Application Program Interface (API)
PSS®E 36.2.1

January 2025

SIEMENS

Siemens Industry, Inc.
Siemens Power Technologies International
400 State Street
Schenectady, NY 12301-1058 USA
+1 518-395-5000
www.siemens.com/power-technologies

Copyright © 1990, 2020 Siemens Power Technologies International
Information in this manual and any software described herein is confidential and subject to change without notice and does not represent a commitment on the part of Siemens Industry, Inc., Siemens Power Technologies International. The software described in this manual is furnished under a license agreement or nondisclosure agreement and may be used or copied only in accordance with the terms of the agreement. No part of this manual may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, for any purpose other than the purchaser's personal use, without the express written permission of Siemens Industry, Inc., Siemens Power Technologies International. PSS®E high-performance transmission planning software is a registered trademark of Siemens Industry, Inc., Siemens Power Technologies International in the United States and other countries. The Windows 10® operating systems, the Visual C++® development system, Microsoft Office Excel® and Microsoft Visual Studio® are registered trademarks of Microsoft Corporation in the United States and other countries. The Python programming language is a trademark of the Python Software Foundation. Other names may be trademarks of their respective owners.

Table of Contents

Preface lxix
Power Flow 1
Power Flow Operation 2
ACCC 2
ACCC_2 4
ACCC_MULTIPLE_MERGE 6
ACCC_MULTIPLE_RUN_REPORT 8
ACCC_MULTIPLE_RUN_REPORT_2 11
ACCC_PARALLEL 15
ACCC_PARALLEL_2 18
ACCC_SINGLE_RUN_REPORT 21
ACCC_SINGLE_RUN_REPORT_2 24
ACCC_SINGLE_RUN_REPORT_3 27
ACCC_SINGLE_RUN_REPORT_4 30
ACCC_SINGLE_RUN_REPORT_5 33
ACCC_SINGLE_RUN_REPORT_6 37
ACCC_TRIP_COR 41
ACCC_TRIP_COR_2 45
ACCC_TRIP_COR_3 49
ACCC_WITH_COR 54
ACCC_WITH_COR_2 58
ACCC_WITH_COR_3 62
ACCC_WITH_DSP 66
ACCC_WITH_DSP_2 69
ACCC_WITH_DSP_3 72
ACCC_WITH_TRIP 75
ACCC_WITH_TRIP_2 78
ACCC_WITH_TRIP_PARALLEL 81
ACCOR 84
ACCOR_2 87
ACCOR_3 90
ALERT_OUTPUT 93
ALLOW_PSSUSERPF 95
ALPH 96
APPEND_ACCC 97
APPLY_VAR_LIMITS 99
AREA 100
AREA_2 101
AREA_ZONE 102
ARNM 103
ARNM_2 105
BASE_FREQUENCY 107
BGEN 108
BRCH 109
BRCH_2 111
BRN_RATING_DEFAULT 113
BSNM 114
BUSN 117

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BUS_INPUT 118
BUS_OUTPUT 119
BUS_SIZE_LEVEL 120
CASE 121
CA_ITERATIONS 122
CHECKVOLTAGELIMITS 123
CHECK_POWERFLOW_DATA 124
CHKCNTDUPLICON 126
CHKCNTDUPLIDFX 127
CLOSE_POWERFLOW 128
CLOSE_REPORT 129
CMPR 130
CNTB 132
CONG 134
CONL 135
CONNECTIVITY_CHECK 137
CONTROL_AREA_INTERCHANGE 138
CSV_TO_RAWX 139
CSV_TO_RAWX_WITH_METAMODEL 140
DCCC 141
DCCC_2 143
DCCOR 145
DCCOR_2 147
DCLF 150
DCLF_2 152
DCPSCOPF 154
DC_TAP_ADJUSTMENT 157
DELTMPFILES 158
DFAX 159
DFAX_2 161
DFAX_CONTINGENCY 163
DFTI 164
DIFF 167
DSCN 171
DUPLICATE_CNTLABEL_CHECK 172
ECDI 173
ECHO 175
EEQV 176
EQRD 178
EXAM 180
FACT 181
FDNS 182
FILE_OVERWRITE 184
FIND 185
FLAT_START 186
FNSL 187
GCAP 189
GCAP_2 190
GDIF 191
GENDSP 193
GENDSP_2 195

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GENS 197
GEOL 198
GETCONTINGENCYSAVEDCASE 199
GIC_PF 200
GNESTATUS 202
GNET 203
IMNET 204
IMOUTAGE 205
IMPC 206
IMPLEMENT_TRANSFER 207
IMPLEMENT_TRANSFER_2 210
INLF 213
INLF_2 215
INTA 217
INTZ 218
ISOLATE_LEVELS 219
JOIN 220
LAMP 221
LINES_PER_PAGE 222
LINES_PER_PAGE_ONE_DEVICE 223
LINE_SHUNT_REPORTING 224
LIST 225
LISTCONTINGENCYSAVEDCASES 227
LLRF 228
LOAD_REDUCTION 230
LOUT 231
LTAP 232
MACCC 233
MACCC_2 238
MACCC_3 243
MACCC_PARALLEL 248
MACCC_PARALLEL_2 253
MACCC_TRIP_COR 258
MACCC_TRIP_COR_2 264
MACCC_TRIP_COR_3 270
MACCC_TRIP_COR_4 277
MACCC_WITH_COR 283
MACCC_WITH_COR_2 288
MACCC_WITH_COR_3 294
MACCC_WITH_COR_4 300
MACCC_WITH_TRIP 306
MACCC_WITH_TRIP_2 311
MACCC_WITH_TRIP_3 316
MACCC_WITH_TRIP_PARALLEL 321
MACCC_WITH_TRIP_PARALLEL_2 326
MATRIX_GROWTH_FACTOR 331
MCRE 332
MODR 333
MOVE3WND 335
MOVEBRN 336
MOVEINDMAC 337

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MOVEINDMACS 338
MOVELOAD 339
MOVELOADS 340
MOVEMAC 341
MOVEPLNT 342
MOVESHUNT 343
MOVESHUNTS 344
MOVESWS 345
MOVESWSHUNT 346
MOVESWSHUNTS 347
MOV_ALPHA 348
MOV_ITERATIONS 349
MOV_TOLERANCE 350
MSLV 351
MSUM 353
MTDC 354
MULTISECTION_REPORTING 355
MWMI 356
MWOMWMSF 358
MWOMWOSF 359
N11_ACCC 360
N11_ACCC_2 364
N11_ACCC_3 368
N11_ACCC_PARALLEL 372
N11_ACCC_PARALLEL_2 376
N11_ACCC_PSCOPF 380
N11_ACCC_PSCOPF_2 387
N11_ACCC_PSCOPF_3 394
NETG 400
NEWCAS 401
NEWCASE_2 402
NEWTON_TOLERANCE 403
NEW_DIMENSION 404
NON_DIVERGENT 405
NON_TRANS_PERCENT_UNITS 406
NSOL 407
NUMBER_THREADS 409
ORDR 410
OTDF 411
OUTPUT_Y_MATRIX 412
OUTS 413
OWNM 414
OWNM_2 416
OWNM_3 418
OWNR 420
PATH 421
PHASE_SHIFT_ADJUSTMENT 422
POLY 423
POUT 426
POWER_OUTPUT 427
PP_ACCC 428

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PP_ACCC_MULTI_CASE 430
PRINT_OUTAGED_BRANCHES 433
PRINT_WINDING_BUSES 434
PROGRESS_OUTPUT 435
PROMPT_OUTPUT 437
PRTI 439
PSCOPF 440
PSCOPF_2 446
PSEB 452
PSSEHALT 453
PSSEHALT_2 454
PV_ENGINE 455
PV_ENGINE_1A 460
PV_ENGINE_2 465
PV_ENGINE_3 471
PV_ENGINE_4 477
PV_ENGINE_5 483
PV_ENGINE_6 489
QV_ENGINE 495
QV_ENGINE_2 498
QV_ENGINE_3 501
QV_ENGINE_4 504
RANK 508
RANK_BRN_AND_MAC 510
RATE 512
RATE_2 514
RATE_3 516
RATE_INTERFACE 518
RATING_SET 519
RAWD 520
RAWDX 522
RAWD_2 524
RAWX_READ_MODE 526
RAWX_TO_CSV 527
RDCH 528
RDCHRAWVERSION 529
RDEQ 530
READ 532
READCAPCURVES 533
READRAWVERSION 534
READRAWX 535
READSUB 536
READSUBRAWVERSION 538
READX 540
RECN 541
REGB 542
RELIND 543
RELIND_2 546
REMM 549
REPORT_OUTPUT 550
RESQ 552

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

RESQVERSION 553
RESULT_TABLE_OUTPUT 554
RETI 555
RETRY_PSSUSERPF 556
REV29_NAMES 557
RNFI 558
RSOL 560
RUNLASTSOLUTION 562
RWCM 563
RWMA 564
RWMM 566
RWSQ 567
RWSQ_2 569
SAVE 571
SCAL 572
SCAL_2 575
SCAL_3 578
SCAL_4 581
SENSITIVITY_FLOW 584
SENSITIVITY_FLOWS 588
SENSITIVITY_INTERFACE 591
SENSITIVITY_VOLTAGE 594
SENSITIVITY_VOLTAGES 597
SET_INPUT_DEV 600
SET_PROGRESS_VERBOSE 601
SHNT 602
SHORT_CIRCUIT_COORDINATES 603
SHORT_CIRCUIT_MODELING 604
SHORT_CIRCUIT_UNITS 605
SHORT_CIRCUIT_WARNING 606
SHORT_CIRCUIT_Z_COORDINATES 607
SHORT_CIRCUIT_Z_UNITS 608
SHOW 609
SHOWTABLE 610
SHOWTABLEBYINDEX 611
SIZE 612
SOLV 613
SPCB 615
SPIL 617
SPLT 621
SQLI 622
SRAIND 624
STOP 626
STOP_2 627
SUBS 628
SWD_RATING_DEFAULT 629
SWITCHED_SHUNT_ADJUSTMENT 630
TAP_ADJUSTMENT 631
TEXT 632
TFLG 633
TIES 634

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

TIEZ 635
TIME 636
TLST 637
TLTG 638
TPCH 642
TRANSFORMER_PERCENT_UNITS 645
TRANSMISSION_LINE_UNITS 646
TREE 647
TYSL 648
USER 649
VAMM 650
VAMO 651
VCHK 652
VOLTAGE_INPUT 653
VOLTAGE_OUTPUT 654
VSMO 655
WRITECAPCURVES 656
WRITERAWVERSION 658
WRITERAWX 659
WRITERAWXSUBSYS 660
WRITESEQVERSION 662
WRITESEQVERSION_2 663
WRITE_OPTIONS_FILE 664
XEQV 665
ZONE 667
ZONE_2 668
ZONE_AREA 669
ZONM 670
ZONM_2 672
Power Flow Data 674
AREA_DATA 674
BRANCH_CHNG 676
BRANCH_CHNG_3 678
BRANCH_CHNG_4 681
BRANCH_DATA 684
BRANCH_DATA_3 686
BRANCH_DATA_4 689
BRN_RATING_NAME_SET 692
BUS_CHNG_3 693
BUS_CHNG_4 695
BUS_DATA 697
BUS_DATA_2 699
BUS_DATA_3 701
BUS_DATA_4 703
BUS_NUMBER 705
CASE_TITLE_DATA 706
CASOLUTION_PARAMETERS 707
CASOLUTION_PARAMETERS_2 708
CASOLUTION_PARAMETERS_3 710
EXTR 712
FACTS_CHNG_2 713

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

FACTS_CHNG_3 716
FACTS_DATA 719
FACTS_DATA_2 722
FACTS_DATA_3 725
GNE_CHNG 728
GNE_DATA 730
IMPEDANCE_CORRECTION_DATA 732
IMPEDANCE_CORRECTION_DATA_3 733
INDUCTION_MACHINE_CHNG 734
INDUCTION_MACHINE_CHNG_2 737
INDUCTION_MACHINE_DATA 740
INDUCTION_MACHINE_DATA_2 743
INTERFACEELEMENT_CHNG 746
INTERFACEELEMENT_DATA 748
INTERFACE_CHNG 750
INTERFACE_DATA 753
LOADTYPE_CHNG 756
LOADTYPE_DATA 757
LOAD_CHNG_4 758
LOAD_CHNG_5 760
LOAD_CHNG_6 762
LOAD_CHNG_7 764
LOAD_DATA 766
LOAD_DATA_3 768
LOAD_DATA_4 770
LOAD_DATA_5 772
LOAD_DATA_6 774
LOAD_DATA_7 776
LONG_TITLE_DATA 778
MACHINE_CAP_CURVE_CHNG 779
MACHINE_CAP_CURVE_DATA 781
MACHINE_CHNG_2 783
MACHINE_CHNG_3 785
MACHINE_CHNG_4 787
MACHINE_CHNG_5 789
MACHINE_DATA 792
MACHINE_DATA_2 794
MACHINE_DATA_3 796
MACHINE_DATA_4 799
MACHINE_DATA_5 802
MBID2DC 805
MBID3WND 806
MBIDATRN 807
MBIDBRN 808
MBIDFACTS 809
MBIDGNE 810
MBIDINDMAC 811
MBIDINTF 812
MBIDLOAD 813
MBIDMAC 814
MBIDMDC 815

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MBIDMSL 816
MBIDSHUNT 817
MBIDSWSHUNT 818
MBIDVD 819
MBIDVSC 820
MBID_SWD_RATING 821
MULTI_SECTION_LINE_DATA 822
MULTI_SECTION_LINE_EDIT 824
MULTI_TERM_DC_BUS_DATA 825
MULTI_TERM_DC_CONVR_DATA 827
MULTI_TERM_DC_LINE_CHNG 830
MULTI_TERM_DC_LINE_DATA 832
MULTI_TERM_DC_LINK_DATA 834
OWNER_DATA 836
PBUS_ADD_MOD 837
PBUS_DELETE 838
PLANT_CHNG 839
PLANT_CHNG_3 840
PLANT_CHNG_4 841
PLANT_DATA 843
PLANT_DATA_3 844
PLANT_DATA_4 845
PURG 847
PURG2DC 849
PURG3WND 850
PURGAREA 851
PURGATRN 852
PURGBRN 853
PURGCAPCURVE 854
PURGE_MULTI_TERM_DC_BUS 855
PURGE_MULTI_TERM_DC_CONVR 856
PURGE_MULTI_TERM_DC_LINK 858
PURGFACTS 859
PURGGNE 860
PURGINDMAC 861
PURGINDMACS 862
PURGLOAD 863
PURGLOADS 864
PURGMAC 865
PURGMDC 866
PURGMSL 867
PURGMUT 868
PURGOWNER 869
PURGPLNT 870
PURGSHUNT 871
PURGSHUNTS 872
PURGSWS 873
PURGSWSHUNT 874
PURGVSC 875
PURGZONE 876
PURG_INTERFACE 877

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PURG_INTERFACE_ELEMENT 878
PURG_LOADTYPE 879
PURG_SWD_RATING 880
PURG_VOLTAGE_DROOP 881
RATINGSETTEXTDATA 882
SHUNT_CHNG 883
SHUNT_CHNG_2 884
SHUNT_DATA 885
SHUNT_DATA_2 886
SOLUTION_PARAMETERS 887
SOLUTION_PARAMETERS_2 889
SOLUTION_PARAMETERS_3 891
SOLUTION_PARAMETERS_4 893
SOLUTION_PARAMETERS_5 895
SWD_RATING_CHNG 898
SWD_RATING_DATA 900
SWD_RATING_NAME_SET 902
SWITCHED_SHUNT_CHNG_3 903
SWITCHED_SHUNT_CHNG_4 906
SWITCHED_SHUNT_CHNG_5 909
SWITCHED_SHUNT_CHNG_6 912
SWITCHED_SHUNT_DATA 915
SWITCHED_SHUNT_DATA_3 918
SWITCHED_SHUNT_DATA_4 921
SWITCHED_SHUNT_DATA_5 924
SWITCHED_SHUNT_DATA_6 927
THREE_WINDING_DATA 930
THREE_WND_IMPEDANCE_DATA 934
THREE_WND_IMPED_CHNG_3 937
THREE_WND_IMPED_CHNG_4 940
THREE_WND_IMPED_DATA_3 943
THREE_WND_IMPED_DATA_4 946
THREE_WND_WINDING_DATA 949
THREE_WND_WINDING_DATA_3 951
THREE_WND_WINDING_DATA_4 953
THREE_WND_WINDING_DATA_5 956
TRANSACTION_ADD_MOD 959
TRANSACTION_DELETE 960
TRANSFER_CHNG 961
TRANSFER_DATA 962
TWO_TERMINAL_DC_LINE_CHNG 964
TWO_TERMINAL_DC_LINE_DATA 966
TWO_TERM_DC_CONVERTER_DATA_3 968
TWO_TERM_DC_CONVR_DATA 971
TWO_WINDING_CHNG_4 974
TWO_WINDING_CHNG_5 978
TWO_WINDING_CHNG_6 982
TWO_WINDING_DATA 987
TWO_WINDING_DATA_3 991
TWO_WINDING_DATA_4 995
TWO_WINDING_DATA_5 999

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

TWO_WINDING_DATA_6 1003
VOLTAGE_DROOP_CHNG 1008
VOLTAGE_DROOP_DATA 1010
VSC_DC_CONVERTER_DATA 1012
VSC_DC_CONVERTER_DATA_3 1014
VSC_DC_LINE_CHNG 1016
VSC_DC_LINE_DATA 1017
ZONE_DATA 1018
Node Breaker 1020
Node Breaker Operation 1021
CONVERT_SECTION_TO_BUS 1021
ISOLATE2DCLINEBYBREAKER 1022
ISOLATE3WTBYBREAKER 1023
ISOLATEBUSBYBREAKER 1024
ISOLATEFACTSBYBREAKER 1025
ISOLATEINDMACHINEBYBREAKER 1026
ISOLATELINEBYBREAKER 1027
ISOLATELOADBYBREAKER 1028
ISOLATEMACHINEBYBREAKER 1029
ISOLATEMSLINEBYBREAKER 1030
ISOLATEMTDCLINEBYBREAKER 1031
ISOLATENODEBYBREAKER 1032
ISOLATESHUNTBYBREAKER 1033
ISOLATESWDBYBREAKER 1034
ISOLATESWSHUNTBYBREAKER 1035
ISOLATESWSHUNTBYBREAKER_2 1036
ISOLATEVSCDCLINEBYBREAKER 1037
RESTORESTATESFROMISOLATE 1038
STATION_AMPOUT 1039
STATION_BUILD_CONFIG 1040
STATION_LIST 1042
STATION_POUT 1043
STATION_POUT_2 1044
STATION_TREE 1045
Node Breaker Data 1047
FETCH_ADJVAR_TERM_NODE 1047
FETCH_INDMACHINE_TERM_NODE 1048
FETCH_LOAD_TERM_NODE 1049
FETCH_MACHINE_TERM_NODE 1050
FETCH_SHUNT_TERM_NODE 1051
FETCH_SWS_SHUNT_TERM_NODE 1052
FETCH_SWS_SHUNT_TERM_NODE_2 1053
PLANT_BUS_SECTION_CHNG 1054
STATION_2DC_BUS_TERM_CHNG 1055
STATION_2DC_CONV_TERM_CHNG 1056
STATION_ADJVAR_TERM_CHNG 1057
STATION_BRANCH_TERM_CHNG 1058
STATION_BRANCH_TERM_MOVE 1059
STATION_BUS_PURG 1060
STATION_CHNG 1061
STATION_DATA 1062

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATION_FACTS_BUS_TERM_CHNG 1063
STATION_FACTS_TERM_CHNG 1064
STATION_INDMACHINE_TERM_CHNG 1065
STATION_LOAD_TERM_CHNG 1066
STATION_MACHINE_TERM_CHNG 1067
STATION_MTDC_BUS_TERM_CHNG 1068
STATION_MTDC_TERM_CHNG 1069
STATION_NODE_CHNG 1070
STATION_NODE_DATA 1071
STATION_NODE_NUMBER 1072
STATION_NODE_PURG 1073
STATION_NUMBER 1074
STATION_PURG 1075
STATION_SHUNT_TERM_CHNG 1076
STATION_SWD_CHNG 1077
STATION_SWD_CHNG_2 1079
STATION_SWD_DATA 1081
STATION_SWD_DATA_2 1083
STATION_SWD_MBID 1085
STATION_SWD_MOVE 1086
STATION_SWD_PURG 1087
STATION_SWS_SHUNT_TERM_CHNG 1088
STATION_SWS_SHUNT_TERM_CHNG_2 1089
STATION_THREE_WND_TERM_CHNG 1090
STATION_THREE_WND_TERM_MOVE 1091
STATION_VSCDC_BUS_TERM_CHNG 1092
STATION_VSCDC_CONV_TERM_CHNG 1093
SYSTEM_SWD_CHNG 1094
SYSTEM_SWD_CHNG_2 1096
SYSTEM_SWD_DATA 1098
SYSTEM_SWD_DATA_2 1100
Optimal Power Flow 1103
Optimal Power Flow Operation 1104
ADD_DETAILS_TO_OPF_LOG 1104
APPLY_ALTERNATE_STEP_SIZE 1105
APPLY_AUTOMATIC_SCALING 1106
BAD_ITER_COARSE_LIMIT 1107
BAD_ITER_FINE_LIMIT 1108
CLAMP_NONOPTIMIZED_GENS 1109
CONSTRAIN_INTERFACE_FLOWS 1110
FINAL_OPF_BARRIER_COEFF 1111
IMPORT_ECDI 1112
INITIAL_OPF_BARRIER_COEFF 1113
INTERFACE_FLOW_COST_COEFF 1114
LSTO 1115
MINIMIZE_ADJ_BUS_SHUNTS 1117
MINIMIZE_FUEL_COST 1118
MINIMIZE_INTERFACE_FLOWS 1119
MINIMIZE_LOAD_ADJUSTMENTS 1120
MINIMIZE_P_LOSSES 1121
MINIMIZE_P_SLACK 1122

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MINIMIZE_Q_LOSSES 1123
MINIMIZE_Q_SLACK 1124
MINIMIZE_REACTIVE_RESERVE 1125
MINIMIZE_SERIES_COMP 1126
NOPF 1127
OPEN_BUS_VOLTAGE_LIMITS 1128
OPF_BARRIER_STEP_LENGTH 1129
OPF_CLAMP_DECAY_FACTOR 1130
OPF_FINAL_CLAMP_TOL 1131
OPF_FIXED_VOLTAGE_PENALTY 1132
OPF_FIX_ALL_GENERATORS 1133
OPF_FIX_PHASE_SHIFTERS 1134
OPF_FIX_SWITCHED_SHUNTS 1135
OPF_FIX_TAP_RATIOS 1136
OPF_INITIAL_CLAMP_TOL 1137
OPF_INTERIOR_SHIFT_FACTOR 1138
OPF_LAMBDA_TOLERANCE 1139
OPF_LF_CONTROL_PENALTY 1140
OPF_MAX_TAP_RATIO_STEP 1141
OPF_MIN_TAP_RATIO_STEP 1142
OPF_REGULATE_AREA_INT 1143
OPF_ROUND_SWITCHED_SHUNTS 1144
OPF_ROUND_TAP_RATIOS 1145
OPF_SCALE_QGEN_LIMITS 1146
OPF_STEP_LENGTH_TOLERANCE 1147
OPF_USE_GENERATOR_VSCHED 1148
OPTO 1149
PRODUCE_OPF_LOG_FILE 1155
P_LOSSES_COST_COEFF 1156
Q_LOSSES_COST_COEFF 1157
REACTIVE_RESV_COST_COEFF 1158
ROPF 1159
RWOP 1160
SET_OPF_REPORT_SUBSYSTEM 1161
USE_DUAL_CRITERIA 1162
USE_EMERGENCY_FLOW_LIMITS 1163
USE_EMERGENCY_VOLT_LIMITS 1164
WRITE_OPF_OPTIONS_FILE 1165
Optimal Power Flow Data 1167
NEWOPF 1167
OPF_ADJBRX_INDV 1168
OPF_ADJBRX_SUBSYS 1170
OPF_ADJLOAD_TBL 1173
OPF_ADJVAR_INDV 1175
OPF_ADJVAR_INDV_2 1177
OPF_ADJVAR_SUBSYS 1179
OPF_APDSP_TBL 1182
OPF_BRFLW_3WT_INDV 1184
OPF_BRFLW_BRN_INDV 1186
OPF_BRFLW_SUBSYS 1188
OPF_BUS_INDV 1191

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPF_BUS_INDV_2 1193
OPF_BUS_SUBSYS 1195
OPF_CHANGE_3WT_FLOW_ID 1198
OPF_CHANGE_ADJVAR_ID 1199
OPF_CHANGE_BRN_FLOW_ID 1200
OPF_CSTTBL_LIN 1201
OPF_CSTTBL_POLY 1202
OPF_CSTTBL_QUAD 1203
OPF_GENDSP_INDV 1204
OPF_GENDSP_SUBSYS 1205
OPF_GENRSV_INDV 1206
OPF_GENRSV_SUBSYS 1207
OPF_GEN_RCAP_INDV 1209
OPF_GEN_RCAP_SUBSYS 1211
OPF_INTFLW_3WT 1214
OPF_INTFLW_BRN 1215
OPF_INTFLW_MAIN 1216
OPF_INTFLW_RESET 1218
OPF_LNCEQN_ADJLOAD 1219
OPF_LNCEQN_ADJVAR 1220
OPF_LNCEQN_BRFLOW 1221
OPF_LNCEQN_INTFLOW 1222
OPF_LNCEQN_MAIN 1223
OPF_LNCEQN_PGEN 1224
OPF_LNCEQN_QGEN 1225
OPF_LNCEQN_RESET 1226
OPF_LNCEQN_SWSHUNT 1227
OPF_LNCEQN_SWSHUNT_2 1228
OPF_LNCEQN_TRAN 1229
OPF_LNCEQN_VANG 1230
OPF_LNCEQN_VANG_2 1231
OPF_LNCEQN_VMAG 1232
OPF_LNCEQN_VMAG_2 1233
OPF_LOAD_INDV 1234
OPF_LOAD_SUBSYS 1235
OPF_PERRSV_GEN 1236
OPF_PERRSV_MAIN 1237
OPF_PERRSV_RESET 1238
PURGE_ALL_OPF_DATA 1239
PURGE_OPF_ADJBRX_INDV 1240
PURGE_OPF_ADJBRX_SUBSYS 1241
PURGE_OPF_ADJLOAD_TBL 1242
PURGE_OPF_ADJVAR_INDV 1243
PURGE_OPF_ADJVAR_INDV_3 1244
PURGE_OPF_ADJVAR_SUBSYS 1245
PURGE_OPF_APDSP_TBL 1246
PURGE_OPF_BRFLW_3WT 1247
PURGE_OPF_BRFLW_BRN 1248
PURGE_OPF_BRFLW_SUBSYS 1249
PURGE_OPF_BUS_INDV 1250
PURGE_OPF_BUS_INDV_3 1251

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PURGE_OPF_BUS_SUBSYS 1252
PURGE_OPF_CSTTBL_LIN 1253
PURGE_OPF_CSTTBL_POLY 1254
PURGE_OPF_CSTTBL_QUAD 1255
PURGE_OPF_GENDSP_INDV 1256
PURGE_OPF_GENDSP_SUBSYS 1257
PURGE_OPF_GENRSV_INDV 1258
PURGE_OPF_GENRSV_SUBSYS 1259
PURGE_OPF_GEN_RCAP_INDV 1260
PURGE_OPF_GEN_RCAP_SUBSYS 1261
PURGE_OPF_INTFLW 1262
PURGE_OPF_INTFLW_3WT 1263
PURGE_OPF_INTFLW_BRN 1264
PURGE_OPF_LNCEQN 1265
PURGE_OPF_LNCEQN_ADJLOAD 1266
PURGE_OPF_LNCEQN_ADJVAR 1267
PURGE_OPF_LNCEQN_BRFLOW 1268
PURGE_OPF_LNCEQN_INTFLOW 1269
PURGE_OPF_LNCEQN_PGEN 1270
PURGE_OPF_LNCEQN_QGEN 1271
PURGE_OPF_LNCEQN_SWSHUNT 1272
PURGE_OPF_LNCEQN_SWSHUNT_2 1273
PURGE_OPF_LNCEQN_TRAN 1274
PURGE_OPF_LNCEQN_VANG 1275
PURGE_OPF_LNCEQN_VANG_2 1276
PURGE_OPF_LNCEQN_VMAG 1277
PURGE_OPF_LNCEQN_VMAG_2 1278
PURGE_OPF_LOAD_INDV 1279
PURGE_OPF_LOAD_SUBSYS 1280
PURGE_OPF_PERRSV 1281
PURGE_OPF_PERRSV_GEN 1282
Fault Analysis 1284
Fault Analysis Operation 1285
ANSI 1285
ANSI_2 1287
ANSI_3 1289
ASCC 1292
ASCC_1A 1294
ASCC_2 1296
ASCC_3 1298
ASCC_SCFILE 1302
BKDY 1303
CHECK_SEQUENCE_DATA 1305
FLAT 1306
FLAT_2 1308
FLAT_3 1311
IECS 1314
IECS_2 1316
IECS_3 1319
IECS_4 1322
IECS_SCFILE 1326

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

NEWSEQ 1327
SCEQ 1328
SCGR 1330
SCMU 1332
SCMU_2 1339
SCOP 1346
SEQD 1348
SEQD_2 1349
SEQUENCE_NETWORK_SETUP 1350
SQEX 1351
Fault Analysis Data 1353
MBIDNCS 1353
SEQ_3_WIND_GROUNDING_DATA 1354
SEQ_3_WIND_WINDING_DATA 1355
SEQ_BRANCH_DATA 1356
SEQ_BRANCH_DATA_3 1358
SEQ_BUS_DATA 1360
SEQ_FIXED_SHUNT_DATA 1361
SEQ_INDUCTION_MACH_DATA 1362
SEQ_INDUCTION_MACH_DATA_2 1364
SEQ_LOAD_DATA 1366
SEQ_MACHINE_DATA 1368
SEQ_MACHINE_DATA_3 1370
SEQ_MACHINE_DATA_4 1372
SEQ_MACHINE_NCS_DATA 1374
SEQ_MACHINE_NCS_DATA_PURG 1376
SEQ_MUTUAL_DATA 1377
SEQ_NCS_FLT_CNTRB_CHNG 1379
SEQ_NCS_FLT_CNTRB_DATA 1380
SEQ_NCS_FLT_CNTRB_PURG 1381
SEQ_NCS_TABLE_DATA 1382
SEQ_NCS_TABLE_ONEPT 1383
SEQ_NCS_TABLE_PURG 1384
SEQ_NCS_TABLE_PURG_ONEPT 1385
SEQ_NCS_TABLE_TYPE 1386
SEQ_SWITCHED_SHUNT_DATA 1387
SEQ_SWITCHED_SHUNT_DATA_3 1389
SEQ_THREE_WINDING_DATA 1391
SEQ_THREE_WINDING_DATA_3 1393
SEQ_TWO_WINDING_DATA 1395
SEQ_TWO_WINDING_DATA_3 1397
Harmonics 1400
Harmonics Operation 1401
HAR_ANALYSIS 1401
HAR_ANALYSIS_2 1404
HAR_CREATE_PFCASE 1410
HAR_EXISTS_DSTN_RESULTS 1411
HAR_EXISTS_FSCAN_RESULTS 1412
HAR_EXPORT_DSTN 1413
HAR_EXPORT_FSCAN 1415
HAR_LIMITS_SC_CURRENTS 1416

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

HAR_LIMITS_VIOLATIONS_CURRENT 1417
HAR_LIMITS_VIOLATIONS_VOLTAGE 1419
HAR_NEW 1421
HAR_PURG 1422
HAR_SET_RESN_THRESHOLDS 1423
HAR_SET_RESN_THRESHOLDS_DEFAULT 1424
Harmonics Data 1426
HAR_2TDC 1426
HAR_2TDC_CHNG 1427
HAR_2TDC_PURG 1428
HAR_BRN 1429
HAR_BRN_CHNG 1430
HAR_BRN_PURG 1431
HAR_CURSRC 1432
HAR_CURSRC_CHNG 1433
HAR_CURSRC_CHNG_NAME 1434
HAR_CURSRC_CHNG_ONEPT 1435
HAR_CURSRC_NAME 1436
HAR_CURSRC_ONEPT 1437
HAR_CURSRC_PURG 1438
HAR_CURSRC_PURG_ONEPT 1439
HAR_FACTS 1440
HAR_FACTS_CHNG 1441
HAR_FACTS_PURG 1442
HAR_IMPCHAR 1443
HAR_IMPCHAR_CHNG 1444
HAR_IMPCHAR_CHNG_NAME 1445
HAR_IMPCHAR_CHNG_ONEPT 1446
HAR_IMPCHAR_NAME 1447
HAR_IMPCHAR_ONEPT 1448
HAR_IMPCHAR_PURG 1449
HAR_IMPCHAR_PURG_ONEPT 1450
HAR_INDMC 1451
HAR_INDMC_CHNG 1452
HAR_INDMC_PURG 1453
HAR_LOAD 1454
HAR_LOAD_CHNG 1455
HAR_LOAD_PURG 1456
HAR_MACH 1457
HAR_MACH_CHNG 1458
HAR_MACH_PURG 1459
HAR_MTDC 1460
HAR_MTDC_CHNG 1461
HAR_MTDC_PURG 1462
HAR_PASSIVE_FILTER 1463
HAR_PASSIVE_FILTER_CHNG 1465
HAR_PASSIVE_FILTER_PURG 1467
HAR_SET_PAR_LODMDL_CIGRE_MEASUREMENT 1468
HAR_SET_PAR_LODMDL_CIGRE_MOTIVE 1469
HAR_SET_PAR_LODMDL_CIGRE_PASSIVE 1470
HAR_SET_PAR_LODMDL_IEEE_2RL_PARALLEL 1471

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

HAR_SET_PAR_LODMDL_IEEE_IM 1472
HAR_SET_PAR_LODMDL_IEEE_MEASUREMENT 1473
HAR_SET_PAR_LODMDL_IEEE_SKIN 1474
HAR_SET_PAR_LODMDL_LARGE_ASYNC_MOTOR 1475
HAR_TRN 1476
HAR_TRN_CHNG 1477
HAR_TRN_PURG 1478
HAR_VLTSRC 1479
HAR_VLTSRC_CHNG 1480
HAR_VLTSRC_CHNG_NAME 1481
HAR_VLTSRC_CHNG_ONEPT 1482
HAR_VLTSRC_NAME 1483
HAR_VLTSRC_ONEPT 1484
HAR_VLTSRC_PURG 1485
HAR_VLTSRC_PURG_ONEPT 1486
HAR_VSCDC 1487
HAR_VSCDC_CHNG 1488
HAR_VSCDC_PURG 1489
GIC 1491
GIC Operation 1492
GIC 1492
GIC_2 1495
GIC_3 1501
GIC_4 1507
GIC_5 1514
GIC_6 1521
GIC_7 1530
GIC_8 1539
GIC_BRANCH_EFIELD_NN 1550
GIC_EARTH1D_USR_PURG 1552
GIC_EFIELD_WAVESHAPE 1553
GIC_EFIELD_WAVESHAPE_REF 1554
GIC_EFIELD_WAVESHAPE_SUPP 1555
GIC_MVARLOSS_SCALING_FACTORS 1556
GIC_NEW 1557
GIC_PF_OPTIONS 1558
GIC_PURG 1560
GIC_READ 1561
GIC_READ_EFIELD_GRID 1562
GIC_THERMAL_IMPACT 1563
GIC_THERMAL_IMPACT_1 1564
GIC_WRITE 1565
GIC_WRITE_STN 1566
GIC Data 1568
GIC_2TDC 1568
GIC_2TDC_CHNG 1569
GIC_2TDC_PURG 1570
GIC_BRN 1571
GIC_BRN_CHNG 1573
GIC_BRN_PURG 1575
GIC_EARTH1D_USR 1576

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GIC_EARTH1D_USR_CHNG 1577
GIC_FACTS 1578
GIC_FACTS_CHNG 1579
GIC_FACTS_PURG 1580
GIC_FXSH 1581
GIC_FXSH_CHNG 1582
GIC_FXSH_PURG 1583
GIC_LOAD 1584
GIC_LOAD_CHNG 1585
GIC_LOAD_PURG 1586
GIC_MTDC 1587
GIC_MTDC_CHNG 1588
GIC_MTDC_PURG 1589
GIC_SUBSTN 1590
GIC_SUBSTN_CHNG 1591
GIC_SUBSTN_PURG 1592
GIC_SWSH 1593
GIC_SWSH_CHNG 1594
GIC_SWSH_PURG 1595
GIC_TRN 1596
GIC_TRN_CHNG 1599
GIC_TRN_PURG 1602
GIC_VSCDC 1603
GIC_VSCDC_CHNG 1604
GIC_VSCDC_PURG 1605
Time Series Power Flow 1607
Time Series Power Flow Operation 1608
IMPOSEPROFILE 1608
TSPF 1610
TSPF_OUTPUT_PMATRIX 1613
TSPF_OUTPUT_PMATRIX_2 1614
Time Series Power Flow Data 1616
PURGE_TSPF_CHANNEL 1616
PURGE_TSPF_CHANNEL_ALL 1617
PURGE_TSPF_PROFILE 1618
PURGE_TSPF_PROFILECURVE 1619
PURGE_TSPF_PROFILECURVESET 1620
PURGE_TSPF_PROFILEDATA 1621
TSPF_CHANNEL_CHNG 1622
TSPF_CHANNEL_DATA 1623
TSPF_PROFILECURVESET_CHNG 1624
TSPF_PROFILECURVESET_DATA 1625
TSPF_PROFILEDATA_CHNG 1626
TSPF_PROFILEDATA_DATA 1627
TSPF_PROFILE_CHNG 1628
TSPF_PROFILE_DATA 1630
Dynamics Simulation 1633
Dynamic Simulation Operation 1634
ADDMODELLIBRARY 1634
ASTR 1635
AULIST 1638

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHANGE_CHANNEL_OUT_FILE 1639
CHANGE_CHRICN 1640
CHANGE_CON 1641
CHANGE_GREF 1642
CHANGE_ICON 1643
CHANGE_PREF 1644
CHANGE_QVREF 1645
CHANGE_STATE 1646
CHANGE_SWSREF 1647
CHANGE_SWSREF_2 1648
CHANGE_VAR 1649
CHANGE_VREF 1650
CRCTRUN 1651
DCLIST 1653
DLST 1654
DOCU 1655
DROPMODELLIBRARY 1658
DROPMODELPROGRAM 1659
DYDA 1660
DYNAMICSMODE 1663
DYNAMICS_SOLUTION_PARAMS 1664
DYNAMICS_SOLUTION_PARAM_2 1666
DYNEXPORTCSV 1668
DYRE_ADD 1669
DYRE_ADD_2 1670
DYRE_NEW 1671
DYRE_NEW_2 1672
ERUN 1673
ESTR_OPEN_CIRCUIT_TEST 1674
ESTR_RESPONSE_RATIO_TEST 1675
FCLIST 1677
GRUN 1678
GSTR 1679
INCREMENT_GREF 1680
INCREMENT_PREF 1681
INCREMENT_QVREF 1682
INCREMENT_SWSREF 1683
INCREMENT_SWSREF_2 1684
INCREMENT_VREF 1685
LDCLIST 1686
LDLIST 1688
LOAD_ARRAY_CHANNEL 1689
MLST 1690
MRUN 1692
MSTR 1693
POWERFLOWMODE 1694
PSAS 1695
RLLIST 1696
RSTR 1697
RUN 1698
RWDY 1699

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SET_CHNFIL_TYPE 1700
SET_DISABLE_RUN 1701
SET_FREQ_RELAY_V_THRESH 1702
SET_GENANG 1703
SET_GENANG_2 1704
SET_GENANG_3 1706
SET_GENANG_SUBSYS_FLAG 1708
SET_GENPWR 1709
SET_GENSPDEV 1710
SET_GENSPDEV_SUBSYS_FLAG 1711
SET_LOAD_MODEL_THRESH 1712
SET_MODEL_DEBUG_OUTPUT_FLAG 1713
SET_NETFRQ 1714
SET_NEXT_CHANNEL 1715
SET_NEXT_ICON 1716
SET_NEXT_VAR 1717
SET_OSSCAN 1718
SET_OSSCAN_2 1719
SET_OSSCAN_SUBSYS_FLAG 1720
SET_RELANG 1721
SET_RELSCN 1723
SET_RELSCN_SUBSYS_FLAG 1724
SET_VLTSCN 1725
SET_VLTSCN_SUBSYS_FLAG 1726
SET_VOLTAGE_DIP_CHECK 1727
SET_VOLTAGE_REC_CHECK 1728
SET_VOLT_VIOL_SUBSYS_FLAG 1730
SET_ZSORCE_RECONCILE_FLAG 1731
SIZE_DS 1732
SNAP 1733
STRT 1734
STRT_2 1735
SWSLIST 1736
TRIG_VOLT_VIOLATION_CHECK 1737
WNLIST 1738
Output Channel Operations 1740
Dynamic Simulation Output Channel Operation 1740
CHANGE_CHANNEL 1740
CHSB 1741
CRT_PLOT_CHANNEL 1744
DELETE_ALL_PLOT_CHANNELS 1745
DMPC 1746
LIST_CHANNEL_MODELS 1748
REMOVE_UNUSED_CHAN_MODELS 1749
SCANCHANNELS 1750
Assign Machine Quantities to Output Channels 1751
MACHINE_APP_R_X_CHANNEL 1751
MACHINE_ARRAY_CHANNEL 1753
MACHINE_ITERM_CHANNEL 1755
Assign Branch Quantities to Output Channels 1757
BRANCH_APP_R_X_CHANNEL 1757

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BRANCH_MVA_CHANNEL 1759
BRANCH_P_AND_Q_CHANNEL 1761
BRANCH_P_CHANNEL 1763
THREE_WND_APP_R_X_CHANNEL 1765
THREE_WND_MVA_CHANNEL 1767
THREE_WND_P_AND_Q_CHANNEL 1769
THREE_WND_P_CHANNEL 1771
Assign Other Quantities to Output Channels 1773
BUS_FREQUENCY_CHANNEL 1773
STATE_CHANNEL 1774
VAR_CHANNEL 1775
VOLTAGE_AND_ANGLE_CHANNEL 1776
VOLTAGE_CHANNEL 1778
Plant Related Models 1779
ADD_PLANT_MODEL 1779
CHANGE_PLMOD_CHRICN 1781
CHANGE_PLMOD_CON 1782
CHANGE_PLMOD_DATA 1783
CHANGE_PLMOD_ICON 1785
CHANGE_PLMOD_VAR 1786
GMB_ADD_PLANT_MODEL 1787
PLMOD_CONSISTENCY 1789
PLMOD_PACK 1790
PLMOD_REMOVE 1791
PLMOD_STATUS 1792
PLMOD_UNCONNECTED 1793
PLMOD_USER 1794
Wind Related Models 1795
ADD_WIND_MODEL 1795
CHANGE_WNMOD_CHRICN 1797
CHANGE_WNMOD_CON 1798
CHANGE_WNMOD_ICON 1799
CHANGE_WNMOD_VAR 1800
WNMOD_CONSISTENCY 1801
WNMOD_PACK 1802
WNMOD_REMOVE 1803
WNMOD_STATUS 1804
WNMOD_UNCONNECTED 1805
WNMOD_USER 1806
Load Related Models 1807
ADD_LOAD_MODEL 1807
CHANGE_LDMOD_CHRICN 1809
CHANGE_LDMOD_CON 1810
CHANGE_LDMOD_DATA 1811
CHANGE_LDMOD_ICON 1813
CHANGE_LDMOD_VAR 1814
LDMODC_PACK 1815
LDMODC_REMOVE 1816
LDMODC_STATUS 1818
LDMODC_UNCONNECTED 1820
LDMODC_USER 1821

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

LDMOD_PACK 1822
LDMOD_REMOVE 1823
LDMOD_STATUS 1824
LDMOD_STATUS2 1825
LDMOD_UNCONNECTED 1826
LDMOD_USER 1827
Load Component Models 1828
ADD_LOADC_MODEL 1828
CHANGE_LDMODC_CDESC 1830
CHANGE_LDMODC_CHRICN 1832
CHANGE_LDMODC_CON 1834
CHANGE_LDMODC_ICON 1836
CHANGE_LDMODC_VAR 1838
Line Relay Models 1840
ADD_RELAY_MODEL 1840
CHANGE_RLMOD_CHRICN 1842
CHANGE_RLMOD_CON 1843
CHANGE_RLMOD_DATA 1844
CHANGE_RLMOD_ICON 1846
CHANGE_RLMOD_VAR 1847
RLMOD_PACK 1848
RLMOD_REMOVE 1849
RLMOD_STATUS 1850
RLMOD_UNCONNECTED 1851
RLMOD_USER 1852
DC Line Related Models 1853
ADD_MTDCLINE_MODEL 1853
ADD_TTDCLINE_MODEL 1855
ADD_VSDCLINE_MODEL 1857
CHANGE_MTDCLMOD_CHRICN 1858
CHANGE_MTDCLMOD_CON 1859
CHANGE_MTDCLMOD_DATA 1860
CHANGE_MTDCLMOD_ICON 1862
CHANGE_MTDCLMOD_VAR 1863
CHANGE_TTDCLMOD_CHRICN 1864
CHANGE_TTDCLMOD_CON 1865
CHANGE_TTDCLMOD_DATA 1866
CHANGE_TTDCLMOD_ICON 1868
CHANGE_TTDCLMOD_VAR 1869
CHANGE_VSDCLMOD_CHRICN 1870
CHANGE_VSDCLMOD_CON 1871
CHANGE_VSDCLMOD_DATA 1872
CHANGE_VSDCLMOD_ICON 1874
CHANGE_VSDCLMOD_VAR 1875
DCLMOD_PACK 1876
DCLMOD_UNCONNECTED 1877
DCLMOD_USER 1878
MTDCLMOD_REMOVE 1879
MTDCLMOD_STATUS 1880
TTDCLMOD_REMOVE 1881
TTDCLMOD_STATUS 1882

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

VSDCLMOD_REMOVE 1883
VSDCLMOD_STATUS 1884
FACTS Device Models 1885
ADD_FACTS_DEVICE_MODEL 1885
CHANGE_FCTMOD_CHRICN 1887
CHANGE_FCTMOD_CON 1888
CHANGE_FCTMOD_DATA 1889
CHANGE_FCTMOD_ICON 1891
CHANGE_FCTMOD_VAR 1892
FCTMOD_PACK 1893
FCTMOD_REMOVE 1894
FCTMOD_STATUS 1895
FCTMOD_UNCONNECTED 1896
FCTMOD_USER 1897
GMB_ADD_FACTS_DEV_MODEL 1898
Switched Shunt Models 1900
ADD_SWSHUNT_MODEL 1900
ADD_SWSHUNT_MODEL_2 1902
CHANGE_SWSMOD_CHRICN 1904
CHANGE_SWSMOD_CHRICN_2 1905
CHANGE_SWSMOD_CON 1906
CHANGE_SWSMOD_CON_2 1907
CHANGE_SWSMOD_ICON 1908
CHANGE_SWSMOD_ICON_2 1909
CHANGE_SWSMOD_VAR 1910
CHANGE_SWSMOD_VAR_2 1911
GMB_ADD_SWSHUNT_MODEL 1912
GMB_ADD_SWSHUNT_MODEL_2 1914
SWSMOD_PACK 1916
SWSMOD_REMOVE 1917
SWSMOD_REMOVE_2 1918
SWSMOD_STATUS 1919
SWSMOD_STATUS_2 1920
SWSMOD_UNCONNECTED 1921
SWSMOD_USER 1922
Induction Machine Models 1923
ADD_INDMAC_MODEL 1923
CHANGE_IMMOD_CHRICN 1925
CHANGE_IMMOD_CON 1926
CHANGE_IMMOD_ICON 1927
CHANGE_IMMOD_VAR 1928
IMMOD_PACK 1929
IMMOD_REMOVE 1930
IMMOD_STATUS 1931
IMMOD_UNCONNECTED 1932
IMMOD_USER 1933
CCT Models 1934
ADD_CCT2DCO_MODEL 1934
ADD_CCT2WTD_MODEL 1936
ADD_CCT3WTD_MODEL 1938
ADD_CCTBRND_MODEL 1940

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ADD_CCTBRNO_MODEL 1942
ADD_CCTBUSO_MODEL 1944
ADD_CCTMCNO_MODEL 1945
ADD_CCTMCNP_MODEL 1947
ADD_CCTMSCO_MODEL 1949
ADD_CCTSWSO_MODEL 1950
ADD_CCTSWSO_MODEL_2 1952
CCT2DCOMOD_REMOVE 1954
CCT2DCOMOD_STATUS 1955
CCT2DCO_LIST 1956
CCT2WTDMOD_REMOVE 1957
CCT2WTDMOD_STATUS 1958
CCT2WTD_LIST 1959
CCT3WTDMOD_REMOVE 1960
CCT3WTDMOD_STATUS 1961
CCT3WTD_LIST 1962
CCTBRNDMOD_REMOVE 1963
CCTBRNDMOD_STATUS 1964
CCTBRND_LIST 1965
CCTBRNOMOD_REMOVE 1966
CCTBRNOMOD_STATUS 1967
CCTBRNO_LIST 1968
CCTBUSOMOD_REMOVE 1969
CCTBUSOMOD_STATUS 1970
CCTBUSO_LIST 1971
CCTMCNOMOD_REMOVE 1972
CCTMCNOMOD_STATUS 1973
CCTMCNO_LIST 1974
CCTMCNPMOD_REMOVE 1975
CCTMCNPMOD_STATUS 1976
CCTMCNP_LIST 1977
CCTMOD_PACK 1978
CCTMOD_USER 1979
CCTMSCOMOD_REMOVE 1980
CCTMSCOMOD_STATUS 1981
CCTMSCO_LIST 1982
CCTSWSOMOD_REMOVE 1983
CCTSWSOMOD_REMOVE_2 1984
CCTSWSOMOD_STATUS 1985
CCTSWSOMOD_STATUS_2 1986
CCTSWSO_LIST 1987
CHANGE_CCT2DCOMOD_CHRICN 1988
CHANGE_CCT2DCOMOD_CON 1989
CHANGE_CCT2DCOMOD_ICON 1990
CHANGE_CCT2DCOMOD_VAR 1991
CHANGE_CCT2WTDMOD_CHRICN 1992
CHANGE_CCT2WTDMOD_CON 1993
CHANGE_CCT2WTDMOD_ICON 1994
CHANGE_CCT2WTDMOD_VAR 1995
CHANGE_CCT3WTDMOD_CHRICN 1996
CHANGE_CCT3WTDMOD_CON 1997

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHANGE_CCT3WTDMOD_ICON 1998
CHANGE_CCT3WTDMOD_VAR 1999
CHANGE_CCTBRNDMOD_CHRICN 2000
CHANGE_CCTBRNDMOD_CON 2001
CHANGE_CCTBRNDMOD_ICON 2002
CHANGE_CCTBRNDMOD_VAR 2003
CHANGE_CCTBRNOMOD_CHRICN 2004
CHANGE_CCTBRNOMOD_CON 2005
CHANGE_CCTBRNOMOD_ICON 2006
CHANGE_CCTBRNOMOD_VAR 2007
CHANGE_CCTBUSOMOD_CHRICN 2008
CHANGE_CCTBUSOMOD_CON 2009
CHANGE_CCTBUSOMOD_ICON 2010
CHANGE_CCTBUSOMOD_VAR 2011
CHANGE_CCTMCNOMOD_CHRICN 2012
CHANGE_CCTMCNOMOD_CON 2013
CHANGE_CCTMCNOMOD_ICON 2014
CHANGE_CCTMCNOMOD_VAR 2015
CHANGE_CCTMCNPMOD_CHRICN 2016
CHANGE_CCTMCNPMOD_CON 2017
CHANGE_CCTMCNPMOD_ICON 2018
CHANGE_CCTMCNPMOD_VAR 2019
CHANGE_CCTMSCOMOD_CHRICN 2020
CHANGE_CCTMSCOMOD_CON 2021
CHANGE_CCTMSCOMOD_ICON 2022
CHANGE_CCTMSCOMOD_VAR 2023
CHANGE_CCTSWSOMOD_CHRICN 2024
CHANGE_CCTSWSOMOD_CHRICN_2 2025
CHANGE_CCTSWSOMOD_CON 2026
CHANGE_CCTSWSOMOD_CON_2 2027
CHANGE_CCTSWSOMOD_ICON 2028
CHANGE_CCTSWSOMOD_ICON_2 2029
CHANGE_CCTSWSOMOD_VAR 2030
CHANGE_CCTSWSOMOD_VAR_2 2031
Auxiliary Signal Models 2032
ADD_FCTSAUXSIGNAL_MODEL 2032
ADD_MTDCAUXSIGNAL_MODEL 2034
ADD_TTDCAUXSIGNAL_MODEL 2036
ADD_VSDCAUXSIGNAL_MODEL 2038
AUXMOD_PACK 2040
AUXMOD_UNCONNECTED 2041
AUXMOD_USER 2042
CHANGE_FCTSAUXMOD_CHRICN 2043
CHANGE_FCTSAUXMOD_CON 2045
CHANGE_FCTSAUXMOD_DATA 2047
CHANGE_FCTSAUXMOD_ICON 2049
CHANGE_FCTSAUXMOD_VAR 2051
CHANGE_MTDCAUXMOD_CHRICN 2053
CHANGE_MTDCAUXMOD_CON 2055
CHANGE_MTDCAUXMOD_DATA 2057
CHANGE_MTDCAUXMOD_ICON 2059

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHANGE_MTDCAUXMOD_VAR 2061
CHANGE_TTDCAUXMOD_CHRICN 2063
CHANGE_TTDCAUXMOD_CON 2065
CHANGE_TTDCAUXMOD_DATA 2067
CHANGE_TTDCAUXMOD_ICON 2069
CHANGE_TTDCAUXMOD_VAR 2071
CHANGE_VSDCAUXMOD_CHRICN 2073
CHANGE_VSDCAUXMOD_CON 2074
CHANGE_VSDCAUXMOD_DATA 2075
CHANGE_VSDCAUXMOD_ICON 2077
CHANGE_VSDCAUXMOD_VAR 2078
FCTSAUXMOD_REMOVE 2079
FCTSAUXMOD_STATUS 2080
MTDCAUXMOD_REMOVE 2081
MTDCAUXMOD_STATUS 2082
TTDCAUXMOD_REMOVE 2083
TTDCAUXMOD_STATUS 2084
VSDCAUXMOD_REMOVE 2085
VSDCAUXMOD_STATUS 2086
Set Disturbance 2087
DIST_3PHASE_BUS_FAULT 2087
DIST_3WIND_CLOSE 2089
DIST_3WIND_FAULT 2090
DIST_3WIND_TRIP 2092
DIST_BRANCH_CLOSE 2093
DIST_BRANCH_FAULT 2094
DIST_BRANCH_TRIP 2096
DIST_BUS_FAULT 2097
DIST_BUS_FAULT_2 2098
DIST_BUS_FAULT_3 2100
DIST_BUS_TRIP 2102
DIST_CLEAR_FAULT 2103
DIST_MACHINE_TRIP 2105
DIST_SCMU_FAULT 2106
DIST_SCMU_FAULT_2 2108
DIST_SCMU_FAULT_3 2110
DIST_SPCB_FAULT 2112
DIST_SPCB_FAULT_2 2114
Set Disturbance Definition 2116
DIST_DEF_3PHASE_BUS_FAULT 2116
DIST_DEF_3WIND_FAULT 2118
DIST_DEF_BRANCH_FAULT 2120
DIST_DEF_SCMU_FAULT_3 2122
DIST_DEF_SPCB_FAULT_2 2124
Subsystem Definition 2127
Area Subsystems 2128
ASYS 2128
ASYSDEF 2129
ASYSINIT 2130
Bus Subsystems 2131
BSYS 2131

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BSYSADD 2133
BSYSDEF 2135
BSYSDELO 2136
BSYSINIT 2137
BSYSMEM 2138
BSYSO 2139
BSYSRCL 2140
Owner Subsystems 2141
OSYS 2141
OSYSDEF 2142
OSYSINIT 2143
Zone Subsystems 2144
ZSYS 2144
ZSYSDEF 2145
ZSYSINIT 2146
Diagram View 2148
CHANGEDIAGCONTOUR 2149
CHECKDIAGFILE 2151
CLOSEDIAGFILE 2152
DELETEDIAGCONTOUR 2153
ENABLEDIAGCONTOUR 2154
EXPORTBUSLOCFILE 2156
EXPORTBUSLOCFILE_2 2157
EXPORTIMAGEFILE 2158
EXPORTIMAGEFILE_2 2159
EXPORTIMAGEFILE_3 2160
GEXMBUS 2161
GROWBUS 2162
GROWBUSLEVELS 2163
GROWDIAGRAM 2164
GROWDIAGRAM_2 2165
GROWSTATION 2167
GRPG 2168
IMPORTDRAWFILE 2169
IMPORTIMAGEFILE 2170
NEWDIAGFILE 2171
NEXTDIAGFILEDATASET 2172
OPENACCFILE 2173
OPENACCFILE_2 2174
OPENACCFILE_3 2175
OPENBUSLOCFILE 2176
OPENDIAGFILE 2177
PREVDIAGFILEDATASET 2178
PRINTDIAGFILE 2179
PRINTDIAGFILE_2 2180
REFRESHDIAGCONTOUR 2181
REFRESHDIAGFILE 2182
RENUMDIAGFILE 2183
SAVEDIAGFILE 2184
SETDIAGAUTOFILE 2185
SETDIAGFONT 2186

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SETDIAGPREC 2187
SETDIAGPREC_2 2188
SETDIAGPROP 2189
SETDIAGPROP_2 2192
SETDIAGPROP_3 2195
SETDIAGPROP_4 2198
SETDIAGPROP_5 2201
SETDIAGRESACCC 2204
SETDIAGRESACCC_2 2205
SETDIAGRESACCC_3 2206
SETDIAGRESASCC 2207
SETDIAGRESASCC_2 2209
SETDIAGRESASCC_3 2211
SETDIAGRESDATA 2213
SETDIAGRESDATA_2 2216
SETDIAGRESDATA_3 2217
SETDIAGRESDYN 2218
SETDIAGRESDYN_2 2219
SETDIAGRESGDIF 2220
SETDIAGRESIEC 2223
SETDIAGRESIEC_2 2225
SETDIAGRESIEC_3 2227
SETDIAGRESMUST 2229
SETDIAGRESMUST_2 2230
SETDIAGRESOPF 2231
SETDIAGRESOPF_2 2232
SETDIAGRESOPF_3 2233
SETDIAGRESPFLOWCHECK 2234
SETDIAGRESPFLOWCHECK_2 2238
SETDIAGRESPFLOWCHECK_3 2241
SETDIAGRESPFLOWCHECK_4 2244
SETDIAGRESPFLOWOPTIONS 2247
SETDIAGRESPFLOWOPTIONS_2 2249
SETDIAGRESPFLOWOPTIONS_3 2251
SETDIAGRESREL 2253
SETDIAGRESREL_2 2255
SETDIAGRESREL_3 2257
SETDIAGRESSCGR 2259
SETDIAGRESTYPEACC 2262
SETDIAGRESTYPEASCC 2263
SETDIAGRESTYPEDATA 2264
SETDIAGRESTYPEDYN 2265
SETDIAGRESTYPEGDIF 2266
SETDIAGRESTYPEGIC 2267
SETDIAGRESTYPEHARM 2268
SETDIAGRESTYPEIEC 2269
SETDIAGRESTYPEMUST 2270
SETDIAGRESTYPEOPF 2271
SETDIAGRESTYPEPFLOW 2272
SETDIAGRESTYPEREL 2273
SETDIAGRESTYPESCGR 2274

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SETDIAGRESTYPETSPF 2275
SETDIAGRESVRCS 2276
SETDIAGRESVRCS_2 2279
SETDIAGTITLE 2282
SWITCHDIAGFILEDATASET 2283
UPDATEBUSLOCDIAGFILE 2284
Single Element Data 2285
APIERRSTR 2286
ARDAT 2287
AREDAT 2289
AREINT 2290
ARENAM 2291
ARENUM 2292
AREUSE 2293
ARITOJ 2294
BRNCUR 2295
BRNDAT 2296
BRNDT2 2298
BRNFLO 2300
BRNINT 2301
BRNMSC 2303
BRNMVA 2305
BRNNAM 2306
BRNSTT 2307
BRN_RATING_NAME_DATA 2308
BSYSISDEF 2309
BUSDAT 2310
BUSDT1 2311
BUSDT2 2313
BUSEXS 2315
BUSINT 2316
BUSMSM 2317
BUSORDPOS 2318
BUSSECTDAT 2319
BUSSECTDT1 2320
BUSSECTDT2 2322
BUSSECTEXS 2324
BUSSECTINT 2325
CCTMDLNAM_2DCO 2326
CCTMDLNAM_2WTD 2327
CCTMDLNAM_3WTD 2328
CCTMDLNAM_BRND 2329
CCTMDLNAM_BRNO 2330
CCTMDLNAM_BUSO 2331
CCTMDLNAM_MCNO 2332
CCTMDLNAM_MCNP 2333
CCTMDLNAM_MSCO 2334
CCTMDLNAM_SWSO_2 2335
CCTMIND_2DCO 2336
CCTMIND_2WTD 2337
CCTMIND_3WTD 2339

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CCTMIND_BRND 2341
CCTMIND_BRNO 2343
CCTMIND_BUSO 2345
CCTMIND_MCNO 2346
CCTMIND_MCNP 2347
CCTMIND_MSCO 2348
CCTMIND_SWSO 2349
CCTMIND_SWSO_2 2351
CHKTRE 2353
CHNVAL 2354
DC2AUXMIND 2355
DC2AUXMNAM 2357
DC2DAT 2358
DC2DAT_2 2360
DC2INT 2362
DC2INT_2 2363
DC2MIND 2364
DC2MNAM 2366
DCNAUXMIND 2367
DCNAUXMNAM 2369
DCNCIN 2370
DCNCIN_2 2371
DCNDAT 2372
DCNDAT_2 2374
DCNINT 2376
DCNINT_2 2377
DCNMIND 2378
DCNMNAM 2380
DSCVAL 2381
DSIVAL 2382
DSRVAL 2383
FCDAUXMIND 2384
FCDAUXMNAM 2386
FCDDAT 2387
FCDDAT_2 2390
FCDINT 2393
FCDINT_2 2394
FCDMIND 2395
FCDMNAM 2396
FINDMODELPROGRAM 2397
FXSBUS 2398
FXSDT1 2399
FXSDT2 2401
FXSINT 2403
FXSNAME 2404
GENCNV 2405
GENDAT 2406
GENDT1 2407
GENSECTDAT 2408
GENSECTDT1 2409
GETHOMEPATH 2410

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_MSTATE 2411
GNECHR 2412
GNEDAT 2413
GNEINT 2414
INDBUS 2416
INDDT1 2417
INDDT2 2419
INDINT 2421
INI2DC 2423
INIBRN 2424
INIBRN_2 2425
INIBRX 2426
INIBRX_2 2427
INIBUS 2428
INIBUS_2 2429
INIBUX 2430
INIFAX 2431
INIFXS 2432
INIIND 2433
INILOD 2434
INIMAC 2435
INIMDC 2436
INIMSL 2437
ININAM 2438
ININAM_2 2439
ININAX 2440
INISTABUSSECT 2441
INISTANODE 2442
INISTASWDEV 2443
INISWS 2444
INITIE 2445
INITIX 2446
INIVSC 2447
INTFLOW 2448
ISLAND 2449
ITERAT 2450
LMODIND 2451
LMODNAM 2453
LODBUS 2454
LODCNV 2455
LODDT1 2456
LODDT2 2458
LODIND 2460
LODINT 2461
LODNAME 2462
LODTYPE 2463
MACBUS 2464
MACCHR 2465
MACDAT 2466
MACDT2 2468
MACIND 2470

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MACINT 2471
MACSTT 2473
MAXMSM 2474
MDLIND 2475
MDLLIBCNT 2477
MDLNAM 2478
MDLPGMCNT 2479
NATONO 2480
NOTONA 2481
NOTONASECT 2482
NXT2DC 2483
NXTBRN 2484
NXTBRN3 2485
NXTBRN3_2 2486
NXTBRN_2 2487
NXTBUS 2488
NXTBUS_2 2489
NXTFAX 2490
NXTFXS 2491
NXTIND 2492
NXTLOD 2493
NXTMAC 2494
NXTMDC 2495
NXTMSL 2496
NXTSTABUSSECT 2497
NXTSTANODE 2498
NXTSTASWDEV 2499
NXTSWS 2500
NXTTIE 2501
NXTTIE3 2502
NXTVSC 2503
OKSTRT 2504
OPENUN 2505
ORDBUS 2506
ORDBUSSECT 2507
OWNDAT 2508
OWNNAM 2510
OWNNUM 2511
OWNUSE 2512
PRMDAT 2513
PRMINT 2515
RATCHR 2516
RMODIND 2517
RMODNAM 2519
SBSGTBKV 2520
SC3WND 2521
SCBRN2 2523
SCBUS2 2524
SCDONE 2526
SCINIT 2527
SCMAC2 2528

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SELCTR 2529
SETHOMEPATH 2530
SFILES 2531
SLMODIND 2532
SLMODNAM 2534
SOLSTR 2535
SOLVED 2536
SOLVED_CA 2537
STADAT 2538
STAINT 2539
STANAME 2540
STANODEINT 2541
STANODENAME 2542
STAREA 2543
STASWDEVDAT 2544
STASWDEVINT 2546
STASWDEVNAME 2547
STASWDEVRSETNAME 2548
STBSKV 2549
STZONE 2550
SWD_RATING_DESC 2551
SWD_RATING_DESC_LENGTH 2552
SWD_RATING_NAME_DATA 2553
SWD_RATING_VALUE 2554
SWSBLK 2555
SWSBLK_2 2556
SWSBLZ 2558
SWSBLZ2 2559
SWSBUS 2560
SWSCHR 2561
SWSDAT 2562
SWSDAT_2 2563
SWSDT1 2564
SWSINT 2565
SWSINT_2 2566
SWSMIND 2567
SWSMIND_2 2569
SWSMNAM 2571
SWSMNAM_2 2572
SYSMSM 2573
SYSMVA 2574
SYSSWDEVRSETNAM 2575
SYSTOT 2576
TITLDT 2577
TOTBUS 2578
TR3DAT 2579
TR3DT2 2580
TR3INT 2582
TR3NAM 2584
TRNDAT 2585
TRXDAT 2586

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

TSPFPROFILECHR 2587
TSPFPROFILECURVECHR 2588
TSPFPROFILECURVEINT 2589
TSPFPROFILECURVEREAL 2590
TSPFPROFILECURVEVALUE 2591
TSPFPROFILEINT 2592
TSPFPROFILEREAL 2593
VOLTAGEDROOPDATA 2594
VOLTAGEDROOPINT 2595
VSCAUXMIND 2596
VSCAUXMNAM 2598
VSCCDT 2599
VSCCIN 2601
VSCINT 2602
VSCMIND 2603
VSCMNAM 2604
WINDMIND 2605
WINDMNAM 2607
WNDDAT 2608
WNDDT2 2610
WNDINT 2612
XFRDAT 2614
XFRINT 2616
XFRNAM 2618
ZNDAT 2619
ZNITOJ 2621
ZONNAM 2622
ZONNUM 2623
ZONUSE 2624
Extension Data Access 2625
GET_EXT_ACLINE_CHAR 2626
GET_EXT_ACLINE_INT 2627
GET_EXT_ACLINE_LOGICAL 2628
GET_EXT_ACLINE_REAL 2629
GET_EXT_AREA_CHAR 2630
GET_EXT_AREA_INT 2631
GET_EXT_AREA_LOGICAL 2632
GET_EXT_AREA_REAL 2633
GET_EXT_BUS_CHAR 2634
GET_EXT_BUS_INT 2635
GET_EXT_BUS_LOGICAL 2636
GET_EXT_BUS_REAL 2637
GET_EXT_CHAR 2638
GET_EXT_FACTS_CHAR 2639
GET_EXT_FACTS_INT 2640
GET_EXT_FACTS_LOGICAL 2641
GET_EXT_FACTS_REAL 2642
GET_EXT_FIXSHUNT_CHAR 2643
GET_EXT_FIXSHUNT_INT 2644
GET_EXT_FIXSHUNT_LOGICAL 2645
GET_EXT_FIXSHUNT_REAL 2646

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_EXT_GENERATOR_CHAR 2647
GET_EXT_GENERATOR_INT 2648
GET_EXT_GENERATOR_LOGICAL 2649
GET_EXT_GENERATOR_REAL 2650
GET_EXT_GNE_CHAR 2651
GET_EXT_GNE_INT 2652
GET_EXT_GNE_LOGICAL 2653
GET_EXT_GNE_REAL 2654
GET_EXT_IATRANS_CHAR 2655
GET_EXT_IATRANS_INT 2656
GET_EXT_IATRANS_LOGICAL 2657
GET_EXT_IATRANS_REAL 2658
GET_EXT_INDMACH_CHAR 2659
GET_EXT_INDMACH_INT 2660
GET_EXT_INDMACH_LOGICAL 2661
GET_EXT_INDMACH_REAL 2662
GET_EXT_INT 2663
GET_EXT_LOAD_CHAR 2664
GET_EXT_LOAD_INT 2665
GET_EXT_LOAD_LOGICAL 2666
GET_EXT_LOAD_REAL 2667
GET_EXT_LOGICAL 2668
GET_EXT_MSLINE_CHAR 2669
GET_EXT_MSLINE_INT 2670
GET_EXT_MSLINE_LOGICAL 2671
GET_EXT_MSLINE_REAL 2672
GET_EXT_NTERMDC_CHAR 2673
GET_EXT_NTERMDC_INT 2674
GET_EXT_NTERMDC_LOGICAL 2675
GET_EXT_NTERMDC_REAL 2676
GET_EXT_OWNER_CHAR 2677
GET_EXT_OWNER_INT 2678
GET_EXT_OWNER_LOGICAL 2679
GET_EXT_OWNER_REAL 2680
GET_EXT_RATING_CHAR 2681
GET_EXT_RATING_INT 2682
GET_EXT_RATING_LOGICAL 2683
GET_EXT_RATING_REAL 2684
GET_EXT_REAL 2685
GET_EXT_ROWCOUNT 2686
GET_EXT_SUBNODE_CHAR 2687
GET_EXT_SUBNODE_INT 2688
GET_EXT_SUBNODE_LOGICAL 2689
GET_EXT_SUBNODE_REAL 2690
GET_EXT_SUBSWD_CHAR 2691
GET_EXT_SUBSWD_INT 2692
GET_EXT_SUBSWD_LOGICAL 2693
GET_EXT_SUBSWD_REAL 2694
GET_EXT_SUB_CHAR 2695
GET_EXT_SUB_INT 2696
GET_EXT_SUB_LOGICAL 2697

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_EXT_SUB_REAL 2698
GET_EXT_SWSHUNT_CHAR 2699
GET_EXT_SWSHUNT_INT 2700
GET_EXT_SWSHUNT_LOGICAL 2701
GET_EXT_SWSHUNT_REAL 2702
GET_EXT_SYSSWD_CHAR 2703
GET_EXT_SYSSWD_INT 2704
GET_EXT_SYSSWD_LOGICAL 2705
GET_EXT_SYSSWD_REAL 2706
GET_EXT_TRANSFORMER_CHAR 2707
GET_EXT_TRANSFORMER_INT 2708
GET_EXT_TRANSFORMER_LOGICAL 2709
GET_EXT_TRANSFORMER_REAL 2710
GET_EXT_TWOTERMDC_CHAR 2711
GET_EXT_TWOTERMDC_INT 2712
GET_EXT_TWOTERMDC_LOGICAL 2713
GET_EXT_TWOTERMDC_REAL 2714
GET_EXT_VSCDC_CHAR 2715
GET_EXT_VSCDC_INT 2716
GET_EXT_VSCDC_LOGICAL 2717
GET_EXT_VSCDC_REAL 2718
GET_EXT_ZONE_CHAR 2719
GET_EXT_ZONE_INT 2720
GET_EXT_ZONE_LOGICAL 2721
GET_EXT_ZONE_REAL 2722
SET_EXT_ACLINE_CHAR 2723
SET_EXT_ACLINE_INT 2724
SET_EXT_ACLINE_LOGICAL 2725
SET_EXT_ACLINE_REAL 2726
SET_EXT_AREA_CHAR 2727
SET_EXT_AREA_INT 2728
SET_EXT_AREA_LOGICAL 2729
SET_EXT_AREA_REAL 2730
SET_EXT_BUS_CHAR 2731
SET_EXT_BUS_INT 2732
SET_EXT_BUS_LOGICAL 2733
SET_EXT_BUS_REAL 2734
SET_EXT_CHAR 2735
SET_EXT_FACTS_CHAR 2736
SET_EXT_FACTS_INT 2737
SET_EXT_FACTS_LOGICAL 2738
SET_EXT_FACTS_REAL 2739
SET_EXT_FIXSHUNT_CHAR 2740
SET_EXT_FIXSHUNT_INT 2741
SET_EXT_FIXSHUNT_LOGICAL 2742
SET_EXT_FIXSHUNT_REAL 2743
SET_EXT_GENERATOR_CHAR 2744
SET_EXT_GENERATOR_INT 2745
SET_EXT_GENERATOR_LOGICAL 2746
SET_EXT_GENERATOR_REAL 2747
SET_EXT_GNE_CHAR 2748

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SET_EXT_GNE_INT 2749
SET_EXT_GNE_LOGICAL 2750
SET_EXT_GNE_REAL 2751
SET_EXT_IATRANS_CHAR 2752
SET_EXT_IATRANS_INT 2753
SET_EXT_IATRANS_LOGICAL 2754
SET_EXT_IATRANS_REAL 2755
SET_EXT_INDMACH_CHAR 2756
SET_EXT_INDMACH_INT 2757
SET_EXT_INDMACH_LOGICAL 2758
SET_EXT_INDMACH_REAL 2759
SET_EXT_INT 2760
SET_EXT_LOAD_CHAR 2761
SET_EXT_LOAD_INT 2762
SET_EXT_LOAD_LOGICAL 2763
SET_EXT_LOAD_REAL 2764
SET_EXT_LOGICAL 2765
SET_EXT_MSLINE_CHAR 2766
SET_EXT_MSLINE_INT 2767
SET_EXT_MSLINE_LOGICAL 2768
SET_EXT_MSLINE_REAL 2769
SET_EXT_NTERMDC_CHAR 2770
SET_EXT_NTERMDC_INT 2771
SET_EXT_NTERMDC_LOGICAL 2772
SET_EXT_NTERMDC_REAL 2773
SET_EXT_OWNER_CHAR 2774
SET_EXT_OWNER_INT 2775
SET_EXT_OWNER_LOGICAL 2776
SET_EXT_OWNER_REAL 2777
SET_EXT_RATING_CHAR 2778
SET_EXT_RATING_INT 2779
SET_EXT_RATING_LOGICAL 2780
SET_EXT_RATING_REAL 2781
SET_EXT_REAL 2782
SET_EXT_SUBNODE_CHAR 2783
SET_EXT_SUBNODE_INT 2784
SET_EXT_SUBNODE_LOGICAL 2785
SET_EXT_SUBNODE_REAL 2786
SET_EXT_SUBSWD_CHAR 2787
SET_EXT_SUBSWD_INT 2788
SET_EXT_SUBSWD_LOGICAL 2789
SET_EXT_SUBSWD_REAL 2790
SET_EXT_SUB_CHAR 2791
SET_EXT_SUB_INT 2792
SET_EXT_SUB_LOGICAL 2793
SET_EXT_SUB_REAL 2794
SET_EXT_SWSHUNT_CHAR 2795
SET_EXT_SWSHUNT_INT 2796
SET_EXT_SWSHUNT_LOGICAL 2797
SET_EXT_SWSHUNT_REAL 2798
SET_EXT_SYSSWD_CHAR 2799

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SET_EXT_SYSSWD_INT 2800
SET_EXT_SYSSWD_LOGICAL 2801
SET_EXT_SYSSWD_REAL 2802
SET_EXT_TRANSFORMER_CHAR 2803
SET_EXT_TRANSFORMER_INT 2804
SET_EXT_TRANSFORMER_LOGICAL 2805
SET_EXT_TRANSFORMER_REAL 2806
SET_EXT_TWOTERMDC_CHAR 2807
SET_EXT_TWOTERMDC_INT 2808
SET_EXT_TWOTERMDC_LOGICAL 2809
SET_EXT_TWOTERMDC_REAL 2810
SET_EXT_VSCDC_CHAR 2811
SET_EXT_VSCDC_INT 2812
SET_EXT_VSCDC_LOGICAL 2813
SET_EXT_VSCDC_REAL 2814
SET_EXT_ZONE_CHAR 2815
SET_EXT_ZONE_INT 2816
SET_EXT_ZONE_LOGICAL 2817
SET_EXT_ZONE_REAL 2818
Subsystem Data Retrieval 2819
Character Data 2820
GET_CHAR_LENGTH 2820
Bus Data 2821
ABUSCHAR 2821
ABUSCOUNT 2823
ABUSCPLX 2824
ABUSINT 2826
ABUSREAL 2828
ABUSTYPES 2830
Plant Bus Data 2831
AGENBUSCHAR 2831
AGENBUSCOUNT 2833
AGENBUSCPLX 2834
AGENBUSINT 2836
AGENBUSREAL 2838
AGENBUSTYPES 2841
Machine Data 2842
AMACHCHAR 2842
AMACHCOUNT 2844
AMACHCPLX 2845
AMACHINT 2847
AMACHREAL 2849
AMACHTYPES 2852
Load Bus Data 2853
ALODBUSCHAR 2853
ALODBUSCOUNT 2855
ALODBUSCPLX 2857
ALODBUSINT 2860
ALODBUSREAL 2862
ALODBUSTYPES 2865
Load Data 2866

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ALOADCHAR 2866
ALOADCOUNT 2868
ALOADCPLX 2869
ALOADINT 2872
ALOADREAL 2874
ALOADTYPES 2877
Fixed Shunt Bus Data 2878
AFXSHNTBUSCHAR 2878
AFXSHNTBUSCOUNT 2880
AFXSHNTBUSCPLX 2881
AFXSHNTBUSINT 2883
AFXSHNTBUSREAL 2885
AFXSHNTBUSTYPES 2887
Fixed Shunt Data 2888
AFXSHUNTCHAR 2888
AFXSHUNTCOUNT 2890
AFXSHUNTCPLX 2891
AFXSHUNTINT 2893
AFXSHUNTREAL 2895
AFXSHUNTTYPES 2897
Switched Shunt Bus Data 2898
ASWSHUNTBUSCHAR 2898
ASWSHUNTBUSCOUNT 2900
ASWSHUNTBUSCPLX 2901
ASWSHUNTBUSINT 2903
ASWSHUNTBUSREAL 2905
ASWSHUNTBUSTYPES 2907
Switched Shunt Data 2908
ASWSHCHAR 2908
ASWSHCOUNT 2910
ASWSHCPLX 2911
ASWSHINT 2913
ASWSHREAL 2915
ASWSHTYPES 2919
Branch Flow Data 2920
AFLOWCHAR 2920
AFLOWCOUNT 2923
AFLOWCPLX 2925
AFLOWINT 2927
AFLOWREAL 2929
AFLOWTYPES 2932
Branch Data 2933
ABRNCHAR 2933
ABRNCOUNT 2936
ABRNCPLX 2938
ABRNINT 2940
ABRNREAL 2943
ABRNTYPES 2951
Two-Winding Transformer Data 2952
ATRNCHAR 2952
ATRNCOUNT 2955

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ATRNCPLX 2957
ATRNINT 2960
ATRNREAL 2963
ATRNTYPES 2971
Three-Winding Transformer Data 2972
ATR3CHAR 2972
ATR3COUNT 2975
ATR3CPLX 2977
ATR3INT 2980
ATR3REAL 2983
ATR3TYPES 2985
Three-Winding Transformer Winding Data 2986
AWNDCHAR 2986
AWNDCOUNT 2989
AWNDCPLX 2991
AWNDINT 2993
AWNDREAL 2996
AWNDTYPES 3004
Two-Terminal dc Line Data 3005
A2TRMDCCHAR 3005
A2TRMDCCOUNT 3007
A2TRMDCCPLX 3008
A2TRMDCINT 3010
A2TRMDCREAL 3012
A2TRMDCTYPES 3016
Two-Terminal dc Line Converter Data 3017
A2TRMDCCONVCHAR 3017
A2TRMDCCONVCOUNT 3020
A2TRMDCCONVCPLX 3022
A2TRMDCCONVINT 3024
A2TRMDCCONVREAL 3026
A2TRMDCCONVTYPES 3029
Multi-Terminal dc Line Data 3030
AMULTITRMDCCHAR 3030
AMULTITRMDCCOUNT 3032
AMULTITRMDCCPLX 3033
AMULTITRMDCINT 3035
AMULTITRMDCREAL 3037
AMULTITRMDCTYPES 3039
Multi-Terminal dc Line Converter Data 3040
AMULTITRMDCCONVCHAR 3040
AMULTITRMDCCONVCOUNT 3043
AMULTITRMDCCONVCPLX 3045
AMULTITRMDCCONVINT 3047
AMULTITRMDCCONVREAL 3049
AMULTITRMDCCONVTYPES 3052
VSC dc Line Data 3053
AVSCDCCHAR 3053
AVSCDCCOUNT 3055
AVSCDCCPLX 3056
AVSCDCINT 3058

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

AVSCDCREAL 3060
AVSCDCTYPES 3062
VSC dc Line Converter Data 3063
AVSCDCCONVCHAR 3063
AVSCDCCONVCOUNT 3066
AVSCDCCONVCPLX 3068
AVSCDCCONVINT 3070
AVSCDCCONVREAL 3072
AVSCDCCONVTYPES 3075
FACTS Device Data 3076
AFACTSCHAR 3076
AFACTSCOUNT 3078
AFACTSCPLX 3080
AFACTSINT 3082
AFACTSREAL 3084
AFACTSTYPES 3088
FACTS Device Bus Data 3089
AFACTSBUSCHAR 3089
AFACTSBUSCOUNT 3092
AFACTSBUSCPLX 3094
AFACTSBUSINT 3096
AFACTSBUSREAL 3099
AFACTSBUSTYPES 3102
Area Data 3103
AAREACHAR 3103
AAREACOUNT 3105
AAREACPLX 3106
AAREAINT 3108
AAREAREAL 3110
AAREATYPES 3113
Owner Data 3114
AOWNERCHAR 3114
AOWNERCOUNT 3116
AOWNERCPLX 3117
AOWNERINT 3119
AOWNERREAL 3121
AOWNERTYPES 3124
Zone Data 3125
AZONECHAR 3125
AZONECOUNT 3127
AZONECPLX 3128
AZONEINT 3130
AZONEREAL 3132
AZONETYPES 3135
Induction Machine Bus Data 3136
AINDMACBUSCHAR 3136
AINDMACBUSCOUNT 3138
AINDMACBUSCPLX 3140
AINDMACBUSINT 3142
AINDMACBUSREAL 3144
AINDMACBUSTYPES 3147

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Induction Machine Data 3148
AINDMACCHAR 3148
AINDMACCOUNT 3150
AINDMACCPLX 3152
AINDMACINT 3154
AINDMACREAL 3157
AINDMACTYPES 3160
Substation Data 3161
ASTATIONCHAR 3161
ASTATIONCOUNT 3163
ASTATIONINT 3164
ASTATIONREAL 3166
ASTATIONTYPES 3167
Substation Node Data 3168
ANODECHAR 3168
ANODECOUNT 3170
ANODEINT 3171
ANODEREAL 3173
ANODETYPES 3174
Substation Switching Device Data 3175
ASTASWDEVCHAR 3175
ASTASWDEVCOUNT 3177
ASTASWDEVCPLX 3178
ASTASWDEVINT 3180
ASTASWDEVREAL 3182
ASTASWDEVTYPES 3185
Substation Terminal Data 3186
ATERMINALCHAR 3186
ATERMINALCOUNT 3188
ATERMINALINT 3189
ATERMINALTYPES 3191
Zero Sequence Mutuals Data 3192
AZMUTCHAR 3192
AZMUTCOUNT 3194
AZMUTCPLX 3196
AZMUTINT 3198
AZMUTREAL 3200
AZMUTTYPES 3202
Graphical Analysis Output 3204
POLY_PRINT 3205
PV_PRINT 3206
QV_PRINT 3208
SETFULLVIEWGRAPHAREA 3209
SETFULLVIEWSCALE 3212
Study Project 3214
CLOSESCENARIOFILE 3215
NEWSCENARIOFILE 3216
OPENSCENARIOFILE 3217
SAVESCENARIOFILE 3218
SCENARIOCLOSE 3219
SCENARIOCOPYFILE 3220

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SCENARIOFILEADD 3221
SCENARIOFILECOUNT 3222
SCENARIOFILEGETATTRS 3223
SCENARIOFILEGETCOMMENTS 3224
SCENARIOFILELENGTHCOMMENTS 3225
SCENARIOFILELIST 3226
SCENARIOFILEREMOVE 3227
SCENARIOFILERENAME 3228
SCENARIOFILESETATTRS 3229
SCENARIOFILESETCOMMENTS 3230
SCENARIOGETATTRS 3231
SCENARIOGETCOMMENTS 3232
SCENARIOGETOPT 3233
SCENARIOGROUPADD 3235
SCENARIOGROUPCOUNT 3236
SCENARIOGROUPGETCOMMENTS 3237
SCENARIOGROUPLENGTHCOMMENTS 3238
SCENARIOGROUPLIST 3239
SCENARIOGROUPOPEN 3240
SCENARIOGROUPREMOVE 3241
SCENARIOGROUPRENAME 3242
SCENARIOGROUPSETCOMMENTS 3243
SCENARIOISDIRTY 3244
SCENARIOISVALID 3245
SCENARIOLENGTHCOMMENTS 3246
SCENARIOMODELGROUP 3247
SCENARIOMOVEFILE 3248
SCENARIONEW 3249
SCENARIOOPEN 3250
SCENARIOSAVE 3251
SCENARIOSETATTRS 3252
SCENARIOSETCOMMENTS 3253
SCENARIOSETOPT 3254
SCENARIOTRACKFILEADD 3256
SCENARIOTRACKFILELIST 3257
SCENARIOTRACKFILENUM 3258
SCENARIOUNZIP 3259
SCENARIOUNZIPPROBLEMS 3260
SCENARIOUNZIPSOLUTIONS 3261
SCENARIOZIP 3262
SCENARIOZIPACTIVE 3263
SCENARIOZIPOPEN 3264
SCENARIOZIPSAVE 3265
STDYPROJCHECKALLTASKS 3266
STDYPROJCURRENTREPORTSETLOCKED 3267
STDYPROJCURRENTVARIANTGETLOCKED 3268
STDYPROJCURRENTVARIANTSETLOCKED 3269
STDYPROJEXTRACTDATASET 3270
STDYPROJEXTRACTSIMULATIONRESULT 3271
STDYPROJGETCURRENTSIMULATION 3272
STDYPROJGETCURRENTVARIANT 3273

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STDYPROJGETSETTINGS 3274
STDYPROJINPUTDATASETADD 3275
STDYPROJINPUTDATASETREMOVE 3276
STDYPROJIPLANLAUNCH 3277
STDYPROJNEW 3278
STDYPROJOPEN 3279
STDYPROJPYTHONLAUNCH 3280
STDYPROJRENAME 3281
STDYPROJREPORTADD 3282
STDYPROJREPORTREMOVE 3283
STDYPROJREPORTRENAME 3284
STDYPROJREPORTSETLOCK 3285
STDYPROJRESPONSELAUNCH 3286
STDYPROJSETCURRENTREPORT 3287
STDYPROJSETCURRENTSIMULATION 3288
STDYPROJSETCURRENTVARIANT 3289
STDYPROJSETSETTINGS 3290
STDYPROJSIMULATIONADD 3291
STDYPROJSIMULATIONREMOVE 3292
STDYPROJSIMULATIONRENAME 3293
STDYPROJSIMULATIONSETLOCK 3294
STDYPROJTASKOPT 3295
STDYPROJTASKSTATUS 3296
STDYPROJVARIANTADD 3297
STDYPROJVARIANTCLONE 3298
STDYPROJVARIANTCOMPARE 3299
STDYPROJVARIANTGETLOCK 3300
STDYPROJVARIANTREMOVE 3301
STDYPROJVARIANTRENAME 3302
STDYPROJVARIANTSETLOCK 3303
STDYPROJWAITALLTASKS 3304
Event Studies 3305
CLOSEEVENTSTUDYFILE 3306
OPENEVENTSTUDYFILE 3307
RUNDYNAMICEVENTSTUDY 3308
RUNPOWERFLOWEVENTSTUDY 3309
SAVEEVENTSTUDYFILE 3310
SETACTIVEEVENTSTUDY 3311
Measurement Interface 3312
Multiple Element Data 3313
AGCAPCOUNT 3314
AGCAPREAL 3315
AINTERFACECHAR 3316
AINTERFACECOUNT 3317
AINTERFACEELEMENTCHAR 3318
AINTERFACEELEMENTCOUNT 3319
AINTERFACEELEMENTCPLX 3320
AINTERFACEELEMENTINT 3321
AINTERFACEELEMENTREAL 3322
AINTERFACEINT 3323
AINTERFACEREAL 3324

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

AVOLTAGEDROOPCHAR 3326
AVOLTAGEDROOPCOUNT 3327
AVOLTAGEDROOPINT 3328
AVOLTAGEDROOPREAL 3329
GETMODELLIBRARIES 3331
GETMODELPROGRAMS 3332
GETTSPFPROFILEBYELEMENT 3333
GETTSPFPROFILECOUNTBYELEMENT 3334
GETTSPFPROFILEDATA 3335
GETTSPFPROFILEDATABYSET 3336
GETTSPFPROFILEDATACOLUMNCOUNT 3337
GETTSPFPROFILEDATACOUNT 3338
LAST_PF 3339
RXPATH 3340
SWD_RATINGS 3341
TREEDAT 3342
Using the Vendor Neutral Contingency and RAS Auxiliary File 3344
ACCC_RAS 3345
ACCC_RAS_2 3346
ADD2WINDINGCONDITIONELEMENT 3348
ADD2WINDINGCONTINGENCYELEMENT 3350
ADD2WINDINGREMEDIALACTIONELEMENT 3353
ADD3WINDINGCONDITIONELEMENT 3356
ADD3WINDINGCONTINGENCYELEMENT 3358
ADD3WINDINGREMEDIALACTIONELEMENT 3361
ADDACLINECONDITIONELEMENT 3365
ADDACLINECONTINGENCYELEMENT 3367
ADDACLINEREMEDIALACTIONELEMENT 3370
ADDBUSCONDITIONELEMENT 3373
ADDBUSCONTINGENCYELEMENT 3375
ADDBUSREMEDIALACTIONELEMENT 3377
ADDCONDITION 3380
ADDCONDITIONELEMENT 3382
ADDCONTINGENCY 3384
ADDCONTINGENCYELEMENT 3385
ADDDCCONVCONDITIONELEMENT 3387
ADDDCCONVCONTINGENCYELEMENT 3389
ADDDCCONVREMEDIALACTIONELEMENT 3391
ADDDCLINECONDITIONELEMENT 3394
ADDDCLINECONTINGENCYELEMENT 3396
ADDDCLINEREMEDIALACTIONELEMENT 3398
ADDFIXSHUNTCONDITIONELEMENT 3401
ADDFIXSHUNTCONTINGENCYELEMENT 3403
ADDFIXSHUNTREMEDIALACTIONELEMENT 3405
ADDGENCONDITIONELEMENT 3408
ADDGENCONTINGENCYELEMENT 3410
ADDGENREMEDIALACTIONELEMENT 3412
ADDLOADCONDITIONELEMENT 3415
ADDLOADCONTINGENCYELEMENT 3417
ADDLOADREMEDIALACTIONELEMENT 3419
ADDNESTEDCONDITION 3422

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ADDPYTHONCONDITIONELEMENT 3423
ADDPYTHONCONTINGENCYELEMENT 3425
ADDPYTHONREMEDIALACTIONELEMENT 3427
ADDREMEDIALACTION 3429
ADDREMEDIALACTIONELEMENT 3430
ADDSUBCONDITIONELEMENT 3433
ADDSUBCONTINGENCYELEMENT 3435
ADDSUBREMEDIALACTIONELEMENT 3437
ADDVSCDCCONDITIONELEMENT 3440
ADDVSCDCCONTINGENCYELEMENT 3442
ADDVSCDCREMEDIALACTIONELEMENT 3444
APPEND_RAS 3447
READ_RAS 3448
VALIDATE_RAS 3449
WRITE_RAS 3450
Non-Engineering 3452
ALERT 3453
APPENDRECORDING 3454
BEGINREPORT 3455
BEGINREPORT_2 3456
CLEARALERTOUTPUT 3457
CLEARPROGRESSOUTPUT 3458
FILEIN 3459
GETBATDEFAULTS 3460
GETDEFAULTCHAR 3461
GETDEFAULTINT 3462
GETDEFAULTREAL 3463
GETLOADEDMODULES 3464
GETMODFUNCLIST 3465
IOFLUSH 3466
LAUNCH_PROGRAM 3467
PAGEREPORT 3468
PAUSERECORDING 3469
PROGRESS 3470
PROMPT 3471
PSSEINIT 3472
PSSEVERSION 3473
REFRESHGUI 3474
REPORT 3475
RUNIPLANFILE 3476
RUNRSPNSFILE 3477
SHOWLOADEDLIBRARIES 3478
SHOWLOADEDMODULES 3479
STARTRECORDING 3480
STOPRECORDING 3481
T_ALERT_OUTPUT 3482
T_PROGRESS_OUTPUT 3484
T_PROMPT_OUTPUT 3486
T_REPORT_OUTPUT 3488
USERIN 3490
PSSPLOT Module 3491

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ADDFUNCTION 3492
CHANNELFILEEXCELEXPORT 3493
DELETESELPLOT 3494
DRAGDROPPLOTDATA 3495
EXPORTPLOT 3496
INSERTPAGE 3497
INSERTPLOT 3498
MULTICHANNEXCELEXPORT 3499
NEWPLOTBOOK 3500
OPENCHANDATAFILE 3501
PLOTPAGEPRINTSETUP 3502
PLOT_BOOK 3503
PLOT_BOOK_MBID 3504
PLOT_BOOK_PRINT 3505
PLOT_BOOK_PURG 3507
PLOT_BOOK_SET_ACTIVE 3508
PLOT_PAGE 3509
PLOT_PAGE_CHNG 3510
PLOT_PAGE_PRINT 3511
PLOT_PAGE_PURG 3513
PLOT_PAGE_SET_ACTIVE 3514
PLOT_PLOT 3515
PLOT_PLOT_CHNG 3517
PLOT_PLOT_EXPORT_IMAGE 3519
PLOT_PLOT_PURG 3520
PLOT_PLOT_SET_ACTIVE 3521
PLOT_TABLE_COLUMN 3522
PLOT_TABLE_COLUMN_RANGE 3524
PLOT_TRACE_CHANNEL 3526
PLOT_TRACE_PURG 3528
PLOT_TRACE_XY 3529
PRINTACTIVEPAGE 3531
PRINTSELPLOT 3532
SETSELECTEDPAGE 3533
SETSELECTEDPLOT 3534
SHOWPLOTEDITOR 3535
XYPLOT 3536
PSSRDB Module 3537
MUSTPY Module 3538
MUST Analysis Functions 3539
ACCDEFAULTS 3539
ACCONTBYCONT 3541
ACCONTBYMONEL 3543
ACCSOLVE 3545
ACLFDC 3547
ACLFNS 3549
ACSOLVE_LFD 3551
ACSOLVE_LFN 3552
ACTIVITYACCONT 3553
ACTIVITYMUST 3554
ACTRANSFERLIMITSTUDY 3555

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ADDTRANSACTION 3556
APPLYSINGLECONT 3557
AREABUBBLE 3558
AREAIMPACT 3559
AREASINZONES 3561
AREASLACKS 3562
AREASUMMARY 3563
BRANCHSENSITIVITY 3564
BUBBLEREPORT 3566
CDETAILBRANCHES 3567
CDETAILINTERFACES 3568
CDETAILVOLTAGEBUSES 3569
CHANGEPARAMS 3570
CLEARMULTISTUDY 3577
CNALLTRANSELBRANCHES 3578
CNALLTRANSELFLOWGATES 3579
CNALLTRANSELINTERFACES 3580
CNALLVIOLATED 3581
CNALLVIOLFREQ 3582
CNBASEVIOLATED 3583
CNCINITSELBRANCHES 3584
CNCINITSELINTERFACES 3585
CNCOMPSELBRANCHES 3586
CNCOMPSELINTERFACES 3587
CNCOMPSELVOLTAGEBUSES 3588
CNCRESULTSSELBRANCHES 3589
CNCRESULTSSELINTERFACES 3590
CNDETAILSELFLOWGATES 3591
CNDETAILSELINTERFACES 3592
CNFACTORSSELFLOWGATES 3593
CNFREQSELBRANCHES 3594
CNFREQSELINTERFACES 3595
CNFREQSELVOLTAGEBUSES 3596
CNGAPPSELBRANCHES 3597
CNGAPPSELFLOWGATES 3598
CNGAPPSELINTERFACES 3599
CNIMPACTSELBRANCHES 3600
CNIMPACTSELFLOWGATES 3601
CNIMPACTSELINTERFACES 3602
CNLISTSELBRANCHES 3603
CNLISTSELFLOWGATES 3604
CNLISTSELINTERFACES 3605
CNLISTSELVOLTAGEBUSES 3606
CNOTDFSELBRANCHES 3607
CNOTDFSELINTERFACES 3608
CNSELECTBRANCHBYALL 3609
CNSELECTBRANCHBYAREAS 3610
CNSELECTBRANCHBYBRANCHES 3611
CNSELECTBRANCHBYBUSES 3612
CNSELECTBRANCHBYIMPACT 3613
CNSELECTBRANCHBYTIES 3614

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CNSELECTBRANCHBYVIOL 3615
CNSELECTBRANCHBYZONES 3616
CNSELECTFGATEBYALL 3617
CNSELECTFGATEBYCONT 3618
CNSELECTFGATEBYID 3619
CNSELECTFGATEBYIMPACT 3620
CNSELECTFGATEBYVIOL 3621
CNSELECTINTERBYALL 3622
CNSELECTINTERBYIMPACT 3623
CNSELECTINTERBYNAME 3624
CNSELECTINTERBYNUMBER 3625
CNSELECTINTERBYVIOL 3626
CNSELECTVOLTAGEBUSESBYVIOL 3627
CNSELECTVOLTBUSBYALL 3628
CNSELECTVOLTBUSBYAREAS 3629
CNSELECTVOLTBUSBYBUSES 3630
CNSELECTVOLTBUSBYZONES 3631
CNSELECTVOLTBUSESBYDEVIATION 3632
CNSENSUBSELBRANCHES 3633
CNSENSUBSELFLOWGATES 3634
CNSENSUBSELINTERFACES 3635
CNSTUDYSELBRANCHES 3636
CNSTUDYSELFLOWGATES 3637
CNSTUDYSELINTERFACES 3638
CNSUBSELBRANCHES 3639
CNSUBSELFLOWGATES 3640
CNSUBSELINTERFACES 3641
CNSUMMARYVIOLATIONS 3642
CNTRANSACTSELBRANCHES 3643
CNTRANSACTSELFLOWGATES 3644
CNTRANSACTSELINTERFACES 3645
CNVARSELBRANCHES 3646
CNVARSELINTERFACES 3647
CNVARSELVOLTAGEBUSES 3648
CNVIOLSELBRANCHES 3649
CNVIOLSELFLOWGATES 3650
CNVIOLSELINTERFACES 3651
CNVIOLSELVOLTAGEBUSES 3652
COMPAREFCITC 3653
CONSTRDEFS 3654
DCCONTBYCONT 3656
DCCONTBYMONEL 3658
DCSOLVE 3660
DCTRANSFERLIMITSTUDY 3661
DETAILSELCONTINGENCIES 3662
DETAILSELCONTINGENCIESSP 3663
DIRECT 3664
EXCLUDESELECTEDCONTS 3665
EXECUTEACCONT 3666
EXITACTIVITYACCONT 3667
EXITACTIVITYMUST 3668

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

FCITC2LINE 3669
FCITCAC 3670
FCITCACMULT 3671
FCITCACSINGLE 3672
FCITCBASE 3673
FCITCBASEMULT 3674
FCITCBASESINGLE 3675
FCITCMULT 3676
FCITCSINGLE 3677
FCITCSUBS 3678
FCITCTLTG 3679
FCITCWIDE 3680
FLOWGATESENSITIVITY 3681
FORGETSTUDYTRANSFER 3683
GENDETAILS 3684
GENERATIONSENSITIVITY 3685
GENSENDETAIL 3687
GENSENSUMMARY 3688
ILFDEFAULTS 3689
IMPACTINTRFACEPARTFACT 3691
IMPACTSETBASECASE 3693
IMPACTSETCONTINGENCY 3694
IMPACTTIESPARTFACT 3695
IMPACTTOTALPARTFACT 3697
INCLUDESELECTEDCONTS 3698
INTERFACESENSITIVITY 3699
LISTACCONTDETAILS 3701
LISTACCONTSUMMARY 3702
LISTAREAS 3703
LISTBUSES 3704
LISTEXCLUDES 3705
LISTGENERATORS 3706
LISTPHASESHIFTERS 3707
LISTSELCONTINGENCIES 3708
LISTZONES 3709
LIST_BFLOW 3710
LIST_BIMPACT 3711
LIST_CFLOW 3712
LIST_CIMPACT 3713
LIST_CONTINS 3714
LIST_CONTINS_WISL 3715
LIST_LODF 3716
LIST_LODF_WISL 3717
LIST_MONELS 3718
LIST_PTDF 3719
LIST_PTDFC 3720
LIST_SYSTEMS 3721
MDETAILBRANCHES 3722
MDETAILINTERFACES 3723
MDETAILVOLTAGEBUSES 3724
MODIFYDISPATCH 3725

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MONELIMPACT 3726
MONSENSALLSUBBRANCHES 3728
MONSENSALLSUBFLOWGATES 3729
MONSENSALLSUBINTERFACES 3730
MONSENSALLTRANBRANCHES 3731
MONSENSALLTRANFLOWGATES 3732
MONSENSALLTRANINTERFACES 3733
MONSENSSINGLESUBBRANCHES 3734
MONSENSSINGLESUBFLOWGATES 3735
MONSENSSINGLESUBINTERFACES 3736
MONSENSSUBSPARTBRANCHES 3737
MONSENSSUBSPARTFLOWGATES 3738
MONSENSSUBSPARTINTERFACES 3739
MONSENSTRANBRANCHES 3740
MONSENSTRANFLOWGATES 3741
MONSENSTRANINTERFACES 3742
MULTACFCITC 3743
MULTDCFCITC 3745
MULTIPLEPARALLELTRANSFERS 3746
MULTIPLESTUDYTRANSFERS 3748
MUSTOPTDEFS 3750
MUSTSTART 3752
MUSTSTOP 3753
PARALLELDETAILTRANSFER 3754
PARALLELFCITCSUMMARY 3755
PARALLELTRANSFERDETAIL 3756
PRINTRPT 3758
PSCASE 3759
RDEXCHGFILE 3760
RDMONCONFILE 3761
RDMUSTFILES 3762
RDSUBFILE 3764
READACCONTFILE 3765
READRAWDATA 3766
REAREABUS 3767
REMOVEEXCLUDES 3768
REZONEBUS 3769
SAVEACCONTFILE 3770
SAVEFLOWGATES 3771
SAVESELECTEDCONTS 3772
SAVESUBSYSTEMS 3773
SAVEXCHGFILE 3774
SCEDANALYSIS 3775
SELECTACTIVECONTS 3777
SELECTALLCONTS 3778
SELECTCOMPLEXCONTS 3779
SELECTCONVERGECONTS 3780
SELECTDISABLEDCONTS 3781
SELECTEXCLUDEDCONTS 3782
SELECTGENCONTS 3783
SELECTHISEVCONTS 3784

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SELECTIMBALANCECONTS 3785
SELECTISLANDCONTS 3786
SELECTLOADCONTS 3787
SELECTLOSEVCONTS 3788
SELECTNOTCONVERGECONTS 3789
SELECTNOVIOLCONTS 3790
SELECTSINGLEBRANCHCONTS 3791
SELECTSPECIFICCONTS 3792
SELECTVIOLCONTS 3793
SETEXCLUDEDBRANCHES 3794
SETEXCLUDEDCONTINGENCIES 3795
SETEXCLUDEDFLOWGATES 3796
SETEXCLUDEDINTERFACES 3797
SETEXCLUDEDVOLTAGEBUSES 3798
SGLACFCITC 3799
SGLDCFCITC 3801
SHOWPARAMS 3803
SUBSYSTEMS 3805
SUMMARYCONTINGENCIES 3806
SWINGBUSES 3807
VIOLSELCONTINGENCIES 3808
WRITEEXCLUDES 3809
WRITERAWDATA 3810
ZONEBUBBLE 3811
ZONESINAREAS 3812
ZONESUMMARY 3813
MUST Data Retrieval 3814
GETAUTOBUBBLE 3814
GETEXCLUDEDBRANCHES 3815
GETEXCLUDEDCONTINGENCIES 3816
GETEXCLUDEDFLOWGATES 3817
GETEXCLUDEDINTERFACES 3818
GETEXCLUDEDVOLTAGEBUSES 3819
GETFINALBALANCE 3820
GETFINALFLOW 3821
GETIDOFCONTINGENCYBYNAME 3822
GETIDOFFLOWGATEBYNAME 3823
GETIDOFINTERFACEBYNAME 3824
GETIDOFVOLTAGEBUSBYNAME 3825
GETINITBALANCE 3826
GETINITIALFLOW 3827
GETIPFFACTOR 3828
GETNAMEOFCONTINGENCYBYID 3829
GETNAMEOFFLOWGATEBYID 3830
GETNAMEOFINTERFACEBYID 3831
GETNAMEOFVOLTAGEBUSBYID 3832
GETNUMAUTOBUBBLE 3833
GETNUMOFEXCLUDEDBRANCHES 3834
GETNUMOFEXCLUDEDCONTINGENCIES 3835
GETNUMOFEXCLUDEDFLOWGATES 3836
GETNUMOFEXCLUDEDINTERFACES 3837

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GETNUMOFEXCLUDEDVOLTAGEBUSES 3838
GETTPFBALANCE 3839
MUSTRESULTSEXIST 3840
PSSDYNMDL Module 3842
Dynamics model parameter retrieval functions 3843
GET_CONMSGERROR 3843
GET_CONMSGNONE 3844
GET_CONMSGWARN 3845
GET_DOCUCHKERR_EQUAL 3846
GET_DOCUCHKERR_GREATEROREQ 3847
GET_DOCUCHKERR_GREATERTHAN 3848
GET_DOCUCHKERR_INCLHIRANGE 3849
GET_DOCUCHKERR_INCLLOWRANGE 3850
GET_DOCUCHKERR_INCLUSIVERANGE 3851
GET_DOCUCHKERR_LESSOREQ 3852
GET_DOCUCHKERR_LESSTHAN 3853
GET_DOCUCHKERR_NOTEQUAL 3854
GET_DOCUCHKERR_NOTINCLHIRANGE 3855
GET_DOCUCHKERR_NOTINCLLOWRANGE 3856
GET_DOCUCHKERR_NOTINCLRANGE 3857
GET_DOCUCHKERR_NOTRANGE 3858
GET_DOCUCHKERR_RANGE 3859
GET_MSGTYPE_ERROR 3860
GET_MSGTYPE_INFO 3861
GET_MSGTYPE_WARN 3862
GET_W7PRG 3863
GET_W7RPT 3864
Dynamics model value retrieval functions 3865
GET_ACBUS 3865
GET_ALFA 3866
GET_ALFMIN 3867
GET_ALFMX 3868
GET_ANGLE 3869
GET_ANGMAX 3870
GET_ANGMIN 3871
GET_B2S 3872
GET_BASFRQ 3873
GET_BASVLT 3874
GET_BSFREQ 3875
GET_BSWICH 3876
GET_BUSNAM 3877
GET_CDGNFR 3878
GET_CFDLFR 3879
GET_CHRICN 3880
GET_CLODFR 3881
GET_CNVANG 3882
GET_CNVCUR 3883
GET_CNVTAP 3884
GET_CNVTYP 3885
GET_CON 3886
GET_CON_DSCRPT 3887

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_CURNT 3888
GET_DC 3889
GET_DC2NAM 3890
GET_DC2SIG 3891
GET_DCIND 3892
GET_DCMSIG 3893
GET_DCNAME 3894
GET_DCPF 3895
GET_DCRES 3896
GET_DCSETP 3897
GET_DELT 3898
GET_DELTI 3899
GET_DERMWTHRESH 3900
GET_DERVTTHRESH 3901
GET_DSTATE 3902
GET_EBASE 3903
GET_EBASEI 3904
GET_EBASER 3905
GET_ECOMP 3906
GET_EFD 3907
GET_ETERM 3908
GET_FCNAME 3909
GET_FCREM 3910
GET_FCSBUS 3911
GET_FCSHMX 3912
GET_FCTBUS 3913
GET_FCTREF 3914
GET_FCTSIG 3915
GET_FCVSET 3916
GET_FQSHNT 3917
GET_FRELVTHRESH 3918
GET_FREQS 3919
GET_GAMA 3920
GET_GAMMIN 3921
GET_GAMMX 3922
GET_GB 3923
GET_GBLINE 3924
GET_GENTAP 3925
GET_GREF 3926
GET_ICOM 3927
GET_ICON 3928
GET_ICON_DSCRPT 3929
GET_IFLAG 3930
GET_IMARYS 3931
GET_IMCCNX 3932
GET_IMELTRQ 3933
GET_IMLDTRQ 3934
GET_IMSPARE 3935
GET_IMTPLOD 3936
GET_IMTQLOD 3937
GET_INDX3W 3938

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_INTICN 3939
GET_IPI 3940
GET_IPR 3941
GET_IPRT 3942
GET_ISORCE 3943
GET_ITER 3944
GET_ITMXDS 3945
GET_JCODE 3946
GET_KNUM 3947
GET_KPAUSE 3948
GET_KTRIP 3949
GET_LCREF 3950
GET_LDAREA 3951
GET_LDDGNFL 3952
GET_LDOWNR 3953
GET_LDSTAT 3954
GET_LDSTR2 3955
GET_LDSTRT 3956
GET_LDZONE 3957
GET_LOADID 3958
GET_LODBKVTHRESH 3959
GET_LODID 3960
GET_LODMWTHRESH 3961
GET_LODPQTHRESH 3962
GET_LODVTTHRESH 3963
GET_LSYSAN 3964
GET_MACBLKD 3965
GET_MACHID 3966
GET_MACNUM 3967
GET_MARGIN 3968
GET_MBASE 3969
GET_MCARYS 3970
GET_MCSTAT 3971
GET_MDC 3972
GET_MDCNAM 3973
GET_MIDTRM 3974
GET_MODE 3975
GET_MSTATE 2411
GET_NBI 3977
GET_NBR 3978
GET_NBRIDG 3979
GET_NBUS 3980
GET_NCONV 3981
GET_NINTEG 3982
GET_NLIN 3983
GET_NLWTRB 3984
GET_NMACHS 3985
GET_NSWSTP 3986
GET_NTAPOS 3987
GET_NUMARE 3988
GET_NUMLOD 3989

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_NUMOWN 3990
GET_NUMSWS 3991
GET_NUMTRM 3992
GET_NUMZON 3993
GET_OEL_ACTIVE 3994
GET_OLDERV 3995
GET_ORGDCMODE 3996
GET_ORGFCSTAT 3997
GET_ORGICOM 3998
GET_ORGJCODE 3999
GET_ORGMDC 4000
GET_ORGVSCMDC 4001
GET_ORGVSCTYP 4002
GET_PAC 4003
GET_PACI 4004
GET_PACR 4005
GET_PELEC 4006
GET_PGENMC 4007
GET_PHSHTB 4008
GET_PHSVLT 4009
GET_PMAXMC 4010
GET_PMAXMCD 4011
GET_PMECH 4012
GET_PMINMC 4013
GET_PMINMCD 4014
GET_PQCNOD 4015
GET_PQCONT 4016
GET_PSANGL 4017
GET_QAC 4018
GET_QACI 4019
GET_QACR 4020
GET_QELEC 4021
GET_QGENMC 4022
GET_QMAXMC 4023
GET_QMINMC 4024
GET_RCI 4025
GET_RCOMP 4026
GET_RCONV 4027
GET_RCR 4028
GET_RDC 4029
GET_RLSTRT 4030
GET_RMAX 4031
GET_RMIN 4032
GET_SBASE 4033
GET_SETVAL 4034
GET_SHFRST 4035
GET_SHNEXT 4036
GET_SHNTGB 4037
GET_SHNTID 4038
GET_SHSTAT 4039
GET_SPEED 4040

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_STATE 4041
GET_STATE_DSCRPT 4042
GET_STORE 4043
GET_STRT2D 4044
GET_STRTAU 4045
GET_STRTCCT 4046
GET_STRTFC 4047
GET_STRTIM 4048
GET_STRTIN 4049
GET_STRTMD 4050
GET_STRTSS 4051
GET_STRTVS 4052
GET_SWSAUX 4053
GET_SWSCNX 4054
GET_SWSHID 4055
GET_SWSHNB 4056
GET_SWSHRM 4057
GET_SWSREF 4058
GET_TABLNO 4059
GET_TAPI 4060
GET_TAPNO 4061
GET_TAPR 4062
GET_TBRATE 4063
GET_THRSHZ 4064
GET_TIME 4065
GET_TOL 4066
GET_TPLOAD 4067
GET_TPNABL 4068
GET_TQLOAD 4069
GET_TRCONV 4070
GET_TRI 4071
GET_TRR 4072
GET_UEL_ACTIVE 4073
GET_VAR 4074
GET_VAR_DSCRPT 4075
GET_VCMODE 4076
GET_VDC 4077
GET_VDCI 4078
GET_VDCR 4079
GET_VFEOUT 4080
GET_VG2IDX 4081
GET_VMAX 4082
GET_VMIN 4083
GET_VOEL 4084
GET_VOLT 4085
GET_VOTHSG 4086
GET_VREF 4087
GET_VRFDBK 4088
GET_VSCACS 4089
GET_VSCCNV 4090
GET_VSCDCS 4091

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_VSCHED 4092
GET_VSCIDC 4093
GET_VSCIMX 4094
GET_VSCLSA 4095
GET_VSCLSB 4096
GET_VSCLSM 4097
GET_VSCMDC 4098
GET_VSCMOD 4099
GET_VSCNAM 4100
GET_VSCPAC 4101
GET_VSCPSV 4102
GET_VSCPWF 4103
GET_VSCQAC 4104
GET_VSCQMN 4105
GET_VSCQMX 4106
GET_VSCQSV 4107
GET_VSCRDC 4108
GET_VSCREM 4109
GET_VSCSIG 4110
GET_VSCSMX 4111
GET_VSCTYP 4112
GET_VSHNHI 4113
GET_VSHNLO 4114
GET_VUEL 4115
GET_W0 4116
GET_WAEROT 4117
GET_WAUXSG 4118
GET_WEQCMD 4119
GET_WINDRP 4120
GET_WINDRS 4121
GET_WIPCMD 4122
GET_WIQCMD 4123
GET_WMCARYS 4124
GET_WNDNUM 4125
GET_WNDSP1 4126
GET_WNDSP10 4127
GET_WNDSP2 4128
GET_WNDSP3 4129
GET_WNDSP4 4130
GET_WNDSP5 4131
GET_WNDSP6 4132
GET_WNDSP7 4133
GET_WNDSP8 4134
GET_WNDSP9 4135
GET_WPCMND 4136
GET_WPITCH 4137
GET_WPORDR 4138
GET_WPRATE 4139
GET_WPREFPTQ 4140
GET_WPREFWGO 4141
GET_WQCMND 4142

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_WROTID 4143
GET_WROTIQ 4144
GET_WROTRI 4145
GET_WROTRV 4146
GET_WROTVD 4147
GET_WROTVQ 4148
GET_WSTRTIN 4149
GET_WTRBSP 4150
GET_WVLCTY 4151
GET_XADIFD 4152
GET_XCI 4153
GET_XCONV 4154
GET_XCR 4155
GET_XTRAN 4156
GET_ZERNUM 4157
GET_ZSORCE 4158
Dynamics model value setting routines 4159
SET_ANGLE 4159
SET_BSWICH 4160
SET_CDGNFR 4161
SET_CFDLFR 4162
SET_CHRICN 4163
SET_CLODFR 4164
SET_CON 4165
SET_CON_DSCRPT 4166
SET_CURNT 4167
SET_DC2SIG 4168
SET_DCMSIG 4169
SET_DSTATE 4170
SET_ECOMP 4171
SET_EFD 4172
SET_ETERM 4173
SET_FCTREF 4174
SET_FCTSIG 4175
SET_GB 4176
SET_GBLINE 4177
SET_GREF 4178
SET_ICON 4179
SET_ICON_DSCRPT 4180
SET_IMARYS 4181
SET_IMCCNX 4182
SET_IMELTRQ 4183
SET_IMLDTRQ 4184
SET_IMSPARE 4185
SET_IMTPLOD 4186
SET_IMTQLOD 4187
SET_INTICN 4188
SET_ISORCE 4189
SET_KPAUSE 4190
SET_KTRIP 4191
SET_LCREF 4192

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SET_LDDGNFL 4193
SET_LDSTAT 4194
SET_MACBLKD 4195
SET_MCARYS 4196
SET_MCSTAT 4197
SET_MDC 4198
SET_NINTEG 4199
SET_NLWTRB 4200
SET_NUMLOD 4201
SET_NUMSWS 4202
SET_NUMTRM 4203
SET_OEL_ACTIVE 4204
SET_OLDERV 4205
SET_PELEC 4206
SET_PHSVLT 4207
SET_PMAXMCD 4208
SET_PMECH 4209
SET_PMINMCD 4210
SET_PSANGL 4211
SET_QELEC 4212
SET_SHNTGB 4213
SET_SHSTAT 4214
SET_SPEED 4215
SET_STATE 4216
SET_STATE_DSCRPT 4217
SET_STORE 4218
SET_SWSAUX 4219
SET_SWSCNX 4220
SET_SWSHNB 4221
SET_SWSREF 4222
SET_TBRATE 4223
SET_TPLOAD 4224
SET_TQLOAD 4225
SET_UEL_ACTIVE 4226
SET_VAR 4227
SET_VAR_DSCRPT 4228
SET_VDC 4229
SET_VFEOUT 4230
SET_VOEL 4231
SET_VOTHSG 4232
SET_VREF 4233
SET_VRFDBK 4234
SET_VSCMDC 4235
SET_VSCPAC 4236
SET_VSCQAC 4237
SET_VSCSIG 4238
SET_VSCTYP 4239
SET_VUEL 4240
SET_WAEROT 4241
SET_WAUXSG 4242
SET_WEQCMD 4243

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SET_WINDRP 4244
SET_WINDRS 4245
SET_WIPCMD 4246
SET_WIQCMD 4247
SET_WMCARYS 4248
SET_WNDSP1 4249
SET_WNDSP10 4250
SET_WNDSP2 4251
SET_WNDSP3 4252
SET_WNDSP4 4253
SET_WNDSP5 4254
SET_WNDSP6 4255
SET_WNDSP7 4256
SET_WNDSP8 4257
SET_WNDSP9 4258
SET_WPCMND 4259
SET_WPITCH 4260
SET_WPORDR 4261
SET_WPRATE 4262
SET_WPREFPTQ 4263
SET_WPREFWGO 4264
SET_WQCMND 4265
SET_WROTID 4266
SET_WROTIQ 4267
SET_WROTRI 4268
SET_WROTRV 4269
SET_WROTVD 4270
SET_WROTVQ 4271
SET_WTRBSP 4272
SET_WVLCTY 4273
SET_XADIFD 4274
Operational functions and calculations for dynamics models 4275
AUXOUT 4275
BLSTAT 4276
BSDSCN 4277
BSDSCN1 4278
BSSQNC 4279
BUS_ENCODE_TRIM 4280
CALCULATE_VFE 4281
CKTIDS 4282
CNSTCR 4283
CNSTPQ 4284
DEADBD2 4285
DEADBN2 4286
DELAYU 4287
DOCUCHK 4288
DOCUCHKBRANCH 4289
DOCUCHKICON 4290
DOCUCHKICONABS 4291
DOCUCHKICONMSG 4292
DOCUCHKINT 4293

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

DOCUCHKLOAD 4294
DOCUCHKMACHINE 4295
DOCUCHKMSG 4296
DOCUCHKPAIR 4297
DOCUCHKPARALLELBRANCH 4298
DOCUCHKREMBUS 4299
DOCUHEADING 4300
DYN_ADD_SHUNT 4301
DYN_REMOVE_SHUNT 4302
ENCODE_BUS_NAME 4303
EXCSAT 4304
EXC_FEX 4305
EXC_FEX_INI 4306
EXC_PW_SOURCE_2 4307
EXC_PW_SOURCE_A 4308
EXC_PW_SOURCE_B 4309
FACTDS_NOGINT 4310
FCTBLOCK 4311
FCTCNTL0 4312
FCTSTAT 4313
FLOW1M 4314
GEESAT 4315
GENCHK 4316
GENCHK2 4317
GENSAT 4318
GENTRP 4319
GETBRANCHSEQCURRENTS 4320
GETFTC 4321
GETFTC3 4322
GETMACHSEQCURRENTS 4323
GETPHASEVOLTAGES 4324
GETSEQUENCEVOLTAGES 4325
GET_EXCITER_VEMAX 4326
GTRSUB 4327
INT_MODE1 4328
INT_MODE2 4329
INT_MODE3 4330
LAG_MODE1 4331
LAG_MODE2 4332
LAG_MODE3 4333
LDLG_MODE1 4334
LDLG_MODE2 4335
LDLG_MODE3 4336
LDSHED 4337
LINESW 4338
LINRCL 4339
LINTRP 4340
LOADTYPE 4341
LODCHK 4342
LOGSUBSYSTEMLOADMODEL 4343
MODELID 4344

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MODESW 4345
NOTMID 4346
NUMBUS 4347
NWINT_MODE1 4348
NWINT_MODE2 4349
NWINT_MODE3 4350
NWLAG_MODE1 4351
NWLAG_MODE2 4352
NWLAG_MODE3 4353
NWLDLG_MODE1 4354
NWLDLG_MODE2 4355
NWLDLG_MODE3 4356
NWPIB_MODE1 4357
NWPIB_MODE2 4358
NWPIB_MODE3 4359
NWPID_MODE1 4360
NWPID_MODE2 4361
NWPID_MODE3 4362
NWPIPW_MODE1 4363
NWPIPW_MODE2 4364
NWPIPW_MODE3 4365
NWPI_MODE1 4366
NWPI_MODE2 4367
NWPI_MODE3 4368
ORD2_MODE1 4369
ORD2_MODE2 4370
ORD2_MODE3 4371
OUTPUTMODELMSGS 4372
PID_MODE1 4373
PID_MODE2 4374
PID_MODE3 4375
PI_MODE1 4376
PI_MODE2 4377
PI_MODE3 4378
PSILD 4379
PTOTAL 4380
PTOTAR 4381
PTOTOW 4382
PTOTZN 4383
RCL3WT 4384
SHOWMODEL 4385
SHOW_MODEL_INDICIES 4386
SHUNT 4387
SWCAP 4388
SWSTAT 4389
TRP3WT 4390
UEL2CUDM 4391
WSHOUT_MODE1 4392
WSHOUT_MODE2 4393
WSHOUT_MODE3 4394
PSSARRAYS Module 4396

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ACCC_SUMMARY 4397
ACCC_SOLUTION 4400
ACCC_SUMMARY_REPORT 4402
SENSITIVITY_FLOW_TO_MW 4403
ACCC_SOLUTION_REPORT 4404
ACCC_VIOLATIONS_REPORT 4405
ASCC_CURRENTS 4406
IECS_CURRENTS 4411
OTDF_FACTORS 4417
PV_SUMMARY 4419
PV_SUMMARY_REPORT 4421
PV_SOLUTION 4422
PV_SOLUTION_REPORT 4424
QV_SOLUTION 4425
QV_SOLUTION_REPORT 4426
QV_SUMMARY 4427
QV_SUMMARY_REPORT 4429
PSSEXCEL Module 4430
ACCC 4431
iec_data_file 4435
pv 4437
qv 4440
EXCELPY Module 4442
Methods 4446
Workbook Functions 4452
GICDATA Module 4453
PFCONTROL Module 4456
PFDSCNBUS 4457
PFCNCTBUS 4458
PFBRCHINT 4459
PFBRCHREAL 4460
PFSUBSWDINT 4461
PFFXSINT 4462
PFFXSREAL 4463
PFSWSINT 4464
PFSWSREAL 4465
PFSWSINT_2 4466
PFSWSREAL_2 4467
PFLODINT 4468
PFLODREAL 4469
PFMACINT 4470
PFMACREAL 4471
PFPLNTREAL 4472
PFCNTLVSC 4474
PSSACCSS Module 4476
ACCC_FILE_INIT 4477
ACCC_FILE_SIZE 4479
ACCC_FILE_GET_SUM_VOLT 4481
ACCC_FILE_CLOSE 4484
ACCC_FILE_SUMMARY 4485
ACCC_FILE_SUMMARY_2 4489

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ACCC_FILE_SUMMARY_3 4493
ACCC_FILE_SUMMARY_4 4498
ACCC_FILE_SUM_AREAS 4503
ACCC_FILE_CNTG_VOLT 4506
ACCC_FILE_CNTG_FLOW 4509
ACCC_FILE_CNTG_FLOW_2 4512
ACCC_FILE_CNTG_FLOW_3 4515
ACCC_FILE_CNTG_FLOW_4 4518
ACCC_FILE_CNTG_DSCR 4521
ACCC_FILE_CNTG_LOAD 4524
ACCC_FILE_SOLUTION 4527
ACCC_FILE_SOLUTION_2 4531
Other Python Modules 4535
Modules Supplied with a GUI 4536
Modules that Define Functions 4537
Index 4538

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Preface

Structure of Manual

This document defines the PSS®E Application Program Interface (API) to the various engineering functions in PSS®E.

How API Routines are Described

The description for each API routine starts with a brief explanation of its function, followed by one or more language specific syntaxes for calling the routine, followed by a detailed description of the arguments used in those calling sequences. The nomenclature used in the argument descriptions is most consistent with the Fortran programming language.

The syntaxes for calling the API routines will include at least one of the PSS®E automation file languages, and will usually include a syntax for use from a Fortran program. The set of syntaxes will include all the PSS®E automation file languages where the API routine can be used. Some general comments about those syntaxes may be helpful, as follows:

PSS®E Automation File Languages

Batch commands:

This is a free-format input syntax. Items can be separated by a comma or spaces, strings need only be quoted if they contain spaces or commas, arrays are entered as a sequence of scalar values, and, except for quoted strings or unquoted filenames, a forward slash will terminate a line. Commands may be continued over multiple lines. The sequence @! at the beginning of the line will cause that line to be ignored.

Handling of omitted values is discussed below under "Default Values". Omitted items can be indicated by consecutive commas. Also a semi-colon can be used to terminate the command and the remaining items are treated as omitted (it must be a separate data item, i.e., it must be separated from the previous data item by a space or comma).

Python commands:

Python is an open source language and its syntax is well documented. Although some arguments are defined as arrays (in keeping with Fortran nomenclature, as mentioned above), Python does not have arrays. In general you should interpret arrays to mean lists (although tuples will usually work on input). Arrays that have some elements as input and some as output in the Fortran syntax are represented in the Python syntax as two separate arrays (i.e. lists), an input array and an output array. Keywords can be used for all arguments. Any element of an input array can be specified with any individual keyword. E.g. myarray6=2 specifies that the 6th element of the array myarray has the value 2. If an alias is documented for a specific array element then it can also be used as a keyword for that element. Multiple line input and comments follow normal Python language rules.

Handling of omitted values is discussed below under "Default Values". Omitted items can be indicated using keywords to avoid specifying all arguments. Per Python syntax positional arguments must precede keyword arguments. It is also possible to incompletely specify an array; the remaining values will be treated as omitted. Additionally it is possible to explicitly specify an omitted value using the values described in "Default Values" below for recording omitted values in Python.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Fortran call statements:

Fortran is a well-documented language (where it matters, we will use the Fortran 2018 standard). We will not use features that require explicit interfaces without stating so clearly in the description.

Unless specifically stated, arguments cannot be omitted.

IPLAN commands:

The IPLAN language is described in the IPLAN manual. For purposes of the CALL statement it resembles Fortran.

Unless specifically stated, arguments cannot be omitted.

Return Codes / Error Codes

Most API routines return a integer return code, where zero indicates success. In Python, where multiple values are returned (a tuple) the return code will always be the first value. When non-successful (usually non-zero) error codes are returned, other returned values are set to None in Python, and are either uninitialized or unchanged (for input/output values) in other syntaxes. There are a small number of API routines that can return non-zero non-error return codes. This is indicated in the description of that code.

The Python interface can be made to throw exceptions instead of returning error code values (the default). Enter:

```
psspy.setThrowPsseExceptions(True)
```

or

```
psspy.param.throwPsseExceptions = True
```

To see if the Python interface is set to throw exceptions instead of returning error code values:

```
val = psspy.getThrowPsseExceptions()
```

Default Values

When values are omitted, special "no input" values are supplied. These special values - for each variable type - are meant to indicate that "no input" was supplied. Many of the API routines permit this and will use documented default values when "no input" values are supplied.

The actual "no input" values can be retrieved from the API routines GETDEFAULTINT, GETDEFAULTREAL, and GETDEFAULTCHAR (capitalization depends on syntax). Users should never depend on the actual values returned, by these functions. They should be used only for supplying values to the PSS®E API routines.

When automation files are recorded, items that have the default value are omitted in batch commands (i.e. recorded as ",,") and recorded as _i, _f, _s, or _o (for integer, real/float, character/string, and filename, respectively). In the PSS®E embedded Python environment there will be local variables _i, _f, and _s set to the values returned from GETDEFAULTxxx routines mentioned above (_o will be treated as blanks). Outside of PSS®E it is the user's responsibility to set those variables.

Recording

Two recording formats are available: batch commands and Python.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Not all API routines are recordable. Routines that only retrieve data do not record, for example. If an API routine is recordable, it will have a batch command format specified; routines without a batch command format specified are not recordable.

Python Modules

Most of the PSS®E API is defined in the module psspy. Later chapters discuss routines defined in other modules. In those cases the Chapter name will include the name of the module.

In keeping with normal Python documentation the namespace is not included as part of the syntax for the individual routines. This may seem confusing to newcomers to Python. For those whose first exposure to Python is inside of PSS®E, be aware that, normally, you will use the, say, case API routine by entering:

psspy.case(filename)

not

case(filename)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 1
Power Flow

This chapter describes the API routines corresponding to the operational and data changing functions of the PSS®E Power Flow.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1. Power Flow Operation

This section describes the API routines corresponding to the operational functions of the PSS®E Power Flow. Also included are those routines that control basic program operations and the Open Access option.

1.1.1. ACCC

This API routine is obsolete. It has been replaced by the API routine accc_with_dsp_2 .

Use this API to apply the AC contingency calculation function (activity ACCC).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ACCC  TOL  OPTIONS(1)..OPTIONS(7)  DFXFILE  ACCFILE  THRFILE
```

Python command syntax:

```
ierr = accc(tol, options, dfxfile, accfile, thrfile)
```

Fortran command syntax:

```
CALL ACCCAPI(TOL, OPTIONS, DFXFILE, ACCFILE, THRFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTIONS (7)	array of 7 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	solution method flag (0 by default)
	OPTIONS(6) = 0	FDNS
	OPTIONS(6) = 1	FNSL
	OPTIONS(6) = 2	optimized FDNS
	OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTIONS(7) = 0	disable
	OPTIONS(7) = 1	enable
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output file (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	no Distribution Factor Data File specified
	IERR = 7	no Contingency Solution Output File specified
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	bus type code and series element status inconsistencies
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.2. ACCC_2

This API routine is obsolete. It has been replaced by the API routine accc_with_dsp_3 .

Use this API to apply the AC contingency calculation function (activity ACCC).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ACCC_2 TOL OPTIONS(1)..OPTIONS(7) DFXFILE ACCFILE THRFILE
```

Python command syntax:

```
ierr = accc_2(tol, options, dfxfile, accfile, thrfile)
```

Fortran command syntax:

```
CALL ACCC_2(TOL, OPTIONS, DFXFILE, ACCFILE, THRFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTIONS (9)	array of 9 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(6)	solution method flag (0 by default)
	OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTIONS(8)	induction motor treatment flag (0 by default)
	OPTIONS(9)	induction machine failure flag (0 by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output file (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR > 0	as for ACCC_WITH_DSP_3.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.3. ACCC_MULTIPLE_MERGE

Use this API to merge a number of AC contingency solution files (acc files) into one acc file. The acc files must be based on the same base case and MON file.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_ACCC_MULTIPLE_MERGE  OPTION NFILES ACFILES(1)..ACFILES(NFILES) ACCFILE
```

Python command syntax:

```
ierr = accc_multiple_merge(option, nfiles, acfiles, accfile)
```

Fortran command syntax:

```
CALL ACCC_MULTIPLE_MERGE(OPTION, NFILES, ACFILES, ACCFILE, IERR)
```

Where:

Integer OPTION	Options for merging acc files. (input).	
	OPTION = 0	Do not merge .acc files if some .acc files have different solving options, sub files or .mon files.
	OPTION = 1	Merge the.acc files even if some .acc files have different solving options, sub files or .mon files.
Integer NFILES	The number of contingency acc files to be merged. (input).	
Character*260 ACFILES	An array of names of acc files. (input).	
Character*260 ACCFILE	The name of the merged contingency acc file. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occured
	IERR = 1	invalid OPTION value
	IERR = 2	invalid NFILES value
	IERR = 3	error opening a Contingency Solution Output File
	IERR = 4	file is not in the form of a current Contingency Solution Output File
	IERR = 5	some input .acc files have different information (.sav, .sub, .mon, PSSE version, solution options)
	IERR = 6	error reading or writing Contingency Solution Output file
	IERR = 7	error reading a Contingency Solution Output File
	IERR = 8	error closing a Contingency Solution Output file
	IERR = 9	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.4. ACCC_MULTIPLE_RUN_REPORT

This API routine is obsolete. It has been replaced by the API routine accc_multiple_run_report_2 .

Use this API to report the results of up to nine executions of the AC Contingency Calculation function.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ACCC_MULTIPLE_RUN_REPORT OPTIONS(1)..OPTIONS(14) VALUES(1)..VALUES(7)
 NFILES ACFILES(1)..ACFILES(NFILES)
```

Python command syntax:

```
ierr = accc_multiple_run_report(options, values, nfiles, acfiles)
```

Fortran command syntax:

```
CALL ACCC_MULTIPLE_RUN_REPORT(OPTIONS, VALUES, NFILES, ACFILES, IERR)
```

Where:

Integer OPTIONS (14)	array of 14 elements specifying defining reporting options(input).	
	OPTIONS(1)	column headings code (1 by default)
	OPTIONS(1) = 1	Contingency Solution Output File names
	OPTIONS(1) = 2	Saved Case File names as contained in the Contingency Solution Output Files
	OPTIONS(2)	base case rating set (rating set program option setting by default)
	OPTIONS(2) = 1	rate A
	OPTIONS(2) = 2	rate B
	OPTIONS(2) = 3	rate C
	OPTIONS(3)	contingency case rating set (rating set program option setting by default)
	OPTIONS(3) = 1	rate A
	OPTIONS(3) = 2	rate B
	OPTIONS(3) = 3	rate C
	OPTIONS(4)	print monitored elements summary report (0 by default)
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
	OPTIONS(5)	print missing monitored elements report (0 by default)
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
	OPTIONS(6)	print missing monitored voltage buses report (0 by default)
	OPTIONS(6) = 0	no
	OPTIONS(6) = 1	yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(7)	print contingency legend (2 by default)
OPTIONS(7) = 0	no
OPTIONS(7) = 1	complete master legend report only
OPTIONS(7) = 2	reduced legend with each table
OPTIONS(7) = 3	both master legend report and legend with each table
OPTIONS(8)	print missing contingencies report (0 by default)
OPTIONS(8) = 0	no
OPTIONS(8) = 1	yes
OPTIONS(9)	print non-converged contingencies report (0 by default)
OPTIONS(9) = 0	no
OPTIONS(9) = 1	yes
OPTIONS(10)	print loading violation reports (0 by default)
OPTIONS(10) = 0	no
OPTIONS(10) = 1	base case and worst case contingency violations only
OPTIONS(10) = 2	base case and all contingency violations only
OPTIONS(10) = 3	base case, worst case contingency, and all contingency violations
OPTIONS(11)	print voltage violation reports (0 by default)
OPTIONS(11) = 0	no
OPTIONS(11) = 1	base case and worst case contingency violations only
OPTIONS(11) = 2	base case and all contingency violations only
OPTIONS(11) = 3	base case, worst case contingency, and all contingency violations
OPTIONS(12)	option for processing interfaces in loading violation reports (1 by default)
OPTIONS(12) = 0	exclude interfaces from loading violation checking and reporting
OPTIONS(12) = 1	check and report interface loading violations
OPTIONS(13)	in loading violation reports, exclude monitored branches and interfaces that show loading violations in the base case from being checked and reported in contingency case reports (0 by default)
OPTIONS(13) = 0	no
OPTIONS(13) = 1	yes
OPTIONS(14)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports (0 by default)
OPTIONS(14) = 0	no

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(14) = 1	yes
Real VALUES (7)	array of 7 elements specifying reporting parameters(input).	
	VALUES(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	VALUES(2)	system mismatch converged tolerance (MVA) (5.0 by default)
	VALUES(3)	in loading violation reports, percent of flow rating for reporting (100.0 by default)
	VALUES(4)	in the worst case contingency loading violation reports, percent of flow rating for counting (> VALUES(3)) (100.0 by default)
	VALUES(5)	in loading violation reports, minimum contingency case flow change from base case value for reporting and counting (0.0 by default)
	VALUES(6)	in loading violation reports, minimum contingency case percent loading increase from base case value for reporting and counting (0.0 by default)
	VALUES(7)	minimum contingency case voltage change for reporting and counting in voltage range violations (0.0 by default)
Integer NFILES	number of Contingency Solution Output Files to be processed (1 through 22) (input; no default allowed).	
Character*260 ACFILES (NFILES)	NFILES names of Contingency Solution Output Files (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	invalid NFILES value
	IERR = 4	the percent for counting is less than the percent for reporting
	IERR = 5	file is not in the form of a current Contingency Solution Output File
	IERR = 6	error opening a Contingency Solution Output File
	IERR = 7	error reading a Contingency Solution Output File
	IERR = 8	error opening a temporary file
	IERR = 9	error opening a subfile in the temporary file
	IERR = 10	error reading or writing the temporary file
	IERR = 11	error closing a subfile in the temporary file
	IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.5. ACCC_MULTIPLE_RUN_REPORT_2

Use this API to report the results of up to twenty two executions of the AC Contingency Calculation function.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_ACCC_MULTIPLE_RUN_REPORT_2 OPTIONS(1)..OPTIONS(16)
 VALUES(1)..VALUES(7) NFILES ACFILES(1)..ACFILES(NFILES)
```

Python command syntax:

```
ierr = accc_multiple_run_report_2(options, values, nfiles, acfiles)
```

Fortran command syntax:

```
CALL ACCC_MULTIPLE_RUN_REPORT_2(OPTIONS, VALUES, NFILES, ACFILES, IERR)
```

Where:

Integer OPTIONS (16)	array of 16 elements specifying defining reporting options(input).	
	OPTIONS(1)	column headings code (1 by default)
	OPTIONS(1) = 1	Contingency Solution Output File names
	OPTIONS(1) = 2	Saved Case File names as contained in the Contingency Solution Output Files
	OPTIONS(2)	base case rating set (rating set program option setting by default)
	OPTIONS(2) = 1	rate A
	OPTIONS(2) = 2	rate B
	OPTIONS(2) = 3	rate C
	OPTIONS(3)	contingency case rating set (rating set program option setting by default)
	OPTIONS(3) = 1	rate A
	OPTIONS(3) = 2	rate B
	OPTIONS(3) = 3	rate C
	OPTIONS(4)	base case voltage limit when STATUS(1) is 0, 1, 3 or 4
	OPTIONS(4) = 1	normal limit
	OPTIONS(4) = 2	emergency limit
	OPTIONS(5)	contingency case voltage limit when STATUS(1) is 0, 1, 3 or 4
	OPTIONS(5) = 1	normal limit
	OPTIONS(5) = 2	emergency limit
	OPTIONS(6)	print monitored elements summary report (0 by default)
	OPTIONS(6) = 0	no
	OPTIONS(6) = 1	yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(7)	print missing monitored elements report (0 by default)
OPTIONS(7) = 0	no
OPTIONS(7) = 1	yes
OPTIONS(8)	print missing monitored voltage buses report (0 by default)
OPTIONS(8) = 0	no
OPTIONS(8) = 1	yes
OPTIONS(9)	print contingency legend (2 by default)
OPTIONS(9) = 0	no
OPTIONS(9) = 1	complete master legend report only
OPTIONS(9) = 2	reduced legend with each table
OPTIONS(9) = 3	both master legend report and legend with each table
OPTIONS(10)	print missing contingencies report (0 by default)
OPTIONS(10) = 0	no
OPTIONS(10) = 1	yes
OPTIONS(11)	print non-converged contingencies report (0 by default)
OPTIONS(11) = 0	no
OPTIONS(11) = 1	yes
OPTIONS(12)	print loading violation reports (0 by default)
OPTIONS(12) = 0	no
OPTIONS(12) = 1	base case and worst case contingency violations only
OPTIONS(12) = 2	base case and all contingency violations only
OPTIONS(12) = 3	base case, worst case contingency, and all contingency violations
OPTIONS(13)	print voltage violation reports (0 by default)
OPTIONS(13) = 0	no
OPTIONS(13) = 1	base case and worst case contingency violations only
OPTIONS(13) = 2	base case and all contingency violations only
OPTIONS(13) = 3	base case, worst case contingency, and all contingency violations
OPTIONS(14)	option for processing interfaces in loading violation reports (1 by default)
OPTIONS(14) = 0	exclude interfaces from loading violation checking and reporting
OPTIONS(14) = 1	check and report interface loading violations
OPTIONS(15)	in loading violation reports, exclude monitored branches and interfaces that show loading violations in the base case from be-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		ing checked and reported in contingency case reports (0 by default)
	OPTIONS(15) = 0	no
	OPTIONS(15) = 1	yes
	OPTIONS(16)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports (0 by default)
	OPTIONS(16) = 0	no
	OPTIONS(16) = 1	yes
Real VALUES (7)	array of 7 elements specifying reporting parameters(input).	
	VALUES(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	VALUES(2)	system mismatch converged tolerance (MVA) (5.0 by default)
	VALUES(3)	in loading violation reports, percent of flow rating for reporting (100.0 by default)
	VALUES(4)	in the worst case contingency loading violation reports, percent of flow rating for counting (> VALUES(3)) (100.0 by default)
	VALUES(5)	in loading violation reports, minimum contingency case flow change from base case value for reporting and counting (0.0 by default)
	VALUES(6)	in loading violation reports, minimum contingency case percent loading increase from base case value for reporting and counting (0.0 by default)
	VALUES(7)	minimum contingency case voltage change for reporting and counting in voltage range violations (0.0 by default)
Integer NFILES	number of Contingency Solution Output Files to be processed (1 through 9) (input; no default allowed).	
Character*260 ACFILES (NFILES)	NFILES names of Contingency Solution Output Files (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	invalid NFILES value
	IERR = 4	the percent for counting is less than the percent for reporting
	IERR = 5	file is not in the form of a current Contingency Solution Output File

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	error opening a Contingency Solution Output File
IERR = 7	error reading a Contingency Solution Output File
IERR = 8	error opening a temporary file
IERR = 9	error opening a subfile in the temporary file
IERR = 10	error reading or writing the temporary file
IERR = 11	error closing a subfile in the temporary file
IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.6. ACCC_PARALLEL

This API routine is obsolete. It has been replaced by the API routine accc_parallel_2 .

Use this API routine to run the initial release of the parallel implementation of the AC contingency calculation function (use ACCC_WITH_DSP_3 to run AC contingency calculation function serially). The API routine distributes contingency cases among multiple processors evenly, and tests the contingency cases assigned to each processor in parallel.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_ACCC_PARALLEL  TOL OPTACC(1)..OPTACC(10) LABEL DFXFILE ACCFILE THRFILE
 INLFILE
```

Python command syntax:

```
ierr = accc_parallel(tol, optacc, label, dfxfile, accfile, thrfile,
 inlfile)
```

Fortran command syntax:

```
CALL ACCC_PARALLEL(TOL, OPTACC, LABEL, DFXFILE, ACCFILE, THRFILE, INLFILE,
 IERR)
```

Where:

Real TOL	Is the mismatch tolerance (input; Newton solution convergence tolerance, TOLN, by default).	
Integer OPTACC (10)	array of elements specifying solution options. The values are as follows:(input).	
	OPTACC(1)	tap adjustment flag. (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag. (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag. (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag. (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTACC(4) = 0	disable
OPTACC(4) = 1	enable
OPTACC(5)	switched shunt adjustment flag. (switched shunt adjustment option setting by default)
OPTACC(5) = 0	disable
OPTACC(5) = 1	enable
OPTACC(5) = 2	enable continuous mode, disable discrete mode
OPTACC(6)	solution method flag. (0 by default)
OPTACC(6) = 0	FDNS
OPTACC(6) = 1	FNSL
OPTACC(6) = 2	optimized FDNS
OPTACC(7)	non-divergent solution flag. (non-divergent solution option setting by default)
OPTACC(7) = 0	disable
OPTACC(7) = 1	enable
OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage) (0 by default)
OPTACC(8) = 0	stall
OPTACC(8) = 1	trip
OPTACC(9)	induction machine failure flag (0 by default)
OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the stalled or tripped state
OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the stalled or tripped state.
OPTACC(10)	dispatch mode (0 by default)
OPTACC(10) = 0	disable
OPTACC(10) = 1	subsystem machines (reserve)
OPTACC(10) = 2	subsystem machines (pmax)
OPTACC(10) = 3	subsystem machines (inertia)
OPTACC(10) = 4	subsystem machines (governor droop)

Character*32 LABEL	Is the name of the generation dispatch subsystem (blank by default, no default allowed if OPTIONS(10) is not 0). (input).
Character*260 DFXFILE	Is the name of the Distribution Factor Data file. (input; no default allowed).
Character*260 ACCFILE	Is the name of the Contingency Solution Output file. (input; no default allowed).
Character*260 THRFILE	Is the name of the Load Throwover Data file. (input; blank by default).
Character*260 INLFILE	Is the name of the Unit Inertia and Governor Data File. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	no Distribution Factor Data File specified
	IERR = 9	no Contingency Solution Output File specified
	IERR = 10	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 11	buses with bus type code and series element status inconsistencies
	IERR = 12	no ZIP Archive Output File specified
	IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	error opening Unit Inertia and Governor Data File
	IERR = 55	error opening ZIP Archive Output File
	IERR = 56	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.7. ACCC_PARALLEL_2

Use this API routine to run the second release of the parallel implementation of the AC contingency calculation function (use ACCC_WITH_DSP_3 to run AC contingency calculation function serially). The API routine distributes contingency cases among multiple processors evenly, and tests the contingency cases assigned to each processor in parallel.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_ACCC_PARALLEL_2  TOL OPTACC(1)..OPTACC(11) LABEL DFXFILE ACCFILE
 THRFILE INLFILE ZIPFILE
```

Python command syntax:

```
ierr = accc_parallel_2(tol, optacc, label, dfxfile, accfile, thrfile,
 inlfile, zipfile)
```

Fortran command syntax:

```
CALL ACCC_PARALLEL_2(TOL, OPTACC, LABEL, DFXFILE, ACCFILE, THRFILE,
 INLFILE, ZIPFILE, IERR)
```

Where:

Real TOL	Is the mismatch tolerance (input; Newton solution convergence tolerance, TOLN, by default).	
Integer OPTACC (11)	array of elements specifying solution options. The values are as follows:(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
OPTACC(5) = 0	disable
OPTACC(5) = 1	enable
OPTACC(5) = 2	enable continuous mode, disable discrete mode
OPTACC(6)	solution method flag (0 by default)
OPTACC(6) = 0	FDNS
OPTACC(6) = 1	FNSL
OPTACC(6) = 2	optimized FDNS
OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTACC(7) = 0	disable
OPTACC(7) = 1	enable
OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage) (0 by default)
OPTACC(8) = 0	stall
OPTACC(8) = 1	trip
OPTACC(9)	induction machine failure flag (0 by default)
OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(10)	dispatch mode (0 by default)
OPTACC(10) = 0	disable
OPTACC(10) = 1	subsystem machines (reserve)
OPTACC(10) = 2	subsystem machines (pmax)
OPTACC(10) = 3	subsystem machines (inertia)
OPTACC(10) = 4	subsystem machines (governor droop)
OPTACC(11)	ZIP archive flag (0 by default)
OPTACC(11) = 0	do not write a ZIP archive file
OPTACC(11) = 1	write a ZIP archive using the file specified as ZIPFILE

Character*32 LABEL	Is the name of the generation dispatch subsystem (blank by default, no default allowed if OPTIONS(10) is not 0) (input).
Character*260 DFXFILE	Is the name of the Distribution Factor Data file (input; no default allowed).
Character*260 ACCFILE	Is the name of the Contingency Solution Output file (input; no default allowed).
Character*260 THRFILE	Is the name of the Load Throwover Data file (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 INLFILE	Is the name of the Unit Inertia and Governor Data File (input; blank by default).
Character*260 ZIPFILE	Is the name of the ZIP Archive Output File (input; blank by default).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTIONS value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation dispatch subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	no Distribution Factor Data File specified
IERR = 9	no Contingency Solution Output File specified
IERR = 10	in-service induction machines are in the "stalled" or "tripped" state
IERR = 11	buses with bus type code and series element status inconsistencies
IERR = 12	no ZIP Archive Output File specified
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	error opening Unit Inertia and Governor Data File
IERR = 55	error opening ZIP Archive Output File
IERR = 56	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.8. ACCC_SINGLE_RUN_REPORT

This API routine is obsolete. It has been replaced by the API routine accc_single_run_report_2 .

Use this API, the AC Contingency Report function, to report the results of the AC Contingency Calculation function.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ACCC_SINGLE_RUN_REPORT STATUS(1)..STATUS(8) INTVAL(1)..INTVAL(5)
 REALVAL(1)..REALVAL(7) RFILE
```

Python command syntax:

```
ierr = accc_single_run_report(status, intval, realval, rfile)
```

Fortran command syntax:

```
CALL ACCC_SINGLE_RUN_REPORT(STATUS, INTVAL, REALVAL, RFILE, IERR)
```

Where:

Integer STATUS (8)	array of 8 elements specifying (input).	
	STATUS(1)	report format (3 by default)
	STATUS(1) = 0	spreadsheet overload report
	STATUS(1) = 1	spreadsheet loading table
	STATUS(1) = 2	available capacity table
	STATUS(1) = 3	non-spreadsheet overload report
	STATUS(1) = 4	non-spreadsheet loading table
	STATUS(1) = 5	non-converged networks report
	STATUS(2)	base case rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
	STATUS(2) = 1	rate A
	STATUS(2) = 2	rate B
	STATUS(2) = 3	rate C
	STATUS(3)	contingency case rating set when STATUS(1) is 0, 1, 3 or 4; base case and contingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
	STATUS(3) = 1	rate A
	STATUS(3) = 2	rate B
	STATUS(3) = 3	rate C
	STATUS(4)	exclude interfaces from report; used only when STATUS(1) is 0, 1, 2, 3 or 4 (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5)	run voltage limit check; used only when STATUS(1) is 0, 1, 3 or 4 (0 by default)
	STATUS(5) = 0	no
	STATUS(5) = 1	yes
	STATUS(6)	in overload reports, exclude monitored branches and interfaces that show loading violations in the base case from being checked and reported in contingency cases; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(6) = 0	no
	STATUS(6) = 1	yes
	STATUS(7)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(7) = 0	no
	STATUS(7) = 1	yes
	STATUS(8)	exclude cases with no overloads from non-spreadsheet overload report; used only when STATUS(1) is 3 (0 by default)
	STATUS(8) = 0	no
	STATUS(8) = 1	yes
Integer INTVAL (5)	array of 5 elements specifying (input).	
	INTVAL(1)	number of low voltage range violations filtering criterion (0 by default)
	INTVAL(2)	number of high voltage range violations filtering criterion (0 by default)
	INTVAL(3)	number of voltage deviation violations filtering criterion; not applied to base case (0 by default)
	INTVAL(4)	number of buses in the largest disconnected island filtering criterion; not applied to base case (0 by default)
	INTVAL(5)	maximum number of elements in the available capacity table (no limit by default)
Real REALVAL (7)	array of 7 elements specifying (input).	
	REALVAL(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	REALVAL(2)	system mismatch converged tolerance (MVA) (5.0 by default)
	REALVAL(3)	percent of flow rating; used only when STATUS(1) is 0, 3 or 4 (100.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALVAL(4)	in overload reports, minimum contingency case flow change from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(5)	in overload reports, minimum contingency case percent loading increase from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(6)	in voltage range violation reports, minimum contingency case voltage change from base case value; used only when STATUS(1) is 0, 1, 3 or 4 (0.0 by default)
	REALVAL(7)	cutoff threshold for available capacity table; used only when STATUS(1) is 2 (99999.0 by default)
Character*260 RFILE	Contingency Solution Output File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	invalid INTVAL value
	IERR = 3	invalid REALVAL value
	IERR = 4	error opening RFILE
	IERR = 5	error reading RFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.9. ACCC_SINGLE_RUN_REPORT_2

This API routine is obsolete. It has been replaced by the API routine accc_single_run_report_3 .

This API is the second release of the AC Contingency Report function. Use this API to report the results of the AC Contingency Calculation function.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_ACCC_SINGLE_RUN_REPORT_2 STATUS(1)..STATUS(9) INTVAL(1)..INTVAL(5)
 REALVAL(1)..REALVAL(7) RFILE
```

Python command syntax:

```
ierr = accc_single_run_report_2(status, intval, realval, rfile)
```

Fortran command syntax:

```
CALL ACCC_SINGLE_RUN_REPORT_2(STATUS, INTVAL, REALVAL, RFILE, IERR)
```

Where:

Integer STATUS (9)	array of 9 elements specifying (input).	
	STATUS(1)	report format (3 by default)
	STATUS(1) = 0	spreadsheet overload report
	STATUS(1) = 1	spreadsheet loading table
	STATUS(1) = 2	available capacity table
	STATUS(1) = 3	non-spreadsheet overload report
	STATUS(1) = 4	non-spreadsheet loading table
	STATUS(1) = 5	non-converged networks report
	STATUS(1) = 6	non-spreadsheet corrective actions report
	STATUS(2)	base case rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
	STATUS(2) = 1	rate A
	STATUS(2) = 2	rate B
	STATUS(2) = 3	rate C
	STATUS(3)	contingency case rating set when STATUS(1) is 0, 1, 3 or 4; base case and contingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
	STATUS(3) = 1	rate A
	STATUS(3) = 2	rate B
	STATUS(3) = 3	rate C
	STATUS(4)	exclude interfaces from report; used only when STATUS(1) is 0, 1, 2, 3, 4 (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5)	run voltage limit check; used only when STATUS(1) is 0, 1, 3, 4 (0 by default)
	STATUS(5) = 0	no
	STATUS(5) = 1	yes
	STATUS(6)	in overload reports, exclude monitored branches and interfaces that show loading violations in the base case from being checked and reported in contingency cases; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(6) = 0	no
	STATUS(6) = 1	yes
	STATUS(7)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(7) = 0	no
	STATUS(7) = 1	yes
	STATUS(8)	exclude cases with no overloads from non-spreadsheet overload report or non-spreadsheet corrective actions report; used only when STATUS(1) is 3 (0 by default)
	STATUS(8) = 0	no
	STATUS(8) = 1	yes
	STATUS(9)	report post-tripping action solutions (0 by default)
	STATUS(9) = 0	no
	STATUS(9) = 1	yes
Integer INTVAL (5)	array of 5 elements specifying (input).	
	INTVAL(1)	number of low voltage range violations filtering criterion (0 by default)
	INTVAL(2)	number of high voltage range violations filtering criterion (0 by default)
	INTVAL(3)	number of voltage deviation violations filtering criterion; not applied to base case (0 by default)
	INTVAL(4)	number of buses in the largest disconnected island filtering criterion; not applied to base case (0 by default)
	INTVAL(5)	maximum number of elements in the available capacity table (no limit by default)
Real REALVAL (7)	array of 7 elements specifying (input).	
	REALVAL(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALVAL(2)	system mismatch converged tolerance (MVA) (5.0 by default)
	REALVAL(3)	percent of flow rating; used only when STATUS(1) is 0, 3 or 4 (100.0 by default)
	REALVAL(4)	in overload reports, minimum contingency case flow change from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(5)	in overload reports, minimum contingency case percent loading increase from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(6)	in voltage range violation reports, minimum contingency case voltage change from base case value; used only when STATUS(1) is 0, 1, 3 or 4 (0.0 by default)
	REALVAL(7)	cutoff threshold for available capacity table; used only when STATUS(1) is 2 (99999.0 by default)
Character*260 RFILE	Contingency Solution Output File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	invalid INTVAL value
	IERR = 3	invalid REALVAL value
	IERR = 4	error opening RFILE
	IERR = 5	error reading RFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.10. ACCC_SINGLE_RUN_REPORT_3

This API routine is obsolete. It has been replaced by the API routine accc_single_run_report_4 .

This API is the third release of the AC Contingency Report function. Use this API to report the results of the AC Contingency Calculation function.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ACCC_SINGLE_RUN_REPORT_3 STATUS(1)..STATUS(10) INTVAL(1)..INTVAL(5)
 REALVAL(1)..REALVAL(7) RFILE
```

Python command syntax:

```
ierr = accc_single_run_report_3(status, intval, realval, rfile)
```

Fortran command syntax:

```
CALL ACCC_SINGLE_RUN_REPORT_3(STATUS, INTVAL, REALVAL, RFILE, IERR)
```

Where:

Integer STATUS (10)	array of 10 elements specifying (input).	
	STATUS(1)	report format (3 by default)
	STATUS(1) = 0	spreadsheet overload report
	STATUS(1) = 1	spreadsheet loading table
	STATUS(1) = 2	available capacity table
	STATUS(1) = 3	non-spreadsheet overload report
	STATUS(1) = 4	non-spreadsheet loading table
	STATUS(1) = 5	non-converged networks report
	STATUS(1) = 6	non-spreadsheet corrective actions report
	STATUS(2)	base case rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
	STATUS(2) = 1	rate A
	STATUS(2) = 2	rate B
	STATUS(2) = 3	rate C
	STATUS(3)	contingency case rating set when STATUS(1) is 0, 1, 3 or 4; base case and contingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
	STATUS(3) = 1	rate A
	STATUS(3) = 2	rate B
	STATUS(3) = 3	rate C
	STATUS(4)	exclude interfaces from report; used only when STATUS(1) is 0, 1, 2, 3, 4 (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5)	run voltage limit check; used only when STATUS(1) is 0, 1, 3, 4 (0 by default)
	STATUS(5) = 0	no
	STATUS(5) = 1	yes
	STATUS(6)	in overload reports, exclude monitored branches and interfaces that show loading violations in the base case from being checked and reported in contingency cases; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(6) = 0	no
	STATUS(6) = 1	yes
	STATUS(7)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(7) = 0	no
	STATUS(7) = 1	yes
	STATUS(8)	exclude cases with no overloads from non-spreadsheet overload report or non-spreadsheet corrective actions report; used only when STATUS(1) is 3 (0 by default)
	STATUS(8) = 0	no
	STATUS(8) = 1	yes
	STATUS(9)	report post-tripping action solutions (0 by default)
	STATUS(9) = 0	no
	STATUS(9) = 1	yes
	STATUS(10)	report loss of loads (0 by default)
	STATUS(10) = 0	no
	STATUS(10) = 1	yes
Integer INTVAL (5)	array of 5 elements specifying (input).	
	INTVAL(1)	number of low voltage range violations filtering criterion (0 by default)
	INTVAL(2)	number of high voltage range violations filtering criterion (0 by default)
	INTVAL(3)	number of voltage deviation violations filtering criterion; not applied to base case (0 by default)
	INTVAL(4)	number of buses in the largest disconnected island filtering criterion; not applied to base case (0 by default)
	INTVAL(5)	maximum number of elements in the available capacity table (no limit by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real REALVAL (7)	array of 7 elements specifying (input).	
	REALVAL(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	REALVAL(2)	system mismatch converged tolerance (MVA) (5.0 by default)
	REALVAL(3)	percent of flow rating; used only when STATUS(1) is 0, 3 or 4 (100.0 by default)
	REALVAL(4)	in overload reports, minimum contingency case flow change from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(5)	in overload reports, minimum contingency case percent loading increase from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(6)	in voltage range violation reports, minimum contingency case voltage change from base case value; used only when STATUS(1) is 0, 1, 3 or 4 (0.0 by default)
	REALVAL(7)	cutoff threshold for available capacity table; used only when STATUS(1) is 2 (99999.0 by default)
Character*260 RFILE	Contingency Solution Output File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	invalid INTVAL value
	IERR = 3	invalid REALVAL value
	IERR = 4	error opening RFILE
	IERR = 5	error reading RFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.11. ACCC_SINGLE_RUN_REPORT_4

This API routine is obsolete. It has been replaced by the API routine accc_single_run_report_5 .

This API is the fourth release of the AC Contingency Report function. Use this API to report the results of the AC Contingency Calculation function.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_ACCC_SINGLE_RUN_REPORT_4 STATUS(1)..STATUS(12) INTVAL(1)..INTVAL(5)
 REALVAL(1)..REALVAL(7) RFILE
```

Python command syntax:

```
ierr = accc_single_run_report_4(status, intval, realval, rfile)
```

Fortran command syntax:

```
CALL ACCC_SINGLE_RUN_REPORT_4(STATUS, INTVAL, REALVAL, RFILE, IERR)
```

Where:

Integer STATUS (12)	array of 12 elements specifying (input).	
	STATUS(1)	report format (3 by default)
	STATUS(1) = 0	spreadsheet overload report
	STATUS(1) = 1	spreadsheet loading table
	STATUS(1) = 2	available capacity table
	STATUS(1) = 3	non-spreadsheet overload report
	STATUS(1) = 4	non-spreadsheet loading table
	STATUS(1) = 5	non-converged networks report
	STATUS(1) = 6	non-spreadsheet corrective actions report
	STATUS(2)	base case rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
	STATUS(2) = 1	rate A
	STATUS(2) = 2	rate B
	STATUS(2) = 3	rate C
	STATUS(3)	contingency case rating set when STATUS(1) is 0, 1, 3 or 4; base case and contingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
	STATUS(3) = 1	rate A
	STATUS(3) = 2	rate B
	STATUS(3) = 3	rate C
	STATUS(4)	base case voltage limit when STATUS(1) is 0, 1, 3 or 4

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(4) = 1	normal limit
	STATUS(4) = 2	emergency limit
	STATUS(5)	contingency case voltage limit when STATUS(1) is 0, 1, 3 or 4
	STATUS(5) = 1	normal limit
	STATUS(5) = 2	emergency limit
	STATUS(6)	exclude interfaces from report; used only when STATUS(1) is 0, 1, 2, 3, 4 (0 by default)
	STATUS(6) = 0	no
	STATUS(6) = 1	yes
	STATUS(7)	run voltage limit check; used only when STATUS(1) is 0, 1, 3, 4 (0 by default)
	STATUS(7) = 0	no
	STATUS(7) = 1	yes
	STATUS(8)	in overload reports, exclude monitored branches and interfaces that show loading violations in the base case from being checked and reported in contingency cases; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(8) = 0	no
	STATUS(8) = 1	yes
	STATUS(9)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(9) = 0	no
	STATUS(9) = 1	yes
	STATUS(10)	exclude cases with no overloads from non-spreadsheet overload report or non-spreadsheet corrective actions report; used only when STATUS(1) is 3 (0 by default)
	STATUS(10) = 0	no
	STATUS(10) = 1	yes
	STATUS(11)	report post-tripping action solutions (0 by default)
	STATUS(11) = 0	no
	STATUS(11) = 1	yes
	STATUS(12)	report loss of loads (0 by default)
	STATUS(12) = 0	no
	STATUS(12) = 1	yes
Integer INTVAL (5)	array of 5 elements specifying (input).	
	INTVAL(1)	number of low voltage range violations filtering criterion (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTVAL(2)	number of high voltage range violations filtering criterion (0 by default)
	INTVAL(3)	number of voltage deviation violations filtering criterion; not applied to base case (0 by default)
	INTVAL(4)	number of buses in the largest disconnected island filtering criterion; not applied to base case (0 by default)
	INTVAL(5)	maximum number of elements in the available capacity table (no limit by default)
Real REALVAL (7)	array of 7 elements specifying (input).	
	REALVAL(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	REALVAL(2)	system mismatch converged tolerance (MVA) (5.0 by default)
	REALVAL(3)	percent of flow rating; used only when STATUS(1) is 0, 3 or 4 (100.0 by default)
	REALVAL(4)	in overload reports, minimum contingency case flow change from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(5)	in overload reports, minimum contingency case percent loading increase from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(6)	in voltage range violation reports, minimum contingency case voltage change from base case value; used only when STATUS(1) is 0, 1, 3 or 4 (0.0 by default)
	REALVAL(7)	cutoff threshold for available capacity table; used only when STATUS(1) is 2 (99999.0 by default)
Character*260 RFILE	Contingency Solution Output File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	invalid INTVAL value
	IERR = 3	invalid REALVAL value
	IERR = 4	error opening RFILE
	IERR = 5	error reading RFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.12. ACCC_SINGLE_RUN_REPORT_5

This API routine is obsolete. It has been replaced by the API routine accc_single_run_report_6 .

This API is the fourth release of the AC Contingency Report function. Use this API to report the results of the AC Contingency Calculation function.

This API routine was first introduced in release 34.5.0.

Batch command syntax:

```
BAT_ACCC_SINGLE_RUN_REPORT_5 STATUS(1)..STATUS(14) INTVAL(1)..INTVAL(5)
 REALVAL(1)..REALVAL(7) RFILE
```

Python command syntax:

```
ierr = accc_single_run_report_5(status, intval, realval, rfile)
```

Fortran command syntax:

```
CALL ACCC_SINGLE_RUN_REPORT_5(STATUS, INTVAL, REALVAL, RFILE, IERR)
```

Where:

Integer STATUS (14)	array of 14 elements specifying (input).	
	STATUS(1)	report format (3 by default)
	STATUS(1) = 0	spreadsheet overload report
	STATUS(1) = 1	spreadsheet loading table
	STATUS(1) = 2	available capacity table
	STATUS(1) = 3	non-spreadsheet overload report
	STATUS(1) = 4	non-spreadsheet loading table
	STATUS(1) = 5	non-converged networks report
	STATUS(1) = 6	non-spreadsheet corrective actions report
	STATUS(2)	base case branch rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
	STATUS(2) = 1	rate 1
	STATUS(2) = 2	rate 2
	STATUS(2) = 3	rate 3
	STATUS(2) = 4	rate 4
	STATUS(2) = 5	rate 5
	STATUS(2) = 6	rate 6
	STATUS(2) = 7	rate 7
	STATUS(2) = 8	rate 8
	STATUS(2) = 9	rate 9
	STATUS(2) = 10	rate 10
	STATUS(2) = 11	rate 11
	STATUS(2) = 12	rate 12
	STATUS(3)	contingency case branch rating set when STATUS(1) is 0, 1, 3 or 4; base case and con-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	tingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
STATUS(3) = 1	rate 1
STATUS(3) = 2	rate 2
STATUS(3) = 3	rate 3
STATUS(3) = 4	rate 4
STATUS(3) = 5	rate 5
STATUS(3) = 6	rate 6
STATUS(3) = 7	rate 7
STATUS(3) = 8	rate 8
STATUS(3) = 9	rate 9
STATUS(3) = 10	rate 10
STATUS(3) = 11	rate 11
STATUS(3) = 12	rate 12
STATUS(4)	base case SWD rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
STATUS(4) = 1	rate 1
STATUS(4) = 2	rate 2
STATUS(4) = 3	rate 3
STATUS(5)	contingency case SWD rating set when STATUS(1) is 0, 1, 3 or 4; base case and contingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
STATUS(5) = 1	rate 1
STATUS(5) = 2	rate 2
STATUS(5) = 3	rate 3
STATUS(6)	base case voltage limit when STATUS(1) is 0, 1, 3 or 4
STATUS(6) = 1	normal limit
STATUS(6) = 2	emergency limit
STATUS(7)	contingency case voltage limit when STATUS(1) is 0, 1, 3 or 4
STATUS(7) = 1	normal limit
STATUS(7) = 2	emergency limit
STATUS(8)	exclude interfaces from report; used only when STATUS(1) is 0, 1, 2, 3, 4 (0 by default)
STATUS(8) = 0	no
STATUS(8) = 1	yes
STATUS(9)	run voltage limit check; used only when STATUS(1) is 0, 1, 3, 4 (0 by default)
STATUS(9) = 0	no
STATUS(9) = 1	yes
STATUS(10)	in overload reports, exclude monitored branches and interfaces that show load-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		ing violations in the base case from being checked and reported in contingency cases; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(10) = 0	no
	STATUS(10) = 1	yes
	STATUS(11)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(11) = 0	no
	STATUS(11) = 1	yes
	STATUS(12)	exclude cases with no overloads from non-spreadsheet overload report or non-spreadsheet corrective actions report; used only when STATUS(1) is 3 (0 by default)
	STATUS(12) = 0	no
	STATUS(12) = 1	yes
	STATUS(13)	report post-tripping action solutions (0 by default)
	STATUS(13) = 0	no
	STATUS(13) = 1	yes
	STATUS(14)	report loss of loads (0 by default)
	STATUS(14) = 0	no
	STATUS(14) = 1	yes
Integer INTVAL (5)	array of 5 elements specifying (input).	
	INTVAL(1)	number of low voltage range violations filtering criterion (0 by default)
	INTVAL(2)	number of high voltage range violations filtering criterion (0 by default)
	INTVAL(3)	number of voltage deviation violations filtering criterion; not applied to base case (0 by default)
	INTVAL(4)	number of buses in the largest disconnected island filtering criterion; not applied to base case (0 by default)
	INTVAL(5)	maximum number of elements in the available capacity table (no limit by default)
Real REALVAL (7)	array of 7 elements specifying (input).	
	REALVAL(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	REALVAL(2)	system mismatch converged tolerance (MVA) (5.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALVAL(3)	percent of flow rating; used only when STATUS(1) is 0, 3 or 4 (100.0 by default)
	REALVAL(4)	in overload reports, minimum contingency case flow change from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(5)	in overload reports, minimum contingency case percent loading increase from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(6)	in voltage range violation reports, minimum contingency case voltage change from base case value; used only when STATUS(1) is 0, 1, 3 or 4 (0.0 by default)
	REALVAL(7)	cutoff threshold for available capacity table; used only when STATUS(1) is 2 (99999.0 by default)
Character*260 RFILE	Contingency Solution Output File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	invalid INTVAL value
	IERR = 3	invalid REALVAL value
	IERR = 4	error opening RFILE
	IERR = 5	error reading RFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.13. ACCC_SINGLE_RUN_REPORT_6

This API is the sixth release of the AC Contingency Report function. Use this API to report the results of the AC Contingency Calculation function.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ACCC_SINGLE_RUN_REPORT_6 STATUS(1)..STATUS(15) INTVAL(1)..INTVAL(5)
 REALVAL(1)..REALVAL(8) CHAVAL RFILE
```

Python command syntax:

```
ierr = accc_single_run_report_6(status, intval, realval, chaval, rfile)
```

Fortran command syntax:

```
CALL ACCC_SINGLE_RUN_REPORT_6(STATUS, INTVAL, REALVAL, CHAVAL, RFILE,
 IERR)
```

Where:

Integer STATUS (15)	array of 15 elements specifying (input).	
	STATUS(1)	report format (3 by default)
	STATUS(1) = 0	spreadsheet overload report
	STATUS(1) = 1	spreadsheet loading table
	STATUS(1) = 2	available capacity table
	STATUS(1) = 3	non-spreadsheet overload report
	STATUS(1) = 4	non-spreadsheet loading table
	STATUS(1) = 5	non-converged networks report
	STATUS(1) = 6	non-spreadsheet corrective actions report
	STATUS(2)	base case branch rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
	STATUS(2) = 1	rate 1
	STATUS(2) = 2	rate 2
	STATUS(2) = 3	rate 3
	STATUS(2) = 4	rate 4
	STATUS(2) = 5	rate 5
	STATUS(2) = 6	rate 6
	STATUS(2) = 7	rate 7
	STATUS(2) = 8	rate 8
	STATUS(2) = 9	rate 9
	STATUS(2) = 10	rate 10
	STATUS(2) = 11	rate 11
	STATUS(2) = 12	rate 12
	STATUS(3)	contingency case branch rating set when STATUS(1) is 0, 1, 3 or 4; base case and con-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	tingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
STATUS(3) = 1	rate 1
STATUS(3) = 2	rate 2
STATUS(3) = 3	rate 3
STATUS(3) = 4	rate 4
STATUS(3) = 5	rate 5
STATUS(3) = 6	rate 6
STATUS(3) = 7	rate 7
STATUS(3) = 8	rate 8
STATUS(3) = 9	rate 9
STATUS(3) = 10	rate 10
STATUS(3) = 11	rate 11
STATUS(3) = 12	rate 12
STATUS(4)	base case SWD rating set; used only when STATUS(1) is 0, 1, 3 or 4 (rating set program option setting by default)
STATUS(4) = 1	rate 1
STATUS(4) = 2	rate 2
STATUS(4) = 3	rate 3
STATUS(5)	contingency case SWD rating set when STATUS(1) is 0, 1, 3 or 4; base case and contingency case rating set when STATUS(1) is 2 (rating set program option setting by default)
STATUS(5) = 1	rate 1
STATUS(5) = 2	rate 2
STATUS(5) = 3	rate 3
STATUS(6)	base case voltage limit when STATUS(1) is 0, 1, 3 or 4
STATUS(6) = 1	normal limit
STATUS(6) = 2	emergency limit
STATUS(7)	contingency case voltage limit when STATUS(1) is 0, 1, 3 or 4
STATUS(7) = 1	normal limit
STATUS(7) = 2	emergency limit
STATUS(8)	exclude interfaces from report; used only when STATUS(1) is 0, 1, 2, 3, 4 (0 by default)
STATUS(8) = 0	no
STATUS(8) = 1	yes
STATUS(9)	run voltage limit check; used only when STATUS(1) is 0, 1, 3, 4 (0 by default)
STATUS(9) = 0	no
STATUS(9) = 1	yes
STATUS(10)	in overload reports, exclude monitored branches and interfaces that show load-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		ing violations in the base case from being checked and reported in contingency cases; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(10) = 0	no
	STATUS(10) = 1	yes
	STATUS(11)	in voltage range violation reports, exclude monitored buses that show voltage range violations in the base case from the corresponding check in contingency case reports; used only when STATUS(1) is 0 or 3 (0 by default)
	STATUS(11) = 0	no
	STATUS(11) = 1	yes
	STATUS(12)	exclude cases with no overloads from non-spreadsheet overload report or non-spreadsheet corrective actions report; used only when STATUS(1) is 3 (0 by default)
	STATUS(12) = 0	no
	STATUS(12) = 1	yes
	STATUS(13)	report post-tripping action solutions (0 by default)
	STATUS(13) = 0	no
	STATUS(13) = 1	yes
	STATUS(14)	report loss of loads (0 by default)
	STATUS(14) = 0	no
	STATUS(14) = 1	yes
	STATUS(15)	run angle difference report; used only when STATUS(1) is 0, 1, 3, 4 (0 by default)
	STATUS(15) = 0	no
	STATUS(15) = 1	yes
Integer INTVAL (5)	array of 5 elements specifying (input).	
	INTVAL(1)	number of low voltage range violations filtering criterion (0 by default)
	INTVAL(2)	number of high voltage range violations filtering criterion (0 by default)
	INTVAL(3)	number of voltage deviation violations filtering criterion; not applied to base case (0 by default)
	INTVAL(4)	number of buses in the largest disconnected island filtering criterion; not applied to base case (0 by default)
	INTVAL(5)	maximum number of elements in the available capacity table (no limit by default)
Real REALVAL (8)	array of 8 elements specifying (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALVAL(1)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	REALVAL(2)	system mismatch converged tolerance (MVA) (5.0 by default)
	REALVAL(3)	percent of flow rating; used only when STATUS(1) is 0, 3 or 4 (100.0 by default)
	REALVAL(4)	in overload reports, minimum contingency case flow change from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(5)	in overload reports, minimum contingency case percent loading increase from base case value; used only when STATUS(1) is 0 or 3 (0.0 by default)
	REALVAL(6)	in voltage range violation reports, minimum contingency case voltage change from base case value; used only when STATUS(1) is 0, 1, 3 or 4 (0.0 by default)
	REALVAL(7)	cutoff threshold for available capacity table; used only when STATUS(1) is 2 (99999.0 by default)
	REALVAL(8)	in angle difference report, cutoff threshold for angle differences, used only when STATUS(1) is 0, 1, 3 or 4 (0.0 by default)
character*80 CHAVAL (1)	array of elements specifying Selected Contingency Category label for the ACCC report. (input).	
	CHAVAL(1)	Selected Contingency Category label (blank by default)
Character*260 RFILE	Contingency Solution Output File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	invalid INTVAL value
	IERR = 3	invalid REALVAL value
	IERR = 4	error opening RFILE
	IERR = 5	error reading RFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.14. ACCC_TRIP_COR

This API routine is obsolete. It has been replaced by the API routine accc_trip_cor_2 .

Use this API to apply the AC contingency calculation with tripping simulations and corrective actions. When there are violations in the post-tripping simulation solutions, the corrections actions analysis is launched to eliminate the violations, with the objective of minimizing the controls adjustments, including load shedding, MW generation dispatch and phase shifter angle adjustments. Rating set program option setting and 100.0 percent of rating set are used to check branch and interface flow overload violations.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_ACCC_TRIP_COR  TOL  OPTACC(1)..OPTACC(8) OPTTRP(1)..OPTTRP(3)
 OPTCOR(1)..OPTCOR(5)  VALUES(1)..VALUES(5)  LABELS(1)..LABELS(4)  DFXFILE
  ACCFILE  THRFILE  INLFILE  TRPFILE
```

Python command syntax:

```
ierr = accc_trip_cor(tol, optacc, opttrp, optcor, values, labels, dfxfile,
 accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL ACCC_TRIP_COR(TOL, OPTACC, OPTTRP, OPTCOR, VALUES, LABELS, DFXFILE,
 ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Integer OPTCOR (5)	array of 5 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective actions (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flow solutions (from 0 to 10, 1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(5) = 1	enable
Real VALUES (5)	array of 5 elements specifying corrective actions parameters(input).	
	VALUES(1)	bus voltage violation tolerance (0.1 by default)
	VALUES(2)	branch flow overload tolerance (0.1 by default)
	VALUES(3)	generator control weighting factor (1.0 by default)
	VALUES(4)	load control weighting factor (1.0 by default)
	VALUES(5)	phase shifter control weighting factor (1.0 by default)
Character*32 LABELS (4)	array of 4 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(3)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(4)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTTRP value
	IERR = 9	invalid OPTCOR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 10	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.15. ACCC_TRIP_COR_2

This API routine is obsolete. It has been replaced by the API routine accc_trip_cor_3 .

This API is the second release of the function to run AC contingency analysis with tripping simulation and corrective actions. Following a post contingency solution, trip events are simulated if their tripping conditions are satisfied. Corrective actions are then specified automatically to eliminate the violations in each post tripping solution with the objective of minimizing the control adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls. Please note that flows are checked against rating set program option setting and 100.0 percent of the rating to determine flow overload violations.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_ACCC_TRIP_COR_2  TOL OPTACC(1)..OPTACC(8) OPTTRP(1)..OPTTRP(3)
 OPTCOR(1)..OPTCOR(8) VALUES(1)..VALUES(8) LABELS(1)..LABELS(7) DFXFILE
 ACCFILE THRFILE INLFILE TRPFILE
```

Python command syntax:

```
ierr = accc_trip_cor_2(tol, optacc, opttrp, optcor, values, labels,
 dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL ACCC_TRIP_COR_2(TOL, OPTACC, OPTTRP, OPTCOR, VALUES, LABELS, DFXFILE,
 ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (RESERVE)
	OPTACC(8) = 2	subsystem machines (PMAX)
	OPTACC(8) = 3	subsystem machines (INERTIA)
	OPTACC(8) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flow solutions (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (8)	array of 8 elements specifying corrective actions parameters(input).	
	VALUES(1)	bus voltage violation tolerance (0.1 by default)
	VALUES(2)	branch overload violation tolerance (0.1 by default)
	VALUES(3)	generator control weighting factor (1.0 by default)
	VALUES(4)	load control weighting factor (1.0 by default)
	VALUES(5)	phase shifter control weighting factor (1.0 by default)
	VALUES(6)	off-line generator control weighting factor (1.0 by default)
	VALUES(7)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(8)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (7)	array of 7 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(3)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(4)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
	LABELS(5)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(6)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(7)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Character*260 TRPFILE	name of Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTTRP value
	IERR = 9	invalid OPTCOR value
	IERR = 10	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multisection line members
	IERR = 49	error opening Tripping Element Data File
	IERR = 50	error opening Unit Inertia Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.16. ACCC_TRIP_COR_3

This API is the third release of the function to run AC contingency analysis with tripping simulation and corrective actions. Following a post contingency solution, trip events are simulated if their tripping conditions are satisfied. Corrective actions are then specified automatically to eliminate the violations in each post tripping solution with the objective of minimizing the control adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls. Please note that flows are checked against rating set program option setting and 100.0 percent of the rating to determine flow overload violations.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ACCC_TRIP_COR_3  TOL OPTACC(1)..OPTACC(10) OPTTRP(1)..OPTTRP(3)
 OPTCOR(1)..OPTCOR(8) VALUES(1)..VALUES(8) LABELS(1)..LABELS(7) DFXFILE
 ACCFILE THRFILE INLFILE TRPFILE
```

Python command syntax:

```
ierr = accc_trip_cor_3(tol, optacc, opttrp, optcor, values, labels,
 dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL ACCC_TRIP_COR_3(TOL, OPTACC, OPTTRP, OPTCOR, VALUES, LABELS, DFXFILE,
 ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (RESERVE)
	OPTACC(10) = 2	subsystem machines (PMAX)
	OPTACC(10) = 3	subsystem machines (INERTIA)
	OPTACC(10) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flow solutions (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (8)	array of 8 elements specifying corrective actions parameters(input).	
	VALUES(1)	bus voltage violation tolerance (0.1 by default)
	VALUES(2)	branch overload violation tolerance (0.1 by default)
	VALUES(3)	generator control weighting factor (1.0 by default)
	VALUES(4)	load control weighting factor (1.0 by default)
	VALUES(5)	phase shifter control weighting factor (1.0 by default)
	VALUES(6)	off-line generator control weighting factor (1.0 by default)
	VALUES(7)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(8)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (7)	array of 7 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(3)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(4)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
	LABELS(5)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(6)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(7)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Character*260 TRPFILE	name of Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	invalid OPTTRP value
	IERR = 11	invalid OPTCOR value
	IERR = 12	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multisection line members
	IERR = 49	error opening Tripping Element Data File
	IERR = 50	error opening Unit Inertia Data File
	IERR = 51	error opening Contingency Solution Output File

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.17. ACCC_WITH_COR

This API routine is obsolete. It has been replaced by the API routine accc_with_cor_2 .

Use this API to apply the AC contingency calculation with corrective actions. When there are violations in the post-contingency solutions, the corrections actions analysis is launched to eliminate the violations, with the objective of minimizing the controls adjustments, including load shedding, MW generation dispatch, and phase shifter angle adjustments. Rating set program option setting and 100.0 percent of the rating set are used to check branch and interface flow overload violations.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_ACCC_WITH_COR  TOL  OPTACC(1)..OPTACC(8)  OPTCOR(1)..OPTCOR(5)
 VALUES(1)..VALUES(5)  LABELS(1)..LABELS(4)  DFXFILE  ACCFILE  THRFILE
 INLFILE
```

Python command syntax:

```
ierr = accc_with_cor(tol, optacc, optcor, values, labels, dfxfile,
 accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_COR(TOL, OPTACC, OPTCOR, VALUES, LABELS, DFXFILE, ACCFILE,
 THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Integer OPTCOR (5)	array of 5 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective actions (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flow solutions (from 0 to 10, 1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load curtailment flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
Real VALUES (5)	array of 5 elements specifying corrective actions parameters(input).	
	VALUES(1)	bus voltage violation tolerance (0.1 by default)
	VALUES(2)	branch flow overload tolerance (0.1 by default)
	VALUES(3)	generator control weighting factor (1.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(4)	load control weighting factor (1.0 by default)
	VALUES(5)	phase shifter control weighting factor (1.0 by default)
Character*32 LABELS (4)	array of 4 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(3)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(4)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTCOR value
	IERR = 9	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 50	error opening Unit Inertia and Governor Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.18. ACCC_WITH_COR_2

This API routine is obsolete. It has been replaced by the API routine accc_with_cor_3 .

This API is the second release of function to run AC contingency analysis with corrective actions. Corrective actions are specified automatically to eliminate the violations in each post contingency solution with the objective of minimizing the controls adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls. Please note that flows are checked against rating set program option setting and 100.0 percent of the rating to determine flow overload violations.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_ACCC_WITH_COR_2  TOL OPTACC(1)..OPTACC(8) OPTCOR(1)..OPTCOR(8)
 VALUES(1)..VALUES(8) LABELS(1)..LABELS(7)  DFXFILE ACCFILE THRFILE
 INLFILE
```

Python command syntax:

```
ierr = accc_with_cor_2(tol, optacc, optcor, values, labels,  dfxfile,
 accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_COR_2(TOL, OPTACC, OPTCOR, VALUES, LABELS,  DFXFILE,
 ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (RESERVE)
	OPTACC(8) = 2	subsystem machines (PMAX)
	OPTACC(8) = 3	subsystem machines (INERTIA)
	OPTACC(8) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective actions (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (8)	array of 8 elements specifying corrective actions parameters(input).	
	VALUES(1)	bus voltage violation tolerance (0.1 by default)
	VALUES(2)	branch overload violation tolerance (0.1 by default)
	VALUES(3)	generator control weighting factor (1.0 by default)
	VALUES(4)	load control weighting factor (1.0 by default)
	VALUES(5)	phase shifter control weighting factor (1.0 by default)
	VALUES(6)	off-line generator control weighting factor (1.0 by default)
	VALUES(7)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(8)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (7)	array of 7 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(3)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(4)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
	LABELS(5)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(6)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(7)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid TOL value
IERR = 2	invalid OPTACC value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	invalid OPTCOR value
IERR = 9	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multisection line members
IERR = 50	error opening Unit Inertia Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.19. ACCC_WITH_COR_3

This API is the third release of function to run AC contingency analysis with corrective actions. Corrective actions are specified automatically to eliminate the violations in each post contingency solution with the objective of minimizing the controls adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls. Please note that flows are checked against rating set program option setting and 100.0 percent of the rating to determine flow overload violations.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ACCC_WITH_COR_3  TOL OPTACC(1)..OPTACC(10) OPTCOR(1)..OPTCOR(8)
 VALUES(1)..VALUES(8) LABELS(1)..LABELS(7)  DFXFILE ACCFILE THRFILE
 INLFILE
```

Python command syntax:

```
ierr = accc_with_cor_3(tol, optacc, optcor, values, labels,  dfxfile,
 accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_COR_3(TOL, OPTACC, OPTCOR, VALUES, LABELS,  DFXFILE,
 ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN, by default).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (RESERVE)
	OPTACC(10) = 2	subsystem machines (PMAX)
	OPTACC(10) = 3	subsystem machines (INERTIA)
	OPTACC(10) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective actions (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (8)	array of 8 elements specifying corrective actions parameters(input).	
	VALUES(1)	bus voltage violation tolerance (0.1 by default)
	VALUES(2)	branch overload violation tolerance (0.1 by default)
	VALUES(3)	generator control weighting factor (1.0 by default)
	VALUES(4)	load control weighting factor (1.0 by default)
	VALUES(5)	phase shifter control weighting factor (1.0 by default)
	VALUES(6)	off-line generator control weighting factor (1.0 by default)
	VALUES(7)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(8)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (7)	array of 7 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(3)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(4)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
	LABELS(5)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(6)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(7)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	invalid OPTCOR value
	IERR = 11	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multisection line members
	IERR = 50	error opening Unit Inertia Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.20. ACCC_WITH_DSP

This API routine is obsolete. It has been replaced by the API routine accc_with_dsp_2 .

Use this API to apply the AC contingency calculation function with a generation dispatch function to handle imbalances in power resources and demand due to contingencies.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ACCC_WITH_DSP  TOL  OPTACC(1)..OPTACC(8)  LABEL  DFXFILE  ACCFILE
 THRFILE  INLFILE
```

Python command syntax:

```
ierr = accc_with_dsp(tol, optacc, label, dfxfile, accfile, thrfile,
 inlfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_DSP(TOL, OPTACC, LABEL, DFXFILE, ACCFILE, THRFILE, INLFILE,
 IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Character*32 LABEL	name of the generation dispatch subsystem (input; blank by default, no default allowed if OPTACC(8) is not 0).	
Character*260 DFXFILE	name of the Distribution Factor Data file (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output file (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data file (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	no Distribution Factor Data File specified
	IERR = 9	no Contingency Solution Output File specified
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	error opening Unit Inertia and Governor Data File
IERR = 55	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.21. ACCC_WITH_DSP_2

This API routine is obsolete. It has been replaced by the API routine accc_with_dsp_3 .

Use this API to run the second release of AC contingency calculation function (ACCC, ACCC_WITH_DSP). The API provides the options to handle induction machines in a contingency analysis.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ACCC_WITH_DSP_2  TOL  OPTIONS(1)..OPTIONS(10)  LABEL  DFXFILE  ACCFILE
  THRFILE  INLFILE
```

Python command syntax:

```
ierr = accc_with_dsp_2(tol, options, label, dfxfile, accfile, thrfile,
 inlfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_DSP_2(TOL, OPTIONS, LABEL, DFXFILE, ACCFILE, THRFILE,
 INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTIONS (10)	array of 10 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	solution method flag (0 by default)
OPTIONS(6) = 0	FDNS
OPTIONS(6) = 1	FNSL
OPTIONS(6) = 2	optimized FDNS
OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(7) = 0	disable
OPTIONS(7) = 1	enable
OPTIONS(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(8) = 0	stall
OPTIONS(8) = 1	trip
OPTIONS(9)	induction machine failure flag (0 by default)
OPTIONS(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
OPTIONS(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
OPTIONS(10)	dispatch mode (0 by default)
OPTIONS(10) = 0	disable
OPTIONS(10) = 1	subsystem machines (reserve)
OPTIONS(10) = 2	subsystem machines (pmax)
OPTIONS(10) = 3	subsystem machines (inertia)
OPTIONS(10) = 4	subsystem machines (governor droop)

Character*32 LABEL — name of the generation dispatch subsystem (input; blank by default, no default allowed if OPTIONS(10) is not 0).

Character*260 DFXFILE — name of the Distribution Factor Data file (input; no default allowed).

Character*260 ACCFILE — name of the Contingency Solution Output file (input; no default allowed).

Character*260 THRFILE — name of the Load Throwover Data file (input; blank by default).

Character*260 INLFILE — name of the Unit Inertia and Governor Data File (input; blank by default).

Integer IERR — error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTIONS value
IERR = 3	generators are converted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation dispatch subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	no Distribution Factor Data File specified
IERR = 9	no Contingency Solution Output File specified
IERR = 10	in-service induction machines are in the "stalled" or "tripped" state
IERR = 11	buses with bus type code and series element status inconsistencies
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	error opening Unit Inertia and Governor Data File
IERR = 55	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.22. ACCC_WITH_DSP_3

Use this API to run the second release of AC contingency calculation function (ACCC, ACCC_WITH_DSP). The API provides the options to handle induction machines in a contingency analysis.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_ACCC_WITH_DSP_3  TOL OPTACC(1)..OPTACC(11) LABEL DFXFILE ACCFILE
 THRFILE INLFILE ZIPFILE
```

Python command syntax:

```
ierr = accc_with_dsp_3(tol, optacc, label, dfxfile, accfile, thrfile,
 inlfile, zipfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_DSP_3(TOL, OPTACC, LABEL, DFXFILE, ACCFILE, THRFILE,
 INLFILE, ZIPFILE, IERR)
```

Where:

Real TOL	Is the mismatch tolerance. (input; Newton solution convergence tolerance, TOLN, by default).	
Integer OPTACC (11)	array of elements specifying solution options. The values are as follows:(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTACC(5) = 1	enable
OPTACC(5) = 2	enable continuous mode, disable discrete mode
OPTACC(6)	solution method flag (0 by default)
OPTACC(6) = 0	FDNS
OPTACC(6) = 1	FNSL
OPTACC(6) = 2	optimized FDNS
OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTACC(7) = 0	disable
OPTACC(7) = 1	enable
OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage)
OPTACC(8) = 0	stall
OPTACC(8) = 1	trip
OPTACC(9)	induction machine failure flag (0 by default)
OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(10)	dispatch mode (0 by default)
OPTACC(10) = 0	disable
OPTACC(10) = 1	subsystem machines (reserve)
OPTACC(10) = 2	subsystem machines (pmax)
OPTACC(10) = 3	subsystem machines (inertia)
OPTACC(10) = 4	subsystem machines (governor droop)
OPTACC(11)	ZIP archive flag (0 by default)
OPTACC(11) = 0	do not write a ZIP archive file
OPTACC(11) = 1	write a ZIP archive using the file specified as ZIPFILE

Character*32 LABEL	Is the name of the generation dispatch subsystem (blank by default, no default allowed if OPTIONS(10) is not 0). (input).
Character*260 DFXFILE	Is the name of the Distribution Factor Data file (input; no default allowed).
Character*260 ACCFILE	Is the name of the Contingency Solution Output file (input; no default allowed).
Character*260 THRFILE	Is the name of the Load Throwover Data file (input; blank by default).
Character*260 INLFILE	Is the name of the Unit Inertia and Governor Data File (input; blank by default).
Character*260 ZIPFILE	Is the name of the ZIP Archive Output File (input; blank by default).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTIONS value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation dispatch subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	no Distribution Factor Data File specified
IERR = 9	no AC Contingency Solution Output File specified
IERR = 10	in-service induction machines are in the "stalled" or "tripped" state
IERR = 11	buses with bus type code and series element status inconsistencies
IERR = 12	no ZIP Archive Output File specified
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	error opening Unit Inertia and Governor Data File
IERR = 55	error opening ZIP Archive Output File
IERR = 56	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.23. ACCC_WITH_TRIP

This API routine is obsolete. It has been replaced by the API routine accc_with_trip_2 .

Use this API to apply the AC contingency calculation function with a post-contingency tripping function. This may be used, for example, to simulate switchings that occur when specified monitored quantities are beyond threshold values.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ACCC_WITH_TRIP  TOL  OPTACC(1)..OPTACC(8)  OPTTRP(1)..OPTTRP(3)  LABEL
   DFXFILE   ACCFILE   THRFILE  INLFILE  TRPFILE
```

Python command syntax:

```
ierr = accc_with_trip(tol, optacc, opttrp, label, dfxfile, accfile,
 thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_TRIP(TOL, OPTACC, OPTTRP, LABEL, DFXFILE, ACCFILE, THRFILE,
 INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Character*32 LABEL	name of the generation dispatch subsystem (input; blank by default; no default allowed if OPTACC(8) is not 0).	
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	Invalid OPTTRP value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.24. ACCC_WITH_TRIP_2

Use this API to run the second version of AC contingency calculation function with a post-contingency tripping function. This may be used, for example, to simulate switchings that occur when specified monitored quantities are beyond threshold values.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ACCC_WITH_TRIP_2  TOL  OPTACC(1)..OPTACC(10)  OPTTRP(1)..OPTTRP(3)
 LABEL   DFXFILE   ACCFILE   THRFILE  INLFILE  TRPFILE
```

Python command syntax:

```
ierr = accc_with_trip_2(tol, optacc, opttrp, label, dfxfile, accfile,
 thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_TRIP_2(TOL, OPTACC, OPTTRP, LABEL, DFXFILE, ACCFILE,
 THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (reserve)
	OPTACC(10) = 2	subsystem machines (pmax)
	OPTACC(10) = 3	subsystem machines (inertia)
	OPTACC(10) = 4	subsystem machines (governor droop)
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Character*32 LABEL	name of the generation dispatch subsystem (input; blank by default; no default allowed if OPTACC(8) is not 0).	
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTACC value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	Invalid OPTTRP value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-sec-tion line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.25. ACCC_WITH_TRIP_PARALLEL

Use this API to run the parallel version of AC contingency calculation function with a post-contingency tripping function (use ACCC_WITH_TRIP_2 to run the function sequentially). The API distributes contingency cases among multiple processors evenly, and tests the contingency cases and following tripping actions assigned to each processor in parallel.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_ACCC_WITH_TRIP_PARALLEL  TOL OPTACC(1)..OPTACC(10)
 OPTTRP(1)..OPTTRP(3) LABEL DFXFILE ACCFILE THRFILE INLFILE TRPFILE
```

Python command syntax:

```
ierr = accc_with_trip_parallel(tol, optacc, opttrp, label, dfxfile,
 accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL ACCC_WITH_TRIP_PARALLEL(TOL, OPTACC, OPTTRP, LABEL, DFXFILE, ACCFILE,
 THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	Is the mismatch tolerance (input; Newton solution convergence tolerance, TOLN, by default).	
Integer OPTACC (10)	array of elements specifying power flow solution options. The values are as follows:(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable.
	OPTACC(1) = 1	enable stepping adjustment.
	OPTACC(1) = 2	enable direct adjustment.
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable.
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange.
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange.
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable.
	OPTACC(3) = 1	enable.
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable.
	OPTACC(4) = 1	enable.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable.
	OPTACC(5) = 1	enable.
	OPTACC(5) = 2	enable continuous mode, disable discrete mode.
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS.
	OPTACC(6) = 1	FNSL.
	OPTACC(6) = 2	optimized FDNS.
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable.
	OPTACC(7) = 1	enable.
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage). (0 by default)
	OPTACC(8) = 0	stall.
	OPTACC(8) = 1	trip.
	OPTACC(9)	induction machine failure flag. (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state.
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state.
	OPTACC(10)	dispatch mode. (0 by default)
	OPTACC(10) = 0	disable.
	OPTACC(10) = 1	subsystem machines (reserve).
	OPTACC(10) = 2	subsystem machines (pmax).
	OPTACC(10) = 3	subsystem machines (inertia).
	OPTACC(10) = 4	subsystem machines (governor droop).
Integer OPTTRP (3)	array of elements specifying tripping simulation options. The values are as follows:(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable.
	OPTTRP(1) = 1	enable tripping simulation.
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Character*32 LABEL	Is the name of the generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0). (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 DFXFILE	Is the name of the Distribution Factor Data File. (input; no default allowed).
Character*260 ACCFILE	Is the name of the Contingency Solution Output File. (input; no default allowed).
Character*260 THRFILE	Is the name of the Load Throwover Data File. (input; blank by default).
Character*260 INLFILE	Is the name of the Unit Inertia and Governor Data File. (input; blank by default).
Character*260 TRPFILE	Is the name of the Tripping Element Data File (input, blank for none; no default allowed if OPTTRP(1) is 1). (input).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTACC value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation dispatch subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	Invalid OPTTRP value.
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 49	error opening Tripping Element Data File.
IERR = 50	error opening Unit Inertia and Governor Data File.
IERR = 51	error opening Contingency Solution Output File.
IERR = 52	error opening Distribution Factor Data File.
IERR = 53	error opening Load Throwover Data File.
IERR = 54	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.26. ACCOR

This API routine is obsolete. It has been replaced by the API routine accor_2 .

Use this API to apply corrective actions to the base case using successive linear programming methods. The corrective actions are to eliminate flow overloads and voltage violations with the objective of minimizing the control adjustments, including load shedding, MW generations dispatch, and phase shifter angle adjustments.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_ACCOR  OPTIONS(1)..OPTIONS(10)  VALUES(1)..VALUES(7)
 LABELS(1)..LABELS(3)  DFXFILE
```

Python command syntax:

```
ierr = accor(options, values, labels, dfxfile)
```

Fortran command syntax:

```
CALL ACCORAPI(OPTIONS, VALUES, LABELS, DFXFILE, IERR)
```

Where:

Integer OPTIONS (10)	array of 10 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	rating set (rating set program option setting by default)
	OPTIONS(6) = 1	rating A
	OPTIONS(6) = 2	rating B
	OPTIONS(6) = 3	rating C
	OPTIONS(7)	number of power flow solutions (from 1 to 10; 1 by default)
	OPTIONS(8)	generator control flag (1 by default)
	OPTIONS(8) = 0	disable
	OPTIONS(8) = 1	enable
	OPTIONS(9)	load control flag (0 by default)
	OPTIONS(9) = 0	disable
	OPTIONS(9) = 1	enable
	OPTIONS(10)	phase shifter control flag (1 by default)
	OPTIONS(10) = 0	disable
	OPTIONS(10) = 1	enable
Real VALUES (7)	array of 7 elements specifying AC corrective actions parameters(input).	
	VALUES(1)	MW mismatch tolerance (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating set (100.0 by default)
	VALUES(3)	bus voltage violation tolerance (0.1 by default)
	VALUES(4)	branch flow overload tolerance (0.1 by default)
	VALUES(5)	generator control weighting factor (1.0 by default)
	VALUES(6)	load control weighting factor (1.0 by default)
	VALUES(7)	phase shifter control weighting factor (1.0 by default)
Character*32 LABELS (3)	array of 3 elements specifying subsystem labels(input).	
	LABELS(1)	label of generator control subsystem (blank by default; no default allowed if OPTIONS(8) is 1)
	LABELS(2)	label of load control subsystem (blank by default; no default allowed if OPTIONS(9) is 1)
	LABELS(3)	label of phase shifter control subsystem (blank by default; no default allowed if OPTIONS(10) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid OPTIONS value
IERR = 2	invalid VALUES value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	No controls
IERR = 6	control subsystem not defined
IERR = 7	Distribution Factor Data File not specified
IERR = 8	largest mismatch exceeds tolerance
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Distribution Factor Data File
IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.27. ACCOR_2

This API routine is obsolete. It has been replaced by the API routine accor_3 .

This API is the second release of corrective action function. Use this API to augment control strategies with off-line generator control, tap setting adjustments and switched shunt controls.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_ACCOR_2  OPTIONS(1)..OPTIONS(13)  VALUES(1)..VALUES(10)
 LABELS(1)..LABELS(6)  DFXFILE
```

Python command syntax:

```
ierr = accor_2(options, values, labels, dfxfile)
```

Fortran command syntax:

```
CALL ACCORAPI_2(OPTIONS, VALUES, LABELS, DFXFILE, IERR)
```

Where:

Integer OPTIONS (13)	array of 13 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	rating set (rating set program option setting by default)
	OPTIONS(6) = 1	rating A
	OPTIONS(6) = 2	rating B
	OPTIONS(6) = 3	rating C
	OPTIONS(7)	number of power flow solutions (from 1 to 10; 1 by default)
	OPTIONS(8)	generator control flag (1 by default)
	OPTIONS(8) = 0	disable
	OPTIONS(8) = 1	enable
	OPTIONS(9)	load control flag (0 by default)
	OPTIONS(9) = 0	disable
	OPTIONS(9) = 1	enable
	OPTIONS(10)	phase shifter control flag (1 by default)
	OPTIONS(10) = 0	disable
	OPTIONS(10) = 1	enable
	OPTIONS(11)	off-line generator control flag (0 by default)
	OPTIONS(11) = 0	disable
	OPTIONS(11) = 1	enable
	OPTIONS(12)	tap setting adjustment flag (0 by default)
	OPTIONS(12) = 0	disable
	OPTIONS(12) = 1	enable
	OPTIONS(13)	switched shunt control flag (0 by default)
	OPTIONS(13) = 0	disable
	OPTIONS(13) = 1	enable
Real VALUES (10)	array of 10 elements specifying AC corrective actions parameters(input).	
	VALUES(1)	MW mismatch tolerance (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating set (100.0 by default)
	VALUES(3)	bus voltage violation tolerance (0.1 by default)
	VALUES(4)	branch flow overload tolerance (0.1 by default)
	VALUES(5)	generator control weighting factor (1.0 by default)
	VALUES(6)	load control weighting factor (1.0 by default)
	VALUES(7)	phase shifter control weighting factor (1.0 by default)
	VALUES(8)	off-line generator control weighting factor (1.0 by default)
	VALUES(9)	tap setting adjustment weighting factor (1.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(10)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (6)	array of 6 elements specifying subsystem labels(input).	
	LABELS(1)	label of generator control subsystem (blank by default; no default allowed if OPTIONS(8) is 1)
	LABELS(2)	label of load control subsystem (blank by default; no default allowed if OPTIONS(9) is 1)
	LABELS(3)	label of phase shifter control subsystem (blank by default; no default allowed if OPTIONS(10) is 1)
	LABELS(4)	off-line generator control subsystem (blank by default; no default allowed if OPTIONS(11) is 1)
	LABELS(5)	tap setting adjustment subsystem (blank by default; no default allowed if OPTIONS(12) is 1)
	LABELS(6)	switched shunt control subsystem (blank by default; no default allowed if OPTIONS(13) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus, use activity TREE
	IERR = 5	No controls
	IERR = 6	control subsystem not defined
	IERR = 7	Distribution Factor Data File not specified
	IERR = 8	largest mismatch exceeds tolerance
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file, run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 51	error opening Distribution Factor Data File
	IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.28. ACCOR_3

This API is the third release of corrective action function. Use this API to specify voltage limits in determining voltage violations.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_ACCOR_3  OPTIONS(1)..OPTIONS(14) VALUES(1)..VALUES(10)
 LABELS(1)..LABELS(6) DFXFILE
```

Python command syntax:

```
ierr = accor_3(options, values, labels, dfxfile)
```

Fortran command syntax:

```
CALL ACCORAPI_3(OPTIONS, VALUES, LABELS, DFXFILE, IERR)
```

Where:

Integer OPTIONS (14)	array of elements specifying solution options. The values are as follows:(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable.
	OPTIONS(1) = 1	enable stepping adjustment.
	OPTIONS(1) = 2	enable direct adjustment.
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable.
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange.
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange.
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable.
	OPTIONS(3) = 1	enable.
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable.
	OPTIONS(4) = 1	enable.
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable.
	OPTIONS(5) = 1	enable.
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(6)	rating set (rating set program option setting by default)
OPTIONS(6) = 1	rating A
OPTIONS(6) = 2	rating B
OPTIONS(6) = 3	rating C
OPTIONS(7)	voltage limit
OPTIONS(7) = 1	normal
OPTIONS(7) = 2	emergency
OPTIONS(8)	number of power flow solutions (from 1 to 10; 1 by default)
OPTIONS(9)	generator control flag (1 by default)
OPTIONS(9) = 0	disable.
OPTIONS(9) = 1	enable.
OPTIONS(10)	load control flag. (0 by default)
OPTIONS(10) = 0	disable.
OPTIONS(10) = 1	enable.
OPTIONS(11)	phase shifter control flag (1 by default)
OPTIONS(11) = 0	disable.
OPTIONS(11) = 1	enable.
OPTIONS(12)	off-line generator control flag (0 by default)
OPTIONS(12) = 0	disable.
OPTIONS(12) = 1	enable.
OPTIONS(13)	tap setting adjustment flag (0 by default)
OPTIONS(13) = 0	disable.
OPTIONS(13) = 1	enable.
OPTIONS(14)	switched shunt control flag (0 by default)
OPTIONS(14) = 0	disable.
OPTIONS(14) = 1	enable.

Real VALUES (10) array of elements specifying AC corrective actions parameters. The values are as follows:(input).

VALUES(1)	MW mismatch tolerance (Newton solution convergence tolerance, TOLN by default)
VALUES(2)	percent of rating set (100.0 by default)
VALUES(3)	bus voltage violation tolerance (0.1 by default)
VALUES(4)	branch flow overload tolerance (0.1 by default)
VALUES(5)	generator control weighting factor (1.0 by default)
VALUES(6)	load control weighting factor (1.0 by default)
VALUES(7)	phase shifter control weighting factor (1.0 by default)
VALUES(8)	off-line generator control weighting factor (1.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(9)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(10)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (6)	array of elements specifying subsystem labels (input). The values are as follows:(input).	
	LABELS(1)	label of generator control subsystem (blank by default; no default allowed if OPTIONS(9) is 1).
	LABELS(2)	label of load control subsystem (blank by default; no default allowed if OPTIONS(10) is 1).
	LABELS(3)	label of phase shifter control subsystem (blank by default; no default allowed if OPTIONS(11) is 1).
	LABELS(4)	off-line generator control subsystem (blank by default; no default allowed if OPTIONS(12) is 1).
	LABELS(5)	tap setting adjustment subsystem (blank by default; no default allowed if OPTIONS(13) is 1).
	LABELS(6)	switched shunt control subsystem (blank by default; no default allowed if OPTIONS(14) is 1).
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	No controls.
	IERR = 6	control subsystem not defined.
	IERR = 7	Distribution Factor Data File not specified.
	IERR = 8	largest mismatch exceeds tolerance.
	IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX.
	IERR = 22	monitored elements exceed limit when adding multi-section line members.
	IERR = 51	error opening Distribution Factor Data File.
	IERR = 52	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.29. ALERT_OUTPUT

Use this API to specify the alert device.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_ALERT_OUTPUT ISLCT FILARG OPTIONS(1) OPTIONS(2)
```

Python command syntax:

```
ierr = alert_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL OPENALERTDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input; 1 by default).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for alerts is a message box and a copy to the Alerts/Warnings tab of the output bar.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input; blank by default).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options (0 by default)
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.30. ALLOW_PSSUSERPF

Use this API to specify or return the option to allow or disallow the loading or use of the Powerflow Customization Interface (PCI) implementation module (pssuserpf).

This API routine was first introduced in release 33.6.0 and 34.0.1.

Batch command syntax:

```
BAT_ALLOW_PSSUSERPF IVAL
```

Python command syntax:

```
ierr = allow_pssuserpf(ival)

ierr,ival = allow_pssuserpf()
```

Fortran command syntax:

```
CALL ALLOW_PSSUSERPF(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	PCI disabled.
	IVAL = 1	PCI enabled.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.31. ALPH

Use this API to print an alphabetically sorted table of all buses in a specified subsystem of the working case (activity ALPH).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ALPH SID ALL
```

Python command syntax:

```
ierr = alph(sid, all)
```

Fortran command syntax:

```
CALL ALPHNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.32. APPEND_ACCC

Use this API to replicate system conditions of a contingency case solution, as contained in a designated Saved Case File, in the form of a Contingency Solution Output File. Results may be appended to an existing Contingency Solution Output File, or an new Contingency Solution Output File may be created.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_APPEND_ACCC  OPTION  DFXFILE  BASFILE  CNTFILE  ACCFILE  LABEL
 NUMDESC  DESCRPT(1)..DESCRPT(NUMDESC)
```

Python command syntax:

```
ierr = append_accc(option, dfxfile, basfile, cntfile, accfile, label,
 numdesc, descrpt)
```

Fortran command syntax:

```
CALL APPEND_ACCC(OPTION, DFXFILE, BASFILE, CNTFILE, ACCFILE, LABEL,
 NUMDESC, DESCRPT, IERR)
```

Where:

Integer OPTION	append flag (input; 1 by default). OPTION = 0 create a new Contingency Solution Output File in the file specified in ACCFILE. OPTION = 1 append results to the existing Contingency Solution Output File specified in ACCFILE
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).
Character*260 BASFILE	name of the Saved Case File containing the pre-contingency power flow working case. This is the case that was present when the Distribution Factor Data File specified in DFXFILE was constructed; it is also the case from which the contingency case was formed. Its solution is in the Saved Case File specified in CNTFILE (input; no default allowed).
Character*260 CNTFILE	name of the Saved Case File containing the contingency case solution for which results are to be placed in the Contingency Solution Output File specified in ACCFILE. This case must be formed by imposing status and/or boundary condition changes on the pre-contingency case specified in BASFILE, and then solving using one or more of the power flow solution activities (input; no default allowed).
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).
Character*32 LABEL	12 character contingency label (input; blank by default).
Integer NUMDESC	number of lines of text used to describe the contingency case; must be between 1 and 16 (input; no default allowed).
Character*160 DESCRPT (NUMDESC)	Are NUMDESC lines of text describing the contingency case (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTION value
	IERR = 2	invalid NUMDESC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	no base case Saved Case File specified
	IERR = 6	no contingency case Saved Case File specified
	IERR = 7	Contingency Solution Output File not found
	IERR = 8	DFAX File and Contingency Solution Output File are not compatible
	IERR = 9	no Distribution Factor Data File specified
	IERR = 10	no Contingency Solution Output File specified
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening pre-contingency Saved Case File
	IERR = 54	error opening contingency case Saved Case File
	IERR = 55	error reading Contingency Solution Output File
	IERR = 56	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.33. APPLY_VAR_LIMITS

Use this API to specify or return the option for the default VAR limits setting; either apply automatically, apply immediately, ignore, or apply on a specific iteration

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_APPLY_VAR_LIMITS IVAL
```

Python command syntax:

```
ierr = apply_var_limits(ival)

ierr, ival = apply_var_limits()
```

Fortran command syntax:

```
CALL APPLY_VAR_LIMITS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = -1	ignore
	IVAL = 0	apply immediately
	IVAL = >0	apply on the specified iteration or when the largest reactive power mismatch is less than a pre-programmed tolerance
	IVAL = 99	apply automatically
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.34. AREA

This API routine is obsolete. It has been replaced by the API routine area_2 .

Use this API to tabulate area totals by area, as well as the desired area net interchange (activity AREA).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_AREA SID ALL
```

Python command syntax:

```
ierr = area(sid, all)
```

Fortran command syntax:

```
CALL AREANW(SID, ALL, IERR)
```

Where:

Integer SID	a valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all areas or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only areas in area subsystem SID.
	ALL = 1	process all areas.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.35. AREA_2

Use this API to tabulate area totals by area, as well as the desired area net interchange (activity AREA).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_AREA_2 SID ALL OPT
```

Python command syntax:

```
ierr = area_2(sid, all, opt)
```

Fortran command syntax:

```
CALL AREA_2(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all areas or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only areas in area subsystem SID.
	ALL = 1	process all areas.
Integer OPT	load and induction machine reporting option (input; 2 by default).	
	OPT = 1	tabulate loads, induction generators and induction motors connected to area buses
	OPT = 2	tabulate loads, induction generators and induction motors assigned to the area
	OPT = 3	tabulate three load and induction machine subtotals for each area: -area load and induction machines connected to area buses -area load and induction machines connected to non-area buses -non-area load and induction machines connected to area buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	Invalid OPT value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.36. AREA_ZONE

Use this API to tabulate area totals by area, along with subtotals by zone. The desired net interchange of each area reported is also tabulated.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_AREA_ZONE SID ALL
```

Python command syntax:

```
ierr = area_zone(sid, all)
```

Fortran command syntax:

```
CALL AREA_ZONE(SID, ALL, IERR)
```

Where:

Integer SID	a valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all areas or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only areas in area subsystem SID.
	ALL = 1	process all areas.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.37. ARNM

This API routine is obsolete. It has been replaced by the API routine arnm_2 .

Use this API to reassign the buses and loads in a specified subsystem of the working case from their original area to a designated area (activity ARNM).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ARNM  SID  ALL  STATUS(1)  STATUS(2)  IANEW
```

Python command syntax:

```
ierr = arnm(sid, all, status, ianew)
```

Fortran command syntax:

```
CALL ARNMNW(SID, ALL, STATUS, IANEW, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer STATUS (2)	array of 2 elements specifying (input).	
	STATUS(1)	Area assignments of buses to IANEW; (0 by default)
	STATUS(1) = 1	Change area assignments of buses to IANEW
	STATUS(1) = 0	Leave area assignments of buses unchanged
	STATUS(2)	Area assignments of loads to IANEW; (0 by default)
	STATUS(2) = 1	Change area assignments of loads to IANEW
	STATUS(2) = 0	Leave area assignments of loads unchanged
Integer IANEW	new area to be assigned to specified buses and/or loads. IANEW must be between 1 and 9999 (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid new area number
	IERR = 5	area table is full; cannot add area to the working case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.38. ARNM_2

This API routine is the second release of the area renumbering function. Use this API routine to reassign the buses, loads and/or induction machines in a specified subsystem of the working case from their original area to a designated area (activity ARNM).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ARNM_2  SID  ALL  STATUS(1)  STATUS(2)  STATUS(3)  IANEW
```

Python command syntax:

```
ierr = arnm_2(sid, all, status, ianew)
```

Fortran command syntax:

```
CALL ARNM_2(SID, ALL, STATUS, IANEW, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer STATUS (3)	array of 3 elements specifying (input).	
	STATUS(1)	Area assignments of buses to IANEW; (0 by default)
	STATUS(1) = 1	Change area assignments of buses to IANEW
	STATUS(1) = 0	Leave area assignments of buses unchanged
	STATUS(2)	Area assignments of loads to IANEW; (0 by default)
	STATUS(2) = 1	Change area assignments of loads to IANEW
	STATUS(2) = 0	Leave area assignments of loads unchanged
	STATUS(3)	Area assignments of induction machines to IANEW; (0 by default)
	STATUS(3) = 1	Change area assignments of induction machines to IANEW
	STATUS(3) = 0	Leave area assignments of induction machines unchanged
Integer IANEW	new area to be assigned to specified buses, load, and/or induction machines. IANEW must be between 1 and 9999 (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ALL value
IERR = 3	invalid STATUS value
IERR = 4	invalid new area number
IERR = 5	area table is full; cannot add area to the working case
IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.39. BASE_FREQUENCY

Use this API to specify or return the value of the base frequency option setting.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_BASE_FREQUENCY RVAL
```

Python command syntax:

```
ierr = base_frequency(rval)

ierr, rval = base_frequency()
```

Fortran command syntax:

```
CALL BASE_FREQUENCY_API(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	base frequency (usually 50 or 60) (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 60.0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.40. BGEN

Use this API to convert the mismatch at boundary buses to equivalent load and/or generation (activity BGEN). It is intended to be used following the removal of a subsystem from a solved working case, with the flows to the deleted subsystem replaced by load and/or generation at those retained buses that were directly connected to the removed subsystem.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BGEN  SID  ALL  STATUS
```

Python command syntax:

```
ierr = bgen(sid, all, status)
```

Fortran command syntax:

```
CALL BGENNW(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer STATUS	type of boundary condition modification to be used (input; 0 by default).	
	STATUS = 0	generator for inflow; load for outflow.
	STATUS = 1	all equivalent generators.
	STATUS = 2	all equivalent loads.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.41. BRCH

This API routine is obsolete. It has been replaced by the API routine brch_2 .

Use this API to tabulate those branches where impedances or other characteristics are such that they may be detrimental to the rate of convergence of one or more of the power flow solution activities.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BRCH SID ALL DOTEST(1)..DOTEST(10) TOLVAL(1)..TOLVAL(7)
```

Python command syntax:

```
ierr = brch(sid, all, dotest, tolval)
```

Fortran command syntax:

```
CALL BRCHNW(SID, ALL, DOTEST, TOLVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).
Integer ALL	all buses or specified subsystem flag (input; 1 by default). ALL = 0 process only buses in subsystem SID ALL = 1 process all buses
Integer DOTEST (10)	array of 10 elements specifying options. Each element determines whether a specific branch check should be applied. For DOTEST(I) = 1, the corresponding branch check is applied; for DOTEST(I) = 0, the check is bypassed(input). DOTEST(1) check for small reactances DOTEST(2) check for large reactances DOTEST(3) check for large resistance to reactance ratios (R/X) DOTEST(4) check for negative reactances DOTEST(5) check for large ratio of largest to smallest reactance at each bus DOTEST(6) check for high or negative charging DOTEST(7) check data of parallel transformers DOTEST(8) check for high tap ratios DOTEST(9) check for low tap ratios DOTEST(10) check for zero sequence non-transformer branch impedances of 0.0
Real TOLVAL (7)	array of 7 elements specifying options. Each element contains a limit or threshold value used in one of the branch checks(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TOLVAL(1)	>0.0 reactance lower limit (0.0005 by default)
	TOLVAL(2)	>0.0 reactance upper limit (1.0 by default)
	TOLVAL(3)	>0.0 R/X threshold (0.66667 by default)
	TOLVAL(4)	>=10.0 Xmax/Xmin threshold (500.0 by default)
	TOLVAL(5)	>=0.0 charging upper limit (5.0 by default)
	TOLVAL(6)	>0.8 high tap ratio limit (1.1 by default)
	TOLVAL(7)	>0.0 and < 1.2 low tap ratio limit (0.9 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid DOTEST value
	IERR = 4	invalid TOLVAL value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.42. BRCH_2

Use this API to tabulate those branches where impedances or other characteristics are such that they may be detrimental to the rate of convergence of one or more of the power flow solution activities.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_BRCH_2 SID ALL DOTEST(1)..DOTEST(11) TOLVAL(1)..TOLVAL(7)
```

Python command syntax:

```
ierr = brch_2(sid, all, dotest, tolval)
```

Fortran command syntax:

```
CALL BRCH_2(SID, ALL, DOTEST, TOLVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer DOTEST (11)	array of elements specifying options. Each element determines whether a specific branch check should be applied. For DOTEST(I) = 1, the corresponding branch check is applied; for DOTEST(I) = 0, the check is bypassed(input).	
	DOTEST(1)	check for small reactances.
	DOTEST(2)	check for large reactances.
	DOTEST(3)	check for large resistance to reactance ratios (R/X).
	DOTEST(4)	check for negative reactances.
	DOTEST(5)	check for large ratio of largest to smallest reactance at each bus.
	DOTEST(6)	check for high or negative charging.
	DOTEST(7)	check data of parallel transformers.
	DOTEST(8)	check for high tap ratios.
	DOTEST(9)	check for low tap ratios.
	DOTEST(10)	check for inconsistent transformer loss data values (copper loss and no-load loss).
	DOTEST(11)	check for zero sequence non-transformer branch impedances of 0.0.
Real TOLVAL (7)	array of elements specifying options. Each element contains a limit or threshold value used in one of the branch checks (input).	
	TOLVAL(1)	>0.0. reactance lower limit (0.0005 by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TOLVAL(2)	>0.0. reactance upper limit (1.0 by default).
	TOLVAL(3)	>0.0. R/X threshold (0.66667 by default).
	TOLVAL(4)	>=10.0. Xmax/Xmin threshold (500.0 by default).
	TOLVAL(5)	>=0.0. charging upper limit (5.0 by default).
	TOLVAL(6)	>0.8. high tap ratio limit (1.1 by default).
	TOLVAL(7)	> 0.0 and < 1.2. low tap ratio limit (0.9 by default).
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid DOTEST value
	IERR = 4	invalid TOLVAL value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.43. BRN_RATING_DEFAULT

Use this API to specify or return the current default branch rating name.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_BRN_RATING_DEFAULT IVAL
```

Python command syntax:

```
ierr = brn_rating_default(ival)

ierr, ival = brn_rating_default()
```

Fortran command syntax:

```
CALL BRN_RATING_DEFAULT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the current default branch rating name, from 1 to 12 (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
Integer IOCODE	indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL
	IOCODE = 1	return the current value of the option setting in IVAL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.44. BSNM

Use this API to change the bus numbers of specified network buses in the working case and retain a tabulation, in file form, of bus number changes made (activity BSNM). This API must be referenced with one OPT 0 reference followed by one of the following:
- one OPT 5, 11 or 12 reference.
- one or more OPT 1 references, followed by one OPT 7 reference.
- one or more OPT 2 references, followed by one OPT 7 reference.
- one or more OPT 3 references, followed by one OPT 7 reference.
- one or more OPT 4 references, followed by one OPT 7 reference.
- one OPT 15 reference for each area in the case containing at least one bus, in ascending area number order, followed by one OPT 25 reference.
- zero or more OPT 15 references in the same order as above, followed by one OPT 35 reference.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BSNM  SID  ALL  OPT  STATUS  BUSRNG(1)  BUSRNG(2)  OLDNAM  TFILE  OUT
 OFILE
```

Python command syntax:

```
ierr = bsnm(sid, all, opt, status, busrng, oldnam, tfile, out, ofile)
```

Fortran command syntax:

```
CALL BSNMNW(SID, ALL, OPT, STATUS, BUSRNG, OLDNAM, TFILE, OUT, OFILE,
 IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer OPT	method used to renumber the buses (input; no default allowed).	
	OPT = 0	initialize for renumbering.
	OPT = 1	renumber buses based on name/new number pairs.
	OPT = 11	renumber using name translation file.
	OPT = 2	renumber buses based on old number/new number pairs.
	OPT = 12	renumber buses using number translation file.
	OPT = 3	pack all buses in specified low/high pairs.
	OPT = 4	renumber buses based on the subsystem specified in the OPT 0 reference.
	OPT = 5	renumber all buses.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPT = 15	renumber all buses blocking by area.
	OPT = 25	apply blocking by area changes.
	OPT = 35	cancel blocking by area changes.
	OPT = 7	cleanup from renumbering.
Integer STATUS	If OPT=15, is the number of the area where the bus number range is specified in BUSRNG. STATUS must be between 1 and AREAS (the highest area number allowed at the present size level of PSSE). If OPT=4: STATUS = 0 indicates a new bus number range is provided. STATUS = 1 indicates a bus number offset is provided. Otherwise STATUS is ignored (input).	
Integer BUSRNG (2)	If OPT = 1: BUSRNG(1) is the new bus number. If OPT = 2: BUSRNG(1) is the old bus number. BUSRNG(2) is the new bus number. If OPT = 3: BUSRNG(1) is the bus number at which to start packing sequentially. BUSRNG(2) is the bus number at which to end packing sequentially. If OPT = 4 and STATUS = 0: BUSRNG(1) is the starting bus number for the range into which the subsystem is to be renumbered. BUSRNG(2) is the ending bus number for the range into which the subsystem is to be renumbered. If OPT = 4 and STATUS = 1: BUSRNG(1) is amount by which all buses in the specified subsystem are offset. If OPT = 5: BUSRNG(1) bus number at which to start renumbering all buses in the working case. Buses are re-sequenced in increments of 1. If OPT = 15: BUSRNG(1) is the starting bus number for the range into which the area is to be renumbered. BUSRNG(2) is the ending bus number for the range into which the area is to be renumbered (input).	
Character*18 OLDNAM	extended bus name of the bus to be renumbered when OPT is 1 (when OPT is 1) (input; no default allowed).	
Character*260 TFILE	bus name/number translation file. This file is only used if OPT = 11 or 12. Data in the translation file must be in the form specified in the PSSE Program Operation Manual. (when OPT is 11 or 12) (input; no default allowed).	
Integer OUT	option selection when OPT is 0 (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE.
	OUT = 1	direct output to the current report device.
	OUT = 2	no output.
Character*260 OFILE	Bus Renumbering Translation file for the records of activity BSNM (when OPT is 0 and OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OUT value
	IERR = 4	invalid OPT 0 call; bus renumbering already initialized
	IERR = 5	prior OPT is not 0 (OPT=1,2,11,12,5,25)
	IERR = 6	bus not found (OPT=1,2,11,12)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	invalid new bus number (OPT=1,2,11,12,4)
IERR = 8	bus number is already assigned to another bus (OPT=1,2,11,12,4)
IERR = 9	prior OPT not 0 or 3 (OPT=3,4,15,35)
IERR = 10	invalid starting bus number (OPT=3,4,5,15)
IERR = 11	invalid ending bus number (OPT=3,4,15)
IERR = 12	starting bus number is greater than ending bus number (OPT=3,4,15)
IERR = 13	invalid STATUS value (OPT=4)
IERR = 14	invalid offset value (OPT=4)
IERR = 15	not enough unused bus numbers in specified bus number range (OPT=4)
IERR = 16	not enough numbers between BUSRNG(1) and 999997 (OPT=5)
IERR = 17	area specified but no more areas left to process (OPT=15)
IERR = 18	area out of sequence (OPT=15)
IERR = 19	bus number range of an area overlaps that of another area (OPT=15)
IERR = 20	bus number range of an area is not large enough (OPT=15)
IERR = 21	not enough OPT 15 calls (OPT=25)
IERR = 22	invalid OPT 7 call; bus renumbering is not initialized (OPT=7)
IERR = 23	invalid OPT value
IERR = 24	error opening TFILE (OPT=11,12)
IERR = 25	error opening OFILE
IERR = 26	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.45. BUSN

Use this API to tabulate unused bus numbers within a specified bus number range (activity BUSN).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BUSN BUSLO BUSHI
```

Python command syntax:

```
ierr = busn(buslo, bushi)
```

Fortran command syntax:

```
CALL BUSNAPI(BUSLO, BUSHI, IERR)
```

Where:

Integer BUSLO	low limit of bus number range (input; 1 by default).	
Integer BUSHI	high limit of bus number range (input; 999,997).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid starting bus number
	IERR = 2	starting bus number is greater than ending bus number
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.46. BUS_INPUT

Use this API to specify or return the bus input option setting, for either numbers or names.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_BUS_INPUT IVAL
```

Python command syntax:

```
ierr = bus_input(ival)

ierr, ival = bus_input()
```

Fortran command syntax:

```
CALL BUS_INPUT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	bus numbers.
	IVAL = 1	extended bus names.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.47. BUS_OUTPUT

Use this API to specify or return the bus output option setting, for either numbers or names.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_BUS_OUTPUT IVAL
```

Python command syntax:

```
ierr = bus_output(ival)

ierr, ival = bus_output()
```

Fortran command syntax:

```
CALL BUS_OUTPUT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	bus numbers.
	IVAL = 1	extended bus names.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.48. BUS_SIZE_LEVEL

Use this API to specify or return the value of the PSSE size level option setting to a multiple of 1,000 between 1,000 and 200,000.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_BUS_SIZE_LEVEL IVAL
```

Python command syntax:

```
ierr = bus_size_level(ival)

ierr, ival = bus_size_level()
```

Fortran command syntax:

```
CALL BUS_SIZE_LEVEL_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1000 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met
	IERR = 4	unable to change bus size. PSSE Xplore is limited to 50 buses.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.49. CASE

Use this API to open a PSSE Saved Case file and transfers its data into the PSSE working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CASE  SFILE
```

Python command syntax:

```
ierr = case(sfile)
```

Fortran command syntax:

```
CALL CASENW(SFILE, IERR)
```

Where:

Character*260 SFILE	name of the PSSE saved case file If SFILE is "*", CASENW attempts to reopen the most recently accessed Saved Case File. If no Saved Case File was accessed in the current execution of PSSE, CASENW returns an error code (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SFILE is blank
	IERR = 2	error reading from SFILE
	IERR = 3	error opening SFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.50. CA_ITERATIONS

Use this API to specify or return the contingency iterations limit setting.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_CA_ITERATIONS IVAL
```

Python command syntax:

```
ierr = ca_iterations(ival)

ierr, ival = ca_iterations()
```

Fortran command syntax:

```
CALL CA_ITERATIONS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 20 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.51. CHECKVOLTAGELIMITS

Use this API to tabulate those buses where voltage magnitude is beyond their normal or emergency voltage limits. This check may be performed for all buses in the working case or for all buses in a specified subsystem.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_CHECKVOLTAGELIMITS SID ALL OPT THRSH
```

Python command syntax:

```
ierr = checkvoltagelimits(sid, all, opt, thrsh)
```

Fortran command syntax:

```
CALL CHECKVOLTAGELIMITS(SID, ALL, OPT, THRSH, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	set of voltage limits to be used (input; 1 by default).	
	OPT = 1	normal limits
	OPT = 2	emergency limits
Real THRSH	voltage limit threshold (-0.05 < THRSH < 0.05) (input0.0).	
	THRSH = 0.0	log voltages above upper limit or below lower limit
	THRSH > 0.0	log voltages beyond or within THRSH of a limit
	THRSH < 0.0	log voltage outside a limit by more than (-THRSH)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	invalid THRSH value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.52. CHECK_POWERFLOW_DATA

Use this API routine to perform data checks on the selected categories of powerflow data for all buses in the working case or for all buses in a specified subsystem.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_CHECK_POWERFLOW_DATA SID ALL OPT
```

Python command syntax:

```
ierr = check_powerflow_data(sid, all, opt)
```

Fortran command syntax:

```
CALL CHECK_POWERFLOW_DATA(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	powerflow data category to be checked (input; 0 by default).	
	OPT = 0	all data categories
	OPT = 1	bus data
	OPT = 2	load data
	OPT = 3	plant data
	OPT = 4	generator unit data
	OPT = 5	induction machine data
	OPT = 6	fixed bus shunt data
	OPT = 7	switched shunt data
	OPT = 8	non-transformer branch data
	OPT = 9	two-winding transformer data
	OPT = 10	three-winding transformer data
	OPT = 11	transformer impedance table correction data
	OPT = 12	multi-section line data
	OPT = 13	two-terminal dc line data
	OPT = 14	multi-terminal dc line data
	OPT = 15	VSC dc line data
	OPT = 16	FACTS device data
	OPT = 17	GNE device data
	OPT = 18	area interchange data
	OPT = 19	owner data

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPT = 20	zone data
	OPT = 21	capability curve data
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	capability curve data is not in the working case
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.53. CHKCNTDUPLICON

Check duplicate labels in Contingency (CON) file.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_CHKCNTDUPLICON CONFILE
```

Python command syntax:

```
ierr, dnum = chkcntduplicon(confile)
```

Fortran command syntax:

```
CALL CHKCNTDUPLICON(CONFILE, DNUM, IERR)
```

Where:

Character*260 CONFILE	contingency file name (input).	
integer DNUM	total number of duplicated contingency labels in the contingency file (output; 0 by default).	
Integer IERR	error code (output).	
	IERR = 1	no CON file specified
	IERR = 2	error opening CON file
	IERR = 3	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.54. CHKCNTDUPLIDFX

Check duplicate labels in DFAX file.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_CHKCNTDUPLIDFX DFXFILE
```

Python command syntax:

```
ierr, dnum = chkcntduplidfx(dfxfile)
```

Fortran command syntax:

```
CALL CHKCNTDUPLIDFX(DFXFILE, DNUM, IERR)
```

Where:

Character*260 DFXFILE	contingency file name (input).	
integer DNUM	total number of duplicated contingency labels in the dfax file (output; 0 by default).	
Integer IERR	error code (output).	
	IERR = 1	no DFAX file specified
	IERR = 2	error opening DFAX file
	IERR = 3	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.55. CLOSE_POWERFLOW

Removes the current powerflow working case from PSSE's working memory. Thus, the PSSE powerflow arrays are in the same state as they are immediately following PSSE startup. The default PSSE program option settings are unchanged.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CLOSE_POWERFLOW
```

Python command syntax:

```
ierr = close_powerflow()
```

Fortran command syntax:

```
CALL CLOSE_POWERFLOW(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.56. CLOSE_REPORT

Sets the report output device to the standard output and sets the command line to request device selection for individual reporting activities (Activity CLOS).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_CLOSE_REPORT
```

Python command syntax:

```
ierr = close_report()
```

Fortran command syntax:

```
CALL CLOSEREPORTDEVICE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.57. CMPR

Use this API to tabulate certain case totals, as contained in the working case, with those of a designated Saved Case (activity CMPR). Either system totals, area totals, owner totals, or zone totals may be compared. It must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by any number of references with APIOPT=2, followed by one reference with APIOPT=3.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference, followed by one APIOPT 3 reference.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CMPR SID APIOPT STATUS(1)..STATUS(4) THRSH CFILE
```

Python command syntax:

```
ierr = cmpr(sid, apiopt, status, thrsh, cfile)
```

Fortran command syntax:

```
CALL CMPRNW(SID, APIOPT, STATUS, THRSH, CFILE, IERR)
```

Where:

Integer SID	a valid area, zone or owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; used only when APIOPT is 0 or 1; 0).	
Integer APIOPT	mode of operation in the API (input; 0 by default).	
	APIOPT = 0	initialize for case totals comparison, run the comparison specified by STATUS(3), and run post-processing clean up.
	APIOPT = 1	initialize system for system, area, owner, zone totals.
	APIOPT = 2	calculate and report system, area, owner, zone totals.
	APIOPT = 3	postprocessing housekeeping.
Integer STATUS (4)	array of 4 elements specifying The value of each element is as follows(input).	
	STATUS(1)	tabulation method (used only when APIOPT is 0 or 1; 0 by default)
	STATUS(1) = 0	tabulate system totals
	STATUS(1) = 1	tabulate area totals for area subsystem SID
	STATUS(1) = 2	tabulate owner totals for owner subsystem SID
	STATUS(1) = 3	tabulate zone totals for zone subsystem SID
	STATUS(2)	units for the difference threshold (used only when APIOPT is 0 or 2; 0 by default)
	STATUS(2) = 0	threshold in engineering units

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(2) = 1	threshold in percent
	STATUS(3)	quantity to be compared (used only when APIOPT is 0 or 2; 1 by default)
	STATUS(3) = 1	compare generation
	STATUS(3) = 2	compare load
	STATUS(3) = 3	compare losses
	STATUS(3) = 4	compare mismatch
	STATUS(3) = 5	compare interchange; not valid for STATUS(1) = 2
	STATUS(4)	file to restore into the working case upon completion (used only when APIOPT is 0 or 3; 0 by default)
	STATUS(4) = 0	restore the working case
	STATUS(4) = 1	restore the comparison file to the working case
Real THRSH	difference threshold, in either engineering units or a percent, based on the value of STATUS(2) (input; used only when APIOPT is 0 or 2; 0.0).	
Character*260 CFILE	Saved Case file to be compared against the working case (when APIOPT is 0 or 1) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid APIOPT value
	IERR = 3	invalid STATUS value
	IERR = 4	owner (STATUS(1)=2) interchange (STATUS(3)=5) combination is invalid
	IERR = 5	invalid THRSH value
	IERR = 6	CFILE is blank
	IERR = 7	error writing the working case to a temporary file
	IERR = 8	unexpected APIOPT value
	IERR = 9	error reading the comparison case
	IERR = 10	error opening the temporary file
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.58. CNTB

Use this API to tabulate the voltage setpoints and desired voltage bands of voltage-controlling equipment in the working case (activity CNTB). Optionally, new scheduled voltages may be specified.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CNTB  SID  ALL  APIOPT  STATUS(1)..STATUS(3)  V(1)  V(2)
```

Python command syntax:

```
ierr, next, bus, ckt, v = cntb(sid, all, apiopt, status, v)
```

Fortran command syntax:

```
CALL CNTBNW(SID, ALL, APIOPT, STATUS, NEXT, BUS, CKT, V, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (used only when APIOPT is 2) (input; 0 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer APIOPT	mode of operation in the API (input).	
	APIOPT = 1	initialize.
	APIOPT = 2	specify voltage setpoint for controlled bus.
	APIOPT = 3	specify voltage band for load drop compensating transformer.
Integer STATUS (3)	array of 3 elements specifying used to control reporting and processing options(input).	
	STATUS(1)	varies by APIOPT
	STATUS(1) = 0	For APIOPT=1, list subsystem controlled buses with conflicts. For APIOPT=2 or 3, no change
	STATUS(1) = 1	For APIOPT=1, list all subsystem controlled buses. For APIOPT=2, change voltage setpoints to new setpoint in V(1). For APIOPT=3, change voltage band to new voltage band in V(1) and V(2)
	STATUS(2)	varies by APIOPT
	STATUS(2) = 0	For APIOPT=1, report at OPENed output device. For APIOPT=2 or 3, not used
	STATUS(2) = 1	For APIOPT=1, list in progress area and allow voltage schedule change. For APIOPT=2 or 3, not used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(3)	varies by APIOPT
	STATUS(3) = 0	For APIOPT=1, process all controlling elements and controlled buses. For APIOPT=2 or 3, not used
	STATUS(3) = 1	For APIOPT=1, process active controlling elements and controlled buses. For APIOPT=2 or 3, not used
Integer NEXT	returned as the next APIOPT value expected by the API (output).	
	NEXT = 1	done with this execution of CNTBNW.
	NEXT = 2	change voltage setpoint/schedule.
	NEXT = 3	change VMIN, VMAX of load drop compensating transformer.
Integer BUS (3)	array of 3 elements specifying a returned three bus values(output).	
	BUS(1)	Returned as controlled bus for which voltage schedule may be changed when Next=2. Returned as controlled bus for which voltage schedule may be changed when Next=3
	BUS(2)	Returned as the from bus of the transformer when Next=3
	BUS(3)	Returned as the to bus of the transformer when Next=3
Character*2 CKT	returned as the circuit ID of the transformer returned in BUS(2) and BUS(3) when NEXT returned as 3 (output).	
Real V (2)	array of 2 elements specifying Voltage values(input).	
	V(1)	New setpoint when APIOPT=2, new VMIN when APIOPT=3. Returned as present setpoint when NEXT=2, returned as present VMIN when NEXT=3
	V(2)	New VMAX when APIOPT=3. Returned as present VMAX when NEXT=3
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid new voltage setpoint value (V(1) <= 0.5 or V(1) >= 1.5)
	IERR = 5	VMIN is not less than VMAX
	IERR = 6	invalid APIOPT value
	IERR = 7	unexpected APIOPT value
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.59. CONG

Use this API to convert generators from their power flow representation in preparation for switching studies and dynamic simulations (activity CONG).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CONG  OPT
```

Python command syntax:

```
ierr = cong(opt)
```

Fortran command syntax:

```
CALL CONGAPI(OPT, IERR)
```

Where:

Integer OPT	machine impedance option (input; 0 by default).	
	OPT = 0	use ZSORCE and IMZSRC.
	OPT = 1	use fault analysis subtransient impedance and IMZPOS.
	OPT = 2	use fault analysis transient impedance and IMZPOS.
	OPT = 3	use fault analysis synchronous impedance and IMZPOS.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	generators are already converted
	IERR = 3	generator conversion failed due to a machine impedance of zero
	IERR = 4	generator conversion failed due to the presence of in-service induction machines that are in the "stalled" or "tripped" state
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.60. CONL

Use this API to convert the constant MVA load for a specified grouping of network loads to a specified mixture of the constant MVA, constant current, and constant admittance load characteristics (activity CONL).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CONL  SID  ALL  APIOPT  STATUS(1)  STATUS(2)  LOADIN(1)..  LOADIN(2)
 LOADIN(3)  LOADIN(4)
```

Python command syntax:

```
ierr, rlods = conl(sid, all, apiopt, status, loadin)
```

Fortran command syntax:

```
CALL CONLNW(SID, ALL, APIOPT, STATUS, LOADIN, RLODS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; used only when APIOPT is 2; 0).	
Integer ALL	all buses or specified subsystem flag (input; used only when APIOPT is 2; 0).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer APIOPT	mode of operation in the API (input; no default allowed).	
	APIOPT = 1	initialize for load conversion.
	APIOPT = 2	convert loads.
	APIOPT = 3	postprocessing housekeeping.
Integer STATUS (2)	array of 2 elements specifying The value of each element is as follows(input).	
	STATUS(1)	method of conversion (used only when APIOPT is 1; 0 by default)
	STATUS(1) = 0	convert existing constant MVA load
	STATUS(1) = 1	reconstruct loads using present voltage, then convert
	STATUS(1) = 2	reconstruct loads using unity voltage, then convert
	STATUS(2)	a special OPF bus type code flag (used only when APIOPT is 2; 0 by default)
	STATUS(2) = 0	all buses in subsystem
	STATUS(2) = 1	only Type 1 buses in subsystem
	STATUS(2) = 2	only Type 2 and 3 buses in subsystem
Real LOADIN (4)	array of 4 elements specifying (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LOADIN(1)	percent of active power load to be converted to the constant current characteristic
	LOADIN(2)	percent of active power load to be converted to the constant admittance characteristic
	LOADIN(3)	percent of reactive power load to be converted to the constant current characteristic
	LOADIN(4)	percent of reactive power load to be converted to the constant admittance characteristic
Integer RLODS	returned as the number of unconverted loads remaining in the working case. If the value is nonzero after a call with APIOPT = 2, and the remaining loads are to be converted, the API is called again with ALL = 1 and APIOPT = 2 (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid APIOPT value
	IERR = 4	invalid STATUS value
	IERR = 5	unexpected APIOPT value
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.61. CONNECTIVITY_CHECK

Use this API to specify or return the option to enable or disable the solution connectivity checking option setting.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_CONNECTIVITY_CHECK IVAL
```

Python command syntax:

```
ierr = connectivity_check(ival)

ierr, ival = connectivity_check()
```

Fortran command syntax:

```
CALL CONNECTIVITY_CHECK_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	no solution connectivity check before power flow solutions.
	IVAL = 1	run solution connectivity check before each power flow solution.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.62. CONTROL_AREA_INTERCHANGE

Use this API to to specify or return the area interchange control option setting to disabled, or enabled with tie lines only, or enabled with tie lines and loads.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_CONTROL_AREA_INTERCHANGE IVAL
```

Python command syntax:

```
ierr = control_area_interchange(ival)

ierr, ival = control_area_interchange()
```

Fortran command syntax:

```
CALL CONTROL_AREA_INTERCHANGE_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	disabled.
	IVAL = 1	enabled with interchange determined from tie lines only.
	IVAL = 2	enabled with interchange determined from tie lines and loads.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.63. CSV_TO_RAWX

Use this API to import a rawx (extended raw data) data table from a CSV (comma-separated values) file.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_CSV_TO_RAWX
```

Python command syntax:

```
ierr = csv_to_rawx(tblname, sfile)
```

Fortran command syntax:

```
CALL CSV_TO_RAWX(TBLNAME, SFILE, IERR)
```

Where:

Character*100 TBLNAME	name of the data table to import into (not case sensitive) (input; no default allowed).	
Character*260 SFILE	file path of the CSV input file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid tblname
	IERR = 2	error opening SFILE
	IERR = 3	error generating RAWX data
	IERR = 4	error reading RAWX data
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.64. CSV_TO_RAWX_WITH_METAMODEL

Use this API to import a rawx (extended raw data) data table from a CSV (comma-separated values) file with metamodel.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_CSV_TO_RAWX_WITH_METAMODEL INFILE OUTFILE DOMAINNAME ELEMENTNAME
```

Python command syntax:

```
ierr = csv_to_rawx_with_metamodel(infile, outfile, domainname,
 elementname)
```

Fortran command syntax:

```
CALL CSV_TO_RAWX_WITH_METAMODEL(INFILE, OUTFILE, DOMAINNAME, ELEMENTNAME,
 IERR)
```

Where:

Character*260 INFILE	name of the input file (input; no default allowed).	
Character*260 OUTFILE	name of the output file (input; no default allowed).	
Character*260 DOMAINNAME	file path of the CSV input file (input; no default allowed).	
Character*260 ELEMENTNAME	file path of the CSV input file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 1	Error opening "infile"
	IERR = 2	Empty "outfile"
	IERR = 10	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.65. DCCC

This API routine is obsolete. It has been replaced by the API routine dccc_2 .

Use this API to run contingency case solutions using a linear network (dc) model (activity DCCC).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_DCCC  OPTIONS(1)..OPTIONS(6)  VALUES(1)..VALUES(3)  DFXFILE
```

Python command syntax:

```
ierr = dccc(options, values, dfxfile)
```

Fortran command syntax:

```
CALL DCCCAPI(OPTIONS, VALUES, DFXFILE, IERR)
```

Where:

Integer OPTIONS (6)	array of 6 elements specifying calculation and reporting options. The value of each element is as follows(input).	
	OPTIONS(1)	contingency case rating set (rating set program option setting by default)
	OPTIONS(1) = 1	RATEA
	OPTIONS(1) = 2	RATEB
	OPTIONS(1) = 3	RATEC
	OPTIONS(2)	base case line flow code (0 by default)
	OPTIONS(2) = 0	dc base case
	OPTIONS(2) = 1	ac base case
	OPTIONS(3)	format code (0 by default)
	OPTIONS(3) = 0	overload output
	OPTIONS(3) = 1	full output
	OPTIONS(4)	exclude cases with no overloads from the overload report (0 by default)
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
	OPTIONS(5)	convert MVA ratings to estimated MW ratings (0 by default)
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
	OPTIONS(6)	code to discontinue activity if MW mismatch exceeds TOLS(1) (0 by default)
	OPTIONS(6) = 0	discontinue
	OPTIONS(6) = 1	continue
Real VALUES (3)	array of 3 elements specifying tolerances and thresholds. The value of each element is as follows(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(1)	required MW mismatch tolerance for continuing (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating for reporting an element as overloaded (100.0 by default)
	VALUES(3)	minimum contingency case flow change for including an element in the overload report (0.0 by default)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	no Distribution Factor Data File specified
	IERR = 4	generators are converted
	IERR = 5	buses in island(s) without a swing bus; use activity TREE
	IERR = 6	largest mismatch exceeds mismatch tolerance
	IERR = 7	error opening file DFXFILE
	IERR = 8	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 9	file DFXFILE does not contain distribution factors; run DFAX with distribution factor option enabled
	IERR = 10	no monitored branches or interfaces defined
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.66. DCCC_2

Use this API to run contingency case solutions using a linear network (dc) model (activity DCCC).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_DCCC_2  OPTIONS(1)..OPTIONS(8) VALUES(1)..VALUES(3) DFXFILE
```

Python command syntax:

```
ierr = dccc_2(options, values, dfxfile)
```

Fortran command syntax:

```
CALL DCCC_2 (OPTIONS, VALUES, DFXFILE, IERR)
```

Where:

Integer OPTIONS (8) — array of elements specifying calculation and reporting options. The value of each element is as follows.(input).

OPTIONS(1)	is the loss estimate option (1 by default)
OPTIONS(1) = 0	exclude loss estimate
OPTIONS(1) = 1	use the present voltage vector to estimate losses and apply them as "hidden loads".
OPTIONS(2)	is the regulating phase shifter code. (0 by default)
OPTIONS(2) = 0	fixed phase shift angles in the base case and the contingency cases.
OPTIONS(2) = 1	fixed MW in the base case, fixed phase shift angles in the contingency cases.
OPTIONS(2) = 2	fixed MW in the base case and the contingency cases.
OPTIONS(3)	contingency case rating set; 1 through 12 (rating set program option setting by default)
OPTIONS(4)	base case line flow code (0 by default)
OPTIONS(4) = 0	dc base case.
OPTIONS(4) = 1	ac base case.
OPTIONS(5)	output format code (0 by default)
OPTIONS(5) = 0	overload output.
OPTIONS(5) = 1	full output.
OPTIONS(6)	exclude cases with no overloads from the overload report (0 by default)
OPTIONS(6) = 0	no.
OPTIONS(6) = 1	yes.
OPTIONS(7)	convert MVA ratings to estimated MW ratings. (0 by default)
OPTIONS(7) = 0	no.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(7) = 1	yes.
	OPTIONS(8)	code to discontinue activity if MW mismatch exceeds VALUES(1). (0 by default)
	OPTIONS(8) = 0	discontinue.
	OPTIONS(8) = 1	continue.
Real VALUES (3)	array of elements specifying tolerances and thresholds. The value of each element is as follows.(input).	
	VALUES(1)	required MW mismatch tolerance for continuing (Newton solution convergence tolerance, TOLN, by default)
	VALUES(2)	percent of rating for reporting an element as overloaded. (100 by default)
	VALUES(3)	minimum contingency case flow change for including an element in the overload report. (0.0 by default)
Character*260 DFXFILE	Is the name of Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	no Distribution Factor Data File specified
	IERR = 4	generators are converted
	IERR = 5	buses in island(s) without a swing bus; use activity TREE
	IERR = 6	largest mismatch exceeds mismatch tolerance
	IERR = 7	error opening DFXFILE
	IERR = 8	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
	IERR = 9	file DFXFILE does not contain distribution factors; run DFAX with distribution factor option enabled.
	IERR = 10	no monitored branches or interfaces defined.
	IERR = 11	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.67. DCCOR

Use this API to apply corrective actions to the base case using linear programming methods. The corrective actions are to eliminate flow overloads with the objective of minimizing the control adjustments, including load shedding, MW generations dispatch, and phase shifter angle adjustments.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_DCCOR  OPTIONS(1)..OPTIONS(7)  VALUES(1)..VALUES(6)
 LABELS(1)..LABELS(3)  DFXFILE
```

Python command syntax:

```
ierr = dccor(options, values, labels, dfxfile)
```

Fortran command syntax:

```
CALL DCCORAPI(OPTIONS, VALUES, LABELS, DFXFILE, IERR)
```

Where:

Integer OPTIONS (7) array of 7 elements specifying DC corrective actions solution option-s(input).

OPTIONS(1)	rating set (rating set program option setting by default)
OPTIONS(1) = 1	rating A
OPTIONS(1) = 2	rating B
OPTIONS(1) = 3	rating C
OPTIONS(2)	generator control flag (1 by default)
OPTIONS(2) = 0	disable
OPTIONS(2) = 1	enable
OPTIONS(3)	load control flag (0 by default)
OPTIONS(3) = 0	disable
OPTIONS(3) = 1	enable
OPTIONS(4)	phase shifter control flag (1 by default)
OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	convert MVA ratings to estimated MW ratings (0 by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(6)	code to discontinue activity if MW mismatch exceeds tolerance (VALUES(1) (0 by default)
OPTIONS(6) = 0	disable (i.e. continue, do not discontinue activity)
OPTIONS(6) = 1	enable (i.e. discontinue)
OPTIONS(7)	modify network data with corrective actions solutions (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(7) = 0	disable (i.e. do not modify network data)
	OPTIONS(7) = 1	enable (i.e. allow network data to be modified)
Real VALUES (6)	array of 6 elements specifying DC corrective actions parameters(input).	
	VALUES(1)	MW mismatch tolerance for continuing (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating set (100.0 by default)
	VALUES(3)	branch flow overload tolerance (0.1 by default)
	VALUES(4)	generator control weighting factor (1.0 by default)
	VALUES(5)	load control weighting factor (1.0 by default)
	VALUES(6)	phase shifter control weighting factor (1.0 by default)
Character*32 LABELS (3)	array of 3 elements specifying subsystem labels(input).	
	LABELS(1)	label of generator control subsystem (blank by default; no default allowed if OPTIONS(2) is 1)
	LABELS(2)	label of load control subsystem (blank by default; no default allowed if OPTIONS(3) is 1)
	LABELS(3)	label of phase shifter control subsystem (blank by default; no default allowed if OPTIONS(4) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	No controls
	IERR = 6	control subsystem not defined
	IERR = 7	Distribution Factor Data File not specified
	IERR = 8	largest mismatch exceeds tolerance
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 51	error opening Distribution Factor Data File
	IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.68. DCCOR_2

Use this API to apply corrective actions to the base case using linear programming methods. The corrective actions are to eliminate flow overloads with the objective of minimizing the control adjustments, including load shedding, MW generations dispatch, and phase shifter angle adjustments.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_DCCOR_2  OPTIONS(1)..OPTIONS(7)  VALUES(1)..VALUES(6)
 LABELS(1)..LABELS(3)  DFXFILE
```

Python command syntax:

```
ierr = dccor_2(options, values, labels, dfxfile)
```

Fortran command syntax:

```
CALL DCCORAPI_2(OPTIONS, VALUES, LABELS, DFXFILE, IERR)
```

Where:

Integer OPTIONS (8)	array of 8 elements specifying DC corrective actions solution options(input).	
	OPTIONS(1)	rating set (rating set program option setting by default)
	OPTIONS(1) = 1	rating A
	OPTIONS(1) = 2	rating B
	OPTIONS(1) = 3	rating C
	OPTIONS(2)	generator control flag (1 by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable
	OPTIONS(3)	load control flag (0 by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	phase shifter control flag (1 by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	off-line generator control flag (1 by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(6)	convert MVA ratings to estimated MW ratings (0 by default)
	OPTIONS(6) = 0	disable
	OPTIONS(6) = 1	enable
	OPTIONS(7)	code to discontinue activity if MW mismatch exceeds tolerance (VALUES(1) (0 by default)
	OPTIONS(7) = 0	disable (i.e. continue, do not discontinue activity)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(7) = 1	enable (i.e. discontinue)
	OPTIONS(8)	modify network data with corrective actions solutions (1 by default)
	OPTIONS(8) = 0	disable (i.e. do not modify network data)
	OPTIONS(8) = 1	enable (i.e. allow network data to be modified)
Real VALUES (7)	array of 7 elements specifying DC corrective actions parameters(input).	
	VALUES(1)	MW mismatch tolerance for continuing (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating set (100.0 by default)
	VALUES(3)	branch flow overload tolerance (0.1 by default)
	VALUES(4)	generator control weighting factor (1.0 by default)
	VALUES(5)	load control weighting factor (1.0 by default)
	VALUES(6)	phase shifter control weighting factor (1.0 by default)
	VALUES(7)	off-line generator control weighting factor (1.0 by default)
Character*32 LABELS (4)	array of 4 elements specifying subsystem labels(input).	
	LABELS(1)	label of generator control subsystem (blank by default; no default allowed if OPTIONS(2) is 1)
	LABELS(2)	label of load control subsystem (blank by default; no default allowed if OPTIONS(3) is 1)
	LABELS(3)	label of phase shifter control subsystem (blank by default; no default allowed if OPTIONS(4) is 1)
	LABELS(4)	label of off-line generator control subsystem (blank by default; no default allowed if OPTIONS(5) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	No controls
	IERR = 6	control subsystem not defined
	IERR = 7	Distribution Factor Data File not specified
	IERR = 8	largest mismatch exceeds tolerance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Distribution Factor Data File
IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.69. DCLF

This API routine is obsolete. It has been replaced by the API routine dclf_2 .

Use this API to apply the dc analogy network solution algorithm to the network modeled in the working case (activity DCLF).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_DCLF  SID  ALL  STATUS(1)..STATUS(4)  BRNCH(1)..BRNCH(3) CKT
```

Python command syntax:

```
ierr = dclf(sid, all, status, brnch, ckt)
```

Fortran command syntax:

```
CALL DCLFAPI(SID, ALL, STATUS, BRNCH, CKT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer STATUS (4)	array of 4 elements specifying The value of each element is as follows(input).	
	STATUS(1)	rating used in tabulating the report (default rating set program option setting)
	STATUS(1) = 1	RATEA
	STATUS(1) = 2	RATEB
	STATUS(1) = 3	RATEC
	STATUS(2)	voltage that should be saved (0 by default)
	STATUS(2) = 0	original voltages
	STATUS(2) = 1	base case dc power flow voltages
	STATUS(2) = 2	change case dc power flow voltages
	STATUS(3)	status of the changed branch upon completion (0 by default)
	STATUS(3) = 0	leave unchanged
	STATUS(3) = 1	save changed status
	STATUS(4)	action code if the MW mismatch exceeds 0.5 MW (1 by default)
	STATUS(4) = 0	discontinue
	STATUS(4) = 1	continue

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCH (3)	array of 3 elements specifying The use of each element is as follows. The values can all be zero or defaulted if no branch is to be changed for the solution(input).
	BRNCH(1) from bus of the branch for which status is to be changed for the solution (0 by default)
	BRNCH(2) to bus of the branch for which status is to be changed for the solution (0 by default)
	BRNCH(3) third bus connected by a three-winding transformer, or zero for a nontransformer branch or a two-winding transformer (0 by default)
Character*2 CKT	circuit ID of the branch for which status is to be changed for the solution (input; '1' by default).
Integer IERR	error code (output).
	IERR = 0 no error occurred
	IERR = 1 invalid SID value or subsystem SID is not defined
	IERR = 2 invalid ALL value
	IERR = 3 invalid STATUS value
	IERR = 4 bus not found
	IERR = 5 branch not found
	IERR = 6 three-winding transformer not found
	IERR = 7 multi-section line specified but multi-section line reporting option is disabled
	IERR = 8 closing a multi-section line is invalid in DCLF
	IERR = 9 returning a Type 4 bus to service is not allowed in DCLF
	IERR = 10 closing a three-winding transformer is not allowed in DCLF
	IERR = 11 generators are converted
	IERR = 12 buses in island(s) without a swing bus; use activity TREE
	IERR = 13 largest mismatch exceeds 0.5 MW
	IERR = 14 prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.70. DCLF_2

Use this API routine to apply the dc analogy network solution algorithm to the network modeled in the working case (activity DCLF).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_DCLF_2  SID ALL STATUS(1)..STATUS(6) BRNCH(1)..BRNCH(3) CKT
```

Python command syntax:

```
ierr = dclf_2(sid, all, status, brnch, ckt)
```

Fortran command syntax:

```
CALL DCLF_2(SID, ALL, STATUS, BRNCH, CKT, IERR)
```

Where:

Integer SID	Is a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined. (input; 1 by default).
Integer ALL	Is the all buses or specified subsystem flag. (input; 1 by default).
Integer STATUS (6)	array of elements specifying calculation options. The value of each element is as follows.(input).

STATUS(1)	is the loss estimate option. (1 by default)
STATUS(1) = 0	exclude loss estimate.
STATUS(1) = 1	use the present voltage vector to estimate losses and apply them as "hidden loads".
STATUS(2)	is the regulating phase shifter code (0 by default)
STATUS(2) = 0	fixed phase shift angles in the base case and the change case.
STATUS(2) = 1	fixed MW in the base case, fixed phase shift angles in the change case.
STATUS(2) = 2	fixed MW in the base case and the change case.
STATUS(3)	is rating used in tabulating the report; 1 through 12 (default rating set program option setting by default).
STATUS(4)	is the voltage that should be saved. (0 by default)
STATUS(4) = 0	original voltages.
STATUS(4) = 1	base case dc power flow voltages.
STATUS(4) = 2	change case dc power flow voltages.
STATUS(5)	is status of the changed branch upon completion (0 by default)
STATUS(5) = 0	leave unchanged.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 1	save changed status.
	STATUS(6)	is action code if the MW mismatch exceeds 0.5 MW. (1 by default)
	STATUS(6) = 0	discontinue.
	STATUS(6) = 1	continue.
Integer BRNCH (3)	array of elements specifying calculation options. The use of each element is as follows. The values can all be zero or defaulted if no branch is to be changed for the solution.(input).	
	BRNCH(1)	is the from bus of the branch for which status is to be changed for the solution (0 by default)
	BRNCH(2)	is the to bus of the branch for which status is to be changed for the solution (0 by default)
	BRNCH(3)	is the third bus connected by a three-winding transformer, or zero for a nontransformer branch or a two-winding transformer (0 by default)
Character*2 CKT	Is the circuit ID of the branch for which status is to be changed for the solution (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined.
	IERR = 2	invalid ALL value.
	IERR = 3	invalid STATUS value.
	IERR = 4	bus not found.
	IERR = 5	branch not found.
	IERR = 6	three-winding transformer not found.
	IERR = 7	multi-section line specified but multi-section line reporting option is disabled.
	IERR = 8	closing a multi-section line is invalid in DCLF.
	IERR = 9	returning a Type 4 bus to service is not allowed in DCLF.
	IERR = 10	closing a three-winding transformer is not allowed in DCLF.
	IERR = 11	generators are converted.
	IERR = 12	buses in island(s) without a swing bus; use activity TREE
	IERR = 13	largest mismatch exceeds 0.5 MW.
	IERR = 14	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.71. DCPSCOPF

This API is the DC based Preventive Security Constrained Optimal Power Flow solution (DCPSCOPF). This API is used to find the settings of controls for a system to comply with base case and contingency case operation limits. The API is implemented with DC preventive security constrained optimal power flow method (DCPSCOPF). The objective of the DCPSCOPF is to minimize control adjustments subject to operation limits constraints. The available controls are generation MW outputs of on-line and off-line generators, loads, phase shifters.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_DCPSCOPF  OPTIONS(1)..OPTIONS(7) VALUES(1)..VALUES(6)
 LABELS(1)..LABELS(5) DFXFILE INLFILE THRFILE
```

Python command syntax:

```
ierr = dcpscopf(options, values, labels, dfxfile, inlfile, thrfile)
```

Fortran command syntax:

```
CALL DCPSCOPFAPI(OPTIONS, VALUES, LABELS, DFXFILE, INLFILE, THRFILE, IERR)
```

Where:

Integer OPTIONS (7)	array of elements specifying power flow options(input).	
	OPTIONS(1)	dispatch mode (0 by default)
	OPTIONS(1) = 0	disable.
	OPTIONS(1) = 1	subsystem machines (RESERVE).
	OPTIONS(1) = 2	subsystem machines (PMAX).
	OPTIONS(1) = 3	subsystem machines (INERTIA).
	OPTIONS(1) = 4	subsystem machines (GOVERNOR DROOP).
	OPTIONS(2)	Ignore initial violations in the base case (1 by default)
	OPTIONS(2) = 0	No
	OPTIONS(2) = 1	Yes
	OPTIONS(3)	Abort if MW mismatch exceeds tolerance (0 by default)
	OPTIONS(3) = 0	No
	OPTIONS(3) = 1	Yes
	OPTIONS(4)	generator control flag (1 by default)
	OPTIONS(4) = 0	disable generation MW dispatch
	OPTIONS(4) = 1	enable
	OPTIONS(5)	load control flag (0 by default)
	OPTIONS(5) = 0	disable load shedding control
	OPTIONS(5) = 1	enable
	OPTIONS(6)	phase shifter control flag (1 by default)
	OPTIONS(6) = 0	disable phase shifter control
	OPTIONS(6) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(7)	off-line generator control flag (0 by default)
	OPTIONS(7) = 0	disable generation MW dispatch of off-line generators
	OPTIONS(7) = 1	enable
Real VALUES (6)	array of elements specifying power flow options(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Flow tolerance in linear programming
	VALUES(3)	Weighting factor of generation MW dispatch (1.0 by default)
	VALUES(4)	Weighting factor of load control (1.0 by default)
	VALUES(5)	Weighting factor of phase shifter control (1.0 by default)
	VALUES(6)	Weighting factor of generation MW dispatch of off-line generators (1.0 by default)
Character*32 LABELS (5)	array of elements specifying labels(input).	
	LABELS(1)	The label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation output (blank for none).
	LABELS(2)	The label of generation dispatch control system. MW injections of the generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(3)	The label of load control system. Loads in the load control system are available for curtailment in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(4)	The label of phase shifter adjustment system. The phase angles of phase shifters in the system are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
	LABELS(5)	The label of generation MW control of off-line generator control system. MW injections of the off-line generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 INLFILE	Is the name of Load Throwover Data File (input, blank for none). (input).	
Character*260 THRFILE	Is the name of Unit Inertia Data File (input, blank for none). (input).	
Integer IERR	error code (output).	
	IERR = -1	PSCOPF terminates with active constraints
	IERR = 0	No error
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	Generation subsystem not specified
	IERR = 7	too many islands in base case
	IERR = 8	buses with bus type code and series element status inconsistencies
	IERR = 9	no Distribution Factor Data File specified
	IERR = 10	No controls are specified
	IERR = 11	No constraints are specified
	IERR = 12	Simulation STOP. Duplicated contingency labels in DFAX file
	IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 51	error opening Distribution Factor Data File
	IERR = 52	error opening Load Throwover Data File
	IERR = 53	error opening Unit Inertia and Governor Data File
	IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.72. DC_TAP_ADJUSTMENT

Use this API to specify or return the option to enable or disable the dc tap adjustment option setting.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_DC_TAP_ADJUSTMENT IVAL
```

Python command syntax:

```
ierr = dc_tap_adjustment(ival)

ierr, ival = dc_tap_adjustment()
```

Fortran command syntax:

```
CALL DC_TAP_ADJUSTMENT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	disabled.
	IVAL = 1	enabled.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.73. DELTMPFILES

Delete closed temporary files. PSSE creates a subdirectory within the user's "temporary" directory; only files there are removed.

This API routine was first introduced in release 32.2.0.

Batch command syntax:

```
BAT_DELTMPFILES
```

Python command syntax:

```
ierr = deltmpfiles()
```

Fortran command syntax:

```
CALL DELTMPFILES(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 1	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.74. DFAX

This API routine is obsolete. It has been replaced by the API routine dfax_2 .

Use this API to construct a Distribution Factor Data File (activity DFAX).

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_DFAX  OPTIONS(1)  OPTIONS(2)  SUBFILE  MONFILE  CONFILE  DFXFILE
```

Python command syntax:

```
ierr = dfax(options, subfile, monfile, confile, dfxfile)
```

Fortran command syntax:

```
CALL DFAXAPI(OPTIONS, SUBFILE, MONFILE, CONFILE, DFXFILE, IERR)
```

Where:

Integer OPTIONS (2)	array of 2 elements specifying calculation options. The value of each element is as follows(input).
	OPTIONS(1) — distribution factor option flag (1 by default)
	OPTIONS(1) = 0 — do not calculate distribution factors (i.e., DFAX,AC)
	OPTIONS(1) = 1 — calculate distribution factors
	OPTIONS(2) — monitored element sorting flag (0 by default)
	OPTIONS(2) = 0 — do not sort (i.e., leave in Monitored Element Description File order)
	OPTIONS(2) = 1 — sort
Character*260 SUBFILE	name of the Subsystem Description File; blank for none (input; blank by default).
Character*260 MONFILE	name of Monitored Element Description File (input; no default allowed).
Character*260 CONFILE	name of Contingency Description Data File (input; no default allowed).
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).
Integer IERR	error code (output).
	IERR = 0 — no error occurred
	IERR = 1 — invalid OPTIONS value
	IERR = 2 — generators are converted
	IERR = 3 — buses in island(s) without a swing bus; use activity TREE
	IERR = 4 — no Distribution Factor Data File specified
	IERR = 5 — no Monitored Element Data input file specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no Contingency Description Data file specified
IERR = 7	fatal error reading input file
IERR = 8	error opening output file DFXFILE
IERR = 9	error opening input file SUBFILE
IERR = 10	error opening input file MONFILE
IERR = 11	error opening input file CONFILE
IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.75. DFAX_2

Use this API to construct a Distribution Factor Data File (activity DFAX).

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_DFAX_2  OPTIONS(1)..OPTIONS(3) SUBFILE MONFILE CONFILE DFXFILE
```

Python command syntax:

```
ierr = dfax_2(options, subfile, monfile, confile, dfxfile)
```

Fortran command syntax:

```
CALL DFAXAPI_2(OPTIONS, SUBFILE, MONFILE, CONFILE, DFXFILE, IERR)
```

Where:

Integer OPTIONS (3)	array of elements specifying calculation options. The value of each element is as follows.(input).	
	OPTIONS(1)	distribution factor option flag. (1 by default)
	OPTIONS(1) = 0	do not calculate distribution factors (i.e., DFAX,AC).
	OPTIONS(1) = 1	calculate distribution factors.
	OPTIONS(2)	monitored element sorting flag (0 by default)
	OPTIONS(2) = 0	do not sort (i.e., leave in Monitored Element Description File order).
	OPTIONS(2) = 1	sort.
	OPTIONS(3)	out-of-service monitored branch flag (0 by default)
	OPTIONS(3) = 0	eliminate out-of-service branch from monitored branches.
	OPTIONS(3) = 1	keep out-of-service branch in monitored branches.
Character*260 SUBFILE	Is the name of the Subsystem Description File (input; blank by default).	
Character*260 MONFILE	Is the name of Monitored Element Description File (input; no default allowed).	
Character*260 CONFILE	Is the name of Contingency Description Data File (input; no default allowed).	
Character*260 DFXFILE	Is the name of Distribution Factor Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	buses in island(s) without a swing bus; use activity TREE
IERR = 4	no Distribution Factor Data File specified
IERR = 5	no Monitored Element Data input file specified.
IERR = 6	no Contingency Description Data file specified.
IERR = 7	fatal error reading input file.
IERR = 8	error opening output file DFXFILE.
IERR = 9	error opening input file SUBFILE.
IERR = 10	error opening input file MONFILE.
IERR = 11	error opening input file CONFILE.
IERR = 12	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.76. DFAX_CONTINGENCY

Use this API to combine one contingency in the first Distribution Factor file with one contingency in the second Distribution Factor file and so on, till specified contingency level is reached, to create multiple event contingencies and then export resulting contingencies to user defined report device.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr = dfax_contingency(option, confile, dfxfile)
```

Fortran command syntax:

```
CALL DFAX_CONTINGENCY(OPTION, CONFILE, DFXFILE, IERR)
```

Where:

Integer OPTION	level of multiple contingencies (input; from 1 to 3; 1).	
Character*260 CONFILE	name of Contingency Description Data File (input; blank by default).	
Character*260 DFXFILE (OPTION)	OPTION names of Distribution Factor Data Files where contingencies are combined each other to build OPTION level contingencies (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTION value
	IERR = 2	Input file is not a Distribution Factor Data File
	IERR = 3	error reading Distribution Factor Data File
	IERR = 51	error opening Contingency Description Data File
	IERR = 52	error opening Distribution Factor Data File

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.77. DFTI

Use this API to compare tie lines, as contained in the working case, with those of a designated Saved Case (activity DFTI). It must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by any number of references with APIOPT=2, followed by one reference with APIOPT=3.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference, followed by one APIOPT 3 reference.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_DFTI SID APIOPT STATUS(1)..STATUS(5) THRSH CFILE
```

Python command syntax:

```
ierr, seqflg = dfti(sid, apiopt, status, thrsh, cfile)
```

Fortran command syntax:

```
CALL DFTIAPI(SID, APIOPT, STATUS, THRSH, CFILE, SEQFLG, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; used only when APIOPT is 0 or 1; 0).	
Integer APIOPT	mode of operation of DFTIAPI (input; 0 by default).	
	APIOPT = 0	initialize for case comparison, run the comparison specified by STATUS(4), and run post-processing clean up.
	APIOPT = 1	initialize for case comparison.
	APIOPT = 2	run a comparison.
	APIOPT = 3	post-processing clean up.
Integer STATUS (5)	array of 5 elements specifying comparison options(input).	
	STATUS(1)	defines the working case content following an APIOPT=0 or 3 reference (used only when APIOPT is 0, 1 or 3; 0 by default)
	STATUS(1) = 0	original working case
	STATUS(1) = 1	Saved Case specified in CFILE
	STATUS(2)	flag for method of considering buses in the two cases to be the same bus (used only when APIOPT is 0 or 1; 0 by default)
	STATUS(2) = 0	same bus numbers
	STATUS(2) = 1	same extended bus names
	STATUS(2) = 2	same bus numbers and extended bus names
	STATUS(3)	flag for specifying THRESH (used only when APIOPT is 0 or 2; 0 by default)
	STATUS(3) = 0	engineering units

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(3) = 1	percent
	STATUS(4)	code for the comparison to be run (used only when APIOPT is 0 or 2; 1 by default)
	STATUS(4) = 1	branch status
	STATUS(4) = 2	line R, X, B
	STATUS(4) = 3	line shunts
	STATUS(4) = 4	line ratings
	STATUS(4) = 5	metered end
	STATUS(4) = 6	line length
	STATUS(4) = 7	transformers
	STATUS(4) = 8	flow MW or MVAR (from bus)
	STATUS(4) = 9	flow MW or MVAR (from & to)
	STATUS(4) = 10	line MW or MVAR losses
	STATUS(4) = 11	zero sequence R, X, B
	STATUS(4) = 12	zero sequence line shunts
	STATUS(4) = 13	transformer connection codes
	STATUS(4) = 14	flow MW (from bus)
	STATUS(4) = 15	flow MVAR (from bus)
	STATUS(4) = 16	flow MW (from and to buses)
	STATUS(4) = 17	flow MVAR (from and to buses)
	STATUS(4) = 18	line MW losses
	STATUS(4) = 19	line MVAR losses
	STATUS(5)	line rating to be compared when STATUS(4) is 4 (used only when APIOPT is 0 or 2; 0 by default)
	STATUS(5) = 0	all ratings
	STATUS(5) = 1	RATEA
	STATUS(5) = 2	RATEB
	STATUS(5) = 3	RATEC
Real THRSH	difference threshold (input; used only when APIOPT is 0 or 2; 0.0).	
Character*260 CFILE	name of Saved Case File in which tie lines are to be compared to those of the working case (when APIOPT is 0 or 1) (input; no default allowed).	
Logical SEQFLG (2)	array of 2 elements specifying returned when APIOPT=0 or 1(output).	
	SEQFLG(1)	true if there is sequence data in the working case
	SEQFLG(2)	true if there is sequence data in the Saved Case in file CFILE
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid STATUS value
	IERR = 3	CFILE is blank

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	no ties to compare
IERR = 5	invalid APIOPT value
IERR = 6	unexpected APIOPT value
IERR = 7	invalid THRSH value
IERR = 8	error reading the comparison case
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.78. DIFF

Use this API to compare specified power flow data and solution results, as contained in the working case, with those of a designated Saved Case (activity DIFF). It must be referenced first with APIOPT=1, followed by any number of references with APIOPT=2, followed by one reference with APIOPT=3:

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_DIFF SID ALL APIOPT STATUS(1)..STATUS(4) THRSH(1)..THRSH(3) CFILE
```

Python command syntax:

```
ierr, seqflg = diff(sid, all, apiopt, status, thrsh, cfile)
```

Fortran command syntax:

```
CALL DIFFNW(SID, ALL, APIOPT, STATUS, THRSH, CFILE, SEQFLG, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer APIOPT	mode of operation in the API (input; no default allowed).	
	APIOPT = 1	initialize for case comparison.
	APIOPT = 2	run case comparison for specified data category.
	APIOPT = 3	postprocessing housekeeping.
Integer STATUS (4)	array of elements specifying STATUS contains various processing options and has different values or meanings depending on the value of APIOPT(input).	
	STATUS(1)	working case or threshold option (0 by default)
	STATUS(1) = 0	If APIOPT=1 or 3, leave original case as working case. If APIOPT=2, thresholds are in enginering units.
	STATUS(1) = 1	If APIOPT=1 or 3, leave comparison case (CFILE) as working case. If APIOPT=2, thresholds are percentages.
	STATUS(2)	comparison options
	STATUS(2) = 0	If APIOPT=1, bus numbers must be the same to match (default for APIOPT=1 or 3).
	STATUS(2) = 1	If APIOPT=1, bus names must be the same to match. If APIOPT=2, compare bus identifiers (default for APIOPT=2).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(2) = 2	If APIOPT=1, bus numbers and names must be the same to match. If APIOPT=2, compare bus type codes.
STATUS(2) = 3	(If APIOPT=2) compare machine statuses.
STATUS(2) = 4	(If APIOPT=2) compare generator MW.
STATUS(2) = 5	(If APIOPT=2) compare generator MW or MVAR.
STATUS(2) = 6	(If APIOPT=2) compare bus loads.
STATUS(2) = 7	(If APIOPT=2) compare bus shunts.
STATUS(2) = 8	(If APIOPT=2) compare switched shunts.
STATUS(2) = 9	(If APIOPT=2) compare voltage.
STATUS(2) = 10	(If APIOPT=2) compare bus voltage & angle.
STATUS(2) = 11	(If APIOPT=2) compare MBASE & ZSORCE.
STATUS(2) = 12	(If APIOPT=2) compare MBASE & RPOS+jX.
STATUS(2) = 13	(If APIOPT=2) compare MBASE & ZNEG.
STATUS(2) = 14	(If APIOPT=2) compare MBASE & ZZERO.
STATUS(2) = 15	(If APIOPT=2) compare negative sequence loads.
STATUS(2) = 16	(If APIOPT=2) compare zero sequence loads.
STATUS(2) = 17	(If APIOPT=2) compare branch status.
STATUS(2) = 18	(If APIOPT=2) compare line R, X, & B.
STATUS(2) = 19	(If APIOPT=2) compare line shunts.
STATUS(2) = 20	(If APIOPT=2) compare line ratings.
STATUS(2) = 21	(If APIOPT=2) compare metered end.
STATUS(2) = 22	(If APIOPT=2) compare transformers.
STATUS(2) = 23	(If APIOPT=2) compare from P or Q flow.
STATUS(2) = 24	(If APIOPT=2) compare from and to P or Q flow.
STATUS(2) = 25	(If APIOPT=2) compare P or Q losses.
STATUS(2) = 26	(If APIOPT=2) compare zero sequence R, X, & B.
STATUS(2) = 27	(If APIOPT=2) compare zero sequence line shunts.
STATUS(2) = 28	(If APIOPT=2) compare connection codes.
STATUS(2) = 29	(If APIOPT=2) compare zero sequence mutuals.
STATUS(2) = 30	(If APIOPT=2) compare multisection lines.
STATUS(2) = 31	(If APIOPT=2) compare multisection line metered end.
STATUS(2) = 32	(If APIOPT=2) compare load status.
STATUS(2) = 33	(If APIOPT=2) compare line lengths.
STATUS(2) = 34	(If APIOPT=2) compare generator Q.
STATUS(2) = 35	(If APIOPT=2) compare from P flow.
STATUS(2) = 36	(If APIOPT=2) compare from Q flow.
STATUS(2) = 37	(If APIOPT=2) compare from and to P flow.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(2) = 38	(If APIOPT=2) compare from and to Q flow.
	STATUS(2) = 39	(If APIOPT=2) compare line P losses.
	STATUS(2) = 40	(If APIOPT=2) compare line Q losses.
	STATUS(2) = 41	(If APIOPT=2) compare shunt status.
	STATUS(2) = 42	(If APIOPT=2) compare switched shunt status.
	STATUS(2) = 43	(If APIOPT=2) compare load scalable flag.
	STATUS(2) = 44	(If APIOPT=2) compare MBASE & RPOS+jX'.
	STATUS(2) = 45	(If APIOPT=2) compare MBASE & RPOS+jXs.
	STATUS(2) = 46	(If APIOPT=2) compare bus substation.
	STATUS(2) = 47	(If APIOPT=2) compare node status.
	STATUS(2) = 48	(If APIOPT=2) compare switching device status.
	STATUS(2) = 49	(If APIOPT=2) compare switching device P or Q flow.
	STATUS(2) = 50	(If APIOPT=2) compare switching device P flow.
	STATUS(2) = 51	(If APIOPT=2) compare switching device Q flow.
	STATUS(3)	load characteric comparison option (0 by default)
	STATUS(3) = 0	use total nominal load (only used if APIOPT=2)
	STATUS(3) = 1	use contant MVA (only used if APIOPT=2)
	STATUS(3) = 2	use constant current (only used if APIOPT=2)
	STATUS(3) = 3	use constant admittance (only used if APIOPT=2)
	STATUS(4)	line rating to use. (1 by default)
	STATUS(4) = 0	use all ratings (only used if APIOPT=2)
	STATUS(4) = 1-12	use rating specified (only used if APIOPT=2)
Real THRSH (3)	array of 3 elements specifying thresholds (only used when APIOPT is 2)(input).	
	THRSH(1)	difference threshold (0.0 by default)
	THRSH(2)	voltage or tap ratio threshold (0.0 by default)
	THRSH(3)	angle threshold (0.0 by default)
Character*260 CFILE	saved case to be compared against the data currently in the working case (only used when APIOPT is 1) (input; no default allowed).	
Logical SEQFLG (2)	array of 2 elements specifying returned when APIOPT = 1(output).	
	SEQFLG(1)	true if there is sequence data in the working case
	SEQFLG(2)	true if there is sequence data in the Saved Case in file CFILE
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid SID value or subsystem SID is not defined
IERR = 2	invalid ALL value
IERR = 3	invalid APIOPT value
IERR = 4	invalid STATUS value
IERR = 5	invalid THRSH value
IERR = 6	CFILE is blank
IERR = 7	unexpected APIOPT value
IERR = 8	error building the case to case translation files
IERR = 9	error reading the comparison case
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.79. DSCN

Use this API to electrically disconnect a bus (activity DSCN).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_DSCN  BUS
```

Python command syntax:

```
ierr = dscn(bus)
```

Fortran command syntax:

```
CALL DSCNAPI(BUS, IERR)
```

Where:

Integer BUS	number of the bus to be disconnected (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus BUS not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.80. DUPLICATE_CNTLABEL_CHECK

Use this API to specify or return the option to enable or disable the duplicated contingency labels check when Distribution Factor Data File (*.dfx) or Contingency Description Data File (*.con) files are used.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_DUPLICATE_CNTLABEL_CHECK STATUS
```

Python command syntax:

```
ierr = duplicate_cntlabel_check(status)

ierr, status = duplicate_cntlabel_check()
```

Fortran command syntax:

```
CALL DUPLICATE_CNTLABEL_CHECK(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, no default allowed; else output).	
	STATUS = 0	disable checking duplicated contingency labels in DFAX or CON file.
	STATUS = 1	enable checking duplicated contingency labels in DFAX or CON file.
Integer ICODE	Indicates the operation mode of the API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in STATUS.
	ICODE = 1	return the current value of the option setting in STATUS.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	Invalid STATUS value
	IERR = 2	Invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.81. ECDI

Use this API to place machines in a specified subsystem on- or off-line to satisfy a given subsystem minimum capacity; the in-service machines in the subsystem are then dispatched on the basis of incremental cost to meet a specified total subsystem generation (activity ECDI).
This API must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by one reference with APIOPT=2, followed by one or more references with APIOPT=3, followed by one reference with APIOPT=4.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference, followed by one APIOPT 3 reference, followed by one APIOPT 4 reference.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ECDI  SID  ALL  APIOPT  CFILE  STATUS  VALUES(1)  VALUES(2)
```

Python command syntax:

```
ierr = ecdi(sid, all, apiopt, cfile, status, values)
```

Fortran command syntax:

```
CALL ECDIAPI(SID, ALL, APIOPT, CFILE, STATUS, VALUES, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; used only when APIOPT is 0 or 2; 0).	
Integer ALL	all buses or specified subsystem flag (input; used only when APIOPT is 0 or 2; 1).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer APIOPT	mode of operation of the API (input; 0 by default).	
	APIOPT = 0	initialize, specify subsystem, process, and clean up.
	APIOPT = 1	initialize for dispatch.
	APIOPT = 2	specify subsystem.
	APIOPT = 3	apply calculation.
	APIOPT = 4	postprocessing housekeeping.
Character*260 CFILE	name of the Economic Dispatch Data File (when APIOPT is 0 or 1) (input; no default allowed).	
Integer STATUS	unit commitment option (input; used only when APIOPT is 0 or 2; 0).	
	STATUS = 0	start from current commitment profile.
	STATUS = 1	new commitment profile (start with all units off-line).
	STATUS = 2	unit commitment disabled.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real VALUES (2)	array of 2 elements specifying (used only when APIOPT is 0 or 3)(input).	
	VALUES(1)	desired loading (present loading by default)
	VALUES(2)	desired minimum capacity of units being dispatched (0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid APIOPT value
	IERR = 3	invalid ALL value
	IERR = 4	invalid STATUS value
	IERR = 5	dispatch unable to meet desired power and capacity
	IERR = 6	no dispatch data for any subsystem machines
	IERR = 7	negative VALUES value
	IERR = 8	unexpected APIOPT value
	IERR = 9	error opening CFILE
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.82. ECHO

Use this API to enable or disable response echoing (activity ECHO).

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_ECHO  IDVFIL
```

Python command syntax:

```
ierr = echo(idvfil)
```

Fortran command syntax:

```
CALL ECHOAPI(IDVFIL, IERR)
```

Where:

Character*260 IDVFIL	name of Response File to be constructed (blank to disable) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error opening IDVFIL
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.83. EEQV

Use this API to construct an electrical equivalent of a specified subsystem of the working case (activity EEQV).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_EEQV  SID  ALL  STATUS(1)..STATUS(6)  DVAL(1)  DVAL(2)
```

Python command syntax:

```
ierr = eeqv(sid, all, status, dval)
```

Fortran command syntax:

```
CALL EEQVNW(SID, ALL, STATUS, DVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer STATUS (6)	array of 6 elements specifying They are used to control equivalencing options. A value of 0 means the option is not enabled. A value of 1 means the option is enabled. They are as follows(input).	
	STATUS(1)	retain area boundary buses (0 by default)
	STATUS(2)	retain zone boundary buses (0 by default)
	STATUS(3)	suppress the equivalencing option for phase shifters (0 by default)
	STATUS(4)	retain controlled buses (0 by default)
	STATUS(5)	retain existing branches between retained buses (0 by default)
	STATUS(6)	net load and shunt at retained buses (0 by default)
Real DVAL (2)	array of 2 elements specifying They are as follows(input).	
	DVAL(1)	minimum active or reactive power generation for retaining generator buses (0.0 by default)
	DVAL(2)	branch threshold tolerance (10.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	invalid STATUS value
IERR = 4	invalid DVAL value
IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.84. EQRD

Use this API to build an electrical equivalent of radial and, optionally, two-point Type 1 buses in a specified subsystem of the working case (activity EQRD).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_EQRD  SID  ALL  STATUS(1)..STATUS(8)
```

Python command syntax:

```
ierr = eqrd(sid, all, status)
```

Fortran command syntax:

```
CALL EQRDNW(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer STATUS (8)	array of 8 elements specifying They are used to control equivalencing options. A value of 0 means the option is not enabled. A value of 1 means the option is enabled. They are as follows(input).	
	STATUS(1)	build an electrical equivalent of radial buses only (0 by default)
	STATUS(2)	suppress the equivalencing option for transformers (0 by default)
	STATUS(3)	suppress the equivalencing option for zero impedance lines (0 by default)
	STATUS(4)	suppress the equivalencing option for controlled buses (0 by default)
	STATUS(5)	suppress the equivalencing option for mutually coupled branches (0 by default)
	STATUS(6)	suppress the equivalencing option for area boundary buses (0 by default)
	STATUS(7)	suppress the equivalencing option for zone boundary buses (0 by default)
	STATUS(8)	apply transformer impedance correction to zero sequence network (1 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ALL value
IERR = 3	invalid STATUS value
IERR = 4	error writing the working case to a temporary file
IERR = 5	load table is full; restoring original working case
IERR = 6	fixed bus shunt table is full; restoring original working case
IERR = 7	error opening temporary file
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.85. EXAM

Use this API to tabulate all power flow data pertaining to a specified bus (activity EXAM).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_EXAM SID ALL
```

Python command syntax:

```
ierr = exam(sid, all)
```

Fortran command syntax:

```
CALL EXAMNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.86. FACT

Use this API to factorize the network admittance matrix in preparation for switching studies and dynamic simulations (activity FACT).

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_FACT
```

Python command syntax:

```
ierr = fact()
```

Fortran command syntax:

```
CALL FACTAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	generators are not converted
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.87. FDNS

Use this API to apply the fixed slope decoupled Newton-Raphson power flow calculation (activity FDNS).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_FDNS  OPTIONS(1)..OPTIONS(8)
```

Python command syntax:

```
ierr = fdns(options)
```

Fortran command syntax:

```
CALL FDNSAPI(OPTIONS, IERR)
```

Where:

Integer OPTIONS (8)	array of 8 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (use tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (use area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (use phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (use dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (use switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	flat start flag (0 by default)
	OPTIONS(6) = 0	do not flat start
	OPTIONS(6) = 1	flat start
	OPTIONS(6) = 2	flat start, then estimate voltage magnitudes
	OPTIONS(6) = 3	flat start, then estimate voltage phase angles
	OPTIONS(6) = 4	flat start, then estimate voltage magnitudes and phase angles
	OPTIONS(7)	var limit flag (99 by default)
	OPTIONS(7) = 0	apply var limits immediately
	OPTIONS(7) > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	OPTIONS(7) = -1	ignore var limits
	OPTIONS(8)	non-divergent solution flag (use non-divergent solution option setting by default)
	OPTIONS(8) = 0	disable
	OPTIONS(8) = 1	enable
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted
	IERR = 3	buses in island(s) without a swing bus; use activity TREE
	IERR = 4	bus type code and series element status inconsistencies
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.88. FILE_OVERWRITE

Use this API to specify or return the option to set the file overwrite option setting to either ask first or overwrite.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_FILE_OVERWRITE IVAL
```

Python command syntax:

```
ierr = file_overwrite(ival)

ierr, ival = file_overwrite()
```

Fortran command syntax:

```
CALL FILE_OVERWRITE_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	ask first.
	IVAL = 1	overwrite.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.89. FIND

Use this API to tabulate a list of buses matching a partial extended bus name (activity FIND).

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_FIND NAME BSKV
```

Python command syntax:

```
ierr = find(name, bskv)
```

Fortran command syntax:

```
CALL FINDAPI(NAME, BSKV, IERR)
```

Where:

Character*24 NAME	bus name, including imbedded asterisks that are treated as wildcard characters (input; blank by default).	
Character*12 BSKV	bus base voltage (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error decoding BSKV
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.90. FLAT_START

Use this API to specify or return the option to enable or disable the voltage flat start option setting.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_FLAT_START IVAL
```

Python command syntax:

```
ierr = flat_start(ival)

ierr, ival = flat_start()
```

Fortran command syntax:

```
CALL FLAT_START_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	use present voltages as the starting point in power flow solutions.
	IVAL = 1	flat start voltages before power flow solutions.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.91. FNSL

Use this API to apply the Newton-Raphson power flow calculation (activity FNSL).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_FNSL  OPTIONS(1)..OPTIONS(8)
```

Python command syntax:

```
ierr = fnsl(options)
```

Fortran command syntax:

```
CALL FNSLAPI(OPTIONS, IERR)
```

Where:

Integer OPTIONS (8)	array of 8 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (use tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (use area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (use phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (use dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (use switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	flat start flag (0 by default)
	OPTIONS(6) = 0	do not flat start
	OPTIONS(6) = 1	flat start
	OPTIONS(6) = 2	flat start, then estimate voltage magnitudes
	OPTIONS(6) = 3	flat start, then estimate voltage phase angles
	OPTIONS(6) = 4	flat start, then estimate voltage magnitudes and phase angles
	OPTIONS(7)	var limit flag (99 by default)
	OPTIONS(7) = 0	apply var limits immediately
	OPTIONS(7) > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	OPTIONS(7) = -1	ignore var limits
	OPTIONS(8)	non-divergent solution flag (use non-divergent solution option setting by default)
	OPTIONS(8) = 0	disable
	OPTIONS(8) = 1	enable
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted
	IERR = 3	buses in island(s) without a swing bus; use activity TREE
	IERR = 4	bus type code and series element status inconsistencies
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.92. GCAP

This API routine is obsolete. It has been replaced by the API routines ReadCapCurves and GCAP_2 .

Use this API to read a data file containing capability curve data and print a report of machine loading and limit data (activity GCAP). Optionally, machine reactive power limits in the working case may be updated.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_GCAP  SID  ALL  OPT  GFILE
```

Python command syntax:

```
ierr = gcap(sid, all, opt, gfile)
```

Fortran command syntax:

```
CALL GCAPNW(SID, ALL, OPT, GFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer OPT	update var limits option (input; 0 by default).	
	OPT = 0	leave var limits unchanged.
	OPT = 1	update var limits in the working case.
Character*260 GFILE	Machine Capability Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	error opening GFILE
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.93. GCAP_2

Use this API to print a report of machine loading and reactive power limit data (activity GCAP). Optionally, machine reactive power limits in the working case may be updated.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_GCAP_2 SID ALL OPT
```

Python command syntax:

```
ierr = gcap_2(sid, all, opt)
```

Fortran command syntax:

```
CALL GCAP_2(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer OPT	update var limits option (input; 0 by default).	
	OPT = 0	leave var limits unchanged.
	OPT = 1	update var limits in the working case.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	capability curve data has not been read into this case
	IERR = 2	invalid SID value or subsystem SID is not defined
	IERR = 3	invalid ALL value
	IERR = 4	invalid OPT value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.94. GDIF

Use this API to calculate differences between the working case and a designated Saved Case (activity GDIF). Results may be programmatically accessed via the internally accessible fetch routines DIFTXT, DIFBUS, DIFBRN, DIF3WN, DIF2DC, DIFMDC and DIFFCT.
Usage is: call to GDIFAPI with APIOPT = 0;
multiple calls to fetch routines;
optionally, repeat of above steps for different Saved Case and/or subsystem;
final call to GDIFAPI with APIOPT = 1.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_GDIF SID ALL APIOPT NAMARG FILARG
```

Python command syntax:

```
ierr = gdif(sid, all, apiopt, namarg, filarg)
```

Fortran command syntax:

```
CALL GDIFAPI(SID, ALL, APIOPT, NAMARG, FILARG, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; used only when APIOPT is 0; 0).	
Integer ALL	all buses or specified subsystem flag (input; used only when APIOPT is 0; 1).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer APIOPT	mode of operation of GDIFAPI (input; no default allowed).	
	APIOPT = 0	apply difference calculations and fill arrays used by fetch routines.
	APIOPT = 1	deallocate arrays used by fetch routines.
Integer NAMARG	flag indicating how to match up buses in the two cases (input; used only when APIOPT is 0; 0).	
	NAMARG = 0	by bus number only.
	NAMARG = 1	by extended bus name only.
	NAMARG = 2	by both number and name.
Character*260 FILARG	name of Saved Case File to be compared to the working case (when APIOPT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	invalid APIOPT value
IERR = 4	invalid NAMARG value
IERR = 5	FILARG is blank
IERR = 6	error reading the comparison case
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.95. GENDSP

Use this API to impose a contingency specified in the Distribution Factor Data file and apply the generation dispatch algorithm used in contingency analysis on the working case. The network can then be solved by power flow solutions.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_GENDSP  OPTIONS(1)  OPTIONS(2)  LABELS(1)  LABELS(2)  DFXFILE  THRFILE
  INLFILE
```

Python command syntax:

```
ierr = gendsp(options, labels, dfxfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL GENDSPAPI(OPTIONS, LABELS, DFXFILE, THRFILE, INLFILE, IERR)
```

Where:

Integer OPTIONS (2)	array of 2 elements specifying options(input).	
	OPTIONS(1)	dispatch mode (1 by default)
	OPTIONS(1) = 0	disable dispatch
	OPTIONS(1) = 1	subsystem machines (RESERVE)
	OPTIONS(1) = 2	subsystem machines (PMAX)
	OPTIONS(1) = 3	subsystem machines (INERTIA)
	OPTIONS(1) = 4	subsystem machines (GOVERNOR DROOP)
	OPTIONS(2)	study case (1 by default)
	OPTIONS(2) = 0	impose a contingency case on the working case
	OPTIONS(2) = 1	working case
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTIONS(1) is from 1 to 4)
	LABELS(2)	contingency label (blank by default, no default allowed if OPTIONS(2) is 0)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPTIONS value
	IERR = 2	no dispatch subsystem specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	no contingency specified
IERR = 4	no Distribution Factor Data File specified
IERR = 5	generators are converted
IERR = 6	too many islands in base case
IERR = 7	contingency not found in Distribution Factor Data File
IERR = 21	file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 23	no subsystem defined in Distribution Factor Data file
IERR = 24	subsystem not found
IERR = 51	error opening Distribution Factor Data File
IERR = 52	error opening Load Throwover Data File
IERR = 53	error opening Unit Inertia and Governor Data File
IERR = 54	error opening or writing temporary Saved Case File
IERR = 55	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.96. GENDSP_2

Use this API to impose a contingency specified in the Distribution Factor Data file and apply the generation dispatch algorithm used in contingency analysis on the working case. The network can then be solved by power flow solutions.

This API routine was first introduced in release 35.3.3.

Batch command syntax:

```
BAT_GENDSP_2 OPTIONS(1) OPTIONS(2) SYSLABEL CNTLABEL DFXFILE THRFILE
 INLFILE
```

Python command syntax:

```
ierr = gendsp_2(options, syslabel, cntlabel, dfxfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL GENDSPAPI_2(OPTIONS, SYSLABEL, CNTLABEL, DFXFILE, THRFILE, INLFILE,
 IERR)
```

Where:

Integer OPTIONS (2)	array of 2 elements specifying options(input).	
	OPTIONS(1)	dispatch mode (1 by default)
	OPTIONS(1) = 0	disable dispatch
	OPTIONS(1) = 1	subsystem machines (RESERVE)
	OPTIONS(1) = 2	subsystem machines (PMAX)
	OPTIONS(1) = 3	subsystem machines (INERTIA)
	OPTIONS(1) = 4	subsystem machines (GOVERNOR DROOP)
	OPTIONS(2)	study case (1 by default)
	OPTIONS(2) = 0	impose a contingency case on the working case
	OPTIONS(2) = 1	working case
Character*32 SYSLABEL	subsystem label (input).	
Character*160 CNTLABEL	contingency label (input).	
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPTIONS value
	IERR = 2	no dispatch subsystem specified
	IERR = 3	no contingency specified
	IERR = 4	no Distribution Factor Data File specified
	IERR = 5	generators are converted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	too many islands in base case
IERR = 7	contingency not found in Distribution Factor Data File
IERR = 21	file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 23	no subsystem defined in Distribution Factor Data file
IERR = 24	subsystem not found
IERR = 51	error opening Distribution Factor Data File
IERR = 52	error opening Load Throwover Data File
IERR = 53	error opening Unit Inertia and Governor Data File
IERR = 54	error opening or writing temporary Saved Case File
IERR = 55	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.97. GENS

Use this API to tabulate the loading and voltage conditions at plant buses (activity GENS).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_GENS SID ALL OPT VREV
```

Python command syntax:

```
ierr = gens(sid, all, opt, vrev)
```

Fortran command syntax:

```
CALL GENSNW(SID, ALL, OPT, VREV, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer OPT	option (input; 4 by default).	
	OPT = 1	tabulate var limited plants with unequal var limits.
	OPT = 2	tabulate all var limited plants.
	OPT = 3	tabulate on-line plants.
	OPT = 4	tabulate all plants.
Integer VREV	option to select the units in which voltages are tabulated (input; 0 by default).	
	VREV = 0	use default voltage units.
	VREV = 1	use alternate voltage units.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	invalid VREV value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.98. GEOL

Use this API to tabulate the loading and voltage conditions at the generator terminals for online machines at Type 2 and 3 buses in the working case (activity GEOL).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_GEOL SID ALL OPT
```

Python command syntax:

```
ierr = geol(sid, all, opt)
```

Fortran command syntax:

```
CALL GEOLNW(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer OPT	option (input; 1 by default).	
	OPT = 1	tabulate overloaded machines only.
	OPT = 2	tabulate all machines.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.99. GETCONTINGENCYSAVEDCASE

Use this API routine to place the working case in the form of a specified system condition as calculated during a previous run of one of the members of the the AC contingency calculation family. This function retrieves data from a ZIP Archive Ouptut File that was created during the contingency calculation.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_GETCONTINGENCYSAVEDCASE PATHZIP ISVFILE
```

Python command syntax:

```
ierr = getcontingencysavedcase(pathzip, isvfile)
```

Fortran command syntax:

```
CALL GETCONTINGENCYSAVEDCASE(PATHZIP, ISVFILE, IERR)
```

Where:

Character*260 PATHZIP	Is the name of the ZIP Archive Output File (input; no default allowed).	
Character*260 ISVFILE	Is the name of an Incremental Saved Case File (.isv file) contained in PATHZIP (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 1	unable to pick up the base Saved Case File from the ZIP Archive File.
	IERR = 2	error opening the Incremental Saved Case File.
	IERR = 3	error closing the Incremental Saved Case File.
	IERR = 4	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.100. GIC_PF

Add activity GIC calculated Mvar Qloss in scaled manner to the working case to arrive at the converged power flow solution.

This API routine was first introduced in release 34.5.0.

Python command syntax:

```
ierr = gic_pf(addfile, qpct, method, tap, area, phshift, dctap, swsh,
 flat, varlimit, nondiv, pfprogress)
```

Where:

Character ADDFILE	RDCH raw file created by GIC activity that represents Constant Current Mvar losses due to GMD corresponding to the working case (input; no default allowed).
real QPCT	Percent Mvar loss step size that will be incrementally added to the working case (input; 100 by default [add all (100%) Mvar losses in one step]).
character METHOD	Power flow solution method (input; last by default). method = last — last power flow method used by working case method = fnsl — full Newton-Raphson method = fdns — fixed slope decoupled Newton-Raphson method = nsol — decoupled Newton-Raphson
integer TAP	power flow tap adjustment flag (input; use last solution tap option by default). tap = 0 — disable tap = 1 — enable stepping adjustment tap = 2 — enable direct adjustment
integer AREA	power flow area interchange adjustment flag (input; use last solution area option by default). area = 0 — disable area = 1 — enable using tie line flows only in calculating area interchange area = 2 — enable using tie line flows and loads in calculating area interchange
integer PHSHIFT	power flow phase shift adjustment flag (input; use last solution phshift option by default). phshift = 0 — disable phshift = 1 — enable
integer DCTAP	power flow dc tap adjustment flag (input; use last solution dctap option by default). dctap = 0 — disable dctap = 1 — enable
integer SWSH	power flow switched shunt adjustment flag (input; use last solution swsh option by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	swsh = 0	disable
	swsh = 1	enable
	swsh = 2	enable continuous mode, disable discrete mode
integer FLAT	power flow flat start flag (input; use last solution flat option by default).	
	flat = 0	do not flat start
	flat = 1	flat start
	flat = 2	flat start, then estimate voltage magnitudes
	flat = 3	flat start, then estimate voltage phase angles
	flat = 4	flat start, then estimate voltage magnitudes and phase angles
integer VARLIMIT	power flow var limit flag (input; use last solution varlimit option by default).	
	varlimit = 0	apply var limits immediately
	varlimit > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	varlimit = -1	ignore var limits
integer NONDIV	power flow non-divergent solution flag (not used in NSOL) (input; use last solution nondiv option by default).	
	nondiv = 0	disable
	nondiv = 1	enable
integer PFPROGRESS	flag to show or suppress data change and power flow solution progres messages (input; 0 by default).	
	pfprogress = 0	suppress progress
	pfprogress = 1	show progress
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	Data change or PF solution error, check progress for details

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.101. GNESTATUS

Use this API to specify or return the status of a GNE device.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_GNESTATUS 'NAME' IVAL
```

Python command syntax:

```
ierr = gnestatus(name, ival)

ierr, ival = gnestatus(name)
```

Fortran command syntax:

```
CALL GNESTATUS(NAME, IVAL, IOCODE, IERR)
```

Where:

Character*12 NAME	name of the GNE device for which the status is to be changed (input; no default allowed).	
Integer IVAL	value of the status of the specified GNE device (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, no default allowed; else output).	
	IVAL = 0	out-of-service.
	IVAL = 1	in-service.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the status to the value specified in IVAL.
	IOCODE = 1	return the current value of the status in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	GNE device not found
	IERR = 2	invalid GNE status value
	IERR = 3	invalid IOCODE value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.102. GNET

Use this API to change in-service generation to negative MVA load at all Type 2 and 3 buses in the subsystem specified by the user.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_GNET  SID  ALL
```

Python command syntax:

```
ierr = gnet(sid, all)
```

Fortran command syntax:

```
CALL GNETNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.103. IMNET

Use this API routine to change the status of induction machines that are in-service to out-of-service, and to replace the power flowing from the network into the machine with constant power load. The routine can be directed to process either all in-service induction machines, all in-service induction machines within a specified subsystem, or all in-service induction machines outside of a specified subsystem.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IMNET  SID  ALL  INOUT
```

Python command syntax:

```
ierr = imnet(sid, all, inout)
```

Fortran command syntax:

```
CALL IMNET(SID, ALL, INOUT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only induction machines within, or outside of, subsystem SID
	ALL = 1	process all induction machines
Integer INOUT	include or exclude induction machines in subsystem SID (input; 0 by default).	
	INOUT = 0	process only machines inside of subsystem SID
	INOUT = 1	process only machines outside of subsystem SID
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid INOUT value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.104. IMOUTAGE

Use this API routine to change the status of induction machines that are in-service, but that have been set to the "stalled" (for motors) or "tripped" (for generators) state by the power flow solution, to out-of-service.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IMOUTAGE  SID  ALL  MOTOPT  GENOPT
```

Python command syntax:

```
ierr = imoutage(sid, all, motopt, genopt)
```

Fortran command syntax:

```
CALL IMOUTAGE(SID, ALL, MOTOPT, GENOPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only induction machines in subsystem SID
	ALL = 1	process all induction machines
Integer MOTOPT	"stalled" induction motor option (input; 0 by default).	
	MOTOPT = 0	set to out-of-service
	MOTOPT = 1	leave in-service
Integer GENOPT	"tripped" induction generator option (input; 0 by default).	
	GENOPT = 0	set to out-of-service
	GENOPT = 1	leave in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid MOTOPT value
	IERR = 4	invalid GENOPT value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.105. IMPC

Use this API to calculate the impact of transaction events on MW flows using a linear network (dc) model.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_IMPC  EVENT DFXFILE ATCFILE
```

Python command syntax:

```
ierr = impc(event, dfxfile, atcfile)
```

Fortran command syntax:

```
CALL IMPCAPI(EVENT, DFXFILE, ATCFILE, IERR)
```

Where:

Integer EVENT	transaction event number (input; no default allowed).	
Character*260 DFXFILE	name of DFAX data file (input; no default allowed).	
Character*260 ATCFILE	name of ATC updates file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no transactions defined
	IERR = 2	transaction event not defined
	IERR = 3	no name of DFAX data file specified
	IERR = 4	error opening ATC updates file
	IERR = 5	error opening file DFXFILE
	IERR = 6	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 8	no monitored branches or interfaces defined
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.106. IMPLEMENT_TRANSFER

This API routine is obsolete. It has been replaced by the API routine implement_transfer_2 .

Use this API to apply a specified transfer using the same transfer dispatch methods that are available in the PV analysis calculation engine.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_IMPLEMENT_TRANSFER  OPTIONS(1)..OPTIONS(4)  RVAL  LABELS(1)  LABELS(2)
  DFXFILE  ECDFILE
```

Python command syntax:

```
ierr = implement_transfer(options, rval, labels, dfxfile, ecdfile)
```

Fortran command syntax:

```
CALL IMPLEMENT_TRANSFER(OPTIONS, RVAL, LABELS, DFXFILE, ECDFILE, IERR)
```

Where:

Integer OPTIONS (4)	array of 4 elements specifying transfer implementation options(input).	
	OPTIONS(1)	study (source) system transfer dispatch method (1 by default)
	OPTIONS(1) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
	OPTIONS(1) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(1) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(1) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(1) = 5	subsystem buses with positive MW machines in proportion to their MW output
	OPTIONS(1) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
	OPTIONS(1) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
	OPTIONS(1) = 8	subsystem buses with positive MW machines via ECDI
	OPTIONS(2)	opposing (sink) system transfer dispatch method (1 by default)
	OPTIONS(2) = 1	buses and distribution factors from DFAX file for buses with positive MW machines

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(2) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(2) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(2) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(2) = 5	subsystem buses with positive MW machines in proportion to their MW output
	OPTIONS(2) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
	OPTIONS(2) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
	OPTIONS(2) = 8	subsystem buses with positive MW machines via ECDI
	OPTIONS(3)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
	OPTIONS(3) = 0	no limits
	OPTIONS(3) = 1	honor machine active power limits
	OPTIONS(4)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
	OPTIONS(4) = 0	no limits
	OPTIONS(4) = 1	enforce non-negative net active power constant MVA load
Real RVAL	transfer increment in MW (input; no default allowed).	
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which load is to be increased (no default allowed)
Character*260 DFXFILE	name of the DFAX data file (input; no default allowed).	
Character*260 ECDFILE	name of the Economic Dispatch Data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid RVAL value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	no DFAX data file specified
IERR = 6	no economic dispatch data file specified
IERR = 7	no participating dispatch buses in study system
IERR = 8	no participating dispatch buses in opposing system
IERR = 9	subsystem engine error
IERR = 10	unable to implement transfer in study system
IERR = 11	unable to implement transfer in opposing system
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 52	error opening DFAX data file
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.107. IMPLEMENT_TRANSFER_2

Use this API routine to apply a specified transfer using the same transfer dispatch methods that are available in the PV analysis calculation engine.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_IMPLEMENT_TRANSFER_2  OPTIONS(1)..OPTIONS(4) RVAL(1)..RVAL(2)
 LABELS(1)..LABELS(2) DFXFILE ECDFILE
```

Python command syntax:

```
ierr = implement_transfer_2(options, rval, labels, dfxfile, ecdfile)
```

Fortran command syntax:

```
CALL IMPLEMENT_TRANSFER_2(OPTIONS, RVAL, LABELS, DFXFILE, ECDFILE, IERR)
```

Where:

Integer OPTIONS (4)	array of elements specifying transfer implementation options (input). The values are as follows:(input).	
	OPTIONS(1)	study (source) system transfer dispatch method (1 by default)
	OPTIONS(1) = 1	buses and distribution factors from DFAX file for buses with positive MW machines.
	OPTIONS(1) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(1) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(1) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(1) = 5	subsystem buses with positive MW machines in proportion to their MW output.
	OPTIONS(1) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
	OPTIONS(1) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC).
	OPTIONS(1) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
	OPTIONS(1) = 9	subsystem buses with positive MW machines via ECDI with unit commitment enabled
	OPTIONS(2)	opposing (sink) system transfer dispatch method (1 by default)
	OPTIONS(2) = 1	buses and distribution factors from DFAX file for buses with positive MW machines.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(2) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(2) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(2) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(2) = 5	subsystem buses with positive MW machines in proportion to their MW output.
	OPTIONS(2) = 6	subsystem buses with positive MW machines in proportion to their MBASEs.
	OPTIONS(2) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC).
	OPTIONS(2) = 8	subsystem buses with positive MW machines via ECDI. with unit commitment disabled
	OPTIONS(2) = 9	subsystem buses with positive MW machines via ECDI with unit commitment enabled
	OPTIONS(3)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
	OPTIONS(3) = 0	no limits.
	OPTIONS(3) = 1	honor machine active power limits.
	OPTIONS(4)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
	OPTIONS(4) = 0	no limits.
	OPTIONS(4) = 1	enforce non-negative net active power constant MVA load.
Real RVAL (2)	array of elements specifying transfer implementation options. The values are as follows:(input).	
	RVAL(1)	the transfer increment in MW
	RVAL(2)	power factor for load increases in dispatch methods 2, 3 and 4 (0.0 by default). Specify RVAL(2) as < 0.0 to retain the original power factor at each load that is changed.
Character*32 LABELS (2)	array of elements specifying subsystem labels (input). The values are as follows:(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which load is to be decreased
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which load is to be increased
Character*260 DFXFILE	Is the name of the Distribution Factor Data file (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 ECDFILE	Is the name of the Economic Dispatch Data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 1	invalid RVAL value.
	IERR = 2	invalid OPTIONS value.
	IERR = 3	generators are converted.
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	no Distribution Factor Data File specified.
	IERR = 6	no Economic Dispatch Data File specified.
	IERR = 7	no participating dispatch buses in study system.
	IERR = 8	no participating dispatch buses in opposing system
	IERR = 9	subsystem engine error.
	IERR = 10	Unable to implement transfer in study system.
	IERR = 11	Unable to implement transfer in opposing system
	IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 23	no subsystems defined.
	IERR = 24	subsystem not found.
	IERR = 25	specified subsystems overlap.
	IERR = 26	no in-service buses in subsystem WORLD.
	IERR = 50	error opening Economic Dispatch Data File.
	IERR = 52	error opening Distribution Factor Data File.
	IERR = 54	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.108. INLF

This API routine is obsolete. It has been replaced by the API routine inlf_2 .

Use this API to apply an inertial or governor response power flow calculation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_INLF  OPTIONS(1)..OPTIONS(7)  IFILE
```

Python command syntax:

```
ierr = inlf(options, ifile)
```

Fortran command syntax:

```
CALL INLFAPI(OPTIONS, IFILE, IERR)
```

Where:

Integer OPTIONS (7)	array of 7 elements specifying solution options(input).	
	OPTIONS(1)	solution type (0 by default)
	OPTIONS(1) = 0	inertial
	OPTIONS(1) = 1	governor response
	OPTIONS(2)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(2) = -1	solution type default (inertial => disable; governor => stepping)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable stepping adjustment
	OPTIONS(2) = 2	enable direct adjustment
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = -1	solution type default (inertial => disable; governor => enable)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = -1	solution type default (inertial => disable; governor => enable)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = -1	solution type default (inertial => enable; governor => enable)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	generator var limit flag (-1 (inertial) or 99 (governor) by default)
	OPTIONS(6) = 0	apply var limits immediately
	OPTIONS(6) > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	OPTIONS(6) = -1	ignore var limits
	OPTIONS(7)	missing active power limits flag (only used in governor response power flow) (0 by default)
	OPTIONS(7) = 0	pre-PSSE-29 approach (0.0 to 1.0 pu for machines with no Unit Inertia and Governor Data File record read; working case values for machines with PMAX defaulted on ECDI data record)
	OPTIONS(7) = 1	0.0 to 1.0 pu for both categories of machines
	OPTIONS(7) = 2	working case values for both categories of machines
Character*260 IFILE	name of unit inertia and governor data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted
	IERR = 3	bus type code and series element status inconsistencies
	IERR = 4	error opening IFILE
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.109. INLF_2

Use this API routine to run the second release of the inertial and governor response power flow calculation.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_INLF_2  OPTIONS(1)..OPTIONS(8)  IFILE
```

Python command syntax:

```
ierr = inlf_2(options, ifile)
```

Fortran command syntax:

```
CALL INLF_2(OPTIONS, IFILE, IERR)
```

Where:

Integer OPTIONS (8)	array of 8 elements specifying solution options(input).	
	OPTIONS(1)	solution type (0 by default)
	OPTIONS(1) = 0	inertial
	OPTIONS(1) = 1	governor response
	OPTIONS(2)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(2) = -1	solution type default (inertial => disable; governor => stepping)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable stepping adjustment
	OPTIONS(2) = 2	enable direct adjustment
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = -1	solution type default (inertial => disable; governor => enable)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = -1	solution type default (inertial => disable; governor => enable)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = -1	solution type default (inertial => enable; governor => enable)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	generator var limit flag (-1 (inertial) or 99 (governor) by default)
	OPTIONS(6) = 0	apply var limits immediately
	OPTIONS(6) > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	OPTIONS(6) = -1	ignore var limits
	OPTIONS(7)	induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
	OPTIONS(7) = 0	stall
	OPTIONS(7) = 1	trip
	OPTIONS(8)	missing active power limits flag (only used in governor response power flow) (0 by default)
	OPTIONS(8) = 0	pre-PSSE-29 approach (0.0 to 1.0 pu for machines with no Unit Inertia and Governor Data File record read; working case values for machines with PMAX defaulted on ECDI data record)
	OPTIONS(8) = 1	0.0 to 1.0 pu for both categories of machines
	OPTIONS(8) = 2	working case values for both categories of machines
Character*260 IFILE	name of unit inertia and governor data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted
	IERR = 3	bus type code and series element status inconsistencies
	IERR = 4	error opening IFILE
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.110. INTA

Use this API to summarize tie flows between an interchange area and all other areas in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_INTA SID ALL
```

Python command syntax:

```
ierr = inta(sid, all)
```

Fortran command syntax:

```
CALL INTANW(SID, ALL, IERR)
```

Where:

Integer SID	a valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all areas or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only areas in area subsystem SID
	ALL = 1	process all areas
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.111. INTZ

Use this API to summarize tie flows between each zone and all other zones in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_INTZ SID ALL
```

Python command syntax:

```
ierr = intz(sid, all)
```

Fortran command syntax:

```
CALL INTZNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Zone subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all zones or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only zones in zone subsystem SID
	ALL = 1	process all zones
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.112. ISOLATE_LEVELS

Use this API to specify or return the maximum number of levels to go outward when isolating an element by breaker.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATE_LEVELS IVAL IOCODE
```

Python command syntax:

```
ierr = isolate_levels(ival, iocode)
```

Fortran command syntax:

```
CALL ISOLATE_LEVELS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 4 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.113. JOIN

The API combines two buses into a single bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_JOIN    IBUS    JBUS    STATUS
```

Python command syntax:

```
ierr = join(ibus, jbus, status)

ierr, idtrns = join(ibus, jbus, status, nid)
```

Fortran command syntax:

```
CALL JOINAPI(IBUS, JBUS, STATUS, NID, IDTRNS, IERR)
```

Where:

Integer IBUS	retained bus number (input).	
Integer JBUS	eliminated bus number (input).	
Integer STATUS	line shunt option (input; 1 by default).	
	STATUS = 0	ignore line shunts of deleted branches
	STATUS = 1	add line shunts of deleted branches to shunt of retained bus
Integer NID	number of elements in the IDTRNS array; 0 to suppress setting IDTRNS; 0 by default	
Character*18 IDTRNS (NID)	returned as the ID translation originally connected to bus JBUS. IDTRNS contents: Columns Content 1-2 LD for load. BR for non-transformer branch. IM for induction machine MC for machine. SH for fixed bus shunt T2 for two-winding transformer. T3 for three-winding transformer. 3-8 For BR and T2 records, bus number at the other end of the branch. For T3 records, bus number of one of the other buses connected by the transformer. 9-14 For T3 records, bus number of the other bus connected by the transformer. 15-16 Old identifier. 17-18 New identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	bus not found
	IERR = 3	invalid NID value
	IERR = 4	IDTRNS is not big enough
	IERR = 5	JOIN processing error
	IERR = 6	bus is part of a substation
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.114. LAMP

Use this API to print power flow solution output, including loadings in amps, in a traditional power flow report format. Bus related information is on the left side of the report and branch related information on the right side.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LAMP SID ALL
```

Python command syntax:

```
ierr = lamp(sid, all)
```

Fortran command syntax:

```
CALL LAMPNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.115. LINES_PER_PAGE

Use this API to specify or return the page length limits for the four output devices; also retrieve device names.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_LINES_PER_PAGE IVAL(1) IVAL(2) IVAL(3) IVAL(4)
```

Python command syntax:

```
ierr = lines_per_page(ival)

ierr, ival,devnam = lines_per_page()
```

Fortran command syntax:

```
CALL LINES_PER_PAGE_API(IVAL, DEVNAM, IOCODE, IERR)
```

Where:

Integer IVAL (4)	array of 4 elements specifying the page length limits(input).	
	IVAL(1)	lines per page for disk files (no change by default)
	IVAL(2)	lines per page for the report window (no change by default)
	IVAL(3)	lines per page for the first primary hard copy device (no change by default)
	IVAL(4)	lines per page for the second primary hard copy device (no change by default)
Character*40 DEVNAM (4)	array of 4 elements specifying the four devices (when IVAL is output)(output).	
	DEVNAM(1)	'FILE
	DEVNAM(2)	'REPORT WINDOW'
	DEVNAM(3)	name of first primary hard copy device
	DEVNAM(4)	name of second primary hard copy device
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option settings to the values specified in IVAL.
	IOCODE = 1	return the current values of the option settings in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	negative IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.116. LINES_PER_PAGE_ONE_DEVICE

Use this API to specify or return the page length limit and device name for one of the four output devices.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_LINES_PER_PAGE_ONE_DEVICE DEVICE IVAL
```

Python command syntax:

```
ierr = lines_per_page_one_device(device, ival)

ierr, ival,devnam = lines_per_page_one_device(device)
```

Fortran command syntax:

```
CALL LINES_PER_PAGE_ONE_DEVICE_API(DEVICE, IVAL, DEVNAM, IOCODE, IERR)
```

Where:

Integer DEVICE	Indicates which of the four devices is to be processed (input; no default allowed).	
	DEVICE = 1	for disk files.
	DEVICE = 2	for the report window.
	DEVICE = 3	for the first primary hard copy output device.
	DEVICE = 4	for the second primary hard copy output device.
Integer IVAL	value of the lines per page option setting of the device corresponding to DEVICE (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, no change by default; else output).	
Character*40 DEVNAM	returned as the device name of the device corresponding to DEVICE (when IVAL is output) (output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	negative IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	invalid DEVICE value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.117. LINE_SHUNT_REPORTING

Use this API to specify or return the option to enable or disable the line shunt reporting.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_LINE_SHUNT_REPORTING IVAL
```

Python command syntax:

```
ierr = line_shunt_reporting(ival)

ierr, ival = line_shunt_reporting()
```

Fortran command syntax:

```
CALL LINE_SHUNT_REPORTING_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	no special reporting of line shunts in POUT, LOUT and LAMP.
	IVAL = 1	include special reporting of line shunts in POUT, LOUT and LAMP.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.118. LIST

Use this API routine to tabulate the power flow working case in a form suitable for problem data documentation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LIST SID ALL OPT VREV
```

Python command syntax:

```
ierr = list(sid, all, opt, vrev)
```

Fortran command syntax:

```
CALL LISTNW(SID, ALL, OPT, VREV, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	power flow data category to be tabulated (input; 1 by default).	
	OPT = 1	case summary
	OPT = 2	bus data
	OPT = 3	switched shunt data
	OPT = 4	plant data
	OPT = 5	generator unit data
	OPT = 6	branch data (single entry)
	OPT = 7	branch data (double entry)
	OPT = 8	two-winding transformer impedance data
	OPT = 9	line shunt data
	OPT = 10	dc line data
	OPT = 11	area interchange data
	OPT = 12	full listing (single entry)
	OPT = 13	full listing (double entry)
	OPT = 14	impedance table correction data
	OPT = 15	multi-section line data
	OPT = 16	zone data
	OPT = 17	inter-area transfer data
	OPT = 18	load data
	OPT = 19	owner data
	OPT = 20	FACTS device sending end data

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPT = 21	two-winding transformer winding data
	OPT = 22	three-winding transformer impedance data
	OPT = 23	three-winding transformer winding data
	OPT = 24	three-winding transformer control data
	OPT = 25	three-winding transformer general data
	OPT = 26	two-winding transformer control data
	OPT = 27	FACTS device terminal end data
	OPT = 28	fixed bus shunt data
	OPT = 29	GNE device data
	OPT = 30	induction machine circuit data
	OPT = 31	induction machine miscellaneous data
	OPT = 32	capability curve data
Integer VREV	units used for displaying voltages (input; 0 by default).	
	VREV = 0	use default voltage units
	VREV = 1	use alternate voltage units
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	invalid VREV value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.119. LISTCONTINGENCYSAVEDCASES

Use this API routine to obtain a report listing some or all of the system conditions preserved in a ZIP Archive Output File that was created during a previous run of one of the members of the the AC contingency calculation family.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_LISTCONTINGENCYSAVEDCASES PATHZIP LABEL
```

Python command syntax:

```
ierr = listcontingencysavedcases(pathzip, label)
```

Fortran command syntax:

```
CALL LISTCONTINGENCYSAVEDCASES(PATHZIP, LABEL, IERR)
```

Where:

Character*260 PATHZIP	Is the name of the ZIP Archive Output File (input; no default allowed).	
Character*32 LABEL	Is a label indicating the system condition(s) that are to be tabulated. blank: list all system conditions. contingency label: list all system conditions for the specified contingency label. Base Case: list all system conditions for the base case; use only for ZIP Archive Output Files written by the P-V and Q-V analysis functions. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 1	error opening the file Names.phy in the ZIP Archive Output File.
	IERR = 2	error reading the file Names.phy in the ZIP Archive Output File.
	IERR = 3	error closing the file Names.phy in the ZIP Archive Output File.
	IERR = 4	internal error.
	IERR = 5	internal error.
	IERR = 6	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.120. LLRF

Use this API to apply the line loading relief calculation using a linear network (dc) model. It must first be called once with APIOPT set to 1, then once for each priority level for which the adjustment method is to be changed with APIOPT set to 2, and finally once with APIOPT set to 3.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_LLRF APIOPT OPTIONS(1) OPTIONS(2) RVAL DFXFILE
```

Python command syntax:

```
ierr = llrf(apiopt, options, rval, dfxfile)
```

Fortran command syntax:

```
CALL LLRFAPI(APIOPT, OPTIONS, RVAL, DFXFILE, IERR)
```

Where:

Integer APIOPT	mode of operation (input).	
	APIOPT = 1	initialize
	APIOPT = 2	specify a priority level and the adjustment method to be applied; must be called once for each priority level, adjustment method pair to be specified
	APIOPT = 3	specify a line loading relief function, apply the required calculations, report results, and terminate
Integer OPTIONS (2)	array of 2 elements specifying options and values to be used in the current calculation phase of LLRFAPI; ignored when APIOPT is 1. When APIOPT is 2, the elements of OPTIONS define a priority level and the adjustment method to be used in curtailing or restoring transactions. When APIOPT is 3, the first element of OPTIONS defines the line loading relief function to be applied. For the curtail transactions and restore transactions functions, the second element of OPTIONS defines the monitored element to be used as the transaction adjustment target(input).	
	OPTIONS(1)	When APIOPT is 2, priority level When APIOPT is 3 (no default allowed)
	OPTIONS(1) = 0	curtail transactions
	OPTIONS(1) = 1	restore transactions
	OPTIONS(1) = 2	report distribution factor matrix
	OPTIONS(2)	When APIOPT is 3, monitored element list index (1 by default when OPTIONS(1) is 0 or 1).. When APIOPT is 2)
	OPTIONS(2) = 0	first in, last out (FILO)
	OPTIONS(2) = 1	decreasing order of distribution factor magnitude (DF-ORDER)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(2) = 2	distribution factor pro rata (DF-PRO RATA)
	OPTIONS(2) = 3	pro rata based on the product of distribution factor and transaction schedule (DF-SCHED-PRO RATA)
Real RVAL	used in the current calculation phase of LLRFAPI; ignored when APIOPT is 2. When APIOPT is 1, RVAL is the distribution factor tolerance used to preclude those transactions with negligible effect on the target monitored element from the transaction adjustment functions (input; 0.05 by default). When APIOPT is 3, RVAL is the target element adjustment MW (10.0 by default).	
Character*260 DFXFILE	name of DFAX data file when APIOPT is 1 (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no transactions defined
	IERR = 2	no DFAX data file specified
	IERR = 3	invalid OPTIONS value
	IERR = 4	invalid RVAL value
	IERR = 5	unexpected APIOPT value
	IERR = 6	invalid APIOPT value
	IERR = 7	error opening file DFXFILE
	IERR = 8	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 10	no monitored branches or interfaces defined
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.121. LOAD_REDUCTION

Use this API to tabulate the amount of load reduction in a specified subsystem due to the voltage at the bus to which the load is connected being below PQBRAK (for constant MVA load) or 0.5 (for constant current load.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_LOAD_REDUCTION SID ALL AOZOPT
```

Python command syntax:

```
ierr = load_reduction(sid, all, aozopt)
```

Fortran command syntax:

```
CALL LOAD_REDUCTION(SID, ALL, AOZOPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only loads in subsystem SID
	ALL = 1	process all loads
Integer AOZOPT	subsystem selection option when area, owner and/or zone are included in the subsystem selection criteria (input; 1 by default).	
	AOZOPT = 1	use the area, owner and/or zone assignments of the bus to which the load is connected
	AOZOPT = 2	use the area, owner and/or zone assignments of the load
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid AOZOPT value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.122. LOUT

Use this API to print the power flow solution results in a traditional power flow report format. Bus related information is on the left side of the report and branch related information on the right side.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LOUT SID ALL
```

Python command syntax:

```
ierr = lout(sid, all)
```

Fortran command syntax:

```
CALL LOUTNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.123. LTAP

Use this API to insert a bus at a designated location along a line.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LTAP  FRMBUS  TOBUS  'CKT'  FRACTION  NEWNUM  'NEWNAM'  NEWKV
```

Python command syntax:

```
ierr = ltap(frmbus, tobus, ckt, fraction, newnum, newnam, newkv)
```

Fortran command syntax:

```
CALL LTAPAPI(FRMBUS, TOBUS, CKT, FRACTION, NEWNUM, NEWNAM, NEWKV, IERR)
```

Where:

Integer FRMBUS	one of the buses connected by the branch to be tapped (input; no default allowed).	
Integer TOBUS	other bus connected by the branch to be tapped (input; no default allowed).	
Character*2 CKT	circuit identifier of the branch to be tapped (input; '1' by default).	
Real FRACTION	new bus location relative to bus FRMBUS in per unit of total line length (input; 0.5).	
Integer NEWNUM	number to be assigned to the new bus (input; no default allowed).	
Character*12 NEWNAM	name to be assigned to the new bus (input; blank by default).	
Real NEWKV	base voltage to be assigned to the new bus (input; base voltage of bus FRMBUS).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	branch not found
	IERR = 3	branch is a transformer
	IERR = 4	invalid FRACTION value
	IERR = 5	invalid NEWNUM value
	IERR = 6	bus number is already assigned to another bus
	IERR = 7	invalid NEWKV value
	IERR = 8	bus, branch, or branch ownership tables full
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.124. MACCC

This API routine is obsolete. It has been replaced by the API routine maccc_2 .

Use this API to run multiple level contingency analysis. User-specified and automatically-selected contingencies from built-in contingency ranking functions are evaluated individually and in combination with each other as overlapping outages up to the specified evaluation level.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MACCC  TOL  OPTACC(1)..OPTACC(8)  OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17)  VALUES(1)..VALUES(3)  LABELS(1)  LABELS(2) DFXFILE
  ACCFILE  THRFILE  INLFILE
```

Python command syntax:

```
ierr = maccc(tol, optacc, optmac, optrnk, values, labels, dfxfile,
 accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCCAPI(TOL, OPTACC, OPTMAC, OPTRNK, VALUES, LABELS, DFXFILE,
 ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1<=OPTMAC (2)<=3; 1 by default)
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single machine and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations, otherwise
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTRNK(2)	contingency rating (rating set program option setting by default)
OPTRNK(2) = 1	rate A
OPTRNK(2) = 2	rate B
OPTRNK(2) = 3	rate C
OPTRNK(3)	convert ratings to MW (0 by default)
OPTRNK(3) = 0	disable
OPTRNK(3) = 1	enable
OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
OPTRNK(4) = 0	disable
OPTRNK(4) = 1	enable
OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
OPTRNK(7)	non-failure cutoff of machine contingencies (>=0; 5 by default)
OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
OPTRNK(8) = 0	disable
OPTRNK(8) = 1	enable
OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
OPTRNK(12) = 0	disable
OPTRNK(12) = 1	enable
OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
OPTRNK(16)	include all branches in voltage depression ranking (0 by default)
OPTRNK(16) = 0	include only with non-zero rating

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Real VALUES (3)	array of 3 elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	percent of rating set (0<= VALUES (1)<=200.0; 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (VALUES (3) >=0.0; 0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTMAC value
	IERR = 9	invalid OPTRNK value
	IERR = 10	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 50	error opening Unit Inertia and Governor Data File

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.125. MACCC_2

Use this API to run the second release of multiple level contingency analysis. User-specified and automatically-selected contingencies from built-in contingency ranking functions are evaluated individually and in combination with each other as overlapping outages up to the specified evaluation level.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MACCC_2  TOL  OPTACC(1)..OPTACC(10)  OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17)  VALUES(1)..VALUES(3)  LABELS(1)  LABELS(2) DFXFILE
  ACCFILE  THRFILE  INLFILE
```

Python command syntax:

```
ierr = maccc_2(tol, optacc, optmac, optrnk, values, labels, dfxfile,
 accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCCAPI_2(TOL, OPTACC, OPTMAC, OPTRNK, VALUES, LABELS, DFXFILE,
 ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTACC(5) = 0	disable
OPTACC(5) = 1	enable
OPTACC(5) = 2	enable continuous mode, disable discrete mode
OPTACC(6)	solution method flag (0 by default)
OPTACC(6) = 0	FDNS
OPTACC(6) = 1	FNSL
OPTACC(6) = 2	optimized FDNS
OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTACC(7) = 0	disable
OPTACC(7) = 1	enable
OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTACC(8) = 0	stall
OPTACC(8) = 1	trip
OPTACC(9)	induction machine failure flag (0 by default)
OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(10)	dispatch mode (0 by default)
OPTACC(10) = 0	disable
OPTACC(10) = 1	subsystem machines (reserve)
OPTACC(10) = 2	subsystem machines (pmax)
OPTACC(10) = 3	subsystem machines (inertia)
OPTACC(10) = 4	subsystem machines (governor droop)

Integer OPTMAC (41) array of 41 elements specifying contingency evaluation level and combinations(input).

OPTMAC(1)	run multiple contingency analysis (1 by default)
OPTMAC(1) = 0	disable
OPTMAC(1) = 1	enable
OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels)
OPTMAC(2) = 1	N-1 contingency analysis
OPTMAC(2) = 2	N-2 contingency analysis
OPTMAC(2) = -2	N-2 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single machine and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations, otherwise
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A
	OPTRNK(2) = 2	rate B
	OPTRNK(2) = 3	rate C
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable
	OPTRNK(3) = 1	enable
	OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(4) = 0	disable
	OPTRNK(4) = 1	enable
	OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(7)	non-failure cutoff of machine contingencies (>=0; 5 by default)
	OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(8) = 0	disable
	OPTRNK(8) = 1	enable
	OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
	OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(12) = 0	disable
	OPTRNK(12) = 1	enable
	OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
	OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to eval-uate (>=0; 10 by default)
	OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(16)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Real VALUES (3)	array of 3 elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	percent of rating set (0<= VALUES (1)<=200.0; 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0.0; 0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	invalid OPTMAC value
	IERR = 11	invalid OPTRNK value
	IERR = 12	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 50	error opening Unit Inertia and Governor Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.126. MACCC_3

Use this API to run the third release of multiple level contingency analysis. User-specified and automatically-selected contingencies from built-in contingency ranking functions are evaluated individually and in combination with each other as overlapping outages up to the specified evaluation level.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MACCC_2  TOL  OPTACC(1)..OPTACC(10)  OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17)  VALUES(1)..VALUES(2)  LABELS(1)  LABELS(2) DFXFILE
  ACCFILE  THRFILE  INLFILE
```

Python command syntax:

```
ierr = maccc_3(tol, optacc, optmac, optrnk, values, labels, dfxfile,
 accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCCAPI_3(TOL, OPTACC, OPTMAC, OPTRNK, VALUES, LABELS, DFXFILE,
 ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (reserve)
	OPTACC(10) = 2	subsystem machines (pmax)
	OPTACC(10) = 3	subsystem machines (inertia)
	OPTACC(10) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels)
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(3)	Each element from 3 to 41 of OPTMAC array represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit (machine) and single branch outages respectively
	OPTMAC(3) = 0	(0 by default)
	OPTMAC(3) = 1	include the contingency combinations
Integer OPTRNK (15)	array of 15 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(2) = 0	disable
	OPTRNK(2) = 1	enable
	OPTRNK(3)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(4)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(5)	non-failure cutoff of machine contingencies (>=0; 5 by default)
	OPTRNK(6)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(6) = 0	disable
	OPTRNK(6) = 1	enable
	OPTRNK(7)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
	OPTRNK(8)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
	OPTRNK(9)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(10)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(10) = 0	disable
	OPTRNK(10) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(11)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
	OPTRNK(12)	maximum of branch contingencies from voltage depression ranking to eval-uate (>=0; 10 by default)
	OPTRNK(13)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(14)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(14) = 0	include only with non-zero rating
	OPTRNK(14) = 1	all
	OPTRNK(15)	include contingencies that create swingless island (0 by default)
	OPTRNK(15) = 0	disable
	OPTRNK(15) = 1	enable
Real VALUES (2)	array of 2 elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	MW threshold for islanding contingencies (>=0.0; 0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value
IERR = 11	invalid OPTRNK value
IERR = 12	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.127. MACCC_PARALLEL

Use this API to run the multiple level contingency analysis in parallel when the number of contingency processors in Program Settings is greater than 1. User-specified and automatically selected contingencies from built-in contingency ranking functions are evaluated individually and in combination with each other as overlapping outages up to the specified evaluation level.

This API routine was first introduced in release 33.3.0.

Batch command syntax:

```
BAT_MACCC_PARALLEL  TOL OPTACC(1)..OPTACC(10) OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) VALUES(1)..VALUES(3) LABELS(1)..LABELS(2) DFXFILE
 ACCFILE THRFILE INLFILE
```

Python command syntax:

```
ierr = maccc_parallel(tol, optacc, optmac, optrnk, values, labels,
 dfxfile, accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCC_PARALLEL(TOL, OPTACC, OPTMAC, OPTRNK, VALUES, LABELS, DFXFILE,
 ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	Is the mismatch tolerance (input; Newton solution convergence tolerance, TOLN, by default).	
Integer OPTACC (10)	array of elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable.
	OPTACC(1) = 1	enable stepping adjustment.
	OPTACC(1) = 2	enable direct adjustment.
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable.
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange.
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange.
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable.
	OPTACC(3) = 1	enable.
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable.
	OPTACC(4) = 1	enable.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable.
	OPTACC(5) = 1	enable.
	OPTACC(5) = 2	enable continuous mode, disable discrete mode.
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS.
	OPTACC(6) = 1	FNSL.
	OPTACC(6) = 2	optimized FDNS.
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable.
	OPTACC(7) = 1	enable.
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default) (applied when an induction motor fails to solve due to low by default)
	OPTACC(8) = 0	stall.
	OPTACC(8) = 1	trip.
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state.
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable.
	OPTACC(10) = 1	subsystem machines (reserve).
	OPTACC(10) = 2	subsystem machines (pmax).
	OPTACC(10) = 3	subsystem machines (inertia).
	OPTACC(10) = 4	subsystem machines (governor droop).
Integer OPTMAC (41)	array of elements specifying contingency evaluation level and combinations (input). The values are as follows:(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable.
	OPTMAC(1) = 1	enable.
	OPTMAC(2)	contingency evaluation level.
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(3)	each element represents a type of contingency combinations in the following order. (0 by default)
	OPTMAC(3) = 0	do not include the contigency combination
	OPTMAC(3) = 1	include the "S" contingency combination. This and the remaining elements of the array (i.e. 3-41) individually indicate whether particular contingency combinations are to be included. In the following S, U and B refer to user specified cases, single machine and single branch outages, respectively: {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}
Integer OPTRNK (17)	array of elements specifying ranking options. The values are as follows:(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable.
	OPTRNK(1) = 1	enable.
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A.
	OPTRNK(2) = 2	rate B.
	OPTRNK(2) = 3	rate C.
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable.
	OPTRNK(3) = 1	enable.
	OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(4) = 0	disable.
	OPTRNK(4) = 1	enable.
	OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(7)	non-failure cutoff of machine contingencies (>=0; 5 by default)
	OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(8) = 0	disable.
	OPTRNK(8) = 1	enable.
	OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
	OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(12) = 0	disable.
	OPTRNK(12) = 1	enable.
	OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
	OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
	OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(16)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating.
	OPTRNK(16) = 1	all.
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable.
	OPTRNK(17) = 1	enable.
Real VALUES (3)	array of elements specifying ranking parameters.The values are as follows: (input).	
	VALUES(1)	small reactance threshold (0<= VALUES(1)<=0.005; 0.0005 by default).
	VALUES(2)	percent of rating set (0<= VALUES(1)<=200.0; 100.0 by default).
	VALUES(3)	MW threshold for islanding contingencies (>=0.0; 0.0 by default).
Character*32 LABELS (2)	array of elements specifying subsystem labels. The values are as follows:(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0).
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	Is the name of the Contingency Solution Output File (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 THRFILE	Is the name of the Load Throwover Data File (input; blank by default).
Character*260 INLFILE	Is the name of the Unit Inertia and Governor Data File (input; blank by default).
Integer IERR	error code (output).

IERR = 0	no error occurred.
IERR = 1	invalid TOL value
IERR = 2	invalid OPTACC value
IERR = 3	generators are converted.
IERR = 4	buses in island(s) without a swing bus; use activity TREE.
IERR = 5	largest mismatch exceeds mismatch tolerance.
IERR = 6	generation subsystem is not defined.
IERR = 7	too many islands in base case.
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value.
IERR = 11	invalid OPTRNK value.
IERR = 12	invalid VALUES value.
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.128. MACCC_PARALLEL_2

Use this API to run the multiple level contingency analysis in parallel when the number of contingency processors in Program Settings is greater than 1. User-specified and automatically selected contingencies from built-in contingency ranking functions are evaluated individually and in combination with each other as overlapping outages up to the specified evaluation level.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MACCC_PARALLEL_2  TOL OPTACC(1)..OPTACC(10) OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) VALUES(1)..VALUES(3) LABELS(1)..LABELS(2) DFXFILE
 ACCFILE THRFILE INLFILE
```

Python command syntax:

```
ierr = maccc_parallel_2(tol, optacc, optmac, optrnk, values, labels,
 dfxfile, accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCC_PARALLEL_2(TOL, OPTACC, OPTMAC, OPTRNK, VALUES, LABELS,
 DFXFILE, ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	Is the mismatch tolerance (input; Newton solution convergence tolerance, TOLN, by default).	
Integer OPTACC (10)	array of elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable.
	OPTACC(1) = 1	enable stepping adjustment.
	OPTACC(1) = 2	enable direct adjustment.
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable.
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange.
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange.
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable.
	OPTACC(3) = 1	enable.
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable.
	OPTACC(4) = 1	enable.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable.
	OPTACC(5) = 1	enable.
	OPTACC(5) = 2	enable continuous mode, disable discrete mode.
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS.
	OPTACC(6) = 1	FNSL.
	OPTACC(6) = 2	optimized FDNS.
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable.
	OPTACC(7) = 1	enable.
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default) (applied when an induction motor fails to solve due to low by default)
	OPTACC(8) = 0	stall.
	OPTACC(8) = 1	trip.
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state.
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable.
	OPTACC(10) = 1	subsystem machines (reserve).
	OPTACC(10) = 2	subsystem machines (pmax).
	OPTACC(10) = 3	subsystem machines (inertia).
	OPTACC(10) = 4	subsystem machines (governor droop).
Integer OPTMAC (41)	array of elements specifying contingency evaluation level and combinations (input). The values are as follows:(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable.
	OPTMAC(1) = 1	enable.
	OPTMAC(2)	contingency evaluation level.
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(3)	Each element from 3 to 41 of OPTMAC array represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit (machine) and single branch outages respectively
	OPTMAC(3) = 0	(0 by default)
	OPTMAC(3) = 1	include the contingency combinations
Integer OPTRNK (15)	array of elements specifying ranking options. The values are as follows:(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable.
	OPTRNK(1) = 1	enable.
	OPTRNK(2)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(2) = 0	disable.
	OPTRNK(2) = 1	enable.
	OPTRNK(3)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(4)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(5)	non-failure cutoff of machine contingencies (>=0; 5 by default)
	OPTRNK(6)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(6) = 0	disable.
	OPTRNK(6) = 1	enable.
	OPTRNK(7)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
	OPTRNK(8)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
	OPTRNK(9)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(10)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(10) = 0	disable.
	OPTRNK(10) = 1	enable.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(11)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
	OPTRNK(12)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
	OPTRNK(13)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(14)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(14) = 0	include only with non-zero rating.
	OPTRNK(14) = 1	all.
	OPTRNK(15)	include contingencies that create swingless island (0 by default)
	OPTRNK(15) = 0	disable.
	OPTRNK(15) = 1	enable.
Real VALUES (2)	array of elements specifying ranking parameters.The values are as follows: (input).	
	VALUES(1)	small reactance threshold (0<= VALUES(1)<=0.005; 0.0005 by default).
	VALUES(2)	MW threshold for islanding contingencies (>=0.0; 0.0 by default).
Character*32 LABELS (2)	array of elements specifying subsystem labels. The values are as follows:(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0).
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	Is the name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	Is the name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	Is the name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted.
	IERR = 4	buses in island(s) without a swing bus; use activity TREE.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	largest mismatch exceeds mismatch tolerance.
IERR = 6	generation subsystem is not defined.
IERR = 7	too many islands in base case.
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value.
IERR = 11	invalid OPTRNK value.
IERR = 12	invalid VALUES value.
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.129. MACCC_TRIP_COR

This API routine is obsolete. It has been replaced by the API routine maccc_trip_cor_2 .

Use this API to run multiple AC contingency analysis with tripping simulations and corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination with each other as overlapping contingencies. When there are violations in post-tripping simulation solutions, the corrective actions analysis will be launched to eliminate the violations, with the objective of minimizing the controls adjustments, including load shedding, MW generation dispatch and phase shifter angle adjustments.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_MACCC_TRIP_COR  TOL OPTACC(1)..OPTACC(8) OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) OPTTRP(1)..OPTTRP(3) OPTCOR(1)..OPTCOR(5)
 VALUES(1)..VALUES(8)  LABELS(1)..LABELS(5)  DFXFILE  ACCFILE  THRFILE
 INLFILE TRPFILE
```

Python command syntax:

```
ierr = maccc_trip_cor(tol, optacc, optmac, optrnk, opttrp, optcor, values,
 labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_TRIP_COR(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, OPTCOR, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1<=OPTMAC (2)<=3; 1 by default)
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single machine and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations, otherwise
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTRNK(1)	include ties from contingency subsystem (0 by default)
OPTRNK(1) = 0	disable
OPTRNK(1) = 1	enable
OPTRNK(2)	contingency rating (rating set program option setting by default)
OPTRNK(2) = 1	rate A
OPTRNK(2) = 2	rate B
OPTRNK(2) = 3	rate C
OPTRNK(3)	convert ratings to MW (0 by default)
OPTRNK(3) = 0	disable
OPTRNK(3) = 1	enable
OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
OPTRNK(4) = 0	disable
OPTRNK(4) = 1	enable
OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
OPTRNK(7)	non-failure cutoff of machine contingencies (>=0; 5 by default)
OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
OPTRNK(8) = 0	disable
OPTRNK(8) = 1	enable
OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
OPTRNK(12) = 0	disable
OPTRNK(12) = 1	enable
OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(16)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Integer OPTCOR (5)	array of 5 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective actions (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flow solutions (from 0 to 10; 1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
Real VALUES (8)	array of 8 elements specifying ranking and corrective action parameters(input).	
	VALUES(1)	small reactance threshold (0<=VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	percent of rating (0<=VALUES (1)<=200.0; 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0.0; 0.0 by default)
	VALUES(4)	bus voltage violation tolerance (0.1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(5)	branch flow overload tolerance (0.1 by default)
	VALUES(6)	generator control weighting factor (1.0 by default)
	VALUES(7)	load control weighting factor (1.0 by default)
	VALUES(8)	phase shifter control weighting factor (1.0 by default)
Character*32 LABELS (5)	array of 5 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
	LABELS(3)	label of generator control subsystem (blank by default; no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default; no default allowed if OPTCOR(4) is 1)
	LABELS(5)	label of phase shifter control subsystem (blank by default; no default allowed if OPTCOR(5) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTMAC value
	IERR = 9	invalid OPTRNK value
	IERR = 10	invalid OPTTRP value
	IERR = 11	invalid OPTCOR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 12	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.130. MACCC_TRIP_COR_2

This API routine is obsolete. It has been replaced by the API routine maccc_trip_cor_3 .

This API is the second release of function to run multiple AC contingency analysis with tripping simulations and corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination each other as overlapping contingencies. Following a post contingency solution, trip events are simulated if their tripping conditions are satisfied. Corrective actions are then specified automatically to eliminate the violations in each post tripping solution with the objective of minimizing the control adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_MACCC_TRIP_COR_2  TOL  OPTACC(1)..OPTACC(8) OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) OPTTRP(1)..OPTTRP(3) OPTCOR(1)..OPTCOR(8)
 VALUES(1)..VALUES(11)  LABELS(1)..LABELS(8) DFXFILE ACCFILE THRFILE
 INLFILE TRPFILE
```

Python command syntax:

```
ierr = maccc_trip_cor_2(tol, optacc, optmac, optrnk, opttrp, optcor,
 values, labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_TRIP_COR_2(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, OPTCOR, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (RESERVE)
	OPTACC(8) = 2	subsystem machines (PMAX)
	OPTACC(8) = 3	subsystem machines (INERTIA)
	OPTACC(8) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	allow multiple contingency analysis (0 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(1) = 2	enabled with failure cutoff disabled
	OPTMAC(2)	contingency evaluation level (1 by default)
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A
	OPTRNK(2) = 2	rate B
	OPTRNK(2) = 3	rate C
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable
	OPTRNK(3) = 1	enable
	OPTRNK(4)	machine overload contingencies (0 by default)
	OPTRNK(4) = 0	disable
	OPTRNK(4) = 1	enable
	OPTRNK(5)	minimum of machine overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(6)	maximum of machine overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(7)	cutoff of machine overload contingencies to evaluate (>=0, 5 by default)
	OPTRNK(8)	line overload contingencies (0 by default)
	OPTRNK(8) = 0	disable
	OPTRNK(8) = 1	enable
	OPTRNK(9)	minimum of line overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(10)	maximum of line overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(11)	cutoff of line overload contingencies (>=0, 5 by default)
	OPTRNK(12)	voltage collapse line contingencies (0 by default)
	OPTRNK(12) = 0	disable
	OPTRNK(12) = 1	enable
	OPTRNK(13)	minimum of voltage collapse line contingencies to evaluate (>=0, 1 by default)
	OPTRNK(14)	maximum of voltage collapse line contingencies to evaluate (>=0, 10 by default)
	OPTRNK(15)	cutoff of voltage collapse line contingencies (>=0, 5 be default)
	OPTRNK(16)	include all branches in voltage collapse ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (11)	array of 11 elements specifying ranking parameters and corrective actions parameters(input).	
	VALUES(1)	small reactance threshold, (0<= VALUES (1)<=0.005, 0.0005 by default)
	VALUES(2)	percent of flow rating, (0<= VALUES (2)<=200.0, 100.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(3)	MW threshold for islanding contingencies (>=0, 0.0 by default)
	VALUES(4)	bus voltage violation tolerance (0.1 by default)
	VALUES(5)	branch overload violation tolerance (0.1 by default)
	VALUES(6)	generator control weighting factor (1.0 by default)
	VALUES(7)	load control weighting factor (1.0 by default)
	VALUES(8)	phase shifter control weighting factor (1.0 by default)
	VALUES(9)	off-line generator control weighting factor (1.0 by default)
	VALUES(10)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(11)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (8)	array of 8 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
	LABELS(3)	label of generator control subsystem (blank by default for none, no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(5)	label of phase shifter control subsystem (blank by default for none, no default allowed if OPTCOR(5) is 1)
	LABELS(6)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(7)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(8)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank for none).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTMAC value
	IERR = 9	invalid OPTRNK value
	IERR = 10	invalid OPTTRP value
	IERR = 11	invalid OPTCOR value
	IERR = 12	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multisection line members
	IERR = 49	error opening Tripping Element Data File
	IERR = 50	error opening Unit Inertia Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.131. MACCC_TRIP_COR_3

This API is the third release of function to run multiple AC contingency analysis with tripping simulations and corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination each other as overlapping contingencies. Following a post contingency solution, trip events are simulated if their tripping conditions are satisfied. Corrective actions are then specified automatically to eliminate the violations in each post tripping solution with the objective of minimizing the control adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MACCC_TRIP_COR_3  TOL  OPTACC(1)..OPTACC(10) OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) OPTTRP(1)..OPTTRP(3) OPTCOR(1)..OPTCOR(8)
 VALUES(1)..VALUES(11)  LABELS(1)..LABELS(8) DFXFILE ACCFILE THRFILE
 INLFILE TRPFILE
```

Python command syntax:

```
ierr = maccc_trip_cor_3(tol, optacc, optmac, optrnk, opttrp, optcor,
 values, labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_TRIP_COR_3(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, OPTCOR, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (RESERVE)
	OPTACC(10) = 2	subsystem machines (PMAX)
	OPTACC(10) = 3	subsystem machines (INERTIA)
	OPTACC(10) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	allow multiple contingency analysis (0 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(1) = 2	enabled with failure cutoff disabled
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3,

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		the event list at the first level is used for any higher levels)
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A
	OPTRNK(2) = 2	rate B
	OPTRNK(2) = 3	rate C
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable
	OPTRNK(3) = 1	enable
	OPTRNK(4)	machine overload contingencies (0 by default)
	OPTRNK(4) = 0	disable
	OPTRNK(4) = 1	enable
	OPTRNK(5)	minimum of machine overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(6)	maximum of machine overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(7)	cutoff of machine overload contingencies to evaluate (>=0, 5 by default)
	OPTRNK(8)	line overload contingencies (0 by default)
	OPTRNK(8) = 0	disable
	OPTRNK(8) = 1	enable
	OPTRNK(9)	minimum of line overload contingencies to evaluate (>=0, 1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(10)	maximum of line overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(11)	cutoff of line overload contingencies (>=0, 5 by default)
	OPTRNK(12)	voltage collapse line contingencies (0 by default)
	OPTRNK(12) = 0	disable
	OPTRNK(12) = 1	enable
	OPTRNK(13)	minimum of voltage collapse line contingencies to evaluate (>=0, 1 by default)
	OPTRNK(14)	maximum of voltage collapse line contingencies to evaluate (>=0, 10 by default)
	OPTRNK(15)	cutoff of voltage collapse line contingencies (>=0, 5 be default)
	OPTRNK(16)	include all branches in voltage collapse ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (11)	array of 11 elements specifying ranking parameters and corrective actions parameters(input).	
	VALUES(1)	small reactance threshold, (0<= VALUES (1)<=0.005, 0.0005 by default)
	VALUES(2)	percent of flow rating, (0<= VALUES (2)<=200.0, 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0, 0.0 by default)
	VALUES(4)	bus voltage violation tolerance (0.1 by default)
	VALUES(5)	branch overload violation tolerance (0.1 by default)
	VALUES(6)	generator control weighting factor (1.0 by default)
	VALUES(7)	load control weighting factor (1.0 by default)
	VALUES(8)	phase shifter control weighting factor (1.0 by default)
	VALUES(9)	off-line generator control weighting factor (1.0 by default)
	VALUES(10)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(11)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (8)	array of 8 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
	LABELS(3)	label of generator control subsystem (blank by default for none, no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(5)	label of phase shifter control subsystem (blank by default for none, no default allowed if OPTCOR(5) is 1)
	LABELS(6)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(7)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(8)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank for none).	
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	invalid OPTMAC value
	IERR = 11	invalid OPTRNK value
	IERR = 12	invalid OPTTRP value
	IERR = 13	invalid OPTCOR value
	IERR = 14	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multisection line members
	IERR = 49	error opening Tripping Element Data File
	IERR = 50	error opening Unit Inertia Data File

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.132. MACCC_TRIP_COR_4

This API is the third release of function to run multiple AC contingency analysis with tripping simulations and corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination each other as overlapping contingencies. Following a post contingency solution, trip events are simulated if their tripping conditions are satisfied. Corrective actions are then specified automatically to eliminate the violations in each post tripping solution with the objective of minimizing the control adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MACCC_TRIP_COR_4  TOL  OPTACC(1)..OPTACC(10) OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) OPTTRP(1)..OPTTRP(3) OPTCOR(1)..OPTCOR(8)
 VALUES(1)..VALUES(11)  LABELS(1)..LABELS(8) DFXFILE ACCFILE THRFILE
 INLFILE TRPFILE
```

Python command syntax:

```
ierr = maccc_trip_cor_4(tol, optacc, optmac, optrnk, opttrp, optcor,
 values, labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_TRIP_COR_4(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, OPTCOR, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (RESERVE)
	OPTACC(10) = 2	subsystem machines (PMAX)
	OPTACC(10) = 3	subsystem machines (INERTIA)
	OPTACC(10) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	allow multiple contingency analysis (0 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(1) = 2	enabled with failure cutoff disabled
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3,

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		the event list at the first level is used for any higher levels)
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(3)	Each element from 3 to 41 of OPTMAC array represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit (machine) and single branch outages respectively
	OPTMAC(3) = 0	(0 by default)
	OPTMAC(3) = 1	include the contingency combinations
Integer OPTRNK (15)	array of 15 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	machine overload contingencies (0 by default)
	OPTRNK(2) = 0	disable
	OPTRNK(2) = 1	enable
	OPTRNK(3)	minimum of machine overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(4)	maximum of machine overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(5)	cutoff of machine overload contingencies to evaluate (>=0, 5 by default)
	OPTRNK(6)	line overload contingencies (0 by default)
	OPTRNK(6) = 0	disable
	OPTRNK(6) = 1	enable
	OPTRNK(7)	minimum of line overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(8)	maximum of line overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(9)	cutoff of line overload contingencies (>=0, 5 by default)
	OPTRNK(10)	voltage collapse line contingencies (0 by default)
	OPTRNK(10) = 0	disable
	OPTRNK(10) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(11)	minimum of voltage collapse line contingencies to evaluate (>=0, 1 by default)
	OPTRNK(12)	maximum of voltage collapse line contingencies to evaluate (>=0, 10 by default)
	OPTRNK(13)	cutoff of voltage collapse line contingencies (>=0, 5 be default)
	OPTRNK(14)	include all branches in voltage collapse ranking (0 by default)
	OPTRNK(14) = 0	include only with non-zero rating
	OPTRNK(14) = 1	all
	OPTRNK(15)	include contingencies that create swingless island (0 by default)
	OPTRNK(15) = 0	disable
	OPTRNK(15) = 1	enable
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable tripping simulation
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(8) = 1	enable
Real VALUES (10)	array of 10 elements specifying ranking parameters and corrective actions parameters(input).	
	VALUES(1)	small reactance threshold, (0<= VALUES (1)<=0.005, 0.0005 by default)
	VALUES(2)	MW threshold for islanding contingencies (>=0, 0.0 by default)
	VALUES(3)	bus voltage violation tolerance (0.1 by default)
	VALUES(4)	branch overload violation tolerance (0.1 by default)
	VALUES(5)	generator control weighting factor (1.0 by default)
	VALUES(6)	load control weighting factor (1.0 by default)
	VALUES(7)	phase shifter control weighting factor (1.0 by default)
	VALUES(8)	off-line generator control weighting factor (1.0 by default)
	VALUES(9)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(10)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (8)	array of 8 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
	LABELS(3)	label of generator control subsystem (blank by default for none, no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(5)	label of phase shifter control subsystem (blank by default for none, no default allowed if OPTCOR(5) is 1)
	LABELS(6)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(7)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(8)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank for none).
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTACC value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value
IERR = 11	invalid OPTRNK value
IERR = 12	invalid OPTTRP value
IERR = 13	invalid OPTCOR value
IERR = 14	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multisection line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.133. MACCC_WITH_COR

This API routine is obsolete. It has been replaced by the API routine maccc_with_cor_2 .

Use this API to run multiple AC contingency analysis with corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination with each other as overlapping contingencies. The corrective actions analysis will be launched for the contingencies with flow and voltage violations to eliminate the violations with the objective of minimizing the controls adjustments including load shedding, MW generation dispatch and phase shifter angle adjustments.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_MACCC_WITH_COR  TOL  OPTACC(1)..OPTACC(8) OPTMAC(1)..OPTMAC(41)
 OPTRNK(1)..OPTRNK(17)  OPTCOR(1)..OPTCOR(5) VALUES(1)..VALUES(8)
 LABELS(1)..LABELS(5)  DFXFILE  ACCFILE  THRFILE  INLFILE
```

Python command syntax:

```
ierr = maccc_with_cor(tol, optacc, optmac, optrnk, optcor, values, labels,
 dfxfile, accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_COR(TOL, OPTACC, OPTMAC, OPTRNK, OPTCOR, VALUES, LABELS,
 DFXFILE, ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1<=OPTMAC(2)<=3; 1 by default)
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single machine and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying contingency ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTRNK(1) = 1	enable
OPTRNK(2)	contingency rating (rating set program option setting by default)
OPTRNK(2) = 1	rate A
OPTRNK(2) = 2	rate B
OPTRNK(2) = 3	rate C
OPTRNK(3)	convert ratings to MW (0 by default)
OPTRNK(3) = 0	disable
OPTRNK(3) = 1	enable
OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
OPTRNK(4) = 0	disable
OPTRNK(4) = 1	enable
OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
OPTRNK(7)	non-failure cutoff of machine contingencies (>=0)
OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
OPTRNK(8) = 0	disable
OPTRNK(8) = 1	enable
OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
OPTRNK(12) = 0	disable
OPTRNK(12) = 1	enable
OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
OPTRNK(16)	include all branches in voltage depression ranking (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTCOR (5)	array of 5 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective actions (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number of power flow solutions (from 0 to 10; 1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
Real VALUES (8)	array of 8 elements specifying ranking and corrective actions parameters(input).	
	VALUES(1)	small reactance threshold (0<=VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	percent of rating set (0<=VALUES (1)<=200.0; 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0.0; 0.0 by default)
	VALUES(4)	bus voltage violation tolerance (0.1 by default)
	VALUES(5)	branch flow overload tolerance (0.1 by default)
	VALUES(6)	generator control weighting factor (1.0 by default)
	VALUES(7)	load control weighting factor (1.0 by default)
	VALUES(8)	phase shifter control weighting factor (1.0 by default)
Character*32 LABELS (5)	array of 5 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(3)	label of generator control subsystem (blank by default; no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default; no default allowed if OPTCOR(4) is 1)
	LABELS(5)	label of phase shifter control subsystem (blank by default; no default allowed if OPTCOR(5) is 1)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTMAC value
	IERR = 9	invalid OPTRNK value
	IERR = 10	invalid OPTCOR value
	IERR = 11	invalid VALUES value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 50	error opening Unit Inertia and Governor Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.134. MACCC_WITH_COR_2

This API routine is obsolete. It has been replaced by the API routine maccc_with_cor_3 .

This API is the second release of function to run multiple AC contingency analysis with corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination each other as overlapping contingencies. Corrective actions are specified automatically to eliminate the violations in each post contingency solution with the objective of minimizing the controls adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_MACCC_WITH_COR_2  TOL  OPTACC(1)..OPTACC(8) OPTMAC(1).. OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) OPTCOR(1)..OPTCOR(8) VALUES(1)..VALUES(11)
 LABELS(1)..LABELS(8) DFXFILE ACCFILE THRFILE INLFILE
```

Python command syntax:

```
ierr = maccc_with_cor_2(tol, optacc, optmac, optrnk, optcor, values,
 labels, dfxfile, accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_COR_2(TOL, OPTACC, OPTMAC, OPTRNK, OPTCOR, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (RESERVE)
	OPTACC(8) = 2	subsystem machines (PMAX)
	OPTACC(8) = 3	subsystem machines (INERTIA)
	OPTACC(8) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	allow multiple contingency analysis (0 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(1) = 2	enabled with failure cutoff disabled
	OPTMAC(2)	contingency evaluation level (1 by default)
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations
	OPTMAC(3) = 0	(0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A
	OPTRNK(2) = 2	rate B
	OPTRNK(2) = 3	rate C
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable
	OPTRNK(3) = 1	enable
	OPTRNK(4)	machine overload contingencies (0 by default)
	OPTRNK(4) = 0	disable
	OPTRNK(4) = 1	enable
	OPTRNK(5)	minimum of machine overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(6)	maximum of machine overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(7)	cutoff of machine overload contingencies to evaluate (>=0, 5 by default)
	OPTRNK(8)	line overload contingencies (0 by default)
	OPTRNK(8) = 0	disable
	OPTRNK(8) = 1	enable
	OPTRNK(9)	minimum of line overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(10)	maximum of line overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(11)	cutoff of line overload contingencies (>=0, 5 by default)
	OPTRNK(12)	voltage collapse line contingencies (0 by default)
	OPTRNK(12) = 0	disable
	OPTRNK(12) = 1	enable
	OPTRNK(13)	minimum of voltage collapse line contingencies to evaluate (>=0, 1 by default)
	OPTRNK(14)	maximum of voltage collapse line contingencies to evaluate (>=0, 10 by default)
	OPTRNK(15)	cutoff of voltage collapse line contingencies (>=0, 5 be default)
	OPTRNK(16)	include all branches in voltage collapse ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (11)	array of 11 elements specifying ranking parameters and corrective actions parameters(input).	
	VALUES(1)	small reactance threshold, (0<= VALUES (1)<=0.005, 0.0005 by default)
	VALUES(2)	percent of flow rating, (0<= VALUES (2)<=200.0, 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0, 0.0 by default)
	VALUES(4)	bus voltage violation tolerance (0.1 by default)
	VALUES(5)	branch overload violation tolerance (0.1 by default)
	VALUES(6)	generator control weighting factor (1.0 by default)
	VALUES(7)	load control weighting factor (1.0 by default)
	VALUES(8)	phase shifter control weighting factor (1.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(9)	off-line generator control weighting factor (1.0 by default)
	VALUES(10)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(11)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (8)	array of 8 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is 1)
	LABELS(2)	label of contingency subsystem (blank by default)
	LABELS(3)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(5)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
	LABELS(6)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(7)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(8)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia and Governor Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	invalid OPTMAC value
	IERR = 9	invalid OPTRNK value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 10	invalid OPTCOR value
IERR = 11	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multisection line members
IERR = 50	error opening Unit Inertia Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.135. MACCC_WITH_COR_3

This API is the third release of function to run multiple AC contingency analysis with corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination each other as overlapping contingencies. Corrective actions are specified automatically to eliminate the violations in each post contingency solution with the objective of minimizing the controls adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MACCC_WITH_COR_3  TOL  OPTACC(1)..OPTACC(10) OPTMAC(1).. OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) OPTCOR(1)..OPTCOR(8) VALUES(1)..VALUES(11)
 LABELS(1)..LABELS(8) DFXFILE ACCFILE THRFILE INLFILE
```

Python command syntax:

```
ierr = maccc_with_cor_3(tol, optacc, optmac, optrnk, optcor, values,
 labels, dfxfile, accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_COR_3(TOL, OPTACC, OPTMAC, OPTRNK, OPTCOR, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (RESERVE)
	OPTACC(10) = 2	subsystem machines (PMAX)
	OPTACC(10) = 3	subsystem machines (INERTIA)
	OPTACC(10) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	allow multiple contingency analysis (0 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(1) = 2	enabled with failure cutoff disabled
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A
	OPTRNK(2) = 2	rate B
	OPTRNK(2) = 3	rate C
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable
	OPTRNK(3) = 1	enable
	OPTRNK(4)	machine overload contingencies (0 by default)
	OPTRNK(4) = 0	disable
	OPTRNK(4) = 1	enable
	OPTRNK(5)	minimum of machine overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(6)	maximum of machine overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(7)	cutoff of machine overload contingencies to evaluate (>=0, 5 by default)
	OPTRNK(8)	line overload contingencies (0 by default)
	OPTRNK(8) = 0	disable
	OPTRNK(8) = 1	enable
	OPTRNK(9)	minimum of line overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(10)	maximum of line overload contingencies to evaluate (>=0, 10 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(11)	cutoff of line overload contingencies (>=0, 5 by default)
	OPTRNK(12)	voltage collapse line contingencies (0 by default)
	OPTRNK(12) = 0	disable
	OPTRNK(12) = 1	enable
	OPTRNK(13)	minimum of voltage collapse line contingencies to evaluate (>=0, 1 by default)
	OPTRNK(14)	maximum of voltage collapse line contingencies to evaluate (>=0, 10 by default)
	OPTRNK(15)	cutoff of voltage collapse line contingencies (>=0, 5 be default)
	OPTRNK(16)	include all branches in voltage collapse ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real VALUES (11)	array of 11 elements specifying ranking parameters and corrective actions parameters(input).	
	VALUES(1)	small reactance threshold, (0<= VALUES (1)<=0.005, 0.0005 by default)
	VALUES(2)	percent of flow rating, (0<= VALUES (2)<=200.0, 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0, 0.0 by default)
	VALUES(4)	bus voltage violation tolerance (0.1 by default)
	VALUES(5)	branch overload violation tolerance (0.1 by default)
	VALUES(6)	generator control weighting factor (1.0 by default)
	VALUES(7)	load control weighting factor (1.0 by default)
	VALUES(8)	phase shifter control weighting factor (1.0 by default)
	VALUES(9)	off-line generator control weighting factor (1.0 by default)
	VALUES(10)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(11)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (8)	array of 8 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is 1)
	LABELS(2)	label of contingency subsystem (blank by default)
	LABELS(3)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(5)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
	LABELS(6)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(7)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(8)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).
Character*260 INLFILE	name of Unit Inertia and Governor Data File (input; blank for none).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTACC value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value
IERR = 11	invalid OPTRNK value
IERR = 12	invalid OPTCOR value
IERR = 13	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multisection line members
IERR = 50	error opening Unit Inertia Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.136. MACCC_WITH_COR_4

This API is the third release of function to run multiple AC contingency analysis with corrective actions. The user-specified and automatically-selected contingencies are evaluated individually and in combination each other as overlapping contingencies. Corrective actions are specified automatically to eliminate the violations in each post contingency solution with the objective of minimizing the controls adjustments including load shedding, MW generation dispatch, phase shifter angle adjustments, off-line generator controls, tap setting adjustments and switched shunt controls.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MACCC_WITH_COR_4  TOL  OPTACC(1)..OPTACC(10) OPTMAC(1).. OPTMAC(41)
 OPTRNK(1)..OPTRNK(17) OPTCOR(1)..OPTCOR(8) VALUES(1)..VALUES(11)
 LABELS(1)..LABELS(8) DFXFILE ACCFILE THRFILE INLFILE
```

Python command syntax:

```
ierr = maccc_with_cor_4(tol, optacc, optmac, optrnk, optcor, values,
 labels, dfxfile, accfile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_COR_4(TOL, OPTACC, OPTMAC, OPTRNK, OPTCOR, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous, disable discrete
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (RESERVE)
	OPTACC(10) = 2	subsystem machines (PMAX)
	OPTACC(10) = 3	subsystem machines (INERTIA)
	OPTACC(10) = 4	subsystem machines (GOVERNOR DROOP)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	allow multiple contingency analysis (0 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(1) = 2	enabled with failure cutoff disabled
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(3)	Each element from 3 to 41 of OPTMAC array represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit (machine) and single branch outages respectively
	OPTMAC(3) = 0	(0 by default)
	OPTMAC(3) = 1	include the contingency combinations
Integer OPTRNK (15)	array of 15 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	machine overload contingencies (0 by default)
	OPTRNK(2) = 0	disable
	OPTRNK(2) = 1	enable
	OPTRNK(3)	minimum of machine overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(4)	maximum of machine overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(5)	cutoff of machine overload contingencies to evaluate (>=0, 5 by default)
	OPTRNK(6)	line overload contingencies (0 by default)
	OPTRNK(6) = 0	disable
	OPTRNK(6) = 1	enable
	OPTRNK(7)	minimum of line overload contingencies to evaluate (>=0, 1 by default)
	OPTRNK(8)	maximum of line overload contingencies to evaluate (>=0, 10 by default)
	OPTRNK(9)	cutoff of line overload contingencies (>=0, 5 by default)
	OPTRNK(10)	voltage collapse line contingencies (0 by default)
	OPTRNK(10) = 0	disable
	OPTRNK(10) = 1	enable
	OPTRNK(11)	minimum of voltage collapse line contingencies to evaluate (>=0, 1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(12)	maximum of voltage collapse line contingencies to evaluate (>=0, 10 by default)
	OPTRNK(13)	cutoff of voltage collapse line contingencies (>=0, 5 be default)
	OPTRNK(14)	include all branches in voltage collapse ranking (0 by default)
	OPTRNK(14) = 0	include only with non-zero rating
	OPTRNK(14) = 1	all
	OPTRNK(15)	include contingencies that create swingless island (0 by default)
	OPTRNK(15) = 0	disable
	OPTRNK(15) = 1	enable
Integer OPTCOR (8)	array of 8 elements specifying corrective actions options(input).	
	OPTCOR(1)	run corrective action (0 by default)
	OPTCOR(1) = 0	disable
	OPTCOR(1) = 1	enable
	OPTCOR(2)	number power flows (1 by default)
	OPTCOR(3)	generator control flag (1 by default)
	OPTCOR(3) = 0	disable
	OPTCOR(3) = 1	enable
	OPTCOR(4)	load control flag (0 by default)
	OPTCOR(4) = 0	disable
	OPTCOR(4) = 1	enable
	OPTCOR(5)	phase shifter control flag (1 by default)
	OPTCOR(5) = 0	disable
	OPTCOR(5) = 1	enable
	OPTCOR(6)	off-line generator control flag (0 by default)
	OPTCOR(6) = 0	disable
	OPTCOR(6) = 1	enable
	OPTCOR(7)	tap setting adjustment flag (0 by default)
	OPTCOR(7) = 0	disable
	OPTCOR(7) = 1	enable
	OPTCOR(8)	switched shunt control flag (0 by default)
	OPTCOR(8) = 0	disable
	OPTCOR(8) = 1	enable
Real VALUES (10)	array of 10 elements specifying ranking parameters and corrective actions parameters(input).	
	VALUES(1)	small reactance threshold, (0<= VALUES (1)<=0.005, 0.0005 by default)
	VALUES(2)	MW threshold for islanding contingencies (>=0, 0.0 by default)
	VALUES(3)	bus voltage violation tolerance (0.1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(4)	branch overload violation tolerance (0.1 by default)
	VALUES(5)	generator control weighting factor (1.0 by default)
	VALUES(6)	load control weighting factor (1.0 by default)
	VALUES(7)	phase shifter control weighting factor (1.0 by default)
	VALUES(8)	off-line generator control weighting factor (1.0 by default)
	VALUES(9)	tap setting adjustment weighting factor (1.0 by default)
	VALUES(10)	switched shunt control weighting factor (1.0 by default)
Character*32 LABELS (8)	array of 8 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default, no default allowed if OPTACC(8) is 1)
	LABELS(2)	label of contingency subsystem (blank by default)
	LABELS(3)	label of generator control subsystem (blank by default, no default allowed if OPTCOR(3) is 1)
	LABELS(4)	label of load control subsystem (blank by default, no default allowed if OPTCOR(4) is 1)
	LABELS(5)	label of phase shifter control subsystem (blank by default, no default allowed if OPTCOR(5) is 1)
	LABELS(6)	off-line generator control subsystem (blank by default; no default allowed if OPTCOR(6) is 1)
	LABELS(7)	tap setting adjustment subsystem (blank by default; no default allowed if OPTCOR(7) is 1)
	LABELS(8)	switched shunt control subsystem (blank by default; no default allowed if OPTCOR(8) is 1)
Character*260 DFXFILE	name of Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia and Governor Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value
IERR = 11	invalid OPTRNK value
IERR = 12	invalid OPTCOR value
IERR = 13	invalid VALUES value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multisection line members
IERR = 50	error opening Unit Inertia Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.137. MACCC_WITH_TRIP

This API routine is obsolete. It has been replaced by the API routine maccc_with_trip_2 .

Use this API to run multiple level contingency analysis with tripping simulation.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MACCC_WITH_TRIP  TOL OPTACC(1)..OPTACC(8) OPTMAC(1).. OPTMAC(41)
 OPTRNK(1).. OPTRNK(17) OPTTRP(1)..OPTTRP(3) VALUES(1).. VALUES(3)
 LABELS(1) LABELS(2) DFXFILE ACCFILE THRFILE INLFILE TRPFILE
```

Python command syntax:

```
ierr = maccc_with_trip(tol, optacc, optmac, optrnk, opttrp, values,
 labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_TRIP(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, VALUES,  LABELS,
 DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (8)	array of 8 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	dispatch mode (0 by default)
	OPTACC(8) = 0	disable
	OPTACC(8) = 1	subsystem machines (reserve)
	OPTACC(8) = 2	subsystem machines (pmax)
	OPTACC(8) = 3	subsystem machines (inertia)
	OPTACC(8) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1<=OPTMAC(2)<=3; 1 by default)
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single machine and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations, otherwise
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTRNK(2) = 1	rate A
OPTRNK(2) = 2	rate B
OPTRNK(2) = 3	rate C
OPTRNK(3)	convert ratings to MW (0 by default)
OPTRNK(3) = 0	disable
OPTRNK(3) = 1	enable
OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
OPTRNK(4) = 0	disable
OPTRNK(4) = 1	enable
OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
OPTRNK(7)	non-failure cutoff of machine contingencies (>=0; 5 by default)
OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
OPTRNK(8) = 0	disable
OPTRNK(8) = 1	enable
OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
OPTRNK(12) = 0	disable
OPTRNK(12) = 1	enable
OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
OPTRNK(16)	include all branches in voltage depression ranking (0 by default)
OPTRNK(16) = 0	include only with non-zero rating
OPTRNK(16) = 1	all

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Real VALUES (3)	array of 3 elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	percent of rating (0<= VALUES (1)<=200.0; 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0; 0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation subsystem is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	too many islands in base case
IERR = 8	invalid OPTMAC value
IERR = 9	invalid OPTRNK value
IERR = 10	invalid VALUES value
IERR = 11	invalid OPTTRP value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.138. MACCC_WITH_TRIP_2

Use this API to run the second version of multiple level contingency analysis with tripping simulation.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MACCC_WITH_TRIP_2  TOL OPTACC(1)..OPTACC(10) OPTMAC(1).. OPTMAC(41)
 OPTRNK(1).. OPTRNK(17) OPTTRP(1)..OPTTRP(3) VALUES(1).. VALUES(3)
 LABELS(1) LABELS(2) DFXFILE ACCFILE THRFILE INLFILE TRPFILE
```

Python command syntax:

```
ierr = maccc_with_trip_2(tol, optacc, optmac, optrnk, opttrp, values,
 labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_TRIP_2(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (reserve)
	OPTACC(10) = 2	subsystem machines (pmax)
	OPTACC(10) = 3	subsystem machines (inertia)
	OPTACC(10) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels)
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(2) = -3	N-3 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single machine and single branch outages respectively
	OPTMAC(3) = 1	include the contingency combinations, otherwise
	OPTMAC(3) = 0	(0 by default)
Integer OPTRNK (17)	array of 17 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A
	OPTRNK(2) = 2	rate B
	OPTRNK(2) = 3	rate C
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable
	OPTRNK(3) = 1	enable
	OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(4) = 0	disable
	OPTRNK(4) = 1	enable
	OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(7)	non-failure cutoff of machine contingencies (>=0; 5 by default)
	OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(8) = 0	disable
	OPTRNK(8) = 1	enable
	OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
	OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(12) = 0	disable
	OPTRNK(12) = 1	enable
	OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
	OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
	OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(16)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Real VALUES (3)	array of 3 elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	percent of rating (0<= VALUES (1)<=200.0; 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0; 0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTACC value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value
IERR = 11	invalid OPTRNK value
IERR = 12	invalid VALUES value
IERR = 13	invalid OPTTRP value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-sec-tion line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.139. MACCC_WITH_TRIP_3

Use this API to run the second version of multiple level contingency analysis with tripping simulation.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MACCC_WITH_TRIP_3  TOL OPTACC(1)..OPTACC(10) OPTMAC(1).. OPTMAC(41)
 OPTRNK(1).. OPTRNK(15) OPTTRP(1)..OPTTRP(3) VALUES(1).. VALUES(2)
 LABELS(1) LABELS(2) DFXFILE ACCFILE THRFILE INLFILE TRPFILE
```

Python command syntax:

```
ierr = maccc_with_trip_3(tol, optacc, optmac, optrnk, opttrp, values,
 labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_TRIP_3(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, TOLN).	
Integer OPTACC (10)	array of 10 elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable
	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (reserve)
	OPTACC(10) = 2	subsystem machines (pmax)
	OPTACC(10) = 3	subsystem machines (inertia)
	OPTACC(10) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of 41 elements specifying contingency evaluation level and combinations(input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels)
	OPTMAC(2) = -3	N-3 contingency analysis
	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(3)	Each element from 3 to 41 of OPTMAC array represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit (machine) and single branch outages respectively
	OPTMAC(3) = 0	(0 by default)
	OPTMAC(3) = 1	include the contingency combinations
Integer OPTRNK (15)	array of 15 elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(2) = 0	disable
	OPTRNK(2) = 1	enable
	OPTRNK(3)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(4)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(5)	non-failure cutoff of machine contingencies (>=0; 5 by default)
	OPTRNK(6)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(6) = 0	disable
	OPTRNK(6) = 1	enable
	OPTRNK(7)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
	OPTRNK(8)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
	OPTRNK(9)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(10)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(10) = 0	disable
	OPTRNK(10) = 1	enable
	OPTRNK(11)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(12)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
	OPTRNK(13)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(14)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(14) = 0	include only with non-zero rating
	OPTRNK(14) = 1	all
	OPTRNK(15)	include contingencies that create swingless island (0 by default)
	OPTRNK(15) = 0	disable
	OPTRNK(15) = 1	enable
Integer OPTTRP (3)	array of 3 elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Real VALUES (2)	array of 2 elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES (1)<=0.005; 0.0005 by default)
	VALUES(2)	MW threshold for islanding contingencies (>=0; 0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0)
	LABELS(2)	label of contingency subsystem (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid OPTACC value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	invalid OPTMAC value
IERR = 11	invalid OPTRNK value
IERR = 12	invalid VALUES value
IERR = 13	invalid OPTTRP value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-sec-tion line members
IERR = 49	error opening Tripping Element Data File
IERR = 50	error opening Unit Inertia and Governor Data File
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 54	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.140. MACCC_WITH_TRIP_PARALLEL

Use this API to run the multiple level contingency analysis with tripping simulation in parallel when the number of contingency processors in Program Settings is greater than 1. User-specified and automatically selected contingencies from built-in contingency ranking functions are evaluated individually and in combination with each other as overlapping outages up to the specified evaluation level.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_MACCC_WITH_TRIP_PARALLEL  TOL OPTACC(1)..OPTACC(10)
 OPTMAC(1)..OPTMAC(41) OPTRNK(1)..OPTRNK(17) OPTTRP(1)..OPTTRP(3)
 VALUES(1)..VALUES(3) LABELS(1)..LABELS(2) DFXFILE ACCFILE THRFILE INLFILE
 TRPFILE
```

Python command syntax:

```
ierr = maccc_with_trip_parallel(tol, optacc, optmac, optrnk, opttrp,
 values, labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_TRIP_PARALLEL(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP, VALUES,
 LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, by default).	
Integer OPTACC (10)	array of elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (reserve)
	OPTACC(10) = 2	subsystem machines (pmax)
	OPTACC(10) = 3	subsystem machines (inertia)
	OPTACC(10) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of elements specifying contingency evaluation level and combinations. (input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels by default)
	OPTMAC(2) = -3	N-3 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(3)	each element represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single machine and single branch outages respectively. (0 by default)
	OPTMAC(3) = 0	not include the contingency combinations
	OPTMAC(3) = 1	include the contingency combinations
Integer OPTRNK (17)	array of elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	contingency rating (rating set program option setting by default)
	OPTRNK(2) = 1	rate A
	OPTRNK(2) = 2	rate B
	OPTRNK(2) = 3	rate C
	OPTRNK(3)	convert ratings to MW (0 by default)
	OPTRNK(3) = 0	disable
	OPTRNK(3) = 1	enable
	OPTRNK(4)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(4) = 0	disable
	OPTRNK(4) = 1	enable
	OPTRNK(5)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(6)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(7)	non-failure cutoff of machine contingencies ((>=0; 5 by default)
	OPTRNK(8)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(8) = 0	disable
	OPTRNK(8) = 1	enable
	OPTRNK(9)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(10)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
	OPTRNK(11)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(12)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(12) = 0	disable
	OPTRNK(12) = 1	enable
	OPTRNK(13)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
	OPTRNK(14)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
	OPTRNK(15)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(16)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(16) = 0	include only with non-zero rating
	OPTRNK(16) = 1	all
	OPTRNK(17)	include contingencies that create swingless island (0 by default)
	OPTRNK(17) = 0	disable
	OPTRNK(17) = 1	enable
Integer OPTTRP (3)	array of elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Real VALUES (3)	array of elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES(1)<=0.005; 0.0005 by default)
	VALUES(2)	percent of rating (0<= VALUES (1)<=200.0; 100.0 by default)
	VALUES(3)	MW threshold for islanding contingencies (>=0; 0.0 by default)
Character*32 LABELS (2)	array of elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(2)	label of contingency subsystem (blank by default) (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	
Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1). (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	invalid OPTMAC value
	IERR = 11	invalid OPTRNK value
	IERR = 12	invalid VALUES value
	IERR = 13	invalid OPTTRP value
	IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 49	error opening Tripping Element Data File
	IERR = 50	error opening Unit Inertia and Governor Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for function not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.141. MACCC_WITH_TRIP_PARALLEL_2

Use this API to run the multiple level contingency analysis with tripping simulation in parallel when the number of contingency processors in Program Settings is greater than 1. User-specified and automatically selected contingencies from built-in contingency ranking functions are evaluated individually and in combination with each other as overlapping outages up to the specified evaluation level.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MACCC_WITH_TRIP_PARALLEL_2  TOL OPTACC(1)..OPTACC(10)
 OPTMAC(1)..OPTMAC(41) OPTRNK(1)..OPTRNK(17) OPTTRP(1)..OPTTRP(3)
 VALUES(1)..VALUES(3) LABELS(1)..LABELS(2) DFXFILE ACCFILE THRFILE INLFILE
 TRPFILE
```

Python command syntax:

```
ierr = maccc_with_trip_parallel_2(tol, optacc, optmac, optrnk, opttrp,
 values, labels, dfxfile, accfile, thrfile, inlfile, trpfile)
```

Fortran command syntax:

```
CALL MACCC_WITH_TRIP_PARALLEL_2(TOL, OPTACC, OPTMAC, OPTRNK, OPTTRP,
 VALUES, LABELS, DFXFILE, ACCFILE, THRFILE, INLFILE, TRPFILE, IERR)
```

Where:

Real TOL	mismatch tolerance (input; Newton solution convergence tolerance, by default).	
Integer OPTACC (10)	array of elements specifying power flow solution options(input).	
	OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTACC(1) = 0	disable
	OPTACC(1) = 1	enable stepping adjustment
	OPTACC(1) = 2	enable direct adjustment
	OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTACC(2) = 0	disable
	OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
	OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTACC(3) = 0	disable
	OPTACC(3) = 1	enable
	OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTACC(4) = 0	disable
	OPTACC(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTACC(5) = 0	disable
	OPTACC(5) = 1	enable
	OPTACC(5) = 2	enable continuous mode, disable discrete mode
	OPTACC(6)	solution method flag (0 by default)
	OPTACC(6) = 0	FDNS
	OPTACC(6) = 1	FNSL
	OPTACC(6) = 2	optimized FDNS
	OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTACC(7) = 0	disable
	OPTACC(7) = 1	enable
	OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTACC(8) = 0	stall
	OPTACC(8) = 1	trip
	OPTACC(9)	induction machine failure flag (0 by default)
	OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
	OPTACC(10)	dispatch mode (0 by default)
	OPTACC(10) = 0	disable
	OPTACC(10) = 1	subsystem machines (reserve)
	OPTACC(10) = 2	subsystem machines (pmax)
	OPTACC(10) = 3	subsystem machines (inertia)
	OPTACC(10) = 4	subsystem machines (governor droop)
Integer OPTMAC (41)	array of elements specifying contingency evaluation level and combinations. (input).	
	OPTMAC(1)	run multiple contingency analysis (1 by default)
	OPTMAC(1) = 0	disable
	OPTMAC(1) = 1	enable
	OPTMAC(2)	contingency evaluation level (1 by default). If it is 2 or 3, a newly ordered event list is built for the next level after solving AC power flow each case at current level; if it is -2 or -3, the event list at the first level is used for any higher levels by default)
	OPTMAC(2) = -3	N-3 contingency analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTMAC(2) = -2	N-2 contingency analysis
	OPTMAC(2) = 1	N-1 contingency analysis
	OPTMAC(2) = 2	N-2 contingency analysis
	OPTMAC(2) = 3	N-3 contingency analysis
	OPTMAC(3)	Each element from 3 to 41 of OPTMAC array represents a type of contingency combinations in the following order. {S, U, B, SS, SU, SB, US, UU, UB, BS, BU, BB, SSS, SSB, SSU, SBS, SBB, SBU, SUS, SUB, SUU, USS, USU, USB, UUS, UUB, UUU, UBS, UBU, UBB, BSS, BSU, BSB, SUS, SUB, SUU, BBS, BBU, BBB}, where S, U and B are user specified cases, single unit (machine) and single branch outages respectively
	OPTMAC(3) = 0	(0 by default)
	OPTMAC(3) = 1	include the contingency combinations
Integer OPTRNK (15)	array of elements specifying ranking options(input).	
	OPTRNK(1)	include ties from contingency subsystem (0 by default)
	OPTRNK(1) = 0	disable
	OPTRNK(1) = 1	enable
	OPTRNK(2)	include machine contingencies from overload ranking (0 by default)
	OPTRNK(2) = 0	disable
	OPTRNK(2) = 1	enable
	OPTRNK(3)	minimum of machine contingencies to evaluate (>=0; 1 by default)
	OPTRNK(4)	maximum of machine contingencies to evaluate (>=0; 10 by default)
	OPTRNK(5)	non-failure cutoff of machine contingencies ((>=0; 5 by default)
	OPTRNK(6)	include branch contingencies from overload ranking (0 by default)
	OPTRNK(6) = 0	disable
	OPTRNK(6) = 1	enable
	OPTRNK(7)	minimum of branch contingencies from overload ranking to evaluate (>=0; 1 by default)
	OPTRNK(8)	maximum of branch contingencies from overload ranking to evaluate (>=0; 10 by default)
	OPTRNK(9)	non-failure cutoff of branch contingencies from overload ranking (>=0; 5 by default)
	OPTRNK(10)	include branch contingencies from voltage depression ranking (0 by default)
	OPTRNK(10) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTRNK(10) = 1	enable
	OPTRNK(11)	minimum of branch contingencies from voltage depression ranking to evaluate (>=0; 1 by default)
	OPTRNK(12)	maximum of branch contingencies from voltage depression ranking to evaluate (>=0; 10 by default)
	OPTRNK(13)	non-failure cutoff of branch contingencies from voltage depression ranking (>=0; 5 by default)
	OPTRNK(14)	include all branches in voltage depression ranking (0 by default)
	OPTRNK(14) = 0	include only with non-zero rating
	OPTRNK(14) = 1	all
	OPTRNK(15)	include contingencies that create swingless island (0 by default)
	OPTRNK(15) = 0	disable
	OPTRNK(15) = 1	enable
Integer OPTTRP (3)	array of elements specifying tripping simulation options(input).	
	OPTTRP(1)	tripping simulation flag (0 by default)
	OPTTRP(1) = 0	disable
	OPTTRP(1) = 1	enable
	OPTTRP(2)	number of tripping events per power flow (1 by default)
	OPTTRP(3)	number of power flows per contingency (1 by default)
Real VALUES (2)	array of elements specifying ranking parameters(input).	
	VALUES(1)	small reactance threshold (0<= VALUES(1)<=0.005; 0.0005 by default)
	VALUES(2)	MW threshold for islanding contingencies (>=0; 0.0 by default)
Character*32 LABELS (2)	array of elements specifying subsystem labels(input).	
	LABELS(1)	label of generation dispatch subsystem (blank by default; no default allowed if OPTACC(8) is not 0 by default)
	LABELS(2)	label of contingency subsystem (blank by default) (blank by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of the Contingency Solution Output File (input; no default allowed).	
Character*260 THRFILE	name of the Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	name of the Unit Inertia and Governor Data File (input; blank by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 TRPFILE	name of the Tripping Element Data File (input; blank for none; no default allowed if OPTTRP(1) is 1). (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid TOL value
	IERR = 2	invalid OPTACC value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	invalid OPTMAC value
	IERR = 11	invalid OPTRNK value
	IERR = 12	invalid VALUES value
	IERR = 13	invalid OPTTRP value
	IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 49	error opening Tripping Element Data File
	IERR = 50	error opening Unit Inertia and Governor Data File
	IERR = 51	error opening Contingency Solution Output File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 54	prerequisite requirements for function not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.142. MATRIX_GROWTH_FACTOR

Use this API to specify or return the value of the matrix growth factor option setting.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_MATRIX_GROWTH_FACTOR RVAL
```

Python command syntax:

```
ierr = matrix_growth_factor(rval)

ierr, rval = matrix_growth_factor()
```

Fortran command syntax:

```
CALL MATRIX_GROWTH_FACTOR_API(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 2.0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.143. MCRE

Use this API to read a Machine Impedance Data File and add the data specified in it to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MCRE  STATUS(1)  STATUS(2)  IFILE
```

Python command syntax:

```
ierr = mcre(status, ifile)
```

Fortran command syntax:

```
CALL MCREAPI(STATUS, IFILE, IERR)
```

Where:

Integer STATUS (2)	array of 2 elements specifying options(input).	
	STATUS(1)	new machine status option (0 by default)
	STATUS(1) = 0	set from STAT field on input record
	STATUS(1) = 1	set from status of other machines at the bus
	STATUS(2)	option for treatment of machines with no data record specified and other machines at the bus with record specified (0 by default)
	STATUS(2) = 0	place out-of-service
	STATUS(2) = 1	delete
Character*260 IFILE	name of machine impedance data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	generators are converted
	IERR = 3	error opening IFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.144. MODR

Use this API to uniformly increase or decrease the line resistances of in-service nontransformer branches. It must first be called once with APIOPT set to 1, then once for each subsystem to be processed with APIOPT set to 2, and finally once with APIOPT set to 3.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MODR  SID  ALL  APIOPT  IRAT  RLOAD  FMAX
```

Python command syntax:

```
ierr, nchng = modr(sid, all, apiopt, irat, rload, fmax)
```

Fortran command syntax:

```
CALL MODRNW(SID, ALL, APIOPT, IRAT, RLOAD, FMAX, NCHNG, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; used only when APIOPT is 2; 0).	
Integer ALL	all buses or specified subsystem flag (input; used only when APIOPT is 2; 1).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer APIOPT	mode of operation of the API (input; no default allowed).	
	APIOPT = 1	initialize system for line resistance modification
	APIOPT = 2	modify line resistances
	APIOPT = 3	postprocessing housekeeping
Integer IRAT	rating set on which the present values of branch resistances are based (input; used only when APIOPT is 1; default rating set program option setting).	
	IRAT = 1	RATEA
	IRAT = 2	RATEB
	IRAT = 3	RATEC
Real RLOAD	base line loading in percent. 0.0001 < RLOAD < 2000.0 (input; used only when APIOPT is 1; 100.0).	
Real FMAX	line resistance scaling factor in pu 0.0 < FMAX < 0.1 (input; used only when APIOPT is 1; 0.01).	
Integer NCHNG	returned as the number of branch resistances changed (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ALL value
IERR = 3	invalid APIOPT value
IERR = 4	invalid IRAT value
IERR = 5	invalid RLOAD value
IERR = 6	invalid FMAX value
IERR = 7	unexpected APIOPT value
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.145. MOVE3WND

Use this API to disconnect the third bus of a specified three-winding transformer from the bus to which it is currently connected, and reconnect it to a designated bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MOVE3WND  IBUS  JBUS1  JBUS2  'CKT'  NEWTOBUS2  'NEWCKT'
```

Python command syntax:

```
ierr = move3wnd(ibus, jbus1, jbus2, ckt, newtobus2, newckt)
```

Fortran command syntax:

```
CALL MOVE3WND(IBUS, JBUS1, JBUS2, CKT, NEWTOBUS2, NEWCKT, IERR)
```

Where:

Integer IBUS	one of the buses connected by the three-winding transformer; the transformer will remain connected to this bus (input; no default allowed).
Integer JBUS1	other bus connected by the three-winding transformer; the transformer will remain connected to this bus (input; no default allowed).
Integer JBUS2	third bus connected by the three-winding transformer; this connection will be severed and the transformer connected to bus NEWTOBUS2 (input; no default allowed).
Character*2 CKT	circuit identifier of the three-winding transformer to be rerouted (input; '1' by default).
Integer NEWTOBUS2	number of the bus to which the three-winding transformer is to be connected (input; no default allowed).
Character*2 NEWCKT	new circuit identifier to be assigned to the rerouted three-winding transformer (input; CKT).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	bus not found
IERR = 2	three-winding transformer not found
IERR = 3	new connection bus must not be the present from or to buses
IERR = 4	circuit NEWCKT from IBUS to JBUS1 to NEWTOBUS2 already exists
IERR = 5	invalid new identifier; cannot start with &, * or @
IERR = 6	invalid new identifier; must not be blank
IERR = 7	node-breaker terminal connection table is full
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.146. MOVEBRN

Use this API to disconnect the to bus of a specified non-transformer branch or two-winding transformer from the bus to which it is currently connected, and reconnect it to a designated bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MOVEBRN  IBUS  JBUS  'CKT'  NEWTOBUS  'NEWCKT'
```

Python command syntax:

```
ierr = movebrn(ibus, jbus, ckt, newtobus, newckt)
```

Fortran command syntax:

```
CALL MOVEBRN(IBUS, JBUS, CKT, NEWTOBUS, NEWCKT, IERR)
```

Where:

Integer IBUS	one of the buses connected by the branch; the branch will remain connected to this bus (input; no default allowed).	
Integer JBUS	other bus connected by the branch; this connection will be severed and the branch connected to bus NEWTOBUS (input; no default allowed).	
Character*2 CKT	circuit identifier of the branch to be rerouted (input; '1' by default).	
Integer NEWTOBUS	number of the bus to which the branch is to be connected (input; no default allowed).	
Character*2 NEWCKT	new circuit identifier to be assigned to the rerouted branch (input; CKT).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	circuit CKT from IBUS to JBUS not found
	IERR = 3	new connection bus must not be the present from or to bus
	IERR = 4	circuit NEWCKT from IBUS to NEWTOBUS already exists
	IERR = 5	invalid new identifier; cannot start with &, * or @
	IERR = 6	invalid new identifier; must not be blank
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.147. MOVEINDMAC

Use this API routine to disconnect the specified induction machine from the bus to which it is currently connected, and reconnect it to a designated bus.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MOVEINDMAC  IBUS  'ID'  NEWBUS  'NEWID'
```

Python command syntax:

```
ierr = moveindmac(ibus, id, newbus, newid)
```

Fortran command syntax:

```
CALL MOVEINDMAC(IBUS, ID, NEWBUS, NEWID, IERR)
```

Where:

Integer IBUS	bus to which induction machine ID is connected (input; no default allowed).	
Character*2 ID	machine identifier of the induction machine to be moved (input; '1' by default).	
Integer NEWBUS	number of the bus to which the induction machine is to be moved (input; ID).	
Character*2 NEWID	new induction machine identifier to be assigned to the relocated induction machine (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	induction machine not found
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	induction machine NEWID at bus NEWBUS already exists
	IERR = 5	invalid new identifier; must not be blank
	IERR = 6	node-breaker terminal connection table is full
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.148. MOVEINDMACS

Use this API routine to disconnect all of the induction machines at the specified bus, and reconnect them to a designated bus. Bus TOBUS must not have any induction machines connected to it at the time this API routine is called.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MOVEINDMACS  IBUS  NEWBUS
```

Python command syntax:

```
ierr = moveindmacs(ibus, newbus)
```

Fortran command syntax:

```
CALL MOVEINDMACS(IBUS, NEWBUS, IERR)
```

Where:

Integer IBUS	bus at which one or more induction machines currently exist (input; no default allowed).	
Integer NEWBUS	bus to which the induction machines on bus IBUS are to be moved (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no induction machines at bus IBUS
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	NEWBUS already has induction machines (move individual machines)
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.149. MOVELOAD

Use this API to disconnect the specified load from the bus to which it is currently connected, and reconnect it to a designated bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MOVELOAD  IBUS  'ID'  NEWBUS  'NEWID'
```

Python command syntax:

```
ierr = moveload(ibus, id, newbus, newid)
```

Fortran command syntax:

```
CALL MOVELOAD(IBUS, ID, NEWBUS, NEWID, IERR)
```

Where:

Integer IBUS	bus to which the load ID is connected (input; no default allowed).	
Character*2 ID	load identifier of the load to be moved (input; '1' by default).	
Integer NEWBUS	number of the bus to which the load is to be moved (input; no default allowed).	
Character*2 NEWID	new load identifier to be assigned to the relocated load (input; ID).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	load not found
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	load NEWID at bus NEWBUS already exists
	IERR = 5	invalid new identifier; must not be blank
	IERR = 6	node-breaker terminal connection table is full
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.150. MOVELOADS

Use this API to disconnect all of the load from the specified bus, and reconnect it to a designated bus. Bus TOBUS must not have any load connected to it at the time this API is called.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MOVELOADS  IBUS  NEWBUS
```

Python command syntax:

```
ierr = moveloads(ibus, newbus)
```

Fortran command syntax:

```
CALL MOVELOADS(IBUS, NEWBUS, IERR)
```

Where:

Integer IBUS	bus on which one or more loads currently exist (input; no default allowed).	
Integer NEWBUS	bus to which the loads on bus IBUS are to be moved (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no loads at bus IBUS
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	NEWBUS already has loads (move individual loads)
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.151. MOVEMAC

Use this API to disconnect the specified machine from the bus to which it is currently connected, and re-connect it to a designated bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MOVEMAC  IBUS  'ID'  NEWBUS  'NEWID'
```

Python command syntax:

```
ierr = movemac(ibus, id, newbus, newid)
```

Fortran command syntax:

```
CALL MOVEMAC(IBUS, ID, NEWBUS, NEWID, IERR)
```

Where:

Integer IBUS	bus to which the machine ID is connected (input; no default allowed).	
Character*2 ID	machine identifier of the machine to be moved (input; '1' by default).	
Integer NEWBUS	number of the bus to which the machine is to be moved (input; ID).	
Character*2 NEWID	new machine identifier to be assigned to the relocated machine (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	machine NEWID at bus NEWBUS already exists
	IERR = 5	plant table full (move machine to an existing plant)
	IERR = 6	invalid new identifier; must not be blank
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.152. MOVEPLNT

Use this API to disconnect all of the machines from the specified bus, and reconnect it to a designated bus. Bus TOBUS must not have any plant data associated with it at the time this API is called.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MOVEPLNT  IBUS  NEWBUS
```

Python command syntax:

```
ierr = moveplnt(ibus, newbus)
```

Fortran command syntax:

```
CALL MOVEPLNT(IBUS, NEWBUS, IERR)
```

Where:

Integer IBUS	bus on which plant data currently exists (input; no default allowed).	
Integer NEWBUS	bus to which the plant on bus IBUS is to be moved (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no plant at bus IBUS
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	NEWBUS already has plant data (move individual machines)
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.153. MOVESHUNT

Use this API to disconnect the specified fixed shunt from the bus to which it is currently connected, and re-connect it to a designated bus.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_MOVESHUNT  IBUS  'ID'  NEWBUS  'NEWID'
```

Python command syntax:

```
ierr = moveshunt(ibus, id, newbus, newid)
```

Fortran command syntax:

```
CALL MOVESHUNT(IBUS, ID, NEWBUS, NEWID, IERR)
```

Where:

Integer IBUS	bus to which the shunt ID is connected (input; no default allowed).	
Character*2 ID	shunt identifier of the fixed shunt to be moved (input; '1' by default).	
Integer NEWBUS	number of the bus to which the shunt is to be moved (input; no default allowed).	
Character*2 NEWID	new shunt identifier to be assigned to the relocated fixed shunt (input; ID).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	fixed shunt not found
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	shunt NEWID at bus NEWBUS already exists
	IERR = 5	invalid new identifier; must not be blank
	IERR = 6	node-breaker terminal connection table is full
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.154. MOVESHUNTS

Use this API to disconnect all of the fixed shunts from the specified bus and reconnect them to a designated bus. Bus TOBUS must not have any fixed shunt connected to it at the time this API is called.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_MOVESHUNTS IBUS NEWBUS
```

Python command syntax:

```
ierr = moveshunts(ibus, newbus)
```

Fortran command syntax:

```
CALL MOVESHUNTS(IBUS, NEWBUS, IERR)
```

Where:

Integer IBUS	bus on which one or more fixed shunts currently exist (input; no default allowed).	
Integer NEWBUS	bus to which the fixed shunts on bus IBUS are to be moved (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no fixed shunts at bus IBUS
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	NEWBUS already has fixed shunts (move individual shunts)
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.155. MOVESWS

This API routine is obsolete. It has been replaced by the API routine MoveSwShunt .

Use this API to disconnect the switched shunt load from the specified bus and reconnect it to a designated bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MOVESWS IBUS NEWBUS
```

Python command syntax:

```
ierr = movesws(ibus, newbus)
```

Fortran command syntax:

```
CALL MOVESWS(IBUS, NEWBUS, IERR)
```

Where:

Integer IBUS	bus on which a switched shunt currently exists (input; no default allowed).	
Integer NEWBUS	bus to which the switched shunt on bus IBUS is to be moved (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no switched shunt at bus IBUS
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.156. MOVESWSHUNT

Use this API to disconnect the specified switched shunt from the bus to which it is currently connected, and reconnect it to a designated bus.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MOVESWSHUNT IBUS 'ID' NEWBUS 'NEWID'
```

Python command syntax:

```
ierr = moveswshunt(ibus, id, newbus, newid)
```

Fortran command syntax:

```
CALL MOVESWSHUNT(IBUS, ID, NEWBUS, NEWID, IERR)
```

Where:

Integer IBUS	bus to which switched shunt ID is connected (input; no default allowed).	
Character*2 ID	shunt identifier of the switched shunt to be moved (input; '1' by default).	
Integer NEWBUS	number of the bus to which the shunt is to be moved (input; no default allowed).	
Character*2 NEWID	new shunt identifier to be assigned to the relocated switched shunt (input; ID).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt not found
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	switched shunt NEWID at bus NEWBUS already exists
	IERR = 5	invalid new identifier; must not be blank
	IERR = 6	node-breaker terminal connection table is full
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.157. MOVESWSHUNTS

Use this API to disconnect all of the switched shunts from the specified bus and reconnect them to a designated bus. Bus TOBUS must not have any switched shunt connected to it at the time this API is called.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MOVESWSHUNTS IBUS NEWBUS
```

Python command syntax:

```
ierr = moveswshunts(ibus, newbus)
```

Fortran command syntax:

```
CALL MOVESWSHUNTS(IBUS, NEWBUS, IERR)
```

Where:

Integer IBUS	bus on which one or more switched shunts currently exist (input; no default allowed).	
Integer NEWBUS	bus to which the switched shunts on bus IBUS are to be moved (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no switched shunts at bus IBUS
	IERR = 3	IBUS and NEWBUS are the same bus
	IERR = 4	NEWBUS already has switched shunts (move individual switched shunts)
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.158. MOV_ALPHA

Use this API to specify or return the value of the MOV iteration ALPHA option setting.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_MOV_ALPHA RVAL IOCODE
```

Python command syntax:

```
ierr = mov_alpha(rval, iocode)
```

Fortran command syntax:

```
CALL MOV_ALPHA(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	MOV iteration ALPHA option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0.3 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.159. MOV_ITERATIONS

Use this API to specify or return the value of the maximum MOV iterations option setting.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_MOV_ITERATIONS IVAL IOCODE
```

Python command syntax:

```
ierr = mov_iterations(ival, iocode)
```

Fortran command syntax:

```
CALL MOV_ITERATIONS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 20 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.160. MOV_TOLERANCE

Use this API to specify or return the value of the MOV iteration tolerance option setting.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_MOV_TOLERANCE RVAL IOCODE
```

Python command syntax:

```
ierr = mov_tolerance(rval, iocode)
```

Fortran command syntax:

```
CALL MOV_TOLERANCE(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	MOV iteration tolerance (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0.01 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.161. MSLV

Use this API to apply the modified Gauss-Seidel power flow calculation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MSLV  OPTIONS(1)..OPTIONS(6)
```

Python command syntax:

```
ierr = mslv(options)
```

Fortran command syntax:

```
CALL MSLVAPI(OPTIONS, IERR)
```

Where:

Integer OPTIONS (6)	array of 6 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (use tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable
	OPTIONS(2)	area interchange adjustment flag (use area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	flat start flag (0 by default)
	OPTIONS(3) = 0	do not flat start
	OPTIONS(3) = 1	flat start
	OPTIONS(3) = 2	flat start, then estimate voltage magnitudes
	OPTIONS(3) = 3	flat start, then estimate voltage phase angles
	OPTIONS(3) = 4	flat start, then estimate voltage magnitudes and phase angles
	OPTIONS(4)	dc tap adjustment flag (use dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (use switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	var limit flag (0 by default)
	OPTIONS(6) = 0	apply var limits
	OPTIONS(6) = 1	ignore var limits
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted
	IERR = 3	buses in island(s) without a swing bus; use activity TREE
	IERR = 4	bus type code and series element status inconsistencies
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.162. MSUM

Use this API to print a summary of mileage by owner.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MSUM
```

Python command syntax:

```
ierr = msum()
```

Fortran command syntax:

```
CALL MSUMAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.163. MTDC

Use this API to produce a report of bus voltages and flows for each in-service multi-terminal dc line.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MTDC
```

Python command syntax:

```
ierr = mtdc()
```

Fortran command syntax:

```
CALL MTDCAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no unblocked multi-terminal dc lines in this case
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.164. MULTISECTION_REPORTING

Use this API to specify or return the option to enable or disable multi-section line reporting.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MULTISECTION_REPORTING IVAL
```

Python command syntax:

```
ierr = multisection_reporting(ival)

ierr, ival = multisection_reporting()
```

Fortran command syntax:

```
CALL MULTISECTION_LINE_REPORTING_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	off.
	IVAL = 1	on.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.165. MWMI

Use this API to apply the MAPP MW-mile calculation
.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MWMI  SID  ALL  STATUS(1)..STATUS(4)  VALUES(1)..VALUES(3)  INPFILE
 TRNFILE  OUTFILE
```

Python command syntax:

```
ierr = mwmi(sid, all, status, values, inpfile, trnfile, outfile)
```

Fortran command syntax:

```
CALL MWMIAPI(SID, ALL, STATUS, VALUES, INPFILE, TRNFILE, OUTFILE, IERR)
```

Where:

Integer SID	a valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all areas or specified subsystem flag (input; 1 by default).	
	ALL = 0	only areas in area subsystem SID are in the pool
	ALL = 1	all areas are in the pool
Integer STATUS (4)	array of 4 elements specifying solution options(input).	
	STATUS(1)	rating used (default rating set program option setting)
	STATUS(1) = 1	RATEA
	STATUS(1) = 2	RATEB
	STATUS(1) = 3	RATEC
	STATUS(2)	option to list data (0 by default)
	STATUS(2) = 0	no
	STATUS(2) = 1	yes
	STATUS(3)	option to include buyer and seller branches in the detailed branch output file (0 by default)
	STATUS(3) = 0	no
	STATUS(3) = 1	yes
	STATUS(4)	action code if the MW mismatch exceeds 0.5 MW (0 by default)
	STATUS(4) = 0	discontinue
	STATUS(4) = 1	continue
Real VALUES (3)	array of 3 elements specifying solution parameters(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(1)	percent load reduction for calculating capacity for serving load (10.0 by default)
	VALUES(2)	MW for each transaction; used when no transactions data file is specified (10.0 by default)
	VALUES(3)	distribution factor threshold for detailed branch (0.0 by default)
Character*260 INPFILE	name of MWMI data file (input; no default allowed).	
Character*260 TRNFILE	name of the transactions raw data file; blank for none (input; blank by default).	
Character*260 OUTFILE	name of the detailed branch output file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid VALUES value
	IERR = 5	INPFILE is blank
	IERR = 6	generators are converted
	IERR = 7	buses in island(s) without a swing bus; use activity TREE
	IERR = 8	largest mismatch exceeds 0.5 MW
	IERR = 9	error opening INPFILE
	IERR = 10	error opening OUTFILE
	IERR = 11	error opening TRNFILE
	IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.166. MWOMWMSF

Use this API to tabulate generation (MW) on MW-mile shift factors for a specified transaction event.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MWOMWMSF EVENT
```

Python command syntax:

```
ierr = mwomwmsf(event)
```

Fortran command syntax:

```
CALL MWOMWMSFAPI(EVENT, IERR)
```

Where:

Integer EVENT	transaction event number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no transactions defined
	IERR = 2	invalid transaction event number
	IERR = 3	transaction event not defined
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.167. MWOMWOSF

Use this API to tabulate generation (MW) on MW-ohm shift factors for a specified transaction event.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MWOMWOSF EVENT
```

Python command syntax:

```
ierr = mwomwosf(event)
```

Fortran command syntax:

```
CALL MWOMWOSFAPI(EVENT, IERR)
```

Where:

Integer EVENT	transaction event number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no transactions defined
	IERR = 2	invalid transaction event number
	IERR = 3	transaction event not defined
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.168. N11_ACCC

This API routine is obsolete. It has been replaced by the API routine n11_accc_2 .

This API is used to perform N-1-1 contingency analysis. Each pair of N-1-1 contingencies is simulated in the way that the first N-1 contingency is simulated by a full AC load flow solution with the power flow options for the first N-1 contingency; the post solution of the first N-1 contingency is used as the base case for the second N-1 contingency, the second N-1 contingency is imposed and solved by a full AC load flow solution with power flow options for the second N-1 contingency.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_N11_ACCC  OPTIONS(1)..OPTIONS(18) VALUES(1) VALUES(2) LABEL DFXFILE
 ACCFILE CONFILE INLFILE THRFILE
```

Python command syntax:

```
ierr = n11_accc(options,values,label,dfxfile,accfile,confile,inlfile,
 thrfile)
```

Fortran command syntax:

```
CALL N11_ACCC(OPTIONS,VALUES,LABEL,DFXFILE,ACCFILE,CONFILE,INLFILE,
 THRFILE,IERR)
```

Where:

Integer OPTIONS (18)	array of 18 elements specifying integer specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable
OPTIONS(9) = 1	enable stepping adjustment
OPTIONS(9) = 2	enable direct adjustment
OPTIONS(10)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(11)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable
OPTIONS(11) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(12)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
	OPTIONS(12) = 0	disable
	OPTIONS(12) = 1	enable
Real VALUES (2)	array of 2 elements specifying a real(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Percent of rating for the first N-1 contingency and second N-1 contingency case (100.0 by default)
Character*32 LABEL	label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation (input).	
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of contingency analysis output file (input; no default allowed).	
Character*260 CONFILE	name of Contingency Description Data File for the second N-1 contingencies (input; blank for none).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	Generators are converted
	IERR = 4	Buses in island(s) without a swing bus--use activity TREE
	IERR = 5	Largest mismatch exceeds tolerance
	IERR = 6	Generation subsystem not specified
	IERR = 7	Too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	No Distribution Factor Data input file specified
	IERR = 11	No Contingency Analysis Output File specified
	IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
	IERR = 13	No second N-1 contingencies are specified
	IERR = 14	Invalid base/contingency case flag combination
	IERR = 21	File is not in the form of a PSSE-25 or later DFAX file--run DFAX

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 22	Monitored elements exceed limit when adding multi-section line members
IERR = 51	Error opening distribution factor data file
IERR = 52	Error opening load throwover data file
IERR = 53	Error opening inertia data file
IERR = 54	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.169. N11_ACCC_2

This API is the second release of N-1-1 contingency analysis solution. This API is used to perform N-1-1 contingency analysis. Each pair of N-1-1 contingencies is simulated in the way that the first N-1 contingency is simulated by a full AC load flow solution with the power flow options for the first N-1 contingency; the post solution of the first N-1 contingency is used as the base case for the second N-1 contingency, the second N-1 contingency is imposed and solved by a full AC load flow solution with power flow options for the second N-1 contingency.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_N11_ACCC_2  OPTIONS(1)..OPTIONS(20) VALUES(1) VALUES(2) LABEL DFXFILE
 ACCFILE CONFILE INLFILE THRFILE
```

Python command syntax:

```
ierr =
 n11_accc_2(options,values,label,dfxfile,accfile,confile,inlfile,thrfile)
```

Fortran command syntax:

```
CALL
 N11_ACCC_2(OPTIONS,VALUES,LABEL,DFXFILE,ACCFILE,CONFILE,INLFILE,THRFILE,IERR)
```

Where:

Integer OPTIONS (20)	array of 20 elements specifying integer specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable
OPTIONS(9) = 1	enable stepping adjustment
OPTIONS(9) = 2	enable direct adjustment
OPTIONS(10)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(11)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable
OPTIONS(11) = 1	enable
OPTIONS(12)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(12) = 0	disable
	OPTIONS(12) = 1	enable
	OPTIONS(13)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
	OPTIONS(13) = 0	disable
	OPTIONS(13) = 1	enable
	OPTIONS(13) = 2	enable continuous, disable discrete
	OPTIONS(14)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTIONS(14) = 0	stall
	OPTIONS(14) = 1	trip
	OPTIONS(15)	generation dispatch method
	OPTIONS(15) = 1	subsystem machines (PMAX)
	OPTIONS(15) = 2	subsystem machines (PMAX)
	OPTIONS(15) = 3	subsystem machines (INERTIA)
	OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP)
	OPTIONS(16)	flag to skip the second N-1 contingencies if the first N-1 contingency is not clean
	OPTIONS(16) = 0	No
	OPTIONS(16) = 1	Yes
	OPTIONS(17)	Rating set for the first N-1 contingency case
	OPTIONS(17) = 1	RATE A
	OPTIONS(17) = 2	RATE B
	OPTIONS(17) = 3	RATE C
	OPTIONS(18)	Rating set for the second N-1 contingency case
	OPTIONS(18) = 1	RATE A
	OPTIONS(18) = 2	RATE B
	OPTIONS(18) = 3	RATE C
	OPTIONS(19)	Voltage limit for the first N-1 contingency case
	OPTIONS(19) = 1	normal
	OPTIONS(19) = 2	emergency
	OPTIONS(20)	Voltage limit for the second N-1 contingency case
	OPTIONS(20) = 1	normal
	OPTIONS(20) = 2	emergency
Real VALUES (2)	array of 2 elements specifying a real(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(2)	Percent of rating for the first N-1 contingency and second N-1 contingency case (100.0 by default)
Character*32 LABEL	label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation (input).	
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of contingency analysis output file (input; no default allowed).	
Character*260 CONFILE	name of Contingency Description Data File for the second N-1 contingencies (input; blank for none).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	Generators are converted
	IERR = 4	Buses in island(s) without a swing bus--use activity TREE
	IERR = 5	Largest mismatch exceeds tolerance
	IERR = 6	Generation subsystem not specified
	IERR = 7	Too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	No Distribution Factor Data input file specified
	IERR = 11	No Contingency Analysis Output File specified
	IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
	IERR = 13	No second N-1 contingencies are specified
	IERR = 14	Invalid base/contingency case flag combination
	IERR = 21	File is not in the form of a PSSE-25 or later DFAX file--run DFAX
	IERR = 22	Monitored elements exceed limit when adding multi-section line members
	IERR = 51	Error opening distribution factor data file
	IERR = 52	Error opening load throwover data file
	IERR = 53	Error opening inertia data file
	IERR = 54	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.170. N11_ACCC_3

This API is the second release of N-1-1 contingency analysis solution. This API is used to perform N-1-1 contingency analysis. Each pair of N-1-1 contingencies is simulated in the way that the first N-1 contingency is simulated by a full AC load flow solution with the power flow options for the first N-1 contingency; the post solution of the first N-1 contingency is used as the base case for the second N-1 contingency, the second N-1 contingency is imposed and solved by a full AC load flow solution with power flow options for the second N-1 contingency.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_N11_ACCC_3  OPTIONS(1)..OPTIONS(16) VALUES LABEL DFXFILE ACCFILE
 CONFILE INLFILE THRFILE
```

Python command syntax:

```
ierr =
 n11_accc_3(options,values,label,dfxfile,accfile,confile,inlfile,thrfile)
```

Fortran command syntax:

```
CALL
 N11_ACCC_3(OPTIONS,VALUES,LABEL,DFXFILE,ACCFILE,CONFILE,INLFILE,THRFILE,IERR)
```

Where:

Integer OPTIONS (16)	array of 16 elements specifying integer specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable
OPTIONS(9) = 1	enable stepping adjustment
OPTIONS(9) = 2	enable direct adjustment
OPTIONS(10)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(11)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable
OPTIONS(11) = 1	enable
OPTIONS(12)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(12) = 0	disable
	OPTIONS(12) = 1	enable
	OPTIONS(13)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
	OPTIONS(13) = 0	disable
	OPTIONS(13) = 1	enable
	OPTIONS(13) = 2	enable continuous, disable discrete
	OPTIONS(14)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTIONS(14) = 0	stall
	OPTIONS(14) = 1	trip
	OPTIONS(15)	generation dispatch method
	OPTIONS(15) = 1	subsystem machines (PMAX)
	OPTIONS(15) = 2	subsystem machines (PMAX)
	OPTIONS(15) = 3	subsystem machines (INERTIA)
	OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP)
	OPTIONS(16)	flag to skip the second N-1 contingencies if the first N-1 contingency is not clean
	OPTIONS(16) = 0	No
	OPTIONS(16) = 1	Yes
Real VALUES (1)	array of 1 elements specifying a real(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
Character*32 LABEL	label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation (input).	
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of contingency analysis output file (input; no default allowed).	
Character*260 CONFILE	name of Contingency Description Data File for the second N-1 contingencies (input; blank for none).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	Generators are converted
	IERR = 4	Buses in island(s) without a swing bus--use activity TREE
	IERR = 5	Largest mismatch exceeds tolerance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Generation subsystem not specified
IERR = 7	Too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	No Distribution Factor Data input file specified
IERR = 11	No Contingency Analysis Output File specified
IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
IERR = 13	No second N-1 contingencies are specified
IERR = 14	Invalid base/contingency case flag combination
IERR = 21	File is not in the form of a PSSE-25 or later DFAX file--run DFAX
IERR = 22	Monitored elements exceed limit when adding multi-section line members
IERR = 51	Error opening distribution factor data file
IERR = 52	Error opening load throwover data file
IERR = 53	Error opening inertia data file
IERR = 54	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.171. N11_ACCC_PARALLEL

Use this API to run the N-1-1 contingency analysis in parallel when the number of contingency processors in Program Settings is greater than 1. The API distributes contingency cases among multiple processors evenly, and tests the contingency cases and following tripping actions assigned to each processor in parallel.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_N11_ACCC_PARALLEL  OPTIONS(1)..OPTIONS(20) VALUES(1)..VALUES(2) LABEL
 DFXFILE ACCFILE CONFILE THRFILE INLFILE
```

Python command syntax:

```
ierr = n11_accc_parallel(options, values, label, dfxfile, accfile,
 confile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL N11_ACCC_PARALLEL(OPTIONS, VALUES, LABEL, DFXFILE, ACCFILE, CONFILE,
 THRFILE, INLFILE, IERR)
```

Where:

Integer OPTIONS (20)	array of elements specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete.
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall.
OPTIONS(6) = 1	trip.
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable.
OPTIONS(8) = 1	Enable.
OPTIONS(9)	tap adjustment flag for the second N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable.
OPTIONS(9) = 1	enable stepping adjustment.
OPTIONS(9) = 2	enable direct adjustment.
OPTIONS(10)	area interchange adjustment flag for the second N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable.
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange.
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange.
OPTIONS(11)	phase shift adjustment flag for the second N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable.
OPTIONS(11) = 1	enable.
OPTIONS(12)	dc tap adjustment flag for the second N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(12) = 0	disable.
OPTIONS(12) = 1	enable.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(13)	switched shunt adjustment flag for the second N-1 contingency (switched shunt adjustment option setting by default)
	OPTIONS(13) = 0	disable
	OPTIONS(13) = 1	enable
	OPTIONS(13) = 2	enable continuous, disable discrete.
	OPTIONS(14)	induction motor treatment flag for the second N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTIONS(14) = 0	stall
	OPTIONS(14) = 1	trip
	OPTIONS(15)	generation dispatch method
	OPTIONS(15) = 0	disable.
	OPTIONS(15) = 1	subsystem machines (RESERVE).
	OPTIONS(15) = 2	subsystem machines (PMAX).
	OPTIONS(15) = 3	subsystem machines (INERTIA).
	OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP).
	OPTIONS(16)	flag to skip the second N-1 contingencies if the first N-1 contingency is not clean
	OPTIONS(16) = 0	No
	OPTIONS(16) = 1	Yes
	OPTIONS(17)	Rating set for the first N-1 contingency case
	OPTIONS(17) = 1	rate A
	OPTIONS(17) = 2	rate B
	OPTIONS(17) = 3	rate C
	OPTIONS(18)	Rating set for the second N-1 contingency case
	OPTIONS(18) = 1	rate A
	OPTIONS(18) = 2	rate B
	OPTIONS(18) = 3	rate C
	OPTIONS(19)	voltage limit for the first N-1 contingency case
	OPTIONS(19) = 1	normal
	OPTIONS(19) = 2	emergency
	OPTIONS(20)	voltage limit for the second N-1 contingency case
	OPTIONS(20) = 1	normal
	OPTIONS(20) = 2	emergency
Real VALUES (2)	array of 2 elements specifying power flow options(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Percent of rating for the first N-1 contingency and second N-1 contingency case (100.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*32 LABEL	The label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation output. (input).
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).
Character*260 ACCFILE	Is the name of contingency analysis output file (input; no default allowed).
Character*260 CONFILE	Is the name of Contingency Description Data File for the second N-1 contingencies (input; blank by default).
Character*260 THRFILE	Is the name of Load Throwover Data File (input; blank by default).
Character*260 INLFILE	Is the name of Unit Inertia Data File (input; blank by default).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	Invalid OPTIONS value
IERR = 2	Invalid VALUES value
IERR = 3	Generators are converted
IERR = 4	Buses in island(s) without a swing bus--use activity TREE
IERR = 5	Largest mismatch exceeds tolerance
IERR = 6	Generation subsystem not specified
IERR = 7	Too many islands in base case.
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	No Distribution Factor Data input file specified
IERR = 11	No Contingency Solution Output File specified
IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
IERR = 13	No second N-1 contingencies are specified
IERR = 14	Invalid base/contingency case flag combination.
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Distribution Factor Data File
IERR = 52	error opening Load Throwover Data File
IERR = 53	error opening Unit Inertia and Governor Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.172. N11_ACCC_PARALLEL_2

Use this API to run the N-1-1 contingency analysis in parallel when the number of contingency processors in Program Settings is greater than 1. The API distributes contingency cases among multiple processors evenly, and tests the contingency cases and following tripping actions assigned to each processor in parallel.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_N11_ACCC_PARALLEL_2  OPTIONS(1)..OPTIONS(16) VALUES LABEL DFXFILE
 ACCFILE CONFILE THRFILE INLFILE
```

Python command syntax:

```
ierr = n11_accc_parallel_2(options, values, label, dfxfile, accfile,
 confile, thrfile, inlfile)
```

Fortran command syntax:

```
CALL N11_ACCC_PARALLEL_2(OPTIONS, VALUES, LABEL, DFXFILE, ACCFILE,
 CONFILE, THRFILE, INLFILE, IERR)
```

Where:

Integer OPTIONS (16)	array of elements specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete.
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall.
OPTIONS(6) = 1	trip.
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable.
OPTIONS(8) = 1	Enable.
OPTIONS(9)	tap adjustment flag for the second N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable.
OPTIONS(9) = 1	enable stepping adjustment.
OPTIONS(9) = 2	enable direct adjustment.
OPTIONS(10)	area interchange adjustment flag for the second N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable.
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange.
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange.
OPTIONS(11)	phase shift adjustment flag for the second N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable.
OPTIONS(11) = 1	enable.
OPTIONS(12)	dc tap adjustment flag for the second N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(12) = 0	disable.
OPTIONS(12) = 1	enable.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(13)	switched shunt adjustment flag for the second N-1 contingency (switched shunt adjustment option setting by default)
	OPTIONS(13) = 0	disable
	OPTIONS(13) = 1	enable
	OPTIONS(13) = 2	enable continuous, disable discrete.
	OPTIONS(14)	induction motor treatment flag for the second N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTIONS(14) = 0	stall
	OPTIONS(14) = 1	trip
	OPTIONS(15)	generation dispatch method
	OPTIONS(15) = 0	disable.
	OPTIONS(15) = 1	subsystem machines (RESERVE).
	OPTIONS(15) = 2	subsystem machines (PMAX).
	OPTIONS(15) = 3	subsystem machines (INERTIA).
	OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP).
	OPTIONS(16)	flag to skip the second N-1 contingencies if the first N-1 contingency is not clean
	OPTIONS(16) = 0	No
	OPTIONS(16) = 1	Yes
Real VALUES (1)	array of 1 elements specifying power flow options(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
Character*32 LABEL	The label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation output. (input).	
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	Is the name of contingency analysis output file (input; no default allowed).	
Character*260 CONFILE	Is the name of Contingency Description Data File for the second N-1 contingencies (input; blank by default).	
Character*260 THRFILE	Is the name of Load Throwover Data File (input; blank by default).	
Character*260 INLFILE	Is the name of Unit Inertia Data File (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	Generators are converted
	IERR = 4	Buses in island(s) without a swing bus--use activity TREE
	IERR = 5	Largest mismatch exceeds tolerance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Generation subsystem not specified
IERR = 7	Too many islands in base case.
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	No Distribution Factor Data input file specified
IERR = 11	No Contingency Solution Output File specified
IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
IERR = 13	No second N-1 contingencies are specified
IERR = 14	Invalid base/contingency case flag combination.
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Distribution Factor Data File
IERR = 52	error opening Load Throwover Data File
IERR = 53	error opening Unit Inertia and Governor Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.173. N11_ACCC_PSCOPF

This API routine is obsolete. It has been replaced by the API routine n11_accc_pscopf_2 .

The API is used to perform N-1-1 contingency analysis in either corrective mode or security constrained optimal power flow mode (SCOPF). In corrective mode, for each pair of N-1 contingencies a full AC load flow is performed for the first N-1 contingency; the post contingency solution is checked against the operation limits and corrective actions are specified to mitigate the limit violations; the post-corrective action solution is used as the base case for the secondary contingency and a full AC load flow is performed to solve the second N-1 contingency.
In the SCOPF mode, a primary contingency is imposed and solved with a full AC load flow solution. A secure control is conducted on the system condition existing after the primary contingency with the secondary contingencies, control adjustments are specified by security constrained optimal power flow to ensure security of the system after the primary contingency. The system condition after the primary contingency and adjustments is used as the base case for secondary contingency; a full AC load flow solution is then performed to solve a secondary N-1 contingency.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_N11_ACCC_PSCOPF  OPTIONS(1)..OPTIONS(18) OPTCOR(1)..OPTCOR(9)
 VALUES(1)..VALUES(10)  LABELS(1)..LABELS(7) DFXFILE ACCFILE  CONFILE
 INLFILE THRFILE
```

Python command syntax:

```
ierr = n11_accc_pscopf(options, optcor, values, labels, dfxfile, accfile,
 confile, inlfile, thrfile)
```

Fortran command syntax:

```
CALL N11_ACCC_PSCOPF(OPTIONS, OPTCOR, VALUES, LABELS, DFXFILE, ACCFILE,
 CONFILE, INLFILE, THRFILE, IERR)
```

Where:

Integer OPTIONS (18)	array of 18 elements specifying integer specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(3) = 0	disable
OPTIONS(3) = 1	enable
OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable
OPTIONS(9) = 1	enable stepping adjustment
OPTIONS(9) = 2	enable direct adjustment
OPTIONS(10)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(11)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable
OPTIONS(11) = 1	enable
OPTIONS(12)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(12) = 0	disable
OPTIONS(12) = 1	enable
OPTIONS(13)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(13) = 0	disable
OPTIONS(13) = 1	enable
OPTIONS(13) = 2	enable continuous, disable discrete
OPTIONS(14)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(14) = 0	stall
OPTIONS(14) = 1	trip
OPTIONS(15)	dispatch mode (0 by default)
OPTIONS(15) = 0	disable
OPTIONS(15) = 1	subsystem machines (RESERVE)
OPTIONS(15) = 2	subsystem machines (PMAX)
OPTIONS(15) = 3	subsystem machines (INERTIA)
OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP)
OPTIONS(16)	flag to skip the second N-1 contingencies if the first N-1 contingency is not clean
OPTIONS(16) = 0	No
OPTIONS(16) = 1	Yes
OPTIONS(17)	Rating set for the first N-1 contingency case
OPTIONS(17) = 1	RATE A
OPTIONS(17) = 2	RATE B
OPTIONS(17) = 3	RATE C
OPTIONS(18)	Rating set for the second N-1 contingency case
OPTIONS(18) = 1	RATE A
OPTIONS(18) = 2	RATE B
OPTIONS(18) = 3	RATE C
Integer OPTCOR (9)	array of 9 elements specifying OPTCOR Is an integer specifying control options(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(1)	System adjustment mode
	OPTCOR(1) = 0	local adjustment only
	OPTCOR(1) = 1	Corrective action mode
	OPTCOR(1) = 2	Preventive security constrained optimal power flow mode
	OPTCOR(2)	maximum iteration number of PSCOPF, required if OPTCOR(1) is 2
	OPTCOR(3)	maximum number of a contingency that does not cause limit violations and instability issue to be tested in the iterative process of PSCOPF, required if OPTCOR(1) is 2
	OPTCOR(4)	generator control flag (1 by default)
	OPTCOR(4) = 0	disable generation MW dispatch in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(4) = 1	enable
	OPTCOR(5)	load control flag (0 by default)
	OPTCOR(5) = 0	disable load shedding control in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(5) = 1	enable
	OPTCOR(6)	phase shifter control flag (1 by default)
	OPTCOR(6) = 0	disable phase shifter control in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(6) = 1	enable
	OPTCOR(7)	off-line generator control flag (0 by default)
	OPTCOR(7) = 0	disable generation MW dispatch of off-line generators in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(7) = 1	enable
	OPTCOR(8)	tap setting adjustment flag (0 by default)
	OPTCOR(8) = 0	disable tap setting adjustment in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(8) = 1	enable
	OPTCOR(9)	switched shunt control flag (0 by default)
	OPTCOR(9) = 0	disable switch shunt control in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(9) = 1	enable
Real VALUES (10)	array of 10 elements specifying VALUES Is a real(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Percentage of rating for the base case (100.0 bye default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(3)	Voltage tolerance in linear programming
	VALUES(4)	Flow tolerance in linear programming
	VALUES(5)	Weighting factor of generation MW dispatch (1.0 bye default)
	VALUES(6)	Weighting factor of load control (1.0 bye default)
	VALUES(7)	Weighting factor of phase shifter control (1.0 bye default)
	VALUES(8)	Weighting factor of generation MW dispatch of off-line generators (1.0 bye default)
	VALUES(9)	Weighting factor of tap setting adjustment (1.0 by default)
	VALUES(10)	Weighting factor of switched shunt control (1.0 by default)
Character*32 LABELS (7)	array of 7 elements specifying specifiying subsystem labels(input).	
	LABELS(1)	label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation (blank for none by default)
	LABELS(2)	label of generation dispatch control system. MW injections of the generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none by default)
	LABELS(3)	label of load control system. Loads in the load control system are available for curtailment in the corrective action mode or preventive security constrained optimal power flow mode (blank for none by default)
	LABELS(4)	label of phase shifter adjustment system. The phase angles of phase shifters in the system are regulated either by corrective action or preventive security constrained optimal power flow (blank for none by default)
	LABELS(5)	label of generation MW control of off-line generator control system. MW injections of the off-line generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none by default)
	LABELS(6)	label of tap adjustment system. The tap positions of transformers in voltage control mode are regulated either by corrective action or

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		preventive security constrained optimal power flow (blank for none by default)
	LABELS(7)	label of switched shunt control system. The admittances of switched shunts in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	name of contingency analysis output file (input; no default allowed).	
Character*260 CONFILE	name of Contingency Description Data File for the second N-1 contingencies (input; blank for none).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	Generators are converted
	IERR = 4	Buses in island(s) without a swing bus--use activity TREE
	IERR = 5	Largest mismatch exceeds tolerance
	IERR = 6	Generation subsystem not specified
	IERR = 7	Too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	No Distribution Factor Data input file specified
	IERR = 11	No Contingency Analysis Output File specified
	IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
	IERR = 13	No second N-1 contingencies are specified
	IERR = 14	Invalid base/contingency case flag combination
	IERR = 15	Invalid OPTCOR value
	IERR = 21	File is not in the form of a PSSE-25 or later DFAX file--run DFAX
	IERR = 22	Monitored elements exceed limit when adding multi-section line members
	IERR = 51	Error opening distribution factor data file
	IERR = 52	Error opening load throwover data file
	IERR = 53	Error opening inertia data file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 54	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.174. N11_ACCC_PSCOPF_2

The API is the second release of N-1-1 contingency analysis solution in either corrective mode or security constrained optimal power flow mode (SCOPF). In corrective mode, for each pair of N-1 contingencies a full AC load flow is performed for the first N-1 ontingency; the post contingency solution is checked against the operation limits and corrective actions are specified to mitigate the limit violations; the post-corrective action solution is used as the base case for the secondary contingency and a full AC load flow is performed to solve the second N-1 contingency.
In the PSCOPF mode, a primary contingency is imposed and solved with a full AC load flow solution. A secure control is conducted on the system condition existing after the primary contingency with the secondary contingencies, control adjustments are specified by security constrained optimal power flow to ensure security of the system after the primary contingency. The system condition after the primary contingency and adjustments is used as the base case for secondary contingency; a full AC load flow solution is then performed to solve a secondary N-1 contingency.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_N11_ACCC_PSCOPF_2  OPTIONS(1)..OPTIONS(20) OPTCOR(1)..OPTCOR(9)
 VALUES(1)..VALUES(10) LABELS(1)..LABELS(7) DFXFILE ACCFILE CONFILE
 THRFILE INLFILE
```

Python command syntax:

```
ierr = n11_accc_pscopf_2(options, optcor, values, labels, dfxfile,
 accfile, confile, inlfile, thrfile)
```

Fortran command syntax:

```
CALL N11_ACCC_PSCOPF_2(OPTIONS, OPTCOR, VALUES, LABELS, DFXFILE, ACCFILE,
 CONFILE, INLFILE, THRFILE, IERR)
```

Where:

Integer OPTIONS (20)	array of elements specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(3) = 0	disable
OPTIONS(3) = 1	enable
OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete.
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall.
OPTIONS(6) = 1	trip.
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the second N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable.
OPTIONS(9) = 1	enable stepping adjustment.
OPTIONS(9) = 2	enable direct adjustment.
OPTIONS(10)	area interchange adjustment flag for the second N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable.
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange.
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(11)	phase shift adjustment flag for the second N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable.
OPTIONS(11) = 1	enable.
OPTIONS(12)	dc tap adjustment flag for the second N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(12) = 0	disable.
OPTIONS(12) = 1	enable.
OPTIONS(13)	switched shunt adjustment flag for the second N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(13) = 0	disable.
OPTIONS(13) = 1	enable.
OPTIONS(13) = 2	enable continuous, disable discrete.
OPTIONS(14)	induction motor treatment flag for the second N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(14) = 0	stall.
OPTIONS(14) = 1	trip.
OPTIONS(15)	dispatch mode (0 by default)
OPTIONS(15) = 0	disable.
OPTIONS(15) = 1	subsystem machines (RESERVE).
OPTIONS(15) = 2	subsystem machines (PMAX).
OPTIONS(15) = 3	subsystem machines (INERTIA).
OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP).
OPTIONS(16)	flag to skip the second N-1 contingencies if the first N-1 contingency is not clean
OPTIONS(16) = 0	No
OPTIONS(16) = 1	Yes
OPTIONS(17)	Rating set for the first N-1 contingency case
OPTIONS(17) = 1	rate A
OPTIONS(17) = 2	rate B
OPTIONS(17) = 3	rate C
OPTIONS(18)	Rating set for the second N-1 contingency case
OPTIONS(18) = 1	rate A
OPTIONS(18) = 2	rate B
OPTIONS(18) = 3	rate C
OPTIONS(19)	voltage limit for the first N-1 contingency case
OPTIONS(19) = 1	normal
OPTIONS(19) = 2	emergency

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(20)	voltage limit for the second N-1 contingency case
	OPTIONS(20) = 1	normal
	OPTIONS(20) = 2	emergency
Integer OPTCOR (9)	array of elements specifying control options(input).	
	OPTCOR(1)	System adjustment mode
	OPTCOR(1) = 0	local adjustment only
	OPTCOR(1) = 1	Corrective action mode
	OPTCOR(1) = 2	Preventive security constrained optimal power flow mode
	OPTCOR(2)	maximum iteration number of PSCOPF, required if OPTCOR(1) is 2
	OPTCOR(3)	maximum number of a contingency that does not cause limit violations and instability issue to be tested in the iterative process of PSCOPF, required if OPTCOR(1) is 2.
	OPTCOR(4)	generator control flag (0 by default)
	OPTCOR(4) = 0	disable generation MW dispatch in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(4) = 1	enable
	OPTCOR(5)	load control flag (0 by default)
	OPTCOR(5) = 0	disable load shedding control in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(5) = 1	enable
	OPTCOR(6)	phase shifter control flag (1 by default)
	OPTCOR(6) = 0	disable phase shifter control in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(6) = 1	enable
	OPTCOR(7)	off-line generator control flag (0 by default)
	OPTCOR(7) = 0	disable generation MW dispatch of off-line generators in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(7) = 1	enable
	OPTCOR(8)	tap setting adjustment flag (0 by default)
	OPTCOR(8) = 0	disable tap setting adjustment in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(8) = 1	enable
	OPTCOR(9)	switched shunt control flag (0 by default)
	OPTCOR(9) = 0	disable switch shunt control in the corrective action mode or preventive security constrained optimal power flow

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(9) = 1	enable
Real VALUES (10)	array of elements specifying power flow settings(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Percentage of rating for the base case (100.0 by default)
	VALUES(3)	Voltage tolerance in linear programming
	VALUES(4)	Flow tolerance in linear programming
	VALUES(5)	Weighting factor of generation MW dispatch (1.0 by default)
	VALUES(6)	Weighting factor of load control (1.0 by default)
	VALUES(7)	Weighting factor of phase shifter control (1.0 by default)
	VALUES(8)	Weighting factor of generation MW dispatch of off-line generators (1.0 by default)
	VALUES(9)	Weighting factor of tap setting adjustment (1.0 by default)
	VALUES(10)	Weighting factor of switched shunt control (1.0 by default)
Character*32 LABELS (7)	array of elements specifying subsystem labels (input).	
	LABELS(1)	The label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation output (blank for none).
	LABELS(2)	The label of generation dispatch control system. MW injections of the generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(3)	The label of load control system. Loads in the load control system are available for curtailment in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(4)	The label of phase shifter adjustment system. The phase angles of phase shifters in the system are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
	LABELS(5)	The label of generation MW control of off-line generator control system. MW injections of the off-line generators within the system are available for dispatch in the corrective ac-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		tion mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(6)	The label of tap adjustment system. The tap positions of transformers in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
	LABELS(7)	The label of switched shunt control system. The admittances of switched shunts in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	Is the name of contingency analysis output file (input; no default allowed).	
Character*260 CONFILE	Is the name of Contingency Description Data File for the second N-1 contingencies (blank for none) (input).	
Character*260 THRFILE	Is the name of Load Throwover Data File (blank for none). (input).	
Character*260 INLFILE	Is the name of Unit Inertia Data File (blank for none). (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	generation dispatch subsystem is not defined
	IERR = 7	too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 9	buses with bus type code and series element status inconsistencies
	IERR = 10	no Distribution Factor Data File specified
	IERR = 11	no Contingency Analysis Output File specified
	IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
	IERR = 13	No second N-1 contingencies are specified
	IERR = 14	Invalid base/contingency case flag combination.
	IERR = 15	Invalid OPTCOR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Distribution Factor Data File
IERR = 52	error opening Load Throwover Data File
IERR = 53	error opening Unit Inertia and Governor Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.175. N11_ACCC_PSCOPF_3

The API is the second release of N-1-1 contingency analysis solution in either corrective mode or security constrained optimal power flow mode (SCOPF). In corrective mode, for each pair of N-1 contingencies a full AC load flow is performed for the first N-1 ontingency; the post contingency solution is checked against the operation limits and corrective actions are specified to mitigate the limit violations; the post-corrective action solution is used as the base case for the secondary contingency and a full AC load flow is performed to solve the second N-1 contingency.
In the PSCOPF mode, a primary contingency is imposed and solved with a full AC load flow solution. A secure control is conducted on the system condition existing after the primary contingency with the secondary contingencies, control adjustments are specified by security constrained optimal power flow to ensure security of the system after the primary contingency. The system condition after the primary contingency and adjustments is used as the base case for secondary contingency; a full AC load flow solution is then performed to solve a secondary N-1 contingency.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_N11_ACCC_PSCOPF_3  OPTIONS(1)..OPTIONS(16) OPTCOR(1)..OPTCOR(9)
 VALUES(1)..VALUES(9) LABELS(1)..LABELS(7) DFXFILE ACCFILE CONFILE THRFILE
 INLFILE
```

Python command syntax:

```
ierr = n11_accc_pscopf_3(options, optcor, values, labels, dfxfile,
 accfile, confile, inlfile, thrfile)
```

Fortran command syntax:

```
CALL N11_ACCC_PSCOPF_3(OPTIONS, OPTCOR, VALUES, LABELS, DFXFILE, ACCFILE,
 CONFILE, INLFILE, THRFILE, IERR)
```

Where:

Integer OPTIONS (16)	array of elements specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the first N-1 contingency (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(3)	phase shift adjustment flag for the first N-1 contingency (phase shift adjustment option setting by default)
OPTIONS(3) = 0	disable
OPTIONS(3) = 1	enable
OPTIONS(4)	dc tap adjustment flag for the first N-1 contingency (dc tap adjustment option setting by default)
OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	switched shunt adjustment flag for the first N-1 contingency (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete.
OPTIONS(6)	induction motor treatment flag for the first N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall.
OPTIONS(6) = 1	trip.
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the second N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable.
OPTIONS(9) = 1	enable stepping adjustment.
OPTIONS(9) = 2	enable direct adjustment.
OPTIONS(10)	area interchange adjustment flag for the second N-1 contingency (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable.
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange.
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(11)	phase shift adjustment flag for the second N-1 contingency (phase shift adjustment option setting by default)
	OPTIONS(11) = 0	disable.
	OPTIONS(11) = 1	enable.
	OPTIONS(12)	dc tap adjustment flag for the second N-1 contingency (dc tap adjustment option setting by default)
	OPTIONS(12) = 0	disable.
	OPTIONS(12) = 1	enable.
	OPTIONS(13)	switched shunt adjustment flag for the second N-1 contingency (switched shunt adjustment option setting by default)
	OPTIONS(13) = 0	disable.
	OPTIONS(13) = 1	enable.
	OPTIONS(13) = 2	enable continuous, disable discrete.
	OPTIONS(14)	induction motor treatment flag for the second N-1 contingency (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
	OPTIONS(14) = 0	stall.
	OPTIONS(14) = 1	trip.
	OPTIONS(15)	dispatch mode (0 by default)
	OPTIONS(15) = 0	disable.
	OPTIONS(15) = 1	subsystem machines (RESERVE).
	OPTIONS(15) = 2	subsystem machines (PMAX).
	OPTIONS(15) = 3	subsystem machines (INERTIA).
	OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP).
	OPTIONS(16)	flag to skip the second N-1 contingencies if the first N-1 contingency is not clean
	OPTIONS(16) = 0	No
	OPTIONS(16) = 1	Yes
Integer OPTCOR (9)	array of elements specifying control options(input).	
	OPTCOR(1)	System adjustment mode
	OPTCOR(1) = 0	local adjustment only
	OPTCOR(1) = 1	Corrective action mode
	OPTCOR(1) = 2	Preventive security constrained optimal power flow mode
	OPTCOR(2)	maximum iteration number of PSCOPF, required if OPTCOR(1) is 2
	OPTCOR(3)	maximum number of a contingency that does not cause limit violations and instability issue to be tested in the iterative process of PSCOPF, required if OPTCOR(1) is 2.
	OPTCOR(4)	generator control flag (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTCOR(4) = 0	disable generation MW dispatch in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(4) = 1	enable
	OPTCOR(5)	load control flag (0 by default)
	OPTCOR(5) = 0	disable load shedding control in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(5) = 1	enable
	OPTCOR(6)	phase shifter control flag (1 by default)
	OPTCOR(6) = 0	disable phase shifter control in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(6) = 1	enable
	OPTCOR(7)	off-line generator control flag (0 by default)
	OPTCOR(7) = 0	disable generation MW dispatch of off-line generators in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(7) = 1	enable
	OPTCOR(8)	tap setting adjustment flag (0 by default)
	OPTCOR(8) = 0	disable tap setting adjustment in the corrective action mode or preventive security constrained optimal power flow mode
	OPTCOR(8) = 1	enable
	OPTCOR(9)	switched shunt control flag (0 by default)
	OPTCOR(9) = 0	disable switch shunt control in the corrective action mode or preventive security constrained optimal power flow
	OPTCOR(9) = 1	enable
Real VALUES (9)	array of elements specifying power flow settings(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Voltage tolerance in linear programming
	VALUES(3)	Flow tolerance in linear programming
	VALUES(4)	Weighting factor of generation MW dispatch (1.0 by default)
	VALUES(5)	Weighting factor of load control (1.0 by default)
	VALUES(6)	Weighting factor of phase shifter control (1.0 by default)
	VALUES(7)	Weighting factor of generation MW dispatch of off-line generators (1.0 by default)
	VALUES(8)	Weighting factor of tap setting adjustment (1.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(9)	Weighting factor of switched shunt control (1.0 by default)
Character*32 LABELS (7)	array of elements specifying subsystem labels (input).	
	LABELS(1)	The label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation output (blank for none).
	LABELS(2)	The label of generation dispatch control system. MW injections of the generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(3)	The label of load control system. Loads in the load control system are available for curtailment in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(4)	The label of phase shifter adjustment system. The phase angles of phase shifters in the system are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
	LABELS(5)	The label of generation MW control of off-line generator control system. MW injections of the off-line generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(6)	The label of tap adjustment system. The tap positions of transformers in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
	LABELS(7)	The label of switched shunt control system. The admittances of switched shunts in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	
Character*260 ACCFILE	Is the name of contingency analysis output file (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 CONFILE	Is the name of Contingency Description Data File for the second N-1 contingencies (blank for none) (input).
Character*260 THRFILE	Is the name of Load Throwover Data File (blank for none). (input).
Character*260 INLFILE	Is the name of Unit Inertia Data File (blank for none). (input).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid OPTIONS value
IERR = 2	invalid VALUES value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	generation dispatch subsystem is not defined
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	no Distribution Factor Data File specified
IERR = 11	no Contingency Analysis Output File specified
IERR = 12	Can not read the Contingency Description Data File for the second N-1 contingencies
IERR = 13	No second N-1 contingencies are specified
IERR = 14	Invalid base/contingency case flag combination.
IERR = 15	Invalid OPTCOR value
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Distribution Factor Data File
IERR = 52	error opening Load Throwover Data File
IERR = 53	error opening Unit Inertia and Governor Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.176. NETG

Use this API to change the in-service generation to negative MVA load at all Type 2 and 3 buses except those in the subsystem specified by the user.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_NETG  SID  ALL
```

Python command syntax:

```
ierr = netg(sid, all)
```

Fortran command syntax:

```
CALL NETGNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses outside of subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.177. NEWCAS

This API routine is obsolete. It has been replaced by the API routine newcase_2 .

Use this API to initialize a new power flow case. All existing working case data will be lost upon execution of this API.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_NEWCAS  BASEMVA  TITL1  TITL2
```

Python command syntax:

```
ierr = newcas(basemva, titl1, titl2)
```

Fortran command syntax:

```
CALL NEWCASNW(BASEMVA, TITL1, TITL2, IERR)
```

Where:

Real BASEMVA	new system base MVA. Valid base MVA values range from 1.0 to 10,000.0 (input; 100.0).	
Character*60 TITL1	first line of the new case title (input; blank by default).	
Character*60 TITL2	second line of the new case title (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid BASEMVA value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.178. NEWCASE_2

This API is the second release of the function used to initialize a new power flow case. All existing working case data will be lost upon execution of this API.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_NEWCASE_2  OPTIONS(1)  OPTIONS(2)  BASEMVA  BASEFREQ  TITL1  TITL2
```

Python command syntax:

```
ierr = newcase_2(options, basemva, basefreq, titl1, titl2)
```

Fortran command syntax:

```
CALL NEWCASE_2(OPTIONS, BASEMVA, BASEFREQ, TITL1, TITL2, IERR)
```

Where:

Integer OPTIONS (2)	array of 2 elements specifying units for ratings (used to set the per-cent loading program option settings)(input). OPTIONS(1) units for transformer ratings OPTIONS(1) > 0 current expressed as MVA OPTIONS(1) < 0 MVA OPTIONS(2) units for ratings of non-transformer branches OPTIONS(2) > 0 current expressed as MVA OPTIONS(2) < 0 MVA
Real BASEMVA	system base MVA. Valid base MVA values range from 1.0 to 10,000.0 (input; 100.0).
Real BASEFREQ	system base frequency. Valid base frequency values range from greater than 0.0 to less than 100.0 (input; present base frequency option setting).
Character*60 TITL1	first line of the new case title (input; blank by default).
Character*60 TITL2	second line of the new case title (input; blank by default).
Integer IERR	error code (output). IERR = 0 no error occurred IERR = 1 invalid BASEMVA value IERR = 2 invalid BASEFREQ value IERR = 3 prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.179. NEWTON_TOLERANCE

Use this API to specify or return the value of the default Newton-Raphson convergence tolerance.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_NEWTON_TOLERANCE RVAL
```

Python command syntax:

```
ierr = newton_tolerance(rval)

ierr, rval = newton_tolerance()
```

Fortran command syntax:

```
CALL NEWTON_TOLERANCE_API(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0.1 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.180. NEW_DIMENSION

Use this API to reset PSSE to an increased bus size level.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_NEW_DIMENSION BUSES
```

Python command syntax:

```
ierr = new_dimension(buses)
```

Fortran command syntax:

```
CALL NEW_DIMENSION(BUSES, IERR)
```

Where:

Integer BUSES	maximum number of buses allowed at the new size level If BUSES is less than the current maximum number of buses, PSSE's capacity limits are not changed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid BUSES value
	IERR = 2	prerequisite requirements for API are not met
	IERR = 3	Unable to change network to new bus size. PSSE Xplore is limited to 50 buses.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.181. NON_DIVERGENT

Use this API to specify or return the option to specify or return the option to enable or disable the non-divergent Newton power flow solution.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_NON_DIVERGENT IVAL
```

Python command syntax:

```
ierr = non_divergent(ival)

ierr, ival = non_divergent()
```

Fortran command syntax:

```
CALL NON_DIVERGENT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	disable non-divergent Newton solution option.
	IVAL = 1	enable non-divergent Newton solution option.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.182. NON_TRANS_PERCENT_UNITS

Use this API to set the non-transformer branch percent units option setting to either MVA or current expressed as MVA.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_NON_TRANS_PERCENT_UNITS IVAL
```

Python command syntax:

```
ierr = non_trans_percent_units(ival)

ierr, ival = non_trans_percent_units()
```

Fortran command syntax:

```
CALL NON_TRANS_PERCENT_UNITS_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	MVA.
	IVAL = 1	current expressed as MVA.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.183. NSOL

Use this API to apply the decoupled Newton-Raphson power flow calculation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_NSOL  OPTIONS(1)  OPTIONS(2)  OPTIONS(3)  OPTIONS(4)  OPTIONS(5)
 OPTIONS(6)  OPTIONS(7)
```

Python command syntax:

```
ierr = nsol(options)
```

Fortran command syntax:

```
CALL NSOLAPI(OPTIONS, IERR)
```

Where:

Integer OPTIONS (7)	array of 7 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (use tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (use area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (use phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (use dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (use switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	flat start flag (0 by default)
	OPTIONS(6) = 0	do not flat start
	OPTIONS(6) = 1	flat start
	OPTIONS(6) = 2	flat start, then estimate voltage magnitudes
	OPTIONS(6) = 3	flat start, then estimate voltage phase angles
	OPTIONS(6) = 4	flat start, then estimate voltage magnitudes and phase angles
	OPTIONS(7)	var limit flag (99 by default)
	OPTIONS(7) = 0	apply var limits immediately
	OPTIONS(7) > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	OPTIONS(7) = -1	ignore var limits
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted
	IERR = 3	buses in island(s) without a swing bus; use activity TREE
	IERR = 4	bus type code and series element status inconsistencies
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.184. NUMBER_THREADS

Use this API to specify or return the number of processors available for parallel operations.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_NUMBER_THREADS IVAL
```

Python command syntax:

```
ierr = number_threads(ival)

ierr, ival = number_threads()
```

Fortran command syntax:

```
CALL NUMBER_THREADS_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting between 1 and the number of processors of the computer (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.185. ORDR

Use this API to calculate a sparsity preserving ordering of buses in preparation for the processing of network matrices.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ORDR  OPT
```

Python command syntax:

```
ierr = ordr(opt)
```

Fortran command syntax:

```
CALL ORDRAPI(OPT, IERR)
```

Where:

Integer OPT	flag for ordering option (input; 0 by default).	
	OPT = 0	assume all branches between in-service buses are in-service
	OPT = 1	ignore out-of-service branches in calculating the bus ordering
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.186. OTDF

Use this API to tabulate vectors of distribution factors using a linear network (dc) model.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OTDF DFXFILE
```

Python command syntax:

```
ierr = otdf(dfxfile)
```

Fortran command syntax:

```
CALL OTDFAPI(DFXFILE, IERR)
```

Where:

Character*260 DFXFILE	name of DFAX data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no DFAX data file specified
	IERR = 2	generators are converted
	IERR = 3	buses in island(s) without a swing bus; use activity TREE
	IERR = 4	error opening file DFXFILE
	IERR = 5	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 6	file DFXFILE does not contain distribution factors; run DFAX with distribution factor option enabled
	IERR = 7	no monitored branches or interfaces defined
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.187. OUTPUT_Y_MATRIX

Use this API to display the network admittance matrix for a subsystem of the working case in the form of a list of matrix terms.

This API routine was first introduced in release 30.1.0.

Batch command syntax:

```
BAT_OUTPUT_Y_MATRIX SID ALL TIES OUT OFILE
```

Python command syntax:

```
ierr = output_y_matrix(sid, all, ties, out, ofile)
```

Fortran command syntax:

```
CALL OUTPUT_Y_MATRIX(SID, ALL, TIES, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only rows corresponding to buses in subsystem SID
	ALL = 1	process the complete case
Integer TIES	option for the omission or inclusion of off-diagonal terms corresponding to ties from the specified subsystem; ignored if ALL is 1 (input; 0 by default).	
	TIES = 0	omit ties
	TIES = 1	include ties
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid TIES value
	IERR = 4	invalid OUT value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.188. OUTS

Use this API to tabulate those components in the working case that are removed from service.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OUTS SID ALL
```

Python command syntax:

```
ierr = outs(sid, all)
```

Fortran command syntax:

```
CALL OUTSNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.189. OWNM

This API routine is obsolete. It has been replaced by the API routine ownm_2 .

Use this API routine to reassign the buses, load, machines, and branches in a specified subsystem of the working case from their original owner to a designated owner.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OWNM  SID  ALL  STATUS(1)..STATUS(7)  IONEW
```

Python command syntax:

```
ierr = ownm(sid, all, status, ionew)
```

Fortran command syntax:

```
CALL OWNMAPI(SID, ALL, STATUS, IONEW, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (7)	array of 7 elements specifying They are used to control reassignment options(input).	
	STATUS(1)	bus owners to IONEW (0 by default)
	STATUS(1) = 0	leave bus ownership unchanged
	STATUS(1) = 1	Change bus owners to IONEW
	STATUS(2)	load owners to IONEW (0 by default)
	STATUS(2) = 0	leave load ownership unchanged
	STATUS(2) = 1	change load owners to IONEW
	STATUS(3)	machine owners to IONEW (0 by default)
	STATUS(3) = 0	leave machine ownership unchanged
	STATUS(3) = 1	change machine owners to IONEW
	STATUS(4)	branch owners to IONEW (0 by default)
	STATUS(4) = 0	leave branch ownership unchanged
	STATUS(4) = 1	change branch owners to IONEW
	STATUS(5)	FACTS device owners to IONEW (0 by default)
	STATUS(5) = 0	leave FACTS device ownership unchanged
	STATUS(5) = 1	change FACTS device owners to IONEW
	STATUS(6)	VSC dc line owners to IONEW (0 by default)
	STATUS(6) = 0	leave VSC dc line ownership unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(6) = 1	change VSC dc line owners to IONEW
	STATUS(7)	branch, series FACTS device, VSC dc line option (1 by default)
	STATUS(7) = 1	subsystem branches only
	STATUS(7) = 2	subsystem ties only
	STATUS(7) = 3	both subsystem branches and ties
Integer IONEW	new owner to which the specified equipment items are to be assigned. IONEW must be between 1 and 9999 (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid new owner number
	IERR = 5	owner table is full; cannot add owner to the working case
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.190. OWNM_2

This API routine is obsolete. It has been replaced by the API routine ownm_3 .

This API routine is the second release of the owner renumbering function. Use this API routine to reassign the buses, loads, machines, branches, FACTS devices, VSC dc lines, and/or GNE devices in a specified subsystem of the working case from their original owner to a designated owner.

This API routine was first introduced in release 32.0.2.

Batch command syntax:

```
BAT_OWNM_2  SID  ALL  STATUS(1)..STATUS(8)  IONEW
```

Python command syntax:

```
ierr = ownm_2(sid, all, status, ionew)
```

Fortran command syntax:

```
CALL OWNMAPI_2(SID, ALL, STATUS, IONEW, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (8)	array of 8 elements specifying They are used to control reassignment options(input).	
	STATUS(1)	bus owners to IONEW (0 by default)
	STATUS(1) = 0	leave bus ownership unchanged
	STATUS(1) = 1	Change bus owners to IONEW
	STATUS(2)	load owners to IONEW (0 by default)
	STATUS(2) = 0	leave load ownership unchanged
	STATUS(2) = 1	change load owners to IONEW
	STATUS(3)	machine owners to IONEW (0 by default)
	STATUS(3) = 0	leave machine ownership unchanged
	STATUS(3) = 1	change machine owners to IONEW
	STATUS(4)	branch owners to IONEW (0 by default)
	STATUS(4) = 0	leave branch ownership unchanged
	STATUS(4) = 1	change branch owners to IONEW
	STATUS(5)	FACTS device owners to IONEW (0 by default)
	STATUS(5) = 0	leave FACTS device ownership unchanged
	STATUS(5) = 1	change FACTS device owners to IONEW
	STATUS(6)	VSC dc line owners to IONEW (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(6) = 0	leave VSC dc line ownership unchanged
	STATUS(6) = 1	change VSC dc line owners to IONEW
	STATUS(7)	GNE device owners to IONEW (0 by default)
	STATUS(7) = 0	leave GNE device ownership unchanged
	STATUS(7) = 1	change GNE device owners to IONEW
	STATUS(8)	branch, series FACTS device, VSC dc line, and series GNE device option (1 by default)
	STATUS(8) = 1	subsystem branches only
	STATUS(8) = 2	subsystem ties only
	STATUS(8) = 3	both subsystem branches and ties
Integer IONEW	new owner to which the specified equipment items are to be assigned. IONEW must be between 1 and 9999 (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid new owner number
	IERR = 5	owner table is full; cannot add owner to the working case
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.191. OWNM_3

This API routine is the third release of the owner renumbering function. Use this API routine to reassign the buses, loads, machines, induction machines, branches, FACTS devices, VSC dc lines, and/or GNE devices in a specified subsystem of the working case from their original owner to a designated owner.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_OWNM_3  SID  ALL  STATUS(1)..STATUS(9)  IONEW
```

Python command syntax:

```
ierr = ownm_3(sid, all, status, ionew)
```

Fortran command syntax:

```
CALL OWNM_3(SID, ALL, STATUS, IONEW, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (9)	array of 9 elements specifying They are used to control reassignment options(input).	
	STATUS(1)	bus owners to IONEW (0 by default)
	STATUS(1) = 0	leave bus ownership unchanged
	STATUS(1) = 1	change bus owners to IONEW
	STATUS(2)	load owners to IONEW (0 by default)
	STATUS(2) = 0	leave load ownership unchanged
	STATUS(2) = 1	change load owners to IONEW
	STATUS(3)	machine owners to IONEW (0 by default)
	STATUS(3) = 0	leave machine ownership unchanged
	STATUS(3) = 1	change machine owners to IONEW
	STATUS(4)	induction machine owners to IONEW (0 by default)
	STATUS(4) = 0	leave induction machine ownership unchanged
	STATUS(4) = 1	change induction machine owners to IONEW
	STATUS(5)	branch owners to IONEW (0 by default)
	STATUS(5) = 0	leave branch ownership unchanged
	STATUS(5) = 1	change branch owners to IONEW
	STATUS(6)	FACTS device owners to IONEW (0 by default)
	STATUS(6) = 0	leave FACTS device ownership unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(6) = 1	change FACTS device owners to IONEW
	STATUS(7)	VSC dc line owners to IONEW (0 by default)
	STATUS(7) = 0	leave VSC dc line ownership unchanged
	STATUS(7) = 1	change VSC dc line owners to IONEW
	STATUS(8)	GNE device owners to IONEW (0 by default)
	STATUS(8) = 0	leave GNE device ownership unchanged
	STATUS(8) = 1	change GNE device owners to IONEW
	STATUS(9)	branch, series FACTS device, VSC dc line, and series GNE device option (1 by default)
	STATUS(9) = 1	subsystem branches only
	STATUS(9) = 2	subsystem ties only
	STATUS(9) = 3	both subsystem branches and ties
Integer IONEW	new owner to which the specified equipment items are to be assigned. IONEW must be between 1 and 9999 (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid new owner number
	IERR = 5	owner table is full; cannot add owner to the working case
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.192. OWNR

Use this API to tabulate owner totals by owner.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OWNR SID ALL
```

Python command syntax:

```
ierr = ownr(sid, all)
```

Fortran command syntax:

```
CALL OWNRNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Owner subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all owners or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only owners in owner subsystem SID
	ALL = 1	process all owners
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.193. PATH

Use this API to specify a directory pathname. Then, whenever a simple filename preceded by an ampersand (for example, &myfile) is specified as input to a PSSE API, the file is opened in the directory specified rather than in the user's current directory.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_PATH PATHNAME
```

Python command syntax:

```
ierr = path(pathname)
```

Fortran command syntax:

```
CALL PATHAPI(PATHNAME, IERR)
```

Where:

Character*260 PATHNAME	name of path to be used (blank to disable) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error setting path PATHNAME
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.194. PHASE_SHIFT_ADJUSTMENT

Use this API to specify or return the option to enable or disable the phase shift adjustment.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_PHASE_SHIFT_ADJUSTMENT IVAL
```

Python command syntax:

```
ierr = phase_shift_adjustment(ival)

ierr, ival = phase_shift_adjustment()
```

Fortran command syntax:

```
CALL PHASE_SHIFT_ADJUSTMENT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	disabled.
	IVAL = 1	enabled.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.195. POLY

Use this API to calculate interchange limits of a study system against two opposing systems using a linear network (dc) model.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_POLY  OPTIONS(1)..OPTIONS(15)  VALUES(1)..VALUES(6)
 LABELS(1)..LABELS(3)  DFXFILE  RESFILE
```

Python command syntax:

```
ierr = poly(options, values, labels, dfxfile, resfile)
```

Fortran command syntax:

```
CALL POLYAPI(OPTIONS, VALUES, LABELS, DFXFILE, RESFILE, IERR)
```

Where:

Integer OPTIONS (15) array of 15 elements specifying calculation and reporting options(input).

OPTIONS(1)	base case rating set (rating set program option setting by default)
OPTIONS(1) = 1	RATEA
OPTIONS(1) = 2	RATEB
OPTIONS(1) = 3	RATEC
OPTIONS(2)	contingency case rating set (rating set program option setting by default)
OPTIONS(2) = 1	RATEA
OPTIONS(2) = 2	RATEB
OPTIONS(2) = 3	RATEC
OPTIONS(3)	base case line flow code (0 by default)
OPTIONS(3) = 0	dc base case
OPTIONS(3) = 1	ac base case
OPTIONS(4)	phase shifter code (0 by default)
OPTIONS(4) = 0	locked in base case shift solutions
OPTIONS(4) = 1	regulating in base case shift solutions
OPTIONS(5)	code for the treatment of base case line flow constraints in contingency cases (0 by default)
OPTIONS(5) = 0	ignore
OPTIONS(5) = 1	include
OPTIONS(6)	list study system buses (0 by default)
OPTIONS(6) = 0	no
OPTIONS(6) = 1	yes
OPTIONS(7)	list opposing system buses (0 by default)
OPTIONS(7) = 0	no

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(7) = 1	yes
	OPTIONS(8)	list study system tie lines (0 by default)
	OPTIONS(8) = 0	no
	OPTIONS(8) = 1	yes
	OPTIONS(9)	add study system tie lines to monitored line list (0 by default)
	OPTIONS(9) = 0	no
	OPTIONS(9) = 1	yes
	OPTIONS(10)	format code (0 by default)
	OPTIONS(10) = 0	summary output
	OPTIONS(10) = 1	full output
	OPTIONS(11)	convert MVA ratings to estimated MW ratings (0 by default)
	OPTIONS(11) = 0	no
	OPTIONS(11) = 1	yes
	OPTIONS(12)	discontinue activity if MW mismatch exceeds VALUES(1) (0 by default)
	OPTIONS(12) = 0	discontinue
	OPTIONS(12) = 1	continue
	OPTIONS(13)	graphical option (0 by default)
	OPTIONS(13) = 0	plot combined case only
	OPTIONS(13) = 1	plot combined case only
	OPTIONS(13) = 2	plot for all system conditions
	OPTIONS(14)	option for plotting constraints (0 by default)
	OPTIONS(14) = 0	plot all constraints
	OPTIONS(14) = 1	plot only constraints forming the boundary of the feasible region
	OPTIONS(15)	graphical format (0 by default)
	OPTIONS(15) = 0	single page per case
	OPTIONS(15) = 1	two pages per case
Real VALUES (6)	array of 6 elements specifying tolerances and thresholds(input).	
	VALUES(1)	required MW mismatch tolerance for continuing (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating for reporting an element as overloaded (100.0 by default)
	VALUES(3)	maximum opposing system generation shift (10000.0 by default)
	VALUES(4)	maximum opposing system delta interchange for plotting (1000.0 by default)
	VALUES(5)	minimum distribution factor magnitude (0.0 by default)
	VALUES(6)	study system generation shift (100.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*32 LABELS (3)	array of 3 elements specifying subsystem labels(input).	
	LABELS(1)	study system label (no default allowed)
	LABELS(2)	first opposing system label (no default allowed)
	LABELS(3)	second opposing system label (no default allowed)
Character*260 DFXFILE	name of DFAX data file (input; no default allowed).	
Character*260 RESFILE	name of POLY results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	no Distribution Factor Data File specified
	IERR = 4	same label specified for at least two subsystems
	IERR = 5	no POLY Results Output File specified
	IERR = 6	generators are converted
	IERR = 7	buses in island(s) without a swing bus; use activity TREE
	IERR = 8	largest mismatch exceeds mismatch tolerance
	IERR = 9	error opening file DFXFILE
	IERR = 10	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 11	file DFXFILE does not contain distribution factors; run DFAX with distribution factor option enabled
	IERR = 12	no monitored branches or interfaces defined
	IERR = 13	no subsystems defined
	IERR = 14	subsystem not found
	IERR = 15	specified subsystems overlap
	IERR = 16	no in-service buses in subsystem WORLD
	IERR = 17	no tie lines from study system
	IERR = 18	only one subsystem defined
	IERR = 19	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.196. POUT

Use this API to print the power flow solution results with boundary condition and flow information on the left side of the report and other information on the right side.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_POUT SID ALL
```

Python command syntax:

```
ierr = pout(sid, all)
```

Fortran command syntax:

```
CALL POUTNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.197. POWER_OUTPUT

Use this API to specify or return the option to display power output in either MVA or kVA.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_POWER_OUTPUT IVAL
```

Python command syntax:

```
ierr = power_output(ival)

ierr, ival = power_output()
```

Fortran command syntax:

```
CALL POWER_OUTPUT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	kVA.
	IVAL = 1	MVA.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.198. PP_ACCC

Use this API to report the results of the AC contingency calculation function.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PP_ACCC OPTIONS(1)..OPTIONS(5) INTVAL(1)..INTVAL(5)
 REALVAL(1)..REALVAL(6) ACCFILE
```

Python command syntax:

```
ierr = pp_accc(options, intval, realval, accfile)
```

Fortran command syntax:

```
CALL PP_ACCC(OPTIONS, INTVAL, REALVAL, ACCFILE, IERR)
```

Where:

Integer OPTIONS (5)	array of 5 elements specifying reporting options(input).	
	OPTIONS(1)	contingency case rating (rating set program option setting by default)
	OPTIONS(1) = 0	RATEA
	OPTIONS(1) = 1	RATEB
	OPTIONS(1) = 2	RATEC
	OPTIONS(2)	report format (3 by default)
	OPTIONS(2) = 0	spreadsheet overload report
	OPTIONS(2) = 1	spreadsheet loading table
	OPTIONS(2) = 2	available capacity table
	OPTIONS(2) = 3	non-spreadsheet overload report
	OPTIONS(2) = 4	non-spreadsheet loading table
	OPTIONS(2) = 5	non-converged networks report
	OPTIONS(3)	exclude interfaces from report (0 by default)
	OPTIONS(3) = 0	no
	OPTIONS(3) = 1	yes
	OPTIONS(4)	run voltage limit check (0 by default)
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
	OPTIONS(5)	exclude cases with no overloads from non-spreadsheet overload report (0 by default)
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
Integer INTVAL (5)	array of 5 elements specifying integer reporting parameters(input).	
	INTVAL(1)	number of low voltage range violations (0 by default)
	INTVAL(2)	number of high voltage range violations (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTVAL(3)	number of voltage deviation violations (0 by default)
	INTVAL(4)	number of buses in the largest disconnected island (0 by default)
	INTVAL(5)	maximum number of elements in available capacity table (no limit by default)
Real REALVAL (6)	array of 6 elements specifying real reporting parameters(input).	
	REALVAL(1)	percent of flow rating (100.0 by default)
	REALVAL(2)	minimum contingency case flow change for overload report (0.0 by default)
	REALVAL(3)	cutoff threshold for available capacity table (99999.0 by default)
	REALVAL(4)	minimum contingency case voltage change for range violations (0.0 by default)
	REALVAL(5)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	REALVAL(6)	system mismatch converged tolerance (MVA) (5.0 by default)
Character*260 ACCFILE	name of the contingency solution output file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid INTVAL value
	IERR = 3	invalid REALVAL value
	IERR = 4	error opening RFILE
	IERR = 5	error reading RFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.199. PP_ACCC_MULTI_CASE

This API routine is obsolete. It has been replaced by the API routine accc_multiple_run_report .

Use this API to report the results of up to nine executions of the AC contingency calculation function.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_PP_ACCC_MULTI_CASE OPTIONS(1)..OPTIONS(11) VALUES(1)..VALUES(6) NFILES
 ACCFILES(1)..ACCFILES(NFILES)
```

Python command syntax:

```
ierr = pp_accc_multi_case(options, values, nfiles, accfiles)
```

Fortran command syntax:

```
CALL PP_ACCC_MULTI_CASE(OPTIONS, VALUES, NFILES, ACCFILES, IERR)
```

Where:

Integer OPTIONS (11)	array of 11 elements specifying reporting options(input).	
	OPTIONS(1)	column headings code (1 by default)
	OPTIONS(1) = 1	contingency solution output file names
	OPTIONS(1) = 2	saved case file names as contained in the contingency solution output files
	OPTIONS(2)	rating set (rating set program option setting by default)
	OPTIONS(2) = 1	RateA
	OPTIONS(2) = 2	RateB
	OPTIONS(2) = 3	RateC
	OPTIONS(3)	print monitored elements summary report (0 by default)
	OPTIONS(3) = 0	no
	OPTIONS(3) = 1	yes
	OPTIONS(4)	print missing monitored elements report (0 by default)
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
	OPTIONS(5)	print missing monitored voltage buses report (0 by default)
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
	OPTIONS(6)	print contingency legend (2 by default)
	OPTIONS(6) = 0	no
	OPTIONS(6) = 1	complete master legend report only
	OPTIONS(6) = 2	reduced legend with each table

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6) = 3	both master legend report and legend with each table
	OPTIONS(7)	print missing contingencies report (0 by default)
	OPTIONS(7) = 0	no
	OPTIONS(7) = 1	yes
	OPTIONS(8)	print non-converged contingencies report (0 by default)
	OPTIONS(8) = 0	no
	OPTIONS(8) = 1	yes
	OPTIONS(9)	print loading violation reports (0 by default)
	OPTIONS(9) = 0	no
	OPTIONS(9) = 1	base case and worst case contingency violations only
	OPTIONS(9) = 2	base case and all contingency violations only
	OPTIONS(9) = 3	base case, worst case contingency, and all contingency violations
	OPTIONS(10)	print voltage violation reports (0 by default)
	OPTIONS(10) = 0	no
	OPTIONS(10) = 1	base case and worst case contingency violations only
	OPTIONS(10) = 2	base case and all contingency violations only
	OPTIONS(10) = 3	base case, worst case contingency, and all contingency violations
	OPTIONS(11)	option for processing interfaces in loading violation reports (1 by default)
	OPTIONS(11) = 0	exclude interfaces from loading violation checking and reporting
	OPTIONS(11) = 1	check and report interface loading violations
Real VALUES (6)	array of 6 elements specifying integer reporting parameters(input).	
	VALUES(1)	percent of flow rating for reporting (100.0 by default)
	VALUES(2)	percent of flow rating for counting in worst case contingency violations report (> VALUES(1)) (100.0 by default)
	VALUES(3)	minimum contingency case flow change for reporting and counting (0.0 by default)
	VALUES(4)	minimum contingency case voltage change for reporting and counting in voltage range violations (0.0 by default)
	VALUES(5)	bus mismatch converged tolerance (MW or Mvar) (0.5 by default)
	VALUES(6)	system mismatch converged tolerance (MVA) (5.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NFILES	number of contingency solution output files to be processed (1 through 9) (input; no default allowed).	
Character*260 ACCFILES (NFILES)	NFILES names of contingency solution output files (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	invalid NFILES value
	IERR = 4	the percent for counting is less than the percent for reporting
	IERR = 5	file is not in the form of a current contingency solution output file
	IERR = 6	error opening a contingency solution output file
	IERR = 7	error reading a contingency solution output file
	IERR = 8	error opening a temporary file
	IERR = 9	error opening a subfile in the temporary file
	IERR = 10	error reading or writing the temporary file
	IERR = 11	error closing a subfile in the temporary file
	IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.200. PRINT_OUTAGED_BRANCHES

Use this API to specify or return the option to enable or disable whether to print out-of-service branches.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PRINT_OUTAGED_BRANCHES IVAL
```

Python command syntax:

```
ierr = print_outaged_branches(ival)

ierr, ival = print_outaged_branches()
```

Fortran command syntax:

```
CALL PRINT_OUTAGED_BRANCHES_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	do not print out-of-service branches.
	IVAL = 1	print out-of-service branches.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.201. PRINT_WINDING_BUSES

Use this API to specify or return the option to enable or disable whether to print winding "to" buses of three-winding transformers.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PRINT_WINDING_BUSES IVAL
```

Python command syntax:

```
ierr = print_winding_buses(ival)

ierr, ival = print_winding_buses()
```

Fortran command syntax:

```
CALL PRINT_WINDING_BUSES_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	do not print winding "to" buses.
	IVAL = 1	print winding "to" buses.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.202. PROGRESS_OUTPUT

Use this API to specify the progress output device.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PROGRESS_OUTPUT ISLCT FILARG OPTIONS(1) OPTIONS(2)
```

Python command syntax:

```
ierr = progress_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL OPENPROGRESSDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input; 1 by default).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for progress is the Progress tab of the output bar.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input; blank by default).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options (0 by default)
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only) (0 by default)
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error occurred
IERR = 1	prerequisite requirements for API are not met
IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.203. PROMPT_OUTPUT

Use this API to specify the prompt output device.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PROMPT_OUTPUT ISLCT FILARG OPTIONS(1) OPTIONS(2)
```

Python command syntax:

```
ierr = prompt_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL OPENPROMPTDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input; 1 by default).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for prompts is the input window generated by user input requests.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input; blank by default).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options (0 by default)
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only) (0 by default)
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error occurred
IERR = 1	prerequisite requirements for API are not met
IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.204. PRTI

Use this API to print the 16 line long title.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_PRTI
```

Python command syntax:

```
ierr = prti()
```

Fortran command syntax:

```
CALL PRTIAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.205. PSCOPF

This API routine is obsolete. It has been replaced by the API routine pscopf_2 .

This API is used to find the settings of controls for a system to comply with base case and contingency case operation limits. The API is implemented with preventive security constrained optimal power flow method (PSCOPF). The objective of the PSCOPF is to minimize control adjustments subject to operation limits constraints. The available controls are generation MW outputs of on-line and off-line generators, loads, phase shifters, tap settings and switched shunts.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_PSCOPF  OPTIONS(1)..OPTIONS(26) VALUES(1)..VALUES(11)
 LABELS(1)..LABELS(7) DFXFILE INLFILE THRFILE
```

Python command syntax:

```
ierr = pscopf(options,values,labels,dfxfile,inlfile,thrfile)
```

Fortran command syntax:

```
CALL PSCOPFAPI(OPTIONS,VALUES, LABELS,DFXFILE,INLFILE,THRFILE,IERR)
```

Where:

Integer OPTIONS (26)	array of 26 elements specifying integer specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the base case (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the base case (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag for the base case (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag for the base case (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5)	switched shunt adjustment flag for the base case (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete
OPTIONS(6)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable
OPTIONS(9) = 1	enable stepping adjustment
OPTIONS(9) = 2	enable direct adjustment
OPTIONS(10)	area interchange adjustment flag for the contingency case (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(11)	phase shift adjustment flag for the contingency case (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable
OPTIONS(11) = 1	enable
OPTIONS(12)	dc tap adjustment flag for the contingency case (dc tap adjustment option setting by default)
OPTIONS(12) = 0	disable
OPTIONS(12) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(13)	switched shunt adjustment flag for the contingency case (switched shunt adjustment option setting by default)
OPTIONS(13) = 0	disable
OPTIONS(13) = 1	enable
OPTIONS(13) = 2	enable continuous, disable discrete
OPTIONS(14)	induction motor treatment flag for the contingency case (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(14) = 0	stall
OPTIONS(14) = 1	trip
OPTIONS(15)	dispatch mode (0 by default)
OPTIONS(15) = 0	disable
OPTIONS(15) = 1	subsystem machines (RESERVE)
OPTIONS(15) = 2	subsystem machines (PMAX)
OPTIONS(15) = 3	subsystem machines (INERTIA)
OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP)
OPTIONS(16)	Rating set for the base case
OPTIONS(16) = 1	RATE A
OPTIONS(16) = 2	RATE B
OPTIONS(16) = 3	RATE C
OPTIONS(17)	Rating set for the contingency case
OPTIONS(17) = 1	RATE A
OPTIONS(17) = 2	RATE B
OPTIONS(17) = 3	RATE C
OPTIONS(18)	maximum iteration number of PSCOPF
OPTIONS(18) = 1	RATE A
OPTIONS(18) = 2	RATE B
OPTIONS(18) = 3	RATE C
OPTIONS(19)	maximum number of a contingency that does not cause limit violations and instability issue to be tested in the iterative process of PSCOPF
OPTIONS(20)	Handle initial violations in the base case
OPTIONS(20) = 0	Ignore the initial violations in the base case from the PSCOPF analysis
OPTIONS(20) = 1	Clean the initial violations in the base case
OPTIONS(21)	generator control flag (1 by default)
OPTIONS(21) = 0	disable generation MW dispatch
OPTIONS(21) = 1	enable
OPTIONS(22)	load control flag (0 by default)
OPTIONS(22) = 0	disable load shedding control
OPTIONS(22) = 1	enable
OPTIONS(23)	phase shifter control flag (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(23) = 0	disable phase shifter control
	OPTIONS(23) = 1	enable
	OPTIONS(24)	off-line generator control flag (0 by default)
	OPTIONS(24) = 0	disable generation MW dispatch of off-line generators
	OPTIONS(24) = 1	enable
	OPTIONS(25)	tap setting adjustment flag (0 by default)
	OPTIONS(25) = 0	disable tap setting adjustment
	OPTIONS(25) = 1	enable
	OPTIONS(26)	switched shunt control flag (0 by default)
	OPTIONS(26) = 0	disable switch shunt control
	OPTIONS(26) = 1	enable
Real VALUES (11)	array of 11 elements specifying VALUES Is a real(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Percentage of rating for the base case (100.0 by default)
	VALUES(3)	Percentage of rating for the contingency cases (100.0 by default)
	VALUES(4)	Voltage tolerance in linear programming
	VALUES(5)	Flow tolerance in linear programming
	VALUES(6)	Weighting factor of generation MW dispatch (1.0 by default)
	VALUES(7)	Weighting factor of load control (1.0 by default)
	VALUES(8)	Weighting factor of phase shifter control (1.0 by default)
	VALUES(9)	Weighting factor of generation MW dispatch of off-line generators (1.0 by default)
	VALUES(10)	Weighting factor of tap setting adjustment (1.0 by default)
	VALUES(11)	Weighting factor of switched shunt control (1.0 by default)
Character*32 LABELS (7)	array of 7 elements specifying a character(input).	
	LABELS(1)	label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation (blank for none by default)
	LABELS(2)	label of generation dispatch control system. MW injections of the generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(3)	label of load control system. Loads in the load control system are available for curtailment in the corrective action mode or preventive security constrained optimal power flow mode (blank for none by default)
	LABELS(4)	label of phase shifter adjustment system. The phase angles of phase shifters in the system are regulated either by corrective action or preventive security constrained optimal power flow (blank for none by default)
	LABELS(5)	label of generation MW control of off-line generator control system. MW injections of the off-line generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none by default)
	LABELS(6)	label of tap adjustment system. The tap positions of transformers in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none by default)
	LABELS(7)	label of switched shunt control system. The admittances of switched shunts in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none by default)
Character*260 DFXFILE	name of the Distribution Factor Data File (input; no default allowed).	
Character*260 INLFILE	name of Unit Inertia Data File (input; blank for none).	
Character*260 THRFILE	name of Load Throwover Data File (input; blank for none).	
Integer IERR	error code (output).	
	IERR = -2	PSCOPF terminates with active constraints
	IERR = -1	PSCOPF reaches the maximum iteration
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	Generators are converted
	IERR = 4	Buses in island(s) without a swing bus--use activity TREE
	IERR = 5	Largest mismatch exceeds tolerance
	IERR = 6	Generation subsystem not specified
	IERR = 7	Too many islands in base case
	IERR = 8	in-service induction machines are in the "stalled" or "tripped" state

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	No Distribution Factor Data input file specified
IERR = 11	No controls are specified
IERR = 12	No constraints are specified
IERR = 13	Invalid base/contingency case flag combination
IERR = 21	File is not in the form of a PSSE-25 or later DFAX file--run DFAX
IERR = 22	Monitored elements exceed limit when adding multi-section line members
IERR = 51	Error opening distribution factor data file
IERR = 52	Error opening load throwover data file
IERR = 53	Error opening inertia data file
IERR = 54	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.206. PSCOPF_2

This API is the second release of Preventive Security Constrained Optimal Power Flow solution (PSCOPF). This API is used to find the settings of controls for a system to comply with base case and contingency case operation limits. The API is implemented with preventive security constrained optimal power flow method (PSCOPF). The objective of the PSCOPF is to minimize control adjustments subject to operation limits constraints. The available controls are generation MW outputs of on-line and off-line generators, loads, phase shifters, tap settings and switched shunts.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_PSCOPF_2 OPTIONS(1)..OPTIONS(28) VALUES(1)..VALUES(11)
 LABELS(1)..LABELS(7) DFXFILE INLFILE THRFILE
```

Python command syntax:

```
ierr = pscopf_2(options, values, labels, dfxfile, inlfile, thrfile)
```

Fortran command syntax:

```
CALL PSCOPFAPI_2(OPTIONS, VALUES, LABELS, DFXFILE, INLFILE, THRFILE, IERR)
```

Where:

Integer OPTIONS (28)	array of elements specifying power flow options(input).	
	OPTIONS(1)	tap adjustment flag for the base case (tap adjustment option by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag for the base case (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange.
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag for the base case (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag for the base case (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5)	switched shunt adjustment flag for the base case (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable all
OPTIONS(5) = 2	enable continuous, disable discrete.
OPTIONS(6)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage, 0 by default)
OPTIONS(6) = 0	stall.
OPTIONS(6) = 1	trip.
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	Fixed slope decoupled Newton-Raphson method
OPTIONS(7) = 1	Full Newton-Raphson method
OPTIONS(7) = 2	Optimized fixed slope decoupled Newton-Raphson method
OPTIONS(8)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(8) = 0	Disable
OPTIONS(8) = 1	Enable
OPTIONS(9)	tap adjustment flag for the first N-1 contingency (tap adjustment option setting by default)
OPTIONS(9) = 0	disable.
OPTIONS(9) = 1	enable stepping adjustment.
OPTIONS(9) = 2	enable direct adjustment.
OPTIONS(10)	area interchange adjustment flag for the contingency case (area interchange adjustment option setting by default)
OPTIONS(10) = 0	disable.
OPTIONS(10) = 1	enable using tie line flows only in calculating area interchange.
OPTIONS(10) = 2	enable using tie line flows and loads in calculating area interchange.
OPTIONS(11)	phase shift adjustment flag for the contingency case (phase shift adjustment option setting by default)
OPTIONS(11) = 0	disable.
OPTIONS(11) = 1	enable.
OPTIONS(12)	dc tap adjustment flag for the contingency case (dc tap adjustment option setting by default)
OPTIONS(12) = 0	disable.
OPTIONS(12) = 1	enable.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(13)	switched shunt adjustment flag for the contingency case (switched shunt adjustment option setting by default)
OPTIONS(13) = 0	disable.
OPTIONS(13) = 1	enable.
OPTIONS(13) = 2	enable continuous, disable discrete.
OPTIONS(14)	induction motor treatment flag for the contingency case(applied when an induction motor fails to solve due to low terminal bus voltage) (0 by default)
OPTIONS(14) = 0	stall.
OPTIONS(14) = 1	trip.
OPTIONS(15)	dispatch mode (0 by default)
OPTIONS(15) = 0	disable.
OPTIONS(15) = 1	subsystem machines (RESERVE).
OPTIONS(15) = 2	subsystem machines (PMAX).
OPTIONS(15) = 3	subsystem machines (INERTIA).
OPTIONS(15) = 4	subsystem machines (GOVERNOR DROOP).
OPTIONS(16)	Rating set for the base case
OPTIONS(16) = 1	rate A
OPTIONS(16) = 2	rate B
OPTIONS(16) = 3	rate C
OPTIONS(17)	Rating set for the contingency case
OPTIONS(17) = 1	rate A
OPTIONS(17) = 2	rate B
OPTIONS(17) = 3	rate C
OPTIONS(18)	base case voltage limit
OPTIONS(18) = 1	normal
OPTIONS(18) = 2	emergency
OPTIONS(19)	contingency case voltage limit
OPTIONS(19) = 1	normal
OPTIONS(19) = 2	emergency
OPTIONS(20)	maximum iteration number of PSCOPF
OPTIONS(21)	maximum number of a contingency that does not cause limit violations and instability issue to be tested in the iterative process of PSCOPF
OPTIONS(22)	Handle initial violations in the base case
OPTIONS(22) = 0	Ignore the initial violations in the base case from the PSCOPF analysis
OPTIONS(22) = 1	Clean the initial violations in the base case
OPTIONS(23)	generator control flag (1 by default)
OPTIONS(23) = 0	disable generation MW dispatch
OPTIONS(23) = 1	enable
OPTIONS(24)	load control flag (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(24) = 0	disable load shedding control
	OPTIONS(24) = 1	enable
	OPTIONS(25)	phase shifter control flag (1 by default)
	OPTIONS(25) = 0	disable phase shifter control
	OPTIONS(25) = 1	enable
	OPTIONS(26)	off-line generator control flag (0 by default)
	OPTIONS(26) = 0	disable generation MW dispatch of off-line generators
	OPTIONS(26) = 1	enable
	OPTIONS(27)	tap setting adjustment flag (0 by default)
	OPTIONS(27) = 0	disable tap setting adjustment
	OPTIONS(27) = 1	enable
	OPTIONS(28)	switched shunt control flag (0 by default)
	OPTIONS(28) = 0	disable switch shunt control
	OPTIONS(28) = 1	enable
Real VALUES (11)	array of elements specifying power flow options(input).	
	VALUES(1)	Mismatch tolerance (TOLN by default)
	VALUES(2)	Percentage of rating for the base case (100.0 by default)
	VALUES(3)	Percentage of rating for the contingency cases (100.0 by default)
	VALUES(4)	Voltage tolerance in linear programming
	VALUES(5)	Flow tolerance in linear programming
	VALUES(6)	Weighting factor of generation MW dispatch (1.0 by default)
	VALUES(7)	Weighting factor of load control (1.0 by default)
	VALUES(8)	Weighting factor of phase shifter control (1.0 by default)
	VALUES(9)	Weighting factor of generation MW dispatch of off-line generators (1.0 by default)
	VALUES(10)	Weighting factor of tap setting adjustment (1.0 by default)
	VALUES(11)	Weighting factor of switched shunt control (1.0 by default)
Character*32 LABELS (7)	array of elements specifying labels(input).	
	LABELS(1)	The label of generation dispatch system, required if the dispatch mode of OPTIONS(15) is not zero. The dispatch mode and the system provide for a default dispatch method for contingencies causing unbalance between load demand and generation output (blank for none).
	LABELS(2)	The label of generation dispatch control system. MW injections of the generators within

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(3)	The label of load control system. Loads in the load control system are available for curtailment in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(4)	The label of phase shifter adjustment system. The phase angles of phase shifters in the system are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
	LABELS(5)	The label of generation MW control of off-line generator control system. MW injections of the off-line generators within the system are available for dispatch in the corrective action mode or preventive security constrained optimal power flow mode (blank for none).
	LABELS(6)	The label of tap adjustment system. The tap positions of transformers in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
	LABELS(7)	The label of switched shunt control system. The admittances of switched shunts in voltage control mode are regulated either by corrective action or preventive security constrained optimal power flow (blank for none).
Character*260 DFXFILE	Is the name of the Distribution Factor Data File (input; no default allowed).	
Character*260 INLFILE	Is the name of Load Throwover Data File (input, blank for none). (input).	
Character*260 THRFILE	Is the name of Unit Inertia Data File (input, blank for none). (input).	
Integer IERR	error code (output).	
	IERR = -2	PSCOPF terminates with active constraints
	IERR = -1	PSCOPF reaches the maximum iteration
	IERR = 0	No error
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Generation subsystem not specified
IERR = 7	too many islands in base case
IERR = 8	in-service induction machines are in the "stalled" or "tripped" state
IERR = 9	buses with bus type code and series element status inconsistencies
IERR = 10	no Distribution Factor Data File specified
IERR = 11	No controls are specified
IERR = 12	No constraints are specified
IERR = 13	Invalid base/contingency case flag combination.
IERR = 14	Initial violations in base case can not be eliminated.
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening Distribution Factor Data File
IERR = 52	error opening Load Throwover Data File
IERR = 53	error opening Unit Inertia and Governor Data File
IERR = 54	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.207. PSEB

Use this API to convert a PSEB command file into a PSSE response file.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PSEB INPFILE OUTFILE
```

Python command syntax:

```
ierr, respfil = pseb(inpfile, outfile)
```

Fortran command syntax:

```
CALL PSEBAPI(INPFILE, OUTFILE, RESPFIL, IERR)
```

Where:

Character*260 INPFILE	name of the PSEB command file (input; no default allowed).	
Character*260 OUTFILE	name of response file; blank to have PSEB assign the filename (input; blank by default).	
Character*260 RESPFIL	pathname of the response file constructed by PSEB (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error opening INPFILE
	IERR = 2	error opening OUTFILE
	IERR = 3	errors detected while reading the PSEB Command Data input file
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.208. PSSEHALT

This API routine is obsolete. It has been replaced by the API routine pssehalt_2 .

Use this API to end the operation of PSSE, closes all associated files and returns to the calling application.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PSSEHALT DELETE
```

Python command syntax:

```
ierr = pssehalt(delete)
```

Fortran command syntax:

```
CALL HALTAPI(DELETE, IERR)
```

Where:

Integer DELETE	delete working files flag (input).	
	DELETE = 1	delete working files
	DELETE != 1	keep working files
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.209. PSSEHALT_2

Use this API to end the operation of PSSE, closes all associated files and returns to the calling application.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_PSSEHALT_2
```

Python command syntax:

```
ierr = pssehalt_2()
```

Fortran command syntax:

```
CALL HALTAPI_2(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.210. PV_ENGINE

This API routine is obsolete. It has been replaced by the API routine pv_engine_1a .

Use this API routine to run the first release of the PV analysis calculation engine. The same set of automatic adjustments is applied in base case and contingency case transfer increment solutions.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PV_ENGINE  OPTIONS(1)..OPTIONS(16)  VALUES(1)..VALUES(6)  LABELS(1)
 LABELS(2)  DFXFILE  THRFILE  ECDFILE  ACCFILE
```

Python command syntax:

```
ierr = pv_engine(options, values, labels, dfxfile, thrfile, ecdfile,
 accfile)
```

Fortran command syntax:

```
CALL PV_ENGINE(OPTIONS, VALUES, LABELS, DFXFILE, THRFILE, ECDFILE,
 ACCFILE, IERR)
```

Where:

Integer OPTIONS (16)	array of 16 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(6) = 0	disable
OPTIONS(6) = 1	enable
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	FDNS
OPTIONS(7) = 1	FNSL
OPTIONS(7) = 2	optimized FDNS
OPTIONS(8)	var limit code for the contingency case initial power flow solution (0 by default)
OPTIONS(8) = 0	apply immediately
OPTIONS(8) = 1	initially ignore, then apply automatically
OPTIONS(9)	var limit code for the subsequent transfer increment cases (0 by default)
OPTIONS(9) = 0	apply immediately
OPTIONS(9) = 1	initially ignore, then apply automatically
OPTIONS(10)	rating set (rating set program option setting by default)
OPTIONS(10) = 1	RATEA
OPTIONS(10) = 2	RATEB
OPTIONS(10) = 3	RATEC
OPTIONS(11)	study (source) system transfer dispatch method (1 by default)
OPTIONS(11) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(11) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(11) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(11) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(11) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(11) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(11) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
OPTIONS(11) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(12)	opposing (sink) system transfer dispatch method (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(12) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
	OPTIONS(12) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(12) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(12) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(12) = 5	subsystem buses with positive MW machines in proportion to their MW output
	OPTIONS(12) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
	OPTIONS(12) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
	OPTIONS(12) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
	OPTIONS(13)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
	OPTIONS(13) = 0	no limits
	OPTIONS(13) = 1	honor machine active power limits
	OPTIONS(14)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
	OPTIONS(14) = 0	no limits
	OPTIONS(14) = 1	enforce non-negative net active power constant MVA load
	OPTIONS(15)	flag to check for low voltage at any monitored bus (0 by default)
	OPTIONS(15) = 0	disable check
	OPTIONS(15) = 1	enable check
	OPTIONS(16)	flag to check for excessive loading on any monitored branch (0 by default)
	OPTIONS(16) = 0	disable check
	OPTIONS(16) = 1	enable check
Real VALUES (6)	array of 6 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial transfer increment in MW (100.0 by default)
	VALUES(3)	transfer increment tolerance in MW (10.0 by default)
	VALUES(4)	maximum incremental transfer in MW (1000.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(5)	low voltage threshold in pu in the low voltage check (0.8 by default)
	VALUES(6)	percent of rating in the excessive branch loading check (100.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which the load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which the load is to be increased (no default allowed)
Character*260 DFXFILE	name of distribution factor data file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ECDFILE	name of economic dispatch data file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of PV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	initial transfer increment is less than transfer increment tolerance
	IERR = 7	no participating dispatch buses in study system
	IERR = 8	no participating dispatch buses in opposing system
	IERR = 9	subsystem engine error
	IERR = 10	ECDI engine in subsystem aaa
	IERR = 11	base case ECDI solution did not converge
	IERR = 12	no DFAX file specified
	IERR = 13	no PV results file specified
	IERR = 14	no economic dispatch data file specified
	IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 16	bus type code and series element status inconsistencies

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 21	DFXFILE file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX file
IERR = 53	error opening load throwover data file
IERR = 54	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.211. PV_ENGINE_1A

This API routine is obsolete. It has been replaced by the API routine pv_engine_2 .

Use this API routine to run the PV analysis calculation engine. The same set of automatic adjustments and induction machine treatment is applied in base case and contingency case transfer increment solutions.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_PV_ENGINE_1A  OPTIONS(1)..OPTIONS(17)  VALUES(1)..VALUES(6)  LABELS(1)
  LABELS(2)  DFXFILE  THRFILE  ECDFILE  ACCFILE
```

Python command syntax:

```
ierr = pv_engine_1a(options, values, labels, dfxfile, thrfile, ecdfile,
 accfile)
```

Fortran command syntax:

```
CALL PV_ENGINE_1A(OPTIONS, VALUES, LABELS, DFXFILE, THRFILE, ECDFILE,
 ACCFILE, IERR)
```

Where:

Integer OPTIONS (17)	array of 17 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(7) = 0	disable
OPTIONS(7) = 1	enable
OPTIONS(8)	solution method flag (0 by default)
OPTIONS(8) = 0	FDNS
OPTIONS(8) = 1	FNSL
OPTIONS(8) = 2	optimized FDNS
OPTIONS(9)	var limit code for the contingency case initial power flow solution (0 by default)
OPTIONS(9) = 0	apply immediately
OPTIONS(9) = 1	initially ignore, then apply automatically
OPTIONS(10)	var limit code for the subsequent transfer increment cases (0 by default)
OPTIONS(10) = 0	apply immediately
OPTIONS(10) = 1	initially ignore, then apply automatically
OPTIONS(11)	rating set (rating set program option setting by default)
OPTIONS(11) = 1	RATEA
OPTIONS(11) = 2	RATEB
OPTIONS(11) = 3	RATEC
OPTIONS(12)	study (source) system transfer dispatch method (1 by default)
OPTIONS(12) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(12) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(12) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(12) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(12) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(12) = 6	subsystem buses with positive MW machines in proportion to their MBASEs

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(12) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
OPTIONS(12) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(13)	opposing (sink) system transfer dispatch method (1 by default)
OPTIONS(13) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(13) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(13) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(13) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(13) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(13) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(13) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
OPTIONS(13) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(14)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
OPTIONS(14) = 0	no limits
OPTIONS(14) = 1	honor machine active power limits
OPTIONS(15)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
OPTIONS(15) = 0	no limits
OPTIONS(15) = 1	enforce non-negative net active power constant MVA load
OPTIONS(16)	flag to check for low voltage at any monitored bus (0 by default)
OPTIONS(16) = 0	disable check
OPTIONS(16) = 1	enable check
OPTIONS(17)	flag to check for excessive loading on any monitored branch (0 by default)
OPTIONS(17) = 0	disable check
OPTIONS(17) = 1	enable check

Real VALUES (6) array of 6 elements specifying solution values(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial transfer increment in MW (100.0 by default)
	VALUES(3)	transfer increment tolerance in MW (10.0 by default)
	VALUES(4)	maximum incremental transfer in MW (1000.0 by default)
	VALUES(5)	low voltage threshold in pu in the low voltage check (0.8 by default)
	VALUES(6)	percent of rating in the excessive branch loading check (100.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which the load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which the load is to be increased (no default allowed)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ECDFILE	name of cconomic dispatch data file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of PV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	initial transfer increment is less than transfer increment tolerance
	IERR = 7	no participating dispatch buses in study system
	IERR = 8	no participating dispatch buses in opposing system
	IERR = 9	subsystem engine error
	IERR = 10	ECDI engine error in subsystem aaa

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 11	base case ECDI solution did not converge
IERR = 12	no DFAX file specified
IERR = 13	no PV results file specified
IERR = 14	no economic dispatch data file specified
IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
IERR = 16	bus type code and series element status in-consistencies
IERR = 21	file DFAX file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX File
IERR = 53	error opening load throwover data file
IERR = 54	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.212. PV_ENGINE_2

This API routine is obsolete. It has been replaced by the API routine pv_engine_3 .

Use this API to run PV analysis calculation engine. It provides for a different set of automatic adjustments in contingency case solutions than is used in the base case transfer increment solutions.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_PV_ENGINE_2  OPTIONS(1)..OPTIONS(21)  VALUES(1)..VALUES(6)  LABELS(1)
 LABELS(2)  DFXFILE  THRFILE  ECDFILE  ACCFILE
```

Python command syntax:

```
ierr = pv_engine_2(options, values, labels, dfxfile, thrfile, ecdfile,
 accfile)
```

Fortran command syntax:

```
CALL PV_ENGINE_2(OPTIONS, VALUES, LABELS, DFXFILE, THRFILE, ECDFILE,
 ACCFILE, IERR)
```

Where:

Integer OPTIONS (21)	array of 21 elements specifying solution options(input).	
	OPTIONS(1)	base case tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	base case area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	base case phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	base case dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	base case switched shunt adjustment flag (switched shunt adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(6) = 0	disable
OPTIONS(6) = 1	enable
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	FDNS
OPTIONS(7) = 1	FNSL
OPTIONS(7) = 2	optimized FDNS
OPTIONS(8)	var limit code for the contingency case power flow solutions (0 by default)
OPTIONS(8) = 0	apply immediately
OPTIONS(8) = 1	initially ignore, then apply automatically
OPTIONS(9)	var limit code for the base case transfer increment solutions (0 by default)
OPTIONS(9) = 0	apply immediately
OPTIONS(9) = 1	initially ignore, then apply automatically
OPTIONS(10)	rating set (rating set program option setting by default)
OPTIONS(10) = 1	RATEA
OPTIONS(10) = 2	RATEB
OPTIONS(10) = 3	RATEC
OPTIONS(11)	study (source) system transfer dispatch method (1 by default)
OPTIONS(11) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(11) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(11) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(11) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(11) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(11) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(11) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
OPTIONS(11) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(12)	opposing (sink) system transfer dispatch method (1 by default)
OPTIONS(12) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(12) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(12) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(12) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(12) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(12) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(12) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
OPTIONS(12) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(13)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
OPTIONS(13) = 0	no limits
OPTIONS(13) = 1	honor machine active power limits
OPTIONS(14)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
OPTIONS(14) = 0	no limits
OPTIONS(14) = 1	enforce non-negative net active power constant MVA load
OPTIONS(15)	flag to check for low voltage at any monitored bus (0 by default)
OPTIONS(15) = 0	disable check
OPTIONS(15) = 1	enable check
OPTIONS(16)	flag to check for excessive loading on any monitored branch (0 by default)
OPTIONS(16) = 0	disable check
OPTIONS(16) = 1	enable check
OPTIONS(17)	contingency case tap adjustment flag (base case tap adjustment flag, OPTIONS (1) by default)
OPTIONS(17) = 0	disable
OPTIONS(17) = 1	enable stepping adjustment
OPTIONS(17) = 2	enable direct adjustment

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(18)	contingency case area interchange adjustment flag (base case area interchange adjustment flag, OPTIONS (2) by default)
	OPTIONS(18) = 0	disable
	OPTIONS(18) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(18) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(19)	contingency case phase shift adjustment flag (base case phase shift adjustment flag, OPTIONS (3) by default)
	OPTIONS(19) = 0	disable
	OPTIONS(19) = 1	enable
	OPTIONS(20)	contingency case dc tap adjustment flag (base case dc tap adjustment flag, OPTIONS (4) by default)
	OPTIONS(20) = 0	disable
	OPTIONS(20) = 1	enable
	OPTIONS(21)	contingency case switched shunt adjustment flag (base case switched shunt adjustment flag, OPTIONS (5) by default)
	OPTIONS(21) = 0	disable
	OPTIONS(21) = 1	enable
	OPTIONS(21) = 2	enable continuous mode, disable discrete mode
Real VALUES (6)	array of 6 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial transfer increment in MW (100.0 by default)
	VALUES(3)	transfer increment tolerance in MW (10.0 by default)
	VALUES(4)	maximum incremental transfer in MW (1000.0 by default)
	VALUES(5)	low voltage threshold in pu in the low voltage check (0.8 by default)
	VALUES(6)	percent of rating in the excessive branch loading check (100.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which the load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

which the load is to be increased (no default allowed)

Character*260 DFXFILE — name of DFAX file (input; no default allowed).

Character*260 THRFILE — name of load throwover data file; blank for none (input; blank by default).

Character*260 ECDFILE — name of economic dispatch data file; blank for none (input; blank by default).

Character*260 ACCFILE — name of PV results file (input; no default allowed).

Integer IERR — error code (output).

IERR = 0	no error occurred
IERR = 1	invalid VALUES value
IERR = 2	invalid OPTIONS value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	initial transfer increment is less than transfer increment tolerance
IERR = 7	no participating dispatch buses in study system
IERR = 8	no participating dispatch buses in opposing system
IERR = 9	subsystem engine error
IERR = 10	ECDI engine in subsystem aaa
IERR = 11	base case ECDI solution did not converge
IERR = 12	no DFAX file specified
IERR = 14	no economic dispatch data file specified
IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
IERR = 16	bus type code and series element status inconsistencies
IERR = 21	DFXFILE file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX file
IERR = 53	error opening load throwover data file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 54	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.213. PV_ENGINE_3

This API routine is obsolete. It has been replaced by the API routine pv_engine_4 .

Use this API to run the PV analysis calculation engine. It provides for a different set of automatic adjustments in contingency case solutions than is used in the base case transfer increment solutions, and allows a negative minimum incremental transfer to be specified..

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_PV_ENGINE_3  OPTIONS(1)..OPTIONS(21)  VALUES(1)..VALUES(7)  LABELS(1)
 LABELS(2)  DFXFILE  THRFILE  ECDFILE  ACCFILE
```

Python command syntax:

```
ierr = pv_engine_3(options, values, labels, dfxfile, thrfile, ecdfile,
 accfile)
```

Fortran command syntax:

```
CALL PV_ENGINE_3(OPTIONS, VALUES, LABELS, DFXFILE, THRFILE, ECDFILE,
 ACCFILE, IERR)
```

Where:

Integer OPTIONS (21)	array of 21 elements specifying solution options(input).	
	OPTIONS(1)	base case tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	base case area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	base case phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	base case dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5)	base case switched shunt adjustment flag (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(6) = 0	disable
OPTIONS(6) = 1	enable
OPTIONS(7)	solution method flag (0 by default)
OPTIONS(7) = 0	FDNS
OPTIONS(7) = 1	FNSL
OPTIONS(7) = 2	optimized FDNS
OPTIONS(8)	var limit code for the contingency case power flow solutions (0 by default)
OPTIONS(8) = 0	apply immediately
OPTIONS(8) = 1	initially ignore, then apply automatically
OPTIONS(9)	var limit code for the base case transfer increment solutions (0 by default)
OPTIONS(9) = 0	apply immediately
OPTIONS(9) = 1	initially ignore, then apply automatically
OPTIONS(10)	rating set (rating set program option setting by default)
OPTIONS(10) = 1	RATEA
OPTIONS(10) = 2	RATEB
OPTIONS(10) = 3	RATEC
OPTIONS(11)	study (source) system transfer dispatch method (1 by default)
OPTIONS(11) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(11) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(11) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(11) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(11) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(11) = 6	subsystem buses with positive MW machines in proportion to their MBASEs

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(11) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
OPTIONS(11) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(12)	opposing (sink) system transfer dispatch method (1 by default)
OPTIONS(12) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(12) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(12) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(12) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(12) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(12) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(12) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
OPTIONS(12) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(13)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
OPTIONS(13) = 0	no limits
OPTIONS(13) = 1	honor machine active power limits
OPTIONS(14)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
OPTIONS(14) = 0	no limits
OPTIONS(14) = 1	enforce non-negative net active power constant MVA load
OPTIONS(15)	flag to check for low voltage at any monitored bus (0 by default)
OPTIONS(15) = 0	disable check
OPTIONS(15) = 1	enable check
OPTIONS(16)	flag to check for excessive loading on any monitored branch (0 by default)
OPTIONS(16) = 0	disable check
OPTIONS(16) = 1	enable check
OPTIONS(17)	contingency case tap adjustment flag (base case tap adjustment flag, OPTIONS (1) by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(17) = 0	disable
	OPTIONS(17) = 1	enable stepping adjustment
	OPTIONS(17) = 2	enable direct adjustment
	OPTIONS(18)	contingency case area interchange adjustment flag (base case area interchange adjustment flag, OPTIONS (2) by default)
	OPTIONS(18) = 0	disable
	OPTIONS(18) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(18) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(19)	contingency case phase shift adjustment flag (base case phase shift adjustment flag, OPTIONS (3) by default)
	OPTIONS(19) = 0	disable
	OPTIONS(19) = 1	enable
	OPTIONS(20)	contingency case dc tap adjustment flag (base case dc tap adjustment flag, OPTIONS (4) by default)
	OPTIONS(20) = 0	disable
	OPTIONS(20) = 1	enable
	OPTIONS(21)	contingency case switched shunt adjustment flag (base case switched shunt adjustment flag, OPTIONS (5) by default)
	OPTIONS(21) = 0	disable
	OPTIONS(21) = 1	enable
	OPTIONS(21) = 2	enable continuous mode, disable discrete mode
Real VALUES (7)	array of 7 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial transfer increment in MW (100.0 by default)
	VALUES(3)	transfer increment tolerance in MW (10.0 by default)
	VALUES(4)	maximum incremental transfer in MW (1000.0 by default)
	VALUES(5)	low voltage threshold in pu in the low voltage check (0.8 by default)
	VALUES(6)	percent of rating in the excessive branch loading check (100.0 by default)
	VALUES(7)	minimum incremental transfer in MW (< 0.0) (0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which the load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which the load is to be increased (no default allowed)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ECDFILE	name of economic dispatch data file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of PV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	initial transfer increment is less than transfer increment tolerance
	IERR = 7	no participating dispatch buses in study system
	IERR = 8	no participating dispatch buses in opposing system
	IERR = 9	subsystem engine error
	IERR = 10	ECDI engine in subsystem aaa
	IERR = 11	base case ECDI solution did not converge
	IERR = 12	no DFAX File specified
	IERR = 13	no PV results file specified
	IERR = 14	no economic dispatch data file specified
	IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 16	invalid base/contingency case adjustment flag combination
	IERR = 17	bus type code and series element status inconsistencies
	IERR = 21	DFXFILE file is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX file
IERR = 53	error opening load throwover data file
IERR = 54	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.214. PV_ENGINE_4

This API routine is obsolete. It has been replaced by the API routine pv_engine_5 .

Use this API routine to run the PV analysis calculation engine. It provides for a different set of automatic adjustments and induction machine treatment in contingency case solutions than is used in the base case transfer increment solutions, and allows a negative minimum incremental transfer to be specified..

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_PV_ENGINE_4  OPTIONS(1)..OPTIONS(23)  VALUES(1)..VALUES(7)  LABELS(1)
 LABELS(2)  DFXFILE  THRFILE  ECDFILE  ACCFILE
```

Python command syntax:

```
ierr = pv_engine_4(options, values, labels, dfxfile, thrfile, ecdfile,
 accfile)
```

Fortran command syntax:

```
CALL PV_ENGINE_4(OPTIONS, VALUES, LABELS, DFXFILE, THRFILE, ECDFILE,
 ACCFILE, IERR)
```

Where:

Integer OPTIONS (23)	array of 23 elements specifying solution options(input).	
	OPTIONS(1)	base case tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	base case area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	base case phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	base case dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5)	base case switched shunt adjustment flag (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	base case induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(7) = 0	disable
OPTIONS(7) = 1	enable
OPTIONS(8)	solution method flag (0 by default)
OPTIONS(8) = 0	FDNS
OPTIONS(8) = 1	FNSL
OPTIONS(8) = 2	optimized FDNS
OPTIONS(9)	var limit code for the contingency case power flow solutions (0 by default)
OPTIONS(9) = 0	apply immediately
OPTIONS(9) = 1	initially ignore, then apply automatically
OPTIONS(10)	var limit code for the base case transfer increment solutions (0 by default)
OPTIONS(10) = 0	apply immediately
OPTIONS(10) = 1	initially ignore, then apply automatically
OPTIONS(11)	rating set (rating set program option setting by default)
OPTIONS(11) = 1	RATEA
OPTIONS(11) = 2	RATEB
OPTIONS(11) = 3	RATEC
OPTIONS(12)	study (source) system transfer dispatch method (1 by default)
OPTIONS(12) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(12) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(12) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(12) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(12) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(12) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(12) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
OPTIONS(12) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(13)	opposing (sink) system transfer dispatch method (1 by default)
OPTIONS(13) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(13) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(13) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(13) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(13) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(13) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(13) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
OPTIONS(13) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(14)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
OPTIONS(14) = 0	no limits
OPTIONS(14) = 1	honor machine active power limits
OPTIONS(15)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
OPTIONS(15) = 0	no limits
OPTIONS(15) = 1	enforce non-negative net active power constant MVA load
OPTIONS(16)	flag to check for low voltage at any monitored bus (0 by default)
OPTIONS(16) = 0	disable check
OPTIONS(16) = 1	enable check
OPTIONS(17)	flag to check for excessive loading on any monitored branch (0 by default)
OPTIONS(17) = 0	disable check

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(17) = 1	enable check
	OPTIONS(18)	contingency case tap adjustment flag (base case tap adjustment flag, OPTIONS (1) by default)
	OPTIONS(18) = 0	disable
	OPTIONS(18) = 1	enable stepping adjustment
	OPTIONS(18) = 2	enable direct adjustment
	OPTIONS(19)	contingency case area interchange adjustment flag (base case area interchange adjustment flag, OPTIONS (2) by default)
	OPTIONS(19) = 0	disable
	OPTIONS(19) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(19) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(20)	contingency case phase shift adjustment flag (base case phase shift adjustment flag, OPTIONS (3) by default)
	OPTIONS(20) = 0	disable
	OPTIONS(20) = 1	enable
	OPTIONS(21)	contingency case dc tap adjustment flag (base case dc tap adjustment flag, OPTIONS (4) by default)
	OPTIONS(21) = 0	disable
	OPTIONS(21) = 1	enable
	OPTIONS(22)	contingency case switched shunt adjustment flag (base case switched shunt adjustment flag, OPTIONS (5) by default)
	OPTIONS(22) = 0	disable
	OPTIONS(22) = 1	enable
	OPTIONS(22) = 2	enable continuous mode, disable discrete mode
	OPTIONS(23)	contingency case induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
	OPTIONS(23) = 0	stall
	OPTIONS(23) = 1	trip
Real VALUES (7)	array of 7 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial transfer increment in MW (100.0 by default)
	VALUES(3)	transfer increment tolerance in MW (10.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(4)	maximum incremental transfer in MW (1000.0 by default)
	VALUES(5)	low voltage threshold in pu in the low voltage check (0.8 by default)
	VALUES(6)	percent of rating in the excessive branch loading check (100.0 by default)
	VALUES(7)	minimum incremental transfer in MW (< 0.0) (0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which the load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which the load is to be increased (no default allowed)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ECDFILE	name of economic dispatch data file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of PV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	initial transfer increment is less than transfer increment tolerance
	IERR = 7	no participating dispatch buses in study system
	IERR = 8	no participating dispatch buses in opposing system
	IERR = 9	subsystem engine error
	IERR = 10	ECDI engine in subsystem aaa
	IERR = 11	base case ECDI solution did not converge
	IERR = 12	no DFAX file specified
	IERR = 13	no PV results file specified
	IERR = 14	no economic dispatch data file specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
IERR = 16	bus type code and series element status inconsistencies
IERR = 17	invalid base/contingency case flag combination
IERR = 21	DFXFILE file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX file
IERR = 53	error opening load throwover data file
IERR = 54	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.215. PV_ENGINE_5

This API routine is obsolete. It has been replaced by the API routine pv_engine_6 .

Use this API routine to run the PV analysis calculation engine. It provides for a different set of automatic adjustments, and for different treatment of induction motors that fail to solve due to low terminal voltage, in contingency case solutions than is used in the base case transfer increment solutions; it allows a negative minimum incremental transfer to be specified; for transfer increment methods involving load, it allows a power factor to be specified; and it provides for dispatching power unbalances resulting from the application of contingencies.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_PV_ENGINE_5  OPTIONS(1)..OPTIONS(24) VALUES(1)..VALUES(8)
 LABELS(1)..LABELS(3) DFXFILE THRFILE ECDFILE INLFILE ACCFILE
```

Python command syntax:

```
ierr = pv_engine_5(options, values, labels, dfxfile, thrfile, ecdfile,
 inlfile, accfile)
```

Fortran command syntax:

```
CALL PV_ENGINE_5(OPTIONS, VALUES, LABELS, DFXFILE, THRFILE, ECDFILE,
 INLFILE, ACCFILE, IERR)
```

Where:

Integer OPTIONS (24)	array of 24 elements specifying solution options(input).	
	OPTIONS(1)	base case tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	base case area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	base case phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	base case dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(4) = 0	disable
OPTIONS(4) = 1	enable
OPTIONS(5)	base case switched shunt adjustment flag (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	base case induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(7) = 0	disable
OPTIONS(7) = 1	enable
OPTIONS(8)	solution method flag (0 by default)
OPTIONS(8) = 0	FDNS
OPTIONS(8) = 1	FNSL
OPTIONS(8) = 2	optimized FDNS
OPTIONS(9)	var limit code for the contingency case power flow solutions (0 by default)
OPTIONS(9) = 0	apply immediately
OPTIONS(9) = 1	initially ignore, then apply automatically
OPTIONS(10)	var limit code for the base case transfer increment solutions (0 by default)
OPTIONS(10) = 0	apply immediately
OPTIONS(10) = 1	initially ignore, then apply automatically
OPTIONS(11)	rating set (rating set program option setting by default)
OPTIONS(11) = 1	RATEA
OPTIONS(11) = 2	RATEB
OPTIONS(11) = 3	RATEC
OPTIONS(12)	study (source) system transfer dispatch method (1 by default)
OPTIONS(12) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(12) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(12) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(12) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(12) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(12) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(12) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
OPTIONS(12) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(13)	opposing (sink) system transfer dispatch method (1 by default)
OPTIONS(13) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(13) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(13) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(13) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(13) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(13) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(13) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
OPTIONS(13) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(14)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
OPTIONS(14) = 0	no limits
OPTIONS(14) = 1	honor machine active power limits
OPTIONS(15)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
OPTIONS(15) = 0	no limits
OPTIONS(15) = 1	enforce non-negative net active power constant MVA load
OPTIONS(16)	flag to check for low voltage at any monitored bus (0 by default)
OPTIONS(16) = 0	disable check
OPTIONS(16) = 1	enable check
OPTIONS(17)	flag to check for excessive loading on any monitored branch (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(17) = 0	disable check
OPTIONS(17) = 1	enable check
OPTIONS(18)	dispatch mode for power unbalances resulting from the application of contingencies (1 by default)
OPTIONS(18) = 0	disable
OPTIONS(18) = 1	in-service subsystem machines using reserve
OPTIONS(18) = 2	in-service subsystem machines using Pmax
OPTIONS(18) = 3	in-service subsystem machines using inertia
OPTIONS(18) = 4	in-service subsystem machines using governor droop
OPTIONS(19)	contingency case tap adjustment flag (base case tap adjustment flag, OPTIONS (1) by default)
OPTIONS(19) = 0	disable
OPTIONS(19) = 1	enable stepping adjustment
OPTIONS(19) = 2	enable direct adjustment
OPTIONS(20)	contingency case area interchange adjustment flag (base case area interchange adjustment flag, OPTIONS (2) by default)
OPTIONS(20) = 0	disable
OPTIONS(20) = 1	enable using tie line flows only in calculating area interchange
OPTIONS(20) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(21)	contingency case phase shift adjustment flag (base case phase shift adjustment flag, OPTIONS (3) by default)
OPTIONS(21) = 0	disable
OPTIONS(21) = 1	enable
OPTIONS(22)	contingency case dc tap adjustment flag (base case dc tap adjustment flag, OPTIONS (4) by default)
OPTIONS(22) = 0	disable
OPTIONS(22) = 1	enable
OPTIONS(23)	contingency case switched shunt adjustment flag (base case switched shunt adjustment flag, OPTIONS (5) by default)
OPTIONS(23) = 0	disable
OPTIONS(23) = 1	enable
OPTIONS(23) = 2	enable continuous mode, disable discrete mode
OPTIONS(24)	contingency case induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(24) = 0	stall
	OPTIONS(24) = 1	trip
Real VALUES (8)	array of 8 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial transfer increment in MW (100.0 by default)
	VALUES(3)	transfer increment tolerance in MW (10.0 by default)
	VALUES(4)	maximum incremental transfer in MW (1000.0 by default)
	VALUES(5)	low voltage threshold in pu in the low voltage check (0.8 by default)
	VALUES(6)	percent of rating in the excessive branch loading check (100.0 by default)
	VALUES(7)	minimum incremental transfer in MW (< 0.0) (0.0 by default)
	VALUES(8)	power factor for load increases in dispatch methods 2, 3 and 4. Specify as <= 0.0 to retain the original power factor at each load that is changed (0.0 by default)
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which the load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which the load is to be increased (no default allowed)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ECDFILE	name of economic dispatch data file; blank for none (input; blank by default).	
Character*260 INLFILE	name of inertia and governor response file (input).	
Character*260 ACCFILE	name of PV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	initial transfer increment is less than transfer increment tolerance
IERR = 7	no participating dispatch buses in study system
IERR = 8	no participating dispatch buses in opposing system
IERR = 9	subsystem engine error
IERR = 10	ECDI engine in subsystem aaa
IERR = 11	base case ECDI solution did not converge
IERR = 12	no DFAX file specified
IERR = 13	no PV results file specified
IERR = 14	no economic dispatch data file specified
IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
IERR = 16	bus type code and series element status inconsistencies
IERR = 17	invalid base/contingency case flag combination
IERR = 21	DFXFILE file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX file
IERR = 53	error opening load throwover data file
IERR = 54	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.216. PV_ENGINE_6

Use this API routine to run the PV analysis calculation engine. It provides for a different set of automatic adjustments, and for different treatment of induction motors that fail to solve due to low terminal voltage, in contingency case solutions than is used in the base case transfer increment solutions; it allows a negative minimum incremental transfer to be specified; for transfer increment methods involving load, it allows a power factor to be specified; it provides for dispatching power unbalances resulting from the application of contingencies; and it allows for the preservation of milestone results in a ZIP Archive Output File.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_PV_ENGINE_6  OPTIONS(1)..OPTIONS(25) VALUES(1)..VALUES(8)
 LABELS(1)..LABELS(3) DFXFILE THRFILE ECDFILE INLFILE ACCFILE ZIPFILE
```

Python command syntax:

```
ierr = pv_engine_6(options, values, labels, dfxfile, thrfile, ecdfile,
 inlfile, accfile, zipfile)
```

Fortran command syntax:

```
CALL PV_ENGINE_6(OPTIONS, VALUES, LABELS, DFXFILE, THRFILE, ECDFILE,
 INLFILE, ACCFILE, ZIPFILE, IERR)
```

Where:

Integer OPTIONS (25)	array of 25 elements specifying solution options(input).	
	OPTIONS(1)	base case tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	base case area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	base case phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	base case dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(5)	base case switched shunt adjustment flag (switched shunt adjustment option setting by default)
OPTIONS(5) = 0	disable
OPTIONS(5) = 1	enable
OPTIONS(5) = 2	enable continuous mode, disable discrete mode
OPTIONS(6)	base case induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
OPTIONS(6) = 0	stall
OPTIONS(6) = 1	trip
OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTIONS(7) = 0	disable
OPTIONS(7) = 1	enable
OPTIONS(8)	solution method flag (0 by default)
OPTIONS(8) = 0	FDNS
OPTIONS(8) = 1	FNSL
OPTIONS(8) = 2	optimized FDNS
OPTIONS(9)	var limit code for the contingency case power flow solutions (0 by default)
OPTIONS(9) = 0	apply immediately
OPTIONS(9) = 1	initially ignore, then apply automatically
OPTIONS(10)	var limit code for the base case transfer increment solutions (0 by default)
OPTIONS(10) = 0	apply immediately
OPTIONS(10) = 1	initially ignore, then apply automatically
OPTIONS(11)	rating set (rating set program option setting by default)
OPTIONS(11) = 1	RATEA
OPTIONS(11) = 2	RATEB
OPTIONS(11) = 3	RATEC
OPTIONS(12)	study (source) system transfer dispatch method (1 by default)
OPTIONS(12) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(12) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(12) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(12) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(12) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(12) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(12) = 7	subsystem buses with positive MW machines in proportion to their reserve (PMAXMC - PGENMC)
OPTIONS(12) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(13)	opposing (sink) system transfer dispatch method (1 by default)
OPTIONS(13) = 1	buses and distribution factors from DFAX file for buses with positive MW machines
OPTIONS(13) = 2	buses and distribution factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(13) = 3	buses and distribution factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(13) = 4	subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(13) = 5	subsystem buses with positive MW machines in proportion to their MW output
OPTIONS(13) = 6	subsystem buses with positive MW machines in proportion to their MBASEs
OPTIONS(13) = 7	subsystem buses with positive MW machines in proportion to their reserve (PGENMC - PMINMC)
OPTIONS(13) = 8	subsystem buses with positive MW machines via ECDI with unit commitment disabled
OPTIONS(14)	generation plant limits flag for transfer methods 1, 3, 5, 6 and 7 (0 by default)
OPTIONS(14) = 0	no limits
OPTIONS(14) = 1	honor machine active power limits
OPTIONS(15)	positive load flag for transfer methods 2, 3 and 4 (0 by default)
OPTIONS(15) = 0	no limits
OPTIONS(15) = 1	enforce non-negative net active power constant MVA load
OPTIONS(16)	flag to check for low voltage at any monitored bus (0 by default)
OPTIONS(16) = 0	disable check
OPTIONS(16) = 1	enable check
OPTIONS(17)	flag to check for excessive loading on any monitored branch (0 by default)
OPTIONS(17) = 0	disable check

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(17) = 1	enable check
OPTIONS(18)	dispatch mode for power unbalances resulting from the application of contingencies (1 by default)
OPTIONS(18) = 0	disable
OPTIONS(18) = 1	in-service subsystem machines using reserve
OPTIONS(18) = 2	in-service subsystem machines using Pmax
OPTIONS(18) = 3	in-service subsystem machines using inertia
OPTIONS(18) = 4	in-service subsystem machines using governor droop
OPTIONS(19)	write ZIP archive flag (0 by default)
OPTIONS(19) = 0	no ZIP archive
OPTIONS(19) = 1	Write ZIP Archive file ZIPFILE; preserve each system condition at its largest solved incremental transfer level.
OPTIONS(19) = 2	Write ZIP Archive file ZIPFILE; preserve each system condition at all of its solved incremental transfer levels.
OPTIONS(20)	contingency case tap adjustment flag (base case tap adjustment flag, OPTIONS (1) by default)
OPTIONS(20) = 0	disable
OPTIONS(20) = 1	enable stepping adjustment
OPTIONS(20) = 2	enable direct adjustment
OPTIONS(21)	contingency case area interchange adjustment flag (base case area interchange adjustment flag, OPTIONS (2) by default)
OPTIONS(21) = 0	disable
OPTIONS(21) = 1	enable using tie line flows only in calculating area interchange
OPTIONS(21) = 2	enable using tie line flows and loads in calculating area interchange
OPTIONS(22)	contingency case phase shift adjustment flag (base case phase shift adjustment flag, OPTIONS (3) by default)
OPTIONS(22) = 0	disable
OPTIONS(22) = 1	enable
OPTIONS(23)	contingency case dc tap adjustment flag (base case dc tap adjustment flag, OPTIONS (4) by default)
OPTIONS(23) = 0	disable
OPTIONS(23) = 1	enable
OPTIONS(24)	contingency case switched shunt adjustment flag (base case switched shunt adjustment flag, OPTIONS (5) by default)
OPTIONS(24) = 0	disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(24) = 1	enable
	OPTIONS(24) = 2	enable continuous mode, disable discrete mode
	OPTIONS(25)	contingency case induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
	OPTIONS(25) = 0	stall
	OPTIONS(25) = 1	trip
Real VALUES (8)	array of 8 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial transfer increment in MW (100.0 by default)
	VALUES(3)	transfer increment tolerance in MW (10.0 by default)
	VALUES(4)	maximum incremental transfer in MW (1000.0 by default)
	VALUES(5)	low voltage threshold in pu in the low voltage check (0.8 by default)
	VALUES(6)	percent of rating in the excessive branch loading check (100.0 by default)
	VALUES(7)	minimum incremental transfer in MW (< 0.0) (0.0 by default)
	VALUES(8)	power factor for load increases in dispatch methods 2, 3 and 4. Specify as <= 0.0 to retain the original power factor at each load that is changed (0.0 by default)
Character*32 LABELS (3)	array of 3 elements specifying subsystem labels(input).	
	LABELS(1)	label of the study (source) system for which generation is to be increased and/or for which the load is to be decreased (no default allowed)
	LABELS(2)	label of the opposing (sink) system for which generation is to be decreased and/or for which the load is to be increased (no default allowed)
	LABELS(3)	label of the dispatch subsystem; used if OPTIONS(18) is 1 through 4 (blank by default)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ECDFILE	name of economic dispatch data file; blank for none (input; blank by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 INLFILE	name of inertia and governor response file (input).
Character*260 ACCFILE	name of PV results file (input; no default allowed).
Character*260 ZIPFILE	name of ZIP archive output file (input).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid VALUES value
IERR = 2	invalid OPTIONS value
IERR = 3	generators are converted
IERR = 4	buses in island(s) without a swing bus; use activity TREE
IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	initial transfer increment is less than transfer increment tolerance
IERR = 7	no participating dispatch buses in study system
IERR = 8	no participating dispatch buses in opposing system
IERR = 9	subsystem engine error
IERR = 10	ECDI engine in subsystem aaa
IERR = 11	base case ECDI solution did not converge
IERR = 12	no DFAX file specified
IERR = 13	no PV results file specified
IERR = 14	no economic dispatch data file specified
IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
IERR = 16	bus type code and series element status inconsistencies
IERR = 17	invalid base/contingency case flag combination
IERR = 21	DFXFILE file is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX file
IERR = 53	error opening load throwover data file
IERR = 54	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.217. QV_ENGINE

This API routine is obsolete. It has been replaced by the API routine qv_engine_2 .

Use this API to run the QV analysis calculation engine.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_QV_ENGINE  OPTIONS(1)..OPTIONS(10)  VALUES(1)..VALUES(4)  DFXFILE
 THRFILE  ACCFILE
```

Python command syntax:

```
ierr = qv_engine(options, values, dfxfile, thrfile, accfile)
```

Fortran command syntax:

```
CALL QV_ENGINE(OPTIONS, VALUES, DFXFILE, THRFILE, ACCFILE, IERR)
```

Where:

Integer OPTIONS (10)	array of 10 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTIONS(6) = 0	disable
	OPTIONS(6) = 1	enable
	OPTIONS(7)	solution method flag (0 by default)
	OPTIONS(7) = 0	FDNS
	OPTIONS(7) = 1	FNSL
	OPTIONS(7) = 2	optimized FDNS
	OPTIONS(8)	var limit code for the VHI power flow solution (0 by default)
	OPTIONS(8) = 0	apply immediately
	OPTIONS(8) = 1	initially ignore, then apply automatically
	OPTIONS(9)	var limit code for the subsequent voltage decrement cases (0 by default)
	OPTIONS(9) = 0	apply immediately
	OPTIONS(9) = 1	initially ignore, then apply automatically
	OPTIONS(10)	study bus number (no default allowed)
Real VALUES (4)	array of 4 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial (maximum) pu voltage setpoint at the study bus (VHI) (1.1 by default)
	VALUES(3)	minimum pu voltage setpoint at the study bus (VLO) (0.9 by default)
	VALUES(4)	pu voltage setpoint decrement (positive) at the study bus (DLTAV) (0.01 by default)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of QV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	VHI is not greater than VLO
	IERR = 7	study bus not found
	IERR = 8	invalid study bus
	IERR = 9	error when adding a plant to the study bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 10	error when adding a machine to the study bus
IERR = 11	base case VHI solution did not converge
IERR = 12	no Distribution Factor Data File specified
IERR = 13	no Q-V Solution Output File specified
IERR = 14	in-service induction machines are in the "stalled" or "tripped" state
IERR = 15	bus type code and series element status in-consistencies
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening QV Results File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 55	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.218. QV_ENGINE_2

This API routine is obsolete. It has been replaced by the API routine qv_engine_3 .

Use this API routine to run the QV analysis calculation engine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_QV_ENGINE_2  OPTIONS(1)..OPTIONS(11)  VALUES(1)..VALUES(4)  DFXFILE
 THRFILE  ACCFILE
```

Python command syntax:

```
ierr = qv_engine_2(options, values, dfxfile, thrfile, accfile)
```

Fortran command syntax:

```
CALL QV_ENGINE_2(OPTIONS, VALUES, DFXFILE, THRFILE, ACCFILE, IERR)
```

Where:

Integer OPTIONS (11)	array of 11 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
	OPTIONS(6) = 0	stall
	OPTIONS(6) = 1	trip
	OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTIONS(7) = 0	disable
	OPTIONS(7) = 1	enable
	OPTIONS(8)	solution method flag (0 by default)
	OPTIONS(8) = 0	FDNS
	OPTIONS(8) = 1	FNSL
	OPTIONS(8) = 2	optimized FDNS
	OPTIONS(9)	var limit code for the VHI power flow solution (0 by default)
	OPTIONS(9) = 0	apply immediately
	OPTIONS(9) = 1	initially ignore, then apply automatically
	OPTIONS(10)	var limit code for the subsequent voltage decrement cases (0 by default)
	OPTIONS(10) = 0	apply immediately
	OPTIONS(10) = 1	initially ignore, then apply automatically
	OPTIONS(11)	study bus number (no default allowed)
Real VALUES (4)	array of 4 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial (maximum) pu voltage setpoint at the study bus (VHI) (1.1 by default)
	VALUES(3)	minimum pu voltage setpoint at the study bus (VLO) (0.9 by default)
	VALUES(4)	pu voltage setpoint decrement (positive) at the study bus (DLTAV) (0.01 by default)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of QV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	largest mismatch exceeds mismatch tolerance
IERR = 6	VHI is not greater than VLO
IERR = 7	study bus not found
IERR = 8	invalid study bus
IERR = 9	error when adding a plant to the study bus
IERR = 10	error when adding a machine to the study bus
IERR = 11	base case VHI solution did not converge
IERR = 12	no Distribution Factor Data File specified
IERR = 13	no Q-V Solution Output File specified
IERR = 14	in-service induction machines are in the "stalled" or "tripped" state
IERR = 15	bus type code and series element status inconsistencies
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening QV Results File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Load Throwover Data File
IERR = 55	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.219. QV_ENGINE_3

This API routine is obsolete. It has been replaced by the API routine qv_engine_4 .

Use this API routine to run the QV analysis calculation engine.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_QV_ENGINE_3  OPTIONS(1)..OPTIONS(12) VALUES(1)..VALUES(4) LABEL
 DFXFILE THRFILE INLFILE ACCFILE
```

Python command syntax:

```
ierr = qv_engine_3(options, values, label, dfxfile, thrfile, inlfile,
 accfile)
```

Fortran command syntax:

```
CALL QV_ENGINE_3(OPTIONS, VALUES, LABEL, DFXFILE, THRFILE, INLFILE,
 ACCFILE, IERR)
```

Where:

Integer OPTIONS (12)	array of 12 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
	OPTIONS(6) = 0	stall
	OPTIONS(6) = 1	trip
	OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTIONS(7) = 0	disable
	OPTIONS(7) = 1	enable
	OPTIONS(8)	solution method flag (0 by default)
	OPTIONS(8) = 0	FDNS
	OPTIONS(8) = 1	FNSL
	OPTIONS(8) = 2	optimized FDNS
	OPTIONS(9)	var limit code for the VHI power flow solution (0 by default)
	OPTIONS(9) = 0	apply immediately
	OPTIONS(9) = 1	initially ignore, then apply automatically
	OPTIONS(10)	var limit code for the subsequent voltage decrement cases (0 by default)
	OPTIONS(10) = 0	apply immediately
	OPTIONS(10) = 1	initially ignore, then apply automatically
	OPTIONS(11)	study bus number (no default allowed)
	OPTIONS(12)	dispatch mode for power unbalances resulting from the application of contingencies (1 by default)
	OPTIONS(12) = 0	disable
	OPTIONS(12) = 1	in-service subsystem machines using reserve
	OPTIONS(12) = 2	in-service subsystem machines using Pmax
	OPTIONS(12) = 3	in-service subsystem machines using inertia
	OPTIONS(12) = 4	in-service subsystem machines using governor droop
Real VALUES (4)	array of 4 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial (maximum) pu voltage setpoint at the study bus (VHI) (1.1 by default)
	VALUES(3)	minimum pu voltage setpoint at the study bus (VLO) (0.9 by default)
	VALUES(4)	pu voltage setpoint decrement (positive) at the study bus (DLTAV) (0.01 by default)
Character*32 LABEL (1)	array of 1 elements specifying subsystem label(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABEL(1)	label of the dispatch subsystem; used if OPTIONS(12) is 1 through 4 (blank by default)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 INLFILE	name of inertia and governor response file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of QV results file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	VHI is not greater than VLO
	IERR = 7	study bus not found
	IERR = 8	invalid study bus
	IERR = 9	error when adding a plant to the study bus
	IERR = 10	error when adding a machine to the study bus
	IERR = 11	base case VHI solution did not converge
	IERR = 12	no Distribution Factor Data File specified
	IERR = 13	no Q-V Solution Output File specified
	IERR = 14	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 15	bus type code and series element status inconsistencies
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 22	monitored elements exceed limit when adding multi-section line members
	IERR = 51	error opening QV Results File
	IERR = 52	error opening Distribution Factor Data File
	IERR = 53	error opening Load Throwover Data File
	IERR = 55	prerequisite requirements for API are not met
	IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.220. QV_ENGINE_4

Use this API routine to run the QV analysis calculation engine.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_QV_ENGINE_4  OPTIONS(1)..OPTIONS(13) VALUES(1)..VALUES(4) LABEL
 DFXFILE THRFILE INLFILE ACCFILE ZIPFILE
```

Python command syntax:

```
ierr = qv_engine_4(options, values, label, dfxfile, thrfile, inlfile,
 accfile, zipfile)
```

Fortran command syntax:

```
CALL QV_ENGINE_4(OPTIONS, VALUES, LABEL, DFXFILE, THRFILE, INLFILE,
 ACCFILE, ZIPFILE, IERR)
```

Where:

Integer OPTIONS (13)	array of 13 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	induction motor treatment flag; applied when an induction motor fails to solve due to low terminal voltage (0 by default)
	OPTIONS(6) = 0	stall
	OPTIONS(6) = 1	trip
	OPTIONS(7)	non-divergent solution flag (non-divergent solution option setting by default)
	OPTIONS(7) = 0	disable
	OPTIONS(7) = 1	enable
	OPTIONS(8)	solution method flag (0 by default)
	OPTIONS(8) = 0	FDNS
	OPTIONS(8) = 1	FNSL
	OPTIONS(8) = 2	optimized FDNS
	OPTIONS(9)	var limit code for the VHI power flow solution (0 by default)
	OPTIONS(9) = 0	apply immediately
	OPTIONS(9) = 1	initially ignore, then apply automatically
	OPTIONS(10)	var limit code for the subsequent voltage decrement cases (0 by default)
	OPTIONS(10) = 0	apply immediately
	OPTIONS(10) = 1	initially ignore, then apply automatically
	OPTIONS(11)	study bus number (no default allowed)
	OPTIONS(12)	dispatch mode for power unbalances resulting from the application of contingencies (1 by default)
	OPTIONS(12) = 0	disable
	OPTIONS(12) = 1	in-service subsystem machines using reserve
	OPTIONS(12) = 2	in-service subsystem machines using Pmax
	OPTIONS(12) = 3	in-service subsystem machines using inertia
	OPTIONS(12) = 4	in-service subsystem machines using governor droop
	OPTIONS(13)	write ZIP archive flag (0 by default)
	OPTIONS(13) = 0	no ZIP archive
	OPTIONS(13) = 1	Write ZIP Archive file ZIPFILE; preserve each system condition at its lowest solved study bus voltage level.
	OPTIONS(13) = 2	Write ZIP Archive file ZIPFILE; preserve each system condition at all solved study bus voltage levels.
Real VALUES (4)	array of 4 elements specifying solution values(input).	
	VALUES(1)	mismatch tolerance in MW and Mvar (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	initial (maximum) pu voltage setpoint at the study bus (VHI) (1.1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(3)	minimum pu voltage setpoint at the study bus (VLO) (0.9 by default)
	VALUES(4)	pu voltage setpoint decrement (positive) at the study bus (DLTAV) (0.01 by default)
Character*32 LABEL (1)	array of 1 elements specifying subsystem label(input).	
	LABEL(1)	label of the dispatch subsystem; used if OPTIONS(12) is 1 through 4 (blank by default)
Character*260 DFXFILE	name of DFAX file (input; no default allowed).	
Character*260 THRFILE	name of load throwover data file; blank for none (input; blank by default).	
Character*260 INLFILE	name of inertia and governor response file; blank for none (input; blank by default).	
Character*260 ACCFILE	name of QV results file (input; no default allowed).	
Character*260 ZIPFILE	name of ZIP archive output file (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VALUES value
	IERR = 2	invalid OPTIONS value
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 6	initial transfer increment is less than transfer increment tolerance
	IERR = 7	no participating dispatch buses in study system
	IERR = 8	no participating dispatch buses in opposing system
	IERR = 9	subsystem engine error
	IERR = 10	ECDI engine in subsystem aaa
	IERR = 11	base case ECDI solution did not converge
	IERR = 12	no DFAX file specified
	IERR = 13	no PV results file specified
	IERR = 14	no economic dispatch data file specified
	IERR = 15	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 16	bus type code and series element status inconsistencies
	IERR = 17	invalid base/contingency case flag combination
	IERR = 21	DFXFILE file is not in the form of a PSSE-25 or later DFAX file; run DFAX

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 23	no subsystems defined
IERR = 24	subsystem not found
IERR = 25	specified subsystems overlap
IERR = 26	no in-service buses in subsystem WORLD
IERR = 50	error opening economic dispatch data file
IERR = 51	error opening PV results file
IERR = 52	error opening DFAX file
IERR = 53	error opening load throwover data file
IERR = 54	error opening inertia and governor response data file
IERR = 55	error opening ZIP archive output file
IERR = 56	prerequisite requirements for API are not met
IERR = 99	terminated by user interrupt or file error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.221. RANK

Use this API to estimate the severity of designated single branch outage contingencies and builds a contingency solution output file with contingencies specified in decreasing order of their estimated severities.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RANK  SID  ALL  IVAL(1)..IVAL(7) RVAL(1)..RVAL(4)  SSFILE  MNFILE
 ACCFILE
```

Python command syntax:

```
ierr = rank(sid, all, ival, rval, ssfile, mnfile, accfile)
```

Fortran command syntax:

```
CALL RANKNW(SID, ALL, IVAL, RVAL, SSFILE, MNFILE, ACCFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IVAL (7)	array of 7 elements specifying ranking options(input).	
	IVAL(1)	allow ties from specified subsystem as contingencies (0 = no, 1 = yes; 0 by default)
	IVAL(2)	number of branch contingencies from overload ranking to include (>= 0) (0 by default)
	IVAL(3)	rating set (1=RATEA, 2=RATEB, 3=RATEC) (rating set program option setting by default)
	IVAL(4)	convert ratings to estimated MW ratings (0 = no, 1 = yes; 0 by default)
	IVAL(5)	number of voltage depression contingencies to include (>=0) (0 by default)
	IVAL(6)	code indicating the set of branches to include in the voltage depression performance index (0 by default)
	IVAL(6) = 0	include all branches
	IVAL(6) = 1	include only those with a nonzero rating
	IVAL(7)	include contingencies that create swingless islands (0 by default)
	IVAL(7) = 0	no
	IVAL(7) = 1	yes
Real RVAL (4)	array of 4 elements specifying ranking parameters(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RVAL(1)	MW mismatch threshold (>= 0.0) (Newton solution convergence tolerance, TOLN by default)
	RVAL(2)	small reactance threshold (THRSHZ <= RVAL(2) <= 0.005) (0.0005 by default)
	RVAL(3)	percent of rating (0.0 <= RVAL(3) <= 200.0) (100.0 by default)
	RVAL(4)	MW threshold for islanding contingencies (>= 0.0) (100.0 by default)
Character*260 SSFILE	name of the subsystem description file, or blank for no subsystem (input; blank by default).	
Character*260 MNFILE	name of the monitored element description file (if IVAL(2) > 0 (i.e., if the branch outage overload ranker is enabled)) (input; no default allowed).	
Character*260 ACCFILE	name of the contingency solution output file, or blank for output to the progress window (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid IVAL value
	IERR = 4	invalid RVAL value
	IERR = 5	MNFILE is blank
	IERR = 6	no contingency types enabled
	IERR = 7	error opening SSFILE or MNFILE
	IERR = 8	generators are converted
	IERR = 9	buses in island(s) without a swing bus; use activity TREE
	IERR = 10	largest mismatch exceeds mismatch tolerance
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.222. RANK_BRN_AND_MAC

Use this API to estimate the severity of designated single element outage contingencies and builds a contingency description data file with contingencies specified in decreasing order of their estimated severities.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_RANK_BRN_AND_MAC  SID  ALL  IVAL(1)..IVAL(8) RVAL(1)..RVAL(4)  SSFILE
 MNFILE  ACCFILE
```

Python command syntax:

```
ierr = rank_brn_and_mac(sid, all, ival, rval, ssfile, mnfile, accfile)
```

Fortran command syntax:

```
CALL RANKAPI(SID, ALL, IVAL, RVAL, SSFILE, MNFILE, ACCFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IVAL (8)	array of 8 elements specifying ranking options(input).	
	IVAL(1)	allow ties from specified subsystem as contingencies (0 by default)
	IVAL(1) = 0	no
	IVAL(1) = 1	yes
	IVAL(2)	number of branch contingencies from overload ranking to include (>= 0) (0 by default)
	IVAL(3)	rating set (1=RATEA, 2=RATEB, 3=RATEC) (rating set program option setting by default)
	IVAL(4)	convert ratings to estimated MW ratings (0 by default)
	IVAL(4) = 0	no
	IVAL(4) = 1	yes
	IVAL(5)	number of voltage depression contingencies to include (>=0) (0 by default)
	IVAL(6)	code indicating the set of branches to include in the voltage depression performance index (0 by default)
	IVAL(6) = 0	include all branches
	IVAL(6) = 1	include only those with a nonzero rating
	IVAL(7)	include contingencies that create swingless islands (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	IVAL(7) = 0	no
	IVAL(7) = 1	yes
	IVAL(8)	number of machine contingencies from overload ranking to include (>=0) (0 by default)
Real RVAL (4)	array of 4 elements specifying ranking parameters(input).	
	RVAL(1)	MW mismatch threshold (>= 0.0) (Newton solution convergence tolerance, TOLN by default)
	RVAL(2)	small reactance threshold (THRSHZ <= RVAL(2) <= 0.005) (0.0005 by default)
	RVAL(3)	percent of rating (0.0 <= RVAL(3) <= 200.0) (100.0 by default)
	RVAL(4)	MW threshold for islanding contingencies (>= 0.0) (100.0 by default)
Character*260 SSFILE	name of the subsystem description file, or blank for no subsystem (input; blank by default).	
Character*260 MNFILE	name of the monitored element description file (if IVAL(2) > 0 (i.e., if the branch outage overload ranker is enabled) or if IVAL(8) > 0 (i.e., if the machine outage overload ranker is enabled)) (input; no default allowed).	
Character*260 ACCFILE	name of the contingency description data file, or blank for output to the progress window (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid IVAL value
	IERR = 4	invalid RVAL value
	IERR = 5	MNFILE is blank
	IERR = 6	no contingency types enabled
	IERR = 7	error opening SSFILE or MNFILE
	IERR = 8	generators are converted
	IERR = 9	buses in island(s) without a swing bus; use activity TREE
	IERR = 10	largest mismatch exceeds mismatch tolerance
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.223. RATE

This API routine is obsolete. It has been replaced by the API routine rate_2 .

Use this API to check branch loadings.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RATE SID ALL LINFLG RATING FMAX
```

Python command syntax:

```
ierr = rate(sid, all, linflg, rating, fmax)
```

Fortran command syntax:

```
CALL RATENW(SID, ALL, LINFLG, RATING, FMAX, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only branches connected to buses in subsystem SID
	ALL = 1	process all branches
Integer LINFLG	branch check to be applied (input; 1 by default).	
	LINFLG = 1	check all lines
	LINFLG = 2	check non-transformer branches
	LINFLG = 3	check transformers
	LINFLG = 4	check all lines, all ratings
Integer RATING	rating to be used (input; default rating set program option setting).	
	RATING = 1	RATEA
	RATING = 2	RATEB
	RATING = 3	RATEC
Real FMAX	line loading limit, in percent; 0.0 =< FMAX =< 500.0 (input; 100.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid LINFLG value
	IERR = 4	invalid RATING value
	IERR = 5	invalid FMAX value
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.224. RATE_2

This API routine is obsolete. It has been replaced by the API routine rate_3 .

Use this API to check branch loadings.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_RATE_2 SID ALL LINFLG TRNFLG BRKFLG RATING FMAX
```

Python command syntax:

```
ierr = rate_2(sid, all, linflg, trnflg, brkflg, rating, fmax)
```

Fortran command syntax:

```
CALL RATE_2(SID, ALL, LINFLG, TRNFLG, BRKFLG, RATING, FMAX, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only branches connected to buses in subsystem SID
	ALL = 1	process all branches
Integer LINFLG	include non-transformer branches that are not breakers or switches in the branch check to be applied (input; 1 by default).	
	LINFLG = 0	no
	LINFLG = 1	yes
Integer TRNFLG	include transformer branches in the branch check to be applied (input; 1 by default).	
	TRNFLG = 0	no
	TRNFLG = 1	yes
Integer BRKFLG	include branches that are breakers or switches in the branch check to be applied (input; 1 by default).	
	BRKFLG = 0	no
	BRKFLG = 1	yes
Integer RATING	rating set 1 through 12 (input; default rating set program option setting).	
	RATING = 0	all rating sets
Real FMAX	line loading limit, in percent; 0.0 =< FMAX =< 500.0 (input; 100.0).	
Integer IERR	error code (output).	
	IERR = 0	no error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid SID value or subsystem SID is not defined
IERR = 2	invalid ALL value
IERR = 3	invalid LINFLG value
IERR = 4	invalid TRNFLG value
IERR = 5	invalid BRKFLG value
IERR = 6	invalid RATING value
IERR = 7	invalid FMAX value
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.225. RATE_3

Use this API to check branch and system switching device loadings.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_RATE_3 SID ALL LINFLG TRNFLG BRKFLG BRNRATING FMAX SWDRATING
```

Python command syntax:

```
ierr = rate_3(sid, all, linflg, trnflg, brkflg, brnrating, fmax,
 swdrating)
```

Fortran command syntax:

```
CALL RATE_3(SID, ALL, LINFLG, TRNFLG, BRKFLG, BRNRATING, FMAX, SWDRATING,
 IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only branches connected to buses in subsystem SID
	ALL = 1	process all branches
Integer LINFLG	include non-transformer branches that are not breakers or switches in the branch check to be applied (input; 1 by default).	
	LINFLG = 0	no
	LINFLG = 1	yes
Integer TRNFLG	include transformer branches in the branch check to be applied (input; 1 by default).	
	TRNFLG = 0	no
	TRNFLG = 1	yes
Integer BRKFLG	include branches that are breakers or switches in the branch check to be applied (input; 1 by default).	
	BRKFLG = 0	no
	BRKFLG = 1	yes
Integer BRNRATING	branch rating set 1 through 12 (input; default rating set program option setting).	
	BRNRATING = 0	all rating sets
Real FMAX	branch loading limit, in percent; 0.0 =< FMAX =< 500.0 (input; 100.0).	
Integer SWDRATING	system and station switching device rating set 1 through 12 (input; default rating set program option setting).	
	SWDRATING = 0	all rating sets

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid LINFLG value
	IERR = 4	invalid TRNFLG value
	IERR = 5	invalid BRKFLG value
	IERR = 6	invalid RATING value
	IERR = 7	invalid FMAX value
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.226. RATE_INTERFACE

Use this API to check interface loadings.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_RATE_INTERFACE SID ALL RATING FMAX
```

Python command syntax:

```
ierr = rate_interface(sid, all, rating, fmax)
```

Fortran command syntax:

```
CALL RATE_INTERFACE(SID, ALL, RATING, FMAX, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only interfaces which have at least one of interface elements connected to the specified subsystem SID
	ALL = 1	process all interfaces
Integer RATING	rating set 1 through 12 (input; default rating set program option setting).	
	RATING = 0	all rating sets
Real FMAX	line loading limit, in percent; 0.0 =< FMAX =< 500.0 (input; 100.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid RATING value
	IERR = 4	invalid FMAX value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.227. RATING_SET

This API routine is obsolete. It has been replaced by the API routine brn_rating_default .

Use this API to specify or return the current rating set, from 1 to 12.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_RATING_SET IVAL
```

Python command syntax:

```
ierr = rating_set(ival)

ierr, ival = rating_set()
```

Fortran command syntax:

```
CALL RATING_SET_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the current rating set, from 1 to 12 (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
Integer IOCODE	indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL
	IOCODE = 1	return the current value of the option setting in IVAL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.228. RAWD

This API routine is obsolete. It has been replaced by the API routine rawd_2 .

Use this API to replicate the working case in the form of a power flow raw data file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RAWD SID ALL STATUS(1)..STATUS(6) OUT OFILE
```

Python command syntax:

```
ierr = rawd(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL RAWDAPI(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses when STATUS(4) = 0; area ties only when STATUS(4) = 1
Integer STATUS (6)	array of 6 elements specifying options(input).	
	STATUS(1)	include records for Type 4 buses (1 by default)
	STATUS(1) = 0	no
	STATUS(1) = 1	yes
	STATUS(2)	include records for out-of-service branches (1 by default)
	STATUS(2) = 0	no
	STATUS(2) = 1	yes
	STATUS(3)	include records for equipment in the subsystem (1 by default)
	STATUS(3) = 0	no
	STATUS(3) = 1	yes
	STATUS(4)	include records for subsystem ties (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes
	STATUS(5)	use bus names as bus identifiers (0 by default)
	STATUS(5) = 0	no

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 1	yes
	STATUS(6)	Raw Data File type (0 by default)
	STATUS(6) = 0	for activity READ with IC=0
	STATUS(6) = 1	for activity READ with IC=1
	STATUS(6) = 2	for activity RDCH
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid OUT value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.229. RAWDX

This API routine is obsolete. It has been replaced by the API routine writerawxsubsys .

Use this API to replicate the working case in the form of an extended power flow raw data file.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_RAWDX SID ALL STATUS(1)..STATUS(6) OUT OFILE
```

Python command syntax:

```
ierr = rawdx(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL RAWDX(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses when STATUS(4) = 0; area ties only when STATUS(4) = 1
Integer STATUS (6)	array of 6 elements specifying options(input).	
	STATUS(1)	include records for Type 4 buses (1 by default)
	STATUS(1) = 0	no
	STATUS(1) = 1	yes
	STATUS(2)	include records for out-of-service branches (1 by default)
	STATUS(2) = 0	no
	STATUS(2) = 1	yes
	STATUS(3)	include records for equipment in the subsystem (1 by default)
	STATUS(3) = 0	no
	STATUS(3) = 1	yes
	STATUS(4)	include records for subsystem ties (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes
	STATUS(5)	include load record option (honored when ALL=0) (0 by default)
	STATUS(5) = 0	for all loads at subsystem buses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 1	for subsystem loads at all buses
	STATUS(5) = 2	for all loads at subsystem buses and subsystem loads at non-subsystem buses
	STATUS(6)	Raw Data File type (0 by default)
	STATUS(6) = 0	for activity READX with IC=0
	STATUS(6) = 1	for activity READX with IC=1
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 2	rawx write error
	IERR = 3	error opening SFILE
	IERR = 4	prerequisite requirements for API are not met
	IERR = 5	invalid SID value or subsystem SID is not defined
	IERR = 6	invalid ALL value
	IERR = 7	invalid STATUS value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.230. RAWD_2

Use this API to replicate the working case in the form of a power flow raw data file.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_RAWD_2 SID ALL STATUS(1)..STATUS(7) OUT OFILE
```

Python command syntax:

```
ierr = rawd_2(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL RAWDAPI_2(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses when STATUS(4) = 0; only area tie branches when STATUS(4) = 1
Integer STATUS (7)	array of 7 elements specifying options(input).	
	STATUS(1)	include records for Type 4 buses (1 by default)
	STATUS(1) = 0	no
	STATUS(1) = 1	yes
	STATUS(2)	include records for out-of-service branches (1 by default)
	STATUS(2) = 0	no
	STATUS(2) = 1	yes
	STATUS(3)	include records for equipment in the subsystem (1 by default)
	STATUS(3) = 0	no
	STATUS(3) = 1	yes
	STATUS(4)	include records for subsystem tie branches (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes
	STATUS(5)	include load record option (honored when ALL=0) (0 by default)
	STATUS(5) = 0	for all loads at subsystem buses
	STATUS(5) = 1	for subsystem loads at all buses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 2	for all loads at subsystem buses and subsystem loads at non-subsystem buses
	STATUS(6)	use bus names as bus identifiers (0 by default)
	STATUS(6) = 0	no
	STATUS(6) = 1	yes
	STATUS(7)	Raw Data File type (0 by default)
	STATUS(7) = 0	for activity READ with IC=0
	STATUS(7) = 1	for activity READ with IC=1
	STATUS(7) = 2	for activity RDCH; include all data categories
	STATUS(7) = 3	for activity RDCH; exclude the substation data category
	STATUS(7) = 4	for activity RDCH; include only the substation data category
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid OUT value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.231. RAWX_READ_MODE

Use this API to enable/disable saving intermediate raw file when processing RAWx read activity. Reading RAWx file activity creates intermediate temporary raw file. When enabled, intermediate raw file will be saved to working folder. By default, this intermediate raw file is deleted at the end of RAWx read activity.

This API routine was first introduced in release 36.2.1.

Batch command syntax:

```
BAT_RAWX_READ_MODE IVAL
```

Python command syntax:

```
ierr = rawx_read_mode(ival)
```

Fortran command syntax:

```
CALL RAWX_READ_MODE(IVAL, IERR)
```

Where:

Integer IVAL	value of the mode setting (input; 0 by default).	
	IVAL = 0	do not save intermediate raw file
	IVAL = 1	save intermediate raw file
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.232. RAWX_TO_CSV

Use this API to export rawx (extended raw data) table to a CSV (comma-separated values) file.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_RAWX_TO_CSV TBLNAME OFILE
```

Python command syntax:

```
ierr = rawx_to_csv(tblname, ofile)
```

Fortran command syntax:

```
CALL RAWX_TO_CSV(TBLNAME, OFILE, IERR)
```

Where:

Character*100 TBLNAME	name of the data table to export (not case sensitive) (input; no default allowed).	
Character*260 OFILE	file path of the CSV output file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid tblname
	IERR = 2	error opening OFILE
	IERR = 3	error generating RAWX data
	IERR = 4	error reading RAWX data
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.233. RDCH

Use this API to read power flow change data into the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RDCH  NUMNAM  IFILE
```

Python command syntax:

```
ierr = rdch(numnam, ifile)
```

Fortran command syntax:

```
CALL RDCHAPI(NUMNAM, IFILE, IERR)
```

Where:

Integer NUMNAM	flag for bus number or name specification on records (input; 0 by default).	
	NUMNAM = 0	bus numbers
	NUMNAM = 1	bus names
Character*260 IFILE	name of power flow data file, Options (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid NUMNAM
	IERR = 10	error opening IFILE
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.234. RDCHRAWVERSION

Use this API to read a Power Flow Change Raw Data File into the working case. Allowed raw file formats are PSSE-15 through current format.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_RDCHRAWVERSION  NUMNAM VERNUM IFILE
```

Python command syntax:

```
ierr = rdchrawversion(numnam, vernum, ifile)
```

Fortran command syntax:

```
CALL RDCHRAWVERSION(NUMNAM, VERNUM, IFILE, IERR)
```

Where:

Integer NUMNAM	flag for bus number or name specification on records (input; 0 by default).	
	NUMNAM = 0	bus numbers
	NUMNAM = 1	bus names
Character*14 VERNUM	version number corresponding to the format of IFILE VERNUM is in the format of a PSSE release number. Example: If IFILE format is of PSSE-29.5.1: VERNUM = '29' or = '29.5' or = '29.5.1' (input; no default allowed).	
Character*260 IFILE	name of power flow raw data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid NUMNAM value
	IERR = 2	invalid VERNUM value
	IERR = 3	unable to convert file
	IERR = 4	error opening temporary file
	IERR = 5	invalid revision number
	IERR = 10	error opening IFILE
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.235. RDEQ

Use this API to build an electrical equivalent of radial and, optionally, two-point Type 1 buses outside of a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RDEQ  SID  ALL  STATUS(1)..STATUS(8)
```

Python command syntax:

```
ierr = rdeq(sid, all, status)
```

Fortran command syntax:

```
CALL RDEQNW(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses outside of subsystem SID
	ALL = 1	process all buses
Integer STATUS (8)	array of 8 elements specifying used to control equivalencing options. A value of 0 means the option is not enabled. A value of 1 means the option is enabled. They are as follows(input).	
	STATUS(1)	build an electrical equivalent of radial buses only (0 by default)
	STATUS(2)	suppress the equivalencing option for transformers (0 by default)
	STATUS(3)	suppress the equivalencing option for zero impedance lines (0 by default)
	STATUS(4)	suppress the equivalencing option for controlled buses (0 by default)
	STATUS(5)	suppress the equivalencing option for mutually coupled branches (0 by default)
	STATUS(6)	suppress the equivalencing option for area boundary buses (0 by default)
	STATUS(7)	suppress the equivalencing option for zone boundary buses (0 by default)
	STATUS(8)	apply transformer impedance correction to zero sequence network (1 by default)
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ALL value
IERR = 3	invalid STATUS value
IERR = 4	error writing the working case to a temporary file
IERR = 5	load table is full; restoring original working case
IERR = 6	fixed bus shunt table is full; restoring original working case
IERR = 7	error opening temporary file
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.236. READ

Use this API to read a power flow raw data file and add all the data specified in it to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_READ  NUMNAM  IFILE
```

Python command syntax:

```
ierr = read(numnam, ifile)
```

Fortran command syntax:

```
CALL READAPI(NUMNAM, IFILE, IERR)
```

Where:

Integer NUMNAM	flag for bus number or name specification on records (input; 0 by default).	
	NUMNAM = 0	bus numbers
	NUMNAM = 1	bus names
Character*260 IFILE	filename of the power flow raw data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid NUMNAM value
	IERR = 2	invalid revision number
	IERR = 3	unable to convert file
	IERR = 4	error opening temporary file
	IERR = 10	error opening IFILE
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.237. READCAPCURVES

Use this API to read data contained in a Machine Capability Curve Data File into the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_READCAPCURVES FILE
```

Python command syntax:

```
ierr = readcapcurves(file)
```

Fortran command syntax:

```
CALL READCAPCURVES(FILE, IERR)
```

Where:

Character*260 FILE	filename of the Machine Capability Curve Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error opening FILE
	IERR = 2	invalid revision number
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.238. READRAWVERSION

Use this API to read a power flow raw data file and add all the data specified in it to the working case. Allowed raw file formats are PSSE-15 through current format.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_READRAWVERSION  NUMNAM VERNUM IFILE
```

Python command syntax:

```
ierr = readrawversion(numnam, vernum, ifile)
```

Fortran command syntax:

```
CALL READRAWVERSION(NUMNAM, VERNUM, IFILE, IERR)
```

Where:

Integer NUMNAM	flag for bus number or name specification on records (input; 0 by default).	
	NUMNAM = 0	bus numbers
	NUMNAM = 1	bus names
Character*14 VERNUM	version number corresponding to the format of IFILE. VERNUM is in the format of a PSSE release number. Example: If IFILE format is of PSSE-29.5.1: VERNUM = '29' or = '29.5' or = '29.5.1' (input; no default allowed).	
Character*260 IFILE	name of power flow raw data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid NUMNAM value
	IERR = 2	invalid VERNUM value
	IERR = 3	unable to convert file
	IERR = 4	error opening temporary file
	IERR = 5	invalid revision number
	IERR = 10	error opening IFILE
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.239. READRAWX

Use this API to read an extended raw data file and add all the data specified to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_READRAWX SFILE DOMAINS MODE
```

Python command syntax:

```
ierr = readrawx(sfile, domains, mode)
```

Fortran command syntax:

```
CALL READRAWX(SFILE, DOMAINS, MODE, IERR)
```

Where:

Character*260 SFILE	filename of the extended power flow raw data file (input; no default allowed).	
Character*512 DOMAINS	comma separated string of RAWX domains to read (input; all domains).	
	DOMAINS = all	all domains present in file
	DOMAINS = network	network data
	DOMAINS = time-series	time series power flow data
	DOMAINS = harmonics	harmonics data
Character*512 MODE	mode to operate in (input; new).	
	MODE = new	base
	MODE = update	update
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 2	rawx read error
	IERR = 3	error opening SFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.240. READSUB

Use this API to read a power flow raw data file and add subsystem data specified in it to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_READSUB  NUMNAM  IFILE  STATUS(1)  STATUS(2)  STATUS(3)  USEKV
 BASEKV(1)  BASEKV(2)  SIDAREA  SIDOWNER  SIDZONE
```

Python command syntax:

```
ierr = readsub(numnam, ifile, status, usekv, basekv, sidarea, sidowner,
 sidzone)
```

Fortran command syntax:

```
CALL READSUB(NUMNAM, IFILE, STATUS, USEKV, BASEKV, SIDAREA, SIDOWNER,
 SIDZONE, IERR)
```

Where:

Integer NUMNAM	flag for bus number or name specification on records (input; 0 by default).	
	NUMNAM = 0	bus numbers
	NUMNAM = 1	bus names
Character*260 IFILE	name of the power flow raw data file (input; no default allowed).	
Integer STATUS (3)	array of 3 elements specifying options(input).	
	STATUS(1)	append flag (0 by default)
	STATUS(1) = 0	initialize working case before reading data
	STATUS(1) = 1	append data to that already in the working case
	STATUS(2)	boundary bus identification flag (0 by default)
	STATUS(2) = 0	do not change type codes of boundary buses
	STATUS(2) = 1	change type codes of boundary buses in preparation for the equivalencing calculation
	STATUS(3)	data option (1 by default)
	STATUS(3) = 1	add only data from within subsystem
	STATUS(3) = 2	add only ties from subsystem
	STATUS(3) = 3	add subsystem data plus ties
Integer USEKV	subsystem base voltage flag (input; 0 by default).	
	USEKV = 0	do not use base voltage as a subsystem selection criterion
	USEKV = 1	use base voltage as a subsystem selection criterion
Real BASEKV (2)	array of 2 elements specifying the base voltage range(input).	
	BASEKV(1)	minimum base voltage (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BASEKV(2)	maximum base voltage (BASEKV (1) by default)
Integer SIDAREA	area subsystem identifier (0 through 11), or -1 to bypass subsystem specification by area (input; -1).	
Integer SIDOWNER	owner subsystem identifier (0 through 11), or -1 to bypass subsystem specification by owner (input; -1).	
Integer SIDZONE	zone subsystem identifier (0 through 11), or -1 to bypass subsystem specification by zone (input; -1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid NUMNAM value
	IERR = 2	invalid STATUS value
	IERR = 3	invalid USEKV value
	IERR = 4	invalid BASEKV value(s)
	IERR = 5	BASEKV(1) is greater than BASEKV(2)
	IERR = 6	invalid SIDAREA value or area subsystem SIDAREA is not defined
	IERR = 7	invalid SIDOWNER value or owner subsystem SIDOWNER is not defined
	IERR = 8	invalid SIDZONE value or zone subsystem SIDZONE is not defined
	IERR = 9	no subsystem specified
	IERR = 10	error opening IFILE
	IERR = 11	invalid revision number
	IERR = 12	prerequisite requirements for API are not met
	IERR = 14	unable to convert file
	IERR = 15	error opening temporary file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.241. READSUBRAWVERSION

Use this API to read a power flow raw data file and add subsystem data specified in it to the working case. Allowed raw file formats are PSSE-15 through current format.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_READSUBRAWVERSION  NUMNAM  VERNUM  IFILE  STATUS(1)  STATUS(2)
 STATUS(3)  USEKV  BASEKV(1)  BASEKV(2)  SIDAREA  SIDOWNER  SIDZONE
```

Python command syntax:

```
ierr = readsubrawversion(numnam, vernum, ifile, status, usekv, basekv,
 sidarea, sidowner, sidzone)
```

Fortran command syntax:

```
CALL READSUBRAWVERSION(NUMNAM, VERNUM, IFILE, STATUS, USEKV, BASEKV,
 SIDAREA, SIDOWNER, SIDZONE, IERR)
```

Where:

Integer NUMNAM	flag for bus number or name specification on records (input; 0 by default).	
	NUMNAM = 0	bus numbers
	NUMNAM = 1	bus names
Character*14 VERNUM	version number corresponding to the format of IFILE. VERNUM is in the format of a PSSE release number. Example: If IFILE format is of PSSE-29.5.1: VERNUM = '29' or = '29.5' or = '29.5.1' (input; no default allowed).	
Character*260 IFILE	filename of the power flow raw data file (input; no default allowed).	
Integer STATUS (3)	array of 3 elements specifying options(input).	
	STATUS(1)	append flag (0 by default)
	STATUS(1) = 0	initialize working case before reading data
	STATUS(1) = 1	append data to that already in the working case
	STATUS(2)	boundary bus identification flag (0 by default)
	STATUS(2) = 0	do not change type codes of boundary buses
	STATUS(2) = 1	change type codes of boundary buses in preparation for the equivalencing calculation
	STATUS(3)	data option (1 by default)
	STATUS(3) = 1	add only data from within subsystem
	STATUS(3) = 2	add only ties from subsystem
	STATUS(3) = 3	add subsystem data plus ties
Integer USEKV	subsystem base voltage flag (input; 0 by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	USEKV = 0	do not use base voltage as a subsystem selection criterion
	USEKV = 1	use base voltage as a subsystem selection criterion
Real BASEKV (2)	array of 2 elements specifying the base voltage range(input).	
	BASEKV(1)	minimum base voltage (0.0 by default)
	BASEKV(2)	maximum base voltage (BASEKV (1) by default)
Integer SIDAREA	area subsystem identifier (0 through 11), or -1 to bypass subsystem specification by area (input; -1).	
Integer SIDOWNER	owner subsystem identifier (0 through 11), or -1 to bypass subsystem specification by owner (input; -1).	
Integer SIDZONE	zone subsystem identifier (0 through 11), or -1 to bypass subsystem specification by zone (input; -1).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid NUMNAM value
	IERR = 2	invalid STATUS value
	IERR = 3	invalid USEKV value
	IERR = 4	invalid BASEKV value(s)
	IERR = 5	BASEKV(1) is greater than BASEKV(2)
	IERR = 6	invalid SIDAREA value or area subsystem SIDAREA is not defined
	IERR = 7	invalid SIDOWNER value or owner subsystem SIDOWNER is not defined
	IERR = 8	invalid SIDZONE value or zone subsystem SIDZONE is not defined
	IERR = 9	no subsystem specified
	IERR = 10	error opening IFILE
	IERR = 11	invalid revision number
	IERR = 12	prerequisite requirements for API are not met
	IERR = 13	invalid VERNUM value
	IERR = 14	unable to convert file
	IERR = 15	error opening temporary file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.242. READX

This API routine is obsolete. It has been replaced by the API routine readrawx .

Use this API to read an extended power flow raw data file and add all the data specified in it to the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_READX SFILE
```

Python command syntax:

```
ierr = readx(sfile)
```

Fortran command syntax:

```
CALL READX(SFILE, IERR)
```

Where:

Character*260 SFILE	filename of the extended power flow raw data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid revision number
	IERR = 2	unable to convert file
	IERR = 3	error opening SFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.243. RECN

Use this API to electrically reconnect a bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RECN  IBUS
```

Python command syntax:

```
ierr = recn(ibus)
```

Fortran command syntax:

```
CALL RECNAPI(IBUS, IERR)
```

Where:

Integer IBUS	bus number of bus to be reconnected (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.244. REGB

Use this API to tabulate those buses where voltages are controlled by generation, switched shunts, and/or other voltage controlling equipment.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_REGB SID ALL
```

Python command syntax:

```
ierr = regb(sid, all)
```

Fortran command syntax:

```
CALL REGBNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.245. RELIND

This API routine is obsolete. It has been replaced by the API routine relind_2 .

Use this API to run probabilistic reliability assessment for transmission systems. Probabilistic indices on system problems are calculated in entire system or a given subsystem, these problems include flow overloading, voltage violation and loss of load, etc.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_RELIND  OPTIONS(1)..OPTIONS(7)  VALUES(1)..VALUES(3)  LABEL   ACCFILE
 DFXFILE  PRBFILE  STSFILE
```

Python command syntax:

```
ierr = relind(options, values, label, accfile, dfxfile, prbfile, stsfile)
```

Fortran command syntax:

```
CALL RELINDAPI(OPTIONS, VALUES, LABEL, ACCFILE, DFXFILE, PRBFILE, STSFILE,
 IERR)
```

Where:

Integer OPTIONS (7)	array of 7 elements specifying options(input).	
	OPTIONS(1)	report format (1 by default)
	OPTIONS(1) = 1	system problem summary report
	OPTIONS(1) = 2	system loss of load report
	OPTIONS(1) = 3	bus loss of load report
	OPTIONS(1) = 4	branch flow overloading report
	OPTIONS(1) = 5	bus voltage violation report
	OPTIONS(1) = 6	contingency summary
	OPTIONS(1) = 7	system problem indices
	OPTIONS(1) = 8	system loss of load indices
	OPTIONS(1) = 9	bus loss of load indices
	OPTIONS(1) = 10	branch overload problem indices
	OPTIONS(1) = 11	bus voltage problem indices
	OPTIONS(1) = 12	contingency summary with outage statistics
	OPTIONS(2)	mode (1 by default)
	OPTIONS(2) = 1	post contingency mode
	OPTIONS(2) = 2	post tripping mode
	OPTIONS(2) = 3	post corrective action mode
	OPTIONS(3)	rating set (rating set program option setting by default)
	OPTIONS(3) = 1	RATEA
	OPTIONS(3) = 2	RATEB
	OPTIONS(3) = 3	RATEC

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(4)	include ties from study subsystem (0 by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	include interfaces (0 by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(6)	Outage Statistics Data File (0 by default)
	OPTIONS(6) = 0	disable
	OPTIONS(6) = 1	enable
	OPTIONS(7)	normalize probability (0 by default)
	OPTIONS(7) = 0	disable
	OPTIONS(7) = 1	enable
Real VALUES (3)	array of 3 elements specifying reliability assessment parameters(input).	
	VALUES(1)	percent of rating set (>=0 and <=200.0, 100.0 by default)
	VALUES(2)	probability cutoff (>=0; 0.0001 by default)
	VALUES(3)	frequency cutoff (>=0; 0.0001 by default)
Character*32 LABEL	name of the study subsystem (input; blank by default, probabilistic reliability assessment is carried out in entire system).	
Character*260 ACCFILE	contingency solution output file (input; no default allowed).	
Character*260 DFXFILE	DFAX data file (input; no default allowed).	
Character*260 PRBFILE	outage statistics data file (input; blank for none if OPTIONS(1) is between 1 and 6, inclusive; no default allowed if OPTIONS(1) is greater).	
Character*260 STSFILE	output outage statistics data file (input; blank for none if OPTIONS(6) is zero or OPTIONS(1) is from 1 to 6; otherwise no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPTIONS value
	IERR = 2	no dispatch subsystem specified
	IERR = 3	no Contingency Solution Output File specified
	IERR = 4	no Distribution Factor Data File specified
	IERR = 5	no Outage Statistic Data File specified
	IERR = 6	no Output Outage Statistic Data File specified
	IERR = 7	generators are converted
	IERR = 8	buses in island(s) without a swing bus--use activity TREE
	IERR = 9	subsystem not found in Distribution Factor Data File
	IERR = 10	Contingency Solution Output File is not created by Multiple Level Contingency Analysis of PSSE 31 or later

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 11	case data is not the one used in contingency analysis
IERR = 12	Distribution Factor Data File is not the one used in contingency analysis
IERR = 13	error reading Contingency Solution Output file
IERR = 51	error opening Contingency Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 53	error opening Outage Statistic Data File
IERR = 54	error opening Output Outage Statistic Data File
IERR = 55	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.246. RELIND_2

Use this API to run probabilistic reliability assessment for transmission systems. Probabilistic indices on system problems are calculated in entire system or a given subsystem, these problems include flow overloading, voltage violation and loss of load, etc.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_RELIND_2  OPTREL(1)..OPTREL(8) RELREL(1)..RELREL(3) CHAARG ACCFILE
 DFXFILE PRBFILE STSFILE
```

Python command syntax:

```
ierr = relind_2(optrel, relrel, chaarg, accfile, dfxfile, prbfile,
 stsfile)
```

Fortran command syntax:

```
CALL RELINDAPI_2(OPTREL, RELREL, CHAARG, ACCFILE, DFXFILE, PRBFILE,
 STSFILE, IERR)
```

Where:

Integer OPTREL (8)	array of elements specifying output options(input).	
	OPTREL(1)	report format (1 by default)
	OPTREL(1) = 1	system problem summary report
	OPTREL(1) = 2	system loss of load report
	OPTREL(1) = 3	bus loss of load report
	OPTREL(1) = 4	branch flow overloading report
	OPTREL(1) = 5	bus voltage violation report
	OPTREL(1) = 6	contingency summary
	OPTREL(1) = 7	system problem indices
	OPTREL(1) = 8	system loss of load indices
	OPTREL(1) = 9	bus loss of load indices
	OPTREL(1) = 10	branch overload problem indices
	OPTREL(1) = 11	bus voltage problem indices
	OPTREL(1) = 12	contingency summary with outage statistics
	OPTREL(2)	output mode (1 by default)
	OPTREL(2) = 1	post contingency mode
	OPTREL(2) = 2	post tripping mode
	OPTREL(2) = 3	post corrective action mode
	OPTREL(3)	rating set (rating set program option setting by default)
	OPTREL(3) = 1	RATEA
	OPTREL(3) = 2	RATEB
	OPTREL(3) = 3	RATEC
	OPTREL(4)	voltage limit.
	OPTREL(4) = 1	normal

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTREL(4) = 2	emergency
	OPTREL(5)	include ties from study subsystem (0 by default)
	OPTREL(5) = 0	disable
	OPTREL(5) = 1	enable
	OPTREL(6)	include interfaces (0 by default)
	OPTREL(6) = 0	disable
	OPTREL(6) = 1	enable
	OPTREL(7)	output Outage Statistics Data File (0 by default)
	OPTREL(7) = 0	disable
	OPTREL(7) = 1	enable
	OPTREL(8)	normalize probability (0 by default)
	OPTREL(8) = 0	disable
	OPTREL(8) = 1	enable
Real RELREL (3)	array of elements specifying reliability assessment parameters(input).	
	RELREL(1)	percent of rating set, (>=0 and <=200.0, 100.0 by default)
	RELREL(2)	probability cutoff (>=0; 0.0001 by default).
	RELREL(3)	frequency cutoff (>=0; 0.0001 by default).
Character*32 CHAARG	name of the study subsystem (blank by default, probabilistic reliability assessment is carried out in entire system). (input).	
Character*260 ACCFILE	contingency solution output file (input; no default allowed).	
Character*260 DFXFILE	DFAX data file (input; no default allowed).	
Character*260 PRBFILE	outage statistics data file (blank for none if OPTIONS(1) is between 1 and 6, inclusive; no default allowed if OPTIONS(1) is greater). (input).	
Character*260 STSFILE	output outage statistics data file (blank for none if OPTIONS(6) is zero or OPTIONS(1) is from 1 to 6; otherwise no default allowed). (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	no dispatch subsystem specified
	IERR = 3	no contingency solution output file specified
	IERR = 4	no DFAX data file specified
	IERR = 5	no outage statistic data file specified
	IERR = 6	no output outage statistic data file specified
	IERR = 7	generators are converted
	IERR = 8	buses in island(s) without a swing bus; use activity TREE
	IERR = 9	subsystem not found in DFAX data file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 10	contingency solution output file is not created by Multiple Level Contingency Analysis of PSSE 31 or later
IERR = 11	case data is not the one used in contingency analysis
IERR = 12	DFAX data file is not the one used in contingency analysis
IERR = 13	error reading contingency solution output file
IERR = 51	error opening contingency solution output file
IERR = 52	error opening DFAX data file
IERR = 53	error opening outage statistic data file
IERR = 54	error opening output outage statistic data file
IERR = 55	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.247. REMM

Use this API to read transaction event data from a transactions raw data file and add it to PSSE working memory.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_REMM  OPT  IFILE
```

Python command syntax:

```
ierr = remm(opt, ifile)
```

Fortran command syntax:

```
CALL REMMAPI(OPT, IFILE, IERR)
```

Where:

Integer OPT	flag for new or change data records (input; 0 by default).	
	OPT = 0	new data; replaces existing transactions data.
	OPT = 1	change data; modifies existing transactions data.
Character*260 IFILE	name of transactions raw data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPT value
	IERR = 2	no transactions defined; change mode invalid
	IERR = 3	error opening IFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.248. REPORT_OUTPUT

Use this API to specify the report output device.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_REPORT_OUTPUT ISLCT FILARG OPTIONS(1) OPTIONS(2)
```

Python command syntax:

```
ierr = report_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL OPENREPORTDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input; 1 by default).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for reports is the Report tab of the output bar.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input; blank by default).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options (0 by default)
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only) (0 by default)
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error occurred
IERR = 1	prerequisite requirements for API are not met
IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.249. RESQ

Use this API to read sequence data from a sequence data file and add it to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RESQ  IFILE
```

Python command syntax:

```
ierr = resq(ifile)
```

Fortran command syntax:

```
CALL RESQAPI(IFILE, IERR)
```

Where:

Character*260 IFILE	name of the power flow sequence data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error opening IFILE
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.250. RESQVERSION

Use this API to read a network sequence data file and add all the data specified in it to the working case. Allowed seq file formats are PSSE-15 through the current format.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_RESQVERSION  VERNUM  IFILE
```

Python command syntax:

```
ierr = resqversion(vernum, ifile)
```

Fortran command syntax:

```
CALL RESQVERSION(VERNUM, IFILE, IERR)
```

Where:

Character*14 VERNUM	version number corresponding to the format of the IFILE. VERNUM is in the format of a PSSE release number. Example: If IFILE format is of PSSE-29.5.1: VERNUM = '29' or = '29.5' or = '29.5.1' (input; no default allowed).	
Character*260 IFILE	name of the power flow sequence data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 2	invalid VERNUM value
	IERR = 3	unable to convert file
	IERR = 4	error opening temporary file
	IERR = 5	invalid revision number
	IERR = 10	error opening IFILE
	IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.251. RESULT_TABLE_OUTPUT

Use this API to get and set the default tabular output.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_RESULT_TABLE_OUTPUT IVAL
```

Python command syntax:

```
ierr = result_table_output(ival)

ierr, ival = result_table_output()
```

Fortran command syntax:

```
CALL RESULT_TABLE_OUTPUT(IVAL, ICODE, IERR)
```

Where:

Integer IVAL	output option setting. If no interactive spreadsheet device is available, report device text output will be used instead. (input; no default allowed).	
	IVAL = 0	no output
	IVAL = 1	interactive spreadsheet device output
	IVAL = 2	report device text output
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in IVAL
	ICODE = 1	return the current value of the option setting in IVAL
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.252. RETI

Use this API to read the long title from an input file into the working case.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_RETI  IFILE
```

Python command syntax:

```
ierr = reti(ifile)
```

Fortran command syntax:

```
CALL RETIAPI(IFILE, IERR)
```

Where:

Character*260 IFILE	name of long title data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error opening IFILE
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.253. RETRY_PSSUSERPF

Use this API to allow the loading or use of the Powerflow Customization Interface (PCI) implementation module (pssuserpf) after a failure. For efficiency reasons loading the PCI module is only attempted once. You may reset the flag controlling this with this API.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_RETRY_PSSUSERPF
```

Python command syntax:

```
ierr = retry_pssuserpf()
```

Fortran command syntax:

```
CALL RETRYPSSUSERPF_FC(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 1	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.254. REV29_NAMES

Use this API to set the extended bus name input format to either the PSSE-29 or PSSE-30 format.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_REV29_NAMES IVAL
```

Python command syntax:

```
ierr = rev29_names(ival)
```

Fortran command syntax:

```
CALL REV29_NAMES_API(IVAL, IERR)
```

Where:

Integer IVAL	extended bus name option flag (input; 0 by default).	
	IVAL = 0	PSSE-30 format (12 characters plus base voltage)
	IVAL = 1	PSSE-29 or earlier format (8 characters plus base voltage)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.255. RNFI

Use this API to reproduce the results of a working case bus renumbering operation in auxiliary data input files.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_RNFI  FTYPE  TRNFILE  INPFILE  OUTFILE
```

Python command syntax:

```
ierr = rnfi(ftype, trnfile, inpfile, outfile)
```

Fortran command syntax:

```
CALL RNFIAPI(FTYPE, TRNFILE, INPFILE, OUTFILE, IERR)
```

Where:

Integer FTYPE	type of data file to be processed (input; 1 by default).	
	FTYPE = 1	sequence data file
	FTYPE = 2	drawing coordinate data file
	FTYPE = 3	dynamics data file
	FTYPE = 4	machine impedance data file
	FTYPE = 5	economic dispatch data file
	FTYPE = 6	inertia and governor response data file
	FTYPE = 7	breaker duty data file
	FTYPE = 8	fault dspecification data file
	FTYPE = 9	fault control data file
	FTYPE = 10	machine capability Curve data file
	FTYPE = 11	load throwover data file
	FTYPE = 12	ANSI fault specification data file
	FTYPE = 13	subsystem description data file
	FTYPE = 14	monitored element data file
	FTYPE = 15	contingency description data file
	FTYPE = 16	tripping element data file
Character*260 TRNFILE	name of bus renumbering translation file (input; no default allowed).	
Character*260 INPFILE	name of data file to be renumbered (input; no default allowed).	
Character*260 OUTFILE	name of data file (blank for output to the report window) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid FTYPE value
	IERR = 2	error opening translation file
	IERR = 3	error opening input data file
	IERR = 4	error opening output data file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.256. RSOL

Use this API to perform a robust power flow solution (activity RSOL).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_RSOL OPTIONS(1)...OPTIONS(10) REALAR(1)...REALAR(2)
```

Python command syntax:

```
ierr = rsol(options, realar)
```

Fortran command syntax:

```
CALL RSOLAPI(OPTIONS, REALAR, IERR)
```

Where:

Integer OPTIONS (10)	array of 10 elements specifying (input).	
	OPTIONS(1)	Initial solution method (0 by default)
	OPTIONS(1) = 0	Fixed Slope Decoupled Newton-Raphson (FDNS)
	OPTIONS(1) = 1	Full Newton-Raphson (FNSL)
	OPTIONS(1) = 2	Decoupled Newton-Raphson (NSOL)
	OPTIONS(2)	Initial tap adjustment flag (0 by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable stepping adjustment
	OPTIONS(2) = 2	enable direct adjustment
	OPTIONS(3)	Initial area interchange adjustment flag (0 by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(3) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(4)	Initial phase shift adjustment flag (0 by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	Initial dc tap adjustment flag (0 by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(6)	Initial switch shunt adjustment flag (0 by default)
	OPTIONS(6) = 0	disable
	OPTIONS(6) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6) = 2	enable continuous mode, disable discrete mode
	OPTIONS(7)	Initial flat start flag (0 by default)
	OPTIONS(7) = 0	do not flat start
	OPTIONS(7) = 1	flat start
	OPTIONS(7) = 2	flat start, then estimate voltage magnitudes
	OPTIONS(7) = 3	flat start, then estimate voltage phase angles
	OPTIONS(7) = 4	flat start, then estimate voltage magnitudes and phase angles
	OPTIONS(8)	Initial var limit flag (0 by default)
	OPTIONS(8) = 0	apply var limits immediately
	OPTIONS(8) > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	OPTIONS(8) = -1	ignore var limits
	OPTIONS(9)	Initial non-divergent flag (0 by default)
	OPTIONS(9) = 0	disable
	OPTIONS(9) = 1	enable
	OPTIONS(10)	Solution failure option (0 by default)
	OPTIONS(10) = 0	do not restore solution or settings on solution failure
	OPTIONS(10) = 1	restore original solution and settings on solution failure
	OPTIONS(10) = 2	restore best found (lowest mismatch) solution and settings on solution failure
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	Bus mismatch threshold for smoothing (500.0 by default)
	REALAR(2)	Var band (5.0 by default)
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 1	Prerequisites for api not met
	IERR = 2	Invalid OPTIONS value
	IERR = 3	Invalid realar value
	IERR = 4	Solution error occured
	IERR = 5	Error getting mismatch
	IERR = 6	Error creating temporary case files
	IERR = 7	Error reading from temporary case files
	IERR = 8	Error writing to temporary case files
	IERR = 9	Error deleting temporary case files
	IERR = 10	Error creating script file
	IERR = 11	Error writing to script file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.257. RUNLASTSOLUTION

Use this API to run the last known power flow solution. A power flow solution is 'know' if the solution method and parameters are contained within the SAV or RAW file. It is useful for solving a case from an outside source for which there is no documentation on solution methods.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_RUNLASTSOLUTION
```

Python command syntax:

```
ierr = runlastsolution()
```

Fortran command syntax:

```
CALL RUNLASTSOLUTION(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.258. RWCM

Use this API to replicate the working case in IEEE common tape format.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RWCM OPT OUT OFILE
```

Python command syntax:

```
ierr = rwcm(opt, out, ofile)
```

Fortran command syntax:

```
CALL RWCMAPI(OPT, OUT, OFILE, IERR)
```

Where:

Integer OPT	dc line/FACTS/GNE device option (input; 0 by default).	
	OPT = 0	ignore dc lines, FACTS devices, and GNE devices
	OPT = 1	add powers to loads
Integer OUT	selection option (input; 1 by default).	
	OUT = 0	direct output to the data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the output file (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPT value
	IERR = 2	invalid OUT value
	IERR = 3	buses in case with 5 or 6 digit bus numbers
	IERR = 4	n buses in case but IEEE common format can handle only 9999
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.259. RWMA

Use this API to replicate machine parametric data from the working case in the form of a machine impedance data file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RWMA SID ALL STATUS(1) STATUS(2) STATUS(3) OUT OFILE
```

Python command syntax:

```
ierr = rwma(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL RWMANW(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (3)	array of 3 elements specifying control options(input).	
	STATUS(1)	include records for out-of-service machines (0 by default)
	STATUS(1) = 0	exclude them
	STATUS(1) = 1	include them
	STATUS(2)	used when calculating P fractions (0 by default)
	STATUS(2) = 0	use PGEN
	STATUS(2) = 1	use MBASE
	STATUS(2) = 2	use PMAX
	STATUS(3)	used when calculating Q fractions (0 by default)
	STATUS(3) = 0	use QGEN
	STATUS(3) = 1	use MBASE
	STATUS(3) = 2	use QMAX
Integer OUT	option selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE.
	OUT = 1	direct output to the current report device.
Character*260 OFILE	output filename (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid SID value or subsystem SID is not defined
IERR = 2	invalid ALL value
IERR = 3	invalid STATUS value
IERR = 4	invalid OUT value
IERR = 5	error opening OFILE
IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.260. RWMM

Use this API to replicate transaction event data in the form of a transactions data file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RWMM OUT OFILE
```

Python command syntax:

```
ierr = rwmm(out, ofile)
```

Fortran command syntax:

```
CALL RWMMAPI(OUT, OFILE, IERR)
```

Where:

Integer OUT	selection option (input; 1 by default).	
	OUT = 0	direct output to the data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OUT value
	IERR = 2	error opening OFILE
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.261. RWSQ

This API routine is obsolete. It has been replaced by the API routine rwsq_2 .

Use this API to replicate the sequence data contained in the working case in the form of a sequence data file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RWSQ SID ALL STATUS(1)..STATUS(4) OUT OFILE
```

Python command syntax:

```
ierr = rwsq(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL RWSQNW(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses when STATUS(4) is 0; area ties only when STATUS(4) is 1
Integer STATUS (4)	array of 4 elements specifying options(input).	
	STATUS(1)	include records for Type 4 buses (1 by default)
	STATUS(1) = 0	no
	STATUS(1) = 1	yes
	STATUS(2)	include records for out-of-service branches (1 by default)
	STATUS(2) = 0	no
	STATUS(2) = 1	yes
	STATUS(3)	include records for equipment in the subsystem (1 by default)
	STATUS(3) = 0	no
	STATUS(3) = 1	yes
	STATUS(4)	include records for subsystem ties (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes
Integer OUT	option selection (input; 1 by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid OUT value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.262. RWSQ_2

Use this API to replicate the sequence data contained in the working case in the form of a sequence data file.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_RWSQ_2 SID ALL STATUS(1)..STATUS(5) OUT OFILE
```

Python command syntax:

```
ierr = rwsq_2(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL RWSQNW_2(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses when STATUS(4) is 0; area ties only when STATUS(4) is 1
Integer STATUS (5)	array of 5 elements specifying options(input).	
	STATUS(1)	include records for Type 4 buses (1 by default)
	STATUS(1) = 0	no
	STATUS(1) = 1	yes
	STATUS(2)	include records for out-of-service branches (1 by default)
	STATUS(2) = 0	no
	STATUS(2) = 1	yes
	STATUS(3)	include records for equipment in the subsystem (1 by default)
	STATUS(3) = 0	no
	STATUS(3) = 1	yes
	STATUS(4)	include records for subsystem ties (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes
	STATUS(5)	sequence data file type (0 by default)
	STATUS(5) = 0	for activity RESQ with IC=0
	STATUS(5) = 1	for activity RESQ with IC=1
Integer OUT	option selection (input; 1 by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid OUT value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.263. SAVE

Use this API to save the PSSE working case in a saved case file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SAVE SFILE
```

Python command syntax:

```
ierr = save(sfile)
```

Fortran command syntax:

```
CALL SAVENW(SFILE, IERR)
```

Where:

Character*260 SFILE	name of the PSSE saved case file If SFILE is "*", SAVENW attempts to use the most recently accessed Saved Case File. If no Saved Case File was accessed in the current execution of PSSE, SAVENW returns an error code (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SFILE is blank
	IERR = 2	error writing to SFILE
	IERR = 3	error opening SFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.264. SCAL

This API routine is obsolete. It has been replaced by the API routine scal_4 .

Use this API to uniformly increase or decrease any or all specified bus quantities for a specified group of buses (activity SCAL). It must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by one reference with APIOPT=2.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SCAL SID ALL APIOPT STATUS(1)..STATUS(4) SCALVAL(1)..SCALVAL(7)
```

Python command syntax:

```
ierr, totals, moto = scal(sid, all, apiopt, status, scalval)
```

Fortran command syntax:

```
CALL SCALNW(SID, ALL, APIOPT, TOTALS, MOTO, STATUS, SCALVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer APIOPT	mode of operation in the API (input; 0 by default).	
	APIOPT = 0	initialize for scaling, then run the scaling and post-processing housekeeping.
	APIOPT = 1	initialize for scaling.
	APIOPT = 2	run the scaling and post-processing housekeeping.
Real TOTALS (11)	array of 11 elements specifying returned when APIOPT = 0 or 1. They are as follows(output).	
	TOTALS(1)	load Mvar total
	TOTALS(2)	load MW total
	TOTALS(3)	generation MW total
	TOTALS(4)	shunt MW total
	TOTALS(5)	reactor Mvar total
	TOTALS(6)	capacitor Mvar total
	TOTALS(7)	motor load MW total
	TOTALS(8)	generator PMAX
	TOTALS(9)	generator PMIN
	TOTALS(10)	motor load PMIN

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TOTALS(11)	motor load PMAX
Integer MOTO	returned when APIOPT = 0 or 1 (output).	
	MOTO = 0	no motors in the specified subsystem.
	MOTO = 1	motors present in the specified subsystem.
Integer STATUS (4)	array of 4 elements specifying that are used to control scaling. The first three elements are checked and used when APIOPT = 0 or 2; the fourth element is checked and saved when APIOPT = 0 or 1. They are as follows(input).	
	STATUS(1)	method used to scale active power load, generation and motor data, and bus shunt data (0 by default)
	STATUS(1) = 0	no scaling
	STATUS(1) = 1	specify new total powers
	STATUS(1) = 2	specify percent changes
	STATUS(1) = 3	specify incremental powers
	STATUS(2)	flag to enforce machine power limits (0 by default)
	STATUS(2) = 0	ignore machine power limits
	STATUS(2) = 1	enforce machine power limits
	STATUS(3)	flag to specify the scaling rule to be enforced when changing the reactive power load (0 by default)
	STATUS(3) = 0	no change
	STATUS(3) = 1	constant P/Q ratio
	STATUS(3) = 2	new total Q load
	STATUS(3) = 3	percent change
	STATUS(3) = 4	new power factor
	STATUS(3) = 5	incremental Q load
	STATUS(4)	bus type code flag for load scaling; normally defaulted except when used by OPF (0 by default)
	STATUS(4) = 0	all buses in subsystem
	STATUS(4) = 1	only Type 1 buses in subsystem
	STATUS(4) = 2	only Type 2 and 3 buses in subsystem
Real SCALVAL (7)	array of 7 elements specifying used as scaling targets. Based on the value of STATUS(1), entries (1) through (6) are either total powers(input).	
	SCALVAL(1)	load MW total/percent/increment
	SCALVAL(2)	generation MW total/percent/increment
	SCALVAL(3)	shunt MW total/percent/increment
	SCALVAL(4)	reactor Mvar total/percent/increment
	SCALVAL(5)	capacitor Mvar total/percent/increment
	SCALVAL(6)	motor load MW total/percent/increment
	SCALVAL(7)	reactive load scaling parameter

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	If STATUS(3)	
	SCALVAL(7) = 2	new total Mvar load (present total Mvar load by default)."
	SCALVAL(7) = 3	percent change (0.0 by default)."
	SCALVAL(7) = 4	new power factor (1.0 by default)."
	SCALVAL(7) = 5	incremental Mvar load change (0.0 by default)."
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.265. SCAL_2

This API routine is obsolete. It has been replaced by the API routine scal_4 .

Use this API routine to uniformly increase or decrease any or all specified bus quantities for a specified group of buses. It must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by one reference with APIOPT=2.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SCAL_2 SID ALL APIOPT STATUS(1)..STATUS(5) SCALVAL(1)..SCALVAL(7)
```

Python command syntax:

```
ierr, totals, moto= scal_2(sid, all, apiopt, status, scalval)
```

Fortran command syntax:

```
CALL SCAL_2(SID, ALL, APIOPT, TOTALS, MOTO, STATUS, SCALVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer APIOPT	mode of operation in the API (input; 0 by default).	
	APIOPT = 0	initialize for scaling, then run the scaling and post-processing housekeeping.
	APIOPT = 1	initialize for scaling.
	APIOPT = 2	run the scaling and post-processing housekeeping.
Real TOTALS (11)	array of elements specifying returned when APIOPT = 0 or 1. They are as follows.(output).	
	TOTALS(1)	load Mvar total.
	TOTALS(2)	load MW total.
	TOTALS(3)	generation MW total.
	TOTALS(4)	shunt MW total.
	TOTALS(5)	reactor Mvar total.
	TOTALS(6)	capacitor Mvar total.
	TOTALS(7)	motor load MW total.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TOTALS(8)	generator PMAX.
	TOTALS(9)	generator PMIN.
	TOTALS(10)	motor load PMIN.
	TOTALS(11)	motor load PMAX.
Integer MOTO	returned when APIOPT = 0 or 1. It indicates the presence of motors that are modeled as conventional generators with negative active power settings. (output).	
	MOTO = 0	no motors in the specified subsystem.
	MOTO = 1	motors present in the specified subsystem.
Integer STATUS (5)	array of elements specifying control scaling. The first and fifth elements are checked and saved when APIOPT = 0 or 1; the second, third and fourth elements are checked and used when APIOPT = 0 or 2. They are as follows.(input).	
	STATUS(1)	interruptible load scaling option (0 by default)
	STATUS(1) = 0	scale both interruptible and non-interruptible scalable loads.
	STATUS(1) = 1	scale only non-interruptible scalable loads.
	STATUS(1) = 2	scale only interruptible scalable loads.
	STATUS(2)	method used to scale active power load, generation and motor data, and bus shunt data (0 by default)
	STATUS(2) = 0	no scaling.
	STATUS(2) = 1	specify new total powers.
	STATUS(2) = 2	specify percent changes.
	STATUS(2) = 3	specify incremental powers.
	STATUS(3)	flag to enforce machine power limits (0 by default)
	STATUS(3) = 0	ignore machine power limits.
	STATUS(3) = 1	enforce machine power limits.
	STATUS(4)	flag to specify the scaling rule to be enforced when changing the reactive power load (0 by default)
	STATUS(4) = 0	no change.
	STATUS(4) = 1	constant P/Q ratio.
	STATUS(4) = 2	new total Q load.
	STATUS(4) = 3	percent change.
	STATUS(4) = 4	new power factor.
	STATUS(4) = 5	incremental Q load.
	STATUS(5)	is the bus type code flag for load scaling; normally defaulted except when used by OPF (0 by default)
	STATUS(5) = 0	all buses in subsystem.
	STATUS(5) = 1	only Type 1 buses in subsystem.
	STATUS(5) = 2	only Type 2 and 3 buses in subsystem.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real SCALVAL (7)	array of elements specifying scaling targets. Based on the value of STATUS(1), entries (1) through (6) are either total powers (present total power by default), percent changes (0.0 by default), or incremental changes (0.0 by default). If STATUS(4) = 2, SCALVAL(7) = new total Mvar load (present total Mvar load by default). If STATUS(4) = 3, SCALVAL(7) = percent change (0.0 by default). If STATUS(4) = 4, SCALVAL(7) = new power factor (1.0 by default). If STATUS(4) = 5, SCALVAL(7) = incremental Mvar load change (0.0 by default).(input).
SCALVAL(1)	load MW total/percent/increment.
SCALVAL(2)	generation MW total/percent/increment.
SCALVAL(3)	shunt MW total/percent/increment.
SCALVAL(4)	reactor Mvar total/percent/increment.
SCALVAL(5)	capacitor Mvar total/percent/increment.
SCALVAL(6)	motor load MW total/percent/increment.
SCALVAL(7)	reactive load scaling parameter.
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.266. SCAL_3

This API routine is obsolete. It has been replaced by the API routine scal_4 .

Use this API routine to uniformly increase or decrease any or all specified bus quantities for a specified group of buses. It must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by one reference with APIOPT=2.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SCAL_3 SID ALL APIOPT STATUS(1)..STATUS(5) SCALVAL(1)..SCALVAL(7)
```

Python command syntax:

```
ierr, totals, moto= scal_3(sid, all, apiopt, status, scalval)
```

Fortran command syntax:

```
CALL SCAL_3(SID, ALL, APIOPT, TOTALS, MOTO, STATUS, SCALVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer APIOPT	mode of operation in the API (input; 0 by default).	
	APIOPT = 0	initialize for scaling, then run the scaling and post-processing housekeeping.
	APIOPT = 1	initialize for scaling.
	APIOPT = 2	run the scaling and post-processing housekeeping.
Real TOTALS (11)	array of elements specifying returned when APIOPT = 0 or 1. They are as follows.(output).	
	TOTALS(1)	load Mvar total.
	TOTALS(2)	load MW total.
	TOTALS(3)	generation MW total.
	TOTALS(4)	shunt MW total.
	TOTALS(5)	reactor Mvar total.
	TOTALS(6)	capacitor Mvar total.
	TOTALS(7)	motor load MW total.
	TOTALS(8)	generator PMAX.
	TOTALS(9)	generator PMIN.
	TOTALS(10)	motor load PMIN.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TOTALS(11)	motor load PMAX.
Integer MOTO	returned when APIOPT = 0 or 1. It indicates the presence of motors that are modeled as conventional generators with negative active power settings. (output).	
	MOTO = 0	no motors in the specified subsystem.
	MOTO = 1	motors present in the specified subsystem.
Integer STATUS (5)	array of elements specifying scaling options. The first and second elements are checked and saved when APIOPT = 0 or 1; the third, fourth and fifth elements are checked and used when APIOPT = 0 or 2. They are as follows.(input).	
	STATUS(1)	interruptible load scaling option (0 by default)
	STATUS(1) = 0	scale both interruptible and non-interruptible loads.
	STATUS(1) = 1	scale only non-interruptible loads.
	STATUS(1) = 2	scale only interruptible loads.
	STATUS(2)	base loaded generation scaling option (0 by default)
	STATUS(2) = 0	scale all subsystem machines and ignore their base load flags.
	STATUS(2) = 1	scale all subsystem machines and honor their base load flags.
	STATUS(2) = 2	scale only non-base loaded subsystem machines.
	STATUS(2) = 3	scale only base loaded subsystem machines and ignore their base load flags.
	STATUS(2) = 4	scale only base loaded subsystem machines and honor their base load flags.
	STATUS(3)	method used to scale active power load, generation and motor data, and bus shunt data (0 by default)
	STATUS(3) = 0	no scaling.
	STATUS(3) = 1	specify new total powers.
	STATUS(3) = 2	specify percent changes.
	STATUS(3) = 3	specify incremental powers.
	STATUS(4)	flag to enforce machine power limits (0 by default)
	STATUS(4) = 0	ignore machine power limits.
	STATUS(4) = 1	enforce machine power limits.
	STATUS(5)	flag to specify the scaling rule to be enforced when changing the reactive power load (0 by default)
	STATUS(5) = 0	no change.
	STATUS(5) = 1	constant P/Q ratio.
	STATUS(5) = 2	new total Q load.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 3	percent change.
	STATUS(5) = 4	new power factor.
	STATUS(5) = 5	incremental Q load.
Real SCALVAL (7)	array of elements specifying scaling targets. Based on the value of STATUS(1), entries (1) through (6) are either total powers (present total power by default), percent changes (0.0 by default), or incremental changes (0.0 by default). If STATUS(5) = 2, SCALVAL(7) = new total Mvar load (present total Mvar load by default). If STATUS(5) = 3, SCALVAL(7) = percent change (0.0 by default). If STATUS(5) = 4, SCALVAL(7) = new power factor (1.0 by default). If STATUS(5) = 5, SCALVAL(7) = incremental Mvar load change (0.0 by default).(input).	
	SCALVAL(1)	load MW total/percent/increment.
	SCALVAL(2)	generation MW total/percent/increment.
	SCALVAL(3)	shunt MW total/percent/increment.
	SCALVAL(4)	reactor Mvar total/percent/increment.
	SCALVAL(5)	capacitor Mvar total/percent/increment.
	SCALVAL(6)	motor load MW total/percent/increment.
	SCALVAL(7)	reactive load scaling parameter.
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.267. SCAL_4

Use this API routine to uniformly increase or decrease any or all specified bus quantities for a specified group of buses. It must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by one reference with APIOPT=2.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SCAL_4 SID ALL APIOPT STATUS(1)..STATUS(6) SCALVAL(1)..SCALVAL(7)
```

Python command syntax:

```
ierr, totals, moto= scal_4(sid, all, apiopt, status, scalval)
```

Fortran command syntax:

```
CALL SCAL_4(SID, ALL, APIOPT, TOTALS, MOTO, STATUS, SCALVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer APIOPT	mode of operation in the API (input; 0 by default).	
	APIOPT = 0	initialize for scaling, then run the scaling and post-processing housekeeping.
	APIOPT = 1	initialize for scaling.
	APIOPT = 2	run the scaling and post-processing housekeeping.
Real TOTALS (11)	array of elements specifying returned when APIOPT = 0 or 1. They are as follows.(output).	
	TOTALS(1)	load Mvar total.
	TOTALS(2)	load MW total.
	TOTALS(3)	generation MW total.
	TOTALS(4)	shunt MW total.
	TOTALS(5)	reactor Mvar total.
	TOTALS(6)	capacitor Mvar total.
	TOTALS(7)	motor load MW total.
	TOTALS(8)	generator PMAX.
	TOTALS(9)	generator PMIN.
	TOTALS(10)	motor load PMIN.
	TOTALS(11)	motor load PMAX.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer MOTO	returned when APIOPT = 0 or 1. It indicates the presence of motors that are modeled as conventional generators with negative active power settings. (output).
	MOTO = 0 no motors in the specified subsystem.
	MOTO = 1 motors present in the specified subsystem.
Integer STATUS (6)	array of elements specifying scaling options. The first, second and third elements are checked and saved when APIOPT = 0 or 1; the fourth, fifth and sixth elements are checked and used when APIOPT = 0 or 2. They are as follows.(input).
	STATUS(1) interruptible load scaling option (0 by default)
	STATUS(1) = 0 scale both interruptible and non-interruptible loads.
	STATUS(1) = 1 scale only non-interruptible loads.
	STATUS(1) = 2 scale only interruptible loads.
	STATUS(2) load component scaling opotion (0 by default)
	STATUS(2) = 0 scale in-service net load
	STATUS(2) = 1 scale only in-service load
	STATUS(2) = 2 scale only in-service distributed generation
	STATUS(3) base loaded generation scaling option (0 by default)
	STATUS(3) = 0 scale all subsystem machines and ignore their base load flags.
	STATUS(3) = 1 scale all subsystem machines and honor their base load flags.
	STATUS(3) = 2 scale only non-base loaded subsystem machines.
	STATUS(3) = 3 scale only base loaded subsystem machines and ignore their base load flags.
	STATUS(3) = 4 scale only base loaded subsystem machines and honor their base load flags.
	STATUS(4) method used to scale active power load, generation and motor data, and bus shunt data (0 by default)
	STATUS(4) = 0 no scaling.
	STATUS(4) = 1 specify new total powers.
	STATUS(4) = 2 specify percent changes.
	STATUS(4) = 3 specify incremental powers.
	STATUS(5) flag to enforce machine power limits (0 by default)
	STATUS(5) = 0 ignore machine power limits.
	STATUS(5) = 1 enforce machine power limits.
	STATUS(6) flag to specify the scaling rule to be enforced when changing the reactive power load (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(6) = 0	no change.
STATUS(6) = 1	constant P/Q ratio.
STATUS(6) = 2	new total Q load.
STATUS(6) = 3	percent change.
STATUS(6) = 4	new power factor.
STATUS(6) = 5	incremental Q load.

Real SCALVAL (7)

array of elements specifying scaling targets. Based on the value of STATUS(1), entries (1) through (6) are either total powers (present total power by default), percent changes (0.0 by default), or incremental changes (0.0 by default). If STATUS(5) = 2, SCALVAL(7) = new total Mvar load (present total Mvar load by default). If STATUS(5) = 3, SCALVAL(7) = percent change (0.0 by default). If STATUS(5) = 4, SCALVAL(7) = new power factor (1.0 by default). If STATUS(5) = 5, SCALVAL(7) = incremental Mvar load change (0.0 by default).(input).

SCALVAL(1)	load MW total/percent/increment.
SCALVAL(2)	generation MW total/percent/increment.
SCALVAL(3)	shunt MW total/percent/increment.
SCALVAL(4)	reactor Mvar total/percent/increment.
SCALVAL(5)	capacitor Mvar total/percent/increment.
SCALVAL(6)	motor load MW total/percent/increment.
SCALVAL(7)	reactive load scaling parameter.

Integer IERR

error code (output).

IERR = 0	no error occurred
IERR = 1	invalid SID value or subsystem SID is not defined
IERR = 2	invalid ALL value
IERR = 3	invalid APIOPT value
IERR = 4	invalid STATUS value
IERR = 5	unexpected APIOPT value
IERR = 6	invalid power factor in SCALVAL(7)
IERR = 7	generator or motor totals beyond limits
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.268. SENSITIVITY_FLOW

This API is used to calculate sensitivity factors of a branch flow to MW power at buses, MW power at generator buses, MW at load buses, phase angle of phase shifters, tap postion of tap changing transformers as well as admittance of switched shunts:

This API routine was first introduced in release 32.1.0.

Batch command syntax:

```
BAT_SENSITIVITY_FLOW FLWOPT(1) .. FLWOPT(5) OPTIONS(1) .. OPTIONS(9)
 VALUES(1) VALUES(2) BRNCKT LABELS(1) LABELS(2) DFXFILE
```

Python command syntax:

```
ierr = sensitivity_flow(flwopt,options,values,brnckt,labels,dfxfile)
```

Fortran command syntax:

```
CALL SENSITIVITY_FLOW(FLWOPT,OPTIONS,VALUES,BRNCKT,LABELS,DFXFILE,IERR)
```

Where:

Integer FLWOPT (5)	array of 5 elements specifying (input).	
	FLWOPT(1)	from bus of the branch for which sensitivity factors are calculated (no default is allowed)
	FLWOPT(2)	to bus of the branch (no default is allowed)
	FLWOPT(3)	third bus connected by a three-winding transformer, or zero for a non-transformer branch or a two-winding transformer (0 by default)
	FLWOPT(4)	network model on which sensitivity anlayiss is based
	FLWOPT(4) = 0	DC network model
	FLWOPT(4) = 1	AC network model
	FLWOPT(5)	branch flow type to which the sensitivity factors are calculated (If DC network is used, only the branch MW flow is valid for sensitivity analysis. 0 by default)
	FLWOPT(5) = 0	MW flow
	FLWOPT(5) = 1	MVar flow
	FLWOPT(5) = 2	MVA flow
	FLWOPT(5) = 3	Amps flow
Integer OPTIONS (9)	array of 9 elements specifying options for sensitvity analysis to a branch flow(input).	
	OPTIONS(1)	flag to calculate sensitivity factors at buses
	OPTIONS(1) = 0	Disable, do not calculate sensitivity factors at buses
	OPTIONS(1) = 1	Enable, calculate sensitivity factors at buses
	OPTIONS(2)	flag to calculate sensitivity factors at generator buses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(2) = 0	Disable
OPTIONS(2) = 1	Enable
OPTIONS(3)	flag to calculate sensitivity factors at load buses
OPTIONS(3) = 0	Disable
OPTIONS(3) = 1	Enable
OPTIONS(4)	flag to calculate sensitivity factors of the branch flow to phase shifters
OPTIONS(4) = 0	Disable
OPTIONS(4) = 1	Enable
OPTIONS(5)	flag to calculate sensitivity factors of the branch flow to tap changing transformers (ignore if DC network model is used)
OPTIONS(5) = 0	Disable
OPTIONS(5) = 1	Enable
OPTIONS(6)	flag to calculate sensitivity factors of the branch flow to switched shunts (ignore if DC network model is used)
OPTIONS(6) = 0	Disable
OPTIONS(6) = 1	Enable
OPTIONS(7)	flag to specifiy the transfer from the components to which sensitivity factors of the branch flow are calculated to the opposing subsystem (the flag is ignored if none of the bus, generator, or load sensitivity calculation flags is selected)
OPTIONS(7) = 0	Import. The positive transfer is from the components to the opposing subsystem
OPTIONS(7) = 1	Export. The positive transfer is from the opposing subsystem to the components
OPTIONS(8)	opposing subsystem type
OPTIONS(8) = 0	Slack bus
OPTIONS(8) = 1	Single bus
OPTIONS(8) = 2	Subsystem
OPTIONS(9)	ignored if opposing subsystem type is slack bus. It is a bus number when opposing subsystem type is a single bus, or one of the following seven dispatch modes if the opposing subsystem type is a subsystem (1 by default)
OPTIONS(9) = 1	Buses & participating factors from DFAX file for buses with positive MW machines
OPTIONS(9) = 2	Buses & participating factors from DFAX file for buses with positive MW constant MVA load

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(9) = 3	Buses & participating factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(9) = 4	Subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(9) = 5	Subsystem buses with positive MW machines in proportion to their MW output
	OPTIONS(9) = 6	Subsystem buses with positive MW machines in proportion to their MBASE
	OPTIONS(9) = 7	Subsystem buses with positive MW machines in proportion to their reserves (Pmaxmc-Pgenmc)
Real VALUES (2)	array of 2 elements specifying sensitivity analysis parameters(input).	
	VALUES(1)	mismatch tolerance for continuing sensitivity analysis in AC mode, ignored in DC mode
	VALUES(2)	sensitivity cutoff. Sensitivity factors less than the cutoff are not presented
Character*2 BRNCKT	branch circuit ID (input).	
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the subsystem containing components to which the sensitivity factors of the branch flow are calcualted
	LABELS(2)	opposing subsystem label when the opposing subsystem type is a subsystem
Character*260 DFXFILE	DFAX file name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	The branch can not be found
	IERR = 4	Generators are converted
	IERR = 5	Buses in island(s) without a swing bus--use activity TREE
	IERR = 6	Dispatch bus does not exist
	IERR = 7	No Distribution Factor Data input file specified
	IERR = 8	Subsystem containing components to which sensitivity factors are calculated is not specified
	IERR = 9	Opposing subsystem is not specified
	IERR = 10	Largest mismatch exceeds mismatch tolerance
	IERR = 11	The branch bus does not exist
	IERR = 12	The branch is out-of-service
	IERR = 13	The branch is a zero impedance line

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 14	The branch is a multi-section line but the Report Multi-Section line option is disabled
IERR = 15	Invalid FLWOPT value
IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file, run DFAX
IERR = 22	Error linearizing Jacobian matrix in AC network model
IERR = 51	error opening Distribution Factor Data File
IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.269. SENSITIVITY_FLOWS

This API is used to calculate sensitivity factors of flows on the branches in a subsystem to MW power at buses, MW power at generator buses, MW power at load buses, phase angle of phase shifters, tap postion of tap changing transformers as well as admittance of switched shunts.

This API routine was first introduced in release 32.1.0.

Batch command syntax:

```
BAT_SENSITIVITY_FLOWS FLWOPT(1) FLWOPT(2) OPTIONS(1) .. OPTIONS(9)
 VALUES(1) VALUES(2) LABELS(1) .. LABELS(3) DFXFILE
```

Python command syntax:

```
ierr = sensitivity_flows(flwopt,options,values,labels,dfxfile)
```

Fortran command syntax:

```
CALL SENSITIVITY_FLOWS(FLWOPT,OPTIONS,VALUES,LABELS,DFXFILE,IERR)
```

Where:

Integer FLWOPT (2)	array of 2 elements specifying options for sensitivity analysis for branches in a subsystem(input).	
	FLWOPT(1)	network model on which sensitivity analysis is based
	FLWOPT(1) = 0	DC network model
	FLWOPT(1) = 1	AC network model
	FLWOPT(2)	branch flow type to which the sensitivity factors are calculated (when DC network is used, only the branch MW flow is valid for sensitivity analysis.)
	FLWOPT(2) = 0	MW flow
	FLWOPT(2) = 1	MVar flow
	FLWOPT(2) = 2	MVA flow
	FLWOPT(2) = 3	Amps flow
Integer OPTIONS (9)	array of 9 elements specifying options for sensitvity analysis to a branch flow(input).	
	OPTIONS(1)	flag to calculate sensitivity factors at buses
	OPTIONS(1) = 0	Disable, do not calculate sensitivity factors at buses
	OPTIONS(1) = 1	Enable, calculate sensitivity factors at buses
	OPTIONS(2)	flag to calculate sensitivity factors at generator buses
	OPTIONS(2) = 0	Disable
	OPTIONS(2) = 1	Enable
	OPTIONS(3)	flag to calculate sensitivity factors at load buses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(3) = 0	Disable
OPTIONS(3) = 1	Enable
OPTIONS(4)	flag to calculate sensitivity factors of the branch flow to phase shifters
OPTIONS(4) = 0	Disable
OPTIONS(4) = 1	Enable
OPTIONS(5)	flag to calculate sensitivity factors of the branch flow to tap changing transformers (ignore if DC network model is used)
OPTIONS(5) = 0	Disable
OPTIONS(5) = 1	Enable
OPTIONS(6)	flag to calculate sensitivity factors of the branch flow to switched shunts (ignore if DC network model is used)
OPTIONS(6) = 0	Disable
OPTIONS(6) = 1	Enable
OPTIONS(7)	flag to specifiy the transfer from the components to which sensitivity factors of the branch flow are calculated to the opposing subsystem (the flag is ignored if none of the bus, generator, or load sensitivity calculation flags is selected)
OPTIONS(7) = 0	Import. The positive transfer is from the components to the opposing subsystem
OPTIONS(7) = 1	Export. The positive transfer is from the opposing subsystem to the components
OPTIONS(8)	opposing subsystem type
OPTIONS(8) = 0	Slack bus
OPTIONS(8) = 1	Single bus
OPTIONS(8) = 2	Subsystem
OPTIONS(9)	ignored if opposing subsystem type is slack bus. It is a bus number when opposing subsystem type is a single bus, or one of the following seven dispatch modes if the opposing subsystem type is a subsystem (1 by default)
OPTIONS(9) = 1	Buses & participating factors from DFAX file for buses with positive MW machines
OPTIONS(9) = 2	Buses & participating factors from DFAX file for buses with positive MW constant MVA load
OPTIONS(9) = 3	Buses & participating factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
OPTIONS(9) = 4	Subsystem buses with positive MW constant MVA load in proportion to their MW load
OPTIONS(9) = 5	Subsystem buses with positive MW machines in proportion to their MW output

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(9) = 6	Subsystem buses with positive MW machines in proportion to their MBASE
	OPTIONS(9) = 7	Subsystem buses with positive MW machines in proportion to their reserves (Pmaxmc-Pgenmc)
Real VALUES (2)	array of 2 elements specifying sensitivity analysis parameters(input).	
	VALUES(1)	mismatch tolerance for continuing sensitivity analysis in AC mode, ignored in DC mode
	VALUES(2)	sensitivity cutoff. Sensitivity factors less than the cutoff are not presented
Character*32 LABELS (3)	array of 3 elements specifying subsystem labels(input).	
	LABELS(1)	label of the subsystem containing components to which the sensitivity factors of the branch flow are calcualted
	LABELS(2)	label of opposing subsystem when the opposing subsystem type is a subsystem
	LABELS(3)	label of the subsystem including branches for which sensitivity factors are calculated
Character*260 DFXFILE	DFAX file name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	The branch can not be found
	IERR = 4	Generators are converted
	IERR = 5	Buses in island(s) without a swing bus--use activity TREE
	IERR = 6	Dispatch bus does not exist
	IERR = 7	No Distribution Factor Data input file specified
	IERR = 8	Subsystem containing components to which sensitivity factors are calculated is not specified
	IERR = 9	Opposing subsystem is not specified
	IERR = 10	Largest mismatch exceeds mismatch tolerance
	IERR = 11	Invalid FLWOPT value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file, run DFAX
	IERR = 22	Error linearizing Jacobian matrix in AC network model
	IERR = 51	error opening Distribution Factor Data File
	IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.270. SENSITIVITY_INTERFACE

This API is used to calculate sensitivity factors of an interface flow to MW power at buses, MW power at generator buses, MW power at load buses, phase angle of phase shifters, tap position of tap changing transformers as well as admittance of switched shunts.

This API routine was first introduced in release 32.1.0.

Batch command syntax:

```
BAT_SENSITIVITY_INTERFACE MODE OPTIONS(1) .. OPTIONS(9) VALUES(1)
 VALUES(2) LABELS(1) .. LABELS(3) DFXFILE
```

Python command syntax:

```
ierr = sensitivity_interface(mode,options,values,labels,dfxfile)
```

Fortran command syntax:

```
CALL SENSITIVITY_INTERFACE(MODE,OPTIONS,VALUES,LABELS,DFXFILE,IERR)
```

Where:

Integer MODE	network model on which sensitivity analysis is based (input; 0 by default).	
	MODE = 0	DC network model
	MODE = 1	AC network model
Integer OPTIONS (9)	array of 9 elements specifying options for sensitvity analysis to a branch flow(input).	
	OPTIONS(1)	flag to calculate sensitivity factors at buses
	OPTIONS(1) = 0	Disable, do not calculate sensitivity factors at buses
	OPTIONS(1) = 1	Enable, calculate sensitivity factors at buses
	OPTIONS(2)	flag to calculate sensitivity factors at generator buses
	OPTIONS(2) = 0	Disable
	OPTIONS(2) = 1	Enable
	OPTIONS(3)	flag to calculate sensitivity factors at load buses
	OPTIONS(3) = 0	Disable
	OPTIONS(3) = 1	Enable
	OPTIONS(4)	flag to calculate sensitivity factors of the branch flow to phase shifters
	OPTIONS(4) = 0	Disable
	OPTIONS(4) = 1	Enable
	OPTIONS(5)	flag to calculate sensitivity factors of the branch flow to tap changing transformers (ignore if DC network model is used)
	OPTIONS(5) = 0	Disable
	OPTIONS(5) = 1	Enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	flag to calculate sensitivity factors of the branch flow to switched shunts (ignore if DC network model is used)
	OPTIONS(6) = 0	Disable
	OPTIONS(6) = 1	Enable
	OPTIONS(7)	flag to specifiy the transfer from the components to which sensitivity factors of the branch flow are calculated to the opposing subsystem (the flag is ignored if none of the bus, generator, or load sensitivity calculation flags is selected)
	OPTIONS(7) = 0	Import. The positive transfer is from the components to the opposing subsystem
	OPTIONS(7) = 1	Export. The positive transfer is from the opposing subsystem to the components
	OPTIONS(8)	opposing subsystem type
	OPTIONS(8) = 0	Slack bus
	OPTIONS(8) = 1	Single bus
	OPTIONS(8) = 2	Subsystem
	OPTIONS(9)	ignored if opposing subsystem type is slack bus. It is a bus number when opposing subsystem type is a single bus, or one of the following seven dispatch modes if the opposing subsystem type is a subsystem (1 by default)
	OPTIONS(9) = 1	Buses & participating factors from DFAX file for buses with positive MW machines
	OPTIONS(9) = 2	Buses & participating factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(9) = 3	Buses & participating factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(9) = 4	Subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(9) = 5	Subsystem buses with positive MW machines in proportion to their MW output
	OPTIONS(9) = 6	Subsystem buses with positive MW machines in proportion to their MBASE
	OPTIONS(9) = 7	Subsystem buses with positive MW machines in proportion to their reserves (Pmaxmc-Pgenmc)
Real VALUES (2)	array of 2 elements specifying sensitivity analysis parameters(input).	
	VALUES(1)	mismatch tolerance for continuing sensitivity analysis in AC mode, ignored in DC mode
	VALUES(2)	sensitivity cutoff. Sensitivity factors less than the cutoff are not presented

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*32 LABELS (3)	array of 3 elements specifying subsystem/interface labels(input).	
	LABELS(1)	label of the subsystem containing components to which the sensitivity factors of the interface flow are calculated
	LABELS(2)	label of opposing subsystem when the opposing subsystem type is a subsystem
	LABELS(3)	label of the interface for which sensitivity factors are calculated
Character*260 DFXFILE	DFAX file name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	Interface is not specified
	IERR = 4	Generators are converted
	IERR = 5	Buses in island(s) without a swing bus--use activity TREE
	IERR = 6	Dispatch bus does not exist
	IERR = 7	No Distribution Factor Data input file specified
	IERR = 8	Subsystem containing components to which sensitivity factors are calculated is not specified
	IERR = 9	Opposing subsystem is not specified
	IERR = 10	Largest mismatch exceeds mismatch tolerance
	IERR = 11	Invalid MODE value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file, run DFAX
	IERR = 22	Error in linearizing Jacobian matrix with AC network model
	IERR = 51	error opening Distribution Factor Data File
	IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.271. SENSITIVITY_VOLTAGE

This API is used to calculate sensitivity factors of a bus voltage to MW and MVar power at buses, MW power at generator buses, MW and MVar at load buses, phase angle of phase shifters, tap postion of tap changing transformers as well as admittance of switched shunts.

This API routine was first introduced in release 32.1.0.

Batch command syntax:

```
BAT_SENSITIVITY_VOLTAGE IBUS OPTIONS(1) .. OPTIONS(9) VALUES(1) VALUES(2)
 LABELS(1) LABELS(2) DFXFILE
```

Python command syntax:

```
ierr = sensitivity_voltage(ibus,options,values,labels,dfxfile)
```

Fortran command syntax:

```
CALL SENSITIVITY_VOLTAGE(IBUS,OPTIONS,VALUES,LABELS,DFXFILE,IERR)
```

Where:

Integer IBUS	bus number for which sensitivity factors are calculated (input).
Integer OPTIONS (9)	array of 9 elements specifying options for sensitvity analysis to a branch flow(input).

OPTIONS(1)	flag to calculate sensitivity factors at buses
OPTIONS(1) = 0	Disable, do not calculate sensitivity factors at buses
OPTIONS(1) = 1	Enable, calculate sensitivity factors at buses
OPTIONS(2)	flag to calculate sensitivity factors at generator buses
OPTIONS(2) = 0	Disable
OPTIONS(2) = 1	Enable
OPTIONS(3)	flag to calculate sensitivity factors at load buses
OPTIONS(3) = 0	Disable
OPTIONS(3) = 1	Enable
OPTIONS(4)	flag to calculate sensitivity factors of the branch flow to phase shifters
OPTIONS(4) = 0	Disable
OPTIONS(4) = 1	Enable
OPTIONS(5)	flag to calculate sensitivity factors of the branch flow to tap changing transformers (ignore if DC network model is used)
OPTIONS(5) = 0	Disable
OPTIONS(5) = 1	Enable
OPTIONS(6)	flag to calculate sensitivity factors of the branch flow to switched shunts (ignore if DC network model is used)
OPTIONS(6) = 0	Disable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6) = 1	Enable
	OPTIONS(7)	flag to specifiy the transfer from the components to which sensitivity factors of the branch flow are calculated to the opposing subsystem (the flag is ignored if none of the bus, generator, or load sensitivity calculation flags is selected)
	OPTIONS(7) = 0	Import. The positive transfer is from the components to the opposing subsystem
	OPTIONS(7) = 1	Export. The positive transfer is from the opposing subsystem to the components
	OPTIONS(8)	opposing subsystem type
	OPTIONS(8) = 0	Slack bus
	OPTIONS(8) = 1	Single bus
	OPTIONS(8) = 2	Subsystem
	OPTIONS(9)	ignored if opposing subsystem type is slack bus. It is a bus number when opposing subsystem type is a single bus, or one of the following seven dispatch modes if the opposing subsystem type is a subsystem (1 by default)
	OPTIONS(9) = 1	Buses & participating factors from DFAX file for buses with positive MW machines
	OPTIONS(9) = 2	Buses & participating factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(9) = 3	Buses & participating factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(9) = 4	Subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(9) = 5	Subsystem buses with positive MW machines in proportion to their MW output
	OPTIONS(9) = 6	Subsystem buses with positive MW machines in proportion to their MBASE
	OPTIONS(9) = 7	Subsystem buses with positive MW machines in proportion to their reserves (Pmaxmc-Pgenmc)
Real VALUES (2)	array of 2 elements specifying sensitivity analysis parameters(input).	
	VALUES(1)	mismatch tolerance for continuing sensitivity analysis in AC mode, ignored in DC mode
	VALUES(2)	sensitivity cutoff. Sensitivity factors less than the cutoff are not presented
Character*32 LABELS (2)	array of 2 elements specifying subsystem labels(input).	
	LABELS(1)	label of the subsystem containing components to which the sensitivity factors of the bus voltage are calcualted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(2)	opposing subsystem label when the opposing subsystem type is a subsystem
Character*260 DFXFILE	DFAX file name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	The bus does not exist
	IERR = 4	Generators are converted
	IERR = 5	Buses in island(s) without a swing bus--use activity TREE
	IERR = 6	Dispatch bus does not exist
	IERR = 7	No Distribution Factor Data input file specified
	IERR = 8	Subsystem containing components to which sensitivity factors are calculated is not specified
	IERR = 9	Opposing subsystem is not specified
	IERR = 10	Largest mismatch exceeds mismatch tolerance
	IERR = 11	The bus is a type 2 or type 3 bus
	IERR = 12	Invalid IBUS value
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file, run DFAX
	IERR = 22	Error linearizing Jacobian matrix in AC network model
	IERR = 51	error opening Distribution Factor Data File
	IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.272. SENSITIVITY_VOLTAGES

This API is used to calculate sensitivity factors of bus voltages in a subsystem to MW and MVar power at buses, MW power at generator buses, MW and MVar at load buses, phase angle of phase shifters, tap postion of tap changing transformers as well as admittance of switched shunts.

This API routine was first introduced in release 32.1.0.

Batch command syntax:

```
BAT_SENSITIVITY_VOLTAGES OPTIONS(1) .. OPTIONS(9) VALUES(1) VALUES(2)
 LABELS(1) .. LABELS(3) DFXFILE
```

Python command syntax:

```
ierr = sensitivity_voltages(options,values,labels,dfxfile)
```

Fortran command syntax:

```
CALL SENSITIVITY_VOLTAGES(OPTIONS,VALUES,LABELS,DFXFILE,IERR)
```

Where:

Integer OPTIONS (9)	array of 9 elements specifying options for sensitvity analysis to a branch flow(input).	
	OPTIONS(1)	flag to calculate sensitivity factors at buses
	OPTIONS(1) = 0	Disable, do not calculate sensitivity factors at buses
	OPTIONS(1) = 1	Enable, calculate sensitivity factors at buses
	OPTIONS(2)	flag to calculate sensitivity factors at generator buses
	OPTIONS(2) = 0	Disable
	OPTIONS(2) = 1	Enable
	OPTIONS(3)	flag to calculate sensitivity factors at load buses
	OPTIONS(3) = 0	Disable
	OPTIONS(3) = 1	Enable
	OPTIONS(4)	flag to calculate sensitivity factors of the branch flow to phase shifters
	OPTIONS(4) = 0	Disable
	OPTIONS(4) = 1	Enable
	OPTIONS(5)	flag to calculate sensitivity factors of the branch flow to tap changing transformers (ignore if DC network model is used)
	OPTIONS(5) = 0	Disable
	OPTIONS(5) = 1	Enable
	OPTIONS(6)	flag to calculate sensitivity factors of the branch flow to switched shunts (ignore if DC network model is used)
	OPTIONS(6) = 0	Disable
	OPTIONS(6) = 1	Enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(7)	flag to specifiy the transfer from the components to which sensitivity factors of the branch flow are calculated to the opposing subsystem (the flag is ignored if none of the bus, generator, or load sensitivity calculation flags is selected)
	OPTIONS(7) = 0	Import. The positive transfer is from the components to the opposing subsystem
	OPTIONS(7) = 1	Export. The positive transfer is from the opposing subsystem to the components
	OPTIONS(8)	opposing subsystem type
	OPTIONS(8) = 0	Slack bus
	OPTIONS(8) = 1	Single bus
	OPTIONS(8) = 2	Subsystem
	OPTIONS(9)	ignored if opposing subsystem type is slack bus. It is a bus number when opposing subsystem type is a single bus, or one of the following seven dispatch modes if the opposing subsystem type is a subsystem (1 by default)
	OPTIONS(9) = 1	Buses & participating factors from DFAX file for buses with positive MW machines
	OPTIONS(9) = 2	Buses & participating factors from DFAX file for buses with positive MW constant MVA load
	OPTIONS(9) = 3	Buses & participating factors from DFAX file for buses with either positive MW machines or positive MW constant MVA load
	OPTIONS(9) = 4	Subsystem buses with positive MW constant MVA load in proportion to their MW load
	OPTIONS(9) = 5	Subsystem buses with positive MW machines in proportion to their MW output
	OPTIONS(9) = 6	Subsystem buses with positive MW machines in proportion to their MBASE
	OPTIONS(9) = 7	Subsystem buses with positive MW machines in proportion to their reserves (Pmaxmc-Pgenmc)
Real VALUES (2)	array of 2 elements specifying sensitivity analysis parameters(input).	
	VALUES(1)	mismatch tolerance for continuing sensitivity analysis in AC mode, ignored in DC mode
	VALUES(2)	sensitivity cutoff. Sensitivity factors less than the cutoff are not presented
Character*32 LABELS (3)	array of 3 elements specifying subsystem labels(input).	
	LABELS(1)	label of the subsystem containing components to which the sensitivity factors of the branch flow are calcualted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(2)	label of opposing subsystem when the opposing subsystem type is a subsystem
	LABELS(3)	label of the subsystem including branches for which sensitivity factors are calculated
Character*260 DFXFILE	DFAX file name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS value
	IERR = 2	Invalid VALUES value
	IERR = 3	The bus does not exist
	IERR = 4	Generators are converted
	IERR = 5	Buses in island(s) without a swing bus--use activity TREE
	IERR = 6	Dispatch bus does not exist
	IERR = 7	No Distribution Factor Data input file specified
	IERR = 8	Subsystem containing components to which sensitivity factors are calculated is not specified
	IERR = 9	Opposing subsystem is not specified
	IERR = 10	Largest mismatch exceeds mismatch tolerance
	IERR = 21	file DFXFILE is not in the form of a PSSE-25 or later DFAX file, run DFAX
	IERR = 22	Error linearizing Jacobian matrix in AC network model
	IERR = 51	error opening Distribution Factor Data File
	IERR = 52	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.273. SET_INPUT_DEV

Use this API to set the terminal input device to a file.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SET_INPUT_DEV IDVFIL
```

Python command syntax:

```
ierr = set_input_dev(idvfil)
```

Fortran command syntax:

```
CALL IDEVAPI(IDVFIL, IERR)
```

Where:

Character*260 IDVFIL	name of response file to be used for terminal (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error opening IDVFIL
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.274. SET_PROGRESS_VERBOSE

Use this API to to set the option progress message verbose value. This value is used to show or not show some repetitive progress messages when reading sequence data file.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Batch command syntax:

```
BAT_SET_PROGRESS_VERBOSE IVAL
```

Python command syntax:

```
ierr = set_progress_verbose(ival)
```

Fortran command syntax:

```
CALL SET_PROGRESS_VERBOSE(IVAL, IERR)
```

Where:

Integer IVAL	value of the option setting (input-1).	
	IVAL = -1	show all messages
	IVAL = 0	do not show repetitive messages
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.275. SHNT

Use this API to tabulate fixed and/or switched bus shunts contained in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SHNT SID ALL OPT
```

Python command syntax:

```
ierr = shnt(sid, all, opt)
```

Fortran command syntax:

```
CALL SHNTNW(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	option to select the type of bus shunts tabulated (input; 1 by default).	
	OPT = 1	all shunts
	OPT = 2	fixed bus shunts
	OPT = 3	switched shunts
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.276. SHORT_CIRCUIT_COORDINATES

Use this API to specify or return the option to set the fault analysis voltage and current output coordinates to either rectangular or polar coordinates.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SHORT_CIRCUIT_COORDINATES IVAL
```

Python command syntax:

```
ierr = short_circuit_coordinates(ival)

ierr, ival = short_circuit_coordinates()
```

Fortran command syntax:

```
CALL SHORT_CIRCUIT_COORDINATES_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	rectangular coordinates.
	IVAL = 1	polar coordinates.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.277. SHORT_CIRCUIT_MODELING

Use this API to specify or return the option for the fault analysis modeling setting; either normal three-phase or center tapped two-phase.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SHORT_CIRCUIT_MODELING IVAL
```

Python command syntax:

```
ierr = short_circuit_modeling(ival)

ierr, ival = short_circuit_modeling()
```

Fortran command syntax:

```
CALL SHORT_CIRCUIT_MODELING_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	center tapped two-phase.
	IVAL = 1	normal three-phase.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.278. SHORT_CIRCUIT_UNITS

Use this API to specify or return the option to set the fault analysis voltage and current output units to either per unit or physical units.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SHORT_CIRCUIT_UNITS  IVAL
```

Python command syntax:

```
ierr = short_circuit_units(ival)

ierr, ival = short_circuit_units()
```

Fortran command syntax:

```
CALL SHORT_CIRCUIT_UNITS_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	per unit.
	IVAL = 1	physical units.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.279. SHORT_CIRCUIT_WARNING

Use this API to specify or return the option to enable or disable the fault analysis warning option setting.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SHORT_CIRCUIT_WARNING IVAL
```

Python command syntax:

```
ierr = short_circuit_warning(ival)

ierr, ival = short_circuit_warning()
```

Fortran command syntax:

```
CALL SHORT_CIRCUIT_WARNING_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
	IVAL = 0	disable printing of RESQ/TRSQ/solution warnings.
	IVAL = 1	print RESQ/TRSQ/solution warnings.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.280. SHORT_CIRCUIT_Z_COORDINATES

Use this API to specify or return the option to set the fault analysis output impedance coordinates to either rectangular or polar coordinates.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SHORT_CIRCUIT_Z_COORDINATES IVAL
```

Python command syntax:

```
ierr = short_circuit_z_coordinates(ival)

ierr, ival = short_circuit_z_coordinates()
```

Fortran command syntax:

```
CALL SHORT_CIRCUIT_Z_COORDINATES(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	rectangular coordinates.
	IVAL = 1	polar coordinates.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.281. SHORT_CIRCUIT_Z_UNITS

Use this API to specify or return the option to set the fault analysis output impedance units to either per unit or ohms.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SHORT_CIRCUIT_Z_UNITS IVAL
```

Python command syntax:

```
ierr = short_circuit_z_units(ival)

ierr, ival = short_circuit_z_units()
```

Fortran command syntax:

```
CALL SHORT_CIRCUIT_Z_UNITS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	per unit.
	IVAL = 1	physical units.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.282. SHOW

Use this API to tabulate summaries of Saved Case and/or Snapshot Files.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SHOW OPTION SFILE
```

Python command syntax:

```
ierr = show(option, sfile)
```

Fortran command syntax:

```
CALL SHOWAPI(OPTION, SFILE, IERR)
```

Where:

Integer OPTION	code for the type(s) of files to report (input; 1 by default).	
	OPTION = 0	file specified in SFILE
	OPTION = 1	all .savx, .sav and .snp files
	OPTION = 2	all .savx and .sav files
	OPTION = 3	all .snp files
Character*260 SFILE	name of saved case or snapshot file (when OPTION is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTION value
	IERR = 2	SFILE is blank when OPTION is 0
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.283. SHOWTABLE

Directs a report table to output console. Use API_NAME instead to redirect report table output to the Progress Window or Report Window.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr = showtable(tblnam, direction)
```

Fortran command syntax:

```
CALL SHOWTABLE(TBLNAM, DIRECTION, IERR)
```

Where:

Character*512 TBLNAM	(input).	
Integer DIRECTION	(input).	
	DIRECTION = 1	Progress window
	DIRECTION = 2	Report window
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.284. SHOWTABLEBYINDEX

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr = showtablebyindex(tblid, direction)
```

Fortran command syntax:

```
CALL SHOWTABLEBYINDEX(TBLID, DIRECTION, IERR)
```

Where:

Integer TBLID	(input).	
Integer DIRECTION	(input).	
	DIRECTION = 1	Progress window
	DIRECTION = 2	Report window
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.285. SIZE

Use this API to obtain a summary of the number of components in the working case (activity SIZE).

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SIZE
```

Python command syntax:

```
ierr = size()
```

Fortran command syntax:

```
CALL SIZEAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.286. SOLV

Use this API to apply the Gauss-Seidel power flow calculation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SOLV  OPTIONS(1)..OPTIONS(6)
```

Python command syntax:

```
ierr = solv(options)
```

Fortran command syntax:

```
CALL SOLVAPI(OPTIONS, IERR)
```

Where:

Integer OPTIONS (6)	array of 6 elements specifying solution options(input).	
	OPTIONS(1)	tap adjustment flag (use tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable
	OPTIONS(2)	area interchange adjustment flag (use area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	flat start flag (0 by default)
	OPTIONS(3) = 0	do not flat start
	OPTIONS(3) = 1	flat start
	OPTIONS(3) = 2	flat start, then estimate voltage magnitudes
	OPTIONS(3) = 3	flat start, then estimate voltage phase angles
	OPTIONS(3) = 4	flat start, then estimate voltage magnitudes and phase angles
	OPTIONS(4)	dc tap adjustment flag (use dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (use switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 2	enable continuous mode, disable discrete mode
	OPTIONS(6)	var limit flag (0 by default)
	OPTIONS(6) = 0	apply var limits
	OPTIONS(6) = 1	ignore var limits
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	generators are converted
	IERR = 3	buses in island(s) without a swing bus; use activity TREE
	IERR = 4	bus type code and series element status inconsistencies
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.287. SPCB

Use this API to calculate positive sequence equivalents of branch unbalances.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SPCB IBUS JBUS CKT OPTIONS(1)..OPTIONS(4) VALUES(1)..VALUES(5)
```

Python command syntax:

```
ierr = spcb(ibus, jbus, ckt, options, values)
```

Fortran command syntax:

```
CALL SPCBAPI(IBUS, JBUS, CKT, OPTIONS, VALUES, IERR)
```

Where:

Integer IBUS	from bus of branch subjected to unbalance (input; no default allowed).	
Integer JBUS	to bus of branch subjected to unbalance (input; no default allowed).	
Character*2 CKT	circuit identifier of branch subjected to unbalance (input; '1' by default).	
Integer OPTIONS (4)	array of 4 elements specifying unbalance OPTIONS(input).	
	OPTIONS(1)	unbalance code (0 by default)
	OPTIONS(1) = 0	no unbalance; Thevenin impedances only
	OPTIONS(1) = 1	one phase open
	OPTIONS(1) = 2	two phases open
	OPTIONS(1) = 3	in-line fault
	OPTIONS(1) = 4	one breaker open
	OPTIONS(1) = 5	no unbalance
	OPTIONS(2)	path to ground code (used when OPTIONS(1) is 1 or 4) (0 by default)
	OPTIONS(2) = 0	no path to ground
	OPTIONS(2) = 1	include a path to ground
	OPTIONS(3)	type of in-line fault code (used when OPTIONS(1) is 3) (1 by default)
	OPTIONS(3) = 1	line-to-ground
	OPTIONS(3) = 2	line-to-line-to-ground
	OPTIONS(3) = 3	three-phase
	OPTIONS(4)	open end code (used when OPTIONS(1) is 4) (1 by default)
	OPTIONS(4) = 0	breaker at to bus is open
	OPTIONS(4) = 1	breaker at from bus is open
Real VALUES (3)	array of 3 elements specifying unbalance parameters(input).	
	VALUES(1)	fault location as fraction of line from from bus; 0.0 < VALUES(1) < 1.0 (used when OP-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		TIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3) (0.5 by default)
	VALUES(2)	& (3) Rl-g and Xl-g (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3 and OPTIONS(3) is 1 or 2) (0.0 by default)
	VALUES(3)	& (5) Rl-l and Xl-l (used when OPTIONS(1) is 3 and OPTIONS(3) is 2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid with two phase modeling option enabled
	IERR = 2	bus not found
	IERR = 3	invalid OPTIONS value
	IERR = 4	invalid VALUES value
	IERR = 5	SEQD temporary file is not open
	IERR = 6	SEQD results are not compatible with the working case
	IERR = 7	terminated due to invalid fault specification or application
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.288. SPIL

Use this API to calculate transmission interchange limits using a linear network (dc) model.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SPIL OPTIONS(1)..OPTIONS(16) VALUES(1)..VALUES(7) LABELS(1)..LABELS(8)
 DFXFILE PRTFILE
```

Python command syntax:

```
ierr = spil(options, values, labels, dfxfile, prtfile)
```

Fortran command syntax:

```
CALL SPILAPI(OPTIONS, VALUES, LABELS, DFXFILE, PRTFILE, IERR)
```

Where:

Integer OPTIONS (16) array of 16 elements specifying calculation and reporting options(input).

OPTIONS(1)	base case rating set (rating set program option setting by default)
OPTIONS(1) = 1	RATEA
OPTIONS(1) = 2	RATEB
OPTIONS(1) = 3	RATEC
OPTIONS(2)	contingency case rating set (rating set program option setting by default)
OPTIONS(2) = 1	RATEA
OPTIONS(2) = 2	RATEB
OPTIONS(2) = 3	RATEC
OPTIONS(3)	base case line flow code (0 by default)
OPTIONS(3) = 0	dc base case
OPTIONS(3) = 1	ac base case
OPTIONS(4)	phase shifter code (0 by default)
OPTIONS(4) = 0	locked in base case shift solutions
OPTIONS(4) = 1	regulating in base case shift solutions
OPTIONS(5)	list study system buses (0 by default)
OPTIONS(5) = 0	no
OPTIONS(5) = 1	yes
OPTIONS(6)	list opposing system buses (0 by default)
OPTIONS(6) = 0	no
OPTIONS(6) = 1	yes
OPTIONS(7)	list study system tie lines (0 by default)
OPTIONS(7) = 0	no
OPTIONS(7) = 1	yes
OPTIONS(8)	add study system tie lines to monitored line list (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(8) = 0	no
	OPTIONS(8) = 1	yes
	OPTIONS(9)	format code (0 by default)
	OPTIONS(9) = 0	summary output
	OPTIONS(9) = 1	full output
	OPTIONS(10)	interchange limit code (0 by default)
	OPTIONS(10) = 0	incremental transfer capability
	OPTIONS(10) = 1	total transfer capability
	OPTIONS(11)	apply summary minimum distribution factor, VALUES(4), to solution reports (0 by default)
	OPTIONS(11) = 0	no
	OPTIONS(11) = 1	yes
	OPTIONS(12)	convert MVA ratings to estimated MW ratings (0 by default)
	OPTIONS(12) = 0	no
	OPTIONS(12) = 1	yes
	OPTIONS(13)	code to discontinue activity if MW mismatch exceeds VALUES(1) (0 by default)
	OPTIONS(13) = 0	discontinue
	OPTIONS(13) = 1	continue
	OPTIONS(14)	summary table contingency descriptions (1 by default)
	OPTIONS(14) = 0	contingency labels
	OPTIONS(14) = 1	contingency events
	OPTIONS(14) = 2	both contingency labels and events
	OPTIONS(15)	number of elements to include in flow tables (60000 by default)
	OPTIONS(16)	summary table maximum times for reporting the same element (5 by default)
Real VALUES (7)	array of 7 elements specifying tolerances and thresholds(input).	
	VALUES(1)	required MW mismatch tolerance for continuing (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating for reporting an element as overloaded (100.0 by default)
	VALUES(3)	summary table maximum import or export (>0.) (99999.0 by default)
	VALUES(4)	summary table minimum distribution factor magnitude (0.0 by default)
	VALUES(5)	minimum contingency case pre-shift flow change (0.0 by default)
	VALUES(6)	minimum contingency case distribution factor change (0.0 by default)
	VALUES(7)	study system generation shift (100.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*32 LABELS (8)	array of 8 elements specifying subsystem and interface labels(input).	
	LABELS(1)	study system label (no default allowed)
	LABELS(2)	opposing system label (no default allowed)
	LABELS(3)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(4)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(5)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(6)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(7)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(8)	interface label for optional interface limits summary table (blank for none) (blank by default)
Character*260 DFXFILE	name of DFAX data file (input; no default allowed).	
Character*260 PRTFILE	name of subsystem participation data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	no DFAX data file specified
	IERR = 4	same label specified for study and opposing systems
	IERR = 5	generators are converted
	IERR = 6	buses in island(s) without a swing bus; use activity TREE
	IERR = 7	largest mismatch exceeds mismatch tolerance
	IERR = 8	error opening file DFXFILE
	IERR = 9	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 10	file DFXFILE does not contain distribution factors; run DFAX with distribution factor option enabled
	IERR = 11	no monitored branches or interfaces defined
	IERR = 12	no subsystems defined
	IERR = 13	subsystem not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 14	specified subsystems overlap
IERR = 15	no in-service buses in subsystem WORLD
IERR = 16	no tie lines from study system
IERR = 17	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.289. SPLT

Use this API to add a bus to the working case and place a zero impedance line between bus BUS and the new bus. No other equipment is connected to the new bus by this API.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SPLT  BUS  NEWNUM  'NEWNAM'  NEWKV
```

Python command syntax:

```
ierr = splt(bus, newnum, newnam, newkv)
```

Fortran command syntax:

```
CALL SPLTAPI(BUS, NEWNUM, NEWNAM, NEWKV, IERR)
```

Where:

Integer BUS	number of the bus to be split (input; no default allowed).	
Integer NEWNUM	number to be assigned to the new bus (input; no default allowed).	
Character*12 NEWNAM	name to be assigned to the new bus (input; blank by default).	
Real NEWKV	base voltage to be assigned to the new bus (input; base voltage of bus BUS).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	invalid NEWNUM value
	IERR = 3	bus number is already assigned to another bus
	IERR = 4	invalid NEWKV value
	IERR = 5	bus, branch, or branch ownership tables are full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.290. SQLI

Use this API to tabulate the sequence data in a form suitable for problem data documentation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SQLI SID ALL OPT
```

Python command syntax:

```
ierr = sqli(sid, all, opt)
```

Fortran command syntax:

```
CALL SQLINW(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	category of sequence data to be tabulated (input; 1 by default).	
	OPT = 1	full listing
	OPT = 2	bus load data
	OPT = 3	generator data
	OPT = 4	branch data
	OPT = 5	two-winding positive sequence data
	OPT = 6	mutual coupling data
	OPT = 7	switched shunt data
	OPT = 8	three-winding general data
	OPT = 9	line shunt data
	OPT = 10	fixed shunt data
	OPT = 11	induction machine data
	OPT = 12	two-winding zero sequence data
	OPT = 13	three-winding positive sequence data
	OPT = 14	three-winding zero sequence data
	OPT = 15	non-conventional source machine data
	OPT = 16	non-conventional source table data
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	invalid OPT value
IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.291. SRAIND

Use this API to run the substation reliability analysis. It applies failure modes and affects analysis of arbitrary bus-breaker configurations and measures the impact of substation equipment failures upon the power transfer capability of the substation.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SRAIND OPTIONS(1)..OPTIONS(3) VALUES(1)..VALUES(5)
 LABELS(1)..LABELS(3) DFXFILE PRBFILE
```

Python command syntax:

```
ierr = sraind(options, values, labels, dfxfile, prbfile)
```

Fortran command syntax:

```
CALL SRAINDAPI(OPTIONS, VALUES, LABELS, DFXFILE, PRBFILE, IERR)
```

Where:

Integer OPTIONS (3)	array of 3 elements specifying options(input).	
	OPTIONS(1)	report format (1 by default)
	OPTIONS(1) = 1	contingency report
	OPTIONS(1) = 2	bus load curtailment report
	OPTIONS(1) = 3	substation load curtailment report
	OPTIONS(2)	rating set (rating set program option setting by default)
	OPTIONS(2) = 1	RATEA
	OPTIONS(2) = 2	RATEB
	OPTIONS(2) = 3	RATEC
	OPTIONS(3)	include system ties (0 by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
Real VALUES (5)	array of 5 elements specifying substation reliability assessment parameters(input).	
	VALUES(1)	percent of flow rating (>=0, 100.0 by default)
	VALUES(2)	frequency cutoff (>=0, 0.0001 by default)
	VALUES(3)	percent of peak load (>=0, 100.0 by default)
	VALUES(4)	switching time in hours (>=0, 0.0 by default)
	VALUES(5)	breaker failure rate in hours per year (>=0, 0.0 by default)
Character*32 LABELS (3)	array of 3 elements specifying subsystem labels(input).	
	LABELS(1)	label of substation subsystem (no default is allowed)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LABELS(2)	label of source subsystem (no default is allowed)
	LABELS(3)	label of sink subsystem (no default is allowed)
Character*260 DFXFILE	name of DFAX data file (input; no default allowed).	
Character*260 PRBFILE	name of outage statistic data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	substation/source/sink subsytem not specified
	IERR = 4	no DFAX data file specified
	IERR = 5	no outage Statistic Data File specified
	IERR = 6	file is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 7	subsystem not found in DFAX data file
	IERR = 8	source/sink buses not in substation subsystem
	IERR = 9	source and sink systems overlap
	IERR = 51	error opening DFAX data file
	IERR = 52	error opening outage statistic data file
	IERR = 53	file system errors

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.292. STOP

This API routine is obsolete. It has been replaced by the API routine stop_2 .

Use this API to end the operation of PSSE.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_STOP DELETE
```

Python command syntax:

```
ierr = stop(delete)
```

Fortran command syntax:

```
CALL STOPAPI(DELETE, IERR)
```

Where:

Integer DELETE	delete working files (input; 0 by default).	
	DELETE = 1	delete working files
	DELETE != 1	keep working files
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.293. STOP_2

Use this API to end the operation of PSSE.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_STOP_2
```

Python command syntax:

```
ierr = stop_2()
```

Fortran command syntax:

```
CALL STOPAPI_2(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.294. SUBS

Use this API to summarize conditions in the working case by tabulating the conditions at each swing system bus, conditions at each area slack area bus, number of components, generation/ load/shunt totals, and loss/line shunt/charging totals by voltage levels.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SUBS SID ALL
```

Python command syntax:

```
ierr = subs(sid, all)
```

Fortran command syntax:

```
CALL SUBSNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.295. SWD_RATING_DEFAULT

Use this API to specify or return the current default swditching device rating set.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SWD_RATING_DEFAULT IVAL
```

Python command syntax:

```
ierr = swd_rating_default(ival)

ierr, ival = swd_rating_default()
```

Fortran command syntax:

```
CALL SWD_RATING_DEFAULT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the current default switching device rating name, from 1 to 12 (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 1 by default; else output).	
Integer IOCODE	indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL
	IOCODE = 1	return the current value of the option setting in IVAL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.296. SWITCHED_SHUNT_ADJUSTMENT

Use this API to specify or return the option to enable or disable switched shunt adjustment.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_ADJUSTMENT IVAL
```

Python command syntax:

```
ierr = switched_shunt_adjustment(ival)

ierr, ival = switched_shunt_adjustment()
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_ADJUSTMENT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	disabled.
	IVAL = 1	enabled.
	IVAL = 2	continuous mode switched shunts enabled, discrete mode switched shunts locked.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.297. TAP_ADJUSTMENT

Use this API to specify or return the option for the default tap adjustment setting; either disabled, stepping or direct.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_TAP_ADJUSTMENT IVAL
```

Python command syntax:

```
ierr = tap_adjustment(ival)

ierr, ival = tap_adjustment()
```

Fortran command syntax:

```
CALL TAP_ADJUSTMENT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	locked (i.e., tap adjustment is disabled).
	IVAL = 1	tap adjustment using the stepping method enabled.
	IVAL = 2	tap adjustment using the direct method enabled.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.298. TEXT

This API does nothing. Its purpose is to provide recordable documentation.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_TEXT 'CARG'
```

Python command syntax:

```
ierr = text(carg)
```

Fortran command syntax:

```
CALL TEXT_API(CARG, IERR)
```

Where:

Character*220 CARG	string of text (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.299. TFLG

Use this API to set or reset the adjustment control mode flags for all automatically adjustable transformers contained in the specified subsystem.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TFLG  SID  ALL  OPT
```

Python command syntax:

```
ierr = tflg(sid, all, opt)
```

Fortran command syntax:

```
CALL TFLGNW(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	option to select the adjustment control mode flags of those transformers being processed (input; 1 by default).	
	OPT = 0	disable
	OPT = 1	enable
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.300. TIES

Use this API to tabulate the flows on all area tie lines, with tie flows grouped by area.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TIES SID ALL
```

Python command syntax:

```
ierr = ties(sid, all)
```

Fortran command syntax:

```
CALL TIESNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all areas or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only areas in area subsystem SID
	ALL = 1	process all areas
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.301. TIEZ

Use this API to tabulate the flows on all zone tie lines, with tie flows grouped by zone.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TIEZ SID ALL
```

Python command syntax:

```
ierr = tiez(sid, all)
```

Fortran command syntax:

```
CALL TIEZNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Zone subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all zones or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only zones in zone subsystem SID
	ALL = 1	process all zones
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.302. TIME

Use this API to tabulate timing statistics.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_TIME INIT
```

Python command syntax:

```
ierr = time(init)
```

Fortran command syntax:

```
CALL TIMEAPI(INIT, IERR)
```

Where:

Integer INIT	timing flag (input; positive).	
	INIT <= 0	do not initialize
	INIT > 0	initialize
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.303. TLST

Use this API to tabulate those transformers in the working case where off-nominal turns ratio or phase shift angle may be adjusted by the power flow solution activities.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TLST SID ALL OPT VREV
```

Python command syntax:

```
ierr = tlst(sid, all, opt, vrev)
```

Fortran command syntax:

```
CALL TLSTNW(SID, ALL, OPT, VREV, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	option to select the transformers tabulated (input; 0 by default).	
	OPT = 0	all
	OPT = 1	violations only
Integer VREV	option to change voltage units from the default units to the alternate units (input; 0 by default).	
	VREV = 0	use default voltage units
	VREV = 1	use alternate voltage units
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	invalid VREV value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.304. TLTG

Use this API to calculate transmission interchange limits using a linear network (dc) model.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_TLTG OPTIONS(1)..OPTIONS(17) VALUES(1)..VALUES(7) LABELS(1)..LABELS(8)
 DFXFILE
```

Python command syntax:

```
ierr = tltg(options, values, labels, dfxfile)
```

Fortran command syntax:

```
CALL TLTGAPI(OPTIONS, VALUES, LABELS, DFXFILE, IERR)
```

Where:

Integer OPTIONS (17) array of 17 elements specifying calculation and reporting options(input).

OPTIONS(1)	base case rating set (rating set program option setting by default)
OPTIONS(1) = 1	RATEA
OPTIONS(1) = 2	RATEB
OPTIONS(1) = 3	RATEC
OPTIONS(2)	contingency case rating set (rating set program option setting by default)
OPTIONS(2) = 1	RATEA
OPTIONS(2) = 2	RATEB
OPTIONS(2) = 3	RATEC
OPTIONS(3)	base case line flow code (0 by default)
OPTIONS(3) = 0	dc base case
OPTIONS(3) = 1	ac base case
OPTIONS(4)	phase shifter code (0 by default)
OPTIONS(4) = 0	locked in base case shift solutions
OPTIONS(4) = 1	regulating in base case shift solutions
OPTIONS(5)	code for the treatment of base case line flow constraints in contingency cases (0 by default)
OPTIONS(5) = 0	ignore
OPTIONS(5) = 1	include
OPTIONS(6)	list study system buses (0 by default)
OPTIONS(6) = 0	no
OPTIONS(6) = 1	yes
OPTIONS(7)	list opposing system buses (0 by default)
OPTIONS(7) = 0	no
OPTIONS(7) = 1	yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(8)	list study system tie lines (0 by default)
	OPTIONS(8) = 0	no
	OPTIONS(8) = 1	yes
	OPTIONS(9)	add study system tie lines to monitored line list (0 by default)
	OPTIONS(9) = 0	no
	OPTIONS(9) = 1	yes
	OPTIONS(10)	format code (0 by default)
	OPTIONS(10) = 0	summary output
	OPTIONS(10) = 1	full output
	OPTIONS(11)	interchange limit code (0 by default)
	OPTIONS(11) = 0	incremental transfer capability
	OPTIONS(11) = 1	total transfer capability
	OPTIONS(12)	apply summary minimum distribution factor, VALUES(4), to solution reports (0 by default)
	OPTIONS(12) = 0	no
	OPTIONS(12) = 1	yes
	OPTIONS(13)	convert MVA ratings to estimated MW ratings (0 by default)
	OPTIONS(13) = 0	no
	OPTIONS(13) = 1	yes
	OPTIONS(14)	code to discontinue activity if MW mismatch exceeds VALUES(1) (0 by default)
	OPTIONS(14) = 0	discontinue
	OPTIONS(14) = 1	continue
	OPTIONS(15)	summary table contingency descriptions (1 by default)
	OPTIONS(15) = 0	contingency labels
	OPTIONS(15) = 1	contingency events
	OPTIONS(15) = 2	both contingency labels and events
	OPTIONS(16)	number of elements to include in flow tables (60000 by default)
	OPTIONS(17)	summary table maximum times for reporting the same element (5 by default)
Real VALUES (7)	array of 7 elements specifying tolerances and thresholds(input).	
	VALUES(1)	required MW mismatch tolerance for continuing (Newton solution convergence tolerance, TOLN by default)
	VALUES(2)	percent of rating for reporting an element as overloaded (100.0 by default)
	VALUES(3)	summary table maximum import or export (>0.) (99999.0 by default)
	VALUES(4)	summary table minimum distribution factor magnitude (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(5)	minimum contingency case pre-shift flow change (0.0 by default)
	VALUES(6)	minimum contingency case distribution factor change (0.0 by default)
	VALUES(7)	study system generation shift (100.0 by default)
Character*32 LABELS (8)	array of 8 elements specifying subsystem and interface labels(input).	
	LABELS(1)	study system label (no default allowed)
	LABELS(2)	opposing system label (no default allowed)
	LABELS(3)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(4)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(5)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(6)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(7)	interface label for optional interface limits summary table (blank for none) (blank by default)
	LABELS(8)	interface label for optional interface limits summary table (blank for none) (blank by default)
Character*260 DFXFILE	name of DFAX data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	no DFAX data file specified
	IERR = 4	same label specified for study and opposing systems
	IERR = 5	generators are converted
	IERR = 6	buses in island(s) without a swing bus; use activity TREE
	IERR = 7	largest mismatch exceeds mismatch tolerance
	IERR = 8	error opening file DFXFILE
	IERR = 9	file DFXFILE is not in the form of a PSSE-25 or later DFAX file; run DFAX
	IERR = 10	file DFXFILE does not contain distribution factors; run DFAX with distribution factor option enabled

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 11	no monitored branches or interfaces defined
IERR = 12	no subsystems defined
IERR = 13	subsystem not found
IERR = 14	specified subsystems overlap
IERR = 15	no in-service buses in subsystem WORLD
IERR = 16	no tie lines from study system
IERR = 17	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.305. TPCH

Use this API to check the adjustment data associated with voltage or flow controlling transformers. It must be referenced using one of the following procedures:
- One reference with APIOPT=1, followed by any number of either APIOPT=2 references and/or pairs of references with APIOPT=2 and then 3, followed by one reference with APIOPT=4.
- One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference, followed by one APIOPT 3 reference if NTRN from the APIOPT 2 reference is greater than zero, followed by one APIOPT 4 reference.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TPCH  SID  ALL  APIOPT  OPT  STATUS(1)  STATUS(2)  THRSH(1)  THRSH(2)
```

Python command syntax:

```
ierr, ttyp, ntrn = tpch(sid, all, apiopt, opt, status, thrsh)
```

Fortran command syntax:

```
CALL TPCHNW(SID, ALL, APIOPT, OPT, STATUS, THRSH, TTYP, NTRN, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; used only when APIOPT is 0 or 1; 0).	
Integer ALL	all buses or specified subsystem flag (input; used only when APIOPT is 0 or 1; 1).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer APIOPT	mode of operation (input; 0 by default).	
	APIOPT = 0	initialize, check, modify adjustment data, and terminate
	APIOPT = 1	initialize for check and data modification
	APIOPT = 2	check transformers
	APIOPT = 3	modify adjustment data
	APIOPT = 4	terminate transformer checks
Integer OPT	option to select the type of adjustment (input; used only when APIOPT is 0 or 2; 1).	
	OPT = 1	tap step = 0. (V/Mvar)
	OPT = 2	0. < step < minimum (V/ Mvar)
	OPT = 3	tap step > maximum (V/ Mvar)
	OPT = 4	voltage band < 2*step (V)
	OPT = 5	voltage band < minimum (V)
	OPT = 6	voltage band > maximum (V)
	OPT = 8	flow band < minimum (MW/Mvar)
	OPT = 9	flow band > maximum (MW/Mvar)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer STATUS (2)	array of 2 elements specifying data modification options (used only when APIOPT is 0 or 3)(input).
	STATUS(1) modify steps or voltage bands, when OPT = 4 (1 by default)
	STATUS(1) = 1 modify steps
	STATUS(1) = 2 modify voltage bands
	STATUS(2) used to indicate which transformers to change where OPTs 1, 2, 3, 8 or 9 (0 by default)
	STATUS(2) = 0 change both types
	STATUS(2) = 1 change voltage controlling transformers when OPT = 1, 2 or 3; change MW controlling transformers when OPT = 8 or 9
	STATUS(2) = 2 change Mvar controlling transformers
Real THRSH (2)	array of 2 elements specifying data checking and modification parameters(input).
	THRSH(1) threshold to use in initial test (used only when APIOPT is 0 or 2; defaults: 0.00625 when OPT is 2 or 3; 0.02 when OPT is 5 or 6; 5.0 when OPT is 8 or 9)
	THRSH(2) new tap step, voltage band, or flow band (used only when APIOPT is 0 or 3)
	If OPT
	THRSH(2) = 1 new tap step (0.00625 by default)""
	THRSH(2) = 2,3 new tap step (THRSH(1) by default)""
	THRSH(2) = 5,6 new voltage band (THRSH(1) by default)""
	THRSH(2) = 8,9 new flow band (THRSH(1) by default)""
Logical TTYP	returned when APIOPT = 0, 2 or 3 and is TRUE. if both voltage (or MW if OPT = 8 or 9) and Mvar controlling transformers were found (output).
Integer NTRN	returned when APIOPT = 0, 2 or 3 and is the number of transformers that need adjustment when applying the adjustment type indicated by OPT (output).
Integer IERR	error code (output).
	IERR = 0 no error occurred
	IERR = 1 invalid SID value or subsystem SID is not defined
	IERR = 2 invalid ALL value
	IERR = 3 invalid APIOPT value
	IERR = 4 invalid OPT value
	IERR = 5 invalid STATUS value
	IERR = 6 invalid THRSH value
	IERR = 7 unexpected APIOPT value
	IERR = 8 no controlling transformers in case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.306. TRANSFORMER_PERCENT_UNITS

Use this API to specify or return the option to set the transformer percent units to either MVA or current expressed as MVA.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_TRANSFORMER_PERCENT_UNITS IVAL
```

Python command syntax:

```
ierr = transformer_percent_units(ival)

ierr, ival = transformer_percent_units()
```

Fortran command syntax:

```
CALL TRANSFORMER_PERCENT_UNITS_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	MVA.
	IVAL = 1	current expressed as MVA.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.307. TRANSMISSION_LINE_UNITS

Use this API to to specify or return the option set the transmission line units to either per unit or ohms.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_TRANSMISSION_LINE_UNITS IVAL
```

Python command syntax:

```
ierr = transmission_line_units(ival)

ierr, ival = transmission_line_units()
```

Fortran command syntax:

```
CALL TRANSMISSION_LINE_UNITS_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	per unit.
	IVAL = 1	ohms and microfarads.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.308. TREE

Use this API to check for the existence of in-service ac islands that do not contain a Type 3 (swing) bus. Following each successful call, it returns BUSES as the number of buses in a swingless island (0 for no more swingless islands). When a BUSES value of 0 is returned, no further calls are needed.
The API must be called once with APIOPT set to 1. If BUSES is returned as 0 (i.e., there are no swingless islands), no further calls are needed. Otherwise, if BUSES is greater than zero, it must be called one or more times with APIOPT set to 2 and OPTION set to indicate the disposition of the current swingless island. APIOPT 2 calls are required until either BUSES is returned as zero or an APIOPT 2 call is made with OPTION set to a negative value.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_TREE APIOPT OPTION
```

Python command syntax:

```
ierr, buses = tree(apiopt, option)
```

Fortran command syntax:

```
CALL TREEAPI(APIOPT, OPTION, BUSES, IERR)
```

Where:

Integer APIOPT	mode of operation (input; no default allowed).	
	APIOPT = 1	initialize and check for the presence of a swingless island
	APIOPT = 2	process previously detected island as dictated by OPTION; then check for the presence of another swingless island
Integer OPTION	option for the handling of previously detected swingless island (input; used only when APIOPT is 2; -1).	
	OPTION < 0	leave this island alone and terminate activity TREE
	OPTION = 0	leave this island alone and check for another swingless island
	OPTION > 0	disconnect this island, then check for another swingless island
Integer BUSES	returned as the number of buses in this swingless island; 0 if no more swingless islands (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid APIOPT value
	IERR = 2	unexpected APIOPT value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.309. TYSL

Use this API to run switching study network solutions.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TYSL  OPT
```

Python command syntax:

```
ierr = tysl(opt)
```

Fortran command syntax:

```
CALL TYSLAPI(OPT, IERR)
```

Where:

Integer OPT	flat start flag (input; 0 by default).	
	OPT = 0	use present voltage vector as starting point
	OPT = 1	flat start
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPT value
	IERR = 2	generators are not converted
	IERR = 3	network is not ordered
	IERR = 4	error reading the admittance matrix temporary file; one of the following occurred: Admittance matrix temporary file is not open; Reading matrix pointers: different sizes specified than were used in writing them; Reading matrix terms: different sizes specified than were used in writing them
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.310. USER

Use this API to run the user-written activity, subroutine USERAC.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_USER
```

Python command syntax:

```
ierr = user()
```

Fortran command syntax:

```
CALL USERAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.311. VAMM

Use this API to tabulate the vector absolute MW-mile report.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_VAMM
```

Python command syntax:

```
ierr = vamm()
```

Fortran command syntax:

```
CALL VAMMAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no transactions defined
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.312. VAMO

Use this API to tabulate the vector absolute MW-ohm report.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_VAMO
```

Python command syntax:

```
ierr = vamo()
```

Fortran command syntax:

```
CALL VAMOAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no transactions defined
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.313. VCHK

Use this API to tabulate those buses where voltage magnitude is outside a specified range.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_VCHK SID ALL VLO VHI
```

Python command syntax:

```
ierr = vchk(sid, all, vlo, vhi)
```

Fortran command syntax:

```
CALL VCHKNW(SID, ALL, VLO, VHI, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Real VLO	lower limit of the voltage band (input; 0.95).	
Real VHI	upper limit of the voltage band (input; 1.05).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.314. VOLTAGE_INPUT

Use this API to specify or return the voltage input option setting; either per unit or kV.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_VOLTAGE_INPUT IVAL
```

Python command syntax:

```
ierr = voltage_input(ival)

ierr, ival = voltage_input()
```

Fortran command syntax:

```
CALL VOLTAGE_INPUT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	per unit.
	IVAL = 1	kV.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.315. VOLTAGE_OUTPUT

Use this API to specify or return the voltage output option setting; either per unit or kV.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_VOLTAGE_OUTPUT IVAL
```

Python command syntax:

```
ierr = voltage_output(ival)

ierr, ival = voltage_output()
```

Fortran command syntax:

```
CALL VOLTAGE_OUTPUT_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	per unit.
	IVAL = 1	kV.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.316. VSMO

Use this API to tabulate the vector MW-ohm report.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_VSMO
```

Python command syntax:

```
ierr = vsmo()
```

Fortran command syntax:

```
CALL VSMOAPI(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	no transactions defined
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.317. WRITECAPCURVES

Use this API to write the capability curve data in the working case in the form of a Capability Curve Raw Data File.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_WRITECAPCURVES SID ALL STATUS(1) STATUS(2) OUT OFILE
```

Python command syntax:

```
ierr = writecapcurves(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL WRITECAPCURVES(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all machines or those in the specified subsystem (input; 1 by default).	
	ALL = 0	process only machines in subsystem SID
	ALL = 1	process all machines
Integer STATUS (2)	array of 2 elements specifying options(input).	
	STATUS(1)	include records for out-of-service machines (1 by default)
	STATUS(1) = 0	no
	STATUS(1) = 1	yes
	STATUS(2)	'IC" value on the first record (0 by default)
	STATUS(2) = 0	for API routine ReadCapCurves with IC=0
	STATUS(2) = 1	for API routine ReadCapCurves with IC=1
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid OUT value
	IERR = 5	capability curve data is not in the working case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	error opening OFILE
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.318. WRITERAWVERSION

Use this API to replicate the working case in the form of power flow raw data file compatible with PSSE 15 or later.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_WRITERAWVERSION 'VERNUM' OUT OFILE
```

Python command syntax:

```
ierr = writerawversion(vernum, out, ofile)
```

Fortran command syntax:

```
CALL WRITERAWVERSION(VERNUM, OUT, OFILE, IERR)
```

Where:

Integer or Character*14 VERNUM	value of OFILE version number VERNUM ='15' or later. Example: If OFILE format is to be PSSE 29.5.1: VERNUM='29' or ='29.5' or ='29.5.1' (input; no default allowed).	
Integer OUT	output destination (input; 1 by default).	
	OUT = 0	to OFILE
	OUT = 1	to current report device
Character*260 OFILE	name of the power flow raw data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VERNUM value
	IERR = 2	invalid OUT value
	IERR = 3	memory allocation Error
	IERR = 4	error reading data
	IERR = 5	error saving temporary 'sav' file
	IERR = 6	error opening temporary 'sav' file
	IERR = 7	error deleting temporary 'sav' file
	IERR = 8	error opening OFILE
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.319. WRITERAWX

Use this API to write an extended raw data file from data in the current working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_WRITERAWX SFILE DOMAINS
```

Python command syntax:

```
ierr = writerawx(sfile, domains)
```

Fortran command syntax:

```
CALL WRITERAWX(SFILE, DOMAINS, IERR)
```

Where:

Character*260 SFILE	filename of the extended power flow raw data file (input; no default allowed).	
Character*512 DOMAINS	comma separated string of RAWX domains to write to SFILE (input; all domains).	
	DOMAINS = all	all domains
	DOMAINS = network	network data
	DOMAINS = time-series	time series power flow data
	DOMAINS = harmonics	harmonics data
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 2	rawx write error
	IERR = 3	error opening SFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.320. WRITERAWXSUBSYS

Use this API to write an extended raw data file from data in the current working case, specifying a subsystem.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_WRITERAWXSUBSYS SID ALL STATUS SFILE DOMAINS
```

Python command syntax:

```
ierr = writerawxsubsys(sid, all, status, sfile, domains)
```

Fortran command syntax:

```
CALL WRITERAWXSUBSYS(SID, ALL, STATUS, SFILE, DOMAINS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses when STATUS(4) = 0; area ties only when STATUS(4) = 1
Integer STATUS (5)	array of 5 elements specifying options(input).	
	STATUS(1)	include records for Type 4 buses (1 by default)
	STATUS(1) = 0	no
	STATUS(1) = 1	yes
	STATUS(2)	include records for out-of-service branches (1 by default)
	STATUS(2) = 0	no
	STATUS(2) = 1	yes
	STATUS(3)	include records for equipment in the subsystem (1 by default)
	STATUS(3) = 0	no
	STATUS(3) = 1	yes
	STATUS(4)	include records for subsystem ties (0 by default)
	STATUS(4) = 0	no
	STATUS(4) = 1	yes
	STATUS(5)	include load record option (honored when ALL=0) (0 by default)
	STATUS(5) = 0	for all loads at subsystem buses
	STATUS(5) = 1	for subsystem loads at all buses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 2	for all loads at subsystem buses and subsystem loads at non-subsystem buses
Character*260 SFILE	filename of the extended power flow raw data file (input; no default allowed).	
Character*512 DOMAINS	comma separated string of RAWX domains to write to SFILE (input; all domains).	
	DOMAINS = all	all domains
	DOMAINS = network	network data
	DOMAINS = time-series	time series power flow data
	DOMAINS = harmonics	harmonics data
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 2	rawx write error
	IERR = 3	error opening SFILE
	IERR = 4	prerequisite requirements for API are not met
	IERR = 5	invalid SID value or subsystem SID is not defined
	IERR = 6	invalid ALL value
	IERR = 7	invalid STATUS value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.321. WRITESEQVERSION

This API routine is obsolete. It has been replaced by the API routine writeseqversion_2 .

Use this API to replicate the working case in the form of a network sequence data file compatible with PSSE 27 or later.

This API routine was first introduced in release 32.0.4.

Batch command syntax:

```
BAT_WRITESEQVERSION 'VERNUM' OUT OFILE
```

Python command syntax:

```
ierr = writeseqversion(vernum, out, ofile)
```

Fortran command syntax:

```
CALL WRITESEQVERSION(VERNUM, OUT, OFILE, IERR)
```

Where:

Integer or Character*14 VERNUM	value of OFILE version number VERNUM ='27' or later. Example: If OFILE format is to be PSSE 29.5.1: VERNUM='29' or ='29.5' or ='29.5.1' (input; no default allowed).	
Integer OUT	output destination (input; 1 by default).	
	OUT = 0	to OFILE
	OUT = 1	to current report device
Character*260 OFILE	name of the network sequence data file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VERNUM value
	IERR = 2	invalid OUT value
	IERR = 3	error opening OFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.322. WRITESEQVERSION_2

Use this API to replicate the working case in the form of a network sequence data file compatible with PSSE 27 or later.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_WRITESEQVERSION_2 'VERNUM' OUT OFILE GZOPTN
```

Python command syntax:

```
ierr = writeseqversion_2(vernum, out, ofile, gzoptn)
```

Fortran command syntax:

```
CALL WRITESEQVERSION_2(VERNUM, OUT, OFILE, GZOPTN, IERR)
```

Where:

Integer or Character*14 VERNUM	value of OFILE version number VERNUM ='27' or later. Example: If OFILE format is to be PSSE 29.5.1: VERNUM='29' or ='29.5' or ='29.5.1' (input; no default allowed).	
Integer OUT	output destination (input; 1 by default).	
	OUT = 0	to OFILE
	OUT = 1	to current report device
Character*260 OFILE	name of the network sequence data file (input; no default allowed).	
Integer GZOPTN	generator reactance option (input; 0 by default).	
	GZOPTN = 0	save subtransient reactance
	GZOPTN = 1	save transient reactance
	GZOPTN = 2	save synchronous reactance
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VERNUM value
	IERR = 2	invalid OUT value
	IERR = 3	error opening OFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.323. WRITE_OPTIONS_FILE

Use this API to save the present values of the PSSE program option settings to the file psse.opt.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_WRITE_OPTIONS_FILE
```

Python command syntax:

```
ierr = write_options_file()
```

Fortran command syntax:

```
CALL WRITE_OPTIONS_FILE_API(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.324. XEQV

Use this API to build an electrical equivalent of the portion of the working case outside of a specified subsystem of the working case.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_XEQV  SID  ALL  STATUS(1)..STATUS(6)  DVAL(1)  DVAL(2)
```

Python command syntax:

```
ierr = xeqv(sid, all, status, dval)
```

Fortran command syntax:

```
CALL XEQV(SID, ALL, STATUS, DVAL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses outside of subsystem SID
	ALL = 1	process all buses
Integer STATUS (6)	array of 6 elements specifying They are used to control equivalencing options. A value of 0 means the option is not enabled. A value of 1 means the option is enabled. They are as follows(input).	
	STATUS(1)	retain area boundary buses (0 by default)
	STATUS(2)	retain zone boundary buses (0 by default)
	STATUS(3)	suppress the equivalencing option for phase shifters (0 by default)
	STATUS(4)	retain controlled buses (0 by default)
	STATUS(5)	retain existing branches between retained buses (0 by default)
	STATUS(6)	net load and shunt at retained buses (0 by default)
Real DVAL (2)	array of 2 elements specifying They are as follows(input).	
	DVAL(1)	minimum active or reactive power generation for retaining generator buses (0.0 by default)
	DVAL(2)	branch threshold tolerance (10.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	invalid STATUS value
IERR = 4	invalid DVAL value
IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.325. ZONE

This API routine is obsolete. It has been replaced by the API routine zone_2 .

Use this API to tabulate zone totals by zone.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ZONE SID ALL
```

Python command syntax:

```
ierr = zone(sid, all)
```

Fortran command syntax:

```
CALL ZONENW(SID, ALL, IERR)
```

Where:

Integer SID	a valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Zone subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all zones or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only zones in zone subsystem SID
	ALL = 1	process all zones
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.326. ZONE_2

Use this API to tabulate zone totals by zone.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_ZONE_2 SID ALL OPT
```

Python command syntax:

```
ierr = zone_2(sid, all, opt)
```

Fortran command syntax:

```
CALL ZONE_2(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Zone subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all zones or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only zones in zone subsystem SID
	ALL = 1	process all zones
Integer OPT	load reporting option (input; 2 by default).	
	OPT = 1	tabulate load connected to zone buses
	OPT = 2	tabulate load assigned to the zone
	OPT = 3	tabulate three load subtotals for each zone: zone load connected to zone buses; zone load connected to non-zone buses; non-zone load connected to zone buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.327. ZONE_AREA

Use this API to tabulate zone totals by zone, along with subtotals by area.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ZONE_AREA SID ALL
```

Python command syntax:

```
ierr = zone_area(sid, all)
```

Fortran command syntax:

```
CALL ZONE_AREA(SID, ALL, IERR)
```

Where:

Integer SID	a valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Zone subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all zones or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only zones in zone subsystem SID
	ALL = 1	process all zones
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.328. ZONM

This API routine is obsolete. It has been replaced by the API routine zonm_2 .

Use this API to reassign the buses and loads in a specified subsystem of the working case from their original zone to a designated zone.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ZONM  SID  ALL  STATUS(1)  STATUS(2)  IZNEW
```

Python command syntax:

```
ierr = zonm(sid, all, status, iznew)
```

Fortran command syntax:

```
CALL ZONMNW(SID, ALL, STATUS, IZNEW, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (2)	array of 2 elements specifying reporting options(input).	
	STATUS(1)	zone assignments of buses to IZNEW (0 by default)
	STATUS(1) = 0	don't change zone assignments of buses to IZNEW
	STATUS(1) = 1	change zone assignments of buses to IZNEW
	STATUS(2)	zone assignments of loads to IZNEW (0 by default)
	STATUS(2) = 0	Don't change zone assignments of loads to IZNEW
	STATUS(2) = 1	Change zone assignments of loads to IZNEW
Integer IZNEW	new zone to be assigned to specified buses and/or loads. IZNEW must be between 1 and 9999 (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	invalid new zone number
IERR = 5	zone table is full; cannot add zone to the working case
IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.1.329. ZONM_2

Use this API routine to reassign the buses, loads and/or induction machines in a specified subsystem of the working case from their original zone to a designated zone.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ZONM_2  SID  ALL  STATUS(1)  STATUS(2)  STATUS(3)  IZNEW
```

Python command syntax:

```
ierr = zonm_2(sid, all, status, iznew)
```

Fortran command syntax:

```
CALL ZONM_2(SID, ALL, STATUS, IZNEW, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (3)	array of 3 elements specifying reporting options(input).	
	STATUS(1)	zone assignments of buses to IZNEW (0 by default)
	STATUS(1) = 0	leave zone assignments of buses unchanged
	STATUS(1) = 1	change zone assignments of buses to IZNEW
	STATUS(2)	zone assignments of loads to IZNEW (0 by default)
	STATUS(2) = 0	leave zone assignments of loads unchanged
	STATUS(2) = 1	change zone assignments of loads to IZNEW
	STATUS(3)	zone assignments of induction machines to IZNEW (0 by default)
	STATUS(3) = 0	leave zone assignments of induction machines unchanged
	STATUS(3) = 1	change zone assignments of induction machines to IZNEW
Integer IZNEW	new zone to be assigned to specified buses, load, and/or induction machines. IZNEW must be between 1 and 9999 (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	invalid STATUS value
IERR = 4	invalid new zone number
IERR = 5	zone table is full; cannot add zone to the working case
IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2. Power Flow Data

This section describes the API routines used to add, modify, or remove power flow data, other than substation data.

For most categories of power flow data, there are a pair of API routines for specifying data; one of these contains "_DATA" as part of the API name, and the other contains "_CHNG" in its API name. The "_DATA" API routines may be used to add a new network element to the working case, or to modify the data of an existing network element. The "_CHNG" API routines may be used only to modify the data of an existing network element; specifying a network element that does not already exist in the working case to one of the "_CHNG" API routines is treated as a fatal error by the API routine.

Generally, when an API is adding a new element to the working case, data items that are defaulted are set to the value listed in the data item's description. When an API is modifying the data of an existing element, defaulted data items are unchanged.

1.2.1. AREA_DATA

Use this API to modify area data in the working case (see PSSE Program Operation Manual, Areas, Zones and Owners).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_AREA_DATA  IAREA   IBUS   REALAR(1)   REALAR(2)   'ARNAME'
```

Python command syntax:

```
ierr = area_data(iarea, ibus, realar, arname)
```

Fortran command syntax:

```
CALL AREA_API_2(IAREA, IBUS, REALAR, ARNAME, IERR)
```

Where:

Integer IAREA	area number between 1 and 9999 (input; no default allowed).	
Integer IBUS	area slack bus number (input; no default allowed).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	PDES, desired MW area interchange (0.0 by default)
	REALAR(2)	PTOL, desired MW interchange tolerance (0.1 * SBASE by default; for example, when system base MVA is 100.0, the default value of PTOL is 10 MW)
Character*12 ARNAME	area name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid area number
IERR = 2	area table is full
IERR = 3	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Area slack bus not found. - Slack bus is not in this area. - No plant at slack bus. - Type code of slack bus is not 2 or 3. - A Type 3 bus is in the area but it is not the area slack bus. - No equipment is assigned to area

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.2. BRANCH_CHNG

This API routine is obsolete. It has been replaced by the API routine branch_chng_3 .

Use this API routine to modify the data of an existing non-transformer branch in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_BRANCH_CHNG  IBUS  JBUS  'CKT'  INTGAR(1)..INTGAR(6)
 REALAR(1)..REALAR(15)
```

Python command syntax:

```
ierr = branch_chng(ibus, jbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL BRANCH_CHNG(IBUS, JBUS, CKT, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	ST, branch status (alias is ST) (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (alias is METBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (alias is O1) (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (alias is O2) (0 by default)
	INTGAR(5)	O3, third owner number (alias is O3) (0 by default)
	INTGAR(6)	O4, fourth owner number (alias is O4) (0 by default)
Real REALAR (15)	array of 15 elements specifying (input).	
	REALAR(1)	R, nominal branch resistance (alias is R) (0.0 by default)
	REALAR(2)	X, nominal branch reactance (alias is X) (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALAR(3)	B, total line charging (alias is B) (0.0 by default)
	REALAR(4)	RATEA, rating set A line rating (alias is RATEA) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(5)	RATEB, rating set B line rating (alias is RATEB) (0.0 by default)
	REALAR(6)	RATEC, rating set C line rating (alias is RATEC) (0.0 by default)
	REALAR(7)	GI, real line shunt at bus IBUS end (alias is GI) (0.0 by default)
	REALAR(8)	BI, reactive line shunt at bus IBUS end (alias is BI) (0.0 by default)
	REALAR(9)	GJ, real line shunt at bus JBUS end (alias is GJ) (0.0 by default)
	REALAR(10)	BJ, reactive line shunt at bus JBUS end (alias is BJ) (0.0 by default)
	REALAR(11)	LEN, line length (alias is LEN) (0.0 by default)
	REALAR(12)	F1, first owner fraction (alias is F1) (1.0 by default)
	REALAR(13)	F2, second owner fraction (alias is F2) (1.0 by default)
	REALAR(14)	F3, third owner fraction (alias is F3) (1.0 by default)
	REALAR(15)	F4, fourth owner fraction (alias is F4) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a two-winding transformer
	IERR = 9	branch not found
	IERR = 10	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - branch reactance is 0.0 - line length is negative - zero impedance line table is full; branch is treated as a normal line - invalid metered end bus - invalid branch status - invalid owner number - invalid ownership fraction - no ownership data is specified - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - branch is no longer treated as a zero impedance line - owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.3. BRANCH_CHNG_3

This API routine is obsolete. It has been replaced by the API routine branch_chng_4 .

Use this API routine to modify the data of an existing non-transformer branch in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_BRANCH_CHNG_3 IBUS JBUS CKT INTGAR(1)..INTGAR(6) REALAR(1)..REALAR(12)
 RATINGS(1)..RATINGS(12) NAMEAR
```

Python command syntax:

```
ierr = branch_chng_3(ibus, jbus, ckt, intgar, realar, ratings, namear)
```

Fortran command syntax:

```
CALL BRANCH_CHNG_3(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	ST, branch status (alias is ST) (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (alias is METBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (alias is O1) (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (alias is O2) (0 by default)
	INTGAR(5)	O3, third owner number (alias is O3) (0 by default)
	INTGAR(6)	O4, fourth owner number (alias is O4) (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	R, nominal branch resistance (alias is R) (0.0 by default)
	REALAR(2)	X, nominal branch reactance (alias is X) (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALAR(3)	B, total line charging (alias is B) (0.0 by default)
	REALAR(4)	GI, real line shunt at bus IBUS end (alias is GI) (0.0 by default)
	REALAR(5)	BI, reactive line shunt at bus IBUS end (alias is BI) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(6)	GJ, real line shunt at bus JBUS end (alias is GJ) (0.0 by default)
	REALAR(7)	BJ, reactive line shunt at bus JBUS end (alias is BJ) (0.0 by default)
	REALAR(8)	LEN, line length (alias is LEN) (0.0 by default)
	REALAR(9)	F1, first owner fraction (alias is F1) (1.0 by default)
	REALAR(10)	F2, second owner fraction (alias is F2) (1.0 by default)
	REALAR(11)	F3, third owner fraction (alias is F3) (1.0 by default)
	REALAR(12)	F4, fourth owner fraction (alias is F4) (1.0 by default)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	branch name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	branch exists but it is a two-winding transformer
IERR = 4	branch from a bus to itself
IERR = 5	blank circuit identifier is invalid
IERR = 6	circuit identifier may not start with &
IERR = 7	branch table is full
IERR = 8	branch ownership table is full
IERR = 9	branch not found
IERR = 10	prerequisite requirements for API are not met
IERR = 11	branch exists but it is a system switching device
IERR = 13	node-breaker terminal connection table is full
IERR = -1	data error, warning and information messages; one or more of: - branch reactance is 0.0 - line length is negative - zero impedance line table is full; branch is treated as a normal line - invalid metered end bus - invalid branch status - invalid owner number - invalid ownership fraction - no ownership data is specified - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - branch is no longer treated as a zero impedance line - owner table is full - rating is negative - branch exists but is a system switching device - name is already assigned to a branch - name contains invalid characters - non-transformer branch connects buses with different base voltages

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.4. BRANCH_CHNG_4

Use this API routine to modify the data of an existing non-transformer branch in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_BRANCH_CHNG_4 IBUS JBUS CKT INTGAR(1)..INTGAR(6) REALAR(1)..REALAR(12)
 RATINGS(1)..RATINGS(12) NAMEAR
```

Python command syntax:

```
ierr = branch_chng_4(ibus, jbus, ckt, intgar, realar, ratings, namear)
```

Fortran command syntax:

```
CALL BRANCH_CHNG_4(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	ST, branch status (alias is ST) (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (alias is METBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (alias is O1) (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (alias is O2) (0 by default)
	INTGAR(5)	O3, third owner number (alias is O3) (0 by default)
	INTGAR(6)	O4, fourth owner number (alias is O4) (0 by default)
	INTGAR(7)	BYPASS, branch bypass flag (alias is BYPASS) (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	R, nominal branch resistance (alias is R) (0.0 by default)
	REALAR(2)	X, nominal branch reactance (alias is X) (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALAR(3)	B, total line charging (alias is B) (0.0 by default)
	REALAR(4)	GI, real line shunt at bus IBUS end (alias is GI) (0.0 by default)
	REALAR(5)	BI, reactive line shunt at bus IBUS end (alias is BI) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(6)	GJ, real line shunt at bus JBUS end (alias is GJ) (0.0 by default)
	REALAR(7)	BJ, reactive line shunt at bus JBUS end (alias is BJ) (0.0 by default)
	REALAR(8)	LEN, line length (alias is LEN) (0.0 by default)
	REALAR(9)	F1, first owner fraction (alias is F1) (1.0 by default)
	REALAR(10)	F2, second owner fraction (alias is F2) (1.0 by default)
	REALAR(11)	F3, third owner fraction (alias is F3) (1.0 by default)
	REALAR(12)	F4, fourth owner fraction (alias is F4) (1.0 by default)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	branch name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	branch exists but it is a two-winding transformer
IERR = 4	branch from a bus to itself
IERR = 5	blank circuit identifier is invalid
IERR = 6	circuit identifier may not start with &
IERR = 7	branch table is full
IERR = 8	branch ownership table is full
IERR = 9	branch not found
IERR = 10	prerequisite requirements for API are not met
IERR = 11	branch exists but it is a system switching device
IERR = 13	node-breaker terminal connection table is full
IERR = -1	data error, warning and information messages; one or more of: - branch reactance is 0.0 - line length is negative - zero impedance line table is full; branch is treated as a normal line - invalid metered end bus - invalid branch status - invalid owner number - invalid ownership fraction - no ownership data is specified - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - branch is no longer treated as a zero impedance line - owner table is full - rating is negative - branch exists but is a system switching device - name is already assigned to a branch - name contains invalid characters - non-transformer branch connects buses with different base voltages

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.5. BRANCH_DATA

This API routine is obsolete. It has been replaced by the API routine branch_data_3 .

Use this API to modify the data of an existing non-transformer branch in the working case or to add a new non-transformer branch to the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BRANCH_DATA  IBUS  JBUS  'CKT'  INTGAR(1)..INTGAR(6)
 REALAR(1)..REALAR(15)
```

Python command syntax:

```
ierr = branch_data(ibus, jbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL BRANCH_API_2(IBUS, JBUS, CKT, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	ST, branch status (alias is ST) (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (alias is METBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (alias is O1) (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (alias is O2) (0 by default)
	INTGAR(5)	O3, third owner number (alias is O3) (0 by default)
	INTGAR(6)	O4, fourth owner number (alias is O4) (0 by default)
Real REALAR (15)	array of 15 elements specifying (input).	
	REALAR(1)	R, nominal branch resistance (alias is R) (0.0 by default)
	REALAR(2)	X, nominal branch reactance (alias is X) (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALAR(3)	B, total line charging (alias is B) (0.0 by default)
	REALAR(4)	RATEA, rating set A line rating (alias is RATEA) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(5)	RATEB, rating set B line rating (alias is RATEB) (0.0 by default)
	REALAR(6)	RATEC, rating set C line rating (alias is RATEC) (0.0 by default)
	REALAR(7)	GI, real line shunt at bus IBUS end (alias is GI) (0.0 by default)
	REALAR(8)	BI, reactive line shunt at bus IBUS end (alias is BI) (0.0 by default)
	REALAR(9)	GJ, real line shunt at bus JBUS end (alias is GJ) (0.0 by default)
	REALAR(10)	BJ, reactive line shunt at bus JBUS end (alias is BJ) (0.0 by default)
	REALAR(11)	LEN, line length (alias is LEN) (0.0 by default)
	REALAR(12)	F1, first owner fraction (alias is F1) (1.0 by default)
	REALAR(13)	F2, second owner fraction (alias is F2) (1.0 by default)
	REALAR(14)	F3, third owner fraction (alias is F3) (1.0 by default)
	REALAR(15)	F4, fourth owner fraction (alias is F4) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a two-winding transformer
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &
	IERR = 7	branch table is full
	IERR = 8	branch ownership table is full
	IERR = 10	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - branch reactance is 0.0 - line length is negative - zero impedance line table is full; branch is treated as a normal line - invalid metered end bus - invalid branch status - invalid owner number - invalid ownership fraction - no ownership data is specified - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - branch is no longer treated as a zero impedance line - owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.6. BRANCH_DATA_3

This API routine is obsolete. It has been replaced by the API routine branch_data_4 .

Use this API to modify the data of an existing non-transformer branch in the working case or to add a new non-transformer branch to the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_BRANCH_DATA_3 IBUS JBUS CKT INTGAR(1)..INTGAR(6) REALAR(1)..REALAR(12)
 RATINGS(1)..RATINGS(12) NAMEAR
```

Python command syntax:

```
ierr = branch_data_3(ibus, jbus, ckt, intgar, realar, ratings, namear)
```

Fortran command syntax:

```
CALL BRANCH_DATA_3(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	ST, branch status (alias is ST) (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (alias is METBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (alias is O1) (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (alias is O2) (0 by default)
	INTGAR(5)	O3, third owner number (alias is O3) (0 by default)
	INTGAR(6)	O4, fourth owner number (alias is O4) (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	R, nominal branch resistance (alias is R) (0.0 by default)
	REALAR(2)	X, nominal branch reactance (alias is X) (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALAR(3)	B, total line charging (alias is B) (0.0 by default)
	REALAR(4)	GI, real line shunt at bus IBUS end (alias is GI) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(5)	BI, reactive line shunt at bus IBUS end (alias is BI) (0.0 by default)
	REALAR(6)	GJ, real line shunt at bus JBUS end (alias is GJ) (0.0 by default)
	REALAR(7)	BJ, reactive line shunt at bus JBUS end (alias is BJ) (0.0 by default)
	REALAR(8)	LEN, line length (alias is LEN) (0.0 by default)
	REALAR(9)	F1, first owner fraction (alias is F1) (1.0 by default)
	REALAR(10)	F2, second owner fraction (alias is F2) (1.0 by default)
	REALAR(11)	F3, third owner fraction (alias is F3) (1.0 by default)
	REALAR(12)	F4, fourth owner fraction (alias is F4) (1.0 by default)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	branch name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	circuit identifier is more than two characters
IERR = 3	branch exists but it is a two-winding transformer
IERR = 4	branch from a bus to itself
IERR = 5	blank circuit identifier is invalid
IERR = 6	circuit identifier may not start with &
IERR = 7	branch table is full
IERR = 8	branch ownership table is full
IERR = 10	prerequisite requirements for API are not met
IERR = 11	branch exists but it is a system switching device
IERR = 12	branch already exists
IERR = 13	node-breaker terminal connection table is full
IERR = -1	data error, warning and information messages; one or more of: - branch reactance is 0.0 - line length is negative - zero impedance line table is full; branch is treated as a normal line - invalid metered end bus - invalid branch status - invalid owner number - invalid ownership fraction - no ownership data is specified - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - branch is no longer treated as a zero impedance line - owner table is full - rating is negative - name already assigned to a branch - name contains invalid characters - non-transformer branch connects buses with different base voltages

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.7. BRANCH_DATA_4

Use this API to modify the data of an existing non-transformer branch in the working case or to add a new non-transformer branch to the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_BRANCH_DATA_4 IBUS JBUS CKT INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(12)
 RATINGS(1)..RATINGS(12) NAMEAR
```

Python command syntax:

```
ierr = branch_data_4(ibus, jbus, ckt, intgar, realar, ratings, namear)
```

Fortran command syntax:

```
CALL BRANCH_DATA_4(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	ST, branch status (alias is ST) (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (alias is METBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (alias is O1) (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (alias is O2) (0 by default)
	INTGAR(5)	O3, third owner number (alias is O3) (0 by default)
	INTGAR(6)	O4, fourth owner number (alias is O4) (0 by default)
	INTGAR(7)	BYPASS, branch bypass flag (alias is BYPASS) (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	R, nominal branch resistance (alias is R) (0.0 by default)
	REALAR(2)	X, nominal branch reactance (alias is X) (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALAR(3)	B, total line charging (alias is B) (0.0 by default)
	REALAR(4)	GI, real line shunt at bus IBUS end (alias is GI) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(5)	BI, reactive line shunt at bus IBUS end (alias is BI) (0.0 by default)
	REALAR(6)	GJ, real line shunt at bus JBUS end (alias is GJ) (0.0 by default)
	REALAR(7)	BJ, reactive line shunt at bus JBUS end (alias is BJ) (0.0 by default)
	REALAR(8)	LEN, line length (alias is LEN) (0.0 by default)
	REALAR(9)	F1, first owner fraction (alias is F1) (1.0 by default)
	REALAR(10)	F2, second owner fraction (alias is F2) (1.0 by default)
	REALAR(11)	F3, third owner fraction (alias is F3) (1.0 by default)
	REALAR(12)	F4, fourth owner fraction (alias is F4) (1.0 by default)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	branch name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	circuit identifier is more than two characters
IERR = 3	branch exists but it is a two-winding transformer
IERR = 4	branch from a bus to itself
IERR = 5	blank circuit identifier is invalid
IERR = 6	circuit identifier may not start with &
IERR = 7	branch table is full
IERR = 8	branch ownership table is full
IERR = 10	prerequisite requirements for API are not met
IERR = 11	branch exists but it is a system switching device
IERR = 12	branch already exists
IERR = 13	node-breaker terminal connection table is full
IERR = -1	data error, warning and information messages; one or more of: - branch reactance is 0.0 - line length is negative - zero impedance line table is full; branch is treated as a normal line - invalid metered end bus - invalid branch status - invalid owner number - invalid ownership fraction - no ownership data is specified - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - branch is no longer treated as a zero impedance line - owner table is full - rating is negative - name already assigned to a branch - name contains invalid characters - non-transformer branch connects buses with different base voltages

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.8. BRN_RATING_NAME_SET

Use this API to change the column and descriptions for a particular branch rating name set used in reporting.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_BRN_RATING_NAME_SET IRATE NAME DESCR
```

Python command syntax:

```
ierr = brn_rating_name_set(irate, name, descr)
```

Fortran command syntax:

```
CALL BRN_RATING_NAME_SET(IRATE, NAME, DESCR, IERR)
```

Where:

Integer IRATE	branch rating name set number (1 through 12) (input; no default allowed).	
Character*6 NAME	branch rating name name (input; blank by default).	
Character*32 DESCR	branch rating name description (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid rating set number
	IERR = 2	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - name of branch rating name truncated to 6 characters. - description of branch rating name truncated to 32 characters.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.9. BUS_CHNG_3

This API routine is obsolete. It has been replaced by the API routine bus_chng_4 .

Use this API routine to modify the data of an existing bus in the working case (see PSSE Program Operation Manual, Bus Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_BUS_CHNG_3  IBUS  INTGAR(1)..INTGAR(4)  REALAR(1)..REALAR(7)  'NAME'
```

Python command syntax:

```
ierr = bus_chng_3(ibus, intgar, realar, name)
```

Fortran command syntax:

```
CALL BUS_CHNG_3(IBUS, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IDE, bus type code (1 by default)
	INTGAR(2)	AREA, area number (1 by default)
	INTGAR(3)	ZONE, zone number (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
Real REALAR (7)	array of 7 elements specifying (input).	
	REALAR(1)	BASKV, bus base voltage in kV (0.0 by default)
	REALAR(2)	VM, bus voltage magnitude in pu (1.0 by default)
	REALAR(3)	VA, bus voltage phase angle (0.0 by default)
	REALAR(4)	NMAXV, normal bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(5)	NMINV, normal bus voltage magnitude low limit in pu (0.9 by default)
	REALAR(6)	EMAXV, emergency bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(7)	EMINV, emergency bus voltage magnitude low limit in pu (0.9 by default)
Character*12 NAME	bus name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid bus number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	working case is not initialized
IERR = 4	bus not found
IERR = 5	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid base voltage. - Invalid bus type code. - Invalid area, zone or owner number. - Bus name is more than 12 characters. - New area number but bus is the area swing for its old area. - Bus type code is 3 but is not the swing bus for its area. - Area table is full. - Zone table is full. - Owner table is full. - High voltage limit is not greater than the low voltage limit. - Normal voltage limit is less restrictive than the emergency limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.10. BUS_CHNG_4

Use this API routine to modify the data of an existing bus in the working case (see PSSE Program Operation Manual, Bus Data).

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_BUS_CHNG_4  IBUS INODE INTGAR(1)..INTGAR(4)  REALAR(1)..REALAR(7)
 'NAME'
```

Python command syntax:

```
ierr = bus_chng_4(ibus, inode, intgar, realar, name)
```

Fortran command syntax:

```
CALL BUS_CHNG_4(IBUS, INODE, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INODE	bus section node number (input; no default allowed).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IDE, bus type code (1 by default)
	INTGAR(2)	AREA, area number (1 by default)
	INTGAR(3)	ZONE, zone number (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
Real REALAR (7)	array of 7 elements specifying (input).	
	REALAR(1)	BASKV, bus base voltage in kV (0.0 by default)
	REALAR(2)	VM, bus voltage magnitude in pu (1.0 by default)
	REALAR(3)	VA, bus voltage phase angle (0.0 by default)
	REALAR(4)	NMAXV, normal bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(5)	NMINV, normal bus voltage magnitude low limit in pu (0.9 by default)
	REALAR(6)	EMAXV, emergency bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(7)	EMINV, emergency bus voltage magnitude low limit in pu (0.9 by default)
Character*12 NAME	bus name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid bus number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	working case is not initialized
IERR = 4	bus not found
IERR = 5	invalid node number
IERR = 6	bus does not belong to a substation
IERR = 7	inode is not found in the substation
IERR = 8	inode is not connected to any bus section of bus
IERR = 10	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid base voltage. - Invalid bus type code. - Invalid area, zone or owner number. - Bus name is more than 12 characters. - New area number but bus is the area swing for its old area. - Bus type code is 3 but is not the swing bus for its area. - Area table is full. - Zone table is full. - Owner table is full. - High voltage limit is not greater than the low voltage limit. - Normal voltage limit is less restrictive than the emergency limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.11. BUS_DATA

This API routine is obsolete. It has been replaced by the API routine bus_data_2 .

Use this API to modify the data of an existing bus in the working case or to add a new bus to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BUS_DATA  IBUS  INTGAR(1)..INTGAR(4)  REALAR(1)..REALAR(5)  'NAME'
```

Python command syntax:

```
ierr = bus_data(ibus, intgar, realar, name)
```

Fortran command syntax:

```
CALL BUS_API(IBUS, INTGAR, REALAR, NAME, IERR, NERR)
```

Where:

Integer IBUS	bus number (input).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IDE, bus type code
	INTGAR(2)	AREA, area number
	INTGAR(3)	ZONE, zone number
	INTGAR(4)	OWNER, owner number
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	GL, fixed shunt admittance (conductance)
	REALAR(2)	BL, fixed shunt admittance (susceptance)
	REALAR(3)	BASKV, bus base voltage in kV
	REALAR(4)	VM, bus voltage magnitude in pu
	REALAR(5)	VA, bus voltage phase angle
Character*12 NAME	bus name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid bus number
	IERR = 2	bus table is full
	IERR = 3	working case is not initialized
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid base voltage. - Invalid bus type code. - Invalid area, zone or owner number. - Bus name is more than 12 characters. - New area number but bus is the

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	area swing for its old area. - Bus type code is 3 but is not the swing bus for its area. - Area table is full. - Zone table is full. - Owner table is full
Integer NERR	number of entries in PBUF02 containing error messages (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.12. BUS_DATA_2

This API routine is obsolete. It has been replaced by the API routine bus_data_3 .

Use this API to modify the data of an existing bus in the working case or to add a new bus to the working case (see PSSE Program Operation Manual, Bus Data).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_BUS_DATA_2  IBUS  INTGAR(1)..INTGAR(4)  REALAR(1)..REALAR(3)  'NAME'
```

Python command syntax:

```
ierr = bus_data_2(ibus, intgar, realar, name)
```

Fortran command syntax:

```
CALL BUS_API_2(IBUS, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IDE, bus type code (1 by default)
	INTGAR(2)	AREA, area number (1 by default)
	INTGAR(3)	ZONE, zone number (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
Real REALAR (3)	array of 3 elements specifying (input).	
	REALAR(1)	BASKV, bus base voltage in kV (0.0 by default)
	REALAR(2)	VM, bus voltage magnitude in pu (1.0 by default)
	REALAR(3)	VA, bus voltage phase angle (0.0 by default)
Character*12 NAME	bus name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid bus number
	IERR = 2	bus table is full
	IERR = 3	working case is not initialized
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid base voltage. - Invalid bus type code. - Invalid area, zone or owner number. - Bus name is more than 12 characters. - New area number but bus is the

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

area swing for its old area. - Bus type code is 3 but is not the swing bus for its area. - Area table is full. - Zone table is full. - Owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.13. BUS_DATA_3

This API routine is obsolete. It has been replaced by the API routine bus_data_4 .

Use this API routine to modify the data of an existing bus in the working case, or to add a new bus to the working case (see PSSE Program Operation Manual, Bus Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_BUS_DATA_3  IBUS  INTGAR(1)..INTGAR(4)  REALAR(1)..REALAR(7)  'NAME'
```

Python command syntax:

```
ierr = bus_data_3(ibus, intgar, realar, name)
```

Fortran command syntax:

```
CALL BUS_API_3(IBUS, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IDE, bus type code (1 by default)
	INTGAR(2)	AREA, area number (1 by default)
	INTGAR(3)	ZONE, zone number (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
Real REALAR (7)	array of 7 elements specifying (input).	
	REALAR(1)	BASKV, bus base voltage in kV (0.0 by default)
	REALAR(2)	VM, bus voltage magnitude in pu (1.0 by default)
	REALAR(3)	VA, bus voltage phase angle (0.0 by default)
	REALAR(4)	NMAXV, normal bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(5)	NMINV, normal bus voltage magnitude low limit in pu (0.9 by default)
	REALAR(6)	EMAXV, emergency bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(7)	EMINV, emergency bus voltage magnitude low limit in pu (0.9 by default)
Character*12 NAME	bus name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid bus number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	bus table is full
IERR = 3	working case is not initialized
IERR = 5	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid base voltage. - Invalid bus type code. - Invalid area, zone or owner number. - Bus name is more than 12 characters. - New area number but bus is the area swing for its old area. - Bus type code is 3 but is not the swing bus for its area. - Area table is full. - Zone table is full. - Owner table is full. - High voltage limit is not greater than the low voltage limit. - Normal voltage limit is less restrictive than the emergency limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.14. BUS_DATA_4

Use this API routine to modify the data of an existing bus in the working case, or to add a new bus to the working case (see PSSE Program Operation Manual, Bus Data).

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_BUS_DATA_4  IBUS INODE INTGAR(1)..INTGAR(4)  REALAR(1)..REALAR(7)
 'NAME'
```

Python command syntax:

```
ierr = bus_data_4(ibus, inode, intgar, realar, name)
```

Fortran command syntax:

```
CALL BUS_API_4(IBUS, INODE, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INODE	bus section node number (input; no default allowed).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IDE, bus type code (1 by default)
	INTGAR(2)	AREA, area number (1 by default)
	INTGAR(3)	ZONE, zone number (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
Real REALAR (7)	array of 7 elements specifying (input).	
	REALAR(1)	BASKV, bus base voltage in kV (0.0 by default)
	REALAR(2)	VM, bus voltage magnitude in pu (1.0 by default)
	REALAR(3)	VA, bus voltage phase angle (0.0 by default)
	REALAR(4)	NMAXV, normal bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(5)	NMINV, normal bus voltage magnitude low limit in pu (0.9 by default)
	REALAR(6)	EMAXV, emergency bus voltage magnitude high limit in pu (1.1 by default)
	REALAR(7)	EMINV, emergency bus voltage magnitude low limit in pu (0.9 by default)
Character*12 NAME	bus name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid bus number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	bus table is full
IERR = 3	working case is not initialized
IERR = 5	invalid node number
IERR = 6	bus does not belong to a substation
IERR = 7	inode is not found in the substation
IERR = 8	inode is not connected to any bus section of bus
IERR = 9	inode must be 0 for bus not in the case
IERR = 10	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid base voltage. - Invalid bus type code. - Invalid area, zone or owner number. - Bus name is more than 12 characters. - New area number but bus is the area swing for its old area. - Bus type code is 3 but is not the swing bus for its area. - Area table is full. - Zone table is full. - Owner table is full. - High voltage limit is not greater than the low voltage limit. - Normal voltage limit is less restrictive than the emergency limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.15. BUS_NUMBER

Use this API to change the bus number of an existing bus in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BUS_NUMBER  IBUS  NEWBUS
```

Python command syntax:

```
ierr = bus_number(ibus, newbus)
```

Fortran command syntax:

```
CALL BUS_NUMBER_API_2(IBUS, NEWBUS, IERR)
```

Where:

Integer IBUS	current bus number of the bus for which the number is to be changed (input; no default allowed).	
Integer NEWBUS	new bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	invalid new bus number
	IERR = 3	bus number NEWBUS is already assigned to another bus
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - None

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.16. CASE_TITLE_DATA

Use this API to change the two line case title in the working case (see PSSE Program Operation Manual, Case Identification Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CASE_TITLE_DATA  'LINE1'  'LINE2'
```

Python command syntax:

```
ierr = case_title_data(line1, line2)
```

Fortran command syntax:

```
CALL CASE_TITLE_API_2(LINE1, LINE2, IERR)
```

Where:

Character*60 LINE1	first title line (input; unchanged).	
Character*60 LINE2	second title line (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Title line exceeds 60 characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.17. CASOLUTION_PARAMETERS

Use this API to modify the contingency analysis solution parameters.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CASOLUTION_PARAMETERS INTGAR(1)..INTGAR(9) REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = casolution_parameters(intgar, realar)
```

Fortran command syntax:

```
CALL CASOLUTION_PARAMETER_API(INTGAR, REALAR, IERR)
```

Where:

Integer INTGAR (9)	array of 9 elements specifying (input).	
	INTGAR(1)	RATBAS, base case rating (1 by default)
	INTGAR(2)	RATCNT, contingency case rating (2 by default)
	INTGAR(3)	MWRATE, Convert MVA to MW for DC solution (1 by default)
	INTGAR(4)	VLTBAS, base case voltage limit (1 by default)
	INTGAR(5)	VLTCNT, contingency case voltage limit (2 by default)
	INTGAR(6)	SCRCAS, enable AC contingency case screening (0 by default)
	INTGAR(7)	INCLOS, include loss estimates in DC solution (0 by default)
	INTGAR(8)	ROSOLN, Enable Robust Power flow solution for non-converged cases (0 by default)
	INTGAR(9)	BASLOD, Base loaded machine option (0 by default)
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	RATPCT, percent of loading (100 by default)
	REALAR(2)	LOADCT, loading cutoff (2 by default)
	REALAR(3)	CHAGMW, flow change cutoff in MW (100 by default)
	REALAR(4)	TOTGEN,Total generation dispatch limit (10000 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.18. CASOLUTION_PARAMETERS_2

Use this API to modify the contingency analysis solution parameters.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_CASOLUTION_PARAMETERS INTGAR(1)..INTGAR(10) REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = casolution_parameters_2(intgar, realar)
```

Fortran command syntax:

```
CALL CASOLUTION_PARAMETER_API_2(INTGAR, REALAR, IERR)
```

Where:

Integer INTGAR (10)	array of 10 elements specifying (input).	
	INTGAR(1)	RATBAS, base case rating (1 by default)
	INTGAR(2)	RATCNT, contingency case rating (2 by default)
	INTGAR(3)	MWRATE, Convert MVA to MW for DC solution (1 by default)
	INTGAR(4)	VLTBAS, base case voltage limit (1 by default)
	INTGAR(5)	VLTCNT, contingency case voltage limit (2 by default)
	INTGAR(6)	SCRCAS, enable AC contingency case screening (0 by default)
	INTGAR(7)	INCLOS, include loss estimates in DC solution (0 by default)
	INTGAR(8)	ROSOLN, Enable Robust Power flow solution for non-converged cases (0 by default)
	INTGAR(9)	BASLOD, Base loaded machine option (0 by default)
	INTGAR(10)	RASFLG, enable RAS (0 by default)
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	RATPCT, percent of loading (100 by default)
	REALAR(2)	LOADCT, loading cutoff (2 by default)
	REALAR(3)	CHAGMW, flow change cutoff in MW (100 by default)
	REALAR(4)	TOTGEN,Total generation dispatch limit (10000 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.19. CASOLUTION_PARAMETERS_3

Use this API to modify the contingency analysis solution parameters.

This API routine was first introduced in release 35.4.1.

Batch command syntax:

```
BAT_CASOLUTION_PARAMETERS INTGAR(1)..INTGAR(10) REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = casolution_parameters_3(intgar, realar)
```

Fortran command syntax:

```
CALL CASOLUTION_PARAMETER_API_3(INTGAR, REALAR, IERR)
```

Where:

Integer INTGAR (10)	array of 10 elements specifying (input).	
	INTGAR(1)	RATBAS, base case rating (1 by default)
	INTGAR(2)	RATCNT, contingency case rating (2 by default)
	INTGAR(3)	MWRATE, Convert MVA to MW for DC solution (1 by default)
	INTGAR(4)	VLTBAS, base case voltage limit (1 by default)
	INTGAR(5)	VLTCNT, contingency case voltage limit (2 by default)
	INTGAR(6)	SCRCAS, enable AC contingency case screening (0 by default)
	INTGAR(7)	INCLOS, include loss estimates in DC solution (0 by default)
	INTGAR(8)	ROSOLN, Enable Robust Power flow solution for non-converged cases (0 by default)
	INTGAR(9)	BASLOD, Base loaded machine option (0 by default)
	INTGAR(10)	RASFLG, enable RAS (0 by default)
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	RATPCT, percent of loading (100 by default)
	REALAR(2)	LOADCT, loading cutoff (2 by default)
	REALAR(3)	CHAGMW, flow change cutoff in MW (100 by default)
	REALAR(4)	TOTGEN,Total generation dispatch limit (10000 by default)
	REALAR(5)	bus angle difference across a branch (90.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR > 0	fatal error occurred
IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.20. EXTR

Use this API to purge specified buses and all branches connected to them from the working case (activity EXTR).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_EXTR  SID  ALL  STATUS(1)  STATUS(2)
```

Python command syntax:

```
ierr = extr(sid, all, status)
```

Fortran command syntax:

```
CALL EXTRNW(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (when ALL is 0) (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; 0 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer STATUS (2)	array of 2 elements specifying They are used to control subsystem removal options. A value of 0 means the option is not enabled. A value of 1 means the option is enabled. They are as follows(input).	
	STATUS(1)	remove generator entries for out-of-service plants (0 by default)
	STATUS(2)	change codes of boundary buses (0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.21. FACTS_CHNG_2

This API routine is obsolete. It has been replaced by the API routine facts_chng_3 .

Use this API routine to modify the data of an existing FACTS device in the working case (see PSSE Program Operation Manual, FACTS Device Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_FACTS_CHNG_2 'NAME' INTGAR(1)..INTGAR(6) REALAR(1)..REALAR(13) 'MNAME'
```

Python command syntax:

```
ierr, realaro = facts_chng_2(name, intgar, realari, mname)
```

Fortran command syntax:

```
CALL FACTS_CHNG_2(NAME, INTGAR, REALAR, MNAME, IERR)
```

Where:

Character*12 NAME	name of the FACTS device (input; no default allowed).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	I, sending end bus number (no default allowed)
	INTGAR(2)	J, terminal end bus number (0 for STATCON) (0 by default)
	INTGAR(3)	MODE, control mode (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
	INTGAR(5)	VSREF, series voltage reference code for modes 4 and 7 (0 for sending end voltage; 1 for series current) (0 by default)
	INTGAR(6)	REMOT, remote bus number (0 for sending end bus) (0 by default)
Real REALAR (19)	(first thirteen are input values; last six are output values) (input (output).).	
Real REALARI (13)	array of 13 elements specifying corresponding to values REALAR(1) through REALAR(13) as described above(input).	
	REALARI(1)	PDES, desired real power at terminal bus (0.0 by default)
	REALARI(2)	QDES, desired reactive power at terminal bus (0.0 by default)
	REALARI(3)	VSET, sending bus voltage setpoint (1.0 by default)
	REALARI(4)	SHMX, sending end maximum shunt current (MVA at rated voltage) (99.99 * SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(5)	TRMX, maximum bridge real power transfer (99.99 * SBASE by default)
	REALARI(6)	VTMN, minimum terminal bus voltage magnitude (0.9 by default)
	REALARI(7)	VTMX, maximum terminal bus voltage magnitude (1.1 by default)
	REALARI(8)	VSMX, maximum series voltage magnitude (1.0 by default)
	REALARI(9)	IMX, maximum series current (MVA at rated voltage) (0.0 by default)
	REALARI(10)	LINX, pu reactance of temporary series element (0.05 by default)
	REALARI(11)	RMPCT, percent of contributed reactive power (100.0 by default)
	REALARI(12)	SET1, real component of constant Z (MODE = 3), magnitude of constant V (MODE = 4), or VD (MODE = 7 or 8) (0.0 by default)
	REALARI(13)	SET2, imaginary component of constant Z (MODE = 3), angle of constant V (MODE = 4), or VQ (MODE = 7 or 8) (0.005 by default when MODE = 3; 0.0 otherwise)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(14) through REALAR(19) as described above(output).	
	REALARO(1)	real component of present effective series Z (returned)
	REALARO(2)	imaginary component of present effective series Z (returned)
	REALARO(3)	magnitude of present series voltage, FDVREF reference (returned)
	REALARO(4)	le of present series voltage, FDVREF reference (returned)
	REALARO(5)	real component of present series voltage, FDVREF reference (returned)
	REALARO(6)	imaginary component of present series voltage, FDVREF reference (returned)
Character*12 MNAME	name of the FACTS device that is the master device for this IPFC slave device (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 2	no sending end bus is specified
	IERR = 3	bus not found
	IERR = 4	the same bus is specified for the sending and terminal end buses
	IERR = 5	blank FACTS device name is invalid
	IERR = 6	FACTS device not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	prerequisite requirements for API are not met
IERR = -1	warnings - one or more of: - Invalid FACTS device control mode. - Invalid owner number. - Invalid series voltage reference code. - Regulated bus not found. - Shunt element specified as remotely regulating itself. - Type code of regulated bus is not 1 or 2. - Shunt element voltage setpoint must be positive. - Terminal bus voltage limit must be positive. - VTMX is less than VTMN. - Series voltage limit must be positive. - Shunt current limit must not be negative. - Bridge real power transfer limit must not be negative. - Series current limit must not be negative. - Series reactance must be positive. - Shunt current limit is not 0.0 for an IPFC device. - Bridge real power transfer limit is not 0.0 for an IPFC device. - Impedance setpoint has reactance of 0.0. - RMPCT is not positive. - Master IPFC FACTS device not found. - Bus is dummy bus of a multi-section line. - Multi-section line grouping deleted. - Owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.22. FACTS_CHNG_3

Use this API routine to modify the data of an existing FACTS device in the working case (see PSSE Program Operation Manual, FACTS Device Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_FACTS_CHNG_3 'NAME' INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(13) 'MNAME'
```

Python command syntax:

```
ierr, realaro = facts_chng_3(name, intgar, realari, mname)
```

Fortran command syntax:

```
CALL FACTS_CHNG_3(NAME, INTGAR, REALAR, MNAME, IERR)
```

Where:

Character*12 NAME	name of the FACTS device (input; no default allowed).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	I, sending end bus number (no default allowed)
	INTGAR(2)	J, terminal end bus number (0 for STATCON) (0 by default)
	INTGAR(3)	MODE, control mode (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
	INTGAR(5)	VSREF, series voltage reference code for modes 4 and 7 (0 for sending end voltage; 1 for series current) (0 by default)
	INTGAR(6)	IREG, regulated bus number (0 by default)
	INTGAR(7)	NODE, regulated node number (0 by default)
Real REALAR (19)	(first thirteen are input values; last six are output values) (input (output).).	
Real REALARI (13)	array of 13 elements specifying corresponding to values REALAR(1) through REALAR(13) as described above(input).	
	REALARI(1)	PDES, desired real power at terminal bus (0.0 by default)
	REALARI(2)	QDES, desired reactive power at terminal bus (0.0 by default)
	REALARI(3)	VSET, sending bus voltage setpoint (1.0 by default)
	REALARI(4)	SHMX, sending end maximum shunt current (MVA at rated voltage) (99.99 * SBASE by default)
	REALARI(5)	TRMX, maximum bridge real power transfer (99.99 * SBASE by default)
	REALARI(6)	VTMN, minimum terminal bus voltage magnitude (0.9 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(7)	VTMX, maximum terminal bus voltage magnitude (1.1 by default)
	REALARI(8)	VSMX, maximum series voltage magnitude (1.0 by default)
	REALARI(9)	IMX, maximum series current (MVA at rated voltage) (0.0 by default)
	REALARI(10)	LINX, pu reactance of temporary series element (0.05 by default)
	REALARI(11)	RMPCT, percent of contributed reactive power (100.0 by default)
	REALARI(12)	SET1, real component of constant Z (MODE = 3), magnitude of constant V (MODE = 4), or VD (MODE = 7 or 8) (0.0 by default)
	REALARI(13)	SET2, imaginary component of constant Z (MODE = 3), angle of constant V (MODE = 4), or VQ (MODE = 7 or 8) (0.005 by default when MODE = 3; 0.0 otherwise)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(14) through REALAR(19) as described above(output).	
	REALARO(1)	real component of present effective series Z (returned)
	REALARO(2)	imaginary component of present effective series Z (returned)
	REALARO(3)	magnitude of present series voltage, FDVREF reference (returned)
	REALARO(4)	le of present series voltage, FDVREF reference (returned)
	REALARO(5)	real component of present series voltage, FDVREF reference (returned)
	REALARO(6)	imaginary component of present series voltage, FDVREF reference (returned)
Character*12 MNAME	name of the FACTS device that is the master device for this IPFC slave device (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 2	no sending end bus is specified
	IERR = 3	bus not found
	IERR = 4	the same bus is specified for the sending and terminal end buses
	IERR = 5	blank FACTS device name is invalid
	IERR = 6	FACTS device not found
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid FACTS device control mode. - Invalid owner number. -

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Invalid series voltage reference code. - Regulated bus not found. - Remote regulated bus is specified for a type 3 bus. - Type code of regulated bus is not 1 or 2. - Shunt element voltage setpoint must be positive. - Terminal bus voltage limit must be positive. - VTMX is less than VTMN. - Series voltage limit must be positive. - Shunt current limit must not be negative. - Bridge real power transfer limit must not be negative. - Series current limit must not be negative. - Series reactance must be positive. - Shunt current limit is not 0.0 for an IPFC device. - Bridge real power transfer limit is not 0.0 for an IPFC slave. - Impedance setpoint has reactance of 0.0. - RMPCT is not positive. - Master IPFC FACTS device not found. - Bus is dummy bus of a multi-section line. - Multi-section line grouping deleted. - Owner table is full - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.23. FACTS_DATA

This API routine is obsolete. It has been replaced by the API routine facts_data_2 .

Use this API to modify the data of an existing FACTS device in the working case or to add a new FACTS device to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_FACTS_DATA FD INTGAR(1)..INTGAR(5) REALAR(1)..REALAR(13)
```

Python command syntax:

```
ierr, realaro = facts_data(fd, intgar, realari)
```

Fortran command syntax:

```
CALL FACTS_API(FD, INTGAR, REALAR, IERR, NERR)
```

Where:

Integer FD	number (which the API converts to a left-justified name) of the FACTS device for which data is to be specified (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	I, sending end bus number (alias is I)
	INTGAR(2)	J, terminal end bus number (0 for STATCON) (alias is J)
	INTGAR(3)	MODE, control mode (alias is MODE)
	INTGAR(4)	OWNER, owner number (alias is OWNER)
	INTGAR(5)	VSREF, series voltage reference code for modes 4 and 7 (0 for sending end voltage; 1 for series current) (alias is VSREF)
Real REALAR (19)	(first thirteen are input values; last six are output values) (input (output).).	
Real REALARI (13)	array of 13 elements specifying corresponding to values REALAR(1) through REALAR(13) as described above(input).	
	REALARI(1)	PDES, desired real power at terminal bus (alias is PDES)
	REALARI(2)	QDES, desired reactive power at terminal bus (alias is QDES)
	REALARI(3)	VSET, sending bus voltage setpoint (alias is VSET)
	REALARI(4)	SHMX, sending end maximum shunt current (MVA at rated voltage) (alias is SHMAX)
	REALARI(5)	TRMX, maximum bridge real power transfer (alias is TRMX)
	REALARI(6)	VTMN, minimum terminal bus voltage magnitude (alias is VTMN)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(7)	VTMX, maximum terminal bus voltage magnitude (alias is VTMX)
	REALARI(8)	VSMX, maximum series voltage magnitude (alias is VSMX)
	REALARI(9)	IMX, maximum series current (MVA at rated voltage) (alias is IMX)
	REALARI(10)	LINX, pu reactance of temporary series element (alias is LINX)
	REALARI(11)	RMPCT, percent of contributed reactive power (alias is RMPCT)
	REALARI(12)	SET1, real component of constant Z (mode = 3), magnitude of constant V (mode = 4), or VD (mode = 7 or 8) (alias is SET1)
	REALARI(13)	SET2, imaginary component of constant Z (mode = 3), angle of constant V (mode = 4), or VQ (mode = 7 or 8) (alias is SET2)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(14) through REALAR(19) as described above(output).	
	REALARO(1)	real component of present effective series Z (returned)
	REALARO(2)	imaginary component of present effective series Z (returned)
	REALARO(3)	magnitude of present series voltage, FDVREF reference (returned)
	REALARO(4)	le of present series voltage, FDVREF reference (returned)
	REALARO(5)	real component of present series voltage, FDVREF reference (returned)
	REALARO(6)	imaginary component of present series voltage, FDVREF reference (returned)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	FACTS device table is full
	IERR = 2	no sending end bus is specified
	IERR = 3	bus not found
	IERR = 4	the same bus is specified for the sending and terminal end buses
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid FACTS device control model. - Invalid owner number. - Invalid series voltage reference code. - Shunt element voltage setpoint must be positive. - Terminal bus voltage limit must be positive. - VTMX is less than VTMN. - Series voltage lim-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	it must be positive. - Shunt current limit must not be negative. - Bridge real power transfer limit must not be negative. - Series current limit must not be negative. - Series reactance must be positive. - Shunt current limit is not 0.0 for an IPFC device. - Bridge real power transfer limit is not 0.0 for an IPFC device. - Impedance setpoint has reactance of 0.0. - RMPCT is not positive. - Master IPFC FACTS device not found. - Bus is dummy bus of a multi-section line. - Multi-section line grouping deleted. - Owner table is full
Integer NERR	number of entries in PBUF02 containing error messages (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.24. FACTS_DATA_2

This API routine is obsolete. It has been replaced by the API routine facts_data_3 .

Use this API to modify the data of an existing FACTS device in the working case, or to add a new FACTS device to the working case (see PSSE Program Operation Manual, FACTS Device Data).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_FACTS_DATA_2 'NAME' INTGAR(1)..INTGAR(6) REALAR(1)..REALAR(13) 'MNAME'
```

Python command syntax:

```
ierr, realaro = facts_data_2(name, intgar, realari, mname)
```

Fortran command syntax:

```
CALL FACTS_API_2(NAME, INTGAR, REALAR, MNAME, IERR)
```

Where:

Character*12 NAME	name of the FACTS device (input; no default allowed).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	I, sending end bus number (no default allowed)
	INTGAR(2)	J, terminal end bus number (0 for STATCON) (0 by default)
	INTGAR(3)	MODE, control mode (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
	INTGAR(5)	VSREF, series voltage reference code for modes 4 and 7 (0 for sending end voltage; 1 for series current) (0 by default)
	INTGAR(6)	REMOT, remote bus number (0 for sending end bus) (0 by default)
Real REALAR (19)	(first thirteen are input values; last six are output values) (input (output).).	
Real REALARI (13)	array of 13 elements specifying corresponding to values REALAR(1) through REALAR(13) as described above(input).	
	REALARI(1)	PDES, desired real power at terminal bus (0.0 by default)
	REALARI(2)	QDES, desired reactive power at terminal bus (0.0 by default)
	REALARI(3)	VSET, sending bus voltage setpoint (1.0 by default)
	REALARI(4)	SHMX, sending end maximum shunt current (MVA at rated voltage) (99.99 * SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(5)	TRMX, maximum bridge real power transfer (99.99 * SBASE by default)
	REALARI(6)	VTMN, minimum terminal bus voltage magnitude (0.9 by default)
	REALARI(7)	VTMX, maximum terminal bus voltage magnitude (1.1 by default)
	REALARI(8)	VSMX, maximum series voltage magnitude (1.0 by default)
	REALARI(9)	IMX, maximum series current (MVA at rated voltage) (0.0 by default)
	REALARI(10)	LINX, pu reactance of temporary series element (0.05 by default)
	REALARI(11)	RMPCT, percent of contributed reactive power (100.0 by default)
	REALARI(12)	SET1, real component of constant Z (MODE = 3), magnitude of constant V (MODE = 4), or VD (MODE = 7 or 8) (0.0 by default)
	REALARI(13)	SET2, imaginary component of constant Z (MODE = 3), angle of constant V (MODE = 4), or VQ (MODE = 7 or 8) (0.005 by default when MODE = 3; 0.0 otherwise)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(14) through REALAR(19) as described above(output).	
	REALARO(1)	real component of present effective series Z (returned)
	REALARO(2)	imaginary component of present effective series Z (returned)
	REALARO(3)	magnitude of present series voltage, FDVREF reference (returned)
	REALARO(4)	le of present series voltage, FDVREF reference (returned)
	REALARO(5)	real component of present series voltage, FDVREF reference (returned)
	REALARO(6)	imaginary component of present series voltage, FDVREF reference (returned)
Character*12 MNAME	name of the FACTS device that is the master device for this IPFC slave device (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	FACTS device table is full
	IERR = 2	no sending end bus is specified
	IERR = 3	bus not found
	IERR = 4	the same bus is specified for the sending and terminal end buses
	IERR = 5	blank FACTS device name is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	prerequisite requirements for API are not met
IERR = -1	warnings - one or more of: - Invalid FACTS device control mode. - Invalid owner number. - Invalid series voltage reference code. - Regulated bus not found. - Shunt element specified as remotely regulating itself. - Type code of regulated bus is not 1 or 2. - Shunt element voltage setpoint must be positive. - Terminal bus voltage limit must be positive. - VTMX is less than VTMN. - Series voltage limit must be positive. - Shunt current limit must not be negative. - Bridge real power transfer limit must not be negative. - Series current limit must not be negative. - Series reactance must be positive. - Shunt current limit is not 0.0 for an IPFC device. - Bridge real power transfer limit is not 0.0 for an IPFC device. - Impedance setpoint has reactance of 0.0. - RMPCT is not positive. - Master IPFC FACTS device not found. - Bus is dummy bus of a multi-section line. - Multi-section line grouping deleted. - Owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.25. FACTS_DATA_3

Use this API to modify the data of an existing FACTS device in the working case, or to add a new FACTS device to the working case (see PSSE Program Operation Manual, FACTS Device Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_FACTS_DATA_3 'NAME' INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(13) 'MNAME'
```

Python command syntax:

```
ierr, realaro = facts_data_3(name, intgar, realari, mname)
```

Fortran command syntax:

```
CALL FACTS_API_3(NAME, INTGAR, REALAR, MNAME, IERR)
```

Where:

Character*12 NAME	name of the FACTS device (input; no default allowed).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	I, sending end bus number (no default allowed)
	INTGAR(2)	J, terminal end bus number (0 for STATCON) (0 by default)
	INTGAR(3)	MODE, control mode (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
	INTGAR(5)	VSREF, series voltage reference code for modes 4 and 7 (0 for sending end voltage; 1 for series current) (0 by default)
	INTGAR(6)	IREG, regulated bus number (0 by default)
	INTGAR(7)	NODE, regulated node number (0 by default)
Real REALAR (19)	(first thirteen are input values; last six are output values) (input (output).).	
Real REALARI (13)	array of 13 elements specifying corresponding to values REALAR(1) through REALAR(13) as described above(input).	
	REALARI(1)	PDES, desired real power at terminal bus (0.0 by default)
	REALARI(2)	QDES, desired reactive power at terminal bus (0.0 by default)
	REALARI(3)	VSET, sending bus voltage setpoint (1.0 by default)
	REALARI(4)	SHMX, sending end maximum shunt current (MVA at rated voltage) (99.99 * SBASE by default)
	REALARI(5)	TRMX, maximum bridge real power transfer (99.99 * SBASE by default)
	REALARI(6)	VTMN, minimum terminal bus voltage magnitude (0.9 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(7)	VTMX, maximum terminal bus voltage magnitude (1.1 by default)
	REALARI(8)	VSMX, maximum series voltage magnitude (1.0 by default)
	REALARI(9)	IMX, maximum series current (MVA at rated voltage) (0.0 by default)
	REALARI(10)	LINX, pu reactance of temporary series element (0.05 by default)
	REALARI(11)	RMPCT, percent of contributed reactive power (100.0 by default)
	REALARI(12)	SET1, real component of constant Z (MODE = 3), magnitude of constant V (MODE = 4), or VD (MODE = 7 or 8) (0.0 by default)
	REALARI(13)	SET2, imaginary component of constant Z (MODE = 3), angle of constant V (MODE = 4), or VQ (MODE = 7 or 8) (0.005 by default when MODE = 3; 0.0 otherwise)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(14) through REALAR(19) as described above(output).	
	REALARO(1)	real component of present effective series Z (returned)
	REALARO(2)	imaginary component of present effective series Z (returned)
	REALARO(3)	magnitude of present series voltage, FDVREF reference (returned)
	REALARO(4)	le of present series voltage, FDVREF reference (returned)
	REALARO(5)	real component of present series voltage, FDVREF reference (returned)
	REALARO(6)	imaginary component of present series voltage, FDVREF reference (returned)
Character*12 MNAME	name of the FACTS device that is the master device for this IPFC slave device (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	FACTS device table is full
	IERR = 2	no sending end bus is specified
	IERR = 3	bus not found
	IERR = 4	the same bus is specified for the sending and terminal end buses
	IERR = 5	blank FACTS device name is invalid
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid FACTS device control mode. - Invalid owner number. -

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Invalid series voltage reference code. - Regulated bus not found. - Remote regulated bus is specified for a type 3 bus. - Type code of regulated bus is not 1 or 2. - Shunt element voltage setpoint must be positive. - Terminal bus voltage limit must be positive. - VTMX is less than VTMN. - Series voltage limit must be positive. - Shunt current limit must not be negative. - Bridge real power transfer limit must not be negative. - Series current limit must not be negative. - Series reactance must be positive. - Shunt current limit is not 0.0 for an IPFC device. - Bridge real power transfer limit is not 0.0 for an IPFC slave. - Impedance setpoint has reactance of 0.0. - RMPCT is not positive. - Master IPFC FACTS device not found. - Bus is dummy bus of a multi-section line. - Multi-section line grouping deleted. - Owner table is full - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.26. GNE_CHNG

Use this API routine to modify the data of an existing GNE device in the working case (see PSSE Program Operation Manual, GNE Device Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_GNE_CHNG  'NAME'  'MODEL'  NTERM  TRMNLS(1)..TRMNLS(NTERM)
 INTGAR(1)..INTGAR(3)  NRELCN  RELCON(1)..RELCON(NRELCN)  NINTCN
 INTCON(1)..INTCON(NINTCN)  NCHRCN  CHRCON(1)..CHRCON(NCHRCN)
```

Python command syntax:

```
ierr = gne_chng(name, model, nterm, trmnls, intgar, nrelcn, relcon,
 nintcn, intcon, nchrcn, chrcon)
```

Fortran command syntax:

```
CALL GNE_CHNG(NAME, MODEL, NTERM, TRMNLS, INTGAR, NRELCN, RELCON, NINTCN,
 INTCON, NCHRCN, CHRCON, IERR)
```

Where:

Character*12 NAME	name of the GNE device (input; no default allowed).
Character*16 MODEL	name of the GMB/BOSL model to be used by this GNE device. The GMB/BOSL model file "MODEL.mac" must be available (input; no default allowed).
Integer NTERM	number of terminals (i.e., buses) to which the model is connected. NTERM must be 1 for GNE-PQ and GNE-I models, and it must be either 1 or 2 for a GNE-Y model (input; no default allowed).
Integer TRMNLS (NTERM)	containing the bus numbers of the buses to which this model is connected (input; no default allowed).
Integer INTGAR (3)	array of 3 elements specifying (input).
	INTGAR(1) STATUS, model status (0 for out-of-service, 1 for in-service) (1 by default)
	INTGAR(2) OWNER, owner number (the owner to which bus TRMNLS (1) is assigned by default)
	INTGAR(3) NMETR, non-metered end bus number (TRMNLS (NTERM) by default)
Integer NRELCN	number of real constants required by the model (input; no default allowed).
Real RELCON (NRELCN)	real model constants (input; 0.0).
Integer NINTCN	number of integer constants required by the model (input; no default allowed).
Integer INTCON (NINTCN)	integer model constants (usually bus numbers) (TRMNLS) (input; 1 by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NCHRCN	number of two-character identifiers required required by the model (input; no default allowed).	
Character*12 CHRCON (NCHRCN)	containing character model constants (e.g., circuit identifiers) (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	blank GNE device name is invalid
	IERR = 2	invalid number of terminal buses for this GMB/BOSL model
	IERR = 3	bus not found
	IERR = 4	model is not a GNE model
	IERR = 5	number of constants specified differs from those required in the model's.mac file
	IERR = 7	blank model name is invalid for a new GNE device instance
	IERR = 8	GNE.mac file is not found or an error occurred while reading it
	IERR = 9	GNE device not found
	IERR = 10	prerequisite requirements for API are not met
	IERR = 18	BOSL/GMB not installed/initialized
	IERR = -1	warnings - one or more of: - Invalid GNE device status. - Invalid owner number. - Invalid non-metered end bus. - Bus specified in an integer constant is not found. - Owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.27. GNE_DATA

Use this API routine to run the GNE device data specification function. It modifies the data of an existing GNE device in the working case, or adds a new GNE device to the working case (see PSSE Program Operation Manual, GNE Device Data).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_GNE_DATA  'NAME'  'MODEL'  NTERM  TRMNLS(1)..TRMNLS(NTERM)
 INTGAR(1)..INTGAR(3)  NRELCN  RELCON(1)..RELCON(NRELCN)  NINTCN
 INTCON(1)..INTCON(NINTCN)  NCHRCN  CHRCON(1)..CHRCON(NCHRCN)
```

Python command syntax:

```
ierr = gne_data(name, model, nterm, trmnls, intgar, nrelcn, relcon,
 nintcn, intcon, nchrcn, chrcon)
```

Fortran command syntax:

```
CALL GNE_API(NAME, MODEL, NTERM, TRMNLS, INTGAR, NRELCN, RELCON, NINTCN,
 INTCON, NCHRCN, CHRCON, IERR)
```

Where:

Character*12 NAME	name of the GNE device (input; no default allowed).
Character*16 MODEL	name of the GMB/BOSL model to be used by this GNE device. The GMB/BOSL model file "MODEL.mac" must be available (input; no default allowed).
Integer NTERM	number of terminals (i.e., buses) to which the model is connected. NTERM must be 1 for GNE-PQ and GNE-I models, and it must be either 1 or 2 for a GNE-Y model (input; no default allowed).
Integer TRMNLS (NTERM)	containing the bus numbers of the buses to which this model is connected (input; no default allowed).
Integer INTGAR (3)	array of 3 elements specifying (input).
	INTGAR(1) STATUS, model status (0 for out-of-service, 1 for in-service) (1 by default)
	INTGAR(2) OWNER, owner number (the owner to which bus TRMNLS (1) is assigned by default)
	INTGAR(3) NMETR, non-metered end bus number (TRMNLS (NTERM) by default)
Integer NRELCN	number of real constants required by the model (input; no default allowed).
Real RELCON (NRELCN)	real model constants (input; 0.0).
Integer NINTCN	number of integer constants required by the model (input; no default allowed).
Integer INTCON (NINTCN)	integer model constants (usually bus numbers) (TRMNLS) (input; 1 by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NCHRCN	number of two-character identifiers required required by the model (input; no default allowed).	
Character*12 CHRCON (NCHRCN)	containing character model constants (e.g., circuit identifiers) (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	blank GNE device name is invalid
	IERR = 2	invalid number of terminal buses for this GMB/BOSL model
	IERR = 3	bus not found
	IERR = 4	model is not a GNE model
	IERR = 5	number of constants specified differs from those required in the model's.mac file
	IERR = 6	GNE device instance table is full
	IERR = 7	blank model name is invalid for a new GNE device instance
	IERR = 8	GNE.mac file is not found or an error occurred while reading it
	IERR = 10	prerequisite requirements for API are not met
	IERR = 18	BOSL/GMB not installed/initialized
	IERR = -1	warnings - one or more of: - Invalid GNE device status. - Invalid owner number. - Invalid non-metered end bus. - Bus specified in an integer constant is not found. - Owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.28. IMPEDANCE_CORRECTION_DATA

This API routine is obsolete. It has been replaced by the API routine impedance_correction_data_3 .

Use this API to modify the data of an existing transformer impedance correction table in the working case or to add a new transformer impedance correction table to the working case (see PSSE Program Operation Manual, Transformer Impedance Correction Table Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_IMPEDANCE_CORRECTION_DATA  I   REALAR(1,1)  REALAR(2,1)  REALAR(1,2)
 REALAR(2,2)..REALAR(1,11)  REALAR(2,11)
```

Python command syntax:

```
ierr, intgar = impedance_correction_data(i, realar)
```

Fortran command syntax:

```
CALL IMPEDANCE_CORRECTION_TAB_API_2(I, INTGAR, REALAR, IERR)
```

Where:

Integer I	table number (must be between 1 and the number of tables allowed at the present size level of PSSE) (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (output).	
	INTGAR(1)	number of entries in table (returned)
	INTGAR(2)	table type flag (0=ratio, 1=angle) (returned)
Real REALAR (22)	22 values dimensioned (2,11), where REALAR(1,i) specifies Ti, and REALAR(2,i) specifies Fi. The values are specified such that the left subscript varies more rapidly: (1,1), (2,1), (1,2), (2,2), .. (1,11), (2,11). (REALAR(1,1) = REALAR (input; 2,1) =1.0 by default; all other values are 0.0 by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid transformer impedance correction table number
	IERR = 3	ABSCISSA values are not in ascending order
	IERR = 4	ABSCISSA value is outside of the range of -180 to +180
	IERR = 5	table has fewer than 2 points
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - point is beyond previous end of table - invalid scaling factor

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.29. IMPEDANCE_CORRECTION_DATA_3

Use this API to modify the data of an existing transformer impedance correction table in the working case or to add a new transformer impedance correction table to the working case (see PSSE Program Operation Manual, Transformer Impedance Correction Table Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_IMPEDANCE_CORRECTION_DATA_3  I POINTS REALAR(1)..REALAR(POINTS)
 CPLXAR(1)..CPLXAR(POINTS)
```

Python command syntax:

```
ierr = impedance_correction_data_3(i, points, realar, cplxar)
```

Fortran command syntax:

```
CALL IMPEDANCE_CORRECTION_TAB_API_3(I, POINTS, REALAR, CPLXAR, IERR)
```

Where:

Integer I	table number (must be between 1 and the number of tables allowed at the present size level (input).	
Integer POINTS	number of points specified in table 'I' (2 through 99) (input; no default allowed).	
Real REALAR (POINTS)	array of 'POINTS' values specifying either off-nominal turns ratios in pu or phase shift angles in degrees in strictly ascending order (input).	
Complex CPLXAR (POINTS)	array of 'POINTS' complex scaling factors by which transformer nominal impedance is to be multiplied to obtain the actual transformer impedance for the corresponding 'REALAR' value (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid transformer impedance correction table number
	IERR = 3	REALAR values are not in ascending order
	IERR = 4	REALAR value is outside of the range of -180 to +180
	IERR = 5	table has fewer than 2 points
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - point is beyond previous end of table - invalid scaling factor

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.30. INDUCTION_MACHINE_CHNG

This API routine is obsolete. It has been replaced by the API routine induction_machine_chng_2 .

Use this API routine to modify the data of an existing induction machine in the working case (see PSSE Program Operation Manual, Induction Machine Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_INDUCTION_MACHINE_CHNG IBUS 'ID' INTGAR(1)..INTGAR(9)
 REALAR(1)..REALAR(23)
```

Python command syntax:

```
ierr = induction_machine_chng(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL INDUCTION_MACHINE_CHNG(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (9)	array of 9 elements specifying (input).	
	INTGAR(1)	STAT, induction machine status (1 by default)
	INTGAR(2)	SCODE, machine standard type (1 for NEMA, 2 for IEC) (1 by default)
	INTGAR(3)	DCODE, machine design code (0 for custom, 1 through 5 for MEMA designs A through E) (2 by default)
	INTGAR(4)	AREA, area number (area of bus IBUS by default)
	INTGAR(5)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(6)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(7)	TCODE, type of mechanical torque variation (1 for simple power law, 2 for WECC model) (1 by default)
	INTGAR(8)	BCODE, base machine power code (1 for mechanical power 2 for electrical power drawn) (1 by default)
	INTGAR(9)	PCODE, power setpoint code (1 for mechanical power 2 for electrical power drawn) (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real REALAR (23)	array of 23 elements specifying (input).	
	REALAR(1)	MBASE, machine base power (SBASE by default)
	REALAR(2)	RATEKV, machine rated voltage (kV line to line); 0.0 for bus base voltage (0.0 by default)
	REALAR(3)	PSET, machine power setpoint in MW (positive for a motor) (0.0 by default)
	REALAR(4)	H, machine inertia in pu on machine base power (1.0 by default)
	REALAR(5)	A, load torque variation constant (1.0 by default)
	REALAR(6)	B, load torque variation constant (1.0 by default)
	REALAR(7)	D, load torque variation constant (1.0 by default)
	REALAR(8)	E, load torque variation constant (1.0 by default)
	REALAR(9)	RA, armature resistance (0.02494 by default)
	REALAR(10)	XA, armature leakage reactance (0.07482 by default)
	REALAR(11)	XM, magnetizing reactance (2.32786 by default)
	REALAR(12)	R1, first cage resistance (0.02078 by default)
	REALAR(13)	X1, first cage reactance (0.09145 by default)
	REALAR(14)	R2, second cage resistance (0.12471 by default)
	REALAR(15)	X2, second cage reactance (0.03326 by default)
	REALAR(16)	X3, third reactance (0.0 by default)
	REALAR(17)	E1, first voltage point on saturation curve (1.0 by default)
	REALAR(18)	SE1, saturation factor at E1 (0.04 by default)
	REALAR(19)	E2, second voltage point on saturation curve (1.2 by default)
	REALAR(20)	SE2, saturation factor at E2 (0.2 by default)
	REALAR(21)	IA1, starting stator current for applying stator leakage reactance saturation (1.0 by default)
	REALAR(22)	IA2, stator current at wiich the stator leakage reactance is fully saturated (5.0 by default)
	REALAR(23)	XAMULT, Xa multiplier at full leakage reactance saturation (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	machine identifier is more than two characters
IERR = 5	induction machine not found
IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid machine status. - invalid machine standard code. - invalid machine design code. - invalid area, zone or owner number. - invalid mechanical torque variation method code. - invalid base power code. - invalid power setpoint code. - machine base power is not positive. - machine rated voltage is negative. - machine rated voltage is ignored because the bus has no base voltage specified for it. - bus base voltage and machine rated voltage differ by more than 20%. - power setpoint exceeds the threshold determined from the standard code and the power setpoint code. - invalid machine inertia. - magnitude of load torque variation constant D is not between 0.2 and 5.0. - the sign of load torque variation constant D is not consistent with that of the machine power setpoint. - machine parameter is negative. - XM is not positive. - Rn or Xn exceeds 999.0. - Rn and Xn are both 0.0. - one of R1 and Xn is 999.0 and the other is not. - E1 is greater than E2; interchanging E1 and E2. - E1 and E2 are equal. - SE1 is greater than SE2; interchanging SE1 and SE2. - IA1 is greater than IA2; interchanging IA1 and IA2. - IA1 and IA2 are equal. - XAMULT must be between 0.0 and 1.0. - area, zone or owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.31. INDUCTION_MACHINE_CHNG_2

Use this API routine to modify the data of an existing induction machine in the working case (see PSSE Program Operation Manual, Induction Machine Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INDUCTION_MACHINE_CHNG_2 IBUS  'ID'  INTGAR(1)..INTGAR(9)
 REALAR(1)..REALAR(23) IMNAMEAR
```

Python command syntax:

```
ierr = induction_machine_chng_2(ibus, id, intgar, realar, imnamear)
```

Fortran command syntax:

```
CALL INDUCTION_MACHINE_CHNG_2(IBUS, ID, INTGAR, REALAR, IMNAMEAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (9)	array of 9 elements specifying (input).	
	INTGAR(1)	STAT, induction machine status (1 by default)
	INTGAR(2)	SCODE, machine standard type (1 for NEMA, 2 for IEC) (1 by default)
	INTGAR(3)	DCODE, machine design code (0 for custom, 1 through 5 for MEMA designs A through E) (2 by default)
	INTGAR(4)	AREA, area number (area of bus IBUS by default)
	INTGAR(5)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(6)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(7)	TCODE, type of mechanical torque variation (1 for simple power law, 2 for WECC model) (1 by default)
	INTGAR(8)	BCODE, base machine power code (1 for mechanical power 2 for electrical power drawn) (1 by default)
	INTGAR(9)	PCODE, power setpoint code (1 for mechanical power 2 for electrical power drawn) (1 by default)
Real REALAR (23)	array of 23 elements specifying (input).	
	REALAR(1)	MBASE, machine base power (SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(2)	RATEKV, machine rated voltage (kV line to line); 0.0 for bus base voltage (0.0 by default)
	REALAR(3)	PSET, machine power setpoint in MW (positive for a motor) (0.0 by default)
	REALAR(4)	H, machine inertia in pu on machine base power (1.0 by default)
	REALAR(5)	A, load torque variation constant (1.0 by default)
	REALAR(6)	B, load torque variation constant (1.0 by default)
	REALAR(7)	D, load torque variation constant (1.0 by default)
	REALAR(8)	E, load torque variation constant (1.0 by default)
	REALAR(9)	RA, armature resistance (0.02494 by default)
	REALAR(10)	XA, armature leakage reactance (0.07482 by default)
	REALAR(11)	XM, magnetizing reactance (2.32786 by default)
	REALAR(12)	R1, first cage resistance (0.02078 by default)
	REALAR(13)	X1, first cage reactance (0.09145 by default)
	REALAR(14)	R2, second cage resistance (0.12471 by default)
	REALAR(15)	X2, second cage reactance (0.03326 by default)
	REALAR(16)	X3, third reactance (0.0 by default)
	REALAR(17)	E1, first voltage point on saturation curve (1.0 by default)
	REALAR(18)	SE1, saturation factor at E1 (0.04 by default)
	REALAR(19)	E2, second voltage point on saturation curve (1.2 by default)
	REALAR(20)	SE2, saturation factor at E2 (0.2 by default)
	REALAR(21)	IA1, starting stator current for applying stator leakage reactance saturation (1.0 by default)
	REALAR(22)	IA2, stator current at wiich the stator leakage reactance is fully saturated (5.0 by default)
	REALAR(23)	XAMULT, Xa multiplier at full leakage reactance saturation (1.0 by default)
character*40 IMNAMEAR	machine name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	machine identifier is more than two characters
IERR = 5	induction machine not found
IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid machine status. - invalid machine standard code. - invalid machine design code. - invalid area, zone or owner number. - invalid mechanical torque variation method code. - invalid base power code. - invalid power setpoint code. - machine base power is not positive. - machine rated voltage is negative. - machine rated voltage is ignored because the bus has no base voltage specified for it. - bus base voltage and machine rated voltage differ by more than 20%. - power setpoint exceeds the threshold determined from the standard code and the power setpoint code. - invalid machine inertia. - magnitude of load torque variation constant D is not between 0.2 and 5.0. - the sign of load torque variation constant D is not consistent with that of the machine power setpoint. - machine parameter is negative. - XM is not positive. - Rn or Xn exceeds 999.0. - Rn and Xn are both 0.0. - one of R1 and Xn is 999.0 and the other is not. - E1 is greater than E2; interchanging E1 and E2. - E1 and E2 are equal. - SE1 is greater than SE2; interchanging SE1 and SE2. - IA1 is greater than IA2; interchanging IA1 and IA2. - IA1 and IA2 are equal. - XAMULT must be between 0.0 and 1.0. - area, zone or owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.32. INDUCTION_MACHINE_DATA

This API routine is obsolete. It has been replaced by the API routine induction_machine_data_2 .

Use this API routine to modify the data of an existing induction machine in the working case, or to add a new induction machine to the working case (see PSSE Program Operation Manual, Induction Machine Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_INDUCTION_MACHINE_DATA IBUS 'ID' INTGAR(1)..INTGAR(9)
 REALAR(1)..REALAR(23)
```

Python command syntax:

```
ierr = induction_machine_data(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL INDUCTION_MACHINE_API(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (9)	array of 9 elements specifying (input).	
	INTGAR(1)	STAT, induction machine status (1 by default)
	INTGAR(2)	SCODE, machine standard type (1 for NEMA, 2 for IEC) (1 by default)
	INTGAR(3)	DCODE, machine design code (0 for custom, 1 through 5 for MEMA designs A through E) (2 by default)
	INTGAR(4)	AREA, area number (area of bus IBUS by default)
	INTGAR(5)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(6)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(7)	TCODE, type of mechanical torque variation (1 for simple power law, 2 for WECC model) (1 by default)
	INTGAR(8)	BCODE, base machine power code (1 for mechanical power 2 for electrical power drawn) (1 by default)
	INTGAR(9)	PCODE, power setpoint code (1 for mechanical power 2 for electrical power drawn) (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real REALAR (23)	array of 23 elements specifying (input).	
	REALAR(1)	MBASE, machine base power (SBASE by default)
	REALAR(2)	RATEKV, machine rated voltage (kV line to line); 0.0 for bus base voltage (0.0 by default)
	REALAR(3)	PSET, machine power setpoint in MW (positive for a motor) (0.0 by default)
	REALAR(4)	H, machine inertia in pu on machine base power (1.0 by default)
	REALAR(5)	A, load torque variation constant (1.0 by default)
	REALAR(6)	B, load torque variation constant (1.0 by default)
	REALAR(7)	D, load torque variation constant (1.0 by default)
	REALAR(8)	E, load torque variation constant (1.0 by default)
	REALAR(9)	RA, armature resistance (0.02494 by default)
	REALAR(10)	XA, armature leakage reactance (0.07482 by default)
	REALAR(11)	XM, magnetizing reactance (2.32786 by default)
	REALAR(12)	R1, first cage resistance (0.02078 by default)
	REALAR(13)	X1, first cage reactance (0.09145 by default)
	REALAR(14)	R2, second cage resistance (0.12471 by default)
	REALAR(15)	X2, second cage reactance (0.03326 by default)
	REALAR(16)	X3, third reactance (0.0 by default)
	REALAR(17)	E1, first voltage point on saturation curve (1.0 by default)
	REALAR(18)	SE1, saturation factor at E1 (0.04 by default)
	REALAR(19)	E2, second voltage point on saturation curve (1.2 by default)
	REALAR(20)	SE2, saturation factor at E2 (0.2 by default)
	REALAR(21)	IA1, starting stator current for applying stator leakage reactance saturation (1.0 by default)
	REALAR(22)	IA2, stator current at wiich the stator leakage reactance is fully saturated (5.0 by default)
	REALAR(23)	XAMULT, Xa multiplier at full leakage reactance saturation (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	machine identifier is more than two characters
IERR = 3	blank machine identifier is invalid
IERR = 4	induction machine table is full
IERR = 5	node-breaker terminal connection table is full
IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid machine status. - invalid machine standard code. - invalid machine design code. - invalid area, zone or owner number. - invalid mechanical torque variation method code. - invalid base power code. - invalid power setpoint code. - machine base power is not positive. - machine rated voltage is negative. - machine rated voltage is ignored because the bus has no base voltage specified for it. - bus base voltage and machine rated voltage differ by more than 20%. - power setpoint exceeds the threshold determined from the standard code and the power setpoint code. - invalid machine inertia. - magnitude of load torque variation constant D is not between 0.2 and 5.0. - the sign of load torque variation constant D is not consistent with that of the machine power setpoint. - machine parameter is negative. - XM is not positive. - Rn or Xn exceeds 999.0. - Rn and Xn are both 0.0. - one of R1 and Xn is 999.0 and the other is not. - E1 is greater than E2; interchanging E1 and E2. - E1 and E2 are equal. - SE1 is greater than SE2; interchanging SE1 and SE2. - IA1 is greater than IA2; interchanging IA1 and IA2. - IA1 and IA2 are equal. - XAMULT must be between 0.0 and 1.0. - area, zone or owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.33. INDUCTION_MACHINE_DATA_2

Use this API routine to modify the data of an existing induction machine in the working case, or to add a new induction machine to the working case (see PSSE Program Operation Manual, Induction Machine Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INDUCTION_MACHINE_DATA_2 IBUS 'ID' INTGAR(1)..INTGAR(9)
 REALAR(1)..REALAR(23) INDNAMEAR
```

Python command syntax:

```
ierr = induction_machine_data_2(ibus, id, intgar, realar, indnamear)
```

Fortran command syntax:

```
CALL INDUCTION_MACHINE_API_2(IBUS, ID, INTGAR, REALAR, INDNAMEAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (9)	array of 9 elements specifying (input).	
	INTGAR(1)	STAT, induction machine status (1 by default)
	INTGAR(2)	SCODE, machine standard type (1 for NEMA, 2 for IEC) (1 by default)
	INTGAR(3)	DCODE, machine design code (0 for custom, 1 through 5 for MEMA designs A through E) (2 by default)
	INTGAR(4)	AREA, area number (area of bus IBUS by default)
	INTGAR(5)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(6)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(7)	TCODE, type of mechanical torque variation (1 for simple power law, 2 for WECC model) (1 by default)
	INTGAR(8)	BCODE, base machine power code (1 for mechanical power 2 for electrical power drawn) (1 by default)
	INTGAR(9)	PCODE, power setpoint code (1 for mechanical power 2 for electrical power drawn) (1 by default)
Real REALAR (23)	array of 23 elements specifying (input).	
	REALAR(1)	MBASE, machine base power (SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(2)	RATEKV, machine rated voltage (kV line to line); 0.0 for bus base voltage (0.0 by default)
	REALAR(3)	PSET, machine power setpoint in MW (positive for a motor) (0.0 by default)
	REALAR(4)	H, machine inertia in pu on machine base power (1.0 by default)
	REALAR(5)	A, load torque variation constant (1.0 by default)
	REALAR(6)	B, load torque variation constant (1.0 by default)
	REALAR(7)	D, load torque variation constant (1.0 by default)
	REALAR(8)	E, load torque variation constant (1.0 by default)
	REALAR(9)	RA, armature resistance (0.02494 by default)
	REALAR(10)	XA, armature leakage reactance (0.07482 by default)
	REALAR(11)	XM, magnetizing reactance (2.32786 by default)
	REALAR(12)	R1, first cage resistance (0.02078 by default)
	REALAR(13)	X1, first cage reactance (0.09145 by default)
	REALAR(14)	R2, second cage resistance (0.12471 by default)
	REALAR(15)	X2, second cage reactance (0.03326 by default)
	REALAR(16)	X3, third reactance (0.0 by default)
	REALAR(17)	E1, first voltage point on saturation curve (1.0 by default)
	REALAR(18)	SE1, saturation factor at E1 (0.04 by default)
	REALAR(19)	E2, second voltage point on saturation curve (1.2 by default)
	REALAR(20)	SE2, saturation factor at E2 (0.2 by default)
	REALAR(21)	IA1, starting stator current for applying stator leakage reactance saturation (1.0 by default)
	REALAR(22)	IA2, stator current at wiich the stator leakage reactance is fully saturated (5.0 by default)
	REALAR(23)	XAMULT, Xa multiplier at full leakage reactance saturation (1.0 by default)
character*40 INDNAMEAR	machine name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	machine identifier is more than two characters
IERR = 3	blank machine identifier is invalid
IERR = 4	induction machine table is full
IERR = 5	node-breaker terminal connection table is full
IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid machine status. - invalid machine standard code. - invalid machine design code. - invalid area, zone or owner number. - invalid mechanical torque variation method code. - invalid base power code. - invalid power setpoint code. - machine base power is not positive. - machine rated voltage is negative. - machine rated voltage is ignored because the bus has no base voltage specified for it. - bus base voltage and machine rated voltage differ by more than 20%. - power setpoint exceeds the threshold determined from the standard code and the power setpoint code. - invalid machine inertia. - magnitude of load torque variation constant D is not between 0.2 and 5.0. - the sign of load torque variation constant D is not consistent with that of the machine power setpoint. - machine parameter is negative. - XM is not positive. - Rn or Xn exceeds 999.0. - Rn and Xn are both 0.0. - one of R1 and Xn is 999.0 and the other is not. - E1 is greater than E2; interchanging E1 and E2. - E1 and E2 are equal. - SE1 is greater than SE2; interchanging SE1 and SE2. - IA1 is greater than IA2; interchanging IA1 and IA2. - IA1 and IA2 are equal. - XAMULT must be between 0.0 and 1.0. - area, zone or owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.34. INTERFACEELEMENT_CHNG

Use this API to modify the data of an existing interface element in the working case (see PSSE Program Operation Manual, Interface Element Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INTERFACEELEMENT_CHNG NAMEAR ELETYP INTBUS(1)...INTBUS(3) DEVNAM
 INTGAR(1)..INTGAR(2) REALAR(1)
```

Python command syntax:

```
ierr = interfaceelement_chng(namear, eletyp, intbus, devnam, intgar,
 realar)
```

Fortran command syntax:

```
CALL INTERFACEELEMENT_CHNG(NAMEAR, ELETYP, INTBUS, DEVNAM, INTGAR, REALAR,
 IERR)
```

Where:

Character*40 NAMEAR	interface name (input).	
Integer ELETYP	interface element type (input; no default allowed).	
Integer INTBUS (3)	array of 3 elements specifying (input).	
	INTBUS(1)	from bus number or from area/zone number (alias is FBUS) (0 by default)
	INTBUS(2)	to bus number or to area/zone number (alias is TBUS) (0 by default)
	INTBUS(3)	the last bus of a three winding transformer (alias is LBUS) (0 by default)
Character*40 DEVNAM	interface element device id or device name (input).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	flow direction, flow into the from bus or out of the from bus (alias is DIRECTION) (1 by default)
	INTGAR(2)	Metered bus end where the flow quantiity is taken (alias is METUS) (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	percentage of interface element flow contributed to interface flow (alias is weight) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	Invalid interface name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Interface not found
IERR = 3	Interface element type not found
IERR = 4	The bus of Interface element not found
IERR = 5	Interface element not found
IERR = 6	Interface element device id is too long
IERR = 7	Interface element table is full
IERR = 8	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Interface name is too long - Invalid interface numnber - Invalid interface type - Interface number has been used - Incorrect interface flow limit set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.35. INTERFACEELEMENT_DATA

Use this API to modify the data of an existing interface element in the working case or to add a new interface element to the working case (see PSSE Program Operation Manual, interface Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INTERFACEELEMENT_DATA NAMEAR ELETYP INTBUS(1)...INTBUS(3) DEVNAM
 INTGAR(1)..INTGAR(2) REALAR(1)
```

Python command syntax:

```
ierr = interfaceelement_data(namear, eletyp, intbus, devnam, intgar,
 realar)
```

Fortran command syntax:

```
CALL INTERFACEELEMENT_API(NAMEAR, ELETYP, INTBUS, DEVNAM, INTGAR, REALAR,
 IERR)
```

Where:

Character*40 NAMEAR	interface name (input).	
Integer ELETYP	interface element type (input; no default allowed).	
Integer INTBUS (3)	array of 3 elements specifying (input).	
	INTBUS(1)	from bus number or from area/zone number (alias is FBUS) (0 by default)
	INTBUS(2)	to bus number or to area/zone number (alias is TBUS) (0 by default)
	INTBUS(3)	the last bus of a three winding transformer (alias is LBUS) (0 by default)
Character*40 DEVNAM	interface element device id or device name (input).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	flow direction, flow into the from bus or out of the from bus (alias is DIRECTION) (1 by default)
	INTGAR(2)	Metered bus end where the flow quantiity is taken (alias is METUS) (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	percentage of interface element flow contributed to interface flow (alias is weight) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	Invalid interface name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Interface not found
IERR = 3	Interface element type not found
IERR = 4	The bus of Interface element not found
IERR = 5	Interface element not found
IERR = 6	Interface element device id is too long
IERR = 7	Interface element table is full
IERR = 8	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Interface name is too long - Invalid interface numnber - Invalid interface type - Interface number has been used - Incorrect interface flow limit set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.36. INTERFACE_CHNG

Use this API to modify the data of an existing interface in the working case (see PSSE Program Operation Manual, Interface Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INTERFACE_CHNG NAMEAR INTGAR(1)..INTGAR(2) REALAR(1)..REALAR(24)
```

Python command syntax:

```
ierr = interface_chng(namear, intgar, realar)
```

Fortran command syntax:

```
CALL INTERFACE_CHNG(NAMEAR, INTGAR, REALAR, IERR)
```

Where:

Character*40 NAMEAR	interface name (input).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	NUMBER, interface number (alias is NUMBER) (next available interface number by default)
	INTGAR(2)	TYPE, interface monitor flow type (alias is TYPE) (1 by default)
Real REALAR (24)	array of 24 elements specifying (input).	
	REALAR(1)	RATEMAX1, rating set 1 maximum interface rating (alias is RATEMAX1) (9999.0 by default)
	REALAR(2)	RATEMAX2, rating set 2 maximum interface rating (alias is RATEMAX2) (9999.0 by default)
	REALAR(3)	RATEMAX3, rating set 3 maximum interface rating (alias is RATEMAX3) (9999.0 by default)
	REALAR(4)	RATEMAX4, rating set 4 maximum interface rating (alias is RATEMAX4) (9999.0 by default)
	REALAR(5)	RATEMAX5, rating set 5 maximum interface rating (alias is RATEMAX5) (9999.0 by default)
	REALAR(6)	RATEMAX6, rating set 6 maximum interface rating (alias is RATEMAX6) (9999.0 by default)
	REALAR(7)	RATEMAX7, rating set 7 maximum interface rating (alias is RATEMAX7) (9999.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(8)	RATEMAX8, rating set 8 maximum interface rating (alias is RATEMAX8) (9999.0 by default)
REALAR(9)	RATEMAX9, rating set 9 maximum interface rating (alias is RATEMAX9) (9999.0 by default)
REALAR(10)	RATEMAX10, rating set 10 maximum interface rating (alias is RATEMAX10) (9999.0 by default)
REALAR(11)	RATEMAX11, rating set 11 maximum interface rating (alias is RATEMAX11) (9999.0 by default)
REALAR(12)	RATEMAX12, rating set 12 maximum interface rating (alias is RATEMAX12) (9999.0 by default)
REALAR(13)	RATEMIN1, rating set 1 minmum interface rating (alias is RATEMIN1) (-9999.0 by default)
REALAR(14)	RATEMIN2, rating set 2 minmum interface rating (alias is RATEMIN2) (-9999.0 by default)
REALAR(15)	RATEMIN3, rating set 3 minmum interface rating (alias is RATEMIN3) (-9999.0 by default)
REALAR(16)	RATEMIN4, rating set 4 minmum interface rating (alias is RATEMIN4) (-9999.0 by default)
REALAR(17)	RATEMIN5, rating set 5 minmum interface rating (alias is RATEMIN5) (-9999.0 by default)
REALAR(18)	RATEMIN6, rating set 6 minmum interface rating (alias is RATEMIN6) (-9999.0 by default)
REALAR(19)	RATEMIN7, rating set 7 minmum interface rating (alias is RATEMIN7) (-9999.0 by default)
REALAR(20)	RATEMIN8, rating set 8 minmum interface rating (alias is RATEMIN8) (-9999.0 by default)
REALAR(21)	RATEMIN9, rating set 9 minmum interface rating (alias is RATEMIN9) (-9999.0 by default)
REALAR(22)	RATEMIN10, rating set 10 minmum interface rating (alias is RATEMIN10) (-9999.0 by default)
REALAR(23)	RATEMIN11, rating set 11 minmum interface rating (alias is RATEMIN11) (-9999.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(24)	RATEMIN12, rating set 12 minmum interface rating (alias is RATEMIN12) (-9999.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	Invalid interface name
	IERR = 2	Interface not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Interface name is too long - Invalid interface numnber - Invalid interface type - Interface number has been used - Incorrect interface flow limit set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.37. INTERFACE_DATA

Use this API to modify the data of an existing interface in the working case or to add a new interface to the working case (see PSSE Program Operation Manual, interface Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INTERFACE_DATA NAMEAR INTGAR(1)..INTGAR(2) REALAR(1)..REALAR(24)
```

Python command syntax:

```
ierr = interface_data(namear, intgar, realar)
```

Fortran command syntax:

```
CALL INTERFACE_API(NAMEAR, INTGAR, REALAR, IERR)
```

Where:

Character*40 NAMEAR	interface name (input).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	NUMBER, interface number (alias is NUMBER) (next available interface number by default)
	INTGAR(2)	TYPE, interface monitor flow type (alias is TYPE) (1 by default)
Real REALAR (24)	array of 24 elements specifying (input).	
	REALAR(1)	RATEMAX1, rating set 1 maximum interface rating (alias is RATEMAX1) (9999.0 by default)
	REALAR(2)	RATEMAX2, rating set 2 maximum interface rating (alias is RATEMAX2) (9999.0 by default)
	REALAR(3)	RATEMAX3, rating set 3 maximum interface rating (alias is RATEMAX3) (9999.0 by default)
	REALAR(4)	RATEMAX4, rating set 4 maximum interface rating (alias is RATEMAX4) (9999.0 by default)
	REALAR(5)	RATEMAX5, rating set 5 maximum interface rating (alias is RATEMAX5) (9999.0 by default)
	REALAR(6)	RATEMAX6, rating set 6 maximum interface rating (alias is RATEMAX6) (9999.0 by default)
	REALAR(7)	RATEMAX7, rating set 7 maximum interface rating (alias is RATEMAX7) (9999.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(8)	RATEMAX8, rating set 8 maximum interface rating (alias is RATEMAX8) (9999.0 by default)
REALAR(9)	RATEMAX9, rating set 9 maximum interface rating (alias is RATEMAX9) (9999.0 by default)
REALAR(10)	RATEMAX10, rating set 10 maximum interface rating (alias is RATEMAX10) (9999.0 by default)
REALAR(11)	RATEMAX11, rating set 11 maximum interface rating (alias is RATEMAX11) (9999.0 by default)
REALAR(12)	RATEMAX12, rating set 12 maximum interface rating (alias is RATEMAX12) (9999.0 by default)
REALAR(13)	RATEMIN1, rating set 1 minmum interface rating (alias is RATEMIN1) (-9999.0 by default)
REALAR(14)	RATEMIN2, rating set 2 minmum interface rating (alias is RATEMIN2) (-9999.0 by default)
REALAR(15)	RATEMIN3, rating set 3 minmum interface rating (alias is RATEMIN3) (-9999.0 by default)
REALAR(16)	RATEMIN4, rating set 4 minmum interface rating (alias is RATEMIN4) (-9999.0 by default)
REALAR(17)	RATEMIN5, rating set 5 minmum interface rating (alias is RATEMIN5) (-9999.0 by default)
REALAR(18)	RATEMIN6, rating set 6 minmum interface rating (alias is RATEMIN6) (-9999.0 by default)
REALAR(19)	RATEMIN7, rating set 7 minmum interface rating (alias is RATEMIN7) (-9999.0 by default)
REALAR(20)	RATEMIN8, rating set 8 minmum interface rating (alias is RATEMIN8) (-9999.0 by default)
REALAR(21)	RATEMIN9, rating set 9 minmum interface rating (alias is RATEMIN9) (-9999.0 by default)
REALAR(22)	RATEMIN10, rating set 10 minmum interface rating (alias is RATEMIN10) (-9999.0 by default)
REALAR(23)	RATEMIN11, rating set 11 minmum interface rating (alias is RATEMIN11) (-9999.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(24)	RATEMIN12, rating set 12 minmum interface rating (alias is RATEMIN12) (-9999.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	Invalid interface name
	IERR = 2	Interface table is full
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Interface name is too long - Invalid interface numnber - Invalid interface type - Interface number has been used - Incorrect interface flow limit set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.38. LOADTYPE_CHNG

Use this API to modify load type data in the working case (see PSSE Program Operation Manual, Load Type Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_LOADTYPE_CHNG 'NAME' 'DES'
```

Python command syntax:

```
ierr = loadtype_chng(name,des)
```

Fortran command syntax:

```
CALL LOADTYPE_CHNG(NAME,DES, IERR)
```

Where:

Character*12 NAME	load type name (input; no default allowed).	
character*72 DES	load type description (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	load type name not found
	IERR = 2	invalid load type name
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - No load is assigned to load type

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.39. LOADTYPE_DATA

Use this API to modify load type data in the working case (see PSSE Program Operation Manual, Load Type Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_LOADTYPE_DATA 'NAME' 'DES'
```

Python command syntax:

```
ierr = loadtype_data(name,des)
```

Fortran command syntax:

```
CALL LOADTYPE_DATA(NAME,DES, IERR)
```

Where:

Character*12 NAME	load type name (input; no default allowed).	
character*72 DES	load type description (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	load type table is full
	IERR = 2	invalid load type name
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - No load is assigned to load type

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.40. LOAD_CHNG_4

This API routine is obsolete. It has been replaced by the API routine load_chng_5 .

Use this API routine to modify the data of an existing load in the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_LOAD_CHNG_4  IBUS 'ID' INTGAR(1)..INTGAR(6) REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = load_chng_4(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL LOAD_CHNG_4(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
Real REALAR (6)	array of 6 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)
	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 5	load not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.41. LOAD_CHNG_5

This API routine is obsolete. It has been replaced by the API routine load_chng_6 .

Use this API routine to modify the data of an existing load in the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_LOAD_CHNG_5  IBUS ID INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(8)
```

Python command syntax:

```
ierr = load_chng_5(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL LOAD_CHNG_5(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
	INTGAR(7)	DGNFLG, distributed generation flag (0 = distributed generation not in-service; 1 = distributed generation in-service) (0 by default)
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
	REALAR(7)	PG, distributed generation real power (0.0 by default)
	REALAR(8)	QG, distributed generation reactive power (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 5	load not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.42. LOAD_CHNG_6

This API routine is obsolete. It has been replaced by the API routine load_chng_7 .

Use this API routine to modify the data of an existing load in the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LOAD_CHNG_6 IBUS ID INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(8) LODTYP
```

Python command syntax:

```
ierr = load_chng_6(ibus, id, intgar, realar, lodtyp)
```

Fortran command syntax:

```
CALL LOAD_CHNG_6(IBUS, ID, INTGAR, REALAR, LODTYP, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
	INTGAR(7)	DGNFLG, distributed generation flag (0 = distributed generation not in-service; 1 = distributed generation in-service) (0 by default)
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
	REALAR(7)	PG, distributed generation real power (0.0 by default)
	REALAR(8)	QG, distributed generation reactive power (0.0 by default)
character*12 LODTYP	load type description (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 5	load not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag - invalid distributed generation flag - load type description is more than 12 characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.43. LOAD_CHNG_7

Use this API routine to modify the data of an existing load in the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_LOAD_CHNG_7  IBUS ID INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(8)
 'LODTYP' 'LDNAME'
```

Python command syntax:

```
ierr = load_chng_7(ibus, id, intgar, realar, lodtyp, ldname)
```

Fortran command syntax:

```
CALL LOAD_CHNG_7(IBUS, ID, INTGAR, REALAR, LODTYP, LDNAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
	INTGAR(7)	DGNFLG, distributed generation flag (0 = distributed generation not in-service; 1 = distributed generation in-service) (0 by default)
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)
	REALAR(4)	IQ, constant current reactive load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
	REALAR(7)	PG, distributed generation real power (0.0 by default)
	REALAR(8)	QG, distributed generation reactive power (0.0 by default)
character*12 LODTYP	load type description (input; blank by default).	
character*40 LDNAME	load name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 5	load not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag - invalid distributed generation flag - load type description is more than 12 characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.44. LOAD_DATA

This API routine is obsolete. It has been replaced by the API routine load_data_3 .

Use this API to modify the data of an existing load in the working case or to add a new load to the working case.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_LOAD_DATA  IBUS  'ID'  INTGAR(1)..INTGAR(4)  REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = load_data(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL LOAD_API_2(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
Real REALAR (6)	array of 6 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)
	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR > 0	fatal error occurred
IERR = 1	bus not found
IERR = 2	load identifier is more than two characters
IERR = 3	blank load identifier is invalid
IERR = 4	load table is full
IERR = 6	prerequisite requirements for API are not me
IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - Area table is full. - Zone table is full. - Owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.45. LOAD_DATA_3

This API routine is obsolete. It has been replaced by the API routine load_data_4 .

Use this API to modify the data of an existing load in the working case or to add a new load to the working case.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_LOAD_DATA_3  IBUS 'ID' INTGAR(1)..INTGAR(5) REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = load_data_3(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL LOAD_API_3(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
Real REALAR (6)	array of 6 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)
	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 3	blank load identifier is invalid
	IERR = 4	load table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.46. LOAD_DATA_4

This API routine is obsolete. It has been replaced by the API routine load_data_5 .

Use this API routine to modify the data of an existing load in the working case, or to add a new load to the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_LOAD_DATA_4  IBUS 'ID' INTGAR(1)..INTGAR(6) REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = load_data_4(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL LOAD_API_4(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
Real REALAR (6)	array of 6 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)
	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 3	blank load identifier is invalid
	IERR = 4	load table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.47. LOAD_DATA_5

This API routine is obsolete. It has been replaced by the API routine load_data_6 .

Use this API routine to modify the data of an existing load in the working case, or to add a new load to the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_LOAD_DATA_5  IBUS ID INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(8)
```

Python command syntax:

```
ierr = load_data_5(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL LOAD_API_5(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
	INTGAR(7)	DGNFLG, distributed generation flag (0 = distributed generation not in-service; 1 = distributed generation in-service) (0 by default)
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
	REALAR(7)	PG, distributed generation real power (0.0 by default)
	REALAR(8)	QG, distributed generation reactive power (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 3	blank load identifier is invalid
	IERR = 4	load table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.48. LOAD_DATA_6

This API routine is obsolete. It has been replaced by the API routine load_data_7 .

Use this API routine to modify the data of an existing load in the working case, or to add a new load to the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LOAD_DATA_6 IBUS ID INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(8) LODTYP
```

Python command syntax:

```
ierr = load_data_6(ibus, id, intgar, realar, lodtyp)
```

Fortran command syntax:

```
CALL LOAD_API_6(IBUS, ID, INTGAR, REALAR, LODTYP, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
	INTGAR(7)	DGNFLG, distributed generation flag (0 = distributed generation not in-service; 1 = distributed generation in-service) (0 by default)
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(4)	IQ, constant current reactive load (0.0 by default)
	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
	REALAR(7)	PG, distributed generation real power (0.0 by default)
	REALAR(8)	QG, distributed generation reactive power (0.0 by default)
character*12 LODTYP	load type description (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 3	blank load identifier is invalid
	IERR = 4	load table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	node-breaker terminal connection table is full
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag - invalid distributed generation flag - load type description is more than 12 characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.49. LOAD_DATA_7

Use this API routine to modify the data of an existing load in the working case, or to add a new load to the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_LOAD_DATA_7 IBUS ID INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(8) 'LODTYP'
 'LDNAME'
```

Python command syntax:

```
ierr = load_data_7(ibus, id, intgar, realar, lodtyp, ldname)
```

Fortran command syntax:

```
CALL LOAD_API_7(IBUS, ID, INTGAR, REALAR, LODTYP, LDNAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STATUS, load status (1 by default)
	INTGAR(2)	AREA, area number (area of bus IBUS by default)
	INTGAR(3)	ZONE, zone number (zone of bus IBUS by default)
	INTGAR(4)	OWNER, owner number (owner of bus IBUS by default)
	INTGAR(5)	SCALE, load scaling flag (0 = fixed, non-conforming; 1 = scalable, conforming) (1 by default)
	INTGAR(6)	INTRPT, interruptible load flag (0 = non-interruptible; 1 = interruptible) (0 by default)
	INTGAR(7)	DGNFLG, distributed generation flag (0 = distributed generation not in-service; 1 = distributed generation in-service) (0 by default)
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	PL, constant power active load (0.0 by default)
	REALAR(2)	QL, constant power reactive load (0.0 by default)
	REALAR(3)	IP, constant current active load (0.0 by default)
	REALAR(4)	IQ, constant current reactive load (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(5)	YP, constant admittance active load (0.0 by default)
	REALAR(6)	YQ, constant admittance reactive load (0.0 by default)
	REALAR(7)	PG, distributed generation real power (0.0 by default)
	REALAR(8)	QG, distributed generation reactive power (0.0 by default)
character*12 LODTYP	load type description (input; blank by default).	
character*40 LDNAME	load name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 3	blank load identifier is invalid
	IERR = 4	load table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	node-breaker terminal connection table is full
	IERR = -1	data error, warning and information messages; one or more of: - invalid load status. - invalid area, zone or owner number. - area table is full. - zone table is full. - owner table is full. - invalid load scaling flag. - invalid interruptible load flag - invalid distributed generation flag - load type description is more than 12 characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.50. LONG_TITLE_DATA

Use this API to change the sixteen line long title in the working case (see PSSE Program Operation Manual, activity CHTI).

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_LONG_TITLE_DATA  'LNGLNS(1)'..'LNGLNS(16)'
```

Python command syntax:

```
ierr = long_title_data(lnglns)
```

Fortran command syntax:

```
CALL LONG_TITLE_API_2(LNGLNS, IERR)
```

Where:

Character*72 LNGLNS (16)	sixteen title lines (defaulted lines are unchanged) (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Title line exceeds 72 characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.51. MACHINE_CAP_CURVE_CHNG

Use this API routine to modify the capability curve data of an existing machine in the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MACHINE_CAP_CURVE_CHNG IBUS ID POINTS PGENAR(1)...PGENAR(POINTS)
 QMAXAR(1)...QMAXAR(POINTS) QMINAR(1)...QMINAR(POINTS)
```

Python command syntax:

```
ierr = machine_cap_curve_chng(ibus, id, points, pgenar, qmaxar, qminar)
```

Fortran command syntax:

```
CALL MACHINE_CAP_CURVE_CHNG(IBUS, ID, POINTS, PGENAR, QMAXAR, QMINAR,
 IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer POINTS	number of points specified in the capability curve (input).	
real PGENAR (POINTS)	array of 'POINTS' values specifying machine active power outputs in MW along the MW axis of the machine's capability curve (input).	
real QMAXAR (POINTS)	array of 'POINTS' values specifying machine maximum reactive power outputs in MVAR at the corresponding PGENAR values of the machine's capability curve (input).	
real QMINAR (POINTS)	array of 'POINTS' values specifying machine minimum reactive power outputs in MVAR at the corresponding PGENAR values of the machine's capability curve (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	invalid POINTS value
	IERR = 5	a motor capability curve is specified for a generator
	IERR = 6	a generator capability curve is specified for a motor
	IERR = 7	capability curve P"s for a generator are not in increasing order
	IERR = 8	capability curve P"s for a motor are not in decreasing order

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	capability curve not found for this machine
IERR = 10	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Machine active power is outside the limits of the capability curve. - For at least one curve point, QMAX is less than QMIN - Non-conventional machine with machine control mode greater than 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.52. MACHINE_CAP_CURVE_DATA

Use this API routine to specify or modify the capability curve data of an existing machine in the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MACHINE_CAP_CURVE_DATA IBUS ID POINTS PGENAR(1)...PGENAR(POINTS)
 QMAXAR(1)...QMAXAR(POINTS) QMINAR(1)...QMINAR(POINTS)
```

Python command syntax:

```
ierr = machine_cap_curve_data(ibus, id, points, pgenar, qmaxar, qminar)
```

Fortran command syntax:

```
CALL MACHINE_CAP_CURVE_DATA(IBUS, ID, POINTS, PGENAR, QMAXAR, QMINAR,
 IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer POINTS	number of points specified in the capability curve (input).	
real PGENAR (POINTS)	array of 'POINTS' values specifying machine active power outputs in MW along the MW axis of the machine's capability curve (input).	
real QMAXAR (POINTS)	array of 'POINTS' values specifying machine maximum reactive power outputs in MVAR at the corresponding PGENAR values of the machine's capability curve (input).	
real QMINAR (POINTS)	array of 'POINTS' values specifying machine minimum reactive power outputs in MVAR at the corresponding PGENAR values of the machine's capability curve (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	invalid POINTS value
	IERR = 5	a motor capability curve is specified for a generator
	IERR = 6	a generator capability curve is specified for a motor
	IERR = 7	capability curve P"s for a generator are not in increasing order

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	capability curve P"s for a motor are not in decreasing order
IERR = 10	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Machine active power is outside the limits of the capability curve. - For at least one curve point, QMAX is less than QMIN - Non-conventional machine with machine control mode greater than 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.53. MACHINE_CHNG_2

This API routine is obsolete. It has been replaced by the API routine machine_chng_3 .

Use this API routine to modify the data of an existing machine in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MACHINE_CHNG_2  IBUS  'ID'  INTGAR(1)..INTGAR(6)
 REALAR(1)..REALAR(17)
```

Python command syntax:

```
ierr = machine_chng_2(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL MACHINE_CHNG_2(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, wind machine reactive power limits mode (0 if this machine is not a wind machine) (0 by default)
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(8)	ZR, machine resistance (0.0 by default)
	REALAR(9)	ZX, machine reactance (1.0 by default)
	REALAR(10)	RT, step-up transformer resistance (0.0 by default)
	REALAR(11)	XT, step-up transformer reactance (0.0 by default)
	REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
	REALAR(13)	F1, first owner fraction (1.0 by default)
	REALAR(14)	F2, second owner fraction (1.0 by default)
	REALAR(15)	F3, third owner fraction (1.0 by default)
	REALAR(16)	F4, fourth owner fraction (1.0 by default)
	REALAR(17)	WPF, wind machine power factor (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at this bus
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid wind machine control mode - invalid wind machine power factor - invalid ZSORCE reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - non-conventional machine has negative active power - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - capability curve is assigned to a non-conventional machine with machine control mode greater than 1 - machine ownership table is full--only first n owners retained - owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.54. MACHINE_CHNG_3

Use this API routine to modify the data of an existing machine in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MACHINE_CHNG_3  IBUS  'ID'  INTGAR(1)..INTGAR(7)
 REALAR(1)..REALAR(17)
```

Python command syntax:

```
ierr = machine_chng_3(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL MACHINE_CHNG_3(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, non-conventional machine reactive power limits mode (0 if this machine is a conventional machine) (0 by default)
	INTGAR(7)	BASFLG, baseloaded machine flag
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)
	REALAR(8)	ZR, machine resistance (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(9)	ZX, machine reactance (1.0 by default)
	REALAR(10)	RT, step-up transformer resistance (0.0 by default)
	REALAR(11)	XT, step-up transformer reactance (0.0 by default)
	REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
	REALAR(13)	F1, first owner fraction (1.0 by default)
	REALAR(14)	F2, second owner fraction (1.0 by default)
	REALAR(15)	F3, third owner fraction (1.0 by default)
	REALAR(16)	F4, fourth owner fraction (1.0 by default)
	REALAR(17)	WPF, non-conventional machine power factor (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at this bus
	IERR = 8	machine not found
	IERR = 9	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid non-conventional (renewable and infeed type) machine control mode - invalid renewable machine power factor - invalid ZSORCE reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - machine ownership table is full--only first n owners retained - owner table is full - infeed machine QGEN is greater than QMAX - QGEN set equal to QMAX - infeed machine QGEN is less than QMIN - QGEN set equal to QMIN - capability curve is assigned to a non-conventional machine with machine control mode greater than 1 - invalid baseloaded machine flag - baseloaded machine has negative active power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.55. MACHINE_CHNG_4

This API routine is obsolete. It has been replaced by the API routine machine_chng_5 .

Use this API routine to modify the data of an existing machine in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_MACHINE_CHNG_4 IBUS 'ID' INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(17)
 NAME
```

Python command syntax:

```
ierr = machine_chng_4(ibus, id, intgar, realar, name)
```

Fortran command syntax:

```
CALL MACHINE_CHNG_4(IBUS, ID, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, non-conventional machine reactive power limits mode (0 if this machine is a conventional machine) (0 by default)
	INTGAR(7)	BASFLG, baseloaded machine flag
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(8)	ZR, machine resistance (0.0 by default)
	REALAR(9)	ZX, machine reactance (1.0 by default)
	REALAR(10)	RT, step-up transformer resistance (0.0 by default)
	REALAR(11)	XT, step-up transformer reactance (0.0 by default)
	REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
	REALAR(13)	F1, first owner fraction (1.0 by default)
	REALAR(14)	F2, second owner fraction (1.0 by default)
	REALAR(15)	F3, third owner fraction (1.0 by default)
	REALAR(16)	F4, fourth owner fraction (1.0 by default)
	REALAR(17)	WPF, non-conventional machine power factor (1.0 by default)
Character*40 NAME	name of the voltage droop control (input; " ").	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at this bus
	IERR = 8	machine not found
	IERR = 9	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid non-conventional (renewable and infeed type) machine control mode - invalid renewable machine power factor - invalid ZSORCE reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - machine ownership table is full--only first n owners retained - owner table is full - infeed machine QGEN is greater than QMAX - QGEN set equal to QMAX - infeed machine QGEN is less than QMIN - QGEN set equal to QMIN - capability curve is assigned to a non-conventional machine with machine control mode greater than 1 - invalid baseloaded machine flag - baseloaded machine has negative active power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.56. MACHINE_CHNG_5

Use this API routine to modify the data of an existing machine in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_MACHINE_CHNG_5  IBUS  'ID'  INTGAR(1)..INTGAR(7)
 REALAR(1)..REALAR(17),NAMEAR(1)...NAMEAR(2)
```

Python command syntax:

```
ierr = machine_chng_5(ibus, id, intgar, realar, namear)
```

Fortran command syntax:

```
CALL MACHINE_CHNG_5(IBUS, ID, INTGAR, REALAR, NAMEAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, non-conventional machine reactive power limits mode (0 if this machine is a conventional machine) (0 by default)
	INTGAR(7)	BASFLG, baseloaded machine flag
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)
	REALAR(8)	ZR, machine resistance (0.0 by default)
	REALAR(9)	ZX, machine reactance (1.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(10)	RT, step-up transformer resistance (0.0 by default)
	REALAR(11)	XT, step-up transformer reactance (0.0 by default)
	REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
	REALAR(13)	F1, first owner fraction (1.0 by default)
	REALAR(14)	F2, second owner fraction (1.0 by default)
	REALAR(15)	F3, third owner fraction (1.0 by default)
	REALAR(16)	F4, fourth owner fraction (1.0 by default)
	REALAR(17)	WPF, non-conventional machine power factor (1.0 by default)
Character*40 NAMEAR (2)	array of 2 elements specifying name of the voltage droop control and machine(input).	
	NAMEAR(1)	name of the voltage droop control
	NAMEAR(2)	name of the machine
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at this bus
	IERR = 8	machine not found
	IERR = 9	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid non-conventional (renewable and infeed type) machine control mode - invalid renewable machine power factor - invalid ZSORCE reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - machine ownership table is full--only first n owners retained - owner table is full - infeed machine QGEN is greater than QMAX - QGEN set equal to QMAX - infeed machine QGEN is less than QMIN - QGEN set equal to QMIN - capability curve is assigned to a non-conventional machine with machine control mode greater than 1 - invalid base-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

loaded machine flag - baseloaded machine has negative active power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.57. MACHINE_DATA

This API routine is obsolete. It has been replaced by the API routine machine_data_2 .

Use this API to modify the data of an existing machine in the working case or to add a new machine to a plant bus in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MACHINE_DATA  IBUS  'ID'  INTGAR(1)..INTGAR(5)  REALAR(1)..REALAR(16)
```

Python command syntax:

```
ierr = machine_data(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL MACHINE_API(IBUS, ID, INTGAR, REALAR, IERR, NERR)
```

Where:

Integer IBUS	bus number (input).	
Character*2 ID	machine identifier (input).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	STAT, machine status
	INTGAR(2)	O1, first owner number
	INTGAR(3)	O2, second owner number
	INTGAR(4)	O3, third owner number
	INTGAR(5)	O4, fourth owner number
Real REALAR (16)	array of 16 elements specifying (input).	
	REALAR(1)	PG, machine active power
	REALAR(2)	QG, machine reactive power
	REALAR(3)	QT, machine reactive power upper limit
	REALAR(4)	QB, machine reactive power lower limit
	REALAR(5)	PT, machine active power upper limit
	REALAR(6)	PB, machine active power lower limit
	REALAR(7)	MBASE, machine MVA base
	REALAR(8)	ZR, machine resistance
	REALAR(9)	ZX, machine reactance
	REALAR(10)	RT, step-up transformer resistance
	REALAR(11)	XT, step-up transformer reactance
	REALAR(12)	GTAP, step-up transformer tap ratio
	REALAR(13)	F1, first owner fraction
	REALAR(14)	F2, second owner fraction
	REALAR(15)	F3, third owner fraction

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(16)	F4, fourth owner fraction
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at bus
	IERR = 4	blank machine identifier is invalid
	IERR = 5	machine table full
	IERR = 6	machine ownership table full
	IERR = 7	prerequisite requirements for API are not met
	IERR = 8	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid ZSORCE reactance - invalid ZSORCE: (0.0,0.0) - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - machine ownership table is full--only first n owners retained - owner table is full
Integer NERR	number of entries in PBUF02 containing error messages (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.58. MACHINE_DATA_2

This API routine is obsolete. It has been replaced by the API routine machine_data_3 .

Use this API to modify the data of an existing machine in the working case, or to add a new machine to a plant bus in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_MACHINE_DATA_2  I  'ID'  INTGAR(1)..INTGAR(6)  REALAR(1)..REALAR(17)
```

Python command syntax:

```
ierr = machine_data_2(i, id, intgar, realar)
```

Fortran command syntax:

```
CALL MACHINE_API_2(I, ID, INTGAR, REALAR, IERR)
```

Where:

Integer I	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, wind machine reactive power limits mode (0 if this machine is not a wind machine) (0 by default)
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(8)	ZR, machine resistance (0.0 by default)
	REALAR(9)	ZX, machine reactance (1.0 by default)
	REALAR(10)	RT, step-up transformer resistance (0.0 by default)
	REALAR(11)	XT, step-up transformer reactance (0.0 by default)
	REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
	REALAR(13)	F1, first owner fraction (1.0 by default)
	REALAR(14)	F2, second owner fraction (1.0 by default)
	REALAR(15)	F3, third owner fraction (1.0 by default)
	REALAR(16)	F4, fourth owner fraction (1.0 by default)
	REALAR(17)	WPF, wind machine power factor (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at this bus
	IERR = 4	blank machine identifier is invalid
	IERR = 5	machine table is full
	IERR = 6	machine ownership table is full
	IERR = 8	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid wind machine control mode - invalid wind machine power factor - invalid ZSORCE: reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - non-conventional machine has negative active power - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - capability curve is assigned to a non-conventional machine with machine control mode greater than 1 - machine ownership table is full--only first n owners retained - owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.59. MACHINE_DATA_3

Use this API to modify the data of an existing machine in the working case, or to add a new machine to a plant bus in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MACHINE_DATA_3  IBUS  'ID'  INTGAR(1)..INTGAR(7)
 REALAR(1)..REALAR(17)
```

Python command syntax:

```
ierr = machine_data_3(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL MACHINE_API_3(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, non-conventional machine reactive power limits mode (0 if this machine is a conventional machine) (0 by default)
	INTGAR(7)	BASFLG, baseloaded machine flag
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)
	REALAR(8)	ZR, machine resistance (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(9)	ZX, machine reactance (1.0 by default)
	REALAR(10)	RT, step-up transformer resistance (0.0 by default)
	REALAR(11)	XT, step-up transformer reactance (0.0 by default)
	REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
	REALAR(13)	F1, first owner fraction (1.0 by default)
	REALAR(14)	F2, second owner fraction (1.0 by default)
	REALAR(15)	F3, third owner fraction (1.0 by default)
	REALAR(16)	F4, fourth owner fraction (1.0 by default)
	REALAR(17)	WPF, non-conventional machine power factor (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at this bus
	IERR = 4	blank machine identifier is invalid
	IERR = 5	machine table is full
	IERR = 6	machine ownership table is full
	IERR = 7	node-breaker terminal connection table is full
	IERR = 9	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid non-conventional (renewable and infeed type) machine control mode - invalid renewable machine power factor - invalid ZSORCE reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - machine ownership table is full--only first n owners retained - owner table is full - infeed machine QGEN is greater than QMAX - QGEN set equal to QMAX - infeed machine QGEN is less than QMIN - QGEN set equal to QMIN - capability curve is assigned to a non-conventional machine with machine control mode greater than 1 - invalid base-loaded machine flag - baseloaded machine has negative active power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.60. MACHINE_DATA_4

This API routine is obsolete. It has been replaced by the API routine machine_data_5 .

Use this API to modify the data of an existing machine in the working case, or to add a new machine to a plant bus in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_MACHINE_DATA_4 IBUS 'ID' INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(17)
 NAME
```

Python command syntax:

```
ierr = machine_data_4(ibus, id, intgar, realar, name)
```

Fortran command syntax:

```
CALL MACHINE_API_4(IBUS, ID, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, non-conventional machine reactive power limits mode (0 if this machine is a conventional machine) (0 by default)
	INTGAR(7)	BASFLG, baseloaded machine flag
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(8)	ZR, machine resistance (0.0 by default)
REALAR(9)	ZX, machine reactance (1.0 by default)
REALAR(10)	RT, step-up transformer resistance (0.0 by default)
REALAR(11)	XT, step-up transformer reactance (0.0 by default)
REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
REALAR(13)	F1, first owner fraction (1.0 by default)
REALAR(14)	F2, second owner fraction (1.0 by default)
REALAR(15)	F3, third owner fraction (1.0 by default)
REALAR(16)	F4, fourth owner fraction (1.0 by default)
REALAR(17)	WPF, non-conventional machine power factor (1.0 by default)

Character*40 NAME — name of the voltage droop control (input; " ").

Integer IERR — error code (output).

IERR = 0	no errors occurred
IERR > 0	fatal error occurred
IERR = 1	bus not found
IERR = 2	machine identifier is more than two characters
IERR = 3	no plant data at this bus
IERR = 4	blank machine identifier is invalid
IERR = 5	machine table is full
IERR = 6	machine ownership table is full
IERR = 7	node-breaker terminal connection table is full
IERR = 9	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid non-conventional (renewable and infeed type) machine control mode - invalid renewable machine power factor - invalid ZSORCE reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - machine ownership table is full--only first n owners retained - owner table is full - infeed machine QGEN is greater than QMAX - QGEN set equal to QMAX - infeed machine QGEN is less than QMIN - QGEN set equal to QMIN - capability curve is assigned to a non-conventional machine with machine

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

control mode greater than 1 - invalid base-loaded machine flag - baseloaded machine has negative active power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.61. MACHINE_DATA_5

Use this API to modify the data of an existing machine in the working case, or to add a new machine to a plant bus in the working case (see PSSE Program Operation Manual, Plant Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_MACHINE_DATA_5 IBUS 'ID' INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(17)
 NAMEAR(1)...NAMEAR(2)
```

Python command syntax:

```
ierr = machine_data_5(ibus, id, intgar, realar, namear)
```

Fortran command syntax:

```
CALL MACHINE_API_5(IBUS, ID, INTGAR, REALAR, NAMEAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (7)	array of 7 elements specifying (input).	
	INTGAR(1)	STAT, machine status (1 by default)
	INTGAR(2)	O1, first owner number (owner of bus I by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
	INTGAR(6)	WMOD, non-conventional machine reactive power limits mode (0 if this machine is a conventional machine) (0 by default)
	INTGAR(7)	BASFLG, baseloaded machine flag
Real REALAR (17)	array of 17 elements specifying (input).	
	REALAR(1)	PG, machine active power (0.0 by default)
	REALAR(2)	QG, machine reactive power (0.0 by default)
	REALAR(3)	QT, machine reactive power upper limit (9999.0 by default)
	REALAR(4)	QB, machine reactive power lower limit (-9999.0 by default)
	REALAR(5)	PT, machine active power upper limit (9999.0 by default)
	REALAR(6)	PB, machine active power lower limit (-9999.0 by default)
	REALAR(7)	MBASE, machine MVA base (SBASE by default)
	REALAR(8)	ZR, machine resistance (0.0 by default)
	REALAR(9)	ZX, machine reactance (1.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(10)	RT, step-up transformer resistance (0.0 by default)
	REALAR(11)	XT, step-up transformer reactance (0.0 by default)
	REALAR(12)	GTAP, step-up transformer tap ratio (1.0 by default)
	REALAR(13)	F1, first owner fraction (1.0 by default)
	REALAR(14)	F2, second owner fraction (1.0 by default)
	REALAR(15)	F3, third owner fraction (1.0 by default)
	REALAR(16)	F4, fourth owner fraction (1.0 by default)
	REALAR(17)	WPF, non-conventional machine power factor (1.0 by default)
Character*40 NAMEAR (2)	array of 2 elements specifying name of the voltage droop control and machine(input).	
	NAMEAR(1)	name of the voltage droop control
	NAMEAR(2)	name of the machine
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	no plant data at this bus
	IERR = 4	blank machine identifier is invalid
	IERR = 5	machine table is full
	IERR = 6	machine ownership table is full
	IERR = 7	node-breaker terminal connection table is full
	IERR = 9	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid machine status - invalid non-conventional (renewable and infeed type) machine control mode - invalid renewable machine power factor - invalid ZSORCE reactance - invalid owner number - invalid ownership fraction - no ownership data is specified - QMAX is less than QMIN - PMAX is less than PMIN - MBASE is not positive - machine is no longer treated as a non-conventional machine - non-conventional machine table is full--retained as a conventional machine - machine ownership table is full--only first n owners retained - owner table is full - infeed machine QGEN is greater than QMAX - QGEN set equal to QMAX - infeed machine QGEN is less than QMIN - QGEN set equal to QMIN - capability curve is assigned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

to a non-conventional machine with machine control mode greater than 1 - invalid base-loaded machine flag - baseloaded machine has negative active power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.62. MBID2DC

Use this API to change the name of the specified two-terminal dc line.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBID2DC  DC  DCNEW
```

Python command syntax:

```
ierr = mbid2dc(dc, dcnew)
```

Fortran command syntax:

```
CALL MBID2DC(NUM, NUMNEW, IERR)

CALL MBID2DC_II(NUM, NUMNEW, IERR)

CALL MBID2DC_IC(NUM, NAMENEW, IERR)

CALL MBID2DC_CI(NAME, NUMNEW, IERR)

CALL MBID2DC_CC(NAME, NAMENEW, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which the dc line name is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer or Character*12 DCNEW	quoted new name to be assigned to the two-terminal dc line; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which the dc line name is to be changed (input; no default allowed).	
Integer NUMNEW	new number to be assigned to the two-terminal dc line as a left-justified name (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line for which the dc line name is to be changed (input; no default allowed).	
Character*12 NAMENEW	new dc line name to be assigned to the two-terminal dc line (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	dc line DC not found
	IERR = 2	dc line DCNEW already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.63. MBID3WND

Use this API to change the identifier of the specified three-winding transformer.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBID3WND  IBUS  JBUS1  JBUS2  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbid3wnd(ibus, jbus1, jbus2, id, idnew)
```

Fortran command syntax:

```
CALL MBID3WND(IBUS, JBUS1, JBUS2, ID, IDNEW, IERR)
```

Where:

Integer IBUS	one of the buses connected by the three-winding transformer (input; no default allowed).	
Integer JBUS1	other bus connected by the three-winding transformer (input; no default allowed).	
Integer JBUS2	other bus connected by the three-winding transformer (input; no default allowed).	
Character*2 ID	present circuit identifier of the three-winding transformer for which the identifier is to be changed (input; '1' by default).	
Character*2 IDNEW	new identifier to be assigned to the three-winding transformer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	circuit ID bus IBUS to JBUS1 to JBUS2 not found
	IERR = 3	circuit IDNEW bus IBUS to JBUS1 to JBUS2 already exists
	IERR = 4	invalid new identifier; cannot start with &, * or @
	IERR = 5	a non-blank new identifier must be specified
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.64. MBIDATRN

Use this API to change the identifier of the specified inter-area transfer.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBIDATRN  FRMAREA  TOAREA  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidatrn(frmarea, toarea, id, idnew)
```

Fortran command syntax:

```
CALL MBIDATRN(FRMAREA, TOAREA, ID, IDNEW, IERR)
```

Where:

Integer FRMAREA	one of the areas involved in the transfer (input; no default allowed).	
Integer TOAREA	other area involved in the transfer (input; no default allowed).	
Character*1 ID	present inter-area transfer identifier (input; '1' by default).	
Character*1 IDNEW	new identifier to be assigned to the inter-area transfer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid area number
	IERR = 2	inter-area transfer ID from area FRMAREA to TOAREA not found
	IERR = 3	inter-area transfer ID from area FRMAREA to TOAREA already exists
	IERR = 4	invalid new identifier--must be 0 through 9 or A through Z
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.65. MBIDBRN

Use this API to change the identifier of the specified non-transformer branch or two-winding transformer.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBIDBRN  IBUS  JBUS  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidbrn(ibus, jbus, id, idnew)
```

Fortran command syntax:

```
CALL MBIDBRN(IBUS, JBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	one of the buses connected by the branch (input; no default allowed).	
Integer JBUS	other bus connected by the branch (input; no default allowed).	
Character*2 ID	present circuit identifier of the branch for which the identifier is to be changed (input; '1' by default).	
Character*2 IDNEW	new identifier to be assigned to the branch (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	circuit ID from IBUS to JBUS not found
	IERR = 3	circuit ID from IBUS to JBUS already exists
	IERR = 4	invalid new identifier; cannot start with &, * or @
	IERR = 5	a non-blank new identifier must be specified
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.66. MBIDFACTS

Use this API to change the name of the specified FACTS device.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBIDFACTS  FD  FDNEW
```

Python command syntax:

```
ierr = mbidfacts(fd, fdnew)
```

Fortran command syntax:

```
CALL MBIDFACTS(NUM, NUMNEW, IERR)

CALL MBIDFACTS_II(NUM, NUMNEW, IERR)

CALL MBIDFACTS_IC(NUM, NAMENEW, IERR)

CALL MBIDFACTS_CI(NAME, NUMNEW, IERR)

CALL MBIDFACTS_CC(NAME, NAMENEW, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device for which the FACTS device name is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer or Character*12 FDNEW	quoted new name to be assigned to the FACTS device; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device for which the FACTS device name is to be changed (input; no default allowed).	
Integer NUMNEW	new number to be assigned to the FACTS device as a left-justified name (input; no default allowed).	
Character*12 NAME	name of the FACTS device for which the FACTS device name is to be changed (input; no default allowed).	
Character*12 NAMENEW	new FACTS device name to be assigned to the FACTS device (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	FACTS device FD not found
	IERR = 2	FACTS device FDNEW already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.67. MBIDGNE

Use this API routine to change the GNE device name of the specified GNE device.

This API routine was first introduced in release 32.0.2.

Batch command syntax:

```
BAT_MBIDGNE  'NAME'  'NAMNEW'
```

Python command syntax:

```
ierr = mbidgne(name, namnew)
```

Fortran command syntax:

```
CALL MBIDGNE(NAME, NAMNEW, IERR)
```

Where:

Character*12 NAME	name of the GNE device for which the GNE device name is to be changed (input; no default allowed).	
Character*12 NAMNEW	new name to be assigned to the GNE device (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	GNE device NAME not found
	IERR = 2	GNE device NEWNAM already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.68. MBIDINDMAC

Use this API routine to change the identifier of the specified induction machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MBIDINDMAC  IBUS  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidindmac(ibus, id, idnew)
```

Fortran command syntax:

```
CALL MBIDINDMAC(IBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	bus to which induction machine ID is connected (input; no default allowed).	
Character*2 ID	present induction machine identifier of the induction machine for which the identifier is to be changed (input; '1' by default).	
Character*2 IDNEW	new identifier to be assigned to the induction machine (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	induction machine ID at bus IBUS not found
	IERR = 3	induction machine ID at bus IBUS already exists
	IERR = 4	a non-blank new identifier must be specified
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.69. MBIDINTF

Use this API to change the interface name of the specified interface flow model in the working case.

This API routine was first introduced in release 36.

Batch command syntax:

```
BAT_MBIDINTF 'NAME' 'NAMNEW'
```

Python command syntax:

```
ierr = mbidintf(name, namnew)
```

Fortran command syntax:

```
CALL MBIDINTF(NAME, NAMNEW, IERR)
```

Where:

Character*40 NAME	name of the interface for which the interface name is to be changed (input; no default allowed).	
Character*40 NAMNEW	new name to be assigned to the interface (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	interface NAME not found
	IERR = 2	interface NEWNAM already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.70. MBIDLOAD

Use this API to change the identifier of the specified load.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBIDLOAD  IBUS  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidload(ibus, id, idnew)
```

Fortran command syntax:

```
CALL MBIDLOAD(IBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	bus to which the load ID is connected (input; no default allowed).	
Character*2 ID	present load identifier of the load for which the identifier is to be changed (input; '1' by default).	
Character*2 IDNEW	new identifier to be assigned to the load (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	load ID at bus IBUS not found
	IERR = 3	load ID at bus IBUS already exists
	IERR = 4	a non-blank new identifier must be specified
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.71. MBIDMAC

Use this API to change the identifier of the specified machine.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBIDMAC  IBUS  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidmac(ibus, id, idnew)
```

Fortran command syntax:

```
CALL MBIDMAC(IBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	bus to which the machine ID is connected (input; no default allowed).	
Character*2 ID	present machine identifier of the machine for which the identifier is to be changed (input; '1' by default).	
Character*2 IDNEW	new identifier to be assigned to the machine (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine ID at bus IBUS not found
	IERR = 3	machine ID at bus IBUS already exists
	IERR = 4	a non-blank new identifier must be specified
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.72. MBIDMDC

Use this API to change the name of the specified multi-terminal dc line.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBIDMDC  DC  DCNEW
```

Python command syntax:

```
ierr = mbidmdc(dc, dcnew)
```

Fortran command syntax:

```
CALL MBIDMDC(NUM, NUMNEW, IERR)

CALL MBIDMDC_II(NUM, NUMNEW, IERR)

CALL MBIDMDC_IC(NUM, NAMENEW, IERR)

CALL MBIDMDC_CI(NAME, NUMNEW, IERR)

CALL MBIDMDC_CC(NAME, NAMENEW, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which the dc line name is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer or Character*12 DCNEW	quoted new name to be assigned to the multi-terminal dc line; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which the dc line name is to be changed (input; no default allowed).	
Integer NUMNEW	new number to be assigned to the multi-terminal dc line as a left-justified name (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line for which the dc line name is to be changed (input; no default allowed).	
Character*12 NAMENEW	new dc line name to be assigned to the multi-terminal dc line (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	dc line DC not found
	IERR = 2	dc line DCNEW already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.73. MBIDMSL

Use this API to change the identifier of the specified multi-section line.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MBIDMSL  IBUS  JBUS  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidmsl(ibus, jbus, id, idnew)
```

Fortran command syntax:

```
CALL MBIDMSL(IBUS, JBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	one of the endpoint buses connected by the multi-section line (input; no default allowed).	
Integer JBUS	other endpoint bus connected by the multi-section line (input; no default allowed).	
Character*2 ID	present multi-section identifier of the multi-section line for which the identifier is to be changed (input; '&1' by default).	
Character*2 IDNEW	new identifier to be assigned to the multi-section line (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	multi-section line ID from IBUS to JBUS not found
	IERR = 3	multi-section line ID from IBUS to JBUS already exists
	IERR = 4	invalid new identifier; must start with &
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.74. MBIDSHUNT

Use this API to change the identifier of the specified fixed bus shunt.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_MBIDSHUNT  IBUS  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidshunt(ibus, id, idnew)
```

Fortran command syntax:

```
CALL MBIDSHUNT(IBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	bus to which the shunt ID is connected (input; no default allowed).	
Character*2 ID	present shunt identifier of the shunt for which the identifier is to be changed (input; '1' by default).	
Character*2 IDNEW	new identifier to be assigned to the shunt (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	shunt ID at bus IBUS not found
	IERR = 3	shunt ID at bus IBUS already exists
	IERR = 4	a non-blank new identifier must be specified
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.75. MBIDSWSHUNT

Use this API to change the identifier of the specified switched bus shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MBIDSWSHUNT  IBUS  'ID'  'IDNEW'
```

Python command syntax:

```
ierr = mbidswshunt(ibus, id, idnew)
```

Fortran command syntax:

```
CALL MBIDSWSHUNT(IBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	bus to which switched shunt "ID" is connected (input; no default allowed).	
Character*2 ID	present shunt identifier of the shunt for which the identifier is to be changed (input; '1' by default).	
Character*2 IDNEW	new identifier to be assigned to the shunt (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt "ID" at bus IBUS not found
	IERR = 3	switched shunt "ID" at bus IBUS already exists
	IERR = 4	a non-blank new identifier must be specified
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.76. MBIDVD

Use this API to change the voltage droop control name of the specified voltage droop control in the working case.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_MBIDVD 'NAME' 'NAMNEW'
```

Python command syntax:

```
ierr = mbidvd(name, namnew)
```

Fortran command syntax:

```
CALL MBIDVD(NAME, NAMNEW, IERR)
```

Where:

Character*40 NAME	name of the voltage droop control for which the voltage droop control name is to be changed (input; no default allowed).	
Character*40 NAMNEW	new name to be assigned to the voltage droop control (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	voltage droop control NAME not found
	IERR = 2	voltage droop control NEWNAM already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.77. MBIDVSC

Use this API to change the VSC dc line name of the specified VSC dc line.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_MBIDVSC  'NAME'  'NAMNEW'
```

Python command syntax:

```
ierr = mbidvsc(name, namnew)
```

Fortran command syntax:

```
CALL MBIDVSC(NAME, NAMNEW, IERR)
```

Where:

Character*12 NAME	name of the VSC dc line for which the VSC dc line name is to be changed (input; no default allowed).	
Character*12 NAMNEW	new name to be assigned to the VSC dc line (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	dc line NAME not found
	IERR = 2	dc line NEWNAM already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.78. MBID_SWD_RATING

Use this API to change the SWD rating table name

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_MBID_SWD_RATING EXISTNAME NEWNAME
```

Python command syntax:

```
ierr = mbid_swd_rating (existname, newname)
```

Fortran command syntax:

```
CALL MBID_SWD_RATING (EXISTNAME, NEWNAME, IERR)
```

Where:

character*40 EXISTNAME	name of the SWD rating table to be changed (input; no default allowed).	
character*40 NEWNAME	New name to be assigned to the SWD rating table (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No Error
	IERR = 1	SWD rating table NOT found
	IERR = 2	Invalide SWD rating table name
	IERR = 3	New SWD rating table name being taken
	IERR = 4	Prerequisite requirements for function not met
	IERR = -1	data error, warning and information messages; one or more of: - New SWD rating table name is too long

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.79. MULTI_SECTION_LINE_DATA

Use this API to modify the definition of an existing multi-section line grouping in the working case or to add a new multi-section line grouping to the working case (see PSSE Program Operation Manual, Multi-Section Line Grouping Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MULTI_SECTION_LINE_DATA  IBUS  JBUS  'ID'  INTGAR(1)..INTGAR(10)
```

Python command syntax:

```
ierr = multi_section_line_data(ibus, jbus, id, intgar)
```

Fortran command syntax:

```
CALL MULTI_SECTION_LINE_API_2(IBUS, JBUS, ID, INTGAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 ID	a multi-section line identifier; ID(1:1) must be '&' (input; '&1' by default).	
Integer INTGAR (10)	array of 10 elements specifying (input).	
	INTGAR(1)	METBUS, metered end bus number (must be IBUS or JBUS) (IBUS by default)
	INTGAR(2)	DUM1 through DUM9 entries 2 through 10 are the bus numbers of the dummy buses encountered along this multi-section line, starting with the bus to which bus 'IBUS' is connected and ending with the bus to which bus 'JBUS' is connected; entries used are 2 through n+1, where n is the number of dummy buses; if n+1 is less than 10, entries n+2 through 10 must be set to 0 (No default is allowed for entry 2; entries 3 through 10 are 0)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	multi-section line identifier is more than two characters
	IERR = 3	multi-section line identifier does not start with &
	IERR = 4	no dummy buses specified
	IERR = 5	line section not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	circuit is already a member of another multi-section line
IERR = 7	bus appears more than once in the multi-section line path
IERR = 8	dummy bus does not have exactly two branches connected to it
IERR = 9	dummy bus is a converter bus of a dc line
IERR = 10	a FACTS device is connected to a dummy bus
IERR = 11	line section table is full
IERR = 12	multi-section line table is full
IERR = 13	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - branch status and bus type code changes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.80. MULTI_SECTION_LINE_EDIT

Use this API to modify the status and/or metered end designation of an existing multi-section line grouping in the working case (see PSSE Program Operation Manual, Multi-Section Line Grouping Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MULTI_SECTION_LINE_EDIT  IBUS  JBUS  'ID'  INTGAR(1)  INTGAR(2)
```

Python command syntax:

```
ierr = multi_section_line_edit(ibus, jbus, id, intgar)
```

Fortran command syntax:

```
CALL MULTI_SECTION_LINE_EDIT_API_2(IBUS, JBUS, ID, INTGAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 ID	multi-section line identifier; ID(1:1) must be '&' (input; '&1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	STAT, branch status (unchanged by default)
	INTGAR(1) = 1	for in-service
	INTGAR(1) = 0	for out-of-service
	INTGAR(2)	METBUS, metered end bus number (must be IBUS or JBUS) (unchanged by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	multi-section line identifier is more than two characters
	IERR = 3	multi-section line identifier does not start with &
	IERR = 4	multi-section line not found
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus. - invalid branch status

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.81. MULTI_TERM_DC_BUS_DATA

Use this API to modify the data of a dc bus of an existing multi-terminal dc line in the working case or to add a new dc bus to a multi-terminal dc line in the working case (see PSSE Program Operation Manual, Multi-Terminal DC Transmission Line Data). The dc line must have previously been specified via the MULTI_TERM_DC_LINE_DATA API.
just some stupid words to see if they show up

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MULTI_TERM_DC_BUS_DATA  DC  IDC  INTGAR(1)..INTGAR(5)  REALAR(1)
 'DCNAME'
```

Python command syntax:

```
ierr, realaro = multi_term_dc_bus_data(dc, idc, intgar, realari, dcname)
```

Fortran command syntax:

```
CALL MULTI_TERMINAL_DC_BUS_API(NUM, IDC, INTGAR, REALAR, DCNAME, IERR)

CALL MULTI_TERMINAL_DC_BUS_API_I(NUM, IDC, INTGAR, REALAR, DCNAME, IERR)

CALL MULTI_TERMINAL_DC_BUS_API_C(NAME, IDC, INTGAR, REALAR, DCNAME, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which data for a dc bus is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which data for a dc bus is to be specified; no default allowed	
Character*12 NAME	name of the multi-terminal dc line for which data for a dc bus is to be specified; no default allowed	
Integer IDC	dc bus number (must be between 1 and the number of dc buses in multi-terminal dc line DC) (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	IB, converter bus number or zero (0 by default)
	INTGAR(2)	AREA, area number (1 by default)
	INTGAR(3)	ZONE, zone number (1 by default)
	INTGAR(4)	OWNER, owner number (1 by default)
	INTGAR(5)	IDC2, second dc bus number (0 by default)
Real REALAR (2)	(first value is input; second value is output) (input (output).).	
Real REALARI (1)	array of 1 elements specifying one element corresponding to values REALAR(1) as described above(input).	
	REALARI(1)	RGRND, resistance to ground (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real REALARO (1)	array of 1 elements specifying one element corresponding to value REALAR(2) as described above(output). REALARO(1) dc voltage (kV) (returned)
Character*12 DCNAME	dc bus name (input; blank by default).
Integer IERR	error code (output). IERR = 0 no errors occurred IERR > 0 fatal error occurred IERR = 1 dc line not found IERR = 2 invalid dc bus number IERR = 3 prerequisite requirements for API are not met IERR = -1 data error, warning and information messages; one or more of: - data for lower numbered dc bus has not yet been entered. - converter bus not found. - bus is not a converter of this multi-terminal dc line. - this dc bus is not the first dc bus of any converter. - the first dc bus of some converter is set to this dc bus. - invalid second dc bus number. - this dc bus is being connected to a converter bus. - dc bus no longer connected to some other converter. - dc bus is being specified with no converter bus. - dc bus no longer connected to converter as its first dc bus. - dc bus no longer connected to converter as its second dc bus. - bus invalid as second dc bus because it is the first dc bus of another converter. - invalid area/zone/owner number. - area table is full. - zone table is full. - owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.82. MULTI_TERM_DC_CONVR_DATA

Use this API to modify the data of a dc converter of an existing multi-terminal dc line in the working case or to add a new converter to a multi-terminal dc line in the working case (see PSSE Program Operation Manual, Multi-Terminal DC Transmission Line Data). The dc line must have previously been specified via the MULTI_TERM_DC_LINE_DATA API.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MULTI_TERM_DC_CONVR_DATA  DC  INTGAR(1)..INTGAR(4)
 REALAR(1)..REALAR(13)
```

Python command syntax:

```
ierr, realaro = multi_term_dc_convr_data(dc, intgar, realari)
```

Fortran command syntax:

```
CALL MULTI_TERM_DC_CONVERTER_API(NUM,INTGAR,REALAR,IERR)

CALL MULTI_TERM_DC_CONVERTER_API_I(NUM,INTGAR,REALAR,IERR)

CALL MULTI_TERM_DC_CONVERTER_API_C(NAME,INTGAR,REALAR,IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which data for a converter is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which data for a converter is to be specified; no default allowed	
Character*12 NAME	name of the multi-terminal dc line for which data for a converter is to be specified; no default allowed	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IB, converter bus number (no default allowed)
	INTGAR(2)	N, number of bridges in series (1 by default)
	INTGAR(3)	CNVCOD, pole code (0 by default)
	INTGAR(3) >= 0	for a positive pole converter
	INTGAR(3) < 0	for a negative pole converter
	INTGAR(4)	VCCFLG, pole's voltage controlling converter flag (0 by default)
	INTGAR(4) <= 0	no
	INTGAR(4) > 0	yes
Real REALAR (17)	(first thirteen are input values; last four are output values) (input (output).).	
Real REALARI (13)	array of 13 elements specifying corresponding to values REALAR(1) through REALAR(13) as described above(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(1)	SETVL, scheduled current (amps), power (MW) or voltage (kv) (0.0 by default)
	REALARI(2)	MN, minimum firing angle (0.0 by default)
	REALARI(3)	MX, maximum firing angle (90.0 by default)
	REALARI(4)	TR, transformer ratio (1.0 by default)
	REALARI(5)	RC, commutating resistance (0.0 by default)
	REALARI(6)	XC, commutating reactance (0.0 by default)
	REALARI(7)	EBAS, primary base ac voltage (1.0 by default)
	REALARI(8)	TAP, tap setting (1.0 by default)
	REALARI(9)	TPMX, maximum tap setting (1.5 by default)
	REALARI(10)	TPMN, minimum tap setting (0.51 by default)
	REALARI(11)	TSTP, tap step (0.00625 by default)
	REALARI(12)	MARG, margin (pu) (0.0 by default)
	REALARI(13)	DCPF, converter participation factor (1.0 by default)
Real REALARO (4)	array of 4 elements specifying corresponding to values REALAR(14) through REALAR(17) as described above(output).	
	REALARO(1)	firing angle (returned)
	REALARO(2)	converter current (amps) (returned)
	REALARO(3)	active power into converter (MW or kW) (returned)
	REALARO(4)	reactive power into converter (MW or kW) (returned)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	dc line not found
	IERR = 2	no converter bus is specified
	IERR = 3	converter bus not found
	IERR = 4	too many converters specified
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - converter bus is a dummy bus of a multi-section line. - multi-section line grouping deleted. - invalid number of bridges. - invalid base ac voltage. - converter maximum angle is less than its minimum angle. - converter maximum tap is less than its minimum tap. - invalid tap step. - dc voltage setpoint is not positive. - scheduled voltage is less than mode switch voltage. - this converter bus is now on the other pole; it

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

is no longer the voltage controlling converter of its previous pole. - there is no voltage controlling converter on a pole. - this converter replaces another converter as its pole's voltage controlling converter. - this converter must remain its pole's voltage controlling converter because it is the only converter on the pole

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.83. MULTI_TERM_DC_LINE_CHNG

Use this API routine to modify the control mode and mode switch voltage of an existing multi-terminal dc line in the working case (see PSSE Program Operation Manual, Multi-Terminal DC Transmission Line Data). Converter data, dc bus data, and dc link data are specified via the MULTI_TERM_DC_CONVR_DATA, MULTI_TERM_DC_BUS_DATA, and MULTI_TERM_DC_LINK_DATA API routines respectively.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_MULTI_TERM_DC_LINE_CHNG  DC  INTGAR(1)  REALAR(1)
```

Python command syntax:

```
ierr, intgaro = multi_term_dc_line_chng(dc, intgari, realar)
```

Fortran command syntax:

```
CALL MULTI_TERMINAL_DC_LINE_CHNG(NUM, INTGAR, REALAR, IERR)

CALL MULTI_TERMINAL_DC_LINE_CHNG_I(NUM, INTGAR, REALAR, IERR)

CALL MULTI_TERMINAL_DC_LINE_CHNG_C(NAME, INTGAR, REALAR, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which data is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which data is to be specified; no default allowed	
Character*12 NAME	name of the multi-terminal dc line for which data is to be specified; no default allowed	
Integer INTGAR (8)	(first value is input; last seven are output values) (input (output).).	
Integer INTGARI (1)	array of 1 elements specifying one element corresponding to value INTGAR(1) as described above(input).	
	INTGARI(1)	MDC, control mode (0 by default)
Integer INTGARO (7)	array of 7 elements specifying corresponding to values INTGAR(2) through INTGAR(8) as described above(output).	
	INTGARO(1)	positive pole voltage controlling converter index (returned)
	INTGARO(2)	negative pole voltage controlling converter index, or zero if the negative pole is not being modeled (returned)
	INTGARO(3)	positive pole voltage controlling converter bus number (returned)
	INTGARO(4)	positive pole voltage controlling converter bus number (returned)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGARO(5)	number of converters (returned)
	INTGARO(6)	number of dc buses (returned)
	INTGARO(7)	number of dc links (returned)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	VCMOD, mode switch dc voltage (kV) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 2	blank dc line name is invalid
	IERR = 3	dc line not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid dc line control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.84. MULTI_TERM_DC_LINE_DATA

Use this API to modify the control mode and mode switch voltage of an existing multi-terminal dc line in the working case or to add a new multi-terminal dc line to the working case (see PSSE Program Operation Manual, Multi-Terminal DC Transmission Line Data). Converter data, dc bus data, and dc link data are specified via the MULTI_TERM_DC_CONVR_DATA, MULTI_TERM_DC_BUS_DATA, and MULTI_TERM_DC_LINK_DATA APIs respectively.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MULTI_TERM_DC_LINE_DATA  DC  INTGAR(1)  REALAR(1)
```

Python command syntax:

```
ierr, intgaro = multi_term_dc_line_data(dc, intgari, realar)
```

Fortran command syntax:

```
CALL MULTI_TERMINAL_DC_LINE_API(NUM, INTGAR, REALAR, IERR)

CALL MULTI_TERMINAL_DC_LINE_API_I(NUM, INTGAR, REALAR, IERR)

CALL MULTI_TERMINAL_DC_LINE_API_C(NAME, INTGAR, REALAR, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which data is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which data is to be specified; no default allowed
Character*12 NAME	name of the multi-terminal dc line for which data is to be specified; no default allowed
Integer INTGAR (8)	(first value is input; last seven are output values) (input (output).).
Integer INTGARI (1)	array of 1 elements specifying one element corresponding to value INTGAR(1) as described above(input).
	INTGARI(1) MDC, control mode (0 by default)
Integer INTGARO (7)	array of 7 elements specifying corresponding to values INTGAR(2) through INTGAR(8) as described above(output).
	INTGARO(1) positive pole voltage controlling converter index (returned)
	INTGARO(2) negative pole voltage controlling converter index, or zero if the negative pole is not being modeled (returned)
	INTGARO(3) positive pole voltage controlling converter bus number (returned)
	INTGARO(4) positive pole voltage controlling converter bus number (returned)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGARO(5)	number of converters (returned)
	INTGARO(6)	number of dc buses (returned)
	INTGARO(7)	number of dc links (returned)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	VCMOD, mode switch dc voltage (kV) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	multi-terminal dc line table is full
	IERR = 2	blank dc line name is invalid
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid dc line control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.85. MULTI_TERM_DC_LINK_DATA

Use this API to modify the data of a dc link of an existing multi-terminal dc line in the working case or to add a new dc link to a multi-terminal dc line in the working case (see PSSE Program Operation Manual, Multi-Terminal DC Transmission Line Data). The dc line must have previously been specified via the MULTI_TERM_DC_LINE_DATA API.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MULTI_TERM_DC_LINK_DATA  DC  IDC  JDC  'DCCKT'  INTGAR(1)  REALAR(1)
 REALAR(2)
```

Python command syntax:

```
ierr = multi_term_dc_link_data(dc,idc,jdc,dcckt,intgar,realar)
```

Fortran command syntax:

```
CALL MULTI_TERMINAL_DC_LINK_API(NUM, IDC, JDC, DCCKT, INTGAR, REALAR,
 IERR)

CALL MULTI_TERMINAL_DC_LINK_API_I(NUM, IDC, JDC, DCCKT, INTGAR, REALAR,
 IERR)

CALL MULTI_TERMINAL_DC_LINK_API_C(NAME, IDC, JDC, DCCKT, INTGAR, REALAR,
 IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which data for a dc link is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which data for a dc link is to be specified; no default allowed	
Character*12 NAME	name of the multi-terminal dc line for which data for a dc link is to be specified; no default allowed	
Integer IDC	dc bus number of from bus (must be between 1 and the number of dc buses in multi-terminal dc line DC) (input; no default allowed).	
Integer JDC	dc bus number of to bus (must be between 1 and the number of dc buses in multi-terminal dc line DC) (input; no default allowed).	
Character*1 DCCKT	circuit identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	METBUS, metered end dc bus number (IDC by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	RDC, dc link resistance (0.0 by default)
	REALAR(2)	LDC, dc link inductance (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	dc line not found
	IERR = 2	invalid dc bus number
	IERR = 3	circuit identifier is more than one character
	IERR = 4	dc bus not present in this multi-terminal dc line
	IERR = 5	branch from a bus to itself
	IERR = 6	too many dc links in this multi-terminal dc line
	IERR = 7	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus. - invalid dc link resistance. - invalid dc link inductance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.86. OWNER_DATA

Use this API to modify owner data in the working case (see PSSE Program Operation Manual, Interarea Transfer Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OWNER_DATA  IOWNER  'OWNAME'
```

Python command syntax:

```
ierr = owner_data(iowner, owname)
```

Fortran command syntax:

```
CALL OWNER_API_2(IOWNER, OWNAME, IERR)
```

Where:

Integer IOWNER	owner number between 1 and 9999 (input; no default allowed).	
Character*12 OWNAME	owner name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid owner number
	IERR = 2	owner table is full
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - no equipment is assigned to owner

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.87. PBUS_ADD_MOD

Use this API to add or modify transaction event participating bus data.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PBUS_ADD_MOD  EVENT  IBUS  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = pbus_add_mod(event, ibus, realar)
```

Fortran command syntax:

```
CALL PBUS_ADD_MOD(EVENT, IBUS, REALAR, IERR, NERR)
```

Where:

Integer EVENT	transaction event number (input).	
Integer IBUS	number of the participating bus (input).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	participating bus load value
	REALAR(2)	participating bus generation value
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	error occurred
	IERR = 2	invalid transaction event number
	IERR = 3	transaction event not found
	IERR > 3	unable to add or modify participating bus data; error condition detected in data access routine
	IERR = 10	prerequisite requirements for API are not met
	IERR = -1	warnings: - None
Integer NERR	number of error entries in PBUF02 (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.88. PBUS_DELETE

Use this API to remove a bus from the set of buses that are participating in a transaction event.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PBUS_DELETE  EVENT  IBUS
```

Python command syntax:

```
ierr = pbus_delete(event, ibus)
```

Fortran command syntax:

```
CALL PBUS_DELETE(EVENT, IBUS, IERR, NERR)
```

Where:

Integer EVENT	transaction event number (input).	
Integer IBUS	number of the participating bus (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid transaction event number
	IERR = 2	transaction event not found
	IERR = 3	prerequisite requirements for API are not met
Integer NERR	number of error entries in PBUF02 (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.89. PLANT_CHNG

This API routine is obsolete. It has been replaced by the API routine plant_chng_3 .

Use this API routine to modify the data of an existing plant in the working case, if the plant bus is not a bus section within a substation. Use plant_chng_3 if data is to be modified at a plant associated with a bus section.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_PLANT_CHNG  IBUS  INTGAR(1)  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = plant_chng(ibus, intgar, realar)
```

Fortran command syntax:

```
CALL PLANT_CHNG(IBUS, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	IREG, remote bus number (0 to control voltage at bus IBUS) (0 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	VS, scheduled voltage magnitude (1.0 by default)
	REALAR(2)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 3	no plant at bus
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid scheduled voltage - regulated bus not found - plant specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - RMPCT is not positive

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.90. PLANT_CHNG_3

This API routine is obsolete. It has been replaced by the API routine plant_chng_4 .

Use this API routine to modify the data of an existing plant that is part of a substation in the working case. If the substation has been split into multiple bus sections, use this API to designate the bus number and a node number within the bus section of the plant that the data is to be changed.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_PLANT_CHNG_3  IBUS  INODE INTGAR(1)  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = plant_chng_3(ibus, inode, intgar, realar)
```

Fortran command syntax:

```
CALL PLANT_CHNG_3(IBUS, INODE, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INODE	node number within the bus section for which the plant resides (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	IREG, remote bus number (0 to control voltage at bus IBUS) (0 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	VS, scheduled voltage magnitude (1.0 by default)
	REALAR(2)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 3	no plant at bus
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid scheduled voltage - regulated bus not found - plant specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - RMPCT is not positive

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.91. PLANT_CHNG_4

Use this API routine to modify the data of an existing plant that is part of a substation in the working case. If the substation has been split into multiple bus sections, use this API to designate the bus number and a node number within the bus section of the plant that the data is to be changed.

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_PLANT_CHNG_4  IBUS  INODE INTGAR(1)  INTGAR(2) REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = plant_chng_4(ibus, inode, intgar, realar)
```

Fortran command syntax:

```
CALL PLANT_CHNG_4(IBUS, INODE, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INODE	node number within the bus section for which the plant resides (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	IREG, remote bus number (0 to control voltage at bus IBUS) (0 by default)
	INTGAR(2)	NREG, regulated bus node number (0 to control voltage at the "parent" bus section of bus IBUS)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	VS, scheduled voltage magnitude (1.0 by default)
	REALAR(2)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 3	no plant at bus
	IERR = 4	invalid bus number
	IERR = 5	INODE is an invalid node number
	IERR = 6	bus IBUS does not belong to a substation
	IERR = 7	node INODE is not found in the substation of bus IBUS
	IERR = 8	node INODE is not connected to any bus section of bus IBUS

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	INODE must be 0 for a bus that is not yet a generator bus
IERR = 10	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid scheduled voltage - regulated bus not found - type code of regulated bus is not 1 or 2 - RMPCT is not positive - per unit scheduled voltage is outside of the band 0.5 to 1.5 p.u. - invalid node number - non-zero regulated node specified but regulated bus does not belong to a substation - regulated node not found - regulated node is not connected to any bus section of the regulated bus - a remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.92. PLANT_DATA

This API routine is obsolete. It has been replaced by the API routine plant_data_3 .

Use this API to modify the data of an existing plant in the working case, or to add a new plant to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PLANT_DATA  IBUS  INTGAR(1)  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = plant_data(ibus, intgar, realar)
```

Fortran command syntax:

```
CALL PLANT_API_2(IBUS, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	IREG, remote bus number (0 to control voltage at bus IBUS) (0 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	VS, scheduled voltage magnitude (1.0 by default)
	REALAR(2)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	plant table is full
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid scheduled voltage - regulated bus not found - plant specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - RMPCT is not positive

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.93. PLANT_DATA_3

This API routine is obsolete. It has been replaced by the API routine plant_data_4 .

Use this API to modify the data of an existing plant in the working case, or to add a new plant to the working case (see PSSE Program Operation Manual, Fixed Bus Shunt Data).

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_PLANT_DATA_3  IBUS  INODE INTGAR(1)  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = plant_data_3(ibus, inode, intgar, realar)
```

Fortran command syntax:

```
CALL PLANT_API_3(IBUS, INODE, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INODE	bus section node number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	IREG, remote bus number (0 to control voltage at bus IBUS) (0 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	VS, scheduled voltage magnitude (1.0 by default)
	REALAR(2)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	plant table is full
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid scheduled voltage - regulated bus not found - plant specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - RMPCT is not positive

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.94. PLANT_DATA_4

Use this API to modify the data of an existing plant in the working case, or to add a new plant to the working case (see PSSE Program Operation Manual, Fixed Bus Shunt Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_PLANT_DATA_4  IBUS  INODE INTGAR(1)  INTGAR(2) REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = plant_data_4(ibus, inode, intgar, realar)
```

Fortran command syntax:

```
CALL PLANT_API_4(IBUS, INODE, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INODE	bus section node number (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	IREG, remote bus number (0 to control voltage at bus IBUS) (0 by default)
	INTGAR(2)	NREG, regulated bus node number (0 to control voltage at the "parent" bus section of bus IBUS) (0 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	VS, scheduled voltage magnitude (1.0 by default)
	REALAR(2)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	plant table is full
	IERR = 4	invalid bus number
	IERR = 5	INODE is an invalid node number
	IERR = 6	bus IBUS does not belong to a substation
	IERR = 7	node INODE is not found in the substation of bus IBUS
	IERR = 8	node INODE is not connected to any bus section of bus IBUS
	IERR = 9	INODE must be 0 for a bus that is not yet a generator bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 10	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid scheduled voltage - regulated bus not found - type code of regulated bus is not 1 or 2 - RMPCT is not positive - per unit scheduled voltage is outside of the band 0.5 to 1.5 p.u. - invalid node number - non-zero regulated node specified but regulated bus does not belong to a substation - regulated node not found - regulated node is not connected to any bus section of the regulated bus - a remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.95. PURG

Use this API routine to delete specified outaged equipment items from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURG  SID  ALL  OPT  STATUS
```

Python command syntax:

```
ierr = purg(sid, all, opt, status)
```

Fortran command syntax:

```
CALL PURGNW(SID, ALL, OPT, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer OPT	equipment type to remove (input; 1 by default).	
	OPT = 1	ac lines
	OPT = 2	switched shunts
	OPT = 3	machines
	OPT = 4	plants
	OPT = 5	two-terminal dc lines
	OPT = 6	multi-terminal dc lines
	OPT = 7	loads
	OPT = 8	FACTS devices
	OPT = 9	three-winding transformers
	OPT = 10	VSC dc lines
	OPT = 11	fixed bus shunts
	OPT = 12	GNE devices
	OPT = 13	induction machines
Integer STATUS	option for the treatment of out-of-service ties to other systems (input; 0 by default).	
	STATUS = 0	keep out-of-service ties
	STATUS = 1	remove out-of-service ties
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ALL value
IERR = 3	invalid OPT value
IERR = 4	invalid STATUS value
IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.96. PURG2DC

Use this API to delete the specified two-terminal dc line from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURG2DC  DC
```

Python command syntax:

```
ierr = purg2dc(dc)
```

Fortran command syntax:

```
CALL PURG2DC(NUM, IERR)

CALL PURG2DC_I(NUM, IERR)

CALL PURG2DC_C(NAME, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line to be removed; if an un-quoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line to be removed; no default allowed	
Character*12 NAME	name of the two-terminal dc line to be removed; no default allowed	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	dc line not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.97. PURG3WND

Use this API to delete the specified three-winding transformer from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURG3WND  IBUS  JBUS  KBUS  'CKT'
```

Python command syntax:

```
ierr = purg3wnd(ibus, jbus, kbus, ckt)
```

Fortran command syntax:

```
CALL PURG3WND(IBUS, JBUS, KBUS, CKT, IERR)
```

Where:

Integer IBUS	one of the buses connected by the three-winding transformer (input; no default allowed).	
Integer JBUS	the bus connected by the three-winding transformer (input; no default allowed).	
Integer KBUS	other bus connected by the three-winding transformer (input; no default allowed).	
Character*2 CKT	circuit identifier of the three-winding transformer to be deleted (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	three-winding transformer not found
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.98. PURGAREA

Use this API to delete areas with no equipment assigned to them from the working case.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_PURGAREA  SID  ALL
```

Python command syntax:

```
ierr = purgarea(sid, all)
```

Fortran command syntax:

```
CALL PURGAREA(SID, ALL, IERR)
```

Where:

Integer SID	a valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all areas or specified subsystem flag (input; 1 by default).	
	ALL = 0	delete only areas in area subsystem SID with no equipment assigned to them.
	ALL = 1	delete all areas with no equipment assigned to them.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.99. PURGATRN

Use this API to delete the specified inter-area transfer from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGATRN  FRMAREA  TOAREA  'ID'
```

Python command syntax:

```
ierr = purgatrn(frmarea, toarea, id)
```

Fortran command syntax:

```
CALL PURGATRN(FRMAREA, TOAREA, ID, IERR)
```

Where:

Integer FRMAREA	one of the areas involved in the transfer (input; no default allowed).	
Integer TOAREA	other area involved in the transfer (input; no default allowed).	
Character*1 ID	inter-area transfer identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	area not found
	IERR = 2	inter-area transfer not found
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.100. PURGBRN

Use this API to delete the specified non-transformer branch or two-winding transformer from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGBRN  IBUS  JBUS  'CKT'
```

Python command syntax:

```
ierr = purgbrn(ibus, jbus, ckt)
```

Fortran command syntax:

```
CALL PURGBRN(IBUS, JBUS, CKT, IERR)
```

Where:

Integer IBUS	one of the buses connected by the branch (input; no default allowed).	
Integer JBUS	other bus connected by the branch (input; no default allowed).	
Character*2 CKT	circuit identifier of the branch to be deleted (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	branch not found
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.101. PURGCAPCURVE

Use this API to delete the specified machine's capability curve from the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_PURGCAPCURVE  IBUS  'ID'
```

Python command syntax:

```
ierr = purgcapcurve(ibus, id)
```

Fortran command syntax:

```
CALL PURGCAPCURVE(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus to which the specified machine is connected (input; no default allowed).	
Character*2 ID	identifier of the machine whose capability curve is to be removed (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no machines at the bus
	IERR = 3	machine not found
	IERR = 4	capability curve not found for this machine
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.102. PURGE_MULTI_TERM_DC_BUS

Use this API to delete the specified dc bus from the specified multi-terminal dc line in the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_MULTI_TERM_DC_BUS  DC  IBUS
```

Python command syntax:

```
ierr = purge_multi_term_dc_bus(dc, ibus)
```

Fortran command syntax:

```
CALL PURGMDC_DC_BUS_API(NUM, IBUS, IERR, NERR)

CALL PURGMDC_DC_BUS_API_I(NUM, IBUS, IERR)

CALL PURGMDC_DC_BUS_API_C(NAME, IBUS, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line from which a dc bus is to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line from which a dc bus is to be removed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line from which a dc bus is to be removed (input; no default allowed).	
Integer IBUS	number of the dc bus to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	dc line not found
	IERR = 2	invalid dc bus number
	IERR = 3	dc bus not present in this multi-terminal dc line
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - dc bus no longer connected to converter as first dc bus. - dc bus no longer connected to converter as second dc bus. - dc link deleted
Integer NERR	number of error entries in PBUF02	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.103. PURGE_MULTI_TERM_DC_CONVR

Use this API to delete the specified converter from the specified multi-terminal dc line in the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_MULTI_TERM_DC_CONVR  DC  IBUS
```

Python command syntax:

```
ierr = purge_multi_term_dc_convr(dc, ibus)
```

Fortran command syntax:

```
CALL PURGMDC_CONVERTER_API(NUM, IBUS, IERR)

CALL PURGMDC_CONVERTER_API_I(NUM, IBUS, IERR)

CALL PURGMDC_CONVERTER_API_C(NAME, IBUS, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line from which a converter is to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line from which a converter is to be removed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line from which a converter is to be removed (input; no default allowed).	
Integer IBUS	number of the ac bus from which the converter is to be removed (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	dc line not found
	IERR = 2	converter bus not found
	IERR = 3	bus is not a converter bus of this multi-terminal dc line
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - this converter bus is no longer its pole's voltage controlling converter. - the voltage setpoint of the new voltage controlling converter is not positive. - the voltage setpoint of the new voltage controlling converter is less than the mode switch voltage

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.104. PURGE_MULTI_TERM_DC_LINK

Use this API to delete the specified dc link from the specified multi-terminal dc line in the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_MULTI_TERM_DC_LINK  DC  IBUS  JBUS  'CKT'
```

Python command syntax:

```
ierr = purge_multi_term_dc_link(dc, ibus, jbus, ckt)
```

Fortran command syntax:

```
CALL PURGMDC_DC_LINK_API(NUM, IBUS, JBUS, CKT, IERR)

CALL PURGMDC_DC_LINK_API_I(NUM, IBUS, JBUS, CKT, IERR)

CALL PURGMDC_DC_LINK_API_C(NAME, IBUS, JBUS, CKT, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line from which a dc link is to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line from which a dc link is to be removed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line from which a dc link is to be removed (input; no default allowed).	
Integer IBUS	one of the dc buses connected by the dc link to be removed (input).	
Integer JBUS	other dc bus connected by the dc link (input).	
Character*1 CKT	circuit identifier of the dc link to be deleted (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	dc line not found
	IERR = 2	invalid dc bus number
	IERR = 3	dc bus not present in this multi-terminal dc line
	IERR = 4	circuit identifier is more than one character
	IERR = 5	dc link not found in this multi-terminal dc line
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.105. PURGFACTS

Use this API to delete the specified FACTS device from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGFACTS   FD
```

Python command syntax:

```
ierr = purgfacts(fd)
```

Fortran command syntax:

```
CALL PURGFACTS(NUM, IERR)

CALL PURGFACTS_I(NUM, IERR)

CALL PURGFACTS_C(NAME, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device to be removed (input; no default allowed).	
Character*12 NAME	name of the FACTS device to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	FACTS device not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.106. PURGGNE

Use this API routine to delete the specified GNE device from the working case.

This API routine was first introduced in release 32.0.2.

Batch command syntax:

```
BAT_PURGGNE  'NAME'
```

Python command syntax:

```
ierr = purggne(name)
```

Fortran command syntax:

```
CALL PURGGNE(NAME, IERR)
```

Where:

Character*12 NAME	name of the GNE device to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	GNE device not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.107. PURGINDMAC

Use this API routine to delete the specified induction machine from the working case.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_PURGINDMAC  IBUS  'ID'
```

Python command syntax:

```
ierr = purgindmac(ibus, id)
```

Fortran command syntax:

```
CALL PURGINDMAC(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus to which induction machine ID is connected (input; no default allowed).	
Character*2 ID	identifier of the induction machine to be removed (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no induction machines at bus
	IERR = 3	induction machine not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.108. PURGINDMACS

Use this API routine to delete all induction machines from the specified bus in the working case.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_PURGINDMACS  IBUS
```

Python command syntax:

```
ierr = purgindmacs(ibus)
```

Fortran command syntax:

```
CALL PURGINDMACS(IBUS, IERR)
```

Where:

Integer IBUS	bus from which all induction machines are to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no induction machines at bus
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.109. PURGLOAD

Use this API to delete the specified load from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGLOAD  IBUS  'ID'
```

Python command syntax:

```
ierr = purgload(ibus, id)
```

Fortran command syntax:

```
CALL PURGLOAD(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus to which the load ID is connected (input; no default allowed).	
Character*2 ID	identifier of the load to be removed (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no loads at bus
	IERR = 3	load not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.110. PURGLOADS

Use this API to delete all loads from the specified bus in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGLOADS  IBUS
```

Python command syntax:

```
ierr = purgloads(ibus)
```

Fortran command syntax:

```
CALL PURGLOADS(IBUS, IERR)
```

Where:

Integer IBUS	bus from which all loads are to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no loads at bus
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.111. PURGMAC

Use this API to delete the specified machine from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGMAC  IBUS  'ID'
```

Python command syntax:

```
ierr = purgmac(ibus, id)
```

Fortran command syntax:

```
CALL PURGMAC(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus to which the machine ID is connected (input; no default allowed).	
Character*2 ID	identifier of the machine to be removed (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no machines at bus
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.112. PURGMDC

Use this API to delete the specified multi-terminal dc line from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGMDC  DC
```

Python command syntax:

```
ierr = purgmdc(dc)
```

Fortran command syntax:

```
CALL PURGMDC(NUM, IERR)

CALL PURGMDC_I(NUM, IERR)

CALL PURGMDC_C(NAME, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line to be removed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	dc line not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.113. PURGMSL

Use this API to delete the specified multi-section line grouping from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGMSL  IBUS  JBUS  'CKT'
```

Python command syntax:

```
ierr = purgmsl(ibus, jbus, ckt)
```

Fortran command syntax:

```
CALL PURGMSL(IBUS, JBUS, CKT, IERR)
```

Where:

Integer IBUS	one of the endpoint buses of the multi-section line (input; no default allowed).	
Integer JBUS	other endpoint bus of the multi-section line (input; no default allowed).	
Character*2 CKT	circuit identifier of the multi-section line to be deleted; the first character must be an ampersand (input; '&1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	multi-section line not found
	IERR = 3	invalid multi-section line identifier (first character not &)
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.114. PURGMUT

Use this API to delete the specified zero sequence mutual coupling from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGMUT  IBUS1  JBUS1  'CKT1'  IBUS2  JBUS2  'CKT2'
```

Python command syntax:

```
ierr = purgmut(ibus1, jbus1, ckt1, ibus2, jbus2, ckt2)
```

Fortran command syntax:

```
CALL PURGMUT(IBUS1, JBUS1, CKT1, IBUS2, JBUS2, CKT2, IERR)
```

Where:

Integer IBUS1	one of the buses connected by the first branch of the mutual coupling (input; no default allowed).	
Integer JBUS1	other bus connected by the first branch of the mutual coupling (input; no default allowed).	
Character*2 CKT1	circuit identifier of the first branch of the mutual coupling (input; '1' by default).	
Integer IBUS2	one of the buses connected by the second branch of the mutual coupling (input; no default allowed).	
Integer JBUS2	other bus connected by the second branch of the mutual coupling (input; no default allowed).	
Character*2 CKT2	circuit identifier of the second branch of the mutual coupling (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	branch not found
	IERR = 3	mutual coupling not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.115. PURGOWNER

Use this API to delete owners with no equipment assigned to them from the working case.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_PURGOWNER  SID  ALL
```

Python command syntax:

```
ierr = purgowner(sid, all)
```

Fortran command syntax:

```
CALL PURGOWNER(SID, ALL, IERR)
```

Where:

Integer SID	a valid owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Owner subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all owners or specified subsystem flag (input; 1 by default).	
	ALL = 0	delete only owners in owner subsystem SID with no equipment assigned to them
	ALL = 1	delete all owners with no equipment assigned to them
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.116. PURGPLNT

Use this API to delete the plant and machine data at the specified bus from the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGPLNT  IBUS
```

Python command syntax:

```
ierr = purgplnt(ibus)
```

Fortran command syntax:

```
CALL PURGPLNT(IBUS, IERR)
```

Where:

Integer IBUS	bus from which the plant is to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no machines at bus
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.117. PURGSHUNT

Use this API to delete the specified fixed bus shunt from the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGSHUNT  IBUS  'ID'
```

Python command syntax:

```
ierr = purgshunt(ibus, id)
```

Fortran command syntax:

```
CALL PURGSHUNT(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus to which the shunt is connected (input; no default allowed).	
Character*2 ID	identifier of the shunt to be removed (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no fixed shunts at bus
	IERR = 3	shunt not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.118. PURGSHUNTS

Use this API to delete all fixed shunts from the specified bus in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGSHUNTS  IBUS
```

Python command syntax:

```
ierr = purgshunts(ibus)
```

Fortran command syntax:

```
CALL PURGSHUNTS(IBUS, IERR)
```

Where:

Integer IBUS	bus from which all fixed shunts are to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no fixed shunts at bus
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.119. PURGSWS

Use this API to delete all switched shunts from the specified bus in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PURGSWS  IBUS
```

Python command syntax:

```
ierr = purgsws(ibus)
```

Fortran command syntax:

```
CALL PURGSWS(IBUS, IERR)
```

Where:

Integer IBUS	bus from which all switched shunts are to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no switched shunts at bus
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.120. PURGSWSHUNT

Use this API to delete the specified switched bus shunt from the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_PURGSWSHUNT  IBUS  'ID'
```

Python command syntax:

```
ierr = purgswshunt(ibus, id)
```

Fortran command syntax:

```
CALL PURGSWSHUNT(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus to which the switched shunt is connected (input; no default allowed).	
Character*2 ID	identifier of the switched shunt to be removed (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	no switched shunts at bus
	IERR = 3	switched shunt not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.121. PURGVSC

Use this API to delete the specified VSC dc line from the working case.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_PURGVSC  'NAME'
```

Python command syntax:

```
ierr = purgvsc(name)
```

Fortran command syntax:

```
CALL PURGVSC(NAME, IERR)
```

Where:

Character*12 NAME	name of the VSC dc line to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	dc line not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.122. PURGZONE

Use this API to delete zones with no equipment assigned to them from the working case.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_PURGZONE  SID  ALL
```

Python command syntax:

```
ierr = purgzone(sid, all)
```

Fortran command syntax:

```
CALL PURGZONE(SID, ALL, IERR)
```

Where:

Integer SID	a valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Zone subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all zones or specified subsystem flag (input; 1 by default).	
	ALL = 0	delete only zones in zone subsystem SID with no equipment assigned to them
	ALL = 1	delete all zones with no equipment assigned to them
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.123. PURG_INTERFACE

Use this API to delete an existing interface flow model from the working case.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_PURG_INTERFACE  'NAME'
```

Python command syntax:

```
ierr = purg_interface(name)
```

Fortran command syntax:

```
CALL PURG_INTERFACE(NAME, IERR)
```

Where:

character*40 NAME	interface name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	interface not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.124. PURG_INTERFACE_ELEMENT

Use this API to delete an existing interface element from the speficied interface.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_PURG_INTERFACE_ELEMENT 'NAME' ELETYP INTBUS(1)...INTBUS(3) DEVNAM
```

Python command syntax:

```
ierr = purg_interface_element(name,eletyp,intbus,devnam)
```

Fortran command syntax:

```
CALL PURG_INTERFACE_ELEMENT( NAME,ELETYP,INTBUS,DEVNAM,IERR)
```

Where:

character*40 NAME	interface name (input).	
Integer ELETYP	interface element type (input).	
Integer INTBUS (3)	array of 3 elements specifying (input).	
	INTBUS(1)	from bus number or from area/zone number (alias is FBUS) (0 by default)
	INTBUS(2)	to bus number or to area/zone number (alias is TBUS) (0 by default)
	INTBUS(3)	the last bus of a three winding transformer (alias is LBUS) (0 by default)
Character*40 DEVNAM	interface element device id or device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	Invalid interface name
	IERR = 2	Interface not found
	IERR = 3	Interface element type not found
	IERR = 4	Interface element bus not found
	IERR = 5	Interface element area not found
	IERR = 6	Interface element zone not found
	IERR = 7	Interface element not found in the case
	IERR = 8	Interface element not found in the interface
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.125. PURG_LOADTYPE

Use this API to delete an existing load type from the working case.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_PURG_LOADTYPE  'NAME'
```

Python command syntax:

```
ierr = purg_loadtype(name)
```

Fortran command syntax:

```
CALL PURG_LOADTYPE(NAME, IERR)
```

Where:

character*12 NAME	load type name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	voltage droop control not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.126. PURG_SWD_RATING

Use this API to purge an entry of Switching device rating table

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_PURG_SWD_RATING RNAME IERR
```

Python command syntax:

```
ierr = purg_swd_rating(rname)
```

Fortran command syntax:

```
CALL PURG_SWD_RATING (RNAME, IERR)
```

Where:

character*40 RNAME	Name of the switching device rating table that needs to be deleted. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No Error
	IERR = 1	Name NOT available in Switching Device rating table
	IERR = 2	No switching device rating table exists
	IERR = 3	Pre-requisities not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.127. PURG_VOLTAGE_DROOP

Use this API to delete an existing voltage droop control from the working case.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_PURG_VOLTAGE_DROOP  'NAME'
```

Python command syntax:

```
ierr = purg_voltage_droop(name)
```

Fortran command syntax:

```
CALL PURG_VOLTAGE_DROOP(NAME, IERR)
```

Where:

character*40 NAME	voltage droop control name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	voltage droop control not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.128. RATINGSETTEXTDATA

This API routine is obsolete. It has been replaced by the API routine brn_ratings_name_data .

Use this API to change the column and descriptions for a particular rating set used in reporting.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_RATINGSETTEXTDATA IRATE NAME DESCR
```

Python command syntax:

```
ierr = ratingsettextdata(irate, name, descr)
```

Fortran command syntax:

```
CALL RATINGSETTEXTDATA(IRATE, NAME, DESCR, IERR)
```

Where:

Integer IRATE	rating set number (1 through 12) (input; no default allowed).	
Character*6 NAME	rating set column heading (input; blank by default).	
Character*32 DESCR	rating set description (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors
	IERR = 1	invalid rating set number
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.129. SHUNT_CHNG

This API routine is obsolete. It has been replaced by the API routine shunt_chng_2 .

Use this API routine to modify the data of an existing fixed bus shunt in the working case (see PSSE Program Operation Manual, Fixed Bus Shunt Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SHUNT_CHNG IBUS 'ID' INTGAR(1) REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = shunt_chng(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL SHUNT_CHNG(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	shunt identifier (input; '1' by default).	
Integer INTGAR (1)	See the SHUNT_DATA API routine for details (input).	
Real REALAR (2)	See the SHUNT_DATA API routine for details (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	shunt identifier is more than two characters
	IERR = 5	fixed bus shunt not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid shunt status

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.130. SHUNT_CHNG_2

Use this API routine to modify the data of an existing fixed bus shunt in the working case (see PSSE Program Operation Manual, Fixed Bus Shunt Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SHUNT_CHNG_2  IBUS  'ID'  INTGAR(1)  REALAR(1)  REALAR(2) SHNAME
```

Python command syntax:

```
ierr = shunt_chng_2(ibus, id, intgar, realar, shname)
```

Fortran command syntax:

```
CALL SHUNT_CHNG_2(IBUS, ID, INTGAR, REALAR, SHNAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	shunt identifier (input; '1' by default).	
Integer INTGAR (1)	See the SHUNT_DATA API routine for details (input).	
Real REALAR (2)	See the SHUNT_DATA API routine for details (input).	
character*40 SHNAME	bus shunt name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	shunt identifier is more than two characters
	IERR = 5	fixed bus shunt not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid shunt status

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.131. SHUNT_DATA

Use this API to modify the data of an existing fixed bus shunt in the working case or to add a new fixed bus shunt to the working case (see PSSE Program Operation Manual, Fixed Bus Shunt Data).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SHUNT_DATA IBUS 'ID' INTGAR(1) REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = shunt_data(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL SHUNT_API(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	shunt identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	STATUS, shunt status (1 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	GL, fixed shunt admittance (conductance) (0.0 by default)
	REALAR(2)	BL, fixed shunt admittance (susceptance) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	shunt identifier is more than two characters
	IERR = 3	blank shunt identifier is invalid
	IERR = 4	fixed bus shunt table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	node-breaker terminal connection table is full
	IERR = -1	data error, warning and information messages; one or more of: - invalid shunt status

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.132. SHUNT_DATA_2

Use this API to modify the data of an existing fixed bus shunt in the working case or to add a new fixed bus shunt to the working case (see PSSE Program Operation Manual, Fixed Bus Shunt Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SHUNT_DATA_2  IBUS  'ID'  INTGAR(1)  REALAR(1)  REALAR(2) 'SHNAME'
```

Python command syntax:

```
ierr = shunt_data_2(ibus, id, intgar, realar, shname)
```

Fortran command syntax:

```
CALL SHUNT_API_2(IBUS, ID, INTGAR, REALAR, SHNAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	shunt identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	STATUS, shunt status (1 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	GL, fixed shunt admittance (conductance) (0.0 by default)
	REALAR(2)	BL, fixed shunt admittance (susceptance) (0.0 by default)
character*40 SHNAME	bus shunt name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	shunt identifier is more than two characters
	IERR = 3	blank shunt identifier is invalid
	IERR = 4	fixed bus shunt table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	node-breaker terminal connection table is full
	IERR = -1	data error, warning and information messages; one or more of: - invalid shunt status

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.133. SOLUTION_PARAMETERS

This API routine is obsolete. It has been replaced by the API routine solution_parameters_2 .

Use this API to modify the power flow solution parameters in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SOLUTION_PARAMETERS INTGAR(1)..INTGAR(3) REALAR(1)..REALAR(16)
```

Python command syntax:

```
ierr = solution_parameters(intgar, realar)
```

Fortran command syntax:

```
CALL SOLUTION_PARAMETER_API(INTGAR, REALAR, IERR, NERR)
```

Where:

Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	ITMX, Gauss-Seidel maximum number of iterations
	INTGAR(2)	ITMXN, Newton-Raphson maximum number of iterations
	INTGAR(3)	ITMXTY, TYSL maximum number of iterations
Real REALAR (16)	array of 16 elements specifying (input).	
	REALAR(1)	ACCP, Gauss-Seidel real component acceleration factor
	REALAR(2)	ACCQ, Gauss-Seidel imaginary component acceleration factor
	REALAR(3)	ACCM, special MSLV acceleration factor
	REALAR(4)	TOL, Gauss-Seidel convergence tolerance
	REALAR(5)	ACCN, Newton-Raphson acceleration factor
	REALAR(6)	TOLN, Newton-Raphson mismatch convergence tolerance
	REALAR(7)	ACCTY, TYSL acceleration factor
	REALAR(8)	TOLTY, TYSL convergence tolerance
	REALAR(9)	BLOWUP, blow-up threshold
	REALAR(10)	PQBRAK, constant power load characteristic breakpoint
	REALAR(11)	THRSHZ, zero impedance line threshold
	REALAR(12)	ADJTHR, automatic adjustment threshold tolerance
	REALAR(13)	ACCTAP, tap movement deceleration factor
	REALAR(14)	TAPLIM, maximum tap change per adjustment

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(15)	DVLIM, largest abs(delta v)/abs(v) for Newton solutions
	REALAR(16)	NDVFCT, non-divergent required improvement factor
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = -1	warnings: - None
Integer NERR	number of entries in PBUF02 containing error messages (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.134. SOLUTION_PARAMETERS_2

This API routine is obsolete. It has been replaced by the API routine solution_parameters_3 .

Use this API to modify the power flow solution parameters in the working case.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_SOLUTION_PARAMETERS_2 INTGAR(1)..INTGAR(3) REALAR(1)..REALAR(18)
```

Python command syntax:

```
ierr = solution_parameters_2(intgar, realar)
```

Fortran command syntax:

```
CALL SOLUTION_PARAMETER_API_2(INTGAR, REALAR, IERR, NERR)
```

Where:

Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	ITMX, Gauss-Seidel maximum number of iterations
	INTGAR(2)	ITMXN, Newton-Raphson maximum number of iterations
	INTGAR(3)	ITMXTY, TYSL maximum number of iterations
Real REALAR (18)	array of 18 elements specifying (input).	
	REALAR(1)	ACCP, Gauss-Seidel real component acceleration factor
	REALAR(2)	ACCQ, Gauss-Seidel imaginary component acceleration factor
	REALAR(3)	ACCM, special MSLV acceleration factor
	REALAR(4)	TOL, Gauss-Seidel convergence tolerance
	REALAR(5)	ACCN, Newton-Raphson acceleration factor
	REALAR(6)	TOLN, Newton-Raphson mismatch convergence tolerance
	REALAR(7)	ACCTY, TYSL acceleration factor
	REALAR(8)	TOLTY, TYSL convergence tolerance
	REALAR(9)	BLOWUP, blow-up threshold
	REALAR(10)	PQBRAK, constant power load characteristic breakpoint
	REALAR(11)	THRSHZ, zero impedance line threshold
	REALAR(12)	ADJTHR, automatic adjustment threshold tolerance
	REALAR(13)	ACCTAP, tap movement deceleration factor
	REALAR(14)	TAPLIM, maximum tap change per adjustment

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(15)	DVLIM, largest abs(delta v)/abs(v) for Newton solutions
	REALAR(16)	NDVFCT, non-divergent required improvement factor
	REALAR(17)	VCTOLQ, Newton-Raphson voltage controlled bus reactive power mismatch convergence tolerance
	REALAR(18)	VCTOLV, Newton-Raphson voltage controlled bus voltage error convergence tolerance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = -1	warnings: - None
Integer NERR	number of entries in PBUF02 containing error messages (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.135. SOLUTION_PARAMETERS_3

This API routine is obsolete. It has been replaced by the API routine solution_parameters_4 .

Use this API to modify the power flow solution parameters in the working case (see PSSE Program Operation Manual, Sections 6.3.13, 6.3.18, 6.5.1 and 11.7).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SOLUTION_PARAMETERS_3 INTGAR(1)..INTGAR(3) REALAR(1)..REALAR(19)
```

Python command syntax:

```
ierr = solution_parameters_3(intgar, realar)
```

Fortran command syntax:

```
CALL SOLUTION_PARAMETER_API_3(INTGAR, REALAR, IERR)
```

Where:

Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	ITMX, Gauss-Seidel maximum number of iterations (100 by default)
	INTGAR(2)	ITMXN, Newton-Raphson maximum number of iterations (20 by default)
	INTGAR(3)	ITMXTY, TYSL maximum number of iterations (20 by default)
Real REALAR (19)	array of 19 elements specifying (input).	
	REALAR(1)	ACCP, Gauss-Seidel real component acceleration factor (1.6 by default)
	REALAR(2)	ACCQ, Gauss-Seidel imaginary component acceleration factor (1.6 by default)
	REALAR(3)	ACCM, special MSLV acceleration factor (1.0 by default)
	REALAR(4)	TOL, Gauss-Seidel convergence tolerance (0.0001 by default)
	REALAR(5)	ACCN, Newton-Raphson acceleration factor (1.0 by default)
	REALAR(6)	TOLN, Newton-Raphson mismatch convergence tolerance (default Newton power flow solution tolerance option setting)
	REALAR(7)	ACCTY, TYSL acceleration factor (1.0 by default)
	REALAR(8)	TOLTY, TYSL convergence tolerance (0.00001 by default)
	REALAR(9)	BLOWUP, blow-up threshold (5.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(10)	PQBRAK, constant power load characteristic breakpoint (0.7 by default)
	REALAR(11)	THRSHZ, zero impedance line threshold (0.0001 by default)
	REALAR(12)	ADJTHR, automatic adjustment threshold tolerance (0.005 by default)
	REALAR(13)	ACCTAP, tap movement deceleration factor (1.0 by default)
	REALAR(14)	TAPLIM, maximum tap change per adjustment (0.05 by default)
	REALAR(15)	DVLIM, largest abs(delta v)/abs(v) for Newton solutions (0.99 by default)
	REALAR(16)	NDVFCT, non-divergent required improvement factor (0.99 by default)
	REALAR(17)	VCTOLQ, Newton-Raphson voltage controlled bus reactive power mismatch convergence tolerance (default Newton power flow solution tolerance option setting)
	REALAR(18)	VCTOLV, Newton-Raphson voltage controlled bus voltage error convergence tolerance (0.00001 by default)
	REALAR(19)	SWVBND, percent of voltage controlling band mode switched shunts to be adjusted per power flow iteration (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid percent value for switched shunt adjustment

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.136. SOLUTION_PARAMETERS_4

This API routine is obsolete. It has been replaced by the API routine solution_parameters_5 .

Use this API to modify the power flow solution parameters in the working case (see PSSE Program Operation Manual, Sections 6.3.13, 6.3.18, 6.5.1 and 11.7).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SOLUTION_PARAMETERS_4 INTGAR(1)..INTGAR(5) REALAR(1)..REALAR(19)
```

Python command syntax:

```
ierr = solution_parameters_4(intgar, realar)
```

Fortran command syntax:

```
CALL SOLUTION_PARAMETER_API_4(INTGAR, REALAR, IERR)
```

Where:

Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	ITMX, Gauss-Seidel maximum number of iterations (100 by default)
	INTGAR(2)	ITMXN, Newton-Raphson maximum number of iterations (20 by default)
	INTGAR(3)	ITMXTY, TYSL maximum number of iterations (20 by default)
	INTGAR(4)	MXTPSS, maximum number of times taps and/or switched shunts are adjusted during power flow solutions (100 by default)
	INTGAR(5)	MXSWIM, maximum number of times an induction machine can change state (e.g., from "running" to "stalled") during power flow solutions (10 by default)
Real REALAR (19)	array of 19 elements specifying (input).	
	REALAR(1)	ACCP, Gauss-Seidel real component acceleration factor (1.6 by default)
	REALAR(2)	ACCQ, Gauss-Seidel imaginary component acceleration factor (1.6 by default)
	REALAR(3)	ACCM, special MSLV acceleration factor (1.0 by default)
	REALAR(4)	TOL, Gauss-Seidel convergence tolerance (0.0001 by default)
	REALAR(5)	ACCN, Newton-Raphson acceleration factor (1.0 by default)
	REALAR(6)	TOLN, Newton-Raphson mismatch convergence tolerance (default Newton power flow solution tolerance option setting)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(7)	ACCTY, TYSL acceleration factor (1.0 by default)
	REALAR(8)	TOLTY, TYSL convergence tolerance (0.00001 by default)
	REALAR(9)	BLOWUP, blow-up threshold (5.0 by default)
	REALAR(10)	PQBRAK, constant power load characteristic breakpoint (0.7 by default)
	REALAR(11)	THRSHZ, zero impedance line threshold (0.0001 by default)
	REALAR(12)	ADJTHR, automatic adjustment threshold tolerance (0.005 by default)
	REALAR(13)	ACCTAP, tap movement deceleration factor (1.0 by default)
	REALAR(14)	TAPLIM, maximum tap change per adjustment (0.05 by default)
	REALAR(15)	DVLIM, largest abs(delta v)/abs(v) for Newton solutions (0.99 by default)
	REALAR(16)	NDVFCT, non-divergent required improvement factor (0.99 by default)
	REALAR(17)	VCTOLQ, Newton-Raphson voltage controlled bus reactive power mismatch convergence tolerance (default Newton power flow solution tolerance option setting)
	REALAR(18)	VCTOLV, Newton-Raphson voltage controlled bus voltage error convergence tolerance (0.00001 by default)
	REALAR(19)	SWVBND, percent of voltage controlling band mode switched shunts to be adjusted per power flow iteration (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid percent value for switched shunt adjustment. - Invalid induction motor treatment code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.137. SOLUTION_PARAMETERS_5

Use this API to modify the power flow solution parameters in the working case (see PSSE Program Operation Manual, Sections 6.3.13, 6.3.18, 6.5.1 and 11.7).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SOLUTION_PARAMETERS_5 INTGAR(1)..INTGAR(9) REALAR(1)..REALAR(21)
```

Python command syntax:

```
ierr = solution_parameters_5(intgar, realar)
```

Fortran command syntax:

```
CALL SOLUTION_PARAMETER_API_5(INTGAR, REALAR, IERR)
```

Where:

Integer INTGAR (9)	array of 9 elements specifying (input).	
	INTGAR(1)	ITMX, Gauss-Seidel maximum number of iterations (100 by default)
	INTGAR(2)	ITMXN, Newton-Raphson maximum number of iterations (20 by default)
	INTGAR(3)	ITMXTY, TYSL maximum number of iterations (20 by default)
	INTGAR(4)	MXTPSS, maximum number of times taps and/or switched shunts are adjusted during power flow solutions (100 by default)
	INTGAR(5)	MXSWIM, maximum number of times an induction machine can change state (e.g., from "running" to "stalled") during power flow solutions (10 by default)
	INTGAR(6)	MovIterMax, maximum number of MOV iterations in short circuit solutions (20 by default)
	INTGAR(7)	IsolLvl, Maximum number of levels to go outward when isolating an element by breaker actions (4 by default)
	INTGAR(8)	CaIterMax, Maximum number of repeat solutions for each contingency case (20 by default)
	INTGAR(9)	CntChk, Check for duplicate contingency labels (<=0 = no, >0 = yes) (0 by default)
Real REALAR (21)	array of 21 elements specifying (input).	
	REALAR(1)	ACCP, Gauss-Seidel real component acceleration factor (1.6 by default)
	REALAR(2)	ACCQ, Gauss-Seidel imaginary component acceleration factor (1.6 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(3)	ACCM, special MSLV acceleration factor (1.0 by default)
	REALAR(4)	TOL, Gauss-Seidel convergence tolerance (0.0001 by default)
	REALAR(5)	ACCN, Newton-Raphson acceleration factor (1.0 by default)
	REALAR(6)	TOLN, Newton-Raphson mismatch convergence tolerance (default Newton power flow solution tolerance option setting)
	REALAR(7)	ACCTY, TYSL acceleration factor (1.0 by default)
	REALAR(8)	TOLTY, TYSL convergence tolerance (0.00001 by default)
	REALAR(9)	BLOWUP, blow-up threshold (5.0 by default)
	REALAR(10)	PQBRAK, constant power load characteristic breakpoint (0.7 by default)
	REALAR(11)	THRSHZ, zero impedance line threshold (0.0001 by default)
	REALAR(12)	ADJTHR, automatic adjustment threshold tolerance (0.005 by default)
	REALAR(13)	ACCTAP, tap movement deceleration factor (1.0 by default)
	REALAR(14)	TAPLIM, maximum tap change per adjustment (0.05 by default)
	REALAR(15)	DVLIM, largest abs(delta v)/abs(v) for Newton solutions (0.99 by default)
	REALAR(16)	NDVFCT, non-divergent required improvement factor (0.99 by default)
	REALAR(17)	VCTOLQ, Newton-Raphson voltage controlled bus reactive power mismatch convergence tolerance (default Newton power flow solution tolerance option setting)
	REALAR(18)	VCTOLV, Newton-Raphson voltage controlled bus voltage error convergence tolerance (0.00001 by default)
	REALAR(19)	SWVBND, percent of voltage controlling band mode switched shunts to be adjusted per power flow iteration (100.0 by default)
	REALAR(20)	MovTolerance, MOV iteration tolerance in short circuit solutions (0.01 by default)
	REALAR(21)	MovAlpha, MOV acceleration factor in short circuit solutions (0.3 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = -1	data error, warning and information messages; one or more of: - Invalid percent value for switched shunt adjustment. - Invalid induction motor treatment code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.138. SWD_RATING_CHNG

Use this API to modify the description or rating 1 thru 12 in the switching device rating table specified by switching device rating name.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SWD_RATING_CHNG RNAME RDESC RATE(1) .. RATE(12)
```

Python command syntax:

```
ierr = swd_rating_chng( rname, rdesc, rate)
```

Fortran command syntax:

```
CALL SWD_RATING_CHNG ( RNAME, RDESC, RATE , IERR)
```

Where:

character*40 RNAME	Alphanumeric Identifier assigned to rating table. NAME must be enclosed in single or double quotes when using special characters. Must be unique. (input; no default allowed).
character RDESC	Alphanumeric description obout switching device. Description is used for information only. (input; blank by default).
real RATE (12)	array of elements specifying nth rating; entered in the unit specified for that nth SWDRAATNAM specified on the SWITCHING DEVICE RATING NAME record. Each RATEn default 0.0(input).
	RATE(1) Rating1 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate1) (0.0 by default)
	RATE(2) Rating2 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate2) (0.0 by default)
	RATE(3) Rating3 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate3) (0.0 by default)
	RATE(4) Rating4 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate4) (0.0 by default)
	RATE(5) Rating5 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate5) (0.0 by default)
	RATE(6) Rating6 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate6) (0.0 by default)
	RATE(7) Rating7 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate7) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RATE(8)	Rating8 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate8) (0.0 by default)
	RATE(9)	Rating9 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate9) (0.0 by default)
	RATE(10)	Rating10 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate10) (0.0 by default)
	RATE(11)	Rating11 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate11) (0.0 by default)
	RATE(12)	Rating12 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate12) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 1	Invalid switching device rating table
	IERR = 2	switching device rating table not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Switching device rating table name is too long; - switching device rating is negative

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.139. SWD_RATING_DATA

Use this API to add and modify the description or rating 1 thru 12 in the existing switching device rating table specified by switching device rating name.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SWD_RATING_DATA RNAME RDESC RATE(1) .. RATE(12)
```

Python command syntax:

```
ierr = swd_rating_data( rname, rdesc, rate)
```

Fortran command syntax:

```
CALL SWD_RATING_DATA ( RNAME, RDESC, RATE , IERR)
```

Where:

character*40 RNAME	Alphanumeric Identifier assigned to rating table. NAME must be enclosed in single or double quotes when using special characters. Must be unique. (input; no default allowed).
character RDESC	Alphanumeric description obout switching device. Description is used for information only. Blank by default (input; blank by default).
real RATE (12)	array of elements specifying nth rating; entered in the unit specified for that nth SWDRAATNAM specified on the SWITCHING DEVICE RATING NAME record. Each RATEn default 0.0(input).
RATE(1)	Rating1 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate1) (0.0 by default)
RATE(2)	Rating2 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate2) (0.0 by default)
RATE(3)	Rating3 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate3) (0.0 by default)
RATE(4)	Rating4 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate4) (0.0 by default)
RATE(5)	Rating5 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate5) (0.0 by default)
RATE(6)	Rating6 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate6) (0.0 by default)
RATE(7)	Rating7 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate7) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RATE(8)	Rating8 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate8) (0.0 by default)
	RATE(9)	Rating9 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate9) (0.0 by default)
	RATE(10)	Rating10 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate10) (0.0 by default)
	RATE(11)	Rating11 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate11) (0.0 by default)
	RATE(12)	Rating12 of the specified on the SWITCHING DEVICE RATING NAME record. (alias is rate12) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 1	Invalid switching device rating table
	IERR = 2	switching device rating table is full
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Switching device rating table name is too long; - switching device rating is negative

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.140. SWD_RATING_NAME_SET

Use this API to change the names, types, units and descriptions for a particular switching device rating name set used in reporting.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SWD_RATING_NAME_SET IRATE NAME ITYPE UNIT DESCR
```

Python command syntax:

```
ierr = swd_rating_name_set(irate, name, itype, unit, descr)
```

Fortran command syntax:

```
CALL SWD_RATING_NAME_SET(IRATE, NAME, ITYPE, UNIT, DESCR, IERR)
```

Where:

Integer IRATE	switching device rating name set number (1 through 12) (input; no default allowed).	
Character*32 NAME	switching device rating name name (input; Rate).	
Integer ITYPE	switching device rating name type (0 or 1) (input; 0 by default).	
Character*3 UNIT	rswitching device rating name unit (input; MVA).	
Character*32 DESCR	switching device rating name description (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid switching device rating name set number
	IERR = 2	Invalid switching device rating name type
	IERR = 3	Invalid switching device rating name unit
	IERR = 4	The unit/type of rating set can not be changed because it has been used for a station switching device
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - name of switching device rating name truncated to 6 characters. - description of switching device rating name truncated to 32 characters.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.141. SWITCHED_SHUNT_CHNG_3

This API routine is obsolete. It has been replaced by the API routine switched_shunt_chng_4 .

Use this API routine to modify the data of an existing switched shunt in the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_CHNG_3  IBUS  INTGAR(1)..INTGAR(12)
 REALAR(1)..REALAR(12)  'RMIDNT'
```

Python command syntax:

```
ierr = switched_shunt_chng_3(ibus, intgar, realar, rmidnt)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_CHNG_3(IBUS, INTGAR, REALAR, RMIDNT, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (12)	array of 12 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, remote bus number (0 by default)
	INTGAR(11)	STAT, status (1 by default)
	INTGAR(12)	ADJM, adjustment method (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)

Character*12 RMIDNT — VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).

Integer IERR — error code (output).

IERR = 0	no errors occurred
IERR > 0	error occurred
IERR = 1	bus not found
IERR = 3	switched shunt not found
IERR = 4	prerequisite requirements for API are not met
IERR = -1	warnings - one or more of: - invalid switched shunt control mode - control mode is 4 and a blank VSC dc line name is specified - VSC dc line not found - control mode is 6 and a blank FACTS device name is specified - FACTS device not found - non-blank device name specified with control mode 0, 1, 2, 3 or 5 - regulated bus not found - voltage controlling switched shunt specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - no regulated bus specified for plant, VSC converter, or switched shunt control - control mode is 3 and the regulated bus has no generation - control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

is 5 and the regulated bus has no switched shunt - regulated bus is not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - block has N or B zero and the other non-zero - invalid number of steps in block - block with non-zero N is beyond a previous N=0 block - block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.142. SWITCHED_SHUNT_CHNG_4

This API routine is obsolete. It has been replaced by the API routine switched_shunt_chng_5 .

Use this API routine to modify the data of an existing switched shunt in the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_CHNG_4  IBUS  INTGAR(1)..INTGAR(13)
 REALAR(1)..REALAR(12)  'RMIDNT'
```

Python command syntax:

```
ierr = switched_shunt_chng_4(ibus, intgar, realar, rmidnt)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_CHNG_4(IBUS, INTGAR, REALAR, RMIDNT, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (13)	array of 13 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, regulated bus number (0 by default)
	INTGAR(11)	NODE, regulated node number (0 by default)
	INTGAR(12)	STAT, status (1 by default)
	INTGAR(13)	ADJM, adjustment method (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)

Character*12 RMIDNT — VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).

Integer IERR — error code (output).

IERR = 0	no errors occurred
IERR > 0	error occurred
IERR = 1	bus not found
IERR = 3	switched shunt not found
IERR = 4	prerequisite requirements for API are not met
IERR = -1	warnings - one or more of: - Invalid switched shunt control mode - Control mode is 4 and a blank VSC dc line name is specified - VSC dc line not found - Control mode is 6 and a blank FACTS device name is specified - FACTS device not found - Non-blank device name specified with control mode 0, 1, 2, 3 or 5 - Regulated bus not found - Type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - No regulated bus specified for plant, VSC converter, or switched shunt control - Control mode is 3 and the regulated bus has no generation - Control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - Control mode is 5 and the regulated bus has no switched shunt - Regulated bus is

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - Block has N or B zero and the other non-zero - Invalid number of steps in block - Block with non-zero N is beyond a previous N=0 block - Block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus - Regulated bus is 0 but a non-zero regulated node is specified - A remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.143. SWITCHED_SHUNT_CHNG_5

This API routine is obsolete. It has been replaced by the API routine switched_shunt_chng_6 .

Use this API routine to modify the data of an existing switched shunt in the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_CHNG_5 IBUS 'ID' INTGAR(1)..INTGAR(21)
 REALAR(1)..REALAR(12) 'RMIDNT'
```

Python command syntax:

```
ierr = switched_shunt_chng_5(ibus, id, intgar, realar, rmidnt)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_CHNG_5(IBUS, ID, INTGAR, REALAR, RMIDNT, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
character*2 ID	Switched shunt identifier (input; '1' by default).	
Integer INTGAR (21)	array of 21 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, regulated bus number (0 by default)
	INTGAR(11)	NODE, regulated node number (0 by default)
	INTGAR(12)	STAT, status (1 by default)
	INTGAR(13)	ADJM, adjustment method (0 by default)
	INTGAR(14)	ST1, status of block 1 (1 by default)
	INTGAR(15)	ST2, status of block 2 (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(16)	ST3, status of block 3 (1 by default)
	INTGAR(17)	ST4, status of block 4 (1 by default)
	INTGAR(18)	ST5, status of block 5 (1 by default)
	INTGAR(19)	ST6, status of block 6 (1 by default)
	INTGAR(20)	ST7, status of block 7 (1 by default)
	INTGAR(21)	ST8, status of block 8 (1 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)
	REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
	REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
	REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
	REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
	REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
	REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
	REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
	REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
	REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
	REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
	REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)
Character*12 RMIDNT	VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 3	identifier is more than two characters
	IERR = 5	switched shunt not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid switched shunt control mode - Control mode is 4 and a blank VSC dc line name is specified - VSC dc line not found - Control mode is 6 and a blank FACTS device name is specified - FAC-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

TS device not found - Non-blank device name specified with control mode 0, 1, 2, 3 or 5 - Regulated bus not found - Type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - No regulated bus specified for plant, VSC converter, or switched shunt control - Control mode is 3 and the regulated bus has no generation - Control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - Control mode is 5 and the regulated bus has no switched shunt - Regulated bus is not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - Block has N or B zero and the other non-zero - Invalid number of steps in block - Block with non-zero N is beyond a previous N=0 block - Block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus - Regulated bus is 0 but a non-zero regulated node is specified - A remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.144. SWITCHED_SHUNT_CHNG_6

Use this API routine to modify the data of an existing switched shunt in the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_CHNG_6 IBUS  'ID' INTGAR(1)..INTGAR(21)
 REALAR(1)..REALAR(12) 'RMIDNT' 'SWSNAM'
```

Python command syntax:

```
ierr = switched_shunt_chng_6(ibus, id, intgar, realar, rmidnt, swsnam)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_CHNG_6(IBUS, ID, INTGAR, REALAR, RMIDNT, SWSNAM, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
character*2 ID	Switched shunt identifier (input; '1' by default).	
Integer INTGAR (21)	array of 21 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, regulated bus number (0 by default)
	INTGAR(11)	NODE, regulated node number (0 by default)
	INTGAR(12)	STAT, status (1 by default)
	INTGAR(13)	ADJM, adjustment method (0 by default)
	INTGAR(14)	ST1, status of block 1 (1 by default)
	INTGAR(15)	ST2, status of block 2 (1 by default)
	INTGAR(16)	ST3, status of block 3 (1 by default)
	INTGAR(17)	ST4, status of block 4 (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(18)	ST5, status of block 5 (1 by default)
	INTGAR(19)	ST6, status of block 6 (1 by default)
	INTGAR(20)	ST7, status of block 7 (1 by default)
	INTGAR(21)	ST8, status of block 8 (1 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)
	REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
	REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
	REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
	REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
	REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
	REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
	REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
	REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
	REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
	REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
	REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)
Character*12 RMIDNT	VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).	
character*40 SWSNAM	switched shunt name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 3	identifier is more than two characters
	IERR = 5	switched shunt not found
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid switched shunt control mode - Control mode is 4 and a blank VSC dc line name is specified - VSC dc line not found - Control mode is 6 and a blank FACTS device name is specified - FACTS device not found - Non-blank device name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

specified with control mode 0, 1, 2, 3 or 5 - Regulated bus not found - Type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - No regulated bus specified for plant, VSC converter, or switched shunt control - Control mode is 3 and the regulated bus has no generation - Control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - Control mode is 5 and the regulated bus has no switched shunt - Regulated bus is not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - Block has N or B zero and the other non-zero - Invalid number of steps in block - Block with non-zero N is beyond a previous N=0 block - Block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus - Regulated bus is 0 but a non-zero regulated node is specified - A remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.145. SWITCHED_SHUNT_DATA

This API routine is obsolete. It has been replaced by the API routine switched_shunt_data_3 .

Use this API to modify the data of an existing switched shunt in the working case or to add a new switched shunt to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_DATA  IBUS  INTGAR(1)..INTGAR(10)
 REALAR(1)..REALAR(12)  'RMIDNT'
```

Python command syntax:

```
ierr = switched_shunt_data(ibus, intgar, realar, rmidnt)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_API_2(IBUS, INTGAR, REALAR, RMIDNT, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (10)	array of 10 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, remote bus number (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)
	REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
	REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)

Character*12 RMIDNT — VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).

Integer IERR — error code (output).

IERR = 0	no errors occurred
IERR > 0	error occurred
IERR = 1	bus not found
IERR = 2	switched shunt table is full
IERR = 4	prerequisite requirements for API are not met
IERR = -1	warnings - one or more of: - invalid switched shunt control mode - control mode is 4 and a blank VSC dc line name is specified - VSC dc line not found - control mode is 6 and a blank FACTS device name is specified - FACTS device not found - non-blank device name specified with control mode 0, 1, 2, 3 or 5 - regulated bus not found - voltage controlling switched shunt specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - no regulated bus specified for plant, VSC converter, or switched shunt control - control mode is 3 and the regulated bus has no generation - control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - control mode is 5 and the regulated bus has no switched shunt - regulated bus is not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC convert-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

er, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - - RMPCT is not positive - block has N or B zero and the other non-zero - invalid number of steps in block - block with non-zero N is beyond a previous N=0 block - block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.146. SWITCHED_SHUNT_DATA_3

This API routine is obsolete. It has been replaced by the API routine switched_shunt_data_4 .

Use this API to modify the data of an existing switched shunt in the working case or to add a new switched shunt to the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_DATA_3  IBUS  INTGAR(1)..INTGAR(12)
 REALAR(1)..REALAR(12)  'RMIDNT'
```

Python command syntax:

```
ierr = switched_shunt_data_3(ibus, intgar, realar, rmidnt)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_API_3(IBUS, INTGAR, REALAR, RMIDNT, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (12)	array of 12 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, remote bus number (0 by default)
	INTGAR(11)	STAT, status (1 by default)
	INTGAR(12)	ADJM, adjustment method (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
	REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
	REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
	REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
	REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
	REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
	REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
	REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
	REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
	REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
	REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)
Character*12 RMIDNT	VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt table is full
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - invalid switched shunt control mode - control mode is 4 and a blank VSC dc line name is specified - VSC dc line not found - control mode is 6 and a blank FACTS device name is specified - FACTS device not found - non-blank device name specified with control mode 0, 1, 2, 3 or 5 - regulated bus not found - voltage controlling switched shunt specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - no regulated bus specified for plant, VSC converter, or switched shunt control - control mode is 3 and the regulated bus has no generation - control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

is 5 and the regulated bus has no switched shunt - regulated bus is not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - block has N or B zero and the other non-zero - invalid number of steps in block - block with non-zero N is beyond a previous N=0 block - block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.147. SWITCHED_SHUNT_DATA_4

This API routine is obsolete. It has been replaced by the API routine switched_shunt_data_5 .

Use this API to modify the data of an existing switched shunt in the working case or to add a new switched shunt to the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_DATA_4  IBUS  INTGAR(1)..INTGAR(13)
 REALAR(1)..REALAR(12)  'RMIDNT'
```

Python command syntax:

```
ierr = switched_shunt_data_4(ibus, intgar, realar, rmidnt)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_API_4(IBUS, INTGAR, REALAR, RMIDNT, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INTGAR (13)	array of 13 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, regulated bus number (0 by default)
	INTGAR(11)	NODE, regulated node number (0 by default)
	INTGAR(12)	STAT, status (1 by default)
	INTGAR(13)	ADJM, adjustment method (0 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)

Character*12 RMIDNT — VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).

Integer IERR — error code (output).

IERR = 0	no errors occurred
IERR > 0	error occurred
IERR = 1	bus not found
IERR = 2	switched shunt table is full
IERR = 4	prerequisite requirements for API are not met
IERR = -1	warnings - one or more of: - Invalid switched shunt control mode - Control mode is 4 and a blank VSC dc line name is specified - VSC dc line not found - Control mode is 6 and a blank FACTS device name is specified - FACTS device not found - Non-blank device name specified with control mode 0, 1, 2, 3 or 5 - Regulated bus not found - Type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - No regulated bus specified for plant, VSC converter, or switched shunt control - Control mode is 3 and the regulated bus has no generation - Control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - Control mode is 5 and the regulated bus has no switched shunt - Regulated bus is

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - Block has N or B zero and the other non-zero - Invalid number of steps in block - Block with non-zero N is beyond a previous N=0 block - Block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus - Regulated bus is 0 but a non-zero regulated node is specified - A remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.148. SWITCHED_SHUNT_DATA_5

This API routine is obsolete. It has been replaced by the API routine switched_shunt_data_6 .

Use this API to modify the data of an existing switched shunt in the working case or to add a new switched shunt to the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_DATA_5 IBUS 'ID' INTGAR(1)..INTGAR(21)
 REALAR(1)..REALAR(12) 'RMIDNT'
```

Python command syntax:

```
ierr = switched_shunt_data_5(ibus, id, intgar, realar, rmidnt)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_API_5(IBUS, ID, INTGAR, REALAR, RMIDNT, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
character*2 ID	Switched shunt identifier (input; '1' by default).	
Integer INTGAR (21)	array of 21 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, regulated bus number (0 by default)
	INTGAR(11)	NODE, regulated node number (0 by default)
	INTGAR(12)	STAT, status (1 by default)
	INTGAR(13)	ADJM, adjustment method (0 by default)
	INTGAR(14)	ST1, status of block 1 (1 by default)
	INTGAR(15)	ST2, status of block 2 (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(16)	ST3, status of block 3 (1 by default)
	INTGAR(17)	ST4, status of block 4 (1 by default)
	INTGAR(18)	ST5, status of block 5 (1 by default)
	INTGAR(19)	ST6, status of block 6 (1 by default)
	INTGAR(20)	ST7, status of block 7 (1 by default)
	INTGAR(21)	ST8, status of block 8 (1 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)
	REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
	REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
	REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
	REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
	REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
	REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
	REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
	REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
	REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
	REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
	REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)
Character*12 RMIDNT	VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt table is full
	IERR = 3	identifier is more than two characters
	IERR = 4	blank shunt identifier is invalid
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	node-breaker terminal connection table is full
	IERR = -1	warnings - one or more of: - Invalid switched shunt control mode - Control mode is 4 and a blank VSC dc line name is specified - VSC

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

dc line not found - Control mode is 6 and a blank FACTS device name is specified - FACTS device not found - Non-blank device name specified with control mode 0, 1, 2, 3 or 5 - Regulated bus not found - Type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - No regulated bus specified for plant, VSC converter, or switched shunt control - Control mode is 3 and the regulated bus has no generation - Control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - Control mode is 5 and the regulated bus has no switched shunt - Regulated bus is not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - Block has N or B zero and the other non-zero - Invalid number of steps in block - Block with non-zero N is beyond a previous N=0 block - Block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus - Regulated bus is 0 but a non-zero regulated node is specified - A remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.149. SWITCHED_SHUNT_DATA_6

Use this API to modify the data of an existing switched shunt in the working case or to add a new switched shunt to the working case (see PSSE Program Operation Manual, Switched Shunt Data).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SWITCHED_SHUNT_DATA_6 IBUS 'ID' INTGAR(1)..INTGAR(21)
 REALAR(1)..REALAR(12) 'RMIDNT' 'SWSNAM'
```

Python command syntax:

```
ierr = switched_shunt_data_6(ibus, id, intgar, realar, rmidnt, swsnam)
```

Fortran command syntax:

```
CALL SWITCHED_SHUNT_API_6(IBUS, ID, INTGAR, REALAR, RMIDNT, SWSNAM, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
character*2 ID	Switched shunt identifier (input; '1' by default).	
Integer INTGAR (21)	array of 21 elements specifying (input).	
	INTGAR(1)	N1, number of steps for block 1 (0 by default)
	INTGAR(2)	N2, number of steps for block 2 (0 by default)
	INTGAR(3)	N3, number of steps for block 3 (0 by default)
	INTGAR(4)	N4, number of steps for block 4 (0 by default)
	INTGAR(5)	N5, number of steps for block 5 (0 by default)
	INTGAR(6)	N6, number of steps for block 6 (0 by default)
	INTGAR(7)	N7, number of steps for block 7 (0 by default)
	INTGAR(8)	N8, number of steps for block 8 (0 by default)
	INTGAR(9)	MODSW, control mode (1 by default)
	INTGAR(10)	SWREM, regulated bus number (0 by default)
	INTGAR(11)	NODE, regulated node number (0 by default)
	INTGAR(12)	STAT, status (1 by default)
	INTGAR(13)	ADJM, adjustment method (0 by default)
	INTGAR(14)	ST1, status of block 1 (1 by default)
	INTGAR(15)	ST2, status of block 2 (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(16)	ST3, status of block 3 (1 by default)
	INTGAR(17)	ST4, status of block 4 (1 by default)
	INTGAR(18)	ST5, status of block 5 (1 by default)
	INTGAR(19)	ST6, status of block 6 (1 by default)
	INTGAR(20)	ST7, status of block 7 (1 by default)
	INTGAR(21)	ST8, status of block 8 (1 by default)
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	B1, admittance increment per step for block 1 (0.0 by default)
	REALAR(2)	B2, admittance increment per step for block 2 (0.0 by default)
	REALAR(3)	B3, admittance increment per step for block 3 (0.0 by default)
	REALAR(4)	B4, admittance increment per step for block 4 (0.0 by default)
	REALAR(5)	B5, admittance increment per step for block 5 (0.0 by default)
	REALAR(6)	B6, admittance increment per step for block 6 (0.0 by default)
	REALAR(7)	B7, admittance increment per step for block 7 (0.0 by default)
	REALAR(8)	B8, admittance increment per step for block 8 (0.0 by default)
	REALAR(9)	VSWHI, desired voltage upper limit (1.0 by default)
	REALAR(10)	VSWLO, desired voltage lower limit (1.0 by default)
	REALAR(11)	BINIT, present switched shunt admittance (0.0 by default)
	REALAR(12)	RMPCT, percent of contributed reactive power (100.0 by default)
Character*12 RMIDNT	VSC dc line name (MODSW =4) or FACTS device name (MODSW = 6) (input; blank by default).	
character*40 SWSNAM	switched shunt name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt table is full
	IERR = 3	identifier is more than two characters
	IERR = 4	blank shunt identifier is invalid
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	node-breaker terminal connection table is full
	IERR = -1	warnings - one or more of: - Invalid switched shunt control mode - Control mode is 4 and

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

a blank VSC dc line name is specified - VSC dc line not found - Control mode is 6 and a blank FACTS device name is specified - FACTS device not found - Non-blank device name specified with control mode 0, 1, 2, 3 or 5 - Regulated bus not found - Type code of regulated bus is not 1 or 2 - VHIGH is less than VLOW for voltage controlling switched shunt - No regulated bus specified for plant, VSC converter, or switched shunt control - Control mode is 3 and the regulated bus has no generation - Control mode is 4 and the regulated bus is not a converter of the specified VSC dc line - Control mode is 5 and the regulated bus has no switched shunt - Regulated bus is not the sending end bus of the specified FACTS device - VLOW is not less than VHIGH for plant, VSC converter, switched shunt, or FACTS device control - VHIGH is greater than 1.0 for plant, VSC converter, switched shunt, or FACTS device control - VLOW is less than 0.0 for plant, VSC converter, switched shunt, or FACTS device control - RMPCT is not positive - Block has N or B zero and the other non-zero - Invalid number of steps in block - Block with non-zero N is beyond a previous N=0 block - Block 1 has no steps. - Reactor block is out of sequence. - Initial admittance is below minimum. - Initial admittance is above maximum. - Initial admittance is not on a step. - Block has admittances with different signs in positive and zero sequences. - Invalid switched shunt status. - Invalid switched shunt adjustment method - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus - Regulated bus is 0 but a non-zero regulated node is specified - A remote regulated bus is specified for a type 3 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.150. THREE_WINDING_DATA

This API routine is obsolete. It has been replaced by the API routines three_wnd_impedance_data and three_wnd_winding_data .

Use this API to modify the data of an existing three-winding transformer in the working case or to add a new three-winding transformer to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_THREE_WINDING_DATA  IBUS  JBUS  KBUS  'CKT'  INTGAR(1)..INTGAR(17)
 REALAR(1)..REALAR(41)  'NAME'
```

Python command syntax:

```
ierr, realaro = three_winding_data(ibus, jbus, kbus, ckt, intgar, realari,
 name)
```

Fortran command syntax:

```
CALL THREE_WINDING_API(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR, NAME, IERR,
 NERR)
```

Where:

Integer IBUS	bus number of one of the buses (input).	
Integer JBUS	bus number one of the other buses (input).	
Integer KBUS	bus number of the third bus (input).	
Character*2 CKT	transformer circuit identifier (input).	
Integer INTGAR (17)	array of 17 elements specifying (input).	
	INTGAR(1)	O1, first owner number
	INTGAR(2)	O2, second owner number
	INTGAR(3)	O3, third owner number
	INTGAR(4)	O4, fourth owner number
	INTGAR(5)	NTP1, number of tap positions
	INTGAR(6)	TAB1, impedance correction table number
	INTGAR(7)	CONT1, controlled bus number
	INTGAR(8)	SICOD1, negative for controlled bus on winding one side
	INTGAR(9)	COD1, adjustment control mode flag (-3 through +3)
	INTGAR(10)	CW, winding data I/O code
	INTGAR(11)	CZ, impedance data I/O code
	INTGAR(12)	CM, magnetizing admittance data I/O code
	INTGAR(13)	STAT, branch status
	INTGAR(14)	NMETBS, non-metered end bus number (IBUS, JBUS or KBUS)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(15)	WN1BUS, winding one side bus number (IBUS, JBUS or KBUS)
	INTGAR(16)	WN2BUS, winding two side bus number (IBUS, JBUS or KBUS)
	INTGAR(17)	WN3BUS, winding three side bus number (IBUS, JBUS or KBUS)
Real REALAR (53)	array of elements specifying (first 41 are input values; last 12 are output values)(input).(output).	
	REALAR(1)	R1-2, nominal bus one to two transformer resistance
	REALAR(2)	X1-2, nominal bus one to two transformer reactance
	REALAR(3)	R3-1, nominal bus three to one transformer resistance
	REALAR(4)	X3-1, nominal bus three to one transformer reactance
	REALAR(5)	R2-3, nominal bus two to three transformer resistance
	REALAR(6)	X2-3, nominal bus two to three transformer reactance
	REALAR(7)	SBS1-2, winding one to two base MVA
	REALAR(8)	SBS2-3, winding two to three base MVA
	REALAR(9)	SBS3-1, winding three to one base MVA
	REALAR(10)	MAG1, magnetizing conductance or no-load losses
	REALAR(11)	MAG2, magnetizing susceptance or exciting current
	REALAR(12)	WINDV1, winding one ratio or voltage
	REALAR(13)	NOMV1, winding one nominal voltage
	REALAR(14)	1, winding one phase shift angle
	REALAR(15)	RATA1, winding one rating set A line rating
	REALAR(16)	RATB1, winding one rating set B line rating
	REALAR(17)	RATC1, winding one rating set C line rating
	REALAR(18)	WINDV2, winding two ratio or voltage
	REALAR(19)	NOMV2, winding two nominal voltage
	REALAR(20)	2, winding two phase shift angle
	REALAR(21)	RATA2, winding two rating set A line rating
	REALAR(22)	RATB2, winding two rating set B line rating
	REALAR(23)	RATC2, winding two rating set C line rating
	REALAR(24)	WINDV3, winding three ratio or voltage
	REALAR(25)	NOMV3, winding three nominal voltage
	REALAR(26)	3, winding three phase shift angle
	REALAR(27)	RATA3, winding three rating set A line rating
	REALAR(28)	RATB3, winding three rating set B line rating
	REALAR(29)	RATC3, winding three rating set C line rating

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALAR(30)	F1, first owner fraction
REALAR(31)	F2, second owner fraction
REALAR(32)	F3, third owner fraction
REALAR(33)	F4, fourth owner fraction
REALAR(34)	RMA1, winding one ratio/angle high limit
REALAR(35)	RMI1, winding one ratio/angle low limit
REALAR(36)	VMA1, voltage or flow upper limit
REALAR(37)	VMI1, voltage or flow lower limit
REALAR(38)	CR1, load drop compensating resistance
REALAR(39)	CX1, load drop compensating reactance
REALAR(40)	VMSTAR, star bus voltage magnitude
REALAR(41)	TAR, star bus voltage angle
REALAR(42)	actual bus one to two resistance (returned)
REALAR(43)	actual bus one to two reactance (returned)
REALAR(44)	actual bus three to one resistance (returned)
REALAR(45)	actual bus three to one reactance (returned)
REALAR(46)	nominal winding one resistance (returned)
REALAR(47)	nominal winding one reactance (returned)
REALAR(48)	nominal winding two resistance (returned)
REALAR(49)	nominal winding two reactance (returned)
REALAR(50)	nominal winding three resistance (returned)
REALAR(51)	nominal winding three reactance (returned)
REALAR(52)	actual winding one resistance (returned)
REALAR(53)	actual winding one reactance (returned)

Character*40 NAME — transformer name (input).

Real REALARI — (input).

Real REALARO — (output).

Integer IERR — error code (output).

IERR = 0	no error occurred
IERR > 0	error occurred
IERR = 1	bus not found
IERR = 2	circuit identifier is more than two characters
IERR = 3	invalid CW, CZ and/or CM value
IERR = 4	CW, CZ and/or CM >1 but no base voltage at one or more buses
IERR = 5	three different buses must be specified
IERR = 6	blank circuit identifier
IERR = 7	circuit identifier may not start with '&'
IERR = 8	three-winding transformer table full
IERR = 9	bus table full
IERR = 10	not enough branch table entries available
IERR = 11	not enough room in two winding transformer table

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	IERR = 12	not enough room in branch ownership table
	IERR = 13	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid set of winding buses. - Invalid transformer status. - Invalid non-metered end bus. - Winding side buses interchanged. - Losses are too big relative to total impedance at winding base MVA. - Magnitude of winding impedance is less than 0.00001. - Controlled bus not found. - Invalid adjustment control mode flag. - Any two-winding transformer warnings
Integer NERR	number of entries in PBUF02 containing error messages (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.151. THREE_WND_IMPEDANCE_DATA

This API routine is obsolete. It has been replaced by the API routine three_wnd_imped_data_3 .

Use this API routine to modify the impedance data of an existing three-winding transformer in the working case, or to add a new three-winding transformer to the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_THREE_WND_IMPEDANCE_DATA  IBUS  JBUS  KBUS  'CKT'
 INTGAR(1)..INTGAR(12)  REALAR(1)..REALAR(17)  'NAME'
```

Python command syntax:

```
ierr, realaro = three_wnd_impedance_data(ibus, jbus, kbus, ckt, intgar,
 realari, name)
```

Fortran command syntax:

```
CALL THREE_WINDING_IMPEDANCE_API_2(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR,
 NAME, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).
Integer JBUS	bus number one of the other buses (input; no default allowed).
Integer KBUS	bus number of the third bus (input; no default allowed).
Character*2 CKT	transformer circuit identifier (input; '1' by default).
Integer INTGAR (12)	array of 12 elements specifying (input).

INTGAR(1)	O1, first owner number (owner of bus IBUS by default)
INTGAR(2)	O2, second owner number (0 by default)
INTGAR(3)	O3, third owner number (0 by default)
INTGAR(4)	O4, fourth owner number (0 by default)
INTGAR(5)	CW, winding data I/O code (1 by default)
INTGAR(6)	CZ, impedance data I/O code (1 by default)
INTGAR(7)	CM, magnetizing admittance data I/O code (1 by default)
INTGAR(8)	STAT, branch status (1 by default)
INTGAR(9)	NMETBS, non-metered end bus number (IBUS, JBUS or KBUS) (JBUS by default)
INTGAR(10)	WN1BUS, winding one side bus number (IBUS, JBUS or KBUS) (IBUS by default)
INTGAR(11)	WN2BUS, winding two side bus number (IBUS, JBUS or KBUS) (JBUS by default)
INTGAR(12)	WN3BUS, winding three side bus number (IBUS, JBUS or KBUS) (KBUS by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real REALAR (23)	(first seventeen are input values; last six are output values) (input (output).).
Real REALARI (17)	array of 17 elements specifying corresponding to values REALAR(1) through REALAR(17) as described above(input).
	REALARI(1) R1-2, nominal bus one to two transformer resistance (0.0 by default)
	REALARI(2) X1-2, nominal bus one to two transformer reactance (0.0002 by default)
	REALARI(3) R2-3, nominal bus two to three transformer resistance (0.0 by default)
	REALARI(4) X2-3, nominal bus two to three transformer reactance (0.0002 by default)
	REALARI(5) R3-1, nominal bus three to one transformer resistance (0.0 by default)
	REALARI(6) X3-1, nominal bus three to one transformer reactance (0.0002 by default)
	REALARI(7) SBS1-2, winding one to two base MVA (SBASE by default)
	REALARI(8) SBS2-3, winding two to three base MVA (SBASE by default)
	REALARI(9) SBS3-1, winding three to one base MVA (SBASE by default)
	REALARI(10) MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(11) MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(12) F1, first owner fraction (1.0 by default)
	REALARI(13) F2, second owner fraction (0.0 by default)
	REALARI(14) F3, third owner fraction (0.0 by default)
	REALARI(15) F4, fourth owner fraction (0.0 by default)
	REALARI(16) VMSTAR, star bus voltage magnitude (1.0 by default)
	REALARI(17) TAR, star bus voltage angle (0.0 by default)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(18) through REALAR(23) as described above(output).
	REALARO(1) actual bus one to two resistance (returned)
	REALARO(2) actual bus one to two reactance (returned)
	REALARO(3) actual bus two to three resistance (returned)
	REALARO(4) actual bus two to three reactance (returned)
	REALARO(5) actual bus three to one resistance (returned)
	REALARO(6) actual bus three to one reactance (returned)
Character*40 NAME	transformer name (input; blank by default).
Integer IERR	error code (output).
	IERR = 0 no error occurred
	IERR > 0 fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	bus not found
IERR = 2	circuit identifier is more than two characters
IERR = 3	invalid CW, CZ and/or CM value
IERR = 4	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
IERR = 5	three different buses must be specified
IERR = 6	blank circuit identifier is invalid
IERR = 7	transformer circuit identifier may not start with &, * or @
IERR = 8	three-winding transformer table is full
IERR = 9	bus table is full
IERR = 10	not enough branch table entries are available
IERR = 11	not enough two-winding transformer table entries are available
IERR = 12	not enough branch ownership table entries are available
IERR = 14	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid set of winding buses. - Invalid CW, CZ and/or CM data value. - Invalid transformer status. - Invalid non-metered end bus. - Winding side buses interchanged. - Losses are too big relative to total impedance at winding base MVA. - Magnitude of winding impedance is less than 0.00001. - Any two-winding transformer warnings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.152. THREE_WND_IMPED_CHNG_3

This API routine is obsolete. It has been replaced by the API routine three_wnd_imped_chng_4 .

Use this API routine to modify the impedance data of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_THREE_WND_IMPED_CHNG_3  IBUS  JBUS  KBUS  'CKT'  INTGAR(1)..INTGAR(12)
  REALAR(1)..REALAR(17)  'CHARAR(1)'  'CHARAR(2)'
```

Python command syntax:

```
ierr, realaro = three_wnd_imped_chng_3(ibus, jbus, kbus, ckt, intgar,
 realari, charar)
```

Fortran command syntax:

```
CALL THREE_WINDING_IMPEDANCE_CHNG_3(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR,
 CHARAR, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer INTGAR (12)	array of 12 elements specifying (input).	
	INTGAR(1)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(2)	O2, second owner number (0 by default)
	INTGAR(3)	O3, third owner number (0 by default)
	INTGAR(4)	O4, fourth owner number (0 by default)
	INTGAR(5)	CW, winding data I/O code (1 by default)
	INTGAR(6)	CZ, impedance data I/O code (1 by default)
	INTGAR(7)	CM, magnetizing admittance data I/O code (1 by default)
	INTGAR(8)	STAT, branch status (1 by default)
	INTGAR(9)	NMETBS, non-metered end bus number (IBUS, JBUS or KBUS) (JBUS by default)
	INTGAR(10)	WN1BUS, winding one side bus number (IBUS, JBUS or KBUS) (IBUS by default)
	INTGAR(11)	WN2BUS, winding two side bus number (IBUS, JBUS or KBUS) (JBUS by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(12)	WN3BUS, winding three side bus number (IBUS, JBUS or KBUS) (KBUS by default)
Real REALAR (23)	(first seventeen are input values; last six are output values) (input (output).).	
Real REALARI (17)	array of 17 elements specifying corresponding to values REALAR(1) through REALAR(17) as described above(input).	
	REALARI(1)	R1-2, nominal bus one to two transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal bus one to two transformer reactance (0.0002 by default)
	REALARI(3)	R2-3, nominal bus two to three transformer resistance (0.0 by default)
	REALARI(4)	X2-3, nominal bus two to three transformer reactance (0.0002 by default)
	REALARI(5)	R3-1, nominal bus three to one transformer resistance (0.0 by default)
	REALARI(6)	X3-1, nominal bus three to one transformer reactance (0.0002 by default)
	REALARI(7)	SBS1-2, winding one to two base MVA (SBASE by default)
	REALARI(8)	SBS2-3, winding two to three base MVA (SBASE by default)
	REALARI(9)	SBS3-1, winding three to one base MVA (SBASE by default)
	REALARI(10)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(11)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(12)	F1, first owner fraction (1.0 by default)
	REALARI(13)	F2, second owner fraction (0.0 by default)
	REALARI(14)	F3, third owner fraction (0.0 by default)
	REALARI(15)	F4, fourth owner fraction (0.0 by default)
	REALARI(16)	VMSTAR, star bus voltage magnitude (1.0 by default)
	REALARI(17)	TAR, star bus voltage angle (0.0 by default)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(18) through REALAR(23) as described above(output).	
	REALARO(1)	actual bus one to two resistance (returned)
	REALARO(2)	actual bus one to two reactance (returned)
	REALARO(3)	actual bus two to three resistance (returned)
	REALARO(4)	actual bus two to three reactance (returned)
	REALARO(5)	actual bus three to one resistance (returned)
	REALARO(6)	actual bus three to one reactance (returned)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	NAME, transformer name (blank by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CHARAR(2)	VGRP, vector group name (blank by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	invalid CW, CZ and/or CM value
	IERR = 4	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
	IERR = 13	three-winding transformer not found
	IERR = 14	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid set of winding buses. - Invalid CW, CZ and/or CM data value. - Invalid transformer status. - Invalid non-metered end bus. - Winding side buses interchanged. - Losses are too big relative to total impedance at winding base MVA. - Magnitude of winding impedance is less than 0.00001. - Vector group not found. - Any two-winding transformer warnings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.153. THREE_WND_IMPED_CHNG_4

Use this API routine to modify the impedance data of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_THREE_WND_IMPED_CHNG_4  IBUS JBUS KBUS CKT INTGAR(1)..INTGAR(13)
 REALAR(1)..REALAR(17) NAMEAR VGRPAR
```

Python command syntax:

```
ierr, realaro = three_wnd_imped_chng_4(ibus, jbus, kbus, ckt, intgar,
 realari, namear, vgrpar)
```

Fortran command syntax:

```
CALL THREE_WINDING_IMPEDANCE_CHNG_4(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR,
 NAMEAR, VGRPAR, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer INTGAR (13)	array of 13 elements specifying (input).	
	INTGAR(1)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(2)	O2, second owner number (0 by default)
	INTGAR(3)	O3, third owner number (0 by default)
	INTGAR(4)	O4, fourth owner number (0 by default)
	INTGAR(5)	CW, winding data I/O code (1 by default)
	INTGAR(6)	CZ, impedance data I/O code (1 by default)
	INTGAR(7)	CM, magnetizing admittance data I/O code (1 by default)
	INTGAR(8)	STAT, branch status (1 by default)
	INTGAR(9)	NMETBS, non-metered end bus number (IBUS, JBUS or KBUS) (JBUS by default)
	INTGAR(10)	WN1BUS, winding one side bus number (IBUS, JBUS or KBUS) (IBUS by default)
	INTGAR(11)	WN2BUS, winding two side bus number (IBUS, JBUS or KBUS) (JBUS by default)
	INTGAR(12)	WN3BUS, winding three side bus number (IBUS, JBUS or KBUS) (KBUS by default)
	INTGAR(13)	ZCOD, impedance adjustment code (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real REALAR (23)	(first seventeen are input values; last six are output values) (input (output).).
Real REALARI (17)	array of 17 elements specifying corresponding to values REALAR(1) through REALAR(17) as described above(input).
	REALARI(1) R1-2, nominal bus one to two transformer resistance (0.0 by default)
	REALARI(2) X1-2, nominal bus one to two transformer reactance (0.0002 by default)
	REALARI(3) R2-3, nominal bus two to three transformer resistance (0.0 by default)
	REALARI(4) X2-3, nominal bus two to three transformer reactance (0.0002 by default)
	REALARI(5) R3-1, nominal bus three to one transformer resistance (0.0 by default)
	REALARI(6) X3-1, nominal bus three to one transformer reactance (0.0002 by default)
	REALARI(7) SBS1-2, winding one to two base MVA (SBASE by default)
	REALARI(8) SBS2-3, winding two to three base MVA (SBASE by default)
	REALARI(9) SBS3-1, winding three to one base MVA (SBASE by default)
	REALARI(10) MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(11) MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(12) F1, first owner fraction (1.0 by default)
	REALARI(13) F2, second owner fraction (0.0 by default)
	REALARI(14) F3, third owner fraction (0.0 by default)
	REALARI(15) F4, fourth owner fraction (0.0 by default)
	REALARI(16) VMSTAR, star bus voltage magnitude (1.0 by default)
	REALARI(17) TAR, star bus voltage angle (0.0 by default)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(18) through REALAR(23) as described above(output).
	REALARO(1) actual bus one to two resistance (returned)
	REALARO(2) actual bus one to two reactance (returned)
	REALARO(3) actual bus two to three resistance (returned)
	REALARO(4) actual bus two to three reactance (returned)
	REALARO(5) actual bus three to one resistance (returned)
	REALARO(6) actual bus three to one reactance (returned)
Character*40 NAMEAR	transformer name (input; blank by default).
Character*12 VGRPAR	vector group name (input; blank by default).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error occurred
IERR > 0	fatal error occurred
IERR = 1	bus not found
IERR = 2	circuit identifier is more than two characters
IERR = 3	invalid CW, CZ and/or CM value
IERR = 4	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
IERR = 13	three-winding transformer not found
IERR = 14	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid set of winding buses. - Invalid CW, CZ and/or CM data value. - Invalid transformer status. - Invalid non-metered end bus. - Winding side buses interchanged. - Losses are too big relative to total impedance at winding base MVA. - Magnitude of winding impedance is less than 0.00001. - Vector group not found. - Any two-winding transformer warnings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.154. THREE_WND_IMPED_DATA_3

This API routine is obsolete. It has been replaced by the API routine three_wnd_imped_data_4 .

Use this API routine to modify the impedance data of an existing three-winding transformer in the working case, or to add a new three-winding transformer to the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_THREE_WND_IMPED_DATA_3  IBUS  JBUS  KBUS  'CKT'  INTGAR(1)..INTGAR(12)
  REALAR(1)..REALAR(17)  'CHARAR(1)'  'CHARAR(2)'
```

Python command syntax:

```
ierr, realaro = three_wnd_imped_data_3(ibus, jbus, kbus, ckt, intgar,
 realari, charar)
```

Fortran command syntax:

```
CALL THREE_WINDING_IMPEDANCE_API_3(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR,
 CHARAR, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer INTGAR (12)	array of 12 elements specifying (input).	
	INTGAR(1)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(2)	O2, second owner number (0 by default)
	INTGAR(3)	O3, third owner number (0 by default)
	INTGAR(4)	O4, fourth owner number (0 by default)
	INTGAR(5)	CW, winding data I/O code (1 by default)
	INTGAR(6)	CZ, impedance data I/O code (1 by default)
	INTGAR(7)	CM, magnetizing admittance data I/O code (1 by default)
	INTGAR(8)	STAT, branch status (1 by default)
	INTGAR(9)	NMETBS, non-metered end bus number (IBUS, JBUS or KBUS) (JBUS by default)
	INTGAR(10)	WN1BUS, winding one side bus number (IBUS, JBUS or KBUS) (IBUS by default)
	INTGAR(11)	WN2BUS, winding two side bus number (IBUS, JBUS or KBUS) (JBUS by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(12)	WN3BUS, winding three side bus number (IBUS, JBUS or KBUS) (KBUS by default)
Real REALAR (23)	(first seventeen are input values; last six are output values) (input (output).).	
Real REALARI (17)	array of 17 elements specifying corresponding to values REALAR(1) through REALAR(17) as described above(input).	
	REALARI(1)	R1-2, nominal bus one to two transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal bus one to two transformer reactance (0.0002 by default)
	REALARI(3)	R2-3, nominal bus two to three transformer resistance (0.0 by default)
	REALARI(4)	X2-3, nominal bus two to three transformer reactance (0.0002 by default)
	REALARI(5)	R3-1, nominal bus three to one transformer resistance (0.0 by default)
	REALARI(6)	X3-1, nominal bus three to one transformer reactance (0.0002 by default)
	REALARI(7)	SBS1-2, winding one to two base MVA (SBASE by default)
	REALARI(8)	SBS2-3, winding two to three base MVA (SBASE by default)
	REALARI(9)	SBS3-1, winding three to one base MVA (SBASE by default)
	REALARI(10)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(11)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(12)	F1, first owner fraction (1.0 by default)
	REALARI(13)	F2, second owner fraction (0.0 by default)
	REALARI(14)	F3, third owner fraction (0.0 by default)
	REALARI(15)	F4, fourth owner fraction (0.0 by default)
	REALARI(16)	VMSTAR, star bus voltage magnitude (1.0 by default)
	REALARI(17)	TAR, star bus voltage angle (0.0 by default)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(18) through REALAR(23) as described above(output).	
	REALARO(1)	actual bus one to two resistance (returned)
	REALARO(2)	actual bus one to two reactance (returned)
	REALARO(3)	actual bus two to three resistance (returned)
	REALARO(4)	actual bus two to three reactance (returned)
	REALARO(5)	actual bus three to one resistance (returned)
	REALARO(6)	actual bus three to one reactance (returned)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	NAME, transformer name (blank by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CHARAR(2)	VGRP, vector group name (blank by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	invalid CW, CZ and/or CM value
	IERR = 4	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
	IERR = 5	three different buses must be specified
	IERR = 6	blank circuit identifier is invalid
	IERR = 7	transformer circuit identifier may not start with &, * or @
	IERR = 8	three-winding transformer table is full
	IERR = 9	bus table is full
	IERR = 10	not enough branch table entries are available
	IERR = 11	not enough two-winding transformer table entries are available
	IERR = 12	not enough branch ownership table entries are available
	IERR = 14	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid set of winding buses. - Invalid CW, CZ and/or CM data value. - Invalid transformer status. - Invalid non-metered end bus. - Winding side buses interchanged. - Losses are too big relative to total impedance at winding base MVA. - Magnitude of winding impedance is less than 0.00001. - Vector group not found. - Any two-winding transformer warnings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.155. THREE_WND_IMPED_DATA_4

Use this API routine to modify the impedance data of an existing three-winding transformer in the working case, or to add a new three-winding transformer to the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_THREE_WND_IMPED_DATA_4  IBUS JBUS KBUS CKT INTGAR(1)..INTGAR(13)
 REALAR(1)..REALAR(17) NAMEAR VGRPAR
```

Python command syntax:

```
ierr, realaro = three_wnd_imped_data_4(ibus, jbus, kbus, ckt, intgar,
 realari, namear, vgrpar)
```

Fortran command syntax:

```
CALL THREE_WINDING_IMPEDANCE_API_4(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR,
 NAMEAR, VGRPAR, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer INTGAR (13)	array of 13 elements specifying (input).	
	INTGAR(1)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(2)	O2, second owner number (0 by default)
	INTGAR(3)	O3, third owner number (0 by default)
	INTGAR(4)	O4, fourth owner number (0 by default)
	INTGAR(5)	CW, winding data I/O code (1 by default)
	INTGAR(6)	CZ, impedance data I/O code (1 by default)
	INTGAR(7)	CM, magnetizing admittance data I/O code (1 by default)
	INTGAR(8)	STAT, branch status (1 by default)
	INTGAR(9)	NMETBS, non-metered end bus number (IBUS, JBUS or KBUS) (JBUS by default)
	INTGAR(10)	WN1BUS, winding one side bus number (IBUS, JBUS or KBUS) (IBUS by default)
	INTGAR(11)	WN2BUS, winding two side bus number (IBUS, JBUS or KBUS) (JBUS by default)
	INTGAR(12)	WN3BUS, winding three side bus number (IBUS, JBUS or KBUS) (KBUS by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(13)	ZCOD, transformer impedance adjustment code (0 for winding impedances, 1 for bus-to-bus impedances) (0 by default)
Real REALAR (23)	(first seventeen are input values; last six are output values) (input (output).).	
Real REALARI (17)	array of 17 elements specifying corresponding to values REALAR(1) through REALAR(17) as described above(input).	
	REALARI(1)	R1-2, nominal bus one to two transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal bus one to two transformer reactance (0.0002 by default)
	REALARI(3)	R2-3, nominal bus two to three transformer resistance (0.0 by default)
	REALARI(4)	X2-3, nominal bus two to three transformer reactance (0.0002 by default)
	REALARI(5)	R3-1, nominal bus three to one transformer resistance (0.0 by default)
	REALARI(6)	X3-1, nominal bus three to one transformer reactance (0.0002 by default)
	REALARI(7)	SBS1-2, winding one to two base MVA (SBASE by default)
	REALARI(8)	SBS2-3, winding two to three base MVA (SBASE by default)
	REALARI(9)	SBS3-1, winding three to one base MVA (SBASE by default)
	REALARI(10)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(11)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(12)	F1, first owner fraction (1.0 by default)
	REALARI(13)	F2, second owner fraction (0.0 by default)
	REALARI(14)	F3, third owner fraction (0.0 by default)
	REALARI(15)	F4, fourth owner fraction (0.0 by default)
	REALARI(16)	VMSTAR, star bus voltage magnitude (1.0 by default)
	REALARI(17)	TAR, star bus voltage angle (0.0 by default)
Real REALARO (6)	array of 6 elements specifying corresponding to values REALAR(18) through REALAR(23) as described above(output).	
	REALARO(1)	actual bus one to two resistance (returned)
	REALARO(2)	actual bus one to two reactance (returned)
	REALARO(3)	actual bus two to three resistance (returned)
	REALARO(4)	actual bus two to three reactance (returned)
	REALARO(5)	actual bus three to one resistance (returned)
	REALARO(6)	actual bus three to one reactance (returned)
Character*40 NAMEAR	transformer name (input; blank by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*12 VGRPAR	vector group name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	invalid CW, CZ and/or CM value
	IERR = 4	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
	IERR = 5	three different buses must be specified
	IERR = 6	blank circuit identifier is invalid
	IERR = 7	transformer circuit identifier may not start with &, * or @
	IERR = 8	three-winding transformer table is full
	IERR = 9	bus table is full
	IERR = 10	not enough branch table entries are available
	IERR = 11	not enough two-winding transformer table entries are available
	IERR = 12	not enough branch ownership table entries are available
	IERR = 14	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid set of winding buses. - Invalid CW, CZ and/or CM data value. - Invalid transformer status. - Invalid non-metered end bus. - Winding side buses interchanged. - Losses are too big relative to total impedance at winding base MVA. - Magnitude of winding impedance is less than 0.00001. - Vector group not found. - Any two-winding transformer warnings defined in two_winding_data_6

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.156. THREE_WND_WINDING_DATA

This API routine is obsolete. It has been replaced by the API routine three_wnd_winding_data_3 .

Use this API to modify the data of one winding of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_THREE_WND_WINDING_DATA  IBUS  JBUS  KBUS  'CKT'  WARG
 INTGAR(1)..INTGAR(5)  REALAR(1)..REALAR(12)
```

Python command syntax:

```
ierr, realaro = three_wnd_winding_data(ibus, jbus, kbus, ckt, warg,
 intgar, realari)
```

Fortran command syntax:

```
CALL THREE_WINDING_WINDING_API_2(IBUS, JBUS, KBUS, CKT, WARG, INTGAR,
 REALAR, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer WARG	winding number (1, 2 or 3) (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	NTPi, number of tap positions (33 by default)
	INTGAR(2)	TABi, impedance correction table number (0 by default)
	INTGAR(3)	CONTi, controlled bus number (0 by default)
	INTGAR(4)	SICODi, negative for controlled bus on winding bus side (1 by default)
	INTGAR(5)	CODi, adjustment control mode flag (-3 through +3) (0 by default)
Real REALAR (16)	(first twelve are input values; last four are output values) (input (output).).	
Real REALARI (12)	array of 12 elements specifying corresponding to values REALAR(1) through REALAR(12) as described above(input).	
	REALARI(1)	WINDVi, winding ratio or voltage (1.0 by default if CW of this transformer is 1 or 3; base voltage of the winding bus by default if CW is 2)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(2)	NOMVi, winding nominal voltage (0.0 by default)
	REALARI(3)	i, winding phase shift angle (0.0 by default)
	REALARI(4)	RATAi, winding rating set A line rating (0.0 by default)
	REALARI(5)	RATBi, winding rating set B line rating (0.0 by default)
	REALARI(6)	RATCi, winding rating set C line rating (0.0 by default)
	REALARI(7)	RMAi, winding ratio/angle high limit (1.1 by default)
	REALARI(8)	RMIi, winding ratio/angle low limit (0.9 by default)
	REALARI(9)	VMAi, winding voltage or flow upper limit (1.1 by default)
	REALARI(10)	VMIi, winding voltage or flow lower limit (0.9 by default)
	REALARI(11)	CRi, winding load drop compensating resistance (0.0 by default)
	REALARI(12)	CXi, winding load drop compensating reactance (0.0 by default)
Real REALARO (4)	array of 4 elements specifying corresponding to values REALAR(13) through REALAR(16) as described above(output).	
	REALARO(1)	winding nominal resistance (returned)
	REALARO(2)	winding nominal reactance (returned)
	REALARO(3)	winding actual resistance (returned)
	REALARO(4)	winding actual reactance (returned)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	invalid CW, CZ and/or CM value
	IERR = 4	CW and/or CZ >1 but no base voltage at winding bus
	IERR = 5	three winding transformer not found
	IERR = 6	invalid winding number
	IERR = 7	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Controlled bus not found. - Invalid adjustment control mode flag. - Any two-winding transformer warnings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.157. THREE_WND_WINDING_DATA_3

This API routine is obsolete. It has been replaced by the API routine three_wnd_winding_data_4 .

Use this API to modify the data of one winding of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_THREE_WND_WINDING_DATA_3  IBUS  JBUS  KBUS  'CKT'  WARG
 INTGAR(1)..INTGAR(5)  REALAR(1)..REALAR(12)
```

Python command syntax:

```
ierr, realaro = three_wnd_winding_data_3(ibus, jbus, kbus, ckt, warg,
 intgar, realari)
```

Fortran command syntax:

```
CALL THREE_WINDING_WINDING_API_3(IBUS, JBUS, KBUS, CKT, WARG, INTGAR,
 REALAR, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer WARG	winding number (1, 2 or 3) (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	NTPi, number of tap positions (33 by default)
	INTGAR(2)	TABi, impedance correction table number (0 by default)
	INTGAR(3)	CONTi, controlled bus number (0 by default)
	INTGAR(4)	SICODi, negative for controlled bus on winding bus side (1 by default)
	INTGAR(5)	CODi, adjustment control mode flag (-3 through +3, -5 or 5) (0 by default)
Real REALAR (17)	(first thirteen are input values; last four are output values) (input (output).).	
Real REALARI (13)	array of 13 elements specifying corresponding to values REALAR(1) through REALAR(13) as described above(input).	
	REALARI(1)	WINDVi, winding ratio or voltage (1.0 by default if CW of this transformer is 1 or 3; base voltage of the winding bus by default if CW is 2)
	REALARI(2)	NOMVi, winding nominal voltage (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(3)	i, winding phase shift angle (0.0 by default)
	REALARI(4)	RATAi, winding rating set A line rating (0.0 by default)
	REALARI(5)	RATBi, winding rating set B line rating (0.0 by default)
	REALARI(6)	RATCi, winding rating set C line rating (0.0 by default)
	REALARI(7)	RMAi, winding ratio/angle high limit (1.1 by default)
	REALARI(8)	RMIi, winding ratio/angle low limit (0.9 by default)
	REALARI(9)	VMAi, winding voltage or flow upper limit (1.1 by default)
	REALARI(10)	VMIi, winding voltage or flow lower limit (0.9 by default)
	REALARI(11)	CRi, winding load drop compensating resistance (0.0 by default)
	REALARI(12)	CXi, winding load drop compensating reactance (0.0 by default)
	REALARI(13)	CNXAi, winding connection angle (0.0 by default). Used with adjustment control mode 5 (unsymmetric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (4)	array of 4 elements specifying corresponding to values REALAR(14) through REALAR(17) as described above(output).	
	REALARO(1)	winding nominal resistance (returned)
	REALARO(2)	winding nominal reactance (returned)
	REALARO(3)	winding actual resistance (returned)
	REALARO(4)	winding actual reactance (returned)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	invalid CW, CZ and/or CM value
	IERR = 4	CW and/or CZ >1 but no base voltage at winding bus
	IERR = 5	three winding transformer not found
	IERR = 6	invalid winding number
	IERR = 7	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Controlled bus not found. - Invalid adjustment control mode flag. - Any two-winding transformer warnings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.158. THREE_WND_WINDING_DATA_4

This API routine is obsolete. It has been replaced by the API routine three_wnd_winding_data_5 .

Use this API to modify the data of one winding of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_THREE_WND_WINDING_DATA_4  IBUS JBUS KBUS CKT WARG INTGAR(1)..INTGAR(5)
 REALAR(1)..REALAR(10) RATINGS(1)..RATINGS(12)
```

Python command syntax:

```
ierr, realaro = three_wnd_winding_data_4(ibus, jbus, kbus, ckt, warg,
 intgar, realari, ratings)
```

Fortran command syntax:

```
CALL THREE_WINDING_WINDING_DATA_4(IBUS, JBUS, KBUS, CKT, WARG, INTGAR,
 REALAR, RATINGS, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer WARG	winding number (1, 2 or 3) (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	NTPi, number of tap positions (33 by default)
	INTGAR(2)	TABi, impedance correction table number (0 by default)
	INTGAR(3)	CONTi, controlled bus number (0 by default)
	INTGAR(4)	SICODi, negative for controlled bus on winding bus side (1 by default)
	INTGAR(5)	CODi, adjustment control mode flag (-3 through +3, -5 or 5) (0 by default)
Real REALAR (14)	(first ten are input values; last four are output values) (input (output).).	
Real REALARI (10)	array of 10 elements specifying corresponding to values REALAR(1) through REALAR(10) as described above(input).	
	REALARI(1)	WINDVi, winding ratio or voltage (1.0 by default if CW of this transformer is 1 or 3; base voltage of the winding bus by default if CW is 2)
	REALARI(2)	NOMVi, winding nominal voltage (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(3)	i, winding phase shift angle (0.0 by default)
	REALARI(4)	RMAi, winding ratio/angle high limit (1.1 by default)
	REALARI(5)	RMIi, winding ratio/angle low limit (0.9 by default)
	REALARI(6)	VMAi, winding voltage or flow upper limit (1.1 by default)
	REALARI(7)	VMIi, winding voltage or flow lower limit (0.9 by default)
	REALARI(8)	CRi, winding load drop compensating resistance (0.0 by default)
	REALARI(9)	CXi, winding load drop compensating reactance (0.0 by default)
	REALARI(10)	CNXAi, winding connection angle (0.0 by default). Used with adjustment control mode 5 (unsymmetric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (4)	array of 4 elements specifying corresponding to values REALAR(11) through REALAR(14) as described above(output).	
	REALARO(1)	winding nominal resistance (returned)
	REALARO(2)	winding nominal reactance (returned)
	REALARO(3)	winding actual resistance (returned)
	REALARO(4)	winding actual reactance (returned)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1i, winding rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2i, winding rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3i, winding rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4i, winding rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5i, winding rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6i, winding rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7i, winding rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8i, winding rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9i, winding rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10i, winding rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11i, winding rating set 11 line rating (alias is RATE11) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RATINGS(12)	RATE12i, winding rating set 12 line rating (alias is RATE12) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	invalid CW, CZ and/or CM value
	IERR = 4	CW and/or CZ >1 but no base voltage at winding bus
	IERR = 5	three winding transformer not found
	IERR = 6	invalid winding number
	IERR = 7	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Controlled bus not found. - Invalid adjustment control mode flag. - Any two-winding transformer warnings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.159. THREE_WND_WINDING_DATA_5

Use this API to modify the data of one winding of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Non-Transformer Branch Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_THREE_WND_WINDING_DATA_5  IBUS JBUS KBUS CKT WARG INTGAR(1)..INTGAR(6)
 REALAR(1)..REALAR(10) RATINGS(1)..RATINGS(12)
```

Python command syntax:

```
ierr, realaro = three_wnd_winding_data_5(ibus, jbus, kbus, ckt, warg,
 intgar, realari, ratings)
```

Fortran command syntax:

```
CALL THREE_WINDING_WINDING_DATA_5(IBUS, JBUS, KBUS, CKT, WARG, INTGAR,
 REALAR, RATINGS, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer WARG	winding number (1, 2 or 3) (input; no default allowed).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	NTPi, number of tap positions (33 by default)
	INTGAR(2)	TABi, impedance correction table number (0 by default)
	INTGAR(3)	CONTi, controlled bus number (0 by default)
	INTGAR(4)	NODEi, controlled node number (0 by default)
	INTGAR(5)	SICODi, negative for controlled bus on winding bus side (1 by default)
	INTGAR(6)	CODi, adjustment control mode flag (-3 through +3, -5 or 5) (0 by default)
Real REALAR (14)	(first ten are input values; last four are output values) (input (output).).	
Real REALARI (10)	array of 10 elements specifying corresponding to values REALAR(1) through REALAR(10) as described above(input).	
	REALARI(1)	WINDVi, winding ratio or voltage (1.0 by default if CW of this transformer is 1 or 3; base voltage of the winding bus by default if CW is 2)
	REALARI(2)	NOMVi, winding nominal voltage (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(3)	i, winding phase shift angle (0.0 by default)
	REALARI(4)	RMAi, winding ratio/angle high limit (1.1 by default)
	REALARI(5)	RMIi, winding ratio/angle low limit (0.9 by default)
	REALARI(6)	VMAi, winding voltage or flow upper limit (1.1 by default)
	REALARI(7)	VMIi, winding voltage or flow lower limit (0.9 by default)
	REALARI(8)	CRi, winding load drop compensating resistance (0.0 by default)
	REALARI(9)	CXi, winding load drop compensating reactance (0.0 by default)
	REALARI(10)	CNXAi, winding connection angle (0.0 by default). Used with adjustment control mode 5 (unsymmetric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (4)	array of 4 elements specifying corresponding to values REALAR(11) through REALAR(14) as described above(output).	
	REALARO(1)	winding nominal resistance (returned)
	REALARO(2)	winding nominal reactance (returned)
	REALARO(3)	winding actual resistance (returned)
	REALARO(4)	winding actual reactance (returned)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1i, winding rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2i, winding rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3i, winding rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4i, winding rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5i, winding rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6i, winding rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7i, winding rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8i, winding rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9i, winding rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10i, winding rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11i, winding rating set 11 line rating (alias is RATE11) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RATINGS(12)	RATE12i, winding rating set 12 line rating (alias is RATE12) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	invalid CW, CZ and/or CM value
	IERR = 4	CW and/or CZ >1 but no base voltage at winding bus
	IERR = 5	three winding transformer not found
	IERR = 6	invalid winding number
	IERR = 7	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Controlled bus not found. - Invalid adjustment control mode flag. - Any two-winding transformer warnings defined in two_winding_data_6

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.160. TRANSACTION_ADD_MOD

Use this API to add or modify transaction event data in PSSE working memory.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_TRANSACTION_ADD_MOD  EVENT  INTGAR(1)  INTGAR(2)  REALAR(1)  REALAR(2)
  'DSCRP'
```

Python command syntax:

```
ierr = transaction_add_mod(event, intgar, realar, dscrp)
```

Fortran command syntax:

```
CALL TRANSACTION_ADD_MOD(EVENT, INTGAR, REALAR, DSCRP, IERR, NERR)
```

Where:

Integer EVENT	transaction event number (input).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	transaction event status (0 = out-of-service, 1 = in-service)
	INTGAR(2)	transaction event priority
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	transaction event magnitude in MW
	REALAR(2)	transaction event curtailment in MW
Character*24 DSCRP	transaction event description (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	error occurred
	IERR = 2	invalid transaction event number
	IERR > 2	unable to add or modify transaction event data: error condition detected in data access routine
	IERR = 10	prerequisite requirements for API are not met
	IERR = -1	warnings: - None
Integer NERR	number of error entries in PBUF02 (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.161. TRANSACTION_DELETE

Use this API to delete a transaction event from PSSE working memory.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_TRANSACTION_DELETE  EVENT
```

Python command syntax:

```
ierr = transaction_delete(event)
```

Fortran command syntax:

```
CALL TRANSACTION_DELETE(EVENT, IERR, NERR)
```

Where:

Integer EVENT	transaction event number (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid transaction event number
	IERR = 2	prerequisite requirements for API are not met
Integer NERR	number of error entries in PBUF02 (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.162. TRANSFER_CHNG

Use this API routine to modify the data of an inter-area transfer in the working case (see PSSE Program Operation Manual, Zone Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_TRANSFER_CHNG  APPLYF  ARFROM  ARTO  'TRID'  REALAR(1)
```

Python command syntax:

```
ierr = transfer_chng(applyf, arfrom, arto, trid, realar)
```

Fortran command syntax:

```
CALL TRANSFER_CHNG(APPLYF, ARFROM, ARTO, TRID, REALAR, IERR)
```

Where:

Integer APPLYF	APPLYF = 1 to modify from and to area desired net interchange entries (DESINT) (input).	
	APPLYF = 2	to leave DESINT entries unchanged. (Input; no default allowed.)
Integer ARFROM	from area number (must be an area already defined) (input; no default allowed).	
Integer ARTO	to area number (must be an area already defined) (input; no default allowed).	
Character*1 TRID	inter-area transfer identifier (input; '1' by default).	
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	PTRAN, MW comprising this inter-area transfer (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid area number
	IERR = 2	inter-area transfer identifier is more than one character
	IERR = 3	invalid APPLYF value
	IERR = 8	area not found
	IERR = 9	inter-area transfer not found
	IERR = 10	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Changes to desired area net interchanges if APPLYF = 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.163. TRANSFER_DATA

Use this API routine to modify the data of an inter-area transfer in the working case, or to add a new inter-area transfer to the working case (see PSSE Program Operation Manual, Zone Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TRANSFER_DATA  APPLYF  ARFROM  ARTO  'TRID'  REALAR(1)
```

Python command syntax:

```
ierr = transfer_data(applyf, arfrom, arto, trid, realar)
```

Fortran command syntax:

```
CALL TRANSFER_API_2(APPLYF, ARFROM, ARTO, TRID, REALAR, IERR)
```

Where:

Integer APPLYF	APPLYF = 1 to modify from and to area desired net interchange entries (DESINT) (input).	
	APPLYF = 2	to leave DESINT entries unchanged. (Input; no default allowed.)
Integer ARFROM	from area number (must be an area already defined) (input; no default allowed).	
Integer ARTO	to area number (must be an area already defined) (input; no default allowed).	
Character*1 TRID	inter-area transfer identifier (input; '1' by default).	
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	PTRAN, MW comprising this inter-area transfer (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid area number
	IERR = 2	inter-area transfer identifier is more than one character
	IERR = 3	invalid APPLYF value
	IERR = 4	invalid inter-area transfer identifier
	IERR = 5	transfer from an area to itself
	IERR = 6	inter-area transfer table is full
	IERR = 7	inter-area transfer already exists
	IERR = 8	area not found
	IERR = 10	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Changes to desired area net interchanges if APPLYF = 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.164. TWO_TERMINAL_DC_LINE_CHNG

Use this API routine to modify the link data of an existing two-terminal dc line in the working case (see PSSE Program Operation Manual, Area Interchange Data). Converter data for the rectifier and inverter are specified via the TWO_TERM_DC_CONVR_DATA API routine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_TWO_TERMINAL_DC_LINE_CHNG DC INTGAR(1) INTGAR(2) REALAR(1)..REALAR(8)
 'METER'
```

Python command syntax:

```
ierr, realaro = two_terminal_dc_line_chng(dc, intgar, realari, meter)
```

Fortran command syntax:

```
CALL TWO_TERMINAL_DC_LINE_CHNG(NUM, INTGAR, REALAR, METER, IERR)

CALL TWO_TERMINAL_DC_LINE_CHNG_I(NUM, INTGAR, REALAR, METER, IERR)

CALL TWO_TERMINAL_DC_LINE_CHNG_C(NAME, INTGAR, REALAR, METER, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which data is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which data is to be specified; no default allowed	
Character*12 NAME	name of the two-terminal dc line for which data is to be specified; no default allowed	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	MDC, control mode (0, 1 or 2) (0 by default)
	INTGAR(2)	CCITMX, CCC Newton solution iteration limit (20 by default)
Real REALAR (9)	first eight are input values; last value is output (input (output).).	
Real REALARI (8)	array of 8 elements specifying values REALAR(1) through REALAR(8) as described above(input).	
	REALARI(1)	SETVL, scheduled current (amps) or power (MW) demand (0.0 by default)
	REALARI(2)	VSCHD, scheduled dc voltage (kV) (1.0 by default)
	REALARI(3)	VCMOD, mode switch dc voltage (kV) (0.0 by default)
	REALARI(4)	DELTI, margin (pu, > 0) (0.0 by default)
	REALARI(5)	RDC, dc line resistance (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(6)	RCOMP, compounding resistance (0.0 by default)
	REALARI(7)	DCVMIN, minimum dc voltage (kV) (0.0 by default)
	REALARI(8)	CCCACC, CCC Newton solution acceleration (1.0 by default)
Real REALARO (1)	array of 1 elements specifying value REALAR(9) as described above(output).	
	REALARO(1)	dc line current (amps) (returned)
Character*1 METER	metered end flag: 'R' or 'I' (input; 'I' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 2	metered end code is more than one character
	IERR = 3	blank dc line name is invalid
	IERR = 4	two-terminal dc line not found
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid dc line control mode - invalid dc line resistance - invalid compounding resistance - compounding resistance exceeds line resistance - scheduled voltage must exceed mode switch voltage - invalid metered end code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.165. TWO_TERMINAL_DC_LINE_DATA

Use this API routine to modify the link data of an existing two-terminal dc line in the working case, or to add a new two-terminal dc line and its link data to the working case (see PSSE Program Operation Manual, Area Interchange Data). Converter data for the rectifier and inverter are specified via the TWO_TERM_DC_CONVR_DATA API routine.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TWO_TERMINAL_DC_LINE_DATA DC INTGAR(1) INTGAR(2) REALAR(1)..REALAR(8)
 'METER'
```

Python command syntax:

```
ierr, realaro = two_terminal_dc_line_data(dc, intgar, realari, meter)
```

Fortran command syntax:

```
CALL TWO_TERMINAL_DC_LINE_API(NUM, INTGAR, REALAR, METER, IERR)

CALL TWO_TERMINAL_DC_LINE_API_I(NUM, INTGAR, REALAR, METER, IERR)

CALL TWO_TERMINAL_DC_LINE_API_C(NAME, INTGAR, REALAR, METER, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which data is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which data is to be specified; no default allowed	
Character*12 NAME	name of the two-terminal dc line for which data is to be specified; no default allowed	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	MDC, control mode (0, 1 or 2) (0 by default)
	INTGAR(2)	CCITMX, CCC Newton solution iteration limit (20 by default)
Real REALAR (9)	first eight are input values; last value is output (input (output).).	
Real REALARI (8)	array of 8 elements specifying values REALAR(1) through REALAR(8) as described above(input).	
	REALARI(1)	SETVL, scheduled current (amps) or power (MW) demand (0.0 by default)
	REALARI(2)	VSCHD, scheduled dc voltage (kV) (1.0 by default)
	REALARI(3)	VCMOD, mode switch dc voltage (kV) (0.0 by default)
	REALARI(4)	DELTI, margin (pu, > 0) (0.0 by default)
	REALARI(5)	RDC, dc line resistance (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(6)	RCOMP, compounding resistance (0.0 by default)
	REALARI(7)	DCVMIN, minimum dc voltage (kV) (0.0 by default)
	REALARI(8)	CCCACC, CCC Newton solution acceleration (1.0 by default)
Real REALARO (1)	array of 1 elements specifying value REALAR(9) as described above(output).	
	REALARO(1)	dc line current (amps) (returned)
Character*1 METER	metered end flag: 'R' or 'I' (input; 'I' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	two-terminal dc line table is full
	IERR = 2	metered end code is more than one character
	IERR = 3	blank dc line name is invalid
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid dc line control mode - invalid dc line resistance - invalid compounding resistance - compounding resistance exceeds line resistance - scheduled voltage must exceed mode switch voltage - invalid metered end code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.166. TWO_TERM_DC_CONVERTER_DATA_3

Use this API to modify the data of a dc converter of an existing two-terminal dc line in the working case (see PSSE Program Operation Manual, Area Interchange Data). The dc line must have previously been specified via the TWO_TERMINAL_DC_LINE_DATA API.

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_TWO_TERM_DC_CONVERTER_DATA_3  CNVFLG  NAME  INTGAR(1)..INTGAR(6)
 REALAR(1)..REALAR(11)  'ID'
```

Python command syntax:

```
ierr, realaro = two_term_dc_converter_data_3(cnvflg, name, intgar,
 realari, id)
```

Fortran command syntax:

```
CALL TWO_TERM_DC_CONVERTER_DATA_3 (CNVFLG, NAME, INTGAR, REALAR, ID, IERR)
```

Where:

Integer CNVFLG	converter type (input; no default allowed).	
	CNVFLG = 1	for rectifier data.
	CNVFLG = 2	for inverter data.
Character*12 NAME	name of the two-terminal dc line for which data for one of its converters is to be specified (input; no default allowed).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	IPR or IPI, converter bus number (no default allowed)
	INTGAR(2)	NBR or NBI, number of bridges in series (0 by default)
	INTGAR(3)	ICR or ICI, zero or bus number of firing angle bus (0 by default)
	INTGAR(4)	NDR or NDI, zero or node number of firing angle bus node (0 by default)
	INTGAR(5)	IFR or IFI, zero or bus number of winding one side of a two-winding ac transformer (0 by default)
	INTGAR(6)	ITR or ITI, zero or bus number of winding two side of a two-winding ac transformer (0 by default)
Real REALAR (15)	(first eleven are input values; last four are output values) (input (output).).	
Real REALARI (11)	array of elements specifying values REALAR(1) through REALAR(11) as described above(input).	
	REALARI(1)	NR or ANMNI, minimum firing angle (90.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(2)	XR or ANMXI, maximum firing angle (0.0 by default)
	REALARI(3)	RCR or RCI, commutating resistance (0.0 by default)
	REALARI(4)	XCR or XCI, commutating reactance (0.0 by default)
	REALARI(5)	EBASR or EBASI, primary base ac voltage (0.0 by default)
	REALARI(6)	TRR or TRI, transformer ratio (1.0 by default)
	REALARI(7)	TAPR or TAPI, tap setting (1.0 by default)
	REALARI(8)	TMXR or TMXI, maximum tap setting (1.5 by default)
	REALARI(9)	TMNR or TMNI, minimum tap setting (0.51 by default)
	REALARI(10)	STPR or STPI, tap step (0.00625 by default)
	REALARI(11)	XCAPR or XCAPI, capacitive reactance of CCC type converter (0.0 by default)
Real REALARO (4)	array of elements specifying values REALAR(12) through REALAR(15) as described above(output).	
	REALARO(1)	firing angle (returned)
	REALARO(2)	dc voltage (kV) (returned)
	REALARO(3)	real power into converter (PWRCOD) (returned)
	REALARO(4)	reactive power into converter (PWRCOD) (returned)
Character*2 ID	ac transformer circuit identifier, IDR or IDI (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	dc line not found
	IERR = 2	converter type is not 1 or 2
	IERR = 3	transformer circuit identifier is more than two characters
	IERR = 4	no converter bus is specified
	IERR = 5	converter bus not found
	IERR = 6	node-breaker terminal connection table is full
	IERR = 7	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Converter bus is dummy bus of a multi-section line. - Multi-section line grouping deleted. - Converter maximum angle is less than its minimum angle. - Converter maximum tap is less than its minimum tap. - Invalid tap step. - Commutating bus not found. - ac transformer bus not

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

found. - Specified branch not found. - Specified branch is not a transformer. - From bus of specified branch is not winding one side bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.167. TWO_TERM_DC_CONVR_DATA

This API routine is obsolete. It has been replaced by the API routine two_term_dc_converter_data_3 .

Use this API to modify the data of a dc converter of an existing two-terminal dc line in the working case (see PSSE Program Operation Manual, Area Interchange Data). The dc line must have previously been specified via the TWO_TERMINAL_DC_LINE_DATA API.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_TWO_TERM_DC_CONVR_DATA  CNVFLG  DC  INTGAR(1)..INTGAR(5)
 REALAR(1)..REALAR(11)  'ID'
```

Python command syntax:

```
ierr, realaro = two_term_dc_convr_data(cnvflg, dc, intgar, realari, id)
```

Fortran command syntax:

```
CALL TWO_TERMINAL_DC_CONVERTER_API (CNVFLG, NUM, INTGAR, REALAR, ID, IERR)

CALL TWO_TERMINAL_DC_CONVERTER_API_I (CNVFLG, NUM, INTGAR, REALAR, ID,
 IERR)

CALL TWO_TERMINAL_DC_CONVERTER_API_C (CNVFLG, NAME, INTGAR, REALAR, ID,
 IERR)
```

Where:

Integer CNVFLG	converter type (input; no default allowed).	
	CNVFLG = 1	for rectifier data.
	CNVFLG = 2	for inverter data.
Integer or Character*12 DC	quoted name of the two-terminal dc line for which data for one of its converters is to be specified; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which data for one of its converters is to be specified; no default allowed	
Character*12 NAME	name of the two-terminal dc line for which data for one of its converters is to be specified; no default allowed	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	IPR or IPI, converter bus number (no default allowed)
	INTGAR(2)	NBR or NBI, number of bridges in series (0 by default)
	INTGAR(3)	ICR or ICI, zero or bus number of firing angle bus (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(4)	IFR or IFI, zero or bus number of winding one side of a two-winding ac transformer (0 by default)
	INTGAR(5)	ITR or ITI, zero or bus number of winding two side of a two-winding ac transformer (0 by default)
Real REALAR (15)	(first eleven are input values; last four are output values) (input (output).).	
Real REALARI (11)	array of elements specifying values REALAR(1) through REALAR(11) as described above(input).	
	REALARI(1)	NR or ANMNI, minimum firing angle (90.0 by default)
	REALARI(2)	XR or ANMXI, maximum firing angle (0.0 by default)
	REALARI(3)	RCR or RCI, commutating resistance (0.0 by default)
	REALARI(4)	XCR or XCI, commutating reactance (0.0 by default)
	REALARI(5)	EBASR or EBASI, primary base ac voltage (0.0 by default)
	REALARI(6)	TRR or TRI, transformer ratio (1.0 by default)
	REALARI(7)	TAPR or TAPI, tap setting (1.0 by default)
	REALARI(8)	TMXR or TMXI, maximum tap setting (1.5 by default)
	REALARI(9)	TMNR or TMNI, minimum tap setting (0.51 by default)
	REALARI(10)	STPR or STPI, tap step (0.00625 by default)
	REALARI(11)	XCAPR or XCAPI, capacitive reactance of CCC type converter (0.0 by default)
Real REALARO (4)	array of elements specifying values REALAR(12) through REALAR(15) as described above(output).	
	REALARO(1)	firing angle (returned)
	REALARO(2)	dc voltage (kV) (returned)
	REALARO(3)	real power into converter (PWRCOD) (returned)
	REALARO(4)	reactive power into converter (PWRCOD) (returned)
Character*2 ID	ac transformer circuit identifier, IDR or IDI (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	dc line not found
	IERR = 2	converter type is not 1 or 2
	IERR = 3	transformer circuit identifier is more than two characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	no converter bus is specified
IERR = 5	converter bus not found
IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Converter bus is dummy bus of a multi-section line. - Multi-section line grouping deleted. - Converter maximum angle is less than its minimum angle. - Converter maximum tap is less than its minimum tap. - Invalid tap step. - Commutating bus not found. - ac transformer bus not found. - Specified branch not found. - Specified branch is not a transformer. - From bus of specified branch is not winding one side bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.168. TWO_WINDING_CHNG_4

This API routine is obsolete. It has been replaced by the API routine two_winding_chng_5 .

Use this API routine to modify the data of an existing two-winding transformer in the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_TWO_WINDING_CHNG_4  IBUS  JBUS  'CKT'  INTGAR(1)..INTGAR(15)
 REALAR(1)..REALAR(24)  'CHARAR(1)'  'CHARAR(2)'
```

Python command syntax:

```
ierr, realaro = two_winding_chng_4(ibus, jbus, ckt, intgar, realari,
 charar)
```

Fortran command syntax:

```
CALL TWO_WINDING_CHNG_4(IBUS, JBUS, CKT, INTGAR, REALAR, CHARAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (15)	array of 15 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(12)	COD1, adjustment control mode flag (-5 through +5) (0 by default)
	INTGAR(13)	CW, winding data I/O code (1 by default)
	INTGAR(14)	CZ, impedance data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(15)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (26)	(first twenty-four are input values; last two are output values) (input (output).).	
Real REALARI (24)	array of 24 elements specifying corresponding to values REALAR(1) through REALAR(24) as described above(input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	RATA1, rating set A line rating (0.0 by default)
	REALARI(10)	RATB1, rating set B line rating (0.0 by default)
	REALARI(11)	RATC1, rating set C line rating (0.0 by default)
	REALARI(12)	F1, first owner fraction (1.0 by default)
	REALARI(13)	F2, second owner fraction (1.0 by default)
	REALARI(14)	F3, third owner fraction (1.0 by default)
	REALARI(15)	F4, fourth owner fraction (1.0 by default)
	REALARI(16)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(17)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(18)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(19)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(20)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(21)	VMI1, voltage or flow lower limit (0.9 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(22)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(23)	CX1, load drop compensating reactance (0.0 by default)
	REALARI(24)	CNXA1, winding connection angle (0.0 by default). Used with adjustment control mode 5 (unsymmatric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(25) through REALAR(26) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	NAME, transformer name (blank by default)
	CHARAR(2)	VGRP, vector group name (blank by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
	IERR = 11	invalid CW, CZ and/or CM value
	IERR = 12	two-winding transformer not found
	IERR = 13	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nominal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base volt-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

age - invalid tap ratio - tap ratio is outside the band 0.5 to 1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - magnetizing susceptance MAG2 is positive - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged). - winding connection angle normalized between -360 and 360 - adjustment control mode flag and the winding connection angle are inconsistent. - for asymmetric phase shifter, abnormally large or small tap setting results at RMA1 (or RMI1) and CNXA1. - for asymmetric phase shifter, the specified and derived winding tap settings differ. - Vector group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.169. TWO_WINDING_CHNG_5

This API routine is obsolete. It has been replaced by the API routine two_winding_chng_6 .

Use this API routine to modify the data of an existing two-winding transformer in the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_TWO_WINDING_CHNG_5  IBUS JBUS CKT INTGAR(1)..INTGAR(15)
 REALAR(1)..REALAR(21) RATINGS(1)..RATINGS(12) NAMEAR VGRPAR
```

Python command syntax:

```
ierr, realaro = two_winding_chng_5(ibus, jbus, ckt, intgar, realari,
 ratings, namear, vgrpar)
```

Fortran command syntax:

```
CALL TWO_WINDING_CHNG_5(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR,
 VGRPAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (15)	array of 15 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(12)	COD1, adjustment control mode flag (-5 through +5) (0 by default)
	INTGAR(13)	CW, winding data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(14)	CZ, impedance data I/O code (1 by default)
	INTGAR(15)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (23)	(first twenty-four are input values; last two are output values) (input (output).).	
Real REALARI (21)	array of 21 elements specifying (input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	F1, first owner fraction (1.0 by default)
	REALARI(10)	F2, second owner fraction (1.0 by default)
	REALARI(11)	F3, third owner fraction (1.0 by default)
	REALARI(12)	F4, fourth owner fraction (1.0 by default)
	REALARI(13)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(14)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(15)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(16)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(17)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(18)	VMI1, voltage or flow lower limit (0.9 by default)
	REALARI(19)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(20)	CX1, load drop compensating reactance (0.0 by default)
	REALARI(21)	CNXA1, winding connection angle (0.0 by default). Used with adjustment control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		5 (unsymmatric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(25) through REALAR(26) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	transformer name (input; blank by default).	
Character*12 VGRPAR	vector group name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &, * or @
	IERR = 7	branch table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	branch ownership table is full
IERR = 9	two-winding transformer table is full
IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
IERR = 11	invalid CW, CZ and/or CM value
IERR = 13	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nominal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base voltage - invalid tap ratio - tap ratio is outside the band 0.5 to 1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - magnetizing susceptance MAG2 is positive - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged). - winding connection angle normalized between -360 and 360 - adjustment control mode flag and the winding connection angle are inconsistent. - for asymmetric phase shifter, abnormally large or small tap setting results at RMA1 (or RMI1) and CNXA1. - for asymmetric phase shifter, the specified and derived winding tap settings differ. - vector group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.170. TWO_WINDING_CHNG_6

Use this API routine to modify the data of an existing two-winding transformer in the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_TWO_WINDING_CHNG_6( IBUS JBUS CKT INTGAR(1)..INTGAR(16)
 REALAR(1)..REALAR(21) RATINGS(1)..RATINGS(12) NAMEAR VGRPAR
```

Python command syntax:

```
ierr, realaro = two_winding_chng_6(ibus, jbus, ckt, intgar, realari,
 ratings, namear, vgrpar)
```

Fortran command syntax:

```
CALL TWO_WINDING_CHNG_6(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR,
 VGRPAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (16)	array of 16 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	NODE1, controlled node number (0 by default)
	INTGAR(12)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(13)	COD1, adjustment control mode flag (-5 through +5) (0 by default)
	INTGAR(14)	CW, winding data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(15)	CZ, impedance data I/O code (1 by default)
	INTGAR(16)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (23)	(first twenty-one are input values; last two are output values) (input (output).).	
Real REALARI (21)	array of 21 elements specifying (input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	F1, first owner fraction (1.0 by default)
	REALARI(10)	F2, second owner fraction (1.0 by default)
	REALARI(11)	F3, third owner fraction (1.0 by default)
	REALARI(12)	F4, fourth owner fraction (1.0 by default)
	REALARI(13)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(14)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(15)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(16)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(17)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(18)	VMI1, voltage or flow lower limit (0.9 by default)
	REALARI(19)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(20)	CX1, load drop compensating reactance (0.0 by default)
	REALARI(21)	CNXA1, winding connection angle (0.0 by default). Used with adjustment control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		5 (unsymmatric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(25) through REALAR(26) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	transformer name (input; blank by default).	
Character*12 VGRPAR	vector group name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &, * or @
	IERR = 7	branch table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	branch ownership table is full
IERR = 9	two-winding transformer table is full
IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
IERR = 11	invalid CW, CZ and/or CM value
IERR = 12	branch not found
IERR = 13	node-breaker terminal connection table is full
IERR = 14	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nominal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base voltage - invalid tap ratio - tap ratio is outside the band 0.5 to 1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - magnetizing susceptance MAG2 is positive - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged). - winding connection angle normalized between -360 and 360 - adjustment control mode flag and the winding connection angle are inconsistent. - for asymmetric phase shifter, abnormally large or small tap setting results at RMA1 (or RMI1) and CNXA1. - for asymmetric phase shifter, the specified and derived winding tap settings differ. - vector group not found - Invalid node number - Non-zero con-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

trolled node specified but controlled bus does not belong to a substation - Controlled node not found - Controlled node is not connected to any bus section of the regulated bus - Controlled bus is 0 but a non-zero controlled node is specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.171. TWO_WINDING_DATA

This API routine is obsolete. It has been replaced by the API routine two_winding_data_3 .

Use this API to modify the data of an existing two-winding transformer in the working case, or to add a new two-winding transformer to the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_TWO_WINDING_DATA  IBUS  JBUS  'CKT'  INTGAR(1)..INTGAR(15)
 REALAR(1)..REALAR(23)  'NAME'
```

Python command syntax:

```
ierr, realaro = two_winding_data(ibus, jbus, ckt, intgar, realari, name)
```

Fortran command syntax:

```
CALL TWO_WINDING_API_2(IBUS, JBUS, CKT, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (15)	array of 15 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(12)	COD1, adjustment control mode flag (-4 through +4) (0 by default)
	INTGAR(13)	CW, winding data I/O code (1 by default)
	INTGAR(14)	CZ, impedance data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(15)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (25)	(first twenty-three are input values; last two are ouptut values) (input (output).).	
Real REALARI (23)	array of 23 elements specifying corresponding to values REALAR(1) through REALAR(23) as described above(input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	RATA1, rating set A line rating (0.0 by default)
	REALARI(10)	RATB1, rating set B line rating (0.0 by default)
	REALARI(11)	RATC1, rating set C line rating (0.0 by default)
	REALARI(12)	F1, first owner fraction (1.0 by default)
	REALARI(13)	F2, second owner fraction (1.0 by default)
	REALARI(14)	F3, third owner fraction (1.0 by default)
	REALARI(15)	F4, fourth owner fraction (1.0 by default)
	REALARI(16)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(17)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(18)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(19)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(20)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(21)	VMI1, voltage or flow lower limit (0.9 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(22)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(23)	CX1, load drop compensating reactance (0.0 by default)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(24) through REALAR(25) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Character*12 NAME	transformer name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &, * or @
	IERR = 7	branch table is full
	IERR = 8	branch ownership table is full
	IERR = 9	two-winding transformer table is full
	IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
	IERR = 11	invalid CW, CZ and/or CM value
	IERR = 13	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nominal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base voltage - invalid tap ratio - tap ratio is outside the band 0.5 to

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.172. TWO_WINDING_DATA_3

This API routine is obsolete. It has been replaced by the API routine two_winding_data_4 .

Use this API to modify the data of an existing two-winding transformer in the working case, or to add a new two-winding transformer to the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_TWO_WINDING_DATA_3  IBUS  JBUS  'CKT'  INTGAR(1)..INTGAR(15)
 REALAR(1)..REALAR(24)  'NAME'
```

Python command syntax:

```
ierr, realaro = two_winding_data_3(ibus, jbus, ckt, intgar, realari, name)
```

Fortran command syntax:

```
CALL TWO_WINDING_API_3(IBUS, JBUS, CKT, INTGAR, REALAR, NAME, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (15)	array of 15 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(12)	COD1, adjustment control mode flag (-5 through +5) (0 by default)
	INTGAR(13)	CW, winding data I/O code (1 by default)
	INTGAR(14)	CZ, impedance data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(15)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (26)	(first twenty-four are input values; last two are output values) (input (output).).	
Real REALARI (24)	array of 24 elements specifying corresponding to values REALAR(1) through REALAR(24) as described above(input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	RATA1, rating set A line rating (0.0 by default)
	REALARI(10)	RATB1, rating set B line rating (0.0 by default)
	REALARI(11)	RATC1, rating set C line rating (0.0 by default)
	REALARI(12)	F1, first owner fraction (1.0 by default)
	REALARI(13)	F2, second owner fraction (1.0 by default)
	REALARI(14)	F3, third owner fraction (1.0 by default)
	REALARI(15)	F4, fourth owner fraction (1.0 by default)
	REALARI(16)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(17)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(18)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(19)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(20)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(21)	VMI1, voltage or flow lower limit (0.9 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(22)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(23)	CX1, load drop compensating reactance (0.0 by default)
	REALARI(24)	CNXA1, winding connection angle (0.0 by default). Used with adjustment control mode 5 (unsymmatric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(25) through REALAR(26) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Character*12 NAME	transformer name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &, * or @
	IERR = 7	branch table is full
	IERR = 8	branch ownership table is full
	IERR = 9	two-winding transformer table is full
	IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
	IERR = 11	invalid CW, CZ and/or CM value
	IERR = 13	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nominal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base voltage - invalid tap ratio - tap ratio is outside the band 0.5 to 1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - magnetizing susceptance MAG2 is positive - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged). - winding connection angle normalized between -360 and 360 - adjustment control mode flag and the winding connection angle are inconsistent. - for asymmetric phase shifter, abnormally large or small tap setting results at RMA1 (or RMI1) and CNXA1. - for asymmetric phase shifter, the specified and derived winding tap settings differ

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.173. TWO_WINDING_DATA_4

This API routine is obsolete. It has been replaced by the API routine two_winding_data_5 .

Use this API to modify the data of an existing two-winding transformer in the working case, or to add a new two-winding transformer to the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_TWO_WINDING_DATA_4  IBUS  JBUS  'CKT'  INTGAR(1)..INTGAR(15)
 REALAR(1)..REALAR(24)  'CHARAR(1)'  'CHARAR(2)'
```

Python command syntax:

```
ierr, realaro = two_winding_data_4(ibus, jbus, ckt, intgar, realari,
 charar)
```

Fortran command syntax:

```
CALL TWO_WINDING_API_4(IBUS, JBUS, CKT, INTGAR, REALAR, CHARAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (15)	array of 15 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(12)	COD1, adjustment control mode flag (-5 through +5) (0 by default)
	INTGAR(13)	CW, winding data I/O code (1 by default)
	INTGAR(14)	CZ, impedance data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(15)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (26)	(first twenty-four are input values; last two are output values) (input (output).).	
Real REALARI (24)	array of 24 elements specifying corresponding to values REALAR(1) through REALAR(24) as described above(input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	RATA1, rating set A line rating (0.0 by default)
	REALARI(10)	RATB1, rating set B line rating (0.0 by default)
	REALARI(11)	RATC1, rating set C line rating (0.0 by default)
	REALARI(12)	F1, first owner fraction (1.0 by default)
	REALARI(13)	F2, second owner fraction (1.0 by default)
	REALARI(14)	F3, third owner fraction (1.0 by default)
	REALARI(15)	F4, fourth owner fraction (1.0 by default)
	REALARI(16)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(17)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(18)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(19)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(20)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(21)	VMI1, voltage or flow lower limit (0.9 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALARI(22)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(23)	CX1, load drop compensating reactance (0.0 by default)
	REALARI(24)	CNXA1, winding connection angle (0.0 by default). Used with adjustment control mode 5 (unsymmatric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(25) through REALAR(26) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	NAME, transformer name (blank by default)
	CHARAR(2)	VGRP, vector group name (blank by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &, * or @
	IERR = 7	branch table is full
	IERR = 8	branch ownership table is full
	IERR = 9	two-winding transformer table is full
	IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
	IERR = 11	invalid CW, CZ and/or CM value
	IERR = 13	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nom-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

inal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base voltage - invalid tap ratio - tap ratio is outside the band 0.5 to 1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - magnetizing susceptance MAG2 is positive - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged). - winding connection angle normalized between -360 and 360 - adjustment control mode flag and the winding connection angle are inconsistent. - for asymmetric phase shifter, abnormally large or small tap setting results at RMA1 (or RMI1) and CNXA1. - for asymmetric phase shifter, the specified and derived winding tap settings differ. - vector group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.174. TWO_WINDING_DATA_5

This API routine is obsolete. It has been replaced by the API routine two_winding_data_6 .

Use this API to modify the data of an existing two-winding transformer in the working case, or to add a new two-winding transformer to the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_TWO_WINDING_DATA_5  IBUS JBUS CKT INTGAR(1)..INTGAR(15)
 REALAR(1)..REALAR(21) RATINGS(1)..RATINGS(12) NAMEAR VGRPAR
```

Python command syntax:

```
ierr, realaro = two_winding_data_5(ibus, jbus, ckt, intgar, realari,
 ratings, namear, vgrpar)
```

Fortran command syntax:

```
CALL TWO_WINDING_DATA_5(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR,
 VGRPAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (15)	array of 15 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(12)	COD1, adjustment control mode flag (-5 through +5) (0 by default)
	INTGAR(13)	CW, winding data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(14)	CZ, impedance data I/O code (1 by default)
	INTGAR(15)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (23)	(first twenty-one are input values; last two are output values) (input (output).).	
Real REALARI (21)	array of 21 elements specifying (input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	F1, first owner fraction (1.0 by default)
	REALARI(10)	F2, second owner fraction (1.0 by default)
	REALARI(11)	F3, third owner fraction (1.0 by default)
	REALARI(12)	F4, fourth owner fraction (1.0 by default)
	REALARI(13)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(14)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(15)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(16)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(17)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(18)	VMI1, voltage or flow lower limit (0.9 by default)
	REALARI(19)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(20)	CX1, load drop compensating reactance (0.0 by default)
	REALARI(21)	CNXA1, winding connection angle (0.0 by default). Used with adjustment control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		5 (unsymmatric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(25) through REALAR(26) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	transformer name (input; blank by default).	
Character*12 VGRPAR	vector group name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &, * or @
	IERR = 7	branch table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	branch ownership table is full
IERR = 9	two-winding transformer table is full
IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
IERR = 11	invalid CW, CZ and/or CM value
IERR = 13	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nominal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base voltage - invalid tap ratio - tap ratio is outside the band 0.5 to 1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - magnetizing susceptance MAG2 is positive - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged). - winding connection angle normalized between -360 and 360 - adjustment control mode flag and the winding connection angle are inconsistent. - for asymmetric phase shifter, abnormally large or small tap setting results at RMA1 (or RMI1) and CNXA1. - for asymmetric phase shifter, the specified and derived winding tap settings differ. - vector group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.175. TWO_WINDING_DATA_6

Use this API to modify the data of an existing two-winding transformer in the working case, or to add a new two-winding transformer to the working case (see PSSE Program Operation Manual, Owner Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_TWO_WINDING_DATA_6  IBUS JBUS CKT INTGAR(1)..INTGAR(16)
 REALAR(1)..REALAR(21) RATINGS(1)..RATINGS(12) NAMEAR VGRPAR
```

Python command syntax:

```
ierr, realaro = two_winding_data_6(ibus, jbus, ckt, intgar, realari,
 ratings, namear, vgrpar)
```

Fortran command syntax:

```
CALL TWO_WINDING_DATA_6(IBUS, JBUS, CKT, INTGAR, REALAR, RATINGS, NAMEAR,
 VGRPAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (16)	array of 16 elements specifying (input).	
	INTGAR(1)	STAT, branch status (1 by default)
	INTGAR(2)	METBUS, metered end bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(3)	O1, first owner number (owner of bus IBUS by default)
	INTGAR(4)	O2, second owner number (0 by default)
	INTGAR(5)	O3, third owner number (0 by default)
	INTGAR(6)	O4, fourth owner number (0 by default)
	INTGAR(7)	NTP1, number of tap positions (33 by default)
	INTGAR(8)	TAB1, impedance correction table number (0 by default)
	INTGAR(9)	WN1BUS, winding one side bus number (IBUS or JBUS) (IBUS by default)
	INTGAR(10)	CONT1, controlled bus number (0 by default)
	INTGAR(11)	NODE1, controlled node number (0 by default)
	INTGAR(12)	SICOD1, negative for controlled bus on winding one side (1 by default)
	INTGAR(13)	COD1, adjustment control mode flag (-5 through +5) (0 by default)
	INTGAR(14)	CW, winding data I/O code (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(15)	CZ, impedance data I/O code (1 by default)
	INTGAR(16)	CM, magnetizing admittance data I/O code (1 by default)
Real REALAR (23)	(first twenty-one are input values; last two are output values) (input (output).).	
Real REALARI (21)	array of 21 elements specifying (input).	
	REALARI(1)	R1-2, nominal transformer resistance (0.0 by default)
	REALARI(2)	X1-2, nominal transformer reactance (THRSHZ by default; 0.0001 if THRSHZ = 0.0)
	REALARI(3)	SBS1-2, winding base MVA (SBASE by default)
	REALARI(4)	WINDV1, winding one ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus IBUS by default if CW is 2)
	REALARI(5)	NOMV1, winding one nominal voltage (0.0 by default)
	REALARI(6)	1, winding one phase shift angle (0.0 by default)
	REALARI(7)	WINDV2, winding two ratio/voltage (1.0 by default if CW is 1 or 3; base voltage of bus JBUS by default if CW is 2)
	REALARI(8)	NOMV2, winding two nominal voltage (0.0 by default)
	REALARI(9)	F1, first owner fraction (1.0 by default)
	REALARI(10)	F2, second owner fraction (1.0 by default)
	REALARI(11)	F3, third owner fraction (1.0 by default)
	REALARI(12)	F4, fourth owner fraction (1.0 by default)
	REALARI(13)	MAG1, magnetizing conductance or no-load losses (0.0 by default)
	REALARI(14)	MAG2, magnetizing susceptance or exciting current (0.0 by default)
	REALARI(15)	RMA1, winding one ratio/angle high limit (1.1 by default)
	REALARI(16)	RMI1, winding one ratio/angle low limit (0.9 by default)
	REALARI(17)	VMA1, voltage or flow upper limit (1.1 by default)
	REALARI(18)	VMI1, voltage or flow lower limit (0.9 by default)
	REALARI(19)	CR1, load drop compensating resistance (0.0 by default)
	REALARI(20)	CX1, load drop compensating reactance (0.0 by default)
	REALARI(21)	CNXA1, winding connection angle (0.0 by default). Used with adjustment control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		5 (unsymmatric phase shift control of active power) implemented in PSSE version 32.1)
Real REALARO (2)	array of 2 elements specifying corresponding to values REALAR(25) through REALAR(26) as described above(output).	
	REALARO(1)	actual transformer resistance (returned)
	REALARO(2)	actual transformer reactance (returned)
Real RATINGS (12)	array of 12 elements specifying (input).	
	RATINGS(1)	RATE1, rating set 1 line rating (alias is RATE1) (0.0 by default)
	RATINGS(2)	RATE2, rating set 2 line rating (alias is RATE2) (0.0 by default)
	RATINGS(3)	RATE3, rating set 3 line rating (alias is RATE3) (0.0 by default)
	RATINGS(4)	RATE4, rating set 4 line rating (alias is RATE4) (0.0 by default)
	RATINGS(5)	RATE5, rating set 5 line rating (alias is RATE5) (0.0 by default)
	RATINGS(6)	RATE6, rating set 6 line rating (alias is RATE6) (0.0 by default)
	RATINGS(7)	RATE7, rating set 7 line rating (alias is RATE7) (0.0 by default)
	RATINGS(8)	RATE8, rating set 8 line rating (alias is RATE8) (0.0 by default)
	RATINGS(9)	RATE9, rating set 9 line rating (alias is RATE9) (0.0 by default)
	RATINGS(10)	RATE10, rating set 10 line rating (alias is RATE10) (0.0 by default)
	RATINGS(11)	RATE11, rating set 11 line rating (alias is RATE11) (0.0 by default)
	RATINGS(12)	RATE12, rating set 12 line rating (alias is RATE12) (0.0 by default)
Character*40 NAMEAR	transformer name (input; blank by default).	
Character*12 VGRPAR	vector group name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a non-transformer branch
	IERR = 4	branch from a bus to itself
	IERR = 5	blank circuit identifier is invalid
	IERR = 6	circuit identifier may not start with &, * or @
	IERR = 7	branch table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	branch ownership table is full
IERR = 9	two-winding transformer table is full
IERR = 10	CW, CZ and/or CM >1 but at least one end-point bus has no base voltage
IERR = 11	invalid CW, CZ and/or CM value
IERR = 13	node-breaker terminal connection table is full
IERR = 14	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid metered end bus - invalid transformer status - invalid owner number - invalid ownership fraction - no ownership data specified. - multi-section line grouping deleted - branch ownership table is full--only first n owners retained - owner table is full - rating is negative - invalid winding 1 side bus number - invalid CW, CZ and/or CM data value - controlled bus not found - invalid adjustment control mode flag - invalid impedance correction table number - invalid number of tap positions - winding nominal voltage is negative - winding bus base voltage and nominal voltage differ by more than 20% - invalid winding voltage - winding voltage and bus base voltage differ by more than 50% - RMAX is greater than 150% of bus base voltage - RMIN is less than 50% of bus base voltage - invalid tap ratio - tap ratio is outside the band 0.5 to 1.5 pu - RMAX is greater than 1.5 - RMIN is less than 0.5 - RMAX is less than RMIN - VMAX is less than VMIN - phase shift angle normalized between -180 and 180 - winding base MVA must be positive - MAG1 is too big relative to MAG2 at winding base MVA - magnetizing susceptance MAG2 is positive - losses are too big relative to total impedance at winding base MVA - branch reactance is 0.0 - winding 1 and winding 2 sides are interchanged (any two-terminal dc lines controlled by this transformer are logged). - winding connection angle normalized between -360 and 360 - adjustment control mode flag and the winding connection angle are inconsistent. - for asymmetric phase shifter, abnormally large or small tap setting results at RMA1 (or RMI1) and CNXA1. - for asymmetric phase shifter, the specified and derived winding tap settings differ. - vector group not found - Invalid node number - Non-zero controlled node specified but controlled bus does not belong to a substation - Con-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

trolled node not found - Controlled node is not connected to any bus section of the regulated bus - Controlled bus is 0 but a non-zero controlled node is specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.176. VOLTAGE_DROOP_CHNG

Use this API routine to modify the voltage droop control in the working case(see PSSE Program Operation Manual.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_VOLTAGE_DROOP_CHNG NAME INTGAR(1) INTGAR(2) REALAR(1)..REALAR(7)
```

Python command syntax:

```
ierr = voltage_droop_chng(name, intgar, realar)
```

Fortran command syntax:

```
CALL VOLTAGE_DROOP_CHNG(NAME, INTGAR, REALAR, IERR)
```

Where:

Character*40 NAME	name of the voltage droop control for which data is to be specified (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying array of 2 elements specifying(input).	
	INTGAR(1)	STATUS, voltage droop control status (0, 1) (0 by default)
	INTGAR(2)	REGBUS, regulated bus of voltage droop control (20 by default)
Real REALAR (7)	array of 7 elements specifying array of 7 elements specifying(input).	
	REALAR(1)	QMAX, upper Q limit of voltage droop control (1.0 by default)
	REALAR(2)	QDB, Q output when voltage is within deadband (0.0 by default)
	REALAR(3)	QMIN, lower Q limit of voltage droop control (-1.0 by default)
	REALAR(4)	VLOW, lower voltage limit of voltage droop control (0.9 by default)
	REALAR(5)	VDBLOW, lower voltage deadband limit of voltage droop control (0.98 by default)
	REALAR(6)	VDBHIGH, higher voltage deadband limit of voltage droop control (1.02 by default)
	REALAR(7)	VHIGH, higher voltage limit of voltage droop control (1.1 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	voltage droop control table is full
	IERR = 2	blank voltage droop control name is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	voltage droop control characteristic curve is invalid
IERR = 4	Voltage Droop control not found
IERR = 5	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid voltage droop control status - qmax is less than qmin - vhigh is less than vlow - vdbhigh is less than vdblow - vdblow is less than vlow

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.177. VOLTAGE_DROOP_DATA

Use this API routine to modify the voltage droop control in the working case, or to add a new voltage droop control to the working case (see PSSE Program Operation Manual.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_VOLTAGE_DROOP_DATA NAME INTGAR(1) INTGAR(2) REALAR(1)..REALAR(7)
```

Python command syntax:

```
ierr = voltage_droop_data(name, intgar, realar)
```

Fortran command syntax:

```
CALL VOLTAGE_DROOP_DATA(NAME, INTGAR, REALAR, IERR)
```

Where:

Character*40 NAME	name of the voltage droop control for which data is to be specified (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying array of 2 elements specifying(input).	
	INTGAR(1)	STATUS, voltage droop control status (0, 1) (0 by default)
	INTGAR(2)	REGBUS, regulated bus of voltage droop control (0 by default)
Real REALAR (7)	array of 7 elements specifying array of 7 elements specifying(input).	
	REALAR(1)	QMAX, upper Q limit of voltage droop control (1.0 by default)
	REALAR(2)	QDB, Q output when voltage is within deadband (0.0 by default)
	REALAR(3)	QMIN, lower Q limit of voltage droop control (-1.0 by default)
	REALAR(4)	VLOW, lower voltage limit of voltage droop control (0.9 by default)
	REALAR(5)	VDBLOW, lower voltage deadband limit of voltage droop control (0.98 by default)
	REALAR(6)	VDBHIGH, higher voltage deadband limit of voltage droop control (1.02 by default)
	REALAR(7)	VHIGH, higher voltage limit of voltage droop control (1.1 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	voltage droop control table is full
	IERR = 2	blank voltage droop control name is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	voltage droop control characteristic curve is invalid
IERR = 5	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid voltage droop control status - qmax is less than qmin - vhigh is less than vlow - vdbhigh is less than vdblow - vdblow is less than vlow

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.178. VSC_DC_CONVERTER_DATA

This API routine is obsolete. It has been replaced by the API routine vsc_dc_converter_data_3 .

Use this API to modify the data of a dc converter of an existing VSC dc line in the working case (see PSSE Program Operation Manual, Voltage Source Converter (VSC) DC Transmission Line Data).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_VSC_DC_CONVERTER_DATA  'NAME'  IC  INTGAR(1)..INTGAR(4)
 REALAR(1)..REALAR(11)
```

Python command syntax:

```
ierr = vsc_dc_converter_data(name, ic, intgar, realar)
```

Fortran command syntax:

```
CALL VSC_DC_LINE_CONVERTER_API_2(NAME, IC, INTGAR, REALAR, IERR)
```

Where:

Character*12 NAME	VSC dc line name (input; no default allowed).	
Integer IC	dc converter index (1 or 2) (input; no default allowed).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	IBUS, converter bus number (no default allowed)
	INTGAR(2)	TYPE, dc control code (1 by default)
	INTGAR(2) = 0	out-of-service
	INTGAR(2) = 1	kV
	INTGAR(2) = 2	MW
	INTGAR(3)	MODE, ac control code (1 = voltage, 2 = power factor) (1 by default)
	INTGAR(4)	REMOT, remote bus number (0 for this converter to regulate voltage at bus IBUS) (0 by default)
Real REALAR (11)	array of 11 elements specifying (input).	
	REALAR(1)	DCSET, dc setpoint (kV or MW) (1.0 by default)
	REALAR(2)	ACSET, ac setpoint (pu voltage or power factor) (1.0 by default)
	REALAR(3)	ALOSS, A loss coefficient (kW) (0.0 by default)
	REALAR(4)	BLOSS, B loss coefficient (kW/amps) (0.0 by default)
	REALAR(5)	MINLOS, minimum converter losses (kW) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(6)	SMAX, converter ac MVA rating (0.0 by default)
	REALAR(7)	IMAX, converter ac current rating (amps) (0.0 by default)
	REALAR(8)	PWF, power weighting factor fraction (1.0 by default)
	REALAR(9)	MAXQ, reactive power upper limit (Mvar) (9999.0 by default)
	REALAR(10)	MINQ, reactive power lower limit (Mvar) (-9999.0 by default)
	REALAR(11)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid converter index
	IERR = 2	no converter bus is specified
	IERR = 3	dc line not found
	IERR = 4	converter bus not found
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Converter bus is dummy bus of a multi-section line - Multi-section line grouping deleted. - Invalid dc control code - Invalid ac control mode - Regulated bus not found - Converter specified as remotely regulating itself. - Type code of regulated bus is not 1 or 2. - dc voltage setpoint is not positive - ac voltage setpoint is outside the band 0.75 to 1.25 pu - ac power factor setpoint is in error (magnitude exceeds 1.0) - Invalid power weighting factor fraction - MAXQ is less than MINQ. - RMPCT is not positive

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.179. VSC_DC_CONVERTER_DATA_3

Use this API to modify the data of a dc converter of an existing VSC dc line in the working case (see PSSE Program Operation Manual, Voltage Source Converter (VSC) DC Transmission Line Data).

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_VSC_DC_CONVERTER_DATA_3  'NAME'  IC  INTGAR(1)..INTGAR(5)
 REALAR(1)..REALAR(11)
```

Python command syntax:

```
ierr = vsc_dc_converter_data_3(name, ic, intgar, realar)
```

Fortran command syntax:

```
CALL VSC_DC_LINE_CONVERTER_API_3(NAME, IC, INTGAR, REALAR, IERR)
```

Where:

Character*12 NAME	VSC dc line name (input; no default allowed).	
Integer IC	dc converter index (1 or 2) (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	IBUS, converter bus number (no default allowed)
	INTGAR(2)	TYPE, dc control code (1 by default)
	INTGAR(2) = 0	out-of-service
	INTGAR(2) = 1	kV
	INTGAR(2) = 2	MW
	INTGAR(3)	MODE, ac control code (1 = voltage, 2 = power factor) (1 by default)
	INTGAR(4)	VSREG, regulated bus number (0 by default)
	INTGAR(5)	NODE, regulated node number (0 by default)
Real REALAR (11)	array of 11 elements specifying (input).	
	REALAR(1)	DCSET, dc setpoint (kV or MW) (1.0 by default)
	REALAR(2)	ACSET, ac setpoint (pu voltage or power factor) (1.0 by default)
	REALAR(3)	ALOSS, A loss coefficient (kW) (0.0 by default)
	REALAR(4)	BLOSS, B loss coefficient (kW/amps) (0.0 by default)
	REALAR(5)	MINLOS, minimum converter losses (kW) (0.0 by default)
	REALAR(6)	SMAX, converter ac MVA rating (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(7)	IMAX, converter ac current rating (amps) (0.0 by default)
	REALAR(8)	PWF, power weighting factor fraction (1.0 by default)
	REALAR(9)	MAXQ, reactive power upper limit (Mvar) (9999.0 by default)
	REALAR(10)	MINQ, reactive power lower limit (Mvar) (-9999.0 by default)
	REALAR(11)	RMPCT, percent of contributed reactive power (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid converter index
	IERR = 2	no converter bus is specified
	IERR = 3	dc line not found
	IERR = 4	converter bus not found
	IERR = 5	prerequisite requirements for API are not met
	IERR = 6	node-breaker terminal connection table is full
	IERR = -1	data error, warning and information messages; one or more of: - Converter bus is dummy bus of a multi-section line - Multi-section line grouping deleted. - Invalid dc control code - Invalid ac control mode - Regulated bus not found - A remote regulated bus is specified for a type 3 bus - Type code of regulated bus is not 1 or 2 - dc voltage setpoint is not positive - ac voltage setpoint is outside the band 0.75 to 1.25 pu - ac power factor setpoint is in error (magnitude exceeds 1.0) - Invalid power weighting factor fraction - MAXQ is less than MINQ. - RMPCT is not positive - Invalid node number - Non-zero regulated node specified but regulated bus does not belong to a substation - Regulated node not found - Regulated node is not connected to any bus section of the regulated bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.180. VSC_DC_LINE_CHNG

Use this API routine to modify the link data of an existing VSC dc line in the working case (see PSSE Program Operation Manual, Voltage Source Converter (VSC) DC Transmission Line Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_VSC_DC_LINE_CHNG  'NAME'  INTGAR(1)..INTGAR(5)  REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = vsc_dc_line_chng(name, intgar, realar)
```

Fortran command syntax:

```
CALL VSC_DC_LINE_CHNG(NAME, INTGAR, REALAR, IERR)
```

Where:

Character*12 NAME	VSC dc line name (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	MDC, control mode (0 or 1) (0 by default)
	INTGAR(2)	O1, first owner number (1 by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	RDC, dc line resistance (0.0 by default)
	REALAR(2)	F1, first owner fraction (1.0 by default)
	REALAR(3)	F2, second owner fraction (1.0 by default)
	REALAR(4)	F3, third owner fraction (1.0 by default)
	REALAR(5)	F4, fourth owner fraction (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 2	blank dc line name is invalid
	IERR = 3	dc line not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid dc line control mode - invalid dc line resistance - invalid owner number - invalid ownership fraction - no ownership data specified - owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.181. VSC_DC_LINE_DATA

Use this API routine to modify the link data of an existing VSC dc line in the working case, or to add a new VSC dc line to the working case (see PSSE Program Operation Manual, Voltage Source Converter (VSC) DC Transmission Line Data).

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_VSC_DC_LINE_DATA  'NAME'  INTGAR(1)..INTGAR(5)  REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = vsc_dc_line_data(name, intgar, realar)
```

Fortran command syntax:

```
CALL VSC_DC_LINE_API_2(NAME, INTGAR, REALAR, IERR)
```

Where:

Character*12 NAME	VSC dc line name (input; no default allowed).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	MDC, control mode (0 or 1) (0 by default)
	INTGAR(2)	O1, first owner number (1 by default)
	INTGAR(3)	O2, second owner number (0 by default)
	INTGAR(4)	O3, third owner number (0 by default)
	INTGAR(5)	O4, fourth owner number (0 by default)
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	RDC, dc line resistance (0.0 by default)
	REALAR(2)	F1, first owner fraction (1.0 by default)
	REALAR(3)	F2, second owner fraction (1.0 by default)
	REALAR(4)	F3, third owner fraction (1.0 by default)
	REALAR(5)	F4, fourth owner fraction (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	VSC dc line table is full
	IERR = 2	blank dc line name is invalid
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid dc line control mode - invalid dc line resistance - invalid owner number - invalid ownership fraction - no ownership data specified - owner table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

1.2.182. ZONE_DATA

Use this API to modify zone data in the working case (see PSSE Program Operation Manual, Zone Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ZONE_DATA IZONE 'ZONAME'
```

Python command syntax:

```
ierr = zone_data(izone, zoname)
```

Fortran command syntax:

```
CALL ZONE_API_2(IZONE, ZONAME, IERR)
```

Where:

Integer IZONE	zone number between 1 and 9999 (input; no default allowed).	
Character*12 ZONAME	zone name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid zone number
	IERR = 2	zone table is full
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - No equipment is assigned to zone

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 2
Node Breaker

This chapter describes the API routines used in PSS®E Node Breaker operations and data changing functions for substation data.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1. Node Breaker Operation

2.1.1. CONVERT_SECTION_TO_BUS

Use this API routine to convert a bus section, one that had been automatically created within a substation due to open substation switching devices, into a main network bus. After the conversion, the new network bus will reside in the same substation and all prior network connections will remain.
Only bus sections that are not the main parent bus can be converted.
If a plant exists at the parent bus of the bus section to be converted and if all machines are connected to the bus section, then the plant will be removed from the parent bus once the conversion of the bus section to a main network bus is made. The newly converted network bus will remain as a plant.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_CONVERT_SECTION_TO_BUS IBUS INODE IARG 'NAME'
```

Python command syntax:

```
ierr = convert_section_to_bus(ibus, inode, iarg, name)
```

Fortran command syntax:

```
CALL CONVERT_SECTION_TO_BUS(IBUS, INODE, IARG, NAME, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Integer INODE	node number (input; no default allowed).	
Integer IARG	bus number of the new network bus that is being converted from a bus section (input; no default allowed).	
Character*12 NAME	new bus name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid bus number
	IERR = 4	bus not found
	IERR = 5	invalid node number
	IERR = 6	bus does not belong to a substation
	IERR = 7	node is not found in the substation
	IERR = 8	node is not connected to any bus section of the specified bus
	IERR = 9	selected bus section is the parent bus and therefore is already a network bus
	IERR = -1	data error, warning and information messages; one or more of: - Bus name is more than 12 characters.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.2. ISOLATE2DCLINEBYBREAKER

Use this API to isolate a two-terminal DC line using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATE2DCLINEBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the two-terminal dc line. If inadequate system or substation breakers are found, such that the line is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the line.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_ISOLATE2DCLINEBYBREAKER  NAMEARG
```

Python command syntax:

```
ierr = isolate2dclinebybreaker(namearg)
```

Fortran command syntax:

```
CALL ISOLATE2DCLINEBYBREAKER(NAMEARG, IERR)
```

Where:

Character*12 NAMEARG	Two-terminal DC line identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	two-terminal dc line is not found
	IERR = 2	topology error in bus branch model
	IERR = 3	maximum levels reached and unable to isolate line
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.3. ISOLATE3WTBYBREAKER

Use this API to isolate a three-winding transformer using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATE3WTBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the three-winding transformer. If inadequate system or substation breakers are found, such that the three-winding transformer is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATE3WTBYBREAKER  IBUS JBUS KBUS CKTARG
```

Python command syntax:

```
ierr = isolate3wtbybreaker(ibus, jbus, kbus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATE3WTBYBREAKER(IBUS, JBUS, KBUS, CKTARG, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Integer KBUS	last bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	circuit identifier may not start with &
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	circuit is not found
	IERR = 5	topology error in bus branch model
	IERR = 6	maximum levels reached and unable to isolate line
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.4. ISOLATEBUSBYBREAKER

Use this API to isolate a bus using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATEBUSBYBREAKER API can search outward into the network for breakers to open when attempting to successfully isolate the bus. If inadequate system or substation breakers are found, such that the bus is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the bus.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATEBUSBYBREAKER  IBUS
```

Python command syntax:

```
ierr = isolatebusbybreaker(ibus)
```

Fortran command syntax:

```
CALL ISOLATEBUSBYBREAKER(IBUS, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	topology error in bus branch model
	IERR = 3	maximum levels reached and unable to isolate bus
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.5. ISOLATEFACTSBYBREAKER

Use this API to isolate a FACTS device using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATE-FACTSBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the FACTS device. If inadequate system or substation breakers are found, such that the device is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the FACTS device.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_ISOLATEFACTSBYBREAKER  NAMEARG
```

Python command syntax:

```
ierr = isolatefactsbybreaker(namearg)
```

Fortran command syntax:

```
CALL ISOLATEFACTSBYBREAKER(NAMEARG, IERR)
```

Where:

Character*12 NAMEARG	FACTS device name identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	FACTS device is not found
	IERR = 2	topology error in bus branch model
	IERR = 3	maximum levels reached and unable to isolate device
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.6. ISOLATEINDMACHINEBYBREAKER

Use this API to isolate an induction machine using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATE-LOADBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the induction machine. If inadequate system or substation breakers are found, such that the induction machine is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the induction machine.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_ISOLATEINDMACHINEBYBREAKER  IBUS CKTARG
```

Python command syntax:

```
ierr = isolateindmachinebybreaker(ibus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATEINDMACHINEBYBREAKER(IBUS, CKTARG, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	induction machine identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	induction machine is not found
	IERR = 3	topology error in bus branch model
	IERR = 4	maximum levels reached and unable to isolate line
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.7. ISOLATELINEBYBREAKER

Use this API to isolate a two-winding transformer or non-transformer line using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATELINEBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the two-winding transformer or non-transformer line. If inadequate system or substation breakers are found, such that the line is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the line.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATELINEBYBREAKER  IBUS JBUS CKTARG
```

Python command syntax:

```
ierr = isolatelinebybreaker(ibus, jbus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATELINEBYBREAKER(IBUS, JBUS, CKTARG, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	circuit identifier may not start with &
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	circuit is not found
	IERR = 5	topology error in bus branch model
	IERR = 6	maximum levels reached and unable to isolate line
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.8. ISOLATELOADBYBREAKER

Use this API to isolate a load using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATE-LOADBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the load. If inadequate system or substation breakers are found, such that the load is not suc-cessfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the load.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATELOADBYBREAKER  IBUS CKTARG
```

Python command syntax:

```
ierr = isolateloadbybreaker(ibus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATELOADBYBREAKER(IBUS, CKTARG, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	load identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	load is not found
	IERR = 3	topology error in bus branch model
	IERR = 4	maximum levels reached and unable to isolate line
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.9. ISOLATEMACHINEBYBREAKER

Use this API to isolate a machine using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATEMACHINEBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the machine. If inadequate system or substation breakers are found, such that the machine is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the machine.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATEMACHINEBYBREAKER  IBUS CKTARG
```

Python command syntax:

```
ierr = isolatemachinebybreaker(ibus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATEMACHINEBYBREAKER(IBUS, CKTARG, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	machine identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	machine is not found
	IERR = 3	topology error in bus branch model
	IERR = 4	maximum levels reached and unable to isolate line
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.10. ISOLATEMSLINEBYBREAKER

Use this API to isolate a multi-section line using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API used to set the number of levels the ISOLATEMSLINEBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the multi-section line. If inadequate system or substation breakers are found, such that the multi-section line is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the multi-section line.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATEMSLINEBYBREAKER  IBUS JBUS CKTARG
```

Python command syntax:

```
ierr = isolatemslinebybreaker(ibus, jbus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATEMSLINEBYBREAKER(IBUS, JBUS, CKTARG, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	circuit is not found
	IERR = 4	topology error in bus branch model
	IERR = 5	maximum levels reached and unable to isolate line
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.11. ISOLATEMTDCLINEBYBREAKER

Use this API to isolate a multi-terminal dc line using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATEMTDCLINEBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the multi-terminal dc line. If inadequate system or substation breakers are found, such that the line is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the line.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_ISOLATEMTDCLINEBYBREAKER  NAMEARG
```

Python command syntax:

```
ierr = isolatemtdclinebybreaker(namearg)
```

Fortran command syntax:

```
CALL ISOLATEMTDCLINEBYBREAKER(NAMEARG, IERR)
```

Where:

Character*12 NAMEARG	Multi-terminal dc line identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	multi-terminal dc line is not found
	IERR = 2	topology error in bus branch model
	IERR = 3	maximum levels reached and unable to isolate line
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.12. ISOLATENODEBYBREAKER

Use this API to isolate a substation node using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATENODEBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the substation node. If inadequate system or substation breakers are found, such that the node is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the node.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATENODEBYBREAKER  ISTARG IARG
```

Python command syntax:

```
ierr = isolatenodebybreaker(istarg, iarg)
```

Fortran command syntax:

```
CALL ISOLATENODEBYBREAKER(ISTARG, IARG, IERR)
```

Where:

Integer ISTARG	substation number (input).	
Integer IARG	node number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	substation in not foung
	IERR = 2	node not found in substation
	IERR = 3	topology error in bus branch model
	IERR = 4	maximum levels reached and unable to isolate line
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.13. ISOLATESHUNTBYBREAKER

Use this API to isolate a fixed shunt using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATESHUNTBYBREAKER API can search outward into the network for breakers to open when attempting to successfully isolate the fixed shunt. If inadequate system or substation breakers are found, such that the shunt is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the shunt.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATESHUNTBYBREAKER  IBUS CKTARG
```

Python command syntax:

```
ierr = isolateshuntbybreaker(ibus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATESHUNTBYBREAKER(IBUS, CKTARG, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	fixed shunt identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	fixed shunt is not found
	IERR = 3	topology error in bus branch model
	IERR = 4	maximum levels reached and unable to isolate line
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.14. ISOLATESWDBYBREAKER

Use this API to isolate a substation switching device using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATESWDBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the substation switching device. If inadequate system or substation breakers are found, such that the substation switching device is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATESWDBYBREAKER  STATION IBUS JBUS CKTARG
```

Python command syntax:

```
ierr = isolateswdbybreaker(station, ibus, jbus, cktarg)
```

Fortran command syntax:

```
CALL ISOLATESWDBYBREAKER(STATION, IBUS, JBUS, CKTARG, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	substation in not foung
	IERR = 2	node not found in substation
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	circuit is not found
	IERR = 5	topology error in bus branch model
	IERR = 6	maximum levels reached and unable to isolate line
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.15. ISOLATESWSHUNTBYBREAKER

This API routine is obsolete. It has been replaced by the API routine IsolateSwShuntByBreaker_2 .

Use this API to isolate a switched shunt using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATESWSHUNTBYBREAKER API can search outward into the network for breakers to open when attempting to successfully isolate the switched shunt. If inadequate system or substation breakers are found, such that the switched shunt is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_ISOLATESWSHUNTBYBREAKER  IBUS
```

Python command syntax:

```
ierr = isolateswshuntbybreaker(ibus)
```

Fortran command syntax:

```
CALL ISOLATESWSHUNTBYBREAKER(IBUS, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	switched shunt is not found
	IERR = 3	topology error in bus branch model
	IERR = 4	maximum levels reached and unable to isolate line
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.16. ISOLATESWSHUNTBYBREAKER_2

Use this API to isolate a switched shunt using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATESWSHUNTBYBREAKER API can search outward into the network for breakers to open when attempting to successfully isolate the switched shunt. If inadequate system or substation breakers are found, such that the switched shunt is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ISOLATESWSHUNTBYBREAKER_2  IBUS, 'ID'
```

Python command syntax:

```
ierr = isolateswshuntbybreaker_2(ibus, id)
```

Fortran command syntax:

```
CALL ISOLATESWSHUNTBYBREAKER_2(IBUS, ID, IERR)
```

Where:

Integer IBUS	from bus number (input).	
character*2 ID	Switched shunt identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	switched shunt is not found
	IERR = 3	topology error in bus branch model
	IERR = 4	maximum levels reached and unable to isolate line
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.17. ISOLATEVSCDCLINEBYBREAKER

Use this API to isolate a VSC DC line using system or substation breakers.
This API works in conjunction with the ISOLATE_LEVELS() API which sets the number of levels the ISOLATEVSCDCLINEBYBREAKER API can search outward into the network for breakers to open when attempting to isolate the VSC dc line. If inadequate system or substation breakers are found, such that the line is not successfully isolated, then specifying a larger ISOLATE_LEVELS() value may find enough additional breakers to open to achieve isolation of the line.
Use the RESTORESTATESFROMISOLATE API to restore the original network working case after an Isolate. It is recommended that you save your network case prior to running any "Isolate". Multiple "Isolate" actions can be run, and other network changes can be made, but only the original network working case prior to the initial "Isolate", or prior to the first "Isolate" after a "Restore", will be restored.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_ISOLATEVSCDCLINEBYBREAKER  NAMEARG
```

Python command syntax:

```
ierr = isolatevscdclinebybreaker(namearg)
```

Fortran command syntax:

```
CALL ISOLATEVSCDCLINEBYBREAKER(NAMEARG, IERR)
```

Where:

Character*12 NAMEARG	VSC DC line identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	vsc dc line is not found
	IERR = 2	topology error in bus branch model
	IERR = 3	maximum levels reached and unable to isolate line
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.18. RESTORESTATESFROMISOLATE

Use this API to restore the network working case back to its original state; the state prior to all "Isolate By Breaker" actions that have been taken. Caution: Any network changes that may have been made since the initial "Isolate" action will be lost.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_RESTORESTATESFROMISOLATE IERR
```

Python command syntax:

```
restorestatesfromisolate(ierr)
```

Fortran command syntax:

```
CALL RESTORESTATESFROMISOLATE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	error restoring working case
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.19. STATION_AMPOUT

Use this API to print current within a substation.

This API routine was first introduced in release 35.3.3.

Batch command syntax:

```
BAT_STATION_AMPOUT ALL STATION
```

Python command syntax:

```
ierr = station_ampout(all, station)
```

Fortran command syntax:

```
CALL STATION_AMPOUT(ALL, STATION, IERR)
```

Where:

Integer ALL	(input).	
	ALL = 1	report the power flows of all substations
	ALL = 2	report the power flows of substation STATION
Integer STATION	substation number (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid ALL value
	IERR = 2	invalid STATION number
	IERR = 3	STATION does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.20. STATION_BUILD_CONFIG

Use this API to automatically build a general layout configuration for a bus being built within a substation. Once the substation is built, the layout can be further modified within the substation diagram and substation editor in order to match specific configurations.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_BUILD_CONFIG IBUS ISTARG NAMEAR SUBTYP
```

Python command syntax:

```
ierr = station_build_config(ibus, istarg, namear, subtyp)
```

Fortran command syntax:

```
CALL BUILDBUSCONFIGURATION(IBUS, ISTARG, NAMEAR, SUBTYP, IERR)
```

Where:

Integer IBUS	bus being built within the substation (input).	
Integer ISTARG	station number used to uniquely identify this substation (input).	
Character*40 NAMEAR	station name used to uniquely identify this substation (input).	
Integer SUBTYP	basic layout configuration to build for the specified bus (input; 1 by default).	
	SUBTYP = 1	single bus
	SUBTYP = 2	ring bus
	SUBTYP = 3	double breaker, double bus
	SUBTYP = 4	breaker and a half bus
	SUBTYP = 5	single breaker, double bus
	SUBTYP = 6	main transfer bus
	SUBTYP = 7	breaker and a half bus, with odd/even nodes in columns
	SUBTYP = 8	single breaker, double bus, with odd/even nodes in columns
Integer IERR	error code (output).	
	IERR = 1	bus does not exist
	IERR = 2	invalid substation number
	IERR = 3	substation model already exists for bus
	IERR = 4	nodes to be added in the substation greater than maximum
	IERR = 5	total nodes in the substation greater than maximum
	IERR = 6	error building the substation
	IERR = 7	error building the nodes and switching devices

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.21. STATION_LIST

Use this API to list components in a substation.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_LIST OPTION STATION
```

Python command syntax:

```
ierr = station_list(option, station)
```

Fortran command syntax:

```
CALL SLIST(OPTION, STATION, IERR)
```

Where:

Integer OPTION	(input).	
	OPTION = 1	list all substations
	OPTION = 2	list substation STATION
Integer STATION	substation number (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid OPTION value
	IERR = 2	invalid STATION number
	IERR = 3	STATION does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.22. STATION_POUT

This API routine is obsolete. It has been replaced by the API routine station_pout_2 .

Use this API to print power flows in a substation.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_POUT OPTION STATION
```

Python command syntax:

```
ierr = station_pout(option, station)
```

Fortran command syntax:

```
CALL SPOUT(OPTION, STATION, IERR)
```

Where:

Integer OPTION	(input).	
	OPTION = 1	report the power flows of all substations
	OPTION = 2	report the power flows of substation STATION
Integer STATION	substation number (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid OPTION value
	IERR = 2	STATION does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.23. STATION_POUT_2

Use this API to print power flows within a substation.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_STATION_POUT_2 ALL STATION OPTION
```

Python command syntax:

```
ierr = station_pout_2(all, station, option)
```

Fortran command syntax:

```
CALL SPOUT_2(ALL, STATION, OPTION, IERR)
```

Where:

Integer ALL	(input).	
	ALL = 1	report the power flows of all substations
	ALL = 2	report the power flows of substation STATION
Integer STATION	substation number (input).	
Integer OPTION	(input).	
	OPTION = 0	Do not display loop flow values
	OPTION = 1	Show all flow values
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid ALL value
	IERR = 2	invalid STATION number
	IERR = 3	STATION does not exist
	IERR = 4	invalid OPTION value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.1.24. STATION_TREE

Use this API routine to check the node breaker model and the consistency between node breaker and bus branch models.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_TREE INTOPT STATION
```

Python command syntax:

```
ierr = station_tree(intopt, station)
```

Fortran command syntax:

```
CALL STREE(INTOPT, STATION, IERR)
```

Where:

Integer INTOPT	operation mode (input; 1 by default).	
	INTOPT = 0	check all changed substations
	INTOPT = 1	check all substations
	INTOPT = 2	check individual substation
Integer STATION	substation number. Used if INTOPT = 2. (input).	
Integer IERR	error code (output).	
	IERR = 1	invalid INTOPT value
	IERR = 2	invalid substation number
	IERR = 3	substation does not exist
	IERR = 4	topology error in bus branch model
	IERR = 5	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

This chapter describes the API routines used in PSS®E Node Breaker operations.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2. Node Breaker Data

2.2.1. FETCH_ADJVAR_TERM_NODE

Use this API to obtain the terminal node to which an OPF adjustable bus shunt is connected to.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Python command syntax:

```
ierr,intgar = fetch_adjvar_term_node(ibus, cktarg)
```

Fortran command syntax:

```
CALL FETCH_ADJVAR_TERM_NODE(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	OPF adjustable var identifier (input).	
Integer INTGAR	node terminal number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	OPF adjustable bus shunt identifier is more than two characters
	IERR = 3	OPF adjustable bus shunt not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.2. FETCH_INDMACHINE_TERM_NODE

Use this API to obtain the terminal node to which a machine is connected to.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Python command syntax:

```
ierr,intgar = fetch_indmachine_term_node(ibus, cktarg)
```

Fortran command syntax:

```
CALL FETCH_INDMACHINE_TERM_NODE(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	induction machine identifier (input).	
Integer INTGAR	node terminal number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.3. FETCH_LOAD_TERM_NODE

Use this API to obtain the terminal node to which a load is connected to.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Python command syntax:

```
ierr,intgar = fetch_load_term_node(ibus, cktarg)
```

Fortran command syntax:

```
CALL FETCH_LOAD_TERM_NODE(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	load identifier (input).	
Integer INTGAR	node terminal number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	load identifier is more than two characters
	IERR = 3	load not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.4. FETCH_MACHINE_TERM_NODE

Use this API to obtain the terminal node to which a machine is connected to.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Python command syntax:

```
ierr,intgar = fetch_machine_term_node(ibus, cktarg)
```

Fortran command syntax:

```
CALL FETCH_MACHINE_TERM_NODE(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	machine identifier (input).	
Integer INTGAR	node terminal number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.5. FETCH_SHUNT_TERM_NODE

Use this API to obtain the terminal node to which a shunt is connected to.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Python command syntax:

```
ierr,intgar = fetch_shunt_term_node(ibus, cktarg)
```

Fortran command syntax:

```
CALL FETCH_SHUNT_TERM_NODE(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	shunt identifier (input).	
Integer INTGAR	node terminal number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	shunt identifier is more than two characters
	IERR = 3	shunt not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.6. FETCH_SWS_SHUNT_TERM_NODE

This API routine is obsolete. It has been replaced by the API routine fetch_sws_shunt_term_node_2 .

Use this API to obtain the terminal node to which a switched shunt is connected to.

This API routine was first introduced in release 34.8.0.

Python command syntax:

```
ierr,intgar = fetch_sws_shunt_term_node(ibus)
```

Fortran command syntax:

```
CALL FETCH_SWS_SHUNT_TERM_NODE(IBUS, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer INTGAR	node terminal number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	switched shunt not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.7. FETCH_SWS_SHUNT_TERM_NODE_2

Use this API to obtain the terminal node to which a switched shunt is connected to.

This API routine was first introduced in release 35.2.

Python command syntax:

```
ierr,intgar = fetch_sws_shunt_term_node_2(ibus, cktarg)
```

Fortran command syntax:

```
CALL FETCH_SWS_SHUNT_TERM_NODE_2(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	switched shunt identifier (input).	
Integer INTGAR	node terminal number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	switched shunt identifier is more than two characters
	IERR = 3	switched shunt not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.8. PLANT_BUS_SECTION_CHNG

This API routine is obsolete. It has been replaced by the API routine plant_chng_3 .

Use this API routine to modify the data of an existing plant in the working case (see PSSE Data Formats Reference Manual, Generator Data).

This API routine was first introduced in release 34.0.1.

Batch command syntax:

```
BAT_PLANT_BUS_SECTION_CHNG  I NODE INTGAR REALAR(1)..REALAR(2)
```

Python command syntax:

```
ierr = plant_bus_section_chng(i, node, intgar, realar)
```

Fortran command syntax:

```
CALL PLANT_BUS_SECTION_CHNG(I, NODE, INTGAR, REALAR, IERR)
```

Where:

Integer I	bus number (input; no default allowed).	
Integer NODE	node number (input; no default allowed).	
Integer INTGAR (1)	one element. See the PLANT_DATA_3 API routine for details (input).	
Real REALAR (2)	See the PLANT_DATA_3 API routine for details (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 3	no plant at bus
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid scheduled voltage - regulated bus not found - plant specified as remotely regulating itself - type code of regulated bus is not 1 or 2 - RMPCT is not positive

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.9. STATION_2DC_BUS_TERM_CHNG

Use this API to change the station two-term dc rectifier or inverter node terminal connection in the working case. This api is defined using the rectifier or inverter end bus number and the desired node number to connect to that end.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
STATION_2DC_BUS_TERM_CHNG NAME IBUS INODE
```

Python command syntax:

```
ierr = station_2dc_bus_term_chng(name, ibus, inode)
```

Fortran command syntax:

```
CALL STATION_2DC_BUS_TERM_CHNG(NAME, IBUS, INODE, IERR)
```

Where:

character*12 NAME	quoted name of the two-terminal dc line whose node terminal connection data is being changed (input; no default allowed).	
Integer IBUS	bus number of desired dc rectifier or inverter end being modified (input; no default allowed).	
Integer INODE	node number to which the dc rectifier or inverter end is to be attached. Node must correspond to IBUS (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	two-terminal dc line not found
	IERR = 2	bus is not part of a substation
	IERR = 3	node not found in substation
	IERR = 4	Invalid node number
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	bus not found
	IERR = 8	bus not a converter bus of dc line

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.10. STATION_2DC_CONV_TERM_CHNG

This API changes both node terminal connections of an existing two-term dc rectifier and inverter terminal data in the working case.
A new API, station_2dc_bus_term_chng, has been introduced at 35.5 in which the node terminal connection at one end is changed by specifying the bus and desired node.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_STATION_2DC_CONV_TERM_CHNG  NAME NODES(1) NODES(2)
```

Python command syntax:

```
ierr = station_2dc_conv_term_chng(name, nodes)
```

Fortran command syntax:

```
CALL ST_NTWOTERMDC(NAME, NODES, IERR)
```

Where:

character*12 NAME	quoted name of the two-terminal dc line for which node terminal data for one or both of its converters is to be specified (input; no default allowed).	
Integer NODES (2)	array of elements specifying (input).	
	NODES(1)	node number to which the dc rectifier end is attached
	NODES(2)	node number to which the dc inverter end is attached
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	two-terminal dc line not found
	IERR = 2	bus associated with node is not part of a substation
	IERR = 3	node does not exist
	IERR = 4	Invalid node number
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.11. STATION_ADJVAR_TERM_CHNG

Use this API to change a substation OPF adjustable bus shunt node terminal connection in the working case.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_STATION_ADJVAR_TERM_CHNG  BUS CKTARG INTGAR
```

Python command syntax:

```
ierr = station_adjvar_term_chng(bus, cktarg, intgar)
```

Fortran command syntax:

```
CALL ST_NADJVAR(BUS, CKTARG, INTGAR, IERR)
```

Where:

Integer BUS	from bus number (input).	
Character*2 CKTARG	adjustable bus shunt identifier (input).	
Integer INTGAR	node terminal number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	OPF adjustable bus shunt identifier is more than two characters
	IERR = 3	OPF adjustable bus shunt is not found
	IERR = 4	bus does not belong to a substation
	IERR = 5	node not found in substation
	IERR = 6	invalid node number
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	error when moving OPF adjustable bus shunt terminal connection
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.12. STATION_BRANCH_TERM_CHNG

Use this API to access existing station branch terminal data in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_BRANCH_TERM_CHNG  IBUS JBUS CKTARG NODES(1) NODES(2)
```

Python command syntax:

```
ierr = station_branch_term_chng(ibus, jbus, cktarg, nodes)
```

Fortran command syntax:

```
CALL ST_NBRANCH(IBUS, JBUS, CKTARG, NODES, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer NODES (2)	array of 2 elements specifying (input).	
	NODES(1)	from node number
	NODES(2)	to node number
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	bus does not belong to a substation
	IERR = 3	node does not exist in substation
	IERR = 4	invalid node number
	IERR = 5	circuit identifier is more than two characters
	IERR = 6	circuit identifier may not start with &
	IERR = 7	circuit not found
	IERR = 8	error when moving branch terminal connection
	IERR = 9	node-breaker terminal connection table is full
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.13. STATION_BRANCH_TERM_MOVE

Use this API to move one terminal node connection of a branch in the working case. The second bus argument (JBUS) is the bus at which the bus at which the node connection will be moved to the node designated in the DSNODE argument.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_BRANCH_TERM_MOVE  IBUS JBUS CKTARG DSNODE
```

Python command syntax:

```
ierr = station_branch_term_move(ibus, jbus, cktarg, dsnode)
```

Fortran command syntax:

```
CALL MOVEBRANCHTERMINAL(IBUS, JBUS, CKTARG, DSNODE, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number. This is the bus for which the node connection will be moved to the node designated in DSNODE. (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer DSNODE	new terminal node number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	bus does not belong to a substation
	IERR = 3	node not found in substation
	IERR = 4	invalid node number
	IERR = 5	circuit identifier is more than two characters
	IERR = 6	circuit identifier may not start with &
	IERR = 7	circuit not found
	IERR = 8	error when moving branch terminal connection
	IERR = 9	node-breaker terminal connection table is full
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.14. STATION_BUS_PURG

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_STATION_BUS_PURG  STATION IBUS
```

Python command syntax:

```
ierr = station_bus_purg(station,ibus)
```

Fortran command syntax:

```
CALL PURG_STATION_BUS(STATION, IBUS, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer IBUS	bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid STATION number
	IERR = 2	STATION does not exist
	IERR = 3	BUS does not exist
	IERR = 4	unable to move all connections back to original network bus
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.15. STATION_CHNG

Use this API to modify the data of an existing substation data in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_CHNG  STATION REALAR(1)..REALAR(3) NAMEAR
```

Python command syntax:

```
ierr = station_chng(station, realar, namear)
```

Fortran command syntax:

```
CALL STATION_CHNG(STATION, REALAR, NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input; no default allowed).	
Real REALAR (3)	array of elements specifying (input).	
	REALAR(1)	substation latitude
	REALAR(2)	substation longitude
	REALAR(3)	substation grounding resistance
Character*40 NAMEAR	substation name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid substation number
	IERR = 2	substation not found
	IERR = 3	substation name has been used
	IERR = 4	substation table is full
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Latitude/longitude out of range - Substation grounding dc resistance is less than 0.0 - Substation name is more than STATIONNAMLEN characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.16. STATION_DATA

Use this API to modify the data of an existing substation data in the working case, or to add a new substation to the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_DATA  STATION REALAR(1)..REALAR(3) NAMEAR
```

Python command syntax:

```
ierr = station_data(station, realar, namear)
```

Fortran command syntax:

```
CALL STATION_API(STATION, REALAR, NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input; no default allowed).	
Real REALAR (3)	array of elements specifying (input).	
	REALAR(1)	substation latitude
	REALAR(2)	substation longitude
	REALAR(3)	substation grounding resistance
Character*40 NAMEAR	substation name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid substation number
	IERR = 2	substation not found
	IERR = 3	substation name has been used
	IERR = 4	substation table is full
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Latitude/longitude out of range - Substation grounding dc resistance is less than 0.0 - Substation name is more than STATIONNAMLEN characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.17. STATION_FACTS_BUS_TERM_CHNG

Use this API to change the station FACTS device node terminal data connection in the working case. This api is defined using the sending or terminal bus number and specifying the desired node number to connect to at that end.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_STATION_FACTS_BUS_TERM_CHNG  NAME IBUS INODE
```

Python command syntax:

```
ierr = station_facts_bus_term_chng(name, ibus, inode)
```

Fortran command syntax:

```
CALL STATION_FACTS_BUS_TERM_CHNG(NAME, IBUS, INODE, IERR)
```

Where:

character*12 NAME	quoted name of the FACTS device whose node terminal connection data is being changed (input; no default allowed).	
Integer IBUS	bus number of desired FACTS device sending or terminal end being modified (input; no default allowed).	
Integer INODE	node number to which the FACTS device sending or terminal end is to be attached. Node must correspond to IBUS (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	FACTS device not found
	IERR = 2	bus is not part of a substation
	IERR = 3	node does not exist
	IERR = 4	Invalid node number
	IERR = 5	Node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	bus not found
	IERR = 8	bus not associated with FACTS device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.18. STATION_FACTS_TERM_CHNG

This API changes the FACTS device sending and terminal end node terminal connections.
A new API, station_facts_bus_term_chng, has been introduced at 35.5 in which the node terminal connection at one end is changed by specifying the bus and desired node.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_STATION_FACTS_TERM_CHNG  NAME INTGAR(1) INTGAR(2)
```

Python command syntax:

```
ierr = station_facts_term_chng(name, intgar)
```

Fortran command syntax:

```
CALL ST_NFACTS(NAME, INTGAR, IERR)
```

Where:

character*12 NAME	quoted name of the FACTS device for which node terminal data for the sending and terminal end is to be changed (input; no default allowed).	
Integer INTGAR (2)	array of elements specifying (input).	
	INTGAR(1)	node number to which the FACTS device sending end is attached
	INTGAR(2)	node number to which the FACTS device terminal end is attached
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	FACTS device not found
	IERR = 2	bus associated with node does not belong to substation
	IERR = 3	node does not exist
	IERR = 4	Invalid node number
	IERR = 5	Node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.19. STATION_INDMACHINE_TERM_CHNG

Use this API to change a substation induction machine node terminal connection in the working case.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_STATION_INDMACHINE_TERM_CHNG  IBUS CKTARG INTGAR
```

Python command syntax:

```
ierr = station_indmachine_term_chng(ibus, cktarg, intgar)
```

Fortran command syntax:

```
CALL ST_NINDMAC(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	induction machine identifier (input).	
Integer INTGAR	node terminal number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	induction machine identifier is more than two characters
	IERR = 3	induction machine not found
	IERR = 4	bus does not belong to a substation
	IERR = 5	node does not exist in substation
	IERR = 6	invalid node number
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	error when moving induction machine terminal connection
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.20. STATION_LOAD_TERM_CHNG

Use this API to access existing station load terminal data in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_LOAD_TERM_CHNG  IBUS CKTARG INTGAR
```

Python command syntax:

```
ierr = station_load_term_chng(ibus, cktarg, intgar)
```

Fortran command syntax:

```
CALL ST_NLOAD(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	load identifier (input).	
Integer INTGAR	node terminal number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	load identifier is more than two characters
	IERR = 3	load is not found
	IERR = 4	bus does not belong to a substation
	IERR = 5	node does not exist in substation
	IERR = 6	invalid node number
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	error when moving load terminal connection
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.21. STATION_MACHINE_TERM_CHNG

Use this API to change a substation machine terminal connection in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_MACHINE_TERM_CHNG  IBUS CKTARG INTGAR
```

Python command syntax:

```
ierr = station_machine_term_chng(ibus, cktarg, intgar)
```

Fortran command syntax:

```
CALL ST_NMACHINE(IBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Character*2 CKTARG	machine identifier (input).	
Integer INTGAR	node terminal number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	bus does not belong to a substation
	IERR = 5	node does not exist in substation
	IERR = 6	Invalid node number
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	error when moving machine terminal connection
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.22. STATION_MTDC_BUS_TERM_CHNG

Use this API to change the station multi-terminal converter node terminal connection in the working case. This api is defined using the converter bus number and desired node number to connect to at that end.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_STATION_MTDC_BUS_TERM_CHNG  NAME IBUS INODE
```

Python command syntax:

```
ierr = station_mtdc_bus_term_chng(name, ibus, inode)
```

Fortran command syntax:

```
CALL STATION_MTDC_BUS_TERM_CHNG(NAME, IBUS, INODE, IERR)
```

Where:

character*12 NAME	quoted name of the multi-terminal dc line whose node terminal connection data is being changed (input; no default allowed).	
Integer IBUS	bus number of desired converter end being modified (input; no default allowed).	
Integer INODE	node number to which the converter end is to be attached. Node must correspond to IBUS. (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	multi-terminal dc line not found
	IERR = 2	bus is not part of a substation
	IERR = 3	node does not exist
	IERR = 4	Invalid node number
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	bus not found
	IERR = 7	bus is not a converter bus of this multi-term dc line
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.23. STATION_MTDC_TERM_CHNG

This API changes both node terminal connections of an existing multi-terminal converter in the working case.
A new API, station_mtdc_bus_term_chng, has been introduced at 35.5 in which the node terminal connection at one end is changed by specifying the converter bus number and desired node.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_STATION_MTDC_TERM_CHNG  NAME INTGAR(1) INTGAR(2)
```

Python command syntax:

```
ierr = station_mtdc_term_chng(name, intgar)
```

Fortran command syntax:

```
CALL ST_NMTDC(NAME, INTGAR, IERR)
```

Where:

character*12 NAME	quoted name of the multi-terminal dc line for which node terminal data is to be specified for one of the ac converter buses (input; no default allowed).	
Integer INTGAR (2)	array of elements specifying (input).	
	INTGAR(1)	converter bus to which the node is attached
	INTGAR(2)	node number to which the converter end is attached
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	multi-terminal dc line not found
	IERR = 2	bus does not belong to substation
	IERR = 3	node does not exist
	IERR = 4	Invalid node number
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	bus not found
	IERR = 7	bus is not a converter bus of this multi-term dc line
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.24. STATION_NODE_CHNG

Use this API to modify the data of an existing substation node in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_NODE_CHNG  STATION NODE INTGAR(1) INTGAR(2) NAMEAR
```

Python command syntax:

```
ierr = station_node_chng(station, node, intgar, namear)
```

Fortran command syntax:

```
CALL NODE_CHNG(STATION, NODE, INTGAR, NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input; no default allowed).	
Integer NODE	node number (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying array of two integer values(input).	
	INTGAR(1)	bus number to which the node is connected
	INTGAR(2)	node status
	INTGAR(2) = 0	out-of-service
	INTGAR(2) = 1	in-service
Character*40 NAMEAR	node name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = -1	data error, warning and information messages
	IERR = 0	no errors
	IERR = 1	invalid substation number
	IERR = 2	substation not found
	IERR = 3	invalid node number
	IERR = 4	node not found
	IERR = 5	no bus associated with this node
	IERR = 6	node number already in use
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.25. STATION_NODE_DATA

Use this API to modify the data of an existing substation node in the working case, or to add a new substation node to the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_NODE_DATA  STATION NODE INTGAR(1) INTGAR(2) NAMEAR
```

Python command syntax:

```
ierr = station_node_data(station, node, intgar, namear)
```

Fortran command syntax:

```
CALL NODE_API(STATION, NODE, INTGAR, NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input; no default allowed).
Integer NODE	node number (input; no default allowed).
Integer INTGAR (2)	array of 2 elements specifying Array of two integer values(input). INTGAR(1) bus number to which the node is connected INTGAR(2) node status INTGAR(2) = 0 out-of-service INTGAR(2) = 1 in-service
Character*40 NAMEAR	Node name (input; blank by default).
Integer IERR	error code (output). IERR = -1 data error, warning and information messages IERR = 0 no errors IERR = 1 invalid substation number IERR = 2 substation not found IERR = 3 invalid node number IERR = 4 node not found IERR = 5 no bus associated with this node IERR = 6 node number already in use IERR = 7 node table is full IERR = 8 node-breaker terminal connection table is full IERR = 9 prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.26. STATION_NODE_NUMBER

Use this API to change a node number in an existing substation.

This API routine was first introduced in release 34.0.1.

Batch command syntax:

```
BAT_STATION_NODE_NUMBER  STATION NODE NEWNODE
```

Python command syntax:

```
ierr = station_node_number(station, node, newnode)
```

Fortran command syntax:

```
CALL NODE_NUMBER(STATION, NODE, NEWNODE, IERR)
```

Where:

Integer STATION	substation number (input; no default allowed).	
Integer NODE	number of the node to be changed (input; no default allowed).	
Integer NEWNODE	new number to be assigned to the node (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 1	invalid substation number.
	IERR = 2	substation not found.
	IERR = 3	invalid node number.
	IERR = 4	node not found.
	IERR = 5	invalid new node number.
	IERR = 6	node number is already assigned to another node in the substation.
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.27. STATION_NODE_PURG

Use this API to delete a node in an existing substation.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_NODE_PURG  STATION NODE
```

Python command syntax:

```
ierr = station_node_purg(station, node)
```

Fortran command syntax:

```
CALL PURG_NODE(STATION, NODE, IERR)
```

Where:

Integer STATION	substation number (input; no default allowed).	
Integer NODE	node number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	working case is empty
	IERR = 2	invalid substation number
	IERR = 3	substation not found
	IERR = 4	invalid node number
	IERR = 5	node not found
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.28. STATION_NUMBER

Use this API to change a substation number for an existing substation.

This API routine was first introduced in release 34.0.1.

Batch command syntax:

```
BAT_STATION_NUMBER  STATION NEWSTATION
```

Python command syntax:

```
ierr = station_number(station, newstation)
```

Fortran command syntax:

```
CALL STATION_NUMBER(STATION, NEWSTATION, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer NEWSTATION	new substation number (input).	
Integer IERR	error code (output).	
	IERR = 1	invalid substation number.
	IERR = 2	substation not found.
	IERR = 3	invalid new substation number.
	IERR = 4	substation number is already assigned to another substation.
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.29. STATION_PURG

Use this API to delete a substation and all node-breaker components within that substation from the working case. All network elements will remain in the working case and any bus sections will be merged back into the original bus.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_PURG  STATION
```

Python command syntax:

```
ierr = station_purg(station)
```

Fortran command syntax:

```
CALL PURG_STATION(STATION, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	invalid substation number
	IERR = 2	substation not found
	IERR = 3	unable to move all connections back to original network bus
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.30. STATION_SHUNT_TERM_CHNG

Use this API to change a substation fixed shunt terminal connection in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_SHUNT_TERM_CHNG  BUS CKTARG INTGAR
```

Python command syntax:

```
ierr = station_shunt_term_chng(bus, cktarg, intgar)
```

Fortran command syntax:

```
CALL ST_NSHUNT(BUS, CKTARG, INTGAR, IERR)
```

Where:

Integer BUS	from bus number (input).	
Character*2 CKTARG	fixed bus shunt identifier (input).	
Integer INTGAR	node terminal number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	fixed shunt identifier is more than two characters
	IERR = 3	fixed shunt is not found
	IERR = 4	bus does not have substation model
	IERR = 5	node does not exist in substation
	IERR = 6	invalid node number
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	error when moving fixed shunt terminal connection
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.31. STATION_SWD_CHNG

This API routine is obsolete. It has been replaced by the API routine station_swd_chng_2 .

Use this API to modify the data of an existing station switching device in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_SWD_CHNG STATION INODE JNODE CKTARG INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(4) NAMEAR
```

Python command syntax:

```
ierr = station_swd_chng(station, inode, jnode, cktarg, intgar, realar,
 namear)
```

Fortran command syntax:

```
CALL STATION_SWD_CHNG(STATION, INODE, JNODE, CKTARG, INTGAR, REALAR,
 NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer INODE	from node number (input).	
Integer JNODE	to node number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	device status
	INTGAR(2)	device normal status
	INTGAR(3)	device type
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	device reactance
	REALAR(2)	RateA rating
	REALAR(3)	RateB rating
	REALAR(4)	RateC rating
Character*40 NAMEAR	substation switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	substation is not found
	IERR = 2	node is not found
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	switching device from node to itself
	IERR = 5	circuit identifier is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	switching device is not found
IERR = 7	cannot place substation switching device between nodes residing in different substations
IERR = 8	cannot place substation switching device between nodes belonging to two different buses in the same substation
IERR = 9	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0. - switching device rating is less than 0.0. - invalid status. - switching device name truncated to 40 characters. - switching device name contains invalid characters. - switching device name already assigned to another switching device in substation.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.32. STATION_SWD_CHNG_2

Use this API to modify the data of an existing station switching device in the working case.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STATION_SWD_CHNG_2  STATION INODE JNODE CKTARG INTGAR(1)..INTGAR(3)
 REALAR(1) RSETNAM NAMEAR
```

Python command syntax:

```
ierr = station_swd_chng_2(station, inode, jnode, cktarg, intgar, realar,
 rsetnam, namear)
```

Fortran command syntax:

```
CALL STATION_SWD_CHNG_2(STATION, INODE, JNODE, CKTARG, INTGAR, REALAR,
 RSETNAM, NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer INODE	from node number (input).	
Integer JNODE	to node number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	device status (alias is stat)
	INTGAR(2)	device normal status (alias is nstat)
	INTGAR(3)	device type (alias is type)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	device reactance (alias is xpu) (0.0001 by default)
Character*40 RSETNAM	switching device rating set name (input).	
Character*40 NAMEAR	substation switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	substation is not found
	IERR = 2	node is not found
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	switching device from node to itself
	IERR = 5	circuit identifier is invalid
	IERR = 6	switching device is not found
	IERR = 7	cannot place substation switching device between nodes residing in different substations

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	cannot place substation switching device between nodes belonging to two different buses in the same substation
IERR = 9	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0. - switching device rating is less than 0.0. - invalid status. - switching device name truncated to 40 characters. - switching device name contains invalid characters. - switching device name already assigned to another switching device in substation. - rating set name has too many characters. - rating set name is set to blank. - rating set name is not in the SWD rating table.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.33. STATION_SWD_DATA

This API routine is obsolete. It has been replaced by the API routine station_swd_data_2 .

Use this API to modify the data of an existing substation switching device in the working case, or to add a new substation switching device to the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_SWD_DATA STATION INODE JNODE CKTARG INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(4) NAMEAR
```

Python command syntax:

```
ierr = station_swd_data(station, inode, jnode, cktarg, intgar, realar,
 namear)
```

Fortran command syntax:

```
CALL STATION_SWD_DATA(STATION, INODE, JNODE, CKTARG, INTGAR, REALAR,
 NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer INODE	from node number (input).	
Integer JNODE	to node number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	device status
	INTGAR(2)	device normal status
	INTGAR(3)	device type
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	device reactance
	REALAR(2)	RateA rating
	REALAR(3)	RateB rating
	REALAR(4)	RateC rating
Character*40 NAMEAR	substation switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	substation is not found
	IERR = 2	node is not found
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	switching device from node to itself

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	circuit identifier is invalid
IERR = 6	switching device already exists
IERR = 7	cannot place substation switching device between nodes residing in different substations
IERR = 8	cannot place substation switching device between nodes belonging to two different buses in the same substation
IERR = 9	station switching device table is full
IERR = 10	system switching device table is full
IERR = 11	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0. - switching device rating is less than 0.0. - invalid status. - switching device name truncated to 40 characters. - switching device name contains invalid characters. - switching device name already assigned to another switching device in substation.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.34. STATION_SWD_DATA_2

Use this API to modify the data of an existing substation switching device in the working case, or to add a new substation switching device to the working case.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STATION_SWD_DATA_2 STATION INODE JNODE CKTARG INTGAR(1)..INTGAR(3)
 REALAR(1) RSETNAM NAMEAR
```

Python command syntax:

```
ierr = station_swd_data_2(station, inode, jnode, cktarg, intgar, realar,
 rsetnam, namear)
```

Fortran command syntax:

```
CALL STATION_SWD_DATA_2(STATION, INODE, JNODE, CKTARG, INTGAR, REALAR,
 RSETNAM, NAMEAR, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer INODE	from node number (input).	
Integer JNODE	to node number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	device status: 0=open, 1=closed, 2=stuck closed (alias is stat)
	INTGAR(2)	device normal status: 0=open, 1=closed, 2=stuck closed (alias is nstat)
	INTGAR(3)	device type (alias is type)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	device reactance (alias is xpu) (0.0001 by default)
Character*40 RSETNAM	switching device rating set name (input).	
Character*40 NAMEAR	substation switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	substation is not found
	IERR = 2	node is not found
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	switching device from node to itself
	IERR = 5	circuit identifier is invalid
	IERR = 6	switching device already exists

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	cannot place substation switching device between nodes residing in different substations
IERR = 8	cannot place substation switching device between nodes belonging to two different buses in the same substation
IERR = 9	station switching device table is full
IERR = 10	system switching device table is full
IERR = 11	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0. - switching device rating is less than 0.0. - invalid status. - switching device name truncated to 40 characters. - switching device name contains invalid characters. - switching device name already assigned to another switching device in substation. - rating set name has too many characters. - rating set name is set to blank. - rating set name is not in the SWD rating table.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.35. STATION_SWD_MBID

Use this API to modify the circuit identifier of an existing station switching device in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_SWD_MBID  STATION INODE JNODE CKTARG CKTNEW
```

Python command syntax:

```
ierr = station_swd_mbid(station, inode, jnode, cktarg, cktnew)
```

Fortran command syntax:

```
CALL STATION_MSWDID(STATION, INODE, JNODE, CKTARG, CKTNEW, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer INODE	from node number (input).	
Integer JNODE	to node number (input).	
Character*2 CKTARG	circuit identifier (input).	
Character*2 CKTNEW	new circuit identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	substation is not found
	IERR = 2	node is not found
	IERR = 3	circuit identifier is more than two characters
	IERR = 4	switching device from node to itself
	IERR = 5	circuit identifier is invalid
	IERR = 6	switching device is not found
	IERR = 7	circuit identifier connecting these nodes already exists
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.36. STATION_SWD_MOVE

Use this API to move the "to" node of an existing substation switching device in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_SWD_MOVE  STATION INODE JNODE CKTARG KNODE CKTNEW
```

Python command syntax:

```
ierr = station_swd_move(station, inode, jnode, cktarg, knode, cktnew)
```

Fortran command syntax:

```
CALL MOVE_STATION_SWD(STATION, INODE, JNODE, CKTARG, KNODE, CKTNEW, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer INODE	from node number (input).	
Integer JNODE	to node number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer KNODE	new to node number (input).	
Character*2 CKTNEW	new circuit identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	substation is not found
	IERR = 2	node not found in substation
	IERR = 3	switching device not found in substation
	IERR = 4	switching device from node to itself
	IERR = 5	cannot place substation switching device between nodes belonging to two different buses in the same substation
	IERR = 6	switching device already exists
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.37. STATION_SWD_PURG

Use this API to delete an existing substation switching device in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_SWD_PURG  STATION INODE JNODE CKTARG
```

Python command syntax:

```
ierr = station_swd_purg(station, inode, jnode, cktarg)
```

Fortran command syntax:

```
CALL PURG_STATION_SWD(STATION, INODE, JNODE, CKTARG, IERR)
```

Where:

Integer STATION	substation number (input).	
Integer INODE	from node number (input).	
Integer JNODE	to node number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	working case is empty
	IERR = 2	invalid substation number
	IERR = 3	substation is not found
	IERR = 4	invalid node number
	IERR = 5	node number is not found in substation
	IERR = 6	switching device is not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.38. STATION_SWS_SHUNT_TERM_CHNG

This API routine is obsolete. It has been replaced by the API routine station_sws_shunt_term_chng_2 .

Use this API to change a substation switched shunt node terminal connection in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_SWS_SHUNT_TERM_CHNG  BUS INTGAR
```

Python command syntax:

```
ierr = station_sws_shunt_term_chng(bus, intgar)
```

Fortran command syntax:

```
CALL ST_NSWSHUNT(BUS, INTGAR, IERR)
```

Where:

Integer BUS	(input).	
Integer INTGAR	(input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	switched shunt does not exist
	IERR = 3	bus does not belong to a substation
	IERR = 4	node not found in substation
	IERR = 5	invalid node number
	IERR = 6	error when moving switched shunt terminal connection
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.39. STATION_SWS_SHUNT_TERM_CHNG_2

Use this API to change a substation switched shunt node terminal connection in the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_STATION_SWS_SHUNT_TERM_CHNG_2  BUS ID INTGAR
```

Python command syntax:

```
ierr = station_sws_shunt_term_chng_2(bus, id, intgar)
```

Fortran command syntax:

```
CALL ST_NSWSHUNT_2(BUS, ID, INTGAR, IERR)
```

Where:

Integer BUS	Number of the bus to which the switched shunt is connected (input).	
character*2 ID	switched shunt identifier (input).	
Integer INTGAR	Terminal node number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt identifier is more than two characters
	IERR = 3	switched shunt not found
	IERR = 4	bus does not belong to a substation
	IERR = 5	node not found in substation
	IERR = 6	invalid node number
	IERR = 7	node-breaker terminal connection table is full
	IERR = 8	error when moving switched shunt terminal connection
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.40. STATION_THREE_WND_TERM_CHNG

Use this API to access existing substation three-winding transformer terminal data in the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_THREE_WND_TERM_CHNG  IBUS JBUS KBUS CKTARG
 INTGAR(1)..INTGAR(3)
```

Python command syntax:

```
ierr = station_three_wnd_term_chng(ibus, jbus, kbus, cktarg, intgar)
```

Fortran command syntax:

```
CALL ST_NTHREE_WINDING(IBUS, JBUS, KBUS, CKTARG, INTGAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Integer KBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	from node number
	INTGAR(2)	to node number
	INTGAR(3)	to node number
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	bus does not belong to a substation
	IERR = 3	node does not exist
	IERR = 4	node number is invalid
	IERR = 5	circuit identifier is more than two characters
	IERR = 6	circuit identifier may not start with &
	IERR = 7	circuit does not exist
	IERR = 8	error when moving winding terminal connection
	IERR = 9	node-breaker terminal connection table is full
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.41. STATION_THREE_WND_TERM_MOVE

Use this API to move one terminal node connection of a three-winding transformer in the working case. The third bus argument (KBUS) is the bus at which the node connection will be moved to the node designated in the DSNODE argument. IBUS and JBUS are the other two three winding transformer bus connections.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_STATION_THREE_WND_TERM_MOVE  IBUS JBUS KBUS CKTARG DSNODE
```

Python command syntax:

```
ierr = station_three_wnd_term_move(ibus, jbus, kbus, cktarg, dsnode)
```

Fortran command syntax:

```
CALL MOVE3WTERMINAL(IBUS, JBUS, KBUS, CKTARG, DSNODE, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Integer KBUS	third bus number. This is the bus for which the node connection will be moved to the node designated in DSNODE. (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer DSNODE	new terminal node number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	bus does not exist
	IERR = 2	bus does not belong to a substation
	IERR = 3	node does not exist
	IERR = 4	invalid node number
	IERR = 5	circuit identifier is more than two characters
	IERR = 6	circuit identifier may not start with &
	IERR = 7	circuit does not exist
	IERR = 8	error when moving winding terminal connection
	IERR = 9	node-breaker terminal connection table is full
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.42. STATION_VSCDC_BUS_TERM_CHNG

Use this API to change the vsc dc converter node terminal connection in the working case.
This api is defined using the converter end bus number and desired node number to connect to at that end.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_STATION_VSCDC_BUS_TERM_CHNG  NAME IBUS INODE
```

Python command syntax:

```
ierr = station_vscdc_bus_term_chng(name, ibus, inode)
```

Fortran command syntax:

```
CALL STATION_VSCDC_BUS_TERM_CHNG(NAME, IBUS, INODE, IERR)
```

Where:

character*12 NAME	quoted name of the vsc dc line whose node terminal data is being changed (input; no default allowed).	
Integer IBUS	bus number of desired converter end being modified (input; no default allowed).	
Integer INODE	node number to which the converter end is to be attached. Node must correspond to IBUS. (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	VSC DC line not found
	IERR = 2	bus is not part of a substation
	IERR = 3	node not found in substation
	IERR = 4	Invalid node number
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = 7	bus not found
	IERR = 8	bus not a converter bus of vsc dc line

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.43. STATION_VSCDC_CONV_TERM_CHNG

This API changes both node terminal connections of an existing vsc dc converter in the working case. A new API station_vscdc_bus_term_chng has been introduced at 35.5 in which the node terminal connection at one end is changed by specifying the converter bus number and desired node connection.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_STATION_VSCDC_CONV_TERM_CHNG  NAME INTGAR(1) INTGAR(2)
```

Python command syntax:

```
ierr = station_vscdc_conv_term_chng(name, intgar)
```

Fortran command syntax:

```
CALL ST_NVSCDC(NAME, INTGAR, IERR)
```

Where:

character*12 NAME	quoted name of the vsc dc line for which node terminal data for one or both of its converters is to be specified (input; no default allowed).	
Integer INTGAR (2)	array of elements specifying (input).	
	INTGAR(1)	node number to which the first converter end is attached
	INTGAR(2)	node number to which the second converter end is attached
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR > 0	fatal error occurred
	IERR = 1	vsc dc line not found
	IERR = 2	bus associated with node is not part of a substation
	IERR = 3	node does not exist
	IERR = 4	Invalid node number
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.44. SYSTEM_SWD_CHNG

This API routine is obsolete. It has been replaced by the API routine system_swd_chng_2 .

Use this API to modify the data of an existing system switching device in the working case, or to add a new system switching device to the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SYSTEM_SWD_CHNG IBUS JBUS CKTARG INTGAR(1)..INTGAR(4) RBRXAR
 REALAR(1)..REALAR(12) NAMEAR
```

Python command syntax:

```
ierr = system_swd_chng(ibus, jbus, cktarg, intgar, rbrxar, realar, namear)
```

Fortran command syntax:

```
CALL SYSTEM_SWD_CHNG(IBUS, JBUS, CKTARG, INTGAR, RBRXAR, REALAR, NAMEAR,
 IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	device status
	INTGAR(2)	device normal status
	INTGAR(3)	metered end bus number
	INTGAR(4)	device type
Real RBRXAR	nominal branch reactance (input).	
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	rating set 1 rating
	REALAR(2)	rating set 2 rating
	REALAR(3)	rating set 3 rating
	REALAR(4)	rating set 4 rating
	REALAR(5)	rating set 5 rating
	REALAR(6)	rating set 6 rating
	REALAR(7)	rating set 7 rating
	REALAR(8)	rating set 8 rating
	REALAR(9)	rating set 9 rating
	REALAR(10)	rating set 10 rating
	REALAR(11)	rating set 11 rating
	REALAR(12)	rating set 12 rating

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*40 NAMEAR	switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus is not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a two-winding transformer
	IERR = 4	branch exists but it is a regular, non-transformer barhcn
	IERR = 5	switching device from bus to itself
	IERR = 6	blank circuit identifier is invalid
	IERR = 7	circuit identifier may not start with &
	IERR = 8	branch table is full
	IERR = 9	branch ownership table is full
	IERR = 10	switching device table is full
	IERR = 11	prerequisite requirements for API are not met
	IERR = 12	zero impedance line table is full
	IERR = 13	branch not found
	IERR = 14	node-breaker terminal connection table is full
	IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0 or greater than zero impedance line threshold. - invalid metered end bus. - invalid switching device status. - system switching device name has too many characters - system switching device rate is negative - zero impedance line table is full - multi-section line grouping deleted - name is already assigned to a branch - name contains invalid characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.45. SYSTEM_SWD_CHNG_2

Use this API to modify the data of an existing system switching device in the working case, or to add a new system switching device to the working case.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SYSTEM_SWD_CHNG_2  IBUS JBUS CKTARG INTGAR(1)..INTGAR(4) REALAR(1)
 RSETNAM NAMEAR
```

Python command syntax:

```
ierr = system_swd_chng_2(ibus, jbus, cktarg, intgar, realar, rsetnam,
 namear)
```

Fortran command syntax:

```
CALL SYSTEM_SWD_CHNG_2(IBUS, JBUS, CKTARG, INTGAR, REALAR, RSETNAM,
 NAMEAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	device status (alias is stat)
	INTGAR(2)	device normal status (alias is nstat)
	INTGAR(3)	metered end bus number (alias is met)
	INTGAR(4)	device type (alias is stype)
Real REALAR (1)	array of 1 elements specifying nominal branch reactance(input).	
	REALAR(1)	device reactance (alias is xpu) (0.0001 by default)
Character*40 RSETNAM	switching device rating set name (input).	
Character*40 NAMEAR	switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus is not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a two-winding transformer
	IERR = 4	branch exists but it is a regular, non-transformer barhcn
	IERR = 5	switching device from bus to itself
	IERR = 6	blank circuit identifier is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	circuit identifier may not start with &
IERR = 8	branch table is full
IERR = 9	branch ownership table is full
IERR = 10	switching device table is full
IERR = 11	prerequisite requirements for API are not met
IERR = 12	zero impedance line table is full
IERR = 13	branch not found
IERR = 14	node-breaker terminal connection table is full
IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0 or greater than zero impedance line threshold. - invalid metered end bus. - invalid switching device status. - system switching device name has too many characters - system switching device rate is negative - zero impedance line table is full - multi-section line grouping deleted - name is already assigned to a branch - name contains invalid characters - rating set name has too many characters - rating set name is set to blank - rating set name is not in the SWD rating table

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.46. SYSTEM_SWD_DATA

This API routine is obsolete. It has been replaced by the API routine system_swd_data_2 .

Use this API to add or modify the data of an existing system switching device in the working case, or to add a new system switching device to the working case.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SYSTEM_SWD_DATA IBUS JBUS CKTARG INTGAR(1)..INTGAR(4) RBRXAR
 REALAR(1)..REALAR(12) NAMEAR
```

Python command syntax:

```
ierr = system_swd_data(ibus, jbus, cktarg, intgar, rbrxar, realar, namear)
```

Fortran command syntax:

```
CALL SYSTEM_SWD_DATA(IBUS, JBUS, CKTARG, INTGAR, RBRXAR, REALAR, NAMEAR,
 IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	device status
	INTGAR(2)	device normal status
	INTGAR(3)	metered end bus number
	INTGAR(4)	device type
Real RBRXAR	nominal branch reactance (input).	
Real REALAR (12)	array of 12 elements specifying (input).	
	REALAR(1)	rating set 1 rating
	REALAR(2)	rating set 2 rating
	REALAR(3)	rating set 3 rating
	REALAR(4)	rating set 4 rating
	REALAR(5)	rating set 5 rating
	REALAR(6)	rating set 6 rating
	REALAR(7)	rating set 7 rating
	REALAR(8)	rating set 8 rating
	REALAR(9)	rating set 9 rating
	REALAR(10)	rating set 10 rating
	REALAR(11)	rating set 11 rating
	REALAR(12)	rating set 12 rating

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*40 NAMEAR	switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus is not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a two-winding transformer
	IERR = 4	branch exists but it is a regular, non-transformer barhcn
	IERR = 5	switching device from bus to itself
	IERR = 6	blank circuit identifier is invalid
	IERR = 7	circuit identifier may not start with &
	IERR = 8	branch table is full
	IERR = 9	branch ownership table is full
	IERR = 10	switching device table is full
	IERR = 11	prerequisite requirements for API are not met
	IERR = 12	zero impedance line table is full
	IERR = 13	branch already exists
	IERR = 14	node-breaker terminal connection table is full
	IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0 or greater than zero impedance line threshold. - invalid metered end bus. - invalid switching device status. - system switching device name has too many characters - system switching device rate is negative - zero impedance line table is full - multi-section line grouping deleted. - name is already assigned to a branch - name contains invalid characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

2.2.47. SYSTEM_SWD_DATA_2

Use this API to add or modify the data of an existing system switching device in the working case, or to add a new system switching device to the working case.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SYSTEM_SWD_DATA_2  IBUS JBUS CKTARG INTGAR(1)..INTGAR(4) REALAR(1)
 RSETNAM NAMEAR
```

Python command syntax:

```
ierr = system_swd_data_2(ibus, jbus, cktarg, intgar, realar, rsetnam,
 namear)
```

Fortran command syntax:

```
CALL SYSTEM_SWD_DATA_2(IBUS, JBUS, CKTARG, INTGAR, REALAR, RSETNAM,
 NAMEAR, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKTARG	circuit identifier (input).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	device status (alias is stat)
	INTGAR(2)	device normal status (alias is nstat)
	INTGAR(3)	metered end bus number (alias is met)
	INTGAR(4)	device type (alias is stype)
Real REALAR (1)	array of 1 elements specifying nominal branch reactance(input).	
	REALAR(1)	device reactance (alias is xpu) (0.0001 by default)
Character*40 RSETNAM	switching device rating set name (input).	
Character*40 NAMEAR	switching device name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus is not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	branch exists but it is a two-winding transformer
	IERR = 4	branch exists but it is a regular, non-transformer barhcn
	IERR = 5	switching device from bus to itself
	IERR = 6	blank circuit identifier is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	circuit identifier may not start with &
IERR = 8	branch table is full
IERR = 9	branch ownership table is full
IERR = 10	switching device table is full
IERR = 11	prerequisite requirements for API are not met
IERR = 12	zero impedance line table is full
IERR = 13	branch already exists
IERR = 14	node-breaker terminal connection table is full
IERR = -1	data error, warning and information messages; one or more of: - switching device reactance is 0.0 or greater than zero impedance line threshold. - invalid metered end bus. - invalid switching device status. - system switching device name has too many characters. - system switching device rate is negative. - zero impedance line table is full. - multi-section line grouping deleted. - name is already assigned to a branch. - name contains invalid characters. - rating set name has too many characters. - rating set name is set to blank. - rating set name is not in the SWD rating table.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

This section describes the API routines used to add or modify power flow data pertaining to substation data.

For most categories of power flow substation data, there are a pair of API routines for specifying data; one of these contains "_DATA" as part of the API name, and the other contains "_CHNG" in its API name. The "_DATA" API routines may be used to add a new network element to the working case, or to modify the data of an existing network element. The "_CHNG" API routines may be used only to modify the data of an existing network element; specifying a network element that does not already exist in the working case to one of the "_CHNG" API routines is treated as a fatal error by the API routine.

Generally, when an API is adding a new element to the working case, data items that are defaulted are set to the value listed in the data item's description. When an API is modifying the data of an existing element, defaulted data items are unchanged.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 3
Optimal Power Flow

This chapter describes the API routines used in PSS®E Optimal Power Flow, or OPF.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1. Optimal Power Flow Operation

3.1.1. ADD_DETAILS_TO_OPF_LOG

Use this API to specify or return the value of the OPF solution option to add details of the sensitivity values and Lagrange multipliers to the OPF Output Log file.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_ADD_DETAILS_TO_OPF_LOG  IVAL
```

Python command syntax:

```
ierr = add_details_to_opf_log(ival)

ierr, ival = add_details_to_opf_log()
```

Fortran command syntax:

```
CALL ADD_DETAILS_TO_OPF_LOG(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to append the OPF log file with information on sensitivity values and Lagrange multipliers (input; input if Batch command or for Python if specified as an argument, or for FORTRAN if IOCODE is 0; else output, 0 by default).	
	IVAL = 0	do not print additional details in log file.
	IVAL = 1	print additional details.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.2. APPLY_ALTERNATE_STEP_SIZE

Use this API to specify or return the value of the OPF solution option for applying an alternate lambda mismatch step size. This option works in conjunction with the OPF_BARRIER_STEP_LENGTH option setting (Section 3.1.25), which specifies the alternate barrier step length value.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_APPLY_ALTERNATE_STEP_SIZE    IVAL
```

Python command syntax:

```
ierr = apply_alternate_step_size(ival)

ierr, ival = apply_alternate_step_size()
```

Fortran command syntax:

```
CALL APPLY_ALTERNATE_STEP_SIZE(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to limit the step size taken during the OPF solution (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not limit the step size.
	IVAL = 1	limit the step size.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.3. APPLY_AUTOMATIC_SCALING

Use this API to specify or return the value of the OPF solution option to apply automatic scaling of the objective coefficients and derivatives to improve convergence of the OPF solution.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_APPLY_AUTOMATIC_SCALING    IVAL
```

Python command syntax:

```
ierr = apply_automatic_scaling(ival)

ierr, ival = apply_automatic_scaling()
```

Fortran command syntax:

```
CALL APPLY_AUTOMATIC_SCALING(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to apply automatic scaling of the objective during the solution (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not apply automatic scaling of the objective.
	IVAL = 1	apply automatic scaling of the objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.4. BAD_ITER_COARSE_LIMIT

Use this API to specify or return the number of bad iterations allowed when the barrier coefficient is greater than 0.011 (mu greater then -2) before the solution progress is terminated as being infeasible.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_BAD_ITER_COARSE_LIMIT IVAL
```

Python command syntax:

```
ierr =  bad_iter_coarse_limit(ival)

ierr, ival = bad_iter_coarse_limit()
```

Fortran command syntax:

```
CALL BAD_ITER_COARSE_LIMIT(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Bad iteration coarse limit value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.5. BAD_ITER_FINE_LIMIT

Use this API to specify or return the the bad iteration limit when the barrier coefficient is less than, or equal to 0.011 (mu less than or equal to -2) before the solution progress is terminated as being infeasible.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_BAD_ITER_FINE_LIMIT IVAL
```

Python command syntax:

```
ierr =  bad_iter_fine_limit(ival)

ierr, ival = bad_iter_fine_limit()
```

Fortran command syntax:

```
CALL BAD_ITER_FINE_LIMIT(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Bad iteration fine limit value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.6. CLAMP_NONOPTIMIZED_GENS

Use this API to specify or return the value of the OPF solution option to apply the clamp equation to non-optimized generators. The behavior of the clamp equation may be further customized through the OPF_INITIAL_CLAMP_TOL, OPF_FINAL_CLAMP_TOL and the OPF_CLAMP_DECAY_FACTOR option settings. For additional information on the clamp treatment of non-optimized generators, refer to Clamp Treatment in the PSSE Program Operation Manual.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_CLAMP_NONOPTIMIZED_GENS    IVAL
```

Python command syntax:

```
ierr = clamp_nonoptimized_gens(ival)

ierr, ival = clamp_nonoptimized_gens()
```

Fortran command syntax:

```
CALL CLAMP_NONOPTIMIZED_GENS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to apply the clamp equation to nonoptimized generators (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not clamp nonoptimized generators.
	IVAL = 1	clamp nonoptimized generators.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.7. CONSTRAIN_INTERFACE_FLOWS

Use this API to specify or return the value of the OPF option to constrain interface flows. The setting of this option affects recognition of any interface flow constraints and the behavior of the MINIMIZE_INTERFACE_FLOWS objective function option.
If the Constrain Interface Flows option is not enabled, the Minimize Interrface Flows objective and any interface flow constraints will be ignored and not used.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_CONSTRAIN_INTERFACE_FLOWS    IVAL
```

Python command syntax:

```
ierr = constrain_interface_flows(ival)

ierr, ival = constrain_interface_flows()
```

Fortran command syntax:

```
CALL CONSTRAIN_INTERFACE_FLOWS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the option to constrain interface flows (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not constrain interface flows.
	IVAL = 1	constrain interface flows.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.8. FINAL_OPF_BARRIER_COEFF

Use this API to specify or return the value of the final coefficient value that the barrier function is attempting to attain. It may be used in conjunction with the INITIAL_OPF_BARRIER_COEFF value.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_FINAL_OPF_BARRIER_COEFF    RVAL
```

Python command syntax:

```
ierr =  final_opf_barrier_coeff(rval)

ierr, rval = final_opf_barrier_coeff()
```

Fortran command syntax:

```
CALL FINAL_OPF_BARRIER_COEFF(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	final barrier coefficient value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.9. IMPORT_ECDI

Use this API to import data from an Economics Dispatch Data File, as prepared for use in activity ECDI, into the OPF working data. This function removes all existing Optimal Power Flow Generator Dispatch Data, Active Power Dispatch Data, and Cost Curve Data, before adding new records from the ECD file.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_IMPORT_ECDI  FILE
```

Python command syntax:

```
ierr = import_ecdi(file)
```

Fortran command syntax:

```
CALL IMPORT_ECDI(FILE, IERR)
```

Where:

Character*260 FILE	name of the Economics Dispatch Data File (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	NEWOPF_API error occurred
	IERR = 2	ECDIAPI error occurred
	IERR = 3	CSTTBL_QUAD_API error occurred
	IERR = 4	OPFDSP_TBL_API error occurred
	IERR = 5	GENDSP_INDV_API error occurred
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.10. INITIAL_OPF_BARRIER_COEFF

Use this API to specify or return the value of the initial OPF barrier coefficient. It may be used in conjunction with the FINAL_OPF_BARRIER_COEFF value.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_INITIAL_OPF_BARRIER_COEFF    RVAL
```

Python command syntax:

```
ierr =  initial_opf_barrier_coeff(rval)

ierr, rval = initial_opf_barrier_coeff()
```

Fortran command syntax:

```
CALL INITIAL_OPF_BARRIER_COEFF(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	initial barrier coefficient value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.11. INTERFACE_FLOW_COST_COEFF

Use this API to specify or return the value of the coefficient for the interface flow objective. This API works in conjunction with the MINIMIZE_INTERFACE_FLOWS objective function option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_INTERFACE_FLOW_COST_COEFF    RVAL
```

Python command syntax:

```
ierr =  interface_flow_cost_coeff(rval)

ierr, rval = interface_flow_cost_coeff()
```

Fortran command syntax:

```
CALL INTERFACE_FLOW_COST_COEFF(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the interface flow cost coefficient (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.12. LSTO

Use this API to list OPF working case data in a form suitable for data documentation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LSTO SID ALL OPT VREV
```

Python command syntax:

```
ierr = lsto(sid, all, opt, vrev)
```

Fortran command syntax:

```
CALL LSTONW(SID, ALL, OPT, VREV, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer OPT	number corresponding to the category of OPF data that is to be tabulated (input; no default allowed).	
	OPT = 1	case summary
	OPT = 2	bus attribute data
	OPT = 3	adjustable bus shunt data
	OPT = 4	bus load data
	OPT = 5	adjustable bus load table data
	OPT = 6	generator dispatch data
	OPT = 7	active power dispatch data
	OPT = 8	generation reserve data
	OPT = 9	generator reactive capability data
	OPT = 10	adjustable branch reactance data
	OPT = 11	linear cost table data
	OPT = 12	quadratic cost table data
	OPT = 13	polynomial and exponential cost table data
	OPT = 14	period reserve constraint data
	OPT = 15	branch flow constraint data
	OPT = 16	interface flow constraint data
	OPT = 17	linear constraint dependency data
	OPT = 21	full listing
Integer VREV	option to change voltage units from the default units to the alternate units (input; no default allowed).	
	VREV = 0	use default voltage units.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VREV = 1	use alternate voltage units.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 4	invalid VREV value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.13. MINIMIZE_ADJ_BUS_SHUNTS

Use this API to specify or return the value of the OPF minimize adjustable bus shunts (var compensation) objective function option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_ADJ_BUS_SHUNTS    IVAL
```

Python command syntax:

```
ierr = minimize_adj_bus_shunts(ival)

ierr, ival = minimize_adj_bus_shunts()
```

Fortran command syntax:

```
CALL MINIMIZE_ADJ_BUS_SHUNTS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize adjustable bus shunts objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize adjustable bus shunts objective.
	IVAL = 1	enable the minimize adjustable bus shunts objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.14. MINIMIZE_FUEL_COST

Use this API to specify or return the value of the OPF minimize fuel cost objective option. The Minimize fuel cost objective and Minimize active power loss objective cannot be simultaneously employed due to the competing goals.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_FUEL_COST   IVAL
```

Python command syntax:

```
ierr = minimize_fuel_cost(ival)

ierr, ival = minimize_fuel_cost()
```

Fortran command syntax:

```
CALL MINIMIZE_FUEL_COST(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize fuel cost objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize fuel cost objective.
	IVAL = 1	enable the minimize fuel cost objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.15. MINIMIZE_INTERFACE_FLOWS

Use this API to specify or return the value of the OPF minimize interface flows objective option. Used in conjuction with INTERFACE_FLOW_COST_COEFFICIENT. A negative coefficient value will maximize interface flows.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_INTERFACE_FLOWS    IVAL
```

Python command syntax:

```
ierr = minimize_interface_flows(ival)

ierr, ival = minimize_interface_flows()
```

Fortran command syntax:

```
CALL MINIMIZE_INTERFACE_FLOWS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize interface flows objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize interface flows objective.
	IVAL = 1	enable the minimize interface flows objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.16. MINIMIZE_LOAD_ADJUSTMENTS

Use this API to specify or return the value of the OPF minimize adjustable bus load objective option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_LOAD_ADJUSTMENTS    IVAL
```

Python command syntax:

```
ierr = minimize_load_adjustments(ival)

ierr, ival = minimize_load_adjustments()
```

Fortran command syntax:

```
CALL MINIMIZE_LOAD_ADJUSTMENTS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize adjustable bus loads objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize adjustable bus loads objective.
	IVAL = 1	enable the minimize adjustable bus loads objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.17. MINIMIZE_P_LOSSES

Use this API to specify or return the value of the OPF minimize active power loss objective option. The Minimize active power loss objective and Minimize fuel cost objective cannot be simultaneously employed due to the competing goals

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_P_LOSSES    IVAL
```

Python command syntax:

```
ierr = minimize_p_losses(ival)

ierr, ival = minimize_p_losses()
```

Fortran command syntax:

```
CALL MINIMIZE_P_LOSSES(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize active power loss objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize active power loss objective.
	IVAL = 1	enable the minimize active power loss objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.18. MINIMIZE_P_SLACK

Use this API to specify or return the value of the OPF minimize active power slack objective option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_P_SLACK    IVAL
```

Python command syntax:

```
ierr = minimize_p_slack(ival)

ierr, ival = minimize_p_slack()
```

Fortran command syntax:

```
CALL MINIMIZE_P_SLACK(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize active power slack objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize active power slack objective.
	IVAL = 1	enable the minimize active power slack objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.19. MINIMIZE_Q_LOSSES

Use this API to specify or return the value of the OPF minimize reactive power loss objective option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_Q_LOSSES    IVAL
```

Python command syntax:

```
ierr = minimize_q_losses(ival)

ierr, ival = minimize_q_losses()
```

Fortran command syntax:

```
CALL MINIMIZE_Q_LOSSES(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize reactive power loss objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize reactive power loss objective.
	IVAL = 1	enable the minimize reactive power loss objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.20. MINIMIZE_Q_SLACK

Use this API to specify or return the value of the OPF minimize reactive power slack objective option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_Q_SLACK    IVAL
```

Python command syntax:

```
ierr = minimize_q_slack(ival)

ierr, ival = minimize_q_slack()
```

Fortran command syntax:

```
CALL MINIMIZE_Q_SLACK(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize reactive power slack objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize reactive power slack objective.
	IVAL = 1	enable the minimize reactive power slack objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.21. MINIMIZE_REACTIVE_RESERVE

Use this API to specify or return the value of the OPF minimize reactive reserve objective option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_REACTIVE_RESERVE    IVAL
```

Python command syntax:

```
ierr = minimize_reactive_reserve(ival)

ierr, ival = minimize_reactive_reserve()
```

Fortran command syntax:

```
CALL MINIMIZE_REACTIVE_RESERVE(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize reactive reserves objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize reactive reserves objective.
	IVAL = 1	enable the minimize reactive reserves objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.22. MINIMIZE_SERIES_COMP

Use this API to specify or return the value of the OPF minimize series compensation (adjustable branch reactance) objective option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_MINIMIZE_SERIES_COMP   IVAL
```

Python command syntax:

```
ierr = minimize_series_comp(ival)

ierr, ival = minimize_series_comp()
```

Fortran command syntax:

```
CALL MINIMIZE_SERIES_COMP(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the minimize series compensation objective (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not enable the minimize series compensation objective.
	IVAL = 1	enable the minimize series compensation objective.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.23. NOPF

Use this API to run the Optimal Power Flow solution. Subsystems that have been specified will be optimized to the objectives and tolerances specified, if a solution is achievable.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_NOPF   SID  ALL
```

Python command syntax:

```
ierr = nopf(sid, all)
```

Fortran command syntax:

```
CALL NOPF_SOLVE(SID, ALL, IERR)
```

Where:

Integer SID	a valid bus subsystem ID from 0 to 11. A bus subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	error initializing OPF data
	IERR = 4	generators are converted
	IERR = 5	Topology error: buses exist with bus type code and series element status inconsistencies
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.24. OPEN_BUS_VOLTAGE_LIMITS

Use this API to specify or return the value of the OPF solution option to automatically adjust bus voltage limits, making them more open for initial feasibility.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPEN_BUS_VOLTAGE_LIMITS    IVAL
```

Python command syntax:

```
ierr =  open_bus_voltage_limits(ival)

ierr, ival = open_bus_voltage_limits()
```

Fortran command syntax:

```
CALL OPEN_BUS_VOLTAGE_LIMITS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to open bus voltage limits for initial feasibility (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not open up bus voltage limits.
	IVAL = 1	open bus voltage limits for initial feasibility.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.25. OPF_BARRIER_STEP_LENGTH

Use this API to specify or return the value of the barrier function step length setting.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_BARRIER_STEP_LENGTH    RVAL
```

Python command syntax:

```
ierr =  opf_barrier_step_length(rval)

ierr, rval = opf_barrier_step_length()
```

Fortran command syntax:

```
CALL OPF_BARRIER_STEP_LENGTH(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the barrier function step length (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.26. OPF_CLAMP_DECAY_FACTOR

Use this API to specify or return the value of the clamp equation decay factor. This API works in conjunction with the API option to CLAMP_NONOPTIMIZED_GENS, and is associated with the APIs for OPF_INITIAL_CLAMP_TOL and OPF_FINAL_CLAMP_TOL.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_CLAMP_DECAY_FACTOR    RVAL
```

Python command syntax:

```
ierr =  opf_clamp_decay_factor(rval)

ierr, rval = opf_clamp_decay_factor()
```

Fortran command syntax:

```
CALL OPF_CLAMP_DECAY_FACTOR(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the clamp equation decay factor (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.27. OPF_FINAL_CLAMP_TOL

Use this API to specify or return the value of the final clamp tolerance. This API works in conjunction with the API option to CLAMP_NONOPTIMIZED_GENS, and is associated with the APIs for OPF_INITIAL_CLAMP_TOL and the OPF_CLAMP_DECAY_FACTOR.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_FINAL_CLAMP_TOL    RVAL
```

Python command syntax:

```
ierr =  opf_final_clamp_tol(rval)

ierr, rval = opf_final_clamp_tol()
```

Fortran command syntax:

```
CALL OPF_FINAL_CLAMP_TOL(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the final clamp tolerance (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.28. OPF_FIXED_VOLTAGE_PENALTY

Use this API to specify or return the value of the penalty for fixed voltage excursions.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_FIXED_VOLTAGE_PENALTY    RVAL
```

Python command syntax:

```
ierr =  opf_fixed_voltage_penalty(rval)

ierr, rval = opf_fixed_voltage_penalty()
```

Fortran command syntax:

```
CALL OPF_FIXED_VOLTAGE_PENALTY(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the penalty for fixed voltage excursions (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.29. OPF_FIX_ALL_GENERATORS

Use this API to specify or return the value of the OPF option to treat all generators as nonoptimized (fixed).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_FIX_ALL_GENERATORS    IVAL
```

Python command syntax:

```
ierr = opf_fix_all_generators(ival)

ierr, ival = opf_fix_all_generators()
```

Fortran command syntax:

```
CALL OPF_FIX_ALL_GENERATORS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the option to treat all generators as nonoptimized and fix their settings (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not treat all generators as nonoptimized.
	IVAL = 1	treat all generators as nonoptimized.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.30. OPF_FIX_PHASE_SHIFTERS

Use this API to specify or return the value of the OPF option to fix transformer phase shift angle settings.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_FIX_PHASE_SHIFTERS    IVAL
```

Python command syntax:

```
ierr = opf_fix_phase_shifters(ival)

ierr, ival = opf_fix_phase_shifters()
```

Fortran command syntax:

```
CALL OPF_FIX_PHASE_SHIFTERS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the option to fix transformer phase shift angles (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not fix transformer phase shift angle settings.
	IVAL = 1	fix transformer phase shift angle settings.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.31. OPF_FIX_SWITCHED_SHUNTS

Use this API to specify or return the value of the OPF option to fix switched shunt settings.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_FIX_SWITCHED_SHUNTS    IVAL
```

Python command syntax:

```
ierr = opf_fix_switched_shunts(ival)

ierr, ival = opf_fix_switched_shunts()
```

Fortran command syntax:

```
CALL OPF_FIX_SWITCHED_SHUNTS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the option to fix switched shunt settings (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not fix switched shunt settings.
	IVAL = 1	fix switched shunt settings.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.32. OPF_FIX_TAP_RATIOS

Use this API to specify or return the value of the OPF option to fix transformer tap ratio settings.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_FIX_TAP_RATIOS    IVAL
```

Python command syntax:

```
ierr = opf_fix_tap_ratios(ival)

ierr, ival = opf_fix_tap_ratios()
```

Fortran command syntax:

```
CALL OPF_FIX_TAP_RATIOS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the option to fix transformer tap ratio settings (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not fix transformer tap ratio settings.
	IVAL = 1	fix transformer tap ratio settings.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.33. OPF_INITIAL_CLAMP_TOL

Use this API to specify or return the value of the initial clamp tolerance. This API works in conjunction with the API option to CLAMP_NONOPTIMIZED_GENS, and is associated with the APIs for OPF_FINAL_CLAMP_TOL and the OPF_CLAMP_DECAY_FACTOR.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_INITIAL_CLAMP_TOL    RVAL
```

Python command syntax:

```
ierr =  opf_initial_clamp_tol(rval)

ierr, rval = opf_initial_clamp_tol()
```

Fortran command syntax:

```
CALL OPF_INITIAL_CLAMP_TOL(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the initial clamp tolerance (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.34. OPF_INTERIOR_SHIFT_FACTOR

Use this API to specify or return the value of the interior shift factor multiplier. This value indicates by how much a variable should be moved "more interior" of the upper and lower limits.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPF_INTERIOR_SHIFT_FACTOR    RVAL
```

Python command syntax:

```
ierr =  opf_interior_shift_factor(rval)

ierr, rval = opf_interior_shift_factor()
```

Fortran command syntax:

```
CALL OPF_INTERIOR_SHIFT_FACTOR(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	interior shift factor multiplier value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.35. OPF_LAMBDA_TOLERANCE

Use this API to specify or return the value of the Lagrange multiplier blow-up tolerance during an optimal power flow solution.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_LAMBDA_TOLERANCE    RVAL
```

Python command syntax:

```
ierr =  opf_lambda_tolerance(rval)

ierr, rval = opf_lambda_tolerance()
```

Fortran command syntax:

```
CALL OPF_LAMBDA_TOLERANCE(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	lambda tolerance value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.36. OPF_LF_CONTROL_PENALTY

Use this API to specify or return the value of the quadratic penalty for loadflow controls. This penalty is applied to optimized loadflow control variable excursions from their initial condition, in $/pu.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_LF_CONTROL_PENALTY    RVAL
```

Python command syntax:

```
ierr =  opf_lf_control_penalty(rval)

ierr, rval = opf_lf_control_penalty()
```

Fortran command syntax:

```
CALL OPF_LF_CONTROL_PENALTY(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the quadratic penalty for loadflow controls (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.37. OPF_MAX_TAP_RATIO_STEP

Use this API to specify or return the value of the maximum transformer tap ratio step.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_MAX_TAP_RATIO_STEP    RVAL
```

Python command syntax:

```
ierr =  opf_max_tap_ratio_step(rval)

ierr, rval = opf_max_tap_ratio_step()
```

Fortran command syntax:

```
CALL OPF_MAX_TAP_RATIO_STEP(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	maximum tap ratio step value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.38. OPF_MIN_TAP_RATIO_STEP

Use this API to set or return the value of the minimum transformer tap ratio step.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_MIN_TAP_RATIO_STEP    RVAL
```

Python command syntax:

```
ierr =  opf_min_tap_ratio_step(rval)

ierr, rval = opf_min_tap_ratio_step()
```

Fortran command syntax:

```
CALL OPF_MIN_TAP_RATIO_STEP(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	minimum tap ratio step value (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.39. OPF_REGULATE_AREA_INT

Use this API to set or return the value of the OPF option to regulate area interchange.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_REGULATE_AREA_INT    IVAL
```

Python command syntax:

```
ierr = opf_regulate_area_int(ival)

ierr, ival = opf_regulate_area_int()
```

Fortran command syntax:

```
CALL OPF_REGULATE_AREA_INT(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to enable the option to regulate area interchange (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not regulate area interchange.
	IVAL = 1	regulate area interchange.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.40. OPF_ROUND_SWITCHED_SHUNTS

Use this API to set or return the value of the OPF solution option to discretize switched shunts.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_ROUND_SWITCHED_SHUNTS    IVAL
```

Python command syntax:

```
ierr = opf_round_switched_shunts(ival)

ierr, ival = opf_round_switched_shunts()
```

Fortran command syntax:

```
CALL OPF_ROUND_SWITCHED_SHUNTS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to discretize switched shunt settings after initial solution. (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not discretize switched shunt settings.
	IVAL = 1	discretize switched shunt settings.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.41. OPF_ROUND_TAP_RATIOS

Use this API to set or return the value of the OPF solution option to round transformer tap ratio settings.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_ROUND_TAP_RATIOS    IVAL
```

Python command syntax:

```
ierr = opf_round_tap_ratios(ival)

ierr, ival = opf_round_tap_ratios()
```

Fortran command syntax:

```
CALL OPF_ROUND_TAP_RATIOS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to round transformer tap ratio settings (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not round transformer tap ratio settings.
	IVAL = 1	round transformer tap ratio settings.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.42. OPF_SCALE_QGEN_LIMITS

Use this API to set or return the value of the OPF option to scale reactive generation limits in the clamped constraint equation.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_SCALE_QGEN_LIMITS   IVAL
```

Python command syntax:

```
ierr = opf_scale_qgen_limits(ival)

ierr, ival = opf_scale_qgen_limits()
```

Fortran command syntax:

```
CALL OPF_SCALE_QGEN_LIMITS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to scale reactive generation limits in the clamp equation (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not scale reactive generation limits.
	IVAL = 1	scale reactive generation limits.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.43. OPF_STEP_LENGTH_TOLERANCE

Use this API to set or return the value of the OPF minimum barrier step length tolerance.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_STEP_LENGTH_TOLERANCE    RVAL
```

Python command syntax:

```
ierr =  opf_step_length_tolerance(rval)

ierr, rval = opf_step_length_tolerance()
```

Fortran command syntax:

```
CALL OPF_STEP_LENGTH_TOLERANCE(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the minimum barrier step length tolerance (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.44. OPF_USE_GENERATOR_VSCHED

Use this API to set or return the value of the OPF option to employ the generator scheduled voltage.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPF_USE_GENERATOR_VSCHED    IVAL
```

Python command syntax:

```
ierr = opf_use_generator_vsched(ival)

ierr, ival = opf_use_generator_vsched()
```

Fortran command syntax:

```
CALL OPF_USE_GENERATOR_VSCHED(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to use generator scheduled voltage values (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not use generator scheduled voltages.
	IVAL = 1	use generator scheduled voltages.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.45. OPTO

Please use the individual OPF option setting API's described in this section.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPTO   SID  ALL  IARY(1) .. IARY(36)  RARY(1) .. RARY(19)  OPFILE
 RCFILE
```

Python command syntax:

```
ierr = opto(sid, all, iary, rary, opfile, rcfile)
```

Fortran command syntax:

```
CALL OPTONW(SID, ALL, IARY, RARY, OPFILE, RCFILE, IERR)
```

Where:

Integer SID	a valid bus subsystem ID from 0 to 11. A bus subsystem must have been previously defined (input).	
Integer ALL	all buses or specified subsystem flag (input).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer IARY (36)	array of 36 elements specifying 36 integers, which are defined as follows(input).	
	IARY(1)	specifies the fuel cost objective
	IARY(1) = 0	do not employ minimize fuel cost objective
	IARY(1) = 1	employ the minimize fuel cost objective
	IARY(2)	specifies the active power slack bus generation objective
	IARY(2) = 0	do not employ minimize active power slack bus generation objective
	IARY(2) = 1	employ the minimize active power slack bus generation objective
	IARY(3)	specifies the reactive power slack generation objective
	IARY(3) = 0	do not employ the minimize reactive power slack generation objective
	IARY(3) = 1	employ the minimize reactive power slack generation objective
	IARY(4)	specifies the active power loss objective
	IARY(4) = 0	do not employ the minimize active power loss objective
	IARY(4) = 1	employ the minimize active power loss objective
	IARY(5)	specifies the reactive power loss objective
	IARY(5) = 0	do not employ the minimize reactive power loss objective

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IARY(5) = 1	employ the minimize reactive power loss objective
IARY(6)	specifies the adjustable branch reactance (series compensation) objective
IARY(6) = 0	do not employ the minimize adjustable branch reactance objective
IARY(6) = 1	employ the minimize adjustable branch reactance objective
IARY(7)	specifies the adjustable bus shunt (var compensation) objective
IARY(7) = 0	do not employ the minimize adjustable bus shunt objective
IARY(7) = 1	employ the minimize adjustable bus shunt objective
IARY(8)	specifies the adjustable bus load (load shedding/increase) objective
IARY(8) = 0	do not employ the adjustable bus load objective
IARY(8) = 1	employ the adjustable bus load objective
IARY(9)	specifies the interface flow objective
IARY(9) = 0	do not employ the interface flow objective
IARY(9) = 1	employ the interface flow objective
IARY(10)	specifies the reactive generation reserve objective
IARY(10) = 0	do not employ the reactive generation reserve objective
IARY(10) = 1	employ the reactive generation reserve objective
IARY(11)	specifies whether to apply the power flow mismatch tolerance to the dual variable problem
IARY(11) = 0	apply tolerance to the mismatch equation only
IARY(11) = 1	apply tolerance to the entire equation
IARY(12)	specifies whether to apply the clamp equation to nonoptimized generators
IARY(12) = 0	do not clamp nonoptimized generators
IARY(12) = 1	clamp nonoptimized generators
IARY(13)	specifies whether to apply the clamp equation to nonoptimized transformers. (Not currently functional)
IARY(13) = 0	do not clamp nonoptimized transformers
IARY(13) = 1	clamp nonoptimized transformers
IARY(14)	specifies whether to constrain interface flows
IARY(14) = 0	do not constrain interface flows

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IARY(14) = 1	constrain interface flows
IARY(15)	specifies whether to employ automatic scaling of the objective
IARY(15) = 0	do not employ automatic scaling
IARY(15) = 1	employ automatic scaling
IARY(16)	specifies whether to regulate area interchange
IARY(16) = 0	do not regulate area interchange
IARY(16) = 1	regulate area interchange
IARY(17)	specifies whether to fix transformer tap ratio settings
IARY(17) = 0	do not fix transformer tap ratio settings
IARY(17) = 1	fix transformer tap ratio settings
IARY(18)	specifies whether to fix transformer phase shift angles
IARY(18) = 0	do not fix transformer phase shift angles
IARY(18) = 1	fix transformer phase shift angles
IARY(19)	specifies whether to fix switched shunt settings
IARY(19) = 0	do not fix switched shunt settings
IARY(19) = 1	fix switched shunt settings
IARY(20)	specifies whether to treat all generators as nonoptimized
IARY(20) = 0	do not treat all generators as nonoptimized
IARY(20) = 1	treat all generators as nonoptimized
IARY(21)	specifies whether to employ the generator scheduled voltage
IARY(21) = 0	do not employ generator scheduled voltage
IARY(21) = 1	use generator scheduled voltage
IARY(22)	specifies whether to scale reactive generation limits on clamped constraint equations
IARY(22) = 0	do not scale reactive generation limits
IARY(22) = 1	scale reactive generation limits
IARY(23)	specifies whether to round transformer tap settings
IARY(23) = 0	do not round transformer tap settings
IARY(23) = 1	round transformer tap settings
IARY(24)	specifies whether to round switched shunt settings
IARY(24) = 0	do not round switched shunt settings
IARY(24) = 1	round switched shunt settings
IARY(25)	specifies whether to limit the step size taken during solution
IARY(25) = 0	do not limit the step size
IARY(25) = 1	limit the step size taken

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IARY(26)	specifies whether to produce an optimization log file
IARY(26) = 0	do not produce an optimization log file
IARY(26) = 1	produce an optimization log file
IARY(27)	specifies whether to include a detailed listing of Lagrange multipliers in the optimization log file. IARY(26) must be set to 1 for this to take effect
IARY(27) = 0	do not print details
IARY(27) = 1	print Lagrange multipliers
IARY(28)	specifies whether to produce a PSSE raw data change file. (Not currently functional)
IARY(28) = 0	do not produce an Read/Change File
IARY(28) = 1	produce an Read/Change File
IARY(29)	specifies whether to save the OPF solution option settings to the OPF Options File, PSSOPF.OPT
IARY(29) = 0	do not save settings
IARY(29) = 1	save settings to PSSOPF.OPT
IARY(30)	indicates whether the program should open voltage limits to ensure an initial feasible condition
IARY(30) = 0	do not have the program open the limits for an initial feasible condition
IARY(30) = 1	have the program open the limits for an initial feasible condition
IARY(31)	specifies whether to impose emergency bus voltage limits or normal bus voltage limits
IARY(31) = 0	impose normal voltage limits
IARY(31) = 1	impose emergency bus voltage limits
IARY(32)	specifies whether to impose emergency branch flow limits or normal branch flow limits
IARY(32) = 0	impose normal branch flow limits
IARY(32) = 1	impose emergency branch flow limits
IARY(33)	specifies whether to include transformer adjustment data in the Read/Change File (if one is produced according to the setting of IARY(28)). (Not currently functional)
IARY(33) = 0	do not include transformer adjustment data
IARY(33) = 1	include transformer adjustment data
IARY(34)	specifies the maximum iteration limit (corresponds to ITMXN of the power flow solution parameters)
IARY(35)	specifies the bad iteration coarse limit
IARY(36)	specifies the bad iteration fine limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real RARY (19) array of 19 elements specifying which are defined as follows(input).

RARY(1)	specifies the initial barrier function coefficient
RARY(2)	specifies the final barrier function coefficient
RARY(3)	specifies the step length blow-up tolerance
RARY(4)	specifies the final clamp equation tolerance value
RARY(5)	specifies the initial clamp equation tolerance value
RARY(6)	specifies the clamp equation decay factor
RARY(7)	specifies the cost coefficient for the active power loss objective
RARY(8)	specifies the cost coefficient for the reactive power loss objective
RARY(9)	specifies the penalty value for power flow controls
RARY(10)	specifies the cost coefficient for the interface flow objective
RARY(11)	specifies the cost coefficient for the reactive generation reserve objective
RARY(12)	specifies the penalty value for fixed voltage
RARY(13)	specifies the lambda (Lagrange multiplier) blow-up tolerance
RARY(14)	specifies the minimum amount to open up tap settings by
RARY(15)	specifies the maximum amount to open up tap settings by
RARY(16)	specifies the amount to shift the interior point by during solution
RARY(17)	specifies the step length taken for the barrier method
RARY(18)	specifies the tolerance value used to determine which bus shunts get added to the Read/Change File as switched shunts (if an RDCH file is produced according to IARY(28) and the minimize added vars objective is employed according to IARY(7)). (Not currently functional)
RARY(19)	specifies the solution convergence tolerance, in MVA (corresponds to TOLN of the power flow solution parameters)

Character*260 OPFILE name of the OPF Optimization Log File that will be produced if IARY(26) is set to 1. It is a good idea to specify the filename in quotes (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 RCFILE	name of the PTI Read Change Data File that will be produced if IARY(28) is set to 1. It is a good idea to specify the filename in quotes. (Not currently functional) (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid IARY value
	IERR = 4	invalid RARY value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.46. PRODUCE_OPF_LOG_FILE

Use this API to set or return the value of the OPF solution option to produce an OPF solution log file, and if so, the name of the log file. This API can be used in conjunction with the ADD_DETAILS_TO_OPF_LOG API if additional details of the Lagrange multipliers are desired.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PRODUCE_OPF_LOG_FILE   IVAL  OFILE
```

Python command syntax:

```
ierr = produce_opf_log_file(ival, ofile)

ierr, ival,ofile = produce_opf_log_file()
```

Fortran command syntax:

```
CALL PRODUCE_OPF_LOG_FILE(IVAL, IOCODE, OFILE, IERR)
```

Where:

Integer IVAL	Indicates whether to produce an OPF Optimization Log File (input; if Batch command or for Python if specified as an argument, or for Fortran if IOCODE is 0; else 0 by default for input).	
	IVAL = 0	do not produce an OPF solution log file.
	IVAL = 1	produce an OPF solution log file.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Character*260 OFILE	OPF Optimization Log output filename (input; if Batch command or for Python if specified as an argument, or for FORTRAN if IOCODE is 0; else DETAIL.DAT by default for input).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.47. P_LOSSES_COST_COEFF

Use this API to set or return the value of the coefficient for the active power loss objective (OPF Solution Options). This API works in conjunction with the MINIMIZE_P_LOSSES objective function option.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_P_LOSSES_COST_COEFF    RVAL
```

Python command syntax:

```
ierr =  p_losses_cost_coeff(rval)

ierr, rval = p_losses_cost_coeff()
```

Fortran command syntax:

```
CALL P_LOSSES_COST_COEFF(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the active power loss cost coefficient (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.48. Q_LOSSES_COST_COEFF

Use this API to set or return the value of the coefficient for the reactive power loss objective. This API works in conjunction with the API option to MINIMIZE_Q_LOSSES.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_Q_LOSSES_COST_COEFF    RVAL
```

Python command syntax:

```
ierr =  q_losses_cost_coeff(rval)

ierr, rval = q_losses_cost_coeff()
```

Fortran command syntax:

```
CALL Q_LOSSES_COST_COEFF(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the reactive power loss cost coefficient (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.49. REACTIVE_RESV_COST_COEFF

Use this API to set or return the value of the coefficient for the reactive reserve objective. This API works in conjunction with the API option to MINIMIZE_REACTIVE_RESERVE.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_REACTIVE_RESV_COST_COEFF    RVAL
```

Python command syntax:

```
ierr =  reactive_resv_cost_coeff(rval)

ierr, rval = reactive_resv_cost_coeff()
```

Fortran command syntax:

```
CALL REACTIVE_RESV_COST_COEFF(RVAL, IOCODE, IERR)
```

Where:

Real RVAL	value of the reactive reserve cost coefficient (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in RVAL.
	IOCODE = 1	return the current value of the option setting in RVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.50. ROPF

Use this API to read an Optimal Power Flow Raw Data File.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ROPF   RFILE
```

Python command syntax:

```
ierr = ropf(rfile)
```

Fortran command syntax:

```
CALL ROPFNW(RFILE, IERR)
```

Where:

Character*260 RFILE	Optimal Power Flow Data File name, or '1' if is coming directly from the terminal or Response file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	error opening file
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.51. RWOP

Use this API to replicate the OPF data contained in the working case in the form of an Optimal Power Flow Data File.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RWOP SID ALL STATUS(1)..STATUS(4) OUT OFILE
```

Python command syntax:

```
ierr = rwop(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL RWOPNW(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid bus subsystem ID from 0 to 11. A bus subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified buses subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer STATUS (4)	array of 4 elements specifying Specifies whether to use various control options. A value of 0 means the option is not enabled. A value of 1 means the option is enabled.(input).	
	STATUS(1)	include isolated buses
	STATUS(2)	include out-of-service branches
	STATUS(3)	include subsystem data
	STATUS(4)	include subsystem tie lines
Integer OUT	option selection (input; no default allowed).	
	OUT = 0	direct output to a data file specified by OFILE.
	OUT = 1	direct output to the current report device.
Character*260 OFILE	output filename (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS(n) value
	IERR = 4	invalid OUT value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.52. SET_OPF_REPORT_SUBSYSTEM

Use this API to define the subsystem to be used when producing the opf output report. The records contained in the report will be limited to just the equipment residing within the defined subsystem. This includes bounded and infeasible reactive power generation, regulated buses, transformer tap ratios, transformer phase shift angles and bounded switched shunts.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SET_OPF_REPORT_SUBSYSTEM  OSID ALL
```

Python command syntax:

```
ierr = set_opf_report_subsystem(osid, all)
```

Fortran command syntax:

```
CALL SET_OPF_REPORT_SUBSYSTEM(OSID, ALL, IERR)
```

Where:

Integer OSID	Indicates the previously defined subsystem number to be used for the opf report (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.53. USE_DUAL_CRITERIA

Use this API to set or return the value of the OPF solution option to apply the power flow mismatch tolerance to the dual variable problem.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_USE_DUAL_CRITERIA    IVAL
```

Python command syntax:

```
ierr = use_dual_criteria(ival)

ierr, ival = use_dual_criteria()
```

Fortran command syntax:

```
CALL USE_DUAL_CRITERIA(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to apply the power flow mismatch tolerance to the dual variable problem (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	apply tolerance to the mismatch equation only.
	IVAL = 1	apply tolerance to the entire equation.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.54. USE_EMERGENCY_FLOW_LIMITS

Use this API to set or return the value of the OPF solution option to impose the emergency flow limits instead of the normal flow limits.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_USE_EMERGENCY_FLOW_LIMITS   IVAL
```

Python command syntax:

```
ierr =  use_emergency_flow_limits(ival)

ierr, ival = use_emergency_flow_limits()
```

Fortran command syntax:

```
CALL USE_EMERGENCY_FLOW_LIMITS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to use the emergency flow limits instead of the normal bus voltage limits (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not impose the emergency flow limits (normal flow limits will be imposed).
	IVAL = 1	impose emergency flow limits.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.55. USE_EMERGENCY_VOLT_LIMITS

Use this API to set or return the value of the OPF solution option to impose the emergency bus voltage limits instead of the normal bus voltage limits.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_USE_EMERGENCY_VOLT_LIMITS   IVAL
```

Python command syntax:

```
ierr =  use_emergency_volt_limits(ival)

ierr, ival = use_emergency_volt_limits()
```

Fortran command syntax:

```
CALL USE_EMERGENCY_VOLT_LIMITS(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	Indicates whether to use the emergency bus voltage limits instead of the normal bus voltage limits (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, then input of 0 by default; else output).	
	IVAL = 0	do not impose the emergency bus voltage limits (normal voltage limits will be imposed).
	IVAL = 1	impose emergency bus voltage limits.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.1.56. WRITE_OPF_OPTIONS_FILE

Use this API to save the PSSE OPF solution option settings to the PSSOPF.OPT file.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_WRITE_OPF_OPTIONS_FILE
```

Python command syntax:

```
ierr =  write_opf_options_file()
```

Fortran command syntax:

```
CALL WRITE_OPF_OPTIONS_FILE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Optimal Power Flow Operation APIs contains API routines corresponding to the operational functions of the PSS®E OPF.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2. Optimal Power Flow Data

3.2.1. NEWOPF

Use this API to initialize the working case with default OPF data for all bus oriented data records. It must be called prior to introducing any OPF data records. It has no effect if OPF data already exists in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_NEWOPF
```

Python command syntax:

```
ierr = newopf()
```

Fortran command syntax:

```
CALL NEWOPF_API(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.2. OPF_ADJBRX_INDV

Use this API to add an individual adjustable branch reactance data record to the working case. It may also be used to modify an existing adjustable branch reactance data record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_ADJBRX_INDV   EIBUS  EJBUS  CKTID  INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = opf_adjbrx_indv(eibus, ejbus, cktid, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_ADJBRX_INDV(EIBUS, EJBUS, CKTID, INTGAR, REALAR, IERR)
```

Where:

Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	cost curve type. This value is not currently used by the program
	INTGAR(1) = 1	polynomial and exponential cost curve
	INTGAR(1) = 2	linear cost curve
	INTGAR(1) = 3	quadratic cost curve
	INTGAR(2)	adjustable branch reactance status (default of 1 if new record, current value otherwise)
	INTGAR(2) = 0	out of service
	INTGAR(2) = 1	in service
	INTGAR(3)	cost table cross reference number. This value is not currently used by the program
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	reactance multiplier in pu (default value of 1.0 if new record, current value otherwise)
	REALAR(2)	maximum reactance multiplier in pu (default value of 1.0 if new record, current value otherwise)
	REALAR(3)	minimum reactance multiplier in pu (default value of 1.0 if new record, current value otherwise)
	REALAR(4)	cost scale coefficient in $/pu (default value of 1.0 if new record, current value otherwise)
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = -1	non-fatal error occurred. Warning messages are as follows: - Invalid cost curve type specified - Invalid branch reactance status specified. - Maximum branch reactance multiplier less than minimum multiplier - Branch reactance multiplier cannot be less than 0.1. - Branch reactance multiplier cannot be set to zero
IERR = 0	no error occurred
IERR = 1	invalid bus number
IERR = 2	specified branch does not exist
IERR = 3	branch reactance table full
IERR = 4	specified branch cannot be a transformer
IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.3. OPF_ADJBRX_SUBSYS

Use this API to add or modify all OPF branch reactance data records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_ADJBRX_SUBSYS   SID  ALL  INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(4)  USEVAL(1)..USEVAL(4)
```

Python command syntax:

```
ierr = opf_adjbrx_subsys(sid, all, intgar, realar, useval)
```

Fortran command syntax:

```
CALL OPF_ADJBRX_SUBSYS(SID, ALL, INTGAR, REALAR, USEVAL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	cost curve type. This value is not currently used by the program
	INTGAR(1) = 1	polynomial and exponential cost curve
	INTGAR(1) = 2	linear cost curve
	INTGAR(1) = 3	quadratic cost curve
	INTGAR(2)	adjustable branch reactance status (default value of 1 for new records, current values otherwise)
	INTGAR(2) = 0	out of service
	INTGAR(2) = 1	in service
	INTGAR(3)	cost table cross reference number. This value is not currently used by the program
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	reactance multiplier in pu (default value of 1.0 for new records, current value otherwise)
	REALAR(2)	maximum reactance multiplier in pu (default value of 1.0 for new records, current value otherwise)
	REALAR(3)	minimum reactance multiplier in pu (default value of 1.0 for new records, current value otherwise)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(4)	cost scale coefficient in $/pu (default value of 1.0 for new records, current value otherwise)
Integer USEVAL (4)	array of 4 elements specifying (default treatment same as value of 0) (input).	
	USEVAL(1)	indicates how the value of REALAR(1) should be used
	USEVAL(1) = 0	if the value of REALAR(1) is to be used as defined
	USEVAL(1) = 1	if the value of REALAR(1) is to be multiplied by the present value of the branch reactance multiplier
	USEVAL(1) = 2	if the value of REALAR(1) is to be added to the present value of the branch reactance multiplier
	USEVAL(2)	indicates how the value of REALAR(2) should be used
	USEVAL(2) = 0	if the value of REALAR(2) is to be used as defined
	USEVAL(2) = 1	if the value of REALAR(2) is to be multiplied by the present value of the maximum branch reactance multiplier
	USEVAL(2) = 2	if the value of REALAR(2) is to be added to the present value of the maximum branch reactance multiplier
	USEVAL(3)	indicates how the value of REALAR(3) should be used
	USEVAL(3) = 0	if the value of REALAR(3) is to be used as defined
	USEVAL(3) = 1	if the value of REALAR(3) is to be multiplied by the present value of the minimum branch reactance multiplier
	USEVAL(3) = 2	if the value of REALAR(3) is to be added to the present value of the minimum branch reactance multiplier
	USEVAL(4)	indicates how the value of REALAR(4) should be used
	USEVAL(4) = 0	if the value of REALAR(4) is to be used as defined
	USEVAL(4) = 1	if the value of REALAR(4) is to be multiplied by the present value of the cost scale coefficient
	USEVAL(4) = 2	if the value of REALAR(4) is to be added to the present value of the cost scale coefficient
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Warning messages are as follows: - Invalid cost curve type spec-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ified - Invalid branch reactance status specified. - Maximum branch reactance multiplier less than minimum multiplier - Branch reactance multiplier cannot be less than 0.1. - Branch reactance multiplier cannot be set to zero
IERR = 0	no error occurred
IERR = 1	invalid SID value or subsystem SID is not defined
IERR = 2	invalid ALL value
IERR = 3	adjustable branch reactance table is full
IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.4. OPF_ADJLOAD_TBL

Use this API to add an individual adjustable bus load table record to the working case. It may also be used to modify an existing adjustable bus load table record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_ADJLOAD_TBL   TBL  INTGAR(1)..INTGAR(3)   REALAR(1)..REALAR(7)
```

Python command syntax:

```
ierr = opf_adjload_tbl(tbl, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_ADJLOAD_TBL(TBL, INTGAR, REALAR, IERR)
```

Where:

Integer TBL	adjustable bus load table number (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	cost curve type. This value is not currently used by the program
	INTGAR(1) = 1	polynomial and exponential cost curve
	INTGAR(1) = 2	linear cost curve
	INTGAR(1) = 3	quadratic cost curve
	INTGAR(2)	adjustable bus load status (default of 1 if new record, current value otherwise)
	INTGAR(2) = 0	out of service
	INTGAR(2) = 1	in service
	INTGAR(3)	cost table cross reference number. This value is not currently used by the program
Real REALAR (7)	array of 7 elements specifying (input).	
	REALAR(1)	load multiplier (default value of 1.0 if new record, current value otherwise)
	REALAR(2)	maximum load multiplier (default value of 1.0 if new record, current value otherwise)
	REALAR(3)	minimum load multiplier (default value of 1.0 if new record, current value otherwise)
	REALAR(4)	load ratio multiplier. This value is not currently used by the program
	REALAR(5)	maximum load ratio multiplier. This value is not currently used by the program
	REALAR(6)	minimum load ratio multiplier. This value is not currently used by the program
	REALAR(7)	load cost scale coefficient (default value of 1.0 if new record, current value otherwise)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Warning messages are as follows: - invalid cost curve type specified - invalid load table status specified - Maximum load multiplier less than 0.0 - Minimum load multiplier less than 0.0 - maximum load multiplier less than minimum load multiplier - maximum load ratio multiplier less than minimum load ratio multiplier
	IERR = 0	no error occurred
	IERR = 1	adjustable bus load table is full
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.5. OPF_ADJVAR_INDV

This API routine is obsolete. It has been replaced by the API routine opf_adjvar_indv_2 .

Use this API to add an individual OPF adjustable bus shunt data record to the working case. It may also be used to modify an existing adjustable bus shunt data record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_ADJVAR_INDV   EIBUS  SHNTID  INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = opf_adjvar_indv(eibus, shntid, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_ADJVAR_INDV(EIBUS, SHNTID, INTGAR, REALAR, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 SHNTID	bus shunt identifier (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying The value of each is as follows(input).	
	INTGAR(1)	cost curve type. This value is not currently used by the program
	INTGAR(1) = 1	polynomial and exponential cost curve
	INTGAR(1) = 2	linear cost curve
	INTGAR(1) = 3	quadratic cost curve
	INTGAR(2)	status (default value of 1 if new record, current value otherwise)
	INTGAR(2) = 0	out of service
	INTGAR(2) = 1	in service
	INTGAR(3)	cost table cross reference number. This value is not currently used by the program
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	initial bus shunt susceptance in Mvar at nominal voltage (default value of 0.0 if new record, current value otherwise)
	REALAR(2)	maximum bus shunt susceptance in Mvar (default value of 0.0 if new record, current value otherwise)
	REALAR(3)	minimum bus shunt susceptance in Mvar (default value of 0.0 if new record, current value otherwise)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(4)	cost scale coefficient in $/Mvar (default value of 1.0 if new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Possible warning messages are as follows: - Invalid cost curve type specified. - Invalid adjustable bus shunt status specified. - Maximum shunt susceptance less than minimum shunt susceptance
	IERR = 0	no error occurred
	IERR = 1	adjustable bus shunt identifier is more than two characters
	IERR = 2	invalid bus number specified
	IERR = 3	blank adjustable bus shunt identifier is invalid
	IERR = 4	adjustable bus shunt table is full
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.6. OPF_ADJVAR_INDV_2

Use this API to add an individual OPF adjustable bus shunt data record to the working case. It may also be used to modify an existing adjustable bus shunt data record.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_OPF_ADJVAR_INDV_2 EIBUS ENOD SHNTID INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = opf_adjvar_indv_2(eibus, enod, shntid, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_ADJVAR_INDV_2(EIBUS, ENOD, SHNTID, INTGAR, REALAR, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Integer ENOD	Bus section number of the node number of one of the nodes in the bus section of interest. 0 if bus is represented by a single bus section. If 0 is specified and bus is represented by multiple bus sections, the bus data of the parent bus and not that of one of the topology bus sections will be changed. (input; no default allowed).	
Character*2 SHNTID	bus shunt identifier (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying The value of each is as follows(input).	
	INTGAR(1)	cost curve type. This value is not currently used by the program
	INTGAR(1) = 1	polynomial and exponential cost curve
	INTGAR(1) = 2	linear cost curve
	INTGAR(1) = 3	quadratic cost curve
	INTGAR(2)	status (default value of 1 if new record, current value otherwise)
	INTGAR(2) = 0	out of service
	INTGAR(2) = 1	in service
	INTGAR(3)	cost table cross reference number. This value is not currently used by the program
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	initial bus shunt susceptance in Mvar at nominal voltage (default value of 0.0 if new record, current value otherwise)
	REALAR(2)	maximum bus shunt susceptance in Mvar (default value of 0.0 if new record, current value otherwise)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(3)	minimum bus shunt susceptance in Mvar (default value of 0.0 if new record, current value otherwise)
	REALAR(4)	cost scale coefficient in $/Mvar (default value of 1.0 if new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = -1	warnings - one or more of: - Invalid cost curve type specified. - Invalid adjustable bus shunt status specified. - Maximum shunt susceptance less than minimum shunt susceptance, values have been switched
	IERR = 0	no error occurred
	IERR = 1	adjustable bus shunt identifier is more than two characters
	IERR = 2	invalid bus number
	IERR = 3	blank adjustable bus shunt identifier is invalid
	IERR = 4	adjustable bus shunt table is full
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	bus not found
	IERR = 7	invalid node number
	IERR = 8	bus does not belong to a substation
	IERR = 9	node is not found in the substation
	IERR = 10	node is not connected to any bus section of bus
	IERR = 11	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.7. OPF_ADJVAR_SUBSYS

Use this API to add or modify all OPF adjustable bus shunt records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_ADJVAR_SUBSYS   SID  ALL  AUTOADD  SHNTID   INTGAR(1)..INTGAR(4)
 REALAR(1)..REALAR(4)   USEVAL(1)..USEVAL(4)
```

Python command syntax:

```
ierr = opf_adjvar_subsys(sid, all, autoadd, shntid, intgar, realar,
 useval)
```

Fortran command syntax:

```
CALL OPF_ADJVAR_SUBSYS(SID, ALL, AUTOADD, SHNTID, INTGAR, REALAR, USEVAL,
 IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer AUTOADD	Indicates whether new records can be added (default treatment same as value of 3) (input).	
	AUTOADD = 0	do not add a new var record if it does not already exist.
	AUTOADD = 1	add a new var record if it does not already exist.
	AUTOADD = 2	add a separate shunt record at the bus.
	AUTOADD = 3	only modify existing bus shunt records that correspond to the shunt identifier given in SHNTID.
Character*2 SHNTID	adjustable bus shunt identifier (input; '1' by default).	
Integer INTGAR (4)	array of 4 elements specifying (input).	
	INTGAR(1)	cost curve type. This value is not currently used by the program
	INTGAR(1) = 1	polynomial and exponential cost curve
	INTGAR(1) = 2	piece-wise linear cost curve
	INTGAR(1) = 3	piece-wise quadratic cost curve
	INTGAR(2)	status (default value of 1 for new records, current value otherwise)
	INTGAR(2) = 0	out of service
	INTGAR(2) = 1	in service

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(3)	cost table cross reference number. This value is not currently used by the program
	INTGAR(4)	indicates which bus types should be processed (default treatment same as value of 0)
	INTGAR(4) = 0	process all buses in subsystem
	INTGAR(4) = 1	process only Type 1 buses in subsystem
	INTGAR(4) = 2	process only Type 2 and 3 buses in subsystem
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	bus shunt susceptance in Mvar at nominal voltage (default value of 0.0 for new records, current values used for existing ones)
	REALAR(2)	maximum bus shunt susceptance in Mvar (default value of 0.0 for new records, current value otherwise)
	REALAR(3)	minimum bus shunt susceptance in Mvar (default value of 0.0 for new records, current value otherwise)
	REALAR(4)	cost scale coefficient in $/Mvar (default value of 1.0 for new records, current value otherwise)
Integer USEVAL (4)	array of 4 elements specifying Used to indicate actions to be taken on the REALAR values. If USEVAL is 1 then value REALAR value is multiplied by the current value. If USEVAL is 2 then REALAR value is added to current real value. If USEVAL is 0 (default) then REALAR is used as provided.(input).	
	USEVAL(1)	indicates how the value of REALAR(1) should be adjusted
	USEVAL(1) = 0	if the value of REALAR(1) is to be used as defined
	USEVAL(1) = 1	if the value of REALAR(1) is to be multiplied by the present value of the bus shunt susceptance
	USEVAL(1) = 2	if the value of REALAR(1) is to be added to the present value of the bus shunt susceptance
	USEVAL(2)	indicates how the value of REALAR(2) should be used
	USEVAL(2) = 0	if the value of REALAR(2) is to be used as defined
	USEVAL(2) = 1	if the value of REALAR(2) is to be multiplied by the present value of the maximum bus shunt susceptance
	USEVAL(2) = 2	if the value of REALAR(2) is to be added to the present value of the maximum bus shunt susceptance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	USEVAL(3)	indicates how the value of REALAR(3) should be used
	USEVAL(3) = 0	if the value of REALAR(3) is to be used as defined
	USEVAL(3) = 1	if the value of REALAR(3) is to be multiplied by the present value of the minimum bus shunt susceptance
	USEVAL(3) = 2	if the value of REALAR(3) is to be added to the present value of the minimum bus shunt susceptance
	USEVAL(4)	indicates how the value of REALAR(4) should be used
	USEVAL(4) = 0	if the value of REALAR(4) is to be used as defined
	USEVAL(4) = 1	if the value of REALAR(4) is to be multiplied by the present value of the cost scale coefficient
	USEVAL(4) = 2	if the value of REALAR(4) is to be added to the present value of the cost scale coefficient
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	adjustable bus shunt identifier is more than two characters
	IERR = 2	invalid SID value or subsystem SID is not defined
	IERR = 3	invalid ALL value
	IERR = 4	adjustable bus shunt table is full
	IERR = 5	node-breaker terminal connection table is full
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid cost curve type specified. - Invalid adjustable bus shunt status specified. - Maximum shunt susceptance less than minimum shunt susceptance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.8. OPF_APDSP_TBL

Use this API to add an individual active power dispatch table record to the working case. It may also be used to modify an existing active power dispatch table record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_APDSP_TBL   TBL  INTGAR(1)..INTGAR(3)   REALAR(1)..REALAR(3)
```

Python command syntax:

```
ierr = opf_apdsp_tbl(tbl, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_APDSP_TBL(TBL, INTGAR, REALAR, IERR)
```

Where:

Integer TBL	active power dispatch table number (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	cost curve type (default value of 1 if new record, current value otherwise)
	INTGAR(1) = 1	polynomial and exponential cost curve
	INTGAR(1) = 2	linear cost curve
	INTGAR(1) = 3	quadratic cost curve
	INTGAR(2)	active power dispatch table status (default value of 1 if new record, current value otherwise)
	INTGAR(2) = 0	out of service
	INTGAR(2) = 1	in service
	INTGAR(3)	cost table cross reference number (default value of 0, current value otherwise)
Real REALAR (3)	array of 3 elements specifying (input).	
	REALAR(1)	maximum active power generation, in MW (default value of 9999.0, current value otherwise)
	REALAR(2)	minimum active power generation, in MW (default value of -9999.0, current value otherwise)
	REALAR(3)	fuel cost scale coefficient (default value of 1.0, current value otherwise)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Warning messages are as follows: - invalid cost curve type specified - invalid active power dispatch table status specified - maximum active power gener-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ation less than minimum active power generation
IERR = 0	no error occurred
IERR = 1	active power dispatch table is full
IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.9. OPF_BRFLW_3WT_INDV

Use this API to add a flow constraint record for an individual winding of a three-winding transformer in the working case. It may also be used to modify an existing three-winding transformer flow constraint record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_BRFLW_3WT_INDV   EIBUS  EJBUS  EKBUS  CKTID  BFLWID  INTGAR(1)
 INTGAR(2)  REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = opf_brflw_3wt_indv(eibus, ejbus, ekbus, cktid, bflwid, intgar,
 realar)
```

Fortran command syntax:

```
CALL OPF_BRFLW_3WT_INDV(EIBUS, EJBUS, EKBUS, CKTID, BFLWID, INTGAR,
 REALAR, IERR)
```

Where:

Integer EIBUS	from bus number The value of EIBUS indicates the from bus of the winding for which the flow constraint is being introduced (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Integer EKBUS	third bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Character*1 BFLWID	branch flow constraint identifier (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	branch flow type (default value of 4 if new record, current value otherwise)
	INTGAR(1) = 1	MW
	INTGAR(1) = 2	Mvar
	INTGAR(1) = 3	MVA
	INTGAR(1) = 4	Ampere
	INTGAR(2)	limit type imposed on the branch flow limits (default value of 1 if new record, current value otherwise)
	INTGAR(2) = 0	reporting only
	INTGAR(2) = 1	hard limit
	INTGAR(2) = 2	linear soft limit
	INTGAR(2) = 3	quadratic soft limit
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	maximum normal flow limit in physical units (default value of 0.0 for new record, current value otherwise)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(2)	minimum normal flow limit in physical units (default value of 0.0 for new record, current value otherwise)
	REALAR(3)	maximum emergency flow limit in physical units (default value of 0.0 for new record, current value otherwise)
	REALAR(4)	minimum emergency flow limit in physical units (default value of 0.0 for new record, current value otherwise)
	REALAR(5)	soft limit penalty weight (default value of 1.0 for new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Possible warning messages are: - Invalid branch flow type specified. - Invalid branch flow limit type specified. - Maximum normal flow limit is less than the minimum normal flow limit. - Maximum emergency flow limit is less than the minimum emergency flow limit
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified three-winding transformer does not exist
	IERR = 3	specified winding of three-winding transformer not found
	IERR = 4	branch flow constraint table is full
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.10. OPF_BRFLW_BRN_INDV

Use this API to add an individual branch or two-winding transformer flow constraint record to the working case. It may also be used to modify an existing branch or two-winding transformer flow constraint.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_BRFLW_BRN_INDV   EIBUS  EJBUS  CKTID  BFLWID  INTGAR(1)  INTGAR(2)
  REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = opf_brflw_brn_indv(eibus, ejbus, cktid, bflwid, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_BRFLW_BRN_INDV(EIBUS, EJBUS, CKTID, BFLWID, INTGAR, REALAR, IERR)
```

Where:

Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Character*1 BFLWID	branch flow constraint identifier (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	branch flow type (default value of 4 if new record, current value otherwise)
	INTGAR(1) = 1	MW
	INTGAR(1) = 2	Mvar
	INTGAR(1) = 3	MVA
	INTGAR(1) = 4	Ampere
	INTGAR(2)	limit type imposed on the branch flow limits (default value of 1 if new record, current value otherwise)
	INTGAR(2) = 0	reporting only
	INTGAR(2) = 1	hard limit
	INTGAR(2) = 2	linear soft limit
	INTGAR(2) = 3	quadratic soft limit
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	maximum normal flow limit in physical units (default value of 0.0 if new record, current value otherwise)
	REALAR(2)	minimum normal flow limit in physical units (default value of 0.0 if new record, current value otherwise)
	REALAR(3)	maximum emergency flow limit in physical units (default value of 0.0 if new record, current value otherwise)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(4)	minimum emergency flow limit in physical units (default value of 0.0 if new record, current value otherwise)
	REALAR(5)	soft limit penalty weight (default value of 1.0 if new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Possible warning messages are: - Invalid branch flow type specified. - Invalid branch flow limit type specified. - Maximum normal flow limit is less than the minimum normal flow limit. - Maximum emergency flow limit is less than the minimum emergency flow limit
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified branch does not exist
	IERR = 3	branch flow constraint table is full
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.11. OPF_BRFLW_SUBSYS

Use this API to add or modify all branch flow data records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_BRFLW_SUBSYS   SID  ALL  AUTOADD  BFLWID  INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(5)  USEVAL(1)..USEVAL(5)
```

Python command syntax:

```
ierr = opf_brflw_subsys(sid, all, autoadd, bflwid, intgar, realar, useval)
```

Fortran command syntax:

```
CALL OPF_BRFLW_SUBSYS(SID, ALL, AUTOADD, BFLWID, INTGAR, REALAR, USEVAL,
 IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer AUTOADD	Indicates whether new records can be added, or are to be modified (input; no default allowed).	
	AUTOADD = 0	do not add a new branch flow record if it does not already exist.
	AUTOADD = 1	add a new branch flow record if it does not already exist.
	AUTOADD = 2	only modify existing branch flow records that correspond to the flow identifier given in BFLWID.
Character*1 BFLWID	branch flow identifier (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	branch flow type (default value of 4 for new records, current value otherwise)
	INTGAR(1) = 1	MW
	INTGAR(1) = 2	Mvar
	INTGAR(1) = 3	MVA
	INTGAR(1) = 4	Ampere
	INTGAR(2)	limit type imposed on the branch flow limits (default value of 1 for new records, current value otherwise)
	INTGAR(2) = 0	reporting only
	INTGAR(2) = 1	hard limit
	INTGAR(2) = 2	linear soft limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(2) = 3	quadratic soft limit
	INTGAR(3)	default rate setting (default value of 0)
	INTGAR(3) = 0	use the limits that have been defined
	INTGAR(3) = 1	base flow limits on RATEA
	INTGAR(3) = 2	base flow limits on RATEB
	INTGAR(3) = 3	base flow limits on RATEC
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	maximum normal flow limit in physical units (default value of 0.0 for new records, current value otherwise)
	REALAR(2)	minimum normal flow limit in physical units (default value of 0.0 for new records, current value otherwise)
	REALAR(3)	maximum emergency flow limit in physical units (default value of 0.0 for new records, current value otherwise)
	REALAR(4)	minimum emergency flow limit in physical units (default value of 0.0 for new records, current value otherwise)
	REALAR(5)	soft limit penalty weight (default value of 1.0 for new records, current value otherwise)
Integer USEVAL (5)	array of 5 elements specifying Specifies whether the corresponding REALAR value should be applied as provided, or multiplied by the present value or added to the present value.(input).	
	USEVAL(1)	indicates how the value of REALAR(1) should be used
	USEVAL(1) = 0	if the value of REALAR(1) is to be used as defined
	USEVAL(1) = 1	if the value of REALAR(1) is to be multiplied by the present value of the maximum normal flow limit
	USEVAL(1) = 2	if the value of REALAR(1) is to be added to the present value of the maximum normal flow limit
	USEVAL(2)	indicates how the value of REALAR(2) should be used
	USEVAL(2) = 0	if the value of REALAR(2) is to be used as defined
	USEVAL(2) = 1	if the value of REALAR(2) is to be multiplied by the present value of the minimum normal flow limit
	USEVAL(2) = 2	if the value of REALAR(2) is to be added to the present value of the minimum normal flow limit
	USEVAL(3)	indicates how the value of REALAR(3) should be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	USEVAL(3) = 0	if the value of REALAR(3) is to be used as defined
	USEVAL(3) = 1	if the value of REALAR(3) is to be multiplied by the present value of the maximum emergency flow limit
	USEVAL(3) = 2	if the value of REALAR(3) is to be added to the present value of the maximum emergency flow limit
	USEVAL(4)	indicates how the value of REALAR(4) should be used
	USEVAL(4) = 0	if the value of REALAR(4) is to be used as defined
	USEVAL(4) = 1	if the value of REALAR(4) is to be multiplied by the present value of the minimum emergency flow limit
	USEVAL(4) = 2	if the value of REALAR(4) is to be added to the present value of the minimum emergency flow limit
	USEVAL(5)	indicates how the value of REALAR(5) should be used
	USEVAL(5) = 0	if the value of REALAR(5) is to be used as defined
	USEVAL(5) = 1	if the value of REALAR(5) is to be multiplied by the present value of the soft limit penalty weight
	USEVAL(5) = 2	if the value of REALAR(5) is to be added to the present value of the soft limit penalty weight
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Possible warning messages are: - Invalid flow type specified. - Invalid branch flow limit type specified. - Maximum normal flow limit is less than the minimum normal flow limit. - Maximum emergency flow limit is less than the minimum emergency flow limit
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	branch flow constraint table is full
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.12. OPF_BUS_INDV

This API routine is obsolete. It has been replaced by the API routine opf_bus_indv_2 .

Use this API to add OPF bus voltage magnitude data to the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_BUS_INDV   EIBUS  INTGAR(1)  INTGAR(2)  REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = opf_bus_indv(eibus, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_BUS_INDV(EIBUS, INTGAR, REALAR, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	limit type imposed on the bus voltage limits (default is the current value; initialized to 1)
	INTGAR(1) = 0	reporting only
	INTGAR(1) = 1	hard limit
	INTGAR(1) = 2	linear soft limit
	INTGAR(1) = 3	quadratic soft limit
	INTGAR(2)	indicates what to do with the defined voltage limits (default treatment same as value of 0)
	INTGAR(2) = 0	leave voltage limits as defined by REALAR(1) and REALAR(2)
	INTGAR(2) = 1	fix normal voltage limit at present voltage values, ignoring any values defined by REALAR(1) and REALAR(2)
	INTGAR(2) = 2	allow normal voltage limits to open up if they exceed the bounds defined by REALAR(1) or REALAR(2)
	INTGAR(2) = 3	set normal and emergency OPF voltage limits to the normal and emergency voltage limits as defined in the network bus data
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	normal maximum voltage limit in pu (default is the current value; initialized to 9999.0)
	REALAR(2)	normal minimum voltage limit in pu (default is the current value; initialized to -9999.0)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(3)	emergency maximum voltage limit in pu (default is the current value; initialized to 9999.0)
	REALAR(4)	emergency minimum voltage limit in pu (default is the current value; initialized to -9999.0)
	REALAR(5)	soft-limit penalty weight (default is the current value; initialized to 1.0)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Warning messages are as follows: - Invalid limit type specified. - Maximum normal voltage limit less than minimum limit. - Maximum emergency voltage limit less than minimum limit
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.13. OPF_BUS_INDV_2

Use this API to add OPF bus voltage magnitude data to the working case.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_OPF_BUS_INDV_2   EIBUS  ENOD  INTGAR(1)  INTGAR(2)
 REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = opf_bus_indv_2(eibus, enod, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_BUS_INDV_2(EIBUS, ENOD, INTGAR, REALAR, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Integer ENOD	The bus section number or the external node number of one of the nodes in the bus section of interest. Specify 0 if bus is represented by a single bus section. If 0 is specified and bus is represented by multiple bus sections, the data of the parent bus and not that of the bus sections will be changed. (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	limit type imposed on the bus voltage limits (default is the current value; initialized to 1)
	INTGAR(1) = 0	reporting only
	INTGAR(1) = 1	hard limit
	INTGAR(1) = 2	linear soft limit
	INTGAR(1) = 3	quadratic soft limit
	INTGAR(2)	indicates what to do with the defined voltage limits (default treatment same as value of 0)
	INTGAR(2) = 0	leave voltage limits as defined by REALAR(1) and REALAR(2)
	INTGAR(2) = 1	fix normal voltage limit at present voltage values, ignoring any values defined by REALAR(1) and REALAR(2)
	INTGAR(2) = 2	allow normal voltage limits to open up if they exceed the bounds defined by REALAR(1) or REALAR(2)
	INTGAR(2) = 3	set normal and emergency OPF voltage limits to the normal and emergency voltage limits as defined in the network bus data
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	normal maximum voltage limit in pu (default is the current value; initialized to 9999.0)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(2)	normal minimum voltage limit in pu (default is the current value; initialized to -9999.0)
	REALAR(3)	emergency maximum voltage limit in pu (default is the current value; initialized to 9999.0)
	REALAR(4)	emergency minimum voltage limit in pu (default is the current value; initialized to -9999.0)
	REALAR(5)	soft-limit penalty weight (default is the current value; initialized to 1.0)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Warning messages are as follows: - Invalid limit type specified. - Maximum normal voltage limit less than minimum limit. - Maximum emergency voltage limit less than minimum limit
	IERR = 0	no error occurred
	IERR = 1	invalid bus number
	IERR = 2	bus not found
	IERR = 3	invalid node number
	IERR = 4	node not found in substation of bus
	IERR = 5	node not connected to any bus section of bus in substation
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.14. OPF_BUS_SUBSYS

Use this API to add OPF bus attribute data for all records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_BUS_SUBSYS   SID  ALL  INTGAR(1)..INTGAR(3)  REALAR(1)..REALAR(5)
 USEVAL(1)..USEVAL(5)
```

Python command syntax:

```
ierr = opf_bus_subsys(sid, all, intgar, realar, useval)
```

Fortran command syntax:

```
CALL OPF_BUS_SUBSYS(SID, ALL, INTGAR, REALAR, USEVAL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses, use existing data values.
	ALL = 2	process all buses, use default data values.
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	limit type imposed on the bus voltage limits (default is the current value; initialized to 1)
	INTGAR(1) = 0	reporting only
	INTGAR(1) = 1	hard limit
	INTGAR(1) = 2	linear soft limit
	INTGAR(1) = 3	quadratic soft limit
	INTGAR(2)	indicates what to do with the defined voltage limits (default treatment same as value of 0)
	INTGAR(2) = 0	leave voltage limits as defined by REALAR(1) and REALAR(2)
	INTGAR(2) = 1	fix normal voltage limit at present voltage values, ignoring any values defined by REALAR(1) and REALAR(2)
	INTGAR(2) = 2	allow normal voltage limits to open up if they exceed the bounds defined by REALAR(1) or REALAR(2)
	INTGAR(2) = 3	set normal and emergency OPF voltage limits to the normal and emergency voltage limits as defined in the network bus data
	INTGAR(3)	indicates which bus types should be processed (default treatment same as value of 0)
	INTGAR(3) = 0	all buses in subsystem

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(3) = 1	only Type 1 buses in subsystem
	INTGAR(3) = 2	only Type 2 and 3 buses in subsystem
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	normal maximum voltage limit in pu (default is the current value; initialized to 9999.0)
	REALAR(2)	normal minimum voltage limit in pu (default is the current value; initialized to -9999.0)
	REALAR(3)	emergency maximum voltage limit in pu (default is the current value; initialized to 9999.0)
	REALAR(4)	emergency minimum voltage limit in pu (default is the current value; initialized to -9999.0)
	REALAR(5)	soft-limit penalty weight (default is the current value; initialized to 1.0)
Integer USEVAL (5)	array of 5 elements specifying (default treatment same as value of 0) (input).	
	USEVAL(1)	indicates how the value of REALAR(1) should be used
	USEVAL(1) = 0	if the value of REALAR(1) is to be used as defined
	USEVAL(1) = 1	if the value of REALAR(1) is to be multiplied by the present value of the normal bus voltage minimum limit
	USEVAL(1) = 2	if the value of REALAR(1) is to be added to the present value of the normal bus voltage minimum limit
	USEVAL(2)	indicates how the value of REALAR(2) should be used
	USEVAL(2) = 0	if the value of REALAR(2) is to be used as defined
	USEVAL(2) = 1	if the value of REALAR(2) is to be multiplied by the present value of the normal bus voltage maximum limit
	USEVAL(2) = 2	if the value of REALAR(2) is to be added to the present value of the normal bus voltage maximum limit
	USEVAL(3)	indicates how the value of REALAR(3) should be used
	USEVAL(3) = 0	if the value of REALAR(3) is to be used as defined
	USEVAL(3) = 1	if the value of REALAR(3) is to be multiplied by the present value of the emergency bus voltage minimum limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	USEVAL(3) = 2	if the value of REALAR(3) is to be added to the present value of the emergency bus voltage minimum limit
	USEVAL(4)	indicates how the value of REALAR(4) should be used
	USEVAL(4) = 0	if the value of REALAR(4) is to be used as defined
	USEVAL(4) = 1	if the value of REALAR(4) is to be multiplied by the present value of the emergency bus voltage maximum limit
	USEVAL(4) = 2	if the value of REALAR(4) is to be added to the present value of the emergency bus voltage maximum limit
	USEVAL(5)	indicates how the value of PENWGT should be used
	USEVAL(5) = 0	if the value of PENWGT is to be used as defined
	USEVAL(5) = 1	if the value of PENWGT is to be multiplied by the present value of the soft limit penalty weight
	USEVAL(5) = 2	if the value of PENWGT is to be added to the present value of the soft limit penalty weight
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	non-fatal error occurred. Warning messages are as follows: - Invalid limit type specified. - Maximum normal voltage limit less than minimum limit. - Maximum emergency voltage limit less than minimum limit

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.15. OPF_CHANGE_3WT_FLOW_ID

Use this API to change the identifier of a flow constraint record for an individual winding of a three-winding transformer in the working case.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Batch command syntax:

```
OPF_CHANGE_3WT_FLOW_ID   IBUS   JBUS   KBUS   CKTID   ID   IDNEW
```

Python command syntax:

```
ierr = opf_change_3wt_flow_id(ibus, jbus, kbus, cktid, id, idnew)
```

Fortran command syntax:

```
CALL OPF_CHANGE_3WT_FLOW_ID(IBUS, JBUS, KBUS, CKTID, ID, IDNEW, IERR)
```

Where:

Integer IBUS	from bus number The value of IBUS indicates the from bus of the winding for which the flow constraint is being introduced (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Character*1 ID	branch flow constraint identifier (input; no default allowed).	
Character*1 IDNEW	new branch flow constraint identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	a non-blank new identifier must be specified
	IERR = 3	specified three-winding transformer does not exist
	IERR = 4	branch flow identifier is more than one character
	IERR = 5	specified winding of three-winding transformer not found
	IERR = 6	specified OPF branch flow constraint does not exist
	IERR = 7	branch flow constraint already exists
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.16. OPF_CHANGE_ADJVAR_ID

This API routine is obsolete. It has been replaced by the API routine opf_change_adjvar_id .

Use this API to change the identifier of an OPF adjustable bus shunt data record in the working case.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Batch command syntax:

```
BAT_OPF_CHANGE_ADJVAR_ID   IBUS  ID  IDNEW
```

Python command syntax:

```
ierr = opf_change_adjvar_id(ibus, id, idnew)
```

Fortran command syntax:

```
CALL OPF_CHANGE_ADJVAR_ID(IBUS, ID, IDNEW, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	OPF adjustable bus shunt identifier (input; no default allowed).	
Character*2 IDNEW	new OPF adjustable bus shunt identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	blank adjustable bus shunt identifier is invalid
	IERR = 3	adjustable bus shunt identifier is more than two characters
	IERR = 4	adjustable bus shunt not found
	IERR = 5	adjustable bus shunt already exists
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.17. OPF_CHANGE_BRN_FLOW_ID

Use this API to change the identifier of a branch or two-winding transformer OPF flow constraint record in the working case.

This API routine was first introduced in release 34.8.0 and 35.2.0.

Batch command syntax:

```
OPF_CHANGE_BRN_FLOW_ID   IBUS   JBUS   CKTID   ID   IDNEW
```

Python command syntax:

```
ierr = opf_change_brn_flow_id(ibus, jbus, cktid, id, idnew)
```

Fortran command syntax:

```
CALL OPF_CHANGE_BRN_FLOW_ID(IBUS, JBUS, CKTID, ID, IDNEW, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).
Integer JBUS	to bus number (input; no default allowed).
Character*2 CKTID	circuit identifier (input; no default allowed).
Character*1 ID	branch flow constraint identifier (input; no default allowed).
Character*1 IDNEW	new branch flow constraint identifier (input; no default allowed).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid bus number specified
IERR = 2	a non-blank new identifier must be specified
IERR = 3	specified branch does not exist
IERR = 4	branch flow identifier is more than one character
IERR = 5	branch flow constraint does not exist
IERR = 6	branch flow constraint already exists
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.18. OPF_CSTTBL_LIN

Use this API to add a linear cost curve table record to the working case. It may also be used to modify an existing linear cost curve table record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_CSTTBL_LIN   TBL   LABL   NPRS
```

Python command syntax:

```
ierr = opf_csttbl_lin(tbl, labl, nprs, xy)
```

Fortran command syntax:

```
CALL OPF_CSTTBL_LIN(TBL, LABL, NPRS, XY, IERR)
```

Where:

Integer TBL	linear cost curve table number (input; no default allowed).	
Character*12 LABL	cost table description of, at most, 12 characters (input; ' ' by default).	
Integer NPRS	number of (Xi, Yi) coordinate pairs being specified (input; no default allowed).	
Real XY (2*NPRS)	Coordinate pairs from 1 to NPRS. Xi is the control variable value. Yi is the total cost or energy consumption value. (input; Xi and Yi have defaults of 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	cost table data table is full
	IERR = 2	exceeded maximum number of allowable cost curve data pairs
	IERR = 3	number of data pairs must not be less than zero
	IERR = 4	incremental costs are not monotonically increasing
	IERR = 5	MW values are not monotonically increasing
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.19. OPF_CSTTBL_POLY

Use this API to add a polynomial and exponential cost curve table record to the working case. It may also be used to modify an existing polynomial and exponential cost curve table record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_CSTTBL_POLY   TBL   LABL   COST   LINCST   QUACST   EXPCST   EXPON
```

Python command syntax:

```
ierr = opf_csttbl_poly(tbl, labl, cost, lincst, quacst, expcst, expon)
```

Fortran command syntax:

```
CALL OPF_CSTTBL_POLY(TBL, LABL, COST, LINCST, QUACST, EXPCST, EXPON, IERR)
```

Where:

Integer TBL	polynomial and exponential cost curve table number (input; no default allowed).	
Character*12 LABL	cost table description of, at most, 12 characters (input; ' ' by default).	
Real COST	cost integration constant (input; 0.0).	
Real LINCST	linear cost coefficient value (input; 1.0).	
Real QUACST	quadratic cost coefficient value (input; 1.0).	
Real EXPCST	exponential cost coefficient value (input; 0.0).	
Real EXPON	exponent of the exponential cost coefficient (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	cost table data table full
	IERR = 2	exceeded maximum number of allowable cost curve data values
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.20. OPF_CSTTBL_QUAD

Use this API to add a quadratic cost curve table record to the working case. It may also be used to modify an existing quadratic cost curve table record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_CSTTBL_QUAD   TBL   LABL   COST   NPRS
```

Python command syntax:

```
ierr = opf_csttbl_quad(tbl, labl, cost, nprs, xy)
```

Fortran command syntax:

```
CALL OPF_CSTTBL_QUAD(TBL, LABL, COST, NPRS, XY, IERR)
```

Where:

Integer TBL	quadratic cost curve table number (input; no default allowed).	
Character*12 LABL	cost table description of, at most, 12 characters (input; ' ' by default).	
Real COST	cost integration constant (input; 0.0).	
Integer NPRS	number of (Xi, Yi) coordinate pairs to be specified (no default) (input).	
Real XY (2*NPRS)	Coordinate pairs from 1 to NPRS. Xi is the control variable value. Yi is the total cost or energy consumption value. (input; Xi and Yi have defaults of 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	cost table data table is full
	IERR = 2	exceeded maximum number of allowable cost curve data pairs
	IERR = 3	number of data pairs must not be negative
	IERR = 4	incremental costs are not monotonically increasing
	IERR = 5	MW values are not monotonically increasing
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.21. OPF_GENDSP_INDV

Use this API to add generator dispatch data to an individual record in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_GENDSP_INDV   EIBUS  GENID  APDTBL  DSPTCH
```

Python command syntax:

```
ierr = opf_gendsp_indv(eibus, genid, apdtbl, dsptch)
```

Fortran command syntax:

```
CALL OPF_GENDSP_INDV(EIBUS, GENID, APDTBL, DSPTCH, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Integer APDTBL	active power dispatch table cross reference number (input; default of 0 if new record, current value otherwise).	
Real DSPTCH	dispatch fraction (default value of 1.0 if new record, current value otherwise) (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified machine does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.22. OPF_GENDSP_SUBSYS

Use this API to add OPF generator dispatch data to all records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_GENDSP_SUBSYS   SID  ALL  APDTBL  DSPTCH  USEVAL
```

Python command syntax:

```
ierr = opf_gendsp_subsys(sid, all, apdtbl, dsptch, useval)
```

Fortran command syntax:

```
CALL OPF_GENDSP_SUBSYS(SID, ALL, APDTBL, DSPTCH, USEVAL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses, use existing data values.
	ALL = 2	process all buses, use default data values.
Integer APDTBL	active power dispatch table cross reference number (input; default of 0 for new record, current value otherwise).	
Real DSPTCH	dispatch fraction (input; default of 1.0 for new record, current value otherwise).	
Integer USEVAL	Indicates how the value of DSPTCH should be applied (default treatment same as value of 0) (input).	
	useval = 0	if the value of DSPTCH is to be used as defined.
	useval = 1	if the value of DSPTCH is to be multiplied by the present value of the dispatch fraction.
	useval = 2	if the value of DSPTCH is to be added to the present value of the dispatch fraction.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.23. OPF_GENRSV_INDV

Use this API to add an individual generator reserve data record to the working case. It may also be used to modify an existing generator reserve data record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_GENRSV_INDV   EIBUS  GENID  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = opf_genrsv_indv(eibus, genid, realar)
```

Fortran command syntax:

```
CALL OPF_GENRSV_INDV(EIBUS, GENID, REALAR, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	unit ramp rate in MW/minute (default value of 9999.0 if new record, current value otherwise)
	REALAR(2)	unit capability in MW (default value of 9999.0 if new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified machine does not exist
	IERR = 3	generator reserve unit table full
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.24. OPF_GENRSV_SUBSYS

Use this API to add or modify all OPF generator reserve records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_GENRSV_SUBSYS   SID  ALL  REALAR(1)  REALAR(2)  USEVAL(1)
 USEVAL(2)
```

Python command syntax:

```
ierr = opf_genrsv_subsys(sid, all, realar, useval)
```

Fortran command syntax:

```
CALL OPF_GENRSV_SUBSYS(SID, ALL, REALAR, USEVAL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	unit ramp rate in MW/minute (default value of 9999.0 for new records, current value used otherwise)
	REALAR(2)	unit capability in MW (default value of 9999.0 for new records, current value used otherwise)
Integer USEVAL (2)	array of 2 elements specifying (default treatment same as value of 0) (input).	
	USEVAL(1)	indicates how the value of REALAR(1) should be used
	USEVAL(1) = 0	if the value of REALAR(1) is to be used as defined
	USEVAL(1) = 1	if the value of REALAR(1) is to be multiplied by the present value of the unit ramp rate
	USEVAL(1) = 2	if the value of REALAR(1) is to be added to the present value of the unit ramp rate
	USEVAL(2)	indicates how the value of REALAR(2) should be used
	USEVAL(2) = 0	if the value of REALAR(2) is to be used as defined
	USEVAL(2) = 1	if the value of REALAR(2) is to be multiplied by the present value of the unit capability

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	USEVAL(2) = 2	if the value of REALAR(2) is to be added to the present value of the unit capability
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	generator reserve unit table is full
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.25. OPF_GEN_RCAP_INDV

Use this API to add an individual generation reactive capability data record to the working case. It may also be used to modify an existing generation reactive capability data record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_GEN_RCAP_INDV   EIBUS   GENID   RCSTAT   REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = opf_gen_rcap_indv(eibus, genid, rcstat, realar)
```

Fortran command syntax:

```
CALL OPF_GEN_RCAP_INDV(EIBUS, GENID, RCSTAT, REALAR, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; ' ' by default).	
Integer RCSTAT	reactive capability limit status (input; 4 by default).	
	rcstat = 0	out of service.
	rcstat = 1	in service, no reactive generation limits.
	rcstat = 2	in service, inhibiting field increase.
	rcstat = 3	in service, inhibiting field decrease.
	rcstat = 4	in service, fixed Efd.
	rcstat = 5	in service, apply reactive generation limits.
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	direct-axis synchronous reactance in pu on Mbase (default of 1.0 if new record, current value otherwise)
	REALAR(2)	stator current limit in pu on Mbase (default of 1.0 if new record, current value otherwise)
	REALAR(3)	rated lagging power factor (default of 1.0 if new record, current value otherwise)
	REALAR(4)	rated leading power factor (default of 1.0 if new record, current value otherwise)
	REALAR(5)	maximum reactive absorption in pu on Mbase (default of 1.0 if new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified machine does not exist
	IERR = 3	generator reactive capability data table is full

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	prerequisite requirements for API are not met
IERR = -1	non-fatal error occurred. Possible warning messages are: - Invalid reactive capability status specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.26. OPF_GEN_RCAP_SUBSYS

Use this API to add or modify all OPF generator reactive capability records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_GEN_RCAP_SUBSYS   SID  ALL  RCSTAT  REALAR(1)..REALAR(5)
 USEVAL(1)..USEVAL(5)
```

Python command syntax:

```
ierr = opf_gen_rcap_subsys(sid, all, rcstat, realar, useval)
```

Fortran command syntax:

```
CALL OPF_GEN_RCAP_SUBSYS(SID, ALL, RCSTAT, REALAR, USEVAL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer RCSTAT	reactive capability limit status (default value of 4 for new records, current value otherwise) (input).	
	rcstat = 0	out of service.
	rcstat = 1	in service, no reactive generation limits.
	rcstat = 2	in service, inhibiting field increase.
	rcstat = 3	in service, inhibiting field decrease.
	rcstat = 4	in service, fixed Efd.
	rcstat = 5	in service, apply reactive generation limits.
Real REALAR (5)	array of 5 elements specifying The value of each is as follows(input).	
	REALAR(1)	direct-axis synchronous reactance in pu on Mbase (default value of 1.0 for new records, current value otherwise)
	REALAR(2)	stator current limit in pu on Mbase (default value of 1.0 for new records, current value otherwise)
	REALAR(3)	rated lagging power factor (default value of 1.0 for new records, current value otherwise)
	REALAR(4)	rated leading power factor (default value of 1.0 for new records, current value otherwise)
	REALAR(5)	maximum reactive absorption in pu on Mbase (default value of 1.0 for new records, current value otherwise)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer USEVAL (5) array of 5 elements specifying (default treatment same as value of 0) (input).

USEVAL(1)	indicates how the value of REALAR(1) should be used
USEVAL(1) = 0	if the value of REALAR(1) is to be used as defined
USEVAL(1) = 1	if the value of REALAR(1) is to be multiplied by the present value of the direct-axis synchronous reactance
USEVAL(1) = 2	if the value of REALAR(1) is to be added to the present value of the direct-axis synchronous reactance
USEVAL(2)	indicates how the value of REALAR(2) should be used
USEVAL(2) = 0	if the value of REALAR(2) is to be used as defined
USEVAL(2) = 1	if the value of REALAR(2) is to be multiplied by the present value of the stator current limit
USEVAL(2) = 2	if the value of REALAR(2) is to be added to the present value of the stator current limit
USEVAL(3)	indicates how the value of REALAR(3) should be used
USEVAL(3) = 0	if the value of REALAR(3) is to be used as defined
USEVAL(3) = 1	if the value of REALAR(3) is to be multiplied by the present value of the rated lagging power factor
USEVAL(3) = 2	if the value of REALAR(3) is to be added to the present value of the rated lagging power factor
USEVAL(4)	indicates how the value of REALAR(4) should be used
USEVAL(4) = 0	if the value of REALAR(4) is to be used as defined
USEVAL(4) = 1	if the value of REALAR(4) is to be multiplied by the present value of the rated leading power factor
USEVAL(4) = 2	if the value of REALAR(4) is to be added to the present value of the rated leading power factor
USEVAL(5)	indicates how the value of REALAR(5) should be used
USEVAL(5) = 0	if the value of REALAR(5) is to be used as defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	USEVAL(5) = 1	if the value of REALAR(5) is to be multiplied by the present value of the maximum reactive absorption
	USEVAL(5) = 2	if the value of REALAR(5) is to be added to the present value of the maximum reactive absorption
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	generator reactive capability data table is full
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	non-fatal error occurred. Possible warning messages are: - Invalid reactive capability status specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.27. OPF_INTFLW_3WT

Use this API to add a participating three-winding transformer winding to a specified interface flow constraint in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_INTFLW_3WT   IFID  EIBUS  EJBUS  EKBUS  CKTID
```

Python command syntax:

```
ierr = opf_intflw_3wt(ifid, eibus, ejbus, ekbus, cktid)
```

Fortran command syntax:

```
CALL OPF_INTFLW_3WT(IFID, EIBUS, EJBUS, EKBUS, CKTID, IERR)
```

Where:

Integer IFID	interface flow identifier (input; no default allowed).	
Integer EIBUS	from bus number The value of EIBUS indicates the from bus of the winding for which the flow constraint is being introduced (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Integer EKBUS	third bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	participating branch table is full
	IERR = 2	invalid interface flow identifier specified
	IERR = 3	invalid bus number specified
	IERR = 4	specified three-winding transformer does not exist
	IERR = 5	specified winding is already participating in the interface
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.28. OPF_INTFLW_BRN

Use this API to add a participating branch to a specified interface flow constraint in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_INTFLW_BRN   IFID  EIBUS  EJBUS  CKTID
```

Python command syntax:

```
ierr = opf_intflw_brn(ifid, eibus, ejbus, cktid)
```

Fortran command syntax:

```
CALL OPF_INTFLW_BRN(IFID, EIBUS, EJBUS, CKTID, IERR)
```

Where:

Integer IFID	interface flow identifier (input; no default allowed).	
Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	participating branch table is full
	IERR = 2	invalid interface flow identifier specified
	IERR = 3	invalid bus number specified
	IERR = 4	specified branch does not exist
	IERR = 5	specified branch is already participating in the interface
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.29. OPF_INTFLW_MAIN

Use this API to add an interface flow constraint record to the working case. It may also be used to modify an existing interface flow constraint record.
This API must be invoked prior to adding any participating branch or three-winding transformer flows to the interface flow constraint.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_INTFLW_MAIN   IFID  LABL  INTGAR(1)  INTGAR(2)
 REALAR(1)..REALAR(3)
```

Python command syntax:

```
ierr = opf_intflw_main(ifid, labl, intgar, realar)
```

Fortran command syntax:

```
CALL OPF_INTFLW_MAIN(IFID, LABL, INTGAR, REALAR, IERR)
```

Where:

Integer IFID	interface flow identifier (input; no default allowed).	
Character*32 LABL	interface flow description made up of, at most, 32 characters (input; ' ' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	interface flow type (default value of 1 if new record, current value otherwise)
	INTGAR(1) = 1	MW
	INTGAR(1) = 2	Mvar
	INTGAR(2)	interface flow limit type (default value of 1 if new record, current value otherwise)
	INTGAR(2) = 0	reporting only
	INTGAR(2) = 1	hard limit
	INTGAR(2) = 2	linear soft limit
	INTGAR(2) = 3	quadratic soft limit
Real REALAR (3)	array of 3 elements specifying (input).	
	REALAR(1)	maximum interface flow, in physical units (default value of 0.0 if new record, current value otherwise)
	REALAR(2)	minimum interface flow, in physical units (default value of 0.0 if new record, current value otherwise)
	REALAR(3)	soft limit penalty weight (default value of 1.0 if new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	interface flow constraint table is full
IERR = 2	prerequisite requirements for API are not met
IERR = -1	non-fatal error occurred. Possible warning messages are: - Invalid flow type specified. - Invalid interface flow limit type specified. - Maximum interface flow is less than the minimum interface flow

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.30. OPF_INTFLW_RESET

Use this API to remove all interface branches associated with a particular interface flow constraint in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_INTFLW_RESET   IFID
```

Python command syntax:

```
ierr = opf_intflw_reset(ifid)
```

Fortran command syntax:

```
CALL OPF_INTFLW_RESET(IFID, IERR)
```

Where:

Integer IFID	interface flow identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid interface flow identifier specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.31. OPF_LNCEQN_ADJLOAD

Use this API to add a participating adjustable bus load variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_ADJLOAD   IQID  ITBL  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_adjload(iqid, itbl, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_ADJLOAD(IQID, ITBL, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer ITBL	adjustable bus load table identifier (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; default of 1.0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	specified adjustable bus load table does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.32. OPF_LNCEQN_ADJVAR

Use this API to add a participating adjustable bus shunt variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_ADJVAR IQID EIBUS SHNTID COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_adjvar(iqid, eibus, shntid, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_ADJVAR(IQID, EIBUS, SHNTID, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Character*2 SHNTID	OPF adjustable bus shunt identifier (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; default of 1.0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid bus number specified
	IERR = 4	specified adjustable bus shunt does not exist
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.33. OPF_LNCEQN_BRFLOW

Use this API to add a participating branch flow variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_BRFLOW   IQID  EIBUS  EJBUS  EKBUS  CKTID  FLWID  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_brflow(iqid, eibus, ejbus, ekbus, cktid, flwid, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_BRFLOW(IQID, EIBUS, EJBUS, EKBUS, CKTID, FLWID, COEFF,
 IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Integer EKBUS	third bus number for a three-winding transformer and zero for a two-winding transformer (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Character*1 FLWID	branch flow identifier (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable. (input; Default value of 1.0 if new record, current value otherwise.).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid bus number specified
	IERR = 4	specified branch/transformer does not exist
	IERR = 5	specified three-winding transformer does not exist
	IERR = 6	flow constraint does not exist at the specified branch
	IERR = 7	flow constraint does not exist at the specified three-winding transformer
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.34. OPF_LNCEQN_INTFLOW

Use this API to add a participating interface flow constraint variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_INTFLOW   IQID  ITBL  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_intflow(iqid, itbl, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_INTFLOW(IQID, ITBL, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer ITBL	interface flow identifier (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise.).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	specified interface identifier does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.35. OPF_LNCEQN_MAIN

Use this API to add a linear constraint dependency record to the working case. It may also be used to modify an existing linear constraint dependency record.
This API must be invoked prior to adding any participating linear constraint equation variable records to the specified linear constraint dependency equation.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_MAIN   IQID  LABL  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = opf_lnceqn_main(iqid, labl, realar)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_MAIN(IQID, LABL, REALAR, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Character*12 LABL	linear constraint description made up of, at most, 12 characters (input; ' ' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	maximum constraint slack value (Default value of 1.0 if new record, current value otherwise. by default)
	REALAR(2)	minimum constraint slack value (Default value of 1.0 if new record, current value otherwise. by default)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Possible warning messages are: - Maximum linear constraint slack is less than the minimum linear constraint slack
	IERR = 0	no error occurred
	IERR = 1	linear constraint table is full
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.36. OPF_LNCEQN_PGEN

Use this API to add a participating active power dispatch variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_PGEN   IQID  ITBL  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_pgen(iqid, itbl, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_PGEN(IQID, ITBL, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer ITBL	active power dispatch table number (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise.).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid active power dispatch table number specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.37. OPF_LNCEQN_QGEN

Use this API to add a participating reactive power dispatch variable of a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_QGEN   IQID  EIBUS  GENID  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_qgen(iqid, eibus, genid, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_QGEN(IQID, EIBUS, GENID, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise.).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	specified bus does not exist
	IERR = 4	specified machine does not exist at bus
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.38. OPF_LNCEQN_RESET

Use this API to remove all participating variables from a particular OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_RESET   IQID
```

Python command syntax:

```
ierr = opf_lnceqn_reset(iqid)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_RESET(IQID, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	Invalid linear constraint identifier specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.39. OPF_LNCEQN_SWSHUNT

This API routine is obsolete. It has been replaced by the API routine opf_lnceqn_swshunt_2 .

Use this API to add a participating switched shunt variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_SWSHUNT   IQID  EIBUS  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_swshunt(iqid, eibus, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_SWSHUNT(IQID, EIBUS, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).
Integer EIBUS	bus number (input; no default allowed).
Real COEFF	value of the coefficient associated with the equation variable (input; Default of 1.0 if new record, current value otherwise).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.40. OPF_LNCEQN_SWSHUNT_2

Use this API to add a participating switched shunt variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_SWSHUNT_2 IQID EIBUS SWSHID COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_swshunt_2(iqid, eibus, swshid, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_SWSHUNT_2(IQID, EIBUS, SWSHID, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
character*2 SWSHID	Switched shunt identifier (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default of 1.0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid bus number specified
	IERR = 4	specified switched shunt does not exist
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.41. OPF_LNCEQN_TRAN

Use this API to add a participating transformer control variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_TRAN   IQID  EIBUS  EJBUS  EKBUS  CKTID  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_tran(iqid, eibus, ejbus, ekbus, cktid, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_TRAN(IQID, EIBUS, EJBUS, EKBUS, CKTID, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).
Integer EIBUS	from bus number (input; no default allowed).
Integer EJBUS	to bus number (input; no default allowed).
Integer EKBUS	third bus number if a three-winding transformer is specified and zero for a two-winding transformer (input; no default allowed).
Character*2 CKTID	circuit identifier (input; no default allowed).
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	specified linear constraint equation does not exist
IERR = 2	participating equation variable table is full
IERR = 3	invalid bus number specified
IERR = 3	specified branch/transformer does not exist
IERR = 4	specified three-winding transformer does not exist
IERR = 5	specified branch is not a transformer
IERR = 6	specified branch is not a three-winding transformer
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.42. OPF_LNCEQN_VANG

This API routine is obsolete. It has been replaced by the API routine opf_lnceqn_vang_2 .

Use this API to add a participating bus voltage angle variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_VANG   IQID  EIBUS  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_vang(iqid, eibus, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_VANG(IQID, EIBUS, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid bus number specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.43. OPF_LNCEQN_VANG_2

Use this API to add a participating bus voltage angle variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_VANG_2 IQID EIBUS ENODE COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_vang_2(iqid, eibus, enode, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_VANG_2(IQID, EIBUS, ENODE, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Integer ENODE	The bus section number or the external node number of one of the nodes in the bus section of interest. Specify 0 if bus is represented by a single bus section. If 0 is specified and bus is represented by multiple bus sections, then the data of the main parent bus and not that of the bus sections will be changed. (input; Default of 0).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid bus number
	IERR = 4	bus not found
	IERR = 5	invalid node number
	IERR = 6	node not found in substation of bus
	IERR = 7	node not connected to any bus section of bus in substation
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.44. OPF_LNCEQN_VMAG

This API routine is obsolete. It has been replaced by the API routine opf_lnceqn_vmag_2 .

Use this API to add a participating bus voltage magnitude variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_VMAG   IQID  EIBUS  COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_vmag(iqid, eibus, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_VMAG(IQID, EIBUS, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid bus number specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.45. OPF_LNCEQN_VMAG_2

Use this API to add a participating bus voltage magnitude variable to a specified linear constraint dependency equation in the working case. It may also be used to modify an existing record.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_OPF_LNCEQN_VMAG_2 IQID EIBUS ENODE COEFF
```

Python command syntax:

```
ierr = opf_lnceqn_vmag_2(iqid, eibus, enode, coeff)
```

Fortran command syntax:

```
CALL OPF_LNCEQN_VMAG_2(IQID, EIBUS, ENODE, COEFF, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Integer ENODE	The bus section number or the external node number of one of the nodes in the bus section of interest. Specify 0 if bus is represented by a single bus section. If 0 is specified and bus is represented by multiple bus sections, then the data of the main parent bus and not that of the bus sections will be changed. (input; Default of 0).	
Real COEFF	value of the coefficient associated with the equation variable (input; Default value of 1.0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	participating equation variable table is full
	IERR = 3	invalid bus number
	IERR = 4	bus not found
	IERR = 5	invalid node number
	IERR = 6	node not found in substation of bus
	IERR = 7	node not connected to any bus section of bus in substation
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.46. OPF_LOAD_INDV

Use this API to add an individual OPF bus load data record to the working case. It may also be used to modify an existing bus load data record.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LOAD_INDV   EIBUS  LOADID  INTGAR
```

Python command syntax:

```
ierr = opf_load_indv(eibus, loadid, intgar)
```

Fortran command syntax:

```
CALL OPF_LOAD_INDV(EIBUS, LOADID, INTGAR, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 LOADID	bus load identifier (input; no default allowed).	
Integer INTGAR	adjustable bus load table cross reference number (input; Default value of 0 if new record, current value otherwise).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified load does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.47. OPF_LOAD_SUBSYS

Use this API to add or modify all OPF bus load data records in a specified subsystem of the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_LOAD_SUBSYS   SID  ALL  INTGAR(1)  INTGAR(2)
```

Python command syntax:

```
ierr = opf_load_subsys(sid, all, intgar)
```

Fortran command syntax:

```
CALL OPF_LOAD_SUBSYS(SID, ALL, INTGAR, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses, use existing data values.
	ALL = 2	process all buses, use default data values.
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	bus load table cross reference number (default value of 0, current value otherwise)
	INTGAR(2)	indicates which bus types should be processed (default treatment same as value of 0)
	INTGAR(2) = 0	all buses in subsystem
	INTGAR(2) = 1	only Type 1 buses in subsystem
	INTGAR(2) = 2	only Type 2 and 3 buses in subsystem
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.48. OPF_PERRSV_GEN

Use this API to add a participating generation reserve unit to a previously defined period reserve constraint in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_PERRSV_GEN   RSVID  EIBUS  GENID
```

Python command syntax:

```
ierr = opf_perrsv_gen(rsvid, eibus, genid)
```

Fortran command syntax:

```
CALL OPF_PERRSV_GEN(RSVID, EIBUS, GENID, IERR)
```

Where:

Integer RSVID	period reserve identifier (1 through 15) (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid period reserve identifier specified
	IERR = 2	invalid bus number specified
	IERR = 3	specified machine does not exist
	IERR = 4	specified unit does not have any generator reserve data defined
	IERR = 5	specified unit is already participating in the reserve constraint
	IERR = 6	generator reserve participating unit table full
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.49. OPF_PERRSV_MAIN

Use this API to add a period reserve constraint record to the working case. It may also be used to modify an existing period reserve constraint record.
This API must be invoked prior to adding any participating generator reserve units (via OPF_PERRSV_GEN) to a specified period reserve constraint.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_PERRSV_MAIN   RSVID  STATUS  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = opf_perrsv_main(rsvid, status, realar)
```

Fortran command syntax:

```
CALL OPF_PERRSV_MAIN(RSVID, STATUS, REALAR, IERR)
```

Where:

Integer RSVID	a valid period reserve identifier (1 through 15) (input; no default allowed).	
Integer STATUS	status of the period reserve constraint (input; default value of 1 if new record, current value otherwise).	
	status = 0	out of service.
	status = 1	in service.
Real REALAR (2)	array of 2 elements specifying The value for each is as follows(input).	
	REALAR(1)	reserve limit, in MW (default value of 0.0 if new record, current value otherwise)
	REALAR(2)	period, in minutes (default value of 9999.0 if new record, current value otherwise)
Integer IERR	error code (output).	
	IERR = -1	non-fatal error occurred. Possible warning messages are: - Invalid period reserve status specified
	IERR = 0	no error occurred
	IERR = 1	invalid period reserve identifier specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.50. OPF_PERRSV_RESET

Use this API to remove all generation reserve units associated with a particular period reserve constraint in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_OPF_PERRSV_RESET    RSVID
```

Python command syntax:

```
ierr = opf_perrsv_reset(rsvid)
```

Fortran command syntax:

```
CALL OPF_PERRSV_RESET(RSVID, IERR)
```

Where:

Integer RSVID	period reserve identifier (1 through 15) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid period reserve identifier specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.51. PURGE_ALL_OPF_DATA

Use this API to purge and re-initialize all OPF data records in the working case. Unlike the NEWOPF_API call, any OPF data in the working case will be purged and re-initialized.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_ALL_OPF_DATA
```

Python command syntax:

```
ierr = purge_all_opf_data()
```

Fortran command syntax:

```
CALL PURGE_ALL_OPF_DATA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.52. PURGE_OPF_ADJBRX_INDV

Use this API to purge an individual adjustable branch reactance data record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_ADJBRX_INDV   EIBUS  EJBUS  CKTID
```

Python command syntax:

```
ierr = purge_opf_adjbrx_indv(eibus, ejbus, cktid)
```

Fortran command syntax:

```
CALL PURGE_OPF_ADJBRX_INDV_2(EIBUS, EJBUS, CKTID, IERR)
```

Where:

Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified branch does not exist
	IERR = 3	prerequisite requirements for API are not met
	IERR = 4	specified branch cannot be a transformer

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.53. PURGE_OPF_ADJBRX_SUBSYS

Use this API to purge all OPF branch reactance data records from a specified subsystem of the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_ADJBRX_SUBSYS    SID   ALL
```

Python command syntax:

```
ierr = purge_opf_adjbrx_subsys(sid, all)
```

Fortran command syntax:

```
CALL PURGE_OPF_ADJBRX_SUBSYS_2(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.54. PURGE_OPF_ADJLOAD_TBL

Use this API to purge an individual adjustable bus load table from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_ADJLOAD_TBL    TBL
```

Python command syntax:

```
ierr = purge_opf_adjload_tbl(tbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_ADJLOAD_TBL_2(TBL, IERR)
```

Where:

Integer TBL	adjustable bus load table number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	adjustable bus load table does not exist
	IERR = 2	load table still referenced by bus load(s) -- unable to delete
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.55. PURGE_OPF_ADJVAR_INDV

This API routine is obsolete. It has been replaced by the API routine purge_opf_adjvar_indv_3 .

Use this API to purge an individual OPF adjustable bus shunt data record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_ADJVAR_INDV   EIBUS   SHNTID
```

Python command syntax:

```
ierr = purge_opf_adjvar_indv(eibus, shntid)
```

Fortran command syntax:

```
CALL PURGE_OPF_ADJVAR_INDV_2(EIBUS, SHNTID, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 SHNTID	bus shunt identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	adjustable bus shunt identifier is more than two characters
	IERR = 2	invalid bus number specified
	IERR = 3	blank adjustable bus shunt identifier is invalid
	IERR = 4	OPF adjustable bus shunt at bus not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.56. PURGE_OPF_ADJVAR_INDV_3

Use this API to purge an individual OPF adjustable bus shunt data record from the working case.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_PURGE_OPF_ADJVAR_INDV_3   EIBUS  ENOD SHNTID
```

Python command syntax:

```
ierr = purge_opf_adjvar_indv_3(eibus, enod, shntid)
```

Fortran command syntax:

```
CALL PURGE_OPF_ADJVAR_INDV_3(EIBUS, ENOD, SHNTID, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Integer ENOD	The section number or the node number of one of the nodes in the bus section of interest. Specify 0 (zero) if the bus is represented by a single bus section. If 0 is specified and the bus is represented by multiple bus sections, then the data of the parent bus and not that of one of the bus sections is changed. (input).	
Character*2 SHNTID	bus shunt identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	adjustable bus shunt identifier is more than two characters
	IERR = 2	invalid bus number specified
	IERR = 3	blank adjustable bus shunt identifier is invalid
	IERR = 4	OPF adjustable bus shunt at bus not found
	IERR = 5	prerequisite requirements for API are not met
	IERR = 6	bus not found
	IERR = 7	invalid node number
	IERR = 8	bus does not belong to a substation
	IERR = 9	node is not found in the substation
	IERR = 10	node is not connected to any bus section of bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.57. PURGE_OPF_ADJVAR_SUBSYS

Use this API to purge all OPF adjustable bus shunt records from a specified subsystem of the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_ADJVAR_SUBSYS   SID  ALL  SHNTID
```

Python command syntax:

```
ierr = purge_opf_adjvar_subsys(sid, all, shntid)
```

Fortran command syntax:

```
CALL PURGE_OPF_ADJVAR_SUBSYS_2(SID, ALL, SHNTID, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Character*2 SHNTID	Adjustable bus shunt identifier. If blank then all adjustable bus shunts in subsystem will be deleted. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	adjustable bus shunt identifier is more than two characters
	IERR = 2	invalid SID value or subsystem SID is not defined
	IERR = 3	invalid ALL value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.58. PURGE_OPF_APDSP_TBL

Use this API to purge an individual active power dispatch table record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_APDSP_TBL    TBL
```

Python command syntax:

```
ierr = purge_opf_apdsp_tbl(tbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_APDSP_TBL_2(TBL, IERR)
```

Where:

Integer TBL	active power dispatch table number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	active power dispatch table does not exist
	IERR = 2	active power dispatch table still referenced -- unable to delete
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.59. PURGE_OPF_BRFLW_3WT

Use this API to purge an individual flow constraint record of a three-winding transformer from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_BRFLW_3WT   EIBUS  EJBUS  EKBUS  CKTID  BFLWID
```

Python command syntax:

```
ierr = purge_opf_brflw_3wt(eibus, ejbus, ekbus, cktid, bflwid)
```

Fortran command syntax:

```
CALL PURGE_OPF_BRFLW_3WT_2(EIBUS, EJBUS, EKBUS, CKTID, BFLWID, IERR)
```

Where:

Integer EIBUS	from bus of the winding from which the flow constraint is being purged (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Integer EKBUS	third bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Character*1 BFLWID	branch flow constraint identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified three-winding transformer does not exist
	IERR = 3	specified winding of three-winding transformer not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.60. PURGE_OPF_BRFLW_BRN

Use this API to purge an individual branch or two-winding transformer flow constraint record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_BRFLW_BRN   EIBUS  EJBUS  CKTID  BFLWID
```

Python command syntax:

```
ierr = purge_opf_brflw_brn(eibus, ejbus, cktid, bflwid)
```

Fortran command syntax:

```
CALL PURGE_OPF_BRFLW_BRN_2(EIBUS, EJBUS, CKTID, BFLWID, IERR)
```

Where:

Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Character*1 BFLWID	branch flow constraint identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified branch does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.61. PURGE_OPF_BRFLW_SUBSYS

Use this API to purge all branch flow data records in a specified subsystem in the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_BRFLW_SUBSYS   SID  ALL  BFLWID
```

Python command syntax:

```
ierr = purge_opf_brflw_subsys(sid, all, bflwid)
```

Fortran command syntax:

```
CALL PURGE_OPF_BRFLW_SUBSYS_2(SID, ALL, BFLWID, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Character*1 BFLWID	branch flow identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.62. PURGE_OPF_BUS_INDV

This API routine is obsolete. It has been replaced by the API routine purge_opf_bus_indv_3 .

Use this API to purge (re-initialize) OPF bus voltage magnitude data for an individual record in the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_BUS_INDV   EIBUS
```

Python command syntax:

```
ierr = purge_opf_bus_indv(eibus)
```

Fortran command syntax:

```
CALL PURGE_OPF_BUS_INDV_2(EIBUS, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.63. PURGE_OPF_BUS_INDV_3

Use this API to purge (re-initialize) OPF bus voltage magnitude data for an individual bus or bus section record in the working case.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_PURGE_OPF_BUS_INDV_3   EIBUS ENOD
```

Python command syntax:

```
ierr = purge_opf_bus_indv_3(eibus, enod)
```

Fortran command syntax:

```
CALL PURGE_OPF_BUS_INDV_3(EIBUS, ENOD, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Integer ENOD	The bus section number or the external node number of one of the nodes in the bus sections of interest. Specify 0 if bus is represented by a single bus section. If 0 is specified and bus is represented by multiple bus sections, then the data of parent bus and not that of one of the bus sections is changed. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.64. PURGE_OPF_BUS_SUBSYS

Use this API to purge (re-initialize) OPF bus attribute data for all records in a specified subsystem of the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_BUS_SUBSYS    SID  ALL
```

Python command syntax:

```
ierr = purge_opf_bus_subsys(sid, all)
```

Fortran command syntax:

```
CALL PURGE_OPF_BUS_SUBSYS_2(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses, use existing data values.
	ALL = 2	process all buses, use default data values.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.65. PURGE_OPF_CSTTBL_LIN

Use this API to purge an optimal power flow linear cost curve record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_CSTTBL_LIN   TBL
```

Python command syntax:

```
ierr = purge_opf_csttbl_lin(tbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_CSTTBL_LIN_2(TBL, IERR)
```

Where:

Integer TBL	linear cost curve table number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear cost table does not exist
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.66. PURGE_OPF_CSTTBL_POLY

Use this API to purge an optimal power flow polynomial and exponential cost curve record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_CSTTBL_POLY    TBL
```

Python command syntax:

```
ierr = purge_opf_csttbl_poly(tbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_CSTTBL_POLY_2(TBL, IERR)
```

Where:

Integer TBL	polynomial and exponential cost curve table number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified polynomial cost table does not exist
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.67. PURGE_OPF_CSTTBL_QUAD

Use this API to purge an optimal power flow quadratic cost curve record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_CSTTBL_QUAD    TBL
```

Python command syntax:

```
ierr = purge_opf_csttbl_quad(tbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_CSTTBL_QUAD_2(TBL, IERR)
```

Where:

Integer TBL	quadratic cost curve table number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified quadratic cost table does not exist
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.68. PURGE_OPF_GENDSP_INDV

Use this API to purge generator dispatch data for an individual record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_GENDSP_INDV   EIBUS  GENID
```

Python command syntax:

```
ierr = purge_opf_gendsp_indv(eibus, genid)
```

Fortran command syntax:

```
CALL PURGE_OPF_GENDSP_INDV_2(EIBUS, GENID, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified machine does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.69. PURGE_OPF_GENDSP_SUBSYS

Use this API to purge generator dispatch data for all records from a specified subsystem of the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_GENDSP_SUBSYS   SID  ALL
```

Python command syntax:

```
ierr = purge_opf_gendsp_subsys(sid, all)
```

Fortran command syntax:

```
CALL PURGE_OPF_GENDSP_SUBSYS_2(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses, use existing data values.
	ALL = 2	process all buses, use default data values.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.70. PURGE_OPF_GENRSV_INDV

Use this API to purge an individual optimal power flow generator reserve data record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_GENRSV_INDV   EIBUS  GENID
```

Python command syntax:

```
ierr = purge_opf_genrsv_indv(eibus, genid)
```

Fortran command syntax:

```
CALL PURGE_OPF_GENRSV_INDV_2(EIBUS, GENID, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified machine does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.71. PURGE_OPF_GENRSV_SUBSYS

Use this API to purge all OPF generator reserve records in a specified subsystem from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_GENRSV_SUBSYS   SID  ALL
```

Python command syntax:

```
ierr = purge_opf_genrsv_subsys(sid, all)
```

Fortran command syntax:

```
CALL PURGE_OPF_GENRSV_SUBSYS_2(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.72. PURGE_OPF_GEN_RCAP_INDV

Use this API to purge an individual OPF generation reactive capability data record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_GEN_RCAP_INDV   EIBUS  GENID
```

Python command syntax:

```
ierr = purge_opf_gen_rcap_indv(eibus, genid)
```

Fortran command syntax:

```
CALL PURGE_OPF_GEN_RCAP_INDV_2(EIBUS, GENID, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified machine does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.73. PURGE_OPF_GEN_RCAP_SUBSYS

Use this API to purge all OPF generator reactive capability records in a specified subsystem from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_GEN_RCAP_SUBSYS   SID  ALL
```

Python command syntax:

```
ierr = purge_opf_gen_rcap_subsys(sid, all)
```

Fortran command syntax:

```
CALL PURGE_OPF_GEN_RCAP_SUBSYS_2(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.74. PURGE_OPF_INTFLW

Use this API to purge an OPF interface flow constraint record, including all of its participating branches, from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_INTFLW   IFID
```

Python command syntax:

```
ierr = purge_opf_intflw(ifid)
```

Fortran command syntax:

```
CALL PURGE_OPF_INTFLW_2(IFID, IERR)
```

Where:

Integer IFID	interface flow identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	interface flow identifier not found
	IERR = 2	unable to remove interface flow record due to participation in linear constraint record
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.75. PURGE_OPF_INTFLW_3WT

Use this API to remove a participating three-winding transformer winding from a specified OPF interface flow constraint in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_INTFLW_3WT   IFID  EIBUS  EJBUS  EKBUS  CKTID
```

Python command syntax:

```
ierr = purge_opf_intflw_3wt(ifid, eibus, ejbus, ekbus, cktid)
```

Fortran command syntax:

```
CALL PURGE_OPF_INTFLW_3WT(IFID, EIBUS, EJBUS, EKBUS, CKTID, IERR)
```

Where:

Integer IFID	interface flow identifier (input; no default allowed).	
Integer EIBUS	from bus number The value of EIBUS indicates the from bus of the winding to be removed from the flow constraint (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Integer EKBUS	third bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid interface flow identifier specified
	IERR = 2	invalid bus number specified
	IERR = 3	specified three-winding transformer does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.76. PURGE_OPF_INTFLW_BRN

Use this API to remove a participating branch from a specified OPF interface flow constraint in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_INTFLW_BRN   IFID  EIBUS  EJBUS  CKTID
```

Python command syntax:

```
ierr = purge_opf_intflw_brn(ifid, eibus, ejbus, cktid)
```

Fortran command syntax:

```
CALL PURGE_OPF_INTFLW_BRN(IFID, EIBUS, EJBUS, CKTID, IERR)
```

Where:

Integer IFID	interface flow identifier (input; no default allowed).	
Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid interface flow identifier specified
	IERR = 2	invalid bus number specified
	IERR = 3	specified branch does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.77. PURGE_OPF_LNCEQN

Use this API to purge an OPF linear constraint dependency record, including all of its participating variables, from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN   IQID
```

Python command syntax:

```
ierr = purge_opf_lnceqn(iqid)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_2(IQID, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	linear constraint equation number not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.78. PURGE_OPF_LNCEQN_ADJLOAD

Use this API to remove a participating adjustable bus load variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_ADJLOAD   IQID  ITBL
```

Python command syntax:

```
ierr = purge_opf_lnceqn_adjload(iqid, itbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_ADJLOAD(IQID, ITBL, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer ITBL	adjustable bus load table identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	specified adjustable bus load table does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.79. PURGE_OPF_LNCEQN_ADJVAR

Use this API to remove a participating adjustable bus shunt variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_ADJVAR   IQID  EIBUS  SHNTID
```

Python command syntax:

```
ierr = purge_opf_lnceqn_adjvar(iqid, eibus, shntid)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_ADJVAR(IQID, EIBUS, SHNTID, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Character*2 SHNTID	adjustable bus shunt identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number specified
	IERR = 3	specified adjustable bus shunt does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.80. PURGE_OPF_LNCEQN_BRFLOW

Use this API to remove a participating branch flow variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_BRFLOW   IQID  EIBUS  EJBUS  EKBUS  CKTID  FLWID
```

Python command syntax:

```
ierr = purge_opf_lnceqn_brflow(iqid, eibus, ejbus, ekbus, cktid, flwid)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_BRFLOW(IQID, EIBUS, EJBUS, EKBUS, CKTID, FLWID,
 IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Integer EKBUS	third bus number for a three-winding transformer and zero for a two-winding transformer (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Character*1 FLWID	branch flow identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number specified
	IERR = 3	specified branch/transformer does not exist
	IERR = 4	specified three-winding transformer does not exist
	IERR = 5	flow constraint does not exist at the specified branch
	IERR = 6	flow constraint does not exist at the specified three-winding transformer
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.81. PURGE_OPF_LNCEQN_INTFLOW

Use this API to remove a participating interface flow constraint variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_INTFLOW   IQID  ITBL
```

Python command syntax:

```
ierr = purge_opf_lnceqn_intflow(iqid, itbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_INTFLOW(IQID, ITBL, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer ITBL	interface flow identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	specified interface identifier does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.82. PURGE_OPF_LNCEQN_PGEN

Use this API to remove a participating active power dispatch variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_PGEN   IQID  ITBL
```

Python command syntax:

```
ierr = purge_opf_lnceqn_pgen(iqid, itbl)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_PGEN(IQID, ITBL, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer ITBL	active power dispatch table number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid active power dispatch table number specified
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.83. PURGE_OPF_LNCEQN_QGEN

Use this API to remove a participating reactive power dispatch variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_QGEN   IQID  EIBUS  GENID
```

Python command syntax:

```
ierr = purge_opf_lnceqn_qgen(iqid, eibus, genid)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_QGEN(IQID, EIBUS, GENID, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	specified bus does not exist
	IERR = 3	specified machine does not exist at bus
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.84. PURGE_OPF_LNCEQN_SWSHUNT

This API routine is obsolete. It has been replaced by the API routine purge_opf_lnceqn_swshunt_2 .

Use this API to remove a participating switched shunt variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_SWSHUNT   IQID  EIBUS
```

Python command syntax:

```
ierr = purge_opf_lnceqn_swshunt(iqid, eibus)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_SWSHUNT(IQID, EIBUS, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).
Integer EIBUS	bus number (input; no default allowed).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.85. PURGE_OPF_LNCEQN_SWSHUNT_2

Use this API to remove a participating switched shunt variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_SWSHUNT_2   IQID  EIBUS SWSHID
```

Python command syntax:

```
ierr = purge_opf_lnceqn_swshunt_2(iqid, eibus, swshid)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_SWSHUNT_2(IQID, EIBUS, SWSHID, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
character*2 SWSHID	Switched shunt identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number specified
	IERR = 3	specified switched shunt does not exist
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.86. PURGE_OPF_LNCEQN_TRAN

Use this API to remove a participating transformer control variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_TRAN   IQID  EIBUS  EJBUS  EKBUS  CKTID
```

Python command syntax:

```
ierr = purge_opf_lnceqn_tran(iqid, eibus, ejbus, ekbus, cktid)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_TRAN(IQID, EIBUS, EJBUS, EKBUS, CKTID, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	from bus number (input; no default allowed).	
Integer EJBUS	to bus number (input; no default allowed).	
Integer EKBUS	third bus number if a three-winding transformer is specified and zero for a two-winding transformer (input; no default allowed).	
Character*2 CKTID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number specified
	IERR = 3	specified branch/transformer does not exist
	IERR = 4	specified three-winding transformer does not exist
	IERR = 5	specified branch is not a transformer
	IERR = 6	specified branch is not a three-winding transformer
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.87. PURGE_OPF_LNCEQN_VANG

This API routine is obsolete. It has been replaced by the API routine purge_opf_lnceqn_vang_2 .

Use this API to remove a participating bus voltage angle variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_VANG   IQID  EIBUS
```

Python command syntax:

```
ierr = purge_opf_lnceqn_vang(iqid, eibus)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_VANG(IQID, EIBUS, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number specified
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.88. PURGE_OPF_LNCEQN_VANG_2

Use this API to remove a participating bus voltage angle variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_VANG_2   IQID  EIBUS ENODE
```

Python command syntax:

```
ierr = purge_opf_lnceqn_vang_2(iqid, eibus, enode)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_VANG_2(IQID, EIBUS, ENODE, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Integer ENODE	The bus section number or the external node number of one of the nodes in the bus section of interest. Specify 0 if bus is represented by a single bus section. If 0 is specified and bus is represented by multiple bus sections, the data of the parent bus and not that of the bus sections will be changed. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number
	IERR = 3	prerequisite requirements for API are not met
	IERR = 4	bus not found
	IERR = 5	invalid node number
	IERR = 6	node not found in substation of bus
	IERR = 7	node not connected to any bus section of bus in substation

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.89. PURGE_OPF_LNCEQN_VMAG

This API routine is obsolete. It has been replaced by the API routine purge_opf_lnceqn_vmag_2 .

Use this API to remove a participating bus voltage magnitude variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_VMAG   IQID  EIBUS
```

Python command syntax:

```
ierr = purge_opf_lnceqn_vmag(iqid, eibus)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_VMAG(IQID, EIBUS, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number specified
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.90. PURGE_OPF_LNCEQN_VMAG_2

Use this API to remove a participating bus voltage magnitude variable from a specified OPF linear constraint dependency equation in the working case.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_PURGE_OPF_LNCEQN_VMAG_2   IQID  EIBUS ENODE
```

Python command syntax:

```
ierr = purge_opf_lnceqn_vmag_2(iqid, eibus, enode)
```

Fortran command syntax:

```
CALL PURGE_OPF_LNCEQN_VMAG_2(IQID, EIBUS, ENODE, IERR)
```

Where:

Integer IQID	linear constraint equation identification number (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Integer ENODE	The bus section number or the external node number of one of the nodes in the bus section of interest. Specify 0 if bus is represented by a single bus section. If 0 is specified and bus is represented by multiple bus sections, the data of the parent bus and not that of the bus sections will be changed. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	specified linear constraint equation does not exist
	IERR = 2	invalid bus number
	IERR = 3	prerequisite requirements for API are not met
	IERR = 4	bus not found
	IERR = 5	invalid node number
	IERR = 6	node not found in substation of bus
	IERR = 7	node not connected to any bus section of bus in substation

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.91. PURGE_OPF_LOAD_INDV

Use this API to purge (re-initialize) an individual OPF bus load data record from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LOAD_INDV   EIBUS  LOADID
```

Python command syntax:

```
ierr = purge_opf_load_indv(eibus, loadid)
```

Fortran command syntax:

```
CALL PURGE_OPF_LOAD_INDV_2(EIBUS, LOADID, IERR)
```

Where:

Integer EIBUS	bus number (input; no default allowed).	
Character*2 LOADID	bus load identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid bus number specified
	IERR = 2	specified load does not exist
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.92. PURGE_OPF_LOAD_SUBSYS

Use this API to purge (re-initialize) all OPF bus load data records in a specified subsystem of the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_LOAD_SUBSYS   SID  ALL
```

Python command syntax:

```
ierr = purge_opf_load_subsys(sid, all)
```

Fortran command syntax:

```
CALL PURGE_OPF_LOAD_SUBSYS_2(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem ID from 0 to 11. A subsystem must have been previously defined (input; no default allowed).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only specified buses.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.93. PURGE_OPF_PERRSV

Use this API to purge an OPF period reserve constraint record and its association with participating reserve units from the working case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_PERRSV    RSVID
```

Python command syntax:

```
ierr = purge_opf_perrsv(rsvid)
```

Fortran command syntax:

```
CALL PURGE_OPF_PERRSV_2(RSVID, IERR)
```

Where:

Integer RSVID	a valid period reserve identifier (1 through 15) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid period reserve identifier specified
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

3.2.94. PURGE_OPF_PERRSV_GEN

Use this API to remove a participating period reserve unit from a specified OPF period reserve constraint.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PURGE_OPF_PERRSV_GEN   RSVID  EIBUS  GENID
```

Python command syntax:

```
ierr = purge_opf_perrsv_gen(rsvid, eibus, genid)
```

Fortran command syntax:

```
CALL PURGE_OPF_PERRSV_GEN(RSVID, EIBUS, GENID, IERR)
```

Where:

Integer RSVID	a valid period reserve identifier (1 through 15) (input; no default allowed).	
Integer EIBUS	bus number (input; no default allowed).	
Character*2 GENID	machine identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	period reserve constraint does not exist
	IERR = 2	bus not found
	IERR = 3	machine not found
	IERR = 4	machine does not have generator reserve data defined
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Optimal Power Flow Data Changing APIs contains API routines specific to adding, modifying, and deleting optimal power flow data.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 4
Fault Analysis

This chapter describes the API routines used in PSS®E Fault Analysis, also called PSS®E Short Circuit.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1. Fault Analysis Operation

4.1.1. ANSI

This API routine is obsolete. It has been replaced by the API routine ansi_2 .

Use this API to calculate fault currents according to the ANSI standard (activity ANSI). To designate fault specification data as part of the API's calling sequence, INARG must be positive; to designate fault specification data in an ANSI Fault Specification Data input file, INARG must be zero and the filename specified as FILARG.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ANSI  OPTIONS(1)..OPTIONS(3)  VALUES(1)..VALUES(4)
 INARG  BUSARY(1)..BUSARY(INARG)  VLTARY(1)..VLTARY(INARG)
 CPTARY(1)..CPTARY(INARG)  FILARG
```

Python command syntax:

```
ierr = ansi(options, values, inarg, busary, vltary, cptary, filarg)
```

Fortran command syntax:

```
CALL ANSIAPI(OPTIONS, VALUES, INARG, BUSARY, VLTARY, CPTARY, FILARG, IERR)
```

Where:

Integer OPTIONS (3)	array of 3 elements specifying solution and reporting options(input).	
	OPTIONS(1)	transformer impedance correction option (1 by default)
	OPTIONS(1) = 0	do not apply to zero sequence
	OPTIONS(1) = 1	apply to zero sequence
	OPTIONS(2)	set of ANSI decrement factors to use (1 by default)
	OPTIONS(2) = 0	account for dc decrement only (use ANSI table 3)
	OPTIONS(2) = 1	account for ac and dc decrements (use ANSI tables 1 and 2)
	OPTIONS(3)	option (0 by default)
	OPTIONS(3) = 0	summary output using ANSI X only, report format (blank delimited output fields)
	OPTIONS(3) = 1	summary output using ANSI X only, spreadsheet input format (comma delimited output fields)
	OPTIONS(3) = 2	detailed output using ANSI X only
	OPTIONS(3) = 3	detailed output using ANSI R and X
	OPTIONS(3) = 4	summary output using ANSI R and X, report format (blank delimited output fields)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(3) = 5	summary output using ANSI R and X, spreadsheet input format (comma delimited output fields)
Real VALUES (4)	array of 4 elements specifying divisors for calculating resistances of network elements where resistance is zero in the working case(input).	
	VALUES(1)	used for branches in the positive sequence network (40.0 by default)
	VALUES(2)	used for machines in the positive sequence network (80.0 by default)
	VALUES(3)	used for branches in the zero sequence network (40.0 by default)
	VALUES(4)	used for machines in the zero sequence network (80.0 by default)
Integer INARG	fault specification option (input; 0 by default).	
	INARG = 0	fault specifications specified in input file.
	INARG > 0	fault specifications specified in calling sequence.
Integer BUSARY (INARG)	a list of faulted buses; ignored if INARG is 0 (if INARG > 0) (input; no default allowed).	
Real VLTARY (INARG)	a list of maximum operating voltages in pu; ignored if INARG is 0 (input; 1.0).	
Real CPTARY (INARG)	a list of contact parting times in seconds; ignored if INARG is 0 (input; 1 cycle).	
Character*260 FILARG	filename of ANSI Fault Specification Data File; ignored if INARG > 0 (if INARG is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	invalid INARG value
	IERR = 4	fault analysis is invalid due to the presence of in-service induction machines that are in the "stalled" or "tripped" state
	IERR = 5	bus type code and series element status inconsistencies
	IERR = 6	error opening file FILARG
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.2. ANSI_2

This API routine is obsolete. It has been replaced by the API routine ansi_3 .

This API routine executes the second release of the ANSI fault calculation function. Use this API routine to calculate fault currents according to the ANSI standard (activity ANSI). To designate fault specification data as part of the API's calling sequence, INARG must be positive; to designate fault specification data in an ANSI Fault Specification Data input file, INARG must be zero and the filename specified as FILARG.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ANSI_2  OPTIONS(1)..OPTIONS(5)  VALUES(1)..VALUES(4)
 INARG  BUSARY(1)..BUSARY(INARG)  VLTARY(1)..VLTARY(INARG)
 CPTARY(1)..CPTARY(INARG)  FILARG
```

Python command syntax:

```
ierr = ansi_2(options, values, inarg, busary, vltary, cptary, filarg)
```

Fortran command syntax:

```
CALL ANSI_2(OPTIONS,VALUES,INARG,BUSARY,VLTARY,CPTARY,FILARG,IERR)
```

Where:

Integer OPTIONS (5)	array of 5 elements specifying solution and reporting options(input).	
	OPTIONS(1)	fault analysis generator reactance option (0 by default)
	OPTIONS(1) = 0	use subtransient reactance
	OPTIONS(1) = 1	use transient reactance
	OPTIONS(1) = 2	use synchronous reactance
	OPTIONS(2)	transformer impedance correction option (1 by default)
	OPTIONS(2) = 0	do not apply to zero sequence
	OPTIONS(2) = 1	apply to zero sequence
	OPTIONS(3)	set of ANSI decrement factors to use (1 by default)
	OPTIONS(3) = 0	account for dc decrement only (use ANSI table 3)
	OPTIONS(3) = 1	account for ac and dc decrements (use ANSI tables 1 and 2)
	OPTIONS(4)	option (0 by default)
	OPTIONS(4) = 0	summary output using ANSI X only, report format (blank delimited output fields)
	OPTIONS(4) = 1	summary output using ANSI X only, spreadsheet input format (comma delimited output fields)
	OPTIONS(4) = 2	detailed output using ANSI X only

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(4) = 3	detailed output using ANSI R and X
	OPTIONS(4) = 4	summary output using ANSI R and X, report format (blank delimited output fields)
	OPTIONS(4) = 5	summary output using ANSI R and X, spread-sheet input format (comma delimited output fields)
	OPTIONS(5)	breaker rating option (0 by default)
	OPTIONS(5) = 0	IEEE total current basis
	OPTIONS(5) = 1	IEEE symmetrical current basis
Real VALUES (4)	array of 4 elements specifying divisors for calculating resistances of network elements where resistance is zero in the working case(input).	
	VALUES(1)	used for branches in the positive sequence network (40.0 by default)
	VALUES(2)	used for machines in the positive sequence network (80.0 by default)
	VALUES(3)	used for branches in the zero sequence network (40.0 by default)
	VALUES(4)	used for machines in the zero sequence network (80.0 by default)
Integer INARG	fault specification option (input; 0 by default).	
	INARG = 0	fault specifications specified in input file.
	INARG > 0	fault specifications specified in calling sequence.
Integer BUSARY (INARG)	a list of faulted buses; ignored if INARG is 0 (if INARG > 0) (input; no default allowed).	
Real VLTARY (INARG)	a list of maximum operating voltages in pu; ignored if INARG is 0 (input; 1.0).	
Real CPTARY (INARG)	a list of contact parting times in seconds; ignored if INARG is 0 (input; 1 cycle).	
Character*260 FILARG	filename of ANSI Fault Specification Data File; ignored if INARG > 0 (if INARG is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	invalid INARG value
	IERR = 4	fault analysis is invalid due to the presence of in-service induction machines that are in the "stalled" or "tripped" state
	IERR = 5	bus type code and series element status inconsistencies
	IERR = 6	error opening file FILARG
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.3. ANSI_3

This API routine executes the third release of the ANSI fault calculation function. Use this API routine to calculate fault currents according to the ANSI standard (activity ANSI). To designate fault specification data as part of the API's calling sequence, INARG must be positive; to designate fault specification data in an ANSI Fault Specification Data input file, INARG must be zero and the filename specified as FILARG.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_ANSI_3  OPTIONS(1)..OPTIONS(5)  VALUES(1)..VALUES(4)
 INARG  BUSARY(1)..BUSARY(INARG)  NODARY(1)..NODARY(INARG)
 VLTARY(1)..VLTARY(INARG)  CPTARY(1)..CPTARY(INARG)  FILARG
```

Python command syntax:

```
ierr = ansi_3(options, values, inarg, busary, nodary, vltary, cptary,
 filarg)
```

Fortran command syntax:

```
CALL ANSI_3(OPTIONS,VALUES,INARG,BUSARY,NODARY,VLTARY,CPTARY,FILARG,IERR)
```

Where:

Integer OPTIONS (5)	array of 5 elements specifying solution and reporting options(input).	
	OPTIONS(1)	fault analysis generator reactance option (0 by default)
	OPTIONS(1) = 0	use subtransient reactance
	OPTIONS(1) = 1	use transient reactance
	OPTIONS(1) = 2	use synchronous reactance
	OPTIONS(2)	transformer impedance correction option (1 by default)
	OPTIONS(2) = 0	do not apply to zero sequence
	OPTIONS(2) = 1	apply to zero sequence
	OPTIONS(3)	set of ANSI decrement factors to use (1 by default)
	OPTIONS(3) = 0	account for dc decrement only (use ANSI table 3)
	OPTIONS(3) = 1	account for ac and dc decrements (use ANSI tables 1 and 2)
	OPTIONS(4)	option (0 by default)
	OPTIONS(4) = 0	summary output using ANSI X only, report format (blank delimited output fields)
	OPTIONS(4) = 1	summary output using ANSI X only, spreadsheet input format (comma delimited output fields)
	OPTIONS(4) = 2	detailed output using ANSI X only
	OPTIONS(4) = 3	detailed output using ANSI R and X

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(4) = 4	summary output using ANSI R and X, report format (blank delimited output fields)
	OPTIONS(4) = 5	summary output using ANSI R and X, spreadsheet input format (comma delimited output fields)
	OPTIONS(5)	breaker rating option (0 by default)
	OPTIONS(5) = 0	IEEE total current basis
	OPTIONS(5) = 1	IEEE symmetrical current basis
Real VALUES (4)	array of 4 elements specifying divisors for calculating resistances of network elements where resistance is zero in the working case(input).	
	VALUES(1)	used for branches in the positive sequence network (40.0 by default)
	VALUES(2)	used for machines in the positive sequence network (80.0 by default)
	VALUES(3)	used for branches in the zero sequence network (40.0 by default)
	VALUES(4)	used for machines in the zero sequence network (80.0 by default)
Integer INARG	fault specification option (input; 0 by default).	
	INARG = 0	fault specifications specified in input file.
	INARG > 0	fault specifications specified in calling sequence.
Integer BUSARY (INARG)	a list of faulted buses; ignored if INARG is 0 (if INARG > 0) (input; no default allowed).	
Integer NODARY (INARG)	a list of node numbers whose bus sections are to be faulted; ignored if INARG is 0 (if INARG > 0) (input; 0 by default).	
Real VLTARY (INARG)	a list of maximum operating voltages in pu; ignored if INARG is 0 (input; 1.0).	
Real CPTARY (INARG)	a list of contact parting times in seconds; ignored if INARG is 0 (input; 1 cycle).	
Character*260 FILARG	filename of ANSI Fault Specification Data File; ignored if INARG > 0 (if INARG is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	invalid INARG value
	IERR = 4	fault analysis is invalid due to the presence of in-service induction machines that are in the "stalled" or "tripped" state
	IERR = 5	bus type code and series element status inconsistencies
	IERR = 6	error opening file FILARG

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.4. ASCC

This API routine is obsolete. It has been replaced by the API routine ASCC_1A .

This API is used to apply a series of faults at various locations in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ASCC  SID  ALL  STATUS(1)..STATUS(10)  RELFILE  FCDFILE
```

Python command syntax:

```
ierr = ascc(sid, all, status, relfile, fcdfile)
```

Fortran command syntax:

```
CALL ASCCNW(SID, ALL, STATUS, RELFILE, FCDFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (10)	array of 10 elements specifying integer options(input).	
	STATUS(1)	Three phase and line-to-ground fault (alias is flt3phlg) (0 by default)
	STATUS(1) = 0	three phase faults only
	STATUS(1) = 1	three phase and line-to-ground faults
	STATUS(2)	line-out faults (alias is linout) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-end faults (alias is linend) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	flat conditions option (alias is flatop) (0 by default)
	STATUS(4) = 0	pre-fault conditions taken from working case
	STATUS(4) = 1	impose flat conditions as pre-fault condition
	STATUS(5)	double-precision fault calculation option (alias is nouse1) (ignored, but must be specified as 0 or 1; 0 by default; double precision always used)
	STATUS(6)	report option (alias is rptop) (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(6) = 1	full output at home bus and N levels away
STATUS(6) = 2	full output at home bus, summary N levels away
STATUS(6) = 3	fault current summary table
STATUS(7)	number of levels back for contributions if STATUS(6) = 1 or 2. STATUS(7) > 0 (alias is rptlvl) (0 by default)
STATUS(8)	branch quantity code for line to ground faults (alias is brnop) (0 by default)
STATUS(8) = 0	A phase
STATUS(8) = 1	3 * IO
STATUS(8) = 2	both
STATUS(9)	dc line and FACTS device option (alias is dcload) (0 by default)
STATUS(9) = 0	blocked
STATUS(9) = 1	represent as load
STATUS(10)	transformer impedance correction option (alias is zcorec) (1 by default)
STATUS(10) = 0	do not apply to zero sequence
STATUS(10) = 1	apply to zero sequence

Character*260 RELFILE	Relay Data (.rel) file (input; blank by default).
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.5. ASCC_1A

This API routine is obsolete. It has been replaced by the API routine ASCC_2 .

This API is used to apply a series of faults at various locations in the working case.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ASCC_1A  SID  ALL  STATUS(1)..STATUS(10)  RELFILE  FCDFILE
```

Python command syntax:

```
ierr = ascc_1a(sid, all, status, relfile, fcdfile)
```

Fortran command syntax:

```
CALL ASCC_1A(SID, ALL, STATUS, RELFILE, FCDFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (10)	array of 10 elements specifying integer options(input).	
	STATUS(1)	Three phase and line-to-ground fault (alias is flt3phlg) (0 by default)
	STATUS(1) = 0	three phase faults only
	STATUS(1) = 1	three phase and line-to-ground faults
	STATUS(2)	line-out faults (alias is linout) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-end faults (alias is linend) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	flat conditions option (alias is flatop) (0 by default)
	STATUS(4) = 0	pre-fault conditions taken from working case
	STATUS(4) = 1	impose flat conditions as pre-fault condition
	STATUS(5)	fault analysis generator reactance option; ignored if generators are already "convert-ed" (alias is genxop) (0 by default)
	STATUS(5) = 0	use subtransient reactance
	STATUS(5) = 1	use transient reactance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 2	use synchronous reactance
	STATUS(6)	option (alias is rptop) (1 by default)
	STATUS(6) = 1	full output at home bus and N levels away
	STATUS(6) = 2	full output at home bus, summary N levels away
	STATUS(6) = 3	fault current summary table
	STATUS(7)	number of levels back for contributions if STATUS(6) = 1 or 2. STATUS(7) > 0 (alias is rptlvl) (0 by default)
	STATUS(8)	branch quantity code for line to ground faults (alias is brnop) (0 by default)
	STATUS(8) = 0	A phase
	STATUS(8) = 1	3 * IO
	STATUS(8) = 2	both
	STATUS(9)	dc line and FACTS device option (alias is dcload) (0 by default)
	STATUS(9) = 0	blocked
	STATUS(9) = 1	represent as load
	STATUS(10)	transformer impedance correction option (alias is zcorec) (1 by default)
	STATUS(10) = 0	do not apply to zero sequence
	STATUS(10) = 1	apply to zero sequence
Character*260 RELFILE	Relay Data (.rel) file (input; blank by default).	
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	line end fault is invalid; bus table is full
	IERR = 5	fault analysis is invalid due to the presence of in-service induction machines that are in the "stalled" or "tripped" state
	IERR = 6	bus type code and series element status inconsistencies
	IERR = 7	error opening RELFILE
	IERR = 8	error opening FCDFILE
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.6. ASCC_2

This API routine is obsolete. It has been replaced by the API routine ASCC_3 .

This API is used to apply a series of faults at various locations in the working case.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_ASCC_2  SID ALL STATUS(1)..STATUS(13) RELFILE FCDFILE SCFILE
```

Python command syntax:

```
ierr = ascc_2(sid, all, status, relfile, fcdfile, scfile)
```

Fortran command syntax:

```
CALL ASCC2(SID, ALL, STATUS, RELFILE, FCDFILE, SCFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or selected subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (13)	array of 13 elements specifying integer options(input).	
	STATUS(1)	line-to-ground faults (alias is fltlg) (0 by default)
	STATUS(1) = 0	omit
	STATUS(1) = 1	include
	STATUS(2)	line-out faults (alias is linout) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-end faults (alias is linend) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	flat conditions option (alias is flatop) (0 by default)
	STATUS(4) = 0	pre-fault conditions taken from working case
	STATUS(4) = 1	impose flat conditions as pre-fault condition
	STATUS(5)	double-precision fault calculation option (alias is nouse1) (ignored, but must be specified as 0 or 1; 0 by default; double precision always used)
	STATUS(6)	report option (alias is rptop) (1 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(6) = 1	total fault currents
	STATUS(6) = 2	fault contributions to N levels away
	STATUS(6) = 3	total fault currents and fault contributions to N levels away
	STATUS(7)	number of levels back for contributions (alias is rptlvl) (0 by default)
	STATUS(7) >= 0	used if STATUS(6) = 2 or 3
	STATUS(8)	branch quantity code (alias is nouse2) (ignored, but must be specified as 0 or 1; 0 by default)
	STATUS(9)	dc line and FACTS device option (alias is dcload) (0 by default)
	STATUS(9) = 0	blocked
	STATUS(9) = 1	represent as load
	STATUS(10)	zero sequence transformer impedance correction option (alias is zcorec) (0 by default)
	STATUS(10) = 0	ignore
	STATUS(10) = 1	apply
	STATUS(11)	three phase faults (alias is flt3ph) (0 by default)
	STATUS(11) = 0	omit
	STATUS(11) = 1	include
	STATUS(12)	line-line-to-ground faults (alias is fltllg) (0 by default)
	STATUS(12) = 0	omit
	STATUS(12) = 1	include
	STATUS(13)	line-to-line faults (alias is fltll) (0 by default)
	STATUS(13) = 0	omit
	STATUS(13) = 1	include
Character*260 RELFILE	Relay Data (.rel) file (input; blank by default).	
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).	
Character*260 SCFILE	ASCC_2 Results (.sc) file (input; blank by default).	
	scfile =	temporary file created
	scfile = nooutput	no file created
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.7. ASCC_3

This API is used to apply a series of faults at various locations in the working case.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ASCC_3  SID ALL STATUS(1)..STATUS(17) VALUES(1) RELFILE FCDFILE SCFILE
```

Python command syntax:

```
ierr = ascc_3(sid, all, status, values, relfile, fcdfile, scfile)
```

Fortran command syntax:

```
CALL ASCC3(SID, ALL, STATUS, VALUES, RELFILE, FCDFILE, SCFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or selected subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (17)	array of 17 elements specifying integer options(input).	
	STATUS(1)	line-to-ground faults (alias is fltlg) (0 by default)
	STATUS(1) = 0	omit
	STATUS(1) = 1	include
	STATUS(2)	line-out faults (alias is linout) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-end faults (alias is linend) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	bus voltage option (alias is voltop) (0 by default)
	STATUS(4) = 0	use bus voltages from power flow solution
	STATUS(4) = 1	set all bus voltages at specified value and at 0 deg
	STATUS(4) = 2	set faulted bus voltage at specified value and at 0 deg
	STATUS(5)	fault analysis generator reactance option; ignored if generators are already "converted" (alias is genxop) (0 by default)
	STATUS(5) = 0	use subtransient reactance
	STATUS(5) = 1	use transient reactance
	STATUS(5) = 2	use synchronous reactance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(6)	report option (alias is rptop) (1 by default)
STATUS(6) = 0	fault current summary table
STATUS(6) = 1	total fault currents
STATUS(6) = 2	fault contributions to N levels away
STATUS(6) = 3	total fault currents and fault contributions to N levels away
STATUS(7)	number of levels back for contributions (alias is rptlvl) (0 by default)
STATUS(7) >= 0	used if STATUS(6) = 2 or 3
STATUS(8)	transformer tap ratios and phase shift angles option (alias is tpunty) (0 by default)
STATUS(8) = 0	leave tap ratios and phase shift angles unchanged
STATUS(8) = 1	set tap ratios to 1.0 pu and phase shift angles to 0
STATUS(8) = 2	set tap ratios to 1.0 pu and phase shift angles unchanged
STATUS(8) = 3	set tap ratios unchanged and phase shift angles to 0
STATUS(8) = 4	set tap ratios to nominal winding kV and phase shift angle unchanged
STATUS(9)	dc line and FACTS device option (alias is dcload) (0 by default)
STATUS(9) = 0	blocked
STATUS(9) = 1	represent as load
STATUS(10)	zero sequence transformer impedance correction option (alias is zcorec) (1 by default)
STATUS(10) = 0	ignore
STATUS(10) = 1	apply
STATUS(11)	three phase faults (alias is flt3ph) (0 by default)
STATUS(11) = 0	omit
STATUS(11) = 1	include
STATUS(12)	line-line-to-ground faults (alias is fltllg) (0 by default)
STATUS(12) = 0	omit
STATUS(12) = 1	include
STATUS(13)	line-to-line faults (alias is fltll) (0 by default)
STATUS(13) = 0	omit
STATUS(13) = 1	include
STATUS(14)	line charging option (alias is lnchrg) (0 by default)
STATUS(14) = 0	leave unchanged
STATUS(14) = 1	set to 0.0 in the positive and negative sequences

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(14) = 2	set to 0.0 in all sequences
	STATUS(15)	line, fixed and switched shunts, and magnetizing admittance option (alias is shntop) (0 by default)
	STATUS(15) = 0	leave unchanged
	STATUS(15) = 1	set to 0.0 in the positive and negative sequences
	STATUS(15) = 2	set to 0.0 in all sequences
	STATUS(16)	load option (alias is loadop) (0 by default)
	STATUS(16) = 0	leave unchanged
	STATUS(16) = 1	set to 0.0 in the positive and negative sequences
	STATUS(16) = 2	set to 0.0 in all sequences
	STATUS(16) = 3	set to 0.0 in the positive sequences
	STATUS(17)	synchronous and asynchronous machines power option (alias is machpq) (0 by default)
	STATUS(17) = 0	use real and reactive power outputs from power flow solution
	STATUS(17) = 1	set real and reactive power outputs to 0.0
Real VALUES (1)	array of 1 elements specifying real values(input).	
	VALUES(1)	User specified pu bus voltage value (Used when STATUS (4) = 1 or 2) (alias is volts)
Character*260 RELFILE	Relay Data (.rel) file (input; blank by default).	
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).	
Character*260 SCFILE	Results (.sc) file (input; blank by default).	
	scfile =	temporary file created
	scfile = nooutput	no file created
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	line end fault invalid; bus table is full
	IERR = 5	error opening RELFILE
	IERR = 6	error opening FCDFILE
	IERR = 7	prerequisite requirements for function not met
	IERR = 8	no faults specified
	IERR = 9	error opening SCFILE
	IERR = 10	SCFILE error
	IERR = 11	bus SC results not available
	IERR = 12	branch SC results not available

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 13	three winding transformer SC results not available
IERR = 14	incorrect SCFILE
IERR = 15	SCFILE not allowed
IERR = 16	fault analysis invalid, in-service induction machines stalled or tripped state
IERR = 17	invalid VALUES value
IERR = 18	SC result file size exceeded. Define smaller size bus subsystems and re-run if SCFILE is required.
IERR = 19	topology error - bus(es) with bus type code and series element status inconsistencies
IERR = 20	Number of branches at faulted bus are greater than NBRLINKS (only arrbox.fault module)
IERR = 21	Re-run by specifying higher NBRLNKS value (only arrbox.fault module)
IERR = 22	Network case has number of MOV and/or NCS, cannot use arrbox.ascc_currents, use arrbox.fault module

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.8. ASCC_SCFILE

This API reads ASCC short circuit results from SCFILE and produces same ASCC output report that was generated when SCFILE was created. SCFILE is read into PSSE memory to enable displaying ASCC results onto slider diagrams. For this API to work properly, it is required that working case in PSSE memory should be the same Saved case that was used when creating the SCFILE.

This API routine was first introduced in release 32.0.2.

Batch command syntax:

```
BAT_ASCC_SCFILE  SCFILE
```

Python command syntax:

```
ierr = ascc_scfile(scfile)
```

Fortran command syntax:

```
CALL ASCC_SCFILE(SCFILE,IERR)
```

Where:

Character*260 SCFILE	Activity ASCC Results (.sc) file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 7	prerequisite requirements for function not met
	IERR = 9	error opening SCFILE
	IERR = 10	SCFILE error
	IERR = 11	bus SC results not available
	IERR = 12	branch SC results not available
	IERR = 13	three winding transformer SC results not available
	IERR = 14	incorrect SCFILE
	IERR = 15	SCFILE not allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.9. BKDY

Use this API to calculate and report circuit breaker interrupting duty for 3-phase faults at all buses in a specified subsystem of the working case (activity BKDY).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BKDY  SID  ALL  APIOPT  LVLBAK  FLTTIM  BFILE  FFILE
```

Python command syntax:

```
ierr = bkdy(sid, all, apiopt, lvlbak, flttim, bfile, ffile)
```

Fortran command syntax:

```
CALL BKDYNW(SID, ALL, APIOPT, LVLBAK, FLTTIM, BFILE, FFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses.
	ALL = 0	process only buses in subsystem SID.
Integer APIOPT	mode of operation in the API (input; 1 by default).	
	APIOPT = 1	fault applications specified in subsystem.
	APIOPT = 2	fault applications specified in FFILE.
Integer LVLBAK	number of levels back for contributions (input; 0 by default).	
	LVLBAK < 0	fault currents only.
	LVLBAK = 0	fault currents and local contributions.
	LVLBAK > 0	fault currents and local contributions, plus contributions LVLBAK levels back.
Real FLTTIM	default fault duty time. FLTTIM >= 0.0 (input; 0.0).	
Character*260 BFILE	Breaker Duty Data file (input; no default allowed).	
Character*260 FFILE	Fault Specification Data file (if APIOPT is 2) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid APIOPT value
	IERR = 4	invalid FLTTIM value
	IERR = 5	BFILE is blank
	IERR = 6	FFILE is blank

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	generators are not converted
IERR = 8	fixed bus shunt table is full
IERR = 9	error opening BFILE or FFILE
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.10. CHECK_SEQUENCE_DATA

Use this API routine to perform data checks on the selected categories of sequence data for all buses in the working case or for all buses in a specified subsystem.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_CHECK_SEQUENCE_DATA SID ALL OPT
```

Python command syntax:

```
ierr = check_sequence_data(sid, all, opt)
```

Fortran command syntax:

```
CALL CHECK_SEQUENCE_DATA(SID, ALL, OPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer OPT	sequence data category to be checked (input; 0 by default).	
	OPT = 0	all data categories
	OPT = 1	load data
	OPT = 2	generator unit data
	OPT = 3	induction machine data
	OPT = 4	fixed bus shunt data
	OPT = 5	switched shunt data
	OPT = 6	non-transformer branch data
	OPT = 7	two-winding transformer data
	OPT = 8	three-winding transformer data
	OPT = 9	zero sequence mutual data
	OPT = 10	non-conforming source table data
	OPT = 11	non-conforming source machine data
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid OPT value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.11. FLAT

This API routine is obsolete. It has been replaced by the API routine flat_2 .

Use this API to specify various levels of flat conditions in preparation for fault analysis calculations (activity FLAT).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_FLAT  OPTIONS(1)..OPTIONS(4)  VALUES(1)  VALUES(2)
```

Python command syntax:

```
ierr = flat(options, values)
```

Fortran command syntax:

```
CALL FLATAPI(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (4)	array of 4 elements specifying boundary condition options(input).	
	OPTIONS(1)	flat option (0 by default)
	OPTIONS(1) = 0	only change voltages (activity FLAT)
	OPTIONS(1) = 1	set classical fault analysis conditions (activity FLAT,CL)
	OPTIONS(1) = 2	set conditions in preparation for IEC standard 909 calculations (activity FLAT,IEC)
	OPTIONS(2)	tap ratio treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(2) = 0	leave tap ratios unchanged
	OPTIONS(2) = 1	set tap ratios to 1.0
	OPTIONS(3)	charging treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(3) = 0	leave charging unchanged
	OPTIONS(3) = 1	set charging to 0.0
	OPTIONS(4)	shunt treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(4) = 0	leave shunts unchanged
	OPTIONS(4) = 1	set line shunts to 0.0 in the positive (and hence negative) sequence, and fixed and switched shunts to 0.0 in all three sequence networks
	OPTIONS(4) = 2	set line shunts, fixed shunts, and switched shunts to 0.0 in all sequence networks, and negative and zero sequence shunt loads to 0.0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real VALUES (2)	array of 2 elements specifying boundary condition values (ignored unless OPTIONS(1) is 2)(input).	
	VALUES(1)	voltage magnitude for all buses, or 0.0 to use present voltage magnitudes (0.0 by default)
	VALUES(2)	generator power factor, or 0.0 to leave generator Mvar unchanged (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid VALUES value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.12. FLAT_2

This API routine is obsolete. It has been replaced by the API routine flat_3 .

This API routine is the second release of the flat conditions function. Use this API routine to specify various levels of flat conditions in preparation for fault analysis calculations (activity FLAT)

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_FLAT_2  OPTIONS(1)..OPTIONS(8)  VALUES(1)  VALUES(2)
```

Python command syntax:

```
ierr = flat_2(options, values)
```

Fortran command syntax:

```
CALL FLAT_2(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (8)	array of 8 elements specifying boundary condition options(input).	
	OPTIONS(1)	flat option (0 by default)
	OPTIONS(1) = 0	only change voltages (activity FLAT)
	OPTIONS(1) = 1	set classical fault analysis conditions (activity FLAT,CL)
	OPTIONS(1) = 2	set conditions in preparation for IEC standard 909 calculations (activity FLAT,IEC)
	OPTIONS(2)	tap ratio treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(2) = 0	leave tap ratios unchanged
	OPTIONS(2) = 1	set tap ratios to 1.0
	OPTIONS(3)	charging treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(3) = 0	leave charging unchanged
	OPTIONS(3) = 1	set charging to 0.0
	OPTIONS(4)	fixed bus shunt treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(4) = 0	leave fixed bus shunts unchanged
	OPTIONS(4) = 1	set fixed bus shunts to 0.0 in the positive (and hence negative) sequence
	OPTIONS(4) = 2	set fixed bus shunts to 0.0 in all sequence networks
	OPTIONS(5)	switched shunt treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(5) = 0	leave switched shunts unchanged
	OPTIONS(5) = 1	set switched shunts to 0.0 in all sequence networks

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	line shunt treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(6) = 0	leave line shunts unchanged
	OPTIONS(6) = 1	set line shunts to 0.0 in the positive (and hence negative) sequence
	OPTIONS(6) = 2	set line shunts to 0.0 in all sequence networks
	OPTIONS(7)	transformer magnetizing admittance treatment flag (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(7) = 0	leave transformer magnetizing admittances unchanged
	OPTIONS(7) = 1	set transformer magnetizing admittances to 0.0 in the positive (and hence negative) sequence
	OPTIONS(8)	load treatment flag; this option applies to the constant power, current, and admittance load characteristic data (specified on Load Data records in the Power Flow Raw Data File), and to the exceptional negative and zero sequence load data (specified on Load Data records in the Sequence Data File) (ignored if OPTIONS(1) is 0) (0 by default)
	OPTIONS(8) = 0	leave loads unchanged
	OPTIONS(8) = 1	set the constant power, current and admittance load to 0.0 in all sequence networks; model (GNEG + j BNEG) in the negative sequence, and (GZERO + j BZERO) in the zero sequence
	OPTIONS(8) = 2	set the constant power, current and admittance load to 0.0 in all sequence networks, and (GNEG + j BNEG) to 0.0 in the negative sequence; model (GZERO + j BZERO) in the zero sequence
	OPTIONS(8) = 3	set the constant power, current and admittance load to 0.0 in all sequence networks, (GNEG + j BNEG) to 0.0 in the negative sequence, and (GZERO + j BZERO) to 0.0 in the zero sequence
Real VALUES (2)	array of 2 elements specifying boundary condition values (ignored unless OPTIONS(1) is 2)(input).	
	VALUES(1)	voltage magnitude for all buses, or 0.0 to use present voltage magnitudes (0.0 by default)
	VALUES(2)	generator power factor, or 0.0 to leave generator Mvar unchanged (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid OPTIONS value
IERR = 2	invalid VALUES value
IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.13. FLAT_3

This API routine is the third release of the flat conditions function. Use this API routine to specify various levels of flat conditions in preparation for fault analysis calculations (activity FLAT)

This API routine was first introduced in release 36.2.

Batch command syntax:

```
BAT_FLAT_3  OPTIONS(1)..OPTIONS(8)  VALUES(1)  VALUES(2)
```

Python command syntax:

```
ierr = flat_3(options, values)
```

Fortran command syntax:

```
CALL FLAT_3(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (11)	array of 11 elements specifying boundary condition options(input).	
	OPTIONS(1)	flat option (alias is method) (0 by default)
	OPTIONS(1) = 0	only change voltages (activity FLAT)
	OPTIONS(1) = 1	set classical fault analysis conditions (activity FLAT,CL)
	OPTIONS(1) = 2	set conditions in preparation for IEC standard 60909 calculations (activity FLAT,IEC)
	OPTIONS(1) = 3	User Specified Network Changes
	OPTIONS(2)	Transformer tap ratio and phase angle treatment flag (ignored if OPTIONS(1) is 0) (alias is tpunty) (0 by default)
	OPTIONS(2) = 0	leave tap ratios unchanged and set phase shift angles to 0
	OPTIONS(2) = 1	set tap ratios to 1.0 pu and phase shift angles to 0
	OPTIONS(2) = 2	set tap ratios to 1.0 pu and phase shift angles unchanged
	OPTIONS(2) = 3	set tap ratios to nominal winding kV and phase shift angle unchanged
	OPTIONS(2) = 4	set tap ratios and phase shift angle unchanged
	OPTIONS(3)	line charging treatment flag (ignored if OPTIONS(1) is 0) (alias is lnchrg) (0 by default)
	OPTIONS(3) = 0	leave line charging unchanged
	OPTIONS(3) = 1	set line charging to 0.0 in the positive and negative sequence
	OPTIONS(3) = 2	set line charging to 0.0 in all sequence networks
	OPTIONS(4)	fixed bus shunt treatment flag (ignored if OPTIONS(1) is 0) (alias is fxshop) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(4) = 0	leave fixed bus shunts unchanged
OPTIONS(4) = 1	set fixed bus shunts to 0.0 in the positive and negative sequence
OPTIONS(4) = 2	set fixed bus shunts to 0.0 in all sequence networks
OPTIONS(5)	switched shunt treatment flag (ignored if OPTIONS(1) is 0) (alias is swshop) (0 by default)
OPTIONS(5) = 0	leave switched shunts unchanged
OPTIONS(5) = 1	set switched shunts to 0.0 in the positive and negative sequence
OPTIONS(5) = 2	set switched shunts to 0.0 in all sequence networks
OPTIONS(6)	line shunt treatment flag (ignored if OPTIONS(1) is 0) (alias is lnshop) (0 by default)
OPTIONS(6) = 0	leave line shunts unchanged
OPTIONS(6) = 1	set line shunts to 0.0 in the positive and negative sequence
OPTIONS(6) = 2	set line shunts to 0.0 in all sequence networks
OPTIONS(7)	transformer magnetizing admittance treatment flag (ignored if OPTIONS(1) is 0) (alias is magadm) (0 by default)
OPTIONS(7) = 0	leave transformer magnetizing admittances unchanged
OPTIONS(7) = 1	set transformer magnetizing admittances to 0.0 in the positive and negative sequence
OPTIONS(7) = 2	set transformer magnetizing admittances to 0.0 in all sequence networks
OPTIONS(8)	load treatment flag; this option applies to the constant power, current, and admittance load characteristic data (specified on Load Data records in the Power Flow Raw Data File), and to the exceptional negative and zero sequence load data (specified on Load Data records in the Sequence Data File) (ignored if OPTIONS(1) is 0) (alias is loadop) (0 by default)
OPTIONS(8) = 0	leave loads unchanged
OPTIONS(8) = 1	set the constant power, current and admittance load to 0.0 in all sequence networks; model (GNEG + j BNEG) in the negative sequence, and (GZERO + j BZERO) in the zero sequence
OPTIONS(8) = 2	set the constant power, current and admittance load to 0.0 in all sequence networks, and (GNEG + j BNEG) to 0.0 in the negative

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		sequence; model (GZERO + j BZERO) in the zero sequence
	OPTIONS(8) = 3	set the constant power, current and admittance load to 0.0 in all sequence networks, (GNEG + j BNEG) to 0.0 in the negative sequence, and (GZERO + j BZERO) to 0.0 in the zero sequence
	OPTIONS(9)	zero sequence transformer impedance correction option (ignored if OPTIONS(1) is 0) (alias is zcorec) (1 by default)
	OPTIONS(9) = 0	ignore
	OPTIONS(9) = 1	apply
	OPTIONS(10)	synchronous and asynchronous machines power option (ignored if OPTIONS(1) is 0) (alias is machpq) (0 by default)
	OPTIONS(10) = 0	use real and reactive power outputs from power flow solution
	OPTIONS(10) = 1	set real and reactive power outputs to 0.0
Real VALUES (2)	array of 2 elements specifying boundary condition values (ignored unless OPTIONS(1) is 2)(input).	
	VALUES(1)	voltage magnitude for all buses, or 0.0 to use present voltage magnitudes (alias is busvlt) (0.0 by default)
	VALUES(2)	generator power factor, or 0.0 to leave generator Mvar unchanged (alias is genpwf) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid FLAT method value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.14. IECS

This API routine is obsolete. It has been replaced by the API routine iecs_2 .

Use this API to apply a series of faults at various locations in the working case to compute short circuit currents according to IEC 60909 standard.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_IECS  SID ALL STATUS(1)..STATUS(8) BRKTIM IECFILE FCDFILE
```

Python command syntax:

```
ierr = iecs(sid, all, status, brktim, iecfile, fcdfile)
```

Fortran command syntax:

```
CALL IECSAPI(SID, ALL, STATUS, BRKTIM, IECFILE, FCDFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses
	ALL = 0	process only buses in subsystem SID
Integer STATUS (8)	array of 8 elements specifying integer options(input).	
	STATUS(1)	three phase faults (alias is flt3ph) (0 by default)
	STATUS(1) = 0	omit
	STATUS(1) = 1	include
	STATUS(2)	line-to-ground faults (alias is fltlg) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-line-to-ground faults (alias is fltllg) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	line-to-line faults (alias is fltll) (0 by default)
	STATUS(4) = 0	omit
	STATUS(4) = 1	include
	STATUS(5)	report option (alias is rptop) (1 by default)
	STATUS(5) = 1	total fault currents
	STATUS(5) = 2	fault contributions to N levels away

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 3	total fault currents and fault contributions to N levels away
	STATUS(6)	number of levels back for contributions (alias is rptlvl) (0 by default)
	STATUS(6) >= 0	used if STATUS(5) = 2 or 3
	STATUS(7)	location of fault (alias is fltloc) (0 by default)
	STATUS(7) = 1	fault at network bus
	STATUS(7) = 2	fault at LV bus of Power Station Unit (PSU)
	STATUS(7) = 3	fault at auxiliary transformer (connected to PSU) LV bus
	STATUS(8)	report total fault currents in descending order of magnitude (AMPS) (0 by default)
	STATUS(8) = 0	do not order
	STATUS(8) = 1	descending order
Real BRKTIM	breaker contact parting time in seconds (input; 5/BaseFrequency).	
Character*260 IECFILE	IEC Fault Calculation Data (.iec) file (input; blank by default).	
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.15. IECS_2

This API routine is obsolete. It has been replaced by the API routine iecs_3 .

This API executes the second release of the IEC short circuit calculations function. This API is used to apply a series of faults at various locations in the working case to compute short circuit currents according to IEC 60909 standard.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_IECS_2  SID ALL STATUS(1)..STATUS(14) BRKTIM IECFILE FCDFILE SCFILE
```

Python command syntax:

```
ierr = iecs_2(sid, all, status, brktim, iecfile, fcdfile, scfile)
```

Fortran command syntax:

```
CALL IECSAPI2(SID, ALL, STATUS, BRKTIM, IECFILE, FCDFILE, SCFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses
	ALL = 0	process only buses in subsystem SID
Integer STATUS (14)	array of 14 elements specifying integer options(input).	
	STATUS(1)	three phase faults (alias is flt3ph) (0 by default)
	STATUS(1) = 0	omit
	STATUS(1) = 1	include
	STATUS(2)	line-to-ground faults (alias is fltlg) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-line-to-ground faults (alias is fltllg) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	line-to-line faults (alias is fltll) (0 by default)
	STATUS(4) = 0	omit
	STATUS(4) = 1	include
	STATUS(5)	report option (alias is rptop) (1 by default)
	STATUS(5) = 1	total fault currents
	STATUS(5) = 2	fault contributions to N levels away

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(5) = 3	total fault currents and fault contributions to N levels away
STATUS(6)	number of levels back for contributions (alias is rptlvl) (0 by default)
STATUS(6) >= 0	used if STATUS(5) = 2 or 3
STATUS(7)	location of fault (alias is fltloc) (1 by default)
STATUS(7) = 1	fault at network bus
STATUS(7) = 2	fault at LV bus of Power Station Unit (PSU)
STATUS(7) = 3	fault at auxiliary transformer (connected to PSU) LV bus
STATUS(8)	line-out faults (alias is linout) (0 by default)
STATUS(8) = 0	omit
STATUS(8) = 1	include
STATUS(9)	line-end faults (alias is linend) (0 by default)
STATUS(9) = 0	omit
STATUS(9) = 1	include
STATUS(10)	transformer tap ratios and phase angles option (alias is tpunty) (0 by default)
STATUS(10) = 0	leave tap ratios and phase angles unchanged
STATUS(10) = 1	set tap ratios 1.0 pu and phase angles to 0.0 deg
STATUS(11)	line charging option (alias is lnchrg) (1 by default)
STATUS(11) = 1	set to 0.0 in positive and negative sequences
STATUS(11) = 2	set to 0.0 in all sequences
STATUS(11) = 3	leave line charging unchanged
STATUS(12)	loads, fixed and switched shunts and and magnetizing admittance option (alias is shntop) (1 by default)
STATUS(12) = 1	set to 0.0 in the positive and negative sequences
STATUS(12) = 2	set to 0.0 in all sequences
STATUS(12) = 3	leave unchanged
STATUS(13)	dc line and FACTS device option (alias is dcload) (0 by default)
STATUS(13) = 0	blocked
STATUS(13) = 1	represent as load
STATUS(14)	zero sequence transformer impedance correction option (alias is zcorec) (0 by default)
STATUS(14) = 0	ignore
STATUS(14) = 1	apply

Real BRKTIM	breaker contact parting time in seconds (input; 5/BaseFrequency).
Character*260 IECFILE	IEC Fault Calculation Data (.iec) file (input; blank by default).
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 SCFILE	Results (.sc) file (input; blank by default). scfile = temporary file created scfile = nooutput no file created
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.16. IECS_3

This API routine is obsolete. It has been replaced by the API routine iecs_4 .

This API executes the third release of the IEC short circuit calculations function. This API is used to apply a series of faults at various locations in the working case to compute short circuit currents according to IEC 60909 standard.

This API routine was first introduced in release 32.0.5.

Batch command syntax:

```
BAT_IECS_3  SID ALL STATUS(1)..STATUS(15) VALUES(1) VALUES(2) IECFILE
 FCDFILE SCFILE
```

Python command syntax:

```
ierr = iecs_3(sid, all, status, values, iecfile, fcdfile, scfile)
```

Fortran command syntax:

```
CALL IECSAPI3(SID, ALL, STATUS, VALUES, IECFILE, FCDFILE, SCFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses
	ALL = 0	process only buses in subsystem SID
Integer STATUS (15)	array of 15 elements specifying integer options(input).	
	STATUS(1)	three phase faults (alias is flt3ph) (0 by default)
	STATUS(1) = 0	omit
	STATUS(1) = 1	include
	STATUS(2)	line-to-ground faults (alias is fltlg) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-line-to-ground faults (alias is fltllg) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	line-to-line faults (alias is fltll) (0 by default)
	STATUS(4) = 0	omit
	STATUS(4) = 1	include
	STATUS(5)	report option (alias is rptop) (1 by default)
	STATUS(5) = 1	total fault currents
	STATUS(5) = 2	fault contributions to N levels away

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(5) = 3	total fault currents and fault contributions to N levels away
STATUS(6)	number of levels back for contributions (alias is rptlvl) (0 by default)
STATUS(6) >= 0	used if STATUS(5) = 2 or 3
STATUS(7)	location of fault (alias is fltloc) (1 by default)
STATUS(7) = 1	fault at network bus
STATUS(7) = 2	fault at LV bus of Power Station Unit (PSU)
STATUS(7) = 3	fault at auxiliary transformer (connected to PSU) LV bus
STATUS(8)	line-out faults (alias is linout) (0 by default)
STATUS(8) = 0	omit
STATUS(8) = 1	include
STATUS(9)	line-end faults (alias is linend) (0 by default)
STATUS(9) = 0	omit
STATUS(9) = 1	include
STATUS(10)	transformer tap ratios and phase angles option (alias is tpunty) (0 by default)
STATUS(10) = 0	leave tap ratios and phase angles unchanged
STATUS(10) = 1	set tap ratios 1.0 pu and phase angles to 0.0 deg
STATUS(11)	line charging option (alias is lnchrg) (1 by default)
STATUS(11) = 1	set to 0.0 in positive and negative sequences
STATUS(11) = 2	set to 0.0 in all sequences
STATUS(11) = 3	leave line charging unchanged
STATUS(12)	loads, fixed and switched shunts and and magnetizing admittance option (alias is shntop) (1 by default)
STATUS(12) = 1	set to 0.0 in the positive and negative sequences
STATUS(12) = 2	set to 0.0 in all sequences
STATUS(12) = 3	leave unchanged
STATUS(13)	dc line and FACTS device option (alias is dcload) (0 by default)
STATUS(13) = 0	blocked
STATUS(13) = 1	represent as load
STATUS(14)	zero sequence transformer impedance correction option (alias is zcorec) (0 by default)
STATUS(14) = 0	ignore
STATUS(14) = 1	apply
STATUS(15)	option for Voltage Factor C (alias is cfactor) (1 by default)
STATUS(15) = 1	Voltage Factor C for maximum fault currents (default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(15) = 2	Voltage Factor C for minimum fault currents
	STATUS(15) = 3	Voltage Factor C as specified by 'vfactorc' value
Real VALUES (2)	array of 2 elements specifying real values(input).	
	VALUES(1)	breaker contact parting time in seconds (alias is brktim) (5/base Frequency by default)
	VALUES(2)	user specified voltage factor c (alias is ucfactor) (1.0 by default) This is used only when STATUS(15)=3)
Character*260 IECFILE	IEC Fault Calculation Data (.iec) file (input; blank by default).	
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).	
Character*260 SCFILE	Results (.sc) file (input; blank by default).	
	scfile =	temporary file created
	scfile = nooutput	no file created
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.17. IECS_4

This API executes the fourth release of the IEC short circuit calculations function. This API is used to apply a series of faults at various locations in the working case to compute short circuit currents according to IEC 60909 standard.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IECS_4  SID ALL STATUS(1)..STATUS(17) VALUES(1) VALUES(2) IECFILE
 FCDFILE SCFILE
```

Python command syntax:

```
ierr = iecs_4(sid, all, status, values, iecfile, fcdfile, scfile)
```

Fortran command syntax:

```
CALL IECSAPI4(SID, ALL, STATUS, VALUES, IECFILE, FCDFILE, SCFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses
	ALL = 0	process only buses in subsystem SID
Integer STATUS (17)	array of 17 elements specifying integer options(input).	
	STATUS(1)	three phase faults (alias is flt3ph) (0 by default)
	STATUS(1) = 0	omit
	STATUS(1) = 1	include
	STATUS(2)	line-to-ground faults (alias is fltlg) (0 by default)
	STATUS(2) = 0	omit
	STATUS(2) = 1	include
	STATUS(3)	line-line-to-ground faults (alias is fltllg) (0 by default)
	STATUS(3) = 0	omit
	STATUS(3) = 1	include
	STATUS(4)	line-to-line faults (alias is fltll) (0 by default)
	STATUS(4) = 0	omit
	STATUS(4) = 1	include
	STATUS(5)	report option (alias is rptop) (1 by default)
	STATUS(5) = 0	fault current summary table
	STATUS(5) = 1	total fault currents
	STATUS(5) = 2	fault contributions to N levels away

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(5) = 3	total fault currents and fault contributions to N levels away
STATUS(6)	number of levels back for contributions (alias is rptlvl) (0 by default)
STATUS(6) >= 0	used if STATUS(5) = 2 or 3
STATUS(7)	location of fault (alias is fltloc) (0 by default)
STATUS(7) = 0	fault at network bus
STATUS(7) = 1	fault at LV bus of Power Station Unit (PSU)
STATUS(7) = 2	fault at auxiliary transformer (connected to PSU) LV bus
STATUS(8)	line-out faults (alias is linout) (0 by default)
STATUS(8) = 0	omit
STATUS(8) = 1	include
STATUS(9)	line-end faults (alias is linend) (0 by default)
STATUS(9) = 0	omit
STATUS(9) = 1	include
STATUS(10)	transformer tap ratios and phase shift angles option (alias is tpunty) (0 by default)
STATUS(10) = 0	leave tap ratios and phase shift angles unchanged
STATUS(10) = 1	set tap ratios to 1.0 pu and phase shift angles to 0 deg
STATUS(10) = 2	set tap ratios to 1.0 pu and phase shift angles unchanged
STATUS(10) = 3	set tap ratios unchanged and phase shift angles to 0 deg
STATUS(11)	line charging option (alias is lnchrg) (1 by default)
STATUS(11) = 0	leave unchanged
STATUS(11) = 1	set to 0.0 in the positive and negative sequences
STATUS(11) = 2	set to 0.0 in all sequences
STATUS(12)	line, fixed and switched shunts, and magnetizing admittance option (alias is shntop) (1 by default)
STATUS(12) = 0	leave unchanged
STATUS(12) = 1	set to 0.0 in the positive and negative sequences
STATUS(12) = 2	set to 0.0 in all sequences
STATUS(13)	dc line and FACTS device option (alias is dcload) (0 by default)
STATUS(13) = 0	blocked
STATUS(13) = 1	represent as load
STATUS(14)	zero sequence transformer impedance correction option (alias is zcorec) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(14) = 0	ignore
	STATUS(14) = 1	apply
	STATUS(15)	option for Voltage Factor C (alias is cfactor) (0 by default)
	STATUS(15) = 0	Voltage Factor C for maximum fault currents
	STATUS(15) = 1	Voltage Factor C for minimum fault currents
	STATUS(15) = 2	Voltage Factor C as specified by 'ucfactor' value for maximum fault current calculations
	STATUS(15) = 3	Voltage Factor C as specified by 'ucfactor' value for minimum fault current calculations
	STATUS(16)	load option (alias is loadop) (1 by default)
	STATUS(16) = 0	leave unchanged
	STATUS(16) = 1	set to 0.0 in the positive and negative sequences
	STATUS(16) = 2	set to 0.0 in all sequences
	STATUS(17)	fault analysis generator reactance option; ignored if generators are already "converted" (alias is genxop) (0 by default)
	STATUS(17) = 0	use subtransient reactance
	STATUS(17) = 1	use transient reactance
	STATUS(17) = 2	use synchronous reactance
Real VALUES (2)	array of 2 elements specifying real values(input).	
	VALUES(1)	breaker contact parting time in seconds (alias is brktim) (5/base Frequency by default)
	VALUES(2)	user specified voltage factor c (alias is ucfactor) (1.0 by default) This is used only when STATUS(15)=2 or 3)
Character*260 IECFILE	IEC Fault Calculation Data (.iec) file (input; blank by default).	
Character*260 FCDFILE	Fault Control (.fcd) file (input; blank by default).	
Character*260 SCFILE	Results (.sc) file (input; blank by default).	
	scfile =	temporary file created
	scfile = nooutput	no file created
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid VALUES value
	IERR = 5	no faults specified
	IERR = 6	error opening IECFILE
	IERR = 7	error opening FCDFILE
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	error opening SCFILE
IERR = 10	SCFILE error
IERR = 11	bus SC results not available
IERR = 12	branch SC results not available
IERR = 13	three winding transformer SC results not available
IERR = 14	incorrect SCFILE
IERR = 15	SCFILE not allowed
IERR = 16	Fault analysis invalid, in-service induction machines stalled or tripped state
IERR = 17	Line end fault invalid--bus table is full
IERR = 18	SC result file size exceeded. Define smaller size bus subsystems and re-run if SCFILE is required.
IERR = 19	topology error - bus(es) with bus type code and series element status inconsistencies
IERR = 20	Number of branches at faulted bus are greater than NBRLINKS (only arrbox.fault module)
IERR = 21	Re-run by specifying higher NBRLNKS value (only arrbox.fault module)
IERR = 22	Network case has number of MOV and/or NCS, cannot use arrbox.iecs_currents, use arrbox.fault module

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.18. IECS_SCFILE

This API reads IECS short circuit results from SCFILE and produces same IECS output report that was generated when SCFILE was created. SCFILE is read into PSSE memory to enable displaying IECS results onto slider diagrams. For this API to work properly, it is required that working case in PSSE memory should be the same Saved case that was used when creating the SCFILE.

This API routine was first introduced in release 32.0.3.

Batch command syntax:

```
BAT_IECS_SCFILE  SCFILE
```

Python command syntax:

```
ierr = iecs_scfile(scfile)
```

Fortran command syntax:

```
CALL IECS_SCFILE(SCFILE,IERR)
```

Where:

Character*260 SCFILE	IEC Results (.sc) file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 7	prerequisite requirements for function not met
	IERR = 9	error opening SCFILE
	IERR = 10	SCFILE error
	IERR = 11	bus SC results not available
	IERR = 12	branch SC results not available
	IERR = 13	three winding transformer SC results not available
	IERR = 14	incorrect SCFILE
	IERR = 15	SCFILE not allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.19. NEWSEQ

Use this API to initialize the fault analysis data arrays, set them to default values, and set the flag indicating that sequence data is present in the working case.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_NEWSEQ
```

Python command syntax:

```
ierr = newseq()
```

Fortran command syntax:

```
CALL NEWSEQ(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.20. SCEQ

Use this API to construct network equivalents of the positive and zero sequence networks and calculates source impedances at equivalent source nodes for all three sequences in preparation for the unbalanced fault analysis activities of PSSE.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SCEQ  SID  ALL  STATUS  THRESH  RFILE  SFILE
```

Python command syntax:

```
ierr = sceq(sid, all, status, thresh, rfile, sfile)
```

Fortran command syntax:

```
CALL SCEQNW(SID, ALL, STATUS, THRESH, RFILE, SFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	transformer impedance correction option (input; 1 by default).	
	STATUS = 0	do not apply to zero sequence
	STATUS = 1	apply to zero sequence
Real THRESH	equivalent branch threshold tolerance (input; 10.0).	
Character*260 RFILE	power flow raw data file; blank for none (input; blank by default).	
Character*260 SFILE	sequence data file; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	case is not subject to classical assumptions
	IERR = 5	generators are not converted
	IERR = 6	error writing the working case to a temporary file
	IERR = 7	no buses being retained
	IERR = 8	no buses being processed for equivalence
	IERR = 9	too many branches in the equivalent
	IERR = 10	bus type codes other than 1, 2, 4, 5 and 6 present

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 11	substations present in subsystem to be equivalenced
IERR = 12	error opening RFILE or SFILE
IERR = 13	error opening temporary file
IERR = 14	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.21. SCGR

This API routine is obsolete. It has been replaced by the API routine ascc_2 .

This API no longer performs any action. Usage in previous releases described below. Use this API routine to calculate three phase and line-to-ground fault solutions. Results may be programmatically accessed via the internally accessible fetch routines SCGTXT, SCGBUS, SCGBRN, and SCG3WN.
Usage is:
- invoke SCGR API with APIOPT = 0;
- multiple calls to fetch routines;
- optionally, repeat of above steps for different fault and/or different pre-fault conditions;
- final invocation of SCGR API with APIOPT = 1.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SCGR SID ALL APIOPT STATUS(1)..STATUS(8)
```

Python command syntax:

```
ierr = scgr(sid, all, apiopt, status)
```

Fortran command syntax:

```
CALL SCGRAPI(SID, ALL, APIOPT, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer APIOPT	mode of operation of SCGRAPI (input; 0 by default).	
	APIOPT = 0	apply fault calculations and fill arrays used by fetch routines
	APIOPT = 1	deallocate arrays used by fetch routines
Integer STATUS (8)	array of 8 elements specifying fault and options(input).	
	STATUS(1)	fault type (0 by default)
	STATUS(1) = 0	three-phase
	STATUS(1) = 1	line-to-ground
	STATUS(2)	L-G fault (ignored if STATUS(1) is 0) (0 by default)
	STATUS(2) = 0	A phase
	STATUS(2) = 1	3 * I0
	STATUS(3)	fault location (0 by default)
	STATUS(3) = 0	all buses in subsystem

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(3) > 0	bus number of the subsystem bus to be faulted
	STATUS(4)	flat conditions option (0 by default)
	STATUS(4) = 0	working case conditions
	STATUS(4) = 1	flat conditions
	STATUS(5)	double-precision fault calculation option (ignored, but must be specified as 0 or 1; 0 by default; double precision always used)
	STATUS(6)	dc line and FACTS device option (ignored if STATUS(4) is 1) (0 by default)
	STATUS(6) = 0	block
	STATUS(6) = 1	represent as load
	STATUS(7)	transformer impedance correction option (ignored if STATUS(1) is 0 or STATUS(4) is 1 (1 by default)
	STATUS(7) = 0	do not apply to zero sequence
	STATUS(7) = 1	apply to zero sequence
	STATUS(8)	branch results option (0 by default)
	STATUS(8) = 0	currents
	STATUS(8) = 1	apparent impedances
Integer IERR	error code (output).	
	IERR = 1	SCGR is obsolete; use "Fault>Automatic sequencing fault calculation (ASCC).." (API routine ASCC_2)
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.22. SCMU

This API routine is obsolete. It has been replaced by the API routine scmu_2 .

Use this API to calculate simultaneous unbalances. It must be called once with APIOPT set to 1, once for each of the simultaneous faults to be applied with APIOPT set to 2, and once with APIOPT set to 3.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SCMU  APIOPT  OPTIONS(1)..OPTIONS(7)  RVAL(1)..RVAL(5)  CKTARG
```

Python command syntax:

```
ierr = scmu(apiopt, options, rval, cktarg)
```

Fortran command syntax:

```
CALL SCMUAPI(APIOPT, OPTIONS, RVAL, CKTARG, IERR)
```

Where:

Integer APIOPT	mode of operation (input; no default allowed).	
	APIOPT = 1	Initialize; automatically runs activity SEQD if the sequence network setup option setting is enabled.
	APIOPT = 2	Specify an unbalance to be applied; must be called once for each simultaneous unbalance to be imposed.
	APIOPT = 3	Solve in the presence of the specified unbalances and terminate.
	APIOPT = 4	Remove specified unbalances (cancels any unbalances specified by any preceding APIOPT 2 references, and leaves memory as it is immediately following an APIOPT 1 reference).
Integer OPTIONS (7)	array of 7 elements specifying options and input values to be used in the current calculation phase. It is ignored when APIOPT is 3 or 4. When APIOPT is 1, OPTIONS(1), OPTIONS(2) and OPTIONS(3) define options to be used in the construction of the sequence admittance matrices. These values are used only when the sequence network setup option is enabled, and activity SEQD is to be automatically executed.(input).	
	OPTIONS(1)	Specify it per additional conditions as below.
	When APIOPT=1, dc line and FACTS device option, 0 by default.	
	OPTIONS(1) = 0	Block and ignore
	OPTIONS(1) = 1	Convert to constant admittance load
	When APIOPT=2, define the type of unbalance to be applied, no default allowed	
	OPTIONS(1) = 1	First line-to-ground fault

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(1) = 2	Second line-to-ground fault
OPTIONS(1) = 3	First line-to-line or line-to-line-to-ground fault
OPTIONS(1) = 4	Second line-to-line or line-to-line-to-ground fault
OPTIONS(1) = 5	One phase closed
OPTIONS(1) = 6	Two phases closed
OPTIONS(1) = 7	Three-phase fault (uses fault types 2 and 4)
OPTIONS(1) = 8	Branch unbalance with one end opened
OPTIONS(1) = 9	Branch unbalance with both ends closed
OPTIONS(2)	Specify it per additional conditions as below.

When APIOPT=1, transformer impedance correction option, 1 by default.

OPTIONS(2) = 0	Do not apply to zero sequence
OPTIONS(2) = 1	Apply to zero sequence

When APIOPT=2 and OPTIONS(1)=1 or 2 (LG), no default allowed.

OPTIONS(2) = None	Bus to be faulted

When APIOPT=2 and OPTIONS(1)=3 or 4 (LLG), no default allowed.

OPTIONS(2) = None	Bus to be faulted

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), no default allowed.

OPTIONS(2) = None	One of the buses to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), no default allowed.

OPTIONS(2) = None	One of the buses to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=7 (3PH), no default allowed.

OPTIONS(2) = None	Bus to be faulted

When APIOPT=2 and OPTIONS(1)=8 (one end open), no default allowed.

OPTIONS(2) = None	One of the buses connected by the branch to be faulted

When APIOPT=2 and OPTIONS(1)=9 (both ends closed), no default allowed.

OPTIONS(2) = None	One of the buses connected by the branch to be faulted
OPTIONS(3)	Specify it per additional conditions as below.

When APIOPT=1, fault analysis generator reactance option, ignored if generators are already "converted", 0 by default.

OPTIONS(3) = 0	Use subtransient reactance
OPTIONS(3) = 1	Use transient reactance
OPTIONS(3) = 2	Use synchronous reactance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

When APIOPT=2 and OPTIONS(1)=1 or 2 (LG), faulted phase, 1 by default.

OPTIONS(3) = 1	Phase A
OPTIONS(3) = 2	Phase B
OPTIONS(3) = 3	Phase C

When APIOPT=2 and OPTIONS(1)=3 or 4 (LLG), excluded phase, 1 by default.

OPTIONS(3) = 1	Phase A
OPTIONS(3) = 2	Phase B
OPTIONS(3) = 3	Phase C

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), no default allowed.

OPTIONS(3) = None	The other bus to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), no default allowed.

OPTIONS(3) = None	The other bus to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=8 (one end open), no default allowed.

OPTIONS(3) = None	Another bus connected by the branch to be faulted

When APIOPT=2 and OPTIONS(1)=9 (both ends closed), no default allowed.

OPTIONS(3) = None	The other bus connected by the branch to be faulted

OPTIONS(4) Specify it per additional conditions as below.

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), phase to be closed, 1 by default.

OPTIONS(4) = 1	Phase A
OPTIONS(4) = 2	Phase B
OPTIONS(4) = 3	Phase C

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), excluded phase, 1 by default.

OPTIONS(4) = 1	Phase A
OPTIONS(4) = 2	Phase B
OPTIONS(4) = 3	Phase C

When APIOPT=2 and OPTIONS(1)=8 (one end open), 0 by default.

OPTIONS(4) = None	Third bus connected by the three-winding transformer to be faulted, or zero for a two terminal branch

When APIOPT=2 and OPTIONS(1)=9 (both ends closed), type of in-line fault to be applied (1 by default).

OPTIONS(4) = 1	Line-to-ground

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(4) = 2	Line-to-line or line-to-line-to-ground fault
OPTIONS(4) = 3	Three-phase
OPTIONS(5)	Specify it per additional conditions as below.

When APIOPT=2 and OPTIONS(1)=8 (one end open), type of in-line fault to be applied (1 by default).

OPTIONS(5) = 1	Line-to-ground
OPTIONS(5) = 2	Line-to-line or line-to-line-to-ground fault
OPTIONS(5) = 3	Three-phase

When APIOPT=2, OPTIONS(1)=9 (both ends closed) and OPTIONS(4)=1, faulted phase (1 by default).

OPTIONS(5) = 1	Phase A
OPTIONS(5) = 2	Phase B
OPTIONS(5) = 3	Phase C

When APIOPT=2, OPTIONS(1)=9 (both ends closed) and OPTIONS(4)=2, excluded phase (1 by default).

OPTIONS(5) = 1	Phase A
OPTIONS(5) = 2	Phase B
OPTIONS(5) = 3	Phase C
OPTIONS(6)	Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=8 (one end open) and OPTIONS(5)=1, faulted phase (1 by default).

OPTIONS(6) = 1	Phase A
OPTIONS(6) = 2	Phase B
OPTIONS(6) = 3	Phase C

When APIOPT=2, OPTIONS(1)=8 (one end open) and OPTIONS(5)=2, excluded phase (1 by default).

OPTIONS(6) = 1	Phase A
OPTIONS(6) = 2	Phase B
OPTIONS(6) = 3	Phase C
OPTIONS(7)	Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(2) by default.

OPTIONS(7) = None	Opened end bus (must be identical to one of OPTIONS(2), OPTIONS(3), or, if it is non-zero, OPTIONS(4))

Real RVAL (5)

array of 5 elements specifying input values used to define the current fault; ignored except when APIOPT is 2. The elements of RVAL are dependent on the value specified for OPTIONS(1).(input).

RVAL(1)	Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=1 or 2 (LG), 0.0 by default.

RVAL(1) = None	Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

RVAL(1) = None Resistance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=5 (1 phase closed), 0.0 by default.

RVAL(1) = None Resistance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=6 (2 phases closed), 0.0 by default.

RVAL(1) = None Resistance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=1, 0.0 by default.

RVAL(1) = None Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.

RVAL(1) = None Resistance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=1, 0.0 by default.

RVAL(1) = None Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.

RVAL(1) = None Resistance of the complex L-L fault impedance in pu

RVAL(2) Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=1 or 2 (LG), 0.0 by default.

RVAL(2) = None Reactance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

RVAL(2) = None Reactance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=5 (1 phase closed), 0.0 by default.

RVAL(2) = None Reactance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=6 (2 phases closed), 0.0 by default.

RVAL(2) = None Reactance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=1, 0.0 by default.

RVAL(2) = None Reactance of the complex L-G fault impedance in pu

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.

RVAL(2) = None — Reactance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=1, 0.0 by default.

RVAL(2) = None — Reactance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.

RVAL(2) = None — Reactance of the complex L-L fault impedance in pu

RVAL(3) — Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

RVAL(3) = None — Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.

RVAL(3) = None — Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.

RVAL(3) = None — Resistance of the complex L-G fault impedance in pu

RVAL(4) — Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

RVAL(4) = None — Reactance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.

RVAL(4) = None — Reactance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.

RVAL(4) = None — Reactance of the complex L-G fault impedance in pu

RVAL(5) — Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=8 (one end open), 1.0 by default.

RVAL(5) = None — fault location from closed end bus in pu of total line length; must be 1.0 if the branch is a transformer

When APIOPT=2, OPTIONS(1)=9 (both ends closed), 0.5 by default.

RVAL(5) = None — fault location from bus OPTIONS(2) in pu of total line length

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*2 CKTARG	circuit identifier of branch to be faulted; ignored except when APIOPT is 2 and OPTIONS(1) is 8 or 9 (input; '1' by default).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.23. SCMU_2

Use this API to calculate simultaneous unbalances. It must be called once with APIOPT set to 1, once for each of the simultaneous faults to be applied with APIOPT set to 2, and once with APIOPT set to 3.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_SCMU_2  APIOPT  OPTIONS(1)..OPTIONS(7)  RVAL(1)..RVAL(5)  CKTARG
```

Python command syntax:

```
ierr = scmu_2(apiopt, options, rval, cktarg)
```

Fortran command syntax:

```
CALL SCMU_2(APIOPT, OPTIONS, RVAL, CKTARG, IERR)
```

Where:

Integer APIOPT — mode of operation (input; no default allowed).

APIOPT = 1	Initialize; automatically runs activity SEQD if the sequence network setup option setting is enabled.
APIOPT = 2	Specify an unbalance to be applied; must be called once for each simultaneous unbalance to be imposed.
APIOPT = 3	Solve in the presence of the specified unbalances and terminate.
APIOPT = 4	Remove specified unbalances (cancels any unbalances specified by any preceding APIOPT 2 references, and leaves memory as it is immediately following an APIOPT 1 reference).

Integer OPTIONS (7) — array of 7 elements specifying options and input values to be used in the current calculation phase. It is ignored when APIOPT is 3 or 4. When APIOPT is 1, OPTIONS(1), OPTIONS(2) and OPTIONS(3) define options to be used in the construction of the sequence admittance matrices. These values are used only when the sequence network setup option is enabled, and activity SEQD is to be automatically executed.(input).

OPTIONS(1)	Specify it per additional conditions as below.
When APIOPT=1, dc line and FACTS device option, 0 by default.	
OPTIONS(1) = 0	Block and ignore
OPTIONS(1) = 1	Convert to constant admittance load
When APIOPT=2, define the type of unbalance to be applied, no default allowed	
OPTIONS(1) = 1	First line-to-ground fault
OPTIONS(1) = 2	Second line-to-ground fault
OPTIONS(1) = 3	First line-to-line or line-to-line-to-ground fault

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTIONS(1) = 4	Second line-to-line or line-to-line-to-ground fault
OPTIONS(1) = 5	One phase closed
OPTIONS(1) = 6	Two phases closed
OPTIONS(1) = 7	Three-phase fault (uses fault types 2 and 4)
OPTIONS(1) = 8	Branch unbalance with one end opened
OPTIONS(1) = 9	Branch unbalance with both ends closed
OPTIONS(2)	Specify it per additional conditions as below.

When APIOPT=1, transformer impedance correction option, 1 by default.

OPTIONS(2) = 0	Do not apply to zero sequence
OPTIONS(2) = 1	Apply to zero sequence

When APIOPT=2 and OPTIONS(1)=1 or 2 (LG), no default allowed.

OPTIONS(2) = None	Bus to be faulted

When APIOPT=2 and OPTIONS(1)=3 or 4 (LLG), no default allowed.

OPTIONS(2) = None	Bus to be faulted

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), no default allowed.

OPTIONS(2) = None	One of the buses to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), no default allowed.

OPTIONS(2) = None	One of the buses to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=7 (3PH), no default allowed.

OPTIONS(2) = None	Bus to be faulted

When APIOPT=2 and OPTIONS(1)=8 (one end open), no default allowed.

OPTIONS(2) = None	One of the buses connected by the branch to be faulted

When APIOPT=2 and OPTIONS(1)=9 (both ends closed), no default allowed.

OPTIONS(2) = None	One of the buses connected by the branch to be faulted
OPTIONS(3)	Specify it per additional conditions as below.

When APIOPT=1, fault analysis generator reactance option, ignored if generators are already "converted", 0 by default.

OPTIONS(3) = 0	Use subtransient reactance
OPTIONS(3) = 1	Use transient reactance
OPTIONS(3) = 2	Use synchronous reactance

When APIOPT=2 and OPTIONS(1)=1 or 2 (LG), 0 by default.

OPTIONS(3) = None	Number of node whose bus section is to be faulted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

When APIOPT=2 and OPTIONS(1)=3 or 4 (LLG), 0 by default.

OPTIONS(3) = None — Number of node whose bus section is to be faulted

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), 0 by default.

OPTIONS(3) = None — Number of node whose bus section is to be connected

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), 0 by default.

OPTIONS(3) = None — Number of node whose bus section is to be connected

When APIOPT=2 and OPTIONS(1)=7 (3PH), 0 by default.

OPTIONS(3) = None — Number of node whose bus section is to be faulted

When APIOPT=2 and OPTIONS(1)=8 (one end open), no default allowed.

OPTIONS(3) = None — Another bus connected by the branch to be faulted

When APIOPT=2 and OPTIONS(1)=9 (both ends closed), no default allowed.

OPTIONS(3) = None — The other bus connected by the branch to be faulted

OPTIONS(4) — Specify it per additional conditions as below.

When APIOPT=2 and OPTIONS(1)=1 or 2 (LG), faulted phase, 1 by default.

OPTIONS(4) = 1 — Phase A

OPTIONS(4) = 2 — Phase B

OPTIONS(4) = 3 — Phase C

When APIOPT=2 and OPTIONS(1)=3 or 4 (LLG), excluded phase, 1 by default.

OPTIONS(4) = 1 — Phase A

OPTIONS(4) = 2 — Phase B

OPTIONS(4) = 3 — Phase C

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), no default allowed.

OPTIONS(4) = None — The other bus to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), no default allowed.

OPTIONS(4) = None — The other bus to be connected by the phase closed unbalance

When APIOPT=2 and OPTIONS(1)=8 (one end open), 0 by default.

OPTIONS(4) = None — Third bus connected by the three-winding transformer to be faulted, or zero for a two terminal branch

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

When APIOPT=2 and OPTIONS(1)=9 (both ends closed), type of in-line fault to be applied (1 by default).

OPTIONS(4) = 1	Line-to-ground
OPTIONS(4) = 2	Line-to-line or line-to-line-to-ground fault
OPTIONS(4) = 3	Three-phase

OPTIONS(5) Specify it per additional conditions as below.

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), 0 by default.

OPTIONS(5) = None	Number of node whose bus section is to be connected

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), 0 by default.

OPTIONS(5) = None	Number of node whose bus section is to be connected

When APIOPT=2 and OPTIONS(1)=8 (one end open), type of in-line fault to be applied (1 by default).

OPTIONS(5) = 1	Line-to-ground
OPTIONS(5) = 2	Line-to-line or line-to-line-to-ground fault
OPTIONS(5) = 3	Three-phase

When APIOPT=2, OPTIONS(1)=9 (both ends closed) and OPTIONS(4)=1, faulted phase (1 by default).

OPTIONS(5) = 1	Phase A
OPTIONS(5) = 2	Phase B
OPTIONS(5) = 3	Phase C

When APIOPT=2, OPTIONS(1)=9 (both ends closed) and OPTIONS(4)=2, excluded phase (1 by default).

OPTIONS(5) = 1	Phase A
OPTIONS(5) = 2	Phase B
OPTIONS(5) = 3	Phase C

OPTIONS(6) Specify it per additional conditions as below.

When APIOPT=2 and OPTIONS(1)=5 (1 phase closed), phase to be closed, 1 by default.

OPTIONS(6) = 1	Phase A
OPTIONS(6) = 2	Phase B
OPTIONS(6) = 3	Phase C

When APIOPT=2 and OPTIONS(1)=6 (2 phases closed), excluded phase, 1 by default.

OPTIONS(6) = 1	Phase A
OPTIONS(6) = 2	Phase B
OPTIONS(6) = 3	Phase C

When APIOPT=2, OPTIONS(1)=8 (one end open) and OPTIONS(5)=1, faulted phase (1 by default).

OPTIONS(6) = 1	Phase A
OPTIONS(6) = 2	Phase B
OPTIONS(6) = 3	Phase C

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

When APIOPT=2, OPTIONS(1)=8 (one end open) and OPTIONS(5)=2, excluded phase (1 by default).

OPTIONS(6) = 1 Phase A

OPTIONS(6) = 2 Phase B

OPTIONS(6) = 3 Phase C

OPTIONS(7) Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(2) by default.

OPTIONS(7) = None Opened end bus (must be identical to one of OPTIONS(2), OPTIONS(3), or, if it is non-zero, OPTIONS(4))

Real RVAL (5) array of 5 elements specifying input values used to define the current fault; ignored except when APIOPT is 2. The elements of RVAL are dependent on the value specified for OPTIONS(1).(input).

RVAL(1) Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=1 or 2 (LG), 0.0 by default.

RVAL(1) = None Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

RVAL(1) = None Resistance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=5 (1 phase closed), 0.0 by default.

RVAL(1) = None Resistance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=6 (2 phases closed), 0.0 by default.

RVAL(1) = None Resistance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=1, 0.0 by default.

RVAL(1) = None Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.

RVAL(1) = None Resistance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=1, 0.0 by default.

RVAL(1) = None Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.

RVAL(1) = None Resistance of the complex L-L fault impedance in pu

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

RVAL(2) Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=1 or 2 (LG), 0.0 by default.

RVAL(2) = None Reactance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

RVAL(2) = None Reactance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=5 (1 phase closed), 0.0 by default.

RVAL(2) = None Reactance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=6 (2 phases closed), 0.0 by default.

RVAL(2) = None Reactance of the complex phase impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=1, 0.0 by default.

RVAL(2) = None Reactance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.

RVAL(2) = None Reactance of the complex L-L fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=1, 0.0 by default.

RVAL(2) = None Reactance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.

RVAL(2) = None Reactance of the complex L-L fault impedance in pu

RVAL(3) Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

RVAL(3) = None Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.

RVAL(3) = None Resistance of the complex L-G fault impedance in pu

When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.

RVAL(3) = None Resistance of the complex L-G fault impedance in pu

RVAL(4) Specify it per additional conditions as below.

When APIOPT=2, OPTIONS(1)=3 or 4 (LLG), 0.0 by default.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RVAL(4) = None	Reactance of the complex L-G fault impedance in pu
	When APIOPT=2, OPTIONS(1)=8 (one end open), OPTIONS(5)=2, 0.0 by default.	
	RVAL(4) = None	Reactance of the complex L-G fault impedance in pu
	When APIOPT=2, OPTIONS(1)=9 (both ends closed), OPTIONS(4)=2, 0.0 by default.	
	RVAL(4) = None	Reactance of the complex L-G fault impedance in pu
	RVAL(5)	Specify it per additional conditions as below.
	When APIOPT=2, OPTIONS(1)=8 (one end open), 1.0 by default.	
	RVAL(5) = None	fault location from closed end bus in pu of total line length; must be 1.0 if the branch is a transformer
	When APIOPT=2, OPTIONS(1)=9 (both ends closed), 0.5 by default.	
	RVAL(5) = None	fault location from bus OPTIONS(2) in pu of total line length
Character*2 CKTARG	circuit identifier of branch to be faulted; ignored except when APIOPT is 2 and OPTIONS(1) is 8 or 9 (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	unexpected APIOPT value
	IERR = 2	invalid APIOPT value
	IERR = 3	invalid unbalance type
	IERR = 4	bus not found
	IERR = 5	bus is out-of-service
	IERR = 6	invalid phase value
	IERR = 7	branch not found
	IERR = 8	three-winding transformer not found
	IERR = 9	invalid open end bus
	IERR = 10	invalid distance factor
	IERR = 11	terminated due to invalid fault specification or application
	IERR = 12	SEQD results are not compatible with the working case
	IERR = 13	fault analysis is invalid due to the presence of in-service induction machines that are in the "stalled" or "tripped" state
	IERR = 14	bus type code and series element status inconsistencies
	IERR = 15	invalid OPTIONS value (from SEQD)
	IERR = 16	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.24. SCOP

Use this API to tabulate unbalanced fault solution output. It gives fault currents arriving at a bus. It must be preceded by the execution of activity SCMU, either interactively, from a response file, or via Python. It must then be referenced using one of the following procedures: (a) One reference with APIOPT=1, followed by any number of references with APIOPT=2, followed by one reference with APIOPT=3. (b) One reference with APIOPT=0. This automatically combines the processing of one APIOPT 1 reference, followed by one APIOPT 2 reference, followed by one APIOPT 3 reference.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SCOP SID ALL UNITS APIOPT
```

Python command syntax:

```
ierr = scop(sid, all, units, apiopt)
```

Fortran command syntax:

```
CALL SCOPAPI(SID, ALL, UNITS, APIOPT, IERR)
```

Where:

Integer SID	a valid subsystem identifier; used only when APIOPT is 0 or 2 and ALL is 0. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag; used only when APIOPT is 0 or 2 (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer UNITS	branch quantity flag; used only when APIOPT is 0 or 2 (input; 1 by default).	
	UNITS = 1	currents
	UNITS = 2	apparent impedances
	UNITS = 3	apparent admittances
Integer APIOPT	mode of operation (input; 0 by default).	
	APIOPT = 0	initialize for short circuit output, produce the output, and run post-processing clean up
	APIOPT = 1	initialize for short circuit output
	APIOPT = 2	produce output
	APIOPT = 3	post-processing clean up
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid UNITS value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	invalid APIOPT value
IERR = 5	unexpected APIOPT value
IERR = 6	SCMU results error; one of the these occurred: No SCMU results file is open; SCMU results are not compatible with the working case; Error reading the SCMU results file
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.25. SEQD

This API routine is obsolete. It has been replaced by the API routine seqd_2 .

Use this API to factorize the sequence matrices in preparation for unbalanced fault calculations.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SEQD  OPTIONS(1)  OPTIONS(2)
```

Python command syntax:

```
ierr = seqd(options)
```

Fortran command syntax:

```
CALL SEQDAPI(OPTIONS, IERR)
```

Where:

Integer OPTIONS (2)	array of 2 elements specifying options to be used in constructing the sequence matrices(input).	
	OPTIONS(1)	dc line and FACTS device option (0 by default)
	OPTIONS(1) = 0	block and ignore
	OPTIONS(1) = 1	convert to constant admittance load
	OPTIONS(2)	transformer impedance correction option (1 by default)
	OPTIONS(2) = 0	do not apply to zero sequence
	OPTIONS(2) = 1	apply to zero sequence
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.26. SEQD_2

Use this API routine to factorize the sequence matrices in preparation for unbalanced fault calculations SCMU and SPCB.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SEQD_2  OPTIONS(1)  OPTIONS(2)  OPTIONS(3)
```

Python command syntax:

```
ierr = seqd_2(options)
```

Fortran command syntax:

```
CALL SEQD_2(OPTIONS, IERR)
```

Where:

Integer OPTIONS (3)	array of 3 elements specifying options to be used in constructing the sequence matrices(input).	
	OPTIONS(1)	dc line and FACTS device option (0 by default)
	OPTIONS(1) = 0	block and ignore
	OPTIONS(1) = 1	convert to constant admittance load
	OPTIONS(2)	transformer impedance correction option (1 by default)
	OPTIONS(2) = 0	do not apply to zero sequence
	OPTIONS(2) = 1	apply to zero sequence
	OPTIONS(3)	fault analysis generator reactance option; ignored if generators are already "converted" (0 by default)
	OPTIONS(3) = 0	use subtransient reactance
	OPTIONS(3) = 1	use transient reactance
	OPTIONS(3) = 2	use synchronous reactance
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	fault analysis is invalid due to the presence of in-service induction machines that are in the "stalled" or "tripped" state
	IERR = 3	bus type code and series element status inconsistencies
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.27. SEQUENCE_NETWORK_SETUP

Use this API to specify or return the option to enable or disable automatic fault analysis sequence network setup.

This API routine was first introduced in release 29.0.0.

Batch command syntax:

```
BAT_SEQUENCE_NETWORK_SETUP IVAL
```

Python command syntax:

```
ierr = sequence_network_setup(ival)

ierr, ival = sequence_network_setup()
```

Fortran command syntax:

```
CALL SEQUENCE_NETWORK_SETUP_API(IVAL, IOCODE, IERR)
```

Where:

Integer IVAL	value of the option setting (input; if Batch command, or for Python if specified as an argument, or for Fortran if IOCODE is 0, 0 by default; else output).	
	IVAL = 0	SCMU assumes SEQD has already been executed.
	IVAL = 1	SCMU always executes activity SEQD.
Integer IOCODE	Indicates the operation mode of the API (no default allowed).	
	IOCODE = 0	set the option setting to the value specified in IVAL.
	IOCODE = 1	return the current value of the option setting in IVAL.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid IVAL value
	IERR = 2	invalid IOCODE value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.1.28. SQEX

Use this API to tabulate all fault analysis data pertaining to a specified bus.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SQEX SID ALL
```

Python command syntax:

```
ierr = sqex(sid, all)
```

Fortran command syntax:

```
CALL SQEXNW(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Fault Analysis Operation APIs contains API routines corresponding to the operational functions of PSS®E Fault Analysis.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2. Fault Analysis Data

Fault Analysis, or Short Circuit, Data Changing APIs contains API routines specific to adding, modifying and deleting short circuit data.

Note that, except for the SEQ_MUTUAL_DATA API, the SEQ_ APIs always modify data of an existing element.

4.2.1. MBIDNCS

Use this API to change the table name of the specified non-conventional source fault contribution table in the working case.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_MBIDNCS  'NAME'  'NAMNEW'
```

Python command syntax:

```
ierr = mbidncs(name, namnew)
```

Fortran command syntax:

```
CALL MBIDNCS(NAME, NAMNEW, IERR)
```

Where:

Character*12 NAME	name of the NCSFCC table for which the NCSFCC table name is to be changed (input; no default allowed).	
Character*12 NAMNEW	new name to be assigned to the NCSFCC table (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	NCSFCC table NAME not found
	IERR = 2	NCSFCC table NEWNAM already exists
	IERR = 3	a non-blank new identifier must be specified
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.2. SEQ_3_WIND_GROUNDING_DATA

Use this API to modify the zero sequence grounding data of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Zero Sequence Transformer Data).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SEQ_3_WIND_GROUNDING_DATA  IBUS  JBUS  KBUS  'CKT'  INTGAR(1)
 REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = seq_3_wind_grounding_data(ibus, jbus, kbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_3_WINDING_GROUNDING_API_2(IBUS,JBUS,KBUS,CKT,INTGAR,REALAR,IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying one element (defaulted data items are unchanged)(input).	
	INTGAR(1)	CC, connection code
Real REALAR (2)	array of 2 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	RG, zero sequence grounding resistance
	REALAR(2)	XG, zero sequence grounding reactance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	three-winding transformer not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid winding connection code. - Connection code is such that more than one winding requires a grounding impedance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.3. SEQ_3_WIND_WINDING_DATA

Use this API to modify the zero sequence impedance data of one winding of an existing three-winding transformer in the working case (see PSSE Program Operation Manual, Zero Sequence Transformer Data).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SEQ_3_WIND_WINDING_DATA  IBUS  JBUS  KBUS  'CKT'  WARG  REALAR(1)
 REALAR(2)
```

Python command syntax:

```
ierr = seq_3_wind_winding_data(ibus, jbus, kbus, ckt, warg, realar)
```

Fortran command syntax:

```
CALL SEQ_THREE_WINDING_WINDING_API_2(IBUS,JBUS,KBUS,CKT,WARG,REALAR,IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer WARG	winding number (1, 2 or 3) (input; no default allowed).	
Real REALAR (2)	array of 2 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	Ri, zero sequence winding resistance
	REALAR(2)	Xi, zero sequence winding reactance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	three-winding transformer not found
	IERR = 4	invalid winding number
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - None

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.4. SEQ_BRANCH_DATA

This API routine is obsolete. It has been replaced by the API routine seq_branch_data_3 .

Use this API to modify the zero sequence data of an existing non-transformer branch in the working case (see PSSE Program Operation Manual, Zero Sequence Non-Transformer Branch Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SEQ_BRANCH_DATA  IBUS  JBUS  'CKT'  REALAR(1)..REALAR(7)
```

Python command syntax:

```
ierr = seq_branch_data(ibus, jbus, ckt, realar)
```

Fortran command syntax:

```
CALL SEQ_BRANCH_API_2(IBUS, JBUS, CKT, REALAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Real REALAR (7)	array of 7 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	RLINZ, zero sequence branch resistance
	REALAR(2)	XLINZ, zero sequence branch reactance
	REALAR(3)	BCHZ, zero sequence total line charging
	REALAR(4)	GI, zero sequence active line shunt at bus IBUS end
	REALAR(5)	BI, zero sequence reactive line shunt at bus IBUS end
	REALAR(6)	GJ, zero sequence active line shunt at bus JBUS end
	REALAR(7)	BJ, zero sequence reactive line shunt at bus JBUS end
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	circuit identifier may not start with &
	IERR = 4	branch not found
	IERR = 5	branch exists but it is a two-winding transformer

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - branch is treated as a zero impedance line but its zero sequence impedance is non-zero

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.5. SEQ_BRANCH_DATA_3

Use this API to modify the zero sequence data of an existing non-transformer branch in the working case (see PSSE Program Operation Manual, Zero Sequence Non-Transformer Branch Data).

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SEQ_BRANCH_DATA_3  IBUS JBUS CKT INTGAR REALAR(1)..REALAR(8)
```

Python command syntax:

```
ierr = seq_branch_data_3(ibus, jbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_BRANCH_API_3(IBUS, JBUS, CKT, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying MOV-protected series capacitor flag(input).	
	INTGAR(1)	MOV-protected series capacitor flag
	INTGAR(1) = 0	not a MOV-protected branch
	INTGAR(1) = 1	MOV-protected series capacitor with MOV protection enabled
	INTGAR(1) = 2	MOV-protected series capacitor with MOV protection disabled
	INTGAR(1) = 3	spark gap protection enabled
Real REALAR (8)	array of 8 elements specifying (defaulted data items are un-changed)(input).	
	REALAR(1)	RLINZ, zero sequence branch resistance
	REALAR(2)	XLINZ, zero sequence branch reactance
	REALAR(3)	BCHZ, zero sequence total line charging
	REALAR(4)	GI, zero sequence active line shunt at bus IBUS end
	REALAR(5)	BI, zero sequence reactive line shunt at bus IBUS end
	REALAR(6)	GJ, zero sequence active line shunt at bus JBUS end
	REALAR(7)	BJ, zero sequence reactive line shunt at bus JBUS end
	REALAR(8)	MOVI, mov protected current level in kA
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no errors occurred
IERR > 0	fatal error occurred
IERR = 1	bus not found
IERR = 2	circuit identifier is more than two characters
IERR = 3	circuit identifier may not start with &
IERR = 4	branch not found
IERR = 5	branch exists but it is a two-winding transformer
IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - branch is treated as a zero impedance line but its zero sequence impedance is non-zero

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.6. SEQ_BUS_DATA

Bus load sequence data is now accessible as load sequence data.
Use this API routine to modify the sequence data of an existing bus in the working case (see PSSE Program Operation Manual, Negative Sequence Shunt Load Data and Zero Sequence Shunt Load Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SEQ_BUS_DATA  IBUS  REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = seq_bus_data(ibus, realar)
```

Fortran command syntax:

```
CALL SEQ_BUS_API_2(IBUS, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Real REALAR (4)	array of 4 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	GNEG, negative sequence shunt admittance active component
	REALAR(2)	BNEG, negative sequence shunt admittance reactive component
	REALAR(3)	GZERO, zero sequence shunt admittance active component
	REALAR(4)	BZERO, zero sequence shunt admittance reactive component
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	the SEQ_BUS_DATA API routine is obsolete
	IERR = 2	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - None

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.7. SEQ_FIXED_SHUNT_DATA

Use this API to modify the zero sequence data of an existing fixed bus shunt in the working case (see PSSE Program Operation Manual, Zero Sequence Fixed Shunt Data).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SEQ_FIXED_SHUNT_DATA  IBUS  'ID'  REALAR(1)  REALAR(2)
```

Python command syntax:

```
ierr = seq_fixed_shunt_data(ibus, id, realar)
```

Fortran command syntax:

```
CALL SEQ_FIXED_SHUNT_API(IBUS, ID, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	shunt identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	GSZERO, active power component of fixed shunt zero sequence admittance (conductance)
	REALAR(2)	BSZERO, reactive power component of fixed shunt zero sequence admittance (susceptance)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	shunt identifier is more than two characters
	IERR = 3	fixed shunt not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Admittance components with different signs in the positive and zero sequences

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.8. SEQ_INDUCTION_MACH_DATA

This API routine is obsolete. It has been replaced by the API routine seq_induction_mach_data_2 .

Use this API routine to modify the sequence data of an existing induction machine in the working case (see PSSE Program Operation Manual, Induction Machine Impedance Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SEQ_INDUCTION_MACH_DATA  IBUS  'ID'  INTGAR(1)  INTGAR(2)
 REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = seq_induction_mach_data(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_INDUCTION_MACH_API(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (defaulted data items are unchanged)(input).	
	INTGAR(1)	CZG, code indicating the units in which the zero sequence grounding impedance is specified
	INTGAR(1) = 1	for per unit
	INTGAR(1) = 2	for ohms
	INTGAR(2)	CGR, grounding code (0 = ungrounded; 1 = grounded) (0 by default)
Real REALAR (6)	array of 6 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	ILK2IR, ratio of positive sequence locked rotor current to rated current
	REALAR(2)	ROVERX, ratio of positive sequence resistance to reactance
	REALAR(3)	R0, machine zero sequence fault analysis resistance
	REALAR(4)	X0, machine zero sequence fault analysis reactance
	REALAR(5)	RG, machine zero sequence grounding resistance
	REALAR(6)	XG, machine zero sequence grounding reactance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	induction machine not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid CZG data value. - invalid grounding code. - CZG is 2 but the bus has no base voltage specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.9. SEQ_INDUCTION_MACH_DATA_2

Use this API routine to modify the sequence data of an existing induction machine in the working case (see PSSE Program Operation Manual, Induction Machine Impedance Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SEQ_INDUCTION_MACH_DATA_2  IBUS ID INTGAR(1)..INTGAR(2)
 REALAR(1)..REALAR(10)
```

Python command syntax:

```
ierr = seq_induction_mach_data_2(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_INDUCTION_MACH_API_2(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (defaulted data items are unchanged)(input).	
	INTGAR(1)	CZG, code indicating the units in which the zero sequence grounding impedance is specified
	INTGAR(1) = 1	for per unit
	INTGAR(1) = 2	for ohms
	INTGAR(2)	CGR, grounding code (0 = ungrounded; 1 = grounded) (0 by default)
Real REALAR (10)	array of 10 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	ILK2IR, ratio of positive sequence locked rotor current to rated current
	REALAR(2)	ROVERX, ratio of positive sequence resistance to reactance
	REALAR(3)	R0, machine zero sequence fault analysis resistance
	REALAR(4)	X0, machine zero sequence fault analysis reactance
	REALAR(5)	RG, machine zero sequence grounding resistance
	REALAR(6)	XG, machine zero sequence grounding reactance
	REALAR(7)	RT, ratio of locked rotor current to rated current (transient)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(8)	XT, ratio of Rpos to Xpos (these two used to set IMZPSN)(transient)
	REALAR(9)	RN, ratio of locked rotor current to rated current (negative sequence)
	REALAR(10)	XN, ratio of Rpos to Xpos (these two used to set IMZPSN)(negative sequence)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	induction machine not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid CZG data value. - invalid grounding code. - CZG is 2 but the bus has no base voltage specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.10. SEQ_LOAD_DATA

Use this API routine to modify the sequence data of an existing load in the working case (see PSSE Program Operation Manual, Load Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SEQ_LOAD_DATA  IBUS  'ID'  INTGAR(1)  REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = seq_load_data(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_LOAD_API(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying one element(input).	
	INTGAR(1)	CGR, load grounding code (0 = ungrounded; 1 = grounded) (0 by default)
Real REALAR (4)	array of 4 elements specifying (input).	
	REALAR(1)	GNEG, negative sequence active power load component; entered as admittance in MW at 1.0 per unit voltage (0.0 by default)
	REALAR(2)	BNEG, negative sequence reactive power load component; BNEG is negative for an inductive load and positive for a capacitive load; entered as admittance in MVar at 1.0 per unit voltage (0.0 by default)
	REALAR(3)	GZERO, zero sequence active power load component; entered as admittance in MW at 1.0 per unit voltage (0.0 by default)
	REALAR(4)	BZERO, zero sequence reactive power load component; BZERO is negative for an inductive load and positive for a capacitive load; entered as admittance in MVar at 1.0 per unit voltage (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	load identifier is more than two characters
	IERR = 3	load not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - invalid grounding code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.11. SEQ_MACHINE_DATA

This API routine is obsolete. It has been replaced by the API routine seq_machine_data_3 .

Use this API routine to modify the sequence data of an existing machine in the working case (see PSSE Program Operation Manual, Positive Sequence Generator Impedance Data, Negative Sequence Generator Impedance Data and Zero Sequence Generator Impedance Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SEQ_MACHINE_DATA  IBUS  'ID'  REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = seq_machine_data(ibus, id, realar)
```

Fortran command syntax:

```
CALL SEQ_MACHINE_API_2(IBUS, ID, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real REALAR (6)	array of 6 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	ZRPOS, machine positive sequence fault analysis resistance
	REALAR(2)	ZXPOS, machine positive sequence fault analysis reactance
	REALAR(3)	ZRNEG, machine negative sequence fault analysis resistance
	REALAR(4)	ZXNEG, machine negative sequence fault analysis reactance
	REALAR(5)	RZERO, machine zero sequence fault analysis resistance
	REALAR(6)	XZERO, machine zero sequence fault analysis reactance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = -1	data error, warning and information messages; one or more of: - Machine impedance is (0.0,0.0)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.12. SEQ_MACHINE_DATA_3

This API routine is obsolete. It has been replaced by the API routine seq_machine_data_4 .

Use this API routine to modify the sequence data of an existing machine in the working case (see PSSE Program Operation Manual, Positive Sequence Generator Impedance Data, Negative Sequence Generator Impedance Data and Zero Sequence Generator Impedance Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SEQ_MACHINE_DATA_3  IBUS  'ID'  INTGAR(1)  REALAR(1)..REALAR(10)
```

Python command syntax:

```
ierr = seq_machine_data_3(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_MACHINE_API_3(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying one element (defaulted data items are unchanged)(input).	
	INTGAR(1)	CZG, code indicating the units in which the zero sequence grounding impedance is specified
	INTGAR(1) = 1	for per unit
	INTGAR(1) = 2	for ohms
Real REALAR (10)	array of 10 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	RPOS, machine positive sequence fault analysis resistance
	REALAR(2)	X", machine positive sequence fault analysis subtransient reactance
	REALAR(3)	RNEG, machine negative sequence fault analysis resistance
	REALAR(4)	XNEG, machine negative sequence fault analysis reactance
	REALAR(5)	RZERO, machine zero sequence fault analysis resistance
	REALAR(6)	XZERO, machine zero sequence fault analysis reactance
	REALAR(7)	X', machine positive sequence fault analysis transient reactance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(8)	Xs, machine positive sequence fault analysis synchronous reactance
	REALAR(9)	RG, machine zero sequence grounding resistance
	REALAR(10)	XG, machine zero sequence grounding reactance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Grounding impedance units code indicates input in ohms but no base voltage has been specified for the bus. - Invalid grounding impedance units code. - Invalid grounding impedance units code. - Invalid positive sequence reactance value. - Negative sequence impedance is (0.0,0.0). - Zero sequence impedance is (0.0,0.0)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.13. SEQ_MACHINE_DATA_4

Use this API routine to modify the sequence data of an existing machine in the working case (see PSSE Program Operation Manual, Positive Sequence Generator Impedance Data, Negative Sequence Generator Impedance Data and Zero Sequence Generator Impedance Data).

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SEQ_MACHINE_DATA_4  IBUS ID INTGAR REALAR(1)..REALAR(11)
```

Python command syntax:

```
ierr = seq_machine_data_4(ibus, id, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_MACHINE_API_4(IBUS, ID, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying one element (defaulted data items are unchanged)(input).	
	INTGAR(1)	CZG, code indicating the units in which the zero sequence grounding impedance is specified
	INTGAR(1) = 1	for per unit
	INTGAR(1) = 2	for ohms
Real REALAR (11)	array of 11 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	RPOS, machine positive sequence fault analysis resistance
	REALAR(2)	X", machine positive sequence fault analysis subtransient reactance
	REALAR(3)	RNEG, machine negative sequence fault analysis resistance
	REALAR(4)	XNEG, machine negative sequence fault analysis reactance
	REALAR(5)	RZERO, machine zero sequence fault analysis resistance
	REALAR(6)	XZERO, machine zero sequence fault analysis reactance
	REALAR(7)	X', machine positive sequence fault analysis transient reactance
	REALAR(8)	Xs, machine positive sequence fault analysis synchronous reactance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(9)	RG, machine zero sequence grounding resistance
	REALAR(10)	XG, machine zero sequence grounding reactance
	REALAR(11)	RFDG, machine reference angle in degrees
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Grounding impedance units code indicates input in ohms but no base voltage has been specified for the bus. - Invalid grounding impedance units code. - Invalid grounding impedance units code. - Invalid positive sequence reactance value. - Negative sequence impedance is (0.0,0.0). - Zero sequence impedance is (0.0,0.0)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.14. SEQ_MACHINE_NCS_DATA

Use this API routine to modify a machine's existing non-conventional source fault current contribution (NCSFCC) data or to add NCSFCC data to an existing machine in the working case.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SEQ_MACHINE_NCS_DATA  IBUS ID INTGAR REALAR(1) REALAR(2) REALAR(3)
 NAMEAR(1) NAMEAR(2)
```

Python command syntax:

```
ierr = seq_machine_ncs_data(ibus, id, intgar, realar, namear)
```

Fortran command syntax:

```
CALL SEQ_MACHINE_NCS_DATA(IBUS, ID, INTGAR, REALAR, NAMEAR, IERR)
```

Where:

Integer IBUS	bus number (input).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR	characteristic type; 0 for none, 1 for time dependent, 2 for voltage dependent (input; 0 by default).	
real REALAR (3)	array of 3 elements specifying Real Options(input).	
	REALAR(1)	IFMAX, per unit total current magnitude allowed during a fault (1.3 by default)
	REALAR(2)	DBVMIN, per unit dead-band minimum voltage at POI (0.9 by default)
	REALAR(3)	DBVMAX, per unit dead-band maximum voltage at POI (1.1 by default)
character*12 NAMEAR (2)	array of 2 elements specifying Specifies the names of the non-conventional source fault contribution (NCSFCC) tables(input).	
	NAMEAR(1)	positive sequence table name (by default)
	NAMEAR(2)	negative sequence table name (by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	no table specified for the positive sequence
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid characteristic type flag. - Table not found. - Machine and ta-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ble characteristic type mismatch. - IFMAX is less than 1.0. - DBVMAX is less than DBVMIN.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.15. SEQ_MACHINE_NCS_DATA_PURG

Use this API routine to delete a machine's existing non-conventional source fault current contribution (NCSFCC) data.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SEQ_MACHINE_NCS_DATA_PURG  IBUS ID
```

Python command syntax:

```
ierr = seq_machine_ncs_data_purg(ibus, id)
```

Fortran command syntax:

```
CALL SEQ_MACHINE_NCS_DATA_PURG(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus number (input).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	no non-conventional source data specified for this machine

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.16. SEQ_MUTUAL_DATA

Use this API to modify existing zero sequence mutual impedance data in the working case or to add a new zero sequence mutual coupling to the working case (see PSSE Program Operation Manual, Zero Sequence Mutual Impedance Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SEQ_MUTUAL_DATA  IBUS  JBUS  'CKT1'  KBUS  LBUS  'CKT2'
 REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = seq_mutual_data(ibus, jbus, ckt1, kbus, lbus, ckt2, realar)
```

Fortran command syntax:

```
CALL SEQ_MUTUAL_API_2(IBUS, JBUS, CKT1, KBUS, LBUS, CKT2, REALAR, IERR)
```

Where:

Integer IBUS	bus number of the from bus of the first branch (input; no default allowed).	
Integer JBUS	bus number of the to bus of the first branch (input; no default allowed).	
Character*2 CKT1	first branch circuit identifier (input; '1' by default).	
Integer KBUS	bus number of the from bus of the second branch (input; no default allowed).	
Integer LBUS	bus number of the to bus of the second branch to bus (input; no default allowed).	
Character*2 CKT2	second branch circuit identifier (input; '1' by default).	
Real REALAR (6)	array of 6 elements specifying (input).	
	REALAR(1)	RM, branch-to-branch mutual resistance (0.0 by default)
	REALAR(2)	XM, branch-to-branch mutual reactance (0.0 by default)
	REALAR(3)	BIJ1, B factor at IBUS end of the first branch (0.0 by default)
	REALAR(4)	BIJ2, B factor at JBUS end of the first branch (1.0 by default)
	REALAR(5)	BKL1, B factor at KBUS end of the second branch (0.0 by default)
	REALAR(6)	BKL2, B factor at LBUS end of the second branch (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	bus not found
IERR = 2	circuit identifier is more than two characters
IERR = 3	circuit identifier may not start with &
IERR = 4	branch not found
IERR = 5	mutual couples a branch to itself
IERR = 6	branch is a transformer
IERR = 7	branch is a zero impedance line
IERR = 8	mutual coupling table is full
IERR = 9	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid B factor (<0.0 or >1.0). - B1 is not less than B2

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.17. SEQ_NCS_FLT_CNTRB_CHNG

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SEQ_NCS_FLT_CNTRB_CHNG  IBUS ID POINTS REALAR(1)..REALAR(POINTS)
 CPLXAR(1)..CPLXAR(POINTS)
```

Python command syntax:

```
ierr = seq_ncs_flt_cntrb_chng(ibus, id, points, realar, cplxar)
```

Fortran command syntax:

```
CALL SEQ_NCS_FLT_CNTRB_CHNG(IBUS, ID, POINTS, REALAR, CPLXAR, IERR)
```

Where:

Integer IBUS	bus number (input).
Character*2 ID	machine identifier (input; '1' by default).
Integer POINTS	number of data points (input; no default allowed).
Real REALAR	Array of 'POINTS' values specifying Ti in seconds, in strictly ascending order of Ti (input; no default allowed).
Complex CPLXAR	Array of 'POINTS' values specifying CiP and CiQ (input; no default allowed).
Integer IERR	error code (output). IERR = 0 no errors occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.18. SEQ_NCS_FLT_CNTRB_DATA

This API routine is obsolete. It has been replaced by the API routines SEQ_NCS_TABLE_DATA and SEQ_MACHINE_NCS_DATA .

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SEQ_NCS_FLT_CNTRB_DATA  IBUS ID POINTS REALAR(1)..REALAR(POINTS)
 CPLXAR(1)..CPLXAR(POINTS)
```

Python command syntax:

```
ierr = seq_ncs_flt_cntrb_data(ibus, id, points, realar, cplxar)
```

Fortran command syntax:

```
CALL SEQ_NCS_FLT_CNTRB_DATA(IBUS, ID, POINTS, REALAR, CPLXAR, IERR)
```

Where:

Integer IBUS	bus number (input).
Character*2 ID	machine identifier (input; '1' by default).
Integer POINTS	number of data points (input; no default allowed).
Real REALAR	Array of 'POINTS' values specifying Ti in seconds, in strictly ascending order of Ti (input; no default allowed).
Complex CPLXAR	Array of 'POINTS' values specifying CiP and CiQ (input; no default allowed).
Integer IERR	error code (output). IERR = 0 no errors occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.19. SEQ_NCS_FLT_CNTRB_PURG

This API routine is obsolete. It has been replaced by the API routine SEQ_MACHINE_NCS_DATA_PURG .

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SEQ_NCS_FLT_CNTRB_PURG  IBUS ID
```

Python command syntax:

```
ierr = seq_ncs_flt_cntrb_purg(ibus, id)
```

Fortran command syntax:

```
CALL SEQ_NCS_FLT_CNTRB_PURG(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus number (input).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	machine identifier is more than two characters
	IERR = 3	machine not found
	IERR = 4	Non-Conventional Source data not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.20. SEQ_NCS_TABLE_DATA

Use this API to modify the data of an existing non-conventional source fault contribution table in the working case or to add a new non-conventional source fault contribution table to the working case.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SEQ_NCS_TABLE_DATA  'NAME'  POINTS  TYPARG  REALAR(1)..REALAR(POINTS)
 CPLXAR(1)..CPLXAR(POINTS)
```

Python command syntax:

```
ierr = seq_ncs_table_data(name, points, typarg, realar, cplxar)
```

Fortran command syntax:

```
CALL SEQ_NCS_TABLE_DATA(NAME, POINTS, TYPARG, REALAR, CPLXAR, IERR)
```

Where:

character*12 NAME	table name (input; no default allowed).	
Integer POINTS	number of points specified in table 'NAME' (1 through 20) (input; no default allowed).	
Integer TYPARG	characteristic type; 1 for time dependent, 2 for voltage dependent (input; 1 by default).	
Real REALAR (POINTS)	array of 'POINTS' values specifying Ti in seconds or Vi in per unit, in strictly ascending order of Ti or Vi (input).	
Complex CPLXAR (POINTS)	array of 'POINTS' values specifying CiP and CiQ (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	blank table name is invalid
	IERR = 2	invalid POINTS value
	IERR = 3	table does not have at least 1 point
	IERR = 4	maximum number of tables are already in the case
	IERR = 5	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - invalid characteristic type flag - point is beyond the previous end of table - Ti or Vi is negative - Ci is (0.0,0.0) - Ti or Vi values are not in increasing order

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.21. SEQ_NCS_TABLE_ONEPT

Use this API to modify the one data point of an existing non-conventional source fault contribution table in the working case or to add a new one data point to non-conventional source fault contribution table to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_SEQ_NCS_TABLE_ONEPT 'NAME' TPTOLD TPT CIPQ
```

Python command syntax:

```
ierr = seq_ncs_table_onept(name, tptold, tpt, cipq)
```

Fortran command syntax:

```
CALL SEQ_NCS_TABLE_ONEPT(NAME, TPTOLD, TPT, CIPQ, IERR)
```

Where:

character*12 NAME	table name (input; no default allowed).	
Real TPTOLD	Existing Table point specifying Ti in seconds or Vi in per unit. Table data at this point is added or changed. When TPTOLD is specified and found, TPT and CIPQ values specified will be used to change existing data. When TPTOLD is zero and TPT greater than 0, TPT and CIPQ values specified will be used to add new data point to existing data. Specify TPTOLD=TPT to change just CIPQ data at that point. (input; no default allowed).	
Real TPT	New Table point specifying Ti in seconds or Vi in per unit. (input; no default allowed).	
Complex CIPQ	Value specifying CiP and CiQ (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	blank table name is invalid
	IERR = 2	maximum number of points in table are already in the case
	IERR = 3	Ti or Vi is invalid
	IERR = 4	Ci is (0.0,0.0), but NCS fault current contribution cannot be zero
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.22. SEQ_NCS_TABLE_PURG

Use this API to delete an existing non-conventional source fault contribution table from the working case.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SEQ_NCS_TABLE_PURG  'NAME'
```

Python command syntax:

```
ierr = seq_ncs_table_purg(name)
```

Fortran command syntax:

```
CALL SEQ_NCS_TABLE_PURG(NAME, IERR)
```

Where:

character*12 NAME	table name (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	table not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.23. SEQ_NCS_TABLE_PURG_ONEPT

Use this API to delete specified data point in an existing non-conventional source fault contribution table.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SEQ_NCS_TABLE_PURG_ONEPT 'NAME' TPT
```

Python command syntax:

```
ierr = seq_ncs_table_purg_onept(name, tpt)
```

Fortran command syntax:

```
CALL SEQ_NCS_TABLE_PURG_ONEPT(NAME, TPT, IERR)
```

Where:

character*12 NAME	table name (input).	
Real TPT	Table Time or Voltage at a point in data table where Ti, Ci or Vi, Ci data is purged (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	table not found
	IERR = 2	prerequisite requirements for API are not met
	IERR = 3	table point is not valid
	IERR = 4	table has no points
	IERR = 3	table point does not exist

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.24. SEQ_NCS_TABLE_TYPE

Use this API to get NCS table type.

This API routine was first introduced in release 35.2.0.

Python command syntax:

```
ierr, ttyp = seq_ncs_table_type(name)
```

Fortran command syntax:

```
CALL SEQ_NCS_TABLE_TYPE(NAME, TTYP, IERR)
```

Where:

character*12 NAME	table name (input).	
Integer TTYP	Table Type (1=Time-Current, 2=Voltage-Current (output).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	table not found
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.25. SEQ_SWITCHED_SHUNT_DATA

This API routine is obsolete. It has been replaced by the API routine SEQ_SWITCHED_SHUNT_DATA_3 .

Use this API to modify the zero sequence data of an existing switched shunt in the working case (see PSSE Program Operation Manual, Zero Sequence Switched Shunt Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SEQ_SWITCHED_SHUNT_DATA  IBUS  REALAR(1)..REALAR(8)
```

Python command syntax:

```
ierr, realaro = seq_switched_shunt_data(ibus, realari)
```

Fortran command syntax:

```
CALL SEQ_SWITCHED_SHUNT_API_2(IBUS, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Real REALAR (9)	(first eight are input values; last value is output) (defaulted data items are unchanged) (input (output).).	
Real REALARI (8)	array of 8 elements specifying corresponding to values REALAR(1) through REALAR(8) as described above(input).	
	REALARI(1)	BZ1, zero sequence increment in MVAR per step for block 1
	REALARI(2)	BZ2, zero sequence increment in MVAR per step for block 2
	REALARI(3)	BZ3, zero sequence increment in MVAR per step for block 3
	REALARI(4)	BZ4, zero sequence increment in MVAR per step for block 4
	REALARI(5)	BZ5, zero sequence increment in MVAR per step for block 5
	REALARI(6)	BZ6, zero sequence increment in MVAR per step for block 6
	REALARI(7)	BZ7, zero sequence increment in MVAR per step for block 7
	REALARI(8)	BZ8, zero sequence increment in MVAR per step for block 8
Real REALARO (1)	array of 1 elements specifying one element corresponding to value REALAR(9) as described above(output).	
	REALARO(1)	present zero sequence switched shunt admittance in MVAR (returned)
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no errors occurred
IERR > 0	fatal error occurred
IERR = 1	bus not found
IERR = 2	no switched shunt at bus
IERR = 3	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Block has N zero and B non-zero. - Block has admittances with different signs in positive and zero sequences

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.26. SEQ_SWITCHED_SHUNT_DATA_3

Use this API to modify the zero sequence data of an existing switched shunt in the working case (see PSSE Program Operation Manual, Zero Sequence Switched Shunt Data).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SEQ_SWITCHED_SHUNT_DATA_3  IBUS  'ID', REALAR(1)..REALAR(8)
```

Python command syntax:

```
ierr, realaro = seq_switched_shunt_data_3(ibus, id, realari)
```

Fortran command syntax:

```
CALL SEQ_SWITCHED_SHUNT_API_3(IBUS, ID, REALAR, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
character*2 ID	Switched shunt identifier (input; 1 by default).	
Real REALAR (9)	(first eight are input values; last value is output) (defaulted data items are unchanged) (input (output).).	
Real REALARI (8)	array of 8 elements specifying corresponding to values REALAR(1) through REALAR(8) as described above(input).	
	REALARI(1)	BZ1, zero sequence increment in MVAR per step for block 1
	REALARI(2)	BZ2, zero sequence increment in MVAR per step for block 2
	REALARI(3)	BZ3, zero sequence increment in MVAR per step for block 3
	REALARI(4)	BZ4, zero sequence increment in MVAR per step for block 4
	REALARI(5)	BZ5, zero sequence increment in MVAR per step for block 5
	REALARI(6)	BZ6, zero sequence increment in MVAR per step for block 6
	REALARI(7)	BZ7, zero sequence increment in MVAR per step for block 7
	REALARI(8)	BZ8, zero sequence increment in MVAR per step for block 8
Real REALARO (1)	array of 1 elements specifying one element corresponding to value REALAR(9) as described above(output).	
	REALARO(1)	present zero sequence switched shunt admittance in MVAR (returned)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR > 0	fatal error occurred
IERR = 1	bus not found
IERR = 2	shunt identifier is more than two characters
IERR = 2	switched shunt not found
IERR = 3	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Block has N zero and B non-zero. - Block has admittances with different signs in positive and zero sequences

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.27. SEQ_THREE_WINDING_DATA

This API routine is obsolete. It has been replaced by the API routine seq_three_winding_data_3 .

Use this API to modify the zero sequence data of an existing three-winding transformer in the working case.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SEQ_THREE_WINDING_DATA  IBUS  JBUS  KBUS  'CKT'  INTGAR(1)
 REALAR(1)..REALAR(8)
```

Python command syntax:

```
ierr = seq_three_winding_data(ibus, jbus, kbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_THREE_WINDING_API(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR, IERR,
 NERR)
```

Where:

Integer IBUS	bus number of one of the buses (input).	
Integer JBUS	bus number one of the other buses (input).	
Integer KBUS	bus number of the third bus (input).	
Character*2 CKT	transformer circuit identifier (input).	
Integer INTGAR (1)	array of 1 elements specifying one element(input).	
	INTGAR(1)	CC, connection code
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	RG, zero sequence grounding resistance
	REALAR(2)	XG, zero sequence grounding reactance
	REALAR(3)	R1, zero sequence winding one resistance
	REALAR(4)	X1, zero sequence winding one reactance
	REALAR(5)	R2, zero sequence winding two resistance
	REALAR(6)	X2, zero sequence winding two reactance
	REALAR(7)	R3, zero sequence winding three resistance
	REALAR(8)	X3, zero sequence winding three reactance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	three-winding transformer not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	IERR = -1	warnings - one or more of: - Invalid winding connection code. - Connection code is such that more than one winding requires a grounding impedance
Integer NERR	number of entries in PBUF02 containing error messages (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.28. SEQ_THREE_WINDING_DATA_3

Use this API to modify the zero sequence data of an existing three-winding transformer in the working case.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SEQ_THREE_WINDING_DATA_3  IBUS  JBUS  KBUS  'CKT'
 INTGAR(1)..INTGAR(3)  REALAR(1)..REALAR(14)
```

Python command syntax:

```
ierr = seq_three_winding_data_3(ibus, jbus, kbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_THREE_WINDING_API_3(IBUS, JBUS, KBUS, CKT, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input).	
Integer JBUS	bus number one of the other buses (input).	
Integer KBUS	bus number of the third bus (input).	
Character*2 CKT	transformer circuit identifier (input).	
Integer INTGAR (3)	array of 3 elements specifying (defaulted data items are unchanged)(input).	
	INTGAR(1)	CZ0, non-grounding impedance data I/O code
	INTGAR(2)	CZG, grounding impedance data I/O code
	INTGAR(3)	CC, connection code
Real REALAR (14)	array of 14 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	RG1, grounding resistance on winding 1 for an impedance grounded transformer
	REALAR(2)	XG1, grounding reactance on winding 1 for an impedance grounded transformer
	REALAR(3)	R01, winding 1, or winding 1 bus to winding 2 bus, leakage resistance
	REALAR(4)	X01, winding 1, or winding 1 bus to winding 2 bus, leakage reactance
	REALAR(5)	RG2, grounding resistance on winding 2 for an impedance grounded transformer
	REALAR(6)	XG2, grounding reactance on winding 2 for an impedance grounded transformer
	REALAR(7)	R02, winding 2, or winding 2 bus to winding 3 bus, leakage resistance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(8)	X02, winding 2, or winding 2 bus to winding 3 bus, leakage reactance
	REALAR(9)	RG3, grounding resistance on winding 3 for an impedance grounded transformer
	REALAR(10)	XG3, grounding reactance on winding 3 for an impedance grounded transformer
	REALAR(11)	R03, winding 3, or winding 3 bus to winding 1 bus, leakage resistance
	REALAR(12)	X03, winding 3, or winding 3 bus to winding 1 bus, leakage reactance
	REALAR(13)	RN, common neutral grounding resistance
	REALAR(14)	XN, common neutral grounding reactance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	three-winding transformer not found
	IERR = 4	CZG is 3 but at least one endpoint bus has no base voltage specified
	IERR = 5	invalid CZ0 or CZG value
	IERR = 6	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - Invalid winding connection code. - Invalid CZ0 or CZG data value. - CZ0 or CZG is not 1 with connection code 1 through 9 or a three-digit connection code. - CZG data value is 3 but at least one endpoint bus has no base voltage specified. - Non-zero neutral impedance specified but at least one endpoint bus has no base or nominal voltage specified. - A non-zero value is specified for an impedance that is not used with the specified connection code. - A value of zero is specified for an impedance that is used with the specified connection code. - Both ZGn and Z0n are used with the specified connection code and both are (0.0,0.0). - Connection code is 17 or 18 but at least one endpoint bus has no base or nominal voltage specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.29. SEQ_TWO_WINDING_DATA

This API routine is obsolete. It has been replaced by the API routine seq_two_winding_data_3 .

Use this API to modify the zero sequence data of an existing two-winding transformer in the working case (see PSSE Program Operation Manual, Zero Sequence Transformer Data).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SEQ_TWO_WINDING_DATA  IBUS  JBUS  'CKT'  INTGAR(1)
 REALAR(1)..REALAR(6)
```

Python command syntax:

```
ierr = seq_two_winding_data(ibus, jbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_TWO_WINDING_API_2(IBUS, JBUS, CKT, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (1)	array of 1 elements specifying one element (defaulted data items are unchanged)(input).	
	INTGAR(1)	CC, connection code (1 through 8)
Real REALAR (6)	array of 6 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	RG, grounding resistance if connection code < 8; winding 1 side resistance to ground if CC = 8
	REALAR(2)	XG, grounding reactance if connection code < 8; winding 1 side reactance to ground if CC = 8
	REALAR(3)	R1, zero sequence transformer resistance
	REALAR(4)	X1, zero sequence transformer reactance
	REALAR(5)	RG2, winding 2 side resistance to ground if CC = 8; 0.0 otherwise
	REALAR(6)	XG2, winding 2 side reactance to ground if CC = 8; 0.0 otherwise
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	circuit identifier is more than two characters
IERR = 3	circuit identifier may not start with &, * or @
IERR = 4	branch not found
IERR = 5	branch exists but it is a non-transformer branch
IERR = 6	prerequisite requirements for API are not met
IERR = -1	data error, warning and information messages; one or more of: - Invalid winding connection code. - Zero sequence impedance is (0.0,0.0). - Non-zero winding 2 impedance to ground not used with the specified connection code. - Winding n impedance to ground is (0.0,0.0) with connection code 8. - Grounding impedance is (0.0,0.0) with connection code 9. - A specified series impedance is (0.0,0.0) with connection code 9

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

4.2.30. SEQ_TWO_WINDING_DATA_3

Use this API to modify the zero sequence data of an existing two-winding transformer in the working case (see PSSE Program Operation Manual, Zero Sequence Transformer Data).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SEQ_TWO_WINDING_DATA_3  IBUS  JBUS  'CKT'  INTGAR(1)..INTGAR(3)
 REALAR(1)..REALAR(10)
```

Python command syntax:

```
ierr = seq_two_winding_data_3(ibus, jbus, ckt, intgar, realar)
```

Fortran command syntax:

```
CALL SEQ_TWO_WINDING_API_3(IBUS, JBUS, CKT, INTGAR, REALAR, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer INTGAR (3)	array of 3 elements specifying (defaulted data items are unchanged)(input).	
	INTGAR(1)	CC, connection code
	INTGAR(2)	CZ0, leakage impedance data I/O code
	INTGAR(3)	CZG, grounding impedance data I/O code
Real REALAR (10)	array of 10 elements specifying (defaulted data items are unchanged)(input).	
	REALAR(1)	RG1, grounding resistance at the winding 1 bus for an impedance grounded transformer
	REALAR(2)	XG1, grounding reactance at the winding 1 bus for an impedance grounded transformer
	REALAR(3)	R01, zero sequence leakage resistance connected to the winding 1 bus
	REALAR(4)	X01, zero sequence leakage reactance connected to the winding 1 bus
	REALAR(5)	RG2, grounding resistance at the winding 2 bus for an impedance grounded transformer
	REALAR(6)	XG2, grounding reactance at the winding 2 bus for an impedance grounded transformer
	REALAR(7)	R02, zero sequence leakage resistance connected to the winding 2 bus
	REALAR(8)	X02, zero sequence leakage reactance connected to the winding 2 bus
	REALAR(9)	RN, common neutral grounding resistance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(10)	XN, common neutral grounding reactance
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	bus not found
	IERR = 2	circuit identifier is more than two characters
	IERR = 3	circuit identifier may not start with &, * or @
	IERR = 4	branch not found
	IERR = 5	branch exists but it is a non-transformer branch
	IERR = 6	CZG is 3 but at least one endpoint bus has no base voltage specified
	IERR = 7	invalid CZ0 or CZG value
	IERR = 8	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - Invalid winding connection code. - Z01 is set to its positive sequence value. - A non-zero value is specified for an impedance that is not used with the specified connection code.. - A value of zero is specified for an impedance that is used with the specified connection code. - Invalid CZ0 or CZG data value. - CZ0 or CZG is not 1 with connection code 1 through 9. - CZG data value is 3 but at least one endpoint bus has no base voltage specified. - Non-zero neutral impedance specified but at least one endpoint bus has no base or nominal voltage specified. - Connection code is 18 through 22 but at least one endpoint bus has no base or nominal voltage specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 5
Harmonics

This chapter describes the API routines used in PSS®E Harmonics.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1. Harmonics Operation

5.1.1. HAR_ANALYSIS

This API routine is obsolete. It has been replaced by the API routine har_analysis_2 .

Use this API to run harmonics analysis.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_ANALYSIS SID ALL INTGAR REALAR FILEAR
```

Python command syntax:

```
ierr = har_analysis(sid, all, intgar, realar, filear)
```

Fortran command syntax:

```
CALL HAR_ANALYSIS(SID, ALL, INTGAR, REALAR, FILEAR, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGAR (8)	array of 8 elements specifying Integer Argument Inputs(input).	
	INTGAR(1)	Analysis Option (alias is anaoptn) (0 by default)
	INTGAR(1) = 0	Frequency scan, voltage and current distortion calculations
	INTGAR(1) = 1	Only frequency scan calculations
	INTGAR(1) = 2	Only voltage and vurrent distortion calculations
	INTGAR(2)	Transmission Line Model (alias is linmdl) (0 by default)
	INTGAR(2) = 0	Nominal PI Line Model, no long line effects, no skin effects
	INTGAR(2) = 1	Distributed Parameter Line Model, long line effects and skin effects modeled
	INTGAR(2) = 2	Distributed Parameter Line Model, long line effects modeled, but skin effects not modeled
	INTGAR(3)	Bus report option in distortion analysis. Note- Frequency scan is always done for study subsystem buses only. The distortion cal-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		culations are done for these buses. (alias is dstnbuop) (0 by default)
	INTGAR(3) = 0	All buses in the case
	INTGAR(3) = 1	Only study subsystem buses
	INTGAR(4)	Report format option in distortion analysis (alias is dstnrptop) (0 by default)
	INTGAR(4) = 0	Report voltage and current Total Harmonic Distortion (THD) Factors
	INTGAR(4) = 1	Report THDs and current contributions
	INTGAR(5)	Induction Machine Impedance Option. Note - The zero sequence impedance data always derived from sequence data. When sequence data is not specified, they are treated as open circuit. (alias is imachimpop) (0 by default)
	INTGAR(5) = 0	Use positive sequence impedance derived from power flow data
	INTGAR(5) = 1	Use positive sequence impedance derived from sequence data
	INTGAR(6)	Two Terminal, VSC and Multi Terminal DC equivalent Impedance Option in frequency scan. Note - In distortion calculations, these are represented by either harmonic current sources when specified or equivalent impedances. (alias is dcimpop) (0 by default)
	INTGAR(6) = 0	Do not add equivalent impedance to admittance matrix during frequency scan
	INTGAR(6) = 1	Add equivalent impedance to admittance matrix during frequency scan
	INTGAR(7)	Triplen harmonics treatment option in distortion calculations (alias is triplenop) (0 by default)
	INTGAR(7) = 0	Ignore triplen harmonics
	INTGAR(7) = 1	Consider triplen harmonics
	INTGAR(8)	Generator Impedance Option. (alias is genimpop) (0 by default)
	INTGAR(8) = 0	Use ZSORCE impedance from power flow data
	INTGAR(8) = 1	Use subtransient impedance from sequence data
	INTGAR(8) = 2	Use transient impedance from sequence data
	INTGAR(8) = 3	Use synchronous impedance from sequence data
	INTGAR(8) = 4	Use negative sequence impedance from sequence data
Real REALAR (3)	array of 3 elements specifying Real Argument Inputs(input).	
	REALAR(1)	Minimum Harmonic Order, must be >0. Specify hord_min=1.0 for fundamental frequen-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		cy. When hord_min=0.0, hord_min is set to hord_stp. (alias is hord_min) (0.0 by default)
	REALAR(1) = 0.0	Use default
	REALAR(1) > 0	Use specified
	REALAR(2)	Maximum Harmonic Order, must be >0. A frequency range upto 50th order harmonics is used in typical power quality assessments. When hord_max=0.0, hord_max is set to hord_stp. (alias is hord_max) (50.0 by default)
	REALAR(2) = 0.0	Use default
	REALAR(2) > 0	Use specified
	REALAR(3)	Harmonic Order Step, must be >0. Specify hord_stp=1.0 for scan in increments of base power frequency. (alias is hord_stp) (0.1 by default)
	REALAR(3) = 0.0	Use default
	REALAR(3) > 0	Use specified
Character*260 FILEAR (3)	array of 3 elements specifying File Argument Inputs(input).	
	FILEAR(1)	Harmonics Analysis Results Output file name (.hrs). (alias is hrsfile) (blank by default)
	FILEAR(1) = ' '	temporary file created
	FILEAR(1) = nooutput	no file created
	FILEAR(2)	Harmonic Analysis Distortion Calculation Report File Name (.txt). (alias is dstnfile) (blank by default)
	FILEAR(2) = ' '	report created in report window
	FILEAR(2) = nooutput	no report file created
	FILEAR(3)	Harmonic Analysis Frequency Scan Resonance Report File Name (.txt). (alias is resnfile) (blank by default)
	FILEAR(3) = ' '	report created in report window
	FILEAR(3) = nooutput	no report file created
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.2. HAR_ANALYSIS_2

Use this API to run harmonics analysis. Harmonics analysis is performed to calculate (1) frequency scan (driving point impedance scan) and (2) voltage and current distortions due to harmonics sources. The working case impedances are recalculated considering harmonics models of network elements and equipment for specified harmonic order range and driving point impedances calculated at specifed buses. The voltage and current distortions are calculated for all harmonic sources present.

This API routine was first introduced in release 35.6.0.

Batch command syntax:

```
BAT_HAR_ANALYSIS_2 SID ALL INTGAR REALAR FILEAR
```

Python command syntax:

```
ierr = har_analysis_2(sid, all, intgar, realar, filear)
```

Fortran command syntax:

```
CALL HAR_ANALYSIS_2(SID, ALL, INTGAR, REALAR, FILEAR, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGAR (17)	array of 17 elements specifying Integer Argument Inputs(input).	
	INTGAR(1)	Frequency Scan option (alias is fscanoptn) (1 by default)
	INTGAR(1) = 0	No
	INTGAR(1) = 1	Yes, run frequency scan calculations
	INTGAR(2)	Distortion Calculations option (alias is dstnoptn) (1 by default)
	INTGAR(2) = 0	No
	INTGAR(2) = 1	Yes, run voltage and current distortion calculations
	INTGAR(3)	Transmission Line Model (alias is linmdl) (0 by default)
	INTGAR(3) = 0	Nominal PI Line Model, no long line effects, no skin effects
	INTGAR(3) = 1	Distributed Parameter Line Model, long line effects and skin effects modeled
	INTGAR(3) = 2	Distributed Parameter Line Model, long line effects modeled, but skin effects not modeled
	INTGAR(4)	Bus report option in distortion analysis. Note- Frequency scan is always done for study

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	subsystem buses only. The distortion calculations are done for these buses. (alias is dstnbuop) (0 by default)
INTGAR(4) = 0	All buses in the case
INTGAR(4) = 1	Only study subsystem buses
INTGAR(5)	Report format option in distortion analysis (alias is dstnrptop) (0 by default)
INTGAR(5) = 0	Report voltage and current Total Harmonic Distortion (THD) Factors
INTGAR(5) = 1	Report THDs and current contributions
INTGAR(6)	Induction Machine Impedance Option. Note - The zero sequence impedance data always derived from sequence data. When sequence data is not specified, they are treated as open circuit. (alias is imachimpop) (0 by default)
INTGAR(6) = 0	Use positive sequence impedance derived from power flow data
INTGAR(6) = 1	Use positive sequence impedance derived from sequence data
INTGAR(7)	Two Terminal, VSC and Multi Terminal DC equivalent Impedance Option in frequency scan. Note - In distortion calculations, these are represented by either harmonic current sources when specified or equivalent impedances. (alias is dcimpop) (0 by default)
INTGAR(7) = 0	Do not add equivalent impedance to admittance matrix during frequency scan
INTGAR(7) = 1	Add equivalent impedance to admittance matrix during frequency scan
INTGAR(8)	Triplen harmonics treatment option in distortion calculations (alias is triplenop) (0 by default)
INTGAR(8) = 0	Ignore triplen harmonics
INTGAR(8) = 1	Consider triplen harmonics
INTGAR(9)	Generator Impedance Option. (alias is genimpop) (0 by default)
INTGAR(9) = 0	Use ZSORCE impedance from power flow data
INTGAR(9) = 1	Use subtransient impedance from sequence data
INTGAR(9) = 2	Use transient impedance from sequence data
INTGAR(9) = 3	Use synchronous impedance from sequence data
INTGAR(9) = 4	Use negative sequence impedance from sequence data
INTGAR(10)	Harmonics Model Type Option (alias is mdltypop) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

INTGAR(10) = 0	Use Model Type as specified in Harmonics Data File. When specified as inherit model type, use Model Type specified in API options.
INTGAR(10) = 1	Use Model Type specified in API options. The Model Type specified in Harmoics Data File is ignored.
INTGAR(11)	Load Model Type Option. (alias is load_mdltyp) (0 by default)
INTGAR(11) = 0	No frequency dependency, R and X used as in power flow data. R stays constant, X reactance changes per frequency.
INTGAR(11) = 1	Quality R constant (serial)
INTGAR(11) = 2	Quality X/R constant (serial)
INTGAR(11) = 3	Impedance characteristics
INTGAR(11) = 4	Quality X/R constant (parallel)
INTGAR(11) = 5	CIGRE Measurements model
INTGAR(11) = 6	Quality R constant (parallel)
INTGAR(11) = 7	IEEE Measurements model
INTGAR(11) = 8	IEEE RL Series Model
INTGAR(11) = 9	IEEE RL Parallel Model
INTGAR(11) = 10	Infinite, ignored in harmonics network
INTGAR(11) = 11	IEEE Skin Effect Model
INTGAR(11) = 12	IEEE Induction Motor Model
INTGAR(11) = 13	IEEE 2RL Parallel Model
INTGAR(11) = 14	CIGRE Passive Model
INTGAR(11) = 15	CIGRE Motive Model
INTGAR(11) = 16	Large Area Asynchronous Motor Model
INTGAR(12)	Generator Model Type Option. (alias is gen_mdltyp) (0 by default)
INTGAR(12) = 0	No frequency dependency, R and X used as in power flow data. R stays constant, X reactance changes per frequency.
INTGAR(12) = 1	quality constant R series
INTGAR(12) = 2	quality constant X/R series (generator R data if any, ignored and R calculated from HQUALITY)
INTGAR(12) = 3	Impedance characteristics
INTGAR(12) = 4	Model 1 IEEE
INTGAR(12) = 5	Model 2 CIGRE Electra 167
INTGAR(12) = 6	Model 3 HQ (Hydro Quebec)
INTGAR(12) = 9	Infinite, ignored in harmonics network
INTGAR(13)	Branch Shunt Model Type Option. (alias is brnsh_mdltyp) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

INTGAR(13) = 0	No frequency dependency, R and X used as in power flow data. R stays constant, X reactance changes per frequency.
INTGAR(13) = 1	quality constant R series
INTGAR(13) = 2	quality constant X/R series (applicable only when branch shunt is inductive), (Line shunt R data if any, ignored and shunt R calculated from HQUALITY)
INTGAR(13) = 3	quality constant R parallel
INTGAR(13) = 4	quality constant X/R parallel (applicable only when branch shunt is inductive), (Line shunt R data if any, ignored and shunt R calculated from HQUALITY)
INTGAR(14)	Transformer Model Type Option. (alias is trn_mdltyp) (0 by default)
INTGAR(14) = 0	No frequency dependency, R and X used as in power flow data. R stays constant, X reactance changes per frequency.
INTGAR(14) = 1	Model 1 CIGRE Electra No. 167
INTGAR(14) = 2	Model 2 IEEE Std. 399
INTGAR(14) = 3	Model 3 CIGRE Electra No. 164
INTGAR(14) = 4	Model 4 Arrillaga
INTGAR(14) = 5	Model 5 Funk
INTGAR(14) = 6	Impedance characteristics (allowed only for 2 winding transformers)
INTGAR(15)	FACTS Device Shunt Model Type Option. (alias is fctsh_mdltyp) (0 by default)
INTGAR(15) = 0	No frequency dependency, R and X used as in power flow data. R stays constant, X reactance changes per frequency.
INTGAR(15) = 1	Quality R constant (serial)
INTGAR(15) = 2	Quality X/R constant (serial)
INTGAR(15) = 3	Impedance characteristics
INTGAR(15) = 4	Quality X/R constant (parallel)
INTGAR(15) = 5	CIGRE Measurements model
INTGAR(15) = 6	Quality R constant (parallel)
INTGAR(15) = 9	Infinite
INTGAR(16)	FACTS Device Series Model Type Option. (alias is fctse_mdltyp) (0 by default)
INTGAR(16) = 0	No frequency dependency, R and X used as in power flow data. R stays constant, X reactance changes per frequency.
INTGAR(16) = 1	Quality R constant (serial)
INTGAR(16) = 2	Quality X/R constant (serial)
INTGAR(16) = 3	Impedance characteristics

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(17)	Induction Machine Model Type Option. (alias is imach_mdltyp) (0 by default)
	INTGAR(17) = 0	No frequency dependency, R and X used as in power flow data. R stays constant, X reactance changes per frequency.
	INTGAR(17) = 1	Quality R constant
	INTGAR(17) = 2	Quality X/R constant
	INTGAR(17) = 3	Impedance characteristics
	INTGAR(17) = 9	Infinite
Real REALAR (3)	array of 3 elements specifying Real Argument Inputs(input).	
	REALAR(1)	Minimum Harmonic Order, must be >0. Specify hord_min=1.0 for fundamental frequency. When hord_min=0.0, hord_min is set to hord_stp. (alias is hord_min) (0.0 by default)
	REALAR(1) = 0.0	Use default
	REALAR(1) > 0	Use specified
	REALAR(2)	Maximum Harmonic Order, must be >0. A frequency range upto 50th order harmonics is used in typical power quality assessments. When hord_max=0.0, hord_max is set to hord_stp. (alias is hord_max) (50.0 by default)
	REALAR(2) = 0.0	Use default
	REALAR(2) > 0	Use specified
	REALAR(3)	Harmonic Order Step, must be >0. Specify hord_stp=1.0 for scan in increments of base power frequency. (alias is hord_stp) (0.1 by default)
	REALAR(3) = 0.0	Use default
	REALAR(3) > 0	Use specified
Character*260 FILEAR (3)	array of 3 elements specifying File Argument Inputs(input).	
	FILEAR(1)	Harmonics Analysis Results Output file name (.hrs). (alias is hrsfile) (blank by default)
	FILEAR(1) = ' '	temporary file created
	FILEAR(1) = nooutput	no file created
	FILEAR(2)	Harmonic Analysis Distortion Calculation Report File Name (.txt). (alias is dstnfile) (blank by default)
	FILEAR(2) = ' '	report created in report window
	FILEAR(2) = nooutput	no report file created
	FILEAR(3)	Harmonic Analysis Frequency Scan Resonance Report File Name (.txt). (alias is resnfile) (blank by default)
	FILEAR(3) = ' '	report created in report window
	FILEAR(3) = nooutput	no report file created

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	Error - Prerequisite requirements for API are not met
	IERR = 2	Topology error, bus type code and series element status inconsistencies
	IERR = 3	Error - Invalid SID value or subsystem SID is not defined
	IERR = 4	Error - Invalid ALL value
	IERR = 5	Error - Invalid INTGAR value
	IERR = 6	Error - Invalid REALAR value
	IERR = 7	Info - No study subsystem buses, frequency scan not done
	IERR = 8	Info - No harmonics sources, distortion analysis not done
	IERR = 9	Error - working case generators are converted, harmonics analysis not done
	IERR = 10	Error - working case is not solved power flow case, harmonics analysis not done
	IERR = 11	Info - error adding PQ loads for fundamental harmonics sources, distortion analysis not done
	IERR = 12	Info - case not converged with PQ loads added for fundamental harmonics sources, distortion analysis not done
	IERR = 13	Error - restoring working case to start harmonics analysis
	IERR = 14	Error - Opening Binary Results (hrsfile) or Report File
	IERR = 15	Error - Writing to Binary Results (hrsfile) or Report File
	IERR = 16	Info - Frequency Scan Harmonic Order minimum is more than maximum, values swapped
	IERR = 17	Info - Info - Frequency Scan and Distortion Report file names are same, added 'fscan' and 'dstr' suffix

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.3. HAR_CREATE_PFCASE

Use this API to save working case updated for harmonic frequency and passive filters added as fixed shunts. The working case impedances are recalculated considering harmonics models of network elements and equipment for specified harmonic order.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CREATE_PFCASE HORD LINMDL SAVOUTFILE
```

Python command syntax:

```
ierr = har_create_pfcase(hord, linmdl, savoutfile)
```

Fortran command syntax:

```
CALL HAR_CREATE_PFCASE(HORD, LINMDL, SAVOUTFILE, IERR)
```

Where:

Real HORD	Harmonic order (=HarmonicFrequency/BaseFrequency), must be >0.0. Specify HORD=1.0 for fundamental frequency. (input; no default allowed).	
Integer LINMDL	Transmission Line Model (input; 0 by default).	
	LINMDL = 0	nominal PI Line Model, no long line effects, no skin effects
	LINMDL = 1	distributed Parameter Line Model, long line effects and skin effects modeled
	LINMDL = 2	distributed Parameter Line Model, long line effects modeled, but skin effects not modeled
Character*260 SAVOUTFILE	File name to save output file. If SAVOUTFILE exists, it will be overwritten. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	Error - Invalid harmonic order (HORD) value, must be >0.0
	IERR = 2	Error - SAVOUTFILE name not provided
	IERR = 3	Error - Prerequisite requirements for API are not met
	IERR = 4	Error opening SAVOUTFILE
	IERR = 5	Error writing to SAVOUTFILE

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.4. HAR_EXISTS_DSTN_RESULTS

Check if Harmonics Distortion Calculation results available in PSSE working memory from previously run harmonic analysis activity.

This API routine was first introduced in release 35.5.3.

Batch command syntax:

```
BAT_HAR_EXISTS_DSTN_RESULTS
```

Python command syntax:

```
ival = har_exists_dstn_results()
```

Fortran command syntax:

```
CALL HAR_EXISTS_DSTN_RESULTS(IVAL)
```

Where:

Integer IVAL	Flag value (output).	
	IVAL = 0	PSSE working memory does not have Distortion Calculation results
	IVAL = 1	PSSE working memory has Distortion Calculation results

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.5. HAR_EXISTS_FSCAN_RESULTS

Check if Harmonics Frequency Scan results available in PSSE working memory from previously run harmonic analysis activity.

This API routine was first introduced in release 35.5.3.

Batch command syntax:

```
BAT_HAR_EXISTS_FSCAN_RESULTS
```

Python command syntax:

```
ival = har_exists_fscan_results()
```

Fortran command syntax:

```
CALL HAR_EXISTS_FSCAN_RESULTS(IVAL)
```

Where:

Integer IVAL	Flag value (output).	
	IVAL = 0	PSSE working memory does not have Frequency Scan results
	IVAL = 1	PSSE working memory has Frequency Scan results

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.6. HAR_EXPORT_DSTN

Use this API to export harmonics distortion calculation results to comma separated (.csv) file. The harmonic analysis must be previously performed and working memory contains those results. Three CSV files are created with '_volt', '_flow' and '_cursrc' suffixes added to the input file name. File with '_volt' suffix contains bus voltages. File with'_flow' suffix contains flows in all network elements and equipment. File with '_cursrc' suffix contains harmonic current source spectrum applied in distortion calculations. For harmonic order 1, the column "%THDor%INDV" provides %THD (percent total harmonic distortion) and for all other harmonic orders it provides %INDV (percent individual harmonic in terms of fundamental).

This API routine was first introduced in release 35.4.2.

Batch command syntax:

```
BAT_HAR_EXPORT_DSTN CSVNAME INTGAR
```

Python command syntax:

```
ierr = har_export_dstn(csvname, intgar)
```

Fortran command syntax:

```
CALL HAR_EXPORT_DSTN(CSVNAME, INTGAR, IERR)
```

Where:

Character*260 CSVNAME	name of the file to which harmonics distortion calculation results are exported. (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying Integer Options for quantities to export(input).	
	INTGAR(1)	Bus Voltage Export Option (alias is voltoptn) (1 by default)
	INTGAR(1) = 0	No bus voltage export
	INTGAR(1) = 1	Export bus voltages for HORD=1 (only THDs exported)
	INTGAR(1) = 2	Export bus voltages for all HORD (THDs and individual harmonic levels exported)
	INTGAR(2)	Current Flow Export Option (alias is flowoptn) (0 by default)
	INTGAR(2) = 0	No current flows export
	INTGAR(2) = 1	Export flows for HORD=1 (only THDs exported)
	INTGAR(2) = 2	Export flows for all HORD (THDs and individual harmonic levels exported)
	INTGAR(3)	Harmonic Current Source Spectrum Option (alias is cursrcoptn) (0 by default)
	INTGAR(3) = 0	No harmonic current source export
	INTGAR(3) = 1	Export harmonic current sources for HORD=1 (only THDs exported)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(3) = 2	Export harmonic current sources for all HORD (THDs and individual harmonic levels exported)
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	no harmonics results exists
	IERR = 3	error opening CSVNAME file
	IERR = 4	CSVNAME is blank
	IERR = 5	invalid INTGAR value
	IERR = 6	error opening all output CSV files
	IERR = 7	error no INTGAR option to export specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.7. HAR_EXPORT_FSCAN

Use this API to export harmonics frequency scan results to comma separated (.csv) file. The harmonic analysis must be previously performed and working memory contains those results.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_EXPORT_FSCAN CSVNAME
```

Python command syntax:

```
ierr = har_export_fscan(csvname)
```

Fortran command syntax:

```
CALL HAR_EXPORT_FSCAN(CSVNAME, IERR)
```

Where:

Character*260 CSVNAME	name of the file to which harmonics frequency scan results are exported (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	no harmonics results exists
	IERR = 3	error opening CSVNAME file
	IERR = 4	CSVNAME is blank

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.8. HAR_LIMITS_SC_CURRENTS

Use this API to calculate maximum short circuit currents to be used by api HAR_LIMITS_VIOLATIONS_CURRENT. The current violations report created for buses in this file, so define subsystem accordingly. The short circuit calculations are run by activity ASCC for the working case in the memory with default arguments of activity ASCC and only three phase fault.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_HAR_LIMITS_SC_CURRENTS SID ALL
```

Python command syntax:

```
ierr = har_limits_sc_currents(sid, all)
```

Fortran command syntax:

```
CALL HAR_LIMITS_SC_CURRENTS(SID, ALL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; no default allowed).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	Error - Prerequisite requirements for API are not met
	IERR = 2	Error - Invalid SID value or subsystem SID is not defined
	IERR = 3	Error - Invalid ALL value
	IERR = 4	Error - No study subsystem buses, fault currents calculation not done
	IERR = 5	Error - ASCC activity error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.9. HAR_LIMITS_VIOLATIONS_CURRENT

Use this API to export harmonics current distortion limits violations to text file, csv file or report tab. The harmonic analysis must be previously performed and working memory contains those results.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_HAR_LIMITS_VIOLATIONS_CURRENT INTGAR(1) INTGAR(2) INTGAR(3) FILEAR(1)
```

Python command syntax:

```
ierr = har_limits_violations_current(intgar, filear)
```

Fortran command syntax:

```
CALL HAR_LIMITS_VIOLATIONS_CURRENT(INTGAR, FILEAR, IERR)
```

Where:

Integer INTGAR (3)	array of 3 elements specifying Integer Argument Inputs(input).	
	INTGAR(1)	harmonic limits standard option (alias is stdoptn) (1 by default)
	INTGAR(1) = 0	no standard [this reports THD and INDV for all buses]
	INTGAR(1) = 1	IEEE 519
	INTGAR(2)	report violations option (alias is viooptn) (1 by default)
	INTGAR(2) = 0	report all buses
	INTGAR(2) = 1	report only violated buses
	INTGAR(3)	report file option (alias is rptoptn) (0 by default)
	INTGAR(3) = 0	Combine THD and Individual Limits violations report in one file
	INTGAR(3) = 1	Separate THD and Individual Limits violations report but in in separate files. Added _thd and _indv suffix to filename. Used only when FILEAR name is specified.
Character*260 FILEAR (1)	array of 1 elements specifying name of the file to which harmonics current distortion limits violations report is written. For blank file name, report directed to Report Device. When file name extension is csv, comma separated values file is created.(input).	
	FILEAR(1)	name of the file to which harmonics current distortion limits violations report is written. For blank file name, report directed to Report Device. When file name extension is csv, comma separated values file is created. (alias is rptfil) (blank by default)
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error occurred
IERR = 1	Error - Prerequisite requirements for API are not met
IERR = 2	Error - No harmonics results exists
IERR = 3	Error - Opening output file, output saved to report device
IERR = 4	Invalid INTGAR value, set to default

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.10. HAR_LIMITS_VIOLATIONS_VOLTAGE

Use this API to export harmonics voltage distortion limits violations to text file, csv file or report tab. The harmonic analysis must be previously performed and working memory contains those results.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_HAR_LIMITS_VIOLATIONS_VOLTAGE INTGAR(1) INTGAR(2) INTGAR(3) RPTFIL
```

Python command syntax:

```
ierr = har_limits_violations_voltage(intgar, rptfil)
```

Fortran command syntax:

```
CALL HAR_LIMITS_VIOLATIONS_VOLTAGE(INTGAR, RPTFIL, IERR)
```

Where:

Integer INTGAR (3)	array of 3 elements specifying Integer Argument Inputs(input).	
	INTGAR(1)	harmonic limits standard option (alias is stdoptn) (1 by default)
	INTGAR(1) = 0	no standard [this reports THD and INDV for all buses]
	INTGAR(1) = 1	IEEE 519
	INTGAR(1) = 2	IEEE 2800
	INTGAR(2)	report violations option (alias is viooptn) (1 by default)
	INTGAR(2) = 0	report all buses
	INTGAR(2) = 1	report only violated buses
	INTGAR(3)	report file option (alias is rptoptn) (0 by default)
	INTGAR(3) = 0	Combine THD and Individual Limits violations report in one file
	INTGAR(3) = 1	Separate THD and Individual Limits violations report but in in separate files. Added _thd and _indv suffix to filename. Used only when RPTFIL name is specified.
Character*260 RPTFIL	name of the file to which harmonics voltage distortion limits violations report is written. For blank file name, report directed to Report Device. When file name extension is csv, comma separated values file is created. (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	Error - Prerequisite requirements for API are not met
	IERR = 2	Error - No harmonics results exists

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	Error opening output file, output saved to report device
IERR = 4	Invalid INTGAR value, set to default

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.11. HAR_NEW

Use this API to initialize all data records required for Harmonics analysis. It must be called prior to introducing any Harmonics data records. It has no effect if Harmonics data already exists in the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_NEW
```

Python command syntax:

```
ierr = har_new()
```

Fortran command syntax:

```
CALL HAR_NEW(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.12. HAR_PURG

Use this API to purge all harmonics data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_PURG
```

Python command syntax:

```
ierr = har_purg()
```

Fortran command syntax:

```
CALL HAR_PURG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.13. HAR_SET_RESN_THRESHOLDS

Use this API to set parameters to filter resonances from harmonic analysis frequency scan response. Two criterions are used to filters peaks that could be considered as resonances. Criterion 1: Slow rising resonance peak, specifiy PCTMX1 and DPCTPN1. Criterion 2: Fast rising resonance peak, specifiy PCTMX2 and DPCTPN2.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_SET_RESN_THRESHOLDS REALAR(1) REALAR(2) REALAR(3) REALAR(4)
```

Python command syntax:

```
har_set_resn_thresholds(realar)
```

Fortran command syntax:

```
CALL HAR_SET_RESN_THRESHOLDS(REALAR)
```

Where:

Real REALAR (4)	array of 4 elements specifying Real Argument Inputs(input).	
	REALAR(1)	current point in percent of maximum peak value (alias is PCTMX1) (60.0 by default)
	REALAR(1) = 0.0	Use default
	REALAR(1) > 0	Use specified
	REALAR(2)	delta previous and delta next points in percent of current point (alias is DPCTPN1) (75.0 by default)
	REALAR(2) = 0.0	Use default
	REALAR(2) > 0	Use specified
	REALAR(3)	current point in percent of maximum peak value (alias is PCTMX2) (20.0 by default)
	REALAR(3) = 0.0	Use default
	REALAR(3) > 0	Use specified
	REALAR(4)	delta previous and delta next points in percent of current point (alias is DPCTPN2) (85.0 by default)
	REALAR(4) = 0.0	Use default
	REALAR(4) > 0	Use specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.1.14. HAR_SET_RESN_THRESHOLDS_DEFAULT

Use this API to set default parameters to filter resonances from harmonic analysis frequency scan response. Two criterions are used to filters peaks that could be considered as resonances. Criterion 1: Slow rising resonance peaks. Criterion 2: Fast rising resonance peaks.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_SET_RESN_THRESHOLDS_DEFAULT
```

Python command syntax:

```
har_set_resn_thresholds_default()
```

Fortran command syntax:

```
CALL HAR_SET_RESN_THRESHOLDS_DEFAULT
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Harmonics Operation APIs contains API routines corresponding to the operational functions of the Harmonics features of PSS®E.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2. Harmonics Data

5.2.1. HAR_2TDC

Use this API to modify or add new harmonics Two Terminal DC data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_2TDC NAME IBUS INTGAR(1) CHARAR(1) CHARAR(2)
```

Python command syntax:

```
ierr = har_2tdc(name,ibus,intgar,charar)
```

Fortran command syntax:

```
CALL HAR_2TDC(NAME,IBUS,INTGAR,CHARAR,IERR)
```

Where:

Character*12 NAME	two term dc line name (input; no default allowed).	
Integer IBUS	rectifier or inverter bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Voltage Source Table Name
	CHARAR(2)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.2. HAR_2TDC_CHNG

Use this API to modify harmonics Two Terminal DC data in the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_2TDC_CHNG NAME IBUS INTGAR(1) CHARAR(1) CHARAR(2)
```

Python command syntax:

```
ierr = har_2tdc_chng(name,ibus,intgar,charar)
```

Fortran command syntax:

```
CALL HAR_2TDC_CHNG(NAME,IBUS,INTGAR,CHARAR,IERR)
```

Where:

Character*12 NAME	two term dc line name (input; no default allowed).	
Integer IBUS	rectifier or inverter bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Voltage Source Table Name
	CHARAR(2)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.3. HAR_2TDC_PURG

Use this API to purge specified harmonics Two Terminal DC data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_2TDC_PURG NAME IBUS
```

Python command syntax:

```
ierr = har_2tdc_purg(name, ibus)
```

Fortran command syntax:

```
CALL HAR_2TDC_PURG(NAME, IBUS, IERR)
```

Where:

Character*12 NAME	two term dc line name (input; no default allowed).	
Integer IBUS	rectifier or inverter bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.4. HAR_BRN

Use this API to modify or add new harmonics branch data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_BRN IBUS JBUS CKT INTGAR(1)..INTGAR(2) REALAR(1) CHARAR(1)
```

Python command syntax:

```
ierr = har_brn(ibus,jbus,ckt,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_BRN(IBUS,JBUS,CKT,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	branch identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (1)	array of 1 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.5. HAR_BRN_CHNG

Use this API to modify harmonics branch data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_BRN_CHNG IBUS JBUS CKT INTGAR(1)..INTGAR(2) REALAR(1) CHARAR(1)
```

Python command syntax:

```
ierr = har_brn_chng(ibus,jbus,ckt,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_BRN_CHNG(IBUS,JBUS,CKT,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	branch identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (1)	array of 1 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.6. HAR_BRN_PURG

Use this API to purge specified harmonics branch data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_BRN_PURG IBUS JBUS CKT
```

Python command syntax:

```
ierr = har_brn_purg(ibus,jbus,ckt)
```

Fortran command syntax:

```
CALL HAR_BRN_PURG(IBUS, JBUS, CKT, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.7. HAR_CURSRC

Use this API to modify or add new harmonics current source table data to the working case. This API is used to add new table of current and angle type data, harmonic numbers and corresponding I/I0 and Angle data points. This can be used to add a new data point of harmonic number, I/I0 and Angle data to existing table. This can be used to change current and angle type data, I/I0 and Angle data points of existing harmonic number in the table. This API cannot be used to change table name and harmonic numbers.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC NAME NPTS INTGAR(1) INTGAR(2) HAR(1)..HAR(NPTS)
 IAR(1)..IAR(NPTS) ANAR(1)..ANAR(NPTS)
```

Python command syntax:

```
ierr = har_cursrc(name, npts, intgar, har, iar, anar)
```

Fortran command syntax:

```
CALL HAR_CURSRC(NAME, NPTS, INTGAR, HAR, IAR, ANAR, IERR)
```

Where:

Character*40 NAME	Harmonics current Source Data table name (input; no default allowed).	
Integer NPTS	number of data points specified in the table (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	CRTYP, Harmonics current Source type (1 by default)
	INTGAR(2)	ANTYP, Harmonics current Source angle type (1 by default)
Real HAR (NPTS)	array of 'NPTS' values specifying harmonic numbers. All the harmonic numbers in HAR array must be non zero. (input).	
Real IAR (NPTS)	array of 'NPTS' values specifying current magnitude (input).	
Real ANAR (NPTS)	array of 'NPTS' values specifying angle (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.8. HAR_CURSRC_CHNG

Use this API to change harmonics current source table data in the working case. This API is used to change current and angle type data, I/I0 and Angle data points of existing harmonic number and existing table in working case harmonics data. This API cannot be used to change table name and harmonic numbers. All the harmonic numbers in HAR array must be non zero.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC_CHNG NAME NPTS INTGAR(1) INTGAR(2) HAR(1)..HAR(NPTS)
 IAR(1)..IAR(NPTS) ANAR(1)..ANAR(NPTS)
```

Python command syntax:

```
ierr = har_cursrc_chng(name, npts, intgar, har, iar, anar)
```

Fortran command syntax:

```
CALL HAR_CURSRC_CHNG(NAME, NPTS, INTGAR, HAR, IAR, ANAR, IERR)
```

Where:

Character*40 NAME	Harmonics current Source Data table name (input; no default allowed).	
Integer NPTS	number of data points specified in the table (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	CRTYP, Harmonics current Source type (1 by default)
	INTGAR(2)	ANTYP, Harmonics current Source angle type (1 by default)
Real HAR (NPTS)	array of 'NPTS' values specifying harmonic numbers. All the harmonic numbers in HAR array must be non zero. (input).	
Real IAR (NPTS)	array of 'NPTS' values specifying current magnitude (input).	
Real ANAR (NPTS)	array of 'NPTS' values specifying angle (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.9. HAR_CURSRC_CHNG_NAME

Use this API to change harmonics current source table name in the working case. This API is used to change table name from working case harmonics data. NAMNEW must not exist in working case harmonics data. NAME must be provided and it must exist in working case harmonics data, then table name changed from NAME to NAMNEW.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC_CHNG_NAME  'NAME'  'NAMNEW' INTGAR
```

Python command syntax:

```
ierr = har_cursrc_chng_name(name, namnew, intgar)
```

Fortran command syntax:

```
CALL HAR_CURSRC_CHNG_NAME(NAME, NAMNEW, INTGAR, IERR)
```

Where:

Character*40 NAME	Harmonics current Source table name whose name is to be changed. NAME must be provided and it must exist in working case harmonics data. (input; no default allowed).	
Character*40 NAMNEW	New Harmonics current Source table name to be assigned. NAMNEW must not exist in working case harmonics data. (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	CRTYP, Harmonics current Source type (1 by default)
	INTGAR(2)	ANTYP, Harmonics current Source angle type (1 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.10. HAR_CURSRC_CHNG_ONEPT

Use this API to modify one data point to harmonics current source table in the working case. This API is used to change H, I/I0, and Angle data of existing harmonic number (HPTOLD) in the existing working case harmonics data table. This API cannot be used to change table name.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC_CHNG_ONEPT NAME HPTOLD HPT IPT ANPT
```

Python command syntax:

```
ierr = har_cursrc_chng_onept(name, hptold, hpt, ipt, anpt)
```

Fortran command syntax:

```
CALL HAR_CURSRC_CHNG_ONEPT(NAME, HPTOLD, HPT, IPT, ANPT, IERR)
```

Where:

Character*40 NAME	Harmonics current Source Data table name (input; no default allowed).	
Real HPTOLD	Harmonics number at point in data table where provided data is changed (input; no default allowed).	
Real HPT	New Harmonics number to be changed (input; 1 by default).	
Real IPT	current Source magnitude for harmonics number HPT to be changed (input; 1 by default).	
Real ANPT	current Source angle for harmonics number HPT to be changed (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.11. HAR_CURSRC_NAME

Use this API to add or change harmonics current source table name in the working case. This API is used to add new table name with no harmonics data points or change table name from working case harmonics data.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC_CHNG_NAME  'NAME'  'NAMNEW' INTGAR
```

Python command syntax:

```
ierr = har_cursrc_name(name, namnew, intgar)
```

Fortran command syntax:

```
CALL HAR_CURSRC_NAME(NAME, NAMNEW, INTGAR, IERR)
```

Where:

Character*40 NAME	Harmonics current Source table name whose name is to be changed. When NAME is provided, it must exist in working case harmonics data, then table name changed from NAME to NAMNEW. If NAME is blank, a new table NAMNEW will be added with no harmonics data points. (input; " by default).	
Character*40 NAMNEW	New Harmonics current Source table name to be assigned. NAMNEW must not exist in working case harmonics data. (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	CRTYP, Harmonics current Source type (1 by default)
	INTGAR(2)	ANTYP, Harmonics current Source angle type (1 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.12. HAR_CURSRC_ONEPT

Use this API to modify or add new one data point to harmonics current source table in the working case. This API is used to add one new data point of H, I/I0, and Angle data to existing table of existing working case harmonics data. This can be used to change H, V/V0, and Angle data points of existing harmonic number in the table. This API cannot be used to change table name.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC_ONEPT NAME HPTOLD HPT IPT ANPT
```

Python command syntax:

```
ierr = har_cursrc_onept(name, hptold, hpt, ipt, anpt)
```

Fortran command syntax:

```
CALL HAR_CURSRC_ONEPT(NAME, HPTOLD, HPT, IPT, ANPT, IERR)
```

Where:

Character*40 NAME	Harmonics current Source Data table name (input; no default allowed).	
Real HPTOLD	Harmonics number at point in data table where provided data is added or changed. When HPTOLD is specified and found, the H, I/I0, and Angle values specified will be used to change existing data. When HPTOLD is zero, the H, I/I0, and Angle values specified will be used to add data point to existing data. (input; no default allowed).	
Real HPT	New Harmonics number to be added or changed at specified data point (input; no default allowed).	
Real IPT	current Source magnitude for harmonics number HPT to be added or changed (input; 1 by default).	
Real ANPT	current source angle for harmonics number HPT to be added or changed (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.13. HAR_CURSRC_PURG

Use this API to purge specified harmonics current source table data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC_PURG NAME
```

Python command syntax:

```
ierr = har_cursrc_purg(name)
```

Fortran command syntax:

```
CALL HAR_CURSRC_PURG(NAME, IERR)
```

Where:

Character*40 NAME	Harmonics current source table name (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.14. HAR_CURSRC_PURG_ONEPT

Use this API to purge specified data point in harmonics current source table.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_CURSRC_PURG_ONEPT NAME HPT
```

Python command syntax:

```
ierr = har_cursrc_purg_onept(name, hpt)
```

Fortran command syntax:

```
CALL HAR_CURSRC_PURG_ONEPT(NAME, HPT, IERR)
```

Where:

Character*40 NAME	Harmonics current source table name (input; no default allowed).	
Real HPT	Harmonic number (H) at a point in data table where H, R/R0, L/) and C/C0 data is purged (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.15. HAR_FACTS

Use this API to modify or add new harmonics FACTS device data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_FACTS NAME SENDBUS SENDBUS INTGAR(1) INTGAR(2) CHARAR(1) CHARAR(2)
 CHARAR(3)
```

Python command syntax:

```
ierr = har_facts(name,sendbus,termbus,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_FACTS(NAME,SENDBUS,TERMBUS,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Character*12 NAME	FACTS device name (input; no default allowed).	
Integer SENDBUS	FACTS Device sending bus number (input; no default allowed).	
Integer TERMBUS	FACTS Device terminal bus number (input; 0 by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.16. HAR_FACTS_CHNG

Use this API to modify harmonics FACTS device data in the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_FACTS_CHNG NAME SENDBUS SENDBUS INTGAR(1) INTGAR(2) CHARAR(1)
 CHARAR(2) CHARAR(3)
```

Python command syntax:

```
ierr = har_facts_chng(name,sendbus,termbus,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_FACTS_CHNG(NAME,SENDBUS,TERMBUS,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Character*12 NAME	FACTS device name (input; no default allowed).	
Integer SENDBUS	FACTS Device sending bus number (input; no default allowed).	
Integer TERMBUS	FACTS Device terminal bus number (input; 0 by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.17. HAR_FACTS_PURG

Use this API to purge specified harmonics FACTS device data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_FACTS_PURG NAME SENDBUS TERMBUS
```

Python command syntax:

```
ierr = har_facts_purg(name,sendbus,termbus)
```

Fortran command syntax:

```
CALL HAR_FACTS_PURG(NAME,SENDBUS,TERMBUS,IERR)
```

Where:

Character*12 NAME	FACTS device name (input; no default allowed).	
Integer SENDBUS	FACTS Device sending bus number (input; no default allowed).	
Integer TERMBUS	FACTS Device terminal bus number (input; 0 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.18. HAR_IMPCHAR

Use this API to modify or add new harmonics impedance characteristics table data to the working case. This API is used to add new table of harmonic numbers and corresponding R/R0, L/L0 and C/C0 data points. This can be used to add a new data point of harmonic number, R/R0, L/L0 and C/C0 data to existing table. This can be used to change R/R0, L/L0 and C/C0 data points of existing harmonic number in the table. This API cannot be used to change table name and harmonic numbers.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR NAME NPTS HAR(1)..HAR(NPTS) RAR(1)..RAR(NPTS)
 LAR(1)..LAR(NPTS) CAR(1)..CAR(NPTS)
```

Python command syntax:

```
ierr = har_impchar(name, npts, har, rar, lar, car)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR(NAME, NPTS, HAR, RAR, LAR, CAR, IERR)
```

Where:

Character*40 NAME	Harmonics Impedance Characteristics Data table name (input; no default allowed).	
Integer NPTS	number of data points specified in the table (input; no default allowed).	
Real HAR (NPTS)	array of 'NPTS' values specifying harmonic numbers. All the harmonic numbers in HAR array must be non zero. (input).	
Real RAR (NPTS)	array of 'NPTS' values specifying R/R0 ratio (input).	
Real LAR (NPTS)	array of 'NPTS' values specifying L/L0 ratio (input).	
Real CAR (NPTS)	array of 'NPTS' values specifying C/C0 ratio (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.19. HAR_IMPCHAR_CHNG

Use this API to change harmonics impedance characteristics table data in the working case. This API is used to change R/R0, L/L0 and C/C0 data points of existing harmonic number and existing table in working case harmonics data. This API cannot be used to change table name and harmonic numbers. All the harmonic numbers in HAR array must be non zero.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR_CHNG NAME NPTS HAR(1)..HAR(NPTS) RAR(1)..RAR(NPTS)
 LAR(1)..LAR(NPTS) CAR(1)..CAR(NPTS)
```

Python command syntax:

```
ierr = har_impchar_chng(name, npts, har, rar, lar, car)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR_CHNG(NAME, NPTS, HAR, RAR, LAR, CAR, IERR)
```

Where:

Character*40 NAME	Harmonics Impedance Characteristics Data table name (input; no default allowed).	
Integer NPTS	number of data points specified in the table (input; no default allowed).	
Real HAR (NPTS)	array of 'NPTS' values specifying harmonic numbers. All the harmonic numbers in HAR array must be non zero. (input).	
Real RAR (NPTS)	array of 'NPTS' values specifying R/R0 ratio (input).	
Real LAR (NPTS)	array of 'NPTS' values specifying L/L0 ratio (input).	
Real CAR (NPTS)	array of 'NPTS' values specifying C/C0 ratio (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.20. HAR_IMPCHAR_CHNG_NAME

Use this API to change harmonics impedance characteristics table name in the working case. This API is used to change table name from working case harmonics data. NAMNEW must not exist in working case harmonics data. NAME must be provided and it must exist in working case harmonics data, then table name changed from NAME to NAMNEW.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR_CHNG_NAME  'NAME'  'NAMNEW'
```

Python command syntax:

```
ierr = har_impchar_chng_name(name, namnew)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR_CHNG_NAME(NAME, NAMNEW, IERR)
```

Where:

Character*40 NAME	Harmonics impedance characteristics table name whose name is to be changed. NAME must be provided and it must exist in working case harmonics data. (input; no default allowed).	
Character*40 NAMNEW	New Harmonics Impedance Characteristics table name to be assigned. NAMNEW must not exist in working case harmonics data. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.21. HAR_IMPCHAR_CHNG_ONEPT

Use this API to modify one data point to harmonics impedance characteristics table in the working case. This API is used to change H, R/R0, L/L0 and C/C0 data of existing harmonic number (HPTOLD) in the existing working case harmonics data table. This API cannot be used to change table name.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR_CHNG_ONEPT NAME HPTOLD HPT RPT LPT CPT
```

Python command syntax:

```
ierr = har_impchar_chng_onept(name, hptold, hpt, rpt, lpt, cpt)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR_CHNG_ONEPT(NAME, HPTOLD, HPT, RPT, LPT, CPT, IERR)
```

Where:

Character*40 NAME	Harmonics Impedance Characteristics Data table name (input; no default allowed).	
Real HPTOLD	Harmonics number at point in data table where provided data is changed (input; no default allowed).	
Real HPT	New Harmonics number to be changed (input; 1 by default).	
Real RPT	Impedance characteristics R/R0 for harmonics number HPT to be changed (input; 1 by default).	
Real LPT	Impedance characteristics L/L0 for harmonics number HPT to be changed (input; 1 by default).	
Real CPT	Impedance characteristics C/C0 for harmonics number HPT to be changed (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.22. HAR_IMPCHAR_NAME

Use this API to add or change harmonics impedance characteristics table name in the working case. This API is used to add new table name with no harmonics data points or change table name from working case harmonics data.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR_CHNG_NAME  'NAME'  'NAMNEW'
```

Python command syntax:

```
ierr = har_impchar_name(name, namnew)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR_NAME(NAME, NAMNEW, IERR)
```

Where:

Character*40 NAME	Harmonics Impedance Characteristics table name whose name is to be changed. When NAME is provided, it must exist in working case harmonics data, then table name changed from NAME to NAMNEW. If NAME is blank, a new table NAMNEW will be added with no harmonics data points. (input; '' by default).	
Character*40 NAMNEW	New Harmonics Impedance Characteristics table name to be assigned. NAMNEW must not exist in working case harmonics data. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.23. HAR_IMPCHAR_ONEPT

Use this API to modify or add new one data point to harmonics impedance characteristics table in the working case. This API is used to add one new data point of H, R/R0, L/L0 and C/C0 data to existing table of existing working case harmonics data. This can be used to change H, R/R0, L/L0 and C/C0 data points of existing harmonic number in the table. This API cannot be used to change table name.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR_ONEPT NAME HPTOLD HPT RPT LPT CPT
```

Python command syntax:

```
ierr = har_impchar_onept(name, hptold, hpt, rpt, lpt, cpt)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR_ONEPT(NAME, HPTOLD, HPT, RPT, LPT, CPT, IERR)
```

Where:

Character*40 NAME	Harmonics Impedance Characteristics Data table name (input; no default allowed).	
Real HPTOLD	Harmonics number at point in data table where provided data is added or changed. When HPTOLD is specified and found, the H, R/R0, L/L0 and C/C0 values specified will be used to change existing data. When HPTOLD is zero, the H, R/R0, L/L0 and C/C0 values specified will be used to add data point to existing data. (input; no default allowed).	
Real HPT	New Harmonics number to be added or changed at specified data point (input; no default allowed).	
Real RPT	Impedance characteristics R/R0 for harmonics number HPT to be added or changed (input; 1 by default).	
Real LPT	Impedance characteristics L/L0 for harmonics number HPT to be added or changed (input; 1 by default).	
Real CPT	Impedance characteristics C/C0 for harmonics number HPT to be added or changed (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.24. HAR_IMPCHAR_PURG

Use this API to purge specified harmonics impedance characteristics table data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR_PURG NAME
```

Python command syntax:

```
ierr = har_impchar_purg(name)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR_PURG(NAME, IERR)
```

Where:

Character*40 NAME	Harmonics impedance characteristics table name (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.25. HAR_IMPCHAR_PURG_ONEPT

Use this API to purge specified data point in harmonics impedance characteristics table.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_IMPCHAR_PURG_ONEPT NAME HPT
```

Python command syntax:

```
ierr = har_impchar_purg_onept(name, hpt)
```

Fortran command syntax:

```
CALL HAR_IMPCHAR_PURG_ONEPT(NAME, HPT, IERR)
```

Where:

Character*40 NAME	Harmonics impedance characteristics table name (input; no default allowed).	
Real HPT	Harmonic number (H) at a point in data table where H, R/R0, L/) and C/C0 data is purged (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.26. HAR_INDMC

Use this API to modify or add new harmonics induction machine data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_INDMC IBUS ID INTGAR(1)..INTGAR(2) REALAR(1) CHARAR(1)..CHARAR(3)
```

Python command syntax:

```
ierr = har_indmc(ibus,id,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_INDMC(IBUS,ID,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.27. HAR_INDMC_CHNG

Use this API to modify harmonics induction machine data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_INDMC_CHNG IBUS ID INTGAR(1)..INTGAR(2) REALAR(1)
 CHARAR(1)..CHARAR(3)
```

Python command syntax:

```
ierr = har_indmc_chng(ibus,id,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_INDMC_CHNG(IBUS,ID,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.28. HAR_INDMC_PURG

Use this API to purge specified harmonics induction machine data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_INDMC_PURG IBUS ID
```

Python command syntax:

```
ierr = har_indmc_purg(ibus,id)
```

Fortran command syntax:

```
CALL HAR_INDMC_PURG(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.29. HAR_LOAD

Use this API to modify or add new harmonics load data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_LOAD IBUS ID INTGAR(1)..INTGAR(2) REALAR(1) CHARAR(1)..CHARAR(3)
```

Python command syntax:

```
ierr = har_load(ibus,id,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_LOAD(IBUS,ID,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (3)	array of 3 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
	REALAR(2)	Control Current (0.0 by default)
	REALAR(3)	Reference Compensation Power (0.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.30. HAR_LOAD_CHNG

Use this API to modify harmonics load data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_LOAD_CHNG IBUS ID INTGAR(1)..INTGAR(2) REALAR(1)
 CHARAR(1)..CHARAR(3)
```

Python command syntax:

```
ierr = har_load_chng(ibus,id,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_LOAD_CHNG(IBUS,ID,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (3)	array of 3 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
	REALAR(2)	Control Current (0.0 by default)
	REALAR(3)	Reference Compensation Power (0.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.31. HAR_LOAD_PURG

Use this API to purge specified harmonics load data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_LOAD_PURG IBUS ID
```

Python command syntax:

```
ierr = har_load_purg(ibus,id)
```

Fortran command syntax:

```
CALL HAR_LOAD_PURG(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.32. HAR_MACH

Use this API to modify or add new harmonics machine data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_MACH IBUS ID INTGAR(1)..INTGAR(2) REALAR(1) CHARAR(1)..CHARAR(3)
```

Python command syntax:

```
ierr = har_mach(ibus,id,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_MACH(IBUS,ID,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.33. HAR_MACH_CHNG

Use this API to modify harmonics machine data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_MACH_CHNG IBUS ID INTGAR(1)..INTGAR(2) REALAR(1)
 CHARAR(1)..CHARAR(3)
```

Python command syntax:

```
ierr = har_mach_chng(ibus,id,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_MACH_CHNG(IBUS,ID,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (3)	array of 3 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Voltage Source Table Name
	CHARAR(3)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.34. HAR_MACH_PURG

Use this API to purge specified harmonics machine data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_MACH_PURG IBUS ID
```

Python command syntax:

```
ierr = har_mach_purg(ibus,id)
```

Fortran command syntax:

```
CALL HAR_MACH_PURG(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	mach identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.35. HAR_MTDC

Use this API to modify or add new harmonics Multi Terminal DC data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_MTDC NAME IBUS INTGAR(1) CHARAR(1) CHARAR(2)
```

Python command syntax:

```
ierr = har_mtdc(name,ibus,intgar,charar)
```

Fortran command syntax:

```
CALL HAR_MTDC(NAME,IBUS,INTGAR,CHARAR,IERR)
```

Where:

Character*12 NAME	multi term dc line name (input; no default allowed).	
Integer IBUS	converter bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Voltage Source Table Name
	CHARAR(2)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.36. HAR_MTDC_CHNG

Use this API to modify harmonics Multi Terminal DC data in the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_MTDC_CHNG NAME IBUS INTGAR(1) CHARAR(1) CHARAR(2)
```

Python command syntax:

```
ierr = har_mtdc_chng(name,ibus,intgar,charar)
```

Fortran command syntax:

```
CALL HAR_MTDC_CHNG(NAME,IBUS,INTGAR,CHARAR,IERR)
```

Where:

Character*12 NAME	multi term dc line name (input; no default allowed).	
Integer IBUS	converter bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Voltage Source Table Name
	CHARAR(2)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.37. HAR_MTDC_PURG

Use this API to purge specified harmonics Multi Terminal DC data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_MTDC_PURG NAME IBUS
```

Python command syntax:

```
ierr = har_mtdc_purg(name, ibus)
```

Fortran command syntax:

```
CALL HAR_MTDC_PURG(NAME, IBUS, IERR)
```

Where:

Character*12 NAME	multi term dc line name (input; no default allowed).	
Integer IBUS	converter bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.38. HAR_PASSIVE_FILTER

Use this API to modify or add new harmonics passive filter data to the working case. The passive filter added gets represented as the fixed bus shunt in the power flow case. Therefore, the passive filter identifier (ibus, id) specified should not exist as the fixed bus shunt identifier (ibus, id) in the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_PASSIVE_FILTER IBUS ID INTGAR(1) INTGAR(2) INTGAR(3) REALAR(1)
 REALAR(2) REALAR(3)
```

Python command syntax:

```
ierr = har_passive_filter(ibus,id,intgar,realar)
```

Fortran command syntax:

```
CALL HAR_PASSIVE_FILTER(IBUS,ID,INTGAR,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	passive filter identifier (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	Current Control State (alias is HSTATE) (1 by default)
	INTGAR(1) = 0	Not active in harmonics
	INTGAR(1) = 1	Active in harmonics
	INTGAR(2)	units in which impedance or admittance data (RG, XB1, XB2) are specified (alias is CZY) (0 by default)
	INTGAR(2) = 0	Impedance expressed in PU on bus voltage and system MVA base
	INTGAR(2) = 1	Admittance expressed in PU on bus voltage and system MVA base
	INTGAR(2) = 2	Admittance expressed in MW and MVAR at 1 PU voltage
	INTGAR(2) = 3	Impedance expressed in ohms (XB1 and XB2 expressed as positive for inductive and negative for capacitive reactances)
	INTGAR(2) = 4	RG is resistance, XB1 is inductive reactance and XB2 is capacitive susceptance expressed in PU on bus voltage and system MVA base
	INTGAR(3)	configuration of impedance or admittance data (RG, XB1, XB2) (alias is CFG) (0 by default)
	INTGAR(3) = 0	RG, XB1, XB2 are connected in series
	INTGAR(3) = 1	RG, XB1, XB2 are connected in parallel

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(3) = 2	RG and XB1 are connected in parallel and that equivalent in series with XB2
	INTGAR(3) = 3	RG and XB2 are connected in parallel and that equivalent in series with XB1
Real REALAR (3)	array of 3 elements specifying (input).	
	REALAR(1)	RG, passive filter resistance or conductance or MW at fundamental frequency (alias is RG) (0.0 by default)
	REALAR(2)	XB1, passive filter reactance or susceptance or MVAR at fundamental frequency. Specify positive for inductive reactance, inductive susceptance or capacitive MVAR. Specify negative for capacitive reactance, capacitive susceptance or inductive MVAR. (alias is XB1) (0.0 by default)
	REALAR(3)	XB2, passive filter reactance or susceptance or MVAR at fundamental frequency. Specify positive for inductive reactance, inductive susceptance or capacitive MVAR. Specify negative for capacitive reactance, capacitive susceptance or inductive MVAR. (alias is XB2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.39. HAR_PASSIVE_FILTER_CHNG

Use this API to modify harmonics passive filter data in the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_PASSIVE_FILTER_CHNG IBUS ID INTGAR(1) INTGAR(2) INTGAR(3)
 REALAR(1) REALAR(2) REALAR(3)
```

Python command syntax:

```
ierr = har_passive_filter_chng(ibus,id,intgar,realar)
```

Fortran command syntax:

```
CALL HAR_PASSIVE_FILTER_CHNG(IBUS,ID,INTGAR,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	passive filter identifier (input; no default allowed).	
Integer INTGAR (3)	array of 3 elements specifying (input).	
	INTGAR(1)	Current Control State (alias is HSTATE) (1 by default)
	INTGAR(1) = 0	Not active in harmonics
	INTGAR(1) = 1	Active in harmonics
	INTGAR(2)	units in which impedance or admittance data (RG, XB1, XB2) are specified (alias is CZY) (0 by default)
	INTGAR(2) = 0	Impedance expressed in PU on bus voltage and system MVA base
	INTGAR(2) = 1	Admittance expressed in PU on bus voltage and system MVA base
	INTGAR(2) = 2	Admittance expressed in MW and MVAR at 1 PU voltage
	INTGAR(2) = 3	Impedance expressed in ohms (XB1 and XB2 expressed as positive for inductive and negative for capacitive reactances)
	INTGAR(2) = 4	RG is resistance, XB1 is inductive reactance and XB2 is capacitive susceptance expressed in PU on bus voltage and system MVA base
	INTGAR(3)	configuration of impedance or admittance data (RG, XB1, XB2) (alias is CFG) (0 by default)
	INTGAR(3) = 0	RG, XB1, XB2 are connected in series
	INTGAR(3) = 1	RG, XB1, XB2 are connected in parallel
	INTGAR(3) = 2	RG and XB1 are connected in parallel and that equivalent in series with XB2

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(3) = 3	RG and XB2 are connected in parallel and that equivalent in series with XB1
Real REALAR (3)	array of 3 elements specifying (input).	
	REALAR(1)	RG, passive filter resistance or conductance or MW at fundamental frequency (alias is RG) (0.0 by default)
	REALAR(2)	XB1, passive filter reactance or susceptance or MVAR at fundamental frequency. Specify positive for inductive reactance, inductive susceptance or capacitive MVAR. Specify negative for capacitive reactance, capacitive susceptance or inductive MVAR. (alias is XB1) (0.0 by default)
	REALAR(3)	XB2, passive filter reactance or susceptance or MVAR at fundamental frequency. Specify positive for inductive reactance, inductive susceptance or capacitive MVAR. Specify negative for capacitive reactance, capacitive susceptance or inductive MVAR. (alias is XB2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.40. HAR_PASSIVE_FILTER_PURG

Use this API to purge specified harmonics passive filter data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_PASSIVE_FILTER_PURG IBUS ID
```

Python command syntax:

```
ierr = har_passive_filter_purg(ibus,id)
```

Fortran command syntax:

```
CALL HAR_PASSIVE_FILTER_PURG(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	passive filter identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.41. HAR_SET_PAR_LODMDL_CIGRE_MEASUREMENT

Use this API to set parameters for harmonics load data user model CIGRE_MEASUREMENT.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_CIGRE_MEASUREMENT IBUS ID REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_cigre_measurement(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_CIGRE_MEASUREMENT(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (4)	array of 4 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (0.073 by default)
	REALAR(2)	Real Constant K2 (alias is K2) (6.7 by default)
	REALAR(3)	Real Constant K3 (alias is K3) (0.74 by default)
	REALAR(4)	Real Constant K4 (alias is K4) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.42. HAR_SET_PAR_LODMDL_CIGRE_MOTIVE

Use this API to set parameters for harmonics load data user model CIGRE_MOTIVE.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_CIGRE_MOTIVE IBUS ID REALAR(1)..REALAR(2)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_cigre_motive(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_CIGRE_MOTIVE(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (1.0 by default)
	REALAR(2)	Real Constant K2 (alias is K2) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.43. HAR_SET_PAR_LODMDL_CIGRE_PASSIVE

Use this API to set parameters for harmonics load data user model CIGRE_PASSIVE.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_CIGRE_PASSIVE IBUS ID REALAR(1)..REALAR(1)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_cigre_passive(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_CIGRE_PASSIVE(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (1)	array of 1 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.44. HAR_SET_PAR_LODMDL_IEEE_2RL_PARALLEL

Use this API to set parameters for harmonics load data user model IEEE_2RL_PARALLEL.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_IEEE_2RL_PARALLEL IBUS ID REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_ieee_2rl_parallel(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_IEEE_2RL_PARALLEL(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (5)	array of 5 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (0.8 by default)
	REALAR(2)	Real Constant KM (alias is KM) (1.2 by default)
	REALAR(3)	Real Constant XM (alias is XM) (0.15 by default)
	REALAR(4)	Real Constant K2 (alias is K2) (0.1 by default)
	REALAR(5)	Real Constant K3 (alias is K3) (8.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.45. HAR_SET_PAR_LODMDL_IEEE_IM

Use this API to set parameters for harmonics load data user model IEEE_IM.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_IEEE_IM IBUS ID REALAR(1)..REALAR(3)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_ieee_im(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_IEEE_IM(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (3)	array of 3 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (0.8 by default)
	REALAR(2)	Real Constant KM (alias is KM) (1.2 by default)
	REALAR(3)	Real Constant XM (alias is XM) (0.15 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.46. HAR_SET_PAR_LODMDL_IEEE_MEASUREMENT

Use this API to set parameters for harmonics load data user model IEEE_MEASUREMENT.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_IEEE_MEASUREMENT IBUS ID REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_ieee_measurement(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_IEEE_MEASUREMENT(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (4)	array of 4 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (0.073 by default)
	REALAR(2)	Real Constant K2 (alias is K2) (6.7 by default)
	REALAR(3)	Real Constant K3 (alias is K3) (0.74 by default)
	REALAR(4)	Real Constant K4 (alias is K4) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.47. HAR_SET_PAR_LODMDL_IEEE_SKIN

Use this API to set parameters for harmonics load data user model IEEE_SKIN.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_IEEE_SKIN IBUS ID REALAR(1)..REALAR(2)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_ieee_skin(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_IEEE_SKIN(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (0.1 by default)
	REALAR(2)	Real Constant K2 (alias is K2) (0.9 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.48. HAR_SET_PAR_LODMDL_LARGE_ASYNC_MOTOR

Use this API to set parameters for harmonics load data user model LARGE_ASYNC_MOTOR.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_HAR_SET_PAR_LODMDL_LARGE_ASYNC_MOTOR IBUS ID REALAR(1)..REALAR(7)
```

Python command syntax:

```
ierr = har_set_par_lodmdl_large_async_motor(ibus,id,realar)
```

Fortran command syntax:

```
CALL HAR_SET_PAR_LODMDL_LARGE_ASYNC_MOTOR(IBUS,ID,REALAR,IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Real REALAR (7)	array of 7 elements specifying Refer model schematic diagram in PAG Harmonics chapter for information on each real constant.(input).	
	REALAR(1)	Real Constant K1 (alias is K1) (0.8 by default)
	REALAR(2)	Real Constant K2 (alias is K2) (0.05 by default)
	REALAR(3)	Real Constant K3 (alias is K3) (0.5 by default)
	REALAR(4)	Real Constant K4 (alias is K4) (0.5 by default)
	REALAR(5)	Real Constant K5 (alias is K5) (-0.15 by default)
	REALAR(6)	Real Constant XM (alias is XM) (0.15 by default)
	REALAR(7)	Real Constant RM (alias is RM) (0.03 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	Harmonics load model parameters not set: load not found
	IERR = 3	Harmonics load model parameters not set: model HTYP not allowed
	IERR = 4	Harmonics load model parameters not set: model does not take parameters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.49. HAR_TRN

Use this API to modify or add new harmonics transformer data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_TRN IBUS JBUS KBUS CKT INTGAR(1)..INTGAR(2) REALAR(1)
 CHARAR(1)..CHARAR(2)
```

Python command syntax:

```
ierr = har_trn(ibus,jbus,kbus,ckt,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_TRN(IBUS,JBUS,KBUS,CKT,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Integer KBUS	bus number of third bus (input; 0 by default).	
Character*2 CKT	transformer identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Current Source Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.50. HAR_TRN_CHNG

Use this API to modify harmonics transformer data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_TRN_CHNG IBUS JBUS KBUS CKT INTGAR(1)..INTGAR(2) REALAR(1)
 CHARAR(1)..CHARAR(2)
```

Python command syntax:

```
ierr = har_trn_chng(ibus,jbus,kbus,ckt,intgar,realar,charar)
```

Fortran command syntax:

```
CALL HAR_TRN_CHNG(IBUS,JBUS,KBUS,CKT,INTGAR,REALAR,CHARAR,IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Integer KBUS	bus number of third bus (input; 0 by default).	
Character*2 CKT	transformer identifier (input; '1' by default).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
	INTGAR(2)	Harmonics Type (1 by default)
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Harmonics Quality (1.0 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Impedance Characteristics Name
	CHARAR(2)	Harmonics Current Source Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.51. HAR_TRN_PURG

Use this API to purge specified harmonics transformer data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_TRN_PURG IBUS JBUS KBUS CKT
```

Python command syntax:

```
ierr = har_trn_purg(ibus,jbus,kbus,ckt)
```

Fortran command syntax:

```
CALL HAR_TRN_PURG(IBUS, JBUS, KBUS, CKT, IERR)
```

Where:

Integer IBUS	bus number of from bus (input; no default allowed).	
Integer JBUS	bus number of to bus (input; no default allowed).	
Integer KBUS	bus number of third bus (input; 0 by default).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.52. HAR_VLTSRC

Use this API to modify or add new harmonics voltage source table data to the working case. This API is used to add new table of angle type data, harmonic numbers and corresponding V/V0 and Angle data points. This can be used to add a new data point of harmonic number, V/V0 and Angle data to existing table. This can be used to change angle type data, V/V0 and Angle data points of existing harmonic number in the table. This API cannot be used to change table name and harmonic numbers.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC NAME NPTS INTGAR(1) HAR(1)..HAR(NPTS) VAR(1)..VAR(NPTS)
 ANAR(1)..ANAR(NPTS)
```

Python command syntax:

```
ierr = har_vltsrc(name, npts, intgar, har, var, anar)
```

Fortran command syntax:

```
CALL HAR_VLTSRC(NAME, NPTS, INTGAR, HAR, VAR, ANAR, IERR)
```

Where:

Character*40 NAME	Harmonics Voltage Source Data table name (input; no default allowed).	
Integer NPTS	number of data points specified in the table (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	ANTYP, Harmonics Voltage Source angle type (1 by default)
Real HAR (NPTS)	array of 'NPTS' values specifying harmonic numbers. All the harmonic numbers in HAR array must be non zero. (input).	
Real VAR (NPTS)	array of 'NPTS' values specifying R/R0 ratio (input).	
Real ANAR (NPTS)	array of 'NPTS' values specifying L/L0 ratio (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.53. HAR_VLTSRC_CHNG

Use this API to change harmonics voltage source table data in the working case. This API is used to change angle type data, V/V0 and Angle data points of existing harmonic number and existing table in working case harmonics data. This API cannot be used to change table name and harmonic numbers. All the harmonic numbers in HAR array must be non zero.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC_CHNG NAME NPTS INTGAR(1) HAR(1)..HAR(NPTS)
 VAR(1)..VAR(NPTS) ANAR(1)..ANAR(NPTS)
```

Python command syntax:

```
ierr = har_vltsrc_chng(name, npts, intgar, har, var, anar)
```

Fortran command syntax:

```
CALL HAR_VLTSRC_CHNG(NAME, NPTS, INTGAR, HAR, VAR, ANAR, IERR)
```

Where:

Character*40 NAME	Harmonics Voltage Source Data table name (input; no default allowed).	
Integer NPTS	number of data points specified in the table (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	ANTYP, Harmonics Voltage Source angle type (1 by default)
Real HAR (NPTS)	array of 'NPTS' values specifying harmonic numbers. All the harmonic numbers in HAR array must be non zero. (input).	
Real VAR (NPTS)	array of 'NPTS' values specifying magnitude (input).	
Real ANAR (NPTS)	array of 'NPTS' values specifying angle (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.54. HAR_VLTSRC_CHNG_NAME

Use this API to change harmonics voltage source table name in the working case. This API is used to change table name from working case harmonics data. NAMNEW must not exist in working case harmonics data. NAME must be provided and it must exist in working case harmonics data, then table name changed from NAME to NAMNEW.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC_CHNG_NAME  'NAME'  'NAMNEW' INTGAR
```

Python command syntax:

```
ierr = har_vltsrc_chng_name(name, namnew, intgar)
```

Fortran command syntax:

```
CALL HAR_VLTSRC_CHNG_NAME(NAME, NAMNEW, INTGAR, IERR)
```

Where:

Character*40 NAME	Harmonics Voltage Source table name whose name is to be changed. NAME must be provided and it must exist in working case harmonics data. (input; no default allowed).	
Character*40 NAMNEW	New Harmonics Voltage Source table name to be assigned. NAMNEW must not exist in working case harmonics data. (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	ANTYP, Harmonics Voltage Source angle type (1 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.55. HAR_VLTSRC_CHNG_ONEPT

Use this API to modify one data point to harmonics voltage source table in the working case. This API is used to change H, V/V0, and Angle data of existing harmonic number (HPTOLD) in the existing working case harmonics data table. This API cannot be used to change table name.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC_CHNG_ONEPT NAME HPTOLD HPT VPT ANPT
```

Python command syntax:

```
ierr = har_vltsrc_chng_onept(name, hptold, hpt, vpt, anpt)
```

Fortran command syntax:

```
CALL HAR_VLTSRC_CHNG_ONEPT(NAME, HPTOLD, HPT, VPT, ANPT, IERR)
```

Where:

Character*40 NAME	Harmonics Voltage Source Data table name (input; no default allowed).	
Real HPTOLD	Harmonics number at point in data table where provided data is changed (input; no default allowed).	
Real HPT	New Harmonics number to be changed (input; no default allowed).	
Real VPT	Voltage Source magnitude for harmonics number HPT to be changed (input; 1 by default).	
Real ANPT	Voltage Source angle for harmonics number HPT to be changed (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.56. HAR_VLTSRC_NAME

Use this API to add or change harmonics voltage source table name in the working case. This API is used to add new table name with no harmonics data points or change table name from working case harmonics data.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC_CHNG_NAME  'NAME'  'NAMNEW' INTGAR
```

Python command syntax:

```
ierr = har_vltsrc_name(name, namnew, intgar)
```

Fortran command syntax:

```
CALL HAR_VLTSRC_NAME(NAME, NAMNEW, INTGAR, IERR)
```

Where:

Character*40 NAME	Harmonics Voltage Source table name whose name is to be changed. When NAME is provided, it must exist in working case harmonics data, then table name changed from NAME to NAMNEW. If NAME is blank, a new table NAMNEW will be added with no harmonics data points. (input; " by default).	
Character*40 NAMNEW	New Harmonics Voltage Source table name to be assigned. NAMNEW must not exist in working case harmonics data. (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	ANTYP, Harmonics Voltage Source angle type (1 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.57. HAR_VLTSRC_ONEPT

Use this API to modify or add new one data point to harmonics voltage source table in the working case. This API is used to add one new data point of H, V/V0, and Angle data to existing table of existing working case harmonics data. This can be used to change H, V/V0, and Angle data points of existing harmonic number in the table. This API cannot be used to change table name.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC_ONEPT NAME HPTOLD HPT VPT ANPT
```

Python command syntax:

```
ierr = har_vltsrc_onept(name, hptold, hpt, vpt, anpt)
```

Fortran command syntax:

```
CALL HAR_VLTSRC_ONEPT(NAME, HPTOLD, HPT, VPT, ANPT, IERR)
```

Where:

Character*40 NAME	Harmonics Voltage Source Data table name (input; no default allowed).	
Real HPTOLD	Harmonics number at point in data table where provided data is added or changed. When HPTOLD is specified and found, the H, V/V0, and Angle values specified will be used to change existing data. When HPTOLD is zero, the H, V/V0, and Angle values specified will be used to add data point to existing data. (input; no default allowed).	
Real HPT	New Harmonics number to be added or changed at specified data point (input; no default allowed).	
Real VPT	Voltage Source magnitude for harmonics number HPT to be added or changed (input; 1 by default).	
Real ANPT	Voltage Source angle for harmonics number HPT to be added or changed (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.58. HAR_VLTSRC_PURG

Use this API to purge specified harmonics voltage source table data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC_PURG NAME
```

Python command syntax:

```
ierr = har_vltsrc_purg(name)
```

Fortran command syntax:

```
CALL HAR_VLTSRC_PURG(NAME, IERR)
```

Where:

Character*40 NAME	Harmonics voltage source table name (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.59. HAR_VLTSRC_PURG_ONEPT

Use this API to purge specified data point in harmonics voltage source table.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VLTSRC_PURG_ONEPT NAME HPT
```

Python command syntax:

```
ierr = har_vltsrc_purg_onept(name, hpt)
```

Fortran command syntax:

```
CALL HAR_VLTSRC_PURG_ONEPT(NAME, HPT, IERR)
```

Where:

Character*40 NAME	Harmonics voltage source table name (input; no default allowed).	
Real HPT	Harmonic number (H) at a point in data table where H, magnitude, L/) and C/C0 data is purged (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.60. HAR_VSCDC

Use this API to modify or add new harmonics VSCDC data to the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VSCDC NAME IBUS INTGAR(1) CHARAR(1) CHARAR(2)
```

Python command syntax:

```
ierr = har_vscdc(name,ibus,intgar,charar)
```

Fortran command syntax:

```
CALL HAR_VSCDC(NAME,IBUS,INTGAR,CHARAR,IERR)
```

Where:

Character*12 NAME	VSC DC line name (input; no default allowed).	
Integer IBUS	converter bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Voltage Source Table Name
	CHARAR(2)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.61. HAR_VSCDC_CHNG

Use this API to modify harmonics VSCDC data in the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VSCDC_CHNG NAME IBUS INTGAR(1) CHARAR(1) CHARAR(2)
```

Python command syntax:

```
ierr = har_vscdc_chng(name,ibus,intgar,charar)
```

Fortran command syntax:

```
CALL HAR_VSCDC_CHNG(NAME,IBUS,INTGAR,CHARAR,IERR)
```

Where:

Character*12 NAME	VSC DC line name (input; no default allowed).	
Integer IBUS	converter bus number (input; no default allowed).	
Integer INTGAR (1)	array of 1 elements specifying (input).	
	INTGAR(1)	Harmonics State (1 by default)
Character*40 CHARAR (2)	array of 2 elements specifying (input).	
	CHARAR(1)	Harmonics Voltage Source Table Name
	CHARAR(2)	Harmonics Current Source Table Name
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

5.2.62. HAR_VSCDC_PURG

Use this API to purge specified harmonics VSCDC data from the working case.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_HAR_VSCDC_PURG NAME IBUS
```

Python command syntax:

```
ierr = har_vscdc_purg(name, ibus)
```

Fortran command syntax:

```
CALL HAR_VSCDC_PURG(NAME, IBUS, IERR)
```

Where:

Character*12 NAME	VSC DC line name (input; no default allowed).	
Integer IBUS	converter bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Harmonics Data Changing APIs contains API routines specific to adding, modifying, and deleting Harmonics data.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 6
GIC

This chapter describes the API routines used in PSS®E GIC.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1. GIC Operation

6.1.1. GIC

This API routine is obsolete. It has been replaced by the API routine gic_2 .

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 32.2.0 and 33.3.0.

Batch command syntax:

```
BAT_GIC  SID ALL APIOPTNS(1)..APIOPTNS(4) VALUES(1)..VALUES(5)
 RPTOPTNS(1)..RPTOPTNS(7) GICDATAFIL GICADDFIL GICPURGFIL GICRNWKFIL
```

Python command syntax:

```
ierr = gic(sid, all, apioptns, values, rptoptns, gicdatafil, gicaddfil,
 gicpurgfil, gicrnwkfil)
```

Fortran command syntax:

```
CALL GIC(SID, ALL, APIOPTNS, VALUES, RPTOPTNS, GICDATAFIL, GICADDFIL,
 GICPURGFIL, GICRNWKFIL, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer APIOPTNS (4)	array of 4 elements specifying Integer Options(input).	
	APIOPTNS(1)	Electric Field Type (alias is efield_type) (0 by default)
	APIOPTNS(1) = 0	for uniform geoelectric field
	APIOPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (0 by default)
	APIOPTNS(2) = 0	for volts/km
	APIOPTNS(2) = 1	for volts/mile
	APIOPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (1 by default)
	APIOPTNS(3) = 0	add updates to Saved Case
	APIOPTNS(3) = 1	create RDCH raw file of GIC updates
	APIOPTNS(4)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	APIOPTNS(4) = 0	consider only study subsystem buses, no buses from inter-ties
	APIOPTNS(4) > 0	consider only study subsystem buses, no buses from inter-ties
Real VALUES (5)	array of 5 elements specifying Real Options(input).	
	VALUES(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag) (0 by default)
	VALUES(1) > 0	when charoptns(1)=uniform, no default allowed
	VALUES(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	VALUES(1) > 0	for charoptns(1)=benchmark, used as specified
	VALUES(2)	electric field direction in degrees, range 0 to 360 degrees (alias is efield_deg) (0 by default)
	VALUES(2) > 0	when charoptns(1)=uniform or benchmark, not used when charoptns(1)=nonuniform
	VALUES(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
	VALUES(3) = 0	Substation is ungrounded
	VALUES(3) > 0	Use specified grounding dc resistance
	VALUES(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in network data (alias is branch_xbyr) (30 by default)
	VALUES(4) = 0	Use default
	VALUES(4) > 0	Use specified
	VALUES(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
	VALUES(5) = 0	Use default
	VALUES(5) > 0	Use specified
Integer RPTOPTNS (7)	array of 7 elements specifying Report Options(input).	
	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RPTOPTNS(2) = 0	No
	RPTOPTNS(2) = 1	Yes
	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)
	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes
	RPTOPTNS(5)	report transformer GIC flows (alias is rpttrn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_gic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
Character*260 GICDATAFIL	GIC data file name (input). (input; no default allowed).	
Character*260 GICADDFIL	GIC updates to Base Case file name (output). (input; blank by default).	
Character*260 GICPURGFIL	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (input; blank by default).	
Character*260 GICRNWKFIL	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow (output). (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.2. GIC_2

This API routine is obsolete. It has been replaced by the API routine gic_3 .

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_GIC_2  SID ALL INTGOPTNS(1)..INTGOPTNS(4) REALOPTNS(1)..REALOPTNS(7)
 CHAROPTNS(1)..CHAROPTNS(7) EJETOPTNS(1)..EJETOPTNS(5)
 FILEOPTNS(1)..FILEOPTNS(4) RPTOPTNS(1)..RPTOPTNS(7)
```

Python command syntax:

```
ierr = gic_2(sid, all, intgoptns, realoptns, charoptns, ejetoptns,
 fileoptns, rptoptns)
```

Fortran command syntax:

```
CALL GIC_2(SID, ALL, INTGOPTNS, REALOPTNS, CHAROPTNS, EJETOPTNS,
 FILEOPTNS, RPTOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (4)	array of 4 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(2)	Year number to scale benchmark event GMD storm. These scaling factors account in the influence of geomagnetic latitude and earth model on the estimated geoelectric field magnitude and are provided in NERC TPL-007-1. (alias is study_year) (0 by default)
	INTGOPTNS(2) = 0	current year, used in benchmark geoelectric field calculations only.
	INTGOPTNS(2) > 0	specific year

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGOPTNS(3)	Subsystem sid for local GMD hot spots (alias is sid_local) (0 by default)
	INTGOPTNS(3) = 0	No local GMD hot spots
	INTGOPTNS(3) > 0	subsystem number (1 - 11). Buses and branches in this subsystem will have Efield due to local GMD hot spots.
	INTGOPTNS(4)	Option for Transformer Thermal Analysis (not implemented yet) (alias is thermal_ana_optn) (0 by default)
	INTGOPTNS(4) = -1	perform on all transformers
	INTGOPTNS(4) = 0	do not perform
	INTGOPTNS(4) > 0	perform on these many top transformers ordered by effective GIC flow
Real REALOPTNS (7)	array of 7 elements specifying Real Options(input).	
	REALOPTNS(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag) (0 by default)
	REALOPTNS(1) > 0	when charoptns(1)=uniform, no default allowed
	REALOPTNS(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	REALOPTNS(1) > 0	for charoptns(1)=benchmark, used as specified
	REALOPTNS(2)	electric field direction in degrees, range 0 to 360 degrees (alias is efield_deg) (0 by default)
	REALOPTNS(2) > 0	when charoptns(1)=uniform or benchmark, not used when charoptns(1)=nonuniform
	REALOPTNS(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
	REALOPTNS(3) = 0	Substation is ungrounded
	REALOPTNS(3) > 0	Use specified grounding dc resistance
	REALOPTNS(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in network data (alias is branch_xbyr) (30 by default)
	REALOPTNS(4) = 0	Use default
	REALOPTNS(4) > 0	Use specified
	REALOPTNS(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
	REALOPTNS(5) = 0	Use default
	REALOPTNS(5) > 0	Use specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALOPTNS(6)	local GMD hot spots electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag_local) (realoptns(1) by default)
	REALOPTNS(6) > 0	when charoptns(1)=uniform or benchmark, not used when charoptns(1)=nonuniform
	REALOPTNS(7)	local GMD hot spots electric field direction in degrees, range 0 to 360 degrees (alias is efield_deg_local) (realoptns(2) by default)
	REALOPTNS(7) > 0	when charoptns(1)=uniform or benchmark, not used when charoptns(1)=nonuniform
Character*12 CHAROPTNS (7)	array of 7 elements specifying Character Options(input).	
	CHAROPTNS(1)	Electric Field Type (alias is efield_type) (uniform by default)
	CHAROPTNS(1) = uniform	for uniform geoelectric field
	CHAROPTNS(1) = benchmark	for benchmark event geoelectric field
	CHAROPTNS(1) = nonuniform	for nonuniform geoelectric field
	CHAROPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (v/km by default)
	CHAROPTNS(2) = v/km	for volts/km
	CHAROPTNS(2) = v/mi	for volts/mile
	CHAROPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (rdch by default)
	CHAROPTNS(3) = sav	add updates to Saved Case
	CHAROPTNS(3) = rdch	create RDCH raw file of GIC updates
	CHAROPTNS(4)	Option to select method for GIC to Mvar Calculation (alias is gic2mvar_optn) (kfactors by default)
	CHAROPTNS(4) = kfactors	for GIC to Mvar scaling factors from GIC data file when provided, otherwise default scaling factors
	CHAROPTNS(4) = capchar	for GIC to Mvar scaling factors from GIC data file when provided, otherwise scaling factors from default transformer Mvar and GIC characteristics
	CHAROPTNS(5)	Earth Model Name. A Standard or User defined model name must be provided when Benchmark Event or Non-uniform electric field is to be modeled or Transformer Thermal Analysis is to be performed. (alias is earth_model_name) (by default)
	CHAROPTNS(6)	Option to scan storm event scenarios (alias is scan_storm_event) (by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CHAROPTNS(6) =	no storm orientation and/or magnitude scan
	CHAROPTNS(6) = scan_deg	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for only orientation that gives maximum VAR losses
	CHAROPTNS(6) = scan_deg_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for each orientation scanned
	CHAROPTNS(6) = scan_mag	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for only final electric field magnitude.
	CHAROPTNS(6) = scan_mag_rpt	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for each scanned electric field magnitude.
	CHAROPTNS(6) = scan_d_m	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans tiil they yield converged power flow solution, create report for only orientation that gives maximum VAR losses and final electric field magnitude
	CHAROPTNS(6) = scan_d_m_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans till they yield converged power flow solution, create report for all orientation scans and mangitude scans performed
	CHAROPTNS(7)	Option to solve Power Flow with GIC losses added to the base case (alias is power_flow_optn) (by default)
	CHAROPTNS(7) =	do not perform power flow solution
	CHAROPTNS(7) = fdns	use fixed slope decoupled Newton-Raphson method
	CHAROPTNS(7) = fnsl	use Full Newton-Raphson method
	CHAROPTNS(7) = nsol	use Decoupled Newton-Raphson method
Real EJETOPTNS (5)	array of 5 elements specifying Electrojet Characteristics, used only if charoptns(1)=nonuniform(input).	
	EJETOPTNS(1)	eletrojet current in million amperes, must be >0 (alias is ejet_million_amps) (no default allowed)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	EJETOPTNS(2)	Cauchy distribution half-width in km, must be >0 (alias is ejet_halfwidth_km) (no default allowed)
	EJETOPTNS(3)	period of variation in minutes, must be >0 (alias is ejet_period_min) (no default allowed)
	EJETOPTNS(4)	height of current in km, must be >0 (alias is ejet_height_km) (no default allowed)
	EJETOPTNS(5)	latitude of center of electrojet in degrees (alias is ejet_center_deg) (no default allowed)
Character*260 FILEOPTNS (4)	array of 4 elements specifying File Used(input).	
	FILEOPTNS(1)	GIC data file name (input). (alias is gicfile) (no default allowed)
	FILEOPTNS(2)	GIC updates to Base Case file name (output). (alias is addfile) (by default)
	FILEOPTNS(2) =	no file created
	FILEOPTNS(3)	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (alias is purgfile) (by default)
	FILEOPTNS(3) =	no file created
	FILEOPTNS(4)	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow. (output) (alias is rnwkfile) (by default)
	FILEOPTNS(4) =	no file created
Integer RPTOPTNS (7)	array of 7 elements specifying Report Options(input).	
	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)
	RPTOPTNS(2) = 0	No
	RPTOPTNS(2) = 1	Yes
	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)
	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RPTOPTNS(5)	report transformer GIC flows (alias is rpt-trn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_g-ic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
Integer IERR	error code (output).	
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.3. GIC_3

This API routine is obsolete. It has been replaced by the API routine gic_4 .

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_GIC_3  SID ALL INTGOPTNS(1)..INTGOPTNS(4) REALOPTNS(1)..REALOPTNS(7)
 CHAROPTNS(1)..CHAROPTNS(7) EJETOPTNS(1)..EJETOPTNS(5)
 FILEOPTNS(1)..FILEOPTNS(5) RPTOPTNS(1)..RPTOPTNS(7)
```

Python command syntax:

```
ierr = gic_3(sid, all, intgoptns, realoptns, charoptns, ejetoptns,
 fileoptns, rptoptns)
```

Fortran command syntax:

```
CALL GIC_3(SID, ALL, INTGOPTNS, REALOPTNS, CHAROPTNS, EJETOPTNS,
 FILEOPTNS, RPTOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (4)	array of 4 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(2)	Year number to scale benchmark event GMD storm. These scaling factors account in the influence of geomagnetic latitude and earth model on the estimated geoelectric field magnitude and are provided in NERC TPL-007-1. (alias is study_year) (0 by default)
	INTGOPTNS(2) = 0	current year, used in benchmark geoelectric field calculations only.
	INTGOPTNS(2) > 0	specific year

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGOPTNS(3)	Subsystem sid for local GMD hot spots (alias is sid_local) (0 by default)
	INTGOPTNS(3) = 0	No local GMD hot spots
	INTGOPTNS(3) > 0	subsystem number (1 - 11). Buses and branches in this subsystem will have Efield due to local GMD hot spots.
	INTGOPTNS(4)	Option for Transformer Thermal Analysis (not implemented yet) (alias is thermal_ana_optn) (0 by default)
	INTGOPTNS(4) = -1	perform on all transformers
	INTGOPTNS(4) = 0	do not perform
	INTGOPTNS(4) > 0	perform on these many top transformers ordered by effective GIC flow
Real REALOPTNS (7)	array of 7 elements specifying Real Options(input).	
	REALOPTNS(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag) (0 by default)
	REALOPTNS(1) > 0	when charoptns(1)=uniform, no default allowed
	REALOPTNS(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	REALOPTNS(1) > 0	for charoptns(1)=benchmark, used as specified
	REALOPTNS(2)	electric field direction in degrees, range 0 to 360 degrees (alias is efield_deg) (0 by default)
	REALOPTNS(2) > 0	when charoptns(1)=uniform or benchmark, not used when charoptns(1)=nonuniform
	REALOPTNS(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
	REALOPTNS(3) = 0	Substation is ungrounded
	REALOPTNS(3) > 0	Use specified grounding dc resistance
	REALOPTNS(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in network data (alias is branch_xbyr) (30 by default)
	REALOPTNS(4) = 0	Use default
	REALOPTNS(4) > 0	Use specified
	REALOPTNS(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
	REALOPTNS(5) = 0	Use default
	REALOPTNS(5) > 0	Use specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALOPTNS(6)	local GMD hot spots electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag_local) (realoptns(1) by default)
	REALOPTNS(6) > 0	when charoptns(1)=uniform or benchmark, not used when charoptns(1)=nonuniform
	REALOPTNS(7)	local GMD hot spots electric field direction in degrees, range 0 to 360 degrees (alias is efield_deg_local) (realoptns(2) by default)
	REALOPTNS(7) > 0	when charoptns(1)=uniform or benchmark, not used when charoptns(1)=nonuniform
Character*12 CHAROPTNS (7)	array of 7 elements specifying Character Options(input).	
	CHAROPTNS(1)	Electric Field Type (alias is efield_type) (uniform by default)
	CHAROPTNS(1) = uniform	for uniform geoelectric field
	CHAROPTNS(1) = benchmark	for benchmark event geoelectric field
	CHAROPTNS(1) = nonuniform	for nonuniform geoelectric field
	CHAROPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (v/km by default)
	CHAROPTNS(2) = v/km	for volts/km
	CHAROPTNS(2) = v/mi	for volts/mile
	CHAROPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (rdch by default)
	CHAROPTNS(3) = sav	add updates to Saved Case
	CHAROPTNS(3) = rdch	create RDCH raw file of GIC updates
	CHAROPTNS(4)	Option to select method for GIC to Mvar Calculation (alias is gic2mvar_optn) (kfactors by default)
	CHAROPTNS(4) = kfactors	for GIC to Mvar scaling factors from GIC data file when provided, otherwise default scaling factors
	CHAROPTNS(4) = capchar	for GIC to Mvar scaling factors from GIC data file when provided, otherwise scaling factors from default transformer Mvar and GIC characteristics
	CHAROPTNS(5)	Earth Model Name. A Standard or User defined model name must be provided when Benchmark Event or Non-uniform electric field is to be modeled or Transformer Thermal Analysis is to be performed. (alias is earth_model_name) (by default)
	CHAROPTNS(6)	Option to scan storm event scenarios (alias is scan_storm_event) (by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CHAROPTNS(6) =	no storm orientation and/or magnitude scan
	CHAROPTNS(6) = scan_deg	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for only orientation that gives maximum VAR losses
	CHAROPTNS(6) = scan_deg_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for each orientation scanned
	CHAROPTNS(6) = scan_mag	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for only final electric field magnitude.
	CHAROPTNS(6) = scan_mag_rpt	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for each scanned electric field magnitude.
	CHAROPTNS(6) = scan_d_m	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans tiil they yield converged power flow solution, create report for only orientation that gives maximum VAR losses and final electric field magnitude
	CHAROPTNS(6) = scan_d_m_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans till they yield converged power flow solution, create report for all orientation scans and mangitude scans performed
	CHAROPTNS(7)	Option to solve Power Flow with GIC losses added to the base case (alias is power_flow_optn) (by default)
	CHAROPTNS(7) =	do not perform power flow solution
	CHAROPTNS(7) = fdns	use fixed slope decoupled Newton-Raphson method
	CHAROPTNS(7) = fnsl	use Full Newton-Raphson method
	CHAROPTNS(7) = nsol	use Decoupled Newton-Raphson method
Real EJETOPTNS (5)	array of 5 elements specifying Electrojet Characteristics, used only if charoptns(1)=nonuniform(input).	
	EJETOPTNS(1)	eletrojet current in million amperes, must be >0 (alias is ejet_million_amps) (no default allowed)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	EJETOPTNS(2)	Cauchy distribution half-width in km, must be >0 (alias is ejet_halfwidth_km) (no default allowed)
	EJETOPTNS(3)	period of variation in minutes, must be >0 (alias is ejet_period_min) (no default allowed)
	EJETOPTNS(4)	height of current in km, must be >0 (alias is ejet_height_km) (no default allowed)
	EJETOPTNS(5)	latitude of center of electrojet in degrees (alias is ejet_center_deg) (no default allowed)
Character*260 FILEOPTNS (5)	array of 5 elements specifying File Used(input).	
	FILEOPTNS(1)	GIC data file name (input). (alias is gicfile) (no default allowed)
	FILEOPTNS(2)	GIC updates to Base Case file name (output). (alias is addfile) (by default)
	FILEOPTNS(2) =	no file created
	FILEOPTNS(3)	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (alias is purgfile) (by default)
	FILEOPTNS(3) =	no file created
	FILEOPTNS(4)	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow. (output) (alias is rnwkfile) (by default)
	FILEOPTNS(4) =	no file created
	FILEOPTNS(5)	GIC Results map data file for given Efield magnitude and degrees OR Efield magnitude and degrees scans which give maximum Var losses when scans are performed (output). This is used by GICMAPS to plot GIC results on network map. (alias is pygicfile) (by default)
	FILEOPTNS(5) =	temporary file created
	FILEOPTNS(5) = nooutput	no file created
Integer RPTOPTNS (7)	array of 7 elements specifying Report Options(input).	
	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)
	RPTOPTNS(2) = 0	No

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RPTOPTNS(2) = 1	Yes
	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)
	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes
	RPTOPTNS(5)	report transformer GIC flows (alias is rpt-trn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_g-ic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
Integer IERR	error code (output).	
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.4. GIC_4

This API routine is obsolete. It has been replaced by the API routine gic_5 .

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_GIC_4  SID ALL INTGOPTNS(1)..INTGOPTNS(4) REALOPTNS(1)..REALOPTNS(9)
 CHAROPTNS(1)..CHAROPTNS(7) EJETOPTNS(1)..EJETOPTNS(5)
 FILEOPTNS(1)..FILEOPTNS(6) RPTOPTNS(1)..RPTOPTNS(7)
```

Python command syntax:

```
ierr = gic_4(sid, all, intgoptns, realoptns, charoptns, ejetoptns,
 fileoptns, rptoptns)
```

Fortran command syntax:

```
CALL GIC_4(SID, ALL, INTGOPTNS, REALOPTNS, CHAROPTNS, EJETOPTNS,
 FILEOPTNS, RPTOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (4)	array of 4 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(2)	Year number to scale benchmark event GMD storm. These scaling factors account in the influence of geomagnetic latitude on the estimated geoelectric field magnitude and are provided in NERC TPL-007. (alias is study_year) (0 by default)
	INTGOPTNS(2) = 0	current year, used in benchmark and supplemental geoelectric field calculations only.
	INTGOPTNS(2) > 0	specific year

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGOPTNS(3)	Subsystem sid for supplemental GMD event (alias is sid_supp) (0 by default)
	INTGOPTNS(3) = 0	No supplemental GMD event
	INTGOPTNS(3) > 0	subsystem number (1 - 11). Buses and branches in this subsystem will have Efield due to supplemental GMD event.
	INTGOPTNS(4)	Option for Transformer Thermal Analysis (alias is thermal_ana_optn) (0 by default)
	INTGOPTNS(4) = -1	perform on all transformers
	INTGOPTNS(4) = 0	do not perform
	INTGOPTNS(4) > 0	perform on these many top transformers ordered by effective GIC flow
Real REALOPTNS (9)	array of 9 elements specifying Real Options(input).	
	REALOPTNS(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform or supplemental (alias is efield_mag) (0 by default)
	REALOPTNS(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	REALOPTNS(1) > 0	when charoptns(1)=uniform, no default allowed
	REALOPTNS(1) > 0	when charoptns(1)=benchmark, used as specified
	REALOPTNS(2)	electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform or supplemental (alias is efield_deg) (0 by default)
	REALOPTNS(2) > 0	when charoptns(1)=uniform or benchmark
	REALOPTNS(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
	REALOPTNS(3) = 0	Substation is ungrounded
	REALOPTNS(3) > 0	Use specified grounding dc resistance
	REALOPTNS(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in network data (alias is branch_xbyr) (30 by default)
	REALOPTNS(4) = 0	Use default
	REALOPTNS(4) > 0	Use specified
	REALOPTNS(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
	REALOPTNS(5) = 0	Use default
	REALOPTNS(5) > 0	Use specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALOPTNS(6)	supplemental event electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag_supp) (realoptns(1) by default)
	REALOPTNS(6) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is not zero
	REALOPTNS(6) > 0	when charoptns(1)=supplemental
	REALOPTNS(7)	local GMD hot spots electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform (alias is efield_deg_supp) (realoptns(2) by default)
	REALOPTNS(7) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is non-zero
	REALOPTNS(7) > 0	when charoptns(1)=supplemental
	REALOPTNS(8)	transmission line AC to DC resistance conversion factor, must be >0 (alias is branch_rac2rdc) (1.0 by default)
	REALOPTNS(8) = 0	Use default
	REALOPTNS(8) > 0	Use specified
	REALOPTNS(9)	transformer winding AC to DC resistance conversion factor, must be >0 (alias is transformer_rac2rdc) (1.0 by default)
	REALOPTNS(9) = 0	Use default
	REALOPTNS(9) > 0	Use specified
Character*12 CHAROPTNS (7)	array of 7 elements specifying Character Options(input).	
	CHAROPTNS(1)	Electric Field Type (alias is efield_type) (uniform by default)
	CHAROPTNS(1) = uniform	for uniform geoelectric field and also supplemental event electric field when intgoptns(3) and realoptns(6) are non-zero
	CHAROPTNS(1) = benchmark	for benchmark event geoelectric field and beta factors (B) and also supplemental event electric field and beta factors (Bs) when intgoptns(3) and realoptns(6) are non-zero
	CHAROPTNS(1) = nonuniform	for nonuniform geoelectric field
	CHAROPTNS(1) = supplemental	for supplemental event geoelectric field and beta factors (Bs)
	CHAROPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (v/km by default)
	CHAROPTNS(2) = v/km	for volts/km
	CHAROPTNS(2) = v/mi	for volts/mile
	CHAROPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (rdch by default)
	CHAROPTNS(3) = sav	add updates to Saved Case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHAROPTNS(3) = rdch	create RDCH raw file of GIC updates
CHAROPTNS(4)	Option to select method for GIC to Mvar Calculation (alias is gic2mvar_optn) (kfactors by default)
CHAROPTNS(4) = kfactors	for GIC to Mvar scaling factors from GIC data file when provided, otherwise default scaling factors
CHAROPTNS(4) = capchar	for GIC to Mvar scaling factors from GIC data file when provided, otherwise scaling factors from default transformer Mvar and GIC characteristics
CHAROPTNS(5)	Earth Model Name. A Standard or User defined model name must be provided when Benchmark Event or Non-uniform electric field is to be modeled or Transformer Thermal Analysis is to be performed. (alias is earth_model_name) (by default)
CHAROPTNS(6)	Option to scan storm event scenarios (alias is scan_storm_event) (by default)
CHAROPTNS(6) =	no storm orientation and/or magnitude scan
CHAROPTNS(6) = scan_deg	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for only orientation that gives maximum VAR losses
CHAROPTNS(6) = scan_deg_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for each orientation scanned
CHAROPTNS(6) = scan_mag	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for only final electric field magnitude.
CHAROPTNS(6) = scan_mag_rpt	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for each scanned electric field magnitude.
CHAROPTNS(6) = scan_d_m	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans tiil they yield converged power flow solution, create report for only orientation that gives maximum VAR losses and final electric field magnitude
CHAROPTNS(6) = scan_d_m_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		uniform or benchmark, then perform magnitude scans till they yield converged power flow solution, create report for all orientation scans and mangitude scans performed
	CHAROPTNS(7)	Option to solve Power Flow with GIC losses added to the base case (alias is power_flow_optn) (by default)
	CHAROPTNS(7) =	do not perform power flow solution
	CHAROPTNS(7) = fdns	use fixed slope decoupled Newton-Raphson method
	CHAROPTNS(7) = fnsl	use Full Newton-Raphson method
	CHAROPTNS(7) = nsol	use Decoupled Newton-Raphson method
Real EJETOPTNS (5)	array of 5 elements specifying Electrojet Characteristics, used only if charoptns(1)=nonuniform(input).	
	EJETOPTNS(1)	eletrojet current in million amperes, must be >0 (alias is ejet_million_amps) (no default allowed)
	EJETOPTNS(2)	Cauchy distribution half-width in km, must be >0 (alias is ejet_halfwidth_km) (no default allowed)
	EJETOPTNS(3)	period of variation in minutes, must be >0 (alias is ejet_period_min) (no default allowed)
	EJETOPTNS(4)	height of current in km, must be >0 (alias is ejet_height_km) (no default allowed)
	EJETOPTNS(5)	latitude of center of electrojet in degrees (alias is ejet_center_deg) (no default allowed)
Character*260 FILEOPTNS (6)	array of 6 elements specifying File Used(input).	
	FILEOPTNS(1)	GIC data file name (input). (alias is gicfile) (no default allowed)
	FILEOPTNS(2)	GIC updates to Base Case file name (output). (alias is addfile) (blank by default)
	FILEOPTNS(2) = ' '	no file created
	FILEOPTNS(3)	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (alias is purgfile) (blank by default)
	FILEOPTNS(3) = ' '	no file created
	FILEOPTNS(4)	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow (output). (alias is rnwkfile) (blank by default)
	FILEOPTNS(4) = ' '	no file created
	FILEOPTNS(5)	GIC Results map data file for given Efield magnitude and degrees OR Efield magnitude and degrees scans which give maximum Var

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		losses when scans are performed (output). This is used by GICMAPS to plot GIC results on network map. (alias is pygicfile) (nooutput by default)
	FILEOPTNS(5) = ' '	temporary file created
	FILEOPTNS(5) = nooutput	no file created
	FILEOPTNS(6)	Transformer Thermal Analysis GIC(t) CSV file (output). (alias is gictfile) (nooutput by default)
	FILEOPTNS(6) = ' '	temporary file created
	FILEOPTNS(6) = nooutput	no file created
Integer RPTOPTNS (7)	array of 7 elements specifying Report Options(input).	
	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)
	RPTOPTNS(2) = 0	No
	RPTOPTNS(2) = 1	Yes
	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)
	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes
	RPTOPTNS(5)	report transformer GIC flows (alias is rpttrn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_gic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.5. GIC_5

This API routine is obsolete. It has been replaced by the API routine gic_6 .

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 34.5.0.

Batch command syntax:

```
BAT_GIC_5  SID ALL INTGOPTNS(1)..INTGOPTNS(5) REALOPTNS(1)..REALOPTNS(13)
 CHAROPTNS(1)..CHAROPTNS(7) EJETOPTNS(1)..EJETOPTNS(5)
 FILEOPTNS(1)..FILEOPTNS(6) RPTOPTNS(1)..RPTOPTNS(7)
```

Python command syntax:

```
ierr = gic_5(sid, all, intgoptns, realoptns, charoptns, ejetoptns,
 fileoptns, rptoptns)
```

Fortran command syntax:

```
CALL GIC_5(SID, ALL, INTGOPTNS, REALOPTNS, CHAROPTNS, EJETOPTNS,
 FILEOPTNS, RPTOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (5)	array of 5 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	1 through 5, consider study subsystem buses + these many levels of inter-tie buses and ignore subsystem definition voltage limits for intertie buses added
	INTGOPTNS(1) < 0	-1 through -5, consider study subsystem buses + these many levels of inter-tie buses and consider subsystem definition voltage limits for intertie buses added
	INTGOPTNS(2)	Year number to scale benchmark event GMD storm. These scaling factors account in the influence of geomagnetic latitude on the estimated geoelectric field magnitude

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		and are provided in NERC TPL-007. (alias is study_year) (0 by default)
	INTGOPTNS(2) = 0	current year, used in benchmark and supplemental geoelectric field calculations only.
	INTGOPTNS(2) > 0	specific year
	INTGOPTNS(3)	Subsystem sid for supplemental GMD event (alias is sid_supp) (0 by default)
	INTGOPTNS(3) = 0	No supplemental GMD event
	INTGOPTNS(3) > 0	subsystem number (1 - 11). Buses and branches in this subsystem will have Efield due to supplemental GMD event.
	INTGOPTNS(4)	Option for Transformer Thermal Analysis (alias is thermal_ana_optn) (0 by default)
	INTGOPTNS(4) = -1	perform on all transformers
	INTGOPTNS(4) = 0	do not perform
	INTGOPTNS(4) > 0	perform on these many top transformers ordered by effective GIC flow
	INTGOPTNS(5)	Option to run power flow for each degree scan calculation (alias is degscan_pf_optn) (0 by default)
	INTGOPTNS(5) = 0	Solve power flow only for worst degree
	INTGOPTNS(5) = 1	Solve power flow for each degree scan calculation
Real REALOPTNS (13)	array of 13 elements specifying Real Options(input).	
	REALOPTNS(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform or supplemental (alias is efield_mag) (0 by default)
	REALOPTNS(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	REALOPTNS(1) > 0	when charoptns(1)=uniform, no default allowed
	REALOPTNS(1) > 0	when charoptns(1)=benchmark, used as specified
	REALOPTNS(2)	electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform or supplemental (alias is efield_deg) (0 by default)
	REALOPTNS(2) > 0	when charoptns(1)=uniform or benchmark
	REALOPTNS(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
	REALOPTNS(3) = 0	Substation is ungrounded
	REALOPTNS(3) > 0	Use specified grounding dc resistance
	REALOPTNS(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	network data (alias is branch_xbyr) (30 by default)
REALOPTNS(4) = 0	Use default
REALOPTNS(4) > 0	Use specified
REALOPTNS(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
REALOPTNS(5) = 0	Use default
REALOPTNS(5) > 0	Use specified
REALOPTNS(6)	supplemental event electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag_supp) (realoptns(1) by default)
REALOPTNS(6) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is not zero
REALOPTNS(6) > 0	when charoptns(1)=supplemental
REALOPTNS(7)	local GMD hot spots electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform (alias is efield_deg_supp) (realoptns(2) by default)
REALOPTNS(7) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is non-zero
REALOPTNS(7) > 0	when charoptns(1)=supplemental
REALOPTNS(8)	transmission line AC to DC resistance conversion factor, must be >0 (alias is branch_rac2rdc) (1.0 by default)
REALOPTNS(8) = 0	Use default
REALOPTNS(8) > 0	Use specified
REALOPTNS(9)	transformer winding AC to DC resistance conversion factor, must be >0 (alias is transformer_rac2rdc) (1.0 by default)
REALOPTNS(9) = 0	Use default
REALOPTNS(9) > 0	Use specified
REALOPTNS(10)	Degree Scan step size, range 1.0 to 180 degrees (alias is degscan_step) (10.0 by default)
REALOPTNS(10) = 0	Use default
REALOPTNS(10) > 0	Use specified
REALOPTNS(11)	Magnitude Scan step size, must be >1.0 V/km (alias is magscan_step) (4.0 by default)
REALOPTNS(11) = 0	Use default
REALOPTNS(11) > 0	Use specified
REALOPTNS(12)	Percent GMD Mvar loss step size. Total GMD Mvar losses added incrementally to the base case to obtain power flow solution, must

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		be >1.0 (alias is pf_qpct_step) (100.0 by default)
	REALOPTNS(12) = 0	Use default
	REALOPTNS(12) > 0	Use specified
	REALOPTNS(13)	Magnitude Scan maximum storm strength, must be >1.0 (alias is magscan_max) (20.0 by default)
	REALOPTNS(13) = 0	Use default
	REALOPTNS(13) > 0	Use specified
Character*12 CHAROPTNS (7)	array of 7 elements specifying Character Options(input).	
	CHAROPTNS(1)	Electric Field Type (alias is efield_type) (uniform by default)
	CHAROPTNS(1) = uniform	for uniform geoelectric field and also supplemental event electric field when intgoptns(3) and realoptns(6) are non-zero
	CHAROPTNS(1) = benchmark	for benchmark event geoelectric field and beta factors (B) and also supplemental event electric field and beta factors (Bs) when intgoptns(3) and realoptns(6) are non-zero
	CHAROPTNS(1) = nonuniform	for nonuniform geoelectric field
	CHAROPTNS(1) = supplemental	for supplemental event geoelectric field and beta factors (Bs)
	CHAROPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (v/km by default)
	CHAROPTNS(2) = v/km	for volts/km
	CHAROPTNS(2) = v/mi	for volts/mile
	CHAROPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (rdch by default)
	CHAROPTNS(3) = sav	add updates to Saved Case
	CHAROPTNS(3) = rdch	create RDCH raw file of GIC updates
	CHAROPTNS(4)	Option to select method for GIC to Mvar Calculation (alias is gic2mvar_optn) (kfactors by default)
	CHAROPTNS(4) = kfactors	for GIC to Mvar scaling factors from GIC data file when provided, otherwise default scaling factors
	CHAROPTNS(4) = capchar	for GIC to Mvar scaling factors from GIC data file when provided, otherwise scaling factors from default transformer Mvar and GIC characteristics
	CHAROPTNS(5)	Earth Model Name. A Standard or User defined model name must be provided when Benchmark Event or Non-uniform electric field is to be modeled or Transformer Ther-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	mal Analysis is to be performed. (alias is earth_model_name) (by default)
CHAROPTNS(6)	Option to scan storm event scenarios (alias is scan_storm_event) (by default)
CHAROPTNS(6) =	no storm orientation and/or magnitude scan
CHAROPTNS(6) = scan_deg	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for only orientation that gives maximum VAR losses
CHAROPTNS(6) = scan_deg_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for each orientation scanned
CHAROPTNS(6) = scan_mag	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for only final electric field magnitude.
CHAROPTNS(6) = scan_mag_rpt	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for each scanned electric field magnitude.
CHAROPTNS(6) = scan_d_m	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans tiil they yield converged power flow solution, create report for only orientation that gives maximum VAR losses and final electric field magnitude
CHAROPTNS(6) = scan_d_m_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans till they yield converged power flow solution, create report for all orientation scans and mangitude scans performed
CHAROPTNS(7)	Option to solve Power Flow with GIC losses added to the base case (alias is power_flow_optn) (by default)
CHAROPTNS(7) =	do not perform power flow solution
CHAROPTNS(7) = fdns	use fixed slope decoupled Newton-Raphson method
CHAROPTNS(7) = fnsl	use Full Newton-Raphson method
CHAROPTNS(7) = nsol	use Decoupled Newton-Raphson method
Real EJETOPTNS (5)	array of 5 elements specifying Electrojet Characteristics, used only if charoptns(1)=nonuniform(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	EJETOPTNS(1)	eletrojet current in million amperes, must be >0 (alias is ejet_million_amps) (no default allowed)
	EJETOPTNS(2)	Cauchy distribution half-width in km, must be >0 (alias is ejet_halfwidth_km) (no default allowed)
	EJETOPTNS(3)	period of variation in minutes, must be >0 (alias is ejet_period_min) (no default allowed)
	EJETOPTNS(4)	height of current in km, must be >0 (alias is ejet_height_km) (no default allowed)
	EJETOPTNS(5)	latitude of center of electrojet in degrees (alias is ejet_center_deg) (no default allowed)
Character*260 FILEOPTNS (6)	array of 6 elements specifying File Used(input).	
	FILEOPTNS(1)	GIC data file name (input). (alias is gicfile) (no default allowed)
	FILEOPTNS(2)	GIC updates to Base Case file name (output). (alias is addfile) (blank by default)
	FILEOPTNS(2) = ''	no file created
	FILEOPTNS(3)	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (alias is purgfile) (blank by default)
	FILEOPTNS(3) = ''	no file created
	FILEOPTNS(4)	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow (output). (alias is rnwkfile) (blank by default)
	FILEOPTNS(4) = ''	no file created
	FILEOPTNS(5)	GIC Results map data file for given Efield magnitude and degrees OR Efield magnitude and degrees scans which give maximum Var losses when scans are performed (output). This is used by GICMAPS to plot GIC results on network map. (alias is pygicfile) (nooutput by default)
	FILEOPTNS(5) = ''	temporary file created
	FILEOPTNS(5) = nooutput	no file created
	FILEOPTNS(6)	Transformer Thermal Analysis GIC(t) CSV file (output). (alias is gictfile) (nooutput by default)
	FILEOPTNS(6) = ''	temporary file created
	FILEOPTNS(6) = nooutput	no file created
Integer RPTOPTNS (7)	array of 7 elements specifying Report Options(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)
	RPTOPTNS(2) = 0	No
	RPTOPTNS(2) = 1	Yes
	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)
	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes
	RPTOPTNS(5)	report transformer GIC flows (alias is rpttrn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_gic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
Integer IERR	error code (output).	
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.6. GIC_6

This API routine is obsolete. It has been replaced by the API routine gic_7 .

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_GIC_6  SID ALL INTGOPTNS(1)..INTGOPTNS(10) REALOPTNS(1)..REALOPTNS(17)
 CHAROPTNS(1)..CHAROPTNS(7) EJETOPTNS(1)..EJETOPTNS(5)
 FILEOPTNS(1)..FILEOPTNS(6) RPTOPTNS(1)..RPTOPTNS(8)
```

Python command syntax:

```
ierr = gic_6(sid, all, intgoptns, realoptns, charoptns, ejetoptns,
 fileoptns, rptoptns)
```

Fortran command syntax:

```
CALL GIC_6(SID, ALL, INTGOPTNS, REALOPTNS, CHAROPTNS, EJETOPTNS,
 FILEOPTNS, RPTOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (10)	array of 10 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	1 through 5, consider study subsystem buses + these many levels of inter-tie buses and ignore subsystem definition voltage limits for intertie buses added
	INTGOPTNS(1) < 0	-1 through -5, consider study subsystem buses + these many levels of inter-tie buses and consider subsystem definition voltage limits for intertie buses added
	INTGOPTNS(2)	Year number to scale benchmark event GMD storm. These scaling factors account in the influence of geomagnetic latitude on the estimated geoelectric field magnitude

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	and are provided in NERC TPL-007. (alias is study_year) (0 by default)
INTGOPTNS(2) = 0	current year, used in benchmark and supplemental geoelectric field calculations only.
INTGOPTNS(2) > 0	specific year
INTGOPTNS(3)	Subsystem sid for supplemental GMD event (alias is sid_supp) (0 by default)
INTGOPTNS(3) = 0	No supplemental GMD event
INTGOPTNS(3) > 0	subsystem number (1 - 11). Buses and branches in this subsystem will have Efield due to supplemental GMD event.
INTGOPTNS(4)	Option for Transformer Thermal Analysis (alias is thermal_ana_optn) (0 by default)
INTGOPTNS(4) = -1	perform on all transformers
INTGOPTNS(4) = 0	do not perform
INTGOPTNS(4) > 0	perform on these many top transformers ordered by effective GIC flow
INTGOPTNS(5)	Option to run power flow for each degree scan calculation (alias is degscan_pf_optn) (0 by default)
INTGOPTNS(5) = 0	Solve power flow only for worst degree
INTGOPTNS(5) = 1	Solve power flow for each degree scan calculation
INTGOPTNS(6)	Option to include buses of boundary transformers in study subsystem (alias is boundary_trn) (0 by default)
INTGOPTNS(6) = 0	consider only study subsystem buses
INTGOPTNS(6) = 1	Include buses of boundary transformers considering subsystem definition voltage limits
INTGOPTNS(6) = 2	Include buses of boundary transformers ignoring subsystem definition voltage limits
INTGOPTNS(7)	Option for Transformers to include in worst case determination (alias is worstcase_trn) (0 by default)
INTGOPTNS(7) = 0	All transformers in study subsystem
INTGOPTNS(7) = 1	All transformers in report subsystem
INTGOPTNS(8)	Option for Supplemental event Moving Box (alias is supp_evt) (0 by default)
INTGOPTNS(8) = 0	No supplemental event
INTGOPTNS(8) = 1	Use supplemental event subsytem SID as the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
INTGOPTNS(8) = 2	Rank substations with maximum GIC flows as center of the moving box from storm orientation scans that results in maximum VAR losses. For CHAROPTNS(6)='', Orientation of

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		Efield=Supplemental Efield=Orienation that gives Maximum VAR losses
	INTGOPTNS(8) = 3	Rank transformers with maximum GIC flows as center of the moving box from storm orientation scans. For CHAROPTNS(6)='', Orientation of Efield=Orienation that gives Maximum VAR losses, Orientation of Supplemental Efield=Orienation that gives Maximum GICs in transformer
	INTGOPTNS(8) = 4	Use substation number provided as center of the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
	INTGOPTNS(8) = 5	Use location provided as center of the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
	INTGOPTNS(9)	Option for number of Supplemental event moving boxes. It is not used when intgoptns(8)=0 and intgoptns(8)=4 (alias is supp_box_num) (0 by default)
	INTGOPTNS(9) = 0	Do not perform moving box calculations when when intgoptns(8)=1, 2, 3
	INTGOPTNS(9) > 0	When intgoptns(8)=1, number of substations to consider for center of moving box
	INTGOPTNS(9) > 0	When intgoptns(8)=2, number of transformers to consider for center of moving box
	INTGOPTNS(9) > 0	When intgoptns(8)=3, valid substation number to consider for center of moving box
	INTGOPTNS(10)	Option for treatment of the transmission line that intersect with Supplemental event moving box (alias is brn_seg_efld) (0 by default)
	INTGOPTNS(10) = 0	Apply Supplemental event Efield to line segment inside the box and Benchmark event Efield to line segments outside the box
	INTGOPTNS(10) = 1	Apply Supplemental event Efield to entire line when any segment of the line is inside the box
Real REALOPTNS (17)	array of 17 elements specifying Real Options(input).	
	REALOPTNS(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform or supplemental (alias is efield_mag) (0 by default)
	REALOPTNS(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	REALOPTNS(1) > 0	when charoptns(1)=uniform, no default allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALOPTNS(1) > 0	when charoptns(1)=benchmark, used as specified
REALOPTNS(2)	electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform or supplemental (alias is efield_deg) (0 by default)
REALOPTNS(2) > 0	when charoptns(1)=uniform or benchmark
REALOPTNS(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
REALOPTNS(3) = 0	Substation is ungrounded
REALOPTNS(3) > 0	Use specified grounding dc resistance
REALOPTNS(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in network data (alias is branch_xbyr) (30 by default)
REALOPTNS(4) = 0	Use default
REALOPTNS(4) > 0	Use specified
REALOPTNS(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
REALOPTNS(5) = 0	Use default
REALOPTNS(5) > 0	Use specified
REALOPTNS(6)	supplemental event electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag_supp) (realoptns(1) by default)
REALOPTNS(6) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is not zero
REALOPTNS(6) > 0	when charoptns(1)=supplemental
REALOPTNS(7)	local GMD hot spots electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform (alias is efield_deg_supp) (realoptns(2) by default)
REALOPTNS(7) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is non-zero
REALOPTNS(7) > 0	when charoptns(1)=supplemental
REALOPTNS(8)	transmission line AC to DC resistance conversion factor, must be >0 (alias is branch_rac2rdc) (1.0 by default)
REALOPTNS(8) = 0	Use default
REALOPTNS(8) > 0	Use specified
REALOPTNS(9)	transformer winding AC to DC resistance conversion factor, must be >0 (alias is transformer_rac2rdc) (1.0 by default)
REALOPTNS(9) = 0	Use default
REALOPTNS(9) > 0	Use specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALOPTNS(10)	Degree Scan step size, range 1.0 to 180 degrees (alias is degscan_step) (10.0 by default)
REALOPTNS(10) = 0	Use default
REALOPTNS(10) > 0	Use specified
REALOPTNS(11)	Magnitude Scan step size, must be >1.0 V/km (alias is magscan_step) (4.0 by default)
REALOPTNS(11) = 0	Use default
REALOPTNS(11) > 0	Use specified
REALOPTNS(12)	Percent GMD Mvar loss step size. Total GMD Mvar losses added incrementally to the base case to obtain power flow solution, must be >1.0 (alias is pf_qpct_step) (100.0 by default)
REALOPTNS(12) = 0	Use default
REALOPTNS(12) > 0	Use specified
REALOPTNS(13)	Magnitude Scan maximum storm strength, must be >1.0 (alias is magscan_max) (20.0 by default)
REALOPTNS(13) = 0	Use default
REALOPTNS(13) > 0	Use specified
REALOPTNS(14)	Supplemental event moving box North-South length in km, must be >1.0, used when intgoptns(8)>0 (alias is supp_box_ns_km) (100.0 by default)
REALOPTNS(14) = 0	Use default
REALOPTNS(14) > 0	Use specified
REALOPTNS(15)	Supplemental event moving box East-West length in km, must be >1.0, used when intgoptns(8)>0 (alias is supp_box_ew_km) (500.0 by default)
REALOPTNS(15) = 0	Use default
REALOPTNS(15) > 0	Use specified
REALOPTNS(16)	Supplemental event moving box center point longitude in degrees, used only when intgoptns(8)=4 (alias is supp_box_lon_c) (0.0 by default)
REALOPTNS(16) = 0	Use default
REALOPTNS(16) > 0	Use specified
REALOPTNS(17)	Supplemental event moving box center point latitude in degrees, used only when intgoptns(8)=4 (alias is supp_box_lat_c) (0.0 by default)
REALOPTNS(17) = 0	Use default
REALOPTNS(17) > 0	Use specified

Character*12 CHAROPTNS (7) array of 7 elements specifying Character Options(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHAROPTNS(1)	Electric Field Type (alias is efield_type) (uniform by default)
CHAROPTNS(1) = uniform	for uniform geoelectric field and also supplemental event electric field when intgoptns(3) and realoptns(6) are non-zero
CHAROPTNS(1) = benchmark	for benchmark event geoelectric field and beta factors (B) and also supplemental event electric field and beta factors (Bs) when intgoptns(3) and realoptns(6) are non-zero
CHAROPTNS(1) = nonuniform	for nonuniform geoelectric field
CHAROPTNS(1) = supplemental	for supplemental event geoelectric field and beta factors (Bs)
CHAROPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (v/km by default)
CHAROPTNS(2) = v/km	for volts/km
CHAROPTNS(2) = v/mi	for volts/mile
CHAROPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (rdch by default)
CHAROPTNS(3) = sav	add updates to Saved Case
CHAROPTNS(3) = rdch	create RDCH raw file of GIC updates
CHAROPTNS(4)	Option to select method for GIC to Mvar Calculation (alias is gic2mvar_optn) (kfactors by default)
CHAROPTNS(4) = kfactors	for GIC to Mvar scaling factors from GIC data file when provided, otherwise default scaling factors
CHAROPTNS(4) = capchar	for GIC to Mvar scaling factors from GIC data file when provided, otherwise scaling factors from default transformer Mvar and GIC characteristics
CHAROPTNS(5)	Earth Model Name. A Standard or User defined model name must be provided when Benchmark Event or Non-uniform electric field is to be modeled or Transformer Thermal Analysis is to be performed. (alias is earth_model_name) (by default)
CHAROPTNS(6)	Option to scan storm event scenarios (alias is scan_storm_event) (by default)
CHAROPTNS(6) =	no storm orientation and/or magnitude scan
CHAROPTNS(6) = scan_deg	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for only orientation that gives maximum VAR losses
CHAROPTNS(6) = scan_deg_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		uniform or benchmark, create report for each orientation scanned
	CHAROPTNS(6) = scan_mag	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for only final electric field magnitude.
	CHAROPTNS(6) = scan_mag_rpt	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for each scanned electric field magnitude.
	CHAROPTNS(6) = scan_d_m	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans tiil they yield converged power flow solution, create report for only orientation that gives maximum VAR losses and final electric field magnitude
	CHAROPTNS(6) = scan_d_m_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans till they yield converged power flow solution, create report for all orientation scans and mangitude scans performed
	CHAROPTNS(7)	Option to solve Power Flow with GIC losses added to the base case (alias is power_flow_optn) (by default)
	CHAROPTNS(7) =	do not perform power flow solution
	CHAROPTNS(7) = fdns	use fixed slope decoupled Newton-Raphson method
	CHAROPTNS(7) = fnsl	use Full Newton-Raphson method
	CHAROPTNS(7) = nsol	use Decoupled Newton-Raphson method
Real EJETOPTNS (5)	array of 5 elements specifying Electrojet Characteristics, used only if charoptns(1)=nonuniform(input).	
	EJETOPTNS(1)	eletrojet current in million amperes, must be >0 (alias is ejet_million_amps) (no default allowed)
	EJETOPTNS(2)	Cauchy distribution half-width in km, must be >0 (alias is ejet_halfwidth_km) (no default allowed)
	EJETOPTNS(3)	period of variation in minutes, must be >0 (alias is ejet_period_min) (no default allowed)
	EJETOPTNS(4)	height of current in km, must be >0 (alias is ejet_height_km) (no default allowed)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	EJETOPTNS(5)	latitude of center of electrojet in degrees (alias is ejet_center_deg) (no default allowed)
Character*260 FILEOPTNS (6)	array of 6 elements specifying File Used(input).	
	FILEOPTNS(1)	GIC data file name (input). (alias is gicfile) (no default allowed)
	FILEOPTNS(2)	GIC updates to Base Case file name (output). (alias is addfile) (blank by default)
	FILEOPTNS(2) = ' '	no file created
	FILEOPTNS(3)	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (alias is purgfile) (blank by default)
	FILEOPTNS(3) = ' '	no file created
	FILEOPTNS(4)	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow (output). (alias is rnwkfile) (blank by default)
	FILEOPTNS(4) = ' '	no file created
	FILEOPTNS(5)	GIC Results map data file for given Efield magnitude and degrees OR Efield magnitude and degrees scans which give maximum Var losses when scans are performed (output). This is used by GICMAPS to plot GIC results on network map. (alias is pygicfile) (nooutput by default)
	FILEOPTNS(5) = ' '	temporary file created
	FILEOPTNS(5) = nooutput	no file created
	FILEOPTNS(6)	Transformer Thermal Analysis GIC(t) CSV file (output). (alias is gictfile) (nooutput by default)
	FILEOPTNS(6) = ' '	temporary file created
	FILEOPTNS(6) = nooutput	no file created
Integer RPTOPTNS (8)	array of 8 elements specifying Report Options(input).	
	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)
	RPTOPTNS(2) = 0	No
	RPTOPTNS(2) = 1	Yes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)
	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes
	RPTOPTNS(5)	report transformer GIC flows (alias is rpt-trn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_g-ic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
	RPTOPTNS(8)	Subsystem sid for report (alias is rpt_sid) (0 by default)
	RPTOPTNS(8) = 0	Use study subsystem as report subsystem
	RPTOPTNS(8) = 1	Subsystem number (1 - 11). GIC report created for buses and branches in this subsystem.
Integer IERR	error code (output).	
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.7. GIC_7

This API routine is obsolete. It has been replaced by the API routine gic_8 .

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_7  SID ALL INTGOPTNS(1)..INTGOPTNS(10) REALOPTNS(1)..REALOPTNS(17)
 CHAROPTNS(1)..CHAROPTNS(7) EJETOPTNS(1)..EJETOPTNS(5)
 FILEOPTNS(1)..FILEOPTNS(5) RPTOPTNS(1)..RPTOPTNS(8)
```

Python command syntax:

```
ierr = gic_7(sid, all, intgoptns, realoptns, charoptns, ejetoptns,
 fileoptns, rptoptns)
```

Fortran command syntax:

```
CALL GIC_7(SID, ALL, INTGOPTNS, REALOPTNS, CHAROPTNS, EJETOPTNS,
 FILEOPTNS, RPTOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (10)	array of 10 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	1 through 5, consider study subsystem buses + these many levels of inter-tie buses and ignore subsystem definition voltage limits for intertie buses added
	INTGOPTNS(1) < 0	-1 through -5, consider study subsystem buses + these many levels of inter-tie buses and consider subsystem definition voltage limits for intertie buses added
	INTGOPTNS(2)	Year number to scale benchmark event GMD storm. These scaling factors account in the influence of geomagnetic latitude on the estimated geoelectric field magnitude

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	and are provided in NERC TPL-007. (alias is study_year) (0 by default)
INTGOPTNS(2) = 0	current year, used in benchmark and supplemental geoelectric field calculations only.
INTGOPTNS(2) > 0	specific year
INTGOPTNS(3)	Subsystem sid for supplemental GMD event (alias is sid_supp) (0 by default)
INTGOPTNS(3) = 0	No supplemental GMD event
INTGOPTNS(3) > 0	subsystem number (1 - 11). Buses and branches in this subsystem will have Efield due to supplemental GMD event.
INTGOPTNS(4)	Option for Transformer Thermal Analysis (alias is thermal_ana_optn) (0 by default)
INTGOPTNS(4) = -1	perform on all transformers
INTGOPTNS(4) = 0	do not perform
INTGOPTNS(4) > 0	perform on these many top transformers ordered by effective GIC flow
INTGOPTNS(5)	Option to run power flow for each degree scan calculation (alias is degscan_pf_optn) (0 by default)
INTGOPTNS(5) = 0	Solve power flow only for worst degree
INTGOPTNS(5) = 1	Solve power flow for each degree scan calculation
INTGOPTNS(6)	Option to include buses of boundary transformers in study subsystem (alias is boundary_trn) (0 by default)
INTGOPTNS(6) = 0	consider only study subsystem buses
INTGOPTNS(6) = 1	Include buses of boundary transformers considering subsystem definition voltage limits
INTGOPTNS(6) = 2	Include buses of boundary transformers ignoring subsystem definition voltage limits
INTGOPTNS(7)	Option for Transformers to include in worst case determination (alias is worstcase_trn) (0 by default)
INTGOPTNS(7) = 0	All transformers in study subsystem
INTGOPTNS(7) = 1	All transformers in report subsystem
INTGOPTNS(8)	Option for Supplemental event Moving Box (alias is supp_evt) (0 by default)
INTGOPTNS(8) = 0	No supplemental event
INTGOPTNS(8) = 1	Use supplemental event subsytem SID as the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
INTGOPTNS(8) = 2	Rank substations with maximum GIC flows as center of the moving box from storm orientation scans that results in maximum VAR losses. For CHAROPTNS(6)='', Orientation of

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		Efield=Supplemental Efield=Orienation that gives Maximum VAR losses
	INTGOPTNS(8) = 3	Rank transformers with maximum GIC flows as center of the moving box from storm orientation scans. For CHAROPTNS(6)='', Orientation of Efield=Orienation that gives Maximum VAR losses, Orientation of Supplemental Efield=Orienation that gives Maximum GICs in transformer
	INTGOPTNS(8) = 4	Use substation number provided as center of the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
	INTGOPTNS(8) = 5	Use location provided as center of the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
	INTGOPTNS(9)	Option for number of Supplemental event moving boxes. It is not used when intgoptns(8)=0 and intgoptns(8)=4 (alias is supp_box_num) (0 by default)
	INTGOPTNS(9) = 0	Do not perform moving box calculations when when intgoptns(8)=1, 2, 3
	INTGOPTNS(9) > 0	When intgoptns(8)=1, number of substations to consider for center of moving box
	INTGOPTNS(9) > 0	When intgoptns(8)=2, number of transformers to consider for center of moving box
	INTGOPTNS(9) > 0	When intgoptns(8)=3, valid substation number to consider for center of moving box
	INTGOPTNS(10)	Option for treatment of the transmission line that intersect with Supplemental event moving box (alias is brn_seg_efld) (0 by default)
	INTGOPTNS(10) = 0	Apply Supplemental event Efield to line segment inside the box and Benchmark event Efield to line segments outside the box
	INTGOPTNS(10) = 1	Apply Supplemental event Efield to entire line when any segment of the line is inside the box
Real REALOPTNS (17)	array of 17 elements specifying Real Options(input).	
	REALOPTNS(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform or supplemental (alias is efield_mag) (0 by default)
	REALOPTNS(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	REALOPTNS(1) > 0	when charoptns(1)=uniform, no default allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALOPTNS(1) > 0	when charoptns(1)=benchmark, used as specified
REALOPTNS(2)	electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform or supplemental (alias is efield_deg) (0 by default)
REALOPTNS(2) > 0	when charoptns(1)=uniform or benchmark
REALOPTNS(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
REALOPTNS(3) = 0	Substation is ungrounded
REALOPTNS(3) > 0	Use specified grounding dc resistance
REALOPTNS(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in network data (alias is branch_xbyr) (30 by default)
REALOPTNS(4) = 0	Use default
REALOPTNS(4) > 0	Use specified
REALOPTNS(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
REALOPTNS(5) = 0	Use default
REALOPTNS(5) > 0	Use specified
REALOPTNS(6)	supplemental event electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag_supp) (realoptns(1) by default)
REALOPTNS(6) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is not zero
REALOPTNS(6) > 0	when charoptns(1)=supplemental
REALOPTNS(7)	local GMD hot spots electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform (alias is efield_deg_supp) (realoptns(2) by default)
REALOPTNS(7) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is non-zero
REALOPTNS(7) > 0	when charoptns(1)=supplemental
REALOPTNS(8)	transmission line AC to DC resistance conversion factor, must be >0 (alias is branch_rac2rdc) (1.0 by default)
REALOPTNS(8) = 0	Use default
REALOPTNS(8) > 0	Use specified
REALOPTNS(9)	transformer winding AC to DC resistance conversion factor, must be >0 (alias is transformer_rac2rdc) (1.0 by default)
REALOPTNS(9) = 0	Use default
REALOPTNS(9) > 0	Use specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALOPTNS(10)	Degree Scan step size, range 1.0 to 180 degrees (alias is degscan_step) (10.0 by default)
REALOPTNS(10) = 0	Use default
REALOPTNS(10) > 0	Use specified
REALOPTNS(11)	Magnitude Scan step size, must be >1.0 V/km (alias is magscan_step) (4.0 by default)
REALOPTNS(11) = 0	Use default
REALOPTNS(11) > 0	Use specified
REALOPTNS(12)	Percent GMD Mvar loss step size. Total GMD Mvar losses added incrementally to the base case to obtain power flow solution, must be >1.0 (alias is pf_qpct_step) (100.0 by default)
REALOPTNS(12) = 0	Use default
REALOPTNS(12) > 0	Use specified
REALOPTNS(13)	Magnitude Scan maximum storm strength, must be >1.0 (alias is magscan_max) (20.0 by default)
REALOPTNS(13) = 0	Use default
REALOPTNS(13) > 0	Use specified
REALOPTNS(14)	Supplemental event moving box North-South length in km, must be >1.0, used when intgoptns(8)>0 (alias is supp_box_ns_km) (100.0 by default)
REALOPTNS(14) = 0	Use default
REALOPTNS(14) > 0	Use specified
REALOPTNS(15)	Supplemental event moving box East-West length in km, must be >1.0, used when intgoptns(8)>0 (alias is supp_box_ew_km) (500.0 by default)
REALOPTNS(15) = 0	Use default
REALOPTNS(15) > 0	Use specified
REALOPTNS(16)	Supplemental event moving box center point longitude in degrees, used only when intgoptns(8)=4 (alias is supp_box_lon_c) (0.0 by default)
REALOPTNS(16) = 0	Use default
REALOPTNS(16) > 0	Use specified
REALOPTNS(17)	Supplemental event moving box center point latitude in degrees, used only when intgoptns(8)=4 (alias is supp_box_lat_c) (0.0 by default)
REALOPTNS(17) = 0	Use default
REALOPTNS(17) > 0	Use specified

Character*12 CHAROPTNS (7) array of 7 elements specifying Character Options(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHAROPTNS(1)	Electric Field Type (alias is efield_type) (uniform by default)
CHAROPTNS(1) = uniform	for uniform geoelectric field and also supplemental event electric field when intgoptns(3) and realoptns(6) are non-zero
CHAROPTNS(1) = benchmark	for benchmark event geoelectric field and beta factors (B) and also supplemental event electric field and beta factors (Bs) when intgoptns(3) and realoptns(6) are non-zero
CHAROPTNS(1) = nonuniform	for nonuniform geoelectric field
CHAROPTNS(1) = supplemental	for supplemental event geoelectric field and beta factors (Bs)
CHAROPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (v/km by default)
CHAROPTNS(2) = v/km	for volts/km
CHAROPTNS(2) = v/mi	for volts/mile
CHAROPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (rdch by default)
CHAROPTNS(3) = sav	add updates to Saved Case
CHAROPTNS(3) = rdch	create RDCH raw file of GIC updates
CHAROPTNS(4)	Option to select method for GIC to Mvar Calculation (alias is gic2mvar_optn) (kfactors by default)
CHAROPTNS(4) = kfactors	for GIC to Mvar scaling factors from GIC data file when provided, otherwise default scaling factors
CHAROPTNS(4) = capchar	for GIC to Mvar scaling factors from GIC data file when provided, otherwise scaling factors from default transformer Mvar and GIC characteristics
CHAROPTNS(5)	Earth Model Name. A Standard or User defined model name must be provided when Benchmark Event or Non-uniform electric field is to be modeled or Transformer Thermal Analysis is to be performed. (alias is earth_model_name) (by default)
CHAROPTNS(6)	Option to scan storm event scenarios (alias is scan_storm_event) (by default)
CHAROPTNS(6) =	no storm orientation and/or magnitude scan
CHAROPTNS(6) = scan_deg	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for only orientation that gives maximum VAR losses
CHAROPTNS(6) = scan_deg_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		uniform or benchmark, create report for each orientation scanned
	CHAROPTNS(6) = scan_mag	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for only final electric field magnitude.
	CHAROPTNS(6) = scan_mag_rpt	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for each scanned electric field magnitude.
	CHAROPTNS(6) = scan_d_m	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans tiil they yield converged power flow solution, create report for only orientation that gives maximum VAR losses and final electric field magnitude
	CHAROPTNS(6) = scan_d_m_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans till they yield converged power flow solution, create report for all orientation scans and mangitude scans performed
	CHAROPTNS(7)	Option to solve Power Flow with GIC losses added to the base case (alias is power_flow_optn) (by default)
	CHAROPTNS(7) =	do not perform power flow solution
	CHAROPTNS(7) = fdns	use fixed slope decoupled Newton-Raphson method
	CHAROPTNS(7) = fnsl	use Full Newton-Raphson method
	CHAROPTNS(7) = nsol	use Decoupled Newton-Raphson method
Real EJETOPTNS (5)	array of 5 elements specifying Electrojet Characteristics, used only if charoptns(1)=nonuniform(input).	
	EJETOPTNS(1)	eletrojet current in million amperes, must be >0 (alias is ejet_million_amps) (no default allowed)
	EJETOPTNS(2)	Cauchy distribution half-width in km, must be >0 (alias is ejet_halfwidth_km) (no default allowed)
	EJETOPTNS(3)	period of variation in minutes, must be >0 (alias is ejet_period_min) (no default allowed)
	EJETOPTNS(4)	height of current in km, must be >0 (alias is ejet_height_km) (no default allowed)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	EJETOPTNS(5)	latitude of center of electrojet in degrees (alias is ejet_center_deg) (no default allowed)
Character*260 FILEOPTNS (5)	array of 5 elements specifying File Used(input).	
	FILEOPTNS(1)	GIC updates to Base Case file name (output). (alias is addfile) (blank by default)
	FILEOPTNS(1) = ' '	no file created
	FILEOPTNS(2)	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (alias is purgfile) (blank by default)
	FILEOPTNS(2) = ' '	no file created
	FILEOPTNS(3)	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow (output). (alias is rnwkfile) (blank by default)
	FILEOPTNS(3) = ' '	no file created
	FILEOPTNS(4)	GIC Results map data file for given Efield magnitude and degrees OR Efield magnitude and degrees scans which give maximum Var losses when scans are performed (output). This is used by GICMAPS to plot GIC results on network map. (alias is pygicfile) (nooutput by default)
	FILEOPTNS(4) = ' '	temporary file created
	FILEOPTNS(4) = nooutput	no file created
	FILEOPTNS(5)	Transformer Thermal Analysis GIC(t) CSV file (output). (alias is gictfile) (nooutput by default)
	FILEOPTNS(5) = ' '	temporary file created
	FILEOPTNS(5) = nooutput	no file created
Integer RPTOPTNS (8)	array of 8 elements specifying Report Options(input).	
	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)
	RPTOPTNS(2) = 0	No
	RPTOPTNS(2) = 1	Yes
	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes
	RPTOPTNS(5)	report transformer GIC flows (alias is rpt-trn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_g-ic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
	RPTOPTNS(8)	Subsystem sid for report (alias is rpt_sid) (0 by default)
	RPTOPTNS(8) = 0	Use study subsystem as report subsystem
	RPTOPTNS(8) = 1	Subsystem number (1 - 11). GIC report creat-ed for buses and branches in this subsystem.
Integer IERR	error code (output).	
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.8. GIC_8

Use this API to calculate Geomagnetically Induced Currents (GIC) in electric power system network as a result of Geomagnetic Disturbance (GMD).

This API routine was first introduced in release 34.8.0 and 35.1.0.

Batch command syntax:

```
BAT_GIC_8  SID ALL INTGOPTNS(1)..INTGOPTNS(10) REALOPTNS(1)..REALOPTNS(19)
 CHAROPTNS(1)..CHAROPTNS(7) EJETOPTNS(1)..EJETOPTNS(5)
 FILEOPTNS(1)..FILEOPTNS(5) RPTOPTNS(1)..RPTOPTNS(8)
```

Python command syntax:

```
ierr = gic_8(sid, all, intgoptns, realoptns, charoptns, ejetoptns,
 fileoptns, rptoptns)
```

Fortran command syntax:

```
CALL GIC_8(SID, ALL, INTGOPTNS, REALOPTNS, CHAROPTNS, EJETOPTNS,
 FILEOPTNS, RPTOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (10)	array of 10 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	1 through 5, consider study subsystem buses + these many levels of inter-tie buses and ignore subsystem definition voltage limits for intertie buses added
	INTGOPTNS(1) < 0	-1 through -5, consider study subsystem buses + these many levels of inter-tie buses and consider subsystem definition voltage limits for intertie buses added
	INTGOPTNS(2)	Year number to scale benchmark event GMD storm. These scaling factors account in the influence of geomagnetic latitude on the estimated geoelectric field magnitude and are provided in NERC TPL-007. (alias is study_year) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

INTGOPTNS(2) = 0	current year, used in benchmark and supplemental geoelectric field calculations only.
INTGOPTNS(2) > 0	specific year
INTGOPTNS(3)	Subsystem sid for supplemental GMD event (alias is sid_supp) (0 by default)
INTGOPTNS(3) = 0	No supplemental GMD event
INTGOPTNS(3) > 0	subsystem number (1 - 11). Buses and branches in this subsystem will have Efield due to supplemental GMD event.
INTGOPTNS(4)	Option for Transformer Thermal Analysis (alias is thermal_ana_optn) (0 by default)
INTGOPTNS(4) = -1	perform on all transformers
INTGOPTNS(4) = 0	do not perform
INTGOPTNS(4) > 0	perform on these many top transformers ordered by effective GIC flow
INTGOPTNS(5)	Option to run power flow for each degree scan calculation (alias is degscan_pf_optn) (0 by default)
INTGOPTNS(5) = 0	Solve power flow only for worst degree
INTGOPTNS(5) = 1	Solve power flow for each degree scan calculation
INTGOPTNS(6)	Option to include buses of boundary transformers in study subsystem (alias is boundary_trn) (0 by default)
INTGOPTNS(6) = 0	consider only study subsystem buses
INTGOPTNS(6) = 1	Include buses of boundary transformers considering subsystem definition voltage limits
INTGOPTNS(6) = 2	Include buses of boundary transformers ignoring subsystem definition voltage limits
INTGOPTNS(7)	Option for Transformers to include in worst case determination (alias is worstcase_trn) (0 by default)
INTGOPTNS(7) = 0	All transformers in study subsystem
INTGOPTNS(7) = 1	All transformers in report subsystem
INTGOPTNS(8)	Option for Supplemental event Moving Box (alias is supp_evt) (0 by default)
INTGOPTNS(8) = 0	No supplemental event
INTGOPTNS(8) = 1	Use supplemental event subsytem SID as the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
INTGOPTNS(8) = 2	Rank substations with maximum GIC flows as center of the moving box from storm orientation scans that results in maximum VAR losses. For CHAROPTNS(6)='', Orientation of Efield=Supplemental Efield=Orienation that gives Maximum VAR losses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGOPTNS(8) = 3	Rank transformers with maximum GIC flows as center of the moving box from storm orientation scans. For CHAROPTNS(6)='', Orientation of Efield=Orienation that gives Maximum VAR losses, Orientation of Supplemental Efield=Orienation that gives Maximum GICs in transformer
	INTGOPTNS(8) = 4	Use substation number provided as center of the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
	INTGOPTNS(8) = 5	Use location provided as center of the moving box. For CHAROPTNS(6)='', Orientation of Efield=REALOPTNS(2), Orientation of Supplemental Efield=REALOPTNS(7)
	INTGOPTNS(9)	Option for number of Supplemental event moving boxes. It is not used when intgoptns(8)=0 and intgoptns(8)=4 (alias is supp_box_num) (0 by default)
	INTGOPTNS(9) = 0	Do not perform moving box calculations when when intgoptns(8)=1, 2, 3
	INTGOPTNS(9) > 0	When intgoptns(8)=1, number of substations to consider for center of moving box
	INTGOPTNS(9) > 0	When intgoptns(8)=2, number of transformers to consider for center of moving box
	INTGOPTNS(9) > 0	When intgoptns(8)=3, valid substation number to consider for center of moving box
	INTGOPTNS(10)	Option for treatment of the transmission line that intersect with Supplemental event moving box (alias is brn_seg_efld) (0 by default)
	INTGOPTNS(10) = 0	Apply Supplemental event Efield to line segment inside the box and Benchmark event Efield to line segments outside the box
	INTGOPTNS(10) = 1	Apply Supplemental event Efield to entire line when any segment of the line is inside the box
Real REALOPTNS (19)	array of 19 elements specifying Real Options(input).	
	REALOPTNS(1)	electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform or supplemental (alias is efield_mag) (0 by default)
	REALOPTNS(1) = 0	when charoptns(1)=benchmark, when specified as 0.0, it will be set to 8.0 V/km (default benchmark event strength)
	REALOPTNS(1) > 0	when charoptns(1)=uniform, no default allowed
	REALOPTNS(1) > 0	when charoptns(1)=benchmark, used as specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALOPTNS(2)	electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform or supplemental (alias is efield_deg) (0 by default)
REALOPTNS(2) > 0	when charoptns(1)=uniform or benchmark
REALOPTNS(3)	substation grounding dc resistance in ohms (alias is substation_r) (0.1 by default)
REALOPTNS(3) = 0	Substation is ungrounded
REALOPTNS(3) > 0	Use specified grounding dc resistance
REALOPTNS(4)	transmission line X/R ratio, must be >0, used to calculate branch DC resistance if R=0.0 in network data (alias is branch_xbyr) (30 by default)
REALOPTNS(4) = 0	Use default
REALOPTNS(4) > 0	Use specified
REALOPTNS(5)	transformer winding X/R ratio, must be >0, used to calculate winding DC resistance if R=0.0 in network data (alias is transformer_xbyr) (30 by default)
REALOPTNS(5) = 0	Use default
REALOPTNS(5) > 0	Use specified
REALOPTNS(6)	supplemental event electric field magnitude in units defined by charoptns(2), not used when charoptns(1)=nonuniform (alias is efield_mag_supp) (realoptns(1) by default)
REALOPTNS(6) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is not zero
REALOPTNS(6) > 0	when charoptns(1)=supplemental
REALOPTNS(7)	local GMD hot spots electric field direction in degrees, range 0 to 360 degrees, not used when charoptns(1)=nonuniform (alias is efield_deg_supp) (realoptns(2) by default)
REALOPTNS(7) > 0	when charoptns(1)=uniform or benchmark and intgoptns(3) is non-zero
REALOPTNS(7) > 0	when charoptns(1)=supplemental
REALOPTNS(8)	transmission line AC to DC resistance conversion factor, must be >0 (alias is branch_rac2rdc) (1.0 by default)
REALOPTNS(8) = 0	Use default
REALOPTNS(8) > 0	Use specified
REALOPTNS(9)	transformer winding AC to DC resistance conversion factor, must be >0 (alias is transformer_rac2rdc) (1.0 by default)
REALOPTNS(9) = 0	Use default
REALOPTNS(9) > 0	Use specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REALOPTNS(10)	Degree Scan step size, range 1.0 to 180 degrees (alias is degscan_step) (10.0 by default)
REALOPTNS(10) = 0	Use default
REALOPTNS(10) > 0	Use specified
REALOPTNS(11)	Magnitude Scan step size, must be >1.0 V/km (alias is magscan_step) (4.0 by default)
REALOPTNS(11) = 0	Use default
REALOPTNS(11) > 0	Use specified
REALOPTNS(12)	Percent GMD Mvar loss step size. Total GMD Mvar losses added incrementally to the base case to obtain power flow solution, must be >1.0 (alias is pf_qpct_step) (100.0 by default)
REALOPTNS(12) = 0	Use default
REALOPTNS(12) > 0	Use specified
REALOPTNS(13)	Magnitude Scan maximum storm strength, must be >1.0 (alias is magscan_max) (20.0 by default)
REALOPTNS(13) = 0	Use default
REALOPTNS(13) > 0	Use specified
REALOPTNS(14)	Supplemental event moving box North-South length in km, must be >1.0, used when intgoptns(8)>0 (alias is supp_box_ns_km) (100.0 by default)
REALOPTNS(14) = 0	Use default
REALOPTNS(14) > 0	Use specified
REALOPTNS(15)	Supplemental event moving box East-West length in km, must be >1.0, used when intgoptns(8)>0 (alias is supp_box_ew_km) (500.0 by default)
REALOPTNS(15) = 0	Use default
REALOPTNS(15) > 0	Use specified
REALOPTNS(16)	Supplemental event moving box center point longitude in degrees, used only when intgoptns(8)=4 (alias is supp_box_lon_c) (0.0 by default)
REALOPTNS(16) = 0	Use default
REALOPTNS(16) > 0	Use specified
REALOPTNS(17)	Supplemental event moving box center point latitude in degrees, used only when intgoptns(8)=4 (alias is supp_box_lat_c) (0.0 by default)
REALOPTNS(17) = 0	Use default
REALOPTNS(17) > 0	Use specified
REALOPTNS(18)	Branch segment length in km, used to calculate branch induced voltage from Efield

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		Grid, must be >0.0. Branch is subdivided into number of segements (points along the branch route) and NN algorithm finds nearest geoelectric efield grid data point to arrive efield at that branch route point. (alias is brn_segkm) (5.0 by default)
	REALOPTNS(18) = 0.0	Use default
	REALOPTNS(18) > 0	Use specified
	REALOPTNS(19)	Branch subsegment length in km, used to calculate branch induced voltage from Efield Grid, range 0.02 to 1.0 km. When efield value at branch route point changes, that branch segment is further subdivided into smaller segments to arrive at branch route point where NN algorithm efield change occurs. (alias is brn_subsegkm) (0.1 by default)
	REALOPTNS(19) = 0.0	Use default
	REALOPTNS(19) > 0	Use specified
Character*12 CHAROPTNS (7)	array of 7 elements specifying Character Options(input).	
	CHAROPTNS(1)	Electric Field Type (alias is efield_type) (uniform by default)
	CHAROPTNS(1) = uniform	for uniform geoelectric field and also supplemental event electric field when intgoptns(3) and realoptns(6) are non-zero
	CHAROPTNS(1) = benchmark	for benchmark event geoelectric field and beta factors (B) and also supplemental event electric field and beta factors (Bs) when intgoptns(3) and realoptns(6) are non-zero
	CHAROPTNS(1) = nonuniform	for nonuniform geoelectric field per elctrojet characteristics and complex image method
	CHAROPTNS(1) = supplemental	for supplemental event geoelectric field and beta factors (Bs)
	CHAROPTNS(1) = efieldgrid	for nonuniform geoelectric field per specified efield grid data and nearest neighbor algorithm
	CHAROPTNS(2)	Units of Electric Field Magnitude (alias is efield_unit) (v/km by default)
	CHAROPTNS(2) = v/km	for volts/km
	CHAROPTNS(2) = v/mi	for volts/mile
	CHAROPTNS(3)	Option to add GIC updates to base case (alias is addfile_optn) (rdch by default)
	CHAROPTNS(3) = sav	add updates to Saved Case
	CHAROPTNS(3) = rdch	create RDCH raw file of GIC updates
	CHAROPTNS(4)	Option to select method for GIC to Mvar Calculation (alias is gic2mvar_optn) (kfactors by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHAROPTNS(4) = kfactors	for GIC to Mvar scaling factors from GIC data file when provided, otherwise default scaling factors
CHAROPTNS(4) = capchar	for GIC to Mvar scaling factors from GIC data file when provided, otherwise scaling factors from default transformer Mvar and GIC characteristics
CHAROPTNS(5)	Earth Model Name. A Standard or User defined model name must be provided when Benchmark Event or Non-uniform electric field is to be modeled or Transformer Thermal Analysis is to be performed. (alias is earth_model_name) (by default)
CHAROPTNS(6)	Option to scan storm event scenarios (alias is scan_storm_event) (by default)
CHAROPTNS(6) =	no storm orientation and/or magnitude scan
CHAROPTNS(6) = scan_deg	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for only orientation that gives maximum VAR losses
CHAROPTNS(6) = scan_deg_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, create report for each orientation scanned
CHAROPTNS(6) = scan_mag	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for only final electric field magnitude.
CHAROPTNS(6) = scan_mag_rpt	perform storm electric field magnitude that yield converged power flow solution when charoptns(1) is uniform or benchmark and charoptns(7) specifies PF method. It uses orienation specified and creates report for each scanned electric field magnitude.
CHAROPTNS(6) = scan_d_m	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans tiil they yield converged power flow solution, create report for only orientation that gives maximum VAR losses and final electric field magnitude
CHAROPTNS(6) = scan_d_m_rpt	perform storm orientation scan that give maximum Var losses when charoptns(1) is uniform or benchmark, then perform magnitude scans till they yield converged power flow solution, create report for all orientation scans and mangitude scans performed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CHAROPTNS(7)	Option to solve Power Flow with GIC losses added to the base case (alias is power_flow_optn) (by default)
	CHAROPTNS(7) =	do not perform power flow solution
	CHAROPTNS(7) = fdns	use fixed slope decoupled Newton-Raphson method
	CHAROPTNS(7) = fnsl	use Full Newton-Raphson method
	CHAROPTNS(7) = nsol	use Decoupled Newton-Raphson method
	CHAROPTNS(7) = rsol	use Robust solution method
	CHAROPTNS(7) = last	use solution method used in input Saved case file
Real EJETOPTNS (5)	array of 5 elements specifying Electrojet Characteristics, used only if charoptns(1)=nonuniform(input).	
	EJETOPTNS(1)	eletrojet current in million amperes, must be >0 (alias is ejet_million_amps) (no default allowed)
	EJETOPTNS(2)	Cauchy distribution half-width in km, must be >0 (alias is ejet_halfwidth_km) (no default allowed)
	EJETOPTNS(3)	period of variation in minutes, must be >0 (alias is ejet_period_min) (no default allowed)
	EJETOPTNS(4)	height of current in km, must be >0 (alias is ejet_height_km) (no default allowed)
	EJETOPTNS(5)	latitude of center of electrojet in degrees (alias is ejet_center_deg) (no default allowed)
Character*260 FILEOPTNS (5)	array of 5 elements specifying File Used(input).	
	FILEOPTNS(1)	GIC updates to Base Case file name (output). (alias is addfile) (blank by default)
	FILEOPTNS(1) = ''	no file created
	FILEOPTNS(2)	RDCH file to remove GIC updates from GIC updated case in working memory to set it back to Base Case network condition (output). (alias is purgfile) (blank by default)
	FILEOPTNS(2) = ''	no file created
	FILEOPTNS(3)	GIC dc resistive network raw file. This represents the dc network used to calculate GIC flow (output). (alias is rnwkfile) (blank by default)
	FILEOPTNS(3) = ''	no file created
	FILEOPTNS(4)	GIC Results map data file for given Efield magnitude and degrees OR Efield magnitude and degrees scans which give maximum Var losses when scans are performed (output). This is used by GICMAPS to plot GIC results

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		on network map. (alias is pygicfile) (nooutput by default)
	FILEOPTNS(4) = ' '	temporary file created
	FILEOPTNS(4) = nooutput	no file created
	FILEOPTNS(5)	Transformer Thermal Analysis GIC(t) CSV file (output). (alias is gictfile) (nooutput by default)
	FILEOPTNS(5) = ' '	temporary file created
	FILEOPTNS(5) = nooutput	no file created
Integer RPTOPTNS (8)	array of 8 elements specifying Report Options(input).	
	RPTOPTNS(1)	what to report (alias is rptoptn) (-1 by default)
	RPTOPTNS(1) = -1	report all
	RPTOPTNS(1) = 0	no report
	RPTOPTNS(1) > 0	Report these many maximum values in each category
	RPTOPTNS(2)	report induced branch voltages (alias is rptbrn_indv) (0 by default)
	RPTOPTNS(2) = 0	No
	RPTOPTNS(2) = 1	Yes
	RPTOPTNS(3)	report DC bus voltages (alias is rptdc_busv) (0 by default)
	RPTOPTNS(3) = 0	No
	RPTOPTNS(3) = 1	Yes
	RPTOPTNS(4)	report branch GIC flows (alias is rptbrn_gic) (0 by default)
	RPTOPTNS(4) = 0	No
	RPTOPTNS(4) = 1	Yes
	RPTOPTNS(5)	report transformer GIC flows (alias is rpttrn_gic) (1 by default)
	RPTOPTNS(5) = 0	No
	RPTOPTNS(5) = 1	Yes
	RPTOPTNS(6)	report substation GIC flows (alias is rptstn_gic) (0 by default)
	RPTOPTNS(6) = 0	No
	RPTOPTNS(6) = 1	Yes
	RPTOPTNS(7)	report transformer losses (alias is rpttrn_q) (1 by default)
	RPTOPTNS(7) = 0	No
	RPTOPTNS(7) = 1	Yes
	RPTOPTNS(8)	Subsystem sid for report (alias is rpt_sid) (0 by default)
	RPTOPTNS(8) = 0	Use study subsystem as report subsystem

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	RPTOPTNS(8) = 1	Subsystem number (1 - 11). GIC report created for buses and branches in this subsystem.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	Invalid SID value or subsystem SID is not defined
	IERR = 2	Invalid ALL value
	IERR = 3	Topology error, bus type code and series element status inconsistencies
	IERR = 4	Invalid INTGOPTNS value, set to default
	IERR = 5	Invalid REALOPTNS value, no default allowed
	IERR = 6	Invalid REALOPTNS value, set to default
	IERR = 7	Invalid RPTOPTNS value, set to default value
	IERR = 8	Electric Field Value must be greater than 0.0
	IERR = 9	Error opening GIC File
	IERR = 10	Error opening GIC ADD File
	IERR = 11	Error opening GIC PURG File
	IERR = 12	Error opening GIC DC NETWORK File
	IERR = 13	No substation data provided
	IERR = 14	No substation data specified for following GIC study subsystem buses
	IERR = 15	No buses in GIC study subsystem
	IERR = 16	No GIC data input file provided
	IERR = 17	No non-transformer branches in working case
	IERR = 18	No non-transformer branches in GIC study subsystem
	IERR = 19	Prerequisite requirements for API are not met
	IERR = 20	Invalid localized GMD hot spots subsystem SID
	IERR = 21	No buses in local GMD hot spots subsystem
	IERR = 22	Invalid CHAROPTNS value, set to default
	IERR = 23	Invalid EJETOPTNS value
	IERR = 24	Benchmark Event default GMD strength different than standard strength
	IERR = 25	Invalid REALOPTNS value, set to default value
	IERR = 26	Transformer T model GIC data: WRI, WRJ and WRK dc resistances must be specified for T model
	IERR = 27	Info - The Geographic Coordinates of the magnetic dipole axis for study year %% are not defined, used data from year %%
	IERR = 28	gic.dll not loaded or entry point not found
	IERR = 29	Error - All transformers must have vector group specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 30	Invalid Magnitude Scan maximum storm strength
IERR = 31	Error - No substations exist in specified moving box
IERR = 32	Error - Specified moving box center substation does not exist in the case
IERR = 33	Error - No Geoelectric Efield Grid Data provided

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.9. GIC_BRANCH_EFIELD_NN

Use this API to calculate geoelectric efield induced in the network branches from specified geoelectric efield grid data using nearest neigbor (NN) alogorithm.

This API routine was first introduced in release 34.8.0 and 35.1.0.

Batch command syntax:

```
BAT_GIC_BRANCH_EFIELD_NN  SID ALL INTGOPTNS(1)..INTGOPTNS(3)
 REALOPTNS(1)..REALOPTNS(2)
```

Python command syntax:

```
ierr = gic_branch_efield_nn(sid, all, intgoptns, realoptns)
```

Fortran command syntax:

```
CALL GIC_BRANCH_EFIELD_NN(SID, ALL, INTGOPTNS, REALOPTNS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer INTGOPTNS (4)	array of 4 elements specifying Integer Options(input).	
	INTGOPTNS(1)	Number of levels of inter-tie buses to add to study subsystem (alias is tielevels) (0 by default)
	INTGOPTNS(1) = 0	consider only study subsystem buses, no buses from inter-ties
	INTGOPTNS(1) > 0	1 through 5, consider study subsystem buses + these many levels of inter-tie buses and ignore subsystem definition voltage limits for intertie buses added
	INTGOPTNS(1) < 0	-1 through -5, consider study subsystem buses + these many levels of inter-tie buses and consider subsystem definition voltage limits for intertie buses added
	INTGOPTNS(2)	Flag to do branch route segment scan to find efield grid index same as previous route point efield grid index (alias is brn_segscan) (1 by default)
	INTGOPTNS(2) = 0	no branch segment scan, efield grid index at each segment point is used to get segment induced voltage
	INTGOPTNS(2) = 1	perform branch segment scan

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGOPTNS(3)	Flag to merge branch route segments with same efield grid index. This only affects how the branch route induced voltage is reported. (alias is brn_segmerge) (1 by default)
	INTGOPTNS(3) = 0	do not merge segments
	INTGOPTNS(3) = 1	merge segments
	INTGOPTNS(4)	Report Option (alias is rptoptn) (1 by default)
	INTGOPTNS(4) = 0	no report
	INTGOPTNS(4) = 1	summary report
	INTGOPTNS(4) = 2	detailed report
	INTGOPTNS(4) = 3	both summary and detailed reports
Real REALOPTNS (2)	array of 2 elements specifying Real Options(input).	
	REALOPTNS(1)	Branch segment length in km, used to calculate branch induced voltage from Efield Grid, must be >0.0. Branch is subdivided into number of segements (points along the branch route) and NN algorithm finds nearest geoelectric efield grid data point to arrive efield at that branch route point. (alias is brn_segkm) (5.0 by default)
	REALOPTNS(1) = 0.0	Use default
	REALOPTNS(1) > 0	Use specified
	REALOPTNS(2)	Branch subsegment length in km, used to calculate branch induced voltage from Efield Grid, range 0.02 to 1.0 km. When efield value at branch route point changes, that branch segment is further subdivided into smaller segments to arrive at branch route point where NN efield change occurs. (alias is brn_subsegkm) (0.1 by default)
	REALOPTNS(2) = 0.0	Use default
	REALOPTNS(2) > 0	Use specified
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	Invalid SID value or subsystem SID is not defined
	IERR = 2	Invalid ALL value
	IERR = 3	Invalid INTGOPTNS value, set to default
	IERR = 4	Invalid REALOPTNS value, set to default
	IERR = 5	Topology error, bus type code and series element status inconsistencies
	IERR = 6	No Efield Grid Data provided'
	IERR = 7	No substations in the working case (or added from GIC data)
	IERR = 8	No buses in GIC study subsystem

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.10. GIC_EARTH1D_USR_PURG

Use this API to purge GIC User 1D Earth Model data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_EARTH1D_USR_PURG NAME
```

Python command syntax:

```
ierr = gic_earth1d_usr_purg(name)
```

Fortran command syntax:

```
CALL GIC_EARTH1D_USR_PURG(NAME,IERR)
```

Where:

Character*32 NAME	1D Earth Model name. The non-blank alphanumeric identifier assigned to earth model. The name should be unique and different than standard earth model names defined in TPL-007. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.11. GIC_EFIELD_WAVESHAPE

For Transformer Thermal Impact assessement due to geomagnetic disturbance event, set the geoelectric field waveshape as provided in EFLDFILE.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_GIC_EFIELD_WAVESHAPE  EFLDFILE
```

Python command syntax:

```
ierr = gic_efield_waveshape(efldfile)
```

Fortran command syntax:

```
CALL GIC_EFIELD_WAVESHAPE(EFLDFILE, IERR)
```

Where:

Character*260 EFLDFILE	name of the Geoelectric Field Waveshape data points file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 1	Prerequisite requirements for API are not met, GIC(t) calculations not done
	IERR = 2	Error opening GIC Electric Field Waveshape Data file, GIC(t) calculations not done
	IERR = 3	Error reading GIC Electric Field Waveshape Data file, GIC(t) calculations not done

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.12. GIC_EFIELD_WAVESHAPE_REF

For Transformer Thermal Impact assessement due to geomagnetic disturbance event, set the geoelectric field waveshape as NERC TPL-007-2 benhcmark event.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_GIC_EFIELD_WAVESHAPE_REF
```

Python command syntax:

```
ierr = gic_efield_waveshape_ref()
```

Fortran command syntax:

```
CALL GIC_EFIELD_WAVESHAPE_REF(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.13. GIC_EFIELD_WAVESHAPE_SUPP

For Transformer Thermal Impact assessement due to geomagnetic disturbance event, set the geoelectric field waveshape as NERC TPL-007-2 supplemental event.

This API routine was first introduced in release 33.12.0 and 34.4.0.

Batch command syntax:

```
BAT_GIC_EFIELD_WAVESHAPE_SUPP
```

Python command syntax:

```
ierr = gic_efield_waveshape_supp()
```

Fortran command syntax:

```
CALL GIC_EFIELD_WAVESHAPE_SUPP(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.14. GIC_MVARLOSS_SCALING_FACTORS

Specify KFACTORS to determine transformer reactive power losses due to geomagnetic currents (GICs) flow.

This API routine was first introduced in release 32.2.1.

Batch command syntax:

```
BAT_GIC_MVARLOSS_SCALING_FACTORS  KFACTORS(1)..KFACTORS(9)
```

Python command syntax:

```
ierr = gic_mvarloss_scaling_factors(kfactors)
```

Fortran command syntax:

```
CALL GIC_MVARLOSS_SCALING_FACTORS(KFACTORS, IERR)
```

Where:

Real KFACTORS (9)	array of elements specifying Kfactor values. The values are as follows.(input).	
	KFACTORS(1)	Voltage level at which these kfactors are specified
	KFACTORS(2)	kfactor value for Shell form transformers
	KFACTORS(3)	kfactor value for core design - Single Phase (three separate cores) transformers
	KFACTORS(4)	kfactor value for core design - Three Phase 3 legged core form transformers
	KFACTORS(5)	kfactor value for core design - Three Phase 5 legged core form transformers
	KFACTORS(6)	kfactor value for core design - Three Phase 7 legged core form transformers
	KFACTORS(7)	kfactor value for unknown core, voltage level of <=200 kV
	KFACTORS(8)	kfactor value for unknown core, voltage level of <200 kV and <=400 kV
	KFACTORS(9)	kfactor value for unknown core, voltage level of >400 kV
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	DLL not loaded or entry point not found.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.15. GIC_NEW

Use this API to initialize all data records required for GIC analysis. It must be called prior to introducing any GIC data records. It has no effect if GIC data already exists in the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_NEW
```

Python command syntax:

```
ierr = gic_new()
```

Fortran command syntax:

```
CALL GIC_NEW(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	error initializing GIC data

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.16. GIC_PF_OPTIONS

Use this API to specify the Newton-Raphson power flow solution options used in GIC.

This API routine was first introduced in release 34.5.0.

Batch command syntax:

```
BAT_GIC_PF_OPTIONS OPTIONS(1)..OPTIONS(8)
```

Python command syntax:

```
ierr = gic_pf_options(options)
```

Fortran command syntax:

```
CALL GIC_PF_OPTIONS(OPTIONS, IERR)
```

Where:

Integer OPTIONS (8)	array of 8 elements specifying Newton-Raphson power flow solution options(input).	
	OPTIONS(1)	tap adjustment flag (use tap adjustment option setting by default)
	OPTIONS(1) = 0	disable
	OPTIONS(1) = 1	enable stepping adjustment
	OPTIONS(1) = 2	enable direct adjustment
	OPTIONS(2)	area interchange adjustment flag (use area interchange adjustment option setting by default)
	OPTIONS(2) = 0	disable
	OPTIONS(2) = 1	enable using tie line flows only in calculating area interchange
	OPTIONS(2) = 2	enable using tie line flows and loads in calculating area interchange
	OPTIONS(3)	phase shift adjustment flag (use phase shift adjustment option setting by default)
	OPTIONS(3) = 0	disable
	OPTIONS(3) = 1	enable
	OPTIONS(4)	dc tap adjustment flag (use dc tap adjustment option setting by default)
	OPTIONS(4) = 0	disable
	OPTIONS(4) = 1	enable
	OPTIONS(5)	switched shunt adjustment flag (use switched shunt adjustment option setting by default)
	OPTIONS(5) = 0	disable
	OPTIONS(5) = 1	enable
	OPTIONS(5) = 2	enable continuous mode, disable discrete mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6)	flat start flag (0 by default)
	OPTIONS(6) = 0	do not flat start
	OPTIONS(6) = 1	flat start
	OPTIONS(6) = 2	flat start, then estimate voltage magnitudes
	OPTIONS(6) = 3	flat start, then estimate voltage phase angles
	OPTIONS(6) = 4	flat start, then estimate voltage magnitudes and phase angles
	OPTIONS(7)	var limit flag (99 by default)
	OPTIONS(7) = 0	apply var limits immediately
	OPTIONS(7) > 0	apply var limits on iteration n (or sooner if mismatch gets small)
	OPTIONS(7) = -1	ignore var limits
	OPTIONS(8)	non-divergent solution flag (use non-divergent solution option setting by default)
	OPTIONS(8) = 0	disable
	OPTIONS(8) = 1	enable
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid OPTIONS value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.17. GIC_PURG

Use this API to purge all GIC data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_PURG
```

Python command syntax:

```
ierr = gic_purg()
```

Fortran command syntax:

```
CALL GIC_PURG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.18. GIC_READ

Use this API to read GIC Data Text (.gic) File into the working case memory.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_READ RFILE
```

Python command syntax:

```
ierr = gic_read(rfile)
```

Fortran command syntax:

```
CALL GIC_READ(RFILE, IERR)
```

Where:

Character*260 RFILE	GIC Data File name (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	error opening file
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.19. GIC_READ_EFIELD_GRID

Use this API to read Geoelectric Efield Grid Data File into the working case memory.

This API routine was first introduced in release 34.8.0 and 35.1.0.

Batch command syntax:

```
BAT_GIC_READ_EFIELD_GRID FTYP, FNAM
```

Python command syntax:

```
ierr = gic_read_efield_grid(ftyp, fnam)
```

Fortran command syntax:

```
CALL GIC_READ_EFIELD_GRID(FTYP, FNAM, IERR)
```

Where:

Integer FTYP	File format type (input; 0 by default).	
	FTYP = 0	csv format, column order: latitude (deg), longitude (deg), Ee (V/km), En (V/km)
	FTYP = 1	USGS json format [coordinates [lon, lat] in deg, Ex(=En) and Ey(=Ee) in mV/km
Character*260 FNAM	Efield Grid Data File name (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	error opening Efield Grid data file
	IERR = 3	error reading Efield Grid data file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.20. GIC_THERMAL_IMPACT

This API routine is obsolete. It has been replaced by the API routine gic_thermal_impact_1 .

Use this API to calculate Transformer Thermal Impact assessement GIC(t) curve using specified effective Eastward GIC(E) and Nortward GIC(N) for one specific transformer. The geoelectric field waveshape used could be NERC TPL-007-2 Benchmark event, NERC TPL-007-2 Supplemental event or User specified event.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_GIC_THERMAL_IMPACT GICEARG, GICNARG, XMERLBL, GICTFARG
```

Python command syntax:

```
ierr = gic_thermal_impact(gicearg, gicnarg, xmerlbl, gictfarg)
```

Fortran command syntax:

```
CALL GIC_THERMAL_IMPACT(GICEARG, GICNARG, XMERLBL, GICTFARG, IERR)
```

Where:

Real GICEARG	Transfomer effective per phase GIC due to Eastward geoelectic field, GIC(E) due to Ee=1 V/km and En=0, with alpha and beta factors applied (input; 0 by default).	
Real GICNARG	Transfomer effective per phase GIC due to Northward geoelectic field, GIC(N) due to En=1 V/km and Ee=0, with alpha and beta factors applied (input; 0 by default).	
Character*40 XMERLBL	Transfomer Name (input; '' by default).	
Character*260 GICTFARG	Transformer Thermal Analysis GIC(t) CSV file (output) (input; '' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	No or invalid GICEARG value provided
	IERR = 1	No or invalid GICNARG value provided
	IERR = 3	No GMD Event EE(t) and EN(t) data available
	IERR = 4	Error opening GIC(t) File

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.21. GIC_THERMAL_IMPACT_1

Use this API to calculate Transformer Thermal Impact assessement GIC(t) curve using specified effective Eastward GIC(E) and Nortward GIC(N) for one specific transformer. The geoelectric field waveshape used could be NERC TPL-007 Benchmark event, NERC TPL-007 Supplemental event or User specified event.

This API routine was first introduced in release 34.8.0 and 35.1.0.

Batch command syntax:

```
BAT_GIC_THERMAL_IMPACT_1 GICEARG, GICNARG, XMERLBL, GICTFARG IEVT
```

Python command syntax:

```
ierr = gic_thermal_impact_1(gicearg, gicnarg, xmerlbl, gictfarg, ievt)
```

Fortran command syntax:

```
CALL GIC_THERMAL_IMPACT_1(GICEARG, GICNARG, XMERLBL, GICTFARG, IEVT, IERR)
```

Where:

Real GICEARG	Transfomer effective per phase GIC due to Eastward geoelectic field, GIC(E) due to Ee=1 V/km and En=0, with alpha and beta factors applied (input; 0 by default).	
Real GICNARG	Transfomer effective per phase GIC due to Northward geoelectic field, GIC(N) due to En=1 V/km and Ee=0, with alpha and beta factors applied (input; 0 by default).	
Character*40 XMERLBL	Transfomer Name (input; " by default).	
Character*260 GICTFARG	Transformer Thermal Analysis GIC(t) CSV file (output) (input; " by default).	
Integer IEVT	GMD Event Type (input; 0 by default).	
	IEVT = 0	TPL-007 Benchmark or User Specified Reference GMD Event
	IEVT = 1	TPL-007 Supplemental GMD Event
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	No or invalid GICEARG value provided
	IERR = 1	No or invalid GICNARG value provided
	IERR = 3	No GMD Event EE(t) and EN(t) data available
	IERR = 4	Error opening GIC(t) File
	IERR = 5	No or invalid IEVT value provided

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.22. GIC_WRITE

Use this API to write working case GIC Data to text (.gic) file.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_WRITE RFILE
```

Python command syntax:

```
ierr = gic_write(rfile)
```

Fortran command syntax:

```
CALL GIC_WRITE(RFILE, IERR)
```

Where:

Character*260 RFILE	GIC Data File name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	error opening file
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.1.23. GIC_WRITE_STN

Use this API to write working case GIC Data to text (.gic) file version 4 that has substation data.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_GIC_WRITE_STN RFILE
```

Python command syntax:

```
ierr = gic_write_stn(rfile)
```

Fortran command syntax:

```
CALL GIC_WRITE_STN(RFILE, IERR)
```

Where:

Character*260 RFILE	GIC Data File name (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	error opening file
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GIC Operation APIs contains API routines corresponding to the operational functions of the GIC features of PSS®E.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2. GIC Data

6.2.1. GIC_2TDC

Use this API to modify or add new two terminal dc GIC shunt data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_2TDC NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_2tdc(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_2TDC(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	DC line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, two terminal dc shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, two terminal dc is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.2. GIC_2TDC_CHNG

Use this API to modify new two terminal dc GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_2TDC_CHNG NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_2tdc_chng(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_2TDC_CHNG(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	DC line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, two terminal dc shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, two terminal dc is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.3. GIC_2TDC_PURG

Use this API to purge new two terminal dc GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_2TDC_PURG NAME BUS CKT
```

Python command syntax:

```
ierr = gic_2tdc_purg(name,bus,ckt)
```

Fortran command syntax:

```
CALL GIC_2TDC_PURG(NAME,BUS,CKT,IERR)
```

Where:

Character*12 NAME	DC line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.4. GIC_BRN

Use this API to modify or add new GIC branch data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_BRN IBUS JBUS CKT REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = gic_brn(ibus,jbus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_BRN(IBUS,JBUS,CKT,REALAR,IERR)
```

Where:

Integer IBUS	From bus number of the branch as in the working case. (input; no default allowed).	
Integer JBUS	To bus number of the branch as in the working case. (input; no default allowed).	
Character*2 CKT	branch circuit identifier (input; '1' by default).	
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	DC resistance of the branch in ohms/phase. When RBRN is not specified, working case resistance is used to determine RBRN. (alias is rbrn) (0.0 by default)
	REALAR(2)	Real part of total branch GMD-induced geoelectric field in volts. When INDVP is not specified, it is calculated as per GMD event. (alias is indvp) (no default allowed)
	REALAR(3)	Imaginary part of total branch GMD-induced geoelectric field in volts. When INDVP is not specified, it is calculated as per GMD event. (alias is indvq) (no default allowed)
	REALAR(4)	DC resistance in ohms/phase of the line shunt at bus IBUS end of the branch. When RLNSHI=0.0 or not specified, there is no ground path this line shunt in DC network. (alias is rlnshi) (0.0 by default)
	REALAR(5)	DC resistance in ohms/phase of the line shunt at bus JBUS end of the branch. When RLNSHJ=0.0 or not specified, there is no ground path this line shunt in DC network. (alias is rlnshj) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.5. GIC_BRN_CHNG

Use this API to modify new GIC branch data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_BRN_CHNG IBUS JBUS CKT REALAR(1)..REALAR(5)
```

Python command syntax:

```
ierr = gic_brn_chng(ibus,jbus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_BRN_CHNG(IBUS,JBUS,CKT,REALAR,IERR)
```

Where:

Integer IBUS	From bus number of the branch as in the working case. (input; no default allowed).	
Integer JBUS	To bus number of the branch as in the working case. (input; no default allowed).	
Character*2 CKT	branch circuit identifier (input; '1' by default).	
Real REALAR (5)	array of 5 elements specifying (input).	
	REALAR(1)	DC resistance of the branch in ohms/phase. When RBRN is not specified, working case resistance is used to determine RBRN. (alias is rbrn) (0.0 by default)
	REALAR(2)	Real part of total branch GMD-induced geoelectric field in volts. When INDVP is not specified, it is calculated as per GMD event. (alias is indvp) (no default allowed)
	REALAR(3)	Imaginary part of total branch GMD-induced geoelectric field in volts. When INDVP is not specified, it is calculated as per GMD event. (alias is indvq) (no default allowed)
	REALAR(4)	DC resistance in ohms/phase of the line shunt at bus IBUS end of the branch. When RLNSHI=0.0 or not specified, there is no ground path this line shunt in DC network. (alias is rlnshi) (0.0 by default)
	REALAR(5)	DC resistance in ohms/phase of the line shunt at bus JBUS end of the branch. When RLNSHJ=0.0 or not specified, there is no ground path this line shunt in DC network. (alias is rlnshj) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.6. GIC_BRN_PURG

Use this API to purge GIC branch data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_BRN_PURG IBUS JBUS CKT
```

Python command syntax:

```
ierr = gic_brn_purg(ibus,jbus,ckt)
```

Fortran command syntax:

```
CALL GIC_BRN_PURG(IBUS,JBUS,CKT,IERR)
```

Where:

Integer IBUS	From bus number of the branch as in the working case. (input; no default allowed).	
Integer JBUS	To bus number of the branch as in the working case. (input; no default allowed).	
Character*2 CKT	branch circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.7. GIC_EARTH1D_USR

Use this API to modify or add new GIC User 1D Earth Model data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_EARTH1D_USR NAME DESC REALAR(1) REALAR(2) LAYERS RHO(1)..RHO(N)
 THICKNESS(1)..THICKNESS(N)
```

Python command syntax:

```
ierr = gic_earth1d_usr(name,desc,realar,layers,rho,thickness)
```

Fortran command syntax:

```
CALL GIC_EARTH1D_USR(NAME,DESC,REALAR,LAYERS,RHO,THICKNESS,IERR)
```

Where:

Character*32 NAME	1D Earth Model name. The non-blank alphanumeric identifier assigned to earth model. The name should be unique and different than standard earth model names defined in TPL-007. (input; no default allowed).	
Character*32 DESC	Earth Model description. It is used for information purpose only.. (input; blank).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	Benchmark event scaling factor (alias is beta) (1.0 by default)
	REALAR(2)	Supplemental event scaling factor (alias is betas) (1.0 by default)
Integer LAYERS	number of layers of earth model rho and thickness (input; no default allowed).	
Real RHO (layers)	array of 'layers' values specifying earth model layer resistivity in ohm-m. All values must be > 0. (input; no default allowed).	
Real THICKNESS (layers)	array of 'layers' values specifying earth model layer thickness in km. All values must be > 0 except last layer. The last layer thickness is infinity, specified as any number < 0. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.8. GIC_EARTH1D_USR_CHNG

Use this API to modify GIC User 1D Earth Model data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_EARTH1D_USR_CHNG NAME DESC REALAR(1) REALAR(2) LAYERS
 RHO(1)..RHO(N) THICKNESS(1)..THICKNESS(N)
```

Python command syntax:

```
ierr = gic_earth1d_usr_chng(name,desc,realar,layers,rho,thickness)
```

Fortran command syntax:

```
CALL GIC_EARTH1D_USR_CHNG(NAME,DESC,REALAR,LAYERS,RHO,THICKNESS,IERR)
```

Where:

Character*32 NAME	1D Earth Model name. The non-blank alphanumeric identifier assigned to earth model. The name should be unique and different than standard earth model names defined in TPL-007. (input; no default allowed).	
Character*32 DESC	Earth Model description. It is used for information purpose only.. (input; blank).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	Benchmark event scaling factor (alias is beta) (1.0 by default)
	REALAR(2)	Supplemental event scaling factor (alias is betas) (1.0 by default)
Integer LAYERS	number of layers of earth model rho and thickness (input; no default allowed).	
Real RHO (layers)	array of 'layers' values specifying earth model layer resistivity in ohm-m. All values must be > 0. (input; no default allowed).	
Real THICKNESS (layers)	array of 'layers' values specifying earth model layer thickness in km. All values must be > 0 except last layer. The last layer thickness is infinity, specified as any number < 0. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.9. GIC_FACTS

Use this API to modify or add new FACTS device GIC shunt data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_FACTS NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_facts(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_FACTS(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	FACTS device name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, FACTS device shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, FACTS device is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.10. GIC_FACTS_CHNG

Use this API to modify new FACTS device GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_FACTS_CHNG NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_facts_chng(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_FACTS_CHNG(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	FACTS device name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, FACTS device shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, FACTS device is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.11. GIC_FACTS_PURG

Use this API to purge new FACTS device GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_FACTS_PURG NAME BUS CKT
```

Python command syntax:

```
ierr = gic_facts_purg(name,bus,ckt)
```

Fortran command syntax:

```
CALL GIC_FACTS_PURG(NAME,BUS,CKT,IERR)
```

Where:

Character*12 NAME	FACTS device name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.12. GIC_FXSH

Use this API to modify or add new fixed shunt GIC shunt data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_FXSH BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_fxsh(bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_FXSH(BUS,CKT,REALAR,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, fixed shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, fixed shunt is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.13. GIC_FXSH_CHNG

Use this API to modify fixed shunt GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_FXSH_CHNG BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_fxsh_chng(bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_FXSH_CHNG(BUS,CKT,REALAR,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, fixed shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, fixed shunt is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.14. GIC_FXSH_PURG

Use this API to purge fixed shunt GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_FXSH_PURG BUS CKT
```

Python command syntax:

```
ierr = gic_fxsh_purg(bus,ckt)
```

Fortran command syntax:

```
CALL GIC_FXSH_PURG(BUS,CKT,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.15. GIC_LOAD

Use this API to modify or add new load GIC shunt data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_LOAD BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_load(bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_LOAD(BUS,CKT,REALAR,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, fixed shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, fixed shunt is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.16. GIC_LOAD_CHNG

Use this API to modify load GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_LOAD_CHNG BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_load_chng(bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_LOAD_CHNG(BUS,CKT,REALAR,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, fixed shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, fixed shunt is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.17. GIC_LOAD_PURG

Use this API to purge load GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_LOAD_PURG BUS CKT
```

Python command syntax:

```
ierr = gic_load_purg(bus,ckt)
```

Fortran command syntax:

```
CALL GIC_LOAD_PURG(BUS,CKT,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.18. GIC_MTDC

Use this API to modify or add new multi terminal dc GIC shunt data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_MTDC NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_mtdc(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_MTDC(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	mtdc line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, multi terminal dc shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, multi terminal dc is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.19. GIC_MTDC_CHNG

Use this API to modify new multi terminal dc GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_MTDC_CHNG NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_mtdc_chng(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_MTDC_CHNG(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	mtdc line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, multi terminal dc shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, multi terminal dc is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.20. GIC_MTDC_PURG

Use this API to purge new multi terminal dc GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_MTDC_PURG NAME BUS CKT
```

Python command syntax:

```
ierr = gic_mtdc_purg(name,bus,ckt)
```

Fortran command syntax:

```
CALL GIC_MTDC_PURG(NAME,BUS,CKT,IERR)
```

Where:

Character*12 NAME	mtdc line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.21. GIC_SUBSTN

Use this API to modify or add new GIC sub station data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_SUBSTN SUBNUM REALAR(1) CHARAR(1) EARTHMDL
```

Python command syntax:

```
ierr = gic_substn(subnum,realar,charar,earthmdl)
```

Fortran command syntax:

```
CALL GIC_SUBSTN(SUBNUM,REALAR,CHARAR,EARTHMDL,IERR)
```

Where:

Integer SUBNUM	sub station number (input; no default allowed).	
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Substation grounding DC resistance in ohms. For ungrounded substation RG <=0 or RG >=99 (alias is rg) (0.1 by default)
Character*40 CHARAR (1)	array of 1 elements specifying (input).	
	CHARAR(1)	Method used to specify RG value, RGFLAG may be up to 40 characters. It is used for informational purposes only. (alias is rgflag)
Character*32 EARTHMDL	sub station earth conductivity model name (input; '' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.22. GIC_SUBSTN_CHNG

Use this API to modify GIC sub station data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_SUBSTN_CHNG SUBNUM REALAR(1) CHARAR(1) EARTHMDL
```

Python command syntax:

```
ierr = gic_substn_chng(subnum,realar,charar,earthmdl)
```

Fortran command syntax:

```
CALL GIC_SUBSTN_CHNG(SUBNUM,REALAR,CHARAR,EARTHMDL,IERR)
```

Where:

Integer SUBNUM	sub station number (input; no default allowed).	
Real REALAR (1)	array of 1 elements specifying (input).	
	REALAR(1)	Substation grounding DC resistance in ohms. For ungrounded substation RG <=0 or RG >=99 (alias is rg) (0.1 by default)
Character*40 CHARAR (1)	array of 1 elements specifying (input).	
	CHARAR(1)	Method used to specify RG value, RGFLAG may be up to 40 characters. It is used for informational purposes only. (alias is rgflag)
Character*32 EARTHMDL	sub station earth conductivity model name (input; '' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.23. GIC_SUBSTN_PURG

Use this API to purge GIC sub station data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_SUBSTN_PURG SUBNUM
```

Python command syntax:

```
ierr = gic_substn_purg(subnum)
```

Fortran command syntax:

```
CALL GIC_SUBSTN_PURG(SUBNUM,IERR)
```

Where:

Integer SUBNUM	sub station number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.24. GIC_SWSH

Use this API to modify or add new GIC switched shunt data to the working case. Use this when switched shunt is described by the bus number and its circuit identifier.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_SWSH BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_swsh(bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_SWSH(BUS,CKT,REALAR,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, switched shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, switched shunt is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.25. GIC_SWSH_CHNG

Use this API to modify GIC switched shunt data from the working case. Use this when switched shunt is described by the bus number and its circuit identifier.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_SWSH_CHNG BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_swsh_chng(bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_SWSH_CHNG(BUS,CKT,REALAR,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, switched shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, switched shunt is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.26. GIC_SWSH_PURG

Use this API to purge GIC switched shunt data from the working case. Use this when switched shunt is described by the bus number and its circuit identifier.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_SWSH_PURG BUS CKT
```

Python command syntax:

```
ierr = gic_swsh_purg(bus,ckt)
```

Fortran command syntax:

```
CALL GIC_SWSH_PURG(BUS,CKT,IERR)
```

Where:

Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.27. GIC_TRN

Use this API to modify or add new GIC transformer data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_TRN IBUS JBUS KBUS CKT INTGAR(1)..INTGAR(5) REALAR(1)..REALAR(7)
 VECGRP
```

Python command syntax:

```
ierr = gic_trn(ibus,jbus,kbus,ckt,intgar,realar,vecgrp)
```

Fortran command syntax:

```
CALL GIC_TRN(IBUS,JBUS,KBUS,CKT,INTGAR,REALAR,VECGRP,IERR)
```

Where:

Integer IBUS	The bus number of the bus to which Winding 1 is connected. It must be same Winding 1 bus of the same transformer in the working case. (input; no default allowed).	
Integer JBUS	The bus number of the bus to which Winding 2 is connected. It must be same Winding 2 bus of the same transformer in the working case. (input; no default allowed).	
Integer KBUS	The bus number of the bus to which Winding 3 is connected. It must be same Winding 3 bus of the same transformer in the working case. KBUS=0 for two winding transformers. No default allowed for three winding transformers. (input; 0 by default).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	GIC blocking device in neutral of Winding 1. For an auto-transformers, if either BDI=1 or BDJ=1, that auto-transformer is treated as if it has GIC blocking device present. (alias is bdi) (0 by default)
	INTGAR(1) = 0	no GIC blocking device present
	INTGAR(1) = 1	GIC blocking device present
	INTGAR(2)	GIC blocking device in neutral of Winding 2. For an auto-transformers, if either BDI=1 or BDJ=1, that auto-transformer is treated as if it has GIC blocking device present. (alias is bdj) (0 by default)
	INTGAR(2) = 0	no GIC blocking device present
	INTGAR(2) = 1	GIC blocking device present
	INTGAR(3)	GIC blocking device in neutral of Winding 3. (alias is bdk) (0 by default)
	INTGAR(3) = 0	no GIC blocking device present
	INTGAR(3) = 1	GIC blocking device present

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(4)	Number of cores in transformer core design. This information is used to calculate transformer reactive power loss from GIC flowing its winding. (alias is core) (0 by default)
	INTGAR(4) = -1	three phase shell form
	INTGAR(4) = 0	unknown core design
	INTGAR(4) = 1	single phase core
	INTGAR(4) = 3	three phase 3-legged core form
	INTGAR(4) = 5	three phase 5-legged core form
	INTGAR(5)	Model transformer as T-model in GIC DC Network (alias is tmodel) (0 by default)
	INTGAR(5) = 0	two/three/auto transformer model as defined by its vector group
	INTGAR(5) = 1	transformer as T-model in DC network
Real REALAR (7)	array of 7 elements specifying (input).	
	REALAR(1)	DC resistance of Winding 1 in ohms/phase. When WRI is not specified, working case resistance is used to determine WRI. (alias is wri) (0.0 by default)
	REALAR(2)	DC resistance of Winding 2 in ohms/phase. When WRJ is not specified, working case resistance is used to determine WRJ. (alias is wrj) (0.0 by default)
	REALAR(3)	DC resistance of Winding 2 in ohms/phase. When WRK is not specified, working case resistance is used to determine WRK. (alias is wrk) (0.0 by default)
	REALAR(4)	Factor to calculate transformer reactive power loss from the GIC flow in its winding (MVAR/AMP) (alias is kfactor) (0.0 by default)
	REALAR(5)	Winding 1 grounding DC resistance in ohms. When GRDWRI=0.0, there is no grounding resistance and winding is solidly grounded. (alias is grdwri) (0.0 by default)
	REALAR(6)	Winding 2 grounding DC resistance in ohms. When GRDWRJ=0.0, there is no grounding resistance and winding is solidly grounded. (alias is grdwrj) (0.0 by default)
	REALAR(7)	Winding 3 grounding DC resistance in ohms. When GRDWRK=0.0, there is no grounding resistance and winding is solidly grounded. (alias is grdwrk) (0.0 by default)
Character*12 VECGRP	Alphanumeric identifier specifying vector group based on transformer winding connections and phase angles. When vector group is blank, its value is taken form power flow data. (input; " by default).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no errors occurred
IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.28. GIC_TRN_CHNG

Use this API to modify GIC transformer data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_TRN_CHNG IBUS JBUS KBUS CKT INTGAR(1)..INTGAR(5)
 REALAR(1)..REALAR(7) VECGRP
```

Python command syntax:

```
ierr = gic_trn_chng(ibus,jbus,kbus,ckt,intgar,realar,vecgrp)
```

Fortran command syntax:

```
CALL GIC_TRN_CHNG(IBUS,JBUS,KBUS,CKT,INTGAR,REALAR,VECGRP,IERR)
```

Where:

Integer IBUS	The bus number of the bus to which Winding 1 is connected. It must be same Winding 1 bus of the same transformer in the working case. (input; no default allowed).	
Integer JBUS	The bus number of the bus to which Winding 2 is connected. It must be same Winding 2 bus of the same transformer in the working case. (input; no default allowed).	
Integer KBUS	The bus number of the bus to which Winding 3 is connected. It must be same Winding 3 bus of the same transformer in the working case. KBUS=0 for two winding transformers. No default allowed for three winding transformers. (input; 0 by default).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer INTGAR (5)	array of 5 elements specifying (input).	
	INTGAR(1)	GIC blocking device in neutral of Winding 1. For an auto-transformers, if either BDI=1 or BDJ=1, that auto-transformer is treated as if it has GIC blocking device present. (alias is bdi) (0 by default)
	INTGAR(1) = 0	no GIC blocking device present
	INTGAR(1) = 1	GIC blocking device present
	INTGAR(2)	GIC blocking device in neutral of Winding 2. For an auto-transformers, if either BDI=1 or BDJ=1, that auto-transformer is treated as if it has GIC blocking device present. (alias is bdj) (0 by default)
	INTGAR(2) = 0	no GIC blocking device present
	INTGAR(2) = 1	GIC blocking device present
	INTGAR(3)	GIC blocking device in neutral of Winding 3. (alias is bdk) (0 by default)
	INTGAR(3) = 0	no GIC blocking device present
	INTGAR(3) = 1	GIC blocking device present

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(4)	Number of cores in transformer core design. This information is used to calculate transformer reactive power loss from GIC flowing its winding. (alias is core) (0 by default)
	INTGAR(4) = -1	three phase shell form
	INTGAR(4) = 0	unknown core design
	INTGAR(4) = 1	single phase core
	INTGAR(4) = 3	three phase 3-legged core form
	INTGAR(4) = 5	three phase 5-legged core form
	INTGAR(5)	Model transformer as T-model in GIC DC Network (alias is tmodel) (0 by default)
	INTGAR(5) = 0	two/three/auto transformer model as defined by its vector group
	INTGAR(5) = 1	transformer as T-model in DC network
Real REALAR (7)	array of 7 elements specifying (input).	
	REALAR(1)	DC resistance of Winding 1 in ohms/phase. When WRI is not specified, working case resistance is used to determine WRI. (alias is wri) (0.0 by default)
	REALAR(2)	DC resistance of Winding 2 in ohms/phase. When WRJ is not specified, working case resistance is used to determine WRJ. (alias is wrj) (0.0 by default)
	REALAR(3)	DC resistance of Winding 2 in ohms/phase. When WRK is not specified, working case resistance is used to determine WRK. (alias is wrk) (0.0 by default)
	REALAR(4)	Factor to calculate transformer reactive power loss from the GIC flow in its winding (MVAR/AMP) (alias is kfactor) (0.0 by default)
	REALAR(5)	Winding 1 grounding DC resistance in ohms. When GRDWRI=0.0, there is no grounding resistance and winding is solidly grounded. (alias is grdwri) (0.0 by default)
	REALAR(6)	Winding 2 grounding DC resistance in ohms. When GRDWRJ=0.0, there is no grounding resistance and winding is solidly grounded. (alias is grdwrj) (0.0 by default)
	REALAR(7)	Winding 3 grounding DC resistance in ohms. When GRDWRK=0.0, there is no grounding resistance and winding is solidly grounded. (alias is grdwrk) (0.0 by default)
Character*12 VECGRP	Alphanumeric identifier specifying vector group based on transformer winding connections and phase angles. When vector group is blank, its value is taken form power flow data. (input; " by default).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no errors occurred
IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.29. GIC_TRN_PURG

Use this API to purge GIC transformer data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_TRN_PURG IBUS JBUS KBUS CKT
```

Python command syntax:

```
ierr = gic_trn_purg(ibus,jbus,kbus,ckt)
```

Fortran command syntax:

```
CALL GIC_TRN_PURG(IBUS,JBUS,KBUS,CKT,IERR)
```

Where:

Integer IBUS	The bus number of the bus to which Winding 1 is connected. It must be same Winding 1 bus of the same transformer in the working case. (input; no default allowed).	
Integer JBUS	The bus number of the bus to which Winding 2 is connected. It must be same Winding 2 bus of the same transformer in the working case. (input; no default allowed).	
Integer KBUS	The bus number of the bus to which Winding 3 is connected. It must be same Winding 3 bus of the same transformer in the working case. KBUS=0 for two winding transformers. No default allowed for three winding transformers. (input; 0 by default).	
Character*2 CKT	transformer circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.30. GIC_VSCDC

Use this API to modify or add new vscdc GIC shunt data to the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_VSCDC NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_vscdc(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_VSCDC(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	vscdc line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, vscdc shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, vscdc is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.31. GIC_VSCDC_CHNG

Use this API to modify new vscdc GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_VSCDC_CHNG NAME BUS CKT REALAR(1) REALAR(2)
```

Python command syntax:

```
ierr = gic_vscdc_chng(name,bus,ckt,realar)
```

Fortran command syntax:

```
CALL GIC_VSCDC_CHNG(NAME,BUS,CKT,REALAR,IERR)
```

Where:

Character*12 NAME	vscdc line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	DC resistance in ohms/phase. R must be > 0. When R is not specified or R=0, vscdc shunt is ignored in DC network. (alias is r) (0.0 by default)
	REALAR(2)	Grounding DC resistance in ohms/phase. When RG is not specified or RG=0, vscdc is treated as solidly grounded when part of DC network. (alias is rg) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

6.2.32. GIC_VSCDC_PURG

Use this API to purge new vscdc GIC shunt data from the working case.

This API routine was first introduced in release 34.8.0.

Batch command syntax:

```
BAT_GIC_VSCDC_PURG NAME BUS CKT
```

Python command syntax:

```
ierr = gic_vscdc_purg(name,bus,ckt)
```

Fortran command syntax:

```
CALL GIC_VSCDC_PURG(NAME,BUS,CKT,IERR)
```

Where:

Character*12 NAME	vscdc line name, must exist in the working case (input; no default allowed).	
Integer BUS	base case bus number, must exist in the working case (input; no default allowed).	
Character*2 CKT	GIC DC shunt circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GIC Data Changing APIs contains API routines specific to adding, modifying, and deleting GIC data.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 7
Time Series Power Flow

This chapter describes the API routines used in PSS®E Time Series Power Flow.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.1. Time Series Power Flow Operation

7.1.1. IMPOSEPROFILE

Use this API to impose generation/load at the given time point on profile curves.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_IMPOSEPROFILE PROFILENAME OPTIONS(1) OPTIONS(2)  CHAARG DFAXFILE
 INFILE
```

Python command syntax:

```
ierr = imposeprofile(profilename,options,chaarg, dfaxfile, infile)
```

Fortran command syntax:

```
CALL IMPOSEPROFILE(PROFILENAME, OPTIONS,CHAARG, DFAXFILE, INFILE, IERR)
```

Where:

integer PROFILENAME	Is the name of the profile (input).	
Integer OPTIONS (2)	array of elements specifying integer options(input).	
	OPTIONS(1)	time point (1 by default)
	OPTIONS(2)	generation dispatch flag
	OPTIONS(2) = 0	disable generation dispatch
	OPTIONS(2) = 1	enable generation dispatch
character*32 CHAARG	generation dispatch system (input; blank by default).	
character*260 DFAXFILE	name of the Distribution Factor Data file (input; no default allowed). (input; blank by default).	
character*260 INFILE	name of the Unit Inertia and Governor Data File (input; blank by default). (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid OPTIONS
	IERR = 2	Generators are converted
	IERR = 3	Topology error: bus(es) with bus type code and series element status inconsistencies
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	No profile curve sets defined
	IERR = 6	Profile curve set not found
	IERR = 7	No profile curves defined in profile curve set
	IERR = 8	Time point not found in the profile curve set
	IERR = 9	Profiles not defined
	IERR = 10	Dispatch system not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 11	Too many islands in base case
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 52	Error opening Distribution Factor Data File
IERR = 54	Error opening Unit Inertia and Governor Data File
IERR = 56	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.1.2. TSPF

Use this API to run the time series power flow solution.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF PROFILE STARTTIME ENDTIME TOL OPTACC(1)..OPTACC(11) LABEL DFXFILE
 ACCFILE INLFILE ZIPFILE
```

Python command syntax:

```
ierr = tspf(profile, starttime, endtime, tol, optacc, label, dfxfile,
 accfile, inlfile, zipfile)
```

Fortran command syntax:

```
CALL TSPFAPI(PROFILE,STARTTIME, ENDTIME, TOL, OPTACC, LABEL, DFXFILE,
 ACCFILE, INLFILE, ZIPFILE, IERR)
```

Where:

integer PROFILE	Profile Name. (input; no default allowed).
Integer STARTTIME	Start time point. when start time point is less than 0, all time points are simulated (input; -1, by default).
Integer ENDTIME	end time point. when end time is less than start time, only start time point is simulated (input; -1, by default).
Real TOL	Is the mismatch tolerance. (input; Newton solution convergence tolerance, TOLN, by default).
Integer OPTACC (11)	array of elements specifying solution options. The values are as follows:(input).

OPTACC(1)	tap adjustment flag (tap adjustment option setting by default)
OPTACC(1) = 0	disable
OPTACC(1) = 1	enable stepping adjustment
OPTACC(1) = 2	enable direct adjustment
OPTACC(2)	area interchange adjustment flag (area interchange adjustment option setting by default)
OPTACC(2) = 0	disable
OPTACC(2) = 1	enable using tie line flows only in calculating area interchange
OPTACC(2) = 2	enable using tie line flows and loads in calculating area interchange
OPTACC(3)	phase shift adjustment flag (phase shift adjustment option setting by default)
OPTACC(3) = 0	disable
OPTACC(3) = 1	enable
OPTACC(4)	dc tap adjustment flag (dc tap adjustment option setting by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OPTACC(4) = 0	disable
OPTACC(4) = 1	enable
OPTACC(5)	switched shunt adjustment flag (switched shunt adjustment option setting by default)
OPTACC(5) = 0	disable
OPTACC(5) = 1	enable
OPTACC(5) = 2	enable continuous mode, disable discrete mode
OPTACC(6)	solution method flag (0 by default)
OPTACC(6) = 0	FDNS
OPTACC(6) = 1	FNSL
OPTACC(6) = 2	optimized FDNS
OPTACC(7)	non-divergent solution flag (non-divergent solution option setting by default)
OPTACC(7) = 0	disable
OPTACC(7) = 1	enable
OPTACC(8)	induction motor treatment flag (applied when an induction motor fails to solve due to low terminal bus voltage)
OPTACC(8) = 0	stall
OPTACC(8) = 1	trip
OPTACC(9)	induction machine failure flag (0 by default)
OPTACC(9) = 0	treat contingency as non-converged if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(9) = 1	treat contingency as solved if it converges, even if any induction machines are placed in the "stalled" or "tripped" state
OPTACC(10)	dispatch mode (0 by default)
OPTACC(10) = 0	disable
OPTACC(10) = 1	subsystem machines (reserve)
OPTACC(10) = 2	subsystem machines (pmax)
OPTACC(10) = 3	subsystem machines (inertia)
OPTACC(10) = 4	subsystem machines (governor droop)
OPTACC(11)	ZIP archive flag (0 by default)
OPTACC(11) = 0	do not write a ZIP archive file
OPTACC(11) = 1	write a ZIP archive using the file specified as ZIPFILE

Character*32 LABEL	Is the name of the generation dispatch subsystem (blank by default, no default allowed if OPTIONS(10) is not 0). (input).
Character*260 DFXFILE	Is the name of the Distribution Factor Data file (input; no default allowed).
Character*260 ACCFILE	Is the name of the Contingency Solution Output file (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 INLFILE	Is the name of the Unit Inertia and Governor Data File (input; blank by default).
Character*260 ZIPFILE	Is the name of the ZIP Archive Output File (input; blank by default).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	invalid TOL value
IERR = 2	invalid OPTIONS value
IERR = 3	generators are converted
IERR = 4	generation dispatch subsystem is not defined
IERR = 5	buses with bus type code and series element status inconsistencies
IERR = 6	buses in island(s) without a swing bus; use activity TREE
IERR = 7	too many islands in base case
IERR = 8	no Distribution Factor Data File specified
IERR = 9	no TSPF Output File specified
IERR = 10	no ZIP Archive Output File specified
IERR = 11	No profile curve sets defined
IERR = 12	Profile curve set not found
IERR = 13	No profile curves defined in profile curve set
IERR = 14	Time point not found in the profile curve set
IERR = 15	No profile data defined
IERR = 21	file DFXFILE is not in the form of a PSSE 25 or later DFAX file; run DFAX
IERR = 22	monitored elements exceed limit when adding multi-section line members
IERR = 51	error opening TSPF Solution Output File
IERR = 52	error opening Distribution Factor Data File
IERR = 54	error opening Unit Inertia and Governor Data File
IERR = 55	error opening ZIP Archive Output File
IERR = 56	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.1.3. TSPF_OUTPUT_PMATRIX

Use this API to output the profile curves within one profile set in the matrix format.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_OUTPUT_PMATRIX PROFILE OPTACC(1)..OPTACC(2)
```

Python command syntax:

```
ierr = tspf_output_pmatrix(profile,optacc)
```

Fortran command syntax:

```
CALL TSPF_OUTPUT_PMATRIX(PROFILE, OPTACC, IERR)
```

Where:

integer PROFILE	Profile Name. (input; no default allowed).	
Integer OPTACC (2)	array of elements specifying profile output options(input).	
	OPTACC(1)	Start time point. when start time point is less than 0, all time points are simulated (-1, by default)
	OPTACC(2)	end time point. when end time is less than start time, only start time point is simulated (1 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	No profile curve sets defined
	IERR = 2	Profile curve set not found
	IERR = 3	No profile curves defined in profile curve set
	IERR = 4	Time point not found in the profile curve set
	IERR = 56	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.1.4. TSPF_OUTPUT_PMATRIX_2

Use this API to output the profile curves within one profile set in the matrix format.

This API routine was first introduced in release 35.4.2.

Batch command syntax:

```
BAT_TSPF_OUTPUT_PMATRIX_2 PROFILE OPTACC(1)..OPTACC(3)
```

Python command syntax:

```
ierr = tspf_output_pmatrix_2(profile,optacc)
```

Fortran command syntax:

```
CALL TSPF_OUTPUT_PMATRIX_2(PROFILE, OPTACC, IERR)
```

Where:

integer PROFILE	Profile Name. (input; no default allowed).	
Integer OPTACC (3)	array of elements specifying profile output options(input).	
	OPTACC(1)	Start time point. when start time point is less than 0, all time points are simulated (-1, by default)
	OPTACC(2)	end time point. when end time is less than start time, only start time point is simulated (1 by default)
	OPTACC(3)	profile curve selection (0 by default)
	OPTACC(3) = 0	report all profile curves in the given profile set
	OPTACC(3) = 1	report profile curves assigned to elements in the given profile set
	OPTACC(3) = 2	report profile curves assigned to elements with non-schedule mode in the given profile set
	OPTACC(3) = 3	report profile curves assigned to elements with schedule mode in the given profile set
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	No profile curve sets defined
	IERR = 2	Profile curve set not found
	IERR = 3	No profile curves defined in profile curve set
	IERR = 4	Time point not found in the profile curve set
	IERR = 56	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Time Series Power Flow Operation APIs contains API routines corresponding to the operational functions of the Time Series Power Flow features of PSS®E.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2. Time Series Power Flow Data

7.2.1. PURGE_TSPF_CHANNEL

Use this API to delete all Time Series Power Flow profiles for an element.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PURGE_TSPF_CHANNEL ELEMENT FIELD IERR
```

Python command syntax:

```
ierr = purge_tspf_channel(element, field)
```

Fortran command syntax:

```
CALL PURGE_TSPF_CHANNEL(ELEMENT, FIELD, IERR)
```

Where:

Character*160 ELEMENT	Name of the element that profiles will be deleted. (input; no default allowed).	
Character*32 FIELD	data quantity to be deleted (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Error while deleting profiles

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.2. PURGE_TSPF_CHANNEL_ALL

Use this API to remover all time series power flow channels for a selected element.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PURGE_TSPF_CHANNEL_ALL ELEMENT
```

Python command syntax:

```
ierr = purg_tspf_channel_all(element)
```

Fortran command syntax:

```
CALL PURGE_TSPF_CHANNEL_ALL(ELEMENT,IERR)
```

Where:

Character*32 ELEMENT	element to be monitored (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 1	element not found
	IERR = 2	Unrecognized data type of the element
	IERR = 3	element and data type not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.3. PURGE_TSPF_PROFILE

Use this API to delete Time Series Power Flow profile by profile name.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PURGE_TSPF_PROFILE NAME IERR
```

Python command syntax:

```
ierr = purge_tspf_profile(name)
```

Fortran command syntax:

```
CALL PURGE_TSPF_PROFILE(NAME,IERR)
```

Where:

Character*160 NAME	Unique name of the profile data (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Error while deleting profile

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.4. PURGE_TSPF_PROFILECURVE

Use this API to delete Time Series Power Flow profile curve by curvesetid and curvecol and index.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PURGE_TSPF_PROFILECRUVE CURVESETID CURVECOL IERR
```

Python command syntax:

```
ierr = purge_tspf_profilecurve(curvesetid,curvecol)
```

Fortran command syntax:

```
CALL PURGE_TSPF_PROFILECURVE(CURVESETID,CURVECOL,IERR)
```

Where:

Integer CURVESETID	Profile Curve Number (input; no default allowed).	
Integer CURVECOL	Profile Curve Collumn (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	The profile curve not found
	IERR = 2	The profile curve not found
	IERR = 3	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.5. PURGE_TSPF_PROFILECURVESET

Use this API to delete Time Series Power Flow profile curve set by profile curvesetid.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PURGE_TSPF_PROFILECURVESET CURVESETID IERR
```

Python command syntax:

```
ierr = purge_tspf_profilecurveset(curvesetid)
```

Fortran command syntax:

```
CALL PURGE_TSPF_PROFILECURVESET(CURVESETID,IERR)
```

Where:

Integer CURVESETID	Profile Curve Number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Error while deleting profile curve

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.6. PURGE_TSPF_PROFILEDATA

Use this API to delete Time Series Power Flow profile data by curvesetid and curvecol and index.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PURGE_TSPF_PROFILEDATA CURVESETID CURVECOL INDEX IERR
```

Python command syntax:

```
ierr = purge_tspf_profiledata(curvesetid,curvecol, index)
```

Fortran command syntax:

```
CALL PURGE_TSPF_PROFILEDATA(CURVESETID,CURVECOL,INDEX,IERR)
```

Where:

Integer CURVESETID	Profile Curve Number (input; no default allowed).	
Integer CURVECOL	Profile Curve Collumn (input; no default allowed).	
Integer INDEX	Time point index (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Time point of the curve not found
	IERR = 2	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.7. TSPF_CHANNEL_CHNG

Use this API to select time series power flow Cchannels.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_CHANNEL_CHNG ELEMENT FIELD OPTION
```

Python command syntax:

```
ierr = tspf_channel_chng(element,field,option)
```

Fortran command syntax:

```
CALL TSPF_CHANNEL_CHNG(ELEMENT,FIELD,OPTION,IERR)
```

Where:

Character*32 ELEMENT	element to be monitored (input; no default allowed).	
Character*32 FIELD	data quantity to be monitored (input; no default allowed).	
Integer OPTION	flag to skip the monitored element (input; 0 by default).	
Integer IERR	error code (output).	
	IERR = 1	element not found
	IERR = 2	Unrecognized data type of the element
	IERR = 3	element exists
	IERR = 4	status is wrong
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.8. TSPF_CHANNEL_DATA

Use this API to select time series power flow Cchannels.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_CHANNEL_DATA ELEMENT FIELD OPTION
```

Python command syntax:

```
ierr = tspf_channel_data(element,field,option)
```

Fortran command syntax:

```
CALL TSPF_CHANNEL_DATA(ELEMENT,FIELD,OPTION,IERR)
```

Where:

Character*32 ELEMENT	element to be monitored (input; no default allowed).	
Character*32 FIELD	data quantity to be monitored (input; no default allowed).	
Integer OPTION	flag to skip the monitored element (input; 0 by default).	
Integer IERR	error code (output).	
	IERR = 1	element not found
	IERR = 2	Unrecognized data type of the element
	IERR = 3	element table is full
	IERR = 4	status is wrong
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.9. TSPF_PROFILECURVESET_CHNG

Use this API routine to modify the data of an existing Time Series Power Flow profile curve set.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_PROFILECURVESET_CHNG CURVESETID INTGAR(1)..INTGAR(2) INTMULT
 START
```

Python command syntax:

```
ierr = tspf_profilecurveset_chng(curvesetid, intgar, intmult, start)
```

Fortran command syntax:

```
CALL TSPF_PROFILECURVESET_CHNG(CURVESETID, INTGAR, INTMULT, START, IERR)
```

Where:

Integer CURVESETID	Profile Curve Set Number (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	time step (1 by default)
	INTGAR(2)	time interval (5 by default)
	INTGAR(2) = 1	year
	INTGAR(2) = 2	month
	INTGAR(2) = 3	week
	INTGAR(2) = 4	day
	INTGAR(2) = 5	hour
	INTGAR(2) = 6	minute
	INTGAR(2) = 7	second
Real INTMULT	Time Interval Multiplier (input; 1.0).	
Character*160 START	Start time (input; "2001-01-01 00:00:00").	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Profile curve set not found
	IERR = 2	Error: invalid time step, set to 1
	IERR = 3	Error: invalid time interval, set to 5
	IERR = 4	profile curve set table I/O error
	IERR = 5	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.10. TSPF_PROFILECURVESET_DATA

Use this API routine to modify the data of an existing Time Series Power Flow profile curve set, or to add a new Time Series Power Flow profile curve set.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_PROFILECURVESET_DATA CURVESETID INTGAR(1) INTGAR(2) INTMULT START
```

Python command syntax:

```
ierr = tspf_profilecurveset_data(curvesetid, intgar, intmult, start)
```

Fortran command syntax:

```
CALL TSPF_PROFILECURVESET_DATA(CURVESETID, INTGAR, INTMULT, START, IERR)
```

Where:

Integer CURVESETID	Profile Curve Set Number (input; no default allowed).	
Integer INTGAR (2)	array of 2 elements specifying (input).	
	INTGAR(1)	time step (1 by default)
	INTGAR(2)	time interval (5 by default)
	INTGAR(2) = 1	year
	INTGAR(2) = 2	month
	INTGAR(2) = 3	week
	INTGAR(2) = 4	day
	INTGAR(2) = 5	hour
	INTGAR(2) = 6	minute
	INTGAR(2) = 7	second
Real INTMULT	Time Interval Multiplier (input; 1.0).	
Character*160 START	Start time (input; "2001-01-01 00:00:00").	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid curve set number
	IERR = 2	Error: invalid time step, set to 1
	IERR = 3	Error: invalid time interval, set to 5
	IERR = 4	profile curve set table I/O error
	IERR = 5	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.11. TSPF_PROFILEDATA_CHNG

Use this API routine to modify the data of an existing Time Series Power Flow profile curve value.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_PROFILEDATA_CHNG CURVESETID CURVECOL INDEX VALUE IERR
```

Python command syntax:

```
ierr = tspf_profiledata_chng(curvesetid,curvecol,index,value)
```

Fortran command syntax:

```
CALL TSPF_PROFILEDATA_CHNG(CURVESETID, CURVECOL ,INDEX,VALUE,IERR)
```

Where:

Integer CURVESETID	Profile Curve Set Number (input; no default allowed).	
Integer CURVECOL	Profile Curve Col Number (input; no default allowed).	
Integer INDEX	Curve value Index (input; no default allowed).	
Real VALUE	Value (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	The time point of the given profile curve set and curve number not found
	IERR = 2	Profile table I/O error
	IERR = 3	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.12. TSPF_PROFILEDATA_DATA

Use this API routine to modify the data of an existing Time Series Power Flow profile curve value, or to add a new Time Series Power Flow profile curve value.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_PROFILEDATA_DATA CURVESETID CURVECOL INDEX VALUE IERR
```

Python command syntax:

```
ierr = tspf_profiledata_data(curvesetid,curvecol,index,value)
```

Fortran command syntax:

```
CALL TSPF_PROFILEDATA_DATA(CURVESETID, CURVECOL ,INDEX,VALUE,IERR)
```

Where:

Integer CURVESETID	Profile Curve Set Number (input; no default allowed).	
Integer CURVECOL	Profile Curve Col Number (input; no default allowed).	
Integer INDEX	Curve value Index (input; no default allowed).	
Real VALUE	Value (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid input for non-optional field.
	IERR = 2	Internal database error.
	IERR = 3	Prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.13. TSPF_PROFILE_CHNG

Use this API routine to modify the data of an existing Time Series Power Flow profile.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_PROFILE_CHNG NAME INTGAR(1)..INTGAR(6) REALAR(1) REALAR(2)
 ELEMENT
```

Python command syntax:

```
ierr = tspf_profile_chng(name, intgar, realar, element)
```

Fortran command syntax:

```
CALL TSPF_PROFILE_CHNG(NAME, INTGAR, REALAR, ELEMENT, IERR)
```

Where:

Character*160 NAME	Unique name of the profile data (input; no default allowed).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	field (1 by default)
	INTGAR(1) = 1	LOADMW
	INTGAR(1) = 2	LOADWVAR
	INTGAR(1) = 3	GENMW
	INTGAR(1) = 4	STATUS
	INTGAR(2)	status (1 by default)
	INTGAR(2) = 0	Off
	INTGAR(2) = 1	On
	INTGAR(3)	profile Curve Set Number (1 by default)
	INTGAR(4)	profile Curve column Number (1 by default)
	INTGAR(5)	profile Adjustment Mode (1 by default)
	INTGAR(5) = 1	Absolute
	INTGAR(5) = 2	Scale
	INTGAR(5) = 3	Deviation
	INTGAR(5) = 3	Schedule
	INTGAR(6)	enforce Machine MW Limits (0 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	Offset (0.0 by default)
	REALAR(2)	Scale Factor (1.0 by default)
Character*160 ELEMENT	Name of the element which profile is applied. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	profile not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	profile table I/O error
IERR = 3	Element not found
IERR = 4	Error: invalid status, set to 1
IERR = 5	Error: invalid adjustment mode
IERR = 6	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

7.2.14. TSPF_PROFILE_DATA

Use this API routine to modify the data of an existing Time Series Power Flow profile, or to add a new Time Series Power Flow profile.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_TSPF_PROFILE_DATA NAME INTGAR(1)..INTGAR(6) REALAR(1) REALAR(2)
 ELEMENT
```

Python command syntax:

```
ierr = tspf_profile_data(name, intgar, realar, element)
```

Fortran command syntax:

```
CALL TSPF_PROFILE_DATA(NAME, INTGAR, REALAR, ELEMENT, IERR)
```

Where:

Character*160 NAME	Unique name of the profile data (input; no default allowed).	
Integer INTGAR (6)	array of 6 elements specifying (input).	
	INTGAR(1)	field (1 by default)
	INTGAR(1) = 1	LOADMW
	INTGAR(1) = 2	LOADWVAR
	INTGAR(1) = 3	GENMW
	INTGAR(1) = 4	STATUS
	INTGAR(2)	status (1 by default)
	INTGAR(2) = 0	Off
	INTGAR(2) = 1	On
	INTGAR(3)	profile Curve Set Number (1 by default)
	INTGAR(4)	profile Curve column Number (1 by default)
	INTGAR(5)	profile Adjustment Mode (1 by default)
	INTGAR(5) = 1	Absolute
	INTGAR(5) = 2	Scale
	INTGAR(5) = 3	Deviation
	INTGAR(5) = 3	Schedule
	INTGAR(6)	enforce Machine MW Limits (0 by default)
Real REALAR (2)	array of 2 elements specifying (input).	
	REALAR(1)	Offset (0.0 by default)
	REALAR(2)	Scale Factor (1.0 by default)
Character*160 ELEMENT	Name of the element which profile is applied. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Empty profile name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	profile table I/O error
IERR = 3	Element not found
IERR = 4	Error: invalid status, set to 1
IERR = 5	Error: invalid adjustment mode
IERR = 6	prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Time Series Power Flow Data Changing APIs contains API routines specific to adding, modifying, and deleting Time Series Power Flow data.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 8
Dynamics Simulation

This chapter describes the API routines used in PSS®E Dynamic Simulation.

This chapter is divided into sections for General Operations, Output Channel Operations, detailed routines for dealing with models, by type, and finally setting disturbances.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1. Dynamic Simulation Operation

Dynamic Simulation Operation contains API routines that deals with general operations of dynamics simulation functions, including option settings and reports.

8.1.1. ADDMODELLIBRARY

Use this API to add a library to the list to be searched for library models.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_ADDMODELLIBRARY  LIBRARYNAME
```

Python command syntax:

```
ierr = addmodellibrary(libraryname)
```

Fortran command syntax:

```
CALL ADDMODELLIBRARY(LIBRARYNAME, IERR)
```

Where:

Character*260 LIBRARYNAME	name of the library of model programs (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	library not found
	IERR = 2	library load failed
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.2. ASTR

Use this API to calculate and replicates the state variable system matrices in the form required by the Linear Dynamic Analysis Program, LSYSAN (activity ASTR).

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ASTR  OPTION  MAXDERIV  PERTURB  NSTATE  MINSTATE(1)..MINSTATE(NSTATE)
   MAXSTATE(1)..MAXSTATE(NSTATE)  NOUTPUT  MINCHAN(1)..MINCHAN(NOUTPUT)
  MAXCHAN(1)..MAXCHAN(NOUTPUT)  NINPUT  INTYPE(1)..INTYPE(NINPUT)
 INPERT(1)..INPERT(NINPUT)  INBUS(1)..INBUS(NINPUT)
 INMCID(1)..INMCID(NINPUT)  INIDENT(1)..INIDENT(NINPUT)  OUTFILE
```

Python command syntax:

```
ierr = astr(option, maxderiv, perturb, nstate, minstate, maxstate,
 noutput, minchan, maxchan, ninput, intype, inpert, inbus, inmcid,
 inident, outfile)
```

Fortran command syntax:

```
CALL ASTRAPI(OPTION, MAXDERIV, PERTURB, NSTATE, MINSTATE, MAXSTATE,
 NOUTPUT, MINCHAN, MAXCHAN, NINPUT, INTYPE, INPERT, INBUS, INMCID,
 INIDENT, OUTFILE, IERR)
```

Where:

Integer OPTION	network solution convergence monitor option (input; 0 by default). OPTION = 1 automatically print the convergence monitor. OPTION = 0 print the convergence monitor only if it is enabled via the CM interrupt control code.
Real MAXDERIV	largest derivative threshold for considering that the initialization reflects a steady-state condition (input; 0.01).
Real PERTURB	amount by which the specified state variables are to be perturbed (input; 0.01).
Integer NSTATE	number of state variable ranges defining those state variables that are to be perturbed; 0 to include all STATEs (input; 0 by default).
Integer MINSTATE (NSTATE)	a list of starting STATE indices defining the ranges of state variables to be perturbed; ignored if NSTATE = 0 (input; no default allowed).
Integer MAXSTATE (NSTATE)	a list of ending STATE indices defining the ranges of state variables to be perturbed; ignored if NSTATE = 0 (input; no default allowed).
Integer NOUTPUT	number of channel ranges defining those output channels that are to be included in the system output vector; 0 to use the first 'NCHAN' or 50, whichever is less (input; 0 by default).
Integer MINCHAN (NOUTPUT)	a list of starting channel indices defining the ranges of output channels to be included in the system output vector; ignored if NOUTPUT = 0 (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer MAXCHAN (NOUTPUT)	a list of ending channel indices defining the ranges of output channels to be included in the system output vector; ignored if NOUTPUT = 0 (input; no default allowed).
Integer NINPUT	number of system variables that are to be perturbed (input; 0 by default).
Integer INTYPE (NINPUT)	a list of variable type codes; ignored if NINPUT = 0 (input; no default allowed).
	INTYPE = 1 EFD, generator main field voltage (pu)
	INTYPE = 2 PMECH, turbine mechanical power (pu on MBASE)
	INTYPE = 3 VOTHSG, stabilizer output signal (pu)
	INTYPE = 4 VREF, voltage regulator voltage setpoint (pu)
	INTYPE = 5 VAR
Real INPERT (NINPUT)	a list of perturbation factors to be applied to those system variables that are to be perturbed; ignored if NINPUT = 0 (input; PERTURB).
Integer INBUS (NINPUT)	a list of buses (when the corresponding INTYPE is 1 through 4) and/or VAR indices (when the corresponding INTYPE is 5) used to define those system variables that are to be perturbed; ignored if NINPUT = 0 (input; no default allowed).
Character*2 INMCID (NINPUT)	a list of machine identifiers used to define those system variables that are to be perturbed (when the corresponding INTYPE is 1 through 4; ignored when the corresponding INTYPE is 5); ignored if NINPUT = 0 (input; '1' by default).
Character*32 INIDENT (NINPUT)	a list of text strings used to identify those system variables that are to be perturbed; ignored if NINPUT = 0. If a blank identifier is specified, the ASTR API provides an appropriate identifier (input; blank by default).
Character*260 OUTFILE	name of the Matrix Output File into which the results of this API's calculation are to be placed; this file may then be used as to the LSYSAN program (input; no default allowed).
Integer IERR	error code (output).
	IERR = 0 no error occurred
	IERR = 1 generators are not converted
	IERR = 2 invalid OPTION value
	IERR = 3 invalid MAXDERIV value
	IERR = 4 invalid PERTURB value
	IERR = 5 derivative threshold exceeded--not in steady state
	IERR = 6 invalid code of input to be perturbed
	IERR = 7 invalid perturbation amount
	IERR = 8 machine is out-of-service
	IERR = 9 invalid VAR index
	IERR = 10 attempting to select more STATEs than are in the system model
	IERR = 11 too many channels selected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 12	prior initialization modified the loads--pick up original converted case
IERR = 13	error opening OUTFILE
IERR = 14	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.3. AULIST

Use this API to list auxiliary signal models that inject signals into dc lines or FACTS devices connected to subsystem buses, along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_AULIST SID ALL STATUS
```

Python command syntax:

```
ierr = aulist(sid, all, status)
```

Fortran command syntax:

```
CALL AULIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.4. CHANGE_CHANNEL_OUT_FILE

Use this API to modify the dynamic simulation channel output filename in dynamics working memory (see PSSE Program Operation Manual, activity ALTR).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_CHANNEL_OUT_FILE  OUTFILE
```

Python command syntax:

```
ierr = change_channel_out_file(outfile)
```

Fortran command syntax:

```
CALL CHANGE_CHANNEL_OUT_FILE(OUTFILE, IERR)
```

Where:

Character*260 OUTFILE	name of the Channel Output File (blank to bypass recording of the output channel values in a Channel Output File) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	error opening OUTFILE
	IERR = 2	prerequisite requirements for API are not met
	IERR = -1	warnings: - None

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.5. CHANGE_CHRICN

Use this API to change the value of an element of the CHRICN array (a character ICON). The element is specified relative to the beginning of the CHRICN array.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CHANGE_CHRICN  NUMBER  'NEWVAL'
```

Python command syntax:

```
ierr = change_chricn(number, newval)
```

Fortran command syntax:

```
CALL CHANGE_CHRICN(NUMBER, NEWVAL, IERR)
```

Where:

Integer NUMBER	CHRICN number or array index (must be between 1 and the number of ICONs allowed at the present size level of PSSE) (input; no default allowed).	
Character*12 NEWVAL	new value to be assigned to CHRICN (NUMBER) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid CHRICN index
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.6. CHANGE_CON

Use this API to change the value of an element of the CON array. The element is specified relative to the beginning of the CON array.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CHANGE_CON  NUMBER  NEWVAL
```

Python command syntax:

```
ierr = change_con(number, newval)
```

Fortran command syntax:

```
CALL CHANGE_CON(NUMBER, NEWVAL, IERR)
```

Where:

Integer NUMBER	CON number or array index (must be between 1 and the number of CONs allowed at the present size level of PSSE) (input; no default allowed).	
Real NEWVAL	new value to be assigned to CON (NUMBER) (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid CON index
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.7. CHANGE_GREF

Use this API to change the value of the element of the GREF (governor reference) array associated with a specified machine.

This API routine was first introduced in release 31.0.2.

Batch command syntax:

```
BAT_CHANGE_GREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = change_gref(ibus, id, newval)
```

Fortran command syntax:

```
CALL CHANGE_GREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be assigned to the GREF array entry associated with machine ID at bus IBUS (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - This machine is not a synchronous machine - No turbine-governor model at this machine - Turbine-governor model at this machine is bypassed - Machine is out of service - Bus is disconnected - Bus is not a generator bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.8. CHANGE_ICON

Use this API to change the value of an element of the ICON array (an integer ICON). The element is specified relative to the beginning of the ICON array.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CHANGE_ICON  NUMBER  NEWVAL
```

Python command syntax:

```
ierr = change_icon(number, newval)
```

Fortran command syntax:

```
CALL CHANGE_ICON(NUMBER, NEWVAL, IERR)
```

Where:

Integer NUMBER	ICON number or array index (must be between 1 and the number of ICONs allowed at the present size level of PSSE) (input; no default allowed).	
Integer NEWVAL	new value to be assigned to ICON (NUMBER) (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid ICON index
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.9. CHANGE_PREF

Use this API to change the value of PREF (Plant Controller active power reference) associated with a specified renewable machine.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_CHANGE_PREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = change_pref(ibus, id, newval)
```

Fortran command syntax:

```
CALL CHANGE_PREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be assigned to the PREF associated with machine ID at bus IBUS (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	this machine is not a renewable machine
	IERR = 4	no electrical control model at this machine
	IERR = 5	electrical control model at this machine is bypassed
	IERR = 6	machine is out of service
	IERR = 7	bus is disconnected
	IERR = 8	bus is not a generator bus
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.10. CHANGE_QVREF

Use this API to change the value of QVREF (Plant Controller reactive power / voltage reference) associated with a specified renewable machine.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_CHANGE_QVREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = change_qvref(ibus, id, newval)
```

Fortran command syntax:

```
CALL CHANGE_QVREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be assigned to QVREF associated with machine ID at bus IBUS (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	this machine is not a renewable machine
	IERR = 4	no electrical control model at this machine
	IERR = 5	electrical control model at this machine is by-passed
	IERR = 6	machine is out of service
	IERR = 7	bus is disconnected
	IERR = 8	bus is not a generator bus
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.11. CHANGE_STATE

Use this API to change the value of an element of the STATE array, as well as the values of the corresponding time derivative (DSTATE) and the associated memory cell used in the integration algorithm (STORE). The element is specified relative to the beginning of the STATE array.
Note: Although this API may be used to modify the value of a state variable (along with its time derivative and integration memory cell) used in state-space and response test simulations, this ability is rarely required in normal dynamic simulation work. Unless some very special purpose is being served, this API should not be used.
Note: This API should never be used during extended term simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_STATE  NUMBER  NEWVAL(1)..NEWVAL(3)
```

Python command syntax:

```
ierr = change_state(number, newval)
```

Fortran command syntax:

```
CALL CHANGE_STATE(NUMBER, NEWVAL, IERR)
```

Where:

Integer NUMBER	STATE number or array index (must be between 1 and the number of STATEs allowed at the present size level of PSSE) (input; no default allowed).	
Real NEWVAL (3)	array of 3 elements specifying (input).	
	NEWVAL(1)	new value to be assigned to STATE(NUMBER) (unchanged by default)
	NEWVAL(2)	new value to be assigned to DSTATE(NUMBER) (unchanged by default)
	NEWVAL(3)	new value to be assigned to STORE(NUMBER) (unchanged by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATE index
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.12. CHANGE_SWSREF

This API routine is obsolete. It has been replaced by the API routine change_swsref_2 .

Use this API to change the value of the element of the SWREF array associated with a specified switched shunt.

This API routine was first introduced in release 32.2.1.

Batch command syntax:

```
BAT_CHANGE_SWSREF  IBUS NEWVAL
```

Python command syntax:

```
ierr = change_swsref(ibus, newval)
```

Fortran command syntax:

```
CALL CHANGE_SWSREF(IBUS, NEWVAL, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input).	
Real NEWVAL	new value to be assigned to the SWSREF array entry associated with switched shunt at bus IBUS (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - no table mode for switched shunt at this bus. - switched shunt dynamic model at this switched shunt is bypassed.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.13. CHANGE_SWSREF_2

Use this API to change the value of the element of the SWREF array associated with a specified switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSREF_2  IBUS ID  NEWVAL
```

Python command syntax:

```
ierr = change_swsref_2(ibus, id, newval)
```

Fortran command syntax:

```
CALL CHANGE_SWSREF_2(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Real NEWVAL	new value to be assigned to the SWSREF array entry associated with switched shunt at bus IBUS (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	data error, warning and information messages; one or more of: - no table mode for switched shunt at this bus. - switched shunt dynamic model at this switched shunt is bypassed.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.14. CHANGE_VAR

Use this API to change the value of an element of the VAR array. The element is specified relative to the beginning of the VAR array.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VAR  NUMBER  NEWVAL
```

Python command syntax:

```
ierr = change_var(number, newval)
```

Fortran command syntax:

```
CALL CHANGE_VAR(NUMBER, NEWVAL, IERR)
```

Where:

Integer NUMBER	VAR number or array index (must be between 1 and the number of VARs allowed at the present size level of PSSE) (input; no default allowed).	
Real NEWVAL	new value to be assigned to VAR (NUMBER) (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid VAR index
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.15. CHANGE_VREF

Use this API to change the value of the element of the VREF (voltage reference) array associated with a specified machine.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = change_vref(ibus, id, newval)
```

Fortran command syntax:

```
CALL CHANGE_VREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be assigned to the VREF array entry associated with machine ID at bus IBUS (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - This machine is not a synchronous machine - No excitation system model at this machine - Excitation system model at this machine is bypassed - Machine is out of service - Bus is disconnected - Bus is not a generator bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.16. CRCTRUN

Use this API to run critical clearing time calculation in dynamic simulations.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_CRCTRUN INITIAL_FD TOTAL_SIM_T STP_SRCH_FLG STP_SRCH_FD GENANG_THR
 IBUSEX ID FD_UNIT
```

Python command syntax:

```
ierr, crct_val = crctrun(initial_fd, total_sim_t, stp_srch_flg,
 stp_srch_fd, genang_thr, ibusex, id, fd_unit)
```

Fortran command syntax:

```
CALL CRCTRUN(INITIAL_FD, TOTAL_SIM_T, STP_SRCH_FLG, STP_SRCH_FD,
 GENANG_THR, IBUSEX, ID, FD_UNIT, CRCT_VAL, IERR)
```

Where:

Real INITIAL_FD	Initial fault duration while starting critical clearing time calculation (unit depends on 'FD_UNIT' argument) (input; 5.0).	
Real TOTAL_SIM_T	Total simulation time (second) (input; 10.0).	
Logical STP_SRCH_FLG	Flag for stop searching Critical Clearing Time Calculation (if network remain stable after STP_SRCH_FD duration) (input; TRUE).	
Real STP_SRCH_FD	Stop searching Critical Clearing Time Calculation, if network remain stable after STP_SRCH_FD duration (unit depends on 'FD_UNIT' argument) (input; 30.0).	
Real GENANG_THR	Value of the deviation threshold for conventional generators to determine stability (degree) (input; 180.0).	
Integer IBUSEX	relative angle calculation to be applied (input; if batch command; unchanged by default)).	
	IBUSEX > 0	use machine ID at bus IBUSEX as the reference machine
	IBUSEX = 0	use system average angle
	IBUSEX = -1	use system weighted average angle
Character*2 ID	Machine identifier (input; if batch command; '1' by default)).	
Integer FD_UNIT	Unit for 'INITIAL_FD', 'STP_SRCH_FD', and 'CRCT_VAL' arguments (input; 1 by default).	
	FD_UNIT = 1	cycle
	FD_UNIT = 2	second
Real CRCT_VAL	Critical Clearing Time output value (output).	
	CRCT_VAL > 0	Critical Clearing Time output value (unit depends on 'FD_UNIT' argument)
	CRCT_VAL = -1.0	CrCT search stopped because CrCT is longer than (Total Simulation Time - Contingency Starting Time)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CRCT_VAL = -2.0	CrCT search stopped because of 'STP_SRCH_FLG' being Active
	CRCT_VAL = -3.0	CrCT can not be found, there is no stable fault duration
	CRCT_VAL = -4.0	CrCT Calculation aborted
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	fault definition not found
	IERR = 2	invalid INITIAL_FD value
	IERR = 3	invalid TOTAL_SIM_T value
	IERR = 4	invalid STP_SRCH_FD value
	IERR = 5	invalid GENANG_THR value
	IERR = 6	invalid IBUSEX value
	IERR = 7	Invalid FD_UNIT value
	IERR = 8	bus not found (for relative angle calculation)
	IERR = 9	bus is not a type 2 bus (for relative angle calculation)
	IERR = 10	machine does not have a synchronous machine model (for relative angle calculation)
	IERR = 11	machine not found (for relative angle calculation)
	IERR = 12	machine is out of service (for relative angle calculation)
	IERR = 13	RUN has already been executed
	IERR = 14	fault already been applied
	IERR = 15	an internal error occured, please check messages
	IERR = 16	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.17. DCLIST

Use this API to list dc line models connected to subsystem buses along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DCLIST SID ALL STATUS(1) STATUS(2)
```

Python command syntax:

```
ierr = dclist(sid, all, status)
```

Fortran command syntax:

```
CALL DCLIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer STATUS (2)	array of 2 elements specifying a two-element array(input).	
	STATUS(1)	active/bypassed flag (0 by default)
	STATUS(1) = 0	list active and bypassed models
	STATUS(1) = 1	list active models only
	STATUS(1) = 2	list bypassed models only
	STATUS(2)	dc line model type (1 by default)
	STATUS(2) = 1	two-terminal dc line models
	STATUS(2) = 2	multi-terminal dc line models
	STATUS(2) = 3	VSC dc line models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.18. DLST

Use this API to tabulate specified portions of one or more of the PSSE dynamics data storage arrays.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DLST NTYPE ATYPE(1)..ATYPE(NTYPE) FIRST(1)..FIRST(NTYPE)
 LAST(1)..LAST(NTYPE)
```

Python command syntax:

```
ierr = dlst(ntype, atype, first, last)
```

Fortran command syntax:

```
CALL DLSTAPI(NTYPE, ATYPE, FIRST, LAST, IERR)
```

Where:

Integer NTYPE	number of array portion specifications to be supplied; must be between 1 and 6, inclusive (input; no default allowed).	
Integer ATYPE (NTYPE)	a list of PSSE array type codes. No more than one occurrence of each of the valid ATYPE values is allowed in each use of this API (input; no default allowed).	
	ATYPE = 1	for CONs
	ATYPE = 2	for STATEs
	ATYPE = 3	for VARs
	ATYPE = 4	for ICONs and CHRICNs
	ATYPE = 5	for output channels
	ATYPE = 6	for CRT plot channels
Integer FIRST (NTYPE)	a list of starting indices in the data array(s) specified by the corresponding ATYPE entry (input; 1 by default).	
Integer LAST (NTYPE)	a list of ending indices in the data array(s) specified by the corresponding ATYPE entry (input; next available - 1 by default for ATYPE = 1 through 5; 6 by default for ATYPE = 6).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid NTYPE value
	IERR = 2	invalid ATYPE value (entry skipped)
	IERR = 3	ATYPE value previously specified (entry skipped)
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.19. DOCU

Use this API to tabulate the data associated with equipment models referenced in the user's simulation set-up.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_DOCU SID ALL STATUS(1)..STATUS(3)
```

Python command syntax:

```
ierr = docu(sid, all, status)
```

Fortran command syntax:

```
CALL DOCUNW(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (3)	array of 3 elements specifying reporting options(input).	
	STATUS(1)	processing mode (0 by default)
	STATUS(1) = 0	reporting mode
	STATUS(1) = 1	data checking mode
	STATUS(2)	models to process (3 by default)
	STATUS(2) = 2	process all CHAN models
	STATUS(2) = 3	process models by subsystem
	STATUS(2) = 4	process Miscellaneous, "other," type models
	STATUS(3)	value depends on the value of STATUS(2). If STATUS(2) = 3 then STATUS(3) is the type of model for which data is to be processed (1 by default)
	If STATUS(2) = 4:	
	STATUS(3) = 0	Miscellaneous "Other" model. DOCU for every model instance
	STATUS(3) > 0	Miscellaneous "Other" model. DOCU for the specified model instance
	If STATUS(2) = 3:	
	STATUS(3) = 1	all models
	STATUS(3) = 2	all plant models
	STATUS(3) = 3	generator models
	STATUS(3) = 4	compensator models

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(3) = 5	stabilizer models
STATUS(3) = 6	minimum excitation limiter models
STATUS(3) = 7	maximum excitation limiter models
STATUS(3) = 8	excitation system models
STATUS(3) = 9	turbine governor models
STATUS(3) = 10	load characteristic models
STATUS(3) = 11	load relay models
STATUS(3) = 12	line relay models
STATUS(3) = 13	auxiliary signal models
STATUS(3) = 14	two-terminal dc line models
STATUS(3) = 15	multi-terminal dc line models
STATUS(3) = 16	VSC dc line models
STATUS(3) = 17	FACTS device models
STATUS(3) = 20	CHAN models
STATUS(3) = 21	turbine load controller
STATUS(3) = 22	switched shunt models
STATUS(3) = 23	all renewable models
STATUS(3) = 24	renewable generator models
STATUS(3) = 25	renewable electrical control models
STATUS(3) = 26	renewable mechanical system models
STATUS(3) = 27	renewable pitch models
STATUS(3) = 28	renewable aerodynamic models
STATUS(3) = 29	renewable gust models
STATUS(3) = 30	renewable plant control models
STATUS(3) = 31	induction machine models
STATUS(3) = 32	branch device models
STATUS(3) = 33	two-winding transformer device models
STATUS(3) = 34	three-winding transformer models
STATUS(3) = 35	protection models associated with machines
STATUS(3) = 36	"other" models associated with machines
STATUS(3) = 37	"other" models associated with bus
STATUS(3) = 38	"other" models associated with two-terminal dc lines
STATUS(3) = 40	renewable torque control models
STATUS(3) = 41	"other" models associated with branch
STATUS(3) = 42	"other" models associated with switched shunt
STATUS(3) = 43	renewable weak grid control models
STATUS(3) = 44	renewable Paux Control models
STATUS(3) = 45	all load component models
STATUS(3) = 46	load main component models
STATUS(3) = 47	load distribution component models
STATUS(3) = 48	load individualcomponent # 1 models
STATUS(3) = 49	load individualcomponent # 2 models

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(3) = 50	load individualcomponent # 3 models
	STATUS(3) = 51	load individualcomponent # 4 models
	STATUS(3) = 52	load individualcomponent # 5 models
	STATUS(3) = 53	load individualcomponent # 6 models
	STATUS(3) = 54	load individualcomponent # 7 models
	STATUS(3) = 55	load individualcomponent # 8 models
	STATUS(3) = 56	load individualcomponent # 9 models
	STATUS(3) = 57	load individualcomponent # 10 models
	STATUS(3) = 58	load individualcomponent # 11 models
	STATUS(3) = 59	Stator Current Limiter models
	STATUS(3) = 60	Power Factor Controller models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.20. DROPMODELLIBRARY

Use this API to remove a library from the list being searched for library models.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_DROPMODELLIBRARY LIBRARYNAME
```

Python command syntax:

```
ierr = dropmodellibrary(libraryname)
```

Fortran command syntax:

```
CALL DROPMODELLIBRARY(LIBRARYNAME, IERR)
```

Where:

Character*260 LIBRARYNAME	name of the library of model programs (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	library not found
	IERR = 2	there are still models in use from this library; unload not done
	IERR = 3	library unload failed
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.21. DROPMODELPROGRAM

Use this API to remove a model from the list of known library models; library models are added to this list the first time that they are called.

This API routine was first introduced in release 30.3.0.

Batch command syntax:

```
BAT_DROPMODELPROGRAM PROGRAMNAME
```

Python command syntax:

```
ierr = dropmodelprogram(programname)
```

Fortran command syntax:

```
CALL DROPMODELPROGRAM(PROGRAMNAME, IERR)
```

Where:

Character*260 PROGRAMNAME	name of the model program (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	program not found
	IERR = 2	program unload failed
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.22. DYDA

Use this API to replicate dynamics model data in the form of a Dynamics Data Input file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_DYDA SID ALL STATUS(1)..STATUS(3) OUT OFILE
```

Python command syntax:

```
ierr = dyda(sid, all, status, out, ofile)
```

Fortran command syntax:

```
CALL DYDANW(SID, ALL, STATUS, OUT, OFILE, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer STATUS (3)	array of 3 elements specifying a three-element array(input).	
	STATUS(1)	models to process (2 by default)
	STATUS(1) = 2	process models by subsystem
	STATUS(1) = 3	process Miscellaneous other type models
	STATUS(2)	value depends on the value of STATUS(1). If STATUS(1) = 2 then STATUS(2) is the type of model for which data is to be processed (1 by default)
	If STATUS(1) = 3:	
	STATUS(2) = 0	Miscellaneous, "Other," model DYDA for every model instance.
	STATUS(2) > 0	Miscellaneous, "Other," model DYDA for the specificed model instance.
	If STATUS(1) = 2:	
	STATUS(2) = 1	all models
	STATUS(2) = 2	all plant models
	STATUS(2) = 3	generator models
	STATUS(2) = 4	compensator models
	STATUS(2) = 5	stabilizer models
	STATUS(2) = 6	minimum excitation limiter models
	STATUS(2) = 7	maximum excitation limiter models
	STATUS(2) = 8	excitation system models
	STATUS(2) = 9	turbine governor models

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(2) = 10	load characteristic models
STATUS(2) = 11	load relay models
STATUS(2) = 12	line relay models
STATUS(2) = 13	auxiliary signal models
STATUS(2) = 14	two-terminal dc line models
STATUS(2) = 15	multi-terminal dc line models
STATUS(2) = 16	VSC dc line models
STATUS(2) = 17	FACTS device models
STATUS(2) = 20	turbine load controller
STATUS(2) = 21	switched shunt models
STATUS(2) = 22	all renewable models
STATUS(2) = 23	renewable generator models
STATUS(2) = 24	renewable electrical control models
STATUS(2) = 25	renewable mechanical system models
STATUS(2) = 26	renewable pitch models
STATUS(2) = 27	renewable aerodynamic models
STATUS(2) = 28	renewable gust models
STATUS(2) = 29	renewable plant control models
STATUS(2) = 30	induction machine models
STATUS(2) = 31	branch device models
STATUS(2) = 32	two-winding transformer device models
STATUS(2) = 33	three-winding transformer models
STATUS(2) = 34	protection models associated with machines
STATUS(2) = 35	"other" models associated with machines
STATUS(2) = 36	"other" models associated with bus
STATUS(2) = 37	"other" models associated with two-terminal dc lines
STATUS(2) = 39	renewable torque control models
STATUS(2) = 40	"other" models associated with branch
STATUS(2) = 41	"other" models associated with switched shunt
STATUS(2) = 42	renewable weak grid control models
STATUS(2) = 43	renewable Paux Control models
STATUS(2) = 44	all load component models
STATUS(2) = 45	load main component models
STATUS(2) = 46	load distribution component models
STATUS(2) = 47	load individualcomponent # 1 models
STATUS(2) = 48	load individualcomponent # 2 models
STATUS(2) = 49	load individualcomponent # 3 models
STATUS(2) = 50	load individualcomponent # 4 models
STATUS(2) = 51	load individualcomponent # 5 models
STATUS(2) = 52	load individualcomponent # 6 models
STATUS(2) = 53	load individualcomponent # 7 models
STATUS(2) = 54	load individualcomponent # 8 models

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(2) = 55	load individualcomponent # 9 models
	STATUS(2) = 56	load individualcomponent # 10 models
	STATUS(2) = 57	load individualcomponent # 11 models
	STATUS(2) = 58	Stator Current Limiter models
	STATUS(2) = 59	Power Factor Controller models
	STATUS(3)	load model option (0 by default)
	STATUS(3) = 0	include all subsystem load models attached to loads in the specified subsystem
	STATUS(3) = 1	exclude subsystem load models attached to loads in the specified subsystem except when subsystem number is explicitly in the list
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE.
	OUT = 1	direct output to the current report device.
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	invalid OUT value
	IERR = 5	error opening OFILE
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.23. DYNAMICSMODE

Use this API to return PSSE to dynamics mode following the use of the POWERFLOWMODE API while in dynamics mode. (The POWERFLOWMODE API would normally be followed by the use of one or more of the power flow data editor APIs).

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_DYNAMICSMODE  OPT
```

Python command syntax:

```
ierr = dynamicsmode(opt)
```

Fortran command syntax:

```
CALL DSMODE(OPT, IERR)
```

Where:

Logical OPT	factorize admittance matrix flag (input; FALSE).	
	OPT = TRUE	(or T or 1 in batch command and Python) to factorize the admittance matrix
	OPT = FALSE	(or F or 0 in batch command and Python) to omit the matrix factorization
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	PSSE was started up at its powerflow entry point
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.24. DYNAMICS_SOLUTION_PARAMS

This API routine is obsolete. It has been replaced by the API routine dynamics_solution_param_2 .

Use this API to modify the dynamic simulation solution parameters in dynamics working memory.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DYNAMICS_SOLUTION_PARAMS  INTGAR(1)..INTGAR(8)  REALAR(1)..REALAR(8)
 OUTFILE
```

Python command syntax:

```
ierr = dynamics_solution_params(intgar, realar, outfile)
```

Fortran command syntax:

```
CALL DYNAMICS_SOLUTION_PARAMETERS(INTGAR, REALAR, OUTFILE, IERR)
```

Where:

Integer INTGAR (8)	array of 8 elements specifying (input).	
	INTGAR(1)	network solution maximum number of iterations
	INTGAR(2)	number of channels being monitored; must be less than or equal to the number of output channels defined
	INTGAR(3)	number of state variables in use in the simulation model; should not normally be changed by the user
	INTGAR(4)	next available location in the CON array
	INTGAR(5)	next available location in the STATE array
	INTGAR(6)	next available location in the VAR array
	INTGAR(7)	next available location in the ICON and CHRICN arrays
	INTGAR(8)	next available location in the channel arrays
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	acceleration sfactor used in the network solution
	REALAR(2)	convergence tolerance used in the network solution
	REALAR(3)	simulation time step (DELT)
	REALAR(4)	filter time constant used in calculating bus frequency deviations
	REALAR(5)	intermediate simulation mode time step threshold used in extended term simulations

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(6)	large (island frequency) simulation mode time step threshold used in extended term simulations
	REALAR(7)	large (island frequency) simulation mode acceleration factor used in extended term simulations
	REALAR(8)	large (island frequency) simulation mode convergence tolerance used in extended term simulations
Character*260 OUTFILE	name of the Channel Output File (blank to bypass recording of the output channel values in a Channel Output File) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	error opening OUTFILE
	IERR = 2	prerequisite requirements for API are not met
	IERR = -1	warnings: - None

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.25. DYNAMICS_SOLUTION_PARAM_2

Use this API to modify the dynamic simulation solution parameters (except the channel output filename) in dynamics working memory (see PSSE Program Operation Manual, activity ALTR).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_DYNAMICS_SOLUTION_PARAM_2  INTGAR(1)..INTGAR(8)  REALAR(1)..REALAR(8)
```

Python command syntax:

```
ierr = dynamics_solution_param_2(intgar, realar)
```

Fortran command syntax:

```
CALL DYNAMICS_SOLUTION_PARAMETERS_2(INTGAR, REALAR, IERR)
```

Where:

Integer INTGAR (8)	array of 8 elements specifying (input).	
	INTGAR(1)	network solution maximum number of iterations
	INTGAR(2)	number of channels being monitored; must be less than or equal to the number of output channels defined
	INTGAR(3)	number of state variables in use in the simulation model; should not normally be changed by the user
	INTGAR(4)	next available location in the CON array
	INTGAR(5)	next available location in the STATE array
	INTGAR(6)	next available location in the VAR array
	INTGAR(7)	next available location in the ICON and CHRICN arrays
	INTGAR(8)	next available location in the channel arrays
Real REALAR (8)	array of 8 elements specifying (input).	
	REALAR(1)	acceleration sfactor used in the network solution
	REALAR(2)	convergence tolerance used in the network solution
	REALAR(3)	simulation time step (DELT)
	REALAR(4)	filter time constant used in calculating bus frequency deviations
	REALAR(5)	intermediate simulation mode time step threshold used in extended term simulations
	REALAR(6)	large (island frequency) simulation mode time step threshold used in extended term simulations

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	REALAR(7)	large (island frequency) simulation mode acceleration factor used in extended term simulations
	REALAR(8)	large (island frequency) simulation mode convergence tolerance used in extended term simulations
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	error occurred
	IERR = 1	prerequisite requirements for API are not met
	IERR = -1	warnings: - None

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.26. DYNEXPORTCSV

Use this API to export dynamics engine and dynamics model messages that are displayed after running a dynamics simulation to CSV file format. If applicable the following data will be included in the specified output file: type, severity, model, model type, bus number, bus name, base kv, area number, owner number, and engine/model messages

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_DYNEXPORTCSV CSVFILENAME
```

Python command syntax:

```
ierr = dynexportcsv(csvfilename)
```

Fortran command syntax:

```
CALL DYNEXPORTCSV(CSVFILENAME, IERR)
```

Where:

character*260 CSVFILENAME	name of the CSV file to hold exported data (input).	
Integer IERR	error code (output).	
	IERR = -1	Prerequisite requirements for API not met
	IERR = 0	no error
	IERR = 1	error opening csv file
	IERR = 2	error closing csv file; data may not have been written to file
	IERR = 3	error writing data to csv file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.27. DYRE_ADD

This API routine is obsolete. It has been replaced by the API routine dyre_add_2 .

Use this API to read a Dynamics Model Raw Data File and append the model references specified in its data records to the simulation data already contained in dynamics working memory (activity DYRE,ADD).

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DYRE_ADD  STARTINDX(1)..STARTINDX(4)  DYREFILE  CONECFILE  CONETFILE
```

Python command syntax:

```
ierr = dyre_add(startindx, dyrefile, conecfile, conetfile)
```

Fortran command syntax:

```
CALL DYRE_ADD(STARTINDX, DYREFILE, CONECFILE, CONETFILE, IERR)
```

Where:

Integer STARTINDX (4)	array of 4 elements specifying starting locations in the dynamics data storage arrays(input).	
	STARTINDX(1)	starting CON index (the next available CON by default)
	STARTINDX(2)	starting STATE index (the next available STATE by default)
	STARTINDX(3)	starting VAR index (the next available VAR by default)
	STARTINDX(4)	starting ICON index (the next available ICON by default)
Character*260 DYREFILE	name of the Dynamics Model Raw Data File (input; no default allowed).	
Character*260 CONECFILE	name of the output file for CONEC additions; blank to route data to the progress area (input; blank by default).	
Character*260 CONETFILE	name of the output file for CONET additions; blank to route data to the progress area (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STARTINDX value
	IERR = 2	machine model connection tables full--use pack plant model tables function
	IERR = 3	error opening output file DYREFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.28. DYRE_ADD_2

Use this API to read a Dynamics Model Raw Data File and either append or replace (replace if data already exists in memory for that element)
the model references specified in its data records to the simulation data already contained in dynamics working memory (activity DYRE,ADD).

This API routine was first introduced in release 36.0.

Batch command syntax:

```
BAT_DYRE_ADD_2  STARTINDX(1)..STARTINDX(4)  DYREFILE
```

Python command syntax:

```
ierr = dyre_add_2(startindx, dyrefile)
```

Fortran command syntax:

```
CALL DYRE_ADD_2(STARTINDX, DYREFILE, IERR)
```

Where:

Integer STARTINDX (4)	array of 4 elements specifying starting locations in the dynamics data storage arrays(input).	
	STARTINDX(1)	starting CON index (the next available CON by default)
	STARTINDX(2)	starting STATE index (the next available STATE by default)
	STARTINDX(3)	starting VAR index (the next available VAR by default)
	STARTINDX(4)	starting ICON index (the next available ICON by default)
Character*260 DYREFILE	name of the Dynamics Model Raw Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STARTINDX value
	IERR = 2	machine model connection tables full--use pack plant model tables function
	IERR = 3	error opening output file DYREFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.29. DYRE_NEW

This API routine is obsolete. It has been replaced by the API routine dyre_new_2 .

Use this API to clear dynamics working memory, read a Dynamics Data File, and place the model references specified on its data records into dynamics working memory. It optionally creates a command file for compiling the CONEC and CONET subroutines.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DYRE_NEW  STARTINDX(1)..STARTINDX(4)  DYREFILE  CONECFILE  CONETFILE
 COMPILFIL
```

Python command syntax:

```
ierr = dyre_new(startindx,dyrefile,conecfile,conetfile,compilfil)
```

Fortran command syntax:

```
CALL DYRE_NEW(STARTINDX, DYREFILE, CONECFILE, CONETFILE, COMPILFIL, IERR)
```

Where:

Integer STARTINDX (4)	array of 4 elements specifying starting locations in the dynamics data storage arrays(input).	
	STARTINDX(1)	starting CON index (1 by default)
	STARTINDX(2)	starting STATE index (1 by default)
	STARTINDX(3)	starting VAR index (1 by default)
	STARTINDX(4)	starting ICON index (1 by default)
Character*260 DYREFILE	name of the Dynamics Model Raw Data File (input; no default allowed).	
Character*260 CONECFILE	name of CONEC output file; blank for output to the progress area (input; blank by default).	
Character*260 CONETFILE	name of CONET output file; blank for output to the progress area (input; blank by default).	
Character*260 COMPILFIL	name of file containing commands to compile the CONEC and CONET output files; blank for none (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STARTINDX value
	IERR = 3	error opening output file DYREFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.30. DYRE_NEW_2

Use this API to clear dynamics working memory, read a Dynamics Data File, and place the model references specified on its data records into dynamics working memory. It optionally creates a command file for compiling the CONEC and CONET subroutines.

This API routine was first introduced in release 36.0.

Batch command syntax:

```
BAT_DYRE_NEW_2  STARTINDX(1)..STARTINDX(4)  DYREFILE
```

Python command syntax:

```
ierr = dyre_new_2(startindx,dyrefile)
```

Fortran command syntax:

```
CALL DYRE_NEW_2(STARTINDX, DYREFILE, IERR)
```

Where:

Integer STARTINDX (4)	array of 4 elements specifying starting locations in the dynamics data storage arrays(input).	
	STARTINDX(1)	starting CON index (1 by default)
	STARTINDX(2)	starting STATE index (1 by default)
	STARTINDX(3)	starting VAR index (1 by default)
	STARTINDX(4)	starting ICON index (1 by default)
Character*260 DYREFILE	name of the Dynamics Model Raw Data File (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STARTINDX value
	IERR = 3	error opening output file DYREFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.31. ERUN

Use this API to calculate PSSE state-space dynamic simulations of excitation system response tests.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ERUN  TPAUSE  NPRT  NPLT  CRTPLT
```

Python command syntax:

```
ierr = erun(tpause, nprt, nplt, crtplt)
```

Fortran command syntax:

```
CALL ERUNAPI(TPAUSE, NPRT, NPLT, CRTPLT, IERR)
```

Where:

Real TPAUSE	value of simulated time at which the simulation should next pause (input; no default allowed).	
Integer NPRT	number of time steps between the printing of the channel values (input; unchanged).	
Integer NPLT	number of time steps between the writing of the output channel values to the current Channel Output File (input; unchanged).	
Integer CRTPLT	number of time steps between the plotting of those channel values that have been designated as CRT output channels (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	activity ESTR needs to be executed
	IERR = 2	error opening the current Channel Output File
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.32. ESTR_OPEN_CIRCUIT_TEST

Use this API to initialize a PSSE dynamic simulation for excitation system open circuit response simulations (i.e., in preparation for activity ERUN) and to specify the Channel Output File into which the output channel values are to be recorded during the simulation.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ESTR_OPEN_CIRCUIT_TEST  OPTION  VDELTA  OUTFILE
```

Python command syntax:

```
ierr = estr_open_circuit_test(option, vdelta, outfile)
```

Fortran command syntax:

```
CALL ESTR_OPEN_CIRCUIT_TEST(OPTION, VDELTA, OUTFILE, IERR)
```

Where:

Integer OPTION	bus flag indicating those buses to be tested (input; 0 by default).	
	OPTION = 0	run test on all in-service machines with excitation system models.
	OPTION > 0	bus number for which machines are to be tested.
Real VDELTA	per unit voltage reference step applied at TIME = 0.0 (input; 0.05).	
Character*260 OUTFILE	name of the Channel Output File (blank to bypass recording of the output channel values in a Channel Output File) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	error opening OUTFILE
	IERR = 3	generators are not "converted"
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.33. ESTR_RESPONSE_RATIO_TEST

Use this API to initialize a PSSE dynamic simulation for excitation system response ratio test simulations (i.e., in preparation for activity ERUN) and to specify the Channel Output File into which the output channel values are to be recorded during the simulation.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ESTR_RESPONSE_RATIO_TEST  OPTION  PFDEFALT  NMACHS
  BUSARY(1)..BUSARY(NMACHS)   IDARY(1)..IDARY(NMACHS)
 PFARY(1)..PFARY(NMACHS)  OUTFILE
```

Python command syntax:

```
ierr = estr_response_ratio_test(option, pfdefalt, nmachs, busary, idary,
 pfary, outfile)
```

Fortran command syntax:

```
CALL ESTR_RESPONSE_RATIO_TEST(OPTION, PFDEFALT, NMACHS, BUSARY, IDARY,
 PFARY, OUTFILE, IERR)
```

Where:

Integer OPTION	bus flag indicating those buses to be tested (input; 0 by default).	
	OPTION = 0	run test on all in-service machines with excitation system models.
	OPTION > 0	bus number for which machines are to be tested.
Real PFDEFALT	default power factor at which machines are initialized (input; 0.95).	
Integer NMACHS	number of machines to be initialized at other power factors (input; 0 by default).	
Integer BUSARY (NMACHS)	a list of bus numbers identifying machines to be initialized at other power factors; ignored if NMACHS = 0 (input; no default allowed).	
Character*2 IDARY (NMACHS)	a list of machine identifiers identifying machines to be initialized at other power factors; ignored if NMACHS = 0 (input; '1' by default).	
Real PFARY (NMACHS)	a list of power factors; ignored if NMACHS = 0. Machine IDARY(n) at bus BUSARY(n) is initialized to the power factor specified in PFARY(n) (input; PFDEFALT).	
Character*260 OUTFILE	name of the Channel Output File (blank to bypass recording of the output channel values in a Channel Output File) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	invalid power factor
	IERR = 3	error opening OUTFILE

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	generators are not "converted"
IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.34. FCLIST

Use this API to list FACTS device models connected to subsystem buses along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCLIST SID ALL STATUS
```

Python command syntax:

```
ierr = fclist(sid, all, status)
```

Fortran command syntax:

```
CALL FCLIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.35. GRUN

Use this API to calculate PSSE state-space dynamic simulations of turbine-governor response tests.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_GRUN  TPAUSE  NPRT  NPLT  CRTPLT
```

Python command syntax:

```
ierr = grun(tpause, nprt, nplt, crtplt)
```

Fortran command syntax:

```
CALL GRUNAPI(TPAUSE, NPRT, NPLT, CRTPLT, IERR)
```

Where:

Real TPAUSE	value of simulated time at which the simulation should next pause (input; no default allowed).	
Integer NPRT	number of time steps between the printing of the channel values (input; unchanged).	
Integer NPLT	number of time steps between the writing of the output channel values to the current Channel Output File (input; unchanged).	
Integer CRTPLT	number of time steps between the plotting of those channel values that have been designated as CRT output channels (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	activity GSTR needs to be executed
	IERR = 2	error opening the current Channel Output File
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.36. GSTR

Use this API to initialize a PSSE dynamic simulation for governor response simulations and to specify the Channel Output File into which the output channel values are to be recorded during the simulation.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_GSTR  OPTION  PINITIAL  PDELTA  OUTFILE
```

Python command syntax:

```
ierr = gstr(option, pinitial, pdelta, outfile)
```

Fortran command syntax:

```
CALL GSTRAPI(OPTION, PINITIAL, PDELTA, OUTFILE, IERR)
```

Where:

Integer OPTION	bus flag indicating those buses to be tested (input; 0 by default).	
	OPTION = 0	run test on all in-service machines with turbine-governor models.
	OPTION = 0	bus number for which machines are to be tested.
Real PINITIAL	initial per unit loading of all machines being tested (input; 0.8).	
Real PDELTA	per unit loading step applied at TIME = 0.0 (input; 0.1).	
Character*260 OUTFILE	name of the Channel Output File (blank to bypass recording of the output channel values in a Channel Output File) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	error opening OUTFILE
	IERR = 3	generators are not, "converted"
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.37. INCREMENT_GREF

Use this API to add a specified value to the value of the element of the GREF (governor reference) array associated with a specified machine.

This API routine was first introduced in release 31.0.2.

Batch command syntax:

```
BAT_INCREMENT_GREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = increment_gref(ibus, id, newval)
```

Fortran command syntax:

```
CALL INCREMENT_GREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be added to the GREF array entry associated with machine ID at bus IBUS (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - This machine is not a synchronous machine - No turbine-governor model at this machine - Turbine-governor model at this machine is bypassed - Machine is out of service - Bus is disconnected - Bus is not a generator bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.38. INCREMENT_PREF

Use this API to add a specified value to the value of PREF (Plant Controller active power reference) associated with a specified renewable machine.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INCREMENT_PREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = increment_pref(ibus, id, newval)
```

Fortran command syntax:

```
CALL INCREMENT_PREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be added to the PREF associated with machine ID at bus IBUS (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	this machine is not a renewable machine
	IERR = 4	no electrical control model at this machine
	IERR = 5	electrical control model at this machine is bypassed
	IERR = 6	machine is out of service
	IERR = 7	bus is disconnected
	IERR = 8	bus is not a generator bus
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.39. INCREMENT_QVREF

Use this API to add a specified value to QVREF (Plant Controller reactive power / voltage reference) associated with a specified renewable machine.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_INCREMENT_QVREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = increment_qvref(ibus, id, newval)
```

Fortran command syntax:

```
CALL INCREMENT_QVREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be added to QVREF associated with machine ID at bus IBUS (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	this machine is not a renewable machine
	IERR = 4	no electrical control model at this machine
	IERR = 5	electrical control model at this machine is bypassed
	IERR = 6	machine is out of service
	IERR = 7	bus is disconnected
	IERR = 8	bus is not a generator bus
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.40. INCREMENT_SWSREF

This API routine is obsolete. It has been replaced by the API routine increment_swsref_2 .

Use this API to add a specified value to the value of the element of the SWREF (switched shunt reference) array associated with a specified switched shunt.

This API routine was first introduced in release 32.2.1.

Batch command syntax:

```
BAT_INCREMENT_SWSREF  IBUS NEWVAL
```

Python command syntax:

```
ierr = increment_swsref(ibus, newval)
```

Fortran command syntax:

```
CALL INCREMENT_SWSREF(IBUS, NEWVAL, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input).	
Real NEWVAL	new value to be added to the SWSREF array entry associated with switched shunt at bus IBUS (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - no table model for switched shunt at this bus. - switched shunt dynamic model as this bus is bypassed.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.41. INCREMENT_SWSREF_2

Use this API to add a specified value to the value of the element of the SWREF (switched shunt reference) array associated with a specified switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_INCREMENT_SWSREF_2  IBUS ID  NEWVAL
```

Python command syntax:

```
ierr = increment_swsref_2(ibus, id, newval)
```

Fortran command syntax:

```
CALL INCREMENT_SWSREF_2(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Real NEWVAL	new value to be added to the SWSREF array entry associated with switched shunt at bus IBUS (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	error occurred
	IERR = 1	bus not found
	IERR = 2	switched shunt not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - no table model for switched shunt at this bus. - switched shunt dynamic model as this bus is bypassed.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.42. INCREMENT_VREF

Use this API to add a specified value to the value of the element of the VREF (voltage reference) array associated with a specified machine.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_INCREMENT_VREF  IBUS  'ID'  NEWVAL
```

Python command syntax:

```
ierr = increment_vref(ibus, id, newval)
```

Fortran command syntax:

```
CALL INCREMENT_VREF(IBUS, ID, NEWVAL, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Real NEWVAL	new value to be added to the VREF array entry associated with machine ID at bus IBUS (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	bus not found
	IERR = 2	machine not found
	IERR = 3	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - This machine is not a synchronous machine - No excitation system model at this machine - Excitation system model at this machine is bypassed - Machine is out of service - Bus is disconnected - Bus is not a generator bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.43. LDCLIST

Use this API to list load models connected to subsystem loads along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 35.x.0.

Batch command syntax:

```
BAT_LDCLIST SID ALL STATUS(1) STATUS(2) STATUS(3)
```

Python command syntax:

```
ierr = ldclist(sid, all, status)
```

Fortran command syntax:

```
CALL LDCLIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (3)	array of 3 elements specifying a three-element array(input).	
	STATUS(1)	active/bypassed flag (0 by default)
	STATUS(1) = 0	active and bypassed models
	STATUS(1) = 1	active models only
	STATUS(1) = 2	bypassed models only
	STATUS(2)	model connection report option (0 by default)
	STATUS(2) = 0	list models connected to loads
	STATUS(2) = 1	list loads connected to model
	STATUS(3)	load component model type (1 by default)
	STATUS(3) = 1	all load component models
	STATUS(3) = 10	main component models
	STATUS(3) = 11	distribution component models
	STATUS(3) = 21	individual component 1 models
	STATUS(3) = 22	individual component 2 models
	STATUS(3) = 23	individual component 3 models
	STATUS(3) = 24	individual component 4 models
	STATUS(3) = 25	individual component 5 models
	STATUS(3) = 26	individual component 6 models
	STATUS(3) = 27	individual component 7 models
	STATUS(3) = 28	individual component 8 models

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(3) = 29	individual component 9 models
	STATUS(3) = 30	individual component 10 models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.44. LDLIST

Use this API to list load models connected to subsystem loads along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LDLIST SID ALL STATUS(1) STATUS(2)
```

Python command syntax:

```
ierr = ldlist(sid, all, status)
```

Fortran command syntax:

```
CALL LDLIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (2)	array of 2 elements specifying a two-element array(input).	
	STATUS(1)	active/bypassed flag (0 by default)
	STATUS(1) = 0	active and bypassed models
	STATUS(1) = 1	active models only
	STATUS(1) = 2	bypassed models only
	STATUS(2)	model connection report option (0 by default)
	STATUS(2) = 0	list models connected to loads
	STATUS(2) = 1	list loads connected to model
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.45. LOAD_ARRAY_CHANNEL

Use this API to add an output channel containing the active or reactive component of a designated load.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_LOAD_ARRAY_CHANNEL  STATUS(1)..STATUS(3)  'ID'  'IDENT'
```

Python command syntax:

```
ierr = load_array_channel(status, id, ident)
```

Fortran command syntax:

```
CALL LOAD_ARRAY_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (3)	array of 3 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	used to indicate the quantity to be placed in an channel (1 by default)
	STATUS(2) = 1	PLOAD
	STATUS(2) = 2	QLOAD
	STATUS(3)	number of the bus to which the load for which quantity is to be placed in an channel is connected (no default allowed)
Character*2 ID	load identifier (input; '1' by default).	
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	bus not found
	IERR = 4	load not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.46. MLST

Use this API to list plant models connected to subsystem machines along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_MLST SID ALL STATUS(1)..STATUS(3)
```

Python command syntax:

```
ierr = mlst(sid, all, status)
```

Fortran command syntax:

```
CALL MLSTNW(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS (3)	array of 3 elements specifying a three-element array(input).	
	STATUS(1)	connected/unconnected flag (1 by default)
	STATUS(1) = 0	list all unconnected models
	STATUS(1) = 1	list connected subsystem models
	STATUS(2)	active/bypassed flag (0 by default)
	STATUS(2) = 0	active and bypassed models
	STATUS(2) = 1	active models only
	STATUS(2) = 2	bypassed models only
	STATUS(3)	plant model type (1 by default)
	STATUS(3) = 1	all plant models
	STATUS(3) = 2	generator models
	STATUS(3) = 3	compensator models
	STATUS(3) = 4	stabilizer models
	STATUS(3) = 5	minimum excitation models
	STATUS(3) = 6	maximum excitation models
	STATUS(3) = 7	excitation system models
	STATUS(3) = 8	governor models
	STATUS(3) = 9	turbine load controller models
Integer IERR	error code (output).	
	IERR = 0	no error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	invalid SID value or subsystem SID is not defined
IERR = 2	invalid ALL value
IERR = 3	invalid STATUS value
IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.47. MRUN

Use this API to calculate PSSE extended term dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MRUN  OPTION  TPAUSE  NPRT  NPLT  CRTPLT
```

Python command syntax:

```
ierr = mrun(option, tpause, nprt, nplt, crtplt)
```

Fortran command syntax:

```
CALL MRUNAPI(OPTION, TPAUSE, NPRT, NPLT, CRTPLT, IERR)
```

Where:

Integer OPTION	network solution convergence monitor option (input; 0 by default).	
	OPTION = 1	automatically print the convergence monitor.
	OPTION = 0	print the convergence monitor only if it is enabled via the CM interrupt control code.
Real TPAUSE	value of simulated time at which the simulation should next pause (input; no default allowed).	
Integer NPRT	number of time steps between the printing of the channel values (input; unchanged).	
Integer NPLT	number of time steps between the writing of the output channel values to the current Channel Output File (input; unchanged).	
Integer CRTPLT	number of time steps between the plotting of those channel values that have been designated as CRT output channels (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	long time step mode threshold is at or below intermediate mode threshold
	IERR = 2	activity MSTR needs to be executed
	IERR = 3	invalid OPTION value
	IERR = 4	generators are not converted
	IERR = 5	error opening the current Channel Output File
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.48. MSTR

Use this API to initialize a PSSE dynamic simulation for extended term simulations and to specify the Channel Output File into which the output channel values are to be recorded during the dynamic simulation.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MSTR  OPTION  OUTFILE
```

Python command syntax:

```
ierr = mstr(option, outfile)
```

Fortran command syntax:

```
CALL MSTRAPI(OPTION, OUTFILE, IERR)
```

Where:

Integer OPTION	network solution convergence monitor option (input; 0 by default).	
	OPTION = 1	automatically print the convergence monitor.
	OPTION = 0	print the convergence monitor only if it is enabled via the CM interrupt control code.
Character*260 OUTFILE	name of the Channel Output File (blank to bypass recording of the output channel values in a Channel Output File) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	long time step mode threshold is at or below intermediate mode threshold
	IERR = 2	DELT is greater than intermediate time step mode threshold
	IERR = 3	generators are not converted
	IERR = 4	invalid OPTION value
	IERR = 5	Prior initialization modified the loads--pick up original converted case
	IERR = 6	error opening OUTFILE
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.49. POWERFLOWMODE

Use this API to switch PSSE from dynamics mode to power flow mode. (This API would normally be followed by the use of one or more of the power flow data editor APIs, and then the DYNAMICSMODE API).
Batch command syntax.
BAT_POWERFLOWMODE

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_POWERFLOWMODE
```

Python command syntax:

```
ierr = powerflowmode()
```

Fortran command syntax:

```
CALL PFMODE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.50. PSAS

Use this API to convert a PSAS Command File into a PSSE Response File.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PSAS INPFILE OUTFILE
```

Python command syntax:

```
ierr, respfil = psas(inpfile, outfile)
```

Fortran command syntax:

```
CALL PSASAPI(INPFILE, OUTFILE, RESPFIL, IERR)
```

Where:

Character*260 INPFILE	name of the PSAS command file (input; no default allowed).	
Character*260 OUTFILE	name of Response file; blank to have PSAS assign the filename (input; blank by default).	
Character*260 RESPFIL	pathname of the Response file constructed by PSAS (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	error opening INPFILE
	IERR = 2	error opening OUTFILE
	IERR = 3	errors detected while reading the PSAS command data input file
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.51. RLLIST

Use this API to list line relay models connected to subsystem buses along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RLLIST SID ALL STATUS
```

Python command syntax:

```
ierr = rllist(sid, all, status)
```

Fortran command syntax:

```
CALL RLLIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.52. RSTR

Use this API to read a dynamics Snapshot File into PSSE working memory.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RSTR  SFILE
```

Python command syntax:

```
ierr = rstr(sfile)
```

Fortran command syntax:

```
CALL RSTRNW(SFILE, IERR)
```

Where:

Character*260 SFILE	snapshot file; '*' to restore the most recently accessed snapshot file in the current execution of PSSE (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	error opening SFILE
	IERR = 2	error reading from SFILE
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.53. RUN

Use this API to calculate PSSE state-space dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_RUN  OPTION  TPAUSE  NPRT  NPLT  CRTPLT
```

Python command syntax:

```
ierr = run(option, tpause, nprt, nplt, crtplt)
```

Fortran command syntax:

```
CALL RUNAPI(OPTION, TPAUSE, NPRT, NPLT, CRTPLT, IERR)
```

Where:

Integer OPTION	network solution convergence monitor option (input; 0 by default).	
	OPTION = 1	automatically print the convergence monitor
	OPTION = 0	print the convergence monitor only if it is enabled via the CM interrupt control code
Real TPAUSE	value of simulated time at which the simulation should next pause (input; no default allowed).	
Integer NPRT	number of time steps between the printing of the channel values (input; unchanged).	
Integer NPLT	number of time steps between the writing of the output channel values to the current channel output file (input; unchanged).	
Integer CRTPLT	number of time steps between the plotting of those channel values that have been designated as CRT output channels (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	activity STRT needs to be executed
	IERR = 2	invalid OPTION value
	IERR = 3	generators are not converted
	IERR = 4	error opening the current channel output file
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.54. RWDY

Use this API to replicate portions of dynamics model data in the form of either a breaker duty data file, unit inertia and governor data file, or PSSPLT relay characteristic data file.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_RWDY OPTIONS(1)..OPTIONS(3) OUT OFILE
```

Python command syntax:

```
ierr = rwdy(options, out, ofile)
```

Fortran command syntax:

```
CALL RWDYAPI(OPTIONS, OUT, OFILE, IERR)
```

Where:

Integer OPTIONS (3)	array of 3 elements specifying (input).	
	OPTIONS(1)	type of file to be (1 by default)
	OPTIONS(1) = 1	breaker duty data file
	OPTIONS(1) = 2	unit inertia and governor data file
	OPTIONS(1) = 3	PSSPLT relay characteristic data file
	OPTIONS(2)	machine limits flag for activity INLF (ignored unless OPTIONS(1) = 2) (0 by default)
	OPTIONS(2) = 0	include machine limits
	OPTIONS(2) = 1	suppress machine limits
	OPTIONS(3)	flag indicating the treatment of machines without governor models in activity INLF (ignored unless OPTIONS(1) = 2) (0 by default)
	OPTIONS(3) = 0	use the working case limits
	OPTIONS(3) = 1	treat them as non-dispatchable
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPTIONS value
	IERR = 2	invalid OUT value
	IERR = 3	error opening OFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.55. SET_CHNFIL_TYPE

Use this API to set/Get the channel output file type (extended type or not).

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_SET_CHNFIL_TYPE  STATUS ICODE
```

Python command syntax:

```
ierr = set_chnfil_type(status)

ierr, status = set_chnfil_type()
```

Fortran command syntax:

```
CALL SET_CHNFIL_TYPE(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	value of the option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 0	channel output file is not of extended type (.out)
	STATUS = 1	channel output file is of extended type (.outx)
Integer ICODE	indicates the operation mode of the API (1 by default).	
	ICODE = 0	set the option setting to the value specified in STATUS
	ICODE > 0	return the current value of the option setting in STATUS
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.56. SET_DISABLE_RUN

Use this API to enable or disable the simulation option setting that precludes dynamic simulation runs in the event there are fatal errors in the model data.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_SET_DISABLE_RUN STATUS
```

Python command syntax:

```
ierr = set_disable_run(status)

ierr, status = set_disable_run()
```

Fortran command syntax:

```
CALL SET_DISABLE_RUN(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 0	Do not disable dynamic simulation run if there is fatal data error
	STATUS = 1	Disable dynamic simulation run if there is fatal data error
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in STATUS
	ICODE > 0	return the current value of the option setting in STATUS
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	pre-requisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.57. SET_FREQ_RELAY_V_THRESH

This API is used to set voltage threshold for use in frequency relay models.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_SET_FREQ_RELAY_V_THRESH  FRELVTHRESH
```

Python command syntax:

```
ierr = set_freq_relay_v_thresh(frelvthresh)

ierr, frelvthresh  = set_freq_relay_v_thresh()
```

Fortran command syntax:

```
CALL SET_FREQ_RELAY_V_THRESH(FRELVTHRESH, ICODE, IERR)
```

Where:

Real FRELVTHRESH	(input; Voltage threshold for use in frequency relay models: if batch command or ICODE is 0 or negative; unchanged by default if ICODE is positive. This value can be used in frequency relay models as the minimum PU voltage for frequency measurement.).	
Integer ICODE	operation mode of API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in FRelVThresh.
	ICODE = 1	return the current value of the option setting in FRelVThresh.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.58. SET_GENANG

This API routine is obsolete. It has been replaced by the API routine set_genang_2 .

Use this API to enable or disable the simulation option setting that scans for generators for which the angle differs from the angular average by more than a specified threshold.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SET_GENANG  STATUS  ANGLE
```

Python command syntax:

```
ierr = set_genang(status, angle)

ierr, status,angle = set_genang()
```

Fortran command syntax:

```
CALL SET_GENANG(STATUS, ANGLE, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 1	enable scanning for generators exceeding angle threshold.
	STATUS = 0	disable scanning option.
Real ANGLE	deviation threshold (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS and the threshold to the value specified in ANGLE.
	ICODE > 0	return the current value of the option setting in STATUS and the threshold in ANGLE.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid angle threshold--must be >0.0 and <360.0
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.59. SET_GENANG_2

This API routine is obsolete. It has been replaced by the API routine set_genang_3 .

Use this API to enable or disable the simulation option setting that scans for generators for which the angle differs from the angular average by more than a specified threshold. Scanning, if enabled, will be done for all machines whose machine MVA is greater than the specified value

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SET_GENANG_2  STATUS  ANGLE    MBASE
```

Python command syntax:

```
ierr = set_genang_2(status, angle, mbase)

ierr, status,angle,mbase = set_genang_2()
```

Fortran command syntax:

```
CALL SET_GENANG_2(STATUS, ANGLE, MBASE, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 1	enable scanning for generators exceeding angle threshold.
	STATUS = 0	disable scanning option.
Real ANGLE	deviation threshold (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
Real MBASE	machine size (MVA) (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive). All machines whose MVAR ratings are above this threshold will be scanned).	
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS and the threshold values specified in ANGLE and MBASE.
	ICODE > 0	return the current value of the option setting in STATUS and the threshold values in ANGLE and MBASE
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid angle threshold--must be >0.0 and <360.0
	IERR = 2	invalid angle threshold--must be >= 0.0
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.60. SET_GENANG_3

Use this API to enable or disable the simulation option setting that scans for generators for which the angle differs from the angular average by more than a specified threshold. Scanning, if enabled, will be done for all machines whose machine MVA is greater than the specified value

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SET_GENANG_3  STATUS ANGLE MBASE TRIP
```

Python command syntax:

```
ierr = set_genang_3(status, angle, mbase, trip)

ierr, status, angle, mbase, trip = set_genang_3()
```

Fortran command syntax:

```
CALL SET_GENANG_3(STATUS, ANGLE, MBASE, TRIP, ICODE, IERR)
```

Where:

Integer STATUS	value of the option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 1	enable scanning for generators exceeding angle threshold.
	STATUS = 0	disable scanning option.
Real ANGLE	value of the deviation threshold (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
Real MBASE	machine size (MVA) (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive). All machines whose MVAR ratings are above this threshold will be scanned).	
Integer TRIP	monitor trip flag (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive). All machines whose MVAR ratings are above this threshold will be scanned).	
	TRIP = 1	trip generators
	TRIP = 0	monitor generators only
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS and the threshold values specified in ANGLE, MBASE and TRIP
	ICODE > 0	return the current value of the option setting in STATUS and the threshold values in ANGLE, MBASE and TRIP
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid angle threshold--must be >0.0 and <360.0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid angle threshold--must be >= 0.0
IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.61. SET_GENANG_SUBSYS_FLAG

Use this API api for get/set the flag that is used for the subsystem defintion for performing scan for generators exceeding angle threshold. When the subsystem flag is set, the API will also flag those buses contained in subsystem SID.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SET_GENANG_SUBSYS_FLAG  ALL SID
```

Python command syntax:

```
ierr = set_genang_subsys_flag(all, sid)

ierr, all = set_genang_subsys_flag()
```

Fortran command syntax:

```
CALL SET_GENANG_SUBSYS_FLAG(ALL, SID, ICODE, IERR)
```

Where:

Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in ALL
	ICODE > 0	return the current value of the option setting in ALL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid ALL value
	IERR = 2	invalid SID value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.62. SET_GENPWR

Use this API to enable or disable the simulation option setting that scans for generators for which the mechanical power differs from its electrical power by more than a specified threshold.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SET_GENPWR  STATUS  POWER
```

Python command syntax:

```
ierr = set_genpwr(status, power)

ierr, status,power = set_genpwr()
```

Fortran command syntax:

```
CALL SET_GENPWR(STATUS, POWER, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 1	enable scanning for generators exceeding power unbalance threshold
	STATUS = 0	disable scanning option
Real POWER	power unbalance threshold (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS and the threshold values specified in POWER
	ICODE > 0	return the current value of the option setting in STATUS and the threshold values in POWER
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid power unbalance threshold - must be >0.0 and <10.0
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.63. SET_GENSPDEV

Use this API to enable or disable the simulation option setting that scans for generators for which the speed deviation is more than a specified threshold.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SET_GENSPDEV  STATUS SPEED TRIP
```

Python command syntax:

```
ierr = set_genspdev(status, speed, trip)

ierr, status, speed, trip = set_genspdev()
```

Fortran command syntax:

```
CALL SET_GENSPDEV(STATUS, SPEED, TRIP, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 1	enable scanning for generator speeds exceeding speed deviation threshold
	STATUS = 0	disable scanning option
Real SPEED	speed deviation threshold (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
Integer TRIP	monitor trip flag (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive). All machines whose MVAR ratings are above this threshold will be scanned).	
	TRIP = 1	trip generators
	TRIP = 0	monitor generators only
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS and the threshold values specified in SPEED and TRIP
	ICODE > 0	return the current value of the option setting in STATUS and the threshold values in SPEED and TRIP
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid speed deviation threshold - must be >0.0
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.64. SET_GENSPDEV_SUBSYS_FLAG

Use this API api for get/set the flag that is used for the subsystem defintion for performing scans for generators for which the speed deviation is more than a specified threshold.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SET_GENSPDEV_SUBSYS_FLAG  ALL SID
```

Python command syntax:

```
ierr = set_genspdev_subsys_flag(all, sid)

ierr, all = set_genspdev_subsys_flag()
```

Fortran command syntax:

```
CALL SET_GENSPDEV_SUBSYS_FLAG(ALL, SID, ICODE, IERR)
```

Where:

Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in ALL
	ICODE > 0	return the current value of the option setting in ALL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid ALL value
	IERR = 2	invalid SID value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.65. SET_LOAD_MODEL_THRESH

This API is used to set the MW, P over Q, and load bus voltage thresholds for load models.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_SET_LOAD_MODEL_THRESH  LMWTHRESH LPQTHRESH LVTTHRESH
```

Python command syntax:

```
ierr = set_load_model_thresh(lmwthresh, lpqthresh, lvtthresh)

ierr, lmwthresh, lpqthresh, lvtthresh = set_load_model_thresh()
```

Fortran command syntax:

```
CALL SET_LOAD_MODEL_THRESH(LMWTHRESH, LPQTHRESH, LVTTHRESH, ICODE, IERR)
```

Where:

Real LMWTHRESH	(input; MW threshold for applying load models: if batch command or ICODE is 0 or negative; unchanged by default if ICODE is positive. This threshold can be used in load models to decide whether the load model should be applied or not.).	
Real LPQTHRESH	(input; P by Q (absolute value) threshold for applying load models: if batch command or ICODE is 0 or negative; unchanged by default if ICODE is positive. This threshold can be used in load models to decide whether the load model should be applied or not.).	
Real LVTTHRESH	(input; pu load bus voltage threshold for applying load models: if batch command or ICODE is 0 or negative; unchanged by default if ICODE is positive. This threshold can be used in load models to decide whether the load model should be applied or not.).	
Integer ICODE	operation mode of API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in LMWThresh, LPQThresh, and LVtThresh.
	ICODE = 1	return the current value of the option setting in LMWThresh, LPQThresh, and LVtThresh.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.66. SET_MODEL_DEBUG_OUTPUT_FLAG

Use this API to enable or disable the simulation option setting for dynamic model debug output.

This API routine was first introduced in release 33.12.0 and 34.5.0.

Batch command syntax:

```
BAT_SET_MODEL_DEBUG_OUTPUT_FLAG  STATUS
```

Python command syntax:

```
ierr = set_model_debug_output_flag(status)

ierr, status = set_model_debug_output_flag()
```

Fortran command syntax:

```
CALL SET_MODEL_DEBUG_OUTPUT_FLAG(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if Batch command or for Python if specified as an argument, or for Fortran if ICODE is 0; no default allowed).	
	STATUS = 0	disable dynamic model debug output
	STATUS = 1	enable dynamic model debug output
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in STATUS
	ICODE = 1	return the current value of the option setting in STATUS
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.67. SET_NETFRQ

Use this API to enable or disable the simulation option setting that models the frequency dependence of network parameters.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SET_NETFRQ  STATUS
```

Python command syntax:

```
ierr = set_netfrq(status)

ierr, status = set_netfrq()
```

Fortran command syntax:

```
CALL SET_NETFRQ(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 0	disable network frequency dependence
	STATUS = 1	enable network frequency dependence
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in STATUS
	ICODE > 0	return the current value of the option setting in STATUS
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.68. SET_NEXT_CHANNEL

Use this API to assign the next available channel index value.

This API routine was first introduced in release 31.0.2.

Batch command syntax:

```
BAT_SET_NEXT_CHANNEL  IVAL
```

Python command syntax:

```
ierr = set_next_channel(ival)
```

Fortran command syntax:

```
CALL SET_NEXT_CHANNEL(IVAL, IERR)
```

Where:

Integer IVAL	value to assign as the next available channel index (input).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid channel index
	IERR = 2	index is beyond the maximum allowed
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.69. SET_NEXT_ICON

Use this API to assign the next available ICON index value.

This API routine was first introduced in release 31.0.2.

Batch command syntax:

```
BAT_SET_NEXT_ICON  IVAL
```

Python command syntax:

```
ierr = set_next_icon(ival)
```

Fortran command syntax:

```
CALL SET_NEXT_ICON(IVAL, IERR)
```

Where:

Integer IVAL	value to assign as the next available ICON index (input).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid ICON index
	IERR = 2	index is beyond the maximum allowed
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.70. SET_NEXT_VAR

Use this API to assign the next available VAR index value.

This API routine was first introduced in release 31.0.2.

Batch command syntax:

```
BAT_SET_NEXT_VAR  IVAL
```

Python command syntax:

```
ierr = set_next_var(ival)
```

Fortran command syntax:

```
CALL SET_NEXT_VAR(IVAL, IERR)
```

Where:

Integer IVAL	value to assign as the next available VAR index (input).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid VAR index
	IERR = 2	index is beyond the maximum allowed
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.71. SET_OSSCAN

Use this API to enable or disable the simulation option setting that scans for out-of-step conditions using a generic relay.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SET_OSSCAN  STATUS  TRIP
```

Python command syntax:

```
ierr = set_osscan(status, trip)

ierr, status,trip = set_osscan()
```

Fortran command syntax:

```
CALL SET_OSSCAN(STATUS, TRIP, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 0	disable out-of-step scanning
	STATUS = 1	enable out-of-step scanning
Integer TRIP	monitor trip flag (input; unchanged).	
	TRIP = 0	monitor only
	TRIP = 1	trip out-of-step circuits
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS and the trip flag to the value specified in TRIP.
	ICODE > 0	return the current value of the option setting in STATUS and the trip flag in TRIP.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.72. SET_OSSCAN_2

This API routine is obsolete. It has been replaced by the API routine SET_OSSCAN_2 .

Use this API to enable or disable the simulation option setting that scans for out-of-step conditions using a generic relay.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SET_OSSCAN_2  STATUS  TRIP  EXCLXFMR
```

Python command syntax:

```
ierr = set_osscan_2(status, trip, exclxfmr)

ierr, status,trip,exclxfmr = set_osscan_2()
```

Fortran command syntax:

```
CALL SET_OSSCAN_2(STATUS, TRIP, EXCLXFMR, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 0	disable out-of-step scanning
	STATUS = 1	enable out-of-step scanning
Integer TRIP	monitor or trip flag (input; unchanged).	
	TRIP = 0	monitor only
	TRIP = 1	trip out-of-step circuits
Integer EXCLXFMR	flag to include or exclude transformer for out-of-step check (input; no default allowed).	
	EXCLXFMR = 1	exclude transformer in out-of-step scan
	EXCLXFMR = 0	do not exclude (i.e., include) transformer in out-of-step scan
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS, TRIP, and EXCLXFMR.
	ICODE > 0	return the current value of the option setting in STATUS, TRIP, and EXCLXFMR.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.73. SET_OSSCAN_SUBSYS_FLAG

Use this API api for get/set the flag that is used for the subsystem defintion for performing scans for out-of-step conditions using a generic relay.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SET_OSSCAN_SUBSYS_FLAG  ALL SID
```

Python command syntax:

```
ierr = set_osscan_subsys_flag(all, sid)

ierr, all = set_osscan_subsys_flag()
```

Fortran command syntax:

```
CALL SET_OSSCAN_SUBSYS_FLAG(ALL, SID, ICODE, IERR)
```

Where:

Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ICODE	indicates the operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in ALL
	ICODE > 0	return the current value of the option setting in ALL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid ALL value
	IERR = 2	invalid SID value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.74. SET_RELANG

Use this API to enable or disable the simulation option setting that expresses the ANGLE array relative to a designated reference angle.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SET_RELANG  SWITCH  IBUSEX  'ID'
```

Python command syntax:

```
ierr = set_relang(switch, ibusex, id)

ierr, switch,ibusex,id = set_relang()
```

Fortran command syntax:

```
CALL SET_RELANG(SWITCH, IBUSEX, ID, ICODE, IERR)
```

Where:

Integer SWITCH	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	SWITCH = 0	disable relative angle calculation (use absolute angles)
	SWITCH = 1	enable relative angle calculation
Integer IBUSEX	relative angle calculation to be applied (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	IBUSEX > 0	use machine ID at bus IBUSEX as the reference machine
	IBUSEX = 0	use system average angle
	IBUSEX = -1	use system weighted average angle
Character*2 ID	machine identifier (input; if batch command or ICODE is 0 or negative; '1' by default) (if ICODE is positive).	
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE < 0	set the option setting to the value specified in SWITCH and the relative angle parameters to the values specified in IBUSEX and ID.
	ICODE > 0	return the current value of the option setting in SWITCH and the relative angle parameters in IBUSEX and ID.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid IBUSEX value
	IERR = 2	bus not found
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.75. SET_RELSCN

Use this API to enable or disable the simulation option setting that scans branches using a generic branch relay.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SET_RELSCN  STATUS
```

Python command syntax:

```
ierr = set_relscn(status)

ierr, status = set_relscn()
```

Fortran command syntax:

```
CALL SET_RELSCN(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 0	disable relay scanning
	STATUS = 1	enable relay scanning
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE < 0	set the option setting to the value specified in STATUS
	ICODE > 0	return the current value of the option setting in STATUS
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.76. SET_RELSCN_SUBSYS_FLAG

Use this API api for get/set the flag that is used for the subsystem defintion for performing scans of branches using a generic branch relay.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SET_RELSCN_SUBSYS_FLAG  ALL SID
```

Python command syntax:

```
ierr = set_relscn_subsys_flag(all, sid)

ierr, all = set_relscn_subsys_flag()
```

Fortran command syntax:

```
CALL SET_RELSCN_SUBSYS_FLAG(ALL, SID, ICODE, IERR)
```

Where:

Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ICODE	indicates the operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in ALL
	ICODE > 0	return the current value of the option setting in ALL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid ALL value
	IERR = 2	invalid SID value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.77. SET_VLTSCN

Use this API to enable or disable the simulation option setting that scans buses for high or low values of voltage magnitude.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SET_VLTSCN  STATUS  VHI  VLO
```

Python command syntax:

```
ierr = set_vltscn(status, vhi, vlo)

ierr, status,vhi,vlo = set_vltscn()
```

Fortran command syntax:

```
CALL SET_VLTSCN(STATUS, VHI, VLO, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	STATUS = 1	enable voltage scanning
	STATUS = 1	disable voltage scanning
Real VHI	high voltage threshold (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
Real VLO	low voltage threshold (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE < 0	set the option setting to the value specified in STATUS and the voltage limits to VHI and VLO.
	ICODE > 0	return the current value of the option setting in STATUS and the voltage limits in VHI and VLO.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid specification of VHI and VLO
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.78. SET_VLTSCN_SUBSYS_FLAG

Use this API api for get/set the flag that is used for the subsystem definition for performing scans of buses for high or low voltage magnitudes.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SET_VLTSCN_SUBSYS_FLAG  ALL SID
```

Python command syntax:

```
ierr = set_vltscn_subsys_flag(all, sid)

ierr, all = set_vltscn_subsys_flag()
```

Fortran command syntax:

```
CALL SET_VLTSCN_SUBSYS_FLAG(ALL, SID, ICODE, IERR)
```

Where:

Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ICODE	indicates the operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in ALL
	ICODE > 0	return the current value of the option setting in ALL
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid ALL value
	IERR = 2	invalid SID value
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.79. SET_VOLTAGE_DIP_CHECK

This API is used to enable or disable the simulation option setting which scans buses for voltage dip (back-swing).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SET_VOLTAGE_DIP_CHECK  VDPCHK   VDPTHR   VDPDUR
```

Python command syntax:

```
ierr = set_voltage_dip_check(vdpchk, vdpthr, vdpdur)

ierr, vdpchk, vdpthr, vdpdur = set_voltage_dip_check()
```

Fortran command syntax:

```
CALL SET_VOLTAGE_DIP_CHECK(VDPCHK, VDPTHR, VDPDUR, ICODE, IERR)
```

Where:

Integer VDPCHK	(input; flag to set voltage dip check: if command or ICODE is 0 or negative; unchanged).	
	VDPCHK = 0	disable voltage dip check
	VDPCHK = 1	enable voltage dip check
Real VDPTHR	(input; voltage dip threshold in per unit: if batch command or ICODE is 0 or negative; unchanged by default. if ICODE is positive. If the voltage swings-back below this threshold a timer is imitated to check for the dip duration).	
Real VDPDUR	(input; voltage dip duration in seconds: if batch command or ICODE is 0 or negative; unchanged).	
Integer ICODE	operation mode of API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in VDPCHK, VDPTHR, and VDPDUR.
	ICODE > 0	return the current value of the option setting in VDPCHK, VDPTHR, and VDPDUR.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.80. SET_VOLTAGE_REC_CHECK

This API is used to enable or disable the simulation option setting which scans buses for voltage recovery (primary and secondary recovery).

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SET_VOLTAGE_REC_CHECK  VPRCHK   VSRCHK   VPRTHR   VPRTIM   VSRTHR
 VSRTIM
```

Python command syntax:

```
ierr = set_voltage_rec_check(vprchk, vsrchk, vprthr, vprtim, vsrthr,
 vsrtim)

ierr, vprchk, vsrchk, vprthr, vprtim, vsrthr, vsrtim =
 set_voltage_rec_check()
```

Fortran command syntax:

```
CALL SET_VOLTAGE_REC_CHECK(VPRCHK, VSRCHK, VPRTHR, VPRTIM, VSRTHR, VSRTIM,
 ICODE, IERR)
```

Where:

Integer VPRCHK	(input; flag to set voltage primary recovery check: if batch command or ICODE is 0 or negative; unchanged).	
	VPRCHK = 0	disable primary recovery check
	VPRCHK = 1	enable primary recovery check
Integer VSRCHK	(input; flag to set voltage secondary recovery check: if batch command or ICODE is 0 or negative; unchanged).	
	VSRCHK = 0	disable secondary recovery check
	VSRCHK = 1	enable secondary recovery check
Real VPRTHR	(input; voltage primary recovery threshold in per unit: if batch command or ICODE is 0 or negative; unchanged).	
Real VPRTIM	(input; voltage primary recovery time in seconds: if batch command or ICODE is 0 or negative; unchanged).	
Real VSRTHR	(input; voltage secondary recovery threshold in per unit: if batch command or ICODE is 0 or negative; unchanged).	
Real VSRTIM	(input; voltage secondary recovery time in seconds: if batch command or ICODE is 0 or negative; unchanged).	
Integer ICODE	operation mode of API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in VPRCHK, VSRCHK, VPRTHR, VPRTIM, VSRTHR and VSRTIM.
	ICODE > 0	return the current value of the option setting in VPRCHK, VSRCHK, VPRTHR, VPRTIM, VSRTHR and VSRTIM.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid specification of VPRTHR and VSRTHR
	IERR = 2	invalid specification of VPRTIM and VSRTIM
	IERR = 3	prerequisite requirements for API not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.81. SET_VOLT_VIOL_SUBSYS_FLAG

This API is used to set the flag to denote the bus subsystem to be used for voltage violation (voltage recovery and voltage dip) checks.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SET_VOLT_VIOL_SUBSYS_FLAG  ALL
```

Python command syntax:

```
ierr = set_volt_viol_subsys_flag(all)

ierr, all = set_volt_viol_subsys_flag()
```

Fortran command syntax:

```
CALL SET_VOLT_VIOL_SUBSYS_FLAG(ALL, ICODE, IERR)
```

Where:

Integer ALL	flag to set the subsystem definition for performing voltage violation check (input; if batch command or ICODE is 0 or negative; unchanged by default) (if ICODE is positive).	
	ALL = 0	Use existing bus subsystem definition
	ALL = 1	Do voltage violation check on all buses
Integer ICODE	Indicates the operation mode of API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in ALL
	ICODE > 0	return the current value of the option setting in ALL
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	Invalid ALL value
	IERR = 2	prerequisite requirements for API not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.82. SET_ZSORCE_RECONCILE_FLAG

Use this API to enable or disable the simulation option setting for automatic ZSORCE reconciliation.

This API routine was first introduced in release 33.10.0 and 34.2.0.

Batch command syntax:

```
BAT_SET_ZSORCE_RECONCILE_FLAG  STATUS
```

Python command syntax:

```
ierr = set_zsorce_reconcile_flag(status)

ierr, status = set_zsorce_reconcile_flag()
```

Fortran command syntax:

```
CALL SET_ZSORCE_RECONCILE_FLAG(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input; if Batch command or for Python if specified as an argument, or for Fortran if ICODE is 0; no default allowed).	
	STATUS = 0	disable ZSORCE reconciliation
	STATUS = 1	enable ZSORCE reconciliation
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE = 0	set the option setting to the value specified in STATUS
	ICODE = 1	return the current value of the option setting in STATUS
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.83. SIZE_DS

Use this API to obtain a summary of utilization in the general purpose dynamics data storage arrays as well as in the various model connection and allocation tables contained in dynamics working memory.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_SIZE_DS
```

Python command syntax:

```
ierr = size_ds()
```

Fortran command syntax:

```
CALL SIZE_DS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.84. SNAP

Use this API to save PSSE dynamics working memory into a Snapshot file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_SNAP STATUS(1)..STATUS(5) SFILE
```

Python command syntax:

```
ierr = snap(status, sfile)
```

Fortran command syntax:

```
CALL SNAPNW(STATUS, SFILE, IERR)
```

Where:

Integer STATUS (5)	array of 5 elements specifying save options. For each entry, -1 may be specified to indicate that the number of elements of the corresponding array(s) to save in the Snapshot File is 1 through the next available - 1(input).	
	STATUS(1)	number of CONs to save (next available-1 by default)
	STATUS(2)	number of STATEs to save (next available-1 by default)
	STATUS(3)	number of VARs to save (next available-1 by default)
	STATUS(4)	number of ICONs to save (next available-1 by default)
	STATUS(5)	number of channels to save (next available-1 by default)
Character*260 SFILE	snapshot file; '*' to use the most recently accessed snapshot file in the current execution of PSSE (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	error opening SFILE
	IERR = 3	error writing to SFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.85. STRT

This API routine is obsolete. It has been replaced by the API routine strt_2 .

Use this API to initialize a PSSE dynamic simulation for state-space simulations (i.e., in preparation for activity RUN) and to specify the channel output file into which the output channel values are to be recorded during the dynamic simulation.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_STRT  OPTION  OUTFILE
```

Python command syntax:

```
ierr = strt(option, outfile)
```

Fortran command syntax:

```
CALL STRTAPI(OPTION, OUTFILE, IERR)
```

Where:

Integer OPTION	network solution convergence monitor option (input; 0 by default).	
	OPTION = 0	print the convergence monitor only if it is enabled via the CM interrupt control code.
	OPTION = 1	automatically print the convergence monitor.
Character*260 OUTFILE	name of the channel output file (blank to bypass recording of the output channel values in a Channel Output File) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	generators are not converted
	IERR = 2	invalid OPTION value
	IERR = 3	Prior initialization modified the loads--pick up original converted case
	IERR = 4	error opening OUTFILE
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.86. STRT_2

This API routine is the second release of the dynamic simulation initialization function. Use this API routine to initialize a PSSE dynamic simulation for state-space simulations (i.e., in preparation for activity RUN) and to specify the channel output file into which the output channel values are to be recorded during the dynamic simulation.

This API routine was first introduced in release 33.9.0 and 34.2.0.

Batch command syntax:

```
BAT_STRT_2  OPTIONS  OUTFILE
```

Python command syntax:

```
ierr = strt_2(options, outfile)
```

Fortran command syntax:

```
CALL STRT_2(OPTIONS, OUTFILE, IERR)
```

Where:

Integer OPTIONS (2)	array of 2 elements specifying simulation initialization options(input).	
	OPTIONS(1)	Network solution convergence monitor option
	OPTIONS(1) = 0	bypass printing of the network solution convergence monitor
	OPTIONS(1) = 1	print the network solution convergence monitor
	OPTIONS(2)	Missing machine model netting option.
	OPTIONS(2) = 0	treat as a fatal error
	OPTIONS(2) = 1	net machine power
Character*260 OUTFILE	channel output file (blank to bypass recording of the output channel values in a channel output file) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	generators are not converted
	IERR = 2	invalid OPTIONS value
	IERR = 3	prior initialization modified the loads - pick up original converted case
	IERR = 4	error opening OUTFILE
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.87. SWSLIST

Use this API to list switched shunt models connected to subsystem buses along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SWSLIST SID ALL STATUS
```

Python command syntax:

```
ierr = swslist(sid, all, status)
```

Fortran command syntax:

```
CALL SWSLIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.88. TRIG_VOLT_VIOLATION_CHECK

This API is used to trigger voltage violation (voltage recovery, voltage dip) checks.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_TRIG_VOLT_VIOLATION_CHECK  STATUS
```

Python command syntax:

```
ierr = trig_volt_violation_check(status)

ierr, status = trig_volt_violation_check()
```

Fortran command syntax:

```
CALL TRIG_VOLT_VIOLATION_CHECK(STATUS, ICODE, IERR)
```

Where:

Integer STATUS	option setting (input).	
	STATUS = 0	disable voltage violation check
	STATUS = 1	trigger voltage violation
Integer ICODE	operation mode of the API (no default allowed).	
	ICODE <= 0	set the option setting to the value specified in STATUS
	ICODE > 0	return the current value of the option setting in STATUS
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.1.89. WNLIST

Use this API to list wind models connected to subsystem buses along with their storage locations in the dynamics data arrays.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_WNLIST SID ALL STATUS
```

Python command syntax:

```
ierr = wnlist(sid, all, status)
```

Fortran command syntax:

```
CALL WNLIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 0	process only buses in subsystem SID.
	ALL = 1	process all buses.
Integer STATUS (2)	array of 2 elements specifying API options(input).	
	STATUS(1)	active/bypassed flag (0 by default)
	STATUS(1) = 0	active and bypassed models
	STATUS(1) = 1	active models only
	STATUS(1) = 2	bypassed models only
	STATUS(2)	wind model type (1 by default)
	STATUS(2) = 1	all wind models
	STATUS(2) = 2	wind generator models
	STATUS(2) = 3	wind electrical models
	STATUS(2) = 4	wind mechanical models
	STATUS(2) = 5	wind pitch models
	STATUS(2) = 6	wind aerodynamic models
	STATUS(2) = 7	wind gust models
	STATUS(2) = 8	wind auxiliary signal models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.2. Output Channel Operations

Output Channel Operations contains API routines that deals with managing channel output operations, and with assigning quantities to output channels.

8.2.1. Dynamic Simulation Output Channel Operation

CHANGE_CHANNEL

Use this API to change the channel address and the channel identifier of a specified output channel.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGE_CHANNEL  CHNARG  ADRARG  IDNARG
```

Python command syntax:

```
ierr = change_channel(chnarg,adrarg, idnarg)
```

Fortran command syntax:

```
CALL CHANGE_CHANNEL(CHNARG, ADRARG, IDNARG, IERR)
```

Where:

Integer CHNARG	channel number for which the channel address and/or channel identifier has to be changed (input; no default allowed).	
Integer ADRARG	channel address (input; unchanged).	
Character*32 IDNARG	channel identifier (input; unchanged).	
Integer IERR	error code (output).	
	IERR = -1	warnings - one or more of: - channel number is larger than NCHAN
	IERR = 0	no error
	IERR = 1	invalid channel number
	IERR = 2	channel number is at or beyond the next available channel
	IERR = 3	invalid channel address
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHSB

Use this API to specify, on a subsystem basis, the simulation variables to monitor during dynamic simulation runs (activity CHSB).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CHSB  SID  ALL  STATUS(1)..STATUS(6)
```

Python command syntax:

```
ierr = chsb(sid, all, status)
```

Fortran command syntax:

```
CALL CHSBNW(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid area, bus, owner, or zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Area, bus, owner, or zone subsystem SID must have been previously defined (input; 0 by default).	
Integer ALL	all buses or specified subsystem flag (input; 1 by default).	
	ALL = 1	process all buses
	ALL = 0	process only buses in subsystem SID
Integer STATUS (6)	array of 6 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	used to indicate the type of quantities to be placed in channels (1 by default)
	STATUS(4) = 1	machine, bus, load, branch quantities by bus subsystem
	STATUS(4) = 2	tie quantity by area subsystem
	STATUS(4) = 3	tie quantity by zone subsystem
	STATUS(4) = 4	power totals for an area subsystem
	STATUS(4) = 5	power totals for a zone subsystem
	STATUS(4) = 6	power totals for an owner subsystem
	STATUS(4) = 7	power totals for all buses
	STATUS(4) = 8	machine angle statistics
	STATUS(5)	quantity code (1 by default). If STATUS(4) = 4, 5, 6, 7 or 8 then STATUS(5) is ignored)
	If STATUS(4) = 1:	
	STATUS(5) = 1	ANGLE, machine relative rotor angle (degrees).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATUS(5) = 2	PELEC, machine electrical power (pu on SBASE).
STATUS(5) = 3	QELEC, machine reactive power.
STATUS(5) = 4	ETERM, machine terminal voltage (pu).
STATUS(5) = 5	EFD, generator main field voltage (pu).
STATUS(5) = 6	PMECH, turbine mechanical power (pu on MBASE).
STATUS(5) = 7	SPEED, machine speed deviation from nominal (pu).
STATUS(5) = 8	XADIFD, machine field current (pu).
STATUS(5) = 9	ECOMP, voltage regulator compensated voltage (pu).
STATUS(5) = 10	VOTHSG, stabilizer output signal (pu).
STATUS(5) = 11	VREF, voltage regulator voltage setpoint (pu).
STATUS(5) = 12	BSFREQ, bus pu frequency deviations.
STATUS(5) = 13	VOLT, bus pu voltages (complex).
STATUS(5) = 14	voltage and angle
STATUS(5) = 15	flow (P).
STATUS(5) = 16	flow (P and Q).
STATUS(5) = 17	flow (MVA).
STATUS(5) = 18	apparent impedance (R and X).
STATUS(5) = 21	ITERM.
STATUS(5) = 22	machine apparent impedance
STATUS(5) = 23	VUEL, minimum excitation limiter output signal (pu).
STATUS(5) = 24	VOEL, maximum excitation limiter output signal (pu).
STATUS(5) = 25	PLOAD.
STATUS(5) = 26	QLOAD.
STATUS(5) = 27	GREF, turbine governor reference.
STATUS(5) = 28	LCREF, turbine load control reference.
STATUS(5) = 29	WVLCTY, wind velocity (m/s).
STATUS(5) = 30	WTRBSP, wind turbine rotor speed deviation (pu).
STATUS(5) = 31	WPITCH, pitch angle (degrees).
STATUS(5) = 32	WAEROT, aerodynamic torque (pu on MBASE).
STATUS(5) = 33	WROTRV, rotor voltage (pu on MBASE).
STATUS(5) = 34	WROTRI, rotor current (pu on MBASE).
STATUS(5) = 35	WPCMND, active power command from wind control (pu on MBASE).
STATUS(5) = 36	WQCMND, reactive power command from wind control (pu on MBASE).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(5) = 37	WAUXSG, output of wind auxiliary control (pu on MBASE).
	STATUS(6)	out-of-service equipment option (0 by default)
	STATUS(6) = 0	omit out-of-service equipment
	STATUS(6) = 1	include out-of-service equipment
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	have just assigned channel n, which is the highest channel number
	IERR = 5	trying to assign locations beyond the end of the VAR or ICON array
	IERR = 6	machine angle statistics are already assigned to channels
	IERR = 7	n monitoring models specified--pointer table full
	IERR = 8	next channel is greater than the maximum allowed channel number
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CRT_PLOT_CHANNEL

Use this API to assign one of the main simulation output channels to a designated CRT plot channel.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CRT_PLOT_CHANNEL  CRTCHAN  CHANNEL  CMIN  CMAX
```

Python command syntax:

```
ierr = crt_plot_channel(crtchan, channel, cmin, cmax)
```

Fortran command syntax:

```
CALL CRT_PLOT_CHANNEL(CRTCHAN, CHANNEL, CMIN, CMAX, IERR)
```

Where:

Integer CRTCHAN	CRT plot channel number (1 through 6) (input; no default allowed).	
Integer CHANNEL	number of the main simulation channel to be assigned to CRT channel number CRTCHAN (must be between 1 and the number of output channels allowed at the present size level of PSSE), or 0 to disable CRT plot channel CRTCHAN (input; unchanged).	
Real CMIN	minimum plotting scale value for this CRT plot channel (input; unchanged).	
Real CMAX	maximum plotting scale value for this CRT plot channel (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	error occurred
	IERR = 1	invalid CRTCHAN value
	IERR = 2	invalid CHANNEL value
	IERR = 3	CMIN is not less than CMAX
	IERR = 4	prerequisite requirements for API are not met
	IERR = -1	warnings - one or more of: - CHANNEL is larger than NCHAN

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

DELETE_ALL_PLOT_CHANNELS

Use this API to delete all the plot channels in the working case. Use of this API will reset the number of plot channels to zero and the next channel to one.

This API routine was first introduced in release 32.2.1.

Batch command syntax:

```
BAT_DELETE_ALL_PLOT_CHANNELS
```

Python command syntax:

```
ierr = delete_all_plot_channels()
```

Fortran command syntax:

```
CALL DELETE_ALL_PLOT_CHANNELS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

DMPC

Use this API to replicate the output channel specifications from dynamics working memory in the form of a response file suitable for transferring them to another snapshot that models essentially the same system.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DMPC OPTIONS(1)..OPTIONS(4) OUT OFILE
```

Python command syntax:

```
ierr = dmpc(options, out, ofile)
```

Fortran command syntax:

```
CALL DMPCAPI(OPTIONS, OUT, OFILE, IERR)
```

Where:

Integer OPTIONS (4)	array of 4 elements specifying (input).	
	OPTIONS(1)	channel selection code (1 by default)
	OPTIONS(1) = 0	skip channels containing STATEs or VARs not associated with channel monitoring models
	OPTIONS(1) = 1	include all channels
	OPTIONS(2)	starting channel code (-1 by default)
	OPTIONS(2) < 0	start at next available channel
	OPTIONS(2) = 0	keep existing channel numbers
	OPTIONS(2) > 0	starting channel number
	OPTIONS(3)	starting VAR code for channel monitoring models (-1 by default)
	OPTIONS(3) < 0	start at next available VAR
	OPTIONS(3) = 0	keep existing VAR indices
	OPTIONS(3) > 0	starting VAR index
	OPTIONS(4)	starting ICON code for channel monitoring models (-1 by default)
	OPTIONS(4) < 0	start at next available ICON
	OPTIONS(4) = 0	keep existing ICON indices
	OPTIONS(4) > 0	starting ICON index
Integer OUT	destination selection (input; 1 by default).	
	OUT = 0	direct output to a data file specified by OFILE
	OUT = 1	direct output to the current report device
Character*260 OFILE	name of the file to which data records are to be written (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPTIONS(1) value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	starting channel number is greater than the largest channel number allowed
IERR = 3	starting VAR index is greater than the largest VAR index allowed
IERR = 4	starting ICON index is greater than the largest ICON index allowed
IERR = 5	invalid OUT value
IERR = 6	error opening file OFILE
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

LIST_CHANNEL_MODELS

Use this API to tabulate the output channel monitoring models referenced in the dynamic model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_LIST_CHANNEL_MODELS STATUS
```

Python command syntax:

```
ierr = list_channel_models(status)
```

Fortran command syntax:

```
CALL LIST_CHANNEL_MODELS(STATUS, IERR)
```

Where:

Integer STATUS	category of channel monitoring models to be listed (input; 0 by default).	
	STATUS = 0	all output channel monitoring models
	STATUS = 1	only models monitoring equipment that exists in the power flow working case
	STATUS = 2	only models monitoring equipment that is not present in the power flow working case
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid STATUS value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

REMOVE_UNUSED_CHAN_MODELS

Use this API to delete from the output channel monitoring model table those model references indicating equipment that is not present in the power flow working case.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_REMOVE_UNUSED_CHAN_MODELS
```

Python command syntax:

```
ierr = remove_unused_chan_models()
```

Fortran command syntax:

```
CALL REMOVE_UNUSED_CHAN_MODELS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SCANCHANNELS

Scan Channels

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_SCANCHANNELS NSCANTYPE NNUMCHANNSTOOUTPUT NCHANNMIN NCHANNMAX
 NNUMQUANTFILTERS NFILTERS(1)..NFILTERS(NNUMQUANTFILTERS) FTIMEMIN
 FTIMEMAX FBANDMIN FBANDMAX SFILTERMASK SFILENAME
```

Python command syntax:

```
scanchannels(nscantype, nnumchannstooutput, nchannmin, nchannmax,
 nnumquantfilters, nfilters, ftimemin, ftimemax, fbandmin, fbandmax,
 sfiltermask, sfilename)
```

Fortran command syntax:

```
CALL SCANCHANNELS(NSCANTYPE, NNUMCHANNSTOOUTPUT, NCHANNMIN, NCHANNMAX,
 NNUMQUANTFILTERS, NFILTERS, FTIMEMIN, FTIMEMAX, FBANDMIN, FBANDMAX,
 SFILTERMASK, SFILENAME)
```

Where:

Integer NSCANTYPE	(input).
Integer NNUMCHANNSTOOUTPUT	(input).
Integer NCHANNMIN	(input).
Integer NCHANNMAX	(input).
Integer NNUMQUANTFILTERS	(input).
Integer NFILTERS (nNumQuantFilters)	Integer Array with Filter Quanitity Types (input).
Real FTIMEMIN	(input).
Real FTIMEMAX	(input).
Real FBANDMIN	(input).
Real FBANDMAX	(input).
character*8 SFILTERMASK	Channel Identifier Mask (input).
character*260 SFILENAME	The filename or filepath to the respective .out or .outx file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.2.2. Assign Machine Quantities to Output Channels

MACHINE_APP_R_X_CHANNEL

Use this API to add a pair of output channels containing the apparent impedance of the system, as seen from the terminals of a specified machine, along with a corresponding call to the GENTMZ monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MACHINE_APP_R_X_CHANNEL  STATUS(1)..STATUS(4)  'ID'  'IDENT(1)'
 'IDENT(2)'
```

Python command syntax:

```
ierr = machine_app_r_x_channel(status, id, ident)
```

Fortran command syntax:

```
CALL MACHINE_APP_R_X_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (4)	array of 4 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the bus to which the machine for which the apparent impedance is to be placed in channels is connected (no default allowed)
Character*2 ID	machine identifier (input; '1' by default).	
Character*32 IDENT (2)	two identifiers to be assigned to the two channels. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	model GENTMZ needs 2 VARs but the last one exceeds the largest VAR index allowed
IERR = 6	model GENTMZ needs 2 ICONs but the last one exceeds the largest ICON index allowed
IERR = 7	the maximum number of channel monitoring models has already been specified
IERR = 8	model GENTMZ needs 2 channels but the last one exceeds the largest channel number allowed
IERR = 9	bus not found
IERR = 10	machine not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MACHINE_ARRAY_CHANNEL

Use this API to add an output channel containing a plant related model variable of a designated type for a specified machine.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MACHINE_ARRAY_CHANNEL  STATUS(1)..STATUS(3)  'ID'  'IDENT'
```

Python command syntax:

```
ierr = machine_array_channel(status, id, ident)
```

Fortran command syntax:

```
CALL MACHINE_ARRAY_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (3)	array of 3 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	used to indicate the quantity to be placed in an channel (1 by default)
	STATUS(2) = 1	ANGLE, machine relative rotor angle (degrees)
	STATUS(2) = 2	PELEC, machine electrical power (pu on SBASE)
	STATUS(2) = 3	QELEC, machine reactive power
	STATUS(2) = 4	ETERM, machine terminal voltage (pu)
	STATUS(2) = 5	EFD, generator main field voltage (pu)
	STATUS(2) = 6	PMECH, turbine mechanical power (pu on MBASE)
	STATUS(2) = 7	SPEED, machine speed deviation from nominal (pu)
	STATUS(2) = 8	XADIFD, machine field current (pu)
	STATUS(2) = 9	ECOMP, voltage regulator compensated voltage (pu)
	STATUS(2) = 10	VOTHSG, stabilizer output signal (pu)
	STATUS(2) = 11	VREF, voltage regulator voltage setpoint (pu)
	STATUS(2) = 12	VUEL, minimum excitation limiter output signal (pu)
	STATUS(2) = 13	VOEL, maximum excitation limiter output signal (pu)
	STATUS(2) = 14	GREF, turbine governor reference
	STATUS(2) = 15	LCREF, turbine load control reference
	STATUS(2) = 16	WVLCTY, wind velocity (m/s)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	STATUS(2) = 17	WTRBSP, wind turbine rotor speed deviation (pu)
	STATUS(2) = 18	WPITCH, pitch angle (degrees)
	STATUS(2) = 19	WAEROT, aerodynamic torque (pu on MBASE)
	STATUS(2) = 20	WROTRV, rotor voltage (pu on MBASE)
	STATUS(2) = 21	WROTRI, rotor current (pu on MBASE)
	STATUS(2) = 22	WPCMND, active power command from wind control (pu on MBASE)
	STATUS(2) = 23	WQCMND, reactive power command from wind control (pu on MBASE)
	STATUS(2) = 24	WAUXSG, output of wind auxiliary control (pu on MBASE)
	STATUS(3)	number of the bus to which the machine for which the quantity is to be placed in an channel is connected (no default allowed)
Character*2 ID	machine identifier (input; '1' by default).	
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	bus not found
	IERR = 4	machine not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MACHINE_ITERM_CHANNEL

Use this API to add an output channel containing the terminal current of a specified machine, along with a corresponding call to the GENTMC monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MACHINE_ITERM_CHANNEL  STATUS(1)..STATUS(4)  'ID'  'IDENT'
```

Python command syntax:

```
ierr = machine_iterm_channel(status, id, ident)
```

Fortran command syntax:

```
CALL MACHINE_ITERM_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (4)	array of 4 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the bus to which the machine for which the terminal current is to be placed in an channel is connected (no default allowed)
Character*2 ID	machine identifier (input; '1' by default).	
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed
	IERR = 5	model GENTMC needs 2 VARs but the last one exceeds the largest VAR index allowed
	IERR = 6	model GENTMC needs 2 ICONs but the last one exceeds the largest ICON index allowed
	IERR = 7	the maximum number of channel monitoring models has already been specified

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	bus not found
IERR = 9	machine not found
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.2.3. Assign Branch Quantities to Output Channels

BRANCH_APP_R_X_CHANNEL

Use this API to add a pair of output channels containing the apparent impedance, as seen at the from bus of a specified branch, along with a corresponding call to the RELAY2 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_BRANCH_APP_R_X_CHANNEL  STATUS(1)..STATUS(5)  'ID'  'IDENT(1)'
 'IDENT(2)'
```

Python command syntax:

```
ierr = branch_app_r_x_channel(status, id, ident)
```

Fortran command syntax:

```
CALL BRANCH_APP_R_X_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (5)	array of 5 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the branch for which apparent impedance is to be placed in channels (no default allowed)
	STATUS(5)	number of the to bus of the branch for which apparent impedance is to be placed in channels (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT (2)	two identifiers to be assigned to the two channels. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	model RELAY2 needs 3 ICONs but the last one exceeds the largest ICON index allowed
IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	model RELAY2 needs 2 VARs but the last one exceeds the largest VAR index allowed
IERR = 8	model RELAY2 needs 2 channels but the last one exceeds the largest channel number allowed
IERR = 9	bus not found
IERR = 10	branch not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BRANCH_MVA_CHANNEL

Use this API to add an output channel containing the MVA flow at the from bus of a specified branch, along with a corresponding call to the FLOW1 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_BRANCH_MVA_CHANNEL  STATUS(1)..STATUS(5)  'ID'  'IDENT'
```

Python command syntax:

```
ierr = branch_mva_channel(status, id, ident)
```

Fortran command syntax:

```
CALL BRANCH_MVA_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (5)	array of 5 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the branch for which MVA flow is to be placed in an channel (no default allowed)
	STATUS(5)	number of the to bus of the branch for which MVA is to be placed in an channel (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed
	IERR = 5	model FLOW1 needs 3 ICONs but the last one exceeds the largest ICON index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	bus not found
IERR = 8	branch not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BRANCH_P_AND_Q_CHANNEL

Use this API to add a pair of output channels containing the active and reactive power flow at the from bus of a specified branch, along with a corresponding call to the FLOW1 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_BRANCH_P_AND_Q_CHANNEL  STATUS(1)..STATUS(5)  'ID'  'IDENT(1)'
 'IDENT(2)'
```

Python command syntax:

```
ierr = branch_p_and_q_channel(status, id, ident)
```

Fortran command syntax:

```
CALL BRANCH_P_AND_Q_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (5)	array of 5 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the branch for which active and reactive power flows are to be placed in channels (no default allowed)
	STATUS(5)	number of the to bus of the branch for which active and reactive power flows are to be placed in channels (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT (2)	two identifiers to be assigned to the two channels. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed
	IERR = 5	model FLOW1 needs 3 ICONs but the last one exceeds the largest ICON index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	model FLOW1 needs 2 VARs but the last one exceeds the largest VAR index allowed
IERR = 8	model FLOW1 needs 2 channels but the last one exceeds the largest channel number allowed
IERR = 9	bus not found
IERR = 10	branch not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BRANCH_P_CHANNEL

Use this API to add an output channel containing the active power flow at the from bus of a specified branch, along with a corresponding call to the FLOW1 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_BRANCH_P_CHANNEL  STATUS(1)..STATUS(5)  'ID'  'IDENT'
```

Python command syntax:

```
ierr = branch_p_channel(status, id, ident)
```

Fortran command syntax:

```
CALL BRANCH_P_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (5)	array of 5 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the branch for which active power flow is to be placed in an channel (no default allowed)
	STATUS(5)	number of the to bus of the branch for which active power flow is to be placed in an channel (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed
	IERR = 5	model FLOW1 needs 3 ICONs but the last one exceeds the largest ICON index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	bus not found
IERR = 8	branch not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

THREE_WND_APP_R_X_CHANNEL

Use this API to add a pair of output channels containing the apparent impedance as seen at the from bus of a specified three-winding transformer, along with a corresponding call to the RELAY3 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_THREE_WND_APP_R_X_CHANNEL  STATUS(1)..STATUS(6)  'ID'  'IDENT(1)'
 'IDENT(2)'
```

Python command syntax:

```
ierr = three_wnd_app_r_x_channel(status, id, ident)
```

Fortran command syntax:

```
CALL THREE_WINDING_APP_R_X_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (6)	array of 6 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the three-winding transformer for which apparent impedance is to be placed in channels (no default allowed)
	STATUS(5)	number of one of the to buses of the three-winding transformer for which apparent impedance is to be placed in channels (no default allowed)
	STATUS(6)	number of the third bus of the three-winding transformer for which apparent impedance is to be placed in channels (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT (2)	two identifiers to be assigned to the two channels. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	starting ICON index is greater than the largest ICON index allowed
IERR = 5	model RELAY3 needs 4 ICONs but the last one exceeds the largest ICON index allowed
IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	model RELAY3 needs 2 VARs but the last one exceeds the largest VAR index allowed
IERR = 8	model RELAY3 needs 2 channels but the last one exceeds the largest channel number allowed
IERR = 9	bus not found
IERR = 10	branch not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

THREE_WND_MVA_CHANNEL

Use this API to add an output channel containing the MVA flow at the from bus of a specified three-winding transformer, along with a corresponding call to the FLOW3 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_THREE_WND_MVA_CHANNEL  STATUS(1)..STATUS(6)  'ID'  'IDENT'
```

Python command syntax:

```
ierr = three_wnd_mva_channel(status, id, ident)
```

Fortran command syntax:

```
CALL THREE_WINDING_MVA_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (6)	array of 6 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the three-winding transformer for which MVA flow is to be placed in an channel (no default allowed)
	STATUS(5)	number of one of the to buses of the three-winding transformer for which MVA flow is to be placed in an channel (no default allowed)
	STATUS(6)	number of the third bus of the three-winding transformer for which MVA flow is to be placed in an channel (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	starting ICON index is greater than the largest ICON index allowed
IERR = 5	model FLOW3 needs 4 ICONs but the last one exceeds the largest ICON index allowed
IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	bus not found
IERR = 8	branch not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

THREE_WND_P_AND_Q_CHANNEL

Use this API to add a pair of output channels containing the active and reactive power flow at the from bus of a specified three-winding transformer, along with a corresponding call to the FLOW3 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_THREE_WND_P_AND_Q_CHANNEL  STATUS(1)..STATUS(6)  'ID'  'IDENT(1)'
 'IDENT(2)'
```

Python command syntax:

```
ierr = three_wnd_p_and_q_channel(status, id, ident)
```

Fortran command syntax:

```
CALL THREE_WINDING_P_AND_Q_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (6)	array of 6 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the three-winding transformer for which active and reactive power flows are to be placed in channels (no default allowed)
	STATUS(5)	number of one of the to buses of the three-winding transformer for which active and reactive power flows are to be placed in channels (no default allowed)
	STATUS(6)	number of the third bus of the three-winding transformer for which active and reactive power flows are to be placed in channels (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT (2)	two identifiers to be assigned to the two channels. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	starting VAR index is greater than the largest VAR index allowed
IERR = 4	starting ICON index is greater than the largest ICON index allowed
IERR = 5	model FLOW3 needs 4 ICONs but the last one exceeds the largest ICON index allowed
IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	model FLOW3 needs 2 VARs but the last one exceeds the largest VAR index allowed
IERR = 8	model FLOW3 needs 2 channels but the last one exceeds the largest channel number allowed
IERR = 9	bus not found
IERR = 10	branch not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

THREE_WND_P_CHANNEL

Use this API to add an output channel containing the active power flow at the from bus of a specified three-winding transformer, along with a corresponding call to the FLOW3 monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_THREE_WND_P_CHANNEL  STATUS(1)..STATUS(6)  'ID'  'IDENT'
```

Python command syntax:

```
ierr = three_wnd_p_channel(status, id, ident)
```

Fortran command syntax:

```
CALL THREE_WINDING_P_CHANNEL(STATUS, ID, IDENT, IERR)
```

Where:

Integer STATUS (6)	array of 6 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the from bus of the three-winding transformer for which active power flow is to be placed in an channel (no default allowed)
	STATUS(5)	number of one of the to buses of the three-winding transformer for which active power flow is to be placed in an channel (no default allowed)
	STATUS(6)	number of the third bus of the three-winding transformer for which active power flow is to be placed in an channel (no default allowed)
Character*2 ID	circuit identifier (input; '1' by default).	
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	starting ICON index is greater than the largest ICON index allowed
IERR = 5	model FLOW3 needs 4 ICONs but the last one exceeds the largest ICON index allowed
IERR = 6	the maximum number of channel monitoring models has already been specified
IERR = 7	bus not found
IERR = 8	branch not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.2.4. Assign Other Quantities to Output Channels

BUS_FREQUENCY_CHANNEL

Use this API to add an output channel containing the per unit frequency deviation at a specified bus.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_BUS_FREQUENCY_CHANNEL  STATUS(1)  STATUS(2)  'IDENT'
```

Python command syntax:

```
ierr = bus_frequency_channel(status, ident)
```

Fortran command syntax:

```
CALL BUS_FREQUENCY_CHANNEL(STATUS, IDENT, IERR)
```

Where:

Integer STATUS (2)	array of 2 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	number of the bus for which frequency deviation is to be placed in an channel is connected (no default allowed)
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	bus not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

STATE_CHANNEL

Use this API to add an output channel containing the value of a specified STATE.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_STATE_CHANNEL  STATUS(1)  STATUS(2)  'IDENT'
```

Python command syntax:

```
ierr = state_channel(status, ident)
```

Fortran command syntax:

```
CALL STATE_CHANNEL(STATUS, IDENT, IERR)
```

Where:

Integer STATUS (2)	array of 2 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	index of the STATE for which the value is to be placed in an channel (no default allowed)
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	invalid STATE index
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

VAR_CHANNEL

Use this API to add an output channel containing the value of a specified VAR.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_VAR_CHANNEL  STATUS(1)  STATUS(2)  'IDENT'
```

Python command syntax:

```
ierr = var_channel(status, ident)
```

Fortran command syntax:

```
CALL VAR_CHANNEL(STATUS, IDENT, IERR)
```

Where:

Integer STATUS (2)	array of 2 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	index of the VAR for which the value is to be placed in an channel (no default allowed)
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	invalid VAR index
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

VOLTAGE_AND_ANGLE_CHANNEL

Use this API to add a pair of output channels containing the voltage magnitude in per unit and phase angle in degrees of a specified bus, along with a corresponding call to the VOLMAG monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_VOLTAGE_AND_ANGLE_CHANNEL  STATUS(1)..STATUS(4)  'IDENT(1)'
 'IDENT(2)'
```

Python command syntax:

```
ierr = voltage_and_angle_channel(status, ident)
```

Fortran command syntax:

```
CALL VOLTAGE_AND_ANGLE_CHANNEL(STATUS, IDENT, IERR)
```

Where:

Integer STATUS (4)	array of 4 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the bus for which voltage magnitude and phase angle are to be placed in channels (no default allowed)
Character*32 IDENT (2)	two identifiers to be assigned to the two channels. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed
	IERR = 5	the maximum number of channel monitoring models has already been specified
	IERR = 6	bus not found
	IERR = 7	model VOLMAG needs 2 VARs but the last one exceeds the largest VAR index allowed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	model VOLMAG needs 2 channels but the last one exceeds the largest channel number allowed
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

VOLTAGE_CHANNEL

Use this API to add an output channel containing the voltage magnitude in per unit of a specified bus, along with a corresponding call to the VOLMAG monitoring model.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_VOLTAGE_CHANNEL  STATUS(1)..STATUS(4)  'IDENT'
```

Python command syntax:

```
ierr = voltage_channel(status, ident)
```

Fortran command syntax:

```
CALL VOLTAGE_CHANNEL(STATUS, IDENT, IERR)
```

Where:

Integer STATUS (4)	array of 4 elements specifying (input).	
	STATUS(1)	starting channel index, or -1 for the next available (next available by default)
	STATUS(2)	starting VAR index, or -1 for the next available (next available by default)
	STATUS(3)	starting ICON index, or -1 for the next available (next available by default)
	STATUS(4)	number of the bus for which voltage magnitude is to be placed in an channel (no default allowed)
Character*32 IDENT	identifier to be assigned to the channel. If a blank channel identifier is specified, the API generates an appropriate channel identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid STATUS value
	IERR = 2	starting channel number is greater than the largest channel number allowed
	IERR = 3	starting VAR index is greater than the largest VAR index allowed
	IERR = 4	starting ICON index is greater than the largest ICON index allowed
	IERR = 5	the maximum number of channel monitoring models has already been specified
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3. Plant Related Models

8.3.1. ADD_PLANT_MODEL

Use this API to add a plant related model of a designated type to a specified machine. If a model of the designated type already exists at the machine, it is replaced.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ADD_PLANT_MODEL  IBUS  'ID'  MTYPE  'NAME'  IBUS2 'ID2'  NICN  IDATA
 CHDATA  NCON  RDATA
```

Python command syntax:

```
ierr = add_plant_model(ibus, id, mtype, name, ibus2, id2, nicn, idata,
 chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_PLANT_MODEL(IBUS, ID, MTYPE, NAME, IBUS2, ID2, NICN, IDATA,
 CHDATA, NCON, RDATA, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).
Character*2 ID	machine identifier (input; blank by default).
Integer MTYPE	model type (input; 1 by default). MTYPE = 1 generator model. MTYPE = 2 compensator model. MTYPE = 3 stabilizer model. MTYPE = 4 minimum excitation limiter model. MTYPE = 5 maximum excitation limiter model. MTYPE = 6 excitation system model. MTYPE = 7 turbine-governor model. MTYPE = 8 turbine-load controller model MTYPE = 9 stator current limiter model MTYPE = 10 power factor/var controller model
Character*16 NAME	model name (input; no default allowed).
Integer IBUS2	bus number of the second machine if this is a two machine model, or 0 for a single machine model (input; 0 by default).
Character*2 ID2	machine identifier of the second machine if this is a two machine model, or ' ' for a single machine model (input; blank by default).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer NCON	number of CONs required by the model (input; no default allowed).
Real RDATA (NCON)	CON data (input; 0.0).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	machine model connection tables full--use pack plant model tables function
IERR = 2	plant model tables are corrupt
IERR = 3	invalid MTYPE value
IERR = 4	unknown model NAME
IERR = 5	model is not consistent with MTYPE
IERR = 6	# CONs specified is not the number expected
IERR = 7	# ICONs specified is not the number expected
IERR = 8	second machine not found where one is required
IERR = 9	same machine specified as the first and second machines
IERR = 10	maximum number of array elements exceeded
IERR = 11	bus not found
IERR = 12	machine not found
IERR = 13	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.2. CHANGE_PLMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of a designated plant related model at a specified machine. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_PLMOD_CHRICN  IBUS  'ID'  'NAME'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_plmod_chricn(ibus, id, name, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_PLMOD_CHRICN(IBUS, ID, NAME, J, CHDATA, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a plant-related model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no input ICONs used in model NAME
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	bus not found
	IERR = 8	machine not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.3. CHANGE_PLMOD_CON

Use this API to change the value of a CON of a designated plant related model at a specified machine.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_PLMOD_CON  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_plmod_con(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_PLMOD_CON(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a plant-related model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no CONs used in model NAME
	IERR = 6	CON number outside of the CON range of this model
	IERR = 7	bus not found
	IERR = 8	machine not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.4. CHANGE_PLMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_PLMOD_CON, CHANGE_PLMOD_ICON, and CHANGE_PLMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of a plant related model of a designated type at a specified machine.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CHANGE_PLMOD_DATA  IBUS  'ID'  MTYPE  'NAME'  J  RDATA  I  IDATA
 'CHDATA'
```

Python command syntax:

```
ierr = change_plmod_data(ibus,id,mtype,name,j,rdata,i,idata,chdata)
```

Fortran command syntax:

```
CALL CHANGE_PLMOD_DATA(IBUS,ID,MTYPE,NAME,J,RDATA,I,IDATA,CHDATA,IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).
Character*2 ID	machine identifier (input; '1' by default).
Integer MTYPE	model type (input; 1 by default). MTYPE = 1 generator model. MTYPE = 2 compensator model. MTYPE = 3 stabilizer model. MTYPE = 4 minimum excitation limiter model. MTYPE = 5 maximum excitation limiter model. MTYPE = 6 excitation system model. MTYPE = 7 turbine-governor model. MTYPE = 8 turbine-load controller model.
Character*16 NAME	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).
Real RDATA	new CON value (input; unchanged).
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value,

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer IERR	error code (output).
	IERR = 0 no error
	IERR = 1 invalid MTYPE value
	IERR = 2 unknown model NAME
	IERR = 3 no model of type MTYPE at this machine
	IERR = 4 type MTYPE model at this machine is aaa not xxx
	IERR = 5 CON number exceeds the number of CONs used by the model
	IERR = 6 ICON number exceeds the number of ICONs used by the model
	IERR = 7 neither the CON nor ICON number is positive
	IERR = 8 bus not found
	IERR = 9 machine not found
	IERR = 10 prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.5. CHANGE_PLMOD_ICON

Use this API to change the value of an integer ICON of a designated plant related model at a specified machine. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_PLMOD_ICON  IBUS  'ID'  'NAME'  J  IDATA
```

Python command syntax:

```
ierr = change_plmod_icon(ibus, id, name, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_PLMOD_ICON(IBUS, ID, NAME, J, IDATA, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a plant-related model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no input ICONs used in model NAME
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	bus not found
	IERR = 8	machine not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.6. CHANGE_PLMOD_VAR

Use this API to change the value of a VAR of a designated plant related model at a specified machine.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_PLMOD_VAR  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_plmod_var(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_PLMOD_VAR(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a plant-related model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no VARs used in model NAME
	IERR = 6	VAR number outside of the VAR range of this model
	IERR = 7	bus not found
	IERR = 8	machine not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.7. GMB_ADD_PLANT_MODEL

Use this API to add a plant related GMB model to a specified machine. If a model of the designated type already exists at the machine, it is replaced.

This API routine was first introduced in release 32.0.2.

Batch command syntax:

```
BAT_GMB_ADD_PLANT_MODEL  IBUS  'ID'  'NAME'  IBUS2 'ID2'  NICN  IDATA
 CHDATA  NCON  RDATA
```

Python command syntax:

```
ierr = gmb_add_plant_model(ibus, id, name, ibus2, id2, nicn, idata,
 chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL GMB_ADD_PLANT_MODEL(IBUS, ID, NAME, IBUS2, ID2, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer IBUS2	bus number of the second machine if this is a two machine model, or 0 for a single machine model (input; 0 by default).	
Character*2 ID2	machine identifier of the second machine if this is a two machine model, or ' ' for a single machine model (input; blank by default).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	machine model connection tables full--use pack plant model tables function
	IERR = 2	plant model tables are corrupt
	IERR = 3	unlicensed GMB model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	error reading GMB model
IERR = 5	GMB model is not a plant type model
IERR = 6	# CONs specified is not the number expected
IERR = 7	# ICONs specified is not the number expected
IERR = 8	second machine not found where one is required
IERR = 9	same machine specified as the first and second machines
IERR = 10	maximum number of array elements exceeded
IERR = 11	number of user-defined models exceeds maximum allowed
IERR = 12	bus not found
IERR = 13	machine not found
IERR = 14	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.8. PLMOD_CONSISTENCY

Use this API to check consistency among the plant related models referenced at each machine.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PLMOD_CONSISTENCY
```

Python command syntax:

```
ierr = plmod_consistency()
```

Fortran command syntax:

```
CALL PLMOD_CONSISTENCY(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.9. PLMOD_PACK

Use this API to remove entries that are marked as unused from the plant model connection tables and the plant model array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PLMOD_PACK
```

Python command syntax:

```
ierr = plmod_pack()
```

Fortran command syntax:

```
CALL PLMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.10. PLMOD_REMOVE

Use this API to remove a plant related model of a designated type from a specified machine.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PLMOD_REMOVE  IBUS  'ID'  MTYPE
```

Python command syntax:

```
ierr = plmod_remove(ibus, id, mtype)
```

Fortran command syntax:

```
CALL PLMOD_REMOVE(IBUS, ID, MTYPE, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; blank by default).	
Integer MTYPE	model type (input; 1 by default).	
	MTYPE = 1	generator model
	MTYPE = 2	compensator model
	MTYPE = 3	stabilizer model
	MTYPE = 4	minimum excitation limiter model
	MTYPE = 5	maximum excitation limiter model
	MTYPE = 6	excitation system model
	MTYPE = 7	turbine-governor model
	MTYPE = 8	turbine-load controller model
	MTYPE = 9	stator current limiter model
	MTYPE = 10	power factor/var controller model
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid MTYPE value
	IERR = 2	no model of type MTYPE at this machine
	IERR = 3	bus not found
	IERR = 4	machine not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.11. PLMOD_STATUS

Use this API to change the status of a plant related model of a designated type at a specified machine.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PLMOD_STATUS  IBUS  'ID'  MTYPE  STATUS
```

Python command syntax:

```
ierr = plmod_status(ibus, id, mtype, status)
```

Fortran command syntax:

```
CALL PLMOD_STATUS(IBUS, ID, MTYPE, STATUS, IERR)
```

Where:

Integer IBUS	plant bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; blank by default).	
Integer MTYPE	model type (input; 1 by default).	
	MTYPE = 1	generator model
	MTYPE = 2	compensator model
	MTYPE = 3	stabilizer model
	MTYPE = 4	minimum excitation limiter model
	MTYPE = 5	maximum excitation limiter model
	MTYPE = 6	excitation system model
	MTYPE = 7	turbine-governor model
	MTYPE = 8	turbine-load controller model
	MTYPE = 9	stator current limiter model
	MTYPE = 10	power factor/var controller model
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid MTYPE value
	IERR = 2	no model of type MTYPE at this machine
	IERR = 3	invalid STATUS value
	IERR = 4	bus not found
	IERR = 5	machine not found
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.12. PLMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those plant related models that are assigned to machines that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PLMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = plmod_unconnected(opt)
```

Fortran command syntax:

```
CALL PLMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected plant models
	OPT = 1	remove unconnected plant models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.3.13. PLMOD_USER

Use this API to list user-written plant model definitions or to remove user-written plant model definitions that are not assigned to any machines (unused) from the user model definition tables.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_PLMOD_USER  OPT
```

Python command syntax:

```
ierr = plmod_user(opt)
```

Fortran command syntax:

```
CALL PLMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written plant model definitions
	OPT = 1	remove unused user-written plant model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4. Wind Related Models

8.4.1. ADD_WIND_MODEL

Use this API to add a renewable related model of a designated type to a specified renewable machine. If a model of the designated type already exists at the machine, it is replaced.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_ADD_WIND_MODEL  IBUS  'ID'  MTYPE  'NAME'  NICN  IDATA  CHDATA  NCON
 RDATA
```

Python command syntax:

```
ierr = add_wind_model(ibus, id, mtype, name, nicn, idata, chdata, ncon,
 rdata)
```

Fortran command syntax:

```
CALL ADD_WIND_MODEL(IBUS, ID, MTYPE, NAME, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)
```

Where:

Integer IBUS	renewable machine bus number (input; no default allowed).
Character*2 ID	renewable machine identifier (input; blank by default).
Integer MTYPE	model type (input; 1 by default).MTYPE = 1 wind generator model). MTYPE = 2 renewable electrical control model MTYPE = 3 renewable mechanical system model MTYPE = 4 renewable pitch control model MTYPE = 5 renewable aerodynamic model MTYPE = 6 renewable gust model MTYPE = 7 renewable auxiliary control model MTYPE = 8 renewable torque control model MTYPE = 9 renewable weak grid control model MTYPE = 10 renewable paux control model
Character*16 NAME	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	renewable machine model connection tables full--use pack renewable model tables function
	IERR = 2	renewable model tables are corrupt
	IERR = 3	invalid MTYPE value
	IERR = 4	unknown model NAME
	IERR = 5	model is not consistent with MTYPE
	IERR = 6	# CONs specified is not the number expected
	IERR = 7	# ICONs specified is not the number expected
	IERR = 8	maximum number of array elements exceeded
	IERR = 9	machine is not designated as a renewable machine in power flow
	IERR = 10	bus not found
	IERR = 11	machine not found
	IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.2. CHANGE_WNMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of a designated renewable related model at a specified renewable machine. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGE_WNMOD_CHRICN  IBUS  'ID'  'NAME'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_wnmod_chricn(ibus, id, name, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_WNMOD_CHRICN(IBUS, ID, NAME, J, CHDATA, IERR)
```

Where:

Integer IBUS	renewable machine bus number (input; no default allowed).	
Character*2 ID	renewable machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a renewable machine model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no input ICONs used in model NAME
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	machine is not designated as a renewable machine in power flow
	IERR = 8	bus not found
	IERR = 9	machine not found
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.3. CHANGE_WNMOD_CON

Use this API to change the value of a CON of a designated renewable machine related model at a specified renewable machine.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGE_WNMOD_CON  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_wnmod_con(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_WNMOD_CON(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	renewable machine bus number (input; no default allowed).	
Character*2 ID	renewable machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a renewable machine model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no input CONs used in model NAME
	IERR = 6	CON number outside of the CON range of this model
	IERR = 7	machine is not designated as a renewable machine in power flow
	IERR = 8	bus not found
	IERR = 9	machine not found
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.4. CHANGE_WNMOD_ICON

Use this API to change the value of an integer ICON of a designated renewable machine related model at a specified renewable machine. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGE_WNMOD_ICON  IBUS  'ID'  'NAME'  J  IDATA
```

Python command syntax:

```
ierr = change_wnmod_icon(ibus, id, name, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_WNMOD_ICON(IBUS, ID, NAME, J, IDATA, IERR)
```

Where:

Integer IBUS	renewable machine bus number (input; no default allowed).	
Character*2 ID	renewable machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a renewable machine model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no input ICONs used in model NAME
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	machine is not designated as a renewable machine in power flow
	IERR = 8	bus not found
	IERR = 9	machine not found
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.5. CHANGE_WNMOD_VAR

Use this API to change the value of a VAR of a designated renewable machine related model at a specified renewable machine.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_WNMOD_VAR  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_wnmod_var(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_WNMOD_VAR(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	renewable machine bus number (input; no default allowed).	
Character*2 ID	renewable machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a renewable machine model
	IERR = 3	no <model type> model at this machine
	IERR = 4	the <model type> model at this machine is aaa not NAME
	IERR = 5	no input VARs used in model NAME
	IERR = 6	VAR number outside of the VAR range of this model
	IERR = 7	machine is not designated as a renewable machine in power flow
	IERR = 8	bus not found
	IERR = 9	machine not found
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.6. WNMOD_CONSISTENCY

Use this API to check consistency among the renewable machine related models referenced at each machine.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_WNMOD_CONSISTENCY
```

Python command syntax:

```
ierr = wnmod_consistency()
```

Fortran command syntax:

```
CALL WNMOD_CONSISTENCY(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.7. WNMOD_PACK

Use this API to remove entries that are marked as unused from the wind model connection tables and the renewable machine model array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_WNMOD_PACK
```

Python command syntax:

```
ierr = wnmod_pack()
```

Fortran command syntax:

```
CALL WNMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.8. WNMOD_REMOVE

Use this API to remove a renewable machine related model of a designated type from a specified renewable machine.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_WNMOD_REMOVE  IBUS  'ID'  MTYPE
```

Python command syntax:

```
ierr = wnmod_remove(ibus, id, mtype)
```

Fortran command syntax:

```
CALL WNMOD_REMOVE(IBUS, ID, MTYPE, IERR)
```

Where:

Integer IBUS	renewable machine bus number (input; no default allowed).	
Character*2 ID	renewable machine identifier (input; blank by default).	
Integer MTYPE	model type (input; 1 by default).	
	MTYPE = 1	renewable generator model
	MTYPE = 2	renewable electrical control model
	MTYPE = 3	renewable mechanical system model
	MTYPE = 4	renewable pitch control model
	MTYPE = 5	renewable aerodynamic model
	MTYPE = 6	renewable gust model
	MTYPE = 7	renewable auxiliary control model
	MTYPE = 8	renewable torque control model
	MTYPE = 9	renewable weak grid control model
	MTYPE = 10	renewable paux control model
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid MTYPE value
	IERR = 2	no model of type MTYPE at this machine
	IERR = 3	machine is not designated as a renewable machine in power flow
	IERR = 4	bus not found
	IERR = 5	machine not found
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.9. WNMOD_STATUS

Use this API to change the status of a renewable machine related model of a designated type at a specified renewable machine.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_WNMOD_STATUS  IBUS  'ID'  MTYPE  STATUS
```

Python command syntax:

```
ierr = wnmod_status(ibus, id, mtype, status)
```

Fortran command syntax:

```
CALL WNMOD_STATUS(IBUS, ID, MTYPE, STATUS, IERR)
```

Where:

Integer IBUS	renewable machine bus number (input; no default allowed).	
Character*2 ID	renewable machine identifier (input; blank by default).	
Integer MTYPE	model type (input; 1 by default).	
	MTYPE = 1	renewable generator model
	MTYPE = 2	renewable electrical control model
	MTYPE = 3	renewable mechanical system model
	MTYPE = 4	renewable pitch control model
	MTYPE = 5	renewable aerodynamic model
	MTYPE = 6	renewable gust model
	MTYPE = 7	renewable auxiliary control model
	MTYPE = 8	renewable torque control model
	MTYPE = 9	renewable weak grid control model
	MTYPE = 10	renewable paux control model
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid MTYPE value
	IERR = 2	no model of type MTYPE at this machine
	IERR = 3	invalid STATUS value
	IERR = 4	machine is not designated as a renewable machine in power flow
	IERR = 5	bus not found
	IERR = 6	machine not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.10. WNMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those renewable machine related models that are assigned to renewable machines that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_WNMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = wnmod_unconnected(opt)
```

Fortran command syntax:

```
CALL WNMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected renewable machine models
	OPT = 1	remove unconnected renewable machine models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.4.11. WNMOD_USER

Use this API to list user-written renewable machine model definitions or to remove user-written renewable machine model definitions that are not assigned to any renewable machines (unused) from the user model definition tables.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_WNMOD_USER  OPT
```

Python command syntax:

```
ierr = wnmod_user(opt)
```

Fortran command syntax:

```
CALL WNMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written renewable machine model definitions
	OPT = 1	remove unused user-written renewable machine model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5. Load Related Models

8.5.1. ADD_LOAD_MODEL

Use this API to add a load related model of a designated type to a specified load or subsystem. If a model of the designated type already exists at the load or subsystem, it is replaced.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ADD_LOAD_MODEL  IBUS  'ID'  MTYPE  LTYPE  'NAME'  NICN  IDATA  CHDATA
  NCON  RDATA
```

Python command syntax:

```
ierr = add_load_model(ibus, id, mtype, ltype, name, nicn, idata, chdata,
 ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_LOAD_MODEL(IBUS, ID, MTYPE, LTYPE, NAME, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when MTYPE is 4 (when MTYPE is less than 4) (input; no default allowed).
Character*2 ID	load identifier, or "*" to apply the model to all loads in the subsystem (input; blank by default).
Integer MTYPE	subsystem type (input; 0 by default). MTYPE = 0 bus model MTYPE = 1 owner model MTYPE = 2 zone model MTYPE = 3 area model MTYPE = 4 all model
Integer LTYPE	load model type (input; 1 by default). LTYPE = 1 load characteristic model LTYPE = 2 load relay model
Character*16 NAME	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NCON	number of CONs required by the model (input; no default allowed).
Real RDATA (NCON)	CON data (input; 0.0).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	load model connection tables full--use pack load model tables function
IERR = 2	invalid MTYPE value
IERR = 3	invalid LTYPE value
IERR = 4	unknown model NAME
IERR = 5	model is not consistent with LTYPE
IERR = 6	model is not consistent with MTYPE
IERR = 7	# CONs specified is not the number expected
IERR = 8	# ICONs specified is not the number expected
IERR = 9	maximum number of array elements exceeded
IERR = 10	type MTYPE load model table is full
IERR = 11	no loads with the specified load identifier
IERR = 12	invalid area/owner/zone number
IERR = 13	no loads with the specified load identifier for specified subsystem
IERR = 14	no loads at bus
IERR = 15	bus not found
IERR = 16	load not found
IERR = 17	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.2. CHANGE_LDMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of a load related model at a specified load or subsystem. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_LDMOD_CHRICN  IBUS  'ID'  'NAME'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_ldmod_chricn(ibus, id, name, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_LDMOD_CHRICN(IBUS, ID, NAME, J, CHDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load-related model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no all type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	all type <load type> model for this id is aaa not xxx
	IERR = 7	no input ICONs used in model NAME
	IERR = 8	ICON number outside of the ICON range of this model
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.3. CHANGE_LDMOD_CON

Use this API to change the value of a CON of a load related model at a specified load or subsystem.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_LDMOD_CON  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_ldmod_con(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_LDMOD_CON(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load-related model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no all type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	all type <load type> model for this id is aaa not xxx
	IERR = 7	no CONs used in model NAME
	IERR = 8	CON number outside of the CON range of this model
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.4. CHANGE_LDMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_LDMOD_CON, CHANGE_LDMOD_ICON, and CHANGE_LDMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of a load related model of a designated type at a specified load or subsystem.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CHANGE_LDMOD_DATA  IBUS  'ID'  MTYPE  LTYPE  'NAME'  J  RDATA  I
 IDATA  'CHDATA'
```

Python command syntax:

```
ierr = change_ldmod_data(ibus,id,mtype,ltype,name,j,rdata,i,idata,chdata)
```

Fortran command syntax:

```
CALL CHANGE_LDMOD_DATA(IBUS, ID, MTYPE, LTYPE, NAME, J, RDATA, I, IDATA,
 CHDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when MTYPE is 4 (when MTYPE is less than 4) (input; no default allowed).
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).
Integer MTYPE	subsystem type (input; 0 by default). MTYPE = 0 bus model MTYPE = 1 owner model MTYPE = 2 zone model MTYPE = 3 area model MTYPE = 4 all model
Integer LTYPE	load model type (input; 1 by default). LTYPE = 1 load characteristic model LTYPE = 2 load relay model
Character*16 NAME	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).
Real RDATA	new CON value (input; unchanged).
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	invalid MTYPE value
IERR = 2	invalid LTYPE value
IERR = 3	unknown model NAME
IERR = 4	no all type LTYPE model for this id in tables
IERR = 5	no STYPE type LTYPE model for this id in tables for MTYPE IBUS
IERR = 6	all type LTYPE model for this id is aaa not xxx
IERR = 7	MTYPE type LTYPE model for this id for MTYPE IBUS is aaa not xxx
IERR = 8	CON number exceeds the number of CONs used by the model
IERR = 9	ICON number exceeds the number of input ICONs used by the model
IERR = 10	neither the CON nor ICON number is positive
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.5. CHANGE_LDMOD_ICON

Use this API to change the value of an integer ICON of a load related model at a specified load or subsystem. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_LDMOD_ICON  IBUS  'ID'  'NAME'  J  IDATA
```

Python command syntax:

```
ierr = change_ldmod_icon(ibus, id, name, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_LDMOD_ICON(IBUS, ID, NAME, J, IDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load-related model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no all type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	all type <load type> model for this id is aaa not xxx
	IERR = 7	no input ICONs used in model NAME
	IERR = 8	ICON number outside of the ICON range of this model
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.6. CHANGE_LDMOD_VAR

Use this API to change the value of a VAR of a load related model at a specified load or subsystem. This API can be used only for bus based load models. It cannot be used for changing VARs of owner, zone, area, and "all" type load models.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_LDMOD_VAR  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_ldmod_var(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_LDMOD_VAR(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	load bus number (input).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load-related model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no "all" type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	"all" type <load type> model for this id is aaa not xxx
	IERR = 7	no VARs used in model NAME
	IERR = 8	VAR number outside of the VAR range of this model
	IERR = 9	VARs of only bus based (load type) models can be changed using this API
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.7. LDMODC_PACK

Use this API to remove entries that are marked as unused from the load component model connection tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_LDMODC_PACK
```

Python command syntax:

```
ierr = ldmodc_pack()
```

Fortran command syntax:

```
CALL LDMODC_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.8. LDMODC_REMOVE

Use this API to remove a load component related model of a designated type from a specified load or subsystem.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_LDMODC_REMOVE STYPE IBUS 'LTYPNAME' 'ID' LTYPE
```

Python command syntax:

```
ierr = ldmodc_remove(stype, ibus, ltypname, id, ltype)
```

Fortran command syntax:

```
CALL LDMODC_REMOVE(STYPE, IBUS, LTYPNAME, ID, LTYPE, IERR)
```

Where:

Integer STYPE	subsystem type (input; 0 by default).	
	STYPE = 0	bus model.
	STYPE = 1	owner model.
	STYPE = 2	zone model.
	STYPE = 3	area model.
	STYPE = 4	all model.
Integer IBUS	bus, owner, zone or area number. Ignored when MTYPE is 4 (when MTYPE is less than 4) (input; no default allowed).	
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Integer LTYPE	load model type (input; 1 by default).	
	LTYPE = 11	load main component model
	LTYPE = 12	load distribution component model
	LTYPE = 21	Component # 1 load Individual component model
	LTYPE = 22	Component # 2 load Individual component model
	LTYPE = 23	Component # 3 load Individual component model
	LTYPE = 24	Component # 4 load Individual component model
	LTYPE = 25	Component # 5 load Individual component model
	LTYPE = 26	Component # 6 load Individual component model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LTYPE = 27	Component # 7 load Individual component model
	LTYPE = 28	Component # 8 load Individual component model
	LTYPE = 29	Component # 9 load Individual component model
	LTYPE = 30	Component # 10 load Individual component model
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid number specified for IBUS
	IERR = 2	invalid LTYPE value
	IERR = 3	invalid STYPE value
	IERR = 4	no all type LTYPE model for load id
	IERR = 5	no STYPE type LTYPE model for load id for MTYPE IBUS
	IERR = 6	no all type LTYPE model at component # for load id
	IERR = 7	no STYPE type LTYPE model at component # for load id for MTYPE IBUS
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.9. LDMODC_STATUS

Use this API to change the status of a load related model of a designated type for all subsystem loads at which it is applied.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_LDMODC_STATUS MTYPE IBUS 'LTYPNAME' 'ID' LTYPE STATUS
```

Python command syntax:

```
ierr = ldmodc_status(mtype, ibus, ltypname, id, ltype, status)
```

Fortran command syntax:

```
CALL LDMODC_STATUS(MTYPE, IBUS, LTYPNAME, ID, LTYPE, STATUS, IERR)
```

Where:

Integer MTYPE	subsystem type (input; 0 by default).	
	MTYPE = 0	bus model
	MTYPE = 1	owner model
	MTYPE = 2	zone model
	MTYPE = 3	area model
	MTYPE = 4	all model
Integer IBUS	bus, owner, zone or area number. Ignored when MTYPE is 4 (when MTYPE is less than 4) (input; no default allowed).	
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Integer LTYPE	load model type (input; no default allowed).	
	LTYPE = 11	load main component model
	LTYPE = 12	load distribution component model
	LTYPE = 21	Component # 1 load Individual component model
	LTYPE = 22	Component # 2 load Individual component model
	LTYPE = 23	Component # 3 load Individual component model
	LTYPE = 24	Component # 4 load Individual component model
	LTYPE = 25	Component # 5 load Individual component model
	LTYPE = 26	Component # 6 load Individual component model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LTYPE = 27	Component # 7 load Individual component model
	LTYPE = 28	Component # 8 load Individual component model
	LTYPE = 29	Component # 9 load Individual component model
	LTYPE = 30	Component # 10 load Individual component model
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid number specified for IBUS
	IERR = 2	invalid LTYPE value
	IERR = 3	invalid MTYPE value
	IERR = 4	no all type LTYPE model for this id in tables
	IERR = 5	no MTYPE type LTYPE model for this id in tables for MTYPE IBUS
	IERR = 6	invalid STATUS value
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.10. LDMODC_UNCONNECTED

Use this API to list or remove from dynamics working memory those load Component models that are assigned to loads that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_LDMODC_UNCONNECTED OPT
```

Python command syntax:

```
ierr = ldmodc_unconnected(opt)
```

Fortran command syntax:

```
CALL LDMODC_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected load component models
	OPT = 1	remove unconnected load component models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.11. LDMODC_USER

Use this API to list user-written load component model definitions or to remove user-written load model definitions that are not assigned to any loads (unused) from the user model definition tables.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_LDMODC_USER OPT
```

Python command syntax:

```
ierr = ldmodc_user(opt)
```

Fortran command syntax:

```
CALL LDMODC_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written load model definitions
	OPT = 1	remove unused user-written load model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.12. LDMOD_PACK

Use this API to remove entries that are marked as unused from the load model connection tables and the load model array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LDMOD_PACK
```

Python command syntax:

```
ierr = ldmod_pack()
```

Fortran command syntax:

```
CALL LDMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.13. LDMOD_REMOVE

Use this API to remove a load related model of a designated type from a specified load or subsystem.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LDMOD_REMOVE  MTYPE  IBUS  'ID'  LTYPE
```

Python command syntax:

```
ierr = ldmod_remove(mtype, ibus, id, ltype)
```

Fortran command syntax:

```
CALL LDMOD_REMOVE(MTYPE, IBUS, ID, LTYPE, IERR)
```

Where:

Integer MTYPE	subsystem type (input; 0 by default).	
	MTYPE = 0	bus model.
	MTYPE = 1	owner model.
	MTYPE = 2	zone model.
	MTYPE = 3	area model.
	MTYPE = 4	all model.
Integer IBUS	bus, owner, zone or area number. Ignored when MTYPE is 4 (when MTYPE is less than 4) (input; no default allowed).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Integer LTYPE	load model type (input; 1 by default).	
	LTYPE = 1	load characteristic model
	LTYPE = 2	load relay model
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid number specified for IBUS
	IERR = 2	invalid LTYPE value
	IERR = 3	invalid MTYPE value
	IERR = 4	no all type LTYPE model for this id in tables
	IERR = 5	no MTYPE type LTYPE model for this id in tables for MTYPE IBUS
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.14. LDMOD_STATUS

Use this API to change the status of a load related model of a designated type for all subsystem loads at which it is applied.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LDMOD_STATUS  MTYPE  IBUS  'ID'  LTYPE  STATUS
```

Python command syntax:

```
ierr = ldmod_status(mtype, ibus, id, ltype, status)
```

Fortran command syntax:

```
CALL LDMOD_STATUS(MTYPE, IBUS, ID, LTYPE, STATUS, IERR)
```

Where:

Integer MTYPE	subsystem type (input; 0 by default).	
	MTYPE = 0	bus model
	MTYPE = 1	owner model
	MTYPE = 2	zone model
	MTYPE = 3	area model
	MTYPE = 4	all model
Integer IBUS	bus, owner, zone or area number. Ignored when MTYPE is 4 (when MTYPE is less than 4) (input; no default allowed).	
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).	
Integer LTYPE	load model type (input; 1 by default).	
	LTYPE = 1	load characteristic model
	LTYPE = 2	load relay model
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid number specified for IBUS
	IERR = 2	invalid LTYPE value
	IERR = 3	invalid MTYPE value
	IERR = 4	no all type LTYPE model for this id in tables
	IERR = 5	no MTYPE type LTYPE model for this id in tables for MTYPE IBUS
	IERR = 6	invalid STATUS value
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.15. LDMOD_STATUS2

Use this API to change the status of a load related model of a designated type at a specified load.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LDMOD_STATUS2  IBUS  'ID'  LTYPE  STATUS
```

Python command syntax:

```
ierr = ldmod_status2(ibus, id, ltype, status)
```

Fortran command syntax:

```
CALL LDMOD_STATUS2(IBUS, ID, LTYPE, STATUS, IERR)
```

Where:

Integer IBUS	load bus number (input; no default allowed).	
Character*2 ID	load identifier (input; '1' by default).	
Integer LTYPE	load model type (input; 1 by default).	
	LTYPE = 1	load characteristic model
	LTYPE = 2	load relay model
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid LTYPE value
	IERR = 2	invalid STATUS value
	IERR = 3	no LTYPE model at this load
	IERR = 4	bus not found
	IERR = 5	load not found
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.16. LDMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those load related models that are assigned to loads that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LDMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = ldmod_unconnected(opt)
```

Fortran command syntax:

```
CALL LDMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected load models
	OPT = 1	remove unconnected load models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.5.17. LDMOD_USER

Use this API to list user-written load model definitions or to remove user-written load model definitions that are not assigned to any loads (unused) from the user model definition tables.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_LDMOD_USER  OPT
```

Python command syntax:

```
ierr = ldmod_user(opt)
```

Fortran command syntax:

```
CALL LDMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written load model definitions
	OPT = 1	remove unused user-written load model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.6. Load Component Models

8.6.1. ADD_LOADC_MODEL

Use this API to add a load component model of a designated type to a specified load or subsystem. If a component model of the designated type already exists at the load or subsystem, it is replaced.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_ADD_LOADC_CMODEL IBUS 'LTYPNAME' 'ID' 'NAME' MCOMP NICN IDATA CHDATA
 NCON RDATA
```

Python command syntax:

```
ierr = add_loadc_model(ibus, ltypname, id, name, mcomp, nicn, idata,
 chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_LOADC_MODEL(IBUS, LTYPNAME, ID, NAME, MCOMP, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).
Character*16 NAME	model name (input; no default allowed).
Integer MCOMP	Model Component number (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer NCON	number of CONs required by the model (input; no default allowed).
Real RDATA (NCON)	CON data (input; 0.0).
Integer IERR	error code (output). IERR = 0 no error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	load model connection tables full--use pack load model tables function
IERR = 2	model NAME is not a load component model
IERR = 3	invalid Load Model <NAME>(5th and 6th chacaracters in Model Name must be BL/OW/ZN/AR/AL/LT)
IERR = 4	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem> IBUS, load id <ID>
IERR = 5	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem type> IBUS, load id <ID>
IERR = 6	unknown model NAME
IERR = 7	model NAME is not a load component model
IERR = 8	model NAME is not a <STYPE> model
IERR = 9	# CONs specified is not the number expected
IERR = 10	# ICONs specified is not the number expected
IERR = 11	maximum number of array elements exceeded
IERR = 12	<STYPE> type load model table is full
IERR = 13	no loads with the specified load identifier
IERR = 14	invalid area or owner or zone number
IERR = 15	no loads with the specified load identifier
IERR = 16	no loads at bus
IERR = 17	bus not found
IERR = 18	load not found
IERR = 19	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.6.2. CHANGE_LDMODC_CDESC

Use this API to change the value of a Model Description of a load component model at a specified load or subsystem).

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_CHANGE_LDMODC_CDESC  IBUS  'LTYPENAME' 'ID'  'NAME' MCOMP 'DESC'
```

Python command syntax:

```
ierr = change_ldmodc_cdesc(ibus, ltypname,id, name, mcomp, desc)
```

Fortran command syntax:

```
CALL CHANGE_LDMODC_CDESC(IBUS, LTYPNAME, ID, NAME, MCOMP, DESC, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).
Character*16 NAME	model name (input; no default allowed).
Integer MCOMP	Model Individual Component number (input; no default allowed).

Mcomp = 21	Component # 1 load Individual component model
Mcomp = 22	Component # 2 load Individual component model
Mcomp = 23	Component # 3 load Individual component model
Mcomp = 24	Component # 4 load Individual component model
Mcomp = 25	Component # 5 load Individual component model
Mcomp = 26	Component # 6 load Individual component model
Mcomp = 27	Component # 7 load Individual component model
Mcomp = 28	Component # 8 load Individual component model
Mcomp = 29	Component # 9 load Individual component model
Mcomp = 30	Component # 10 load Individual component model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*32 DESC	component model description (input; blank by default).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	unknown model NAME
IERR = 2	model NAME is not a load component model
IERR = 3	no <subsystem type> type <load type> model for load at <subsystem type> IBUS load id ID Component number Mcomp
IERR = 4	no All type <load type> model for load id ID Component number Mcomp
IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS component number Mcomp is aaa not xxx
IERR = 6	all type <load type> model for this id component number Mcomp is aaa not xxx
IERR = 7	Model <NAME> applied to load <subsystem type> IBUS load id <ID> is a <load type> model; component description does not apply to this model type
IERR = 8	Model <NAME> applied to load id <ID> is a <load type> model; component description does not apply to this model type
IERR = 9	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem> IBUS, load id <ID>
IERR = 10	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem type> IBUS, load id <ID>
IERR = 11	invalid Load Model <NAME>(5th and 6th chacaracters in Model Name must be BL/OW/ZN/AR/AL/LT)
IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.6.3. CHANGE_LDMODC_CHRICN

Use this API to change the value of a CHRICN of a load component model at a specified load or subsystem).

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_CHANGE_LDMODC_CHRICN IBUS 'LTYPNAME' 'ID' 'NAME' MCOMP J CHDATA
```

Python command syntax:

```
ierr = change_ldmodc_chricn(ibus, ltypname,id, name, mcomp, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_LDMODC_CHRICN(IBUS, LTYPNAME, ID, NAME, MCOMP, J, CHDATA,
 IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).
Character*16 NAME	model name (input; no default allowed).
Integer MCOMP	Model Component number (input; no default allowed).

Mcomp = 11	load main component model
Mcomp = 12	load distribution component model
Mcomp = 21	Component # 1 load Individual component model
Mcomp = 22	Component # 2 load Individual component model
Mcomp = 23	Component # 3 load Individual component model
Mcomp = 24	Component # 4 load Individual component model
Mcomp = 25	Component # 5 load Individual component model
Mcomp = 26	Component # 6 load Individual component model
Mcomp = 27	Component # 7 load Individual component model
Mcomp = 28	Component # 8 load Individual component model
Mcomp = 29	Component # 9 load Individual component model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Mcomp = 30	Component # 10 load Individual component model
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(M), 2 for ICON(M+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (enter within quotes) (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load component model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no all type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	all type <load type> model for this id is aaa not xxx
	IERR = 7	no ICONs used in model NAME
	IERR = 8	ICON number outside of the ICON range of this model
	IERR = 9	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem> IBUS, load id <ID>
	IERR = 10	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem type> IBUS, load id <ID>
	IERR = 11	invalid Load Model <NAME>(5th and 6th chacaracters in Model Name must be BL/OW/ZN/AR/AL/LT)
	IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.6.4. CHANGE_LDMODC_CON

Use this API to change the value of a CON of a load component model at a specified load or subsystem.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_CHANGE_LDMODC_CON IBUS 'LTYPNAME' 'ID' 'NAME' MCOMP J RDATA
```

Python command syntax:

```
ierr = change_ldmodc_con(ibus, ltypname,id, name, mcomp, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_LDMODC_CON(IBUS, LTYPNAME, ID, NAME, MCOMP, J, RDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).
Character*16 NAME	model name (input; no default allowed).
Integer MCOMP	Model Component number (input; no default allowed).

Mcomp = 11	load main component model
Mcomp = 12	load distribution component model
Mcomp = 21	Component # 1 load Individual component model
Mcomp = 22	Component # 2 load Individual component model
Mcomp = 23	Component # 3 load Individual component model
Mcomp = 24	Component # 4 load Individual component model
Mcomp = 25	Component # 5 load Individual component model
Mcomp = 26	Component # 6 load Individual component model
Mcomp = 27	Component # 7 load Individual component model
Mcomp = 28	Component # 8 load Individual component model
Mcomp = 29	Component # 9 load Individual component model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Mcomp = 30	Component # 10 load Individual component model
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load component model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no all type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	all type <load type> model for this id is aaa not xxx
	IERR = 7	no CONs used in model NAME
	IERR = 8	CON number outside of the CON range of this model
	IERR = 9	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem> IBUS, load id <ID>
	IERR = 10	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem type> IBUS, load id <ID>
	IERR = 11	invalid Load Model <NAME>(5th and 6th chacaracters in Model Name must be BL/OW/ZN/AR/AL/LT)
	IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.6.5. CHANGE_LDMODC_ICON

Use this API to change the value of a ICON of a load component model at a specified load or subsystem.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_CHANGE_LDMODC_ICON  IBUS  'LTYPENAME' 'ID'  'NAME' MCOMP J  IDATA
```

Python command syntax:

```
ierr = change_ldmodc_icon(ibus, ltypname,id, name, mcomp, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_LDMODC_ICON(IBUS, LTYPNAME, ID, NAME, MCOMP, J, IDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).
Character*16 NAME	model name (input; no default allowed).
Integer MCOMP	Model Component number (input; no default allowed).

Mcomp = 11	load main component model
Mcomp = 12	load distribution component model
Mcomp = 21	Component # 1 load Individual component model
Mcomp = 22	Component # 2 load Individual component model
Mcomp = 23	Component # 3 load Individual component model
Mcomp = 24	Component # 4 load Individual component model
Mcomp = 25	Component # 5 load Individual component model
Mcomp = 26	Component # 6 load Individual component model
Mcomp = 27	Component # 7 load Individual component model
Mcomp = 28	Component # 8 load Individual component model
Mcomp = 29	Component # 9 load Individual component model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Mcomp = 30	Component # 10 load Individual component model
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(M), 2 for ICON(M+1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load component model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no all type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	all type <load type> model for this id is aaa not xxx
	IERR = 7	no ICONs used in model NAME
	IERR = 8	ICON number outside of the ICON range of this model
	IERR = 9	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem> IBUS, load id <ID>
	IERR = 10	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem type> IBUS, load id <ID>
	IERR = 11	invalid Load Model <NAME>(5th and 6th chacaracters in Model Name must be BL/OW/ZN/AR/AL/LT)
	IERR = 12	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.6.6. CHANGE_LDMODC_VAR

Use this API to change the value of a VAR of a load component model at a specified load or subsystem.

This API routine was first introduced in release 35.5.

Batch command syntax:

```
BAT_CHANGE_LDMODC_VAR IBUS 'LTYPNAME' 'ID' 'NAME' MCOMP J RDATA
```

Python command syntax:

```
ierr = change_ldmodc_var(ibus, ltypname,id, name, mcomp, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_LDMODC_VAR(IBUS, LTYPNAME, ID, NAME, MCOMP, J, RDATA, IERR)
```

Where:

Integer IBUS	bus, owner, zone or area number. Ignored when an all type load model is specified (unless an all type load model is specified) (input; no default allowed).
Character*12 LTYPNAME	load type, or " " if the model is not applied to applied to "Load Type"subsystem (input; blank by default).
Character*2 ID	load identifier, or "*" if the model is applied to all loads in the subsystem (input; blank by default).
Character*16 NAME	model name (input; no default allowed).
Integer MCOMP	Model Component number (input; no default allowed).

Mcomp = 11	load main component model
Mcomp = 12	load distribution component model
Mcomp = 21	Component # 1 load Individual component model
Mcomp = 22	Component # 2 load Individual component model
Mcomp = 23	Component # 3 load Individual component model
Mcomp = 24	Component # 4 load Individual component model
Mcomp = 25	Component # 5 load Individual component model
Mcomp = 26	Component # 6 load Individual component model
Mcomp = 27	Component # 7 load Individual component model
Mcomp = 28	Component # 8 load Individual component model
Mcomp = 29	Component # 9 load Individual component model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Mcomp = 30	Component # 10 load Individual component model
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	model NAME is not a load component model
	IERR = 3	no <subsystem type> type <load type> model for this id in tables for <subsystem type> IBUS
	IERR = 4	no all type <load type> model for this id in tables
	IERR = 5	<subsystem type> <load type> model for this id for <subsystem type> IBUS is aaa not xxx
	IERR = 6	all type <load type> model for this id is aaa not xxx
	IERR = 7	no VARs used in model NAME
	IERR = 8	VAR number outside of the VAR range of this model
	IERR = 9	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem> IBUS, load id <ID>
	IERR = 10	invalid Component number Mcomp for <load type> Model <NAME> for <subsystem type> IBUS, load id <ID>
	IERR = 11	invalid Load Model <NAME>(5th and 6th chacaracters in Model Name must be BL/OW/ZN/AR/AL/LT)
	IERR = 12	VARs of only bus based (load type) models can be changed using this API
	IERR = 13	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7. Line Relay Models

8.7.1. ADD_RELAY_MODEL

Use this API to add a line relay model in the designated relay slot of the from bus end of a specified branch. If a model already exists in the designated relay slot at the from bus end of the branch, it is replaced.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_ADD_RELAY_MODEL  IBUS  JBUS  'ID'  RS  'NAME'  NICN  IDATA  CHDATA
 NCON  RDATA
```

Python command syntax:

```
ierr = add_relay_model(ibus,jbus,id,rs,name,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL ADD_RELAY_MODEL(IBUS, JBUS, ID, RS, NAME, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; blank by default).	
Integer RS	relay slot (1 or 2) (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RS value
	IERR = 2	unknown model NAME
	IERR = 3	model is not a line relay model
	IERR = 4	# CONs specified is not the number expected
	IERR = 5	# ICONs specified is not the number expected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	maximum number of array elements exceeded
IERR = 7	line relay model connection tables full--use pack line relay model tables function
IERR = 8	line relay model tables are corrupt
IERR = 9	bus not found
IERR = 10	branch not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.2. CHANGE_RLMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the line relay model in the designated relay slot of the from bus end of a specified branch. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_RLMOD_CHRICN  IBUS  JBUS  'ID'  RS  'NAME'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_rlmod_chricn(ibus, jbus, id, rs, name, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_RLMOD_CHRICN(IBUS,JBUS,ID,RS,NAME,J,CHDATA,IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; blank by default).	
Integer RS	relay slot (1 or 2) (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RS value
	IERR = 2	unknown model NAME
	IERR = 3	no relay model on this branch at the bus IBUS end
	IERR = 4	relay model at bus IBUS end is aaa not xxx
	IERR = 5	no input ICONs used in model NAME
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	bus not found
	IERR = 8	branch not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.3. CHANGE_RLMOD_CON

Use this API to change the value of a CON of the line relay model in the designated relay slot of the from bus end of a specified branch.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_RLMOD_CON  IBUS  JBUS  'ID'  RS  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_rlmod_con(ibus, jbus, id, rs, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_RLMOD_CON(IBUS, JBUS, ID, RS, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; blank by default).	
Integer RS	relay slot (1 or 2) (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RS value
	IERR = 2	unknown model NAME
	IERR = 3	no relay model on this branch at the bus IBUS end
	IERR = 4	relay model at bus IBUS end is aaa not xxx
	IERR = 5	no CONs used in model NAME
	IERR = 6	CON number outside of the CON range of this model
	IERR = 7	bus not found
	IERR = 8	branch not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.4. CHANGE_RLMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_RLMOD_CON, CHANGE_RLMOD_ICON, and CHANGE_RLMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of the line relay model in the designated relay slot of the from bus end of a specified branch.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_CHANGE_RLMOD_DATA  IBUS  JBUS  'ID'  RS  'NAME'  J  RDATA  I  IDATA
 'CHDATA'
```

Python command syntax:

```
ierr = change_rlmod_data(ibus,jbus,id,rs,name,j,rdata,i,idata,chdata)
```

Fortran command syntax:

```
CALL CHANGE_RLMOD_DATA(IBUS, JBUS, ID, RS, NAME, J, RDATA, I, IDATA,
 CHDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).
Integer JBUS	to bus number (input; no default allowed).
Character*2 ID	circuit identifier (input; blank by default).
Integer RS	relay slot (1 or 2) (input; 1 by default).
Character*16 NAME	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).
Real RDATA	new CON value (input; unchanged).
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer IERR	error code (output). IERR = 0 no error IERR = 1 invalid RS value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	unknown model NAME
IERR = 3	no relay model on this branch at the bus IBUS end
IERR = 4	relay model at bus IBUS end is aaa not xxx
IERR = 5	CON number exceeds the number of CONs used by the model
IERR = 6	ICON number exceeds the number of ICONs used by the model
IERR = 7	neither the CON nor ICON number is positive
IERR = 8	bus not found
IERR = 9	branch not found
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.5. CHANGE_RLMOD_ICON

Use this API to change the value of an integer ICON of the line relay model in the designated relay slot of the from bus end of a specified branch. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_RLMOD_ICON  IBUS  JBUS  'ID'  RS  'NAME'  J  IDATA
```

Python command syntax:

```
ierr = change_rlmod_icon(ibus, jbus, id, rs, name, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_RLMOD_ICON(IBUS, JBUS, ID, RS, NAME, J, IDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; blank by default).	
Integer RS	relay slot (1 or 2) (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RS value
	IERR = 2	unknown model NAME
	IERR = 3	no relay model on this branch at the bus IBUS end
	IERR = 4	relay model at bus IBUS end is aaa not xxx
	IERR = 5	no input ICONs used in model NAME
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	bus not found
	IERR = 8	branch not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.6. CHANGE_RLMOD_VAR

Use this API to change the value of a VAR of the line relay model in the designated relay slot of the from bus end of a specified branch.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_RLMOD_VAR  IBUS  JBUS  'ID'  RS  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_rlmod_var(ibus, jbus, id, rs, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_RLMOD_VAR(IBUS, JBUS, ID, RS, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; blank by default).	
Integer RS	relay slot (1 or 2) (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RS value
	IERR = 2	unknown model NAME
	IERR = 3	no relay model on this branch at the bus IBUS end
	IERR = 4	relay model at bus IBUS end is aaa not xxx
	IERR = 5	no VARs used in model NAME
	IERR = 6	VAR number outside of the VAR range of this model
	IERR = 7	bus not found
	IERR = 8	branch not found
	IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.7. RLMOD_PACK

Use this API to remove entries that are marked as unused from the line relay model connection tables and the line relay model array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RLMOD_PACK
```

Python command syntax:

```
ierr = rlmod_pack()
```

Fortran command syntax:

```
CALL RLMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.8. RLMOD_REMOVE

Use this API to remove the line relay model in the designated relay slot of the from bus end of a specified branch.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RLMOD_REMOVE  IBUS  JBUS 'ID'  RS
```

Python command syntax:

```
ierr = rlmod_remove(ibus, jbus, id, rs)
```

Fortran command syntax:

```
CALL RLMOD_REMOVE(IBUS, JBUS, ID, RS, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer RS	relay slot (1 or 2) (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RS value
	IERR = 2	no relay model on this branch at the bus IBUS end
	IERR = 3	bus not found
	IERR = 4	branch not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.9. RLMOD_STATUS

Use this API to change the status of the line relay model in the designated relay slot of the from bus end of a specified branch.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RLMOD_STATUS  IBUS  JBUS  'ID'  RS  STATUS
```

Python command syntax:

```
ierr = rlmod_status(ibus, jbus, id, rs, status)
```

Fortran command syntax:

```
CALL RLMOD_STATUS(IBUS, JBUS, ID, RS, STATUS, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer RS	relay slot (1 or 2) (input; 1 by default).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid RS value
	IERR = 2	no relay model on this branch at the bus IBUS end
	IERR = 3	invalid STATUS value
	IERR = 4	bus not found
	IERR = 5	branch not found
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.10. RLMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those line relay models that are assigned to branches that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RLMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = rlmod_unconnected(opt)
```

Fortran command syntax:

```
CALL RLMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected line relay models
	OPT = 1	remove unconnected line relay models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.7.11. RLMOD_USER

Use this API to list user-written line relay model definitions or to remove user-written line relay model definitions that are not assigned to any branches (unused) from the user model definition tables.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_RLMOD_USER  OPT
```

Python command syntax:

```
ierr = rlmod_user(opt)
```

Fortran command syntax:

```
CALL RLMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written line relay definitions
	OPT = 1	remove unused user-written line relay definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8. DC Line Related Models

8.8.1. ADD_MTDCLINE_MODEL

Use this API to add a multi-terminal dc line model to the specified multi-terminal dc line. If a model already exists at the multi-terminal dc line, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_MTDCLINE_MODEL  DC  'MODEL'  NICN  IDATA  CHDATA  NCON  RDATA
 IERR
```

Python command syntax:

```
ierr = add_mtdcline_model(dc, model, nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_MTDCLINE_MODEL(NUM, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)

CALL ADD_MTDCLINE_MODEL_I(NUM, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)

CALL ADD_MTDCLINE_MODEL_C(NAME, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line to which the model is to be added; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line to which the model is to be added (input; no default allowed).
Character*12 NAME	name of the multi-terminal dc line to which the model is to be added (input; no default allowed).
Character*16 MODEL	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer NCON	number of CONs required by the model (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	multi-terminal dc line model connection tables full--use pack dc line model tables function
	IERR = 3	unknown model name
	IERR = 4	model is not a multi-terminal dc line model
	IERR = 5	# CONs specified is not the number expected
	IERR = 6	# ICONs specified is not the number expected
	IERR = 7	maximum number of array elements exceeded
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.2. ADD_TTDCLINE_MODEL

Use this API to add a two-terminal dc line model to the specified two-terminal dc line. If a model already exists at the two-terminal dc line, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_TTDCLINE_MODEL  DC  'MODEL'  NICN  IDATA  CHDATA  NCON  RDATA
 IERR
```

Python command syntax:

```
ierr = add_ttdcline_model(dc, model, nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_TTDCLINE_MODEL(NUM, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)

CALL ADD_TTDCLINE_MODEL_I(NUM, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)

CALL ADD_TTDCLINE_MODEL_C(NAME, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line to which the model is to be added; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line to which the model is to be added (input; no default allowed).
Character*12 NAME	name of the two-terminal dc line to which the model is to be added (input; no default allowed).
Character*16 MODEL	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer NCON	number of CONs required by the model (input; no default allowed).
Real RDATA (NCON)	CON data (input; 0.0).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error
IERR = 1	dc line not found
IERR = 2	two-terminal dc line model connection tables full--use pack dc line model tables function
IERR = 3	unknown model name
IERR = 4	model is not a two-terminal dc line model
IERR = 5	# CONs specified is not the number expected
IERR = 6	# ICONs specified is not the number expected
IERR = 7	maximum number of array elements exceeded
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.3. ADD_VSDCLINE_MODEL

Use this API to add a VSC dc line model to the specified VSC dc line. If a model already exists at the VSC dc line, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_VSDCLINE_MODEL  'DCNAME'  'MODEL'  NICN  IDATA  CHDATA  NCON
 RDATA  IERR
```

Python command syntax:

```
ierr = add_vsdcline_model(dcname,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL ADD_VSDCLINE_MODEL(DCNAME,MODEL,NICN,IDATA,CHDATA,NCON,RDATA,IERR)
```

Where:

Character*12 DCNAME	VSC dc line name (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	VSC dc line model connection tables full--use pack dc line model tables function
	IERR = 3	unknown model name
	IERR = 4	model is not a VSC dc line model
	IERR = 5	# CONs specified is not the number expected
	IERR = 6	# ICONs specified is not the number expected
	IERR = 7	maximum number of array elements exceeded
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.4. CHANGE_MTDCLMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the multi-terminal dc line model of a specified multi-terminal dc line. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCLMOD_CHRICN  DC  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_mtdclmod_chricn(dc, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCLMOD_CHRICN(NUM, MODEL, J, CHDATA, IERR)

CALL CHANGE_MTDCLMOD_CHRICN_I(NUM, MODEL, J, CHDATA, IERR)

CALL CHANGE_MTDCLMOD_CHRICN_C(NAME, MODEL, J, CHDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.5. CHANGE_MTDCLMOD_CON

Use this API to change the value of a CON of the multi-terminal dc line model of a specified multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCLMOD_CON  DC  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_mtdclmod_con(dc, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCLMOD_CON(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCLMOD_CON_I(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCLMOD_CON_C(NAME, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no CONs used in model
	IERR = 6	CON number outside of the CON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.6. CHANGE_MTDCLMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_MTDCLMOD_CON, CHANGE_MTDCLMOD_ICON, and CHANGE_MTDCLMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of the multi-terminal dc line model of a specified multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCLMOD_DATA  DC  'NAME'  J  RDATA  I  IDATA  'CHDATA'
```

Python command syntax:

```
ierr = change_mtdclmod_data(dc, name, j, rdata, i, idata, chdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCLMOD_DATA(DC, NAME, J, RDATA, I, IDATA, CHDATA, IERR)
```

Where:

Integer DC	multi-terminal dc line number (input; no default allowed).
Character*16 NAME	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).
Real RDATA	new CON value (input; unchanged).
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer IERR	error code (output).
	IERR = 1 dc line not found
	IERR = 2 unknown model name
	IERR = 3 no model at this dc line
	IERR = 4 model at this dc line is aaa not xxx
	IERR = 5 no CONs used in model; or
	IERR = 5 no input ICONs used in model
	IERR = 6 CON number outside of the CON range of this model; or

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	ICON number outside of the ICON range of this model
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.7. CHANGE_MTDCLMOD_ICON

Use this API to change the value of an integer ICON of the multi-terminal dc line model of a specified multi-terminal dc line. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCLMOD_ICON  DC  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_mtdclmod_icon(dc, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCLMOD_ICON(NUM, MODEL, J, IDATA, IERR)

CALL CHANGE_MTDCLMOD_ICON_I(NUM, MODEL, J, IDATA, IERR)

CALL CHANGE_MTDCLMOD_ICON_C(NAME, MODEL, J, IDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.8. CHANGE_MTDCLMOD_VAR

Use this API to change the value of a VAR of the multi-terminal dc line model of a specified multi-terminal dc line.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_MTDCLMOD_VAR  DC  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_mtdclmod_var(dc, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCLMOD_VAR(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCLMOD_VAR_I(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCLMOD_VAR_C(NAME, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no VARs used in model
	IERR = 6	VAR number outside of the VAR range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.9. CHANGE_TTDCLMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the two-terminal dc line model of a specified two-terminal dc line. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCLMOD_CHRICN  DC  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_ttdclmod_chricn(dc, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCLMOD_CHRICN(NUM, MODEL, J, CHDATA, IERR)

CALL CHANGE_TTDCLMOD_CHRICN_I(NUM, MODEL, J, CHDATA, IERR)

CALL CHANGE_TTDCLMOD_CHRICN_C(NAME, MODEL, J, CHDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.10. CHANGE_TTDCLMOD_CON

Use this API to change the value of a CON of the two-terminal dc line model of a specified two-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCLMOD_CON  DC  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_ttdclmod_con(dc, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCLMOD_CON(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCLMOD_CON_I(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCLMOD_CON_C(NAME, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no CONs used in model
	IERR = 6	CON number outside of the CON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.11. CHANGE_TTDCLMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_TTDCLMOD_CON, CHANGE_TTDCLMOD_ICON, and CHANGE_TTDCLMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of the two-terminal dc line model of a specified two-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCLMOD_DATA  IDC  'NAME'  J  RDATA  I  IDATA  'CHDATA'
```

Python command syntax:

```
ierr = change_ttdclmod_data(idc, name, j, rdata, i, idata, chdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCLMOD_DATA(IDC, NAME, J, RDATA, I, IDATA, CHDATA, IERR)
```

Where:

Integer IDC	two-terminal dc line number (input; no default allowed).
Character*16 NAME	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).
Real RDATA	new CON value (input; unchanged).
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer IERR	error code (output).
	IERR = 0 no error
	IERR = 1 dc line not found
	IERR = 2 unknown model name
	IERR = 3 no model at this dc line
	IERR = 4 model at this dc line is aaa not xxx
	IERR = 5 no CONs used in model; or
	IERR = 5 no input ICONs used in model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	CON number outside of the CON range of this model; or
IERR = 6	ICON number outside of the ICON range of this model
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.12. CHANGE_TTDCLMOD_ICON

Use this API to change the value of an integer ICON of the two-terminal dc line model of a specified two-terminal dc line. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCLMOD_ICON  DC  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_ttdclmod_icon(dc, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCLMOD_ICON(NUM, MODEL, J, IDATA, IERR)

CALL CHANGE_TTDCLMOD_ICON_I(NUM, MODEL, J, IDATA, IERR)

CALL CHANGE_TTDCLMOD_ICON_C(NAME, MODEL, J, IDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.13. CHANGE_TTDCLMOD_VAR

Use this API to change the value of a VAR of the two-terminal dc line model of a specified two-terminal dc line.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_TTDCLMOD_VAR  DC  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_ttdclmod_var(dc, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCLMOD_VAR(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCLMOD_VAR_I(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCLMOD_VAR_C(NAME, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no VARs used in model
	IERR = 6	VAR number outside of the VAR range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.14. CHANGE_VSDCLMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the VSC dc line model of a specified VSC dc line. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCLMOD_CHRICN  'DCNAME'  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_vsdclmod_chricn(dcname, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCLMOD_CHRICN(DCNAME, MODEL, J, CHDATA, IERR)
```

Where:

Character*12 DCNAME	VSC dc line name (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.15. CHANGE_VSDCLMOD_CON

Use this API to change the value of a CON of the VSC dc line model of a specified VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCLMOD_CON  'DCNAME'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_vsdclmod_con(dcname, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCLMOD_CON(DCNAME, MODEL, J, RDATA, IERR)
```

Where:

Character*12 DCNAME	VSC dc line name (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	VSC dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no CONs used in model
	IERR = 6	CON number outside of the CON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.16. CHANGE_VSDCLMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_VSDCLMOD_CON, CHANGE_VSDCLMOD_ICON, and CHANGE_VSDCLMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of the VSC dc line model of a specified VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCLMOD_DATA  'VSDCNM'  'NAME'  J  RDATA  I  IDATA  'CHDATA'
```

Python command syntax:

```
ierr = change_vsdclmod_data(vsdcnm,name,j,rdata,i,idata,chdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCLMOD_DATA(VSDCNM,NAME,J,RDATA,I,IDATA,CHDATA,IERR)
```

Where:

Character*12 VSDCNM	VSC dc line name (input; no default allowed).
Character*16 NAME	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).
Real RDATA	new CON value (input; unchanged).
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer IERR	error code (output).
	IERR = 0 no error
	IERR = 1 dc line not found
	IERR = 2 unknown model name
	IERR = 3 no model at this dc line
	IERR = 4 model at this dc line is aaa not xxx
	IERR = 5 no CONs used in model; or
	IERR = 5 no input ICONs used in model
	IERR = 6 CON number outside of the CON range of this model; or

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	ICON number outside of the ICON range of this model
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.17. CHANGE_VSDCLMOD_ICON

Use this API to change the value of an integer ICON of the VSC dc line model of a specified VSC dc line. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCLMOD_ICON  'DCNAME'  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_vsdclmod_icon(dcname, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCLMOD_ICON(DCNAME, MODEL, J, IDATA, IERR)
```

Where:

Character*12 DCNAME	VSC dc line name (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.18. CHANGE_VSDCLMOD_VAR

Use this API to change the value of a VAR of the VSC dc line model of a specified VSC dc line.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_VSDCLMOD_VAR  'DCNAME'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_vsdclmod_var(dcname, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCLMOD_VAR(DCNAME, MODEL, J, RDATA, IERR)
```

Where:

Character*12 DCNAME	VSC dc line name (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	VSC dc line not found
	IERR = 2	unknown model name
	IERR = 3	no model at this dc line
	IERR = 4	model at this dc line is aaa not xxx
	IERR = 5	no VARs used in model
	IERR = 6	VAR number outside of the VAR range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.19. DCLMOD_PACK

Use this API to remove entries that are marked as unused from the dc line model connection tables and the dc line array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DCLMOD_PACK
```

Python command syntax:

```
ierr = dclmod_pack()
```

Fortran command syntax:

```
CALL DCLMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.20. DCLMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those dc line models that are assigned to dc lines that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DCLMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = dclmod_unconnected(opt)
```

Fortran command syntax:

```
CALL DCLMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected dc line models
	OPT = 1	remove unconnected dc line models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.21. DCLMOD_USER

Use this API to list user-written dc line model definitions or to remove user-written dc line model definitions that are not assigned to any dc lines (unused) from the user model definition tables.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DCLMOD_USER  OPT
```

Python command syntax:

```
ierr = dclmod_user(opt)
```

Fortran command syntax:

```
CALL DCLMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written dc line model definitions.
	OPT = 1	remove unused user-written dc line model definitions.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.22. MTDCLMOD_REMOVE

Use this API to remove the multi-terminal dc line model from a specified multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MTDCLMOD_REMOVE  DC
```

Python command syntax:

```
ierr = mtdclmod_remove(dc)
```

Fortran command syntax:

```
CALL MTDCLMOD_REMOVE(NUM, IERR)

CALL MTDCLMOD_REMOVE_I(NUM, IERR)

CALL MTDCLMOD_REMOVE_C(NAME, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which the model is to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which the model is to be removed (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line for which the model is to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	no model at this dc line
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.23. MTDCLMOD_STATUS

Use this API to change the status of the multi-terminal dc line model at a specified multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MTDCLMOD_STATUS  DC  STATUS
```

Python command syntax:

```
ierr = mtdclmod_status(dc, status)
```

Fortran command syntax:

```
CALL MTDCLMOD_STATUS(NUM, STATUS, IERR)

CALL MTDCLMOD_STATUS_I(NUM, STATUS, IERR)

CALL MTDCLMOD_STATUS_C(NAME, STATUS, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line for which the model status is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line for which the model status is to be changed (input; no default allowed).
Character*12 NAME	name of the multi-terminal dc line for which the model status is to be changed (input; no default allowed).
Integer STATUS	new model status (input; 1 by default). STATUS = 0 out-of-service STATUS = 1 in-service
Integer IERR	error code (output). IERR = 0 no error IERR = 1 dc line not found IERR = 2 no model at this dc line IERR = 3 invalid STATUS value IERR = 4 prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.24. TTDCLMOD_REMOVE

Use this API to remove the two-terminal dc line model from a specified two-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_TTDCLMOD_REMOVE  DC
```

Python command syntax:

```
ierr = ttdclmod_remove(dc)
```

Fortran command syntax:

```
CALL TTDCLMOD_REMOVE(NUM, IERR)

CALL TTDCLMOD_REMOVE_I(NUM, IERR)

CALL TTDCLMOD_REMOVE_C(NAME, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which the model is to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which the model is to be removed (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line for which the model is to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	no model at this dc line
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.25. TTDCLMOD_STATUS

Use this API to change the status of the two-terminal dc line model at a specified two-terminal end line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_TTDCLMOD_STATUS  DC  STATUS
```

Python command syntax:

```
ierr = ttdclmod_status(dc, status)
```

Fortran command syntax:

```
CALL TTDCLMOD_STATUS(NUM, STATUS, IERR)

CALL TTDCLMOD_STATUS_I(NUM, STATUS, IERR)

CALL TTDCLMOD_STATUS_C(NAME, STATUS, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line for which the model status is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line for which the model status is to be changed (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line for which the model status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	no model at this dc line
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.26. VSDCLMOD_REMOVE

Use this API to remove the VSC dc line model from a specified VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_VSDCLMOD_REMOVE  'DCNAME'
```

Python command syntax:

```
ierr = vsdclmod_remove(dcname)
```

Fortran command syntax:

```
CALL VSDCLMOD_REMOVE(DCNAME, IERR)
```

Where:

Character*12 DCNAME	VSC dc line name (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	VSC dc line not found
	IERR = 2	no model at this VSC dc line
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.8.27. VSDCLMOD_STATUS

Use this API to change the status of the VSC dc line model at a specified VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_VSDCLMOD_STATUS  'DCNAME'  STATUS
```

Python command syntax:

```
ierr = vsdclmod_status(dcname, status)
```

Fortran command syntax:

```
CALL VSDCLMOD_STATUS(DCNAME, STATUS, IERR)
```

Where:

Character*12 DCNAME	VSC dc line name (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	VSC dc line not found
	IERR = 2	no model at this VSC dc line
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9. FACTS Device Models

8.9.1. ADD_FACTS_DEVICE_MODEL

Use this API to add a FACTS device model to the specified FACTS device. If a model already exists at the FACTS device, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_FACTS_DEVICE_MODEL  FD  'MODEL'  NICN  IDATA  CHDATA  NCON  RDATA
 IERR
```

Python command syntax:

```
ierr = add_facts_device_model(fd, model, nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_FACTS_DEVICE_MODEL(NUM, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)

CALL ADD_FACTS_DEVICE_MODEL_I(NUM, MODEL, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)

CALL ADD_FACTS_DEVICE_MODEL_C(NAME, MODEL, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device to which the model is to be added; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device to which the model is to be added (input; no default allowed).
Character*12 NAME	name of the FACTS device to which the model is to be added (input; no default allowed).
Character*16 MODEL	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer NCON	number of CONs required by the model (input; no default allowed).
Real RDATA (NCON)	CON data (input; 0.0).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	unknown model name
	IERR = 3	model is not a FACTS device model
	IERR = 4	# CONs specified is not the number expected
	IERR = 5	# ICONs specified is not the number expected
	IERR = 6	maximum number of array elements exceeded
	IERR = 7	FACTS device model connection tables full; use pack FACTS device model tables function
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.2. CHANGE_FCTMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the FACTS device model of a specified FACTS device. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTMOD_CHRICN  FD  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_fctmod_chricn(fd, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTMOD_CHRICN(NUM, MODEL, J, CHDATA, IERR)

CALL CHANGE_FCTMOD_CHRICN_I(NUM, MODEL, J, CHDATA, IERR)

CALL CHANGE_FCTMOD_CHRICN_C(NAME, MODEL, J, CHDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the FACTS device for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	unknown model name
	IERR = 3	no model at this FACTS device
	IERR = 4	model at this FACTS device is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.3. CHANGE_FCTMOD_CON

Use this API to change the value of a CON of the FACTS device model of a specified FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTMOD_CON  FD  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_fctmod_con(fd, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTMOD_CON(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTMOD_CON_I(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTMOD_CON_C(NAME, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device for which model data is to be changed (input; no default allowed).
Character*12 NAME	name of the FACTS device for which model data is to be changed (input; no default allowed).
Character*16 MODEL	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).
Real RDATA	new CON value (input; unchanged).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	FACTS device not found
IERR = 2	unknown model name
IERR = 3	no model at this FACTS device
IERR = 4	model at this FACTS device is aaa not xxx
IERR = 5	no CONs used in model
IERR = 6	CON number outside of the CON range of this model
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.4. CHANGE_FCTMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_FCTMOD_CON, CHANGE_FCTMOD_ICON, and CHANGE_FCTMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of the FACTS device model of a specified FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTMOD_DATA  FD  'NAME'  J  RDATA  I  IDATA  'CHDATA'
```

Python command syntax:

```
ierr = change_fctmod_data(fd,  name,  j,  rdata,  i,  idata,  chdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTMOD_DATA(FD, NAME, J, RDATA, I, IDATA, CHDATA, IERR)
```

Where:

Integer FD	FACTS device number (input; no default allowed).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).	
Real RDATA	new CON value (input; unchanged).	
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).	
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	unknown model name
	IERR = 3	no model at this FACTS device
	IERR = 4	model at this FACTS device is aaa not xxx
	IERR = 5	no CONs used in model
	IERR = 5	no input ICONs used in model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	CON number outside of the CON range of this model
IERR = 6	ICON number outside of the ICON range of this model
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.5. CHANGE_FCTMOD_ICON

Use this API to change the value of an integer ICON of the FACTS device model of a specified FACTS device. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTMOD_ICON  FD  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_fctmod_icon(fd, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_FCTMOD_ICON(NUM, MODEL, J, IDATA, IERR)

CALL CHANGE_FCTMOD_ICON_I(NUM, MODEL, J, IDATA, IERR)

CALL CHANGE_FCTMOD_ICON_C(NAME, MODEL, J, IDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device for which model data is to be changed (input; no default allowed).	
Character*12 NAME	name of the FACTS device for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	unknown model name
	IERR = 3	no model at this FACTS device
	IERR = 4	model at this FACTS device is aaa not xxx
	IERR = 5	no input ICONs used in model
	IERR = 6	ICON number outside of the ICON range of this model
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.6. CHANGE_FCTMOD_VAR

Use this API to change the value of a VAR of the FACTS device model of a specified FACTS device.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_FCTMOD_VAR  FD  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_fctmod_var(fd, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTMOD_VAR(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTMOD_VAR_I(NUM, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTMOD_VAR_C(NAME, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device for which model data is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device for which model data is to be changed (input; no default allowed).
Character*12 NAME	name of the FACTS device for which model data is to be changed (input; no default allowed).
Character*16 MODEL	model name (input; no default allowed).
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).
Real RDATA	new VAR value (input; unchanged).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	FACTS device not found
IERR = 2	unknown model name
IERR = 3	no model at this FACTS device
IERR = 4	model at this FACTS device is aaa not xxx
IERR = 5	no VARs used in model
IERR = 6	VAR number outside of the VAR range of this model
IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.7. FCTMOD_PACK

Use this API to remove entries that are marked as unused from the FACTS device model connection tables and the FACTS device model array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCTMOD_PACK
```

Python command syntax:

```
ierr = fctmod_pack()
```

Fortran command syntax:

```
CALL FCTMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.8. FCTMOD_REMOVE

Use this API to remove the FACTS device model from a specified FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCTMOD_REMOVE  FD
```

Python command syntax:

```
ierr = fctmod_remove(fd)
```

Fortran command syntax:

```
CALL FCTMOD_REMOVE(NUM, IERR)

CALL FCTMOD_REMOVE_I(NUM, IERR)

CALL FCTMOD_REMOVE_C(NAME, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device for which the model is to be removed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device for which the model is to be removed (input; no default allowed).	
Character*12 NAME	name of the FACTS device for which the model is to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	no model at this FACTS device
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.9. FCTMOD_STATUS

Use this API to change the status of the FACTS device model at a specified FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCTMOD_STATUS  FD  STATUS
```

Python command syntax:

```
ierr = fctmod_status(fd, status)
```

Fortran command syntax:

```
CALL FCTMOD_STATUS(NUM, STATUS, IERR)

CALL FCTMOD_STATUS_I(NUM, STATUS, IERR)

CALL FCTMOD_STATUS_C(NAME, STATUS, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device for which the model status is to be changed; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device for which the model status is to be changed (input; no default allowed).	
Character*12 NAME	name of the FACTS device for which the model status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	no model at this FACTS device
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.10. FCTMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those FACTS device models that are assigned to FACTS device that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCTMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = fctmod_unconnected(opt)
```

Fortran command syntax:

```
CALL FCTMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected FACTS device models
	OPT = 1	remove unconnected FACTS device models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.11. FCTMOD_USER

Use this API to list user-written FACTS device model definitions or to remove user-written FACTS device model definitions that are not assigned to any FACTS devices (unused) from the user model definition tables.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCTMOD_USER  OPT
```

Python command syntax:

```
ierr = fctmod_user(opt)
```

Fortran command syntax:

```
CALL FCTMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written FACTS device model definitions
	OPT = 1	remove unused user-written FACTS device model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.9.12. GMB_ADD_FACTS_DEV_MODEL

Use this API to add a GMB FACTS device model to the specified FACTS device. If a model already exists at the FACTS device, it is replaced.

This API routine was first introduced in release 32.0.2.

Batch command syntax:

```
BAT_GMB_ADD_FACTS_DEV_MODEL FD 'MODEL' NICN IDATA CHDATA NCON RDATA
  IERR
```

Python command syntax:

```
ierr = gmb_add_facts_dev_model(fd,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL GMB_ADD_FACTS_DEV_MODEL(FD,MODEL,NICN,IDATA,CHDATA,NCON,RDATA, IERR)
```

Where:

Character*12 FD	quoted name of the FACTS device to which the model is to be added; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device to which the model is to be added (input; no default allowed).	
Character*12 NAME	name of the FACTS device to which the model is to be added (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	error reading GMB model name
	IERR = 3	model is not a FACTS device model
	IERR = 4	# CONs specified is not the number expected
	IERR = 5	# ICONs specified is not the number expected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	maximum number of array elements exceeded
IERR = 7	FACTS device model connection tables full; use pack FACTS device model tables function
IERR = 8	unlicensed GMB model name
IERR = 9	number of user-defined models exceeds maximum allowed
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10. Switched Shunt Models

8.10.1. ADD_SWSHUNT_MODEL

This API routine is obsolete. It has been replaced by the API routine add_swshunt_model_2 .

Use this API to add a switched shunt model to the specified switched shunt device. If a model already exists at the switched shunt, it is replaced.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_ADD_SWSHUNT_MODEL  IBUS  'MODEL'  NICN  IDATA  CHDATA  NCON  RDATA
 IERR
```

Python command syntax:

```
ierr = add_swshunt_model(ibus,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL ADD_SWSHUNT_MODEL(IBUS,MODEL,NICN,IDATA,CHDATA,NCON,RDATA,IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a switched shunt model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	switched shunt model connection tables full--use pack switched shunt model tables function
IERR = 7	bus not found
IERR = 8	switched shunt not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.2. ADD_SWSHUNT_MODEL_2

Use this API to add a switched shunt model to the specified switched shunt device. If a model already exists at the switched shunt, it is replaced.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ADD_SWSHUNT_MODEL_2  IBUS  ID  'MODEL'  NICN  IDATA  CHDATA  NCON
 RDATA  IERR
```

Python command syntax:

```
ierr = add_swshunt_model_2(ibus,id,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL ADD_SWSHUNT_MODEL_2(IBUS,ID,MODEL,NICN,IDATA,CHDATA,NCON,RDATA,IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a switched shunt model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	switched shunt model connection tables full--use pack switched shunt model tables function
	IERR = 7	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	switched shunt not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.3. CHANGE_SWSMOD_CHRICN

This API routine is obsolete. It has been replaced by the API routine change_swsmod_chricn_2 .

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the switched shunt model of a specified switched shunt device. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_CHRICN  IBUS  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_swsmod_chricn(ibus, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_CHRICN(IBUS, MODEL, J, CHDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input ICONs used in model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.4. CHANGE_SWSMOD_CHRICN_2

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the switched shunt model of a specified switched shunt device. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_CHRICN_2  IBUS  ID  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_swsmod_chricn_2(ibus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_CHRICN_2(IBUS, ID, MODEL, J, CHDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input ICONs used in model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.5. CHANGE_SWSMOD_CON

This API routine is obsolete. It has been replaced by the API routine change_swsmod_con_2 .

Use this API to change the value of a CON of the switched shunt model of a specified switched shunt device.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_CON  IBUS  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_swsmod_con(ibus, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_CON(IBUS, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input CONs used in model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.6. CHANGE_SWSMOD_CON_2

Use this API to change the value of a CON of the switched shunt model of a specified switched shunt device.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_CON_2  IBUS  ID  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_swsmod_con_2(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_CON_2(IBUS, ID, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input CONs used in model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.7. CHANGE_SWSMOD_ICON

This API routine is obsolete. It has been replaced by the API routine change_swsmod_icon_2 .

Use this API to change the value of an integer ICON (i.e., an element of the ICON array) of the switched shunt model of a specified switched shunt device. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_ICON  IBUS  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_swsmod_icon(ibus, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_ICON(IBUS, MODEL, J, IDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input ICONs used in model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.8. CHANGE_SWSMOD_ICON_2

Use this API to change the value of an integer ICON (i.e., an element of the ICON array) of the switched shunt model of a specified switched shunt device. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_ICON_2  IBUS  ID  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_swsmod_icon_2(ibus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_ICON_2(IBUS, ID, MODEL, J, IDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input ICONs used in model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.9. CHANGE_SWSMOD_VAR

This API routine is obsolete. It has been replaced by the API routine change_swsmod_var_2 .

Use this API to change the value of a VAR of the switched shunt model of a specified switched shunt device.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_VAR  IBUS  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_swsmod_var(ibus, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_VAR(IBUS, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input VARs used in model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.10. CHANGE_SWSMOD_VAR_2

Use this API to change the value of a VAR of the switched shunt model of a specified switched shunt device.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_SWSMOD_VAR_2  IBUS  ID  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_swsmod_var_2(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_SWSMOD_VAR_2(IBUS, ID, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no model at this switched shunt device
	IERR = 3	model at this switched shunt is aaa not xxx
	IERR = 4	no input VARs used in model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	switched shunt not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.11. GMB_ADD_SWSHUNT_MODEL

This API routine is obsolete. It has been replaced by the API routine gmb_add_swshunt_model_2 .

Use this API to add a GMB switched shunt model to the specified switched shunt device. If a model already exists at the switched shunt, it is replaced.

This API routine was first introduced in release 32.0.2.

Batch command syntax:

```
BAT_GMB_ADD_SWSHUNT_MODEL IBUS 'MODEL' NICN IDATA CHDATA NCON RDATA
  IERR
```

Python command syntax:

```
ierr = gmb_add_swshunt_model(ibus,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL GMB_ADD_SWSHUNT_MODEL(IBUS,MODEL,NICN,IDATA,CHDATA,NCON,RDATA,IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	error reading GMB model name
	IERR = 2	model is not a switched shunt model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	switched shunt model connection tables full--use pack switched shunt model tables function

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	unlicensed GMB model name
IERR = 8	number of user-defined models exceeds maximum allowed
IERR = 9	bus not found
IERR = 10	switched shunt not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.12. GMB_ADD_SWSHUNT_MODEL_2

Use this API to add a GMB switched shunt model to the specified switched shunt device. If a model already exists at the switched shunt, it is replaced.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_GMB_ADD_SWSHUNT_MODEL_2  IBUS  ID  'MODEL'  NICN  IDATA  CHDATA  NCON
 RDATA  IERR
```

Python command syntax:

```
ierr = gmb_add_swshunt_model_2(ibus,id,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL
 GMB_ADD_SWSHUNT_MODEL_2(IBUS,ID,MODEL,NICN,IDATA,CHDATA,NCON,RDATA,IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	error reading GMB model name
	IERR = 2	model is not a switched shunt model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	switched shunt model connection tables full--use pack switched shunt model tables function
	IERR = 7	unlicensed GMB model name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	number of user-defined models exceeds maximum allowed
IERR = 9	bus not found
IERR = 10	switched shunt not found
IERR = 11	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.13. SWSMOD_PACK

Use this API to remove entries that are marked as unused from the switched shunt model connection tables and the switched shunt model array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SWSMOD_PACK
```

Python command syntax:

```
ierr = swsmod_pack()
```

Fortran command syntax:

```
CALL SWSMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.14. SWSMOD_REMOVE

This API routine is obsolete. It has been replaced by the API routine swsmod_remove_2 .

Use this API to remove the switched shunt model from a specified switched shunt.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SWSMOD_REMOVE  IBUS
```

Python command syntax:

```
ierr = swsmod_remove(ibus)
```

Fortran command syntax:

```
CALL SWSMOD_REMOVE(IBUS, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no model at this switched shunt device
	IERR = 2	bus not found
	IERR = 3	switched shunt not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.15. SWSMOD_REMOVE_2

Use this API to remove the switched shunt model from a specified switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SWSMOD_REMOVE_2  IBUS  ID
```

Python command syntax:

```
ierr = swsmod_remove_2(ibus,id)
```

Fortran command syntax:

```
CALL SWSMOD_REMOVE_2(IBUS, ID, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no model at this switched shunt device
	IERR = 2	bus not found
	IERR = 3	switched shunt not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.16. SWSMOD_STATUS

This API routine is obsolete. It has been replaced by the API routine swsmod_status_2 .

Use this API to change the status of the switched shunt model at a specified switched shunt device.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SWSMOD_STATUS  IBUS  STATUS
```

Python command syntax:

```
ierr = swsmod_status(ibus, status)
```

Fortran command syntax:

```
CALL SWSMOD_STATUS(IBUS, STATUS, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service.
	STATUS = 1	in-service.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no model at this switched shunt device
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	switched shunt not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.17. SWSMOD_STATUS_2

Use this API to change the status of the switched shunt model at a specified switched shunt device.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SWSMOD_STATUS_2  IBUS  ID  STATUS
```

Python command syntax:

```
ierr = swsmod_status_2(ibus, id, status)
```

Fortran command syntax:

```
CALL SWSMOD_STATUS_2(IBUS, ID, STATUS, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service.
	STATUS = 1	in-service.
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no model at this switched shunt device
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	switched shunt not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.18. SWSMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those switched shunt models that are assigned to switched shunt that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SWSMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = swsmod_unconnected(opt)
```

Fortran command syntax:

```
CALL SWSMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected switched shunt models
	OPT = 1	remove unconnected switched shunt models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.10.19. SWSMOD_USER

Use this API to list user-written switched shunt model definitions or to remove user-written switched shunt model definitions that are not assigned to any switched shunt devices (unused) from the user model definition tables.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SWSMOD_USER  OPT
```

Python command syntax:

```
ierr = swsmod_user(opt)
```

Fortran command syntax:

```
CALL SWSMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written switched shunt model definitions
	OPT = 1	remove unused user-written switched shunt model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11. Induction Machine Models

8.11.1. ADD_INDMAC_MODEL

Use this API to add an induction machine model to a specified machine. If a model of the designated type already exists at the machine, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_INDMAC_MODEL  IBUS  'ID'  'NAME'  NICN  IDATA  CHDATA  NCON  RDATA
```

Python command syntax:

```
ierr = add_indmac_model(ibus, id, name, nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_INDMAC_MODEL(IBUS, ID, NAME, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Integer IBUS	induction machine bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	Induction machine model connection tables full--use pack induction machine model tables function
	IERR = 2	Induction machine model tables are corrupt
	IERR = 3	unknown model NAME
	IERR = 4	model is not an induction machine model
	IERR = 5	# CONs specified is not the number expected
	IERR = 6	# ICONs specified is not the number expected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	maximum number of array elements exceeded
IERR = 8	bus not found
IERR = 9	machine not found
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.2. CHANGE_IMMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of an induction machine model at a specified machine. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_IMMOD_CHRICN  IBUS  'ID'  'NAME'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_immod_chricn(ibus, id, name, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_IMMOD_CHRICN(IBUS, ID, NAME, J, CHDATA, IERR)
```

Where:

Integer IBUS	induction machine bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	No model at this induction machine
	IERR = 3	model at this machine is aaa not NAME
	IERR = 4	no input ICONs used in model NAME
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.3. CHANGE_IMMOD_CON

Use this API to change the value of a CON of a designated induction machine model at a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_IMMOD_CON  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_immod_con(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_IMMOD_CON(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	induction machine bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	No model at this induction machine
	IERR = 3	model at this machine is aaa not NAME
	IERR = 4	no input ICONs used in model NAME
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.4. CHANGE_IMMOD_ICON

Use this API to change the value of an integer ICON of a designated induction machine model at a specified machine. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_IMMOD_ICON  IBUS  'ID'  'NAME'  J  IDATA
```

Python command syntax:

```
ierr = change_immod_icon(ibus, id, name, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_IMMOD_ICON(IBUS, ID, NAME, J, IDATA, IERR)
```

Where:

Integer IBUS	induction machine bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	No model at this induction machine
	IERR = 3	model at this machine is aaa not NAME
	IERR = 4	no input CONs used in model NAME
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.5. CHANGE_IMMOD_VAR

Use this API to change the value of a VAR of a designated induction machine model at a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_IMMOD_VAR  IBUS  'ID'  'NAME'  J  RDATA
```

Python command syntax:

```
ierr = change_immod_var(ibus, id, name, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_IMMOD_VAR(IBUS, ID, NAME, J, RDATA, IERR)
```

Where:

Integer IBUS	induction machine bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; blank by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model NAME
	IERR = 2	No model at this induction machine
	IERR = 3	model at this machine is aaa not NAME
	IERR = 4	no input VARs used in model NAME
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.6. IMMOD_PACK

Use this API to remove entries that are marked as unused from the induction machine model contables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IMMOD_PACK
```

Python command syntax:

```
ierr = immod_pack()
```

Fortran command syntax:

```
CALL IMMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.7. IMMOD_REMOVE

Use this API to remove an induction machine model of a designated type from a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IMMOD_REMOVE  IBUS  'ID'
```

Python command syntax:

```
ierr = immod_remove(ibus, id)
```

Fortran command syntax:

```
CALL IMMOD_REMOVE(IBUS, ID, IERR)
```

Where:

Integer IBUS	induction machine bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no induction machine model at this machine
	IERR = 2	bus not found
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.8. IMMOD_STATUS

Use this API to change the status of an induction machine model at a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IMMOD_STATUS  IBUS  'ID'  STATUS
```

Python command syntax:

```
ierr = immod_status(ibus, id, status)
```

Fortran command syntax:

```
CALL IMMOD_STATUS(IBUS, ID, STATUS, IERR)
```

Where:

Integer IBUS	induction machine bus number (input; no default allowed).	
Character*2 ID	induction machine identifier (input; blank by default).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no induction machine model at this machine
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	machine not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.9. IMMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those induction machine models that are assigned to machines that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IMMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = immod_unconnected(opt)
```

Fortran command syntax:

```
CALL IMMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected induction machine models
	OPT = 1	remove unconnected induction machine models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.11.10. IMMOD_USER

Use this API to list user-written induction machine model definitions or to remove user-written induction machine model definitions that are not assigned to any machines (unused) from the user model definition tables.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_IMMOD_USER  OPT
```

Python command syntax:

```
ierr = immod_user(opt)
```

Fortran command syntax:

```
CALL IMMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written induction machine model definitions
	OPT = 1	remove unused user-written induction machine model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12. CCT Models

8.12.1. ADD_CCT2DCO_MODEL

Use this API to add an "other" type model attached to a 2-terminal dc line. If another model exists, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCT2DCO_MODEL  DC  'MODEL'  NICN  IDATA  CHDATA  NCON  RDATA
```

Python command syntax:

```
ierr = add_cct2dco_model(dc, model, nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_CCT2DCO_MODEL(DC, MODEL, NICN, IDATA, CHDATA, NCON, RDATA, IERR)
```

Where:

Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a 2-terminal dc line "other" type model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	dc line not found
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.2. ADD_CCT2WTD_MODEL

Use this API to add a 2-winding transformer device model to the specified 2-winding transformer. If a model already exists at the 2-winding transformer, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCT2WTD_MODEL  IBUS  JBUS  'ID'  'MODEL'  NICN  IDATA  CHDATA
 NCON  RDATA
```

Python command syntax:

```
ierr = add_cct2wtd_model(ibus,jbus,id,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL ADD_CCT2WTD_MODEL(IBUS, JBUS, ID, MODEL, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a 2-winding transformer device model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	branch is not a transformer
IERR = 8	bus not found
IERR = 9	2-winding transformer not found
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.3. ADD_CCT3WTD_MODEL

Use this API to add a 3-winding transformer device model to the specified 3-winding transformer. If a model already exists at the 3-winding transformer, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCT3WTD_MODEL  IBUS JBUS KBUS ID MODEL NICN IDATA CHDATA NCON
 RDATA
```

Python command syntax:

```
ierr = add_cct3wtd_model(ibus, jbus, kbus, id, model, nicn, idata, chdata,
 ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_CCT3WTD_MODEL(IBUS, JBUS, KBUS, ID, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a 3-winding transformer device model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	CCT model connection tables full-use pack CCT model tables function
IERR = 7	branch is not a transformer
IERR = 8	bus not found
IERR = 9	3-winding transformer not found
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.4. ADD_CCTBRND_MODEL

Use this API to add a branch device model to the specified branch. If a model already exists at the branch, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCTBRND_MODEL  IBUS  JBUS  'ID'  'MODEL'  NICN  IDATA  CHDATA
 NCON  RDATA
```

Python command syntax:

```
ierr = add_cctbrnd_model(ibus,jbus,id,model,nicn,idata,chdata,ncon,rdata)
```

Fortran command syntax:

```
CALL ADD_CCTBRND_MODEL(IBUS, JBUS, ID, MODEL, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a branch device model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	branch is a transformer
IERR = 8	bus not found
IERR = 9	branch not found
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.5. ADD_CCTBRNO_MODEL

Use this API to add a branch "other" model to the specified branch. If a model already exists at the branch, it is replaced.

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_ADD_CCTBRNO_MODEL  IBUS JBUS ID MODEL NICN IDATA(1)..IDATA(NICN)
 CHDATA(1)..CHDATA(NICN) NCON RDATA(1)..RDATA(NCON)
```

Python command syntax:

```
ierr = add_cctbrno_model(ibus, jbus, id, model, nicn, idata, chdata, ncon,
 rdata)
```

Fortran command syntax:

```
CALL ADD_CCTBRNO_MODEL(IBUS, JBUS, ID, MODEL, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a branch device model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	branch is a transformer
IERR = 8	bus not found
IERR = 9	branch not found
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.6. ADD_CCTBUSO_MODEL

Use this API to add an "other" type model attached to a bus. If another model exists, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCTBUSO_MODEL  IBUS  'MODEL'  NICN  IDATA  CHDATA  NCON  RDATA
```

Python command syntax:

```
ierr = add_cctbuso_model(ibus, model, nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_CCTBUSO_MODEL(IBUS, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a bus "other" type model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function
	IERR = 7	bus not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.7. ADD_CCTMCNO_MODEL

Use this API to add a machine other model to the specified machine. If another model exists, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCTMCNO_MODEL  IBUS  'ID'  'MODEL'  NICN  IDATA  CHDATA  NCON
 RDATA
```

Python command syntax:

```
ierr = add_cctmcno_model(ibus, id, model, nicn, idata, chdata, ncon,
 rdata)
```

Fortran command syntax:

```
CALL ADD_CCTMCNO_MODEL(IBUS, ID, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a machine protection model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function
	IERR = 7	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	machine not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.8. ADD_CCTMCNP_MODEL

Use this API to add a machine protection model to the specified machine. If another model exists, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCTMCNP_MODEL  IBUS  'ID'  'MODEL'  NICN  IDATA  CHDATA  NCON
 RDATA
```

Python command syntax:

```
ierr = add_cctmcnp_model(ibus, id, model, nicn, idata, chdata, ncon,
 rdata)
```

Fortran command syntax:

```
CALL ADD_CCTMCNP_MODEL(IBUS, ID, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a machine protection model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function
	IERR = 7	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	machine not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.9. ADD_CCTMSCO_MODEL

Use this API to add a miscellaneous "other" type model with the specified model instance. If another model exists with the same model instance, it is replaced.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_ADD_CCTMSCO_MODEL  'MODEL'  MINS NICN  IDATA  CHDATA  NCON  RDATA
```

Python command syntax:

```
ierr = add_cctmsco_model(model, mins,  nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_CCTMSCO_MODEL(MODEL, MINS, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Character*16 MODEL	model name (input; no default allowed).	
Integer MINS	model instance (>0) (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a bus "other" type model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function
	IERR = 7	invalid model instance
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.10. ADD_CCTSWSO_MODEL

This API routine is obsolete. It has been replaced by the API routine add_cctswso_model_2 .

Use this API to add a switched shunt "other" model to the specified bus. If a model already exists at the bus, it is replaced.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_ADD_CCTSWSO_MODEL  IBUS MODEL NICN IDATA(1)..IDATA(NICN)
 CHDATA(1)..CHDATA(NICN) NCON RDATA(1)..RDATA(NCON)
```

Python command syntax:

```
ierr = add_cctswso_model(ibus, model, nicn, idata, chdata, ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_CCTSWSO_MODEL(IBUS, MODEL, NICN, IDATA, CHDATA, NCON, RDATA,
 IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a switched shunt other model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	bus not found
IERR = 8	switched shunt not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.11. ADD_CCTSWSO_MODEL_2

Use this API to add a switched shunt "other" model to the specified bus. If a model already exists at the bus, it is replaced.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ADD_CCTSWSO_MODEL_2  IBUS ID MODEL NICN IDATA(1)..IDATA(NICN)
 CHDATA(1)..CHDATA(NICN) NCON RDATA(1)..RDATA(NCON)
```

Python command syntax:

```
ierr = add_cctswso_model_2(ibus, id, model, nicn, idata, chdata, ncon,
 rdata)
```

Fortran command syntax:

```
CALL ADD_CCTSWSO_MODEL_2(IBUS, ID, MODEL, NICN, IDATA, CHDATA, NCON,
 RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	model is not a switched shunt other model
	IERR = 3	# CONs specified is not the number expected
	IERR = 4	# ICONs specified is not the number expected
	IERR = 5	maximum number of array elements exceeded
	IERR = 6	CCT model connection tables full-use pack CCT model tables function
	IERR = 7	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	switched shunt not found
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.12. CCT2DCOMOD_REMOVE

Use this API to remove an "other" type model attached to a 2-terminal dc line.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCT2DCOMOD_REMOVE  DC  MODEL
```

Python command syntax:

```
ierr = cct2dcomod_remove(dc, model)
```

Fortran command syntax:

```
CALL CCT2DCOMOD_REMOVE(DC, MODEL, IERR)
```

Where:

Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	name of the model to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no "other" model at this dc line
	IERR = 2	dc line not found
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.13. CCT2DCOMOD_STATUS

Use this API to change status of an "other" type model attached to a 2-terminal dc line.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCT2DCOMOD_STATUS  DC  MODEL  STATUS
```

Python command syntax:

```
ierr = cct2dcomod_status(dc, model, status)
```

Fortran command syntax:

```
CALL CCT2DCOMOD_STATUS(DC, MODEL, STATUS, IERR)
```

Where:

Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	name of the model whose status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no "other" model at this dc line
	IERR = 2	invalid STATUS value
	IERR = 3	dc line not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.14. CCT2DCO_LIST

Use this API to list "other" type models attached to 2-terminal dc lines.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCT2DCO_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cct2dco_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCT2DCO_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.15. CCT2WTDMOD_REMOVE

Use this API to remove a 2-winding transformer device model attached to the specified 2-winding transformer.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCT2WTDMOD_REMOVE   IBUS  JBUS  'ID'
```

Python command syntax:

```
ierr = cct2wtdmod_remove(ibus, jbus, id)
```

Fortran command syntax:

```
CALL CCT2WTDMOD_REMOVE(IBUS, JBUS, ID, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no 2-winding transformer device model on this branch
	IERR = 2	bus not found
	IERR = 3	2-winding transformer not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.16. CCT2WTDMOD_STATUS

Use this API to change the status of a 2-winding transformer device model attached to the specified 2-winding transformer.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCT2WTDMOD_STATUS   IBUS  JBUS  'ID'  STATUS
```

Python command syntax:

```
ierr = cct2wtdmod_status(ibus, jbus, id, status)
```

Fortran command syntax:

```
CALL CCT2WTDMOD_STATUS(IBUS, JBUS, ID, STATUS, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no 2-winding transformer device model on this branch
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	2-winding transformer not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.17. CCT2WTD_LIST

Use this API to list "other" type models attached to 2-winding transformers.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCT2WTD_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cct2wtd_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCT2WTD_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.18. CCT3WTDMOD_REMOVE

Use this API to remove a 3-winding transformer device model attached to the specified 3-winding transformer.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCT3WTDMOD_REMOVE  IBUS  JBUS  KBUS 'ID'
```

Python command syntax:

```
ierr = cct3wtdmod_remove(ibus, jbus, kbus, id)
```

Fortran command syntax:

```
CALL CCT3WTDMOD_REMOVE(IBUS, JBUS, KBUS, ID, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no 3-winding transformer device model on this branch
	IERR = 2	bus not found
	IERR = 3	3-winding transformer not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.19. CCT3WTDMOD_STATUS

Use this API to change the status of a 3-winding transformer device model attached to the specified 3-winding transformer.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCT3WTDMOD_STATUS  IBUS  JBUS  KBUS 'ID'  STATUS
```

Python command syntax:

```
ierr = cct3wtdmod_status(ibus, jbus, kbus, id, status)
```

Fortran command syntax:

```
CALL CCT3WTDMOD_STATUS(IBUS, JBUS, KBUS, ID, STATUS, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no 3-winding transformer device model on this branch
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	3-winding transformer not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.20. CCT3WTD_LIST

Use this API to list "other" type models attached to 3-winding transformers.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCT3WTD_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cct3wtd_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCT3WTD_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.21. CCTBRNDMOD_REMOVE

Use this API to remove a branch device model attached to the specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTBRNMOD_REMOVE  IBUS JBUS ID
```

Python command syntax:

```
ierr = cctbrndmod_remove(ibus, jbus, id)
```

Fortran command syntax:

```
CALL CCTBRNDMOD_REMOVE(IBUS, JBUS, ID, IERR)
```

Where:

Integer IBUS	from bus number (input; Not Allowed).	
Integer JBUS	to bus number (input; Not Allowed).	
Character*2 ID	circuit identifier (input; Not Allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no branch device model on the branch
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.22. CCTBRNDMOD_STATUS

Use this API to change the status of a branch device model attached to the specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTBRNDMOD_STATUS  IBUS  JBUS  'ID'  STATUS
```

Python command syntax:

```
ierr = cctbrndmod_status(ibus, jbus, id, status)
```

Fortran command syntax:

```
CALL CCTBRNDMOD_STATUS(IBUS, JBUS, ID, STATUS, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input; Not Allowed).	
Character*2 ID	circuit identifier (input).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no branch device model on this branch
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	branch not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.23. CCTBRND_LIST

Use this API to list device type models attached to branches.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCTBRND_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cctbrnd_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCTBRND_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.24. CCTBRNOMOD_REMOVE

Use this API to remove a branch "other" model attached to the specified branch.

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_CCTBRNOMOD_REMOVE  IBUS JBUS ID MODEL
```

Python command syntax:

```
ierr = cctbrnomod_remove(ibus, jbus, id, model)
```

Fortran command syntax:

```
CALL CCTBRNOMOD_REMOVE(IBUS, JBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	from bus number (input; Not Allowed).	
Integer JBUS	to bus number (input; Not Allowed).	
Character*2 ID	circuit identifier (input; Not Allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no branch other model on the branch
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.25. CCTBRNOMOD_STATUS

Use this API to change the status of a branch "other" model attached to the specified branch.

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_CCTBRNOMOD_STATUS  IBUS JBUS ID MODEL STATUS
```

Python command syntax:

```
ierr = cctbrnomod_status(ibus, jbus, id, model, status)
```

Fortran command syntax:

```
CALL CCTBRNOMOD_STATUS(IBUS, JBUS, ID, MODEL, STATUS, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input; Not Allowed).	
Character*2 ID	circuit identifier (input).	
Character*16 MODEL	model name (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no branch other model on this branch
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	branch not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.26. CCTBRNO_LIST

Use this API to list "other" type models attached to branches.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCTBRNO_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cctbrno_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCTBRNO_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.27. CCTBUSOMOD_REMOVE

Use this API to remove an "other" type model attached to a bus.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTBUSOMOD_REMOVE  IBUS  MODEL
```

Python command syntax:

```
ierr = cctbusomod_remove(ibus, model)
```

Fortran command syntax:

```
CALL CCTBUSOMOD_REMOVE(IBUS, MODEL, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*16 MODEL	name of the model to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no bus other model
	IERR = 2	bus not found
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.28. CCTBUSOMOD_STATUS

Use this API to change status of an "other" type model attached to a bus.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTBUSOMOD_STATUS  IBUS  MODEL  STATUS
```

Python command syntax:

```
ierr = cctbusomod_status(ibus, model, status)
```

Fortran command syntax:

```
CALL CCTBUSOMOD_STATUS(IBUS, MODEL, STATUS, IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*16 MODEL	name of the model whose status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no bus other model at this bus
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.29. CCTBUSO_LIST

Use this API to list "other" type models attached to buses.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCTBUSO_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cctbuso_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCTBUSO_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.30. CCTMCNOMOD_REMOVE

Use this API to remove a machine "other" model attached to the specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMCNOMOD_REMOVE  IBUS  'ID'  'MODEL'
```

Python command syntax:

```
ierr = cctmcnomod_remove(ibus, id, model)
```

Fortran command syntax:

```
CALL CCTMCNOMOD_REMOVE(IBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	name of the model to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no machine protection model at this machine
	IERR = 2	bus not found
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.31. CCTMCNOMOD_STATUS

Use this API to change status of a machine "other" model attached to the specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMCNOMOD_STATUS  IBUS  'ID'  MODEL  STATUS
```

Python command syntax:

```
ierr = cctmcnomod_status(ibus, id, model, status)
```

Fortran command syntax:

```
CALL CCTMCNOMOD_STATUS(IBUS, ID, MODEL, STATUS, IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	name of the model whose status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no machine other model at this machine
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	machine not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.32. CCTMCNO_LIST

Use this API to list "other" type models attached to machines.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCTMCNO_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cctmcno_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCTMCNO_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.33. CCTMCNPMOD_REMOVE

Use this API to remove a machine protection model attached to the specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMCNPMOD_REMOVE  IBUS  'ID'  'MODEL'
```

Python command syntax:

```
ierr = cctmcnpmod_remove(ibus, id, model)
```

Fortran command syntax:

```
CALL CCTMCNPMOD_REMOVE(IBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	name of the model to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no machine protection model at this machine
	IERR = 2	bus not found
	IERR = 3	machine not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.34. CCTMCNPMOD_STATUS

Use this API to change status of a machine protection model attached to the specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMCNPMOD_STATUS  IBUS  'ID'  MODEL  STATUS
```

Python command syntax:

```
ierr = cctmcnpmod_status(ibus, id, model, status)
```

Fortran command syntax:

```
CALL CCTMCNPMOD_STATUS(IBUS, ID, MODEL, STATUS, IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	name of the model whose status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no machine protection model at this machine
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	machine not found
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.35. CCTMCNP_LIST

Use this API to list protection models attached to machines.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCTMCNP_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cctmcnp_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCTMCNP_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.36. CCTMOD_PACK

Use this API to pack CCT model definition tables.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMOD_PACK
```

Python command syntax:

```
ierr = cctmod_pack()
```

Fortran command syntax:

```
CALL CCTMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.37. CCTMOD_USER

Use this API to list or remove user-written CCT model definitions.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMOD_USER    OPT
```

Python command syntax:

```
ierr = cctmod_user(opt)
```

Fortran command syntax:

```
CALL CCTMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written model definitions
	OPT = 1	remove unused user-written model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.38. CCTMSCOMOD_REMOVE

Use this API to remove a miscellaneous "other" type model with the specified model instance.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMSCOMOD_REMOVE  MINS
```

Python command syntax:

```
ierr = cctmscomod_remove(mins)
```

Fortran command syntax:

```
CALL CCTMSCOMOD_REMOVE(MINS, IERR)
```

Where:

Integer MINS	model instance (>0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no bus other model with this model instance at this bus
	IERR = 2	invalid model instance
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.39. CCTMSCOMOD_STATUS

Use this API to change status of a miscellaneous "other" type model with the specified model instance.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CCTMSCOMOD_STATUS  MINS  STATUS
```

Python command syntax:

```
ierr = cctmscomod_status(mins, status)
```

Fortran command syntax:

```
CALL CCTMSCOMOD_STATUS(MINS, STATUS, IERR)
```

Where:

Integer MINS	model instance (>0) (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no miscellaneous other model with this model instance
	IERR = 2	invalid STATUS value
	IERR = 3	invalid model instance
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.40. CCTMSCO_LIST

Use this API to list miscellaneous "other" type models.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCTMSCO_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cctmsco_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCTMSCO_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.41. CCTSWSOMOD_REMOVE

This API routine is obsolete. It has been replaced by the API routine cctswsomod_remove_2 .

Use this API to remove an "other" type model attached to a switched shunt.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_CCTSWSOMOD_REMOVE  IBUS MODEL
```

Python command syntax:

```
ierr = cctswsomod_remove(ibus, model)
```

Fortran command syntax:

```
CALL CCTSWSOMOD_REMOVE(IBUS, MODEL, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	name of the model to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no bus other model
	IERR = 2	bus not found
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.42. CCTSWSOMOD_REMOVE_2

Use this API to remove an "other" type model attached to a switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CCTSWSOMOD_REMOVE_2  IBUS ID MODEL
```

Python command syntax:

```
ierr = cctswsomod_remove_2(ibus, id, model)
```

Fortran command syntax:

```
CALL CCTSWSOMOD_REMOVE_2(IBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	name of the model to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no bus other model
	IERR = 2	bus not found
	IERR = 3	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.43. CCTSWSOMOD_STATUS

This API routine is obsolete. It has been replaced by the API routine cctswsomod_status_2 .

Use this API to change status of an "other" type model attached to a switched shunt.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_CCTSWSOMOD_STATUS  IBUS MODEL STATUS
```

Python command syntax:

```
ierr = cctswsomod_status(ibus, model, status)
```

Fortran command syntax:

```
CALL CCTSWSOMOD_STATUS(IBUS, MODEL, STATUS, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	name of the model whose status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no bus other model at this bus
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.44. CCTSWSOMOD_STATUS_2

Use this API to change status of an "other" type model attached to a switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CCTSWSOMOD_STATUS_2  IBUS ID MODEL STATUS
```

Python command syntax:

```
ierr = cctswsomod_status_2(ibus, id, model, status)
```

Fortran command syntax:

```
CALL CCTSWSOMOD_STATUS_2(IBUS, ID, MODEL, STATUS, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	name of the model whose status is to be changed (input; no default allowed).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	no bus other model at this bus
	IERR = 2	invalid STATUS value
	IERR = 3	bus not found
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.45. CCTSWSO_LIST

Use this API to list "other" type models attached to switched shunts.

This API routine was first introduced in release 33.12.0 and 34.1.0.

Batch command syntax:

```
BAT_CCTSWSO_LIST SID ALL STATUS
```

Python command syntax:

```
ierr = cctswso_list(sid, all, status)
```

Fortran command syntax:

```
CALL CCTSWSO_LIST(SID, ALL, STATUS, IERR)
```

Where:

Integer SID	valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (0 by default). (input).	
Integer ALL	all buses or specified subsystem flag (1 by default) (input).	
	ALL = 0	process only buses in subsystem SID
	ALL = 1	process all buses
Integer STATUS	active/bypassed flag (input; 0 by default).	
	STATUS = 0	active and bypassed models
	STATUS = 1	active models only
	STATUS = 2	bypassed models only
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid SID value or subsystem SID is not defined
	IERR = 2	invalid ALL value
	IERR = 3	invalid STATUS value
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.46. CHANGE_CCT2DCOMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the "other" type model attached to a specified 2-terminal dc line. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2DCOMOD_CHRICN  DC  'MODEL' J  CHDATA
```

Python command syntax:

```
ierr = change_cct2dcomod_chricn(dc, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2DCOMOD_CHRICN(DC, MODEL, J, CHDATA,  IERR)
```

Where:

Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	dc line not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.47. CHANGE_CCT2DCOMOD_CON

Use this API to change the value of a real constant (CON) of the "other" type model attached to a specified 2-terminal dc line.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2DCOMOD_CON  DC  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cct2dcomod_con(dc, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2DCOMOD_CON(DC, MODEL, J, RDATA,  IERR)
```

Where:

Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	dc line not found
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.48. CHANGE_CCT2DCOMOD_ICON

Use this API to change the value of an integer ICON of the "other" type model attached to a specified 2-terminal dc line.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2DCOMOD_ICON  DC  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_cct2dcomod_icon(dc, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2DCOMOD_ICON(DC, MODEL, J, IDATA,  IERR)
```

Where:

Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	dc line not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.49. CHANGE_CCT2DCOMOD_VAR

Use this API to change the value of VAR of the "other" type model attached to a specified 2-terminal dc line.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2DCOMOD_VAR  DC  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cct2dcomod_var(dc, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2DCOMOD_VAR(DC, MODEL, J, RDATA,  IERR)
```

Where:

Character*12 DC	quoted name of the two-terminal dc line for which model data is to be changed (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	dc line not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.50. CHANGE_CCT2WTDMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the 2-winding transformer device model of a specified branch. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2WTDMOD_CHRICN  IBUS  JBUS  'ID'  'MODEL'  J  CHDATA
```

Python command syntax:

```
ierr = change_cct2wtdmod_chricn(ibus, jbus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2WTDMOD_CHRICN(IBUS, JBUS, ID, MODEL, J, CHDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.51. CHANGE_CCT2WTDMOD_CON

Use this API to change the value of a real constant (CON) of the 2-winding transformer device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2WTDMOD_CON  IBUS  JBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cct2wtdmod_con(ibus, jbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2WTDMOD_CON(IBUS, JBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.52. CHANGE_CCT2WTDMOD_ICON

Use this API to change the value of an integer ICON of the 2-winding transformer device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2WTDMOD_ICON  IBUS  JBUS  'ID'  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_cct2wtdmod_icon(ibus, jbus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2WTDMOD_ICON(IBUS, JBUS, ID, MODEL, J, IDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.53. CHANGE_CCT2WTDMOD_VAR

Use this API to change the value of VAR of the 2-winding transformer device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT2WTDMOD_VAR  IBUS  JBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cct2wtdmod_var(ibus, jbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT2WTDMOD_VAR(IBUS, JBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.54. CHANGE_CCT3WTDMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the 3-winding transformer device model of a specified branch. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT3WTDMOD_CHRICN  IBUS  JBUS  KBUS 'ID'  'MODEL'  J  CHDATA
```

Python command syntax:

```
ierr = change_cct3wtdmod_chricn(ibus, jbus, kbus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT3WTDMOD_CHRICN(IBUS, JBUS, KBUS, ID, MODEL, J, CHDATA,
 IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.55. CHANGE_CCT3WTDMOD_CON

Use this API to change the value of a real constant (CON) of the 3-winding transformer device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT3WTDMOD_CON  IBUS  JBUS  KBUS 'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cct3wtdmod_con(ibus, jbus, kbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT3WTDMOD_CON(IBUS, JBUS, KBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.56. CHANGE_CCT3WTDMOD_ICON

Use this API to change the value of an integer ICON of the 3-winding transformer device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT3WTDMOD_ICON  IBUS  JBUS  KBUS 'ID'  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_cct3wtdmod_icon(ibus, jbus, kbus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCT3WTDMOD_ICON(IBUS, JBUS, KBUS, ID, MODEL, J, IDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.57. CHANGE_CCT3WTDMOD_VAR

Use this API to change the value of VAR of the 3-winding transformer device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCT3WTDMOD_VAR  IBUS  JBUS  KBUS 'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cct3wtdmod_var(ibus, jbus, kbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCT3WTDMOD_VAR(IBUS, JBUS, KBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Integer KBUS	third bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.58. CHANGE_CCTBRNDMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the branch device model of a specified branch. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNDMOD_CHRICN  IBUS  JBUS  'ID'  'MODEL'  J  CHDATA
```

Python command syntax:

```
ierr = change_cctbrndmod_chricn(ibus, jbus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNDMOD_CHRICN(IBUS, JBUS, ID, MODEL, J, CHDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.59. CHANGE_CCTBRNDMOD_CON

Use this API to change the value of a real constant (CON) of the branch device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNDMOD_CON  IBUS  JBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctbrndmod_con(ibus, jbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNDMOD_CON(IBUS, JBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.60. CHANGE_CCTBRNDMOD_ICON

Use this API to change the value of an integer ICON of the branch device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNDMOD_ICON  IBUS  JBUS  'ID'  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_cctbrndmod_icon(ibus, jbus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNDMOD_ICON(IBUS, JBUS, ID, MODEL, J, IDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.61. CHANGE_CCTBRNDMOD_VAR

Use this API to change the value of VAR of the branch device model of a specified branch.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNDMOD_VAR  IBUS  JBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctbrndmod_var(ibus, jbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNDMOD_VAR(IBUS, JBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.62. CHANGE_CCTBRNOMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the branch "other" model of a specified branch. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNOMOD_CHRICN  IBUS JBUS ID MODEL J CHDATA
```

Python command syntax:

```
ierr = change_cctbrnomod_chricn(ibus, jbus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNOMOD_CHRICN(IBUS, JBUS, ID, MODEL, J, CHDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2004

8.12.63. CHANGE_CCTBRNOMOD_CON

Use this API to change the value of a real constant (CON) of the branch "other" model of a specified branch.

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNOMOD_CON  IBUS JBUS ID MODEL J RDATA
```

Python command syntax:

```
ierr = change_cctbrnomod_con(ibus, jbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNOMOD_CON(IBUS, JBUS, ID, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2005

8.12.64. CHANGE_CCTBRNOMOD_ICON

Use this API to change the value of an integer ICON of the branch "other" model of a specified branch.

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNOMOD_ICON  IBUS JBUS ID MODEL J IDATA
```

Python command syntax:

```
ierr = change_cctbrnomod_icon(ibus, jbus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNOMOD_ICON(IBUS, JBUS, ID, MODEL, J, IDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2006

8.12.65. CHANGE_CCTBRNOMOD_VAR

Use this API to change the value of VAR of the branch "other" model of a specified branch.

This API routine was first introduced in release 33.6.0.

Batch command syntax:

```
BAT_CHANGE_CCTBRNOMOD_VAR  IBUS JBUS ID MODEL J RDATA
```

Python command syntax:

```
ierr = change_cctbrnomod_var(ibus, jbus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBRNOMOD_VAR(IBUS, JBUS, ID, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	from bus number (input; no default allowed).	
Integer JBUS	to bus number (input; no default allowed).	
Character*2 ID	circuit identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	branch not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2007

8.12.66. CHANGE_CCTBUSOMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the "other" type model attached to a specified bus. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBUSOMOD_CHRICN  IBUS  'MODEL' J  CHDATA
```

Python command syntax:

```
ierr = change_cctbusomod_chricn(ibus, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBUSOMOD_CHRICN(IBUS, MODEL, J, CHDATA,  IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2008

8.12.67. CHANGE_CCTBUSOMOD_CON

Use this API to change the value of a real constant (CON) of the "other" type model attached to a specified bus.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBUSOMOD_CON  IBUS  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctbusomod_con(ibus, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBUSOMOD_CON(IBUS, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.68. CHANGE_CCTBUSOMOD_ICON

Use this API to change the value of an integer ICON of the "other" type model attached to a specified bus.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBUSOMOD_ICON  IBUS  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_cctbusomod_icon(ibus, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBUSOMOD_ICON(IBUS, MODEL, J, IDATA,  IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2010

8.12.69. CHANGE_CCTBUSOMOD_VAR

Use this API to change the value of VAR of the "other" type model attached to a specified bus.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTBUSOMOD_VAR  IBUS  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctbusomod_var(ibus, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTBUSOMOD_VAR(IBUS, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2011

8.12.70. CHANGE_CCTMCNOMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the machine "other" model for a specified machine. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNOMOD_CHRICN  IBUS  'ID'  'MODEL' J  CHDATA
```

Python command syntax:

```
ierr = change_cctmcnomod_chricn(ibus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNOMOD_CHRICN(IBUS, ID, MODEL, J, CHDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2012

8.12.71. CHANGE_CCTMCNOMOD_CON

Use this API to change the value of a real constant (CON) of the machine "other" model for a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNOMOD_CON  IBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctmcnomod_con(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNOMOD_CON(IBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.72. CHANGE_CCTMCNOMOD_ICON

Use this API to change the value of an integer ICON of the machine "other" model for a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNOMOD_ICON  IBUS  'ID'  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_cctmcnomod_icon(ibus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNOMOD_ICON(IBUS, ID, MODEL, J, IDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.73. CHANGE_CCTMCNOMOD_VAR

Use this API to change the value of VAR of the machine "other" model for a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNOMOD_VAR  IBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctmcnomod_var(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNOMOD_VAR(IBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2015

8.12.74. CHANGE_CCTMCNPMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the machine protection model of a specified machine. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNPMOD_CHRICN  IBUS  'ID'  'MODEL' J  CHDATA
```

Python command syntax:

```
ierr = change_cctmcnpmod_chricn(ibus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNPMOD_CHRICN(IBUS, ID, MODEL, J, CHDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2016

8.12.75. CHANGE_CCTMCNPMOD_CON

Use this API to change the value of a real constant (CON) of the machine protection model of a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNPMOD_CON  IBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctmcnpmod_con(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNPMOD_CON(IBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2017

8.12.76. CHANGE_CCTMCNPMOD_ICON

Use this API to change the value of an integer ICON of the machine protection model of a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNPMOD_ICON  IBUS  'ID'  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_cctmcnpmod_icon(ibus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNPMOD_ICON(IBUS, ID, MODEL, J, IDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2018

8.12.77. CHANGE_CCTMCNPMOD_VAR

Use this API to change the value of VAR of the machine protection model of a specified machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMCNPMOD_VAR  IBUS  'ID'  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_cctmcnpmod_var(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMCNPMOD_VAR(IBUS, ID, MODEL, J, RDATA,  IERR)
```

Where:

Integer IBUS	machine bus number (input; no default allowed).	
Character*2 ID	machine identifier (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	machine not found
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2019

8.12.78. CHANGE_CCTMSCOMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the miscellaneous "other" type model with the specified model instance. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMSCOMOD_CHRICN  MODEL MINS J CHDATA
```

Python command syntax:

```
ierr = change_cctmscomod_chricn(model, mins, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMSCOMOD_CHRICN(MODEL, MINS, J, CHDATA,  IERR)
```

Where:

Character*16 MODEL	model name (input; no default allowed).	
Integer MINS	model instance (>0) (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model aaa with model instance xxx
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	invalid model instance value
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2020

8.12.79. CHANGE_CCTMSCOMOD_CON

Use this API to change the value of a real constant (CON) of the miscellaneous "other" type model with the specified model instance.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMSCOMOD_CON  'MODEL'  MINS J  RDATA
```

Python command syntax:

```
ierr = change_cctmscomod_con(model, mins, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMSCOMOD_CON(MODEL, MINS, J, RDATA,  IERR)
```

Where:

Character*16 MODEL	model name (input; no default allowed).	
Integer MINS	model instance (>0) (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model aaa with model instance xxx
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	invalid model instance value
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2021

8.12.80. CHANGE_CCTMSCOMOD_ICON

Use this API to change the value of an integer ICON of the miscellaneous "other" type model with the specified model instance.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMSCOMOD_ICON  'MODEL'  MINS J  IDATA
```

Python command syntax:

```
ierr = change_cctmscomod_icon(model, mins, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMSCOMOD_ICON(MODEL, MINS, J, IDATA,  IERR)
```

Where:

Character*16 MODEL	model name (input; no default allowed).	
Integer MINS	model instance (>0) (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model aaa with model instance xxx
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	invalid model instance value
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2022

8.12.81. CHANGE_CCTMSCOMOD_VAR

Use this API to change the value of VAR of the miscellaneous "other" type model with the specified model instance.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTMSCOMOD_VAR  'MODEL'  MINS J  RDATA
```

Python command syntax:

```
ierr = change_cctmscomod_var(model, mins, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTMSCOMOD_VAR(MODEL, MINS, J, RDATA,  IERR)
```

Where:

Character*16 MODEL	model name (input; no default allowed).	
Integer MINS	model instance (>0) (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model aaa with model instance xxx
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	invalid model instance value
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2023

8.12.82. CHANGE_CCTSWSOMOD_CHRICN

This API routine is obsolete. It has been replaced by the API routine change_cctswsomod_chricn_2 .

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the "other" type model attached to a specified switched shunt. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_CHRICN  IBUS MODEL J CHDATA
```

Python command syntax:

```
ierr = change_cctswsomod_chricn(ibus, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_CHRICN(IBUS, MODEL, J, CHDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2024

8.12.83. CHANGE_CCTSWSOMOD_CHRICN_2

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of the "other" type model attached to a specified switched shunt. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_CHRICN_2  IBUS ID MODEL J CHDATA
```

Python command syntax:

```
ierr = change_cctswsomod_chricn_2(ibus, id, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_CHRICN_2(IBUS, ID, MODEL, J, CHDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2025

8.12.84. CHANGE_CCTSWSOMOD_CON

This API routine is obsolete. It has been replaced by the API routine change_cctswsomod_con_2 .

Use this API to change the value of a real constant (CON) of the "other" type model attached to a specified switched shunt.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_CON  IBUS MODEL J RDATA
```

Python command syntax:

```
ierr = change_cctswsomod_con(ibus, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_CON(IBUS, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International
2026

8.12.85. CHANGE_CCTSWSOMOD_CON_2

Use this API to change the value of a real constant (CON) of the "other" type model attached to a specified switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_CON_2  IBUS ID MODEL J RDATA
```

Python command syntax:

```
ierr = change_cctswsomod_con_2(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_CON_2(IBUS, ID, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change (J is relative to the start of the block of CONs assigned to this model, e,g., 1 for first CON(K), 2 for CON(K+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no CONs used in the model
	IERR = 5	CON number outside of the CON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.86. CHANGE_CCTSWSOMOD_ICON

This API routine is obsolete. It has been replaced by the API routine change_cctswsomod_icon_2 .

Use this API to change the value of an integer ICON of the "other" type model attached to a specified switched shunt.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_ICON  IBUS MODEL J IDATA
```

Python command syntax:

```
ierr = change_cctswsomod_icon(ibus, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_ICON(IBUS, MODEL, J, IDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.87. CHANGE_CCTSWSOMOD_ICON_2

Use this API to change the value of an integer ICON of the "other" type model attached to a specified switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_ICON_2  IBUS ID MODEL J IDATA
```

Python command syntax:

```
ierr = change_cctswsomod_icon_2(ibus, id, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_ICON_2(IBUS, ID, MODEL, J, IDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change (J is relative to the start of the block of ICONs assigned to this model, e,g., 1 for first ICON(M), 2 for ICON(M +1), etc.) (input; no default allowed).	
Integer IDATA	new ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no ICONs used in the model
	IERR = 5	ICON number outside of the ICON range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.88. CHANGE_CCTSWSOMOD_VAR

This API routine is obsolete. It has been replaced by the API routine change_cctswsomod_var_2 .

Use this API to change the value of VAR of the "other" type model attached to a specified switched shunt.

This API routine was first introduced in release 33.8.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_VAR  IBUS MODEL J RDATA
```

Python command syntax:

```
ierr = change_cctswsomod_var(ibus, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_VAR(IBUS, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.12.89. CHANGE_CCTSWSOMOD_VAR_2

Use this API to change the value of VAR of the "other" type model attached to a specified switched shunt.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGE_CCTSWSOMOD_VAR_2  IBUS ID MODEL J RDATA
```

Python command syntax:

```
ierr = change_cctswsomod_var_2(ibus, id, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_CCTSWSOMOD_VAR_2(IBUS, ID, MODEL, J, RDATA, IERR)
```

Where:

Integer IBUS	switched shunt bus number (input; no default allowed).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change (J is relative to the start of the block of VARs assigned to this model, e,g., 1 for first VAR(L), 2 for VAR(L+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	unknown model name
	IERR = 2	no CCT table entry for the model
	IERR = 3	model names do not match - model is aaa not xxx
	IERR = 4	no VARs used in the model
	IERR = 5	VAR number outside of the VAR range of this model
	IERR = 6	bus not found
	IERR = 7	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13. Auxiliary Signal Models

Auxiliary Signal Model APIs handle the models that inject supplementary signals into dc lines and FACTS devices (auxiliary signal models).

8.13.1. ADD_FCTSAUXSIGNAL_MODEL

Use this API to add a FACTS device auxiliary signal model at the specified injection point of the designated FACTS device. If a model already exists at the specified injection point of the FACTS device, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_FCTSAUXSIGNAL_MODEL  FD  ISGNDX  'MODEL'  NICN  IDATA  CHDATA
 NCON  RDATA  IERR
```

Python command syntax:

```
ierr = add_fctsauxsignal_model(fd, isgndx, model, nicn, idata, chdata,
 ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_FCTSAUXSIGNAL_MODEL(NUM, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)

CALL ADD_FCTSAUXSIGNAL_MODEL_I(NUM, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)

CALL ADD_FCTSAUXSIGNAL_MODEL_C(NAME, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).
Character*12 NAME	name of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).
Character*16 MODEL	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer NCON	number of CONs required by the model (input; no default allowed).
Real RDATA (NCON)	CON data (input; 0.0).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	FACTS device not found
IERR = 2	invalid ISGNDX value for a FACTS device
IERR = 3	unknown model name
IERR = 4	model is not an auxiliary signal model
IERR = 5	# CONs specified is not the number expected
IERR = 6	# ICONs specified is not the number expected
IERR = 7	maximum number of array elements exceeded
IERR = 8	auxiliary signal model connection tables full--use pack auxiliary signal model tables function
IERR = 9	this auxiliary signal may be used only with two-terminal dc lines
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.2. ADD_MTDCAUXSIGNAL_MODEL

Use this API to add a multi-terminal dc line auxiliary signal model at the specified auxiliary signal index of the designated multi-terminal dc line. If a model already exists at the specified auxiliary signal index of the multi-terminal dc line, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_MTDCAUXSIGNAL_MODEL  DC  ISGNDX  'MODEL'  NICN  IDATA  CHDATA
 NCON  RDATA  IERR
```

Python command syntax:

```
ierr = add_mtdcauxsignal_model(dc, isgndx, model, nicn, idata, chdata,
 ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_MTDCAUXSIGNAL_MODEL(NUM, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)

CALL ADD_MTDCAUXSIGNAL_MODEL_I(NUM, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)

CALL ADD_MTDCAUXSIGNAL_MODEL_C(NAME, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).
Character*12 NAME	name of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).
Character*16 MODEL	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NCON	number of CONs required by the model (input; no default allowed).
Real RDATA (NCON)	CON data (input; 0.0).
Integer IERR	error code (output).

IERR = 0	no error
IERR = 1	dc line not found
IERR = 2	invalid ISGNDX value for a multi-terminal dc line
IERR = 3	unknown model name
IERR = 4	model is not an auxiliary signal model
IERR = 5	# CONs specified is not the number expected
IERR = 6	# ICONs specified is not the number expected
IERR = 7	maximum number of array elements exceeded
IERR = 8	auxiliary signal model connection tables full--use pack auxiliary signal model tables function
IERR = 9	this auxiliary signal may be used only with two-terminal dc lines
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.3. ADD_TTDCAUXSIGNAL_MODEL

Use this API to add a two-terminal dc line auxiliary signal model at the specified injection point of the designated two-terminal dc line. If a model already exists at the specified injection point of the two-terminal dc line, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_TTDCAUXSIGNAL_MODEL  DC  ISGNDX  'MODEL'  NICN  IDATA  CHDATA
 NCON  RDATA  IERR
```

Python command syntax:

```
ierr = add_ttdcauxsignal_model(dc, isgndx, model, nicn, idata, chdata,
 ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_TTDCAUXSIGNAL_MODEL(NUM, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)

CALL ADD_TTDCAUXSIGNAL_MODEL_I(NUM, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)

CALL ADD_TTDCAUXSIGNAL_MODEL_C(NAME, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).
Character*12 NAME	name of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).
Character*16 MODEL	model name (input; no default allowed).
Integer NICN	number of ICONs required by the model (input; no default allowed).
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a two-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	model is not an auxiliary signal model
	IERR = 5	# CONs specified is not the number expected
	IERR = 6	# ICONs specified is not the number expected
	IERR = 7	maximum number of array elements exceeded
	IERR = 8	auxiliary signal model connection tables full--use pack auxiliary signal model tables function
	IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.4. ADD_VSDCAUXSIGNAL_MODEL

Use this API to add a VSC dc line auxiliary signal model at the specified injection point of the designated VSC dc line. If a model already exists at the specified injection point of the VSC dc line, it is replaced.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_ADD_VSDCAUXSIGNAL_MODEL  'DCNAME'  ISGNDX  'MODEL'  NICN  IDATA
 CHDATA  NCON  RDATA  IERR
```

Python command syntax:

```
ierr = add_vsdcauxsignal_model(dcname, isgndx, model, nicn, idata, chdata,
 ncon, rdata)
```

Fortran command syntax:

```
CALL ADD_VSDCAUXSIGNAL_MODEL(DCNAME, ISGNDX, MODEL, NICN, IDATA, CHDATA,
 NCON, RDATA, IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer NICN	number of ICONs required by the model (input; no default allowed).	
Integer IDATA (NICN)	ICON data (input; 0 by default). When a character value is intended for an ICON value, the corresponding IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA (NICN)	character ICON data (no default allowed for an ICON for which a character value is to be specified). When an integer value is intended for an ICON value, the corresponding CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer NCON	number of CONs required by the model (input; no default allowed).	
Real RDATA (NCON)	CON data (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a VSC dc line
	IERR = 3	unknown model name
	IERR = 4	model is not an auxiliary signal model
	IERR = 5	# CONs specified is not the number expected
	IERR = 6	# ICONs specified is not the number expected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	maximum number of array elements exceeded
IERR = 8	auxiliary signal model connection tables full--use pack auxiliary signal model tables function
IERR = 9	this auxiliary signal may be used only with two-terminal dc lines
IERR = 10	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.5. AUXMOD_PACK

Use this API to remove entries that are marked as unused from the auxiliary signal model connection tables and the auxiliary signal array allocation tables. It packs the entries remaining in these tables at the front of the various arrays.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_AUXMOD_PACK
```

Python command syntax:

```
ierr = auxmod_pack()
```

Fortran command syntax:

```
CALL AUXMOD_PACK(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.6. AUXMOD_UNCONNECTED

Use this API to list or remove from dynamics working memory those auxiliary signal models that are assigned to dc lines or FACTS devices that are not present in the current power flow working case (unconnected).

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_AUXMOD_UNCONNECTED  OPT
```

Python command syntax:

```
ierr = auxmod_unconnected(opt)
```

Fortran command syntax:

```
CALL AUXMOD_UNCONNECTED(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list unconnected auxiliary signal models
	OPT = 1	remove unconnected auxiliary signal models
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.7. AUXMOD_USER

Use this API to list user-written auxiliary signal model definitions or to remove user-written auxiliary signal model definitions that are not assigned to any dc lines or FACTS devices (unused) from the user model definition tables.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_AUXMOD_USER  OPT
```

Python command syntax:

```
ierr = auxmod_user(opt)
```

Fortran command syntax:

```
CALL AUXMOD_USER(OPT, IERR)
```

Where:

Integer OPT	option code (input; 0 by default).	
	OPT = 0	list user-written auxiliary signal model definitions
	OPT = 1	remove unused user-written auxiliary signal model definitions
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	invalid OPT value
	IERR = 2	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.8. CHANGE_FCTSAUXMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of an auxiliary signal model of a specified FACTS device. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTSAUXMOD_CHRICN  FD  ISGNDX  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_fctsauxmod_chricn(fd, isgndx, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTSAUXMOD_CHRICN(NUM, ISGNDX, MODEL, J, CHDATA, IERR)

CALL CHANGE_FCTSAUXMOD_CHRICN_I(NUM, ISGNDX, MODEL, J, CHDATA, IERR)

CALL CHANGE_FCTSAUXMOD_CHRICN_C(NAME, ISGNDX, MODEL, J, CHDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	invalid ISGNDX value for a FACTS device
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this FACTS device
	IERR = 5	model at this index for this FACTS device is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no input ICONs used in model
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.9. CHANGE_FCTSAUXMOD_CON

Use this API to change the value of a CON of an auxiliary signal model of a specified FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTSAUXMOD_CON  FD  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_fctsauxmod_con(fd, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTSAUXMOD_CON(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTSAUXMOD_CON_I(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTSAUXMOD_CON_C(NAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	invalid ISGNDX value for a FACTS device
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this FACTS device
	IERR = 5	model at this index for this FACTS device is aaa not xxx
	IERR = 6	no CONs used in model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	CON number outside of the CON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.10. CHANGE_FCTSAUXMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_FCTSAUXMOD_CON, CHANGE_FCTSAUXMOD_ICON, and CHANGE_FCTSAUXMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of an auxiliary signal model of a specified FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTSAUXMOD_DATA  FD  ISGNDX  'NAME'  J  RDATA  I  IDATA
 'CHDATA'
```

Python command syntax:

```
ierr = change_fctsauxmod_data(fd, isgndx, name, j, rdata, i, idata,
 chdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTSAUXMOD_DATA(FD, ISGNDX, NAME, J, RDATA, I, IDATA, CHDATA,
 IERR)
```

Where:

Integer FD	number of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).	
Real RDATA	new CON value (input; unchanged).	
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).	
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ISGNDX value for a FACTS device
IERR = 3	unknown model name
IERR = 4	no auxiliary signal model at this index for this FACTS device
IERR = 5	model at this index for this FACTS device is aaa not xxx
IERR = 6	no CONs used in model; or
IERR = 6	no input ICONs used in model; or
IERR = 7	CON number outside of the CON range of this model; or
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.11. CHANGE_FCTSAUXMOD_ICON

Use this API to change the value of an integer ICON of an auxiliary signal model of a specified FACTS device. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_FCTSAUXMOD_ICON  FD  ISGNDX  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_fctsauxmod_icon(fd, isgndx, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_FCTSAUXMOD_ICON(NUM, ISGNDX, MODEL, J, IDATA, IERR)

CALL CHANGE_FCTSAUXMOD_ICON_I(NUM, ISGNDX, MODEL, J, IDATA, IERR)

CALL CHANGE_FCTSAUXMOD_ICON_C(NAME, ISGNDX, MODEL, J, IDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 2	FACTS device not found
	IERR = 3	invalid ISGNDX value for a FACTS device
	IERR = 4	unknown model name
	IERR = 5	no auxiliary signal model at this index for this FACTS device
	IERR = 6	model at this index for this FACTS device is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	no input ICONs used in model
IERR = 8	ICON number outside of the ICON range of this model
IERR = 9	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.12. CHANGE_FCTSAUXMOD_VAR

Use this API to change the value of a VAR of an auxiliary signal model of a specified FACTS device.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_FCTSAUXMOD_VAR  FD  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_fctsauxmod_var(fd, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_FCTSAUXMOD_VAR(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTSAUXMOD_VAR_I(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_FCTSAUXMOD_VAR_C(NAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	invalid ISGNDX value for a FACTS device
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this FACTS device
	IERR = 5	model at this index for this FACTS device is aaa not xxx
	IERR = 6	no VARs used in model

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	VAR number outside of the VAR range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.13. CHANGE_MTDCAUXMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of an auxiliary signal model of a specified multi-terminal dc line. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCAUXMOD_CHRICN  DC  ISGNDX  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_mtdcauxmod_chricn(dc, isgndx, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCAUXMOD_CHRICN(NUM, ISGNDX, MODEL, J, CHDATA, IERR)

CALL CHANGE_MTDCAUXMOD_CHRICN_I(NUM, ISGNDX, MODEL, J, CHDATA, IERR)

CALL CHANGE_MTDCAUXMOD_CHRICN_C(NAME, ISGNDX, MODEL, J, CHDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a multi-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this multi-terminal dc line
	IERR = 5	model at this index for this multi-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no input ICONs used in model
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.14. CHANGE_MTDCAUXMOD_CON

Use this API to change the value of a CON of an auxiliary signal model of a specified multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCAUXMOD_CON  DC  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_mtdcauxmod_con(dc, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCAUXMOD_CON(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCAUXMOD_CON_I(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCAUXMOD_CON_C(NAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a multi-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this multi-terminal dc line
	IERR = 5	model at this index for this multi-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no CONs used in model
IERR = 7	CON number outside of the CON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.15. CHANGE_MTDCAUXMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_MTDCAUXMOD_CON, CHANGE_MTDCAUXMOD_ICON, and CHANGE_MTDCAUXMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of an auxiliary signal model of a specified multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCAUXMOD_DATA  DC  ISGNDX  'NAME'  J  RDATA  I  IDATA
 'CHDATA'
```

Python command syntax:

```
ierr = change_mtdcauxmod_data(dc,isgndx,name,j,rdata,i,idata,chdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCAUXMOD_DATA(DC, ISGNDX, NAME, J, RDATA, I, IDATA, CHDATA,
 IERR)
```

Where:

Integer DC	number of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).	
Real RDATA	new CON value (input; unchanged).	
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).	
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ISGNDX value for a multi-terminal dc line
IERR = 3	unknown model name
IERR = 4	no auxiliary signal model at this index for this multi-terminal dc line
IERR = 5	model at this index for this multi-terminal dc line is aaa not xxx
IERR = 6	no CONs used in model; or
IERR = 6	no input ICONs used in model; or
IERR = 7	CON number outside of the CON range of this model; or
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.16. CHANGE_MTDCAUXMOD_ICON

Use this API to change the value of an integer ICON of an auxiliary signal model of a specified multi-terminal dc line. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_MTDCAUXMOD_ICON  DC  ISGNDX  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_mtdcauxmod_icon(dc, isgndx, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCAUXMOD_ICON(NUM, ISGNDX, MODEL, J, IDATA, IERR)

CALL CHANGE_MTDCAUXMOD_ICON_I(NUM, ISGNDX, MODEL, J, IDATA, IERR)

CALL CHANGE_MTDCAUXMOD_ICON_C(NAME, ISGNDX, MODEL, J, IDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a multi-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this multi-terminal dc line
	IERR = 5	model at this index for this multi-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no input ICONs used in model
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.17. CHANGE_MTDCAUXMOD_VAR

Use this API to change the value of a VAR of an auxiliary signal model of a specified multi-terminal dc line.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_MTDCAUXMOD_VAR  DC  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_mtdcauxmod_var(dc, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_MTDCAUXMOD_VAR(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCAUXMOD_VAR_I(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_MTDCAUXMOD_VAR_C(NAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a multi-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this multi-terminal dc line
	IERR = 5	model at this index for this multi-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no VARs used in model
IERR = 7	VAR number outside of the VAR range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.18. CHANGE_TTDCAUXMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of an auxiliary signal model of a specified two-terminal dc line. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCAUXMOD_CHRICN  DC  ISGNDX  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_ttdcauxmod_chricn(dc, isgndx, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCAUXMOD_CHRICN(NUM, ISGNDX, MODEL, J, CHDATA, IERR)

CALL CHANGE_TTDCAUXMOD_CHRICN_I(NUM, ISGNDX, MODEL, J, CHDATA, IERR)

CALL CHANGE_TTDCAUXMOD_CHRICN_C(NAME, ISGNDX, MODEL, J, CHDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a two-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this two-terminal dc line
	IERR = 5	model at this index for this two-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no input ICONs used in model
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.19. CHANGE_TTDCAUXMOD_CON

Use this API to change the value of a CON of an auxiliary signal model of a specified two-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCAUXMOD_CON  DC  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_ttdcauxmod_con(dc, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCAUXMOD_CON( NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCAUXMOD_CON_I(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCAUXMOD_CON_C(NAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a two-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this two-terminal dc line
	IERR = 5	model at this index for this two-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no CONs used in model
IERR = 7	CON number outside of the CON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.20. CHANGE_TTDCAUXMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_TTDCAUXMOD_CON, CHANGE_TTDCAUXMOD_ICON, and CHANGE_TTDCAUXMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of an auxiliary signal model of a specified two-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCAUXMOD_DATA  DC  ISGNDX  'NAME'  J  RDATA  I  IDATA
 'CHDATA'
```

Python command syntax:

```
ierr = change_ttdcauxmod_data(dc,isgndx,name,j,rdata,i,idata,chdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCAUXMOD_DATA(DC, ISGNDX, NAME, J, RDATA, I, IDATA, CHDATA,
 IERR)
```

Where:

Integer DC	number of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 NAME	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).	
Real RDATA	new CON value (input; unchanged).	
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).	
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).	
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	invalid ISGNDX value for a two-terminal dc line
IERR = 3	unknown model name
IERR = 4	no auxiliary signal model at this index for this two-terminal dc line
IERR = 5	model at this index for this two-terminal dc line is aaa not xxx
IERR = 6	no CONs used in model
IERR = 6	no input ICONs used in model
IERR = 7	CON number outside of the CON range of this model
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.21. CHANGE_TTDCAUXMOD_ICON

Use this API to change the value of an integer ICON of an auxiliary signal model of a specified two-terminal dc line. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_TTDCAUXMOD_ICON  DC  ISGNDX  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_ttdcauxmod_icon(dc, isgndx, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCAUXMOD_ICON(NUM, ISGNDX, MODEL, J, IDATA, IERR)

CALL CHANGE_TTDCAUXMOD_ICON_I(NUM, ISGNDX, MODEL, J, IDATA, IERR)

CALL CHANGE_TTDCAUXMOD_ICON_C(NAME, ISGNDX, MODEL, J, IDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a two-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this two-terminal dc line
	IERR = 5	model at this index for this two-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no input ICONs used in model
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.22. CHANGE_TTDCAUXMOD_VAR

Use this API to change the value of a VAR of an auxiliary signal model of a specified two-terminal dc line.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_TTDCAUXMOD_VAR  DC  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_ttdcauxmod_var(dc, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_TTDCAUXMOD_VAR(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCAUXMOD_VAR_I(NUM, ISGNDX, MODEL, J, RDATA, IERR)

CALL CHANGE_TTDCAUXMOD_VAR_C(NAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a two-terminal dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this two-terminal dc line
	IERR = 5	model at this index for this two-terminal dc line is aaa not xxx

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	no VARs used in model
IERR = 7	VAR number outside of the VAR range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.23. CHANGE_VSDCAUXMOD_CHRICN

Use this API to change the value of a character ICON (i.e., an element of the CHRICN array) of an auxiliary signal model of a specified VSC dc line. The corresponding integer ICON is set to 0.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCAUXMOD_CHRICN  'DCNAME'  ISGNDX  'MODEL'  J  'CHDATA'
```

Python command syntax:

```
ierr = change_vsdcauxmod_chricn(dcname, isgndx, model, j, chdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCAUXMOD_CHRICN(DCNAME,ISGNDX,MODEL,J,CHDATA,IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Character*12 CHDATA	new character ICON value (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a VSC dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this VSC dc line
	IERR = 5	model at this index for this VSC dc line is aaa not xxx
	IERR = 6	no input ICONs used in model
	IERR = 7	ICON number outside of the ICON range of this model
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.24. CHANGE_VSDCAUXMOD_CON

Use this API to change the value of a CON of an auxiliary signal model of a specified VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCAUXMOD_CON  'DCNAME'  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_vsdcauxmod_con(dcname, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCAUXMOD_CON(DCNAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	CON number to change relative to the start of the block of CONs assigned to this model (1 for CON(J), 2 for CON(J+1), etc.) (input; no default allowed).	
Real RDATA	new CON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a VSC dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this VSC dc line
	IERR = 5	model at this index for this VSC dc line is aaa not xxx
	IERR = 6	no CONs used in model
	IERR = 7	CON number outside of the CON range of this model
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.25. CHANGE_VSDCAUXMOD_DATA

This API routine is obsolete. It has been replaced by the API routines CHANGE_VSDCAUXMOD_CON, CHANGE_VSDCAUXMOD_ICON, and CHANGE_VSDCAUXMOD_CHRICN .

Use this API to change the value of an ICON and/or CON of an auxiliary signal model of a specified VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCAUXMOD_DATA 'DCNAME' ISGNDX 'NAME' J RDATA I IDATA
 'CHDATA'
```

Python command syntax:

```
ierr = change_vsdcauxmod_data(dcname,isgndx,name,j,rdata,i,idata,chdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCAUXMOD_DATA(DCNAME, ISGNDX, NAME, J, RDATA, I, IDATA,
 CHDATA, IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).
Character*16 NAME	model name (input; no default allowed).
Integer J	CON number to change relative to the start of the block of CONs assigned to this model, or 0 for no CON change (1 for CON(J), 2 for CON(J+1), etc.) (input; 0 by default).
Real RDATA	new CON value (input; unchanged).
Integer I	ICON number to change relative to the start of the block of ICONs assigned to this model, or 0 for no ICON change (1 for ICON(I), 2 for ICON(I+1), etc.) (input; 0 by default).
Integer IDATA	new integer ICON value (input; unchanged by default). When a character value is intended for an ICON value, the IDATA entry is ignored and the integer ICON is set to 0).
Character*12 CHDATA	new character ICON value (no default allowed if the character ICON is to be changed). When an integer value is intended for an ICON value, the CHDATA entry must be either defaulted or specified as either '$$' or ' '; the character ICON is set to '$$' (input; blank by default).
Integer IERR	error code (output). IERR = 0 no error IERR = 1 dc line not found IERR = 2 invalid ISGNDX value for a VSC dc line

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	unknown model name
IERR = 4	no auxiliary signal model at this index for this VSC dc line
IERR = 5	model at this index for this VSC dc line is aaa not xxx
IERR = 6	no CONs used in model
IERR = 6	no input ICONs used in model
IERR = 7	CON number outside of the CON range of this model
IERR = 7	ICON number outside of the ICON range of this model
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.26. CHANGE_VSDCAUXMOD_ICON

Use this API to change the value of an integer ICON of an auxiliary signal model of a specified VSC dc line. The corresponding character CHRICN is set to '$$'.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CHANGE_VSDCAUXMOD_ICON  'DCNAME'  ISGNDX  'MODEL'  J  IDATA
```

Python command syntax:

```
ierr = change_vsdcauxmod_icon(dcname, isgndx, model, j, idata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCAUXMOD_ICON(DCNAME, ISGNDX, MODEL, J, IDATA, IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	ICON number to change relative to the start of the block of ICONs assigned to this model (1 for ICON(I), 2 for ICON(I+1), etc.) (input; no default allowed).	
Integer IDATA	new integer ICON value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a VSC dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this VSC dc line
	IERR = 5	model at this index for this VSC dc line is aaa not xxx
	IERR = 6	no input ICONs used in model
	IERR = 7	ICON number outside of the ICON range of this model
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.27. CHANGE_VSDCAUXMOD_VAR

Use this API to change the value of a VAR of an auxiliary signal model of a specified VSC dc line.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CHANGE_VSDCAUXMOD_VAR  'DCNAME'  ISGNDX  'MODEL'  J  RDATA
```

Python command syntax:

```
ierr = change_vsdcauxmod_var(dcname, isgndx, model, j, rdata)
```

Fortran command syntax:

```
CALL CHANGE_VSDCAUXMOD_VAR(DCNAME, ISGNDX, MODEL, J, RDATA, IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Character*16 MODEL	model name (input; no default allowed).	
Integer J	VAR number to change relative to the start of the block of VARs assigned to this model (1 for VAR(J), 2 for VAR(J+1), etc.) (input; no default allowed).	
Real RDATA	new VAR value (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a VSC dc line
	IERR = 3	unknown model name
	IERR = 4	no auxiliary signal model at this index for this VSC dc line
	IERR = 5	model at this index for this VSC dc line is aaa not xxx
	IERR = 6	no VARs used in model
	IERR = 7	VAR number outside of the VAR range of this model
	IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.28. FCTSAUXMOD_REMOVE

Use this API to remove an auxiliary signal model from the specified injection point of a designated FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCTSAUXMOD_REMOVE  FD  ISGNDX
```

Python command syntax:

```
ierr = fctsauxmod_remove(fd, isgndx)
```

Fortran command syntax:

```
CALL FCTSAUXMOD_REMOVE(NUM, ISGNDX, IERR)

CALL FCTSAUXMOD_REMOVE_I(NUM, ISGNDX, IERR)

CALL FCTSAUXMOD_REMOVE_C(NAME, ISGNDX, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	invalid ISGNDX value for a FACTS device
	IERR = 3	no auxiliary signal model at this index for this FACTS device
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.29. FCTSAUXMOD_STATUS

Use this API to change the status of the auxiliary signal model at the specified injection point of a designated FACTS device.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_FCTSAUXMOD_STATUS  FD  ISGNDX  STATUS
```

Python command syntax:

```
ierr = fctsauxmod_status(fd, isgndx, status)
```

Fortran command syntax:

```
CALL FCTSAUXMOD_STATUS(NUM
       , ISGNDX, STATUS, IERR)

CALL FCTSAUXMOD_STATUS_I(NUM, ISGNDX, STATUS, IERR)

CALL FCTSAUXMOD_STATUS_C(NAME, ISGNDX, STATUS, IERR)
```

Where:

Integer or Character*12 FD	quoted name of the FACTS device receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the FACTS device receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	FACTS device not found
	IERR = 2	invalid ISGNDX value for a FACTS device
	IERR = 3	no auxiliary signal model at this index for this FACTS device
	IERR = 4	invalid STATUS value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.30. MTDCAUXMOD_REMOVE

Use this API to remove an auxiliary signal model from the specified auxiliary signal index of the designated multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MTDCAUXMOD_REMOVE  DC  ISGNDX
```

Python command syntax:

```
ierr = mtdcauxmod_remove(dc, isgndx)
```

Fortran command syntax:

```
CALL MTDCAUXMOD_REMOVE(NUM, ISGNDX, IERR)

CALL MTDCAUXMOD_REMOVE_I(NUM, ISGNDX, IERR)

CALL MTDCAUXMOD_REMOVE_C(NAME, ISGNDX, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a multi-terminal dc line
	IERR = 3	no auxiliary signal model at this index for this multi-terminal dc line
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.31. MTDCAUXMOD_STATUS

Use this API to change the status of the auxiliary signal model at the specified auxiliary signal index of the designated multi-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_MTDCAUXMOD_STATUS  DC  ISGNDX  STATUS
```

Python command syntax:

```
ierr = mtdcauxmod_status(dc, isgndx, status)
```

Fortran command syntax:

```
CALL MTDCAUXMOD_STATUS(NUM, ISGNDX, STATUS, IERR)

CALL MTDCAUXMOD_STATUS_I(NUM, ISGNDX, STATUS, IERR)

CALL MTDCAUXMOD_STATUS_C(NAME, ISGNDX, STATUS, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the multi-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the multi-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a multi-terminal dc line
	IERR = 3	no auxiliary signal model at this index for this multi-terminal dc line
	IERR = 4	invalid STATUS value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.32. TTDCAUXMOD_REMOVE

Use this API to remove an auxiliary signal model from the specified injection point of the designated two-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_TTDCAUXMOD_REMOVE  DC  ISGNDX
```

Python command syntax:

```
ierr = ttdcauxmod_remove(dc, isgndx)
```

Fortran command syntax:

```
CALL TTDCAUXMOD_REMOVE(NUM, ISGNDX, IERR)

CALL TTDCAUXMOD_REMOVE_I(NUM, ISGNDX, IERR)

CALL TTDCAUXMOD_REMOVE_C(NAME, ISGNDX, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a two-terminal dc line
	IERR = 3	no auxiliary signal model at this index for this two-terminal dc line
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.33. TTDCAUXMOD_STATUS

Use this API to change the status of the auxiliary signal model at the specified injection point of the designated two-terminal dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_TTDCAUXMOD_STATUS  DC  ISGNDX  STATUS
```

Python command syntax:

```
ierr = ttdcauxmod_status(dc, isgndx, status)
```

Fortran command syntax:

```
CALL TTDCAUXMOD_STATUS(NUM, ISGNDX, STATUS, IERR)

CALL TTDCAUXMOD_STATUS_I(NUM, ISGNDX, STATUS, IERR)

CALL TTDCAUXMOD_STATUS_C(NAME, ISGNDX, STATUS, IERR)
```

Where:

Integer or Character*12 DC	quoted name of the two-terminal dc line receiving the of the auxiliary signal model; if an unquoted integer value is specified, the API converts it to a left-justified name (input; no default allowed).	
Integer NUM	number (which the API converts to a left-justified name) of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Character*12 NAME	name of the two-terminal dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 2	dc line not found
	IERR = 3	invalid ISGNDX value for a two-terminal dc line
	IERR = 4	no auxiliary signal model at this index for this two-terminal dc line
	IERR = 5	invalid STATUS value
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.34. VSDCAUXMOD_REMOVE

Use this API to remove an auxiliary signal model from the specified injection point of the designated VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_VSDCAUXMOD_REMOVE  'DCNAME'  ISGNDX
```

Python command syntax:

```
ierr = vsdcauxmod_remove(dcname, isgndx)
```

Fortran command syntax:

```
CALL VSDCAUXMOD_REMOVE(DCNAME, ISGNDX, IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	dc line not found
	IERR = 2	invalid ISGNDX value for a VSC dc line
	IERR = 3	no auxiliary signal model at this index for this VSC dc line
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.13.35. VSDCAUXMOD_STATUS

Use this API to change the status of the auxiliary signal model at the specified injection point of the designated VSC dc line.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_VSDCAUXMOD_STATUS  'DCNAME'  ISGNDX  STATUS
```

Python command syntax:

```
ierr = vsdcauxmod_status(dcname, isgndx, status)
```

Fortran command syntax:

```
CALL VSDCAUXMOD_STATUS(DCNAME, ISGNDX, STATUS, IERR)
```

Where:

Character*12 DCNAME	name of the VSC dc line receiving the of the auxiliary signal model (input; no default allowed).	
Integer ISGNDX	auxiliary signal injection point number, or the auxiliary signal index (input; 1 by default).	
Integer STATUS	new model status (input; 1 by default).	
	STATUS = 0	out-of-service
	STATUS = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	VSC dc line not found
	IERR = 2	invalid ISGNDX value for a VSC dc line
	IERR = 3	no auxiliary signal model at this index for this VSC dc line
	IERR = 4	invalid STATUS value
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14. Set Disturbance

Set Disturbance, contains API routines for applying and removing disturbances during dynamics simulations.

8.14.1. DIST_3PHASE_BUS_FAULT

Use this API to apply a three phase fault at a bus during dynamic simulations.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_DIST_3PHASE_BUS_FAULT  IBUS  NODE  UNITS  BASEKV  VALUES(1)  VALUES(2)
```

Python command syntax:

```
ierr = dist_3phase_bus_fault(ibus, node, units, basekv, values)
```

Fortran command syntax:

```
CALL DIST_3PHASE_BUS_FAULT(IBUS, NODE, UNITS, BASEKV, VALUES, IERR)
```

Where:

Integer IBUS	bus number of the bus at which the fault is to be placed (input; no default allowed).	
Integer NODE	node number of a substation node connected to the specified bus; this defines the bus section to be faulted. If 0 is specified for a bus in a substation, the 'parent' bus is faulted; 0 must be specified for a bus that is not in a substation (input; 0 by default).	
Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default).	
	UNITS = 1	admittance in MVA
	UNITS = 2	admittance in mhos
	UNITS = 3	impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).	
Real VALUES (2)	array of 2 elements specifying (input).	
	VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
	VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	fixed bus shunt table is full
IERR = 3	bus is out-of-service
IERR = 4	invalid UNITS value
IERR = 5	invalid BASEKV value (<0.0)
IERR = 6	both BASEKV and the base voltage of bus IBUS are 0.0
IERR = 7	invalid bus number
IERR = 8	bus not found
IERR = 9	invalid node number
IERR = 10	bus does not belong to a substation
IERR = 11	node not found in the substation of this bus
IERR = 12	node is not connected to any bus section of this bus
IERR = 13	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.2. DIST_3WIND_CLOSE

Use this API to set a three-winding transformer to in-service during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_3WIND_CLOSE  IBUS  JBUS  KBUS  'ID'
```

Python command syntax:

```
ierr = dist_3wind_close(ibus, jbus, kbus, id)
```

Fortran command syntax:

```
CALL DIST_3WIND_CLOSEAPI(IBUS, JBUS, KBUS, ID, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number of one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	three-winding transformer not found
	IERR = 4	three-winding transformer is already in-service
	IERR = 5	bus is out-of-service
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.3. DIST_3WIND_FAULT

Use this API to apply a fault at the IBUS end of a three-winding transformer during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_3WIND_FAULT  IBUS  JBUS  KBUS  'ID'  UNITS  BASEKV  VALUES(1)
 VALUES(2)
```

Python command syntax:

```
ierr = dist_3wind_fault(ibus,jbus,kbus,id,units,basekv,values)
```

Fortran command syntax:

```
CALL DIST_3WIND_FAULTAPI(IBUS,JBUS,KBUS,ID,UNITS,BASEKV,VALUES,IERR)
```

Where:

Integer IBUS	bus number of the bus at which the fault is to be placed (input; no default allowed).	
Integer JBUS	bus number of one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default).	
	UNITS = 1	admittance in MVA
	UNITS = 2	admittance in mhos
	UNITS = 3	impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).	
Real VALUES (2)	array of 2 elements specifying (input).	
	VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
	VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	three-winding transformer not found
	IERR = 4	three-winding transformer winding is out-of-service

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	invalid UNITS value
IERR = 6	invalid BASEKV value (<0.0)
IERR = 7	both BASEKV and the base voltage of bus IBUS are 0.0
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.4. DIST_3WIND_TRIP

Use this API to set a three-winding transformer to out-of-service during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_3WIND_TRIP  IBUS  JBUS  KBUS  'ID'
```

Python command syntax:

```
ierr = dist_3wind_trip(ibus, jbus, kbus, id)
```

Fortran command syntax:

```
CALL DIST_3WIND_TRIPAPI(IBUS, JBUS, KBUS, ID, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number of one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	three-winding transformer not found
	IERR = 4	three-winding transformer is already out-of-service
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.5. DIST_BRANCH_CLOSE

Use this API to set a non-transformer branch or a two-winding transformer to in-service during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_BRANCH_CLOSE  IBUS  JBUS  'ID'
```

Python command syntax:

```
ierr = dist_branch_close(ibus, jbus, id)
```

Fortran command syntax:

```
CALL DIST_BRANCH_CLOSEAPI(IBUS, JBUS, ID, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number of the other bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	branch is already in-service
	IERR = 5	bus is out-of-service
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.6. DIST_BRANCH_FAULT

Use this API to apply a fault at the IBUS end of a non-transformer branch or a two-winding transformer during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_BRANCH_FAULT  IBUS  JBUS  'ID'  UNITS  BASEKV  VALUES(1)
 VALUES(2)
```

Python command syntax:

```
ierr = dist_branch_fault(ibus, jbus, id, units, basekv, values)
```

Fortran command syntax:

```
CALL DIST_BRANCH_FAULTAPI(IBUS,JBUS,ID,UNITS,BASEKV,VALUES,IERR)
```

Where:

Integer IBUS	bus number of the bus at which the fault is to be placed (input; no default allowed).	
Integer JBUS	bus number of the other bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default).	
	UNITS = 1	admittance in MVA
	UNITS = 2	admittance in mhos
	UNITS = 3	impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).	
Real VALUES (2)	array of 2 elements specifying (input).	
	VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
	VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	branch is out-of-service
	IERR = 5	invalid UNITS value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	invalid BASEKV value (<0.0)
IERR = 7	both BASEKV and the base voltage of bus IBUS are 0.0
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.7. DIST_BRANCH_TRIP

Use this API to set a non-transformer branch or a two-winding transformer to out-of-service during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_BRANCH_TRIP  IBUS  JBUS  'ID'
```

Python command syntax:

```
ierr = dist_branch_trip(ibus, jbus, id)
```

Fortran command syntax:

```
CALL DIST_BRANCH_TRIPAPI(IBUS, JBUS, ID, IERR)
```

Where:

Integer IBUS	bus number of one of the buses (input; no default allowed).	
Integer JBUS	bus number of the other bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	branch is already out-of-service
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.8. DIST_BUS_FAULT

This API routine is obsolete. It has been replaced by the API routine dist_3phase_bus_fault .

Use this API to apply a fault at a bus during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_BUS_FAULT  IBUS  UNITS  BASEKV  VALUES(1)  VALUES(2)
```

Python command syntax:

```
ierr = dist_bus_fault(ibus, units, basekv, values)
```

Fortran command syntax:

```
CALL DIST_BUS_FAULTAPI(IBUS, UNITS, BASEKV, VALUES, IERR)
```

Where:

Integer IBUS	bus number of the bus at which the fault is to be placed (input; no default allowed).	
Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default).	
	UNITS = 1	admittance in MVA
	UNITS = 2	admittance in mhos
	UNITS = 3	impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).	
Real VALUES (2)	array of 2 elements specifying (input).	
	VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
	VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	as in DIST_3PHASE_BUS_FAULT

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.9. DIST_BUS_FAULT_2

This API routine is obsolete. It has been replaced by the API routine dist_bus_fault_3 .

Use this API to apply a fault at a bus during dynamic simulations.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_DIST_BUS_FAULT_2  UNITS BASEKV OPTIONS(1)..OPTIONS(4)
 VALUES(1)..VALUES(6)
```

Python command syntax:

```
ierr = dist_bus_fault_2(units, basekv, options, values)
```

Fortran command syntax:

```
CALL DIST_BUS_FAULT_2(UNITS, BASEKV, OPTIONS, VALUES, IERR)
```

Where:

Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default). UNITS = 1 admittance in MVA UNITS = 2 admittance in mhos UNITS = 3 impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).
Integer OPTIONS (4)	array of 4 elements specifying options and values. The first element of OPTIONS defines the option to be used in the construction of the sequence admittance matrices; the remaining entries define the fault to be applied(input). OPTIONS(1) transformer impedance correction option (1 by default) OPTIONS(1) = 0 do not apply to zero sequence OPTIONS(1) = 1 apply to zero sequence OPTIONS(2) unbalanced type (1 by default) OPTIONS(2) = 1 line-to-ground fault OPTIONS(2) = 2 line-to-line or line-to-line-to-ground fault OPTIONS(3) bus to be faulted (no default allowed) OPTIONS(4) exceptional phase (1 by default) OPTIONS(4) = 1 phase A OPTIONS(4) = 2 phase B OPTIONS(4) = 3 phase C
Real VALUES (6)	array of 6 elements specifying values used to define the fault. The first two elements of VALUES define the fault admittance or imped-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ance to be used in the simulation; the remaining entries are used to define the fault if phase voltages are to be calculated during the simulation.(input).

VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
VALUES(3)	resistance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(2) is 2) fault impedance in pu (0.0 by default)
VALUES(4)	reactance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(2) is 2) fault impedance in pu (0.0 by default)
VALUES(5)	resistance of the complex L-G fault impedance in pu (if OPTIONS(2) is 2) (0.0 by default)
VALUES(6)	reactance of the complex L-G fault impedance in pu (if OPTIONS(2) is 2) (0.0 by default)

Integer IERR — error code (output).

IERR = 0	no error occurred
IERR > 0	as in DIST_BUS_FAULT_3

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.10. DIST_BUS_FAULT_3

Use this API to apply an unbalanced fault at a bus during dynamic simulations.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_DIST_BUS_FAULT_3  UNITS BASEKV OPTIONS(1)..OPTIONS(5)
 VALUES(1)..VALUES(6)
```

Python command syntax:

```
ierr = dist_bus_fault_3(units, basekv, options, values)
```

Fortran command syntax:

```
CALL DIST_BUS_FAULT_3(UNITS, BASEKV, OPTIONS, VALUES, IERR)
```

Where:

Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default).	
	UNITS = 1	admittance in MVA
	UNITS = 2	admittance in mhos
	UNITS = 3	impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).	
Integer OPTIONS (5)	array of 5 elements specifying options and values. The first element of OPTIONS defines the option to be used in the construction of the sequence admittance matrices; the remaining entries define the fault to be applied(input).	
	OPTIONS(1)	transformer impedance correction option (1 by default)
	OPTIONS(1) = 0	do not apply to zero sequence
	OPTIONS(1) = 1	apply to zero sequence
	OPTIONS(2)	unbalanced type (1 by default)
	OPTIONS(2) = 1	line-to-ground fault
	OPTIONS(2) = 2	line-to-line or line-to-line-to-ground fault
	OPTIONS(3)	bus to be faulted (no default allowed)
	OPTIONS(4)	node number of a substation node connected to the specified bus; this defines the bus section to be faulted. If 0 is specified for a bus in a substation, the 'parent' bus is faulted; 0 must be specified for a bus that is not in a substation (0 by default)
	OPTIONS(5)	exceptional phase (1 by default)
	OPTIONS(5) = 1	phase A
	OPTIONS(5) = 2	phase B

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 3	phase C
Real VALUES (6)	array of 6 elements specifying values used to define the fault. The first two elements of VALUES define the fault admittance or impedance to be used in the simulation; the remaining entries are used to define the fault if phase voltages are to be calculated during the simulation.(input).	
	VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
	VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
	VALUES(3)	resistance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(2) is 2) fault impedance in pu (0.0 by default)
	VALUES(4)	reactance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(2) is 2) fault impedance in pu (0.0 by default)
	VALUES(5)	resistance of the complex L-G fault impedance in pu (if OPTIONS(2) is 2) (0.0 by default)
	VALUES(6)	reactance of the complex L-G fault impedance in pu (if OPTIONS(2) is 2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	fixed bus shunt table is full
	IERR = 3	bus is out-of-service
	IERR = 4	invalid UNITS value
	IERR = 5	invalid BASEKV value (<0.0)
	IERR = 6	both BASEKV and the base voltage of bus IBUS are 0.0
	IERR = 7	invalid bus number
	IERR = 8	bus not found
	IERR = 9	invalid node number
	IERR = 10	bus does not belong to a substation
	IERR = 11	node not found in the substation of this bus
	IERR = 12	node is not connected to any bus section of this bus
	IERR = 13	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.11. DIST_BUS_TRIP

Use this API to disconnect a bus during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_BUS_TRIP  IBUS
```

Python command syntax:

```
ierr = dist_bus_trip(ibus)
```

Fortran command syntax:

```
CALL DIST_BUS_TRIPAPI(IBUS, IERR)
```

Where:

Integer IBUS	bus number of the bus to be disconnected (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR not successfully executed
	IERR = 2	bus not found
	IERR = 3	bus is already disconnected
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.12. DIST_CLEAR_FAULT

Use this API to clear a fault during dynamic simulations. The fault must have previously been applied using one of the following APIs:
- DIST_3WIND_FAULT
- DIST_BRANCH_FAULT
- DIST_BUS_FAULT_3
- DIST_SCMU_FAULT_3
- DIST_SPCB_FAULT_2

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_CLEAR_FAULT  FAULT
```

Python command syntax:

```
ierr = dist_clear_fault(fault)
```

Fortran command syntax:

```
CALL DIST_CLEAR_FAULTAPI(FAULT, IERR)
```

Where:

Integer FAULT	index in the fault memory tables of the fault to be cleared (input; 1 by default). Faults are stored in the fault memory tables in the order in which they are applied; the index assigned to a fault is the next available location in the tables. Each time a fault is cleared using this API, the fault memory tables are compressed. Thus, if there are three faults active, the most recently applied fault will have index number 3).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR not successfully executed
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	three-winding transformer not found
	IERR = 5	fixed bus shunt not found
	IERR = 6	no faults in the fault memory tables
	IERR = 7	invalid bus number
	IERR = 8	bus not found
	IERR = 9	invalid node number
	IERR = 10	bus does not belong to a substation
	IERR = 11	node not found in the substation of this bus
	IERR = 12	node is not connected to any bus section of this bus
	IERR = 13	invalid FAULT value
	IERR = 14	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.13. DIST_MACHINE_TRIP

Use this API to set a machine to out-of-service during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_MACHINE_TRIP  IBUS  'ID'
```

Python command syntax:

```
ierr = dist_machine_trip(ibus,id)
```

Fortran command syntax:

```
CALL DIST_MACHINE_TRIPAPI(IBUS, ID, IERR)
```

Where:

Integer IBUS	bus number of machine to be tripped (input; no default allowed).	
Character*2 ID	machine identifier (input; '1' by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR not successfully executed
	IERR = 2	bus not found
	IERR = 3	machine not found
	IERR = 4	machine is already out-of-service
	IERR = 5	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.14. DIST_SCMU_FAULT

This API routine is obsolete. It has been replaced by the API routine dist_scmu_fault_2 .

Use this API to calculate an unbalanced fault and apply the equivalent positive sequence fault admittance at a bus during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_SCMU_FAULT  OPTIONS(1)..OPTIONS(4)  VALUES(1)..VALUES(4)
```

Python command syntax:

```
ierr = dist_scmu_fault(options, values)
```

Fortran command syntax:

```
CALL DIST_SCMU_FAULTAPI(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (4)	array of 4 elements specifying options and values. The first two elements of OPTIONS define options to be used in the construction of the sequence admittance matrices; the remaining entries define the fault to be applied(input).
OPTIONS(1)	dc line and FACTS device option (0 by default)
OPTIONS(1) = 0	block and ignore
OPTIONS(1) = 1	convert to constant admittance load
OPTIONS(2)	transformer impedance correction option (1 by default)
OPTIONS(2) = 0	do not apply to zero sequence
OPTIONS(2) = 1	apply to zero sequence
OPTIONS(3)	unbalanced type (1 by default)
OPTIONS(3) = 1	line-to-ground fault
OPTIONS(3) = 2	line-to-line or line-to-line-to-ground fault
OPTIONS(4)	bus to be faulted (no default allowed)
Real VALUES (4)	array of 4 elements specifying values used to define the fault. They are dependent on the value specified for OPTIONS(3)(input).
VALUES(1)	resistance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)
VALUES(2)	reactance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)
VALUES(3)	resistance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(4)	reactance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR not successfully executed
	IERR = 2	fixed bus shunt table is full
	IERR = 3	error encountered in executing SCMUAPI
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.15. DIST_SCMU_FAULT_2

This API routine is obsolete. It has been replaced by the API routine dist_scmu_fault_3 .

Use this API to calculate an unbalanced fault and apply the equivalent positive sequence fault admittance at a bus during dynamic simulations.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_DIST_SCMU_FAULT_2  OPTIONS(1)..OPTIONS(5) VALUES(1)..VALUES(4)
```

Python command syntax:

```
ierr = dist_scmu_fault_2(options, values)
```

Fortran command syntax:

```
CALL DIST_SCMU_FAULT_2(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (5)	array of 5 elements specifying options and values. The first two elements of OPTIONS define options to be used in the construction of the sequence admittance matrices; the remaining entries define the fault to be applied(input).	
	OPTIONS(1)	dc line and FACTS device option (0 by default)
	OPTIONS(1) = 0	block and ignore
	OPTIONS(1) = 1	convert to constant admittance load
	OPTIONS(2)	transformer impedance correction option (1 by default)
	OPTIONS(2) = 0	do not apply to zero sequence
	OPTIONS(2) = 1	apply to zero sequence
	OPTIONS(3)	unbalanced type (1 by default)
	OPTIONS(3) = 1	line-to-ground fault
	OPTIONS(3) = 2	line-to-line or line-to-line-to-ground fault
	OPTIONS(4)	bus to be faulted (no default allowed)
	OPTIONS(5)	exceptional phase (1 by default)
	OPTIONS(5) = 1	phase A
	OPTIONS(5) = 2	phase B
	OPTIONS(5) = 3	phase C
Real VALUES (4)	array of 4 elements specifying values used to define the fault. They are dependent on the value specified for OPTIONS(3)(input).	
	VALUES(1)	resistance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(2)	reactance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)
	VALUES(3)	resistance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)
	VALUES(4)	reactance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	as in DIST_SCMU_FAULT_3

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.16. DIST_SCMU_FAULT_3

Use this API to calculate an unbalanced fault and apply the equivalent positive sequence fault admittance at a bus during dynamic simulations.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_DIST_SCMU_FAULT_3  OPTIONS(1)..OPTIONS(6) VALUES(1)..VALUES(4)
```

Python command syntax:

```
ierr = dist_scmu_fault_3(options, values)
```

Fortran command syntax:

```
CALL DIST_SCMU_FAULT_3(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (6)	array of 6 elements specifying options and values. The first two elements of OPTIONS define options to be used in the construction of the sequence admittance matrices; the remaining entries define the fault to be applied(input).	
	OPTIONS(1)	dc line and FACTS device option (0 by default)
	OPTIONS(1) = 0	block and ignore
	OPTIONS(1) = 1	convert to constant admittance load
	OPTIONS(2)	transformer impedance correction option (1 by default)
	OPTIONS(2) = 0	do not apply to zero sequence
	OPTIONS(2) = 1	apply to zero sequence
	OPTIONS(3)	unbalanced type (1 by default)
	OPTIONS(3) = 1	line-to-ground fault
	OPTIONS(3) = 2	line-to-line or line-to-line-to-ground fault
	OPTIONS(4)	bus to be faulted (no default allowed)
	OPTIONS(5)	node number of a substation node connected to the specified bus; this defines the bus section to be faulted. If 0 is specified for a bus in a substation, the 'parent' bus is faulted; 0 must be specified for a bus that is not in a substation (0 by default)
	OPTIONS(6)	exceptional phase
	OPTIONS(6) = 1	phase A
	OPTIONS(6) = 2	phase B
	OPTIONS(6) = 3	phase C
Real VALUES (4)	array of 4 elements specifying values used to define the fault. They are dependent on the value specified for OPTIONS(3)(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(1)	resistance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)
	VALUES(2)	reactance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)
	VALUES(3)	resistance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)
	VALUES(4)	reactance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR not successfully executed
	IERR = 2	fixed bus shunt table is full
	IERR = 3	error encountered in executing SCMU_2
	IERR = 4	prerequisite requirements for API are not met
	IERR = 7	invalid bus number
	IERR = 8	bus not found
	IERR = 9	invalid node number
	IERR = 10	bus does not belong to a substation
	IERR = 11	node not found in the substation of this bus
	IERR = 12	node is not connected to any bus section of this bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.17. DIST_SPCB_FAULT

This API routine is obsolete. It has been replaced by the API routine dist_spcb_fault_2 .

Use this API to calculate a branch unbalance and apply the equivalent positive sequence pi model at a branch during dynamic simulations.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_DIST_SPCB_FAULT  IBUS  JBUS  'ID'  OPTIONS(1)..OPTIONS(6)
 VALUES(1)..VALUES(5)
```

Python command syntax:

```
ierr = dist_spcb_fault(ibus, jbus, id, options, values)
```

Fortran command syntax:

```
CALL DIST_SPCB_FAULTAPI(IBUS, JBUS, ID, OPTIONS, VALUES, IERR)
```

Where:

Integer IBUS	from bus of branch to be subjected to unbalance (input; no default allowed).	
Integer JBUS	to bus of branch to be subjected to unbalance (input; no default allowed).	
Character*2 ID	circuit identifier of branch to be subjected to unbalance (input; '1' by default).	
Integer OPTIONS (6)	array of 6 elements specifying unbalance options(input).	
	OPTIONS(1)	unbalance code (1 by default)
	OPTIONS(1) = 1	one phase open
	OPTIONS(1) = 2	two phases open
	OPTIONS(1) = 3	in-line fault
	OPTIONS(1) = 4	one breaker open
	OPTIONS(2)	path to ground code (used when OPTIONS(1) is 1 or 4) (0 by default)
	OPTIONS(2) = 0	no path to ground
	OPTIONS(2) = 1	include a path to ground
	OPTIONS(3)	type of in-line fault code (used when OPTIONS(1) is 3) (1 by default)
	OPTIONS(3) = 1	line-to-ground fault
	OPTIONS(3) = 2	line-to-line-to-ground fault
	OPTIONS(3) = 3	three phase fault
	OPTIONS(4)	open end code (used when OPTIONS(1) is 4) (1 by default)
	OPTIONS(4) = 0	breaker at bus JBUS is open

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(4) = 1	breaker at bus IBUS is open
	OPTIONS(5)	dc line and FACTS device option (0 by default)
	OPTIONS(5) = 0	block and ignore
	OPTIONS(5) = 1	convert to constant admittance load
	OPTIONS(6)	transformer impedance correction option (1 by default)
	OPTIONS(6) = 0	do not apply to zero sequence
	OPTIONS(6) = 1	apply to zero sequence
Real VALUES (5)	array of 5 elements specifying unbalance parameters(input).	
	VALUES(1)	fault location as fraction of line from bus IBUS; 0.0 < VALUES(1) < 1.0 (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3) (0.5 by default)
	VALUES(2)	Rl-g (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3 and OPTIONS(3) is 1 or 2) (0.0 by default)
	VALUES(3)	Xl-g (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3 and OPTIONS(3) is 1 or 2) (0.0 by default)
	VALUES(4)	Rl-l used when OPTIONS(1) is 3 and OPTIONS(3) is 2) (0.0 by default)
	VALUES(5)	Xl-l (used when OPTIONS(1) is 3 and OPTIONS(3) is 2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	branch is out-of-service
	IERR = 5	error encountered in executing SPCBAPI
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.14.18. DIST_SPCB_FAULT_2

Use this API routine to run the second release of the dynamics branch unbalance function. Use this API routine to calculate a branch unbalance and apply the equivalent positive sequence pi model at a branch during dynamic simulations.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_DIST_SPCB_FAULT_2  IBUS  JBUS  'ID'  OPTIONS(1)..OPTIONS(7)
 VALUES(1)..VALUES(5)
```

Python command syntax:

```
ierr = dist_spcb_fault_2(ibus, jbus, id, options, values)
```

Fortran command syntax:

```
CALL DIST_SPCB_FAULT_2(IBUS, JBUS, ID, OPTIONS, VALUES, IERR)
```

Where:

Integer IBUS	from bus of branch to be subjected to unbalance (input; no default allowed).	
Integer JBUS	to bus of branch to be subjected to unbalance (input; no default allowed).	
Character*2 ID	circuit identifier of branch to be subjected to unbalance (input; '1' by default).	
Integer OPTIONS (7)	array of 7 elements specifying unbalance options(input).	
	OPTIONS(1)	unbalance code (1 by default)
	OPTIONS(1) = 1	one phase open
	OPTIONS(1) = 2	two phases open
	OPTIONS(1) = 3	in-line fault
	OPTIONS(1) = 4	one breaker open
	OPTIONS(2)	path to ground code (used when OPTIONS(1) is 1 or 4) (0 by default)
	OPTIONS(2) = 0	no path to ground
	OPTIONS(2) = 1	include a path to ground
	OPTIONS(3)	type of in-line fault code (used when OPTIONS(1) is 3) (1 by default)
	OPTIONS(3) = 1	line-to-ground fault
	OPTIONS(3) = 2	line-to-line-to-ground fault
	OPTIONS(3) = 3	three phase fault
	OPTIONS(4)	open end code (used when OPTIONS(1) is 4) (1 by default)
	OPTIONS(4) = 0	breaker at bus JBUS is open
	OPTIONS(4) = 1	breaker at bus IBUS is open

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5)	dc line and FACTS device option (0 by default)
	OPTIONS(5) = 0	block and ignore
	OPTIONS(5) = 1	convert to constant admittance load
	OPTIONS(6)	transformer impedance correction option (1 by default)
	OPTIONS(6) = 0	do not apply to zero sequence
	OPTIONS(6) = 1	apply to zero sequence
	OPTIONS(7)	fault analysis generator reactance option (0 by default)
	OPTIONS(7) = 0	use subtransient reactance
	OPTIONS(7) = 1	use transient reactance
	OPTIONS(7) = 2	use synchronous reactance
Real VALUES (5)	array of 5 elements specifying unbalance parameters(input).	
	VALUES(1)	fault location as fraction of line from bus IBUS; 0.0 < VALUES(1) < 1.0 (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3) (0.5 by default)
	VALUES(2)	Rl-g (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3 and OPTIONS(3) is 1 or 2) (0.0 by default)
	VALUES(3)	Xl-g (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3 and OPTIONS(3) is 1 or 2) (0.0 by default)
	VALUES(4)	Rl-l used when OPTIONS(1) is 3 and OPTIONS(3) is 2) (0.0 by default)
	VALUES(5)	Xl-l (used when OPTIONS(1) is 3 and OPTIONS(3) is 2) (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	STRT or MSTR has not been successfully executed
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	branch is out-of-service
	IERR = 5	error encountered in executing SPCBAPI
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.15. Set Disturbance Definition

Set Disturbance Definition, contains API routines for Critical Clearing Time Calculation.

8.15.1. DIST_DEF_3PHASE_BUS_FAULT

Use this API to define a three phase fault at a bus in dynamic simulations.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_DIST_DEF_3PHASE_BUS_FAULT IBUS NODE UNITS BASEKV VALUES(1) VALUES(2)
 DISTTIME
```

Python command syntax:

```
ierr = dist_def_3phase_bus_fault(ibus, node, units, basekv, values,
 disttime)
```

Fortran command syntax:

```
CALL DIST_DEF_3PHASE_BUS_FAULT(IBUS, NODE, UNITS, BASEKV, VALUES,
 DISTTIME, IERR)
```

Where:

Integer IBUS	bus number of the bus at which the fault is to be placed (input; no default allowed).
Integer NODE	node number of a substation node connected to the specified bus; this defines the bus section to be faulted. If 0 is specified for a bus in a substation, the 'parent' bus is faulted; 0 must be specified for a bus that is not in a substation (input; 0 by default).
Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default). UNITS = 1 admittance in MVA UNITS = 2 admittance in mhos UNITS = 3 impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).
Real VALUES (2)	array of 2 elements specifying (input). VALUES(1) real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default) VALUES(2) reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
Real DISTTIME	Time at which the fault is applied in seconds (input; 1.0).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid DISTTIME value
	IERR = 2	fixed bus shunt table is full
	IERR = 3	bus is out-of-service
	IERR = 4	invalid UNITS value
	IERR = 5	invalid BASEKV value (<0.0)
	IERR = 6	both BASEKV and the base voltage of bus IBUS are 0.0
	IERR = 7	invalid bus number
	IERR = 8	bus not found
	IERR = 9	invalid node number
	IERR = 10	bus does not belong to a substation
	IERR = 11	node not found in the substation of this bus
	IERR = 12	node is not connected to any bus section of this bus
	IERR = 13	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.15.2. DIST_DEF_3WIND_FAULT

Use this API to define a fault at the IBUS end of a three-winding transformer in dynamic simulations.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_DIST_DEF_3WIND_FAULT IBUS JBUS KBUS 'ID' UNITS BASEKV VALUES(1)
 VALUES(2) DISTTIME
```

Python command syntax:

```
ierr = dist_def_3wind_fault(ibus,jbus,kbus,id,units,basekv,values,
 disttime)
```

Fortran command syntax:

```
CALL
 DIST_DEF_3WIND_FAULTAPI(IBUS,JBUS,KBUS,ID,UNITS,BASEKV,VALUES,DISTTIME,IERR)
```

Where:

Integer IBUS	bus number of the bus at which the fault is to be placed (input; no default allowed).	
Integer JBUS	bus number of one of the other buses (input; no default allowed).	
Integer KBUS	bus number of the third bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default).	
	UNITS = 1	admittance in MVA
	UNITS = 2	admittance in mhos
	UNITS = 3	impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).	
Real VALUES (2)	array of 2 elements specifying (input).	
	VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
	VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
Real DISTTIME	Time at which the fault is applied in seconds (input; 1.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid DISTTIME value
	IERR = 2	bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	three-winding transformer not found
IERR = 4	three-winding transformer winding is out-of-service
IERR = 5	invalid UNITS value
IERR = 6	invalid BASEKV value (<0.0)
IERR = 7	both BASEKV and the base voltage of bus IBUS are 0.0
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.15.3. DIST_DEF_BRANCH_FAULT

Use this API to define a fault at the IBUS end of a non-transformer branch or a two-winding transformer in dynamic simulations.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_DIST_DEF_BRANCH_FAULT IBUS JBUS 'ID' UNITS BASEKV VALUES(1) VALUES(2)
 DISTTIME
```

Python command syntax:

```
ierr = dist_def_branch_fault(ibus, jbus, id, units, basekv, values,
 disttime)
```

Fortran command syntax:

```
CALL
 DIST_DEF_BRANCH_FAULTAPI(IBUS,JBUS,ID,UNITS,BASEKV,VALUES,DISTTIME,IERR)
```

Where:

Integer IBUS	bus number of the bus at which the fault is to be placed (input; no default allowed).	
Integer JBUS	bus number of the other bus (input; no default allowed).	
Character*2 ID	circuit identifier (input; '1' by default).	
Integer UNITS	units in which fault admittance or impedance is specified in VALUES (input; 1 by default).	
	UNITS = 1	admittance in MVA
	UNITS = 2	admittance in mhos
	UNITS = 3	impedance in ohms
Real BASEKV	base voltage in kV used to calculate the per unit fault admittance if UNITS is 2 or 3; ignored if UNITS is 1. If BASEKV is specified as 0.0, the base voltage of bus IBUS is used (input; 0.0).	
Real VALUES (2)	array of 2 elements specifying (input).	
	VALUES(1)	real component of the complex fault admittance or impedance according to the value specified for UNITS (0.0 by default)
	VALUES(2)	reactive component of the complex fault admittance or impedance according to the value specified for UNITS (-2.0E11 by default)
Real DISTTIME	Time at which the fault is applied in seconds (input; 1.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid DISTTIME value
	IERR = 2	bus not found
	IERR = 3	branch not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	branch is out-of-service
IERR = 5	invalid UNITS value
IERR = 6	invalid BASEKV value (<0.0)
IERR = 7	both BASEKV and the base voltage of bus IBUS are 0.0
IERR = 8	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.15.4. DIST_DEF_SCMU_FAULT_3

Use this API to define an unbalanced fault at a bus in dynamic simulations.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_DIST_DEF_SCMU_FAULT_3 OPTIONS(1)..OPTIONS(6) VALUES(1)..VALUES(4)
 DISTTIME
```

Python command syntax:

```
ierr = dist_def_scmu_fault_3(options, values, disttime)
```

Fortran command syntax:

```
CALL DIST_DEF_SCMU_FAULT_3(OPTIONS, VALUES, DISTTIME, IERR)
```

Where:

Integer OPTIONS (6)	array of 6 elements specifying options and values. The first two elements of OPTIONS define options to be used in the construction of the sequence admittance matrices; the remaining entries define the fault to be applied(input).
OPTIONS(1)	dc line and FACTS device option (0 by default)
OPTIONS(1) = 0	block and ignore
OPTIONS(1) = 1	convert to constant admittance load
OPTIONS(2)	transformer impedance correction option (1 by default)
OPTIONS(2) = 0	do not apply to zero sequence
OPTIONS(2) = 1	apply to zero sequence
OPTIONS(3)	unbalanced type (1 by default)
OPTIONS(3) = 1	line-to-ground fault
OPTIONS(3) = 2	line-to-line or line-to-line-to-ground fault
OPTIONS(4)	bus to be faulted (no default allowed)
OPTIONS(5)	node number of a substation node connected to the specified bus; this defines the bus section to be faulted. If 0 is specified for a bus in a substation, the 'parent' bus is faulted; 0 must be specified for a bus that is not in a substation (0 by default)
OPTIONS(6)	exceptional phase
OPTIONS(6) = 1	phase A
OPTIONS(6) = 2	phase B
OPTIONS(6) = 3	phase C
Real VALUES (4)	array of 4 elements specifying values used to define the fault. They are dependent on the value specified for OPTIONS(3)(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(1)	resistance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)
	VALUES(2)	reactance of the complex L-G (if OPTIONS(3) is 1) or L-L (if OPTIONS(3) is 2) fault impedance in pu (0.0 by default)
	VALUES(3)	resistance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)
	VALUES(4)	reactance of the complex L-G fault impedance in pu (if OPTIONS(3) is 2) (0.0 by default)
Real DISTTIME	Time at which the fault is applied in seconds (input; 1.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid DISTTIME value
	IERR = 4	prerequisite requirements for API are not met
	IERR = 7	invalid bus number
	IERR = 8	bus not found
	IERR = 9	invalid node number
	IERR = 10	bus does not belong to a substation
	IERR = 11	node not found in the substation of this bus
	IERR = 12	node is not connected to any bus section of this bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

8.15.5. DIST_DEF_SPCB_FAULT_2

Use this API routine to define a branch unbalance in dynamic simulations.

This API routine was first introduced in release 35.5.0.

Batch command syntax:

```
BAT_DIST_DEF_SPCB_FAULT_2 IBUS JBUS 'ID' OPTIONS(1)..OPTIONS(7)
 VALUES(1)..VALUES(5) DISTTIME
```

Python command syntax:

```
ierr = dist_def_spcb_fault_2(ibus, jbus, id, options, values, disttime)
```

Fortran command syntax:

```
CALL DIST_DEF_SPCB_FAULT_2(IBUS, JBUS, ID, OPTIONS, VALUES, DISTTIME,
 IERR)
```

Where:

Integer IBUS	from bus of branch to be subjected to unbalance (input; no default allowed).	
Integer JBUS	to bus of branch to be subjected to unbalance (input; no default allowed).	
Character*2 ID	circuit identifier of branch to be subjected to unbalance (input; '1' by default).	
Integer OPTIONS (7)	array of 7 elements specifying unbalance options(input).	
	OPTIONS(1)	unbalance code (1 by default)
	OPTIONS(1) = 1	one phase open
	OPTIONS(1) = 2	two phases open
	OPTIONS(1) = 3	in-line fault
	OPTIONS(1) = 4	one breaker open
	OPTIONS(2)	path to ground code (used when OPTIONS(1) is 1 or 4) (0 by default)
	OPTIONS(2) = 0	no path to ground
	OPTIONS(2) = 1	include a path to ground
	OPTIONS(3)	type of in-line fault code (used when OPTIONS(1) is 3) (1 by default)
	OPTIONS(3) = 1	line-to-ground fault
	OPTIONS(3) = 2	line-to-line-to-ground fault
	OPTIONS(3) = 3	three phase fault
	OPTIONS(4)	open end code (used when OPTIONS(1) is 4) (1 by default)
	OPTIONS(4) = 0	breaker at bus JBUS is open
	OPTIONS(4) = 1	breaker at bus IBUS is open
	OPTIONS(5)	dc line and FACTS device option (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 0	block and ignore
	OPTIONS(5) = 1	convert to constant admittance load
	OPTIONS(6)	transformer impedance correction option (1 by default)
	OPTIONS(6) = 0	do not apply to zero sequence
	OPTIONS(6) = 1	apply to zero sequence
	OPTIONS(7)	fault analysis generator reactance option (0 by default)
	OPTIONS(7) = 0	use subtransient reactance
	OPTIONS(7) = 1	use transient reactance
	OPTIONS(7) = 2	use synchronous reactance
Real VALUES (5)	array of 5 elements specifying unbalance parameters(input).	
	VALUES(1)	fault location as fraction of line from bus IBUS; 0.0 < VALUES(1) < 1.0 (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3) (0.5 by default)
	VALUES(2)	Rl-g (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3 and OPTIONS(3) is 1 or 2) (0.0 by default)
	VALUES(3)	Xl-g (used when OPTIONS(1) is 1 or 4 and OPTIONS(2) is 1; or when OPTIONS(1) is 3 and OPTIONS(3) is 1 or 2) (0.0 by default)
	VALUES(4)	Rl-l used when OPTIONS(1) is 3 and OPTIONS(3) is 2) (0.0 by default)
	VALUES(5)	Xl-l (used when OPTIONS(1) is 3 and OPTIONS(3) is 2) (0.0 by default)
Real DISTTIME	Time at which the fault is applied in seconds (input; 1.0).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid DISTTIME value
	IERR = 2	bus not found
	IERR = 3	branch not found
	IERR = 4	branch is out-of-service
	IERR = 5	error encountered in executing SPCBAPI
	IERR = 6	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 9
Subsystem Definition

This chapter defines API routines that control subsystems. There are four kinds of subsystems in PSS®E: area subsystems, bus subsystems, owner subsystems, and zone subsystems. Area subsystems are a set of areas, bus subsystems are a set of buses, etc.

Many PSS®E API routines accept a subsystem definition to limit the scope of operation of the routine. Most such use bus subsystems. Bus subsystems can be defined in terms of areas, zones, and/or owners; but a bus subsystem defined by areas, for example, is not an area subsystem, which is a set of areas, not a set of buses in an area.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.1. Area Subsystems

9.1.1. ASYS

Use this API to define an area subsystem.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_ASYS SID NUM AREAS(1) .. AREAS(NUM)
```

Python command syntax:

```
ierr = asys(sid, num, areas)
```

Fortran command syntax:

```
CALL SBSASET(SID, NUM, AREAS, IERR)
```

Where:

Integer SID	valid area subsystem ID. Valid area subsystem IDs range from 0 to 11 (input).	
Integer NUM	number of areas to set (input).	
Integer AREAS (NUM)	array that contains the areas to set (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid
	IERR = 2	SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.1.2. ASYSDEF

Use this API to set the definition of an area subsystem. This can be used to toggle a particular area subsystem on or off.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_ASYSDEF SID DEFINED
```

Python command syntax:

```
ierr = asysdef(sid, defined)
```

Fortran command syntax:

```
CALL SBSASETDEF(SID, DEFINED, IERR)
```

Where:

Integer SID	valid area subsystem ID. Valid area subsystem IDs range from 0 to 11 (input).	
Integer DEFINED	flag used to mark the area subsystem as undefined/defined (input).	
	DEFINED = 0	mark the area subsystem as undefined
	DEFINED = 1	mark the area subsystem as defined
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.1.3. ASYSINIT

Use this API to initialize or re-initialize an area subsystem.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_ASYSINIT SID
```

Python command syntax:

```
ierr = asysinit(sid)
```

Fortran command syntax:

```
CALL SBSACREATE(SID, IERR)
```

Where:

Integer SID	valid area subsystem ID. Valid area subsystem IDs range from 0 to 11 (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2. Bus Subsystems

9.2.1. BSYS

Use this API to define a bus subsystem. The criteria used to define the bus subsystem are evaluated using an AND criteria. That is to say, if a bus kV range, Areas and Zones are specified, to be in the bus subsystem a bus must be in the specified kV range AND must be in one of the Areas defined AND must be in one of the Zones defined. Failure to meet ANY of the specified criteria will remove the bus from the bus subsystem.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_BSYS SID USEKV BASEKV(1) BASEKV(2) NUMAREA AREAS(1)..AREAS(NUMAREA)
 NUMBUS BUSES(1)..BUSES(NUMBUS) NUMOWNER OWNERS(1)..OWNERS(NUMOWNER)
 NUMZONE ZONES(1)..ZONES(NUMZONE)
```

Python command syntax:

```
ierr = bsys(sid, usekv, basekv, numarea, areas, numbus, buses, numowner,
 owners, numzone, zones)
```

Fortran command syntax:

```
CALL SBSBSET_A (SID, USEKV, BASEKV, NUMAREA, AREAS, NUMBUS, BUSES_C,
 NUMOWNER, OWNERS, NUMZONE, ZONES, IERR)

CALL SBSBSET_N (SID, USEKV, BASEKV, NUMAREA, AREAS, NUMBUS, BUSES_I,
 NUMOWNER, OWNERS, NUMZONE, ZONES, IERR)
```

Where:

Integer SID	valid bus subsystem ID. Valid bus subsystem IDs range from 0 to 11 (input).
Integer USEKV	flag to determine whether to use BASEKV values in setting the subsystem (input). USEKV = 0 don't use BASEKV values USEKV = 1 use BASEKV values
Real BASEKV (2)	array of 2 elements specifying (input). BASEKV(1) minimum basekV limit BASEKV(2) maximum basekV limit
Integer NUMAREA	number of areas to set (input).
Integer AREAS (NUMAREA)	array that contains the areas to set (input).
Integer NUMBUS	number of buses to set (input).
Integer or Character*18 BUSES (NUMBUS)	A union type array describing the network element's BUS value. BUSES_I refers to Integer, BUSES_C refers to Character value in the Fortran syntax. (input).
Integer NUMOWNER	number of owners to set (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer OWNERS (NUMOWNER)	array that contains the owners to set (input).
Integer NUMZONE	number of zones to set (input).
Integer ZONES (NUMZONE)	array that contains the zones to set (input).
Integer IERR	error code (output).
	IERR = 0 no error occurred
	IERR = 1 SID is not valid
	IERR = 2 SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2.2. BSYSADD

Use this API to add elements to an existing bus subsystem.
The criteria used to define a bus subsystem are evaluated using an AND criteria. That is to say, if a bus kV range, Areas and Zones are specified, to be in the bus subsystem a bus must be in the specified kV range AND must be in one of the Areas defined AND must be in one of the Zones defined. Failure to meet ANY of the specified criteria will remove the bus from the bus subsystem.
Adding to an existing bus subsystem with this API adds addtional AND critieria, not OR criteria, to be used in the evaluation.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_BSYSADD SID USEKV BASEKV(1) BASEKV(2) NUMAREA AREAS(1)..AREAS(NUMAREA)
 NUMBUS BUSES(1)..BUSES(NUMBUS) NUMOWNER OWNERS(1)..OWNERS(NUMOWNER)
 NUMZONE ZONES(1)..ZONES(NUMZONE)
```

Python command syntax:

```
ierr = bsysadd(sid, usekv, basekv, numarea, areas, numbus, buses,
 numowner, owners, numzone, zones)
```

Fortran command syntax:

```
CALL SBSBADD(SID, USEKV, BASEKV, NUMAREA, AREAS, NUMBUS, BUSES, NUMOWNER,
 OWNERS, NUMZONE, ZONES, IERR)
```

Where:

Integer SID	valid bus subsystem ID. Valid bus subsystem IDs range from 0 to 11 (input).
Integer USEKV	flag to determine whether to use BASEKV values in setting the sub-system (input). USEKV = 0 don't use BASEKV values USEKV = 1 use BASEKV values
Real BASEKV (2)	array of 2 elements specifying (input). BASEKV(1) minimum basekV limit BASEKV(2) maximum basekV limit
Integer NUMAREA	number of areas to set (input).
Integer AREAS (NUMAREA)	array that contains the areas to set (input).
Integer NUMBUS	number of buses to set (input).
Integer BUSES (NUMBUS)	array that contains the buses to set (input).
Integer NUMOWNER	number of owners to set (input).
Integer OWNERS (NUMOWNER)	array that contains the owners to set (input).
Integer NUMZONE	number of zones to set (input).
Integer ZONES (NUMZONE)	array that contains the zones to set (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error occurred
IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2.3. BSYSDEF

Use this API to set the definition of a bus subsystem. This can be used to toggle a particular bus subsystem on or off.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_BSYSDEF SID DEFINED
```

Python command syntax:

```
ierr = bsysdef(sid, defined)
```

Fortran command syntax:

```
CALL SBSBSETDEF(SID, DEFINED, IERR)
```

Where:

Integer SID	valid bus subsystem ID. Valid bus subsystem IDs range from 0 to 11 (input).	
Integer DEFINED	flag used to mark the bus subsystem as undefined/defined (input).	
	DEFINED = 0	mark the bus subsystem as undefined
	DEFINED = 1	mark the bus subsystem as defined
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2.4. BSYSDELO

Use this API to delete a single bus from a bus subsystem.

This API routine was first introduced in release 33.4.0.

Batch command syntax:

```
BAT_BSYSDELO SID BUSNUM
```

Python command syntax:

```
ierr = bsysdelo(sid, busnum)
```

Fortran command syntax:

```
CALL SBSBDELO(SID, BUSNUM, IERR)
```

Where:

Integer SID	valid bus subsystem ID. Valid bus subsystem IDs range from 0 to 11 (input).	
Integer BUSNUM	bus number to delete from the subsystem (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid
	IERR = 2	bus subsystem is not defined
	IERR = 3	no buses defined in subsystem
	IERR = 4	BUSNUM doesn't exist in subsystem

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2.5. BSYSINIT

Use this API to initialize or re-initialize a bus subsystem.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_BSYSINIT SID
```

Python command syntax:

```
ierr = bsysinit(sid)
```

Fortran command syntax:

```
CALL SBSBCREATE(SID, IERR)
```

Where:

Integer SID	valid bus subsystem ID. Valid bus subsystem IDs range from 0 to 11 (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2.6. BSYSMEM

Use this API to replicate the specified bus subsystem in a bus subsystem file. This bus subsystem file may then be recalled at a later time to restore the specified bus subsystem.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BSYSMEM  SID  SFILE
```

Python command syntax:

```
ierr = bsysmem(sid, sfile)
```

Fortran command syntax:

```
CALL SBSBSMEM(SID, SFILE, IERR)
```

Where:

Integer SID	a valid bus subsystem id. Valid bus subsystem ids range from 0 to 11 (input).	
Character*260 SFILE	bus subsystem filename (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid
	IERR = 2	error opening SFILE

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2.7. BSYSO

Use this API to build a bus subsystem one bus at a time.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_BSYSO SID BUSNUM
```

Python command syntax:

```
ierr = bsyso(sid, busnum)
```

Fortran command syntax:

```
CALL SBSBSETO(SID, BUSNUM, IERR)
```

Where:

Integer SID	valid bus subsystem ID. Valid bus subsystem IDs range from 0 to 11 (input).	
Integer BUSNUM	bus number to add to subsystem (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid
	IERR = 2	Bus subsystem contains other elements, not just buses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.2.8. BSYSRCL

Use this API to recall a specified bus subsystem saved in a bus subsystem file.

This API routine was first introduced in release 28.0.0.

Batch command syntax:

```
BAT_BSYSRCL SID SFILE
```

Python command syntax:

```
ierr = bsysrcl(sid, sfile)
```

Fortran command syntax:

```
CALL SBSBSRCL(SID, SFILE, IERR)
```

Where:

Integer SID	a valid bus subsystem id. Valid bus subsystem ids range from 0 to 11 (input).	
Character*260 SFILE	bus subsystem filename (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid
	IERR = 2	error opening SFILE
	IERR = 3	not a valid bus subsystem file
	IERR = 4	items in bus subsystem file not found in working case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.3. Owner Subsystems

9.3.1. OSYS

Use this API to define an owner subsystem.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OSYS SID NUM OWNERS(1) .. OWNERS(NUM)
```

Python command syntax:

```
ierr = osys(sid, num, owners)
```

Fortran command syntax:

```
CALL SBSOSET(SID, NUM, OWNERS, IERR)
```

Where:

Integer SID	valid owner subsystem ID. Valid owner subsystem IDs range from 0 to 11 (input).	
Integer NUM	number of owners to set (input).	
Integer OWNERS (NUM)	array that contains the owners to set (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid
	IERR = 2	SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.3.2. OSYSDEF

Use this API to set the definition of an owner subsystem. This can be used to toggle a particular owner subsystem on or off.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_OSYSDEF  SID  DEFINED
```

Python command syntax:

```
ierr = osysdef(sid, defined)
```

Fortran command syntax:

```
CALL SBSOSETDEF(SID, DEFINED, IERR)
```

Where:

Integer SID	a valid owner subsystem ID. Valid owner subsystem IDs range from 0 to 11 (input).	
Integer DEFINED	flag to mark the owner subsystem as undefined/defined (input).	
	DEFINED = 0	mark the owner subsystem as undefined
	DEFINED = 1	mark the owner subsystem as defined
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.3.3. OSYSINIT

Use this API to initialize or re-initialize an owner subsystem.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_OSYSINIT  SID
```

Python command syntax:

```
ierr = osysinit(sid)
```

Fortran command syntax:

```
CALL SBSOCREATE(SID, IERR)
```

Where:

Integer SID	a valid owner subsystem ID. Valid owner subsystem IDs range from 0 to 11 (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.4. Zone Subsystems

9.4.1. ZSYS

Use this API to define a zone subsystem.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_ZSYS SID NUM ZONES(1) .. ZONES(NUM)
```

Python command syntax:

```
ierr = zsys(sid, num, zones)
```

Fortran command syntax:

```
CALL SBSZSET(SID, NUM, ZONES, IERR)
```

Where:

Integer SID	valid zone subsystem ID. Valid zone subsystem IDs range from 0 to 11 (input).	
Integer NUM	number of zones to set (input).	
Integer ZONES (NUM)	array that contains the zones to set (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid
	IERR = 2	SID is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.4.2. ZSYSDEF

Use this API to set the definition of an zone subsystem. This can be used to toggle a particular zone subsystem on or off.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_ZSYSDEF  SID  DEFINED
```

Python command syntax:

```
ierr = zsysdef(sid, defined)
```

Fortran command syntax:

```
CALL SBSZSETDEF(SID, DEFINED, IERR)
```

Where:

Integer SID	a valid zone subsystem ID. Valid zone subsystem IDs range from 0 to 11 (input).	
Integer DEFINED	flag to mark the zone subsystem as undefined/defined (input).	
	DEFINED = 0	mark the zone subsystem as undefined
	DEFINED = 1	mark the zone subsystem as defined
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

9.4.3. ZSYSINIT

Use this API to initialize or re-initialize a zone subsystem.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_ZSYSINIT  SID
```

Python command syntax:

```
ierr = zsysinit(sid)
```

Fortran command syntax:

```
CALL SBSZCREATE(SID, IERR)
```

Where:

Integer SID	a valid zone subsystem ID. Valid zone subsystem IDs range from 0 to 11 (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	SID is not valid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 10
Diagram View

This chapter defines API routines that are used to manipulate Slider Diagrams in the interface. They will elicit no action if a GUI is not present when invoked.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.1. CHANGEDIAGCONTOUR

Use this API to change the contour parameters for the contour in the active Slider diagram in the GUI. Requires a current network case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_CHANGEDIAGCONTOUR ELEMENT QUANTITY METHOD COLOR ENABLE RES INFLUENCE
 BOUND MAX MIN
```

Python command syntax:

```
changediagcontour(element, quantity, method, color, enable, res,
 influence, bound, max, min)
```

Fortran command syntax:

```
CALL CHANGEDIAGCONTOUR(ELEMENT, QUANTITY, METHOD, COLOR, ENABLE, RES,
 INFLUENCE, BOUND, MAX, MIN)
```

Where:

Integer ELEMENT	item to be contoured e.g. Buses, branches, machines (input).	
	ELEMENT = 1	bus
	ELEMENT = 2	branch
	ELEMENT = 3	machine
Integer QUANTITY	element quantity to generate the contour on (input).	
Integer METHOD	algorithm used to generate the contour (input).	
	METHOD = 1	linear method
	METHOD = 2	linear approximation method
	METHOD = 3	quad tree method
Integer COLOR	color gradiant to use for the contour (input).	
	COLOR = 0	red scale
	COLOR = 1	green scale
	COLOR = 2	blue scale
	COLOR = 3	gray scale
	COLOR = 4	blue/low to red/high
	COLOR = 5	red/low to blue/high
Integer ENABLE	"switch" used to toggle the contour on/off (input).	
	ENABLE = 0	off
	ENABLE = 1	on
Real RES	resolution of the contour (10 to 2000). A higher resolution will result in greater computation times. (input).	
Real INFLUENCE	influence range (1 to 100). A larger range will result in greater blending. (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real BOUND	value that controls how far beyond the outermost ELEMENTs on the diagram the contour extends (input).
Real MAX	maximum "quantity" value associated with the "high" color (input).
Real MIN	minimum "quantity" value associated with the "low" color (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.2. CHECKDIAGFILE

Use this API to check the active Slider Diagram in the GUI against the current network case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_CHECKDIAGFILE
```

Python command syntax:

```
checkdiagfile()
```

Fortran command syntax:

```
CALL CHECKDIAGFILE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.3. CLOSEDIAGFILE

Use this API to close the active Slider Diagram in the GUI without prompting to save the Diagram.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_CLOSEDIAGFILE
```

Python command syntax:

```
closediagfile()
```

Fortran command syntax:

```
CALL CLOSEDIAGFILE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.4. DELETEDIAGCONTOUR

Use this API to delete the contour from the active Slider Diagram in the GUI.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_DELETEDIAGCONTOUR
```

Python command syntax:

```
deletediagcontour()
```

Fortran command syntax:

```
CALL DELETEDIAGCONTOUR
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.5. ENABLEDIAGCONTOUR

Use this API to specify the contour parameters and display a contour in the active Slider Diagram in the GUI. Requires a current network case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_ENABLEDIAGCONTOUR ELEMENT QUANTITY METHOD COLOR ENABLE RES INFLUENCE
 BOUND MAX MIN
```

Python command syntax:

```
enablediagcontour(element, quantity, method, color, enable, res,
 influence, bound, max, min)
```

Fortran command syntax:

```
CALL ENABLEDIAGCONTOUR(ELEMENT, QUANTITY, METHOD, COLOR, ENABLE, RES,
 INFLUENCE, BOUND, MAX, MIN)
```

Where:

Integer ELEMENT	item to be contoured e.g. Buses, branches, machines (input).	
	ELEMENT = 1	Bus
	ELEMENT = 2	Branch
	ELEMENT = 3	Machine
Integer QUANTITY	element quantity to generate the contour on (input).	
Integer METHOD	algorithm used to generate the contour (input).	
	METHOD = 1	Linear method
	METHOD = 2	Linear approximation method
	METHOD = 3	Quad tree method
Integer COLOR	color gradiant to use for the contour (input).	
	COLOR = 0	Red scale
	COLOR = 1	Green scale
	COLOR = 2	Blue scale
	COLOR = 3	Gray scale
	COLOR = 4	Blue/low to Red/high
	COLOR = 5	Red/low to Blue/high
Integer ENABLE	"switch" used to toggle the contour on/off (input).	
	ENABLE = 0	Off
	ENABLE = 1	On
Real RES	is the resolution of the contour (10 to 2000). A higher resolution will result in greater computation times. (input).	
Real INFLUENCE	influence range (1 to 100). A larger range will result in greater blending. (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real BOUND	a value that controls how far beyond the outermost ELEMENTs on the diagram the contour extends (input).
Real MAX	maximum "quantity" value associated with the "high" color (input).
Real MIN	minimum "quantity" value associated with the "low" color (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.6. EXPORTBUSLOCFILE

This API routine is obsolete. It has been replaced by the API routine exportbuslocfile_2 .

Use this API to export the bus location from the active Slider Diagram in the GUI to a file.

This API routine was first introduced in release 30.2.0.

Batch command syntax:

```
BAT_EXPORTBUSLOCFILE BUSLOCFILE
```

Python command syntax:

```
exportbuslocfile(buslocfile)
```

Fortran command syntax:

```
CALL EXPORTBUSLOCFILE(BUSLOCFILE)
```

Where:

Character*260 BUSLOCFILE	name of the Bus Location Data File to be written (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.7. EXPORTBUSLOCFILE_2

Use this API to export the bus location from the active Slider Diagram in the GUI to a file.

This API routine was first introduced in release 34.8.0 and 35.1.0.

Batch command syntax:

```
BAT_EXPORTBUSLOCFILE_2 BUSLOCFILE EXPORTGEO
```

Python command syntax:

```
exportbuslocfile_2(buslocfile, exportgeo)
```

Fortran command syntax:

```
CALL EXPORTBUSLOCFILE_2(BUSLOCFILE, EXPORTGEO)
```

Where:

Character*260 BUSLOCFILE	name of the Bus Location Data File to be written (input).	
Integer EXPORTGEO	export geophysical coordinates or Cartesian coordinates (input; 1 by default).	
	ExportGeo = 0	export in Cartesian coordinates
	ExportGeo = 1	export in geophysical coordinates

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.8. EXPORTIMAGEFILE

This API routine is obsolete. It has been replaced by the API routine exportimagefile_2 .

Use this API to export an image of the active Slider Diagram in the GUI to a file.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_EXPORTIMAGEFILE  TYPE IMAGEFILE QUALITY
```

Python command syntax:

```
exportimagefile(type, imagefile, quality)
```

Fortran command syntax:

```
CALL EXPORTIMAGEFILE(TYPE, IMAGEFILE, QUALITY)
```

Where:

Integer TYPE	type of image to export (input). Type = 1 Bitmap Type = 2 JPEG Type = 3 PNG Type = 4 GIF Type = 5 TIF
Character*260 IMAGEFILE	name of the image file to be written (input).
Integer QUALITY	a value, between 1 and 100, used to control the quality of the exported image if TYPE = 2 (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.9. EXPORTIMAGEFILE_2

Use this API to export an image of the active Slider Diagram in the GUI to a file.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_EXPORTIMAGEFILE_2 TYPE IMAGEFILE QUALITY WIDTH HEIGHT SCALE X Y
```

Python command syntax:

```
exportimagefile_2(type, imagefile, quality, width, height, scale, x, y)
```

Fortran command syntax:

```
CALL EXPORTIMAGEFILE_2(TYPE, IMAGEFILE, QUALITY, WIDTH, HEIGHT, SCALE, X,
 Y)
```

Where:

Integer TYPE	type of image to export (input). Type = 1 Bitmap Type = 2 JPEG Type = 3 PNG Type = 4 GIF Type = 5 TIF
Character*260 IMAGEFILE	name of the image file to be written (input).
Integer QUALITY	a value, between 1 and 100, used to control the quality of the exported image if TYPE = 2 (input).
Integer WIDTH	width of the image to capture in pixels (input).
Integer HEIGHT	height of the image to capture in pixels (input).
real SCALE	scale at wich to capture the image. (input).
real X	x location on the diagram that will be the center of the image (input).
real Y	y location on the diagram that will be the center of the image (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.10. EXPORTIMAGEFILE_3

Use this API to export an image of the active Slider Diagram by defining extents of the bounding box of the image capture area in Slider Diagram units.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_EXPORTIMAGEFILE_3  TYPE IMAGEFILE QUALITY TOP RIGHT BOTTOM LEFT SCALE
```

Python command syntax:

```
exportimagefile_3(type, imagefile, quality, top, right, bottom, left,
 scale)
```

Fortran command syntax:

```
CALL EXPORTIMAGEFILE_3(TYPE, IMAGEFILE, QUALITY, TOP, RIGHT, BOTTOM, LEFT,
 SCALE)
```

Where:

Integer TYPE	type of image to export (input). Type = 1 Bitmap Type = 2 JPEG Type = 3 PNG Type = 4 GIF Type = 5 TIF
Character*260 IMAGEFILE	Name of the image file to be written (input).
Integer QUALITY	A value, between 1 and 100, used to control the quality of the exported image if TYPE = 2 (input).
real TOP	The top extent of the bounding box used to define the image capture area, in Slider Diagram units. (input).
real RIGHT	The right extent of the bounding box used to define the image capture area, in Slider Diagram units. (input).
real BOTTOM	The bottom extent of the bounding box used to define the image capture area, in Slider Diagram units. (input).
real LEFT	The left extent of the bounding box used to define the image capture area, in Slider Diagram units. (input).
real SCALE	Scale at which capture the image. (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.11. GEXMBUS

Use this API to create a GEXM/GOUT display of a bus in an active Slider diagram in the GUI. Requires a current network case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_GEXMBUS BUS
```

Python command syntax:

```
gexmbus(bus)
```

Fortran command syntax:

```
CALL GEXMBUS(BUS)
```

Where:

Integer BUS	bus number of the bus for which to create the GEXM display (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.12. GROWBUS

Use this API to GROW a bus in an active Slider diagram in the GUI. Requires a current network case.

This API routine was first introduced in release 30.1.0.

Batch command syntax:

```
BAT_GROWBUS BUSNUMBER XPOSITION YPOSITION
```

Python command syntax:

```
growbus(busnumber, xposition, yposition)
```

Fortran command syntax:

```
CALL GROWBUS(BUSNUMBER, XPOSITION, YPOSITION)
```

Where:

Integer BUSNUMBER	bus number of the bus to GROW (no default) (input).
Real XPOSITION	X coordinate of the bus on the Slider diagram (input).
Real YPOSITION	Y coordinate of the bus on the Slider diagram (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.13. GROWBUSLEVELS

Use this API to GROW a bus and all the connected buses up to the specified number of levels away in an active Slider diagram in the GUI. Requires a current network case.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_GROWBUSLEVELS BUSNUMBER XPOSITION YPOSITION NUMLEVELS
```

Python command syntax:

```
growbuslevels(busnumber, xposition, yposition, numlevels)
```

Fortran command syntax:

```
CALL GROWBUSLEVELS(BUSNUMBER, XPOSITION, YPOSITION, NUMLEVELS)
```

Where:

Integer BUSNUMBER	bus number of the bus to GROW (no default) (input).
Real XPOSITION	X coordinate of the bus on the Slider diagram (input).
Real YPOSITION	Y coordinate of the bus on the Slider diagram (input).
Integer NUMLEVELS	number of levels; include all buses up to this number of connections from BUSNUMBER in the GROW operation (no default) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.14. GROWDIAGRAM

This API routine is obsolete. It has been replaced by the API routine growdiagram_2 .

Use this API to GROW a bus and all the connected buses in an active Slider diagram in the GUI. Requires a current network case.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_GROWDIAGRAM NUMLEVELS ITEMCOUNT ITEMLIST XPOSITION YPOSITION
```

Python command syntax:

```
growdiagram(numlevels, itemcount, itemlist, xposition, yposition)
```

Fortran command syntax:

```
CALL GROWDIAGRAM(NUMLEVELS, ITEMCOUNT, ITEMLIST, XPOSITION, YPOSITION)
```

Where:

Integer NUMLEVELS	number of levels; include all buses up to this number of connections from each item in ITEMLIST in the GROW operation. A value of -1 can be used to grow all buses (subsystem and in-service options still apply), however attempting to lay out several hundred buses or more will take a very large amount of time to complete as PSSE attempts to calculate best positions for each bus. It is still recommended that the user only grow several levels at once (input).
Integer ITEMCOUNT	number of items in ITEMLIST. ITEMCOUNT can be 0 if NUMLEVELS is -1. Under this condition diagram growth will start at an arbitrary bus (input; 0 by default).
Character*40 ITEMLIST (ITEMCOUNT)	list of items to grow. Each item is given by its generated map string (input).
Real XPOSITION	X coordinate of the first bus on the Slider diagram (input).
Real YPOSITION	X coordinate of the first bus on the Slider diagram (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.15. GROWDIAGRAM_2

Use this API to GROW a bus and all the connected buses in an active Slider diagram in the GUI. Requires a current network case.

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_GROWDIAGRAM_2 NUMLEVELS ITEMCOUNT ITEMLIST XPOSITION YPOSITION
 EXCLUDELIST
```

Python command syntax:

```
growdiagram_2(numlevels, itemcount, itemlist, xposition, yposition,
 excludelist)
```

Fortran command syntax:

```
CALL GROWDIAGRAM_2(NUMLEVELS, ITEMCOUNT, ITEMLIST, XPOSITION, YPOSITION,
 EXCLUDELIST)
```

Where:

Integer NUMLEVELS	number of levels; include all buses up to this number of connections from each item in ITEMLIST in the GROW operation. A value of -1 can be used to grow all buses (subsystem and in-service options still apply), however attempting to lay out several hundred buses or more will take a very large amount of time to complete as PSSE attempts to calculate best positions for each bus. It is still recommended that the user only grow several levels at once (input).
Integer ITEMCOUNT	number of items in ITEMLIST. ITEMCOUNT can be 0 if NUMLEVELS is -1. Under this condition diagram growth will start at an arbitrary bus (input; 0 by default).
Character*40 ITEMLIST (ITEMCOUNT)	list of items to grow. Each item is given by its generated map string (input).
Real XPOSITION	X coordinate of the first bus on the Slider diagram (input).
Real YPOSITION	X coordinate of the first bus on the Slider diagram (input).
integer EXCLUDELIST (13)	array of 13 elements specifying list of element types to exclude from the grow operation(input). EXCLUDELIST(1) Exclude machines (0 by default) EXCLUDELIST(1) = 0 disabled EXCLUDELIST(1) = 1 enabled EXCLUDELIST(2) Exclude loads (0 by default) EXCLUDELIST(2) = 0 disabled EXCLUDELIST(2) = 1 enabled EXCLUDELIST(3) Exclude fixed shunts (0 by default) EXCLUDELIST(3) = 0 disabled EXCLUDELIST(3) = 1 enabled

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

EXCLUDELIST(4)	Exclude switched shunts (0 by default)
EXCLUDELIST(4) = 0	disabled
EXCLUDELIST(4) = 1	enabled
EXCLUDELIST(5)	Exclude induction machines (0 by default)
EXCLUDELIST(5) = 0	disabled
EXCLUDELIST(5) = 1	enabled
EXCLUDELIST(6)	Exclude AC branches (0 by default)
EXCLUDELIST(6) = 0	disabled
EXCLUDELIST(6) = 1	enabled
EXCLUDELIST(7)	Exclude 2 winding transformers (0 by default)
EXCLUDELIST(7) = 0	disabled
EXCLUDELIST(7) = 1	enabled
EXCLUDELIST(8)	Exclude 3 winding transformers (0 by default)
EXCLUDELIST(8) = 0	disabled
EXCLUDELIST(8) = 1	enabled
EXCLUDELIST(9)	Exclude system switching devices (0 by default)
EXCLUDELIST(9) = 0	disabled
EXCLUDELIST(9) = 1	enabled
EXCLUDELIST(10)	Exclude DC lines (0 by default)
EXCLUDELIST(10) = 0	disabled
EXCLUDELIST(10) = 1	enabled
EXCLUDELIST(11)	Exclude VSC DC lines (0 by default)
EXCLUDELIST(11) = 0	disabled
EXCLUDELIST(11) = 1	enabled
EXCLUDELIST(12)	Exclude N-term DC lines (0 by default)
EXCLUDELIST(12) = 0	disabled
EXCLUDELIST(12) = 1	enabled
EXCLUDELIST(13)	Exclude FACTS devices (0 by default)
EXCLUDELIST(13) = 0	disabled
EXCLUDELIST(13) = 1	enabled

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.16. GROWSTATION

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_GROWSTATION STATIONNUM
```

Python command syntax:

```
growstation(stationnum)
```

Fortran command syntax:

```
CALL GROWSTATION(STATIONNUM)
```

Where:

Integer STATIONNUM	substation number (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.17. GRPG

Use this API to produce a GRPG report in the active Slider diagram in the GUI.

This API routine was first introduced in release 30.3.3.

Batch command syntax:

```
BAT_GRPG IMAGEFILE DUMFILE
```

Python command syntax:

```
ierr = grpg(imagefile, dumfile)
```

Fortran command syntax:

```
CALL GRPGAPI(IMAGEFILE, DUMFILE, IERR)
```

Where:

Character*260 IMAGEFILE	name of the GRPG file to be used (input).
Character*260 DUMFILE	dummy filename; this should be initialized to " " (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.18. IMPORTDRAWFILE

Use this API to import a Draw Data File into an active Slider diagram in the GUI.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_IMPORTDRAWFILE DRAWFILE
```

Python command syntax:

```
importdrawfile(drawfile)
```

Fortran command syntax:

```
CALL IMPORTDRAWFILE(DRAWFILE)
```

Where:

Character*260 DRAWFILE	name of an existing Draw Data File, .drw (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.19. IMPORTIMAGEFILE

Use this API to import an image into the background layer of the active Slider diagram in the GUI.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_IMPORTIMAGEFILE IMAGEFILE
```

Python command syntax:

```
importimagefile(imagefile)
```

Fortran command syntax:

```
CALL IMPORTIMAGEFILE(IMAGEFILE)
```

Where:

Character*260 IMAGEFILE	name of the image file to be imported (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.20. NEWDIAGFILE

Use this API to open a new Slider diagram in the GUI.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_NEWDIAGFILE
```

Python command syntax:

```
newdiagfile()
```

Fortran command syntax:

```
CALL NEWDIAGFILE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.21. NEXTDIAGFILEDATASET

This API is used to display the next element in the active data set on the active Slider diagram. Currently, this API is only used with ACCC results.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_NEXTDIAGFILEDATASET
```

Python command syntax:

```
nextdiagfileset()
```

Fortran command syntax:

```
CALL NEXTDIAGFILEDATASET
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.22. OPENACCFILE

This API routine is obsolete. It has been replaced by the API routine openaccfile_2 .

This API is used to open an existing ACC results file for displaying results on diagram.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_OPENACCFILE NBASERATE NCNTGRATE FPERRATE SZFILENAME
```

Python command syntax:

```
openaccfile(nbaserate, ncntgrate, fperrate, szfilename)
```

Fortran command syntax:

```
CALL OPENACCFILE(NBASERATE, NCNTGRATE, FPERRATE, SZFILENAME)
```

Where:

Integer NBASERATE	base case rating to use for results display (input). nBaseRate = 1 RateA nBaseRate = 2 RateB nBaseRate = 3 RateC
Integer NCNTGRATE	contingency case rating to use for results display (input). nCntgRate = 1 RateA nCntgRate = 2 RateB nCntgRate = 3 RateC
Real FPERRATE	loading percentage to use for results display (input).
Character*260 SZFILENAME	ACC file to open (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.23. OPENACCFILE_2

This API is used to open an existing ACC results file for displaying results on diagram.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_OPENACCFILE_2 NBASERATE NCNTGRATE NBASEVOLTLIM NCNTGVOLTLIM FPERRATE
 SZFILENAME
```

Python command syntax:

```
openaccfile_2(nbaserate, ncntgrate, nbasevoltlim, ncntgvoltlim, fperrate,
 szfilename)
```

Fortran command syntax:

```
CALL OPENACCFILE_2(NBASERATE, NCNTGRATE, NBASEVOLTLIM, NCNTGVOLTLIM,
 FPERRATE, SZFILENAME)
```

Where:

Integer NBASERATE	base case rating to use for results display (input).	
	nBaseRate = 1	RateA
	nBaseRate = 2	RateB
	nBaseRate = 3	RateC
Integer NCNTGRATE	contingency case rating to use for results display (input).	
	nCntgRate = 1	RateA
	nCntgRate = 2	RateB
	nCntgRate = 3	RateC
Integer NBASEVOLTLIM	base case voltage limit for results display (input).	
	nBaseVoltLim = 1	Don't use the limit
	nBaseVoltLim = 2	Use the limit
Integer NCNTGVOLTLIM	contingencyt case voltage limit for results display (input).	
	nCntgVoltLim = 1	don't use the limit
	nCntgVoltLim = 2	use the limit
Real FPERRATE	loading percentage to use for results display (input).	
Character*260 SZFILENAME	ACC file to open (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.24. OPENACCFILE_3

This API is used to open an existing ACC results file for displaying results on diagram.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_OPENACCFILE_3 SETSCOPE NBASERATE NCNTGRATE NBASEVOLTLIM NCNTGVOLTLIM
 FPERRATE SZFILENAME
```

Python command syntax:

```
openaccfile_3(setscope, nbaserate, ncntgrate, nbasevoltlim, ncntgvoltlim,
 fperrate, szfilename)
```

Fortran command syntax:

```
CALL OPENACCFILE_3(SETSCOPE, NBASERATE, NCNTGRATE, NBASEVOLTLIM,
 NCNTGVOLTLIM, FPERRATE, SZFILENAME)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer NBASERATE	base case rating to use for results display (input).	
	nBaseRate = 1	RateA
	nBaseRate = 2	RateB
	nBaseRate = 3	RateC
Integer NCNTGRATE	contingency case rating to use for results display (input).	
	nCntgRate = 1	RateA
	nCntgRate = 2	RateB
	nCntgRate = 3	RateC
Integer NBASEVOLTLIM	base case voltage limit for results display (input).	
	nBaseVoltLim = 1	Don't use the limit
	nBaseVoltLim = 2	Use the limit
Integer NCNTGVOLTLIM	contingencyt case voltage limit for results display (input).	
	nCntgVoltLim = 1	don't use the limit
	nCntgVoltLim = 2	use the limit
Real FPERRATE	loading percentage to use for results display (input).	
Character*260 SZFILENAME	ACC file to open (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.25. OPENBUSLOCFILE

Use this API to open an existing bus location file in the GUI. The bus location file can be used to control the placement of buses on a diagram during auto-draw and grow operations

This API routine was first introduced in release 30.0.1.

Batch command syntax:

```
BAT_OPENBUSLOCFILE BUSLOCFILE
```

Python command syntax:

```
openbuslocfile(buslocfile)
```

Fortran command syntax:

```
CALL OPENBUSLOCFILE(BUSLOCFILE)
```

Where:

Character*260 BUSLOCFILE	name of existing Bus Location Data File (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.26. OPENDIAGFILE

Use this API to open an existing Slider diagram in the GUI.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_OPENDIAGFILE SFILE
```

Python command syntax:

```
opendiagfile(sfile)
```

Fortran command syntax:

```
CALL OPENDIAGFILE(SFILE)
```

Where:

Character*260 SFILE	error code (output).	
	IERR = 0	no error occured
	IERR = 1	error opening the diagram file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.27. PREVDIAGFILEDATASET

This API is used to display the previous element in the active data set on the active Slider diagram. Currently, this API is only used with ACCC results.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_PREVDIAGFILEDATASET
```

Python command syntax:

```
prevdiagfiledataset()
```

Fortran command syntax:

```
CALL PREVDIAGFILEDATASET
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.28. PRINTDIAGFILE

This API routine is obsolete. It has been replaced by the API routine printdiagfile_2 .

Use this API to print the active Slider diagram in the GUI.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PRINTDIAGFILE SDEV COPIES ORIENT
```

Python command syntax:

```
printdiagfile(sdev, copies, orient)
```

Fortran command syntax:

```
CALL PRINTDIAGFILE(SDEV, COPIES, ORIENT)
```

Where:

Character*260 SDEV	name of valid attached Windows printer (input).	
Integer COPIES	number of copies to print (input).	
Integer ORIENT	orientation (input).	
	orient = 1	Portrait
	orient = 2	Landscape

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.29. PRINTDIAGFILE_2

Use this API to print the active Slider diagram in the GUI.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_PRINTDIAGFILE_2 PAGESETUP VIEWSETUP SDEV FILENAME
```

Python command syntax:

```
printdiagfile_2(pagesetup, viewsetup, sdev, filename)
```

Fortran command syntax:

```
CALL PRINTDIAGFILE_2(PAGESETUP, VIEWSETUP, SDEV, FILENAME)
```

Where:

Integer PAGESETUP (2)	array of 2 elements specifying print options(input).	
	PAGESETUP(1)	number of copies to print (alias is copies) (1 by default)
	PAGESETUP(2)	orientation (alias is orient) (1 by default)
	PAGESETUP(2) = 1	Portrait
	PAGESETUP(2) = 2	Landscape
Real VIEWSETUP (5)	array of 5 elements specifying view setup options(input).	
	VIEWSETUP(1)	left extent of view to print (alias is xmin)
	VIEWSETUP(2)	bottom extent of view to print (alias is ymin)
	VIEWSETUP(3)	right extent of view to print (alias is xmax)
	VIEWSETUP(4)	top extent of view to print (alias is ymax)
	VIEWSETUP(5)	scale to print view at, 0.0 to fit view extents to a single page (alias is scale) (0.0 by default)
Character*260 SDEV	name of valid attached Windows printer (input).	
character*260 FILENAME	name of a file to save the print to, if printing to a file.This parameter is not written out during the recording of the API from the PSSE GUI due to dependencies on the print device. (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.30. REFRESHDIAGCONTOUR

Use this API to refresh the contour in the active Slider diagram in the GUI. Requires a current network case. This API would be used in an automation file to force a contour refresh that wouldn't normally take place until the automation file completed.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_REFRESHDIAGCONTOUR
```

Python command syntax:

```
refreshdiagcontour()
```

Fortran command syntax:

```
CALL REFRESHDIAGCONTOUR
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.31. REFRESHDIAGFILE

Use this API to refresh the results and styles in the active Slider diagram in the GUI. It requires a current network case. This API would be used in an automation file to force a results refresh that wouldn't normally take place until the automation file completed.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_REFRESHDIAGFILE
```

Python command syntax:

```
refreshdiagfile()
```

Fortran command syntax:

```
CALL REFRESHDIAGFILE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.32. RENUMDIAGFILE

Use this API to renumber buses in the active Slider diagram in the GUI.

This API routine was first introduced in release 30.2.1.

Batch command syntax:

```
BAT_RENUMDIAGFILE TRNFILE
```

Python command syntax:

```
renumdiagfile(trnfile)
```

Fortran command syntax:

```
CALL RENUMDIAGFILE(TRNFILE)
```

Where:

Character*260 TRNFILE	name of Bus Number Translation File (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.33. SAVEDIAGFILE

Use this API to save the active Slider diagram in the GUI.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SAVEDIAGFILE SFILE
```

Python command syntax:

```
savediagfile(sfile)
```

Fortran command syntax:

```
CALL SAVEDIAGFILE(SFILE)
```

Where:

Character*260 SFILE	name of file to save the Slider diagram in (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.34. SETDIAGAUTOFILE

Use this API to specify an automation file that is run each time results are updated on the active Slider diagram.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SETDIAGAUTOFILE AUTONAME
```

Python command syntax:

```
setdiagautofile(autoname)
```

Fortran command syntax:

```
CALL SETDIAGAUTOFILE(AUTONAME)
```

Where:

Character*260 AUTONAME	name of the automation file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.35. SETDIAGFONT

Use this API to specify the default text font in the active Slider diagram.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGFONT FNTSIZ FNTWGT FNTNAM
```

Python command syntax:

```
setdiagfont(fntsiz, fntwgt, fntnam)
```

Fortran command syntax:

```
CALL SETDIAGFONT(FNTSIZ, FNTWGT, FNTNAM)
```

Where:

Integer FNTSIZ	font point size (input).
Integer FNTWGT	font weight (input).
character*260 FNTNAM	name of the font family (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.36. SETDIAGPREC

This API routine is obsolete. It has been replaced by the API routine setdiagprec_2 .

Use this API to specify the precision of results displayed in Slider diagrams.

This API routine was first introduced in release 30.1.0.

Batch command syntax:

```
BAT_SETDIAGPREC DIAGPREC
```

Python command syntax:

```
setdiagprec(diagprec)
```

Fortran command syntax:

```
CALL SETDIAGPREC(DIAGPREC)
```

Where:

Integer DIAGPREC (8)	number of digits after the decimal point for various result types. Currently, only DIAGPREC(1) is used for power flow results; valid values are 1 through 4 (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.37. SETDIAGPREC_2

Use this API to specify the precision of results displayed in Slider Diagrams.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGPREC_2 SETSCOP DIAGPREC(1)..DIAGPREC(8)
```

Python command syntax:

```
setdiagprec_2(setscop, diagprec)
```

Fortran command syntax:

```
CALL SETDIAGPREC_2(SETSCOP, DIAGPREC)
```

Where:

Integer SETSCOP	scope of the API (input).	
	SETSCOP = 0	defaults for new diagrams
	SETSCOP = 1	defaults for new diagrams and settings for the active diagram
	SETSCOP = 2	dettings for the active diagram
Integer DIAGPREC	number of digits after the decimal point for various result types. DIAGPREC(1) is used for power flow results, DIAGPREC(2) is used for short circuit results, DIAGPREC(3) is used for OPF results and DIAGPREC(4) is used for PU values. Valid values are 1 through 4. (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.38. SETDIAGPROP

This API routine is obsolete. It has been replaced by the API routine setdiagprop_2 .

Use this API to specify basic properties in the active Slider diagram.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGPROP UNBNDCLR(1)..UNBNDCLR(3) BNDCLR(1)..BNDCLR(3)
 BKGCLR(1)..BKGCLR(3) PRTCLR(1)..PRTCLR(3) GRDCLR(1)..GRDCLR(3)
 UPARWCLR(1)..UPARWCLR(3) DWNARWCLR(1)..DWNARWCLR(3) LABCLR(1)..LABCLR(3)
 GRDSTYL GRDPTRN FLTVAL(1) FLTVAL(2) DBLVAL(1)..DBLVAL(10)
 LOGVAL(1)..LOGVAL(6) BARCHRT LABLHIDE PRNTMODE IMGDIR
```

Python command syntax:

```
setdiagprop(unbndclr, bndclr, bkgclr, prtclr, grdclr, uparwclr, dwnarwclr,
 labclr, grdstyl, grdptrn, fltval, dblval, logval, barchrt, lablhide,
 prntmode, imgdir)
```

Fortran command syntax:

```
CALL SETDIAGPROP(UNBNDCLR, BNDCLR, BKGCLR, PRTCLR, GRDCLR, UPARWCLR,
 DWNARWCLR, LABCLR, GRDSTYL, GRDPTRN, FLTVAL, DBLVAL, LOGVAL, BARCHRT,
 LABLHIDE, PRNTMODE, IMGDIR)
```

Where:

Integer UNBNDCLR (3)	array of 3 elements specifying unbound item color(input).	
	UNBNDCLR(1)	0->255 Red component
	UNBNDCLR(2)	0->255 Blue component
	UNBNDCLR(3)	0->255 Green component
Integer BNDCLR (3)	array of 3 elements specifying bound item color(input).	
	BNDCLR(1)	0->255 Red component
	BNDCLR(2)	0->255 Blue component
	BNDCLR(3)	0->255 Green component
Integer BKGCLR (3)	array of 3 elements specifying background color(input).	
	BKGCLR(1)	0->255 Red component
	BKGCLR(2)	0->255 Blue component
	BKGCLR(3)	0->255 Green component
Integer PRTCLR (3)	array of 3 elements specifying item connection port color(input).	
	PRTCLR(1)	0->255 Red component
	PRTCLR(2)	0->255 Blue component
	PRTCLR(3)	0->255 Green component
Integer GRDCLR (3)	array of 3 elements specifying on-diagram layout grid color(input).	
	GRDCLR(1)	0->255 Red component
	GRDCLR(2)	0->255 Blue component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	GRDCLR(3)	0->255 Green component
Integer UPARWCLR (3)	array of 3 elements specifying active power animation arrow color(input).	
	UPARWCLR(1)	0->255 Red component
	UPARWCLR(2)	0->255 Blue component
	UPARWCLR(3)	0->255 Green component
Integer DWNARWCLR (3)	array of 3 elements specifying reactive power animation arrow color(input).	
	DWNARWCLR(1)	0->255 Red component
	DWNARWCLR(2)	0->255 Blue component
	DWNARWCLR(3)	0->255 Green component
Integer LABCLR (3)	array of 3 elements specifying label color(input).	
	LABCLR(1)	0->255 Red component
	LABCLR(2)	0->255 Blue component
	LABCLR(3)	0->255 Green component
Integer GRDSTYL	grid style (input).	
	GRDSTYL = 0	Dotted
	GRDSTYL = 1	Lined
Integer GRDPTRN	grid pattern (input).	
Real FLTVAL (2)	array of 2 elements specifying (input).	
	FLTVAL(1)	grid point size
	FLTVAL(2)	default line width in pixels
Real DBLVAL (10)	array of 10 elements specifying (input).	
	DBLVAL(1)	maximum zoom factor x 100, i.e. 10=1000% zoom
	DBLVAL(2)	minimum zoom factor
	DBLVAL(3)	zoom increment
	DBLVAL(4)	zoom at which to hide selection handles
	DBLVAL(5)	zoom at which to hide text
	DBLVAL(6)	zoom at which to hide grid
	DBLVAL(7)	multi-page print scaling
	DBLVAL(8)	pan distance
	DBLVAL(9)	grid spacing
	DBLVAL(10)	grid snap distance
Integer LOGVAL (6)	array of 6 elements specifying Each element has only two possible values. 0 = FALSE and 1 = TRUE. The elements are as follows(input).	
	LOGVAL(1)	no upside down text
	LOGVAL(2)	high quality printing
	LOGVAL(3)	display tooltips
	LOGVAL(4)	stipple grid
	LOGVAL(5)	hide unbound network items
	LOGVAL(6)	display filled busbars

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BARCHRT	Two-winding transformer bar chart display side (input). BARCHRT = 0 high current loading side BARCHRT = 1 low current loading side BARCHRT = 2 both sides
Integer LABLHIDE	Label hide mode (input). LABLHIDE = 0 show all labels and results LABLHIDE = 1 hide results LABLHIDE = 2 hide all labels and results
Integer PRNTMODE	Printing mode (input). PRNTMODE = 0 as seen on the screen, WYSIWYG PRNTMODE = 1 fit the whole diagram on the page PRNTMODE = 2 multi-page
Character*260 IMGDIR	name of directory where image files for this diagram are stored (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.39. SETDIAGPROP_2

This API routine is obsolete. It has been replaced by the API routine setdiagprop_3 .

Use this API to specify basic properties in the active Slider diagram.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SETDIAGPROP_2 BKGCLR(1)..BKGCLR(3) PRTCLR(1)..PRTCLR(3)
 GRDCLR(1)..GRDCLR(3) UPARWCLR(1)..UPARWCLR(3) DWNARWCLR(1)..DWNARWCLR(3)
 LABCLR(1)..LABCLR(3) GRDSTYL GRDPTRN FLTVAL(1) FLTVAL(2)
 DBLVAL(1)..DBLVAL(10) LOGVAL(1)..LOGVAL(6) LABLHIDE PRNTMODE IMGDIR
```

Python command syntax:

```
setdiagprop_2(bkgclr, prtclr, grdclr, uparwclr, dwnarwclr, labclr,
 grdstyl, grdptrn, fltval, dblval, logval, lablhide, prntmode, imgdir)
```

Fortran command syntax:

```
CALL SETDIAGPROP_2(BKGCLR, PRTCLR, GRDCLR, UPARWCLR, DWNARWCLR, LABCLR,
 GRDSTYL, GRDPTRN, FLTVAL, DBLVAL, LOGVAL, LABLHIDE, PRNTMODE, IMGDIR)
```

Where:

Integer BKGCLR (3)	array of 3 elements specifying background color(input).	
	BKGCLR(1)	0->255 Red component
	BKGCLR(2)	0->255 Blue component
	BKGCLR(3)	0->255 Green component
Integer PRTCLR (3)	array of 3 elements specifying item connection port color(input).	
	PRTCLR(1)	0->255 Red component
	PRTCLR(2)	0->255 Blue component
	PRTCLR(3)	0->255 Green component
Integer GRDCLR (3)	array of 3 elements specifying on-diagram layout grid color(input).	
	GRDCLR(1)	0->255 Red component
	GRDCLR(2)	0->255 Blue component
	GRDCLR(3)	0->255 Green component
Integer UPARWCLR (3)	array of 3 elements specifying active power animation arrow color(input).	
	UPARWCLR(1)	0->255 Red component
	UPARWCLR(2)	0->255 Blue component
	UPARWCLR(3)	0->255 Green component
Integer DWNARWCLR (3)	array of 3 elements specifying reactive power animation arrow color(input).	
	DWNARWCLR(1)	0->255 Red component
	DWNARWCLR(2)	0->255 Blue component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	DWNARWCLR(3)	0->255 Green component
Integer LABCLR (3)	array of 3 elements specifying label color(input).	
	LABCLR(1)	0->255 Red component
	LABCLR(2)	0->255 Blue component
	LABCLR(3)	0->255 Green component
Integer GRDSTYL	grid style (input).	
	GRDSTYL = 0	Dotted
	GRDSTYL = 1	Lined
Integer GRDPTRN	grid pattern (input).	
Real FLTVAL (2)	array of 2 elements specifying (input).	
	FLTVAL(1)	grid point size
	FLTVAL(2)	default line width in pixels
Real DBLVAL (10)	array of 10 elements specifying (input).	
	DBLVAL(1)	maximum zoom factor x 100, i.e. 10=1000% zoom
	DBLVAL(2)	minimum zoom factor
	DBLVAL(3)	zoom increment
	DBLVAL(4)	zoom at which to hide selection handles
	DBLVAL(5)	zoom at which to hide text
	DBLVAL(6)	zoom at which to hide grid
	DBLVAL(7)	multi-page print scaling
	DBLVAL(8)	pan distance
	DBLVAL(9)	grid spacing
	DBLVAL(10)	grid snap distance
Integer LOGVAL (6)	array of 6 elements specifying Each element has only two possible values. 0 = FALSE and 1 = TRUE. The elements are as follows(input).	
	LOGVAL(1)	no upside down text
	LOGVAL(2)	high quality printing
	LOGVAL(3)	display tooltips
	LOGVAL(4)	stipple grid
	LOGVAL(5)	hide unbound network items
	LOGVAL(6)	display filled busbars
Integer LABLHIDE	Label hide mode (input).	
	LABLHIDE = 0	show all labels and results
	LABLHIDE = 1	hide results
	LABLHIDE = 2	hide all labels and results
Integer PRNTMODE	Printing mode (input).	
	PRNTMODE = 0	as seen on the screen, WYSIWYG
	PRNTMODE = 1	fit the whole diagram on the page
	PRNTMODE = 2	multi-page
Character*260 IMGDIR	name of the directory where image files for this diagram are stored (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.40. SETDIAGPROP_3

This API routine is obsolete. It has been replaced by the API routine setdiagprop_4 .

Use this API to specify basic properties in the active Slider diagram.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SETDIAGPROP_3 BKGCLR(1)..BKGCLR(3) PRTCLR(1)..PRTCLR(3)
 GRDCLR(1)..GRDCLR(3) UPARWCLR(1)..UPARWCLR(3) DWNARWCLR(1)..DWNARWCLR(3)
 LABCLR(1)..LABCLR(3) GRDSTYL GRDPTRN FLTVAL(1)..FLTVAL(2)
 DBLVAL(1)..DBLVAL(10) LOGVAL(1)..LOGVAL(7) LABLHIDE PRNTMODE IMGDIR
```

Python command syntax:

```
setdiagprop_3(bkgclr, prtclr, grdclr, uparwclr, dwnarwclr, labclr,
 grdstyl, grdptrn, fltval, dblval, logval, lablhide, prntmode, imgdir)
```

Fortran command syntax:

```
CALL SETDIAGPROP_3(BKGCLR, PRTCLR, GRDCLR, UPARWCLR, DWNARWCLR, LABCLR,
 GRDSTYL, GRDPTRN, FLTVAL, DBLVAL, LOGVAL, LABLHIDE, PRNTMODE, IMGDIR)
```

Where:

Integer BKGCLR (3)	array of 3 elements specifying background color(input).	
	BKGCLR(1)	0->255 Red component
	BKGCLR(2)	0->255 Blue component
	BKGCLR(3)	0->255 Green component
Integer PRTCLR (3)	array of 3 elements specifying item connection port color(input).	
	PRTCLR(1)	0->255 Red component
	PRTCLR(2)	0->255 Blue component
	PRTCLR(3)	0->255 Green component
Integer GRDCLR (3)	array of 3 elements specifying on-diagram layout grid color(input).	
	GRDCLR(1)	0->255 Red component
	GRDCLR(2)	0->255 Blue component
	GRDCLR(3)	0->255 Green component
Integer UPARWCLR (3)	array of 3 elements specifying active power animation arrow color(input).	
	UPARWCLR(1)	0->255 Red component
	UPARWCLR(2)	0->255 Blue component
	UPARWCLR(3)	0->255 Green component
Integer DWNARWCLR (3)	array of 3 elements specifying reactive power animation arrow color(input).	
	DWNARWCLR(1)	0->255 Red component
	DWNARWCLR(2)	0->255 Blue component
	DWNARWCLR(3)	0->255 Green component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer LABCLR (3)	array of 3 elements specifying label color(input).	
	LABCLR(1)	0->255 Red component
	LABCLR(2)	0->255 Blue component
	LABCLR(3)	0->255 Green component
Integer GRDSTYL	grid style (input).	
	GRDSTYL = 0	dotted
	GRDSTYL = 1	lined
Integer GRDPTRN	grid pattern (input).	
Real FLTVAL (2)	array of 2 elements specifying (input).	
	FLTVAL(1)	grid point size
	FLTVAL(2)	default line width in pixels
Real DBLVAL (10)	array of 10 elements specifying (input).	
	DBLVAL(1)	maximum zoom factor x 100, i.e. 10=1000% zoom
	DBLVAL(2)	minimum zoom factor
	DBLVAL(3)	zoom increment
	DBLVAL(4)	zoom at which to hide selection handles
	DBLVAL(5)	zoom at which to hide text
	DBLVAL(6)	zoom at which to hide grid
	DBLVAL(7)	multi-page print scaling
	DBLVAL(8)	pan distance
	DBLVAL(9)	grid spacing
	DBLVAL(10)	grid snap distance
Integer LOGVAL (7)	array of 7 elements specifying Each element has only two possible values. 0 = FALSE and 1 = TRUE. The elements are as follows(input).	
	LOGVAL(1)	no upside down text
	LOGVAL(2)	high quality printing
	LOGVAL(3)	display tooltips
	LOGVAL(4)	stipple grid
	LOGVAL(5)	hide unbound network items
	LOGVAL(6)	display filled busbars
	LOGVAL(7)	make branch connections perpendicular to branches
Integer LABLHIDE	label hide mode (input).	
	LABLHIDE = 0	show all labels and results
	LABLHIDE = 1	hide results
	LABLHIDE = 2	hide all labels and results
Integer PRNTMODE	printing mode (input).	
	PRNTMODE = 0	as seen on the screen, WYSIWYG
	PRNTMODE = 1	fit the whole diagram on the page
	PRNTMODE = 2	multi-page

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 IMGDIR	name of directory where image files for this diagram are stored (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.41. SETDIAGPROP_4

This API routine is obsolete. It has been replaced by the API routine setdiagprop_5 .

Use this API to specify basic properties in the active Slider diagram.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SETDIAGPROP_4 BKGCLR(1)..BKGCLR(3) PRTCLR(1)..PRTCLR(3)
 GRDCLR(1)..GRDCLR(3) UPARWCLR(1)..UPARWCLR(3) DWNARWCLR(1)..DWNARWCLR(3)
 LABCLR(1)..LABCLR(3) GRDSTYL GRDPTRN FLTVAL(1)..FLTVAL(2)
 DBLVAL(1)..DBLVAL(11) LOGVAL(1)..LOGVAL(7) LABLHIDE PRNTMODE IMGDIR
```

Python command syntax:

```
setdiagprop_4(bkgclr, prtclr, grdclr, uparwclr, dwnarwclr, labclr,
 grdstyl, grdptrn, fltval, dblval, logval, lablhide, prntmode, imgdir)
```

Fortran command syntax:

```
CALL SETDIAGPROP_4(BKGCLR, PRTCLR, GRDCLR, UPARWCLR, DWNARWCLR, LABCLR,
 GRDSTYL, GRDPTRN, FLTVAL, DBLVAL, LOGVAL, LABLHIDE, PRNTMODE, IMGDIR)
```

Where:

Integer BKGCLR (3)	array of 3 elements specifying background color(input).	
	BKGCLR(1)	0->255 Red component
	BKGCLR(2)	0->255 Blue component
	BKGCLR(3)	0->255 Green component
Integer PRTCLR (3)	array of 3 elements specifying item connection port color(input).	
	PRTCLR(1)	0->255 Red component
	PRTCLR(2)	0->255 Blue component
	PRTCLR(3)	0->255 Green component
Integer GRDCLR (3)	array of 3 elements specifying on-diagram layout grid color(input).	
	GRDCLR(1)	0->255 Red component
	GRDCLR(2)	0->255 Blue component
	GRDCLR(3)	0->255 Green component
Integer UPARWCLR (3)	array of 3 elements specifying active power animation arrow color(input).	
	UPARWCLR(1)	0->255 Red component
	UPARWCLR(2)	0->255 Blue component
	UPARWCLR(3)	0->255 Green component
Integer DWNARWCLR (3)	array of 3 elements specifying reactive power animation arrow color(input).	
	DWNARWCLR(1)	0->255 Red component
	DWNARWCLR(2)	0->255 Blue component
	DWNARWCLR(3)	0->255 Green component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer LABCLR (3)	array of 3 elements specifying label color(input).	
	LABCLR(1)	0->255 Red component
	LABCLR(2)	0->255 Blue component
	LABCLR(3)	0->255 Green component
Integer GRDSTYL	grid style (input).	
	GRDSTYL = 0	dotted
	GRDSTYL = 1	lined
Integer GRDPTRN	grid pattern (input).	
Real FLTVAL (2)	array of 2 elements specifying (input).	
	FLTVAL(1)	grid point size
	FLTVAL(2)	default line width in pixels
Real DBLVAL (11)	array of 11 elements specifying (input).	
	DBLVAL(1)	maximum zoom factor x 100, i.e. 10=1000% zoom
	DBLVAL(2)	minimum zoom factor
	DBLVAL(3)	zoom increment
	DBLVAL(4)	zoom at which to hide selection handles
	DBLVAL(5)	zoom at which to hide text
	DBLVAL(6)	zoom at which to hide grid
	DBLVAL(7)	multi-page print scaling
	DBLVAL(8)	pan distance
	DBLVAL(9)	grid spacing
	DBLVAL(10)	grid snap distance
	DBLVAL(11)	branch spacing
Integer LOGVAL (7)	array of 7 elements specifying Each element has only two possible values. 0 = FALSE and 1 = TRUE. The elements are as follows(input).	
	LOGVAL(1)	no upside down text
	LOGVAL(2)	high quality printing
	LOGVAL(3)	display tooltips
	LOGVAL(4)	stipple grid
	LOGVAL(5)	hide unbound network items
	LOGVAL(6)	display filled busbars
	LOGVAL(7)	make branch connections perpendicular to branches
Integer LABLHIDE	label hide mode (input).	
	LABLHIDE = 0	show all labels and results
	LABLHIDE = 1	hide results
	LABLHIDE = 2	hide all labels and results
Integer PRNTMODE	printing mode (input).	
	PRNTMODE = 0	as seen on the screen, WYSIWYG
	PRNTMODE = 1	fit the whole diagram on the page
	PRNTMODE = 2	multi-page

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 IMGDIR	name of directory where image files for this diagram are stored (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.42. SETDIAGPROP_5

Use this API to specify basic properties in the active Slider diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGPROP_5 SETSCOPE BKGCLR(1)..BKGCLR(3) PRTCLR(1)..PRTCLR(3)
 GRDCLR(1)..GRDCLR(3) UPARWCLR(1)..UPARWCLR(3) DWNARWCLR(1)..DWNARWCLR(3)
 LABCLR(1)..LABCLR(3) GRDSTYL GRDPTRN FLTVAL(1)..FLTVAL(2)
 DBLVAL(1)..DBLVAL(11) LOGVAL(1)..LOGVAL(7) PRNTMODE IMGDIR
```

Python command syntax:

```
setdiagprop_5(setscope, bkgclr, prtclr, grdclr, uparwclr, dwnarwclr,
 labclr, grdstyl, grdptrn, fltval, dblval, logval, prntmode, imgdir)
```

Fortran command syntax:

```
CALL SETDIAGPROP_5(SETSCOPE, BKGCLR, PRTCLR, GRDCLR, UPARWCLR, DWNARWCLR,
 LABCLR, GRDSTYL, GRDPTRN, FLTVAL, DBLVAL, LOGVAL, PRNTMODE, IMGDIR)
```

Where:

Integer SETSCOPE	is the scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer BKGCLR (3)	array of 3 elements specifying background color(input).	
	BKGCLR(1)	0->255 Red component
	BKGCLR(2)	0->255 Blue component
	BKGCLR(3)	0->255 Green component
Integer PRTCLR (3)	array of 3 elements specifying item connection port color(input).	
	PRTCLR(1)	0->255 Red component
	PRTCLR(2)	0->255 Blue component
	PRTCLR(3)	0->255 Green component
Integer GRDCLR (3)	array of 3 elements specifying on-diagram layout grid color(input).	
	GRDCLR(1)	0->255 Red component
	GRDCLR(2)	0->255 Blue component
	GRDCLR(3)	0->255 Green component
Integer UPARWCLR (3)	array of 3 elements specifying active power animation arrow color(input).	
	UPARWCLR(1)	0->255 Red component
	UPARWCLR(2)	0->255 Blue component
	UPARWCLR(3)	0->255 Green component
Integer DWNARWCLR (3)	array of 3 elements specifying reactive power animation arrow color(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	DWNARWCLR(1)	0->255 Red component
	DWNARWCLR(2)	0->255 Blue component
	DWNARWCLR(3)	0->255 Green component
Integer LABCLR (3)	array of 3 elements specifying label color(input).	
	LABCLR(1)	0->255 Red component
	LABCLR(2)	0->255 Blue component
	LABCLR(3)	0->255 Green component
Integer GRDSTYL	grid style (input).	
	GRDSTYL = 0	dotted
	GRDSTYL = 1	lined
Integer GRDPTRN	grid pattern (input).	
Real FLTVAL (2)	array of 2 elements specifying (input).	
	FLTVAL(1)	grid point size
	FLTVAL(2)	default line width in pixels
Real DBLVAL (11)	array of 11 elements specifying (input).	
	DBLVAL(1)	maximum zoom factor x 100, i.e. 10=1000% zoom
	DBLVAL(2)	minimum zoom factor
	DBLVAL(3)	zoom increment
	DBLVAL(4)	zoom at which to hide selection handles
	DBLVAL(5)	zoom at which to hide text
	DBLVAL(6)	zoom at which to hide grid
	DBLVAL(7)	multi-page print scaling
	DBLVAL(8)	pan distance
	DBLVAL(9)	grid spacing
	DBLVAL(10)	grid snap distance
	DBLVAL(11)	branch spacing
Integer LOGVAL (7)	array of 7 elements specifying Each element has only two possible values. 0 = FALSE and 1 = TRUE. The elements are as follows(input).	
	LOGVAL(1)	no upside down text
	LOGVAL(2)	high quality printing
	LOGVAL(3)	display tooltips
	LOGVAL(4)	stipple grid
	LOGVAL(5)	hide unbound network items
	LOGVAL(6)	display filled busbars
	LOGVAL(7)	make branch connections perpendicular to branches
Integer PRNTMODE	printing mode (input).	
	PRNTMODE = 0	as seen on the screen, WYSIWYG
	PRNTMODE = 1	fit the whole diagram on the page
	PRNTMODE = 2	multi-page

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*260 IMGDIR	name of directory where image files for this diagram are stored (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.43. SETDIAGRESACCC

This API routine is obsolete. It has been replaced by the API routine setdiagresaccc_2 .

Use this API to set the ACCC analysis annotation options in the active Slider diagram.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SETDIAGRESACCC RATE CNTGOPT CONTOUR CNTROPT LOADPER FILE
```

Python command syntax:

```
setdiagresaccc(rate, cntgopt, contour, cntropt, loadper, file)
```

Fortran command syntax:

```
CALL SETDIAGRESACCC(RATE, CNTGOPT, CONTOUR, CNTROPT, LOADPER, FILE)
```

Where:

Integer RATE	branch rating set (input).	
	RATE = 1	RateA
	RATE = 2	RateB
	RATE = 3	RateC
Integer CNTGOPT	contingencies to view (input).	
	CNTGOPT = 1	all contingencies
	CNTGOPT = 2	contingencies causing overload violations
	CNTGOPT = 3	contingencies causing any violations
Integer CONTOUR	contour selection (input).	
	CONTOUR = 0	display data values as text
	CONTOUR = 1	display data values as a contour
Integer CNTROPT	contour value selection (input).	
	CNTROPT = 0	contour bus voltages
	CNTROPT = 1	contour % branch flows
Real LOADPER	loading percentage (input).	
Character*260 FILE	ACCC Contingency Solution data file (input; Not Allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.44. SETDIAGRESACCC_2

This API routine is obsolete. It has been replaced by the API routine setdiagresaccc_3 .

Use this API to set the ACCC analysis annotation options in the active Slider diagram

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SETDIAGRESACCC_2 CNTGOPT SELCNTG CONTOUR CNTROPT
```

Python command syntax:

```
setdiagresaccc_2(cntgopt, selcntg, contour, cntropt)
```

Fortran command syntax:

```
CALL SETDIAGRESACCC_2(CNTGOPT, SELCNTG, CONTOUR, CNTROPT)
```

Where:

Integer SELCNTG	selected contingency (input).	
	SELCNTG = 0	none selected
	SELCNTG != 0	selected contingency from list determined by CNTGOPT
Integer CNTGOPT	contingency to view (input).	
	CNTGOPT = 1	all contingencies
	CNTGOPT = 2	contingencies causing overload violations
	CNTGOPT = 3	contingencies causing any violations
Integer CONTOUR	contour selection (input).	
	CONTOUR = 0	display data values as text
	CONTOUR = 1	display data values as a contour
Integer CNTROPT	contour value selection (input).	
	CNTROPT = 0	contour bus voltages
	CNTROPT = 1	contour % branch flows

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.45. SETDIAGRESACCC_3

Use this API to set the ACCC analysis annotation options in the active Slider diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESACCC_3 SETSCOPE CNTGOPT SELCNTG CONTOUR CNTROPT
```

Python command syntax:

```
setdiagresaccc_3(setscope, cntgopt, selcntg, contour, cntropt)
```

Fortran command syntax:

```
CALL SETDIAGRESACCC_3(SETSCOPE, CNTGOPT, SELCNTG, CONTOUR, CNTROPT)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer SELCNTG	selected contingency (input).	
	SELCNTG = 0	none selected
	SELCNTG != 0	selected contingency from list determined by CNTGOPT
Integer CNTGOPT	contingency to view (input).	
	CNTGOPT = 1	all contingencies
	CNTGOPT = 2	contingencies causing overload violations
	CNTGOPT = 3	contingencies causing any violations
Integer CONTOUR	contour selection (input).	
	CONTOUR = 0	display data values as text
	CONTOUR = 1	display data values as a contour
Integer CNTROPT	contour value selection (input).	
	CNTROPT = 0	contour bus voltages
	CNTROPT = 1	contour % branch flows

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.46. SETDIAGRESASCC

This API routine is obsolete. It has been replaced by the API routine setdiagresascc_2 .

This API is used to set the ASCC short circuit analysis annotation options in the active Slider diagram.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SETDIAGRESASCC ANNOT TYPE BUS BOE CTRIB FRM TO LAST ID STYLE WIDTH
 COLOR(1)..COLOR(3)
```

Python command syntax:

```
setdiagresascc(annot, type, bus, boe, ctrib, frm, to, last, id, style,
 width, color)
```

Fortran command syntax:

```
CALL SETDIAGRESASCC(ANNOT, TYPE, BUS, BOE, CTRIB, FRM, TO, LAST, ID,
 STYLE, WIDTH, COLOR)
```

Where:

Integer ANNOT	fault to display (input).	
	ANNOT = 0	fault currents (only bus faults)
	ANNOT = 1	three phase
	ANNOT = 2	line to Ground (LG)
	ANNOT = 3	line Line to Ground (LLG)
	ANNOT = 4	line to Line (LL)
Integer TYPE	fault display quantity (input).	
	TYPE = 1	Ia1
	TYPE = 2	Ia2
	TYPE = 3	Ia0
	TYPE = 4	IA
	TYPE = 5	IB
	TYPE = 6	IC
	TYPE = 7	3Ia0
Integer BUS	faulted bus (input).	
	BUS = 0	no bus faulted
	BUS > 0	bus to be faulted
Integer BOE	fault location (input).	
	BOE = 1	fault located at the faulted bus
	BOE = 2	fault located at the faulted bus with lines out-aged
	BOE = 3	fault located at the end of lines
Integer CTRIB	contributions at home bus for bus faults (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CTRIB = 0	contributions for selected fault
	CTRIB = 1	contributions at home bus for all faulted buses for selected fault
Integer FRM	from bus if a line fault is applied (input).	
	FRM = 0	no line fault applied
	FRM > 0	from bus
Integer TO	to bus if a line fault is applied (input).	
	TO = 0	no line fault applied
	TO > 0	to bus
Integer LAST	third bus if a line fault is applied to a 3-winding transformer (input).	
	LAST = 0	no line fault applied
	LAST > 0	to bus
Character*2 ID	branch/transformer identifier (input).	
Integer STYLE	line style applied to the faulted bus diagram item (input).	
Integer WIDTH	line width applied to the faulted bus diagram item, >= 1 (input).	
Integer COLOR (3)	array of 3 elements specifying color for diagram items that are faulted(input).	
	COLOR(1)	0->255 Red component
	COLOR(2)	0->255 Blue component
	COLOR(3)	0->255 Green component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.47. SETDIAGRESASCC_2

This API routine is obsolete. It has been replaced by the API routine setdiagresascc_3 .

This API is used to set the ASCC short circuit analysis annotation options in the active Slider diagram.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SETDIAGRESASCC_2 SETSCOPE ANNOT TYPE BUS BOE CTRIB FRM TO LAST ID
```

Python command syntax:

```
setdiagresascc_2(setscope, annot, type, bus, boe, ctrib, frm, to, last,
 id)
```

Fortran command syntax:

```
CALL SETDIAGRESASCC_2(SETSCOPE, ANNOT, TYPE, BUS, BOE, CTRIB, FRM, TO,
 LAST, ID)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer ANNOT	fault to display (input).	
	ANNOT = 0	fault currents (only bus faults)
	ANNOT = 1	three phase
	ANNOT = 2	line to Ground (LG)
	ANNOT = 3	line Line to Ground (LLG)
	ANNOT = 4	line to Line (LL)
Integer TYPE	fault display quantity (input).	
	TYPE = 1	Ia1
	TYPE = 2	Ia2
	TYPE = 3	Ia0
	TYPE = 4	IA
	TYPE = 5	IB
	TYPE = 6	IC
	TYPE = 7	3Ia0
Integer BUS	faulted bus (input).	
	BUS = 0	no bus faulted
	BUS > 0	bus to be faulted
Integer BOE	fault location (input).	
	BOE = 1	fault located at the faulted bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BOE = 2	fault located at the faulted bus with lines outaged
	BOE = 3	fault located at the end of lines
Integer CTRIB	contributions at home bus for bus faults (input).	
	CTRIB = 0	contributions for selected fault
	CTRIB = 1	contributions at home bus for all faulted buses for selected fault
Integer FRM	from bus if a line fault is applied (input).	
	FRM = 0	no line fault applied
	FRM > 0	from bus
Integer TO	to bus if a line fault is applied (input).	
	TO = 0	no line fault applied
	TO > 0	to bus
Integer LAST	third bus if a line fault is applied to a 3-winding transformer (input).	
	LAST = 0	no line fault applied
	LAST > 0	to bus
Character*2 ID	branch/transformer identifier (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.48. SETDIAGRESASCC_3

This API is used to set the ASCC short circuit analysis annotation options in the active Slider diagram.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_SETDIAGRESASCC_3 SETSCOPE ANNOT TYPE BUS BUSSECT BOE CTRIB FRM TO LAST
 ID
```

Python command syntax:

```
setdiagresascc_3(setscope, annot, type, bus, bussect, boe, ctrib, frm, to,
 last, id)
```

Fortran command syntax:

```
CALL SETDIAGRESASCC_3(SETSCOPE, ANNOT, TYPE, BUS, BUSSECT, BOE, CTRIB,
 FRM, TO, LAST, ID)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer ANNOT	fault to display (input).	
	ANNOT = 0	fault currents (only bus faults)
	ANNOT = 1	three phase
	ANNOT = 2	line to Ground (LG)
	ANNOT = 3	line Line to Ground (LLG)
	ANNOT = 4	line to Line (LL)
Integer TYPE	fault display quantity (input).	
	TYPE = 1	Ia1
	TYPE = 2	Ia2
	TYPE = 3	Ia0
	TYPE = 4	IA
	TYPE = 5	IB
	TYPE = 6	IC
	TYPE = 7	3Ia0
Integer BUS	faulted bus (input).	
	BUS = 0	no bus faulted
	BUS > 0	bus to be faulted
Integer BUSSECT	faulted bus section (input).	
	BUSSECT = 0	no bus section faulted
	BUSSECT > 0	bus section to be faulted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BOE	fault location (input).	
	BOE = 1	fault located at the faulted bus
	BOE = 2	fault located at the faulted bus with lines outaged
	BOE = 3	fault located at the end of lines
Integer CTRIB	contributions at home bus for bus faults (input).	
	CTRIB = 0	contributions for selected fault
	CTRIB = 1	contributions at home bus for all faulted buses for selected fault
Integer FRM	from bus if a line fault is applied (input).	
	FRM = 0	no line fault applied
	FRM > 0	from bus
Integer TO	to bus if a line fault is applied (input).	
	TO = 0	no line fault applied
	TO > 0	to bus
Integer LAST	third bus if a line fault is applied to a 3-winding transformer (input).	
	LAST = 0	no line fault applied
	LAST > 0	to bus
Character*2 ID	branch/transformer identifier (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.49. SETDIAGRESDATA

This API routine is obsolete. It has been replaced by the API routine setdiagresdata_2 .

Use this API to specify the impedance data voltage and range checking annotation options in the active Slider Diagram.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESDATA USEVLT VLNWDT(1)..VLNWDT(8) VLNSTY(1)..VLNSTY(8)
 VLTVAL(1)..VLTVAL(7) VLNCLRR(1)..VLNCLRR(8) VLNCLRG(1)..VLNCLRG(8)
 VLNCLRB(1)..VLNCLRB(8) OUTWDT OUTSTY OUTCLR(1)..OUTCLR(3) BUSANN BRNANN
 EQPANN BUSVLT
```

Python command syntax:

```
setdiagresdata(usevlt, vlnwdt, vlnsty, vltval, vlnclrr, vlnclrg, vlnclrb,
 outwdt, outsty, outclr, busann, brnann, eqpann, busvlt)
```

Fortran command syntax:

```
CALL SETDIAGRESDATA(USEVLT, VLNWDT, VLNSTY, VLTVAL, VLNCLRR, VLNCLRG,
 VLNCLRB, OUTWDT, OUTSTY, OUTCLR, BUSANN, BRNANN, EQPANN, BUSVLT)
```

Where:

Integer USEVLT	option to apply voltage level annotation to the Diagram elements (input).	
	USEVLT = 0	don't apply voltage level annotation
	USEVLT = 1	apply voltage level annotation
Integer VLNWDT (8)	array of 8 elements specifying used to set the width of Diagram elements that fall in the appropriate voltage range(input).	
	VLNWDT(1)	base voltage < VLTVAL(1)
	VLNWDT(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNWDT(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNWDT(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNWDT(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNWDT(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNWDT(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNWDT(8)	base voltage >= VLTVAL(7)
Integer VLNSTY (8)	array of 8 elements specifying used to set the line style of Diagram elements that fall in the appropriate voltage range(input).	
	VLNSTY(1)	base voltage < VLTVAL(1)
	VLNSTY(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNSTY(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNSTY(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNSTY(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNSTY(6)	VLTVAL(5) >= base voltage < VLTVAL(6)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VLNSTY(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNSTY(8)	base voltage >= VLTVAL(7)
Real VLTVAL (7)	used to set the voltage ranges used to assign annotation Diagram. Base voltage values should be in ascending order (input).	
Integer VLNCLRR (8)	array of 8 elements specifying used to set the Red color component of Diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRR(1)	base voltage < VLTVAL(1)
	VLNCLRR(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRR(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRR(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRR(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRR(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRR(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRR(8)	base voltage >= VLTVAL(7)
Integer VLNCLRG (8)	array of 8 elements specifying used to set Green color component of Diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRG(1)	base voltage < VLTVAL(1)
	VLNCLRG(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRG(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRG(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRG(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRG(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRG(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRG(8)	base voltage >= VLTVAL(7)
Integer VLNCLRB (8)	array of 8 elements specifying used to set Blue color component of Diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRB(1)	base voltage < VLTVAL(1)
	VLNCLRB(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRB(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRB(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRB(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRB(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRB(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRB(8)	base voltage >= VLTVAL(7)
Integer OUTWDT	line width applied to out of service Diagram items, >= 1 (input).	
Integer OUTSTY	line style applied to out of service Diagram items (input).	
Integer OUTCLR (3)	array of 3 elements specifying color of Diagram items that are out of service(input).	
	OUTCLR(1)	0->255 Red component
	OUTCLR(2)	0->255 Blue component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OUTCLR(3)	0->255 Green component
Integer BUSANN	bus ID annotation (input).	
	BUSANN = 0	number, no name
	BUSANN = 1	name, no number
	BUSANN = 2	number and name
	BUSANN = 3	no number, no name
Integer BRNANN	branch annotation (input).	
	BRNANN = 0	MW/MVAR flows
	BRNANN = 1	MVA
	BRNANN = 2	MVA/% rating
	BRNANN = 3	Amps/pu current
	BRNANN = 4	MW/%rating
	BRNANN = 5	Amps/% rating
	BRNANN = 6	None
Integer EQPANN	equipment annotation (input).	
	EQPANN = 0	MW/MVAR
	EQPANN = 2	none
Integer BUSVLT	bus voltage annotation (input).	
	BUSVLT = 0	magnitude (pu)
	BUSVLT = 1	magnitude (kV)
	BUSVLT = 2	angle (degrees)
	BUSVLT = 3	magnitude (pu) and angle (degrees)
	BUSVLT = 4	magnitude (pu) and magnitude (kV)
	BUSVLT = 5	magnitude (kV) and angle (degrees)
	BUSVLT = 7	magnitude (pu), magnitude (kV) and angle (degrees)
	BUSVLT = 8	none

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.50. SETDIAGRESDATA_2

This API routine is obsolete. It has been replaced by the API routine setdiagresdata_3 .

Use this API to specify the basic Power Flow annotation options in the active Slider Diagram.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SETDIAGRESDATA_2 BRNANN BUSANN
```

Python command syntax:

```
setdiagresdata_2(brnann, busann)
```

Fortran command syntax:

```
CALL SETDIAGRESDATA_2(BRNANN, BUSANN)
```

Where:

Integer BRNANN	branch annotation (input).	
	BRNANN = 6	display impedances
	BRNANN = 7	display ratings
	BRNANN = 8	display impedances & ratings
	BRNANN = 9	display nothing
Integer BUSANN	bus voltage annotation (input).	
	BUSANN = 0	don't display base kV
	BUSANN = 1	display base kV

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.51. SETDIAGRESDATA_3

Use this API to specify the basic Power Flow annotation options in the active Slider Diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESDATA_3 SETSCOPE BRNANN BUSANN
```

Python command syntax:

```
setdiagresdata_3(setscope, brnann, busann)
```

Fortran command syntax:

```
CALL SETDIAGRESDATA_3(SETSCOPE, BRNANN, BUSANN)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer BRNANN	branch annotation (input).	
	BRNANN = 6	display impedances
	BRNANN = 7	display ratings
	BRNANN = 8	display impedances & ratings
	BRNANN = 9	display nothing
Integer BUSANN	bus voltage annotation (input).	
	BUSANN = 0	don't display base kV
	BUSANN = 1	display base kV

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.52. SETDIAGRESDYN

This API routine is obsolete. It has been replaced by the API routine setdiagresdyn_2 .

This API is used to set the dynamic simulation annotation options in the active Slider diagram.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SETDIAGRESDYN MODE CONTOUR
```

Python command syntax:

```
setdiagresdyn(mode, contour)
```

Fortran command syntax:

```
CALL SETDIAGRESDYN(MODE, CONTOUR)
```

Where:

Integer MODE	specifies the simulation result/value to display (input).	
	MODE = 0	voltage violations as specified by SET_VOLTAGE_REC_CHECK and SET_VOLTAGE_DIP_CHECK
	MODE = 1	contour simulation quantities
Integer CONTOUR	specifies the dynamic quantity to contour (input).	
	CONTOUR = 0	bus voltage - kV

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.53. SETDIAGRESDYN_2

This API is used to set the dynamic simulation annotation options in the active Slider diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESDYN_2 SETSCOPE MODE CONTOUR
```

Python command syntax:

```
setdiagresdyn_2(setscope, mode, contour)
```

Fortran command syntax:

```
CALL SETDIAGRESDYN_2(SETSCOPE, MODE, CONTOUR)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer MODE	specifies the simulation result/value to display (input).	
	MODE = 0	voltage violations as specified by SET_VOLTAGE_REC_CHECK and SET_VOLTAGE_DIP_CHECK
	MODE = 1	contour simulation quantities
Integer CONTOUR	specifies the dynamic quantity to contour (input).	
	CONTOUR = 0	Bus voltage - kV

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.54. SETDIAGRESGDIF

Use this API to specify the case comparison voltage and range checking annotation options in the active Slider diagram.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESGDIF USEVLT VLNWDT(1)..VLNWDT(8) VLNSTY(1)..VLNSTY(8)
 VLTVAL(1)..VLTVAL(7) VLNCLRR(1)..VLNCLRR(8) VLNCLRG(1)..VLNCLRG(8)
 VLNCLRB(1)..VLNCLRB(8) OUTWDT OUTSTY OUTCLR(1)..OUTCLR(3) BUSANN
```

Python command syntax:

```
setdiagresgdif(usevlt, vlnwdt, vlnsty, vltval, vlnclrr, vlnclrg, vlnclrb,
 outwdt, outsty, outclr, busann)
```

Fortran command syntax:

```
CALL SETDIAGRESGDIF(USEVLT, VLNWDT, VLNSTY, VLTVAL, VLNCLRR, VLNCLRG,
 VLNCLRB, OUTWDT, OUTSTY, OUTCLR, BUSANN)
```

Where:

Integer USEVLT	flag to apply voltage level annotation to the diagram elements (input).	
	USEVLT = 0	don't apply voltage level annotation
	USEVLT = 1	apply voltage level annotation
Integer VLNWDT (8)	array of 8 elements specifying the width of diagram elements that fall in the appropriate voltage range(input).	
	VLNWDT(1)	base voltage < VLTVAL(1)
	VLNWDT(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNWDT(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNWDT(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNWDT(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNWDT(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNWDT(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNWDT(8)	base voltage >= VLTVAL(7)
Integer VLNSTY (8)	array of 8 elements specifying the line style of diagram elements that fall in the appropriate voltage range(input).	
	VLNSTY(1)	base voltage < VLTVAL(1)
	VLNSTY(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNSTY(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNSTY(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNSTY(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNSTY(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNSTY(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNSTY(8)	base voltage >= VLTVAL(7)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real VLTVAL (7) the voltage ranges used to assign annotation diagram. Base voltage values should be in ascending order (input).

Integer VLNCLRR (8) array of 8 elements specifying the Red color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRR(1)	base voltage < VLTVAL(1)
VLNCLRR(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRR(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRR(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRR(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRR(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRR(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRR(8)	base voltage >= VLTVAL(7)

Integer VLNCLRG (8) array of 8 elements specifying the Green color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRG(1)	base voltage < VLTVAL(1)
VLNCLRG(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRG(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRG(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRG(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRG(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRG(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRG(8)	base voltage >= VLTVAL(7)

Integer VLNCLRB (8) array of 8 elements specifying the Blue color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRB(1)	base voltage < VLTVAL(1)
VLNCLRB(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRB(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRB(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRB(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRB(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRB(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRB(8)	base voltage >= VLTVAL(7)

Integer OUTWDT line width applied to out of service diagram items, >= 1 (input).

Integer OUTSTY line style applied to out of service diagram items (input).

Integer OUTCLR (3) array of 3 elements specifying color for diagram items that are out of service(input).

OUTCLR(1)	0->255 Red component
OUTCLR(2)	0->255 Blue component
OUTCLR(3)	0->255 Green component

Integer BUSANN bus ID annotation (input).

BUSANN = 0	number, no name
BUSANN = 1	name, no number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BUSANN = 2	number and name
BUSANN = 3	no number, no name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.55. SETDIAGRESIEC

This API routine is obsolete. It has been replaced by the API routine setdiagresiec_2 .

Use this API to specify the IEC annotation options in the active Slider diagram.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SETDIAGRESIEC ANNOT TYPE FLTBUS BOE CONTRIB FRM TO LAST ID STYLE WIDTH
 COLOR(1).. COLOR(3)
```

Python command syntax:

```
setdiagresiec(annot, type, fltbus, boe, contrib, frm, to, last, id, style,
 width, color)
```

Fortran command syntax:

```
CALL SETDIAGRESIEC(ANNOT, TYPE, FLTBUS, BOE, CONTRIB, FRM, TO, LAST, ID,
 STYLE, WIDTH, COLOR)
```

Where:

Integer ANNOT	specifies the fault to display (input).	
	ANNOT = 0	fault currents (only bus faults)
	ANNOT = 1	three phase
	ANNOT = 2	line to Ground (LG)
	ANNOT = 3	line Line to Ground (LLG)
	ANNOT = 4	line to Line (LL)
Integer TYPE	specifies the fault display quantity (input).	
	TYPE = 1	Ia1
	TYPE = 2	Ia2
	TYPE = 3	Ia0
	TYPE = 4	IA
	TYPE = 5	IB
	TYPE = 6	IC
	TYPE = 7	3Ia0
Integer FLTBUS	specifies the faulted bus (input).	
	FLTBUS = 0	no bus faulted
	FLTBUS > 0	bus to be faulted
Integer BOE	specifies the fault location (input).	
	BOE = 1	fault located at the faulted bus
	BOE = 2	fault located at the faulted bus with lines out-aged
	BOE = 3	fault located at the end of lines
Integer CONTRIB	specifies contributions at home bus for bus faults (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CONTRIB = 0	contributions for selected fault
	CONTRIB = 1	contributions at home bus for all faulted buses for selected fault
Integer FRM	specifies the from bus if a line fault is applied (input).	
	FRM = 0	no line fault applied
	FRM > 0	from bus
Integer TO	specifies the to bus if a line fault is applied (input).	
	TO = 0	no line fault applied
	TO > 0	to bus
Integer LAST	specifies the third bus if a line fault is applied to a 3-winding transformer (input).	
	LAST = 0	no line fault applied
	LAST > 0	to bus
Character*2 ID	branch/transformer identifier (input).	
Integer STYLE	line style applied to the faulted bus diagram item (input).	
Integer WIDTH	line width applied to the faulted bus diagram item, >= 1 (input).	
Integer COLOR (3)	array of 3 elements specifying color for diagram items that are faulted(input).	
	COLOR(1)	0->255 Red component
	COLOR(2)	0->255 Blue component
	COLOR(3)	0->255 Green component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.56. SETDIAGRESIEC_2

This API routine is obsolete. It has been replaced by the API routine setdiagresiec_3 .

Use this API to specify the IEC annotation options in the active Slider diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESIEC_2 SETSCOPE ANNOT TYPE FLTBUS BOE CONTRIB FRM TO LAST ID
```

Python command syntax:

```
setdiagresiec_2(setscope, annot, type, fltbus, boe, contrib, frm, to,
 last, id)
```

Fortran command syntax:

```
CALL SETDIAGRESIEC_2(SETSCOPE, ANNOT, TYPE, FLTBUS, BOE, CONTRIB, FRM, TO,
 LAST, ID)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer ANNOT	fault to display (input).	
	ANNOT = 0	fault currents (only bus faults)
	ANNOT = 1	three phase
	ANNOT = 2	line to Ground (LG)
	ANNOT = 3	line Line to Ground (LLG)
	ANNOT = 4	line to Line (LL)
Integer TYPE	fault display quantity (input).	
	TYPE = 1	Ia1
	TYPE = 2	Ia2
	TYPE = 3	Ia0
	TYPE = 4	IA
	TYPE = 5	IB
	TYPE = 6	IC
	TYPE = 7	3Ia0
Integer FLTBUS	faulted bus (input).	
	FLTBUS = 0	no bus faulted
	FLTBUS > 0	bus to be faulted
Integer BOE	fault location (input).	
	BOE = 1	fault located at the faulted bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BOE = 2	fault located at the faulted bus with lines outaged
	BOE = 3	fault located at the end of lines
Integer CONTRIB	contributions at home bus for bus faults (input).	
	CONTRIB = 0	contributions for selected fault
	CONTRIB = 1	contributions at home bus for all faulted buses for selected fault
Integer FRM	from bus if a line fault is applied (input).	
	FRM = 0	no line fault applied
	FRM > 0	from bus
Integer TO	to bus if a line fault is applied (input).	
	TO = 0	no line fault applied
	TO > 0	to bus
Integer LAST	third bus if a line fault is applied to a 3-winding transformer (input).	
	LAST = 0	no line fault applied
	LAST > 0	to bus
Character*2 ID	branch/transformer identifier (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.57. SETDIAGRESIEC_3

Use this API to specify the IEC annotation options in the active Slider diagram.

This API routine was first introduced in release 34.2.0.

Batch command syntax:

```
BAT_SETDIAGRESIEC_3 SETSCOPE ANNOT TYPE FLTBUS FLTBUSSECT BOE CONTRIB FRM
 TO LAST ID
```

Python command syntax:

```
setdiagresiec_3(setscope, annot, type, fltbus, fltbussect, boe, contrib,
 frm, to, last, id)
```

Fortran command syntax:

```
CALL SETDIAGRESIEC_3(SETSCOPE, ANNOT, TYPE, FLTBUS, FLTBUSSECT, BOE,
 CONTRIB, FRM, TO, LAST, ID)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer ANNOT	fault to display (input).	
	ANNOT = 0	fault currents (only bus faults)
	ANNOT = 1	three phase
	ANNOT = 2	line to Ground (LG)
	ANNOT = 3	line Line to Ground (LLG)
	ANNOT = 4	line to Line (LL)
Integer TYPE	fault display quantity (input).	
	TYPE = 1	Ia1
	TYPE = 2	Ia2
	TYPE = 3	Ia0
	TYPE = 4	IA
	TYPE = 5	IB
	TYPE = 6	IC
	TYPE = 7	3Ia0
Integer FLTBUS	faulted bus (input).	
	FLTBUS = 0	no bus faulted
	FLTBUS > 0	bus to be faulted
Integer FLTBUSSECT	faulted bus section (input).	
	FLTBUSSECT = 0	no bus section faulted
	FLTBUSSECT > 0	bus section to be faulted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BOE	fault location (input).	
	BOE = 1	fault located at the faulted bus
	BOE = 2	fault located at the faulted bus with lines outaged
	BOE = 3	fault located at the end of lines
Integer CONTRIB	contributions at home bus for bus faults (input).	
	CONTRIB = 0	contributions for selected fault
	CONTRIB = 1	contributions at home bus for all faulted buses for selected fault
Integer FRM	from bus if a line fault is applied (input).	
	FRM = 0	no line fault applied
	FRM > 0	from bus
Integer TO	to bus if a line fault is applied (input).	
	TO = 0	no line fault applied
	TO > 0	to bus
Integer LAST	third bus if a line fault is applied to a 3-winding transformer (input).	
	LAST = 0	no line fault applied
	LAST > 0	to bus
Character*2 ID	branch/transformer identifier (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.58. SETDIAGRESMUST

This API is used to set the MUST simulation annotation options in the active Slider diagram.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETDIAGRESMUST SETSCOPE OPTION1 OPTION2 OPTION3 OPTION4
```

Python command syntax:

```
setdiagresmust(setscope, option1, option2, option3, option4)
```

Fortran command syntax:

```
CALL SETDIAGRESMUST(SETSCOPE, OPTION1, OPTION2, OPTION3, OPTION4)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer OPTION1	scope of the API (input).	
	OPTION1 = 0	defaults for new diagrams
	OPTION1 = 1	defaults for new diagrams and settings for the active diagram
	OPTION1 = 2	settings for the active diagram
Integer OPTION2	specifies the simulation result/value to display (input).	
	OPTION2 = 0	voltage violations as specified by SET_VOLTAGE_REC_CHECK and SET_VOLTAGE_DIP_CHECK
	OPTION2 = 1	contour simulation quantities
Integer OPTION3	specifies the dynamic quantity to contour (input).	
	OPTION3 = 0	Bus voltage - kV
Integer OPTION4	specifies the dynamic quantity to contour (input).	
	OPTION4 = 0	Bus voltage - kV

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.59. SETDIAGRESMUST_2

This API is used to set the MUST simulation annotation options in the active Slider diagram.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETDIAGRESMUST_2 SETSCOPE OPTION1 OPTION2 OPTION3 OPTION4 OPTION5
```

Python command syntax:

```
setdiagresmust_2(setscope, option1, option2, option3, option4, option5)
```

Fortran command syntax:

```
CALL SETDIAGRESMUST_2(SETSCOPE, OPTION1, OPTION2, OPTION3, OPTION4,
 OPTION5)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer OPTION1	scope of the API (input).	
	OPTION1 = 0	defaults for new diagrams
	OPTION1 = 1	defaults for new diagrams and settings for the active diagram
	OPTION1 = 2	settings for the active diagram
Integer OPTION2	specifies the simulation result/value to display (input).	
	OPTION2 = 0	voltage violations as specified by SET_VOLTAGE_REC_CHECK and SET_VOLTAGE_DIP_CHECK
	OPTION2 = 1	contour simulation quantities
Integer OPTION3	specifies the dynamic quantity to contour (input).	
	OPTION3 = 0	Bus voltage - kV
Integer OPTION4	specifies the dynamic quantity to contour (input).	
	OPTION4 = 0	Bus voltage - kV
Integer OPTION5	(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.60. SETDIAGRESOPF

This API routine is obsolete. It has been replaced by the API routine setdiagresopf_2 .

Use this API to specify the OPF annotation options in the active Slider diagram.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_SETDIAGRESOPF OPFBUSVOLTMAG OPFBUSPINJ OPFBUSQINJ OPFLOADSENS
 OPFFXSSENS OPFSWSSENS
```

Python command syntax:

```
setdiagresopf(opfbusvoltmag, opfbuspinj, opfbusqinj, opfloadsens,
 opffxssens, opfswssens)
```

Fortran command syntax:

```
CALL SETDIAGRESOPF(OPFBUSVOLTMAG, OPFBUSPINJ, OPFBUSQINJ, OPFLOADSENS,
 OPFFXSSENS, OPFSWSSENS)
```

Where:

Integer OPFBUSVOLTMAG	bus voltage magnitude flag (input).
Integer OPFBUSPINJ	bus P injection flag (input).
Integer OPFBUSQINJ	bus Q injection flag (input).
Integer OPFLOADSENS	load sensitivities flag (input).
Integer OPFFXSSENS	fixed shunt sensitivities flag (input).
Integer OPFSWSSENS	switched shunt sensitivities flag (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.61. SETDIAGRESOPF_2

Use this API to specify the OPF annotation options in the active Slider diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESOPF_2 SETSCOPE OPFBUSVOLTMAG OPFBUSPINJ OPFBUSQINJ
 OPFLOADSENS OPFFXSSENS OPFSWSSENS
```

Python command syntax:

```
setdiagresopf_2(setscope, opfbusvoltmag, opfbuspinj, opfbusqinj,
 opfloadsens, opffxssens, opfswssens)
```

Fortran command syntax:

```
CALL SETDIAGRESOPF_2(SETSCOPE, OPFBUSVOLTMAG, OPFBUSPINJ, OPFBUSQINJ,
 OPFLOADSENS, OPFFXSSENS, OPFSWSSENS)
```

Where:

Integer SETSCOPE	is the scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer OPFBUSVOLTMAG	bus voltage magnitude flag (input).	
Integer OPFBUSPINJ	bus P injection flag (input).	
Integer OPFBUSQINJ	bus Q injection flag (input).	
Integer OPFLOADSENS	load sensitivities flag (input).	
Integer OPFFXSSENS	fixed shunt sensitivities flag (input).	
Integer OPFSWSSENS	switched shunt sensitivities flag (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.62. SETDIAGRESOPF_3

Use this API to specify the OPF annotation options in the active Slider diagram.

This API routine was first introduced in release 34.5.0.

Batch command syntax:

```
BAT_SETDIAGRESOPF_3 SETSCOPE OPFBUSVOLTMAG OPFBUSPINJ OPFBUSQINJ
 OPFLOADSENS OPFFXSSENS OPFSWSSENS OPFMACSENS1 OPFMACSENS2 OPFMACSENS3
 OPFMACSENS4 OPFMACSENS5
```

Python command syntax:

```
setdiagresopf_3(setscope, opfbusvoltmag, opfbuspinj, opfbusqinj,
 opfloadsens, opfffxssens, opfswssens, opfmacsens1, opfmacsens2,
 opfmacsens3, opfmacsens4, opfmacsens5)
```

Fortran command syntax:

```
CALL SETDIAGRESOPF_3(SETSCOPE, OPFBUSVOLTMAG, OPFBUSPINJ, OPFBUSQINJ,
 OPFLOADSENS, OPFFXSSENS, OPFSWSSENS, OPFMACSENS1, OPFMACSENS2,
 OPFMACSENS3, OPFMACSENS4, OPFMACSENS5)
```

Where:

Integer SETSCOPE	is the scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer OPFBUSVOLTMAG	bus voltage magnitude flag (input).	
Integer OPFBUSPINJ	bus P injection flag (input).	
Integer OPFBUSQINJ	bus Q injection flag (input).	
Integer OPFLOADSENS	load sensitivities flag (input).	
Integer OPFFXSSENS	fixed shunt sensitivities flag (input).	
Integer OPFSWSSENS	switched shunt sensitivities flag (input).	
Integer OPFMACSENS1	machine sensitivities flag (input).	
Integer OPFMACSENS2	machine sensitivities flag (input).	
Integer OPFMACSENS3	machine sensitivities flag (input).	
Integer OPFMACSENS4	machine sensitivities flag (input).	
Integer OPFMACSENS5	machine sensitivities flag (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.63. SETDIAGRESPFLOWCHECK

This API routine is obsolete. It has been replaced by the API routine setdiagrespflowcheck_2 .

Use this API to specify the Power Flow voltage and range checking annotation options in the active Slider diagram.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESPFLOWCHECK LOGARR(1)..LOGARR(4) VLNWDT(1)..VLNWDT(8)
 VLNSTY(1)..VLNSTY(8) VLTVAL(1)..VLTVAL(7) VLNCLRR(1)..VLNCLRR(8)
 VLNCLRG(1)..VLNCLRG(8) VLNCLRB(1)..VLNCLRB(8) LINLOD LLODARR(1)
 LLODARR(2) LLODCLR(1)..LLODCLR(3) BUSVLT(1) BUSVLT(2) BUSVLTWDT(1)
 BUSVLTWDT(2) BUSCLRR(1) BUSCLRR(2) BUSCLRG(1) BUSCLRG(2) BUSCLRB(1)
 BUSCLRB(2) OUTARR(1) OUTARR(2) OUTCLR(1)..OUTCLR(3) BRNRAT BARPERCENT
 BARCLRR(1) BARCLRR(2) BARCLRR(3) BARCLRR(4) BARCLRG(1) BARCLRG(2)
 BARCLRG(3) BARCLRG(4) BARCLRB(1) BARCLRB(2) BARCLRB(3) BARCLRB(4)
```

Python command syntax:

```
setdiagrespflowcheck(logarr, vlnwdt, vlnsty, vltval, vlnclrr, vlnclrg,
 vlnclrb, linlod, llodarr, llodclr, busvlt, busvltwdt, busclrr, busclrg,
 busclrb, outarr, outclr, brnrat, barpercent, barclrr, barclrg, barclrb)
```

Fortran command syntax:

```
CALL SETDIAGRESPFLOWCHECK(LOGARR, VLNWDT, VLNSTY, VLTVAL, VLNCLRR,
 VLNCLRG, VLNCLRB, LINLOD, LLODARR, LLODCLR, BUSVLT, BUSVLTWDT, BUSCLRR,
 BUSCLRG, BUSCLRB, OUTARR, OUTCLR, BRNRAT, BARPERCENT, BARCLRR, BARCLRG,
 BARCLRB)
```

Where:

Integer LOGARR (4)	array of 4 elements specifying flag to determine what annotation is applied. Each element has only two possible values. 0 = FALSE and 1 = TRUE(input).	
	LOGARR(1)	use Voltage level thresholds
	LOGARR(2)	use Line ratings
	LOGARR(3)	use Bus voltage
	LOGARR(4)	use out of service
Integer VLNWDT (8)	array of 8 elements specifying the width of diagram elements that fall in the appropriate voltage range(input).	
	VLNWDT(1)	base voltage < VLTVAL(1)
	VLNWDT(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNWDT(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNWDT(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNWDT(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNWDT(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNWDT(7)	VLTVAL(6) >= base voltage < VLTVAL(7)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VLNWDT(8)	base voltage >= VLTVAL(7)
Integer VLNSTY (8)	array of 8 elements specifying the line style of diagram elements that fall in the appropriate voltage range(input).	
	VLNSTY(1)	base voltage < VLTVAL(1)
	VLNSTY(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNSTY(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNSTY(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNSTY(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNSTY(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNSTY(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNSTY(8)	base voltage >= VLTVAL(7)
Real VLTVAL (7)	set the voltage ranges used to assign annotation diagram. Base voltage values should be in ascending order (input).	
Integer VLNCLRR (8)	array of 8 elements specifying the Red color component of diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRR(1)	base voltage < VLTVAL(1)
	VLNCLRR(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRR(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRR(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRR(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRR(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRR(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRR(8)	base voltage >= VLTVAL(7)
Integer VLNCLRG (8)	array of 8 elements specifying the Green color component of diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRG(1)	base voltage < VLTVAL(1)
	VLNCLRG(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRG(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRG(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRG(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRG(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRG(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRG(8)	base voltage >= VLTVAL(7)
Integer VLNCLRB (8)	array of 8 elements specifying the Blue color component of diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRB(1)	base voltage < VLTVAL(1)
	VLNCLRB(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRB(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRB(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRB(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRB(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRB(7)	VLTVAL(6) >= base voltage < VLTVAL(7)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VLNCLRB(8)	base voltage >= VLTVAL(7)
Real LINLOD	% loading for line rating (input).	
Integer LLODARR (2)	array of 2 elements specifying the width and style of diagram items that exceed the % of line loading(input).	
	LLODARR(1)	Line width >= 1
	LLODARR(2)	Line style
Integer LLODCLR (3)	array of 3 elements specifying color for lines that exceed the % loading rating, LINLOD(input).	
	LLODCLR(1)	0->255 Red component
	LLODCLR(2)	0->255 Blue component
	LLODCLR(3)	0->255 Green component
Real BUSVLT (2)	array of 2 elements specifying the bus voltage check Min and Max(input).	
	BUSVLT(1)	Min. bus voltage
	BUSVLT(2)	Max. bus voltage
Integer BUSVLTWDT (2)	array of 2 elements specifying the bus voltage check width of diagram items at the Min and Max(input).	
	BUSVLTWDT(1)	Min. bus voltage width >= 1
	BUSVLTWDT(2)	Max. bus voltage width >= 1
Integer BUSCLRR (2)	array of 2 elements specifying the Red color component of diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRR(1)	base voltage < BUSVLTWDT(1)
	BUSCLRR(2)	BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRG (2)	array of 2 elements specifying the Green color component of diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRG(1)	base voltage < BUSVLTWDT (1)
	BUSCLRG(2)	BUSVLTWDT (1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRB (2)	array of 2 elements specifying the Blue color component of diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRB(1)	base voltage < BUSVLTWDT (1)
	BUSCLRB(2)	BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer OUTARR (2)	array of 2 elements specifying the width and style of diagram items that are out of service(input).	
	OUTARR(1)	Line width >= 1
	OUTARR(2)	Line style
Integer OUTCLR (3)	array of 3 elements specifying color for diagram items that are out of service(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OUTCLR(1)	0->255 Red component
	OUTCLR(2)	0->255 Blue component
	OUTCLR(3)	0->255 Green component
Integer BRNRAT	branch rating set (input).	
	BRNRAT = 0	RateA
	BRNRAT = 1	RateB
	BRNRAT = 2	RateC
Integer BARPERCENT (4)	array of 4 elements specifying percent overloads at which to apply different color schemes for the line loading charts on the diagram(input).	
	BARPERCENT(1)	first overload voltage point
	BARPERCENT(2)	second overload voltage point
	BARPERCENT(3)	third overload voltage point
	BARPERCENT(4)	fourth overload voltage point
Integer BARCLRR (4)	array of 4 elements specifying the Red color component for the line loading charts on the diagram, 0->255(input).	
	BARCLRR(1)	first line loading chart red value
	BARCLRR(2)	second line loading chart red value
	BARCLRR(3)	third line loading chart red value
	BARCLRR(4)	fourth line loading chart red value
Integer BARCLRG (4)	array of 4 elements specifying the Green color component for the line loading charts on the diagram, 0->255(input).	
	BARCLRG(1)	first line loading chart green value
	BARCLRG(2)	second line loading chart green value
	BARCLRG(3)	third line loading chart green value
	BARCLRG(4)	fourth line loading chart green value
Integer BARCLRB (4)	array of 4 elements specifying the Blue color component for the line loading charts on the diagram, 0->255(input).	
	BARCLRB(1)	first line loading chart blue value
	BARCLRB(2)	second line loading chart blue value
	BARCLRB(3)	third line loading chart blue value
	BARCLRB(4)	fourth line loading chart blue value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.64. SETDIAGRESPFLOWCHECK_2

This API routine is obsolete. It has been replaced by the API routine setdiagrespflowcheck_3 .

Use this API to specify the Power Flow voltage and range checking annotation options in the active Slider Diagram.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SETDIAGRESPFLOWCHECK_2 LOGARR(1) LOGARR(2) LINLOD LLODARR(1)
 LLODARR(2) LLODCLR(1)..LLODCLR(3) BUSVLT(1) BUSVLT(2) BUSVLTWDT(1)
 BUSVLTWDT(2) BUSCLRR(1) BUSCLRR(2) BUSCLRG(1) BUSCLRG(2) BUSCLRB(1)
 BUSCLRB(2) BRNRAT BARPERCENT BARCLRR(1) BARCLRR(2) BARCLRR(3) BARCLRR(4)
 BARCLRG(1) BARCLRG(2) BARCLRG(3) BARCLRG(4) BARCLRB(1) BARCLRB(2)
 BARCLRB(3) BARCLRB(4)
```

Python command syntax:

```
setdiagrespflowcheck_2(logarr, linlod, llodarr, llodclr, busvlt,
 busvltwdt, busclrr, busclrg, busclrb, brnrat, barpercent, barclrr,
 barclrg, barclrb)
```

Fortran command syntax:

```
CALL SETDIAGRESPFLOWCHECK_2(LOGARR, LINLOD, LLODARR, LLODCLR, BUSVLT,
 BUSVLTWDT, BUSCLRR, BUSCLRG, BUSCLRB, BRNRAT, BARPERCENT, BARCLRR,
 BARCLRG, BARCLRB)
```

Where:

Integer LOGARR (2)	array of 2 elements specifying flag to determine what annotation is applied. Each element has only two possible values. 0 = FALSE and 1 = TRUE(input). LOGARR(1) use Line ratings LOGARR(2) use Bus voltage
Real LINLOD	% loading for line rating (input).
Integer LLODARR (2)	array of 2 elements specifying the width and style of Diagram items that exceed the % of line loading(input). LLODARR(1) Line width >= 1 LLODARR(2) Line style
Integer LLODCLR (3)	array of 3 elements specifying the color for lines that exceed the % loading rating, LINLOD(input). LLODCLR(1) 0->255 Red component LLODCLR(2) 0->255 Blue component LLODCLR(3) 0->255 Green component
Real BUSVLT (2)	array of 2 elements specifying the bus voltage check Min and Max(input). BUSVLT(1) Min. bus voltage

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BUSVLT(2)	Max. bus voltage
Integer BUSVLTWDT (2)	array of 2 elements specifying the bus voltage check width of diagram items at the Min and Max(input).	
	BUSVLTWDT(1)	Min. bus voltage width >= 1
	BUSVLTWDT(2)	Max. bus voltage width >= 1
Integer BUSCLRR (2)	array of 2 elements specifying the Red color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRR(1)	base voltage < BUSVLTWDT(1)
	BUSCLRR(2)	BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRG (2)	array of 2 elements specifying the Green color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRG(1)	base voltage < BUSVLTWDT (1)
	BUSCLRG(2)	BUSVLTWDT (1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRB (2)	array of 2 elements specifying the Blue color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRB(1)	base voltage < BUSVLTWDT (1)
	BUSCLRB(2)	BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer BRNRAT	branch rating set (input).	
	BRNRAT = 0	RateA
	BRNRAT = 1	RateB
	BRNRAT = 2	RateC
Integer BARPERCENT (4)	array of 4 elements specifying percent overloads at which to apply different color schemes for the line loading charts on the Diagram(input).	
	BARPERCENT(1)	first overload voltage point
	BARPERCENT(2)	second overload voltage point
	BARPERCENT(3)	third overload voltage point
	BARPERCENT(4)	fourth overload voltage point
Integer BARCLRR (4)	array of 4 elements specifying the Red color component for the line loading charts on the Diagram, 0->255(input).	
	BARCLRR(1)	first line loading chart red value
	BARCLRR(2)	second line loading chart red value
	BARCLRR(3)	third line loading chart red value
	BARCLRR(4)	fourth line loading chart red value
Integer BARCLRG (4)	array of 4 elements specifying the Green color component for the line loading charts on the Diagram, 0->255(input).	
	BARCLRG(1)	first line loading chart green value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BARCLRG(2)	second line loading chart green value
	BARCLRG(3)	third line loading chart green value
	BARCLRG(4)	fourth line loading chart green value
Integer BARCLRB (4)	array of 4 elements specifying the Blue color component for the line loading charts on the Diagram, 0->255(input).	
	BARCLRB(1)	first line loading chart blue value
	BARCLRB(2)	second line loading chart blue value
	BARCLRB(3)	third line loading chart blue value
	BARCLRB(4)	fourth line loading chart blue value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.65. SETDIAGRESPFLOWCHECK_3

Use this API to specify the Power Flow voltage and range checking annotation options in the active Slider Diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESPFLOWCHECK_3 SETSCOPE LOGARR(1)..LOGARR(2) LINLOD
 LLODARR(1)..LLODARR(2) LLODCLR(1)..LLODCLR(3) BUSVLT(1)..BUSVLT(2)
 BUSVLTWDT(1)..BUSVLTWDT(2) BUSCLRR(1)..BUSCLRR(2) BUSCLRG(1)..BUSCLRG(2)
 BUSCLRB(1)..BUSCLRB(2) BRNRAT BARPERCENT(1)..BARPERCENT(4)
 BARCLRR(1)..BARCLRR(4) BARCLRG(1)..BARCLRG(4) BARCLRB(1)..BARCLRB(4)
 TBARCHRT
```

Python command syntax:

```
setdiagrespflowcheck_3(setscope, logarr, linlod, llodarr, llodclr, busvlt,
 busvltwdt, busclrr, busclrg, busclrb, brnrat, barpercent, barclrr,
 barclrg, barclrb, tbarchrt)
```

Fortran command syntax:

```
CALL SETDIAGRESPFLOWCHECK_3(SETSCOPE, LOGARR, LINLOD, LLODARR, LLODCLR,
 BUSVLT, BUSVLTWDT, BUSCLRR, BUSCLRG, BUSCLRB, BRNRAT, BARPERCENT,
 BARCLRR, BARCLRG, BARCLRB, TBARCHRT)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	dettings for the active diagram
Integer LOGARR (2)	array of 2 elements specifying flag to determine what annotation is applied. Each element has only two possible values. 0 = FALSE and 1 = TRUE(input).	
	LOGARR(1)	use Line ratings
	LOGARR(2)	use Bus voltage
Real LINLOD	% loading for line rating (input).	
Integer LLODARR (2)	array of 2 elements specifying the width and style of Diagram items that exceed the % of line loading(input).	
	LLODARR(1)	Line width >= 1
	LLODARR(2)	Line style
Integer LLODCLR (3)	array of 3 elements specifying color for lines that exceed the % loading rating, LINLOD(input).	
	LLODCLR(1)	0->255 Red component
	LLODCLR(2)	0->255 Blue component
	LLODCLR(3)	0->255 Green component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real BUSVLT (2)	array of 2 elements specifying the bus voltage check Min and Max(input).
	BUSVLT(1) Min. bus voltage
	BUSVLT(2) Max. bus voltage
Integer BUSVLTWDT (2)	array of 2 elements specifying the bus voltage check width of diagram items at the Min and Max(input).
	BUSVLTWDT(1) Min. bus voltage width >= 1
	BUSVLTWDT(2) Max. bus voltage width >= 1
Integer BUSCLRR (2)	array of 2 elements specifying the Red color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).
	BUSCLRR(1) base voltage < BUSVLTWDT(1)
	BUSCLRR(2) BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRG (2)	array of 2 elements specifying the Green color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).
	BUSCLRG(1) base voltage < BUSVLTWDT (1)
	BUSCLRG(2) BUSVLTWDT (1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRB (2)	array of 2 elements specifying the Blue color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).
	BUSCLRB(1) base voltage < BUSVLTWDT (1)
	BUSCLRB(2) BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer BRNRAT	branch rating set (input).
	BRNRAT = 1 Rate1
	BRNRAT = 2 Rate2
	BRNRAT = 3 Rate3
	BRNRAT = 4 Rate4
	BRNRAT = 5 Rate5
	BRNRAT = 6 Rate6
	BRNRAT = 7 Rate7
	BRNRAT = 8 Rate8
	BRNRAT = 9 Rate9
	BRNRAT = 10 Rate10
	BRNRAT = 11 Rate11
	BRNRAT = 12 Rate12
Integer BARPERCENT (4)	array of 4 elements specifying percent overloads at which to apply different color schemes for the line loading charts on the Diagram(input).
	BARPERCENT(1) first overload voltage point
	BARPERCENT(2) second overload voltage point

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

BARPERCENT(3)	third overload voltage point
BARPERCENT(4)	fourth overload voltage point

Integer BARCLRR (4) — array of 4 elements specifying the Red color component for the line loading charts on the Diagram, 0->255(input).

BARCLRR(1)	first line loading chart red value
BARCLRR(2)	second line loading chart red value
BARCLRR(3)	third line loading chart red value
BARCLRR(4)	fourth line loading chart red value

Integer BARCLRG (4) — array of 4 elements specifying the Green color component for the line loading charts on the Diagram, 0->255(input).

BARCLRG(1)	first line loading chart green value
BARCLRG(2)	second line loading chart green value
BARCLRG(3)	third line loading chart green value
BARCLRG(4)	fourth line loading chart green value

Integer BARCLRB (4) — array of 4 elements specifying the Blue color component for the line loading charts on the Diagram, 0->255(input).

BARCLRB(1)	first line loading chart blue value
BARCLRB(2)	second line loading chart blue value
BARCLRB(3)	third line loading chart blue value
BARCLRB(4)	fourth line loading chart blue value

Integer TBARCHRT — toggle for the display of the line loading bar charts (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.66. SETDIAGRESPFLOWCHECK_4

Use this API to specify the Power Flow voltage and range checking annotation options in the active Slider Diagram.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_SETDIAGRESPFLOWCHECK_4 SETSCOPE LOGARR(1)..LOGARR(3) LINLOD
 LLODARR(1)..LLODARR(2) LLODCLR(1)..LLODCLR(3) BUSVLT(1)..BUSVLT(2)
 BUSVLTWDT(1)..BUSVLTWDT(2) BUSCLRR(1)..BUSCLRR(2) BUSCLRG(1)..BUSCLRG(2)
 BUSCLRB(1)..BUSCLRB(2) BRNRAT BARPERCENT(1)..BARPERCENT(4)
 BARCLRR(1)..BARCLRR(4) BARCLRG(1)..BARCLRG(4) BARCLRB(1)..BARCLRB(4)
 TBARCHRT SWDRAT
```

Python command syntax:

```
setdiagrespflowcheck_4(setscope, logarr, linlod, llodarr, llodclr, busvlt,
 busvltwdt, busclrr, busclrg, busclrb, brnrat, barpercent, barclrr,
 barclrg, barclrb, tbarchrt, swdrat)
```

Fortran command syntax:

```
CALL SETDIAGRESPFLOWCHECK_4(SETSCOPE, LOGARR, LINLOD, LLODARR, LLODCLR,
 BUSVLT, BUSVLTWDT, BUSCLRR, BUSCLRG, BUSCLRB, BRNRAT, BARPERCENT,
 BARCLRR, BARCLRG, BARCLRB, TBARCHRT, SWDRAT)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	dettings for the active diagram
Integer LOGARR (3)	array of 3 elements specifying flag to determine what annotation is applied. Each element has only two possible values. 0 = FALSE and 1 = TRUE(input).	
	LOGARR(1)	use Branch ratings
	LOGARR(2)	use Bus voltage
	LOGARR(3)	use SWD ratings
Real LINLOD	% loading for line rating (input).	
Integer LLODARR (2)	array of 2 elements specifying the width and style of Diagram items that exceed the % of line loading(input).	
	LLODARR(1)	Line width >= 1
	LLODARR(2)	Line style
Integer LLODCLR (3)	array of 3 elements specifying color for lines that exceed the % loading rating, LINLOD(input).	
	LLODCLR(1)	0->255 Red component
	LLODCLR(2)	0->255 Blue component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	LLODCLR(3)	0->255 Green component
Real BUSVLT (2)	array of 2 elements specifying the bus voltage check Min and Max(input).	
	BUSVLT(1)	Min. bus voltage
	BUSVLT(2)	Max. bus voltage
Integer BUSVLTWDT (2)	array of 2 elements specifying the bus voltage check width of diagram items at the Min and Max(input).	
	BUSVLTWDT(1)	Min. bus voltage width >= 1
	BUSVLTWDT(2)	Max. bus voltage width >= 1
Integer BUSCLRR (2)	array of 2 elements specifying the Red color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRR(1)	base voltage < BUSVLTWDT(1)
	BUSCLRR(2)	BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRG (2)	array of 2 elements specifying the Green color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRG(1)	base voltage < BUSVLTWDT (1)
	BUSCLRG(2)	BUSVLTWDT (1) >= base voltage < BUSVLTWDT (2)
Integer BUSCLRB (2)	array of 2 elements specifying the Blue color component of Diagram elements that fall in the appropriate bus voltage range, 0->255(input).	
	BUSCLRB(1)	base voltage < BUSVLTWDT (1)
	BUSCLRB(2)	BUSVLTWDT(1) >= base voltage < BUSVLTWDT (2)
Integer BRNRAT	branch rating set (input).	
	BRNRAT = 1	Rate1
	BRNRAT = 2	Rate2
	BRNRAT = 3	Rate3
	BRNRAT = 4	Rate4
	BRNRAT = 5	Rate5
	BRNRAT = 6	Rate6
	BRNRAT = 7	Rate7
	BRNRAT = 8	Rate8
	BRNRAT = 9	Rate9
	BRNRAT = 10	Rate10
	BRNRAT = 11	Rate11
	BRNRAT = 12	Rate12
Integer BARPERCENT (4)	array of 4 elements specifying percent overloads at which to apply different color schemes for the line loading charts on the Diagram(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BARPERCENT(1)	first overload voltage point
	BARPERCENT(2)	second overload voltage point
	BARPERCENT(3)	third overload voltage point
	BARPERCENT(4)	fourth overload voltage point
Integer BARCLRR (4)	array of 4 elements specifying the Red color component for the line loading charts on the Diagram, 0->255(input).	
	BARCLRR(1)	first line loading chart red value
	BARCLRR(2)	second line loading chart red value
	BARCLRR(3)	third line loading chart red value
	BARCLRR(4)	fourth line loading chart red value
Integer BARCLRG (4)	array of 4 elements specifying the Green color component for the line loading charts on the Diagram, 0->255(input).	
	BARCLRG(1)	first line loading chart green value
	BARCLRG(2)	second line loading chart green value
	BARCLRG(3)	third line loading chart green value
	BARCLRG(4)	fourth line loading chart green value
Integer BARCLRB (4)	array of 4 elements specifying the Blue color component for the line loading charts on the Diagram, 0->255(input).	
	BARCLRB(1)	first line loading chart blue value
	BARCLRB(2)	second line loading chart blue value
	BARCLRB(3)	third line loading chart blue value
	BARCLRB(4)	fourth line loading chart blue value
Integer TBARCHRT	toggle for the display of the line loading bar charts (input).	
Integer SWDRAT	SWD rating set (input).	
	SWDRAT = 1	Rate1
	SWDRAT = 2	Rate2
	SWDRAT = 3	Rate3
	SWDRAT = 4	Rate4
	SWDRAT = 5	Rate5
	SWDRAT = 6	Rate6
	SWDRAT = 7	Rate7
	SWDRAT = 8	Rate8
	SWDRAT = 9	Rate9
	SWDRAT = 10	Rate10
	SWDRAT = 11	Rate11
	SWDRAT = 12	Rate12

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.67. SETDIAGRESPFLOWOPTIONS

This API routine is obsolete. It has been replaced by the API routine setdiagrespflowoptions_2 .

Use this API to specify the basic Power Flow annotation options in the active Slider diagram.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESPFLOWOPTIONS BRNANN FLWOPT BUSANN BUSVLT EQPANN DIAGTTL
 PMVALINE PMVATRAN USEPGMSET
```

Python command syntax:

```
setdiagrespflowoptions(brnann, flwopt, busann, busvlt, eqpann, diagttl,
 pmvaline, pmvatran, usepgmset)
```

Fortran command syntax:

```
CALL SETDIAGRESPFLOWOPTIONS(BRNANN, FLWOPT, BUSANN, BUSVLT, EQPANN,
 DIAGTTL, PMVALINE, PMVATRAN, USEPGMSET)
```

Where:

Integer BRNANN	branch annotation (input).	
	BRNANN = 0	MW/MVAR flows
	BRNANN = 1	MVA
	BRNANN = 2	MVA/% rating
	BRNANN = 3	Amps/pu current
	BRNANN = 4	MW/%rating
	BRNANN = 5	Amps/% rating
	BRNANN = 6	None
Integer FLWOPT	flow direction (input).	
	FLWOPT = 0	arrows
	FLWOPT = 1	signs
Integer BUSANN	bus ID annotation (input).	
	BUSANN = 0	number, no name
	BUSANN = 1	name, no number
	BUSANN = 2	number and name
	BUSANN = 3	no number, no name
Integer BUSVLT	bus voltage annotation (input).	
	BUSVLT = 0	magnitude (pu)
	BUSVLT = 1	magnitude (kV)
	BUSVLT = 2	angle (degrees)
	BUSVLT = 3	magnitude (pu) and angle (degrees)
	BUSVLT = 4	magnitude (pu) and magnitude (kV)
	BUSVLT = 5	magnitude (kV) and angle (degrees)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BUSVLT = 7	magnitude (pu), magnitude (kV) and angle (degrees)
	BUSVLT = 8	none
Integer EQPANN	equipment annotation (input).	
	EQPANN = 0	MW/MVAR
	EQPANN = 1	MVA
	EQPANN = 2	none
Character*260 DIAGTTL	diagram title (input).	
Integer PMVALINE	loading basis for % loading on non-transformer branches (input).	
	PMVALINE = 0	current loading
	PMVALINE = 1	MVA loading
Integer PMVATRAN	loading basis for % loading on transformer branches (input).	
	PMVATRAN = 0	current loading
	PMVATRAN = 1	MVA loading
Integer USEPGMSET	flag for system or diagram settings for displaying MVATrans and MVA-Line values (input).	
	USEPGMSET = 0	use diagram settings
	USEPGMSET = 1	use System settings

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.68. SETDIAGRESPFLOWOPTIONS_2

This API routine is obsolete. It has been replaced by the API routine setdiagrespflowoptions_3 .

Use this API to specify the basic Power Flow annotation options in the active Slider diagram.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SETDIAGRESPFLOWOPTIONS_2 BRNANN FLWOPT BUSVLT PMVALINE PMVATRAN
 USEPGMSET DIAGTTL
```

Python command syntax:

```
setdiagrespflowoptions_2(brnann, flwopt, busvlt, pmvaline, pmvatran,
 usepgmset, diagttl)
```

Fortran command syntax:

```
CALL SETDIAGRESPFLOWOPTIONS_2(BRNANN, FLWOPT, BUSVLT, PMVALINE, PMVATRAN,
 USEPGMSET, DIAGTTL)
```

Where:

Integer BRNANN	branch annotation (input).	
	BRNANN = 0	MW/MVAR flows
	BRNANN = 1	MVA
	BRNANN = 2	MVA/% rating
	BRNANN = 3	Amps/pu current
	BRNANN = 4	MW/%rating
	BRNANN = 5	Amps/% rating
	BRNANN = 6	None
Integer FLWOPT	flow direction (input).	
	FLWOPT = 0	arrows
	FLWOPT = 1	signs
Integer BUSVLT	bus voltage annotation (input).	
	BUSVLT = 0	magnitude (pu)
	BUSVLT = 1	magnitude (kV)
	BUSVLT = 2	angle (degrees)
	BUSVLT = 3	magnitude (pu) and angle (degrees)
	BUSVLT = 4	magnitude (pu) and magnitude (kV)
	BUSVLT = 5	magnitude (kV) and angle (degrees)
	BUSVLT = 7	magnitude (pu), magnitude (kV) and angle (degrees)
	BUSVLT = 8	none
Integer PMVALINE	loading basis for % loading on non-transformer branches (input).	
	PMVALINE = 0	current loading

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	PMVALINE = 1	MVA loading
Integer PMVATRAN	loading basis for % loading on transformer branches (input).	
	PMVATRAN = 0	current loading
	PMVATRAN = 1	MVA loading
Integer USEPGMSET	flag for system or diagram settings for displaying MVATrans and MVA-Line values (input).	
	USEPGMSET = 0	use diagram settings
	USEPGMSET = 1	use System settings
Character*260 DIAGTTL	diagram title (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.69. SETDIAGRESPFLOWOPTIONS_3

Use this API to specify the basic Power Flow annotation options in the active Slider diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESPFLOWOPTIONS_3 SETSCOPE BRNANN FLWOPT BUSVLT EQIPANN
 PMVALINE PMVATRAN USEPGMSET MSLOPT BUSIDANN BRNIDANN EQIPIDANN
```

Python command syntax:

```
setdiagrespflowoptions_3(setscope, brnann, flwopt, busvlt, eqipann,
 pmvaline, pmvatran, usepgmset, mslopt, busidann, brnidann, eqipidann)
```

Fortran command syntax:

```
CALL SETDIAGRESPFLOWOPTIONS_3(SETSCOPE, BRNANN, FLWOPT, BUSVLT, EQIPANN,
 PMVALINE, PMVATRAN, USEPGMSET, MSLOPT, BUSIDANN, BRNIDANN, EQIPIDANN)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer BRNANN	branch annotation (input).	
	BRNANN = 0	MW/MVAR flows
	BRNANN = 1	MVA
	BRNANN = 2	MVA/% rating
	BRNANN = 3	Amps/pu current
	BRNANN = 4	MW/%rating
	BRNANN = 5	Amps/% rating
	BRNANN = 6	none
Integer FLWOPT	flow direction (input).	
	FLWOPT = 0	arrows
	FLWOPT = 1	signs
Integer BUSVLT	bus voltage annotation (input).	
	BUSVLT = 0	magnitude (pu)
	BUSVLT = 1	magnitude (kV)
	BUSVLT = 2	angle (degrees)
	BUSVLT = 3	magnitude (pu) and angle (degrees)
	BUSVLT = 4	magnitude (pu) and magnitude (kV)
	BUSVLT = 5	magnitude (kV) and angle (degrees)
	BUSVLT = 7	magnitude (pu), magnitude (kV) and angle (degrees)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BUSVLT = 8	none
Integer EQIPANN	equipment annotation (input).	
	EQIPANN = 0	MW/Mvar
	EQIPANN = 1	MVA
	EQIPANN = 2	MW
	EQIPANN = 3	Mvar
	EQIPANN = 4	None
Integer PMVALINE	loading basis for % loading on non-transformer branches (input).	
	PMVALINE = 0	current loading
	PMVALINE = 1	MVA loading
Integer PMVATRAN	loading basis for % loading on transformer branches (input).	
	PMVATRAN = 0	current loading
	PMVATRAN = 1	MVA loading
Integer USEPGMSET	flag for system or diagram settings for displaying MVATrans and MVA-Line values (input).	
	USEPGMSET = 0	use diagram settings
	USEPGMSET = 1	use System settings
Integer MSLOPT	multi-section line (MSL) option (input).	
	MSLOPT = 0	MSL are not displayed
	MSLOPT = 1	MSL are displayed
Integer BUSIDANN	bus id annotation (input).	
	BUSIDANN = 0	bus number only
	BUSIDANN = 1	bus name only
	BUSIDANN = 2	bus number and bus name
	BUSIDANN = 3	none
Integer BRNIDANN	branch Id annotation (input).	
	BRNIDANN = 0	branch Ids are not displayed
	BRNIDANN = 1	branch Ids are displayed
Integer EQIPIDANN	equipment Id annotation (input).	
	EQIPIDANN = 0	equipment Ids are not displayed
	EQIPIDANN = 1	equipment Ids are displayed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.70. SETDIAGRESREL

This API routine is obsolete. It has been replaced by the API routine setdiagresrel_2 .

Use this API to specify the reliability annotation options in the active Slider diagram.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SETDIAGRESREL OUTMODE MODE BUSMODE CNTBUS CNTBRN DETBUS DETBRN PRBBUS
 PRBBRN VOLTREC CNTLABEL CNTLIST
```

Python command syntax:

```
setdiagresrel(outmode, mode, busmode, cntbus, cntbrn, detbus, detbrn,
 prbbus, prbbrn, voltrec, cntlabel, cntlist)
```

Fortran command syntax:

```
CALL SETDIAGRESREL(OUTMODE, MODE, BUSMODE, CNTBUS, CNTBRN, DETBUS, DETBRN,
 PRBBUS, PRBBRN, VOLTREC, CNTLABEL, CNTLIST)
```

Where:

Integer OUTMODE	mode of results displayed (input).	
	OUTMODE = 1	post contingency stage
	OUTMODE = 2	post tripping stage
	OUTMODE = 3	post corrective actions stage
Integer MODE	type of reliability results displayed (input).	
	MODE = 0	contingency analysis results
	MODE = 1	deterministic reliability results
	MODE = 2	probabilistic reliability results
Integer BUSMODE	mode of bus reporting (input).	
	BUSMODE = 0	bus voltage violations
	BUSMODE = 1	bus load curtailments
Integer CNTBUS	If MODE = 0, is the option to specify whether Bus voltage is displayed (input).	
	CNTBUS = 0	don't display Bus voltage
	CNTBUS = 1	display Bus voltage
Integer CNTBRN	If MODE = 0, is the option to specify whether Branch flow loading is displayed (input).	
	CNTBRN = 0	don't display Branch flow loading
	CNTBRN = 1	display MVA loading
	CNTBRN = 2	display Percent (%)
	CNTBRN = 3	display both MVA and Percent (%)
Integer DETBUS	If MODE = 1, is the option to specify which bus voltage violations are displayed. Non-zero values may be summed (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	DETBUS = 0	don't display bus voltage violations
	DETBUS = 1	# of contingencies resulting in violations
	DETBUS = 2	contingency with maximum violation
	DETBUS = 4	average violations (AV)
	DETBUS = 8	maximum violation (MV)
Integer DETBRN	If MODE = 1, is the option to specify which branch flow overloading violations are displayed (input).	
	DETBRN = 0	don't display bus voltage violations
	DETBRN = 1	# of contingencies resulting in violations
	DETBRN = 2	Contingency with maximum loading
	DETBRN = 4	available capacity (AC)
	DETBRN = 6	maximum loading % (ML)
	DETBRN = 8	average violations (AV)
Integer PRBBUS	If MODE = 2, is the option to specify which bus voltage violations are displayed (input).	
	PRBBUS = 0	don't display bus voltage violations
	PRBBUS = 1	frequency (OC/Y)
	PRBBUS = 2	impact (VI)
	PRBBUS = 4	probability (H/Y)
	PRBBUS = 8	expected energy curtailed
Integer PRBBRN	If MODE = 2, is the option to specify which branch flow overloading violations are displayed (input).	
	PRBBRN = 0	don't display bus voltage violations
	PRBBRN = 1	frequency (OC/Y)s
	PRBBRN = 2	impact (FI)
	PRBBRN = 4	probability (H/Y)
Integer VOLTREC	index into the voltage limit record array for the current DFAX file (input).	
Character*32 CNTLABEL	contingency to display results for (input).	
Integer CNTLIST	what contingencies are listed for selection (input).	
	CNTLIST = 1	all contingencies are listed
	CNTLIST = 2	only those contingencies that cause overloading violations are listed
	CNTLIST = 3	only those contingencies that cause violations are listed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.71. SETDIAGRESREL_2

This API routine is obsolete. It has been replaced by the API routine setdiagresrel_3 .

Use this API to specify the reliability annotation options in the active Slider diagram.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SETDIAGRESREL_2 OUTMODE RELMODE BUSMODE BUSDETANN BRNDETANN BUSPRBANN
 BRNPRBANN VOLTREC
```

Python command syntax:

```
setdiagresrel_2(outmode, relmode, busmode, busdetann, brndetann,
 busprbann, brnprbann, voltrec)
```

Fortran command syntax:

```
CALL SETDIAGRESREL_2(OUTMODE, RELMODE, BUSMODE, BUSDETANN, BRNDETANN,
 BUSPRBANN, BRNPRBANN, VOLTREC)
```

Where:

Integer OUTMODE	mode of results displayed (input).	
	OUTMODE = 1	post contingency stage
	OUTMODE = 2	post tripping stage
	OUTMODE = 3	post corrective actions stage
Integer RELMODE	type of reliability results displayed (input).	
	RELMODE = 0	deterministic reliability results
	RELMODE = 1	probabilistic reliability results
Integer BUSMODE	mode of bus reporting (input).	
	BUSMODE = 0	bus voltage violations
	BUSMODE = 1	bus load curtailments
Integer BUSDETANN	If MODE = 1, is the option to specify which bus voltage violations are displayed (input).	
	BUSDETANN = 0	don't display Bus voltage violations
	BUSDETANN = BUSDETANN + 1	# of contingencies resulting in violations
	BUSDETANN = BUSDETANN + 2	contingency with maximum violations
	BUSDETANN = BUSDETANN + 4	average violations (AV)
	BUSDETANN = BUSDETANN + 8	maximum violations (MV)
Integer BRNDETANN	If MODE = 1, is the option to specify which branch flow overloading violations are displayed (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	BRNDETANN = 0 don't display branch flow violations
	BRNDETANN = BRNDE-TANN + 1 # of contingencies resulting in violations
	BRNDETANN = BRNDE-TANN + 2 contingency with maximum loading
	BRNDETANN = BRNDE-TANN + 4 available capacity (AC)
	BRNDETANN = BRNDE-TANN + 8 average violations (AV)
	BRNDETANN = BRNDE-TANN + 16 maximum loading % (ML)
Integer BUSPRBANN	If MODE = 2, is the option to specify which bus voltage violations are displayed (input).
	BUSPRBANN = 0 don't display Bus voltage violations
	BUSPRBANN = BUSPRBANN + 1 frequency (OC/Y)
	BUSPRBANN = BUSPRBANN + 2 impact (VI)
	BUSPRBANN = BUSPRBANN + 4 probability (H/Y)
	BUSPRBANN = BUSPRBANN + 8 expected energy curtailed
Integer BRNPRBANN	If MODE = 2, is the option to specify which branch flow overloading violations are displayed (input).
	BRNPRBANN = 0 don't display branch flow violations
	BRNPRBANN = BRN-PRBANN + 1 Ffrequency (OC/Y)
	BRNPRBANN = BRN-PRBANN + 2 impact (VI)
	BRNPRBANN = BRN-PRBANN + 4 probability (H/Y)
Integer VOLTREC	index into the voltage limit record array for the current DFAX file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.72. SETDIAGRESREL_3

Use this API to specify the reliability annotation options in the active Slider diagram.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SETDIAGRESREL_3 SETSCOPE OUTMODE RELMODE BUSMODE BUSDETANN BRNDETANN
 BUSPRBANN BRNPRBANN VOLTREC
```

Python command syntax:

```
setdiagresrel_3(setscope, outmode, relmode, busmode, busdetann, brndetann,
 busprbann, brnprbann, voltrec)
```

Fortran command syntax:

```
CALL SETDIAGRESREL_3(SETSCOPE, OUTMODE, RELMODE, BUSMODE, BUSDETANN,
 BRNDETANN, BUSPRBANN, BRNPRBANN, VOLTREC)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer OUTMODE	mode of results displayed (input).	
	OUTMODE = 1	post contingency stage
	OUTMODE = 2	post tripping stage
	OUTMODE = 3	post corrective actions stage
Integer RELMODE	type of reliability results displayed (input).	
	RELMODE = 0	deterministic reliability results
	RELMODE = 1	probabilistic reliability results
Integer BUSMODE	mode of bus reporting (input).	
	BUSMODE = 0	bus voltage violations
	BUSMODE = 1	bus load curtailments
Integer BUSDETANN	If MODE = 1, is the option to specify which bus voltage violations are displayed (input).	
	BUSDETANN = 0	don't display Bus voltage violations
	BUSDETANN = BUSDETANN + 1	# of contingencies resulting in violations
	BUSDETANN = BUSDETANN + 2	contingency with maximum violations
	BUSDETANN = BUSDETANN + 4	average violations (AV)
	BUSDETANN = BUSDETANN + 8	maximum violations (MV)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNDETANN	If MODE = 1, is the option to specify which branch flow overloading violations are displayed (input). BRNDETANN = 0 don't display branch flow violations BRNDETANN = BRNDETANN + 1 # of contingencies resulting in violations BRNDETANN = BRNDETANN + 2 contingency with maximum loading BRNDETANN = BRNDETANN + 4 available capacity (AC) BRNDETANN = BRNDETANN + 8 average violations (AV) BRNDETANN = BRNDETANN + 16 maximum loading % (ML)
Integer BUSPRBANN	If MODE = 2, is the option to specify which bus voltage violations are displayed (input). BUSPRBANN = 0 don't display Bus voltage violations BUSPRBANN = BUSPRBANN + 1 frequency (OC/Y) BUSPRBANN = BUSPRBANN + 2 impact (VI) BUSPRBANN = BUSPRBANN + 4 probability (H/Y) BUSPRBANN = BUSPRBANN + 8 expected energy curtailed
Integer BRNPRBANN	If MODE = 2, is the option to specify which branch flow overloading violations are displayed (input). BRNPRBANN = 0 don't display branch flow violations BRNPRBANN = BRNPRBANN + 1 frequency (OC/Y) BRNPRBANN = BRNPRBANN + 2 impact (VI) BRNPRBANN = BRNPRBANN + 4 probability (H/Y)
Integer VOLTREC	index into the voltage limit record array for the current DFAX file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.73. SETDIAGRESSCGR

Use this API to specify the short circuit analysis voltage and range checking annotation options in the active Slider diagram.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESSCGR USEVLT VLNWDT(1)..VLNWDT(8) VLNSTY(1)..VLNSTY(8)
 VLTVAL(1)..VLTVAL(7) VLNCLRR(1)..VLNCLRR(8) VLNCLRG(1)..VLNCLRG(8)
 VLNCLRB(1)..VLNCLRB(8) OUTWDT OUTSTY OUTCLR(1)..OUTCLR(3) FLTWDT FLTSTY
 FLTCLR(1)..FLTCLR(3) BUSANN
```

Python command syntax:

```
setdiagresscgr(usevlt, vlnwdt, vlnsty, vltval, vlnclrr, vlnclrg, vlnclrb,
 outwdt, outsty, outclr, fltwdt, fltsty, fltclr, busann)
```

Fortran command syntax:

```
CALL SETDIAGRESSCGR(USEVLT, VLNWDT, VLNSTY, VLTVAL, VLNCLRR, VLNCLRG,
 VLNCLRB, OUTWDT, OUTSTY, OUTCLR, FLTWDT, FLTSTY, FLTCLR, BUSANN)
```

Where:

Integer USEVLT	flag to apply voltage level annotation to the diagram elements (input).	
	USEVLT = 0	don't apply voltage level annotation
	USEVLT = 1	apply voltage level annotation
Integer VLNWDT (8)	array of 8 elements specifying width of diagram elements that fall in the appropriate voltage range(input).	
	VLNWDT(1)	base voltage < VLTVAL(1)
	VLNWDT(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNWDT(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNWDT(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNWDT(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNWDT(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNWDT(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNWDT(8)	base voltage >= VLTVAL(7)
Integer VLNSTY (8)	array of 8 elements specifying line style of diagram elements that fall in the appropriate voltage range(input).	
	VLNSTY(1)	base voltage < VLTVAL(1)
	VLNSTY(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNSTY(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNSTY(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNSTY(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNSTY(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNSTY(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNSTY(8)	base voltage >= VLTVAL(7)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real VLTVAL (7) voltage ranges used to assign annotation diagram. Base voltage values should be in ascending order (input).

Integer VLNCLRR (8) array of 8 elements specifying set the Red color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRR(1)	base voltage < VLTVAL(1)
VLNCLRR(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRR(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRR(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRR(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRR(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRR(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRR(8)	base voltage >= VLTVAL(7)

Integer VLNCLRG (8) array of 8 elements specifying set Green color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRG(1)	base voltage < VLTVAL(1)
VLNCLRG(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRG(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRG(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRG(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRG(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRG(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRG(8)	base voltage >= VLTVAL(7)

Integer VLNCLRB (8) array of 8 elements specifying set Blue color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRB(1)	base voltage < VLTVAL(1)
VLNCLRB(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRB(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRB(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRB(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRB(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRB(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRB(8)	base voltage >= VLTVAL(7)

Integer OUTWDT line width applied to out of service diagram items, >= 1 (input).

Integer OUTSTY line style applied to out of service diagram items (input).

Integer OUTCLR (3) array of 3 elements specifying color for diagram items that are out of service(input).

OUTCLR(1)	0->255 Red component
OUTCLR(2)	0->255 Blue component
OUTCLR(3)	0->255 Green component

Integer FLTWDT line width applied to the faulted bus diagram item, >= 1 (input).

Integer FLTSTY line style applied to the faulted bus diagram item (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer FLTCLR (3)	array of 3 elements specifying color for Diagram items that are out of service(input).
	FLTCLR(1) 0->255 Red component
	FLTCLR(2) 0->255 Blue component
	FLTCLR(3) 0->255 Green component
Integer BUSANN	bus ID annotation (input).
	BUSANN = 0 number, no name
	BUSANN = 1 name, no number
	BUSANN = 2 number and name
	BUSANN = 3 no number, no name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.74. SETDIAGRESTYPEACC

Use this API to set the type of results displayed in the active Slider diagram to ACCC results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 33.1.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEACC
```

Python command syntax:

```
setdiagrestypeacc()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEACC
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.75. SETDIAGRESTYPEASCC

Use this API to set the type of results displayed in the active Slider diagram to ASCC fault calculation results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEASCC
```

Python command syntax:

```
setdiagrestypeascc()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEASCC
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.76. SETDIAGRESTYPEDATA

Use this API to set the type of results displayed in the active Slider diagram to impedance data values. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEDATA
```

Python command syntax:

```
setdiagrestypedata()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEDATA
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.77. SETDIAGRESTYPEDYN

Use this API to set the type of results displayed in the active Slider diagram to Dynamics data results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEDYN
```

Python command syntax:

```
setdiagrestypedyn()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEDYN
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.78. SETDIAGRESTYPEGDIF

Use this API to set the type of results displayed in the active Slider diagram to the differences in values between the current network case and a supplied comparison case.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEGDIF NAMEARG SFILE
```

Python command syntax:

```
setdiagrestypegdif(namearg, sfile)
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEGDIF(NAMEARG, SFILE)
```

Where:

Integer NAMEARG	flag indicating how to match up buses in the two cases (input). NameArg = 0 by bus number only NameArg = 1 by extended bus name only NameArg = 2 by both number and name.
Character*260 SFILE	name of an existing Saved case to compare the current case against (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.79. SETDIAGRESTYPEGIC

Use this API to set the type of results displayed in the active Slider diagram to GIC solution results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 32.2.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEGIC
```

Python command syntax:

```
setdiagrestypegic()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEGIC
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.80. SETDIAGRESTYPEHARM

Use this API to set the type of results displayed in the active Slider diagram to Harmonics solution results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPHARM
```

Python command syntax:

```
setdiagrestypeharm()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEHARM
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.81. SETDIAGRESTYPEIEC

Use this API to set the type of results displayed in the active Slider diagram to IEC Fault calculation results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEIEC
```

Python command syntax:

```
setdiagrestypeiec()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEIEC
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.82. SETDIAGRESTYPEMUST

Use this API to set the type of results displayed in the active Slider diagram to MUST solution results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEMUST
```

Python command syntax:

```
setdiagrestypemust()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEMUST
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.83. SETDIAGRESTYPEOPF

Use this API to set the type of results displayed in the active Slider diagram to OPF solution results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEOPF
```

Python command syntax:

```
setdiagrestypeopf()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEOPF
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.84. SETDIAGRESTYPEPFLOW

Use this API to set the type of results displayed in the active Slider diagram to Power Flow results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEPFLOW
```

Python command syntax:

```
setdiagrestypepflow()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEPFLOW
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.85. SETDIAGRESTYPEREL

Use this API to set the type of results displayed in the active Slider diagram to reliability analysis results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPEREL
```

Python command syntax:

```
setdiagrestyperel()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPEREL
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.86. SETDIAGRESTYPESCGR

Use this API to set the type of results displayed in the active Slider diagram to the results of a short circuit analysis.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPESCGR OPTIONS(1) .. OPTIONS(8)
```

Python command syntax:

```
setdiagrestypescgr(options)
```

Fortran command syntax:

```
CALL SETDIAGRESTYPESCGR(OPTIONS)
```

Where:

Integer OPTIONS (8)	array of 8 elements specifying analysis options(input).	
	OPTIONS(1)	faults
	OPTIONS(1) = 0	3 phase
	OPTIONS(1) = 1	3 phase and L-G
	OPTIONS(2)	L-G fault
	OPTIONS(2) = 0	A phase
	OPTIONS(2) = 1	zero sequence
	OPTIONS(3)	fault location
	OPTIONS(3) = 0	all
	OPTIONS(3) > 0	faulted bus number
	OPTIONS(4)	flat conditions
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
	OPTIONS(5)	double precision
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes (ignored)
	OPTIONS(6)	dc line/FACTS device option
	OPTIONS(6) = 0	block
	OPTIONS(6) = 1	model as load
	OPTIONS(7)	zero sequence transformer impedance correction option
	OPTIONS(7) = 0	ignore
	OPTIONS(7) = 1	apply
	OPTIONS(8)	branch quantity option
	OPTIONS(8) = 0	currents
	OPTIONS(8) = 1	apparent impedances

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.87. SETDIAGRESTYPETSPF

Use this API to set the type of results displayed in the active Slider diagram to TSPF solution results. Requires that current network elements are being displayed in the diagram view.

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_SETDIAGRESTYPETSPF
```

Python command syntax:

```
setdiagrestypetspf()
```

Fortran command syntax:

```
CALL SETDIAGRESTYPETSPF
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.88. SETDIAGRESVRCS

This API routine is obsolete. It has been replaced by the API routine setdiagresvrcs_2 .

Use this API to specify the system diagram defaults for Voltage coloring and style, Out of Service coloring and style etc.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SETDIAGRESVRCS BUSANN EQPANN USEVCLR VLNWDT(1)..VLNWDT(8)
 VLNSTY(1)..VLNSTY(8) VLTVAL(1)..VLTVAL(7) VLNCLRR(1)..VLNCLRR(8)
 VLNCLRG(1)..VLNCLRG(8) VLNCLRB(1)..VLNCLRB(8) USEOOSCLR OOSWDT OOSSTY
 OUTCLR(1)..OUTCLR(3) BNDCLR(1)..BNDCLR(3) UNBNDCLR(1)..UNBNDCLR(3)
 TRANBARCHRT
```

Python command syntax:

```
setdiagresvrcs(busann, eqpann, usevclr, vlnwdt, vlnsty, vltval, vlnclrr,
 vlnclrg, vlnclrb, useoosclr, ooswdt, oossty, outclr, bndclr, unbndclr,
 tranbarchrt)
```

Fortran command syntax:

```
CALL SETDIAGRESVRCS(BUSANN, EQPANN, USEVCLR, VLNWDT, VLNSTY, VLTVAL,
 VLNCLRR, VLNCLRG, VLNCLRB, USEOOSCLR, OOSWDT, OOSSTY, OUTCLR, BNDCLR,
 UNBNDCLR, TRANBARCHRT)
```

Where:

Integer BUSANN	bus ID annotation (input).	
	BUSANN = 0	number, no name
	BUSANN = 1	name, no number
	BUSANN = 2	number and name
	BUSANN = 3	no number, no name
Integer EQPANN	equipment annotation (input).	
	EQPANN = 0	MW/MVAR
	EQPANN = 1	MVA
	EQPANN = 2	none
Integer USEVCLR	use voltage level coloring . The element has only two possible values. 0 = FALSE and 1 = TRUE (input).	
Integer VLNWDT (8)	array of 8 elements specifying width of diagram elements that fall in the appropriate voltage range(input).	
	VLNWDT(1)	base voltage < VLTVAL(1)
	VLNWDT(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNWDT(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNWDT(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNWDT(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNWDT(6)	VLTVAL(5) >= base voltage < VLTVAL(6)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VLNWDT(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNWDT(8)	base voltage >= VLTVAL(7)
Integer VLNSTY (8)	array of 8 elements specifying line style of diagram elements that fall in the appropriate voltage range(input).	
	VLNSTY(1)	base voltage < VLTVAL(1)
	VLNSTY(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNSTY(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNSTY(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNSTY(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNSTY(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNSTY(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNSTY(8)	base voltage >= VLTVAL(7)
Real VLTVAL (7)	voltage ranges used to assign annotation diagram. Base voltage values should be in ascending order (input).	
Integer VLNCLRR (8)	array of 8 elements specifying the Red color component of diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRR(1)	base voltage < VLTVAL(1)
	VLNCLRR(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRR(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRR(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRR(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRR(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRR(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRR(8)	base voltage >= VLTVAL(7)
Integer VLNCLRG (8)	array of 8 elements specifying the Green color component of diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRG(1)	base voltage < VLTVAL(1)
	VLNCLRG(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRG(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRG(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRG(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRG(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNCLRG(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRG(8)	base voltage >= VLTVAL(7)
Integer VLNCLRB (8)	array of 8 elements specifying the Blue color component of diagram elements that fall in the appropriate voltage range, 0->255(input).	
	VLNCLRB(1)	base voltage < VLTVAL(1)
	VLNCLRB(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNCLRB(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNCLRB(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNCLRB(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNCLRB(6)	VLTVAL(5) >= base voltage < VLTVAL(6)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VLNCLRB(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNCLRB(8)	base voltage >= VLTVAL(7)
Integer USEOOSCLR	Out-of-Service coloring. The element has only two possible values. 0 = FALSE and 1 = TRUE (input).	
Integer OOSWDT	width of diagram items that are out of service. OOSWDT Line width >= 1 (input).	
Integer OOSSTY	line style of diagram items that are out of service. OOSSTY Line style (input).	
Integer OUTCLR (3)	array of 3 elements specifying color for diagram items that are out of service(input).	
	OUTCLR(1)	0->255 Red component
	OUTCLR(2)	0->255 Blue component
	OUTCLR(3)	0->255 Green component
Integer BNDCLR (3)	array of 3 elements specifying bound item color(input).	
	BNDCLR(1)	0->255 Red component
	BNDCLR(2)	0->255 Blue component
	BNDCLR(3)	0->255 Green component
Integer UNBNDCLR (3)	array of 3 elements specifying unbound item color(input).	
	UNBNDCLR(1)	0->255 Red component
	UNBNDCLR(2)	0->255 Blue component
	UNBNDCLR(3)	0->255 Green component
Integer TRANBARCHRT	two-winding transformer bar chart display side. TRANBARCHRT = 0 High current loading side (input).	
	TRANBARCHRT = 1	low current loading side
	TRANBARCHRT = 2	both sides

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.89. SETDIAGRESVRCS_2

Use this API to specify the system diagram defaults for Voltage coloring and style, Out of Service coloring and style etc.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGRESVRCS_2 SETSCOPE USEVOLT VLNWDT(1)..VLNWDT(8)
 VLNSTY(1)..VLNSTY(8) VLTVAL(1)..VLTVAL(7) VLNCLRR(1)..VLNCLRR(8)
 VLNCLRG(1)..VLNCLRG(8) VLNCLRB(1)..VLNCLRB(8) USEOOSCLR OOSWDT OOSSTY
 OUTCLR(1)..OUTCLR(3) BNDCLR(1)..BNDCLR(3) UNBNDCLR(1)..UNBNDCLR(3)
 BRKOPNCLR(1)..BRKOPNCLR(3) BRKCLSCLR(1)..BRKCLSCLR(3) USEVOLTLAY
 HIDEUNBND FLTBUSWIDTH FLTBUSSTYLE FLTBUSCLR(1)..FLTBUSCLR(3)
```

Python command syntax:

```
setdiagresvrcs_2(setscope, usevolt, vlnwdt, vlnsty, vltval, vlnclrr,
 vlnclrg, vlnclrb, useoosclr, ooswdt, oossty, outclr, bndclr, unbndclr,
 brkopnclr, brkclsclr, usevoltlay, hideunbnd, fltbuswidth, fltbusstyle,
 fltbusclr)
```

Fortran command syntax:

```
CALL SETDIAGRESVRCS_2(SETSCOPE, USEVOLT, VLNWDT, VLNSTY, VLTVAL, VLNCLRR,
 VLNCLRG, VLNCLRB, USEOOSCLR, OOSWDT, OOSSTY, OUTCLR, BNDCLR, UNBNDCLR,
 BRKOPNCLR, BRKCLSCLR, USEVOLTLAY, HIDEUNBND, FLTBUSWIDTH, FLTBUSSTYLE,
 FLTBUSCLR)
```

Where:

Integer SETSCOPE	scope of the API (input).	
	SETSCOPE = 0	defaults for new diagrams
	SETSCOPE = 1	defaults for new diagrams and settings for the active diagram
	SETSCOPE = 2	settings for the active diagram
Integer USEVOLT	voltage level coloring. The element has only two possible values. 0 = FALSE and 1 = TRUE (input).	
Integer VLNWDT (8)	array of 8 elements specifying width of Diagram elements that fall in the appropriate voltage range(input).	
	VLNWDT(1)	base voltage < VLTVAL(1)
	VLNWDT(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
	VLNWDT(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
	VLNWDT(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
	VLNWDT(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
	VLNWDT(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
	VLNWDT(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
	VLNWDT(8)	base voltage >= VLTVAL(7)
Integer VLNSTY (8)	array of 8 elements specifying line style of diagram elements that fall in the appropriate voltage range(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

VLNSTY(1)	base voltage < VLTVAL(1)
VLNSTY(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNSTY(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNSTY(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNSTY(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNSTY(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNSTY(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNSTY(8)	base voltage >= VLTVAL(7)

Real VLTVAL (7) — voltage ranges used to assign annotation diagram. Base voltage values should be in ascending order (input).

Integer VLNCLRR (8) — array of 8 elements specifying the Red color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRR(1)	base voltage < VLTVAL(1)
VLNCLRR(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRR(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRR(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRR(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRR(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRR(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRR(8)	base voltage >= VLTVAL(7)

Integer VLNCLRG (8) — array of 8 elements specifying the Green color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRG(1)	base voltage < VLTVAL(1)
VLNCLRG(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRG(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRG(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRG(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRG(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRG(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRG(8)	base voltage >= VLTVAL(7)

Integer VLNCLRB (8) — array of 8 elements specifying the Blue color component of diagram elements that fall in the appropriate voltage range, 0->255(input).

VLNCLRB(1)	base voltage < VLTVAL(1)
VLNCLRB(2)	VLTVAL(1) >= base voltage < VLTVAL(2)
VLNCLRB(3)	VLTVAL(2) >= base voltage < VLTVAL(3)
VLNCLRB(4)	VLTVAL(3) >= base voltage < VLTVAL(4)
VLNCLRB(5)	VLTVAL(4) >= base voltage < VLTVAL(5)
VLNCLRB(6)	VLTVAL(5) >= base voltage < VLTVAL(6)
VLNCLRB(7)	VLTVAL(6) >= base voltage < VLTVAL(7)
VLNCLRB(8)	base voltage >= VLTVAL(7)

Integer USEOOSCLR — Out-of-Service coloring. The element has only two possible values. 0 = FALSE and 1 = TRUE (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer OOSWDT	width of diagram items that are out of service. OOSWDT Line width >= 1 (input).
Integer OOSSTY	line style of diagram items that are out of service. OOSSTY Line style (input).
Integer OUTCLR (3)	array of 3 elements specifying color of diagram items that are out of service(input). OUTCLR(1) 0->255 Red component OUTCLR(2) 0->255 Blue component OUTCLR(3) 0->255 Green component
Integer BNDCLR (3)	array of 3 elements specifying bound item color(input). BNDCLR(1) 0->255 Red component BNDCLR(2) 0->255 Blue component BNDCLR(3) 0->255 Green component
Integer UNBNDCLR (3)	array of 3 elements specifying unbound item color(input). UNBNDCLR(1) 0->255 Red component UNBNDCLR(2) 0->255 Blue component UNBNDCLR(3) 0->255 Green component
Integer BRKOPNCLR (3)	array of 3 elements specifying open breaker/switch item color(input). BRKOPNCLR(1) 0->255 Red component BRKOPNCLR(2) 0->255 Blue component BRKOPNCLR(3) 0->255 Green component
Integer BRKCLSCLR (3)	array of 3 elements specifying closed breaker/switch item color(input). BRKCLSCLR(1) 0->255 Red component BRKCLSCLR(2) 0->255 Blue component BRKCLSCLR(3) 0->255 Green component
Integer USEVOLTLAY	assign items to layers based on base kV (input). USEVOLTLAY = 0 don't assign to layer USEVOLTLAY = 1 assign to layer
Integer HIDEUNBND	not used (input).
Integer FLTBUSWIDTH	line width applied to the faulted bus item (input).
Integer FLTBUSSTYLE	line style applied to the faulted bus item (input).
Integer FLTBUSCLR (3)	array of 3 elements specifying faulted bus item color(input). FLTBUSCLR(1) 0->255 Red component FLTBUSCLR(2) 0->255 Blue component FLTBUSCLR(3) 0->255 Green component

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.90. SETDIAGTITLE

Use this API to set the title of the active Slider diagram.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_SETDIAGTITLE DIAGTITLE
```

Python command syntax:

```
setdiagtitle(diagtitle)
```

Fortran command syntax:

```
CALL SETDIAGTITLE(DIAGTITLE)
```

Where:

Character*260 DIAGTITLE	new title of the diagram (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.91. SWITCHDIAGFILEDATASET

This API is used to rotate through values for the current element in the active data set on the active Slider diagram. Currently, this API is only used with ACCC results and is used for displaying the post contingency, post tripping and post corrective actions values.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SWITCHDIAGFILEDATASET
```

Python command syntax:

```
switchdiagfileset()
```

Fortran command syntax:

```
CALL SWITCHDIAGFILEDATASET
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

10.92. UPDATEBUSLOCDIAGFILE

Use this API to update bus locations in the active Slider diagram in the GUI using the positions found in the current bus location file.

This API routine was first introduced in release 30.2.1.

Batch command syntax:

```
BAT_UPDATEBUSLOCDIAGFILE
```

Python command syntax:

```
updatebuslocdiagfile()
```

Fortran command syntax:

```
CALL UPDATEBUSLOCDIAGFILE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 11
Single Element Data

This chapter defines API routines that are used to retrieve data for a single element (e.g., bus, branch, area, etc.) from the PSS®E engine and test on various program states.

These routines do not record themselves when called with recording enabled.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.1. APIERRSTR

Use this API to return the message string associated with a specific error code returned from a recordable PSSE API routine.

This API routine was first introduced in release 34.4.0.

Python command syntax:

```
ierr, string = apierrstr(api, errcode)
```

Fortran command syntax:

```
CALL APIERRSTR(API, ERRCODE, STRING, IERR)
```

IPLAN command syntax:

```
CALL APIERRSTR(API, ERRCODE, STRING, IERR)
```

Where:

Character API	Name of the API. (input).	
Integer ERRCODE	Error code for API (input).	
Character*148 STRING	Error message for API and ERRCODE (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	API not found; 'STRING' set to spaces
	IERR = 2	ERRCODE out of range; 'STRING' set to spaces

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.2. ARDAT

Use this API to return area totals.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval  = ardat(iar, string)
```

Fortran command syntax:

```
CALL ARDAT(IAR, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL ARDAT(IAR, STRING, P, Q, IERR)
```

Where:

Integer IAR	Area number (input).	
Character STRING	String indicating the area total desired (input).	
	'LOAD'	Total area load by bus area assignment (net of load plus in-service distributed generation on load feeder)
	'LOADLD'	Total area load by load area assignment
	'LDGN'	Total distributed generation on load feeder by bus area assignment
	'LDGNLD'	Total distributed generation on load feeder by load area assignment
	'GEN'	Total area generation
	'LOSS'	Total area losses; I**2 * (R + jX) for ac branches with non-metered end in the area
	'XFRMAG'	Magnetizing admittance powers for transformers with winding 1 bus in the area
	'LNSHNT'	Powers flowing into line shunts of non-transformer branches with non-metered end in the area
	'CHARGE'	Powers flowing from line charging of non-transformer branches with non-metered end in the area
	'INT'	Net area interchange
	'INDMAC'	Total area induction machine powers by bus area assignment
	'INDMACMC'	Total area induction machine powers by machine area assignment
	'INDGEN'	Total area induction generator powers by bus area assignment
	'INDGENMC'	Total area induction generator powers by machine area assignment

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'INDMOT'	Total area induction motor powers by bus area assignment
	'INDMOTMC'	Total area induction motor powers by machine area assignment
Real P	Real component of the desired complex power	
Real Q	Reactive component of the desired complex power	
Complex CMPVAL	Desired complex power (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Area number < 0 or > largest allowable area number; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	No in-service buses with in-service loads (for 'LOAD'), no in-service loads (for 'LOADLD'), no type 2 or type 3 buses (for 'GEN'), no branches (for 'LOSS'), or no ties (for 'INT') in area; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Area not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	Bad 'STRING' value; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.3. AREDAT

Use this API to return real area data items.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval  = aredat(iar, string)
```

Fortran command syntax:

```
CALL AREDAT(IAR, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL AREDAT(IAR, STRING, RVAL, IERR)
```

Where:

Integer IAR	Area number (input).	
Character STRING	String indicating the area quantity desired (input).	
	'PDES'	Desired net interchange leaving area in MW
	'PTOL'	Interchange tolerance bandwidth in MW
	'O_PDES'	Same as 'PDES' in units determined by the power output option setting
	'O_PTOL'	Same as 'PTOL' in units determined by the power output option setting
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Area number <= 0 or > largest allowable area number; 'RVAL' unchanged
	IERR = 2	Area not found; 'RVAL' unchanged
	IERR = 3	Bad 'STRING' value; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.4. AREINT

Use this API to return integer area data items.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival  = areint(iar, string)
```

Fortran command syntax:

```
CALL AREINT(IAR, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL AREINT(IAR, STRING, IVAL, IERR)
```

Where:

Integer IAR	Area number (input).	
Character STRING	String indicating the area quantity desired (input).	
	'SWING'	Area slack bus number
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Area number <= 0 or > largest allowable area number; 'IVAL' unchanged
	IERR = 2	Area not found; 'IVAL' unchanged
	IERR = 3	Bad 'STRING' value; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.5. ARENAM

Use this API to return the area name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval  = arenam(iar)
```

Fortran command syntax:

```
CALL ARENAM(IAR, CVAL, IERR)
```

IPLAN command syntax:

```
CALL ARENAM(IAR, CVAL, IERR)
```

Where:

Integer IAR	Area number (input).	
Character*12 CVAL	Area name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Area number <= 0 or > largest allowable area number; 'CVAL' unchanged
	IERR = 2	Area not found; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.6. ARENUM

Use this API to return the area number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, iar  = arenum(cval)
```

Fortran command syntax:

```
CALL ARENUM(IAR, CVAL, IERR)
```

IPLAN command syntax:

```
CALL ARENUM(IAR, CVAL, IERR)
```

Where:

Integer IAR	Area number (output).	
Character CVAL	Area name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IAR' returned
	IERR = 1	Area with name 'CVAL' not found
	IERR > 1	Multiple areas with the name 'CVAL' found, 'IAR' returned as lowest number of these

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.7. AREUSE

Use this API to indicate if an area is in use.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ival = areuse(iar)
```

Fortran command syntax:

```
CALL AREUSE(IAR, IVAL)
```

IPLAN command syntax:

```
CALL AREUSE(IAR, IVAL)
```

Where:

Integer IAR	Area number (input).	
Integer IVAL	Area status (output).	
	IVAL = -1	Area number <= 0 or > largest allowable area number.
	IVAL = 0	If no equipment is assigned to area IAR.
	IVAL = 1	If any equipment is assigned to area IAR.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.8. ARITOJ

Use this API to return the interchange between two areas.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval  = aritoj(iar, jar)
```

Fortran command syntax:

```
CALL ARITOJ(IAR, JAR, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL ARITOJ(IAR, JAR, P, Q, IERR)
```

Where:

Integer IAR	From area number (input).	
Integer JAR	To area number (input).	
Real P	Real component of the complex interchange	
Real Q	Reactive component of the complex interchange	
Complex CMPVAL	Complex interchange (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Area number <= 0 or > largest allowable area number; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	No in-service branches between areas; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Area not found; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.9. BRNCUR

This API routine is obsolete. It has been replaced by the API routine BRNMSC .

Use this API to return branch flow values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = brncur(ibus,jbus,ickt)
```

Fortran command syntax:

```
CALL BRNCUR(IBUS, JBUS, ICKT, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNCUR(IBUS, JBUS, ICKT, RVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*2 ICKT	Circuit or multisection line identifier (input).	
Real RVAL	Flow value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Branch not found; 'RVAL' unchanged
	IERR = 3	Branch out-of-service; 'RVAL' returned set to zero
	IERR = 4	No base kV at IBUS; 'RVAL' returned = per unit current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.10. BRNDAT

Use this API to return real branch parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = brndat(ibus,jbus,ickt,string)
```

Fortran command syntax:

```
CALL BRNDAT(IBUS, JBUS, ICKT, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNDAT(IBUS, JBUS, ICKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*12 ICKT	Circuit identifier (input).	
Character STRING	String indicating the branch quantity desired (input).	
	'RATEn'	Rating from rating set "n" (n = 1 through 12)
	'RATEA'	First rating
	'RATEB'	Second rating
	'RATEC'	Third rating
	'RATE'	Default rating
	'LENGTH'	Line length
	'CHARG'	Total branch charging
	'CHARGZ'	Zero sequence charging
	'FRACT1'	Fraction of total ownership assigned to the first owner
	'FRACT2'	Fraction of total ownership assigned to the second owner
	'FRACT3'	Fraction of total ownership assigned to the third owner
	'FRACT4'	Fraction of total ownership assigned to the fourth owner
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Branch not found; 'RVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 4	'BRNDAT' invalid with multisection line identifier; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	Sequence data not in case; 'RVAL' unchanged
IERR = 6	For STRING = 'FRACTn', n > number of owners; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.11. BRNDT2

Use this API to return complex positive and zero sequence parameters for non-transformer branches, and complex positive sequence parameters for transformer branches.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = brndt2(ibus,jbus,ickt,string)
```

Fortran command syntax:

```
CALL BRNDT2(IBUS, JBUS, ICKT, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNDT2(IBUS, JBUS, ICKT, STRING, P, Q, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*12 ICKT	Circuit identifier (input).	
Character STRING	String indicating the branch quantity desired (input).	
	'RX'	Branch resistance and reactance in per unit
	'ISHNT'	Admittance of line shunt at IBUS end of the branch
	'JSHNT'	Admittance of line shunt at JBUS end of the branch
	'RXZ'	Zero sequence branch resistance and reactance (not valid for transformers)
	'ISHNTZ'	Zero sequence admittance of line shunt at IBUS end of the branch (not valid for transformers)
	'JSHNTZ'	Zero sequence admittance of line shunt at JBUS end of the branch (not valid for transformers)
	'LOSSES'	Branch losses in MW and Mvar
	'O_LOSSES'	Branch losses in units determined by the power output option setting
Real P	Real component of the complex value indicated by STRING	
Real Q	Reactive component of the complex value indicated by STRING	
Complex CMPVAL	Complex value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Branch not found; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 3	Invalid value of 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 4	'BRNDT2' invalid with multisection line identifier; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 5	Sequence data not in case; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 6	'BRNDT2' zero sequence parameter invalid for transformer; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.12. BRNFLO

Use this API to return the complex branch flow (P+jQ) as calculated at IBUS.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = brnflo(ibus,jbus,ickt)
```

Fortran command syntax:

```
CALL BRNFLO(IBUS, JBUS, ICKT, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNFLO(IBUS, JBUS, ICKT, P, Q, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*2 ICKT	Circuit or multisection line identifier (input).	
Real P	Real component of the complex branch flow	
Real Q	Reactive component of the complex branch flow	
Complex CMPVAL	Complex branch flow (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Branch not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Branch is out-of-service; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.13. BRNINT

Use this API to return integer branch parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = brnint(ibus,jbus,ickt,string)
```

Fortran command syntax:

```
CALL BRNINT(IBUS, JBUS, ICKT, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNINT(IBUS, JBUS, ICKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*12 ICKT	Circuit or multisection line identifier (input).	
Character STRING	String indicating the branch quantity desired (input).	
	'STATUS'	Branch status
	'NSTATUS'	Normal system switching device status
	'METER'	Bus number of metered end
	'NMETR'	Bus number of non-metered end
	'OWNERS'	Number of owners for this branch
	'OWN1'	Owner number of the first owner
	'OWN2'	Owner number of the second owner
	'OWN3'	Owner number of the third owner
	'OWN4'	Owner number of the fourth owner
	'STATION_I'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
	'STATION_J'	Bus JBUS substation number (returned as 0 if bus JBUS is not in a substation)
	'SECTION_I'	Bus IBUS bus section number (returned as 0 if bus IBUS is not in a substation)
	'SECTION_J'	Bus JBUS bus section number (returned as 0 if bus JBUS is not in a substation)
	'NODE_I'	Number of the node to which the branch is connected at the bus IBUS end (returned as 0 if bus IBUS is not in a substation)
	'NODE_J'	Number of the node to which the branch is connected at the bus JBUS end (returned as 0 if bus JBUS is not in a substation)
	'TYPE'	System switching device type (returned as 0 if the branch is not a system switching device).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'SCTYPE'	MOV protection mode
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Branch not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	For 'STRING' = 'OWNn', n > number of owners; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.14. BRNMSC

Use this API to return real branch flow values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = brnmsc(ibus,jbus,ickt,string)
```

Fortran command syntax:

```
CALL BRNMSC(IBUS, JBUS, ICKT, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNMSC(IBUS, JBUS, ICKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character ICKT	Circuit or multisection line identifier (input).	
Character STRING	String indicating the branch quantity desired (input).	
	'MVA'	Branch MVA flow
	'O_MVA'	Same as 'MVA' in units determined by the power output option setting
	'AMPS'	Branch current in amps
	'PUCUR'	Branch current in pu
	'CURANG'	Angle of branch current in degrees
	'PCTRTA'	Percent current loading of RATEA
	'PCTRTB'	Percent current loading of RATEB
	'PCTRTC'	Percent current loading of RATEC
	'PCTMVA'	Percent MVA loading of RATEA
	'PCTMVB'	Percent MVA loading of RATEB
	'PCTMVC'	Percent MVA loading of RATEC
	'PCTCPA'	Percent current or MVA loading (according to the appropriate percent loading units program option setting) of RATEA
	'PCTCPB'	Percent current or MVA loading (according to the appropriate percent loading units program option setting) of RATEB
	'PCTCPC'	Percent current or MVA loading (according to the appropriate percent loading units program option setting) of RATEC
	'P'	Branch MW flow at the IBUS end of the branch
	'O_P'	Same as 'P' in units determined by the power output option setting

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'Q'	Branch Mvar flow at the IBUS end of the branch
	'O_Q'	Same as 'Q' in units determined by the power output option setting
	'PLOS'	Branch active power losses in MW
	'O_PLOS'	Branch active power losses in units determined by the power output option setting
	'QLOS'	Branch reactive power losses in Mvar
	'O_QLOS'	Branch reactive power losses in units determined by the power output option setting
Real RVAL	Flow value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Branch not found; 'RVAL' unchanged
	IERR = 3	Branch out-of-service; 'RVAL' returned set to zero
	IERR = 4	No base kV at IBUS; 'RVAL' returned = per unit current
	IERR = 5	Invalid 'STRING'; 'RVAL' unchanged
	IERR = 6	Rating is zero; 'RVAL' returned set to zero

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.15. BRNMVA

This API routine is obsolete. It has been replaced by the API routine BRNMSC .

Use this API to return return branch MVA.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = brnmva(ibus,jbus,ickt)
```

Fortran command syntax:

```
CALL BRNMVA(IBUS, JBUS, ICKT, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNMVA(IBUS, JBUS, ICKT, RVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*2 ICKT	Circuit or multisection line identifier (input).	
Real RVAL	Flow value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Branch not found; 'RVAL' unchanged
	IERR = 3	Branch out-of-service; 'RVAL' returned set to zero

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.16. BRNNAM

Use this API to return the name of an AC branch.

This API routine was first introduced in release 34.3.0.

Python command syntax:

```
ierr, string = brnnam(ibus, jbus, ickt)
```

Fortran command syntax:

```
CALL BRNNAM(IBUS, JBUS, ICKT, STRING, IERR)
```

IPLAN command syntax:

```
CALL BRNNAM(IBUS, JBUS, ICKT, STRING, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character ICKT	Circuit identifier (input).	
Character*40 STRING	Branch name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Bus not found; 'STRING' unchanged
	IERR = 2	Branch not found; 'STRING' unchanged
	IERR = 4	'BRNNAM' invalid with multisection line identifier; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.17. BRNSTT

This API routine is obsolete. It has been replaced by the API routine BRNINT .

Return branch status.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = brnstt(ibus,jbus,ickt)
```

Fortran command syntax:

```
CALL BRNSTT(IBUS, JBUS, ICKT, IVAL, IERR)
```

IPLAN command syntax:

```
CALL BRNSTT(IBUS, JBUS, ICKT, IVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*2 ICKT	Circuit or multisection line identifier (input).	
Integer IVAL	Branch status (output).	
	IVAL = 0	out of service
	IVAL = 1	in-service
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Branch not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.18. BRN_RATING_NAME_DATA

Use this API to return branch rating name parameters.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, strout = brn_rating_name_data(brnratset, strin)
```

Fortran command syntax:

```
CALL BRN_RATING_NAME_DATA(BRNRATSET,STRIN,STROUT,IERR)
```

IPLAN command syntax:

```
CALL BRN_RATING_NAME_DATA(BRNRATSET,STRIN,STROUT,IERR)
```

Where:

Integer BRNRATSET	branch rating name set number (1 through 12) (input).	
Character STRIN	(input).	
	'NAME'	column heading (6 characters).
	'DESC'	description (32 characters).
Character*32 STROUT	character string (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'STROUT', returned
	IERR = 1	invalid branch rating name set number; 'STROUT' unchanged
	IERR = 2	bad value of 'STRIN'; 'STROUT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.19. BSYSISDEF

Use this API to check whether a bus subsystem has been defined for a given subsystem ID. A return value of FALSE means that the given subsystem ID is operating in ALL buses mode, e.g., all buses are operated on.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
exist = bsysisdef(sid)
```

Fortran command syntax:

```
CALL SBSBDEF(SID,EXIST)
```

IPLAN command syntax:

```
CALL BSYSISDEF(SID,EXIST)
```

Where:

Integer SID	a valid bus subsystem ID. Valid bus subsystem IDs range from 0 to 11 (input).	
Logical EXIST	function return (output).	
	EXIST = False	no subsystem defined, operating in ALL buses mode.
	EXIST = True	subsystem is defined.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.20. BUSDAT

Use this API to return real bus values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = busdat(ibus ,string)
```

Fortran command syntax:

```
CALL BUSDAT(IBUS, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSDAT(IBUS, STRING, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character STRING	String indicating the bus quantity desired (input).	
	'BASE'	Bus base voltage, in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV
	'ANGLE'	Bus voltage angle, in radians
	'ANGLED'	Bus voltage angle, in degrees
	'NVLMHI'	Normal high voltage limit, in pu
	'NVLMLO'	Normal low voltage limit, in pu
	'EVLMHI'	Emergency high voltage limit, in pu
	'EVLMLO'	Emergency low voltage limit, in pu
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Bad value of 'STRING'; 'RVAL' unchanged
	IERR = 3	Bus type code 4 or greater; 'RVAL' returned
	IERR = 4	No base voltage and 'STRING' = 'KV'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.21. BUSDT1

Use this API to return complex bus parameters as MVA. Values may be either nominal or actual.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = busdt1(ibus, string1, string2)
```

Fortran command syntax:

```
CALL BUSDT1(IBUS, STRING1, STRING2, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSDT1(IBUS, STRING1, STRING2, RVAL, IERR)
```

Note:

When STRNG1 is one of the following: 'MVA', 'IL', 'YL', 'TOTAL', 'SC_MVA', 'SC_IL', 'SC_YL', 'SC_TOTAL', 'FX_MVA', 'FX_IL', 'FX_YL', 'FX_TOTAL', the values returned are the net load values (i.e., value includes the load plus contribution from any in-service distributed generation on the load feeder at that bus)

Where:

Integer IBUS	Bus number (input).	
Character STRING1	String indicating the bus quantity desired (input).	
	'MVA'	Total in-service constant MVA load
	'IL'	Total in-service constant current load
	'YL'	Total in-service constant admittance load
	'TOTAL'	Total in-service load
	'SC_MVA'	Total in-service scalable constant MVA load
	'SC_IL'	Total in-service scalable constant current load
	'SC_YL'	Total in-service scalable constant admittance load
	'SC_TOTAL'	Total in-service scalable load
	'FX_MVA'	Total in-service fixed constant MVA load
	'FX_IL'	Total in-service fixed constant current load
	'FX_YL'	Total in-service fixed constant admittance load
	'FX_TOTAL'	Total in-service fixed load
	'YS'	Total in-service fixed bus shunt
	'YSZERO'	Total nominal in-service zero sequence fixed bus shunt
	'YSZ'	Total nominal in-service zero sequence fixed bus shunt in per unit
	'YSW'	In-service switched shunt
	'YSWZ'	In-service zero sequence switched shunt

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'SHUNTN'	Total in-service negative sequence load (pu nominal)
	'SHUNTZ'	Total in-service zero sequence load (pu nominal)
Character STRING2	String indicating the units to be used (ignored if STRING1 = 'YSZ' or 'YSWZ') (input).	
	'ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'RVAL' returned in MVA
	'O_ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'RVAL' in units determined by the power output option setting (units determined by power output options)
	'NOM'	Nominal load or shunt (at 1.0 pu voltage). 'RVAL' returned in MVA
	'O_NOM'	Nominal load or shunt (at 1.0 pu voltage). 'RVAL' in units determined by the power output option setting
Real RVAL	Magnitude of the complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Bad value of 'STRING1' or 'STRING2'; 'RVAL' unchanged
	IERR = 3	Bus type code 4 or greater; 'RVAL' returned
	IERR = 4	No loads at bus for 'STRING1' = 'MVA', 'IL', 'YL', or 'TOTAL'; 'RVAL' returned set to zero
	IERR = 5	No switched shunt at bus for 'STRING1' = 'YSW'; 'YSWZERO' or 'YSWZ'; 'RVAL' unchanged
	IERR = 6	Sequence data not in case for 'STRING1' = 'YSZERO', 'YSZ', 'YSWZERO', 'YSWZ', 'SHUNTN' or 'SHUNTZ'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.22. BUSDT2

Use this API to return complex bus parameters. Values may be either nominal or actual. Nominal values return negative values for reactors. Actual values show flow into reactor.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = busdt2(ibus, string1, string2)
```

Fortran command syntax:

```
CALL BUSDT2(IBUS, STRING1, STRING2, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSDT2(IBUS, STRING1, STRING2, P, Q, IERR)
```

Note:

When STRNG1 is one of the following: 'MVA', 'IL', 'YL', 'TOTAL', 'SC_MVA', 'SC_IL', 'SC_YL', 'SC_TOTAL', 'FX_MVA', 'FX_IL', 'FX_YL', 'FX_TOTAL', the values returned are the net load values (i.e., value includes load plus the contribution from any in-service distributed generation on the load feeder at that bus)

Where:

Integer IBUS	Bus number (input).	
Character STRING1	String indicating the bus quantity desired (input).	
	'MVA'	Total in-service constant MVA load
	'IL'	Total in-service constant current load
	'YL'	Total in-service constant admittance load
	'TOTAL'	Total in-service load
	'SC_MVA'	Total in-service scalable constant MVA load
	'SC_IL'	Total in-service scalable constant current load
	'SC_YL'	Total in-service scalable constant admittance load
	'SC_TOTAL'	Total in-service scalable load
	'FX_MVA'	Total in-service fixed constant MVA load
	'FX_IL'	Total in-service fixed constant current load
	'FX_YL'	Total in-service fixed constant admittance load
	'FX_TOTAL'	Total in-service fixed load
	'YS'	Total in-service fixed bus shunt
	'YSZERO'	Total nominal in-service zero sequence fixed bus shunt
	'YSZ'	Total nominal in-service zero sequence fixed bus shunt in per unit
	'YSW'	In-service switched shunt

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'YSWZERO'	Nominal in-service zero sequence switched shunt
	'YSWZ'	Nominal in-service zero sequence switched shunt in per unit
	'SHUNTN'	Total in-service negative sequence load (pu nominal)
	'SHUNTZ'	Total in-service zero sequence load (pu nominal)
Character STRING2	String indicating the units to be used (ignored if STRING1 = 'YSZ' or 'YSWZ') (input).	
	'ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'P' 'Q' returned in MW Mvar
	'O_ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'P' 'Q' in units determined by the power output option setting
	'NOM'	Nominal load (at 1.0 pu voltage). 'P' 'Q' returned in MW Mvar
	'O_NOM'	Nominal load (at 1.0 pu voltage). 'P' 'Q' in units determined by the power output option setting
Real P	Real component of the complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Real Q	Reactive component of the complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Complex CMPVAL	Complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bad value of 'STRING1' or 'STRING2'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Bus type code 4 or greater; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 4	No loads at bus for 'STRING1' = 'MVA', 'IL', 'YL', or 'TOTAL'; 'P' and 'Q' or 'CMPVAL' returned set to zero
	IERR = 5	No switched shunt at bus for 'STRING1' = 'YSW', 'YSWZERO' or 'YSWZ'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 6	Sequence data not in case for 'STRING1' = 'YSZERO', 'YSZ', 'YSWZERO', 'YSWZ', 'SHUNTN' or 'SHUNTZ'; 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.23. BUSEXS

Use this API to check for the existence of a specified bus.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = busexs(ibus)
```

Fortran command syntax:

```
CALL BUSEXS(IBUS, IERR)
```

IPLAN command syntax:

```
CALL BUSEXS(IBUS, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	Bus found
	IERR = 1	Bus not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.24. BUSINT

Use this API to return integer bus parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = busint(ibus ,string)
```

Fortran command syntax:

```
CALL BUSINX(IBUS, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSINT(IBUS, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character STRING	String indicating the bus quantity desired (input).	
	'STATION'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number
	'DUMMY'	Returns 1 (one) if bus is a dummy bus of a multisection line else 0 (zero)
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Bad value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.25. BUSMSM

Use this API to return complex bus mismatch.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = busmsm(ibus)
```

Fortran command syntax:

```
CALL BUSMSM(IBUS, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSMSM(IBUS, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Real P	Real component (MW) of the bus mismatch	
Real Q	Reactive component (Mvar) of the bus mismatch	
Complex CMPVAL	Complex bus mismatch (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bus type code greater than 4; 'P' and 'Q' or 'CMPVAL' returned set to zero

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.26. BUSORDPOS

Use this API to return the ordinal position of the specified bus or bus section in the numerically ordered list of non-star point buses (i.e., star point buses are excluded).

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, iord = busordpos(ibus, inode)
```

Fortran command syntax:

```
CALL BUSORDPOS(IBUS, INODE, IORD, IERR)
```

IPLAN command syntax:

```
CALL BUSORDPOS(IBUS, INODE, IORD, IERR)
```

Where:

Integer IBUS	bus number (input).	
Integer INODE	node number. The position in the collating sequence of the bus section to which node INODE is connected is returned. INODE must be specified as 0 if bus IBUS is not in a substation. INODE may be specified as 0 if bus IBUS is represented by a single bus section. If 0 is specified and bus IBUS is represented by multiple bus sections, the collating sequence position of the parent bus section of bus IBUS is returned. (input).	
Integer IORD	ordinal position; 1 through the number of non-star point buses in the working case. IORD is unchanged if a positive value of IERR is returned. (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'IORD' returned.
	IERR = 1	invalid bus number.
	IERR = 2	bus IBUS not found.
	IERR = 3	invalid node number.
	IERR = 4	bus IBUS does not belong to a substation.
	IERR = 5	node INODE is not in the substation of bus IBUS.
	IERR = 6	node INODE is not connected to any bus section of bus IBUS.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.27. BUSSECTDAT

Use this API routine to return real bus section values.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, rval = bussectdat(ibus, sect, string)
```

Fortran command syntax:

```
CALL BUSSECTDAT(IBUS, SECT, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSSECTDAT(IBUS, SECT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Character STRING	String indicating the bus quantity desired (input).	
	'BASE'	Bus base voltage, in kV
	'PU'	Bus voltage magnitude, in pu
	'KV'	Bus voltage magnitude, in kV
	'ANGLE'	Bus voltage angle, in radians
	'ANGLED'	Bus voltage angle, in degrees
	'NVLMHI'	Normal high voltage limit, in pu
	'NVLMLO'	Normal low voltage limit, in pu
	'EVLMHI'	Emergency high voltage limit, in pu
	'EVLMLO'	Emergency low voltage limit, in pu
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 3	Bus type code 4 or greater; 'RVAL' returned
	IERR = 4	No base voltage and 'STRING' = 'KV'; 'RVAL' unchanged
	IERR = 9	Bus section not found; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.28. BUSSECTDT1

Use this API routine to return complex bus section parameters as MVA. Values may be either nominal or actual.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, rval = bussectdt1(ibus, sect, string1, string2)
```

Fortran command syntax:

```
CALL BUSSECTDT1(IBUS, SECT, STRING1, STRING2, RVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSSECTDT1(IBUS, SECT, STRING1, STRING2, RVAL, IERR)
```

Note:

When STRNG1 is one of the following: 'MVA', 'IL', 'YL', 'TOTAL', 'SC_MVA', 'SC_IL', 'SC_YL', 'SC_TOTAL', 'FX_MVA', 'FX_IL', 'FX_YL', 'FX_TOTAL', the values returned are the net load values (i.e., value includes the load plus contribution from any in-service distributed generation on the load feeder at that bus).

Where:

Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Character STRING1	String indicating the bus quantity desired (input).	
	'MVA'	Net in-service constant MVA load
	'IL'	Net in-service constant current load
	'YL'	Net in-service constant admittance load
	'TOTAL'	Net in-service total load (total of constant MVA constant current and constant admittance portions)
	'LDGEN'	Total in-service distributed generation on the load feeder
	'SC_MVA'	Net in-service scalable constant MVA load
	'SC_IL'	Net in-service scalable constant current load
	'SC_YL'	Net in-service scalable constant admittance load
	'SC_TOTAL'	Net in-service scalable total load
	'FX_MVA'	Net in-service fixed constant MVA load
	'FX_IL'	Net in-service fixed constant current load
	'FX_YL'	Net in-service fixed constant admittance load
	'FX_TOTAL'	Net in-service fixed total load
	'YS'	Total in-service fixed bus shunt
	'YSZERO'	Total nominal in-service zero sequence fixed bus shunt

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'YSZ'	Total nominal in-service zero sequence fixed bus shunt in per unit
	'YSW'	In-service switched shunt
	'YSWZ'	In-service zero sequence switched shunt
	'SHUNTN'	Total in-service negative sequence load (pu nominal)
	'SHUNTZ'	Total in-service zero sequence load (pu nominal)
Character STRING2	String indicating the units to be used (ignored if STRING1 = 'YSZ' or 'YSWZ') (input).	
	'ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'RVAL' returned in MVA
	'O_ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'RVAL' in units determined by the power option setting
	'NOM'	Nominal load or shunt (at 1.0 pu voltage). 'RVAL' returned in MVA
	'O_NOM'	Nominal load or shunt (at 1.0 pu voltage). 'RVAL' in units determined by the power option setting
Real RVAL	Magnitude of the complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Bad value of 'STRING1' or 'STRING2'; 'RVAL' unchanged
	IERR = 3	Bus type code 4 or greater; 'RVAL' returned
	IERR = 4	No loads at bus for 'STRING1' = 'MVA', 'IL', 'YL', or 'TOTAL'; 'RVAL' returned set to zero
	IERR = 5	No switched shunt at bus for 'STRING1' = 'YSW'; 'YSWZERO' or 'YSWZ'; 'RVAL' unchanged
	IERR = 6	Sequence data not in case for 'STRING1' = 'YSZERO', 'YSZ', 'YSWZERO', 'YSWZ', 'SHUNTN' or 'SHUNTZ'; 'RVAL' unchanged
	IERR = 9	Bus section not found; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.29. BUSSECTDT2

Use this API routine to return complex bus section parameters. Values may be either nominal or actual. Nominal values return negative values for reactors. Actual values show flow into reactor.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, cmpval = bussectdt2(ibus, sect, string1, string2)
```

Fortran command syntax:

```
CALL BUSSECTDT2(IBUS, SECT, STRING1, STRING2, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSSECTDT2(IBUS, SECT, STRING1, STRING2, P, Q, IERR)
```

Note:

When STRNG1 is one of the following: 'MVA', 'IL', 'YL', 'TOTAL', 'SC_MVA', 'SC_IL', 'SC_YL', 'SC_TOTAL', 'FX_MVA', 'FX_IL', 'FX_YL', 'FX_TOTAL', the values returned are the net load values (i.e., value includes the load plus contribution from any in-service distributed generation on the load feeder at that bus).

Where:

Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Character STRING1	String indicating the bus quantity desired (input).	
	'MVA'	Net in-service constant MVA load
	'IL'	Net in-service constant current load
	'YL'	Net in-service constant admittance load
	'TOTAL'	Net in-service total load (total of constant MVA constant current and constant admittance portions)
	'LDGEN'	Total in-service distributed generation on the load feeder
	'SC_MVA'	Net in-service scalable constant MVA load
	'SC_IL'	Net in-service scalable constant current load
	'SC_YL'	Net in-service scalable constant admittance load
	'SC_TOTAL'	Net in-service scalable total load
	'FX_MVA'	Net in-service fixed constant MVA load
	'FX_IL'	Net in-service fixed constant current load
	'FX_YL'	Net in-service fixed constant admittance load
	'FX_TOTAL'	Net in-service fixed total load
	'YS'	Total in-service fixed bus shunt
	'YSZERO'	Total nominal in-service zero sequence fixed bus shunt

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'YSZ'	Total nominal in-service zero sequence fixed bus shunt in per unit
	'YSW'	In-service switched shunt
	'YSWZ'	In-service zero sequence switched shunt
	'SHUNTN'	Total in-service negative sequence load (pu nominal)
	'SHUNTZ'	Total in-service zero sequence load (pu nominal)
Character STRING2	String indicating the units to be used (ignored if STRING1 = 'YSZ' or 'YSWZ') (input).	
	'ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'RVAL' returned in MVA
	'O_ACT'	Actual load or shunt (nominal shunt for STRING1 = 'YSZERO' or 'YSWZERO'). 'RVAL' in units determined by the power option setting
	'NOM'	Nominal load or shunt (at 1.0 pu voltage). 'RVAL' returned in MVA
	'O_NOM'	Nominal load or shunt (at 1.0 pu voltage). 'RVAL' in units determined by the power option setting
Real P	Real component of the complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Real Q	Reactive component of the complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Complex CMPVAL	Complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bad value of 'STRING1' or 'STRING2'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Bus type code 4 or greater; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 4	No loads at bus for 'STRING1' = 'MVA', 'IL', 'YL', or 'TOTAL'; 'P' and 'Q' or 'CMPVAL' returned set to zero
	IERR = 5	No switched shunt at bus for 'STRING1' = 'YSW', 'YSWZERO' or 'YSWZ'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 6	Sequence data not in case for 'STRING1' = 'YSZERO', 'YSZ', 'YSWZERO', 'YSWZ', 'SHUNTN' or 'SHUNTZ'; 'CMPVAL' unchanged
	IERR = 9	Bus section not found; 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.30. BUSSECTEXS

Use this API routine to check for the existence of a specified bus section.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr = bussectexs(ibus, sect)
```

Fortran command syntax:

```
CALL BUSSECTEXS(IBUS, SECT, IERR)
```

IPLAN command syntax:

```
CALL BUSSECTEXS(IBUS, SECT, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 9	Bus section not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.31. BUSSECTINT

Use this API routine to return real bus section values.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, ival = bussectint(ibus, sect, string)
```

Fortran command syntax:

```
CALL BUSSECTINT(IBUS, SECT, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL BUSSECTINT(IBUS, SECT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Character STRING	String indicating the bus quantity desired (input).	
	'STATION'	Bus IBUS substation number (returned as zero if bus IBUS is not in a substation)
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number
	'DUMMY'	Returns 1 (one) if bus is a dummy bus of a multi-section line else 0 (zero)
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 9	Bus section not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.32. CCTMDLNAM_2DCO

Use this API to return model name of CCT 2-terminal dc line other models.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, model = cctmdlnam_2dco(name)
```

Fortran command syntax:

```
CALL CCTMDLNAM_2DCO (NAME, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_2DCO (NAME, MODEL, IERR)
```

Where:

Character*12 NAME	Two-terminal dc line name (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	dc line not in case
	IERR = 2	No CCT models in case
	IERR = 3	No 2-terminal dc other model attached to the specified dc line
	IERR = 4	Dynamics data not present in working memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.33. CCTMDLNAM_2WTD

Use this API to return model name of the CCT 2-winding transformer device model.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, model = cctmdlnam_2wtd(ibus, jbus, id)
```

Fortran command syntax:

```
CALL CCTMDLNAM_2WTD(IBUS, JBUS, ID, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_2WTD(IBUS, JBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the 2-winding transformer (input).	
Integer JBUS	"To" Bus number of the 2-winding transformer (input).	
Character*2 ID	2-winding transformer circuit identifier (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	2-winding transformer not in case
	IERR = 2	No CCT models in case
	IERR = 3	No device model attached to the specified 2-winding transformer
	IERR = 4	Dynamics data not present in working memory
	IERR = 5	Specified branch is not a transformer

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.34. CCTMDLNAM_3WTD

Use this API to return model name of the CCT 3-winding transformer device model.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, model = cctmdlnam_3wtd(ibus, jbus, kbus, id)
```

Fortran command syntax:

```
CALL CCTMDLNAM_3WTD(IBUS, JBUS, KBUS, ID, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_3WTD(IBUS, JBUS, KBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the branch (input).	
Integer JBUS	"To" Bus number of the branch (input).	
Integer KBUS	"Third" Bus number of the branch (input).	
Character*2 ID	3-winding transformer branch circuit identifier (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	branch not in case;
	IERR = 2	No CCT models in case
	IERR = 3	No device model attached to the specified 3-winding transformer
	IERR = 4	Dynamics data not present in working memory
	IERR = 5	Specified branch is a transformer

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.35. CCTMDLNAM_BRND

Use this API to return model name of the CCT branch device model.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, model = cctmdlnam_brnd(ibus, jbus, id)
```

Fortran command syntax:

```
CALL CCTMDLNAM_BRND(IBUS, JBUS, ID, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_BRND(IBUS, JBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the branch (input).	
Integer JBUS	"To" Bus number of the branch (input).	
Character*2 ID	Branch circuit identifier (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	branch not in case;
	IERR = 2	No CCT models in case
	IERR = 3	No branch device model attached to the specified branch
	IERR = 4	Dynamics data not present in working memory
	IERR = 5	Specified branch is a transformer

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.36. CCTMDLNAM_BRNO

Use this API to return model name of the CCT branch other model.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, model = cctmdlnam_brno(ibus, jbus, id)
```

Fortran command syntax:

```
CALL CCTMDLNAM_BRNO(IBUS, JBUS, ID, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_BRNO(IBUS, JBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the branch (input).	
Integer JBUS	"To" Bus number of the branch (input).	
Character*2 ID	Branch circuit identifier (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	branch not in case
	IERR = 2	No CCT models in case
	IERR = 3	No branch other model attached to the specified branch
	IERR = 4	Dynamics data not present in working memory
	IERR = 5	Specified branch is a transformer

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.37. CCTMDLNAM_BUSO

Use this API to return model name of CCT Bus other models.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, model = cctmdlnam_buso(ibus)
```

Fortran command syntax:

```
CALL CCTMDLNAM_BUSO(IBUS, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_BUSO(IBUS, MODEL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	bus not in case
	IERR = 2	No CCT models in case
	IERR = 3	No bus other model attached to the specified bus
	IERR = 4	Dynamics data not present in working memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.38. CCTMDLNAM_MCNO

Use this API to return model name of the CCT machine other models.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, model = cctmdlnam_mcno(ibus, id)
```

Fortran command syntax:

```
CALL CCTMDLNAM_MCNO(IBUS, ID, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_MCNO(IBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Machine identifier (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	machine not in case
	IERR = 2	No CCT models in case
	IERR = 3	No machine other model attached to the specified machine
	IERR = 4	Dynamics data not present in working memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.39. CCTMDLNAM_MCNP

Use this API to return model name of the CCT machine protection models.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, model = cctmdlnam_mcnp(ibus, id)
```

Fortran command syntax:

```
CALL CCTMDLNAM_MCNP(IBUS, ID, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_MCNP(IBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Machine identifier (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	machine not in case
	IERR = 2	No CCT models in case
	IERR = 3	No machine protection model attached to the specified machine
	IERR = 4	Dynamics data not present in working memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.40. CCTMDLNAM_MSCO

Use this API to return model name of the CCT Miscellaneous other model.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, model = cctmdlnam_msco(mins)
```

Fortran command syntax:

```
CALL CCTMDLNAM_MSCO(MINS, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_MSCO(MINS, MODEL, IERR)
```

Where:

Integer MINS	Model instance number of the model (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	Invalid MINS value
	IERR = 2	No CCT models in case
	IERR = 3	No model attached for the specified MINS
	IERR = 4	Dynamics data not present in working memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.41. CCTMDLNAM_SWSO_2

Use this API to return model name of the CCT switched shunt other type models.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, model = cctmdlnam_swso_2(ibus, id)
```

Fortran command syntax:

```
CALL CCTMDLNAM_SWSO_2(IBUS, ID, MODEL, IERR)
```

IPLAN command syntax:

```
CALL CCTMDLNAM_SWSO_2(IBUS, ID, MODEL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	switched shunt identifier (input).	
Character*16 MODEL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; MODEL name is returned
	IERR = 1	bus not in case; MODEL name is returned as blank
	IERR = 2	No CCT models in case
	IERR = 3	No switched shunt other type model attached to the specified switched shunte
	IERR = 4	Dynamics data not present in working memory
	IERR = 5	switched shunt not in case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.42. CCTMIND_2DCO

Use this API to return starting array indices and status of CCT 2-terminal dc line other models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_2dco(name, model, string)
```

Fortran command syntax:

```
CALL CCTMIND_2DCO (NAME, MODEL, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_2DCO (NAME, MODEL, STRING, IVAL, IERR)
```

Where:

Character*12 NAME	Two-terminal dc line name (input).	
Character*16 MODEL	2-terminal dc other Model name (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect dc line status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	Model name specified in 'MODEL' not attached to this dc line; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.43. CCTMIND_2WTD

Use this API to return starting array indices and status of CCT 2-winding transformer device models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_2wtd(ibus, jbus, id, string)
```

Fortran command syntax:

```
CALL CCTMIND_2WTD(IBUS, JBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_2WTD(IBUS, JBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the 2-winding transformer (input).	
Integer JBUS	"To" Bus number of the 2-winding transformer (input).	
Character*2 ID	2-winding transformer circuit identifier (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect 2-winding transformer status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	2-winding transformer not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	No device model attached to the specified 2-winding transformer; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	Specified branch is not a transformer; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.44. CCTMIND_3WTD

Use this API to return starting array indices and status of CCT 3-winding transformer device models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_3wtd(ibus, jbus, kbus, id, string)
```

Fortran command syntax:

```
CALL CCTMIND_3WTD(IBUS, JBUS, KBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_3WTD(IBUS, JBUS, KBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the 3-winding transformer (input).	
Integer JBUS	"To" Bus number of the 3-winding transformer (input).	
Integer KBUS	"Third" Bus number of the 3-winding transformer (input).	
Character*2 ID	2-winding transformer circuit identifier (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect 3-winding transformer status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	4-winding transformer not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	No device model attached to the specified 3-winding transformer; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged
IERR = 7	Specified branch is not a transformer; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.45. CCTMIND_BRND

Use this API to return starting array indices and status of CCT branch device models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_brnd(ibus, jbus, id, string)
```

Fortran command syntax:

```
CALL CCTMIND_BRND(IBUS, JBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_BRND(IBUS, JBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the branch (input).	
Integer JBUS	"To" Bus number of the branch (input).	
Character*2 ID	Branch circuit identifier (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect branch status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	branch not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	No device model attached to the specified branch; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged
	IERR = 7	Specified branch is a transformer; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.46. CCTMIND_BRNO

Use this API to return starting array indices and status of CCT branch other models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_brno(ibus, jbus, id, model, string)
```

Fortran command syntax:

```
CALL CCTMIND_BRNO(IBUS, JBUS, ID, MODEL, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_BRNO(IBUS, JBUS, ID, MODEL, STRING, IVAL, IERR)
```

Where:

Integer IBUS	"From" Bus number of the branch (input).	
Integer JBUS	"To" Bus number of the branch (input).	
Character*2 ID	Branch circuit identifier (input).	
Character*16 MODEL	Branch other Model name (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect branch status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	branch not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	Model name specified in 'MODEL' not attached to the specified branch; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	Branch is a transformer; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.47. CCTMIND_BUSO

Use this API to return starting array indices and status of CCT Bus other models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_buso(ibus, model, string)
```

Fortran command syntax:

```
CALL CCTMIND_BUSO(IBUS, MODEL, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_BUSO(IBUS, MODEL, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*16 MODEL	Bus other Model name (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect bus status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	bus not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	Model name specified in 'MODEL' not attached to this bus; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.48. CCTMIND_MCNO

Use this API to return starting array indices and status of CCT machine other models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_mcno(ibus, id, model, string)
```

Fortran command syntax:

```
CALL CCTMIND_MCNO(IBUS, ID, MODEL, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_MCNO(IBUS, ID, MODEL, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Machine identifier (input).	
Character*16 MODEL	Machine other Model name (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect machine status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	machine not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	Model name specified in 'MODEL' not attached to this machine; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.49. CCTMIND_MCNP

Use this API to return starting array indices and status of CCT machine protection models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_mcnp(ibus, id, model, string)
```

Fortran command syntax:

```
CALL CCTMIND_MCNP(IBUS, ID, MODEL, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_MCNP(IBUS, ID, MODEL, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Machine identifier (input).	
Character*16 MODEL	Machine protection Model name (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect machine status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	machine not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	Model name specified in 'MODEL' not attached to this machine; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.50. CCTMIND_MSCO

Use this API to return starting array indices and status of CCT Miscellaneous other models.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_msco(mins, string)
```

Fortran command syntax:

```
CALL CCTMIND_MSCO(MINS, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_MSCO(MINS, STRING, IVAL, IERR)
```

Where:

Integer MINS	Model instance number of the model (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	CON Starting CON index
	'STATE'	STATE Starting STATE index
	'VAR'	VAR Starting VAR index
	'ICON'	ICON Starting ICON index
	'NCON'	NCON Number of CON values used by the model
	'NSTATE'	NSTATE Number of STATE values used by the model
	'NVAR'	NVAR Number of VAR values used by the model
	'NICON'	NICON Number of ICON values used by the model
	'STATUS'	STATUS Bypassed flag: 0 = bypassed; 1 = not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Invalid MINS value; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	No model attached for the specified MINS; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.51. CCTMIND_SWSO

This API routine is obsolete. It has been replaced by the API routine cctmind_swso_2 .

In versions prior to 35.0, this API would return the starting array indices and status of CCT switched shunt other model types. Starting version 35.0, this API is being retained only for reasons of backward compatibility of python scripts.
Use of this API is not recommended anymore as this will always pick the first switched shunt at a bus.

This API routine was first introduced in release 33.9.0.

Python command syntax:

```
ierr, ival = cctmind_swso(ibus, model, string)
```

Fortran command syntax:

```
CALL CCTMIND_SWSO(IBUS, MODEL, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_SWSO(IBUS, MODEL, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number at which switched shunt is attached (input).	
Character*16 MODEL	Switched shunt other Model name (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect switched shunt status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	bus not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	Model name specified in 'MODEL' not attached to this bus; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged
IERR = 7	No switched shunt at the specified bus; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.52. CCTMIND_SWSO_2

Use this API to return starting array indices and status of CCT Switched Shunt other models.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, ival = cctmind_swso_2(ibus, id, model, string)
```

Fortran command syntax:

```
CALL CCTMIND_SWSO_2(IBUS, ID, MODEL, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL CCTMIND_SWSO_2(IBUS, ID, MODEL, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number at which switched shunt is attached (input).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 MODEL	Switched shunt other Model name (input).	
Character STRING	String indicating the model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect switched shunt status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	bus not in case; 'IVAL' returned as 0
	IERR = 2	No CCT models in case
	IERR = 3	Model name specified in 'MODEL' not attached to this bus; 'IVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 6	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	No switched shunt at the specified bus; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.53. CHKTRE

Use this API to check for Type 4 (or greater) buses with in-service branches connected to them. If none are found, it then counts number of ac islands not containing a Type 3 (swing) bus. It may be inappropriate to call this subroutine with generators converted; in this case, there are no Type 3 buses, so there is normally one or more swingless islands.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ival = chktre()
```

Fortran command syntax:

```
CALL CHKTRE(IVAL)
```

IPLAN command syntax:

```
CALL CHKTRE(IVAL)
```

Where:

Integer IVAL	Return code (output).	
	IVAL = 0	None of the following found.
	IVAL = -1	No in-service buses remaining.
	IVAL = -2	There are Type 4 buses with in-service branches connected to them.
	IVAL > 0	Number of islands without a Type 3 bus.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.54. CHNVAL

Use this API to return the present value of the simulation variable assigned to a specified output channel.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = chnval(n)
```

Fortran command syntax:

```
CALL CHNVAL(N, RVAL, IERR)
```

IPLAN command syntax:

```
CALL CHNVAL(N, RVAL, IERR)
```

Where:

Integer N	Channel number (input).	
Real RVAL	Channel value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Invalid channel index; 'RVAL' unchanged
	IERR = 2	Invalid channel address; 'RVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.55. DC2AUXMIND

Use this API to return the starting array indices and status of auxiliary signal model associated with 2-terminal dc line for the specified signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = dc2auxmind(name, isgndx, string)
```

Fortran command syntax:

```
CALL DC2AUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2AUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

Where:

Character*12 NAME	Two-terminal dc line name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character STRING	String indicating the 2-terminal dc line auxiliary signal model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. For a model to be called during simulations the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not in network but auxiliary signal model in model tables; 'IVAL' returned
	IERR = 2	dc line not in network and auxiliary signal model not in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.56. DC2AUXMNAM

Use this API to return the auxiliary signal model name associated with a 2-terminal dc line for the specified auxiliary signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = dc2auxmnam(name, isgndx)
```

Fortran command syntax:

```
CALL DC2AUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2AUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

Where:

Character NAME	Two-terminal dc line name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	dc line not in network but auxiliary signal model in model tables; 'CVAL' returned
	IERR = 2	dc line not in network and auxiliary signal model not in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.57. DC2DAT

This API routine is obsolete. It has been replaced by the API routine dc2dat_2 .

Use this API to return real two-terminal dc line quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = dc2dat(n, ibus, string)
```

Fortran command syntax:

```
CALL DC2DAT(N, IBUS, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2DAT(N, IBUS, STRING, RVAL, IERR)
```

Where:

Integer N	Two-terminal dc line number (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'DCCUR'	dc line current in amps (Line Quantity)
	'RDC'	dc line resistance in ohms (Line Quantity)
	'SETVAL'	Current (amps) or power (MW) demand (Line Quantity)
	'VSCHD'	Scheduled compounded dc voltage in kV (Line Quantity)
	'VCMOD'	Mode switch dc voltage in kV (Line Quantity)
	'RCOMP'	Compounding resistance in ohms (Line Quantity)
	'DELTI'	Margin in pu of desired dc power or current (Line Quantity)
	'DCVMIN'	Minimum compounded dc voltage in kV (Line Quantity)
	'PAC'	Apparent active power flow into the dc line in MW (Converter Quantity)
	'O_PAC'	Same as 'PAC' in units determined by the power output option setting (Converter Quantity)
	'QAC'	Apparent reactive power flow into the dc line in Mvar (Converter Quantity)
	'O_QAC'	Same as 'QAC' in units determined by the power output option setting (Converter Quantity)
	'MVA'	Apparent ac flow into the dc line in MVA (Converter Quantity)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_MVA'	Same as 'MVA' in units determined by the power output option setting (Converter Quantity)
	'ACAMPS'	ac current flow into the dc line in amps (Converter Quantity)
	'PUCUR'	ac current flow into the dc line in pu (Converter Quantity)
	'KVDC'	dc voltage in kV (Converter Quantity)
	'ANGLE'	Converter angle in degrees (Converter Quantity)
	'ANGMIN'	Minimum steady-state converter angle in degrees (Converter Quantity)
	'ANGMAX'	Nominal maximum converter angle in degrees (Converter Quantity)
	'RC'	Commutating transformer resistance per bridge in ohms (Converter Quantity)
	'XC'	Commutating transformer reactance per bridge in ohms (Converter Quantity)
	'EBASE'	Primary base ac voltage in kV (Converter Quantity)
	'TR'	Transformer ratio in pu (Converter Quantity)
	'TAP'	Tap setting in pu (Converter Quantity)
	'TPMX'	Maximum tap setting in pu (Converter Quantity)
	'TPMN'	Minimum tap setting in pu (Converter Quantity)
	'STP'	Tap step in pu (Converter Quantity)
	'XCAP'	Commutating capacitor reactance magnitude per bridge in ohms (Converter Quantity)
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	dc line not found; 'RVAL' unchanged
	IERR = 2	Bus not a converter bus of dc line 'N'; 'RVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 4	dc line blocked; 'RVAL' of 0.0 returned
	IERR = 5	No base kV at bus; 'RVAL' returned as the per unit current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.58. DC2DAT_2

This API is the second release of the API that returns real two-terminal dc line quantities.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, rval = dc2dat_2(name, ibus, string)
```

Fortran command syntax:

```
CALL DC2DAT_2(NAME, IBUS, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2DAT_2(NAME, IBUS, STRING, RVAL, IERR)
```

Where:

Character NAME	Two-terminal dc line name (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'DCCUR'	dc line current in amps (Line Quantity)
	'RDC'	dc line resistance in ohms (Line Quantity)
	'SETVAL'	Current (amps) or power (MW) demand (Line Quantity)
	'VSCHD'	Scheduled compounded dc voltage in kV (Line Quantity)
	'VCMOD'	Mode switch dc voltage in kV (Line Quantity)
	'RCOMP'	Compounding resistance in ohms (Line Quantity)
	'DELTI'	Margin in pu of desired dc power or current (Line Quantity)
	'DCVMIN'	Minimum compounded dc voltage in kV (Line Quantity)
	'PAC'	Apparent active power flow into the dc line in MW (Converter Quantity)
	'O_PAC'	Same as 'PAC' in units determined by the power output option setting (Converter Quantity)
	'QAC'	Apparent reactive power flow into the dc line in Mvar (Converter Quantity)
	'O_QAC'	Same as 'QAC' in units determined by the power output option setting (Converter Quantity)
	'MVA'	Apparent ac flow into the dc line in MVA (Converter Quantity)
	'O_MVA'	Same as 'MVA' in units determined by the power output option setting (Converter Quantity)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'ACAMPS'	ac current flow into the dc line in amps (Converter Quantity)
	'PUCUR'	ac current flow into the dc line in pu (Converter Quantity)
	'KVDC'	dc voltage in kV (Converter Quantity)
	'ANGLE'	Converter angle in degrees (Converter Quantity)
	'ANGMIN'	Minimum steady-state converter angle in degrees (Converter Quantity)
	'ANGMAX'	Nominal maximum converter angle in degrees (Converter Quantity)
	'RC'	Commutating transformer resistance per bridge in ohms (Converter Quantity)
	'XC'	Commutating transformer reactance per bridge in ohms (Converter Quantity)
	'EBASE'	Primary base ac voltage in kV (Converter Quantity)
	'TR'	Transformer ratio in pu (Converter Quantity)
	'TAP'	Tap setting in pu (Converter Quantity)
	'TPMX'	Maximum tap setting in pu (Converter Quantity)
	'TPMN'	Minimum tap setting in pu (Converter Quantity)
	'STP'	Tap step in pu (Converter Quantity)
	'XCAP'	Commutating capacitor reactance magnitude per bridge in ohms (Converter Quantity)
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	dc line not found; 'RVAL' unchanged
	IERR = 2	Bus not a converter bus of dc line 'NAME'; 'RVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 4	dc line blocked; 'RVAL' of 0.0 returned
	IERR = 5	No base kV at bus; 'RVAL' returned as the per unit current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.59. DC2INT

This API routine is obsolete. It has been replaced by the API routine dc2int_2 .

Use this API to return integer two-terminal dc line quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = dc2int(n, string)
```

Fortran command syntax:

```
CALL DC2INT(N, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2INT(N, STRING, IVAL, IERR)
```

Where:

Integer N	Two-terminal dc line number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'MDC'	dc line control mode
	'RECT'	Bus number of rectifier end of dc line
	'INV'	Bus number of inverter end of dc line
	'METER'	Bus number of metered end of dc line
	'NBR'	Number of bridges in series rectifier end
	'NBI'	Number of bridges in series inverter end
	'ICR'	Rectifier firing angle measuring bus
	'ICI'	Inverter firing angle measuring bus
	'NDR'	Rectifier firing angle measuring node
	'NDI'	Inverter firing angle measuring node
	'ANSTTR'	Firing angle status rectifier end: (-1) low; (0) regulating; (1) high
	'ANSTTI'	Firing angle status inverter end: (-1) low; (0) regulating (1) high
	'TPSTTR'	Tap status rectifier end: (-1) low; (0) regulating; (1) high; (-2) locked
	'TPSTTI'	Tap status inverter end: (-1) low; (0) regulating; (1) high; (-2) locked
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 2	dc line not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.60. DC2INT_2

This API is the second release of the API that returns integer two-terminal dc line quantities.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = dc2int_2(name, string)
```

Fortran command syntax:

```
CALL DC2INT_2(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2INT_2(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	Two-terminal dc line name (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'MDC'	dc line control mode
	'RECT'	Bus number of rectifier end of dc line
	'INV'	Bus number of inverter end of dc line
	'METER'	Bus number of metered end of dc line
	'NBR'	Number of bridges in series rectifier end
	'NBI'	Number of bridges in series inverter end
	'ICR'	Rectifier firing angle measuring bus
	'ICI'	Inverter firing angle measuring bus
	'NDR'	Rectifier firing angle measuring node
	'NDI'	Inverter firing angle measuring node
	'ANSTTR'	Firing angle status rectifier end: (-1) low; (0) regulating; (1) high
	'ANSTTI'	Firing angle status inverter end: (-1) low; (0) regulating (1) high
	'TPSTTR'	Tap status rectifier end: (-1) low; (0) regulating; (1) high; (-2) locked
	'TPSTTI'	Tap status inverter end: (-1) low; (0) regulating; (1) high; (-2) locked
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 2	dc line not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.61. DC2MIND

Use this API to return 2-terminal dc line model starting array indices and status.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = dc2mind(name, string)
```

Fortran command syntax:

```
CALL DC2MIND(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2MIND(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	Two-terminal dc line name (input).	
Character STRING	String indicating the 2-terminal dc line model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect dc line status. For a model to be called during simulations the dc line must be in service the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not in network but in model tables; 'IVAL' returned
	IERR = 2	dc line not in network or in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.62. DC2MNAM

Use this API to return 2-terminal dc line model name.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = dc2mnam(name)
```

Fortran command syntax:

```
CALL DC2MNAM(NAME, CVAL, IERR)
```

IPLAN command syntax:

```
CALL DC2MNAM(NAME, CVAL, IERR)
```

Where:

Character NAME	Two-terminal dc line name (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	dc line not in network but in model tables; 'CVAL' returned
	IERR = 2	dc line not in network or in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.63. DCNAUXMIND

Use this API to return the starting array indices and status of auxiliary signal model associated with N-terminal dc line for the specified signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = dcnauxmind(name, isgndx, string)
```

Fortran command syntax:

```
CALL DCNAUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNAUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

Where:

Character NAME	N-terminal dc line name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character STRING	String indicating the N-terminal dc line auxiliary signal model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. For a model to be called during simulations the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not in network but auxiliary signal model in model tables; 'IVAL' returned
	IERR = 2	dc line not in network and auxiliary signal model not in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.64. DCNAUXMNAM

Use this API to return the auxiliary signal model name associated with a N-terminal dc line for the specified auxiliary signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = dcnauxmnam(name, isgndx)
```

Fortran command syntax:

```
CALL DCNAUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNAUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

Where:

Character NAME	N-terminal dc line name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	dc line not in network but auxiliary signal model in model tables; 'CVAL' returned
	IERR = 2	dc line not in network and auxiliary signal model not in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.65. DCNCIN

This API routine is obsolete. It has been replaced by the API routine dcncin_2 .

Use this API to return integer multi-terminal dc line converter quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = dcncin(n, ibus, string)
```

Fortran command syntax:

```
CALL DCNCIN(N, IBUS, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNCIN(N, IBUS, STRING, IVAL, IERR)
```

Where:

Integer N	Multi-terminal dc line number (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'ANSTT'	Firing angle status: (-1) low; (0) regulating; (1) high
	'TPSTT'	Tap status: (-1) low; (0) regulating; (1) high; (-2) locked
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not found; 'IVAL' unchanged
	IERR = 2	Bus not a converter bus of dc line 'N'; 'IVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.66. DCNCIN_2

This API is the second release of the API that returns integer multi-terminal dc line converter quantities.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = dcncin_2(name, ibus, string)
```

Fortran command syntax:

```
CALL DCNCIN_2(NAME, IBUS, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNCIN_2(NAME, IBUS, STRING, IVAL, IERR)
```

Where:

Character NAME	Multi-terminal dc line name (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'ANSTT'	Firing angle status: (-1) low; (0) regulating; (1) high
	'TPSTT'	Tap status: (-1) low; (0) regulating; (1) high; (-2) locked
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not found; 'IVAL' unchanged
	IERR = 2	Bus not a converter bus of dc line 'NAME'; 'IVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.67. DCNDAT

This API routine is obsolete. It has been replaced by the API routine dcndat_2 .

Use this API to return real multi-terminal dc line quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = dcndat(n, ibus, string)
```

Fortran command syntax:

```
CALL DCNDAT(N, IBUS, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNDAT(N, IBUS, STRING, RVAL, IERR)
```

Where:

Integer N	Multi-terminal dc line number (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'VCMOD'	Mode switch dc voltage in kV (Line Quantity)
	'PAC'	Active power into dc line at converter bus in MW (Converter Quantity)
	'O_PAC'	Same as 'PAC' in units determined by the power output option setting (Converter Quantity)
	'QAC'	Reactive power into dc line at converter bus in Mvar (Converter Quantity)
	'O_QAC'	Same as 'QAC' in units determined by the power output option setting (Converter Quantity)
	'KVDC'	dc voltage at converter bus end (Converter Quantity)
	'DCCUR'	dc current (Converter Quantity)
	'SETVAL'	Converter setpoint (Converter Quantity)
	'ANGLE'	Converter angle (Converter Quantity)
	'TAP'	Tap setting (Converter Quantity)
	'ANGMIN'	Minimum steady-state ALPHA or GAMMA angle in degrees (Converter Quantity)
	'ANGMAX'	Nominal maximum ALPHA or GAMMA angle in degrees (Converter Quantity)
	'RC'	Commutating resistance per bridge in ohms (Converter Quantity)
	'XC'	Commutating reactance per bridge in ohms (Converter Quantity)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'MVA'	Apparent ac flow into the dc line in MVA (Converter Quantity)
	'O_MVA'	Same as 'MVA' in units determined by the power output option setting (Converter Quantity)
	'ACAMPS'	ac current flow into the dc line in amps (Converter Quantity)
	'PUCUR'	ac current flow into the dc line in pu (Converter Quantity)
	'EBASE'	Primary base ac voltage in kV (Converter Quantity)
	'TR'	Transformer ratio in pu (Converter Quantity)
	'TPMX'	Maximum tap setting in pu (Converter Quantity)
	'TPMN'	Minimum tap setting in pu (Converter Quantity)
	'STP'	Tap step in pu (Converter Quantity)
	'DELTI'	Rectifier margin in pu of desired dc power or current (Converter Quantity)
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	dc line not found; 'RVAL' unchanged
	IERR = 2	Bus not a converter bus of dc line 'N'; 'RVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 4	dc line blocked; 'RVAL' of 0.0 returned
	IERR = 5	No base kV at bus; 'RVAL' returned as the per unit current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.68. DCNDAT_2

This API is the second release of the API that returns real multi-terminal dc line quantities.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, rval = dcndat_2(name, ibus, string)
```

Fortran command syntax:

```
CALL DCNDAT_2(NAME, IBUS, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNDAT_2(NAME, IBUS, STRING, RVAL, IERR)
```

Where:

Character NAME	Multi-terminal dc line name (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'VCMOD'	Mode switch dc voltage in kV (Line Quantity)
	'PAC'	Active power into dc line at converter bus in MW (Converter Quantity)
	'O_PAC'	Same as 'PAC' in units determined by the power output option setting (Converter Quantity)
	'QAC'	Reactive power into dc line at converter bus in Mvar (Converter Quantity)
	'O_QAC'	Same as 'QAC' in units determined by the power output option setting (Converter Quantity)
	'KVDC'	dc voltage at converter bus end (Converter Quantity)
	'DCCUR'	dc current (Converter Quantity)
	'SETVAL'	Converter setpoint (Converter Quantity)
	'ANGLE'	Converter angle (Converter Quantity)
	'TAP'	Tap setting (Converter Quantity)
	'ANGMIN'	Minimum steady-state ALPHA or GAMMA angle in degrees (Converter Quantity)
	'ANGMAX'	Nominal maximum ALPHA or GAMMA angle in degrees (Converter Quantity)
	'RC'	Commutating resistance per bridge in ohms (Converter Quantity)
	'XC'	Commutating reactance per bridge in ohms (Converter Quantity)
	'MVA'	Apparent ac flow into the dc line in MVA (Converter Quantity)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_MVA'	Same as 'MVA' in units determined by the power output option setting (Converter Quantity)
	'ACAMPS'	ac current flow into the dc line in amps (Converter Quantity)
	'PUCUR'	ac current flow into the dc line in pu (Converter Quantity)
	'EBASE'	Primary base ac voltage in kV (Converter Quantity)
	'TR'	Transformer ratio in pu (Converter Quantity)
	'TPMX'	Maximum tap setting in pu (Converter Quantity)
	'TPMN'	Minimum tap setting in pu (Converter Quantity)
	'STP'	Tap step in pu (Converter Quantity)
	'DELTI'	Rectifier margin in pu of desired dc power or current (Converter Quantity)
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	dc line not found; 'RVAL' unchanged
	IERR = 2	Bus not a converter bus of dc line 'NAME'; 'RVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 4	dc line blocked; 'RVAL' of 0.0 returned
	IERR = 5	No base kV at bus; 'RVAL' returned as the per unit current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.69. DCNINT

This API routine is obsolete. It has been replaced by the API routine dcnint_2 .

Use this API to return integer multi-terminal dc line quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = dcnint(n, string)
```

Fortran command syntax:

```
CALL DCNINT(N, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNINT(N, STRING, IVAL, IERR)
```

Where:

Integer N	Multi-terminal dc line number (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'MDC'	Control mode
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 2	dc line not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.70. DCNINT_2

This API is the second release of the API that returns integer multi-terminal dc line quantities.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = dcnint_2(name, string)
```

Fortran command syntax:

```
CALL DCNINT_2(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNINT_2(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	Multi-terminal dc line name (input).	
Character STRING	String indicating the dc line quantity desired (input).	
	'MDC'	Control mode
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 2	dc line not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.71. DCNMIND

Use this API to return N-terminal dc line model starting array indices and status.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = dcnmind(name, string)
```

Fortran command syntax:

```
CALL DCNMIND(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNMIND(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	N-terminal dc line name (input).	
Character STRING	String indicating the N-terminal dc line model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect dc line status. For a model to be called during simulations the dc line must be in service the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not in network but in model tables; 'IVAL' returned
	IERR = 2	dc line not in network or in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.72. DCNMNAM

Use this API to return N-terminal dc line model name.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = dcnmnam(name)
```

Fortran command syntax:

```
CALL DCNMNAM(NAME, CVAL, IERR)
```

IPLAN command syntax:

```
CALL DCNMNAM(NAME, CVAL, IERR)
```

Where:

Character NAME	N-terminal dc line name (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	dc line not in network but in model tables; 'CVAL' returned
	IERR = 2	dc line not in network or in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.73. DSCVAL

Use this API to return dynamics character array values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = dscval(string, indx)
```

Fortran command syntax:

```
CALL DSCVAL(STRING, INDX, CVAL, IERR)
```

IPLAN command syntax:

```
CALL DSCVAL(STRING, INDX, CVAL, IERR)
```

Where:

Character STRING	String indicating the dynamics quantity desired (input).	
	'ICON'	Character model parameters
Integer INDX	Index to the dynamics character array (input).	
Character*12 CVAL	Character dynamics array value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Invalid value of 'STRING'; 'CVAL' unchanged
	IERR = 2	'INDX' invalid; 'CVAL' unchanged
	IERR = 3	'INDX' points to integer icon; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.74. DSIVAL

Use this API to return dynamics integer array values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = dsival(string, indx)
```

Fortran command syntax:

```
CALL DSIVAL(STRING, INDX, IVAL, IERR)
```

IPLAN command syntax:

```
CALL DSIVAL(STRING, INDX, IVAL, IERR)
```

Where:

Character STRING	String indicating the dynamics quantity desired (input).	
	'ICON'	Integer model parameters
Integer INDX	Index to the dynamics integer array (input).	
Integer IVAL	Integer dynamics array value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 2	'INDX' invalid; 'IVAL' unchanged
	IERR = 3	'INDX' points to character icon; 'IVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.75. DSRVAL

Use this API to return dynamics real values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = dsrval(string, indx)
```

Fortran command syntax:

```
CALL DSRVAL(STRING, INDX, RVAL, IERR)
```

IPLAN command syntax:

```
CALL DSRVAL(STRING, INDX, RVAL, IERR)
```

Where:

Character STRING	String indicating the dynamics quantity desired (input).	
	'TIME'	Simulation time
	'DELT'	Simulation time step
	'STATE'	State variable values
	'CON'	Real model parameters
	'VAR'	Real model variable values
Integer INDX	Index to dynamics array Ignored for 'TIME' and 'DELT' (input).	
Real RVAL	Real dynamics quantity or array value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 2	'INDX' invalid; 'RVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.76. FCDAUXMIND

Use this API to return the starting array indices and status of auxiliary signal model associated with FACTS device for the specified signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = fcdauxmind(name, isgndx, string)
```

Fortran command syntax:

```
CALL FCDAUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDAUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

Where:

Character NAME	FACTS device name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character STRING	String indicating the FACTS device auxiliary signal model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. For a model to be called during simulations the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	FACTS device not in network but auxiliary signal model in model tables; 'IVAL' returned
	IERR = 2	FACTS device not in network and auxiliary signal model not in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.77. FCDAUXMNAM

Use this API to return the auxiliary signal model name associated with a FACTS device for the specified auxiliary signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = fcdauxmnam(name, isgndx)
```

Fortran command syntax:

```
CALL FCDAUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDAUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

Where:

Character NAME	FACTS device name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	FACTS device not in network but auxiliary signal model in model tables; 'CVAL' returned
	IERR = 2	FACTS device not in network and auxiliary signal model not in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.78. FCDDAT

This API routine is obsolete. It has been replaced by the API routine fcddat_2 .

Use this API to return FACTS device real quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = fcddat(n, string)
```

Fortran command syntax:

```
CALL FCDDAT(N, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDDAT(N, STRING, RVAL, IERR)
```

Where:

Integer N	FACTS device number (input).	
Character STRING	String indicating the FACTS device quantity desired (input).	
	'PDES'	Desired active power flow at terminal end bus in MW
	'O_PDES'	Same as 'PDES' in units determined by the power output option setting
	'QDES'	Desired reactive power flow at terminal end bus in Mvar
	'O_QDES'	Same as 'QDES' in units determined by the power output option setting
	'VSET'	Voltage setpoint at sending end bus
	'SHMX'	Maximum shunt current at sending end bus in MVA at unity voltage
	'O_SHMX'	Same as 'SHMX' in units determined by the power output option setting
	'TRMX'	Maximum bridge real power transfer in MW
	'O_TRMX'	Same as 'TRMX' in units determined by the power output option setting
	'VTMN'	Minimum voltage at terminal end bus in pu
	'VTMX'	Maximum voltage at terminal end bus in pu
	'VSMX'	Maximum series voltage in pu
	'IMX'	Maximum series current in MVA at unity voltage
	'O_IMX'	Same as 'IMX' in units determined by the power output option setting
	'LINX'	Reactance of the dummy series element in pu
	'PSEND'	Active power into series element at sending end bus in MW

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'O_PSEND'	Same as 'PSEND' in units determined by the power output option setting
'QSEND'	Reactive power into series element at sending end bus in Mvar
'O_QSEND'	Same as 'QSEND' in units determined by the power output option setting
'PTERM'	Active power into series element at terminal end bus in MW
'O_PTERM'	Same as 'PTERM' in units determined by the power output option setting
'QTERM'	Reactive power into series element at terminal end bus in Mvar
'O_QTERM'	Same as 'QTERM' in units determined by the power output option setting
'PBRDG'	Active bridge power exchange in MW
'O_PBRDG'	Same as 'PBRDG' in units determined by the power output option setting
'QSHNT'	Reactive power into shunt element in Mvar
'O_QSHNT'	Same as 'QSHNT' in units determined by the power output option setting
'VSERES'	Series voltage magnitude
'MVASND'	Flow into series element at sending bus end in MVA
'O_MVASND'	Same as 'MVASND' in units determined by the power output option setting
'MVATRM'	Flow into series element at terminal bus end in MVA
'O_MVATRM'	Same as 'MVATRM' in units determined by the power output option setting
'MVASHN'	Flow into shunt element in MVA
'O_MVASHN'	Same as 'MVASHN' in units determined by the power output option setting
'IPUSND'	Current flow into series element at sending bus end in pu
'IPUTRM'	Current flow into series element at terminal bus end in pu
'IPUSHN'	Current flow into shunt element in pu
'AMPSND'	Current flow into series element at sending bus end in amps
'AMPTRM'	Current flow into series element at terminal bus end in amps
'AMPSHN'	Current flow into shunt element in amps
'PCTSND'	Percent series current loading at sending bus end
'PCTTRM'	Percent series current loading at terminal bus end

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PCTSHN'	Percent shunt current loading
	'PCTPBR'	Percent bridge real power loading
	'RMPCT'	Percent of contributed vars
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 2	FACTS device 'N' not found; 'RVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 4	No base kV at bus; 'RVAL' returned = per unit current
	IERR = 5	For quantities which are solution results, the FACTS device is blocked or out-of-service; "V" of 0.0 returned.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.79. FCDDAT_2

This API is the second release of the API that returns FACTS device real quantities.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, rval = fcddat_2(name, string)
```

Fortran command syntax:

```
CALL FCDDAT_2(NAME, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDDAT_2(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	FACTS device name (input).	
Character STRING	String indicating the FACTS device quantity desired (input).	
	'PDES'	Desired active power flow at terminal end bus in MW
	'O_PDES'	Same as 'PDES' in units determined by the power output option setting
	'QDES'	Desired reactive power flow at terminal end bus in Mvar
	'O_QDES'	Same as 'QDES' in units determined by the power output option setting
	'VSET'	Voltage setpoint at sending end bus
	'SHMX'	Maximum shunt current at sending end bus in MVA at unity voltage
	'O_SHMX'	Same as 'SHMX' in units determined by the power output option setting
	'TRMX'	Maximum bridge real power transfer in MW
	'O_TRMX'	Same as 'TRMX' in units determined by the power output option setting
	'VTMN'	Minimum voltage at terminal end bus in pu
	'VTMX'	Maximum voltage at terminal end bus in pu
	'VSMX'	Maximum series voltage in pu
	'IMX'	Maximum series current in MVA at unity voltage
	'O_IMX'	Same as 'IMX' in units determined by the power output option setting
	'LINX'	Reactance of the dummy series element in pu
	'PSEND'	Active power into series element at sending end bus in MW
	'O_PSEND'	Same as 'PSEND' in units determined by the power output option setting

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'QSEND'	Reactive power into series element at sending end bus in Mvar
'O_QSEND'	Same as 'QSEND' in units determined by the power output option setting
'PTERM'	Active power into series element at terminal end bus in MW
'O_PTERM'	Same as 'PTERM' in units determined by the power output option setting
'QTERM'	Reactive power into series element at terminal end bus in Mvar
'O_QTERM'	Same as 'QTERM' in units determined by the power output option setting
'PBRDG'	Active bridge power exchange in MW
'O_PBRDG'	Same as 'PBRDG' in units determined by the power output option setting
'QSHNT'	Reactive power into shunt element in Mvar
'O_QSHNT'	Same as 'QSHNT' in units determined by the power output option setting
'VSERES'	Series voltage magnitude
'MVASND'	Flow into series element at sending bus end in MVA
'O_MVASND'	Same as 'MVASND' in units determined by the power output option setting
'MVATRM'	Flow into series element at terminal bus end in MVA
'O_MVATRM'	Same as 'MVATRM' in units determined by the power output option setting
'MVASHN'	Flow into shunt element in MVA
'O_MVASHN'	Same as 'MVASHN' in units determined by the power output option setting
'IPUSND'	Current flow into series element at sending bus end in pu
'IPUTRM'	Current flow into series element at terminal bus end in pu
'IPUSHN'	Current flow into shunt element in pu
'AMPSND'	Current flow into series element at sending bus end in amps
'AMPTRM'	Current flow into series element at terminal bus end in amps
'AMPSHN'	Current flow into shunt element in amps
'PCTSND'	Percent series current loading at sending bus end
'PCTTRM'	Percent series current loading at terminal bus end
'PCTSHN'	Percent shunt current loading
'PCTPBR'	Percent bridge real power loading

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'RMPCT'	Percent of contributed vars
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 2	FACTS device 'NAME' not found; 'RVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 4	No base kV at bus; 'RVAL' returned = per unit current
	IERR = 5	For quantities which are solution results, the FACTS is blocked or out-of-service; "V" of 0.0 returned.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.80. FCDINT

This API routine is obsolete. It has been replaced by the API routine fcdint_2 .

Use this API to return FACTS device integer quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = fcdint(n, string)
```

Fortran command syntax:

```
CALL FCDINT(N, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDINT(N, STRING, IVAL, IERR)
```

Where:

Integer N	FACTS device number (input).	
Character STRING	String indicating the FACTS device quantity desired (input).	
	'SEND'	Sending end bus number
	'TERM'	Terminal end bus number
	'REMOTE'	Bus number of remote bus or zero for local control
	'OWNER'	Owner number
	'MODE'	Control mode (negative for blocked during solution)
	'STATVT'	Terminal end voltage condition code returns: 0 Terminal end voltage not constrained. 1 Holding terminal end voltage high limit. -1 Holding terminal end voltage low limit
	'STATVS'	Series voltage limit code returns. 0 Series voltage not constrained. 1 Holding series voltage limit
	'STATBR'	Bridge active power transfer limit code returns. 0 Bridge real power transfer not constrained. 1 Holding bridge real power transfer limit
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 2	FACTS device 'N' not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.81. FCDINT_2

This API is the second release of the API that returns FACTS device integer quantities.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = fcdint_2(name, string)
```

Fortran command syntax:

```
CALL FCDINT_2(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDINT_2(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	FACTS device name (input).	
Character STRING	String indicating the FACTS device quantity desired (input).	
	'SEND'	Sending end bus number
	'TERM'	Terminal end bus number
	'REMOTE'	Bus number of regulated bus
	'IREG'	Bus number of regulated bus
	'NREG'	Node number of regulated node
	'OWNER'	Owner number
	'MODE'	Control mode (negative for blocked during solution)
	'STATVT'	Terminal end voltage condition code returns: 0 Terminal end voltage not constrained. 1 Holding terminal end voltage high limit. -1 Holding terminal end voltage low limit
	'STATVS'	Series voltage limit code returns: 0 Series voltage not constrained. 1 Holding series voltage limit
	'STATBR'	Bridge active power transfer limit code returns: 0 Bridge real power transfer not constrained. 1 Holding bridge real power transfer limit
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 2	FACTS device 'NAME' not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.82. FCDMIND

Use this API to return FACTS device model starting array indices and status.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = fcdmind(name, string)
```

Fortran command syntax:

```
CALL FCDMIND(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDMIND(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	FACTS device name (input).	
Character STRING	String indicating the FACTS device model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect status of FACTS device. For a model to be called during simulations the FACTS device must be in service the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	FACTS device not in network but in model tables; 'IVAL' returned
	IERR = 2	FACTS device not in network or in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.83. FCDMNAM

Use this API to return FACTS device model name.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = fcdmnam(name)
```

Fortran command syntax:

```
CALL FCDMNAM(NAME, CVAL, IERR)
```

IPLAN command syntax:

```
CALL FCDMNAM(NAME, CVAL, IERR)
```

Where:

Character NAME	FACTS device name (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	FACTS device not in network but in model tables; 'CVAL' returned
	IERR = 2	FACTS device not in network or in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.84. FINDMODELPROGRAM

Look for a particular program name in the loaded dynamics model libraries and return the name of the library where it is found.
Note that if the program name is in more than one library only the first library found will be returned. The libraries will be searched in the order that they were loaded.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr,libname = findmodelprogram(prgname,args)
```

Fortran command syntax:

```
CALL FINDMODELPROGRAM(PRGNAME,ARGS,LIBNAME,IERR)
```

Where:

Character*260 PRGNAME	(input).	
Integer ARGS	number of arguments for the program. (input).	
Character*260 LIBNAME	name of the library where the program was found. (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LIBNAME' returned
	IERR = 1	Cannot load program
	IERR = 3	Allocation failed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.85. FXSBUS

Use this API to return the shunt bus and shunt id by shunt name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, ibus, id = fxsbus(name)
```

Fortran command syntax:

```
CALL FXSBUS(NAME, IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL FXSBUS(NAME, IBUS, ID, IERR)
```

Where:

Character*40 NAME	String indicating the shunt name (input).	
Integer IBUS	Bus number (output).	
Character*2 ID	shunt identifier (output; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error; shunt bus and 'ID' returned
	IERR = 1	Shunt not found
	IERR = 2	Shunt name is empty
	IERR = 3	Bus type code is not 1, 2 or 3; shunt bus and 'ID' returned
	IERR = 4	Shunt out-of-service; shunt bus and 'ID' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.86. FXSDT1

Use this API to return the magnitude of a specified fixed bus shunt.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, rval = fxsdt1(ibus, id ,string)
```

Fortran command syntax:

```
CALL FXSDT1(IBUS, ID, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL FXSDT1(IBUS, ID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Shunt identifier (input).	
Character STRING	String indicating the units to be used (input).	
	'ACT'	Actual shunt. 'RVAL' returned in MVA
	'O_ACT'	Actual shunt. 'RVAL' in units determined by the power output option setting
	'NOM'	Nominal shunt (at 1.0 pu voltage). 'RVAL' returned in MVA
	'O_NOM'	Nominal shunt (at 1.0 pu voltage). 'RVAL' in units determined by the power output option setting
	'PQZERO'	Nominal zero sequence shunt (at 1.0 pu voltage). 'RVAL' returned in per unit
	'PQZ'	Nominal zero sequence shunt (at 1.0 pu voltage). 'RVAL' returned in MVA
	'O_PQZ'	Nominal zero sequence shunt (at 1.0 pu voltage). 'RVAL' returned in units determined by the power output option setting
Real RVAL	Magnitude of the complex shunt in the units indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Shunt not found; 'RVAL' unchanged
	IERR = 3	Bus type code is not 1, 2 or 3; 'RVAL' returned
	IERR = 4	Shunt out-of-service; 'RVAL' returned
	IERR = 5	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 6	Sequence data not in case for 'STRING1' = 'PQZERO', 'PQZ' or 'O_PQZ'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.87. FXSDT2

Use this API to return complex fixed bus shunt.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cmpval = fxsdt2(ibus, id ,string)
```

Fortran command syntax:

```
CALL FXSDT2(IBUS, ID, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL FXSDT2(IBUS, ID, STRING, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Shunt identifier (input).	
Character STRING	String indicating the units to be used (input).	
	'ACT'	Actual shunt. 'CMPVAL' returned in MW Mvar
	'O_ACT'	Actual shunt. 'CMPVAL' in units determined by the power output option setting
	'NOM'	Nominal shunt (at 1.0 pu voltage). 'CMPVAL' returned in MW Mvar
	'O_NOM'	Nominal shunt (at 1.0 pu voltage). 'CMPVAL' in units determined by the power output option setting
	'PQZERO'	Nominal zero sequence shunt (at 1.0 pu voltage). 'CMPVAL' returned in per unit
	'PQZ'	Nominal zero sequence shunt (at 1.0 pu voltage). 'CMPVAL' returned in MW Mvar
	'O_PQZ'	Nominal zero sequence shunt (at 1.0 pu voltage). 'CMPVAL' returned in units determined by the power output option setting
Real P	Active power component of the complex shunt in the units indicated by STRING (output).	
Real Q	Reactive power component of the complex shunt in the units indicated by STRING (output).	
Complex CMPVAL	Complex shunt in the units indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Shunt not found; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	Bus type code is not 1, 2 or 3; 'P' and 'Q' or 'CMPVAL' returned
IERR = 4	Shunt out-of-service; 'P' and 'Q' or 'CMPVAL' returned
IERR = 5	Invalid value of 'STRING'; 'P' and 'Q' or 'CMP-VAL' unchanged
IERR = 6	Sequence data not in case for 'STRING1' = 'PQZERO', 'PQZ' or 'O_PQZ'; 'P' and 'Q' or 'CMP-VAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.88. FXSINT

Use this API to return integer quantities of the specified fixed bus shunt.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = fxsint(ibus, id, string)
```

Fortran command syntax:

```
CALL FXSINT(IBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL FXSINT(IBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Shunt identifier (input).	
Character STRING	String indicating the shunt quantity desired (input).	
	'STATION'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
	'SECTION'	Bus IBUS bus section number (returned as 0 if bus IBUS is not in a substation).
	'NODE'	Number of the node to which the fixed shunt is connected (returned as 0 if bus IBUS is not in a substation)
	'STATUS'	Shunt status; 1 (one) for in-service 0 (zero) for out-of-service
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Shunt not found; 'IVAL' unchanged
	IERR = 3	Bus type code is not 1, 2 or 3; 'IVAL' returned
	IERR = 4	Shunt out-of-service; 'IVAL' returned
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.89. FXSNAME

Use this API to return name of the specified fixed bus shunt.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, string = fxsname(ibus, id)
```

Fortran command syntax:

```
CALL FXSNAME(IBUS, ID, STRING, IERR)
```

IPLAN command syntax:

```
CALL FXSNAME(IBUS, ID, STRING, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Shunt identifier (input).	
Character*40 STRING	String indicating the load name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Shunt not found; 'IVAL' unchanged
	IERR = 3	Bus type code is not 1, 2 or 3; 'IVAL' returned
	IERR = 4	Shunt out-of-service; 'IVAL' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.90. GENCNV

Use this API to return a flag indicating whether generators are converted.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
icode = gencnv()
```

Fortran command syntax:

```
CALL GENCNV(ICODE)
```

IPLAN command syntax:

```
CALL GENCNV(ICODE)
```

Where:

Integer ICODE	0 if generators are converted, otherwise 1 Stand-alone simulator returns a 1 (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.91. GENDAT

Use this API to return plant total power output.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = gendat(ibus)
```

Fortran command syntax:

```
CALL GENDAT(IBUS, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL GENDAT(IBUS, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Real P	Real component of the complex generation (output).	
Real Q	Reactive component of the complex generation (output).	
Complex CMPVAL	Complex generation (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bus type code is not 2 or 3; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 3	No machines at bus; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	No online machines; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.92. GENDT1

Returns total plant power output in MVA.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = gendt1(ibus)
```

Fortran command syntax:

```
CALL GENDT1(IBUS, RVAL, IERR)
```

IPLAN command syntax:

```
CALL GENDT1(IBUS, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Real RVAL	Magnitude of total plant generation (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Bus type code is not 2 or 3; 'RVAL' returned
	IERR = 3	No machines at bus; 'RVAL' unchanged
	IERR = 4	No online machines; 'RVAL' of 0.0 returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.93. GENSECTDAT

Use this API routine to return a bus section's plant total power output.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, cmpval = gensectdat(ibus, sect)
```

Fortran command syntax:

```
CALL GENSECTDAT(IBUS, SECT, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL GENSECTDAT(IBUS, SECT, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Real P	Real component of the complex generation (output).	
Real Q	Reactive component of the complex generation (output).	
Complex CMPVAL	Complex generation (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bus type code is not 2 or 3; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 3	No machines at bus; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	No online machines; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned
	IERR = 9	Bus section not found; 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.94. GENSECTDT1

Use this API routine to return a bus section's total plant power output in MVA.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, rval = gensectdt1(ibus, sect)
```

Fortran command syntax:

```
CALL GENSECTDT1(IBUS, SECT, RVAL, IERR)
```

IPLAN command syntax:

```
CALL GENSECTDT1(IBUS, SECT, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Real RVAL	Magnitude of total plant generation (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Bus type code is not 2 or 3; 'RVAL' returned
	IERR = 3	No machines at bus; 'RVAL' unchanged
	IERR = 4	No online machines; 'RVAL' of 0.0 returned
	IERR = 9	Bus section not found; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.95. GETHOMEPATH

Use this API to retrieve a user's HOMEPATH directory.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
sfile = gethomepath()
```

Fortran command syntax:

```
CALL GETHOMPTH(SFILE)
```

IPLAN command syntax:

```
CALL GETHOMEPATH(SFILE)
```

Where:

Character*260 SFILE	user's HOMEPATH directory (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.96. GET_MSTATE

Use this API to return the MSTATE value. MSTATE is meaningful in MODEs one through four and indicates the type of simulation being run by activities STRT/RUN, MSTR/MRUN, ESTR/ERUN, GSTR/GRUN or ASTR.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
n = get_mstate()
```

Fortran command syntax:

```
CALL GET_MSTATE(N)
```

IPLAN command syntax:

```
CALL GET_MSTATE(N)
```

Where:

Integer N	MSTATE value (output).	
	N = -1	No dynamics data is present.
	N = 0	Standard state-space dynamic simulation via activities STRT and RUN, or activity ASTR.
	N = 1	Excitation system response ratio test via activities ESTR and ERUN. Models other than generator and excitation system models are not called.
	N = 2	Excitation system open circuit step response test via activities ESTR and ERUN. Models other than generator and excitation system models are not called.
	N = 3	Governor response test via activities GSTR and GRUN. Models other than generator and turbine governor models are not called.
	N = 4	Extended term dynamic simulation via activities MSTR and MRUN.
	N = 5	Dynamics data is present but no initialization activity has been successfully executed.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.97. GNECHR

Use this API routine to return character quantities of a specified GNE device.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, cval = gnechr(name, string)
```

Fortran command syntax:

```
CALL GNECHR(NAME, STRING, CVAL, IERR)
```

IPLAN command syntax:

```
CALL GNECHR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	GNE device name (input).	
Character STRING	String indicating the GNE device quantity desired (input).	
	'MODEL'	Name of the BOSL model used by this GNE device
	'CHARn'	nth two-character identifier required by this model; n must be between 1 and the number of character identifiers required by the model to calculate its inputs
Character*16 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	GNE device not found; 'CVAL' unchanged
	IERR = 2	Invalid value of 'STRING'; 'CVAL' unchanged
	IERR = 3	For 'STRING' value 'CHARn', n is not valid for this model; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.98. GNEDAT

Use this API routine to return real quantities of a specified GNE device.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, rval = gnedat(name, string)
```

Fortran command syntax:

```
CALL GNEDAT(NAME, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL GNEDAT(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	GNE device name (input).	
Character STRING	String indicating the GNE device quantity desired (input).	
	'REALn'	nth real data item required by this model; n must be between 1 and the number of real data items used by the model
	'Pn'	active power in MW flowing into the GNE device at terminal n; n must be between 1 and the number of terminals used by the model
	'Qn'	reactive power in MVAR flowing into the GNE device at terminal n; n must be between 1 and the number of terminals used by the model
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	GNE device not found; 'RVAL' unchanged
	IERR = 2	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 3	For 'STRING' value 'REALn', n is not valid for this model; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.99. GNEINT

Use this API routine to return integer quantities of a specified GNE device.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, ival = gneint(name, string)
```

Fortran command syntax:

```
CALL GNEINT(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL GNEINT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	GNE device name (input).	
Character STRING	String indicating the GNE device quantity desired (input).	
	'NTERM'	Number of terminals used by this model (1 or 2)
	'BUS1'	Number of the first (or only) bus to which this model is connected
	'BUS2'	Number of the second bus to which this model is connected; 0 is returned for a 1 terminal model
	'STATUS'	Device status
	'OWNER'	Owner to which the device is assigned
	'NMETER'	Number of the non-metered end bus; used for GNE devices with more than one terminal
	'NREAL'	Number of floating point data items required by this model
	'NINTG'	Number of buses required by the model to calculate its inputs
	'NCHAR'	Number of number of two-character identifiers required by the model to calculate its inputs
	'INTGn'	nth bus number required by this model; n must be between 1 and the number of buses required by the model to calculate its inputs
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	GNE device not found; 'IVAL' unchanged
	IERR = 2	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 3	For 'STRING' value 'INTGn', n is not valid for this model; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.100. INDBUS

Use this API to return the induction machine bus and id by induction machine name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, ibus, id = indbus(name)
```

Fortran command syntax:

```
CALL INDBUS(NAME, IBUS, ID, IERR)
```

Where:

Character*40 NAME	String indicating the induction machine name (input).	
Integer IBUS	Bus number (output).	
Character*2 ID	induction machine identifier (output; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error; induction machine bus and 'ID' returned
	IERR = 1	Induction machine not found
	IERR = 2	Induction machine name is empty
	IERR = 3	Bus type code is not 1, 2 or 3; induction machine bus and 'ID' returned
	IERR = 4	induction machine out-of-service; induction machine bus and 'ID' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.101. INDDT1

Use this API routine to return real induction machine quantities.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, rval = inddt1(ibus, id, string)
```

Fortran command syntax:

```
CALL INDDT1(IBUS, ID, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL INDDT1(IBUS, ID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Induction machine identifier (input).	
Character STRING	String indicating the induction machine quantity desired (input).	
	'MBASE'	Machine power base
	'RATEKV'	Machine rated voltage
	'PSET'	Power setpoint (positive for a motor)
	'RA'	Armature resistance
	'XA'	Armature leakage reactance
	'R1'	First cage resistance
	'X1'	First cage reactance
	'R2'	Second cage resistance
	'X2'	Second cage reactance
	'X3'	Third reactance
	'E1'	Main saturation parameter
	'SE1'	Main saturation parameter
	'E2'	Main saturation parameter
	'SE2'	Main saturation parameter
	'IA1'	Leakage saturation parameter
	'IA2'	Leakage saturation parameter
	'XAMULT'	Leakage saturation parameter
	'TRQA'	Load torque variation constant
	'TRQB'	Load torque variation constant
	'TRQD'	Load torque variation constant
	'TRQE'	Load torque variation constant
	'H'	Machine inertia constant
	'IRATIO'	Ratio of locked rotor current to rated current
	'ROVERX'	Ratio of Rpos to Xpos

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'RZERO'	Machine zero sequence fault analysis resistance
	'XZERO'	Machine zero sequence fault analysis reactance
	'RGRND'	Machine zero sequence grounding resistance
	'XGRND'	Machine zero sequence grounding reactance
	'P'	Active power flowing from the bus to the machine in MW
	'O_P'	Same as 'P' in units determined by the power output option setting
	'Q'	Reactive power flowing from the bus to the machine in Mvar
	'O_Q'	Same as 'Q' in units determined by the power output option setting
	'MVA'	Complex power flowing from the bus to the machine in MVA
	'O_MVA'	Same as 'MVA' in units determined by the power output option setting
	'SLIP'	Machine slip
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Induction machine not found; 'RVAL' unchanged
	IERR = 3	Bus type code is not 2 or 3; 'RVAL' returned
	IERR = 4	Induction machine is out-of-service; 'RVAL' returned
	IERR = 5	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 6	Sequence data not in case (when STRING = 'IRATIO', 'ROVERX', 'RZERO', 'XZERO', 'RGRND' or 'XGRND')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.102. INDDT2

Use this API routine to return complex induction machine quantities.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, cmpval = inddt2(ibus, id, string)
```

Fortran command syntax:

```
CALL INDDT2(IBUS, ID, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL INDDT2(IBUS, ID, STRING, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Induction machine identifier (input).	
Character STRING	String indicating the induction machine quantity desired (input).	
	'ZA'	Armature impedance (Ra + j Xa)
	'Z1'	First cage impedance (R1 + j X1)
	'Z2'	Second cage impedance (R2 + j X2)
	'ZZERO'	Machine zero sequence fault analysis impedance
	'ZGRND'	Machine zero sequence grounding impedance
	'PQ'	Actual power flowing from the bus to the machine in MW Mvar
	'O_PQ'	Same as 'PQ' in units determined by the power output option setting
Real P	Real component of the complex value indicated by STRING	
Real Q	Reactive component of the complex value indicated by STRING	
Complex CMPVAL	Complex value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Induction machine not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Bus type code is not 2 or 3; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 4	Induction machine is out-of-service; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 5	Invalid value of 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Sequence data not in case (when STRING = 'ZZERO' or ZGRND')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.103. INDINT

Use this API routine to return integer induction machine quantities.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, ival = indint(ibus, id, string)
```

Fortran command syntax:

```
CALL INDINT(IBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL INDINT(IBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Induction machine identifier (input).	
Character STRING	String indicating the induction machine quantity desired (input).	
	'STATUS'	Machine status; 1 (one) for in-service 0 (zero) for out-of-service
	'SCODE'	Machine standard type; 1 for NEMA 2 for IEC)
	'DCODE'	Machine design code; 0 for custom 1 through 5 for NEMA designs A through E
	'AREA'	Area number of its area
	'ZONE'	Zone number of its zone
	'OWNER'	Owner number of its owner
	'TCODE'	Type of mechanical torque variation; 1 for a simple power law 2 for the WECC model)
	'BCODE'	Base power code; 1 for mechanical power (MW) output of the machine 2 for apparent electrical power (MVA) drawn by the machine
	'PCODE'	Scheduled power code; 1 for mechanical power (MW) output of the machine 2 for electrical active power (MW) drawn by the machine
	'CZG'	Grounding impedance data input/outout (I/O) code (1 for per unit 2 for ohms)
	'CGR'	Grounding code (0 for an ungrounded machine 1 for a grounded machine)
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Induction machine not found; 'IVAL' unchanged
IERR = 3	Bus type code is not 2 or 3; 'IVAL' returned
IERR = 4	Induction machine is out-of-service, 'IVAL' returned
IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
IERR = 6	Sequence data not in case (when STRING = 'CZG' or 'CGR')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.104. INI2DC

Initializes the two-terminal dc line fetching routine 'NXT2DC' for retrieving two-terminal dc lines in dc line name alphabetical order.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr = ini2dc()
```

Fortran command syntax:

```
CALL INI2DC(IERR)
```

IPLAN command syntax:

```
CALL INI2DC(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error, 'NXT2DC' is properly initialized
	IERR = 1	No two-terminal dc lines in case; 'NXT2DC' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.105. INIBRN

Initializes the branch fetching routine 'NXTBRN' or 'NXTBRN3' for returning branches connected to IBUS. For multisection lines, 'NXTBRN' or 'NXTBRN3' returns the adjacent dummy bus as the to bus.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = inibrn(ibus, single)
```

Fortran command syntax:

```
CALL INIBRN(IBUS, SINGLE, IERR)
```

IPLAN command syntax:

```
CALL INIBRN(IBUS, SINGLE, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer SINGLE	If 1, 'NXTBRN' or 'NXTBRN3' will return only branches for which the from bus ('IBUS') number (or bus name) is lower than the to bus (or buses); if 2, then all branches connected to 'IBUS' are returned by 'NXTBRN' or 'NXTBRN3' (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NXTBRN/NXTBRN3' is properly initialized
	IERR = 1	Bus not found; 'NXTBRN/NXTBRN3' may not be used
	IERR = 2	Invalid 'SINGLE' value; 'NXTBRN/NXTBRN3' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.106. INIBRN_2

Initializes the branch fetching routine 'NXTBRN_2' or 'NXTBRN3_2' for returning branches connected to bus sections described by IBUS and INODE. For multisection lines, 'NXTBRN_2' or 'NXTBRN3_2' returns the adjacent dummy bus as the to bus.

This API routine was first introduced in release 34.2.0.

Python command syntax:

```
ierr, isect = inibrn_2(ibus, inode, single)
```

Fortran command syntax:

```
CALL INIBRN_2(IBUS, INODE, SINGLE, ISECT, IERR)
```

IPLAN command syntax:

```
CALL INIBRN_2(IBUS, INODE, SINGLE, ISECT, IERR)
```

Where:

Integer IBUS	bus number (input).	
Integer INODE	bus section node number (input).	
Integer SINGLE	if 1, 'NXTBRN_2' or 'NXTBRN3_2' will return only branches for which the from bus ('IBUS') number (or bus name) is lower than the to bus (or buses); if 2, then all branches connected to 'IBUS' are returned by 'NXTBRN_2' or 'NXTBRN3_2' (input).	
Integer ISECT	bus section number (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTBRN_2/NXTBRN3_2' is properly initialized
	IERR = 1	invalid bus number; 'NXTBRN_2/NXTBRN3_3' may not be used
	IERR = 2	IBUS not found; 'NXTBRN_2/NXTBRN3_3' may not be used
	IERR = 3	invalid node number; 'NXTBRN_2/ NXTBRN3_2' may not be used
	IERR = 4	IBUS is not in a substation; 'NXTBRN_2/ NXTBRN3_2' may not be used
	IERR = 5	INODE is not in the substation; 'NXTBRN_2/ NXTBRN3_2' may not be used
	IERR = 6	INODE is not associated with IBUS; 'NXTBRN_2/NXTBRN3_2' may not be used
	IERR = 7	invalid 'SINGLE' value; 'NXTBRN_2/ NXTBRN3_2' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.107. INIBRX

Initializes the branch fetching routine, 'NXTBRN' or 'NXTBRN3' for returning branches connected to IBUS. For multisection lines, 'NXTBRN' or 'NXTBRN3' returns the other endpoint bus as the to bus.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = inibrx(ibus, single)
```

Fortran command syntax:

```
CALL INIBRX(IBUS, SINGLE, IERR)
```

IPLAN command syntax:

```
CALL INIBRX(IBUS, SINGLE, IERR)
```

Where:

Integer IBUS	From bus (input).	
Integer SINGLE	If 1, 'NXTBRN' or 'NXTBRN3' will return only branches for which the from bus ('IBUS') number (or bus name) is lower than the to bus (or buses); if 2, then all branches connected to 'IBUS' are returned by 'NXTBRN' or 'NXTBRN3' (input).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTBRN/NXTBRN3' properly initialized
	IERR = 1	Bus not found; 'NXTBRN/NXTBRN3' may not be used
	IERR = 2	Invalid 'SINGLE' value; 'NXTBRN/NXTBRN3' may not be used
	IERR = 3	'IBUS' is a multi-section line dummy bus; 'NXTBRN/NXTBRN3' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.108. INIBRX_2

Initializes the branch fetching routine, 'NXTBRN_2' or 'NXTBRN3_2' for returning branches connected to IBUS. For multisection lines, 'NXTBRN_2' or 'NXTBRN3_2' returns the other endpoint bus as the to bus.

This API routine was first introduced in release 34.2.0.

Python command syntax:

```
ierr, isect = inibrx_2(ibus, inode, single)
```

Fortran command syntax:

```
CALL INIBRX_2(IBUS, INODE, SINGLE, ISECT, IERR)
```

IPLAN command syntax:

```
CALL INIBRX_2(IBUS, INODE, SINGLE, ISECT, IERR)
```

Where:

Integer IBUS	bus number (input).	
Integer INODE	bus section node number (input).	
Integer SINGLE	if 1, 'NXTBRN_2' or 'NXTBRN3_2' will return only branches for which the from bus ('IBUS') number (or bus name) is lower than the to bus (or buses); if 2, then all branches connected to 'IBUS' are returned by 'NXTBRN_2' or 'NXTBRN3_2' (input).	
Integer ISECT	bus section number (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTBRN_2/NXTBRN3_2' is properly initialized
	IERR = 1	invalid bus number; 'NXTBRN_2/NXTBRN3_3' may not be used
	IERR = 2	IBUS not found; 'NXTBRN_2/NXTBRN3_3' may not be used
	IERR = 3	invalid node number; 'NXTBRN_2/ NXTBRN3_2' may not be used
	IERR = 4	IBUS is not in a substation; 'NXTBRN_2/ NXTBRN3_2' may not be used
	IERR = 5	INODE is not in the substation; 'NXTBRN_2/ NXTBRN3_2' may not be used
	IERR = 6	INODE is not associated with IBUS; 'NXTBRN_2/NXTBRN3_2' may not be used
	IERR = 7	invalid 'SINGLE' value; 'NXTBRN_2/ NXTBRN3_2' may not be used
	IERR = 8	IBUS is a multi-section line dummy bus; 'NXTBRN_2/NXTBRN3_2' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.109. INIBUS

Initializes the bus fetching routine 'NXTBUS' for retrieving buses in ascending numerical order. Dummy buses for multisection lines are included. Hidden star point buses of three-winding transformers are excluded.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = inibus(ibus)
```

Fortran command syntax:

```
CALL INIBUS(IBUS, IERR)
```

IPLAN command syntax:

```
CALL INIBUS(IBUS, IERR)
```

Where:

Integer IBUS	starting bus number, 0 for first bus in ordered list (input).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTBUS' is properly initialized
	IERR = 1	bus not found; 'NXTBUS' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.110. INIBUS_2

Initializes the bus fetching routine 'NXTBUS_2' for retrieving buses in ascending numerical order. Hidden star point buses of three-winding transformers are excluded.

This API routine was first introduced in release 34.2.0.

Python command syntax:

```
ierr = inibus_2(ibus,msl,topo)
```

Fortran command syntax:

```
CALL INIBUS_2(IBUS, MSL, TOPO, IERR)
```

IPLAN command syntax:

```
CALL INIBUS_2(IBUS, MSL, TOPO, IERR)
```

Where:

Integer IBUS	starting bus number, 0 for first bus in ordered list (input).	
Integer MSL	multi-section line interior bus code (input).	
	MSL = 0	exclude
	MSL = 1	include
Integer TOPO	topological bus code (input).	
	TOPO = 0	exclude (i.e., for a bus with multiple sections, only one bus section is included)
	TOPO = 1	include (i.e., for a bus with multiple sections, all bus sections are included)
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTBUS_2' is properly initialized
	IERR = 1	bus not found; 'NXTBUS_2' may not be used
	IERR = 2	invalid MSL value; 'NXTBUS_2' may not be used
	IERR = 3	invalid TOPO value; 'NXTBUS_2' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.111. INIBUX

Initializes the bus fetching routine 'NXTBUS' for retrieving buses in ascending numerical order. Dummy buses of multisection lines are excluded. Hidden star point buses of three-winding transformers are excluded.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = inibux(ibus)
```

Fortran command syntax:

```
CALL INIBUX(IBUS, IERR)
```

IPLAN command syntax:

```
CALL INIBUX(IBUS, IERR)
```

Where:

Integer IBUS	Starting bus number, 0 for first bus in ordered list (input).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTBUS' properly initialized
	IERR = 1	Bus not found; 'NXTBUS' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.112. INIFAX

Initializes the FACTS device fetching routine 'NXTFAX' for retrieving FACTS devices in FACTS device name alphabetical order.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr = inifax()
```

Fortran command syntax:

```
CALL INIFAX(IERR)
```

IPLAN command syntax:

```
CALL INIFAX(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTFAX' is properly initialized
	IERR = 1	No FACTS devices in case; 'NXTFAX' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.113. INIFXS

Initializes the fixed bus shunt fetching routine 'NXTFXS' for returning fixed shunts attached to bus IBUS.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr = inifxs(ibus)
```

Fortran command syntax:

```
CALL INIFXS(IBUS, IERR)
```

IPLAN command syntax:

```
CALL INIFXS(IBUS, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NXTFXS' is properly initialized
	IERR = 1	Bus not found; 'NXTFXS' may not be used
	IERR = 2	Bus has no fixed shunts; 'NXTFXS' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.114. INIIND

Initializes induction machine fetching routine 'NXTIND' for returning induction machines attached to IBUS.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr = iniind(ibus)
```

Fortran command syntax:

```
CALL INIIND(IBUS, IERR)
```

IPLAN command syntax:

```
CALL INIIND(IBUS, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NXTIND' is properly initialized
	IERR = 1	Bus not found; 'NXTIND' may not be used
	IERR = 2	Bus has no induction machines connected to it; 'NXTIND' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.115. INILOD

Initializes load fetching routine 'NXTLOD' for returning loads attached to IBUS.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = inilod(ibus)
```

Fortran command syntax:

```
CALL INILOD(IBUS, IERR)
```

IPLAN command syntax:

```
CALL INILOD(IBUS, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NXTLOD' is properly initialized
	IERR = 1	Bus not found; 'NXTLOD' may not be used
	IERR = 2	Bus has no load entries; 'NXTLOD' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.116. INIMAC

Initializes machine fetching routine 'NXTMAC' for returning machines attached to IBUS.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = inimac(ibus)
```

Fortran command syntax:

```
CALL INIMAC(IBUS, IERR)
```

IPLAN command syntax:

```
CALL INIMAC(IBUS, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NXTMAC' is properly initialized
	IERR = 1	Bus not found; 'NXTMAC' may not be used
	IERR = 2	Bus not a generator bus; 'NXTMAC' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.117. INIMDC

Initializes the multi-terminal dc line fetching routine 'NXTMDC' for retrieving multi-terminal dc lines in dc line name alphabetical order.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr = inimdc()
```

Fortran command syntax:

```
CALL INIMDC(IERR)
```

IPLAN command syntax:

```
CALL INIMDC(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTMDC' is properly initialized
	IERR = 1	No multi-terminal dc lines in case; 'NXTMDC' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.118. INIMSL

Initializes the multi-section line member fetching routine 'NXTMSL' for retrieving lines that are sections of the specified multi-section line from IBUS to JBUS.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = inimsl(ibus, jbus, ickt)
```

Fortran command syntax:

```
CALL INIMSL(IBUS, JBUS, ICKT, IERR)
```

IPLAN command syntax:

```
CALL INIMSL(IBUS, JBUS, ICKT, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*2 ICKT	Multi-section line identifier (must begin with &) (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; NXTMSL properly initialized
	IERR = 1	Bus not found; NXTMSL may not be used
	IERR = 2	& not the first character of 'ICKT'; NXTMSL may not be used
	IERR = 3	Multi-section line not found; NXTMSL may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.119. ININAM

Initializes the bus fetching routine 'NXTBUS' for retrieving buses in ascending alphabetical order. Dummy buses for multisection lines are included. Hidden star point buses of three winding transformers are excluded.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = ininam(string)
```

Fortran command syntax:

```
CALL ININAM(STRING, IERR)
```

IPLAN command syntax:

```
CALL ININAM(STRING, IERR)
```

Where:

Character STRING	Extended bus name, blank for first bus in ordered list (input).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTBUS' is properly initialized
	IERR = 1	Bus not found; 'NXTBUS' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.120. ININAM_2

Initializes the bus fetching routine 'NXTBUS_2' for retrieving buses in ascending alphabetical order. Hidden star point buses of three winding transformers are excluded.

This API routine was first introduced in release 34.2.0.

Python command syntax:

```
ierr = ininam_2(string, msl, topo)
```

Fortran command syntax:

```
CALL ININAM_2(STRING, MSL, TOPO, IERR)
```

IPLAN command syntax:

```
CALL ININAM_2(STRING, MSL, TOPO, IERR)
```

Where:

Character STRING	extended bus name, blank for first bus in ordered list (input).	
Integer MSL	multi-section line interior bus code (input).	
	MSL = 0	exclude
	MSL = 1	include
Integer TOPO	topological bus code (input).	
	TOPO = 0	exclude (i.e., for a bus with multiple sections, only one bus section is included)
	TOPO = 1	include (i.e., for a bus with multiple sections, all bus sections are included)
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTBUS_2' is properly initialized
	IERR = 1	bus not found; 'NXTBUS_2' may not be used
	IERR = 2	invalid MSL value; 'NXTBUS_2' may not be used
	IERR = 3	invalid TOPO value; 'NXTBUS_2' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.121. ININAX

Initializes the bus fetching routine 'NXTBUS' for retrieving buses in ascending alphabetical order. Dummy buses of multisection lines are excluded. Hidden star point buses of three winding transformers are excluded.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = ininax(string)
```

Fortran command syntax:

```
CALL ININAX(STRING, IERR)
```

IPLAN command syntax:

```
CALL ININAX(STRING, IERR)
```

Where:

Character STRING	Extended bus name, blank for first bus in ordered list (input).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTBUS' properly initialized
	IERR = 1	Bus not found; 'NXTBUS' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.122. INISTABUSSECT

Use this API routine to initialize the bus section fetching routine 'NXTSTABUSSECT' for retrieving the bus sections in a substation.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr = inistabussect(ista)
```

Fortran command syntax:

```
CALL INISTABUSSECT(ISTA, IERR)
```

IPLAN command syntax:

```
CALL INISTABUSSECT(ISTA, IERR)
```

Note:

Initialize the bus section fetching routine 'NXTSTABUSSECT'.

Where:

Integer ISTA	substation number (input).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTSTABUSSECT' is properly initialized
	IERR = 1	substation not found; ' NXTSTABUSSECT' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.123. INISTANODE

Use this API routine to initialize the node fetching routine 'NXTSTANODE' for retrieving the nodes in a sub-station.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr = inistanode(ista)
```

Fortran command syntax:

```
CALL INISTANODE(ISTA, IERR)
```

IPLAN command syntax:

```
CALL INISTANODE(ISTA, IERR)
```

Note:

Initialize the node fetching routine 'NXTSTANODE'.

Where:

Integer ISTA	substation number (input).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTSTANODE' is properly initialized
	IERR = 1	substation not found; ' NXTSTANODE' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.124. INISTASWDEV

Use this API routine to initialize the switching device fetching routine 'NXTSTASWDEV' for retrieving the switching devices in a substation.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr = inistaswdev(ista)
```

Fortran command syntax:

```
CALL INISTASWDEV(ISTA, IERR)
```

IPLAN command syntax:

```
CALL INISTASWDEV(ISTA, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'NXTSTASWDEV' is properly initialized
	IERR = 1	substation not found; ' NXTSTASWDEV' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.125. INISWS

Initializes the switched shunt fetching routine 'NXTSWS' for returning switched shunts attached to bus IBUS.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr = inisws(ibus)
```

Fortran command syntax:

```
CALL INISWS(IBUS, IERR)
```

IPLAN command syntax:

```
CALL INISWS(IBUS, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NXTSWS' is properly initialized
	IERR = 1	Bus not found; 'NXTSWS' may not be used
	IERR = 2	Bus has no switched shunts; 'NXTSWS' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.126. INITIE

Initializes the tie branch fetching routine 'NXTTIE' or 'NXTTIE3' for returning tie branches from area IAR. For multisection lines connected to a from area bus, the adjacent dummy bus is treated as the to bus. Dummy buses for multisection lines are included.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = initie(iar)
```

Fortran command syntax:

```
CALL INITIE(IAR, IERR)
```

IPLAN command syntax:

```
CALL INITIE(IAR, IERR)
```

Where:

Integer IAR	From area number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTTIE' or 'NXTTIE3' is properly initialized
	IERR = 1	Area number <= 0 or > largest allowable area number; 'NXTTIE' or 'NXTTIE3' may not be used
	IERR = 2	No buses in area 'IAR'; 'NXTTIE' or 'NXTTIE3' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.127. INITIX

Initializes the tie branch fetching routine 'NXTTIE' or 'NXTTIE3' for returning tie branches from area IAR. For multisection lines connected to a from area bus, the other endpoint bus is treated as the to bus. Dummy buses of multisection lines are excluded.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = initix(iar)
```

Fortran command syntax:

```
CALL INITIX(IAR, IERR)
```

IPLAN command syntax:

```
CALL INITIX(IAR, IERR)
```

Where:

Integer IAR	From area number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTTIE' or 'NXTTIE3' is properly initialized
	IERR = 1	Area number <= 0 or > largest allowable area number; 'NXTTIE' or 'NXTTIE3' may not be used
	IERR = 2	No buses in area 'IAR'; 'NXTTIE' or 'NXTTIE3' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.128. INIVSC

Initializes the VSC dc line fetching routine 'NXTVSC' for retrieving VSC dc lines in dc line name alphabetical order.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr = inivsc()
```

Fortran command syntax:

```
CALL INIVSC(IERR)
```

IPLAN command syntax:

```
CALL INIVSC(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error, 'NXTVSC' is properly initialized
	IERR = 1	No VSC dc lines in case; 'NXTVSC' may not be used

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.129. INTFLOW

Use this API to return interface flow for the specified interface.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, cmpval = intflow(name)
```

Fortran command syntax:

```
CALL INTFLOW(NAME, CMPVAL, IERR)
```

Where:

Character*40 NAME	String indicating the interface name (input).	
Complex CMPVAL	Complex value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CMPVAL' returned
	IERR = 1	Invalid interface;
	IERR = 2	Interface not found;
	IERR = -1	data error, warning and information messages; one or more of: - Interface name is too long

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.130. ISLAND

Trips in-service branches connected to Type 4 (or greater) buses, and disconnects islands not containing a swing bus.
Note: Calling ISLAND with generators converted will disconnect all buses.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = island()
```

Fortran command syntax:

```
CALL ISLAND(IERR)
```

IPLAN command syntax:

```
CALL ISLAND(IERR)
```

Note:

Calling ISLAND with generators converted will disconnect all buses.

Where:

Integer IERR	error code (output).	
	IERR = 0	No errors.
	IERR = -1	No in-service buses remaining.
	IERR > 0	Number of islands tripped.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.131. ITERAT

Use this API to return the number of iterations used in the last solution attempt.

This API routine was first introduced in release 30.1.0.

Python command syntax:

```
ival = iterat()
```

Fortran command syntax:

```
CALL ITERAT(IVAL)
```

IPLAN command syntax:

```
CALL ITERAT(IVAL)
```

Where:

Integer IVAL	Number of iterations used in the last solution attempt (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.132. LMODIND

Returns load-related model starting array indices and status.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = lmodind(ibus, id ,string1, string2)
```

Fortran command syntax:

```
CALL LMODIND(IBUS, ID, STRING1, STRING2, IVAL, IERR)
```

IPLAN command syntax:

```
CALL LMODIND(IBUS, ID, STRING1, STRING2, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Load identifier (input).	
Character STRING1	String indicating the type of load model desired (input).	
	'CHARAC'	Load characteristic model
	'RELAY'	Load relay model
Character STRING2	String indicating the load model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'RESICON'	Starting reserved ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'NRESICON'	Number of reserved ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect load status. For a model to be called during simulations the load must be in-service bus type code must be 1 or 2 and the model's bypassed flag must be not bypassed
Integer IVAL	Load model array index or status (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Load not in network but in model tables; 'IVAL' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Load not in network or in model tables; 'IVAL' unchanged
IERR = 3	Invalid value of 'STRING1'; 'IVAL' unchanged
IERR = 4	No model of type 'STRING1' for this load; 'IVAL' unchanged
IERR = 5	Invalid value of 'STRING2'; 'IVAL' unchanged
IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.133. LMODNAM

Returns load-related model name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = lmodnam(ibus, id ,string)
```

Fortran command syntax:

```
CALL LMODNAM(IBUS, ID, STRING, CVAL, IERR)
```

IPLAN command syntax:

```
CALL LMODNAM(IBUS, ID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Load identifier (input).	
Character STRING	String indicating the type of load model desired (input).	
	'CHARAC'	Load characteristic model
	'RELAY'	Load relay model
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Load not in network but in model tables; 'CVAL' returned
	IERR = 2	Load not in network or in model tables; 'CVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'CVAL' unchanged
	IERR = 4	No model of type 'STRING' for this load; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.134. LODBUS

Use this API to return the load bus and load id by load name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, ibus, id = lodbus(name)
```

Fortran command syntax:

```
CALL LODBUS(NAME, IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL LODBUS(NAME, IBUS, ID, IERR)
```

Where:

Character*40 NAME	String indicating the load name (input).	
Integer IBUS	Bus number (output).	
Character*2 ID	Load identifier (output; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error; load bus and 'ID' returned
	IERR = 1	Load not found
	IERR = 2	Load name is empty
	IERR = 3	Bus type code is not 1, 2 or 3; load bus and 'ID' returned
	IERR = 4	Load out-of-service; load bus and 'ID' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.135. LODCNV

This obsolete API always returns a 0.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
icode = lodcnv()
```

Fortran command syntax:

```
CALL LODCNV(ICODE)
```

IPLAN command syntax:

```
CALL LODCNV(ICODE)
```

Where:

Integer ICODE	Returns a 0 (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.136. LODDT1

Returns load quantities in MVA.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = loddt1(ibus, id ,string1, string2)
```

Fortran command syntax:

```
CALL LODDT1(IBUS, ID, STRING1, STRING2, RVAL, IERR)
```

IPLAN command syntax:

```
CALL LODDT1(IBUS, ID, STRING1, STRING2, RVAL, IERR)
```

Note:

When STRNG1 is one of the following: 'MVA', 'IL', 'YL', 'TOTAL', the values returned are the net load values (i.e., value includes load plus contribution from any in-service distributed generation on the load feeder at that bus)

Where:

Integer IBUS	bus number (input).	
Character ID	load identifier (input).	
Character STRING1	string indicating the load characteristic desired (input).	
	'MVA'	constant MVA load
	'IL'	constant current load
	'YL'	constant admittance load
	'TOTAL'	total load
	'YNEG'	exceptional nominal negative sequence load
	'YZERO'	exceptional nominal zero sequence load
Character STRING2	string indicating the units to be used (input).	
	'ACT'	actual load (nominal load for STRING1 = 'YNEG' or 'YZERO'). 'P' 'Q' returned in MW Mvar
	'O_ACT'	actual load (nominal load for STRING1 = 'YNEG' or 'YZERO'). 'P' 'Q' in units determined by the power output option setting
	'NOM'	nominal load (at 1.0 pu voltage). 'RVAL' returned in MVA
	'O_NOM'	nominal load (at 1.0 pu voltage). 'RVAL' in units determined by the power output option setting
Real RVAL	magnitude of the complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	no error; 'RVAL' returned
IERR = 1	Bbus not found; 'RVAL' unchanged
IERR = 2	Load not found; 'RVAL' unchanged
IERR = 3	Bbus type code is not 1, 2 or 3; 'RVAL' returned
IERR = 4	Lload out-of-service; 'RVAL' returned
IERR = 5	Iinvalid value of 'STRING1' or 'STRING2'; 'RVAL' unchanged
IERR = 6	Ssequence data not in case for 'STRING1' = 'YNEG' or 'YZERO'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.137. LODDT2

Use this API to return complex load quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = loddt2(ibus, id ,string1, string2)
```

Fortran command syntax:

```
CALL LODDT2(IBUS, ID, STRING1, STRING2, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL LODDT2(IBUS, ID, STRING1, STRING2, P, Q, IERR)
```

Note:

When STRNG1 is one of the following: 'MVA', 'IL', 'YL', 'TOTAL', the values returned are the net load values (i.e., value includes load plus the contribution from any in-service distributed generation on the load feeder at that bus)

Where:

Integer IBUS	bus number (input).	
Character ID	load identifier (input).	
Character STRING1	string indicating the load characteristic desired (input).	
	'MVA'	constant MVA load
	'IL'	constant current load
	'YL'	constant admittance load
	'TOTAL'	total load
	'YNEG'	exceptional nominal negative sequence load
	'YZERO'	exceptional nominal zero sequence load
Character STRING2	String indicating the units to be used (input).	
	'ACT'	actual load (nominal load for STRING1 = 'YNEG' or 'YZERO'). 'P' 'Q' returned in MW Mvar
	'O_ACT'	actual load (nominal load for STRING1 = 'YNEG' or 'YZERO'). 'P' 'Q' in units determined by the power output option setting
	'NOM'	nominal load (at 1.0 pu voltage). 'P' 'Q' returned in MW Mvar
	'O_NOM'	nominal load (at 1.0 pu voltage). 'P' 'Q' in units determined by the power output option setting
Real P	real component of complex value indicated by STRING1 in the units indicated by STRING2	
Real Q	reactive component of complex value indicated by STRING1 in the units indicated by STRING2	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Complex CMPVAL	complex value indicated by STRING1 in the units indicated by STRING2 (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	load not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	bus type code is not 1, 2 or 3; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 4	load out-of-service; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 5	invalid value of 'STRING1' or 'STRING2'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 6	sequence data not in case for 'STRING1' = 'YNEG' or 'YZERO'; 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.138. LODIND

Use this API to return the load array index.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = lodind(ibus, id)
```

Fortran command syntax:

```
CALL LODIND(IBUS, ID, IVAL, IERR)
```

IPLAN command syntax:

```
CALL LODIND(IBUS, ID, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Load identifier (input).	
Integer IVAL	Load array index (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Load not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.139. LODINT

Use this API to return integer load quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = lodint(ibus, id, string)
```

Fortran command syntax:

```
CALL LODINT(IBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL LODINT(IBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Load identifier (input).	
Character STRING	String indicating the load quantity desired (input).	
	'STATION'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
	'SECTION'	Bus IBUS bus section number (returned as 0 if bus IBUS is not in a substation)
	'NODE'	Node number (0 if bus IBUS is not in a substation)
	'STATUS'	Load status; 1 (one) for in-service else 0 (zero)
	'AREA'	Load area number
	'ZONE'	Load zone number
	'OWNER'	Load owner number
	'SCALE'	Scalable load flag
	'CGR'	Load grounding code (0 for ungrounded 1 for grounded)
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Load not found; 'IVAL' unchanged
	IERR = 3	Bus type code is not 1, 2 or 3; 'IVAL' returned
	IERR = 4	Load out-of-service; 'IVAL' returned
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Sequence data not in case for 'STRING1' = 'CGR'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.140. LODNAME

Use this API to return the load name character string.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, string = lodname(ibus, id)
```

Fortran command syntax:

```
CALL LODNAME(IBUS, ID, STRING, IERR)
```

IPLAN command syntax:

```
CALL LODNAME(IBUS, ID, STRING, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Load identifier (input; no default allowed).	
Character*40 STRING	String indicating the load name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Bus not found; 'STRING' unchanged
	IERR = 2	Load not found; 'STRING' unchanged
	IERR = 3	Bus type code is not 1, 2 or 3; 'STRING' returned
	IERR = 4	Load out-of-service; 'STRING' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.141. LODTYPE

Use this API to return the load type character string.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, string = lodtype(ibus, id)
```

Fortran command syntax:

```
CALL LODTYPE(IBUS, ID, STRING, IERR)
```

IPLAN command syntax:

```
CALL LODTYPE(IBUS, ID, STRING, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Load identifier (input; no default allowed).	
Character*12 STRING	String indicating the load type (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Bus not found; 'STRING' unchanged
	IERR = 2	Load not found; 'STRING' unchanged
	IERR = 3	Bus type code is not 1, 2 or 3; 'STRING' returned
	IERR = 4	Load out-of-service; 'STRING' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.142. MACBUS

Use this API to return the machine bus and machine id by load name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, ibus, id = macbus(name)
```

Fortran command syntax:

```
CALL MACBUS(NAME, IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL MACBUS(NAME, IBUS, ID, IERR)
```

Where:

Character*40 NAME	String indicating the machine name (input).	
Integer IBUS	Bus number (output).	
Character*2 ID	machine identifier (output; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error; machine bus and 'ID' returned
	IERR = 1	Machine not found
	IERR = 2	Machine name is empty
	IERR = 3	Bus type code is not 2 or 3; machine bus and 'ID' returned
	IERR = 4	Machine out-of-service; machine bus and 'ID' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.143. MACCHR

Use this API to return character machine quantities.

This API routine was first introduced in release 35.2.0.

Python command syntax:

```
ierr, cval = macchr(ibus, id, string)
```

Fortran command syntax:

```
CALL MACCHR(IBUS, ID, STRING, CVAL, IERR)
```

IPLAN command syntax:

```
CALL MACCHR(IBUS, ID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING	String indicating the machine quantity desired (input).	
	'POSTBL'	Non-conventional source positive sequence table (12 characters).
	'NEGTBL'	Non-conventional source negative sequence table (12 characters).
	'DROOPNAME'	Voltage droop control name (40 characters).
	'MCNAME'	machine name (40 characters).
character*40 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Bus not found; 'CVAL' unchanged
	IERR = 2	Machine not found; 'CVAL' unchanged
	IERR = 3	Bus type code is not 2 or 3; 'CVAL' returned
	IERR = 4	Machine off-line, 'CVAL' returned
	IERR = 5	Invalid value of 'STRING'; 'CVAL' unchanged
	IERR = 6	Sequence data not in case; 'CVAL' unchanged
	IERR = 7	No non-conventional source data for this machine; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.144. MACDAT

Use this API to return real machine quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = macdat(ibus, id, string)
```

Fortran command syntax:

```
CALL MACDAT(IBUS, ID, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL MACDAT(IBUS, ID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING	String indicating the machine quantity desired (input).	
	'QMAX'	Maximum generator reactive output in Mvar
	'O_QMAX'	Same as 'QMAX' in units determined by the power output option setting
	'QMIN'	Minimum generator reactive output in Mvar
	'O_QMIN'	Same as 'QMIN' in units determined by the power output option setting
	'PMAX'	Maximum generator active output in MW
	'O_PMAX'	Same as 'PMAX' in units determined by the power output option setting
	'PMIN'	Minimum generator real active in MW
	'O_PMIN'	Same as 'PMIN' in units determined by the power output option setting
	'MBASE'	Total MVA base
	'MVA'	Machine loading in MVA
	'O_MVA'	Same as 'MVA' in units determined by the power output option setting
	'P'	Machine loading in MW
	'O_P'	Same as 'P' in units determined by the power output option setting
	'Q'	Machine loading in Mvar
	'O_Q'	Same as 'Q' in units determined by the power output option setting
	'PERCENT'	MVA loading of the machines as a percentage of machine base
	'GENTAP'	Step-up transformer off-nominal turns ratio
	'VSCHED'	Regulated voltage setpoint

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'WPF'	'WPF Power factor used in setting reactive power limits for this wind machine when WMOD is 2 or 3
	'FRACT1'	Fraction of total ownership assigned to the first owner
	'FRACT2'	Fraction of total ownership assigned to the second owner
	'FRACT3'	Fraction of total ownership assigned to the third owner
	'FRACT4'	Fraction of total ownership assigned to the fourth owner
	'RMPCT'	Percentage of total MVAR required to regulate remote bus voltage
	'RPOS'	Positive sequence fault analysis machine resistance
	'XSUBTR'	Positive sequence fault analysis machine subtransient reactance
	'XTRANS'	Positive sequence fault analysis machine transient reactance
	'XSYNCH'	Positive sequence fault analysis machine synchronous reactance
	'IFMAX'	Non-conventional source per unit maximum total fault current magnitude
	'DBVMIN'	Non-conventional source per unit dead-band minimum voltage
	'DBVMAX'	Non-conventional source per unit dead-band maximum voltage
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Machine not found; 'RVAL' unchanged
	IERR = 3	Bus type code is not 2 or 3; 'RVAL' returned
	IERR = 4	Machine off-line; 'RVAL' returned
	IERR = 5	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 6	For STRING = 'FRACTn', n > number of owners; 'RVAL' unchanged
	IERR = 7	For STRING = 'WPF', this machine is not a wind machine; 'RVAL' of 1.0 returned
	IERR = 8	Sequence data not in case for STRING = 'RPOS', 'XSUBTR', 'XTRANS', 'XSYNCH', 'IFMAX', 'DBVMIN' or 'DBVMAX'; 'RVAL' unchanged
	IERR = 9	No non-conventional source data for this maxhine for STRING = 'IFMAX', 'DBVMIN' or 'DBVMAX'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.145. MACDT2

Use this API to return complex machine quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = macdt2(ibus, id, string)
```

Fortran command syntax:

```
CALL MACDT2(IBUS, ID, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL MACDT2(IBUS, ID, STRING, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING	String indicating the machine quantity desired (input).	
	'PQ'	Actual generator power output in MW/Mvar
	'O_PQ'	Same as 'PQ' in units determined by the power output option setting
	'ZSORCE'	Machine impedance
	'XTRAN'	Step-up transformer impedance
	'ZPOS'	Positive sequence fault analysis machine impedance (Rpos + j X)
	'ZNEG'	Negative sequence machine impedance
	'ZZERO'	Zero sequence machine impedance
	'ZGRND'	Zero sequence grounding impedance in per unit or ohms according to the machine's grounding impedance data input/output code
	'ZGRNDPU'	Zero sequence grounding impedance in per unit
Real P	Real component of the complex value indicated by STRING (output).	
Real Q	Reactive component of the complex value indicated by STRING (output).	
Complex CMPVAL	Complex value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Machine not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Bus type code is not 2 or 3; 'P' and 'Q' or 'CMPVAL' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Machine off-line; 'P' and 'Q' or 'CMPVAL' returned
IERR = 5	Invalid value of 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 6	Sequence data not in case for STRING = 'ZPOS', 'ZNEG', or 'ZZERO', 'ZGRND', or 'ZGRNDPU'; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.146. MACIND

Use this API to return the machine array index.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = macind(ibus, id)
```

Fortran command syntax:

```
CALL MACIND(IBUS, ID, IVAL, IERR)
```

IPLAN command syntax:

```
CALL MACIND(IBUS, ID, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Machine identifier (input).	
Integer IVAL	Machine array index (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Machine not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.147. MACINT

Use this API to return integer machine quantities.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = macint(ibus, id, string)
```

Fortran command syntax:

```
CALL MACINT(IBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL MACINT(IBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING	String indicating the machine quantity desired (input).	
	'STATION'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
	'SECTION'	Bus IBUS bus section number (returned as 0 if bus IBUS is not in a substation)
	'NODE'	Number of the node to which the machine is connected (returned as 0 if bus IBUS is not in a substation)
	'STATUS'	Machine status; 1 (one) for in-service else 0 (zero)
	'IREG'	Bus number of regulated bus
	'NREG'	Node number of regulated node
	'BASFLG'	Baseloaded flag value
	'OWNERS'	Number of owners
	'WMOD'	Wind machine reactive power limits mode; 0 if this machine is not a wind machine
	'OWN1'	Owner number of the first owner
	'OWN2'	Owner number of the second owner
	'OWN3'	Owner number of the third owner
	'OWN4'	Owner number of the fourth owner
	'PERCENT'	MVA loading of the machine as a percentage of machine base
	'CZG'	Grounding impedance data input/output code
	'NCSTYP'	Non-conventional source characteristic type
Integer IVAL	Value indicated by STRING (output).	
integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	No error; 'IVAL' returned
IERR = 1	Bus not found; 'IVAL' unchanged
IERR = 2	Machine not found; 'IVAL' unchanged
IERR = 3	Bus type code is not 2 or 3; 'IVAL' returned
IERR = 4	Machine off-line, 'IVAL' returned
IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
IERR = 6	For STRING = 'OWNn', n> number of owners; 'IVAL' unchanged
IERR = 7	Sequence data not in case for STRING = 'CZG' or 'NCSTYP'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.148. MACSTT

Use this API to return the machine status value.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = macstt(ibus, id)
```

Fortran command syntax:

```
CALL MACSTT(IBUS, ID, IVAL, IERR)
```

IPLAN command syntax:

```
CALL MACSTT(IBUS, ID, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Machine identifier (input).	
Integer IVAL	Machine status value; 0 for out of service, 1 for in service (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Machine not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.149. MAXMSM

Use this API to return the complex bus mismatch at the bus with the largest MVA mismatch.

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, ibus, cmpval = maxmsm()
```

Fortran command syntax:

```
CALL MAXMSM(IBUS, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL MAXMSM(IBUS, P, Q, IERR)
```

Where:

Integer IBUS	Bus number of the bus with the largest MVA mismatch (input).	
Real P	Real component (MW) of the bus mismatch (output).	
Real Q	Reactive component (Mvar) of the bus mismatch (output).	
Complex CMPVAL	Complex bus mismatch (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IBUS', and 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	No in-service buses found; 'IBUS' of 0, and 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.150. MDLIND

Use this API to return plant-related model starting array indices and status.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = mdlind(ibus, id, string1, string2)
```

Fortran command syntax:

```
CALL MDLIND(IBUS, ID, STRING1, STRING2, IVAL, IERR)
```

IPLAN command syntax:

```
CALL MDLIND(IBUS, ID, STRING1, STRING2, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING1	String indicating the type of plant-related model (input).	
	'GEN'	Generator model
	'COMP'	Compensator model
	'STAB'	Stabilizer model
	'EXC'	Excitation system model
	'GOV'	Turbine governor model
	'TLC'	Turbine Load Controller model
	'MINXL'	Minimum excitation limiter model
	'MAXXL'	Maximum excitation limiter model
	'SCL'	Stator Current Limiter model
	'PFC'	Power Factor/Var Controller model
Character STRING2	String indicating the plant-related model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect machine status. For a model to be called during simulations the machine must be in service the type code of the bus must be 2 and the model's bypassed flag must be not bypassed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IVAL	Value indicated by STRING Integer (output).
Integer IERR	error code (output).

IERR = 0	No error; 'IVAL' returned
IERR = 1	Machine not in network but in model tables; 'IVAL' returned
IERR = 2	Machine not in network or in model tables; 'IVAL' unchanged
IERR = 3	Invalid value of 'STRING1', 'IVAL' unchanged
IERR = 4	No model of type 'STRING1' for this machine; 'IVAL' unchanged
IERR = 5	Invalid value of 'STRING2'; 'IVAL' unchanged
IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.151. MDLLIBCNT

Use this API to return the number of libraries in the list to be searched for dynamics library models.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
nstr = mdllibcnt()
```

Fortran command syntax:

```
CALL GETMODELLIBRARYCOUNT(NSTR)
```

IPLAN command syntax:

```
CALL MDLLIBCNT(NSTR)
```

Where:

Integer NSTR	number of libraries in the list (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.152. MDLNAM

Use this API to return plant-related model name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = mdlnam(ibus, id, string)
```

Fortran command syntax:

```
CALL MDLNAM(IBUS, ID, STRING, CVAL, IERR)
```

IPLAN command syntax:

```
CALL MDLNAM(IBUS, ID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING	String indicating the type of plant-related model (input).	
	'GEN'	Generator model
	'COMP'	Compensator model
	'STAB'	Stabilizer model
	'EXC'	Excitation system model
	'GOV'	Turbine governor model
	'TLC'	Turbine Load Controller model
	'MINXL'	Minimum excitation limiter model
	'MAXXL'	Maximum excitation limiter model
	'SCL'	Stator Current Limiter model
	'PFC'	Power Factor/Var Controller model
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Machine not in network but in model tables; 'CVAL' returned
	IERR = 2	Machine not in network or in model tables; 'CVAL' unchanged
	IERR = 3	Invalid value of 'STRING', 'CVAL' unchanged
	IERR = 4	No model of type 'STRING' for this machine; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.153. MDLPGMCNT

Use this API to returns the number of model programs loaded from dynamics model libraries.

This API routine was first introduced in release 33.3.0.

Python command syntax:

```
nsiz = mdlpgmcnt()
```

Fortran command syntax:

```
CALL GETMODELPROGRAMCOUNT(NSIZ)
```

IPLAN command syntax:

```
CALL MDLPGMCNT(NSIZ)
```

Where:

Integer NSIZ	number of programs loaded (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.154. NATONO

Use this API to return the bus number for a specified 18-character extended bus name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ibus = natono(string)
```

Fortran command syntax:

```
CALL NATONO(STRING, IBUS, IERR)
```

IPLAN command syntax:

```
CALL NATONO(STRING, IBUS, IERR)
```

Where:

Character STRING	Extended bus name (input).	
Integer IBUS	Bus number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IBUS' returned
	IERR = 1	Bus not found; 'IBUS' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.155. NOTONA

Use this API to return the bus 18-character extended bus name for a specified bus number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = notona(ibus)
```

Fortran command syntax:

```
CALL NOTONA(IBUS, CVAL, IERR)
```

IPLAN command syntax:

```
CALL NOTONA(IBUS, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*18 CVAL	Extended bus name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Bus not found; 'CVAL' unchanged
	IERR = 2	'CVAL' not long enough to contain entire extended bus name; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.156. NOTONASECT

Use this API to return the bus 18-character extended bus name for a specified bus section.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, busname = notona(ibus,isect)
```

Fortran command syntax:

```
CALL NOTONASECT(IBUS,ISECT,BUSNAME,IERR)
```

IPLAN command syntax:

```
CALL NOTONASECT(IBUS,ISECT,BUSNAME,IERR)
```

Where:

Integer IBUS	bus number (input).	
Integer ISECT	section number; ISECT is returned as 0 if bus IBUS is not represented by multiple bus sections (output).	
Character*18 BUSNAME	bus name (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'STRING' returned
	IERR = 1	bus not found; 'STRING' unchanged
	IERR = 2	'STRING' smaller than CHARACTER*16; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.157. NXT2DC

Returns the next two-terminal dc line in dc line name alphabetical order. The routine INI2DC must be called first.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, string = nxt2dc()
```

Fortran command syntax:

```
CALL NXT2DC(STRING, IERR)
```

IPLAN command syntax:

```
CALL NXT2DC(STRING, IERR)
```

Where:

Character*12 STRING	Two-terminal dc line name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	No more two-terminal dc lines; 'STRING' unchanged
	IERR = 2	'INI2DC' was not called; 'STRING' unchanged
	IERR = 3	'STRING' not long enough to contain entire dc line name; 'STRING' returned but contains a truncated dc line name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.158. NXTBRN

Returns the next branch connected to a bus, excluding three-winding transformers. Each time 'NXTBRN' is called it returns the next branch in either numeric or alphabetic ascending order, depending on the PSSE bus output option setting. 'INIBRN' or 'INIBRX' must first be called to establish the from bus number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, jbus, ickt = nxtbrn(ibus)
```

Fortran command syntax:

```
CALL NXTBRN(IBUS, JBUS, ICKT, IERR)
```

IPLAN command syntax:

```
CALL NXTBRN(IBUS, JBUS, ICKT, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (output).	
Character*2 ICKT	Circuit identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'JBUS' and 'ICKT' returned
	IERR = 1	No more branches from 'IBUS'; 'JBUS' is returned set to 0, and 'ICKT' returned set to spaces
	IERR = 2	'INIBRN' or 'INIBRX' was not called for bus 'IBUS'; 'JBUS', and 'ICKT' are unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.159. NXTBRN3

Returns the next branch connected to a bus, including three-winding transformers. Each time 'NXTBRN3' is called it returns the next branch in either numeric or alphabetic ascending order, depending on the PSSE bus output option setting. 'INIBRN' or 'INIBRX' must first be called to establish the from bus number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, jbus, kbus, ickt = nxtbrn3(ibus)
```

Fortran command syntax:

```
CALL NXTBRN3(IBUS, JBUS, KBUS, ICKT, IERR)
```

IPLAN command syntax:

```
CALL NXTBRN3(IBUS, JBUS, KBUS, ICKT, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (output).	
Integer KBUS	Third bus number (output).	
Character*2 ICKT	Circuit identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'JBUS' and 'ICKT' returned
	IERR = 1	No more branches from 'IBUS'; 'JBUS', 'KBUS' are returned set to 0, and 'ICKT' returned set to spaces
	IERR = 2	'INIBRN' or 'INIBRX' was not called for bus 'IBUS'; 'JBUS', 'KBUS', and 'ICKT' are un-changed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.160. NXTBRN3_2

Returns the next branch connected to a bus section, including three-winding transformers. Each time 'NXTBRN3_2' is called it returns the next branch in either numeric or alphabetic ascending order, depending on the PSSE bus output option setting. 'INIBRN' or 'INIBRX_2' must first be called to establish the from bus number.

This API routine was first introduced in release 34.2.0.

Python command syntax:

```
ierr, jbus, jsect, kbus, ksect, ckt = nxtbrn3_2(ibus, inode)
```

Fortran command syntax:

```
CALL NXTBRN3_2(IBUS, INODE, JBUS, JSECT, KBUS, KSECT, CKT, IERR)
```

IPLAN command syntax:

```
CALL NXTBRN3_2(IBUS, INODE, JBUS, JSECT, KBUS, KSECT, CKT, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer INODE	from bus node number (input).	
Integer JBUS	to bus number (output).	
Integer JSECT	to bus section number (output).	
Integer KBUS	third bus number (output).	
Integer KSECT	third bus section number (output).	
Character*2 CKT	circuit identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'JBUS', 'JSECT', 'KBUS', 'KSECT' and 'ICKT' returned
	IERR = 1	no more branches from 'IBUS' and 'INODE'; 'JBUS', 'KBUS', 'JSECT', 'KSECT' are returned set to 0, and 'CKT' returned set to spaces
	IERR = 2	'INIBRN_2' or 'INIBRX_2' was not called for node 'INODE' of bus 'IBUS'; 'JBUS', 'JSECT', 'KBUS', 'KSECT' and 'CKT' are unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.161. NXTBRN_2

Returns the next branch connected to a bus, excluding three-winding transformers. Each time 'NXTBRN_2' is called it returns the next branch in either numeric or alphabetic ascending order, depending on the PSSE bus output option setting. 'INIBRN_2' or 'INIBRX_2' must first be called to establish the from bus number.

This API routine was first introduced in release 34.2.0.

Python command syntax:

```
ierr, jbus, jsect, ckt = nxtbrn_2(ibus, inode)
```

Fortran command syntax:

```
CALL NXTBRN_2(IBUS, INODE, JBUS, JSECT, CKT, IERR)
```

IPLAN command syntax:

```
CALL NXTBRN_2(IBUS, INODE, JBUS, JSECT, CKT, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer INODE	bus section node number (input).	
Integer JBUS	to bus number (output).	
Integer JSECT	to bus section number (output).	
Character*2 CKT	circuit identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'JBUS', 'JSECT, and 'CKT' returned
	IERR = 1	no more branches from 'IBUS' and 'INODE'; 'JBUS', 'JSECT' is returned set to 0, and 'CKT' returned set to spaces
	IERR = 2	'INIBRN_2' or 'INIBRX_2' was not called for node 'INODE' of bus 'IBUS'; 'JBUS', and 'CKT' are unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.162. NXTBUS

Returns the next bus in ordered sequence. One of the routines INIBUS, INIBUX, ININAM or ININAX must be called first. The sequence can also be affected by STAREA, STZONE, and STBSKV (more than one may be used; conditions are applied in an 'AND' fashion).

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ibus, cval = nxtbus()
```

Fortran command syntax:

```
CALL NXTBUS(IBUS, CVAL, IERR)
```

IPLAN command syntax:

```
CALL NXTBUS(IBUS, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (output).	
Character*18 CVAL	Extended bus name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IBUS' and 'CVAL' returned
	IERR = 1	No more buses; 'IBUS' and 'CVAL' unchanged
	IERR = 2	'INIBUS', 'INIBUX','ININAM' or 'ININAX' was not called; 'IBUS' and 'CVAL' unchanged
	IERR = 3	'STRING' not long enough to contain entire extended bus name; 'IBUS' and 'CVAL' returned but 'CVAL' does not contain any part of the base voltage portion of extended bus name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.163. NXTBUS_2

Returns the next bus in ordered sequence. One of the routines INIBUS_2 or ININAM_2 must be called first. The sequence can also be affected by STAREA, STZONE, and STBSKV (more than one may be used; conditions are applied in an 'AND' fashion).

This API routine was first introduced in release 34.2.0.

Python command syntax:

```
ierr, ibus, isect, cval = nxtbus_2()
```

Fortran command syntax:

```
CALL NXTBUS_2(IBUS, ISECT, CVAL, IERR)
```

IPLAN command syntax:

```
CALL NXTBUS_2(IBUS, ISECT, CVAL, IERR)
```

Where:

Integer IBUS	bus number (output).	
Integer ISECT	bus section number (returned as 0 for a bus represented by a single bus section) (output).	
Character*18 CVAL	extended bus name (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'IBUS', 'ISECT' and 'CVAL' returned
	IERR = 1	no more buses; 'IBUS', 'ISECT' and 'CVAL' unchanged
	IERR = 2	'INIBUS_2' or 'ININAM_2' was not called; 'IBUS', 'ISECT' and 'CVAL' unchanged
	IERR = 3	'CVAL' not long enough to contain entire extended bus name; 'IBUS', 'ISECT' and 'CVAL' returned but 'CVAL' does not contain any part of the base voltage portion of extended bus name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.164. NXTFAX

Returns the next FACTS device in FACTS device name alphabetical order. The routine INIFAX must be called first.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, string = nxtfax()
```

Fortran command syntax:

```
CALL NXTFAX(STRING, IERR)
```

IPLAN command syntax:

```
CALL NXTFAX(STRING, IERR)
```

Where:

Character*12 STRING	FACTS device name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	No more FACTS devices; 'STRING' unchanged
	IERR = 2	'INIFAX' was not called; 'STRING' unchanged
	IERR = 3	'STRING' not long enough to contain entire FACTS device name; 'STRING' returned but contains a truncated FACTS device name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.165. NXTFXS

Use this API to return the identifier of the next fixed shunt connected to a bus. Each time 'NXTFXS' is called it returns the next shunt identifier in ascending order. 'INIFXS' must first be called to establish the shunt bus number.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, id = nxtfxs(ibus)
```

Fortran command syntax:

```
CALL NXTFXS(IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL NXTFXS(IBUS, ID, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Shunt identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	No more fixed shunts at 'IBUS'; 'ID' unchanged
	IERR = 2	'INIFXS' was not called for bus 'IBUS'; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.166. NXTIND

Use this API routine to return the identifier of the next induction machine connected to a bus. Each time 'NXTIND' is called it returns the next induction machine identifier in ascending order. 'INIIND' must first be called to establish the induction machine bus number.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, id = nxtind(ibus)
```

Fortran command syntax:

```
CALL NXTIND(IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL NXTIND(IBUS, ID, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Induction machine identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	No more induction machines at 'IBUS'; 'ID' unchanged
	IERR = 2	'INIIND' was not called for bus 'IBUS'; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.167. NXTLOD

Use this API to return the identifier of the next load connected to a bus. Each time 'NXTLOD' is called it returns the next load identifier in ascending order. 'INILOD' must first be called to establish the load bus number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, id = nxtlod(ibus)
```

Fortran command syntax:

```
CALL NXTLOD(IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL NXTLOD(IBUS, ID, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Load identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	No more loads at 'IBUS'; 'ID' unchanged
	IERR = 2	'INILOD' was not called for bus 'IBUS'; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.168. NXTMAC

Use this API to return the identifier of the next machine connected to a bus. Each time 'NXTMAC' is called it returns the next machine identifier in ascending order. 'INIMAC' must first be called to establish the plant bus number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, id = nxtmac(ibus)
```

Fortran command syntax:

```
CALL NXTMAC(IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL NXTMAC(IBUS, ID, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Machine identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	No more machines at 'IBUS'; 'ID' unchanged
	IERR = 2	'INIMAC' was not called for bus 'IBUS'; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.169. NXTMDC

Returns the next multi-terminal dc line in dc line name alphabetical order. The routine INIMDC must be called first.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, string = nxtmdc()
```

Fortran command syntax:

```
CALL NXTMDC(STRING, IERR)
```

IPLAN command syntax:

```
CALL NXTMDC(STRING, IERR)
```

Where:

Character*12 STRING	Multi-terminal dc line name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	No more multi-terminal dc lines; 'STRING' unchanged
	IERR = 2	'INIMDC' was not called; 'STRING' unchanged
	IERR = 3	'STRING' not long enough to contain entire dc line name; 'STRING' returned but contains a truncated dc line name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.170. NXTMSL

Use this API to return the next multi-section line member branch.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ibus, jbus, ickt = nxtmsl()
```

Fortran command syntax:

```
CALL NXTMSL(IBUS, JBUS, ICKT, IERR)
```

IPLAN command syntax:

```
CALL NXTMSL(IBUS, JBUS, ICKT, IERR)
```

Where:

Integer IBUS	From bus number (output).	
Integer JBUS	To bus number (output).	
Character*2 ICKT	Multi-section line identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IBUS', 'JBUS', and 'ICKT' returned
	IERR = 1	No more branches in multi-section line; 'IBUS', 'JBUS', and 'ICKT' unchanged
	IERR = 2	& not the first character of 'ICKT'; NXTMSL may not be used
	IERR = 3	INIMSL was not called; 'IBUS', 'JBUS', and 'ICKT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.171. NXTSTABUSSECT

Use this API routine to retrieve the next bus section in the specified substation. The routine IniStaBusSect must be called first with the same substation number.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, ibus, sect = nxtstabussect(ista)
```

Fortran command syntax:

```
CALL NXTSTABUSSECT(ISTA, IBUS, SECT, IERR)
```

IPLAN command syntax:

```
CALL NXTSTABUSSECT(ISTA, IBUS, SECT, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Integer IBUS	Bus number (input).	
Integer SECT	Bus section number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IBUS' and 'SECT' are returned
	IERR = 1	No more bus sections in this substation; 'IBUS' and 'SECT' are returned as zero
	IERR = 2	'IniStaBusSect' was not called for this substation; 'IBUS' and 'SECT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.172. NXTSTANODE

Use this API routine to retrieve the next node in the specified substation. The routine IniStaNode must be called first with the same substation number.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, node = nxtstanode(ista)
```

Fortran command syntax:

```
CALL NXTSTANODE(ISTA, NODE, IERR)
```

IPLAN command syntax:

```
CALL NXTSTANODE(ISTA, NODE, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Integer NODE	Node number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NODE' is returned
	IERR = 1	No more nodes in this substation; 'NODE' is returned as zero
	IERR = 2	'IniStaNode' was not called for this substation; 'NODE' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.173. NXTSTASWDEV

Use this API routine to retrieve the next switching device in the specified substation. The routine IniStaSwDev must be called first with the same substation number.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, inode, jnode, id = nxtstaswdev(ista)
```

Fortran command syntax:

```
CALL NXTSTASWDEV(ISTA, INODE, JNODE, ID, IERR)
```

IPLAN command syntax:

```
CALL NXTSTASWDEV(ISTA, INODE, JNODE, ID, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Integer INODE	"From" node number (output).	
Integer JNODE	"To" node number (output).	
Character*2 ID	Switching device identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'INODE', 'JNODE' and 'ID' are returned
	IERR = 1	No more switching devices in this substation; 'INODE' and 'JNODE' are returned as zero, and 'ID" is returned as blanks
	IERR = 2	'IniStaSwDev' was not called for this substation; 'INODE', 'JNODE' and 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.174. NXTSWS

Use this API to return the identifier of the next switched shunt connected to a bus. Each time 'NXTSWS' is called it returns the next shunt identifier in ascending order. 'INISWS' must first be called to establish the shunt bus number.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, id = nxtsws(ibus)
```

Fortran command syntax:

```
CALL NXTSWS(IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL NXTSWS(IBUS, ID, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	Shunt identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	No more switched shunts at 'IBUS'; 'ID' unchanged
	IERR = 2	'INISWS' was not called for bus 'IBUS'; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.175. NXTTIE

Use this API to return the next tie branch from an area, excluding three-winding transformers. IBUS will be in the area IAR specified in the call to 'INITIE' or 'INITIX', which must be called first; JBUS will be in the other area.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ibus, jbus, ickt = nxttie()
```

Fortran command syntax:

```
CALL NXTTIE(IBUS, JBUS, ICKT, IERR)
```

IPLAN command syntax:

```
CALL NXTTIE(IBUS, JBUS, ICKT, IERR)
```

Where:

Integer IBUS	From bus number (output).	
Integer JBUS	To bus number (output).	
Character*2 ICKT	Circuit identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'IBUS', 'JBUS', and 'ICKT' returned
	IERR = 1	No more tie branches from area 'IAR'; 'IBUS', 'JBUS' returned set to 0, and 'ICKT' set to spaces
	IERR = 2	'INITIE' or 'INITIX' was not called; 'IBUS', 'JBUS' and 'ICKT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.176. NXTTIE3

Use this API to return the next tie branch from an area, including three-winding transformers. IBUS will be in the area IAR specified in the call to 'INITIE' or 'INITIX', which must be called first; JBUS will be in the other area.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ibus, jbus, kbus, ickt = nxttie3()
```

Fortran command syntax:

```
CALL NXTTIE3(IBUS, JBUS, KBUS, ICKT, IERR)
```

IPLAN command syntax:

```
CALL NXTTIE3(IBUS, JBUS, KBUS, ICKT, IERR)
```

Where:

Integer IBUS	From bus number (output).	
Integer JBUS	To bus number (output).	
Integer KBUS	Third bus number (output).	
Character*2 ICKT	Circuit identifier (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'IBUS', 'JBUS', 'KBUS', and 'ICKT' returned
	IERR = 1	No more tie branches from area 'IAR'; 'IBUS', 'JBUS', 'KBUS' returned set to 0, 'ICKT' returned set to spaces
	IERR = 2	'INITIE' or 'INITIX' was not called; 'IBUS', 'JBUS', 'KBUS', and 'ICKT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.177. NXTVSC

Returns the next VSC dc line in dc line name alphabetical order. The routine INIVSC must be called first.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, string = nxtvsc()
```

Fortran command syntax:

```
CALL NXTVSC(STRING, IERR)
```

IPLAN command syntax:

```
CALL NXTVSC(STRING, IERR)
```

Where:

Character*12 STRING	VSC dc line name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	No more VSC dc lines; 'STRING' unchanged
	IERR = 2	'INIVSC' was not called; 'STRING' unchanged
	IERR = 3	'STRING' not long enough to contain entire dc line name; 'STRING' returned but contains a truncated dc line name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.178. OKSTRT

Routine which may be called after activity STRT or MSTR to indicate the presence of INITIAL CONDITIONS SUSPECT errors (for activity STRT) or at least one MODEL NOT AVAILABLE IN MSTR/MRUN message (for MSTR).

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = okstrt()
```

Fortran command syntax:

```
CALL OKSTRT(IERR)
```

IPLAN command syntax:

```
CALL OKSTRT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Simulation initialization errors were detected
	IERR = 9	Dynamics data not present in working memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.179. OPENUN

Connects an IPLAN unit number to the Fortran unit used by activity OPEN and returns that IPLAN unit number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
n = openun()
```

Fortran command syntax:

```
CALL OPENUN(N)
```

IPLAN command syntax:

```
CALL OPENUN(N)
```

Where:

Integer N	IPLAN unit number; zero if no unit is currently open by activity OPEN (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.180. ORDBUS

This API routine returns the bus number corresponding to the specified position in the ordered list (numeric or alphabetic) of non-hidden buses (i.e., star point buses and topological buses are excluded). Dummy buses of multi-section lines are included; hidden star point buses of three-winding transformers are not included; topological buses of buses represented by multiple bus sections are not included.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ibus = ordbus(iord)
```

Fortran command syntax:

```
CALL ORDBUS(IORD, IBUS, IERR)
```

IPLAN command syntax:

```
CALL ORDBUS(IORD, IBUS, IERR)
```

Where:

Integer IORD	Ordinal position; 1 through the number of non-hidden buses in the working case (input).	
Integer IBUS	Bus number (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IBUS' returned
	IERR = 1	'IORD' outside of range 1 through the number of non-hidden buses in the working case; 'IBUS' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.181. ORDBUSSECT

This API routine returns the bus number and section number corresponding to the specified position in the ordered list (numeric or alphabetic) of non-star point buses (i.e., star point buses are excluded). Dummy buses of multi-section lines are included; hidden star point buses of three-winding transformers are not included; topological buses of buses represented by multiple bus sections are included.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, ibus, isect = ordbussect(iord)
```

Fortran command syntax:

```
CALL ORDBUSSECT(IORD, IBUS, ISECT, IERR)
```

IPLAN command syntax:

```
CALL ORDBUSSECT(IORD, IBUS, ISECT, IERR)
```

Where:

Integer IORD	Ordinal position; 1 through the number of non-star point buses in the working case (input).	
Integer IBUS	Bus number (output).	
Integer ISECT	Section number; ISECT is returned as 0 if bus IBUS is not represented by multiple bus sections (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IBUS' and 'ISECT' returned
	IERR = 1	'IORD' outside of range 1 through the number of non-start point buses in the working case; 'IBUS' and 'ISECT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.182. OWNDAT

Returns the data associated with an owner ('LOAD' gets loads at buses of owner 'IAR', 'LOADLD' gets loads assigned to owner 'IAR').

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = owndat(iar, string)
```

Fortran command syntax:

```
CALL OWDAT(IAR, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL OWNDAT(IAR, STRING, P, Q, IERR)
```

Where:

Integer IAR	Owner number (input).	
Character STRING	String indicating the owner total desired (input).	
	'LOAD'	Total owner load by bus owner assignment (net of load plus in-service distributed generation on load feeder)
	'LOADLD'	Total owner load by load owner assignment
	'LDGN'	Total distributed generation on load feeder by bus owner assignment
	'LDGNLD'	Total distributed generation on load feeder by load owner assignment
	'GEN'	Total owner generation
	'LOSS'	Total owner losses; I**2 * (R + jX) for ac branches
	'XFRMAG'	Magnetizing admittance powers for transformers with winding 1 bus in the owner
	'INDMAC'	Total owner induction machine powers by bus owner assignment
	'INDMACMC'	Total owner induction machine powers by machine owner assignment
	'INDGEN'	Total owner induction generator powers by bus owner assignment
	'INDGENMC'	Total owner induction generator powers by machine owner assignment
	'INDMOT'	Total owner induction motor powers by bus owner assignment
	'INDMOTMC'	Total owner induction motor powers by machine owner assignment
Real P	Real component of complex power (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real Q	Reactive component of complex power (output).	
Complex CMPVAL	Complex power (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	No in-service buses with in-service loads
	IERR = 2	No in-service buses with in-service loads (for 'LOAD'), no in-service loads (for 'LOADLD'), no type 2 or type 3 buses (for 'GEN'), no in-service machines (for 'GENMC'), no in-service branches (for 'LOSS'), no in-service buses with in-service induction machines of the appropriate type (for 'INDMAC', 'INDGEN' and 'INDMOT'), or no inservice induction machines of the appropriate type (for 'INDMACMC', 'INDGENMC' and 'INDMOTMC') in owner; 'P' and 'Q' or 'CMPVAL' unchanged.
	IERR = 3	Owner not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	Bad 'STRING' value; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.183. OWNNAM

Returns the owner name for a specified owner number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = ownnam(iar)
```

Fortran command syntax:

```
CALL OWNNAM(IAR, CVAL, IERR)
```

IPLAN command syntax:

```
CALL OWNNAM(IAR, CVAL, IERR)
```

Where:

Integer IAR	Owner number (input).	
Character*12 CVAL	Owner name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Owner number <= 0 or > largest allowable owner number; 'CVAL' unchanged
	IERR = 2	Owner not found; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.184. OWNNUM

Returns the owner number for a specified owner name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, iar = ownnum(string)
```

Fortran command syntax:

```
CALL OWNNUM(IAR, STRING, IERR)
```

IPLAN command syntax:

```
CALL OWNNUM(IAR, STRING, IERR)
```

Where:

Integer IAR	Owner number (output).	
Character*12 STRING	Owner name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IAR' returned
	IERR = 1	Owner with name 'STRING' not found
	IERR > 1	Multiple owners with the name 'STRING' found; 'IAR' returned as the lowest numbered of these

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.185. OWNUSE

Use this API to indicate whether an owner is in use.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ival = ownuse(iar)
```

Fortran command syntax:

```
CALL OWNUSE(IAR, IVAL)
```

IPLAN command syntax:

```
CALL OWNUSE(IAR, IVAL)
```

Where:

Integer IAR	Owner number (input).	
Integer IVAL	Owner status (output).	
	IVAL = -1	Owner number <= 0 or > largest allowable owner number.
	IVAL = 0	If no equipment is assigned to owner IAR.
	IVAL = 1	If any equipment is assigned to owner IAR.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.186. PRMDAT

Use this API routine to return real-valued solution parameters.

This API routine was first introduced in release 32.2.0.

Python command syntax:

```
ierr, rval = prmdat(string)
```

Fortran command syntax:

```
CALL PRMDAT(STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL PRMDAT(STRING, RVAL, IERR)
```

Where:

Character STRING	String indicating the solution parameter desired (input).	
	'ACCP'	Gauss-Seidel real component acceleration factor
	'ACCQ'	Gauss-Seidel imaginary component acceleration factor
	'ACCM'	special MSLV acceleration factor
	'TOL'	Gauss-Seidel convergence tolerance
	'ACCN'	Newton-Raphson acceleration factor
	'TOLN'	Newton_Raphson mismatch convergence tolerance
	'ACCTY'	TYSL acceleration factor
	'TOLTY'	TYSL convergence tolerance
	'BLOWUP'	blown-up voltage change threshold
	'PQBRAK'	constant power load characteristic breakpoint
	'THRSHZ'	zero impedance line threshold
	'ADJTHR'	automatic adjustment threshold tolerance
	'ACCTAP'	tap movement deceleration factor
	'TAPLIM'	maximum tap change per adjustment cycle
	'DVLIM'	largest abs(delta V) / abs(V) for Newton-Raphson solutions
	'NDVFCT'	non-divergent required improvement factor
	'VCTOLQ'	Newton-Raphson controlled bus reactive power mismatch convergence tolerance
	'VCTOLV'	Newton-Raphson controlled bus voltage error convergence tolerance
	'SWVBND'	percent of voltage controlling band mode switched shunts to be adjusted
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	No error; 'RVAL' returned
IERR = 1	Invalid value of 'STRING'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.187. PRMINT

Use this API routine to return integer-valued solution parameters.

This API routine was first introduced in release 32.2.0.

Python command syntax:

```
ierr, ival = prmint(string)
```

Fortran command syntax:

```
CALL PRMINT(STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL PRMINT(STRING, IVAL, IERR)
```

Where:

Character STRING	String indicating the solution parameter desired (input).	
	'ITMX'	Gauss-Seidel maximum number of iterations
	'ITMXN'	Newton-Raphson maximum number of iterations
	'ITMXTY'	TYSL maximum number of iterations
	'MXTPSS'	maximum number of tap and/or switched shunt adjustment cycles that change data between iterations
	'MXSWIM'	maximum number of induction machine state switchings during a power flow solution
	'PFCODE'	last solution type performed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.188. RATCHR

This API routine is obsolete. It has been replaced by the API routine brn_rating_name_data .

Use this API to get rating set parameters.

This API routine was first introduced in release 34.0.0.

Python command syntax:

```
ierr, strout = ratchr(ratset, strin)
```

Fortran command syntax:

```
CALL RATCHR(RATSET,STRIN,STROUT,IERR)
```

IPLAN command syntax:

```
CALL RATCHR(RATSET,STRIN,STROUT,IERR)
```

Where:

Integer RATSET	rating set number (1 through 12) (input).	
Character STRIN	(input).	
	'NAME'	column heading (6 characters).
	'DESC'	description (32 characters).
Character*32 STROUT	character string (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'STROUT' returned
	IERR = 1	invalid rating set number; 'STROUT' unchanged
	IERR = 2	bad value of 'STRIN'; 'STROUT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.189. RMODIND

Returns the branch relay model starting array indices and status.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = rmodind(ibus, jbus, ckt, slot, string)
```

Fortran command syntax:

```
CALL RMODIND(IBUS, JBUS, CKT, SLOT, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL RMODIND(IBUS, JBUS, CKT, SLOT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character CKT	circuit identifier (input).	
Integer SLOT	relay slot (1 or 2) (input).	
Character STRING	string indicating the relay model quantity desired (input).	
	'CON'	starting CON index
	'STATE'	starting STATE index
	'VAR'	starting VAR index
	'ICON'	starting ICON index
	'NCON'	number of CON values used by the model
	'NSTATE'	number of STATE values used by the model
	'NVAR'	number of VAR values used by the model
	'NICON'	number of ICON values used by the model
	'STATUS'	bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect branch status. For a model to be called during simulations the branch must be in-service and the model's bypassed flag must be not bypassed
Integer IVAL	branch model index or status (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'IVAL' returned
	IERR = 1	branch not in network but in model tables; 'IVAL' returned
	IERR = 2	branch not in network or in model tables; 'IVAL' unchanged
	IERR = 3	invalid value of 'SLOT'; 'IVAL' unchanged
	IERR = 4	no SLOT 'SLOT' model for this branch; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	invalid value of 'STRING'; 'IVAL' unchanged
IERR = 6	model does not use any of this data type; 'IVAL' = 0 returned
IERR = 9	dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.190. RMODNAM

Returns the branch relay model name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = rmodnam(ibus, jbus, ickt, slot)
```

Fortran command syntax:

```
CALL RMODNAM(IBUS, JBUS, ICKT, SLOT, CVAL, IERR)
```

IPLAN command syntax:

```
CALL RMODNAM(IBUS, JBUS, ICKT, SLOT, CVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character*16 ICKT	Circuit identifier (input).	
Integer SLOT	Relay slot (1 or 2) (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Branch not in network but in model tables; 'CVAL' returned
	IERR = 2	Branch not in network or in model tables; 'CVAL' unchanged
	IERR = 3	Invalid value of 'SLOT'; 'CVAL' unchanged
	IERR = 4	No SLOT 'SLOT' model for this branch; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.191. SBSGTBKV

Use this API to get previously defined bus subsystem voltage limits. It returns BASEKV array used in the definition of BSYS.

This API routine was first introduced in release 34.5.0.

Python command syntax:

```
bskvmn, bskvmx = sbsgtbkv(sidarg)
```

Fortran command syntax:

```
CALL SBSGTBKV(SIDARG, BSKVMN, BSKVMX)
```

IPLAN command syntax:

```
CALL SBSGTBKV(SIDARG, BSKVMN, BSKVMX)
```

Where:

Integer SIDARG	Previously defined valid bus subsystem ID (input; no default allowed).
Real BSKVMN	Minimum basekV limit (output).
Real BSKVMX	Maximum basekV limit (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.192. SC3WND

Returns the three-winding transformer short circuit currents arriving at IBUS following activity SCMU.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr, cmpval = sc3wnd(ibus, jbus, kbus, ickt, string)
```

Fortran command syntax:

```
CALL SC3WND(IBUS, JBUS, KBUS, ICKT, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL SC3WND(IBUS, JBUS, KBUS, ICKT, STRING, P, Q, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Integer KBUS	Third bus number (input).	
Character ICKT	Circuit identifier (string (input).	
Character STRING	String indicating the short circuit current desired (input).	
	'IPOS'	Positive sequence
	'INEG'	Negative sequence
	'IZERO'	Zero sequence
	'IA'	A-phase
	'IB'	B-phase
	'IC'	C-phase
Real P	Real component of the complex current indicated by STRING (output).	
Real Q	Reactive component of the complex current indicated by STRING (output).	
Complex CMPVAL	Complex current indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	'SCINIT' not successfully called; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Branch value not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	Invalid 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 5	Branch out of service; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.193. SCBRN2

Returns the branch short circuit currents arriving at IBUS following activity SCMU.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = scbrn2(ibus, jbus, ickt, string)
```

Fortran command syntax:

```
CALL SCBRN2(IBUS, JBUS, ICKT, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL SCBRN2(IBUS, JBUS, ICKT, STRING, P, Q, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character ICKT	Circuit identifier (input).	
Character STRING	String indicating the short circuit current desired (input).	
	'IPOS'	Positive sequence
	'INEG'	Negative sequence
	'IZERO'	Zero sequence
	'IA'	A-phase
	'IB'	B-phase
	'IC'	C-phase
Real P	Real component of the complex current indicated by STRING (output).	
Real Q	Reactive component of the complex current indicated by STRING (output).	
Complex CMPVAL	Complex current indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	'SCINIT' not successfully called; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Branch not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	Bad value of 'STRING1'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 5	Branch out of service; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.194. SCBUS2

Returns complex bus voltages and currents following activity SCMU.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = scbus2(ibus, string)
```

Fortran command syntax:

```
CALL SCBUS2(IBUS, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL SCBUS2(IBUS, STRING, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character STRING	String indicating the quantity desired (input).	
	'VPOS'	Positive sequence voltage
	'VNEG'	Negative sequence voltage
	'VZERO'	Zero sequence voltage
	'VA'	A-phase voltage
	'VB'	B-phase voltage
	'VC'	C-phase voltage
	'SHUNTP'	Positive sequence shunt current
	'SHUNTN'	Negative sequence shunt current
	'SHUNTZ'	Zero sequence shunt current
	'SHUNTA'	A-phase shunt current
	'SHUNTB'	B-phase shunt current
	'SHUNTC'	C-phase shunt current
	'FAULTP'	Positive sequence fault current
	'FAULTN'	Negative sequence fault current
	'FAULTZ'	Zero sequence fault current
	'FAULTA'	A-phase fault current
	'FAULTB'	B-phase fault current
	'FAULTC'	C-phase fault current
Real P	Real component of the complex value indicated by STRING (output).	
Real Q	Reactive component of the complex value indicated by STRING (output).	
Complex CMPVAL	Complex value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	'SCINIT' not successfully called; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 3	Bad value of 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 4	Bus disconnected; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned
IERR = 5	Bus not faulted; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.195. SCDONE

Use this API to restore the working case after 'SCINIT'.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = scdone()
```

Fortran command syntax:

```
CALL SCDONE(IERR)
```

IPLAN command syntax:

```
CALL SCDONE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	'SCINIT' not successfully called

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.196. SCINIT

Use this API to initialize the short-circuit data retrieval APIs. It must be called when PSSE is at the power flow activity level and must follow each execution of activity SCMU. Can be followed by SCBUS2, SCBRN2, SC3WND and SCMAC2 calls. SCDONE must be called after executing these and before any other calls or PSSE activities can be executed.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = scinit()
```

Fortran command syntax:

```
CALL SCINIT(IERR)
```

IPLAN command syntax:

```
CALL SCINIT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Sequence data not in case
	IERR = 2	Unable to access SCMU results; one of the following occurred: -No SCMU results file is open. -SCMU results are not compatible with the working case. -Error reading the SCMU results file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.197. SCMAC2

Use this API to return the machine short circuit currents following activity SCMU.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = scmac2(ibus, ickt, string)
```

Fortran command syntax:

```
CALL SCMAC2(IBUS, ICKT, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL SCMAC2(IBUS, ICKT, STRING, P, Q, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ICKT	Machine identifier (input).	
Character STRING	String indicating the short circuit current desired (input).	
	'IPOS'	Positive sequence
	'INEG'	Negative sequence
	'IZERO'	Zero sequence
	'IA'	A-phase
	'IB'	B-phase
	'IC'	C-phase
Real P	Real component of the complex current (output).	
Real Q	Reactive component of the complex current (output).	
Complex CMPVAL	Complex current (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	'SCINIT' not successfully called; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Machine not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	Bad value of 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 5	Machine out of service; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.198. SELCTR

Use this API to return the activity selector indicator.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ival = selctr()
```

Fortran command syntax:

```
CALL SELCTR(IVAL)
```

IPLAN command syntax:

```
CALL SELCTR(IVAL)
```

Where:

Integer IVAL	Activity indicator (output).	
	IVAL = 0	Not at 'ACTIVITY'.
	IVAL = 1	At power flow activity selector.
	IVAL = 2	At dynamics activity selector.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.199. SETHOMEPATH

Use this API to set a user's HOMEPATH directory.

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_SETHOMPTH  SFILE
```

Python command syntax:

```
sethomepath(sfile)
```

Fortran command syntax:

```
CALL SETHOMPTH(SFILE)
```

IPLAN command syntax:

```
CALL SETHOMEPATH(SFILE)
```

Where:

Character*260 SFILE	user's HOMEPATH directory (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.200. SFILES

Use this API to return the current Saved Case and Snapshot filenames.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
savfilnam, snpfilnam = sfiles()
```

Fortran command syntax:

```
CALL SFILES(SAVFILNAM, SNPFILNAM)
```

IPLAN command syntax:

```
CALL SFILES(SAVFILNAM, SNPFILNAM)
```

Where:

Character*260 SAVFILNAM	Current Saved Case File (output).
Character*260 SNPFILNAM	Current Snapshot File (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.201. SLMODIND

Use this API to return the subsystem load-related model starting array indices and status.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = slmodind(ctype, ibus, id, string1, string2)
```

Fortran command syntax:

```
CALL SLMODIND(CTYPE, ID, IBUS, STRING1, STRING2, IVAL, IERR)
```

IPLAN command syntax:

```
CALL SLMODIND(CTYPE, ID, IBUS, STRING1, STRING2, IVAL, IERR)
```

Where:

Character CTYPE	Subsystem type (input).	
	'BUS'	Subsystem specified by buses
	'AREA'	Subsystem specified by areas
	'ZONE'	Subsystem specified by zones
	'OWNER'	Subsystem specified by owners
	'ALL'	Subsystem of all loads
Integer IBUS	Number of bus, area, zone, or owner, depending on CTYPE. Not used if CTYPE='ALL' (input).	
Character ID	Load identifier (input).	
Character STRING1	String indicating the type of load model desired (input).	
	'CHARAC'	Load characteristic model
	'RELAY'	Load relay model
Character STRING2	String indicating the load model quantity desired (input).	
	'CON'	Starting CON index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect load status. For a model to be called during simulations the load must be in-service the bus type code must be 1 or 2 and the model's bypassed flag must be not bypassed
Integer IVAL	Load model index or status (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Invalid value of 'CTYPE'; 'IVAL' unchanged
	IERR = 2	Invalid value of 'STRING1'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	No model of type 'STRING1' for this load; 'IVAL' unchanged
IERR = 4	Invalid value of 'STRING2'; 'IVAL' unchanged
IERR = 5	Model does not use any of this data type; 'IVAL' = 0 returned
IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.202. SLMODNAM

Use this API to return the subsystem load-related model name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = slmodnam(ctype, ibus, id, string)
```

Fortran command syntax:

```
CALL SLMODNAM(CTYPE, IBUS, ID, STRING, CVAL, IERR)
```

IPLAN command syntax:

```
CALL SLMODNAM(CTYPE, IBUS, ID, STRING, CVAL, IERR)
```

Where:

Character CTYPE	Subsystem type (input).	
	'BUS'	Subsystem specified by buses
	'AREA'	Subsystem specified by areas
	'ZONE'	Subsystem specified by zones
	'OWNER'	Subsystem specified by owners
	'ALL'	Subsystem of all loads
Integer IBUS	Number of bus, area, zone, or owner, depending on CTYPE. Not used if CTYPE='ALL' (input).	
Character ID	Load identifier (input).	
Character STRING	String indicating the type of load model desired (input).	
	'CHARAC'	Load characteristic model
	'RELAY'	Load relay model
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Invalid value of 'CTYPE'; 'CVAL' returned
	IERR = 2	Invalid value of 'STRING'; 'CVAL' unchanged
	IERR = 3	No model of type 'STRING' for this load; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.203. SOLSTR

Use this API to return a string indicating the result of the last solution attempt.

This API routine was first introduced in release 30.1.0.

Python command syntax:

```
desc = solstr()
```

Fortran command syntax:

```
CALL SOLSTR(DESC)
```

IPLAN command syntax:

```
CALL SOLSTR(DESC)
```

Where:

Character*48 DESC	string containing the results of the last solution attempt (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.204. SOLVED

Use this API to check whether the last solution attempt reached tolerance. This is not a test for whether the working case is a solved case.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ival = solved()
```

Fortran command syntax:

```
CALL SOLVED(IVAL)
```

IPLAN command syntax:

```
CALL SOLVED(IVAL)
```

Where:

Integer IVAL	Solution convergence indicator (output).	
	IVAL = 0	Met convergence tolerance.
	IVAL = 1	Iteration limit exceeded.
	IVAL = 2	Blown up (only when non-divergent option disabled).
	IVAL = 3	Terminated by non-divergent option.
	IVAL = 4	Terminated by console interrupt.
	IVAL = 5	Singular Jacobian matrix or voltage of 0.0 detected.
	IVAL = 6	Inertial power flow dispatch error (INLF).
	IVAL = 7	OPF solution met convergence tolerance (NOPF).
	IVAL = 9	Solution not attempted.
	IVAL = 10	RSOL converged with Phase shift locked.
	IVAL = 11	RSOL converged with TOLN increased.
	IVAL = 12	RSOL converged with Y load conversion due to low voltage.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.205. SOLVED_CA

Use this API to check whether the last corrective action solution attempt reached tolerance.

This API routine was first introduced in release 33.3.0.

Python command syntax:

```
ival = solved_ca()
```

Fortran command syntax:

```
CALL SOLVED_CA(IVAL)
```

IPLAN command syntax:

```
CALL SOLVED_CA(IVAL)
```

Where:

Integer IVAL	Corrective action solution convergence indicator (output).	
	IVAL = 0	Removed all limit violations successfully
	IVAL = 1	Exceeded iteration limit (Number of AC load flows)
	IVAL = 2	Terminated with constraint violations
	IVAL = 3	Terminated because of insufficient controls
	IVAL = 4	Terminated because linear programming model cannot be setup
	IVAL = 5	Terminated because Linear Programming solution diverged
	IVAL = 6	Terminated because AC load flow exceeded iteration limit
	IVAL = 7	Terminated because AC load flow blown up (only when non-divergent option disabled)
	IVAL = 8	Terminated because AC load flow terminated by non-divergent option
	IVAL = 9	Terminated because AC load flow terminated by console interrupt
	IVAL = 10	Terminated because AC load flow terminated by singular jacobian matrix (FNSL) or voltage= 0.0 (FDNS)
	IVAL = 13	Terminated because AC load flow converged with stalled and/or tripped induction machines
	IVAL = 14	Solution not attempted

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.206. STADAT

Use this API routine to return real substation values.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, rval = stadat(ista,string)
```

Fortran command syntax:

```
CALL STADAT(ISTA, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL STADAT(ISTA, STRING, RVAL, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Character STRING	String indicating the quantity desired (input).	
	'LATI'	Substation latitude in degrees
	'LONG'	Substation longitude in degrees
	'SRG'	Substation grounding dc resistance in ohms
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Substation not found; 'RVAL' unchanged
	IERR = 2	Invalid value of 'STRING'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.207. STAINT

Use this API routine to return integer substation values.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, ival = staint(ista, string)
```

Fortran command syntax:

```
CALL STAINT(ISTA, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL STAINT(ISTA, STRING, IVAL, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Character STRING	String indicating the quantity desired (input).	
	'NODES'	Number of nodes in the substation
	'PARENTS'	Number of parent buses in the substation
	'SECTIONS'	Number of bus sections in the substation
	'SWDEVS'	Number of switching devices in the substation
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Substation not found; 'IVAL' unchanged
	IERR = 2	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.208. STANAME

Use this API routine to return a substation's name.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, string = staname(ista)
```

Fortran command syntax:

```
CALL STANAME(ISTA, STRING, IERR)
```

IPLAN command syntax:

```
CALL STANAME(ISTA, STRING, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Character*40 STRING	Substation name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Substation not found; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.209. STANODEINT

Use this API routine to return integer substation values.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, ival = stanodeint(ista, node, string)
```

Fortran command syntax:

```
CALL STANODEINT(ISTA, NODE, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL STANODEINT(ISTA, NODE, STRING, IVAL, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Integer NODE	Node number (input).	
Character STRING	String indicating the quantity desired (input).	
	'BUS'	Bus number of the bus section to which the node is connected
	'SECTION'	Section number of the bus section to which the node is connected
	'STATUS'	Node status
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Substation not found; 'IVAL' unchanged
	IERR = 2	Node not found; 'IVAL' unchanged
	IERR = 3	Invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.210. STANODENAME

Use this API routine to return the name of a substation node.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, string = stanodename(ista, node)
```

Fortran command syntax:

```
CALL STANODENAME(ISTA, NODE, STRING, IERR)
```

IPLAN command syntax:

```
CALL STANODENAME(ISTA, NODE, STRING, IERR)
```

Where:

Integer ISTA	Substation number (input).	
Integer NODE	Node number (input).	
Character*40 STRING	Node name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Substation not found; 'STRING' unchanged
	IERR = 2	Node not found; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.211. STAREA

Use this API to restrict 'NXTBUS' to a single area. It must be called after INIBUS / INIBUX / ININAM / ININAX.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = starea(iar)
```

Fortran command syntax:

```
CALL STAREA(IAR, IERR)
```

IPLAN command syntax:

```
CALL STAREA(IAR, IERR)
```

Where:

Integer IAR	Area number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid area number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.212. STASWDEVDAT

Use this API routine to return real substation switching device values.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, rval = staswdevdat(ista, inode, jnode, id, string)
```

Fortran command syntax:

```
CALL STASWDEVDAT(ISTA, INODE, JNODE, ID, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL STASWDEVDAT(ISTA, INODE, JNODE, ID, STRING, RVAL, IERR)
```

Where:

Integer ISTA	substation number (input).	
Integer INODE	"from" node number (input).	
Integer JNODE	"to" node number (input).	
Character*2 ID	switching device identifier (input).	
Character STRING	string indicating the quantity desired (input).	
	'X'	switching device reactance
	'RATE1'	switching device first rating
	'RATE2'	switching device second rating
	'RATE3'	switching device third rating
	'RATE4'	switching device forth rating
	'RATE5'	switching device fifth rating
	'RATE6'	switching device sixth rating
	'RATE7'	switching device seventh rating
	'RATE8'	switching device eighth rating
	'RATE9'	switching device ninth rating
	'RATE10'	switching device tenth rating
	'RATE11'	switching device eleventh rating
	'RATE12'	switching device twelfth rating
	'AMPRATE1'	switching device first rating in AMP
	'AMPRATE2'	switching device second rating in AMP
	'AMPRATE3'	switching device third rating in AMP
	'AMPRATE4'	switching device forth rating in AMP
	'AMPRATE5'	switching device fifth rating in AMP
	'AMPRATE6'	switching device sixth rating in AMP
	'AMPRATE7'	switching device seventh rating in AMP
	'AMPRATE8'	switching device eighth rating in AMP
	'AMPRATE9'	switching device ninth rating in AMP

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'AMPRATE10'	switching device tenth rating in AMP
	'AMPRATE11'	switching device eleventh rating in AMP
	'AMPRATE12'	switching device twelfth rating in AMP
	'AMPS'	switching device current in amps
	'PUCUR'	switching device current in pu
	'P'	MW flow at the INODE end
	'O_P'	same as P
	'Q'	MVAR flow at the INODE end
	'O_Q'	same as Q
	'MVA'	switching device MVA loading
	'O_MVA'	same as MVA
	'PCTRT1'	percent current loading of RATE1
	'PCTRT2'	percent current loading of RATE2
	'PCTRT3'	percent current loading of RATE3
	'PCTRT4'	percent current loading of RATE4
	'PCTRT5'	percent current loading of RATE5
	'PCTRT6'	percent current loading of RATE6
	'PCTRT7'	percent current loading of RATE7
	'PCTRT8'	percent current loading of RATE8
	'PCTRT9'	percent current loading of RATE9
	'PCTRT10'	percent current loading of RATE10
	'PCTRT11'	percent current loading of RATE11
	'PCTRT12'	percent current loading of RATE12
	'PCTMV1'	percent MVA loading of RATE1
	'PCTMV2'	percent MVA loading of RATE2
	'PCTMV3'	percent MVA loading of RATE3
	'PCTMV4'	percent MVA loading of RATE4
	'PCTMV5'	percent MVA loading of RATE5
	'PCTMV6'	percent MVA loading of RATE6
	'PCTMV7'	percent MVA loading of RATE7
	'PCTMV8'	percent MVA loading of RATE8
	'PCTMV9'	percent MVA loading of RATE9
	'PCTMV10'	percent MVA loading of RATE10
	'PCTMV11'	percent MVA loading of RATE11
	'PCTMV12'	percent MVA loading of RATE12
Real RVAL	value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'RVAL' returned
	IERR = 1	substation not found; 'RVAL' unchanged
	IERR = 2	node not found; 'RVAL' unchanged
	IERR = 3	switching device not found; 'RVAL' unchanged
	IERR = 4	invalid value of 'STRING'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.213. STASWDEVINT

Use this API routine to return integer substation switching device values.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, ival = staswdevint(ista, inode, jnode, id, string)
```

Fortran command syntax:

```
CALL STASWDEVINT(ISTA, INODE, JNODE, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL STASWDEVINT(ISTA, INODE, JNODE, ID, STRING, IVAL, IERR)
```

Where:

Integer ISTA	substation number (input).	
Integer INODE	"from" node number (input).	
Integer JNODE	"to" node number (input).	
Character*2 ID	switching device identifier (input).	
Character STRING	string indicating the quantity desired (input).	
	'TYPE'	switching device type (1 through 3)
	'STATUS'	switching device status
	'NSTATUS'	switching device normal status
Integer IVAL	value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'IVAL' returned
	IERR = 1	substation not found; 'IVAL' unchanged
	IERR = 2	node not found; 'IVAL' unchanged
	IERR = 3	switching device not found; 'IVAL' unchanged
	IERR = 4	invalid value of 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.214. STASWDEVNAME

Use this API routine to return the name of a substation switching device.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, string = staswdevname(ista, inode, jnode, id)
```

Fortran command syntax:

```
CALL STASWDEVNAME(ISTA, INODE, JNODE, ID, STRING, IERR)
```

IPLAN command syntax:

```
CALL STASWDEVNAME(ISTA, INODE, JNODE, ID, STRING, IERR)
```

Where:

Integer ISTA	substation number (input).	
Integer INODE	"from" node number (input).	
Integer JNODE	"to" node number (input).	
Character*2 ID	switching device identifier (input).	
Character*40 STRING	switching device name (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'STRING' returned
	IERR = 1	substation not found; 'STRING' unchanged
	IERR = 2	node not found; 'STRING' unchanged
	IERR = 3	switching device not found; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.215. STASWDEVRSETNAME

Use this API routine to return the rating set name of a substation switching device.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, string = staswdevrsetname(ista, inode, jnode, id)
```

Fortran command syntax:

```
CALL STASWDEVRSETNAME(ISTA, INODE, JNODE, ID, STRING, IERR)
```

IPLAN command syntax:

```
CALL STASWDEVRSETNAME(ISTA, INODE, JNODE, ID, STRING, IERR)
```

Where:

Integer ISTA	substation number (input).	
Integer INODE	"from" node number (input).	
Integer JNODE	"to" node number (input).	
Character*2 ID	switching device identifier (input).	
Character*40 STRING	switching device rating set name (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'STRING' returned
	IERR = 1	substation not found; 'STRING' unchanged
	IERR = 2	node not found; 'STRING' unchanged
	IERR = 3	switching device not found; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.216. STBSKV

Use this API to restrict 'NXTBUS' to a single base voltage level. It must be called after INIBUS / INIBUX / ININAM / ININAX.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = stbskv(bskv)
```

Fortran command syntax:

```
CALL STBSKV(BSKV, IERR)
```

IPLAN command syntax:

```
CALL STBSKV(BSKV, IERR)
```

Where:

Real BSKV	Base kV level (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Negative base voltage

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.217. STZONE

Use this API to restrict 'NXTBUS' to a single zone. It must be called after INIBUS / INIBUX / ININAM / ININAX.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr = stzone(iar)
```

Fortran command syntax:

```
CALL STZONE(IAR, IERR)
```

IPLAN command syntax:

```
CALL STZONE(IAR, IERR)
```

Where:

Integer IAR	Zone number (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid zone number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.218. SWD_RATING_DESC

Use this API to get switching device table rating description

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, rdesc = swd_rating_desc (rname)
```

Fortran command syntax:

```
CALL SWD_RATING_DESC (RNAME, RDESC, IERR)
```

IPLAN command syntax:

```
CALL SWD_RATING_DESC (RNAME, RDESC, IERR)
```

Where:

Character*40 RNAME	Name of the switching device rating table for which the required string has to be retrived. (input; no default allowed).	
Character RDESC	The Description of the given switching device rating table entry RNAME that is retrived. (output; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	NO ERROR; 'RDESC', RETURNED
	IERR = 9	INVALID RATING TABLE NAME; 'RDESC', DOES NOT EXIST

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.219. SWD_RATING_DESC_LENGTH

Use this API to get switching device table rating description length

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, ilen = swd_rating_desc_length (rname)
```

Fortran command syntax:

```
CALL SWD_RATING_DESC_LENGTH (RNAME, ILEN, IERR)
```

Where:

Character*40 RNAME	Name of the switching device rating table for which the required string has to be retrived. (input; no default allowed).	
Integer ILEN	length of the description of the given switching device rating table entry RNAME (output).	
Integer IERR	error code (output).	
	IERR = 0	NO ERROR; 'RDESC', RETURNED
	IERR = 9	INVALID RATING TABLE NAME; 'RDESC', DOES NOT EXIST

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.220. SWD_RATING_NAME_DATA

Use this API to return switching device rating name parameters.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr,strout = swd_rating_name_data(swdratset, strin)
```

Fortran command syntax:

```
CALL SWD_RATING_NAME_DATA(SWDRATSET,STRIN,STROUT,IERR)
```

IPLAN command syntax:

```
CALL SWD_RATING_NAME_DATA(SWDRATSET,STRIN,STROUT,IERR)
```

Where:

Integer SWDRATSET	switching device rating name set number (1 through 12) (input).	
Character STRIN	(input).	
	'NAME'	column heading (6 characters).
	'TYPE'	type (12 characters).
	'UNIT'	unit (3 characters).
	'DESC'	description (32 characters).
Character*32 STROUT	character string (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; STROUT' returned
	IERR = 1	invalid switching device rating name set number; 'STROUT' unchanged
	IERR = 2	bad value of 'STRIN'; 'STROUT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.221. SWD_RATING_VALUE

Use this API to get a rating from the switching device table rating set

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr,rating = swd_rating_value(rname, ratingnum)
```

Fortran command syntax:

```
CALL SWD_RATING_VALUE (RNAME, RATINGNUM, RATING, IERR)
```

IPLAN command syntax:

```
CALL SWD_RATING_VALUE (RNAME, RATINGNUM, RATING, IERR)
```

Where:

Character*40 RNAME	Name of the switching device rating table for which the rating value has to be retrived. (input; no default allowed).	
Integer RATINGNUM	Specify the required rating number. Possible values 1 thru 12. (input; no default allowed).	
Real RATING	The value of rating returned specified by the RNAME and RATINGNUM. (output; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	NO ERROR; 'RATING', RETURNED
	IERR = 9	INVALID RATING TABLE NAME; 'RATING', DOES NOT EXIST

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.222. SWSBLK

This API routine is obsolete. It has been replaced by the API routine SWSBLK_2 .

Use this API to return the switched shunt '1' number of steps and step size for block IBLK.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival, v = swsblk(ibus, iblk)
```

Fortran command syntax:

```
CALL SWSBLK(IBUS, IBLK, IVAL, V, IERR)
```

IPLAN command syntax:

```
CALL SWSBLK(IBUS, IBLK, IVAL, V, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IBLK	Switched shunt block number (input).	
Integer IVAL	Number of steps in block 'IBLK' (output).	
Real V	Admittance in Mvar per step in block 'IBLK' (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' and 'V' returned
	IERR > 0	SWSBLK_2 error code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.223. SWSBLK_2

Use this API to return data for a specified block of a specified switched shunt.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, istat, nstep, rval = swsblk_2(ibus, id, iblk, string)
```

Fortran command syntax:

```
CALL SWSBLK_2(IBUS, ID, IBLK, STRING, ISTAT, NSTEP, RVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSBLK_2(IBUS, ID, IBLK, STRING, ISTAT, NSTEP, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
character*2 ID	Switched shunt identifier (input).	
Integer IBLK	Switched shunt block number (1 through 8) (input).	
Character STRING	String indicating the switched shunt quantity desired (input).	
	'BSTEP'	Admittance in Mvar per step in block "IBLK"
	'PU_ZSTEP'	Zero sequence admittance in pu per step in block "IBLK"
	'MVAR_ZSTEP'	Zero sequence admittance in Mvar per step in block "IBLK"
Integer ISTAT	Status of block "IBLK" (output).	
Integer NSTEP	Number of steps in block "IBLK" (output).	
Real RVAL	Admittance value indicated by STRING, per step in block "IBLK" (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ISTAT', 'NSTEP' and 'RVAL' returned
	IERR = 1	Bus not found; 'ISTAT', 'NSTEP' and 'RVAL' unchanged
	IERR = 2	Switched shunt not found; 'ISTAT', 'NSTEP' and 'RVAL' unchanged
	IERR = 3	Bad value of 'IBLK'; 'ISTAT', 'NSTEP' and 'RVAL' unchanged
	IERR = 4	Bus type code greater than or equal to 4; 'ISTAT', 'NSTEP' and 'RVAL' returned
	IERR = 5	Status is 0; 'ISTAT', 'NSTEP' and 'RVAL' returned
	IERR = 6	Invalid 'STRING'; 'ISTAT', 'NSTEP' and 'RVAL' unchanged
	IERR = 7	Sequence data not in case when STRING is 'PU_ZSTEP' or 'MVAR_ZSTEP'; 'ISTAT', 'NSTEP' and 'RVAL' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.224. SWSBLZ

This API routine is obsolete. It has been replaced by the API routine SWSBLK_2 .

Use this API to return the switched shunt '1' number of steps and zero sequence step size in per unit for block IBLK.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival, v = swsblz(ibus, iblk)
```

Fortran command syntax:

```
CALL SWSBLZ(IBUS, IBLK, IVAL, V, IERR)
```

IPLAN command syntax:

```
CALL SWSBLZ(IBUS, IBLK, IVAL, V, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IBLK	Switched shunt block number (input).	
Integer IVAL	Number of steps in block 'IBLK' (output).	
Real V	Admittance in pu per step in block 'IBLK' (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' and 'V' returned
	IERR > 0	SWSBLK_2 error code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.225. SWSBLZ2

This API routine is obsolete. It has been replaced by the API routine SWSBLK_2 .

Use this API to return the switched shunt '1' number of steps and zero sequence step size in Mvar for block IBLK.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, ival, v = swsblz2(ibus, iblk)
```

Fortran command syntax:

```
CALL SWSBLZ2(IBUS, IBLK, IVAL, V, IERR)
```

IPLAN command syntax:

```
CALL SWSBLZ2(IBUS, IBLK, IVAL, V, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Integer IBLK	Switched shunt block number (input).	
Integer IVAL	Number of steps in block 'IBLK' (output).	
Real V	Admittance in Mvar per step in block 'IBLK' (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' and 'V' returned
	IERR > 0	SWSBLK_2 error code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.226. SWSBUS

Use this API to return the switched shunt bus and id by switched shunt name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, ibus, id = swsbus(name)
```

Fortran command syntax:

```
CALL SWSBUS(NAME, IBUS, ID, IERR)
```

IPLAN command syntax:

```
CALL SWSBUS(NAME, IBUS, ID, IERR)
```

Where:

Character*40 NAME	String indicating the switched shunt name (input).	
Integer IBUS	Bus number (output).	
Character*2 ID	switched shunt identifier (output; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error; switched shunt bus and 'ID' returned
	IERR = 1	Switched shunt not found
	IERR = 2	Switched shunt name is empty
	IERR = 3	Bus type code is not 1, 2 or 3; switched shunt bus and 'ID' returned
	IERR = 4	Switched shunt out-of-service; switched shunt bus and 'ID' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.227. SWSCHR

Use this API to return switched shunt character parameters.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, cval = swschr(ibus, id, string)
```

Fortran command syntax:

```
CALL SWSCHR(IBUS, ID, STRING, CVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSCHR(IBUS, ID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
character*2 ID	Switched shunt identifier (input).	
Character STRING	String indicating the switched shunt quantity desired (input).	
	'RMIDNT'	Name of regulated VSCDC or FACTS (12 characters).
	'SWSNAME'	Name of switched shunt (40 characters).
Character*40 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Bus not found; 'CVAL' unchanged
	IERR = 2	Switched shunt not found; 'CVAL' unchanged
	IERR = 3	Bad value of 'STRING'; 'CVAL' unchanged
	IERR = 4	Bus type code greater than or equal to 4; 'CVAL' returned
	IERR = 5	Status is 0; 'CVAL' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.228. SWSDAT

This API routine is obsolete. It has been replaced by the API routine SWSDAT_2 .

Use this API to return the switched shunt '1' scheduled voltage band.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, vhi, vlo = swsdat(ibus)
```

Fortran command syntax:

```
CALL SWSDAT(IBUS, VHI, VLO, IERR)
```

IPLAN command syntax:

```
CALL SWSDAT(IBUS, VHI, VLO, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Real VHI	Desired voltage upper limit (output).	
Real VLO	Desired voltage lower limit (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'VHI' and 'VLO' returned
	IERR > 0	SWSDAT_2 error code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.229. SWSDAT_2

Use this API to return switched shunt real data.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, rval = swsdat_2(ibus, id, string)
```

Fortran command syntax:

```
CALL SWSDAT_2(IBUS, ID, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSDAT_2(IBUS, ID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
character*2 ID	Switched shunt identifier (input).	
Character STRING	String indicating the switched shunt quantity desired (input).	
	'VSWHI'	Controlled quantity upper limit
	'VSWLO'	Controlled quantity lower limit
	'RMPCT'	Percentage of total Mvar required to regulate controlled bus voltage
	'BINIT'	Present admittance switched on in Mvar at unity voltage
	'O_BINIT'	Same as 'BINIT' units determined by power output option
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Switched shunt not found; 'RVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'RVAL' unchanged
	IERR = 4	Bus type code greater than or equal to 4; 'RVAL' returned
	IERR = 5	Status is 0; 'RVAL' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.230. SWSDT1

This API routine is obsolete. It has been replaced by the API routine SWSDAT_2 .

Use this API to return switched shunt '1' real data.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr, rval = swsdt1(ibus, string)
```

Fortran command syntax:

```
CALL SWSDT1(IBUS, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSDT1(IBUS, STRING, RVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character STRING	String indicating the switched shunt quantity desired (input).	
	'VSWHI'	Controlled quantity upper limit
	'VSWLO'	Controlled quantity lower limit
	'RMPCT'	Percentage of total Mvar required to regulate controlled bus voltage
	'BINIT'	Present admittance switched on in Mvar at unity voltage
	'O_BINIT'	Same as 'BINIT' units determined by power output option
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR > 0	SWSDAT_2 error code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.231. SWSINT

This API routine is obsolete. It has been replaced by the API routine SWSINT_2 .

Use this API to return switched shunt '1' integer parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = swsint(ibus, string)
```

Fortran command syntax:

```
CALL SWSINT(IBUS, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSINT(IBUS, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character STRING	String indicating the switched shunt quantity desired (input).	
	'STATION'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
	'SECTION'	Bus IBUS bus section number (returned as 0 if bus IBUS is not in a substation)
	'NODE'	Number of the node to which the switched shunt is connected (returned as 0 if bus IBUS is not in a substation)
	'MODE'	Control mode
	'STATUS'	Status flag
	'ADJM'	Adjustment method
	'REMOTE'	Bus number of regulated bus; zero for none
	'IREG'	Bus number of regulated bus; zero for none
	'NREG'	Node number of regulated node
	'BLOCKS'	Number of blocks
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR > 0	SWSINT_2 error code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.232. SWSINT_2

Use this API to return switched shunt integer parameters.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, ival = swsint_2(ibus, id, string)
```

Fortran command syntax:

```
CALL SWSINT_2(IBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSINT_2(IBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
character*2 ID	Switched shunt identifier (input).	
Character STRING	String indicating the switched shunt quantity desired (input).	
	'STATION'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
	'SECTION'	Bus IBUS bus section number (returned as 0 if bus IBUS is not represented by multiple bus sections)
	'NODE'	Number of the node to which the switched shunt is connected (returned as 0 if bus IBUS is not represented by multiple bus sections)
	'MODE'	Control mode
	'STATUS'	Status flag
	'ADJM'	Adjustment method
	'REMOTE'	Bus number of regulated bus; zero for none
	'IREG'	Bus number of regulated bus; zero for none
	'NREG'	Node number of regulated node
	'BLOCKS'	Number of blocks
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Switched shunt not found; 'IVAL' unchanged
	IERR = 3	Bad value of 'STRING'; 'IVAL' unchanged
	IERR = 4	Bus type code greater than or equal to 4; 'IVAL' returned
	IERR = 5	Status is 0; 'IVAL' returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.233. SWSMIND

This API routine is obsolete. It has been replaced by the API routine swsmind_2 .

In versions prior to 35.0, this API would return switched shunt model starting array indices and status. Starting version 35.0, this API is being retained only for reasons of backward compatibility of python scripts.
Use of this API is not recommended anymore as this will always pick the first switched shunt at a bus.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = swsmind(ibus, string)
```

Fortran command syntax:

```
CALL SWSMIND(IBUS, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSMIND(IBUS, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character STRING	String indicating the switched shunt model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect switched shunt status. For a model to be called during simulations the switched shunt must be in service the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Switched shunt not in network but in model tables; 'IVAL' returned
	IERR = 2	Switched shunt not in network or in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
IERR = 7	Bus not found; 'IVAL' unchanged
IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.234. SWSMIND_2

Use this API to return switched shunt model starting array indices and status.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, ival = swsmind_2(ibus, id, string)
```

Fortran command syntax:

```
CALL SWSMIND_2(IBUS, ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSMIND_2(IBUS, ID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character STRING	String indicating the switched shunt model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect switched shunt status. For a model to be called during simulations the switched shunt must be in service the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Switched shunt not in network but in model tables; 'IVAL' returned
	IERR = 2	Switched shunt not in network or in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 7	Bus not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.235. SWSMNAM

This API routine is obsolete. It has been replaced by the API routine swsmnam_2 .

In versions prior to 35.0, this API would return switched shunt model name.
Starting version 35.0, this API is being retained only for reasons of backward compatibility of python scripts.
Use of this API is not recommended anymore as this will always pick the first switched shunt at a bus.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = swsmnam(ibus)
```

Fortran command syntax:

```
CALL SWSMNAM(IBUS, CVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSMNAM(IBUS, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Switched shunt not in network but in model tables; 'CVAL' returned
	IERR = 2	Switched shunt not in network or in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables; 'CVAL' unchanged
	IERR = 7	Bus not found; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.236. SWSMNAM_2

Use this API to return switched shunt model name.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, cval = swsmnam_2(ibus,id)
```

Fortran command syntax:

```
CALL SWSMNAM_2(IBUS, ID, CVAL, IERR)
```

IPLAN command syntax:

```
CALL SWSMNAM_2(IBUS, ID, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character*2 ID	switched shunt identifier (input; blank by default).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Switched shunt not in network but in model tables; 'CVAL' returned
	IERR = 2	Switched shunt not in network or in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables; 'CVAL' unchanged
	IERR = 7	Bus not found; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.237. SYSMSM

Use this API to return the total system MVA mismatch.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
mva = sysmsm()
```

Fortran command syntax:

```
CALL SYSMSM(MVA)
```

IPLAN command syntax:

```
CALL SYSMSM(MVA)
```

Where:

Real MVA	Total system MVA mismatch (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.238. SYSMVA

Use this API to return the system base MVA.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
mvabas = sysmva()
```

Fortran command syntax:

```
CALL SYSMVA(MVABAS)
```

IPLAN command syntax:

```
CALL SYSMVA(MVABAS)
```

Where:

Real MVABAS	System base MVA (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.239. SYSSWDEVRSETNAM

Use this API to return the rating set name of a system switching device.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, string = sysswdevrsetnam(ibus, jbus, ickt)
```

Fortran command syntax:

```
CALL SYSSWDEVRSETNAM(IBUS, JBUS, ICKT, STRING, IERR)
```

IPLAN command syntax:

```
CALL SYSSWDEVRSETNAM(IBUS, JBUS, ICKT, STRING, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character ICKT	Circuit identifier (input).	
Character*40 STRING	System switching device rating set name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Bus not found; 'STRING' unchanged
	IERR = 2	Branch not found; 'STRING' unchanged
	IERR = 4	'SYSSWDEVRSETNAM' invalid with multisection line identifier; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.240. SYSTOT

Use this API to return complex system-wide values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = systot(string)
```

Fortran command syntax:

```
CALL SYSTOT(STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL SYSTOT(STRING, P, Q, IERR)
```

Where:

Character STRING	String indicating the system total desired (input).	
	'LOAD'	Total system load
	'GEN'	Total system generation
	'LOSS'	Total system losses; I**2 * (R + jX) for ac branches
	'XFRMAG'	Total power consumed by transformer magnetizing admittances
	'INDMAC'	Total system induction machine powers
	'INDGEN'	Total system induction generator powers
	'INDMOT'	Total system induction motor powers
	'FACTSH'	Total of FACTS device shunt elements
	'BSSHNT'	Total of bus fixed and switched shunt elements
	'LNSHNT'	Total of line shunts and transformer magnetizing shunt elements
	'ALSHNT'	Total of FACTS device fixed switched line and transformer magnetizing shunt elements
Real P	Real component of the complex power (output).	
Real Q	Reactive component of the complex power (output).	
Complex CMPVAL	Complex power (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bad 'STRING' value; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	No in-service buses in case; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.241. TITLDT

Use this API to return the two line case title.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
line1, line2 = titldt()
```

Fortran command syntax:

```
CALL TITLDT(LINE1, LINE2)
```

IPLAN command syntax:

```
CALL TITLDT(LINE1, LINE2)
```

Where:

Character*60 LINE1	First 60-character line (output).
Character*60 LINE2	Second 60-character line (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.242. TOTBUS

Use this API to return the total number of buses in the working case. Dummy buses of multisection lines and star point buses of three-winding transformers are included.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
n = totbus()
```

Fortran command syntax:

```
CALL TOTBUS(N)
```

IPLAN command syntax:

```
CALL TOTBUS(N)
```

Where:

Integer N	Number of buses in the working case. If the working case is empty, then the number of buses returned is zero (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.243. TR3DAT

Use this API to return three-winding transformer real values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = tr3dat(ibus, jbus, kbus, ickt, string)
```

Fortran command syntax:

```
CALL TR3DAT(IBUS, JBUS, KBUS, ICKT, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL TR3DAT(IBUS, JBUS, KBUS, ICKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Integer KBUS	Third bus number (input).	
Character ICKT	Circuit identifier (input).	
Character STRING	String indicating the three-winding transformer quantity desired (input).	
	'FRACT1'	Fraction of total ownership assigned to the first owner
	'FRACT2'	Fraction of total ownership assigned to the second owner
	'FRACT3'	Fraction of total ownership assigned to the third owner
	'FRACT4'	Fraction of total ownership assigned to the fourth owner
	'VMSTAR'	Star bus voltage magnitude
	'ANSTAR'	Star bus voltage angle in degrees
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Three-winding transformer not found; 'RVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'RVAL' unchanged
	IERR = 4	Error fetching transformer data; 'RVAL' unchanged
	IERR = 6	With STRING = FRACTn, n > number of owners; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.244. TR3DT2

Use this API to return three-winding transformer complex values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = tr3dt2(ibus, jbus, kbus, ickt, string)
```

Fortran command syntax:

```
CALL TR3DT2(IBUS, JBUS, KBUS, ICKT, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL TR3DT2(IBUS, JBUS, KBUS, ICKT, STRING, P, Q, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Integer KBUS	Third bus number (input).	
Character ICKT	Circuit identifier (input).	
Character STRING	String indicating the three-winding transformer quantity desired (input).	
	'RX1-2'	Measured impedance in per unit between the winding 1 and 2 buses
	'RX2-3'	Measured impedance in per unit between the winding 2 and 3 buses
	'RX3-1'	Measured impedance in per unit between the winding 3 and 1 buses
	'YMAGNT'	Magnetizing admittance in pu
	'ZG1', 'ZGRND'	Zero-sequence winding 1 grounding impedance
	'Z01'	Zero-sequence winding 1 or winding 1 bus to winding 2 bus leakage impedance
	'ZG2'	Zero-sequence winding 2 grounding impedance
	'Z02'	Zero-sequence winding 2 or winding 2 bus to winding 3 bus leakage impedance
	'ZG3'	Zero-sequence winding 3 grounding impedance
	'Z03'	Zero-sequence winding 3 or winding 3 bus to winding 1 bus leakage impedance
	'ZNUTRL'	Zero-sequence common neutral grounding impedance
Real P	Real component of complex value indicated by STRING (output).	
Real Q	Reactive component of complex value indicated by STRING (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Complex CMPVAL	Complex value indicated by STRING (output).
Integer IERR	error code (output).

IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 2	Three-winding transformer not found; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 3	Invalid 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 4	Error fetching transformer data; 'P' and 'Q' or 'CMPVAL' unchanged
IERR = 5	Sequence data not in case for 'STRING' = 'ZGRND', 'ZG1', 'Z01', 'ZG2', ,'Z02', 'ZG3', 'Z03', or 'ZNUTRL'; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.245. TR3INT

Use this API to return three-winding transformer integer values.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = tr3int(ibus, jbus, kbus, ickt, string)
```

Fortran command syntax:

```
CALL TR3INT(IBUS, JBUS, KBUS, ICKT, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL TR3INT(IBUS, JBUS, KBUS, ICKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).
Integer JBUS	To bus number (input).
Integer KBUS	Third bus number (input).
Character ICKT	Circuit identifier (input).
Character STRING	String indicating the three-winding transformer quantity desired (input).

'STATUS'	Three-winding transformer status
'NMETER'	Non-metered bus number
'OWNERS'	Number of owners
'OWN1'	Owner number of first owner
'OWN2'	Owner number of second owner
'OWN3'	Owner number of third owner
'OWN4'	Owner number of fourth owner
'CW'	Winding data I/O code
'CZ'	Impedance data I/O code
'CM'	Magnetizing admittance data I/O code
'CZ0'	Leakage impedance data I/O code
'CZG'	Grounding impedance data I/O code
'CNXCOD'	Winding connection code
'ZADCOD'	Impedance adjustment method code
'STATION_I'	Bus IBUS substation number (returned as 0 if bus IBUS is not in a substation)
'STATION_J'	Bus JBUS substation number (returned as 0 if bus JBUS is not in a substation)
'STATION_K'	Bus KBUS substation number (returned as 0 if bus KBUS is not in a substation)
'SECTION_I'	Bus IBUS bus section number (returned as 0 if bus IBUS is not in a substation)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'SECTION_J'	Bus JBUS bus section number (returned as 0 if bus JBUS is not in a substation)
	'SECTION_K'	Bus KBUS bus section number (returned as 0 if bus KBUS is not in a substation)
	'NODE_I'	Number of the node to which the winding is connected at the bus IBUS end (returned as 0 if bus IBUS is not in a substation)
	'NODE_J'	Number of the node to which the winding is connected at the bus JBUS end (returned as 0 if bus JBUS is not in a substation)
	'NODE_K'	Number of the node to which the winding is connected at the bus KBUS end (returned as 0 if bus KBUS is not in a substation)
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Three-winding transformer not found; 'IVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'IVAL' unchanged
	IERR = 4	Error fetching transformer data; 'IVAL' unchanged
	IERR = 5	Sequence data not in case for 'STRING' = 'CZ0', 'CZG' or 'CNXCOD'; 'IVAL' unchanged
	IERR = 6	With 'STRING' = OWNn, n > number of owners; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.246. TR3NAM

Use this API to return three-winding transformer name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = tr3nam(ibus, jbus, kbus, ickt)
```

Fortran command syntax:

```
CALL TR3NAM(IBUS, JBUS, KBUS, ICKT, CVAL, IERR)
```

IPLAN command syntax:

```
CALL TR3NAM(IBUS, JBUS, KBUS, ICKT, CVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Integer KBUS	Third bus number (input).	
Character ICKT	Circuit identifier (input).	
Character*40 CVAL	Transformer name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Bus not found; 'CVAL' unchanged
	IERR = 2	Three-winding transformer not found; 'CVAL' unchanged
	IERR = 4	Error fetching transformer data; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.247. TRNDAT

This API routine is obsolete. It has been replaced by the API routine XFRDAT .

Return transformer Ratio and Angle.

This API routine was first introduced in release 30.3.3.

Python command syntax:

```
ierr, ratio, angle = trndat(ibus, jbus, ckt)
```

Fortran command syntax:

```
CALL TRNDAT(IBUS, JBUS, CKT, RATIO, ANGLE, IERR)
```

IPLAN command syntax:

```
CALL TRNDAT(IBUS, JBUS, CKT, RATIO, ANGLE, IERR)
```

Where:

Integer IBUS	from bus number (input).	
Integer JBUS	to bus number (input).	
Character*2 CKT	circuit identifier (input).	
Real RATIO	tap position (output).	
Real ANGLE	phase shift angle, in degrees (output).	
Integer IERR	error code (output).	
	IERR = 0	no error; 'RATIO' and 'ANGLE' returned
	IERR = 1	bus not found; 'RATIO' and 'ANGLE' unchanged
	IERR = 2	branch not found; 'RATIO' and 'ANGLE' unchanged
	IERR = 3	branch not a transformer; 'RATIO' and 'ANGLE' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.248. TRXDAT

Use this API to return the inter-area transfer MW.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr, rval = trxdat(iar, jar, id)
```

Fortran command syntax:

```
CALL TRXDAT(IAR, JAR, ID, RVAL, IERR)
```

IPLAN command syntax:

```
CALL TRXDAT(IAR, JAR, ID, RVAL, IERR)
```

Where:

Integer IAR	From area number (input).	
Integer JAR	To area number (input).	
Character*1 ID	Inter-area transfer identifier (input).	
Real RVAL	Value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Area number <= 0 or > largest allowable area number; 'RVAL' unchanged
	IERR = 2	Interarea transfer not found; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.249. TSPFPROFILECHR

Use this API to return string parameters of Time Series Power Flow profile.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, astring = tspfprofilechr(name,string)
```

Fortran command syntax:

```
CALL TSPFPROFILECHR(NAME, STRING, ASTRING, IERR)
```

IPLAN command syntax:

```
CALL TSPFPROFILECHR(NAME, STRING, ASTRING, IERR)
```

Where:

Character*160 NAME	Profile name (input).	
Character*160 STRING	String indicating name of the string parameter desired. (input).	
	'ELEMENT'	Element key
	'FIELD'	Field key
Character*160 ASTRING	Value indicated by input STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Profile not found; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.250. TSPFPROFILECURVECHR

Use this API to return string parameters of Time Series Power Flow profile curve.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, astring = tspfprofilecurvechr(curveid,string)
```

Fortran command syntax:

```
CALL TSPFPROFILECURVECHR(CURVEID, STRING, ASTRING, IERR)
```

IPLAN command syntax:

```
CALL TSPFPROFILECURVECHR(CURVEID, STRING, ASTRING, IERR)
```

Where:

Integer CURVEID	Curve ID (input).	
Character*160 STRING	String indicating name of the string parameter desired. (input).	
	'START'	Start time
Character*160 ASTRING	Value indicated by input STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Curveid not found; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.251. TSPFPROFILECURVEINT

Use this API to return integer parameters of Time Series Power Flow profile curve.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, ival = tspfprofilecurveint(curveid,string)
```

Fortran command syntax:

```
CALL TSPFPROFILECURVEINT(CURVEID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL TSPFPROFILECURVEINT(CURVEID, STRING, IVAL, IERR)
```

Where:

Integer CURVEID	Curve ID (input).	
Character*160 STRING	String indicating name of the integer parameter desired. (input).	
	'INTERVAL'	Time interval
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Curveid not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.252. TSPFPROFILECURVEREAL

Use this API to return real parameters of Time Series Power Flow profile curve.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, rval = tspfprofilecurvereal(curveid,string)
```

Fortran command syntax:

```
CALL TSPFPROFILECURVEREAL(CURVEID, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL TSPFPROFILECURVEREAL(CURVEID, STRING, RVAL, IERR)
```

Where:

Integer CURVEID	Curve ID (input).	
Character*160 STRING	String indicating name of the real parameter desired. (input).	
	'INTMULT'	Time Interval Multiplier
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Curveid not found; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.253. TSPFPROFILECURVEVALUE

Use this API to return real parameters of Time Series Power Flow profile data.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, rval = tspfprofilecurvevalue(curvesetid,curvecol,index,string)
```

Fortran command syntax:

```
CALL TSPFPROFILECURVEVALUE(CURVESETID,CURVECOL, INDEX, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL TSPFPROFILECURVEVALUE(CURVESETID,CURVECOL, INDEX, STRING, RVAL, IERR)
```

Where:

Integer CURVESETID	Curve Set ID (input).	
Integer CURVECOL	Curve Column (input).	
Integer INDEX	Index (input).	
Character*160 STRING	String indicating name of the real parameter desired. (input).	
	'VALUE'	Profile value indicated by index
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Curveid not found; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.254. TSPFPROFILEINT

Use this API to return integer parameters of Time Series Power Flow profile.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, ival = tspfprofileint(name,string)
```

Fortran command syntax:

```
CALL TSPFPROFILEINT(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL TSPFPROFILEINT(NAME, STRING, IVAL, IERR)
```

Where:

Character*160 NAME	Profile name (input).	
Character*160 STRING	String indicating name of the integer parameter desired. (input).	
	'MWLIMITS'	Enforce Machine MW Limits
	'CURVEID'	Profile Curve Number
	'ADJMODE'	Profile Adjustment Mode
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Profile not found; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.255. TSPFPROFILEREAL

Use this API to return real parameters of Time Series Power Flow profile.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, rval = tspfprofilereal(name,string)
```

Fortran command syntax:

```
CALL TSPFPROFILEREAL(NAME, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL TSPFPROFILEREAL(NAME, STRING, RVAL, IERR)
```

Where:

Character*160 NAME	Profile name (input).	
Character*160 STRING	String indicating name of the real parameter desired. (input).	
	'OFFSET'	Offset
	'SCALEFACTOR'	Scale factor
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Profile not found; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.256. VOLTAGEDROOPDATA

Use this API to return voltage droop control real parameters.

This API routine was first introduced in release 35.3.0.

Python command syntax:

```
ierr, rval = voltagedroopdata(name, string)
```

Fortran command syntax:

```
CALL VOLTAGEDROOPDATA(NAME, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL VOLTAGEDROOPDATA(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	voltage droop control name (input).	
Character STRING	String indicating the voltage droop control quantity desired (input).	
	'STATUS'	voltage droop control status
	'REGBUS'	regulated bus of voltage droop control
Real RVAL	Parameter value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	voltage droop control 'NAME' not found; 'RVAL' unchanged
	IERR = 2	Invalid 'STRING'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.257. VOLTAGEDROOPINT

Use this API to return voltage droop control integer parameters.

This API routine was first introduced in release 35.3.0.

Python command syntax:

```
ierr, ival = voltagedroopint(name, string)
```

Fortran command syntax:

```
CALL VOLTAGEDROOPINT(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL VOLTAGEDROOPINT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	voltage droop control name (input).	
Character STRING	String indicating the voltage droop control quantity desired (input).	
	'STATUS'	voltage droop control status
	'REGBUS'	regulated bus of voltage droop control
	'ACTSTATUS'	actual voltage droop control status
	'ACTREGBUS'	actual regulated bus of voltage droop control
Integer IVAL	Parameter value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	voltage droop control 'NAME' not found; 'IVAL' unchanged
	IERR = 2	Invalid 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.258. VSCAUXMIND

Use this API to return the starting array indices and status of auxiliary signal model associated with VSC dc line for the specified signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = vscauxmind(name, isgndx, string)
```

Fortran command syntax:

```
CALL VSCAUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL VSCAUXMIND(NAME, ISGNDX, STRING, IVAL, IERR)
```

Where:

Character NAME	VSC dc line name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character STRING	String indicating the VSC dc line auxiliary signal model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. For a model to be called during simulations the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not in network but auxiliary signal model in model tables; 'IVAL' returned
	IERR = 2	dc line not in network and auxiliary signal model not in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.259. VSCAUXMNAM

Use this API to return the auxiliary signal model name associated with a VSC dc line for the specified auxiliary signal index.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = vscauxmnam(name, isgndx)
```

Fortran command syntax:

```
CALL VSCAUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

IPLAN command syntax:

```
CALL VSCAUXMNAM(NAME, ISGNDX, CVAL, IERR)
```

Where:

Character NAME	VSC dc line name (input).	
Integer ISGNDX	Auxiliary signal index (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	dc line not in network but auxiliary signal model in model tables; 'CVAL' returned
	IERR = 2	dc line not in network and auxiliary signal model not in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables for the given auxiliary signal index; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.260. VSCCDT

Use this API to return VSC dc line real parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = vsccdt(id, ibus, string)
```

Fortran command syntax:

```
CALL VSCCDT(ID, IBUS, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL VSCCDT(ID, IBUS, STRING, RVAL, IERR)
```

Where:

Character ID	VSC dc line name (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the VSC dc line quantity desired (input).	
	'PAC'	Real power into dc line at converter bus in MW
	'O_PAC'	Same as 'PAC' in units determined by the power output option setting
	'QAC'	Reactive power into dc line at converter bus in MV
	'O_QAC'	Same as 'QAC' in units determined by the power output option setting
	'MVA'	Apparent ac flow into the dc line in MVA
	'O_MVA'	Same as 'MVA' in units determined by the power output option setting
	'PCTMVA'	ac loading percent of limit
	'ACAMPS'	ac current in amps
	'PUCUR'	ac current in pu
	'KVDC'	Voltage at controlling converter in kV
	'DCCUR'	dc current in amps
	'DCSET'	dc setpoint in pu
	'ACSET'	ac setpoint in pu
	'RDC'	dc line resistance in ohms
Real RVAL	Parameter value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	VSC dc line 'ID' not found; 'RVAL' unchanged
	IERR = 2	Invalid 'STRNG'; 'RVAL' unchanged
	IERR = 3	Bus 'IBUS' not a converter of this dc line; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	dc line blocked; 'RVAL' returned as 0.0
IERR = 5	No base kV at bus; 'RVAL' returned as the per unit current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.261. VSCCIN

Use this API to return VSC dc line converter integer parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = vsccin(id, ibus, string)
```

Fortran command syntax:

```
CALL VSCCIN(ID, IBUS, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL VSCCIN(ID, IBUS, STRING, IVAL, IERR)
```

Where:

Character ID	VSC dc line name (input).	
Integer IBUS	Converter bus number (input).	
Character STRING	String indicating the VSC dc line quantity desired (input).	
	'DCTYPE'	Converter dc control code
	'ACMODE'	Converter ac control mode
	'VSREG'	Regulated bus
	'NREG'	Regulated node
Integer IVAL	Parameter value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	VSC dc line 'ID' not found; 'IVAL' unchanged
	IERR = 2	Invalid 'STRING'; 'IVAL' unchanged
	IERR = 3	Bus 'IBUS' not a converter of this dc line; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.262. VSCINT

Use this API to return VSC dc line integer parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = vscint(id, string)
```

Fortran command syntax:

```
CALL VSCINT(ID, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL VSCINT(ID, STRING, IVAL, IERR)
```

Where:

Character ID	VSC dc line name (input).	
Character STRING	String indicating the VSC dc line quantity desired (input).	
	'CONV1'	Converter 1 bus number
	'CONV2'	Converter 2 bus number
	'MDC'	Control mode
Integer IVAL	Parameter value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	VSC dc line 'ID' not found; 'IVAL' unchanged
	IERR = 2	Invalid 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.263. VSCMIND

Use this API to return VSC dc line model starting array indices and status.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = vscmind(name, string)
```

Fortran command syntax:

```
CALL VSCMIND(NAME, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL VSCMIND(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	VSC dc line name (input).	
Character STRING	String indicating the VSC dc line model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect dc line status. For a model to be called during simulations the dc line must be in service the model's bypassed flag must be not bypassed
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	dc line not in network but in model tables; 'IVAL' returned
	IERR = 2	dc line not in network or in model tables; 'IVAL' unchanged
	IERR = 4	No model in model tables; 'IVAL' unchanged
	IERR = 5	Invalid value of 'STRING'; 'IVAL' unchanged
	IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
	IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.264. VSCMNAM

Use this API to return VSC dc line model name.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = vscmnam(name)
```

Fortran command syntax:

```
CALL VSCMNAM(NAME, CVAL, IERR)
```

IPLAN command syntax:

```
CALL VSCMNAM(NAME, CVAL, IERR)
```

Where:

Character NAME	VSC dc line name (input).	
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	dc line not in network but in model tables; 'CVAL' returned
	IERR = 2	dc line not in network or in model tables; 'CVAL' unchanged
	IERR = 4	No model in model tables; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.265. WINDMIND

Use this API to return wind model starting array indices and status.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, ival = windmind(ibus, id, string1, string2)
```

Fortran command syntax:

```
CALL WINDMIND(IBUS, ID, STRING1, STRING2, IVAL, IERR)
```

IPLAN command syntax:

```
CALL WINDMIND(IBUS, ID, STRING1, STRING2, IVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING1	String indicating the type of wind model (input).	
	'WGEN'	Renewable Generator model
	'WELEC'	Renewable Electrical model
	'WMECH'	Renewable Mechanical model
	'WPICH or WPITCH'	Wind Pitch model
	'WAERO'	Wind Aerodynamic model
	'WGUST'	Wind Gust Model
	'WAUX'	Renewable Auxiliary Signal model
	'WTRQ'	Wind Torque Control model
	'WWGO'	Renewable Weak Grid model
	'WPAUX'	Renewable Paux Control model
Character STRING2	String indicating the plant-related model quantity desired (input).	
	'CON'	Starting CON index
	'STATE'	Starting STATE index
	'VAR'	Starting VAR index
	'ICON'	Starting ICON index
	'NCON'	Number of CON values used by the model
	'NSTATE'	Number of STATE values used by the model
	'NVAR'	Number of VAR values used by the model
	'NICON'	Number of ICON values used by the model
	'STATUS'	Bypassed flag: 0 = bypassed; 1 = not bypassed. Does not reflect machine status. For a model to be called during simulations the machine must be in service the type code of the bus must be 2 and the model's bypassed flag must be not bypassed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IVAL	Value indicated by STRING (output).
Integer IERR	error code (output).

IERR = 0	No error; 'IVAL' returned
IERR = 1	Machine not in network but in model tables; 'IVAL' returned
IERR = 2	Machine not in network or in model tables; 'IVAL' unchanged
IERR = 3	Invalid value of 'STRING1', 'IVAL' unchanged
IERR = 4	No model of type 'STRING1' for this machine; 'IVAL' unchanged
IERR = 5	Invalid value of 'STRING2'; 'IVAL' unchanged
IERR = 6	Model does not use any of this data type; 'IVAL' = 0 returned
IERR = 9	Dynamics data not present in working memory; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.266. WINDMNAM

Use this API to return wind model name.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, cval = windmnam(ibus, id, string)
```

Fortran command syntax:

```
CALL WINDMNAM(IBUS, ID, STRING, CVAL, IERR)
```

IPLAN command syntax:

```
CALL WINDMNAM(IBUS, ID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	Bus number (input).	
Character ID	Machine identifier (input).	
Character STRING	String indicating the type of plant-related model (input).	
	'WGEN'	Renewable Generator model
	'WELEC'	Renewable Electrical model
	'WMECH'	Renewable Mechanical model
	'WPICH or WPITCH'	Wind Pitch model
	'WAERO'	Wind Aerodynamic model
	'WGUST'	Wind Gust Model
	'WAUX'	Renewable Auxiliary Signal model
	'WTRQ'	Wind Torque Control model
	'WWGO'	Renewable Weak Grid model
	'WPAUX'	Renewable Paux Control model
Character*16 CVAL	Model name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Machine not in network but in model tables; 'CVAL' returned
	IERR = 2	Machine not in network or in model tables; 'CVAL' unchanged
	IERR = 3	Invalid value of 'STRING', 'CVAL' unchanged
	IERR = 4	No model of type 'STRING' for this machine; 'CVAL' unchanged
	IERR = 9	Dynamics data not present in working memory; 'CVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.267. WNDDAT

Use this API to return three-winding transformer real values for winding connected to bus IBUS.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = wnddat(ibus, jbus, kbus, ickt, string)
```

Fortran command syntax:

```
CALL WNDDAT(IBUS, JBUS, KBUS, ICKT, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL WNDDAT(IBUS, JBUS, KBUS, ICKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).
Integer JBUS	To bus number (input).
Integer KBUS	Third bus number (input).
Character ICKT	Circuit identifier (input).
Character STRING	String indicating the transformer winding quantity desired (input).

'RATEA'	Rating set A
'RATEB'	Rating set B
'RATEC'	Rating set C
'RATE'	Default rating set
'RATIO'	Winding tap ratio
'ANGLE'	Winding phase angle shift in degrees
'SBASE'	Winding MVA base
'NOMV'	Nominal winding voltage
'RMAX'	Ratio or angle high limit
'RMIN'	Ratio or angle low limit
'VMAX'	Controlled quantity high limit
'VMIN'	Controlled quantity low limit
'STEP'	Derived tap step
'CNXANG'	Winding connection angle in degrees
'MVA'	Winding MVA flow
'O_MVA'	Same as 'MVA' in units determined by the power output option setting
'AMPS'	Winding current in amps
'PUCUR'	Winding current in pu
'PCTRTA'	Percent current loading of RATEA
'PCTRTB'	Percent current loading of RATEB
'PCTRTC'	Percent current loading of RATEC
'PCTMVA'	Percent MVA loading of RATEA

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PCTMVB'	Percent MVA loading of RATEB
	'PCTMVC'	Percent MVA loading of RATEC
	'PCTCPA'	Percent current or MVA loading (according to the transformer percent loading units program option setting) of RATEA
	'PCTCPB'	Percent current or MVA loading (according to the transformer percent loading units program option setting) of RATEB
	'PCTCPC'	Percent current or MVA loading (according to the transformer percent loading units program option setting) of RATEC
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Three-winding transformer not found; 'RVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'RVAL' unchanged
	IERR = 4	Error fetching transformer data; 'RVAL' unchanged
	IERR = 7	Branch out of service; 'RVAL' returned set to zero
	IERR = 8	No base kV at IBUS; 'RVAL' returned = per unit current
	IERR = 9	Rating is zero; 'RVAL' returned set to zero

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.268. WNDDT2

Use this API to return three-winding transformer complex values for winding connected to bus IBUS.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = wnddt2(ibus, jbus, kbus, ickt, string)
```

Fortran command syntax:

```
CALL WNDDT2(IBUS, JBUS, KBUS, ICKT, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL WNDDT2(IBUS, JBUS, KBUS, ICKT, STRING, P, Q, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Integer KBUS	Third bus number (input).	
Character ICKT	Circuit identifier (input).	
Character STRING	String indicating the transformer winding quantity desired (input).	
	'RX'	Winding impedance in per unit
	'RXZ'	Zero-sequence winding impedance
	'COMPRX'	Compensating impedance
	'FLOW'	Complex flow in MW/Mvar
	'O_FLOW'	Same as 'FLOW' units determined by power output option
Real P	Real component of complex value indicated by STRING (output).	
Real Q	Reactive component of complex value indicated by STRING (output).	
Complex CMPVAL	Complex value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Bus not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	Three-winding transformer not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	Error fetching transformer data; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 5	Sequence data not in case for 'STRING' = 'RXZ'; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 7	Branch out of service; 'P' and 'Q' of 0.0 or 'CMPVAL' of (0.0,0.0) returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.269. WNDINT

Use this API to return three-winding transformer integer values for winding connected to bus IBUS.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = wndint(ibus, jbus, kbus, ickt, string)
```

Fortran command syntax:

```
CALL WNDINT(IBUS, JBUS, KBUS, ICKT, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL WNDINT(IBUS, JBUS, KBUS, ICKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Integer KBUS	Third bus number (input).	
Character ICKT	Circuit identifier (input).	
Character STRING	String indicating the transformer winding quantity desired (input).	
	'STATUS'	Status
	'CNXCOD'	Winding connection code
	'WNDNUM'	Winding number
	'ICONT'	Controlled bus number
	'SIDCOD'	Controlled bus side code (negative if on winding 1 side)
	'NCONT'	Controlled node number
	'TABLE'	Transformer impedance correction table number (0 if none)
	'CNTRL'	Adjustment flag
	'CNTRL2'	Transformer adjustment code
	'NTPOSN'	Number of tap positions
	'TPSTT'	Transformer winding ratio limit code: (-1) low; (0) regulating; (1) high; (-2) locked
	'ANSTT'	Transformer winding angle limit code: (-1) low; (0) regulating; (1) high; (-2) locked
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Bus not found; 'IVAL' unchanged
	IERR = 2	Three-winding transformer not found; 'IVAL' unchanged
	IERR = 3	Invalid 'STRING'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Error fetching transformer data; 'IVAL' unchanged
IERR = 5	Sequence data not in case for 'STRING' = 'CNXCOD'; 'IVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.270. XFRDAT

Use this API to return two-winding transformer real parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, rval = xfrdat(ibus, jbus, ickt, string)
```

Fortran command syntax:

```
CALL XFRDAT(IBUS, JBUS, ICKT, STRING, RVAL, IERR)
```

IPLAN command syntax:

```
CALL XFRDAT(IBUS, JBUS, ICKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character ICKT	Circuit identifier (input).	
Character STRING	String indicating the transformer quantity desired (input).	
	'RATIO'	Winding one off-nominal turns ratio
	'RATIO2'	Winding two off-nominal turns ratio
	'ANGLE'	Winding one-phase shift angle in degrees
	'RMAX'	Winding one ratio or angle upper limit
	'RMIN'	Winding one ratio or angle lower limit
	'VMAX'	Controlled quantity upper limit
	'VMIN'	Controlled quantity lower limit
	'STEP'	Winding one ratio or angle step width
	'CR'	Real component of load drop compensation impedance
	'CX'	Reactive component of load drop compensation impedance
	'CNXANG'	Winding connection angle in degrees
	'SBASE1'	Winding one base MVA
	'NOMV1'	Winding one nominal voltage in kV
	'NOMV2'	Winding two nominal voltage in kV
	'GMAGNT'	Conductance component of transformer magnetizing admittance in pu
	'BMAGNT'	Susceptance component of transformer magnetizing admittance in pu
	"RG1', 'RGRND"	Grounding resistance at the winding 1 bus
	"XG1', 'XGRND"	Grounding reactance at the winding 1 bus
	'R01'	Zero sequence leakage resistance connected to the winding 1 bus

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'X01'	Zero sequence leakage reactance connected to the winding 1 bus
	"RG2', 'RGRND2"	Grounding resistance at the winding 2 bus
	"XG2', 'XGRND2"	Grounding reactance at the winding 1 bus
	'R02'	Zero sequence leakage resistance connected to the winding 2 bus
	'X02'	Zero sequence leakage reactance connected to the winding 2 bus
	'RNUTRL'	Common neutral grounding resistance
	'XNUTRL'	Common neutral grounding reactance
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Bus not found; 'RVAL' unchanged
	IERR = 2	Branch not found; 'RVAL' unchanged
	IERR = 3	Branch not a transformer; 'RVAL' unchanged
	IERR = 4	Invalid value of 'STRING'; 'RVAL' unchanged
	IERR = 5	Sequence data not in case for STRING = 'RGRND', 'XGRND', 'RGRND2', 'XGRND2', 'RG1', 'XG1', 'R01', 'X01', 'RG2', 'XG2', 'R02', 'X02', 'RNUTRL' or 'XNUTRL'; 'RVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.271. XFRINT

Use this API to return two-winding transformer integer parameters.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, ival = xfrint(ibus, jbus, ickt, string)
```

Fortran command syntax:

```
CALL XFRINT(IBUS, JBUS, ICKT, STRING, IVAL, IERR)
```

IPLAN command syntax:

```
CALL XFRINT(IBUS, JBUS, ICKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character ICKT	Circuit identifier (input).	
Character STRING	String indicating the transformer quantity desired (input).	
	'ICONT'	Controlled bus number
	'NCONT'	Controlled node number
	'SIDCOD'	Controlled bus side code (negative if on the winding 1 side)
	'TAPPED'	Tapped side bus number
	'GCIBUS'	Bus IBUS old grounding code (derived from CNXCOD)
	'GCJBUS'	Bus JBUS old grounding code (derived from CNXCOD)
	'TABLE'	Transformer impedance correction table number (0 if none)
	'CNTRL'	Adjustment enable flag
	'CNTRL2'	Transformer adjustment code
	'NTPOSN'	Number of tap positions
	'CNXCOD'	Connection code
	'CW'	Winding data I/O code
	'CZ'	Impedance data I/O code
	'CM'	Magnetizing admittance I/O code
	'CZ0'	Leakage impedance data I/O code
	'CZG'	Grounding impedance data I/O code
	'TPSTT'	Transformer winding one ratio limit code: (-1) low; (0) regulating; (1) high; (-2) locked; (-3) controlling dc line
	'ANSTT'	Transformer winding one angle limit code: (-1) low; (0) regulating; (1) high; (-2) locked

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IVAL	Value indicated by STRING (output).
Integer IERR	error code (output).

IERR = 0	No error; 'IVAL' returned
IERR = 1	Bus not found; 'IVAL' unchanged
IERR = 2	Branch not found; 'IVAL' unchanged
IERR = 3	Branch not a transformer; 'IVAL' unchanged
IERR = 4	Invalid value of 'STRING'; 'IVAL' unchanged
IERR = 5	Sequence data not in case for STRING = 'CNX-COD', 'CZ0', 'CZG', 'GCIBUS' or 'GCJBUS'; IVAL unchanged
IERR = 6	Connection code is 6 or greater for STRING = 'GCIBUS' or 'GCJBUS'; IVAL unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.272. XFRNAM

Use this API to return two-winding transformer name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, string = xfrnam(ibus, jbus, ickt)
```

Fortran command syntax:

```
CALL XFRNAM(IBUS, JBUS, ICKT, STRING, IERR)
```

IPLAN command syntax:

```
CALL XFRNAM(IBUS, JBUS, ICKT, STRING, IERR)
```

Where:

Integer IBUS	From bus number (input).	
Integer JBUS	To bus number (input).	
Character ICKT	Circuit identifier (input).	
Character*40 STRING	Transformer name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'STRING' returned
	IERR = 1	Bus not found; 'STRING' unchanged
	IERR = 2	Transformer not found; 'STRING' unchanged
	IERR = 3	Branch not a transformer; 'STRING' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.273. ZNDAT

Use this API to return zone totals.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = zndat(iar, string)
```

Fortran command syntax:

```
CALL ZNDAT(IAR, STRING, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL ZNDAT(IAR, STRING, P, Q, IERR)
```

Where:

Integer IAR	Zone number (input).	
Character STRING	String indicating the zone total desired (input).	
	'LOAD'	Total zone load by bus zone assignment (net of load plus in-service distributed generation on load feeder)
	'LOADLD'	Total zone load by load zone assignment
	'LDGN'	Total distributed generation on load feeder by bus zone assignment
	'LDGNLD'	Total distributed generation on load feeder by load zone assignment
	'GEN'	Total zone generation
	'LOSS'	Total zone losses; I**2 * (R + jX) for ac branches with non-metered end in the zone
	'XFRMAG'	Magnetizing admittance powers for transformers with winding 1 bus in the zone
	'LNSHNT'	Powers flowing into line shunts of non-transformer branches with non-metered end in the zone
	'CHARGE'	Powers flowing from line charging of non-transformer branches with non-metered end in the zone
	'INT'	Net zone interchange
	'INDMAC'	Total zone induction machine powers by bus zone assignment
	'INDMACMC'	Total zone induction machine powers by machine zone assignment
	'INDGEN'	Total zone induction generator powers by bus zone assignment
	'INDGENMC'	Total zone induction generator powers by machine zone assignment

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'INDMOT'	Total zone induction motor powers by bus zone assignment
	'INDMOTMC'	Total zone induction motor powers by machine zone assignment
Real P	Real component of the desired complex power (output).	
Real Q	Reactive component of the desired complex power (output).	
Complex CMPVAL	Desired complex power (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Zone number < 0 or > largest allowable zone number; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	No in-service buses with in-service loads (for 'LOAD'), no in-service loads (for 'LOADLD'), no type 2 or type 3 buses (for 'GEN'), no branches (for LOSS'), or no ties (for 'INT') in zone; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Zone not found; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 4	Bad 'STRING' value; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.274. ZNITOJ

Use this API to return the interchange between two zones.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cmpval = znitoj(iar, jar)
```

Fortran command syntax:

```
CALL ZNITOJ(IAR, JAR, CMPVAL, IERR)
```

IPLAN command syntax:

```
CALL ZNITOJ(IAR, JAR, P, Q, IERR)
```

Where:

Integer IAR	From zone number (input).	
Integer JAR	To zone number (input).	
Real P	Real component of the complex interchange	
Real Q	Reactive component of the complex interchange	
Complex CMPVAL	Complex interchange (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'P' and 'Q' or 'CMPVAL' returned
	IERR = 1	Zone number <= 0 or > largest allowable zone number; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 2	No in-service branches between zones; 'P' and 'Q' or 'CMPVAL' unchanged
	IERR = 3	Zone not found; 'P' and 'Q' or 'CMPVAL' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.275. ZONNAM

Use this API to return the zone name.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, cval = zonnam(iar)
```

Fortran command syntax:

```
CALL ZONNAM(IAR, CVAL, IERR)
```

IPLAN command syntax:

```
CALL ZONNAM(IAR, CVAL, IERR)
```

Where:

Integer IAR	Zone number (input).	
Character*12 CVAL	Zone name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'CVAL' is returned
	IERR = 1	Zone number <= 0 or > largest allowable zone number; 'CVAL' is unchanged
	IERR = 2	Zone not found; 'CVAL' is unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.276. ZONNUM

Use this API to return the zone number.

This API routine was first introduced in release 28.0.0.

Python command syntax:

```
ierr, iar = zonnum(string)
```

Fortran command syntax:

```
CALL ZONNUM(IAR, STRING, IERR)
```

IPLAN command syntax:

```
CALL ZONNUM(IAR, STRING, IERR)
```

Where:

Integer IAR	Zone number (integer (output).	
Character STRING	Zone name (string (input).	
Integer IERR	error code (output).	
	IERR = 0	No error, 'IAR' is returned
	IERR = 1	Zone with name 'STRING' not found
	IERR > 1	Multiple zones with the name 'STRING' found, 'IAR' returned as lowest number of these

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

11.277. ZONUSE

Use this API to indicate whether a zone is in use.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ival = zonuse(iar)
```

Fortran command syntax:

```
CALL ZONUSE(IAR, IVAL)
```

IPLAN command syntax:

```
CALL ZONUSE(IAR, IVAL)
```

Where:

Integer IAR	Zone number (input).	
Integer IVAL	Zone status (output).	
	IVAL = -1	Zone number <= 0 or > largest allowable zone number.
	IVAL = 0	If no equipment is assigned to zone IAR.
	IVAL = 1	If any equipment is assigned to zone IAR.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 12
Extension Data Access

This chapter defines API routines that are used to retrieve and set extension data for a single element (e.g., bus, branch, area, etc.) in the PSS®E engine.

These routines do not record themselves when called with recording enabled.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.1. GET_EXT_ACLINE_CHAR

Use this API to get ac line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_acline_char(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_ACLINE_CHAR(IBUS, JBUS, CKT, STRING, CVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.2. GET_EXT_ACLINE_INT

Use this API to get ac line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_acline_int(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_ACLINE_INT(IBUS, JBUS, CKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.3. GET_EXT_ACLINE_LOGICAL

Use this API to get ac line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_acline_logical(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_ACLINE_LOGICAL(IBUS, JBUS, CKT, STRING, LVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.4. GET_EXT_ACLINE_REAL

Use this API to get ac line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_acline_real(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_ACLINE_REAL(IBUS, JBUS, CKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.5. GET_EXT_AREA_CHAR

Use this API to get area character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_area_char(iarea, string)
```

Fortran command syntax:

```
CALL GET_EXT_AREA_CHAR(IAREA, STRING, CVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.6. GET_EXT_AREA_INT

Use this API to get area integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_area_int(iarea, string)
```

Fortran command syntax:

```
CALL GET_EXT_AREA_INT(IAREA, STRING, IVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.7. GET_EXT_AREA_LOGICAL

Use this API to get area logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_area_logical(iarea, string)
```

Fortran command syntax:

```
CALL GET_EXT_AREA_LOGICAL(IAREA, STRING, LVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.8. GET_EXT_AREA_REAL

Use this API to get area real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_area_real(iarea, string)
```

Fortran command syntax:

```
CALL GET_EXT_AREA_REAL(IAREA, STRING, RVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.9. GET_EXT_BUS_CHAR

Use this API to get bus character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_bus_char(ibus, string)
```

Fortran command syntax:

```
CALL GET_EXT_BUS_CHAR(IBUS, STRING, CVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.10. GET_EXT_BUS_INT

Use this API to get bus integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_bus_int(ibus, string)
```

Fortran command syntax:

```
CALL GET_EXT_BUS_INT(IBUS, STRING, IVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.11. GET_EXT_BUS_LOGICAL

Use this API to get bus logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_bus_logical(ibus, string)
```

Fortran command syntax:

```
CALL GET_EXT_BUS_LOGICAL(IBUS, STRING, LVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.12. GET_EXT_BUS_REAL

Use this API to get bus real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_bus_real(ibus, string)
```

Fortran command syntax:

```
CALL GET_EXT_BUS_REAL(IBUS, STRING, RVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.13. GET_EXT_CHAR

Use this API to get character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_char(tblname, string, irow)
```

Fortran command syntax:

```
CALL GET_EXT_CHAR(TBLNAME, STRING, IROW, CVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.14. GET_EXT_FACTS_CHAR

Use this API to get facts device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_facts_char(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_FACTS_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.15. GET_EXT_FACTS_INT

Use this API to get facts device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_facts_int(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_FACTS_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.16. GET_EXT_FACTS_LOGICAL

Use this API to get facts device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_facts_logical(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_FACTS_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.17. GET_EXT_FACTS_REAL

Use this API to get facts device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_facts_real(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_FACTS_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.18. GET_EXT_FIXSHUNT_CHAR

Use this API to get fixed shunt character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_fixshunt_char(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_FIXSHUNT_CHAR(IBUS, SHNTID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.19. GET_EXT_FIXSHUNT_INT

Use this API to get fixed shunt integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_fixshunt_int(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_FIXSHUNT_INT(IBUS, SHNTID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.20. GET_EXT_FIXSHUNT_LOGICAL

Use this API to get fixed shunt logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_fixshunt_logical(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_FIXSHUNT_LOGICAL(IBUS, SHNTID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.21. GET_EXT_FIXSHUNT_REAL

Use this API to get fixed shunt real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_fixshunt_real(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_FIXSHUNT_REAL(IBUS, SHNTID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.22. GET_EXT_GENERATOR_CHAR

Use this API to get generator character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_generator_char(ibus, machid, string)
```

Fortran command syntax:

```
CALL GET_EXT_GENERATOR_CHAR(IBUS, MACHID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.23. GET_EXT_GENERATOR_INT

Use this API to get generator integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_generator_int(ibus, machid, string)
```

Fortran command syntax:

```
CALL GET_EXT_GENERATOR_INT(IBUS, MACHID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.24. GET_EXT_GENERATOR_LOGICAL

Use this API to get generator logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_generator_logical(ibus, machid, string)
```

Fortran command syntax:

```
CALL GET_EXT_GENERATOR_LOGICAL(IBUS, MACHID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.25. GET_EXT_GENERATOR_REAL

Use this API to get generator real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_generator_real(ibus, machid, string)
```

Fortran command syntax:

```
CALL GET_EXT_GENERATOR_REAL(IBUS, MACHID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.26. GET_EXT_GNE_CHAR

Use this API to get general network element device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_gne_char(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_GNE_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.27. GET_EXT_GNE_INT

Use this API to get general network element device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_gne_int(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_GNE_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.28. GET_EXT_GNE_LOGICAL

Use this API to get general network element device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_gne_logical(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_GNE_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.29. GET_EXT_GNE_REAL

Use this API to get general network element device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_gne_real(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_GNE_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.30. GET_EXT_IATRANS_CHAR

Use this API to get inter-area transfer character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_iatrans_char(arfrom, arto, trid, string)
```

Fortran command syntax:

```
CALL GET_EXT_IATRANS_CHAR(ARFROM, ARTO, TRID, STRING, CVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.31. GET_EXT_IATRANS_INT

Use this API to get inter-area transfer integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_iatrans_int(arfrom, arto, trid, string)
```

Fortran command syntax:

```
CALL GET_EXT_IATRANS_INT(ARFROM, ARTO, TRID, STRING, IVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.32. GET_EXT_IATRANS_LOGICAL

Use this API to get inter-area transfer logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_iatrans_logical(arfrom, arto, trid, string)
```

Fortran command syntax:

```
CALL GET_EXT_IATRANS_LOGICAL(ARFROM, ARTO, TRID, STRING, LVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.33. GET_EXT_IATRANS_REAL

Use this API to get inter-area transfer real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_iatrans_real(arfrom, arto, trid, string)
```

Fortran command syntax:

```
CALL GET_EXT_IATRANS_REAL(ARFROM, ARTO, TRID, STRING, RVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.34. GET_EXT_INDMACH_CHAR

Use this API to get induction machine character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_indmach_char(ibus, imid, string)
```

Fortran command syntax:

```
CALL GET_EXT_INDMACH_CHAR(IBUS, IMID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.35. GET_EXT_INDMACH_INT

Use this API to get induction machine integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_indmach_int(ibus, imid, string)
```

Fortran command syntax:

```
CALL GET_EXT_INDMACH_INT(IBUS, IMID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.36. GET_EXT_INDMACH_LOGICAL

Use this API to get induction machine logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_indmach_logical(ibus, imid, string)
```

Fortran command syntax:

```
CALL GET_EXT_INDMACH_LOGICAL(IBUS, IMID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.37. GET_EXT_INDMACH_REAL

Use this API to get induction machine real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_indmach_real(ibus, imid, string)
```

Fortran command syntax:

```
CALL GET_EXT_INDMACH_REAL(IBUS, IMID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.38. GET_EXT_INT

Use this API to get integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_int(tblname, string, irow)
```

Fortran command syntax:

```
CALL GET_EXT_INT(TBLNAME, STRING, IROW, IVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.39. GET_EXT_LOAD_CHAR

Use this API to get load character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_load_char(ibus, loadid, string)
```

Fortran command syntax:

```
CALL GET_EXT_LOAD_CHAR(IBUS, LOADID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.40. GET_EXT_LOAD_INT

Use this API to get load integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_load_int(ibus, loadid, string)
```

Fortran command syntax:

```
CALL GET_EXT_LOAD_INT(IBUS, LOADID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.41. GET_EXT_LOAD_LOGICAL

Use this API to get load logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_load_logical(ibus, loadid, string)
```

Fortran command syntax:

```
CALL GET_EXT_LOAD_LOGICAL(IBUS, LOADID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.42. GET_EXT_LOAD_REAL

Use this API to get load real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_load_real(ibus, loadid, string)
```

Fortran command syntax:

```
CALL GET_EXT_LOAD_REAL(IBUS, LOADID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.43. GET_EXT_LOGICAL

Use this API to get logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_logical(tblname, string, irow)
```

Fortran command syntax:

```
CALL GET_EXT_LOGICAL(TBLNAME, STRING, IROW, LVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.44. GET_EXT_MSLINE_CHAR

Use this API to get multi-section line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_msline_char(ibus, jbus, mslid, string)
```

Fortran command syntax:

```
CALL GET_EXT_MSLINE_CHAR(IBUS, JBUS, MSLID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.45. GET_EXT_MSLINE_INT

Use this API to get multi-section line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_msline_int(ibus, jbus, mslid, string)
```

Fortran command syntax:

```
CALL GET_EXT_MSLINE_INT(IBUS, JBUS, MSLID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.46. GET_EXT_MSLINE_LOGICAL

Use this API to get multi-section line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_msline_logical(ibus, jbus, mslid, string)
```

Fortran command syntax:

```
CALL GET_EXT_MSLINE_LOGICAL(IBUS, JBUS, MSLID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.47. GET_EXT_MSLINE_REAL

Use this API to get multi-section line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_msline_real(ibus, jbus, mslid, string)
```

Fortran command syntax:

```
CALL GET_EXT_MSLINE_REAL(IBUS, JBUS, MSLID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.48. GET_EXT_NTERMDC_CHAR

Use this API to get multi-terminal dc line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_ntermdc_char(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_NTERMDC_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.49. GET_EXT_NTERMDC_INT

Use this API to get multi-terminal dc line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_ntermdc_int(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_NTERMDC_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.50. GET_EXT_NTERMDC_LOGICAL

Use this API to get multi-terminal dc line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_ntermdc_logical(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_NTERMDC_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.51. GET_EXT_NTERMDC_REAL

Use this API to get multi-terminal dc line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_ntermdc_real(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_NTERMDC_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.52. GET_EXT_OWNER_CHAR

Use this API to get owner character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_owner_char(iowner, string)
```

Fortran command syntax:

```
CALL GET_EXT_OWNER_CHAR(IOWNER, STRING, CVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.53. GET_EXT_OWNER_INT

Use this API to get owner integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_owner_int(iowner, string)
```

Fortran command syntax:

```
CALL GET_EXT_OWNER_INT(IOWNER, STRING, IVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.54. GET_EXT_OWNER_LOGICAL

Use this API to get owner logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_owner_logical(iowner, string)
```

Fortran command syntax:

```
CALL GET_EXT_OWNER_LOGICAL(IOWNER, STRING, LVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.55. GET_EXT_OWNER_REAL

Use this API to get owner real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_owner_real(iowner, string)
```

Fortran command syntax:

```
CALL GET_EXT_OWNER_REAL(IOWNER, STRING, RVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.56. GET_EXT_RATING_CHAR

Use this API to get rating character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_rating_char(irate, string)
```

Fortran command syntax:

```
CALL GET_EXT_RATING_CHAR(IRATE, STRING, CVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.57. GET_EXT_RATING_INT

Use this API to get rating integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_rating_int(irate, string)
```

Fortran command syntax:

```
CALL GET_EXT_RATING_INT(IRATE, STRING, IVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.58. GET_EXT_RATING_LOGICAL

Use this API to get rating logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_rating_logical(irate, string)
```

Fortran command syntax:

```
CALL GET_EXT_RATING_LOGICAL(IRATE, STRING, LVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.59. GET_EXT_RATING_REAL

Use this API to get rating real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_rating_real(irate, string)
```

Fortran command syntax:

```
CALL GET_EXT_RATING_REAL(IRATE, STRING, RVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.60. GET_EXT_REAL

Use this API to get real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_real(tblname, string, irow)
```

Fortran command syntax:

```
CALL GET_EXT_REAL(TBLNAME, STRING, IROW, RVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.61. GET_EXT_ROWCOUNT

Use this API to get extension data table row count.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, nrows = get_ext_rowcount(tblname)
```

Fortran command syntax:

```
CALL GET_EXT_ROWCOUNT(TBLNAME, NROWS, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Integer NROWS	Number of rows in table indicated by TBLNAME (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NROWS' set
	IERR = 1	Table not found; 'NROWS' not set
	IERR = 4	Unknown error with data access; 'NROWS' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.62. GET_EXT_SUBNODE_CHAR

Use this API to get substation node character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_subnode_char(isub, inode, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBNODE_CHAR(ISUB, INODE, STRING, CVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.63. GET_EXT_SUBNODE_INT

Use this API to get substation node integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_subnode_int(isub, inode, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBNODE_INT(ISUB, INODE, STRING, IVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.64. GET_EXT_SUBNODE_LOGICAL

Use this API to get substation node logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_subnode_logical(isub, inode, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBNODE_LOGICAL(ISUB, INODE, STRING, LVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.65. GET_EXT_SUBNODE_REAL

Use this API to get substation node real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_subnode_real(isub, inode, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBNODE_REAL(ISUB, INODE, STRING, RVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.66. GET_EXT_SUBSWD_CHAR

Use this API to get substation switching device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_subswd_char(isub, inode, jnode, swdid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBSWD_CHAR(ISUB, INODE, JNODE, SWDID, STRING, CVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.67. GET_EXT_SUBSWD_INT

Use this API to get substation switching device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_subswd_int(isub, inode, jnode, swdid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBSWD_INT(ISUB, INODE, JNODE, SWDID, STRING, IVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.68. GET_EXT_SUBSWD_LOGICAL

Use this API to get substation switching device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_subswd_logical(isub, inode, jnode, swdid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBSWD_LOGICAL(ISUB, INODE, JNODE, SWDID, STRING, LVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.69. GET_EXT_SUBSWD_REAL

Use this API to get substation switching device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_subswd_real(isub, inode, jnode, swdid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUBSWD_REAL(ISUB, INODE, JNODE, SWDID, STRING, RVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.70. GET_EXT_SUB_CHAR

Use this API to get substation character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_sub_char(isub, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUB_CHAR(ISUB, STRING, CVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.71. GET_EXT_SUB_INT

Use this API to get substation integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_sub_int(isub, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUB_INT(ISUB, STRING, IVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.72. GET_EXT_SUB_LOGICAL

Use this API to get substation logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_sub_logical(isub, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUB_LOGICAL(ISUB, STRING, LVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.73. GET_EXT_SUB_REAL

Use this API to get substation real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_sub_real(isub, string)
```

Fortran command syntax:

```
CALL GET_EXT_SUB_REAL(ISUB, STRING, RVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.74. GET_EXT_SWSHUNT_CHAR

Use this API to get switched shunt character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_swshunt_char(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SWSHUNT_CHAR(IBUS, SHNTID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.75. GET_EXT_SWSHUNT_INT

Use this API to get switched shunt integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_swshunt_int(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SWSHUNT_INT(IBUS, SHNTID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.76. GET_EXT_SWSHUNT_LOGICAL

Use this API to get switched shunt logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_swshunt_logical(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SWSHUNT_LOGICAL(IBUS, SHNTID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.77. GET_EXT_SWSHUNT_REAL

Use this API to get switched shunt real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_swshunt_real(ibus, shntid, string)
```

Fortran command syntax:

```
CALL GET_EXT_SWSHUNT_REAL(IBUS, SHNTID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.78. GET_EXT_SYSSWD_CHAR

Use this API to get system switching device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_sysswd_char(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_SYSSWD_CHAR(IBUS, JBUS, CKT, STRING, CVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.79. GET_EXT_SYSSWD_INT

Use this API to get system switching device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_sysswd_int(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_SYSSWD_INT(IBUS, JBUS, CKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.80. GET_EXT_SYSSWD_LOGICAL

Use this API to get system switching device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_sysswd_logical(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_SYSSWD_LOGICAL(IBUS, JBUS, CKT, STRING, LVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.81. GET_EXT_SYSSWD_REAL

Use this API to get system switching device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_sysswd_real(ibus, jbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_SYSSWD_REAL(IBUS, JBUS, CKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.82. GET_EXT_TRANSFORMER_CHAR

Use this API to get transformer character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_transformer_char(ibus, jbus, kbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_TRANSFORMER_CHAR(IBUS, JBUS, KBUS, CKT, STRING, CVAL, IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.83. GET_EXT_TRANSFORMER_INT

Use this API to get transformer integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_transformer_int(ibus, jbus, kbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_TRANSFORMER_INT(IBUS, JBUS, KBUS, CKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.84. GET_EXT_TRANSFORMER_LOGICAL

Use this API to get transformer logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_transformer_logical(ibus, jbus, kbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_TRANSFORMER_LOGICAL(IBUS, JBUS, KBUS, CKT, STRING, LVAL,
 IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.85. GET_EXT_TRANSFORMER_REAL

Use this API to get transformer real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_transformer_real(ibus, jbus, kbus, ckt, string)
```

Fortran command syntax:

```
CALL GET_EXT_TRANSFORMER_REAL(IBUS, JBUS, KBUS, CKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.86. GET_EXT_TWOTERMDC_CHAR

Use this API to get two-terminal dc line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_twotermdc_char(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_TWOTERMDC_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.87. GET_EXT_TWOTERMDC_INT

Use this API to get two-terminal dc line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_twotermdc_int(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_TWOTERMDC_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.88. GET_EXT_TWOTERMDC_LOGICAL

Use this API to get two-terminal dc line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_twotermdc_logical(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_TWOTERMDC_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.89. GET_EXT_TWOTERMDC_REAL

Use this API to get two-terminal dc line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_twotermdc_real(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_TWOTERMDC_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.90. GET_EXT_VSCDC_CHAR

Use this API to get voltage source converter dc line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_vscdc_char(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_VSCDC_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.91. GET_EXT_VSCDC_INT

Use this API to get voltage source converter dc line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_vscdc_int(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_VSCDC_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.92. GET_EXT_VSCDC_LOGICAL

Use this API to get voltage source converter dc line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_vscdc_logical(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_VSCDC_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.93. GET_EXT_VSCDC_REAL

Use this API to get voltage source converter dc line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_vscdc_real(name, string)
```

Fortran command syntax:

```
CALL GET_EXT_VSCDC_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.94. GET_EXT_ZONE_CHAR

Use this API to get zone character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, cval = get_ext_zone_char(izone, string)
```

Fortran command syntax:

```
CALL GET_EXT_ZONE_CHAR(IZONE, STRING, CVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Character*100 CVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' returned
	IERR = 1	Element not found; 'CVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'CVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not returned
	IERR = 4	Unknown error with data access; 'CVAL' not returned
	IERR = 10	API Prereqs not met; 'CVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.95. GET_EXT_ZONE_INT

Use this API to get zone integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ival = get_ext_zone_int(izone, string)
```

Fortran command syntax:

```
CALL GET_EXT_ZONE_INT(IZONE, STRING, IVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Integer IVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' returned
	IERR = 1	Element not found; 'IVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'IVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not returned
	IERR = 4	Unknown error with data access; 'IVAL' not returned
	IERR = 10	API Prereqs not met; 'IVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.96. GET_EXT_ZONE_LOGICAL

Use this API to get zone logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, lval = get_ext_zone_logical(izone, string)
```

Fortran command syntax:

```
CALL GET_EXT_ZONE_LOGICAL(IZONE, STRING, LVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Logical LVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' returned
	IERR = 1	Element not found; 'LVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'LVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not returned
	IERR = 4	Unknown error with data access; 'LVAL' not returned
	IERR = 10	API Prereqs not met; 'LVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.97. GET_EXT_ZONE_REAL

Use this API to get zone real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, rval = get_ext_zone_real(izone, string)
```

Fortran command syntax:

```
CALL GET_EXT_ZONE_REAL(IZONE, STRING, RVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Real RVAL	Value indicated by STRING (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' returned
	IERR = 1	Element not found; 'RVAL' not returned
	IERR = 2	Bad value of 'STRING'; 'RVAL' not returned
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not returned
	IERR = 4	Unknown error with data access; 'RVAL' not returned
	IERR = 10	API Prereqs not met; 'RVAL' not returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.98. SET_EXT_ACLINE_CHAR

Use this API to set ac line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_acline_char(ibus, jbus, ckt, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_ACLINE_CHAR(IBUS, JBUS, CKT, STRING, CVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.99. SET_EXT_ACLINE_INT

Use this API to set ac line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_acline_int(ibus, jbus, ckt, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_ACLINE_INT(IBUS, JBUS, CKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.100. SET_EXT_ACLINE_LOGICAL

Use this API to set ac line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_acline_logical(ibus, jbus, ckt, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_ACLINE_LOGICAL(IBUS, JBUS, CKT, STRING, LVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.101. SET_EXT_ACLINE_REAL

Use this API to set ac line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_acline_real(ibus, jbus, ckt, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_ACLINE_REAL(IBUS, JBUS, CKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	ACLINE ibus (input).	
Integer JBUS	ACLINE jbus (input).	
Character CKT	ACLINE ckt (input).	
Character STRING	String indicating the extension ACLINE field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.102. SET_EXT_AREA_CHAR

Use this API to set area character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_area_char(iarea, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_AREA_CHAR(IAREA, STRING, CVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.103. SET_EXT_AREA_INT

Use this API to set area integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_area_int(iarea, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_AREA_INT(IAREA, STRING, IVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.104. SET_EXT_AREA_LOGICAL

Use this API to set area logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_area_logical(iarea, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_AREA_LOGICAL(IAREA, STRING, LVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.105. SET_EXT_AREA_REAL

Use this API to set area real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_area_real(iarea, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_AREA_REAL(IAREA, STRING, RVAL, IERR)
```

Where:

Integer IAREA	AREA iarea (input).	
Character STRING	String indicating the extension AREA field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.106. SET_EXT_BUS_CHAR

Use this API to set bus character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_bus_char(ibus, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_BUS_CHAR(IBUS, STRING, CVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.107. SET_EXT_BUS_INT

Use this API to set bus integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_bus_int(ibus, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_BUS_INT(IBUS, STRING, IVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.108. SET_EXT_BUS_LOGICAL

Use this API to set bus logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_bus_logical(ibus, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_BUS_LOGICAL(IBUS, STRING, LVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.109. SET_EXT_BUS_REAL

Use this API to set bus real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_bus_real(ibus, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_BUS_REAL(IBUS, STRING, RVAL, IERR)
```

Where:

Integer IBUS	BUS ibus (input).	
Character STRING	String indicating the extension BUS field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.110. SET_EXT_CHAR

Use this API to set character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_char(tblname, string, irow, cval)
```

Fortran command syntax:

```
CALL SET_EXT_CHAR(TBLNAME, STRING, IROW, CVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Character CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.111. SET_EXT_FACTS_CHAR

Use this API to set facts device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_facts_char(name, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_FACTS_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.112. SET_EXT_FACTS_INT

Use this API to set facts device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_facts_int(name, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_FACTS_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.113. SET_EXT_FACTS_LOGICAL

Use this API to set facts device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_facts_logical(name, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_FACTS_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.114. SET_EXT_FACTS_REAL

Use this API to set facts device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_facts_real(name, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_FACTS_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	FACTS name (input).	
Character STRING	String indicating the extension FACTS field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.115. SET_EXT_FIXSHUNT_CHAR

Use this API to set fixed shunt character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_fixshunt_char(ibus, shntid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_FIXSHUNT_CHAR(IBUS, SHNTID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.116. SET_EXT_FIXSHUNT_INT

Use this API to set fixed shunt integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_fixshunt_int(ibus, shntid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_FIXSHUNT_INT(IBUS, SHNTID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.117. SET_EXT_FIXSHUNT_LOGICAL

Use this API to set fixed shunt logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_fixshunt_logical(ibus, shntid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_FIXSHUNT_LOGICAL(IBUS, SHNTID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.118. SET_EXT_FIXSHUNT_REAL

Use this API to set fixed shunt real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_fixshunt_real(ibus, shntid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_FIXSHUNT_REAL(IBUS, SHNTID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	FIXSHUNT ibus (input).	
Character SHNTID	FIXSHUNT shntid (input).	
Character STRING	String indicating the extension FIXSHUNT field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.119. SET_EXT_GENERATOR_CHAR

Use this API to set generator character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_generator_char(ibus, machid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_GENERATOR_CHAR(IBUS, MACHID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.120. SET_EXT_GENERATOR_INT

Use this API to set generator integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_generator_int(ibus, machid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_GENERATOR_INT(IBUS, MACHID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.121. SET_EXT_GENERATOR_LOGICAL

Use this API to set generator logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_generator_logical(ibus, machid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_GENERATOR_LOGICAL(IBUS, MACHID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.122. SET_EXT_GENERATOR_REAL

Use this API to set generator real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_generator_real(ibus, machid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_GENERATOR_REAL(IBUS, MACHID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	GENERATOR ibus (input).	
Character MACHID	GENERATOR machid (input).	
Character STRING	String indicating the extension GENERATOR field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.123. SET_EXT_GNE_CHAR

Use this API to set general network element device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_gne_char(name, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_GNE_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.124. SET_EXT_GNE_INT

Use this API to set general network element device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_gne_int(name, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_GNE_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.125. SET_EXT_GNE_LOGICAL

Use this API to set general network element device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_gne_logical(name, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_GNE_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.126. SET_EXT_GNE_REAL

Use this API to set general network element device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_gne_real(name, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_GNE_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	GNE name (input).	
Character STRING	String indicating the extension GNE field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.127. SET_EXT_IATRANS_CHAR

Use this API to set inter-area transfer character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_iatrans_char(arfrom, arto, trid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_IATRANS_CHAR(ARFROM, ARTO, TRID, STRING, CVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.128. SET_EXT_IATRANS_INT

Use this API to set inter-area transfer integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_iatrans_int(arfrom, arto, trid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_IATRANS_INT(ARFROM, ARTO, TRID, STRING, IVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.129. SET_EXT_IATRANS_LOGICAL

Use this API to set inter-area transfer logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_iatrans_logical(arfrom, arto, trid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_IATRANS_LOGICAL(ARFROM, ARTO, TRID, STRING, LVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.130. SET_EXT_IATRANS_REAL

Use this API to set inter-area transfer real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_iatrans_real(arfrom, arto, trid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_IATRANS_REAL(ARFROM, ARTO, TRID, STRING, RVAL, IERR)
```

Where:

Integer ARFROM	IATRANS arfrom (input).	
Integer ARTO	IATRANS arto (input).	
Character TRID	IATRANS trid (input).	
Character STRING	String indicating the extension IATRANS field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.131. SET_EXT_INDMACH_CHAR

Use this API to set induction machine character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_indmach_char(ibus, imid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_INDMACH_CHAR(IBUS, IMID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.132. SET_EXT_INDMACH_INT

Use this API to set induction machine integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_indmach_int(ibus, imid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_INDMACH_INT(IBUS, IMID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.133. SET_EXT_INDMACH_LOGICAL

Use this API to set induction machine logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_indmach_logical(ibus, imid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_INDMACH_LOGICAL(IBUS, IMID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.134. SET_EXT_INDMACH_REAL

Use this API to set induction machine real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_indmach_real(ibus, imid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_INDMACH_REAL(IBUS, IMID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	INDMACH ibus (input).	
Character IMID	INDMACH imid (input).	
Character STRING	String indicating the extension INDMACH field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.135. SET_EXT_INT

Use this API to set integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_int(tblname, string, irow, ival)
```

Fortran command syntax:

```
CALL SET_EXT_INT(TBLNAME, STRING, IROW, IVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.136. SET_EXT_LOAD_CHAR

Use this API to set load character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_load_char(ibus, loadid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_LOAD_CHAR(IBUS, LOADID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.137. SET_EXT_LOAD_INT

Use this API to set load integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_load_int(ibus, loadid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_LOAD_INT(IBUS, LOADID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.138. SET_EXT_LOAD_LOGICAL

Use this API to set load logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_load_logical(ibus, loadid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_LOAD_LOGICAL(IBUS, LOADID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.139. SET_EXT_LOAD_REAL

Use this API to set load real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_load_real(ibus, loadid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_LOAD_REAL(IBUS, LOADID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	LOAD ibus (input).	
Character LOADID	LOAD loadid (input).	
Character STRING	String indicating the extension LOAD field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.140. SET_EXT_LOGICAL

Use this API to set logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_logical(tblname, string, irow, lval)
```

Fortran command syntax:

```
CALL SET_EXT_LOGICAL(TBLNAME, STRING, IROW, LVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.141. SET_EXT_MSLINE_CHAR

Use this API to set multi-section line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_msline_char(ibus, jbus, mslid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_MSLINE_CHAR(IBUS, JBUS, MSLID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.142. SET_EXT_MSLINE_INT

Use this API to set multi-section line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_msline_int(ibus, jbus, mslid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_MSLINE_INT(IBUS, JBUS, MSLID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.143. SET_EXT_MSLINE_LOGICAL

Use this API to set multi-section line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_msline_logical(ibus, jbus, mslid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_MSLINE_LOGICAL(IBUS, JBUS, MSLID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.144. SET_EXT_MSLINE_REAL

Use this API to set multi-section line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_msline_real(ibus, jbus, mslid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_MSLINE_REAL(IBUS, JBUS, MSLID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	MSLINE ibus (input).	
Integer JBUS	MSLINE jbus (input).	
Character MSLID	MSLINE mslid (input).	
Character STRING	String indicating the extension MSLINE field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.145. SET_EXT_NTERMDC_CHAR

Use this API to set multi-terminal dc line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_ntermdc_char(name, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_NTERMDC_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.146. SET_EXT_NTERMDC_INT

Use this API to set multi-terminal dc line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_ntermdc_int(name, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_NTERMDC_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.147. SET_EXT_NTERMDC_LOGICAL

Use this API to set multi-terminal dc line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_ntermdc_logical(name, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_NTERMDC_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.148. SET_EXT_NTERMDC_REAL

Use this API to set multi-terminal dc line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_ntermdc_real(name, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_NTERMDC_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	NTERMDC name (input).	
Character STRING	String indicating the extension NTERMDC field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.149. SET_EXT_OWNER_CHAR

Use this API to set owner character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_owner_char(iowner, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_OWNER_CHAR(IOWNER, STRING, CVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.150. SET_EXT_OWNER_INT

Use this API to set owner integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_owner_int(iowner, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_OWNER_INT(IOWNER, STRING, IVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.151. SET_EXT_OWNER_LOGICAL

Use this API to set owner logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_owner_logical(iowner, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_OWNER_LOGICAL(IOWNER, STRING, LVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.152. SET_EXT_OWNER_REAL

Use this API to set owner real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_owner_real(iowner, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_OWNER_REAL(IOWNER, STRING, RVAL, IERR)
```

Where:

Integer IOWNER	OWNER iowner (input).	
Character STRING	String indicating the extension OWNER field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.153. SET_EXT_RATING_CHAR

Use this API to set rating character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_rating_char(irate, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_RATING_CHAR(IRATE, STRING, CVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.154. SET_EXT_RATING_INT

Use this API to set rating integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_rating_int(irate, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_RATING_INT(IRATE, STRING, IVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.155. SET_EXT_RATING_LOGICAL

Use this API to set rating logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_rating_logical(irate, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_RATING_LOGICAL(IRATE, STRING, LVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.156. SET_EXT_RATING_REAL

Use this API to set rating real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_rating_real(irate, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_RATING_REAL(IRATE, STRING, RVAL, IERR)
```

Where:

Integer IRATE	RATING irate (input).	
Character STRING	String indicating the extension RATING field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.157. SET_EXT_REAL

Use this API to set real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_real(tblname, string, irow, rval)
```

Fortran command syntax:

```
CALL SET_EXT_REAL(TBLNAME, STRING, IROW, RVAL, IERR)
```

Where:

Character TBLNAME	Extension table name (input).	
Character STRING	String indicating the extension field desired (input).	
Integer IROW	Row index of desired data item (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 5	Data table row index 'IROW' out-of-bounds; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.158. SET_EXT_SUBNODE_CHAR

Use this API to set substation node character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subnode_char(isub, inode, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_SUBNODE_CHAR(ISUB, INODE, STRING, CVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.159. SET_EXT_SUBNODE_INT

Use this API to set substation node integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subnode_int(isub, inode, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_SUBNODE_INT(ISUB, INODE, STRING, IVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.160. SET_EXT_SUBNODE_LOGICAL

Use this API to set substation node logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subnode_logical(isub, inode, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_SUBNODE_LOGICAL(ISUB, INODE, STRING, LVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.161. SET_EXT_SUBNODE_REAL

Use this API to set substation node real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subnode_real(isub, inode, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_SUBNODE_REAL(ISUB, INODE, STRING, RVAL, IERR)
```

Where:

Integer ISUB	SUBNODE isub (input).	
Integer INODE	SUBNODE inode (input).	
Character STRING	String indicating the extension SUBNODE field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.162. SET_EXT_SUBSWD_CHAR

Use this API to set substation switching device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subswd_char(isub, inode, jnode, swdid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_SUBSWD_CHAR(ISUB, INODE, JNODE, SWDID, STRING, CVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.163. SET_EXT_SUBSWD_INT

Use this API to set substation switching device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subswd_int(isub, inode, jnode, swdid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_SUBSWD_INT(ISUB, INODE, JNODE, SWDID, STRING, IVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.164. SET_EXT_SUBSWD_LOGICAL

Use this API to set substation switching device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subswd_logical(isub, inode, jnode, swdid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_SUBSWD_LOGICAL(ISUB, INODE, JNODE, SWDID, STRING, LVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.165. SET_EXT_SUBSWD_REAL

Use this API to set substation switching device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_subswd_real(isub, inode, jnode, swdid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_SUBSWD_REAL(ISUB, INODE, JNODE, SWDID, STRING, RVAL, IERR)
```

Where:

Integer ISUB	SUBSWD isub (input).	
Integer INODE	SUBSWD inode (input).	
Integer JNODE	SUBSWD jnode (input).	
Character SWDID	SUBSWD swdid (input).	
Character STRING	String indicating the extension SUBSWD field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.166. SET_EXT_SUB_CHAR

Use this API to set substation character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sub_char(isub, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_SUB_CHAR(ISUB, STRING, CVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.167. SET_EXT_SUB_INT

Use this API to set substation integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sub_int(isub, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_SUB_INT(ISUB, STRING, IVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.168. SET_EXT_SUB_LOGICAL

Use this API to set substation logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sub_logical(isub, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_SUB_LOGICAL(ISUB, STRING, LVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.169. SET_EXT_SUB_REAL

Use this API to set substation real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sub_real(isub, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_SUB_REAL(ISUB, STRING, RVAL, IERR)
```

Where:

Integer ISUB	SUB isub (input).	
Character STRING	String indicating the extension SUB field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.170. SET_EXT_SWSHUNT_CHAR

Use this API to set switched shunt character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_swshunt_char(ibus, shntid, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_SWSHUNT_CHAR(IBUS, SHNTID, STRING, CVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.171. SET_EXT_SWSHUNT_INT

Use this API to set switched shunt integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_swshunt_int(ibus, shntid, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_SWSHUNT_INT(IBUS, SHNTID, STRING, IVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.172. SET_EXT_SWSHUNT_LOGICAL

Use this API to set switched shunt logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_swshunt_logical(ibus, shntid, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_SWSHUNT_LOGICAL(IBUS, SHNTID, STRING, LVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.173. SET_EXT_SWSHUNT_REAL

Use this API to set switched shunt real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_swshunt_real(ibus, shntid, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_SWSHUNT_REAL(IBUS, SHNTID, STRING, RVAL, IERR)
```

Where:

Integer IBUS	SWSHUNT ibus (input).	
Character SHNTID	SWSHUNT shntid (input).	
Character STRING	String indicating the extension SWSHUNT field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.174. SET_EXT_SYSSWD_CHAR

Use this API to set system switching device character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sysswd_char(ibus, jbus, ckt, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_SYSSWD_CHAR(IBUS, JBUS, CKT, STRING, CVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.175. SET_EXT_SYSSWD_INT

Use this API to set system switching device integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sysswd_int(ibus, jbus, ckt, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_SYSSWD_INT(IBUS, JBUS, CKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.176. SET_EXT_SYSSWD_LOGICAL

Use this API to set system switching device logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sysswd_logical(ibus, jbus, ckt, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_SYSSWD_LOGICAL(IBUS, JBUS, CKT, STRING, LVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.177. SET_EXT_SYSSWD_REAL

Use this API to set system switching device real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_sysswd_real(ibus, jbus, ckt, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_SYSSWD_REAL(IBUS, JBUS, CKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	SYSSWD ibus (input).	
Integer JBUS	SYSSWD jbus (input).	
Character CKT	SYSSWD ckt (input).	
Character STRING	String indicating the extension SYSSWD field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.178. SET_EXT_TRANSFORMER_CHAR

Use this API to set transformer character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_transformer_char(ibus, jbus, kbus, ckt, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_TRANSFORMER_CHAR(IBUS, JBUS, KBUS, CKT, STRING, CVAL, IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.179. SET_EXT_TRANSFORMER_INT

Use this API to set transformer integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_transformer_int(ibus, jbus, kbus, ckt, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_TRANSFORMER_INT(IBUS, JBUS, KBUS, CKT, STRING, IVAL, IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.180. SET_EXT_TRANSFORMER_LOGICAL

Use this API to set transformer logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_transformer_logical(ibus, jbus, kbus, ckt, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_TRANSFORMER_LOGICAL(IBUS, JBUS, KBUS, CKT, STRING, LVAL,
 IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.181. SET_EXT_TRANSFORMER_REAL

Use this API to set transformer real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_transformer_real(ibus, jbus, kbus, ckt, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_TRANSFORMER_REAL(IBUS, JBUS, KBUS, CKT, STRING, RVAL, IERR)
```

Where:

Integer IBUS	TRANSFORMER ibus (input).	
Integer JBUS	TRANSFORMER jbus (input).	
Integer KBUS	TRANSFORMER kbus (input).	
Character CKT	TRANSFORMER ckt (input).	
Character STRING	String indicating the extension TRANSFORMER field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.182. SET_EXT_TWOTERMDC_CHAR

Use this API to set two-terminal dc line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_twotermdc_char(name, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_TWOTERMDC_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.183. SET_EXT_TWOTERMDC_INT

Use this API to set two-terminal dc line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_twotermdc_int(name, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_TWOTERMDC_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.184. SET_EXT_TWOTERMDC_LOGICAL

Use this API to set two-terminal dc line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_twotermdc_logical(name, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_TWOTERMDC_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.185. SET_EXT_TWOTERMDC_REAL

Use this API to set two-terminal dc line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_twotermdc_real(name, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_TWOTERMDC_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	TWOTERMDC name (input).	
Character STRING	String indicating the extension TWOTERMDC field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.186. SET_EXT_VSCDC_CHAR

Use this API to set voltage source converter dc line character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_vscdc_char(name, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_VSCDC_CHAR(NAME, STRING, CVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.187. SET_EXT_VSCDC_INT

Use this API to set voltage source converter dc line integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_vscdc_int(name, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_VSCDC_INT(NAME, STRING, IVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.188. SET_EXT_VSCDC_LOGICAL

Use this API to set voltage source converter dc line logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_vscdc_logical(name, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_VSCDC_LOGICAL(NAME, STRING, LVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.189. SET_EXT_VSCDC_REAL

Use this API to set voltage source converter dc line real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_vscdc_real(name, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_VSCDC_REAL(NAME, STRING, RVAL, IERR)
```

Where:

Character NAME	VSCDC name (input).	
Character STRING	String indicating the extension VSCDC field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.190. SET_EXT_ZONE_CHAR

Use this API to set zone character extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_zone_char(izone, string, cval)
```

Fortran command syntax:

```
CALL SET_EXT_ZONE_CHAR(IZONE, STRING, CVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Character*100 CVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'CVAL' set
	IERR = 1	Element not found; 'CVAL' not set
	IERR = 2	Bad value of 'STRING'; 'CVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'CVAL' not set
	IERR = 4	Unknown error with data access; 'CVAL' not set
	IERR = 10	API Prereqs not met; 'CVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.191. SET_EXT_ZONE_INT

Use this API to set zone integer extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_zone_int(izone, string, ival)
```

Fortran command syntax:

```
CALL SET_EXT_ZONE_INT(IZONE, STRING, IVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Integer IVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'IVAL' set
	IERR = 1	Element not found; 'IVAL' not set
	IERR = 2	Bad value of 'STRING'; 'IVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'IVAL' not set
	IERR = 4	Unknown error with data access; 'IVAL' not set
	IERR = 10	API Prereqs not met; 'IVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.192. SET_EXT_ZONE_LOGICAL

Use this API to set zone logical extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_zone_logical(izone, string, lval)
```

Fortran command syntax:

```
CALL SET_EXT_ZONE_LOGICAL(IZONE, STRING, LVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Logical LVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'LVAL' set
	IERR = 1	Element not found; 'LVAL' not set
	IERR = 2	Bad value of 'STRING'; 'LVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'LVAL' not set
	IERR = 4	Unknown error with data access; 'LVAL' not set
	IERR = 10	API Prereqs not met; 'LVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

12.193. SET_EXT_ZONE_REAL

Use this API to set zone real extension data.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr = set_ext_zone_real(izone, string, rval)
```

Fortran command syntax:

```
CALL SET_EXT_ZONE_REAL(IZONE, STRING, RVAL, IERR)
```

Where:

Integer IZONE	ZONE izone (input).	
Character STRING	String indicating the extension ZONE field desired (input).	
Real RVAL	Value indicated by STRING (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'RVAL' set
	IERR = 1	Element not found; 'RVAL' not set
	IERR = 2	Bad value of 'STRING'; 'RVAL' not set
	IERR = 3	Bad data type for field 'STRING'; 'RVAL' not set
	IERR = 4	Unknown error with data access; 'RVAL' not set
	IERR = 10	API Prereqs not met; 'RVAL' not set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 13
Subsystem Data Retrieval

The subsystem data retrieval APIs are used to retrieve data from the PSS®E engine for elements (e.g., buses, branches, etc.) in a subsystem. One or more input data items to each API allow the user to restrict the elements for which data is returned to a subset of subsystem elements (e.g., to omit out-of-service elements).

These APIs are grouped into families with each family containing up to six APIs:

- An API that returns the number of array entries required to hold one set of subsystem data items returned by the data retrieval APIs in the family (e.g., GET_BUS_COUNT returns the number of buses in subsystem SID that meet the editing criterion indicated by FLAG).
- A data retrieval API that returns an array of one or more sets of integer data values.
- A data retrieval API that returns an array of one or more sets of real data values.
- A data retrieval API that returns an array of one or more sets of complex data values.
- A data retrieval API that returns an array of one or more sets of character data values.
- An API that returns information on the data types corresponding to specified STRING values, each of which may be specified to one or more of the data retrieval APIs in the family. STRING values that are valid input values for both the real and complex data retrieval APIs of a family are indicated as being of type complex.

Each of the data retrieval APIs returns an array of one or more columns containing subsystem data items. Input arguments specify the name of the array to be used, the number data items to be retrieved (which is also the minimum number of columns required in the array), and the number of elements in each column (which must be at least as large as the number returned by the first API listed above). Array elements beyond those required to accommodate the requested data items are left unchanged by the data retrieval APIs.

These routines do not record themselves when called with recording enabled.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.1. Character Data

Prior to calling any of the subsystem data retrieval APIs, space for the array that is to receive the set(s) of data items that are to be returned must be allocated. For character data items, the number of characters required for each array entry for the data corresponding to any of the STRING arguments of the character data retrieval APIs (i.e., the a*Char APIs) may be obtained from the GET_CHAR_LENGTH API.

13.1.1. GET_CHAR_LENGTH

Use this API to return the number of characters required to accommodate the data corresponding to the specified STRING entries, which can be for any of the a*Char data retrieval functions.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, nchars = get_char_length(string)
```

Fortran command syntax:

```
CALL GET_CHAR_LENGTH(NSTR, STRING, NCHARS, IERR)
```

Where:

Integer NSTR	number of elements in STRING and NCHARS (1 < NSTR < 50); no default allowed	
Character STRING (NSTR)	NSTR valid strings for the a*Char data retrieval functions to be used (input; no default allowed).	
Integer NCHARS (NSTR)	NSTR entries containing the number of characters of each element of the array associated with the corresponding STRING value (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1-50	Position of an element in the array STRING that contains a value that is not recognized by any of the a*Char API routines
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.2. Bus Data

The bus data family of subsystem data retrieval APIs returns arrays of data for buses in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem buses, or to restrict the array entries to data for in-service buses.

Data for subsystem buses is returned from all APIs in the bus data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced.

13.2.1. ABUSCHAR

Use this API to return an array of character values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = abuschar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_BUS_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service buses.
	FLAG = 2	for all buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus quantities desired (input; no default allowed).	
	'NAME'	Bus name (12 characters).
	'EXNAME'	Extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of CARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.2.2. ABUSCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the bus data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, buses = abuscount(sid, flag)
```

Fortran command syntax:

```
CALL GET_BUS_COUNT(SID, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service buses.
	FLAG = 2	for all buses.
Integer BUSES	number of buses in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.2.3. ABUSCPLX

Use this API to return an array of complex values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = abuscplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_BUS_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service buses.
	FLAG = 2	for all buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus quantities desired (input; no default allowed).	
	'VOLTAGE'	Actual bus voltage, in pu, rectangular coordinates
	'SHUNTACT'	Actual total in-service fixed bus shunt, in MW and Mvar
	'O_SHUNTACT'	Actual total in-service fixed bus shunt, in units determined by the power output option setting
	'SHUNTNOM'	Nominal total in-service fixed bus shunt, in MW and Mvar at 1.0 pu voltage
	'O_SHUNTNOM'	Nominal total in-service fixed bus shunt, in units determined by the power output option setting at 1.0 pu voltage
	'SHUNTN'	Total in-service negative sequence load (pu nominal)
	'SHUNTZ'	Total in-service zero sequence load (pu nominal)
	'MISMATCH'	Bus mismatch, in MW and Mvar (0.0 if bus type code > 3)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_MISMATCH'	Bus mismatch, in units determined by the power output option setting (0.0 if bus type code > 3)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case (when STRING = 'SHUNTN' or 'SHUNTZ')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.2.4. ABUSINT

Use this API to return an array of integer values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = abusint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_BUS_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service buses.
	FLAG = 2	for all buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number
	'SECTION'	Bus section number, 0 if bus is not in a substation
	'STATION'	Bus substation number, 0 if bus is not in a substation
	'DUMMY'	Returns 1 if the bus is a dummy bus of a multisection line, or 0 if it is not
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).

Integer IERR — error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of IARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.2.5. ABUSREAL

Use this API to return an array of real values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = abusreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_BUS_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service buses.
	FLAG = 2	for all buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus quantities desired (input; no default allowed).	
	'BASE'	Bus base voltage, in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV (in pu if base voltage is 0.0)
	'ANGLE'	Bus voltage phase angle, in radians
	'ANGLED'	Bus voltage phase angle, in degrees
	'NVLMHI'	Normal high voltage limit, in pu
	'NVLMLO'	Normal low voltage limit, in pu
	'EVLMHI'	Emergency high voltage limit, in pu
	'EVLMLO'	Emergency low voltage limit, in pu
	'MISMATCH'	Bus mismatch, in MVA (0.0 if bus type code > 3)
	'O_MISMATCH'	Bus mismatch, in units determined by the power output option setting (0.0 if bus type code > 3)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.2.6. ABUSTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the bus data family (aBusInt, aBusReal, aBusCplx and aBusChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = abustypes(string)
```

Fortran command syntax:

```
CALL GET_BUS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the bus data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the bus data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.3. Plant Bus Data

The plant bus data family of subsystem data retrieval APIs returns arrays of data for buses in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem buses, or to restrict the array entries to data for a subset of buses according to their bus, plant, and/or machine configuration and statuses

Data for subsystem buses is returned from all APIs in the plant bus data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced.

13.3.1. AGENBUSCHAR

Use this API to return an array of character values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = agenbuschar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_PLANT_BUS_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service plant buses (type code 2 or 3) with at least one in-service machine.
	FLAG = 2	for only in-service plant buses (type code 2 or 3), including those with no in-service machines.
	FLAG = 3	for all in-service buses, including those that are not plant buses.
	FLAG = 4	for all plant buses, including those with no in-service machines.
	FLAG = 5	for all buses, including those that are not plant buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or plant quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'NAME'	Bus name (12 characters).
	'EXNAME'	Extended bus name (18 characters).
	'IREGNAME'	Bus name of regulated bus (12 characters).
	'IREGEXNAME'	Extended bus name of regulated bus (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_PLANT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_PLANT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.3.2. AGENBUSCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the plant bus data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, buses = agenbuscount(sid, flag)
```

Fortran command syntax:

```
CALL GET_PLANT_BUS_COUNT(SID, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service plant buses (type code 2 or 3) with at least one in-service machine.
	FLAG = 2	for only in-service plant buses (type code 2 or 3), including those with no in-service machines.
	FLAG = 3	for all in-service buses, including those that are not plant buses.
	FLAG = 4	for all plant buses, including those with no in-service machines.
	FLAG = 5	for all buses, including those that are not plant buses.
Integer BUSES	number of buses in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.3.3. AGENBUSCPLX

Use this API to return an array of complex values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = agenbuscplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_PLANT_BUS_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service plant buses (type code 2 or 3) with at least one in-service machine.
	FLAG = 2	for only in-service plant buses (type code 2 or 3), including those with no in-service machines.
	FLAG = 3	for all in-service buses, including those that are not plant buses.
	FLAG = 4	for all plant buses, including those with no in-service machines.
	FLAG = 5	for all buses, including those that are not plant buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or plant quantities desired (input; no default allowed).	
	'VOLTAGE'	Actual bus voltage, in pu, rectangular coordinates
	'PQGEN'	Power output of in-service machines at this plant (returns MW and Mvar)
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (returns MW and Mvar)
	'O_PQGEN'	Power output of in-service machines at this plant (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the GET_PLANT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_PLANT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of XARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.3.4. AGENBUSINT

Use this API to return an array of integer values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = agenbusint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_PLANT_BUS_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service plant buses (type code 2 or 3) with at least one in-service machine.
	FLAG = 2	for only in-service plant buses (type code 2 or 3), including those with no in-service machines.
	FLAG = 3	for all in-service buses, including those that are not plant buses.
	FLAG = 4	for all plant buses, including those with no in-service machines.
	FLAG = 5	for all buses, including those that are not plant buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or plant quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Substation number (0 if bus is not in a substation)
	'SECTION'	Bus section number (0 if bus is not represented by multiple bus sections)
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'DUMMY'	Returns 1 if the bus is a dummy bus of a multisection line or 0 if it is not
	'STATUS'	Inferred plant status. Returns 1 if the bus type code is 2 or 3 and at least one in-service machine is present or 0 otherwise
	'IREG'	Bus number of the regulated bus
	'NREG'	Node number of the regulated node
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_PLANT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BUSES	number of buses in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.3.5. AGENBUSREAL

Use this API to return an array of real values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = agenbusreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_PLANT_BUS_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service plant buses (type code 2 or 3) with at least one in-service machine.
	FLAG = 2	for only in-service plant buses (type code 2 or 3), including those with no in-service machines.
	FLAG = 3	for all in-service buses, including those that are not plant buses.
	FLAG = 4	for all plant buses, including those with no in-service machines.
	FLAG = 5	for all buses, including those that are not plant buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or plant quantities desired (input; no default allowed).	
	'BASE'	Bus base voltage in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV (in pu if base voltage is 0.0)
	'ANGLE'	Bus voltage phase angle, in radians
	'ANGLED'	Bus voltage phase angle, in degrees
	'PERCENT'	MVA loading as a percentage of total plant MVA base, MBASE
	'IREGBASE'	Regulated bus base voltage, in kV
	'IREGPU'	Actual regulated bus voltage magnitude, in pu

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'IREGKV'	Actual regulated bus voltage magnitude, in kV (in pu if base voltage is 0.0)
'VSPU'	Regulated bus scheduled voltage magnitude, in pu
'VSKV'	Regulated bus scheduled voltage magnitude, in kV (in pu if base voltage is 0.0)
'RMPCT'	Percent of total Mvar to be contributed by this plant
'PGEN'	Active power output of in-service machines at this plant, in MW
'QGEN'	Reactive power output of in-service machines at this plant, in Mvar
'MVA'	abs(PGEN+jQGEN) of in-service machines at this plant, in MVA
'PMAX'	Active power upper limit of in-service machines at this plant, in MW
'PMIN'	Active power lower limit of in-service machines at this plant, in MW
'QMAX'	Reactive power upper limit of in-service machines at this plant, in Mvar
'QMIN'	Reactive power lower limit of in-service machines at this plant, in Mvar
'MISMATCH'	Bus mismatch, in MVA (0.0 if bus type code > 3)
'O_PGEN'	Active power output of in-service machines at this plant (units determined by power output options)
'O_QGEN'	Reactive power output of in-service machines at this plant (units determined by power output options)
'O_MVA'	abs(PGEN+jQGEN) of in-service machines at this plant (units determined by power output options)
'O_PMAX'	Active power upper limit of in-service machines at this plant (units determined by power output options)
'O_PMIN'	Active power lower limit of in-service machines at this plant (units determined by power output options)
'O_QMAX'	Reactive power upper limit of in-service machines at this plant (units determined by power output options)
'O_QMIN'	Reactive power lower limit of in-service machines at this plant (units determined by power output options)
'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_PLANT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_PLANT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.3.6. AGENBUSTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the plant bus data family (aGenBusInt, aGenBusReal, aGenBusCplx and aGenBusChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = agenbustypes(string)
```

Fortran command syntax:

```
CALL GET_PLANT_BUS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the plant bus data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the plant bus data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.4. Machine Data

The machine data family of subsystem data retrieval APIs returns arrays of data for machines in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem machines, or to restrict the array entries to data for a subset of machines according to their bus, plant, and/or machine configuration and statuses.

Data for subsystem machines is returned from all APIs in the machine data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced; multiple machines at the same bus are in ascending machine identifier order.

13.4.1. AMACHCHAR

Use this API to return an array of character values for subsystem machines.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = amachchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_MACHINE_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem machines to include (input; 1 by default).	
	FLAG = 1	for only in-service machines at in-service plants (type code 2 or 3).
	FLAG = 2	for all machines at in-service plants (type code 2 or 3).
	FLAG = 3	for only in-service machines, including those at Type 1 and 4 buses.
	FLAG = 4	for all machines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or machine quantities desired (input; no default allowed).	
	'ID'	Machine identifier (2 characters).
	'NAME'	Bus name (12 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'EXNAME'	Extended bus name (18 characters).
	'POSTBL'	Non-conventional source positive sequence table (12 characters).
	'NEGTBL'	Non-conventional source negative sequence table (12 characters).
	'DROOPNAME'	Voltage droop control name (40 characters).
	'MCNAME'	machine name (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of CARRAY that are set by this API. MACHS is identical to the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.4.2. AMACHCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the machine data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, machs = amachcount(sid, flag)
```

Fortran command syntax:

```
CALL GET_MACHINE_COUNT(SID, FLAG, MACHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem machines to include (input; 1 by default).	
	FLAG = 1	for only in-service machines at in-service plants (type code 2 or 3).
	FLAG = 2	for all machines at in-service plants (type code 2 or 3).
	FLAG = 3	for only in-service machines, including those at type 1 and 4 buses.
	FLAG = 4	for all machines.
Integer MACHS	number of machines in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, MACHS returned
	IERR = 1	Working case is empty; MACHS returned as 0
	IERR = 2	Invalid SID value; MACHS returned as 0
	IERR = 3	Invalid FLAG value; MACHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.4.3. AMACHCPLX

Use this API to return an array of complex values for subsystem machines.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = amachcplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_MACHINE_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem machines to include (input; 1 by default).	
	FLAG = 1	for only in-service machines at in-service plants (type code 2 or 3).
	FLAG = 2	for all machines at in-service plants (type code 2 or 3).
	FLAG = 3	for only in-service machines, including those at Type 1 and 4 buses.
	FLAG = 4	for all machines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or machine quantities desired (input; no default allowed).	
	'ZSORCE'	Machine impedance (ZR + j ZX)
	'XTRAN'	Step-up transformer impedance (RT + j XT)
	'ZPOS'	" Positive sequence fault analysis machine impedance (Rpos + j X"")"
	'ZNEG'	Negative sequence machine impedance (ZRNEG + j ZXNEG)
	'ZZERO'	Zero sequence machine impedance (RZERO + j XZERO)
	'ZGRND'	Grounding impedance. If the grounding impedance data I/O code is 2 and the bus base voltage is positive the impedance is returned in ohms; otherwise it is return in per unit.
	'ZGRNDPU'	Grounding impedance in per unit
	'PQGEN'	Power output (MW and Mvar)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_PQGEN'	Power output (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of XARRAY that are set by this API. MACHS is identical to the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case (when STRING = 'ZPOS', 'ZNEG', 'ZZERO', 'ZGRND' or 'ZGRNDPU')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.4.4. AMACHINT

Use this API to return an array of integer values for subsystem machines.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = amachint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_MACHINE_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem machines to include (input; 1 by default).	
	FLAG = 1	for only in-service machines at in-service plants (type code 2 or 3).
	FLAG = 2	for all machines at in-service plants (type code 2 or 3).
	FLAG = 3	for only in-service machines, including those at Type 1 and 4 buses.
	FLAG = 4	for all machines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or machine quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Bus substation number (returned as 0 if the bus is not in a substation)
	'SECTION'	Bus section number (returned as 0 if the bus is not in a substation)
	'NODE'	Number of the node to which the machine is connected (returned as 0 if the bus is not in a substation)
	'BASFLG'	Baseloaded flag value
	'STATUS'	Machine status
	'WMOD'	Wind machine reactive power limits mode; 0 if the machine is not a wind machine
	'OWNERS'	Number of owners
	'OWN1'	First owner

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'OWN2'	Second owner
	'OWN3'	Third owner
	'OWN4'	Fourth owner
	'CZG'	Grounding impedance data input/outout (I/O) code (1 for per unit 2 for ohms)
	'NCSTYP'	Non-conventional source characteristic type
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of IARRAY that are set by this API. MACHS is identical to the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case (when STRING = 'CZG')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.4.5. AMACHREAL

Use this API to return an array of real values for subsystem machines.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = amachreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_MACHINE_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem machines to include (input; 1 by default).	
	FLAG = 1	for only in-service machines at in-service plants (type code 2 or 3).
	FLAG = 2	for all machines at in-service plants (type code 2 or 3).
	FLAG = 3	for only in-service machines, including those at Type 1 and 4 buses.
	FLAG = 4	for all machines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or machine quantities desired (input; no default allowed).	
	'FRACT1'	First owner fraction
	'FRACT2'	Second owner fraction
	'FRACT3'	Third owner fraction
	'FRACT4'	Fourth owner fraction
	'PERCENT'	MVA loading as a percentage of machine MVA base, MBASE
	'MBASE'	Machine MVA base
	'GENTAP'	Step-up transformer tap ratio
	'WPF'	Power factor used in setting reactive power limits for the wind machine when its WMOD is 2 or 3; 1.0 if the machine is not a wind machine
	'RPOS'	Positive sequence fault analysis machine resistance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'XSUBTR'	Positive sequence fault analysis machine sub-transient reactance
	'XTRANS'	Positive sequence fault analysis machine transient reactance
	'XSYNCH'	Positive sequence fault analysis machine synchronous reactance
	'PGEN'	Active power output, in MW
	'QGEN'	Reactive power output, in Mvar
	'MVA'	abs(PGEN+jQGEN), in MVA
	'PMAX'	Active power upper limit, in MW
	'PMIN'	Active power lower limit, in MW
	'QMAX'	Reactive power upper limit, in Mvar
	'QMIN'	Reactive power lower limit, in Mvar
	'O_PGEN'	Active power output (units determined by power output options)
	'O_QGEN'	Reactive power output (units determined by power output options)
	'O_MVA'	abs(PGEN+jQGEN) (units determined by power output options)
	'O_PMAX'	Active power upper limit (units determined by power output options)
	'O_PMIN'	Active power lower limit (units determined by power output options)
	'O_QMAX'	Reactive power upper limit (units determined by power output options)
	'O_QMIN'	Reactive power lower limit (units determined by power output options)
	'IFMAX'	Non-conventional source per unit maximum total fault current magnitude
	'DBVMIN'	Non-conventional source per unit dead-band minimum voltage
	'DBVMAX'	Non-conventional source per unit dead-band maximum voltage
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of RARRAY that are set by this API. MACHS is identical to the value of MACHS returned by the GET_MACHINE_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value
IERR = 7	Sequence data not in case (when STRING = 'RPOS', 'XSUBTR', 'XTRANS', or 'XSYNCH')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.4.6. AMACHTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the machine data family (aMachInt, aMachReal, aMachCplx and aMachChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = amachtypes(string)
```

Fortran command syntax:

```
CALL GET_MACHINE_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the machine data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the machine data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.5. Load Bus Data

The load bus data family of subsystem data retrieval APIs returns arrays of data for buses in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem buses, or to restrict the array entries to data for a subset of buses according to their bus and/or load configuration and statuses.

Data for subsystem buses is returned from all APIs in the load bus data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced.

13.5.1. ALODBUSCHAR

Use this API to return an array of character values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = alodbuschar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_BUS_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service load buses (type code 1, 2 or 3) with at least one in-service load.
	FLAG = 2	for only in-service load buses (type code 1, 2 or 3), including those with only out-of-service loads.
	FLAG = 3	for all in-service buses, including those that are not load buses.
	FLAG = 4	for all load buses, including those with only out-of-service loads.
	FLAG = 5	for all buses, including those that are not load buses.
	FLAG = 10	for only in-service load buses (type code 1, 2 or 3) with at least one in-service scalable load.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 11	for only in-service load buses (type code 1, 2 or 3) with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 12	for all load buses with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 20	for only in-service load buses (type code 1, 2 or 3) with at least one in-service fixed load.
	FLAG = 21	for only in-service load buses (type code 1, 2 or 3) with at least one fixed load, including those with all such loads out-of-service.
	FLAG = 22	for all load buses with at least one fixed load, including those with all such loads out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'NAME'	Bus name (12 characters).
	'EXNAME'	Extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.5.2. ALODBUSCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the load bus data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, buses = alodbuscount(sid, flag)
```

Fortran command syntax:

```
CALL GET_LOAD_BUS_COUNT(SID, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service load buses (type code 1, 2 or 3) with at least one in-service load.
	FLAG = 2	for only in-service load buses (type code 1, 2 or 3), including those with only out-of-service loads.
	FLAG = 3	for all in-service buses, including those that are not load buses.
	FLAG = 4	for all load buses, including those with only out-of-service loads.
	FLAG = 5	for all buses, including those that are not load buses.
	FLAG = 10	for only in-service load buses (type code 1, 2 or 3) with at least one in-service scalable load.
	FLAG = 11	for only in-service load buses (type code 1, 2 or 3) with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 12	for all load buses with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 20	for only in-service load buses (type code 1, 2 or 3) with at least one in-service fixed load.
	FLAG = 21	for only in-service load buses (type code 1, 2 or 3) with at least one fixed load, including those with all such loads out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 22	for all load buses with at least one fixed load, including those with all such loads out-of-service.
Integer BUSES	number of buses in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.5.3. ALODBUSCPLX

Use this API to return an array of complex values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = alodbuscplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_BUS_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service load buses (type code 1, 2 or 3) with at least one in-service load.
	FLAG = 2	for only in-service load buses (type code 1, 2 or 3), including those with only out-of-service loads.
	FLAG = 3	for all in-service buses, including those that are not load buses.
	FLAG = 4	for all load buses, including those with only out-of-service loads.
	FLAG = 5	for all buses, including those that are not load buses.
	FLAG = 10	for only in-service load buses (type code 1, 2 or 3) with at least one in-service scalable load.
	FLAG = 11	for only in-service load buses (type code 1, 2 or 3) with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 12	for all load buses with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 20	for only in-service load buses (type code 1, 2 or 3) with at least one in-service fixed load.
	FLAG = 21	for only in-service load buses (type code 1, 2 or 3) with at least one fixed load, including those with all such loads out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 22	for all load buses with at least one fixed load, including those with all such loads out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'VOLTAGE'	Actual bus voltage in pu rectangular coordinates
	'MVAACT'	Actual in-service constant MVA load
	'MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage
	'ILACT'	Actual in-service constant current load
	'ILNOM'	Nominal in-service constant current load at 1.0 pu voltage
	'YLACT'	Actual total in-service constant admittance load
	'YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage
	'TOTALACT'	Actual in-service load
	'TOTALNOM'	Nominal in-service load at 1.0 pu voltage
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3)
	'O_MVAACT'	Actual in-service constant MVA load (units determined by power output options)
	'O_MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage (units determined by power output options)
	'O_ILACT'	Actual in-service constant current load (units determined by power output options)
	'O_ILNOM'	Nominal in-service constant current load at 1.0 pu voltage (units determined by power output options)
	'O_YLACT'	Actual total in-service constant admittance load (units determined by power output options)
	'O_YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage (units determined by power output options)
	'O_TOTALACT'	Actual in-service load (units determined by power output options)
	'O_TOTALNOM'	Nominal in-service load at 1.0 pu voltage (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of XARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.5.4. ALODBUSINT

Use this API to return an array of integer values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = alodbusint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_BUS_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service load buses (type code 1, 2 or 3) with at least one in-service load.
	FLAG = 2	for only in-service load buses (type code 1, 2 or 3), including those with only out-of-service loads.
	FLAG = 3	for all in-service buses, including those that are not load buses.
	FLAG = 4	for all load buses, including those with only out-of-service loads.
	FLAG = 5	for all buses, including those that are not load buses.
	FLAG = 10	for only in-service load buses (type code 1, 2 or 3) with at least one in-service scalable load.
	FLAG = 11	for only in-service load buses (type code 1, 2 or 3) with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 12	for all load buses with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 20	for only in-service load buses (type code 1, 2 or 3) with at least one in-service fixed load.
	FLAG = 21	for only in-service load buses (type code 1, 2 or 3) with at least one fixed load, including those with all such loads out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 22	for all load buses with at least one fixed load, including those with all such loads out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Substation number (0 if bus is not in a substation)
	'SECTION'	Bus section number (0 if bus is not represented by multiple bus sections)
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number
	'DUMMY'	Returns 1 if the bus is a dummy bus of a multisection line, or 0 if it is not
	'STATUS'	Inferred load status. Returns 1 if the bus type code is 1, 2 or 3 and at least one in-service load is present or 0 otherwise
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.5.5. ALODBUSREAL

Use this API to return an array of real values for subsystem buses.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = alodbusreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_BUS_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service load buses (type code 1, 2 or 3) with at least one in-service load.
	FLAG = 2	for only in-service load buses (type code 1, 2 or 3), including those with only out-of-service loads.
	FLAG = 3	for all in-service buses, including those that are not load buses.
	FLAG = 4	for all load buses, including those with only out-of-service loads.
	FLAG = 5	for all buses, including those that are not load buses.
	FLAG = 10	for only in-service load buses (type code 1, 2 or 3) with at least one in-service scalable load.
	FLAG = 11	for only in-service load buses (type code 1, 2 or 3) with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 12	for all load buses with at least one scalable load, including those with all such loads out-of-service.
	FLAG = 20	for only in-service load buses (type code 1, 2 or 3) with at least one in-service fixed load.
	FLAG = 21	for only in-service load buses (type code 1, 2 or 3) with at least one fixed load, including those with all such loads out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 22	for all load buses with at least one fixed load, including those with all such loads out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'BASE'	Bus base voltage, in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV (in pu if base voltage is 0.0)
	'ANGLE'	Bus voltage phase angle, in radians
	'ANGLED'	Bus voltage phase angle, in degrees
	'MVAACT'	Actual in-service constant MVA load
	'MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage
	'ILACT'	Actual in-service constant current load
	'ILNOM'	Nominal in-service constant current load at 1.0 pu voltage
	'YLACT'	Actual total in-service constant admittance load
	'YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage
	'TOTALACT'	Actual in-service load
	'TOTALNOM'	Nominal in-service load at 1.0 pu voltage
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3)
	'O_MVAACT'	Actual in-service constant MVA load (units determined by power output options)
	'O_MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage (units determined by power output options)
	'O_ILACT'	Actual in-service constant current load (units determined by power output options)
	'O_ILNOM'	Nominal in-service constant current load at 1.0 pu voltage (units determined by power output options)
	'O_YLACT'	Actual total in-service constant admittance load (units determined by power output options)
	'O_YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage (units determined by power output options)
	'O_TOTALACT'	Actual in-service load (units determined by power output options)
	'O_TOTALNOM'	Nominal in-service load at 1.0 pu voltage (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_LOAD_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Real RARRAY (NSTR*DIM)	(output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of RARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.5.6. ALODBUSTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the load bus data family (aLodBusInt, aLodBusReal, aLodBusCplx and aLodBusChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = alodbustypes(string)
```

Fortran command syntax:

```
CALL GET_LOAD_BUS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the load bus data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the load bus data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.6. Load Data

The load data family of subsystem data retrieval APIs returns arrays of data for loads in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem loads, or to restrict the array entries to data for a subset of loads according to their bus and/or load configuration and statuses.

Data for subsystem loads is returned from all APIs in the load data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced; multiple loads at the same bus are in ascending load identifier order.

13.6.1. ALOADCHAR

Use this API to return an array of character values for subsystem loads.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = aloadchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,LOADS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem loads to include (input; 1 by default).	
	FLAG = 1	for only in-service loads at in-service buses (type code < 4).
	FLAG = 2	for all loads at in-service buses (type code < 4).
	FLAG = 3	for only in-service loads, including those at type 4 buses.
	FLAG = 4	for all loads.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'ID'	Load identifier (2 characters).
	'NAME'	Bus name (12 characters).
	'EXNAME'	Extended bus name (18 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'LODTYPE'	Load type description (12 characters).
	'LOADNAME'	Load name description (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer LOADS	number of elements in each column of CARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.6.2. ALOADCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the load data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, loads = aloadcount(sid, flag)
```

Fortran command syntax:

```
CALL GET_LOAD_COUNT(SID, FLAG, LOADS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem loads to include (input; 1 by default).	
	FLAG = 1	for only in-service loads at in-service buses (type code < 4).
	FLAG = 2	for all loads at in-service buses (type code < 4).
	FLAG = 3	for only in-service loads, including those at type 4 buses.
	FLAG = 4	for all loads.
Integer LOADS	number of loads in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, LOADS returned
	IERR = 1	Working case is empty; LOADS returned as 0
	IERR = 2	Invalid SID value; LOADS returned as 0
	IERR = 3	Invalid FLAG value; LOADS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.6.3. ALOADCPLX

Use this API to return an array of complex values for subsystem loads.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = aloadcplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,LOADS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem loads to include (input; 1 by default).	
	FLAG = 1	for only in-service loads at in-service buses (type code < 4).
	FLAG = 2	for all loads at in-service buses (type code < 4).
	FLAG = 3	for only in-service loads, including those at type 4 buses.
	FLAG = 4	for all loads.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired. NOTE: In the description below, the "net" values is the load value minus any in-service distributed generation on that load feeder. (input; no default allowed).	
	'MVAACT'	Actual in-service constant MVA load (in MW and Mvar)
	'MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage (in MW and Mvar)
	'ILACT'	Actual in-service constant current load (in MW and Mvar)
	'ILNOM'	Nominal in-service constant current load at 1.0 pu voltage (in MW and Mvar)
	'YLACT'	Actual total in-service constant admittance load (in MW and Mvar)
	'YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage (in MW and Mvar)
	'TOTALACT'	Actual in-service load (in MW and Mvar)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'TOTALNOM'	Nominal in-service load at 1.0 pu voltage (in MW and Mvar)
	'LDGNACT'	Actual in-service distributed generation
	'LDGNNOM'	Nominal in-service distributed generation
	'YNEG'	Nominal exceptional negative sequence load (at 1.0 pu voltage)
	'YZERO'	Nominal exceptional zero sequence load (at 1.0 pu voltage)
	'O_MVAACT'	Actual in-service constant MVA load (units determined by power output options)
	'O_MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage (units determined by power output options)
	'O_ILACT'	Actual in-service constant current load (units determined by power output options)
	'O_ILNOM'	Nominal in-service constant current load at 1.0 pu voltage (units determined by power output options)
	'O_YLACT'	Actual total in-service constant admittance load (units determined by power output options)
	'O_YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage (units determined by power output options)
	'O_TOTALACT'	Actual in-service load (units determined by power output options)
	'O_TOTALNOM'	Nominal in-service load at 1.0 pu voltage (units determined by power output options)
	'O_LDGNACT'	Actual in-service distributed generation
	'O_LDGNNOM'	Nominal in-service distributed generation
	'O_YNEG'	Nominal exceptional negative sequence load (at 1.0 pu voltage)
	'O_YZERO'	Nominal exceptional zero sequence load (at 1.0 pu voltage)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer LOADS	number of elements in each column of XARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of XARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.6.4. ALOADINT

Use this API to return an array of integer values for subsystem loads.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = aloadint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,LOADS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem loads to include (input; 1 by default).	
	FLAG = 1	for only in-service loads at in-service buses (type code < 4).
	FLAG = 2	for all loads at in-service buses (type code < 4).
	FLAG = 3	for only in-service loads, including those at type 4 buses.
	FLAG = 4	for all loads.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Substation number (0 if bus is not in a substation)
	'SECTION'	Bus section number (0 if bus is not represented by multiple bus sections)
	'NODE'	Node number (0 if bus is not in a substation)
	'AREA'	Load area number
	'ZONE'	Load zone number
	'OWNER'	Load owner number
	'STATUS'	Load status
	'SCALE'	Scalable load flag
	'INTRPT'	Interruptible load flag (1 for interruptible, 0 for non-interruptible)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'CGR'	Grounding code (0 for an ungrounded load, 1 for a grounded load)
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer LOADS	number of elements in each column of IARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case (when STRING = 'CGR')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.6.5. ALOADREAL

Use this API to return an array of real values for subsystem loads.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = aloadreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_LOAD_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,LOADS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem loads to include (input; 1 by default).	
	FLAG = 1	for only in-service loads at in-service buses (type code < 4).
	FLAG = 2	for all loads at in-service buses (type code < 4).
	FLAG = 3	for only in-service loads, including those at type 4 buses.
	FLAG = 4	for all loads.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired. NOTE: In the description below, the "net" values is the load value minus any in-service distributed generation on that load feeder. (input; no default allowed).	
	'MVAACT'	Actual in-service constant MVA load, in MVA
	'MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage, in MVA
	'ILACT'	Actual in-service constant current load, in MVA
	'ILNOM'	Nominal in-service constant current load at 1.0 pu voltage, in MVA
	'YLACT'	Actual total in-service constant admittance load, in MVA
	'YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage, in MVA
	'TOTALACT'	Actual in-service load, in MVA

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'TOTALNOM'	Nominal in-service load at 1.0 pu voltage, in MVA
	'YNEG'	Nominal negative sequence total load at 1.0 pu voltage, in MVA
	'YZERO'	Nominal zero sequence total load at 1.0 pu voltage, in MVA
	'O_MVAACT'	Actual in-service constant MVA load (units determined by power output options)
	'O_MVANOM'	Nominal in-service constant MVA load at 1.0 pu voltage (units determined by power output options)
	'O_ILACT'	Actual in-service constant current load (units determined by power output options)
	'O_ILNOM'	Nominal in-service constant current load at 1.0 pu voltage (units determined by power output options)
	'O_YLACT'	Actual total in-service constant admittance load (units determined by power output options)
	'O_YLNOM'	Nominal total in-service constant admittance load at 1.0 pu voltage (units determined by power output options)
	'O_TOTALACT'	Actual in-service load (units determined by power output options)
	'O_TOTALNOM'	Nominal in-service load at 1.0 pu voltage (units determined by power output options)
	'O_YNEG'	Nominal negative sequence total load at 1.0 pu voltage (units determined by power output options)
	'O_YZERO'	Nominal zero sequence total load at 1.0 pu voltage (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer LOADS	number of elements in each column of RARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_LOAD_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value
IERR = 7	Sequence data not in case (when STRING = 'YNEG', 'O_YNEG', 'YZERO' or 'O_YZERO')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.6.6. ALOADTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the load data family (aLoadInt, aLoadReal, aLoadCplx and aLoadChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = aloadtypes(string)
```

Fortran command syntax:

```
CALL GET_LOAD_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the load data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the load data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.7. Fixed Shunt Bus Data

The fixed shunt bus data family of subsystem data retrieval APIs returns arrays of data for buses in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem buses, or to restrict the array entries to data for a subset of buses according to their bus and/or fixed shunt configuration and statuses.

Data for subsystem buses is returned from all APIs in the fixed shunt bus data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced.

13.7.1. AFXSHNTBUSCHAR

Use this API to return an array of character values for subsystem buses.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, carray = afxshntbuschar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_BUS_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunt buses (type code 1, 2 or 3) with at least one in-service fixed shunt.
	FLAG = 2	for only in-service fixed shunt buses (type code 1, 2 or 3), including those with only out-of-service fixed shunts.
	FLAG = 3	for all in-service buses, including those that are not fixed shunt buses.
	FLAG = 4	for all fixed shunt buses, including those with only out-of-service fixed shunts.
	FLAG = 5	for all buses, including those that are not fixed shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed). 'NAME' Bus name (12 characters). 'EXNAME' Extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output). IERR = 0 No error IERR = 1 Working case is empty IERR = 2 Invalid SID value IERR = 3 Invalid FLAG value IERR = 4 Invalid NSTR value IERR = 5 DIM, and hence the size of CARRAY, is not large enough IERR = 6 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.7.2. AFXSHNTBUSCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the fixed shunt bus data family.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, buses = afxshntbuscount(sid, flag)
```

Fortran command syntax:

```
CALL GET_SHUNT_BUS_COUNT(SID, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunt buses (type code 1, 2 or 3) with at least one in-service fixed shunt.
	FLAG = 2	for only in-service fixed shunt buses (type code 1, 2 or 3), including those with only out-of-service fixed shunts.
	FLAG = 3	for all in-service buses, including those that are not fixed shunt buses.
	FLAG = 4	for all fixed shunt buses, including those with only out-of-service fixed shunts.
	FLAG = 5	for all buses, including those that are not fixed shunt buses.
Integer BUSES	number of buses in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.7.3. AFXSHNTBUSCPLX

Use this API to return an array of complex values for subsystem buses.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, xarray = afxshntbuscplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_BUS_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunt buses (type code 1, 2 or 3) with at least one in-service fixed shunt.
	FLAG = 2	for only in-service fixed shunt buses (type code 1, 2 or 3), including those with only out-of-service fixed shunts.
	FLAG = 3	for all in-service buses, including those that are not fixed shunt buses.
	FLAG = 4	for all fixed shunt buses, including those with only out-of-service fixed shunts.
	FLAG = 5	for all buses, including those that are not fixed shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed).	
	'VOLTAGE'	Actual bus voltage, in pu, rectangular coordinates
	'PU_GBZERO'	Zero sequence total nominal in-service fixed bus shunt, in pu
	'SHUNTACT'	Total actual in-service fixed bus shunt (in MW and Mvar)
	'SHUNTNOM'	Total nominal in-service fixed bus shunt at 1.0 pu voltage (in MW and Mvar)
	'GBZERO'	Zero sequence total nominal in-service fixed bus shunt (in MW and Mvar)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (in MW and Mvar)
	'O_SHUNTACT'	Total actual in-service fixed bus shunt (units determined by power output options)
	'O_SHUNTNOM'	Total nominal in-service fixed bus shunt at 1.0 pu voltage (units determined by power output options)
	'O_GBZERO'	Zero sequence total nominal in-service fixed bus shunt (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case; when STRING = 'PU_GBZERO', 'GBZERO' and 'O_GBZERO'

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.7.4. AFXSHNTBUSINT

Use this API to return an array of integer values for subsystem buses.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, iarray = afxshntbusint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_BUS_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunt buses (type code 1, 2 or 3) with at least one in-service fixed shunt.
	FLAG = 2	for only in-service fixed shunt buses (type code 1, 2 or 3), including those with only out-of-service fixed shunts.
	FLAG = 3	for all in-service buses, including those that are not fixed shunt buses.
	FLAG = 4	for all fixed shunt buses, including those with only out-of-service fixed shunts.
	FLAG = 5	for all buses, including those that are not fixed shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Substation number (0 if bus is not in a substation)
	'SECTION'	Bus section number (0 if bus is not represented by multiple bus sections)
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'DUMMY'	Returns 1 if the bus is a dummy bus of a multisection line or 0 if it is not
	'STATUS'	Inferred shunt status. Returns 1 if the bus type code is 1 2 or 3 and at least one in-service fixed shunt is present or 0 otherwise
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.7.5. AFXSHNTBUSREAL

Use this API to return an array of real values for subsystem buses.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, rarray = afxshntbusreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_BUS_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunt buses (type code 1, 2 or 3) with at least one in-service fixed shunt.
	FLAG = 2	for only in-service fixed shunt buses (type code 1, 2 or 3), including those with only out-of-service fixed shunts.
	FLAG = 3	for all in-service buses, including those that are not fixed shunt buses.
	FLAG = 4	for all fixed shunt buses, including those with only out-of-service fixed shunts.
	FLAG = 5	for all buses, including those that are not fixed shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed).	
	'BASE'	Bus base voltage, in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV (in pu if base voltage is 0.0)
	'ANGLE'	Bus voltage phase angle, in radians
	'ANGLED'	Bus voltage phase angle, in degrees
	'PU_GBZERO'	Zero sequence total nominal in-service fixed bus shunt, in pu
	'SHUNTACT'	Total actual in-service fixed bus shunt, in MVA

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'SHUNTNOM'	Total nominal in-service fixed bus shunt at 1.0 pu voltage, in MVA
	'GBZERO'	Zero sequence total nominal in-service fixed bus shunt, in MVA
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3), in MVA
	'O_SHUNTACT'	Total actual in-service fixed bus shunt (units determined by power output options)
	'O_SHUNTNOM'	Total nominal in-service fixed bus shunt at 1.0 pu voltage (units determined by power output options)
	'O_GBZERO'	Zero sequence total nominal in-service fixed bus shunt (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of RARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case; when STRING = 'PU_GBZERO', 'GBZERO' and 'O_GBZERO'

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.7.6. AFXSHNTBUSTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the fixed shunt bus data family (aFxShntBusInt, aFxShntBusReal, aFxShntBusCplx and aFxShntBusChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = afxshntbustypes(string)
```

Fortran command syntax:

```
CALL GET_SHUNT_BUS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the fixed shunt bus data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the fixed shunt bus data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.8. Fixed Shunt Data

The fixed shunt data family of subsystem data retrieval APIs returns arrays of data for fixed bus shunts in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem fixed bus shunts, or to restrict the array entries to data for a subset of fixed bus shunts according to their bus and/or fixed shunt configuration and statuses.

Data for subsystem fixed bus shunts is returned from all APIs in the fixed shunt data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced; multiple fixed shunts at the same bus are in ascending shunt identifier order.

13.8.1. AFXSHUNTCHAR

Use this API to return an array of character values for subsystem fixed shunts.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, carray = afxshuntchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,SHUNTS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem fixed bus shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunts at in-service buses (type code < 4).
	FLAG = 2	for all fixed shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service fixed shunts, including those at type 4 buses.
	FLAG = 4	for all fixed bus shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed).	
	'ID'	Shunt identifier (2 characters).
	'NAME'	Bus name (12 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'EXNAME'	Extended bus name (18 characters).
	'SHNAME'	Shunt name (40 characters).
Integer DIM	number of elements in each column of the array CARRAY DIM must be as least as large as the value of SHUNTS returned by the GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer SHUNTS	number of elements in each column of CARRAY that are set by this API. SHUNTS is identical to the value of SHUNTS returned by the GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.8.2. AFXSHUNTCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the fixed shunt data family.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, shunts = afxshuntcount(sid, flag)
```

Fortran command syntax:

```
CALL GET_SHUNT_COUNT(SID, FLAG, SHUNTS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem fixed bus shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunts at in-service buses (type code < 4).
	FLAG = 2	for all fixed shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service fixed shunts, including those at type 4 buses.
	FLAG = 4	for all fixed bus shunts.
Integer SHUNTS	number of fixed bus shunts in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, SHUNTS returned
	IERR = 1	Working case is empty; SHUNTS returned as 0
	IERR = 2	Invalid SID value; SHUNTS returned as 0
	IERR = 3	Invalid FLAG value; SHUNTS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.8.3. AFXSHUNTCPLX

Use this API to return an array of complex values for subsystem fixed shunts.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, xarray = afxshuntcplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,SHUNTS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem fixed bus shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunts at in-service buses (type code < 4).
	FLAG = 2	for all fixed shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service fixed shunts, including those at type 4 buses.
	FLAG = 4	for all fixed bus shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed).	
	'PU_GBZERO'	Zero sequence nominal fixed bus shunt, in per unit
	'SHUNTACT'	Actual in-service fixed bus shunt (in MW and Mvar)
	'SHUNTNOM'	Nominal fixed bus shunt at 1.0 pu voltage (in MW and Mvar)
	'GBZERO'	Zero sequence nominal fixed bus shunt (in MW and Mvar)
	'O_SHUNTACT'	Actual in-service fixed bus shunt (units determined by power output options)
	'O_SHUNTNOM'	Nominal fixed bus shunt at 1.0 pu voltage (units determined by power output options)
	'O_GBZERO'	Zero sequence nominal fixed bus shunt (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of SHUNTS returned by the GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer SHUNTS	number of elements in each column of XARRAY that are set by this API. SHUNTS is identical to the value of SHUNTS returned by the GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of XARRAY, is not large enough
IERR = 6	Invalid STRING value
IERR = 7	Sequence data not in case; when STRING = 'PU_GBZERO', 'GBZERO' and 'O_GBZERO'

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.8.4. AFXSHUNTINT

Use this API to return an array of integer values for subsystem fixed shunts.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, iarray = afxshuntint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,SHUNTS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem fixed bus shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunts at in-service buses (type code < 4).
	FLAG = 2	for all fixed shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service fixed shunts, including those at type 4 buses.
	FLAG = 4	for all fixed bus shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Substation number (0 if bus is not in a substation)
	'SECTION'	Bus section number (0 if bus is not represented by multiple bus sections)
	'NODE'	Node number (0 if bus is not in a substation)
	'STATUS'	Shunt status
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of SHUNTS returned by the GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer SHUNTS	number of elements in each column of IARRAY that are set by this API. SHUNTS is identical to the value of SHUNTS returned by the GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.8.5. AFXSHUNTREAL

Use this API to return an array of real values for subsystem fixed shunts.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
ierr, rarray = afxshuntreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SHUNT_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,SHUNTS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem fixed bus shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service fixed shunts at in-service buses (type code < 4).
	FLAG = 2	for all fixed shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service fixed shunts, including those at type 4 buses.
	FLAG = 4	for all fixed bus shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or fixed shunt quantities desired (input; no default allowed).	
	'PU_GBZERO'	Zero sequence nominal fixed bus shunt, in pu
	'SHUNTACT'	Actual in-service fixed bus shunt, in MVA
	'SHUNTNOM'	Nominal fixed bus shunt at 1.0 pu voltage, in MVA
	'GBZERO'	Zero sequence nominal fixed bus shunt, in MVA
	'O_SHUNTACT'	Actual in-service fixed bus shunt (units determined by power output options)
	'O_SHUNTNOM'	Nominal fixed bus shunt at 1.0 pu voltage (units determined by power output options)
	'O_GBZERO'	Zero sequence nominal fixed bus shunt (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of SHUNTS returned by the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer SHUNTS	number of elements in each column of RARRAY that are set by this API. SHUNTS is identical to the value of SHUNTS returned by the GET_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value
IERR = 7	Sequence data not in case; when STRING = 'PU_GBZERO', 'GBZERO' and 'O_GBZERO'

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.8.6. AFXSHUNTTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the fixed shunt data family (aFxShuntInt, aFxShuntReal, aFxShuntCplx and aFxShuntChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = afxshunttypes(string)
```

Fortran command syntax:

```
CALL GET_SHUNT_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the fixed shunt data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the fixed shunt data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.9. Switched Shunt Bus Data

The switched shunt bus data family of subsystem data retrieval APIs returns arrays of data for buses in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem buses, or to restrict the array entries to data for a subset of buses according to their bus and/or switched shunt configuration and statuses.

Data for subsystem buses is returned from all APIs in the switched shunt bus data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced.

13.9.1. ASWSHUNTBUSCHAR

Use this API to return an array of character values for subsystem buses.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, carray = aswshuntbuschar(sid, flag, string)
```

Fortran command syntax:

```
CALL
 GET_SWITCHED_SHUNT_BUS_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunt buses (type code 1, 2 or 3) with at least one in-service switched shunt.
	FLAG = 2	for only in-service switched shunt buses (type code 1, 2 or 3), including those with only out-of-service switched shunts.
	FLAG = 3	for all in-service buses, including those that are not switched shunt buses.
	FLAG = 4	for all switched shunt buses, including those with only out-of-service switched shunts.
	FLAG = 5	for all buses, including those that are not switched shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed). 'NAME' Bus name (12 characters). 'EXNAME' Extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output). IERR = 0 No error IERR = 1 Working case is empty IERR = 2 Invalid SID value IERR = 3 Invalid FLAG value IERR = 4 Invalid NSTR value IERR = 5 DIM, and hence the size of CARRAY, is not large enough IERR = 6 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.9.2. ASWSHUNTBUSCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the switched shunt bus data family.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, buses = aswshntbuscount(sid, flag)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_BUS_COUNT(SID, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunt buses (type code 1, 2 or 3) with at least one in-service switched shunt.
	FLAG = 2	for only in-service switched shunt buses (type code 1, 2 or 3), including those with only out-of-service switched shunts.
	FLAG = 3	for all in-service buses, including those that are not switched shunt buses.
	FLAG = 4	for all switched shunt buses, including those with only out-of-service switched shunts.
	FLAG = 5	for all buses, including those that are not switched shunt buses.
Integer BUSES	number of buses in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.9.3. ASWSHUNTBUSCPLX

Use this API to return an array of complex values for subsystem buses.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, xarray = aswshntbuscplx(sid, flag, string)
```

Fortran command syntax:

```
CALL
 GET_SWITCHED_SHUNT_BUS_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunt buses (type code 1, 2 or 3) with at least one in-service switched shunt.
	FLAG = 2	for only in-service switched shunt buses (type code 1, 2 or 3), including those with only out-of-service switched shunts.
	FLAG = 3	for all in-service buses, including those that are not switched shunt buses.
	FLAG = 4	for all switched shunt buses, including those with only out-of-service switched shunts.
	FLAG = 5	for all buses, including those that are not switched shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed).	
	'VOLTAGE'	Actual bus voltage, in pu, rectangular coordinates
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (in MW and Mvar)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output option setting)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of XARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.9.4. ASWSHUNTBUSINT

Use this API to return an array of integer values for subsystem buses.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, iarray = aswshntbusint(sid, flag, string)
```

Fortran command syntax:

```
CALL
 GET_SWITCHED_SHUNT_BUS_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunt buses (type code 1, 2 or 3) with at least one in-service switched shunt.
	FLAG = 2	for only in-service switched shunt buses (type code 1, 2 or 3), including those with only out-of-service switched shunts.
	FLAG = 3	for all in-service buses, including those that are not switched shunt buses.
	FLAG = 4	for all switched shunt buses, including those with only out-of-service switched shunts.
	FLAG = 5	for all buses, including those that are not switched shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Substation number (0 if bus is not in a substation)
	'SECTION'	Bus section number (0 if bus is not represented by multiple bus sections)
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'OWNER'	Bus owner number
	'DUMMY'	Returns 1 if the bus is a dummy bus of a multisection line or 0 if it is not
	'STATUS'	Inferred shunt status. Returns 1 if the bus type code is 1 2 or 3 and at least one in-service switched shunt is present or 0 otherwise
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.9.5. ASWSHUNTBUSREAL

Use this API to return an array of real values for subsystem buses.

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, rarray = afswhntbusreal(sid, flag, string)
```

Fortran command syntax:

```
CALL
 GET_SWITCHED_SHUNT_BUS_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunt buses (type code 1, 2 or 3) with at least one in-service switched shunt.
	FLAG = 2	for only in-service switched shunt buses (type code 1, 2 or 3), including those with only out-of-service switched shunts.
	FLAG = 3	for all in-service buses, including those that are not switched shunt buses.
	FLAG = 4	for all switched shunt buses, including those with only out-of-service switched shunts.
	FLAG = 5	for all buses, including those that are not switched shunt buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed).	
	'BASE'	Bus base voltage, in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV (in pu if base voltage is 0.0)
	'ANGLE'	Bus voltage phase angle, in radians
	'ANGLED'	Bus voltage phase angle, in degrees
	'PU_BSWZERO'	Zero sequence total nominal in-service switched bus shunt, in pu

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'BSWACT'	Total actual in-service switched bus shunt, in MVAR
	'BSWNOM'	Total nominal in-service switched bus shunt at 1.0 pu voltage, in MVAR
	'BSWZERO'	Zero sequence total nominal in-service switched bus shunt, in MVAR
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3), in MVA
	'O_BSWACT'	Total actual in-service switched bus shunt (units determined by power output option setting)
	'O_BSWNOM'	Total nominal in-service switched bus shunt at 1.0 pu voltage (units determined by power output option setting)
	'O_BSWZERO'	Zero sequence total nominal in-service switched bus shunt (units determined by power output option setting)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output option setting)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_SWITCHED_SHUNT_BUS_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of RARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case; when STRING = 'PU_BSWZERO', BSWBZERO' and 'O_BSWZERO'

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.9.6. ASWSHUNTBUSTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the switched shunt bus data family (aSwShntBusInt, aSwShntBusReal, aSwShntBusCplx and aSwShntBusChar).

This API routine was first introduced in release 35.0.0.

Python command syntax:

```
ierr, types = aswshntbustypes(string)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_BUS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the switched shunt bus data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the switched shunt bus data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.10. Switched Shunt Data

The switched shunt data family of subsystem data retrieval APIs returns arrays of data for switched shunts in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for all subsystem switched shunts, or to restrict the array entries to data for switched shunts at in-service buses.

Data for subsystem switched shunts is returned from all APIs in the switched shunt data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API is referenced.

13.10.1. ASWSHCHAR

Use this API to return an array of character values for subsystem switched shunts.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = aswshchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,SHNTS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem switched shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunts at in-service buses (type code < 4).
	FLAG = 2	for all switched shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service switched shunts, including those at type 4 buses.
	FLAG = 4	for all switched shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed).	
	'VSCNAME'	VSC dc line name; non-blank name is returned only for those switched shunts for which the control mode, MODSW, is 4 (12 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'FACTSNAME'	FACTS device name; non-blank name is returned only for those switched shunts for which the control mode, MODSW, is 6 (12 characters).
	'DEVICENAME'	VSC dc line name or FACTS device name; non-blank name is returned only for those switched shunts for which the control mode, MODSW, is 4 or 6 (12 characters).
	'ID'	Switched shunt identifier (2 characters).
	'NAME'	Bus name (12 characters).
	'EXNAME'	Extended bus name (18 characters).
	'IREGNAME'	Bus name of regulated bus (12 characters).
	'IREGEXNAME'	Extended bus name of regulated bus (18 characters).
	'SWSNAME'	name of switched shunt (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer SHNTS	number of elements in each column of CARRAY that are set by this API. SHNTS is identical to the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.10.2. ASWSHCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the switched shunt data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, shnts = aswshcount(sid, flag)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_COUNT(SID, FLAG, SHNTS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem switched shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunts at in-service buses (type code < 4).
	FLAG = 2	for all switched shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service switched shunts, including those at type 4 buses.
	FLAG = 4	for all switched shunts.
Integer SHNTS	number of switched shunts in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, SHNTS returned
	IERR = 1	Working case is empty; SHNTS returned as 0
	IERR = 2	Invalid SID value; SHNTS returned as 0
	IERR = 3	Invalid FLAG value; SHNTS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.10.3. ASWSHCPLX

Use this API to return an array of complex values for subsystem switched shunts.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = aswshcplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_COMPLEX(SID, FLAG, NSTR, STRING, DIM, XARRAY,
 SHNTS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem switched shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunts at in-service buses (type code < 4).
	FLAG = 2	for all switched shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service switched shunts, including those at type 4 buses.
	FLAG = 4	for all switched shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed).	
	'VOLTAGE'	Actual bus voltage in pu rectangular coordinates
	'YSWACT'	Present shunt switched on
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3)
	'O_YSWACT'	Present shunt switched on (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer SHNTS	number of elements in each column of XARRAY that are set by this API. SHNTS is identical to the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of XARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.10.4. ASWSHINT

Use this API to return an array of integer values for subsystem switched shunts.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = aswshint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_INTEGER(SID, FLAG, NSTR, STRING, DIM, IARRAY,
 SHNTS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem switched shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunts at in-service buses (type code < 4).
	FLAG = 2	for all switched shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service switched shunts, including those at type 4 buses.
	FLAG = 4	for all switched shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'STATION'	Substation number (0 if bus is not in a substation)
	'SECTION'	Bus section number (0 if bus is not represented by multiple bus sections)
	'NODE'	Node number (0 if bus is not in a substation)
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number
	'DUMMY'	Returns 1 if the bus is a dummy bus of a multisection line or 0 if it is not
	'MODE'	Switched shunt control mode

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'STATUS'	Switched shunt status flag
	'ADJMETHOD'	Switched shunt adjustment method
	'IREG'	Bus number of the regulated bus
	'NREG'	Node number of the regulated node
	'BLOCKS'	Number of blocks
	'STATUSBLOCK1'	Status of block 1
	'STATUSBLOCK2'	Status of block 2
	'STATUSBLOCK3'	Status of block 3
	'STATUSBLOCK4'	Status of block 4
	'STATUSBLOCK5'	Status of block 5
	'STATUSBLOCK6'	Status of block 6
	'STATUSBLOCK7'	Status of block 7
	'STATUSBLOCK8'	Status of block 8
	'STEPSBLOCK1'	Number of steps in block 1
	'STEPSBLOCK2'	Number of steps in block 2
	'STEPSBLOCK3'	Number of steps in block 3
	'STEPSBLOCK4'	Number of steps in block 4
	'STEPSBLOCK5'	Number of steps in block 5
	'STEPSBLOCK6'	Number of steps in block 6
	'STEPSBLOCK7'	Number of steps in block 7
	'STEPSBLOCK8'	Number of steps in block 8

Integer DIM

number of elements in each column of the array IARRAY. DIM must be as least as large as the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).

Integer IARRAY (NSTR*DIM)

Is an array of at least NSTR columns, each of which contains DIM elements (output).

Integer SHNTS

number of elements in each column of IARRAY that are set by this API. SHNTS is identical to the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (output).

Integer IERR

error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of IARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.10.5. ASWSHREAL

Use this API to return an array of real values for subsystem switched shunts.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = aswshreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,SHNTS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem switched shunts to include (input; 1 by default).	
	FLAG = 1	for only in-service switched shunts at in-service buses (type code < 4).
	FLAG = 2	for all switched shunts at in-service buses (type code < 4).
	FLAG = 3	for only in-service switched shunts, including those at type 4 buses.
	FLAG = 4	for all switched shunts.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or switched shunt quantities desired (input; no default allowed).	
	'BASE'	Bus base voltage in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV (in pu if base voltage is 0.0)
	'ANGLE'	Bus voltage phase angle, in radians
	'ANGLED'	Bus voltage phase angle, in degrees
	'VSWHI'	Controlled quantity upper limit
	'VSWLO'	Controlled quantity lower limit
	'RMPCT'	Percent of total Mvar to be contributed by this switched shunt
	'PU_BSWZERO'	Present zero sequence susceptance switched on, in pu
	'PUBZERBLOCK1'	Zero sequence pu step size in block 1
	'PUBZERBLOCK2'	Zero sequence pu step size in block 2

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'PUBZERBLOCK3'	Zero sequence pu step size in block 3
'PUBZERBLOCK4'	Zero sequence pu step size in block 4
'PUBZERBLOCK5'	Zero sequence pu step size in block 5
'PUBZERBLOCK6'	Zero sequence pu step size in block 6
'PUBZERBLOCK7'	Zero sequence pu step size in block 7
'PUBZERBLOCK8'	Zero sequence pu step size in block 8
'BSWNOM'	Present susceptance switched on (susceptance)
'BSWMAX'	Admittance with all reactors off, all capacitors on
'BSWMIN'	Admittance with all reactors on, all capacitors off
'BSWACT'	Present susceptance switched on (Mvar)
'BSWZERO'	Present zero sequence susceptance switched on
'BSTPBLOCK1'	Step size in block 1
'BSTPBLOCK2'	Step size in block 2
'BSTPBLOCK3'	Step size in block 3
'BSTPBLOCK4'	Step size in block 4
'BSTPBLOCK5'	Step size in block 5
'BSTPBLOCK6'	Step size in block 6
'BSTPBLOCK7'	Step size in block 7
'BSTPBLOCK8'	Step size in block 8
'BZERBLOCK1'	Zero sequence step size in block 1
'BZERBLOCK2'	Zero sequence step size in block 2
'BZERBLOCK3'	Zero sequence step size in block 3
'BZERBLOCK4'	Zero sequence step size in block 4
'BZERBLOCK5'	Zero sequence step size in block 5
'BZERBLOCK6'	Zero sequence step size in block 6
'BZERBLOCK7'	Zero sequence step size in block 7
'BZERBLOCK8'	Zero sequence step size in block 8
'MISMATCH'	Bus mismatch (0.0 if bus type code > 3)
'O_BSWNOM'	Present susceptance switched on (units determined by power output options)
'O_BSWMAX'	Admittance with all reactors off all capacitors on (units determined by power output options)
'O_BSWMIN'	Admittance with all reactors on all capacitors off (units determined by power output options)
'O_BSWACT'	Present susceptance switched on (units determined by power output options)
'O_BSWZERO'	Present zero sequence susceptance switched on (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_BSTPBLOCK1'	Step size in block 1 (units determined by power output options)
	'O_BSTPBLOCK2'	Step size in block 2 (units determined by power output options)
	'O_BSTPBLOCK3'	Step size in block 3 (units determined by power output options)
	'O_BSTPBLOCK4'	Step size in block 4 (units determined by power output options)
	'O_BSTPBLOCK5'	Step size in block 5 (units determined by power output options)
	'O_BSTPBLOCK6'	Step size in block 6 (units determined by power output options)
	'O_BSTPBLOCK7'	Step size in block 7 (units determined by power output options)
	'O_BSTPBLOCK8'	Step size in block 8 (units determined by power output options)
	'O_BZERBLOCK1'	Zero sequence step size in block 1 (units determined by power output options)
	'O_BZERBLOCK2'	Zero sequence step size in block 2 (units determined by power output options)
	'O_BZERBLOCK3'	Zero sequence step size in block 3 (units determined by power output options)
	'O_BZERBLOCK4'	Zero sequence step size in block 4 (units determined by power output options)
	'O_BZERBLOCK5'	Zero sequence step size in block 5 (units determined by power output options)
	'O_BZERBLOCK6'	Zero sequence step size in block 6 (units determined by power output options)
	'O_BZERBLOCK7'	Zero sequence step size in block 7 (units determined by power output options)
	'O_BZERBLOCK8'	Zero sequence step size in block 8 (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer SHNTS	number of elements in each column of RARRAY that are set by this API. SHNTS is identical to the value of SHNTS returned by the GET_SWITCHED_SHUNT_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value
IERR = 7	Sequence data not in case (when STRING = 'BSWZERO', 'PU_BSWZERO', 'O_BSWZERO', 'BZERBLOCKn', 'PUBZERBLOCKn', 'O_BZERBLOCKn')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.10.6. ASWSHTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the switched shunt data family (aSwshInt, aSwshReal, aSwshCplx and aSwshChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = aswshtypes(string)
```

Fortran command syntax:

```
CALL GET_SWITCHED_SHUNT_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the switched shunt data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the switched shunt data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.11. Branch Flow Data

The branch flow data family of subsystem data retrieval APIs returns arrays of loading data for ac branches in a specified subsystem. These APIs return loading data for non-transformer branches, two-winding transformers, and three-winding transformers.

One of the input items to each API allows the user to elect either to retrieve data for all subsystem branches, for subsystem tie branches, or for both subsystem and tie branches. Another input item allows the user to elect either to retrieve data for all such branches, or to restrict the array entries to data for in-service branches.

Data for subsystem branches is returned from all APIs in the branch flow data family in the same order. Unless otherwise restricted for tie branches, each branch appears in both directions (in all three directions for three-winding transformers) in the arrays returned by the APIs of the branch flow data family. Branches are ordered in ascending numerical or alphabetical order by from bus, according to the bus output option setting in effect at the time the API is referenced. For each from bus, branches are ordered in ascending order by to bus. Parallel branches are ordered in ascending circuit identifier order.

13.11.1. AFLOWCHAR

Use this API to return an array of character values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = aflowchar(sid, owner, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_FLOW_CHAR(SID, OWNER, TIES, FLAG, NSTR, STRING, DIM,
 CARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem branches only.
	TIES = 2	for the subsystem bus end of tie branches only.
	TIES = 3	for the non-subsystem bus end of tie branches only.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TIES = 4	for each end of tie branches only.
	TIES = 5	for each end of interior subsystem branches and the subsystem bus end of tie branches.
	TIES = 6	for each end of interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service branches.
	FLAG = 2	for all branches.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the branch quantities desired (input; no default allowed).	
	'ID'	Circuit identifier (2 characters).
	'FROMNAME'	From bus name (12 characters).
	'FROMEXNAME'	From bus extended bus name (18 characters).
	'TONAME'	To bus name (three-winding transformer name for a three-winding transformer winding) (12 characters).
	'TOEXNAME'	To bus extended bus name (three-winding transformer name and winding number for a three-winding transformer winding) (18 characters).
	'NMETERNAME'	Non-metered bus name (12 characters).
	'NMETEREXNAME'	Non-metered bus extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	DIM, and hence the size of CARRAY, is not large enough
IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.11.2. AFLOWCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the branch flow data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, brnchs = aflowcount(sid, owner, ties, flag)
```

Fortran command syntax:

```
CALL GET_BRANCH_FLOW_COUNT(SID, OWNER, TIES, FLAG, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem branches only.
	TIES = 2	for the subsystem bus end of tie branches only.
	TIES = 3	for the non-subsystem bus end of tie branches only.
	TIES = 4	for each end of tie branches only.
	TIES = 5	for each end of interior subsystem branches and the subsystem bus end of tie branches.
	TIES = 6	for each end of interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service branches.
	FLAG = 2	for all branches.
Integer BRNCHS	number of array entries required for the subsystem indicated by SID and OWNER that meet the editing criteria indicated by TIES and FLAG (output).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	No error, BRNCHS returned
IERR = 1	Working case is empty; BRNCHS returned as 0
IERR = 2	Invalid SID value; BRNCHS returned as 0
IERR = 3	Invalid OWNER value; BRNCHS returned as 0
IERR = 4	Invalid TIES value; BRNCHS returned as 0
IERR = 5	Invalid FLAG value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.11.3. AFLOWCPLX

Use this API to return an array of complex values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = aflowcplx(sid, owner, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_FLOW_COMPLEX(SID, OWNER, TIES, FLAG, NSTR, STRING, DIM,
 XARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem branches only.
	TIES = 2	for the subsystem bus end of tie branches only.
	TIES = 3	for the non-subsystem bus end of tie branches only.
	TIES = 4	for each end of tie branches only.
	TIES = 5	for each end of interior subsystem branches and the subsystem bus end of tie branches.
	TIES = 6	for each end of interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service branches.
	FLAG = 2	for all branches.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the branch quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PQ'	P + j Q flow at from bus end (in MW and Mvar)
	'PQLOSS'	Losses (in MW and Mvar)
	'O_PQ'	P + j Q flow at from bus end (units determined by power output options)
	'O_PQLOSS'	Losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of XARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.11.4. AFLOWINT

Use this API to return an array of integer values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = aflowint(sid, owner, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_FLOW_INTEGER(SID, OWNER, TIES, FLAG, NSTR, STRING, DIM,
 IARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem branches only.
	TIES = 2	for the subsystem bus end of tie branches only.
	TIES = 3	for the non-subsystem bus end of tie branches only.
	TIES = 4	for each end of tie branches only.
	TIES = 5	for each end of interior subsystem branches and the subsystem bus end of tie branches.
	TIES = 6	for each end of interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service branches.
	FLAG = 2	for all branches.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the branch quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'FROMNUMBER'	From bus number
	'TONUMBER'	To bus number (>1000000 for a three-winding transformer winding)
	'FROMSTATION'	From bus substation number (0 if bus is not in a substation)
	'TOSTATION'	To bus substation number (0 if bus is not in a substation)
	'FROMSECTION'	From bus section number (0 if bus is not represented by multiple bus sections)
	'TOSECTION'	To bus section number (0 if bus is not represented by multiple bus sections)
	'FROMNODE'	From bus node number (0 if bus is not in a substation)
	'TONODE'	To bus node number (0 if bus is not in a substation)
	'STATUS'	Branch status
	'NMETERNUMBER'	Non-metered end bus number
	'OWNERS'	Number of owners
	'OWN1'	First owner
	'OWN2'	Second owner
	'OWN3'	Third owner
	'OWN4'	Fourth owner
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of IARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.11.5. AFLOWREAL

Use this API to return an array of real values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = aflowreal(sid, owner, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_FLOW_REAL(SID, OWNER, TIES, FLAG, NSTR, STRING, DIM,
 RARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem branches only.
	TIES = 2	for the subsystem bus end of tie branches only.
	TIES = 3	for the non-subsystem bus end of tie branches only.
	TIES = 4	for each end of tie branches only.
	TIES = 5	for each end of interior subsystem branches and the subsystem bus end of tie branches.
	TIES = 6	for each end of interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service branches.
	FLAG = 2	for all branches.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the branch quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'AMPS'	Branch current, in amps (0.0 if bus base voltage is 0.0)
'PUCUR'	Branch current, in pu
'PCTRATE'	Percent from bus current of default rating set
'PCTRATEA'	Percent from bus current of rating set A
'PCTRATEB'	Percent from bus current of rating set B
'PCTRATEC'	Percent from bus current of rating set C
'PCTMVARATE'	Percent from bus MVA of default rating set
'PCTMVARATEA'	Percent from bus MVA of rating set A
'PCTMVARATEB'	Percent from bus MVA of rating set B
'PCTMVARATEC'	Percent from bus MVA of rating set C
'PCTCORPRATE'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'PCTCORPRATEA'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set A
'PCTCORPRATEB'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set B
'PCTCORPRATEC'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set C
'FRACT1'	First owner fraction
'FRACT2'	Second owner fraction
'FRACT3'	Third owner fraction
'FRACT4'	Fourth owner fraction
'RATE'	Rating from default rating set
'RATEA'	Rating from rating set A
'RATEB'	Rating from rating set B
'RATEC'	Rating from rating set C
'P'	Active power flow, in MW
'Q'	Reactive power flow, in Mvar
'MVA'	abs(P+jQ), in MVA
'PLOSS'	Active power losses, in MW
'QLOSS'	Reactive power losses, in Mvar
'O_P'	Active power flow (units determined by power output options)
'O_Q'	Reactive power flow (units determined by power output options)
'O_MVA'	abs(P+jQ) (units determined by power output options)
'O_PLOSS'	Active power losses (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_FLOW_COUNT API when it is called with the same values specified for SID, OWNER, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of RARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.11.6. AFLOWTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the branch flow data family (aFlowInt, aFlowReal, aFlowCplx and aFlowChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = aflowtypes(string)
```

Fortran command syntax:

```
CALL GET_BRANCH_FLOW_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the branch flow data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the branch flow data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.12. Branch Data

The branch data family of subsystem data retrieval APIs returns arrays of data for two-terminal ac branches in a specified subsystem. These APIs return data for non-transformer branches and/or two-winding transformers.

One of the input items to each API allows the user to elect either to retrieve data for all subsystem branches, for subsystem tie branches, or for both subsystem and tie branches. Another input item allows the user to elect to retrieve data for only non-transformer branches, only two-winding transformers, or both non-transformer branches and two-winding transformers, and either to retrieve data for all such branches, or to restrict the array entries to data for in-service branches.

Data for subsystem branches is returned from all APIs in the branch data family in the same order. One of the input items to each API allows the user to elect that either single entry arrays (each branch appears once) or double entry arrays (each branch appears in both directions) are to be returned. Branches are ordered in ascending numerical or alphabetical order by from bus, according to the bus output option setting in effect at the time the API is referenced. For each from bus, branches are ordered in ascending order by to bus. Parallel branches are ordered in ascending circuit identifier order.

13.12.1. ABRNCHAR

Use this API to return an array of character values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = abrnchar(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_CHAR(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 CARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).
	FLAG = 1 for only in-service non-transformer branches.
	FLAG = 2 for all non-transformer branches.
	FLAG = 3 for only in-service non-transformer branches and two-winding transformers.
	FLAG = 4 for all non-transformer branches and two-winding transformers.
	FLAG = 5 for only in-service two-winding transformers.
	FLAG = 6 for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).
	ENTRY = 1 for single entry (each branch once).
	ENTRY = 2 for double entry (each branch in both directions).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the branch quantities desired (input; no default allowed).
	'ID' Circuit identifier (2 characters).
	'FROMNAME' From bus name (12 characters).
	'FROMEXNAME' From bus extended bus name (18 characters).
	'TONAME' To bus name (three-winding transformer name for a three-winding transformer winding) (12 characters).
	'TOEXNAME' To bus extended bus name (three-winding transformer name and winding number for a three-winding transformer winding) (18 characters).
	'METERNAME' Metered bus name (12 characters).
	'METEREXNAME' Metered bus extended bus name (18 characters).
	'NMETERNAME' Non-metered bus name (12 characters).
	'NMETEREXNAME' Non-metered bus extended bus name (18 characters).
	'BRANCHNAME' Branch name (40 characters).
	'SWDRSETNAME' System switching device rating set name (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of CARRAY, is not large enough
IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.12.2. ABRNCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the branch data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, brnchs = abrncount(sid, owner, ties, flag, entry)
```

Fortran command syntax:

```
CALL GET_BRANCH_COUNT(SID, OWNER, TIES, FLAG, ENTRY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service non-transformer branches.
	FLAG = 2	for all non-transformer branches.
	FLAG = 3	for only in-service non-transformer branches and two-winding transformers.
	FLAG = 4	for all non-transformer branches and two-winding transformers.
	FLAG = 5	for only in-service two-winding transformers.
	FLAG = 6	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCHS	number of array entries required for the subsystem indicated by SID and OWNER that meet the editing criteria indicated by TIES, FLAG and ENTRY (output).
Integer IERR	error code (output).

IERR = 0	No error, BRNCHS returned
IERR = 1	Working case is empty; BRNCHS returned as 0
IERR = 2	Invalid SID value; BRNCHS returned as 0
IERR = 3	Invalid OWNER value; BRNCHS returned as 0
IERR = 4	Invalid TIES value; BRNCHS returned as 0
IERR = 5	Invalid FLAG value; BRNCHS returned as 0
IERR = 6	Invalid ENTRY value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.12.3. ABRNCPLX

Use this API to return an array of complex values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = abrncplx(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_COMPLEX(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 XARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service non-transformer branches.
	FLAG = 2	for all non-transformer branches.
	FLAG = 3	for only in-service non-transformer branches and two-winding transformers.
	FLAG = 4	for all non-transformer branches and two-winding transformers.
	FLAG = 5	for only in-service two-winding transformers.
	FLAG = 6	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the branch quantities desired (input; no default allowed).

'RX'	Branch impedance, in pu
'FROMSHNT'	Line shunt at from bus end, in pu
'TOSHNT'	Line shunt at to bus end, in pu
'RXZERO'	Zero sequence branch impedance, in pu
'FROMSHNTZERO'	Zero sequence line shunt at from bus end, in pu
'TOSHNTZERO'	Zero sequence line shunt at to bus end, in pu
'PQ'	P + j Q flow at from bus end (MW and Mvar)
'PQLOSS'	Losses (MW and Mvar)
'O_PQ'	P + j Q flow at from bus end (units determined by power output options)
'O_PQLOSS'	Losses (units determined by power output options)

Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of XARRAY, is not large enough
IERR = 9	Invalid STRING value
IERR = 10	Sequence data not in case (when STRING = 'RXZERO', 'FROMSHNTZERO' or 'TOSHNTZERO')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.12.4. ABRNINT

Use this API to return an array of integer values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = abrnint(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_INTEGER(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 IARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service non-transformer branches.
	FLAG = 2	for all non-transformer branches.
	FLAG = 3	for only in-service non-transformer branches and two-winding transformers.
	FLAG = 4	for all non-transformer branches and two-winding transformers.
	FLAG = 5	for only in-service two-winding transformers.
	FLAG = 6	for all two-winding transformers.
	FLAG = 7	for only in-service system switching devices.
	FLAG = 8	for all system switching devices.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NSTR	the number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the branch quantities desired (input; no default allowed).

'FROMNUMBER'	From bus number
'TONUMBER'	To bus number
'FROMSTATION'	From bus substation number (0 if bus is not in a substation)
'TOSTATION'	To bus substation number (0 if bus is not in a substation)
'FROMSECTION'	From bus section number (0 if bus is not represented by multiple bus sections)
'TOSECTION'	To bus section number (0 if bus is not represented by multiple bus sections)
'FROMNODE'	From bus node number (0 if bus is not in a substation)
'TONODE'	To bus node number (0 if bus is not in a substation)
'STATUS'	Branch status
'NSTATUS'	Normal system switching device status
'METERNUMBER'	Metered end bus number
'NMETERNUMBER'	Non-metered end bus number
'OWNERS'	Number of owners
'OWN1'	First owner
'OWN2'	Second owner
'OWN3'	Third owner
'OWN4'	Fourth owner
'TYPE'	System switching device type (0 if branch is not a system switching device)
'MOVTYPE'	MOV protection mode

Integer DIM	the number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	the number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY. (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of IARRAY, is not large enough
IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.12.5. ABRNREAL

Use this API to return an array of real values for subsystem branches.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = abrnreal(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_BRANCH_REAL(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 RARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for only in-service non-transformer branches.
	FLAG = 2	for all non-transformer branches.
	FLAG = 3	for only in-service non-transformer branches and two-winding transformers.
	FLAG = 4	for all non-transformer branches and two-winding transformers.
	FLAG = 5	for only in-service two-winding transformers.
	FLAG = 6	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)

NSTR of the following strings indicating the branch quantities desired (input; no default allowed).

'AMPS'	Branch current, in amps (0.0 if bus base voltage is 0.0)
'PUCUR'	Branch current, in pu
'PCTRATE'	Percent from bus current of default rating set
'PCTRATEA'	Percent from bus current of rating set A
'PCTRATEB'	Percent from bus current of rating set B
'PCTRATEC'	Percent from bus current of rating set C
'PCTRATE1'	Percent from bus current of rating set 1
'PCTRATE2'	Percent from bus current of rating set 2
'PCTRATE3'	Percent from bus current of rating set 3
'PCTRATE4'	Percent from bus current of rating set 4
'PCTRATE5'	Percent from bus current of rating set 5
'PCTRATE6'	Percent from bus current of rating set 6
'PCTRATE7'	Percent from bus current of rating set 7
'PCTRATE8'	Percent from bus current of rating set 8
'PCTRATE9'	Percent from bus current of rating set 9
'PCTRATE10'	Percent from bus current of rating set 10
'PCTRATE11'	Percent from bus current of rating set 11
'PCTRATE12'	Percent from bus current of rating set 12
'PCTMVARATE'	Percent from bus MVA of default rating set
'PCTMVARATEA'	Percent from bus MVA of rating set A
'PCTMVARATEB'	Percent from bus MVA of rating set B
'PCTMVARATEC'	Percent from bus MVA of rating set C
'PCTMVARATE1'	Percent from bus MVA of rating set 1
'PCTMVARATE2'	Percent from bus MVA of rating set 2
'PCTMVARATE3'	Percent from bus MVA of rating set 3
'PCTMVARATE4'	Percent from bus MVA of rating set 4
'PCTMVARATE5'	Percent from bus MVA of rating set 5
'PCTMVARATE6'	Percent from bus MVA of rating set 6
'PCTMVARATE7'	Percent from bus MVA of rating set 7
'PCTMVARATE8'	Percent from bus MVA of rating set 8
'PCTMVARATE9'	Percent from bus MVA of rating set 9
'PCTMVARATE10'	Percent from bus MVA of rating set 10
'PCTMVARATE11'	Percent from bus MVA of rating set 11
'PCTMVARATE12'	Percent from bus MVA of rating set 12
'PCTCORPRATE'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'PCTCORPRATEA'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set A

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'PCTCORPRATEB'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set B
'PCTCORPRATEC'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set C
'PCTCRPRATE'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'PCTCRPRATE1'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 1
'PCTCRPRATE2'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 2
'PCTCRPRATE3'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 3
'PCTCRPRATE4'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 4
'PCTCRPRATE5'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 5
'PCTCRPRATE6'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 6
'PCTCRPRATE7'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 7
'PCTCRPRATE8'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 8
'PCTCRPRATE9'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 9
'PCTCRPRATE10'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 10
'PCTCRPRATE11'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 11
'PCTCRPRATE12'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 12
'MAXPCTRATE'	Larger of percent from/to bus current of default rating set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'MAXPCTRATEA'	Larger of percent from/to bus current of rating set A
'MAXPCTRATEB'	Larger of percent from/to bus current of rating set B
'MAXPCTRATEC'	Larger of percent from/to bus current of rating set C
'MAXPCTRATE1'	Larger of percent from/to bus current of rating set 1
'MAXPCTRATE2'	Larger of percent from/to bus current of rating set 2
'MAXPCTRATE3'	Larger of percent from/to bus current of rating set 3
'MAXPCTRATE4'	Larger of percent from/to bus current of rating set 4
'MAXPCTRATE5'	Larger of percent from/to bus current of rating set 5
'MAXPCTRATE6'	Larger of percent from/to bus current of rating set 6
'MAXPCTRATE7'	Larger of percent from/to bus current of rating set 7
'MAXPCTRATE8'	Larger of percent from/to bus current of rating set 8
'MAXPCTRATE9'	Larger of percent from/to bus current of rating set 9
'MAXPCTRATE10'	Larger of percent from/to bus current of rating set 10
'MAXPCTRATE11'	Larger of percent from/to bus current of rating set 11
'MAXPCTRATE12'	Larger of percent from/to bus current of rating set 12
'MXPCTMVARAT'	Larger of percent from/to bus MVA of default rating set
'MXPCTMVARATA'	Larger of percent from/to bus MVA of rating set A
'MXPCTMVARATB'	Larger of percent from/to bus MVA of rating set B
'MXPCTMVARATC'	Larger of percent from/to bus MVA of rating set C
'MXPCTPRATE'	Larger of percent from/to bus MVA of rating set C
'MXPCTPRATE1'	Larger of percent from/to bus MVA of rating set 1
'MXPCTPRATE2'	Larger of percent from/to bus MVA of rating set 2
'MXPCTPRATE3'	Larger of percent from/to bus MVA of rating set 3

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'MXPCTPRATE4'	Larger of percent from/to bus MVA of rating set 4
'MXPCTPRATE5'	Larger of percent from/to bus MVA of rating set 5
'MXPCTPRATE6'	Larger of percent from/to bus MVA of rating set 6
'MXPCTPRATE7'	Larger of percent from/to bus MVA of rating set 7
'MXPCTPRATE8'	Larger of percent from/to bus MVA of rating set 8
'MXPCTPRATE9'	Larger of percent from/to bus MVA of rating set 9
'MXPCTPRATE10'	Larger of percent from/to bus MVA of rating set 10
'MXPCTPRATE11'	Larger of percent from/to bus MVA of rating set 11
'MXPCTPRATE12'	Larger of percent from/to bus MVA of rating set 12
'MXPCTCRPRAT'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'MXPCTCRPRATA'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set A
'MXPCTCRPRATB'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set B
'MXPCTCRPRATC'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set C
'MXPCTCRPRA'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'MXPCTCRPRA1'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 1
'MXPCTCRPRA2'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 2
'MXPCTCRPRA3'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	loading units program option setting) of rating set 3
'MXPCTCRPRA4'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 4
'MXPCTCRPRA5'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 5
'MXPCTCRPRA6'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 6
'MXPCTCRPRA7'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 7
'MXPCTCRPRA8'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 8
'MXPCTCRPRA9'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 9
'MXPCTCRPRA10'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 10
'MXPCTCRPRA11'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 11
'MXPCTCRPRA12'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 12
'FRACT1'	First owner fraction
'FRACT2'	Second owner fraction
'FRACT3'	Third owner fraction
'FRACT4'	Fourth owner fraction
'RATE'	Rating from default rating set
'RATEA'	Rating from rating set A
'RATEB'	Rating from rating set B
'RATEC'	Rating from rating set C
'RATE1'	Rating from rating set 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'RATE2'	Rating from rating set 2
	'RATE3'	Rating from rating set 3
	'RATE4'	Rating from rating set 4
	'RATE5'	Rating from rating set 5
	'RATE6'	Rating from rating set 6
	'RATE7'	Rating from rating set 7
	'RATE8'	Rating from rating set 8
	'RATE9'	Rating from rating set 9
	'RATE10'	Rating from rating set 10
	'RATE11'	Rating from rating set 11
	'RATE12'	Rating from rating set 12
	'LENGTH'	Line length
	'CHARGING'	Total charging capacitance, in pu
	'CHARGINGZERO'	Zero sequence total charging capacitance, in pu
	'MOVIRATED'	MOV Rated Current for series capacitor branch, in kA
	'P'	Active power flow at from bus end, in MW
	'Q'	Reactive power flow at from bus end, in Mvar
	'MVA'	abs(P+jQ) at from bus end, in MVA
	'MAXMVA'	abs(P+jQ) at from bus or to bus end whichever is larger, in MVA
	'PLOSS'	Active power losses, in MW
	'QLOSS'	Reactive power losses, in Mvar
	'O_P'	Active power flow at from bus end (units determined by power output options)
	'O_Q'	Reactive power flow at from bus end (units determined by power output options)
	'O_MVA'	abs(P+jQ) at from bus end (units determined by power output options)
	'O_MAXMVA'	abs(P+jQ) at from bus or to bus end whichever is larger (units determined by power output options)
	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_BRANCH_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of RARRAY, is not large enough
IERR = 9	Invalid STRING value
IERR = 10	Sequence data not in case (when STRING = 'CHARGINGZERO')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.12.6. ABRNTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the branch data family (aBrnInt, aBrnReal, aBrnCplx and aBrnChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = abrntypes(string)
```

Fortran command syntax:

```
CALL GET_BRANCH_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the branch data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the branch data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.13. Two-Winding Transformer Data

The two-winding transformer data family of subsystem data retrieval APIs returns arrays of data for two-winding transformers in a specified subsystem.

One of the input items to each API allows the user to elect either to retrieve data for all subsystem two-winding transformers, for subsystem tie two-winding transformers, or for both subsystem and tie two-winding transformers. Another input item allows the user to elect either to retrieve data for all such transformers, or to restrict the array entries to data for in-service transformers.

Data for subsystem two-winding transformers is returned from all APIs in the two-winding transformer data family in the same order. One of the input items to each API allows the user to elect that either single entry arrays (each transformer appears once) or double entry arrays (each transformer appears in both directions) are to be returned. Transformers are ordered in ascending numerical or alphabetical order by from bus, according to the bus output option setting in effect at the time the API is referenced. For each from bus, transformers are ordered in ascending order by to bus. Parallel transformers are ordered in ascending circuit identifier order.

13.13.1. ATRNCHAR

Use this API to return an array of character values for subsystem two-winding transformers.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = atrnchar(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_2WINDING_CHAR(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 CARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem two-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-winding transformers only.
	TIES = 2	for subsystem tie two-winding transformers only.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TIES = 3	for both interior subsystem and tie two-winding transformers.
Integer FLAG	a flag indicating which subsystem two-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service two-winding transformers.
	FLAG = 2	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'ID'	Circuit identifier (2 characters).
	'FROMNAME'	From bus name (12 characters).
	'FROMEXNAME'	From bus extended bus name (18 characters).
	'TONAME'	To bus name (three-winding transformer name for a three-winding transformer winding) (12 characters).
	'TOEXNAME'	To bus extended bus name (three-winding transformer name and winding number for a three-winding transformer winding) (18 characters).
	'METERNAME'	Metered bus name (12 characters).
	'METEREXNAME'	Metered bus extended bus name (18 characters).
	'NMETERNAME'	Non-metered bus name (12 characters).
	'NMETEREXNAME'	Non-metered bus extended bus name (18 characters).
	'ICONTNAME'	Controlled bus name (12 characters).
	'ICONTEXNAME'	Controlled bus extended bus name (18 characters).
	'WIND1NAME'	Winding 1 bus name (12 characters).
	'WIND1EXNAME'	Winding 1 bus extended bus name (18 characters).
	'WIND2NAME'	Winding 2 bus name (12 characters).
	'WIND2EXNAME'	Winding 2 bus extended bus name (18 characters).
	'XFRNAME'	Transformer name (40 characters).
	'VECTORGROUP'	Vector group name (12 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of CARRAY, is not large enough
IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.13.2. ATRNCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the two-winding transformer data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, brnchs = atrncount(sid, owner, ties, flag, entry)
```

Fortran command syntax:

```
CALL GET_2WINDING_COUNT(SID,OWNER,TIES,FLAG,ENTRY,BRNCHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem two-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-winding transformers only.
	TIES = 2	for subsystem tie two-winding transformers only.
	TIES = 3	for both interior subsystem and tie two-winding transformers.
Integer FLAG	a flag indicating which subsystem two-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service two-winding transformers.
	FLAG = 2	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).
Integer BRNCHS	number of array entries required for the subsystem indicated by SID and OWNER that meet the editing criteria indicated by TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BRNCHS returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	Working case is empty; BRNCHS returned as 0
IERR = 2	Invalid SID value; BRNCHS returned as 0
IERR = 3	Invalid OWNER value; BRNCHS returned as 0
IERR = 4	Invalid TIES value; BRNCHS returned as 0
IERR = 5	Invalid FLAG value; BRNCHS returned as 0
IERR = 6	Invalid ENTRY value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.13.3. ATRNCPLX

Use this API to return an array of complex values for subsystem two-winding transformers.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = atrncplx(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_2WINDING_COMPLEX(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, XARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem two-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-winding transformers only.
	TIES = 2	for subsystem tie two-winding transformers only.
	TIES = 3	for both interior subsystem and tie two-winding transformers.
Integer FLAG	a flag indicating which subsystem two-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service two-winding transformers.
	FLAG = 2	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'RXACT'	Actual transformer impedance in pu

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'RXACTCZ'	Actual transformer impedance in units governed by CZ
	'RXNOM'	Nominal transformer impedance in pu
	'RXNOMCZ'	Nominal transformer impedance in units governed by CZ
	'YMAG'	Magnetizing admittance in pu
	'YMAGCM'	Magnetizing admittance in units governed by CM
	'COMPRX'	Load drop compensation impedance in pu
	'RXZERO'	Zero sequence branch impedance in pu
	'ZG1'	ZGRND' Zero sequence winding 1 grounding impedance in pu
	'ZG1CZG'	Zero sequence winding 1 grounding impedance in units governed by CZG
	'ZG2'	'ZGRND2' Zero sequence winding 2 grounding impedance in pu
	'ZG2CZG'	Zero sequence winding 2 grounding impedance in units governed by CZG
	'Z01'	Zero sequence leakage impedance connected to the winding 1 bus in pu
	'Z01CZ0'	Zero sequence leakage impedance connected to the winding 1 bus in units governed by CZ0
	'Z02'	Zero sequence leakage impedance connected to the winding 2 bus in pu
	'Z02CZ0'	Zero sequence leakage impedance connected to the winding 2 bus in units governed by CZ0
	'ZNUTRL'	Zero sequence common neutral grounding impedance in pu
	'ZNUTRLCZG'	Zero sequence common neutral grounding impedance in units governed by CZG
	'PQ'	P + j Q flow at from bus end
	'PQLOSS'	Losses (MW and Mvar)
	'O_PQ'	P + j Q flow at from bus end (units determined by power output options)
	'O_PQLOSS'	Losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of XARRAY, is not large enough
IERR = 9	Invalid STRING value
IERR = 10	Sequence data not in case (when STRING = 'RXZERO', 'ZGRND', 'ZGRND2', 'ZG1', 'ZG1CZG', 'ZG2', 'ZG2CZG', 'Z01', 'Z01CZ0', 'Z02', 'Z02CZ0', 'ZNUTRL', or 'ZNUTRLCZG')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.13.4. ATRNINT

Use this API to return an array of integer values for subsystem two-winding transformers.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = atrnint(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_2WINDING_INTEGER(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, IARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem two-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-winding transformers only.
	TIES = 2	for subsystem tie two-winding transformers only.
	TIES = 3	for both interior subsystem and tie two-winding transformers.
Integer FLAG	a flag indicating which subsystem two-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service two-winding transformers.
	FLAG = 2	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'FROMNUMBER'	From bus number
	'TONUMBER'	To bus number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'FROMSTATION'	From bus substation number (0 if bus is not in a substation)
'TOSTATION'	To bus substation number (0 if bus is not in a substation)
'FROMSECTION'	From bus section number (0 if bus is not represented by multiple bus sections)
'TOSECTION'	To bus section number (0 if bus is not represented by multiple bus sections)
'FROMNODE'	From bus node number (0 if bus is not in a substation)
'TONODE'	To bus node number (0 if bus is not in a substation)
'STATUS'	Transformer status
'METERNUMBER'	Metered end bus number
'NMETERNUMBER'	Non-metered end bus number
'OWNERS'	Number of owners
'OWN1'	First owner
'OWN2'	Second owner
'OWN3'	Third owner
'OWN4'	Fourth owner
'ICONTNUMBER'	Controlled bus number
'NCONT'	Controlled node number
'WIND1NUMBER'	Winding 1 bus number
'WIND2NUMBER'	Winding 2 bus number
'WIND1STATION'	Winding 1 bus substation number (0 if bus is not in a substation)
'WIND2STATION'	Winding 2 bus substation number (0 if bus is not in a substation)
'WIND1SECTION'	Winding 1 bus section number (0 if bus is not represented by multiple bus sections)
'WIND2SECTION'	Winding 2 bus section number (0 if bus is not represented by multiple bus sections)
'WIND1NODE'	Winding 1 bus node number (0 if bus is not in a substation)
'WIND2NODE'	Winding 2 bus node number (0 if bus is not in a substation)
'TABLE'	Transformer impedance adjustment table number
'CODE'	Transformer control mode for automatic adjustments
'NTPOSN'	Number of tap positions
'CW'	Winding data I/O code
'CZ'	Impedance data I/O code
'CM'	Magnetizing admittance I/O code
'CZ0'	Leakage impedance data I/O code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'CZG'	Grounding impedance data I/O code
	'CNXCOD'	Connection code
	'TPSTT'	Winding 1 ratio limit code: -1 for low, 0 for regulating, 1 for high, -2 for locked, -3 for controlling a dc line
	'ANSTT'	Winding 1 angle limit code: -1 for low, 0 for regulating, 1 for high, -2 for locked
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of IARRAY, is not large enough
	IERR = 9	Invalid STRING value
	IERR = 10	Sequence data not in case (when STRING = 'CZ0', 'CZG' or 'CNXCOD')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.13.5. ATRNREAL

Use this API to return an array of real values for subsystem two-winding transformers.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = atrnreal(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_2WINDING_REAL(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 RARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem two-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-winding transformers only.
	TIES = 2	for subsystem tie two-winding transformers only.
	TIES = 3	for both interior subsystem and tie two-winding transformers.
Integer FLAG	a flag indicating which subsystem two-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service two-winding transformers.
	FLAG = 2	for all two-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for single entry (each branch once).
	ENTRY = 2	for double entry (each branch in both directions).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'AMPS'	Branch current in amps (0.0 if bus base voltage is 0.0)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'PUCUR'	Branch current in pu
'PCTRATE'	Percent from bus current of default rating set
'PCTRATEA'	Percent from bus current of rating set A
'PCTRATEB'	Percent from bus current of rating set B
'PCTRATEC'	Percent from bus current of rating set C
'PCTRATE1'	Percent from bus current of rating set 1
'PCTRATE2'	Percent from bus current of rating set 2
'PCTRATE3'	Percent from bus current of rating set 3
'PCTRATE4'	Percent from bus current of rating set 4
'PCTRATE5'	Percent from bus current of rating set 5
'PCTRATE6'	Percent from bus current of rating set 6
'PCTRATE7'	Percent from bus current of rating set 7
'PCTRATE8'	Percent from bus current of rating set 8
'PCTRATE9'	Percent from bus current of rating set 9
'PCTRATE10'	Percent from bus current of rating set 10
'PCTRATE11'	Percent from bus current of rating set 11
'PCTRATE12'	Percent from bus current of rating set 12
'PCTMVARATE'	Percent from bus MVA of default rating set
'PCTMVARATEA'	Percent from bus MVA of rating set A
'PCTMVARATEB'	Percent from bus MVA of rating set B
'PCTMVARATEC'	Percent from bus MVA of rating set C
'PCTMVARATE1'	Percent from bus MVA of rating set 1
'PCTMVARATE2'	Percent from bus MVA of rating set 2
'PCTMVARATE3'	Percent from bus MVA of rating set 3
'PCTMVARATE4'	Percent from bus MVA of rating set 4
'PCTMVARATE5'	Percent from bus MVA of rating set 5
'PCTMVARATE6'	Percent from bus MVA of rating set 6
'PCTMVARATE7'	Percent from bus MVA of rating set 7
'PCTMVARATE8'	Percent from bus MVA of rating set 8
'PCTMVARATE9'	Percent from bus MVA of rating set 9
'PCTMVARATE10'	Percent from bus MVA of rating set 10
'PCTMVARATE11'	Percent from bus MVA of rating set 11
'PCTMVARATE12'	Percent from bus MVA of rating set 12
'PCTCORPRATE'	Percent from bus current or MVA loading (according to the transformer percent loading units program option setting) of default rating set
'PCTCORPRATEA'	Percent from bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set A
'PCTCORPRATEB'	Percent from bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set B

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'PCTCORPRATEC'	Percent from bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set C
'PCTCRPRATE'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'PCTCRPRATE1'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 1
'PCTCRPRATE2'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 2
'PCTCRPRATE3'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 3
'PCTCRPRATE4'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 4
'PCTCRPRATE5'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 5
'PCTCRPRATE6'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 6
'PCTCRPRATE7'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 7
'PCTCRPRATE8'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 8
'PCTCRPRATE9'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 9
'PCTCRPRATE10'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 10
'PCTCRPRATE11'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 11
'PCTCRPRATE12'	Percent from bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 12
'MAXPCTRATE'	Larger of percent from/to bus current of default rating set
'MAXPCTRATEA'	Larger of percent from/to bus current of rating set A

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'MAXPCTRATEB'	Larger of percent from/to bus current of rating set B
'MAXPCTRATEC'	Larger of percent from/to bus current of rating set C
'MAXPCTRATE1'	Larger of percent from/to bus current of rating set 1
'MAXPCTRATE2'	Larger of percent from/to bus current of rating set 2
'MAXPCTRATE3'	Larger of percent from/to bus current of rating set 3
'MAXPCTRATE4'	Larger of percent from/to bus current of rating set 4
'MAXPCTRATE5'	Larger of percent from/to bus current of rating set 5
'MAXPCTRATE6'	Larger of percent from/to bus current of rating set 6
'MAXPCTRATE7'	Larger of percent from/to bus current of rating set 7
'MAXPCTRATE8'	Larger of percent from/to bus current of rating set 8
'MAXPCTRATE9'	Larger of percent from/to bus current of rating set 9
'MAXPCTRATE10'	Larger of percent from/to bus current of rating set 10
'MAXPCTRATE11'	Larger of percent from/to bus current of rating set 11
'MAXPCTRATE12'	Larger of percent from/to bus current of rating set 12
'MXPCTMVARAT'	Larger of percent from/to bus MVA of default rating set
'MXPCTMVARATA'	Larger of percent from/to bus MVA of rating set A
'MXPCTMVARATB'	Larger of percent from/to bus MVA of rating set B
'MXPCTMVARATC'	Larger of percent from/to bus MVA of rating set C
'MXPCTCRPRAT'	Larger of percent from/to bus current or MVA loading (according to the transformer percent loading units program option setting) of default rating set
'MXPCTCRPRATA'	Larger of percent from/to bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set A
'MXPCTCRPRATB'	Larger of percent from/to bus current or MVA loading (according to the transformer percent

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	loading units program option setting) of rating set B
'MXPCTCRPRATC'	Larger of percent from/to bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set C
'MXPCTPRATE'	Larger of percent from/to bus MVA of default rating set
'MXPCTPRATE1'	Larger of percent from/to bus MVA of rating set 1
'MXPCTPRATE2'	Larger of percent from/to bus MVA of rating set 2
'MXPCTPRATE3'	Larger of percent from/to bus MVA of rating set 3
'MXPCTPRATE4'	Larger of percent from/to bus MVA of rating set 4
'MXPCTPRATE5'	Larger of percent from/to bus MVA of rating set 5
'MXPCTPRATE6'	Larger of percent from/to bus MVA of rating set 6
'MXPCTPRATE7'	Larger of percent from/to bus MVA of rating set 7
'MXPCTPRATE8'	Larger of percent from/to bus MVA of rating set 8
'MXPCTPRATE9'	Larger of percent from/to bus MVA of rating set 9
'MXPCTPRATE10'	Larger of percent from/to bus MVA of rating set 10
'MXPCTPRATE11'	Larger of percent from/to bus MVA of rating set 11
'MXPCTPRATE12'	Larger of percent from/to bus MVA of rating set 12
'MXPCTCRPRA'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'MXPCTCRPRA1'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 1
'MXPCTCRPRA2'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 2
'MXPCTCRPRA3'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	loading units program option setting) of rating set 3
'MXPCTCRPRA4'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 4
'MXPCTCRPRA5'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 5
'MXPCTCRPRA6'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 6
'MXPCTCRPRA7'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 7
'MXPCTCRPRA8'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 8
'MXPCTCRPRA9'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 9
'MXPCTCRPRA10'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 10
'MXPCTCRPRA11'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 11
'MXPCTCRPRA12'	Larger of percent from/to bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 12
'FRACT1'	First owner fraction
'FRACT2'	Second owner fraction
'FRACT3'	Third owner fraction
'FRACT4'	Fourth owner fraction
'RATE'	Rating from default rating set
'RATEA'	Rating from rating set A
'RATEB'	Rating from rating set B
'RATEC'	Rating from rating set C
'RATE1'	Rating from rating set 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'RATE2'	Rating from rating set 2
'RATE3'	Rating from rating set 3
'RATE4'	Rating from rating set 4
'RATE5'	Rating from rating set 5
'RATE6'	Rating from rating set 6
'RATE7'	Rating from rating set 7
'RATE8'	Rating from rating set 8
'RATE9'	Rating from rating set 9
'RATE10'	Rating from rating set 10
'RATE11'	Rating from rating set 11
'RATE12'	Rating from rating set 12
'RATIO'	Winding 1 tap ratio in pu
'RATIOCW'	Winding 1 tap ratio in units governed by CW
'RATIO2'	Winding 2 tap ratio in pu
'RATIO2CW'	Winding 2 tap ratio in units governed by CW
'ANGLE'	Winding 1 phase shift angle in degrees
'RMAX'	Winding 1 ratio or angle upper limit (in pu if ratio)
'RMAXCW'	Winding 1 ratio or angle upper limit (in units governed by CW if ratio)
'RMIN'	Winding 1 ratio or angle lower limit (in pu if ratio)
'RMINCW'	Winding 1 ratio or angle lower limit (in units governed by CW if ratio)
'VMAX'	Controlled quantity upper limit (in pu if voltage)
'VMAXKV'	Controlled quantity upper limit (in units governed by CW if voltage)
'VMIN'	Controlled quantity lower limit (in pu if voltage)
'VMINKV'	Controlled quantity lower limit (in units governed by CW if voltage)
'STEP'	Winding 1 ratio or angle step width (in pu if ratio)
'STEPCW'	Winding 1 ratio or angle step width (in units governed by CW if ratio)
'NOMV1'	Winding 1 nominal voltage in kv (0.0 for bus base voltage)
'NOMV2'	Winding 2 nominal voltage in kv (0.0 for bus base voltage)
'SBASE1'	Winding 1 base MVA
'P'	Active power flow at from bus end, in MW
'Q'	Reactive power flow at from bus end, in Mvar
'MVA'	abs(P+jQ) at from bus end, in MVA

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'MAXMVA'	abs(P+jQ) at from bus or to bus end, whichever is larger, in MVA
	'PLOSS'	Active power losses, in MW
	'QLOSS'	Reactive power losses, in Mvar
	'O_P'	Active power flow at from bus end (units determined by power output options)
	'O_Q'	Reactive power flow at from bus end (units determined by power output options)
	'O_MVA'	abs(P+jQ) at from bus end (units determined by power output options)
	'O_MAXMVA'	abs(P+jQ) at from bus or to bus end whichever is larger (units determined by power output options)
	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_2WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of RARRAY, is not large enough
	IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.13.6. ATRNTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the two-winding transformer data family (aTrnInt, aTrnReal, aTrnCplx and aTrnChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = atrntypes(string)
```

Fortran command syntax:

```
CALL GET_2WINDING_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the two-winding transformer data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the two-winding transformer data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.14. Three-Winding Transformer Data

The three-winding transformer data family of subsystem data retrieval APIs returns arrays of data for three-winding transformers in a specified subsystem.

One of the input items to each API allows the user to elect either to retrieve data for all subsystem three-winding transformers, for subsystem tie three-winding transformers, or for both subsystem and tie three-winding transformers. Another input item allows the user to elect either to retrieve data for all such transformers, or to restrict the array entries to data for in-service transformers.

Data for subsystem three-winding transformers is returned from all APIs in the three-winding transformer data family in the same order. One of the input items to each API allows the user to elect that transformers be ordered either by transformer name, or in ascending numerical or alphabetical order by winding 1 bus, according to the bus output option setting in effect at the time the API is referenced. For each winding 1 bus, transformers are ordered in ascending order by winding 2 bus, then winding 3 bus, then in ascending circuit identifier order.

13.14.1. ATR3CHAR

Use this API to return an array of character values for subsystem three-winding transformers.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = atr3char(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_CHAR(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 CARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service three-winding transformers.
	FLAG = 2	for all three-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'ID'	Circuit identifier (2 characters).
	'WIND1NAME'	Winding 1 bus name (12 characters).
	'WIND1EXNAME'	Winding 1 bus extended bus name (18 characters).
	'WIND2NAME'	Winding 2 bus name (12 characters).
	'WIND2EXNAME'	Winding 2 bus extended bus name (18 characters).
	'WIND3NAME'	Winding 3 bus name (12 characters).
	'WIND3EXNAME'	Winding 3 bus extended bus name (18 characters).
	'NMETERNAME'	Non-metered bus name (12 characters).
	'NMETEREXNAME'	Non-metered bus extended bus name (18 characters).
	'XFRNAME'	Transformer name (40 characters).
	'VECTORGROUP'	Vector group name (12 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of CARRAY, is not large enough
IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.14.2. ATR3COUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the three-winding transformer data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, brnchs = atr3count(sid, owner, ties, flag, entry)
```

Fortran command syntax:

```
CALL GET_3WINDING_COUNT(SID,OWNER,TIES,FLAG,ENTRY,BRNCHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service three-winding transformers.
	FLAG = 2	for all three-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer BRNCHS	number of array entries required for the subsystem indicated by SID and OWNER that meet the editing criteria indicated by TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BRNCHS returned

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	Working case is empty; BRNCHS returned as 0
IERR = 2	Invalid SID value; BRNCHS returned as 0
IERR = 3	Invalid OWNER value; BRNCHS returned as 0
IERR = 4	Invalid TIES value; BRNCHS returned as 0
IERR = 5	Invalid FLAG value; BRNCHS returned as 0
IERR = 6	Invalid ENTRY value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.14.3. ATR3CPLX

Use this API to return an array of complex values for subsystem three-winding transformers.

This API routine was first introduced in release 30.3.2.

Python command syntax:

```
ierr, xarray = atr3cplx(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_COMPLEX(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, XARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service three-winding transformers.
	FLAG = 2	for all three-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'RX1-2ACT'	Actual measured impedance between the winding 1 and winding 2 buses in pu

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'RX1-2ACTCZ'	Actual measured impedance between the winding 1 and winding 2 buses in units governed by CZ
'RX1-2NOM'	Nominal measured impedance between the winding 1 and winding 2 buses in pu
'RX1-2NOMCZ'	Nominal measured impedance between the winding 1 and winding 2 buses in units governed by CZ
'RX2-3ACT'	Actual measured impedance between the winding 2 and winding 3 buses in pu
'RX2-3ACTCZ'	Actual measured impedance between the winding 2 and winding 3 buses in units governed by CZ
'RX2-3NOM'	Nominal measured impedance between the winding 2 and winding 3 buses in pu
'RX2-3NOMCZ'	Nominal measured impedance between the winding 2 and winding 3 buses in units governed by CZ
'RX3-1ACT'	Actual measured impedance between the winding 3 and winding 1 buses in pu
'RX3-1ACTCZ'	Actual measured impedance between the winding 3 and winding 1 buses in units governed by CZ
'RX3-1NOM'	Nominal measured impedance between the winding 3 and winding 1 buses in pu
'RX3-1NOMCZ'	Nominal measured impedance between the winding 3 and winding 1 buses in units governed by CZ
'YMAG'	Magnetizing admittance in pu
'YMAGCM'	Magnetizing admittance in units governed by CM
'ZG1'	ZGRND' Zero sequence winding 1 grounding impedance in pu
'ZG1CZG'	Zero sequence winding 1 grounding impedance in units governed by CZG
'ZG2'	'ZGRND2' Zero sequence winding 2 grounding impedance in pu
'ZG2CZG'	Zero sequence winding 2 grounding impedance in units governed by CZG
'ZG3'	'ZGRND3' Zero sequence winding 3 grounding impedance in pu
'ZG3CZG'	Zero sequence winding 3 grounding impedance in units governed by CZG
'Z01'	Zero sequence winding 1 or bus 1 to bus 2 leakage impedance in pu
'Z01CZ0'	Zero sequence winding 1 or bus 1 to bus 2 leakage impedance in units governed by CZ0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'Z02'	Zero sequence winding 2 or bus 2 to bus 3 leakage impedance in pu
	'Z02CZ0'	Zero sequence winding 2 or bus 2 to bus 3 leakage impedance in units governed by CZ0
	'Z03'	Zero sequence winding 3 or bus 3 to bus 1 leakage impedance in pu
	'Z03CZ0'	Zero sequence winding 3 or bus 3 to bus 1 leakage impedance in units governed by CZ0
	'ZNUTRL'	Zero sequence common neutral grounding impedance in pu
	'ZNUTRLCZG'	Zero sequence common neutral grounding impedance in units governed by CZG
	'PQLOSS'	Losses (in MW and Mvar)
	'O_PQLOSS'	Losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of XARRAY, is not large enough
	IERR = 9	Invalid STRING value
	IERR = 10	Sequence data not in case (when STRING = 'ZGRND', 'ZG1', 'ZG1CZG', 'ZG2', 'ZG2CZG', 'ZG3', 'ZG3CZG', 'Z01', 'Z01CZ0', 'Z02', 'Z02CZ0', 'Z03', 'Z03CZ0', 'ZNUTRL', or 'ZNUTRLCZG')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.14.4. ATR3INT

Use this API to return an array of integer values for subsystem three-winding transformers.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = atr3int(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_INTEGER(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, IARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service three-winding transformers.
	FLAG = 2	for all three-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'WIND1NUMBER'	Winding 1 bus number
	'WIND2NUMBER'	Winding 2 bus number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'WIND3NUMBER'	Winding 3 bus number
	'WIND1STATION'	Winding 1 bus substation number (0 if bus is not in a substation)
	'WIND2STATION'	Winding 2 bus substation number (0 if bus is not in a substation)
	'WIND3STATION'	Winding 3 bus substation number (0 if bus is not in a substation)
	'WIND1SECTION'	Winding 1 bus section number (0 if bus is not represented by multiple bus sections)
	'WIND2SECTION'	Winding 2 bus section number (0 if bus is not represented by multiple bus sections)
	'WIND3SECTION'	Winding 3 bus section number (0 if bus is not represented by multiple bus sections)
	'WIND1NODE'	Winding 1 bus node number (0 if bus is not in a substation)
	'WIND2NODE'	Winding 2 bus node number (0 if bus is not in a substation)
	'WIND3NODE'	Winding 3 bus node number (0 if bus is not in a substation)
	'STATUS'	Transformer status
	'NMETERNUMBER'	Non-metered end bus number
	'OWNERS'	Number of owners
	'OWN1'	First owner
	'OWN2'	Second owner
	'OWN3'	Third owner
	'OWN4'	Fourth owner
	'ZADCOD'	Impedance adjustment method code
	'CW'	Winding data I/O code
	'CZ'	Impedance data I/O code
	'CM'	Magnetizing admittance I/O code
	'CZ0'	Leakage impedance data I/O code
	'CZG'	Grounding impedance data I/O code
	'CNXCOD'	Connection code
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of IARRAY, is not large enough
IERR = 9	Invalid STRING value
IERR = 10	Sequence data not in case (when STRING = 'CZ0', 'CZG' or 'CNXCOD')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.14.5. ATR3REAL

Use this API to return an array of real values for subsystem three-winding transformers.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = atr3real(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_REAL(SID, OWNER, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 RARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service three-winding transformers.
	FLAG = 2	for all three-winding transformers.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	
	'FRACT1'	First owner fraction

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'FRACT2'	Second owner fraction
	'FRACT3'	Third owner fraction
	'FRACT4'	Fourth owner fraction
	'VMSTAR'	Actual bus voltage magnitude at the star point bus, in pu
	'ANSTAR'	Bus voltage phase angle at the star point bus, in degrees
	'PLOSS'	Active power losses, in MW
	'QLOSS'	Reactive power losses, in Mvar
	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of RARRAY, is not large enough
	IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.14.6. ATR3TYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the three-winding transformer data family (aTr3Int, aTr3Real, aTr3Cplx and aTr3Char).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = atr3types(string)
```

Fortran command syntax:

```
CALL GET_3WINDING_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the three-winding transformer data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the three-winding transformer data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.15. Three-Winding Transformer Winding Data

The three-winding transformer winding data family of subsystem data retrieval APIs returns arrays of winding data for three-winding transformers in a specified subsystem.

One of the input items to each API allows the user to elect either to retrieve data for all subsystem three-winding transformers, for subsystem tie three-winding transformers, or for both subsystem and tie three-winding transformers. Another input item allows the user to elect either to retrieve data for all such transformers, to restrict the array entries to data for all windings of in-service transformers, or to restrict the array entries to data for in-service windings of in-service transformers.

Data for subsystem three-winding transformers is returned from all APIs in the three-winding transformer winding data family in the same order. One of the input items to each API allows the user to elect that transformers be ordered either by transformer name with all windings of each transformer grouped together, or in ascending numerical or alphabetical order by winding bus, according to the bus output option setting in effect at the time the API is referenced.

13.15.1. AWNDCHAR

Use this API to return an array of character values for subsystem three-winding transformer windings.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, carray = awndchar(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_WINDINGS_CHAR(SID, OWNER, TIES, FLAG, ENTRY, NSTR,
 STRING, DIM, CARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).
	FLAG = 1 — for only in-service windings of in-service three-winding transformers.
	FLAG = 2 — for all windings of in-service three-winding transformers.
	FLAG = 3 — for all three-winding transformer windings.
Integer ENTRY	a flag indicating organization (input; 1 by default).
	ENTRY = 1 — for winding bus order.
	ENTRY = 2 — for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).
	'ID' — Circuit identifier (2 characters).
	'WNDBUSNAME' — Name of the bus connected to this winding (12 characters).
	'WNDBUSEXNAME' — Extended bus name of the bus connected to this winding (18 characters).
	'OTHER1NAME' — Name of another bus connected to this winding (12 characters).
	'OTHER1EXNAME' — Extended bus name of another bus connected to this winding (18 characters).
	'OTHER2NAME' — Name of another bus connected to this winding (12 characters).
	'OTHER2EXNAME' — Extended bus name of another bus connected to this winding (18 characters).
	'WIND1NAME' — Winding 1 bus name (12 characters).
	'WIND1EXNAME' — Winding 1 bus extended bus name (18 characters).
	'WIND2NAME' — Winding 2 bus name (12 characters).
	'WIND2EXNAME' — Winding 2 bus extended bus name (18 characters).
	'WIND3NAME' — Winding 3 bus name (12 characters).
	'WIND3EXNAME' — Winding 3 bus extended bus name (18 characters).
	'NMETERNAME' — Non-metered bus name (12 characters).
	'NMETEREXNAME' — Non-metered bus extended bus name (18 characters).
	'XFRNAME' — Transformer name (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid OWNER value
IERR = 4	Invalid TIES value
IERR = 5	Invalid FLAG value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of CARRAY, is not large enough
IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.15.2. AWNDCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the three-winding transformer winding data family.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, brnchs = awndcount(sid, owner, ties, flag, entry)
```

Fortran command syntax:

```
CALL GET_3WINDING_WINDINGS_COUNT(SID,OWNER,TIES,FLAG,ENTRY,BRNCHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service windings of in-service three-winding transformers.
	FLAG = 2	for all windings of in-service three-winding transformers.
	FLAG = 3	for all three-winding transformer windings.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer BRNCHS	number of array entries required for the subsystem indicated by SID and OWNER that meet the editing criteria indicated by TIES, FLAG and ENTRY (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	No error, BRNCHS returned
	IERR = 1	Working case is empty; BRNCHS returned as 0
	IERR = 2	Invalid SID value; BRNCHS returned as 0
	IERR = 3	Invalid OWNER value; BRNCHS returned as 0
	IERR = 4	Invalid TIES value; BRNCHS returned as 0
	IERR = 5	Invalid FLAG value; BRNCHS returned as 0
	IERR = 6	Invalid ENTRY value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.15.3. AWNDCPLX

Use this API to return an array of complex values for subsystem three-winding transformer windings.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, xarray = awndcplx(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_WINDINGS_COMPLEX(SID, OWNER, TIES, FLAG, ENTRY, NSTR,
 STRING, DIM, XARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service windings of in-service three-winding transformers.
	FLAG = 2	for all windings of in-service three-winding transformers.
	FLAG = 3	for all three-winding transformer windings.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding bus order.
	ENTRY = 2	for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'RXACT'	Actual transformer impedance, in pu
	'RXNOM'	Nominal transformer impedance, in pu
	'COMPRX'	Load drop compensation impedance, in pu
	'RXZERO'	Zero sequence winding leakage impedance, in pu
	'ZGRND'	Zero sequence winding grounding impedance, in pu
	'PQ'	P + j Q flow at winding bus end, in MW and Mvar
	'PQLOSS'	Losses, in MW and Mvar
	'O_PQ'	P + j Q flow at winding bus end (units determined by power output options)
	'O_PQLOSS'	Losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of XARRAY, is not large enough
	IERR = 9	Invalid STRING value
	IERR = 10	Sequence data not in case (when STRING = 'RXZERO' or 'ZGRND')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.15.4. AWNDINT

Use this API to return an array of integer values for subsystem three-winding transformer windings.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, iarray = awndint(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_WINDINGS_INTEGER(SID, OWNER, TIES, FLAG, ENTRY, NSTR,
 STRING, DIM, IARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service windings of in-service three-winding transformers.
	FLAG = 2	for all windings of in-service three-winding transformers.
	FLAG = 3	for all three-winding transformer windings.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'WNDBUSNUMBER'	Number of the bus connected to this winding
'OTHER1NUMBER'	Number of another bus connected to this transformer
'OTHER2NUMBER'	Number of another bus connected to this transformer
'WNDNUM'	Winding number
'WIND1NUMBER'	Winding 1 bus number
'WIND2NUMBER'	Winding 2 bus number
'WIND3NUMBER'	Winding 3 bus number
'WIND1STATION'	Winding 1 bus substation number (0 if bus is not in a substation)
'WIND2STATION'	Winding 2 bus substation number (0 if bus is not in a substation)
'WIND3STATION'	Winding 3 bus substation number (0 if bus is not in a substation)
'WIND1SECTION'	Winding 1 bus section number (0 if bus is not represented by multiple bus sections)
'WIND2SECTION'	Winding 2 bus section number (0 if bus is not represented by multiple bus sections)
'WIND3SECTION'	Winding 3 bus section number (0 if bus is not represented by multiple bus sections)
'WIND1NODE'	Winding 1 bus node number (0 if bus is not in a substation)
'WIND2NODE'	Winding 2 bus node number (0 if bus is not in a substation)
'WIND3NODE'	Winding 3 bus node number (0 if bus is not in a substation)
'NMETERNUMBER'	Non-metered end bus number
'STATUS'	Winding status
'OWNERS'	Number of owners
'OWN1'	First owner
'OWN2'	Second owner
'OWN3'	Third owner
'OWN4'	Fourth owner
'ICONTNUMBER'	Controlled bus number
'SIDCOD'	Controlled bus side code (negative if on this winding side)
'NCONT'	Controlled node number
'TABLE'	Transformer impedance adjustment table number
'CODE'	Transformer control mode for automatic adjustments
'NTPOSN'	Number of tap positions
'CW'	Winding data I/O code
'CZ'	Impedance data I/O code

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'CM'	Magnetizing admittance I/O code
	'CZ0'	Leakage impedance data I/O code
	'CZG'	Grounding impedance data I/O code
	'CNXCOD'	Connection code
	'TPSTT'	Winding 1 ratio limit code: -1 for low, 0 for regulating, 1 for high, -2 for locked
	'ANSTT'	Winding 1 angle limit code: -1 for low, 0 for regulating, 1 for high, -2 for locked
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of IARRAY, is not large enough
	IERR = 9	Invalid STRING value
	IERR = 10	Sequence data not in case (when STRING = 'CZ0', 'CZG' or 'CNXCOD')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.15.5. AWNDREAL

Use this API to return an array of real values for subsystem three-winding transformer windings.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
ierr, rarray = awndreal(sid, owner, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_3WINDING_WINDINGS_REAL(SID, OWNER, TIES, FLAG, ENTRY, NSTR,
 STRING, DIM, RARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use transformer ownership.
Integer TIES	a flag indicating which subsystem three-winding transformers to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem three-winding transformers only.
	TIES = 2	for subsystem tie three-winding transformers only.
	TIES = 3	for both interior subsystem and tie three-winding transformers.
Integer FLAG	a flag indicating which subsystem three-winding transformers to include (input; 1 by default).	
	FLAG = 1	for only in-service windings of in-service three-winding transformers.
	FLAG = 2	for all windings of in-service three-winding transformers.
	FLAG = 3	for all three-winding transformer windings.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for winding 1 bus order.
	ENTRY = 2	for transformer name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the transformer quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'AMPS'	Branch current in amps (0.0 if bus base voltage is 0.0)
'PUCUR'	Branch current in pu
'PCTRATE'	Percent winding bus current of default rating set
'PCTRATEA'	Percent winding bus current of rating set A
'PCTRATEB'	Percent winding bus current of rating set B
'PCTRATEC'	Percent winding bus current of rating set C
'PCTRATE1'	Percent winding bus current of rating set 1
'PCTRATE2'	Percent winding bus current of rating set 2
'PCTRATE3'	Percent winding bus current of rating set 3
'PCTRATE4'	Percent winding bus current of rating set 4
'PCTRATE5'	Percent winding bus current of rating set 5
'PCTRATE6'	Percent winding bus current of rating set 6
'PCTRATE7'	Percent winding bus current of rating set 7
'PCTRATE8'	Percent winding bus current of rating set 8
'PCTRATE9'	Percent winding bus current of rating set 9
'PCTRATE10'	Percent winding bus current of rating set 10
'PCTRATE11'	Percent winding bus current of rating set 11
'PCTRATE12'	Percent winding bus current of rating set 12
'PCTMVARATE'	Percent winding bus MVA of default rating set
'PCTMVARATEA'	Percent winding bus MVA of rating set A
'PCTMVARATEB'	Percent winding bus MVA of rating set B
'PCTMVARATEC'	Percent winding bus MVA of rating set C
'PCTMVARATE1'	Percent winding bus MVA of rating set 1
'PCTMVARATE2'	Percent winding bus MVA of rating set 2
'PCTMVARATE3'	Percent winding bus MVA of rating set 3
'PCTMVARATE4'	Percent winding bus MVA of rating set 4
'PCTMVARATE5'	Percent winding bus MVA of rating set 5
'PCTMVARATE6'	Percent winding bus MVA of rating set 6
'PCTMVARATE7'	Percent winding bus MVA of rating set 7
'PCTMVARATE8'	Percent winding bus MVA of rating set 8
'PCTMVARATE9'	Percent winding bus MVA of rating set 9
'PCTMVARATE10'	Percent winding bus MVA of rating set 10
'PCTMVARATE11'	Percent winding bus MVA of rating set 11
'PCTMVARATE12'	Percent winding bus MVA of rating set 12
'PCTCRPRATE'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'PCTCRPRATE1'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'PCTCRPRATE2'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 2
'PCTCRPRATE3'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 3
'PCTCRPRATE4'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 4
'PCTCRPRATE5'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 5
'PCTCRPRATE6'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 6
'PCTCRPRATE7'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 7
'PCTCRPRATE8'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 8
'PCTCRPRATE9'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 9
'PCTCRPRATE10'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 10
'PCTCRPRATE11'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 11
'PCTCRPRATE12'	Percent winding bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 12
'PCTCORPRATE'	Percent winding bus current or MVA loading (according to the transformer percent loading units program option setting) of default rating set
'PCTCORPRATEA'	Percent winding bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set A
'PCTCORPRATEB'	Percent winding bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set B
'PCTCORPRATEC'	Percent winding bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set C
'MAXPCTRATE'	Larger of percent winding/star point bus current of default rating set

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'MAXPCTRATEA'	Larger of percent winding/star point bus current of rating set A
'MAXPCTRATEB'	Larger of percent winding/star point bus current of rating set B
'MAXPCTRATEC'	Larger of percent winding/star point bus current of rating set C
'MAXPCTRATE1'	Larger of percent winding/star point bus current of rating set 1
'MAXPCTRATE2'	Larger of percent winding/star point bus current of rating set 2
'MAXPCTRATE3'	Larger of percent winding/star point bus current of rating set 3
'MAXPCTRATE4'	Larger of percent winding/star point bus current of rating set 4
'MAXPCTRATE5'	Larger of percent winding/star point bus current of rating set 5
'MAXPCTRATE6'	Larger of percent winding/star point bus current of rating set 6
'MAXPCTRATE7'	Larger of percent winding/star point bus current of rating set 7
'MAXPCTRATE8'	Larger of percent winding/star point bus current of rating set 8
'MAXPCTRATE9'	Larger of percent winding/star point bus current of rating set 9
'MAXPCTRATE10'	Larger of percent winding/star point bus current of rating set 10
'MAXPCTRATE11'	Larger of percent winding/star point bus current of rating set 11
'MAXPCTRATE12'	Larger of percent winding/star point bus current of rating set 12
'MXPCTMVARAT'	Larger of percent winding/star point bus MVA of default rating set
'MXPCTMVARATA'	Larger of percent winding/star point bus MVA of rating set A
'MXPCTMVARATB'	Larger of percent winding/star point bus MVA of rating set B
'MXPCTMVARATC'	Larger of percent winding/star point bus MVA of rating set C
'MXPCTPRATE'	Larger of percent winding/star point bus MVA of default rating set
'MXPCTPRATE1'	Larger of percent winding/star point bus MVA of rating set 1
'MXPCTPRATE2'	Larger of percent winding/star point bus MVA of rating set 2
'MXPCTPRATE3'	Larger of percent winding/star point bus MVA of rating set 3

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'MXPCTPRATE4'	Larger of percent winding/star point bus MVA of rating set 4
'MXPCTPRATE5'	Larger of percent winding/star point bus MVA of rating set 5
'MXPCTPRATE6'	Larger of percent winding/star point bus MVA of rating set 6
'MXPCTPRATE7'	Larger of percent winding/star point bus MVA of rating set 7
'MXPCTPRATE8'	Larger of percent winding/star point bus MVA of rating set 8
'MXPCTPRATE9'	Larger of percent winding/star point bus MVA of rating set 9
'MXPCTPRATE10'	Larger of percent winding/star point bus MVA of rating set 10
'MXPCTPRATE11'	Larger of percent winding/star point bus MVA of rating set 11
'MXPCTPRATE12'	Larger of percent winding/star point bus MVA of rating set 12
'MXPCTCRPRAT'	Larger of percent winding/star point bus current or MVA loading (according to the transformer percent loading units program option setting) of default rating set
'MXPCTCRPRATA'	Larger of percent winding/star point bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set A
'MXPCTCRPRATB'	Larger of percent winding/star point bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set B
'MXPCTCRPRATC'	Larger of percent winding/star point bus current or MVA loading (according to the transformer percent loading units program option setting) of rating set C
'MXPCTCRPRA'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of default rating set
'MXPCTCRPRA1'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 1
'MXPCTCRPRA2'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 2
'MXPCTCRPRA3'	Larger of percent winding/star point bus current or MVA loading (according to the appro-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	priate percent loading units program option setting) of rating set 3
'MXPCTCRPRA4'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 4
'MXPCTCRPRA5'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 5
'MXPCTCRPRA6'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 6
'MXPCTCRPRA7'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 7
'MXPCTCRPRA8'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 8
'MXPCTCRPRA9'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 9
'MXPCTCRPRA10'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 10
'MXPCTCRPRA11'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 11
'MXPCTCRPRA12'	Larger of percent winding/star point bus current or MVA loading (according to the appropriate percent loading units program option setting) of rating set 12
'FRACT1'	First owner fraction
'FRACT2'	Second owner fraction
'FRACT3'	Third owner fraction
'FRACT4'	Fourth owner fraction
'RATE'	Rating from default rating set
'RATEA'	Rating from rating set A
'RATEB'	Rating from rating set B
'RATEC'	Rating from rating set C
'RATE1'	Rating from rating set 1

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'RATE2'	Rating from rating set 2
'RATE3'	Rating from rating set 3
'RATE4'	Rating from rating set 4
'RATE5'	Rating from rating set 5
'RATE6'	Rating from rating set 6
'RATE7'	Rating from rating set 7
'RATE8'	Rating from rating set 8
'RATE9'	Rating from rating set 9
'RATE10'	Rating from rating set 10
'RATE11'	Rating from rating set 11
'RATE12'	Rating from rating set 12
'RATIO'	Winding tap ratio in pu
'RATIOCW'	Winding tap ratio in units governed by CW
'ANGLE'	Winding phase shift angle in degrees
'RMAX'	Winding ratio or angle upper limit (in pu if ratio)
'RMAXCW'	Winding ratio or angle upper limit (in units governed by CW if ratio)
'RMIN'	Winding ratio or angle lower limit (in pu if ratio)
'RMINCW'	Winding ratio or angle lower limit (in units governed by CW if ratio)
'VMAX'	Controlled quantity upper limit (in pu if voltage)
'VMAXKV'	Controlled quantity upper limit (in units governed by CW if voltage)
'VMIN'	Controlled quantity lower limit (in pu if voltage)
'VMINKV'	Controlled quantity lower limit (in units governed by CW if voltage)
'STEP'	Winding ratio or angle step width (in pu if ratio)
'STEPCW'	Winding ratio or angle step width (in units governed by CW if ratio)
'NOMV'	Winding nominal voltage in kv (0.0 for bus base voltage)
'SBASE'	Winding base MVA
'P'	Active power flow at winding bus end, in MW
'Q'	Reactive power flow at winding bus end, in Mvar
'MVA'	abs(P+jQ) at winding bus end, in MVA
'MAXMVA'	abs(P+jQ) at winding bus or star point bus end whichever is larger, in MVA
'PLOSS'	Active power losses, in MW
'QLOSS'	Reactive power losses, in Mvar

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_P'	Active power flow at winding bus end (units determined by power output options)
	'O_Q'	Reactive power flow at winding bus end (units determined by power output options)
	'O_MVA'	abs(P+jQ) at winding bus end (units determined by power output options)
	'O_MAXMVA'	abs(P+jQ) at winding bus or star point bus end whichever is larger (units determined by power output options)
	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_3WINDING_WINDINGS_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and ENTRY (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of RARRAY, is not large enough
	IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.15.6. AWNDTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the three-winding transformer winding data family (aWndInt, aWndReal, aWndCplx and aWndChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = awndtypes(string)
```

Fortran command syntax:

```
CALL GET_3WINDING_WINDINGS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the three-winding transformer winding data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the three-winding transformer winding data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.16. Two-Terminal dc Line Data

The two-terminal dc line data family of subsystem data retrieval APIs returns arrays of data for two-terminal dc lines in a specified subsystem.

One of the input items to each API allows the user to elect to retrieve data either for all subsystem two-terminal dc lines, for subsystem two-terminal dc lines that are tie branches, or for both subsystem and tie two-terminal dc lines. Another input item allows the user to elect to retrieve data for all such two-terminal dc lines, or to restrict the array entries to data for in-service two-terminal dc lines.

Data for subsystem two-terminal dc lines is returned from all APIs in the two-terminal dc line data family in dc line name alphabetical order.

13.16.1. A2TRMDCCHAR

Use this API to return an array of character values for subsystem two-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = a2trmdcchar(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_LINE_CHAR(SID, TIES, FLAG, NSTR, STRING, DIM, CARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-terminal dc lines only.
	TIES = 2	for subsystem tie two-terminal dc lines only.
	TIES = 3	for both interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line quantities desired (input; no default allowed).	
	'DCNAME'	dc line name (12 characters).
	'FROMNAME'	Rectifier bus name (12 characters).
	'FROMEXNAME'	Rectifier bus extended bus name (18 characters).
	'TONAME'	Inverter bus name (12 characters).
	'TOEXNAME'	Inverter bus extended bus name (18 characters).
	'METERNAME'	Metered bus name (12 characters).
	'METEREXNAME'	Metered bus extended bus name (18 characters).
	'NMETERNAME'	Non-metered bus name (12 characters).
	'NMETEREXNAME'	Non-metered bus extended bus name (18 characters).
	'IFRNAME'	Rectifier firing angle measuring bus name (12 characters).
	'IFREXNAME'	Rectifier firing angle measuring bus extended bus name (18 characters).
	'IFINAME'	Inverter firing angle measuring bus name (12 characters).
	'IFIEXNAME'	Inverter firing angle measuring bus extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid NSTR value
	IERR = 6	DIM, and hence the size of CARRAY, is not large enough
	IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.16.2. A2TRMDCCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the two-terminal dc line data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, brnchs = a2trmdccount(sid, ties, flag)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_LINE_COUNT(SID, TIES, FLAG, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-terminal dc lines only.
	TIES = 2	for subsystem tie two-terminal dc lines only.
	TIES = 3	for both interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer BRNCHS	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BRNCHS returned
	IERR = 1	Working case is empty; BRNCHS returned as 0
	IERR = 2	Invalid SID value; BRNCHS returned as 0
	IERR = 3	Invalid TIES value; BRNCHS returned as 0
	IERR = 4	Invalid FLAG value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.16.3. A2TRMDCCPLX

Use this API to return an array of complex values for subsystem two-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = a2trmdccplx(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_LINE_COMPLEX(SID, TIES, FLAG, NSTR, STRING, DIM,
 XARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-terminal dc lines only.
	TIES = 2	for subsystem tie two-terminal dc lines only.
	TIES = 3	for both interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line quantities desired (input; no default allowed).	
	'PQAC_R'	Apparent ac power flow into the dc line at the rectifier (in MW and Mvar)
	'PQAC_I'	Apparent ac power flow into the dc line at the inverter (in MW and Mvar)
	'PQLOSS'	ac power losses (in MW and Mvar)
	'O_PQAC_R'	Apparent ac power flow into the dc line at the rectifier (units determined by power output options)
	'O_PQAC_I'	Apparent ac power flow into the dc line at the inverter (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_PQLOSS'	ac power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid NSTR value
	IERR = 6	DIM, and hence the size of XARRAY, is not large enough
	IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.16.4. A2TRMDCINT

Use this API to return an array of integer values for subsystem two-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = a2trmdcint(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_LINE_INTEGER(SID, TIES, FLAG, NSTR, STRING, DIM,
 IARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-terminal dc lines only.
	TIES = 2	for subsystem tie two-terminal dc lines only.
	TIES = 3	for both interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line quantities desired (input; no default allowed).	
	'FROMNUMBER'	Rectifier bus number
	'TONUMBER'	Inverter bus number
	'METERNUMBER'	Metered end bus number
	'NMETERNUMBER'	Non-metered end bus number
	'IFRNUMBER'	Rectifier firing angle measuring bus number
	'IFINUMBER'	Inverter extinction angle measuring bus number
	'NDR'	Rectifier firing angle measuring node number
	'NDI'	Inverter extinction angle measuring node number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'MDC'	Control mode
	'NBR'	Rectifier number of bridges in series
	'NBI'	Inverter number of bridges in series
	'ANSTT_R'	Rectifier angle limit code: -1 (at or below low limit), 0 (in range), or 1 (at or above high limit)
	'ANSTT_I'	Inverter angle limit code: -1 (at or below low limit), 0 (in range), or 1 (at or above high limit)
	'TPSTT_R'	Rectifier tap ratio limit code: -1 (at or below low limit), 0 (in range), 1 (at or above high limit) or -2 (locked)
	'TPSTT_I'	Inverter tap ratio limit code: -1 (at or below low limit), 0 (in range), 1 (at or above high limit) or -2 (locked)
	'CCCITMX'	Iteration limit for capacitor commutated converter solution
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid NSTR value
	IERR = 6	DIM, and hence the size of IARRAY, is not large enough
	IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.16.5. A2TRMDCREAL

Use this API to return an array of real values for subsystem two-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = a2trmdcreal(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_LINE_REAL(SID, TIES, FLAG, NSTR, STRING, DIM, RARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem two-terminal dc lines only.
	TIES = 2	for subsystem tie two-terminal dc lines only.
	TIES = 3	for both interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line quantities desired (input; no default allowed).	
	'DCCUR'	dc line current in amps
	'RDC'	dc line resistance in ohms
	'SETVAL'	Desired current (in amps) or power (in MW) demand
	'VSCHD'	Scheduled compounded dc voltage in kV
	'VCMOD'	Mode switch dc voltage in kV
	'RCOMP'	Compounding resistance in ohms
	'DELTI'	Margin in pu of desired dc power or current
	'DCVMIN'	Minimum compounded dc voltage in kV
	'ACAMPS_R'	ac current in amps at the rectifier
	'ACAMPS_I'	ac current in amps at the inverter

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'PUCUR_R'	ac current in pu at the rectifier
'PUCUR_I'	ac current in pu at the inverter
'KVDC_R'	Rectifier dc voltage in kV
'KVDC_I'	Inverter dc voltage in kV
'ANGLE_R'	Rectifier converter angle in degrees
'ANGLE_I'	Inverter converter angle in degrees
'ANGMIN_R'	Rectifier minimum steady-state converter angle in degrees
'ANGMIN_I'	Inverter minimum steady-state converter angle in degrees
'ANGMAX_R'	Rectifier nominal maximum converter angle in degrees
'ANGMAX_I'	Inverter nominal maximum converter angle in degrees
'RC_R'	Rectifier commutating transformer resistance per bridge in ohms
'RC_I'	Inverter commutating transformer resistance per bridge in ohms
'XC_R'	Rectifier commutating transformer reactance per bridge in ohms
'XC_I'	Inverter commutating transformer reactance per bridge in ohms
'EBASE_R'	Rectifier primary base ac voltage in kV
'EBASE_I'	Inverter primary base ac voltage in kV
'TR_R'	Rectifier transformer ratio in pu
'TR_I'	Inverter transformer ratio in pu
'TAP_R'	Rectifier tap setting in pu
'TAP_I'	Inverter tap setting in pu
'TPMX_R'	Rectifier maximum tap setting in pu
'TPMX_I'	Inverter maximum tap setting in pu
'TPMN_R'	Rectifier minimum tap setting in pu
'TPMN_I'	Inverter minimum tap setting in pu
'STP_R'	Rectifier tap step in pu
'STP_I'	Inverter tap step in pu
'XCAP_R'	Rectifier commutating capacitor reactance magnitude per bridge in pu
'XCAP_I'	Inverter commutating capacitor reactance magnitude per bridge in pu
'CCCACC'	Acceleration factor for capacitor commutated converter solution
'PAC_R'	Apparent active power flow into the dc line at the rectifier in MW
'PAC_I'	Apparent active power flow into the dc line at the inverter in MW

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'QAC_R'	Apparent reactive power flow into the dc line at the rectifier in Mvar
	'QAC_I'	Apparent reactive power flow into the dc line at the inverter in Mvar
	'MVA_R'	Apparent ac power flow into the dc line at the rectifierin MVA
	'MVA_I'	Apparent ac power flow into the dc line at the inverter in MVA
	'PLOSS'	Active power losses in MW
	'QLOSS'	Reactive power losses in Mvar
	'O_PAC_R'	Apparent active power flow into the dc line at the rectifier (units determined by power output options)
	'O_PAC_I'	Apparent active power flow into the dc line at the inverter (units determined by power output options)
	'O_QAC_R'	Apparent reactive power flow into the dc line at the rectifier (units determined by power output options)
	'O_QAC_I'	Apparent reactive power flow into the dc line at the inverter (units determined by power output options)
	'O_MVA_R'	Apparent ac power flow into the dc line at the rectifier (units determined by power output options)
	'O_MVA_I'	Apparent ac power flow into the dc line at the inverter (units determined by power output options)
	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_TWO_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid NSTR value
IERR = 6	DIM, and hence the size of RARRAY, is not large enough
IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.16.6. A2TRMDCTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the two-terminal dc line data family (a2TrmDcInt, a2TrmDcReal, a2TrmDcCplx and a2TrmDcChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = a2trmdctypes(string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_LINE_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the two-terminal dc line data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the two-terminal dc line data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.17. Two-Terminal dc Line Converter Data

The two-terminal dc line converter data family of subsystem data retrieval APIs returns arrays of data for two-terminal dc line converters in a specified subsystem.

One of the input items to each API allows the user to elect to retrieve data either for all subsystem two-terminal dc line converters, for subsystem two-terminal dc line converters of two-terminal dc lines that are tie branches, or for both subsystem and tie two-terminal dc line converters. Another input item allows the user to elect to retrieve data for all such two-terminal dc line converters, or to restrict the array entries to data for in-service two-terminal dc lines.

Data for subsystem converter buses is returned from all APIs in the two-terminal dc line converter data family in the same order. One of the input items to each API allows the user to elect that converter buses be ordered either in dc line name order with all converter buses of each two-terminal dc line grouped together, or in numerical or alphabetical order by converter bus, according to the bus output option setting in effect at the time the API is referenced.

13.17.1. A2TRMDCCONVCHAR

Use this API to return an array of character values for subsystem two-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = a2trmdcconvchar(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_CONV_CHAR(SID, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 CARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem two-terminal dc lines only.
	TIES = 2	for the subsystem bus end of tie two-terminal dc lines only.
	TIES = 3	for the non-subsystem bus end of tie two-terminal dc lines only.
	TIES = 4	for each end of tie two-terminal dc lines only.
	TIES = 5	for each end of interior subsystem two-terminal dc lines and the subsystem bus end of tie two-terminal dc lines.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TIES = 6	for each end of interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line converter quantities desired (input; no default allowed).	
	'DCNAME'	dc line name (12 characters).
	'FROMNAME'	From bus name (12 characters).
	'FROMEXNAME'	From bus extended bus name (18 characters).
	'TONAME'	To bus name (12 characters).
	'TOEXNAME'	To bus extended bus name (18 characters).
	'METERNAME'	Metered bus name (12 characters).
	'METEREXNAME'	Metered bus extended bus name (18 characters).
	'NMETERNAME'	Non-metered bus name (12 characters).
	'NMETEREXNAME'	Non-metered bus extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of CARRAY, is not large enough

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.17.2. A2TRMDCCONVCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the two-terminal dc line converter data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, buses = a2trmdcconvcount(sid, ties, flag)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_CONV_COUNT(SID, TIES, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem two-terminal dc lines only.
	TIES = 2	for the subsystem bus end of tie two-terminal dc lines only.
	TIES = 3	for the non-subsystem bus end of tie two-terminal dc lines only.
	TIES = 4	for each end of tie two-terminal dc lines only.
	TIES = 5	for each end of interior subsystem two-terminal dc lines and the subsystem bus end of tie two-terminal dc lines.
	TIES = 6	for each end of interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer BUSES	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid TIES value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.17.3. A2TRMDCCONVCPLX

Use this API to return an array of complex values for subsystem two-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = a2trmdcconvcplx(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_CONV_COMPLEX(SID, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, XARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem two-terminal dc lines only.
	TIES = 2	for the subsystem bus end of tie two-terminal dc lines only.
	TIES = 3	for the non-subsystem bus end of tie two-terminal dc lines only.
	TIES = 4	for each end of tie two-terminal dc lines only.
	TIES = 5	for each end of interior subsystem two-terminal dc lines and the subsystem bus end of tie two-terminal dc lines.
	TIES = 6	for each end of interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line converter quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PQAC'	Apparent ac power flow into the dc line at the from bus (in MW and Mvar)
	'PQLOSS'	ac power losses (in MW and Mvar)
	'O_PQAC'	Apparent ac power flow into the dc line at the from bus (units determined by power output options)
	'O_PQLOSS'	ac power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of XARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.17.4. A2TRMDCCONVINT

Use this API to return an array of integer values for subsystem two-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = a2trmdcconvint(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_CONV_INTEGER(SID, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, IARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem two-terminal dc lines only.
	TIES = 2	for the subsystem bus end of tie two-terminal dc lines only.
	TIES = 3	for the non-subsystem bus end of tie two-terminal dc lines only.
	TIES = 4	for each end of tie two-terminal dc lines only.
	TIES = 5	for each end of interior subsystem two-terminal dc lines and the subsystem bus end of tie two-terminal dc lines.
	TIES = 6	for each end of interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line converter quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'FROMNUMBER'	From bus number
	'TONUMBER'	To bus number
	'METERNUMBER'	Metered end bus number
	'NMETERNUMBER'	Non-metered end bus number
	'CONVERTERTYP'	1 if from bus is the rectifier 2 if the inverter
	'MDC'	Control mode
	'ANSTT'	From bus converter angle limit code: -1 (at or below low limit), 0 (in range), or 1 (at or above high limit)
	'TPSTT'	From bus converter tap ratio limit code: -1 (at or below low limit), 0 (in range), 1 (at or above high limit) or -2 (locked)
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of IARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.17.5. A2TRMDCCONVREAL

Use this API to return an array of real values for subsystem two-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = a2trmdcconvreal(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_CONV_REAL(SID, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 RARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of two-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem two-terminal dc lines only.
	TIES = 2	for the subsystem bus end of tie two-terminal dc lines only.
	TIES = 3	for the non-subsystem bus end of tie two-terminal dc lines only.
	TIES = 4	for each end of tie two-terminal dc lines only.
	TIES = 5	for each end of interior subsystem two-terminal dc lines and the subsystem bus end of tie two-terminal dc lines.
	TIES = 6	for each end of interior subsystem two-terminal dc lines and tie two-terminal dc lines.
Integer FLAG	a flag indicating which subsystem two-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service two-terminal dc lines.
	FLAG = 2	for all two-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the two-terminal dc line converter quantities desired (input; no default allowed).	
	'DCCUR'	dc line current in amps

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'ACAMPS'	ac current in amps at the from bus
	'PUCUR'	ac current in pu at the from bus
	'KVDC'	From bus dc voltage in kV
	'ANGLE'	From bus converter angle in degrees
	'TAP'	From bus tap setting in pu
	'PAC'	Apparent active power flow into the dc line at the from bus
	'QAC'	Apparent reactive power flow into the dc line at the from bus
	'MVA'	Apparent ac power flow into the dc line at the from bus
	'PLOSS'	Active power losses
	'QLOSS'	Reactive power losses
	'O_PAC'	Apparent active power flow into the dc line at the from bus (units determined by power output options)
	'O_QAC'	Apparent reactive power flow into the dc line at the from bus (units determined by power output options)
	'O_MVA'	Apparent ac power flow into the dc line at the from bus (units determined by power output options)
	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_TWO_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	DIM, and hence the size of RARRAY, is not large enough
IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.17.6. A2TRMDCCONVTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the two-terminal dc line converter data family (a2TrmDcConvInt, a2TrmDcConvReal, a2TrmDcConvCplx and a2TrmDcConvChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = a2trmdcconvtypes(string)
```

Fortran command syntax:

```
CALL GET_TWO_TERM_DC_CONV_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the two-terminal dc line converter data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the two-terminal dc line converter data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.18. Multi-Terminal dc Line Data

The multi-terminal dc line data family of subsystem data retrieval APIs returns arrays of data for multi-terminal dc lines in a specified subsystem.

One of the input items to each API allows the user to elect to retrieve data for either all subsystem multi-terminal dc lines, for subsystem multi-terminal dc lines that are tie branches, or for both subsystem and tie multi-terminal dc lines. Another input item allows the user to elect to retrieve data for all such multi-terminal dc lines, or to restrict the array entries to data for in-service multi-terminal dc lines.

Data for subsystem multi-terminal dc lines is returned from all APIs in the multi-terminal dc line data family in dc line name alphabetical order.

13.18.1. AMULTITRMDCCHAR

Use this API to return an array of character values for subsystem multi-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = amultitrmdcchar(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_LINE_CHAR(SID, TIES, FLAG, NSTR, STRING, DIM,
 CARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem multi-terminal dc lines only.
	TIES = 2	for subsystem tie multi-terminal dc lines only.
	TIES = 3	for both interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line quantities desired (input; no default allowed).
	'DCNAME' dc line name (12 characters).
	'VCNPOSNAME' Positive pole voltage controlling converter bus name (12 characters).
	'VCNPOSEXNAME' Positive pole voltage controlling converter bus extended bus name (18 characters).
	'VCNNEGNAME' Negative pole voltage controlling converter bus name (12 characters).
	'VCNNEGEXNAME' Negative pole voltage controlling converter bus extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Working case is empty
	IERR = 2 Invalid SID value
	IERR = 3 Invalid TIES value
	IERR = 4 Invalid FLAG value
	IERR = 5 Invalid NSTR value
	IERR = 6 DIM, and hence the size of CARRAY, is not large enough
	IERR = 7 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.18.2. AMULTITRMDCCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the multi-terminal dc line data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, brnchs = amultitrmdccount(sid, ties, flag)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_LINE_COUNT(SID, TIES, FLAG, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem multi-terminal dc lines only.
	TIES = 2	for subsystem tie multi-terminal dc lines only.
	TIES = 3	for both interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer BRNCHS	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BRNCHS returned
	IERR = 1	Working case is empty; BRNCHS returned as 0
	IERR = 2	Invalid SID value; BRNCHS returned as 0
	IERR = 3	Invalid TIES value; BRNCHS returned as 0
	IERR = 4	Invalid FLAG value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.18.3. AMULTITRMDCCPLX

Use this API to return an array of complex values for subsystem multi-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = amultitrmdccplx(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_LINE_COMPLEX(SID, TIES, FLAG, NSTR, STRING, DIM,
 XARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem multi-terminal dc lines only.
	TIES = 2	for subsystem tie multi-terminal dc lines only.
	TIES = 3	for both interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line quantities desired (input; no default allowed).	
	'PQLOSS'	ac power losses (in MW and Mvar)
	'O_PQLOSS'	ac power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid NSTR value
IERR = 6	DIM, and hence the size of XARRAY, is not large enough
IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.18.4. AMULTITRMDCINT

Use this API to return an array of integer values for subsystem multi-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = amultitrmdcint(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_LINE_INTEGER(SID, TIES, FLAG, NSTR, STRING, DIM,
 IARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem multi-terminal dc lines only.
	TIES = 2	for subsystem tie multi-terminal dc lines only.
	TIES = 3	for both interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line quantities desired (input; no default allowed).	
	'CONVERTERS'	Number of converters
	'DCBUSES'	Number of dc buses
	'DCLINKS'	Number of dc links
	'VCNPOSNUMBER'	Positive pole voltage controlling converter bus number
	'VCNNEGNUMBER'	Negative pole voltage controlling converter bus number
	'MDC'	Control mode
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	same values specified for SID, TIES and FLAG (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid NSTR value
IERR = 6	DIM, and hence the size of IARRAY, is not large enough
IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.18.5. AMULTITRMDCREAL

Use this API to return an array of real values for subsystem multi-terminal dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = amultitrmdcreal(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_LINE_REAL(SID, TIES, FLAG, NSTR, STRING, DIM,
 RARRAY, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem multi-terminal dc lines only.
	TIES = 2	for subsystem tie multi-terminal dc lines only.
	TIES = 3	for both interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line quantities desired (input; no default allowed).	
	'VCMOD'	Mode switch dc voltage, in kV
	'PLOSS'	Active power losses, in MW
	'QLOSS'	Reactive power losses, in Mvar
	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	same values specified for SID, TIES and FLAG (input; no default allowed).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_MULTI_TERM_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid NSTR value
IERR = 6	DIM, and hence the size of RARRAY, is not large enough
IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.18.6. AMULTITRMDCTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the multi-terminal dc line data family (aMultiTrmDcInt, aMultiTrmDcReal, aMultiTrmDcCplx and aMultiTrmDcChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = amultitrmdctypes(string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_LINE_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the multi-terminal dc line data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the multi-terminal dc line data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.19. Multi-Terminal dc Line Converter Data

The multi-terminal dc line converter data family of subsystem data retrieval APIs returns arrays of data for multi-terminal dc line converters in a specified subsystem.

One of the input items to each API allows the user to elect to retrieve data either for all subsystem multi-terminal dc line converters, for subsystem multi-terminal dc line converters of multi-terminal dc lines that are tie branches, or for both subsystem and tie multi-terminal dc line converters. Another input item allows the user to elect to retrieve data for all such multi-terminal dc line converters, or to restrict the array entries to data for in-service multi-terminal dc lines.

Data for subsystem converter buses is returned from all APIs in the multi-terminal dc line converter data family in the same order. One of the input items to each API allows the user to elect that converter buses be ordered either in dc line name order with all converter buses of each multi-terminal dc line grouped together, or in numerical or alphabetical order by converter bus, according to the bus output option setting in effect at the time the API is referenced.

13.19.1. AMULTITRMDCCONVCHAR

Use this API to return an array of character values for subsystem multi-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = amultitrmdcconvchar(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_CONV_CHAR(SID, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, CARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem multi-terminal dc lines only.
	TIES = 2	for the subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 3	for the non-subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 4	for each end of tie multi-terminal dc lines only.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TIES = 5	for each end of interior subsystem multi-terminal dc lines and the subsystem bus ends of tie multi-terminal dc lines.
	TIES = 6	for each end of interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line converter quantities desired (input; no default allowed).	
	'DCNAME'	dc line name (12 characters).
	'NAME'	Converter bus name (12 characters).
	'EXNAME'	Converter bus extended bus name (18 characters).
	'DCBUSNAME'	Name of the dc bus connected to this converter bus (12 characters).
	'DCBUS2NAME'	Name of the optional second dc bus connected to this converter bus (12 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of CARRAY, is not large enough

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.19.2. AMULTITRMDCCONVCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the multi-terminal dc line converter data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, buses = amultitrmdcconvcount(sid, ties, flag)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_CONV_COUNT(SID, TIES, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem multi-terminal dc lines only.
	TIES = 2	for the subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 3	for the non-subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 4	for each end of tie multi-terminal dc lines only.
	TIES = 5	for each end of interior subsystem multi-terminal dc lines and the subsystem bus ends of tie multi-terminal dc lines.
	TIES = 6	for each end of interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer BUSES	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid TIES value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.19.3. AMULTITRMDCCONVCPLX

Use this API to return an array of complex values for subsystem multi-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = amultitrmdcconvcplx(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_CONV_COMPLEX(SID, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, XARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem multi-terminal dc lines only.
	TIES = 2	for the subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 3	for the non-subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 4	for each end of tie multi-terminal dc lines only.
	TIES = 5	for each end of interior subsystem multi-terminal dc lines and the subsystem bus ends of tie multi-terminal dc lines.
	TIES = 6	for each end of interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line converter quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PQAC'	Apparent ac power flow into the dc line at the from bus (in MW and Mvar)
	'PQLOSS'	Total dc line ac power losses (in MW and Mvar)
	'O_PQAC'	Apparent ac power flow into the dc line at the from bus (units determined by power output options)
	'O_PQLOSS'	Total dc line ac power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of XARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.19.4. AMULTITRMDCCONVINT

Use this API to return an array of integer values for subsystem multi-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = amultitrmdcconvint(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_CONV_INTEGER(SID, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, IARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem multi-terminal dc lines only.
	TIES = 2	for the subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 3	for the non-subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 4	for each end of tie multi-terminal dc lines only.
	TIES = 5	for each end of interior subsystem multi-terminal dc lines and the subsystem bus ends of tie multi-terminal dc lines.
	TIES = 6	for each end of interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line converter quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'NUMBER'	Converter bus number
	'CONVERTERTYP'	1 for a voltage controlling converter, 3 for a rectifier, 4 for an inverter
	'POLE'	1 for a positive pole converter, -1 for a negative pole converter
	'DCBUSNUMBER'	Number of the dc bus connected to this converter bus
	'DCBUS2NUMBER'	Number of the optional second dc bus connected to this converter bus
	'NBRIDGES'	Number of bridges in series
	'ANSTT'	Converter angle limit code: -1 (at or below low limit), 0 (in range), or 1 (at or above high limit)
	'TPSTT'	Converter tap ratio limit code: -1 (at or below low limit), 0 (in range), 1 (at or above high limit), or -2 (locked)
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of IARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.19.5. AMULTITRMDCCONVREAL

Use this API to return an array of real values for subsystem multi-terminal dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = amultitrmdcconvreal(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_CONV_REAL(SID, TIES, FLAG, ENTRY, NSTR, STRING,
 DIM, RARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of multi-terminal dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem multi-terminal dc lines only.
	TIES = 2	for the subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 3	for the non-subsystem bus ends of tie multi-terminal dc lines only.
	TIES = 4	for each end of tie multi-terminal dc lines only.
	TIES = 5	for each end of interior subsystem multi-terminal dc lines and the subsystem bus ends of tie multi-terminal dc lines.
	TIES = 6	for each end of interior subsystem multi-terminal dc lines and tie multi-terminal dc lines.
Integer FLAG	a flag indicating which subsystem multi-terminal dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service multi-terminal dc lines.
	FLAG = 2	for all multi-terminal dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the multi-terminal dc line converter quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'DCCUR'	Converter dc current in amps
'SETVAL'	Voltage (kV) current (amps) or power (MW) setpoint
'ACAMPS'	ac current in amps at the converter bus
'PUCUR'	ac current in pu at the converter bus
'KVDC'	Converter bus dc voltage in kV
'ANGLE'	Converter angle in degrees
'ANGMIN'	Minimum steady-state converter angle in degrees
'ANGMAX'	Nominal maximum converter angle in degrees
'TAP'	Converter bus tap setting in pu
'TPMX'	Maximum tap setting in pu
'TPMN'	Minimum tap setting in pu
'STP'	Tap step in pu
'TR'	Transformer ratio in pu
'RC'	Commutating transformer resistance per bridge in ohms
'XC'	Commutating transformer reactance per bridge in ohms
'EBASE'	Primary base ac voltage in kV
'DELTI'	Rectifier margin in pu of desired dc power or current
'DCPF'	Converter participation factor
'PAC'	Apparent active power flow into the dc line at the converter bus in MW
'QAC'	Apparent reactive power flow into the dc line at the converter bus in Mvar
'MVA'	Apparent ac power flow into the dc line at the converter bus in MVA
'PLOSS'	Total dc line active power losses in MW
'QLOSS'	Total dc line reactive power losses in Mvar
'O_PAC'	Apparent active power flow into the dc line at the converter bus (units determined by power output options)
'O_QAC'	Apparent reactive power flow into the dc line at the converter bus (units determined by power output options)
'O_MVA'	Apparent ac power flow into the dc line at the converter bus (units determined by power output options)
'O_PLOSS'	Total dc line active power losses (units determined by power output options)
'O_QLOSS'	Total dc line reactive power losses (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).
Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_MULTI_TERM_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid ENTRY value
IERR = 6	Invalid NSTR value
IERR = 7	DIM, and hence the size of RARRAY, is not large enough
IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.19.6. AMULTITRMDCCONVTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the two-terminal dc line converter data family (aMultiTrmDcConvInt, aMultiTrmDcConvReal, aMultiTrmDcConvCplx and aMultiTrmDcConvChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = amultitrmdcconvtypes(string)
```

Fortran command syntax:

```
CALL GET_MULTI_TERM_DC_CONV_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the two-terminal dc line converter data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the two-terminal dc line converter data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.20. VSC dc Line Data

The VSC dc line data family of subsystem data retrieval APIs returns arrays of data for VSC dc lines in a specified subsystem.

VSC dc lines, for subsystem VSC dc lines that are tie branches, or for both subsystem and tie VSC dc lines. Another input item allows the user to elect to retrieve data for all such VSC dc lines, to restrict the array entries to data for in-service VSC dc lines with one or both converters in-service, or to restrict the array entries to data for in-service VSC dc lines with both converters in-service.

Data for subsystem VSC dc lines is returned from all APIs in the VSC dc line data family in dc line name alphabetical order.

13.20.1. AVSCDCCHAR

Use this API to return an array of character values for subsystem VSC dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = avscdcchar(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_LINE_CHAR(SID, TIES, FLAG, NSTR, STRING, DIM, CARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem VSC dc lines only.
	TIES = 2	for subsystem tie VSC dc lines only.
	TIES = 3	for both interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service VSC dc lines with one or both converters in-service.
	FLAG = 2	for only in-service VSC dc lines with both converters in-service.
	FLAG = 3	for all VSC dc lines.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line quantities desired (input; no default allowed).
	'VSCNAME' dc line name (12 characters).
	'FROMNAME' Converter 1 bus name (12 characters).
	'FROMEXNAME' Converter 1 bus extended bus name (18 characters).
	'TONAME' Converter 2 bus name (12 characters).
	'TOEXNAME' Converter 2 bus extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Working case is empty
	IERR = 2 Invalid SID value
	IERR = 3 Invalid TIES value
	IERR = 4 Invalid FLAG value
	IERR = 5 Invalid NSTR value
	IERR = 6 DIM, and hence the size of CARRAY, is not large enough
	IERR = 7 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.20.2. AVSCDCCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the VSC dc line data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, brnchs = avscdccount(sid, ties, flag)
```

Fortran command syntax:

```
CALL GET_VSC_DC_LINE_COUNT(SID, TIES, FLAG, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem VSC dc lines only.
	TIES = 2	for subsystem tie VSC dc lines only.
	TIES = 3	for both interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service VSC dc lines with one or both converters in-service.
	FLAG = 2	for only in-service VSC dc lines with both converters in-service.
	FLAG = 3	for all VSC dc lines.
Integer BRNCHS	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BRNCHS returned
	IERR = 1	Working case is empty; BRNCHS returned as 0
	IERR = 2	Invalid SID value; BRNCHS returned as 0
	IERR = 3	Invalid TIES value; BRNCHS returned as 0
	IERR = 4	Invalid FLAG value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.20.3. AVSCDCCPLX

Use this API to return an array of complex values for subsystem VSC dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = avscdccplx(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_LINE_COMPLEX(SID, TIES, FLAG, NSTR, STRING, DIM, XARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem VSC dc lines only.
	TIES = 2	for subsystem tie VSC dc lines only.
	TIES = 3	for both interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service VSC dc lines with one or both converters in-service.
	FLAG = 2	for only in-service VSC dc lines with both converters in-service.
	FLAG = 3	for all VSC dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line quantities desired (input; no default allowed).	
	'PQLOSS'	ac power losses (in MW and Mvar)
	'O_PQLOSS'	ac power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid NSTR value
IERR = 6	DIM, and hence the size of XARRAY, is not large enough
IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.20.4. AVSCDCINT

Use this API to return an array of integer values for subsystem VSC dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = avscdcint(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_LINE_INTEGER(SID, TIES, FLAG, NSTR, STRING, DIM, IARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem VSC dc lines only.
	TIES = 2	for subsystem tie VSC dc lines only.
	TIES = 3	for both interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service VSC dc lines with one or both converters in-service.
	FLAG = 2	for only in-service VSC dc lines with both converters in-service.
	FLAG = 3	for all VSC dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line quantities desired (input; no default allowed).	
	'FROMNUMBER'	Converter 1 bus number
	'TONUMBER'	Converter 2 bus number
	'MDC'	Control mode
	'OWNERS'	Number of owners
	'OWN1'	First owner
	'OWN2'	Second owner
	'OWN3'	Third owner
	'OWN4'	Fourth owner

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid NSTR value
IERR = 6	DIM, and hence the size of IARRAY, is not large enough
IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.20.5. AVSCDCREAL

Use this API to return an array of real values for subsystem VSC dc lines.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = avscdcreal(sid, ties, flag, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_LINE_REAL(SID, TIES, FLAG, NSTR, STRING, DIM, RARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem VSC dc lines only.
	TIES = 2	for subsystem tie VSC dc lines only.
	TIES = 3	for both interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc lines to include (input; 1 by default).	
	FLAG = 1	for only in-service VSC dc lines with one or both converters in-service.
	FLAG = 2	for only in-service VSC dc lines with both converters in-service.
	FLAG = 3	for all VSC dc lines.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line quantities desired (input; no default allowed).	
	'DCCUR'	dc line current in amps
	'RDC'	dc line resistance in ohms
	'FRACT1'	First owner fraction
	'FRACT2'	Second owner fraction
	'FRACT3'	Third owner fraction
	'FRACT4'	Fourth owner fraction
	'PLOSS'	Active power losses, in MW
	'QLOSS'	Reactive power losses, in Mvar

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_PLOSS'	Active power losses (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_VSC_DC_LINE_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid NSTR value
	IERR = 6	DIM, and hence the size of RARRAY, is not large enough
	IERR = 7	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.20.6. AVSCDCTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the VSC dc line data family (aVscDcInt, aVscDcReal, aVscDcCplx and aVscDcChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = avscdctypes(string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_LINE_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the VSC dc line data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the VSC dc line data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.21. VSC dc Line Converter Data

The VSC dc line converter data family of subsystem data retrieval APIs returns arrays of data for VSC dc line converters in a specified subsystem.

One of the input items to each API allows the user to elect to retrieve data either for all subsystem VSC dc line converters, for subsystem VSC dc line converters of VSC dc lines that are tie branches, or for both subsystem and tie VSC dc line converters. Another input item allows the user to elect to retrieve data for all such VSC dc line converters, to restrict the array entries to data for all converters of in-service VSC dc lines, or to restrict the array entries to data for in-service converters of in-service VSC dc lines.

Data for subsystem converter buses is returned from all APIs in the VSC dc line converter data family in the same order. One of the input items to each API allows the user to elect that converter buses be ordered either in dc line name order with all converter buses of each VSC dc line grouped together, or in numerical or alphabetical order by converter bus, according to the bus output option setting in effect at the time the API is referenced.

13.21.1. AVSCDCCONVCHAR

Use this API to return an array of character values for subsystem VSC dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = avscdcconvchar(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_CONV_CHAR(SID, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 CARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem VSC dc lines only.
	TIES = 2	for the subsystem bus end of tie VSC dc lines only.
	TIES = 3	for the non-subsystem bus end of tie VSC dc lines only.
	TIES = 4	for each end of tie VSC dc lines only.
	TIES = 5	for each end of interior subsystem VSC dc lines and the subsystem bus end of tie VSC dc lines.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TIES = 6	for each end of interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service converters of in-service VSC dc lines.
	FLAG = 2	for all converters of in-service VSC dc lines.
	FLAG = 3	for all converters of all VSC dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line converter quantities desired (input; no default allowed).	
	'VSCNAME'	dc line name (12 characters).
	'FROMNAME'	From bus name (12 characters).
	'FROMEXNAME'	From bus extended bus name (18 characters).
	'TONAME'	To bus name (12 characters).
	'TOEXNAME'	To bus extended bus name (18 characters).
	'IREGNAME'	Bus name of the remote voltage controlled bus (12 characters).
	'IREGEXNAME'	Extended bus name of the remote voltage controlled bus (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API. DIM must be as least as large as the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid ENTRY value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of CARRAY, is not large enough

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.21.2. AVSCDCCONVCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the VSC dc line converter data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, buses = avscdcconvcount(sid, ties, flag)
```

Fortran command syntax:

```
CALL GET_VSC_DC_CONV_COUNT(SID, TIES, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem VSC dc lines only.
	TIES = 2	for the subsystem bus end of tie VSC dc lines only.
	TIES = 3	for the non-subsystem bus end of tie VSC dc lines only.
	TIES = 4	for each end of tie VSC dc lines only.
	TIES = 5	for each end of interior subsystem VSC dc lines and the subsystem bus end of tie VSC dc lines.
	TIES = 6	for each end of interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service converters of in-service VSC dc lines.
	FLAG = 2	for all converters of in-service VSC dc lines.
	FLAG = 3	for all converters of all VSC dc lines.
Integer BUSES	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BUSES returned
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	Invalid TIES value; BUSES returned as 0
IERR = 4	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.21.3. AVSCDCCONVCPLX

Use this API to return an array of complex values for subsystem VSC dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = avscdcconvcplx(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_CONV_COMPLEX(SID, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 XARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem VSC dc lines only.
	TIES = 2	for the subsystem bus end of tie VSC dc lines only.
	TIES = 3	for the non-subsystem bus end of tie VSC dc lines only.
	TIES = 4	for each end of tie VSC dc lines only.
	TIES = 5	for each end of interior subsystem VSC dc lines and the subsystem bus end of tie VSC dc lines.
	TIES = 6	for each end of interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service converters of in-service VSC dc lines.
	FLAG = 2	for all converters of in-service VSC dc lines.
	FLAG = 3	for all converters of all VSC dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line converter quantities desired (input; no default allowed).
	'PQAC' Apparent ac power flow into the dc line at the from bus (in MW and Mvar)
	'PQLOSS' ac power losses (in MW and Mvar)
	'O_PQAC' Apparent ac power flow into the dc line at the from bus (units determined by power output options)
	'O_PQLOSS' ac power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BUSES	number of elements in each column of XARRAY that are set by this API. DIM must be as least as large as the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Working case is empty
	IERR = 2 Invalid SID value
	IERR = 3 Invalid TIES value
	IERR = 4 Invalid FLAG value
	IERR = 5 Invalid ENTRY value
	IERR = 6 Invalid NSTR value
	IERR = 7 DIM, and hence the size of XARRAY, is not large enough
	IERR = 8 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.21.4. AVSCDCCONVINT

Use this API to return an array of integer values for subsystem VSC dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = avscdcconvint(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_CONV_INTEGER(SID, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 IARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem VSC dc lines only.
	TIES = 2	for the subsystem bus end of tie VSC dc lines only.
	TIES = 3	for the non-subsystem bus end of tie VSC dc lines only.
	TIES = 4	for each end of tie VSC dc lines only.
	TIES = 5	for each end of interior subsystem VSC dc lines and the subsystem bus end of tie VSC dc lines.
	TIES = 6	for each end of interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service converters of in-service VSC dc lines.
	FLAG = 2	for all converters of in-service VSC dc lines.
	FLAG = 3	for all converters of all VSC dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line converter quantities desired (input; no default allowed).
	'FROMNUMBER' From bus number
	'TONUMBER' To bus number
	'IREG' Bus number of the voltage controlled bus
	'CONVERTER' 1 if the from bus is converter 1, 2 if converter 2
	'DCTYPE' Converter dc control code
	'ACMODE' Converter ac control mode
	'NREG' Node number of the voltage controlled node
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Working case is empty
	IERR = 2 Invalid SID value
	IERR = 3 Invalid TIES value
	IERR = 4 Invalid FLAG value
	IERR = 5 Invalid ENTRY value
	IERR = 6 Invalid NSTR value
	IERR = 7 DIM, and hence the size of IARRAY, is not large enough
	IERR = 8 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.21.5. AVSCDCCONVREAL

Use this API to return an array of real values for subsystem VSC dc line converters.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = avscdcconvreal(sid, ties, flag, entry, string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_CONV_REAL(SID, TIES, FLAG, ENTRY, NSTR, STRING, DIM,
 RARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem converter buses of VSC dc lines to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem VSC dc lines only.
	TIES = 2	for the subsystem bus end of tie VSC dc lines only.
	TIES = 3	for the non-subsystem bus end of tie VSC dc lines only.
	TIES = 4	for each end of tie VSC dc lines only.
	TIES = 5	for each end of interior subsystem VSC dc lines and the subsystem bus end of tie VSC dc lines.
	TIES = 6	for each end of interior subsystem VSC dc lines and tie VSC dc lines.
Integer FLAG	a flag indicating which subsystem VSC dc line converters to include (input; 1 by default).	
	FLAG = 1	for only in-service converters of in-service VSC dc lines.
	FLAG = 2	for all converters of in-service VSC dc lines.
	FLAG = 3	for all converters of all VSC dc lines.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for converter bus order.
	ENTRY = 2	for dc line name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character STRING (NSTR)	NSTR of the following strings indicating the VSC dc line converter quantities desired (input; no default allowed).	
	'ACAMPS'	ac current in amps at the from bus
	'PUCUR'	ac current in pu at the from bus.
	'PCTMVA'	MVA as a percent of MVA rating (more restrictive of SMAX and IMAX) at the from bus
	'KVDC'	From bus dc voltage in kV
	'DCSET'	dc setpoint (kV or MW) at the from bus
	'ACSET'	ac setpoint (power factor or pu voltage) at the from bus
	'ALOSS'	From bus A loss coefficient in kW
	'BLOSS'	From bus B loss coefficient in kW/amp
	'MINLOSS'	From bus minimum converter losses in kW
	'SMAX'	From bus converter MVA rating
	'IMAX'	From bus converter current rating in amps
	'PWF'	From bus power weighting factor fraction
	'RMPCT'	Percent of total Mvar to be contributed by this converter
	'PAC'	Apparent active power flow into the dc line at the from bus, in MW
	'QAC'	Apparent reactive power flow into the dc line at the from bus, in Mvar
	'MVA'	Apparent ac power flow into the dc line at the from bus, in MVA
	'PLOSS'	Total active power losses, in MW
	'QLOSS'	Total reactive power losses, in Mvar
	'MAXQ'	Reactive power upper limit, in Mvar
	'MINQ'	Reactive power lower limit, in Mvar
	'O_PAC'	Apparent active power flow into the dc line at the from bus (units determined by power output options)
	'O_QAC'	Apparent reactive power flow into the dc line at the from bus (units determined by power output options)
	'O_MVA'	Apparent ac power flow into the dc line at the from bus (units determined by power output options)
	'O_PLOSS'	Total active power losses (units determined by power output options)
	'O_QLOSS'	Total reactive power losses (units determined by power output options)
	'O_MAXQ'	Reactive power upper limit (units determined by power output options)
	'O_MINQ'	Reactive power lower limit (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (input; no default allowed).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BUSES	number of elements in each column of RARRAY that are set by this API. DIM must be as least as large as the value of BUSES returned by the GET_VSC_DC_CONV_COUNT API when it is called with the same values specified for SID, TIES and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid ENTRY value
IERR = 6	Invalid NSTR value
IERR = 7	DIM, and hence the size of RARRAY, is not large enough
IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.21.6. AVSCDCCONVTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the VSC dc line converter data family (aVscDcConvInt, aVscDcConvReal, aVscDcConvCplx and aVscDcConvChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = avscdcconvtypes(string)
```

Fortran command syntax:

```
CALL GET_VSC_DC_CONV_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the VSC dc line converter data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the VSC dc line converter data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.22. FACTS Device Data

The FACTS device data family of subsystem data retrieval APIs returns arrays of data for FACTS devices in a specified subsystem.

FACTS devices with a series element may be subsystem tie branches. One of the input items to each API allows the user to elect to retrieve data either for all subsystem FACTS devices, for subsystem FACTS devices that are tie branches, or for both subsystem and tie FACTS devices. Another input item allows the user to elect to retrieve data only for FACTS devices without a series element, only for FACTS devices with a series element, or all types of FACTS devices.

Another input item allows the user to elect to retrieve data for all such FACTS devices, or to restrict the array entries to data for in-service FACTS devices.

Data for subsystem FACTS devices is returned from all APIs in the FACTS device data family in ascending FACTS device name alphabetical order.

13.22.1. AFACTSCHAR

Use this API to return an array of character values for subsystem FACTS devices.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = afactschar(sid, ties, flag, fcttyp, string)
```

Fortran command syntax:

```
CALL GET_FACTS_CHAR(SID, TIES, FLAG, FCTTYP, NSTR, STRING, DIM, CARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem FACTS devices only.
	TIES = 2	for subsystem tie FACTS devices only.
	TIES = 3	for both interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default). FCTTYP = 1 for only FACTS devices without a series element. FCTTYP = 2 for only FACTS devices with a series element. FCTTYP = 3 for both types of FACTS devices.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed). 'FACTSNAME' FACTS device name (12 characters). 'FROMNAME' Sending end bus name (12 characters). 'FROMEXNAME' Sending end bus extended bus name (18 characters). 'TONAME' Terminal end bus name (12 characters). 'TOEXNAME' Terminal end bus extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer BRNCHS	number of elements in each column of CARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).
Integer IERR	error code (output). IERR = 0 No error IERR = 1 Working case is empty IERR = 2 Invalid SID value IERR = 3 Invalid TIES value IERR = 4 Invalid FLAG value IERR = 5 Invalid FCTTYP value IERR = 6 Invalid NSTR value IERR = 7 DIM, and hence the size of CARRAY, is not large enough IERR = 8 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.22.2. AFACTSCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the FACTS device data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, brnchs = afactscount(sid, ties, flag, fcttyp)
```

Fortran command syntax:

```
CALL GET_FACTS_COUNT(SID, TIES, FLAG, FCTTYP, BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem FACTS devices only.
	TIES = 2	for subsystem tie FACTS devices only.
	TIES = 3	for both interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.
Integer BRNCHS	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES, FLAG and FCTTYP (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, BRNCHS returned
	IERR = 1	Working case is empty; BRNCHS returned as 0
	IERR = 2	Invalid SID value; BRNCHS returned as 0
	IERR = 3	Invalid TIES value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Invalid FLAG value; BRNCHS returned as 0
IERR = 5	Invalid FCTTYP value; BRNCHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.22.3. AFACTSCPLX

Use this API to return an array of complex values for subsystem FACTS devices.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = afactscplx(sid, ties, flag, fcttyp, string)
```

Fortran command syntax:

```
CALL GET_FACTS_COMPLEX(SID, TIES, FLAG, FCTTYP, NSTR, STRING, DIM, XARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem FACTS devices only.
	TIES = 2	for subsystem tie FACTS devices only.
	TIES = 3	for both interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed).	
	'PQDES'	Desired complex power flow at the terminal bus end (MW and Mvar)
	'PQSEND'	Complex power flow into the series element at the sending end bus (MW and Mvar)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PQTERM'	Complex power flow into the series element at the terminal end bus (MW and Mvar)
	'PQSHNT'	Complex bridge (active) and shunt (reactive) power flow into the shunt element (MW and Mvar)
	'PQLOSS'	Losses (MW and Mvar)
	'O_PQDES'	Desired complex power flow at the terminal bus end (units determined by power output options)
	'O_PQSEND'	Complex power flow into the series element at the sending end bus (units determined by power output options)
	'O_PQTERM'	Complex power flow into the series element at the terminal end bus (units determined by power output options)
	'O_PQSHNT'	Complex bridge (active) and shunt (reactive) power flow into the shunt element (units determined by power output options)
	'O_PQLOSS'	Losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of XARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid FCTTYP value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of RARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.22.4. AFACTSINT

Use this API to return an array of integer values for subsystem FACTS devices.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = afactsint(sid, ties, flag, fcttyp, string)
```

Fortran command syntax:

```
CALL GET_FACTS_INTEGER(SID, TIES, FLAG, FCTTYP, NSTR, STRING, DIM, IARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem FACTS devices only.
	TIES = 2	for subsystem tie FACTS devices only.
	TIES = 3	for both interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed).	
	'SENDNUMBER'	Sending end bus number
	'TERMNUMBER'	Terminal end bus number
	'MODE'	Control mode (negative if blocked during solution)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'OWNER'	Owner number
	'IREG'	Regulated bus number
	'NREG'	Regulated node number
	'VSREF'	Series voltage reference code
	'STATVT'	Terminal end voltage condition code: -1 (holding low limit), 0 (in range), or 1 (holding high limit)
	'STATVS'	Series voltage limit code: 0 (in range), or 1 (holding high limit)
	'STATBR'	Bridge active power transfer limit code: 0 (in range), or 1 (holding high limit)
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer BRNCHS	number of elements in each column of IARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid FCTTYP value
	IERR = 6	Invalid NSTR value
	IERR = 7	DIM, and hence the size of IARRAY, is not large enough
	IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.22.5. AFACTSREAL

Use this API to return an array of real values for subsystem FACTS devices.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = afactsreal(sid, ties, flag, fcttyp, string)
```

Fortran command syntax:

```
CALL GET_FACTS_REAL(SID, TIES, FLAG, FCTTYP, NSTR, STRING, DIM, RARRAY,
 BRNCHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem FACTS devices only.
	TIES = 2	for subsystem tie FACTS devices only.
	TIES = 3	for both interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed).	
	'VSET'	Voltage setpoint at the sending end bus in pu
	'VTMN'	Minimum voltage at the terminal end bus in pu
	'VTMX'	Maximum voltage at the terminal end bus in pu

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'VSMX'	Maximum series voltage magnitude in pu
'LINX'	Reactance of the dummy series element
'RMPCT'	Percentage of the total MVAR required to regulate the controlled bus voltage that are contributed by this device
'VSERES'	Series voltage magnitude in pu
'IPUSND'	Current flow into series element at sending end bus in pu. 'IPUTRM' Current flow into series element at terminal end bus in pu
'IPUSHN'	Current flow into shunt element in pu
'AMPSND'	Current flow into series element at sending end bus in amps
'AMPTRM'	Current flow into series element at terminal end bus in amps
'AMPSHN'	Current flow into shunt element in amps
'PCTSND'	Percent series current loading at sending bus end
'PCTTRM'	Percent series current loading at terminal bus end
'PCTSHN'	Percent shunt current loading
'PCTPBR'	Percent bridge active power loading
'PDES'	Desired active power flow at the terminal bus end, in MW
'QDES'	Desired reactive power flow at the terminal bus end, in Mvar
'SHMX'	Maximum shunt current at the sending end bus at 1.0 pu voltage, in MVA
'TRMX'	Maximum bridge active power transfer, in MVA
'IMX'	Maximum series current at 1.0 pu voltage, in MVA
'PSEND'	Active power flow into the series element at the sending end bus, in MW
'QSEND'	Reactive power flow into the series element at the sending end bus, in Mvar
'PTERM'	Active power flow into the series element at the terminal end bus, in MW
'QTERM'	Reactive power flow into the series element at the terminal end bus, in Mvar
'PBRDG'	Bridge active power exchange, in MW
'QSHNT'	Reactive power flow into the shunt element, in Mvar
'MVASND'	Power flow into the series element at the sending end bus, in MVA
'MVATRM'	Power flow into the series element at the terminal end bus, in MVA

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'MVASHN'	Power flow into the shunt element, in MVA
	'QLOSS'	Reactive power losses, in Mvar
	'O_PDES'	Desired active power flow at the terminal bus end (units determined by power output options)
	'O_QDES'	Desired reactive power flow at the terminal bus end (units determined by power output options)
	'O_SHMX'	Maximum shunt current at the sending end bus at 1.0 pu voltage (units determined by power output options)
	'O_TRMX'	Maximum bridge active power transfer (units determined by power output options)
	'O_IMX'	Maximum series current at 1.0 pu voltage (units determined by power output options)
	'O_PSEND'	Active power flow into the series element at the sending end bus (units determined by power output options)
	'O_QSEND'	Reactive power flow into the series element at the sending end bus (units determined by power output options)
	'O_PTERM'	Active power flow into the series element at the terminal end bus (units determined by power output options)
	'O_QTERM'	Reactive power flow into the series element at the terminal end bus (units determined by power output options)
	'O_PBRDG'	Bridge active power exchange (units determined by power output options)
	'O_QSHNT'	Reactive power flow into the shunt element (units determined by power output options)
	'O_MVASND'	Power flow into the series element at the sending end bus (units determined by power output options)
	'O_MVATRM'	Power flow into the series element at the terminal end bus (units determined by power output options)
	'O_MVASHN'	Power flow into the shunt element (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BRNCHS	number of elements in each column of RARRAY that are set by this API. BRNCHS is identical to the value of BRNCHS returned by the GET_FACTS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid FCTTYP value
IERR = 6	Invalid NSTR value
IERR = 7	DIM, and hence the size of RARRAY, is not large enough
IERR = 8	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.22.6. AFACTSTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the FACTS device data family (aFactsInt, aFactsReal, aFactsCplx and aFactsChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = afactstypes(string)
```

Fortran command syntax:

```
CALL GET_FACTS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the FACTS device data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the FACTS device data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.23. FACTS Device Bus Data

The FACTS device bus data family of subsystem data retrieval APIs returns arrays of data for buses that are in a specified subsystem and to which FACTS devices are connected.

One of the input items to each API allows the user to elect to retrieve data either for all subsystem FACTS device buses, for subsystem buses connected to FACTS devices with a series element that is a tie branch, or for both subsystem and tie FACTS device buses. Another input item allows the user to elect to retrieve data either for FACTS devices without a series element, for FACTS devices with a series element, or for both types of FACTS devices.

Another input item allows the user to elect to retrieve data for all such FACTS device buses, or to restrict the array entries to data for buses connected to in-service FACTS devices.

Data for subsystem buses is returned from all APIs in the FACTS device bus data family in the same order. One of the input items to each API allows the user to elect that FACTS device buses be ordered either in FACTS device name order with all buses of each FACTS device grouped together, or in numerical or alphabetical order by FACTS device bus, according to the bus output option setting in effect at the time the API is referenced.

13.23.1. AFACTSBUSCHAR

Use this API to return an array of character values for subsystem FACTS device buses.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = afactsbuschar(sid, ties, flag, fcttyp, entry, string)
```

Fortran command syntax:

```
CALL GET_FACTS_BUS_CHAR(SID, TIES, FLAG, FCTTYP, ENTRY, NSTR, STRING, DIM,
 CARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem buses connected to FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem FACTS devices only.
	TIES = 2	for the subsystem bus end of tie FACTS devices only.
	TIES = 3	for the non-subsystem bus end of tie FACTS devices only.
	TIES = 4	for each end of tie FACTS devices only.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	TIES = 5	for each end of interior subsystem FACTS devices and the subsystem bus end of tie FACTS devices.
	TIES = 6	for each end of interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.
Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for bus order.
	ENTRY = 2	for FACTS device name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed).	
	'FACTSNAME'	FACTS device name (12 characters).
	'FROMNAME'	Bus name of this FACTS bus (12 characters).
	'FROMEXNAME'	Extended bus name of this FACTS bus (18 characters).
	'TONAME'	Bus name of the other end FACTS bus (12 characters).
	'TOEXNAME'	Extended bus name of the other end FACTS bus (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	Invalid TIES value
IERR = 4	Invalid FLAG value
IERR = 5	Invalid FCTTYP value
IERR = 6	Invalid ENTRY value
IERR = 7	Invalid NSTR value
IERR = 8	DIM, and hence the size of CARRAY, is not large enough
IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.23.2. AFACTSBUSCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the FACTS device bus data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, buses = afactsbuscount(sid, ties, flag, fcttyp)
```

Fortran command syntax:

```
CALL GET_FACTS_BUS_COUNT(SID, TIES, FLAG, FCTTYP, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem buses connected to FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem FACTS devices only.
	TIES = 2	for the subsystem bus end of tie FACTS devices only.
	TIES = 3	for the non-subsystem bus end of tie FACTS devices only.
	TIES = 4	for each end of tie FACTS devices only.
	TIES = 5	for each end of interior subsystem FACTS devices and the subsystem bus end of tie FACTS devices.
	TIES = 6	for each end of interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BUSES	number of array entries required for the subsystem indicated by SID that meet the editing criteria indicated by TIES, FLAG and FCTTYP (output).
Integer IERR	error code (output).

IERR = 0	No error, BUSES returned
IERR = 1	Working case is empty; BUSES returned as 0
IERR = 2	Invalid SID value; BUSES returned as 0
IERR = 3	Invalid TIES value; BUSES returned as 0
IERR = 4	Invalid FLAG value; BUSES returned as 0
IERR = 5	Invalid FCTTYP value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.23.3. AFACTSBUSCPLX

Use this API to return an array of complex values for subsystem FACTS device buses.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = afactsbuscplx(sid, ties, flag, fcttyp, entry, string)
```

Fortran command syntax:

```
CALL GET_FACTS_BUS_COMPLEX(SID, TIES, FLAG, FCTTYP, ENTRY, NSTR, STRING,
 DIM, XARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem buses connected to FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem FACTS devices only.
	TIES = 2	for the subsystem bus end of tie FACTS devices only.
	TIES = 3	for the non-subsystem bus end of tie FACTS devices only.
	TIES = 4	for each end of tie FACTS devices only.
	TIES = 5	for each end of interior subsystem FACTS devices and the subsystem bus end of tie FACTS devices.
	TIES = 6	for each end of interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer ENTRY	a flag indicating organization (input; 1 by default).	
	ENTRY = 1	for bus order.
	ENTRY = 2	for FACTS device name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed).	
	'PQFROM'	Complex power flow into the series element at the from bus end (MW and Mvar)
	'PQSHNT'	Complex bridge (active) and shunt (reactive) power flow into the shunt element (0.0 if the from bus is the terminal end bus) (MW and Mvar)
	'PQLOSS'	Losses (MW and Mvar)
	'O_PQFROM'	Complex power flow into the series element at the from bus end (units determined by power output options)
	'O_PQSHNT'	Complex bridge (active) and shunt (reactive) power flow into the shunt element (0.0 if the from bus is the terminal end bus) (units determined by power output options)
	'O_PQLOSS'	Losses (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).	
Integer BUSES	number of elements in each column of XARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid FCTTYP value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of RARRAY, is not large enough
	IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.23.4. AFACTSBUSINT

Use this API to return an array of integer values for subsystem FACTS device buses.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = afactsbusint(sid, ties, flag, fcttyp, entry, string)
```

Fortran command syntax:

```
CALL GET_FACTS_BUS_INTEGER(SID, TIES, FLAG, FCTTYP, ENTRY, NSTR, STRING,
 DIM, IARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem buses connected to FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem FACTS devices only.
	TIES = 2	for the subsystem bus end of tie FACTS devices only.
	TIES = 3	for the non-subsystem bus end of tie FACTS devices only.
	TIES = 4	for each end of tie FACTS devices only.
	TIES = 5	for each end of interior subsystem FACTS devices and the subsystem bus end of tie FACTS devices.
	TIES = 6	for each end of interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.
Integer ENTRY	a flag indicating organization (input; 1 by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ENTRY = 1	for bus order.
	ENTRY = 2	for FACTS device name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed).	
	'FROMNUMBER'	Bus number of this FACTS bus
	'TONUMBER'	Bus number of the other end FACTS bus
	'FACTSEND'	FACTS device end code of this FACTS bus: 1 (sending end bus) or 2 (terminal end bus)
	'MODE'	Control mode (negative if blocked during solution)
	'OWNER'	Owner number
	'IREG'	Regulated bus number
	'NREG'	Regulated node number
	'VSREF'	Series voltage reference code
	'STATVT'	Terminal end voltage condition code: -1 (holding low limit), 0 (in range), or 1 (holding high limit)
	'STATVS'	Series voltage limit code: 0 (in range), or 1 (holding high limit)
	'STATBR'	Bridge active power transfer limit code: 0 (in range), or 1 (holding high limit)
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid FCTTYP value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of IARRAY, is not large enough
	IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.23.5. AFACTSBUSREAL

Use this API to return an array of real values for subsystem FACTS device buses.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = afactsbusreal(sid, ties, flag, fcttyp, entry, string)
```

Fortran command syntax:

```
CALL GET_FACTS_BUS_REAL(SID, TIES, FLAG, FCTTYP, ENTRY, NSTR, STRING, DIM,
 RARRAY, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer TIES	a flag indicating which subsystem buses connected to FACTS devices with a series element to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for each end of interior subsystem FACTS devices only.
	TIES = 2	for the subsystem bus end of tie FACTS devices only.
	TIES = 3	for the non-subsystem bus end of tie FACTS devices only.
	TIES = 4	for each end of tie FACTS devices only.
	TIES = 5	for each end of interior subsystem FACTS devices and the subsystem bus end of tie FACTS devices.
	TIES = 6	for each end of interior subsystem FACTS devices and tie FACTS devices.
Integer FLAG	a flag indicating which subsystem FACTS devices to include (input; 1 by default).	
	FLAG = 1	for only in-service FACTS devices.
	FLAG = 2	for both in-service and out-of-service FACTS devices.
Integer FCTTYP	a flag indicating which types of FACTS devices to include (input; 1 by default).	
	FCTTYP = 1	for only FACTS devices without a series element.
	FCTTYP = 2	for only FACTS devices with a series element.
	FCTTYP = 3	for both types of FACTS devices.
Integer ENTRY	a flag indicating organization (input; 1 by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ENTRY = 1	for bus order.
	ENTRY = 2	for FACTS device name order.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the FACTS device quantities desired (input; no default allowed).	
	'VSERES'	Series voltage magnitude in pu
	'IPUFROM'	Current flow into series element at the from bus end in pu
	'IPUSHN'	Current flow into shunt element in pu (0.0 if the from bus is the terminal end bus)
	'AMPSFROM'	Current flow into series element at the from bus end in amps
	'AMPSHN'	Current flow into shunt element in amps (0.0 if the from bus is the terminal end bus)
	'PCTFROM'	Percent series current loading at the from bus end
	'PCTSHN'	Percent shunt current loading (0.0 if the from bus is the terminal end bus)
	'PCTPBR'	Percent bridge active power loading (0.0 if the from bus is the terminal end bus)
	'PFROM'	Active power flow into the series element at the from bus end, in MW
	'QFROM'	Reactive power flow into the series element at the from bus end, in Mvar
	'PBRDG'	Bridge active power exchange (0.0 if the from bus is the terminal end bus), in MW
	'QSHNT'	Reactive power flow into the shunt element (0.0 if the from bus is the terminal end bus), in Mvar
	'MVAFROM'	Power flow into the series element at the from bus end, in MVA
	'MVASHN'	Power flow into the shunt element (0.0 if the from bus is the terminal end bus), in MVA
	'QLOSS'	Reactive power losses, in Mvar
	'O_PFROM'	Active power flow into the series element at the from bus end (units determined by power output options)
	'O_QFROM'	Reactive power flow into the series element at the from bus end (units determined by power output options)
	'O_PBRDG'	Bridge active power exchange (0.0 if the from bus is the terminal end bus) (units determined by power output options)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_QSHNT'	Reactive power flow into the shunt element (0.0 if the from bus is the terminal end bus) (units determined by power output options)
	'O_MVAFROM'	Power flow into the series element at the from bus end (units determined by power output options)
	'O_MVASHN'	Power flow into the shunt element (0.0 if the from bus is the terminal end bus) (units determined by power output options)
	'O_QLOSS'	Reactive power losses (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (input; no default allowed).	
Integer BUSES	number of elements in each column of RARRAY that are set by this API. BUSES is identical to the value of BUSES returned by the GET_FACTS_BUS_COUNT API when it is called with the same values specified for SID, TIES, FLAG and FCTTYP (output).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid TIES value
	IERR = 4	Invalid FLAG value
	IERR = 5	Invalid FCTTYP value
	IERR = 6	Invalid ENTRY value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of RARRAY, is not large enough
	IERR = 9	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.23.6. AFACTSBUSTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the FACTS device bus data family (aFactsBusInt, aFactsBusReal, aFactsBusCplx and aFactsBusChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = afactsbustypes(string)
```

Fortran command syntax:

```
CALL GET_FACTS_BUS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the FACTS device bus data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the FACTS device bus data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.24. Area Data

The area data family of subsystem data retrieval APIs returns arrays of data for areas in a specified area subsystem. One of the input items to each API allows the user to elect either to retrieve data for areas with any ac buses, loads, induction machines, or dc buses assigned to them, or to restrict the array entries to data for areas with at least one ac bus assigned to them.

Data for subsystem areas is returned from all APIs in the area data family in the same order: ascending area number order.

13.24.1. AAREACHAR

Use this API to return an array of character values for subsystem areas.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = aareachar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_AREA_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,AREAS,IERR)
```

Where:

Integer SID	Defines the area subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all areas in the working case.
	SID >= 0	valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem areas to include (input; 1 by default).	
	FLAG = 1	for only subsystem areas with at least one ac bus assigned to them.
	FLAG = 2	for subsystem areas with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the area quantities desired (input; no default allowed).	
	'AREANAME'	Area name (12 characters).
	'NAME'	Bus name of the area swing bus (12 characters).
	'EXNAME'	Extended bus name of the area swing bus (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of AREAS returned by the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer AREAS	number of elements in each column of CARRAY that are set by this API. AREAS is identical to the value of AREAS returned by the GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of CARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.24.2. AAREACOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the area data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, areas = aareacount(sid, flag)
```

Fortran command syntax:

```
CALL GET_AREA_COUNT(SID, FLAG, AREAS, IERR)
```

Where:

Integer SID	Defines the area subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all areas in the working case.
	SID >= 0	valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem areas to include (input; 1 by default).	
	FLAG = 1	for only subsystem areas with at least one ac bus assigned to them.
	FLAG = 2	for subsystem areas with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer AREAS	number of areas in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, AREAS returned
	IERR = 1	Working case is empty; AREAS returned as 0
	IERR = 2	Invalid SID value; AREAS returned as 0
	IERR = 3	Invalid FLAG value; AREAS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.24.3. AAREACPLX

Use this API to return an array of complex values for subsystem areas.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = aareacplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_AREA_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,AREAS,IERR)
```

Where:

Integer SID	Defines the area subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all areas in the working case.
	SID >= 0	valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem areas to include (input; 1 by default).	
	FLAG = 1	for only subsystem areas with at least one ac bus assigned to them.
	FLAG = 2	for subsystem areas with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the area quantities desired (input; no default allowed).	
	'PQLOAD'	Total complex load on buses in the area (in MW and Mvar)
	'PQLOADLD'	Total complex load of loads in the area (in MW and Mvar)
	'PQGEN'	Total complex generation on buses in the area (in MW and Mvar)
	'PQLOSS'	Total complex losses assigned to the area (in MW and Mvar)
	'PQXFRMAG'	Total complex power to transformer magnetizing admittance for winding 1 buses in the area (in MW and Mvar)
	'PQINT'	Total complex net interchange from the area (in MW and Mvar)
	'O_PQLOAD'	Total complex load on buses in the area (units determined by power output option)
	'O_PQLOADLD'	Total complex load of loads in the area (units determined by power output option)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_PQGEN'	Total complex generation on buses in the area (units determined by power output option)
	'O_PQLOSS'	Total complex losses assigned to the area (units determined by power output option)
	'O_PQXFRMAG'	Total complex power to transformer magnetizing admittance for winding 1 buses in the area (units determined by power output option)
	'O_PQINT'	Total complex net interchange from the area (units determined by power output option)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of AREAS returned by the GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer AREAS	number of elements in each column of XARRAY that are set by this API. AREAS is identical to the value of AREAS returned by the GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.24.4. AAREAINT

Use this API to return an array of integer values for subsystem areas.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = aareaint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_AREA_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,AREAS,IERR)
```

Where:

Integer SID	Defines the area subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all areas in the working case.
	SID >= 0	valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem areas to include (input; 1 by default).	
	FLAG = 1	for only subsystem areas with at least one ac bus assigned to them.
	FLAG = 2	for subsystem areas with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the area quantities desired (input; no default allowed).	
	'NUMBER'	Area number
	'SWING'	Number of the area swing bus used for area interchange control purposes
	'SWINGSTATION'	Area swing bus substation number (0 if bus is not in a substation)
	'SWINGSECTION'	Area swing bus section number (0 if bus is not represented by multiple bus sections)
	'BUSES'	Number of ac buses assigned to the area
	'LOADS'	Number of loads assigned to the area
	'INDMACS'	Number of induction machines assigned to the area
	'DCBUSES'	Number of dc buses assigned to the area
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of AREAS returned by the GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer AREAS	number of elements in each column of IARRAY that are set by this API. AREAS is identical to the value of AREAS returned by the GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of IARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.24.5. AAREAREAL

Use this API to return an array of real values for subsystem areas.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = aareareal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_AREA_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,AREAS,IERR)
```

Where:

Integer SID	Defines the area subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all areas in the working case.
	SID >= 0	valid area subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem areas to include (input; 1 by default).	
	FLAG = 1	for only subsystem areas with at least one ac bus assigned to them.
	FLAG = 2	for subsystem areas with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the area quantities desired (input; no default allowed).	
	'PDES'	Desired net active power interchange leaving the area, in MW
	'PTOL'	Interchange tolerance bandwidth, in MW
	'PLOAD'	Total active power load on buses in the area, in MW
	'QLOAD'	Total reactive power load on buses in the area, in Mvar
	'PLOADLD'	Total active power load of loads in the area, in MW
	'QLOADLD'	Total reactive power load of loads in the area, in Mvar
	'PGEN'	Total active power generation on buses in the area, in MW
	'QGEN'	Total reactive power generation on buses in the area, in Mvar
	'PLOSS'	Total active power losses assigned to the area, in MW

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

'QLOSS'	Total reactive power losses assigned to the area, in Mvar
'PXFRMAG'	Total active power to transformer magnetizing admittance for winding 1 buses in the area, in MW
'QXFRMAG'	Total reactive power to transformer magnetizing admittance for winding 1 buses in the area, in MVar
'PINT'	Total active power net interchange from the area, in MW
'QINT'	Total reactive power net interchange from the area, in Mvar
'O_PDES'	Desired net active power interchange leaving the area (units determined by power output option)
'O_PTOL'	Interchange tolerance bandwidth (units determined by power output option)
'O_PLOAD'	Total active power load on buses in the area (units determined by power output option)
'O_QLOAD'	Total reactive power load on buses in the area (units determined by power output option)
'O_PLOADLD'	Total active power load of loads in the area (units determined by power output option)
'O_QLOADLD'	Total reactive power load of loads in the area (units determined by power output option)
'O_PGEN'	Total active power generation on buses in the area (units determined by power output option)
'O_QGEN'	Total reactive power generation on buses in the area (units determined by power output option)
'O_PLOSS'	Total active power losses assigned to the area (units determined by power output option)
'O_QLOSS'	Total reactive power losses assigned to the area (units determined by power output option)
'O_PXFRMAG'	Total active power to transformer magnetizing admittance for winding 1 buses in the area (units determined by power output option)
'O_QXFRMAG'	Total reactive power to transformer magnetizing admittance for winding 1 buses in the area (units determined by power output option)
'O_PINT'	Total active power net interchange from the area (units determined by power output option)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_QINT'	Total reactive power net interchange from the area (units determined by power output option)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of AREAS returned by the GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer AREAS	number of elements in each column of RARRAY that are set by this API. AREAS is identical to the value of AREAS returned by the GET_AREA_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of RARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.24.6. AAREATYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the area data family (aAreaInt, aAreaReal, aAreaCplx and aAreaChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = aareatypes(string)
```

Fortran command syntax:

```
CALL GET_AREA_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the area data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the area data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.25. Owner Data

The owner data family of subsystem data retrieval APIs returns arrays of data for owners in a specified owner subsystem. One of the input items to each API allows the user to elect either to retrieve data for owners with any ac buses, loads, machines, induction machines, branches, FACTS devices, VSC dc lines, or dc buses assigned to them, or to restrict the array entries to data for owners with at least one ac bus assigned to them.

Data for subsystem owners is returned from all APIs in the owner data family in the same order: ascending owner number order.

13.25.1. AOWNERCHAR

Use this API to return an array of character values for subsystem owners.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = aownerchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_OWNER_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,OWNERS,IERR)
```

Where:

Integer SID	Defines the owner subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all owners in the working case.
	SID >= 0	valid owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem owners to include (input; 1 by default).	
	FLAG = 1	for only subsystem owners with at least one ac bus assigned to them.
	FLAG = 2	for subsystem owners with any equipment (ac buses, loads, machines, ac branches, FACTS devices, VSC dc lines and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the owner quantities desired (input; no default allowed).	
	'OWNERNAME'	Owner name (12 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer OWNERS	number of elements in each column of CARRAY that are set by this API. OWNERS is identical to the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of CARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.25.2. AOWNERCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the owner data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, owners = aownercount(sid, flag)
```

Fortran command syntax:

```
CALL GET_OWNER_COUNT(SID, FLAG, OWNERS, IERR)
```

Where:

Integer SID	Defines the owner subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all owners in the working case.
	SID >= 0	valid owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem owners to include (input; 1 by default).	
	FLAG = 1	for only subsystem owners with at least one ac bus assigned to them.
	FLAG = 2	for subsystem owners with any equipment (ac buses, loads, machines, ac branches, FACTS devices, VSC dc lines and/or dc buses) assigned to them.
Integer OWNERS	number of owners in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, OWNERS returned
	IERR = 1	Working case is empty; OWNERS returned as 0
	IERR = 2	Invalid SID value; OWNERS returned as 0
	IERR = 3	Invalid FLAG value; OWNERS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.25.3. AOWNERCPLX

Use this API to return an array of complex values for subsystem owners.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = aownercplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_OWNER_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,OWNERS,IERR)
```

Where:

Integer SID	Defines the owner subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all owners in the working case.
	SID >= 0	valid owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem owners to include (input; 1 by default).	
	FLAG = 1	for only subsystem owners with at least one ac bus assigned to them.
	FLAG = 2	for subsystem owners with any equipment (ac buses, loads, machines, ac branches, FACTS devices, VSC dc lines and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the owner quantities desired (input; no default allowed).	
	'PQLOAD'	Total complex load on the owner's buses (in MW and Mvar)
	'PQLOADLD'	Total complex load of the owner's loads (in MW and Mvar)
	'PQGEN'	Total complex generation on the owner's buses (in MW and Mvar)
	'PQGENMC'	Total complex generation on the owner's machines (in MW and Mvar)
	'PQLOSS'	Total complex losses assigned to the owner (in MW and Mvar)
	'PQXFRMAG'	Total complex power to transformer magnetizing admittance assigned to the owner (in MW and Mvar)
	'O_PQLOAD'	Total complex load on the owner's buses (units determined by power output option)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_PQLOADLD'	Total complex load of the owner's loads (units determined by power output option)
	'O_PQGEN'	Total complex generation on the owner's buses (units determined by power output option)
	'O_PQGENMC'	Total complex generation on the owner's machines (units determined by power output option)
	'O_PQLOSS'	Total complex losses assigned to the owner (units determined by power output option)
	'O_PQXFRMAG'	Total complex power to transformer magnetizing admittance assigned to the owner (units determined by power output option)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer OWNERS	number of elements in each column of XARRAY that are set by this API. OWNERS is identical to the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.25.4. AOWNERINT

Use this API to return an array of integer values for subsystem owners.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = aownerint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_OWNER_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,OWNERS,IERR)
```

Where:

Integer SID	Defines the owner subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all owners in the working case.
	SID >= 0	valid owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem owners to include (input; 1 by default).	
	FLAG = 1	for only subsystem owners with at least one ac bus assigned to them.
	FLAG = 2	for subsystem owners with any equipment (ac buses, loads, machines, ac branches, FACTS devices, VSC dc lines and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the owner quantities desired (input; no default allowed).	
	'NUMBER'	Owner number
	'BUSES'	Number of ac buses assigned to the owner
	'LOADS'	Number of loads assigned to the owner
	'MACHINES'	Number of machine ownership specifications for the owner
	'INDMACS'	Number of induction machines assigned to the owner
	'BRANCHES'	Number of branch ownership specifications for the owner
	'FACTS'	Number of FACTS devices assigned to the owner
	'VSCDCLINES'	Number of VSC dc line ownership specifications for the owner
	'DCBUSES'	Number of dc buses assigned to the owner

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer OWNERS	number of elements in each column of IARRAY that are set by this API. OWNERS is identical to the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of IARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.25.5. AOWNERREAL

Use this API to return an array of real values for subsystem owners.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = aownerreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_OWNER_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,OWNERS,IERR)
```

Where:

Integer SID	Defines the owner subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all owners in the working case.
	SID >= 0	valid owner subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem owners to include (input; 1 by default).	
	FLAG = 1	for only subsystem owners with at least one ac bus assigned to them.
	FLAG = 2	for subsystem owners with any equipment (ac buses, loads, machines, ac branches, FACTS devices, VSC dc lines and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the owner quantities desired (input; no default allowed).	
	'PLOAD'	Total active power load on the owner's buses, in MW
	'QLOAD'	Total reactive power load on the owner's buses, in Mvar
	'PLOADLD'	Total active power load of the owner's loads, in MW
	'QLOADLD'	Total reactive power load of the owner's loads, in Mvar
	'PGEN'	Total active power generation on the owner's buses, in Mw
	'QGEN'	Total reactive power generation on the owner's buses, in Mvar
	'PGENMC'	Total active power generation on the owner's machines, in MW
	'QGENMC'	Total reactive power generation on the owner's machines, in Mvar

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PLOSS'	Total active power losses assigned to the owner, in MW
	'QLOSS'	Total reactive power losses assigned to the owner, in Mvar
	'PXFRMAG'	Total active power to transformer magnetizing admittance assigned to the owner, in MW
	'QXFRMAG'	Total reactive power to transformer magnetizing admittance assigned to the owner, in MVar
	'O_PLOAD'	Total active power load on the owner's buses (units determined by power output option)
	'O_QLOAD'	Total reactive power load on the owner's buses (units determined by power output option)
	'O_PLOADLD'	Total active power load of the owner's loads (units determined by power output option)
	'O_QLOADLD'	Total reactive power load of the owner's loads (units determined by power output option)
	'O_PGEN'	Total active power generation on the owner's buses (units determined by power output option)
	'O_QGEN'	Total reactive power generation on the owner's buses (units determined by power output option)
	'O_PGENMC'	Total active power generation on the owner's machines (units determined by power output option)
	'O_QGENMC'	Total reactive power generation on the owner's machines (units determined by power output option)
	'O_PLOSS'	Total active power losses assigned to the owner (units determined by power output option)
	'O_QLOSS'	Total reactive power losses assigned to the owner (units determined by power output option)
	'O_PXFRMAG'	Total active power to transformer magnetizing admittance assigned to the owner (units determined by power output option)
	'O_QXFRMAG'	Total reactive power to transformer magnetizing admittance assigned to the owner (units determined by power output option)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer OWNERS	number of elements in each column of RARRAY that are set by this API. OWNERS is identical to the value of OWNERS returned by the GET_OWNER_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.25.6. AOWNERTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the owner data family (aOwnerInt, aOwnerReal, aOwnerCplx and aOwnerChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = aownertypes(string)
```

Fortran command syntax:

```
CALL GET_OWNER_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the owner data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the owner data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.26. Zone Data

The zone data family of subsystem data retrieval APIs returns arrays of data for zones in a specified zone subsystem. One of the input items to each API allows the user to elect either to retrieve data for zones with any ac buses, loads, induction machines, or dc buses assigned to them, or to restrict the array entries to data for zones with at least one ac bus assigned to them.

Data for subsystem zones is returned from all APIs in the zone data family in the same order: ascending zone number order.

13.26.1. AZONECHAR

Use this API to return an array of character values for subsystem zones.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, carray = azonechar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_ZONE_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,ZONES,IERR)
```

Where:

Integer SID	Defines the zone subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all zones in the working case.
	SID >= 0	valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem zones to include (input; 1 by default).	
	FLAG = 1	for only subsystem zones with at least one ac bus assigned to them.
	FLAG = 2	for subsystem zones with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the zone quantities desired (input; no default allowed).	
	'ZONENAME'	Zone name (12 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of ZONES returned by the GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer ZONES	number of elements in each column of CARRAY that are set by this API. ZONES is identical to the value of ZONES returned by the GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of CARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.26.2. AZONECOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the zone data family.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, zones = azonecount(sid, flag)
```

Fortran command syntax:

```
CALL GET_ZONE_COUNT(SID, FLAG, ZONES, IERR)
```

Where:

Integer SID	Defines the zone subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all zones in the working case.
	SID >= 0	valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem zones to include (input; 1 by default).	
	FLAG = 1	for only subsystem zones with at least one ac bus assigned to them.
	FLAG = 2	for subsystem zones with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer ZONES	number of zones in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, ZONES returned
	IERR = 1	Working case is empty; ZONES returned as 0
	IERR = 2	Invalid SID value; ZONES returned as 0
	IERR = 3	Invalid FLAG value; ZONES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.26.3. AZONECPLX

Use this API to return an array of complex values for subsystem zones.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, xarray = azonecplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_ZONE_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,ZONES,IERR)
```

Where:

Integer SID	Defines the zone subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all zones in the working case.
	SID >= 0	valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem zones to include (input; 1 by default).	
	FLAG = 1	for only subsystem zones with at least one ac bus assigned to them.
	FLAG = 2	for subsystem zones with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the zone quantities desired (input; no default allowed).	
	'PQLOAD'	Total complex load on buses in the zone (in MW and Mvar)
	'PQLOADLD'	Total complex load of loads in the zone (in MW and Mvar)
	'PQGEN'	Total complex generation on buses in the zone (in MW and Mvar)
	'PQLOSS'	Total complex losses assigned to the zone (in MW and Mvar)
	'PQXFRMAG'	Total complex power to transformer magnetizing admittance for winding 1 buses in the zone (in MW and Mvar)
	'PQINT'	Total complex net interchange from the zone (in MW and Mvar)
	'O_PQLOAD'	Total complex load on buses in the zone (units determined by power output option)
	'O_PQLOADLD'	Total complex load of loads in the zone (units determined by power output option)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'O_PQGEN'	Total complex generation on buses in the zone (units determined by power output option)
	'O_PQLOSS'	Total complex losses assigned to the zone (units determined by power output option)
	'O_PQXFRMAG'	Total complex power to transformer magnetizing admittance for winding 1 buses in the zone (units determined by power output option)
	'O_PQINT'	Total complex net interchange from the zone (units determined by power output option)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of ZONES returned by the GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer ZONES	number of elements in each column of XARRAY that are set by this API. ZONES is identical to the value of ZONES returned by the GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.26.4. AZONEINT

Use this API to return an array of integer values for subsystem zones.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, iarray = azoneint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_ZONE_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,ZONES,IERR)
```

Where:

Integer SID	Defines the zone subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all zones in the working case.
	SID >= 0	valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem zones to include (input; 1 by default).	
	FLAG = 1	for only subsystem zones with at least one ac bus assigned to them.
	FLAG = 2	for subsystem zones with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the zone quantities desired (input; no default allowed).	
	'NUMBER'	Zone number
	'BUSES'	Number of ac buses assigned to the zone
	'LOADS'	Number of loads assigned to the zone
	'INDMACS'	Number of induction machines assigned to the zone
	'DCBUSES'	Number of dc buses assigned to the zone
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of ZONES returned by the GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer ZONES	number of elements in each column of IARRAY that are set by this API. ZONES is identical to the value of ZONES returned by the GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.26.5. AZONEREAL

Use this API to return an array of real values for subsystem zones.

This API routine was first introduced in release 30.3.0.

Python command syntax:

```
ierr, rarray = azonereal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_ZONE_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,ZONES,IERR)
```

Where:

Integer SID	Defines the zone subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all zones in the working case.
	SID >= 0	valid zone subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem zones to include (input; 1 by default).	
	FLAG = 1	for only subsystem zones with at least one ac bus assigned to them.
	FLAG = 2	for subsystem zones with any equipment (ac buses, loads, and/or dc buses) assigned to them.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the zone quantities desired (input; no default allowed).	
	'PLOAD'	Total active power load on buses in the zone, in MW
	'QLOAD'	Total reactive power load on buses in the zone, in Mvar
	'PLOADLD'	Total active power load of loads in the zone, in MW
	'QLOADLD'	Total reactive power load of loads in the zone, in Mvar
	'PGEN'	Total active power generation on buses in the zone, in MW
	'QGEN'	Total reactive power generation on buses in the zone, in Mvar
	'PLOSS'	Total active power losses assigned to the zone, in MW
	'QLOSS'	Total reactive power losses assigned to the zone, in Mvar

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'PXFRMAG'	Total active power to transformer magnetizing admittance for winding 1 buses in the zone, in MW
	'QXFRMAG'	Total reactive power to transformer magnetizing admittance for winding 1 buses in the zone, in MVar
	'PINT'	Total active power net interchange from the zone, in MW
	'QINT'	Total reactive power net interchange from the zone, in Mvar
	'O_PLOAD'	Total active power load on buses in the zone (units determined by power output option)
	'O_QLOAD'	Total reactive power load on buses in the zone (units determined by power output option)
	'O_PLOADLD'	Total active power load of loads in the zone (units determined by power output option)
	'O_QLOADLD'	Total reactive power load of loads in the zone (units determined by power output option)
	'O_PGEN'	Total active power generation on buses in the zone (units determined by power output option)
	'O_QGEN'	Total reactive power generation on buses in the zone (units determined by power output option)
	'O_PLOSS'	Total active power losses assigned to the zone (units determined by power output option)
	'O_QLOSS'	Total reactive power losses assigned to the zone (units determined by power output option)
	'O_PXFRMAG'	Total active power to transformer magnetizing admittance for winding 1 buses in the zone (units determined by power output option)
	'O_QXFRMAG'	Total reactive power to transformer magnetizing admittance for winding 1 buses in the zone (units determined by power output option)
	'O_PINT'	Total active power net interchange from the zone (units determined by power output option)
	'O_QINT'	Total reactive power net interchange from the zone (units determined by power output option)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of ZONES returned by the	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer ZONES	number of elements in each column of RARRAY that are set by this API. ZONES is identical to the value of ZONES returned by the GET_ZONE_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.26.6. AZONETYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the zone data family (aZoneInt, aZoneReal, aZoneCplx and aZoneChar).

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
ierr, types = azonetypes(string)
```

Fortran command syntax:

```
CALL GET_ZONE_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the zone data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the zone data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.27. Induction Machine Bus Data

The induction machine bus data family of subsystem data retrieval API routines returns arrays of data for buses in a specified subsystem. One of the input items to each API routine allows the user to elect either to retrieve data for all subsystem buses, or to restrict the array entries to data for a subset of buses according to their bus and/or induction machine configuration and statuses.

Data for subsystem buses is returned from all API routines in the induction machine bus data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API routine is referenced.

13.27.1. AINDMACBUSCHAR

Use this API routine to return an array of character values for subsystem buses.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, carray = aindmacbuschar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_BUS_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction machine.
	FLAG = 2	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 3	for all in-service subsystem buses (type code 1, 2 or 3), including those that have no induction machines at the bus.
	FLAG = 4	for all subsystem buses with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 5	for all subsystem buses, including those that have no induction machines at the bus.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 10	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction generator.
	FLAG = 11	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction generator, even if all induction generators at the bus are out-of-service.
	FLAG = 12	for all subsystem buses with at least one induction generator, even if all induction generators at the bus are out-of-service.
	FLAG = 20	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction motor.
	FLAG = 21	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction motor, even if all induction motors at the bus are out-of-service.
	FLAG = 22	for all subsystem buses with at least one induction motor, even if all induction motors at the bus are out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'NAME'	Bus name (12 characters).
	'EXNAME'	Extended bus name (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of CARRAY that are set by this API routine. BUSES is identical to the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (output).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.27.2. AINDMACBUSCOUNT

Use this API routine to obtain the number of array entries required to accommodate the data to be returned by the remaining members of the induction machine bus data family.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, buses = aindmacbuscount(sid, flag)
```

Fortran command syntax:

```
CALL GET_IND_MAC_BUS_COUNT(SID, FLAG, BUSES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction machine.
	FLAG = 2	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 3	for all in-service subsystem buses (type code 1, 2 or 3), including those that have no induction machines at the bus.
	FLAG = 4	for all subsystem buses with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 5	for all subsystem buses, including those that have no induction machines at the bus.
	FLAG = 10	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction generator.
	FLAG = 11	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction generator, even if all induction generators at the bus are out-of-service.
	FLAG = 12	for all subsystem buses with at least one induction generator, even if all induction generators at the bus are out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 20	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction motor.
	FLAG = 21	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction motor, even if all induction motors at the bus are out-of-service.
	FLAG = 22	for all subsystem buses with at least one induction motor, even if all induction motors at the bus are out-of-service.
Integer BUSES	number of buses in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty; BUSES returned as 0
	IERR = 2	Invalid SID value; BUSES returned as 0
	IERR = 3	Invalid FLAG value; BUSES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.27.3. AINDMACBUSCPLX

Use this API routine to return an array of complex values for subsystem buses.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, xarray = aindmacbuscplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_BUS_COMPLEX(SID, FLAG, NSTR, STRING, DIM, XARRAY, BUSES,
 IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction machine.
	FLAG = 2	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 3	for all in-service subsystem buses (type code 1, 2 or 3), including those that have no induction machines at the bus.
	FLAG = 4	for all subsystem buses with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 5	for all subsystem buses, including those that have no induction machines at the bus.
	FLAG = 10	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction generator.
	FLAG = 11	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction generator, even if all induction generators at the bus are out-of-service.
	FLAG = 12	for all subsystem buses with at least one induction generator, even if all induction generators at the bus are out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 20	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction motor.
	FLAG = 21	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction motor, even if all induction motors at the bus are out-of-service.
	FLAG = 22	for all subsystem buses with at least one induction motor, even if all induction motors at the bus are out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'VOLTAGE'	Actual bus voltage, in pu, rectangular coordinates
	'TOTAL'	Flow from the bus to the in-service induction machines at the bus (in MW and Mvar)
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (in MW and Mvar)
	'O_TOTAL'	Flow from the bus to the in-service induction machines at the bus (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of XARRAY that are set by this API routine. BUSES is identical to the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (output).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.27.4. AINDMACBUSINT

Use this API routine to return an array of integer values for subsystem buses.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, iarray = aindmacbusint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_BUS_INTEGER(SID, FLAG, NSTR, STRING, DIM, IARRAY, BUSES,
 IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction machine.
	FLAG = 2	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 3	for all in-service subsystem buses (type code 1, 2 or 3), including those that have no induction machines at the bus.
	FLAG = 4	for all subsystem buses with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 5	for all subsystem buses, including those that have no induction machines at the bus.
	FLAG = 10	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction generator.
	FLAG = 11	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction generator, even if all induction generators at the bus are out-of-service.
	FLAG = 12	for all subsystem buses with at least one induction generator, even if all induction generators at the bus are out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 20	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction motor.
	FLAG = 21	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction motor, even if all induction motors at the bus are out-of-service.
	FLAG = 22	for all subsystem buses with at least one induction motor, even if all induction motors at the bus are out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'TYPE'	Bus type code
	'AREA'	Bus area number
	'ZONE'	Bus zone number
	'OWNER'	Bus owner number
	'DUMMY'	Returns 1 if the bus is a dummy bus of a multisection line, or 0 if it is not
	'STATUS'	Inferred induction machine status. Returns 1 if the bus type code is 1, 2 or 3, and at least one in-service induction machine is present or 0 otherwise
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of IARRAY that are set by this API routine. BUSES is identical to the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (output).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.27.5. AINDMACBUSREAL

Use this API routine to return an array of real values for subsystem buses.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, rarray = aindmacbusreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_BUS_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,BUSES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem buses to include (input; 1 by default).	
	FLAG = 1	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction machine.
	FLAG = 2	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 3	for all in-service subsystem buses (type code 1, 2 or 3), including those that have no induction machines at the bus.
	FLAG = 4	for all subsystem buses with at least one induction machine, even if all induction machines at the bus are out-of-service.
	FLAG = 5	for all subsystem buses, including those that have no induction machines at the bus.
	FLAG = 10	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction generator.
	FLAG = 11	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction generator, even if all induction generators at the bus are out-of-service.
	FLAG = 12	for all subsystem buses with at least one induction generator, even if all induction generators at the bus are out-of-service.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 20	for all in-service subsystem buses (type code 1, 2 or 3) with at least one in-service induction motor.
	FLAG = 21	for all in-service subsystem buses (type code 1, 2 or 3) with at least one induction motor, even if all induction motors at the bus are out-of-service.
	FLAG = 22	for all subsystem buses with at least one induction motor, even if all induction motors at the bus are out-of-service.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'BASE'	Bus base voltage, in kV
	'PU'	Actual bus voltage magnitude, in pu
	'KV'	Actual bus voltage magnitude, in kV (in pu if base voltage is 0.0)
	'ANGLE'	Bus voltage phase angle, in radians
	'ANGLED'	Bus voltage phase angle, in degrees
	'TOTAL'	Flow from the bus to the in-service induction machines at the bus, in MVA
	'MISMATCH'	Bus mismatch (0.0 if bus type code > 3), in MVA
	'O_TOTAL'	Flow from the bus to the in-service induction machines at the bus (units determined by power output options)
	'O_MISMATCH'	Bus mismatch (0.0 if bus type code > 3) (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer BUSES	number of elements in each column of RARRAY that are set by this API routine. BUSES is identical to the value of BUSES returned by the aIndMacBusCount API routine when it is called with the same values specified for SID and FLAG (output).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.27.6. AINDMACBUSTYPES

Use this API routine to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the induction machine bus data family (aIndMacBusInt, aIndMacBusReal, aIndMacBusCplx and aIndMacBusChar).

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, types = aindmacbustypes(string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_BUS_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the induction machine bus data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the induction machine bus data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.28. Induction Machine Data

The induction machine data family of subsystem data retrieval API routines returns arrays of data for induction machines in a specified subsystem. One of the input items to each API routine allows the user to elect either to retrieve data for all subsystem induction machines, or to restrict the array entries to data for a subset of induction machines according to their bus and/or induction machine configuration and statuses.

Data for subsystem induction machines is returned from all API routines in the induction machine data family in the same order: ascending bus number order or alphabetic extended bus name order, according to the bus output option setting in effect at the time the API routine is referenced; multiple induction machines at the same bus are in ascending induction machine identifier order.

13.28.1. AINDMACCHAR

Use this API routine to return an array of character values for subsystem induction machines.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, carray = aindmacchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem induction machines to include (input; 1 by default).	
	FLAG = 1	for all in-service induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 2	for all induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 3	for all in-service induction machines at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 4	for all induction machines at all subsystem buses.
	FLAG = 11	for all in-service induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 12	all induction generators at in-service subsystem buses (type code 1, 2 or 3).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 13	for all in-service induction generators at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 14	for all induction generators at all subsystem buses.
	FLAG = 21	for all in-service induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 22	all induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 23	for all in-service induction motors at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 24	for all induction motors at all subsystem buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'ID'	Induction machine identifier (2 characters).
	'NAME'	Bus name (12 characters).
	'EXNAME'	Extended bus name (18 characters).
	'IMNAME'	induction machine name description (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of CARRAY that are set by this API routine. MACHS is identical to the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.28.2. AINDMACCOUNT

Use this API routine to obtain the number of array entries required to accommodate the data to be returned by the remaining members of the induction machine data family.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, machs = aindmaccount(sid, flag)
```

Fortran command syntax:

```
CALL GET_IND_MAC_COUNT(SID, FLAG, MACHS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem induction machines to include (input; 1 by default).	
	FLAG = 1	for all in-service induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 2	for all induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 3	for all in-service induction machines at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 4	for all induction machines at all subsystem buses.
	FLAG = 11	for all in-service induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 12	all induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 13	for all in-service induction generators at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 14	for all induction generators at all subsystem buses.
	FLAG = 21	for all in-service induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 22	all induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 23	for all in-service induction motors at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 24	for all induction motors at all subsystem buses.
Integer MACHS	number of induction machines in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty; MACHS returned as 0
	IERR = 2	Invalid SID value; MACHS returned as 0
	IERR = 3	Invalid FLAG value; MACHS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.28.3. AINDMACCPLX

Use this API routine to return an array of complex values for subsystem induction machines.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, xarray = aindmaccplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem induction machines to include (input; 1 by default).	
	FLAG = 1	for all in-service induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 2	for all induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 3	for all in-service induction machines at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 4	for all induction machines at all subsystem buses.
	FLAG = 11	for all in-service induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 12	all induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 13	for all in-service induction generators at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 14	for all induction generators at all subsystem buses.
	FLAG = 21	for all in-service induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 22	all induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 23	for all in-service induction motors at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 24	for all induction motors at all subsystem buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'ZA'	(RA + j XA)
	'Z1'	(R1 + j X1)
	'Z2'	(R2 + j X2)
	'ZZERO'	Machine zero sequence fault analysis impedance
	'ZGROUND'	Machine zero sequence grounding impedance
	'PQ'	Complex power flow from the bus to the machine (in MW and Mvar)
	'O_PQ'	Complex power flow from the bus to the machine (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of XARRAY that are set by this API routine. MACHS is identical to the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case (when STRING = 'ZZERO' or 'ZGROUND')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.28.4. AINDMACINT

Use this API routine to return an array of integer values for subsystem induction machines.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, iarray = aindmacint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem induction machines to include (input; 1 by default).	
	FLAG = 1	for all in-service induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 2	for all induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 3	for all in-service induction machines at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 4	for all induction machines at all subsystem buses.
	FLAG = 11	for all in-service induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 12	all induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 13	for all in-service induction generators at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 14	for all induction generators at all subsystem buses.
	FLAG = 21	for all in-service induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 22	all induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 23	for all in-service induction motors at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 24	for all induction motors at all subsystem buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'NUMBER'	Bus number
	'AREA'	Machine area assignment
	'ZONE'	Machine zone assignment
	'OWNER'	Machine owner assignment
	'STATUS'	Machine status
	'STANDARDCODE'	Machine standard code (1 for NEMA, 2 for IEC)
	'DESIGNCODE'	Machine design code (0 through 5)
	'PSETCODE'	Power setpoint code (1 for mechanical, 2 for electrical)
	'BASECODE'	Machine base power code (1 for mechanical, 2 for electrical)
	'TRQVARCODE'	Mechanical torque variation code (1 for simple power law, 2 for WECC model)
	'CZG'	Grounding impedance data input/outout (I/O) code (1 for per unit, 2 for ohms)
	'CGR'	Grounding code (0 for an ungrounded machine, 1 for a grounded machine)
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of IARRAY that are set by this API routine. MACHS is identical to the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of IARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case (when STRING = 'CZG' or 'CGR')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.28.5. AINDMACREAL

Use this API routine to return an array of real values for subsystem induction machines.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, rarray = aindmacreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,MACHS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API routine to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer FLAG	a flag indicating which subsystem induction machines to include (input; 1 by default).	
	FLAG = 1	for all in-service induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 2	for all induction machines at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 3	for all in-service induction machines at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 4	for all induction machines at all subsystem buses.
	FLAG = 11	for all in-service induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 12	all induction generators at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 13	for all in-service induction generators at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).
	FLAG = 14	for all induction generators at all subsystem buses.
	FLAG = 21	for all in-service induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 22	all induction motors at in-service subsystem buses (type code 1, 2 or 3).
	FLAG = 23	for all in-service induction motors at subsystem buses, even if the bus to which it is connected is out-of-service (type code 4).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FLAG = 24	for all induction motors at all subsystem buses.
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or induction machine quantities desired (input; no default allowed).	
	'MBASE'	Machine base power
	'RATEKV'	Machine rated voltage
	'PSETPOINT'	Power setpoint (positive for a motor)
	'RA'	Armature resistance
	'XA'	Armature leakage reactance
	'XM'	Magnetizing reactance
	'R1'	First cage resistance
	'X1'	First cage reactance
	'R2'	Second cage resistance
	'X2'	Second cage reactance
	'X3'	Third reactance
	'E1'	First saturation voltage
	'SE1'	SE(E1)
	'E2'	Second saturation voltage
	'SE2'	SE(E2)
	'IA1'	Starting stator current for applying stator leakage reactance saturation
	'IA2'	Stator current at which the stator leakage reactance is fully saturated
	'XAMULT'	Xa multiplier at full leakage reactance saturation
	'TRQA'	Load torque variation constant A for WECC model
	'TRQB'	Load torque variation constant B for WECC model
	'TRQD'	Load torque variation constant D for both models
	'TRQE'	Load torque variation constant E for WECC model
	'SLIP'	Slip
	'H'	Machine inertia
	'ILKDOVERIRAT'	Ratio of locked rotor current to rated current
	'ROVERX'	Ratio of Rpos to Xpos
	'RZERO'	Machine zero sequence fault analysis resistance
	'XZERO'	Machine zero sequence fault analysis reactance
	'RGROUND'	Machine zero sequence grounding resistance
	'XGROUND'	Machine zero sequence grounding reactance

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'P'	Active power flow from the bus to the machine, in MW
	'Q'	Reactive power flow from the bus to the machine, in Mvar
	'MVA'	MVA flow from the bus to the machine, in MVA
	'O_P'	Active power flow from the bus to the machine (units determined by power output options)
	'O_Q'	Reactive power flow from the bus to the machine (units determined by power output options)
	'O_MVA'	MVA flow from the bus to the machine (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MACHS	number of elements in each column of RARRAY that are set by this API routine. MACHS is identical to the value of MACHS returned by the aIndMacCount API routine when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of RARRAY, is not large enough
	IERR = 6	Invalid STRING value
	IERR = 7	Sequence data not in case (when STRING = 'ILKDOVERIRAT', 'ROVERX', 'RZERO', 'XZERO', 'XZERO', 'RGROUND' or 'XGROUND')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.28.6. AINDMACTYPES

Use this API routine to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the induction machine data family (aIndMacInt, aIndMacReal, aIndMacCplx and aIndMacChar).

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, types = aindmactypes(string)
```

Fortran command syntax:

```
CALL GET_IND_MAC_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the induction machine data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the induction machine data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.29. Substation Data

The substation data family of subsystem data retrieval APIs returns arrays of data for substations in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for substations with a least one subsystem bus, or to restrict the array entries to data for substations with all of their buses in the subsystem.

Data for subsystem substations is returned from all APIs in the substation data family in ascending substation number order.

13.29.1. ASTATIONCHAR

Use this API to return an array of character values for subsystem substations.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, carray = astationchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SUBSTATION_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,STATIONS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (1 by default) (input).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation quantities desired (input; no default allowed).	
	'STATIONNAME'	Substation name (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of BUSES returned by the GET_BUS_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer STATIONS	number of elements in each column of CARRAY that are set by this API. STATIONS is identical to the value of STATIONS returned by the GET_SUBSTATION_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of CARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.29.2. ASTATIONCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the substation data family.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, stations = astationcount(sid, flag)
```

Fortran command syntax:

```
CALL GET_SUBSTATION_COUNT(SID, FLAG, STATIONS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer STATIONS	number of substations in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, STATIONS returned
	IERR = 1	Working case is empty; STATIONS returned as 0
	IERR = 2	Invalid SID value; STATIONS returned as 0
	IERR = 3	Invalid FLAG value; STATIONS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.29.3. ASTATIONINT

Use this API to return an array of integer values for subsystem substations.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, iarray = astationint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SUBSTATION_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,STATIONS,
 IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).
Character STRING (NSTR)	NSTR of the following strings indicating the substation quantities desired (input; no default allowed).
	'NUMBER' Substation number
	'NODES' Number of nodes in the substation
	'PARENTS' Number of parent buses in the substation
	'SECTIONS' Number of bus sections in the substation
	'SWDEVS' Number of substation switching devices in the substation
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of STATIONS returned by the GET_SUBSTATION_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer STATIONS	number of elements in each column of IARRAY that are set by this API. STATIONS is identical to the value of STATIONS returned by the GET_SUBSTATION_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Working case is empty
	IERR = 2 Invalid SID value
	IERR = 3 Invalid FLAG value
	IERR = 4 Invalid NSTR value
	IERR = 5 DIM, and hence the size of IARRAY, is not large enough

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.29.4. ASTATIONREAL

Use this API to return an array of real values for subsystem substations.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, rarray = astationreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_SUBSTATION_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,STATIONS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the substation quantities desired (input; no default allowed). 'LATITUDE' Substation latitude 'LONGITUDE' Substation longitude 'RGROUNDING' Substation grounding dc resistance
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of STATIONS returned by the GET_SUBSTATION_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer STATIONS	number of elements in each column of RARRAY that are set by this API. STATIONS is identical to the value of STATIONS returned by the GET_SUBSTATION_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output). IERR = 0 No error IERR = 1 Working case is empty IERR = 2 Invalid SID value IERR = 3 Invalid FLAG value IERR = 4 Invalid NSTR value IERR = 5 DIM, and hence the size of RARRAY, is not large enough IERR = 6 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.29.5. ASTATIONTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the substation data family (aStationInt, aStationReal and aStationChar).

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, types = astationtypes(string)
```

Fortran command syntax:

```
CALL GET_SUBSTATION_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation quantities desired (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the fol-lowing characters indicating the data type of the data associated with the cor-responding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the substation data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.30. Substation Node Data

The substation node data family of subsystem data retrieval APIs returns arrays of data for substation nodes in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for substations with a least one subsystem bus, or to restrict the array entries to data for substations with all of their buses in the subsystem.

Data for subsystem substation nodes is returned from all APIs in the substation node data family in ascending substation number order, and in ascending node number order for each substation.

13.30.1. ANODECHAR

Use this API to return an array of character values for subsystem substation nodes.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, carray = anodechar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_NODE_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,NODES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input).	
Integer FLAG	flag indicating which subsystem substations to include (1 by default) (input).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation quantities desired (input; no default allowed).	
	'STATIONNAME'	Substation name (40 characters).
	'NODENAME'	Node name (40 characters).
	'BUSNAME'	Name of the bus to which the node is connected (12 characters).
	'BUSEXNAME'	Extended bus name of the bus to which the node is connected (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of NODES returned by the GET_NODE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements	
Integer NODES	number of elements in each column of CARRAY that are set by this API. NODES is identical to the value of NODES returned by the GET_NODES_COUNT API when it is called with the same values specified for SID and FLAG (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.30.2. ANODECOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the substation node data family.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, nodes = anodecount(sid, flag)
```

Fortran command syntax:

```
CALL GET_NODE_COUNT(SID, FLAG, NODES, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer NODES	number of substation nodes in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, NODES returned
	IERR = 1	Working case is empty; NODES returned as 0
	IERR = 2	Invalid SID value; NODES returned as 0
	IERR = 3	Invalid FLAG value; NODES returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.30.3. ANODEINT

Use this API to return an array of integer values for subsystem substation nodes.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, iarray = anodeint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_NODE_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,NODES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation node quantities desired (input; no default allowed).	
	'STATION'	Substation number
	'NODE'	Node number
	'BUS'	Number of the bus to which the node is connected
	'SECTION'	Section number of the bus to which the node is connected
	'STATUS'	Node status
	'TERMINALS'	Number of terminals connected to the node
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of NODES returned by the GET_NODE_COUNT API when it is called with the same values speci-fied for SID and FLAG (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer NODES	number of elements in each column of IARRAY that are set by this API. NODES is identical to the value of NODES returned by the GET_NODE_COUNT API when it is called with the same values speci-fied for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	DIM, and hence the size of IARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.30.4. ANODEREAL

Use this API to return an array of real values for subsystem substation nodes.

This API routine was first introduced in release 34.4.0.

Python command syntax:

```
ierr, rarray = anodereal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_NODE_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,NODES,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation node quantities desired (input; no default allowed).	
	'PU'	Node voltage magnitude in per unit
	'KV'	Node voltage magnitude in kV
	'ANGLE'	Node voltage angle in radians
	'ANGLED'	Node voltage angle in degrees
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of NODES returned by the GET_NODE_COUNT API when it is called with the same values speci-fied for SID and FLAG (input; no default allowed).	
real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer NODES	number of elements in each column of IARRAY that are set by this API. NODES is identical to the value of NODES returned by the GET_NODE_COUNT API when it is called with the same values speci-fied for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of RARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.30.5. ANODETYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the substation node data family (aNodeInt and aNodeChar).

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, types = anodetypes(string)
```

Fortran command syntax:

```
CALL GET_NODE_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed); no default allowed	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the substation node data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the substation data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.31. Substation Switching Device Data

The substation switching device data family of subsystem data retrieval APIs returns arrays of data for substation switching devices in a specified subsystem. One of the input items to each API allows the user to select either to retrieve data for substations with a least one subsystem bus, or to restrict the array entries to data for substations with all of their buses in the subsystem.

Data for subsystem substation switching devices is returned from all APIs in the substation switching device data family in ascending substation number order. For the nodes connected by each switching device, the node with the smaller node number is returned as the "from" node, and the node with the larger node number is returned as the "to" node. Station switching devices for a substation are returned in ascending "from" node number order, and in ascending "to" node number order for each "from" node.

13.31.1. ASTASWDEVCHAR

Use this API to return an array of character values for subsystem substation switching devices.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, carray = astaswdevchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_STASWDEV_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,SWDEVS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation switching device quantities desired (input; no default allowed).	
	'STATIONNAME'	Substation name (40 characters).
	'FROMNODENAME'	From node name (40 characters).
	'TONODENAME'	To node name (40 characters).
	'ID'	Switching device identifier (2 characters).
	'SWITCHDEVNAME'	Switching device name (40 characters).
	'SWDRSETNAME'	Switching device rating set name (40 characters).
	'BUSNAME'	Name of the bus to which the nodes are connected (12 characters).
	'BUSEXNAME'	Extended bus name of the bus to which the nodes are connected (18 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of SWDEVS returned by the GET_S-	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	taSwDev_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer SWDEVS	number of elements in each column of CARRAY that are set by this API. SWDEVS is identical to the value of SWDEVS returned by the GET_StaSwDev_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of CARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.31.2. ASTASWDEVCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the substation switching device data family.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, swdevs = astaswdevcount(sid, flag)
```

Fortran command syntax:

```
CALL GET_STASWDEV_COUNT(SID, FLAG, SWDEVS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer SWDEVS	number of substation switching devices in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, SWDEVS returned
	IERR = 1	Working case is empty; SWDEVS returned as 0
	IERR = 2	Invalid SID value; SWDEVS returned as 0
	IERR = 3	Invalid FLAG value; SWDEVS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.31.3. ASTASWDEVCPLX

Use this API to return an array of complex values for subsystem substation switching devices.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, xarray = astaswdevcplx(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_STASWDEV_COMPLEX(SID,FLAG,NSTR,STRING,DIM,XARRAY,SWDEVS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation switching device quantities desired For the following, values are returned in MW and Mvar For the following, values are returned in units determined by the power option setting (input; no default allowed).	
	'PQ'	abs(P+jQ) at the from node end (in MW and Mvar)
	'O_PQ'	abs(P+jQ) at the from node end (units determined by power output options)
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of SWDEVS returned by the GET_StaSwDev_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer SWDEVS	number of elements in each column of XARRAY that are set by this API. SWDEVS is identical to the value of SWDEVS returned by the GET_StaSwDev_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.31.4. ASTASWDEVINT

Use this API to return an array of integer values for subsystem substation switching devices.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, iarray = astaswdevint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_STASWDEV_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,SWDEVS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the substation switching device quantities desired (input; no default allowed).
	'STATION' Substation number
	'FROMNODE' From node number
	'TONODE' To node number
	'BUS' Number of the bus to which the nodes are connected
	'FROMSECTION' Section number of the bus to which the from node is connected
	'TOSECTION' Section number of the bus to which the to node is connected
	'TYPE' Switching device type (1 through 3)
	'STATUS' Switching device status
	'NSTATUS' Switching device normal status
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of SWDEVS returned by the GET_StaSwDev_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer SWDEVS	number of elements in each column of IARRAY that are set by this API. SWDEVS is identical to the value of SWDEVS returned by the GET_StaSwDev_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Working case is empty

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of IARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.31.5. ASTASWDEVREAL

Use this API to return an array of real values for subsystem substation switching devices.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, rarray = astaswdevreal(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_STASWDEV_REAL(SID,FLAG,NSTR,STRING,DIM,RARRAY,SWDEVS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).
Integer FLAG	flag indicating which subsystem substations to include (input; 1 by default).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).
Character STRING (NSTR)	NSTR of the following strings indicating the substation switching device quantities desired For the following, values are returned in MW, Mvar, or MVA For the following, values are returned in units determined by the power option setting (input; no default allowed).

'X'	Switching device reactance
'RATE1'	Rating from rating set 1
'RATE2'	Rating from rating set 2
'RATE3'	Rating from rating set 3
'RATE4'	switching device forth rating
'RATE5'	switching device fifth rating
'RATE6'	switching device sixth rating
'RATE7'	switching device seventh rating
'RATE8'	switching device eighth rating
'RATE9'	switching device ninth rating
'RATE10'	switching device tenth rating
'RATE11'	switching device eleventh rating
'RATE12'	switching device twelfth rating
'AMPRATE1'	switching device first rating in AMP
'AMPRATE2'	switching device second rating in AMP
'AMPRATE3'	switching device third rating in AMP
'AMPRATE4'	switching device forth rating in AMP
'AMPRATE5'	switching device fifth rating in AMP
'AMPRATE6'	switching device sixth rating in AMP
'AMPRATE7'	switching device seventh rating in AMP
'AMPRATE8'	switching device eighth rating in AMP
'AMPRATE9'	switching device ninth rating in AMP
'AMPRATE10'	switching device tenth rating in AMP

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'AMPRATE11'	switching device eleventh rating in AMP
	'AMPRATE12'	switching device twelfth rating in AMP
	'AMPS'	Branch current loading in amps
	'PUCUR'	Branch current loading in per unit
	'PCTRATE1'	Percent current loading of rating set 1
	'PCTRATE2'	Percent current loading of rating set 2
	'PCTRATE3'	Percent current loading of rating set 3
	'PCTRATE4'	percent current loading of RATE4
	'PCTRATE5'	percent current loading of RATE5
	'PCTRATE6'	percent current loading of RATE6
	'PCTRATE7'	percent current loading of RATE7
	'PCTRATE8'	percent current loading of RATE8
	'PCTRATE9'	percent current loading of RATE9
	'PCTRATE10'	percent current loading of RATE10
	'PCTRATE11'	percent current loading of RATE11
	'PCTRATE12'	percent current loading of RATE12
	'PCTMVARATE1'	Percent MVA loading of rating set 1
	'PCTMVARATE2'	Percent MVA loading of rating set 2
	'PCTMVARATE3'	Percent MVA loading of rating set 3
	'PCTMVARATE4'	percent MVA loading of RATE4
	'PCTMVARATE5'	percent MVA loading of RATE5
	'PCTMVARATE6'	percent MVA loading of RATE6
	'PCTMVARATE7'	percent MVA loading of RATE7
	'PCTMVARATE8'	percent MVA loading of RATE8
	'PCTMVARATE9'	percent MVA loading of RATE9
	'PCTMVARATE10'	percent MVA loading of RATE10
	'PCTMVARATE11'	percent MVA loading of RATE11
	'PCTMVARATE12'	percent MVA loading of RATE12
	'P'	Active power flow at the from node end, in MW
	'Q'	Reactive power flow at the from node end, in Mvar
	'MVA'	abs(P+jQ) at the from node end, in MVA
	'O_P'	Active power flow at the from node end (units determined by power output options)
	'O_Q'	Reactive power flow at the from node end (units determined by power output options)
	'O_MVA'	abs(P+jQ) at the from node end (units determined by power output options)
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of SWDEVS returned by the GET_StaSwDev_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer SWDEVS	number of elements in each column of RARRAY that are set by this API. SWDEVS is identical to the value of SWDEVS returned by the GET_StaSwDev_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of RARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.31.6. ASTASWDEVTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the substation switching device data family (aStaSwDevInt, aStaSwDevReal, aStaSwDevCplx and aStaSwDevChar).

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, types = astaswdevtypes(string)
```

Fortran command syntax:

```
CALL GET_STASWDEV_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (no default allowed) (input).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the substation node data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the substation switching device data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.32. Substation Terminal Data

The substation terminal data family of subsystem data retrieval APIs returns arrays of data for substation terminals in a specified subsystem. One of the input items to each API allows the user to elect either to retrieve data for substations with a least one subsystem bus, or to restrict the array entries to data for substations with all of their buses in the subsystem.

Data for subsystem substation terminals is returned from all APIs in the substation terminal data family in ascending substation number order, and in ascending bus number order for each substation.

13.32.1. ATERMINALCHAR

Use this API to return an array of integer values for subsystem substation terminals.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, carray = aterminalchar(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_TERMINAL_CHAR(SID,FLAG,NSTR,STRING,DIM,CARRAY,TERMINALS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation switching device quantities desired (input; no default allowed).	
	'STATIONNAME'	Substation name (40 characters).
	'NODENAME'	Name of the node to which the terminal is connected (40 characters).
	'BUSNAME'	Name of the bus to which the terminal's node is connected (12 characters).
	'BUSEXNAME'	Extended bus name of the bus to which the terminal's node is connected (18 characters).
	'OTHER1NAME'	Name of another bus connected to this device; returned as blanks except for two-terminal branches and three-winding transformers (12 characters).
	'OTHER1EXNAME'	Extended bus name of another bus connected to this device; returned as blanks except for two-terminal branches and three-winding transformers (18 characters).
	'OTHER2NAME'	Name of another bus connected to this device; returned as blanks except for three-winding transformers (12 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'OTHER2EXNAME'	Extended bus name of another bus connected to this device; returned as blanks except for three-winding transformers (18 characters).
	'ID'	Machine, load, fixed shunt, or circuit identifier (2 characters).
	'TERMINALTYPE'	Flag indicatinm the type of terminal "L" for Load, "M" for Machine, "F" for Fixed Shunt, "S" for Switched Shunt, "I" for induction machine, "2" for AC Line or Two-Winding Transformer, "3" for Three-Winding Transformer, "A" for FACTs, "V" for VSC DC line, "D" for two terminal DC line and "T" for multiple terminal DC line (1 characters).
	'BRANCHNAME'	Name of branch, transformer, FACTs, DC line, VSC DC line, mutliple terminal DC line (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of TERMINALS returned by the GET_TERMINAL_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer TERMINALS	number of elements in each column of CARRAY that are set by this API. TERMINALS is identical to the value of TERMINALS returned by the GET_TERMINAL_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid FLAG value
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.32.2. ATERMINALCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the substation terminal data family.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, terminals = aterminalcount(sid, flag)
```

Fortran command syntax:

```
CALL GET_TERMINAL_COUNT(SID, FLAG, TERMINALS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).	
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).	
Integer TERMINALS	number of substation terminals in the subsystem indicated by SID that meet the editing criteria indicated by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error, TERMINALS returned
	IERR = 1	Working case is empty; TERMINALS returned as 0
	IERR = 2	Invalid SID value; TERMINALS returned as 0
	IERR = 3	Invalid FLAG value; TERMINALS returned as 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.32.3. ATERMINALINT

Use this API to return an array of integer values for subsystem substation terminals.

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, iarray = aterminalint(sid, flag, string)
```

Fortran command syntax:

```
CALL GET_TERMINAL_INTEGER(SID,FLAG,NSTR,STRING,DIM,IARRAY,TERMINALS, IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input; -1).
Integer FLAG	a flag indicating which subsystem substations to include (input; 1 by default).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the substation switching device quantities desired (input; no default allowed).

'STATION'	Substation number
'NODE'	Number of the node to which the terminal is connected
'BUS'	Number of the bus to which the terminal's node is connected
'SECTION'	Section number of the bus to which the terminal's node is connected
'OTHER1BUS'	Number of another bus connected to this device; returned as 0 except for two-terminal branches and three-winding transformers
'OTHER1SECTION'	Section number of another bus connected to this device; returned as 0 except for two-terminal branches and three-winding transformers
'OTHER2BUS'	Number of another bus connected to this device; returned as 0 except for three-winding transformers
'OTHER2SECTION'	Section number of another bus connected to this device; returned as 0 except for three-winding transformers

Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of TERMINALS returned by the GET_TERMINAL_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer TERMINALS	number of elements in each column of IARRAY that are set by this API. TERMINALS is identical to the value of TERMINALS returned by the GET_TERMINAL_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid SID value
IERR = 3	Invalid FLAG value
IERR = 4	Invalid NSTR value
IERR = 5	DIM, and hence the size of IARRAY, is not large enough
IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.32.4. ATERMINALTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the substation terminal data family (aTerminalInt and aTerminalChar).

This API routine was first introduced in release 34.1.0.

Python command syntax:

```
ierr, types = aterminaltypes(string)
```

Fortran command syntax:

```
CALL GET_TERMINAL_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the substation switching device quantities desired (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the substation terminal data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.33. Zero Sequence Mutuals Data

13.33.1. AZMUTCHAR

Use this API to return an array of character values for subsystem zero sequence mutual branches.

This API routine was first introduced in release 33.12.0 and 34.0.2.

Python command syntax:

```
ierr, carray = azmutchar(sid, owner, ties, flag, brns, string)
```

Fortran command syntax:

```
CALL
 GET_ZMUT_CHAR(SID,OWNER,TIES,FLAG,BRNS,NSTR,STRING,DIM,CARRAY,MUTUALS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for both mutually coupled branches must be in-service
	FLAG = 2	for all mutually coupled branches
Integer BRNS	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	BRNS = 1	for at least one mutually coupled branch in subsystem
	BRNS = 2	for both mutually coupled branches in subsystem

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the branch zero sequence mutual quantities desired (input; no default allowed).
	'BR1_FROMNAME' Bus name of Branch1 From Bus (12 characters).
	'BR1_FROMEXNAME' Extended Bus name of Branch1 From Bus (18 characters).
	'BR1_TONAME' Bus name of Branch1 To Bus (12 characters).
	'BR1_TOEXNAME' Extended Bus name of Branch1 To Bus (18 characters).
	'BR1_ID' Branch1 Circuit identifier (2 characters).
	'BR2_FROMNAME' Bus name of Branch2 From Bus (12 characters).
	'BR2_FROMEXNAME' Extended Bus name of Branch2 From Bus (18 characters).
	'BR2_TONAME' Bus name of Branch2 To Bus (12 characters).
	'BR2_TOEXNAME' Extended Bus name of Branch2 To Bus (18 characters).
	'BR2_ID' Branch2 Circuit identifier (2 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (input; no default allowed).
Character CARRAY (NSTR*DIM)	array of at least NSTR columns, each of which contains DIM elements (output).
Integer MUTUALS	number of elements in each column of RARRAY that are set by this API. MUTUALS is identical to the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (output).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Working case is empty
	IERR = 2 Invalid SID value
	IERR = 3 Invalid OWNER value
	IERR = 4 Invalid TIES value
	IERR = 5 Invalid FLAG value
	IERR = 6 Invalid BRNS value
	IERR = 7 Invalid NSTR value
	IERR = 8 DIM, and hence the size of CARRAY, is not large enough
	IERR = 9 Invalid STRING value
	IERR = 10 Sequence data not in case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.33.2. AZMUTCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the branch zero sequence mutual data family.

This API routine was first introduced in release 33.12.0 and 34.0.2.

Python command syntax:

```
ierr, mutuals = azmutcount(sid, owner, ties, flag, brns)
```

Fortran command syntax:

```
CALL GET_ZMUT_COUNT(SID,OWNER,TIES,FLAG,BRNS,MUTUALS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for both mutually coupled branches must be in-service
	FLAG = 2	for all mutually coupled branches
Integer BRNS	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	BRNS = 1	for at least one mutually coupled branch in subsystem
	BRNS = 2	for both mutually coupled branches in subsystem
Integer MUTUALS	number of array entries required for the subsystem indicated by SID and OWNER that meet the editing criteria indicated by TIES, FLAG and BRNS. (output).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	No error, MUTUALS returned
IERR = 1	Working case is empty; MUTUALS returned as 0
IERR = 2	Invalid SID value; MUTUALS returned as 0
IERR = 3	Invalid OWNER value; MUTUALS returned as 0
IERR = 4	Invalid TIES value; MUTUALS returned as 0
IERR = 5	Invalid FLAG value; MUTUALS returned as 0
IERR = 6	Invalid BRNS value; MUTUALS returned as 0
IERR = 10	Sequence data not in case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.33.3. AZMUTCPLX

Use this API to return an array of complex values for subsystem zero sequence mutual branches.

This API routine was first introduced in release 33.12.0 and 34.0.2.

Python command syntax:

```
ierr, xarray = azmutcplx(sid, owner, ties, flag, brns, string)
```

Fortran command syntax:

```
CALL
 GET_ZMUT_COMPLEX(SID,OWNER,TIES,FLAG,BRNS,NSTR,STRING,DIM,XARRAY,MUTUALS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for both mutually coupled branches must be in-service
	FLAG = 2	for all mutually coupled branches
Integer BRNS	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	BRNS = 1	for at least one mutually coupled branch in subsystem
	BRNS = 2	for both mutually coupled branches in subsystem
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the branch zero sequence mutual quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'ZM'	PU Mutual coupling impedance
Integer DIM	number of elements in each column of the array XARRAY. DIM must be as least as large as the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MUTUALS	number of elements in each column of RARRAY that are set by this API. MUTUALS is identical to the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid BRNS value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of XARRAY, is not large enough
	IERR = 9	Invalid STRING value
	IERR = 10	Sequence data not in case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.33.4. AZMUTINT

Use this API to return an array of integer values for subsystem zero sequence mutual branches.

This API routine was first introduced in release 33.12.0 and 34.0.2.

Python command syntax:

```
ierr, iarray = azmutint(sid, owner, ties, flag, brns, string)
```

Fortran command syntax:

```
CALL
 GET_ZMUT_INTEGER(SID,OWNER,TIES,FLAG,BRNS,NSTR,STRING,DIM,IARRAY,MUTUALS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for both mutually coupled branches must be in-service
	FLAG = 2	for all mutually coupled branches
Integer BRNS	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	BRNS = 1	for at least one mutually coupled branch in subsystem
	BRNS = 2	for both mutually coupled branches in subsystem
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the branch zero sequence mutual quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'BR1_FROMNUMBER'	Bus number of Branch1 From Bus
	'BR1_TONUMBER'	Bus number of Branch1 To Bus
	'BR2_FROMNUMBER'	Bus number of Branch2 From Bus
	'BR2_TONUMBER'	Bus number of Branch2 To Bus
	'BR1_FROMSECTION'	Section number of Branch1 From Bus
	'BR1_TOSECTION'	Section number of Branch1 To Bus
	'BR2_FROMSECTION'	Section number of Branch2 From Bus
	'BR2_TOSECTION'	Section number of Branch2 To Bus
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MUTUALS	number of elements in each column of RARRAY that are set by this API. MUTUALS is identical to the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid BRNS value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of IARRAY, is not large enough
	IERR = 9	Invalid STRING value
	IERR = 10	Sequence data not in case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.33.5. AZMUTREAL

Use this API to return an array of real values for subsystem zero sequence mutual branches.

This API routine was first introduced in release 33.12.0 and 34.0.2.

Python command syntax:

```
ierr, rarray = azmutreal(sid, owner, ties, flag, brns, string)
```

Fortran command syntax:

```
CALL
 GET_ZMUT_REAL(SID,OWNER,TIES,FLAG,BRNS,NSTR,STRING,DIM,RARRAY,MUTUALS,IERR)
```

Where:

Integer SID	Defines the bus subsystem to be used (input-1).	
	SID < 0	to instruct the API to assume a subsystem containing all buses in the working case.
	SID >= 0	valid bus subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined.
Integer OWNER	a flag indicating owner usage if ownership is a subsystem selection criterion (ignored if SID is negative) (input; 1 by default).	
	OWNER = 1	to use bus ownership.
	OWNER = 2	to use branch ownership.
Integer TIES	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	TIES = 1	for interior subsystem branches only.
	TIES = 2	for subsystem tie branches only.
	TIES = 3	for both interior subsystem branches and tie branches.
Integer FLAG	a flag indicating which subsystem branches to include (input; 1 by default).	
	FLAG = 1	for both mutually coupled branches must be in-service
	FLAG = 2	for all mutually coupled branches
Integer BRNS	a flag indicating which subsystem branches to include (ignored if SID is negative) (input; 1 by default).	
	BRNS = 1	for at least one mutually coupled branch in subsystem
	BRNS = 2	for both mutually coupled branches in subsystem
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the branch zero sequence mutual quantities desired (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'BR1_LOC_START'	Branch1 mutual coupling start location
	'BR1_LOC_END'	Branch1 mutual coupling end location
	'BR2_LOC_START'	Branch2 mutual coupling start location
	'BR2_LOC_END'	Branch2 mutual coupling end location
Integer DIM	number of elements in each column of the array RARRAY. DIM must be as least as large as the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (input; no default allowed).	
Real RARRAY (NSTR*DIM)	array of at least NSTR columns, each of which contains DIM elements (output).	
Integer MUTUALS	number of elements in each column of RARRAY that are set by this API. MUTUALS is identical to the value of MUTUALS returned by the GET_ZMUT_COUNT API when it is called with the same values specified for SID, OWNER, TIES, FLAG and BRNS. (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid SID value
	IERR = 3	Invalid OWNER value
	IERR = 4	Invalid TIES value
	IERR = 5	Invalid FLAG value
	IERR = 6	Invalid BRNS value
	IERR = 7	Invalid NSTR value
	IERR = 8	DIM, and hence the size of RARRAY, is not large enough
	IERR = 9	Invalid STRING value
	IERR = 10	Sequence data not in case

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

13.33.6. AZMUTTYPES

Use this API to return an array of character values indicating the data types corresponding to a set of specified STRING values that are valid input values to any of the data retrieval routines of the branch zero sequence mutual data family (aZmutInt, aZmutReal, aZmutCplx and aZmutChar).

This API routine was first introduced in release 33.12.0 and 34.0.2.

Python command syntax:

```
ierr, types = azmuttypes(string)
```

Fortran command syntax:

```
CALL GET_ZMUT_TYPES(NSTR,STRING,TYPES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR strings that are valid STRING values for any of the data retrieval routines of the branch data family (input; no default allowed).	
Character*1 TYPES (NSTR)	each of which contains one of the following characters indicating the data type of the data associated with the corresponding element of STRING (output).	
	'I'	Integer.
	'R'	Real.
	'X'	Complex.
	'C'	Character.
	'?'	Invalid STRING value.
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = n	Position of the first element in STRING that contains a value that is not a valid STRING value for any of the data retrieval routines in the branch data family (1 < n < NSTR)
	IERR = 99	Invalid NSTR value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 14
Graphical Analysis Output

This chapter defines API routines that interact with displays and files associated with the graphical analysis capabilities of the PSS®E GUI. They will elicit no action if a GUI is not present when invoked.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

14.1. POLY_PRINT

Use this API to print the results of activity POLY from a .pol results file.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_POLY_PRINT NPLOT PLOTARRAY(1)..PLOTARRAY(NPLOT) BOUNDVAL PRTPARAMS
 RESFILE
```

Python command syntax:

```
poly_print(nplot, plotarray, boundval, prtparams, resfile)
```

Fortran command syntax:

```
CALL POLY_PRINT(NPLOT, PLOTARRAY, BOUNDVAL, PRTPARAMS, RESFILE)
```

Where:

Integer NPLOT	number of plots (input).
Integer PLOTARRAY (NPLOT)	an array of the numbers of plots, selected for printing. The base case is marked by 0, then there are all the contingencies, beginning from 1, and the last number means all the cases together. The array consists of the set of these numbers. The size of the array is equal to NPLOT (input).
Real BOUNDVAL	boundary value for the plot area. It is equal to maximum opposing system delta interchange used for calculations (input).
Integer PRTPARAMS	print (or not) the set of parameters used for calculations (input). PRTPARAMS = 0 no printing of parameters. PRTPARAMS = 1 the set of parameters will be printed.
Character*260 RESFILE	name of the existing POLY results file (.pol) file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

14.2. PV_PRINT

Use this API to print the results of PV analysis from the .pv results file

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_PV_PRINT RESFILE RESTYPE NCONTING CONTINGARRAY(1)..
 CONTINGARRAY(NCONTING) NOUTPUTS OUTPUTSARRAY(1).. OUTPUTSARRAY(NOUTPUTS)
 INTERFACE ENFORCESCALE USELINESTYLES NUMBERLINES PRTPARAMS
```

Python command syntax:

```
pv_print(resfile, restype, nconting, contingarray, noutputs, outputsarray,
 interface, enforcescale, uselinestyles, numberlines, prtparams)
```

Fortran command syntax:

```
CALL PV_PRINT(RESFILE, RESTYPE, NCONTING, CONTINGARRAY, NOUTPUTS,
 OUTPUTSARRAY, INTERFACE, ENFORCESCALE, USELINESTYLES, NUMBERLINES,
 PRTPARAMS)
```

Where:

Character*260 RESFILE	name of the existing .pv file (input).
Integer RESTYPE	type of the results (input). RESTYPE = 0 bus voltage RESTYPE = 1 generator MW RESTYPE = 2 branch MVA flow RESTYPE = 3 interface MW flow RESTYPE = 4 bus voltage against interface flow
Integer NCONTING	number of specified contingencies (input).
Integer CONTINGARRAY (NCONTING)	an array of the numbers of specified contingencies. The base case is marked by 0. The size of the array is equal to NCONTING (input).
Integer NOUTPUTS	number of specified outputs (input).
Integer OUTPUTSARRAY (NOUTPUTS)	an array of the numbers of specified outputs. The count starts from 0. The size of the array is equal to NOUTPUTS (input).
Integer INTERFACE	number of specified interface. Valid only in case of bus voltage against interface flow (RESTYPE = 4) (input).
Integer ENFORCESCALE	allows to print the results with the different scales (input). ENFORCESCALE = 0 the scales will be not changed according with the results; this allows to print the results from different files in the same scales. ENFORCESCALE = 1 the maximum and minimum values for the used scales will be taken from the file of results, and the printing will be done according with these values.
Integer USELINESTYLES	use (or not) different line styles for plotting (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	USELINESTYLES = 0	all the lines will be solid
	USELINESTYLES = 1	five different (standard) styles of lines will be used: Solid, Dash-Dash, Dot-Dot, Dash-Dot, Dash-Dot-Dot.
Integer NUMBERLINES	put the numbers (or not) on the graphs (input).	
	NUMBERLINES = 0	no numbering of the graphs.
	NUMBERLINES = 1	each graph will have its number for better recognition
Integer PRTPARAMS	print (or not) the set of parameters used for calculations (input).	
	PRTPARAMS = 0	no printing of parameters
	PRTPARAMS = 1	the set of parameters will be printed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

14.3. QV_PRINT

Use this API to print the results of QV analysis from a .qv resultsfile.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_QV_PRINT RESFILE NCONTING CONTINGARRAY(1).. CONTINGARRAY(NCONTING)
 ENFORCESCALE USELINESTYLES NUMBERLINES PRTPARAMS
```

Python command syntax:

```
qv_print(resfile, nconting, contingarray, enforcescale, uselinestyles,
 numberlines, prtparams)
```

Fortran command syntax:

```
CALL QV_PRINT(RESFILE, NCONTING, CONTINGARRAY, ENFORCESCALE,
 USELINESTYLES, NUMBERLINES, PRTPARAMS)
```

Where:

Character*260 RESFILE	name of the existing .qv file (input).	
Integer NCONTING	number of specified contingencies (input).	
Integer CONTINGARRAY (NCONTING)	array of the numbers of specified contingencies. The base case is marked by 0. The size of the array is equal to NCONTING (input).	
Integer ENFORCESCALE	allows to print the results with the different scales (input).	
	ENFORCESCALE = 0	the scales will be not changed according with the results; this allows to print the results from different files in the same scales
	ENFORCESCALE = 1	the maximum and minimum values for the used scales will be taken from the file of results, and the printing will be done according with these values
Integer USELINESTYLES	use (or not) different line styles for plotting (input).	
	USELINESTYLES = 0	all the lines will be solid
	USELINESTYLES = 1	five different (standard) styles of lines will be used: Solid, Dash-Dash, Dot-Dot, Dash-Dot, Dash-Dot-Dot
Integer NUMBERLINES	put the numbers (or not) on the graphs (input).	
	NUMBERLINES = 0	no numbering of the graphs
	NUMBERLINES = 1	each graph will have its number for better recognition
Integer PRTPARAMS	print (or not) the set of parameters used for calculations (input).	
	PRTPARAMS = 0	no printing of parameters
	PRTPARAMS = 1	the set of parameters will be printed

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

14.4. SETFULLVIEWGRAPHAREA

Use this API to specify the parameters of the scales used for plotting the results of PV and QV analysis.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETFULLVIEWGRAPHAREA AREA VAXISSHOW VAXISSTYLE VAXISCLR(1)..
 VAXISCLR(3) VGRIDSHOW VGRIDSTYLE VGRIDCLR(1).. VGRIDCLR(3) HAXISSHOW
 HAXISSTYLE HAXISCLR(1).. HAXISCLR(3) HGRIDSHOW HGRIDSTYLE HGRIDCLR(1)..
 HGRIDCLR(3) BORDERSHOW BORDERSTYLE BORDERCLR(1).. BORDERCLR(3) SETAREACLR
 AREACLR(1).. AREACLR(3)
```

Python command syntax:

```
setfullviewgrapharea(area, vaxisshow, vaxisstyle, vaxisclr, vgridshow,
 vgridstyle, vgridclr, haxisshow, haxisstyle, haxisclr, hgridshow,
 hgridstyle, hgridclr, bordershow, borderstyle, borderclr, setareaclr,
 areaclr)
```

Fortran command syntax:

```
CALL SETFULLVIEWGRAPHAREA(AREA, VAXISSHOW, VAXISSTYLE, VAXISCLR,
 VGRIDSHOW, VGRIDSTYLE, VGRIDCLR, HAXISSHOW, HAXISSTYLE, HAXISCLR,
 HGRIDSHOW, HGRIDSTYLE, HGRIDCLR, BORDERSHOW, BORDERSTYLE, BORDERCLR,
 SETAREACLR, AREACLR)
```

Where:

Integer AREA	number of the plotting area (input).	
	AREA = 0	QV analysis, area for Q(V) plot
	AREA = 1	QV analysis, area for mismatch plot
	AREA = 2	PV analysis, area for bus voltage
	AREA = 3	PV analysis, area for generator MW
	AREA = 4	PV analysis, area for branch MVA flow
	AREA = 5	PV analysis, area for interface MW flow
	AREA = 6	PV analysis, area for bus voltage against interface flow.
Integer VAXISSHOW	show (or not) the vertical axis (input).	
Integer VAXISSTYLE	style of the vertical axis. Valid only if VAXISSHOW=1 (input).	
	VAXISSTYLE = 0	solid line
	VAXISSTYLE = 1	dash - dash - dash line
	VAXISSTYLE = 2	dot - dot - dot line
	VAXISSTYLE = 3	dash - dot line
	VAXISSTYLE = 4	dash - dot - dot line
Integer VAXISCLR (3)	array of 3 elements specifying color of the vertical axis. Valid only if VAXISSHOW=1(input).	
	VAXISCLR(1)	Red component; the value between 0 and 255

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VAXISCLR(2)	Green component; the value between 0 and 255
	VAXISCLR(3)	Blue component; the value between 0 and 255
Integer VGRIDSHOW	show (or not) the vertical grid (input).	
Integer VGRIDSTYLE	style of the vertical grid. Valid only if VGRIDSHOW =1 (input).	
	VGRIDSTYLE = 0	solid line
	VGRIDSTYLE = 1	dash - dash - dash line
	VGRIDSTYLE = 2	dot - dot - dot line
	VGRIDSTYLE = 3	dash - dot line
	VGRIDSTYLE = 4	dash - dot - dot line
Integer VGRIDCLR (3)	array of 3 elements specifying color of the vertical grid. Valid only if VGRIDSHOW =1(input).	
	VGRIDCLR(1)	Red component; the value between 0 and 255
	VGRIDCLR(2)	Green component; the value between 0 and 255
	VGRIDCLR(3)	Blue component; the value between 0 and 255
Integer HAXISSHOW	show (or not) the horizontal axis (input).	
Integer HAXISSTYLE	style of the horizontal axis. Valid only if HAXISSHOW=1 (input).	
	HAXISSTYLE = 0	solid line
	HAXISSTYLE = 1	dash - dash - dash line
	HAXISSTYLE = 2	dot - dot - dot line
	HAXISSTYLE = 3	dash - dot line
	HAXISSTYLE = 4	dash - dot - dot line
Integer HAXISCLR (3)	array of 3 elements specifying color of the horizontal axis. Valid only if HAXISSHOW=1(input).	
	HAXISCLR(1)	Red component; the value between 0 and 255
	HAXISCLR(2)	Green component; the value between 0 and 255
	HAXISCLR(3)	Blue component; the value between 0 and 255
Integer HGRIDSHOW	show (or not) the horizontal grid (input).	
Integer HGRIDSTYLE	style of the horizontal grid. Valid only if HGRIDSHOW =1 (input).	
	HGRIDSTYLE = 0	solid line
	HGRIDSTYLE = 1	dash - dash - dash line
	HGRIDSTYLE = 2	dot - dot - dot line
	HGRIDSTYLE = 3	dash - dot line
	HGRIDSTYLE = 4	dash - dot - dot line
Integer HGRIDCLR (3)	array of 3 elements specifying color of the horizontal grid. Valid only if HGRIDSHOW =1(input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	HGRIDCLR(1)	Red component; the value between 0 and 255
	HGRIDCLR(2)	Green component; the value between 0 and 255
	HGRIDCLR(3)	Blue component; the value between 0 and 255
Integer BORDERSHOW	show (or not) the border around the plot (input).	
Integer BORDERSTYLE	style of the border. Valid only if BORDERSHOW =1 (input).	
	BORDERSTYLE = 0	solid line
	BORDERSTYLE = 1	dash - dash - dash line
	BORDERSTYLE = 2	dot - dot - dot line
	BORDERSTYLE = 3	dash - dot line
	BORDERSTYLE = 4	dash - dot - dot line
Integer BORDERCLR (3)	array of 3 elements specifying color of the border. Valid only if BORDERSHOW =1(input).	
	BORDERCLR(1)	Red component; the value between 0 and 255
	BORDERCLR(2)	Green component; the value between 0 and 255
	BORDERCLR(3)	Blue component; the value between 0 and 255
Integer SETAREACLR	set (or not) the back color of the plotting area (input).	
Integer AREACLR (3)	array of 3 elements specifying color of the plotting area. Valid only if SETAREACLR =1(input).	
	AREACLR(1)	Red component; the value between 0 and 255
	AREACLR(2)	Green component; the value between 0 and 255
	AREACLR(3)	Blue component; the value between 0 and 255

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

14.5. SETFULLVIEWSCALE

Use this API to specify the parameters of the scales used for plotting the results of PV and QV analysis.

This API routine was first introduced in release 30.0.0.

Batch command syntax:

```
BAT_SETFULLVIEWSCALE SCALE LTVAL RBVAL ADJUST(1) ADJUST(2) AXISPLACE
 AXISVALUE GRIDCALCTYPE GRIDSTEP GRIDMINLINES SHOWNUM (1).. SHOWNUM(3)
 LTPOS INPOS RBPOS LTPLACES INPLACES RBPLACES
```

Python command syntax:

```
setfullviewscale(scale, ltval, rbval, adjust, axisplace, axisvalue,
 gridcalctype, gridstep, gridminlines, shownum, ltpos, inpos, rbpos,
 ltplaces, inplaces, rbplaces)
```

Fortran command syntax:

```
CALL SETFULLVIEWSCALE(SCALE, LTVAL, RBVAL, ADJUST, AXISPLACE, AXISVALUE,
 GRIDCALCTYPE, GRIDSTEP, GRIDMINLINES, SHOWNUM, LTPOS, INPOS, RBPOS,
 LTPLACES, INPLACES, RBPLACES)
```

Where:

Integer SCALE	number of the scale (input).	
	SCALE = 0	QV analysis, horizontal scale for Q(V) plot
	SCALE = 1	QV analysis, vertical scale for Q(V) plot
	SCALE = 2	QV analysis, vertical scale for mismatch plot
	SCALE = 3	PV analysis, horizontal scale for incremental transfer
	SCALE = 4	PV analysis, horizontal scale for interface MW flow (bus voltage against interface flow)
	SCALE = 5	PV analysis, vertical scale for bus voltage
	SCALE = 6	PV analysis, vertical scale for generator MW
	SCALE = 7	PV analysis, vertical scale for branch MVA flow
	SCALE = 8	PV analysis, vertical scale for interface MW flow
	SCALE = 9	PV analysis, vertical scale for bus voltage (bus voltage against interface flow)
Real LTVAL	left value (for horizontal scale) or top value (for vertical scale) (input).	
Real RBVAL	right value (for horizontal scale) or bottom value (for vertical scale) (input).	
Integer ADJUST (2)	array of 2 elements specifying (input).	
	ADJUST(1)	to adjust (or not) the left (top) value of the scale

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ADJUST(2)	to adjust (or not) the right (bottom) value of the scale
Integer AXISPLACE	place of the axis (input).	
	AXISPLACE = 0	on the left (top) border
	AXISPLACE = 1	the axis place is set in AXISVALUE
	AXISPLACE = 2	on the right (bottom) border
	AXISPLACE = 3	undefined, the exact value will be calculated
Real AXISVALUE	place of the axis. Valid only in case AXISPLACE = 1 (input).	
Integer GRIDCALCTYPE	way the grid is calculated (input).	
	GRIDCALCTYPE = 0	grid step is set in GRIDSTEP
	GRIDCALCTYPE = 1	grid is defined by minimum grid lines set in GRIDMINLINES
Real GRIDSTEP	tep of the grid. Valid only in case GRIDCALCTYPE = 0 (input).	
Integer GRIDMINLINES	Minimum number of the grid lines. Valid only in case GRIDCALCTYPE = 1 (input).	
Integer SHOWNUM (3)	array of 3 elements specifying (input).	
	SHOWNUM(1)	to show (or not) the left (top) value of the scale
	SHOWNUM(2)	to show (or not) all the intermediate values for the grid
	SHOWNUM(3)	to show (or not) the right (bottom) value of the scale
Integer LTPOS	place of the left (top) value relative to the left (top) border (input).	
	LTPOS = 0	to the left of the border (above the border)
	LTPOS = 1	centered to the border
	LTPOS = 2	to the right of the border (below the border)
Integer INPOS	place of the intermediate values relative to the intermediate grid lines (input).	
	INPOS = 0	to the left of the line (above the line)
	INPOS = 1	centered to the line
	INPOS = 2	to the right of the line (below the line)
Integer RBPOS	place of the right (bottom) value relative to the right (bottom) border (input).	
	RBPOS = 0	to the left of the border (above the border)
	RBPOS = 1	centered to the border
	RBPOS = 2	to the right of the border (below the border)
Integer LTPLACES	number of digits to the right of the decimal point for the left (top) value (input).	
Integer INPLACES	number of digits to the right of the decimal point for intermediate values (input).	
Integer RBPLACES	number of digits to the right of the decimal point for the right (bottom) value (input).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 15
Study Project

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.1. CLOSESCENARIOFILE

This API routine is obsolete. It has been replaced by the API routine SCENARIOCLOSE .

Use this API to close the currently open Scenario. If a file in the active group has was specified to "Load Last", this file will be opened prior to the closing of the Scenario.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CLOSESCENARIOFILE
```

Python command syntax:

```
closescenariofile()
```

Fortran command syntax:

```
CALL CLOSESCENARIOFILE()
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.2. NEWSCENARIOFILE

This API routine is obsolete. It has been replaced by the API routine SCENARIONEW .

Use this API to create a new Scenario. At least one group must be added to the Scenario through SCENARIOGROUPADD before files may be added to it. If adding files through file tracking is also desired, a group must become the active group which can be done by calling SCENARIOGROUPOPEN.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_NEWSCENARIOFILE SFILE
```

Python command syntax:

```
newscenariofile(sfile)
```

Fortran command syntax:

```
CALL NEWSCENARIOFILE(SFILE)
```

Where:

Character*260 SFILE	full file path of the new scenario file (input; blank by default. The name of the Scenario will be determined by the file name portion of the full file path. When the default is used no name is given to the Scenario and a full file name will be required upon saving. The root path of the Scenario will be determined by the path portion of the full file path. When the default is used the root path will default to the current working directory).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.3. OPENSCENARIOFILE

This API routine is obsolete. It has been replaced by the API routine SCENARIOOPEN .

Use this API to open a previously created Scenario.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPENSCENARIOFILE SFILE
```

Python command syntax:

```
openscenariofile(sfile)
```

Fortran command syntax:

```
CALL OPENSCENARIOFILE(SFILE)
```

Where:

Character*260 SFILE	full file path of the scenario file to open (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.4. SAVESCENARIOFILE

This API routine is obsolete. It has been replaced by the API routine SCENARIOSAVE .

Use this API to save the currently open Scenario.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SAVESCENARIOFILE SFILE
```

Python command syntax:

```
savescenariofile(sfile)
```

Fortran command syntax:

```
CALL SAVESCENARIOFILE (SFILE)
```

Where:

Character*260 SFILE	full file path to save the scenario file to (input; unchanged by default. When the default is used the Scenario will be saved to the last file-name given in a SCENARIONEW, SCENARIOOPEN, or SCENARIOSAVE operation, if a call to SCENARIONEW with default arguments was the previous call, the Scenario will not be saved due to no previous file-name being available).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.5. SCENARIOCLOSE

This API routine is obsolete. It has been replaced by the API routine stdyprojnew .

Use this API to close the currently open Scenario. If a file in the active group has was specified to "Load Last", this file will be opened prior to the closing of the Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOCLOSE
```

Python command syntax:

```
ierr = scenarioclose()
```

Fortran command syntax:

```
CALL SCENARIOCLOSE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.6. SCENARIOCOPYFILE

This API routine is obsolete.

Use this API to copy a file from one group to another group.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOCOPYFILE SRCGROUP SFILE DSTGROUP
```

Python command syntax:

```
ierr = scenariocopyfile(srcgroup, sfile, dstgroup)
```

Fortran command syntax:

```
CALL SCENARIOCOPYFILE(SRCGROUP, SFILE, DSTGROUP, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SRCGROUP	name of the group containing the file to be copied (input; no default allowed).	
Character*260 SFILE	name of the file to be copied (input; no default allowed).	
Character*72 DSTGROUP	name of the group the file will be copied to (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid file name
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.7. SCENARIOFILEADD

This API routine is obsolete. It has been replaced by the API routine stdyprojinputdatasetadd .

Use this API to add a file to the specified group.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOFILEADD SGROUP SFILE STYPE LOAD SARGS
```

Python command syntax:

```
ierr = scenariofileadd(sgroup, sfile, stype, load, sargs)
```

Fortran command syntax:

```
CALL SCENARIOFILEADD(SGROUP, SFILE, STYPE, LOAD, SARGS, IERR)
```

Where:

Character*72 SGROUP	name of the group to add a file to (input; no default allowed).	
Character*260 SFILE	name of the file to add (input; no default allowed).	
Character*8 STYPE	file type identifier for the specified file (input; blank by default).	
Integer LOAD	file's position in the load order of its group. A load order of 0 specifies that the file is not loaded (input; 0 by default).	
Character*260 SARGS	argument string to be used for the file when automatically opening the file (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid file name
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.8. SCENARIOFILECOUNT

This API routine is obsolete.

Use this API to retrieve the number of files in a specified group in the Scenario.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, nfiles = scenariofilecount(sgroup)
```

Fortran command syntax:

```
CALL SCENARIOFILECOUNT(SGROUP, NFILES, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group to retrieve the file count of input (input; ignored).	
Integer NFILES	number of the files in the specified group (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.9. SCENARIOFILEGETATTRS

This API routine is obsolete.

Use this API to retrieve the attributes of the specified file.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, stype, load, sargs = scenariofilegetattrs(sgroup, sfile)
```

Fortran command syntax:

```
CALL SCENARIOFILEGETATTRS(SGROUP, SFILE, STYPE, LOAD, SARGS, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group that contains the specified file (input; no default allowed).	
Character*260 SFILE	name of the file whose attributes are to be retrieved (input; no default allowed).	
Character*8 STYPE	file type identifier for the specified file (output).	
Integer LOAD	Denotes the file's position in the load order of its group. A load order of 0 specifies that the file is not loaded (output).	
Character*260 SARGS	argument string to be used for the file when automatically opening the file (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.10. SCENARIOFILEGETCOMMENTS

This API routine is obsolete.

Use this API to retrieve the comments for the specified file.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, comments = scenariofilegetcomments(sgroup, sfile)
```

Fortran command syntax:

```
CALL SCENARIOFILEGETCOMMENTS(SGROUP, SFILE, COMMENTS, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group that contains the specified file (input; no default allowed).	
Character*260 SFILE	name of the file whose attributes are to be retrieved (input; no default allowed).	
Character COMMENTS	the comment string for the specified file. Use SCENARIOFILELENGTH-COMMENTS to get required buffer size, if needed. (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.11. SCENARIOFILELENGTHCOMMENTS

This API routine is obsolete.

Use this API to determine the buffer size needed to hold the comments for the specified file.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, length = scenariofilelengthcomments(sgroup, sfile)
```

Fortran command syntax:

```
CALL SCENARIOFILELENGTHCOMMENTS(SGROUP, SFILE, LENGTH, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group that contains the specified file (input; no default allowed).	
Character*260 SFILE	name of the file whose comment length is to be returned (input; no default allowed).	
Integer LENGTH	length of the Scenario comment string (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.12. SCENARIOFILELIST

This API routine is obsolete.

Use this API to retrieve the list of files in the specified group.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, sfiles = scenariofilelist(sgroup)
```

Fortran command syntax:

```
CALL SCENARIOFILELIST(SGROUP, NFILES, SFILES, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group to retrieve the file list of (input; no default allowed).	
Integer NFILES	max number of files to list (input; no default allowed).	
Character*260 SFILES (NFILES)	list of files in the specified group (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.13. SCENARIOFILEREMOVE

This API routine is obsolete. It has been replaced by the API routine stdyprojinputdatasetremove .

Use this API to remove a file from the specified group.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOFILEREMOVE SGROUP SFILE
```

Python command syntax:

```
ierr = scenariofileremove(sgroup, sfile)
```

Fortran command syntax:

```
CALL SCENARIOFILEREMOVE(SGROUP, SFILE, IERR)
```

Where:

Character*72 SGROUP	name of the group to remove a file from (input; no default allowed).	
Character*260 SFILE	name of the file to be removed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid file name
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.14. SCENARIOFILERENAME

This API routine is obsolete.

Use this API to rename one of the files in the Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOFILERENAME  SGROUP SFILE SNEWNAME
```

Python command syntax:

```
ierr = scenariofilerename(sgroup, sfile, snewname)
```

Fortran command syntax:

```
CALL SCENARIOFILERENAME(SGROUP, SFILE, SNEWNAME, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group that contains the specified file (input; no default allowed).	
Character*260 SFILE	name of the file whose name is to be changed (input; no default allowed).	
Character*260 SNEWNAME	new name to be given to the specified file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid file name
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.15. SCENARIOFILESETATTRS

This API routine is obsolete.

Use this API to modify the attributes of the specified file.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOFILESETATTRS SGROUP SFILE STYPE LOAD SARGS
```

Python command syntax:

```
ierr = scenariofilesetattrs(sgroup, sfile, stype, load, sargs)
```

Fortran command syntax:

```
CALL SCENARIOFILESETATTRS(SGROUP, SFILE, STYPE, LOAD, SARGS, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group that contains the specified file (input; no default allowed).	
Character*260 SFILE	name of the file whose attributes are to be modified (input; no default allowed).	
Character*8 STYPE	new file type identifier for the specified file (input; ignored).	
Integer LOAD	file's new position in the load order of its group. A load order of 0 specifies that the file is not loaded (input; ignored).	
Character*260 SARGS	argument string to be used for the file when automatically opening the file (input; ignored).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.16. SCENARIOFILESETCOMMENTS

This API routine is obsolete.

Use this API to define or modify the comments for the specified file.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOFILESETCOMMENTS SGROUP SFILE NCOMMENTBLOCKS COMMENTS APPEND
```

Python command syntax:

```
ierr = scenariofilesetcomments(sgroup, sfile, ncommentblocks, comments,
 append)
```

Fortran command syntax:

```
CALL SCENARIOFILESETCOMMENTS(SGROUP, SFILE, NCOMMENTBLOCKS, COMMENTS,
 APPEND, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group that contains the specified file (input; no default allowed).	
Character*260 SFILE	name of the file whose attributes are to be modified (input; no default allowed).	
Integer NCOMMENTBLOCKS	number of 72 character strings that make up the comment. In python, this number can be set to 1 and the full comment string given as the first comment block (input; 0 by default).	
Character*72 COMMENTS (NCOMMENTBLOCKS)	comment string to be defined for or added to the comments for the specified file (input; unchanged).	
Logical APPEND	flag for appending comments. If TRUE, COMMENTS is appended to the current comments. If FALSE, the current comments are overwritten by COMMENTS (input; FALSE).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.17. SCENARIOGETATTRS

This API routine is obsolete.

Use this API to retrieve the attributes of the Scenario.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, rootpath, startupgroup, creationdate, createdby, changedate,
 changedby, name, version, versionpsse = scenariogetattrs()
```

Fortran command syntax:

```
CALL SCENARIOGETATTRS(ROOTPATH, STARTUPGROUP, CREATIONDATE, CREATEDBY,
 CHANGEDATE, CHANGEDBY, NAME, VERSION, VERSIONPSSE, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*260 ROOTPATH	root path of the Scenario (output).	
Character*72 STARTUPGROUP	name of the startup group (output).	
Character*23 CREATIONDATE	date of creation (output).	
Character*256 CREATEDBY	username of the creator (output).	
Character*23 CHANGEDATE	last date of modification (output).	
Character*256 CHANGEDBY	username of the last modifier (output).	
Character*260 NAME	name of the Scenario (output).	
Character*14 VERSION	version of Scenario the file was saved under (output).	
Character*14 VERSIONPSSE	version of PSSE the file was saved under (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.18. SCENARIOGETCOMMENTS

This API routine is obsolete.

Use this API to retrieve the Scenario comments.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, comments = scenariogetcomments()
```

Fortran command syntax:

```
CALL SCENARIOGETCOMMENTS(COMMENTS, IERR)
```

Note:

This API routine is deprecated.

Where:

Character COMMENTS	the comment string for the Scenario. Use SCENARIOLENGTHCOMMENTS to get required buffer size, if needed. (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.19. SCENARIOGETOPT

This API routine is obsolete.

Use this API to retrieve the Scenario options. Note: Scenario settings are not attached to the Scenario file. They are user preferences that are saved on the user's machine.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, trackopt, uziprootopt, uziplclpopt, uziplclopt, uziprmtpopt,
 uziprmtopt = scenariogetopt()
```

Fortran command syntax:

```
CALL SCENARIOGETOPT(TRACKOPT, UZIPROOTOPT, UZIPLCLPOPT, UZIPLCLOPT,
 UZIPRMTPOPT, UZIPRMTOPT, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer TRACKOPT	setting for file tracking (output).	
	TRACKOPT = 0	tracking is turned off.
	TRACKOPT = 1	the user will be prompted as soon as possible when a new file is being tracked
	TRACKOPT = 2	the user will be prompted about tracked files when the current active group is being closed
	TRACKOPT = 3	tracked files will be added to the current active group immediately.
Integer UZIPROOTOPT	setting for conflict resolution for root files (output).	
	UZIPROOTOPT = 0	skip the file.
	UZIPROOTOPT = 1	prompt the user for action.
	UZIPROOTOPT = 2	overwrite the file on disk.
Integer UZIPLCLPOPT	setting for placement of local files (output).	
	UZIPLCLPOPT = 0	do not unzip.
	UZIPLCLPOPT = 1	unzip to the root path.
	UZIPLCLPOPT = 2	unzip to the same path the file was at before the file was zipped.
	UZIPLCLPOPT = 3	unzip to the same path the file was at before, relative to the root path.
Integer UZIPLCLOPT	setting for conflict resolution for local files (output).	
	UZIPLCLOPT = 0	skip the file.
	UZIPLCLOPT = 1	prompt the user for action.
	UZIPLCLOPT = 2	overwrite the file on disk.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer UZIPRMTPOPT	setting for placement of remote files Note: Cannot choose to unzip remote files by relative path (output).	
	UZIPRMTPOPT = 0	do not unzip.
	UZIPRMTPOPT = 1	unzip to the root path.
	UZIPRMTPOPT = 2	unzip to the same path the file was at before the file was zipped.
Integer UZIPRMTOPT	setting for conflict resolution for remote files (output).	
	UZIPRMTOPT = 0	skip the file.
	UZIPRMTOPT = 1	prompt the user for action.
	UZIPRMTOPT = 2	overwrite the file on disk.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.20. SCENARIOGROUPADD

This API routine is obsolete. It has been replaced by the API routine stdyprojvariantadd .

Use this API to add a group to the Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOGROUPADD SGROUP
```

Python command syntax:

```
ierr = scenariogroupadd(sgroup)
```

Fortran command syntax:

```
CALL SCENARIOGROUPADD(SGROUP, IERR)
```

Where:

Character*72 SGROUP	name of the new group to add to the Scenario (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid group name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.21. SCENARIOGROUPCOUNT

This API routine is obsolete.

Use this API to retrieve the number of groups in the Scenario.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, ngroups = scenariogroupcount()
```

Fortran command syntax:

```
CALL SCENARIOGROUPCOUNT(NGROUPS, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer NGROUPS	number of the group in the Scenario (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.22. SCENARIOGROUPGETCOMMENTS

This API routine is obsolete.

Use this API to retrieve the comments for the specified group.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, comments = scenariogroupgetcomments(sgroup)
```

Fortran command syntax:

```
CALL SCENARIOGROUPGETCOMMENTS(SGROUP, COMMENTS, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	name of the group whose comments are to be returned (input; no default allowed).	
Character COMMENTS	the comment string for the specified group. Use SCENARIOGROUPLENGTHCOMMENTS to get required buffer size, if needed. (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.23. SCENARIOGROUPLENGTHCOMMENTS

This API routine is obsolete.

Use this API to determine the buffer size needed to hold the comments for the specified group.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, length = scenariogrouplengthcomments(sgroup)
```

Fortran command syntax:

```
CALL SCENARIOGROUPLENGTHCOMMENTS(SGROUP, LENGTH, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	name of the group whose comment length is to be returned (input; no default allowed).	
Integer LENGTH	length of the comment string for the specified group (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.24. SCENARIOGROUPLIST

This API routine is obsolete.

Use this API to retrieve the list of groups in the Scenario.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, sgroups = scenariogrouplist()
```

Fortran command syntax:

```
CALL SCENARIOGROUPLIST(NGROUPS, SGROUPS, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer NGROUPS	max number of groups to list (input; no default allowed).	
Character*72 SGROUPS (NGROUPS)	list of groups in the Scenario (output; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.25. SCENARIOGROUPOPEN

This API routine is obsolete. It has been replaced by the API routine stdyprojsetcurrentvariant .

Use this API to add a group to the Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOGROUPOPEN SGROUP
```

Python command syntax:

```
ierr = scenariogroupopen(sgroup)
```

Fortran command syntax:

```
CALL SCENARIOGROUPOPEN(SGROUP, IERR)
```

Where:

Character*72 SGROUP	name of the group to open (input; ignored by default. When default is used the startup group will be opened if one is defined in the Scenario. If the name of a group in the Scenario is given, that group will be opened. If the string is blank or does not match any of the group names in the Scenario no group is opened).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.26. SCENARIOGROUPREMOVE

This API routine is obsolete. It has been replaced by the API routine stdyprojvariantremove .

Use this API to remove a group from the Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOGROUPREMOVE SGROUP
```

Python command syntax:

```
ierr = scenariogroupremove (sgroup)
```

Fortran command syntax:

```
CALL SCENARIOGROUPREMOVE (SGROUP, IERR)
```

Where:

Character*72 SGROUP	name of the group to be removed from the Scenario (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.27. SCENARIOGROUPRENAME

This API routine is obsolete. It has been replaced by the API routine stdyprojvariantrename .

Use this API to rename one of the groups in the Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOGROUPRENAME SGROUP SNEWNAME
```

Python command syntax:

```
ierr = scenariogrouprename(sgroup, snewname)
```

Fortran command syntax:

```
CALL SCENARIOGROUPRENAME(SGROUP, SNEWNAME, IERR)
```

Where:

Character*72 SGROUP	name of the group to be renamed (input; no default allowed).	
Character*72 SNEWNAME	new name to be given to the specified group (input; no default allowed).	
Integer IERR	error code (output).	
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid group name
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.28. SCENARIOGROUPSETCOMMENTS

This API routine is obsolete.

Use this API to define or modify the comments for the specified group.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOGROUPSETCOMMENTS SGROUP NCOMMENTBLOCKS COMMENTS APPEND
```

Python command syntax:

```
ierr = scenariogroupsetcomments(sgroup, ncommentblocks, comments, append)
```

Fortran command syntax:

```
CALL SCENARIOGROUPSETCOMMENTS(SGROUP, NCOMMENTBLOCKS, COMMENTS, APPEND,
 IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	name of the group whose comments are to be modified (input; no default allowed).	
Integer NCOMMENTBLOCKS	number of 72 character strings that make up the comment. In python, this number can be set to 1 and the full comment string given as the first comment block (input; 0 by default).	
Character*72 COMMENTS (NCOMMENTBLOCKS)	comment string to be defined for or added to the comments for the specified group (input; unchanged).	
Logical APPEND	flag for appending comments. If TRUE, COMMENTS is appended to the current comments. If FALSE, the current comments are overwritten by COMMENTS (input; FALSE).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.29. SCENARIOISDIRTY

This API routine is obsolete.

Use this API to determine if there are any unsaved changes in the current Scenario.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
isdirty = scenarioisdirty()
```

Fortran command syntax:

```
CALL SCENARIOISDIRTY(ISDIRTY)
```

Note:

This API routine is deprecated.

Where:

Logical ISDIRTY	TRUE when the Scenario in memory currently contains unsaved changes (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.30. SCENARIOISVALID

This API routine is obsolete.

Use this API to determine if there is a valid Scenario in memory.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
isvalid = scenarioisvalid()
```

Fortran command syntax:

```
CALL SCENARIOISVALID(ISVALID)
```

Note:

This API routine is deprecated.

Where:

Logical ISVALID	TRUE when a valid Scenario is in memory, currently returns FALSE when in Zip Mode (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.31. SCENARIOLENGTHCOMMENTS

This API routine is obsolete.

Use this API to determine the buffer size needed to hold the current Scenario comments.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, length = scenariolengthcomments()
```

Fortran command syntax:

```
CALL SCENARIOLENGTHCOMMENTS(LENGTH, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer LENGTH	length of the Scenario comment string (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.32. SCENARIOMODELGROUP

This API routine is obsolete. It has been replaced by the API routine stdyprojvariantclone .

Use this API to create a new group that is identical to another group.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOMODELGROUP SGROUP SNEWNAME
```

Python command syntax:

```
ierr = scenariomodelgroup(sgroup, snewname)
```

Fortran command syntax:

```
CALL SCENARIOMODELGROUP(SGROUP, SNEWNAME, IERR)
```

Where:

Character*72 SGROUP	name of the group to be copied (input; no default allowed).	
Character*72 SNEWNAME	name of the new group (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid file name
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.33. SCENARIOMOVEFILE

This API routine is obsolete.

Use this API to move a file from one group to another group.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOMOVEFILE SRCGROUP SFILE DSTGROUP
```

Python command syntax:

```
ierr = scenariomovefile(srcgroup, sfile, dstgroup)
```

Fortran command syntax:

```
CALL SCENARIOMOVEFILE(SRCGROUP, SFILE, DSTGROUP, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SRCGROUP	name of the group containing the file to be moved (input; no default allowed).	
Character*260 SFILE	name of the file to be moved (input; no default allowed).	
Character*72 DSTGROUP	name of the group the file will be moved to (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	invalid file name
	IERR = 3	specified group not found
	IERR = 4	specified file not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.34. SCENARIONEW

This API routine is obsolete. It has been replaced by the API routine stdyprojnew .

Use this API to create a new Scenario. At least one group must be added to the Scenario through SCENARIOGROUPADD before files may be added to it. If adding files through file tracking is also desired, a group must become the active group which can be done by calling SCENARIOGROUPOPEN.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIONEW SFILE
```

Python command syntax:

```
ierr = scenarionew(sfile)
```

Fortran command syntax:

```
CALL SCENARIONEW(SFILE, IERR)
```

Where:

Character*260 SFILE	full file path of the new scenario file (input; blank by default. The name of the Scenario will be determined by the file name portion of the full file path. When the default is used no name is given to the Scenario and a full file name will be required upon saving. The root path of the Scenario will be determined by the path portion of the full file path. When the default is used the root path will default to the current working directory).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.35. SCENARIOOPEN

This API routine is obsolete. It has been replaced by the API routine stdyprojopen .

Use this API to open a previously created Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOOPEN SFILE SGROUP
```

Python command syntax:

```
ierr = scenarioopen(sfile, sgroup)
```

Fortran command syntax:

```
CALL SCENARIOOPEN(SFILE, SGROUP, IERR)
```

Where:

Character*260 SFILE	full file path of the scenario file to open (input; no default allowed).	
Character*72 SGROUP	name of the group to open when the scenario is loaded. When the default is used the startup group will be opened if one is defined in the Scenario. If the name of a group in the Scenario is given, that group will be opened. If the string is blank or does not match any of the group names in the Scenario, no group is opened. (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 2	file path is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.36. SCENARIOSAVE

This API routine is obsolete. It has been replaced by the API routine stdyprojrename .

Use this API to save the currently open Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOSAVE SFILE
```

Python command syntax:

```
ierr = scenariosave(sfile)
```

Fortran command syntax:

```
CALL SCENARIOSAVE(SFILE, IERR)
```

Where:

Character*260 SFILE	full file path to save the scenario file to (input; unchanged by default. When the default is used the Scenario will be saved to the last file-name given in a SCENARIONEW, SCENARIOOPEN, or SCENARIOSAVE operation, if a call to SCENARIONEW with default arguments was the previous call, the Scenario will not be saved due to no previous file-name being available).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 2	file path is invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.37. SCENARIOSETATTRS

This API routine is obsolete.

Use this API to modify the root path and startup group attributes of the Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOSETATTRS ROOTPATH STARTUPGROUP
```

Python command syntax:

```
ierr = scenariosetattrs(rootpath, startupgroup)
```

Fortran command syntax:

```
CALL SCENARIOSETATTRS(ROOTPATH, STARTUPGROUP, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*260 ROOTPATH	directory to be used as the root path of the Scenario (input; unchanged).	
Character*72 STARTUPGROUP	name of the group to be designated as the startup group (input; unchanged).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory
	IERR = 3	specified group not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.38. SCENARIOSETCOMMENTS

This API routine is obsolete.

Use this API to define or modify the Scenario comments.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOSETCOMMENTS NCOMMENTBLOCKS COMMENTS APPEND
```

Python command syntax:

```
ierr = scenariosetcomments(ncommentblocks, comments, append)
```

Fortran command syntax:

```
CALL SCENARIOSETCOMMENTS(NCOMMENTBLOCKS, COMMENTS, APPEND, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer NCOMMENTBLOCKS	number of 72 character strings that make up the comment. In python, this number can be set to 1 and the full comment string given as the first comment block (input; 0 by default).	
Character*72 COMMENTS (NCOMMENTBLOCKS)	comment string to be defined for or added to the comments for the Scenario (input; unchanged).	
Logical APPEND	flag for appending comments. If TRUE, COMMENTS is appended to the current Scenario comments. If FALSE, the current Scenario comments are overwritten by COMMENTS (input; FALSE).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.39. SCENARIOSETOPT

This API routine is obsolete.

Use this API to modify the Scenario options. Note: Scenario settings are not attached to the Scenario file. They are user preferences that are saved on the user's machine.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOSETOPT TRACKOPT UZIPROOTOPT UZIPLCLPCOPT UZIPLCLOPT
 UZIPRMTPCOPT UZIPRMTOPT
```

Python command syntax:

```
ierr = scenariosetopt(trackopt, uziprootopt, uziplclpcopt, uziplclopt,
 uziprmtpcopt, uziprmtopt)
```

Fortran command syntax:

```
CALL SCENARIOSETOPT(TRACKOPT, UZIPROOTOPT, UZIPLCLPCOPT, UZIPLCLOPT,
 UZIPRMTPCOPT, UZIPRMTOPT, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer TRACKOPT	setting for file tracking (input; unchanged).	
	TRACKOPT = 0	The user will not be prompted about tracked files. If there are files that are currently being tracked the user will be prompted as soon as possible for those files.
	TRACKOPT = 1	The user will be prompted as soon as possible when a new file is being tracked
	TRACKOPT = 2	The user will be prompted about tracked files when the current active group is being closed
	TRACKOPT = 3	Tracked files will be added to the current active group immediately. If there are files that are currently being tracked the user will be prompted as soon as possible for those files.
Integer UZIPROOTOPT	setting for conflict resolution for root files (input; unchanged).	
	UZIPROOTOPT = 0	Skip the file.
	UZIPROOTOPT = 1	Prompt the user for action.
	UZIPROOTOPT = 2	Overwrite the file on disk.
Integer UZIPLCLPCOPT	setting for placement of local files (input; unchanged).	
	UZIPLCLPCOPT = 0	Do not unzip.
	UZIPLCLPCOPT = 1	Unzip to the root path.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	UZIPLCLPCOPT = 2	Unzip to the same path the file was at before the file was zipped.
	UZIPLCLPCOPT = 3	Unzip to the same path the file was at before, relative to the root path.
Integer UZIPLCLOPT	setting for conflict resolution for local files (input; unchanged).	
	UZIPLCLOPT = 0	Skip the file.
	UZIPLCLOPT = 1	Prompt the user for action.
	UZIPLCLOPT = 2	Overwrite the file on disk.
Integer UZIPRMTPCOPT	setting for placement of remote files (input; unchanged by default). Note: Cannot choose to unzip remote files by relative path).	
	UZIPRMTPCOPT = 0	Do not unzip.
	UZIPRMTPCOPT = 1	Unzip to the root path.
	UZIPRMTPCOPT = 2	Unzip to the same path the file was at before the file was zipped.
Integer UZIPRMTOPT	setting for conflict resolution for remote files (input; unchanged).	
	UZIPRMTOPT = 0	Skip the file.
	UZIPRMTOPT = 1	Prompt the user for action.
	UZIPRMTOPT = 2	Overwrite the file on disk.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.40. SCENARIOTRACKFILEADD

This API routine is obsolete.

Use this API to add the files currently being tracked.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOTRACKFILEADD SGROUP NFILES INDEXES DISCARD
```

Python command syntax:

```
ierr = scenariotrackfileadd(sgroup, nfiles, indexes, discard)
```

Fortran command syntax:

```
CALL SCENARIOTRACKFILEADD(SGROUP, NFILES, INDEXES, DISCARD, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*72 SGROUP	group to add tracked file to (input; current active group).	
Integer NFILES	number of files currently being tracked to add (input; no default allowed).	
Integer INDEXES (NFILES)	indexes of the files to add based on the list from SCENARIOTRACKFILEADD (input; no default allowed).	
Logical DISCARD	a flag to specify whether or not to discard the files being tracked that were not added (input; FALSE).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.41. SCENARIOTRACKFILELIST

This API routine is obsolete.

Use this API to retrieve the list of files being tracked.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, sfiles, stypes = scenariotrackfilelist()
```

Fortran command syntax:

```
CALL SCENARIOTRACKFILELIST(NFILES, SFILES, STYPES, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer NFILES	max number of files currently being tracked to return (input; no default allowed).	
Character*260 SFILES (NFILES)	list of files currently being tracked (output).	
Character*8 STYPES (NFILES)	list of file type identifiers for each file being tracked (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.42. SCENARIOTRACKFILENUM

This API routine is obsolete.

Use this API to retrieve the current number of files being tracked.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, nfiles = scenariotrackfilenum()
```

Fortran command syntax:

```
CALL SCENARIOTRACKFILENUM(NFILES, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer NFILES	number of files currently being tracked (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.43. SCENARIOUNZIP

This API routine is obsolete.

Use this API to unzip a zipped Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOUNZIP SFILE
```

Python command syntax:

```
ierr = scenariounzip(sfile)
```

Fortran command syntax:

```
CALL SCENARIOUNZIP(SFILE, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*260 SFILE	full file path of the scenario archive to unzip (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no conflicts occurred
	IERR > 0	conflicts occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.44. SCENARIOUNZIPPROBLEMS

This API routine is obsolete.

Use this API to retrieve the problems that occurred during the last unzip attempt.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
ierr, indexes, problems, paths, types = scenariounzipproblems()
```

Fortran command syntax:

```
CALL SCENARIOUNZIPPROBLEMS(ERRCNT, INDEXES, PROBLEMS, PATHS, TYPES, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer ERRCNT	return value from either SCENARIOUNZIP or SCENARIOUNZIPSOLUTIONS (input; no default allowed).	
Integer INDEXES (CNULLTERMLEN)	internal problem index which will be used in SCENARIOUNZIPSOLUTIONS (output).	
Integer PROBLEMS (CNULLTERMLEN)	problems that were encountered (output).	
	PROBLEMS = 0	file already exists.
	PROBLEMS = 1	path does not exist.
	PROBLEMS = 2	a file with the specified name does not exist in the archive.
	PROBLEMS = 3	a directory with the specified file name already exists.
Character*260 PATHS	full file paths of all files that had problems being extracted (output).	
Integer TYPES (CNULLTERMLEN)	file types associated with each file (output).	
Integer IERR	error code (output).	
	IERR = 0	no conflicts occurred
	IERR > 0	conflicts occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.45. SCENARIOUNZIPSOLUTIONS

This API routine is obsolete.

Use this API to submit potential solutions to the problems that occurred during the last unzip attempt.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOUNZIPSOLUTIONS NSOLUTIONS INDEXES SOLUTIONS PATHS
```

Python command syntax:

```
ierr = scenariounzipsolutions(nsolutions, indexes, solutions, paths)
```

Fortran command syntax:

```
CALL SCENARIOUNZIPSOLUTIONS(NSOLUTIONS, INDEXES, SOLUTIONS, PATHS, IERR)
```

Note:

This API routine is deprecated.

Where:

Integer NSOLUTIONS	number of solutions being submitted (input; no default allowed).	
Integer INDEXES (NSOLUTIONS)	internal problem indexes which were obtained from SCENARIOUNZIPPROBLEMS (input; no default allowed).	
Integer SOLUTIONS (NSOLUTIONS)	solutions to attempt Note: These solutions do not apply to all problems. For example if the file is missing from the archive only a solution of 0 will solve the problem (input; no default allowed).	
	SOLUTIONS = 0	skip the file.
	SOLUTIONS = 1	retry to unzip, possibly with a different file name.
	SOLUTIONS = 2	to overwrite the current item on disk.
Character*260 PATHS (NSOLUTIONS)	used to attempt to unzip a file to a new path or under a new name All name changes will be reflected in the Scenario. If no name is specified for a particular problem the name will not change (input).	
Integer IERR	error code (output).	
	IERR = 0	no conflicts occurred
	IERR > 0	conflicts occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.46. SCENARIOZIP

This API routine is obsolete.

Use this API to zip up the current Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOZIP SFILE
```

Python command syntax:

```
ierr = scenariozip(sfile)
```

Fortran command syntax:

```
CALL SCENARIOZIP(SFILE, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*260 SFILE	full file path of the file to zip the scenario to (input; unchanged by default). When the default is used the name of the zip will be the same name as the Scenario but with the ZIP extension instead of the PSSXML extension. If a different name is given the Scenario's name and the Scenario PSSXML file name will be changed to reflect the new name).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no Scenario in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.47. SCENARIOZIPACTIVE

This API routine is obsolete.

Use this API to determine if the Scenario is currently in Zip Mode.

This API routine was first introduced in release 33.0.0.

Python command syntax:

```
active = scenariozipactive()
```

Fortran command syntax:

```
CALL SCENARIOZIPACTIVE(ACTIVE)
```

Note:

This API routine is deprecated.

Where:

Logical ACTIVE	TRUE when the Scenario is in Zip Mode (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.48. SCENARIOZIPOPEN

This API routine is obsolete.

Use this API to open a zipped Scenario.

This API routine was first introduced in release 33.0.0.

Batch command syntax:

```
BAT_SCENARIOZIPOPEN SFILE SGROUP
```

Python command syntax:

```
ierr = scenariozipopen(sfile, sgroup)
```

Fortran command syntax:

```
CALL SCENARIOZIPOPEN(SFILE, SGROUP, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*260 SFILE	full file path of the scenario file to open (input; no default allowed).	
Character*72 SGROUP	Reserved for future use (input; ignored).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.49. SCENARIOZIPSAVE

This API routine is obsolete.

This API routine was first introduced in release 33.2.0.

Batch command syntax:

```
BAT_SCENARIOZIPSAVE SFILE
```

Python command syntax:

```
ierr = scenariozipsave(sfile)
```

Fortran command syntax:

```
CALL SCENARIOZIPSAVE(SFILE, IERR)
```

Note:

This API routine is deprecated.

Where:

Character*260 SFILE	full file path of the scenario file to save to (input; no default allowed).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.50. STDYPROJCHECKALLTASKS

Use this API to get various status info for all of the tasks.

This API routine was first introduced in release 36.2.0.

Python command syntax:

```
ierr, alldone, succeded, failed, running, pending =
 stdyprojcheckalltasks(includepending)
```

Fortran command syntax:

```
CALL STDYPROJCHECKALLTASKS(INCLUDEPENDING, ALLDONE, SUCCEDED, FAILED,
 RUNNING, PENDING, IERR)
```

Where:

integer INCLUDEPENDING	1 to includ pending in the ALLDONE check, 0 to exclude pending in ALLDONE check. (input).	
integer ALLDONE	1 no Tasks are in the "In Progress" state (or "Pending" state if INCLUDEPENDING is non-zero), 0 otherwise. (input).	
integer SUCCEDED	Number of Tasks that have succeded in completing. (input).	
integer FAILED	Number of Tasks that have succeded in completing. (input).	
integer RUNNING	Number of Tasks that are currently in progress. (input).	
integer PENDING	Number of Tasks that are blocked from running due to other states, e.g. a locked Variant or Simulation. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.51. STDYPROJCURRENTREPORTSETLOCKED

Use this API to lock or unlock the current Study Project Report. If Locked, Subsequent Report changes will be put into a new Report.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJCURRENTREPORTSETLOCKED LOCK
```

Python command syntax:

```
stdyprojcurrentreportsetlocked(lock)
```

Fortran command syntax:

```
CALL STDYPROJCURRENTREPORTSETLOCKED(LOCK)
```

Where:

integer LOCK	0 for Unlock, 1 for Lock. (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.52. STDYPROJCURRENTVARIANTGETLOCKED

Use this API to get the lock state of the current Variant.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lock = stdyprojcurrentvariantgetlocked()
```

Fortran command syntax:

```
CALL STDYPROJCURRENTVARIANTGETLOCKED(LOCK)
```

Where:

integer LOCK	0 for Unlock, 1 for Lock. (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.53. STDYPROJCURRENTVARIANTSETLOCKED

Use this API to lock or unlock the current Network Model state Variant. If Locked, Subsequent Network Model changes will be put into a new Variant who's parent Variant is the current Variant.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJCURRENTVARIANTSETLOCKED LOCK
```

Python command syntax:

```
stdyprojcurrentvariantsetlocked(lock)
```

Fortran command syntax:

```
CALL STDYPROJCURRENTVARIANTSETLOCKED(LOCK)
```

Where:

integer LOCK	0 for Unlock, 1 for Lock. (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.54. STDYPROJEXTRACTDATASET

Use this API to extract a data set from the active Study Project and place it in the location specified.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJEXTRACTDATASET SPATH SSETNAME
```

Python command syntax:

```
stdyprojextractdataset(spath, ssetname)
```

Fortran command syntax:

```
CALL STDYPROJEXTRACTDATASET(SPATH, SSETNAME)
```

Where:

character*260 SPATH	Folder location to place the data set file to be extracted. This will overwrite conflicting files. (input).	
character*72 SSETNAME	Name of the data set to be extracted (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid input data set name
	IERR = 4	unable to access destination path

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.55. STDYPROJEXTRACTSIMULATIONRESULT

Use this API to extract simulation results from the active Study Project and place them in the location specified.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJEXTRACTSIMULATIONRESULT SEXTRACTLOC SVARIANTNAME SSIMNAME
```

Python command syntax:

```
stdyprojextractsimulationresult(sextractloc, svariantname, ssimname)
```

Fortran command syntax:

```
CALL STDYPROJEXTRACTSIMULATIONRESULT(SEXTRACTLOC, SVARIANTNAME, SSIMNAME)
```

Where:

character*260 SEXTRACTLOC	Folder location to place the extracted results (input).	
character*72 SVARIANTNAME	Name of the variant to extract results for (input).	
character*72 SSIMNAME	Name of the simulation to extract results for (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant/simulation name combination
	IERR = 4	unable to access main destination path

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.56. STDYPROJGETCURRENTSIMULATION

Use this API to get the name of the active Simulation of the Study Project.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, name = stdyprojgetcurrentsimulation()
```

Fortran command syntax:

```
CALL STDYPROJGETCURRENTSIMULATION(NAME, IERR)
```

Where:

character*72 NAME	Name of the active Simulation (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.57. STDYPROJGETCURRENTVARIANT

Use this API to get the name of the active Variant of the Study Project.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, name = stdyprojgetcurrentvariant()
```

Fortran command syntax:

```
CALL STDYPROJGETCURRENTVARIANT(NAME, IERR)
```

Where:

character*72 NAME	Name of the active Variant (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.58. STDYPROJGETSETTINGS

Use this API to get the current setting of the Study Project.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, usecloud, maxlocaltasks = stdyprojgetsettings()
```

Fortran command syntax:

```
CALL STDYPROJGETSETTINGS(USECLOUD, MAXLOCALTASKS, IERR)
```

Where:

integer USECLOUD	Directs the Study Project Manager to offload work to the PSS(R)E Cloud, if available. (input).	
integer MAXLOCALTASKS	Specifies how many jobs need to be running locally before work is sent to PSS(R)E Cloud. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	study project module not in use

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.59. STDYPROJINPUTDATASETADD

Use this API to add a file to the Study Project as an Input Data Set. This action occurs automatically when a called API references a file that is not in the Study Project, but this API allows the action to be done manually by the user.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJINPUTDATASETADD PATH
```

Python command syntax:

```
stdyprojinputdatasetadd(path)
```

Fortran command syntax:

```
CALL STDYPROJINPUTDATASETADD(PATH)
```

Where:

character*260 PATH	Full path to the file to add to the Study Project as an Input Data Set. If an Input Data Set has the same name as the File pointed to by PATH, the current Input Data Set will be overwritten. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid path

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.60. STDYPROJINPUTDATASETREMOVE

Use this API to remove an Input Data Set from the Study Project, this will remove Simulations and Variants that depend on it.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJINPUTDATASETREMOVE NAME
```

Python command syntax:

```
stdyprojinputdatasetremove(name)
```

Fortran command syntax:

```
CALL STDYPROJINPUTDATASETREMOVE(NAME)
```

Where:

character*72 NAME	Name of the Input Data Set to remove (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid input dat set name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.61. STDYPROJIPLANLAUNCH

This API is used to launch an IPLAN program in the Study Project

This API routine was first introduced in release 36.0.

Python command syntax:

```
ierr = stdyprojiplanlaunch(scriptfile, argcount, args, infilecount,
 infiles, outfilecount, outfiles)
```

Fortran command syntax:

```
CALL STDYPROJIPLANLAUNCH(SCRIPTFILE, ARGCOUNT, ARGS, INFILECOUNT, INFILES,
 OUTFILECOUNT, OUTFILES, IERR)
```

Where:

character*260 SCRIPTFILE	(input).	
integer ARGCOUNT	Number of arguments passed (input; no default allowed).	
character*260 ARGS (ARGCOUNT)	Arguments passed (input; no default allowed).	
Integer INFILECOUNT	Number of Input files passed (input; no default allowed).	
character*260 INFILES (INFILE-COUNT)	Input files passed (input; no default allowed).	
integer OUTFILECOUNT	Number of Output files passed (input; no default allowed).	
character*260 OUTFILES (OUT-FILECOUNT)	Output files passed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Missing Input File %%

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.62. STDYPROJNEW

Use this API to create a new Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJNEW NAME FORMAT COMPRESS
```

Python command syntax:

```
ierr = stdyprojnew(name, format, compress)
```

Fortran command syntax:

```
CALL STDYPROJNEW(NAME, FORMAT, COMPRESS, IERR)
```

Where:

character*72 NAME	Name to give the new Study Project. Blank by default, a unique name staring with "New Study" will be used. (input).	
integer FORMAT	Format to use for the internal script files. 0 for batch (disallows executing custom python scripts as simulations). 1 for python. (input; 1 by default).	
integer COMPRESS	Compress all contents into an archive file. True by default. (input; 0 by default).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 4	unable to access main archive path

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.63. STDYPROJOPEN

Use this API to open a Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJOPEN NAME
```

Python command syntax:

```
ierr = stdyprojopen(name)
```

Fortran command syntax:

```
CALL STDYPROJOPEN(NAME, IERR)
```

Where:

character*72 NAME	Name of the Study Project to Open (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 2	invalid file name
	IERR = 4	unable to access main archive path

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.64. STDYPROJPYTHONLAUNCH

This API is used to launch a Python script in the Study Project

This API routine was first introduced in release 36.0.

Python command syntax:

```
ierr = stdyprojpythonlaunch(scriptfile, argcount, args, infilecount,
 infiles, outfilecount, outfiles)
```

Fortran command syntax:

```
CALL STDYPROJPYTHONLAUNCH(SCRIPTFILE, ARGCOUNT, ARGS, INFILECOUNT,
 INFILES, OUTFILECOUNT, OUTFILES, IERR)
```

Where:

character*260 SCRIPTFILE	(input; no default allowed).	
integer ARGCOUNT	Number of arguments passed (input; no default allowed).	
character*260 ARGS (ARGCOUNT)	Arguments passed (input; no default allowed).	
Integer INFILECOUNT	Number of Input files passed (input; no default allowed).	
character*260 INFILES (INFILE-COUNT)	Input files passed (input; no default allowed).	
integer OUTFILECOUNT	Number of Output files passed (input; no default allowed).	
character*260 OUTFILES (OUT-FILECOUNT)	Output files passed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Missing Input File %%

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.65. STDYPROJRENAME

Use this API to rename the current working Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
STDYPROJSETNAME NEWNAME
```

Python command syntax:

```
ierr = stdyprojrename(newname)
```

Fortran command syntax:

```
CALL STDYPROJRENAME(NEWNAME, IERR)
```

Where:

Character*72 NEWNAME	New name of the study project (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid file name
	IERR = 3	there is another study project file with the same name
	IERR = 4	unable to access main archive path

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.66. STDYPROJREPORTADD

Use this API to add a simualtion to the active Study Project.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJREPORTADD NAME
```

Python command syntax:

```
ierr = stdyprojreportadd(name)
```

Fortran command syntax:

```
CALL STDYPROJREPORTADD(NAME, IERR)
```

Where:

character*72 NAME	Name to give the new report (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 3	there is another report with the same name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.67. STDYPROJREPORTREMOVE

Use this API to remove a simualtion to the active Study Project.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJREPORTREMOVE NAME
```

Python command syntax:

```
ierr = stdyprojreportremove(name)
```

Fortran command syntax:

```
CALL STDYPROJREPORTREMOVE(NAME, IERR)
```

Where:

character*72 NAME	Name of the report to remove (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid report name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.68. STDYPROJREPORTRENAME

Use this API to rename the a report.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
STDYPROJREPORTRENAME NAME NEWNAME
```

Python command syntax:

```
ierr = stdyprojreportrename(name, newname)
```

Fortran command syntax:

```
CALL STDYPROJREPORTRENAME(NAME, NEWNAME, IERR)
```

Where:

character*72 NAME	Current name of the report (input).	
character*72 NEWNAME	New name of the report (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid report name
	IERR = 3	there is another report with the same name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.69. STDYPROJREPORTSETLOCK

Use this API to set the lock status of a Report.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
STDYPROJREPORTSETLOCK NAME LOCK
```

Python command syntax:

```
ierr = stdyprojreportsetlock(name, lock)
```

Fortran command syntax:

```
CALL STDYPROJREPORTSETLOCK(NAME, LOCK, IERR)
```

Where:

character*72 NAME	Name of the report to get information on (input).	
integer LOCK	Specifies the lock status of the Report. 0 is unlocked, 1 is locked. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid report name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.70. STDYPROJRESPONSELAUNCH

This API is used to launch a Response script in the Study Project

This API routine was first introduced in release 36.0.

Python command syntax:

```
ierr = stdyprojresponselaunch(scriptfile, argcount, args, infilecount,
 infiles, outfilecount, outfiles)
```

Fortran command syntax:

```
CALL STDYPROJRESPONSELAUNCH(SCRIPTFILE, ARGCOUNT, ARGS, INFILECOUNT,
 INFILES, OUTFILECOUNT, OUTFILES, IERR)
```

Where:

character*260 SCRIPTFILE	(input).	
integer ARGCOUNT	Number of arguments passed (input; no default allowed).	
character*260 ARGS (ARGCOUNT)	Arguments passed (input; no default allowed).	
Integer INFILECOUNT	Number of Input files passed (input; no default allowed).	
character*260 INFILES (INFILE-COUNT)	Input files passed (input; no default allowed).	
integer OUTFILECOUNT	Number of Output files passed (input; no default allowed).	
character*260 OUTFILES (OUT-FILECOUNT)	Output files passed (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Missing Input File %%

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.71. STDYPROJSETCURRENTREPORT

Use this API to set the active report of the Study Project.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJSETCURRENTREPORT NAME
```

Python command syntax:

```
ierr = stdyprojsetcurrentreport(name)
```

Fortran command syntax:

```
CALL STDYPROJSETCURRENTREPORT(NAME, IERR)
```

Where:

character*72 NAME	Name of the report to make active (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid report name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.72. STDYPROJSETCURRENTSIMULATION

Use this API to set the active Simulation of the Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJSETCURRENTSIMULATION NAME
```

Python command syntax:

```
ierr = stdyprojsetcurrentsimulation(name)
```

Fortran command syntax:

```
CALL STDYPROJSETCURRENTSIMULATION(NAME, IERR)
```

Where:

character*72 NAME	Name of the Simulation to make active (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid simulation name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.73. STDYPROJSETCURRENTVARIANT

Use this API to set the active variant of the Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJSETCURRENTVARIANT NAME
```

Python command syntax:

```
ierr = stdyprojsetcurrentvariant(name)
```

Fortran command syntax:

```
CALL STDYPROJSETCURRENTVARIANT(NAME, IERR)
```

Where:

character*72 NAME	Name of the Variant to make active (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.74. STDYPROJSETSETTINGS

Use this API to set the current setting of the Study Project.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr = stdyprojsetsettings(usecloud, maxlocaltasks)
```

Fortran command syntax:

```
CALL STDYPROJSETSETTINGS(USECLOUD, MAXLOCALTASKS, IERR)
```

Where:

integer USECLOUD	Directs the Study Project Manager to offload work to the PSS(R)E Cloud, if available. Unchanged by default. (input).	
integer MAXLOCALTASKS	Specifies how many jobs need to be running locally before work is sent to PSS(R)E Cloud. Unchanged by defaut. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	study project module not in use

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.75. STDYPROJSIMULATIONADD

Use this API to add a simualtion to the active Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJSIMULATIONADD NAME
```

Python command syntax:

```
ierr = stdyprojsimulationadd(name)
```

Fortran command syntax:

```
CALL STDYPROJSIMULATIONADD(NAME, IERR)
```

Where:

character*72 NAME	Name to give the new simulation (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 3	there is another simulation with the same name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.76. STDYPROJSIMULATIONREMOVE

Use this API to remove a simualtion to the active Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJSIMULATIONREMOVE NAME
```

Python command syntax:

```
ierr = stdyprojsimulationremove(name)
```

Fortran command syntax:

```
CALL STDYPROJSIMULATIONREMOVE(NAME, IERR)
```

Where:

character*72 NAME	Name of the simulation to remove (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid simulation name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.77. STDYPROJSIMULATIONRENAME

Use this API to rename the a simulation.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
STDYPROJSIMULATIONRENAME NAME NEWNAME
```

Python command syntax:

```
ierr = stdyprojsimulationrename(name, newname)
```

Fortran command syntax:

```
CALL STDYPROJSIMULATIONRENAME(NAME, NEWNAME, IERR)
```

Where:

character*72 NAME	Current name of the simulation (input).	
character*72 NEWNAME	New name of the simulation (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid simulation name
	IERR = 3	there is another simulation with the same name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.78. STDYPROJSIMULATIONSETLOCK

Use this API to set the lock status of a simulation.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJSIMULATIONSETLOCK NAME LOCK
```

Python command syntax:

```
ierr = stdyprojsimulationsetlock(name, lock)
```

Fortran command syntax:

```
CALL STDYPROJSIMULATIONSETLOCK(NAME, LOCK, IERR)
```

Where:

character*72 NAME	Name of the simulation to get information on (input).	
integer LOCK	Specifies the lock status of the Simulation. 0 is unlocked, 1 is locked. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid simulation name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.79. STDYPROJTASKOPT

Use this API to adjust the execution of a task.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJTASKOPT SVARIANTNAME SSIMNAME COMMAND
```

Python command syntax:

```
ierr = stdyprojtaskopt(svariantname, ssimname, command)
```

Fortran command syntax:

```
CALL STDYPROJTASKOPT(SVARIANTNAME, SSIMNAME, COMMAND, IERR)
```

Where:

character*72 SVARIANTNAME	Name of the variant related to the task to send a command to. (input).	
character*72 SSIMNAME	Name of the simulation related to the task to send a command to. (input).	
integer COMMAND	Command to direct to the specified task. (input; no default allowed).	
	COMMAND = 1	Run Normally
	COMMAND = 2	Do Not Run
	COMMAND = 3	Re-Run
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant/simulation name combination

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.80. STDYPROJTASKSTATUS

Use this API to retrieve status of a task.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
ierr, status = stdyprojtaskstatus(svariantname, ssimname)
```

Fortran command syntax:

```
CALL STDYPROJTASKSTATUS(SVARIANTNAME, SSIMNAME, STATUS, IERR)
```

Where:

character*72 SVARIANTNAME	Name of the variant related to the task to retrieve the status of. (input).	
character*72 SSIMNAME	Name of the simulation related to the task to retrieve the status of. (input).	
integer STATUS	Status of the task. (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant/simulation name combination

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.81. STDYPROJVARIANTADD

Use this API to add a variant to the active Study Project.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJVARIANTADD NAME PARENTNAME
```

Python command syntax:

```
ierr = stdyprojvariantadd(name, parentname)
```

Fortran command syntax:

```
CALL STDYPROJVARIANTADD(NAME, PARENTNAME, IERR)
```

Where:

character*72 NAME	Name to give the new variant (input).	
character*72 PARENTNAME	Name of the parent variant (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid parent name
	IERR = 3	there is another variant with the same name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.82. STDYPROJVARIANTCLONE

Use this API to clone a variant in the active Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJVARIANTCLONE NAME PARENTNAME NEWNAME
```

Python command syntax:

```
ierr = stdyprojvariantclone(name, parentname, newname)
```

Fortran command syntax:

```
CALL STDYPROJVARIANTCLONE(NAME, PARENTNAME, NEWNAME, IERR)
```

Where:

character*72 NAME	Name of the variant to clone (input).	
character*72 PARENTNAME	Name of the parent variant to place the clone under (input).	
character*72 NEWNAME	Name to give the new variant (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant/parent name combination
	IERR = 3	there is another variant with the same name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.83. STDYPROJVARIANTCOMPARE

Use this API to compare two variants in the active Study Project.

This API routine was first introduced in release 36.1.0.

Batch command syntax:

```
BAT_STDYPROJVARIANTCOMPARE BASISVARIANTNAME SUBJECTVARIANTNAME
```

Python command syntax:

```
ierr = stdyprojvariantcompare(basisvariantname, subjectvariantname)
```

Fortran command syntax:

```
CALL STDYPROJVARIANTCOMPARE(BASISVARIANTNAME, SUBJECTVARIANTNAME, IERR)
```

Where:

character*72 BASISVARIANTNAME	Name of the variant to treat as the basis of the comparison (input).	
character*72 SUBJECTVARIANT-NAME	Name of the variant to treat as the subject of the comparison (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid basis variant name
	IERR = 3	invalid subject variant name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.84. STDYPROJVARIANTGETLOCK

Use this API to get the lock status of a Variant.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, lock = stdyprojvariantgetlock(name)
```

Fortran command syntax:

```
CALL STDYPROJVARIANTGETLOCK(NAME, LOCK, IERR)
```

Where:

character*72 NAME	Name of the variant to get information on (input).	
integer LOCK	Specifies the lock status of the Variant. 0 is unlocked, 1 is locked. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.85. STDYPROJVARIANTREMOVE

Use this API to remove a variant from the active Study Project.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJVARIANTREMOVE SNAME
```

Python command syntax:

```
ierr = stdyprojvariantremove(name)
```

Fortran command syntax:

```
CALL STDYPROJVARIANTREMOVE(NAME, IERR)
```

Where:

character*72 NAME	Name of the variant to remove (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.86. STDYPROJVARIANTRENAME

Use this API to rename the a variant.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJVARIANTRENAME NAME NEWNAME
```

Python command syntax:

```
ierr = stdyprojvariantrename(name, newname)
```

Fortran command syntax:

```
CALL STDYPROJVARIANTRENAME(NAME, NEWNAME, IERR)
```

Where:

character*72 NAME	Current name of the variant (input).	
character*72 NEWNAME	New name of the variant (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant name
	IERR = 3	there is another variant with the same name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.87. STDYPROJVARIANTSETLOCK

Use this API to set the lock status of a Variant.

This API routine was first introduced in release 36.0.0.

Batch command syntax:

```
BAT_STDYPROJVARIANTSETLOCK NAME LOCK
```

Python command syntax:

```
ierr = stdyprojvariantsetlock(name, lock)
```

Fortran command syntax:

```
CALL STDYPROJVARIANTSETLOCK(NAME, LOCK, IERR)
```

Where:

character*72 NAME	Name of the variant to get information on (input).	
integer LOCK	Specifies the lock status of the Variant. 0 is unlocked, 1 is locked. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	invalid variant name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

15.88. STDYPROJWAITALLTASKS

Use this API to have the calling thread wait until all Tasks are out of the "In Progress" state.

This API routine was first introduced in release 36.2.0.

Python command syntax:

```
ierr = stdyprojwaitalltasks(timeout includepending)
```

Fortran command syntax:

```
CALL STDYPROJWAITALLTASKS(TIMEOUT, INCLUDEPENDING, IERR)
```

Where:

integer TIMEOUT	Timeout period in seconds to wait for tasks to complete. Will wait indefinitely by default. (input).	
integer INCLUDEPENDING	1 to include waiting for pending tasks, 0 to exclude waiting for pending. Advisable to always use a timeout when waiting for pending tasks and if used in a loop, only wait for pending on the first check. (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR > 0	fatal error occurred
	IERR = 1	no study project in memory
	IERR = 2	timeout occured

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 16
Event Studies

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

16.1. CLOSEEVENTSTUDYFILE

Use this API to close an open Event Study file.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_CLOSEEVENTSTUDYFILE
```

Python command syntax:

```
closeeventstudyfile()
```

Fortran command syntax:

```
CALL CLOSEEVENTSTUDYFILE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

16.2. OPENEVENTSTUDYFILE

Use this API to open a new or existing event study file in the GUI.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_OPENEVENTSTUDYFILE SFILE
```

Python command syntax:

```
openeventstudyfile(sfile)
```

Fortran command syntax:

```
CALL OPENEVENTSTUDYFILE(SFILE)
```

Where:

Character*260 SFILE	name of the new or existing event study file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

16.3. RUNDYNAMICEVENTSTUDY

Use this API to run a dynamic study using the active event study in the open event study file.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_RUNDYNAMICEVENTSTUDY
```

Python command syntax:

```
rundynamiceventstudy()
```

Fortran command syntax:

```
CALL RUNDYNAMICEVENTSTUDY
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

16.4. RUNPOWERFLOWEVENTSTUDY

Use this API to run a power flow study using the active event study in the open event study file.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_RUNPOWERFLOWEVENTSTUDY
```

Python command syntax:

```
runpowerfloweventstudy()
```

Fortran command syntax:

```
CALL RUNPOWERFLOWEVENTSTUDY
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

16.5. SAVEEVENTSTUDYFILE

Use this API to save existing event studies in an event study file.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SAVEEVENTSTUDYFILE SFILE
```

Python command syntax:

```
saveeventstudyfile(sfile)
```

Fortran command syntax:

```
CALL SAVEEVENTSTUDYFILE(SFILE)
```

Where:

Character*260 SFILE	name of the destination event study file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

16.6. SETACTIVEEVENTSTUDY

Use this API to select an Event Study in an Event file as the active Event Study for running dynamic or power flow event studies.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_SETACTIVEEVENTSTUDY SFILE
```

Python command syntax:

```
setactiveeventstudy(sfile)
```

Fortran command syntax:

```
CALL SETACTIVEEVENTSTUDY(SFILE)
```

Where:

Character*260 SFILE	name of an event study in the open event study file (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 17
Measurement Interface

This chapter describes the API routines used in PSS®E Measurement Interface feature.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 18
Multiple Element Data

This chapter defines API routines that are used to retrieve data that are neither single elements nor based on subsystems.

These routines do not record themselves when called with recording enabled.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.1. AGCAPCOUNT

Use this API to return the number of data points of a GCAP curve.

This API routine was first introduced in release 35.6.1.

Python command syntax:

```
ierr, count = agcapcount(ibus, id)
```

Fortran command syntax:

```
CALL GET_GCAP_COUNT(IBUS, ID, COUNT, IERR)
```

Where:

Integer IBUS	Machine bus number (input; no default allowed).	
Character*2 ID	Machine identifier (input).	
Integer COUNT	number of elements in RARRAY. (output; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error, number of GCAP data points returned
	IERR = 1	machine bus not found
	IERR = 2	no machine at the bus
	IERR = 3	machine not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.2. AGCAPREAL

Use this API to return an array of real values for a GCAP curve.

This API routine was first introduced in release 35.6.1.

Python command syntax:

```
ierr, rarray = agcapreal(ibus,id,string)
```

Fortran command syntax:

```
CALL GET_GCAP_REAL(IBUS,ID,NSTR,STRING,COUNT,RARRAY,PDCOUNT,IERR)
```

Where:

Integer IBUS	Machine bus number (input).	
Character ID	Machine identifier (input).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50); no default allowed	
Character STRING (NSTR)	NSTR of the following strings indicating the GCAP quantities desired (input; no default allowed).	
	'PGEN'	active power output
	'QMAX'	Qmax at the PGEN
	'QMIN'	Qmin at the PGEN
Integer COUNT	number of elements in RARRAY. COUNT must be as least as large as the value of PDCOUNT returned by the GET_CAP_COUNT API; no default allowed	
Real RARRAY (NSTR*COUNT)	Is an array of at least NSTR columns, each of which contains COUNT elements (output).	
Integer PDCOUNT	number of elements in each column of RARRAY that are set by this API. PDCOUNT is identical to the value of COUNT returned by the GET_GCAP_COUNT API (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	bus not found
	IERR = 2	bus has no generators
	IERR = 3	generator not found
	IERR = 4	invalid string

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.3. AINTERFACECHAR

Use this API to return an array of character values for the interfaces.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, carray = ainterfacechar(string)
```

Fortran command syntax:

```
CALL GET_INTERFACE_CHAR(NSTR,STRING,DIM,CARRAY,INTERFACES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the interfaces quantities desired (input; no default allowed).	
	'INTFNAME'	interface name (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of INTERFACES returned by the GET_INTERFACE_COUNT API when it is called (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer INTERFACES	number of elements in each column of CARRAY that are set by this API. INTERFACES is identical to the value of INTERFACE returned by the GET_INTERFACE_COUNT API when it is called (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid NSTR value
	IERR = 3	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.4. AINTERFACECOUNT

Use this API to return the number of interfaces.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, interfaces = ainterfacecount()
```

Fortran command syntax:

```
CALL GET_INTERFACE_COUNT(INTERFACES,IERR)
```

Where:

Integer INTERFACES	number of interfaces (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.5. AINTERFACEELEMENTCHAR

Use this API to return an array of character values of interface element of an interface.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, carray = ainterfaceelementchar(namarg, string)
```

Fortran command syntax:

```
CALL
 GET_ELEMENT_BY_INTERFACE_CHAR(NAMARG,NSTR,STRING,DIM,CARRAY,ELEMENTS,IERR)
```

Where:

Character NAMARG	Defines the load type name to be used (input; no default allowed).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'INTFTYPE'	Interface element type
	'DEVICENAME'	Interface element device id or device name
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of ELEMENTS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer ELEMENTS	number of elements in each column of IARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid Interface name
	IERR = 3	Interface not found
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of CARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.6. AINTERFACEELEMENTCOUNT

Use this API to return the number of array entries required to accommodate the data to be returned by the remaining members of the interface element data family.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, elements = ainterfaceelementcount(namarg)
```

Fortran command syntax:

```
CALL GET_ELEMENT_BY_INTERFACE_COUNT(NAMARG, ELEMENTS, IERR)
```

Where:

Character NAMARG	Defines the load type to be used (input; no default allowed).	
Integer ELEMENTS	number of elements in the interface NAMARG defined by FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid Interface name
	IERR = 3	Interface not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.7. AINTERFACEELEMENTCPLX

Use this API to return an array of complex values of interface element of an interface.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, xarray = ainterfaceelementcplx(namarg, string)
```

Fortran command syntax:

```
CALL
 GET_ELEMENT_BY_INTERFACE_COMPLEX(NAMARG,NSTR,STRING,DIM,XARRAY,ELEMENTS,IERR)
```

Where:

Character NAMARG	Defines the interface name to be used (input; no default allowed).	
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed).	
	'FLOW'	Interface element flow
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of ELEMENTS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).	
Complex XARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer ELEMENTS	number of elements in each column of IARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid Interface name
	IERR = 3	Interface not found
	IERR = 4	Invalid NSTR value
	IERR = 5	DIM, and hence the size of XARRAY, is not large enough
	IERR = 6	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.8. AINTERFACEELEMENTINT

Use this API to return an array of integer values of interface element of an interface.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, iarray = ainterfaceelementint(namarg, string)
```

Fortran command syntax:

```
CALL
 GET_ELEMENT_BY_INTERFACE_INT(NAMARG,NSTR,STRING,DIM,IARRAY,ELEMENTS,IERR)
```

Where:

Character NAMARG	Defines the load type name to be used (input; no default allowed).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed). 'FROMNUMBER' Bus number 'TONUMBER' Bus number 'LASTNUMBER' Bus number 'STATUS' element status 'DIRECTION' Bus number 'METERBUS' Bus number
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of ELEMENTS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer ELEMENTS	number of elements in each column of IARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output). IERR = 0 No error IERR = 1 Working case is empty IERR = 2 Invalid Interface name IERR = 3 Interface not found IERR = 4 Invalid NSTR value IERR = 5 DIM, and hence the size of IARRAY, is not large enough IERR = 6 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.9. AINTERFACEELEMENTREAL

Use this API to return an array of integer values of interface element of an interface.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, rarray = ainterfaceelementreal(namarg, string)
```

Fortran command syntax:

```
CALL
 GET_ELEMENT_BY_INTERFACE_REAL(NAMARG,NSTR,STRING,DIM,RARRAY,ELEMENTS,IERR)
```

Where:

Character NAMARG	Defines the load type name to be used (input; no default allowed).
Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the bus and/or load quantities desired (input; no default allowed). 'FROMNUMBER' Bus number 'TONUMBER' Bus number 'LASTNUMBER' Bus number 'STATUS' element status 'DIRECTION' Bus number 'METERBUS' Bus number
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of ELEMENTS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (input; no default allowed).
real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer ELEMENTS	number of elements in each column of IARRAY that are set by this API. LOADS is identical to the value of LOADS returned by the GET_ELEMENT_BY_INTERFACE_COUNT API when it is called with the same values specified for SID and FLAG (output).
Integer IERR	error code (output). IERR = 0 No error IERR = 1 Working case is empty IERR = 2 Invalid Interface name IERR = 3 Interface not found IERR = 4 Invalid NSTR value IERR = 5 DIM, and hence the size of RARRAY, is not large enough IERR = 6 Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.10. AINTERFACEINT

Use this API to return an array of integer values for the interfaces.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, iarray = ainterfaceint(string)
```

Fortran command syntax:

```
CALL GET_INTERFACE_INT(NSTR,STRING,DIM,IARRAY,INTERFACES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the interfaces quantities desired (input; no default allowed).	
	'NUMBER'	interface number
	'TYPE'	interface type
Integer DIM	number of elements in each column of the array IARRAY. DIM must be as least as large as the value of INTERFACES returned by the GET_INTERFACE_COUNT API when it is called (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer INTERFACES	number of elements in each column of IARRAY that are set by this API. INTERFACES is identical to the value of INTERFACES returned by the GET_INTERFACE_COUNT API when it is called (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid NSTR value
	IERR = 3	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.11. AINTERFACEREAL

Use this API to return an array of real values for the interfaces.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr, rarray = ainterfacereal(string)
```

Fortran command syntax:

```
CALL GET_INTERFACE_REAL(NSTR,STRING,DIM,RARRAY,INTERFACES,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).
Character STRING (NSTR)	NSTR of the following strings indicating the interfaces quantities desired (input; no default allowed).

'MAXRATE1'	Maximum Rating from rating set 1
'MAXRATE2'	Maximum Rating from rating set 2
'MAXRATE3'	Maximum Rating from rating set 3
'MAXRATE4'	Maximum Rating from rating set 4
'MAXRATE5'	Maximum Rating from rating set 5
'MAXRATE6'	Maximum Rating from rating set 6
'MAXRATE7'	Maximum Rating from rating set 7
'MAXRATE8'	Maximum Rating from rating set 8
'MAXRATE9'	Maximum Rating from rating set 9
'MAXRATE10'	Maximum Rating from rating set 10
'MAXRATE11'	Maximum Rating from rating set 11
'MAXRATE12'	Maximum Rating from rating set 12
'MINRATE1'	Minimum Rating from rating set 1
'MINRATE2'	Minimum Rating from rating set 2
'MINRATE3'	Minimum Rating from rating set 3
'MINRATE4'	Minimum Rating from rating set 4
'MINRATE5'	Minimum Rating from rating set 5
'MINRATE6'	Minimum Rating from rating set 6
'MINRATE7'	Minimum Rating from rating set 7
'MINRATE8'	Minimum Rating from rating set 8
'MINRATE9'	Minimum Rating from rating set 9
'MINRATE10'	minimum Rating from rating set 10
'MINRATE11'	minimum Rating from rating set 11
'MINRATE12'	minimum Rating from rating set 12

Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of INTERFACES returned by the GET_INTERFACE_COUNT API when it is called (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).
Integer INTERFACES	number of elements in each column of CARRAY that are set by this API. INTERFACES is identical to the value of INTERFACES returned by the GET_INTERFACE_COUNT API when it is called (output).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Working case is empty
IERR = 2	Invalid NSTR value
IERR = 3	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.12. AVOLTAGEDROOPCHAR

Use this API to return an array of character values for the voltage droop controls.

This API routine was first introduced in release 35.3.0.

Python command syntax:

```
ierr, carray = avoltagedroopchar(string)
```

Fortran command syntax:

```
CALL GET_VD_CHAR(NSTR,STRING,DIM,CARRAY,VDS,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the voltage droop quantities desired (input; no default allowed).	
	'DROOPNAME'	voltage droop name (40 characters).
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of VDS returned by the GET_VD_COUNT API when it is called (input; no default allowed).	
Character CARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer VDS	number of elements in each column of CARRAY that are set by this API. VDS is identical to the value of VDS returned by the GET_VD_COUNT API when it is called (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid NSTR value
	IERR = 3	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.13. AVOLTAGEDROOPCOUNT

Use this API to return the number of voltage droop controls.

This API routine was first introduced in release 35.3.0.

Python command syntax:

```
ierr, vdroops = avoltagedroopcount()
```

Fortran command syntax:

```
CALL GET_VD_COUNT(VDROOPS,IERR)
```

Where:

Integer VDROOPS	number of voltage droop controls (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.14. AVOLTAGEDROOPINT

Use this API to return an array of real values for the voltage droop controls.

This API routine was first introduced in release 35.3.0.

Python command syntax:

```
ierr, iarray = avoltagedroopint(string)
```

Fortran command syntax:

```
CALL GET_VD_INT(NSTR,STRING,DIM,IARRAY,VDS,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the voltage droop quantities desired (input; no default allowed).	
	'STATUS'	defined voltage droop control status
	'ACTSTATUS'	actual voltage droop control status
	'REGBUS'	defined regulated bus of voltage droop control
	'ACTREGBUS'	actual regulated bus of voltage droop control
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of VDS returned by the GET_VD_COUNT API when it is called (input; no default allowed).	
Integer IARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer VDS	number of elements in each column of CARRAY that are set by this API. VDS is identical to the value of VDS returned by the GET_VD_COUNT API when it is called (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid NSTR value
	IERR = 3	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.15. AVOLTAGEDROOPREAL

Use this API to return an array of real values for the voltage droop controls.

This API routine was first introduced in release 35.3.0.

Python command syntax:

```
ierr, rarray = avoltagedroopreal(string)
```

Fortran command syntax:

```
CALL GET_VD_REAL(NSTR,STRING,DIM,RARRAY,VDS,IERR)
```

Where:

Integer NSTR	number of elements in STRING (1 < NSTR < 50) (input; no default allowed).	
Character STRING (NSTR)	NSTR of the following strings indicating the voltage droop quantities desired (input; no default allowed).	
	'QMAX'	QMAX of voltage droop control
	'QDB'	QDB of voltage droop control
	'QMIN'	QMIN of voltage droop control
	'VLOW'	VLOW of voltage droop control
	'VDBLOW'	VDBLOW of voltage droop control
	'VDBHIGH'	VDBHIGH of voltage droop control
	'VHIGH'	VHIGH of voltage droop control
	'VOLTAGE'	voltage at the regulated bus of voltage droop control
	'QINJ'	Q injection to the regulated bus of voltage droop control, Q is positive if the Q enters into the bus
	'QMAX'	desired Q injection to the regulated bus of voltage droop control, Q is positive if the Q enters into the bus
	'QGEN'	total Q outputs of the generators in the voltage droop control
Integer DIM	number of elements in each column of the array CARRAY. DIM must be as least as large as the value of VDS returned by the GET_VD_COUNT API when it is called (input; no default allowed).	
Real RARRAY (NSTR*DIM)	Is an array of at least NSTR columns, each of which contains DIM elements (output).	
Integer VDS	number of elements in each column of CARRAY that are set by this API. VDS is identical to the value of VDS returned by the GET_VD_COUNT API when it is called (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Invalid NSTR value
IERR = 3	Invalid STRING value

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.16. GETMODELLIBRARIES

Return the list of libraries that are searched for dynamically (i.e. run-time) loaded dynamics model programs.
In particular this will be the list of libraries searched for user-written dynamics model programs.

This API routine was first introduced in release 31.0.0.

Python command syntax:

```
names = getmodellibraries()
```

Fortran command syntax:

```
CALL GETMODELLIBRARIES(NSTR, NAMES, NRET)
```

Where:

Integer NSTR	declared size of NAMES (input).
Character*260 NAMES (NSTR)	names of all libraries in the search list (output).
Integer NRET	number of libraries returned (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.17. GETMODELPROGRAMS

Return the list dynamically (i.e. run-time) loaded dynamics model programs currently in use.
In particular this will include all user-written dynamics model programs.

This API routine was first introduced in release 33.3.0.

Python command syntax:

```
progs,libs,args = getmodelprograms()
```

Fortran command syntax:

```
CALL GETMODELPROGRAMS(NSIZ,PROGS,LIBS,ARGS,PCNT)
```

Where:

Integer NSIZ	declared size of PROGS and LIBS (input).
Character*260 PROGS (NSIZ)	(output).
Character*260 LIBS (NSIZ)	names of the libraries each program has been loaded from; the array index of program can be used to identify the correcponding library in this array. (output).
Integer ARGS (NSIZ)	number of arguments for each program; the array index of program can be used to identify the correcponding entry in this array. (output).
Integer PCNT	number of programs returned (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.18. GETTSPFPROFILEBYELEMENT

Use this API to return an array of character values for the Time Series Power Flow profiles by element name.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, carray = gettspfprofilebyelement(element)
```

Fortran command syntax:

```
CALL GETTSPFPROFILEBYELEMENT(ELEMENT,COUNT,CARRAY,PDECOUNT,IERR)
```

Where:

Character*160 ELEMENT	Name of the element that profiles will be returned. (input; no default allowed).	
Integer COUNT	Number of elements in CARRAY; no default allowed	
Character*160 CARRAY (COUNT)	Is an array which contains COUNT elements (output).	
Integer PDECOUNT	Number of elements in CARRAY that are set by this API. PBECOUNT is identical to the value of PBECOUNT returned by the GETTSPFPROFILE-COUNTBYELEMENT API when it is called (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid input for non-optional field.
	IERR = 2	Internal database error.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.19. GETTSPFPROFILECOUNTBYELEMENT

Use this API to return the number of Time Series Power Flow profile by element name.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, count = gettspfprofilecountbyelement(element)
```

Fortran command syntax:

```
CALL GETTSPFPROFILECOUNTBYELEMENT(ELEMENT,COUNT,IERR)
```

Where:

Character*160 ELEMENT	Name of the element that profiles will be returned. (input; no default allowed).	
Integer COUNT	number of Time Series Power Flow profile that defined for ELEMENT. (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'COUNT' returned
	IERR = 1	Curveid not found; 'COUNT' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.20. GETTSPFPROFILEDATA

Use this API to return an array of real values for Time Series Power Flow profile data.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, rarray, pdcount = gettspfprofiledata(curvesetid,curvecol)
```

Fortran command syntax:

```
CALL GETTSPFPROFILEDATA(CURVESETID,CURVECOL,COUNT,RARRAY,PDCOUNT,IERR)
```

Where:

Integer CURVESETID	Curve ID (input; no default allowed).	
Integer CURVECOL	Curve col (input; no default allowed).	
Integer COUNT	number of elements in RARRAY. COUNT must be as least as large as the value of PDCOUNT returned by the gettspfprofiledatacount API when it is called; no default allowed	
Real RARRAY (COUNT)	Is an array of at least COUNT elements (output).	
Integer PDCOUNT	number of elements in RARRAY that are set by this API. PDCOUNT is identical to the value of PDCOUNT returned by the gettspfprofiledata-count API when it is called (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid input for non-optional field.
	IERR = 2	Internal database error.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.21. GETTSPFPROFILEDATABYSET

Use this API to return an array of real values for Time Series Power Flow profile data for given curve set id.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, rarray = gettspfprofiledatabyset(curvesetid,indexcount)
```

Fortran command syntax:

```
CALL GETTSPFPROFILEDATABYSET(CURVESETID,COUNT,INDEXCOUNT,RARRAY,IERR)
```

Where:

Integer CURVESETID	Curve ID (input; no default allowed).	
Integer COUNT	number of elements in RARRAY. COUNT must be as least as large as the value of PDCOUNT returned by the gettspfprofiledatacount API when it is called; no default allowed	
Integer INDEXCOUNT	number of elements in RARRAY. COUNT must be as least as large as the value of PDCOUNT returned by the gettspfprofiledatacount API when it is called (input; no default allowed).	
Real RARRAY (COUNT*INDEX-COUNT)	Is an array of at least COUNT elements (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid input for non-optional field.
	IERR = 2	Internal database error.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.22. GETTSPFPROFILEDATACOLUMNCOUNT

Use this API to return the number of Time Series Power Flow profile data column count for a curve set.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, count = gettspfprofiledatacolumncount(curvesetid)
```

Fortran command syntax:

```
CALL GETTSPFPROFILEDATACOLUMNCOUNT(CURVESETID,COUNT,IERR)
```

Where:

Integer CURVESETID	Curve Set ID (input; no default allowed).	
Integer COUNT	number of Time Series Power Flow data (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid input for non-optional field.
	IERR = 2	Internal database error.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.23. GETTSPFPROFILEDATACOUNT

Use this API to return the number of Time Series Power Flow profile data count.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr, count = gettspfprofiledatacount(curvesetid,curvecol)
```

Fortran command syntax:

```
CALL GETTSPFPROFILEDATACOUNT(CURVESETID,CURVECOL,COUNT,IERR)
```

Where:

Integer CURVESETID	Curve ID (input; no default allowed).	
Integer CURVECOL	Curve col (input; no default allowed).	
Integer COUNT	number of Time Series Power Flow data (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Invalid input for non-optional field.
	IERR = 2	Internal database error.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.24. LAST_PF

Return last attempted network solution data.

This API routine was first introduced in release 34.0.0.

Python command syntax:

```
method,opts = last_pf()
```

Fortran command syntax:

```
CALL LAST_PF(METHOD,OPTS)
```

Where:

Integer METHOD	last solution api routine used (output).	
	METHOD = 0	for Unknown
	METHOD = 1	for SOLV - standard Gauss-Seidel
	METHOD = 2	for MSLV - modified Gauss-Seidel
	METHOD = 3	for FNSL - full Newton-Raphson
	METHOD = 4	for NSOL - simple decoupled Newton-Raphson
	METHOD = 5	for FDNS - fixed slope decoupled Newton-Raphson
Integer OPTS (8)	array of 8 elements specifying solution parameters; values vary by solution type. For options not used by a particular solution type, a very large value is returned.(output).	
	OPTS(1)	Tap adjustment flag
	OPTS(2)	Area interchange adjustment flag
	OPTS(3)	Phase shift adjustment flag
	OPTS(4)	Dc tap adjustment flag
	OPTS(5)	Switched shunt adjustment flag
	OPTS(6)	Flat start flag
	OPTS(7)	Var limit flag
	OPTS(8)	Non-divergent solution flag

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.25. RXPATH

Use this API to return buses from starting bus to ending bus with shortest electricity distance in terms of one of two options, impedance and number of buses. The total values do not include line charging, off-nominal turns ratio, phase shifter angle etc.

This API routine was first introduced in release 34.8.0 and 35.1.0.

Python command syntax:

```
ierr, iarray, rxvalue = rxpath(iopt, ifrom, itobus, tdim)
```

Fortran command syntax:

```
CALL FINDSHORTESTDISTANCEBETWEENTWOBUS(IOPT, IFROM, ITOBUS, TDIM, IARRAY,
 RXVALUE, IERR)
```

Where:

Integer IOPT	the option to calculate the electricity distance (input; 0 by default).	
	IOPT = 0	impedance.
	IOPT = 1	number of buses.
Integer IFROM	starting bus (input; 1 by default).	
Integer ITOBUS	ending bus (input; 1 by default).	
Integer TDIM	number of buses along the path (input; 1 by default).	
Integer IARRAY (TDIM)	the list of buses from starting bus to ending bus (output).	
real RXVALUE	total value defined by IOPT from starting bus to ending bus (output).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Working case is empty
	IERR = 2	Invalid starting bus number
	IERR = 3	Invalid ending bus number
	IERR = 4	TDIM is zero
	IERR = 5	Number of buses along the path is greater than TDIM

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.26. SWD_RATINGS

Use this API to get switching device table rating set

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr,ratings = swd_ratings(rname)
```

Fortran command syntax:

```
CALL SWD_RATINGS(RNAME,RATINGS,IERR)
```

Where:

Character*40 RNAME	Name of the switching device rating table for which the required rating has to be retrived. (input; no default allowed).	
Real RATINGS (12)	The returned array containing 12 elements of the requested switching device rating table entry. (output; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	NO ERROR; 'RATING', RETURNED
	IERR = 1	INVALID RATING TABLE NAME; 'RATE', DOES NOT EXIST

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

18.27. TREEDAT

Use this API to return Swing Bus and Island Data of Working Case Network.

This API routine was first introduced in release 33.5.1.

Python command syntax:

```
treeobj = treedat(sizeislands)
```

Fortran command syntax:

```
CALL TREEDAT(NBUS, SIZEISLANDS, NSWINGBUS, NISLANDS, NB_ISLANDS,
 NP_ISLANDS, BUSNUMXX, BUSNAMEXX, BUSKVXX, BUSAREAXX, GENPQXX, LOADXX,
 SHUNTXX, IERR)
```

Where:

Integer NBUS	number of buses in the case (input; no default allowed).
Integer SIZEISLANDS	number of (possible) islands (input; 20 by default).
Integer NSWINGBUS	number of swing buses in the case (output).
Integer NISLANDS	number of actual islands (output).
Integer NB_ISLANDS (SIZEISLANDS)	number of buses in each island (output).
Integer NP_ISLANDS (SIZEISLANDS)	number of plant buses in each island (output).
Integer BUSNUMXX (NBUS)	bus number array, composed from swing + islanded buses, (1:n_swbus) -- swing buses, (n_swbus+1,n_swbus+nb_islands(1)) -- buses in island 1 and so on (output).
Character*12 BUSNAMEXX (NBUS)	bus name array, composed same way as BUSNUMXX (output).
Real BUSKVXX (NBUS)	bus base kV array, composed same way as BUSNUMXX (output).
Integer BUSAREAXX (NBUS)	bus area array, composed same way as BUSNUMXX (output).
Real GENPQXX (SIZEISLANDS*4)	pgen, qgen, qt, qb values for each island (output).
Complex LOADXX (SIZEISLANDS*3)	(PQ, IL, YL) loads for each island (output).
Complex SHUNTXX (SIZEISLANDS)	shunts for each island (output).
Integer TREEOBJ	Returned dictionary object. (output).
Integer IERR	error code (output).

IERR = 0	no error occurred
IERR = 1	Warning: All islands data is not fetched. Re-run the activity with argument sizeislands value greater than provided value
IERR = 2	Error: Working case is empty
IERR = 3	Error: No sizeislands input value provided
IERR = 4	Error: Invalid sizeislands input value, must be greater than 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

The returned object 'robj' contains the following attributes:
robj['ierr'] --> Error Code
= 0, No error
= 1, All islands data is not fetched. Re-run the activity
with higher argument sizeislands value.
= 2, Working case is empty.
= 3, No sizeislands input value provided.
= 4, Invalid sizeislands input value, must be greater than 0.
Following values are not returned for robj['ierr']=2,3,4.
robj['nswingbus']: Number of Swing Buses
robj['nislands']: Number of Islands
robj['island_nbus']: [ ] of size nislands, Number of buses in each island
robj['island_nplants']: [ ] of size nislands, Number of plant buses in each island
robj['swing_busnum']: [ ] of size nswingbus, Swing Bus Numbers
robj['swing_busname']: [ ] of size nswingbus, Swing Bus Names
robj['swing_buskv']: [ ] of size nswingbus, Swing Bus Base kV
robj['swing_busarea']: [ ] of size nswingbus, Swing Bus Area
robj['island_busnum']: [ ] of size nislands, each item is [ ] of Island Bus Numbers
robj['island_busname']: [ ] of size nislands, each item is [ ] of Island Bus Names
robj['island_buskv']: [ ] of size nislands, each item is [ ] of Island Bus Base kV
robj['island_busarea']: [ ] of size nislands, each item is [ ] of Island Bus Area
robj['island_plants']: [ ] of size nislands, each item is [ ] of Island Plant Bus Numbers
robj['island_pgen']: [ ] of size nislands, Total Pgen of each Island (MVA)
robj['island_qgen']: [ ] of size nislands, Total Qgen of each Island (MVAR)
robj['island_qt']: [ ] of size nislands, Total QT of each Island (maximum Q limit) (MVAR)
robj['island_qb']: [ ] of size nislands, Total QB of each Island (minimum Q limit) (MVAR)
robj['island_load_pq']: [ ] of size nislands, Total Constant MVA (PL+jQL) loads of each Island (MVA)
robj['island_load_il']: [ ] of size nislands, Total Constant Current (IP+jIQ) loads of each Island (MVA)
robj['island_load_yl']: [ ] of size nislands, Total Constant admittance (YP+jYQ) loads of each Island (MVA)
robj['island_shunt']: [ ] of size nislands, Total Bus Shunts (G+jB) of each Island (MVA)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 19
Using the Vendor Neutral Contingency and RAS Auxiliary File

This chapter defines API routines that work with the auxilary contingency/RAS file, a vendor neutral format for providing these specifications.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.1. ACCC_RAS

This API routine is obsolete. It has been replaced by the API routine accc_ras_2 .

Use this API to run AC contingency calculation function with RAS model.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_ACCC_RAS ACCFILE
```

Python command syntax:

```
ierr = accc_ras(accfile)
```

Fortran command syntax:

```
CALL ACCC_RAS(ACCFILE, IERR)
```

Where:

Character*260 ACCFILE	Is the name of the Contingency Solution Output file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid tolerance value from RAS file
	IERR = 2	invalid solution option value from RAS file
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 7	too many islands in base case
	IERR = 9	no AC Contingency Solution Output File specified
	IERR = 10	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 11	buses with bus type code and series element status inconsistencies
	IERR = 16	Contingency and Remedial Actions definitions not in memory
	IERR = 17	Contingency and Remedial Actions definitions have not been validated
	IERR = 51	error opening Contingency Solution Output File
	IERR = 56	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.2. ACCC_RAS_2

Use this API to run AC contingency calculation function with RAS model.

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ACCC_RAS_2 ACCFILE XMLFILE OPTION
```

Python command syntax:

```
ierr = accc_ras_2(accfile,xmlfile,option)
```

Fortran command syntax:

```
CALL ACCC_RAS_2(ACCFILE, XMLFILE, OPTION, IERR)
```

Where:

Character*260 ACCFILE	Is the name of the Contingency Solution Output file (input; no default allowed).	
Character*260 XMLFILE	Is the name of the Apply Tracing Output file (input; no default allowed).	
Integer OPTION	apply tracing option (input; 0 by default).	
	OPTION = 0	Do not create Apply Tracing file.
	OPTION = 1	Create Apply Tracing file.
	OPTION = 2	If Apply Tracing file fails to open, write output to current progress device.
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	invalid tolerance value from RAS file
	IERR = 2	invalid solution option value from RAS file
	IERR = 3	generators are converted
	IERR = 4	buses in island(s) without a swing bus; use activity TREE
	IERR = 5	largest mismatch exceeds mismatch tolerance
	IERR = 7	too many islands in base case
	IERR = 9	no AC Contingency Solution Output File specified
	IERR = 10	in-service induction machines are in the "stalled" or "tripped" state
	IERR = 11	buses with bus type code and series element status inconsistencies
	IERR = 16	Contingency and Remedial Actions definitions not in memory
	IERR = 17	Contingency and Remedial Actions definitions have not been validated

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 51	error opening Contingency Solution Output File
IERR = 56	prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.3. ADD2WINDINGCONDITIONELEMENT

Add 2WindingConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADD2WINDINGCONDITIONELEMENT IBUS JBUS CKT CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = add2windingconditionelement(ibus, jbus, ckt, condname, condnum,
 objfield, comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADD2WINDINGCONDITIONELEMENT_NA (IBUS_I, JBUS_C, CKT, CONDNAME,
 CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD2WINDINGCONDITIONELEMENT_AN (IBUS_C, JBUS_I, CKT, CONDNAME,
 CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD2WINDINGCONDITIONELEMENT_AA (IBUS_C, JBUS_C, CKT, CONDNAME,
 CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD2WINDINGCONDITIONELEMENT_NN (IBUS_I, JBUS_I, CKT, CONDNAME,
 CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO). " YES (3 characters). " NO (2 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.4. ADD2WINDINGCONTINGENCYELEMENT

Add 2WindingContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADD2WINDINGCONTINGENCYELEMENT IBUS JBUS CKT CTGNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = add2windingcontingencyelement(ibus, jbus, ckt, ctgname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADD2WINDINGCONTINGENCYELEMENT_NNV (IBUS_I, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADD2WINDINGCONTINGENCYELEMENT_NAR (IBUS_I, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADD2WINDINGCONTINGENCYELEMENT_NAV (IBUS_I, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADD2WINDINGCONTINGENCYELEMENT_ANR (IBUS_C, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADD2WINDINGCONTINGENCYELEMENT_ANV (IBUS_C, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADD2WINDINGCONTINGENCYELEMENT_AAR (IBUS_C, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADD2WINDINGCONTINGENCYELEMENT_AAV (IBUS_C, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADD2WINDINGCONTINGENCYELEMENT_NNR (IBUS_I, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGY-CHECK and POSTCHECK solution steps. (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.5. ADD2WINDINGREMEDIALACTIONELEMENT

Add 2WindingRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADD2WINDINGREMEDIALACTIONELEMENT IBUS JBUS CKT RACTNAME ACTION
 ACTVALUE ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
 INCLUSIONFILTER ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = add2windingremedialactionelement(ibus, jbus, ckt, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADD2WINDINGREMEDIALACTIONELEMENT_NNV (IBUS_I, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD2WINDINGREMEDIALACTIONELEMENT_NAR (IBUS_I, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD2WINDINGREMEDIALACTIONELEMENT_NAV (IBUS_I, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD2WINDINGREMEDIALACTIONELEMENT_ANR (IBUS_C, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD2WINDINGREMEDIALACTIONELEMENT_ANV (IBUS_C, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD2WINDINGREMEDIALACTIONELEMENT_AAR (IBUS_C, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD2WINDINGREMEDIALACTIONELEMENT_AAV (IBUS_C, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD2WINDINGREMEDIALACTIONELEMENT_NNR (IBUS_I, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGY-CHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.6. ADD3WINDINGCONDITIONELEMENT

Add 3WindingConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADD3WINDINGCONDITIONELEMENT IBUS JBUS KBUS CKT CONDNAME CONDNUM
 OBJFIELD COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = add3windingconditionelement(ibus, jbus, kbus, ckt, condname,
 condnum, objfield, comparelogic, value, othervalue, absolute,
 objectfieldevalref)
```

Fortran command syntax:

```
CALL ADD3WINDINGCONDITIONELEMENT_NNA (IBUS_I, JBUS_I, KBUS_C, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD3WINDINGCONDITIONELEMENT_NAN (IBUS_I, JBUS_C, KBUS_I, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD3WINDINGCONDITIONELEMENT_NAA (IBUS_I, JBUS_C, KBUS_C, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD3WINDINGCONDITIONELEMENT_ANN (IBUS_C, JBUS_I, KBUS_I, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD3WINDINGCONDITIONELEMENT_ANA (IBUS_C, JBUS_I, KBUS_C, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD3WINDINGCONDITIONELEMENT_AAN (IBUS_C, JBUS_C, KBUS_I, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD3WINDINGCONDITIONELEMENT_AAA (IBUS_C, JBUS_C, KBUS_C, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)

CALL ADD3WINDINGCONDITIONELEMENT_NNN (IBUS_I, JBUS_I, KBUS_I, CKT,
 CONDNAME, CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)
```

Where:

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 KBUS	A union type describing the network element KBUS value. KBUS_I refers to Integer, KBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO). " YES (3 characters). " NO (2 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.7. ADD3WINDINGCONTINGENCYELEMENT

Add 3WindingContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADD3WINDINGCONTINGENCYELEMENT IBUS JBUS KBUS CKT CTGNAME ACTION
 ACTVALUE ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = add3windingcontingencyelement(ibus, jbus, kbus, ckt, ctgname,
 action, actvalue, actmodif, critname, critstat, persistent, comment,
 timedelay)
```

Fortran command syntax:

```
CALL ADD3WINDINGCONTINGENCYELEMENT_NNNV (IBUS_I, JBUS_I, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_NNAR (IBUS_I, JBUS_I, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_NNAV (IBUS_I, JBUS_I, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_NANR (IBUS_I, JBUS_C, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_NANV (IBUS_I, JBUS_C, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_NAAR (IBUS_I, JBUS_C, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_NAAV (IBUS_I, JBUS_C, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_ANNR (IBUS_C, JBUS_I, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_ANNV (IBUS_C, JBUS_I, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

```
CALL ADD3WINDINGCONTINGENCYELEMENT_ANAR (IBUS_C, JBUS_I, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_ANAV (IBUS_C, JBUS_I, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_AANR (IBUS_C, JBUS_C, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_AANV (IBUS_C, JBUS_C, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_AAAR (IBUS_C, JBUS_C, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_AAAV (IBUS_C, JBUS_C, KBUS_C, CKT,
 CTGNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)

CALL ADD3WINDINGCONTINGENCYELEMENT_NNNR (IBUS_I, JBUS_I, KBUS_I, CKT,
 CTGNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 KBUS	A union type describing the network element KBUS value. KBUS_I refers to Integer, KBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.8. ADD3WINDINGREMEDIALACTIONELEMENT

Add 3WindingRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADD3WINDINGREMEDIALACTIONELEMENT IBUS JBUS KBUS CKT RACTNAME ACTION
 ACTVALUE ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
 INCLUSIONFILTER ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = add3windingremedialactionelement(ibus, jbus, kbus, ckt, ractname,
 action, actvalue, actmodif, critname, critstat, persistent, comment,
 timedelay, inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADD3WINDINGREMEDIALACTIONELEMENT_NNNV (IBUS_I, JBUS_I, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_NNAR (IBUS_I, JBUS_I, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_NNAV (IBUS_I, JBUS_I, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_NANR (IBUS_I, JBUS_C, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_NANV (IBUS_I, JBUS_C, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_NAAR (IBUS_I, JBUS_C, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_NAAV (IBUS_I, JBUS_C, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_ANNR (IBUS_C, JBUS_I, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_ANNV (IBUS_C, JBUS_I, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

```
CALL ADD3WINDINGREMEDIALACTIONELEMENT_ANAR (IBUS_C, JBUS_I, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_ANAV (IBUS_C, JBUS_I, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_AANR (IBUS_C, JBUS_C, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_AANV (IBUS_C, JBUS_C, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_AAAR (IBUS_C, JBUS_C, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_AAAV (IBUS_C, JBUS_C, KBUS_C, CKT,
 RACTNAME, ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADD3WINDINGREMEDIALACTIONELEMENT_NNNR (IBUS_I, JBUS_I, KBUS_I, CKT,
 RACTNAME, ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT,
 COMMENT, TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 KBUS	A union type describing the network element KBUS value. KBUS_I refers to Integer, KBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).	
	"	Always (6 characters).
	"	Never (5 characters).
	"	Check (5 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.9. ADDACLINECONDITIONELEMENT

Add AclineConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDACLINECONDITIONELEMENT IBUS JBUS CKT CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addaclineconditionelement(ibus, jbus, ckt, condname, condnum,
 objfield, comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDACLINECONDITIONELEMENT_NA (IBUS_I, JBUS_C, CKT, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)

CALL ADDACLINECONDITIONELEMENT_AN (IBUS_C, JBUS_I, CKT, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)

CALL ADDACLINECONDITIONELEMENT_AA (IBUS_C, JBUS_C, CKT, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)

CALL ADDACLINECONDITIONELEMENT_NN (IBUS_I, JBUS_I, CKT, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO). " YES (3 characters). " NO (2 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.10. ADDACLINECONTINGENCYELEMENT

Add AclineContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDACLINECONTINGENCYELEMENT IBUS JBUS CKT CTGNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addaclinecontingencyelement(ibus, jbus, ckt, ctgname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDACLINECONTINGENCYELEMENT_NNV (IBUS_I, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADDACLINECONTINGENCYELEMENT_NAR (IBUS_I, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADDACLINECONTINGENCYELEMENT_NAV (IBUS_I, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADDACLINECONTINGENCYELEMENT_ANR (IBUS_C, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADDACLINECONTINGENCYELEMENT_ANV (IBUS_C, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADDACLINECONTINGENCYELEMENT_AAR (IBUS_C, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADDACLINECONTINGENCYELEMENT_AAV (IBUS_C, JBUS_C, CKT, CTGNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)

CALL ADDACLINECONTINGENCYELEMENT_NNR (IBUS_I, JBUS_I, CKT, CTGNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGY-CHECK and POSTCHECK solution steps. (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.11. ADDACLINEREMEDIALACTIONELEMENT

Add AclineRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDACLINEREMEDIALACTIONELEMENT IBUS JBUS CKT RACTNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER
 ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = addaclineremedialactionelement(ibus, jbus, ckt, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDACLINEREMEDIALACTIONELEMENT_NNV (IBUS_I, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDACLINEREMEDIALACTIONELEMENT_NAR (IBUS_I, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDACLINEREMEDIALACTIONELEMENT_NAV (IBUS_I, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDACLINEREMEDIALACTIONELEMENT_ANR (IBUS_C, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDACLINEREMEDIALACTIONELEMENT_ANV (IBUS_C, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDACLINEREMEDIALACTIONELEMENT_AAR (IBUS_C, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDACLINEREMEDIALACTIONELEMENT_AAV (IBUS_C, JBUS_C, CKT, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDACLINEREMEDIALACTIONELEMENT_NNR (IBUS_I, JBUS_I, CKT, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Integer or Character*18 JBUS	A union type describing the network element JBUS value. JBUS_I refers to Integer, JBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CKT	A string describing the network element circuit id. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGY-CHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.12. ADDBUSCONDITIONELEMENT

Add BusConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDBUSCONDITIONELEMENT IBUS CONDNAME CONDNUM OBJFIELD COMPARELOGIC
 VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addbusconditionelement(ibus, condname, condnum, objfield,
 comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDBUSCONDITIONELEMENT_A (IBUS_C, CONDNAME, CONDNUM, OBJFIELD,
 COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF, IERR)

CALL ADDBUSCONDITIONELEMENT_N (IBUS_I, CONDNAME, CONDNUM, OBJFIELD,
 COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	"	YES (3 characters).
	"	NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.13. ADDBUSCONTINGENCYELEMENT

Add BusContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDBUSCONTINGENCYELEMENT IBUS CTGNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addbuscontingencyelement(ibus, ctgname, action, actvalue, actmodif,
 critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDBUSCONTINGENCYELEMENT_NV (IBUS_I, CTGNAME, ACTION, ACTVALUE_C,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)

CALL ADDBUSCONTINGENCYELEMENT_AR (IBUS_C, CTGNAME, ACTION, ACTVALUE_R,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)

CALL ADDBUSCONTINGENCYELEMENT_AV (IBUS_C, CTGNAME, ACTION, ACTVALUE_C,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)

CALL ADDBUSCONTINGENCYELEMENT_NR (IBUS_I, CTGNAME, ACTION, ACTVALUE_R,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	"	Always (6 characters).
	"	Never (5 characters).
	"	Check (5 characters).
	"	TopologyCheck (13 characters).
	"	PostCheck (9 characters).
	"	SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).	
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.14. ADDBUSREMEDIALACTIONELEMENT

Add BusRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDBUSREMEDIALACTIONELEMENT IBUS RACTNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER ARMINGCRIT
 ARMINGSTAT
```

Python command syntax:

```
ierr = addbusremedialactionelement(ibus, ractname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDBUSREMEDIALACTIONELEMENT_NV (IBUS_I, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDBUSREMEDIALACTIONELEMENT_AR (IBUS_C, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDBUSREMEDIALACTIONELEMENT_AV (IBUS_C, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDBUSREMEDIALACTIONELEMENT_NR (IBUS_I, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).	
	"	Always (6 characters).
	"	Never (5 characters).
	"	Check (5 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.15. ADDCONDITION

Add Condition to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDCONDITION CONDNAME OBJECT CONDLOGIC EVALUATEINREF
 DISABLEIFTRUEINREF TIMEDELAY MEMO
```

Python command syntax:

```
ierr = addcondition(condname, object, condlogic, evaluateinref,
 disableiftrueinref, timedelay, memo)
```

Fortran command syntax:

```
CALL ADDCONDITION(CONDNAME, OBJECT, CONDLOGIC, EVALUATEINREF,
 DISABLEIFTRUEINREF, TIMEDELAY, MEMO, IERR)
```

Where:

Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Character*160 OBJECT	(input; blank by default).
Character*160 CONDLOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE) (input; no default allowed).
Character*3 EVALUATEINREF	Set to YES to cause the model condition to evaluate itself in the reference case and never reevaluate during a contingency solution (input; NO). " YES (3 characters). " NO (2 characters).
Character*3 DISABLEIFTRUEINREF	Set to YES to disable this model condition for the entire contingency run if it evaluated TRUE in the reference (this is used so that a model condition can be created that checks the status of a branch being opened, but you want to restrict it to triggering an event only when the branch is opened as a RESULT of the contingency). (See Model Filter explanation in Section 5.13.1). (input; NO). " YES (3 characters). " NO (2 characters).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGY-CHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 MEMO	Extra string enclosed in quotes (input; blank by default).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	No error occurred
IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.16. ADDCONDITIONELEMENT

Add ConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDCONDITIONELEMENT ELMTTYPE ELMTKEY CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addconditionelement(elmttype, elmtkey, condname, condnum, objfield,
 comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDCONDITIONELEMENT(ELMTTYPE, ELMTKEY, CONDNAME, CONDNUM, OBJFIELD,
 COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF, IERR)
```

Where:

Character*160 ELMTTYPE	Type of the Contingency Element (input; no default allowed).
Character*160 ELMTKEY	Object field of the Contingency Element. For Python, enter parameters as modulename, funcname and path respectively. (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	"	NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.17. ADDCONTINGENCY

Add Contingency to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDCONTINGENCY CTGNAME CATEGORY SKIP MEMO
```

Python command syntax:

```
ierr = addcontingency(ctgname, category, skip, memo)
```

Fortran command syntax:

```
CALL ADDCONTINGENCY(CTGNAME, CATEGORY, SKIP, MEMO, IERR)
```

Where:

Character*160 CTGNAME	Unique name of the Contingency. (input; no default allowed).	
Character*160 CATEGORY	A comma delimited list of strings representing the contingency categories assigned to this contingency. (input; blank by default).	
Character*3 SKIP	Set to "YES" and a contingency processor will ignore this contingency. (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Character*160 MEMO	A comment field. (input; blank by default).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.18. ADDCONTINGENCYELEMENT

Add ContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDCONTINGENCYELEMENT ELMTTYPE ELMTKEY CTGNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addcontingencyelement(elmttype, elmtkey, ctgname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDCONTINGENCYELEMENT_V (ELMTTYPE, ELMTKEY, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDCONTINGENCYELEMENT_R (ELMTTYPE, ELMTKEY, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)
```

Where:

Character*160 ELMTTYPE	Type of the Contingency Element (input; no default allowed).
Character*160 ELMTKEY	Object field of the Contingency Element. For Python, enter parameters as modulename, funcname and path respectively. (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	"	Check (5 characters).
	"	TopologyCheck (13 characters).
	"	PostCheck (9 characters).
	"	SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).	
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.19. ADDDCCONVCONDITIONELEMENT

Add DcconvConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDDCCONVCONDITIONELEMENT DCNAME RECT CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = adddcconvconditionelement(dcname, rect, condname, condnum,
 objfield, comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDDCCONVCONDITIONELEMENT(DCNAME, RECT, CONDNAME, CONDNUM, OBJFIELD,
 COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 RECT	A string describing the network element rect. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	"	NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.20. ADDDCCONVCONTINGENCYELEMENT

Add DcconvContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDDCCONVCONTINGENCYELEMENT DCNAME RECT CTGNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = adddcconvcontingencyelement(dcname, rect, ctgname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDDCCONVCONTINGENCYELEMENT_V (DCNAME, RECT, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDDCCONVCONTINGENCYELEMENT_R (DCNAME, RECT, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 RECT	A string describing the network element rect. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	"	Check (5 characters).
	"	TopologyCheck (13 characters).
	"	PostCheck (9 characters).
	"	SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).	
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.21. ADDDCCONVREMEDIALACTIONELEMENT

Add DcconvRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDDCCONVREMEDIALACTIONELEMENT DCNAME RECT RACTNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER
 ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = adddcconvremedialactionelement(dcname, rect, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDDCCONVREMEDIALACTIONELEMENT_V (DCNAME, RECT, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDDCCONVREMEDIALACTIONELEMENT_R (DCNAME, RECT, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 RECT	A string describing the network element rect. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	" Always (6 characters).
	" Never (5 characters).
	" Check (5 characters).
	" TopologyCheck (13 characters).
	" PostCheck (9 characters).
	" SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO).
	" YES (3 characters).
	" NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).
	" Always (6 characters).
	" Never (5 characters).
	" Check (5 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.22. ADDDCLINECONDITIONELEMENT

Add DclineConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDDCLINECONDITIONELEMENT DCNAME CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = adddclineconditionelement(dcname, condname, condnum, objfield,
 comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDDCLINECONDITIONELEMENT(DCNAME, CONDNAME, CONDNUM, OBJFIELD,
 COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.23. ADDDCLINECONTINGENCYELEMENT

Add DclineContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDDCLINECONTINGENCYELEMENT DCNAME CTGNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = adddclinecontingencyelement(dcname, ctgname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDDCLINECONTINGENCYELEMENT_V (DCNAME, CTGNAME, ACTION, ACTVALUE_C,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)

CALL ADDDCLINECONTINGENCYELEMENT_R (DCNAME, CTGNAME, ACTION, ACTVALUE_R,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.24. ADDDCLINEREMEDIALACTIONELEMENT

Add DclineRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDDCLINEREMEDIALACTIONELEMENT DCNAME RACTNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER
 ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = adddclineremedialactionelement(dcname, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDDCLINEREMEDIALACTIONELEMENT_V (DCNAME, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDDCLINEREMEDIALACTIONELEMENT_R (DCNAME, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	" Check (5 characters).
	" TopologyCheck (13 characters).
	" PostCheck (9 characters).
	" SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO).
	" YES (3 characters).
	" NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).
	" Always (6 characters).
	" Never (5 characters).
	" Check (5 characters).
Integer IERR	error code (output).
	IERR = 0 No error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.25. ADDFIXSHUNTCONDITIONELEMENT

Add FixshuntConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDFIXSHUNTCONDITIONELEMENT IBUS SHNTID CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addfixshuntconditionelement(ibus, shntid, condname, condnum,
 objfield, comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDFIXSHUNTCONDITIONELEMENT_A (IBUS_C, SHNTID, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)

CALL ADDFIXSHUNTCONDITIONELEMENT_N (IBUS_I, SHNTID, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 SHNTID	A string describing the network element fix shunt id. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression :

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO). " YES (3 characters). " NO (2 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.26. ADDFIXSHUNTCONTINGENCYELEMENT

Add FixshuntContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDFIXSHUNTCONTINGENCYELEMENT IBUS SHNTID CTGNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addfixshuntcontingencyelement(ibus, shntid, ctgname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDFIXSHUNTCONTINGENCYELEMENT_NV (IBUS_I, SHNTID, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDFIXSHUNTCONTINGENCYELEMENT_AR (IBUS_C, SHNTID, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDFIXSHUNTCONTINGENCYELEMENT_AV (IBUS_C, SHNTID, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDFIXSHUNTCONTINGENCYELEMENT_NR (IBUS_I, SHNTID, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 SHNTID	A string describing the network element fix shunt id. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.27. ADDFIXSHUNTREMEDIALACTIONELEMENT

Add FixshuntRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDFIXSHUNTREMEDIALACTIONELEMENT IBUS SHNTID RACTNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER
 ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = addfixshuntremedialactionelement(ibus, shntid, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDFIXSHUNTREMEDIALACTIONELEMENT_NV (IBUS_I, SHNTID, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDFIXSHUNTREMEDIALACTIONELEMENT_AR (IBUS_C, SHNTID, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDFIXSHUNTREMEDIALACTIONELEMENT_AV (IBUS_C, SHNTID, RACTNAME,
 ACTION, ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDFIXSHUNTREMEDIALACTIONELEMENT_NR (IBUS_I, SHNTID, RACTNAME,
 ACTION, ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT,
 TIMEDELAY, INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 SHNTID	A string describing the network element fix shunt id. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.28. ADDGENCONDITIONELEMENT

Add GenConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDGENCONDITIONELEMENT IBUS MACHID CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addgenconditionelement(ibus, machid, condname, condnum, objfield,
 comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDGENCONDITIONELEMENT_A (IBUS_C, MACHID, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)

CALL ADDGENCONDITIONELEMENT_N (IBUS_I, MACHID, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 MACHID	A string describing the network element machine id. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression :

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO). " YES (3 characters). " NO (2 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.29. ADDGENCONTINGENCYELEMENT

Add GenContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDGENCONTINGENCYELEMENT IBUS MACHID CTGNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addgencontingencyelement(ibus, machid, ctgname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDGENCONTINGENCYELEMENT_NV (IBUS_I, MACHID, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDGENCONTINGENCYELEMENT_AR (IBUS_C, MACHID, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDGENCONTINGENCYELEMENT_AV (IBUS_C, MACHID, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDGENCONTINGENCYELEMENT_NR (IBUS_I, MACHID, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 MACHID	A string describing the network element machine id. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.30. ADDGENREMEDIALACTIONELEMENT

Add GenRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDGENREMEDIALACTIONELEMENT IBUS MACHID RACTNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER
 ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = addgenremedialactionelement(ibus, machid, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDGENREMEDIALACTIONELEMENT_NV (IBUS_I, MACHID, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDGENREMEDIALACTIONELEMENT_AR (IBUS_C, MACHID, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDGENREMEDIALACTIONELEMENT_AV (IBUS_C, MACHID, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDGENREMEDIALACTIONELEMENT_NR (IBUS_I, MACHID, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 MACHID	A string describing the network element machine id. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).	
	"	Always (6 characters).
	"	Never (5 characters).
	"	Check (5 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.31. ADDLOADCONDITIONELEMENT

Add LoadConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDLOADCONDITIONELEMENT IBUS LOADID CONDNAME CONDNUM OBJFIELD
 COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addloadconditionelement(ibus, loadid, condname, condnum, objfield,
 comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDLOADCONDITIONELEMENT_A (IBUS_C, LOADID, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)

CALL ADDLOADCONDITIONELEMENT_N (IBUS_I, LOADID, CONDNAME, CONDNUM,
 OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF,
 IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 LOADID	A string describing the network element load id. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression :

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO). " YES (3 characters). " NO (2 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.32. ADDLOADCONTINGENCYELEMENT

Add LoadContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDLOADCONTINGENCYELEMENT IBUS LOADID CTGNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addloadcontingencyelement(ibus, loadid, ctgname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDLOADCONTINGENCYELEMENT_NV (IBUS_I, LOADID, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDLOADCONTINGENCYELEMENT_AR (IBUS_C, LOADID, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDLOADCONTINGENCYELEMENT_AV (IBUS_C, LOADID, CTGNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)

CALL ADDLOADCONTINGENCYELEMENT_NR (IBUS_I, LOADID, CTGNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 LOADID	A string describing the network element load id. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.33. ADDLOADREMEDIALACTIONELEMENT

Add LoadRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDLOADREMEDIALACTIONELEMENT IBUS LOADID RACTNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER
 ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = addloadremedialactionelement(ibus, loadid, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDLOADREMEDIALACTIONELEMENT_NV (IBUS_I, LOADID, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDLOADREMEDIALACTIONELEMENT_AR (IBUS_C, LOADID, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDLOADREMEDIALACTIONELEMENT_AV (IBUS_C, LOADID, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDLOADREMEDIALACTIONELEMENT_NR (IBUS_I, LOADID, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Integer or Character*18 IBUS	A union type describing the network element IBUS value. IBUS_I refers to Integer, IBUS_C refers to Character value in the Fortran syntax. (Key) (input; no default allowed).
Character*160 LOADID	A string describing the network element load id. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).	
	"	Always (6 characters).
	"	Never (5 characters).
	"	Check (5 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.34. ADDNESTEDCONDITION

Add NestedCondition to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDNESTEDCONDITION CONDNAME PARENT
```

Python command syntax:

```
ierr = addnestedcondition(condname, parent)
```

Fortran command syntax:

```
CALL ADDNESTEDCONDITION(CONDNAME, PARENT, IERR)
```

Where:

Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).	
Character*160 PARENT	(input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.35. ADDPYTHONCONDITIONELEMENT

Add PythonConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDPYTHONCONDITIONELEMENT PATH MODULENAME FUNCNAME CONDNAME CONDNUM
 OBJFIELD COMPARELOGIC VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addpythonconditionelement(path, modulename, funcname, condname,
 condnum, objfield, comparelogic, value, othervalue, absolute,
 objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDPYTHONCONDITIONELEMENT(PATH, MODULENAME, FUNCNAME, CONDNAME,
 CONDNUM, OBJFIELD, COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE,
 OBJECTFIELDEVALREF, IERR)
```

Where:

Character*160 PATH	Path of the module (input; blank by default).
Character*160 MODULENAME	Module Name of the element (input; no default allowed).
Character*160 FUNCNAME	Function Name of the element (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	"	YES (3 characters).
	"	NO (2 characters).
Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.36. ADDPYTHONCONTINGENCYELEMENT

Add PythonContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDPYTHONCONTINGENCYELEMENT PATH MODULENAME FUNCNAME CTGNAME CRITNAME
 CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addpythoncontingencyelement(path, modulename, funcname, ctgname,
 critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDPYTHONCONTINGENCYELEMENT(PATH, MODULENAME, FUNCNAME, CTGNAME,
 CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)
```

Where:

Character*160 PATH	Path of the module (input; blank by default).
Character*160 MODULENAME	Module Name of the element (input; no default allowed).
Character*160 FUNCNAME	Function Name of the element (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGY-CHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.37. ADDPYTHONREMEDIALACTIONELEMENT

Add PythonRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDPYTHONREMEDIALACTIONELEMENT PATH MODULENAME FUNCNAME RACTNAME
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER ARMINGCRIT
 ARMINGSTAT
```

Python command syntax:

```
ierr = addpythonremedialactionelement(path, modulename, funcname,
 ractname, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDPYTHONREMEDIALACTIONELEMENT(PATH, MODULENAME, FUNCNAME, RACTNAME,
 CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, INCLUSIONFILTER,
 ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Character*160 PATH	Path of the module (input; blank by default).
Character*160 MODULENAME	Module Name of the element (input; no default allowed).
Character*160 FUNCNAME	Function Name of the element (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLO-

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	GYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.38. ADDREMEDIALACTION

Add RemedialAction to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDREMEDIALACTION RACTNAME SKIP MEMO ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = addremedialaction(ractname, skip, memo, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDREMEDIALACTION(RACTNAME, SKIP, MEMO, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Character*160 RACTNAME	Unique name of the remedial action (input; no default allowed).
Character*3 SKIP	Set to "YES" and a remedial action processor will ignore this remedial action. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 MEMO	A comment field. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.39. ADDREMEDIALACTIONELEMENT

Add RemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDREMEDIALACTIONELEMENT ELMTTYPE ELMTKEY RACTNAME ACTION ACTVALUE
 ACTMODIF CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER
 ARMINGCRIT ARMINGSTAT
```

Python command syntax:

```
ierr = addremedialactionelement(elmttype, elmtkey, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDREMEDIALACTIONELEMENT_V (ELMTTYPE, ELMTKEY, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDREMEDIALACTIONELEMENT_R (ELMTTYPE, ELMTKEY, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Character*160 ELMTTYPE	Type of the Contingency Element (input; no default allowed).
Character*160 ELMTKEY	Object field of the Contingency Element. For Python, enter parameters as modulename, funcname and path respectively. (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	" Always (6 characters).
	" Never (5 characters).
	" Check (5 characters).
	" TopologyCheck (13 characters).
	" PostCheck (9 characters).
	" SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO).
	" YES (3 characters).
	" NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).
	" Always (6 characters).
	" Never (5 characters).
	" Check (5 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.40. ADDSUBCONDITIONELEMENT

Add SubConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDSUBCONDITIONELEMENT SUBID CONDNAME CONDNUM OBJFIELD COMPARELOGIC
 VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addsubconditionelement(subid, condname, condnum, objfield,
 comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDSUBCONDITIONELEMENT(SUBID, CONDNAME, CONDNUM, OBJFIELD,
 COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF, IERR)
```

Where:

Character*160 SUBID	A string describing the network element substation id. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.41. ADDSUBCONTINGENCYELEMENT

Add SubContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDSUBCONTINGENCYELEMENT SUBID CTGNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addsubcontingencyelement(subid, ctgname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDSUBCONTINGENCYELEMENT_V (SUBID, CTGNAME, ACTION, ACTVALUE_C,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)

CALL ADDSUBCONTINGENCYELEMENT_R (SUBID, CTGNAME, ACTION, ACTVALUE_R,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)
```

Where:

Character*160 SUBID	A string describing the network element substation id. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.42. ADDSUBREMEDIALACTIONELEMENT

Add SubRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDSUBREMEDIALACTIONELEMENT SUBID RACTNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER ARMINGCRIT
 ARMINGSTAT
```

Python command syntax:

```
ierr = addsubremedialactionelement(subid, ractname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDSUBREMEDIALACTIONELEMENT_V (SUBID, RACTNAME, ACTION, ACTVALUE_C,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDSUBREMEDIALACTIONELEMENT_R (SUBID, RACTNAME, ACTION, ACTVALUE_R,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Character*160 SUBID	A string describing the network element substation id. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	" Check (5 characters).
	" TopologyCheck (13 characters).
	" PostCheck (9 characters).
	" SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO).
	" YES (3 characters).
	" NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check).
	" Always (6 characters).
	" Never (5 characters).
	" Check (5 characters).
Integer IERR	error code (output).
	IERR = 0 No error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.43. ADDVSCDCCONDITIONELEMENT

Add VscdcConditionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDVSCDCCONDITIONELEMENT DCNAME CONDNAME CONDNUM OBJFIELD COMPARELOGIC
 VALUE OTHERVALUE ABSOLUTE OBJECTFIELDEVALREF
```

Python command syntax:

```
ierr = addvscdcconditionelement(dcname, condname, condnum, objfield,
 comparelogic, value, othervalue, absolute, objectfieldevalref)
```

Fortran command syntax:

```
CALL ADDVSCDCCONDITIONELEMENT(DCNAME, CONDNAME, CONDNUM, OBJFIELD,
 COMPARELOGIC, VALUE, OTHERVALUE, ABSOLUTE, OBJECTFIELDEVALREF, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 CONDNAME	Unique name of the Condition. (input; no default allowed).
Integer CONDNUM	Number specifying the number of the condition. Omitting this value, or specifying a value of zero will cause this to automatically add the condition to the existing ModelCondition. Otherwise by specifying a number you can edit existing conditions. (input; no default allowed).
Character*160 OBJFIELD	A field associated with the object to which this model condition applies. A list of fields is shown in Section 2.4. In addition to these fields, you may also specify the string _UseAnotherFilter to signify that this condition refers to another Filter. This will be important for the creation of more complex nested filter definitions as described in Section 3.2. (input; no default allowed).
Character*160 COMPARELOGIC	Logic to apply to the list of comparisons (AND, OR, NAND, NOR,X-OR,OneTRUE). (input; no default allowed).
Character*160 VALUE	Value or Field or Expression. The entry specifies the value to which the field is compared. This may be done in three ways Value : 1.459 or a String Field : start with the tag Field and then follow with a string using the format as the ObjectField string Model Expression : start with the tag Expression and then follow the name of the Model Expression (input; no default allowed).
Character*160 OTHERVALUE	(input; no default allowed).
Character*3 ABSOLUTE	Set to YES to take the absolute value of the ObjectField begin compared. (input; NO). " YES (3 characters). " NO (2 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*3 OBJECTFIELDEVALREF	Set to YES to specify that the ObjectField should be evaluated in the reference state of the contingency analysis tool. Added in Simulator Version 19, December 18, 2015 patch (input; NO).	
	"	YES (3 characters).
	"	NO (2 characters).
Integer IERR	error code (output).	
	IERR = 0	No error occurred
	IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.44. ADDVSCDCCONTINGENCYELEMENT

Add VscdcContingencyElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDVSCDCCONTINGENCYELEMENT DCNAME CTGNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY
```

Python command syntax:

```
ierr = addvscdccontingencyelement(dcname, ctgname, action, actvalue,
 actmodif, critname, critstat, persistent, comment, timedelay)
```

Fortran command syntax:

```
CALL ADDVSCDCCONTINGENCYELEMENT_V (DCNAME, CTGNAME, ACTION, ACTVALUE_C,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)

CALL ADDVSCDCCONTINGENCYELEMENT_R (DCNAME, CTGNAME, ACTION, ACTVALUE_R,
 ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 CTGNAME	Name of the contingency (or TSContingency) to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Integer IERR	error code (output). IERR = 0 No error occurred IERR = 4 Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.45. ADDVSCDCREMEDIALACTIONELEMENT

Add VscdcRemedialActionElement to RAS memory model

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_ADDVSCDCREMEDIALACTIONELEMENT DCNAME RACTNAME ACTION ACTVALUE ACTMODIF
 CRITNAME CRITSTAT PERSISTENT COMMENT TIMEDELAY INCLUSIONFILTER ARMINGCRIT
 ARMINGSTAT
```

Python command syntax:

```
ierr = addvscdcremedialactionelement(dcname, ractname, action,
 actvalue, actmodif, critname, critstat, persistent, comment, timedelay,
 inclusionfilter, armingcrit, armingstat)
```

Fortran command syntax:

```
CALL ADDVSCDCREMEDIALACTIONELEMENT_V (DCNAME, RACTNAME, ACTION,
 ACTVALUE_C, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)

CALL ADDVSCDCREMEDIALACTIONELEMENT_R (DCNAME, RACTNAME, ACTION,
 ACTVALUE_R, ACTMODIF, CRITNAME, CRITSTAT, PERSISTENT, COMMENT, TIMEDELAY,
 INCLUSIONFILTER, ARMINGCRIT, ARMINGSTAT, IERR)
```

Where:

Character*160 DCNAME	A string describing the network element dc name. (Key) (input; no default allowed).
Character*160 RACTNAME	Name of the remedial action to which this element belongs. (input; no default allowed).
Character*5 ACTION	A string describing this action. (input; no default allowed).
Real or Character*160 ACTVALUE	Value of the Action. ACTVALUE_R refers to Real, ACTVALUE_C refers to Character value in the Fortran syntax (input; blank by default).
Character*160 ACTMODIF	Modifier field of the Action (input; blank by default).
Character*160 CRITNAME	Name of either a ModelCondition or a ModelFilter that determines the situation under which the action will actually be implemented. The point at which this model criteria is evaluated will depend on the CriteriaStatus as well as whether this contingency is used for Power Flow Contingencies or Transient Stability Contingencies. (input; blank by default).
Character*13 CRITSTAT	Set to either Always, Never, Check, TopologyCheck ,PostCheck, or SolutionFail. Default value used when none specified is Check. Details are explained in Section 5.15.2. For Transient Stability Elements, the choices are either Always, Never, or CHECK (input; Check). " Always (6 characters). " Never (5 characters).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	" Check (5 characters). " TopologyCheck (13 characters). " PostCheck (9 characters). " SolutionFail (12 characters).
Character*3 PERSISTENT	This field is only defined for the Power Flow Contingency Elements. A YES or NO field. Any action that has Persistent set to YES and also has the CriteriaStatus field set to SOLUTIONFAIL, POSTCHECK or TOPOLOGYCHECK will be applied in the appropriate section of the overall contingency process any time that its Criteria is met. An exception to this is that a SOLUTIONFAIL element will only remain persistent up until a solution is successfully achieved. Once a solution is achieved then the SOLUTIONFAIL elements that had been applied will no longer be evaluated. This is done to ensure the SOLUTIONFAIL loop is not entered repeatedly. Default value used when none specified is NO. (input; NO). " YES (3 characters). " NO (2 characters).
Character*160 COMMENT	Text field that can be used to provide comments attached to the element Default value used when none specified is empty string. (input; blank by default).
Real TIMEDELAY	This field is only defined for the Power Flow Contingency Elements. Time delay to wait before the defined Action takes place specified in a real number in seconds (see note below when using a ModelFilter). This is NOT an absolute time at which an action occurs, but serves as a relative ordering for the implementation of actions. Actions with the smallest time delay will be applied first during the TOPOLOGYCHECK and POSTCHECK solution steps. (input; 0.0).
Character*160 INCLUSIONFILTER	Name of a filter or device filter (See Section 3 or 3.3) that gets applied to each contingency. If the contingency meets the Inclusion filter defined, that contingency will include this element. Otherwise, the remedial action element will be ignored. A description of the format of this string is in Section 3.4. (input; blank by default).
Character*160 ARMINGCRIT	If specified, this is the name of either a ModelCondition or a ModelFilter that determines the situation under which the action will be armed. If not specified, the action is considered to be armed. The specified condition is evaluated in the contingency reference case. For a RemedialActionElement to be armed, its RemedialAction must also be armed. (input; blank by default).
Character*6 ARMINGSTAT	Set to either Always, Never, or Check. Default value used when none specified is Check. Always -> action is considered armed regardless of the ArmingCriteria Never -> action is not armed Check -> action is armed if ArmingCriteria is true or not specified (input; Check). " Always (6 characters). " Never (5 characters). " Check (5 characters).
Integer IERR	error code (output). IERR = 0 No error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	Req are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.46. APPEND_RAS

Use this API to read a Contingency Definition and Remedial Actions auxiliary file, adding to definitions in memory, if present.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_APPEND_RAS IFILE
```

Python command syntax:

```
ierr = append_ras(ifile)
```

Fortran command syntax:

```
CALL APPEND_RAS(IFILE,IERR)
```

Where:

character*260 IFILE	Contingency Definition and Remedial Actions auxiliary file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 2	error opening IFILE
	IERR = 3	error closing IFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.47. READ_RAS

Use this API to read a Contingency Definition and Remedial Actions auxiliary file into memory.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_READ_RAS IFILE
```

Python command syntax:

```
ierr = read_ras(ifile)
```

Fortran command syntax:

```
CALL READ_RAS(IFILE,IERR)
```

Where:

character*260 IFILE	Contingency Definition and Remedial Actions auxiliary file (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 2	error opening IFILE
	IERR = 3	error closing IFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.48. VALIDATE_RAS

Use this API to validate Contingency and Remedial Actions definitions in memory.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_VALIDATE_RAS
```

Python command syntax:

```
ierr = validate_ras()
```

Fortran command syntax:

```
CALL VALIDATE_RAS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	Contingency and Remedial Actions definitions not in memory
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

19.49. WRITE_RAS

Use this API to write a Contingency Definition and Remedial Actions auxiliary file from memory.

This API routine was first introduced in release 34.6.0.

Batch command syntax:

```
BAT_WRITE_RAS IFILE OPTION(1)..OPTION(8)
```

Python command syntax:

```
ierr = write_ras(ifile,options)
```

Fortran command syntax:

```
CALL WRITE_RAS(IFILE,OPTIONS,IERR)
```

Where:

character*260 IFILE	Contingency Definition and Remedial Actions auxiliary file (input; no default allowed).	
Integer OPTIONS (8)	array of 8 elements specifying (input).	
	OPTIONS(1)	write records with bad ID fields (alias is IncludeBadId) (0 by default)
	OPTIONS(1) = 0	do not write records with bad ID fields
	OPTIONS(1) = 1	write records with bad ID fields
	OPTIONS(2)	write internal contents of fields with bad data (alias is IncludeBadData) (0 by default)
	OPTIONS(2) = 0	do not write fields with bad data
	OPTIONS(2) = 1	write all fields with data input, even if values are invalid
	OPTIONS(3)	write all defined fields (alias is includeAllFields) (0 by default)
	OPTIONS(3) = 0	write fields only if at least on record had data for that field
	OPTIONS(3) = 1	write all fields, including fields defined for RAS analysis for which no data was entered on any record
	OPTIONS(3) = 3	label
	OPTIONS(4)	(future use) write bus numbers or names (alias is busOutput) (0 by default)
	OPTIONS(4) = 0	bus number
	OPTIONS(4) = 1	bus name
	OPTIONS(4) = 2	bus name and kV
	OPTIONS(4) = 3	label
	OPTIONS(5)	future use (0 by default)
	OPTIONS(6)	future use (0 by default)
	OPTIONS(7)	future use (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(8)	future use (0 by default)
Integer IERR	error code (output).	
	IERR = 0	no error
	IERR = 1	Contingency and Remedial Actions definitions not in memory
	IERR = 2	error opening IFILE
	IERR = 3	error closing IFILE
	IERR = 4	prerequisite requirements for API are not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 20
Non-Engineering

This chapter defines API routines that provide functions not associated with PSS®E data, analysis, or graphics capabilities.

Most of these routines do not record themselves when called with recording enabled. Those that have a batch format documented can record themselves.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.1. ALERT

Use this API to send output to the alert device.
Note: A Fortran program that calls ALERTS must also include a USE statement for KEYPTI.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
alert(string)
```

Fortran command syntax:

```
CALL ALERTS(STRING)
```

```
CALL ALERTS(STRING-ARRAY)
```

Where:

Character*220 STRING	string to be written (input).
Character*220 STRING-ARRAY	array of strings to be written

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.2. APPENDRECORDING

Start recording program operation at end of specified file.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr = appendrecording(iopt,string)
```

Fortran command syntax:

```
CALL APPENDRECORDING(IOPT, STRING, IERR)
```

Where:

Integer IOPT	type of format to be recorded (input).	
	IOPT = 0	record in batch (BAT_*) format.
	IOPT = 1	record in python format.
Character*260 STRING	filename (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 2	unable to open file.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.3. BEGINREPORT

This API routine is obsolete. It has been replaced by the API routine beginreport_2 .

Use this API to create a new report tab in the GUI.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
beginreport()
```

Fortran command syntax:

```
CALL RPTBEGUSER()
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.4. BEGINREPORT_2

Use this API to create a new report tab in the GUI with name as specified.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
beginreport_2(rptnam)
```

Fortran command syntax:

```
CALL RPTBEG(RPTNAM)
```

Where:

Character RPTNAM	name of the report (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.5. CLEARALERTOUTPUT

Clear the Alerts/Warnings tab of the output bar in the GUI

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_CLEARALERTOUTPUT
```

Python command syntax:

```
clearalertoutput()
```

Fortran command syntax:

```
CALL CLEARALERTOUTPUT
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.6. CLEARPROGRESSOUTPUT

Clear the Progress tab of the output bar in the GUI

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_CLEARPROGRESSOUTPUT
```

Python command syntax:

```
clearprogressoutput()
```

Fortran command syntax:

```
CALL CLEARPROGRESSOUTPUT
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.7. FILEIN

Get input from file (Fortran unit number).

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr, string, flags = filein(lunit)
```

Where:

Integer LUNIT	Fortran logical unit number of the file (input).
Character*512 STRING	buffer read from file (output).
Integer ILEN	declared length (input).
Logical FLAGS	the first element of flags is set to true if an end-of-file is encountered; the second element of flags is set to true if the file is closed after the read attempt (output).
Integer IERR	error code (output). IERR = 0 no error occurred.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.8. GETBATDEFAULTS

Retrieve "no input" value for integers and reals.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
idef,rdef = getbatdefaults()
```

Fortran command syntax:

```
CALL GETBATDEFAULTS(IDEF, RDEF)
```

Where:

Integer IDEF	default integer value used in the batch command processor (output).
Real RDEF	default real value used in the batch command processor (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.9. GETDEFAULTCHAR

Retrieve "no input" value for characters (i.e., strings) other than filenames. Specify this value to indicate that "no input" is intended.

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
cdef = getdefaultchar()
```

Fortran command syntax:

```
CALL GETDEFAULTCHAR(CDEF)
```

Where:

Character*1 CDEF	"no character value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.10. GETDEFAULTINT

Retrieve "no input" value for integers. Specify this value to indicate that "no input" is intended.

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
idef = getdefaultint()
```

Fortran command syntax:

```
CALL GETDEFAULTINT(IDEF)
```

Where:

Integer IDEF	"no integer value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.11. GETDEFAULTREAL

Retrieve "no input" value for reals (floats). Specify this value to indicate that "no input" is intended.

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
rdef = getdefaultreal()
```

Fortran command syntax:

```
CALL GETDEFAULTREAL(RDEF)
```

Where:

Real RDEF	"no real value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.12. GETLOADEDMODULES

Get list of all modules currently loaded.

This API routine was first introduced in release 32.0.1.

Python command syntax:

```
modules = getloadedmodules()
```

Where:

Character MODULES	list of loaded Python modules (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.13. GETMODFUNCLIST

Get list of external callable methods given a module name. Module is imported in process.

This API routine was first introduced in release 32.0.0.

Python command syntax:

```
c = getmodfunclist(mod)
```

Where:

Character MOD	module name (input).
Character C	list of callable methods (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.14. IOFLUSH

Force all files in PSSE to flush buffers to disk, if possible.

This API routine was first introduced in release 30.2.1.

Python command syntax:

```
ioflush()
```

Fortran command syntax:

```
CALL IOFLSH()
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.15. LAUNCH_PROGRAM

This API routine was first introduced in release 34.1.0.

Batch command syntax:

```
BAT_LAUNCH_PROGRAM PROGRAM NUMARGS ARGS(1)..ARGS(NUMARGS)
```

Python command syntax:

```
ierr = launch_program(program, numargs, args)
```

Fortran command syntax:

```
CALL LAUNCH_PROGRAM(PROGRAM, NUMARGS, ARGS, IERR)
```

Where:

Character*260 PROGRAM	name of program to launch (input; no default allowed).
Integer NUMARGS	number of argument strings to pass to launched program (input).
Character*260 ARGS (NUMARGS)	argument strings to pass to launched program (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.16. PAGEREPORT

Use this API to place a report separator on the active report tab of the GUI.

This API routine was first introduced in release 30.2.0.

Python command syntax:

```
pagereport()
```

Fortran command syntax:

```
CALL RPTPGBK()
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.17. PAUSERECORDING

Pause or resume recording program operation.

This API routine was first introduced in release 35.4.0.

Python command syntax:

```
ierr = pauserecording(iopt)
```

Fortran command syntax:

```
CALL PAUSERECORDING(IOPT, IERR)
```

Where:

Integer IOPT	type of format to be recorded (input).	
	IOPT = 0	resume recording.
	IOPT = 1	pause recording.
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 2	not recording.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.18. PROGRESS

Send output to progress device.
Note: A Fortran program that calls PROGRESS must also include a USE statement for KEYPTI.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
progress(string)
```

Fortran command syntax:

```
CALL PROGRESS(STRING)
```

Where:

Character*220 STRING	string to be written (input).
Character*220 STRING-ARRAY	array of strings to be written

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.19. PROMPT

Send output to prompt device. Prompts are stored until the next terminal input request. Up to 36 buffers of up to 220 characters can be stored.
Note: A Fortran program that calls PROMPTS must also include a USE statement for KEYPTI.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
prompt(string)
```

Fortran command syntax:

```
CALL PROMPTS(STRING)

CALL PROMPTS(STRING-ARRAY)
```

Where:

Character*220 STRING	string to be written (input).
Character*220 STRING-ARRAY	array of strings to be written

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.20. PSSEINIT

Initialize PSSE.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr = psseinit(buses)
```

Fortran command syntax:

```
CALL PSSEINIT(BUSES, IERR)
```

Where:

Integer BUSES	requested bus size; zero for default size level (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.21. PSSEVERSION

Get PSSE version information.

This API routine was first introduced in release 30.1.0.

Python command syntax:

```
name,major,minor,update,date,stat = psseversion()
```

Fortran command syntax:

```
CALL REVPSSE(NAME, MAJOR, MINOR, UPDATE, DATE, STAT)
```

Where:

Character*32 NAME	product name (output).
Integer MAJOR	major release number (output).
Integer MINOR	minor release number (output).
Integer UPDATE	update release number (output).
Character*30 DATE	release date (output).
Character*132 STAT	any special release information (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.22. REFRESHGUI

Refresh the graphical user interface.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
refreshgui()
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.23. REPORT

Send output to report device.
Note: A Fortran program that calls REPORTS must also include a USE statement for KEYPTI.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
report(string)
```

Fortran command syntax:

```
CALL REPORTS(STRING)
```

Where:

Character*220 STRING	string to be written (input).
Character*220 STRING-ARRAY	array of strings to be written

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.24. RUNIPLANFILE

Execute an IPLAN program.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
runiplanfile(iplname)
```

Fortran command syntax:

```
CALL EXECRUN(IPLNAME)
```

Where:

Character*260 IPLNAME	the filename; the filename can be followed by arguments as part of a single string, in which case the filename must be delimited (e.g. enclosed in quotes). The -arg option can be specified this way as well. (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.25. RUNRSPNSFILE

Run a response file. Operates by replacing standard input with a file.
Then begin and continue reading from terminal until standard input is no longer redirected using PSSE command line techniques (allowing interaction with IPLAN programs, for example).
Pass any results to Command Line Interface (CLI). The CLI can recognize batch commands and Line Mode commands and direct them to the proper interpreters.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
runrspnsfile(rspname)
```

Where:

Character*260 RSPNAME	the filename; the filename can be followed by arguments as part of a single string, in which case the filename must be delimited (e.g. enclosed in quotes). (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.26. SHOWLOADEDLIBRARIES

Display list of all libraries currently loaded.

This API routine was first introduced in release 34.5.0.

Python command syntax:

```
showloadedlibraries()
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.27. SHOWLOADEDMODULES

Display list of all modules currently loaded.

This API routine was first introduced in release 32.0.2.

Python command syntax:

```
showloadedmodules()
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.28. STARTRECORDING

Start recording program operation.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr = startrecording(iopt,string)
```

Fortran command syntax:

```
CALL STARTRECORDING(IOPT, STRING, IERR)
```

Where:

Integer IOPT	type of format to be recorded (input).	
	IOPT = 0	record in batch (BAT_*) format.
	IOPT = 1	record in python format.
Character*260 STRING	filename (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 2	unable to open file.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.29. STOPRECORDING

Stop recording program operation.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr = stoprecording()
```

Fortran command syntax:

```
CALL STOPRECORDING(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	no error occurred.
	IERR = 1	error closing file.
	IERR = 2	not recording.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.30. T_ALERT_OUTPUT

Use this API to specify a "T"'d alert device, i.e. a destination for a copy of everything sent to the alert device.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_T_ALERT_OUTPUT ISLCT FILARG OPTIONS(1)..OPTIONS(2)
```

Python command syntax:

```
ierr = t_alert_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL TALERTDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for alerts is a message box and a copy to the Alerts/Warnings tab of the output bar.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only)
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	prerequisite requirements for API are not met
IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.31. T_PROGRESS_OUTPUT

Use this API to specify a "T"'d progress device, i.e. a destination for a copy of everything sent to the progress device.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_T_PROGRESS_OUTPUT ISLCT FILARG OPTIONS(1)..OPTIONS(2)
```

Python command syntax:

```
ierr = t_progress_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL TPROGRESSDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for progress is the Progress tab of the output bar.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only)
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	prerequisite requirements for API are not met
IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.32. T_PROMPT_OUTPUT

Use this API to specify a "T"'d prompt device, i.e. a destination for a copy of everything sent to the prompt device.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_T_PROMPT_OUTPUT ISLCT FILARG OPTIONS(1)..OPTIONS(2)
```

Python command syntax:

```
ierr = t_prompt_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL TPROMPTDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for prompts is the input window generated by user input requests.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only)
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	prerequisite requirements for API are not met
IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.33. T_REPORT_OUTPUT

Use this API to specify a "T"'d report device, i.e. a destination for a copy of everything sent to the report device.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_T_REPORT_OUTPUT ISLCT FILARG OPTIONS(1)..OPTIONS(2)
```

Python command syntax:

```
ierr = t_report_output(islct, filarg, options)
```

Fortran command syntax:

```
CALL TREPORTDEVICE(ISLCT, FILARG, OPTIONS, IERR)
```

Where:

Integer ISLCT	virtual device selector (input).	
	ISLCT = 1	standard destination. In the PSSE GUI standard output for reports is the Report tab of the output bar.
	ISLCT = 2	direct output to a file.
	ISLCT = 3	direct output to a printer.
	ISLCT = 4	direct output to progress device.
	ISLCT = 5	direct output to report device.
	ISLCT = 6	no output.
Character*260 FILARG	printer name or filename; ignored if ISLCT is not 2 or 3 (input).	
Integer OPTIONS (2)	array of 2 elements specifying open and printing options; ignored if ISLCT is not 2 or 3(input).	
	OPTIONS(1)	file and printer open options: The OPTIONS(1) values may be summed to select multiple options
	OPTIONS(1) = 0	open with carriage control format and, for files, for overwrite of existing files with buffering
	OPTIONS(1) = 1	open with list format
	OPTIONS(1) = 2	open file for append (file output only)
	OPTIONS(1) = 2	set printer orientation to portrait mode (printer output only)
	OPTIONS(1) = 4	open file with no buffering (file output only)
	OPTIONS(1) = 4	open printer in immediate print mode (printer output only)
	OPTIONS(2)	number of copies to print (printer only)
Integer IERR	error code (output).	
	IERR = 0	no error occurred

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 1	prerequisite requirements for API are not met
IERR = 2	failed to open device

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

20.34. USERIN

Get input from interactive device.

This API routine was first introduced in release 30.0.0.

Python command syntax:

```
ierr,string = userin()
```

Where:

Character*512 STRING	buffer input by user (output).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 21
PSSPLOT Module

This chapter defines API routines that are used to manipulate plots in the interface. They will elicit no action if the GUI is not present when invoked.

Please note that these routines are members of the pssplot Python extension module, not psspy. When recorded for Python, the recording will reference each of these functions within the pssplot namespace.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.1. ADDFUNCTION

Add a Function Definition

This API routine was first introduced in release 34.4.0.

Batch command syntax:

```
BAT_ADDFUNCTION FUNCNAME FUNCSTR NUMVARS CHANNFILES(1)...CHANNFILES(6)
 CHANNINDICES(1)...CHANNINDICES(6)
```

Python command syntax:

```
addfunction(funcname, funcstr, numvars, channfiles, channindices)
```

Fortran command syntax:

```
CALL ADDFUNCTION(FUNCNAME, FUNCSTR, NUMVARS, CHANNFILES, CHANNINDICES)
```

Where:

character*32 FUNCNAME	(input).
character*32 FUNCSTR	(input).
Integer NUMVARS	(input).
character*260 CHANNFILES (6)	(input).
Integer CHANNINDICES (6)	(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.2. CHANNELFILEEXCELEXPORT

Use this API to load a Channel Output File into the plotting book.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_CHANNELFILEEXCELEXPORT  SFILENAME
```

Python command syntax:

```
channelfileexcelexport(sfilename)
```

Fortran command syntax:

```
CALL CHANNELFILEEXCELEXPORT(SFILENAME)
```

Where:

Character*260 SFILENAME	name of the channel file to be opened (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.3. DELETESELPLOT

Use this API to delete the selected plot on the active page in the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_DELETESELPLOT
```

Python command syntax:

```
deleteselplot()
```

Fortran command syntax:

```
CALL DELETESELPLOT
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.4. DRAGDROPPLOTDATA

Use this API to load a Channel Output File into the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_DRAGDROPPLOTDATA  SFILENAME SCHANNELNAME
```

Python command syntax:

```
dragdropplotdata(sfilename, schannelname)
```

Fortran command syntax:

```
CALL DRAGDROPPLOTDATA(SFILENAME, SCHANNELNAME)
```

Where:

Character*260 SFILENAME	Channel File name (input).
Character*260 SCHANNELNAME	Channel to be dragged to the plot (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.5. EXPORTPLOT

Use this API to export the selected plot on the active page in the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_EXPORTPLOT SFILENAME NTYPE
```

Python command syntax:

```
exportplot(sfilename, ntype)
```

Fortran command syntax:

```
CALL EXPORTPLOT(SFILENAME, NTYPE)
```

Where:

Character*260 SFILENAME	filename (input).	
Integer NTYPE	graphics format for the exported file (input).	
	NTYPE = 1	Metafile export
	NTYPE = 2	Bitmap export
	NTYPE = 3	JPEG export
	NTYPE = 4	PDF export

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.6. INSERTPAGE

Use this API to insert a page into the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_INSERTPAGE
```

Python command syntax:

```
insertpage()
```

Fortran command syntax:

```
CALL INSERTPAGE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.7. INSERTPLOT

Use this API to add an empty plot on the active page in the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_INSERTPLOT
```

Python command syntax:

```
insertplot()
```

Fortran command syntax:

```
CALL INSERTPLOT
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.8. MULTICHANNEXCELEXPORT

Use this API to export the selected plot channels in the active plotting book.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_MULTICHANNEXCELEXPORT SFILENAME, NUMCHANNS,
 CHANNINDICE(1),..,CHANNINDICE(NUMCHANNS)
```

Python command syntax:

```
multichannexcelexport(sfilename, numchanns, channindice)
```

Fortran command syntax:

```
CALL MULTICHANNEXCELEXPORT(SFILENAME, NUMCHANNS, CHANNINDICE)
```

Where:

Character*260 SFILENAME	filename (input).
Integer NUMCHANNS	Number of Channels to Export (input).
Integer CHANNINDICE	Channel Number to Export (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.9. NEWPLOTBOOK

Use this API to create a new plotting book.

This API routine was first introduced in release 32.0.0.

Batch command syntax:

```
BAT_NEWPLOTBOOK
```

Python command syntax:

```
newplotbook()
```

Fortran command syntax:

```
CALL NEWPLOTSHEET
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.10. OPENCHANDATAFILE

Use this API to load a Channel Output File into the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_OPENCHANDATAFILE  SFILENAME
```

Python command syntax:

```
openchandatafile(sfilename)
```

Fortran command syntax:

```
CALL OPENCHANDATAFILE(SFILENAME)
```

Where:

Character*260 SFILENAME	name of the channel file to be opened (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.11. PLOTPAGEPRINTSETUP

Plot Page Print Settings

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_PLOTPAGEPRINTSETUP  ORIENT NUMROWS NUMCOLS VERTSPAC HORSPAC
```

Python command syntax:

```
plotpageprintsetup(orient, numrows, numcols, vertspac, horspac)
```

Fortran command syntax:

```
CALL PLOTPAGEPRINTSETUP(ORIENT, NUMROWS, NUMCOLS, VERTSPAC, HORSPAC)
```

Where:

Integer ORIENT	(input).
Integer NUMROWS	(input).
Integer NUMCOLS	(input).
Integer VERTSPAC	(input).
Integer HORSPAC	(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.12. PLOT_BOOK

Use this API to add a new plot book. The new plot book will be set as the active book.

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_BOOK BOOKNAME
```

Python command syntax:

```
ierr = plot_book(bookname)
```

Fortran command syntax:

```
CALL PLOT_BOOK(BOOKNAME, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; plot book name will be generated if blank).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to allocate new plot book

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.13. PLOT_BOOK_MBID

Use this API to rename a plot book

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_BOOK_MBID BOOKNAME NEWBOOKNAME
```

Python command syntax:

```
ierr = plot_book_mbid(bookname, newbookname)
```

Fortran command syntax:

```
CALL PLOT_BOOK_MBID(BOOKNAME, NEWBOOKNAME, IERR)
```

Where:

character*72 BOOKNAME	current name of the plot book for which the name is to be changed (input; active plot book name will be used if blank).	
character*72 NEWBOOKNAME	new name to be assigned to the plot book (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	a plot book with the name NEWBOOKNAME already exists

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.14. PLOT_BOOK_PRINT

Use this API to print pages of plots from a plot book

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_BOOK_PRINT BOOKNAME INTGAR(1)..INTGAR(5) REALAR(1)..REALAR(2)
 SDEV FILENAME
```

Python command syntax:

```
ierr = plot_book_print(bookname, intgar, realar, sdev, filename)
```

Fortran command syntax:

```
CALL PLOT_BOOK_PRINT(BOOKNAME, INTGAR, REALAR, SDEV, FILENAME, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).	
Integer INTGAR (5)	array of elements specifying (input).	
	INTGAR(1)	number of copies to print (alias is COPIES) (1 by default)
	INTGAR(2)	orientation, values outside of normal range are set to default. (alias is ORIENT) (1 by default)
	INTGAR(2) = 1	Portrait
	INTGAR(2) = 2	Landscape
	INTGAR(3)	resize plots to fit page (alias is RESIZE) (1 by default)
	INTGAR(4)	first page to include in print, first page in book by default (alias is START) (0 by default)
	INTGAR(5)	last page to include in print, last page in book by default (alias is END) (0 by default)
real REALAR (2)	array of elements specifying (input).	
	REALAR(1)	percent of area used for horizontal margins and spacing (alias is HMARG) (1.0 by default)
	REALAR(2)	percent of area used for vertical margins and spacing (alias is VMARG) (1.0 by default)
Character*260 SDEV	name of valid attached Windows printer (input).	
character*260 FILENAME	name of a file to save the print to, if printing to a file (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 5	failed to select printer
IERR = 6	failed to start printer job
IERR = 7	error while printing chart

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.15. PLOT_BOOK_PURG

Use this API to delete a plot book

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_BOOK_PURG BOOKNAME
```

Python command syntax:

```
ierr = plot_book_purg(bookname)
```

Fortran command syntax:

```
CALL PLOT_BOOK_PURG(BOOKNAME, IERR)
```

Where:

character*72 BOOKNAME	name of the plot book to be deleted, active plot book name will be used if blank (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.16. PLOT_BOOK_SET_ACTIVE

Use this API to set the active plot book

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_BOOK_SET_ACTIVE BOOKNAME
```

Python command syntax:

```
ierr = plot_book_set_active(bookname)
```

Fortran command syntax:

```
CALL PLOT_BOOK_SET_ACTIVE(BOOKNAME, IERR)
```

Where:

character*72 BOOKNAME	name of the plot book to make active (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.17. PLOT_PAGE

Use this API to add a new plot page

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PAGE BOOKNAME PAGENUMBER PAGETITLE INTGAR(1)..INTGAR(4)
 REALAR(1)..REALAR(2)
```

Python command syntax:

```
ierr = plot_page(bookname, pagenumber, pagetitle, intgar, realar)
```

Fortran command syntax:

```
CALL PLOT_PAGE(BOOKNAME, PAGENUMBER, PAGETITLE, INTGAR, REALAR, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).	
Integer PAGENUMBER	page number (input; automatic page number assignment if 0).	
character*72 PAGETITLE	title of the plot page (input).	
Integer INTGAR (4)	array of elements specifying (input).	
	INTGAR(1)	maximum width or height of plot layout on page (alias is MAX) (2 by default)
	INTGAR(2)	if 0, maximum columns of plots on page clamped to MAX, otherwise maximum rows of plots on page clamped to MAX (alias is CLAMP) (0 by default)
	INTGAR(3)	number of plots to fit horizontally on display (alias is COL) (2 by default)
	INTGAR(4)	number of plots to fit vertically on display (alias is ROW) (2 by default)
real REALAR (2)	array of elements specifying (input).	
	REALAR(1)	percent of area used for horizontal margins and spacing (alias is HMARG) (1.0 by default)
	REALAR(2)	percent of area used for vertical margins and spacing (alias is VMARG) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to allocate new plot page

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.18. PLOT_PAGE_CHNG

Use this API to modify a plot page

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PAGE_CHNG BOOKNAME PAGENUMBER PAGETITLE INTGAR(1)..INTGAR(4)
 REALAR(1)..REALAR(2)
```

Python command syntax:

```
ierr = plot_page_chng(bookname, pagenumber, pagetitle, intgar, realar)
```

Fortran command syntax:

```
CALL PLOT_PAGE_CHNG(BOOKNAME, PAGENUMBER, PAGETITLE, INTGAR, REALAR, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).	
Integer PAGENUMBER	plot page number (input; active plot page number will be used if 0).	
character*72 PAGETITLE	title of the plot page (input).	
Integer INTGAR (4)	array of elements specifying (input).	
	INTGAR(1)	maximum width or height of plot layout on page (alias is MAX) (2 by default)
	INTGAR(2)	if 0, maximum columns of plots on page clamped to MAX, otherwise maximum rows of plots on page clamped to MAX (alias is CLAMP) (0 by default)
	INTGAR(3)	number of plots to fit horizontally on display (alias is COL) (2 by default)
	INTGAR(4)	number of plots to fit vertically on display (alias is ROW) (2 by default)
real REALAR (2)	array of elements specifying (input).	
	REALAR(1)	percent of area used for horizontal margins and spacing (alias is HMARG) (1.0 by default)
	REALAR(2)	percent of area used for vertical margins and spacing (alias is VMARG) (1.0 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.19. PLOT_PAGE_PRINT

Use this API to print plots from a plot page

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PAGE_PRINT BOOKNAME PAGENUMBER INTGAR(1)..INTGAR(9)
 REALAR(1)..REALAR(2) SDEV FILENAME
```

Python command syntax:

```
ierr = plot_page_print(bookname, pagenumber, intgar, realar, sdev,
 filename)
```

Fortran command syntax:

```
CALL PLOT_PAGE_PRINT(BOOKNAME, PAGENUMBER, INTGAR, REALAR, SDEV, FILENAME,
 IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).	
Integer PAGENUMBER	page number (input; active plot page number will be used if 0).	
Integer INTGAR (9)	array of elements specifying (input).	
	INTGAR(1)	number of copies to print (alias is COPIES) (1 by default)
	INTGAR(2)	orientation, values outside of normal range are modulated. (alias is ORIENT) (1 by default)
	INTGAR(2) = 1	Portrait
	INTGAR(2) = 2	Landscape
	INTGAR(3)	resize plots to fit page (alias is RESIZE) (1 by default)
	INTGAR(4)	first column of plots to include in print (alias is XSTART) (0 by default)
	INTGAR(5)	first row of plots to include in print (alias is YSTART) (0 by default)
	INTGAR(6)	number of columns to include in print, all columns after XSTART included by default (alias is XSPAN) (0 by default)
	INTGAR(7)	number of rows to include in print, all rows after YSTART included by default (alias is YSPAN) (0 by default)
	INTGAR(8)	number of columns to layout on per page, all columns to be printed by default (alias is XPAGE) (0 by default)
	INTGAR(9)	number of rows to layout on per page, all rows to be printed by default (alias is YPAGE) (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

real REALAR (2)	array of elements specifying (input).	
	REALAR(1)	percent of area used for horizontal margins and spacing (alias is HMARG) (1.0 by default)
	REALAR(2)	percent of area used for vertical margins and spacing (alias is VMARG) (1.0 by default)
Character*260 SDEV	name of valid attached Windows printer (input).	
character*260 FILENAME	name of a file to save the print to, if printing to a file (input).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 5	failed to select printer
	IERR = 6	failed to start printer job
	IERR = 7	error while printing chart

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.20. PLOT_PAGE_PURG

Use this API to delete a plot page

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PAGE_PURG BOOKNAME PAGENUMBER
```

Python command syntax:

```
ierr = plot_page_purg(bookname, pagenumber)
```

Fortran command syntax:

```
CALL PLOT_PAGE_PURG(BOOKNAME, PAGENUMBER, IERR)
```

Where:

character*72 BOOKNAME	name of the plot book containing the plot page to be deleted (input; active plot book name will be used if blank).	
Integer PAGENUMBER	number of the plot page to be deleted (input; active plot book name will be used if blank).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.21. PLOT_PAGE_SET_ACTIVE

Use this API to set the active plot page on the current active plot book

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PAGE_SET_ACTIVE PAGENUMBER
```

Python command syntax:

```
ierr = plot_page_set_active(pagenumber)
```

Fortran command syntax:

```
CALL PLOT_PAGE_SET_ACTIVE(PAGENUMBER, IERR)
```

Where:

Integer PAGENUMBER	number of the plot page to make active (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	no active plot book
	IERR = 2	failed to locate plot page with given number on active plot book

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.22. PLOT_PLOT

Use this API to add a new plot

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PLOT BOOKNAME PAGENUMBER PLOTNUMBER PLOTTITLE
 INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = plot_plot(bookname, pagenumber, plotnumber, plottitle, intgar,
 realar)
```

Fortran command syntax:

```
CALL PLOT_PLOT(BOOKNAME, PAGENUMBER, PLOTNUMBER, PLOTTITLE, INTGAR,
 REALAR, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).	
Integer PAGENUMBER	page number (input; active page number will be used if 0).	
Integer PLOTNUMBER	plot number (input; automatic plot number assignment if 0).	
character*72 PLOTTITLE	title of the plot (input).	
Integer INTGAR (7)	array of elements specifying (input).	
	INTGAR(1)	X axis type. (alias is XTYP) (0 by default)
	INTGAR(1) = 0	Linear
	INTGAR(1) = 1	Logarithmic
	INTGAR(2)	Y axis type (alias is YTYP) (0 by default)
	INTGAR(2) = 0	Linear
	INTGAR(2) = 1	Logarithmic
	INTGAR(3)	X axis dynamic scaling (alias is XAUTO) (1 by default)
	INTGAR(3) = 0	Static
	INTGAR(3) = 1	Dynamic
	INTGAR(4)	Y axis dynamic scaling (alias is YAUTO) (1 by default)
	INTGAR(4) = 0	Static
	INTGAR(4) = 1	Dynamic
	INTGAR(5)	Each trace given it's own y axis (alias is MULTY) (1 by default)
	INTGAR(6)	Grid style to use. Values outside of normal range are modulated. (alias is GRID) (1 by default)
	INTGAR(6) = 0	No Grid Shown
	INTGAR(6) = 1	Major Grid Lines Only

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(6) = 2	Major and Minor Grid Lines
	INTGAR(7)	show legend (alias is LGND) (1 by default)
real REALAR (4)	array of elements specifying (input).	
	REALAR(1)	minimum x axis value if not dynamic (alias is XMIN)
	REALAR(2)	minimum y axis value if not dynamic (alias is YMIN)
	REALAR(3)	maximum x axis value if not dynamic (alias is XMAX)
	REALAR(4)	maximum y axis value if not dynamic (alias is YMAX)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 3	failed to allocate new plot

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.23. PLOT_PLOT_CHNG

Use this API to modify a plot

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PLOT_CHNG BOOKNAME PAGENUMBER PLOTNUMBER PLOTTITLE
 INTGAR(1)..INTGAR(7) REALAR(1)..REALAR(4)
```

Python command syntax:

```
ierr = plot_plot_chng(bookname, pagenumber, plotnumber, plottitle, intgar,
 realar)
```

Fortran command syntax:

```
CALL PLOT_PLOT_CHNG(BOOKNAME, PAGENUMBER, PLOTNUMBER, PLOTTITLE, INTGAR,
 REALAR, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).
Integer PAGENUMBER	page number (input; active page number will be used if 0).
Integer PLOTNUMBER	plot number (input; active plot number will be used if 0).
character*72 PLOTTITLE	title of the plot (input).
Integer INTGAR (7)	array of elements specifying (input).

INTGAR(1)	x axis type (alias is XTYP) (0 by default)
INTGAR(1) = 0	Linear
INTGAR(1) = 1	Logarithmic
INTGAR(2)	y axis type (alias is YTYP) (0 by default)
INTGAR(2) = 0	Linear
INTGAR(2) = 1	Logarithmic
INTGAR(3)	x axis dynamic scaling (alias is XAUTO) (1 by default)
INTGAR(3) = 0	Static
INTGAR(3) = 1	Dynamic
INTGAR(4)	y axis dynamic scaling (alias is YAUTO) (1 by default)
INTGAR(4) = 0	Static
INTGAR(4) = 1	Dynamic
INTGAR(5)	each trace given it's own y axis (alias is MULTY) (1 by default)
INTGAR(6)	grid style to use. Values outside of normal range are modulated. (alias is GRID) (1 by default)
INTGAR(6) = 0	No Grid Shown
INTGAR(6) = 1	Major Grid Lines Only

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(6) = 2	Major and Minor Grid Lines
	INTGAR(7)	show legend (alias is LGND) (1 by default)
real REALAR (4)	array of elements specifying (input).	
	REALAR(1)	minimum x axis value if not dynamic (alias is XMIN)
	REALAR(2)	minimum y axis value if not dynamic (alias is YMIN)
	REALAR(3)	maximum x axis value if not dynamic (alias is XMAX)
	REALAR(4)	maximum y axis value if not dynamic (alias is YMAX)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 3	failed to locate plot with given number, or no active plot on plot page if PLOTNUMBER is 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.24. PLOT_PLOT_EXPORT_IMAGE

Use this API to export an image of a plot

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PLOT_EXPORT_IMAGE BOOKNAME PAGENUMBER PLOTNUMBER FILENAME TYPE
```

Python command syntax:

```
ierr = plot_plot_export_image(bookname, pagenumber, plotnumber, filename,
 type)
```

Fortran command syntax:

```
CALL PLOT_PLOT_EXPORT_IMAGE(BOOKNAME, PAGENUMBER, PLOTNUMBER, FILENAME,
 TYPE, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).	
Integer PAGENUMBER	page number (input; active page number will be used if 0).	
Integer PLOTNUMBER	plot number (input; active plot number will be used if 0).	
Character*260 FILENAME	name of the image file to be written (input).	
Integer TYPE	type of image to export, values outside this range are discarded. (input).	
	TYPE = 1	Bitmap
	TYPE = 2	JPEG
	TYPE = 3	PNG
	TYPE = 4	GIF
	TYPE = 5	PDF
	TYPE = 6	EMF
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 3	failed to locate plot with given number, or no active plot on plot page if PLOTNUMBER is 0

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.25. PLOT_PLOT_PURG

Use this API to delete a plot

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PLOT_PURG BOOKNAME PAGENUMBER PLOTNUMBER
```

Python command syntax:

```
ierr = plot_plot_purg(bookname, pagenumber, plotnumber)
```

Fortran command syntax:

```
CALL PLOT_PLOT_PURG(BOOKNAME, PAGENUMBER, PLOTNUMBER, IERR)
```

Where:

character*72 BOOKNAME	name of the plot book containing the plot to be deleted (input; active plot book name will be used if blank).	
Integer PAGENUMBER	number of the plot page containing the plot to be deleted (input; active page number will be used if 0).	
Integer PLOTNUMBER	number of the plot to be deleted (input; active plot number will be used if 0).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 3	failed to locate plot with given number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.26. PLOT_PLOT_SET_ACTIVE

Use this API to set the active plot on the current active plot page

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_PLOT_SET_ACTIVE PLOTNUMBER
```

Python command syntax:

```
ierr = plot_plot_set_active(plotnumber)
```

Fortran command syntax:

```
CALL PLOT_PLOT_SET_ACTIVE(PLOTNUMBER, IERR)
```

Where:

Integer PLOTNUMBER	number of the plot to make active (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	no active plot book
	IERR = 2	no active plot page on active plot book
	IERR = 3	failed to locate plot with given number on active plot page

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.27. PLOT_TABLE_COLUMN

This API will plot a column(s) of data from a PssTable

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PLOT_TABLE_COLUMN TABLEID MODE NUMSPREAD THRESHOLD NUMCOLS
 COLUMNIDS(1) .. COLUMNIDS(NUMCOLS)
```

Python command syntax:

```
ierr = plot_table_column(tableid, mode, numspread, threshold, numcols,
 columnids)
```

Fortran command syntax:

```
CALL PLOT_TABLE_COLUMN(TABLEID, MODE, NUMSPREAD, THRESHOLD, NUMCOLS,
 COLUMNIDS, IERR)
```

Where:

Integer TABLEID	the PssTable number to plot columns for (input).	
Integer MODE	is the plotting mode (input).	
	MODE = 0	PLOT_SELECTION - Plot only the selected columns
	MODE = 1	PLOT_SPREAD - Plot the top NUMSPREAD columns that have the largest Min, Max spread
	MODE = 2	PLOT_THRESHOLD - Plot the first NUMSPREAD columns that have a Max value >= THRESHOLD
Integer NUMSPREAD	if this value is not equal to 0, this API will plot the NUMSPREAD columns that have the greatest range between their minimum and maximum values (input).	
Real THRESHOLD	is the threshold value to begin selecting columns for plotting if MODE = PLOT_THRESHOLD (input).	
Integer NUMCOLS	the number of columns contained in COLUMNIDS (input; no default allowed).	
Integer COLUMNIDS (NUMCOLS)	The PssTable column numbers (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	NUMSPREAD and NUMCOLS are both 0
	IERR = 3	TABLEID is not defined
	IERR = 4	No columns satisfied THRESHOLD
	IERR = 10	failed to allocate new plot book
	IERR = 20	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 21	failed to allocate new plot page
IERR = 30	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
IERR = 31	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
IERR = 32	failed to allocate new plot

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.28. PLOT_TABLE_COLUMN_RANGE

This API will plot a column(s) of data from a PssTable for a range of rows

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_PLOT_TABLE_COLUMN_RANGE TABLEID MODE NUMSPREAD THRESHOLD NUMCOLS
 COLUMNIDS(1) .. COLUMNIDS(NUMCOLS) NUMROWS ROWS(1) .. ROWS(NUMROWS)
```

Python command syntax:

```
ierr = plot_table_column_range(tableid, mode, numspread, threshold,
 numcols, columnids, numrows, rows)
```

Fortran command syntax:

```
CALL PLOT_TABLE_COLUMN_RANGE(TABLEID, MODE, NUMSPREAD, THRESHOLD, NUMCOLS,
 COLUMNIDS, NUMROWS, ROWS, IERR)
```

Where:

Integer TABLEID	the PssTable number to plot columns for (input).	
Integer MODE	is the plotting mode (input).	
	MODE = 0	PLOT_SELECTION - Plot only the selected columns
	MODE = 1	PLOT_SPREAD - Plot the top NUMSPREAD columns that have the largest Min, Max spread
	MODE = 2	PLOT_THRESHOLD - Plot the first NUMSPREAD columns that have a Max value >= THRESHOLD
Integer NUMSPREAD	if this value is not equal to 0, this API will plot the NUMSPREAD columns that have the greatest range between their minimum and maximum values (input).	
Real THRESHOLD	is the threshold value to begin selecting columns for plotting if MODE = PLOT_THRESHOLD (input).	
Integer NUMCOLS	the number of columns contained in COLUMNIDS (input; no default allowed).	
Integer COLUMNIDS (NUMCOLS)	The PssTable column numbers (input).	
Integer NUMROWS	the number of rows contained in ROWS (input; no default allowed).	
Integer ROWS (NUMROWS)	The range of row numbers (input).	
Integer IERR	error code (output).	
	IERR = 0	no error occurred
	IERR = 1	NUMSPREAD and NUMCOLS are both 0
	IERR = 2	ROWS contains non-continguous rows
	IERR = 3	TABLEID is not defined
	IERR = 4	No columns satisfied THRESHOLD

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 10	failed to allocate new plot book
IERR = 20	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
IERR = 21	failed to allocate new plot page
IERR = 30	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
IERR = 31	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
IERR = 32	failed to allocate new plot

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.29. PLOT_TRACE_CHANNEL

Use this API to add a new plot trace from a channel file

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_TRACE_CHANNEL BOOKNAME PAGENUMBER PLOTNUMBER TRACENUMBER
 TRACETITLE TYPE FILENAME CHANNELNAME INTGAR(1)..INTGAR(4)
```

Python command syntax:

```
ierr = plot_trace_channel(bookname, pagenumber, plotnumber, tracenumber,
 tracetitle, type, filename, channelname, intgar)
```

Fortran command syntax:

```
CALL PLOT_TRACE_CHANNEL(BOOKNAME, PAGENUMBER, PLOTNUMBER, TRACENUMBER,
 TRACETITLE, TYPE, FILENAME, CHANNELNAME, INTGAR, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).	
Integer PAGENUMBER	page number (input; active page number will be used if 0).	
Integer PLOTNUMBER	plot number (input; active plot number will be used if 0).	
Integer TRACENUMBER	plot trace number (input; Unused, reserved for future use. Plot Trace numbers are currently automatically assigned.).	
character*72 TRACETITLE	title of the plot trace (input).	
Integer TYPE	(input).	
Character*260 FILENAME	Channel File name (input).	
Character*72 CHANNELNAME	Channel to be added to the plot (input).	
Integer INTGAR (4)	array of elements specifying (input).	
	INTGAR(1)	Trace Type (Line, Bar, etc.). Values outside of normal range are modulated. (alias is TTYPE) (-1 by default)
	INTGAR(2)	Line Style (alias is LSTYLE) (-1 by default)
	INTGAR(2) = -1	Auto-Select
	INTGAR(2) = 0	Solid
	INTGAR(2) = 1	Dash
	INTGAR(2) = 2	Dot
	INTGAR(2) = 3	Dash Dot
	INTGAR(2) = 4	Dash Dot Dot
	INTGAR(3)	Symbol Style. Values outside of normal range are modulated. (alias is SSTYLE) (-1 by default)
	INTGAR(3) = -1	Auto-Select
	INTGAR(3) = 0	No Symbol

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(3) = 1	Rectangle
	INTGAR(3) = 2	Circle
	INTGAR(3) = 3	Triangle
	INTGAR(3) = 4	Down Triangle
	INTGAR(3) = 5	Cross
	INTGAR(3) = 6	Diagonal Cross
	INTGAR(3) = 7	Star
	INTGAR(3) = 8	Diamond
	INTGAR(3) = 9	Small Dot
	INTGAR(3) = 10	Nothing
	INTGAR(3) = 11	Left Triangle
	INTGAR(3) = 12	Right Triangle
	INTGAR(3) = 13	Hexagon
	INTGAR(3) = 14	Visual
	INTGAR(3) = 15	Donut
	INTGAR(3) = 16	Arrow
	INTGAR(4)	Trace Color (Auto-Select when equal to plot background color) (alias is COLOR) (16777215 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 3	failed to locate plot with given number, or no active plot on plot page if PLOTNUMBER is 0
	IERR = 4	failed to allocate new plot trace
	IERR = 5	failed to find a file with the given file name
	IERR = 6	failed to find a channel with the given channel name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.30. PLOT_TRACE_PURG

Use this API to delete a plot trace

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_TRACE_PURG BOOKNAME PAGENUMBER PLOTNUMBER TRACENUMBER
```

Python command syntax:

```
ierr = plot_trace_purg(bookname, pagenumber, plotnumber, tracenumber)
```

Fortran command syntax:

```
CALL PLOT_TRACE_PURG(BOOKNAME, PAGENUMBER, PLOTNUMBER, TRACENUMBER, IERR)
```

Where:

character*72 BOOKNAME	name of the plot book containing the plot trace to be deleted (input; active plot book name will be used if blank).	
Integer PAGENUMBER	number of the plot page containing the plot trace to be deleted (input; active page number will be used if 0).	
Integer PLOTNUMBER	number of the plot containing the plot trace to be deleted (input; active plot number will be used if 0).	
Integer TRACENUMBER	number of the plot trace to be deleted (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 3	failed to locate plot with given number, or no active plot on plot page if PLOTNUMBER is 0
	IERR = 4	failed to locate plot tracewith given number

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.31. PLOT_TRACE_XY

Use this API to add a new plot trace from a channel file on both the x and y axis

This API routine was first introduced in release 35.4.

Batch command syntax:

```
BAT_PLOT_TRACE_XY BOOKNAME PAGENUMBER PLOTNUMBER TRACENUMBER TRACETITLE
 TYPE FILENAME CHANNELNAMEX CHANNELNAMEY INTGAR(1)..INTGAR(4)
```

Python command syntax:

```
ierr = plot_trace_xy(bookname, pagenumber, plotnumber, tracenumber,
 tracetitle, type, filename, channelnamex, channelnamey, intgar)
```

Fortran command syntax:

```
CALL PLOT_TRACE_XY(BOOKNAME, PAGENUMBER, PLOTNUMBER, TRACENUMBER,
 TRACETITLE, TYPE, FILENAME, CHANNELNAMEX, CHANNELNAMEY, INTGAR, IERR)
```

Where:

character*72 BOOKNAME	plot book name (input; active plot book name will be used if blank).
Integer PAGENUMBER	page number (input; active page number will be used if 0).
Integer PLOTNUMBER	plot number (input; active plot number will be used if 0).
Integer TRACENUMBER	plot trace number (input; Unused, reserved for future use. Plot Trace numbers are currently automatically assigned.).
character*72 TRACETITLE	title of the plot trace (input).
Integer TYPE	(input).
Character*260 FILENAME	Channel File name (input).
Character*260 CHANNELNAMEX	Channel to be used for the X values of the trace to add to the plot (input).
Character*260 CHANNELNAMEY	Channel to be used for the Y values of the trace to add to the plot (input).
Integer INTGAR (4)	array of elements specifying (input).

INTGAR(1)	Trace Type (Line, Bar, etc.) (alias is TTYPE) (-1 by default)
INTGAR(2)	Line Style (alias is LSTYLE) (-1 by default)
INTGAR(2) = -1	Auto-Select
INTGAR(2) = 0	Solid
INTGAR(2) = 1	Dash
INTGAR(2) = 2	Dot
INTGAR(2) = 3	Dash Dot
INTGAR(2) = 4	Dash Dot Dot
INTGAR(3)	Symbol Style (alias is SSTYLE) (-1 by default)
INTGAR(3) = -1	Auto-Select
INTGAR(3) = 0	No Symbol

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	INTGAR(3) = 1	Rectangle
	INTGAR(3) = 2	Circle
	INTGAR(3) = 3	Triangle
	INTGAR(3) = 4	Down Triangle
	INTGAR(3) = 5	Cross
	INTGAR(3) = 6	Diagonal Cross
	INTGAR(3) = 7	Star
	INTGAR(3) = 8	Diamond
	INTGAR(3) = 9	Small Dot
	INTGAR(3) = 10	Nothing
	INTGAR(3) = 11	Left Triangle
	INTGAR(3) = 12	Right Triangle
	INTGAR(3) = 13	Hexagon
	INTGAR(3) = 14	Visual
	INTGAR(3) = 15	Donut
	INTGAR(3) = 16	Arrow
	INTGAR(4)	Trace Color (Auto-Select when equal to plot background color) (alias is COLOR) (16777215 by default)
Integer IERR	error code (output).	
	IERR = 0	no errors occurred
	IERR > 0	fatal error occurred
	IERR = 1	failed to locate plot book with given name, or no active plot book if BOOKNAME is blank
	IERR = 2	failed to locate plot page with given number, or no active plot page on plot book if PAGENUMBER is 0
	IERR = 3	failed to locate plot with given number, or no active plot on plot page if PLOTNUMBER is 0
	IERR = 4	failed to allocate new plot trace
	IERR = 5	failed to find a file with the given file name
	IERR = 6	failed to find a channel with the given channel name

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.32. PRINTACTIVEPAGE

Use this API to print the active page on the default printer.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_PRINTACTIVEPAGE
```

Python command syntax:

```
printactivepage()
```

Fortran command syntax:

```
CALL PRINTACTIVEPAGE
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.33. PRINTSELPLOT

Use this API to print the selected plot on the active page in the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_PRINTSELPLOT SFILENAME NORIENT NINDEX
```

Python command syntax:

```
printselplot(sfilename, norient, nindex)
```

Fortran command syntax:

```
CALL PRINTSELPLOT(SFILENAME, NORIENT, NINDEX)
```

Where:

Character*260 SFILENAME	name of the file to print (input).
Integer NORIENT	orientation of the paper (input). NORIENT = 0 Portrait NORIENT = 1 Landscape
Integer NINDEX	index for the printer you wish to use; This index refers to the list of printers defined on your system; default = 1 (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.34. SETSELECTEDPAGE

Use this API to set the active page in the plotting book

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SETSELECTEDPAGE    NPAGENUM
```

Python command syntax:

```
setselectedpage(npagenum)
```

Fortran command syntax:

```
CALL SETSELECTEDPAGE(NPAGENUM)
```

Where:

Integer NPAGENUM	page number in the plotting book (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.35. SETSELECTEDPLOT

Use this API to set the selected plot on the active page in the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SETSELECTEDPLOT    NPLOTNUM
```

Python command syntax:

```
setselectedplot(nplotnum)
```

Fortran command syntax:

```
CALL SETSELECTEDPLOT(NPLOTNUM)
```

Where:

Integer NPLOTNUM	plot number on the selected page (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.36. SHOWPLOTEDITOR

Use this API to display the editor for the selected plot in the plotting book.

This API routine was first introduced in release 31.0.0.

Batch command syntax:

```
BAT_SHOWPLOTEDITOR
```

Python command syntax:

```
showploteditor()
```

Fortran command syntax:

```
CALL SHOWPLOTEDITOR
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

21.37. XYPLOT

Use this API to create an X-Y Plot

This API routine was first introduced in release 34.3.0.

Batch command syntax:

```
BAT_XYPLOT  SFILENAME XCHANNELNAME YCHANNELNAME
```

Python command syntax:

```
xyplot(sfilename, xchannelname, ychannelname)
```

Fortran command syntax:

```
CALL XYPLOT(SFILENAME, XCHANNELNAME, YCHANNELNAME)
```

Where:

Character*260 SFILENAME	Channel File name (input).
character*260 XCHANNELNAME	(input).
character*260 YCHANNELNAME	(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 22
PSSRDB Module

Note

The PSSRDB Module has been discontinued starting with PSS®E 34.9.1 due to support for Data Visualization and Reporting (DVRM) ending.

The APIs previously provided with the PSSRDB Module are no longer usable and are considered Obsolete.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 23
MUSTPY Module

This chapter defines API routines that are used for Transfer Analysis and Sensitivities features.

Please note that these routines are members of the mustpy Python extension module, not psspy. When recorded for Python, the recording will reference each of these functions within the mustpy namespace.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1. MUST Analysis Functions

These API routines are recordable and perform system analysis functions.

23.1.1. ACCDEFAULTS

Use this function to enter a set of default AC Contingency options for later use in an the AC contingency analysis

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ACCDEFAULTS  OPTSI(1)..OPTS(10) VALUES(1)..VALUES(3)
```

Python command syntax:

```
ierr = accdefaults(options, values)
```

Fortran command syntax:

```
CALL MAPI_ACCDEFAULTS(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (11)	array of 11 elements specifying calculation options(input).	
	OPTIONS(1)	AC LF method (2 by default)
	OPTIONS(1) = 1	AC Newton
	OPTIONS(1) = 2	AC Decoupled
	OPTIONS(2)	Apply non-divergent LF (0 by default)
	OPTIONS(2) = 0	never
	OPTIONS(2) = 1	always
	OPTIONS(2) = 2	only if regular diverged
	OPTIONS(3)	Transformer Tap (0 by default)
	OPTIONS(3) = 0	disabled
	OPTIONS(3) = 1	enable stepping, No Direct
	OPTIONS(4)	Phase Shifter (0 by default)
	OPTIONS(4) = 0	disabled
	OPTIONS(4) = 1	enabled
	OPTIONS(5)	Switchable Shunts (1 by default)
	OPTIONS(5) = 0	disabled
	OPTIONS(5) = 1	enabled
	OPTIONS(6)	Area Interchange (0 by default)
	OPTIONS(6) = 0	disabled
	OPTIONS(6) = 1	Tie lines only
	OPTIONS(6) = 2	Tie lines and loads
	OPTIONS(7)	DC Tap (1 by default)
	OPTIONS(7) = 0	Locked

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(7) = 1	On
	OPTIONS(8)	Iter. to begin Var limiting (0 by default)
	OPTIONS(8) = 0	Hot Start: apply var limits immediately
	OPTIONS(8) = 99	Automatic:
	OPTIONS(9)	Load Flow Iterations Limit (20 by default)
	OPTIONS(9) > n	Terminate calculation after n iterations
	OPTIONS(10)	Use Flat Start for Load Flow Solution (0 by default)
	OPTIONS(10) = 0	hot
	OPTIONS(10) = 1	flat
	OPTIONS(11)	Iteration to freeze adjustments (15 by default)
	OPTIONS(11) > n	Freeze adjustments after n iterations
Real VALUES (5)	array of 5 elements specifying reporting parameters(input).	
	VALUES(1)	Minimum contingency case flow change cutoff (0.0 by default)
	VALUES(2)	Minimum voltage contingency change cutoff in per-unit (0.0 by default)
	VALUES(3)	Real and Reactive Power Tolerance (0.6 by default)
	VALUES(4)	Reactive Adjustments DeAcceleration Factor (1.0 by default)
	VALUES(5)	Low Voltage break point (pu) for load scaling (0.7 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer option value is missing
	IERR = 3	Float option value is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.2. ACCONTBYCONT

Use this function to generate an AC contingency report by contingency options

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ACCONTBYCONT SEL SELOPT1 SELOPT2 RPTTYPE OPTIONS(1)..OPTIONS(6)
 NUM(1)..NUM(4)
```

Python command syntax:

```
ierr = accontbycont(sel, selopt1, selopt2, rpttype, options, num)
```

Fortran command syntax:

```
CALL MAPI_ACCONTBYCONT(SEL, SELOPT1, SELOPT2, RPTTYPE, OPTIONS, NUM, IERR)
```

Where:

Integer SEL	Indicate how to filter the contingencies for this report (input; no default allowed).	
	'1'	Report contingencies by Category
	'2'	Report contingencies by Range
	'3'	Report contingencies below severity index
	'4'	Report contingencies above severity index
Integer SELOPT1	Option used in conjunction with SEL when SEL=(1 or 2) to specify filter criteria (input; no default allowed).	
	SELOPT1 = n	If SEL = 2 (by Range) then n = lower contingency number in desired range. If SEL = 1 (by Category) then enter one of the following:
	SELOPT1 = 1	Show ALL contingencies
	SELOPT1 = 2	Show contingencies with Violations
	SELOPT1 = 3	Show contingencies without Violations
	SELOPT1 = 4	Show ACTIVE contingencies
	SELOPT1 = 5	Show EXCLUDED contingencies
	SELOPT1 = 6	Show DISABLED contingencies
	SELOPT1 = 7	Show SINGLE BRANCH contingencies
	SELOPT1 = 8	Show COMPLEX contingencies
	SELOPT1 = 9	Show contingencies with Islands
	SELOPT1 = 10	Show IMBALANCED contingencies
	SELOPT1 = 11	Show contingencies with Generation
	SELOPT1 = 12	Show contingencies with Load
	SELOPT1 = 13	Show CONVERGED contingencies
	SELOPT1 = 14	Show NOT CONVERGED contingencies
Integer SELOPT2	Only used if SEL = 2 (by Range) (input).	
	SELOPT2 = n	Upper contingency number in desired range
Integer RPTTYPE	Type of contingency report (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'1'	Selected Contingencies with Violations
	'2'	Summary List of Selected Contingencies
	'3'	AC Contingency Summary of Selected Contingencies
	'4'	Details for Selected Contingencies
Integer OPTIONS (6)	array of 6 elements specifying report options(input).	
	OPTIONS(1)	Report loading above cutoff (0 by default)
	OPTIONS(1) = 0	No
	OPTIONS(1) = 1	Yes
	OPTIONS(2)	Report Low Voltage (0 by default)
	OPTIONS(2) = 0	No
	OPTIONS(2) = 1	Yes
	OPTIONS(3)	Report High Voltage (0 by default)
	OPTIONS(3) = 0	No
	OPTIONS(3) = 1	Yes
	OPTIONS(4)	Include Contingent Events (0 by default)
	OPTIONS(4) = 0	No
	OPTIONS(4) = 1	Yes
	OPTIONS(5)	Include Islands Report (0 by default)
	OPTIONS(5) = 0	No
	OPTIONS(5) = 1	Yes
	OPTIONS(6)	Include Dispatch Data (0 by default)
	OPTIONS(6) = 0	No
	OPTIONS(6) = 1	Yes
Real NUM (4)	array of 4 elements specifying criteria to be used in conjunction with SEL and OPTIONS (input).	
	NUM(1)	Used with SEL=3 or 4 (by Severity Index) (0 by default)
	NUM(1) = x	x = Severity index for selected contingencies
	NUM(2)	Used with OPTIONS(1) = 1 (Report loading above cutoff) (0 by default)
	NUM(2) = y	y = Cutoff (%) for reporting loading
	NUM(3)	Used with OPTIONS(2) = 1 (Report Low Voltage) (0 by default)
	NUM(3) = vL	vL = Low Voltage Margin (PU)
	NUM(4)	Used with OPTIONS(3) = 1 (Report High Voltage) (0 by default)
	NUM(4) = vH	vH = High Voltage Margin (PU)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.3. ACCONTBYMONEL

Use this function to generate an AC contingency report by monitored elements

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ACCONTBYMONEL SEL SELOPT1 SELOPT2 MONELS RPTTYPE NUM(1)..NUM(4)
```

Python command syntax:

```
ierr = accontbymonel(sel, selopt1, selopt2, monels, rpttype, num)
```

Fortran command syntax:

```
CALL MAPI_ACCONTBYMONEL(SEL, SELOPT1, SELOPT2, MONELS, RPTTYPE, NUM, IERR)
```

Where:

Integer SEL	Indicate how to filter the contingencies for this report (input; no default allowed).	
	'1'	Report contingencies by Category
	'2'	Report contingencies by Range
	'3'	Report contingencies below severity index
	'4'	Report contingencies above severity index
Integer SELOPT1	Option used in conjunction with SEL when SEL=(1 or 2) to specify filter criteria (input; no default allowed).	
	SELOPT1 = n	If SEL = 2 (by Range) then n = lower contingency number in desired range. If SEL = 1 (by Category) then enter one of the following:
	SELOPT1 = 1	Show ALL contingencies
	SELOPT1 = 2	Show contingencies with Violations
	SELOPT1 = 3	Show contingencies without Violations
	SELOPT1 = 4	Show ACTIVE contingencies
	SELOPT1 = 5	Show EXCLUDED contingencies
	SELOPT1 = 6	Show DISABLED contingencies
	SELOPT1 = 7	Show SINGLE BRANCH contingencies
	SELOPT1 = 8	Show COMPLEX contingencies
	SELOPT1 = 9	Show contingencies with Islands
	SELOPT1 = 10	Show IMBALANCED contingencies
	SELOPT1 = 11	Show contingencies with Generation
	SELOPT1 = 12	Show contingencies with Load
	SELOPT1 = 13	Show CONVERGED contingencies
	SELOPT1 = 14	Show NOT CONVERGED contingencies
Integer SELOPT2	Only used if SEL = 2 (by Range) (input).	
	SELOPT2 = n	Upper contingency number in desired range
Integer MONELS	Monitored elements for report (input; no default allowed).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'1'	Branches
	'2'	Interfaces
	'3'	Flowgates
	'4'	Voltage Buses
Integer RPTTYPE	Type of contingency report (input; no default allowed).	
	'1'	List of Monitored Elements
	'2'	List of Violations above Cutoff
	'3'	Summary of Frequency of Violations
Real NUM (4)	array of 4 elements specifying criteria to be used in conjunction with SEL and RPTTYPE (input).	
	NUM(1)	Used with SEL=3 or 4 (by Severity Index) (0 by default)
	NUM(1) = x	x = Severity index for selected contingencies
	NUM(2)	Used with RPTTYPE = 2 (Violations above cut-off) (100 by default)
	NUM(2) = y	y = Cutoff (%) for reporting loading
	NUM(3)	Used with RPTTYPE = 2 and MONELS = 4 (Voltage Bus Low Voltage Margin) (0 by default)
	NUM(3) = vL	vL = Low Voltage Margin (PU)
	NUM(4)	Used with RPTTYPE = 2 and MONELS = 4 (Voltage Bus High Voltage Margin) (0 by default)
	NUM(4) = vH	vH = High Voltage Margin (PU)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Entry is missing or invalid
	IERR = 3	Not available for flowgates

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.4. ACCSOLVE

Use this function to run the AC contingency analysis solution with the specified calculation options

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ACCSOLVE  OPTSI(1)..OPTS(10) VALUES(1)..VALUES(3)
```

Python command syntax:

```
ierr = accsolve(options, values)
```

Fortran command syntax:

```
CALL MAPI_ACCSOLVE(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (11)	array of 11 elements specifying calculation options(input).	
	OPTIONS(1)	AC LF method (2 by default)
	OPTIONS(1) = 1	AC Newton
	OPTIONS(1) = 2	AC Decoupled
	OPTIONS(2)	Apply non-divergent LF (0 by default)
	OPTIONS(2) = 0	never
	OPTIONS(2) = 1	always
	OPTIONS(2) = 2	only if regular diverged
	OPTIONS(3)	Transformer Tap (0 by default)
	OPTIONS(3) = 0	disabled
	OPTIONS(3) = 1	enable stepping, No Direct
	OPTIONS(4)	Phase Shifter (0 by default)
	OPTIONS(4) = 0	disabled
	OPTIONS(4) = 1	enabled
	OPTIONS(5)	Switchable Shunts (1 by default)
	OPTIONS(5) = 0	disabled
	OPTIONS(5) = 1	enabled
	OPTIONS(6)	Area Interchange (0 by default)
	OPTIONS(6) = 0	disabled
	OPTIONS(6) = 1	Tie lines only
	OPTIONS(6) = 2	Tie lines and loads
	OPTIONS(7)	DC Tap (1 by default)
	OPTIONS(7) = 0	Locked
	OPTIONS(7) = 1	On
	OPTIONS(8)	Iter. to begin Var limiting (0 by default)
	OPTIONS(8) = 0	Hot Start: apply var limits immediately
	OPTIONS(8) = 99	Automatic:
	OPTIONS(9)	Load Flow Iterations Limit (20 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(9) > n	Terminate calculation after n iterations
	OPTIONS(10)	Use Flat Start for Load Flow Solution (0 by default)
	OPTIONS(10) = 0	hot
	OPTIONS(10) = 1	flat
	OPTIONS(11)	Iteration to freeze adjustments (15 by default)
	OPTIONS(11) > n	Freeze adjustments after n iterations
Real VALUES (5)	array of 5 elements specifying reporting parameters(input).	
	VALUES(1)	Minimum contingency case flow change cutoff (0.0 by default)
	VALUES(2)	Minimum voltage contingency change cutoff in percent (0.0 by default)
	VALUES(3)	Real and Reactive Power Tolerance (0.6 by default)
	VALUES(4)	Reactive Adjustments DeAcceleration Factor (1.0 by default)
	VALUES(5)	Low Voltage break point (pu) for load scaling (0.7 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer option value is missing
	IERR = 3	Float option value is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.5. ACLFDC

Use this function to execute an AC load flow solution using the Fast Decoupled Method and the specified calculation options

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ACLFDC  OPTSI(1)..OPTS(10) VALUES(1)..VALUES(3)
```

Python command syntax:

```
ierr = aclfdc(options, values)
```

Fortran command syntax:

```
CALL MAPI_ACLFDC(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (10)	array of 10 elements specifying calculation options(input).	
	OPTIONS(1)	Base Case Load Flow (4 by default)
	OPTIONS(1) = 0	DC
	OPTIONS(1) = 1	AC Newton
	OPTIONS(1) = 2	AC Decoupled
	OPTIONS(1) = 3	EMS
	OPTIONS(1) = 4	Do Not Solve
	OPTIONS(2)	Transformer Tap (0 by default)
	OPTIONS(2) = 0	disabled
	OPTIONS(2) = 1	enable stepping, No Direct
	OPTIONS(3)	Phase Shifter (0 by default)
	OPTIONS(3) = 0	disabled
	OPTIONS(3) = 1	enabled
	OPTIONS(4)	Switchable Shunts (1 by default)
	OPTIONS(4) = 0	disabled
	OPTIONS(4) = 1	enabled
	OPTIONS(5)	Area Interchange (0 by default)
	OPTIONS(5) = 0	disabled
	OPTIONS(5) = 1	Tie lines only
	OPTIONS(5) = 2	Tie lines and loads
	OPTIONS(6)	DC Tap (1 by default)
	OPTIONS(6) = 0	Locked
	OPTIONS(6) = 1	On
	OPTIONS(7)	Iter. to begin Var limiting (0 by default)
	OPTIONS(7) = 0	Hot Start: apply var limits immediately
	OPTIONS(7) = 99	Automatic:
	OPTIONS(8)	Load Flow Iterations Limit (20 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(8) > n	Terminate calculation after n iterations
	OPTIONS(9)	Use Flat Start for Load Flow Solution (0 by default)
	OPTIONS(9) = 0	hot
	OPTIONS(9) = 1	flat
	OPTIONS(10)	Iteration to freeze adjustments (15 by default)
	OPTIONS(10) > n	Freeze adjustments after n iterations
Real VALUES (3)	array of 3 elements specifying reporting parameters(input).	
	VALUES(1)	Real and Reactive Power Tolerance (0.1 by default)
	VALUES(2)	Reactive Adjustments DeAcceleration Factor (1.0 by default)
	VALUES(3)	Low Voltage break point (pu) for load scaling (0.7 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer option value is missing
	IERR = 3	Float option value is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.6. ACLFNS

Use this function to execute an AC load flow solution using the Full Newton Method and the specified calculation options

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ACLFNS  OPTSI(1)..OPTS(10) VALUES(1)..VALUES(3)
```

Python command syntax:

```
ierr = aclfns(options, values)
```

Fortran command syntax:

```
CALL MAPI_ACLFNS(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (10)	array of 10 elements specifying calculation options(input).	
	OPTIONS(1)	Base Case Load Flow (4 by default)
	OPTIONS(1) = 0	DC
	OPTIONS(1) = 1	AC Newton
	OPTIONS(1) = 2	AC Decoupled
	OPTIONS(1) = 3	EMS
	OPTIONS(1) = 4	Do Not Solve
	OPTIONS(2)	Transformer Tap (0 by default)
	OPTIONS(2) = 0	disabled
	OPTIONS(2) = 1	enable stepping, No Direct
	OPTIONS(3)	Phase Shifter (0 by default)
	OPTIONS(3) = 0	disabled
	OPTIONS(3) = 1	enabled
	OPTIONS(4)	Switchable Shunts (1 by default)
	OPTIONS(4) = 0	disabled
	OPTIONS(4) = 1	enabled
	OPTIONS(5)	Area Interchange (0 by default)
	OPTIONS(5) = 0	disabled
	OPTIONS(5) = 1	Tie lines only
	OPTIONS(5) = 2	Tie lines and loads
	OPTIONS(6)	DC Tap (1 by default)
	OPTIONS(6) = 0	Locked
	OPTIONS(6) = 1	On
	OPTIONS(7)	Iter. to begin Var limiting (0 by default)
	OPTIONS(7) = 0	Hot Start: apply var limits immediately
	OPTIONS(7) = 99	Automatic:
	OPTIONS(8)	Load Flow Iterations Limit (20 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(8) > n	Terminate calculation after n iterations
	OPTIONS(9)	Use Flat Start for Load Flow Solution (0 by default)
	OPTIONS(9) = 0	hot
	OPTIONS(9) = 1	flat
	OPTIONS(10)	Iteration to freeze adjustments (15 by default)
	OPTIONS(10) > n	Freeze adjustments after n iterations
Real VALUES (3)	array of 3 elements specifying reporting parameters(input).	
	VALUES(1)	Real and Reactive Power Tolerance (0.1 by default)
	VALUES(2)	Reactive Adjustments DeAcceleration Factor (1.0 by default)
	VALUES(3)	Low Voltage break point (pu) for load scaling (0.7 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer option value is missing
	IERR = 3	Float option value is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.7. ACSOLVE_LFD

Use this function to execute an AC load flow solution using the Fast Decoupled Method.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ACSOLVE_LFD  DFLAG
```

Python command syntax:

```
ierr = acsolve_lfd(dflag)
```

Fortran command syntax:

```
CALL MAPI_ACSOLVE_LFD(DFLAG, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.8. ACSOLVE_LFN

Use this function to execute an AC load flow solution using the Full Newton Method.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ACSOLVE_LFN  DFLAG
```

Python command syntax:

```
ierr = acsolve_lfn(dflag)
```

Fortran command syntax:

```
CALL MAPI_ACSOLVE_LFN(DFLAG, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.9. ACTIVITYACCONT

Use this function to initiate or startup the ACCONT Activities. This function sets up the memory allocations required to perform the analyses listed below. Attemping to execute any of the listed analyses without first invoking this function will not allow the analysis to proceed. You will be prompted to run this function first. Analyses which require this function to be run first. Note that any or all of the listed analyses can be run with a single call to ACTIVITYACCONT. It is not necessary to EXITACTIVITYACCONT as long as you are traversing these listed analyses: Execution (running) of the AC contingency analysis load flow; Reading or Saving the AC contingency analysis file (.acc); AC Constraints analyses by branches, interfaces, flowgates or voltages; AC Contingency analyses by contingencies

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ACTIVITYACCONT
```

Python command syntax:

```
ierr = activityaccont()
```

Fortran command syntax:

```
CALL MAPI_PREACCONTANAL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.10. ACTIVITYMUST

Use this function to initiate or startup the PSSE MUST module Activities. This function sets up the memory allocations required to perform the analyses listed below. Attemping to execute any of the listed analyses without first invoking this function will not allow the analysis to proceed. You will be prompted to run this function first.
Analyses which require this function to be run first. Note that any or all of the listed analyses can be run with a single call to ACTIVITYMUST. It is not necessary to EXITACTIVITYMUST as long as you are traversing these listed analyses:
- DC Constraints analyses by branches, interfaces or flowgates
- DC Contingency analyses by contingencies
- DC Transfer analyses
- AC Transfer limit analysis
- Transfer versus transfer sensitivities
- Generator sensitivites on study transfer
- Transaction impact analysis

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ACTIVITYMUST
```

Python command syntax:

```
ierr = activitymust()
```

Fortran command syntax:

```
CALL MAPI_PREMUSTANAL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Could not initiate MUST DC Analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.11. ACTRANSFERLIMITSTUDY

Use this function to execute an AC Study Transfer.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_ACTRANSFERLIMITSTUDY  DFLAG SYS1 SYS2 TRANSFER
```

Python command syntax:

```
ierr = actransferlimitstudy(dflag, sys1, sys2, transfer)
```

Fortran command syntax:

```
CALL MAPI_ACTRANSFERLIMITSTUDY(DFLAG, SYS1, SYS2, TRANSFER, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem(s) missing
	IERR = 3	No transfer value
	IERR = 4	Subsystem(s) not valid
	IERR = 5	Sending subsystem already selected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.12. ADDTRANSACTION

Use this function to add any transaction for dispatch analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ADDTRANSACTION  SUBSFR SUBSIN TRANS
```

Python command syntax:

```
ierr = addtransaction(subsfr, subsin, trans)
```

Fortran command syntax:

```
CALL MAPI_ADDTRAN(SUBSFR, SUBSIN, TRANS, IERR)
```

Where:

Character*32 SUBSFR	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SUBSIN	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANS	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	From SubSystem same as TO SubSystem
	IERR = 4	Subsystem label is missing.
	IERR = 5	The MW transfer input is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.13. APPLYSINGLECONT

Use this function to modify the case by applying a single contingency.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_APPLYSINGLECONT  CONTNO
```

Python command syntax:

```
ierr = applysinglecont(contno)
```

Fortran command syntax:

```
CALL MAPI_APPSELCONT(CONTNO, IERR)
```

Where:

Integer CONTNO	Is the number of the contingency to be applied. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Selected Contingency out of range
	IERR = 3	The number of the contingency is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.14. AREABUBBLE

Use this function to update values for the Area Bubble Diagram.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_AREABUBBLE SYS1, SYS2, TRANSFER
```

Python command syntax:

```
ierr = areabubble(sys1, sys2, transfer)
```

Fortran command syntax:

```
CALL MAPI_UPDAREABUBL(SYS1, SYS2, TRANSFER, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem(s) missing
	IERR = 3	No transfer value
	IERR = 4	Subsystem(s) not valid
	IERR = 5	Sending subsystem already selected
	IERR = 6	Overlapping systems

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.15. AREAIMPACT

Use this function to generate the impact report between Areas, Zones and/or Subsystems

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_AREAIMPACT SYS1 SYS2 TRANSFER OPTIONS(1)..OPTIONS(5)
 CUTOFFS(1)..CUTOFFS(3) NUMFROM LISTFROM NUMTO LISTTO
```

Python command syntax:

```
ierr = areaimpact(sys1, sys2, transfer, options, cutoffs, numfrom,
 listfrom, numto, listto )
```

Fortran command syntax:

```
CALL MAPI_AZSREPORT(SYS1, SYS2, TRANSFER, OPTIONS, CUTOFFS, NUMFROM,
 LISTFROM, NUMTO, LISTTO, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).
Integer OPTIONS (5)	array of 5 elements specifying calculation options(input).

OPTIONS(1)	Type of Report: (1 by default)
OPTIONS(1) = 1	Total Participation Factor (TPF)
OPTIONS(1) = 2	Interface Participation Factor (IPF)
OPTIONS(1) = 3	Detailed Report for all Tie branches (Ties)
OPTIONS(2)	Sort Order: (2 by default)
OPTIONS(2) = 1	by Number
OPTIONS(2) = 2	by Impact
OPTIONS(2) = 3	by Name
OPTIONS(3)	Contingency for impact report (0 by default)
OPTIONS(3) = 0	Base Case
OPTIONS(3) = x	x = Contingency number
OPTIONS(4)	Impacted systems group (1 by default)
OPTIONS(4) = 1	Areas
OPTIONS(4) = 2	Zones
OPTIONS(4) = 3	Subsystems
OPTIONS(5)	Neighboring systems group (1 by default)
OPTIONS(5) = 1	Areas
OPTIONS(5) = 2	Zones

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 3	Subsystems
Real CUTOFFS (3)	array of 3 elements specifying cutoff values(input).	
	CUTOFFS(1)	TPF Report Cutoff (0.03 by default)
	CUTOFFS(1) = x	x = Cutoff value for TPF Report
	CUTOFFS(2)	IPF Report Cutoff (0.02 by default)
	CUTOFFS(2) = y	y = Cutoff value for IPF Report
	CUTOFFS(3)	Tie Report Cutoff (0.00 by default)
	CUTOFFS(3) = z	z = Cutoff value for Tie Report
Integer NUMFROM	How many impacted systems to consider. (input; no default allowed).	
Character*32 LISTFROM	Is a set of strings forming a list of Impacted Areas, Zones or Subsystems consistent with OPTIONS(4) (input; no default allowed).	
Integer NUMTO	How many neighboring systems to consider. (input; no default allowed).	
Character*32 LISTTO	Is a set of strings forming a list of Neighboring Areas, Zones or Subsystems consistent with OPTIONS(5) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.16. AREASINZONES

Use this function to a report of areas in zones.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_AREASINZONES
```

Python command syntax:

```
ierr = areasinzones()
```

Fortran command syntax:

```
CALL MAPI_AREASINZONES(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.17. AREASLACKS

Use this function to a report of slack buses.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_AREASLACKS
```

Python command syntax:

```
ierr = areaslacks()
```

Fortran command syntax:

```
CALL MAPI_AREASLACKS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.18. AREASUMMARY

Use this function to generate an area summary report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_AREASUMMARY
```

Python command syntax:

```
ierr = areasummary()
```

Fortran command syntax:

```
CALL MAPI_AREASUMMARY(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.19. BRANCHSENSITIVITY

Use this function to generate the monitored elements sensitivities for specified branches

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_BRANCHSENSITIVITY NUMMONEL LISTML OPTIONS(1)..OPTIONS(4) REFSYS
 IMPACTCUTOFF EXPSYS IMPSYS NUMTRANS LISTTRANS
```

Python command syntax:

```
ierr = branchsensitivity(nummonel, listml, options, refsys, impactcutoff,
 expsys, impsys, numtrans, listtrans)
```

Fortran command syntax:

```
CALL MAPI_BRSENS(NUMMONEL, LISTML, OPTIONS, REFSYS, IMPACTCUTOFF, EXPSYS,
 IMPSYS, NUMTRANS, LISTTRANS, IERR)
```

Where:

Integer NUMMONEL	How many monitored elements to consider. (input; no default allowed).
Character*32 LISTML (3*NUMMONEL)	Is a set of strings forming a list of branches. Each branch consists of a FROM bus followed by a TO bus followed by a Circuit ID. There must be as many sets of FROM TO CktID as indicated by NUMMONEL. (input; no default allowed).
Integer OPTIONS (4)	array of 4 elements specifying calculation options(input).
	OPTIONS(1) — Sensitivity Report to: (1 by default)
	OPTIONS(1) = 1 — Reference Subsystem
	OPTIONS(1) = 2 — Factors for Impact Cutoff
	OPTIONS(1) = 3 — Subsystem Participating Buses
	OPTIONS(1) = 4 — Transactions
	OPTIONS(2) — Contingency for impact/sensitivity reports (0 by default)
	OPTIONS(2) = 0 — Base Case
	OPTIONS(2) = x — x = Contingency number
	OPTIONS(3) — Reference subsystem config for sensitivity analysis (1 by default)
	OPTIONS(3) = 1 — Importing
	OPTIONS(3) = 2 — Exporting
	OPTIONS(4) — Transactions to consider (1 by default)
	OPTIONS(4) = 1 — All possible transactions
	OPTIONS(4) = 2 — Only selected transactions
Character*32 REFSYS	Is a string identifying the Reference Subsystem Name - to be used with Report 1 of OPTION(1). Enter -1 for Swing Bus. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real IMPACTCUTOFF	Impact CutOff Value - to be used with Report 2 of OPTION(1) (input; 0 by default).
Character*32 EXPSYS	Is a string identifying the Exporting Subsystem Name - to be used with Report 3 of OPTION(1) (input; blank by default).
Character*32 IMPSYS	Is a string identifying the Importing Subsystem Name - to be used with Report 3 of OPTION(1) (input; blank by default).
Integer NUMTRANS	How many transactions in LISTTRANS - to be used with Report 4 of OPTION(1). (input; no default allowed).
Character*32 LISTTRANS (2*NUMTRANS)	Is a set of strings forming a list of subsystem names representing transactions with each transaction being made up of a FROM subsystem and a TO subsystem. (input; no default allowed).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Prerequisite requirements for function not met
	IERR = 2 Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.20. BUBBLEREPORT

Use this function to generate values for a Bubble Diagram.

This API routine was first introduced in release 35.6.0.

Batch command syntax:

```
BAT_BUBBLEREPORT SYS1 SYS2 TRANSFER OPTION(1) OPTION(2) CUTOFF(1)
 CUTOFF(2)
```

Python command syntax:

```
ierr = bubblereport(sys1, sys2, transfer, option, cutoff)
```

Fortran command syntax:

```
CALL MAPI_BUBBLEREPORT(SYS1, SYS2, TRANSFER, OPTION, CUTOFF, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer OPTION (2)	Option for the transfer solution (input; no default allowed).	
real CUTOFF (2)	Cutoff values (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem(s) missing
	IERR = 3	No transfer value
	IERR = 4	Subsystem(s) not valid
	IERR = 5	Sending subsystem already selected
	IERR = 6	Overlapping systems

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.21. CDETAILBRANCHES

Use this function to generate a detailed report by contingencies for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CDETAILBRANCHES
```

Python command syntax:

```
ierr = cdetailbranches()
```

Fortran command syntax:

```
CALL MAPI_CNCDETAILBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.22. CDETAILINTERFACES

Use this function to generate a detailed report by contingencies for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CDETAILINTERFACES
```

Python command syntax:

```
ierr = cdetailinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNCDETAILINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.23. CDETAILVOLTAGEBUSES

Use this function to generate a detailed report by contingencies for the selected voltage buses.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CDETAILVOLTAGEBUSES
```

Python command syntax:

```
ierr = cdetailvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_CNCDETAILVOLT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.24. CHANGEPARAMS

Use this function to change any of the option parameters used in the PSSE MUST module.
where:

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CHANGEPARAMS  MENUNM PARMID NEWVALUE
```

Python command syntax:

```
ierr = changeparams(menunm, parmid, newvalue)
```

Fortran command syntax:

```
CALL MAPI_CHANGEPARM(MENUNM, PARMID, NEWVALUE, IERR)
```

Where:

Character*18 MENUNM — Is a string identifying which menu contains the option parameter you wish to change. Acceptable entries are: (input; no default allowed).

'MAINOPTIONS'	General program setting options
'LFADJOPTIONS'	Load Flow Adjustment options
'LFGENOPTIONS'	Generator Var Limits Checking options
'LFSOLOPTIONS'	AC Load Flow Solution options
'LFADVOPTIONS'	Advanced Load Flow Solution options
'CONSTROPTS'	LP/OPF Constraint/Contingency options
'MUSTOPTIONS'	MUST Run time solution options
'ACVEROPTIONS'	AC verification options
'TRANVIOLOPTS'	FCITC Violations report format options
'ACCONTOPTS'	AC Contingency analysis options
'CONTOPTS'	Contingency reporting options
'TRANIMPOPTS'	Transfer impact analysis Options
'TRGENOPTS'	Generation worst dispatch options
'VOLTAGEOPTS'	Voltage violations with respect to specified limits
'BRANREPOPTS'	Branch violations (MW) and impact report analysis options
'BRANAMPOPTS'	Branch violations (MVA) and impact report analysis options
'FGATEREPOPTS'	Flowgate violations and impact report analysis options
'INTRREPOPTS'	Interface violations and impact report analysis options

Integer PARMID (1) — array of 1 elements specifying Is the index of the option parameter you want to change. See below for a full list of menus and indices(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PARMID(1)	MAINOPTIONS : General program setting options
PARMID(1) = 1	Console output lines per page
PARMID(1) = 2	File output lines per page. If >= 99999 set to 9999999
PARMID(1) = 3	Bus Input/Output by NAMES or NUMBERS (1- NUMBERS, 2- NAMES)
PARMID(1) = 4	Line shunts in pout (1- no, 2- yes)
PARMID(1) = 5	Separator in ASC file (1- /, 2- \\, 3- ;, 4- !)
PARMID(1) = 6	Log File (1- Enabled Log File, 0- Disabled Log File)
PARMID(2)	LFADJOPTIONS: Load Flow Adjustment options
PARMID(2) = 1	Transformer Tap (0- disabled, 1- enabled stepping, No Direct)
PARMID(2) = 2	Phase Shifter (0- disabled, 1- enabled)
PARMID(2) = 3	Switchable Shunts (0- disabled, 1- enabled)
PARMID(2) = 4	Area Interchange (0- disabled, 1- Tie lines only, 2- Tie lines and loads)
PARMID(2) = 5	DC TAP (0- Locked, 1- On)
PARMID(3)	LFGENOPTIONS: Generator Var Limits Checking options
PARMID(3) = 1	Enable Reactive Limit Checking (0- disabled, 1- enabled)
PARMID(3) = 2	Iter. to begin Var limiting (flat start), 99 for automatic
PARMID(3) = 3	Iter. to begin Var limiting (hot start), 99 for automatic
PARMID(4)	LFSOLOPTIONS: AC Load Flow Solution options
PARMID(4) = 1	Load Flow Iterations Limit
PARMID(4) = 2	Real and Reactive Power Tolerance
PARMID(4) = 3	Enable Non-divergent Load Flow (0- disabled, 1- enabled)
PARMID(4) = 4	Use Flat Start for Load Flow Solution (0- hot, 1- flat)
PARMID(4) = 5	Reactive Adjustments DeAcceleration Factor
PARMID(4) = 6	Tap adjustments DeAcceleration Factor
PARMID(4) = 7	Low Voltage break point (pu) for load scaling
PARMID(4) = 8	Iteration to freeze adjustments
PARMID(5)	LFADVOPTIONS: Advanced Load Flow Solution options
PARMID(5) = 1	Maximum voltage magnitude change per iteration
PARMID(5) = 2	Number of first LF iterations to limit voltage change

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PARMID(5) = 3	Maximum Tap ratio change per iteration
PARMID(5) = 4	Voltage tolerance (PU) to activate TAP, shunt and DC line adjustments
PARMID(5) = 5	Angle tolerance to activate phase shifters and area interchange adjustments
PARMID(5) = 6	Maximum voltage change in blow up test
PARMID(5) = 7	Maximum angle change in blow up test (except first iteration)
PARMID(5) = 8	Report data warnings
PARMID(6)	CONSTROPTS : LP/OPF Constraint/Contingency options
PARMID(6) = 1	AC MW Mismatch Tolerance
PARMID(6) = 2	Base Case Rating (Select Rating 1-12)
PARMID(6) = 3	Base Case Percent of Rating (branches only)
PARMID(6) = 4	Contingency Case Rating (Select Rating 1-12)
PARMID(6) = 5	Contingency Case Percent of Rating (branches only)
PARMID(6) = 6	Base Case Load Flow (0- DC, 1- AC Newton, 2- AC Decoupled, 3- EMS, 4- Do not solve AC, 5- PSSE Presolved)
PARMID(6) = 7	Convert branch ratings to estimated MW ratings(0- No, 1- Yes)
PARMID(6) = 8	Contingency ID Reporting (1- labels, 2- events, 3- both)
PARMID(6) = 9	Maximum Number of Contingencies to process
PARMID(6) = 10	Line Rating Unit for AC Applications (0- Current expressed as MVA, 1- MVA)
PARMID(7)	MUSTOPTIONS : MUST Run time solution options
PARMID(7) = 1	Phase shifter code (0- fixed, 1- const.flow base case, 2- const.flow in cont)
PARMID(7) = 2	0- ignore base case constraints in FCITC reports, 1- include
PARMID(7) = 3	Max. number of violations to report in FCITC and Gen.Sens. Tables
PARMID(7) = 4	Default minimum distribution factor magnitude cutoff (PTDF&OTDF)
PARMID(7) = 5	Summary table maximum times to report the same element (100- Unlimited)
PARMID(7) = 6	Apply Min.Distr.Factor CutOff for contingency analysis (0- no, 1- yes)
PARMID(7) = 7	Apply minimum distr. factor to FCITC solution reports (0- no, 1- yes)
PARMID(7) = 8	Minimum contingency case (pre&post) flow change

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PARMID(7) = 9	Minimum contingency case distribution factor change
PARMID(7) = 10	Minimum distribution factor for transfer sensitivity analysis
PARMID(8)	ACVEROPTIONS: AC verification options
PARMID(8) = 1	AC LF method (1- Newton, 2- Decoupled)
PARMID(8) = 2	Apply non-divergent LF (0- never, 1- always, 2- only if regular diverged)
PARMID(8) = 3	Maximum number violations to verify using AC
PARMID(8) = 4	Tolerance to check study transfer (MW)
PARMID(8) = 5	Report removed violations with FCITC report (0- no, 1- yes)
PARMID(9)	TRANVIOLOPTS: FCITC Violations report format options
PARMID(9) = 1	FCITC format (1- two lines report, 2- wide, 3- TLTG, 4- wide w xfer, 5- DP2Line, 6- DPWide)
PARMID(9) = 2	Report removed violations with FCITC report (0- no, 1- yes, 2- only removed (for 1 4))
PARMID(9) = 3	Report FCITC violations (1- yes, 0- no)
PARMID(9) = 4	Report Export level(1- yes, 0- no)
PARMID(9) = 5	Report Import level(1- yes, 0- no)
PARMID(9) = 6	Report Flowgate/Interfaces flows (0- no, 1- interface, 2- flowgate)
PARMID(10)	ACCONTOPTS : AC Contingency analysis options
PARMID(10) = 1	AC LF method (1- Newton, 2- Decoupled)
PARMID(10) = 2	Apply non-divergent LF (0- never, 1- always, 2- only if regular diverged)
PARMID(10) = 3	Minimum contingency case flow change cutoff
PARMID(10) = 4	Minimum voltage contingency change cutoff in percent
PARMID(11)	CONTOPTS : Contingency reporting options
PARMID(11) = 1	Report details on contingency violations (0- no report, 1- report)
PARMID(11) = 2	Report details on contingency events (0- no report, 1- report)
PARMID(11) = 3	Report islands details (0- no report, 1- report)
PARMID(11) = 4	Report details on dispatch events (0- no report, 1- report)
PARMID(12)	TRANIMPOPTS : Transfer impact analysis Options
PARMID(12) = 1	Sort method for reporting TPF(1- by number, 2- by Impact, 3- by name)
PARMID(12) = 2	TPF (trans. participation factor) report cut off

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PARMID(12) = 3	IPF (interface participation factor) report cut off
PARMID(12) = 4	Ties (PTDF/OTDF) report cut off
PARMID(13)	TRGENOPTS : Generation worst dispatch options
PARMID(13) = 1	Specify maximum power transfer (MW) to test
PARMID(13) = 2	Maximum number of worst scenarios to report
PARMID(13) = 3	Worst dispatch subsystem (1- import, 2- export)
PARMID(13) = 4	Show all participating generators in detailed report (0- no, 1- yes)
PARMID(13) = 5	Minimum Factor CutOff for individual generators
PARMID(13) = 6	Minimum Factor CutOff for study transfer
PARMID(14)	VOLTAGEOPTS : Voltage violations with respect to specified limits
PARMID(14) = 1	Low voltage violations tolerance (Vmin-Vtol-Low, use negative to check how close)
PARMID(14) = 2	High voltage violations tolerance (Vmax +VtolHigh, negative - within range)
PARMID(14) = 3	Low voltage cutoff
PARMID(14) = 4	High voltage cutoff
PARMID(15)	BRANREPOPTS : Branch violations (MW) and impact report analysis options
PARMID(15) = 1	Loading Cutoff for reporting violations for the Current case(in %)
PARMID(15) = 2	Loading Cutoff for reporting violations for the Initial case(in %)
PARMID(15) = 3	Check/report initial violations (0- no, 1- yes)
PARMID(15) = 4	Report order (1- bus order, 2- loading, 3- impact)
PARMID(15) = 5	Apply Impact CutOff(0- no, 1- yes)
PARMID(15) = 6	Impact CutOff Value in percent
PARMID(15) = 7	Apply Loading CutOff(0- no, 1- yes)
PARMID(15) = 8	Show Study Transfer Margins(0- no, 1- yes)
PARMID(15) = 9	Subsystem for Study transfer sensitivity analysis (1- importing, 2- exporting)
PARMID(15) = 10	Contingency for impact/sensitivity reports (0- base case, >0- Contin.Index)
PARMID(16)	BRANAMPOPTS : Branch violations (MVA) and impact report analysis options
PARMID(16) = 1	Loading Cutoff for reporting violations for the Current case(in %)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PARMID(16) = 2	Loading Cutoff for reporting violations for the Initial case(in %)
PARMID(16) = 3	Check/report initial violations (0- no, 1- yes)
PARMID(16) = 4	Report order (1- bus order, 2- loading, 3- impact)
PARMID(16) = 5	Apply Impact CutOff(0- no, 1- yes)
PARMID(16) = 6	Impact CutOff Value in percent
PARMID(16) = 7	Apply Loading CutOff(0- no, 1- yes)
PARMID(16) = 8	Show Study Transfer Margins(0- no, 1- yes)
PARMID(16) = 9	Subsystem for Study transfer sensitivity analysis (1- importing, 2- exporting)
PARMID(16) = 10	Contingency for impact/sensitivity reports (0- base case, >0- Contin.Index)
PARMID(17)	FGATEREPOPTS: Flowgate violations and impact report analysis options
PARMID(17) = 1	Loading Cutoff for reporting violations for the Current case(in %)
PARMID(17) = 2	Loading Cutoff for reporting violations for the Initial case(in %)
PARMID(17) = 3	Check/report initial violations (0- no, 1- yes)
PARMID(17) = 4	Report order (1- input order, 2- loading, 3- impact)
PARMID(17) = 5	Apply Impact CutOff(0- no, 1- yes)
PARMID(17) = 6	Impact CutOff Value in percent
PARMID(17) = 7	Apply Loading CutOff(0- no, 1- yes)
PARMID(17) = 8	Show Study Transfer Margins(0- no, 1- yes)
PARMID(17) = 9	Subsystem for Study transfer sensitivity analysis (1- importing, 2- exporting)
PARMID(18)	INTRREPOPTS : Interface violations and impact report analysis options
PARMID(18) = 1	Loading Cutoff for reporting violations for the Current case(in %)
PARMID(18) = 2	Loading Cutoff for reporting violations for the Initial case(in %)
PARMID(18) = 3	Check/report initial violations (0- no, 1- yes)
PARMID(18) = 4	Report order (1- input order, 2- loading, 3- impact)
PARMID(18) = 5	Apply Impact CutOff(0- no, 1- yes)
PARMID(18) = 6	Impact CutOff Value in percent
PARMID(18) = 7	Apply Loading CutOff(0- no, 1- yes)
PARMID(18) = 8	Show Study Transfer Margins(0- no, 1- yes)
PARMID(18) = 9	Subsystem for Study transfer sensitivity analysis (1- importing, 2- exporting)
PARMID(18) = 10	Contingency for impact/sensitivity reports (0- base case, >0- Contin.Index)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real NEWVALUE	Is the value you want the option to be. A decimal point is not required for parameters which are integer selections. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid Menu Selected - No changes made
	IERR = 3	Invalid Parameter ID - No changes made
	IERR = 4	New value Out of Bounds - No changes made
	IERR = 5	The menu option setting is missing.
	IERR = 6	The option parameter is missing.
	IERR = 7	The option value is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.25. CLEARMULTISTUDY

The Multiple Study Transfer will continually add transfers to an existing report unless the function is cleared.
Use this api to clear the list of Multiple Study Transfers in order to generate new reports.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_CLEARMULTISTUDY
```

Python command syntax:

```
ierr = clearmultistudy()
```

Fortran command syntax:

```
CALL MAPI_CLEARTSENKNT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.26. CNALLTRANSELBRANCHES

Use this function to display the transactions impact on ALL branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNALLTRANSELBRANCHES
```

Python command syntax:

```
ierr = cnalltranselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNSTALLSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.27. CNALLTRANSELFLOWGATES

Use this function to display the transactions impact on ALL flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNALLTRANSELFLOWGATES
```

Python command syntax:

```
ierr = cnalltranselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNSTALLSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.28. CNALLTRANSELINTERFACES

Use this function to display the transactions impact on ALL interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNALLTRANSELINTERFACES
```

Python command syntax:

```
ierr = cnalltranselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNSTALLSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.29. CNALLVIOLATED

Use this function to show all violated constraints for the selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNALLVIOLATED
```

Python command syntax:

```
ierr = cnallviolated()
```

Fortran command syntax:

```
CALL MAPI_CNALLVIOLBYCASE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.30. CNALLVIOLFREQ

Use this function to show the frequency of violations for the selected contingencies

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNALLVIOLFREQ  FRTYPE
```

Python command syntax:

```
ierr = cnallviolfreq(frtype)
```

Fortran command syntax:

```
CALL MAPI_ALLVIOLFREQ(FRTYPE, IERR)
```

Where:

Character*12 FRTYPE	Is a string identifying the frequency type (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.31. CNBASEVIOLATED

Use this function to report all base case violations.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNBASEVIOLATED
```

Python command syntax:

```
ierr = cnbaseviolated()
```

Fortran command syntax:

```
CALL MAPI_CNBASEVIOLBYCASE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.32. CNCINITSELBRANCHES

Use this function to compare INITIAL contingency flows for selected contingencies for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNCINITSELBRANCHES
```

Python command syntax:

```
ierr = cncinitselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNCINITSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.33. CNCINITSELINTERFACES

Use this function to compare INITIAL contingency flows for selected contingencies for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNCINITSELINTERFACES
```

Python command syntax:

```
ierr = cncinitselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNCINITSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.34. CNCOMPSELBRANCHES

Use this function to compare INITIAL flows for selected contingencies for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNCOMPSELBRANCHES
```

Python command syntax:

```
ierr = cncompselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNCOMPSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.35. CNCOMPSELINTERFACES

Use this function to compare INITIAL flows for selected contingencies for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNCOMPSELINTERFACES
```

Python command syntax:

```
ierr = cncompselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNCOMPSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.36. CNCOMPSELVOLTAGEBUSES

Use this function to compare INITIAL contingency flows for selected contingencies for the selected voltage buses.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNCOMPSELVOLTAGEBUSES
```

Python command syntax:

```
ierr = cncompselvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_CNCOMPSELVOLT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.37. CNCRESULTSSELBRANCHES

Use this function to compare RESULTING contingency flows for selected contingencies for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNCRESULTSSELBRANCHES
```

Python command syntax:

```
ierr = cncresultsselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNCRESSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.38. CNCRESULTSSELINTERFACES

Use this function to compare RESULTING contingency flows for selected contingencies for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNCRESULTSSELINTERFACES
```

Python command syntax:

```
ierr = cncresultsselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNCRESSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.39. CNDETAILSELFLOWGATES

Use this function to display a detailed report of the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNDETAILSELFLOWGATES
```

Python command syntax:

```
ierr = cndetailselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNDETSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.40. CNDETAILSELINTERFACES

Use this function to display a detailed report of the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNDETAILSELINTERFACES
```

Python command syntax:

```
ierr = cndetailselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNDETSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.41. CNFACTORSSELFLOWGATES

Use this function to display the OTDF variation factors for the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNFACTORSSELFLOWGATES
```

Python command syntax:

```
ierr = cnfactorsselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNFACTSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.42. CNFREQSELBRANCHES

Use this function to generate a violations frequency report for selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNFREQSELBRANCHES
```

Python command syntax:

```
ierr = cnfreqselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNFREQSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.43. CNFREQSELINTERFACES

Use this function to generate a violations frequency report for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNFREQSELINTERFACES
```

Python command syntax:

```
ierr = cnfreqselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNFREQSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.44. CNFREQSELVOLTAGEBUSES

Use this function to generate a violations frequency report for the selected buses with monitored voltages.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNFREQSELVOLTAGEBUSES
```

Python command syntax:

```
ierr = cnfreqselvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_CNFREQSELVOLT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.45. CNGAPPSELBRANCHES

Use this function to display the GAPP factors for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNGAPPSELBRANCHES
```

Python command syntax:

```
ierr = cngappselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNGAPPSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.46. CNGAPPSELFLOWGATES

Use this function to display the GAPP factors for the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNGAPPSELFLOWGATES
```

Python command syntax:

```
ierr = cngappselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNGAPPSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.47. CNGAPPSELINTERFACES

Use this function to display the GAPP factors for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNGAPPSELINTERFACES
```

Python command syntax:

```
ierr = cngappselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNGAPPSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.48. CNIMPACTSELBRANCHES

Use this function to report on constraints impacted by transfer for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNIMPACTSELBRANCHES
```

Python command syntax:

```
ierr = cnimpactselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNIMPACTSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.49. CNIMPACTSELFLOWGATES

Use this function to report on constraints impacted by transfer for the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNIMPACTSELFLOWGATES
```

Python command syntax:

```
ierr = cnimpactselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNIMPACTSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.50. CNIMPACTSELINTERFACES

Use this function to report on constraints impacted by transfer for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNIMPACTSELINTERFACES
```

Python command syntax:

```
ierr = cnimpactselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNIMPACTSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.51. CNLISTSELBRANCHES

Use this function to generate a list of selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNLISTSELBRANCHES
```

Python command syntax:

```
ierr = cnlistselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNLISTSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.52. CNLISTSELFLOWGATES

Use this function to generate a list of selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNLISTSELFLOWGATES
```

Python command syntax:

```
ierr = cnlistselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNLISTSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.53. CNLISTSELINTERFACES

Use this function to generate a list of selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNLISTSELINTERFACES
```

Python command syntax:

```
ierr = cnlistselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNLISTSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.54. CNLISTSELVOLTAGEBUSES

Use this function to generate a list of selected buses with monitored voltages.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNLISTSELVOLTAGEBUSES
```

Python command syntax:

```
ierr = cnlistselvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_CNLISTSELVOLT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.55. CNOTDFSELBRANCHES

Use this function to display the OTDF variation factors for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNOTDFSELBRANCHES
```

Python command syntax:

```
ierr = cnotdfselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNOTDFSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.56. CNOTDFSELINTERFACES

Use this function to display the OTDF variation factors for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNOTDFSELINTERFACES
```

Python command syntax:

```
ierr = cnotdfselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNOTDFSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.57. CNSELECTBRANCHBYALL

Use this function to select ALL branches for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYALL
```

Python command syntax:

```
ierr = cnselectbranchbyall()
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYALL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.58. CNSELECTBRANCHBYAREAS

Use this function to select branches by areas for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYAREAS  VLOW VHIGH NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectbranchbyareas(vlow, vhigh, nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYAREAS(VLOW, VHIGH, NUMMONEL, LISTML, IERR)
```

Where:

Character*12 VLOW	Is the minimum voltage limit for selecting branches (input; no default allowed).	
Character*12 VHIGH	Is the maximum voltage limit for selecting branches (input; no default allowed).	
Integer NUMMONEL	(input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of areas. There must be as many areas as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The minimum voltage limit for selecting branches is missing.
	IERR = 3	The maximum voltage limit for selecting branches is missing.
	IERR = 4	The number of monitored elements is missing.
	IERR = 5	One area label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.59. CNSELECTBRANCHBYBRANCHES

Use this function to select branches for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYBRANCHES  NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectbranchbybranches(nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYBRANCHES(NUMMONEL, LISTML, IERR)
```

Where:

Integer NUMMONEL	(input; no default allowed).	
Character*32 LISTML (3*NUMMONEL)	Is a set of strings forming a list of branches. Each branch consists of a FROM bus followed by a TO bus followed by a Circuit ID. There must be as many sets of FROM TO CktID as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of monitored elements is missing.
	IERR = 3	One branch label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.60. CNSELECTBRANCHBYBUSES

Use this function to select branches by buses for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYBUSES  NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectbranchbybuses(nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYBUSES(NUMMONEL, LISTML, IERR)
```

Where:

Integer NUMMONEL	(input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of buses. There must be as many buses as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of monitored elements is missing.
	IERR = 3	One of bus labels is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.61. CNSELECTBRANCHBYIMPACT

Use this function to select branches by impact for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYIMPACT
```

Python command syntax:

```
ierr = cnselectbranchbyimpact()
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYIMP(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.62. CNSELECTBRANCHBYTIES

Use this function to select branches by ties for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYTIES  LISTML
```

Python command syntax:

```
ierr = cnselectbranchbyties(listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYTIES(LISTML, IERR)
```

Where:

Character*12 LISTML	Is a set of strings forming a list of ties. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The tie line labels are missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.63. CNSELECTBRANCHBYVIOL

Use this function to select branches by violation for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYVIOL  VIOLMETH
```

Python command syntax:

```
ierr = cnselectbranchbyviol(violmeth)
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYVIOL(VIOLMETH, IERR)
```

Where:

Integer VIOLMETH	Indicates which violation method to use and can be one of the following entries: (input; no default allowed).	
	VIOLMETH = 1:	Violated in the pre-shift case
	VIOLMETH = 2:	Violated/limiting after adjustment
	VIOLMETH = 3:	Violated in the pre-shift case OR after adjustment
	VIOLMETH = 4:	Violated in the pre-shift case AND after adjustment
	VIOLMETH = 5:	Violated/binding after, but not in the pre-shiftcase
	VIOLMETH = 6:	Violated in the pre-shift case, but not after adjustments
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Chosen Violation Method is out of range
	IERR = 3	The violation method number is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.64. CNSELECTBRANCHBYZONES

Use this function to select branches by zones for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTBRANCHBYZONES  VLOW VHIGH NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectbranchbyzones(vlow, vhigh, nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELBRABYZONES(VLOW, VHIGH, NUMMONEL, LISTML, IERR)
```

Where:

Character*12 VLOW	Is the minimum voltage limit for selecting branches (input; no default allowed).	
Character*12 VHIGH	Is the maximum voltage limit for selecting branches (input; no default allowed).	
Integer NUMMONEL	(input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of zones. There must be as many zones as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The minimum voltage limit for selecting branches is missing.
	IERR = 3	The maximum voltage limit for selecting branches is missing.
	IERR = 4	The number of monitored elements is missing.
	IERR = 5	One of the zone labels is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.65. CNSELECTFGATEBYALL

Use this function to select ALL flowgates for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTFGATEBYALL
```

Python command syntax:

```
ierr = cnselectfgatebyall()
```

Fortran command syntax:

```
CALL MAPI_CNSELFGBYALL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.66. CNSELECTFGATEBYCONT

Use this function to select Contingenct Flowgates Only for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTFGATEBYCONT
```

Python command syntax:

```
ierr = cnselectfgatebycont()
```

Fortran command syntax:

```
CALL MAPI_CNSELFGBYCONT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.67. CNSELECTFGATEBYID

Use this function to select flowgates by flowgate ID for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTFGATEBYID  NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectfgatebyid(nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELFGBYID(NUMMONEL, LISTML, IERR)
```

Where:

Integer NUMMONEL	Is the number of flowgates to be included in LISTML (input; no default allowed).	
Character*5 LISTML	Is a set of strings forming a list of flowgate ID's. There must be as many flowgates as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of flowgate IDs is missing.
	IERR = 3	One of the flowgate IDs is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.68. CNSELECTFGATEBYIMPACT

Use this function to select flowgates by impact for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTFGATEBYIMPACT
```

Python command syntax:

```
ierr = cnselectfgatebyimpact()
```

Fortran command syntax:

```
CALL MAPI_CNSELFGBYIMP(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.69. CNSELECTFGATEBYVIOL

Use this function to select flowgates by violation for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTFGATEBYVIOL  VIOLMETH
```

Python command syntax:

```
ierr = cnselectfgatebyviol(violmeth)
```

Fortran command syntax:

```
CALL MAPI_CNSELFGBYVIOL(VIOLMETH, IERR)
```

Where:

Integer VIOLMETH	Indicates which violation method to use and can be one of the following entries: (input; no default allowed).	
	VIOLMETH = 1:	Violated in the pre-shift case
	VIOLMETH = 2:	Violated/limiting after adjustment
	VIOLMETH = 3:	Violated in the pre-shift case OR after adjustment
	VIOLMETH = 4:	Violated in the pre-shift case AND after adjustment
	VIOLMETH = 5:	Violated/binding after, but not in the pre-shiftcase
	VIOLMETH = 6:	Violated in the pre-shift case, but not after adjustments
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Chosen Violation Method is out of range
	IERR = 3	The number of violation methods is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.70. CNSELECTINTERBYALL

Use this function to select ALL interfaces for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTINTERBYALL
```

Python command syntax:

```
ierr = cnselectinterbyall()
```

Fortran command syntax:

```
CALL MAPI_CNSELINTBYALL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.71. CNSELECTINTERBYIMPACT

Use this function to select interfaces by impact for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTINTERBYIMPACT
```

Python command syntax:

```
ierr = cnselectinterbyimpact()
```

Fortran command syntax:

```
CALL MAPI_CNSELINTBYIMP(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.72. CNSELECTINTERBYNAME

Use this function to select interfaces by interface name for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTINTERBYNAME  NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectinterbyname(nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELINTBYNAM(NUMMONEL, LISTML, IERR)
```

Where:

Integer NUMMONEL	Is the number of interfaces to be included in LISTML (input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of interface names. There must be as many interfaces as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of monitored interfaces is missing.
	IERR = 3	One of the interface labels is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.73. CNSELECTINTERBYNUMBER

Use this function to select interfaces by interface number for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTINTERBYNUMBER  NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectinterbynumber(nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELINTBYNUM(NUMMONEL, LISTML, IERR)
```

Where:

Integer NUMMONEL	Is the number of interfaces to be included in LISTML (input; no default allowed).	
Character*5 LISTML	Is a set of strings forming a list of interface numbers. There must be as many interfaces as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of monitored interfaces is missing.
	IERR = 3	One of interface labels is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.74. CNSELECTINTERBYVIOL

Use this function to select interfaces by violation for a constraints analysis.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTINTERBYVIOL  VIOLMETH
```

Python command syntax:

```
ierr = cnselectinterbyviol(violmeth)
```

Fortran command syntax:

```
CALL MAPI_CNSELINTBYVIOL(VIOLMETH, IERR)
```

Where:

Integer VIOLMETH	Isdicates which violation method to use and can be one of the following entries: (input; no default allowed).	
	VIOLMETH = 1:	Violated in the pre-shift case
	VIOLMETH = 2:	Violated/limiting after adjustment
	VIOLMETH = 3:	Violated in the pre-shift case OR after adjustment
	VIOLMETH = 4:	Violated in the pre-shift case AND after adjustment
	VIOLMETH = 5:	Violated/binding after, but not in the pre-shiftcase
	VIOLMETH = 6:	Violated in the pre-shift case, but not after adjustments
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Chosen Violation Method is out of range
	IERR = 3	The number of violation methods is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.75. CNSELECTVOLTAGEBUSESBYVIOL

Use this function to select buses with monitored voltages by violation.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTVOLTAGEBUSESBYVIOL  VIOLMETH
```

Python command syntax:

```
ierr = cnselectvoltagebusesbyviol(violmeth)
```

Fortran command syntax:

```
CALL MAPI_CNSELVOLTBYVIOL(VIOLMETH, IERR)
```

Where:

Integer VIOLMETH	Indicates which violation method to use and can be one of the following entries: (input; no default allowed).	
	VIOLMETH = 1:	Violated in the pre-shift case
	VIOLMETH = 2:	Violated/limiting after adjustment
	VIOLMETH = 3:	Violated in the pre-shift case OR after adjustment
	VIOLMETH = 4:	Violated in the pre-shift case AND after adjustment
	VIOLMETH = 5:	Violated/binding after, but not in the pre-shiftcase
	VIOLMETH = 6:	Violated in the pre-shift case, but not after adjustments
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Chosen Violation Method is out of range
	IERR = 3	The number of violation methods is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.76. CNSELECTVOLTBUSBYALL

Use this function to select ALL buses with monitored voltages.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTVOLTBUSBYALL
```

Python command syntax:

```
ierr = cnselectvoltbusbyall()
```

Fortran command syntax:

```
CALL MAPI_CNSELVOLTBYALL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.77. CNSELECTVOLTBUSBYAREAS

Use this function to select buses with monitored voltages by areas.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTVOLTBUSBYAREAS  VLOW VHIGH NUMMONEL
 LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectvoltbusbyareas(vlow, vhigh, nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELVOLTBYAREAS(VLOW, VHIGH, NUMMONEL, LISTML, IERR)
```

Where:

Character*12 VLOW	Is the minimum voltage limit for selecting buses (input; no default allowed).	
Character*12 VHIGH	Is the maximum voltage limit for selecting buses (input; no default allowed).	
Integer NUMMONEL	(input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of areas. There must be as many areas as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The minimum voltage limit for selecting buses is missing.
	IERR = 3	The maximum voltage limit for selecting buses is missing.
	IERR = 4	The number of monitored elements is missing.
	IERR = 5	One area label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.78. CNSELECTVOLTBUSBYBUSES

Use this function to select buses with monitored voltages by bus number.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTVOLTBUSBYBUSES  NUMMONEL LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectvoltbusbybuses(nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELVOLTBYBUSES(NUMMONEL, LISTML, IERR)
```

Where:

Integer NUMMONEL	(input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of voltage buses. There must be as many voltage buses as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of voltage buses is missing.
	IERR = 3	One bus label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.79. CNSELECTVOLTBUSBYZONES

Use this function to select buses with monitored voltages by zones.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTVOLTBUSBYZONES  VLOW VHIGH NUMMONEL
 LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = cnselectvoltbusbyzones(vlow, vhigh, nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_CNSELVOLTBYZONES(VLOW, VHIGH, NUMMONEL, LISTML, IERR)
```

Where:

Character*12 VLOW	Is the minimum voltage limit for selecting buses (input; no default allowed).	
Character*12 VHIGH	Is the maximum voltage limit for selecting buses (input; no default allowed).	
Integer NUMMONEL	(input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of zones. There must be as many zones as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The minimum voltage limit for selecting buses is missing.
	IERR = 3	The maximum voltage limit for selecting buses is missing.
	IERR = 4	The number of monitored elements is missing.
	IERR = 5	One zone label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.80. CNSELECTVOLTBUSESBYDEVIATION

Use this function to select branches by deviation.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSELECTVOLTBUSESBYDEVIATION  VLOW VHIGH
```

Python command syntax:

```
ierr = cnselectvoltbusesbydeviation(vlow, vhigh)
```

Fortran command syntax:

```
CALL MAPI_CNSELVOLTBYDEV(VLOW, VHIGH, IERR)
```

Where:

Real VLOW	Is the minimum voltage limit for selecting buses (input; no default allowed).	
Real VHIGH	Is the maximum voltage limit for selecting buses (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	VLOW outside of acceptable limits
	IERR = 3	VHIGH outside of acceptable limits
	IERR = 4	The minimum voltage limit for selecting buses is missing.
	IERR = 5	The maximum voltage limit for selecting buses is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.81. CNSENSUBSELBRANCHES

Use this function to display the subsystem sensitivities for the first selected branch.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSENSUBSELBRANCHES
```

Python command syntax:

```
ierr = cnsensubselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNSENSSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.82. CNSENSUBSELFLOWGATES

Use this function to display the subsystem sensitivities for the first selected flowgate.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSENSUBSELFLOWGATES
```

Python command syntax:

```
ierr = cnsensubselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNSENSSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.83. CNSENSUBSELINTERFACES

Use this function to display the subsystem sensitivities for the first selected interface.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSENSUBSELINTERFACES
```

Python command syntax:

```
ierr = cnsensubselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNSENSSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.84. CNSTUDYSELBRANCHES

Use this function to generate a sensitivity analysis for study transfer on the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSTUDYSELBRANCHES  SUBSFR SUBSIN
```

Python command syntax:

```
ierr = cnstudyselbranches(subsfr, subsin)
```

Fortran command syntax:

```
CALL MAPI_CNSTUDYSELBRA(SUBSFR, SUBSIN, IERR)
```

Where:

Character*32 SUBSFR	Is a string identifying FROM Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SUBSIN	Is a string identifying TO Subsystem Name to be used in the study transfer. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	FROM SubSystem same as TO SubSystem
	IERR = 4	The FROM Subsystem Name is missing.
	IERR = 5	The TO Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.85. CNSTUDYSELFLOWGATES

Use this function to generate a sensitivity analysis for study transfer on the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSTUDYSELFLOWGATES  SUBSFR SUBSIN
```

Python command syntax:

```
ierr = cnstudyselflowgates(subsfr, subsin)
```

Fortran command syntax:

```
CALL MAPI_CNSTUDYSELFG(SUBSFR, SUBSIN, IERR)
```

Where:

Character*32 SUBSFR	Is a string identifying FROM Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SUBSIN	Is a string identifying TO Subsystem Name to be used in the study transfer. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	FROM SubSystem same as TO SubSystem
	IERR = 4	The FROM Subsystem Name is missing.
	IERR = 5	The TO Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.86. CNSTUDYSELINTERFACES

Use this function to generate a sensitivity analysis for study transfer on the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSTUDYSELINTERFACES  SUBSFR SUBSIN
```

Python command syntax:

```
ierr = cnstudyselinterfaces(subsfr, subsin)
```

Fortran command syntax:

```
CALL MAPI_CNSTUDYSELINT(SUBSFR, SUBSIN, IERR)
```

Where:

Character*32 SUBSFR	Is a string identifying FROM Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SUBSIN	Is a string identifying TO Subsystem Name to be used in the study transfer. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	FROM SubSystem same as TO SubSystem
	IERR = 4	The FROM Subsystem Name is missing.
	IERR = 5	The TO Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.87. CNSUBSELBRANCHES

Use this function to report on the subsystem branch factors for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSUBSELBRANCHES   SUBS
```

Python command syntax:

```
ierr = cnsubselbranches(subs)
```

Fortran command syntax:

```
CALL MAPI_CNSUBSELBRA(SUBS, IERR)
```

Where:

Character*32 SUBS	Is a string identifying the Subsystem Name for which to report the branch factors. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	The Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.88. CNSUBSELFLOWGATES

Use this function to report on the subsystem branch factors for the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSUBSELFLOWGATES   SUBS
```

Python command syntax:

```
ierr = cnsubselflowgates(subs)
```

Fortran command syntax:

```
CALL MAPI_CNSUBSELFG(SUBS, IERR)
```

Where:

Character*32 SUBS	Is a string identifying the Subsystem Name for which to report the flowgate factors. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	The Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.89. CNSUBSELINTERFACES

Use this function to report on the subsystem branch factors for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSUBSELINTERFACES  SUBS
```

Python command syntax:

```
ierr = cnsubselinterfaces(subs)
```

Fortran command syntax:

```
CALL MAPI_CNSUBSELINT(SUBS, IERR)
```

Where:

Character*32 SUBS	Is a string identifying the Subsystem Name for which to report the interface factors. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	The Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.90. CNSUMMARYVIOLATIONS

Use this function to report a summary of violations.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNSUMMARYVIOLATIONS
```

Python command syntax:

```
ierr = cnsummaryviolations()
```

Fortran command syntax:

```
CALL MAPI_CNLPINFOSUM(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.91. CNTRANSACTSELBRANCHES

Use this function to display the transactions impact on the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNTRANSACTSELBRANCHES  NUMTRANS LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = cntransactselbranches(numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_CNTRANSELBRA(NUMTRANS, LISTTR, IERR)
```

Where:

Integer NUMTRANS	Is the number of transactions (input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of transactions is missing.
	IERR = 3	One transaction label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.92. CNTRANSACTSELFLOWGATES

Use this function to display the transactions impact on the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNTRANSACTSELFLOWGATES  NUMTRANS LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = cntransactselflowgates(numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_CNTRANSELFG(NUMTRANS, LISTTR, IERR)
```

Where:

Integer NUMTRANS	Is the number of transactions to be included in LISTTR (input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of transactions is missing.
	IERR = 3	One transaction label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.93. CNTRANSACTSELINTERFACES

Use this function to display the transactions impact on the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNTRANSACTSELINTERFACES  NUMTRANS LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = cntransactselinterfaces(numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_CNTRANSELINT(NUMTRANS, LISTTR, IERR)
```

Where:

Integer NUMTRANS	(input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of transactions is missing.
	IERR = 3	One transaction label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.94. CNVARSELBRANCHES

Use this function to display the variations report for the selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNVARSELBRANCHES
```

Python command syntax:

```
ierr = cnvarselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNVARSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.95. CNVARSELINTERFACES

Use this function to display the variations report for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNVARSELINTERFACES
```

Python command syntax:

```
ierr = cnvarselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNVARSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.96. CNVARSELVOLTAGEBUSES

Use this function to display the variations report for the selected voltage buses.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNVARSELVOLTAGEBUSES
```

Python command syntax:

```
ierr = cnvarselvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_CNVARSELVOLT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.97. CNVIOLSELBRANCHES

Use this function to generate a violations report for selected branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNVIOLSELBRANCHES
```

Python command syntax:

```
ierr = cnviolselbranches()
```

Fortran command syntax:

```
CALL MAPI_CNVIOLSELBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.98. CNVIOLSELFLOWGATES

Use this function to generate a violations report for the selected flowgates.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNVIOLSELFLOWGATES
```

Python command syntax:

```
ierr = cnviolselflowgates()
```

Fortran command syntax:

```
CALL MAPI_CNVIOLSELFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.99. CNVIOLSELINTERFACES

Use this function to generate a violations report for the selected interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNVIOLSELINTERFACES
```

Python command syntax:

```
ierr = cnviolselinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNVIOLSELINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.100. CNVIOLSELVOLTAGEBUSES

Use this function to generate a violations report for the selected buses with monitored voltages.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_CNVIOLSELVOLTAGEBUSES
```

Python command syntax:

```
ierr = cnviolselvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_CNVIOLSELVOLT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.101. COMPAREFCITC

Use this function to compare two FCITC reports.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_COMPAREFCITC  MNFILE CNFILE
```

Python command syntax:

```
ierr = comparefcitc(mnfile, cnfile)
```

Fortran command syntax:

```
CALL MAPI_COMPAREFCITC(MNFILE, CNFILE, IERR)
```

Where:

Character*260 MNFILE	(input; no default allowed).	
Character*260 CNFILE	(input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	File 1 - FCITC file missing, invalid name or no access
	IERR = 3	File 2 - FCITC file missing, invalid name or no access

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.102. CONSTRDEFS

Use this function to enter a set of default MUST Options for later use in Transfer Analysis calculations

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_CONSTRDEFS  OPTIONS(1)..OPTIONS(7) VALUES(1)..VALUES(3)
```

Python command syntax:

```
ierr = constrdefs(options, values)
```

Fortran command syntax:

```
CALL MAPI_CONSTRDEFS(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (7)	array of 7 elements specifying calculation options(input).	
	OPTIONS(1)	Base Case Rating (Select Rating 1-12) (1 by default)
	OPTIONS(1) > n	Rating ID
	OPTIONS(2)	Contingency Case Rating (Select Rating 1-12) (2 by default)
	OPTIONS(2) > n	Rating ID
	OPTIONS(3)	Base Case Load Flow (4 by default)
	OPTIONS(3) = 0	DC
	OPTIONS(3) = 1	AC Newton
	OPTIONS(3) = 2	AC Decoupled
	OPTIONS(3) = 3	EMS
	OPTIONS(3) = 4	Do not solve AC
	OPTIONS(3) = 5	PSS/E Presolved
	OPTIONS(4)	Convert branch ratings to estimated MW ratings (0 by default)
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
	OPTIONS(5)	Contingency ID Reporting (3 by default)
	OPTIONS(5) = 1	labels
	OPTIONS(5) = 2	events
	OPTIONS(5) = 3	both
	OPTIONS(6)	Maximum Number of Contingencies to process (900000 by default)
	OPTIONS(6) > n	Number of Contingencies
	OPTIONS(7)	Line Rating Unit for AC Applications (0 by default)
	OPTIONS(7) = 0	Current expressed as MVA
	OPTIONS(7) = 1	MVA

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real VALUES (3)	array of 3 elements specifying reporting parameters(input).	
	VALUES(1)	AC MW Mismatch Tolerance (10.0 by default)
	VALUES(2)	Base Case Percent of Rating (branches only) (100.0 by default)
	VALUES(3)	Contingency Case Percent of Rating (branches only) (100.0 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer option value is missing
	IERR = 3	Float option value is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.103. DCCONTBYCONT

Use this function to generate a DC contingency report by contingency options

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_DCCONTBYCONT SEL SELOPT1 SELOPT2 RPTTYPE OPTIONS(1)..OPTIONS(4)
 NUM(1)..NUM(2)
```

Python command syntax:

```
ierr = dccontbycont(sel, selopt1, selopt2, rpttype, options, num)
```

Fortran command syntax:

```
CALL MAPI_DCCONTBYCONT(SEL, SELOPT1, SELOPT2, RPTTYPE, OPTIONS, NUM, IERR)
```

Where:

Integer SEL	Indicate how to filter the contingencies for this report (input; no default allowed).	
	'1'	Report contingencies by Category
	'2'	Report contingencies by Range
	'3'	Report contingencies below severity index
	'4'	Report contingencies above severity index
Integer SELOPT1	Option used in conjunction with SEL when SEL=(1 or 2) to specify filter criteria (input; no default allowed).	
	SELOPT1 = n	If SEL = 2 (by Range) then n = lower contingency number in desired range. If SEL = 1 (by Category) then enter one of the following:
	SELOPT1 = 1	Show ALL contingencies
	SELOPT1 = 2	Show contingencies with Violations
	SELOPT1 = 3	Show contingencies without Violations
	SELOPT1 = 4	Show ACTIVE contingencies
	SELOPT1 = 5	Show EXCLUDED contingencies
	SELOPT1 = 6	Show DISABLED contingencies
	SELOPT1 = 7	Show SINGLE BRANCH contingencies
	SELOPT1 = 8	Show COMPLEX contingencies
	SELOPT1 = 9	Show contingencies with Islands
	SELOPT1 = 10	Show IMBALANCED contingencies
	SELOPT1 = 11	Show contingencies with Generation
	SELOPT1 = 12	Show contingencies with Load
Integer SELOPT2	Only used if SEL = 2 (by Range) (input).	
	SELOPT2 = n	Upper contingency number in desired range
Integer RPTTYPE	Type of contingency report (input; no default allowed).	
	'1'	Selected Contingencies with Violations

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'2'	Summary List of Selected Contingencies
	'3'	Details for Selected Contingencies
Integer OPTIONS (4)	array of 4 elements specifying report options(input).	
	OPTIONS(1)	Report loading above cutoff (0 by default)
	OPTIONS(1) = 0	No
	OPTIONS(1) = 1	Yes
	OPTIONS(2)	Include Contingent Events (0 by default)
	OPTIONS(2) = 0	No
	OPTIONS(2) = 1	Yes
	OPTIONS(3)	Include Islands Report (0 by default)
	OPTIONS(3) = 0	No
	OPTIONS(3) = 1	Yes
	OPTIONS(4)	Include Dispatch Data (0 by default)
	OPTIONS(4) = 0	No
	OPTIONS(4) = 1	Yes
Real NUM (2)	array of 2 elements specifying criteria to be used in conjunction with SEL and OPTIONS (input).	
	NUM(1)	Used with SEL=3 or 4 (by Severity Index) (0 by default)
	NUM(1) = x	x = Severity index for selected contingencies
	NUM(2)	Used with OPTIONS(1) = 1 (Report loading above cutoff) (0 by default)
	NUM(2) = y	y = Cutoff (%) for reporting loading
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.104. DCCONTBYMONEL

Use this function to generate a DC contingency report by monitored elements

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_DCCONTBYMONEL SEL SELOPT1 SELOPT2 MONELS RPTTYPE NUM(1)..NUM(2)
```

Python command syntax:

```
ierr = dccontbymonel(sel, selopt1, selopt2, monels, rpttype, num)
```

Fortran command syntax:

```
CALL MAPI_DCCONTBYMONEL(SEL, SELOPT1, SELOPT2, MONELS, RPTTYPE, NUM, IERR)
```

Where:

Integer SEL	Indicate how to filter the contingencies for this report (input; no default allowed).	
	'1'	Report contingencies by Category
	'2'	Report contingencies by Range
	'3'	Report contingencies below severity index
	'4'	Report contingencies above severity index
Integer SELOPT1	Option used in conjunction with SEL when SEL=(1 or 2) to specify filter criteria (input; no default allowed).	
	SELOPT1 = n	If SEL = 2 (by Range) then n = lower contingency number in desired range. If SEL = 1 (by Category) then enter one of the following:
	SELOPT1 = 1	Show ALL contingencies
	SELOPT1 = 2	Show contingencies with Violations
	SELOPT1 = 3	Show contingencies without Violations
	SELOPT1 = 4	Show ACTIVE contingencies
	SELOPT1 = 5	Show EXCLUDED contingencies
	SELOPT1 = 6	Show DISABLED contingencies
	SELOPT1 = 7	Show SINGLE BRANCH contingencies
	SELOPT1 = 8	Show COMPLEX contingencies
	SELOPT1 = 9	Show contingencies with Islands
	SELOPT1 = 10	Show IMBALANCED contingencies
	SELOPT1 = 11	Show contingencies with Generation
	SELOPT1 = 12	Show contingencies with Load
Integer SELOPT2	Only used if SEL = 2 (by Range) (input).	
	SELOPT2 = n	Upper contingency number in desired range
Integer MONELS	Monitored elements for report (input; no default allowed).	
	'1'	Branches
	'2'	Interfaces

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	'3'	Flowgates
Integer RPTTYPE	Type of contingency report (input; no default allowed).	
	'1'	List of Monitored Elements
	'2'	List of Violations above Cutoff
	'3'	Summary of Frequency of Violations
Real NUM (2)	array of 2 elements specifying criteria to be used in conjunction with SEL and RPTTYPE (input).	
	NUM(1)	Used with SEL=3 or 4 (by Severity Index) (0 by default)
	NUM(1) = x	x = Severity index for selected contingencies
	NUM(2)	Used with RPTTYPE = 2 (Violations above cut-off) (0 by default)
	NUM(2) = y	y = Cutoff (%) for reporting loading
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.105. DCSOLVE

Use this function to solve the DC load flow .

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_DCSOLVE
```

Python command syntax:

```
ierr = dcsolve()
```

Fortran command syntax:

```
CALL MAPI_DCSOLVE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Initialization error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.106. DCTRANSFERLIMITSTUDY

Use this function to execute a DC Study Transfer.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_DCTRANSFERLIMITSTUDY  DFLAG SYS1 SYS2 TRANSFER
```

Python command syntax:

```
ierr = dctransferlimitstudy(dflag, sys1, sys2, transfer)
```

Fortran command syntax:

```
CALL MAPI_DCTRANSFERLIMITSTUDY(DFLAG, SYS1, SYS2, TRANSFER, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem(s) missing
	IERR = 3	No transfer value
	IERR = 4	Subsystem(s) not valid
	IERR = 5	Sending subsystem already selected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.107. DETAILSELCONTINGENCIES

Use this function to display details of the selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_DETAILSELCONTINGENCIES
```

Python command syntax:

```
ierr = detailselcontingencies()
```

Fortran command syntax:

```
CALL MAPI_DETLSELCONTS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.108. DETAILSELCONTINGENCIESSP

Use this function to display details report on the selected contingencies using the Selected Parameters

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_DETAILSELCONTINGENCIESSP  CVIOL CEVENTS CISLAND CDISPATCH
```

Python command syntax:

```
ierr = detailselcontingenciessp(cviol, cevents, cisland, cdispatch)
```

Fortran command syntax:

```
CALL MAPI_DETLSELCONTSSP(CVIOL, CEVENTS, CISLAND, CDISPATCH, IERR)
```

Where:

Integer CVIOL	Report violation details (0-no report,1 - report) (input; no default allowed).	
Integer CEVENTS	Report details on contingency events (0-no report,1 - report) (input; no default allowed).	
Integer CISLAND	Report islands details (0-no report,1 - report) (input; no default allowed).	
Integer CDISPATCH	Report details on dispatch events (0-no report,1 - report) (input; no default allowed).	
integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Report violation details is missing.
	IERR = 3	Report contingency events details is missing.
	IERR = 4	Report islands details is missing.
	IERR = 5	Report dispatch event details is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.109. DIRECT

Use this function to capture the terminal output to a file. Also use this function to terminate capturing the terminal output. pss must module generally writes all reports to the terminal. This function is used to redirect the reports to the specified file.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_DIRECT  DFLAG MARG FILARG
```

Python command syntax:

```
ierr = direct(dflag, marg, filarg)
```

Fortran command syntax:

```
CALL MAPI_DIRECT(DFLAG, MARG, FILARG, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Character*1 MARG	Is a one character string denoting the destination of the terminal output. Acceptable entries are: (input; no default allowed).	
	MARG = T	send output to terminal (also used to terminate writing to the file)
	MARG = F	send output to the specified file
	MARG = X	append output to the specified file
Character*260 FILARG	Is a string identifying the full pathname of the file to which the terminal output will be written. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	FILARG missing, invalid file name, or no access
	IERR = 3	Invalid file specification
	IERR = 4	MARG invalid specification

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.110. EXCLUDESELECTEDCONTS

Use this function to exclude the selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_EXCLUDESELECTEDCONTS
```

Python command syntax:

```
ierr = excludeselectedconts()
```

Fortran command syntax:

```
CALL MAPI_EXCSELCONTS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.111. EXECUTEACCONT

Use this function to run the AC contingency analysis load flow.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_EXECUTEACCONT
```

Python command syntax:

```
ierr = executeaccont()
```

Fortran command syntax:

```
CALL MAPI_ACCEXE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Could not complete ACCONT Analysis

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.112. EXITACTIVITYACCONT

Use this function to terminate the ACCONT Activities. This function frees up the memory allocations that were required to perform the ACCONT analyses listed below. Analyses for which this function should be run upon completion: Execution (running) of the AC contingency analysis load flow; Reading or Saving the AC contingency analysis file (.acc); AC Constraints analyses by branches, interfaces, flowgates or voltages; AC Contingency analyses by contingencies

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_EXITACTIVITYACCONT
```

Python command syntax:

```
ierr = exitactivityaccont()
```

Fortran command syntax:

```
CALL MAPI_POSTACCONTANAL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	ACCONT Analysis has not been initialized

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.113. EXITACTIVITYMUST

Use this function to terminate the pss must module Activities. This function frees up the memory allocations that were required to perform the pss must module analyses listed below.
Analyses for which this function should be run upon completion:
- DC Constraints analyses by branches, interfaces or flowgates
- DC Contingency analyses by contingencies
- DC Transfer analyses
- AC Transfer limit analysis
- Transfer versus transfer sensitivities
- Generator sensitivites on study transfer
- Transaction impact analysis

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_EXITACTIVITYMUST  EXITC
```

Python command syntax:

```
ierr = exitactivitymust(exitc)
```

Fortran command syntax:

```
CALL MAPI_POSTMUSTANAL(EXITC, IERR)
```

Where:

Integer EXITC	Is Exit Code. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The Exit Code is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.114. FCITC2LINE

Use this function to generate a 2 LINE multiple fcitc report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_FCITC2LINE  MAXREP
```

Python command syntax:

```
ierr = fcitc2line(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITC2LIN(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined
	IERR = 3	Maximum Number of Violations to Report is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.115. FCITCAC

Use this function to generate an AC multiple fcitc report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_FCITCAC  MAXREP
```

Python command syntax:

```
ierr = fcitcac(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITCAC(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Maximum Number of Violations to Report is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.116. FCITCACMULT

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_FCITCACMULT  NVIOL NSTUTRA STUTRALIST
```

Python command syntax:

```
ierr = fcitcacmult(nviol,nstutra,stutralist)
```

Fortran command syntax:

```
CALL MAPI_FCITCACMULT(NVIOL, NSTUTRA, STUTRALIST, IERR)
```

Where:

Integer NVIOL	(input; no default allowed).
Integer NSTUTRA	(input; no default allowed).
Character*32 STUTRALIST (3*NS-TUTRA)	(input; no default allowed).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.117. FCITCACSINGLE

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_FCITCACSINGLE  MAXREP
```

Python command syntax:

```
ierr = fcitcacmultiple(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITCACSINGLE(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.118. FCITCBASE

Use this function to generate a BASE multiple fcitc report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_FCITCBASE  MAXREP
```

Python command syntax:

```
ierr = fcitcbase(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITCBASE(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined
	IERR = 3	Maximum Number of Violations to Report is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.119. FCITCBASEMULT

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MAPI_FCITCBASEMULT   MAXREP NSTUTRA STUTRALIST
```

Python command syntax:

```
ierr = fcitcbasemult(maxrep,nstutra,stutralist)
```

Fortran command syntax:

```
CALL MAPI_FCITCBASEMULT(MAXREP, NSTUTRA, STUTRALIST, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).
Integer NSTUTRA	(input; no default allowed).
Character*32 STUTRALIST (3*NSTUTRA)	(input; no default allowed).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.120. FCITCBASESINGLE

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_FCITCBASESINGLE    MAXREP
```

Python command syntax:

```
ierr = fcitcbasesingle(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITCBASESINGLE(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined
	IERR = 3	Maximum Number of Violations to Report is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.121. FCITCMULT

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_FCITCMULT  NVIOL NSTUTRA STUTRALIST
```

Python command syntax:

```
ierr = fcitcmult(nviol,nstutra,stutralist)
```

Fortran command syntax:

```
CALL MAPI_FCITCMULT(NVIOL, NSTUTRA, STUTRALIST, IERR)
```

Where:

Integer NVIOL	(input).
Integer NSTUTRA	(input).
Character*32 STUTRALIST (3*NSTUTRA)	(input; no default allowed).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.122. FCITCSINGLE

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_FCITCSINGLE    MAXREP
```

Python command syntax:

```
ierr = fcitcsingle(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITCSINGLE(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.123. FCITCSUBS

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_FCITCSUBS  SUBSFROM SUBSTO MWLEV
```

Python command syntax:

```
ierr = fcitcsubs(subsfrom, substo,mwlev)
```

Fortran command syntax:

```
CALL MAPI_FCITCSUBS(SUBSFROM, SUBSTO, MWLEV, IERR)
```

Where:

Character*32 SUBSFROM	Is a string identifying the name of the subsystem From list (input; no default allowed).	
Character*32 SUBSTO	Is a string identifying the name of the subsystem To list (input; no default allowed).	
Real MWLEV	The MW transfer level (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem not found
	IERR = 3	The subsystem label is missing.
	IERR = 4	The MW transfer level is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.124. FCITCTLTG

Use this function to generate a TLTG multiple fcitc report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_FCITCTLTG  MAXREP
```

Python command syntax:

```
ierr = fcitctltg(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITCTLTG(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined
	IERR = 3	The Maximum Number of Violations to Report is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.125. FCITCWIDE

Use this function to generate a WIDE multiple fcitc report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_FCITCWIDE  MAXREP
```

Python command syntax:

```
ierr = fcitcwide(maxrep)
```

Fortran command syntax:

```
CALL MAPI_FCITCWIDE(MAXREP, IERR)
```

Where:

Integer MAXREP	Maximum Number of Violations to Report (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined
	IERR = 3	The Maximum Number of Violations to Report is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.126. FLOWGATESENSITIVITY

Use this function to generate the monitored elements sensitivities for specified flowgates

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_FLOWGATESENSITIVITY NUMMONEL LISTML OPTIONS(1)..OPTIONS(4) REFSYS
 IMPACTCUTOFF EXPSYS IMPSYS NUMTRANS LISTTRANS
```

Python command syntax:

```
ierr = flowgatesensitivity(nummonel, listml, options, refsys,
 impactcutoff, expsys, impsys, numtrans, listtrans)
```

Fortran command syntax:

```
CALL MAPI_FGSENS(NUMMONEL, LISTML, OPTIONS, REFSYS, IMPACTCUTOFF, EXPSYS,
 IMPSYS, NUMTRANS, LISTTRANS, IERR)
```

Where:

Integer NUMMONEL	How many monitored elements to consider. (input; no default allowed).	
Character*5 LISTML (NUMMONEL)	Is a set of strings forming a list of flowgate IDs. There must be as many flowgates as indicated by NUMMONEL. (input; no default allowed).	
Integer OPTIONS (4)	array of 4 elements specifying calculation options(input).	
	OPTIONS(1)	Sensitivity Report to: (1 by default)
	OPTIONS(1) = 1	Reference Subsystem
	OPTIONS(1) = 2	Factors for Impact Cutoff
	OPTIONS(1) = 3	Subsystem Participating Buses
	OPTIONS(1) = 4	Transactions
	OPTIONS(2)	Contingency for impact/sensitivity reports (0 by default)
	OPTIONS(2) = 0	Base Case
	OPTIONS(2) = x	x = Contingency number
	OPTIONS(3)	Reference subsystem config for sensitivity analysis (1 by default)
	OPTIONS(3) = 1	Importing
	OPTIONS(3) = 2	Exporting
	OPTIONS(4)	Transactions to consider (1 by default)
	OPTIONS(4) = 1	All possible transactions
	OPTIONS(4) = 2	Only selected transactions
Character*32 REFSYS	Is a string identifying the Reference Subsystem Name - to be used with Report 1 of OPTION(1). Enter -1 for Swing Bus. (input; blank by default).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real IMPACTCUTOFF	Impact CutOff Value - to be used with Report 2 of OPTION(1) (input; 0 by default).
Character*32 EXPSYS	Is a string identifying the Exporting Subsystem Name - to be used with Report 3 of OPTION(1) (input; blank by default).
Character*32 IMPSYS	Is a string identifying the Importing Subsystem Name - to be used with Report 3 of OPTION(1) (input; blank by default).
Integer NUMTRANS	How many transactions in LISTTRANS - to be used with Report 4 of OPTION(1). (input; no default allowed).
Character*32 LISTTRANS (2*NUMTRANS)	Is a set of strings forming a list of subsystem names representing transactions with each transaction being made up of a FROM subsystem and a TO subsystem. (input; no default allowed).
Integer IERR	error code (output).
	IERR = 0 No error
	IERR = 1 Prerequisite requirements for function not met
	IERR = 2 Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.127. FORGETSTUDYTRANSFER

Use this function to remove the current Study Transfer from memory.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_FORGETSTUDYTRANSFER
```

Python command syntax:

```
ierr = forgetstudytransfer()
```

Fortran command syntax:

```
CALL MAPI_FORGETST(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.128. GENDETAILS

Use this function to list the details for a specific subsystem.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_GENDETAILS  SUBS MWLEV
```

Python command syntax:

```
ierr = gendetails(subs, mwlev)
```

Fortran command syntax:

```
CALL MAPI_GENDET(SUBS, MWLEV, IERR)
```

Where:

Character*32 SUBS	Is a string identifying the name of the subsystem to list (input; no default allowed).	
Real MWLEV	The MW transfer level (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem not found
	IERR = 3	The subsystem label is missing.
	IERR = 4	The MW transfer level is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.129. GENERATIONSENSITIVITY

Use this function to calculate the generator sensitivites for a given study transfer

This API routine was first introduced in release 35.4.0.

Batch command syntax:

```
BAT_GENERATIONSENSITIVITY SYS1 SYS2 TRANSFER OPTIONS(1)..OPTIONS(4)
 CUTOFFIND CUTOFFST
```

Python command syntax:

```
ierr = generationsensitivity(sys1, sys2, transfer, options, cutoffind,
 cutoffst)
```

Fortran command syntax:

```
CALL MAPI_SUMGENSENS(SYS1, SYS2, TRANSFER, OPTIONS, CUTOFFIND, CUTOFFST,
 IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer OPTIONS (4)	array of 4 elements specifying calculation options(input).	
	OPTIONS(1)	Type of Report: (1 by default)
	OPTIONS(1) = 1	Summary Report
	OPTIONS(1) = 2	Detailed Report
	OPTIONS(1) = 3	Both summary and detailed reports
	OPTIONS(2)	Number of worst scenarios to report (0 by default)
	OPTIONS(2) = x	How many
	OPTIONS(3)	Transfer to use for Sensitivity Analysis: (1 by default)
	OPTIONS(3) = 1	Transfer TO
	OPTIONS(3) = 2	Transfer FROM
	OPTIONS(4)	Report participating generators NOT adjusted by worst dispatch (1 by default)
	OPTIONS(4) = 1	No
	OPTIONS(4) = 2	Yes
Real CUTOFFIND	Minimum factor cutoff for individual (input; no default allowed).	
Real CUTOFFST	Minimum factor cutoff for study transfer (input; no default allowed).	
Integer IERR	error code (output).	

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0	No error
IERR = 1	Prerequisite requirements for function not met
IERR = 2	Study Transfer not defined
IERR = 3	Max Power Transfer outside of limits
IERR = 4	Illegal Transfer Step
IERR = 5	No Subsystems Selected
IERR = 6	Specified Subsystem not found
IERR = 7	The maximum power transfer is missing.
IERR = 8	The increment of power transfer is missing.
IERR = 9	The number of subsystems is missing.
IERR = 10	One label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.130. GENSENDETAIL

Use this function to generate a detailed report for generator sensitivites on the current study transfer.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_GENSENDETAIL
```

Python command syntax:

```
ierr = gensendetail()
```

Fortran command syntax:

```
CALL MAPI_GENSENDET(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer Not Defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.131. GENSENSUMMARY

Use this function to generate a summary table for generator sensitivites on the current study transfer.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_GENSENSUMMARY
```

Python command syntax:

```
ierr = gensensummary()
```

Fortran command syntax:

```
CALL MAPI_GENSENSUM(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer Not Defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.132. ILFDEFAULTS

Use this function to enter a set of default Initial Line Flow options for later use in an AC load flow solution

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_ILFDEFAULTS  OPTSI(1)..OPTS(10) VALUES(1)..VALUES(3)
```

Python command syntax:

```
ierr = ilfdefaults(options, values)
```

Fortran command syntax:

```
CALL MAPI_ILFDEFAULTS(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (10)	array of 10 elements specifying calculation options(input).	
	OPTIONS(1)	Base Case Load Flow (4 by default)
	OPTIONS(1) = 0	DC
	OPTIONS(1) = 1	AC Newton
	OPTIONS(1) = 2	AC Decoupled
	OPTIONS(1) = 3	EMS
	OPTIONS(1) = 4	Do Not Solve
	OPTIONS(2)	Transformer Tap (0 by default)
	OPTIONS(2) = 0	disabled
	OPTIONS(2) = 1	enable stepping, No Direct
	OPTIONS(3)	Phase Shifter (0 by default)
	OPTIONS(3) = 0	disabled
	OPTIONS(3) = 1	enabled
	OPTIONS(4)	Switchable Shunts (1 by default)
	OPTIONS(4) = 0	disabled
	OPTIONS(4) = 1	enabled
	OPTIONS(5)	Area Interchange (0 by default)
	OPTIONS(5) = 0	disabled
	OPTIONS(5) = 1	Tie lines only
	OPTIONS(5) = 2	Tie lines and loads
	OPTIONS(6)	DC Tap (1 by default)
	OPTIONS(6) = 0	Locked
	OPTIONS(6) = 1	On
	OPTIONS(7)	Iter. to begin Var limiting (0 by default)
	OPTIONS(7) = 0	Hot Start: apply var limits immediately
	OPTIONS(7) = 99	Automatic:
	OPTIONS(8)	Load Flow Iterations Limit (20 by default)
	OPTIONS(8) > n	Terminate calculation after n iterations

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(9)	Use Flat Start for Load Flow Solution (0 by default)
	OPTIONS(9) = 0	hot
	OPTIONS(9) = 1	flat
	OPTIONS(10)	Iteration to freeze adjustments (15 by default)
	OPTIONS(10) > n	Freeze adjustments after n iterations
Real VALUES (3)	array of 3 elements specifying reporting parameters(input).	
	VALUES(1)	Real and Reactive Power Tolerance (0.1 by default)
	VALUES(2)	Reactive Adjustments DeAcceleration Factor (1.0 by default)
	VALUES(3)	Low Voltage break point (pu) for load scaling (0.7 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer option value is missing
	IERR = 3	Float option value is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.133. IMPACTINTRFACEPARTFACT

Use this function to generate an impact study for the systems interface participation factors (IPF).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_IMPACTINTRFACEPARTFACT  FROBJ TOOBJ NUMFROM LISTFR(1)..LISTFR(NUMFROM)
 NUMTO LISTTO(1)..LISTTO(NUMTO)
```

Python command syntax:

```
ierr = impactintrfacepartfact(frobj, toobj, numfrom, listfr, numto,
 listto)
```

Fortran command syntax:

```
CALL MAPI_IMPINTPF(FROBJ, TOOBJ, NUMFROM, LISTFR, NUMTO, LISTTO, IERR)
```

Where:

Character*12 FROBJ	Indicates the impacted system type. Valid entries can be: (input; no default allowed).	
	FROBJ = AREA:	Impacted systems listed by area
	FROBJ = ZONE:	Impacted systems listed by zone
	FROBJ = SYST:	Impacted systems listed by subsystem
Character*12 TOOBJ	Indicates the neighboring system type. Valid entries can be: (input; no default allowed).	
	TOOBJ = AREA:	Neighboring systems listed by area
	TOOBJ = ZONE:	Neighboring systems listed by zone
	TOOBJ = SYST:	Neighboring systems listed by subsystem
Integer NUMFROM	Is the number of impacted systems to be used in the impact study (input; no default allowed).	
Character*12 LISTFR	Is a set of strings forming a list of impacted systems. There must be as many impacted systems as indicated by NUMFROM. (input; no default allowed).	
Integer NUMTO	Is the number of neighboring systems to be used in the impact study (input; no default allowed).	
Character*12 LISTTO	Is a set of strings forming a list of neighboring systems. There must be as many neighboring systems as indicated by NUMTO. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	From Category must be AREAS, ZONES or SYSTEMS

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 3	To Category must be AREAS, ZONES or SYSTEMS
IERR = 4	No List of Systems Provided for Selected Category
IERR = 5	No List of Opposing Systems Provided for Selected Category
IERR = 6	The number of impacted systems is missing.
IERR = 7	The number of Opposing systems is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.134. IMPACTSETBASECASE

Use this function to generate an impact study on the base case.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_IMPACTSETBASECASE
```

Python command syntax:

```
ierr = impactsetbasecase()
```

Fortran command syntax:

```
CALL MAPI_IMPSETBASECASE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.135. IMPACTSETCONTINGENCY

Use this function to generate an impact study on the specified contingency.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_IMPACTSETCONTINGENCY  CONTNUM
```

Python command syntax:

```
ierr = impactsetcontingency(contnum)
```

Fortran command syntax:

```
CALL MAPI_IMPSETCONTCASE(CONTNUM, IERR)
```

Where:

Integer CONTNUM	Is the number of contingency to be used in the impact study (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Contingency not found
	IERR = 3	The contingency number is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.136. IMPACTTIESPARTFACT

Use this function to generate an impact study providing a detailed report for all tie branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_IMPACTTIESPARTFACT  FROBJ TOOBJ NUMFROM LISTFR(1)..LISTFR(NUMFROM)
 NUMTO LISTTO(1)..LISTTO(NUMTO)
```

Python command syntax:

```
ierr = impacttiespartfact(frobj, toobj, numfrom, listfr, numto, listto)
```

Fortran command syntax:

```
CALL MAPI_IMPTIEPF(FROBJ, TOOBJ, NUMFROM, LISTFR, NUMTO, LISTTO, IERR)
```

Where:

Character*12 FROBJ	Indicates the impacted system type. Valid entries can be: (input; no default allowed).	
	FROBJ = AREA:	Impacted systems listed by area
	FROBJ = ZONE:	Impacted systems listed by zone
	FROBJ = SYST:	Impacted systems listed by subsystem
Character*12 TOOBJ	Indicates the neighboring system type. Valid entries can be: (input; no default allowed).	
	TOOBJ = AREA:	Neighboring systems listed by area
	TOOBJ = ZONE:	Neighboring systems listed by zone
	TOOBJ = SYST:	Neighboring systems listed by subsystem
Integer NUMFROM	Is the number of impacted systems to be used in the impact study (input; no default allowed).	
Character*12 LISTFR	Is a set of strings forming a list of impacted systems. There must be as many impacted systems as indicated by NUMFROM. (input; no default allowed).	
Integer NUMTO	Is the number of neighboring systems to be used in the impact study (input; no default allowed).	
Character*12 LISTTO	Is a set of strings forming a list of neighboring systems. There must be as many neighboring systems as indicated by NUMTO. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	From Category must be AREAS, ZONES or SYSTEMS
	IERR = 3	To Category must be AREAS, ZONES or SYSTEMS

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 4	No List of Systems Provided for Selected Category
IERR = 5	No List of Opposing Systems Provided for Selected Category
IERR = 6	The number of impacted systems is missing.
IERR = 7	The number of Opposing systems is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.137. IMPACTTOTALPARTFACT

Use this function to generate an impact study for the system totals participation factor (TPF).

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_IMPACTTOTALPARTFACT  FROBJ TOOBJ NUMMONEL LISTOP(1)..LISTOP(NUMMONEL)
```

Python command syntax:

```
ierr = impacttotalpartfact(frobj, toobj, nummonel, listop)
```

Fortran command syntax:

```
CALL MAPI_IMPTOTPF(FROBJ, TOOBJ, NUMMONEL, LISTOP, IERR)
```

Where:

Character*12 FROBJ	Indicates the impacted system type. Valid entries can be: (input; no default allowed).	
	FROBJ = AREA:	Impacted systems listed by area
	FROBJ = ZONE:	Impacted systems listed by zone
	FROBJ = SYST:	Impacted systems listed by subsystem
Character*12 TOOBJ	Indicates the neighboring system type. Valid entries can be: (input; no default allowed).	
	TOOBJ = AREA:	Neighboring systems listed by area
	TOOBJ = ZONE:	Neighboring systems listed by zone
	TOOBJ = SYST:	Neighboring systems listed by subsystem
Integer NUMMONEL	(input; no default allowed).	
Character*12 LISTOP	Is a set of strings forming a list of impacted systems. There must be as many impacted systems as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	From Category must be AREAS, ZONES or SYSTEMS
	IERR = 3	To Category must be AREAS, ZONES or SYSTEMS
	IERR = 4	List of systems not found or not proper for Selected Category
	IERR = 5	The number of impacted systems is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.138. INCLUDESELECTEDCONTS

Use this function to include the selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_INCLUDESELECTEDCONTS
```

Python command syntax:

```
ierr = includeselectedconts()
```

Fortran command syntax:

```
CALL MAPI_INCSELCONTS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.139. INTERFACESENSITIVITY

Use this function to generate the monitored elements sensitivities for specified interfaces

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_INTERFACESENSITIVITY NUMMONEL LISTML OPTIONS(1)..OPTIONS(4) REFSYS
 IMPACTCUTOFF EXPSYS IMPSYS NUMTRANS LISTTRANS
```

Python command syntax:

```
ierr = interfacesensitivity(nummonel, listml, options, refsys,
 impactcutoff, expsys, impsys, numtrans, listtrans)
```

Fortran command syntax:

```
CALL MAPI_INSENS(NUMMONEL, LISTML, OPTIONS, REFSYS, IMPACTCUTOFF, EXPSYS,
 IMPSYS, NUMTRANS, LISTTRANS, IERR)
```

Where:

Integer NUMMONEL	How many monitored elements to consider. (input; no default allowed).
Character*32 LISTML (NUMMONEL)	Is a set of strings forming a list of interface numbers. There must be as many interfaces as indicated by NUMMONEL. (input; no default allowed).
Integer OPTIONS (4)	array of 4 elements specifying calculation options(input).

OPTIONS(1)	Sensitivity Report to: (1 by default)
OPTIONS(1) = 1	Reference Subsystem
OPTIONS(1) = 2	Factors for Impact Cutoff
OPTIONS(1) = 3	Subsystem Participating Buses
OPTIONS(1) = 4	Transactions
OPTIONS(2)	Contingency for impact/sensitivity reports (0 by default)
OPTIONS(2) = 0	Base Case
OPTIONS(2) = x	x = Contingency number
OPTIONS(3)	Reference subsystem config for sensitivity analysis (1 by default)
OPTIONS(3) = 1	Importing
OPTIONS(3) = 2	Exporting
OPTIONS(4)	Transactions to consider (1 by default)
OPTIONS(4) = 1	All possible transactions
OPTIONS(4) = 2	Only selected transactions

Character*32 REFSYS	Is a string identifying the Reference Subsystem Name - to be used with Report 1 of OPTION(1). Enter -1 for Swing Bus. (input; blank by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real IMPACTCUTOFF	Impact CutOff Value - to be used with Report 2 of OPTION(1) (input; 0 by default).
Character*32 EXPSYS	Is a string identifying the Exporting Subsystem Name - to be used with Report 3 of OPTION(1) (input; blank by default).
Character*32 IMPSYS	Is a string identifying the Importing Subsystem Name - to be used with Report 3 of OPTION(1) (input; blank by default).
Integer NUMTRANS	How many transactions in LISTTRANS - to be used with Report 4 of OPTION(1). (input; no default allowed).
Character*32 LISTTRANS (2*NUMTRANS)	Is a set of strings forming a list of subsystem names representing transactions with each transaction being made up of a FROM subsystem and a TO subsystem. (input; no default allowed).
Integer IERR	error code (output).
	IERR = 0 — No error
	IERR = 1 — Prerequisite requirements for function not met
	IERR = 2 — Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.140. LISTACCONTDETAILS

Use this function to provide an AC contingency details report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTACCONTDETAILS
```

Python command syntax:

```
ierr = listaccontdetails()
```

Fortran command syntax:

```
CALL MAPI_ACCONTDET(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.141. LISTACCONTSUMMARY

Use this function to provide an AC contingency summary report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTACCONTSUMMARY
```

Python command syntax:

```
ierr = listaccontsummary()
```

Fortran command syntax:

```
CALL MAPI_ACCONTSUM(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.142. LISTAREAS

Use this function for a quick list of all areas.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTAREAS
```

Python command syntax:

```
ierr = listareas()
```

Fortran command syntax:

```
CALL MAPI_AREAS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Error in obtaining list

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.143. LISTBUSES

Use this function to a report of the list of buses.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTBUSES
```

Python command syntax:

```
ierr = listbuses()
```

Fortran command syntax:

```
CALL MAPI_BUSES(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.144. LISTEXCLUDES

Use this function to list the constraints or contingencies to be excluded.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTEXCLUDES
```

Python command syntax:

```
ierr = listexcludes()
```

Fortran command syntax:

```
CALL MAPI_LISTEXCLUDE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.145. LISTGENERATORS

Use this function to a report of the list of generators.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTGENERATORS
```

Python command syntax:

```
ierr = listgenerators()
```

Fortran command syntax:

```
CALL MAPI_GENERS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.146. LISTPHASESHIFTERS

Use this function to a report of the list of phase shifters.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTPHASESHIFTERS
```

Python command syntax:

```
ierr = listphaseshifters()
```

Fortran command syntax:

```
CALL MAPI_PHASESHIFT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.147. LISTSELCONTINGENCIES

Use this function to list the selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTSELCONTINGENCIES
```

Python command syntax:

```
ierr = listselcontingencies()
```

Fortran command syntax:

```
CALL MAPI_LISTSELCONTS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.148. LISTZONES

Use this function for a quick list of all zones.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LISTZONES
```

Python command syntax:

```
ierr = listzones()
```

Fortran command syntax:

```
CALL MAPI_ZONES(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.149. LIST_BFLOW

Use this function for a quick report of base case flows.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_BFLOW
```

Python command syntax:

```
ierr = list_bflow()
```

Fortran command syntax:

```
CALL MAPI_BFLOW(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.150. LIST_BIMPACT

Use this function for a quick report of base case impact on selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_BIMPACT
```

Python command syntax:

```
ierr = list_bimpact()
```

Fortran command syntax:

```
CALL MAPI_BIMPACT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.151. LIST_CFLOW

Use this function for a quick report of contingency flows for selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_CFLOW
```

Python command syntax:

```
ierr = list_cflow()
```

Fortran command syntax:

```
CALL MAPI_CFLOW(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.152. LIST_CIMPACT

Use this function for a quick report of study transfer impact on selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_CIMPACT
```

Python command syntax:

```
ierr = list_cimpact()
```

Fortran command syntax:

```
CALL MAPI_CIMPACT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.153. LIST_CONTINS

Use this function for a quick list of all defined contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_CONTINS
```

Python command syntax:

```
ierr = list_contins()
```

Fortran command syntax:

```
CALL MAPI_LISTCONTINS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.154. LIST_CONTINS_WISL

Use this function for a quick list of all contingencies with island.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_CONTINS_WISL
```

Python command syntax:

```
ierr = list_contins_wisl()
```

Fortran command syntax:

```
CALL MAPI_LISTCONTINS_WISL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.155. LIST_LODF

Use this function for a quick report of LODF factors.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_LODF
```

Python command syntax:

```
ierr = list_lodf()
```

Fortran command syntax:

```
CALL MAPI_LODF(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.156. LIST_LODF_WISL

Use this function for a quick report of LODF factors with island.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_LODF_WISL
```

Python command syntax:

```
ierr = list_lodf_wisl()
```

Fortran command syntax:

```
CALL MAPI_LODF_WISL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.157. LIST_MONELS

Use this function for a quick list of all monitored elements.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_MONELS
```

Python command syntax:

```
ierr = list_monels()
```

Fortran command syntax:

```
CALL MAPI_LISTMONELS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.158. LIST_PTDF

Use this function for a quick report of PTDF/TDF reference bus factors on monitored elements.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_PTDF
```

Python command syntax:

```
ierr = list_ptdf()
```

Fortran command syntax:

```
CALL MAPI_PTDF(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.159. LIST_PTDFC

Use this function for a quick report of ALL reference bus factors on contingency branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_PTDFC
```

Python command syntax:

```
ierr = list_ptdfc()
```

Fortran command syntax:

```
CALL MAPI_PTDFC(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.160. LIST_SYSTEMS

Use this function for a quick list of all subsystems.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_LIST_SYSTEMS
```

Python command syntax:

```
ierr = list_systems()
```

Fortran command syntax:

```
CALL MAPI_LISTSYSTEMS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.161. MDETAILBRANCHES

Use this function to generate a detailed report by monitored branches.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MDETAILBRANCHES
```

Python command syntax:

```
ierr = mdetailbranches()
```

Fortran command syntax:

```
CALL MAPI_CNMDETAILBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	No branches selected

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.162. MDETAILINTERFACES

Use this function to generate a detailed report by monitored interfaces.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MDETAILINTERFACES
```

Python command syntax:

```
ierr = mdetailinterfaces()
```

Fortran command syntax:

```
CALL MAPI_CNMDETAILINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.163. MDETAILVOLTAGEBUSES

Use this function to generate a detailed report by monitored voltage buses.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MDETAILVOLTAGEBUSES
```

Python command syntax:

```
ierr = mdetailvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_CNMDETAILVOLT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.164. MODIFYDISPATCH

Use this function to modify the dispatch study transaction.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MODIFYDISPATCH  TRPOW
```

Python command syntax:

```
ierr = modifydispatch(trpow)
```

Fortran command syntax:

```
CALL MAPI_TRANDISP(TRPOW, IERR)
```

Where:

Real TRPOW	(input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer is not defined
	IERR = 3	Transfer Level out of range (0 - 999,999)
	IERR = 4	The transfer level is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.165. MONELIMPACT

Use this function to generate a monitored elements impact analysis report

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_MONELIMPACT SYS1 SYS2 TRANSFER MONELS OPTIONS(1)..OPTIONS(6)
 LOADCUTOFF IMPACTCUTOFF
```

Python command syntax:

```
ierr = monelimpact(sys1, sys2, transfer, monels, options, loadcutoff,
 impactcutoff)
```

Fortran command syntax:

```
CALL MAPI_MONELIMPACT(SYS1, SYS2, TRANSFER, MONELS, OPTIONS, LOADCUTOFF,
 IMPACTCUTOFF, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer MONELS	Monitored elements for report (input; no default allowed).	
	'1'	Branches
	'2'	Interfaces
	'3'	Flowgates
Integer OPTIONS (6)	array of 6 elements specifying calculation options(input).	
	OPTIONS(1)	OTDF Sensitivity: (0 by default)
	OPTIONS(1) = 0	no
	OPTIONS(1) = 1	yes
	OPTIONS(2)	Report order: (1 by default)
	OPTIONS(2) = 1	bus order
	OPTIONS(2) = 2	by loading
	OPTIONS(2) = 3	by impact reponse
	OPTIONS(3)	Apply Loading CutOff: (0 by default)
	OPTIONS(3) = 0	no
	OPTIONS(3) = 1	yes
	OPTIONS(4)	Apply Impact CutOff: (0 by default)
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
	OPTIONS(5)	Report Transfer Limits (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
	OPTIONS(6)	Contingency for impact/sensitivity reports (0 by default)
	OPTIONS(6) = 0	Base Case
	OPTIONS(6) = x	x = Contingency number
Real LOADCUTOFF	Loading Cutoff for reporting violations; Ignored if OPTION(3) = 0 (input; no default allowed).	
Real IMPACTCUTOFF	Impact CutOff Value; Ignored if OPTION(4) = 0 (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Entry is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.166. MONSENSALLSUBBRANCHES

Use this function to report on the subsystem branch factors for the selected branches.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSALLSUBBRA   SUBS
```

Python command syntax:

```
ierr = monsensallsubbranches(subs)
```

Fortran command syntax:

```
CALL MAPI_MONSENSALLSUBBRA(SUBS, IERR)
```

Where:

Character*32 SUBS	Is a string identifying the Subsystem Name for which to report the branch factors. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	The Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.167. MONSENSALLSUBFLOWGATES

Use this function to report on the subsystem branch factors for the selected flowgates.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSALLSUBFG  SUBS
```

Python command syntax:

```
ierr = monsensallsubflowgates(subs)
```

Fortran command syntax:

```
CALL MAPI_MONSENSALLSUBFG(SUBS, IERR)
```

Where:

Character*32 SUBS	Is a string identifying the Subsystem Name for which to report the flowgate factors. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	The Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.168. MONSENSALLSUBINTERFACES

Use this function to report on the subsystem branch factors for the selected interfaces.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSALLSUBINTER  SUBS
```

Python command syntax:

```
ierr = monsensallsubinterfaces(subs)
```

Fortran command syntax:

```
CALL MAPI_MONSENSALLSUBINT(SUBS, IERR)
```

Where:

Character*32 SUBS	Is a string identifying the Subsystem Name for which to report the interface factors. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	The Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.169. MONSENSALLTRANBRANCHES

Use this function to display the transactions impact on ALL branches.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSALLTRANBRANCHES
```

Python command syntax:

```
ierr = monsensalltranbranches()
```

Fortran command syntax:

```
CALL MAPI_MONSENSALLTRANBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.170. MONSENSALLTRANFLOWGATES

Use this function to display the transactions impact on ALL flowgates.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSALLTRANFLOWGATES
```

Python command syntax:

```
ierr = monsensalltranflowgates()
```

Fortran command syntax:

```
CALL MAPI_MONSENSALLTRANFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.171. MONSENSALLTRANINTERFACES

Use this function to display the transactions impact on ALL interfaces.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSALLTRANINTERFACES
```

Python command syntax:

```
ierr = monsensalltraninterfaces()
```

Fortran command syntax:

```
CALL MAPI_MONSENSALLTRANINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.172. MONSENSSINGLESUBBRANCHES

Use this function to display the subsystem sensitivities for the first selected branch.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSSINGLESUBBRA
```

Python command syntax:

```
ierr = monsenssinglesubbranches()
```

Fortran command syntax:

```
CALL MAPI_MONSENSSINGLESUBBRA(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.173. MONSENSSINGLESUBFLOWGATES

Use this function to display the subsystem sensitivities for the first selected flowgate.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSSINGLESUBFG
```

Python command syntax:

```
ierr = monsenssinglesubflowgates()
```

Fortran command syntax:

```
CALL MAPI_MONSENSSINGLESUBFG(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.174. MONSENSSINGLESUBINTERFACES

Use this function to display the subsystem sensitivities for the first selected interface.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSSINGLESUBINTER
```

Python command syntax:

```
ierr = monsenssinglesubinterfaces()
```

Fortran command syntax:

```
CALL MAPI_MONSENSSINGLESUBINT(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.175. MONSENSSUBSPARTBRANCHES

Use this function to generate a sensitivity analysis for study transfer on the selected branches.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSSUBSPARTBRANCHES  SUBSFR SUBSIN
```

Python command syntax:

```
ierr = monsenssubspartbranches(subsfr, subsin)
```

Fortran command syntax:

```
CALL MAPI_MONSENSSUBSPARTBRA(SUBSFR, SUBSIN, IERR)
```

Where:

Character*32 SUBSFR	Is a string identifying FROM Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SUBSIN	Is a string identifying TO Subsystem Name to be used in the study transfer. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	FROM SubSystem same as TO SubSystem
	IERR = 4	The FROM Subsystem Name is missing.
	IERR = 5	The TO Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.176. MONSENSSUBSPARTFLOWGATES

Use this function to generate a sensitivity analysis for study transfer on the selected flowgates.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSSUBSPARTFLOWGATES  SUBSFR SUBSIN
```

Python command syntax:

```
ierr = monsenssubspartflowgates(subsfr, subsin)
```

Fortran command syntax:

```
CALL MAPI_MONSENSSUBSPARTFG(SUBSFR, SUBSIN, IERR)
```

Where:

Character*32 SUBSFR	Is a string identifying FROM Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SUBSIN	Is a string identifying TO Subsystem Name to be used in the study transfer. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	FROM SubSystem same as TO SubSystem
	IERR = 4	The FROM Subsystem Name is missing.
	IERR = 5	The TO Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.177. MONSENSSUBSPARTINTERFACES

Use this function to generate a sensitivity analysis for study transfer on the selected interfaces.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSSUBSPARTINTERFACES  SUBSFR SUBSIN
```

Python command syntax:

```
ierr = monsenssubspartinterfaces(subsfr, subsin)
```

Fortran command syntax:

```
CALL MAPI_MONSENSSUBSPARTINT(SUBSFR, SUBSIN, IERR)
```

Where:

Character*32 SUBSFR	Is a string identifying FROM Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SUBSIN	Is a string identifying TO Subsystem Name to be used in the study transfer. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid SubSystem Selected
	IERR = 3	FROM SubSystem same as TO SubSystem
	IERR = 4	The FROM Subsystem Name is missing.
	IERR = 5	The TO Subsystem Name is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.178. MONSENSTRANBRANCHES

Use this function to display the transactions impact on the selected branches.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSTRANBRANCHES  NUMTRANS LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = monsenstranbranches(numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_MONSENSTRANBRA(NUMTRANS, LISTTR, IERR)
```

Where:

Integer NUMTRANS	Is the number of transactions (input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of transactions is missing.
	IERR = 3	One transaction label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.179. MONSENSTRANFLOWGATES

Use this function to display the transactions impact on the selected flowgates.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSTRANFLOWGATES  NUMTRANS LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = monsenstranflowgates(numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_MONSENSTRANFG(NUMTRANS, LISTTR, IERR)
```

Where:

Integer NUMTRANS	Is the number of transactions to be included in LISTTR (input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of transactions is missing.
	IERR = 3	One transaction label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.180. MONSENSTRANINTERFACES

Use this function to display the transactions impact on the selected interfaces.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MONSENSTRANINTERFACES  NUMTRANS LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = monsenstraninterfaces(numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_MONSENSTRANINT(NUMTRANS, LISTTR, IERR)
```

Where:

Integer NUMTRANS	(input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of transactions is missing.
	IERR = 3	One transaction label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.181. MULTACFCITC

Use this function to calculation the effects of a series of AC FCITC Transfers

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MULTACFCITC NUMTRANS LISTTRANS OPTIONS(1)..OPTIONS(5) ACTOLN
```

Python command syntax:

```
ierr = multacfcitc(numtrans, listtrans, options, actoln)
```

Fortran command syntax:

```
CALL MAPI_MULTACFCITC(NUMTRANS, LISTTRANS, OPTIONS, ACTOLN, IERR)
```

Where:

Integer NUMTRANS	(input; no default allowed).	
Character*32 LISTTRANS (3*NUMTRANS)	(input; no default allowed).	
Integer OPTIONS (5)	array of 5 elements specifying calculation options(input).	
	OPTIONS(1)	FCITC report format (2 by default)
	OPTIONS(1) = 1	Standard Report
	OPTIONS(1) = 2	Wide Report (Spreadsheet format)
	OPTIONS(2)	Maximum number violations to verify using AC (5 by default)
	OPTIONS(2) > n	Number of DC violations to consider
	OPTIONS(3)	Apply non-divergent LF (0 by default)
	OPTIONS(3) = 0	never
	OPTIONS(3) = 1	always
	OPTIONS(3) = 2	only if regular diverged
	OPTIONS(4)	AC LF method (0 by default)
	OPTIONS(4) = 1	Newton
	OPTIONS(4) = 2	Decoupled
	OPTIONS(5)	Report removed violations with FCITC report (0 by default)
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
Real ACTOLN (1)	array of 1 elements specifying Tolerance to check study transfer (MW)(input).	
	ACTOLN(1)	AC load flow tolerance level(MW) (10.0 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 2	Option is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.182. MULTDCFCITC

Use this function to calculation the effects of a series of Linear FCITC Transfers

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_MULTDCFCITC  NUMTRANS LISTTRANS OPTIONS(1)..OPTIONS(4)
```

Python command syntax:

```
ierr = multdcfcitc(numtrans, listtrans, options)
```

Fortran command syntax:

```
CALL MAPI_MULTDCFCITC(NUMTRANS, LISTTRANS, OPTIONS, IERR)
```

Where:

Integer NUMTRANS	(input; no default allowed).	
Character*32 LISTTRANS (3*NUMTRANS)	(input; no default allowed).	
Integer OPTIONS (4)	array of 4 elements specifying calculation options(input).	
	OPTIONS(1)	FCITC report format (2 by default)
	OPTIONS(1) = 1	2 Line report
	OPTIONS(1) = 2	Wide Report (Spreadsheet format)
	OPTIONS(1) = 3	TLTG Report
	OPTIONS(1) = 4	Wide Report (Old format)
	OPTIONS(2)	Maximum violations to display (5 by default)
	OPTIONS(2) > n	Number of violations to present on report
	OPTIONS(3)	Add separate report for basecase (0 by default)
	OPTIONS(3) = 0	no
	OPTIONS(3) = 1	yes
	OPTIONS(4)	Report removed violations with FCITC report (0 by default)
	OPTIONS(4) = 0	no
	OPTIONS(4) = 1	yes
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Option is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.183. MULTIPLEPARALLELTRANSFERS

Use this function to display the impact of a selected set of parallel transfers on one study transfer FCITC.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_MULTIPLEPARALLELTRANSFERS SYS1 SYS2 TRANSFER MAXTRANLV TRSTEP NUMTRANS
 LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = multipleparalleltransfers(sys1, sys2, transfer, maxtranlv, trstep,
 numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_MULTIPARAL(SYS1, SYS2, TRANSFER, MAXTRANLV, TRSTEP, NUMTRANS,
 LISTTR, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Real MAXTRANLV	Is the maximum power transfer to be considered in this study (input; no default allowed).	
Real TRSTEP	Is the increment of power transfer for each step in this study (input; no default allowed).	
Integer NUMTRANS	How many transactions to consider. (input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer not defined
	IERR = 3	Max Power Transfer outside of limits
	IERR = 4	Illegal Transfer Step
	IERR = 5	No Subsystems Selected
	IERR = 6	Specified Subsystem not found
	IERR = 7	The maximum power transfer is missing.
	IERR = 8	The increment of power transfer is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	The number of subsystems is missing.
IERR = 10	One label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.184. MULTIPLESTUDYTRANSFERS

Use this function to display the impact of one additional transaction on a whole group of study transfers.

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_MULTIPLESTUDYTRANSFERS SYS1 SYS2 TRANSFER MAXTRANLV TRSTEP NUMTRANS
 LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = multiplestudytransfers(sys1, sys2, transfer, maxtranlv, trstep,
 numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_MULTISTUDY(SYS1, SYS2, TRANSFER, MAXTRANLV, TRSTEP, NUMTRANS,
 LISTTR, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Real MAXTRANLV	Is the maximum power transfer to be considered in this study (input; no default allowed).	
Real TRSTEP	Is the increment of power transfer for each step in this study (input; no default allowed).	
Integer NUMTRANS	How many transactions to consider. (input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer not defined
	IERR = 3	Max Power Transfer outside of limits
	IERR = 4	Illegal Transfer Step
	IERR = 5	No Subsystems Selected
	IERR = 6	Specified Subsystem not found
	IERR = 7	The maximum power transfer is missing.
	IERR = 8	The increment of power transfer is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 9	The number of subsystems is missing.
IERR = 10	One label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.185. MUSTOPTDEFS

Use this function to enter a set of default MUST Options for later use in Transfer Analysis calculations

This API routine was first introduced in release 35.3.0.

Batch command syntax:

```
BAT_MUSTOPTDEFS  OPTIONS(1)..OPTIONS(6) VALUES(1)..VALUES(4)
```

Python command syntax:

```
ierr = mustoptdefs(options, values)
```

Fortran command syntax:

```
CALL MAPI_MUSTOPTDEFS(OPTIONS, VALUES, IERR)
```

Where:

Integer OPTIONS (6)	array of 6 elements specifying calculation options(input).	
	OPTIONS(1)	Phase shifter code (0 by default)
	OPTIONS(1) = 0	fixed
	OPTIONS(1) = 1	const.flow base case
	OPTIONS(1) = 2	const.flow in contingency
	OPTIONS(2)	ignore base case constraints in FCITC reports (1 by default)
	OPTIONS(2) = 0	exclude
	OPTIONS(2) = 1	include
	OPTIONS(3)	Max. number of violations to report in FCITC and Gen.Sens. Tables (50000 by default)
	OPTIONS(3) > n	violations
	OPTIONS(4)	Summary table maximum times to report the same element (3 by default)
	OPTIONS(4) = n	Limited
	OPTIONS(4) = 100	Unlimited
	OPTIONS(5)	Apply Min.Distr.Factor CutOff for contingency analysis (0 by default)
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
	OPTIONS(6)	Apply minimum distr. factor to FCITC solution reports (0 by default)
	OPTIONS(6) = 0	no
	OPTIONS(6) = 1	yes
Real VALUES (4)	array of 4 elements specifying reporting parameters(input).	
	VALUES(1)	Default minimum distribution factor magnitude cutoff (PTDF and OTDF) (0.02 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	VALUES(2)	Minimum contingency case (pre and post) flow change (0.0 by default)
	VALUES(3)	Minimum contingency case distribution factor change (0.0 by default)
	VALUES(4)	Minimum distribution factor for transfer sensitivity analysis (0.0 by default)
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer option value is missing
	IERR = 3	Float option value is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.186. MUSTSTART

Use this function to startup pss must module from within the Python environment. This function will allocate memory and initialize the pss must module environment for further commands.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MUSTSTART
```

Python command syntax:

```
ierr = muststart()
```

Fortran command syntax:

```
CALL MAPI_INITMUST(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Failed to start pss must module

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.187. MUSTSTOP

Use this function to shutdown pss must module from within the Python environment.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_MUSTSTOP
```

Python command syntax:

```
ierr = muststop()
```

Fortran command syntax:

```
CALL MAPI_SHUTDOWNMUST(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.188. PARALLELDETAILTRANSFER

Use this function to generate a detailed report on FCITC adjustments for the selected transfer.

This API routine was first introduced in release 34.0.0.

Batch command syntax:

```
BAT_PARALLELDETAILTRANSFER  MAXPOWTR SYSFROM SYSTO REPTYP
```

Python command syntax:

```
ierr = paralleldetailtransfer(maxpowtr, sysfrom, systo, reptyp)
```

Fortran command syntax:

```
CALL MAPI_TSENDET(MAXPOWTR, SYSFROM, SYSTO, REPTYP, IERR)
```

Where:

Real MAXPOWTR	Is the maximum power transfer to be considered in this study (input; no default allowed).
Character*32 SYSFROM	Is the subsystem From which a power transfer parallel to the current study transfer is to be considered (input; no default allowed).
Character*32 SYSTO	Is the subsystem To which a power transfer parallel to the current study transfer is to be considered (input; no default allowed).
Character*32 REPTYP	Is a set of strings forming a list of subsystems. There must be as many subsystems as indicated by NUMTRANS. (input; no default allowed). REPTYP = "SUMMARY" : Provide a summary report REPTYP = "DETAIL" : Provide a detailed report REPTYP = "BOTH" : Provide both a summary report and a detailed report
Integer IERR	error code (output). IERR = 0 No error IERR = 1 Prerequisite requirements for function not met IERR = 2 Study Transfer not defined IERR = 3 Specified Subsystem not found IERR = 4 Specified Report Type not found IERR = 5 The maximum power transfer is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.189. PARALLELFCITCSUMMARY

Use this function to generate a summary on FCITC adjustments for several Parallel transfers.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_PARALLELFCITCSUMMARY  MAXPOWTR TRSTEP NUMTRANS
 LISTTR(1)..LISTTR(NUMTRANS)
```

Python command syntax:

```
ierr = parallelfcitcsummary(maxpowtr, trstep, numtrans, listtr)
```

Fortran command syntax:

```
CALL MAPI_TSENSUM(MAXPOWTR, TRSTEP, NUMTRANS, LISTTR, IERR)
```

Where:

Real MAXPOWTR	Is the maximum power transfer to be considered in this study (input; no default allowed).	
Real TRSTEP	Is the increment of power transfer for each step in this study (input; no default allowed).	
Integer NUMTRANS	(input; no default allowed).	
Character*32 LISTTR (2*NUMTRANS)	Is a set of strings forming a list of transactions. Each transaction consists of a FROM Subsystem Name followed by a TO Subsystem Name. There must be as many sets of FROM and TO subsystem names as indicated by NUMTRANS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer not defined
	IERR = 3	Max Power Transfer outside of limits
	IERR = 4	Illegal Transfer Step
	IERR = 5	No Subsystems Selected
	IERR = 6	Specified Subsystem not found
	IERR = 7	The maximum power transfer is missing.
	IERR = 8	The increment of power transfer is missing.
	IERR = 9	The number of subsystems is missing.
	IERR = 10	One label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.190. PARALLELTRANSFERDETAIL

Use this function to generate a detailed report on FCITC adjustments for a parallel transfer.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_PARALLELTRANSFERDETAIL SYS1 SYS2 STRAN REPORT SYSFROM SYSTO PTRAN
```

Python command syntax:

```
ierr = paralleltransferdetail(sys1, sys2, stran, report, sysfrom, systo,
 ptran)
```

Fortran command syntax:

```
CALL MAPI_PARADETXFER(SYS1, SYS2, STRAN, REPORT, SYSFROM, SYSTO, PTRAN,
 IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real STRAN	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer REPORT	Extent of Report (input; no default allowed).	
	'1'	Provide a summary report
	'2'	Provide a detailed report
	'3'	Provide both a summary report and a detailed report
Character*32 SYSFROM	Is the subsystem from which a power transfer parallel to the current study transfer is to be considered. (input; no default allowed).	
Character*32 SYSTO	Is the subsystem to which a power transfer parallel to the current study transfer is to be considered. (input; no default allowed).	
Real PTRAN	Is the MW Transfer Level for the Parallel Transfer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Study Transfer not defined
	IERR = 3	Max Power Transfer outside of limits
	IERR = 4	Illegal Transfer Step
	IERR = 5	No Subsystems Selected
	IERR = 6	Specified Subsystem not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 7	The maximum power transfer is missing.
IERR = 8	The increment of power transfer is missing.
IERR = 9	The number of subsystems is missing.
IERR = 10	One label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.191. PRINTRPT

Use this function to display a report as specified below.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_PRINTRPT  MARG
```

Python command syntax:

```
ierr = printrpt(marg)
```

Fortran command syntax:

```
CALL MAPI_PRINTRPT(MARG, IERR)
```

Where:

Character*20 MARG	(input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid Menu Selected - No changes made

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.192. PSCASE

Use this function to enter a PSSE SAV case file into pss must module.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_PSCASE  DFLAG CSFILE
```

Python command syntax:

```
ierr = pscase(dflag, csfile)
```

Fortran command syntax:

```
CALL MAPI_READSAVCASE(DFLAG, CSFILE, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Character*260 CSFILE	Is a string identifying the full pathname of the PSSE case file (.sav) to be read. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	sav file missing, invalid name or no access
	IERR = 3	No buses found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.193. RDEXCHGFILE

Use this function to enter a file listing the constraints or contingencies to be excluded.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_RDEXCHGFILE  DFLAG EXFILE
```

Python command syntax:

```
ierr = rdexchgfile(dflag, exfile)
```

Fortran command syntax:

```
CALL MAPI_READEXCCHGFILE(DFLAG, EXFILE, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Character*260 EXFILE	Is a string identifying the full pathname of the excluded monitored elements file (.exc) to be read. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Exclude file missing, invalid name or no access
	IERR = 3	Error reading exclude/change file
	IERR = 4	Memory allocation error. Out of memory
	IERR = 5	Exclude element(s) not recognized

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.194. RDMONCONFILE

Use this function to enter both a monitored elements file and a contingency description file. You must enter a name for both files.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_RDMONCONFILE  DFLAG MNFILE CNFILE
```

Python command syntax:

```
ierr = rdmonconfile(dflag, mnfile, cnfile)
```

Fortran command syntax:

```
CALL MAPI_READMONCONFILE(DFLAG, MNFILE, CNFILE, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Character*260 MNFILE	Is a string identifying the full pathname of the monitored elements file (.mon) to be read. (input; no default allowed).	
Character*260 CNFILE	Is a string identifying the full pathname of the contingency description file (.con) to be read. (input; no default allowed).	
Integer IERR	Is the error code (output) (output).	
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Monitor file missing, invalid name or no access
	IERR = 3	Contingency file missing, invalid name or no access
	IERR = 4	Error reading Monitor/Contingency file
	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Monitor file missing, invalid name or no access
	IERR = 3	Contingency file missing, invalid name or no access
	IERR = 4	No buses
	IERR = 5	Setup Error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.195. RDMUSTFILES

Use this function to enter a subsytems definition file, a monitored elements file and a contingency description file. You must enter a name for all three files.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_RDMUSTFILES  SBFILE MNFILE CNFILE
```

Python command syntax:

```
ierr = rdmustfiles(sbfile, mnfile, cnfile)
```

Fortran command syntax:

```
CALL MAPI_READMUSTFILES(SBFILE, MNFILE, CNFILE, IERR)
```

Where:

Character*260 SBFILE	Is a string identifying the full pathname of the subsystems definition file (.sub) to be read. (input; no default allowed).	
Character*260 MNFILE	Is a string identifying the full pathname of the monitored elements file (.mon) to be read. (input; no default allowed).	
Character*260 CNFILE	Is a string identifying the full pathname of the contingency description file (.con) to be read. (input; no default allowed).	
Integer IERR	Is the error code (output) (output).	
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem file missing, invalid name or no access
	IERR = 3	Monitor file missing, invalid name or no access
	IERR = 4	Contingency file missing, invalid name or no access
	IERR = 5	Error reading SubSystem/Monitor/Contingency file
	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem file missing, invalid name or no access
	IERR = 3	Monitor file missing, invalid name or no access
	IERR = 4	Contingency file missing, invalid name or no access
	IERR = 5	Error reading Subsystem/Monitor/Contingency file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6	Setup Error

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.196. RDSUBFILE

Use this function to enter a subsystem definition file.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_RDSUBFILE  DFLAG SBFILE
```

Python command syntax:

```
ierr = rdsubfile(dflag, sbfile)
```

Fortran command syntax:

```
CALL MAPI_READSUBSYS(DFLAG, SBFILE, IERR)
```

Where:

Integer DFLAG	This argument is not used and kept only for backward compatibility. (input).	
Character*260 SBFILE	Is a string identifying the full pathname of the subsystem definition file (.sub) to be read. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsys file missing, invalid name or no access
	IERR = 3	No subsystems found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.197. READACCONTFILE

Use this function to read an AC contingency analysis results file into the ACCONT activity.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_READACCONTFILE  FNAME
```

Python command syntax:

```
ierr = readaccontfile(fname)
```

Fortran command syntax:

```
CALL MAPI_ACCONTREAD(FNAME, IERR)
```

Where:

Character*260 FNAME	Is a string identifying the full pathname of the AC contingency analysis results file (.acc) to be read. (input).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The full pathname of the AC contingency analysis results file is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.198. READRAWDATA

Use this function to enter a PSSE RAW case file.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_READRAWDATA  NUMNAM WRFILE
```

Python command syntax:

```
ierr = readrawdata(numnam, wrfile)
```

Fortran command syntax:

```
CALL MAPI_READRAWDATA(NUMNAM, WRFILE, IERR)
```

Where:

Integer NUMNAM	Set NUMNAM to read BUSES by NUMBER or NAME (input; no default allowed).	
	NUMNAM = 0	read BUSES by NUMBER
	NUMNAM = 1	read BUSES by NAME
Character*260 WRFILE	Is a string identifying the full pathname of the RAW case file (.raw) to be read. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Request to read by Numbers but MUST options set to read by Names
	IERR = 3	Request to read by Names but MUST options set to read by Numbers
	IERR = 4	The option to read BUSES is missing.
	IERR = 5	The full pathname of the RAW case file is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.199. REAREABUS

Use this function to change the area assigned to a bus.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_REAREABUS  BUSNO AREANO
```

Python command syntax:

```
ierr = reareabus(busno, areano)
```

Fortran command syntax:

```
CALL MAPI_REAREA(BUSNO, AREANO, IERR)
```

Where:

Integer BUSNO	Number of bus to be reassigned (input; no default allowed).	
Integer AREANO	Area number to which bus is to be reassigned (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Bus not found
	IERR = 3	Area not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.200. REMOVEEXCLUDES

Use this function to remove all excluded constraints and contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_REMOVEEXCLUDES
```

Python command syntax:

```
ierr = removeexcludes()
```

Fortran command syntax:

```
CALL MAPI_REMOVEXCLUDE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.201. REZONEBUS

Use this function to change the zone assigned to a bus.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_REZONEBUS  BUSNO ZONENO
```

Python command syntax:

```
ierr = rezonebus(busno, zoneno)
```

Fortran command syntax:

```
CALL MAPI_REZONE(BUSNO, ZONENO, IERR)
```

Where:

Integer BUSNO	Number of bus to be reassigned (input; no default allowed).	
Integer ZONENO	Zone number to which bus is to be reassigned (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Bus not found
	IERR = 3	Zone not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.202. SAVEACCONTFILE

Use this function to save the AC contingency analysis results to a file from the ACCONT activity.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SAVEACCONTFILE  FNAME
```

Python command syntax:

```
ierr = saveaccontfile(fname)
```

Fortran command syntax:

```
CALL MAPI_ACCONTSAVE(FNAME, IERR)
```

Where:

Character*260 FNAME	Is a string identifying the full pathname of the file (.acc) to which the AC contingency analysis will be saved. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	the full pathname of the AC contingency analysis results file is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.203. SAVEFLOWGATES

Use this function to generate a list of flowgates in a format which can be read back into MUST.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SAVEFLOWGATES
```

Python command syntax:

```
ierr = saveflowgates()
```

Fortran command syntax:

```
CALL MAPI_SAVEFGATES(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.204. SAVESELECTEDCONTS

Use this function to display the selected contingencies formatted for inclusion in a contingency definition file.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SAVESELECTEDCONTS
```

Python command syntax:

```
ierr = saveselectedconts()
```

Fortran command syntax:

```
CALL MAPI_SAVSELCONTS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.205. SAVESUBSYSTEMS

Use this function to generate a list of participating subsystems in a format which can be read back into MUST.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SAVESUBSYSTEMS
```

Python command syntax:

```
ierr = savesubsystems()
```

Fortran command syntax:

```
CALL MAPI_SAVSUBS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.206. SAVEXCHGFILE

Use this function to list all excluded constraints and contingencies in a form suitable for capturing to a file.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SAVEXCHGFILE
```

Python command syntax:

```
ierr = savexchgfile()
```

Fortran command syntax:

```
CALL MAPI_SAVEXCCHGFILE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.207. SCEDANALYSIS

Use this function to determine a generation pattern to minimize generation cost or remove branch overheating.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SCEDANALYSIS SMODE MERIT SECUR MONIT RELAX RATIN CRATN XPCNT RCONT
 RPREV SCUTF SECMAR GENER LOAD PSHIFT DCLINE TOTAREA FNAME RNAME PNAME
```

Python command syntax:

```
ierr = scedanalysis(smode, merit, secur, monit, relax, ratin, cratn,
 xpcnt, rcont, rprev, scutf, secmar, gener, load, pshift, dcline, totarea,
 fname, rname, pname, ierr)
```

Fortran command syntax:

```
CALL MAPI_SCEDAPI(SMODE, MERIT, SECUR, MONIT, RELAX, RATIN, CRATN, XPCNT,
 RCONT, RPREV, SCUTF, SECMAR, GENER, LOAD, PSHIFT, DCLINE, TOTAREA, FNAME,
 RNAME, PNAME, IERR)
```

Where:

Integer SMODE	Enter 0 for SCED Analysis or 1 for SCBD Analysis (input; no default allowed).
Integer MERIT	Enter 1 (yes) to include Merit Order or 0 (No) to ignore (input; no default allowed).
Integer SECUR	Enter 1 (yes) to include Security Constraints or 0 (No) to ignore (input; no default allowed).
Integer MONIT	Enter desired Security Check: -1 to ignore; 0 for All Branches; 1 for Monitored Branches Only (input; no default allowed).
Integer RELAX	Is the Relaxation input option (input; no default allowed).
Integer RATIN	Is the Branch Violations Rating for base case (1-12) (input; no default allowed).
Integer CRATN	Is the option for Run Correct (input; no default allowed).
Integer XPCNT	Is the option for Run Load shedding (input; no default allowed).
Integer RCONT	Is the Branch Violations Rating for contingency case (1-12) (input; no default allowed).
Integer RPREV	Is the option for Run Preventive case (input; no default allowed).
Real SCUTF	Is the option for Sensitivity Cutoff (input; no default allowed).
Real SECMAR	Is the Security Margin percentage(0-25) (input; no default allowed).
Integer GENER	Is the MW Controls for Generation (input; no default allowed).
Integer LOAD	Is the MW Controls for Load (input; no default allowed).
Integer PSHIFT	Is the MW Controls for Phase Shifter (input; no default allowed).
Integer DCLINE	Is the MW Controls for DC Lines (input; no default allowed).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer TOTAREA	Is the option for Total Area Generation Persistence (input; no default allowed).
Character*260 FNAME	Is the full pathname of the ECDI File (input; no default allowed).
Character*260 RNAME	Is the full pathname of the BASE Result File (input).
Character*260 PNAME	Is the full pathname of the Preventive case Result File (input).
Integer IERR	error code (output).

IERR = 0	No error
IERR = 1	Prerequisite requirements for function not met
IERR = 2	SCED input option out of range (limited to 0 or 1)
IERR = 3	Merit order input option out of range (limited to 0 or 1)
IERR = 4	Constraint option input out of range (limited to 0 or 1)
IERR = 5	Monitor Branch input option out of range (limited to 0 or 1)
IERR = 6	Relaxation input option out of range (limited to 0 or 1)
IERR = 7	Base rating input option out of range (limited to 1 through 12)
IERR = 8	Run correction input option out of range (limited to 0 or 1)
IERR = 9	Run load shedding input option out of range (limited to 0 or 1)
IERR = 10	Contingency rating input option out of range (limited to 1 through 12)
IERR = 11	Run preventive case input option out of range (limited to 0 or 1)
IERR = 12	Sensitivity cutoff value out of range (limited to 0.05 through 1.0)
IERR = 13	ECDI filename is missing
IERR = 14	Security Marge percentage out of range (limited to 0 through 25)
IERR = 15	Generation input option out of range (limited to 0 or 1)
IERR = 16	Load input option out of range (limited to 0 or 1)
IERR = 17	Phase Shifter input option out of range (limited to 0 or 1)
IERR = 18	DC Lines input option out of range (limited to 0 or 1)
IERR = 19	Total Area Generation Persistence input option out of range (limited to 0 or 1)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.208. SELECTACTIVECONTS

Use this function to select only active contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTACTIVECONTS
```

Python command syntax:

```
ierr = selectactiveconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTACTIVE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.209. SELECTALLCONTS

Use this function to select ALL contingencies to report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTALLCONTS
```

Python command syntax:

```
ierr = selectallconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTALL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.210. SELECTCOMPLEXCONTS

Use this function to select only complex (not single branch opening) contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTCOMPLEXCONTS
```

Python command syntax:

```
ierr = selectcomplexconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTCMPLX(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.211. SELECTCONVERGECONTS

Use this function to select only contingencies that did converge.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTCONVERGECONTS
```

Python command syntax:

```
ierr = selectconvergeconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTCONV(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.212. SELECTDISABLEDCONTS

Use this function to select only contingencies disabled due to data errors.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTDISABLEDCONTS
```

Python command syntax:

```
ierr = selectdisabledconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTDISABLE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.213. SELECTEXCLUDEDCONTS

Use this function to select only user excluded contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTEXCLUDEDCONTS
```

Python command syntax:

```
ierr = selectexcludedconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTEXC(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.214. SELECTGENCONTS

Use this function to select only contingencies with generation adjustments.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTGENCONTS
```

Python command syntax:

```
ierr = selectgenconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTGEN(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.215. SELECTHISEVCONTS

Use this function to select only contingencies with higher severity index.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTHISEVCONTS  SEVCO
```

Python command syntax:

```
ierr = selecthisevconts(sevco)
```

Fortran command syntax:

```
CALL MAPI_SELCONTHSEV(SEVCO, IERR)
```

Where:

Real SEVCO	Is Severity Index. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Severity index out of range (0 - 200)
	IERR = 3	Severity Index is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.216. SELECTIMBALANCECONTS

Use this function to select only imbalanced contingencies (load+gener+losses) to report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTIMBALANCECONTS
```

Python command syntax:

```
ierr = selectimbalanceconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTIMBAL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.217. SELECTISLANDCONTS

Use this function to select only contingencies causing separations.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTISLANDCONTS
```

Python command syntax:

```
ierr = selectislandconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTISL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.218. SELECTLOADCONTS

Use this function to select only contingencies with load adjustments.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTLOADCONTS
```

Python command syntax:

```
ierr = selectloadconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTLD(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.219. SELECTLOSEVCONTS

Use this function to select only contingencies with lower severity index.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTLOSEVCONTS  SEVCO
```

Python command syntax:

```
ierr = selectlosevconts(sevco)
```

Fortran command syntax:

```
CALL MAPI_SELCONTLSEV(SEVCO, IERR)
```

Where:

Real SEVCO	Is severity index. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Severity index out of range (0 - 200)
	IERR = 3	Severity Index is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.220. SELECTNOTCONVERGECONTS

Use this function to select only contingencies that did not converge.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTNOTCONVERGECONTS
```

Python command syntax:

```
ierr = selectnotconvergeconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTNCONV(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.221. SELECTNOVIOLCONTS

Use this function to select only contingencies without violations.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTNOVIOLCONTS
```

Python command syntax:

```
ierr = selectnoviolconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTNOVIOL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.222. SELECTSINGLEBRANCHCONTS

Use this function to select only single branch contingencies opening without islands.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTSINGLEBRANCHCONTS
```

Python command syntax:

```
ierr = selectsinglebranchconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTSINGLE(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.223. SELECTSPECIFICCONTS

Use this function to select only specified contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTSPECIFICCONTS  NUMCONTS CONTLIST(1)..CONTLIST(NUMCONTS)
```

Python command syntax:

```
ierr = selectspecificconts(numconts, contlist)
```

Fortran command syntax:

```
CALL MAPI_SELCONTSPEC(NUMCONTS, CONTLIST, IERR)
```

Where:

Integer NUMCONTS	Is the number of contingencies to be included in CONTLIST (input; no default allowed).	
Integer CONTLIST	Is a set of integers forming a list of contingency numbers. There must be as many contingencies as indicated by NUMCONTS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of contingencies is missing.
	IERR = 3	One label of contingencies is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.224. SELECTVIOLCONTS

Use this function to select only contingencies with violations.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SELECTVIOLCONTS
```

Python command syntax:

```
ierr = selectviolconts()
```

Fortran command syntax:

```
CALL MAPI_SELCONTVIOL(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.225. SETEXCLUDEDBRANCHES

Use this function to exclude branches by from bus, to bus and exclusion option.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETEXCLUDEDBRANCHES  NUMBRA LISTBRA(1)..LISTBRA(NUMBRA)
```

Python command syntax:

```
ierr = setexcludedbranches(numbra, listbra)
```

Fortran command syntax:

```
CALL MAPI_SETEXCLUDEDBRANCHES(NUMBRA, LISTBRA, IERR)
```

Where:

Integer NUMBRA	(input; no default allowed).	
Character*32 LISTBRA (4*NUMBRA)	Is a set of strings forming a list of excluded branches. Each branch consists of a FROM bus followed by a TO bus followed by a Circuit ID and Exclusion Option index. There must be as many sets of FROM TO Exclusion Option as indicated by NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of monitored elements is missing.
	IERR = 3	One branch label is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.226. SETEXCLUDEDCONTINGENCIES

Use this function to exclude contingencies by contingency ID.

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETEXCLUDEDCONTS  NUMCONTS CONTLIST(1)..CONTLIST(NUMCONTS)
```

Python command syntax:

```
ierr = setexcludedcontingencies(numconts, contlist)
```

Fortran command syntax:

```
CALL MAPI_SETEXCLUDEDCONTS(NUMCONTS, CONTLIST, IERR)
```

Where:

Integer NUMCONTS	Is the number of contingencies to be included in CONTLIST (input; no default allowed).	
Integer CONTLIST	Is a set of integers forming a list of contingency numbers. There must be as many contingencies as indicated by NUMCONTS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of contingencies is missing.
	IERR = 3	One of Contingency Id is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.227. SETEXCLUDEDFLOWGATES

Use this function to exclude flowgates by flowgate ID

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETEXCLUDEDFGATES  NUMFGATES LISTFGATES(1)..LISTFGATES(NUMFGATES)
```

Python command syntax:

```
ierr = setexcludedflowgates(numfgates, listfgates)
```

Fortran command syntax:

```
CALL MAPI_SETEXCLUDEDFGATES(NUMFGATES, LISTFGATES, IERR)
```

Where:

Integer NUMFGATES	Is the number of excluded flowgates to be included in LISTFGATES (input; no default allowed).	
Integer LISTFGATES	Is a set of integers forming a list of flowgate ID's. There must be as many flowgates as indicated by NUMFGATES. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of flowgate IDs is missing.
	IERR = 3	One of the flowgate IDs is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.228. SETEXCLUDEDINTERFACES

Use this function to exclude interfaces by interface ID and exclusion option

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETEXCLUDEDINTERS  NUMINTER LISTINTER(1)..LISTINTER(NUMINTER)
```

Python command syntax:

```
ierr = setexcludedinterfaces(numinter, listinter)
```

Fortran command syntax:

```
CALL MAPI_SETEXCLUDEDINTERS(NUMINTER, LISTINTER, IERR)
```

Where:

Integer NUMINTER	Is the number of excluded interfaces to be included in LISTML (input; no default allowed).	
Integer LISTINTER (2*NUMINTER)	Is a set of integers forming a list of interface ID's followed by exclusion option index. There must be as many sets of interface ID to Exclusion Option as indicated by NUMINTER. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of interfaces is missing.
	IERR = 3	One of Interface Id is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.229. SETEXCLUDEDVOLTAGEBUSES

Use this function to exclude Voltage Buses by bus ID and exclusion option

This API routine was first introduced in release 35.1.0.

Batch command syntax:

```
BAT_SETEXCLUDEDBUSES  NUMBUS LISTBUS(1)..LISTBUS(NUMBUS)
```

Python command syntax:

```
ierr = setexcludedbuses(numbus, listbus)
```

Fortran command syntax:

```
CALL MAPI_SETEXCLUDEDBUSES(NUMBUS, LISTBUS, IERR)
```

Where:

Integer NUMBUS	Is the number of excluded voltage buses to be included in LISTML (input; no default allowed).	
Integer LISTBUS (2*NUMBUS)	Is a set of integers forming a list of voltage bus ID's followed by exclusion option index. There must be as many sets of voltage bus ID to Exclusion Option as indicated by NUMBUS. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	The number of voltage bus IDs is missing.
	IERR = 3	One of the voltage bus IDs is missing.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.230. SGLACFCITC

Use this function to calculation the effects of a Single AC FCITC Transfer

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SGLACFCITC SYS1 SYS2 TRANSFER OPTSI(1)..OPTS(5) ACTOLN
```

Python command syntax:

```
ierr = sglacfcitc(sys1, sys2, transfer, options, actoln)
```

Fortran command syntax:

```
CALL MAPI_SGLACFCITC(SYS1, SYS2, TRANSFER, OPTIONS, ACTOLN, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer OPTIONS (5)	array of 5 elements specifying calculation options(input).	
	OPTIONS(1)	FCITC report format (2 by default)
	OPTIONS(1) = 1	Standard Report
	OPTIONS(1) = 2	Wide Report (Spreadsheet format)
	OPTIONS(2)	Maximum number violations to verify using AC (5 by default)
	OPTIONS(2) > n	Number of DC violations to consider
	OPTIONS(3)	Apply non-divergent LF (0 by default)
	OPTIONS(3) = 0	never
	OPTIONS(3) = 1	always
	OPTIONS(3) = 2	only if regular diverged
	OPTIONS(4)	AC LF method (0 by default)
	OPTIONS(4) = 1	Newton
	OPTIONS(4) = 2	Decoupled
	OPTIONS(5)	Report removed violations with FCITC report (0 by default)
	OPTIONS(5) = 0	no
	OPTIONS(5) = 1	yes
Real ACTOLN (1)	array of 1 elements specifying Tolerance to check study transfer (MW)(input).	
	ACTOLN(1)	AC load flow tolerance level(MW) (10.0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Option is missing or invalid

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.231. SGLDCFCITC

Use this function to calculation the effects of a Single Linear FCITC Transfer

This API routine was first introduced in release 35.2.0.

Batch command syntax:

```
BAT_SGLDCFCITC SYS1 SYS2 TRANSFER OPTSI(1)..OPTS(9) MAXTR, NUMMONEL,
 LISTML(1)..LISTML(NUMMONEL)
```

Python command syntax:

```
ierr = sgldcfcitc(sys1, sys2, transfer, options, maxtr, nummonel, listml)
```

Fortran command syntax:

```
CALL MAPI_SGLDCFCITC(SYS1, SYS2, TRANSFER, OPTIONS, MAXTR, NUMMONEL,
 LISTML, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).
Integer OPTIONS (8)	array of 8 elements specifying calculation options(input).

OPTIONS(1)	FCITC report format (2 by default)
OPTIONS(1) = 1	2 Line report
OPTIONS(1) = 2	Wide Report (Spreadsheet format)
OPTIONS(1) = 3	TLTG Report
OPTIONS(1) = 4	Wide Report (Old format)
OPTIONS(2)	Report FCITC violations (0 by default)
OPTIONS(2) = 0	no
OPTIONS(2) = 1	yes
OPTIONS(3)	Report Export level (0 by default)
OPTIONS(3) = 0	no
OPTIONS(3) = 1	yes
OPTIONS(4)	Report Import level (0 by default)
OPTIONS(4) = 0	no
OPTIONS(4) = 1	yes
OPTIONS(5)	Add subsystems adjustments detailed report (0 by default)
OPTIONS(5) = 0	no
OPTIONS(5) = 1	yes
OPTIONS(6)	Add separate report for basecase (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS(6) = 0	no
	OPTIONS(6) = 1	yes
	OPTIONS(7)	Report removed violations with FCITC report (0 by default)
	OPTIONS(7) = 0	no
	OPTIONS(7) = 1	yes
	OPTIONS(8)	Report Flowgates/Interfaces flows (0 by default)
	OPTIONS(8) = 0	no
	OPTIONS(8) = 1	interfaces
	OPTIONS(8) = 2	flowgates
Real MAXTR (1)	array of 1 elements specifying Subsystems adjustments detailed report(input).	
	MAXTR(1)	Maximum transfer level to consider (MW). Ignored if OPTION(5) = 0 (100.0 by default)
Integer NUMMONEL	Is the number of flowgates/interfaces to be included in LISTML (input; no default allowed).	
Character*12 LISTML	Is a set of strings forming a list of flowgate IDs or interface names. There must be as many flowgates/interfaces as indicated in NUMMONEL. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Integer Option is missing or invalid
	IERR = 3	Float Value is missing or invalid
	IERR = 4	Flowgate/Interface not found
	IERR = 5	Flowgate/Interface list missing entries

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.232. SHOWPARAMS

Use this function to display all of the option parameters currently in use for any given menu in pss must module.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SHOWPARAMS   MENUNM
```

Python command syntax:

```
ierr = showparams(menunm)
```

Fortran command syntax:

```
CALL MAPI_SHOWPARMS(MENUNM, IERR)
```

Where:

Character*20 MENUNM	Is a string identifying which menu you wish to display. Acceptable entries are: (input; no default allowed).
	MENUNM = MAINOPTIONS — General program setting options
	MENUNM = LFADJOPTIONS — Load Flow Adjustment options
	MENUNM = LFGENOPTIONS — Generator Var Limits Checking options
	MENUNM = LFSOLOPTIONS — AC Load Flow Solution options
	MENUNM = LFADVOPTIONS — Advanced Load Flow Solution options
	MENUNM = CONSTROPTS — LP/OPF Constraint/Contingency options
	MENUNM = MUSTOPTIONS — MUST Run time solution options
	MENUNM = ACVEROPTIONS — AC verification options
	MENUNM = TRANVIOLOPTS — FCITC Violations report format options
	MENUNM = ACCONTOPTS — AC Contingency analysis options
	MENUNM = CONTOPTS — Contingency reporting options
	MENUNM = TRANIMPOPTS — Transfer impact analysis Options
	MENUNM = TRGENOPTS — Generation worst dispatch options
	MENUNM = VOLTAGEOPTS — Voltage violations with respect to specified limits

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	MENUNM = BRANRE-POPTS	Branch violations (MW) and impact report analysis options
	MENUNM = BRANAM-POPTS	Branch violations (MVA) and impact report analysis options
	MENUNM = FGATERE-POPTS	Flowgate violations and impact report analysis options
	MENUNM = INTRRE-POPTS	Interface violations and impact report analysis options
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Invalid Menu Selected - No changes made

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.233. SUBSYSTEMS

Use this function to generate a subsystem report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SUBSYSTEMS
```

Python command syntax:

```
ierr = subsystems()
```

Fortran command syntax:

```
CALL MAPI_SYSTEMS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.234. SUMMARYCONTINGENCIES

Use this function to display a summary of the selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SUMMARYCONTINGENCIES
```

Python command syntax:

```
ierr = summarycontingencies()
```

Fortran command syntax:

```
CALL MAPI_SUMSELCONTS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.235. SWINGBUSES

Use this function to a report of swing buses.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_SWINGBUSES
```

Python command syntax:

```
ierr = swingbuses()
```

Fortran command syntax:

```
CALL MAPI_SWINGBUSES(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Error in obtaining list

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.236. VIOLSELCONTINGENCIES

Use this function to report the violations by selected contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_VIOLSELCONTINGENCIES
```

Python command syntax:

```
ierr = violselcontingencies()
```

Fortran command syntax:

```
CALL MAPI_VIOLSELCONTS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.237. WRITEEXCLUDES

Use this function to write all excluded constraints and contingencies.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_WRITEEXCLUDES OUT OFILE
```

Python command syntax:

```
ierr = writeexcludes(out, ofile)
```

Fortran command syntax:

```
CALL MAPI_WRITEEXCLUDES(OUT, OFILE, IERR)
```

Where:

Integer OUT	selection option (input; 1 by default).	
	OUT = 0	direct output to the data file specified by OFILE
	OUT = 1	direct output to the report window
Character*260 OFILE	name of the output file (when OUT is 0) (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	prerequisite requirements for API are not met
	IERR = 2	invalid OUT value
	IERR = 3	error opening OFILE

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.238. WRITERAWDATA

Use this function to WRITE a PSSE RAW case file from the current MUST data.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_WRITERAWDATA  NUMNAM WRFILE
```

Python command syntax:

```
ierr = writerawdata(numnam, wrfile)
```

Fortran command syntax:

```
CALL MAPI_WRITERAWDATA(NUMNAM, WRFILE, IERR)
```

Where:

integer NUMNAM	Set NUMNAM to write BUSES by NUMBER or NAME (input; no default allowed).	
	NUMNAM = 0	write BUSES by NUMBER
	NUMNAM = 1	write BUSES by NAME
Character*260 WRFILE	Is a string identifying the full pathname of the RAW case file (.raw) to be written. (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Cannot write RAW file with busnamgen on
	IERR = 3	The option to to write BUSES by NUMBER or NAME is missing.
	IERR = 4	The name of the output file is missing

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.239. ZONEBUBBLE

Use this function to update values for the Zone Bubble Diagram.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ZONEBUBBLE SYS1, SYS2, TRANSFER
```

Python command syntax:

```
ierr = zonebubble(sys1, sys2, transfer)
```

Fortran command syntax:

```
CALL MAPI_UPDZONEBUBL(SYS1, SYS2, TRANSFER, IERR)
```

Where:

Character*32 SYS1	Is a string identifying From Subsystem Name to be used in the study transfer. (input; no default allowed).	
Character*32 SYS2	Is a string identifying To Subsystem Name to be used in the study transfer. (input; no default allowed).	
Real TRANSFER	Is the MW Transfer Level for this Study Transfer (input; no default allowed).	
Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Subsystem(s) missing
	IERR = 3	No transfer value
	IERR = 4	Subsystem(s) not valid
	IERR = 5	Sending subsystem already selected
	IERR = 6	Overlapping systems

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.240. ZONESINAREAS

Use this function to a report of zones in areas.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ZONESINAREAS
```

Python command syntax:

```
ierr = zonesinareas()
```

Fortran command syntax:

```
CALL MAPI_ZONESINAREAS(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.1.241. ZONESUMMARY

Use this function to generate an zone summary report.

This API routine was first introduced in release 35.0.0.

Batch command syntax:

```
BAT_ZONESUMMARY
```

Python command syntax:

```
ierr = zonesummary()
```

Fortran command syntax:

```
CALL MAPI_ZONESUMMARY(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	No error
	IERR = 1	Prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2. MUST Data Retrieval

These API routines return data and results of MUST operations.

These routines do not record themselves when called with recording enabled.

23.2.1. GETAUTOBUBBLE

Use this API to return the numbers for the given area or zone.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
getautobubble(numbubble, bubbleinx)
```

Fortran command syntax:

```
CALL MAPI_GETAUTOBUBBLE(NUMBUBBLE, BUBBLEINX)
```

Where:

Integer NUMBUBBLE	(input).
Integer BUBBLEINX	(input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.2. GETEXCLUDEDBRANCHES

Use this function to get list of exclude branches.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
frombus, tobus, cktid, excopt = getexcludedbranches()
```

Fortran command syntax:

```
CALL MAPI_GETEXCLUDEDBRANCHES(NUMBRA, FROMBUS, TOBUS, CKTID, EXCOPT,
 NUMRETBRA)
```

Where:

Integer NUMBRA	declared size of output arrays
Integer FROMBUS (NUMBRA)	Is an array of Excluded Branch From Buses (output).
Integer TOBUS (NUMBRA)	Is an array of Excluded Branch To Buses (output).
Character*2 CKTID (NUMBRA)	Is an array of Excluded Branch To Circuit ID's (output).
Integer EXCOPT (NUMBRA)	Is an array of Excluded Branch To Exclusion Options (output).
Integer NUMRETBRA	number of excluded branches returned (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.3. GETEXCLUDEDCONTINGENCIES

Use this function to get list of exclude contingencies.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
contid,numretcont = getexcludedcontingencies()
```

Fortran command syntax:

```
CALL MAPI_GETEXCLUDEDCONTS(NUMCONT, CONTID, NUMRETCONT)
```

Where:

Integer NUMCONT	declared size of NAMES
Integer CONTID (NUMCONT)	Is an array of Excluded Contingency Id's (output).
Integer NUMRETCONT	number of libraries returned (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.4. GETEXCLUDEDFLOWGATES

Use this function to get list of exclude flowgates.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
listfgates = getexcludedflowgates()
```

Fortran command syntax:

```
CALL MAPI_GETEXCLUDEDFGATES(NUMFGATE, LISTFGATES, NUMRETFGATE)
```

Where:

Integer NUMFGATE	declared size of LISTFGATES
Integer LISTFGATES (NUMFGATE)	Is an array of Excluded Flowgates Id's (output).
Integer NUMRETFGATE	number of excluded flowgates returned (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.5. GETEXCLUDEDINTERFACES

Use this function to get list of exclude interfaces.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
interfaceid, excopt = getexcludedinterfaces()
```

Fortran command syntax:

```
CALL MAPI_GETEXCLUDEDINTERS(NUMINTER, INTERFACEID, EXCOPT, NUMRETINTER)
```

Where:

Integer NUMINTER	declared size of output arrays
Integer INTERFACEID (NUMINTER)	Is an array of Excluded Interface Id's (output).
Integer EXCOPT (NUMINTER)	Is an array of Excluded Interface Exclusion Option Index's (output).
Integer NUMRETINTER	number of libraries returned (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.6. GETEXCLUDEDVOLTAGEBUSES

Use this function to get list of exclude voltagebuses.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
busid, excopt = getexcludedvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_GETEXCLUDEDBUSES(NUMBUS, BUSID, EXCOPT, NUMRETBUS)
```

Where:

Integer NUMBUS	declared size of output arrays
Integer BUSID (NUMBUS)	Is an array of Excluded Voltage Bus Id's (output).
Integer EXCOPT (NUMBUS)	Is an array of Excluded Voltage Bus Exclusion Option Index's (output).
Integer NUMRETBUS	number of libraries returned (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.7. GETFINALBALANCE

Use this API to return the value of the final balance flow for the given area or zone.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, finalbalflow  = getfinalbalance(azindex)
```

Fortran command syntax:

```
CALL MAPI_GETFINALBAL(AZINDEX, FINALBALFLOW, IERR)
```

Where:

Integer AZINDEX	Area/Zone Number (input).	
Real FINALBALFLOW	Final Balance Flow (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; Final Flow Value returned
	IERR = 1	Final Balance Flow not calculated; Check pre-requisite requirements for function
	IERR = 2	Input Area/Zone Number not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.8. GETFINALFLOW

Use this API to return the value of the final flow between two given areas or zones.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, finalflow  = getfinalflow(azindexfrom, azindexto)
```

Fortran command syntax:

```
CALL MAPI_GETFINALFLOW(AZINDEXFROM, AZINDEXTO, FINALFLOW, IERR)
```

Where:

Integer AZINDEXFROM	From Area/Zone Number (input).	
Integer AZINDEXTO	To Area/Zone Number (input).	
Real FINALFLOW	Final Flow Between From and To Areas/Zones (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; Final Flow Value returned
	IERR = 1	Final Flow not calculated; Check prerequisite requirements for function
	IERR = 2	Input Area/Zone Number(s) not found
	IERR = 3	Connection Between Given Input Area/Zone Number(s) not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.9. GETIDOFCONTINGENCYBYNAME

Use this API to return the contingency id by name .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, id  = getidofcontingencybyname(name)
```

Fortran command syntax:

```
CALL MAPI_GETIDOFCONTINGENCYBYNAME(ID, NAME, IERR)
```

Where:

Integer ID	Contingency id (output).	
Character*12 NAME	Contingency name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Contingency not found; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.10. GETIDOFFLOWGATEBYNAME

Use this API to return the flowgate id by name .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, id  = getidofflowgatebyname(name)
```

Fortran command syntax:

```
CALL MAPI_GETIDOFFLOWGATEBYNAME(ID, NAME, IERR)
```

Where:

Integer ID	Flowgate id (output).	
Character*12 NAME	Flowgate name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Flowgate not found; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.11. GETIDOFINTERFACEBYNAME

Use this API to return the interface id by name .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, id  = getidofinterfacebyname(name)
```

Fortran command syntax:

```
CALL MAPI_GETIDOFINTERFACEBYNAME(ID, NAME, IERR)
```

Where:

Integer ID	Interface id (output).	
Character*12 NAME	Interface name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Interface not found; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.12. GETIDOFVOLTAGEBUSBYNAME

Use this API to return the voltage bus id by name .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, id  = getidofvoltagebusbyname(name)
```

Fortran command syntax:

```
CALL MAPI_GETIDOFVOLTAGEBUSBYNAME(ID, NAME, IERR)
```

Where:

Integer ID	Voltage bus id (output).	
Character*12 NAME	Voltage bus name (input).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'ID' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Voltage bus not found; 'ID' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.13. GETINITBALANCE

Use this API to return the value of the initial balance flow for the given area or zone.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, initbalflow  = getinitbalance(azindex)
```

Fortran command syntax:

```
CALL MAPI_GETINITBAL(AZINDEX, INITBALFLOW, IERR)
```

Where:

Integer AZINDEX	Area/Zone Number (input).	
Real INITBALFLOW	Initial Balance Flow (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; Initial Flow Value returned
	IERR = 1	Initial Balance Flow not calculated; Check pre-requisite requirements for function
	IERR = 2	Input Area/Zone Number not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.14. GETINITIALFLOW

Use this API to return the value of the initial flow between two given areas or zones.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, initialflow  = getinitialflow(azindexfrom, azindexto)
```

Fortran command syntax:

```
CALL MAPI_GETINITFLOW(AZINDEXFROM, AZINDEXTO, INITIALFLOW, IERR)
```

Where:

Integer AZINDEXFROM	From Area/Zone Number (input).	
Integer AZINDEXTO	To Area/Zone Number (input).	
Real INITIALFLOW	Initial Flow Between From and To Areas/Zones (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; Initial Flow Value returned
	IERR = 1	Initial Flow not calculated; Check prerequisite requirements for function
	IERR = 2	Input Area/Zone Number(s) not found
	IERR = 3	Connection Between Given Input Area/Zone Number(s) not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.15. GETIPFFACTOR

Use this API to return the IPF Factor between two given areas or zones.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, ipff  = getipffactor(azindexfrom, azindexto)
```

Fortran command syntax:

```
CALL MAPI_GETIPFFACTOR(AZINDEXFROM, AZINDEXTO, IPFF, IERR)
```

Where:

Integer AZINDEXFROM	From Area/Zone Number (input).	
Integer AZINDEXTO	To Area/Zone Number (input).	
Real IPFF	IPF Factor Between From and To Areas/Zones (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; Final Flow Value returned
	IERR = 1	Final Flow not calculated; Check prerequisite requirements for function
	IERR = 2	Input Area/Zone Number(s) not found
	IERR = 3	Connection Between Given Input Area/Zone Number(s) not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.16. GETNAMEOFCONTINGENCYBYID

Use this API to return the contingency name by id .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, name  = getnameofcontingencybyname(id)
```

Fortran command syntax:

```
CALL MAPI_GETNAMEOFCONTINGENCYBYID(ID, NAME, IERR)
```

Where:

Integer ID	Contingency id (input).	
Character*12 NAME	Contingency name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NAME' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Contingency id is out of index; 'NAME' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.17. GETNAMEOFFLOWGATEBYID

Use this API to return the flowgate name by id .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, name  = getnameofflowgatebyname(id)
```

Fortran command syntax:

```
CALL MAPI_GETNAMEOFFLOWGATEBYID(ID, NAME, IERR)
```

Where:

Integer ID	Flowgate id (input).	
Character*12 NAME	Flowgate name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NAME' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Flowgate not found; 'NAME' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.18. GETNAMEOFINTERFACEBYID

Use this API to return the interface name by id .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, name  = getnameofinterfacebyname(id)
```

Fortran command syntax:

```
CALL MAPI_GETNAMEOFINTERFACEBYID(ID, NAME, IERR)
```

Where:

Integer ID	Interface id (input).	
Character*12 NAME	Interface name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NAME' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Interface id is out of index; 'NAME' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.19. GETNAMEOFVOLTAGEBUSBYID

Use this API to return the voltage bus name by id .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, name  = getnameofvoltagebusbyname(id)
```

Fortran command syntax:

```
CALL MAPI_GETNAMEOFVOLTAGEBUSBYID(ID, NAME, IERR)
```

Where:

Integer ID	Voltage Bus id (input).	
Character*12 NAME	Voltage bus name (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; 'NAME' returned
	IERR = 1	Prerequisite requirements for function not met
	IERR = 2	Voltage bus not found; 'NAME' unchanged

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.20. GETNUMAUTOBUBBLE

Use this API to return the numbers for the given area or zone.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
numbubble = getnumautobubble()
```

Fortran command syntax:

```
CALL MAPI_GETNUMAUTOBUBBLE(NUMBUBBLE)
```

Where:

Integer NUMBUBBLE	Number of areas/zones in the bubble diagram (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.21. GETNUMOFEXCLUDEDBRANCHES

Use this API to return the number of excluded branches.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
numbra = getnumofexcludedbranches()
```

Fortran command syntax:

```
CALL MAPI_GETNUMOFEXCBRA(NUMBRA)
```

Where:

Integer NUMBRA	number of excluded branches (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.22. GETNUMOFEXCLUDEDCONTINGENCIES

Use this API to return the number of excluded contingencies.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
numcont = getnumofexcludedcontingencies()
```

Fortran command syntax:

```
CALL MAPI_GETNUMOFEXCCONT(NUMCONT)
```

Where:

Integer NUMCONT	number of Excluded Contingencies (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.23. GETNUMOFEXCLUDEDFLOWGATES

Use this API to return the number of excluded flowgates.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
numfgate = getnumofexcludedflowgates()
```

Fortran command syntax:

```
CALL MAPI_GETNUMOFEXCFGATE(NUMFGATE)
```

Where:

Integer NUMFGATE	number of Excluded Flowgates (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.24. GETNUMOFEXCLUDEDINTERFACES

Use this API to return the number of excluded interfaces.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
numinter = getnumofexcludedinterfaces()
```

Fortran command syntax:

```
CALL MAPI_GETNUMOFEXCINTER(NUMINTER)
```

Where:

Integer NUMINTER	number of Excluded Interfaces (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.25. GETNUMOFEXCLUDEDVOLTAGEBUSES

Use this API to return the number of excluded ac voltage buses.

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
numbus = getnumofexcludedvoltagebuses()
```

Fortran command syntax:

```
CALL MAPI_GETNUMOFEXCBUS(NUMBUS)
```

Where:

Integer NUMBUS	number of excluded Ac Voltage Buses (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.26. GETTPFBALANCE

Use this API to return the value of the TPF factor for the given area or zone .

This API routine was first introduced in release 35.1.0.

Python command syntax:

```
ierr, tpfbalflow  = gettpfbalance(azindex)
```

Fortran command syntax:

```
CALL MAPI_GETTPFBAL(AZINDEX, TPFBALFLOW, IERR)
```

Where:

Integer AZINDEX	Area/Zone Number (input).	
Real TPFBALFLOW	TPF Factor (output).	
Integer IERR	error code (output).	
	IERR = 0	No error; TPF Factor returned
	IERR = 1	TPF Factor not calculated; Check prerequisite requirements for function
	IERR = 2	Input Area/Zone Number not found

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

23.2.27. MUSTRESULTSEXIST

Use this API to if ALA/MUST bubble results exist in the ALA/MUST engine.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ierr = mustresultsexist()
```

Fortran command syntax:

```
CALL MAPI_MUSTRESULTSEXIST(IERR)
```

Where:

Integer IERR	error code (output).	
	IERR = 0	ALA/MUST results exist
	IERR = 1	ALA/MUST results do NOT ssssexist

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 24
PSSDYNMDL Module

This chapter defines API routines that are used for Dynamic Simulation User-written Model writing.

Please note that these routines are members of the pssdynmdl Python extension module, not psspy. They are not recordable. They cannot be used in automation files. They can only be used for dynamics models.

For Fortran users, include in your code the module PSSDM, i.e.:

USE PSSDM

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1. Dynamics model parameter retrieval functions

This section describes the routines that can return parameters, values that are fixed and do not change.

24.1.1. GET_CONMSGERROR

Use this API to return the severity value of "error" for dsconchk* routines, used for error reporting.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
conmsgerror = get_conmsgerror()
```

Fortran command syntax:

```
CONMSGERROR = GET_CONMSGERROR()
```

Where:

Integer CONMSGERROR	severity value of "error" for dsconchk* routines, used for error reporting (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.2. GET_CONMSGNONE

Use this API to return the severity value of "no error" for dsconchk* routines, used for error reporting.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
conmsgnone = get_conmsgnone()
```

Fortran command syntax:

```
CONMSGNONE = GET_CONMSGNONE()
```

Where:

Integer CONMSGNONE	severity value of "no error" for dsconchk* routines, used for error reporting (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.3. GET_CONMSGWARN

Use this API to return the severity value of "warning" for dsconchk* routines, used for error reporting.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
conmsgwarn = get_conmsgwarn()
```

Fortran command syntax:

```
CONMSGWARN = GET_CONMSGWARN()
```

Where:

Integer CONMSGWARN	severity value of "warning" for dsconchk* routines, used for error reporting (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.4. GET_DOCUCHKERR_EQUAL

Use this API to return the value to signal if CON equal to test value.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_equal = get_docuchkerr_equal()
```

Fortran command syntax:

```
DOCUCHKERR_EQUAL = GET_DOCUCHKERR_EQUAL()
```

Where:

Integer DOCUCHKERR_EQUAL	value to signal if CON equal to test value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.5. GET_DOCUCHKERR_GREATEROREQ

Use this API to return the value to signal if CON greater than or equal to test value.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_greateroreq = get_docuchkerr_greateroreq()
```

Fortran command syntax:

```
DOCUCHKERR_GREATEROREQ = GET_DOCUCHKERR_GREATEROREQ()
```

Where:

Integer DOCUCHKER-R_GREATEROREQ	value to signal if CON greater than or equal to test value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.6. GET_DOCUCHKERR_GREATERTHAN

Use this API to return the value to signal if CON greater than test value.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_greaterthan = get_docuchkerr_greaterthan()
```

Fortran command syntax:

```
DOCUCHKERR_GREATERTHAN = GET_DOCUCHKERR_GREATERTHAN()
```

Where:

Integer DOCUCHKER-R_GREATERTHAN	value to signal if CON greater than test value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.7. GET_DOCUCHKERR_INCLHIRANGE

Use this API to return the value to signal if CON not within range, second value inclusive.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_inclhirange = get_docuchkerr_inclhirange()
```

Fortran command syntax:

```
DOCUCHKERR_INCLHIRANGE = GET_DOCUCHKERR_INCLHIRANGE()
```

Where:

Integer DOCUCHKERR_INCLHIRANGE	value to signal if CON not within range, second value inclusive (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.8. GET_DOCUCHKERR_INCLLOWRANGE

Use this API to return the value to signal if CON not within range, first value inclusive.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_incllowrange = get_docuchkerr_incllowrange()
```

Fortran command syntax:

```
DOCUCHKERR_INCLLOWRANGE = GET_DOCUCHKERR_INCLLOWRANGE()
```

Where:

Integer DOCUCHKERR_INCL-LOWRANGE	value to signal if CON not within range, first value inclusive (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.9. GET_DOCUCHKERR_INCLUSIVERANGE

Use this API to return the value to signal if CON not within inclusive range.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_inclusiverange = get_docuchkerr_inclusiverange()
```

Fortran command syntax:

```
DOCUCHKERR_INCLUSIVERANGE = GET_DOCUCHKERR_INCLUSIVERANGE()
```

Where:

Integer DOCUCHKERR_INCLUSIVERANGE	value to signal if CON not within inclusive range (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.10. GET_DOCUCHKERR_LESSOREQ

Use this API to return the value to signal if CON less than or equal to test value.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_lessoreq = get_docuchkerr_lessoreq()
```

Fortran command syntax:

```
DOCUCHKERR_LESSOREQ = GET_DOCUCHKERR_LESSOREQ()
```

Where:

Integer DOCUCHKERR_LESSOREQ	value to signal if CON less than or equal to test value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.11. GET_DOCUCHKERR_LESSTHAN

Use this API to return the value to signal if CON less than test value.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_lessthan = get_docuchkerr_lessthan()
```

Fortran command syntax:

```
DOCUCHKERR_LESSTHAN = GET_DOCUCHKERR_LESSTHAN()
```

Where:

Integer DOCUCHKERR_LESSTHAN	value to signal if CON less than test value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.12. GET_DOCUCHKERR_NOTEQUAL

Use this API to return the value to signal if CON not equal to test value.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_notequal = get_docuchkerr_notequal()
```

Fortran command syntax:

```
DOCUCHKERR_NOTEQUAL = GET_DOCUCHKERR_NOTEQUAL()
```

Where:

Integer DOCUCHKERR_NOTEQUAL value to signal if CON not equal to test value (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.13. GET_DOCUCHKERR_NOTINCLHIRANGE

Use this API to return the value to signal if CON not outside range, second value inclusive.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_notinclhirange = get_docuchkerr_notinclhirange()
```

Fortran command syntax:

```
DOCUCHKERR_NOTINCLHIRANGE = GET_DOCUCHKERR_NOTINCLHIRANGE()
```

Where:

Integer DOCUCHKERR_NOTINCLHIRANGE	value to signal if CON not outside range, second value inclusive (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.14. GET_DOCUCHKERR_NOTINCLLOWRANGE

Use this API to return the value to signal if CON not outside range, first value inclusive.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_notincllowrange = get_docuchkerr_notincllowrange()
```

Fortran command syntax:

```
DOCUCHKERR_NOTINCLLOWRANGE = GET_DOCUCHKERR_NOTINCLLOWRANGE()
```

Where:

Integer DOCUCHKERR_NOTINCL-LOWRANGE	value to signal if CON not outside range, first value inclusive (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.15. GET_DOCUCHKERR_NOTINCLRANGE

Use this API to return the value to signal if CON not outside inclusive range.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_notinclrange = get_docuchkerr_notinclrange()
```

Fortran command syntax:

```
DOCUCHKERR_NOTINCLRANGE = GET_DOCUCHKERR_NOTINCLRANGE()
```

Where:

Integer DOCUCHKERR_NOTIN-CLRANGE	value to signal if CON not outside inclusive range (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.16. GET_DOCUCHKERR_NOTRANGE

Use this API to return the value to signal if CON not outside exclusive range.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_notrange = get_docuchkerr_notrange()
```

Fortran command syntax:

```
DOCUCHKERR_NOTRANGE = GET_DOCUCHKERR_NOTRANGE()
```

Where:

Integer DOCUCHKERR_NOTRANGE value to signal if CON not outside exclusive range (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.17. GET_DOCUCHKERR_RANGE

Use this API to return the value to signal if CON not within exclusive range.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkerr_range = get_docuchkerr_range()
```

Fortran command syntax:

```
DOCUCHKERR_RANGE = GET_DOCUCHKERR_RANGE()
```

Where:

Integer DOCUCHKERR_RANGE	value to signal if CON not within exclusive range (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.18. GET_MSGTYPE_ERROR

Use this API to return the severity value of "error" for the outputmodelmsgs routine, used for error reporting.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
msgtype_error = get_msgtype_error()
```

Fortran command syntax:

```
MSGTYPE_ERROR = GET_MSGTYPE_ERROR()
```

Where:

Integer MSGTYPE_ERROR	severity value of "error" for the outputmodelmsgs routine, used for error reporting (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.19. GET_MSGTYPE_INFO

Use this API to return the severity value of "informational" for the outputmodelmsgs routine, used for error reporting.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
msgtype_info = get_msgtype_info()
```

Fortran command syntax:

```
MSGTYPE_INFO = GET_MSGTYPE_INFO()
```

Where:

Integer MSGTYPE_INFO	severity value of "informational" for the outputmodelmsgs routine, used for error reporting (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.20. GET_MSGTYPE_WARN

Use this API to return the severity value of "warning" for the outputmodelmsgs routine, used for error reporting.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
msgtype_warn = get_msgtype_warn()
```

Fortran command syntax:

```
MSGTYPE_WARN = GET_MSGTYPE_WARN()
```

Where:

Integer MSGTYPE_WARN	severity value of "warning" for the outputmodelmsgs routine, used for error reporting (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.21. GET_W7PRG

Use this API to return the specify the progress virtual device.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
w7prg = get_w7prg()
```

Fortran command syntax:

```
W7PRG = GET_W7PRG()
```

Where:

Integer W7PRG	specify the progress virtual device (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.1.22. GET_W7RPT

Use this API to return the specify the report virtual device.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
w7rpt = get_w7rpt()
```

Fortran command syntax:

```
W7RPT = GET_W7RPT()
```

Where:

Integer W7RPT	specify the report virtual device (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2. Dynamics model value retrieval functions

This section describes the routines that can return the current values of internal variables to the caller. For those variables that are arrays, single values or array slices may be retrieved.

24.2.1. GET_ACBUS

Use this API to return the multi-terminal dc line converter ac bus array indices.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
acbus = get_acbus(cnvidx, indx)

aacbus = get_acbus(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
ACBUS = GET_ACBUS(CNVIDX, INDX)

AACBUS = GET_ACBUS(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer ACBUS	multi-terminal dc line converter ac bus array indices for the element indicated by indx1 (output).
Integer AACBUS ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter ac bus array indices; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.2. GET_ALFA

Use this API to return the two terminal dc line alpha.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
alfa = get_alfa(indx1)

aalfa = get_alfa(indx1, indx2)
```

Fortran command syntax:

```
ALFA = GET_ALFA(INDX1)

AALFA = GET_ALFA(INDX1, INDX2)
```

Where:

Real ALFA	two terminal dc line alpha for the element indicated by indx1 (output).
Real AALFA (INDX2-INDX1+1)	two terminal dc line alpha; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.3. GET_ALFMIN

Use this API to return the two terminal dc line maximum alpha.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
alfmin = get_alfmin(indx1)

aalfmin = get_alfmin(indx1, indx2)
```

Fortran command syntax:

```
ALFMIN = GET_ALFMIN(INDX1)

AALFMIN = GET_ALFMIN(INDX1, INDX2)
```

Where:

Real ALFMIN	two terminal dc line maximum alpha for the element indicated by indx1 (output).
Real AALFMIN (INDX2-INDX1+1)	two terminal dc line maximum alpha; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.4. GET_ALFMX

Use this API to return the two terminal dc line minimum alpha.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
alfmx = get_alfmx(indx1)

aalfmx = get_alfmx(indx1, indx2)
```

Fortran command syntax:

```
ALFMX = GET_ALFMX(INDX1)

AALFMX = GET_ALFMX(INDX1, INDX2)
```

Where:

Real ALFMX	two terminal dc line minimum alpha for the element indicated by indx1 (output).
Real AALFMX (INDX2-INDX1+1)	two terminal dc line minimum alpha; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.5. GET_ANGLE

Use this API to return the angle.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
angle = get_angle(indx1)

aangle = get_angle(indx1, indx2)
```

Fortran command syntax:

```
ANGLE = GET_ANGLE(INDX1)

AANGLE = GET_ANGLE(INDX1, INDX2)
```

Where:

Real ANGLE	angle for the element indicated by indx1 (output).
Real AANGLE (INDX2-INDX1+1)	angle; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	angle (input).
Integer INDX2	angle, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.6. GET_ANGMAX

Use this API to return the commutating resistance.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
angmax = get_angmax(cnvidx, indx)

aangmax = get_angmax(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
ANGMAX = GET_ANGMAX(CNVIDX, INDX)

AANGMAX = GET_ANGMAX(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real ANGMAX	commutating resistance for the element indicated by indx1 (output).
Real AANGMAX ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	commutating resistance; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.7. GET_ANGMIN

Use this API to return the actual tap ratio.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
angmin = get_angmin(cnvidx, indx)

aangmin = get_angmin(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
ANGMIN = GET_ANGMIN(CNVIDX, INDX)

AANGMIN = GET_ANGMIN(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real ANGMIN	actual tap ratio for the element indicated by indx1 (output).
Real AANGMIN ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	actual tap ratio; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.8. GET_B2S

Use this API to return the half of the line charging shunt.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
b2s = get_b2s(indx1)

ab2s = get_b2s(indx1, indx2)
```

Fortran command syntax:

```
B2S = GET_B2S(INDX1)

AB2S = GET_B2S(INDX1, INDX2)
```

Where:

Real B2S	half of the line charging shunt for the element indicated by indx1 (output).
Real AB2S (INDX2-INDX1+1)	half of the line charging shunt; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.9. GET_BASFRQ

Use this API to return the system base frequency (Hz).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
basfrq = get_basfrq()
```

Fortran command syntax:

```
BASFRQ = GET_BASFRQ()
```

Where:

Real BASFRQ	system base frequency (Hz) (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.10. GET_BASVLT

Use this API to return the bus base kV.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
basvlt = get_basvlt(indx1)

abasvlt = get_basvlt(indx1, indx2)
```

Fortran command syntax:

```
BASVLT = GET_BASVLT(INDX1)

ABASVLT = GET_BASVLT(INDX1, INDX2)
```

Where:

Real BASVLT	bus base kV for the element indicated by indx1 (output).
Real ABASVLT (INDX2-INDX1+1)	bus base kV; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.11. GET_BSFREQ

Use this API to return the bsfreq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
bsfreq = get_bsfreq(indx1)

absfreq = get_bsfreq(indx1, indx2)
```

Fortran command syntax:

```
BSFREQ = GET_BSFREQ(INDX1)

ABSFREQ = GET_BSFREQ(INDX1, INDX2)
```

Where:

Real BSFREQ	bsfreq for the element indicated by indx1 (output).
Real ABSFREQ (INDX2-INDX1+1)	bsfreq; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bsfreq (input).
Integer INDX2	bsfreq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.12. GET_BSWICH

Use this API to return the actual total shunt admittance presently switched on at the bus in MVA.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
bswich = get_bswich(indx1)

abswich = get_bswich(indx1, indx2)
```

Fortran command syntax:

```
BSWICH = GET_BSWICH(INDX1)

ABSWICH = GET_BSWICH(INDX1, INDX2)
```

Where:

Real BSWICH	actual total shunt admittance presently switched on at the bus in MVA for the element indicated by indx1 (output).
Real ABSWICH (INDX2-INDX1+1)	actual total shunt admittance presently switched on at the bus in MVA; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	switched shunt array index (input).
Integer INDX2	switched shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.13. GET_BUSNAM

Use this API to return the bus name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
busnam = get_busnam(indx1)

abusnam = get_busnam(indx1, indx2)
```

Fortran command syntax:

```
BUSNAM = GET_BUSNAM(INDX1)

ABUSNAM = GET_BUSNAM(INDX1, INDX2)
```

Where:

Character*12 BUSNAM	bus name for the element indicated by indx1 (output).
Character*12 ABUSNAM (INDX2-INDX1+1)	bus name; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.14. GET_CDGNFR

Use this API to return the distributed generation load quantities in pu of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
cdgnfr = get_cdgnfr(indx1)

acdgnfr = get_cdgnfr(indx1, indx2)
```

Fortran command syntax:

```
CDGNFR = GET_CDGNFR(INDX1)

ACDGNFR = GET_CDGNFR(INDX1, INDX2)
```

Where:

Complex CDGNFR	distributed generation load quantities in pu of system MVA for the element indicated by indx1 (output).
Complex ACDGNFR (INDX2-INDX1+1)	distributed generation load quantities in pu of system MVA; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	load array index (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.15. GET_CFDLFR

Use this API to return the complex (gross) constant power, current and admittance loads in p.u. of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
cfdlfr = get_cfdlfr(loadtype, indx)

acfdlfr = get_cfdlfr(loadtype, loadtype2, indx, indx2)
```

Fortran command syntax:

```
CFDLFR = GET_CFDLFR(LOADTYPE, INDX)

ACFDLFR = GET_CFDLFR(LOADTYPE, LOADTYPE2, INDX, INDX2)
```

Where:

Complex CFDLFR	complex (gross) constant power, current and admittance loads in p.u. of system MVA for the element indicated by indx1 (output).
Complex ACFDLFR ((LOADTYPE2-LOADTYPE+1)*(INDX2-INDX+1))	complex (gross) constant power, current and admittance loads in p.u. of system MVA; an array of values for the elements indicated by the indices supplied (output).
Integer LOADTYPE	load component: 1. constant power, 2. constant current, 3. constant impendance (input).
Integer LOADTYPE2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).
Integer INDX	load array index (input).
Integer INDX2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.16. GET_CHRICN

Use this API to return the used to stored character ICON (sized to as many as number of ICONs used by any model).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
chricn = get_chricn(indx1)

achricn = get_chricn(indx1, indx2)
```

Fortran command syntax:

```
CHRICN = GET_CHRICN(INDX1)

ACHRICN = GET_CHRICN(INDX1, INDX2)
```

Where:

Character*12 CHRICN	used to stored character ICON (sized to as many as number of ICONs used by any model) for the element indicated by indx1 (output).
Character*12 ACHRICN (INDX2-INDX1+1)	used to stored character ICON (sized to as many as number of ICONs used by any model); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	chricn (input).
Integer INDX2	chricn, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.17. GET_CLODFR

Use this API to return the complex (net) constant power, current and admittance loads in p.u. of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
clodfr = get_clodfr(loadtype, indx)

aclodfr = get_clodfr(loadtype, loadtype2, indx, indx2)
```

Fortran command syntax:

```
CLODFR = GET_CLODFR(LOADTYPE, INDX)

ACLODFR = GET_CLODFR(LOADTYPE, LOADTYPE2, INDX, INDX2)
```

Where:

Complex CLODFR	complex (net) constant power, current and admittance loads in p.u. of system MVA for the element indicated by indx1 (output).
Complex ACLODFR ((LOADTYPE2-LOADTYPE+1)*(INDX2-INDX+1))	complex (net) constant power, current and admittance loads in p.u. of system MVA; an array of values for the elements indicated by the indices supplied (output).
Integer LOADTYPE	load component: 1. constant power, 2. constant current, 3. constant impendance (input).
Integer LOADTYPE2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).
Integer INDX	load array index (input).
Integer INDX2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.18. GET_CNVANG

Use this API to return the multi-terminal dc line converter dc current injection.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
cnvang = get_cnvang(cnvidx, indx)

acnvang = get_cnvang(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
CNVANG = GET_CNVANG(CNVIDX, INDX)

ACNVANG = GET_CNVANG(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real CNVANG	multi-terminal dc line converter dc current injection for the element indicated by indx1 (output).
Real ACNVANG ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter dc current injection; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.19. GET_CNVCUR

Use this API to return the multi-terminal dc line converter minimum alpha/gamma angle.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
cnvcur = get_cnvcur(cnvidx, indx)

acnvcur = get_cnvcur(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
CNVCUR = GET_CNVCUR(CNVIDX, INDX)

ACNVCUR = GET_CNVCUR(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real CNVCUR	multi-terminal dc line converter minimum alpha/gamma angle for the element indicated by indx1 (output).
Real ACNVCUR ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter minimum alpha/gamma angle; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.20. GET_CNVTAP

Use this API to return the multi-terminal dc line converter power/current/voltage setpoint.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
cnvtap = get_cnvtap(cnvidx, indx)

acnvtap = get_cnvtap(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
CNVTAP = GET_CNVTAP(CNVIDX, INDX)

ACNVTAP = GET_CNVTAP(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real CNVTAP	multi-terminal dc line converter power/current/voltage setpoint for the element indicated by indx1 (output).
Real ACNVTAP ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter power/current/voltage setpoint; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.21. GET_CNVTYP

Use this API to return the multi-terminal dc line converter real power seen from ac network.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
cnvtyp = get_cnvtyp(cnvidx, indx)

acnvtyp = get_cnvtyp(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
CNVTYP = GET_CNVTYP(CNVIDX, INDX)

ACNVTYP = GET_CNVTYP(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer CNVTYP	multi-terminal dc line converter real power seen from ac network for the element indicated by indx1 (output).
Integer ACNVTYP ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter real power seen from ac network; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.22. GET_CON

Use this API to return the real (floating point) model data.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
con = get_con(indx1)

acon = get_con(indx1, indx2)
```

Fortran command syntax:

```
CON = GET_CON(INDX1)

ACON = GET_CON(INDX1, INDX2)
```

Where:

Real CON	real (floating point) model data for the element indicated by indx1 (output).
Real ACON (INDX2-INDX1+1)	real (floating point) model data; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	con (input).
Integer INDX2	con, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.23. GET_CON_DSCRPT

Use this API to return the con_dscrpt.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
con_dscrpt = get_con_dscrpt(indx1)

acon_dscrpt = get_con_dscrpt(indx1, indx2)
```

Fortran command syntax:

```
CON_DSCRPT = GET_CON_DSCRPT(INDX1)

ACON_DSCRPT = GET_CON_DSCRPT(INDX1, INDX2)
```

Where:

Character*144 CON_DSCRPT	con_dscrpt for the element indicated by indx1 (output).
Character*144 ACON_DSCRPT (INDX2-INDX1+1)	con_dscrpt; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	con_dscrpt (input).
Integer INDX2	con_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.24. GET_CURNT

Use this API to return the current injection at a bus.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
curnt = get_curnt(indx1)

acurnt = get_curnt(indx1, indx2)
```

Fortran command syntax:

```
CURNT = GET_CURNT(INDX1)

ACURNT = GET_CURNT(INDX1, INDX2)
```

Where:

Complex CURNT	current injection at a bus for the element indicated by indx1 (output).
Complex ACURNT (INDX2-INDX1+1)	current injection at a bus; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.25. GET_DC

Use this API to return the two terminal dc line current .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dc = get_dc(indx1)

adc = get_dc(indx1, indx2)
```

Fortran command syntax:

```
DC = GET_DC(INDX1)

ADC = GET_DC(INDX1, INDX2)
```

Where:

Real DC	two terminal dc line current for the element indicated by indx1 (output).
Real ADC (INDX2-INDX1+1)	two terminal dc line current ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.26. GET_DC2NAM

Use this API to return the two-terminal dc line name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dc2nam = get_dc2nam(indx1)

adc2nam = get_dc2nam(indx1, indx2)
```

Fortran command syntax:

```
DC2NAM = GET_DC2NAM(INDX1)

ADC2NAM = GET_DC2NAM(INDX1, INDX2)
```

Where:

Character*12 DC2NAM	two-terminal dc line name for the element indicated by indx1 (output).
Character*12 ADC2NAM (INDX2-INDX1+1)	two-terminal dc line name; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.27. GET_DC2SIG

Use this API to return the dc2sig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
dc2sig = get_dc2sig(one, two)

adc2sig = get_dc2sig(one, one2, two, two2)
```

Fortran command syntax:

```
DC2SIG = GET_DC2SIG(ONE, TWO)

ADC2SIG = GET_DC2SIG(ONE, ONE2, TWO, TWO2)
```

Where:

Real DC2SIG	dc2sig for the element indicated by indx1 (output).
Real ADC2SIG ((ONE2-ONE +1)*(TWO2-TWO+1))	dc2sig; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	dc2sig (input).
Integer ONE2	dc2sig, upper value of range (input).
Integer TWO	dc2sig (input).
Integer TWO2	dc2sig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.28. GET_DCIND

Use this API to return the multi-terminal dc line dc link inductance (millihenries).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dcind = get_dcind(linkidx, indx)

adcind = get_dcind(linkidx, linkidx2, indx, indx2)
```

Fortran command syntax:

```
DCIND = GET_DCIND(LINKIDX, INDX)

ADCIND = GET_DCIND(LINKIDX, LINKIDX2, INDX, INDX2)
```

Where:

Real DCIND	multi-terminal dc line dc link inductance (millihenries) for the element indicated by indx1 (output).
Real ADCIND ((LINKIDX2-LINKIDX +1)*(INDX2-INDX+1))	multi-terminal dc line dc link inductance (millihenries); an array of values for the elements indicated by the indices supplied (output).
Integer LINKIDX	multi-terminal dc line dc link index (1:MXDCKT) (input).
Integer LINKIDX2	multi-terminal dc line dc link index (1:MXDCKT), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line dc link index (1:MXDCKT), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.29. GET_DCMSIG

Use this API to return the dcmsig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
dcmsig = get_dcmsig(one, two)

adcmsig = get_dcmsig(one, one2, two, two2)
```

Fortran command syntax:

```
DCMSIG = GET_DCMSIG(ONE, TWO)

ADCMSIG = GET_DCMSIG(ONE, ONE2, TWO, TWO2)
```

Where:

Real DCMSIG	dcmsig for the element indicated by indx1 (output).
Real ADCMSIG ((ONE2-ONE +1)*(TWO2-TWO+1))	dcmsig; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	dcmsig (input).
Integer ONE2	dcmsig, upper value of range (input).
Integer TWO	dcmsig (input).
Integer TWO2	dcmsig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.30. GET_DCNAME

Use this API to return the multi-terminal dc line dc bus name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dcname = get_dcname(dcbusidx, indx)

adcname = get_dcname(dcbusidx, dcbusidx2, indx, indx2)
```

Fortran command syntax:

```
DCNAME = GET_DCNAME(DCBUSIDX, INDX)

ADCNAME = GET_DCNAME(DCBUSIDX, DCBUSIDX2, INDX, INDX2)
```

Where:

Character*12 DCNAME	multi-terminal dc line dc bus name for the element indicated by indx1 (output).
Character*12 ADCNAME ((DCBUSIDX2-DCBUSIDX +1)*(INDX2-INDX+1))	multi-terminal dc line dc bus name; an array of values for the elements indicated by the indices supplied (output).
Integer DCBUSIDX	multi-terminal dc line dc bus index (1:MXDCBS) (input).
Integer DCBUSIDX2	multi-terminal dc line dc bus index (1:MXDCBS), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line dc bus index (1:MXDCBS), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.31. GET_DCPF

Use this API to return the multi-terminal dc line converter area number.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dcpf = get_dcpf(cnvidx, indx)

adcpf = get_dcpf(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
DCPF = GET_DCPF(CNVIDX, INDX)

ADCPF = GET_DCPF(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real DCPF	multi-terminal dc line converter area number for the element indicated by indx1 (output).
Real ADCPF ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter area number; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.32. GET_DCRES

Use this API to return the two terminal dc line converter dc link resistance (ohms).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dcres = get_dcres(linkidx, indx)

adcres = get_dcres(linkidx, linkidx2, indx, indx2)
```

Fortran command syntax:

```
DCRES = GET_DCRES(LINKIDX, INDX)

ADCRES = GET_DCRES(LINKIDX, LINKIDX2, INDX, INDX2)
```

Where:

Real DCRES	two terminal dc line converter dc link resistance (ohms) for the element indicated by indx1 (output).
Real ADCRES ((LINKIDX2-LINKIDX +1)*(INDX2-INDX+1))	two terminal dc line converter dc link resistance (ohms); an array of values for the elements indicated by the indices supplied (output).
Integer LINKIDX	multi-terminal dc line dc link index (1:MXDCKT) (input).
Integer LINKIDX2	multi-terminal dc line dc link index (1:MXDCKT), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line dc link index (1:MXDCKT), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.33. GET_DCSETP

Use this API to return the multi-terminal dc line converter maximum alpha/gamma angle.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dcsetp = get_dcsetp(cnvidx, indx)

adcsetp = get_dcsetp(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
DCSETP = GET_DCSETP(CNVIDX, INDX)

ADCSETP = GET_DCSETP(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real DCSETP	multi-terminal dc line converter maximum alpha/gamma angle for the element indicated by indx1 (output).
Real ADCSETP ((CNVIDX2-CN-VIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter maximum alpha/gamma angle; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.34. GET_DELT

Use this API to return the DELT.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
delt = get_delt()
```

Fortran command syntax:

```
DELT = GET_DELT()
```

Where:

Real DELT	DELT (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.35. GET_DELTI

Use this API to return the two terminal dc line margin entered in per unit of desired dc power or current.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
delti = get_delti(indx1)

adelti = get_delti(indx1, indx2)
```

Fortran command syntax:

```
DELTI = GET_DELTI(INDX1)

ADELTI = GET_DELTI(INDX1, INDX2)
```

Where:

Real DELTI	two terminal dc line margin entered in per unit of desired dc power or current for the element indicated by indx1 (output).
Real ADELTI (INDX2-INDX1+1)	two terminal dc line margin entered in per unit of desired dc power or current; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.36. GET_DERMWTHRESH

Use this API to return the DERMWThresh.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dermwthresh = get_dermwthresh()
```

Fortran command syntax:

```
DERMWTHRESH = GET_DERMWTHRESH()
```

Where:

Real DERMWTHRESH	DERMWThresh (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.37. GET_DERVTTHRESH

Use this API to return the DERVtThresh.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dervtthresh = get_dervtthresh()
```

Fortran command syntax:

```
DERVTTHRESH = GET_DERVTTHRESH()
```

Where:

Real DERVTTHRESH	DERVtThresh (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.38. GET_DSTATE

Use this API to return the derivative of state variable.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dstate = get_dstate(indx1)

adstate = get_dstate(indx1, indx2)
```

Fortran command syntax:

```
DSTATE = GET_DSTATE(INDX1)

ADSTATE = GET_DSTATE(INDX1, INDX2)
```

Where:

Real DSTATE	derivative of state variable for the element indicated by indx1 (output).
Real ADSTATE (INDX2-INDX1+1)	derivative of state variable; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dstate (input).
Integer INDX2	dstate, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.39. GET_EBASE

Use this API to return the multi-terminal dc line converter dc bus voltage.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ebase = get_ebase(cnvidx, indx)

aebase = get_ebase(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
EBASE = GET_EBASE(CNVIDX, INDX)

AEBASE = GET_EBASE(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real EBASE	multi-terminal dc line converter dc bus voltage for the element indicated by indx1 (output).
Real AEBASE ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter dc bus voltage; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.40. GET_EBASEI

Use this API to return the two terminal dc line inverter primary base ac voltage in kV.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ebasei = get_ebasei(indx1)

aebasei = get_ebasei(indx1, indx2)
```

Fortran command syntax:

```
EBASEI = GET_EBASEI(INDX1)

AEBASEI = GET_EBASEI(INDX1, INDX2)
```

Where:

Real EBASEI	two terminal dc line inverter primary base ac voltage in kV for the element indicated by indx1 (output).
Real AEBASEI (INDX2-INDX1+1)	two terminal dc line inverter primary base ac voltage in kV; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.41. GET_EBASER

Use this API to return the two terminal dc line rectifier primary base ac voltage in kV.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ebaser = get_ebaser(indx1)

aebaser = get_ebaser(indx1, indx2)
```

Fortran command syntax:

```
EBASER = GET_EBASER(INDX1)

AEBASER = GET_EBASER(INDX1, INDX2)
```

Where:

Real EBASER	two terminal dc line rectifier primary base ac voltage in kV for the element indicated by indx1 (output).
Real AEBASER (INDX2-INDX1+1)	two terminal dc line rectifier primary base ac voltage in kV; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.42. GET_ECOMP

Use this API to return the ecomp.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ecomp = get_ecomp(indx1)

aecomp = get_ecomp(indx1, indx2)
```

Fortran command syntax:

```
ECOMP = GET_ECOMP(INDX1)

AECOMP = GET_ECOMP(INDX1, INDX2)
```

Where:

Real ECOMP	ecomp for the element indicated by indx1 (output).
Real AECOMP (INDX2-INDX1+1)	ecomp; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	ecomp (input).
Integer INDX2	ecomp, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.43. GET_EFD

Use this API to return the efd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
efd = get_efd(indx1)

aefd = get_efd(indx1, indx2)
```

Fortran command syntax:

```
EFD = GET_EFD(INDX1)

AEFD = GET_EFD(INDX1, INDX2)
```

Where:

Real EFD	efd for the element indicated by indx1 (output).
Real AEFD (INDX2-INDX1+1)	efd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	efd (input).
Integer INDX2	efd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.44. GET_ETERM

Use this API to return the eterm.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
eterm = get_eterm(indx1)

aeterm = get_eterm(indx1, indx2)
```

Fortran command syntax:

```
ETERM = GET_ETERM(INDX1)

AETERM = GET_ETERM(INDX1, INDX2)
```

Where:

Real ETERM	eterm for the element indicated by indx1 (output).
Real AETERM (INDX2-INDX1+1)	eterm; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	eterm (input).
Integer INDX2	eterm, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.45. GET_FCNAME

Use this API to return the FACTS device name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fcname = get_fcname(indx1)

afcname = get_fcname(indx1, indx2)
```

Fortran command syntax:

```
FCNAME = GET_FCNAME(INDX1)

AFCNAME = GET_FCNAME(INDX1, INDX2)
```

Where:

Character*12 FCNAME	FACTS device name for the element indicated by indx1 (output).
Character*12 AFCNAME (INDX2-INDX1+1)	FACTS device name; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.46. GET_FCREM

Use this API to return the FACTS device remotely regulated bus.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fcrem = get_fcrem(indx1)

afcrem = get_fcrem(indx1, indx2)
```

Fortran command syntax:

```
FCREM = GET_FCREM(INDX1)

AFCREM = GET_FCREM(INDX1, INDX2)
```

Where:

Integer FCREM	FACTS device remotely regulated bus for the element indicated by indx1 (output).
Integer AFCREM (INDX2-INDX1+1)	FACTS device remotely regulated bus; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.47. GET_FCSBUS

Use this API to return the FACTS device sending bus index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fcsbus = get_fcsbus(indx1)

afcsbus = get_fcsbus(indx1, indx2)
```

Fortran command syntax:

```
FCSBUS = GET_FCSBUS(INDX1)

AFCSBUS = GET_FCSBUS(INDX1, INDX2)
```

Where:

Integer FCSBUS	FACTS device sending bus index for the element indicated by indx1 (output).
Integer AFCSBUS (INDX2-INDX1+1)	FACTS device sending bus index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.48. GET_FCSHMX

Use this API to return the FACTS device sending end maximum shunt current.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
fcshmx = get_fcshmx(indx1)

afcshmx = get_fcshmx(indx1, indx2)
```

Fortran command syntax:

```
FCSHMX = GET_FCSHMX(INDX1)

AFCSHMX = GET_FCSHMX(INDX1, INDX2)
```

Where:

Real FCSHMX	FACTS device sending end maximum shunt current for the element indicated by indx1 (output).
Real AFCSHMX (INDX2-INDX1+1)	FACTS device sending end maximum shunt current; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.49. GET_FCTBUS

Use this API to return the FACTS device terminal bus index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fctbus = get_fctbus(indx1)

afctbus = get_fctbus(indx1, indx2)
```

Fortran command syntax:

```
FCTBUS = GET_FCTBUS(INDX1)

AFCTBUS = GET_FCTBUS(INDX1, INDX2)
```

Where:

Integer FCTBUS	FACTS device terminal bus index for the element indicated by indx1 (output).
Integer AFCTBUS (INDX2-INDX1+1)	FACTS device terminal bus index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.50. GET_FCTREF

Use this API to return the fctref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fctref = get_fctref(indx1)

afctref = get_fctref(indx1, indx2)
```

Fortran command syntax:

```
FCTREF = GET_FCTREF(INDX1)

AFCTREF = GET_FCTREF(INDX1, INDX2)
```

Where:

Real FCTREF	fctref for the element indicated by indx1 (output).
Real AFCTREF (INDX2-INDX1+1)	fctref; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.51. GET_FCTSIG

Use this API to return the fctsig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
fctsig = get_fctsig(one, two)

afctsig = get_fctsig(one, one2, two, two2)
```

Fortran command syntax:

```
FCTSIG = GET_FCTSIG(ONE, TWO)

AFCTSIG = GET_FCTSIG(ONE, ONE2, TWO, TWO2)
```

Where:

Real FCTSIG	fctsig for the element indicated by indx1 (output).
Real AFCTSIG ((ONE2-ONE +1)*(TWO2-TWO+1))	fctsig; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	fctsig (input).
Integer ONE2	fctsig, upper value of range (input).
Integer TWO	fctsig (input).
Integer TWO2	fctsig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.52. GET_FCVSET

Use this API to return the FACTS device sending end bus voltage set point.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
fcvset = get_fcvset(indx1)

afcvset = get_fcvset(indx1, indx2)
```

Fortran command syntax:

```
FCVSET = GET_FCVSET(INDX1)

AFCVSET = GET_FCVSET(INDX1, INDX2)
```

Where:

Real FCVSET	FACTS device sending end bus voltage set point for the element indicated by indx1 (output).
Real AFCVSET (INDX2-INDX1+1)	FACTS device sending end bus voltage set point; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.53. GET_FQSHNT

Use this API to return the FACTS sending reactive power.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fqshnt = get_fqshnt(indx1)

afqshnt = get_fqshnt(indx1, indx2)
```

Fortran command syntax:

```
FQSHNT = GET_FQSHNT(INDX1)

AFQSHNT = GET_FQSHNT(INDX1, INDX2)
```

Where:

Real FQSHNT	FACTS sending reactive power for the element indicated by indx1 (output).
Real AFQSHNT (INDX2-INDX1+1)	FACTS sending reactive power; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.54. GET_FRELVTHRESH

Use this API to return the FRelVThresh.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
frelvthresh = get_frelvthresh()
```

Fortran command syntax:

```
FRELVTHRESH = GET_FRELVTHRESH()
```

Where:

Real FRELVTHRESH	FRelVThresh (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.55. GET_FREQS

Use this API to return the FREQS.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
freqs = get_freqs()
```

Fortran command syntax:

```
FREQS = GET_FREQS()
```

Where:

Logical FREQS	FREQS (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.56. GET_GAMA

Use this API to return the two terminal dc line gamma.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gama = get_gama(indx1)

agama = get_gama(indx1, indx2)
```

Fortran command syntax:

```
GAMA = GET_GAMA(INDX1)

AGAMA = GET_GAMA(INDX1, INDX2)
```

Where:

Real GAMA	two terminal dc line gamma for the element indicated by indx1 (output).
Real AGAMA (INDX2-INDX1+1)	two terminal dc line gamma; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.57. GET_GAMMIN

Use this API to return the two terminal dc line minimum gamma.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gammin = get_gammin(indx1)

agammin = get_gammin(indx1, indx2)
```

Fortran command syntax:

```
GAMMIN = GET_GAMMIN(INDX1)

AGAMMIN = GET_GAMMIN(INDX1, INDX2)
```

Where:

Real GAMMIN	two terminal dc line minimum gamma for the element indicated by indx1 (output).
Real AGAMMIN (INDX2-INDX1+1)	two terminal dc line minimum gamma; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.58. GET_GAMMX

Use this API to return the two terminal dc line maximum gamma.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gammx = get_gammx(indx1)

agammx = get_gammx(indx1, indx2)
```

Fortran command syntax:

```
GAMMX = GET_GAMMX(INDX1)

AGAMMX = GET_GAMMX(INDX1, INDX2)
```

Where:

Real GAMMX	two terminal dc line maximum gamma for the element indicated by indx1 (output).
Real AGAMMX (INDX2-INDX1+1)	two terminal dc line maximum gamma; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.59. GET_GB

Use this API to return the negative reciprocal of the actual branch or transformer impedance in p.u. of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gb = get_gb(indx1)

agb = get_gb(indx1, indx2)
```

Fortran command syntax:

```
GB = GET_GB(INDX1)

AGB = GET_GB(INDX1, INDX2)
```

Where:

Complex GB	negative reciprocal of the actual branch or transformer impedance in p.u. of system MVA for the element indicated by indx1 (output).
Complex AGB (INDX2-INDX1+1)	negative reciprocal of the actual branch or transformer impedance in p.u. of system MVA; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.60. GET_GBLINE

Use this API to return the negative reciprocal of the nominal ac line impedance.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gbline = get_gbline(end, indx)

agbline = get_gbline(end, end2, indx, indx2)
```

Fortran command syntax:

```
GBLINE = GET_GBLINE(END, INDX)

AGBLINE = GET_GBLINE(END, END2, INDX, INDX2)
```

Where:

Complex GBLINE	negative reciprocal of the nominal ac line impedance for the element indicated by indx1 (output).
Complex AGBLINE ((END2-END +1)*(INDX2-INDX+1))	negative reciprocal of the nominal ac line impedance; an array of values for the elements indicated by the indices supplied (output).
Integer END	branch end index: 1 = from bus, 2 = to bus (input).
Integer END2	branch end index: 1 = from bus, 2 = to bus, upper value of range (input).
Integer INDX	branch array index (input).
Integer INDX2	branch end index: 1 = from bus, 2 = to bus, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.61. GET_GENTAP

Use this API to return the machine transformer tap.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gentap = get_gentap(indx1)

agentap = get_gentap(indx1, indx2)
```

Fortran command syntax:

```
GENTAP = GET_GENTAP(INDX1)

AGENTAP = GET_GENTAP(INDX1, INDX2)
```

Where:

Real GENTAP	machine transformer tap for the element indicated by indx1 (output).
Real AGENTAP (INDX2-INDX1+1)	machine transformer tap; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.62. GET_GREF

Use this API to return the gref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gref = get_gref(indx1)

agref = get_gref(indx1, indx2)
```

Fortran command syntax:

```
GREF = GET_GREF(INDX1)

AGREF = GET_GREF(INDX1, INDX2)
```

Where:

Real GREF	gref for the element indicated by indx1 (output).
Real AGREF (INDX2-INDX1+1)	gref; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	gref (input).
Integer INDX2	gref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.63. GET_ICOM

Use this API to return the branch status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
icom = get_icom(indx1)

aicom = get_icom(indx1, indx2)
```

Fortran command syntax:

```
ICOM = GET_ICOM(INDX1)

AICOM = GET_ICOM(INDX1, INDX2)
```

Where:

Integer ICOM	branch status for the element indicated by indx1 (output).
Integer AICOM (INDX2-INDX1+1)	branch status; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.64. GET_ICON

Use this API to return the Integer constants.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
icon = get_icon(indx1)

aicon = get_icon(indx1, indx2)
```

Fortran command syntax:

```
ICON = GET_ICON(INDX1)

AICON = GET_ICON(INDX1, INDX2)
```

Where:

Integer ICON	Integer constants for the element indicated by indx1 (output).
Integer AICON (INDX2-INDX1+1)	Integer constants; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	icon (input).
Integer INDX2	icon, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.65. GET_ICON_DSCRPT

Use this API to return the icon_dscrpt.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
icon_dscrpt = get_icon_dscrpt(indx1)

aicon_dscrpt = get_icon_dscrpt(indx1, indx2)
```

Fortran command syntax:

```
ICON_DSCRPT = GET_ICON_DSCRPT(INDX1)

AICON_DSCRPT = GET_ICON_DSCRPT(INDX1, INDX2)
```

Where:

Character*144 ICON_DSCRPT	icon_dscrpt for the element indicated by indx1 (output).
Character*144 AICON_DSCRPT (INDX2-INDX1+1)	icon_dscrpt; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	icon_dscrpt (input).
Integer INDX2	icon_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.66. GET_IFLAG

Use this API to return the IFLAG.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
iflag = get_iflag()
```

Fortran command syntax:

```
IFLAG = GET_IFLAG()
```

Where:

Logical IFLAG	IFLAG (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.67. GET_IMARYS

Use this API to return the imarys.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
imarys = get_imarys(one, two)

aimarys = get_imarys(one, one2, two, two2)
```

Fortran command syntax:

```
IMARYS = GET_IMARYS(ONE, TWO)

AIMARYS = GET_IMARYS(ONE, ONE2, TWO, TWO2)
```

Where:

Real IMARYS	imarys for the element indicated by indx1 (output).
Real AIMARYS ((ONE2-ONE +1)*(TWO2-TWO+1))	imarys; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	imarys (input).
Integer ONE2	imarys, upper value of range (input).
Integer TWO	imarys (input).
Integer TWO2	imarys, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.68. GET_IMCCNX

Use this API to return the imccnx.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
imccnx = get_imccnx(indx1)

aimccnx = get_imccnx(indx1, indx2)
```

Fortran command syntax:

```
IMCCNX = GET_IMCCNX(INDX1)

AIMCCNX = GET_IMCCNX(INDX1, INDX2)
```

Where:

Integer IMCCNX	imccnx for the element indicated by indx1 (output).
Integer AIMCCNX (INDX2-INDX1+1)	imccnx; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	imccnx (input).
Integer INDX2	imccnx, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.69. GET_IMELTRQ

Use this API to return the imeltrq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
imeltrq = get_imeltrq(indx1)

aimeltrq = get_imeltrq(indx1, indx2)
```

Fortran command syntax:

```
IMELTRQ = GET_IMELTRQ(INDX1)

AIMELTRQ = GET_IMELTRQ(INDX1, INDX2)
```

Where:

Real IMELTRQ	imeltrq for the element indicated by indx1 (output).
Real AIMELTRQ (INDX2-INDX1+1)	imeltrq; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	imeltrq (input).
Integer INDX2	imeltrq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.70. GET_IMLDTRQ

Use this API to return the imldtrq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
imldtrq = get_imldtrq(indx1)

aimldtrq = get_imldtrq(indx1, indx2)
```

Fortran command syntax:

```
IMLDTRQ = GET_IMLDTRQ(INDX1)

AIMLDTRQ = GET_IMLDTRQ(INDX1, INDX2)
```

Where:

Real IMLDTRQ	imldtrq for the element indicated by indx1 (output).
Real AIMLDTRQ (INDX2-INDX1+1)	imldtrq; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	imldtrq (input).
Integer INDX2	imldtrq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.71. GET_IMSPARE

Use this API to return the imspare.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
imspare = get_imspare(indx1)

aimspare = get_imspare(indx1, indx2)
```

Fortran command syntax:

```
IMSPARE = GET_IMSPARE(INDX1)

AIMSPARE = GET_IMSPARE(INDX1, INDX2)
```

Where:

Real IMSPARE	imspare for the element indicated by indx1 (output).
Real AIMSPARE (INDX2-INDX1+1)	imspare; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	imspare (input).
Integer INDX2	imspare, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.72. GET_IMTPLOD

Use this API to return the imtplod.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
imtplod = get_imtplod(indx1)

aimtplod = get_imtplod(indx1, indx2)
```

Fortran command syntax:

```
IMTPLOD = GET_IMTPLOD(INDX1)

AIMTPLOD = GET_IMTPLOD(INDX1, INDX2)
```

Where:

Real IMTPLOD	imtplod for the element indicated by indx1 (output).
Real AIMTPLOD (INDX2-INDX1+1)	imtplod; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	imtplod (input).
Integer INDX2	imtplod, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.73. GET_IMTQLOD

Use this API to return the imtqlod.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
imtqlod = get_imtqlod(indx1)

aimtqlod = get_imtqlod(indx1, indx2)
```

Fortran command syntax:

```
IMTQLOD = GET_IMTQLOD(INDX1)

AIMTQLOD = GET_IMTQLOD(INDX1, INDX2)
```

Where:

Real IMTQLOD	imtqlod for the element indicated by indx1 (output).
Real AIMTQLOD (INDX2-INDX1+1)	imtqlod; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	imtqlod (input).
Integer INDX2	imtqlod, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.74. GET_INDX3W

Use this API to return the the 3-winding transformer index of a 2-winding transformer record used to model one of the windings. Will be 0 if an actual 2-winding transformer.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
indx3w = get_indx3w(indx1)

aindx3w = get_indx3w(indx1, indx2)
```

Fortran command syntax:

```
INDX3W = GET_INDX3W(INDX1)

AINDX3W = GET_INDX3W(INDX1, INDX2)
```

Where:

Integer INDX3W	the 3-winding transformer index of a 2-winding transformer record used to model one of the windings. Will be 0 if an actual 2-winding transformer for the element indicated by indx1 (output).
Integer AINDX3W (INDX2-INDX1+1)	the 3-winding transformer index of a 2-winding transformer record used to model one of the windings. Will be 0 if an actual 2-winding transformer; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.75. GET_INTICN

Use this API to return the scartch memory location for storing integer data (sized to as many as number of ICONs used by any model).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
inticn = get_inticn(indx1)

ainticn = get_inticn(indx1, indx2)
```

Fortran command syntax:

```
INTICN = GET_INTICN(INDX1)

AINTICN = GET_INTICN(INDX1, INDX2)
```

Where:

Integer INTICN	scartch memory location for storing integer data (sized to as many as number of ICONs used by any model) for the element indicated by indx1 (output).
Integer AINTICN (INDX2-INDX1+1)	scartch memory location for storing integer data (sized to as many as number of ICONs used by any model); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	inticn (input).
Integer INDX2	inticn, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.76. GET_IPI

Use this API to return the two terminal dc line inverter converter bus index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ipi = get_ipi(indx1)

aipi = get_ipi(indx1, indx2)
```

Fortran command syntax:

```
IPI = GET_IPI(INDX1)

AIPI = GET_IPI(INDX1, INDX2)
```

Where:

Integer IPI	two terminal dc line inverter converter bus index for the element indicated by indx1 (output).
Integer AIPI (INDX2-INDX1+1)	two terminal dc line inverter converter bus index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.77. GET_IPR

Use this API to return the two terminal dc line rectifier converter bus index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ipr = get_ipr(indx1)

aipr = get_ipr(indx1, indx2)
```

Fortran command syntax:

```
IPR = GET_IPR(INDX1)

AIPR = GET_IPR(INDX1, INDX2)
```

Where:

Integer IPR	two terminal dc line rectifier converter bus index for the element indicated by indx1 (output).
Integer AIPR (INDX2-INDX1+1)	two terminal dc line rectifier converter bus index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.78. GET_IPRT

Use this API to return the current report device unit number.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
iprt = get_iprt()
```

Fortran command syntax:

```
IPRT = GET_IPRT()
```

Where:

Integer IPRT	current report device unit number (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.79. GET_ISORCE

Use this API to return the machine complex Norton current source (p.u. of system MVA) .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
isorce = get_isorce(indx1)

aisorce = get_isorce(indx1, indx2)
```

Fortran command syntax:

```
ISORCE = GET_ISORCE(INDX1)

AISORCE = GET_ISORCE(INDX1, INDX2)
```

Where:

Complex ISORCE	machine complex Norton current source (p.u. of system MVA) for the element indicated by indx1 (output).
Complex AISORCE (INDX2-INDX1+1)	machine complex Norton current source (p.u. of system MVA) ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.80. GET_ITER

Use this API to return the number of dynamic simulation iterations.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
iter = get_iter()
```

Fortran command syntax:

```
ITER = GET_ITER()
```

Where:

Integer ITER	number of dynamic simulation iterations (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.81. GET_ITMXDS

Use this API to return the ITMXDS.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
itmxds = get_itmxds()
```

Fortran command syntax:

```
ITMXDS = GET_ITMXDS()
```

Where:

Integer ITMXDS	ITMXDS (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.82. GET_JCODE

Use this API to return the bus type code.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
jcode = get_jcode(indx1)

ajcode = get_jcode(indx1, indx2)
```

Fortran command syntax:

```
JCODE = GET_JCODE(INDX1)

AJCODE = GET_JCODE(INDX1, INDX2)
```

Where:

Integer JCODE	bus type code for the element indicated by indx1 (output).
Integer AJCODE (INDX2-INDX1+1)	bus type code; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.83. GET_KNUM

Use this API to return the single entry branch index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
knum = get_knum(indx1)

aknum = get_knum(indx1, indx2)
```

Fortran command syntax:

```
KNUM = GET_KNUM(INDX1)

AKNUM = GET_KNUM(INDX1, INDX2)
```

Where:

Integer KNUM	single entry branch index for the element indicated by indx1 (output).
Integer AKNUM (INDX2-INDX1+1)	single entry branch index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	double-entry branch index (input).
Integer INDX2	double-entry branch index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.84. GET_KPAUSE

Use this API to return the KPAUSE.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
kpause = get_kpause()
```

Fortran command syntax:

```
KPAUSE = GET_KPAUSE()
```

Where:

Integer KPAUSE	KPAUSE (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.85. GET_KTRIP

Use this API to return the KTRIP.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ktrip = get_ktrip()
```

Fortran command syntax:

```
KTRIP = GET_KTRIP()
```

Where:

Integer KTRIP	KTRIP (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.86. GET_LCREF

Use this API to return the lcref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lcref = get_lcref(indx1)

alcref = get_lcref(indx1, indx2)
```

Fortran command syntax:

```
LCREF = GET_LCREF(INDX1)

ALCREF = GET_LCREF(INDX1, INDX2)
```

Where:

Real LCREF	lcref for the element indicated by indx1 (output).
Real ALCREF (INDX2-INDX1+1)	lcref; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	lcref (input).
Integer INDX2	lcref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.87. GET_LDAREA

Use this API to return the area array index for load.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ldarea = get_ldarea(indx1)

aldarea = get_ldarea(indx1, indx2)
```

Fortran command syntax:

```
LDAREA = GET_LDAREA(INDX1)

ALDAREA = GET_LDAREA(INDX1, INDX2)
```

Where:

Integer LDAREA	area array index for load for the element indicated by indx1 (output).
Integer ALDAREA (INDX2-INDX1+1)	area array index for load; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	load array index (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.88. GET_LDDGNFL

Use this API to return the DER status on the load record.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lddgnfl = get_lddgnfl(indx1)

alddgnfl = get_lddgnfl(indx1, indx2)
```

Fortran command syntax:

```
LDDGNFL = GET_LDDGNFL(INDX1)

ALDDGNFL = GET_LDDGNFL(INDX1, INDX2)
```

Where:

Integer LDDGNFL	DER status on the load record for the element indicated by indx1 (output).
Integer ALDDGNFL (INDX2-INDX1+1)	DER status on the load record; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	load array index (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.89. GET_LDOWNR

Use this API to return the owner array index for load.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ldownr = get_ldownr(indx1)

aldownr = get_ldownr(indx1, indx2)
```

Fortran command syntax:

```
LDOWNR = GET_LDOWNR(INDX1)

ALDOWNR = GET_LDOWNR(INDX1, INDX2)
```

Where:

Integer LDOWNR	owner array index for load for the element indicated by indx1 (output).
Integer ALDOWNR (INDX2-INDX1+1)	owner array index for load; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	load array index (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.90. GET_LDSTAT

Use this API to return the load status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ldstat = get_ldstat(indx1)

aldstat = get_ldstat(indx1, indx2)
```

Fortran command syntax:

```
LDSTAT = GET_LDSTAT(INDX1)

ALDSTAT = GET_LDSTAT(INDX1, INDX2)
```

Where:

Integer LDSTAT	load status for the element indicated by indx1 (output).
Integer ALDSTAT (INDX2-INDX1+1)	load status; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	load array index (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.91. GET_LDSTR2

Use this API to return the ldstr2.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ldstr2 = get_ldstr2(one, two)

aldstr2 = get_ldstr2(one, one2, two, two2)
```

Fortran command syntax:

```
LDSTR2 = GET_LDSTR2(ONE, TWO)

ALDSTR2 = GET_LDSTR2(ONE, ONE2, TWO, TWO2)
```

Where:

Integer LDSTR2	ldstr2 for the element indicated by indx1 (output).
Integer ALDSTR2 ((ONE2-ONE +1)*(TWO2-TWO+1))	ldstr2; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	ldstr2 (input).
Integer ONE2	ldstr2, upper value of range (input).
Integer TWO	(input).
Integer TWO2	ldstr2, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.92. GET_LDSTRT

Use this API to return the ldstrt.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ldstrt = get_ldstrt(one, two)

aldstrt = get_ldstrt(one, one2, two, two2)
```

Fortran command syntax:

```
LDSTRT = GET_LDSTRT(ONE, TWO)

ALDSTRT = GET_LDSTRT(ONE, ONE2, TWO, TWO2)
```

Where:

Integer LDSTRT	ldstrt for the element indicated by indx1 (output).
Integer ALDSTRT ((ONE2-ONE +1)*(TWO2-TWO+1))	ldstrt; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	ldstrt (input).
Integer ONE2	ldstrt, upper value of range (input).
Integer TWO	(input).
Integer TWO2	ldstrt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.93. GET_LDZONE

Use this API to return the zone array index for load.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ldzone = get_ldzone(indx1)

aldzone = get_ldzone(indx1, indx2)
```

Fortran command syntax:

```
LDZONE = GET_LDZONE(INDX1)

ALDZONE = GET_LDZONE(INDX1, INDX2)
```

Where:

Integer LDZONE	zone array index for load for the element indicated by indx1 (output).
Integer ALDZONE (INDX2-INDX1+1)	zone array index for load; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	load array index (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.94. GET_LOADID

Use this API to return the load id.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
loadid = get_loadid(indx1)

aloadid = get_loadid(indx1, indx2)
```

Fortran command syntax:

```
LOADID = GET_LOADID(INDX1)

ALOADID = GET_LOADID(INDX1, INDX2)
```

Where:

Character*2 LOADID	load id for the element indicated by indx1 (output).
Character*2 ALOADID (INDX2-INDX1+1)	load id; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	load array index (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.95. GET_LODBKVTHRESH

Use this API to return the LodBkVThresh.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lodbkvthresh = get_lodbkvthresh()
```

Fortran command syntax:

```
LODBKVTHRESH = GET_LODBKVTHRESH()
```

Where:

Real LODBKVTHRESH	LodBkVThresh (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.96. GET_LODID

Use this API to return the LODID.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lodid = get_lodid()
```

Fortran command syntax:

```
LODID = GET_LODID()
```

Where:

Character*2 LODID	LODID (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.97. GET_LODMWTHRESH

Use this API to return the LodMWThresh.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lodmwthresh = get_lodmwthresh()
```

Fortran command syntax:

```
LODMWTHRESH = GET_LODMWTHRESH()
```

Where:

Real LODMWTHRESH	LodMWThresh (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.98. GET_LODPQTHRESH

Use this API to return the LodPQThresh.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lodpqthresh = get_lodpqthresh()
```

Fortran command syntax:

```
LODPQTHRESH = GET_LODPQTHRESH()
```

Where:

Real LODPQTHRESH	LodPQThresh (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.99. GET_LODVTTHRESH

Use this API to return the LodVtThresh.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lodvtthresh = get_lodvtthresh()
```

Fortran command syntax:

```
LODVTTHRESH = GET_LODVTTHRESH()
```

Where:

Real LODVTTHRESH	LodVtThresh (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.100. GET_LSYSAN

Use this API to return the LSYSAN.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lsysan = get_lsysan()
```

Fortran command syntax:

```
LSYSAN = GET_LSYSAN()
```

Where:

Logical LSYSAN	LSYSAN (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.101. GET_MACBLKD

Use this API to return the macblkd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
macblkd = get_macblkd(indx1)

amacblkd = get_macblkd(indx1, indx2)
```

Fortran command syntax:

```
MACBLKD = GET_MACBLKD(INDX1)

AMACBLKD = GET_MACBLKD(INDX1, INDX2)
```

Where:

Logical MACBLKD	macblkd for the element indicated by indx1 (output).
Logical AMACBLKD (INDX2-INDX1+1)	macblkd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	macblkd (input).
Integer INDX2	macblkd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.102. GET_MACHID

Use this API to return the machine id.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
machid = get_machid(indx1)

amachid = get_machid(indx1, indx2)
```

Fortran command syntax:

```
MACHID = GET_MACHID(INDX1)

AMACHID = GET_MACHID(INDX1, INDX2)
```

Where:

Character*2 MACHID	machine id for the element indicated by indx1 (output).
Character*2 AMACHID (INDX2-INDX1+1)	machine id; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.103. GET_MACNUM

Use this API to return the machine array index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
macnum = get_macnum(indx1)

amacnum = get_macnum(indx1, indx2)
```

Fortran command syntax:

```
MACNUM = GET_MACNUM(INDX1)

AMACNUM = GET_MACNUM(INDX1, INDX2)
```

Where:

Integer MACNUM	machine array index for the element indicated by indx1 (output).
Integer AMACNUM (INDX2-INDX1+1)	machine array index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	(8*# of machines) (input).
Integer INDX2	(8*# of machines), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.104. GET_MARGIN

Use this API to return the multi-terminal dc line converter resistance to ground.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
margin = get_margin(cnvidx, indx)

amargin = get_margin(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
MARGIN = GET_MARGIN(CNVIDX, INDX)

AMARGIN = GET_MARGIN(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real MARGIN	multi-terminal dc line converter resistance to ground for the element indicated by indx1 (output).
Real AMARGIN ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter resistance to ground; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.105. GET_MBASE

Use this API to return the machine MVA base.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
mbase = get_mbase(indx1)

ambase = get_mbase(indx1, indx2)
```

Fortran command syntax:

```
MBASE = GET_MBASE(INDX1)

AMBASE = GET_MBASE(INDX1, INDX2)
```

Where:

Real MBASE	machine MVA base for the element indicated by indx1 (output).
Real AMBASE (INDX2-INDX1+1)	machine MVA base; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.106. GET_MCARYS

Use this API to return the mcarys.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
mcarys = get_mcarys(one, two)

amcarys = get_mcarys(one, one2, two, two2)
```

Fortran command syntax:

```
MCARYS = GET_MCARYS(ONE, TWO)

AMCARYS = GET_MCARYS(ONE, ONE2, TWO, TWO2)
```

Where:

Real MCARYS	mcarys for the element indicated by indx1 (output).
Real AMCARYS ((ONE2-ONE +1)*(TWO2-TWO+1))	mcarys; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	mcarys (input).
Integer ONE2	mcarys, upper value of range (input).
Integer TWO	mcarys (input).
Integer TWO2	mcarys, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.107. GET_MCSTAT

Use this API to return the machine staus.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
mcstat = get_mcstat(indx1)

amcstat = get_mcstat(indx1, indx2)
```

Fortran command syntax:

```
MCSTAT = GET_MCSTAT(INDX1)

AMCSTAT = GET_MCSTAT(INDX1, INDX2)
```

Where:

Integer MCSTAT	machine staus for the element indicated by indx1 (output).
Integer AMCSTAT (INDX2-INDX1+1)	machine staus; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.108. GET_MDC

Use this API to return the two terminal dc line control mode: 0 = blocked, 1 = power, 2 = current.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
mdc = get_mdc(indx1)

amdc = get_mdc(indx1, indx2)
```

Fortran command syntax:

```
MDC = GET_MDC(INDX1)

AMDC = GET_MDC(INDX1, INDX2)
```

Where:

Integer MDC	two terminal dc line control mode: 0 = blocked, 1 = power, 2 = current for the element indicated by indx1 (output).
Integer AMDC (INDX2-INDX1+1)	two terminal dc line control mode: 0 = blocked, 1 = power, 2 = current; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.109. GET_MDCNAM

Use this API to return the multi-terminal dc line name.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
mdcnam = get_mdcnam(indx1)

amdcnam = get_mdcnam(indx1, indx2)
```

Fortran command syntax:

```
MDCNAM = GET_MDCNAM(INDX1)

AMDCNAM = GET_MDCNAM(INDX1, INDX2)
```

Where:

Character*12 MDCNAM	multi-terminal dc line name for the element indicated by indx1 (output).
Character*12 AMDCNAM (INDX2-INDX1+1)	multi-terminal dc line name; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.110. GET_MIDTRM

Use this API to return the MIDTRM.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
midtrm = get_midtrm()
```

Fortran command syntax:

```
MIDTRM = GET_MIDTRM()
```

Where:

Logical MIDTRM	MIDTRM (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.111. GET_MODE

Use this API to return the MODE.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
mode = get_mode()
```

Fortran command syntax:

```
MODE = GET_MODE()
```

Where:

Integer MODE	MODE (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.112. GET_MSTATE

Use this API to return the MSTATE.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
mstate = get_mstate()
```

Fortran command syntax:

```
MSTATE = GET_MSTATE()
```

Where:

Integer MSTATE	MSTATE (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.113. GET_NBI

Use this API to return the Number of bridges in series for inverter.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
nbi = get_nbi(indx1)

anbi = get_nbi(indx1, indx2)
```

Fortran command syntax:

```
NBI = GET_NBI(INDX1)

ANBI = GET_NBI(INDX1, INDX2)
```

Where:

Integer NBI	Number of bridges in series for inverter for the element indicated by indx1 (output).
Integer ANBI (INDX2-INDX1+1)	Number of bridges in series for inverter; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.114. GET_NBR

Use this API to return the Number of bridges in series for rectifier.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
nbr = get_nbr(indx1)

anbr = get_nbr(indx1, indx2)
```

Fortran command syntax:

```
NBR = GET_NBR(INDX1)

ANBR = GET_NBR(INDX1, INDX2)
```

Where:

Integer NBR	Number of bridges in series for rectifier for the element indicated by indx1 (output).
Integer ANBR (INDX2-INDX1+1)	Number of bridges in series for rectifier; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.115. GET_NBRIDG

Use this API to return the multi-terminal dc line converter reactive power seen from ac network.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
nbridg = get_nbridg(cnvidx, indx)

anbridg = get_nbridg(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
NBRIDG = GET_NBRIDG(CNVIDX, INDX)

ANBRIDG = GET_NBRIDG(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer NBRIDG	multi-terminal dc line converter reactive power seen from ac network for the element indicated by indx1 (output).
Integer ANBRIDG ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter reactive power seen from ac network; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.116. GET_NBUS

Use this API to return the NBUS.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
nbus = get_nbus()
```

Fortran command syntax:

```
NBUS = GET_NBUS()
```

Where:

Integer NBUS	NBUS (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.117. GET_NCONV

Use this API to return the number of multi-terminal dc line terminals in dc network.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
nconv = get_nconv(indx1)

anconv = get_nconv(indx1, indx2)
```

Fortran command syntax:

```
NCONV = GET_NCONV(INDX1)

ANCONV = GET_NCONV(INDX1, INDX2)
```

Where:

Integer NCONV	number of multi-terminal dc line terminals in dc network for the element indicated by indx1 (output).
Integer ANCONV (INDX2-INDX1+1)	number of multi-terminal dc line terminals in dc network; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.118. GET_NINTEG

Use this API to return the NINTEG.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ninteg = get_ninteg()
```

Fortran command syntax:

```
NINTEG = GET_NINTEG()
```

Where:

Integer NINTEG	NINTEG (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.119. GET_NLIN

Use this API to return the NLIN.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
nlin = get_nlin()
```

Fortran command syntax:

```
NLIN = GET_NLIN()
```

Where:

Integer NLIN	NLIN (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.120. GET_NLWTRB

Use this API to return the nlwtrb.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
nlwtrb = get_nlwtrb(indx1)

anlwtrb = get_nlwtrb(indx1, indx2)
```

Fortran command syntax:

```
NLWTRB = GET_NLWTRB(INDX1)

ANLWTRB = GET_NLWTRB(INDX1, INDX2)
```

Where:

Integer NLWTRB	nlwtrb for the element indicated by indx1 (output).
Integer ANLWTRB (INDX2-INDX1+1)	nlwtrb; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	nlwtrb (input).
Integer INDX2	nlwtrb, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.121. GET_NMACHS

Use this API to return the NMACHS.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
nmachs = get_nmachs()
```

Fortran command syntax:

```
NMACHS = GET_NMACHS()
```

Where:

Integer NMACHS	NMACHS (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.122. GET_NSWSTP

Use this API to return the switched shunt number of steps per block.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
nswstp = get_nswstp(block, indx)

answstp = get_nswstp(block, block2, indx, indx2)
```

Fortran command syntax:

```
NSWSTP = GET_NSWSTP(BLOCK, INDX)

ANSWSTP = GET_NSWSTP(BLOCK, BLOCK2, INDX, INDX2)
```

Where:

Integer NSWSTP	switched shunt number of steps per block for the element indicated by indx1 (output).
Integer ANSWSTP ((BLOCK2-BLOCK+1)*(INDX2-INDX+1))	switched shunt number of steps per block; an array of values for the elements indicated by the indices supplied (output).
Integer BLOCK	block number (1:8) (input).
Integer BLOCK2	block number (1:8), upper value of range (input).
Integer INDX	switched shunt array index (input).
Integer INDX2	block number (1:8), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.123. GET_NTAPOS

Use this API to return the transformer number of tap positions available.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ntapos = get_ntapos(indx1)

antapos = get_ntapos(indx1, indx2)
```

Fortran command syntax:

```
NTAPOS = GET_NTAPOS(INDX1)

ANTAPOS = GET_NTAPOS(INDX1, INDX2)
```

Where:

Integer NTAPOS	transformer number of tap positions available for the element indicated by indx1 (output).
Integer ANTAPOS (INDX2-INDX1+1)	transformer number of tap positions available; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.124. GET_NUMARE

Use this API to return the area number.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
numare = get_numare(indx1)

anumare = get_numare(indx1, indx2)
```

Fortran command syntax:

```
NUMARE = GET_NUMARE(INDX1)

ANUMARE = GET_NUMARE(INDX1, INDX2)
```

Where:

Integer NUMARE	area number for the element indicated by indx1 (output).
Integer ANUMARE (INDX2-INDX1+1)	area number; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	area array index (input).
Integer INDX2	area array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.125. GET_NUMLOD

Use this API to return the numlod.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
numlod = get_numlod(indx1)

anumlod = get_numlod(indx1, indx2)
```

Fortran command syntax:

```
NUMLOD = GET_NUMLOD(INDX1)

ANUMLOD = GET_NUMLOD(INDX1, INDX2)
```

Where:

Integer NUMLOD	numlod for the element indicated by indx1 (output).
Integer ANUMLOD (INDX2-INDX1+1)	numlod; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	numlod (input).
Integer INDX2	numlod, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.126. GET_NUMOWN

Use this API to return the owner number.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
numown = get_numown(indx1)

anumown = get_numown(indx1, indx2)
```

Fortran command syntax:

```
NUMOWN = GET_NUMOWN(INDX1)

ANUMOWN = GET_NUMOWN(INDX1, INDX2)
```

Where:

Integer NUMOWN	owner number for the element indicated by indx1 (output).
Integer ANUMOWN (INDX2-INDX1+1)	owner number; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	owner array index (input).
Integer INDX2	owner array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.127. GET_NUMSWS

Use this API to return the numsws.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
numsws = get_numsws(indx1)

anumsws = get_numsws(indx1, indx2)
```

Fortran command syntax:

```
NUMSWS = GET_NUMSWS(INDX1)

ANUMSWS = GET_NUMSWS(INDX1, INDX2)
```

Where:

Integer NUMSWS	numsws for the element indicated by indx1 (output).
Integer ANUMSWS (INDX2-INDX1+1)	numsws; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	numsws (input).
Integer INDX2	numsws, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.128. GET_NUMTRM

Use this API to return the bus array index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
numtrm = get_numtrm(indx1)

anumtrm = get_numtrm(indx1, indx2)
```

Fortran command syntax:

```
NUMTRM = GET_NUMTRM(INDX1)

ANUMTRM = GET_NUMTRM(INDX1, INDX2)
```

Where:

Integer NUMTRM	bus array index for the element indicated by indx1 (output).
Integer ANUMTRM (INDX2-INDX1+1)	bus array index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.129. GET_NUMZON

Use this API to return the zone number.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
numzon = get_numzon(indx1)

anumzon = get_numzon(indx1, indx2)
```

Fortran command syntax:

```
NUMZON = GET_NUMZON(INDX1)

ANUMZON = GET_NUMZON(INDX1, INDX2)
```

Where:

Integer NUMZON	zone number for the element indicated by indx1 (output).
Integer ANUMZON (INDX2-INDX1+1)	zone number; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	zone array index (input).
Integer INDX2	zone array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.130. GET_OEL_ACTIVE

Use this API to return the oel_active.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
oel_active = get_oel_active(indx1)

aoel_active = get_oel_active(indx1, indx2)
```

Fortran command syntax:

```
OEL_ACTIVE = GET_OEL_ACTIVE(INDX1)

AOEL_ACTIVE = GET_OEL_ACTIVE(INDX1, INDX2)
```

Where:

Logical OEL_ACTIVE	oel_active for the element indicated by indx1 (output).
Logical AOEL_ACTIVE (INDX2-INDX1+1)	oel_active; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	oel_active (input).
Integer INDX2	oel_active, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.131. GET_OLDERV

Use this API to return the earlier value of derivative of state variable.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
olderv = get_olderv(indx1)

aolderv = get_olderv(indx1, indx2)
```

Fortran command syntax:

```
OLDERV = GET_OLDERV(INDX1)

AOLDERV = GET_OLDERV(INDX1, INDX2)
```

Where:

Real OLDERV	earlier value of derivative of state variable for the element indicated by indx1 (output).
Real AOLDERV (INDX2-INDX1+1)	earlier value of derivative of state variable; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	olderv (input).
Integer INDX2	olderv, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.132. GET_ORGDCMODE

Use this API to return the original multi-terminal dc line mode.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
orgdcmode = get_orgdcmode(indx1)

aorgdcmode = get_orgdcmode(indx1, indx2)
```

Fortran command syntax:

```
ORGDCMODE = GET_ORGDCMODE(INDX1)

AORGDCMODE = GET_ORGDCMODE(INDX1, INDX2)
```

Where:

Integer ORGDCMODE	original multi-terminal dc line mode for the element indicated by indx1 (output).
Integer AORGDCMODE (INDX2-INDX1+1)	original multi-terminal dc line mode; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	multi-terminal dc line array index (input).
Integer INDX2	multi-terminal dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.133. GET_ORGFCSTAT

Use this API to return the original FACTS device mode.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
orgfcstat = get_orgfcstat(indx1)

aorgfcstat = get_orgfcstat(indx1, indx2)
```

Fortran command syntax:

```
ORGFCSTAT = GET_ORGFCSTAT(INDX1)

AORGFCSTAT = GET_ORGFCSTAT(INDX1, INDX2)
```

Where:

Integer ORGFCSTAT	original FACTS device mode for the element indicated by indx1 (output).
Integer AORGFCSTAT (INDX2-INDX1+1)	original FACTS device mode; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	FACTS device array index (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.134. GET_ORGICOM

Use this API to return the original branch status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
orgicom = get_orgicom(indx1)

aorgicom = get_orgicom(indx1, indx2)
```

Fortran command syntax:

```
ORGICOM = GET_ORGICOM(INDX1)

AORGICOM = GET_ORGICOM(INDX1, INDX2)
```

Where:

Integer ORGICOM	original branch status for the element indicated by indx1 (output).
Integer AORGICOM (INDX2-INDX1+1)	original branch status; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.135. GET_ORGJCODE

Use this API to return the original bus type code.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
orgjcode = get_orgjcode(indx1)

aorgjcode = get_orgjcode(indx1, indx2)
```

Fortran command syntax:

```
ORGJCODE = GET_ORGJCODE(INDX1)

AORGJCODE = GET_ORGJCODE(INDX1, INDX2)
```

Where:

Integer ORGJCODE	original bus type code for the element indicated by indx1 (output).
Integer AORGJCODE (INDX2-INDX1+1)	original bus type code; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.136. GET_ORGMDC

Use this API to return the original 2-terminal dc line status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
orgmdc = get_orgmdc(indx1)

aorgmdc = get_orgmdc(indx1, indx2)
```

Fortran command syntax:

```
ORGMDC = GET_ORGMDC(INDX1)

AORGMDC = GET_ORGMDC(INDX1, INDX2)
```

Where:

Integer ORGMDC	original 2-terminal dc line status for the element indicated by indx1 (output).
Integer AORGMDC (INDX2-INDX1+1)	original 2-terminal dc line status; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	2-teminal dc line array index (input).
Integer INDX2	2-teminal dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.137. GET_ORGVSCMDC

Use this API to return the original VSCDC control mode.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
orgvscmdc = get_orgvscmdc(indx1)

aorgvscmdc = get_orgvscmdc(indx1, indx2)
```

Fortran command syntax:

```
ORGVSCMDC = GET_ORGVSCMDC(INDX1)

AORGVSCMDC = GET_ORGVSCMDC(INDX1, INDX2)
```

Where:

Integer ORGVSCMDC	original VSCDC control mode for the element indicated by indx1 (output).
Integer AORGVSCMDC (INDX2-INDX1+1)	original VSCDC control mode; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vsc dc line array index (input).
Integer INDX2	vsc dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.138. GET_ORGVSCTYP

Use this API to return the original VSCDC converter type.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
orgvsctyp = get_orgvsctyp(linend, indx)

aorgvsctyp = get_orgvsctyp(linend, linend2, indx, indx2)
```

Fortran command syntax:

```
ORGVSCTYP = GET_ORGVSCTYP(LINEND, INDX)

AORGVSCTYP = GET_ORGVSCTYP(LINEND, LINEND2, INDX, INDX2)
```

Where:

Integer ORGVSCTYP	original VSCDC converter type for the element indicated by indx1 (output).
Integer AORGVSCTYP ((LINEND2-LINEND+1)*(INDX2-INDX+1))	original VSCDC converter type; an array of values for the elements indicated by the indices supplied (output).
Integer LINEND	line end: (1 for inverter, 2 for rectifier) (input).
Integer LINEND2	line end: (1 for inverter, 2 for rectifier), upper value of range (input).
Integer INDX	vsc dc line array index (input).
Integer INDX2	line end: (1 for inverter, 2 for rectifier), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.139. GET_PAC

Use this API to return the multi-terminal dc line converter alpha or gamma.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pac = get_pac(cnvidx, indx)

apac = get_pac(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
PAC = GET_PAC(CNVIDX, INDX)

APAC = GET_PAC(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real PAC	multi-terminal dc line converter alpha or gamma for the element indicated by indx1 (output).
Real APAC ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter alpha or gamma; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.140. GET_PACI

Use this API to return the two terminal dc line inverter ac active power.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
paci = get_paci(indx1)

apaci = get_paci(indx1, indx2)
```

Fortran command syntax:

```
PACI = GET_PACI(INDX1)

APACI = GET_PACI(INDX1, INDX2)
```

Where:

Real PACI	two terminal dc line inverter ac active power for the element indicated by indx1 (output).
Real APACI (INDX2-INDX1+1)	two terminal dc line inverter ac active power; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.141. GET_PACR

Use this API to return the two terminal dc line rectifier ac active power.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pacr = get_pacr(indx1)

apacr = get_pacr(indx1, indx2)
```

Fortran command syntax:

```
PACR = GET_PACR(INDX1)

APACR = GET_PACR(INDX1, INDX2)
```

Where:

Real PACR	two terminal dc line rectifier ac active power for the element indicated by indx1 (output).
Real APACR (INDX2-INDX1+1)	two terminal dc line rectifier ac active power; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.142. GET_PELEC

Use this API to return the pelec.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pelec = get_pelec(indx1)

apelec = get_pelec(indx1, indx2)
```

Fortran command syntax:

```
PELEC = GET_PELEC(INDX1)

APELEC = GET_PELEC(INDX1, INDX2)
```

Where:

Real PELEC	pelec for the element indicated by indx1 (output).
Real APELEC (INDX2-INDX1+1)	pelec; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	pelec (input).
Integer INDX2	pelec, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.143. GET_PGENMC

Use this API to return the machine active power (p.u. of system MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pgenmc = get_pgenmc(indx1)

apgenmc = get_pgenmc(indx1, indx2)
```

Fortran command syntax:

```
PGENMC = GET_PGENMC(INDX1)

APGENMC = GET_PGENMC(INDX1, INDX2)
```

Where:

Real PGENMC	machine active power (p.u. of system MVA) for the element indicated by indx1 (output).
Real APGENMC (INDX2-INDX1+1)	machine active power (p.u. of system MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.144. GET_PHSHTB

Use this API to return the flag to indicate if the transformer impedance correction table is sensitive to phase shift angle.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
phshtb = get_phshtb(indx1)

aphshtb = get_phshtb(indx1, indx2)
```

Fortran command syntax:

```
PHSHTB = GET_PHSHTB(INDX1)

APHSHTB = GET_PHSHTB(INDX1, INDX2)
```

Where:

Logical PHSHTB	flag to indicate if the transformer impedance correction table is sensitive to phase shift angle for the element indicated by indx1 (output).
Logical APHSHTB (INDX2-INDX1+1)	flag to indicate if the transformer impedance correction table is sensitive to phase shift angle; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	transformer impedance correction table number (input).
Integer INDX2	transformer impedance correction table number, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.145. GET_PHSVLT

Use this API to return the PHSVLT.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
phsvlt = get_phsvlt()
```

Fortran command syntax:

```
PHSVLT = GET_PHSVLT()
```

Where:

Logical PHSVLT	PHSVLT (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.146. GET_PMAXMC

Use this API to return the machine maximum active power (p.u. of machine MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pmaxmc = get_pmaxmc(indx1)

apmaxmc = get_pmaxmc(indx1, indx2)
```

Fortran command syntax:

```
PMAXMC = GET_PMAXMC(INDX1)

APMAXMC = GET_PMAXMC(INDX1, INDX2)
```

Where:

Real PMAXMC	machine maximum active power (p.u. of machine MVA) for the element indicated by indx1 (output).
Real APMAXMC (INDX2-INDX1+1)	machine maximum active power (p.u. of machine MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.147. GET_PMAXMCD

Use this API to return the pmaxmcd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pmaxmcd = get_pmaxmcd(indx1)

apmaxmcd = get_pmaxmcd(indx1, indx2)
```

Fortran command syntax:

```
PMAXMCD = GET_PMAXMCD(INDX1)

APMAXMCD = GET_PMAXMCD(INDX1, INDX2)
```

Where:

Real PMAXMCD	pmaxmcd for the element indicated by indx1 (output).
Real APMAXMCD (INDX2-INDX1+1)	pmaxmcd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	pmaxmcd (input).
Integer INDX2	pmaxmcd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.148. GET_PMECH

Use this API to return the pmech.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pmech = get_pmech(indx1)

apmech = get_pmech(indx1, indx2)
```

Fortran command syntax:

```
PMECH = GET_PMECH(INDX1)

APMECH = GET_PMECH(INDX1, INDX2)
```

Where:

Real PMECH	pmech for the element indicated by indx1 (output).
Real APMECH (INDX2-INDX1+1)	pmech; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	pmech (input).
Integer INDX2	pmech, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.149. GET_PMINMC

Use this API to return the machine minimum active power (p.u. of machine MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pminmc = get_pminmc(indx1)

apminmc = get_pminmc(indx1, indx2)
```

Fortran command syntax:

```
PMINMC = GET_PMINMC(INDX1)

APMINMC = GET_PMINMC(INDX1, INDX2)
```

Where:

Real PMINMC	machine minimum active power (p.u. of machine MVA) for the element indicated by indx1 (output).
Real APMINMC (INDX2-INDX1+1)	machine minimum active power (p.u. of machine MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.150. GET_PMINMCD

Use this API to return the pminmcd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pminmcd = get_pminmcd(indx1)

apminmcd = get_pminmcd(indx1, indx2)
```

Fortran command syntax:

```
PMINMCD = GET_PMINMCD(INDX1)

APMINMCD = GET_PMINMCD(INDX1, INDX2)
```

Where:

Real PMINMCD	pminmcd for the element indicated by indx1 (output).
Real APMINMCD (INDX2-INDX1+1)	pminmcd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	pminmcd (input).
Integer INDX2	pminmcd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.151. GET_PQCNOD

Use this API to return the .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pqcnod = get_pqcnod(indx1)

apqcnod = get_pqcnod(indx1, indx2)
```

Fortran command syntax:

```
PQCNOD = GET_PQCNOD(INDX1)

APQCNOD = GET_PQCNOD(INDX1, INDX2)
```

Where:

Integer PQCNOD	for the element indicated by indx1 (output).
Integer APQCNOD (INDX2-INDX1+1)	; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.152. GET_PQCONT

Use this API to return the transformer signed bus sequence number of the bus whose voltage is controlled when is 1 or -1.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
pqcont = get_pqcont(indx1)

apqcont = get_pqcont(indx1, indx2)
```

Fortran command syntax:

```
PQCONT = GET_PQCONT(INDX1)

APQCONT = GET_PQCONT(INDX1, INDX2)
```

Where:

Integer PQCONT	transformer signed bus sequence number of the bus whose voltage is controlled when is 1 or -1 for the element indicated by indx1 (output).
Integer APQCONT (INDX2-INDX1+1)	transformer signed bus sequence number of the bus whose voltage is controlled when is 1 or -1; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.153. GET_PSANGL

Use this API to return the transformer winding 1 phase shift angle in radians .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
psangl = get_psangl(indx1)

apsangl = get_psangl(indx1, indx2)
```

Fortran command syntax:

```
PSANGL = GET_PSANGL(INDX1)

APSANGL = GET_PSANGL(INDX1, INDX2)
```

Where:

Real PSANGL	transformer winding 1 phase shift angle in radians for the element indicated by indx1 (output).
Real APSANGL (INDX2-INDX1+1)	transformer winding 1 phase shift angle in radians ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.154. GET_QAC

Use this API to return the multi-terminal dc line converter tap setting.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
qac = get_qac(cnvidx, indx)

aqac = get_qac(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
QAC = GET_QAC(CNVIDX, INDX)

AQAC = GET_QAC(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real QAC	multi-terminal dc line converter tap setting for the element indicated by indx1 (output).
Real AQAC ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter tap setting; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.155. GET_QACI

Use this API to return the two terminal dc line inverter ac reactive power.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
qaci = get_qaci(indx1)

aqaci = get_qaci(indx1, indx2)
```

Fortran command syntax:

```
QACI = GET_QACI(INDX1)

AQACI = GET_QACI(INDX1, INDX2)
```

Where:

Real QACI	two terminal dc line inverter ac reactive power for the element indicated by indx1 (output).
Real AQACI (INDX2-INDX1+1)	two terminal dc line inverter ac reactive power; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.156. GET_QACR

Use this API to return the two terminal dc line rectifier ac reactive power.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
qacr = get_qacr(indx1)

aqacr = get_qacr(indx1, indx2)
```

Fortran command syntax:

```
QACR = GET_QACR(INDX1)

AQACR = GET_QACR(INDX1, INDX2)
```

Where:

Real QACR	two terminal dc line rectifier ac reactive power for the element indicated by indx1 (output).
Real AQACR (INDX2-INDX1+1)	two terminal dc line rectifier ac reactive power; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.157. GET_QELEC

Use this API to return the qelec.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
qelec = get_qelec(indx1)

aqelec = get_qelec(indx1, indx2)
```

Fortran command syntax:

```
QELEC = GET_QELEC(INDX1)

AQELEC = GET_QELEC(INDX1, INDX2)
```

Where:

Real QELEC	qelec for the element indicated by indx1 (output).
Real AQELEC (INDX2-INDX1+1)	qelec; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	qelec (input).
Integer INDX2	qelec, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.158. GET_QGENMC

Use this API to return the machine reactive power (p.u. of system MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
qgenmc = get_qgenmc(indx1)

aqgenmc = get_qgenmc(indx1, indx2)
```

Fortran command syntax:

```
QGENMC = GET_QGENMC(INDX1)

AQGENMC = GET_QGENMC(INDX1, INDX2)
```

Where:

Real QGENMC	machine reactive power (p.u. of system MVA) for the element indicated by indx1 (output).
Real AQGENMC (INDX2-INDX1+1)	machine reactive power (p.u. of system MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.159. GET_QMAXMC

Use this API to return the machine maximum reactive power (p.u. of machine MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
qmaxmc = get_qmaxmc(indx1)

aqmaxmc = get_qmaxmc(indx1, indx2)
```

Fortran command syntax:

```
QMAXMC = GET_QMAXMC(INDX1)

AQMAXMC = GET_QMAXMC(INDX1, INDX2)
```

Where:

Real QMAXMC	machine maximum reactive power (p.u. of machine MVA) for the element indicated by indx1 (output).
Real AQMAXMC (INDX2-INDX1+1)	machine maximum reactive power (p.u. of machine MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.160. GET_QMINMC

Use this API to return the machine minimum reactive power (p.u. of machine MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
qminmc = get_qminmc(indx1)

aqminmc = get_qminmc(indx1, indx2)
```

Fortran command syntax:

```
QMINMC = GET_QMINMC(INDX1)

AQMINMC = GET_QMINMC(INDX1, INDX2)
```

Where:

Real QMINMC	machine minimum reactive power (p.u. of machine MVA) for the element indicated by indx1 (output).
Real AQMINMC (INDX2-INDX1+1)	machine minimum reactive power (p.u. of machine MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.161. GET_RCI

Use this API to return the two-terminal dc line inverter commutating transformer resistance per bridge in ohms..

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rci = get_rci(indx1)

arci = get_rci(indx1, indx2)
```

Fortran command syntax:

```
RCI = GET_RCI(INDX1)

ARCI = GET_RCI(INDX1, INDX2)
```

Where:

Real RCI	two-terminal dc line inverter commutating transformer resistance per bridge in ohms. for the element indicated by indx1 (output).
Real ARCI (INDX2-INDX1+1)	two-terminal dc line inverter commutating transformer resistance per bridge in ohms.; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.162. GET_RCOMP

Use this API to return the two terminal dc line compounding resistance (ohm).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rcomp = get_rcomp(indx1)

arcomp = get_rcomp(indx1, indx2)
```

Fortran command syntax:

```
RCOMP = GET_RCOMP(INDX1)

ARCOMP = GET_RCOMP(INDX1, INDX2)
```

Where:

Real RCOMP	two terminal dc line compounding resistance (ohm) for the element indicated by indx1 (output).
Real ARCOMP (INDX2-INDX1+1)	two terminal dc line compounding resistance (ohm); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.163. GET_RCONV

Use this API to return the multi-terminal dc line converter commutating resistance.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rconv = get_rconv(cnvidx, indx)

arconv = get_rconv(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
RCONV = GET_RCONV(CNVIDX, INDX)

ARCONV = GET_RCONV(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real RCONV	multi-terminal dc line converter commutating resistance for the element indicated by indx1 (output).
Real ARCONV ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter commutating resistance; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.164. GET_RCR

Use this API to return the two terminal dc line rectifier commutating transformer resistance per bridge in ohms..

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rcr = get_rcr(indx1)

arcr = get_rcr(indx1, indx2)
```

Fortran command syntax:

```
RCR = GET_RCR(INDX1)

ARCR = GET_RCR(INDX1, INDX2)
```

Where:

Real RCR	two terminal dc line rectifier commutating transformer resistance per bridge in ohms. for the element indicated by indx1 (output).
Real ARCR (INDX2-INDX1+1)	two terminal dc line rectifier commutating transformer resistance per bridge in ohms.; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.165. GET_RDC

Use this API to return the two terminal dc line resistance in ohms.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rdc = get_rdc(indx1)

ardc = get_rdc(indx1, indx2)
```

Fortran command syntax:

```
RDC = GET_RDC(INDX1)

ARDC = GET_RDC(INDX1, INDX2)
```

Where:

Real RDC	two terminal dc line resistance in ohms for the element indicated by indx1 (output).
Real ARDC (INDX2-INDX1+1)	two terminal dc line resistance in ohms; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.166. GET_RLSTRT

Use this API to return the rlstrt.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rlstrt = get_rlstrt(one, two)

arlstrt = get_rlstrt(one, one2, two, two2)
```

Fortran command syntax:

```
RLSTRT = GET_RLSTRT(ONE, TWO)

ARLSTRT = GET_RLSTRT(ONE, ONE2, TWO, TWO2)
```

Where:

Integer RLSTRT	rlstrt for the element indicated by indx1 (output).
Integer ARLSTRT ((ONE2-ONE +1)*(TWO2-TWO+1))	rlstrt; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	rlstrt (input).
Integer ONE2	rlstrt, upper value of range (input).
Integer TWO	(input).
Integer TWO2	rlstrt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.167. GET_RMAX

Use this API to return the transformer controlling quantity upper limit .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rmax = get_rmax(indx1)

armax = get_rmax(indx1, indx2)
```

Fortran command syntax:

```
RMAX = GET_RMAX(INDX1)

ARMAX = GET_RMAX(INDX1, INDX2)
```

Where:

Real RMAX	transformer controlling quantity upper limit for the element indicated by indx1 (output).
Real ARMAX (INDX2-INDX1+1)	transformer controlling quantity upper limit ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.168. GET_RMIN

Use this API to return the transformer controlling quantity lower limit .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rmin = get_rmin(indx1)

armin = get_rmin(indx1, indx2)
```

Fortran command syntax:

```
RMIN = GET_RMIN(INDX1)

ARMIN = GET_RMIN(INDX1, INDX2)
```

Where:

Real RMIN	transformer controlling quantity lower limit for the element indicated by indx1 (output).
Real ARMIN (INDX2-INDX1+1)	transformer controlling quantity lower limit ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.169. GET_SBASE

Use this API to return the system MVA base.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
sbase = get_sbase()
```

Fortran command syntax:

```
SBASE = GET_SBASE()
```

Where:

Real SBASE	system MVA base (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.170. GET_SETVAL

Use this API to return the two terminal dc line current (A) or power (MW) demand.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
setval = get_setval(indx1)

asetval = get_setval(indx1, indx2)
```

Fortran command syntax:

```
SETVAL = GET_SETVAL(INDX1)

ASETVAL = GET_SETVAL(INDX1, INDX2)
```

Where:

Real SETVAL	two terminal dc line current (A) or power (MW) demand for the element indicated by indx1 (output).
Real ASETVAL (INDX2-INDX1+1)	two terminal dc line current (A) or power (MW) demand; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.171. GET_SHFRST

Use this API to return the index of the first fixed shunt at a bus .

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
shfrst = get_shfrst(indx1)

ashfrst = get_shfrst(indx1, indx2)
```

Fortran command syntax:

```
SHFRST = GET_SHFRST(INDX1)

ASHFRST = GET_SHFRST(INDX1, INDX2)
```

Where:

Integer SHFRST	index of the first fixed shunt at a bus for the element indicated by indx1 (output).
Integer ASHFRST (INDX2-INDX1+1)	index of the first fixed shunt at a bus ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.172. GET_SHNEXT

Use this API to return the index of next fixed shunt.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
shnext = get_shnext(indx1)

ashnext = get_shnext(indx1, indx2)
```

Fortran command syntax:

```
SHNEXT = GET_SHNEXT(INDX1)

ASHNEXT = GET_SHNEXT(INDX1, INDX2)
```

Where:

Integer SHNEXT	index of next fixed shunt for the element indicated by indx1 (output).
Integer ASHNEXT (INDX2-INDX1+1)	index of next fixed shunt; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	fixed shunt array index (input).
Integer INDX2	fixed shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.173. GET_SHNTGB

Use this API to return the fixed shunt admittance (p.u. of system MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
shntgb = get_shntgb(indx1)

ashntgb = get_shntgb(indx1, indx2)
```

Fortran command syntax:

```
SHNTGB = GET_SHNTGB(INDX1)

ASHNTGB = GET_SHNTGB(INDX1, INDX2)
```

Where:

Complex SHNTGB	fixed shunt admittance (p.u. of system MVA) for the element indicated by indx1 (output).
Complex ASHNTGB (INDX2-INDX1+1)	fixed shunt admittance (p.u. of system MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	fixed shunt array index (input).
Integer INDX2	fixed shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.174. GET_SHNTID

Use this API to return the fixed shunt id.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
shntid = get_shntid(indx1)

ashntid = get_shntid(indx1, indx2)
```

Fortran command syntax:

```
SHNTID = GET_SHNTID(INDX1)

ASHNTID = GET_SHNTID(INDX1, INDX2)
```

Where:

Character*2 SHNTID	fixed shunt id for the element indicated by indx1 (output).
Character*2 ASHNTID (INDX2-INDX1+1)	fixed shunt id; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	fixed shunt array index (input).
Integer INDX2	fixed shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.175. GET_SHSTAT

Use this API to return the fixed shunt status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
shstat = get_shstat(indx1)

ashstat = get_shstat(indx1, indx2)
```

Fortran command syntax:

```
SHSTAT = GET_SHSTAT(INDX1)

ASHSTAT = GET_SHSTAT(INDX1, INDX2)
```

Where:

Integer SHSTAT	fixed shunt status for the element indicated by indx1 (output).
Integer ASHSTAT (INDX2-INDX1+1)	fixed shunt status; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	fixed shunt array index (input).
Integer INDX2	fixed shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.176. GET_SPEED

Use this API to return the speed.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
speed = get_speed(indx1)

aspeed = get_speed(indx1, indx2)
```

Fortran command syntax:

```
SPEED = GET_SPEED(INDX1)

ASPEED = GET_SPEED(INDX1, INDX2)
```

Where:

Real SPEED	speed for the element indicated by indx1 (output).
Real ASPEED (INDX2-INDX1+1)	speed; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	speed (input).
Integer INDX2	speed, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.177. GET_STATE

Use this API to return the state variable.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
state = get_state(indx1)

astate = get_state(indx1, indx2)
```

Fortran command syntax:

```
STATE = GET_STATE(INDX1)

ASTATE = GET_STATE(INDX1, INDX2)
```

Where:

Real STATE	state variable for the element indicated by indx1 (output).
Real ASTATE (INDX2-INDX1+1)	state variable; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	state (input).
Integer INDX2	state, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.178. GET_STATE_DSCRPT

Use this API to return the state_dscrpt.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
state_dscrpt = get_state_dscrpt(indx1)

astate_dscrpt = get_state_dscrpt(indx1, indx2)
```

Fortran command syntax:

```
STATE_DSCRPT = GET_STATE_DSCRPT(INDX1)

ASTATE_DSCRPT = GET_STATE_DSCRPT(INDX1, INDX2)
```

Where:

Character*144 STATE_DSCRPT	state_dscrpt for the element indicated by indx1 (output).
Character*144 ASTATE_DSCRPT (INDX2-INDX1+1)	state_dscrpt; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	state_dscrpt (input).
Integer INDX2	state_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.179. GET_STORE

Use this API to return the contains information about past history of state variables.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
store = get_store(indx1)

astore = get_store(indx1, indx2)
```

Fortran command syntax:

```
STORE = GET_STORE(INDX1)

ASTORE = GET_STORE(INDX1, INDX2)
```

Where:

Real STORE	contains information about past history of state variables for the element indicated by indx1 (output).
Real ASTORE (INDX2-INDX1+1)	contains information about past history of state variables; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	store (input).
Integer INDX2	store, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.180. GET_STRT2D

Use this API to return the strt2d.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strt2d = get_strt2d(one, two)

astrt2d = get_strt2d(one, one2, two, two2)
```

Fortran command syntax:

```
STRT2D = GET_STRT2D(ONE, TWO)

ASTRT2D = GET_STRT2D(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRT2D	strt2d for the element indicated by indx1 (output).
Integer ASTRT2D ((ONE2-ONE +1)*(TWO2-TWO+1))	strt2d; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strt2d (input).
Integer ONE2	strt2d, upper value of range (input).
Integer TWO	strt2d (input).
Integer TWO2	strt2d, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.181. GET_STRTAU

Use this API to return the strtau.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtau = get_strtau(one, two)

astrtau = get_strtau(one, one2, two, two2)
```

Fortran command syntax:

```
STRTAU = GET_STRTAU(ONE, TWO)

ASTRTAU = GET_STRTAU(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRTAU	strtau for the element indicated by indx1 (output).
Integer ASTRTAU ((ONE2-ONE +1)*(TWO2-TWO+1))	strtau; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strtau (input).
Integer ONE2	strtau, upper value of range (input).
Integer TWO	strtau (input).
Integer TWO2	strtau, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.182. GET_STRTCCT

Use this API to return the strtcct.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtcct = get_strtcct(one, two)

astrtcct = get_strtcct(one, one2, two, two2)
```

Fortran command syntax:

```
STRTCCT = GET_STRTCCT(ONE, TWO)

ASTRTCCT = GET_STRTCCT(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRTCCT	strtcct for the element indicated by indx1 (output).
Integer ASTRTCCT ((ONE2-ONE +1)*(TWO2-TWO+1))	strtcct; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strtcct (input).
Integer ONE2	strtcct, upper value of range (input).
Integer TWO	strtcct (input).
Integer TWO2	strtcct, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.183. GET_STRTFC

Use this API to return the strtfc.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtfc = get_strtfc(one, two)

astrtfc = get_strtfc(one, one2, two, two2)
```

Fortran command syntax:

```
STRTFC = GET_STRTFC(ONE, TWO)

ASTRTFC = GET_STRTFC(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRTFC	strtfc for the element indicated by indx1 (output).
Integer ASTRTFC ((ONE2-ONE +1)*(TWO2-TWO+1))	strtfc; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strtfc (input).
Integer ONE2	strtfc, upper value of range (input).
Integer TWO	strtfc (input).
Integer TWO2	strtfc, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.184. GET_STRTIM

Use this API to return the strtim.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtim = get_strtim(one, two)

astrtim = get_strtim(one, one2, two, two2)
```

Fortran command syntax:

```
STRTIM = GET_STRTIM(ONE, TWO)

ASTRTIM = GET_STRTIM(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRTIM	strtim for the element indicated by indx1 (output).
Integer ASTRTIM ((ONE2-ONE +1)*(TWO2-TWO+1))	strtim; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strtim (input).
Integer ONE2	strtim, upper value of range (input).
Integer TWO	strtim (input).
Integer TWO2	strtim, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.185. GET_STRTIN

Use this API to return the strtin.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtin = get_strtin(array_type, indx)

astrtin = get_strtin(array_type, array_type2, indx, indx2)
```

Fortran command syntax:

```
STRTIN = GET_STRTIN(ARRAY_TYPE, INDX)

ASTRTIN = GET_STRTIN(ARRAY_TYPE, ARRAY_TYPE2, INDX, INDX2)
```

Where:

Integer STRTIN	strtin for the element indicated by indx1 (output).
Integer ASTRTIN ((ARRAY_TYPE2-ARRAY_TYPE+1)*(INDX2-INDX+1))	strtin; an array of values for the elements indicated by the indices supplied (output).
Integer ARRAY_TYPE	array type (1:4) (input).
Integer ARRAY_TYPE2	array type (1:4), upper value of range (input).
Integer INDX	(8*# of machines (input).
Integer INDX2	array type (1:4), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.186. GET_STRTMD

Use this API to return the strtmd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtmd = get_strtmd(one, two)

astrtmd = get_strtmd(one, one2, two, two2)
```

Fortran command syntax:

```
STRTMD = GET_STRTMD(ONE, TWO)

ASTRTMD = GET_STRTMD(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRTMD	strtmd for the element indicated by indx1 (output).
Integer ASTRTMD ((ONE2-ONE +1)*(TWO2-TWO+1))	strtmd; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strtmd (input).
Integer ONE2	strtmd, upper value of range (input).
Integer TWO	strtmd (input).
Integer TWO2	strtmd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.187. GET_STRTSS

Use this API to return the strtss.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtss = get_strtss(one, two)

astrtss = get_strtss(one, one2, two, two2)
```

Fortran command syntax:

```
STRTSS = GET_STRTSS(ONE, TWO)

ASTRTSS = GET_STRTSS(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRTSS	strtss for the element indicated by indx1 (output).
Integer ASTRTSS ((ONE2-ONE +1)*(TWO2-TWO+1))	strtss; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strtss (input).
Integer ONE2	strtss, upper value of range (input).
Integer TWO	strtss (input).
Integer TWO2	strtss, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.188. GET_STRTVS

Use this API to return the strtvs.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
strtvs = get_strtvs(one, two)

astrtvs = get_strtvs(one, one2, two, two2)
```

Fortran command syntax:

```
STRTVS = GET_STRTVS(ONE, TWO)

ASTRTVS = GET_STRTVS(ONE, ONE2, TWO, TWO2)
```

Where:

Integer STRTVS	strtvs for the element indicated by indx1 (output).
Integer ASTRTVS ((ONE2-ONE +1)*(TWO2-TWO+1))	strtvs; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	strtvs (input).
Integer ONE2	strtvs, upper value of range (input).
Integer TWO	strtvs (input).
Integer TWO2	strtvs, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.189. GET_SWSAUX

Use this API to return the swsaux.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
swsaux = get_swsaux(indx1)

aswsaux = get_swsaux(indx1, indx2)
```

Fortran command syntax:

```
SWSAUX = GET_SWSAUX(INDX1)

ASWSAUX = GET_SWSAUX(INDX1, INDX2)
```

Where:

Real SWSAUX	swsaux for the element indicated by indx1 (output).
Real ASWSAUX (INDX2-INDX1+1)	swsaux; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	swsaux (input).
Integer INDX2	swsaux, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.190. GET_SWSCNX

Use this API to return the swscnx.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
swscnx = get_swscnx(indx1)

aswscnx = get_swscnx(indx1, indx2)
```

Fortran command syntax:

```
SWSCNX = GET_SWSCNX(INDX1)

ASWSCNX = GET_SWSCNX(INDX1, INDX2)
```

Where:

Integer SWSCNX	swscnx for the element indicated by indx1 (output).
Integer ASWSCNX (INDX2-INDX1+1)	swscnx; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	swscnx (input).
Integer INDX2	swscnx, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.191. GET_SWSHID

Use this API to return the switched shunt id.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
swshid = get_swshid(indx1)

aswshid = get_swshid(indx1, indx2)
```

Fortran command syntax:

```
SWSHID = GET_SWSHID(INDX1)

ASWSHID = GET_SWSHID(INDX1, INDX2)
```

Where:

Character*2 SWSHID	switched shunt id for the element indicated by indx1 (output).
Character*2 ASWSHID (INDX2-INDX1+1)	switched shunt id; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	switched shunt array index (input).
Integer INDX2	switched shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.192. GET_SWSHNB

Use this API to return the switched shunt admittance increment for each step in the block in MVA .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
swshnb = get_swshnb(block, indx)

aswshnb = get_swshnb(block, block2, indx, indx2)
```

Fortran command syntax:

```
SWSHNB = GET_SWSHNB(BLOCK, INDX)

ASWSHNB = GET_SWSHNB(BLOCK, BLOCK2, INDX, INDX2)
```

Where:

Real SWSHNB	switched shunt admittance increment for each step in the block in MVA for the element indicated by indx1 (output).
Real ASWSHNB ((BLOCK2-BLOCK +1)*(INDX2-INDX+1))	switched shunt admittance increment for each step in the block in MVA ; an array of values for the elements indicated by the indices supplied (output).
Integer BLOCK	block number (1:8) (input).
Integer BLOCK2	block number (1:8), upper value of range (input).
Integer INDX	switched shunt array index (input).
Integer INDX2	block number (1:8), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.193. GET_SWSHRM

Use this API to return the bus sequence number of the bus where voltage or reactive power output is controlled by a switched shunt (zero if shunt controls it's own voltage).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
swshrm = get_swshrm(indx1)

aswshrm = get_swshrm(indx1, indx2)
```

Fortran command syntax:

```
SWSHRM = GET_SWSHRM(INDX1)

ASWSHRM = GET_SWSHRM(INDX1, INDX2)
```

Where:

Integer SWSHRM	bus sequence number of the bus where voltage or reactive power output is controlled by a switched shunt (zero if shunt controls it's own voltage) for the element indicated by indx1 (output).
Integer ASWSHRM (INDX2-INDX1+1)	bus sequence number of the bus where voltage or reactive power output is controlled by a switched shunt (zero if shunt controls it's own voltage); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	switched shunt array index (input).
Integer INDX2	switched shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.194. GET_SWSREF

Use this API to return the swsref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
swsref = get_swsref(indx1)

aswsref = get_swsref(indx1, indx2)
```

Fortran command syntax:

```
SWSREF = GET_SWSREF(INDX1)

ASWSREF = GET_SWSREF(INDX1, INDX2)
```

Where:

Real SWSREF	swsref for the element indicated by indx1 (output).
Real ASWSREF (INDX2-INDX1+1)	swsref; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	swsref (input).
Integer INDX2	swsref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.195. GET_TABLNO

Use this API to return the transformer impedance correction table number .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tablno = get_tablno(indx1)

atablno = get_tablno(indx1, indx2)
```

Fortran command syntax:

```
TABLNO = GET_TABLNO(INDX1)

ATABLNO = GET_TABLNO(INDX1, INDX2)
```

Where:

Integer TABLNO	transformer impedance correction table number for the element indicated by indx1 (output).
Integer ATABLNO (INDX2-INDX1+1)	transformer impedance correction table number ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.196. GET_TAPI

Use this API to return the two terminal dc line actual inverter tap.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tapi = get_tapi(indx1)

atapi = get_tapi(indx1, indx2)
```

Fortran command syntax:

```
TAPI = GET_TAPI(INDX1)

ATAPI = GET_TAPI(INDX1, INDX2)
```

Where:

Real TAPI	two terminal dc line actual inverter tap for the element indicated by indx1 (output).
Real ATAPI (INDX2-INDX1+1)	two terminal dc line actual inverter tap; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.197. GET_TAPNO

Use this API to return the transformer tap .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tapno = get_tapno(indx1)

atapno = get_tapno(indx1, indx2)
```

Fortran command syntax:

```
TAPNO = GET_TAPNO(INDX1)

ATAPNO = GET_TAPNO(INDX1, INDX2)
```

Where:

Integer TAPNO	transformer tap for the element indicated by indx1 (output).
Integer ATAPNO (INDX2-INDX1+1)	transformer tap ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.198. GET_TAPR

Use this API to return the two terminal dc line actual rectifier tap.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tapr = get_tapr(indx1)

atapr = get_tapr(indx1, indx2)
```

Fortran command syntax:

```
TAPR = GET_TAPR(INDX1)

ATAPR = GET_TAPR(INDX1, INDX2)
```

Where:

Real TAPR	two terminal dc line actual rectifier tap for the element indicated by indx1 (output).
Real ATAPR (INDX2-INDX1+1)	two terminal dc line actual rectifier tap; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.199. GET_TBRATE

Use this API to return the tbrate.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tbrate = get_tbrate(indx1)

atbrate = get_tbrate(indx1, indx2)
```

Fortran command syntax:

```
TBRATE = GET_TBRATE(INDX1)

ATBRATE = GET_TBRATE(INDX1, INDX2)
```

Where:

Real TBRATE	tbrate for the element indicated by indx1 (output).
Real ATBRATE (INDX2-INDX1+1)	tbrate; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	tbrate (input).
Integer INDX2	tbrate, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.200. GET_THRSHZ

Use this API to return the THRSHZ.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
thrshz = get_thrshz()
```

Fortran command syntax:

```
THRSHZ = GET_THRSHZ()
```

Where:

Real THRSHZ	THRSHZ (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.201. GET_TIME

Use this API to return the TIME.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
time = get_time()
```

Fortran command syntax:

```
TIME = GET_TIME()
```

Where:

Real TIME	TIME (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.202. GET_TOL

Use this API to return the TOL.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tol = get_tol()
```

Fortran command syntax:

```
TOL = GET_TOL()
```

Where:

Real TOL	TOL (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.203. GET_TPLOAD

Use this API to return the tpload.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tpload = get_tpload(indx1)

atpload = get_tpload(indx1, indx2)
```

Fortran command syntax:

```
TPLOAD = GET_TPLOAD(INDX1)

ATPLOAD = GET_TPLOAD(INDX1, INDX2)
```

Where:

Real TPLOAD	tpload for the element indicated by indx1 (output).
Real ATPLOAD (INDX2-INDX1+1)	tpload; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	tpload (input).
Integer INDX2	tpload, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.204. GET_TPNABL

Use this API to return the transformer adjustment control flag.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tpnabl = get_tpnabl(indx1)

atpnabl = get_tpnabl(indx1, indx2)
```

Fortran command syntax:

```
TPNABL = GET_TPNABL(INDX1)

ATPNABL = GET_TPNABL(INDX1, INDX2)
```

Where:

Integer TPNABL	transformer adjustment control flag for the element indicated by indx1 (output).
Integer ATPNABL (INDX2-INDX1+1)	transformer adjustment control flag; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.205. GET_TQLOAD

Use this API to return the tqload.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tqload = get_tqload(indx1)

atqload = get_tqload(indx1, indx2)
```

Fortran command syntax:

```
TQLOAD = GET_TQLOAD(INDX1)

ATQLOAD = GET_TQLOAD(INDX1, INDX2)
```

Where:

Real TQLOAD	tqload for the element indicated by indx1 (output).
Real ATQLOAD (INDX2-INDX1+1)	tqload; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	tqload (input).
Integer INDX2	tqload, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.206. GET_TRCONV

Use this API to return the current change participation factor.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
trconv = get_trconv(cnvidx, indx)

atrconv = get_trconv(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
TRCONV = GET_TRCONV(CNVIDX, INDX)

ATRCONV = GET_TRCONV(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real TRCONV	current change participation factor for the element indicated by indx1 (output).
Real ATRCONV ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	current change participation factor; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.207. GET_TRI

Use this API to return the two terminal dc line inverter transformer ratio.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
tri = get_tri(indx1)

atri = get_tri(indx1, indx2)
```

Fortran command syntax:

```
TRI = GET_TRI(INDX1)

ATRI = GET_TRI(INDX1, INDX2)
```

Where:

Real TRI	two terminal dc line inverter transformer ratio for the element indicated by indx1 (output).
Real ATRI (INDX2-INDX1+1)	two terminal dc line inverter transformer ratio; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.208. GET_TRR

Use this API to return the two terminal dc line rectifier transformer ratio.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
trr = get_trr(indx1)

atrr = get_trr(indx1, indx2)
```

Fortran command syntax:

```
TRR = GET_TRR(INDX1)

ATRR = GET_TRR(INDX1, INDX2)
```

Where:

Real TRR	two terminal dc line rectifier transformer ratio for the element indicated by indx1 (output).
Real ATRR (INDX2-INDX1+1)	two terminal dc line rectifier transformer ratio; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.209. GET_UEL_ACTIVE

Use this API to return the uel_active.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
uel_active = get_uel_active(indx1)

auel_active = get_uel_active(indx1, indx2)
```

Fortran command syntax:

```
UEL_ACTIVE = GET_UEL_ACTIVE(INDX1)

AUEL_ACTIVE = GET_UEL_ACTIVE(INDX1, INDX2)
```

Where:

Logical UEL_ACTIVE	uel_active for the element indicated by indx1 (output).
Logical AUEL_ACTIVE (INDX2-INDX1+1)	uel_active; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	uel_active (input).
Integer INDX2	uel_active, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.210. GET_VAR

Use this API to return the scartch memory location for storing model floating point data.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
var = get_var(indx1)

avar = get_var(indx1, indx2)
```

Fortran command syntax:

```
VAR = GET_VAR(INDX1)

AVAR = GET_VAR(INDX1, INDX2)
```

Where:

Real VAR	scartch memory location for storing model floating point data for the element indicated by indx1 (output).
Real AVAR (INDX2-INDX1+1)	scartch memory location for storing model floating point data; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	var (input).
Integer INDX2	var, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.211. GET_VAR_DSCRPT

Use this API to return the var_dscrpt.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
var_dscrpt = get_var_dscrpt(indx1)

avar_dscrpt = get_var_dscrpt(indx1, indx2)
```

Fortran command syntax:

```
VAR_DSCRPT = GET_VAR_DSCRPT(INDX1)

AVAR_DSCRPT = GET_VAR_DSCRPT(INDX1, INDX2)
```

Where:

Character*144 VAR_DSCRPT	var_dscrpt for the element indicated by indx1 (output).
Character*144 AVAR_DSCRPT (INDX2-INDX1+1)	var_dscrpt; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	var_dscrpt (input).
Integer INDX2	var_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.212. GET_VCMODE

Use this API to return the two terminal dc line switch dc voltage in kV.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vcmode = get_vcmode(indx1)

avcmode = get_vcmode(indx1, indx2)
```

Fortran command syntax:

```
VCMODE = GET_VCMODE(INDX1)

AVCMODE = GET_VCMODE(INDX1, INDX2)
```

Where:

Real VCMODE	two terminal dc line switch dc voltage in kV for the element indicated by indx1 (output).
Real AVCMODE (INDX2-INDX1+1)	two terminal dc line switch dc voltage in kV; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.213. GET_VDC

Use this API to return the multi-terminal dc line dc bus voltage.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vdc = get_vdc(dcbusidx, indx)

avdc = get_vdc(dcbusidx, dcbusidx2, indx, indx2)
```

Fortran command syntax:

```
VDC = GET_VDC(DCBUSIDX, INDX)

AVDC = GET_VDC(DCBUSIDX, DCBUSIDX2, INDX, INDX2)
```

Where:

Real VDC	multi-terminal dc line dc bus voltage for the element indicated by indx1 (output).
Real AVDC ((DCBUSIDX2-DCBUSIDX+1)*(INDX2-INDX+1))	multi-terminal dc line dc bus voltage; an array of values for the elements indicated by the indices supplied (output).
Integer DCBUSIDX	multi-terminal dc line dc bus index (1:MXDCBS) (input).
Integer DCBUSIDX2	multi-terminal dc line dc bus index (1:MXDCBS), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line dc bus index (1:MXDCBS), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.214. GET_VDCI

Use this API to return the two terminal dc line inverter dc voltage.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vdci = get_vdci(indx1)

avdci = get_vdci(indx1, indx2)
```

Fortran command syntax:

```
VDCI = GET_VDCI(INDX1)

AVDCI = GET_VDCI(INDX1, INDX2)
```

Where:

Real VDCI	two terminal dc line inverter dc voltage for the element indicated by indx1 (output).
Real AVDCI (INDX2-INDX1+1)	two terminal dc line inverter dc voltage; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.215. GET_VDCR

Use this API to return the two terminal dc line rectifier dc voltage.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vdcr = get_vdcr(indx1)

avdcr = get_vdcr(indx1, indx2)
```

Fortran command syntax:

```
VDCR = GET_VDCR(INDX1)

AVDCR = GET_VDCR(INDX1, INDX2)
```

Where:

Real VDCR	two terminal dc line rectifier dc voltage for the element indicated by indx1 (output).
Real AVDCR (INDX2-INDX1+1)	two terminal dc line rectifier dc voltage; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.216. GET_VFEOUT

Use this API to return the vfeout.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vfeout = get_vfeout(indx1)

avfeout = get_vfeout(indx1, indx2)
```

Fortran command syntax:

```
VFEOUT = GET_VFEOUT(INDX1)

AVFEOUT = GET_VFEOUT(INDX1, INDX2)
```

Where:

Real VFEOUT	vfeout for the element indicated by indx1 (output).
Real AVFEOUT (INDX2-INDX1+1)	vfeout; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vfeout (input).
Integer INDX2	vfeout, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.217. GET_VG2IDX

Use this API to return the index of the two-winding transformer vector group .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vg2idx = get_vg2idx(indx1)

avg2idx = get_vg2idx(indx1, indx2)
```

Fortran command syntax:

```
VG2IDX = GET_VG2IDX(INDX1)

AVG2IDX = GET_VG2IDX(INDX1, INDX2)
```

Where:

Integer VG2IDX	index of the two-winding transformer vector group for the element indicated by indx1 (output).
Integer AVG2IDX (INDX2-INDX1+1)	index of the two-winding transformer vector group ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.218. GET_VMAX

Use this API to return the transformer controlled quantity upper limit .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vmax = get_vmax(indx1)

avmax = get_vmax(indx1, indx2)
```

Fortran command syntax:

```
VMAX = GET_VMAX(INDX1)

AVMAX = GET_VMAX(INDX1, INDX2)
```

Where:

Real VMAX	transformer controlled quantity upper limit for the element indicated by indx1 (output).
Real AVMAX (INDX2-INDX1+1)	transformer controlled quantity upper limit ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.219. GET_VMIN

Use this API to return the transformer controlled quantity lower limit .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vmin = get_vmin(indx1)

avmin = get_vmin(indx1, indx2)
```

Fortran command syntax:

```
VMIN = GET_VMIN(INDX1)

AVMIN = GET_VMIN(INDX1, INDX2)
```

Where:

Real VMIN	transformer controlled quantity lower limit for the element indicated by indx1 (output).
Real AVMIN (INDX2-INDX1+1)	transformer controlled quantity lower limit ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.220. GET_VOEL

Use this API to return the voel.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
voel = get_voel(indx1)

avoel = get_voel(indx1, indx2)
```

Fortran command syntax:

```
VOEL = GET_VOEL(INDX1)

AVOEL = GET_VOEL(INDX1, INDX2)
```

Where:

Real VOEL	voel for the element indicated by indx1 (output).
Real AVOEL (INDX2-INDX1+1)	voel; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	voel (input).
Integer INDX2	voel, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.221. GET_VOLT

Use this API to return the bus complex p.u. voltage in rectangular coordinates .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
volt = get_volt(indx1)

avolt = get_volt(indx1, indx2)
```

Fortran command syntax:

```
VOLT = GET_VOLT(INDX1)

AVOLT = GET_VOLT(INDX1, INDX2)
```

Where:

Complex VOLT	bus complex p.u. voltage in rectangular coordinates for the element indicated by indx1 (output).
Complex AVOLT (INDX2-INDX1+1)	bus complex p.u. voltage in rectangular coordinates ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	bus array index (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.222. GET_VOTHSG

Use this API to return the vothsg.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vothsg = get_vothsg(indx1)

avothsg = get_vothsg(indx1, indx2)
```

Fortran command syntax:

```
VOTHSG = GET_VOTHSG(INDX1)

AVOTHSG = GET_VOTHSG(INDX1, INDX2)
```

Where:

Real VOTHSG	vothsg for the element indicated by indx1 (output).
Real AVOTHSG (INDX2-INDX1+1)	vothsg; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vothsg (input).
Integer INDX2	vothsg, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.223. GET_VREF

Use this API to return the vref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vref = get_vref(indx1)

avref = get_vref(indx1, indx2)
```

Fortran command syntax:

```
VREF = GET_VREF(INDX1)

AVREF = GET_VREF(INDX1, INDX2)
```

Where:

Real VREF	vref for the element indicated by indx1 (output).
Real AVREF (INDX2-INDX1+1)	vref; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vref (input).
Integer INDX2	vref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.224. GET_VRFDBK

Use this API to return the vrfdbk.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vrfdbk = get_vrfdbk(indx1)

avrfdbk = get_vrfdbk(indx1, indx2)
```

Fortran command syntax:

```
VRFDBK = GET_VRFDBK(INDX1)

AVRFDBK = GET_VRFDBK(INDX1, INDX2)
```

Where:

Real VRFDBK	vrfdbk for the element indicated by indx1 (output).
Real AVRFDBK (INDX2-INDX1+1)	vrfdbk; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vrfdbk (input).
Integer INDX2	vrfdbk, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.225. GET_VSCACS

Use this API to return the vscdc converter power weighting factor fraction.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscacs = get_vscacs(cnvidx, indx)

avscacs = get_vscacs(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCACS = GET_VSCACS(CNVIDX, INDX)

AVSCACS = GET_VSCACS(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCACS	vscdc converter power weighting factor fraction for the element indicated by indx1 (output).
Real AVSCACS ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter power weighting factor fraction; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.226. GET_VSCCNV

Use this API to return the vscdc converter reactive power seen from ac network.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vsccnv = get_vsccnv(cnvidx, indx)

avsccnv = get_vsccnv(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCCNV = GET_VSCCNV(CNVIDX, INDX)

AVSCCNV = GET_VSCCNV(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer VSCCNV	vscdc converter reactive power seen from ac network for the element indicated by indx1 (output).
Integer AVSCCNV ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter reactive power seen from ac network; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.227. GET_VSCDCS

Use this API to return the vscdc converter converter ac current rating.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscdcs = get_vscdcs(cnvidx, indx)

avscdcs = get_vscdcs(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCDCS = GET_VSCDCS(CNVIDX, INDX)

AVSCDCS = GET_VSCDCS(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCDCS	vscdc converter converter ac current rating for the element indicated by indx1 (output).
Real AVSCDCS ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter converter ac current rating; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.228. GET_VSCHED

Use this API to return the two terminal dc line scheduled compounded dc voltage in kV.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vsched = get_vsched(indx1)

avsched = get_vsched(indx1, indx2)
```

Fortran command syntax:

```
VSCHED = GET_VSCHED(INDX1)

AVSCHED = GET_VSCHED(INDX1, INDX2)
```

Where:

Real VSCHED	two terminal dc line scheduled compounded dc voltage in kV for the element indicated by indx1 (output).
Real AVSCHED (INDX2-INDX1+1)	two terminal dc line scheduled compounded dc voltage in kV; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.229. GET_VSCIDC

Use this API to return the vscdc line current (amps).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscidc = get_vscidc(indx1)

avscidc = get_vscidc(indx1, indx2)
```

Fortran command syntax:

```
VSCIDC = GET_VSCIDC(INDX1)

AVSCIDC = GET_VSCIDC(INDX1, INDX2)
```

Where:

Real VSCIDC	vscdc line current (amps) for the element indicated by indx1 (output).
Real AVSCIDC (INDX2-INDX1+1)	vscdc line current (amps); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vscdc line array index (input).
Integer INDX2	vscdc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.230. GET_VSCIMX

Use this API to return the vscdc converter ac current rating.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscimx = get_vscimx(cnvidx, indx)

avscimx = get_vscimx(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCIMX = GET_VSCIMX(CNVIDX, INDX)

AVSCIMX = GET_VSCIMX(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCIMX	vscdc converter ac current rating for the element indicated by indx1 (output).
Real AVSCIMX ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter ac current rating; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.231. GET_VSCLSA

Use this API to return the vscdc converter reactive power upper limit in mvar.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vsclsa = get_vsclsa(cnvidx, indx)

avsclsa = get_vsclsa(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCLSA = GET_VSCLSA(CNVIDX, INDX)

AVSCLSA = GET_VSCLSA(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCLSA	vscdc converter reactive power upper limit in mvar for the element indicated by indx1 (output).
Real AVSCLSA ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter reactive power upper limit in mvar; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.232. GET_VSCLSB

Use this API to return the vscdc converter reactive power lower limit in mvar.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vsclsb = get_vsclsb(cnvidx, indx)

avsclsb = get_vsclsb(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCLSB = GET_VSCLSB(CNVIDX, INDX)

AVSCLSB = GET_VSCLSB(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCLSB	vscdc converter reactive power lower limit in mvar for the element indicated by indx1 (output).
Real AVSCLSB ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter reactive power lower limit in mvar; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.233. GET_VSCLSM

Use this API to return the vscdc converter minimum converter losses.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vsclsm = get_vsclsm(cnvidx, indx)

avsclsm = get_vsclsm(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCLSM = GET_VSCLSM(CNVIDX, INDX)

AVSCLSM = GET_VSCLSM(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCLSM	vscdc converter minimum converter losses for the element indicated by indx1 (output).
Real AVSCLSM ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter minimum converter losses; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.234. GET_VSCMDC

Use this API to return the vscdc line dc control mode.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscmdc = get_vscmdc(indx1)

avscmdc = get_vscmdc(indx1, indx2)
```

Fortran command syntax:

```
VSCMDC = GET_VSCMDC(INDX1)

AVSCMDC = GET_VSCMDC(INDX1, INDX2)
```

Where:

Integer VSCMDC	vscdc line dc control mode for the element indicated by indx1 (output).
Integer AVSCMDC (INDX2-INDX1+1)	vscdc line dc control mode; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vscdc line array index (input).
Integer INDX2	vscdc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.235. GET_VSCMOD

Use this API to return the vscdc converter reactive power seen from ac network at cong.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscmod = get_vscmod(cnvidx, indx)

avscmod = get_vscmod(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCMOD = GET_VSCMOD(CNVIDX, INDX)

AVSCMOD = GET_VSCMOD(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer VSCMOD	vscdc converter reactive power seen from ac network at cong for the element indicated by indx1 (output).
Integer AVSCMOD ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter reactive power seen from ac network at cong; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.236. GET_VSCNAM

Use this API to return the vscdc converter ac control mode: 1 = ac voltage control, 2 = fixed ac power factor.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscnam = get_vscnam(indx1)

avscnam = get_vscnam(indx1, indx2)
```

Fortran command syntax:

```
VSCNAM = GET_VSCNAM(INDX1)

AVSCNAM = GET_VSCNAM(INDX1, INDX2)
```

Where:

Character*12 VSCNAM	vscdc converter ac control mode: 1 = ac voltage control, 2 = fixed ac power factor for the element indicated by indx1 (output).
Character*12 AVSCNAM (INDX2-INDX1+1)	vscdc converter ac control mode: 1 = ac voltage control, 2 = fixed ac power factor; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vscdc line array index (input).
Integer INDX2	vscdc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.237. GET_VSCPAC

Use this API to return the vscdc converter a loss coefficient (kw).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscpac = get_vscpac(cnvidx, indx)

avscpac = get_vscpac(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCPAC = GET_VSCPAC(CNVIDX, INDX)

AVSCPAC = GET_VSCPAC(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCPAC	vscdc converter a loss coefficient (kw) for the element indicated by indx1 (output).
Real AVSCPAC ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter a loss coefficient (kw); an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.238. GET_VSCPSV

Use this API to return the vscdc converter minimum converter losses.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscpsv = get_vscpsv(cnvidx, indx)

avscpsv = get_vscpsv(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCPSV = GET_VSCPSV(CNVIDX, INDX)

AVSCPSV = GET_VSCPSV(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCPSV	vscdc converter minimum converter losses for the element indicated by indx1 (output).
Real AVSCPSV ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter minimum converter losses; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.239. GET_VSCPWF

Use this API to return the vscdc converter power weighting factor fraction.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscpwf = get_vscpwf(cnvidx, indx)

avscpwf = get_vscpwf(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCPWF = GET_VSCPWF(CNVIDX, INDX)

AVSCPWF = GET_VSCPWF(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCPWF	vscdc converter power weighting factor fraction for the element indicated by indx1 (output).
Real AVSCPWF ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter power weighting factor fraction; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.240. GET_VSCQAC

Use this API to return the vscdc converter b loss coefficient (kw/amp).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscqac = get_vscqac(cnvidx, indx)

avscqac = get_vscqac(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCQAC = GET_VSCQAC(CNVIDX, INDX)

AVSCQAC = GET_VSCQAC(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCQAC	vscdc converter b loss coefficient (kw/amp) for the element indicated by indx1 (output).
Real AVSCQAC ((CNVIDX2-CN-VIDX+1)*(INDX2-INDX+1))	vscdc converter b loss coefficient (kw/amp); an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.241. GET_VSCQMN

Use this API to return the vscdc converter reactive power lower limit in MVAR.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscqmn = get_vscqmn(cnvidx, indx)

avscqmn = get_vscqmn(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCQMN = GET_VSCQMN(CNVIDX, INDX)

AVSCQMN = GET_VSCQMN(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCQMN	vscdc converter reactive power lower limit in MVAR for the element indicated by indx1 (output).
Real AVSCQMN ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter reactive power lower limit in MVAR; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.242. GET_VSCQMX

Use this API to return the vscdc converter reactive power upper limit in MVAR.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscqmx = get_vscqmx(cnvidx, indx)

avscqmx = get_vscqmx(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCQMX = GET_VSCQMX(CNVIDX, INDX)

AVSCQMX = GET_VSCQMX(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCQMX	vscdc converter reactive power upper limit in MVAR for the element indicated by indx1 (output).
Real AVSCQMX ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter reactive power upper limit in MVAR; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.243. GET_VSCQSV

Use this API to return the vscdc converter converter mva rating.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscqsv = get_vscqsv(cnvidx, indx)

avscqsv = get_vscqsv(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCQSV = GET_VSCQSV(CNVIDX, INDX)

AVSCQSV = GET_VSCQSV(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCQSV	vscdc converter converter mva rating for the element indicated by indx1 (output).
Real AVSCQSV ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter converter mva rating; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.244. GET_VSCRDC

Use this API to return the vscdc line resistance (ohm).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscrdc = get_vscrdc(indx1)

avscrdc = get_vscrdc(indx1, indx2)
```

Fortran command syntax:

```
VSCRDC = GET_VSCRDC(INDX1)

AVSCRDC = GET_VSCRDC(INDX1, INDX2)
```

Where:

Real VSCRDC	vscdc line resistance (ohm) for the element indicated by indx1 (output).
Real AVSCRDC (INDX2-INDX1+1)	vscdc line resistance (ohm); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vscdc line array index (input).
Integer INDX2	vscdc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.245. GET_VSCREM

Use this API to return the vscdc converter converter dc setpoint (kv or mw).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscrem = get_vscrem(cnvidx, indx)

avscrem = get_vscrem(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCREM = GET_VSCREM(CNVIDX, INDX)

AVSCREM = GET_VSCREM(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer VSCREM	vscdc converter converter dc setpoint (kv or mw) for the element indicated by indx1 (output).
Integer AVSCREM ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter converter dc setpoint (kv or mw); an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.246. GET_VSCSIG

Use this API to return the vscsig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
vscsig = get_vscsig(one, two)

avscsig = get_vscsig(one, one2, two, two2)
```

Fortran command syntax:

```
VSCSIG = GET_VSCSIG(ONE, TWO)

AVSCSIG = GET_VSCSIG(ONE, ONE2, TWO, TWO2)
```

Where:

Real VSCSIG	vscsig for the element indicated by indx1 (output).
Real AVSCSIG ((ONE2-ONE +1)*(TWO2-TWO+1))	vscsig; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	vscsig (input).
Integer ONE2	vscsig, upper value of range (input).
Integer TWO	vscsig (input).
Integer TWO2	vscsig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.247. GET_VSCSMX

Use this API to return the vscdc converter MVA rating.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vscsmx = get_vscsmx(cnvidx, indx)

avscsmx = get_vscsmx(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCSMX = GET_VSCSMX(CNVIDX, INDX)

AVSCSMX = GET_VSCSMX(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCSMX	vscdc converter MVA rating for the element indicated by indx1 (output).
Real AVSCSMX ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter MVA rating; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.248. GET_VSCTYP

Use this API to return the vscdc converter active power seen from ac network at cong.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vsctyp = get_vsctyp(cnvidx, indx)

avsctyp = get_vsctyp(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
VSCTYP = GET_VSCTYP(CNVIDX, INDX)

AVSCTYP = GET_VSCTYP(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer VSCTYP	vscdc converter active power seen from ac network at cong for the element indicated by indx1 (output).
Integer AVSCTYP ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter active power seen from ac network at cong; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.249. GET_VSHNHI

Use this API to return the switched shunt high voltage setpoint.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vshnhi = get_vshnhi(indx1)

avshnhi = get_vshnhi(indx1, indx2)
```

Fortran command syntax:

```
VSHNHI = GET_VSHNHI(INDX1)

AVSHNHI = GET_VSHNHI(INDX1, INDX2)
```

Where:

Real VSHNHI	switched shunt high voltage setpoint for the element indicated by indx1 (output).
Real AVSHNHI (INDX2-INDX1+1)	switched shunt high voltage setpoint; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	switched shunt array index (input).
Integer INDX2	switched shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.250. GET_VSHNLO

Use this API to return the switched shunt low voltage setpoint.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vshnlo = get_vshnlo(indx1)

avshnlo = get_vshnlo(indx1, indx2)
```

Fortran command syntax:

```
VSHNLO = GET_VSHNLO(INDX1)

AVSHNLO = GET_VSHNLO(INDX1, INDX2)
```

Where:

Real VSHNLO	switched shunt low voltage setpoint for the element indicated by indx1 (output).
Real AVSHNLO (INDX2-INDX1+1)	switched shunt low voltage setpoint; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	switched shunt array index (input).
Integer INDX2	switched shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.251. GET_VUEL

Use this API to return the vuel.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vuel = get_vuel(indx1)

avuel = get_vuel(indx1, indx2)
```

Fortran command syntax:

```
VUEL = GET_VUEL(INDX1)

AVUEL = GET_VUEL(INDX1, INDX2)
```

Where:

Real VUEL	vuel for the element indicated by indx1 (output).
Real AVUEL (INDX2-INDX1+1)	vuel; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	vuel (input).
Integer INDX2	vuel, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.252. GET_W0

Use this API to return the system base frequency (radian/s).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
w0 = get_w0()
```

Fortran command syntax:

```
W0 = GET_W0()
```

Where:

Real W0	system base frequency (radian/s) (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.253. GET_WAEROT

Use this API to return the waerot.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
waerot = get_waerot(indx1)

awaerot = get_waerot(indx1, indx2)
```

Fortran command syntax:

```
WAEROT = GET_WAEROT(INDX1)

AWAEROT = GET_WAEROT(INDX1, INDX2)
```

Where:

Real WAEROT	waerot for the element indicated by indx1 (output).
Real AWAEROT (INDX2-INDX1+1)	waerot; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	waerot (input).
Integer INDX2	waerot, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.254. GET_WAUXSG

Use this API to return the wauxsg.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wauxsg = get_wauxsg(indx1)

awauxsg = get_wauxsg(indx1, indx2)
```

Fortran command syntax:

```
WAUXSG = GET_WAUXSG(INDX1)

AWAUXSG = GET_WAUXSG(INDX1, INDX2)
```

Where:

Real WAUXSG	wauxsg for the element indicated by indx1 (output).
Real AWAUXSG (INDX2-INDX1+1)	wauxsg; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wauxsg (input).
Integer INDX2	wauxsg, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.255. GET_WEQCMD

Use this API to return the weqcmd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
weqcmd = get_weqcmd(indx1)

aweqcmd = get_weqcmd(indx1, indx2)
```

Fortran command syntax:

```
WEQCMD = GET_WEQCMD(INDX1)

AWEQCMD = GET_WEQCMD(INDX1, INDX2)
```

Where:

Real WEQCMD	weqcmd for the element indicated by indx1 (output).
Real AWEQCMD (INDX2-INDX1+1)	weqcmd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	weqcmd (input).
Integer INDX2	weqcmd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.256. GET_WINDRP

Use this API to return the transformer winding 1 tap ratio .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
windrp = get_windrp(indx1)

awindrp = get_windrp(indx1, indx2)
```

Fortran command syntax:

```
WINDRP = GET_WINDRP(INDX1)

AWINDRP = GET_WINDRP(INDX1, INDX2)
```

Where:

Real WINDRP	transformer winding 1 tap ratio for the element indicated by indx1 (output).
Real AWINDRP (INDX2-INDX1+1)	transformer winding 1 tap ratio ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.257. GET_WINDRS

Use this API to return the transformer winding 2 tap ratio .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
windrs = get_windrs(indx1)

awindrs = get_windrs(indx1, indx2)
```

Fortran command syntax:

```
WINDRS = GET_WINDRS(INDX1)

AWINDRS = GET_WINDRS(INDX1, INDX2)
```

Where:

Real WINDRS	transformer winding 2 tap ratio for the element indicated by indx1 (output).
Real AWINDRS (INDX2-INDX1+1)	transformer winding 2 tap ratio ; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.258. GET_WIPCMD

Use this API to return the wipcmd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wipcmd = get_wipcmd(indx1)

awipcmd = get_wipcmd(indx1, indx2)
```

Fortran command syntax:

```
WIPCMD = GET_WIPCMD(INDX1)

AWIPCMD = GET_WIPCMD(INDX1, INDX2)
```

Where:

Real WIPCMD	wipcmd for the element indicated by indx1 (output).
Real AWIPCMD (INDX2-INDX1+1)	wipcmd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wipcmd (input).
Integer INDX2	wipcmd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.259. GET_WIQCMD

Use this API to return the wiqcmd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wiqcmd = get_wiqcmd(indx1)

awiqcmd = get_wiqcmd(indx1, indx2)
```

Fortran command syntax:

```
WIQCMD = GET_WIQCMD(INDX1)

AWIQCMD = GET_WIQCMD(INDX1, INDX2)
```

Where:

Real WIQCMD	wiqcmd for the element indicated by indx1 (output).
Real AWIQCMD (INDX2-INDX1+1)	wiqcmd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wiqcmd (input).
Integer INDX2	wiqcmd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.260. GET_WMCARYS

Use this API to return the wmcarys.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wmcarys = get_wmcarys(one, two)

awmcarys = get_wmcarys(one, one2, two, two2)
```

Fortran command syntax:

```
WMCARYS = GET_WMCARYS(ONE, TWO)

AWMCARYS = GET_WMCARYS(ONE, ONE2, TWO, TWO2)
```

Where:

Real WMCARYS	wmcarys for the element indicated by indx1 (output).
Real AWMCARYS ((ONE2-ONE +1)*(TWO2-TWO+1))	wmcarys; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	wmcarys (input).
Integer ONE2	wmcarys, upper value of range (input).
Integer TWO	wmcarys (input).
Integer TWO2	wmcarys, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.261. GET_WNDNUM

Use this API to return the renewable machine index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndnum = get_wndnum(indx1)

awndnum = get_wndnum(indx1, indx2)
```

Fortran command syntax:

```
WNDNUM = GET_WNDNUM(INDX1)

AWNDNUM = GET_WNDNUM(INDX1, INDX2)
```

Where:

Integer WNDNUM	renewable machine index for the element indicated by indx1 (output).
Integer AWNDNUM (INDX2-INDX1+1)	renewable machine index; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.262. GET_WNDSP1

Use this API to return the wndsp1.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp1 = get_wndsp1(indx1)

awndsp1 = get_wndsp1(indx1, indx2)
```

Fortran command syntax:

```
WNDSP1 = GET_WNDSP1(INDX1)

AWNDSP1 = GET_WNDSP1(INDX1, INDX2)
```

Where:

Real WNDSP1	wndsp1 for the element indicated by indx1 (output).
Real AWNDSP1 (INDX2-INDX1+1)	wndsp1; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp1 (input).
Integer INDX2	wndsp1, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.263. GET_WNDSP10

Use this API to return the wndsp10.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp10 = get_wndsp10(indx1)

awndsp10 = get_wndsp10(indx1, indx2)
```

Fortran command syntax:

```
WNDSP10 = GET_WNDSP10(INDX1)

AWNDSP10 = GET_WNDSP10(INDX1, INDX2)
```

Where:

Real WNDSP10	wndsp10 for the element indicated by indx1 (output).
Real AWNDSP10 (INDX2-INDX1+1)	wndsp10; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp10 (input).
Integer INDX2	wndsp10, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.264. GET_WNDSP2

Use this API to return the wndsp2.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp2 = get_wndsp2(indx1)

awndsp2 = get_wndsp2(indx1, indx2)
```

Fortran command syntax:

```
WNDSP2 = GET_WNDSP2(INDX1)

AWNDSP2 = GET_WNDSP2(INDX1, INDX2)
```

Where:

Real WNDSP2	wndsp2 for the element indicated by indx1 (output).
Real AWNDSP2 (INDX2-INDX1+1)	wndsp2; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp2 (input).
Integer INDX2	wndsp2, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.265. GET_WNDSP3

Use this API to return the wndsp3.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp3 = get_wndsp3(indx1)

awndsp3 = get_wndsp3(indx1, indx2)
```

Fortran command syntax:

```
WNDSP3 = GET_WNDSP3(INDX1)

AWNDSP3 = GET_WNDSP3(INDX1, INDX2)
```

Where:

Real WNDSP3	wndsp3 for the element indicated by indx1 (output).
Real AWNDSP3 (INDX2-INDX1+1)	wndsp3; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp3 (input).
Integer INDX2	wndsp3, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.266. GET_WNDSP4

Use this API to return the wndsp4.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp4 = get_wndsp4(indx1)

awndsp4 = get_wndsp4(indx1, indx2)
```

Fortran command syntax:

```
WNDSP4 = GET_WNDSP4(INDX1)

AWNDSP4 = GET_WNDSP4(INDX1, INDX2)
```

Where:

Real WNDSP4	wndsp4 for the element indicated by indx1 (output).
Real AWNDSP4 (INDX2-INDX1+1)	wndsp4; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp4 (input).
Integer INDX2	wndsp4, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.267. GET_WNDSP5

Use this API to return the wndsp5.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp5 = get_wndsp5(indx1)

awndsp5 = get_wndsp5(indx1, indx2)
```

Fortran command syntax:

```
WNDSP5 = GET_WNDSP5(INDX1)

AWNDSP5 = GET_WNDSP5(INDX1, INDX2)
```

Where:

Real WNDSP5	wndsp5 for the element indicated by indx1 (output).
Real AWNDSP5 (INDX2-INDX1+1)	wndsp5; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp5 (input).
Integer INDX2	wndsp5, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.268. GET_WNDSP6

Use this API to return the wndsp6.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp6 = get_wndsp6(indx1)

awndsp6 = get_wndsp6(indx1, indx2)
```

Fortran command syntax:

```
WNDSP6 = GET_WNDSP6(INDX1)

AWNDSP6 = GET_WNDSP6(INDX1, INDX2)
```

Where:

Real WNDSP6	wndsp6 for the element indicated by indx1 (output).
Real AWNDSP6 (INDX2-INDX1+1)	wndsp6; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp6 (input).
Integer INDX2	wndsp6, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.269. GET_WNDSP7

Use this API to return the wndsp7.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp7 = get_wndsp7(indx1)

awndsp7 = get_wndsp7(indx1, indx2)
```

Fortran command syntax:

```
WNDSP7 = GET_WNDSP7(INDX1)

AWNDSP7 = GET_WNDSP7(INDX1, INDX2)
```

Where:

Real WNDSP7	wndsp7 for the element indicated by indx1 (output).
Real AWNDSP7 (INDX2-INDX1+1)	wndsp7; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp7 (input).
Integer INDX2	wndsp7, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.270. GET_WNDSP8

Use this API to return the wndsp8.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp8 = get_wndsp8(indx1)

awndsp8 = get_wndsp8(indx1, indx2)
```

Fortran command syntax:

```
WNDSP8 = GET_WNDSP8(INDX1)

AWNDSP8 = GET_WNDSP8(INDX1, INDX2)
```

Where:

Real WNDSP8	wndsp8 for the element indicated by indx1 (output).
Real AWNDSP8 (INDX2-INDX1+1)	wndsp8; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp8 (input).
Integer INDX2	wndsp8, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.271. GET_WNDSP9

Use this API to return the wndsp9.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wndsp9 = get_wndsp9(indx1)

awndsp9 = get_wndsp9(indx1, indx2)
```

Fortran command syntax:

```
WNDSP9 = GET_WNDSP9(INDX1)

AWNDSP9 = GET_WNDSP9(INDX1, INDX2)
```

Where:

Real WNDSP9	wndsp9 for the element indicated by indx1 (output).
Real AWNDSP9 (INDX2-INDX1+1)	wndsp9; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wndsp9 (input).
Integer INDX2	wndsp9, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.272. GET_WPCMND

Use this API to return the wpcmnd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wpcmnd = get_wpcmnd(indx1)

awpcmnd = get_wpcmnd(indx1, indx2)
```

Fortran command syntax:

```
WPCMND = GET_WPCMND(INDX1)

AWPCMND = GET_WPCMND(INDX1, INDX2)
```

Where:

Real WPCMND	wpcmnd for the element indicated by indx1 (output).
Real AWPCMND (INDX2-INDX1+1)	wpcmnd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wpcmnd (input).
Integer INDX2	wpcmnd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.273. GET_WPITCH

Use this API to return the wpitch.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wpitch = get_wpitch(indx1)

awpitch = get_wpitch(indx1, indx2)
```

Fortran command syntax:

```
WPITCH = GET_WPITCH(INDX1)

AWPITCH = GET_WPITCH(INDX1, INDX2)
```

Where:

Real WPITCH	wpitch for the element indicated by indx1 (output).
Real AWPITCH (INDX2-INDX1+1)	wpitch; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wpitch (input).
Integer INDX2	wpitch, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.274. GET_WPORDR

Use this API to return the wpordr.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wpordr = get_wpordr(indx1)

awpordr = get_wpordr(indx1, indx2)
```

Fortran command syntax:

```
WPORDR = GET_WPORDR(INDX1)

AWPORDR = GET_WPORDR(INDX1, INDX2)
```

Where:

Real WPORDR	wpordr for the element indicated by indx1 (output).
Real AWPORDR (INDX2-INDX1+1)	wpordr; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wpordr (input).
Integer INDX2	wpordr, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.275. GET_WPRATE

Use this API to return the wprate.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wprate = get_wprate(indx1)

awprate = get_wprate(indx1, indx2)
```

Fortran command syntax:

```
WPRATE = GET_WPRATE(INDX1)

AWPRATE = GET_WPRATE(INDX1, INDX2)
```

Where:

Real WPRATE	wprate for the element indicated by indx1 (output).
Real AWPRATE (INDX2-INDX1+1)	wprate; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wprate (input).
Integer INDX2	wprate, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.276. GET_WPREFPTQ

Use this API to return the wprefptq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wprefptq = get_wprefptq(indx1)

awprefptq = get_wprefptq(indx1, indx2)
```

Fortran command syntax:

```
WPREFPTQ = GET_WPREFPTQ(INDX1)

AWPREFPTQ = GET_WPREFPTQ(INDX1, INDX2)
```

Where:

Real WPREFPTQ	wprefptq for the element indicated by indx1 (output).
Real AWPREFPTQ (INDX2-INDX1+1)	wprefptq; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wprefptq (input).
Integer INDX2	wprefptq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.277. GET_WPREFWGO

Use this API to return the wprefwgo.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wprefwgo = get_wprefwgo(indx1)

awprefwgo = get_wprefwgo(indx1, indx2)
```

Fortran command syntax:

```
WPREFWGO = GET_WPREFWGO(INDX1)

AWPREFWGO = GET_WPREFWGO(INDX1, INDX2)
```

Where:

Real WPREFWGO	wprefwgo for the element indicated by indx1 (output).
Real AWPREFWGO (INDX2-INDX1+1)	wprefwgo; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wprefwgo (input).
Integer INDX2	wprefwgo, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.278. GET_WQCMND

Use this API to return the wqcmnd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wqcmnd = get_wqcmnd(indx1)

awqcmnd = get_wqcmnd(indx1, indx2)
```

Fortran command syntax:

```
WQCMND = GET_WQCMND(INDX1)

AWQCMND = GET_WQCMND(INDX1, INDX2)
```

Where:

Real WQCMND	wqcmnd for the element indicated by indx1 (output).
Real AWQCMND (INDX2-INDX1+1)	wqcmnd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wqcmnd (input).
Integer INDX2	wqcmnd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.279. GET_WROTID

Use this API to return the wrotid.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wrotid = get_wrotid(indx1)

awrotid = get_wrotid(indx1, indx2)
```

Fortran command syntax:

```
WROTID = GET_WROTID(INDX1)

AWROTID = GET_WROTID(INDX1, INDX2)
```

Where:

Real WROTID	wrotid for the element indicated by indx1 (output).
Real AWROTID (INDX2-INDX1+1)	wrotid; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wrotid (input).
Integer INDX2	wrotid, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.280. GET_WROTIQ

Use this API to return the wrotiq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wrotiq = get_wrotiq(indx1)

awrotiq = get_wrotiq(indx1, indx2)
```

Fortran command syntax:

```
WROTIQ = GET_WROTIQ(INDX1)

AWROTIQ = GET_WROTIQ(INDX1, INDX2)
```

Where:

Real WROTIQ	wrotiq for the element indicated by indx1 (output).
Real AWROTIQ (INDX2-INDX1+1)	wrotiq; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wrotiq (input).
Integer INDX2	wrotiq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.281. GET_WROTRI

Use this API to return the wrotri.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wrotri = get_wrotri(indx1)

awrotri = get_wrotri(indx1, indx2)
```

Fortran command syntax:

```
WROTRI = GET_WROTRI(INDX1)

AWROTRI = GET_WROTRI(INDX1, INDX2)
```

Where:

Real WROTRI	wrotri for the element indicated by indx1 (output).
Real AWROTRI (INDX2-INDX1+1)	wrotri; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wrotri (input).
Integer INDX2	wrotri, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.282. GET_WROTRV

Use this API to return the wrotrv.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wrotrv = get_wrotrv(indx1)

awrotrv = get_wrotrv(indx1, indx2)
```

Fortran command syntax:

```
WROTRV = GET_WROTRV(INDX1)

AWROTRV = GET_WROTRV(INDX1, INDX2)
```

Where:

Real WROTRV	wrotrv for the element indicated by indx1 (output).
Real AWROTRV (INDX2-INDX1+1)	wrotrv; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wrotrv (input).
Integer INDX2	wrotrv, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.283. GET_WROTVD

Use this API to return the wrotvd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wrotvd = get_wrotvd(indx1)

awrotvd = get_wrotvd(indx1, indx2)
```

Fortran command syntax:

```
WROTVD = GET_WROTVD(INDX1)

AWROTVD = GET_WROTVD(INDX1, INDX2)
```

Where:

Real WROTVD	wrotvd for the element indicated by indx1 (output).
Real AWROTVD (INDX2-INDX1+1)	wrotvd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wrotvd (input).
Integer INDX2	wrotvd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.284. GET_WROTVQ

Use this API to return the wrotvq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wrotvq = get_wrotvq(indx1)

awrotvq = get_wrotvq(indx1, indx2)
```

Fortran command syntax:

```
WROTVQ = GET_WROTVQ(INDX1)

AWROTVQ = GET_WROTVQ(INDX1, INDX2)
```

Where:

Real WROTVQ	wrotvq for the element indicated by indx1 (output).
Real AWROTVQ (INDX2-INDX1+1)	wrotvq; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wrotvq (input).
Integer INDX2	wrotvq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.285. GET_WSTRTIN

Use this API to return the wstrtin.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wstrtin = get_wstrtin(one, two)

awstrtin = get_wstrtin(one, one2, two, two2)
```

Fortran command syntax:

```
WSTRTIN = GET_WSTRTIN(ONE, TWO)

AWSTRTIN = GET_WSTRTIN(ONE, ONE2, TWO, TWO2)
```

Where:

Integer WSTRTIN	wstrtin for the element indicated by indx1 (output).
Integer AWSTRTIN ((ONE2-ONE +1)*(TWO2-TWO+1))	wstrtin; an array of values for the elements indicated by the indices supplied (output).
Integer ONE	wstrtin (input).
Integer ONE2	wstrtin, upper value of range (input).
Integer TWO	wstrtin (input).
Integer TWO2	wstrtin, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.286. GET_WTRBSP

Use this API to return the wtrbsp.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wtrbsp = get_wtrbsp(indx1)

awtrbsp = get_wtrbsp(indx1, indx2)
```

Fortran command syntax:

```
WTRBSP = GET_WTRBSP(INDX1)

AWTRBSP = GET_WTRBSP(INDX1, INDX2)
```

Where:

Real WTRBSP	wtrbsp for the element indicated by indx1 (output).
Real AWTRBSP (INDX2-INDX1+1)	wtrbsp; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wtrbsp (input).
Integer INDX2	wtrbsp, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.287. GET_WVLCTY

Use this API to return the wvlcty.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
wvlcty = get_wvlcty(indx1)

awvlcty = get_wvlcty(indx1, indx2)
```

Fortran command syntax:

```
WVLCTY = GET_WVLCTY(INDX1)

AWVLCTY = GET_WVLCTY(INDX1, INDX2)
```

Where:

Real WVLCTY	wvlcty for the element indicated by indx1 (output).
Real AWVLCTY (INDX2-INDX1+1)	wvlcty; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	wvlcty (input).
Integer INDX2	wvlcty, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.288. GET_XADIFD

Use this API to return the xadifd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
xadifd = get_xadifd(indx1)

axadifd = get_xadifd(indx1, indx2)
```

Fortran command syntax:

```
XADIFD = GET_XADIFD(INDX1)

AXADIFD = GET_XADIFD(INDX1, INDX2)
```

Where:

Real XADIFD	xadifd for the element indicated by indx1 (output).
Real AXADIFD (INDX2-INDX1+1)	xadifd; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	xadifd (input).
Integer INDX2	xadifd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.289. GET_XCI

Use this API to return the two terminal dc line inverter commutating transformer reactance per bridge in ohms.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
xci = get_xci(indx1)

axci = get_xci(indx1, indx2)
```

Fortran command syntax:

```
XCI = GET_XCI(INDX1)

AXCI = GET_XCI(INDX1, INDX2)
```

Where:

Real XCI	two terminal dc line inverter commutating transformer reactance per bridge in ohms for the element indicated by indx1 (output).
Real AXCI (INDX2-INDX1+1)	two terminal dc line inverter commutating transformer reactance per bridge in ohms; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.290. GET_XCONV

Use this API to return the multi-terminal dc line converter commutating reactance.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
xconv = get_xconv(cnvidx, indx)

axconv = get_xconv(cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
XCONV = GET_XCONV(CNVIDX, INDX)

AXCONV = GET_XCONV(CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real XCONV	multi-terminal dc line converter commutating reactance for the element indicated by indx1 (output).
Real AXCONV ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	multi-terminal dc line converter commutating reactance; an array of values for the elements indicated by the indices supplied (output).
Integer CNVIDX	multi-terminal dc line converter index (1:MXCONV) (input).
Integer CNVIDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line converter index (1:MXCONV), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.291. GET_XCR

Use this API to return the two terminal dc line rectifier commutating transformer reactance per bridge in ohms.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
xcr = get_xcr(indx1)

axcr = get_xcr(indx1, indx2)
```

Fortran command syntax:

```
XCR = GET_XCR(INDX1)

AXCR = GET_XCR(INDX1, INDX2)
```

Where:

Real XCR	two terminal dc line rectifier commutating transformer reactance per bridge in ohms for the element indicated by indx1 (output).
Real AXCR (INDX2-INDX1+1)	two terminal dc line rectifier commutating transformer reactance per bridge in ohms; an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	dc line array index (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.292. GET_XTRAN

Use this API to return the machine implicit step-up transformer impedance (p.u. of machine MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
xtran = get_xtran(indx1)

axtran = get_xtran(indx1, indx2)
```

Fortran command syntax:

```
XTRAN = GET_XTRAN(INDX1)

AXTRAN = GET_XTRAN(INDX1, INDX2)
```

Where:

Complex XTRAN	machine implicit step-up transformer impedance (p.u. of machine MVA) for the element indicated by indx1 (output).
Complex AXTRAN (INDX2-INDX1+1)	machine implicit step-up transformer impedance (p.u. of machine MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.293. GET_ZERNUM

Use this API to return the flag to indicate if the branch is a zero impedance line, i.e., ZIL (value be > 0 if the branch is a ZIL).

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
zernum = get_zernum(indx1)

azernum = get_zernum(indx1, indx2)
```

Fortran command syntax:

```
ZERNUM = GET_ZERNUM(INDX1)

AZERNUM = GET_ZERNUM(INDX1, INDX2)
```

Where:

Integer ZERNUM	flag to indicate if the branch is a zero impedance line, i.e., ZIL (value be > 0 if the branch is a ZIL) for the element indicated by indx1 (output).
Integer AZERNUM (INDX2-INDX1+1)	flag to indicate if the branch is a zero impedance line, i.e., ZIL (value be > 0 if the branch is a ZIL); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	branch array index (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.2.294. GET_ZSORCE

Use this API to return the machine source impedance (p.u. of machine MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
zsorce = get_zsorce(indx1)

azsorce = get_zsorce(indx1, indx2)
```

Fortran command syntax:

```
ZSORCE = GET_ZSORCE(INDX1)

AZSORCE = GET_ZSORCE(INDX1, INDX2)
```

Where:

Complex ZSORCE	machine source impedance (p.u. of machine MVA) for the element indicated by indx1 (output).
Complex AZSORCE (INDX2-INDX1+1)	machine source impedance (p.u. of machine MVA); an array of values for the elements indicated by the indices supplied (output).
Integer INDX1	machine array index (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3. Dynamics model value setting routines

This section describes the routines that can set the current values of internal variables. For those variables that are arrays, single values or array slices may be specified.

24.3.1. SET_ANGLE

Use this API to set the angle.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_angle(angle, indx1)

set_angle(aangle, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_ANGLE(ANGLE, INDX1)

CALL SET_ANGLE(AANGLE, INDX1, INDX2)
```

Where:

Real ANGLE	angle for the element indicated by indx1 (input).
Real AANGLE (INDX2-INDX1+1)	angle; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	angle, lower value of range (input).
Integer INDX2	angle, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.2. SET_BSWICH

Use this API to set the actual total shunt admittance presently switched on at the bus in MVA.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
set_bswich(bswich, indx1)

set_bswich(abswich, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_BSWICH(BSWICH, INDX1)

CALL SET_BSWICH(ABSWICH, INDX1, INDX2)
```

Where:

Real BSWICH	actual total shunt admittance presently switched on at the bus in MVA for the element indicated by indx1 (input).
Real ABSWICH (INDX2-INDX1+1)	actual total shunt admittance presently switched on at the bus in MVA; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	switched shunt array index, lower value of range (input).
Integer INDX2	switched shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.3. SET_CDGNFR

Use this API to set the distributed generation load quantities in pu of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_cdgnfr(cdgnfr, indx1)

set_cdgnfr(acdgnfr, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_CDGNFR(CDGNFR, INDX1)

CALL SET_CDGNFR(ACDGNFR, INDX1, INDX2)
```

Where:

Complex CDGNFR	distributed generation load quantities in pu of system MVA for the element indicated by indx1 (input).
Complex ACDGNFR (INDX2-INDX1+1)	distributed generation load quantities in pu of system MVA; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	load array index, lower value of range (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.4. SET_CFDLFR

Use this API to set the complex (gross) constant power, current and admittance loads in p.u. of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_cfdlfr(cfdlfr, loadtype, indx)

set_cfdlfr(acfdlfr, loadtype, loadtype2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_CFDLFR(CFDLFR, LOADTYPE, INDX)

CALL SET_CFDLFR(ACFDLFR, LOADTYPE, LOADTYPE2, INDX, INDX2)
```

Where:

Complex CFDLFR	complex (gross) constant power, current and admittance loads in p.u. of system MVA for the element indicated by indx1 (input).
Complex ACFDLFR ((LOADTYPE2-LOADTYPE+1)*(INDX2-INDX+1))	complex (gross) constant power, current and admittance loads in p.u. of system MVA; an array of values for the elements indicated by the indices supplied (input).
Integer LOADTYPE	load component: 1. constant power, 2. constant current, 3. constant impendance (input).
Integer LOADTYPE2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).
Integer INDX	load array index (input).
Integer INDX2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.5. SET_CHRICN

Use this API to set the used to stored character ICON (sized to as many as number of ICONs used by any model).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_chricn(chricn, indx1)

set_chricn(achricn, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_CHRICN(CHRICN, INDX1)

CALL SET_CHRICN(ACHRICN, INDX1, INDX2)
```

Where:

Character*12 CHRICN	used to stored character ICON (sized to as many as number of ICONs used by any model) for the element indicated by indx1 (input).
Character*12 ACHRICN (INDX2-INDX1+1)	used to stored character ICON (sized to as many as number of ICONs used by any model); an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	chricn, lower value of range (input).
Integer INDX2	chricn, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.6. SET_CLODFR

Use this API to set the complex (net) constant power, current and admittance loads in p.u. of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_clodfr(clodfr, loadtype, indx)

set_clodfr(aclodfr, loadtype, loadtype2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_CLODFR(CLODFR, LOADTYPE, INDX)

CALL SET_CLODFR(ACLODFR, LOADTYPE, LOADTYPE2, INDX, INDX2)
```

Where:

Complex CLODFR	complex (net) constant power, current and admittance loads in p.u. of system MVA for the element indicated by indx1 (input).
Complex ACLODFR ((LOAD-TYPE2-LOADTYPE+1)*(INDX2-INDX+1))	complex (net) constant power, current and admittance loads in p.u. of system MVA; an array of values for the elements indicated by the indices supplied (input).
Integer LOADTYPE	load component: 1. constant power, 2. constant current, 3. constant impendance (input).
Integer LOADTYPE2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).
Integer INDX	load array index (input).
Integer INDX2	load component: 1. constant power, 2. constant current, 3. constant impendance, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.7. SET_CON

Use this API to set the real (floating point) model data.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_con(con, indx1)

set_con(acon, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_CON(CON, INDX1)

CALL SET_CON(ACON, INDX1, INDX2)
```

Where:

Real CON	real (floating point) model data for the element indicated by indx1 (input).
Real ACON (INDX2-INDX1+1)	real (floating point) model data; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	con, lower value of range (input).
Integer INDX2	con, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.8. SET_CON_DSCRPT

Use this API to set the con_dscrpt.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_con_dscrpt(con_dscrpt, indx1)

set_con_dscrpt(acon_dscrpt, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_CON_DSCRPT(CON_DSCRPT, INDX1)

CALL SET_CON_DSCRPT(ACON_DSCRPT, INDX1, INDX2)
```

Where:

Character*144 CON_DSCRPT	con_dscrpt for the element indicated by indx1 (input).
Character*144 ACON_DSCRPT (INDX2-INDX1+1)	con_dscrpt; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	con_dscrpt, lower value of range (input).
Integer INDX2	con_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.9. SET_CURNT

Use this API to set the current injection at a bus.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_currnt(currnt, indx1)

set_currnt(acurnt, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_CURNT(CURNT, INDX1)

CALL SET_CURNT(ACURNT, INDX1, INDX2)
```

Where:

Complex CURNT	current injection at a bus for the element indicated by indx1 (input).
Complex ACURNT (INDX2-INDX1+1)	current injection at a bus; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	bus array index, lower value of range (input).
Integer INDX2	bus array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.10. SET_DC2SIG

Use this API to set the dc2sig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
set_dc2sig(dc2sig, one, two)

set_dc2sig(adc2sig, one, one2, two, two2)
```

Fortran command syntax:

```
CALL SET_DC2SIG(DC2SIG, ONE, TWO)

CALL SET_DC2SIG(ADC2SIG, ONE, ONE2, TWO, TWO2)
```

Where:

Real DC2SIG	dc2sig for the element indicated by indx1 (input).
Real ADC2SIG ((ONE2-ONE +1)*(TWO2-TWO+1))	dc2sig; an array of values for the elements indicated by the indices supplied (input).
Integer ONE	dc2sig (input).
Integer ONE2	dc2sig, upper value of range (input).
Integer TWO	dc2sig (input).
Integer TWO2	dc2sig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.11. SET_DCMSIG

Use this API to set the dcmsig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
set_dcmsig(dcmsig, one, two)

set_dcmsig(adcmsig, one, one2, two, two2)
```

Fortran command syntax:

```
CALL SET_DCMSIG(DCMSIG, ONE, TWO)

CALL SET_DCMSIG(ADCMSIG, ONE, ONE2, TWO, TWO2)
```

Where:

Real DCMSIG	dcmsig for the element indicated by indx1 (input).
Real ADCMSIG ((ONE2-ONE +1)*(TWO2-TWO+1))	dcmsig; an array of values for the elements indicated by the indices supplied (input).
Integer ONE	dcmsig (input).
Integer ONE2	dcmsig, upper value of range (input).
Integer TWO	dcmsig (input).
Integer TWO2	dcmsig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.12. SET_DSTATE

Use this API to set the derivative of state variable.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_dstate(dstate, indx1)

set_dstate(adstate, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_DSTATE(DSTATE, INDX1)

CALL SET_DSTATE(ADSTATE, INDX1, INDX2)
```

Where:

Real DSTATE	derivative of state variable for the element indicated by indx1 (input).
Real ADSTATE (INDX2-INDX1+1)	derivative of state variable; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	dstate, lower value of range (input).
Integer INDX2	dstate, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.13. SET_ECOMP

Use this API to set the ecomp.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_ecomp(ecomp, indx1)

set_ecomp(aecomp, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_ECOMP(ECOMP, INDX1)

CALL SET_ECOMP(AECOMP, INDX1, INDX2)
```

Where:

Real ECOMP	ecomp for the element indicated by indx1 (input).
Real AECOMP (INDX2-INDX1+1)	ecomp; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	ecomp, lower value of range (input).
Integer INDX2	ecomp, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.14. SET_EFD

Use this API to set the efd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_efd(efd, indx1)

set_efd(aefd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_EFD(EFD, INDX1)

CALL SET_EFD(AEFD, INDX1, INDX2)
```

Where:

Real EFD	efd for the element indicated by indx1 (input).
Real AEFD (INDX2-INDX1+1)	efd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	efd, lower value of range (input).
Integer INDX2	efd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.15. SET_ETERM

Use this API to set the eterm.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_eterm(eterm, indx1)

set_eterm(aeterm, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_ETERM(ETERM, INDX1)

CALL SET_ETERM(AETERM, INDX1, INDX2)
```

Where:

Real ETERM	eterm for the element indicated by indx1 (input).
Real AETERM (INDX2-INDX1+1)	eterm; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	eterm, lower value of range (input).
Integer INDX2	eterm, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.16. SET_FCTREF

Use this API to set the fctref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_fctref(fctref, indx1)

set_fctref(afctref, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_FCTREF(FCTREF, INDX1)

CALL SET_FCTREF(AFCTREF, INDX1, INDX2)
```

Where:

Real FCTREF	fctref for the element indicated by indx1 (input).
Real AFCTREF (INDX2-INDX1+1)	fctref; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	FACTS device array index, lower value of range (input).
Integer INDX2	FACTS device array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.17. SET_FCTSIG

Use this API to set the fctsig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
set_fctsig(fctsig, one, two)

set_fctsig(afctsig, one, one2, two, two2)
```

Fortran command syntax:

```
CALL SET_FCTSIG(FCTSIG, ONE, TWO)

CALL SET_FCTSIG(AFCTSIG, ONE, ONE2, TWO, TWO2)
```

Where:

Real FCTSIG	fctsig for the element indicated by indx1 (input).
Real AFCTSIG ((ONE2-ONE +1)*(TWO2-TWO+1))	fctsig; an array of values for the elements indicated by the indices supplied (input).
Integer ONE	fctsig (input).
Integer ONE2	fctsig, upper value of range (input).
Integer TWO	fctsig (input).
Integer TWO2	fctsig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.18. SET_GB

Use this API to set the negative reciprocal of the actual branch or transformer impedance in p.u. of system MVA.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_gb(gb, indx1)

set_gb(agb, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_GB(GB, INDX1)

CALL SET_GB(AGB, INDX1, INDX2)
```

Where:

Complex GB	negative reciprocal of the actual branch or transformer impedance in p.u. of system MVA for the element indicated by indx1 (input).
Complex AGB (INDX2-INDX1+1)	negative reciprocal of the actual branch or transformer impedance in p.u. of system MVA; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	branch array index, lower value of range (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.19. SET_GBLINE

Use this API to set the negative reciprocal of the nominal ac line impedance.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_gbline(gbline, end, indx)

set_gbline(agbline, end, end2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_GBLINE(GBLINE, END, INDX)

CALL SET_GBLINE(AGBLINE, END, END2, INDX, INDX2)
```

Where:

Complex GBLINE	negative reciprocal of the nominal ac line impedance for the element indicated by indx1 (input).
Complex AGBLINE ((END2-END +1)*(INDX2-INDX+1))	negative reciprocal of the nominal ac line impedance; an array of values for the elements indicated by the indices supplied (input).
Integer END	branch end index: 1 = from bus, 2 = to bus (input).
Integer END2	branch end index: 1 = from bus, 2 = to bus, upper value of range (input).
Integer INDX	branch array index (input).
Integer INDX2	branch end index: 1 = from bus, 2 = to bus, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.20. SET_GREF

Use this API to set the gref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_gref(gref, indx1)

set_gref(agref, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_GREF(GREF, INDX1)

CALL SET_GREF(AGREF, INDX1, INDX2)
```

Where:

Real GREF	gref for the element indicated by indx1 (input).
Real AGREF (INDX2-INDX1+1)	gref; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	gref, lower value of range (input).
Integer INDX2	gref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.21. SET_ICON

Use this API to set the Integer constants.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_icon(icon, indx1)

set_icon(aicon, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_ICON(ICON, INDX1)

CALL SET_ICON(AICON, INDX1, INDX2)
```

Where:

Integer ICON	Integer constants for the element indicated by indx1 (input).
Integer AICON (INDX2-INDX1+1)	Integer constants; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	icon, lower value of range (input).
Integer INDX2	icon, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.22. SET_ICON_DSCRPT

Use this API to set the icon_dscrpt.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_icon_dscrpt(icon_dscrpt, indx1)

set_icon_dscrpt(aicon_dscrpt, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_ICON_DSCRPT(ICON_DSCRPT, INDX1)

CALL SET_ICON_DSCRPT(AICON_DSCRPT, INDX1, INDX2)
```

Where:

Character*144 ICON_DSCRPT	icon_dscrpt for the element indicated by indx1 (input).
Character*144 AICON_DSCRPT (INDX2-INDX1+1)	icon_dscrpt; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	icon_dscrpt, lower value of range (input).
Integer INDX2	icon_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.23. SET_IMARYS

Use this API to set the imarys.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_imarys(imarys, one, two)

set_imarys(aimarys, one, one2, two, two2)
```

Fortran command syntax:

```
CALL SET_IMARYS(IMARYS, ONE, TWO)

CALL SET_IMARYS(AIMARYS, ONE, ONE2, TWO, TWO2)
```

Where:

Real IMARYS	imarys for the element indicated by indx1 (input).
Real AIMARYS ((ONE2-ONE +1)*(TWO2-TWO+1))	imarys; an array of values for the elements indicated by the indices supplied (input).
Integer ONE	imarys (input).
Integer ONE2	imarys, upper value of range (input).
Integer TWO	imarys (input).
Integer TWO2	imarys, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.24. SET_IMCCNX

Use this API to set the imccnx.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_imccnx(imccnx, indx1)

set_imccnx(aimccnx, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_IMCCNX(IMCCNX, INDX1)

CALL SET_IMCCNX(AIMCCNX, INDX1, INDX2)
```

Where:

Integer IMCCNX	imccnx for the element indicated by indx1 (input).
Integer AIMCCNX (INDX2-INDX1+1)	imccnx; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	imccnx, lower value of range (input).
Integer INDX2	imccnx, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.25. SET_IMELTRQ

Use this API to set the imeltrq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_imeltrq(imeltrq, indx1)

set_imeltrq(aimeltrq, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_IMELTRQ(IMELTRQ, INDX1)

CALL SET_IMELTRQ(AIMELTRQ, INDX1, INDX2)
```

Where:

Real IMELTRQ	imeltrq for the element indicated by indx1 (input).
Real AIMELTRQ (INDX2-INDX1+1)	imeltrq; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	imeltrq, lower value of range (input).
Integer INDX2	imeltrq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.26. SET_IMLDTRQ

Use this API to set the imldtrq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_imldtrq(imldtrq, indx1)

set_imldtrq(aimldtrq, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_IMLDTRQ(IMLDTRQ, INDX1)

CALL SET_IMLDTRQ(AIMLDTRQ, INDX1, INDX2)
```

Where:

Real IMLDTRQ	imldtrq for the element indicated by indx1 (input).
Real AIMLDTRQ (INDX2-INDX1+1)	imldtrq; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	imldtrq, lower value of range (input).
Integer INDX2	imldtrq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.27. SET_IMSPARE

Use this API to set the imspare.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_imspare(imspare, indx1)

set_imspare(aimspare, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_IMSPARE(IMSPARE, INDX1)

CALL SET_IMSPARE(AIMSPARE, INDX1, INDX2)
```

Where:

Real IMSPARE	imspare for the element indicated by indx1 (input).
Real AIMSPARE (INDX2-INDX1+1)	imspare; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	imspare, lower value of range (input).
Integer INDX2	imspare, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.28. SET_IMTPLOD

Use this API to set the imtplod.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_imtplod(imtplod, indx1)

set_imtplod(aimtplod, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_IMTPLOD(IMTPLOD, INDX1)

CALL SET_IMTPLOD(AIMTPLOD, INDX1, INDX2)
```

Where:

Real IMTPLOD	imtplod for the element indicated by indx1 (input).
Real AIMTPLOD (INDX2-INDX1+1)	imtplod; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	imtplod, lower value of range (input).
Integer INDX2	imtplod, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.29. SET_IMTQLOD

Use this API to set the imtqlod.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_imtqlod(imtqlod, indx1)

set_imtqlod(aimtqlod, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_IMTQLOD(IMTQLOD, INDX1)

CALL SET_IMTQLOD(AIMTQLOD, INDX1, INDX2)
```

Where:

Real IMTQLOD	imtqlod for the element indicated by indx1 (input).
Real AIMTQLOD (INDX2-INDX1+1)	imtqlod; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	imtqlod, lower value of range (input).
Integer INDX2	imtqlod, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.30. SET_INTICN

Use this API to set the scartch memory location for storing integer data (sized to as many as number of ICONs used by any model).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_inticn(inticn, indx1)

set_inticn(ainticn, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_INTICN(INTICN, INDX1)

CALL SET_INTICN(AINTICN, INDX1, INDX2)
```

Where:

Integer INTICN	scartch memory location for storing integer data (sized to as many as number of ICONs used by any model) for the element indicated by indx1 (input).
Integer AINTICN (INDX2-INDX1+1)	scartch memory location for storing integer data (sized to as many as number of ICONs used by any model); an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	inticn, lower value of range (input).
Integer INDX2	inticn, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.31. SET_ISORCE

Use this API to set the machine complex Norton current source (p.u. of system MVA) .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_isorce(isorce, indx1)

set_isorce(aisorce, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_ISORCE(ISORCE, INDX1)

CALL SET_ISORCE(AISORCE, INDX1, INDX2)
```

Where:

Complex ISORCE	machine complex Norton current source (p.u. of system MVA) for the element indicated by indx1 (input).
Complex AISORCE (INDX2-INDX1+1)	machine complex Norton current source (p.u. of system MVA) ; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	machine array index, lower value of range (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.32. SET_KPAUSE

Use this API to set the KPAUSE.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_kpause(kpause)
```

Fortran command syntax:

```
SET_KPAUSE(KPAUSE)
```

Where:

Integer KPAUSE	KPAUSE (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.33. SET_KTRIP

Use this API to set the KTRIP.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_ktrip(ktrip)
```

Fortran command syntax:

```
SET_KTRIP(KTRIP)
```

Where:

Integer KTRIP	KTRIP (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.34. SET_LCREF

Use this API to set the lcref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_lcref(lcref, indx1)

set_lcref(alcref, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_LCREF(LCREF, INDX1)

CALL SET_LCREF(ALCREF, INDX1, INDX2)
```

Where:

Real LCREF	lcref for the element indicated by indx1 (input).
Real ALCREF (INDX2-INDX1+1)	lcref; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	lcref, lower value of range (input).
Integer INDX2	lcref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.35. SET_LDDGNFL

Use this API to set the DER status on the load record.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_lddgnfl(lddgnfl, indx1)

set_lddgnfl(alddgnfl, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_LDDGNFL(LDDGNFL, INDX1)

CALL SET_LDDGNFL(ALDDGNFL, INDX1, INDX2)
```

Where:

Integer LDDGNFL	DER status on the load record for the element indicated by indx1 (input).
Integer ALDDGNFL (INDX2-INDX1+1)	DER status on the load record; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	load array index, lower value of range (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.36. SET_LDSTAT

Use this API to set the load status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_ldstat(ldstat, indx1)

set_ldstat(aldstat, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_LDSTAT(LDSTAT, INDX1)

CALL SET_LDSTAT(ALDSTAT, INDX1, INDX2)
```

Where:

Integer LDSTAT	load status for the element indicated by indx1 (input).
Integer ALDSTAT (INDX2-INDX1+1)	load status; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	load array index, lower value of range (input).
Integer INDX2	load array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.37. SET_MACBLKD

Use this API to set the macblkd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_macblkd(macblkd, indx1)

set_macblkd(amacblkd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_MACBLKD(MACBLKD, INDX1)

CALL SET_MACBLKD(AMACBLKD, INDX1, INDX2)
```

Where:

Logical MACBLKD	macblkd for the element indicated by indx1 (input).
Logical AMACBLKD (INDX2-INDX1+1)	macblkd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	macblkd, lower value of range (input).
Integer INDX2	macblkd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.38. SET_MCARYS

Use this API to set the mcarys.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_mcarys(mcarys, one, two)

set_mcarys(amcarys, one, one2, two, two2)
```

Fortran command syntax:

```
CALL SET_MCARYS(MCARYS, ONE, TWO)

CALL SET_MCARYS(AMCARYS, ONE, ONE2, TWO, TWO2)
```

Where:

Real MCARYS	mcarys for the element indicated by indx1 (input).
Real AMCARYS ((ONE2-ONE +1)*(TWO2-TWO+1))	mcarys; an array of values for the elements indicated by the indices supplied (input).
Integer ONE	mcarys (input).
Integer ONE2	mcarys, upper value of range (input).
Integer TWO	mcarys (input).
Integer TWO2	mcarys, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.39. SET_MCSTAT

Use this API to set the machine staus.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_mcstat(mcstat, indx1)

set_mcstat(amcstat, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_MCSTAT(MCSTAT, INDX1)

CALL SET_MCSTAT(AMCSTAT, INDX1, INDX2)
```

Where:

Integer MCSTAT	machine staus for the element indicated by indx1 (input).
Integer AMCSTAT (INDX2-INDX1+1)	machine staus; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	machine array index, lower value of range (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.40. SET_MDC

Use this API to set the two terminal dc line control mode: 0 = blocked, 1 = power, 2 = current.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
set_mdc(mdc, indx1)

set_mdc(amdc, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_MDC(MDC, INDX1)

CALL SET_MDC(AMDC, INDX1, INDX2)
```

Where:

Integer MDC	two terminal dc line control mode: 0 = blocked, 1 = power, 2 = current for the element indicated by indx1 (input).
Integer AMDC (INDX2-INDX1+1)	two terminal dc line control mode: 0 = blocked, 1 = power, 2 = current; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	dc line array index, lower value of range (input).
Integer INDX2	dc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.41. SET_NINTEG

Use this API to set the NINTEG.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_ninteg(ninteg)
```

Fortran command syntax:

```
SET_NINTEG(NINTEG)
```

Where:

Integer NINTEG	NINTEG (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.42. SET_NLWTRB

Use this API to set the nlwtrb.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_nlwtrb(nlwtrb, indx1)

set_nlwtrb(anlwtrb, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_NLWTRB(NLWTRB, INDX1)

CALL SET_NLWTRB(ANLWTRB, INDX1, INDX2)
```

Where:

Integer NLWTRB	nlwtrb for the element indicated by indx1 (input).
Integer ANLWTRB (INDX2-INDX1+1)	nlwtrb; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	nlwtrb, lower value of range (input).
Integer INDX2	nlwtrb, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.43. SET_NUMLOD

Use this API to set the numlod.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_numlod(numlod, indx1)

set_numlod(anumlod, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_NUMLOD(NUMLOD, INDX1)

CALL SET_NUMLOD(ANUMLOD, INDX1, INDX2)
```

Where:

Integer NUMLOD	numlod for the element indicated by indx1 (input).
Integer ANUMLOD (INDX2-INDX1+1)	numlod; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	numlod, lower value of range (input).
Integer INDX2	numlod, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.44. SET_NUMSWS

Use this API to set the numsws.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_numsws(numsws, indx1)

set_numsws(anumsws, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_NUMSWS(NUMSWS, INDX1)

CALL SET_NUMSWS(ANUMSWS, INDX1, INDX2)
```

Where:

Integer NUMSWS	numsws for the element indicated by indx1 (input).
Integer ANUMSWS (INDX2-INDX1+1)	numsws; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	numsws, lower value of range (input).
Integer INDX2	numsws, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.45. SET_NUMTRM

Use this API to set the bus array index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_numtrm(numtrm, indx1)

set_numtrm(anumtrm, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_NUMTRM(NUMTRM, INDX1)

CALL SET_NUMTRM(ANUMTRM, INDX1, INDX2)
```

Where:

Integer NUMTRM	bus array index for the element indicated by indx1 (input).
Integer ANUMTRM (INDX2-INDX1+1)	bus array index; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	machine array index, lower value of range (input).
Integer INDX2	machine array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.46. SET_OEL_ACTIVE

Use this API to set the oel_active.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_oel_active(oel_active, indx1)

set_oel_active(aoel_active, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_OEL_ACTIVE(OEL_ACTIVE, INDX1)

CALL SET_OEL_ACTIVE(AOEL_ACTIVE, INDX1, INDX2)
```

Where:

Logical OEL_ACTIVE	oel_active for the element indicated by indx1 (input).
Logical AOEL_ACTIVE (INDX2-INDX1+1)	oel_active; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	oel_active, lower value of range (input).
Integer INDX2	oel_active, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.47. SET_OLDERV

Use this API to set the earlier value of derivative of state variable.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
set_olderv(olderv, indx1)

set_olderv(aolderv, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_OLDERV(OLDERV, INDX1)

CALL SET_OLDERV(AOLDERV, INDX1, INDX2)
```

Where:

Real OLDERV	earlier value of derivative of state variable for the element indicated by indx1 (input).
Real AOLDERV (INDX2-INDX1+1)	earlier value of derivative of state variable; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	olderv, lower value of range (input).
Integer INDX2	olderv, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.48. SET_PELEC

Use this API to set the pelec.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_pelec(pelec, indx1)

set_pelec(apelec, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_PELEC(PELEC, INDX1)

CALL SET_PELEC(APELEC, INDX1, INDX2)
```

Where:

Real PELEC	pelec for the element indicated by indx1 (input).
Real APELEC (INDX2-INDX1+1)	pelec; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	pelec, lower value of range (input).
Integer INDX2	pelec, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.49. SET_PHSVLT

Use this API to set the PHSVLT.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_phsvlt(phsvlt)
```

Fortran command syntax:

```
SET_PHSVLT(PHSVLT)
```

Where:

Logical PHSVLT	PHSVLT (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.50. SET_PMAXMCD

Use this API to set the pmaxmcd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_pmaxmcd(pmaxmcd, indx1)

set_pmaxmcd(apmaxmcd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_PMAXMCD(PMAXMCD, INDX1)

CALL SET_PMAXMCD(APMAXMCD, INDX1, INDX2)
```

Where:

Real PMAXMCD	pmaxmcd for the element indicated by indx1 (input).
Real APMAXMCD (INDX2-INDX1+1)	pmaxmcd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	pmaxmcd, lower value of range (input).
Integer INDX2	pmaxmcd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.51. SET_PMECH

Use this API to set the pmech.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_pmech(pmech, indx1)

set_pmech(apmech, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_PMECH(PMECH, INDX1)

CALL SET_PMECH(APMECH, INDX1, INDX2)
```

Where:

Real PMECH	pmech for the element indicated by indx1 (input).
Real APMECH (INDX2-INDX1+1)	pmech; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	pmech, lower value of range (input).
Integer INDX2	pmech, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.52. SET_PMINMCD

Use this API to set the pminmcd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_pminmcd(pminmcd, indx1)

set_pminmcd(apminmcd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_PMINMCD(PMINMCD, INDX1)

CALL SET_PMINMCD(APMINMCD, INDX1, INDX2)
```

Where:

Real PMINMCD	pminmcd for the element indicated by indx1 (input).
Real APMINMCD (INDX2-INDX1+1)	pminmcd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	pminmcd, lower value of range (input).
Integer INDX2	pminmcd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.53. SET_PSANGL

Use this API to set the transformer winding 1 phase shift angle in radians .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_psangl(psangl, indx1)

set_psangl(apsangl, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_PSANGL(PSANGL, INDX1)

CALL SET_PSANGL(APSANGL, INDX1, INDX2)
```

Where:

Real PSANGL	transformer winding 1 phase shift angle in radians for the element indicated by indx1 (input).
Real APSANGL (INDX2-INDX1+1)	transformer winding 1 phase shift angle in radians ; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	branch array index, lower value of range (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.54. SET_QELEC

Use this API to set the qelec.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_qelec(qelec, indx1)

set_qelec(aqelec, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_QELEC(QELEC, INDX1)

CALL SET_QELEC(AQELEC, INDX1, INDX2)
```

Where:

Real QELEC	qelec for the element indicated by indx1 (input).
Real AQELEC (INDX2-INDX1+1)	qelec; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	qelec, lower value of range (input).
Integer INDX2	qelec, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.55. SET_SHNTGB

Use this API to set the fixed shunt admittance (p.u. of system MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_shntgb(shntgb, indx1)

set_shntgb(ashntgb, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_SHNTGB(SHNTGB, INDX1)

CALL SET_SHNTGB(ASHNTGB, INDX1, INDX2)
```

Where:

Complex SHNTGB	fixed shunt admittance (p.u. of system MVA) for the element indicated by indx1 (input).
Complex ASHNTGB (INDX2-INDX1+1)	fixed shunt admittance (p.u. of system MVA); an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	fixed shunt array index, lower value of range (input).
Integer INDX2	fixed shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.56. SET_SHSTAT

Use this API to set the fixed shunt status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_shstat(shstat, indx1)

set_shstat(ashstat, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_SHSTAT(SHSTAT, INDX1)

CALL SET_SHSTAT(ASHSTAT, INDX1, INDX2)
```

Where:

Integer SHSTAT	fixed shunt status for the element indicated by indx1 (input).
Integer ASHSTAT (INDX2-INDX1+1)	fixed shunt status; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	fixed shunt array index, lower value of range (input).
Integer INDX2	fixed shunt array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.57. SET_SPEED

Use this API to set the speed.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_speed(speed, indx1)

set_speed(aspeed, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_SPEED(SPEED, INDX1)

CALL SET_SPEED(ASPEED, INDX1, INDX2)
```

Where:

Real SPEED	speed for the element indicated by indx1 (input).
Real ASPEED (INDX2-INDX1+1)	speed; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	speed, lower value of range (input).
Integer INDX2	speed, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.58. SET_STATE

Use this API to set the state variable.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_state(state, indx1)

set_state(astate, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_STATE(STATE, INDX1)

CALL SET_STATE(ASTATE, INDX1, INDX2)
```

Where:

Real STATE	state variable for the element indicated by indx1 (input).
Real ASTATE (INDX2-INDX1+1)	state variable; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	state, lower value of range (input).
Integer INDX2	state, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.59. SET_STATE_DSCRPT

Use this API to set the state_dscrpt.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
set_state_dscrpt(state_dscrpt, indx1)

set_state_dscrpt(astate_dscrpt, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_STATE_DSCRPT(STATE_DSCRPT, INDX1)

CALL SET_STATE_DSCRPT(ASTATE_DSCRPT, INDX1, INDX2)
```

Where:

Character*144 STATE_DSCRPT	state_dscrpt for the element indicated by indx1 (input).
Character*144 ASTATE_DSCRPT (INDX2-INDX1+1)	state_dscrpt; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	state_dscrpt, lower value of range (input).
Integer INDX2	state_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.60. SET_STORE

Use this API to set the contains information about past history of state variables.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_store(store, indx1)

set_store(astore, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_STORE(STORE, INDX1)

CALL SET_STORE(ASTORE, INDX1, INDX2)
```

Where:

Real STORE	contains information about past history of state variables for the element indicated by indx1 (input).
Real ASTORE (INDX2-INDX1+1)	contains information about past history of state variables; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	store, lower value of range (input).
Integer INDX2	store, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.61. SET_SWSAUX

Use this API to set the swsaux.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_swsaux(swsaux, indx1)

set_swsaux(aswsaux, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_SWSAUX(SWSAUX, INDX1)

CALL SET_SWSAUX(ASWSAUX, INDX1, INDX2)
```

Where:

Real SWSAUX	swsaux for the element indicated by indx1 (input).
Real ASWSAUX (INDX2-INDX1+1)	swsaux; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	swsaux, lower value of range (input).
Integer INDX2	swsaux, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.62. SET_SWSCNX

Use this API to set the swscnx.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_swscnx(swscnx, indx1)

set_swscnx(aswscnx, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_SWSCNX(SWSCNX, INDX1)

CALL SET_SWSCNX(ASWSCNX, INDX1, INDX2)
```

Where:

Integer SWSCNX	swscnx for the element indicated by indx1 (input).
Integer ASWSCNX (INDX2-INDX1+1)	swscnx; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	swscnx, lower value of range (input).
Integer INDX2	swscnx, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.63. SET_SWSHNB

Use this API to set the switched shunt admittance increment for each step in the block in MVA .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_swshnb(swshnb, block, indx)

set_swshnb(aswshnb, block, block2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_SWSHNB(SWSHNB, BLOCK, INDX)

CALL SET_SWSHNB(ASWSHNB, BLOCK, BLOCK2, INDX, INDX2)
```

Where:

Real SWSHNB	switched shunt admittance increment for each step in the block in MVA for the element indicated by indx1 (input).
Real ASWSHNB ((BLOCK2-BLOCK +1)*(INDX2-INDX+1))	switched shunt admittance increment for each step in the block in MVA ; an array of values for the elements indicated by the indices supplied (input).
Integer BLOCK	block number (1:8) (input).
Integer BLOCK2	block number (1:8), upper value of range (input).
Integer INDX	switched shunt array index (input).
Integer INDX2	block number (1:8), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.64. SET_SWSREF

Use this API to set the swsref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_swsref(swsref, indx1)

set_swsref(aswsref, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_SWSREF(SWSREF, INDX1)

CALL SET_SWSREF(ASWSREF, INDX1, INDX2)
```

Where:

Real SWSREF	swsref for the element indicated by indx1 (input).
Real ASWSREF (INDX2-INDX1+1)	swsref; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	swsref, lower value of range (input).
Integer INDX2	swsref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.65. SET_TBRATE

Use this API to set the tbrate.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_tbrate(tbrate, indx1)

set_tbrate(atbrate, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_TBRATE(TBRATE, INDX1)

CALL SET_TBRATE(ATBRATE, INDX1, INDX2)
```

Where:

Real TBRATE	tbrate for the element indicated by indx1 (input).
Real ATBRATE (INDX2-INDX1+1)	tbrate; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	tbrate, lower value of range (input).
Integer INDX2	tbrate, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.66. SET_TPLOAD

Use this API to set the tpload.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_tpload(tpload, indx1)

set_tpload(atpload, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_TPLOAD(TPLOAD, INDX1)

CALL SET_TPLOAD(ATPLOAD, INDX1, INDX2)
```

Where:

Real TPLOAD	tpload for the element indicated by indx1 (input).
Real ATPLOAD (INDX2-INDX1+1)	tpload; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	tpload, lower value of range (input).
Integer INDX2	tpload, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.67. SET_TQLOAD

Use this API to set the tqload.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_tqload(tqload, indx1)

set_tqload(atqload, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_TQLOAD(TQLOAD, INDX1)

CALL SET_TQLOAD(ATQLOAD, INDX1, INDX2)
```

Where:

Real TQLOAD	tqload for the element indicated by indx1 (input).
Real ATQLOAD (INDX2-INDX1+1)	tqload; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	tqload, lower value of range (input).
Integer INDX2	tqload, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.68. SET_UEL_ACTIVE

Use this API to set the uel_active.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_uel_active(uel_active, indx1)

set_uel_active(auel_active, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_UEL_ACTIVE(UEL_ACTIVE, INDX1)

CALL SET_UEL_ACTIVE(AUEL_ACTIVE, INDX1, INDX2)
```

Where:

Logical UEL_ACTIVE	uel_active for the element indicated by indx1 (input).
Logical AUEL_ACTIVE (INDX2-INDX1+1)	uel_active; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	uel_active, lower value of range (input).
Integer INDX2	uel_active, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.69. SET_VAR

Use this API to set the scartch memory location for storing model floating point data.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_var(var, indx1)

set_var(avar, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VAR(VAR, INDX1)

CALL SET_VAR(AVAR, INDX1, INDX2)
```

Where:

Real VAR	scartch memory location for storing model floating point data for the element indicated by indx1 (input).
Real AVAR (INDX2-INDX1+1)	scartch memory location for storing model floating point data; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	var, lower value of range (input).
Integer INDX2	var, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.70. SET_VAR_DSCRPT

Use this API to set the var_dscrpt.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
set_var_dscrpt(var_dscrpt, indx1)

set_var_dscrpt(avar_dscrpt, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VAR_DSCRPT(VAR_DSCRPT, INDX1)

CALL SET_VAR_DSCRPT(AVAR_DSCRPT, INDX1, INDX2)
```

Where:

Character*144 VAR_DSCRPT	var_dscrpt for the element indicated by indx1 (input).
Character*144 AVAR_DSCRPT (INDX2-INDX1+1)	var_dscrpt; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	var_dscrpt, lower value of range (input).
Integer INDX2	var_dscrpt, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.71. SET_VDC

Use this API to set the multi-terminal dc line dc bus voltage.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vdc(vdc, dcbusidx, indx)

set_vdc(avdc, dcbusidx, dcbusidx2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_VDC(VDC, DCBUSIDX, INDX)

CALL SET_VDC(AVDC, DCBUSIDX, DCBUSIDX2, INDX, INDX2)
```

Where:

Real VDC	multi-terminal dc line dc bus voltage for the element indicated by indx1 (input).
Real AVDC ((DCBUSIDX2-DCBUSIDX+1)*(INDX2-INDX+1))	multi-terminal dc line dc bus voltage; an array of values for the elements indicated by the indices supplied (input).
Integer DCBUSIDX	multi-terminal dc line dc bus index (1:MXDCBS) (input).
Integer DCBUSIDX2	multi-terminal dc line dc bus index (1:MXDCBS), upper value of range (input).
Integer INDX	multi-terminal dc line index (input).
Integer INDX2	multi-terminal dc line dc bus index (1:MXDCBS), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.72. SET_VFEOUT

Use this API to set the vfeout.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vfeout(vfeout, indx1)

set_vfeout(avfeout, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VFEOUT(VFEOUT, INDX1)

CALL SET_VFEOUT(AVFEOUT, INDX1, INDX2)
```

Where:

Real VFEOUT	vfeout for the element indicated by indx1 (input).
Real AVFEOUT (INDX2-INDX1+1)	vfeout; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	vfeout, lower value of range (input).
Integer INDX2	vfeout, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.73. SET_VOEL

Use this API to set the voel.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_voel(voel, indx1)

set_voel(avoel, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VOEL(VOEL, INDX1)

CALL SET_VOEL(AVOEL, INDX1, INDX2)
```

Where:

Real VOEL	voel for the element indicated by indx1 (input).
Real AVOEL (INDX2-INDX1+1)	voel; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	voel, lower value of range (input).
Integer INDX2	voel, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.74. SET_VOTHSG

Use this API to set the vothsg.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vothsg(vothsg, indx1)

set_vothsg(avothsg, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VOTHSG(VOTHSG, INDX1)

CALL SET_VOTHSG(AVOTHSG, INDX1, INDX2)
```

Where:

Real VOTHSG	vothsg for the element indicated by indx1 (input).
Real AVOTHSG (INDX2-INDX1+1)	vothsg; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	vothsg, lower value of range (input).
Integer INDX2	vothsg, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.75. SET_VREF

Use this API to set the vref.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vref(vref, indx1)

set_vref(avref, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VREF(VREF, INDX1)

CALL SET_VREF(AVREF, INDX1, INDX2)
```

Where:

Real VREF	vref for the element indicated by indx1 (input).
Real AVREF (INDX2-INDX1+1)	vref; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	vref, lower value of range (input).
Integer INDX2	vref, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.76. SET_VRFDBK

Use this API to set the vrfdbk.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vrfdbk(vrfdbk, indx1)

set_vrfdbk(avrfdbk, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VRFDBK(VRFDBK, INDX1)

CALL SET_VRFDBK(AVRFDBK, INDX1, INDX2)
```

Where:

Real VRFDBK	vrfdbk for the element indicated by indx1 (input).
Real AVRFDBK (INDX2-INDX1+1)	vrfdbk; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	vrfdbk, lower value of range (input).
Integer INDX2	vrfdbk, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.77. SET_VSCMDC

Use this API to set the vscdc line dc control mode.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vscmdc(vscmdc, indx1)

set_vscmdc(avscmdc, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VSCMDC(VSCMDC, INDX1)

CALL SET_VSCMDC(AVSCMDC, INDX1, INDX2)
```

Where:

Integer VSCMDC	vscdc line dc control mode for the element indicated by indx1 (input).
Integer AVSCMDC (INDX2-INDX1+1)	vscdc line dc control mode; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	vscdc line array index, lower value of range (input).
Integer INDX2	vscdc line array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.78. SET_VSCPAC

Use this API to set the vscdc converter a loss coefficient (kw).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vscpac(vscpac, cnvidx, indx)

set_vscpac(avscpac, cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_VSCPAC(VSCPAC, CNVIDX, INDX)

CALL SET_VSCPAC(AVSCPAC, CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCPAC	vscdc converter a loss coefficient (kw) for the element indicated by indx1 (input).
Real AVSCPAC ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter a loss coefficient (kw); an array of values for the elements indicated by the indices supplied (input).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.79. SET_VSCQAC

Use this API to set the vscdc converter b loss coefficient (kw/amp).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vscqac(vscqac, cnvidx, indx)

set_vscqac(avscqac, cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_VSCQAC(VSCQAC, CNVIDX, INDX)

CALL SET_VSCQAC(AVSCQAC, CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Real VSCQAC	vscdc converter b loss coefficient (kw/amp) for the element indicated by indx1 (input).
Real AVSCQAC ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter b loss coefficient (kw/amp); an array of values for the elements indicated by the indices supplied (input).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.80. SET_VSCSIG

Use this API to set the vscsig.

This API routine was first introduced in release 36.1.1.

Python command syntax:

```
set_vscsig(vscsig, one, two)

set_vscsig(avscsig, one, one2, two, two2)
```

Fortran command syntax:

```
CALL SET_VSCSIG(VSCSIG, ONE, TWO)

CALL SET_VSCSIG(AVSCSIG, ONE, ONE2, TWO, TWO2)
```

Where:

Real VSCSIG	vscsig for the element indicated by indx1 (input).
Real AVSCSIG ((ONE2-ONE +1)*(TWO2-TWO+1))	vscsig; an array of values for the elements indicated by the indices supplied (input).
Integer ONE	vscsig (input).
Integer ONE2	vscsig, upper value of range (input).
Integer TWO	vscsig (input).
Integer TWO2	vscsig, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.81. SET_VSCTYP

Use this API to set the vscdc converter active power seen from ac network at cong.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vsctyp(vsctyp, cnvidx, indx)

set_vsctyp(avsctyp, cnvidx, cnvidx2, indx, indx2)
```

Fortran command syntax:

```
CALL SET_VSCTYP(VSCTYP, CNVIDX, INDX)

CALL SET_VSCTYP(AVSCTYP, CNVIDX, CNVIDX2, INDX, INDX2)
```

Where:

Integer VSCTYP	vscdc converter active power seen from ac network at cong for the element indicated by indx1 (input).
Integer AVSCTYP ((CNVIDX2-CNVIDX+1)*(INDX2-INDX+1))	vscdc converter active power seen from ac network at cong; an array of values for the elements indicated by the indices supplied (input).
Integer CNVIDX	vscdc line converter index (1:2) (input).
Integer CNVIDX2	vscdc line converter index (1:2), upper value of range (input).
Integer INDX	vscdc line array index (input).
Integer INDX2	vscdc line converter index (1:2), upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.82. SET_VUEL

Use this API to set the vuel.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_vuel(vuel, indx1)

set_vuel(avuel, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_VUEL(VUEL, INDX1)

CALL SET_VUEL(AVUEL, INDX1, INDX2)
```

Where:

Real VUEL	vuel for the element indicated by indx1 (input).
Real AVUEL (INDX2-INDX1+1)	vuel; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	vuel, lower value of range (input).
Integer INDX2	vuel, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.83. SET_WAEROT

Use this API to set the waerot.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_waerot(waerot, indx1)

set_waerot(awaerot, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WAEROT(WAEROT, INDX1)

CALL SET_WAEROT(AWAEROT, INDX1, INDX2)
```

Where:

Real WAEROT	waerot for the element indicated by indx1 (input).
Real AWAEROT (INDX2-INDX1+1)	waerot; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	waerot, lower value of range (input).
Integer INDX2	waerot, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.84. SET_WAUXSG

Use this API to set the wauxsg.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wauxsg(wauxsg, indx1)

set_wauxsg(awauxsg, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WAUXSG(WAUXSG, INDX1)

CALL SET_WAUXSG(AWAUXSG, INDX1, INDX2)
```

Where:

Real WAUXSG	wauxsg for the element indicated by indx1 (input).
Real AWAUXSG (INDX2-INDX1+1)	wauxsg; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wauxsg, lower value of range (input).
Integer INDX2	wauxsg, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.85. SET_WEQCMD

Use this API to set the weqcmd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_weqcmd(weqcmd, indx1)

set_weqcmd(aweqcmd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WEQCMD(WEQCMD, INDX1)

CALL SET_WEQCMD(AWEQCMD, INDX1, INDX2)
```

Where:

Real WEQCMD	weqcmd for the element indicated by indx1 (input).
Real AWEQCMD (INDX2-INDX1+1)	weqcmd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	weqcmd, lower value of range (input).
Integer INDX2	weqcmd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.86. SET_WINDRP

Use this API to set the transformer winding 1 tap ratio .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_windrp(windrp, indx1)

set_windrp(awindrp, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WINDRP(WINDRP, INDX1)

CALL SET_WINDRP(AWINDRP, INDX1, INDX2)
```

Where:

Real WINDRP	transformer winding 1 tap ratio for the element indicated by indx1 (input).
Real AWINDRP (INDX2-INDX1+1)	transformer winding 1 tap ratio ; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	branch array index, lower value of range (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.87. SET_WINDRS

Use this API to set the transformer winding 2 tap ratio .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_windrs(windrs, indx1)

set_windrs(awindrs, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WINDRS(WINDRS, INDX1)

CALL SET_WINDRS(AWINDRS, INDX1, INDX2)
```

Where:

Real WINDRS	transformer winding 2 tap ratio for the element indicated by indx1 (input).
Real AWINDRS (INDX2-INDX1+1)	transformer winding 2 tap ratio ; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	branch array index, lower value of range (input).
Integer INDX2	branch array index, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.88. SET_WIPCMD

Use this API to set the wipcmd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wipcmd(wipcmd, indx1)

set_wipcmd(awipcmd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WIPCMD(WIPCMD, INDX1)

CALL SET_WIPCMD(AWIPCMD, INDX1, INDX2)
```

Where:

Real WIPCMD	wipcmd for the element indicated by indx1 (input).
Real AWIPCMD (INDX2-INDX1+1)	wipcmd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wipcmd, lower value of range (input).
Integer INDX2	wipcmd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.89. SET_WIQCMD

Use this API to set the wiqcmd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wiqcmd(wiqcmd, indx1)

set_wiqcmd(awiqcmd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WIQCMD(WIQCMD, INDX1)

CALL SET_WIQCMD(AWIQCMD, INDX1, INDX2)
```

Where:

Real WIQCMD	wiqcmd for the element indicated by indx1 (input).
Real AWIQCMD (INDX2-INDX1+1)	wiqcmd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wiqcmd, lower value of range (input).
Integer INDX2	wiqcmd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.90. SET_WMCARYS

Use this API to set the wmcarys.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wmcarys(wmcarys, one, two)

set_wmcarys(awmcarys, one, one2, two, two2)
```

Fortran command syntax:

```
CALL SET_WMCARYS(WMCARYS, ONE, TWO)

CALL SET_WMCARYS(AWMCARYS, ONE, ONE2, TWO, TWO2)
```

Where:

Real WMCARYS	wmcarys for the element indicated by indx1 (input).
Real AWMCARYS ((ONE2-ONE +1)*(TWO2-TWO+1))	wmcarys; an array of values for the elements indicated by the indices supplied (input).
Integer ONE	wmcarys (input).
Integer ONE2	wmcarys, upper value of range (input).
Integer TWO	wmcarys (input).
Integer TWO2	wmcarys, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.91. SET_WNDSP1

Use this API to set the wndsp1.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp1(wndsp1, indx1)

set_wndsp1(awndsp1, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP1(WNDSP1, INDX1)

CALL SET_WNDSP1(AWNDSP1, INDX1, INDX2)
```

Where:

Real WNDSP1	wndsp1 for the element indicated by indx1 (input).
Real AWNDSP1 (INDX2-INDX1+1)	wndsp1; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp1, lower value of range (input).
Integer INDX2	wndsp1, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.92. SET_WNDSP10

Use this API to set the wndsp10.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp10(wndsp10, indx1)

set_wndsp10(awndsp10, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP10(WNDSP10, INDX1)

CALL SET_WNDSP10(AWNDSP10, INDX1, INDX2)
```

Where:

Real WNDSP10	wndsp10 for the element indicated by indx1 (input).
Real AWNDSP10 (INDX2-INDX1+1)	wndsp10; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp10, lower value of range (input).
Integer INDX2	wndsp10, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.93. SET_WNDSP2

Use this API to set the wndsp2.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp2(wndsp2, indx1)

set_wndsp2(awndsp2, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP2(WNDSP2, INDX1)

CALL SET_WNDSP2(AWNDSP2, INDX1, INDX2)
```

Where:

Real WNDSP2	wndsp2 for the element indicated by indx1 (input).
Real AWNDSP2 (INDX2-INDX1+1)	wndsp2; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp2, lower value of range (input).
Integer INDX2	wndsp2, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.94. SET_WNDSP3

Use this API to set the wndsp3.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp3(wndsp3, indx1)

set_wndsp3(awndsp3, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP3(WNDSP3, INDX1)

CALL SET_WNDSP3(AWNDSP3, INDX1, INDX2)
```

Where:

Real WNDSP3	wndsp3 for the element indicated by indx1 (input).
Real AWNDSP3 (INDX2-INDX1+1)	wndsp3; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp3, lower value of range (input).
Integer INDX2	wndsp3, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.95. SET_WNDSP4

Use this API to set the wndsp4.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp4(wndsp4, indx1)

set_wndsp4(awndsp4, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP4(WNDSP4, INDX1)

CALL SET_WNDSP4(AWNDSP4, INDX1, INDX2)
```

Where:

Real WNDSP4	wndsp4 for the element indicated by indx1 (input).
Real AWNDSP4 (INDX2-INDX1+1)	wndsp4; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp4, lower value of range (input).
Integer INDX2	wndsp4, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.96. SET_WNDSP5

Use this API to set the wndsp5.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp5(wndsp5, indx1)

set_wndsp5(awndsp5, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP5(WNDSP5, INDX1)

CALL SET_WNDSP5(AWNDSP5, INDX1, INDX2)
```

Where:

Real WNDSP5	wndsp5 for the element indicated by indx1 (input).
Real AWNDSP5 (INDX2-INDX1+1)	wndsp5; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp5, lower value of range (input).
Integer INDX2	wndsp5, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.97. SET_WNDSP6

Use this API to set the wndsp6.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp6(wndsp6, indx1)

set_wndsp6(awndsp6, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP6(WNDSP6, INDX1)

CALL SET_WNDSP6(AWNDSP6, INDX1, INDX2)
```

Where:

Real WNDSP6	wndsp6 for the element indicated by indx1 (input).
Real AWNDSP6 (INDX2-INDX1+1)	wndsp6; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp6, lower value of range (input).
Integer INDX2	wndsp6, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.98. SET_WNDSP7

Use this API to set the wndsp7.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp7(wndsp7, indx1)

set_wndsp7(awndsp7, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP7(WNDSP7, INDX1)

CALL SET_WNDSP7(AWNDSP7, INDX1, INDX2)
```

Where:

Real WNDSP7	wndsp7 for the element indicated by indx1 (input).
Real AWNDSP7 (INDX2-INDX1+1)	wndsp7; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp7, lower value of range (input).
Integer INDX2	wndsp7, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.99. SET_WNDSP8

Use this API to set the wndsp8.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp8(wndsp8, indx1)

set_wndsp8(awndsp8, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP8(WNDSP8, INDX1)

CALL SET_WNDSP8(AWNDSP8, INDX1, INDX2)
```

Where:

Real WNDSP8	wndsp8 for the element indicated by indx1 (input).
Real AWNDSP8 (INDX2-INDX1+1)	wndsp8; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp8, lower value of range (input).
Integer INDX2	wndsp8, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.100. SET_WNDSP9

Use this API to set the wndsp9.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wndsp9(wndsp9, indx1)

set_wndsp9(awndsp9, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WNDSP9(WNDSP9, INDX1)

CALL SET_WNDSP9(AWNDSP9, INDX1, INDX2)
```

Where:

Real WNDSP9	wndsp9 for the element indicated by indx1 (input).
Real AWNDSP9 (INDX2-INDX1+1)	wndsp9; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wndsp9, lower value of range (input).
Integer INDX2	wndsp9, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.101. SET_WPCMND

Use this API to set the wpcmnd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wpcmnd(wpcmnd, indx1)

set_wpcmnd(awpcmnd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WPCMND(WPCMND, INDX1)

CALL SET_WPCMND(AWPCMND, INDX1, INDX2)
```

Where:

Real WPCMND	wpcmnd for the element indicated by indx1 (input).
Real AWPCMND (INDX2-INDX1+1)	wpcmnd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wpcmnd, lower value of range (input).
Integer INDX2	wpcmnd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.102. SET_WPITCH

Use this API to set the wpitch.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wpitch(wpitch, indx1)

set_wpitch(awpitch, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WPITCH(WPITCH, INDX1)

CALL SET_WPITCH(AWPITCH, INDX1, INDX2)
```

Where:

Real WPITCH	wpitch for the element indicated by indx1 (input).
Real AWPITCH (INDX2-INDX1+1)	wpitch; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wpitch, lower value of range (input).
Integer INDX2	wpitch, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.103. SET_WPORDR

Use this API to set the wpordr.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wpordr(wpordr, indx1)

set_wpordr(awpordr, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WPORDR(WPORDR, INDX1)

CALL SET_WPORDR(AWPORDR, INDX1, INDX2)
```

Where:

Real WPORDR	wpordr for the element indicated by indx1 (input).
Real AWPORDR (INDX2-INDX1+1)	wpordr; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wpordr, lower value of range (input).
Integer INDX2	wpordr, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.104. SET_WPRATE

Use this API to set the wprate.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wprate(wprate, indx1)

set_wprate(awprate, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WPRATE(WPRATE, INDX1)

CALL SET_WPRATE(AWPRATE, INDX1, INDX2)
```

Where:

Real WPRATE	wprate for the element indicated by indx1 (input).
Real AWPRATE (INDX2-INDX1+1)	wprate; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wprate, lower value of range (input).
Integer INDX2	wprate, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.105. SET_WPREFPTQ

Use this API to set the wprefptq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wprefptq(wprefptq, indx1)

set_wprefptq(awprefptq, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WPREFPTQ(WPREFPTQ, INDX1)

CALL SET_WPREFPTQ(AWPREFPTQ, INDX1, INDX2)
```

Where:

Real WPREFPTQ	wprefptq for the element indicated by indx1 (input).
Real AWPREFPTQ (INDX2-INDX1+1)	wprefptq; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wprefptq, lower value of range (input).
Integer INDX2	wprefptq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.106. SET_WPREFWGO

Use this API to set the wprefwgo.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wprefwgo(wprefwgo, indx1)

set_wprefwgo(awprefwgo, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WPREFWGO(WPREFWGO, INDX1)

CALL SET_WPREFWGO(AWPREFWGO, INDX1, INDX2)
```

Where:

Real WPREFWGO	wprefwgo for the element indicated by indx1 (input).
Real AWPREFWGO (INDX2-INDX1+1)	wprefwgo; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wprefwgo, lower value of range (input).
Integer INDX2	wprefwgo, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.107. SET_WQCMND

Use this API to set the wqcmnd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wqcmnd(wqcmnd, indx1)

set_wqcmnd(awqcmnd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WQCMND(WQCMND, INDX1)

CALL SET_WQCMND(AWQCMND, INDX1, INDX2)
```

Where:

Real WQCMND	wqcmnd for the element indicated by indx1 (input).
Real AWQCMND (INDX2-INDX1+1)	wqcmnd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wqcmnd, lower value of range (input).
Integer INDX2	wqcmnd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.108. SET_WROTID

Use this API to set the wrotid.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wrotid(wrotid, indx1)

set_wrotid(awrotid, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WROTID(WROTID, INDX1)

CALL SET_WROTID(AWROTID, INDX1, INDX2)
```

Where:

Real WROTID	wrotid for the element indicated by indx1 (input).
Real AWROTID (INDX2-INDX1+1)	wrotid; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wrotid, lower value of range (input).
Integer INDX2	wrotid, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.109. SET_WROTIQ

Use this API to set the wrotiq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wrotiq(wrotiq, indx1)

set_wrotiq(awrotiq, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WROTIQ(WROTIQ, INDX1)

CALL SET_WROTIQ(AWROTIQ, INDX1, INDX2)
```

Where:

Real WROTIQ	wrotiq for the element indicated by indx1 (input).
Real AWROTIQ (INDX2-INDX1+1)	wrotiq; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wrotiq, lower value of range (input).
Integer INDX2	wrotiq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.110. SET_WROTRI

Use this API to set the wrotri.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wrotri(wrotri, indx1)

set_wrotri(awrotri, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WROTRI(WROTRI, INDX1)

CALL SET_WROTRI(AWROTRI, INDX1, INDX2)
```

Where:

Real WROTRI	wrotri for the element indicated by indx1 (input).
Real AWROTRI (INDX2-INDX1+1)	wrotri; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wrotri, lower value of range (input).
Integer INDX2	wrotri, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.111. SET_WROTRV

Use this API to set the wrotrv.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wrotrv(wrotrv, indx1)

set_wrotrv(awrotrv, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WROTRV(WROTRV, INDX1)

CALL SET_WROTRV(AWROTRV, INDX1, INDX2)
```

Where:

Real WROTRV	wrotrv for the element indicated by indx1 (input).
Real AWROTRV (INDX2-INDX1+1)	wrotrv; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wrotrv, lower value of range (input).
Integer INDX2	wrotrv, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.112. SET_WROTVD

Use this API to set the wrotvd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wrotvd(wrotvd, indx1)

set_wrotvd(awrotvd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WROTVD(WROTVD, INDX1)

CALL SET_WROTVD(AWROTVD, INDX1, INDX2)
```

Where:

Real WROTVD	wrotvd for the element indicated by indx1 (input).
Real AWROTVD (INDX2-INDX1+1)	wrotvd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wrotvd, lower value of range (input).
Integer INDX2	wrotvd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.113. SET_WROTVQ

Use this API to set the wrotvq.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wrotvq(wrotvq, indx1)

set_wrotvq(awrotvq, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WROTVQ(WROTVQ, INDX1)

CALL SET_WROTVQ(AWROTVQ, INDX1, INDX2)
```

Where:

Real WROTVQ	wrotvq for the element indicated by indx1 (input).
Real AWROTVQ (INDX2-INDX1+1)	wrotvq; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wrotvq, lower value of range (input).
Integer INDX2	wrotvq, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.114. SET_WTRBSP

Use this API to set the wtrbsp.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wtrbsp(wtrbsp, indx1)

set_wtrbsp(awtrbsp, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WTRBSP(WTRBSP, INDX1)

CALL SET_WTRBSP(AWTRBSP, INDX1, INDX2)
```

Where:

Real WTRBSP	wtrbsp for the element indicated by indx1 (input).
Real AWTRBSP (INDX2-INDX1+1)	wtrbsp; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wtrbsp, lower value of range (input).
Integer INDX2	wtrbsp, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.115. SET_WVLCTY

Use this API to set the wvlcty.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_wvlcty(wvlcty, indx1)

set_wvlcty(awvlcty, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_WVLCTY(WVLCTY, INDX1)

CALL SET_WVLCTY(AWVLCTY, INDX1, INDX2)
```

Where:

Real WVLCTY	wvlcty for the element indicated by indx1 (input).
Real AWVLCTY (INDX2-INDX1+1)	wvlcty; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	wvlcty, lower value of range (input).
Integer INDX2	wvlcty, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.3.116. SET_XADIFD

Use this API to set the xadifd.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
set_xadifd(xadifd, indx1)

set_xadifd(axadifd, indx1, indx2)
```

Fortran command syntax:

```
CALL SET_XADIFD(XADIFD, INDX1)

CALL SET_XADIFD(AXADIFD, INDX1, INDX2)
```

Where:

Real XADIFD	xadifd for the element indicated by indx1 (input).
Real AXADIFD (INDX2-INDX1+1)	xadifd; an array of values for the elements indicated by the indices supplied (input).
Integer INDX1	xadifd, lower value of range (input).
Integer INDX2	xadifd, upper value of range (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4. Operational functions and calculations for dynamics models

This section describes routines that can perform a variety of useful functions for dynamics models.

24.4.1. AUXOUT

Use this API to store auxiliary signal model output in PSSE data structures. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
auxout(idvt, idvx, isgx, varval)
```

Fortran command syntax:

```
CALL AUXOUT(IDVT, IDVX, ISGX, VARVAL)
```

Where:

Integer IDVT	device type (input).
Integer IDVX	device index (input).
Integer ISGX	auxiliary signal index (input).
Real VARVAL	auxiliary signal output value (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.2. BLSTAT

Use this API to get the switched shunt block status.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vblstat = blstat(block, indx)
```

Fortran command syntax:

```
VBLSTAT = BLSTAT(BLOCK, INDX)
```

Where:

Integer VBLSTAT	switched shunt block status (output).
Integer BLOCK	block number (1:8) (input).
Integer INDX	switched shunt array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.3. BSDSCN

Use this API to disconnect bus . Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
bsdscn(indx)
```

Fortran command syntax:

```
CALL BSDSCN(INDX)
```

Where:

Integer INDX	index of ICON that contains the bus to be tripped (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.4. BSDSCN1

Use this API to disconnect bus. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
bsdscn1(indx)
```

Fortran command syntax:

```
CALL BSDSCN1(INDX)
```

Where:

Integer INDX	external bus number (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.5. BSSQNC

Use this API to bus array index from external bus number.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
busindx = bssqnc(busnum)
```

Fortran command syntax:

```
CALL BSSQNC(BUSNUM, BUSINDX)
```

Where:

Integer BUSNUM	external bus number (input).
Integer BUSINDX	bus array index (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.6. BUS_ENCODE_TRIM

Use this API to encode bus number, bus section, name and base voltage left justified in outbuf.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
outbuf = bus_encode_trim(busindx)
```

Fortran command syntax:

```
CALL BUS_ENCODE_TRIM(BUSINDX, OUTBUF)
```

Where:

Integer BUSINDX	bus array index (input).
Character*31 OUTBUF	encoded extended bus name (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.7. CALCULATE_VFE

Use this API to calculate the exciter field current (VFE). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vfe = calculate_vfe(mc)
```

Fortran command syntax:

```
CALL CALCULATE_VFE(MC, VFE)
```

Where:

Integer MC	machine array index (input).
Real VFE	calculated VFE (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.8. CKTIDS

Use this API to encode circuit IDs for DOCU and DYDA output of lines and 2-winding transformers. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
cbuffer = cktids(indx, nbrn)
```

Fortran command syntax:

```
CALL CKTIDS(INDX, NBRN, CBUFFER)
```

Where:

Integer INDX	index of the first ICON that contains the "FROM" bus of the branch(es) (input).
Integer NBRN	number of branches for which the circuit id has to be encoded into the output buffer(s) (input).
Character*6 CBUFFER (nbrn)	output buffer(s) (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.9. CNSTCR

Use this API to get the the factor by which nominal current load is to be multiplied when bus voltage magnitude is less than 0.5 pu.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vcnstcr = cnstcr(vmag)
```

Fortran command syntax:

```
VCNSTCR = CNSTCR(VMAG)
```

Where:

Real VCNSTCR	the factor by which nominal current load is to be multiplied when bus voltage magnitude is less than 0.5 pu (output).
Real VMAG	voltage magnitude (pu) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.10. CNSTPQ

Use this API to get the the factor by which nominal mva load is to be multiplied when bus voltage magnitude is less than the values specified in pqbrak.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vcnstpq = cnstpq(vmag)
```

Fortran command syntax:

```
VCNSTPQ = CNSTPQ(VMAG)
```

Where:

Real VCNSTPQ	the factor by which nominal mva load is to be multiplied when bus voltage magnitude is less than the values specified in pqbrak (output).
Real VMAG	voltage magnitude (pu) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.11. DEADBD2

Use this API to Intentional deadband . Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
y = deadbd2(x, dbh, dbl)
```

Fortran command syntax:

```
CALL DEADBD2(X, DBH, DBL, Y)
```

Where:

Real X	Input to deadband (input).
Real DBH	Deadband high value (input).
Real DBL	deadband low value (input).
Real Y	Deadband output (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.12. DEADBN2

Use this API to model intentional deadband with "Bow-Tie" hysteresis. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
y = deadbn2(dbh, err, dbl, m, x)
```

Fortran command syntax:

```
CALL DEADBN2(DBH, ERR, DBL, M, X, Y)
```

Where:

Real DBH	deadband high value (input).
Real ERR	intentional deadband hysteresis (input).
Real DBL	kdeadband low value (input).
Integer M	ICON index of first of two internal ICONs used as flags (input).
Real X	input signal (input).
Real Y	output signal (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.13. DELAYU

Use this API to simulate time delay. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
delayu(iconinx, conidx, varidx)
```

Fortran command syntax:

```
CALL DELAYU(ICONINX, CONIDX, VARIDX)
```

Where:

Integer ICONINX	index of INTICN used (input).
Integer CONIDX	CON index that holds the delay time (input).
Integer VARIDX	VAR index that holds the timer value (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.14. DOCUCHK

Use this API to validate a CON inputs using test flag. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
errflag = docuchk(connum, varname, severity, test, tstval1, tstval2)
```

Fortran command syntax:

```
CALL DOCUCHK(CONNUM, VARNAME, SEVERITY, TEST, TSTVAL1, TSTVAL2, ERRFLAG)
```

Where:

Integer CONNUM	index for CON being tested (input).
Character VARNAME	string containing the name of the CON to be displayed (input).
Integer SEVERITY	severity of error (use one of the conMsg* parameters) (input).
Integer TEST	type of comparison (use one of the docuChkErr_* parameters) (input).
Real TSTVAL1	first critical value (input).
Real TSTVAL2	second critical value (input).
Logical ERRFLAG	signal test result (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.15. DOCUCHKBRANCH

Use this API to perform DOCU check of a branch and 2-winding transformer specified as ICONs in models. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkbranch(icndx, icnnum, frombus, tobus, cktid)
```

Fortran command syntax:

```
CALL DOCUCHKBRANCH(ICNDX, ICNNUM, FROMBUS, TOBUS, CKTID)
```

Where:

Integer ICNDX	first ICON array index of three consecutive ICONs (input).
Integer ICNNUM	relative ICON index correspoinding to ICNIDX (input).
Character FROMBUS	bus name of from bus (input).
Character TOBUS	bus name of to bus (input).
Character CKTID	circuit ID (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.16. DOCUCHKICON

Use this API to validate an ICON against a range. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
errflag = docuchkicon(icnnum, varname, severity, v1, v2)
```

Fortran command syntax:

```
CALL DOCUCHKICON(ICNNUM, VARNAME, SEVERITY, V1, V2, ERRFLAG)
```

Where:

Integer ICNNUM	index for ICON being tested (input).
Character VARNAME	string containing the name of the ICON to be displayed (input).
Integer SEVERITY	severity of error (use one of the conMsg* parameters) (input).
Integer V1	lower value of range (input).
Integer V2	upper value of range (input).
Logical ERRFLAG	signal test result (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.17. DOCUCHKICONABS

Use this API to validate absolute value of an ICON against a range. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
errflag = docuchkiconabs(icnnum, varname, severity, v1, v2)
```

Fortran command syntax:

```
CALL DOCUCHKICONABS(ICNNUM, VARNAME, SEVERITY, V1, V2, ERRFLAG)
```

Where:

Integer ICNNUM	index for ICON being tested (input).
Character VARNAME	string containing the name of the ICON to be displayed (input).
Integer SEVERITY	severity of error (use one of the conMsg* parameters) (input).
Integer V1	lower value of range (input).
Integer V2	upper value of range (input).
Logical ERRFLAG	signal test result (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.18. DOCUCHKICONMSG

Use this API to produce messages for icon violations. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkiconmsg(icnnum, severity, msg)
```

Fortran command syntax:

```
CALL DOCUCHKICONMSG(ICNNUM, SEVERITY, MSG)
```

Where:

Integer ICNNUM (:)	an array of any length containing index(es) for ICON(s) with a violation. (input).
Integer SEVERITY	severity of error (use one of the conMsg* parameters) (input).
Character MSG (:)	an array of any length (of any length strings) containing error message(s) to be displayed. (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.19. DOCUCHKINT

Use this API to validate a CON against an integer range. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
errflag = docuchkint(connum, varname, severity, v1, v2)
```

Fortran command syntax:

```
CALL DOCUCHKINT(CONNUM, VARNAME, SEVERITY, V1, V2, ERRFLAG)
```

Where:

Integer CONNUM	index for CON being tested (input).
Character VARNAME	string containing the name of the CON to be displayed (input).
Integer SEVERITY	severity of error (use one of the conMsg* parameters) (input).
Integer V1	lower value of range (input).
Integer V2	upper value of range (input).
Logical ERRFLAG	signal test result (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.20. DOCUCHKLOAD

Use this API to perform DOCU check of load specified as ICONs in models. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkload(busidx, icnnum, ididx, idnum, busname, loadid)
```

Fortran command syntax:

```
CALL DOCUCHKLOAD(BUSIDX, ICNNUM, IDIDX, IDNUM, BUSNAME, LOADID)
```

Where:

Integer BUSIDX	index of the ICON that contains the bus number (input).
Integer ICNNUM	relative ICON index corresponding to BUSIDX (input).
Integer IDIDX	index of the ICON that contains the load id (input).
Integer IDNUM	relative ICON index corresponding to IDIDX (input).
Character BUSNAME	Name of bus as it should appear in DOCU check report (input).
Character LOADID	The load ID as it should appear in DOCU check report (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.21. DOCUCHKMACHINE

Use this API to perform DOCU check of machine specified as ICONs in models. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkmachine(busidx, icnnum, ididx, idnum, busname, macid)
```

Fortran command syntax:

```
CALL DOCUCHKMACHINE(BUSIDX, ICNNUM, IDIDX, IDNUM, BUSNAME, MACID)
```

Where:

Integer BUSIDX	index of the ICON that contains the bus number (input).
Integer ICNNUM	relative ICON index corresponding to BUSIDX (input).
Integer IDIDX	index of the ICON that contains the machine id (input).
Integer IDNUM	relative ICON index corresponding to IDIDX (input).
Character BUSNAME	Name of bus as it should appear in DOCU check report (input).
Character MACID	The machine ID as it should appear in DOCU check report (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.22. DOCUCHKMSG

Use this API to produce messages for con violations. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkmsg(connum, severity, msg)
```

Fortran command syntax:

```
CALL DOCUCHKMSG(CONNUM, SEVERITY, MSG)
```

Where:

Integer CONNUM (:)	an array of any length containing index(es) for CON(s) with a violation. (input).
Integer SEVERITY	severity of error (use one of the conMsg* parameters) (input).
Character MSG (:)	an array of any length (of any length strings) containing error message(s) to be displayed. (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.23. DOCUCHKPAIR

Use this API to validate a pair of scaled CON inputs using test flag. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
errflag = docuchkpair(connum1, varname1, f1, connum2, varname2, f2,
 severity, test)
```

Fortran command syntax:

```
CALL DOCUCHKPAIR(CONNUM1, VARNAME1, F1, CONNUM2, VARNAME2, F2, SEVERITY,
 TEST, ERRFLAG)
```

Where:

Integer CONNUM1	first CON index for this model that is being tested (input).
Character VARNAME1	string containing the name of the first CON to be displayed (input).
Real F1	factor for first CON (input).
Integer CONNUM2	second CON for this model that is being tested (input).
Character VARNAME2	string containing the name of the second CON to be displayed (input).
Real F2	factor for second CON (input).
Integer SEVERITY	severity of error (use one of the conMsg* parameters) (input).
Integer TEST	type of comparison (use one of the docuChkErr_* parameters) (input).
Logical ERRFLAG	signal test result (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.24. DOCUCHKPARALLELBRANCH

Use this API to DOCU check of parallel branches specified as ICONs in models. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuchkparallelbranch(icndx, icnidx, frombus, tobus, cktid)
```

Fortran command syntax:

```
CALL DOCUCHKPARALLELBRANCH(ICNDX, ICNIDX, FROMBUS, TOBUS, CKTID)
```

Where:

Integer ICNDX	first ICON array index of three consecutive ICONs (input).
Integer ICNIDX	relative ICON index correspoinding to ICNIDX (input).
Character FROMBUS	name of first bus as it should appear in the DOCU check report (input).
Character TOBUS	name of second bus as it should appear in the DOCU check report (input).
Character CKTID	The circuit ID as it should appear in the DOCU check report (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.25. DOCUCHKREMBUS

Use this API to DOCU check of remote bus specified in models. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
errflg = docuchkrembus(ibus, icnidx, busname)
```

Fortran command syntax:

```
CALL DOCUCHKREMBUS(IBUS, ICNIDX, BUSNAME, ERRFLG)
```

Where:

Integer IBUS	ICON array index that contains the external bus number (input).
Integer ICNIDX	relative ICON index (input).
Character BUSNAME	Name of bus as it should appear in the DOCU check report (input).
Logical ERRFLG	error flag (True: error found, else false) (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.26. DOCUHEADING

Use this API to display model heading and model ID.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
docuheading()
```

Fortran command syntax:

```
CALL DOCUHEADING
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.27. DYN_ADD_SHUNT

Use this API to add a shunt in a model. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dyn_add_shunt(ibus)
```

Fortran command syntax:

```
CALL DYN_ADD_SHUNT(IBUS)
```

Where:

Integer IBUS	bus number (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.28. DYN_REMOVE_SHUNT

Use this API to remove a shunt in a model. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
dyn_remove_shunt(ibus)
```

Fortran command syntax:

```
CALL DYN_REMOVE_SHUNT(IBUS)
```

Where:

Integer IBUS	bus number (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.29. ENCODE_BUS_NAME

Use this API to encode bus name and base voltage left justified in outbuf.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
outbuf = encode_bus_name(busindx)
```

Fortran command syntax:

```
CALL ENCODE_BUS_NAME(BUSINDX, OUTBUF)
```

Where:

Integer BUSINDX	bus array index (input).
Character*18 OUTBUF	encoded bus number, section number, extended bus name (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.30. EXCSAT

Use this API to calculate Exciter saturation factor. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
s = excsat(x, j)
```

Fortran command syntax:

```
CALL EXCSAT(X, S, J)
```

Where:

Real X	input for calculating saturation factor (input).
Real S	Saturation factor (output).
Integer J	first CON array index for 4 values used in calculation (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.31. EXC_FEX

Use this API to Calculates the Fex output (rectfiier regulation equation in IEEE 421p5 Standard). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fex = exc_fex(kc, vinp1, vinp2)
```

Fortran command syntax:

```
CALL EXC_FEX(KC, VINP1, VINP2, FEX)
```

Where:

Real KC	Rectifier regulation factor (input).
Real VINP1	Denominator input (input).
Real VINP2	Numerator input (input).
Real FEX	Fex output (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.32. EXC_FEX_INI

Use this API to Initializes the Fex block (rectifier regulation block). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
three = exc_fex_ini(one, two)
```

Fortran command syntax:

```
CALL EXC_FEX_INI(ONE, TWO, THREE)
```

Where:

Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.33. EXC_PW_SOURCE_2

Use this API to Calculate exciter power source output. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vout = exc_pw_source_2(mc, vinp, ki, kc, vbmax)
```

Fortran command syntax:

```
CALL EXC_PW_SOURCE_2(MC, VINP, KI, KC, VBMAX, VOUT)
```

Where:

Integer MC	machine array index (input).
Real VINP	Input signal (input).
Real KI	Gain (input).
Real KC	Rectifier loading factor (input).
Real VBMAX	Maximum output (input).
Real VOUT	Output of the block (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.34. EXC_PW_SOURCE_A

Use this API to Calculate exciter power source output. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
nine = exc_pw_source_a(mc, vinp, kp, thetap, ki, xl, kc, vbmax)
```

Fortran command syntax:

```
CALL EXC_PW_SOURCE_A(MC, VINP, KP, THETAP, KI, XL, KC, VBMAX, NINE)
```

Where:

Integer MC	machine array index (input).
Real VINP	Input signal (input).
Real KP	Gain (input).
Real THETAP	Angle theta (input).
Real KI	Gain KI (input).
Real XL	reactance (input).
Real KC	Gain KC (input).
Real VBMAX	Maximum block output (input).
Real NINE	ninth argument (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.35. EXC_PW_SOURCE_B

Use this API to Calculate exciter power source output. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vout = exc_pw_source_b(mc, vinp, kp, kc, vbmax)
```

Fortran command syntax:

```
CALL EXC_PW_SOURCE_B(MC, VINP, KP, KC, VBMAX, VOUT)
```

Where:

Integer MC	machine array index (input).
Real VINP	Input signal (input).
Real KP	Gain (input).
Real KC	Rectifier loading factor (input).
Real VBMAX	Maximum output (input).
Real VOUT	Output of the block (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.36. FACTDS_NOGINT

Use this API to admittance matrix factorization without Generic Network Element (GNE) initialization.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
factds_nogint()
```

Fortran command syntax:

```
CALL FACTDS_NOGINT
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.37. FCTBLOCK

Use this API to block a FACTS device.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fctblock(indx)
```

Fortran command syntax:

```
CALL FCTBLOCK(INDX)
```

Where:

Integer INDX	FACTS device array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.38. FCTCNTL0

Use this API to set control mode of a FACTS device to zero.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
fctcntl0(indx)
```

Fortran command syntax:

```
CALL FCTCNTL0(INDX)
```

Where:

Integer INDX	FACTS device array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.39. FCTSTAT

Use this API to get the FACTS device status, true if blocked (i.e. not in-service).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vfctstat = fctstat(indx)
```

Fortran command syntax:

```
VFCTSTAT = FCTSTAT(INDX)
```

Where:

Logical VFCTSTAT	FACTS device status, true if blocked (i.e. not in-service) (output).
Integer INDX	FACTS device array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.40. FLOW1M

Use this API to calculate flow on a branch .

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
flow1m(icnidx, mwidx, mvaridx, mvaidx)
```

Fortran command syntax:

```
CALL FLOW1M(ICNIDX, MWIDX, MVARIDX, MVAIDX)
```

Where:

Integer ICNIDX	ICON index of the first of 3 ICONs used by the model (input).
Integer MWIDX	index of VAR that contains the MW flow (input).
Integer MVARIDX	index of VAR that contains the MVAR flow (input).
Integer MVAIDX	index of VAR that contains the MVA flow (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.41. GEESAT

Use this API to calculate generator saturation factor (exponential saturation). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
s = geesat(x, j)
```

Fortran command syntax:

```
CALL GEESAT(X, S, J)
```

Where:

Real X	input for calculating saturation factor (input).
Real S	Saturation factor (output).
Integer J	first CON array index for 4 values used in calculation (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.42. GENCHK

Use this API to the machine index. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
indx = genchk(ibus, mcid, model)
```

Fortran command syntax:

```
CALL GENCHK(IBUS, MCID, INDX, MODEL)
```

Where:

Integer IBUS	bus array index (input).
Character MCID	machine ID (input).
Integer INDX	machine array slot (output).
Character MODEL	model name (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.43. GENCHK2

Use this API to the machine index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
indx = genchk2(ibus, mcid)
```

Fortran command syntax:

```
CALL GENCHK2(IBUS, MCID, INDX)
```

Where:

Integer IBUS	bus array index (input).
Character MCID	machine ID (input).
Integer INDX	machine array slot (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.44. GENSAT

Use this API to calculate generator saturation factor (quadratic saturation). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
s = gensat(x, j)
```

Fortran command syntax:

```
CALL GENSAT(X, S, J)
```

Where:

Real X	input for calculating saturation factor (input).
Real S	Saturation factor (output).
Integer J	first CON array index for 4 values used in calculation (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.45. GENTRP

Use this API to trip machine. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gentrp(ibus, mcid)
```

Fortran command syntax:

```
CALL GENTRP(IBUS, MCID)
```

Where:

Integer IBUS	bus array index (input).
Character MCID	machine ID (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.46. GETBRANCHSEQCURRENTS

Use this API to calculate branch sequence currents (p.u. of system MVA basse). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
getbranchseqcurrents(ibus, jbus, cktid, vamag, vaang, vbmag, vbang, vcmag,
 vcang)
```

Fortran command syntax:

```
CALL GETBRANCHSEQCURRENTS(IBUS, JBUS, CKTID, VAMAG, VAANG, VBMAG, VBANG,
 VCMAG, VCANG)
```

Where:

Integer IBUS	from bus array index (input).
Integer JBUS	to bus array index (input).
Character CKTID	circuit ID (input).
Integer VAMAG	var index for magnitude of positive sequence current (per unit) (0 to bypass) (input).
Integer VAANG	var index for angle of positive sequence current (degrees) (0 to bypass) (input).
Integer VBMAG	var index for magnitude of negative sequence current (per unit) (0 to bypass) (input).
Integer VBANG	var index for angle of negative sequence current (degrees) (0 to bypass) (input).
Integer VCMAG	var index for magnitude zero sequence current (per unit) (0 to bypass) (input).
Integer VCANG	var index for angle of zero sequence current (degrees) (0 to bypass) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.47. GETFTC

Use this API to the "From", "to", bus index and the circuit id for branches incliding 2-winding transformer.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ibus, jbus, cktid = getftc(dindx)
```

Fortran command syntax:

```
CALL GETFTC(DINDX, IBUS, JBUS, CKTID)
```

Where:

Integer DINDX	branch "double" array index (input).
Integer IBUS	from bus array index (output).
Integer JBUS	to bus array index (output).
Character*2 CKTID	circuit id (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.48. GETFTC3

Use this API to the "From", "to", "third" bus index and the circuit id for 3-winding transformer.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
getftc3(dindx, frbusidx, tobusidx, b3busidx, chkid)
```

Fortran command syntax:

```
CALL GETFTC3(DINDX, FRBUSIDX, TOBUSIDX, B3BUSIDX, CHKID)
```

Where:

Integer DINDX	branch "double" array index (input).
Integer FRBUSIDX	from bus array index (input).
Integer TOBUSIDX	to bus array index (input).
Integer B3BUSIDX	third bus array index (input).
Character CHKID	circuit id (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.49. GETMACHSEQCURRENTS

Use this API to calculate machine sequence currents. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
getmachseqcurrents(ibus, vamag, vaang, vbmag, vbang, vcmag, vcang)
```

Fortran command syntax:

```
CALL GETMACHSEQCURRENTS(IBUS, VAMAG, VAANG, VBMAG, VBANG, VCMAG, VCANG)
```

Where:

Integer IBUS	bus array index (input).
Integer VAMAG	var index for magnitude of positive sequence current (per unit) (0 to bypass) (input).
Integer VAANG	var index for angle of positive sequence current (degrees) (0 to bypass) (input).
Integer VBMAG	var index for magnitude of negative sequence current (per unit) (0 to bypass) (input).
Integer VBANG	var index for angle of negative sequence current (degrees) (0 to bypass) (input).
Integer VCMAG	var index for magnitude zero sequence current (per unit) (0 to bypass) (input).
Integer VCANG	var index for angle of zero sequence current (degrees) (0 to bypass) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.50. GETPHASEVOLTAGES

Use this API to calculate bus phase voltages. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
getphasevoltages(ibus, vamag, vaang, vbmag, vbang, vcmag, vcang)
```

Fortran command syntax:

```
CALL GETPHASEVOLTAGES(IBUS, VAMAG, VAANG, VBMAG, VBANG, VCMAG, VCANG)
```

Where:

Integer IBUS	bus array index (input).
Integer VAMAG	var index for magnitude of positive sequence current (per unit) (0 to bypass) (input).
Integer VAANG	var index for angle of positive sequence current (degrees) (0 to bypass) (input).
Integer VBMAG	var index for magnitude of negative sequence current (per unit) (0 to bypass) (input).
Integer VBANG	var index for angle of negative sequence current (degrees) (0 to bypass) (input).
Integer VCMAG	var index for magnitude zero sequence current (per unit) (0 to bypass) (input).
Integer VCANG	var index for angle of zero sequence current (degrees) (0 to bypass) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.51. GETSEQUENCEVOLTAGES

Use this API to calculate bus sequence voltages. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
getsequencevoltages(ibus, vamag, vaang, vbmag, vbang, vcmag, vcang)
```

Fortran command syntax:

```
CALL GETSEQUENCEVOLTAGES(IBUS, VAMAG, VAANG, VBMAG, VBANG, VCMAG, VCANG)
```

Where:

Integer IBUS	bus array index (input).
Integer VAMAG	var index for magnitude of positive sequence current (per unit) (0 to bypass) (input).
Integer VAANG	var index for angle of positive sequence current (degrees) (0 to bypass) (input).
Integer VBMAG	var index for magnitude of negative sequence current (per unit) (0 to bypass) (input).
Integer VBANG	var index for angle of negative sequence current (degrees) (0 to bypass) (input).
Integer VCMAG	var index for magnitude zero sequence current (per unit) (0 to bypass) (input).
Integer VCANG	var index for angle of zero sequence current (degrees) (0 to bypass) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.52. GET_EXCITER_VEMAX

Use this API to Calculate exciter VEMAX limit (when VEMAX is a function of exciter saturation) . Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vemax = get_exciter_vemax(kd, ke, vfemax, j)
```

Fortran command syntax:

```
CALL GET_EXCITER_VEMAX(KD, KE, VFEMAX, J, VEMAX)
```

Where:

Real KD	Demagnetization factor (input).
Real KE	exciter field resistance constant (input).
Real VFEMAX	Exciter field current maximum (input).
Integer J	first CON array index for 4 values used in calculation (input).
Real VEMAX	Fourth argument (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.53. GTRSUB

Use this API to trip machine.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
gtrsub(indx)
```

Fortran command syntax:

```
CALL GTRSUB(INDX)
```

Where:

Integer INDX	machine array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.54. INT_MODE1

Use this API to get the INTEGRATOR BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vint_mode1, ierr = int_mode1(one, two, three)
```

Fortran command syntax:

```
VINT_MODE1 = INT_MODE1(ONE, TWO, THREE, IERR)
```

Where:

Real VINT_MODE1	INTEGRATOR BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Integer THREE	third argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.55. INT_MODE2

Use this API to get the INTEGRATOR BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vint_mode2 = int_mode2(one, two, three)
```

Fortran command syntax:

```
VINT_MODE2 = INT_MODE2(ONE, TWO, THREE)
```

Where:

Real VINT_MODE2	INTEGRATOR BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Integer THREE	third argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.56. INT_MODE3

Use this API to get the INTEGRATOR BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vint_mode3 = int_mode3(one, two, three)
```

Fortran command syntax:

```
VINT_MODE3 = INT_MODE3(ONE, TWO, THREE)
```

Where:

Real VINT_MODE3	INTEGRATOR BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Integer THREE	third argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.57. LAG_MODE1

Use this API to get the LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vlag_mode1, ierr = lag_mode1(one, two, three, four)
```

Fortran command syntax:

```
VLAG_MODE1 = LAG_MODE1(ONE, TWO, THREE, FOUR, IERR)
```

Where:

Real VLAG_MODE1	LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.58. LAG_MODE2

Use this API to get the LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vlag_mode2 = lag_mode2(one, two, three, four)
```

Fortran command syntax:

```
VLAG_MODE2 = LAG_MODE2(ONE, TWO, THREE, FOUR)
```

Where:

Real VLAG_MODE2	LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.59. LAG_MODE3

Use this API to get the LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vlag_mode3 = lag_mode3(one, two, three, four)
```

Fortran command syntax:

```
VLAG_MODE3 = LAG_MODE3(ONE, TWO, THREE, FOUR)
```

Where:

Real VLAG_MODE3	LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.60. LDLG_MODE1

Use this API to get the LEAD-LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vldlg_mode1, ierr = ldlg_mode1(one, two, three, four, fifth)
```

Fortran command syntax:

```
VLDLG_MODE1 = LDLG_MODE1(ONE, TWO, THREE, FOUR, FIFTH, IERR)
```

Where:

Real VLDLG_MODE1	LEAD-LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.61. LDLG_MODE2

Use this API to get the LEAD-LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vldlg_mode2 = ldlg_mode2(one, two, three, four, fifth)
```

Fortran command syntax:

```
VLDLG_MODE2 = LDLG_MODE2(ONE, TWO, THREE, FOUR, FIFTH)
```

Where:

Real VLDLG_MODE2	LEAD-LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.62. LDLG_MODE3

Use this API to get the LEAD-LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vldlg_mode3 = ldlg_mode3(one, two, three, four, fifth)
```

Fortran command syntax:

```
VLDLG_MODE3 = LDLG_MODE3(ONE, TWO, THREE, FOUR, FIFTH)
```

Where:

Real VLDLG_MODE3	LEAD-LAG BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.63. LDSHED

Use this API to shed fraction of load. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
ldshed(indx, fraction)
```

Fortran command syntax:

```
CALL LDSHED(INDX, FRACTION)
```

Where:

Integer INDX	load array index (input).
Real FRACTION	fraction of load to shed (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.64. LINESW

Use this API to trip and reclose branches. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
linesw(icndx, action)
```

Fortran command syntax:

```
CALL LINESW(ICNDX, ACTION)
```

Where:

Integer ICNDX	index of ICON that contains the "FROM" bus of the branch to be tripped (input).
Integer ACTION	0=TRIP, 1=RECLOSE (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.65. LINRCL

Use this API to reclose branches including 2-winding transformer. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
linrcl(icndx)
```

Fortran command syntax:

```
CALL LINRCL(ICNDX)
```

Where:

Integer ICNDX	STARTING ICON CONTAINING BRANCH DESCRIPTION (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.66. LINTRP

Use this API to LINTRP. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
lintrp(icndx)
```

Fortran command syntax:

```
CALL LINTRP(ICNDX)
```

Where:

Integer ICNDX	STARTING ICON CONTAINING BRANCH DESCRIPTION (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.67. LOADTYPE

Use this API to get the load type.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vloadtype = loadtype(indx)
```

Fortran command syntax:

```
VLOADTYPE = LOADTYPE(INDX)
```

Where:

Character*12 VLOADTYPE	load type (output).
Integer INDX	load array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.68. LODCHK

Use this API to load array index.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
indx = lodchk(ibus, ldid)
```

Fortran command syntax:

```
CALL LODCHK(IBUS, LDID, INDX)
```

Where:

Integer IBUS	bus array index (input).
Character LDID	load ID (input).
Integer INDX	load array slot (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.69. LOGSUBSYSTEMLOADMODEL

Use this API to writing out the load information in load model DOCU output. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
logsubsystemloadmodel(mdlnam, indx)
```

Fortran command syntax:

```
CALL LOGSUBSYSTEMLOADMODEL(MDLNAM, INDX)
```

Where:

Character MDLNAM	model name (input).
Integer INDX	load array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.70. MODELID

Use this API to get the compose model ID.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vmodelid = modelid()
```

Fortran command syntax:

```
VMODELID = MODELID()
```

Where:

Character*180 VMODELID	compose model ID (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.71. MODESW

Use this API to get the switched-shunt control mode, 0: locked, 1: discrete adjustment controlling voltage locally or at bus SWREG, 2: continuous adjustment controlling voltage locally or at bus SWREG, 3: discrete adjustment controlling the reactive power output of the plant at bus SWREG, 4: discrete adjustment controlling the reactive power output of the VSC dc, line converter at bus SWREG of the VSC dc line for which the name is specified as RMIDNT, 5: discrete adjustment controlling the admittance setting of the switched shunt at bus SWREG, 6: discrete adjustment controlling the reactive power output of the shunt element of the FACTS device for which the name is specified as RMIDNT.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vmodesw = modesw(indx)
```

Fortran command syntax:

```
VMODESW = MODESW(INDX)
```

Where:

Integer VMODESW	switched-shunt control mode, 0: locked, 1: discrete adjustment controlling voltage locally or at bus SWREG, 2: continuous adjustment controlling voltage locally or at bus SWREG, 3: discrete adjustment controlling the reactive power output of the plant at bus SWREG, 4: discrete adjustment controlling the reactive power output of the VSC dc, line converter at bus SWREG of the VSC dc line for which the name is specified as RMIDNT, 5: discrete adjustment controlling the admittance setting of the switched shunt at bus SWREG, 6: discrete adjustment controlling the reactive power output of the shunt element of the FACTS device for which the name is specified as RMIDNT (output).
Integer INDX	switched shunt array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.72. NOTMID

Use this API to write out information if the model is not ready for use in extended term simulation. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
notmid()
```

Fortran command syntax:

```
CALL NOTMID
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.73. NUMBUS

Use this API to get the external bus number.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnumbus = numbus(indx)
```

Fortran command syntax:

```
VNUMBUS = NUMBUS(INDX)
```

Where:

Integer VNUMBUS	external bus number (output).
Integer INDX	bus array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.74. NWINT_MODE1

Use this API to get the NON-WINDUP INTEGRATOR BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwint_mode1, ierr = nwint_mode1(one, two, three, four, fifth)
```

Fortran command syntax:

```
VNWINT_MODE1 = NWINT_MODE1(ONE, TWO, THREE, FOUR, FIFTH, IERR)
```

Where:

Real VNWINT_MODE1	NON-WINDUP INTEGRATOR BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.75. NWINT_MODE2

Use this API to get the NON-WINDUP INTEGRATOR BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwint_mode2 = nwint_mode2(one, two, three, four, fifth)
```

Fortran command syntax:

```
VNWINT_MODE2 = NWINT_MODE2(ONE, TWO, THREE, FOUR, FIFTH)
```

Where:

Real VNWINT_MODE2	NON-WINDUP INTEGRATOR BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.76. NWINT_MODE3

Use this API to get the NON-WINDUP INTEGRATOR BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwint_mode3 = nwint_mode3(one, two, three, four, fifth)
```

Fortran command syntax:

```
VNWINT_MODE3 = NWINT_MODE3(ONE, TWO, THREE, FOUR, FIFTH)
```

Where:

Real VNWINT_MODE3	NON-WINDUP INTEGRATOR BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.77. NWLAG_MODE1

Use this API to get the NON-WINDUP LAG BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwlag_mode1, ierr = nwlag_mode1(kp, two, three, four, fifth, six)
```

Fortran command syntax:

```
VNWLAG_MODE1 = NWLAG_MODE1(KP, TWO, THREE, FIFTH, SIX, IERR)
```

Where:

Real VNWLAG_MODE1	NON-WINDUP LAG BLOCK (IEEE Std. 421.5 2005) (output).
Real KP	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Integer SIX	sixth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.78. NWLAG_MODE2

Use this API to get the NON-WINDUP LAG BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwlag_mode2 = nwlag_mode2(one, two, three, four, fifth, six)
```

Fortran command syntax:

```
VNWLAG_MODE2 = NWLAG_MODE2(ONE, TWO, THREE, FOUR, FIFTH, SIX)
```

Where:

Real VNWLAG_MODE2	NON-WINDUP LAG BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Integer SIX	sixth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.79. NWLAG_MODE3

Use this API to get the NON-WINDUP LAG BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwlag_mode3 = nwlag_mode3(one, two, three, four, fifth, six)
```

Fortran command syntax:

```
VNWLAG_MODE3 = NWLAG_MODE3(ONE, TWO, THREE, FOUR, FIFTH, SIX)
```

Where:

Real VNWLAG_MODE3	NON-WINDUP LAG BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Integer SIX	sixth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.80. NWLDLG_MODE1

Use this API to get the NON-WINDUP LEAD-LAG BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwldlg_mode1, ierr = nwldlg_mode1(kp, two, three, four, fifth, six,
 seven)
```

Fortran command syntax:

```
VNWLDLG_MODE1 = NWLDLG_MODE1(KP, TWO, THREE, FIFTH, SIX, SEVEN, IERR)
```

Where:

Real VNWLDLG_MODE1	NON-WINDUP LEAD-LAG BLOCK (IEEE Std. 421.5 2005) (output).
Real KP	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Integer SEVEN	seventh argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.81. NWLDLG_MODE2

Use this API to get the NON-WINDUP LEAD-LAG BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwldlg_mode2 = nwldlg_mode2(one, two, three, four, fifth, six, seven)
```

Fortran command syntax:

```
VNWLDLG_MODE2 = NWLDLG_MODE2(ONE, TWO, THREE, FIFTH, SIX, SEVEN)
```

Where:

Real VNWLDLG_MODE2	NON-WINDUP LEAD-LAG BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Integer SEVEN	seventh argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.82. NWLDLG_MODE3

Use this API to get the NON-WINDUP LEAD-LAG BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwldlg_mode3 = nwldlg_mode3(one, two, three, four, fifth, six, seven)
```

Fortran command syntax:

```
VNWLDLG_MODE3 = NWLDLG_MODE3(ONE, TWO, THREE, FIFTH, SIX, SEVEN)
```

Where:

Real VNWLDLG_MODE3	NON-WINDUP LEAD-LAG BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Integer SEVEN	seventh argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.83. NWPIB_MODE1

Use this API to get the NWPIB_MODE1. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpib_mode1, ierr = nwpib_mode1(kp, ki, vrmax, vrmin, vroutin, kk)
```

Fortran command syntax:

```
VNWPIB_MODE1 = NWPIB_MODE1(KP, KI, VRMAX, VROUTIN, KK, IERR)
```

Where:

Real VNWPIB_MODE1	NWPIB_MODE1 (output).
Real KP	proportional gain (input).
Real KI	integral gain (input).
Real VRMAX	MAX limit (input).
Real VRMIN	MIN limit (input).
Real VROUTIN	input value of the block output (input).
Integer KK	state index (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.84. NWPIB_MODE2

Use this API to get the NWPIB_MODE2. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpib_mode2 = nwpib_mode2(one, ki, three, four, five, six)
```

Fortran command syntax:

```
VNWPIB_MODE2 = NWPIB_MODE2(ONE, KI, THREE, FOUR, FIVE, SIX)
```

Where:

Real VNWPIB_MODE2	NWPIB_MODE2 (output).
Real ONE	first argument (input).
Real KI	integral gain (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIVE	fifth argument (input).
Integer SIX	sixth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.85. NWPIB_MODE3

Use this API to get the NWPIB_MODE3. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpib_mode3 = nwpib_mode3(one, two, three, four, five, six)
```

Fortran command syntax:

```
VNWPIB_MODE3 = NWPIB_MODE3(ONE, TWO, THREE, FOUR, FIVE, SIX)
```

Where:

Real VNWPIB_MODE3	NWPIB_MODE3 (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIVE	fifth argument (input).
Integer SIX	sixth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.86. NWPID_MODE1

Use this API to get the NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpid_mode1, ierr = nwpid_mode1(kp, two, three, four, fifth, six, seven,
 eight, nine)
```

Fortran command syntax:

```
VNWPID_MODE1 = NWPID_MODE1(KP, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT, NINE,
 IERR)
```

Where:

Real VNWPID_MODE1	NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005) (output).
Real KP	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.87. NWPID_MODE2

Use this API to get the NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpid_mode2 = nwpid_mode2(one, two, three, four, fifth, six, seven,
 eight, nine)
```

Fortran command syntax:

```
VNWPID_MODE2 = NWPID_MODE2(ONE, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT,
 NINE)
```

Where:

Real VNWPID_MODE2	NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.88. NWPID_MODE3

Use this API to get the NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpid_mode3 = nwpid_mode3(one, two, three, four, fifth, six, seven,
 eight, nine)
```

Fortran command syntax:

```
VNWPID_MODE3 = NWPID_MODE3(ONE, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT,
 NINE)
```

Where:

Real VNWPID_MODE3	NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.89. NWPIPW_MODE1

Use this API to get the NWPIPW_MODE1. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpipw_mode1, ierr = nwpipw_mode1(kp, two, three, four, fifth, six,
 seven, eight, nine)
```

Fortran command syntax:

```
VNWPIPW_MODE1 = NWPIPW_MODE1(KP, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT,
 NINE, IERR)
```

Where:

Real VNWPIPW_MODE1	NWPIPW_MODE1 (output).
Real KP	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.90. NWPIPW_MODE2

Use this API to get the NWPIPW_MODE2. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpipw_mode2 = nwpipw_mode2(one, two, three, four, fifth, six, seven,
 eight, nine)
```

Fortran command syntax:

```
VNWPIPW_MODE2 = NWPIPW_MODE2(ONE, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT,
 NINE)
```

Where:

Real VNWPIPW_MODE2	NWPIPW_MODE2 (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.91. NWPIPW_MODE3

Use this API to get the NWPIPW_MODE3. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpipw_mode3 = nwpipw_mode3(one, two, three, four, fifth, six, seven,
 eight, nine)
```

Fortran command syntax:

```
VNWPIPW_MODE3 = NWPIPW_MODE3(ONE, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT,
 NINE)
```

Where:

Real VNWPIPW_MODE3	NWPIPW_MODE3 (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.92. NWPI_MODE1

Use this API to get the NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpi_mode1, ierr = nwpi_mode1(one, two, three, four, fifth)
```

Fortran command syntax:

```
VNWPI_MODE1 = NWPI_MODE1(ONE, TWO, THREE, FOUR, FIFTH, IERR)
```

Where:

Real VNWPI_MODE1	NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.93. NWPI_MODE2

Use this API to get the NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpi_mode2 = nwpi_mode2(one, two, three, four, fifth)
```

Fortran command syntax:

```
VNWPI_MODE2 = NWPI_MODE2(ONE, TWO, THREE, FOUR, FIFTH)
```

Where:

Real VNWPI_MODE2	NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.94. NWPI_MODE3

Use this API to get the NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vnwpi_mode3 = nwpi_mode3(one, two, three, four, fifth)
```

Fortran command syntax:

```
VNWPI_MODE3 = NWPI_MODE3(ONE, TWO, THREE, FOUR, FIFTH)
```

Where:

Real VNWPI_MODE3	NON-WINDUP PID BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Integer FIFTH	fifth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.95. ORD2_MODE1

Use this API to get the 2ND ORDER BLOCK WITHOUT LIMITS. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vord2_mode1, ierr = ord2_mode1(kp, two, three, four, fifth, six, seven,
 eight, nine)
```

Fortran command syntax:

```
VORD2_MODE1 = ORD2_MODE1(KP, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT, NINE,
 IERR)
```

Where:

Real VORD2_MODE1	2ND ORDER BLOCK WITHOUT LIMITS (output).
Real KP	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.96. ORD2_MODE2

Use this API to get the 2ND ORDER BLOCK WITHOUT LIMITS. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vord2_mode2 = ord2_mode2(one, two, three, four, fifth, six, seven, eight,
 nine)
```

Fortran command syntax:

```
VORD2_MODE2 = ORD2_MODE2(ONE, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT, NINE)
```

Where:

Real VORD2_MODE2	2ND ORDER BLOCK WITHOUT LIMITS (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.97. ORD2_MODE3

Use this API to get the 2ND ORDER BLOCK WITHOUT LIMITS. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vord2_mode3 = ord2_mode3(one, two, three, four, fifth, six, seven, eight,
 nine)
```

Fortran command syntax:

```
VORD2_MODE3 = ORD2_MODE3(ONE, TWO, THREE, FIFTH, SIX, SEVEN, EIGHT, NINE)
```

Where:

Real VORD2_MODE3	2ND ORDER BLOCK WITHOUT LIMITS (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Real SIX	sixth argument (input).
Real SEVEN	sixth argument (input).
Integer EIGHT	eighth argument (input).
Integer NINE	ninth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.98. OUTPUTMODELMSGS

Use this API to write out messages during simulation initialization.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
outputmodelmsgs(buffers, nlines, severity)
```

Fortran command syntax:

```
CALL OUTPUTMODELMSGS(BUFFERS, NLINES, SEVERITY)
```

Where:

Character BUFFERS (NLINES)	An array of NLIN strings (of any length) containing messages to be displayed. (input).
Integer NLINES	number of lines of messages in BUFFERS (input).
Integer SEVERITY	severity of error (use one of the MsgType_* parameters) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.99. PID_MODE1

Use this API to get the PID BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vpid_mode1, ierr = pid_mode1(kp, two, three, four, fifth, six, seven)
```

Fortran command syntax:

```
VPID_MODE1 = PID_MODE1(KP, TWO, THREE, FIFTH, SIX, SEVEN, IERR)
```

Where:

Real VPID_MODE1	PID BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real KP	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Integer SIX	sixth argument (input).
Integer SEVEN	seventh argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.100. PID_MODE2

Use this API to get the PID BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vpid_mode2 = pid_mode2(one, two, three, four, fifth, six, seven)
```

Fortran command syntax:

```
VPID_MODE2 = PID_MODE2(ONE, TWO, THREE, FIFTH, SIX, SEVEN)
```

Where:

Real VPID_MODE2	PID BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Integer SIX	sixth argument (input).
Integer SEVEN	seventh argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.101. PID_MODE3

Use this API to get the PID BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vpid_mode3 = pid_mode3(one, two, three, four, fifth, six, seven)
```

Fortran command syntax:

```
VPID_MODE3 = PID_MODE3(ONE, TWO, THREE, FIFTH, SIX, SEVEN)
```

Where:

Real VPID_MODE3	PID BLOCK WITHOUT LIMITS (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Real FOUR	fourth argument (input).
Real FIFTH	fifth argument (input).
Integer SIX	sixth argument (input).
Integer SEVEN	seventh argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.102. PI_MODE1

Use this API to get the PI BLOCK WITHOUT LIMITS. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vpi_mode1, ierr = pi_mode1(one, two, three, four)
```

Fortran command syntax:

```
VPI_MODE1 = PI_MODE1(ONE, TWO, THREE, FOUR, IERR)
```

Where:

Real VPI_MODE1	PI BLOCK WITHOUT LIMITS (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.103. PI_MODE2

Use this API to get the PI BLOCK WITHOUT LIMITS. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vpi_mode2 = pi_mode2(one, two, three, four)
```

Fortran command syntax:

```
VPI_MODE2 = PI_MODE2(ONE, TWO, THREE, FOUR)
```

Where:

Real VPI_MODE2	PI BLOCK WITHOUT LIMITS (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.104. PI_MODE3

Use this API to get the PI BLOCK WITHOUT LIMITS. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vpi_mode3 = pi_mode3(one, two, three, four)
```

Fortran command syntax:

```
VPI_MODE3 = PI_MODE3(ONE, TWO, THREE, FOUR)
```

Where:

Real VPI_MODE3	PI BLOCK WITHOUT LIMITS (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.105. PSILD

Use this API to get the load adjustment multiplier.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vpsild = psild(loadidx)
```

Fortran command syntax:

```
VPSILD = PSILD(LOADIDX)
```

Where:

Real VPSILD	load adjustment multiplier (output).
Integer LOADIDX	load array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.106. PTOTAL

Use this API to power totals (AL subsystem). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
ptotal(icnidx, varidx)
```

Fortran command syntax:

```
CALL PTOTAL(ICNIDX, VARIDX)
```

Where:

Integer ICNIDX	'icon' index for area/zone/owner number (input).
Integer VARIDX	starting 'var' index [uses var(j) through var(j+4)] (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.107. PTOTAR

Use this API to power totals (AR subsystem). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
ptotar(icnidx, varidx)
```

Fortran command syntax:

```
CALL PTOTAR(ICNIDX, VARIDX)
```

Where:

Integer ICNIDX	'icon' index for area/zone/owner number (input).
Integer VARIDX	starting 'var' index [uses var(j) through var(j+4)] (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.108. PTOTOW

Use this API to power totals (Owner subsystem). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
ptotow(icnidx, varidx)
```

Fortran command syntax:

```
CALL PTOTOW(ICNIDX, VARIDX)
```

Where:

Integer ICNIDX	'icon' index for area/zone/owner number (input).
Integer VARIDX	starting 'var' index [uses var(j) through var(j+4)] (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.109. PTOTZN

Use this API to power totals (ZN subsystem). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
ptotzn(icnidx, varidx)
```

Fortran command syntax:

```
CALL PTOTZN(ICNIDX, VARIDX)
```

Where:

Integer ICNIDX	'icon' index for area/zone/owner number (input).
Integer VARIDX	starting 'var' index [uses var(j) through var(j+4)] (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.110. RCL3WT

Use this API to reclose 3-winding transformer.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
rcl3wt(indx)
```

Fortran command syntax:

```
CALL RCL3WT(INDX)
```

Where:

Integer INDX	3-winding transformer array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.111. SHOWMODEL

Use this API to display model ID.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
showmodel(vdev)
```

Fortran command syntax:

```
CALL SHOWMODEL(VDEV)
```

Where:

Integer VDEV	virtual device code (use one of the W7* parameters) (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.112. SHOW_MODEL_INDICIES

Use this API to outputs starting CON, STATE, VAR and ICON indices for model DOCU output. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
show_model_indicies(first_icon, last_icon, first_con, last_con,
 first_state, last_state, first_var, last_var, first_ric, last_ric)
```

Fortran command syntax:

```
CALL SHOW_MODEL_INDICIES(FIRST_ICON, LAST_ICON, FIRST_CON, LAST_CON,
 FIRST_STATE, LAST_STATE, FIRST_VAR, LAST_VAR, FIRST_RIC, LAST_RIC)
```

Where:

Integer FIRST_ICON	first ICON index of that model (input).
Integer LAST_ICON	last ICON index of that model (input).
Integer FIRST_CON	first CON index of that model (input).
Integer LAST_CON	last CON index of that model (input).
Integer FIRST_STATE	first STATE index of that model (input).
Integer LAST_STATE	last STATE index of that model (input).
Integer FIRST_VAR	first VAR index of that model (input).
Integer LAST_VAR	last VAR index of that model (input).
Integer FIRST_RIC	first RIC index of that model (input).
Integer LAST_RIC	last RIC index of that model (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.113. SHUNT

Use this API to get the total in-service shunt admittance at bus (MVA).

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vshunt = shunt(busindx)
```

Fortran command syntax:

```
VSHUNT = SHUNT(BUSINDX)
```

Where:

Complex VSHUNT	total in-service shunt admittance at bus (MVA) (output).
Integer BUSINDX	bus array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.114. SWCAP

Use this API to simulate time delay. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
swcap(iconinx, conidx, varidx)
```

Fortran command syntax:

```
CALL SWCAP(ICONINX, CONIDX, VARIDX)
```

Where:

Integer ICONINX	index of ICON that contains the bus number (input).
Integer CONIDX	starting CON index of the model (input).
Integer VARIDX	VAR index that holds the timer value (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.115. SWSSTAT

Use this API to get the switched shunt status, true if in-service.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vswsstat = swsstat(indx)
```

Fortran command syntax:

```
VSWSSTAT = SWSSTAT(INDX)
```

Where:

Logical VSWSSTAT	switched shunt status, true if in-service (output).
Integer INDX	switched shunt array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.116. TRP3WT

Use this API to trip 3-winding transformer.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
trp3wt(indx)
```

Fortran command syntax:

```
CALL TRP3WT(INDX)
```

Where:

Integer INDX	3-winding transformer array index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.117. UEL2CUDM

Use this API to calls the PSSE model UEL2C. Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.1.0.

Python command syntax:

```
uel2cudm(mcindx, alocindx)
```

Fortran command syntax:

```
CALL UEL2CUDM(MCINDX, ALOCINDX)
```

Where:

Integer MCINDX	machine index (input).
Integer ALOCINDX	allocation index (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.118. WSHOUT_MODE1

Use this API to get the WASH-OUT BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vwshout_mode1, ierr = wshout_mode1(one, two, three, four)
```

Fortran command syntax:

```
VWSHOUT_MODE1 = WSHOUT_MODE1(ONE, TWO, THREE, FOUR, IERR)
```

Where:

Real VWSHOUT_MODE1	WASH-OUT BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).
Integer IERR	error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.119. WSHOUT_MODE2

Use this API to get the WASH-OUT BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vwshout_mode2 = wshout_mode2(one, two, three, four)
```

Fortran command syntax:

```
VWSHOUT_MODE2 = WSHOUT_MODE2(ONE, TWO, THREE, FOUR)
```

Where:

Real VWSHOUT_MODE2	WASH-OUT BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

24.4.120. WSHOUT_MODE3

Use this API to get the WASH-OUT BLOCK (IEEE Std. 421.5 2005). Usage is described in the Model Functions Chapter of the Model Library Manual.

This API routine was first introduced in release 36.0.0.

Python command syntax:

```
vwshout_mode3 = wshout_mode3(one, two, three, four)
```

Fortran command syntax:

```
VWSHOUT_MODE3 = WSHOUT_MODE3(ONE, TWO, THREE, FOUR)
```

Where:

Real VWSHOUT_MODE3	WASH-OUT BLOCK (IEEE Std. 421.5 2005) (output).
Real ONE	first argument (input).
Real TWO	second argument (input).
Real THREE	third argument (input).
Integer FOUR	fourth argument (input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 25
PSSARRAYS Module

The pssarrays module provides Python functions to retrieve PSS®E solution results in arrays. These arrays can then be used in a Python script for post-processing, to create customized reports for export to a text file or an Excel spreadsheet, etc.

These functions do not record themselves when called while recording is enabled in PSS®E.

Please note that these routines are members of the pssarrays Python extension module, not psspy. The syntax on the following pages assumes that the module has been imported by executing:

```
import pssarrays
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.1. ACCC_SUMMARY

This function returns AC Contingency monitored element labels, contingency labels etc.

Python syntax:

```
rlst = pssarrays.accc_summary(accfile)
```

where:

string	accfile	Is the name of Contingency Solution Output file (input, no default allowed), maximum string length=260

Returned object 'rlst' contains the following attributes:

integer	rlst.ierr	Is the error code =0 no error occurred =4 error opening accfile =5 error reading accfile =6 prerequisite requirements for function not met
integer	rlst.acccsize.nmline	Is the number of monitored branches
integer	rlst.acccsize.ninter	Is the number of monitored interfaces
integer	rlst.acccsize.ncase	Is the number of contingencies + 1 (for base case)
integer	rlst.acccsize.nmvbus	Is the number of voltage monitored buses
integer	rlst.acccsize.nmvrec	Is the number of voltage monitored records
integer	rlst.acccsize.nmvbusrec	Is the number of voltage monitored bus records
integer	rlst.acccsize.nbus	Is the number of buses in the case
integer	rlst.acccsize. ncntlshed	Is the number of load sheds due to dispatch and contingency
integer	rlst.acccsize. ntrplshed	Is the number of load sheds due to tripping
integer	rlst.acccsize. ncactlshed	Is the number of load sheds due to corrective actions
integer	rlst.acccsize. ncactgdisp	Is the number of generation dispatched due to corrective actions
integer	rlst.acccsize. ncactphsftr	Is the number of phase shifter changed due to corrective actions
integer	rlst.acccsize.nmlmtb	Is the number of voltage limit bus records
integer	rlst.acccsize.nmlmtn	Is the number of voltage limit node records
string	rlst.casetitle.line1	Is the case short title line 1, maximum string length=60
string	rlst.casetitle.line2	Is the case short title line 2, maximum string length=60
string	rlst.file.acc	Is the name of contingency output (.acc) file, maximum string length=260

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

string	rlst.file.sav	Is the name of saved case (.sav) file, maximum string length=260
string	rlst.file.dfx	Is the name of distribution factor data (.dfx) file, maximum string length=260
string	rlst.file.sub	Is the name of subsystem definition data (.sub) file, maximum string length=260
string	rlst.file.mon	Is the name of monitored element data (.mon) file, maximum string length=260
string	rlst.file.con	Is the name of contingency description data (.con) file, maximum string length=260
string	rlst.file.thr	Is the name of load throwover data (.thr) file, maximum string length=260
string	rlst.file.inl	Is the name of unit inertia and governor data (.inl) file, maximum string length=260
string	rlst.file.trp	Is the name of tripping element data (.trp) file, maximum string length=260
string	rlst.melement	Is a list of length (nmline+ninter) of monitored branch and interface names, maximum string length=54
float	rlst.rating.a	Is a list of length (nmline+ninter) of monitored element rating A
float	rlst.rating.b	Is a list of length (nmline+ninter) of monitored element rating B
float	rlst.rating.c	Is a list of length (nmline+ninter) of monitored element rating C
string	rlst.mvbuslabel	Is a list of length (nmvbusrec) of monitored voltage bus label, maximum string length=25
string	rlst.mvreclabel	Is a list of length (nmvbusrec) of monitored voltage record label, maximum string length=20
float	rlst.mvrecmax	Is a list of length (nmvbusrec) of monitored voltage bus maximum
float	rlst.mvrecmin	Is a list of length (nmvbusrec) of monitored voltage bus minimum
string	rlst.mvrectype	Is a list of length (nmvbusrec) of monitored voltage record type (RANGE / DEVIATION), maximum string length=9
string	rlst.colabel	Is a list of length (ncase) of contingency labels, maximum string length=12
string	rlst.busname	Is a list of length (nbus) of extended bus names, maximum string length=25
string	rlst.mvlbusreclabel	Is a list of length (nmlmtb) indicating the ordering of emergency monitored voltage limit bus records (mvbasel and mvemergn)
string	rlst.mvbasel	Is a list of length (nmlmtb) of normal monitored voltage limit bus records in order of [[low_limit],[high_limit]]

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

string	rlst.mvemergn	Is a list of length (nmlmtb) of emergency monitored voltage limit bus records in order of [[low_limit],[high_limit]]
string	rlst.mvlnodreclabel	Is a list of length (nmlmtn) of the ordering of emergency monitored voltage limit node records (mvnbasel and mvnemergn)
string	rlst.mvnbasel	Is a list of length (nmlmtn) of monitored voltage limit node records in order of [[low_limit],[high_limit]]
string	rlst.mvnemergn	Is a list of length (nmlmtn) of emergency monitored voltage limit node records in order of [[low_limit],[high_limit]]

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.2. ACCC_SOLUTION

This function returns ACCC post-contingency, post-tripping or post-corrective action solution monitored MVA flows, ampere flows and bus voltages for one contingency.

Python syntax:

```
rlst = pssarrays.accc_solution(accfile, colabel, stype, busmsm, sysmsm)
```

where:

string	accfile	Is the name of Contingency Solution Output file (input, no default allowed), maximum string length=260
string	colabel	Is the name of contingency label to get ACCC solution for, only one label allowed (input, no default allowed), maximum string length=12
string	stype	Is the name of solution type to get (input, *contingency* by default), allowed values: *contingency*,*con*, *tripping*, *trp*, *caction*,*contingency action*,*cor*
float	busmsm	Is the value of bus mismatch tolerance in MVA (input, 0.5 MVA by default)
float	sysmsm	Is the value of system mismatch tolerance in MVA (input, 5.0 MVA by default)

Returned object 'rlst' contains the following attributes:

integer	rlst.ierr	Is the error code =0 no error occurred =4 error opening accfile =5 error reading accfile =6 prerequisite requirements for function not met
string	rlst.codesc	Is the list of length (up to 16) of contingency events description, maximum string length=152
boolean	rlst.cnvflag	Is the convergence status (True when converged)
string	rlst.cnvcond	Is the convergence condition description, maximum string length=34
integer	rlst.island	Is the number of islands
float	rlst.mvaworst	Is the worst MVA mismatch
float	rlst.mvatotal	Is the total MVA mismatch
float	rlst.volts	Is a list of length (nmvbusrec) of monitored bus voltage in pu
float	rlst.mvaflow	Is a list of length (nmline+ninter) of monitored branch and interface MVA flow
float	rlst.ampflow	Is a list of length (nmline) of monitored branch ampere flow

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

string	rlst.lshedbus	Is a list of extended bus names of the load shed buses, maximum string length=25, list length: = ncntlshed, for post-contingency solution = ntrplshed, for post-tripping solution = ncactlshed, for post-corrective action solution
float	rlst.loadshed	Is a list of loads shed in MW, list length: = ncntlshed, for post-contingency solution = ntrplshed, for post-tripping solution = 2 columns of ncactlshed, for post-corrective action solution, 1st column gives initial load in MW, and 2nd column gives load shed in MW.
string	rlst.gdispbus	Is a list of length (ncactgdisp) of extended bus names of the generation dispatch buses, maximum string length=25, available for post-corrective action solution only.
float	rlst.gendisp	Is a list of generation dispatches in MW, available for post-corrective action solution only, list length = 2 columns of ncactgdisp, 1st column gives initial generation in MW, and 2nd column gives generation dispatch in MW.
string	rlst.phsftr	Is a list of length (ncactphsftr) of phase shifter names, maximum string length=54, available for post-corrective action solution only.
float	rlst.phsftrang	Is a list of phase shifter angles in degrees, available for post-corrective action solution only, list length = 2 columns of ncactphsftr, 1st column gives initial angle in degrees, and 2nd column gives new angle in degrees.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.3. ACCC_SUMMARY_REPORT

This function generates text report of lists returned by accc_summary.

Python syntax:

```
ierr = pssarrays.accc_summary_report(accfile, rptfile)
```

where:

string	accfile	Is the name of Contingency Solution Output file (input, no default allowed), maximum string length=260
string	rptfile	Is the name of report file (input, PSS®E Report by default), maximum string length=260

returns:

integer	rlst.ierr	Is the error code =0 no error occurred =4 error opening accfile =5 error reading accfile =6 prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.4. SENSITIVITY_FLOW_TO_MW

This API is used to calculate sensitivity factors of a branch flow to MW power at generator buses and MW power at load buses.

The documentation for this API is available by running the following commands in either PSS®E CLI or any Python Interpreter:

```
import pssarrays

  help(pssarrays.sensitivity_flow_to_mw)
```

For error codes details please refer to Section 1.1.262, SENSITIVITY_FLOWS.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.5. ACCC_SOLUTION_REPORT

This function generates text report of lists returned by accc_solution.

Python syntax:

```
ierr = pssarrays.accc_solution_report(accfile, colabels, stype, busmsm,
 sysmsm, rating, rptfile)
```

where:

string	accfile	Is the name of Contingency Solution Output file (input, no default allowed), maximum string length=260
string	colabels	Is the list of contingency label names (input, all contingencies in accfile by default), maximum string length=12
string	stype	Is the name of solution type to get (input, *contingency* by default allowed), allowed values: *contingency*,*con*, *tripping*, *trp*, *caction*,*contingency action*,*cor*
float	busmsm	Is the value of bus mismatch tolerance in MVA (input, 0.5 MVA by default)
float	sysmsm	Is the value of system mismatch tolerance in MVA (input, 5.0 MVA by default)
string	rating	Is the name of rating ('a' or 'b' or 'c') to use for calculating percentage loading (input, 'a' by default), maximum string length=1
string	rptfile	Is the name of report file (input, PSS®E Report by default), maximum string length=260

returns:

integer	rlst.ierr	Is the error code =0 no error occurred =4 error opening accfile =5 error reading accfile =6 prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.6. ACCC_VIOLATIONS_REPORT

This function generates text report of monitored flow and bus voltage violations of ACCC post-contingency, post-tripping or post-corrective action solution. For each monitored element flow and bus voltage, only the worst violation and contingency which results that violation are reported. That is each element violations is reported once and that violation is the worst violation.

Python syntax:

```
ierr = pssarrays.accc_violations_report (accfile, stype, busmsm, sysmsm,
rating, flowlimit, rptfile)
```

where:

string	accfile	Is the name of Contingency Solution Output file (input, no default allowed), maximum string length=260
string	stype	Is the name of solution type (input, *contingency* by default allowed), allowed values: *contingency*,*con*, *tripping*, *trp*, *caction*,*contingency action*,*cor*
float	busmsm	Is the value of bus mismatch tolerance in MVA (input, 0.5 MVA by default)
float	sysmsm	Is the value of system mismatch tolerance in MVA (input, 5.0 MVA by default)
string	rating	Is the name of rating ('a' or 'b' or 'c') to use for calculating percentage loading (input, 'a' by default), maximum string length=1
float	flowlimit	Is the value of percent of flow rating (input, 100% by default)
string	rptfile	Is the name of report file (input, PSS®E Report by default), maximum string length=260

returns:

integer	rlst.ierr	Is the error code =0 no error occurred =4 error opening accfile =5 error reading accfile =6 prerequisite requirements for function not met

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.7. ASCC_CURRENTS

This function returns short circuit currents and Thevenin impedance in three-phase ac systems from API. Results are returned for bus faults only. Results for line out and line end faults are not returned.

Python syntax:

```
rlst = pssarrays.ascc_currents  (sid, all, flt3ph, fltlg, fltllg, fltll,
 linout, linend, voltop, genxop, tpunty, dcload, zcorec, lnchrg, shntop,
 loadop, machpq, volts, relfil, fcdfil, scfile)
```

int	sid	Is a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default).
int	all	Is the *all buses*or selected subsystem flag (input; 1 by default) = 1 process all buses = 0 process only buses in subsystem SID
int	flt3ph	Is to report three phase faults (0 by default). = 0 omit = 1 include
int	fltlg	Is to report line-to-ground faults (0 by default). = 0 omit = 1 include
int	fltllg	Is to report line-line-to-ground faults (0 by default). = 0 omit = 1 include
int	fltll	Is to report line-to-line faults (0 by default). = 0 omit = 1 include
int	linout	Is to report line-out faults (0 by default). = 0 omit = 1 include
int	linend	Is to report line-end faults (0 by default). = 0 omit = 1 include
int	voltop	Is the bus voltage option (0 by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= 0 use bus voltages from power flow solution = 1 set all bus voltages at specified value and at 0 deg = 2 set faulted bus voltage at specified value and at 0 deg
int	genxop	Is the synchronous machine reactance option, ignored if generators are already “converted” (0 by default). = 0 use subtransient reactance = 1 use transient reactance = 2 use synchronous reactance
int	tpunty	Is the transformer tap tatios and phase angle option (0 by default) = 0 leave tap ratios and phase shift angles unchanged = 1 set tap ratios to 1.0 pu and phase shift angles to 0 deg = 2 set tap ratios to 1.0 pu and phase shift angles unchanged = 3 set tap ratios unchanged and phase shift angles to 0 deg
int	dcload	Is the dc line and FACTS device option (0 by default) = 0 blocked = 1 represent as load
int	zcorec	Is the zero sequence transformer impedance correction option (0 by default) = 0 ignore = 1 apply
int	lnchrg	Is the line charging option (0 by default) = 0 leave unchanged = 1 set to 0.0 in the positive and negative sequences = 2 set to 0.0 in all sequences
int	shntop	Is the line shunts, fixed shunts, switched shunts and transformer magnetizing admittance option (0 by default)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= 0 leave unchanged = 1 set to 0.0 in the positive and negative sequences = 2 set to 0.0 in all sequences
int	loadop	Is the load option (0 by default) = 0 leave unchanged = 1 set to 0.0 in the positive and negative sequences = 2 set to 0.0 in all sequences
int	machpq	Is the synchronous and asynchronous machines power output option (0 by default) = 0 use real and reactive power outputs from power flow solution = 1 set real and reactive power outputs to 0.0
int	volts	Is the user specified bus voltage value in pu (1.0 by default) This is used only when voltop = 1 or 2.
str	relfil	Is the Relay Output Data (.rel) file; blank for none (input; blank by default); maximum string length=260
str	fcdfil	Is the name of fault control (.fcd) file; blank for none (input; blank by default); maximum string length=260.
str	scfile	Is the ASCC_3 Results (.sc) file; blank for none (output; blank by default); maximum string length=260.

Returned object 'rlst' contains the following attributes:

Index i is the ith faulted bus.

int	rlst.ierr	Is the error code = 0, no error occurred = 1, invalid SID value or subsystem SID is not defined = 2, invalid ALL value = 3, invalid STATUS value = 4, line end fault invalid; bus table is full = 5, error opening RELFIL = 6, error opening FCDFIL

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= 7, prerequisite requirements for function not met = 8, no faults specified = 9, error opening SCFILE = 10, SCFILE error = 11, bus SC results not available. = 12, branch SC results not available. = 13, three winding transformer SC results not available. = 14, incorrect SCFILE. = 15, SCFILE not allowed.
str	rlst.scunit	Is the units of returned fault currents = pu = physical (r.m.s. amperes)
str	rlst.scfmt	Is the coordinates of returned fault currents = rectangular = polar
int	rlst.fltbus	Is a list of faulted bus numbers
complex	rlst.flt3ph[i].ia1	Is three phase fault, positive sequence current = I"k
complex	rlst.flt3ph[i].ia2	Is three phase fault, negative sequence current
complex	rlst.flt3ph[i].ia0	Is three phase fault, zero sequence current
complex	rlst.flt3ph[i].ia	Is three phase fault, Phase A current
complex	rlst.flt3ph[i].ib	Is three phase fault, Phase B current
complex	rlst.flt3ph[i].ic	Is three phase fault, Phase C current
complex	rlst.fltlg[i].ia1	Is line to ground fault, positive sequence current
complex	rlst.fltlg[i].ia2	Is line to ground fault, negative sequence current
complex	rlst.fltlg[i].ia0	Is line to ground fault, zero sequence current = I"k/3
complex	rlst.fltlg[i].ia	Is line to ground fault, Phase A current
complex	rlst.fltlg[i].ib	Is line to ground fault, Phase B current
complex	rlst.fltlg[i].ic	Is line to ground fault, Phase C current
complex	rlst.fltllg[i].ia1	Is line line to ground fault, positive sequence current
complex	rlst.fltllg[i].ia2	Is line line to ground fault, negative sequence current
complex	rlst.fltllg[i].ia0	Is line line to ground fault, zero sequence current = I"k/3
complex	rlst.fltllg[i].ia	Is line line to ground fault, Phase A current
complex	rlst.fltllg[i].ib	Is line line to ground fault, Phase B current
complex	rlst.fltllg[i].ic	Is line line to ground fault, Phase C current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

complex	rlst.fltll[i].ia1	Is line to line fault, positive sequence current
complex	rlst.fltll[i].ia2	Is line to line fault, negative sequence current
complex	rlst.fltll[i].ia0	Is line to line fault, zero sequence current
complex	rlst.fltll[i].ia	Is line to line fault, Phase A current
complex	rlst.fltll[i].ib	Is line to line fault, Phase B current = I"k
complex	rlst.fltll[i].ic	Is line to line fault, Phase C current = -I"k
complex	rlst.thevz[i].z1	Is Thevenin impedance, positive sequence, in PU or OHM as set by SCUNIT
complex	rlst.thevz[i].z2	Is Thevenin impedance, negative sequence, in PU or OHM as set by SCUNIT
complex	rlst.thevz[i].z0	Is Thevenin impedance, zero sequence, in PU or OHM as set by SCUNIT
complex	rlst.thevzpu[i].z1	Is PU Thevenin impedance, positive sequence
complex	rlst.thevzpu[i].z2	Is PU Thevenin impedance, negative sequence
complex	rlst.thevzpu[i].z0	Is PU Thevenin impedance, zero sequence
complex	rlst.maxflt[i].ia1	Is maximum fault current, positive sequence current
complex	rlst.maxflt[i].ia2	Is maximum fault current, negative sequence current
complex	rlst.maxflt[i].ia0	Is maximum fault current, zero sequence current
complex	rlst.maxflt[i].ia	Is maximum fault current, Phase A current
complex	rlst.maxflt[i].ib	Is maximum fault current, Phase B current
complex	rlst.maxflt[i].ic	Is maximum fault current, Phase C current
str	rlst.maxfltdsc[i]	Is description of fault condition for maximum fault current

Note: Maximum fault current is the largest fault current among analyzed faults at a bus.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.8. IECS_CURRENTS

This function returns short circuit currents and Thevenin impedance in three-phase ac systems according to IEC 60909 standard (IECS_3 API). Results are returned for bus faults only. Results for line out and line end faults are not returned.

Python syntax:

```
rlst = pssarrays.iecs_currents(sid, all, flt3ph, fltlg, fltllg, fltll,
 fltloc, linout, linend, tpunty, lnchrg, shntop, dcload, zcorec, optnftrc,
 loadop, genxop, brktime, vfactorc, iecfil, fcdfil, scfile)
```

where:

int	sid	Is a valid subsystem identifier. Valid subsystem identifiers range from 0 to 11. Subsystem SID must have been previously defined (input; 0 by default)
int	all	Is the *all buses* or specified subsystem flag (input; 1 by default). = 1 process all buses. = 0 process only buses in subsystem SID.
int	flt3ph	Is to report three phase faults, (0 by default). = 0 omit = 1 include
int	fltlg	Is to report line-to-ground faults (0 by default). = 0 omit = 1 include
int	fltllg	Is to report line-line-to-ground faults, (0 by default). = 0 omit = 1 include
int	fltll	Is to report line-to-line faults, (0 by default). = 0 omit = 1 include
int	fltloc	Is the location of fault (1 by default) =0 fault at network bus =1 fault at LV bus of Power Station Unit (PSU) =2 fault at auxiliary transformer (connected to PSU) LV bus
int	linout	Is to report line-out faults (0 by default).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= 0 omit = 1 include
int	linend	Is to report line-end faults (0 by default). = 0 omit = 1 include
int	tpunty	Is the transformer tap ratios and phase angle option (0 by default) = 0 leave tap ratios and phase shift angels unchanged = 1 set tap ratios to 1.0 pu and phase shift angels to 0 deg = 2 set tap ratios to 1.0 pu and phase shift angels unchanged = 3 set tap ratios unchanged and phase shift angels to 0 deg
int	lnchrg	Is the line charging option (1 by default). = 0 leave unchanged = 1 set to 0.0 in the positive and negative sequences = 2 set to 0.0 in all sequences
int	shntop	Is the line shunts, fixed shunts, switched shunts and transformer magnetizing admittance option (1 by default). = 0 leave unchanged = 1 set to 0.0 in the positive and negative sequences = 2 set to 0.0 in all sequences
int	dcload	Is the dc line and FACTS device option (0 by default) = 0 blocked = 1 represent as load
int	zcorec	Is the zero sequence transformer impedance correction option (0 by default) = 0 ignore = 1 apply
int	optnftrc	Is the option for Voltage Factor C

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= 0, Voltage Factor C for maximum fault currents (default) = 1, Voltage Factor C for minimum fault currents = 2, Voltage Factor C as specified by 'vfactorc' value
int	loadop	Is the load option (1 by default) = 0 leave unchanged = 1 set to 0.0 in the positive and negative sequences = 2 set to 0.0 in all sequences
int	genxop	Is the synchronous machine reactance option, ignored if generators are already “converted” (0 by default). = 0 use subtransient reactance = 1 use transient reactance = 2 use synchronous reactance
float	brktime	Is the breaker contact parting time in seconds (5/BaseFrequency by default)
float	vfactorc	Is the user specified Voltage Factor C This is used only when optnftrc=3.
str	iecfil	Is the name of IEC data (.iec) file; blank for none (input; blank by default); maximum string length=260
str	fcdfil	Is the name of fault control (.fcd) file; blank for none (input; blank by default); maximum string length=260
str	scfile	Is the IECS_3 Results (.sc) file; blank for none (output; blank by default); maximum string length=260.

Returned object 'rlst' contains the following attributes::

Index i is the i^{th} faulted bus.

int	rlst.ierr	Is the error code = 0 no error occurred. = 1 invalid sid value or subsystem sid is not defined. = 2 invalid all value. = 3 invalid STATUS value. = 4 invalid brktime value. = 5 no faults specified.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= 6 error opening iecfile. = 7 error opening fcdfile. = 8 prerequisite requirements for API are not met. = 9 error opening SCFILE = 10 SCFILE error = 11 bus SC results not available. = 12 branch SC results not available. = 13 three winding transformer SC results not available. = 14 incorrect SCFILE. = 15 SCFILE not allowed.
str	rlst.scunit	Is the units of returned fault currents = pu = physical (r.m.s. amperes)
str	rlst.scfmt	Is the coordinates of returned fault currents = rectangular = polar
int	rlst.fltbus	Is a list of faulted bus numbers
complex	rlst.flt3ph[i].ia1	Is three phase fault, positive sequence current = I"k
complex	rlst.flt3ph[i].ia2	Is three phase fault, negative sequence current
complex	rlst.flt3ph[i].ia0	Is three phase fault, zero sequence current
complex	rlst.flt3ph[i].ia	Is three phase fault, Phase A current
complex	rlst.flt3ph[i].ib	Is three phase fault, Phase B current
complex	rlst.flt3ph[i].ic	Is three phase fault, Phase C current
complex	rlst.flt3ph[i].ipb	Is three phase fault, peak current - Method B, ip(B)
complex	rlst.flt3ph[i].ipc	Is three phase fault, peak current - Method C, ip(C)
complex	rlst.flt3ph[i].idc	Is three phase fault, dc component of asymmetrical breaking current, idc
complex	rlst.flt3ph[i].ibsym	Is three phase fault, symmetrical breaking current (r.m.s.), ib(sym)
complex	rlst.flt3ph[i].ibuns	Is three phase fault, symmetrical breaking current (r.m.s.), ib(uns)
complex	rlst.fltlg[i].ia1	Is line to ground fault, positive sequence current
complex	rlst.fltlg[i].ia2	Is line to ground fault, negative sequence current
complex	rlst.fltlg[i].ia0	Is line to ground fault, zero sequence current = I"k/3
complex	rlst.fltlg[i].ia	Is line to ground fault, Phase A current

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

complex	rlst.fltlg[i].ib	Is line to ground fault, Phase B current
complex	rlst.fltlg[i].ic	Is line to ground fault, Phase C current
complex	rlst.fltlg[i].ipb	Is line to ground fault, peak current - Method B, ip(B)
complex	rlst.fltlg[i].ipc	Is line to ground fault, peak current - Method C, ip(C)
complex	rlst.fltlg[i].idc	Is line to ground fault dc component of asymmetrical breaking current
complex	rlst.fltlg[i].ibsym	Is line to ground fault, symmetrical breaking current (r.m.s.)
complex	rlst.fltlg[i].ibuns	Is line to ground fault, asymmetrical breaking current (r.m.s.)
complex	rlst.fltllg[i].ia1	Is line line to ground fault, positive sequence current
complex	rlst.fltllg[i].ia2	Is line line to ground fault, negative sequence current
complex	rlst.fltllg[i].ia0	Is line line to ground fault, zero sequence current = I"k / 3
complex	rlst.fltllg[i].ia	Is line line to ground fault, Phase A Current
complex	rlst.fltllg[i].ib	Is line line to ground fault, Phase B Current
complex	rlst.fltllg[i].ic	Is line line to ground fault, Phase C Current
complex	rlst.fltllg[i].ipb	Is line line to ground fault, peak current - Method B, ip(B)
complex	rlst.fltllg[i].ipc	Is line line to ground fault, peak current - Method C, ip(C)
complex	rlst.fltllg[i].idc	Is line line to ground fault dc component of asymmetrical breaking current, idc
complex	rlst.fltllg[i].ibsym	Is line line to ground fault, symmetrical breaking current (r.m.s.), ib(sym)
complex	rlst.fltllg[i].ibuns	Is line line to ground fault, asymmetrical breaking current (r.m.s.), ib(uns)
complex	rlst.fltll[i].ia1	Is line to line fault, positive sequence current
complex	rlst.fltll[i].ia2	Is line to line fault, negative sequence current
complex	rlst.fltll[i].ia0	Is line to line fault, zero sequence current
complex	rlst.fltll[i].ia	Is line to line fault, Phase A current
complex	rlst.fltll[i].ib	Is line to line fault, Phase B current = I"k
complex	rlst.fltll[i].ic	Is line to line fault, Phase C current = -I"k
complex	rlst.fltll[i].ipb	Is line to line fault, peak current - Method B, ip(B)
complex	rlst.fltll[i].ipc	Is line to line fault, peak current - Method C, ip(C)
complex	rlst.fltll[i].idc	Is line to line fault, dc component of asymmetrical breaking current, idc
complex	rlst.fltll[i].ibsym	Is line to line fault, symmetrical breaking current (r.m.s.), ib(sym)
complex	rlst.fltll[i].ibuns	Is line to line fault, asymmetrical breaking current (r.m.s.), ib(uns)
complex	rlst.thevz[i].z1	Is PU Thevenin impedance, positive sequence

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

complex	rlst.thevzpu[i].z2	Is PU Thevenin impedance, negative sequence
complex	thevzpu[i].z0	Is PU Thevenin impedance, zero sequence
complex	rlst.maxflt[i].ia1	Is maximum fault current, positive sequence current
complex	rlst.maxflt[i].ia2	Is maximum fault current, negative sequence current
complex	rlst.maxflt[i].ia0	Is maximum fault current, zero sequence current
complex	rlst.maxflt[i].ia	Is maximum fault current, Phase A current
complex	rlst.maxflt[i].ib	Is maximum fault current, Phase B current
complex	rlst.maxflt[i].ic	Is maximum fault current, Phase C current
str	rlst.maxfltdsc[i]	Is description of fault condition for maximum fault current

Note: Maximum fault current is the largest fault current among analyzed faults at a bus.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.9. OTDF_FACTORS

This function returns the OTDF factors array. The PSS®E case must be opened prior to execution of this function. Also the dfxfile provided must correspond to the PSS®E case opened.

Python syntax:

```
rlst = pssarrays.otdf_factors(dfxfile)
```

where:

string	dfxfile	Is the name of distribution factor .dfx file (input, no default allowed), maximum string length=260

Returned object 'rlst' contains the following attributes:

integer	rlst.ierr	Is the error code =0 no error occurred =1 no Distribution Factor Data File specified =2 generators are converted =3 buses in island(s) without a swing bus; use activity TREE =4 error opening file DFXFILE =5 file DFXFILE is not in the form of a PSS®E-25 or later DFAX file; run DFAX =6 file does not contain distribution factors; run DFAX with distribution factor option enabled =7 no monitored branches or interfaces defined =8 prerequisite requirements for API are not met =9 requires opened SAV file, for retrieving OTDF factors
integer	rlst.size.nmline	Is the number of monitored branches
integer	rlst.size.ninter	Is the number of monitored interfaces
integer	rlst.size.ncase	Is the number of contingencies + 1 (for base case)
string	rlst.casetitle.line1	Is the case short title line 1, maximum string length=60
string	rlst.casetitle.line2	Is the case short title line 2, maximum string length=60
string	rlst.file.sav	Is the name of saved case (.sav) file, maximum string length=260
string	rlst.file.dfx	Is the name of distribution factor data (.dfx) file, maximum string length=260

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

string	rlst.file.sub	Is the name of subsystem definition data (.sub) file, maximum string length=260
string	rlst.file.mon	Is the name of monitored element data (.mon) file, maximum string length=260
string	rlst.file.con	Is the name of contingency description data (.con) file, maximum string length=260
string	rlst.melement	Is a list of length (nmline+ninter) of monitored branch and interface names, maximum string length=54
string	rlst.colabel	Is a list of length (ncase) of contingency labels, maximum string length=12
string	rlst.codesc	Is a list of length (ncase) of contingency description, maximum string length=152
float	rlst.factor	Is a list of list OTDF factors, list of ncase elements with each element is a list of nmline+ninter values

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.10. PV_SUMMARY

This functions returns PV analysis summary.

Python syntax:

```
rlst = pssarrays.pv_summary(pvfile)
```

where:

string	pvfile	PV output filename (.pv), (input, no default allowed), maximum string length=260

Returned object 'rlst' contains the following attributes:

int	rlst.ierr	error code: =0 no error =1 file not PV type =4 requires SAV file
int	rlst.pvsize.ncase	number of contingencies + 1 (for base case)
int	rlst.pvsize.nmline	number of monitored branches
int	rlst.pvsize.ninter	number of monitored interfaces
int	rlst.pvsize.nmvbus	number of voltage monitored buses
int	rlst.pvsize.nmvrec	number of voltage monitored records
int	rlst.pvsize.nmgnbus	number of monitored plant (generator) buses
int	rlst.pvsize.nmldbus	number of monitored load buses
int	rlst.pvsize.nmxtrns	maximum number of MW transfer changes
int	rlst.options	PV solution options (same as in API manual)
float	rlst.values	PV solution values (same as in API manual)
string	rlst.brnflowunits.xfrcur	PV solution transformer branch flow units ('mbrnamp' in pv_solution return list)
string	rlst.brnflowunits.nxfrcr	PV solution non-transformer branch flow units ('mbrnamp' in pv_solution return list)
string	rlst.casetitle.line1	short title line 1
string	rlst.casetitle.line2	short title line 2
string	rlst.file.pv	contingency output (.acc) filename
string	rlst.file.sav	saved case (.sav) filename
string	rlst.file.ecd	economic dispatch data (.ecd) filename
string	rlst.file.thr	load throwover data (.thr) filename
string	rlst.file.dfx	distribution factor data (.dfx) filename
string	rlst.file.sub	subsystem definition data (.sub) filename
string	rlst.file.mon	monitored element data (.mon) filename
string	rlst.file.con	contingency description data (.con) filename
string	rlst.srcsink	source and sink subsystem names

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

string	rlst.mbranch	monitored branch names
float	rlst.mbrnrating.a	rating A
float	rlst.mbrnrating.b	rating B
float	rlst.mbrnrating.c	rating C
string	rlst.minterface	monitored interface names
float	rlst.mitfrating	selected rating of monitored interface
string	rlst.mgenbus	monitored plant (generator) bus label
string	rlst.mloadbus	monitored load bus label
string	rlst.mvbuslabel	monitored voltage bus label
string	rlst.mvreclabel	monitored voltage record label
float	rlst.mvrecmax	monitored voltage bus maximum
float	rlst.mvrecmin	monitored voltage bus minimum
string	rlst.mvrectype	monitored voltage record type (range/deviation)
string	rlst.colabel	contingency labels
string	rlst.codesc	contingency description
float	rlst.maxmw	maximum MW transfer

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.11. PV_SUMMARY_REPORT

This function creates PV analysis summary text report.

Python syntax:

```
ierr = pv_summary_report(pvfile, rptfile)
```

where:

string	pvfile	PV output filename (.pv), (input, no default allowed), maximum string length=260
string	rptfile	Is the name of report file (input, PSS®E Report by default), maximum string length=260

Returns:

int	rlst.ierr	error code: =0 no error =1 file not PV type =4 requires SAV file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.12. PV_SOLUTION

This functions returns PV analysis monitored flows and bus voltages for one contingency.

Python syntax:

```
rlst = pssarrays.pv_solution(pvfile, colabel)
```

where:

string	pvfile	PV output filename (.pv), (input, no default allowed), maximum string length=260
string	colabel	Is the name of contingency label to get PV solution for, only one label allowed (input, no default allowed), maximum string length=12

Returned object 'rlst' contains the following attributes:

int	rlst.ierr	error code: =0 no error =1 file not PV type =4 requires SAV file
int	rlst.island	number of islands
float	rlst.mwtransfer	MW transactions, list of size [ntrns]
boolean	rlst.cnvflag	convergence flag (True when converged), list of size [ntrns]
string	rlist.cnvcond	convergence condition
float	rlst.mvaworst	largest bus MVA mismatch, list of size [ntrns]
float	rlst.mvatotal	total system MVA mismatch, list of size [ntrns]
float	rlst.volts	monitored bus voltage (pu), list of size [nmvbus][ntrns]
float	rlst.mgenmw	monitored plant MW, list of size [nmgnbus][ntrns]
float	rlst.mgenmvar	monitored plant MVAR, list of size [nmgnbus][ntrns]
float	rlst.mloadmw	monitored load MW, list of size [nmldbus][ntrns]
float	rlst.mloadmvar	monitored load MVAR, list of size [nmldbus][ntrns]
float	rlst.mbrnmva	monitored branch MVA flow (MVA), list of size [nmline][ntrns]
float	rlst.mbrnamp	monitored branch flow, AMPS expressed in MVA or MVA, list of size [nmline][ntrns]. The brnflowunits variable in pv_summary determine the units. Use this value to calculate branch loadings.
float	rlst.mitfmw	monitored interface MW flow (MW), list of size [ninter][ntrns]

where

ntrans =number of MW transfer changes

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

nmvbus =number of voltage monitored buses

nmgnbus =number of monitored plant (generator) buses

nmldbus =number of monitored load buses

nmline =number of monitored branches

ninter =number of monitored interfaces

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.13. PV_SOLUTION_REPORT

This function creates PV analysis solution text report.

Python syntax:

```
ierr = pv_solution_report(pvfile, colabels, rptfile)
```

where:

string	pvfile	PV output filename (.pv), (input, no default allowed), maximum string length=260
string	colabels	Is the list of contingency label names (input, all contingencies in accfile by default), maximum string length=12
string	rptfile	Is the name of report file (input, PSS®E Report by default), maximum string length=260

Returns:

int	rlst.ierr	error code: =0 no error =1 file not PV type =4 requires SAV file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.14. QV_SOLUTION

This functions returns QV analysis monitored flows and bus voltages for one contingency.

Python syntax:

```
rlst = pssarrays.qv_solution(qvfile, colabel)
```

where:

string	qvfile	QV output filename (.qv), (input, no default allowed), maximum string length=260
string	colabel	Is the name of contingency label to get QV solution for, only one label allowed (input, no default allowed), maximum string length=12

Returned object 'rlst' contains the following attributes:

int	rlst.ierr	error code: =0 no error =2 file not QV type =5 requires SAV file
int	rlst.island	number of islands
float	rlst.vsetpoint	voltage setpoints, list of size [nvstp]
boolean	rlst.cnvflag	convergence flag (True when converged), list of size [nvstp]
string	rlst.convcond	convergence condtion
float	rlst.mvaworst	largest bus MVA mismatch, list of size [nvstp]
float	rlst.mvatotal	total system MVA mismatch, list of size [nvstp]
float	rlst.volts	monitored bus voltage (pu), list of size [nmvbus] [nvstp]
float	rlst.mgenmvar	monitored plant Mvar, list of size [nmgnbus][nvstp]

where

ntrans = number of voltage setpoint changes

nmvbus = number of voltage monitored buses

nmgnbus = number of monitored plant (generator) buses

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.15. QV_SOLUTION_REPORT

This function creates QV analysis solution text report.

Python syntax:

```
ierr = qv_solution_report(qvfile, colabels, rptfile)
```

where:

string	qvfile	QV output filename (.qv), (input, no default allowed), maximum string length=260
string	colabels	Is the list of contingency label names (input, all contingencies in accfile by default), maximum string length=12
string	rptfile	Is the name of report file (input, PSS®E Report by default), maximum string length=260

Returned list (accessed by attributes as):

int	rlst.ierr	error code: =0 no error =2 file not QV type =5 requires SAV file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.16. QV_SUMMARY

This functions returns QV analysis summary.

Python syntax:

```
rlst = pssarrays.qv_summary(qvfile)
```

where:

string	qvfile	QV output filename (.qv), (input, no default allowed), maximum string length=260

Returned object 'rlst' contains the following attributes:

int	rlst.ierr	error code: =0 no error =2 file not QV type, =5 requires SAV file
int	rlst.qvbus	QV analysis bus name
int	rlst.qvsize.ncase	number of contingencies + 1 (for base case)
int	rlst.qvsize.nmvbus	number of voltage monitored buses
int	rlst.qvsize.nmvrec	number of voltage monitored records
int	rlst.qvsize.nmgnbus	number of monitored plant (generator) buses
int	rlst.qvsize.nmxvstp	maximum number of voltage setpoint changes
int	rlst.options	QV solution options (same as in API manual)
string	rlst.values	QV solution values (same as in API manual)
string	rlst.casetitle.line1	short title line 1
string	rlst.casetitle.line2	short title line 2
string	rlst.file.qv	contingency output (.acc) filename
string	rlst.file.sav	saved case (.sav) filename
string	rlst.file.thr	load throwover data (.thr) filename
string	rlst.file.dfx	distribution factor data (.dfx) filename
string	rlst.file.sub	subsystem definition data (.sub) filename
string	rlst.file.mon	monitored element data (.mon) filename
string	rlst.file.con	contingency description data (.con) filename
string	rlst.mgenbus	monitored plant (generator) bus label
string	rlst.mvbuslabel	monitored voltage bus label
string	rlst.mvreclabel	monitored voltage record label
float	rlst.mvrecmax	monitored voltage bus maximum
float	rlst.mvrecmin	monitored voltage bus minimum
string	rlst.mvrectype	monitored voltage record type (range/deviation)
string	rlst.colabel	contingency labels
string	rlst.codesc	contingency description

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

float	rlst.minvstp	minimum voltage setpoint
float	rlst.maxvstp	maximum voltage setpoint
float	rlst.minmvar	minimum MVAR change
float	rlst.maxmvar	maximum MVAR change
float	rlst.maxmsm	maximum MVAR mismatch

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

25.17. QV_SUMMARY_REPORT

This function creates QV analysis summary text report.

Python syntax:

```
ierr = qv_summary_report(qvfile, rptfile)
```

where:

string	qvfile	QV output filename (.qv), (input, no default allowed), maximum string length=260
string	rptfile	Is the name of report file (input, PSS®E Report by default), maximum string length=260

Returned list (accessed by attributes as):

int	rlst.ierr	error code: =0 no error =2 file not QV type =5 requires SAV file

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 26
PSSEXCEL Module

The pssexcel module provides Python functions to export PSS®E data or solution results to Excel spreadsheets.

These functions do not record themselves when called while recording is enabled in PSS®E.

Please note that these routines are members of the pssexcel Python extension module, not psspy. The syntax on the following pages assumes that the module has been imported by executing:

```
import pssexcel
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

26.1. ACCC

This function exports ACCC analysis results to Excel Spreadsheet.

Python syntax:

```
pssexcel.accc(accfile, string, colabel='',
stype='contingency', busmsm=0.5, sysmsm=5.0, rating='a', namesplit=True,
xlsfile='', sheet='', overwritesheet=True, show=True, ratecon='b',
 baseflowvio=True,
basevoltvio=True, flowlimit=100.0, flowchange=0.0, voltchange=0.0,
 swdrating='a',
swdratecon='b',baseswdflowvio=False,basenodevoltvio=False,
 overloadreport=False)
```

where:

string	accfile	ACCC analysis output filename (.acc), no default allowed
string	string	Name or list of names indicating which results to export, no default allowed. Allowed strings: 's' or 'summary' ACCC Analysis Summary 'e' or 'events' Contingency Events Description 'b' or 'branch' Monitored Branch Flow (MVA) 'i' or 'interface' Monitored Interface Flow (MW) 'v' or 'voltage' Monitored Bus Voltage 'l' or 'load' Loads Shed (MW) 'g' or 'generator' Generator Dispatch (MW)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		'p' or 'phase shifter' Phase Shifter Angle 'n' or 'node voltage' Monitored Node Voltage 'w' or 'swd flow' Monitored SWD Flow (MVA) Example: string = 'v' or ['s','v','g','l','b','i','n','w']
string	colabel	Contingency label or list of contingency labels for which the solution is exported, default - all contingencies, not required when exporting only summary
string	stype	Is the name of solution type to get (input, contingency by default), allowed values: contingency, con, tripping, trp, caction, contingency action, cor
float	busmsm	Is the value of bus mismatch tolerance in MVA (input, 0.5 MVA by default)
float	sysmsm	Is the value of system mismatch tolerance in MVA (input, 5.0 MVA by default)
string	rating	s the value of rating to use to calculate base case percentage overload, default 'a', allowed values:'a', 'b', or 'c'
string	stype	Is the name of solution type to get (input, *contingency* by default allowed), allowed values: *contingency*, *con*, *tripping*, *trp*, *caction*, *contingency action*, *cor*
float	busmsm	Is the value of bus mismatch tolerance in MVA (input, 0.5 MVA by default)
float	sysmsm	Is the value of system mismatch tolerance in MVA (input, 5.0 MVA by default)
string	rating	Rating to use to calculate percentage overload. Default 'a', allowed values: 'a', 'b', 'c', ..., 'l'.
boolean	namesplit	Split extended bus names, default True = True bus names split into three values: number name and bus voltage (three columns in worksheet) = False bus names are kept as single value string (one column in worksheet)
string	xlsfile	Excel workbook name, default Book#.xls

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

string	sheet	Excel worksheet name, default (blank string), depending on string input, worksheets created are:
		
		`Summary`
		
		`ACCC`
		`Analysis Summary`
		
		`Contingency Events`
		
		`Events Description`
		
		`Branch Flow`
		
		`Monitored`
		`Branch Flow`
		
		`(MVA)`
		
		`Interface Flow`
		
		`Monitored`
		`Interface Flow`
		
		`(MW)`
		
		`Bus Voltage`
		
		`Monitored`
		`Bus Voltage`
		
		`Load Shed`
		
		`Loads`
		`Shed (MW)`
		
		`Generator Dispatch`
		
		`Generator Dispatch (MW)`
		
		`Phase Shifter Angle`
		
		`Monitored Node Voltage`
		
		`Monitored SWD flow (MVA)`
		
		`(degrees)`
		
		All worksheet names are prefixed with sheet name, when provided.

boolean	overwritesheet	Overwrite worksheets flag, default True = True existing worksheets are overwritten

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= False existing worksheets are copied and their names appended with (#) where # is next sequence number.
boolean	show	Show or Hide Excel Spreadsheet flag, default True = True open and show Excel Spreadsheet = False do not show Excel Spreadsheet (create and save)
string	ratecon	Is the value of rating to use to calculate Contingency Case percentage overload default 'b', allowed values:'a', 'b', 'c', ..., 'l'.
boolean	baseflowvio	Exclude elements with base case loading violations from contingency reports default True, allowed values: True or False
boolean	basevoltvio	Exclude buses with base case voltage range violations from contingency reports default True, allowed values: True or False
float	flowlimit	Percent of flow rating, default=100 for 100% Show monitored elements with flow above flowlimit value. flowlimit=0 will export flows for all elements.
float	flowchange	Minimum contingency case flow change in MVA for overload reports default 0 MVA, allowed values: Any number
float	voltchange	Minimum contingency case voltage change in pu for voltage range violations default 0 pu, allowed values: Any number
string	swdrating	Is the value of rating to use to calculate Base Case SWD percentage overload default 'a', allowed values:'a', 'b', or 'c'.
string	swdratecon	Is the value of rating to use to calculate Contingency Case SWD percentage overload default 'b', allowed values:'a', 'b', or 'c'.
boolean	baseswdflowvio	Exclude SWD flow results with base case loading violations from contingency reports default True, allowed values: True or False
boolean	basenodevoltvio	Exclude node voltage results with base case voltage range violations from contingency reports default True, allowed values: True or False
boolean	overloadreport	Is the Flag to only report overload power flow and overvoltage bus voltage (True) or report all the results (False). percentage overload default False.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

26.2. iec_data_file

This function exports synchronous machines, asynchronous machines and transformer data from PSS®E Saved Case required to assemble IEC fault data input file as an Excel® spreadsheet. IEC data file (.iec) is used by IECS API. The PSS ®E Saved case must be opened prior to execution of this function.

It creates three worksheets 'Machines', 'Transformers' and 'IEC Data'.

This file is used as a convenient way to create an .iec file by copy/paste generator and transformer records to form a power station unit, modify the generator, motor and transformer data as required by IECS API. After appropriate modifications, save IEC Data worksheet as comma separated file with extension .iec. This file then can be used as *Fault data input file* to the IECS API.

Python syntax:

```
pssexcel.iec_data_file(xlsfile, sheet, overwritesheet,
show)
```

where:

string	xlsfile	Is the name of excel workbook filename (input, default savefile prefix and extension .iec)

string	sheet	Excel worksheet name, default (blank string), worksheets created are:
		
		`Machines`
		
		`Saved case machines data`
		
		`Transformers`
		
		`Saved case two and three windings transformers data`
		
		`IEC Data`
		
		`Data required by IECS API`
		
		All worksheet names are prefixed with sheet name, when provided.

boolean	overwritesheet	Overwrite worksheets flag, default True = True existing worksheets are overwritten = False existing worksheets are copied and their names appended with (#), where # is next sequence number.
boolean	show	Show or Hide Excel Spreadsheet flag, default True = True open and show Excel Spreadsheet

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

= False do not show Excel Spreadsheet (create and save)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

26.3. pv

This function exports PV solution results to Excel Spreadsheet.

Python syntax:

```
pssexcel.pv(pvfile, string, colabel, namesplit,
xlsfile, sheet, overwritesheet, show)
```

where:

string	pvfile	PV analysis output filename (.pv), no default allowed
string	string	Name or list of names indicating which results to export, no default allowed. Allowed strings:

```
's' or 'summary'

PV
Solution Summary

'v' or 'voltage'

Monitored
Bus Voltage

'm' or 'mismatch'

Largest and Total Mismatch

'g' or 'generator'

Monitored Plants MW and MVAR

'l' or 'load'

Monitored
Loads MW and MVAR

'b' or 'branch'

Monitored
Branch Flow (MVA)

'i' or 'interface'

Monitored Interface Flow (MW)
```

Example:

```
string = 'v' or

string =
 ['s','m','v','g','l','b','i']
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

string	colabel	Contingency label or list of contingency labels for which the solution is exported, default - all contingencies, not required when exporting only summary
boolean	namesplit	Split extended bus names, default True = True bus names split into three values: number name and bus voltage (three columns in worksheet) = False bus names are kept as single value string (one column in worksheet)
string	xlsfile	Excel workbook name, default Book#.xls

string	sheet	Excel worksheet name, default '' (blank string), depending on string input, worksheets created are:
		
		`Summary`
		
		`PV`
		`Solution Summary`
		
		`Bus Voltage`
		
		`Monitored`
		`Bus Voltage`
		
		`Mismatch`
		
		`Largest`
		`and Total Mismatch`
		
		`Generator Dispatch`
		
		`Monitored Plants MW and MVAR`
		
		`Bus Load`
		
		`Monitored`
		`Loads MW and MVAR`
		
		`Branch Flow`
		
		`Monitored`
		`Branch Flow (MVA)`
		
		`Interface Flow`
		
		`Monitored`
		`Interface Flow (MW)`
		
		All worksheet names are prefixed with sheet name, when provided.

boolean	overwritesheet	Overwrite worksheets flag, default True = True existing worksheets are overwritten

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		= False existing worksheets are copied and their names appended with (#) where # is next sequence number.
boolean	show	Show or Hide Excel Spreadsheet flag, default True = True open and show Excel Spreadsheet = False do not show Excel Spreadsheet (create and save)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

26.4. qv

This function exports QV solution results to Excel Spreadsheet.

Python syntax:

```
pssexcel.qv(qvfile, string, colabel, namesplit,
xlsfile, sheet, overwritesheet, show)
```

where:

string	qvfile	QV analysis output filename (.qv), no default allowed

string	string	Name or list of names indicating which results to export, no default allowed. Allowed strings:
		`'s' or 'summary'`
		`QV Solution Summary`
		`'v' or 'voltage'`
		`Monitored Bus Voltage`
		`'m' or 'mismatch'`
		`Largest and Total Mismatch`
		`'g' or 'generator'`
		`Monitored Plants MW and MVAR`
		Example:
		`string = 'v' or`
		`string = ['s','m','v','g']`

string	colabel	Contingency label or list of contingency labels for which the solution is exported, default - all contingencies, not required when exporting only summary
boolean	namesplit	Split extended bus names, default True = True bus names split into three values: number name and bus voltage (three columns in worksheet) = False bus names are kept as single value string (one column in worksheet)
string	xlsfile	Excel workbook name, default Book#.xls
string	sheet	Excel worksheet name, default '' (blank string), depending on string input, worksheets created are:

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		Summary QV Solution Summary Bus Voltage Monitored Bus Voltage Mismatch Largest and Total Mismatch Generator Dispatch Monitored Plants MVAR All worksheet names are prefixed with sheet name, when provided.
boolean	overwritesheet	Overwrite worksheets flag, default True = True existing worksheets are overwritten = False existing worksheets are copied and their names appended with (#) where # is next sequence number.
boolean	show	Show or Hide Excel Spreadsheet flag, default True = True open and show Excel Spreadsheet = False do not show Excel Spreadsheet (create and save)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 27
EXCELPY Module

The excelpy module provides Python functions to interface with Excel. Use this module to create, populate and format Excel workbooks from Python.

These functions do not record themselves when called while recording is enabled in PSS®E.

Please note that these routines are members of the excelpy Python extension module, not psspy. The syntax on the following pages assumes that the module has been imported by executing:

```
import excelpy
```

Following are the definition of arguments used by various functions defined in this module.

string or int	address	tuple of integers/strings. Allowed address when specifying cell: 'adrstr' or ('adrstr') or (row,'clnstr') or (row, column) Example: Cell address referencing 2nd row, column A, can be provided as- 'a2' or ('a2') or (2,'a') or (2,1) range: ('adrstr1','adrstr2') or (toprow, 'leftcolumnstr', bottomrow, 'rightcolumnstr') or (toprow, leftcolumn, bottomrow, rightcolumn) or ('adrstr1', (bottomrow, rightcolumn)) or ('adrstr1', (bottomrow, 'rightcolumnstr')) ((toprow, leftcolumn), 'adrstr2') or ((toprow, 'leftcolumnstr'), 'adrstr2')

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		where row, column, toprow, leftcolumn, bottomrow, rightcolumn are integer values, adrstr, adrstr1, adrstr2 are cell address string values, and clnstr, leftcolumnstr, rightcolumnstr are column name string values. Note: Column and address refernce string values are case-insensitive.
string	alignv	alignment value for cell, range, rows or columns. Allowed values when specifying horizontal alignment: 'left','right','h_center' vertical alignment: 'top','bottom','v_center'
string	borderv	border name. Allowed border names when formatting: cell: 'diagonaldown', 'diagnalup', 'top', 'bottom', 'left', 'right', 'none', 'outline' range: 'diagonaldown', 'diagonalup', 'top', 'bottom', 'left', 'right', 'inside', 'hinside', 'vinside', 'none', 'outline'
string or int	borderColor	string, color name; or integer, color index. Allowed color names: 'black', 'white', 'red', 'green', 'blue', 'yellow', 'magenta', 'cyan', 'brown', 'dgreen', 'dblue', 'orange'

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		Allowed ColorIndex: `1 to 56 as per ColorIndex property in Excel Visual Basic Reference.`
string	borderStyle	border style name. Allowed names: `'continuous', 'dash','dashdot', 'dashdotdot', 'dot', 'double', 'none', 'slantdashdot'`
string	borderWeight	border weight name. Allowed names: `'hairline', 'medium', 'thick', 'thin'`

string	fontStyle	font style name, specified as style name or tuple of style names.
		Allowed font style names:
		`'regular', 'bold', 'italic', 'underline'`
		Example:
		`(Italic,Bold) or`
		`'bold'`

string or int	fontColor	string, color name; or integer, color index. This is same as color in 'borderColor'.
string	numberFormat	number format Example: 0.000 for three decimal places
int	leftCol	left column number
string	newSheet	name of the new worksheet
string	oldSheet	name of the worksheet used as reference for worksheet operations
boolean	overwritesheet	overwrite if worksheet exists, Allowed values: = True existing worksheet will be overwritten. = False existing sheet copied and its name appended with (#), where # is next sequence number.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

		Used when creating new worksheets or re-naming existing worksheets.
string	sheetspreadsheet	name of the excel worksheet. If sheet is not provided, active worksheet used.
int	topRow	top row number
boolean	wrapText	wrap text or not, Allowed values: True or False
string	xlsfile	name of the Excel workbook

Default Workbook Style

The excel workbook is created with following default font style:

```
Black, Regular, Courier New, size 10, no wrapText,
General number format
```

How to Use?

Create xls file object as below and apply various methods defined here.

(1) When used to create new Excel files or add worksheets to existing Excel files:

testxls = excelpy.workbook()

testxls = excelpy.workbook(r"c:\working dir\ex1.xls")

testxls = excelpy.workbook(r"c:\working dir\ex1.xls", "MySheet") or

testxls = excelpy.workbook(r"c:\working dir\ex1.xls", "MySheet", False) or

testxls = excelpy.workbook(r"c:\working dir\ex1.xls", "MySheet", False, 'w') or

testxls = excelpy.workbook(xlsfile=r"c:\working dir\ex1.xls", sheet="MySheet",

overwritesheet=False, mode='w')

(2) When used to read existing Excel files:

testxls = excelpy.workbook(r"c:\working dir\ex1.xls", mode='r')

Note: When using Excel 2007 and later, create file in either 'xlsx' or 'xls' format.

Excel 2003 and earlier - default save format 'xls'.

Excel 2007 and later - default save format 'xlsx'.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

27.1. Methods

The following methods are members of the excelpy class that can be instantiated using the workbook functions defined in Section 27.2, "Workbook Functions". In all these methods, when sheet=None or not provided, active worksheet is used.

align(address, alignv, sheet=None)

Align Cell or Range.

cell, address = (row,col)

range, address = (topRow,leftCol,bottomRow,rightCol)

align_columns(address, alignv, sheet=None)

Align one or more columns.

One column, address = (row,col)

Multiple columns, address = (topRow,leftCol,topRow,rightCol)

align_rows(address, alignv, sheet=None)

Align one or more rows.

One row, address = (row,col)

Multiple rows, address = (topRow,leftCol,bottomRow,leftCol)

autofit_columns(address, sheet=None)

Autofit column or columns.

One column, address = (row,col)

Multiple columns, address = (topRow,leftCol,topRow,rightCol)

autofit_rows(address, sheet=None)

Autofit row or rows.

One row, address = (row,col)

Multiple rows, address = (topRow,leftCol,bottomRow,leftCol)

border(address, borderv, borderStyle='continuous', borderWeight='hairline', borderColor='black', sheet=None)

Format Cell or Range borders.

cell, address = (row,col)

range, address = (topRow,leftCol,bottomRow,rightCol)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

close()

Close active Excel workbook and quit Excel application.

delete_columns(address, howmany=1, sheet=None)

Delete columns to the right of specified row.

row address = (row,col)

Number of columns deleted is provided by integer value 'howmany'.

delete_rows(address, howmany=1, sheet=None)

Delete rows below specified row.

row address = (row,col)

Number of rows deleted is provided by integer value 'howmany'.

fontaddress, fontStyle=None, fontName=None, fontSize=None, fontColor=None, wrapText=False, numberFormat=None, sheet=None)

Set cell or range font properties: style, name, size, color, wrap, number format

Cell, address = (row,col)

Range, address = (topRow,leftCol,bottomRow,rightCol)

font_color(address, color, sheet=None)

Set cell or range font color.

Cell, address = (row,col)

Range, address = (topRow,leftCol,bottomRow,rightCol)

font_sheet(fontStyle=('Regular',), fontName='Courier

New', fontSize=10, fontColor='black', wrapText=False, numberFormat=None, sheet=None)

Set font properties for Excel sheet: style, name, size, color, wrap, number format

freezepanes(address, sheet=None)

Freeze worksheet panes.

address = (row,1) to freeze worksheet rows above 'row'

address = (1, col) to freeze worksheet columns left of 'col'

address = (row,col to freeze worksheet rows above 'row' and columns

left of 'col'

height(address, height, sheet=None)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Set height of one row or multiple rows. Height is provided in points.

One row, address = (row,col)

Multiple rows, address = (topRow,leftCol,bottomRow,rightCol)

hide()

Hide opened Excel workbook.

merge(address, sheet=None)

Merge cells in the range.

page_footer(left=None, center=None, right=None, sheet=None)

Set page left, center, and right footers. Footer input value is a string.

page_format(orientation='portrait', left=1.0, right=1.0, top=1.0, bottom=1.0, header=0.5, footer=0.5, sheet=None)

Set page format: orientation, margins (given in inches).

```
   Orientation = 'portrait'  or 'p'  or 1
               = 'landscape' or 'l'  or 2
     Defaults:
     orientation = Portrait
     margins: left   = 1.0 inch
              right  = 1.0 inch
              top    = 1.0 inch
              bottom = 1.0 inch
              header = 0.5 inch
              footer = 0.5 inch
```

page_header(left=None, center=None, right=None, sheet=None)

Set page left, center, and right headers. Header input value is a string.

page_landscape(sheet=None)

Set page orientation to landscape.

page_margin(left=1.0, right=1.0, top=1.0, bottom=1.0, header=0.5, footer=0.5, sheet=None)

Set page margins. The margin value is given in inches.

page_portrait(sheet=None)

Set page orientation to portrait.

save(xlsfile=None)

Save active Excel workbook.

xlsfile = save(xlsfile)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Returns name of the saved Excel file.

set_active_sheet(sheet)

Set 'sheet' as active worksheet.

set_cell(address, value, fontStyle=None, fontName=None, fontSize=None, fontColor=None, wrapText=False, numberFormat=None, sheet=None)

Set value to one cell.

set_range(topRow, leftCol, data, transpose=False, fontStyle=None, fontName=None, fontSize=None, fontColor=None, wrapText=False, numberFormat=None, sheet=None)

Set data to cells in the range.

bottomRow, rightCol = set_range(topRow,leftCol, data, ...)

Boolean, transpose: Allowed values: True or False

Excel methods write data row by row. If data in Python list of lists, or tuple of tuples or any combination of these, is not stored row by row, transpose that data. This functions works same as matrix transpose.

Returns:

bottomRow: Bottom row number after writing input data to worksheet.

rightCol : Right column number after writing input data to worksheet.

show( )

Show opened Excel workbook.

show_alerts(display)

Show or suppresses all POP-UP windows, like File Overwrite Yes/No/Cancel, Merge Overwrite.

Boolean, display: Allowed values: True or False, 1 or 0

= True or 1, show alerts

= False or 0, do not show alerts. Data will be overwritten.

transpose_data(data, transpose=True)

Transpose data.

Excel methods write data row by row. If data in Python list of lists, or tuple of tuples or any combination of these, is not stored row by row, transpose that data. This functions works same as matrix transpose.

nrows, nclns, datat = set_range(data, transpose=True)

data : any type of data to be tranposed

transpose: boolean, allowed values: True or False, True by default

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Returns:

nrows - number of rows in transposed data

nclns - number of columns in transposed data

datat - transposed data

width(address, width, sheet=None)

Set width of one column or multiple columns. Width is provided in points.

One column, address = (row,col)

Multiple columns, address = (topRow,leftCol,topRow,rightCol)

worksheet_add_after(newSheet=None, oldSheet=None, overwritesheet=True)spreadsheet

Add a newSheet after the oldSheet or active sheet. If newSheet name is not provided, Sheet# worksheet is added. If oldSheet name is not provided, worksheet is added at the end. Sheet# is a next available sheet number in the active workbook.

worksheet_add_before(newSheet=None, oldSheet=None, overwritesheet=True)

Add a newSheet before the oldSheet or active sheet. If newSheet name is not provided, Sheet# worksheet is added. If oldSheet name is not provided, worksheet is added at the end. Sheet# is a next available sheet number in the active workbook.spreadsheet

worksheet_add_begin(sheet=None, overwritesheet=True)

Add a sheet at the beginning. If sheet name is not provided, Sheet# worksheet is added. Sheet# is a next available sheet number in the active workbook.spreadsheet

worksheet_add_end(sheet=None, overwritesheet=True)spreadsheet

Add a sheet at the end. If sheet name is not provided, Sheet# worksheet is added. Sheet# is a next available sheet number in the active workbook.

worksheet_delete(sheet)

Delete a sheet from active workbook.

worksheet_left_columns2repeat(columns, sheet=None)spreadsheet

Repeat 'columns' at the left of each page of the worksheet.

'columns' is specified as tuple or list of column numbers,

e.g., columns = (1,3) to repeat columns 1 to 3 at the left or

columns = 1 to repeat column 1 at the left.

worksheet_rename(newSheet, oldSheet=None, overwritesheet=True)spreadsheet

Rename Excel 'oldSheet' or active sheet with newSheet name.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

overwritesheet = True, contents of oldSheet are deleted, so newSheet is blank worksheet

= False, contents of oldSheet are written to newSheet.\

worksheet_size_violation(topRow, leftCol, data=None, transpose=False)

Get worksheet size violations and extents.

Inputs:

data - List or List of Lists values to be written transpose - Logical

Excel methods write data row by row. If data in Python list of lists, or tuple of tuples or any combination of these, is not stored that way, transpose that data. This functions same as matrix transpose.

Returns:

Returned tuple contains:

row_violation = rtple[0], True when top/bottom row exceeds maximum allowable rows

col_violation = rtple[2], True when left/right column exceeds maximum allowable columns

topRow = rtple[3], top row number

leftCol = rtple[4], left column number

bottomRow = rtple[5], bottom row number

rightCol = rtple[6], right column number

wshtmaxrows = rtple[7], = 65536, maximum number of rows allowed in a worksheet

wshtmaxcols = rtple[8], = 256, maximum number of columns allowed in a worksheet

worksheet_top_rows2repeat(rows, sheet=None)spreadsheet

Repeat 'rows' at the top of each page of the worksheet.

'rows' is specified as tuple or list of row numbers,

e.g., rows = (1,3) to repeat rows 1 to 3 at the top or

rows = 1 to repeat row 1 at the top.

wraptext(address, sheet=None)

Wrap Text inside the cell or range.

Cell, address = (row,col)

Range, address = (topRow,leftCol,topRow,rightCol)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

27.2. Workbook Functions

Use this function to create an Excel workbook object that will possess the methods defined in Section 27.1, “Methods”.

Python syntax:

```
excelpy.workbook(xlsfile=None, sheet=None,
overwritesheet=True)
```

Examples:

```
testxls = excelpy.workbook()

testxls = excelpy.workbook("c:\working dir\test1.xls")

testxls = excelpy.workbook("c:\working dir\test1.xls",
"MySheet")

testxls = excelpy.workbook("c:\working dir\test1.xls",
"MySheet",
                           False)

testxls = excelpy.workbook(xlsfile="c:\working
dir\test1.xls",
                           sheet="MySheet", overwritesheet=False)
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 28
GICDATA Module

The gicdata module provides Python functions to create the GIC data file required in GIC analysis.

These functions do not record themselves when called while recording is enabled in PSS®E.

Please note that these routines are members of the gicdata Python extension module, not psspy. The syntax on the following pages assumes that the module has been imported by executing:

```
import gicdata
```

GIC (Geomagnetically Induced Current) Analysis requires an additional GIC data file (See Section 7.2, GIC Data File Contents in the *Program Operation Manual*) that contains:

- Substation data
- Bus Substation data and
- Transformer data

Functions in the gicdata module help to create GIC data file applicable to a specific Power Flow Network Case for an entire network or just desired study subsystem.

Creating GIC data using the gicdata module is a two step process.

Step 1

Create GIC data file template using functions:

- template_excel(..)
- template_py(..)

This will create structure of GIC data that is required.

Step 2

Edit/Update the GIC data file template created in Step (1) to provide:

- Substation data
- Bus Substation data
- Transformer data

For transformers, vector group information must be provided. If power flow Saved Case has vector group information, GIC data file template will have this information as well. If PSSE Saved Case does not have vector group information, this needs to be provided. The GIC data file then can default values for all other transformer record data items.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Once GIC data file template is updated, create GIC data file using functions:

- excel2gicfile(..)
- py2gicfile(..)

This GIC data file can be used in GIC Analysis.

Argument Definitions:

string savfile- PSSE Saved Case file name

string pyfile- GIC data file formatted in Python structures

This module defines Python structures and saves in this file (Step 1 above).

User only edits/updates values.

string excelfile- GIC data file formatted in Excel worksheets

This module creates this Excel file and worksheets in it (Step 1 above).

User only edits/updates values.

string gicfile- GIC data file name (output) that can be used in GIC Analysis.

The following optional arguments are used to define subsystem, if GIC study is to be performed for a sub-system.

real basekv- List of two real values: minimum voltage and maximum voltage,

For example [minVolt, maxVolt], default = []

integer areas- List of areas, default = []

integer buses- List of buses, default = []

integer owners- List of owners, default = []

integer zones- List of zones, default = []

integer tielevels- Number of intertie levels, default = 0

Functions:

check_gicfile(gicfile0, gicfile='')

Check GIC data file 'gicfile0' and create new GIC data file 'gicfile'.

Input gicfile0 is read and processed to do following:

- Add quotes (if already not there) around character data items likes "SUBNAM"
- Remove tabs and such non-ASCII characters

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

- Check if same geographical location is provided for different substations and if found remove that substation from 'substation' record groups and replace that substation in 'bus subsation' record group.

Output 'gicfile' created only if it is specified.

excel2gicfile(excelfile, gicfile)

Create GIC data file (.gic) from GIC data template in Excel.

py2gicfile(pyfile, gicfile)

Create GIC data file (.gic) from GIC data template in Python structures.

template_excel(savfile, excelfile, basekv=[], areas=[], buses=[], owners=[], zones=[], tielevels=0)

Create GIC data file template in Excel Spreadsheet.

template_py(savfile, pyfile, basekv=[], areas=[], buses=[], owners=[], zones=[], tielevels=0)

Create GIC data file template in Python structures.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 29
PFCONTROL Module

This chapter describes Python functions available in the PSS®E Remedial Action Scheme (RAS) and Advanced Contingency module.

These functions allow the user to program advanced RAS scenarios and events not supported by the existing SUB/MON/CON and TRP files to be executed in conjunction with the Power Flow Customization Interface (PCI) embedded in the PSS®E engine.

These functions do not record themselves when called while recording is enabled in PSS®E.

Please note that these routines are members of the pfcontrol Python extension module, not psspy. The syntax on the following pages assumes that the module has been imported by executing:

```
import pfcontrol
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.1. PFDSCNBUS

This API is used to disconnect a bus in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr, typcode = pfdscnbus(iocode, frombus)
```

Python syntax:

```
ierr = pfdscnbus(frombus)
```

where:

Integer IOCODE	Is the action code (input). The values are as follows. 0 by default.
	IOCODE = 0 disconnect the bus
	IOCODE = 1 return the type code in typcode
Integer FROMBUS	Is the bus number (input).
Integer TYPCODE	Is the bus type code (output).
Integer IERR	Is the error code (output).
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 7 bus not found.
	IERR = 21 bus is already disconnected (type code > 3).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.2. PFCNCTBUS

This API is used to connect a bus in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.10.0 and 34.2.0

Python syntax:

```
ierr = pfcnctbus(frombus,typcode)
```

where:

Integer FROMBUS	Is the bus number (input).
Integer TYPCODE	Is the bus type code (input). The values are as follows. 0 by default.
	TYPCODE = 1 for an ordinary in-service bus.
	TYPCODE = 2 for a generator bus.
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 7 bus not found.
	IERR = 22 bus is already connected (type code < 4).
	IERR = 23 bus is an interior bus of a multi-section line.
	IERR = 24 new type code is not 1 or 2.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.3. PFBRCHINT

This API is used to get/set specific integer quantities on a branch in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pfbrchint(frombus, tobus, id, namval, ival)
ierr, ival = pfbrchint(frombus, tobus, id, namval)
```

where:

Integer FROMBUS	Is the branch from bus number (input).
Integer TOBUS	Is the branch to bus number (input).
Character ID*2	Is the branch Id (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	STATUS the branch status
Integer IVAL	Is the integer value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 20 branch not found.
	IERR = 25 at least one of the endpoint buses is not energized.
	IERR = 26 multi-section line reporting option setting is disabled.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.4. PFBRCHREAL

This API is used to get/set specific real quantities on a branch in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 35.6.4

Python syntax:

```
ierr = pfbrchreal(frombus, tobus, id, namval, rval)

ierr, rval = pfbrchreal(frombus, tobus, id, namval)
```

where:

Integer FROMBUS	Is the branch from bus number (input).
Integer TOBUS	Is the branch to bus number (input).
Character ID*2	Is the branch Id (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	R the branch resistance; X the branch reactance
Real RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 20 branch not found.
	IERR = 25 at least one of the endpoint buses is not energized.
	IERR = 26 multi-section line reporting option setting is disabled.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.5. PFSUBSWDINT

This API is used to get/set specific integer quantities on a substation switching device in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 34.2.0

Python syntax:

```
ierr = pfsubswdint(statnum, fromnode, tonode, id, namval, ival)

ierr, ival = pfsubswdint(statnum, fromnode, tonode, id, namval)
```

where:

Integer STATNUM	Is the substation switching device substation number (input).
Integer FROMNODE	Is the substation switching device from node number (input).
Integer TONODE	Is the substation switching device to node number (input).
Character ID*2	Is the substation switching device Id (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	STATUS the substation switching device status
Integer IVAL	Is the integer value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 27 substation STANUM not found.
	IERR = 28 node FRNODE not found in substation STANUM.
	IERR = 29 node TONODE not found in substation STANUM.
	IERR = 30 switching device not found in substation STANUM.
	IERR = 31 at least one of the nodes is out-of-service.
	IERR = 32 attempting to open a stuck breaker.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.6. PFFXSINT

This API is used to get/set specific integer quantities on a fixed shunt in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pffxsint(frombus, id, namval, ival)

ierr, ival = pffxsint(frombus, id, namval)
```

where:

Integer FROMBUS	Is the fixed shunt from bus number (input).
Character ID*2	Is the fixed shunt Id (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	STATUS the fixed shunt status
Integer IVAL	Is the integer value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 16 fixed shunt not found.
	IERR = 17 fixed shunt is out-of-service.
	IERR = 39 fatal error from SHUNT_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.7. PFFXSREAL

This API is used to get/set specific real quantities on a fixed shunt in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pffxsreal(frombus, id, namval, rval)

ierr, rval = pffxsreal(frombus, id, namval)
```

where:

Integer FROMBUS	Is the fixed shunt from bus number (input).
Character ID*2	Is the fixed shunt Id (input).
Character NAMVAL*8	Is the value to set or return (input). The values are as follows.
	PNOMINAL the nominal active power in MW
	QNOMINAL the nominal reactive power in MVAR
Integer RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 16 fixed shunt not found.
	IERR = 17 fixed shunt is out-of-service.
	IERR = 39 fatal error from SHUNT_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.8. PFSWSINT

This API is obsolete. It has been replaced by the API routine PFSWSINT_2.

This API is used to get/set specific integer quantities on a switched shunt in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pfswsint(frombus, namval, ival)

ierr, ival = pfswsint(frombus, namval)
```

where:

Integer FROMBUS	Is the switched shunt from bus number (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	STATUS the switched shunt status
Integer IVAL	Is the integer value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 18 switched shunt not found.
	IERR = 19 switched shunt is out-of-service.
	IERR = 40 fatal error from SWITCHED_SHUNT_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.9. PFSWSREAL

This API is obsolete. It has been replaced by the API routine PFSWSREAL_2.

This API is used to get/set specific real quantities on a switched shunt in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pfswsreal(frombus, namval, rval)

ierr, rval = pfswsreal(frombus, namval)
```

where:

Integer FROMBUS	Is the switched shunt from bus number (input).
Character NAMVAL*5	Is the value to set or return (input). The values are as follows.
	QINIT the present switched shunt admittance in MVAR
	VMAX the desired voltage upper limit
	VMIN the desired voltage lower limit
Real RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 18 switched shunt not found.
	IERR = 19 switched shunt is out-of-service.
	IERR = 40 fatal error from SWITCHED_SHUNT_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.10. PFSWSINT_2

This API is used to get/set specific integer quantities on a switched shunt in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 35.3.0

Python syntax:

```
ierr = pfswsint_2(frombus, id, namval, ival)

ierr, ival = pfswsint_2(frombus, id, namval)
```

where:

Integer FROMBUS	Is the switched shunt from bus number (input).
Character ID*2	Is the fixed shunt Id (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	STATUS the switched shunt status
Integer IVAL	Is the integer value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 18 switched shunt not found.
	IERR = 19 switched shunt is out-of-service.
	IERR = 40 fatal error from SWITCHED_SHUNT_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.11. PFSWSREAL_2

This API is used to get/set specific real quantities on a switched shunt in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 35.3.0

Python syntax:

```
ierr = pfswsreal_2(frombus, id, namval, rval)

ierr, rval = pfswsreal_2(frombus, id, namval)
```

where:

Integer FROMBUS	Is the switched shunt from bus number (input).
Character ID*2	Is the fixed shunt Id (input).
Character NAMVAL*5	Is the value to set or return (input). The values are as follows.
	QINIT the present switched shunt admittance in MVAR
	VMAX the desired voltage upper limit
	VMIN the desired voltage lower limit
Real RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 18 switched shunt not found.
	IERR = 19 switched shunt is out-of-service.
	IERR = 40 fatal error from SWITCHED_SHUNT_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.12. PFLODINT

This API is used to get/set specific integer quantities on a load in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax to set an integer quantity:

```
ierr = pflodint(frombus, id, namval, ival)
```

```
ierr, ival = pflodint(frombus, id, namval)
```

where:

Integer FROMBUS	Is the load from bus number (input).
Character ID*2	Is the load Id (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	STATUS the load status
Integer IVAL	Is the integer value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 14 load not found.
	IERR = 15 load is out-of-service.
	IERR = 38 fatal error from LOAD_API_5.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.13. PFLODREAL

This API is used to get/set specific real quantities on a load in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pflodreal(frombus, id, namval, rval)

ierr, rval = pflodreal(frombus, id, namval)
```

where:

Integer FROMBUS	Is the load from bus number (input).
Character ID*2	Is the load Id (input).
Character NAMVAL*5	Is the value to set or return (input). The values are as follows.
	PLOAD the nominal active constant power load in MW
	QLOAD the nominal reactive constant power load in MVAR
Integer RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 14 load not found.
	IERR = 15 load is out-of-service.
	IERR = 38 fatal error from LOAD_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.14. PFMACINT

This API is used to get/set specific integer quantities on a machine in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pfmacint(frombus, id, namval, ival)

ierr, ival = pfmacint(frombus, id, namval)
```

where:

Integer FROMBUS	machine from bus number (input).
Character ID*2	Is the machine Id (input).
Character NAMVAL*6	Is the value to set or return (input). The values are as follows.
	STATUS the machine status
	Is the integer value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 9 machine not found.
	IERR = 10 machine is out-of-service.
	IERR = 37 fatal error from MACHINE_API_2.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.15. PFMACREAL

This API is used to get/set specific real quantities on a machine in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pfmacreal(frombus, id, namval, rval)

ierr, rval = pfmacreal(frombus, id, namval)
```

where:

Integer FROMBUS	Is the machine from bus number (input).
Character ID*2	Is the machine Id (input).
Character NAMVAL*4	Is the value to set or return (input). The values are as follows.
	PGEN the active power generation in MW
	QMAX the reactive power high limit in MVAR
	QMIN the reactive power low limit in MVAR
Integer RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 9 machine not found.
	IERR = 10 machine is out-of-service.
	IERR = 37 fatal error from MACHINE_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.16. PFPLNTREAL

This API is used to get/set specific real quantities on a plant in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.9.0 and 34.2.0

Python syntax:

```
ierr = pfplntreal(frombus, section, namval, rval)

ierr, rval = pfplntreal(frombus, section, namval)
```

where:

Integer FROMBUS	Is the plant from bus number (input).
Integer SECTION	Is the plant bus section number (input).
	SECTION = 0 If bus FROMBUS is not in a substation or if FROMBYS is not represented by multiple bus sections or if the same change applies to all plants at all bus sections
	SECTION != 0 plant bus section to apply change to
Character NAMVAL*4	Is the value to set or return (input). The values are as follows.
	VSET the desired voltage setpoint
Integer RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 2 not called from allowable PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 7 bus not found.
	IERR = 8 bus is out-of-service.
	IERR = 11 no plant present on this bus section.
	IERR = 12 plant is out-of-service.
	IERR = 13 no in-service plant found on any bus section.
	IERR = 33 invalid node number.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 34 bus is not part of a substation.

IERR = 35 node is not found in this substation.

IERR = 36 fatal error from PLANT_BUS_SECTION_CHNG.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

29.17. PFCNTLVSC

This API is used to get/set specific real quantities on a VSC DC line in a PCI automation function. Its use is only supported in the context of a call from a PCI automation function.

This API routine was first introduced in release 33.6.0

Python syntax:

```
ierr = pfcntlvsc(name, cnvbus, namval, rval)

ierr, rval = pfcntlvsc(name, cnvbus, namval)
```

where:

Character NAME*12	Is the VSC DC line name (input).
Integer CNVBUS	Is the VSC DC line converter bus number (input).
Character NAMVAL*5	Is the value to set or return (input). The values are as follows.
	DCSET the converter dc setpoint
	ACSET the converter ac setpoint
	ALOSS the converter loss coefficient
	BLOSS the converter loss coefficient
	QMAX the reactive power upper limit in MVAR
	QMIN the reactive power lower limit in MVAR
	QSET the reactive power setpoint in MVAR
Integer RVAL	Is the real value to set if specified (input) else returned (output).
Integer IERR	Is the error code (output).
	IERR = 0 no error occurred.
	IERR = 1 not called from PCI automation function.
	IERR = 3 cannot change data in post_ca_sol_eval.
	IERR = 4 working case is empty.
	IERR = 6 invalid NAMVAL string.
	IERR = 41 dc line not found.
	IERR = 42 converter bus not found.
	IERR = 43 bus is not a converter bus of this dc line.
	IERR = 44 for getting "QSET", VSQMAX and VSQMIN are not equal.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 45 for setting "QMAX" or "QMIN", called by Pre_Iteration or Post_Iteration.

IERR = 46 fatal error from VSC_DC_LINE_CONVERTER_API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 30
PSSACCSS Module

This chapter describes Python functions available in the PSS®E Remedial Action Scheme (RAS) and Advanced Contingency module.

These functions allow the user to read an ACC file directly using Python calls. In some situations these APIs are faster than the APIs in the pssarrays module since they deal with smaller amounts of data.

These functions do not record themselves when called while recording is enabled in PSS®E.

Please note that these routines are members of the pssaccss Python extension module, not psspy. The syntax on the following pages assumes that the module has been imported by executing:

```
import pssaccss
```


All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.1. ACCC_FILE_INIT

This API is used to initialize an ACCC file for direct access to the contingency data contained within.

Python syntax:

```
ierr = accc_file_init(rfile, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_INITIALIZE(RFILE, ACCCSIZE, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions. ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ACCCSIZE (15) ACCC file type.

ACCCSIZE (15) = 0 ACCC.

ACCCSIZE (15) = 1 N-1-1.

ACCCSIZE (16) Number of Areas.

ACCCSIZE (17) Number of Zones.

ACCCSIZE (18) Number of Owners.

ACCCSIZE (19) Number of Voltage levels.

ACCCSIZE (20) Number of Ratings.

ACCCSIZE (21) Number of Nodes in ACCC case.

ACCCSIZE (22) Number of Monitored Nodes.

ACCCSIZE (23) Number of Monitored Node records (Range and Deviation).

ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)).

ACCCSIZE (25) Number of Monitored Limit Bus records.

ACCCSIZE (26) Number of Monitored Limit Node records.

ACCCSIZE (27) Number of Monitored SWD flows.

ACCCSIZE (28) Number of Monitored SWD ratings.

ACCCSIZE (29) Number of Contingency Categories.

Integer IERR

Is the error code (output).

IERR = 0 no error occurred.

IERR = 1 error opening RFILE.

IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.2. ACCC_FILE_SIZE

This API is used to get the number of elements contained within and ACCC file so that storage may be allocated for retrieving items (if desired).

Python syntax:

```
ierr, acccsize = accc_file_size(rfile)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_SIZE(RFILE, ACCCSIZE, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Output). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions. ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ACCCSIZE (14) = -1 Undecided.

ACCCSIZE (15) ACCC file type.

ACCCSIZE (15) = 0 ACCC.

ACCCSIZE (15) = 1 N-1-1.

ACCCSIZE (16) Number of Areas.

ACCCSIZE (17) Number of Zones.

ACCCSIZE (18) Number of Owners.

ACCCSIZE (19) Number of Voltage levels.

ACCCSIZE (20) Number of Ratings.

ACCCSIZE (21) Number of Nodes in ACCC case.

ACCCSIZE (22) Number of Monitored Nodes.

ACCCSIZE (23) Number of Monitored Node records (Range and Deviation).

ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)).

ACCCSIZE (25) Number of Monitored Limit Bus records.

ACCCSIZE (26) Number of Monitored Limit Node records.

ACCCSIZE (27) Number of Monitored SWD flows.

ACCCSIZE (28) Number of Monitored SWD ratings.

ACCCSIZE (29) Number of Contingency Categories.

Integer IERR

Is the error code (output).

IERR = 0 no error occurred.

IERR = 4 error opening RFILE.

IERR = 5 error reading RFILE.

IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.3. ACCC_FILE_GET_SUM_VOLT

This API is used to return information about monitored voltage records and limits contained in the contingency data file.

Python syntax:

```
ierr, mvrec_labelxx, mvrec_limxx, mvrec_typexx, mvbus_seqxx,
 vbaselmxx, vemergnxx, mvrec_ivbxx, mvrnd_labelxx, mvrnd_limxx,
 mvrnd_typexx, mvnod_seqxx, vnbaselmxx, vnemergnxx, mvrnd_ivbxx =
 accc_file_get_sum_volt(rfile, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_SUMMARY_VOLTAGES(RFILE, ACCCSIZE, MVREC_LABELxx,
 MVREC_LIMxx, MVREC_TYPExx, MVBUS_SEQxx, VBASELMxx, VEMERGNxx,
 MVREC_IVBxx, MVRND_LABELxx, MVRND_LIMxx, MVRND_TYPExx, MVNOD_SEQxx,
 VNBASELMxx, VNEMERGNxx, MVRND_IVBxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions. ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Character MVREC_LABELxx*34 (ACCCSIZE(5))	Is an array containing monitored voltage record label for all monitored voltage records in the file (output).
Real MVREC_LIMxx (2*ACCCSIZE(5))	Is an array containing monitored voltage record limits for all monitored voltage records in the file (output).
Character MVREC_TYPExx*9 (ACCCSIZE(5))	Is an array containing monitored voltage record type for all monitored voltage records in the file (output). Either 'DEVIATION', ‘RANGE' or ‘LIMIT'
Integer MVBUS_SEQxx (ACCCSIZE(7))	Is an array containing monitored bus sequence numbers for all buses in the monitored bus list in the file (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real VBASELMxx (2*ACCCSIZE(7))	Is an array containing monitored bus normal limits for all buses in the monitored bus list in the file (output).
Real VEMERGNxx (2*ACCCSIZE(7))	Is an array containing monitored bus emergency limits for all buses in the monitored bus list in the file (output).
Integer MVREC_IVBxx (ACCCSIZE(7)* ACCCSIZE(5))	Is an array containing flags to indicate if the bus is monitored in a record, indexed by the bus sequence number and monitor record in the file (output).
Character MVRND_LABELxx*34 (ACCCSIZE(23))	Monitored voltage record label
Real MVRND_LIMxx (ACCCSIZE(23))	Monitored voltage record limits
Character MVRND_TYPExx*9 (ACCCSIZE(23))	Monitor voltage record TYPE
Integer MVNOD_SEQxx (ACCCSIZE(21))	Contains the number in monitored buses list, indexed by the bus sequence number in the bus table
Real VNBASELMxx (ACCCSIZE(21))	The normal voltage limits, indexed by the bus sequence in the bus table
Real VNEMERGNxx (ACCCSIZE(21))	The emergency voltage limits, indexed by the bus sequence in the bus table
Integer MVRND_IVBxx (ACCCSIZE(21)* ACCCSIZE(23))	Contains a flag to indicate if the bus is monitored in a record, indexed by the bus sequence number and monitor record
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 1-4 error in internal parsing routines. IERR = 5 RFILE does not match the one called in ACCC_FILE_SIZE. IERR = 6 error in internal ordering of buses. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.4. ACCC_FILE_CLOSE

This API is used to close an open ACCC file.

Python syntax:

```
ierr = accc_file_close()
```

Fortran syntax:

```
CALL ACCC_FILE_CLOSE(IERR)
```

where:

Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = X error closing open ACCC file. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.5. ACCC_FILE_SUMMARY

This API routine is obsolete. It has been replaced by the API routine accc_file_summary_2.

This API is used to get a summary of the data contained within an ACCC file.

Python syntax:

```
ierr, shrttitle, filenames, cktlabels, mon_ratexx, mvbus_labelxx,
 mvrev_labelxx, mvrec_maxxx, mvrec_minxx, mvrec_typexx, lblcntxx,
 busnamaccxx, addcntxx, addtrpxx, addcorxx, mvrec_ivbxx, ratnamxx,
 ratdescxx, mvnod_labelxx, mvrnd_labelxx, mvrnd_maxxx, mvrnd_minxx,
 mvrnd_typexx, nodnameaccxx, mvrnd_ivbxx = accc_file_summary(rfile,
 acccsize)
```

Fortran syntax:

```
CALL GET_ACCC_SUMMARY(RFILE, ACCCSIZE, SHRTTITLE, FILENAMES, CKTLABELS,
 MON_RATExx, MVBUS_LABELxx, MVREV_LABELxx, MVREC_MAXxx, MVREC_MINxx,
 MVREC_TYPExx, LBLCNTxx, BUSNAMACCxx, ADDCNTxx, ADDTRPxx, ADDCORxx,
 MVREC_IVBxx, RATNAMxx, RATDESCxx, MVNOD_LABELxx, MVRND_LABELxx,
 MVRND_MAXxx, MVRND_MINxx, MVRND_TYPExx, NODNAMEACCxx, MVRND_IVBxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; No default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions. ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions.

ACCCSIZE (14) Name Out option.

ACCCSIZE (14) = 0 Number.

ACCCSIZE (14) = 1 Name.

ACCCSIZE (14) = -1 Undecided.

ACCCSIZE (15) ACCC file type.

ACCCSIZE (15) = 0 ACCC.

ACCCSIZE (15) = 1 N-1-1.

ACCCSIZE (16) Number of Areas.

ACCCSIZE (17) Number of Zones.

ACCCSIZE (18) Number of Owners.

ACCCSIZE (19) Number of Voltage levels.

ACCCSIZE (20) Number of Ratings.

ACCCSIZE (21) Number of Nodes in ACCC case.

ACCCSIZE (22) Number of Monitored Nodes.

ACCCSIZE (23) Number of Monitored Node records (Range and Deviation).

ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)).

ACCCSIZE (25) Number of Monitored Limit Bus records.

ACCCSIZE (26) Number of Monitored Limit Node records.

ACCCSIZE (27) Number of Monitored SWD flows.

ACCCSIZE (28) Number of Monitored SWD ratings.

ACCCSIZE (29) Number of Contingency Categories.

Character SHRTTITLE*60(2) Is an array of two elements that contain the two short title lines of the case contained within the ACCC file.

SHRTTITLE (1) First short title line.

SHRTTITLE (2) Second short title line.

Character FILENAMES*260(9) Is an array of nine elements that contain the file names of all files used in the case contained within the ACCC file.

FILENAMES (1) ACC file name.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	FILENAMES (2) SAV file name. FILENAMES (3) DFX file name. FILENAMES (4) SUB file name. FILENAMES (5) MON file name. FILENAMES (6) CON file name. FILENAMES (7) THR file name. FILENAMES (8) INL file name. FILENAMES (9) TRP file name.
Character CKTLABELS*62 (ACCCSIZE(1)* ACCCSIZE(2))	Is an array that contains the labels of all monitored lines contained within the ACCC file. Multi-section line labels are not stored.
Float MON_RATExx ((ACCCSIZE(1)+ ACCCSIZE(2))* ACCCSIZE(20))	Is an array that contains the Monintored Line Flow ratings.
Character MVBUS_LABELxx*29 (ACCCSIZE(6))	Is an array containing the Monitored Bus labels.
Character MVREV_LABELxx*34 (ACCCSIZE(6))	Is an array containing the Monitored Bus record labels.
Real MVREC_MAXxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record maximum.
Real MVREC_MINxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record minimum.
Character MVREC_TYPExx*9 (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record type (Range, Deviation, Limit).
Charcter LBLCNTxx*152 (ACCCSIZE(3))	Is an array containing the Contingency labels.
Character BUSNAMACCxx*29 (ACCCSIZE(7))	Is an array containing the extended Bus names.
Long Integer ADDCNTxx (ACCCSIZE(3))	Is an array containing ACCC Contingency solution result addresses. Used as input to other APIs.
Long Integer ADDTRPxx (ACCCSIZE(3))	Is an array containing ACCC Tripping solution result addresses. Used as input to other APIs.
Long Integer ADDCORxx (ACCCSIZE(3))	Is an array containing ACCC Corrective Action solution result addresses. Used as input to other APIs.
Integer MVREC_IVBxx (ACCCSIZE(4)* ACCCSIZE(5))	Is an array containing flags used to indicate if the Bus is monitored in a record (indexed by the bus sequence number and monitored bus record).
Character RATNAMxx*6 (ACCCSIZE(20))	Is an array containing the Line Flow Rating names.
Character RATDESCxx*32 (ACCCSIZE(20))	Is an array containing the Line Flow Rating descriptions.
Character MVNOD_LABELxx*29 (ACCCSIZE(24))	Is an array containing the Monitored Node labels.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character MVRND_LABELxx*34 (ACCCSIZE(24))	Is an array containing the Monitored Node record labels.
Real MVRND_MAXxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record maximum.
Real MVRND_MINxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record minimum.
Character MVRND_TYPExx*9 (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record type (Range, Deviation, Limit).
Character NODNAMEACCxx*29 (ACCCSIZE(21))	Is an array containing the Node names.
Integer MVRND_IVBxx (ACCCSIZE(23))	Is an array containing flags used to indicate if the Node is monitored in a record (indexed by the bus sequence number and monitored node record).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 RFILE does not match the one called in ACCC_FILE_SIZE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.6. ACCC_FILE_SUMMARY_2

This API routine is obsolete. It has been replaced by the API routine accc_file_summary_3.

This API is used to get a summary of the data contained within an ACCC file.

Python syntax:

```
ierr, shrttitle, filenames, cktlabels, mon_ratexx, mvbus_labelxx,
 mvrev_labelxx, mvrec_maxxx, mvrec_minxx, mvrec_typexx, lblcntxx,
 busnamaccxx, addcntxx, addtrpxx, addcorxx, mvrec_ivbxx, ratnamxx,
 ratdescxx, mvnod_labelxx, mvrnd_labelxx, mvrnd_maxxx, mvrnd_minxx,
 mvrnd_typexx, nodnameaccxx, mvrnd_ivbxx, vbaselLst, vemergnLst,
 vnbaselLst, vnemergnLst, mvlbusrec_labelLst, mvlnodrec_labelLst =
 accc_file_summary(rfile, acccsize, options)
```

Fortran syntax:

```
CALL GET_ACCC_SUMMARY_2(RFILE, ACCCSIZE, OPTIONS, SHRTTITLE, FILENAMES,
 CKTLABELS, MON_RATExx, MVBUS_LABELxx, MVREV_LABELxx, MVREC_MAXxx,
 MVREC_MINxx, MVREC_TYPExx, LBLCNTxx, BUSNAMACCxx, ADDCNTxx, ADDTRPxx,
 ADDCORxx, MVREC_IVBxx, RATNAMxx, RATDESCxx, MVNOD_LABELxx, MVRND_LABELxx,
 MVRND_MAXxx, MVRND_MINxx, MVRND_TYPExx, NODNAMEACCxx, MVRND_IVBxx,
 VBASELxx, VEMERGNxx, VNBASELxx, VNEMERGNxx, VBASEL_LABEL, VNBASEL_LABEL,
 IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; No default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Integer OPTIONS(2)	Is an array of two elements containing base case voltage limit and contingency case voltage limit. OPTIONS (1) Base case voltage limit (default 1) OPTIONS (2) Contingency case voltage limit (default 2)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character SHRTTITLE*60(2)	Is an array of two elements that contain the two short title lines of the case contained within the ACCC file. SHRTTITLE (1) First short title line. SHRTTITLE (2) Second short title line.
Character FILENAMES*260(9)	Is an array of nine elements that contain the file names of all files used in the case contained within the ACCC file. FILENAMES (1) ACC file name. FILENAMES (2) SAV file name. FILENAMES (3) DFX file name. FILENAMES (4) SUB file name. FILENAMES (5) MON file name. FILENAMES (6) CON file name. FILENAMES (7) THR file name. FILENAMES (8) INL file name. FILENAMES (9) TRP file name.
Character CKTLABELS*62 (ACCCSIZE(1)* ACCCSIZE(2))	Is an array that contains the labels of all monitored lines contained within the ACCC file. Multi-section line labels are not stored.
Float MON_RATExx ((ACCCSIZE(1)+ ACCCSIZE(2))* ACCCSIZE(20))	Is an array that contains the Monintored Line Flow ratings.
Character MVBUS_LABELxx*29 (ACCCSIZE(6))	Is an array containing the Monitored Bus labels.
Character MVREV_LABELxx*34 (ACCCSIZE(6))	Is an array containing the Monitored Bus record labels.
Real MVREC_MAXxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record maximum.
Real MVREC_MINxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record minimum.
Character MVREC_TYPExx*9 (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record type (Range, Deviation, Limit).
Charcter LBLCNTxx*152 (ACCCSIZE(3))	Is an array containing the Contingency labels.
Character BUSNAMACCxx*29 (ACCCSIZE(7))	Is an array containing the extended Bus names.
Long Integer ADDCNTxx (ACCCSIZE(3))	Is an array containing ACCC Contingency solution result addresses. Used as input to other APIs.
Long Integer ADDTRPxx (ACCCSIZE(3))	Is an array containing ACCC Tripping solution result addresses. Used as input to other APIs.
Long Integer ADDCORxx (ACCCSIZE(3))	Is an array containing ACCC Corrective Action solution result addresses. Used as input to other APIs.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer MVREC_IVBxx (ACCCSIZE(4)* ACCCSIZE(5))	Is an array containing flags used to indicate if the Bus is monitored in a record (indexed by the bus sequence number and monitored bus record).
Character RATNAMxx*6 (ACCCSIZE(20))	Is an array containing the Line Flow Rating names.
Character RATDESCxx*32 (ACCCSIZE(20))	Is an array containing the Line Flow Rating descriptions.
Character MVNOD_LABELxx*29 (ACCCSIZE(24))	Is an array containing the Monitored Node labels.
Character MVRND_LABELxx*34 (ACCCSIZE(24))	Is an array containing the Monitored Node record labels.
Real MVRND_MAXxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record maximum.
Real MVRND_MINxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record minimum.
Character MVRND_TYPExx*9 (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record type (Range, Deviation, Limit).
Character NODNAMEACCxx*29 (ACCCSIZE(21))	Is an array containing the Node names.
Integer MVRND_IVBxx (ACCCSIZE(23))	Is an array containing flags used to indicate if the Node is monitored in a record (indexed by the bus sequence number and monitored node record).
Integer VBASELxx (ACCCSIZE(25))	Is an array containing the normal Monitored Bus voltage limits.
Integer VEMERGNxx (ACCCSIZE(25))	Is an array containing the emergency Monitored Bus voltage limits.
Integer VNBASELxx (ACCCSIZE(26))	Is an array containing the normal Monitored Node voltage limits.
Integer VNEMERGNxx (ACCCSIZE(26))	Is an array containing the emergency Monitored Node voltage limits.
Character MVLBUSREC_LABELxx*29 (ACCCSIZE(26))	Is an array containing the monitored limit bus numbers in order of values returned for VBASELxx and VEMERGNxx
Character MVLNODREC_LABELxx*29 (ACCCSIZE(26))	Is an array containing the monitored limit node names in order of values returned for VNBASELxx and VNEMERGNxx
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 RFILE does not match the one called in ACCC_FILE_SIZE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.7. ACCC_FILE_SUMMARY_3

This API routine is obsolete. It has been replaced by the API routine accc_file_summary_4.

Python syntax:

```
ierr, shrttitle, filenames, cktlabels, mon_ratexx, mvbus_labelxx,
 mvrev_labelxx, mvrec_maxxx, mvrec_minxx, mvrec_typexx, lblcntxx,
 busnamaccxx, addcntxx, addtrpxx, addcorxx, mvrec_ivbxx,
 ratnamxx, ratdescxx, mvnod_labelxx, mvrnd_labelxx, mvrnd_maxxx,
 mvrnd_minxx, mvrnd_typexx, nodnameaccxx, mvrnd_ivbxx, vbaselLst,
 vemergnLst, vnbaselLst, vnemergnLst, mvlbusrec_labelLst,
 mvlnodrec_labelLst, mswd_labelsxx, acswdflow_outxx, swd_rat_num_outxx =
 accc_file_summary_3(rfile, acccsize, options)
```

Fortran syntax:

```
CALL GET_ACCC_SUMMARY_3(RFILE, ACCCSIZE, OPTIONS, SHRTTITLE, FILENAMES,
 CKTLABELS, MON_RATExx, MVBUS_LABELxx, MVREV_LABELxx, MVREC_MAXxx,
 MVREC_MINxx, MVREC_TYPExx, LBLCNTxx, BUSNAMACCxx, ADDCNTxx, ADDTRPxx,
 ADDCORxx, MVREC_IVBxx, RATNAMxx, RATDESCxx, MVNOD_LABELxx, MVRND_LABELxx,
 MVRND_MAXxx, MVRND_MINxx, MVRND_TYPExx, NODNAMEACCxx, MVRND_IVBxx,
 VBASELxx, VEMERGNxx, VNBASELxx, VNEMERGNxx, VBASEL_LABEL, VNBASEL_LABEL,
 MSWD_LABELSxx, ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; No default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Integer OPTIONS(2)	Is an array of two elements containing base case voltage limit and contingency case voltage limit. OPTIONS (1) Base case voltage limit (default 1) OPTIONS (2) Contingency case voltage limit (default 2)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character SHRTTITLE*60(2)	Is an array of two elements that contain the two short title lines of the case contained within the ACCC file. SHRTTITLE (1) First short title line. SHRTTITLE (2) Second short title line.
Character FILENAMES*260(9)	Is an array of nine elements that contain the file names of all files used in the case contained within the ACCC file. FILENAMES (1) ACC file name. FILENAMES (2) SAV file name. FILENAMES (3) DFX file name. FILENAMES (4) SUB file name. FILENAMES (5) MON file name. FILENAMES (6) CON file name. FILENAMES (7) THR file name. FILENAMES (8) INL file name. FILENAMES (9) TRP file name.
Character CKTLABELS*62 (ACCCSIZE(1)* ACCCSIZE(2))	Is an array that contains the labels of all monitored lines contained within the ACCC file. Multi-section line labels are not stored.
Float MON_RATExx ((ACCCSIZE(1)+ ACCCSIZE(2))* ACCCSIZE(20))	Is an array that contains the Monintored Line Flow ratings.
Character MVBUS_LABELxx*29 (ACCCSIZE(6))	Is an array containing the Monitored Bus labels.
Character MVREV_LABELxx*34 (ACCCSIZE(6))	Is an array containing the Monitored Bus record labels.
Real MVREC_MAXxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record maximum.
Real MVREC_MINxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record minimum.
Character MVREC_TYPExx*9 (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record type (Range, Deviation, Limit).
Charcter LBLCNTxx*152 (ACCCSIZE(3))	Is an array containing the Contingency labels.
Character BUSNAMACCxx*29 (ACCCSIZE(7))	Is an array containing the extended Bus names.
Long Integer ADDCNTxx (ACCCSIZE(3))	Is an array containing ACCC Contingency solution result addresses. Used as input to other APIs.
Long Integer ADDTRPxx (ACCCSIZE(3))	Is an array containing ACCC Tripping solution result addresses. Used as input to other APIs.
Long Integer ADDCORxx (ACCCSIZE(3))	Is an array containing ACCC Corrective Action solution result addresses. Used as input to other APIs.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer MVREC_IVBxx (ACCCSIZE(4)* ACCCSIZE(5))	Is an array containing flags used to indicate if the Bus is monitored in a record (indexed by the bus sequence number and monitored bus record).
Character RATNAMxx*6 (ACCCSIZE(20))	Is an array containing the Line Flow Rating names.
Character RATDESCxx*32 (ACCCSIZE(20))	Is an array containing the Line Flow Rating descriptions.
Character MVNOD_LABELxx*29 (ACCCSIZE(24))	Is an array containing the Monitored Node labels.
Character MVRND_LABELxx*34 (ACCCSIZE(24))	Is an array containing the Monitored Node record labels.
Real MVRND_MAXxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record maximum.
Real MVRND_MINxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record minimum.
Character MVRND_TYPExx*9 (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record type (Range, Deviation, Limit).
Character NODNAMEACCxx*29 (ACCCSIZE(21))	Is an array containing the Node names.
Integer MVRND_IVBxx (ACCCSIZE(23))	Is an array containing flags used to indicate if the Node is monitored in a record (indexed by the bus sequence number and monitored node record).
Integer VBASELxx (ACCCSIZE(25))	Is an array containing the normal Monitored Bus voltage limits.
Integer VEMERGNxx (ACCCSIZE(25))	Is an array containing the emergency Monitored Bus voltage limits.
Integer VNBASELxx (ACCCSIZE(26))	Is an array containing the normal Monitored Node voltage limits.
Integer VNEMERGNxx (ACCCSIZE(26))	Is an array containing the emergency Monitored Node voltage limits.
Character MVLBUSREC_LABELxx*29 (ACCCSIZE(26))	Is an array containing the monitored limit bus numbers in order of values returned for VBASELxx and VEMERGNxx
Character MVLNODREC_LABELxx*45 (ACCCSIZE(26))	Is an array containing the monitored limit node names in order of values returned for VNBASELxx and VNEMERGNxx
Character MSWD_LABELSxx*29 (ACCCSIZE(26))	Is an array containing the monitored switching device names
Real ACSWDFLOW_OUTxx (ACCCSIZE(27))	Is an array containing all the desired MVA flows for the desired SWD (output).
Real SWD_RAT_NUM_OUTxx (ACCCSIZE(27))	Is an array containing all the desired Ampere flows for the desired SWD (output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 RFILE does not match the one called in ACCC_FILE_SIZE.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 6 error in fetching contingency strings.

IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.8. ACCC_FILE_SUMMARY_4

This API is used to get a summary of the data contained within an ACCC file.

Python syntax:

```
ierr, shrttitle, filenames, cktlabels, mon_ratexx, mvbus_labelxx,
 mvrev_labelxx, mvrec_maxxx, mvrec_minxx, mvrec_typexx, lblcntxx,
 busnamaccxx, addcntxx, addtrpxx, addcorxx, mvrec_ivbxx, ratnamxx,
 ratdescxx, mvnod_labelxx, mvrnd_labelxx, mvrnd_maxxx, mvrnd_minxx,
 mvrnd_typexx, nodnameaccxx, mvrnd_ivbxx, vbaselLst, vemergnLst,
 vnbaselLst, vnemergnLst, mvlbusrec_labelLst, mvlnodrec_labelLst,
 mswd_labelsxx, acswdflow_outxx, swd_rat_num_outxx, categoryxx, catelenxx=
 accc_file_summary_4(rfile, acccsize, options)
```

Fortran syntax:

```
CALL GET_ACCC_SUMMARY_4(RFILE, ACCCSIZE, OPTIONS, SHRTTITLE, FILENAMES,
 CKTLABELS, MON_RATExx, MVBUS_LABELxx, MVREV_LABELxx, MVREC_MAXxx,
 MVREC_MINxx, MVREC_TYPExx, LBLCNTxx, BUSNAMACCxx, ADDCNTxx, ADDTRPxx,
 ADDCORxx, MVREC_IVBxx, RATNAMxx, RATDESCxx, MVNOD_LABELxx, MVRND_LABELxx,
 MVRND_MAXxx, MVRND_MINxx, MVRND_TYPExx, NODNAMEACCxx, MVRND_IVBxx,
 VBASELxx, VEMERGNxx, VNBASELxx, VNEMERGNxx, VBASEL_LABEL, VNBASEL_LABEL,
 MSWD_LABELSxx, ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx, CATEGORYxx, CATELENxx,
 IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; No default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions.

ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions.

ACCCSIZE (14) Name Out option.

ACCCSIZE (14) = 0 Number.

ACCCSIZE (14) = 1 Name.

ACCCSIZE (14) = -1 Undecided.

ACCCSIZE (15) ACCC file type.

ACCCSIZE (15) = 0 ACCC.

ACCCSIZE (15) = 1 N-1-1.

ACCCSIZE (16) Number of Areas.

ACCCSIZE (17) Number of Zones.

ACCCSIZE (18) Number of Owners.

ACCCSIZE (19) Number of Voltage levels.

ACCCSIZE (20) Number of Ratings.

ACCCSIZE (21) Number of Nodes in ACCC case.

ACCCSIZE (22) Number of Monitored Nodes.

ACCCSIZE (23) Number of Monitored Node records (Range and Deviation).

ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)).

ACCCSIZE (25) Number of Monitored Limit Bus records.

ACCCSIZE (26) Number of Monitored Limit Node records.

ACCCSIZE (27) Number of Monitored SWD flows.

ACCCSIZE (28) Number of Monitored SWD ratings.

ACCCSIZE (29) Number of Contingency Categories.

Integer OPTIONS(2)

Is an array of two elements containing base case voltage limit and contingency case voltage limit.

OPTIONS (1) Base case voltage limit (default 1)

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	OPTIONS (2) Contingency case voltage limit (default 2)
Character SHRTTITLE*60(2)	Is an array of two elements that contain the two short title lines of the case contained within the ACCC file. SHRTTITLE (1) First short title line. SHRTTITLE (2) Second short title line.
Character FILENAMES*260(9)	Is an array of nine elements that contain the file names of all files used in the case contained within the ACCC file. FILENAMES (1) ACC file name. FILENAMES (2) SAV file name. FILENAMES (3) DFX file name. FILENAMES (4) SUB file name. FILENAMES (5) MON file name. FILENAMES (6) CON file name. FILENAMES (7) THR file name. FILENAMES (8) INL file name. FILENAMES (9) TRP file name.
Character CKTLABELS*62 (ACCCSIZE(1)* ACCCSIZE(2))	Is an array that contains the labels of all monitored lines contained within the ACCC file. Multi-section line labels are not stored.
Float MON_RATExx ((ACCCSIZE(1)+ ACCCSIZE(2))* ACCCSIZE(20))	Is an array that contains the Monintored Line Flow ratings.
Character MVBUS_LABELxx*29 (ACCCSIZE(6))	Is an array containing the Monitored Bus labels.
Character MVREV_LABELxx*34 (ACCCSIZE(6))	Is an array containing the Monitored Bus record labels.
Real MVREC_MAXxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record maximum.
Real MVREC_MINxx (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record minimum.
Character MVREC_TYPExx*9 (ACCCSIZE(6))	Is an array containing the Monitored Bus voltage record type (Range, Deviation, Limit).
Charcter LBLCNTxx*152 (ACCCSIZE(3))	Is an array containing the Contingency labels.
Character BUSNAMACCxx*29 (ACCCSIZE(7))	Is an array containing the extended Bus names.
Long Integer ADDCNTxx (ACCCSIZE(3))	Is an array containing ACCC Contingency solution result addresses. Used as input to other APIs.
Long Integer ADDTRPxx (ACCCSIZE(3))	Is an array containing ACCC Tripping solution result addresses. Used as input to other APIs.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Long Integer ADDCORxx (ACCCSIZE(3))	Is an array containing ACCC Corrective Action solution result addresses. Used as input to other APIs.
Integer MVREC_IVBxx (ACCCSIZE(4)* ACCCSIZE(5))	Is an array containing flags used to indicate if the Bus is monitored in a record (indexed by the bus sequence number and monitored bus record).
Character RATNAMxx*6 (ACCCSIZE(20))	Is an array containing the Line Flow Rating names.
Character RATDESCxx*32 (ACCCSIZE(20))	Is an array containing the Line Flow Rating descriptions.
Character MVNOD_LABELxx*29 (ACCCSIZE(24))	Is an array containing the Monitored Node labels.
Character MVRND_LABELxx*34 (ACCCSIZE(24))	Is an array containing the Monitored Node record labels.
Real MVRND_MAXxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record maximum.
Real MVRND_MINxx (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record minimum.
Character MVRND_TYPExx*9 (ACCCSIZE(24))	Is an array containing the Monitored Node voltage record type (Range, Deviation, Limit).
Character NODNAMEACCxx*29 (ACCCSIZE(21))	Is an array containing the Node names.
Integer MVRND_IVBxx (ACCCSIZE(23))	Is an array containing flags used to indicate if the Node is monitored in a record (indexed by the bus sequence number and monitored node record).
Integer VBASELxx (ACCCSIZE(25))	Is an array containing the normal Monitored Bus voltage limits.
Integer VEMERGNxx (ACCCSIZE(25))	Is an array containing the emergency Monitored Bus voltage limits.
Integer VNBASELxx (ACCCSIZE(26))	Is an array containing the normal Monitored Node voltage limits.
Integer VNEMERGNxx (ACCCSIZE(26))	Is an array containing the emergency Monitored Node voltage limits.
Character MVLBUSREC_LABELxx*29 (ACCCSIZE(26))	Is an array containing the monitored limit bus numbers in order of values returned for VBASELxx and VEMERGNxx
Character MVLNODREC_LABELxx*45 (ACCCSIZE(26))	Is an array containing the monitored limit node names in order of values returned for VNBASELxx and VNEMERGNxx
Character MSWD_LABELSxx*29 (ACCCSIZE(26))	Is an array containing the monitored switching device names
Real ACSWDFLOW_OUTxx (ACCCSIZE(27))	Is an array containing all the desired MVA flows for the desired SWD (output).
Real SWD_RAT_NUM_OUTxx (ACCCSIZE(27))	Is an array containing all the desired Ampere flows for the desired SWD (output).
Character CATEGORYxx (ACCCSIZE(29))	Is an array containing all contingency categories (output).
Integer CATELENxx (ACCCSIZE(29))	Is an array containing the length of contingency categories (output).
Integer IERR	Is the error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0 no error occurred.

IERR = 4 error opening RFILE.

IERR = 5 RFILE does not match the one called in ACCC_FILE_SIZE.

IERR = 6 error in fetching contingency strings.

IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.9. ACCC_FILE_SUM_AREAS

This API is used to return information about areas, owners and zone information contained in the contingency data file.

Python syntax:

```
ierr, bsarea_outxxx, bszone_outxx, bsowne_outxx, bsvolt_outxx,
 nuarea_outxx, nuzone_outxx, nuowne_outxx, nuvolt_outxx, naarea_outxx,
 nazone_outxx, naowne_outxx = accc_file_sum_areas(rfile, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_SUMMARY_AREAS(RFILE, ACCCSIZE, BSAREA_OUTxxx,
 BSZONE_OUTxx, BSOWNE_OUTxx, BSVOLT_OUTxx, NUAREA_OUTxx, NUZONE_OUTxx,
 NUOWNE_OUTxx, NUVOLT_OUTxx, NAAREA_OUTxx, NAZONE_OUTxx, NAOWNE_OUTxx,
 NUVOLT_OUTxx, NAAREA_OUTxx, NAZONE_OUTxx, NAOWNE_OUTxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions. ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (14) = 0 Number.
	ACCCSIZE (14) = 1 Name.
	ACCCSIZE (14) = -1 Undecided.
	ACCCSIZE (15) ACCC file type.
	ACCCSIZE (15) = 0 ACCC.
	ACCCSIZE (15) = 1 N-1-1.
	ACCCSIZE (16) Number of Areas.
	ACCCSIZE (17) Number of Zones.
	ACCCSIZE (18) Number of Owners.
	ACCCSIZE (19) Number of Voltage levels.
	ACCCSIZE (20) Number of Ratings.
	ACCCSIZE (21) Number of Nodes in ACCC case.
	ACCCSIZE (22) Number of Monitored Nodes.
	ACCCSIZE (23) Number of Monitored Node records (Range and Deviation).
	ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)).
	ACCCSIZE (25) Number of Monitored Limit Bus records.
	ACCCSIZE (26) Number of Monitored Limit Node records.
	ACCCSIZE (27) Number of Monitored SWD flows.
	ACCCSIZE (28) Number of Monitored SWD ratings.
	ACCCSIZE (29) Number of Contingency Categories.
Integer BSAREA_OUTxxx (ACCCSIZE(7))	Is an array containing all the Area numbers for the buses in the file (output).
Integer , BSZONE_OUTxxx (ACCCSIZE(7))	Is an array containing all the Zone numbers for the buses in the file (output).
Integer BSOWNE_OUTxxx (ACCCSIZE(7))	Is an array containing all the Owner numbers for the buses in the file (output).
Real BSVOLT_OUTxxx (ACCCSIZE(7))	Is an array containing all the Voltages for the buses in the file (output).
Integer NUAREA_OUTxxx (ACCCSIZE(16))	Is an array containing all the Area numbers in the file (output).
Integer NUZONE_OUTxxx (ACCCSIZE(17))	Is an array containing all the Zone numbers in the file (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer NUOWNE_OUTxxx (ACCCSIZE(18))	Is an array containing all the Owner numbers in the file (output).
Real NUOWNE_OUTxxx (ACCCSIZE(19))	Is an array containing all the Voltage levels numbers in the file (output).
Character NAAREA_OUTxxx*12 (ACCCSIZE(16))	Is an array containing all the Area names in the file (output).
Character NAZONE_OUTxxx*12 (ACCCSIZE(17))	Is an array containing all the Zone names in the file (output).
Character NAOWNE_OUTxxx*12 (ACCCSIZE(18))	Is an array containing all the Owner names in the file (output).
Integer IERR	Is the error code (output). IERR = -1 No results IERR = 0 no error occurred. IERR = 5 RFILE does not match the one called in ACCC_FILE_SIZE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.10. ACCC_FILE_CNTG_VOLT

This API is used to return voltages for a specified contingency and contingency type for all monitored buses in the contingency data file.

Python syntax:

```
ierr, mv_vmag_outxx, mvnode_vmag_outxx = accc_file_cntg_volt(rfile,
 getwhat, cntpos, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_CONTINGENCY_VOLTAGE(RFILE, GETWHAT, CNTPOS, ACCCSIZE,
 MV_VMAG_OUTxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer GETWHAT	Is the voltages to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution voltages GETWHAT = 2 Tripping solution voltages GETWHAT = 3 Corrective action solution voltages
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired voltages (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Real MV_VMAG_OUTxx (ACCCSIZE(4))	Is an array containing all the desired voltages for the desired contingency (output).
Real MVNODE_VMAG_OUTxx (ACCCSIZE(4))	Is an array containing all the desired node voltages for the desired contingency (output).
Integer IERR	Is the error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0 no error occurred.

IERR = 4 error opening RFILE.

IERR = 5 error reading RFILE.

IERR = 6 error in fetching contingency strings.

IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.11. ACCC_FILE_CNTG_FLOW

This API routine is obsolete. It has been replaced by the API routine accc_file_cntg_flow_2.

This API is used to return flows for a specified contingency and contingency type for all monitored lines in the contingency data file.

Python syntax:

```
ierr, acflow_outxx, rat_num_outxx = accc_file_cntg_flow(getwhat, cntpos,
 acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_CONTINGENCY_FLOW(GETWHAT, CNTPOS, ACCCSIZE,
 ACFLOW_OUTxx, RAT_NUM_OUTxx, IERR)
```

where:

Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution flows GETWHAT = 2 Tripping solution flows GETWHAT = 3 Corrective action solution flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Real ACFLOW_OUTxx ((ACCCSIZE(1)+ ACCCSIZE(2))	Is an array containing all the desired MVA flows for the desired contingency (output).
Real RAT_NUM_OUTxx (ACCCSIZE(1))	Is an array containing all the desired Ampere flows for the desired contingency (output).
Integer IERR	Is the error code (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

IERR = 0 no error occurred.

IERR = 4 error opening RFILE.

IERR = 5 error reading RFILE.

IERR = 6 error in fetching contingency strings.

IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.12. ACCC_FILE_CNTG_FLOW_2

This API is used to return flows for a specified contingency and contingency type for all monitored lines in the contingency data file.

Python syntax:

```
ierr, acflow_outxx, rat_num_outxx, acswdflow_outxx, swd_rat_num_outxx =
 accc_file_cntg_flow_2(getwhat, cntpos, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_CONTINGENCY_FLOW_2(GETWHAT, CNTPOS, ACCCSIZE,
 ACFLOW_OUTxx, RAT_NUM_OUTxx, ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx, IERR)
```

where:

Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution flows GETWHAT = 2 Tripping solution flows GETWHAT = 3 Corrective action solution flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions. ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Real ACFLOW_OUTxx ((ACCCSIZE(1)+ ACCCSIZE(2))	Is an array containing all the desired MVA flows for the desired contingency (output).
Real RAT_NUM_OUTxx (ACCCSIZE(1))	Is an array containing all the desired Ampere flows for the desired contingency (output).
Real ACSWDFLOW_OUTxx (ACCCSIZE(27))	Is an array containing all the desired MVA flows for the desired SWD (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real SWD_RAT_NUM_OUTxx (ACCCSIZE(28))	Is an array containing all the desired Ampere flows for the desired SWD (output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 error reading RFILE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.13. ACCC_FILE_CNTG_FLOW_3

This API is used to return flows for a specified contingency and contingency type for all monitored lines in the contingency data file.

Python syntax:

```
ierr, acflow_outxx, rat_num_outxx, acswdflow_outxx, swd_rat_num_outxx,
 swdlpflowxx = accc_file_cntg_flow_3(getwhat, cntpos, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_CONTINGENCY_FLOW_3(GETWHAT, CNTPOS, ACCCSIZE,
 ACFLOW_OUTxx, RAT_NUM_OUTxx, ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx,
 SWDLPFLOWxx, IERR)
```

where:

Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution flows GETWHAT = 2 Tripping solution flows GETWHAT = 3 Corrective action solution flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Real ACFLOW_OUTxx ((ACCCSIZE(1)+ ACCCSIZE(2))	Is an array containing all the desired MVA flows for the specified contingency (output).
Real RAT_NUM_OUTxx (ACCCSIZE(1))	Is an array containing all the desired Ampere flows for the specified contingency (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real ACSWDFLOW_OUTxx (ACCCSIZE(27))	Is an array containing all the desired MVA flows for the monitored SWD (output).
Real SWD_RAT_NUM_OUTxx (ACCCSIZE(27))	Is an array containing all the desired Ampere flows for the monitored SWD (output).
Integer SWDLPFLOWxx (ACCCSIZE(27))	Is an array containing all the desired Active Power flows for the monitored SWD (output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 error reading RFILE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.14. ACCC_FILE_CNTG_FLOW_4

This API is used to return flows for a specified contingency and contingency type for all monitored lines in the contingency data file.

Python syntax:

```
ierr, acflow_outxx, rat_num_outxx, acswdflow_outxx, swd_rat_num_outxx,
 swdlpflowxx, acangledeltxx = accc_file_cntg_flow_4(getwhat, cntpos,
 acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_CONTINGENCY_FLOW_4(GETWHAT, CNTPOS, ACCCSIZE,
 ACFLOW_OUTxx, RAT_NUM_OUTxx, ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx,
 SWDLPFLOWxx, ACANGLEDELTxx, IERR)
```

where:

Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution flows GETWHAT = 2 Tripping solution flows GETWHAT = 3 Corrective action solution flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Real ACFLOW_OUTxx ((ACCCSIZE(1)+ ACCCSIZE(2))	Is an array containing all the desired MVA flows for the desired contingency (output).
Real RAT_NUM_OUTxx (ACCCSIZE(1))	Is an array containing all the desired Ampere flows for the desired contingency (output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real ACSWDFLOW_OUTxx (ACCSIZE(27))	Is an array containing all the MVA flows for the monitored SWD (output).
Real SWD_RAT_NUM_OUTxx (ACCCSIZE(27))	Is an array containing all the Ampere flows for the monitored SWD (output).
Integer SWDLPFLOWxx (ACCSIZE(27))	Is an array containing all the Active Power flows for the monitored SWD (output).
Integer ACANGLEDELTxx (ACCSIZE(1))	Is an array containing the bus angle differences across the monitored lines (output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 error reading RFILE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.15. ACCC_FILE_CNTG_DSCR

This API is used to return voltages and flows for a specified contingency and contingency type for all monitored buses and lines in the contingency data file.

Python syntax:

```
ierr, num_island, case_worst, case_total, cnv_cond, cnt_desc =
 accc_file_cntg_dscr(rfile, getwhat, cntpos, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_CONTINGENCY_DESCRIPTION(RFILE, GETWHAT, CNTPOS,
 ACCCSIZE, NUM_ISLAND, CASE_WORST, CASE_TOTAL, CNV_COND, CNT_DESC, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution voltages and flows GETWHAT = 2 Tripping solution voltages and flows GETWHAT = 3 Corrective action solution voltages and flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping. ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions.
	ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions.
	ACCCSIZE (14) Name Out option.
	ACCCSIZE (14) = 0 Number.
	ACCCSIZE (14) = 1 Name.
	ACCCSIZE (14) = -1 Undecided.
	ACCCSIZE (15) ACCC file type.
	ACCCSIZE (15) = 0 ACCC.
	ACCCSIZE (15) = 1 N-1-1.
	ACCCSIZE (16) Number of Areas.
	ACCCSIZE (17) Number of Zones.
	ACCCSIZE (18) Number of Owners.
	ACCCSIZE (19) Number of Voltage levels.
	ACCCSIZE (20) Number of Ratings.
	ACCCSIZE (21) Number of Nodes in ACCC case.
	ACCCSIZE (22) Number of Monitored Nodes.
	ACCCSIZE (23) Number of Monitored Node records (Range and Deviation).
	ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)).
	ACCCSIZE (25) Number of Monitored Limit Bus records.
	ACCCSIZE (26) Number of Monitored Limit Node records.
	ACCCSIZE (27) Number of Monitored SWD flows.
	ACCCSIZE (28) Number of Monitored SWD ratings.
	ACCCSIZE (29) Number of Contingency Categories.
Integer NUM_ISLAND	Is the number of islanded buses in the desired contingency (Output).
Real CASE_WORST	Is the worst case in the desired contingency (Output).
Real CASE_TOTAL	Is the case total in the desired contingency (Output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Character CNV_COND*34	Is the convergence solution text for the desired contingency (Output).
Character CNT_DESC*160 (32)	Is the description of every contingency in the specified contingency group (Output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 5 error reading RFILE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.16. ACCC_FILE_CNTG_LOAD

This API is used to return voltages and flows for a specified contingency and contingency type for all monitored buses and lines in the contingency data file.

Python syntax:

```
ierr, num_loadshed, lshed_bus_outxx, lshed_outxx, num_gendisp,
 gdisp_bus_outxx, disp_outxx, num_phsshft, pshft_bus_outxx, pshft_outxx =
 accc_file_cntg_load(rfile, getwhat, cntpos, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_CONTINGENCY_LOAD(RFILE, GETWHAT, CNTPOS, ACCCSIZE,
 NUM_LOADSHED, LSHED_BUS_OUTxx, LSHED_OUTxx, NUM_GENDISP, GDISP_BUS_OUTxx,
 GDISP_OUTxx, NUM_PHSSHFT, PSHFT_BUS_OUTxx, PSHFT_OUTxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution loads GETWHAT = 2 Tripping solution loads GETWHAT = 3 Corrective action solution voltages and flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows. ACCCSIZE (1) Number of monitored lines. ACCCSIZE (2) Number of interfaces. ACCCSIZE (3) Number of Contingencies + Base Case. ACCCSIZE (4) Number of voltage buses. ACCCSIZE (5) Number of voltage records. ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)). ACCCSIZE (7) Number of buses in ACCC case. ACCCSIZE (8) Acc file version number. ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency. ACCCSIZE (10) Max # of LOAD SHEDS due to tripping.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions. ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions. ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions. ACCCSIZE (14) Name Out option. ACCCSIZE (14) = 0 Number. ACCCSIZE (14) = 1 Name. ACCCSIZE (14) = -1 Undecided. ACCCSIZE (15) ACCC file type. ACCCSIZE (15) = 0 ACCC. ACCCSIZE (15) = 1 N-1-1. ACCCSIZE (16) Number of Areas. ACCCSIZE (17) Number of Zones. ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Integer NUM_LOADSHED	Is the number of loads shed for the desired contingency (Output).
Real LSHED_BUS_OUTxx (ACCCSIZE(11))	Is an array containing all the load shedding buses for the desired contingency (Output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real LSHED_OUTxx (2*ACCCSIZE(11))	Is an array containing all the load shed at the load shedding buses for the desired contingency (Output).
Integer NUM_GENDISP	Is the number of Generators dispatched for the desired contingency (Output).
Real GDISP_BUS_OUTxx (ACCCSIZE(12))	Is an array containing all the generation dispatching buses for the desired contingency (Output).
Real GDISP_OUTxx (2*ACCCSIZE(12))	Is an array containing all the generation dispatch at the generation dispatching buses for the desired contingency (Output).
Integer NUM_PHSSHFT	Is the number of phase shifters for the desired contingency (Output).
Character PSHFT_BUS_OUTxx*33 (ACCCSIZE(13))	Is an array containing all the phase shifting buses for the desired contingency (Output).
Real PSHFT_OUTxx (2*ACCCSIZE(13))	Is an array containing all the phase shifts for the phase shifting buses for the desired contingency (Output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 5 error reading RFILE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.17. ACCC_FILE_SOLUTION

This API is used to return voltages and flows for a specified contingency and contingency type for all monitored buses and lines in the contingency data file.

Python syntax:

```
ierr, cnt_desc, cnv_flg, cnv_cond, num_island, case_worst, case_total,
 mv_vmag_outxx, acflow_outxx, rat_num_outxx, lshed_bus_outxx, lshed_outxx,
 mvnod_vmagxx = accc_file_solution(rfile, getwhat, cntpos, bus_msm,
 sys_msm, acccsize)
```

or:

```
ierr, cnt_desc, cnv_flg, cnv_cond, num_island, case_worst, case_total,
 mv_vmag_outxx, acflow_outxx, rat_num_outxx, lshed_bus_outxx,
 lshed_outxx, gdispbusxx, gdispxx, phsftnamexx, phsftangxx, mvnod_vmagxx =
 accc_file_solution(rfile, getwhat, cntpos, bus_msm, sys_msm, acccsize)
```

Fortran syntax:

```
CALL ACCC_FILE_GET_1CNTGNCY_SOLUTION(RFILE, GETWHAT, CNTPOS,
 BUS_MSM, SYS_MSM, ACCCSIZE, CNT_DESC, CNV_FLG, CNV_COND, NUM_ISLAND,
 CASE_WORST, CASE_TOTAL,  , MV_VMAG_OUTxx, ACFLOW_OUTxx, RAT_NUM_OUTxx,
 LSHED_BUS_OUTxx, LSHED_OUTxx, MVNOD_VMAGxx, IERR)
```

or:

```
CALL ACCC_FILE_GET_1TRIPPING_SOLUTION(RFILE, GETWHAT, CNTPOS,
 BUS_MSM, SYS_MSM, ACCCSIZE, CNT_DESC, CNV_FLG, CNV_COND, NUM_ISLAND,
 CASE_WORST, CASE_TOTAL,  , MV_VMAG_OUTxx, ACFLOW_OUTxx, RAT_NUM_OUTxx,
 LSHED_BUS_OUTxx, LSHED_OUTxx, MVNOD_VMAGxx, IERR)
```

or:

```
CALL ACCC_FILE_GET_CACTION_SOLUTION(RFILE, GETWHAT, CNTPOS, BUS_MSM,
 SYS_MSM, ACCCSIZE, CNT_DESC, CNV_FLG, CNV_COND, NUM_ISLAND,
 CASE_WORST, CASE_TOTAL,  , MV_VMAG_OUTxx, ACFLOW_OUTxx, RAT_NUM_OUTxx,
 LSHED_BUS_OUTxx, LSHED_OUTxx, GDISPBUSxx, GDISPxx, PHSFTNAMExx,
 PHSFTANGxx, MVNOD_VMAGxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution loads GETWHAT = 2 Tripping solution loads GETWHAT = 3 Corrective action solution voltages and flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BUS_MSM	Is the bus mismatch value to use when reporting contingencies from the contingency file (Input).
Integer SYS_MSM	Is the system mismatch value to use when reporting contingencies from the contingency file (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows.

ACCCSIZE (1) Number of monitored lines.

ACCCSIZE (2) Number of interfaces.

ACCCSIZE (3) Number of Contingencies + Base Case.

ACCCSIZE (4) Number of voltage buses.

ACCCSIZE (5) Number of voltage records.

ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)).

ACCCSIZE (7) Number of buses in ACCC case.

ACCCSIZE (8) Acc file version number.

ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency.

ACCCSIZE (10) Max # of LOAD SHEDS due to tripping.

ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions.

ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions.

ACCCSIZE (14) Name Out option.

ACCCSIZE (14) = 0 Number.

ACCCSIZE (14) = 1 Name.

ACCCSIZE (14) = -1 Undecided.

ACCCSIZE (15) ACCC file type.

ACCCSIZE (15) = 0 ACCC.

ACCCSIZE (15) = 1 N-1-1.

ACCCSIZE (16) Number of Areas.

ACCCSIZE (17) Number of Zones.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Character CNT_DESC*160 (32)	Is the description of every contingency in the specified contingency group (Output).
Logical CNV_FLAG	Is a logical flag indicating whether the specified contingency converged or not (Output).
Character CNV_COND*34	Is the convergence solution text for the desired contingency (Output).
Integer NUM_ISLAND	Is the number of islanded buses in the desired contingency (Output).
Real CASE_WORST	Is the worst case in the desired contingency (Output).
Real CASE_TOTAL	Is the case total in the desired contingency (Output).
Real MV_VMAG_OUTxx * ACCCSIZE(4)	Is an array containing all the desired voltages for the desired contingency (output).
Real ACFLOW_OUTxx (ACCCSIZE(1)+ ACCCSIZE(2))	Is an array containing all the desired MVA flows for the desired contingency (output).
Real RAT_NUM_OUTxx (ACCCSIZE(1))	Is an array containing all the desired Ampere flows for the desired contingency (output).
Real LSHED_BUS_OUTxx (ACCCSIZE(11))	Is an array containing all the load shedding buses for the desired contingency (Output).
Real LSHED_OUTxx (2*ACCCSIZE(11))	Is an array containing all the load shed at the load shedding buses for the desired contingency (Output).
Real MVNOD_VMAGxx (ACCCSIZE(22))	Is an array containing the Monitored Node voltage results (Output).
Integer GDISPBUS (ACCCSIZE(11))	Is an array containing all the generation dispatch buses for the desired contingency (Output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real GDISPxx (ACCCSIZE(11))	Is an array containing all the generation dispatched at the generation dispatch buses for the desired contingency (Output).
Character PHSFTNAMExx*62 (ACCCSIZE(11))	Is an array containing all the Phase shifter names (Output).
Real PHSFTANGxx (ACCCSIZE(11))	Is an array containing all the Phase shifter angles (Output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 error reading RFILE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

30.18. ACCC_FILE_SOLUTION_2

This API is used to return voltages and flows for a specified contingency and contingency type for all monitored buses and lines in the contingency data file.

Python syntax:

```
ierr, cnt_desc, cnv_flg, cnv_cond, num_island, case_worst, case_total,
 mv_vmag_outxx, acflow_outxx, rat_num_outxx, lshed_bus_outxx,
 lshed_outxx, mvnod_vmagxx, acswdflow_outxx, swd_rat_num_outxx =
 accc_file_solution_2(rfile, getwhat, cntpos, bus_msm, sys_msm, acccsize)
```

or:

```
ierr, cnt_desc, cnv_flg, cnv_cond, num_island, case_worst, case_total,
 mv_vmag_outxx, acflow_outxx, rat_num_outxx, lshed_bus_outxx,
 lshed_outxx, gdispbusxx, gdispxx, phsftnamexx, pfsftangxx, mvnod_vmagxx,
 acswdflow_outxx, swd_rat_num_outxx = accc_file_solution_2(rfile, getwhat,
 cntpos, bus_msm, sys_msm, acccsize)
```

Fortran syntax:

```
CALL GET_ACCC_1CNTGNCY_SOLUTION_2(RFILE, GETWHAT, CNTPOS, BUS_MSM,
 SYS_MSM, ACCCSIZE, CNT_DESC, CNV_FLG, CNV_COND, NUM_ISLAND, CASE_WORST,
 CASE_TOTAL, MV_VMAG_OUTxx, ACFLOW_OUTxx, RAT_NUM_OUTxx, LSHED_BUS_OUTxx,
 LSHED_OUTxx, MVNOD_VMAGxx, ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx, IERR)
```

or:

```
CALL GET_ACCC_1TRIPPING_SOLUTION_2(RFILE, GETWHAT, CNTPOS, BUS_MSM,
 SYS_MSM, ACCCSIZE, CNT_DESC, CNV_FLG, CNV_COND, NUM_ISLAND, CASE_WORST,
 CASE_TOTAL, MV_VMAG_OUTxx, ACFLOW_OUTxx, RAT_NUM_OUTxx, LSHED_BUS_OUTxx,
 LSHED_OUTxx, MVNOD_VMAGxx, ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx, IERR)
```

or:

```
CALL GET_ACCC_1CACTION_SOLUTION_2(RFILE, GETWHAT, CNTPOS, BUS_MSM,
 SYS_MSM, ACCCSIZE, CNT_DESC, CNV_FLG, CNV_COND, NUM_ISLAND, CASE_WORST,
 CASE_TOTAL, MV_VMAG_OUTxx, ACFLOW_OUTxx, RAT_NUM_OUTxx, LSHED_BUS_OUTxx,
 LSHED_OUTxx, GDISPBUSxx, GDISPxx, PHSFTNAMExx, PFSFTANGxx, MVNOD_VMAGxx,
 ACSWDFLOW_OUTxx, SWD_RAT_NUM_OUTxx, IERR)
```

where:

Character RFILE*260	Is the Contingency Solution Output File (input; no default allowed).
Integer GETWHAT	Is the flows to return for the specified contingency (Input). The values are as follows. GETWHAT = 1 Contingency solution loads GETWHAT = 2 Tripping solution loads GETWHAT = 3 Corrective action solution voltages and flows
Long Integer CNTPOS	Is the contingency position in the Contingency Solution Output file from which to read the desired flows (Input).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Integer BUS_MSM	Is the bus mismatch value to use when reporting contingencies from the contingency file (Input).
Integer SYS_MSM	Is the system mismatch value to use when reporting contingencies from the contingency file (Input).
Integer ACCCSIZE(29)	Is an array containing the number of the various elements in the ACCC file (Input). The values are as follows.

ACCCSIZE (1) Number of monitored lines.

ACCCSIZE (2) Number of interfaces.

ACCCSIZE (3) Number of Contingencies + Base Case.

ACCCSIZE (4) Number of voltage buses.

ACCCSIZE (5) Number of voltage records.

ACCCSIZE (6) number monitored bus records (could be more than ACCCSIZE(4)).

ACCCSIZE (7) Number of buses in ACCC case.

ACCCSIZE (8) Acc file version number.

ACCCSIZE (9) Max # of LOAD SHEDS due to dispatch and contingency.

ACCCSIZE (10) Max # of LOAD SHEDS due to tripping.

ACCCSIZE (11) Max # of LOAD SHEDS due to corrective actions.

ACCCSIZE (12) Max # of GENERATION DISPACHED due to corrective actions.

ACCCSIZE (13) max # of PHASE SHIFTER changed due to corrective actions.

ACCCSIZE (14) Name Out option.

ACCCSIZE (14) = 0 Number.

ACCCSIZE (14) = 1 Name.

ACCCSIZE (14) = -1 Undecided.

ACCCSIZE (15) ACCC file type.

ACCCSIZE (15) = 0 ACCC.

ACCCSIZE (15) = 1 N-1-1.

ACCCSIZE (16) Number of Areas.

ACCCSIZE (17) Number of Zones.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

	ACCCSIZE (18) Number of Owners. ACCCSIZE (19) Number of Voltage levels. ACCCSIZE (20) Number of Ratings. ACCCSIZE (21) Number of Nodes in ACCC case. ACCCSIZE (22) Number of Monitored Nodes. ACCCSIZE (23) Number of Monitored Node records (Range and Deviation). ACCCSIZE (24) Number of Monitored Node records (Range, Deviation and Limit) (could be more than ACCCSIZE(22)). ACCCSIZE (25) Number of Monitored Limit Bus records. ACCCSIZE (26) Number of Monitored Limit Node records. ACCCSIZE (27) Number of Monitored SWD flows. ACCCSIZE (28) Number of Monitored SWD ratings. ACCCSIZE (29) Number of Contingency Categories.
Character CNT_DESC*160 (32)	Is the description of every contingency in the specified contingency group (Output).
Logical CNV_FLAG	Is a logical flag indicating whether the specified contingency converged or not (Output).
Character CNV_COND*34	Is the convergence solution text for the specified contingency (Output).
Integer NUM_ISLAND	Is the number of islanded buses in the specified contingency (Output).
Real CASE_WORST	Is the worst mismatch in the specified contingency (Output).
Real CASE_TOTAL	Is the mismatch total in the specified contingency (Output).
Real MV_VMAG_OUTxx * ACCCSIZE(4)	Is an array containing all the desired voltages for the specified contingency (output).
Real ACFLOW_OUTxx (ACCCSIZE(1)+ ACCCSIZE(2))	Is an array containing all the desired MVA flows for the specified contingency (output).
Real RAT_NUM_OUTxx (ACCCSIZE(1))	Is an array containing all the desired Ampere flows for the specified contingency (output).
Real LSHED_BUS_OUTxx (ACCCSIZE(11))	Is an array containing all the load shedding buses for the specified contingency (Output).
Real LSHED_OUTxx (2*ACCCSIZE(11))	Is an array containing all the load shed at the load shedding buses for the specified contingency (Output).
Real MVNOD_VMAGxx (ACCCSIZE(22))	Is an array containing the Monitored Node voltage results (Output).
Integer GDISPBUS (ACCCSIZE(11))	Is an array containing all the generation dispatch buses for the specified contingency (Output).

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Real GDISPxx (ACCCSIZE(11))	Is an array containing all the generation dispatched at the generation dispatch buses for the specified contingency (Output).
Character PHSFTNAMExx*62 (ACCCSIZE(11))	Is an array containing all the Phase shifter names (Output).
Real PHSFTANGxx (ACCCSIZE(11))	Is an array containing all the Phase shifter angles (Output).
Real ACSWDFLOW_OUTxx (ACCCSIZE(27))	Is an array containing all the desired MVA flows for the monitored SWD (output).
Real SWD_RAT_NUM_OUTxx (ACCCSIZE(27))	Is an array containing all the desired Ampere flows for the monitored SWD (output).
Integer IERR	Is the error code (output). IERR = 0 no error occurred. IERR = 4 error opening RFILE. IERR = 5 error reading RFILE. IERR = 6 error in fetching contingency strings. IERR = 10 prerequisite requirements for API are not met.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Chapter 31
Other Python Modules

This chapter identifies other Python modules that are delivered as part of the PSS®E API.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

31.1. Modules Supplied with a GUI

These modules can be imported or run by themselves. They display a GUI that presents their functionality.

createusrdllw – Create a user dll for dynamics simulation. Combines the traditional compile and cload4 processes in a more user-friendly way.

excelexportw – Provides tools to export psse results to Excel, e.g., pv, qv, and acc files.

plot2wordw – Provides tools to create Word documents from plots.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

31.2. Modules that Define Functions

Each of these modules contains help information about the functions that they define. Import the module and run the help function on them for more information.

bsntools – Bus renaming tools. Functions to modify bus names by area. Requires psspy.

caspy – Create an object to directly returns arrays of data from a saved case file. Encapsulates the functionality of the Saved Case Data Extraction routines (USRCAS).

dyntools – Tools for processing channel output files, including export to Excel.

pssgrpg – Provides functions intended to be used to supply values to report labels on 1-line diagrams.

pssppe – Convert csv file to a dictionary of tuples.

pssras – Define a Remedial Action Scheme. Used with the Powerflow Customization Interface.

redirect – Tools for directing PSS®E output to Python, and vice versa.

sliderPy – Provides functions to directly manipulate diagram elements.

wordpy – Provides Python functions to interface with Microsoft Word.

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

Index

A

A2TRMDCCHAR, 3005
A2TRMDCCONVCHAR, 3017
A2TRMDCCONVCOUNT, 3020
A2TRMDCCONVCPLX, 3022
A2TRMDCCONVINT, 3024
A2TRMDCCONVREAL, 3026
A2TRMDCCONVTYPES, 3029
A2TRMDCCOUNT, 3007
A2TRMDCCPLX, 3008
A2TRMDCINT, 3010
A2TRMDCREAL, 3012
A2TRMDCTYPES, 3016
AAREACHAR, 3103
AAREACOUNT, 3105
AAREACPLX, 3106
AAREAINT, 3108
AAREAREAL, 3110
AAREATYPES, 3113
ABRNCHAR, 2933
ABRNCOUNT, 2936
ABRNCPLX, 2938
ABRNINT, 2940
ABRNREAL, 2943
ABRNTYPES, 2951
ABUSCHAR, 2821
ABUSCOUNT, 2823
ABUSCPLX, 2824
ABUSINT, 2826
ABUSREAL, 2828
ABUSTYPES, 2830
ACCC, 2
ACCC_2, 4
ACCC_MULTIPLE_MERGE, 6
ACCC_MULTIPLE_RUN_REPORT, 8
ACCC_MULTIPLE_RUN_REPORT_2, 11
ACCC_PARALLEL, 15
ACCC_PARALLEL_2, 18
ACCC_RAS, 3345
ACCC_RAS_2, 3346
ACCC_SINGLE_RUN_REPORT, 21
ACCC_SINGLE_RUN_REPORT_2, 24
ACCC_SINGLE_RUN_REPORT_3, 27
ACCC_SINGLE_RUN_REPORT_4, 30
ACCC_SINGLE_RUN_REPORT_5, 33
ACCC_SINGLE_RUN_REPORT_6, 37
ACCC_TRIP_COR, 41
ACCC_TRIP_COR_2, 45
ACCC_TRIP_COR_3, 49
ACCC_WITH_COR, 54
ACCC_WITH_COR_2, 58
ACCC_WITH_COR_3, 62
ACCC_WITH_DSP, 66
ACCC_WITH_DSP_2, 69
ACCC_WITH_DSP_3, 72
ACCC_WITH_TRIP, 75
ACCC_WITH_TRIP_2, 78
ACCC_WITH_TRIP_PARALLEL, 81
ACCDEFAULTS, 3539
ACCONTBYCONT, 3541
ACCONTBYMONEL, 3543
ACCOR, 84
ACCOR_2, 87
ACCOR_3, 90
ACCSOLVE, 3545
ACLFDC, 3547
ACLFNS, 3549
ACSOLVE_LFD, 3551
ACSOLVE_LFN, 3552
ACTIVITYACCONT, 3553
ACTIVITYMUST, 3554
ACTRANSFERLIMITSTUDY, 3555
ADD_CCT2DCO_MODEL, 1934
ADD_CCT2WTD_MODEL, 1936
ADD_CCT3WTD_MODEL, 1938
ADD_CCTBRND_MODEL, 1940
ADD_CCTBRNO_MODEL, 1942
ADD_CCTBUSO_MODEL, 1944
ADD_CCTMCNO_MODEL, 1945
ADD_CCTMCNP_MODEL, 1947
ADD_CCTMSCO_MODEL, 1949
ADD_CCTSWSO_MODEL, 1950
ADD_CCTSWSO_MODEL_2, 1952
ADD_DETAILS_TO_OPF_LOG, 1104
ADD_FACTS_DEVICE_MODEL, 1885
ADD_FCTSAUXSIGNAL_MODEL, 2032
ADD_INDMAC_MODEL, 1923
ADD_LOAD_MODEL, 1807
ADD_LOADC_MODEL, 1828
ADD_MTDCAUXSIGNAL_MODEL, 2034
ADD_MTDCLINE_MODEL, 1853
ADD_PLANT_MODEL, 1779
ADD_RELAY_MODEL, 1840
ADD_SWSHUNT_MODEL, 1900
ADD_SWSHUNT_MODEL_2, 1902
ADD_TTDCAUXSIGNAL_MODEL, 2036
ADD_TTDCLINE_MODEL, 1855
ADD_VSDCAUXSIGNAL_MODEL, 2038
ADD_VSDCLINE_MODEL, 1857
ADD_WIND_MODEL, 1795
ADD2WINDINGCONDITIONELEMENT, 3348
ADD2WINDINGCONTINGENCYELEMENT, 3350
ADD2WINDINGREMEDIALACTIONELEMENT, 3353
ADD3WINDINGCONDITIONELEMENT, 3356
ADD3WINDINGCONTINGENCYELEMENT, 3358
ADD3WINDINGREMEDIALACTIONELEMENT, 3361
ADDACLINECONDITIONELEMENT, 3365
ADDACLINECONTINGENCYELEMENT, 3367
ADDACLINEREMEDIALACTIONELEMENT, 3370
ADDBUSCONDITIONELEMENT, 3373
ADDBUSCONTINGENCYELEMENT, 3375
ADDBUSREMEDIALACTIONELEMENT, 3377
ADDCONDITION, 3380
ADDCONDITIONELEMENT, 3382
ADDCONTINGENCY, 3384
ADDCONTINGENCYELEMENT, 3385
ADDDCCONVCONDITIONELEMENT, 3387
ADDDCCONVCONTINGENCYELEMENT, 3389
ADDDCCONVREMEDIALACTIONELEMENT, 3391
ADDDCLINECONDITIONELEMENT, 3394
ADDDCLINECONTINGENCYELEMENT, 3396
ADDDCLINEREMEDIALACTIONELEMENT, 3398
ADDFIXSHUNTCONDITIONELEMENT, 3401
ADDFIXSHUNTCONTINGENCYELEMENT, 3403
ADDFIXSHUNTREMEDIALACTIONELEMENT, 3405
ADDFUNCTION, 3492
ADDGENCONDITIONELEMENT, 3408
ADDGENCONTINGENCYELEMENT, 3410
ADDGENREMEDIALACTIONELEMENT, 3412
ADDLOADCONDITIONELEMENT, 3415
ADDLOADCONTINGENCYELEMENT, 3417
ADDLOADREMEDIALACTIONELEMENT, 3419
ADDMODELLIBRARY, 1634
ADDNESTEDCONDITION, 3422
ADDPYTHONCONDITIONELEMENT, 3423
ADDPYTHONCONTINGENCYELEMENT, 3425
ADDPYTHONREMEDIALACTIONELEMENT, 3427
ADDREMEDIALACTION, 3429
ADDREMEDIALACTIONELEMENT, 3430
ADDSUBCONDITIONELEMENT, 3433
ADDSUBCONTINGENCYELEMENT, 3435
ADDSUBREMEDIALACTIONELEMENT, 3437
ADDTRANSACTION, 3556
ADDVSCDCCONDITIONELEMENT, 3440
ADDVSCDCCONTINGENCYELEMENT, 3442
ADDVSCDCREMEDIALACTIONELEMENT, 3444

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

AFACTSBUSCHAR, 3089
AFACTSBUSCOUNT, 3092
AFACTSBUSCPLX, 3094
AFACTSBUSINT, 3096
AFACTSBUSREAL, 3099
AFACTSBUSTYPES, 3102
AFACTSCHAR, 3076
AFACTSCOUNT, 3078
AFACTSCPLX, 3080
AFACTSINT, 3082
AFACTSREAL, 3084
AFACTSTYPES, 3088
AFLOWCHAR, 2920
AFLOWCOUNT, 2923
AFLOWCPLX, 2925
AFLOWINT, 2927
AFLOWREAL, 2929
AFLOWTYPES, 2932
AFXSHNTBUSCHAR, 2878
AFXSHNTBUSCOUNT, 2880
AFXSHNTBUSCPLX, 2881
AFXSHNTBUSINT, 2883
AFXSHNTBUSREAL, 2885
AFXSHNTBUSTYPES, 2887
AFXSHUNTCHAR, 2888
AFXSHUNTCOUNT, 2890
AFXSHUNTCPLX, 2891
AFXSHUNTINT, 2893
AFXSHUNTREAL, 2895
AFXSHUNTTYPES, 2897
AGCAPCOUNT, 3314
AGCAPREAL, 3315
AGENBUSCHAR, 2831
AGENBUSCOUNT, 2833
AGENBUSCPLX, 2834
AGENBUSINT, 2836
AGENBUSREAL, 2838
AGENBUSTYPES, 2841
AINDMACBUSCHAR, 3136
AINDMACBUSCOUNT, 3138
AINDMACBUSCPLX, 3140
AINDMACBUSINT, 3142
AINDMACBUSREAL, 3144
AINDMACBUSTYPES, 3147
AINDMACCHAR, 3148
AINDMACCOUNT, 3150
AINDMACCPLX, 3152
AINDMACINT, 3154
AINDMACREAL, 3157
AINDMACTYPES, 3160
AINTERFACECHAR, 3316
AINTERFACECOUNT, 3317
AINTERFACEELEMENTCHAR, 3318
AINTERFACEELEMENTCOUNT, 3319
AINTERFACEELEMENTCPLX, 3320
AINTERFACEELEMENTINT, 3321
AINTERFACEELEMENTREAL, 3322
AINTERFACEINT, 3323
AINTERFACEREAL, 3324
ALERT, 3453
ALERT_OUTPUT, 93
ALLOW_PSSUSERPF, 95
ALOADCHAR, 2866
ALOADCOUNT, 2868
ALOADCPLX, 2869
ALOADINT, 2872
ALOADREAL, 2874
ALOADTYPES, 2877
ALODBUSCHAR, 2853
ALODBUSCOUNT, 2855
ALODBUSCPLX, 2857
ALODBUSINT, 2860
ALODBUSREAL, 2862
ALODBUSTYPES, 2865
ALPH, 96
AMACHCHAR, 2842
AMACHCOUNT, 2844
AMACHCPLX, 2845
AMACHINT, 2847
AMACHREAL, 2849
AMACHTYPES, 2852
AMULTITRMDCCHAR, 3030
AMULTITRMDCCONVCHAR, 3040
AMULTITRMDCCONVCOUNT, 3043
AMULTITRMDCCONVCPLX, 3045
AMULTITRMDCCONVINT, 3047
AMULTITRMDCCONVREAL, 3049
AMULTITRMDCCONVTYPES, 3052
AMULTITRMDCCOUNT, 3032
AMULTITRMDCCPLX, 3033
AMULTITRMDCINT, 3035
AMULTITRMDCREAL, 3037
AMULTITRMDCTYPES, 3039
ANODECHAR, 3168
ANODECOUNT, 3170
ANODEINT, 3171
ANODEREAL, 3173
ANODETYPES, 3174
ANSI, 1285
ANSI_2, 1287
ANSI_3, 1289
AOWNERCHAR, 3114
AOWNERCOUNT, 3116
AOWNERCPLX, 3117
AOWNERINT, 3119
AOWNERREAL, 3121
AOWNERTYPES, 3124
APIERRSTR, 2286
APPEND_ACCC, 97
APPEND_RAS, 3447
APPENDRECORDING, 3454
APPLY_ALTERNATE_STEP_SIZE, 1105
APPLY_AUTOMATIC_SCALING, 1106
APPLY_VAR_LIMITS, 99
APPLYSINGLECONT, 3557
ARDAT, 2287
AREA, 100
AREA_2, 101
AREA_DATA, 674
AREA_ZONE, 102
AREABUBBLE, 3558
AREAIMPACT, 3559
AREASINZONES, 3561
AREASLACKS, 3562
AREASUMMARY, 3563
AREDAT, 2289
AREINT, 2290
ARENAM, 2291
ARENUM, 2292
AREUSE, 2293
ARITOJ, 2294
ARNM, 103
ARNM_2, 105
ASCC, 1292
ASCC_1A, 1294
ASCC_2, 1296
ASCC_3, 1298
ASCC_SCFILE, 1302
ASTASWDEVCHAR, 3175
ASTASWDEVCOUNT, 3177
ASTASWDEVCPLX, 3178
ASTASWDEVINT, 3180
ASTASWDEVREAL, 3182
ASTASWDEVTYPES, 3185
ASTATIONCHAR, 3161
ASTATIONCOUNT, 3163
ASTATIONINT, 3164
ASTATIONREAL, 3166
ASTATIONTYPES, 3167
ASTR, 1635
ASWSHCHAR, 2908
ASWSHCOUNT, 2910

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ASWSHCPLX, 2911
ASWSHINT, 2913
ASWSHREAL, 2915
ASWSHTYPES, 2919
ASWSHUNTBUSCHAR, 2898
ASWSHUNTBUSCOUNT, 2900
ASWSHUNTBUSCPLX, 2901
ASWSHUNTBUSINT, 2903
ASWSHUNTBUSREAL, 2905
ASWSHUNTBUSTYPES, 2907
ASYS, 2128
ASYSDEF, 2129
ASYSINIT, 2130
ATERMINALCHAR, 3186
ATERMINALCOUNT, 3188
ATERMINALINT, 3189
ATERMINALTYPES, 3191
ATR3CHAR, 2972
ATR3COUNT, 2975
ATR3CPLX, 2977
ATR3INT, 2980
ATR3REAL, 2983
ATR3TYPES, 2985
ATRNCHAR, 2952
ATRNCOUNT, 2955
ATRNCPLX, 2957
ATRNINT, 2960
ATRNREAL, 2963
ATRNTYPES, 2971
AULIST, 1638
AUXMOD_PACK, 2040
AUXMOD_UNCONNECTED, 2041
AUXMOD_USER, 2042
AUXOUT, 4275
AVOLTAGEDROOPCHAR, 3326
AVOLTAGEDROOPCOUNT, 3327
AVOLTAGEDROOPINT, 3328
AVOLTAGEDROOPREAL, 3329
AVSCDCCHAR, 3053
AVSCDCCONVCHAR, 3063
AVSCDCCONVCOUNT, 3066
AVSCDCCONVCPLX, 3068
AVSCDCCONVINT, 3070
AVSCDCCONVREAL, 3072
AVSCDCCONVTYPES, 3075
AVSCDCCOUNT, 3055
AVSCDCCPLX, 3056
AVSCDCINT, 3058
AVSCDCREAL, 3060
AVSCDCTYPES, 3062
AWNDCHAR, 2986
AWNDCOUNT, 2989
AWNDCPLX, 2991
AWNDINT, 2993
AWNDREAL, 2996
AWNDTYPES, 3004
AZMUTCHAR, 3192
AZMUTCOUNT, 3194
AZMUTCPLX, 3196
AZMUTINT, 3198
AZMUTREAL, 3200
AZMUTTYPES, 3202
AZONECHAR, 3125
AZONECOUNT, 3127
AZONECPLX, 3128
AZONEINT, 3130
AZONEREAL, 3132
AZONETYPES, 3135

B

BAD_ITER_COARSE_LIMIT, 1107
BAD_ITER_FINE_LIMIT, 1108
BASE_FREQUENCY, 107
BEGINREPORT, 3455
BEGINREPORT_2, 3456
BGEN, 108
BKDY, 1303
BLSTAT, 4276
BRANCH_APP_R_X_CHANNEL, 1757
BRANCH_CHNG, 676
BRANCH_CHNG_3, 678
BRANCH_CHNG_4, 681
BRANCH_DATA, 684
BRANCH_DATA_3, 686
BRANCH_DATA_4, 689
BRANCH_MVA_CHANNEL, 1759
BRANCH_P_AND_Q_CHANNEL, 1761
BRANCH_P_CHANNEL, 1763
BRANCHSENSITIVITY, 3564
BRCH, 109
BRCH_2, 111
BRN_RATING_DEFAULT, 113
BRN_RATING_NAME_DATA, 2308
BRN_RATING_NAME_SET, 692
BRNCUR, 2295
BRNDAT, 2296
BRNDT2, 2298
BRNFLO, 2300
BRNINT, 2301
BRNMSC, 2303
BRNMVA, 2305
BRNNAM, 2306
BRNSTT, 2307
BSDSCN, 4277
BSDSCN1, 4278
BSNM, 114
BSSQNC, 4279
BSYS, 2131
BSYSADD, 2133
BSYSDEF, 2135
BSYSDELO, 2136
BSYSINIT, 2137
BSYSISDEF, 2309
BSYSMEM, 2138
BSYSO, 2139
BSYSRCL, 2140
BUBBLEREPORT, 3566
BUS_CHNG_3, 693
BUS_CHNG_4, 695
BUS_DATA, 697
BUS_DATA_2, 699
BUS_DATA_3, 701
BUS_DATA_4, 703
BUS_ENCODE_TRIM, 4280
BUS_FREQUENCY_CHANNEL, 1773
BUS_INPUT, 118
BUS_NUMBER, 705
BUS_OUTPUT, 119
BUS_SIZE_LEVEL, 120
BUSDAT, 2310
BUSDT1, 2311
BUSDT2, 2313
BUSEXS, 2315
BUSINT, 2316
BUSMSM, 2317
BUSN, 117
BUSORDPOS, 2318
BUSSECTDAT, 2319
BUSSECTDT1, 2320
BUSSECTDT2, 2322
BUSSECTEXS, 2324
BUSSECTINT, 2325

C

CA_ITERATIONS, 122
CALCULATE_VFE, 4281
CASE, 121
CASE_TITLE_DATA, 706
CASOLUTION_PARAMETERS, 707
CASOLUTION_PARAMETERS_2, 708

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CASOLUTION_PARAMETERS_3, 710
CCT2DCO_LIST, 1956
CCT2DCOMOD_REMOVE, 1954
CCT2DCOMOD_STATUS, 1955
CCT2WTD_LIST, 1959
CCT2WTDMOD_REMOVE, 1957
CCT2WTDMOD_STATUS, 1958
CCT3WTD_LIST, 1962
CCT3WTDMOD_REMOVE, 1960
CCT3WTDMOD_STATUS, 1961
CCTBRND_LIST, 1965
CCTBRNDMOD_REMOVE, 1963
CCTBRNDMOD_STATUS, 1964
CCTBRNO_LIST, 1968
CCTBRNOMOD_REMOVE, 1966
CCTBRNOMOD_STATUS, 1967
CCTBUSO_LIST, 1971
CCTBUSOMOD_REMOVE, 1969
CCTBUSOMOD_STATUS, 1970
CCTMCNO_LIST, 1974
CCTMCNOMOD_REMOVE, 1972
CCTMCNOMOD_STATUS, 1973
CCTMCNP_LIST, 1977
CCTMCNPMOD_REMOVE, 1975
CCTMCNPMOD_STATUS, 1976
CCTMDLNAM_2DCO, 2326
CCTMDLNAM_2WTD, 2327
CCTMDLNAM_3WTD, 2328
CCTMDLNAM_BRND, 2329
CCTMDLNAM_BRNO, 2330
CCTMDLNAM_BUSO, 2331
CCTMDLNAM_MCNO, 2332
CCTMDLNAM_MCNP, 2333
CCTMDLNAM_MSCO, 2334
CCTMDLNAM_SWSO_2, 2335
CCTMIND_2DCO, 2336
CCTMIND_2WTD, 2337
CCTMIND_3WTD, 2339
CCTMIND_BRND, 2341
CCTMIND_BRNO, 2343
CCTMIND_BUSO, 2345
CCTMIND_MCNO, 2346
CCTMIND_MCNP, 2347
CCTMIND_MSCO, 2348
CCTMIND_SWSO, 2349
CCTMIND_SWSO_2, 2351
CCTMOD_PACK, 1978
CCTMOD_USER, 1979
CCTMSCO_LIST, 1982
CCTMSCOMOD_REMOVE, 1980
CCTMSCOMOD_STATUS, 1981
CCTSWSO_LIST, 1987
CCTSWSOMOD_REMOVE, 1983
CCTSWSOMOD_REMOVE_2, 1984
CCTSWSOMOD_STATUS, 1985
CCTSWSOMOD_STATUS_2, 1986
CDETAILBRANCHES, 3567
CDETAILINTERFACES, 3568
CDETAILVOLTAGEBUSES, 3569
CHANGE_CCT2DCOMOD_CHRICN, 1988
CHANGE_CCT2DCOMOD_CON, 1989
CHANGE_CCT2DCOMOD_ICON, 1990
CHANGE_CCT2DCOMOD_VAR, 1991
CHANGE_CCT2WTDMOD_CHRICN, 1992
CHANGE_CCT2WTDMOD_CON, 1993
CHANGE_CCT2WTDMOD_ICON, 1994
CHANGE_CCT2WTDMOD_VAR, 1995
CHANGE_CCT3WTDMOD_CHRICN, 1996
CHANGE_CCT3WTDMOD_CON, 1997
CHANGE_CCT3WTDMOD_ICON, 1998
CHANGE_CCT3WTDMOD_VAR, 1999
CHANGE_CCTBRNDMOD_CHRICN, 2000
CHANGE_CCTBRNDMOD_CON, 2001
CHANGE_CCTBRNDMOD_ICON, 2002
CHANGE_CCTBRNDMOD_VAR, 2003
CHANGE_CCTBRNOMOD_CHRICN, 2004
CHANGE_CCTBRNOMOD_CON, 2005
CHANGE_CCTBRNOMOD_ICON, 2006
CHANGE_CCTBRNOMOD_VAR, 2007
CHANGE_CCTBUSOMOD_CHRICN, 2008
CHANGE_CCTBUSOMOD_CON, 2009
CHANGE_CCTBUSOMOD_ICON, 2010
CHANGE_CCTBUSOMOD_VAR, 2011
CHANGE_CCTMCNOMOD_CHRICN, 2012
CHANGE_CCTMCNOMOD_CON, 2013
CHANGE_CCTMCNOMOD_ICON, 2014
CHANGE_CCTMCNOMOD_VAR, 2015
CHANGE_CCTMCNPMOD_CHRICN, 2016
CHANGE_CCTMCNPMOD_CON, 2017
CHANGE_CCTMCNPMOD_ICON, 2018
CHANGE_CCTMCNPMOD_VAR, 2019
CHANGE_CCTMSCOMOD_CHRICN, 2020
CHANGE_CCTMSCOMOD_CON, 2021
CHANGE_CCTMSCOMOD_ICON, 2022
CHANGE_CCTMSCOMOD_VAR, 2023
CHANGE_CCTSWSOMOD_CHRICN, 2024
CHANGE_CCTSWSOMOD_CHRICN_2, 2025
CHANGE_CCTSWSOMOD_CON, 2026
CHANGE_CCTSWSOMOD_CON_2, 2027
CHANGE_CCTSWSOMOD_ICON, 2028
CHANGE_CCTSWSOMOD_ICON_2, 2029
CHANGE_CCTSWSOMOD_VAR, 2030
CHANGE_CCTSWSOMOD_VAR_2, 2031
CHANGE_CHANNEL, 1740
CHANGE_CHANNEL_OUT_FILE, 1639
CHANGE_CHRICN, 1640
CHANGE_CON, 1641
CHANGE_FCTMOD_CHRICN, 1887
CHANGE_FCTMOD_CON, 1888
CHANGE_FCTMOD_DATA, 1889
CHANGE_FCTMOD_ICON, 1891
CHANGE_FCTMOD_VAR, 1892
CHANGE_FCTSAUXMOD_CHRICN, 2043
CHANGE_FCTSAUXMOD_CON, 2045
CHANGE_FCTSAUXMOD_DATA, 2047
CHANGE_FCTSAUXMOD_ICON, 2049
CHANGE_FCTSAUXMOD_VAR, 2051
CHANGE_GREF, 1642
CHANGE_ICON, 1643
CHANGE_IMMOD_CHRICN, 1925
CHANGE_IMMOD_CON, 1926
CHANGE_IMMOD_ICON, 1927
CHANGE_IMMOD_VAR, 1928
CHANGE_LDMOD_CHRICN, 1809
CHANGE_LDMOD_CON, 1810
CHANGE_LDMOD_DATA, 1811
CHANGE_LDMOD_ICON, 1813
CHANGE_LDMOD_VAR, 1814
CHANGE_LDMODC_CDESC, 1830
CHANGE_LDMODC_CHRICN, 1832
CHANGE_LDMODC_CON, 1834
CHANGE_LDMODC_ICON, 1836
CHANGE_LDMODC_VAR, 1838
CHANGE_MTDCAUXMOD_CHRICN, 2053
CHANGE_MTDCAUXMOD_CON, 2055
CHANGE_MTDCAUXMOD_DATA, 2057
CHANGE_MTDCAUXMOD_ICON, 2059
CHANGE_MTDCAUXMOD_VAR, 2061
CHANGE_MTDCLMOD_CHRICN, 1858
CHANGE_MTDCLMOD_CON, 1859
CHANGE_MTDCLMOD_DATA, 1860
CHANGE_MTDCLMOD_ICON, 1862
CHANGE_MTDCLMOD_VAR, 1863
CHANGE_PLMOD_CHRICN, 1781
CHANGE_PLMOD_CON, 1782
CHANGE_PLMOD_DATA, 1783
CHANGE_PLMOD_ICON, 1785
CHANGE_PLMOD_VAR, 1786
CHANGE_PREF, 1644
CHANGE_QVREF, 1645
CHANGE_RLMOD_CHRICN, 1842
CHANGE_RLMOD_CON, 1843

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CHANGE_RLMOD_DATA, 1844
CHANGE_RLMOD_ICON, 1846
CHANGE_RLMOD_VAR, 1847
CHANGE_STATE, 1646
CHANGE_SWSMOD_CHRICN, 1904
CHANGE_SWSMOD_CHRICN_2, 1905
CHANGE_SWSMOD_CON, 1906
CHANGE_SWSMOD_CON_2, 1907
CHANGE_SWSMOD_ICON, 1908
CHANGE_SWSMOD_ICON_2, 1909
CHANGE_SWSMOD_VAR, 1910
CHANGE_SWSMOD_VAR_2, 1911
CHANGE_SWSREF, 1647
CHANGE_SWSREF_2, 1648
CHANGE_TTDCAUXMOD_CHRICN, 2063
CHANGE_TTDCAUXMOD_CON, 2065
CHANGE_TTDCAUXMOD_DATA, 2067
CHANGE_TTDCAUXMOD_ICON, 2069
CHANGE_TTDCAUXMOD_VAR, 2071
CHANGE_TTDCLMOD_CHRICN, 1864
CHANGE_TTDCLMOD_CON, 1865
CHANGE_TTDCLMOD_DATA, 1866
CHANGE_TTDCLMOD_ICON, 1868
CHANGE_TTDCLMOD_VAR, 1869
CHANGE_VAR, 1649
CHANGE_VREF, 1650
CHANGE_VSDCAUXMOD_CHRICN, 2073
CHANGE_VSDCAUXMOD_CON, 2074
CHANGE_VSDCAUXMOD_DATA, 2075
CHANGE_VSDCAUXMOD_ICON, 2077
CHANGE_VSDCAUXMOD_VAR, 2078
CHANGE_VSDCLMOD_CHRICN, 1870
CHANGE_VSDCLMOD_CON, 1871
CHANGE_VSDCLMOD_DATA, 1872
CHANGE_VSDCLMOD_ICON, 1874
CHANGE_VSDCLMOD_VAR, 1875
CHANGE_WNMOD_CHRICN, 1797
CHANGE_WNMOD_CON, 1798
CHANGE_WNMOD_ICON, 1799
CHANGE_WNMOD_VAR, 1800
CHANGEDIAGCONTOUR, 2149
CHANGEPARAMS, 3570
CHANNELFILEEXCELEXPORT, 3493
CHECK_POWERFLOW_DATA, 124
CHECK_SEQUENCE_DATA, 1305
CHECKDIAGFILE, 2151
CHECKVOLTAGELIMITS, 123
CHKCNTDUPLICON, 126
CHKCNTDUPLIDFX, 127
CHKTRE, 2353
CHNVAL, 2354
CHSB, 1741
CKTIDS, 4282
CLAMP_NONOPTIMIZED_GENS, 1109
CLEARALERTOUTPUT, 3457
CLEARMULTISTUDY, 3577
CLEARPROGRESSOUTPUT, 3458
CLOSE_POWERFLOW, 128
CLOSE_REPORT, 129
CLOSEDIAGFILE, 2152
CLOSEEVENTSTUDYFILE, 3306
CLOSESCENARIOFILE, 3215
CMPR, 130
CNALLTRANSELBRANCHES, 3578
CNALLTRANSELFLOWGATES, 3579
CNALLTRANSELINTERFACES, 3580
CNALLVIOLATED, 3581
CNALLVIOLFREQ, 3582
CNBASEVIOLATED, 3583
CNCINITSELBRANCHES, 3584
CNCINITSELINTERFACES, 3585
CNCOMPSELBRANCHES, 3586
CNCOMPSELINTERFACES, 3587
CNCOMPSELVOLTAGEBUSES, 3588
CNCRESULTSSELBRANCHES, 3589
CNCRESULTSSELINTERFACES, 3590
CNDETAILSELFLOWGATES, 3591
CNDETAILSELINTERFACES, 3592
CNFACTORSSELFLOWGATES, 3593
CNFREQSELBRANCHES, 3594
CNFREQSELINTERFACES, 3595
CNFREQSELVOLTAGEBUSES, 3596
CNGAPPSELBRANCHES, 3597
CNGAPPSELFLOWGATES, 3598
CNGAPPSELINTERFACES, 3599
CNIMPACTSELBRANCHES, 3600
CNIMPACTSELFLOWGATES, 3601
CNIMPACTSELINTERFACES, 3602
CNLISTSELBRANCHES, 3603
CNLISTSELFLOWGATES, 3604
CNLISTSELINTERFACES, 3605
CNLISTSELVOLTAGEBUSES, 3606
CNOTDFSELBRANCHES, 3607
CNOTDFSELINTERFACES, 3608
CNSELECTBRANCHBYALL, 3609
CNSELECTBRANCHBYAREAS, 3610
CNSELECTBRANCHBYBRANCHES, 3611
CNSELECTBRANCHBYBUSES, 3612
CNSELECTBRANCHBYIMPACT, 3613
CNSELECTBRANCHBYTIES, 3614
CNSELECTBRANCHBYVIOL, 3615
CNSELECTBRANCHBYZONES, 3616
CNSELECTFGATEBYALL, 3617
CNSELECTFGATEBYCONT, 3618
CNSELECTFGATEBYID, 3619
CNSELECTFGATEBYIMPACT, 3620
CNSELECTFGATEBYVIOL, 3621
CNSELECTINTERBYALL, 3622
CNSELECTINTERBYIMPACT, 3623
CNSELECTINTERBYNAME, 3624
CNSELECTINTERBYNUMBER, 3625
CNSELECTINTERBYVIOL, 3626
CNSELECTVOLTAGEBUSESBYVIOL, 3627
CNSELECTVOLTBUSBYALL, 3628
CNSELECTVOLTBUSBYAREAS, 3629
CNSELECTVOLTBUSBYBUSES, 3630
CNSELECTVOLTBUSBYZONES, 3631
CNSELECTVOLTBUSESBYDEVIATION, 3632
CNSENSUBSELBRANCHES, 3633
CNSENSUBSELFLOWGATES, 3634
CNSENSUBSELINTERFACES, 3635
CNSTCR, 4283
CNSTPQ, 4284
CNSTUDYSELBRANCHES, 3636
CNSTUDYSELFLOWGATES, 3637
CNSTUDYSELINTERFACES, 3638
CNSUBSELBRANCHES, 3639
CNSUBSELFLOWGATES, 3640
CNSUBSELINTERFACES, 3641
CNSUMMARYVIOLATIONS, 3642
CNTB, 132
CNTRANSACTSELBRANCHES, 3643
CNTRANSACTSELFLOWGATES, 3644
CNTRANSACTSELINTERFACES, 3645
CNVARSELBRANCHES, 3646
CNVARSELINTERFACES, 3647
CNVARSELVOLTAGEBUSES, 3648
CNVIOLSELBRANCHES, 3649
CNVIOLSELFLOWGATES, 3650
CNVIOLSELINTERFACES, 3651
CNVIOLSELVOLTAGEBUSES, 3652
COMPAREFCITC, 3653
CONG, 134
CONL, 135
CONNECTIVITY_CHECK, 137
CONSTRAIN_INTERFACE_FLOWS, 1110
CONSTRDEFS, 3654
CONTROL_AREA_INTERCHANGE, 138
CONVERT_SECTION_TO_BUS, 1021
CRCTRUN, 1651
CRT_PLOT_CHANNEL, 1744
CSV_TO_RAWX, 139

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

CSV_TO_RAWX_WITH_METAMODEL, 140

D

DC_TAP_ADJUSTMENT, 157
DC2AUXMIND, 2355
DC2AUXMNAM, 2357
DC2DAT, 2358
DC2DAT_2, 2360
DC2INT, 2362
DC2INT_2, 2363
DC2MIND, 2364
DC2MNAM, 2366
DCCC, 141
DCCC_2, 143
DCCONTBYCONT, 3656
DCCONTBYMONEL, 3658
DCCOR, 145
DCCOR_2, 147
DCLF, 150
DCLF_2, 152
DCLIST, 1653
DCLMOD_PACK, 1876
DCLMOD_UNCONNECTED, 1877
DCLMOD_USER, 1878
DCNAUXMIND, 2367
DCNAUXMNAM, 2369
DCNCIN, 2370
DCNCIN_2, 2371
DCNDAT, 2372
DCNDAT_2, 2374
DCNINT, 2376
DCNINT_2, 2377
DCNMIND, 2378
DCNMNAM, 2380
DCPSCOPF, 154
DCSOLVE, 3660
DCTRANSFERLIMITSTUDY, 3661
DEADBD2, 4285
DEADBN2, 4286
DELAYU, 4287
DELETE_ALL_PLOT_CHANNELS, 1745
DELETEDIAGCONTOUR, 2153
DELETESELPLOT, 3494
DELTMPFILES, 158
DETAILSELCONTINGENCIES, 3662
DETAILSELCONTINGENCIESSP, 3663
DFAX, 159
DFAX_2, 161
DFAX_CONTINGENCY, 163
DFTI, 164
DIFF, 167
DIRECT, 3664
DIST_3PHASE_BUS_FAULT, 2087
DIST_3WIND_CLOSE, 2089
DIST_3WIND_FAULT, 2090
DIST_3WIND_TRIP, 2092
DIST_BRANCH_CLOSE, 2093
DIST_BRANCH_FAULT, 2094
DIST_BRANCH_TRIP, 2096
DIST_BUS_FAULT, 2097
DIST_BUS_FAULT_2, 2098
DIST_BUS_FAULT_3, 2100
DIST_BUS_TRIP, 2102
DIST_CLEAR_FAULT, 2103
DIST_DEF_3PHASE_BUS_FAULT, 2116
DIST_DEF_3WIND_FAULT, 2118
DIST_DEF_BRANCH_FAULT, 2120
DIST_DEF_SCMU_FAULT_3, 2122
DIST_DEF_SPCB_FAULT_2, 2124
DIST_MACHINE_TRIP, 2105
DIST_SCMU_FAULT, 2106
DIST_SCMU_FAULT_2, 2108
DIST_SCMU_FAULT_3, 2110
DIST_SPCB_FAULT, 2112
DIST_SPCB_FAULT_2, 2114
DLST, 1654
DMPC, 1746
DOCU, 1655
DOCUCHK, 4288
DOCUCHKBRANCH, 4289
DOCUCHKICON, 4290
DOCUCHKICONABS, 4291
DOCUCHKICONMSG, 4292
DOCUCHKINT, 4293
DOCUCHKLOAD, 4294
DOCUCHKMACHINE, 4295
DOCUCHKMSG, 4296
DOCUCHKPAIR, 4297
DOCUCHKPARALLELBRANCH, 4298
DOCUCHKREMBUS, 4299
DOCUHEADING, 4300
DRAGDROPPLOTDATA, 3495
DROPMODELLIBRARY, 1658
DROPMODELPROGRAM, 1659
DSCN, 171
DSCVAL, 2381
DSIVAL, 2382
DSRVAL, 2383
DUPLICATE_CNTLABEL_CHECK, 172
DYDA, 1660
DYN_ADD_SHUNT, 4301
DYN_REMOVE_SHUNT, 4302
DYNAMICS_SOLUTION_PARAM_2, 1666
DYNAMICS_SOLUTION_PARAMS, 1664
DYNAMICSMODE, 1663
DYNEXPORTCSV, 1668
DYRE_ADD, 1669
DYRE_ADD_2, 1670
DYRE_NEW, 1671
DYRE_NEW_2, 1672

E

ECDI, 173
ECHO, 175
EEQV, 176
ENABLEDIAGCONTOUR, 2154
ENCODE_BUS_NAME, 4303
EQRD, 178
ERUN, 1673
ESTR_OPEN_CIRCUIT_TEST, 1674
ESTR_RESPONSE_RATIO_TEST, 1675
EXAM, 180
EXC_FEX, 4305
EXC_FEX_INI, 4306
EXC_PW_SOURCE_2, 4307
EXC_PW_SOURCE_A, 4308
EXC_PW_SOURCE_B, 4309
EXCLUDESELECTEDCONTS, 3665
EXCSAT, 4304
EXECUTEACCONT, 3666
EXITACTIVITYACCONT, 3667
EXITACTIVITYMUST, 3668
EXPORTBUSLOCFILE, 2156
EXPORTBUSLOCFILE_2, 2157
EXPORTIMAGEFILE, 2158
EXPORTIMAGEFILE_2, 2159
EXPORTIMAGEFILE_3, 2160
EXPORTPLOT, 3496
EXTR, 712

F

FACT, 181
FACTDS_NOGINT, 4310
FACTS_CHNG_2, 713
FACTS_CHNG_3, 716
FACTS_DATA, 719
FACTS_DATA_2, 722
FACTS_DATA_3, 725
FCDAUXMIND, 2384

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

FCDAUXMNAM, 2386
FCDDAT, 2387
FCDDAT_2, 2390
FCDINT, 2393
FCDINT_2, 2394
FCDMIND, 2395
FCDMNAM, 2396
FCITC2LINE, 3669
FCITCAC, 3670
FCITCACMULT, 3671
FCITCACSINGLE, 3672
FCITCBASE, 3673
FCITCBASEMULT, 3674
FCITCBASESINGLE, 3675
FCITCMULT, 3676
FCITCSINGLE, 3677
FCITCSUBS, 3678
FCITCTLTG, 3679
FCITCWIDE, 3680
FCLIST, 1677
FCTBLOCK, 4311
FCTCNTL0, 4312
FCTMOD_PACK, 1893
FCTMOD_REMOVE, 1894
FCTMOD_STATUS, 1895
FCTMOD_UNCONNECTED, 1896
FCTMOD_USER, 1897
FCTSAUXMOD_REMOVE, 2079
FCTSAUXMOD_STATUS, 2080
FCTSTAT, 4313
FDNS, 182
FETCH_ADJVAR_TERM_NODE, 1047
FETCH_INDMACHINE_TERM_NODE, 1048
FETCH_LOAD_TERM_NODE, 1049
FETCH_MACHINE_TERM_NODE, 1050
FETCH_SHUNT_TERM_NODE, 1051
FETCH_SWS_SHUNT_TERM_NODE, 1052
FETCH_SWS_SHUNT_TERM_NODE_2, 1053
FILE_OVERWRITE, 184
FILEIN, 3459
FINAL_OPF_BARRIER_COEFF, 1111
FIND, 185
FINDMODELPROGRAM, 2397
FLAT, 1306
FLAT_2, 1308
FLAT_3, 1311
FLAT_START, 186
FLOW1M, 4314
FLOWGATESENSITIVITY, 3681
FNSL, 187
FORGETSTUDYTRANSFER, 3683
FXSBUS, 2398
FXSDT1, 2399
FXSDT2, 2401
FXSINT, 2403
FXSNAME, 2404

G

GCAP, 189
GCAP_2, 190
GDIF, 191
GEESAT, 4315
GENCHK, 4316
GENCHK2, 4317
GENCNV, 2405
GENDAT, 2406
GENDETAILS, 3684
GENDSP, 193
GENDSP_2, 195
GENDT1, 2407
GENERATIONSENSITIVITY, 3685
GENS, 197
GENSAT, 4318
GENSECTDAT, 2408
GENSECTDT1, 2409
GENSENDETAIL, 3687
GENSENSUMMARY, 3688
GENTRP, 4319
GEOL, 198
GET_ACBUS, 3865
GET_ALFA, 3866
GET_ALFMIN, 3867
GET_ALFMX, 3868
GET_ANGLE, 3869
GET_ANGMAX, 3870
GET_ANGMIN, 3871
GET_B2S, 3872
GET_BASFRQ, 3873
GET_BASVLT, 3874
GET_BSFREQ, 3875
GET_BSWICH, 3876
GET_BUSNAM, 3877
GET_CDGNFR, 3878
GET_CFDLFR, 3879
GET_CHAR_LENGTH, 2820
GET_CHRICN, 3880
GET_CLODFR, 3881
GET_CNVANG, 3882
GET_CNVCUR, 3883
GET_CNVTAP, 3884
GET_CNVTYP, 3885
GET_CON, 3886
GET_CON_DSCRPT, 3887
GET_CONMSGERROR, 3843
GET_CONMSGNONE, 3844
GET_CONMSGWARN, 3845
GET_CURNT, 3888
GET_DC, 3889
GET_DC2NAM, 3890
GET_DC2SIG, 3891
GET_DCIND, 3892
GET_DCMSIG, 3893
GET_DCNAME, 3894
GET_DCPF, 3895
GET_DCRES, 3896
GET_DCSETP, 3897
GET_DELT, 3898
GET_DELTI, 3899
GET_DERMWTHRESH, 3900
GET_DERVTTHRESH, 3901
GET_DOCUCHKERR_EQUAL, 3846
GET_DOCUCHKERR_GREATEROREQ, 3847
GET_DOCUCHKERR_GREATERTHAN, 3848
GET_DOCUCHKERR_INCLHIRANGE, 3849
GET_DOCUCHKERR_INCLLOWRANGE, 3850
GET_DOCUCHKERR_INCLUSIVERANGE, 3851
GET_DOCUCHKERR_LESSOREQ, 3852
GET_DOCUCHKERR_LESSTHAN, 3853
GET_DOCUCHKERR_NOTEQUAL, 3854
GET_DOCUCHKERR_NOTINCLHIRANGE, 3855
GET_DOCUCHKERR_NOTINCLLOWRANGE, 3856
GET_DOCUCHKERR_NOTINCLRANGE, 3857
GET_DOCUCHKERR_NOTRANGE, 3858
GET_DOCUCHKERR_RANGE, 3859
GET_DSTATE, 3902
GET_EBASE, 3903
GET_EBASEI, 3904
GET_EBASER, 3905
GET_ECOMP, 3906
GET_EFD, 3907
GET_ETERM, 3908
GET_EXCITER_VEMAX, 4326
GET_EXT_ACLINE_CHAR, 2626
GET_EXT_ACLINE_INT, 2627
GET_EXT_ACLINE_LOGICAL, 2628
GET_EXT_ACLINE_REAL, 2629
GET_EXT_AREA_CHAR, 2630
GET_EXT_AREA_INT, 2631
GET_EXT_AREA_LOGICAL, 2632
GET_EXT_AREA_REAL, 2633
GET_EXT_BUS_CHAR, 2634

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_EXT_BUS_INT, 2635
GET_EXT_BUS_LOGICAL, 2636
GET_EXT_BUS_REAL, 2637
GET_EXT_CHAR, 2638
GET_EXT_FACTS_CHAR, 2639
GET_EXT_FACTS_INT, 2640
GET_EXT_FACTS_LOGICAL, 2641
GET_EXT_FACTS_REAL, 2642
GET_EXT_FIXSHUNT_CHAR, 2643
GET_EXT_FIXSHUNT_INT, 2644
GET_EXT_FIXSHUNT_LOGICAL, 2645
GET_EXT_FIXSHUNT_REAL, 2646
GET_EXT_GENERATOR_CHAR, 2647
GET_EXT_GENERATOR_INT, 2648
GET_EXT_GENERATOR_LOGICAL, 2649
GET_EXT_GENERATOR_REAL, 2650
GET_EXT_GNE_CHAR, 2651
GET_EXT_GNE_INT, 2652
GET_EXT_GNE_LOGICAL, 2653
GET_EXT_GNE_REAL, 2654
GET_EXT_IATRANS_CHAR, 2655
GET_EXT_IATRANS_INT, 2656
GET_EXT_IATRANS_LOGICAL, 2657
GET_EXT_IATRANS_REAL, 2658
GET_EXT_INDMACH_CHAR, 2659
GET_EXT_INDMACH_INT, 2660
GET_EXT_INDMACH_LOGICAL, 2661
GET_EXT_INDMACH_REAL, 2662
GET_EXT_INT, 2663
GET_EXT_LOAD_CHAR, 2664
GET_EXT_LOAD_INT, 2665
GET_EXT_LOAD_LOGICAL, 2666
GET_EXT_LOAD_REAL, 2667
GET_EXT_LOGICAL, 2668
GET_EXT_MSLINE_CHAR, 2669
GET_EXT_MSLINE_INT, 2670
GET_EXT_MSLINE_LOGICAL, 2671
GET_EXT_MSLINE_REAL, 2672
GET_EXT_NTERMDC_CHAR, 2673
GET_EXT_NTERMDC_INT, 2674
GET_EXT_NTERMDC_LOGICAL, 2675
GET_EXT_NTERMDC_REAL, 2676
GET_EXT_OWNER_CHAR, 2677
GET_EXT_OWNER_INT, 2678
GET_EXT_OWNER_LOGICAL, 2679
GET_EXT_OWNER_REAL, 2680
GET_EXT_RATING_CHAR, 2681
GET_EXT_RATING_INT, 2682
GET_EXT_RATING_LOGICAL, 2683
GET_EXT_RATING_REAL, 2684
GET_EXT_REAL, 2685
GET_EXT_ROWCOUNT, 2686
GET_EXT_SUB_CHAR, 2695
GET_EXT_SUB_INT, 2696
GET_EXT_SUB_LOGICAL, 2697
GET_EXT_SUB_REAL, 2698
GET_EXT_SUBNODE_CHAR, 2687
GET_EXT_SUBNODE_INT, 2688
GET_EXT_SUBNODE_LOGICAL, 2689
GET_EXT_SUBNODE_REAL, 2690
GET_EXT_SUBSWD_CHAR, 2691
GET_EXT_SUBSWD_INT, 2692
GET_EXT_SUBSWD_LOGICAL, 2693
GET_EXT_SUBSWD_REAL, 2694
GET_EXT_SWSHUNT_CHAR, 2699
GET_EXT_SWSHUNT_INT, 2700
GET_EXT_SWSHUNT_LOGICAL, 2701
GET_EXT_SWSHUNT_REAL, 2702
GET_EXT_SYSSWD_CHAR, 2703
GET_EXT_SYSSWD_INT, 2704
GET_EXT_SYSSWD_LOGICAL, 2705
GET_EXT_SYSSWD_REAL, 2706
GET_EXT_TRANSFORMER_CHAR, 2707
GET_EXT_TRANSFORMER_INT, 2708
GET_EXT_TRANSFORMER_LOGICAL, 2709
GET_EXT_TRANSFORMER_REAL, 2710
GET_EXT_TWOTERMDC_CHAR, 2711
GET_EXT_TWOTERMDC_INT, 2712
GET_EXT_TWOTERMDC_LOGICAL, 2713
GET_EXT_TWOTERMDC_REAL, 2714
GET_EXT_VSCDC_CHAR, 2715
GET_EXT_VSCDC_INT, 2716
GET_EXT_VSCDC_LOGICAL, 2717
GET_EXT_VSCDC_REAL, 2718
GET_EXT_ZONE_CHAR, 2719
GET_EXT_ZONE_INT, 2720
GET_EXT_ZONE_LOGICAL, 2721
GET_EXT_ZONE_REAL, 2722
GET_FCNAME, 3909
GET_FCREM, 3910
GET_FCSBUS, 3911
GET_FCSHMX, 3912
GET_FCTBUS, 3913
GET_FCTREF, 3914
GET_FCTSIG, 3915
GET_FCVSET, 3916
GET_FQSHNT, 3917
GET_FRELVTHRESH, 3918
GET_FREQS, 3919
GET_GAMA, 3920
GET_GAMMIN, 3921
GET_GAMMX, 3922
GET_GB, 3923
GET_GBLINE, 3924
GET_GENTAP, 3925
GET_GREF, 3926
GET_ICOM, 3927
GET_ICON, 3928
GET_ICON_DSCRPT, 3929
GET_IFLAG, 3930
GET_IMARYS, 3931
GET_IMCCNX, 3932
GET_IMELTRQ, 3933
GET_IMLDTRQ, 3934
GET_IMSPARE, 3935
GET_IMTPLOD, 3936
GET_IMTQLOD, 3937
GET_INDX3W, 3938
GET_INTICN, 3939
GET_IPI, 3940
GET_IPR, 3941
GET_IPRT, 3942
GET_ISORCE, 3943
GET_ITER, 3944
GET_ITMXDS, 3945
GET_JCODE, 3946
GET_KNUM, 3947
GET_KPAUSE, 3948
GET_KTRIP, 3949
GET_LCREF, 3950
GET_LDAREA, 3951
GET_LDDGNFL, 3952
GET_LDOWNR, 3953
GET_LDSTAT, 3954
GET_LDSTR2, 3955
GET_LDSTRT, 3956
GET_LDZONE, 3957
GET_LOADID, 3958
GET_LODBKVTHRESH, 3959
GET_LODID, 3960
GET_LODMWTHRESH, 3961
GET_LODPQTHRESH, 3962
GET_LODVTTHRESH, 3963
GET_LSYSAN, 3964
GET_MACBLKD, 3965
GET_MACHID, 3966
GET_MACNUM, 3967
GET_MARGIN, 3968
GET_MBASE, 3969
GET_MCARYS, 3970
GET_MCSTAT, 3971
GET_MDC, 3972

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_MDCNAM, 3973
GET_MIDTRM, 3974
GET_MODE, 3975
GET_MSGTYPE_ERROR, 3860
GET_MSGTYPE_INFO, 3861
GET_MSGTYPE_WARN, 3862
GET_MSTATE, 2411, 3976
GET_NBI, 3977
GET_NBR, 3978
GET_NBRIDG, 3979
GET_NBUS, 3980
GET_NCONV, 3981
GET_NINTEG, 3982
GET_NLIN, 3983
GET_NLWTRB, 3984
GET_NMACHS, 3985
GET_NSWSTP, 3986
GET_NTAPOS, 3987
GET_NUMARE, 3988
GET_NUMLOD, 3989
GET_NUMOWN, 3990
GET_NUMSWS, 3991
GET_NUMTRM, 3992
GET_NUMZON, 3993
GET_OEL_ACTIVE, 3994
GET_OLDERV, 3995
GET_ORGDCMODE, 3996
GET_ORGFCSTAT, 3997
GET_ORGICOM, 3998
GET_ORGJCODE, 3999
GET_ORGMDC, 4000
GET_ORGVSCMDC, 4001
GET_ORGVSCTYP, 4002
GET_PAC, 4003
GET_PACI, 4004
GET_PACR, 4005
GET_PELEC, 4006
GET_PGENMC, 4007
GET_PHSHTB, 4008
GET_PHSVLT, 4009
GET_PMAXMC, 4010
GET_PMAXMCD, 4011
GET_PMECH, 4012
GET_PMINMC, 4013
GET_PMINMCD, 4014
GET_PQCNOD, 4015
GET_PQCONT, 4016
GET_PSANGL, 4017
GET_QAC, 4018
GET_QACI, 4019
GET_QACR, 4020
GET_QELEC, 4021
GET_QGENMC, 4022
GET_QMAXMC, 4023
GET_QMINMC, 4024
GET_RCI, 4025
GET_RCOMP, 4026
GET_RCONV, 4027
GET_RCR, 4028
GET_RDC, 4029
GET_RLSTRT, 4030
GET_RMAX, 4031
GET_RMIN, 4032
GET_SBASE, 4033
GET_SETVAL, 4034
GET_SHFRST, 4035
GET_SHNEXT, 4036
GET_SHNTGB, 4037
GET_SHNTID, 4038
GET_SHSTAT, 4039
GET_SPEED, 4040
GET_STATE, 4041
GET_STATE_DSCRPT, 4042
GET_STORE, 4043
GET_STRT2D, 4044
GET_STRTAU, 4045
GET_STRTCCT, 4046
GET_STRTFC, 4047
GET_STRTIM, 4048
GET_STRTIN, 4049
GET_STRTMD, 4050
GET_STRTSS, 4051
GET_STRTVS, 4052
GET_SWSAUX, 4053
GET_SWSCNX, 4054
GET_SWSHID, 4055
GET_SWSHNB, 4056
GET_SWSHRM, 4057
GET_SWSREF, 4058
GET_TABLNO, 4059
GET_TAPI, 4060
GET_TAPNO, 4061
GET_TAPR, 4062
GET_TBRATE, 4063
GET_THRSHZ, 4064
GET_TIME, 4065
GET_TOL, 4066
GET_TPLOAD, 4067
GET_TPNABL, 4068
GET_TQLOAD, 4069
GET_TRCONV, 4070
GET_TRI, 4071
GET_TRR, 4072
GET_UEL_ACTIVE, 4073
GET_VAR, 4074
GET_VAR_DSCRPT, 4075
GET_VCMODE, 4076
GET_VDC, 4077
GET_VDCI, 4078
GET_VDCR, 4079
GET_VFEOUT, 4080
GET_VG2IDX, 4081
GET_VMAX, 4082
GET_VMIN, 4083
GET_VOEL, 4084
GET_VOLT, 4085
GET_VOTHSG, 4086
GET_VREF, 4087
GET_VRFDBK, 4088
GET_VSCACS, 4089
GET_VSCCNV, 4090
GET_VSCDCS, 4091
GET_VSCHED, 4092
GET_VSCIDC, 4093
GET_VSCIMX, 4094
GET_VSCLSA, 4095
GET_VSCLSB, 4096
GET_VSCLSM, 4097
GET_VSCMDC, 4098
GET_VSCMOD, 4099
GET_VSCNAM, 4100
GET_VSCPAC, 4101
GET_VSCPSV, 4102
GET_VSCPWF, 4103
GET_VSCQAC, 4104
GET_VSCQMN, 4105
GET_VSCQMX, 4106
GET_VSCQSV, 4107
GET_VSCRDC, 4108
GET_VSCREM, 4109
GET_VSCSIG, 4110
GET_VSCSMX, 4111
GET_VSCTYP, 4112
GET_VSHNHI, 4113
GET_VSHNLO, 4114
GET_VUEL, 4115
GET_W0, 4116
GET_W7PRG, 3863
GET_W7RPT, 3864
GET_WAEROT, 4117
GET_WAUXSG, 4118
GET_WEQCMD, 4119

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GET_WINDRP, 4120
GET_WINDRS, 4121
GET_WIPCMD, 4122
GET_WIQCMD, 4123
GET_WMCARYS, 4124
GET_WNDNUM, 4125
GET_WNDSP1, 4126
GET_WNDSP10, 4127
GET_WNDSP2, 4128
GET_WNDSP3, 4129
GET_WNDSP4, 4130
GET_WNDSP5, 4131
GET_WNDSP6, 4132
GET_WNDSP7, 4133
GET_WNDSP8, 4134
GET_WNDSP9, 4135
GET_WPCMND, 4136
GET_WPITCH, 4137
GET_WPORDR, 4138
GET_WPRATE, 4139
GET_WPREFPTQ, 4140
GET_WPREFWGO, 4141
GET_WQCMND, 4142
GET_WROTID, 4143
GET_WROTIQ, 4144
GET_WROTRI, 4145
GET_WROTRV, 4146
GET_WROTVD, 4147
GET_WROTVQ, 4148
GET_WSTRTIN, 4149
GET_WTRBSP, 4150
GET_WVLCTY, 4151
GET_XADIFD, 4152
GET_XCI, 4153
GET_XCONV, 4154
GET_XCR, 4155
GET_XTRAN, 4156
GET_ZERNUM, 4157
GET_ZSORCE, 4158
GETAUTOBUBBLE, 3814
GETBATDEFAULTS, 3460
GETBRANCHSEQCURRENTS, 4320
GETCONTINGENCYSAVEDCASE, 199
GETDEFAULTCHAR, 3461
GETDEFAULTINT, 3462
GETDEFAULTREAL, 3463
GETEXCLUDEDBRANCHES, 3815
GETEXCLUDEDCONTINGENCIES, 3816
GETEXCLUDEDFLOWGATES, 3817
GETEXCLUDEDINTERFACES, 3818
GETEXCLUDEDVOLTAGEBUSES, 3819
GETFINALBALANCE, 3820
GETFINALFLOW, 3821
GETFTC, 4321
GETFTC3, 4322
GETHOMEPATH, 2410
GETIDOFCONTINGENCYBYNAME, 3822
GETIDOFFLOWGATEBYNAME, 3823
GETIDOFINTERFACEBYNAME, 3824
GETIDOFVOLTAGEBUSBYNAME, 3825
GETINITBALANCE, 3826
GETINITIALFLOW, 3827
GETIPFFACTOR, 3828
GETLOADEDMODULES, 3464
GETMACHSEQCURRENTS, 4323
GETMODELLIBRARIES, 3331
GETMODELPROGRAMS, 3332
GETMODFUNCLIST, 3465
GETNAMEOFCONTINGENCYBYID, 3829
GETNAMEOFFLOWGATEBYID, 3830
GETNAMEOFINTERFACEBYID, 3831
GETNAMEOFVOLTAGEBUSBYID, 3832
GETNUMAUTOBUBBLE, 3833
GETNUMOFEXCLUDEDBRANCHES, 3834
GETNUMOFEXCLUDEDCONTINGENCIES, 3835
GETNUMOFEXCLUDEDFLOWGATES, 3836
GETNUMOFEXCLUDEDINTERFACES, 3837
GETNUMOFEXCLUDEDVOLTAGEBUSES, 3838
GETPHASEVOLTAGES, 4324
GETSEQUENCEVOLTAGES, 4325
GETTPFBALANCE, 3839
GETTSPFPROFILEBYELEMENT, 3333
GETTSPFPROFILECOUNTBYELEMENT, 3334
GETTSPFPROFILEDATA, 3335
GETTSPFPROFILEDATABYSET, 3336
GETTSPFPROFILEDATACOLUMNCOUNT, 3337
GETTSPFPROFILEDATACOUNT, 3338
GEXMBUS, 2161
GIC, 1492
GIC_2, 1495
GIC_2TDC, 1568
GIC_2TDC_CHNG, 1569
GIC_2TDC_PURG, 1570
GIC_3, 1501
GIC_4, 1507
GIC_5, 1514
GIC_6, 1521
GIC_7, 1530
GIC_8, 1539
GIC_BRANCH_EFIELD_NN, 1550
GIC_BRN, 1571
GIC_BRN_CHNG, 1573
GIC_BRN_PURG, 1575
GIC_EARTH1D_USR, 1576
GIC_EARTH1D_USR_CHNG, 1577
GIC_EARTH1D_USR_PURG, 1552
GIC_EFIELD_WAVESHAPE, 1553
GIC_EFIELD_WAVESHAPE_REF, 1554
GIC_EFIELD_WAVESHAPE_SUPP, 1555
GIC_FACTS, 1578
GIC_FACTS_CHNG, 1579
GIC_FACTS_PURG, 1580
GIC_FXSH, 1581
GIC_FXSH_CHNG, 1582
GIC_FXSH_PURG, 1583
GIC_LOAD, 1584
GIC_LOAD_CHNG, 1585
GIC_LOAD_PURG, 1586
GIC_MTDC, 1587
GIC_MTDC_CHNG, 1588
GIC_MTDC_PURG, 1589
GIC_MVARLOSS_SCALING_FACTORS, 1556
GIC_NEW, 1557
GIC_PF, 200
GIC_PF_OPTIONS, 1558
GIC_PURG, 1560
GIC_READ, 1561
GIC_READ_EFIELD_GRID, 1562
GIC_SUBSTN, 1590
GIC_SUBSTN_CHNG, 1591
GIC_SUBSTN_PURG, 1592
GIC_SWSH, 1593
GIC_SWSH_CHNG, 1594
GIC_SWSH_PURG, 1595
GIC_THERMAL_IMPACT, 1563
GIC_THERMAL_IMPACT_1, 1564
GIC_TRN, 1596
GIC_TRN_CHNG, 1599
GIC_TRN_PURG, 1602
GIC_VSCDC, 1603
GIC_VSCDC_CHNG, 1604
GIC_VSCDC_PURG, 1605
GIC_WRITE, 1565
GIC_WRITE_STN, 1566
GMB_ADD_FACTS_DEV_MODEL, 1898
GMB_ADD_PLANT_MODEL, 1787
GMB_ADD_SWSHUNT_MODEL, 1912
GMB_ADD_SWSHUNT_MODEL_2, 1914
GNE_CHNG, 728
GNE_DATA, 730
GNECHR, 2412
GNEDAT, 2413

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

GNEINT, 2414
GNESTATUS, 202
GNET, 203
GROWBUS, 2162
GROWBUSLEVELS, 2163
GROWDIAGRAM, 2164
GROWDIAGRAM_2, 2165
GROWSTATION, 2167
GRPG, 2168
GRUN, 1678
GSTR, 1679
GTRSUB, 4327

H

HAR_2TDC, 1426
HAR_2TDC_CHNG, 1427
HAR_2TDC_PURG, 1428
HAR_ANALYSIS, 1401
HAR_ANALYSIS_2, 1404
HAR_BRN, 1429
HAR_BRN_CHNG, 1430
HAR_BRN_PURG, 1431
HAR_CREATE_PFCASE, 1410
HAR_CURSRC, 1432
HAR_CURSRC_CHNG, 1433
HAR_CURSRC_CHNG_NAME, 1434
HAR_CURSRC_CHNG_ONEPT, 1435
HAR_CURSRC_NAME, 1436
HAR_CURSRC_ONEPT, 1437
HAR_CURSRC_PURG, 1438
HAR_CURSRC_PURG_ONEPT, 1439
HAR_EXISTS_DSTN_RESULTS, 1411
HAR_EXISTS_FSCAN_RESULTS, 1412
HAR_EXPORT_DSTN, 1413
HAR_EXPORT_FSCAN, 1415
HAR_FACTS, 1440
HAR_FACTS_CHNG, 1441
HAR_FACTS_PURG, 1442
HAR_IMPCHAR, 1443
HAR_IMPCHAR_CHNG, 1444
HAR_IMPCHAR_CHNG_NAME, 1445
HAR_IMPCHAR_CHNG_ONEPT, 1446
HAR_IMPCHAR_NAME, 1447
HAR_IMPCHAR_ONEPT, 1448
HAR_IMPCHAR_PURG, 1449
HAR_IMPCHAR_PURG_ONEPT, 1450
HAR_INDMC, 1451
HAR_INDMC_CHNG, 1452
HAR_INDMC_PURG, 1453
HAR_LIMITS_SC_CURRENTS, 1416
HAR_LIMITS_VIOLATIONS_CURRENT, 1417
HAR_LIMITS_VIOLATIONS_VOLTAGE, 1419
HAR_LOAD, 1454
HAR_LOAD_CHNG, 1455
HAR_LOAD_PURG, 1456
HAR_MACH, 1457
HAR_MACH_CHNG, 1458
HAR_MACH_PURG, 1459
HAR_MTDC, 1460
HAR_MTDC_CHNG, 1461
HAR_MTDC_PURG, 1462
HAR_NEW, 1421
HAR_PASSIVE_FILTER, 1463
HAR_PASSIVE_FILTER_CHNG, 1465
HAR_PASSIVE_FILTER_PURG, 1467
HAR_PURG, 1422
HAR_SET_PAR_LODMDL_CIGRE_MEASUREMENT, 1468
HAR_SET_PAR_LODMDL_CIGRE_MOTIVE, 1469
HAR_SET_PAR_LODMDL_CIGRE_PASSIVE, 1470
HAR_SET_PAR_LODMDL_IEEE_2RL_PARALLEL, 1471
HAR_SET_PAR_LODMDL_IEEE_IM, 1472
HAR_SET_PAR_LODMDL_IEEE_MEASUREMENT, 1473
HAR_SET_PAR_LODMDL_IEEE_SKIN, 1474
HAR_SET_PAR_LODMDL_LARGE_ASYNC_MOTOR, 1475
HAR_SET_RESN_THRESHOLDS, 1423
HAR_SET_RESN_THRESHOLDS_DEFAULT, 1424
HAR_TRN, 1476
HAR_TRN_CHNG, 1477
HAR_TRN_PURG, 1478
HAR_VLTSRC, 1479
HAR_VLTSRC_CHNG, 1480
HAR_VLTSRC_CHNG_NAME, 1481
HAR_VLTSRC_CHNG_ONEPT, 1482
HAR_VLTSRC_NAME, 1483
HAR_VLTSRC_ONEPT, 1484
HAR_VLTSRC_PURG, 1485
HAR_VLTSRC_PURG_ONEPT, 1486
HAR_VSCDC, 1487
HAR_VSCDC_CHNG, 1488
HAR_VSCDC_PURG, 1489

I

IECS, 1314
IECS_2, 1316
IECS_3, 1319
IECS_4, 1322
IECS_SCFILE, 1326
ILFDEFAULTS, 3689
IMMOD_PACK, 1929
IMMOD_REMOVE, 1930
IMMOD_STATUS, 1931
IMMOD_UNCONNECTED, 1932
IMMOD_USER, 1933
IMNET, 204
IMOUTAGE, 205
IMPACTINTRFACEPARTFACT, 3691
IMPACTSETBASECASE, 3693
IMPACTSETCONTINGENCY, 3694
IMPACTTIESPARTFACT, 3695
IMPACTTOTALPARTFACT, 3697
IMPC, 206
IMPEDANCE_CORRECTION_DATA, 732
IMPEDANCE_CORRECTION_DATA_3, 733
IMPLEMENT_TRANSFER, 207
IMPLEMENT_TRANSFER_2, 210
IMPORT_ECDI, 1112
IMPORTDRAWFILE, 2169
IMPORTIMAGEFILE, 2170
IMPOSEPROFILE, 1608
INCLUDESELECTEDCONTS, 3698
INCREMENT_GREF, 1680
INCREMENT_PREF, 1681
INCREMENT_QVREF, 1682
INCREMENT_SWSREF, 1683
INCREMENT_SWSREF_2, 1684
INCREMENT_VREF, 1685
INDBUS, 2416
INDDT1, 2417
INDDT2, 2419
INDINT, 2421
INDUCTION_MACHINE_CHNG, 734
INDUCTION_MACHINE_CHNG_2, 737
INDUCTION_MACHINE_DATA, 740
INDUCTION_MACHINE_DATA_2, 743
INI2DC, 2423
INIBRN, 2424
INIBRN_2, 2425
INIBRX, 2426
INIBRX_2, 2427
INIBUS, 2428
INIBUS_2, 2429
INIBUX, 2430
INIFAX, 2431
INIFXS, 2432
INIIND, 2433
INILOD, 2434
INIMAC, 2435
INIMDC, 2436
INIMSL, 2437
ININAM, 2438

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

ININAM_2, 2439
ININAX, 2440
INISTABUSSECT, 2441
INISTANODE, 2442
INISTASWDEV, 2443
INISWS, 2444
INITIAL_OPF_BARRIER_COEFF, 1113
INITIE, 2445
INITIX, 2446
INIVSC, 2447
INLF, 213
INLF_2, 215
INSERTPAGE, 3497
INSERTPLOT, 3498
INT_MODE1, 4328
INT_MODE2, 4329
INT_MODE3, 4330
INTA, 217
INTERFACE_CHNG, 750
INTERFACE_DATA, 753
INTERFACE_FLOW_COST_COEFF, 1114
INTERFACEELEMENT_CHNG, 746
INTERFACEELEMENT_DATA, 748
INTERFACESENSITIVITY, 3699
INTFLOW, 2448
INTZ, 218
IOFLUSH, 3466
ISLAND, 2449
ISOLATE_LEVELS, 219
ISOLATE2DCLINEBYBREAKER, 1022
ISOLATE3WTBYBREAKER, 1023
ISOLATEBUSBYBREAKER, 1024
ISOLATEFACTSBYBREAKER, 1025
ISOLATEINDMACHINEBYBREAKER, 1026
ISOLATELINEBYBREAKER, 1027
ISOLATELOADBYBREAKER, 1028
ISOLATEMACHINEBYBREAKER, 1029
ISOLATEMSLINEBYBREAKER, 1030
ISOLATEMTDCLINEBYBREAKER, 1031
ISOLATENODEBYBREAKER, 1032
ISOLATESHUNTBYBREAKER, 1033
ISOLATESWDBYBREAKER, 1034
ISOLATESWSHUNTBYBREAKER, 1035
ISOLATESWSHUNTBYBREAKER_2, 1036
ISOLATEVSCDCLINEBYBREAKER, 1037
ITERAT, 2450

J

JOIN, 220

L

LAG_MODE1, 4331
LAG_MODE2, 4332
LAG_MODE3, 4333
LAMP, 221
LAST_PF, 3339
LAUNCH_PROGRAM, 3467
LDCLIST, 1686
LDLG_MODE1, 4334
LDLG_MODE2, 4335
LDLG_MODE3, 4336
LDLIST, 1688
LDMOD_PACK, 1822
LDMOD_REMOVE, 1823
LDMOD_STATUS, 1824
LDMOD_STATUS2, 1825
LDMOD_UNCONNECTED, 1826
LDMOD_USER, 1827
LDMODC_PACK, 1815
LDMODC_REMOVE, 1816
LDMODC_STATUS, 1818
LDMODC_UNCONNECTED, 1820
LDMODC_USER, 1821
LDSHED, 4337
LINE_SHUNT_REPORTING, 224
LINES_PER_PAGE, 222
LINES_PER_PAGE_ONE_DEVICE, 223
LINESW, 4338
LINRCL, 4339
LINTRP, 4340
LIST, 225
LIST_BFLOW, 3710
LIST_BIMPACT, 3711
LIST_CFLOW, 3712
LIST_CHANNEL_MODELS, 1748
LIST_CIMPACT, 3713
LIST_CONTINS, 3714
LIST_CONTINS_WISL, 3715
LIST_LODF, 3716
LIST_LODF_WISL, 3717
LIST_MONELS, 3718
LIST_PTDF, 3719
LIST_PTDFC, 3720
LIST_SYSTEMS, 3721
LISTACCONTDETAILS, 3701
LISTACCONTSUMMARY, 3702
LISTAREAS, 3703
LISTBUSES, 3704
LISTCONTINGENCYSAVEDCASES, 227
LISTEXCLUDES, 3705
LISTGENERATORS, 3706
LISTPHASESHIFTERS, 3707
LISTSELCONTINGENCIES, 3708
LISTZONES, 3709
LLRF, 228
LMODIND, 2451
LMODNAM, 2453
LOAD_ARRAY_CHANNEL, 1689
LOAD_CHNG_4, 758
LOAD_CHNG_5, 760
LOAD_CHNG_6, 762
LOAD_CHNG_7, 764
LOAD_DATA, 766
LOAD_DATA_3, 768
LOAD_DATA_4, 770
LOAD_DATA_5, 772
LOAD_DATA_6, 774
LOAD_DATA_7, 776
LOAD_REDUCTION, 230
LOADTYPE, 4341
LOADTYPE_CHNG, 756
LOADTYPE_DATA, 757
LODBUS, 2454
LODCHK, 4342
LODCNV, 2455
LODDT1, 2456
LODDT2, 2458
LODIND, 2460
LODINT, 2461
LODNAME, 2462
LODTYPE, 2463
LOGSUBSYSTEMLOADMODEL, 4343
LONG_TITLE_DATA, 778
LOUT, 231
LSTO, 1115
LTAP, 232

M

MACBUS, 2464
MACCC, 233
MACCC_2, 238
MACCC_3, 243
MACCC_PARALLEL, 248
MACCC_PARALLEL_2, 253
MACCC_TRIP_COR, 258
MACCC_TRIP_COR_2, 264
MACCC_TRIP_COR_3, 270
MACCC_TRIP_COR_4, 277
MACCC_WITH_COR, 283

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

MACCC_WITH_COR_2, 288
MACCC_WITH_COR_3, 294
MACCC_WITH_COR_4, 300
MACCC_WITH_TRIP, 306
MACCC_WITH_TRIP_2, 311
MACCC_WITH_TRIP_3, 316
MACCC_WITH_TRIP_PARALLEL, 321
MACCC_WITH_TRIP_PARALLEL_2, 326
MACCHR, 2465
MACDAT, 2466
MACDT2, 2468
MACHINE_APP_R_X_CHANNEL, 1751
MACHINE_ARRAY_CHANNEL, 1753
MACHINE_CAP_CURVE_CHNG, 779
MACHINE_CAP_CURVE_DATA, 781
MACHINE_CHNG_2, 783
MACHINE_CHNG_3, 785
MACHINE_CHNG_4, 787
MACHINE_CHNG_5, 789
MACHINE_DATA, 792
MACHINE_DATA_2, 794
MACHINE_DATA_3, 796
MACHINE_DATA_4, 799
MACHINE_DATA_5, 802
MACHINE_ITERM_CHANNEL, 1755
MACIND, 2470
MACINT, 2471
MACSTT, 2473
MATRIX_GROWTH_FACTOR, 331
MAXMSM, 2474
MBID_SWD_RATING, 821
MBID2DC, 805
MBID3WND, 806
MBIDATRN, 807
MBIDBRN, 808
MBIDFACTS, 809
MBIDGNE, 810
MBIDINDMAC, 811
MBIDINTF, 812
MBIDLOAD, 813
MBIDMAC, 814
MBIDMDC, 815
MBIDMSL, 816
MBIDNCS, 1353
MBIDSHUNT, 817
MBIDSWSHUNT, 818
MBIDVD, 819
MBIDVSC, 820
MCRE, 332
MDETAILBRANCHES, 3722
MDETAILINTERFACES, 3723
MDETAILVOLTAGEBUSES, 3724
MDLIND, 2475
MDLLIBCNT, 2477
MDLNAM, 2478
MDLPGMCNT, 2479
MINIMIZE_ADJ_BUS_SHUNTS, 1117
MINIMIZE_FUEL_COST, 1118
MINIMIZE_INTERFACE_FLOWS, 1119
MINIMIZE_LOAD_ADJUSTMENTS, 1120
MINIMIZE_P_LOSSES, 1121
MINIMIZE_P_SLACK, 1122
MINIMIZE_Q_LOSSES, 1123
MINIMIZE_Q_SLACK, 1124
MINIMIZE_REACTIVE_RESERVE, 1125
MINIMIZE_SERIES_COMP, 1126
MLST, 1690
MODELID, 4344
MODESW, 4345
MODIFYDISPATCH, 3725
MODR, 333
MONELIMPACT, 3726
MONSENSALLSUBBRANCHES, 3728
MONSENSALLSUBFLOWGATES, 3729
MONSENSALLSUBINTERFACES, 3730
MONSENSALLTRANBRANCHES, 3731
MONSENSALLTRANFLOWGATES, 3732
MONSENSALLTRANINTERFACES, 3733
MONSENSSINGLESUBBRANCHES, 3734
MONSENSSINGLESUBFLOWGATES, 3735
MONSENSSINGLESUBINTERFACES, 3736
MONSENSSUBSPARTBRANCHES, 3737
MONSENSSUBSPARTFLOWGATES, 3738
MONSENSSUBSPARTINTERFACES, 3739
MONSENSTRANBRANCHES, 3740
MONSENSTRANFLOWGATES, 3741
MONSENSTRANINTERFACES, 3742
MOV_ALPHA, 348
MOV_ITERATIONS, 349
MOV_TOLERANCE, 350
MOVE3WND, 335
MOVEBRN, 336
MOVEINDMAC, 337
MOVEINDMACS, 338
MOVELOAD, 339
MOVELOADS, 340
MOVEMAC, 341
MOVEPLNT, 342
MOVESHUNT, 343
MOVESHUNTS, 344
MOVESWS, 345
MOVESWSHUNT, 346
MOVESWSHUNTS, 347
MRUN, 1692
MSLV, 351
MSTR, 1693
MSUM, 353
MTDC, 354
MTDCAUXMOD_REMOVE, 2081
MTDCAUXMOD_STATUS, 2082
MTDCLMOD_REMOVE, 1879
MTDCLMOD_STATUS, 1880
MULTACFCITC, 3743
MULTDCFCITC, 3745
MULTI_SECTION_LINE_DATA, 822
MULTI_SECTION_LINE_EDIT, 824
MULTI_TERM_DC_BUS_DATA, 825
MULTI_TERM_DC_CONVR_DATA, 827
MULTI_TERM_DC_LINE_CHNG, 830
MULTI_TERM_DC_LINE_DATA, 832
MULTI_TERM_DC_LINK_DATA, 834
MULTICHANNEXCELEXPORT, 3499
MULTIPLEPARALLELTRANSFERS, 3746
MULTIPLESTUDYTRANSFERS, 3748
MULTISECTION_REPORTING, 355
MUSTOPTDEFS, 3750
MUSTRESULTSEXIST, 3840
MUSTSTART, 3752
MUSTSTOP, 3753
MWMI, 356
MWOMWMSF, 358
MWOMWOSF, 359

N

N11_ACCC, 360
N11_ACCC_2, 364
N11_ACCC_3, 368
N11_ACCC_PARALLEL, 372
N11_ACCC_PARALLEL_2, 376
N11_ACCC_PSCOPF, 380
N11_ACCC_PSCOPF_2, 387
N11_ACCC_PSCOPF_3, 394
NATONO, 2480
NETG, 400
NEW_DIMENSION, 404
NEWCAS, 401
NEWCASE_2, 402
NEWDIAGFILE, 2171
NEWOPF, 1167
NEWPLOTBOOK, 3500
NEWSCENARIOFILE, 3216

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

NEWSEQ, 1327
NEWTON_TOLERANCE, 403
NEXTDIAGFILEDATASET, 2172
NON_DIVERGENT, 405
NON_TRANS_PERCENT_UNITS, 406
NOPF, 1127
NOTMID, 4346
NOTONA, 2481
NOTONASECT, 2482
NSOL, 407
NUMBER_THREADS, 409
NUMBUS, 4347
NWINT_MODE1, 4348
NWINT_MODE2, 4349
NWINT_MODE3, 4350
NWLAG_MODE1, 4351
NWLAG_MODE2, 4352
NWLAG_MODE3, 4353
NWLDLG_MODE1, 4354
NWLDLG_MODE2, 4355
NWLDLG_MODE3, 4356
NWPI_MODE1, 4366
NWPI_MODE2, 4367
NWPI_MODE3, 4368
NWPIB_MODE1, 4357
NWPIB_MODE2, 4358
NWPIB_MODE3, 4359
NWPID_MODE1, 4360
NWPID_MODE2, 4361
NWPID_MODE3, 4362
NWPIPW_MODE1, 4363
NWPIPW_MODE2, 4364
NWPIPW_MODE3, 4365
NXT2DC, 2483
NXTBRN, 2484
NXTBRN_2, 2487
NXTBRN3, 2485
NXTBRN3_2, 2486
NXTBUS, 2488
NXTBUS_2, 2489
NXTFAX, 2490
NXTFXS, 2491
NXTIND, 2492
NXTLOD, 2493
NXTMAC, 2494
NXTMDC, 2495
NXTMSL, 2496
NXTSTABUSSECT, 2497
NXTSTANODE, 2498
NXTSTASWDEV, 2499
NXTSWS, 2500
NXTTIE, 2501
NXTTIE3, 2502
NXTVSC, 2503

O

OKSTRT, 2504
OPEN_BUS_VOLTAGE_LIMITS, 1128
OPENACCFILE, 2173
OPENACCFILE_2, 2174
OPENACCFILE_3, 2175
OPENBUSLOCFILE, 2176
OPENCHANDATAFILE, 3501
OPENDIAGFILE, 2177
OPENEVENTSTUDYFILE, 3307
OPENSCENARIOFILE, 3217
OPENUN, 2505
OPF_ADJBRX_INDV, 1168
OPF_ADJBRX_SUBSYS, 1170
OPF_ADJLOAD_TBL, 1173
OPF_ADJVAR_INDV, 1175
OPF_ADJVAR_INDV_2, 1177
OPF_ADJVAR_SUBSYS, 1179
OPF_APDSP_TBL, 1182
OPF_BARRIER_STEP_LENGTH, 1129
OPF_BRFLW_3WT_INDV, 1184
OPF_BRFLW_BRN_INDV, 1186
OPF_BRFLW_SUBSYS, 1188
OPF_BUS_INDV, 1191
OPF_BUS_INDV_2, 1193
OPF_BUS_SUBSYS, 1195
OPF_CHANGE_3WT_FLOW_ID, 1198
OPF_CHANGE_ADJVAR_ID, 1199
OPF_CHANGE_BRN_FLOW_ID, 1200
OPF_CLAMP_DECAY_FACTOR, 1130
OPF_CSTTBL_LIN, 1201
OPF_CSTTBL_POLY, 1202
OPF_CSTTBL_QUAD, 1203
OPF_FINAL_CLAMP_TOL, 1131
OPF_FIX_ALL_GENERATORS, 1133
OPF_FIX_PHASE_SHIFTERS, 1134
OPF_FIX_SWITCHED_SHUNTS, 1135
OPF_FIX_TAP_RATIOS, 1136
OPF_FIXED_VOLTAGE_PENALTY, 1132
OPF_GEN_RCAP_INDV, 1209
OPF_GEN_RCAP_SUBSYS, 1211
OPF_GENDSP_INDV, 1204
OPF_GENDSP_SUBSYS, 1205
OPF_GENRSV_INDV, 1206
OPF_GENRSV_SUBSYS, 1207
OPF_INITIAL_CLAMP_TOL, 1137
OPF_INTERIOR_SHIFT_FACTOR, 1138
OPF_INTFLW_3WT, 1214
OPF_INTFLW_BRN, 1215
OPF_INTFLW_MAIN, 1216
OPF_INTFLW_RESET, 1218
OPF_LAMBDA_TOLERANCE, 1139
OPF_LF_CONTROL_PENALTY, 1140
OPF_LNCEQN_ADJLOAD, 1219
OPF_LNCEQN_ADJVAR, 1220
OPF_LNCEQN_BRFLOW, 1221
OPF_LNCEQN_INTFLOW, 1222
OPF_LNCEQN_MAIN, 1223
OPF_LNCEQN_PGEN, 1224
OPF_LNCEQN_QGEN, 1225
OPF_LNCEQN_RESET, 1226
OPF_LNCEQN_SWSHUNT, 1227
OPF_LNCEQN_SWSHUNT_2, 1228
OPF_LNCEQN_TRAN, 1229
OPF_LNCEQN_VANG, 1230
OPF_LNCEQN_VANG_2, 1231
OPF_LNCEQN_VMAG, 1232
OPF_LNCEQN_VMAG_2, 1233
OPF_LOAD_INDV, 1234
OPF_LOAD_SUBSYS, 1235
OPF_MAX_TAP_RATIO_STEP, 1141
OPF_MIN_TAP_RATIO_STEP, 1142
OPF_PERRSV_GEN, 1236
OPF_PERRSV_MAIN, 1237
OPF_PERRSV_RESET, 1238
OPF_REGULATE_AREA_INT, 1143
OPF_ROUND_SWITCHED_SHUNTS, 1144
OPF_ROUND_TAP_RATIOS, 1145
OPF_SCALE_QGEN_LIMITS, 1146
OPF_STEP_LENGTH_TOLERANCE, 1147
OPF_USE_GENERATOR_VSCHED, 1148
OPTO, 1149
ORD2_MODE1, 4369
ORD2_MODE2, 4370
ORD2_MODE3, 4371
ORDBUS, 2506
ORDBUSSECT, 2507
ORDR, 410
OSYS, 2141
OSYSDEF, 2142
OSYSINIT, 2143
OTDF, 411
OUTPUT_Y_MATRIX, 412
OUTPUTMODELMSGS, 4372
OUTS, 413
OWNDAT, 2508

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

OWNER_DATA, 836
OWNM, 414
OWNM_2, 416
OWNM_3, 418
OWNNAM, 2510
OWNNUM, 2511
OWNR, 420
OWNUSE, 2512

P

P_LOSSES_COST_COEFF, 1156
PAGEREPORT, 3468
PARALLELDETAILTRANSFER, 3754
PARALLELFCITCSUMMARY, 3755
PARALLELTRANSFERDETAIL, 3756
PATH, 421
PAUSERECORDING, 3469
PBUS_ADD_MOD, 837
PBUS_DELETE, 838
PHASE_SHIFT_ADJUSTMENT, 422
PI_MODE1, 4376
PI_MODE2, 4377
PI_MODE3, 4378
PID_MODE1, 4373
PID_MODE2, 4374
PID_MODE3, 4375
PLANT_BUS_SECTION_CHNG, 1054
PLANT_CHNG, 839
PLANT_CHNG_3, 840
PLANT_CHNG_4, 841
PLANT_DATA, 843
PLANT_DATA_3, 844
PLANT_DATA_4, 845
PLMOD_CONSISTENCY, 1789
PLMOD_PACK, 1790
PLMOD_REMOVE, 1791
PLMOD_STATUS, 1792
PLMOD_UNCONNECTED, 1793
PLMOD_USER, 1794
PLOT_BOOK, 3503
PLOT_BOOK_MBID, 3504
PLOT_BOOK_PRINT, 3505
PLOT_BOOK_PURG, 3507
PLOT_BOOK_SET_ACTIVE, 3508
PLOT_PAGE, 3509
PLOT_PAGE_CHNG, 3510
PLOT_PAGE_PRINT, 3511
PLOT_PAGE_PURG, 3513
PLOT_PAGE_SET_ACTIVE, 3514
PLOT_PLOT, 3515
PLOT_PLOT_CHNG, 3517
PLOT_PLOT_EXPORT_IMAGE, 3519
PLOT_PLOT_PURG, 3520
PLOT_PLOT_SET_ACTIVE, 3521
PLOT_TABLE_COLUMN, 3522
PLOT_TABLE_COLUMN_RANGE, 3524
PLOT_TRACE_CHANNEL, 3526
PLOT_TRACE_PURG, 3528
PLOT_TRACE_XY, 3529
PLOTPAGEPRINTSETUP, 3502
POLY, 423
POLY_PRINT, 3205
POUT, 426
POWER_OUTPUT, 427
POWERFLOWMODE, 1694
PP_ACCC, 428
PP_ACCC_MULTI_CASE, 430
PREVDIAGFILEDATASET, 2178
PRINT_OUTAGED_BRANCHES, 433
PRINT_WINDING_BUSES, 434
PRINTACTIVEPAGE, 3531
PRINTDIAGFILE, 2179
PRINTDIAGFILE_2, 2180
PRINTRPT, 3758
PRINTSELPLOT, 3532
PRMDAT, 2513
PRMINT, 2515
PRODUCE_OPF_LOG_FILE, 1155
PROGRESS, 3470
PROGRESS_OUTPUT, 435
PROMPT, 3471
PROMPT_OUTPUT, 437
PRTI, 439
PSAS, 1695
PSCASE, 3759
PSCOPF, 440
PSCOPF_2, 446
PSEB, 452
PSILD, 4379
PSSEHALT, 453
PSSEHALT_2, 454
PSSEINIT, 3472
PSSEVERSION, 3473
PTOTAL, 4380
PTOTAR, 4381
PTOTOW, 4382
PTOTZN, 4383
PURG, 847
PURG_INTERFACE, 877
PURG_INTERFACE_ELEMENT, 878
PURG_LOADTYPE, 879
PURG_SWD_RATING, 880
PURG_VOLTAGE_DROOP, 881
PURG2DC, 849
PURG3WND, 850
PURGAREA, 851
PURGATRN, 852
PURGBRN, 853
PURGCAPCURVE, 854
PURGE_ALL_OPF_DATA, 1239
PURGE_MULTI_TERM_DC_BUS, 855
PURGE_MULTI_TERM_DC_CONVR, 856
PURGE_MULTI_TERM_DC_LINK, 858
PURGE_OPF_ADJBRX_INDV, 1240
PURGE_OPF_ADJBRX_SUBSYS, 1241
PURGE_OPF_ADJLOAD_TBL, 1242
PURGE_OPF_ADJVAR_INDV, 1243
PURGE_OPF_ADJVAR_INDV_3, 1244
PURGE_OPF_ADJVAR_SUBSYS, 1245
PURGE_OPF_APDSP_TBL, 1246
PURGE_OPF_BRFLW_3WT, 1247
PURGE_OPF_BRFLW_BRN, 1248
PURGE_OPF_BRFLW_SUBSYS, 1249
PURGE_OPF_BUS_INDV, 1250
PURGE_OPF_BUS_INDV_3, 1251
PURGE_OPF_BUS_SUBSYS, 1252
PURGE_OPF_CSTTBL_LIN, 1253
PURGE_OPF_CSTTBL_POLY, 1254
PURGE_OPF_CSTTBL_QUAD, 1255
PURGE_OPF_GEN_RCAP_INDV, 1260
PURGE_OPF_GEN_RCAP_SUBSYS, 1261
PURGE_OPF_GENDSP_INDV, 1256
PURGE_OPF_GENDSP_SUBSYS, 1257
PURGE_OPF_GENRSV_INDV, 1258
PURGE_OPF_GENRSV_SUBSYS, 1259
PURGE_OPF_INTFLW, 1262
PURGE_OPF_INTFLW_3WT, 1263
PURGE_OPF_INTFLW_BRN, 1264
PURGE_OPF_LNCEQN, 1265
PURGE_OPF_LNCEQN_ADJLOAD, 1266
PURGE_OPF_LNCEQN_ADJVAR, 1267
PURGE_OPF_LNCEQN_BRFLOW, 1268
PURGE_OPF_LNCEQN_INTFLOW, 1269
PURGE_OPF_LNCEQN_PGEN, 1270
PURGE_OPF_LNCEQN_QGEN, 1271
PURGE_OPF_LNCEQN_SWSHUNT, 1272
PURGE_OPF_LNCEQN_SWSHUNT_2, 1273
PURGE_OPF_LNCEQN_TRAN, 1274
PURGE_OPF_LNCEQN_VANG, 1275
PURGE_OPF_LNCEQN_VANG_2, 1276
PURGE_OPF_LNCEQN_VMAG, 1277

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

PURGE_OPF_LNCEQN_VMAG_2, 1278
PURGE_OPF_LOAD_INDV, 1279
PURGE_OPF_LOAD_SUBSYS, 1280
PURGE_OPF_PERRSV, 1281
PURGE_OPF_PERRSV_GEN, 1282
PURGE_TSPF_CHANNEL, 1616
PURGE_TSPF_CHANNEL_ALL, 1617
PURGE_TSPF_PROFILE, 1618
PURGE_TSPF_PROFILECURVE, 1619
PURGE_TSPF_PROFILECURVESET, 1620
PURGE_TSPF_PROFILEDATA, 1621
PURGFACTS, 859
PURGGNE, 860
PURGINDMAC, 861
PURGINDMACS, 862
PURGLOAD, 863
PURGLOADS, 864
PURGMAC, 865
PURGMDC, 866
PURGMSL, 867
PURGMUT, 868
PURGOWNER, 869
PURGPLNT, 870
PURGSHUNT, 871
PURGSHUNTS, 872
PURGSWS, 873
PURGSWSHUNT, 874
PURGVSC, 875
PURGZONE, 876
PV_ENGINE, 455
PV_ENGINE_1A, 460
PV_ENGINE_2, 465
PV_ENGINE_3, 471
PV_ENGINE_4, 477
PV_ENGINE_5, 483
PV_ENGINE_6, 489
PV_PRINT, 3206

Q

Q_LOSSES_COST_COEFF, 1157
QV_ENGINE, 495
QV_ENGINE_2, 498
QV_ENGINE_3, 501
QV_ENGINE_4, 504
QV_PRINT, 3208

R

RANK, 508
RANK_BRN_AND_MAC, 510
RATCHR, 2516
RATE, 512
RATE_2, 514
RATE_3, 516
RATE_INTERFACE, 518
RATING_SET, 519
RATINGSETTEXTDATA, 882
RAWD, 520
RAWD_2, 524
RAWDX, 522
RAWX_READ_MODE, 526
RAWX_TO_CSV, 527
RCL3WT, 4384
RDCH, 528
RDCHRAWVERSION, 529
RDEQ, 530
RDEXCHGFILE, 3760
RDMONCONFILE, 3761
RDMUSTFILES, 3762
RDSUBFILE, 3764
REACTIVE_RESV_COST_COEFF, 1158
READ, 532
READ_RAS, 3448
READACCONTFILE, 3765
READCAPCURVES, 533
READRAWDATA, 3766
READRAWVERSION, 534
READRAWX, 535
READSUB, 536
READSUBRAWVERSION, 538
READX, 540
REAREABUS, 3767
RECN, 541
REFRESHDIAGCONTOUR, 2181
REFRESHDIAGFILE, 2182
REFRESHGUI, 3474
REGB, 542
RELIND, 543
RELIND_2, 546
REMM, 549
REMOVE_UNUSED_CHAN_MODELS, 1749
REMOVEEXCLUDES, 3768
RENUMDIAGFILE, 2183
REPORT, 3475
REPORT_OUTPUT, 550
RESQ, 552
RESQVERSION, 553
RESTORESTATESFROMISOLATE, 1038
RESULT_TABLE_OUTPUT, 554
RETI, 555
RETRY_PSSUSERPF, 556
REV29_NAMES, 557
REZONEBUS, 3769
RLLIST, 1696
RLMOD_PACK, 1848
RLMOD_REMOVE, 1849
RLMOD_STATUS, 1850
RLMOD_UNCONNECTED, 1851
RLMOD_USER, 1852
RMODIND, 2517
RMODNAM, 2519
RNFI, 558
ROPF, 1159
RSOL, 560
RSTR, 1697
RUN, 1698
RUNDYNAMICEVENTSTUDY, 3308
RUNIPLANFILE, 3476
RUNLASTSOLUTION, 562
RUNPOWERFLOWEVENTSTUDY, 3309
RUNRSPNSFILE, 3477
RWCM, 563
RWDY, 1699
RWMA, 564
RWMM, 566
RWOP, 1160
RWSQ, 567
RWSQ_2, 569
RXPATH, 3340

S

SAVE, 571
SAVEACCONTFILE, 3770
SAVEDIAGFILE, 2184
SAVEEVENTSTUDYFILE, 3310
SAVEFLOWGATES, 3771
SAVESCENARIOFILE, 3218
SAVESELECTEDCONTS, 3772
SAVESUBSYSTEMS, 3773
SAVEXCHGFILE, 3774
SBSGTBKV, 2520
SC3WND, 2521
SCAL, 572
SCAL_2, 575
SCAL_3, 578
SCAL_4, 581
SCANCHANNELS, 1750
SCBRN2, 2523
SCBUS2, 2524
SCDONE, 2526

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SCEDANALYSIS, 3775
SCENARIOCLOSE, 3219
SCENARIOCOPYFILE, 3220
SCENARIOFILEADD, 3221
SCENARIOFILECOUNT, 3222
SCENARIOFILEGETATTRS, 3223
SCENARIOFILEGETCOMMENTS, 3224
SCENARIOFILELENGTHCOMMENTS, 3225
SCENARIOFILELIST, 3226
SCENARIOFILEREMOVE, 3227
SCENARIOFILERENAME, 3228
SCENARIOFILESETATTRS, 3229
SCENARIOFILESETCOMMENTS, 3230
SCENARIOGETATTRS, 3231
SCENARIOGETCOMMENTS, 3232
SCENARIOGETOPT, 3233
SCENARIOGROUPADD, 3235
SCENARIOGROUPCOUNT, 3236
SCENARIOGROUPGETCOMMENTS, 3237
SCENARIOGROUPLENGTHCOMMENTS, 3238
SCENARIOGROUPLIST, 3239
SCENARIOGROUPOPEN, 3240
SCENARIOGROUPREMOVE, 3241
SCENARIOGROUPRENAME, 3242
SCENARIOGROUPSETCOMMENTS, 3243
SCENARIOISDIRTY, 3244
SCENARIOISVALID, 3245
SCENARIOLENGTHCOMMENTS, 3246
SCENARIOMODELGROUP, 3247
SCENARIOMOVEFILE, 3248
SCENARIONEW, 3249
SCENARIOOPEN, 3250
SCENARIOSAVE, 3251
SCENARIOSETATTRS, 3252
SCENARIOSETCOMMENTS, 3253
SCENARIOSETOPT, 3254
SCENARIOTRACKFILEADD, 3256
SCENARIOTRACKFILELIST, 3257
SCENARIOTRACKFILENUM, 3258
SCENARIOUNZIP, 3259
SCENARIOUNZIPPROBLEMS, 3260
SCENARIOUNZIPSOLUTIONS, 3261
SCENARIOZIP, 3262
SCENARIOZIPACTIVE, 3263
SCENARIOZIPOPEN, 3264
SCENARIOZIPSAVE, 3265
SCEQ, 1328
SCGR, 1330
SCINIT, 2527
SCMAC2, 2528
SCMU, 1332
SCMU_2, 1339
SCOP, 1346
SELCTR, 2529
SELECTACTIVECONTS, 3777
SELECTALLCONTS, 3778
SELECTCOMPLEXCONTS, 3779
SELECTCONVERGECONTS, 3780
SELECTDISABLEDCONTS, 3781
SELECTEXCLUDEDCONTS, 3782
SELECTGENCONTS, 3783
SELECTHISEVCONTS, 3784
SELECTIMBALANCECONTS, 3785
SELECTISLANDCONTS, 3786
SELECTLOADCONTS, 3787
SELECTLOSEVCONTS, 3788
SELECTNOTCONVERGECONTS, 3789
SELECTNOVIOLCONTS, 3790
SELECTSINGLEBRANCHCONTS, 3791
SELECTSPECIFICCONTS, 3792
SELECTVIOLCONTS, 3793
SENSITIVITY_FLOW, 584
SENSITIVITY_FLOWS, 588
SENSITIVITY_INTERFACE, 591
SENSITIVITY_VOLTAGE, 594
SENSITIVITY_VOLTAGES, 597
SEQ_3_WIND_GROUNDING_DATA, 1354
SEQ_3_WIND_WINDING_DATA, 1355
SEQ_BRANCH_DATA, 1356
SEQ_BRANCH_DATA_3, 1358
SEQ_BUS_DATA, 1360
SEQ_FIXED_SHUNT_DATA, 1361
SEQ_INDUCTION_MACH_DATA, 1362
SEQ_INDUCTION_MACH_DATA_2, 1364
SEQ_LOAD_DATA, 1366
SEQ_MACHINE_DATA, 1368
SEQ_MACHINE_DATA_3, 1370
SEQ_MACHINE_DATA_4, 1372
SEQ_MACHINE_NCS_DATA, 1374
SEQ_MACHINE_NCS_DATA_PURG, 1376
SEQ_MUTUAL_DATA, 1377
SEQ_NCS_FLT_CNTRB_CHNG, 1379
SEQ_NCS_FLT_CNTRB_DATA, 1380
SEQ_NCS_FLT_CNTRB_PURG, 1381
SEQ_NCS_TABLE_DATA, 1382
SEQ_NCS_TABLE_ONEPT, 1383
SEQ_NCS_TABLE_PURG, 1384
SEQ_NCS_TABLE_PURG_ONEPT, 1385
SEQ_NCS_TABLE_TYPE, 1386
SEQ_SWITCHED_SHUNT_DATA, 1387
SEQ_SWITCHED_SHUNT_DATA_3, 1389
SEQ_THREE_WINDING_DATA, 1391
SEQ_THREE_WINDING_DATA_3, 1393
SEQ_TWO_WINDING_DATA, 1395
SEQ_TWO_WINDING_DATA_3, 1397
SEQD, 1348
SEQD_2, 1349
SEQUENCE_NETWORK_SETUP, 1350
SET_ANGLE, 4159
SET_BSWICH, 4160
SET_CDGNFR, 4161
SET_CFDLFR, 4162
SET_CHNFIL_TYPE, 1700
SET_CHRICN, 4163
SET_CLODFR, 4164
SET_CON, 4165
SET_CON_DSCRPT, 4166
SET_CURNT, 4167
SET_DC2SIG, 4168
SET_DCMSIG, 4169
SET_DISABLE_RUN, 1701
SET_DSTATE, 4170
SET_ECOMP, 4171
SET_EFD, 4172
SET_ETERM, 4173
SET_EXT_ACLINE_CHAR, 2723
SET_EXT_ACLINE_INT, 2724
SET_EXT_ACLINE_LOGICAL, 2725
SET_EXT_ACLINE_REAL, 2726
SET_EXT_AREA_CHAR, 2727
SET_EXT_AREA_INT, 2728
SET_EXT_AREA_LOGICAL, 2729
SET_EXT_AREA_REAL, 2730
SET_EXT_BUS_CHAR, 2731
SET_EXT_BUS_INT, 2732
SET_EXT_BUS_LOGICAL, 2733
SET_EXT_BUS_REAL, 2734
SET_EXT_CHAR, 2735
SET_EXT_FACTS_CHAR, 2736
SET_EXT_FACTS_INT, 2737
SET_EXT_FACTS_LOGICAL, 2738
SET_EXT_FACTS_REAL, 2739
SET_EXT_FIXSHUNT_CHAR, 2740
SET_EXT_FIXSHUNT_INT, 2741
SET_EXT_FIXSHUNT_LOGICAL, 2742
SET_EXT_FIXSHUNT_REAL, 2743
SET_EXT_GENERATOR_CHAR, 2744
SET_EXT_GENERATOR_INT, 2745
SET_EXT_GENERATOR_LOGICAL, 2746
SET_EXT_GENERATOR_REAL, 2747
SET_EXT_GNE_CHAR, 2748
SET_EXT_GNE_INT, 2749

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SET_EXT_GNE_LOGICAL, 2750
SET_EXT_GNE_REAL, 2751
SET_EXT_IATRANS_CHAR, 2752
SET_EXT_IATRANS_INT, 2753
SET_EXT_IATRANS_LOGICAL, 2754
SET_EXT_IATRANS_REAL, 2755
SET_EXT_INDMACH_CHAR, 2756
SET_EXT_INDMACH_INT, 2757
SET_EXT_INDMACH_LOGICAL, 2758
SET_EXT_INDMACH_REAL, 2759
SET_EXT_INT, 2760
SET_EXT_LOAD_CHAR, 2761
SET_EXT_LOAD_INT, 2762
SET_EXT_LOAD_LOGICAL, 2763
SET_EXT_LOAD_REAL, 2764
SET_EXT_LOGICAL, 2765
SET_EXT_MSLINE_CHAR, 2766
SET_EXT_MSLINE_INT, 2767
SET_EXT_MSLINE_LOGICAL, 2768
SET_EXT_MSLINE_REAL, 2769
SET_EXT_NTERMDC_CHAR, 2770
SET_EXT_NTERMDC_INT, 2771
SET_EXT_NTERMDC_LOGICAL, 2772
SET_EXT_NTERMDC_REAL, 2773
SET_EXT_OWNER_CHAR, 2774
SET_EXT_OWNER_INT, 2775
SET_EXT_OWNER_LOGICAL, 2776
SET_EXT_OWNER_REAL, 2777
SET_EXT_RATING_CHAR, 2778
SET_EXT_RATING_INT, 2779
SET_EXT_RATING_LOGICAL, 2780
SET_EXT_RATING_REAL, 2781
SET_EXT_REAL, 2782
SET_EXT_SUB_CHAR, 2791
SET_EXT_SUB_INT, 2792
SET_EXT_SUB_LOGICAL, 2793
SET_EXT_SUB_REAL, 2794
SET_EXT_SUBNODE_CHAR, 2783
SET_EXT_SUBNODE_INT, 2784
SET_EXT_SUBNODE_LOGICAL, 2785
SET_EXT_SUBNODE_REAL, 2786
SET_EXT_SUBSWD_CHAR, 2787
SET_EXT_SUBSWD_INT, 2788
SET_EXT_SUBSWD_LOGICAL, 2789
SET_EXT_SUBSWD_REAL, 2790
SET_EXT_SWSHUNT_CHAR, 2795
SET_EXT_SWSHUNT_INT, 2796
SET_EXT_SWSHUNT_LOGICAL, 2797
SET_EXT_SWSHUNT_REAL, 2798
SET_EXT_SYSSWD_CHAR, 2799
SET_EXT_SYSSWD_INT, 2800
SET_EXT_SYSSWD_LOGICAL, 2801
SET_EXT_SYSSWD_REAL, 2802
SET_EXT_TRANSFORMER_CHAR, 2803
SET_EXT_TRANSFORMER_INT, 2804
SET_EXT_TRANSFORMER_LOGICAL, 2805
SET_EXT_TRANSFORMER_REAL, 2806
SET_EXT_TWOTERMDC_CHAR, 2807
SET_EXT_TWOTERMDC_INT, 2808
SET_EXT_TWOTERMDC_LOGICAL, 2809
SET_EXT_TWOTERMDC_REAL, 2810
SET_EXT_VSCDC_CHAR, 2811
SET_EXT_VSCDC_INT, 2812
SET_EXT_VSCDC_LOGICAL, 2813
SET_EXT_VSCDC_REAL, 2814
SET_EXT_ZONE_CHAR, 2815
SET_EXT_ZONE_INT, 2816
SET_EXT_ZONE_LOGICAL, 2817
SET_EXT_ZONE_REAL, 2818
SET_FCTREF, 4174
SET_FCTSIG, 4175
SET_FREQ_RELAY_V_THRESH, 1702
SET_GB, 4176
SET_GBLINE, 4177
SET_GENANG, 1703
SET_GENANG_2, 1704
SET_GENANG_3, 1706
SET_GENANG_SUBSYS_FLAG, 1708
SET_GENPWR, 1709
SET_GENSPDEV, 1710
SET_GENSPDEV_SUBSYS_FLAG, 1711
SET_GREF, 4178
SET_ICON, 4179
SET_ICON_DSCRPT, 4180
SET_IMARYS, 4181
SET_IMCCNX, 4182
SET_IMELTRQ, 4183
SET_IMLDTRQ, 4184
SET_IMSPARE, 4185
SET_IMTPLOD, 4186
SET_IMTQLOD, 4187
SET_INPUT_DEV, 600
SET_INTICN, 4188
SET_ISORCE, 4189
SET_KPAUSE, 4190
SET_KTRIP, 4191
SET_LCREF, 4192
SET_LDDGNFL, 4193
SET_LDSTAT, 4194
SET_LOAD_MODEL_THRESH, 1712
SET_MACBLKD, 4195
SET_MCARYS, 4196
SET_MCSTAT, 4197
SET_MDC, 4198
SET_MODEL_DEBUG_OUTPUT_FLAG, 1713
SET_NETFRQ, 1714
SET_NEXT_CHANNEL, 1715
SET_NEXT_ICON, 1716
SET_NEXT_VAR, 1717
SET_NINTEG, 4199
SET_NLWTRB, 4200
SET_NUMLOD, 4201
SET_NUMSWS, 4202
SET_NUMTRM, 4203
SET_OEL_ACTIVE, 4204
SET_OLDERV, 4205
SET_OPF_REPORT_SUBSYSTEM, 1161
SET_OSSCAN, 1718
SET_OSSCAN_2, 1719
SET_OSSCAN_SUBSYS_FLAG, 1720
SET_PELEC, 4206
SET_PHSVLT, 4207
SET_PMAXMCD, 4208
SET_PMECH, 4209
SET_PMINMCD, 4210
SET_PROGRESS_VERBOSE, 601
SET_PSANGL, 4211
SET_QELEC, 4212
SET_RELANG, 1721
SET_RELSCN, 1723
SET_RELSCN_SUBSYS_FLAG, 1724
SET_SHNTGB, 4213
SET_SHSTAT, 4214
SET_SPEED, 4215
SET_STATE, 4216
SET_STATE_DSCRPT, 4217
SET_STORE, 4218
SET_SWSAUX, 4219
SET_SWSCNX, 4220
SET_SWSHNB, 4221
SET_SWSREF, 4222
SET_TBRATE, 4223
SET_TPLOAD, 4224
SET_TQLOAD, 4225
SET_UEL_ACTIVE, 4226
SET_VAR, 4227
SET_VAR_DSCRPT, 4228
SET_VDC, 4229
SET_VFEOUT, 4230
SET_VLTSCN, 1725
SET_VLTSCN_SUBSYS_FLAG, 1726
SET_VOEL, 4231

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SET_VOLT_VIOL_SUBSYS_FLAG, 1730
SET_VOLTAGE_DIP_CHECK, 1727
SET_VOLTAGE_REC_CHECK, 1728
SET_VOTHSG, 4232
SET_VREF, 4233
SET_VRFDBK, 4234
SET_VSCMDC, 4235
SET_VSCPAC, 4236
SET_VSCQAC, 4237
SET_VSCSIG, 4238
SET_VSCTYP, 4239
SET_VUEL, 4240
SET_WAEROT, 4241
SET_WAUXSG, 4242
SET_WEQCMD, 4243
SET_WINDRP, 4244
SET_WINDRS, 4245
SET_WIPCMD, 4246
SET_WIQCMD, 4247
SET_WMCARYS, 4248
SET_WNDSP1, 4249
SET_WNDSP10, 4250
SET_WNDSP2, 4251
SET_WNDSP3, 4252
SET_WNDSP4, 4253
SET_WNDSP5, 4254
SET_WNDSP6, 4255
SET_WNDSP7, 4256
SET_WNDSP8, 4257
SET_WNDSP9, 4258
SET_WPCMND, 4259
SET_WPITCH, 4260
SET_WPORDR, 4261
SET_WPRATE, 4262
SET_WPREFPTQ, 4263
SET_WPREFWGO, 4264
SET_WQCMND, 4265
SET_WROTID, 4266
SET_WROTIQ, 4267
SET_WROTRI, 4268
SET_WROTRV, 4269
SET_WROTVD, 4270
SET_WROTVQ, 4271
SET_WTRBSP, 4272
SET_WVLCTY, 4273
SET_XADIFD, 4274
SET_ZSORCE_RECONCILE_FLAG, 1731
SETACTIVEEVENTSTUDY, 3311
SETDIAGAUTOFILE, 2185
SETDIAGFONT, 2186
SETDIAGPREC, 2187
SETDIAGPREC_2, 2188
SETDIAGPROP, 2189
SETDIAGPROP_2, 2192
SETDIAGPROP_3, 2195
SETDIAGPROP_4, 2198
SETDIAGPROP_5, 2201
SETDIAGRESACCC, 2204
SETDIAGRESACCC_2, 2205
SETDIAGRESACCC_3, 2206
SETDIAGRESASCC, 2207
SETDIAGRESASCC_2, 2209
SETDIAGRESASCC_3, 2211
SETDIAGRESDATA, 2213
SETDIAGRESDATA_2, 2216
SETDIAGRESDATA_3, 2217
SETDIAGRESDYN, 2218
SETDIAGRESDYN_2, 2219
SETDIAGRESGDIF, 2220
SETDIAGRESIEC, 2223
SETDIAGRESIEC_2, 2225
SETDIAGRESIEC_3, 2227
SETDIAGRESMUST, 2229
SETDIAGRESMUST_2, 2230
SETDIAGRESOPF, 2231
SETDIAGRESOPF_2, 2232
SETDIAGRESOPF_3, 2233
SETDIAGRESPFLOWCHECK, 2234
SETDIAGRESPFLOWCHECK_2, 2238
SETDIAGRESPFLOWCHECK_3, 2241
SETDIAGRESPFLOWCHECK_4, 2244
SETDIAGRESPFLOWOPTIONS, 2247
SETDIAGRESPFLOWOPTIONS_2, 2249
SETDIAGRESPFLOWOPTIONS_3, 2251
SETDIAGRESREL, 2253
SETDIAGRESREL_2, 2255
SETDIAGRESREL_3, 2257
SETDIAGRESSCGR, 2259
SETDIAGRESTYPEACC, 2262
SETDIAGRESTYPEASCC, 2263
SETDIAGRESTYPEDATA, 2264
SETDIAGRESTYPEDYN, 2265
SETDIAGRESTYPEGDIF, 2266
SETDIAGRESTYPEGIC, 2267
SETDIAGRESTYPEHARM, 2268
SETDIAGRESTYPEIEC, 2269
SETDIAGRESTYPEMUST, 2270
SETDIAGRESTYPEOPF, 2271
SETDIAGRESTYPEPFLOW, 2272
SETDIAGRESTYPEREL, 2273
SETDIAGRESTYPESCGR, 2274
SETDIAGRESTYPETSPF, 2275
SETDIAGRESVRCS, 2276
SETDIAGRESVRCS_2, 2279
SETDIAGTITLE, 2282
SETEXCLUDEDBRANCHES, 3794
SETEXCLUDEDCONTINGENCIES, 3795
SETEXCLUDEDFLOWGATES, 3796
SETEXCLUDEDINTERFACES, 3797
SETEXCLUDEDVOLTAGEBUSES, 3798
SETFULLVIEWGRAPHAREA, 3209
SETFULLVIEWSCALE, 3212
SETHOMEPATH, 2530
SETSELECTEDPAGE, 3533
SETSELECTEDPLOT, 3534
SFILES, 2531
SGLACFCITC, 3799
SGLDCFCITC, 3801
SHNT, 602
SHORT_CIRCUIT_COORDINATES, 603
SHORT_CIRCUIT_MODELING, 604
SHORT_CIRCUIT_UNITS, 605
SHORT_CIRCUIT_WARNING, 606
SHORT_CIRCUIT_Z_COORDINATES, 607
SHORT_CIRCUIT_Z_UNITS, 608
SHOW, 609
SHOW_MODEL_INDICIES, 4386
SHOWLOADEDLIBRARIES, 3478
SHOWLOADEDMODULES, 3479
SHOWMODEL, 4385
SHOWPARAMS, 3803
SHOWPLOTEDITOR, 3535
SHOWTABLE, 610
SHOWTABLEBYINDEX, 611
SHUNT, 4387
SHUNT_CHNG, 883
SHUNT_CHNG_2, 884
SHUNT_DATA, 885
SHUNT_DATA_2, 886
SIZE, 612
SIZE_DS, 1732
SLMODIND, 2532
SLMODNAM, 2534
SNAP, 1733
SOLSTR, 2535
SOLUTION_PARAMETERS, 887
SOLUTION_PARAMETERS_2, 889
SOLUTION_PARAMETERS_3, 891
SOLUTION_PARAMETERS_4, 893
SOLUTION_PARAMETERS_5, 895
SOLV, 613
SOLVED, 2536

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SOLVED_CA, 2537
SPCB, 615
SPIL, 617
SPLT, 621
SQEX, 1351
SQLI, 622
SRAIND, 624
STADAT, 2538
STAINT, 2539
STANAME, 2540
STANODEINT, 2541
STANODENAME, 2542
STAREA, 2543
STARTRECORDING, 3480
STASWDEVDAT, 2544
STASWDEVINT, 2546
STASWDEVNAME, 2547
STASWDEVRSETNAME, 2548
STATE_CHANNEL, 1774
STATION_2DC_BUS_TERM_CHNG, 1055
STATION_2DC_CONV_TERM_CHNG, 1056
STATION_ADJVAR_TERM_CHNG, 1057
STATION_AMPOUT, 1039
STATION_BRANCH_TERM_CHNG, 1058
STATION_BRANCH_TERM_MOVE, 1059
STATION_BUILD_CONFIG, 1040
STATION_BUS_PURG, 1060
STATION_CHNG, 1061
STATION_DATA, 1062
STATION_FACTS_BUS_TERM_CHNG, 1063
STATION_FACTS_TERM_CHNG, 1064
STATION_INDMACHINE_TERM_CHNG, 1065
STATION_LIST, 1042
STATION_LOAD_TERM_CHNG, 1066
STATION_MACHINE_TERM_CHNG, 1067
STATION_MTDC_BUS_TERM_CHNG, 1068
STATION_MTDC_TERM_CHNG, 1069
STATION_NODE_CHNG, 1070
STATION_NODE_DATA, 1071
STATION_NODE_NUMBER, 1072
STATION_NODE_PURG, 1073
STATION_NUMBER, 1074
STATION_POUT, 1043
STATION_POUT_2, 1044
STATION_PURG, 1075
STATION_SHUNT_TERM_CHNG, 1076
STATION_SWD_CHNG, 1077
STATION_SWD_CHNG_2, 1079
STATION_SWD_DATA, 1081
STATION_SWD_DATA_2, 1083
STATION_SWD_MBID, 1085
STATION_SWD_MOVE, 1086
STATION_SWD_PURG, 1087
STATION_SWS_SHUNT_TERM_CHNG, 1088
STATION_SWS_SHUNT_TERM_CHNG_2, 1089
STATION_THREE_WND_TERM_CHNG, 1090
STATION_THREE_WND_TERM_MOVE, 1091
STATION_TREE, 1045
STATION_VSCDC_BUS_TERM_CHNG, 1092
STATION_VSCDC_CONV_TERM_CHNG, 1093
STBSKV, 2549
STDYPROJCHECKALLTASKS, 3266
STDYPROJCURRENTREPORTSETLOCKED, 3267
STDYPROJCURRENTVARIANTGETLOCKED, 3268
STDYPROJCURRENTVARIANTSETLOCKED, 3269
STDYPROJEXTRACTDATASET, 3270
STDYPROJEXTRACTSIMULATIONRESULT, 3271
STDYPROJGETCURRENTSIMULATION, 3272
STDYPROJGETCURRENTVARIANT, 3273
STDYPROJGETSETTINGS, 3274
STDYPROJINPUTDATASETADD, 3275
STDYPROJINPUTDATASETREMOVE, 3276
STDYPROJIPLANLAUNCH, 3277
STDYPROJNEW, 3278
STDYPROJOPEN, 3279
STDYPROJPYTHONLAUNCH, 3280
STDYPROJRENAME, 3281
STDYPROJREPORTADD, 3282
STDYPROJREPORTREMOVE, 3283
STDYPROJREPORTRENAME, 3284
STDYPROJREPORTSETLOCK, 3285
STDYPROJRESPONSELAUNCH, 3286
STDYPROJSETCURRENTREPORT, 3287
STDYPROJSETCURRENTSIMULATION, 3288
STDYPROJSETCURRENTVARIANT, 3289
STDYPROJSETSETTINGS, 3290
STDYPROJSIMULATIONADD, 3291
STDYPROJSIMULATIONREMOVE, 3292
STDYPROJSIMULATIONRENAME, 3293
STDYPROJSIMULATIONSETLOCK, 3294
STDYPROJTASKOPT, 3295
STDYPROJTASKSTATUS, 3296
STDYPROJVARIANTADD, 3297
STDYPROJVARIANTCLONE, 3298
STDYPROJVARIANTCOMPARE, 3299
STDYPROJVARIANTGETLOCK, 3300
STDYPROJVARIANTREMOVE, 3301
STDYPROJVARIANTRENAME, 3302
STDYPROJVARIANTSETLOCK, 3303
STDYPROJWAITALLTASKS, 3304
STOP, 626
STOP_2, 627
STOPRECORDING, 3481
STRT, 1734
STRT_2, 1735
STZONE, 2550
SUBS, 628
SUBSYSTEMS, 3805
SUMMARYCONTINGENCIES, 3806
SWCAP, 4388
SWD_RATING_CHNG, 898
SWD_RATING_DATA, 900
SWD_RATING_DEFAULT, 629
SWD_RATING_DESC, 2551
SWD_RATING_DESC_LENGTH, 2552
SWD_RATING_NAME_DATA, 2553
SWD_RATING_NAME_SET, 902
SWD_RATING_VALUE, 2554
SWD_RATINGS, 3341
SWINGBUSES, 3807
SWITCHDIAGFILEDATASET, 2283
SWITCHED_SHUNT_ADJUSTMENT, 630
SWITCHED_SHUNT_CHNG_3, 903
SWITCHED_SHUNT_CHNG_4, 906
SWITCHED_SHUNT_CHNG_5, 909
SWITCHED_SHUNT_CHNG_6, 912
SWITCHED_SHUNT_DATA, 915
SWITCHED_SHUNT_DATA_3, 918
SWITCHED_SHUNT_DATA_4, 921
SWITCHED_SHUNT_DATA_5, 924
SWITCHED_SHUNT_DATA_6, 927
SWSBLK, 2555
SWSBLK_2, 2556
SWSBLZ, 2558
SWSBLZ2, 2559
SWSBUS, 2560
SWSCHR, 2561
SWSDAT, 2562
SWSDAT_2, 2563
SWSDT1, 2564
SWSINT, 2565
SWSINT_2, 2566
SWSLIST, 1736
SWSMIND, 2567
SWSMIND_2, 2569
SWSMNAM, 2571
SWSMNAM_2, 2572
SWSMOD_PACK, 1916
SWSMOD_REMOVE, 1917
SWSMOD_REMOVE_2, 1918
SWSMOD_STATUS, 1919
SWSMOD_STATUS_2, 1920

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

SWSMOD_UNCONNECTED, 1921
SWSMOD_USER, 1922
SWSSTAT, 4389
SYSMSM, 2573
SYSMVA, 2574
SYSSWDEVRSETNAM, 2575
SYSTEM_SWD_CHNG, 1094
SYSTEM_SWD_CHNG_2, 1096
SYSTEM_SWD_DATA, 1098
SYSTEM_SWD_DATA_2, 1100
SYSTOT, 2576

T

T_ALERT_OUTPUT, 3482
T_PROGRESS_OUTPUT, 3484
T_PROMPT_OUTPUT, 3486
T_REPORT_OUTPUT, 3488
TAP_ADJUSTMENT, 631
TEXT, 632
TFLG, 633
THREE_WINDING_DATA, 930
THREE_WND_APP_R_X_CHANNEL, 1765
THREE_WND_IMPED_CHNG_3, 937
THREE_WND_IMPED_CHNG_4, 940
THREE_WND_IMPED_DATA_3, 943
THREE_WND_IMPED_DATA_4, 946
THREE_WND_IMPEDANCE_DATA, 934
THREE_WND_MVA_CHANNEL, 1767
THREE_WND_P_AND_Q_CHANNEL, 1769
THREE_WND_P_CHANNEL, 1771
THREE_WND_WINDING_DATA, 949
THREE_WND_WINDING_DATA_3, 951
THREE_WND_WINDING_DATA_4, 953
THREE_WND_WINDING_DATA_5, 956
TIES, 634
TIEZ, 635
TIME, 636
TITLDT, 2577
TLST, 637
TLTG, 638
TOTBUS, 2578
TPCH, 642
TR3DAT, 2579
TR3DT2, 2580
TR3INT, 2582
TR3NAM, 2584
TRANSACTION_ADD_MOD, 959
TRANSACTION_DELETE, 960
TRANSFER_CHNG, 961
TRANSFER_DATA, 962
TRANSFORMER_PERCENT_UNITS, 645
TRANSMISSION_LINE_UNITS, 646
TREE, 647
TREEDAT, 3342
TRIG_VOLT_VIOLATION_CHECK, 1737
TRNDAT, 2585
TRP3WT, 4390
TRXDAT, 2586
TSPF, 1610
TSPF_CHANNEL_CHNG, 1622
TSPF_CHANNEL_DATA, 1623
TSPF_OUTPUT_PMATRIX, 1613
TSPF_OUTPUT_PMATRIX_2, 1614
TSPF_PROFILE_CHNG, 1628
TSPF_PROFILE_DATA, 1630
TSPF_PROFILECURVESET_CHNG, 1624
TSPF_PROFILECURVESET_DATA, 1625
TSPF_PROFILEDATA_CHNG, 1626
TSPF_PROFILEDATA_DATA, 1627
TSPFPROFILECHR, 2587
TSPFPROFILECURVECHR, 2588
TSPFPROFILECURVEINT, 2589
TSPFPROFILECURVEREAL, 2590
TSPFPROFILECURVEVALUE, 2591
TSPFPROFILEINT, 2592
TSPFPROFILEREAL, 2593
TTDCAUXMOD_REMOVE, 2083
TTDCAUXMOD_STATUS, 2084
TTDCLMOD_REMOVE, 1881
TTDCLMOD_STATUS, 1882
TWO_TERM_DC_CONVERTER_DATA_3, 968
TWO_TERM_DC_CONVR_DATA, 971
TWO_TERMINAL_DC_LINE_CHNG, 964
TWO_TERMINAL_DC_LINE_DATA, 966
TWO_WINDING_CHNG_4, 974
TWO_WINDING_CHNG_5, 978
TWO_WINDING_CHNG_6, 982
TWO_WINDING_DATA, 987
TWO_WINDING_DATA_3, 991
TWO_WINDING_DATA_4, 995
TWO_WINDING_DATA_5, 999
TWO_WINDING_DATA_6, 1003
TYSL, 648

U

UEL2CUDM, 4391
UPDATEBUSLOCDIAGFILE, 2284
USE_DUAL_CRITERIA, 1162
USE_EMERGENCY_FLOW_LIMITS, 1163
USE_EMERGENCY_VOLT_LIMITS, 1164
USER, 649
USERIN, 3490

V

VALIDATE_RAS, 3449
VAMM, 650
VAMO, 651
VAR_CHANNEL, 1775
VCHK, 652
VIOLSELCONTINGENCIES, 3808
VOLTAGE_AND_ANGLE_CHANNEL, 1776
VOLTAGE_CHANNEL, 1778
VOLTAGE_DROOP_CHNG, 1008
VOLTAGE_DROOP_DATA, 1010
VOLTAGE_INPUT, 653
VOLTAGE_OUTPUT, 654
VOLTAGEDROOPDATA, 2594
VOLTAGEDROOPINT, 2595
VSC_DC_CONVERTER_DATA, 1012
VSC_DC_CONVERTER_DATA_3, 1014
VSC_DC_LINE_CHNG, 1016
VSC_DC_LINE_DATA, 1017
VSCAUXMIND, 2596
VSCAUXMNAM, 2598
VSCCDT, 2599
VSCCIN, 2601
VSCINT, 2602
VSCMIND, 2603
VSCMNAM, 2604
VSDCAUXMOD_REMOVE, 2085
VSDCAUXMOD_STATUS, 2086
VSDCLMOD_REMOVE, 1883
VSDCLMOD_STATUS, 1884
VSMO, 655

W

WINDMIND, 2605
WINDMNAM, 2607
WNDDAT, 2608
WNDDT2, 2610
WNDINT, 2612
WNLIST, 1738
WNMOD_CONSISTENCY, 1801
WNMOD_PACK, 1802
WNMOD_REMOVE, 1803
WNMOD_STATUS, 1804
WNMOD_UNCONNECTED, 1805

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International

WNMOD_USER, 1806
WRITE_OPF_OPTIONS_FILE, 1165
WRITE_OPTIONS_FILE, 664
WRITE_RAS, 3450
WRITECAPCURVES, 656
WRITEEXCLUDES, 3809
WRITERAWDATA, 3810
WRITERAWVERSION, 658
WRITERAWX, 659
WRITERAWXSUBSYS, 660
WRITESEQVERSION, 662
WRITESEQVERSION_2, 663
WSHOUT_MODE1, 4392
WSHOUT_MODE2, 4393
WSHOUT_MODE3, 4394

X

XEQV, 665
XFRDAT, 2614
XFRINT, 2616
XFRNAM, 2618
XYPLOT, 3536

Z

ZNDAT, 2619
ZNITOJ, 2621
ZONE, 667
ZONE_2, 668
ZONE_AREA, 669
ZONE_DATA, 1018
ZONEBUBBLE, 3811
ZONESINAREAS, 3812
ZONESUMMARY, 3813
ZONM, 670
ZONM_2, 672
ZONNAM, 2622
ZONNUM, 2623
ZONUSE, 2624
ZSYS, 2144
ZSYSDEF, 2145
ZSYSINIT, 2146

All material contained in this documentation is proprietary to Siemens Industry Inc., Siemens Power Technologies International